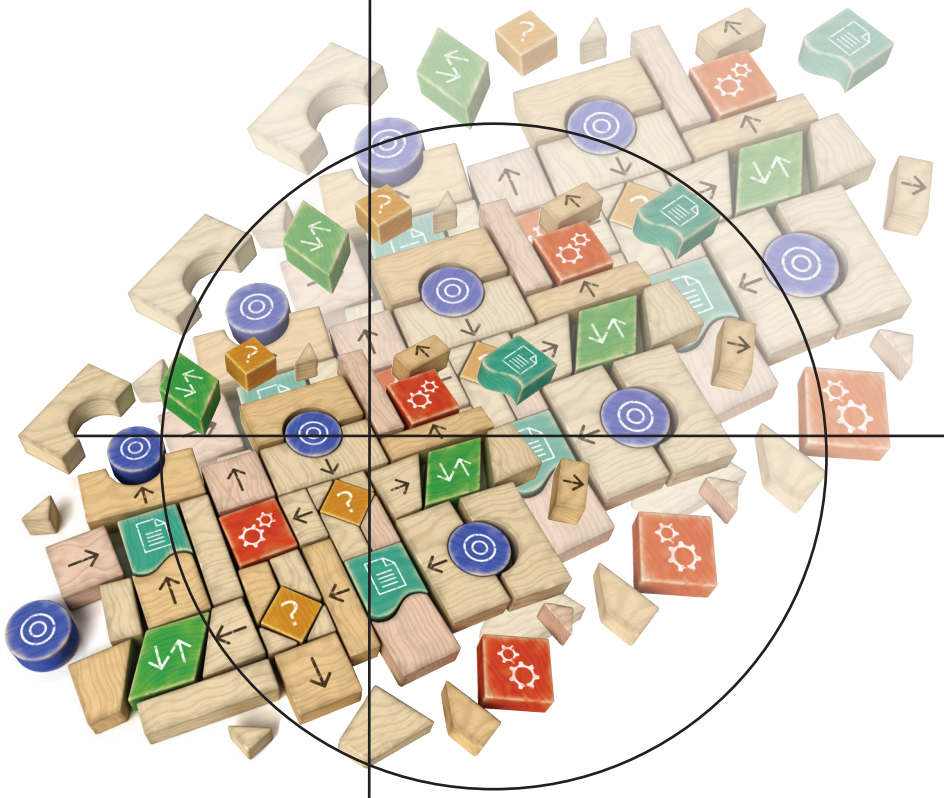
Thomas H. Cormen
Charles E. Leiserson
Ronald L. Rivest
Clifford Stein



Introduzione agli algoritmi e strutture dati

Terza edizione

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Mc Graw Hill

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

collana di istruzione scientifica
serie di informatica

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Al lettore

La realizzazione di un libro comporta costi variabili (carta, stampa e legatura) e costi fissi, cioè indipendenti dal numero di copie stampate (traduzione, preparazione degli originali, redazione, composizione, impaginazione). I fotocopiatori possono contenere il prezzo perché, oltre a non pagare i diritti d'autore, non hanno costi fissi. Ogni fotocopia, d'altra parte, riducendo il numero di copie vendute dall'editore, aumenta l'incidenza dei costi fissi a copia e costringe l'editore ad aumentare il prezzo; questo, naturalmente, fornisce un ulteriore incentivo a fotocopiare. Se questo circolo vizioso non verrà spezzato, arriveremo al punto in cui gli editori non avranno più convenienza economica a realizzare libri di testo per l'università.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

In quel momento non ci saranno più neppure fotocopie.

L'Editore

Thomas H. Cormen
Charles E. Leiserson
Ronald L. Rivest
Clifford Stein

Introduzione agli algoritmi e strutture dati

terza edizione

Edizione italiana a cura di
Livio Colussi

Acquistato da Michele Michele su Web 12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Titolo originale: *Introduction to Algorithms, 3rd edition*
Copyright © 2009 by The Massachusetts Institute of Technology

Copyright © 2010 McGraw-Hill Education (Italy) S.r.l.
Via Ripamonti, 89 - 20141 Milano
Tel. 02.535718.1

I diritti di traduzione, di riproduzione, di memorizzazione elettronica e di adattamento totale e parziale con qualsiasi mezzo (compresi i microfilm e le copie fotostatiche) sono riservati per tutti i Paesi.

Nomi e marchi citati nel testo sono generalmente depositati dalle rispettive case produttrici.

Le fotocopie per *uso personale* del lettore possono essere effettuate nei limiti del 15% di ciascun volume/fa-scicolo di periodico dietro pagamento alla SIAE del compenso previsto dall'art. 68, commi 4 e 5, della legge 22 aprile 1941 n. 633.

Le riproduzioni effettuate per finalità di carattere professionale, economico o commerciale o comunque per *uso diverso da quello personale* possono essere effettuate a seguito di specifica autorizzazione rilasciata da CLEARedi, Corso di Porta Romana n. 108, 20122 Milano, e-mail info@clearedi.org e sito web www.clearedi.org.

Traduzione e realizzazione editoriale: Carmelo Giarratana

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

ISBN 13: 978-88-386-9771-5

Indice

Prefazione *xi*

Presentazione dell'edizione italiana *xvii*

I Fondamenti

Introduzione *3*

1 Ruolo degli algoritmi nell'elaborazione dei dati *5*
1.1 Algoritmi *5*
1.2 Algoritmi come tecnologia *10*

2 Per incominciare *14*
2.1 Insertion sort *14*
2.2 Analisi degli algoritmi *20*
2.3 Progettare gli algoritmi *25*

3 Crescita delle funzioni *37*
3.1 Notazione asintotica *37*
3.2 Notazioni standard e funzioni comuni *46*

4 Divide et impera *55*
4.1 Il problema del massimo sottoarray *57*
4.2 Algoritmo di Strassen per il prodotto di matrici *63*
4.3 Il metodo di sostituzione per risolvere le ricorrenze *70*
4.4 Il metodo dell'albero di ricorsione per risolvere le ricorrenze *74*
4.5 Il metodo dell'esperto per risolvere le ricorrenze *79*
★ 4.6 Dimostrazione del teorema dell'esperto *82*

5 Analisi probabilistica e algoritmi randomizzati *95*
5.1 Il problema delle assunzioni *95*
5.2 Variabili casuali indicatrici *98*
5.3 Algoritmi randomizzati *101*
★ 5.4 Approfondimento dell'analisi probabilistica *108*

II Ordinamento e statistiche d'ordine

Introduzione *123*

6 Heapsort *127*
6.1 Heap *127*
6.2 Conservare la proprietà dell'heap *129*
6.3 Costruire un heap *131*
6.4 L'algoritmo heapsort *134*
6.5 Code di priorità *135*

Acquistato da Michele Michele srl Webster il 2022-07-07 23:13 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

7 Quicksort ***141***
7.1 Descrizione di quicksort *141*
7.2 Prestazioni di quicksort *144*
7.3 Una versione randomizzata di quicksort *148*
7.4 Analisi di quicksort *148*

8 Ordinamento in tempo lineare ***157***
8.1 Limiti inferiori per l'ordinamento *157*
8.2 Counting sort *159*
8.3 Radix sort *162*
8.4 Bucket sort *164*

9 Mediane e statistiche d'ordine ***175***
9.1 Minimo e massimo *175*
9.2 Selezione in tempo atteso lineare *177*
9.3 Selezione in tempo lineare nel caso peggiore *180*

III Strutture dati

Introduzione ***189***

10 Strutture dati elementari ***192***
10.1 Stack e code *192*
10.2 Liste concatenate *195*
10.3 Implementare puntatori e oggetti *199*
10.4 Rappresentazione di alberi radicati *203*

11 Hashing ***209***
11.1 Tavole a indirizzamento diretto *209*
11.2 Tavole hash *211*
11.3 Funzioni hash *216*
11.4 Indirizzamento aperto *223*
★ 11.5 Hashing perfetto *230*

12 Alberi binari di ricerca ***237***
12.1 Che cos'è un albero binario di ricerca? *237*
12.2 Interrogazione di un albero binario di ricerca *239*
12.3 Inserimento e cancellazione *243*
★ 12.4 Alberi binari di ricerca costruiti in modo casuale *248*

13 Alberi rosso-neri ***255***
13.1 Proprietà degli alberi rosso-neri *255*
13.2 Rotazioni *258*
13.3 Inserimento *260*
13.4 Cancellazione *267*

14 Aumentare le strutture dati ***280***
14.1 Statistiche d'ordine dinamiche *280*
14.2 Come aumentare una struttura dati *285*
14.3 Alberi di intervalli *288*

Acquistato da Michele Michele su Webster il 2022-07-07 23:14. Numero licenza: 10000000000000 Copyright © 2022, McGraw-Hill Education (Italy)

IV Tecniche avanzate di progettazione e di analisi

Introduzione ***295***

15 Programmazione dinamica ***297***
15.1 Taglio delle aste *298*
15.2 Moltiplicare una sequenza di matrici *306*
15.3 Elementi della programmazione dinamica *313*
15.4 La più lunga sottosequenza comune (LCS) *324*
15.5 Alberi binari di ricerca ottimi *329*

16 Algoritmi golosi ***344***
16.1 Problema della selezione di attività *345*
16.2 Elementi della strategia golosa *351*
16.3 I codici di Huffman *356*
★ 16.4 Matroidi e metodi golosi *364*
★ 16.5 Un problema di programmazione dei lavori *369*

17 Analisi ammortizzata ***376***
17.1 Il metodo dell'aggregazione *377*
17.2 Il metodo degli accantonamenti *380*
17.3 Il metodo del potenziale *382*
17.4 Tavole dinamiche *386*

V Strutture dati avanzate

Introduzione ***401***

18 B-alberi ***404***
18.1 Introduzione *404*
18.2 Definizione dei B-alberi *407*
18.3 Operazioni fondamentali con i B-alberi *410*
18.4 Cancellare una chiave da un B-albero *416*

19 Haep di Fibonacci ***422***
19.1 Struttura degli heap di Fibonacci *424*
19.2 Operazioni con heap riunibili *426*
19.3 Diminuire il valore di una chiave e cancellare un nodo *433*
19.4 Limitare il grado massimo *436*

20 Alberi di van Emde Boas ***443***
20.1 Metodi preliminari *444*
20.2 Una struttura ricorsiva *447*
20.3 L'albero di van Emde Boas *455*

21 Strutture dati per insiemi disgiunti ***468***
21.1 Operazioni con gli insiemi disgiunti *468*
21.2 Rappresentazione di insiemi disgiunti tramite liste concatenate *471*
21.3 Foreste di insiemi disgiunti *474*
★ 21.4 Unione per rango con compressione del cammino *477*

Acquistato da Michele Michele Sivo (obsidian) 2022-07-07 21:12:4 Università degli Studi di Udine (id 1855899) 9788838693570 © Copyright 2022, McGraw-Hill Education (Italy)

VI Algoritmi per grafi

Introduzione *491*

22 Algoritmi elementari per grafi *493*
22.1 Rappresentazione dei grafi *493*
22.2 Visita in ampiezza *497*
22.3 Visita in profondità *504*
22.4 Ordinamento topologico *512*
22.5 Componenti fortemente connesse *515*

23 Alberi di connessione minimi *523*
23.1 Creare un albero di connessione minimo *524*
23.2 Gli algoritmi di Kruskal e Prim *528*

24 Cammini minimi da sorgente unica *539*
24.1 Definizioni e proprietà dei cammini mimini *539*
24.2 L'algoritmo di Bellman-Ford *546*
24.3 Cammini minimi da sorgente unica nei grafi orientati aciclici *549*
24.4 Algoritmo di Dijkstra *552*
24.5 Vincoli sulle differenze e cammini minimi *557*
24.6 Dimostrazione delle proprietà dei cammini minimi *562*

25 Cammini minimi fra tutte le coppie *573*
25.1 Introduzione *573*
25.2 Cammini minimi e moltiplicazione di matrici *575*
25.3 Algoritmo di Floyd-Warshall *580*
25.4 Algoritmo di Johnson per i grafi sparsi *586*

26 Flusso massimo *593*
26.1 Reti di flusso *594*
26.2 Il metodo di Ford-Fulkerson *598*
26.3 Abbinamento massimo nei grafi bipartiti *612*
★ 26.4 Algoritmi push-relabel *616*
★ 26.5 Algoritmo relabel-to-front *626*

VII Argomenti scelti

Introduzione *643*

27 Algoritmi multithread *645*
27.1 Fondamenti del multithreading dinamico *647*
27.2 Moltiplicazione multithread delle matrici *661*
27.3 Algoritmi multithread per merge sort *665*

28 Operazioni con le matrici *679*
28.1 Risoluzione dei sistemi di equazioni lineari *679*
28.2 Inversione di matrici *691*
28.3 Matrici simmetriche e definite positive e minimi quadrati *695*

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995030016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

29 Programmazione lineare *704*
29.1 Introduzione *704*
29.2 Le forme canoniche e standard *710*
29.3 Formulare i problemi come programmi lineari *717*
29.4 L'algoritmo del simplesso *722*
29.5 Dualità *735*
29.6 La soluzione di base iniziale ammissibile *740*

30 Polinomi e FFT *750*
30.1 Rappresentazione dei polinomi *752*
30.2 DFT e FFT *757*
30.3 Implementazioni efficienti della FFT *764*

31 Algoritmi di teoria dei numeri *773*
31.1 Concetti elementari di teoria dei numeri *774*
31.2 Massimo comun divisore *779*
31.3 Aritmetica modulare *784*
31.4 Risolvere le equazioni lineari modulari *790*
31.5 Il teorema cinese del resto *793*
31.6 Potenze di un elemento *796*
31.7 Crittografia a chiave pubblica RSA *800*
★ 31.8 Test di primalità *805*
★ 31.9 Scomposizione di un numero intero in fattori primi *814*

32 String matching *822*
32.1 Introduzione *822*
32.2 L'algoritmo ingenuo di string matching *824*
32.3 Algoritmo di Rabin-Karp *826*
32.4 String matching con automi a stati finiti *830*
★ 32.5 Algoritmo di Knuth-Morris-Pratt *836*

33 Geometria computazionale *845*
33.1 Proprietà dei segmenti di retta *845*
33.2 Verificare se qualche coppia di segmenti si interseca *851*
33.3 Trovare l'involucro convesso *857*
33.4 Trovare la coppia di punti più vicini *865*

34 NP-Completezza *873*
34.1 Introduzione *873*
34.2 Tempo polinomiale *878*
34.3 Verifica in tempo polinomiale *884*
34.4 NP-completezza e riducibilità *888*
34.5 Dimostrazioni della NP-completezza *897*
34.6 Problemi NP-completi *904*

35 Algoritmi di approssimazione *921*
35.1 Il problema della copertura di vertici *923*
35.2 Il problema del commesso viaggiatore *925*
35.3 Il problema della copertura di insiemi *931*
35.4 Randomizzazione e programmazione lineare *935*
35.5 Il problema della somma di sottoinsieme *939*

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

VIII Appendici: prerequisiti matematici

Introduzione *951*

A Sommatorie *952*
A.1 Formule di sommatoria e loro proprietà *952*
A.2 Limitare le sommatorie *955*

B Insiemi e altro *962*
B.1 Insiemi *962*
B.2 Relazioni *966*
B.3 Funzioni *968*
B.4 Grafi *970*
B.5 Alberi *974*

C Calcolo combinatorio e delle probabilità *982*
C.1 Calcolo combinatorio *982*
C.2 Probabilità *987*
C.3 Variabili casuali discrete *992*
C.4 Distribuzioni geometriche e binomiali *997*
★ C.5 Le code della distribuzione binomiale *1002*

D Matrici *1009*
D.1 Matrici e operazioni sulle matrici *1009*
D.2 Proprietà fondamentali delle matrici *1013*

Bibliografia

Indice analitico

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Prefazione

Prima dei calcolatori c'erano già gli algoritmi. Ma adesso che ci sono i calcolatori, ci sono più algoritmi e gli algoritmi sono il cuore del calcolo.

Questo libro è un'introduzione completa allo studio moderno degli algoritmi per calcolatori. Presenta numerosi algoritmi e li tratta molto approfonditamente, pur rendendone accessibili la progettazione e l'analisi ai lettori di qualsiasi livello. Abbiamo cercato di mantenere una semplicità espositiva, senza sacrificare l'approfondimento delle tematiche e il rigore matematico.

Ogni capitolo presenta una classe di algoritmi, le tecniche di progettazione, un'area di applicazioni e gli argomenti correlati. La descrizione degli algoritmi, sempre molto particolareggiata, si avvale di speciali "pseudocodici" appositamente progettati per essere leggibili da chiunque abbia un minimo di esperienza di programmazione. Il libro contiene 236 figure – molte composte da più parti – che illustrano il funzionamento degli algoritmi. Poiché l'*efficienza* è un criterio fondamentale della progettazione, abbiamo incluso l'analisi dei tempi di esecuzione di tutti i nostri algoritmi.

Il testo è stato ideato principalmente per essere utilizzato in corsi universitari o di specializzazione in algoritmi e strutture dati. Poiché si occupa di tecniche ingegneristiche di progettazione degli algoritmi, come pure di aspetti matematici, è adatto anche a professionisti ed esperti informatici.

In questa terza edizione abbiamo aggiornato di nuovo l'intero libro. Le modifiche riguardano vari aspetti, quali la revisione dello pseudocodice, l'inserimento di nuovi capitoli e uno stile di scrittura più attivo.

Ai docenti

Questo libro è stato progettato per essere uno strumento versatile e completo. Potrà essere utilizzato in vari corsi, da quelli sulle strutture dati a quelli più avanzati sugli algoritmi. Poiché abbiamo incluso molto più materiale di quello che si trova in un tipico corso semestrale, non dovreste incontrare difficoltà a organizzare i vostri corsi utilizzando soltanto i capitoli che vi servono. Abbiamo cercato di rendere i capitoli relativamente autonomi, in modo che non dobbiate preoccuparvi per una imprevista e inutile dipendenza di un capitolo dall'altro. Ogni capitolo tratta prima gli argomenti più semplici e poi quelli più difficili, con la struttura dei paragrafi che marca i punti naturali di separazione. In un corso per studenti universitari, potreste utilizzare soltanto i primi paragrafi di un capitolo; in un corso di specializzazione, potreste trattare l'intero capitolo.

Abbiamo incluso 957 esercizi e 158 problemi. Ogni paragrafo termina con gli esercizi e ogni capitolo termina con i problemi. Generalmente, gli esercizi sono domande concise che verificano la padronanza degli argomenti trattati. Alcuni sono semplici esercizi di autoverifica, mentre altri sono più complessi e sono adatti per essere assegnati come compiti per casa. I problemi sono casi di studio più elaborati che spesso introducono nuovi argomenti; tipicamente, sono formati da più domande che guidano lo studente fino alla soluzione finale.

Sulla base della nostra esperienza con le precedenti edizioni di questo libro, abbiamo reso disponibili le soluzioni di alcuni problemi ed esercizi. Il nostro sito web http://mitpress.mit.edu/algorithms/ contiene i link per queste soluzioni. Visi-

Acquistato da Michele Mabelo su Webster il 2022-07-07 22:15 Numero Ordine Libreria 1095736015 220707 Copyright © 2022 The McGraw-Hill Education (Italy)

tate questo sito per verificare che esso non contenga la soluzione di un problema o esercizio che volete assegnare. Noi prevediamo di aumentare gradualmente nel tempo il numero delle soluzioni, quindi vi consigliamo di visitare il nostro sito prima di iniziare un corso.

I paragrafi e gli esercizi contraddistinti con una stella (⋆) sono più adatti agli studenti che frequentano corsi di specializzazione postlaurea. Un paragrafo con una stella non è necessariamente più difficile di uno senza stella, ma potrebbe richiedere conoscenze matematiche di livello elevato. Analogamente, gli esercizi con le stelle potrebbero richiedere un bagaglio di conoscenze più avanzate o una creatività superiore alla media.

Agli studenti

Ci auguriamo che questo libro possa essere una piacevole introduzione allo studio degli algoritmi. Abbiamo cercato di rendere chiare e interessanti le descrizioni degli algoritmi. Per aiutarvi a capire gli algoritmi difficili o nuovi, abbiamo descritto i singoli passi di ogni algoritmo; abbiamo anche fornito spiegazioni accurate dei concetti matematici che sono richiesti per capire l'analisi degli algoritmi. Abbiamo organizzato i capitoli in modo che, se conoscete già qualche argomento, possiate sorvolare sui paragrafi introduttivi e passare rapidamente agli argomenti più avanzati. Il libro tratta numerosi argomenti. Abbiamo cercato di creare un libro che possa servirvi adesso come testo di riferimento di un corso e in futuro come manuale di consultazione nella vostra carriera professionale.

Quali sono i prerequisiti per leggere questo libro?

- Dovreste avere qualche esperienza di programmazione. In particolare, dovreste conoscere le procedure ricorsive e le strutture dati fondamentali, come gli array e le liste concatenate.
- Dovreste avere una certa dimestichezza con le dimostrazioni per induzione matematica. Poche porzioni del libro richiedono la conoscenza del calcolo infinitesimale. Al di là di questo, le Parti I e VIII del libro descrivono tutte le tecniche matematiche di cui avrete bisogno.

Abbiamo sentito, forte e chiara, la richiesta di fornire le soluzioni degli esercizi e dei problemi. Il nostro sito web http://mitpress.mit.edu/algorithms/ contiene i link per le soluzioni di alcuni esercizi e problemi. Visitate il sito per confrontare le vostre soluzioni con le nostre. Vi chiediamo, però, di non inviarci le vostre soluzioni.

Ai professionisti

La vasta gamma di argomenti trattati fa di questo libro un'eccellente guida agli algoritmi. Poiché ogni capitolo è relativamente autonomo, potrete focalizzare l'attenzione sugli argomenti che maggiormente vi interessano.

Acquistato da Michele Michele su Webster il 2022-01-17 23:13. Numero Ordine Libreria: 1995016-210767. Commerciale 2022 McGraw-Hill Education (Italy)

Quasi tutti gli algoritmi trattati sono di grande utilità pratica; di conseguenza abbiamo analizzato anche i problemi implementativi e altri aspetti della progettazione. Per quei pochi algoritmi che hanno soltanto un interesse squisitamente teorico abbiamo indicato le alternative pratiche.

Se vorrete implementare qualche algoritmo, sarà semplice tradurre il nostro pseudocodice nel vostro linguaggio di programmazione preferito. La struttura della pseudocodifica è stata studiata per presentare gli algoritmi in modo chiaro e conciso; di conseguenza, non considereremo la gestione degli errori e altri argomenti di ingegneria del software che richiedono ipotesi specifiche sul vostro particola-

re ambiente di programmazione. Abbiamo cercato di presentare ogni algoritmo in maniera semplice e diretta, evitando che le idiosincrasie di qualche particolare linguaggio di programmazione ne offuschino l'essenza.

Dal momento che voi utilizzate questo libro al di fuori di un corso, non avete la possibilità di confrontare le vostre soluzioni degli esercizi e dei problemi con quelle di un istruttore. Il nostro sito web http://mitpress.mit.edu/algorithms/ contiene i link per le soluzioni di alcuni esercizi e problemi. Visitate il sito per confrontare le vostre soluzioni con le nostre. Vi chiediamo di non inviarci le vostre soluzioni.

Ai nostri colleghi

Il libro include una ricca bibliografia e numerosi riferimenti alla letteratura corrente. Ogni capitolo termina con uno speciale paragrafo – Note – che fornisce alcuni dettagli storici e riferimenti bibliografici. Questo paragrafo non è una guida bibliografica completa per tutta la materia degli algoritmi. Sebbene possa sembrare difficile da credere per un libro di questa mole, non abbiamo potuto includere molti algoritmi interessanti per mancanza di spazio.

Novità della terza edizione

Che cosa è cambiato tra la seconda e la terza edizione di questo libro? Il numero di modifiche pareggia quello delle modifiche tra la prima e la seconda edizione. Come abbiamo detto a proposito delle modifiche della seconda edizione, a seconda di come si guardano, si può dire che il libro non è cambiato molto o è cambiato un bel po'.

Da una rapido esame dell'indice generale si vede che la maggior parte dei capitoli e dei paragrafi della seconda edizione sono rimasti nella terza edizione. Abbiamo tolto due capitoli e un paragrafo, ma abbiamo aggiunto due nuovi capitoli, una nuova appendice e due nuovi paragrafi.

Abbiamo mantenuto l'organizzazione ibrida delle prime due edizioni. Anziché organizzare i capitoli soltanto in funzione dei problemi o soltanto in funzione delle tecniche, questo libro applica entrambi i criteri di organizzazione. Contiene capitoli basati sulle tecniche che trattano il metodo divide et impera, la programmazione dinamica, gli algoritmi golosi, l'analisi ammortizzata, la NP-completezza e gli algoritmi di approssimazione. Ma il libro include anche intere parti sull'ordinamento, sulle strutture dati per gli insiemi dinamici e sugli algoritmi per i problemi relativi ai grafi. Sappiamo che, sebbene voi abbiate bisogno di sapere come applicare le tecniche per progettare e analizzare gli algoritmi, i problemi raramente indicano quali siano le tecniche più appropriate per essere risolti.

Ecco una sintesi delle modifiche più significative presenti nella terza edizione:

- Abbiamo aggiunto i nuovi capitoli sugli alberi di van Emde Boas e sugli algoritmi multithread; abbiamo dedicato una nuova appendice ai concetti di base delle matrici.
- Abbiamo rivisto il capitolo sulle ricorrenze per trattare in modo più generale la tecnica divide et impera; i primi paragrafi spiegano come applicare questa tecnica per risolvere due problemi. Un altro paragrafo di questo capitolo presenta l'algoritmo di Strassen per il prodotto di matrici, che prima era trattato nel capitolo dedicato alle operazioni con le matrici.

Acquistato da Michele Michela su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950316.220707.0 Copyright © 2022 McGraw-Hill Education (Italy)

- Abbiamo tolto due capitoli che erano raramente trattati nei corsi: gli heap binomiali e le reti di ordinamento. Un concetto chiave del capitolo sulle reti di ordinamento, il principio 0-1, appare nel Problema 8-7 della terza edizione come lemma dell'ordinamento 0-1 per gli algoritmi confronta-scambia. L'analisi degli heap di Fibonacci non si basa più sugli heap binomiali.
- Abbiamo rivisto la nostra trattazione della programmazione dinamica e gli algoritmi golosi. Adesso la programmazione dinamica inizia con un problema più interessante, il taglio delle aste, anziché con la programmazione delle catene di montaggio della seconda edizione. Inoltre, abbiamo trattato un po' più approfonditamente la tecnica dell'annotazione rispetto alla seconda edizione e abbiamo presentato il concetto di grafo dei sottoproblemi come un modo per capire il tempo di esecuzione di un algoritmo di programmazione dinamica. Nel primo esempio di algoritmi golosi, il problema della programmazione delle attività, abbiamo affrontato in modo più diretto la strategia golosa rispetto alla seconda edizione.
- Il modo in cui viene cancellato un nodo da un albero binario di ricerca (che include gli alberi rosso-neri) adesso garantisce che il nodo sia effettivamente cancellato. Nelle prime due edizioni, in alcuni casi, qualche altro nodo veniva cancellato e i suoi contenuti venivano trasferiti al nodo passato alla procedura di cancellazione. Con il nuovo modo di cancellare i nodi nell'albero, se altre componenti di un programma gestiscono i puntatori ai nodi dell'albero, non potrà accadere che esse si trovino erroneamente ad avere puntatori "obsoleti" a nodi che sono stati cancellati.
- La trattazione delle reti di flusso adesso si basa interamente sugli archi. Questo approccio è più intuitivo di quello utilizzato nelle precedenti edizioni del libro.
- Avendo dedicato una nuova appendice alle matrici e avendo trattato l'algoritmo di Strassen in un altro capitolo, il capitolo sulle operazioni con le matrici adesso è più piccolo rispetto alla seconda edizione.
- Abbiamo modificato la trattazione dell'algoritmo di Knuth-Morris-Pratt per le operazioni di string-matching.
- Abbiamo corretto vari errori, molti dei quali ci sono stati segnalati nel nostro sito web della seconda edizione.
- Per soddisfare numerose richieste, abbiamo cambiato la sintassi del nostro pseudocodice. Adesso utilizziamo i segni "=" per indicare l'assegnazione e "==" per il test di uguaglianza, come nei linguaggi C, C++, Java e Python. Abbiamo anche eliminato le parole chiave **do** e **then** e abbiamo utilizzato "**//**" come simbolo di commento delle righe dello pseudocodice. Abbiamo adottato il punto (.) per indicare gli attributi degli oggetti. Il nostro pseudocodice resta procedurale, anziché orientato agli oggetti. In altre parole, anziché eseguire i metodi sugli oggetti, chiamiamo semplicemente le procedure passando gli oggetti come parametri.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

- Abbiamo aggiunto 100 esercizi e 28 problemi nuovi. Abbiamo aggiornato molti riferimenti bibliografici e ne abbiamo aggiunti di nuovi.
- Infine, abbiamo rivisto l'intero libro, modificando frasi e interi paragrafi per rendere più chiaro e attivo lo stile di scrittura.

Sito web

Potete utilizzare il nostro sito web, http://mitpress.mit.edu/algorithms/, per ottenere informazioni supplementari e per comunicare con noi. Il sito web contiene dei link per una lista di errori, per le soluzioni di esercizi e problemi e per altri argomenti. Il sito spiega anche come segnalare errori o suggerimenti.

Ringraziamenti

Sono più di vent'anni ormai che lavoriamo per MIT Press, ed è stata davvero una fantastica esperienza! Ringraziamo Ellen Faran, Bob Prior, Ada Brunstein e Mary Reilly per il loro aiuto e supporto.

Eravamo geograficamente lontani mentre lavoravamo alla terza edizione, trovandoci nel Dartmouth College Department of Computer Science, nel MIT Computer Science and Artificial Intelligence Laboratory e nel Columbia University Department of Industrial Engineering and Operations Research. Ringraziamo le nostre rispettive università e i nostri colleghi per il supporto e gli stimoli che ci hanno fornito.

Julie Sussman, P.P.A., ancora una volta ci ha tolto l'impaccio della redazione tecnica. Siamo rimasti sorpresi dal numero di errori che ci sono sfuggiti, ma che sono stati catturati da Julie. Il suo lavoro ci è servito per migliorare anche la nostra esposizione. Se ci fosse una Hall of Fame per i redattori tecnici, Julie sarebbe certamente eletta alla prima votazione. È stata davvero fenomenale. Grazie, grazie, grazie Julie! Anche Priya Natarajan ha trovato alcuni errori che abbiamo potuto eliminare prima che il libro andasse in stampa. Gli eventuali errori che sono rimasti sono da attribuire agli autori (e probabilmente sono stati commessi dopo che Julie ha letto le bozze).

La trattazione degli alberi di van Emde Boas è tratta dalle note di Erik Demaine che, a sua volta, si è ispirato a Michael Bender. In questa edizione abbiamo utilizzato anche alcuni concetti tratti dalle opere di Javed Aslam, Bradley Kuszmaul e Hui Zha. Il capitolo sul multithreading si basa sulle note originariamente scritte insieme con Harald Prokop. I testi si rifanno al lavoro di molti altri autori che partecipano al progetto Cilk del MIT, tra i quali Bradley Kuszmaul e Matteo Frigo. La realizzazione dello pseudocodice multithread ha subito l'influenza del progetto Cilk del MIT e delle estensioni Cilk++ al C++ distribuite da Cilk Arts.

Ringraziamo anche i molti lettori della prima e della seconda edizione che hanno segnalato gli errori o dato suggerimenti per migliorare questo libro. Abbiamo corretto tutti gli errori che ci sono stati segnalati e abbiamo accolto molti suggerimenti. Siamo contenti perché il numero di questi lettori è aumentato notevolmente, al punto però che è impossibile elencarli tutti.

Infine, ringraziamo le nostre mogli – Nicole Cormen, Wendy Leiserson, Gail Rivest e Rebecca Ivry – e i nostri figli – Ricky, Will, Debby e Katie Leiserson; Alex e Christopher Rivest; Molly, Noah e Benjamin Stein – per l'amore e il supporto che ci hanno dato durante la scrittura di questo libro. La pazienza e l'incoraggiamento dei nostri familiari hanno reso possibile questo progetto. A loro dedichiamo con affetto questo libro.

Acquistato da Michele Mo[illegible] 2021-01-08 13:[illegible] Copia non [illegible] 2022 McGraw-Hill Education (Italy)

THOMAS H. CORMEN — *Lebanon, New Hampshire*
CHARLES E. LEISERSON — *Cambridge, Massachusetts*
RONALD L. RIVEST — *Cambridge, Massachusetts*
CLIFFORD STEIN — *New York, New York*

Febbraio 2009

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Presentazione dell'edizione italiana

Cosa può aggiungere il curatore della traduzione di un libro di così grande successo che non sia già stato ampiamente detto nella prefazione degli autori? Mi limiterò a esporre le ragioni per cui ritengo importante una traduzione di tale libro in italiano e le ragioni di alcune scelte adottate nella traduzione.

Nella mia lunga esperienza didattica nell'insegnamento di corsi di Algoritmi ho avuto occasione di visionare molti testi, alcuni scritti direttamente in italiano, altri tradotti dall'inglese, altri ancora nella loro versione originale in inglese. La maggior parte di essi appartiene a due categorie.

Una prima categoria è costituita dai cosiddetti "testi sacri". I testi in questa prima categoria sono troppo "difficili" per essere usati come libro di testo in un corso universitario di primo livello. Essi sono più adatti come testi di riferimento per studiosi della materia e per professionisti di alto livello. Una seconda categoria è costituita da testi che trattano gli algoritmi nel contesto dell'insegnamento di uno specifico linguaggio di programmazione prescelto all'interno del corso universitario.

Il libro di Cormen, Leiserson, Rivest e Stein si colloca nel mezzo tra queste due categorie: rispetto ai testi sacri la trattazione della materia è stata semplificata omettendo quegli argomenti e approfondimenti la cui trattazione richiede delle nozioni matematiche di più alto livello, pur mantenendo la stessa rigorosità di trattazione degli argomenti non omessi. Questo lo rende accessibile a qualsiasi studente di un corso di laurea scientifico di primo livello interessato alla materia, e certamente a ogni studente di un corso di laurea in Informatica.

Rispetto ai testi della seconda categoria la materia trattata è molto più ampia e questo ha diversi vantaggi. Al docente di un corso di algoritmi permette un'ampia possibilità di scelta degli argomenti da trattare a lezione consentendogli di adattare il corso sia al tipo di uditorio sia alle sue preferenze personali. Per gli studenti particolarmente interessati alla materia (forse pochi) costituisce un utile strumento per allargare le conoscenze. Per gli studenti di Informatica che diventeranno informatici professionisti (sperabilmente molti) esso costituirà un utile riferimento per tutti quei problemi algoritmici che inevitabilmente si presenteranno loro nello svolgimento della professione.

Nel corso del lavoro di traduzione e di curatela si sono dovute fare alcune scelte di carattere generale. La prima scelta sofferta è stata quella di trovare un giusto compromesso tra una traduzione che rimanesse più aderente possibile al testo originale e una traduzione libera che mantenesse soltanto il senso del testo tradotto. Se si fosse trattato di un testo letterario la scelta sarebbe stata senz'altro la seconda in quanto lo stile della prosa italiana sarebbe stato di primaria importanza. Trattandosi di un testo tecnico si è ritenuto che l'aderenza al testo originale fosse importante anche se questo avrebbe inevitabilmente appesantito la prosa italiana con qualche reminiscenza delle costruzioni sintattiche inglesi. Crediamo di aver raggiunto un buon compromesso.

Acquistato da Michele Michelucci Webster il 2022-07-07 23:13 - Numero Ordine Libreria: 1905038163 - 20707.0 Copyright © 2022 McGraw-Hill Education (Italy)

Un'altra decisione sofferta è stata quella relativa alla traduzione dei programmi scritti in pseudocodice. Nessun dubbio sul fatto che le parole chiave, **while**, **if**, **else** ecc., non dovessero essere tradotte. Nessun dubbio neppure sul fatto che le istruzioni astratte e i commenti inseriti nello pseudocodice andassero invece tradotti in italiano.

Il problema era se tradurre anche gli identificatori (nomi dei programmi, delle variabili ecc.) usati nello pseudocodice. Da una parte tradurre il nome SORT di un programma che ordina un array con ORDINA avrebbe certamente facilitato la comprensione da parte di studenti completamente digiuni di inglese (sperabilmente una minoranza). Dall'altra, qualunque implementazione di tale algoritmo presente in una libreria di programmi si chiamerà certamente SORT e non ORDINA. Tradurre tali identificatori avrebbe quindi soltanto spostato (e non eliminato) la difficoltà di tali studenti al momento successivo in cui cercare una implementazione dell'algoritmo.

Abbiamo quindi deciso di lasciare nella loro forma originale inglese gli identificatori. Naturalmente un bravo docente, all'occorrenza, non mancherà di spiegare il significato dei termini inglesi usati come identificatori.

*Livio Colussi **
Padova, marzo 2010

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

* **Livio Colussi** è professore associato di Informatica presso la Facoltà di Scienze MM.FF.NN. dell'Università degli Studi di Padova, dove insegna alcuni corsi di Algoritmi a studenti dei corsi di Laurea triennale e specialistica in Informatica.

I Fondamenti

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Introduzione

Questa parte è un'introduzione alle tecniche di progettazione e analisi degli algoritmi. È stata ideata per presentare gradualmente il modo in cui specifichiamo gli algoritmi, alcune strategie di progettazione che saranno utilizzate in questo libro e molti dei concetti fondamentali dell'analisi degli algoritmi. Le parti successive del libro si fondano su queste basi.

Il Capitolo 1 è una panoramica degli algoritmi e del loro ruolo nei moderni sistemi di elaborazione dei dati. Questo capitolo definisce che cos'è un algoritmo e fornisce alcuni esempi. Ipotizza inoltre che gli algoritmi siano una tecnologia, esattamente come le unità hardware veloci, le interfacce grafiche, i sistemi orientati agli oggetti e le reti.

Nel Capitolo 2 presentiamo i primi algoritmi che risolvono il problema dell'ordinamento di una sequenza di n numeri. Ogni algoritmo è scritto con un semplice pseudocodice che, sebbene non sia direttamente traducibile in uno dei linguaggi di programmazione convenzionali, presenta la struttura dell'algoritmo in modo sufficientemente chiaro per consentire a un programmatore di implementarla nel suo linguaggio preferito. Fra gli algoritmi di ordinamento esaminati figurano *insertion sort*, che usa un approccio incrementale, e *merge sort*, che usa una tecnica ricorsiva detta "divide et impera". Sebbene il tempo di calcolo richiesto da ciascun algoritmo cresca con il valore di n, tuttavia il tasso di crescita di questo tempo varia fra i due algoritmi. Il Capitolo 2 descrive come calcolare i tempi di esecuzione degli algoritmi e presenta un'utile notazione per esprimerli.

Il Capitolo 3 definisce con esattezza questa notazione, che chiameremo *notazione asintotica*. Inizialmente, presenteremo varie notazioni asintotiche, che poi utilizzeremo per definire i limiti dei tempi di esecuzione degli algoritmi. La parte restante del capitolo è essenzialmente una presentazione di notazioni matematiche; lo scopo principale non è quello di insegnarvi nuovi concetti matematici, ma bensì garantire che le vostre notazioni siano conformi a quelle adottate in questo libro.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Il Capitolo 4 tratta più approfonditamente il metodo *divide et impera* introdotto nel Capitolo 2. In particolare, presenteremo i metodi per risolvere le ricorrenze, che sono utili per descrivere i tempi di esecuzione degli algoritmi ricorsivi. Una tecnica molto efficace è il "metodo dell'esperto" che può essere impiegato per risolvere le ricorrenze che derivano dagli algoritmi divide et impera. Gran

parte di questo capitolo è dedicata alla dimostrazione della correttezza del metodo dell'esperto (potete comunque tralasciare questa dimostrazione senza alcun problema).

Il Capitolo 5 introduce l'analisi probabilistica e gli algoritmi randomizzati. Tipicamente, l'analisi probabilistica viene utilizzata per determinare il tempo di esecuzione di un algoritmo nel caso in cui, per la presenza di una particolare distribuzione di probabilità, il tempo di esecuzione vari con input diversi della stessa dimensione. In alcuni casi, supponiamo che gli input siano conformi a una distribuzione di probabilità nota, in modo da mediare il tempo di esecuzione su tutti i possibili input. In altri casi, la distribuzione di probabilità non proviene dagli input, ma da scelte casuali effettuate durante lo svolgimento dell'algoritmo. Un algoritmo il cui comportamento è determinato non soltanto dai suoi input, ma anche dai valori prodotti da un generatore di numeri casuali è detto *algoritmo randomizzato*. È possibile utilizzare gli algoritmi randomizzati per imporre una distribuzione di probabilità agli input – garantendo così che nessun input possa sistematicamente provocare una riduzione delle prestazioni – o anche per limitare il tasso di errore di algoritmi cui è consentito produrre risultati affetti da un errore controllato.

Le Appendici A-D trattano altri concetti matematici che vi saranno particolarmente utili durante la lettura di questo libro. È probabile che conosciate già molti degli argomenti descritti nelle appendici (sebbene le particolari notazioni da noi adottate possano differire in alcuni casi da quelle che conoscete), quindi potete considerare le appendici come materiale di riferimento. D'altra parte, potreste non avere mai visto molti argomenti della Parte I. Tutti i capitoli della Parte I e delle appendici sono scritti con la tipica forma dei tutorial.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Ruolo degli algoritmi nell'elaborazione dei dati 1

Che cosa sono gli algoritmi? Perché è utile studiare gli algoritmi? Qual è il ruolo degli algoritmi rispetto ad altre tecnologie utilizzate nei calcolatori? In questo capitolo risponderemo a queste domande.

1.1 Algoritmi

Informalmente, un ***algoritmo*** è una procedura di calcolo ben definita che prende un certo valore, o un insieme di valori, come ***input*** e genera un valore, o un insieme di valori, come ***output***. Un algoritmo è quindi una sequenza di passi computazionali che trasforma l'input in output.

Possiamo anche considerare un algoritmo come uno strumento per risolvere un ***problema computazionale*** ben definito. La definizione del problema specifica in termini generali la relazione di input/output desiderata. L'algoritmo descrive una specifica procedura computazionale per ottenere tale relazione di input/output.

Per esempio, supponiamo di dovere ordinare una sequenza di numeri in ordine non decrescente. Questo problema si presenta spesso nella pratica e rappresenta un terreno fertile per introdurre vari strumenti di analisi e tecniche di progettazione standard. Il ***problema dell'ordinamento*** può essere formalmente definito nel seguente modo:

Input: una sequenza di n numeri $\langle a_1, a_2, \ldots, a_n \rangle$.

Output: una permutazione (riarrangiamento) $\langle a'_1, a'_2, \ldots, a'_n \rangle$ della sequenza di input tale che $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

Per esempio, data la sequenza di input $\langle 31, 41, 59, 26, 41, 58 \rangle$, un algoritmo di ordinamento restituisce come output la sequenza $\langle 26, 31, 41, 41, 58, 59 \rangle$. Tale sequenza di input è detta ***istanza*** del problema dell'ordinamento. In generale, l'***istanza di un problema*** è formata dall'input (che soddisfa tutti i vincoli imposti nella definizione del problema) richiesto per calcolare una soluzione del problema.

L'ordinamento è un'operazione fondamentale in informatica (molti programmi la usano come passo intermedio), per questo abbiamo a disposizione molti buoni algoritmi di ordinamento. La scelta dell'algoritmo più appropriato a una data applicazione dipende – fra l'altro – dal numero di elementi da ordinare, dal livello di ordinamento iniziale degli elementi, da eventuali vincoli sui valori degli elementi, dall'architettura del calcolatore e dal tipo di unità di memorizzazione da utilizzare: memoria principale, dischi o nastri.

Acquistato da Michele Michele su Webster | 2022-07-07 15:12 Numeo - Online Libreria. 1995039016-2207070 © Copyright © 2022, McGraw-Hill Education (Italy)

Un algoritmo si dice ***corretto*** se, per ogni istanza di input, termina con l'output corretto. Diciamo che un algoritmo corretto ***risolve*** il problema computazionale dato. Un algoritmo errato potrebbe non terminare affatto con qualche istanza di input o potrebbe terminare fornendo un risultato errato. Contrariamente a quello che ci si potrebbe aspettare, gli algoritmi errati possono risultare utili qualora sia

possibile mantenere sotto controllo la percentuale di risultati errati. Vedremo un esempio di algoritmo con una percentuale di errori controllabile nel Capitolo 31 quando studieremo gli algoritmi per trovare dei numeri primi grandi. Di solito, tuttavia, ci occuperemo soltanto di algoritmi corretti.

Un algoritmo può essere specificato in lingua italiana, come un programma per computer, o perfino come un progetto hardware. L'unico requisito è che la specifica deve fornire una descrizione esatta della procedura computazionale da seguire.

Quali problemi risolvono gli algoritmi?

L'ordinamento non è affatto l'unico problema computazionale per cui sono stati sviluppati gli algoritmi (molti lo avranno intuito osservando la mole di questo libro). Le applicazioni pratiche degli algoritmi sono innumerevoli; ne citiamo alcune:

- Il Progetto Genoma Umano ha fatto molti progressi allo scopo di identificare tutti i 100 000 geni del DNA umano, determinando le sequenze di 3 miliardi di paia di basi chimiche che formano il DNA umano, registrando queste informazioni nei database e sviluppando gli strumenti per analizzare i dati. Ciascuno di questi passaggi richiede sofisticati algoritmi. Sebbene le soluzioni di questi problemi esulino dagli obiettivi di questo libro, molti dei metodi usati per risolvere questi problemi biologici sfruttano le idee esposte in diversi capitoli di questo libro consentendo così agli scienziati di svolgere i loro compiti utilizzando in modo efficiente le risorse. Si risparmia tempo (di persone e macchine) e denaro, in quanto è possibile estrarre più informazioni dalle tecniche di laboratorio.
- Internet consente agli utenti di tutto il mondo di accedere rapidamente a grandi quantità di informazioni. I siti Internet, con l'ausilio di algoritmi intelligenti, sono in grado di gestire e manipolare tali enormi volumi di dati. Fra gli esempi di problemi che fanno un uso essenziale di algoritmi citiamo la ricerca dei percorsi ottimali che i dati devono seguire (le tecniche per risolvere questi problemi sono descritte nel Capitolo 24) e l'uso di un motore di ricerca per trovare velocemente le pagine che contengono una particolare informazione (le relative tecniche sono trattate nei Capitoli 11 e 32).
- Il commercio elettronico consente di negoziare e scambiare elettronicamente beni e servizi e questo richiede che sia possibile mantenere riservate alcune informazioni personali quali i numeri delle carte di credito, le password e gli estratti conto bancari. Le tecnologie fondamentali usate nel commercio elettronico comprendono la crittografia a chiave pubblica e la firma digitale (che sono trattate nel Capitolo 31) e queste si basano su algoritmi numerici e sulla teoria dei numeri.
- Nelle attività industriali e commerciali spesso è importante allocare delle risorse limitate nel modo più vantaggioso. Una compagnia petrolifera potrebbe essere interessata a sapere dove disporre i propri pozzi per massimizzare i profitti. Un candidato politico potrebbe essere interessato a determinare in quale campagna pubblicitaria investire i suoi soldi per massimizzare le probabilità di vincere le elezioni. Una compagnia aerea potrebbe essere interessata ad assegnare il personale ai voli nel modo più economico possibile, verificando che ogni volo sia coperto e che siano soddisfatte le disposizioni governative sulla programmazione del personale di volo. Un fornitore di servizi Internet potreb-

Acquistato da Michele Michele su Webster il 2022-07-07 22:12 Numero Ordine Libreria 1003016220707 © Copyright © 2022 McGraw-Hill Education (Italy)

be essere interessato a determinare dove allocare delle risorse addizionali per servire i suoi clienti in modo più efficiente. Tutti questi sono esempi di problemi che possono essere risolti utilizzando la programmazione lineare, che sarà trattata nel Capitolo 29.

Sebbene alcuni dettagli di questi esempi esulino dagli scopi di questo libro, tuttavia è opportuno descrivere le tecniche di base che si applicano a questi tipi di problemi. Spiegheremo inoltre come risolvere molti problemi particolari, inclusi i seguenti:

- Supponiamo di avere una carta stradale dove sono segnate le distanze fra ogni coppia di incroci consecutivi e di voler trovare il percorso più breve da un incrocio all'altro. Il numero di percorsi possibili può essere enorme, anche se escludiamo i percorsi che passano più volte per lo stesso posto. Come scegliere il più breve di tutti i percorsi? In questo caso, creiamo un modello della carta stradale (che a sua volta è un modello delle strade reali) nella forma di un grafo (che descriveremo nella Parte VI e nell'Appendice B) e cerchiamo di determinare il cammino più breve da un vertice all'altro del grafo. Spiegheremo come risolvere efficientemente questo problema nel Capitolo 24.
- Supponiamo di avere due sequenze ordinate di simboli, $X = \langle x_1, x_2, \ldots, x_m \rangle$ e $Y = \langle y_1, y_2, \ldots, y_n \rangle$, e di voler trovare una lunga sottosequenza comune di X e Y. Una sottosequenza di X è la stessa sequenza X alla quale sono stati tolti alcuni (anche tutti o nessuno) dei suoi elementi. Per esempio, una sottosequenza di $\langle A, B, C, D, E, F, G \rangle$ potrebbe essere $\langle B, C, E, G \rangle$. La lunghezza della più lunga sottosequenza comune di X e Y fornisce una misura della somiglianza di queste due sequenze. Per esempio, se le due sequenze sono una coppia di basi nei filamenti del DNA, allora dobbiamo considerarle simili se hanno una più lunga sottosequenza comune. Se X ha m simboli e Y ha n simboli, allora X e Y hanno, rispettivamente, 2^m e 2^n sottosequenze possibili. Selezionare tutte le possibili sottosequenze di X e Y per poi confrontarle potrebbe richiedere un tempo eccessivamente lungo, a meno che m ed n non siano molto piccoli. Nel Capitolo 15 vedremo come utilizzare una tecnica generale, detta programmazione dinamica, per risolvere questo problema in modo molto più efficiente.
- Un progetto meccanico è costituito da un certo numero di parti, ciascuna delle quali può includere altre parti; occorre elencare le parti in modo che ciascuna parte appaia prima delle parti che la usano. Se il progetto è formato da n parti, ci sono $n!$ ordinamenti possibili, dove $n!$ indica la funzione fattoriale. Poiché la funzione fattoriale cresce più rapidamente di una funzione esponenziale, non sarebbe praticabile generare tutti i possibili ordinamenti e poi verificare che, all'interno di ogni ordinamento, ciascuna parte appaia prima delle parti che la usano (a meno che il progetto non abbia poche parti). Questo problema è un'istanza dell'ordinamento topologico e nel Capitolo 22 vedremo come può essere risolto in modo efficiente.
- Dati n punti nel piano, vogliamo determinare l'involucro convesso di questi punti. L'involucro convesso è il più piccolo poligono convesso che contiene i punti. Intuitivamente, possiamo immaginare ogni punto come se fosse rappresentato da un chiodo che fuoriesce da una tavola. L'involucro convesso potrebbe essere rappresentato da un elastico teso che circonda tutti i chiodi.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503616-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Ogni chiodo attorno al quale l'elastico ruota è un vertice dell'involucro convesso (un esempio è illustrato nella Figura 33.6 a pagina 857). Ciascuno dei 2^n sottoinsiemi dei punti potrebbe costituire l'insieme dei vertici dell'involucro. Inoltre, conoscere i punti che formano i vertici dell'involucro convesso non è sufficiente, in quanto occorre sapere anche l'ordine in cui essi si presentano. Ci sono dunque molte possibilità di scelta per i vertici dell'involucro convesso. Il Capitolo 33 descrive due buoni metodi per trovare l'involucro convesso.

Questo elenco non è affatto esaustivo (come probabilmente avrete immaginato dalle dimensioni di questo libro), ma presenta due caratteristiche che sono comuni a molti importanti problemi algoritmici.

1. Esistono numerose soluzioni possibili, la maggior parte delle quali non risolve il nostro particolare problema. Scegliere una soluzione che risolve il problema, e possibilmente quella "migliore", può essere un'impresa ardua.
2. Esistono varie applicazioni pratiche. Fra i problemi precedentemente elencati, determinare il percorso più breve rappresenta l'esempio più semplice. Un'azienda di trasporti su strada o rotaie è interessata a trovare i percorsi minimi nelle reti stradali o ferroviarie, perché tali percorsi consentono di risparmiare costi di manodopera e carburante. Come altro esempio, in un nodo di instradamento su Internet potrebbe essere necessario trovare il percorso più breve nella rete che permette di instradare rapidamente un messaggio. Analogamente, una persona che volesse andare in auto da Milano a Roma potrebbe trovare il percorso da fare in un sito web appropriato oppure potrebbe utilizzare il GPS mentre guida.

Non tutti i problemi risolti dagli algoritmi hanno un gruppo di soluzioni possibili facilmente identificabile.

Per esempio, supponiamo di avere un insieme di valori numerici che rappresentano dei campioni di un segnale e di voler calcolare la trasformata discreta di Fourier di questi campioni. La trasformata discreta di Fourier converte il dominio del tempo nel dominio della frequenza, generando una serie di coefficienti numerici che ci consentono di determinare l'influenza delle varie frequenze sul segnale campionato. Oltre ad essere utilizzate nell'elaborazione dei segnali, le trasformate discrete di Fourier trovano applicazione nella compressione dei dati e nella moltiplicazione di grandi polinomi e numeri interi. Il Capitolo 30 descrive un algoritmo efficiente, la trasformata rapida di Fourier (nota come FFT), per questo problema; il capitolo presenta anche lo schema di un circuito hardware per calcolare la FFT.

Strutture dati

Questo libro contiene anche diverse strutture dati. Una ***struttura dati*** è un modo per memorizzare e organizzare i dati e semplificarne l'accesso e la modifica. Non esiste un'unica struttura dati che vada bene per qualsiasi compito, quindi è importante conoscere vantaggi e svantaggi di queste strutture.

Acquistato da Michele Michele su Webster il 2022-07-19 23:12 - Numero Ordine Libreria: ebs_8038016 - Copyright © 2022, McGraw-Hill Education (Italy)

Tecnica

Sebbene possiate utilizzare questo libro come un "libro di ricette" per algoritmi, tuttavia un giorno potreste incontrare un problema per il quale non riuscite a trovare un algoritmo pubblicato (come molti esercizi e problemi di questo libro). Il libro vi insegna le tecniche per progettare e analizzare gli algoritmi, in modo che possiate sviluppare i vostri algoritmi, dimostrare che forniscono la risposta esatta

e valutare la loro efficienza. Molti capitoli trattano vari aspetti della risoluzione dei problemi tramite algoritmi. Alcuni capitoli trattano problemi specifici, come trovare le mediane e altre statistiche d'ordine nel Capitolo 9, calcolare gli alberi di connessione minimi nel Capitolo 23 e determinare un flusso massimo in una rete nel Capitolo 26. Altri capitoli descrivono tecniche, come divide et impera nel Capitolo 4, la programmazione dinamica nel Capitolo 15 e l'analisi ammortizzata nel Capitolo 17.

Problemi difficili

Gran parte di questo libro è dedicata agli algoritmi efficienti. La tipica unità di misura dell'efficienza è la velocità, ovvero quanto tempo impiega un algoritmo per produrre il suo risultato. Ci sono problemi, tuttavia, per i quali non si conosce una soluzione efficiente. Il Capitolo 34 studia un interessante sottoinsieme di questi problemi, noti come problemi NP-completi.

Perché sono interessanti i problemi NP-completi? In primo luogo, sebbene non sia stato ancora trovato un algoritmo efficiente per un problema NP-completo, tuttavia nessuno ha dimostrato che non possa esistere un algoritmo efficiente per uno di questi problemi. In altre parole, non sappiamo se esistano algoritmi efficienti per i problemi NP-completi. In secondo luogo, l'insieme dei problemi NP-completi gode dell'importante proprietà che, se esiste un algoritmo efficiente per uno di essi, allora esistono algoritmi efficienti per ciascuno di essi. Questa relazione fra i problemi NP-completi rende molto più interessante la mancanza di soluzioni efficienti. In terzo luogo, molti problemi NP-completi sono simili, ma non identici, a problemi per i quali conosciamo algoritmi efficienti. Gli informatici rimangono spesso sorpresi da come una piccola variazione nella definizione del problema può causare una grande variazione dell'efficienza del migliore algoritmo conosciuto.

È importante conoscere i problemi NP-completi perché alcuni di essi si presentano nelle applicazioni reali molto più spesso di quanto ci aspetteremmo. Se vi chiedessero di creare un algoritmo efficiente per un problema NP-completo, rischiereste di sprecare molto del vostro tempo in ricerche inutili. Se riuscite a dimostrare che il problema è NP-completo, allora potrete impiegare il vostro tempo a sviluppare un algoritmo efficiente che fornisce una buona soluzione, non la migliore possibile.

Come esempio concreto, considerate un'impresa di spedizioni che abbia un magazzino centrale. Tutte le mattine ciascun autocarro viene caricato presso il magazzino e poi indirizzato alle varie destinazioni per consegnare le merci. Alla fine della giornata ogni autocarro deve ritornare al magazzino per essere pronto per il giorno successivo. Per ridurre i costi, l'azienda intende scegliere un ordine di fermate per le consegne che consenta a ciascun autocarro di percorrere la distanza minima. Si tratta del cosiddetto "problema del commesso viaggiatore" ed è un problema NP-completo. Non esiste un algoritmo efficiente. Sotto opportune ipotesi, tuttavia, sono noti degli algoritmi efficienti che forniscono una distanza complessiva che non è molto diversa da quella minima. Il Capitolo 35 tratta questi "algoritmi di approssimazione".

Acquistato da Michele Mignone su Webster il 2022-07-07 20:12 - Agenzia Chiavi Libreria. 1955030161220707 © Copyright © 2021, McGraw-Hill Education (Italy)

Parallelismo

Per molti anni, abbiamo potuto confidare su velocità di clock dei processori costantemente in crescita. Tuttavia, le limitazioni fisiche rappresentano un ostacolo

fondamentale per le velocità sempre crescenti: poiché la densità di potenza aumenta in modo superlineare con la velocità di clock, i chip corrono il rischio di fondere una volta che le loro velocità di clock diventano sufficientemente elevate. Per poter eseguire più operazioni al secondo, quindi, i chip sono progettati per contenere non uno ma più "nuclei" (core) di elaborazione. Possiamo paragonare questi computer multicore a più computer sequenziali in un singolo chip; in altre parole, essi sono un tipo di "computer parallelo". Al fine di ottenere le migliori prestazioni dai computer multicore, dobbiamo progettare gli algoritmi tenendo conto del parallelismo. Il Capitolo 27 presenta un modello per algoritmi "multithread" che sono in grado di sfruttare le potenzialità offerte dai chip multicore. Questo modello offre dei vantaggi da un punto di vista teorico ed è alla base di numerosi software di successo, incluso un programma per campionati di scacchi.

Esercizi

1.1-1
Indicate un esempio nel mondo reale in cui è richiesto l'ordinamento o uno in cui è richiesto di trovare l'involucro convesso.

1.1-2
Oltre alla velocità, quali altri indici di efficienza potrebbero essere utilizzati in un ambiente reale?

1.1-3
Scegliete una struttura dati che avete visto in precedenza e analizzatene vantaggi e svantaggi.

1.1-4
In che modo sono simili i problemi del percorso minimo e del commesso viaggiatore? In che modo differiscono?

1.1-5
Descrivete un problema del mondo reale in cui è ammissibile soltanto la soluzione ottima. Poi indicatene uno in cui è accettabile una soluzione che "approssima" quella ottima.

1.2 Algoritmi come tecnologia

Se i computer fossero infinitamente veloci e la memoria dei computer fosse gratuita, avremmo ancora qualche motivo per studiare gli algoritmi? La risposta è sì, se non altro perché vorremmo ugualmente dimostrare che il nostro metodo di risoluzione termina e fornisce la soluzione esatta.

Se i computer fossero infinitamente veloci, qualsiasi metodo corretto per risolvere un problema andrebbe bene. Probabilmente, vorremmo che la nostra implementazione rispettasse le buone norme dell'ingegneria del software (ovvero fosse ben progettata e documentata), ma il più delle volte adotteremmo il metodo più semplice da implementare. Ovviamente, i computer possono essere veloci, ma non infinitamente veloci. La memoria può costare poco, ma non può essere gratuita. Il tempo di elaborazione e lo spazio nella memoria sono risorse limitate, che devono essere saggiamente utilizzate; gli algoritmi che sono efficienti in termini di tempo o spazio ci aiuteranno a farlo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0. Copyright © 2022, McGraw-Hill Education (Italy)

Efficienza

Algoritmi progettati per risolvere lo stesso problema spesso sono notevolmente diversi nella loro efficienza. Queste differenze possono essere molto più significative di quelle dovute all'hardware e al software.

Per esempio, nel Capitolo 2 esamineremo due algoritmi di ordinamento. Il primo, detto ***insertion sort***, impiega un tempo pari a circa $c_1 n^2$ per ordinare n elementi, dove c_1 è una costante che non dipende da n; ovvero occorre un tempo all'incirca proporzionale a n^2. Il secondo algoritmo, ***merge sort***, richiede un tempo pari a circa $c_2 n \lg n$, dove $\lg n$ sta per $\log_2 n$ e c_2 è un'altra costante che non dipende da n. Insertion sort, di solito, ha un fattore costante più piccolo di merge sort ($c_1 < c_2$). Vedremo come i fattori costanti abbiano un impatto molto minore sul tempo di calcolo della dipendenza dalla dimensione n dell'input. Se indichiamo il tempo di esecuzione di insertion sort come $c_1 n \cdot n$ e quello di merge sort come $c_2 n \cdot \lg n$, allora notiamo che quando insertion sort ha un fattore n nel suo tempo di esecuzione, merge sort ha un fattore $\lg n$, che è molto più piccolo. (Per esempio, quando $n = 1000$, $\lg n$ è circa 10, e quando n è uguale a un milione, $\lg n$ è pari a circa 20.) Sebbene insertion sort, di solito, sia più veloce di merge sort per input di piccole dimensioni, tuttavia quando la dimensione dell'input n diventa relativamente grande, il vantaggio di merge sort, $\lg n$ su n, compensa abbondantemente la differenza fra i fattori costanti. Indipendentemente da quanto c_1 sia più piccola rispetto a c_2, ci sarà sempre un punto oltre il quale merge sort è più rapido.

Come esempio concreto, mettiamo a confronto un computer veloce (computer A) che esegue insertion sort e un computer lento (computer B) che esegue merge sort. Entrambi devono ordinare un array di 10 milioni di numeri (sebbene 10 milioni di numeri possano sembrare tanti, se i numeri sono interi a 8 byte, allora l'input richiede circa 80 megabyte, e può essere facilmente contenuto anche nella memoria di un computer economico). Supponiamo che il computer A esegua 10 miliardi di istruzioni al secondo (più veloce di qualsiasi computer sequenziale al momento in cui scriviamo) e il computer B esegua soltanto 10 milioni di istruzioni al secondo, ovvero il computer A è 1000 volte più veloce del computer B in termini di potenza di calcolo. Per rendere la differenza ancora più evidente, supponiamo che il miglior programmatore del mondo abbia codificato insertion sort nel linguaggio macchina del computer A e che il codice risultante richieda $2n^2$ istruzioni per ordinare n numeri. Merge sort, invece, è stato programmato per il computer B da un programmatore medio con un linguaggio di alto livello e un compilatore inefficiente; il codice risultante richiede $50n \lg n$ istruzioni. Per ordinare 10 milioni di numeri, il computer A impiega

$$\frac{2 \cdot (10^7)^2 \text{ istruzioni}}{10^{10} \text{ istruzioni/secondo}} = 20000 \text{ secondi (più di 5,5 ore)}$$

mentre il computer B impiega

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\frac{50 \cdot 10^7 \lg 10^7 \text{ istruzioni}}{10^7 \text{ istruzioni/secondo}} \approx 1163 \text{ secondi (meno di 20 minuti)}$$

Utilizzando un algoritmo il cui tempo di esecuzione cresce più lentamente, perfino con un compilatore scadente, il computer B è 17 volte più veloce del computer A! Il vantaggio di merge sort è ancora più significativo se ordiniamo 100 milioni di numeri: insertion sort impiega più di 23 giorni, merge sort meno di 4 ore. In

generale, al crescere della dimensione del problema, aumenta il vantaggio relativo di merge sort.

Algoritmi e altre tecnologie

L'esempio dimostra che gli algoritmi, come l'hardware, sono una ***tecnologia***. Le prestazioni globali di un sistema dipendono tanto dalla scelta di algoritmi efficienti quanto dalla scelta di un hardware veloce. Analogamente a quanto è avvenuto in altre tecnologie per calcolatori, anche gli algoritmi hanno avuto un loro rapido progresso. Qualcuno potrebbe chiedersi se gli algoritmi siano davvero così importanti per i moderni calcolatori alla luce di altre tecnologie avanzate, quali

- architetture di computer e tecnologie di fabbricazione avanzate
- interfacce grafiche (GUI) intuitive e facili da usare
- sistemi orientati agli oggetti
- tecnologie web integrate
- reti veloci (cablate e wireless)

La risposta è sì. Sebbene ci siano applicazioni che non richiedano esplicitamente un contenuto algoritmico a livello dell'applicazione (per esempio, alcune semplici applicazioni web), molte richiedono una certa dose di contenuto algoritmico. Per esempio, considerate un servizio web che determina come spostarsi da un sito all'altro. La sua implementazione dovrebbe fare affidamento su un hardware veloce, un'interfaccia grafica utente, una rete WAN e, possibilmente, anche su sistemi orientati agli oggetti; ma richiederebbe anche gli algoritmi per svolgere determinate operazioni, come trovare i percorsi (utilizzando un algoritmo per il percorso più breve), rappresentare le mappe e interpolare gli indirizzi.

Inoltre, anche un'applicazione che non richiede un contenuto algoritmico a livello applicazione fa affidamento sugli algoritmi. L'applicazione fa affidamento su un hardware veloce? Il progetto dell'hardware utilizza gli algoritmi. L'applicazione fa affidamento su un'interfaccia grafica utente? Il progetto di qualsiasi interfaccia grafica utilizza pesantemente gli algoritmi. L'applicazione fa affidamento sulle reti? Il routing delle reti impiega gli algoritmi. L'applicazione è stata scritta in un linguaggio diverso dal codice machina? Allora è stata elaborata da un compilatore, un interprete o un assemblatore, ciascuno dei quali utilizza ampiamente gli algoritmi. Gli algoritmi sono il nucleo delle principali tecnologie utilizzate nei moderni calcolatori.

Grazie alle loro sempre crescenti capacità, i calcolatori vengono utilizzati per risolvere problemi più complicati che in passato. Come abbiamo visto nel precedente confronto fra gli algoritmi insertion sort e merge sort, è con i problemi di dimensioni maggiori che le differenze di efficienza fra gli algoritmi diventano particolarmente evidenti.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria 1995909818 220707 Copyright © 2022, McGraw-Hill Education (Italy)

Avere una solida base di conoscenza degli algoritmi e delle tecniche è una caratteristica che contraddistingue i programmatori esperti dai principianti. Con i moderni calcolatori, potete svolgere alcuni compiti senza sapere molto di algoritmi, ma con una buona conoscenza degli algoritmi, potete fare molto, molto di più.

Esercizi

1.2-1

Indicate l'esempio di un'applicazione che richiede un contenuto algoritmico a livello dell'applicazione e descrivete la funzione degli algoritmi richiesti.

1.2-2

Supponete di confrontare le implementazioni di insertion sort e merge sort sulla stessa macchina. Con un input di dimensione n, insertion sort viene eseguito in $8n^2$ passi, mentre merge sort viene eseguito in $64n \lg n$ passi. Per quali valori di n insertion sort batte merge sort?

1.2-3

Qual è il più piccolo valore di n per cui un algoritmo il cui tempo di esecuzione è $100n^2$ viene eseguito più velocemente di un algoritmo il cui tempo di esecuzione è 2^n sulla stessa macchina?

Problemi

1-1 Confronto fra i tempi di esecuzione

Per ogni funzione $f(n)$ e tempo t della seguente tabella, determinate la massima dimensione n di un problema che può essere risolto nel tempo t, supponendo che l'algoritmo che risolve il problema impieghi $f(n)$ microsecondi.

	1 secondo	1 minuto	1 ora	1 giorno	1 mese	1 anno	1 secolo
$\lg n$							
$\sqrt{n}$							
n							
$n \lg n$							
n^2							
n^3							
2^n							
$n!$							

Note

Ci sono molti testi eccellenti che trattano in generale gli algoritmi, fra i quali citiamo: Aho, Hopcroft e Ullman [5, 6]; Baase e Van Gelder [28]; Brassard e Bratley [55]; Dasgupta, Papadimitriou e Vazirani [83]; Goodrich e Tamassia [149]; Hofri [176]; Horowitz, Sahni e Rajasekaran [182]; Johnsonbaugh e Schaefer [194]; Kingston [206]; Kleinberg e Tardos [209]; Knuth [210, 211, 212]; Kozen [221]; Levitin [236]; Manber [243]; Mehlhorn [250, 251, 252]; Purdom e Brown [288]; Reingold, Nievergelt e Deo [294]; Sedgewick [307]; Sedgewick e Flajolet [308]; Skiena [319]; e Wilf [357].

Acquistato da Michele Michele s [illegible]

Alcuni degli aspetti più pratici della progettazione degli algoritmi sono descritti da Bentley [42, 43] e Gonnet [146]. I manuali *Handbook of Theoretical Computer Science*, Volume A [343] e *CRC Handbook on Algorithms and Theory of Computation* [25] riportano alcuni studi sugli algoritmi. Una panoramica sugli algoritmi utilizzati nella biologia computazionale si trova nei libri di testo di Gusfield [157], Pevzner [276], Setubal e Meidanis [311] e Waterman [351].

2 Per incominciare

Questo capitolo consente ai lettori di acquisire familiarità con i concetti fondamentali della progettazione e dell'analisi degli algoritmi. È un capitolo autonomo, sebbene includa riferimenti ad argomenti che saranno introdotti nei Capitoli 3 e 4 (presenta anche alcune sommatorie che saranno descritte nell'Appendice A).

Inizieremo a esaminare l'algoritmo insertion sort per risolvere il problema dell'ordinamento introdotto nel Capitolo 1. Definiremo uno "pseudocodice" che dovrebbe essere familiare a chiunque abbia già scritto qualche programma per computer e lo utilizzeremo per mostrare come specificheremo i nostri algoritmi. Dopo avere specificato l'algoritmo insertion sort, controlleremo che esso effettui correttamente l'ordinamento e analizzeremo il suo tempo di esecuzione. L'analisi introduce una notazione che si concentra su come cresce questo tempo con il numero di elementi da ordinare. Successivamente, introdurremo l'approccio *divide et impera* per progettare algoritmi e sviluppare un algoritmo detto *merge sort*. Il capitolo termina con un'analisi del tempo di esecuzione di merge sort.

2.1 Insertion sort

Il nostro primo algoritmo, insertion sort, risolve il ***problema dell'ordinamento*** introdotto nel Capitolo 1:

Input: una sequenza di n numeri $\langle a_1, a_2, \ldots, a_n \rangle$.

Output: una permutazione (riarrangiamento) $\langle a'_1, a'_2, \ldots, a'_n \rangle$ della sequenza di input tale che $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

I numeri da ordinare sono anche detti ***chiavi***. Sebbene concettualmente stiamo ordinando una sequenza, l'input si presenta nella forma di un array di n elementi.

In questo libro, tipicamente, descriveremo gli algoritmi come programmi scritti in uno ***pseudocodice*** che è simile per molti aspetti ai linguaggi C, C++, Java, Python e Pascal. Se conoscete uno di questi linguaggi, non dovreste incontrare molte difficoltà a leggere i nostri algoritmi. Ciò che distingue lo pseudocodice dal codice "reale" è che nello pseudocodice impieghiamo qualsiasi mezzo espressivo che specifichi nel modo più chiaro e conciso un determinato algoritmo. A volte, il mezzo più chiaro è l'italiano, quindi non sorprendetevi se incontrate una frase in italiano all'interno di una sezione di codice "reale". Un'altra differenza fra pseudocodice e codice reale è che il primo, tipicamente, non si occupa dei problemi di ingegneria del software. Problemi quali l'astrazione dei dati, la modularità e la gestione degli errori, di solito, vengono ignorati per potere esprimere in modo più conciso l'essenza di un algoritmo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016.220707.0 Copyright © 2022, McGraw-Hill Education (Italy)

Iniziamo con ***insertion sort***, che è un algoritmo efficiente per ordinare un piccolo numero di elementi. Questo algoritmo opera nello stesso modo in cui molte persone ordinano le carte da gioco. Iniziamo con la mano sinistra vuota e le carte

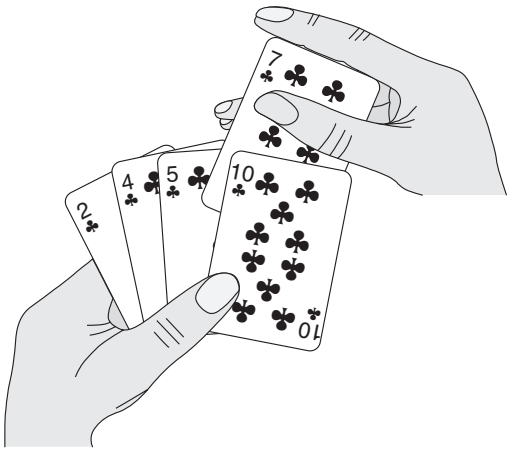


Figura 2.1 Ordinare una mano di carte mediante insertion sort.

coperte poste sul tavolo. Prendiamo una carta alla volta dal tavolo e la inseriamo nella posizione corretta nella mano sinistra. Per trovare la posizione corretta di una carta, la confrontiamo con le singole carte che abbiamo già in mano, da destra a sinistra, come illustra la Figura 2.1. In qualsiasi momento, le carte che teniamo nella mano sinistra sono ordinate; originariamente queste carte erano le prime della pila di carte che erano sul tavolo.

Il nostro pseudocodice per insertion sort è presentato come una procedura chiamata INSERTION-SORT, che prende come parametro un array $A[1\,..\,n]$ contenente una sequenza di lunghezza n che deve essere ordinata (nel codice il numero n di elementi di A è indicato da $A.length$. L'algoritmo ordina i numeri di input ***sul posto***: i numeri sono risistemati all'interno dell'array A avendo, in ogni istante, al più un numero costante di essi memorizzati all'esterno dell'array. Quando la procedura INSERTION-SORT è completata, l'array di input A contiene la sequenza di output ordinata.

```
INSERTION-SORT(A)
1  for j = 2 to A.length
2      key = A[j]
3      // Inserisce A[j] nella sequenza ordinata A[1 .. j − 1].
4      i = j − 1
5      while i > 0 and A[i] > key
6          A[i + 1] = A[i]
7          i = i − 1
8      A[i + 1] = key
```

Invarianti di ciclo e correttezza di insertion sort

La Figura 2.2 mostra come opera questo algoritmo con $A = \langle 5, 2, 4, 6, 1, 3\rangle$. L'indice j identifica la "carta corrente" che viene inserita nelle altre. All'inizio di ogni iterazione del ciclo **for**, il cui indice è j, il sottoarray che è formato dagli elementi $A[1\,..\,j-1]$ costituisce la mano di carte correntemente ordinate e gli elementi $A[j+1\,..\,n]$ corrispondono alla pila delle carte che si trovano ancora sul tavolo. In effetti, gli elementi $A[1\,..\,j-1]$ sono quelli che *originariamente* occupavano le posizioni da 1 a $j-1$, ma che adesso sono ordinati. Definiamo formalmente queste proprietà di $A[1\,..\,j-1]$ come ***invariante di ciclo***:

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995030[illegible] Copyright © 2022, McGraw-Hill Education (Italy)

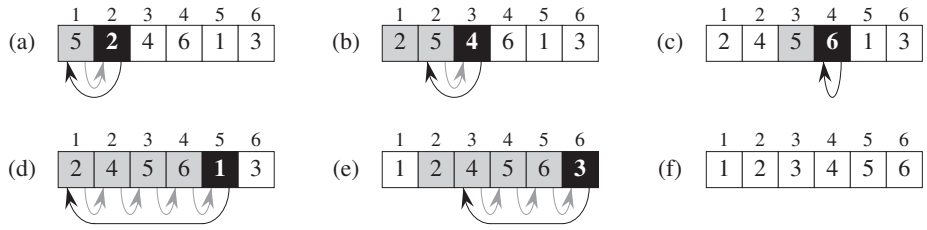


Figura 2.2 Il funzionamento di INSERTION-SORT con l'array $A = \langle 5, 2, 4, 6, 1, 3 \rangle$. Gli indici dell'array sono indicati sopra i rettangoli; i valori memorizzati nelle posizioni dell'array sono indicati all'interno dei rettangoli. **(a)–(e)** Le iterazioni del ciclo **for** (righe 1–8). In ogni iterazione, il rettangolo nero contiene la chiave estratta da $A[j]$, che viene confrontata con i valori nei rettangoli grigi alla sua sinistra nel test della riga 5. Le frecce grige mostrano i valori dell'array che vengono spostati di una posizione verso destra nella riga 6; le frecce nere indicano dove viene spostata la chiave nella riga 8. **(f)** L'array finale ordinato.

> All'inizio di ogni iterazione del ciclo **for** (righe 1–8), il sottoarray $A[1 \,..\, j-1]$ è ordinato ed è formato dagli stessi elementi che originariamente erano in $A[1 \,..\, j-1]$.

Utilizziamo le invarianti di ciclo per aiutarci a capire perché un algoritmo è corretto. Dobbiamo dimostrare tre cose su un'invariante di ciclo:

Inizializzazione: è vera prima della prima iterazione del ciclo.

Conservazione: se è vera prima di un'iterazione del ciclo, rimane vera prima della successiva iterazione.

Conclusione: quando il ciclo termina, l'invariante fornisce un'utile proprietà che ci aiuta a dimostrare che l'algoritmo è corretto.

Quando le prime due proprietà sono valide, l'invariante di ciclo è vera prima di ogni iterazione del ciclo. (Ovviamente, siamo liberi di basarci su dati di fatto diversi dalla stessa invariante di ciclo per provare che l'invariante di ciclo resta vera prima di ogni iterazione.) Notate l'analogia con l'induzione matematica, dove per provare che una proprietà è valida, si prova un caso base e un passo induttivo. Qui, dimostrare che l'invariante è vera prima della prima iterazione equivale al caso base e dimostrare che l'invariante resta vera da un'iterazione all'altra equivale al passo induttivo.

La terza proprietà è forse la più importante, perché utilizziamo l'invariante di ciclo per dimostrare la correttezza. Tipicamente, si usa l'invariante di ciclo insieme con la condizione che ha determinato la conclusione del ciclo. La presenza di una conclusione differisce dall'uso consueto dell'induzione matematica, dove il passo induttivo viene utilizzato all'infinito; qui invece interrompiamo il "processo induttivo" quando il ciclo termina.

Vediamo se queste proprietà sono valide per insertion sort.

Inizializzazione: iniziamo dimostrando che l'invariante di ciclo è vera prima della prima iterazione del ciclo, quando $j = 2$.[1] Il sottoarray $A[1 \,..\, j-1]$, quindi, è formato dal solo elemento $A[1]$, che infatti è l'elemento originale in $A[1]$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[1] Quando il ciclo è un ciclo **for**, il punto in cui verifichiamo l'invariante di ciclo appena prima della prima iterazione è immediatamente dopo l'assegnazione iniziale del contatore del ciclo e appena prima del primo test del ciclo. Nel caso di INSERTION-SORT, questo punto è dopo l'assegnazione di 2 alla variabile j, ma prima del primo test $j \leq A.length$.

Inoltre, questo sottoarray è ordinato (banale, ovviamente) e ciò dimostra che l'invariante di ciclo è vera prima della prima iterazione del ciclo.

Conservazione: passiamo alla seconda proprietà: dimostrare che ogni iterazione conserva l'invariante di ciclo. Informalmente, il corpo del ciclo **for** esterno opera spostando $A[j-1]$, $A[j-2]$, $A[j-3]$ e così via di una posizione verso destra, finché non troverà la posizione appropriata per $A[j]$ (righe 4–7), dove inserirà il valore di $A[j]$ (riga 8). Il sottoarray $A[1\,..\,j]$ quindi è ordinato ed è formato dagli stessi elementi che originariamente erano in $A[1\,..\,j]$. Dunque, l'incremento di j per la successiva iterazione del ciclo **for** preserva l'invariante di ciclo.

Un trattamento più formale della seconda proprietà richiederebbe di definire e dimostrare un'invariante di ciclo per il ciclo **while** delle righe 5–7. A questo punto, tuttavia, preferiamo non impantanarci in simili formalismi; quindi, confidiamo nella nostra analisi informale per dimostrare che la seconda proprietà è vera per il ciclo esterno.

Conclusione: infine, esaminiamo che cosa accade quando il ciclo termina. La condizione che determina la conclusione del ciclo **for** è: $j > A.length = n$. Poiché ogni iterazione del ciclo aumenta j di 1, alla fine del ciclo si avrà $j = n + 1$. Sostituendo j con $n + 1$ nella formulazione dell'invariante di ciclo, otteniamo che il sottoarray $A[1\,..\,n]$ è formato dagli elementi *ordinati* che si trovavano originariamente in $A[1\,..\,n]$. Ma il sottoarray $A[1\,..\,n]$ è l'intero array e dunque tutto l'array è ordinato. Pertanto l'algoritmo è corretto.

Applicheremo ancora questo metodo delle invarianti di ciclo per dimostrare la correttezza più avanti in questo capitolo e in altri capitoli.

Convenzioni di pseudocodifica

Adotteremo le seguenti convenzioni nelle nostre pseudocodifiche.

- L'indentazione (rientro verso destra delle righe) serve a indicare la struttura a blocchi dello pseudocodice. Per esempio, il corpo del ciclo **for**, che inizia nella riga 1, è formato dalla righe 2–8 e il corpo del ciclo **while**, che inizia nella riga 5, contiene le righe 6–7, ma non la riga 8. Il nostro stile di indentazione si applica anche alle istruzioni **if-else**.[2] Utilizzando l'indentazione, anziché gli indicatori convenzionali della struttura a blocchi, come le istruzioni **begin** e **end**, si riduce di molto il disordine, preservando o perfino migliorando la chiarezza.[3]
- I costrutti iterativi **while**, **for** e **repeat-until** e il costrutto condizionale **if-else** hanno interpretazioni simili a quelle dei linguaggi C, C++, Python, Java e

[2] In una istruzione **if-else**, indentiamo la parola chiave **else** allo stesso livello della sua corrispondente **if**. Sebbene omettiamo la parola chiave **then**, occasionalmente facciamo riferimento alla porzione eseguita quando il test che segue **if** è vero come a una clausola ***then***. Per i test a più vie, utilizziamo **elseif** per i test dopo il primo.

[3] In questo libro lo pseudocodice di una procedura sarà sempre contenuto in una sola pagina in modo tale che non sia mai necessario riconoscere i livelli di identazione in un codice suddiviso tra pagine successive.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950310-220707-0 Copyright © 2021, McGraw-Hill Education (Italy)

Pascal.[4] In questo libro il contatore del ciclo mantiene il suo valore dopo la fine del ciclo, diversamente da quanto accade in alcuni casi nei linguaggi C++, Java e Pascal. Quindi, immediatamente dopo un ciclo **for**, il valore del contatore del ciclo è quello che ha appena superato il limite del ciclo **for**. Abbiamo utilizzato questa proprietà nella nostra analisi della correttezza di insertion sort. La prima istruzione del ciclo **for** (riga 1) è **for** $j = 2$ **to** $A.length$; quindi, alla fine di questo ciclo, $j = A.length + 1$ (che equivale a $j = n + 1$, in quanto $n = A.length$). Utilizziamo la parola chiave **to** quando un ciclo **for** aumenta il valore del suo contatore in ogni iterazione e la parola chiave **downto** quando diminuisce tale valore. Quando il contatore del ciclo varia di una quantità maggiore di 1, il valore di tale quantità è indicato dopo la parola chiave facoltativa **by**.

- Il simbolo "**//**" indica che il resto della riga è un commento.
- Un'assegnazione multipla della forma $i = j = e$ assegna a entrambe le variabili i e j il valore dell'espressione e; deve essere considerata equivalente all'assegnazione $j = e$ seguita dall'assegnazione $i = j$.
- Le variabili (come i, j e *key*) sono locali a una determinata procedura. Non useremo mai variabili globali senza un'esplicita indicazione.
- Per identificare un elemento di un array, specifichiamo il nome dell'array seguito dall'indice dell'elemento fra parentesi quadre. Per esempio, $A[i]$ indica l'elemento i-esimo dell'array A. La notazione ". ." è utilizzata per indicare un intervallo di valori all'interno di un array. Quindi, $A[1 \,..\, j]$ indica il sottoarray di A che è composto da j elementi: $A[1], A[2], \ldots, A[j]$.
- I dati composti sono tipicamente organizzati in ***oggetti***, che sono formati da ***attributi***. Un particolare attributo è identificato utilizzando la sintassi tipica di molti linguaggi di programmazione orientata agli oggetti: il nome dell'oggetto seguito da un punto e dal nome dell'attributo. Per esempio, noi trattiamo un array come un oggetto con l'attributo *length* che indica il numero di elementi contenuti nell'array. Per specificare il numero di elementi di un array A, scriviamo $A.length$.

  Una variabile che rappresenta un array o un oggetto è trattata come un puntatore ai dati che costituiscono l'array o l'oggetto. Per tutti gli attributi f di un oggetto x, l'assegnazione $y = x$ implica che $y.f$ sia uguale a $x.f$. Inoltre, se poi impostiamo $x.f = 3$, allora non soltanto $x.f$ sarà uguale a 3, ma anche $y.f$ sarà uguale a 3. In altre parole, x e y puntano allo stesso oggetto dopo l'assegnazione $y = x$.

  La nostra notazione degli attributi può effettuarsi in "cascata". Per esempio, supponiamo che l'attributo f sia un puntatore a un tipo di oggetto che ha un attributo g. La notazione $x.f.g$ viene implicitamente interpretata come $(x.f).g$. In altre parole, se avessimo assegnato $y = x.f$, allora $x.f.g$ sarebbe equivalente a $y.g$.

Acquistato da Michele Michele su Webster il 2022-07-07 2013, Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[4] Molti linguaggi con strutture a blocchi hanno costrutti equivalenti, anche se la sintassi esatta può differire. Python non ha il ciclo **repeat**-**until**, e il suo ciclo **for** opera in modo un po' diverso dai cicli **for** di questo libro.

Un puntatore può non fare riferimento ad alcun oggetto; in questo caso daremo ad esso il valore speciale NIL.

- I parametri vengono passati a una procedura ***per valore***: la procedura chiamata riceve la sua copia dei parametri e, se viene assegnato un valore a un parametro, la modifica *non* viene vista dalla procedura chiamante. Quando viene passato un oggetto, viene copiato il puntatore ai dati che costituiscono l'oggetto, ma non vengono copiati gli attributi dell'oggetto. Per esempio, se x è un parametro di una procedura chiamata, l'assegnazione $x = y$ all'interno della procedura chiamata non è visibile alla procedura chiamante. L'assegnazione $x.f = 3$, invece, è visibile. Analogamente, gli array sono passati per puntatore; quindi a una procedura viene passato il puntatore a un array, anziché l'intero array, e le modifiche dei singoli elementi dell'array sono visibili alla procedura chiamante.
- Un'istruzione **return** restituisce immediatamente il controllo al punto in cui la procedura chiamante ha effettuato la chiamata. La maggior parte delle istruzioni **return** inoltre passano un valore alla procedura chiamante. La nostra pseudocodifica differisce da quella di molti linguaggi di programmazione perché consente di passare più valori con un'unica istruzione **return**.
- Gli operatori booleani "and" e "or" sono ***cortocircuitati***. Questo significa che, quando valutiamo l'espressione "x and y", prima dobbiamo valutare x. Se x è FALSE, allora l'intera espressione non può essere TRUE, quindi non occorre valutare y. Se, invece, x è TRUE, dobbiamo valutare y per determinare il valore dell'intera espressione. Analogamente, se abbiamo l'espressione "x or y", valutiamo y soltanto se x è FALSE. Gli operatori cortocircuitati ci consentono di scrivere espressioni booleane come "$x \neq$ NIL and $x.f = y$" senza preoccuparci di ciò che accade quando tentiamo di valutare $x.f$ quando x è NIL.
- La parola chiave **error** indica che si è verificato un errore perché le condizioni non erano corrette per la procedura chiamata. La procedura chiamante è responsabile della gestione degli errori, e quindi non specifichiamo quale azione intraprendere.

Esercizi

2.1-1

Utilizzando la Figura 2.2 come modello, illustrate l'operazione di INSERTION-SORT sull'array $A = \langle 31, 41, 59, 26, 41, 58 \rangle$.

2.1-2

Modificate la procedura INSERTION-SORT per disporre gli elementi in ordine non crescente, anziché non decrescente.

2.1-3

Considerate il seguente ***problema di ricerca***:

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Input: una sequenza di n numeri $A = \langle a_1, a_2, \ldots, a_n \rangle$ e un valore v.

Output: un indice i tale che $v = A[i]$ o il valore speciale NIL se v non figura in A.

Scrivete uno pseudocodice di ***ricerca lineare*** che esamina gli elementi della sequenza alla ricerca di v. Utilizzando un'invariante di ciclo, dimostrate che il vostro algoritmo è corretto. Verificate che la vostra invariante di ciclo soddisfa le tre proprietà richieste.

2.1-4

Considerate il problema di sommare due numeri interi binari di n-bit, memorizzati in due array A e B di n elementi. La somma dei due interi deve essere memorizzata in forma binaria nell'array C di $n + 1$ elementi. Definite formalmente il problema e scrivete lo pseudocodice per sommare i due interi.

2.2 Analisi degli algoritmi

Analizzare un algoritmo significa prevedere le risorse che l'algoritmo richiede. Raramente sono di primaria importanza risorse come la memoria, la larghezza di banda nelle comunicazioni o l'hardware nei computer, mentre più frequentemente è importante misurare il tempo di elaborazione. In generale, analizzando più algoritmi candidati a risolvere un problema, è facile identificare quello più efficiente. Tale analisi potrebbe indicare più di un candidato, ma di solito in questo processo vengono scartati diversi algoritmi inefficienti.

Prima di analizzare un algoritmo, dobbiamo avere un modello della tecnologia di implementazione che sarà utilizzata, incluso un modello per le risorse di tale tecnologia e dei loro costi. Nella maggior parte dei casi di questo libro, considereremo come tecnologia di implementazione un generico modello di calcolo a un processore, che chiameremo modello ***random-access machine*** (***RAM***); inoltre, i nostri algoritmi saranno implementati come programmi per computer. Nel modello RAM, le istruzioni sono eseguite una dopo l'altra, senza operazioni contemporanee.

A rigor di termini, dovremmo definire con precisione le istruzioni del modello RAM e i loro costi. Purtroppo, tutto questo risulterebbe noioso e non gioverebbe molto a illustrare il processo di analisi e progettazione degli algoritmi. Inoltre dobbiamo stare attenti a non abusare del modello RAM. Per esempio, che cosa accadrebbe se un modello RAM avesse un'istruzione di ordinamento? Potremmo ordinare gli elementi con una sola istruzione. Tale modello non sarebbe realistico, in quanto i computer reali non hanno simili istruzioni. La nostra guida, dunque, è come i computer reali sono progettati. Il modello RAM contiene istruzioni che si trovano comunemente nei computer reali: istruzioni aritmetiche (addizione, sottrazione, moltiplicazione, divisione, resto, parte intera inferiore o floor, parte intera superiore o ceiling), istruzioni per spostare i dati (load, store, copy) e istruzioni di controllo (salto condizionato e incondizionato, chiamata di subroutine e return). Ciascuna di queste istruzioni richiede una quantità costante di tempo.

I tipi di dati nel modello RAM sono integer (numeri interi) e floating point (numeri in virgola mobile). Sebbene di solito non ci cureremo della precisione in questo libro, tuttavia, in alcune applicazioni la precisione potrebbe essere un fattore cruciale. Inoltre, supporremo che ci sia un limite alla dimensione di ogni parola (word) di dati. Per esempio, quando operiamo con input di dimensione n, tipicamente supponiamo che i numeri interi siano rappresentati da $c \lg n$ bit per una costante $c \geq 1$. Noi richiediamo $c \geq 1$ in modo che una parola possa contenere il valore di n, consentendoci di indicizzare i singoli elementi di input, e imponiamo che c sia una costante in modo che la dimensione della parola non cresca in modo arbitrario (se questa dimensione potesse crescere arbitrariamente, potremmo memorizzare enormi quantità di dati in una parola e operare con essa sempre in un tempo costante – chiaramente uno scenario irreale).

Acquistato da Michele Michele su Webster il 2022-0[illegible] Copyright © [illegible] All rights reserved

I computer reali contengono istruzioni non elencate in precedenza; tali istruzioni rappresentano un'area grigia nel modello RAM. Per esempio, l'elevamento a potenza è un'istruzione a tempo costante? Nel caso generale, no; occorrono varie istruzioni per calcolare x^y quando x e y sono numeri reali. In casi particolari, invece, l'elevamento a potenza è un'operazione a tempo costante. Molti computer hanno un'istruzione "shift left" (scorrimento a sinistra), che fa scorrere in un tempo costante i bit di un numero intero di k posizioni a sinistra. In molti computer, lo scorrimento dei bit di un intero di una posizione a sinistra equivale a moltiplicare per 2. Lo scorrimento dei bit di k posizioni a sinistra equivale a moltiplicare per 2^k. Di conseguenza, tali computer possono calcolare 2^k con un'istruzione a tempo costante facendo scorrere l'intero 1 di k posizioni a sinistra, purché k non superi il numero di bit di una parola del computer. Cercheremo di evitare tali aree grige nel modello RAM, tuttavia considereremo il calcolo di 2^k come un'operazione a tempo costante quando k è un intero positivo sufficientemente piccolo.

Nel modello RAM non tenteremo di modellare la struttura gerarchica della memoria che è comune nei computer attuali, ovvero non modelleremo la memoria cache o la memoria virtuale. Vari modelli computazionali tentano di tenere conto degli effetti della gerarchia della memoria, che a volte sono significativi nei programmi o nelle macchine reali. Soltanto in pochi problemi di questo libro esamineremo gli effetti della gerarchia della memoria, ma nella maggior parte dei casi l'analisi ignorerà tali effetti. I modelli che includono la struttura gerarchica della memoria sono un po' più complessi del modello RAM, quindi è difficile operare con essi. Inoltre, l'analisi del modello RAM di solito è un eccellente strumento per prevedere le prestazioni delle macchine reali.

L'analisi di un algoritmo nel modello RAM può risultare complessa anche se l'algoritmo è semplice. Gli strumenti matematici richiesti possono includere la teoria delle probabilità, la combinatorica, destrezza algebrica e capacità di identificare i termini più significativi in una formula. Poiché il comportamento di un algoritmo può essere diverso per ogni possibile input, occorrono strumenti per sintetizzare tale comportamento in formule semplici e facili da capire.

Anche se usiamo un solo un modello di macchina per analizzare un determinato algoritmo, avremo diverse scelte possibili per decidere come esprimere la nostra analisi. Noi vorremmo un metodo che sia semplice da scrivere e manipolare, mostri le caratteristiche importanti delle risorse richieste da un algoritmo ed elimini i dettagli più noiosi.

Analisi di insertion sort

Il tempo richiesto dalla procedura INSERTION-SORT dipende dall'input: occorre più tempo per ordinare un migliaio di numeri che tre numeri. Inoltre, INSERTION-SORT può richiedere quantità di tempo differenti per ordinare due sequenze di input della stessa dimensione a seconda di come gli elementi siano già ordinati. In generale, il tempo richiesto da un algoritmo cresce con la dimensione dell'input, quindi è tradizione descrivere il tempo di esecuzione di un programma come una funzione della dimensione del suo input. Per farlo, dobbiamo definire più precisamente i termini "tempo di esecuzione" e "dimensione dell'input".

Acquistato da Michele Mu... per uso esclusivo... Copyright © 2023 McGraw-Hill Education (Italy)

La definizione migliore della ***dimensione dell'input*** dipende dal problema che si sta studiando. Per la maggior parte dei problemi, come l'ordinamento o il calcolo delle trasformate discrete di Fourier, la misura più naturale è il *numero di elementi dell'input* – per esempio, la dimensione n dell'array per l'ordinamento.

Per molti altri problemi, come la moltiplicazione di due interi, la misura migliore della dimensione dell'input è il *numero totale di bit* richiesti per rappresentare l'input nella normale notazione binaria. A volte, è più appropriato descrivere la dimensione dell'input con due numeri, anziché con uno solo. Per esempio, se l'input di un algoritmo è un grafo, la dimensione dell'input può essere descritta dal numero di vertici e dal numero di lati del grafo. Per ogni problema analizzato dovremo indicare quale misura della dimensione di input sarà adottata.

Il ***tempo di esecuzione*** di un algoritmo per un particolare input è il numero di operazioni primitive che vengono eseguite o "passi". Conviene definire il concetto di passo nel modo più indipendente possibile dal tipo di macchina. Per il momento, adotteremo il seguente punto di vista. Per eseguire una riga del nostro pseudocodice occorre una quantità costante di tempo. Una riga può richiedere una quantità di tempo diversa da un'altra riga, tuttavia supporremo che ogni esecuzione dell'i-esima riga richieda un tempo c_i, dove c_i è una costante. Questa ipotesi è conforme al modello RAM e rispecchia anche il modo in cui lo pseudocodice può essere implementato in molti computer reali.[5]

Nella discussione che segue, la nostra espressione del tempo di esecuzione per INSERTION-SORT si evolverà da una formula complicata che usa tutti i costi c_i delle istruzioni a una notazione molto più semplice, concisa e facilmente manipolabile. Questa notazione semplificata renderà anche più facile determinare se un algoritmo è più efficiente di un altro.

Presentiamo, innanzi tutto, la procedura INSERTION-SORT con il tempo impiegato da ogni istruzione (costo) e il numero di volte che vengono eseguite le singole istruzioni. Per ogni $j = 2, 3, \ldots, n$, dove $n = A.length$, indichiamo con t_j il numero di volte che il test del ciclo **while** nella riga 5 viene eseguito per quel valore di j. Quando un ciclo **for** o **while** termina nel modo consueto (come stabilito dal test all'inizio del ciclo), il test viene eseguito una volta di più del corpo del ciclo. Noi supponiamo che i commenti non siano istruzioni eseguibili e, quindi, il loro costo è nullo.

	INSERTION-SORT(A)	*costo*	*numero di volte*
1	**for** $j = 2$ **to** $A.length$	c_1	n
2	$key = A[j]$	c_2	$n-1$
3	// Inserisce $A[j]$ nella sequenza ordinata $A[1..j-1]$.	0	$n-1$
4	$i = j - 1$	c_4	$n-1$
5	**while** $i > 0$ and $A[i] > key$	c_5	$\sum_{j=2}^{n} t_j$
6	$A[i+1] = A[i]$	c_6	$\sum_{j=2}^{n} (t_j - 1)$
7	$i = i - 1$	c_7	$\sum_{j=2}^{n} (t_j - 1)$
8	$A[i+1] = key$	c_8	$n-1$

Acquistato da Michele Michele su Webster il 2022-05-07 23:12. Numero Ordine Libraria 19950316 220707 0 Copyright© 2022, McGraw-Hill Education (Italy)

[5] Ci sono alcuni particolari da chiarire. I passi computazionali che specifichiamo in italiano spesso sono varianti di una procedura che richiede più di una quantità di tempo costante. Per esempio, più avanti in questo libro potremmo dire di "ordinare i punti in funzione della coordinata x"; come vedremo, questa operazione richiede più di una quantità di tempo costante. Notiamo inoltre che un'istruzione che chiama una subroutine impiega un tempo costante, sebbene la subroutine, una volta chiamata, possa impiegare di più. In altre parole, separiamo il processo della ***chiamata*** della subroutine – passare i parametri alla subroutine, ecc. – dal processo di ***esecuzione*** della subroutine.

Il tempo di esecuzione dell'algoritmo è la somma dei tempi di esecuzione per ogni istruzione eseguita; un'istruzione che richiede c_i passi e viene eseguita n volte contribuirà con $c_i n$ al tempo di esecuzione totale.[6] Per calcolare $T(n)$, il tempo di esecuzione di INSERTION-SORT con un input do n valori, sommiamo i prodotti delle colonne *costo* e *numero di volte*, ottenendo

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^{n} t_j + c_6 \sum_{j=2}^{n} (t_j - 1) \\ &\quad + c_7 \sum_{j=2}^{n} (t_j - 1) + c_8(n-1) \end{aligned}$$

Anche per più input della stessa dimensione, il tempo di esecuzione di un algoritmo può dipendere da *quale* input di quella dimensione viene scelto. Per esempio, in INSERTION-SORT il caso migliore si verifica se l'array è già ordinato. Per ogni $j = 2, 3, \ldots, n$, troviamo che $A[i] \leq key$ nella riga 5, quando i ha il suo valore iniziale $j - 1$. Quindi $t_j = 1$ per $j = 2, 3, \ldots, n$ e il tempo di esecuzione nel caso migliore è

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\ &= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) \end{aligned}$$

Questo tempo di esecuzione può essere espresso come $an + b$, con le *costanti* a e b che dipendono dai costi delle istruzioni c_i; quindi è una ***funzione lineare*** di n.

Se l'array è ordinato in senso inverso – cioè in ordine decrescente – allora si verifica il caso peggiore. Dobbiamo confrontare ogni elemento $A[j]$ con ogni elemento dell'intero sottoarray ordinato $A[1 \ldots j-1]$, e quindi $t_j = j$ per $j = 2, 3, \ldots, n$. Poiché

$$\sum_{j=2}^{n} j = \frac{n(n+1)}{2} - 1$$

e

$$\sum_{j=2}^{n} (j-1) = \frac{n(n-1)}{2}$$

(consultate l'Appendice A per sapere come risolvere queste sommatorie), il tempo di esecuzione di INSERTION-SORT nel caso peggiore è

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) \\ &\quad + c_6\left(\frac{n(n-1)}{2}\right) + c_7\left(\frac{n(n-1)}{2}\right) + c_8(n-1) \\ &= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right)n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8\right)n \\ &\quad - (c_2 + c_4 + c_5 + c_8) \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libraria: 1945030115-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[6] Questa caratteristica non è necessariamente valida per una risorsa come la memoria. Un'istruzione che fa riferimento a m parole di memoria e viene eseguita n volte non necessariamente consuma mn parole di memoria in totale.

Questo tempo di esecuzione può essere espresso come $an^2 + bn + c$, con le costanti a, b e c che, anche in questo caso, dipendono dai costi delle istruzioni c_i; quindi è una ***funzione quadratica*** di n. Generalmente, come accade con insertion sort, il tempo di esecuzione di un algoritmo è fisso per un dato input, sebbene nei successivi capitoli vedremo alcuni interessanti algoritmi "randomizzati" il cui comportamento può variare anche con uno stesso input.

Analisi del caso peggiore e del caso medio

Nell'analisi di insertion sort, abbiamo esaminato sia il caso migliore, in cui l'array di input era già ordinato, sia il caso peggiore, in cui l'array di input era ordinato alla rovescia. Nel seguito del libro, di solito, sono descritte le tecniche per determinare soltanto il ***tempo di esecuzione nel caso peggiore***, ovvero il tempo di esecuzione più lungo per *qualsiasi* input di dimensione n. Ci sono tre ragioni alla base di questo orientamento.

- Il tempo di esecuzione nel caso peggiore di un algoritmo è un limite superiore al tempo di esecuzione per qualsiasi input. Conoscendo questo tempo, abbiamo la garanzia che l'algoritmo non potrà impiegare di più. Non abbiamo bisogno di fare altre ipotesi sul tempo di esecuzione e sperare che questo tempo non venga mai superato.
- Per alcuni algoritmi, il caso peggiore si verifica molto spesso. Per esempio, nella ricerca di una particolare informazione in un database, il caso peggiore dell'algoritmo di ricerca si verifica ogni volta che l'informazione non è presente nel database. In alcune applicazioni di ricerca potrebbe essere frequente ricercare informazioni assenti.
- Il "caso medio" spesso è cattivo quasi quanto quello peggiore. Supponete di avere scelto a caso n numeri e di applicare l'algoritmo insertion sort. Quanto tempo impiegherà l'algoritmo per determinare dove inserire l'elemento $A[j]$ nel sottoarray $A[1 \mathinner{..} j-1]$? In media, metà degli elementi di $A[1 \mathinner{..} j-1]$ sono più piccoli di $A[j]$, mentre gli altri elementi sono più grandi. In media, quindi, verifichiamo metà del sottoarray $A[1 \mathinner{..} j-1]$, pertanto t_j vale circa $j/2$. Il tempo di esecuzione nel caso medio risulta dunque una funzione quadratica della dimensione dell'input, proprio come il tempo di esecuzione nel caso peggiore.

In alcuni casi particolari saremo interessati a determinare il tempo di esecuzione nel ***caso medio*** di un algoritmo; applicheremo la tecnica dell'***analisi probabilistica*** a vari algoritmi. L'applicabilità dell'analisi del caso medio è tuttavia limitata dal fatto che non è sempre evidente ciò che costituisce un input "medio" per un particolare problema. Spesso supporremo che tutti gli input di una data dimensione abbiano la stessa probabilità. Nella pratica questa ipotesi potrebbe essere invalidata, tuttavia in alcuni casi possiamo utilizzare un ***algoritmo randomizzato***, che effettua delle scelte casuali, per consentirci di svolgere l'analisi probabilistica e ottenere un tempo di esecuzione ***atteso***. Analizzeremo più approfondi gli algoritmi randomizzati nel Capitolo 5 e in altri capitoli successivi.

Acquistato da Michele Michele su Webster il 2022-07-12 12:12:1 tramite Online Libraria. 9788891621775 © Copyright © 2022 McGraw-Hill Education (Italy)

Tasso di crescita

In precedenza abbiamo fatto alcune ipotesi per semplificare l'analisi della procedura INSERTION-SORT. Innanzi tutto, abbiamo ignorato il costo effettivo di ogni istruzione, utilizzando le costanti c_i per rappresentare questi costi. Poi, abbiamo osservato che anche queste costanti ci danno più dettagli del necessario: abbiamo espresso il tempo di esecuzione nel caso peggiore come $an^2 + bn + c$, con

le costanti a, b e c che dipendono dai costi delle istruzioni c_i. Quindi, abbiamo ignorato non soltanto i costi effettivi delle istruzioni, ma anche i costi astratti c_i. Adesso faremo un'altra astrazione semplificativa. È la velocità con cui cresce il tempo di esecuzione, ossia il suo ***tasso di crescita***, che effettivamente ci interessa. Di conseguenza, consideriamo soltanto il termine principale di una formula (per esempio an^2), in quanto i termini di ordine inferiore sono relativamente insignificanti per grandi valori di n. Ignoriamo anche il coefficiente costante del termine principale, in quanto i fattori costanti sono meno significativi del tasso di crescita nel determinare l'efficienza computazionale per grandi input. Per insertion sort, dopo aver ignorato i termini di ordine inferiore e il coefficiente del termine principale, rimaniamo con il solo fattore n^2 del termine principale. Scriveremo quindi che insertion sort ha un tempo di esecuzione nel caso peggiore pari a $\Theta(n^2)$ (che si pronuncia "teta di n al quadrato"). In questo capitolo useremo informalmente la notazione Θ, che sarà definita più precisamente nel Capitolo 3. Di solito, un algoritmo è considerato più efficiente di un altro se il suo tempo di esecuzione nel caso peggiore ha un tasso di crescita inferiore. A causa dei fattori costanti e dei termini di ordine inferiore, un algoritmo il cui tempo di esecuzione abbia un tasso di crescita maggiore può richiedere, per input piccoli, meno tempo di un algoritmo il cui tempo di esecuzione abbia un tasso di crescita minore. Tuttavia, per input sufficientemente grandi, un algoritmo $\Theta(n^2)$, per esempio, sarà eseguito più velocemente nel caso peggiore di un algoritmo $\Theta(n^3)$.

Esercizi

2.2-1

Esprimete la funzione $n^3/1000 - 100n^2 - 100n + 3$ nella notazione Θ.

2.2-2

Supponete di ordinare n numeri memorizzati nell'array A trovando prima il più piccolo elemento di A e scambiandolo con l'elemento in $A[1]$; poi, trovate il secondo elemento più piccolo di A e scambiatelo con $A[2]$. Continuate in questo modo per i primi $n-1$ elementi di A. Scrivete lo pseudocodice per questo algoritmo, che è noto come ***selection sort*** (ordinamento per selezione). Quale invariante di ciclo conserva questo algoritmo? Perché basta eseguirlo soltanto per i primi $n-1$ elementi, anziché per tutti gli n elementi? Esprimete nella notazione Θ i tempi di esecuzione nei casi migliore e peggiore dell'algoritmo selection sort.

2.2-3

Considerate di nuovo la ricerca lineare (Esercizio 2.1-3). Quanti elementi della sequenza di input devono essere esaminati in media, supponendo che l'elemento cercato abbia la stessa probabilità di essere un elemento qualsiasi dell'array? Quanti elementi nel caso peggiore? Quali sono i tempi di esecuzione nei casi migliore e peggiore della ricerca lineare nella notazione Θ. Spiegate le vostre risposte.

2.2-4

Come possiamo modificare quasi tutti gli algoritmi in modo da avere un buon tempo di esecuzione nel caso migliore?

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

2.3 Progettare gli algoritmi

Ci sono varie tecniche per progettare gli algoritmi. Per insertion sort abbiamo usato un approccio ***incrementale***: dopo avere ordinato il sottoarray $A[1 \,..\, j-1]$, abbiamo inserito il singolo elemento $A[j]$ nella posizione appropriata, ottenendo

il sottoarray ordinato $A[1\,..\,j]$. Nel prossimo paragrafo esamineremo un metodo di progettazione alternativo, noto come "divide et impera", che sarà descritto più dettagliatamente nel Capitolo 4. Utilizzeremo questo metodo per progettare un algoritmo di ordinamento il cui tempo di esecuzione nel caso peggiore è molto più piccolo di quello di insertion sort. Un vantaggio degli algoritmi divide et impera è che i loro tempi di esecuzione, spesso, possono essere facilmente determinati applicando le tecniche che saranno presentate nel Capitolo 4.

2.3.1 Il metodo divide et impera

Molti utili algoritmi sono ***ricorsivi***: per risolvere un determinato problema, questi algoritmi chiamano sé stessi in modo ricorsivo, una o più volte, per trattare sottoproblemi dello stesso tipo. Generalmente, gli algoritmi ricorsivi adottano un approccio ***divide et impera***: suddividono il problema in vari sottoproblemi, che sono simili al problema originale, ma di dimensioni più piccole, risolvono i sottoproblemi in modo ricorsivo e, poi, combinano le soluzioni per costruire una soluzione del problema originale.

Il paradigma divide et impera prevede tre passi a ogni livello di ricorsione:

Divide: il problema viene diviso in un certo numero di sottoproblemi, che sono istanze più piccole dello stesso problema.

Impera: i sottoproblemi vengono risolti in modo ricorsivo. Comunque, quando i sottoproblemi hanno una dimensione sufficientemente piccola, essi vengono risolti direttamente.

Combina: le soluzioni dei sottoproblemi vengono combinate per generare la soluzione del problema originale.

L'algoritmo ***merge sort*** è conforme al paradigma divide et impera; intuitivamente, opera nel modo seguente.

Divide: divide la sequenza degli n elementi da ordinare in due sottosequenze di $n/2$ elementi ciascuna.

Impera: ordina le due sottosequenze in modo ricorsivo utilizzando l'algoritmo merge sort.

Combina: fonde le due sottosequenze ordinate per generare la sequenza ordinata.

La ricorsione "tocca il fondo" quando la sequenza da ordinare ha lunghezza 1, nel qual caso non c'è più nulla da fare, in quanto ogni sequenza di lunghezza 1 è già ordinata.

L'operazione chiave dell'algoritmo merge sort è la fusione di due sottosequenze ordinate nel passo "combina". Per effettuare la fusione, utilizziamo una procedura ausiliaria MERGE(A, p, q, r), dove A è un array e p, q e r sono indici dell'array tali che $p \le q < r$. La procedura assume che i sottoarray $A[p\,..\,q]$ e $A[q+1\,..\,r]$ siano ordinati. Li ***fonde*** per formare un unico sottoarray ordinato che sostituisce il sottoarray corrente $A[p\,..\,r]$.

Acquistato da Michele Michele su Webster il 2022-[illegible]

La procedura MERGE impiega un tempo $\Theta(n)$, dove $n = r - p + 1$ è il numero totale di elementi da fondere, e opera nel modo seguente. Riprendendo l'esempio delle carte da gioco, supponiamo di avere sul tavolo due mazzi di carte scoperte. Ogni mazzo è ordinato, con le carte più piccole in alto. Vogliamo "fondere" i due mazzi in un unico mazzo ordinato di output, con le carte coperte. Il passo

base consiste nello scegliere la più piccola fra le carte scoperte in cima ai due mazzi, togliere questa carta dal suo mazzo (scoprendo così una nuova carta in cima al mazzo) e deporla coperta sul mazzo di output. Ripetiamo questo passo finché un mazzo di input sarà vuoto; a questo punto, prendiamo le carte rimanenti nell'altro mazzo di input e le poniamo coperte sopra il mazzo di output. Da un punto di vista computazionale, ogni passo base impiega un tempo costante, in quanto dobbiamo solo confrontare le due carte in cima ai mazzi di input. Poiché svolgiamo al massimo n passi base, la fusione dei mazzi impiega un tempo $\Theta(n)$.

Il seguente pseudocodice implementa la precedente idea, con un espediente aggiuntivo che evita di dover controllare se i mazzi sono vuoti in ogni passo base.

```
MERGE(A, p, q, r)
 1  n_1 = q − p + 1
 2  n_2 = r − q
 3  crea due nuovi array L[1 .. n_1 + 1] e R[1 .. n_2 + 1]
 4  for i = 1 to n_1
 5       L[i] = A[p + i − 1]
 6  for j = 1 to n_2
 7       R[j] = A[q + j]
 8  L[n_1 + 1] = ∞
 9  R[n_2 + 1] = ∞
10  i = 1
11  j = 1
12  for k = p to r
13       if L[i] ≤ R[j]
14            A[k] = L[i]
15            i = i + 1
16       else A[k] = R[j]
17            j = j + 1
```

Mettiamo in fondo a ogni mazzo una carta ***sentinella***, che contiene un valore speciale che usiamo per semplificare il nostro codice. In questo esempio usiamo ∞ come valore della sentinella, in modo che quando si presenta una carta con il valore ∞, essa non può essere la carta più piccola, a meno che entrambi i mazzi non abbiano esposto le loro sentinelle. Ma quando accade questo, tutte le carte non-sentinella sono state già poste nel mazzo di output. Poiché sappiamo in anticipo che saranno poste esattamente $r - p + 1$ carte nel mazzo di output, possiamo fermare il processo non appena abbiamo eseguiti questo numero di passi.

In dettaglio, la procedura MERGE opera nel modo seguente. La riga 1 calcola la lunghezza n_1 del sottoarray $A[p..q]$; la riga 2 calcola la lunghezza n_2 del sottoarray $A[q + 1..r]$. Nella riga 3 creiamo gli array L e R (L sta per "left" o sinistro e R sta per "right" o destro), rispettivamente, di lunghezza $n_1 + 1$ e $n_2 + 1$. Il ciclo **for**, righe 4–5, copia il sottoarray $A[p\,..\,q]$ in $L[1\,..\,n_1]$; il ciclo **for**, righe 6–7, copia il sottoarray $A[q + 1\,..\,r]$ in $R[1\,..\,n_2]$. Le righe 8–9 pongono le sentinelle alla fine degli array L e R. Le righe 10–17, illustrate nella Figura 2.3, eseguono $r - p + 1$ passi base mantenendo la seguente invariante di ciclo:

> All'inizio di ogni iterazione del ciclo **for**, righe 12–17, il sottoarray $A[p\,..\,k-1]$ contiene, ordinati, i $k-p$ elementi più piccoli di $L[1\,..\,n_1+1]$ e $R[1\,..\,n_2 + 1]$. Inoltre, $L[i]$ e $R[j]$ sono i più piccoli elementi dei loro array che non sono stati copiati in A.

Acquistato da Michele Michelassi Webster il 2022-07-27 23:12. Numero Ordine Libreria: 1095631162. ©2007 © Copyright © 2013 McGraw-Hill Education (Italy)

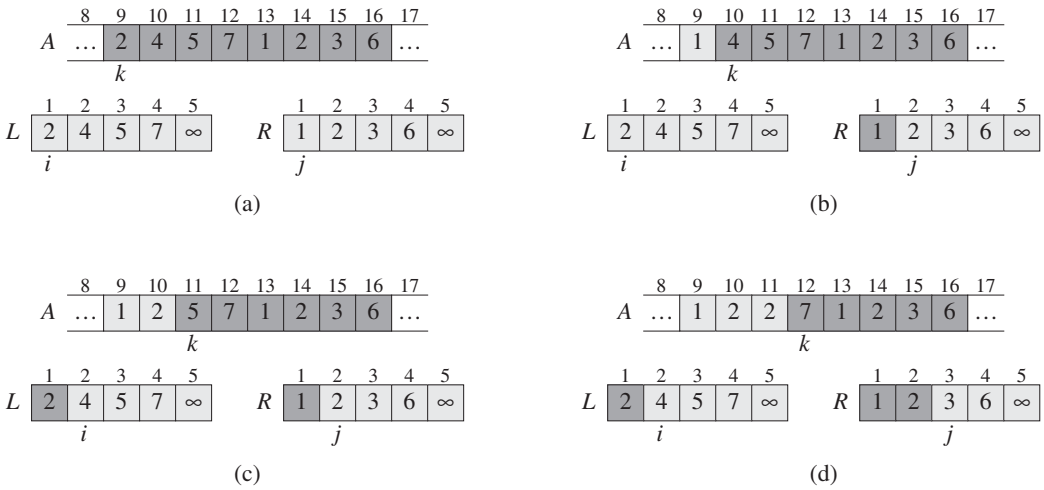


Figura 2.3 Il funzionamento delle righe 10–17 nella chiamata MERGE$(A, 9, 12, 16)$, quando il sottoarray $A[9\,..\,16]$ contiene la sequenza $\langle 2, 4, 5, 7, 1, 2, 3, 6\rangle$. Dopo il ricopiamento e l'aggiunta delle sentinelle, l'array L contiene $\langle 2, 4, 5, 7, \infty\rangle$ e l'array R contiene $\langle 1, 2, 3, 6, \infty\rangle$. Le posizioni di colore grigio chiaro di A contengono i loro valori finali; le posizioni di colore grigio chiaro di L e R contengono i valori che devono essere copiati in A. Tutte insieme, le posizioni di colore grigio chiaro contengono sempre i valori che originariamente erano in $A[9\,..\,16]$ e le due sentinelle. Le posizioni di colore grigio scuro di A contengono valori che verranno successivamente sovrascritti; le posizioni di colore grigio scuro di L e R contengono i valori che sono stati già copiati in A. **(a)–(h)** Gli array A, L e R e i loro rispettivi indici k, i e j prima di ogni iterazione del ciclo 12–17. **(i)** Gli array e gli indici alla fine del ciclo. A questo punto, il sottoarray in $A[9\,..\,16]$ è ordinato e le due sentinelle in L e R sono gli unici due elementi in questi array che non sono stati copiati in A.

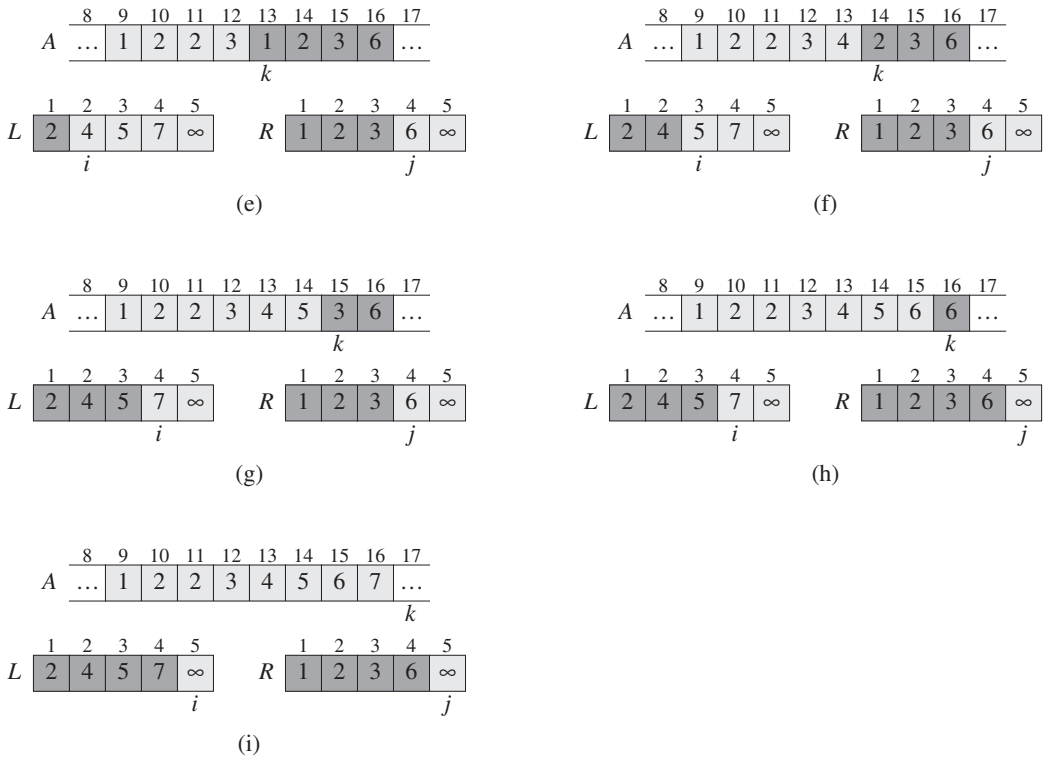
Dobbiamo dimostrare che questa invariante di ciclo è valida prima della prima iterazione del ciclo **for**, righe 12–17, che ogni iterazione del ciclo conserva l'invariante e che l'invariante fornisce un'utile proprietà per dimostrare la correttezza quando il ciclo termina.

Inizializzazione: prima della prima iterazione del ciclo, abbiamo $k = p$, quindi il sottoarray $A[p\,..\,k-1]$ è vuoto. Questo sottoarray vuoto contiene i $k-p = 0$ elementi più piccoli di L e R; poiché $i = j = 1$, $L[i]$ e $R[j]$ sono i più piccoli elementi, nei rispettivi array, tra quelli che non sono ancora stati copiati in A.

Conservazione: per verificare che ogni iterazione conserva l'invariante di ciclo, supponiamo dapprima che $L[i] \le R[j]$; quindi $L[i]$ è l'elemento più piccolo che non è stato ancora copiato in A. Poiché $A[p\,..\,k-1]$ contiene i $k-p$ elementi più piccoli, dopo che la riga 14 ha copiato $L[i]$ in $A[k]$, il sottoarray $A[p\,..\,k]$ conterrà i $k-p+1$ elementi più piccoli. Incrementando k (aggiornamento del ciclo **for**) e i (riga 15), si ristabilisce l'invariante di ciclo per la successiva iterazione. Se, invece, $L[i] > R[j]$, allora le righe 16–17 svolgono l'azione appropriata per conservare l'invariante di ciclo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 . Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Conclusione: alla fine del ciclo, $k = r+1$. Per l'invariante di ciclo, il sottoarray $A[p\,..\,k-1]$, che è $A[p\,..\,r]$, contiene $k-p = r-p+1$ elementi ordinati che sono i più piccoli di $L[1\,..\,n_1+1]$ e $R[1\,..\,n_2+1]$. Assieme, gli array L e R contengono $n_1+n_2+2 = r-p+3$ elementi. Tutti gli elementi, tranne i due più grandi, sono stati copiati in A; questi due elementi sono le sentinelle.



Per verificare che la procedura MERGE viene eseguita nel tempo $\Theta(n)$, con $n = r - p + 1$, notate che ciascuna delle righe 1–3 e 8–11 impiega un tempo costante, i cicli **for** (righe 4–7) impiegano un tempo $\Theta(n_1 + n_2) = \Theta(n)$,[7] e ci sono n iterazioni del ciclo **for** (righe 12–17), ciascuna delle quali impiega un tempo costante. Adesso possiamo utilizzare la procedura MERGE come subroutine nell'algoritmo merge sort. La procedura MERGE-SORT(A, p, r) ordina gli elementi nel sottoarray $A[p\,..\,r]$. Se $p \geq r$, il sottoarray ha al massimo un elemento e, quindi, è già ordinato; altrimenti, il passo "divide" calcola semplicemente un indice q che separa $A[p\,..\,r]$ in due sottoarray: $A[p\,..\,q]$, che contiene $\lceil n/2 \rceil$ elementi, e $A[q + 1\,..\,r]$, che contiene $\lfloor n/2 \rfloor$ elementi.[8]

```
MERGE-SORT(A, p, r)
1  if p < r
2      q = ⌊(p + r)/2⌋
3      MERGE-SORT(A, p, q)
4      MERGE-SORT(A, q + 1, r)
5      MERGE(A, p, q, r)
```

Per ordinare l'intera sequenza $A = \langle A[1], A[2], \ldots, A[n] \rangle$, effettuiamo la chiamata iniziale MERGE-SORT$(A, 1, A.length)$, dove ancora una volta $A.length = n$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[7] Il Capitolo 3 spiega come interpretare formalmente le equazioni che contengono la notazione Θ.

[8] L'espressione $\lceil x \rceil$ indica il più piccolo numero intero che è maggiore o uguale a x; $\lfloor x \rfloor$ indica il più grande numero intero che è minore o uguale a x. Queste notazioni sono definite nel Capitolo 3. Il sistema più semplice per verificare che impostando q a $\lfloor (p + r)/2 \rfloor$ si ottengono i sottoarray $A[p\,..\,q]$ e $A[q + 1\,..\,r]$, rispettivamente, di dimensione $\lceil n/2 \rceil$ e $\lfloor n/2 \rfloor$, consiste nell'esaminare i quattro casi che si presentano a seconda se p e r siano pari o dispari.

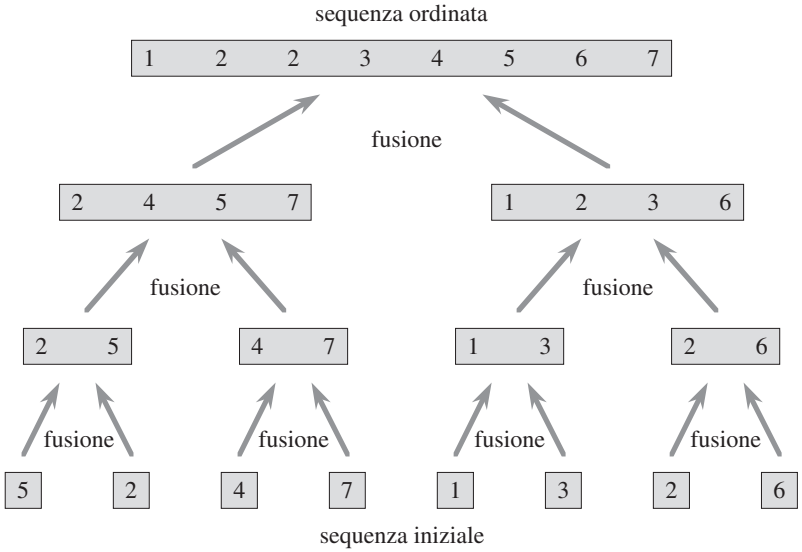


Figura 2.4
Funzionamento di merge sort con l'array $A = \langle 5, 2, 4, 7, 1, 3, 2, 6 \rangle$. Le lunghezze delle sequenze ordinate da fondere aumentano via via che l'algoritmo procede dal basso verso l'alto.

La Figura 2.4 illustra il funzionamento della procedura dal basso verso l'alto, quando n è una potenza di 2. L'algoritmo consiste nel fondere coppie di sequenze di un elemento per formare sequenze ordinate di lunghezza 2, fondere coppie di sequenze di lunghezza 2 per formare sequenze ordinate di lunghezza 4 e così via, finché non si fonderanno due sequenze di lunghezza $n/2$ per formare l'ultima sequenza ordinata di lunghezza n.

2.3.2 Analisi degli algoritmi divide et impera

Quando un algoritmo contiene una chiamata ricorsiva a sé stesso, il suo tempo di esecuzione spesso può essere descritto con una ***equazione di ricorrenza*** o ***ricorrenza***, che esprime il tempo di esecuzione totale di un problema di dimensione n in funzione del tempo di esecuzione per input più piccoli. Poi è possibile utilizzare gli strumenti matematici per risolvere l'equazione di ricorrenza e stabilire i limiti delle prestazioni dell'algoritmo.

Una ricorrenza per il tempo di esecuzione di un algoritmo divide et impera si basa sui tre passi del paradigma di base. Come in precedenza, supponiamo che $T(n)$ sia il tempo di esecuzione di un problema di dimensione n. Se la dimensione del problema è sufficientemente piccola, per esempio $n \leq c$ per qualche costante c, la soluzione diretta richiede un tempo costante, che indichiamo con $\Theta(1)$. Supponiamo che la nostra suddivisione del problema generi a sottoproblemi e che la dimensione di ciascun sottoproblema sia $1/b$ volte la dimensione del problema originale (per merge sort, i valori di a e b sono entrambi pari a 2, ma vedremo vari algoritmi divide et impera in cui $a \neq b$). Serve tempo $T(n/b)$ per risolvere un sottoproblema di dimensione n/b e quindi, per risolverne a, serve tempo $aT(n/b)$. Se impieghiamo un tempo $D(n)$ per dividere il problema in sottoproblemi e un tempo $C(n)$ per combinare le soluzioni dei sottoproblemi nella soluzione del problema originale, otteniamo la ricorrenza

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-2207070 Copyright © 2022, McGraw-Hill Education (Italy)

$$T(n) = \begin{cases} \Theta(1) & \text{se } n \leq c \\ aT(n/b) + D(n) + C(n) & \text{negli altri casi} \end{cases}$$

Nel Capitolo 4 vedremo come risolvere alcune comuni ricorrenze di questa forma.

Analisi di merge sort

Sebbene lo pseudocodice di MERGE-SORT funzioni correttamente quando il numero di elementi non è pari, la nostra analisi basata sulla ricorrenza si semplifica se supponiamo che la dimensione del problema originale sia una potenza di 2. Ogni passo *divide* genera due sottosequenze di dimensione esattamente pari a $n/2$. Nel Capitolo 4, vedremo che questa ipotesi non influisce sul tasso di crescita della soluzione della ricorrenza.

Per trovare la ricorrenza per $T(n)$, il tempo di esecuzione nel caso peggiore di merge sort con n numeri, possiamo fare il seguente ragionamento. L'algoritmo merge sort applicato a un solo elemento impiega un tempo costante. Se abbiamo $n > 1$ elementi, suddividiamo il tempo di esecuzione nel modo seguente.

Divide: questo passo calcola semplicemente il centro del sottoarray. Ciò richiede un tempo costante, quindi $D(n) = \Theta(1)$.

Impera: risolviamo in modo ricorsivo i due sottoproblemi, ciascuno di dimensione $n/2$; ciò contribuisce con $2T(n/2)$ al tempo di esecuzione.

Combina: abbiamo già notato che la procedura MERGE con un sottoarray di n elementi richiede un tempo $\Theta(n)$, quindi $C(n) = \Theta(n)$.

Quando sommiamo le funzioni $D(n)$ e $C(n)$ per l'analisi di merge sort, stiamo sommando una funzione che è $\Theta(1)$ e una funzione che è $\Theta(n)$. Questa somma è una funzione lineare di n, cioè $\Theta(n)$. Sommandola al termine $2T(n/2)$ del passo "impera", si ottiene la ricorrenza per il tempo di esecuzione $T(n)$ nel caso peggiore di merge sort:

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ 2T(n/2) + \Theta(n) & \text{se } n > 1 \end{cases} \tag{2.1}$$

Nel Capitolo 4 vedremo il "teorema dell'esperto", che possiamo utilizzare per dimostrare che $T(n)$ è $\Theta(n \lg n)$, dove $\lg n$ sta per $\log_2 n$. Poiché la funzione logaritmica cresce più lentamente di qualsiasi funzione lineare, per input sufficientemente grandi, l'algoritmo merge sort, con il suo tempo di esecuzione $\Theta(n \lg n)$, supera le prestazioni di insertion sort, il cui tempo di esecuzione è $\Theta(n^2)$, nel caso peggiore. Non occorre il teorema dell'esperto per capire perché la soluzione della ricorrenza (2.1) è $T(n) = \Theta(n \lg n)$. Riscriviamo la ricorrenza (2.1) così

$$T(n) = \begin{cases} c & \text{se } n = 1 \\ 2T(n/2) + cn & \text{se } n > 1 \end{cases} \tag{2.2}$$

La costante c rappresenta sia il tempo richiesto per risolvere i problemi di dimensione 1 sia il tempo per elemento dell'array dei passi *divide* e *combina*.[9]

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[9] È improbabile che la stessa costante rappresenti esattamente sia il tempo richiesto per risolvere i problemi di dimensione 1 sia il tempo per elemento dell'array dei passi *divide* e *combina*. Possiamo aggirare questo problema, assegnando a c il valore più grande di questi tempi, nel qual caso la nostra ricorrenza impone un limite superiore al tempo di esecuzione oppure assegnando a c il valore più piccolo di questi tempi, nel qual caso la nostra ricorrenza impone un limite inferiore al tempo di esecuzione. Entrambi i limiti saranno nell'ordine di $n \lg n$ e, presi insieme, danno un tempo di esecuzione $\Theta(n \lg n)$.

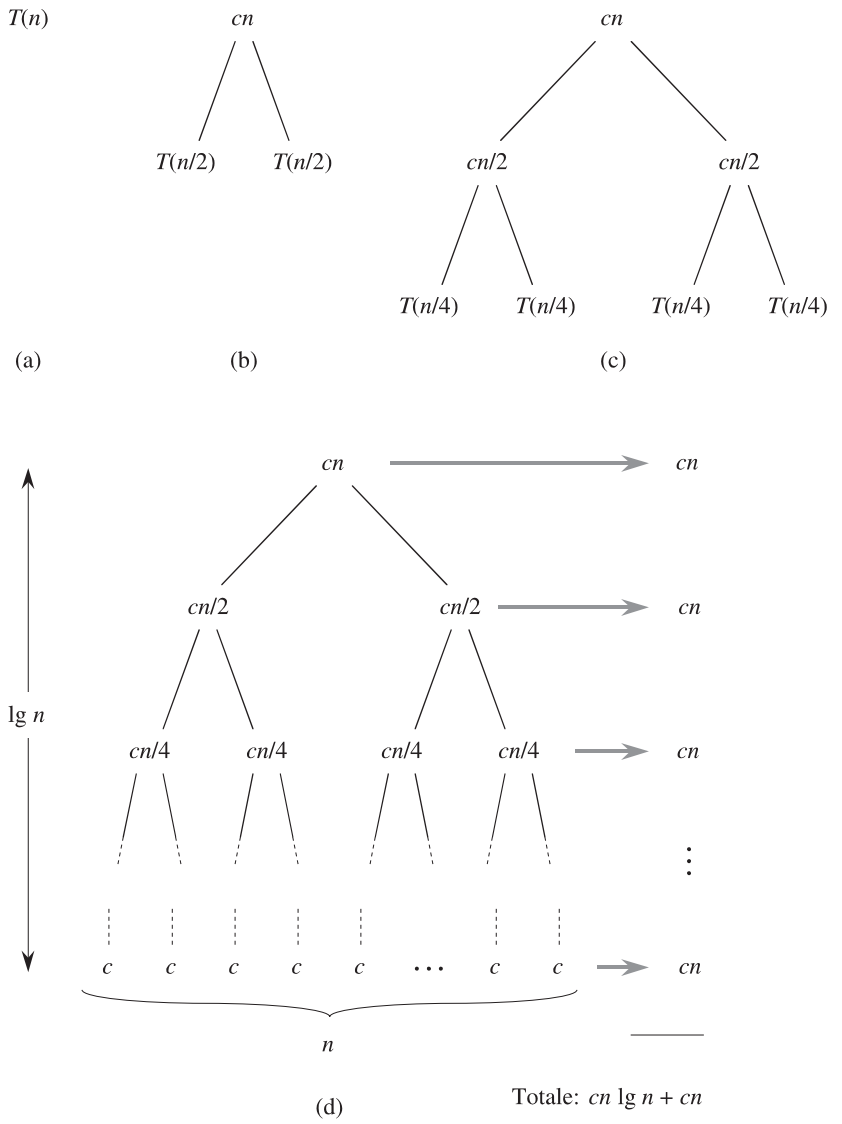


Figura 2.5
La costruzione di un albero di ricorsione per la ricorrenza $T(n) = 2T(n/2) + cn$. La parte **(a)** mostra $T(n)$, che viene progressivamente espanso in **(b)–(d)** per formare l'albero di ricorsione. L'albero completamente espanso nella parte (d) ha $\lg n + 1$ livelli (cioè ha un'altezza $\lg n$, come indicato) e ogni livello ha un costo cn. Di conseguenza, il costo totale è $cn \lg n + cn$, che è $\Theta(n \lg n)$.

La Figura 2.5 mostra come possiamo risolvere la ricorrenza (2.2). Per comodità, supponiamo che n sia una potenza esatta di 2. La parte (a) della figura mostra $T(n)$, che nella parte (b) è stato espanso in un albero equivalente che rappresenta la ricorrenza. Il termine cn è la radice (il costo sostenuto al primo livello di ricorsione) e i due sottoalberi della radice sono le due ricorrenze più piccole $T(n/2)$. La parte (c) mostra questo processo un passo più avanti con l'espansione di $T(n/2)$. Il costo sostenuto per ciascuno dei due sottonodi al secondo livello di ricorsione è $cn/2$. Continuiamo a espandere i nodi nell'albero suddividendolo nelle sue componenti come stabilisce la ricorrenza, finché le dimensioni dei problemi si riducono a 1, ciascuno con un costo c. La parte (d) mostra l'***albero di ricorsione*** risultante.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995003010-2287070 Copyright © 2022, McGraw-Hill Education (Italy)

Sommiamo i costi per ogni livello dell'albero. Il primo livello in alto ha un costo totale cn, il secondo livello ha un costo totale $c(n/2) + c(n/2) = cn$, il terzo livello ha un costo totale $c(n/4) + c(n/4) + c(n/4) + c(n/4) = cn$ e così via. In generale, il livello i ha 2^i nodi, ciascuno dei quali ha un costo $c(n/2^i)$, quindi l'i-esimo livello ha un costo totale $2^i\, c(n/2^i) = cn$. All'ultimo livello in basso ci sono n nodi, ciascuno con un costo c, per un costo totale cn.

Il numero totale di livelli dell'*albero di ricorsione* nella Figura 2.5 è $\lg n + 1$, dove n è il numero di foglie che è anche uguale alla dimensione dell'input. Questo può essere facilmente dimostrato con un ragionamento induttivo informale. Il caso base si verifica quando $n = 1$, nel qual caso c'è un solo livello. Poiché $\lg 1 = 0$, abbiamo che $\lg n + 1$ fornisce il numero corretto di livelli. Adesso supponiamo, come ipotesi induttiva, che il numero di livelli di un albero di ricorsione per 2^i nodi sia $\lg 2^i + 1 = i + 1$ (poiché per qualsiasi valore di i, si ha $\lg 2^i = i$). Poiché stiamo supponendo che la dimensione dell'input originale sia una potenza di 2, la successiva dimensione da considerare è 2^{i+1}. Un albero con 2^{i+1} nodi ha un livello in più di un albero con 2^i nodi; quindi il numero totale di livelli è $(i + 1) + 1 = \lg 2^{i+1} + 1$.

Per calcolare il costo totale rappresentato dalla ricorrenza (2.2), basta sommare i costi di tutti i livelli. Ci sono $\lg n + 1$ livelli, ciascuno di costo cn, per un costo totale di $cn(\lg n + 1) = cn \lg n + cn$. Ignorando il termine di ordine inferiore e la costante c, si ottiene il risultato desiderato $\Theta(n \lg n)$.

Esercizi

2.3-1

Utilizzando la Figura 2.4 come modello, illustrate l'operazione di merge sort sull'array $A = \langle 3, 41, 52, 26, 38, 57, 9, 49 \rangle$.

2.3-2

Modificate la procedura MERGE in modo da non utilizzare le sentinelle; interrompete il processo quando tutti gli elementi di uno dei due array L e R sono stati copiati in A; poi copiate il resto dell'altro array in A.

2.3-3

Applicate l'induzione matematica per dimostrare che, se n è una potenza esatta di 2, allora $T(n) = n \lg n$ è la soluzione della ricorrenza

$$T(n) = \begin{cases} 2 & \text{se } n = 2 \\ 2T(n/2) + n & \text{se } n = 2^k, \text{ per } k > 1 \end{cases}$$

2.3-4

Insertion sort può essere espresso come una procedura ricorsiva nel modo seguente: per ordinare $A[1 \,..\, n]$, si ordina in modo ricorsivo $A[1 \,..\, n-1]$ e poi si inserisce $A[n]$ nell'array ordinato $A[1 \,..\, n-1]$. Scrivete una ricorrenza per il tempo di esecuzione di questa versione ricorsiva di insertion sort.

2.3-5

Riprendendo il problema della ricerca (Esercizio 2.1-3), notiamo che, se la sequenza A è ordinata, possiamo confrontare il punto centrale della sequenza con v ed escludere metà sequenza da ulteriori controlli. La ***ricerca binaria*** è un algoritmo che ripete questa procedura, dimezzando ogni volta la dimensione della porzione restante della sequenza. Scrivete uno pseudocodice, iterativo o ricorsivo, per la ricerca binaria. Dimostrate che il tempo di esecuzione nel caso peggiore della ricerca binaria è $\Theta(\lg n)$.

Acquistato da Michele Mo [illegible] Distribuito da McGraw-Hill Education (Italy)

2.3-6

Il ciclo **while**, righe 5–7, della procedura INSERTION-SORT nel Paragrafo 2.1 usa la ricerca lineare per esplorare (a ritroso) il sottoarray ordinato $A[1 \,..\, j-1]$. È possibile usare la ricerca binaria (Esercizio 2.3-5) per migliorare il tempo di esecuzione complessivo nel caso peggiore di insertion sort fino a $\Theta(n \lg n)$?

2.3-7 ⋆

Descrivete un algoritmo con tempo $\Theta(n \lg n)$ che, dato un insieme S di n numeri interi e un altro intero x, determini se esistono due elementi in S la cui somma è esattamente x.

Problemi

2-1 Insertion sort su piccoli array in merge sort

Sebbene merge sort venga eseguito nel tempo $\Theta(n \lg n)$ nel caso peggiore e insertion sort venga eseguito nel tempo $\Theta(n^2)$ nel caso peggiore, per problemi di piccola dimensione i fattori costanti possono far risultare insertion sort più veloce, nella pratica, su molti tipi di macchina. Quindi, ha senso ingrandire le foglie dell'albero di ricorsione usando insertion sort all'interno di merge sort quando i sottoproblemi diventano sufficientemente piccoli. Considerate una versione modificata di merge sort in cui n/k sottoliste di lunghezza k siano ordinate utilizzando insertion sort e poi fuse mediante il meccanismo standard di merge sort; k è un valore da determinare.

a. Dimostrare che le n/k sottoliste, ciascuna di lunghezza k, possono essere ordinate da insertion sort nel tempo $\Theta(nk)$ nel caso peggiore.

b. Dimostrare che le sottoliste possono essere fuse nel tempo $\Theta(n \lg(n/k))$ nel caso peggiore.

c. Dato che l'algoritmo modificato viene eseguito nel tempo $\Theta(nk + n \lg(n/k))$ nel caso peggiore, qual è il massimo valore asintotico di k (notazione Θ) come funzione di n per cui l'algoritmo modificato ha lo stesso tempo di esecuzione asintotico di merge sort normale?

d. Nella pratica, come dovrebbe essere scelto il valore di k?

2-2 Correttezza di bubblesort

Bubblesort è un noto, ma inefficiente, algoritmo di ordinamento; opera scambiando ripetutamente gli elementi adiacenti che non sono ordinati.

```
BUBBLESORT(A)
1  for i = 1 to A.length − 1
2      for j = A.length downto i + 1
3          if A[j] < A[j − 1]
4              scambia A[j] con A[j − 1]
```

a. Indichiamo con A' l'output di BUBBLESORT(A). Per dimostrare che la procedura BUBBLESORT è corretta, bisogna verificare che termina e che

$$A'[1] \leq A'[2] \leq \cdots \leq A'[n] \tag{2.3}$$

dove $n = A.length$. Che altro bisogna verificare per dimostrare che BUBBLESORT ordina effettivamente gli elementi?

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria 19950301 6:220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

I prossimi due punti dimostrano la disuguaglianza (2.3).

b. Definite con precisione un'invariante di ciclo per il ciclo **for**, righe 2–4, e dimostrate che tale invariante è vera. La vostra dimostrazione dovrebbe usare la struttura della verifica delle invarianti di ciclo presentata in questo capitolo.

c. Utilizzando la condizione di conclusione dell'invariante di ciclo dimostrata nel punto (b), definite un'invariante di ciclo per il ciclo **for**, righe 1–4, che vi consentirà di dimostrare la disuguaglianza (2.3). La vostra dimostrazione dovrebbe usare la struttura della verifica delle invarianti di ciclo presentata in questo capitolo.

d. Qual è il tempo di esecuzione nel caso peggiore di bubblesort? Confrontatelo con il tempo di esecuzione di insertion sort.

2-3 Correttezza della regola di Horner

Il seguente frammento di codice implementa la regola di Horner per il calcolo di un polinomio

$$\begin{aligned} P(x) &= \sum_{k=0}^{n} a_k x^k \\ &= a_0 + x(a_1 + x(a_2 + \cdots + x(a_{n-1} + x a_n)\cdots)) \end{aligned}$$

dati i coefficienti $a_0, a_1, \ldots, a_n$ e un valore di x:

```
1  y = 0
2  for i = n downto 0
3      y = a_i + x · y
```

a. Secondo la notazione Θ, qual è il tempo di esecuzione asintotico di questo frammento di codice per la regola di Horner?

b. Scrivete uno pseudocodice per implementare un semplice algoritmo che calcola ciascun termine del polinomio da zero. Qual è il tempo di esecuzione di questo algoritmo? Confrontatelo con la regola di Horner?

c. Considerate la seguente invariante di ciclo:

All'inizio di ogni iterazione del ciclo **for**, righe 2–3,

$$y = \sum_{k=0}^{n-(i+1)} a_{k+i+1} x^k \, .$$

La vostra dimostrazione dovrebbe usare la struttura della verifica delle invarianti di ciclo presentata in questo capitolo e dovrebbe verificare che, alla fine, $y = \sum_{k=0}^{n} a_k x^k$.

d. Concludete dimostrando che il frammento di codice dato calcola correttamente un polinomio caratterizzato dai coefficienti $a_0, a_1, \ldots, a_n$.

2-4 Inversioni

Sia $A[1..n]$ un array di n numeri distinti. Se $i < j$ e $A[i] > A[j]$, allora la coppia (i, j) è detta ***inversione*** di A.

Acquistato da Michele Mancini - Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

a. Elencate le cinque inversioni dell'array $\langle 2, 3, 8, 6, 1 \rangle$.

b. Quale array con elementi estratti dall'insieme $\{1, 2, \ldots, n\}$ ha più inversioni? Quante inversioni ha?

c. Qual è la relazione fra il tempo di esecuzione di insertion sort e il numero di inversioni nell'array di input? Spiegate la vostra risposta.

d. Create un algoritmo che determina il numero di inversioni in una permutazione di n elementi nel tempo $\Theta(n \lg n)$ nel caso peggiore (*suggerimento*: modificate merge sort).

Note

Nel 1968 Knuth pubblicò il primo di tre volumi con il titolo generale *The Art of Computer Programming* [210, 211, 212]. Il primo volume diede l'avvio allo studio moderno degli algoritmi per calcolatori, con particolare enfasi sull'analisi dei tempi di esecuzione; l'opera completa resta un importante riferimento per molti argomenti trattati nel nostro libro. Secondo Knuth, la parola "algoritmo" deriva da "al-Khowârizmî", un matematico persiano del nono secolo.

Aho, Hopcroft e Ullman [5] sostennero l'analisi asintotica degli algoritmi – con l'uso delle notazioni che verranno introdotte nel Capitolo 3, compresa la notazione Θ – come uno strumento per confrontarne le prestazioni relative. Diffusero inoltre l'uso delle relazioni di ricorrenza per descrivere i tempi di esecuzione degli algoritmi ricorsivi.

Knuth [212] fornisce una trattazione enciclopedica di molti algoritmi di ordinamento. Il suo metodo per confrontare gli algoritmi di ordinamento (pagina 381) usa una analisi del conteggio esatto dei passi, come quella che abbiamo fatto in questo capitolo per insertion sort. La discussione di Knuth sull'algoritmo insertion sort include numerose varianti dell'algoritmo. La più importante di queste è *Shell's sort*, introdotta da D. L. Shell, che applica insertion sort a sottosequenze periodiche dell'input per produrre un algoritmo di ordinamento più veloce. Knuth ha descritto anche merge sort. Egli ricorda che nel 1938 fu inventato un collazionatore meccanico in grado di fondere due pacchi di schede perforate in un solo passo. Pare che nel 1945 J. von Neumann, uno dei pionieri dell'informatica, abbia scritto un programma di merge sort nel calcolatore EDVAC.

Gries [154] ha scritto la storia delle origini delle dimostrazioni della correttezza dei programmi; attribuisce a P. Naur il primo articolo in questo campo e a R. W. Floyd le invarianti di ciclo. Il libro di Mitchell [257] descrive i più recenti progressi nella dimostrazione della correttezza dei programmi.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Crescita delle funzioni 3

Il tasso di crescita del tempo di esecuzione di un algoritmo, definito nel Capitolo 2, fornisce una semplice caratterizzazione dell'efficienza dell'algoritmo; inoltre, ci consente di confrontare le prestazioni relative di algoritmi alternativi. Se la dimensione dell'input n diventa sufficientemente grande, l'algoritmo merge sort, con il suo tempo di esecuzione $\Theta(n \lg n)$ nel caso peggiore, batte insertion sort, il cui tempo di esecuzione nel caso peggiore è $\Theta(n^2)$. Sebbene a volte sia possibile determinare il tempo esatto di esecuzione di un algoritmo, come abbiamo fatto con insertion sort nel Capitolo 2, tuttavia la maggiore precisione, di solito, non compensa lo sforzo per ottenerla. Per input sufficientemente grandi, le costanti moltiplicative e i termini di ordine inferiore che compaiono in un tempo esatto di esecuzione sono dominati dagli effetti della dimensione stessa dell'input.

Quando operiamo con dimensioni dell'input abbastanza grandi da rendere rilevante soltanto il tasso di crescita del tempo di esecuzione, stiamo studiando l'efficienza ***asintotica*** degli algoritmi. In altre parole, ci interessa sapere come aumenta il tempo di esecuzione di un algoritmo al crescere della dimensione dell'input *al limite*, quando la dimensione dell'input cresce senza limitazioni. Di solito, un algoritmo che è asintoticamente più efficiente sarà il migliore con tutti gli input, tranne con quelli molto piccoli.

Questo capitolo descrive diversi metodi standard per semplificare l'analisi asintotica degli algoritmi. Il prossimo paragrafo inizia definendo vari tipi di "notazioni asintotiche", di cui abbiamo già visto un esempio nella notazione Θ. Poi saranno presentate alcune convenzioni sulle notazioni che saranno adottate in tutto il libro. Infine, esamineremo il comportamento delle funzioni che comunemente si usano nell'analisi degli algoritmi.

3.1 Notazione asintotica

Le notazioni che usiamo per descrivere il tempo di esecuzione asintotico di un algoritmo sono definite in termini di funzioni il cui dominio è l'insieme dei numeri naturali $\mathbb{N} = \{0, 1, 2, \ldots\}$. Tali notazioni sono comode per descrivere la funzione $T(n)$, tempo di esecuzione nel caso peggiore, che di solito è definita soltanto con dimensioni intere dell'input. A volte, però, troveremo conveniente *abusare* della notazione asintotica in vari modi. Per esempio, possiamo estendere la notazione al dominio dei numeri reali o limitarla a un sottoinsieme dei numeri naturali. È importante capire il significato esatto della notazione in modo che, quando se ne abusa, non per questo se ne faccia un *uso errato*. Questo paragrafo definisce le notazioni asintotiche di base e presenta anche alcuni tipici abusi.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Notazione asintotica, funzioni e tempi di esecuzione

Utilizzeremo la notazione asintotica principalmente per descrivere i tempi di esecuzione degli algoritmi, come quando abbiamo scritto che il tempo di esecuzione nel caso peggiore di insertion sort era $\Theta(n^2)$. In effetti, però, la notazione asintotica si applica alle funzioni. Ricordiamo che abbiamo caratterizzato il tempo di esecuzione nel caso peggiore di insertion sort come $an^2 + bn + c$, con a, b e c costanti. Scrivendo che il tempo di esecuzione di insertion sort è $\Theta(n^2)$, abbiamo trascurato alcuni dettagli di questa funzione. Poiché la notazione asintotica si applica alle funzioni, ciò che avevamo indicato con $\Theta(n^2)$ era la funzione $an^2 + bn + c$, che in questo caso particolare rappresentava il tempo di esecuzione nel caso peggiore di insertion sort.

In questo libro le funzioni alle quali applichiamo la notazione asintotica rappresentano, generalmente, dei tempi di esecuzione degli algoritmi. Tuttavia la notazione asintotica può essere applicata alle funzioni che caratterizzano qualche altro aspetto degli algoritmi (per esempio, la quantità di spazio che utilizzano) o anche a funzioni che non hanno assolutamente nulla a che fare con gli algoritmi.

Anche quando utilizziamo la notazione asintotica per applicarla al tempo di esecuzione di un algoritmo, dobbiamo sapere a *quale* tempo di esecuzione ci riferiamo. A volte siamo interessati al tempo di esecuzione nel caso peggiore. Talvolta, però, vogliamo caratterizzare il tempo di esecuzione indipendentemente dal particolare input; in altre parole, intendiamo fare un'affermazione generica che includa tutti gli input, non soltanto il caso peggiore. Vedremo notazioni asintotiche che si adattano molto bene a caratterizzare i tempi di esecuzione indipendentemente dal particolare input.

Notazione Θ

Nel Capitolo2 abbiamo visto che il tempo di esecuzione nel caso peggiore di insertion sort è $T(n) = \Theta(n^2)$. Definiamo adesso il significato di questa notazione. Per una data funzione $g(n)$, indichiamo con $\Theta(g(n))$ l'*insieme delle funzioni*

$$\Theta(g(n)) = \{f(n) : \text{ esistono delle costanti positive } c_1, c_2 \text{ e } n_0 \text{ tali che } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ per ogni } n \geq n_0\}\ .^{1}$$

Una funzione $f(n)$ appartiene all'insieme $\Theta(g(n))$ se esistono delle costanti positive c_1 e c_2 tali che essa possa essere "racchiusa" fra $c_1 g(n)$ e $c_2 g(n)$, per valori sufficientemente grandi di n. Poiché $\Theta(g(n))$ è un insieme, dovremmo scrivere "$f(n) \in \Theta(g(n))$" per indicare che $f(n)$ è un membro di $\Theta(g(n))$. Invece, di solito, scriveremo "$f(n) = \Theta(g(n))$" per esprimere lo stesso concetto. Questo abuso del simbolo di uguaglianza per denotare l'appartenenza a un insieme, inizialmente, potrebbe sembrare poco chiaro, ma vedremo più avanti che ha i suoi vantaggi.

La Figura 3.1(a) presenta un quadro intuitivo delle funzioni $f(n)$ e $g(n)$, dove $f(n) = \Theta(g(n))$. Per tutti i valori di n a destra di n_0, il valore di $f(n)$ coincide o sta sopra $c_1 g(n)$ e coincide o sta sotto $c_2 g(n)$. In altre parole, per ogni $n \geq n_0$, la funzione $f(n)$ è uguale a $g(n)$ a meno di un fattore costante. Si dice che $g(n)$ è un ***limite asintoticamente stretto*** per $f(n)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

[1] All'interno della notazione di insieme, i due punti (:) vanno letti "tale che".

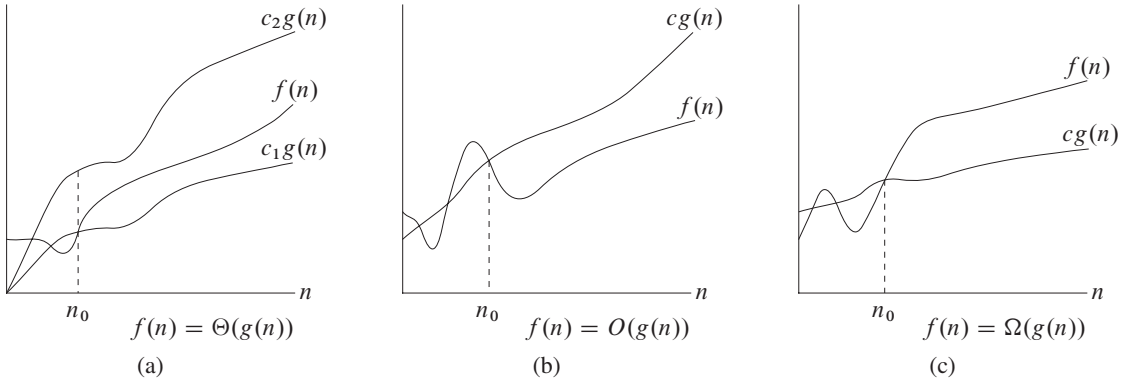


Figura 3.1 Esempi grafici delle notazioni Θ, O e Ω. Nei tre casi il valore di n_0 mostrato è il più piccolo possibile; è accettabile anche qualsiasi valore più grande. **(a)** La notazione Θ limita una funzione a meno di fattori costanti. Si scrive $f(n) = \Theta(g(n))$ se esistono delle costanti positive n_0, c_1 e c_2 tali che, a destra di n_0, il valore di $f(n)$ è sempre compreso fra $c_1 g(n)$ e $c_2 g(n)$, estremi inclusi. **(b)** La notazione O fornisce un limite superiore a una funzione a meno di un fattore costante. Si scrive $f(n) = O(g(n))$ se esistono delle costanti positive n_0 e c tali che, a destra di n_0, il valore di $f(n)$ è sempre minore o uguale di $cg(n)$. **(c)** La notazione Ω fornisce un limite inferiore a una funzione a meno di un fattore costante. Si scrive $f(n) = \Omega(g(n))$ se esistono delle costanti positive n_0 e c tali che, a destra di n_0, il valore di $f(n)$ è sempre maggiore o uguale di $cg(n)$.

La definizione di $\Theta(g(n))$ richiede che ogni membro di $f(n) \in \Theta(g(n))$ sia ***asintoticamente non negativo***, ovvero che $f(n)$ sia non negativa quando n è sufficientemente grande (una funzione ***asintoticamente positiva*** è positiva per qualsiasi valore sufficientemente grande di n). Di conseguenza, la funzione $g(n)$ stessa deve essere asintoticamente non negativa, altrimenti l'insieme $\Theta(g(n))$ è vuoto. Pertanto, supporremo che ogni funzione utilizzata nella notazione Θ sia asintoticamente non negativa. Questa ipotesi vale anche per le altre notazioni asintotiche definite in questo capitolo.

Nel Capitolo 2 abbiamo introdotto un concetto informale della notazione Θ che equivaleva a escludere i termini di ordine inferiore e a ignorare il coefficiente del termine di ordine più elevato. Giustifichiamo brevemente questa intuizione utilizzando la definizione formale per dimostrare che $\frac{1}{2}n^2 - 3n = \Theta(n^2)$. Per farlo, dobbiamo determinare le costanti positive c_1, c_2 e n_0 in modo che

$$c_1 n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2 n^2$$

per qualsiasi $n \geq n_0$. Dividendo per n^2, si ha

$$c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2$$

Acquistato da Michele Mazzucco (mazzucco@cs.ut.ee) - Proprietà letteraria riservata. ISBN 9788838695211 Copyright © 2022, McGraw-Hill Education (Italy)

La disuguaglianza destra può essere resa valida per qualsiasi valore di $n \geq 1$ scegliendo una costante $c_2 \geq 1/2$. Analogamente, la disuguaglianza sinistra può essere resa valida per qualsiasi valore di $n \geq 7$ scegliendo una costante $c_1 \leq 1/14$. Quindi, scegliendo $c_1 = 1/14$, $c_2 = 1/2$ e $n_0 = 7$, possiamo verificare che $\frac{1}{2}n^2 - 3n = \Theta(n^2)$. Certamente è possibile scegliere altri valori delle costanti, ma la cosa importante è che esiste *qualche* scelta. Notate che queste costanti dipendono dalla funzione $\frac{1}{2}n^2 - 3n$; un'altra funzione che appartiene a $\Theta(n^2)$, di solito, richiede costanti differenti. Possiamo applicare anche la definizione formale per verificare

che $6n^3 \neq \Theta(n^2)$. Supponiamo per assurdo che esistano le costanti c_2 e n_0 tali che $6n^3 \leq c_2 n^2$ per ogni $n \geq n_0$; ma allora, dividendo per n^2, si ottiene $n \leq c_2/6$ e questo non può essere assolutamente vero per n arbitrariamente grande, in quanto c_2 è costante.

Intuitivamente, i termini di ordine inferiore di una funzione asintoticamente positiva possono essere ignorati nel determinare i limiti asintoticamente stretti, perché sono insignificanti per grandi valori di n. Per n grande, anche una piccola frazione del termine di ordine più elevato è sufficiente a dominare i termini di ordine inferiore. Quindi, assegnando a c_1 un valore che è leggermente più piccolo del coefficiente del termine di ordine più elevato e a c_2 un valore che è leggermente più grande, è possibile soddisfare le disuguaglianze nella definizione della notazione Θ. In modo analogo, può essere ignorato il coefficiente del termine di ordine più elevato, in quanto esso cambia c_1 e c_2 solo per un fattore costante pari al coefficiente.

Come esempio consideriamo una funzione quadratica $f(n) = an^2 + bn + c$ generica, dove a, b e c sono costanti e $a > 0$. Escludendo i termini di ordine inferiore e ignorando la costante, si ha $f(n) = \Theta(n^2)$. Formalmente, per dimostrare la stessa cosa, prendiamo le costanti $c_1 = a/4$, $c_2 = 7a/4$ e $n_0 = 2 \cdot \max((|b|/a), \sqrt{(|c|/a)})$. Il lettore può verificare che $0 \leq c_1 n^2 \leq an^2 + bn + c \leq c_2 n^2$ per ogni $n \geq n_0$. In generale, per qualsiasi polinomio $p(n) = \sum_{i=0}^{d} a_i n^i$, dove i coefficienti a_i sono costanti e $a_d > 0$, si ha $p(n) = \Theta(n^d)$ (vedere il Problema 3-1).

Poiché qualsiasi costante è un polinomio di grado 0, possiamo esprimere qualsiasi funzione costante come $\Theta(n^0)$ o $\Theta(1)$. Quest'ultima notazione, tuttavia, è un piccolo abuso, perché non specifica quale sia la variabile che tende all'infinito.[2] Adotteremo spesso la notazione $\Theta(1)$ per indicare una costante o una funzione costante rispetto a qualche variabile.

Notazione *O*

La notazione Θ limita asintoticamente una funzione da sopra e da sotto. Quando abbiamo soltanto un ***limite asintotico superiore***, utilizziamo la notazione O. Per una data funzione $g(n)$, denotiamo con $O(g(n))$ (si legge "O grande di g di n" o semplicemente "O di g di n") l'insieme delle funzioni

$$O(g(n)) = \{f(n) : \text{esistono delle costanti positive } c \text{ e } n_0 \text{ tali che } 0 \leq f(n) \leq cg(n) \text{ per ogni } n \geq n_0\}$$

La notazione O si usa per assegnare un limite superiore a una funzione, a meno di un fattore costante. La Figura 3.1(b) illustra il concetto intuitivo che sta dietro questa notazione. Per qualsiasi valore n a destra di n_0, il valore della funzione $f(n)$ coincide o sta sotto $cg(n)$.

Si scrive $f(n) = O(g(n))$ per indicare che una funzione $f(n)$ è un membro dell'insieme $O(g(n))$. Notate che $f(n) = \Theta(g(n))$ implica $f(n) = O(g(n))$, in quanto la notazione Θ è una nozione più forte della notazione O. Secondo la notazione della teoria degli insiemi possiamo scrivere $\Theta(g(n)) \subseteq O(g(n))$. Quindi,

Acquistato da Michele Michele su Webster il 2022-07-07 23:13 Numero Ordine Libreria: 199503816-220787-8 Copyright © 2022 McGraw-Hill Education (Italy)

[2] Il vero problema è che la nostra notazione ordinaria per le funzioni non distingue le funzioni dai valori. Nel lambda-calcolo, i parametri di una funzione sono specificati in modo chiaro: la funzione n^2 deve essere scritta $\lambda n.n^2$ o anche $\lambda r.r^2$. Tuttavia, se adottassimo una notazione più rigorosa, complicheremmo le manipolazioni algebriche; per questo abbiamo scelto di tollerare l'abuso.

la nostra dimostrazione che qualsiasi funzione quadratica $an^2 + bn + c$, con $a > 0$, è in $\Theta(n^2)$ implica anche che tali funzioni quadratiche stanno tutte in $O(n^2)$. Ciò che può sembrare più sorprendente è che qualsiasi funzione *lineare* $an + b$ è in $O(n^2)$; questo è facilmente verificabile ponendo $c = a + |b|$ e $n_0 = \max(1, -b/a)$.

Alcuni lettori che abbiano già visto la notazione O potrebbero trovare strano che noi scriviamo, per esempio, $n = O(n^2)$. Nella letteratura, la notazione O viene a volte utilizzata informalmente per descrivere i limiti asintoticamente stretti, ovvero ciò che noi abbiamo definito con la notazione Θ. In questo libro, tuttavia, quando scriviamo $f(n) = O(g(n))$, stiamo semplicemente affermando che qualche costante multipla di $g(n)$ è un limite asintotico superiore di $f(n)$, senza specificare quanto sia stretto il limite superiore. La distinzione fra limiti superiori asintotici e limiti asintoticamente stretti è diventata standard nella letteratura degli algoritmi. Utilizzando la notazione O, spesso, è possibile descrivere il tempo di esecuzione di un algoritmo, esaminando semplicemente la struttura complessiva dell'algoritmo. Per esempio, la struttura con i cicli doppiamente annidati dell'algoritmo insertion sort del Capitolo 2 genera immediatamente un limite superiore $O(n^2)$ sul tempo di esecuzione nel caso peggiore: il costo di ogni iterazione del ciclo interno è limitato superiormente da $O(1)$ (costante), gli indici i e j sono entrambi al massimo n; il ciclo interno viene eseguito al massimo una volta per ognuna delle n^2 coppie di valori i e j.

Poiché la notazione O descrive un limite superiore, quando la utilizziamo per limitare il tempo di esecuzione nel caso peggiore di un algoritmo, abbiamo un limite sul tempo di esecuzione dell'algoritmo per ogni input – l'affermazione generica di cui si è detto in precedenza. Quindi, il limite $O(n^2)$ sul tempo di esecuzione nel caso peggiore di insertion sort si applica anche al suo tempo di esecuzione per qualsiasi input. Il limite $\Theta(n^2)$ sul tempo di esecuzione nel caso peggiore di insertion sort, tuttavia, non implica un limite $\Theta(n^2)$ sul tempo di esecuzione di insertion sort per *qualsiasi* input. Per esempio, abbiamo visto nel Capitolo 2 che, quando l'input è già ordinato, insertion sort viene eseguito nel tempo $\Theta(n)$.

Tecnicamente, è un abuso dire che il tempo di esecuzione di insertion sort è $O(n^2)$, in quanto, per un dato n, il tempo effettivo di esecuzione varia a seconda del particolare input di dimensione n. Quando scriviamo "il tempo di esecuzione è $O(n^2)$", intendiamo dire che c'è una funzione $f(n)$ che è $O(n^2)$ tale che, per qualsiasi valore di n, indipendentemente da quale input di dimensione n venga scelto, il tempo di esecuzione con quell'input è limitato superiormente dal valore $f(n)$. In modo equivalente, intendiamo che il tempo di esecuzione nel caso peggiore è $O(n^2)$.

Notazione Ω

Così come la notazione O fornisce un limite asintotico *superiore* a una funzione, la notazione Ω fornisce un ***limite asintotico inferiore***. Per una data funzione $g(n)$, denotiamo con $\Omega(g(n))$ (si legge "Omega grande di g di n" o semplicemente "Omega di g di n") l'insieme delle funzioni

$$\Omega(g(n)) = \{f(n) : \text{esistono delle costanti positive } c \text{ e } n_0 \text{ tali che } 0 \le cg(n) \le f(n) \text{ per ogni } n \ge n_0\}$$

Il concetto intuitivo che sta dietro la notazione Ω è illustrato nella Figura 3.1(c). Per tutti i valori di n a destra di n_0, il valore di $f(n)$ coincide o sta sopra $cg(n)$.

Acquistato da Michele Mastrogiovanni - 2012 [illegible] 8386-2007-7 - Copyright © 2023 McGraw-Hill Education (Italy)

Dalle definizioni delle notazioni asintotiche che abbiamo visto finora, è facile dimostrare il seguente importante teorema (vedere l'Esercizio 3.1-5).

Teorema 3.1
Per ogni coppia di funzioni $f(n)$ e $g(n)$, si ha $f(n) = \Theta(g(n))$, se e soltanto se $f(n) = O(g(n))$ e $f(n) = \Omega(g(n))$. ■

Come esempio applicativo di questo teorema, la nostra dimostrazione che $an^2 + bn + c = \Theta(n^2)$ per qualsiasi costante a, b e c, con $a > 0$, implica immediatamente che $an^2 + bn + c = \Omega(n^2)$ e $an^2 + bn + c = O(n^2)$. In pratica, anziché usare il Teorema 3.1 per ottenere i limiti asintotici superiore e inferiore dai limiti asintoticamente stretti, come abbiamo fatto per questo esempio, di solito lo usiamo per dimostrare i limiti asintoticamente stretti dai limiti asintotici superiore e inferiore.

Quando diciamo che il *tempo di esecuzione* (senza ulteriori specifiche) di un algoritmo è $\Omega(g(n))$, intendiamo che *per qualsiasi valore di n e indipendentemente da quale particolare input di dimensione n sia scelto*, il tempo di esecuzione con quell'input è almeno pari a una costante moltiplicata per $g(n)$, con n sufficientemente grande. Ovvero stiamo assegnando un limite inferiore al tempo di esecuzione nel caso migliore di un algoritmo. Per esempio, il tempo di esecuzione nel caso migliore di insertion sort è $\Omega(n)$, che implica che il tempo di esecuzione di insertion sort è $\Omega(n)$.

Il tempo di esecuzione di insertion sort quindi è compreso fra $\Omega(n)$ e $O(n^2)$, in quanto si trova in una zona compresa tra una funzione lineare di n e una funzione quadratica di n. Inoltre, questi limiti sono asintoticamente i più stretti possibili: per esempio, il tempo di esecuzione di insertion sort non è $\Omega(n^2)$, in quanto esiste un input per il quale insertion sort viene eseguito nel tempo $\Theta(n)$ (per esempio, quando l'input è già ordinato). Tuttavia, non è contraddittorio affermare che il tempo di esecuzione nel *caso peggiore* di insertion sort è $\Omega(n^2)$, in quanto esiste un input con il quale l'algoritmo impiega un tempo $\Omega(n^2)$.

Notazione asintotica nelle equazioni e nelle disequazioni

Abbiamo già visto come la notazione asintotica possa essere utilizzata all'interno di formule matematiche. Per esempio, presentando la notazione O, abbiamo scritto "$n = O(n^2)$". Avremmo potuto scrivere anche $2n^2 + 3n + 1 = 2n^2 + \Theta(n)$. Come vanno interpretate queste formule?

Quando la notazione asintotica sta da sola (ossia non all'interno di una formula più grande) sul lato destro di un'equazione (o disuguaglianza), come in $n = O(n^2)$, abbiamo già definito il segno uguale per indicare l'appartenenza all'insieme: $n \in O(n^2)$. In generale, però, quando la notazione asintotica appare in una formula, essa va interpretata come se indicasse qualche funzione anonima, di cui non è importante fare il nome. Per esempio, la formula $2n^2 + 3n + 1 = 2n^2 + \Theta(n)$ significa che $2n^2 + 3n + 1 = 2n^2 + f(n)$, dove $f(n)$ è qualche funzione dell'insieme $\Theta(n)$. In questo caso, $f(n) = 3n + 1$, che appartiene effettivamente a $\Theta(n)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Utilizzando la notazione asintotica in questo modo, è possibile eliminare dettagli superflui e ingombranti da un'equazione. Per esempio, nel Capitolo 2 abbiamo espresso il tempo di esecuzione nel caso peggiore di merge sort con la ricorrenza

$$T(n) = 2T(n/2) + \Theta(n)$$

Se siamo interessati soltanto al comportamento asintotico di $T(n)$, non è importante specificare con esattezza tutti i termini di ordine inferiore; è sottointeso che essi siano tutti inclusi nella funzione anonima indicata dal termine $\Theta(n)$.

Il numero di funzioni anonime in un'espressione è sottointeso che sia uguale al numero di volte che appare la notazione asintotica; per esempio, nell'espressione

$$\sum_{i=1}^{n} O(i)$$

c'è una sola funzione anonima (una funzione di i). Questa espressione *non* è quindi la stessa cosa di $O(1) + O(2) + \cdots + O(n)$ che, in effetti, non ha una chiara interpretazione.

In alcuni casi, la notazione asintotica si trova sul lato sinistro di un'equazione, come in

$$2n^2 + \Theta(n) = \Theta(n^2)$$

Per interpretare simili equazioni, applichiamo la seguente regola: *Indipendentemente dal modo in cui vengano scelte le funzioni anonime a sinistra del segno uguale, c'è un modo di scegliere le funzioni anonime a destra del segno uguale per rendere valida l'equazione*. Quindi, il significato del nostro esempio è che per *qualsiasi* funzione $f(n) \in \Theta(n)$, c'è *qualche* funzione $g(n) \in \Theta(n^2)$ tale che $2n^2 + f(n) = g(n)$ per ogni n. In altre parole, il lato destro di un'equazione fornisce un livello di dettaglio più grossolano del lato sinistro.

Simili relazioni potrebbero essere concatenate in questo modo

$$\begin{aligned} 2n^2 + 3n + 1 &= 2n^2 + \Theta(n) \\ &= \Theta(n^2) \end{aligned}$$

Applicando la precedente regola, possiamo interpretare separatamente le singole equazioni. La prima equazione indica che esiste *qualche* funzione $f(n) \in \Theta(n)$ tale che $2n^2 + 3n + 1 = 2n^2 + f(n)$ per ogni n. La seconda equazione indica che per *qualsiasi* funzione $g(n) \in \Theta(n)$ (come la funzione $f(n)$ appena citata), c'è *qualche* funzione $h(n) \in \Theta(n^2)$ tale che $2n^2 + g(n) = h(n)$ per ogni n. Notate che questa interpretazione implica che $2n^2 + 3n + 1 = \Theta(n^2)$, che è quanto intuitivamente indicano le equazioni concatenate.

Notazione *o*

Il limite asintotico superiore fornito dalla notazione O può essere asintoticamente stretto oppure no. Il limite $2n^2 = O(n^2)$ è asintoticamente stretto, mentre il limite $2n = O(n^2)$ non lo è. Utilizziamo la notazione o per denotare un limite superiore che non è asintoticamente stretto. Definiamo formalmente $o(g(n))$ (si legge "o piccolo di g di n") come l'insieme

$$o(g(n)) = \{f(n) : \text{ per qualsiasi costante } c > 0, \text{ esiste una costante } n_0 > 0 \text{ tale che } 0 \leq f(n) < cg(n) \text{ per ogni } n \geq n_0\}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-2207070 Copyright © 2022, McGraw-Hill Education (Italy)

Per esempio, $2n = o(n^2)$, ma $2n^2 \neq o(n^2)$.

Le definizioni delle notazioni O e o sono simili. La differenza principale è che in $f(n) = O(g(n))$ il limite $0 \leq f(n) \leq cg(n)$ vale per *qualche* costante $c > 0$, mentre in $f(n) = o(g(n))$ il limite $0 \leq f(n) < cg(n)$ vale per *tutte* le costanti $c > 0$. Intuitivamente, nella notazione o la funzione $f(n)$ diventa insignificante rispetto a $g(n)$ quando n tende all'infinito; ovvero

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0 \tag{3.1}$$

Alcuni autori usano questo limite come definizione della notazione o; la definizione in questo libro richiede inoltre che le funzioni anonime siano asintoticamente non negative.

Notazione ω

Per analogia, la notazione ω sta alla notazione Ω come la notazione o sta alla notazione O. Utilizziamo la notazione ω per indicare un limite inferiore che non è asintoticamente stretto. Un modo per definirla è il seguente

$f(n) \in \omega(g(n))$ se e soltanto se $g(n) \in o(f(n))$

Formalmente, tuttavia, definiamo $\omega(g(n))$ ("omega piccolo di g di n") come l'insieme

$$\omega(g(n)) = \{f(n) : \text{ per qualsiasi costante } c > 0, \text{ esiste una costante } n_0 > 0 \text{ tale che } 0 \le cg(n) < f(n) \text{ per ogni } n \ge n_0\}$$

Per esempio, $n^2/2 = \omega(n)$, ma $n^2/2 \neq \omega(n^2)$. La relazione $f(n) = \omega(g(n))$ implica che

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$$

se il limite esiste; cioè $f(n)$ diventa arbitrariamente grande rispetto a $g(n)$ quando n tende all'infinito.

Confronto di funzioni

Molte delle proprietà delle relazioni fra numeri reali si applicano anche ai confronti asintotici. Supponiamo che $f(n)$ e $g(n)$ siano asintoticamente positive.

Proprietà transitiva:

$f(n) = \Theta(g(n))$ e $g(n) = \Theta(h(n))$ implicano $f(n) = \Theta(h(n))$
$f(n) = O(g(n))$ e $g(n) = O(h(n))$ implicano $f(n) = O(h(n))$
$f(n) = \Omega(g(n))$ e $g(n) = \Omega(h(n))$ implicano $f(n) = \Omega(h(n))$
$f(n) = o(g(n))$ e $g(n) = o(h(n))$ implicano $f(n) = o(h(n))$
$f(n) = \omega(g(n))$ e $g(n) = \omega(h(n))$ implicano $f(n) = \omega(h(n))$

Proprietà riflessiva:

$f(n) = \Theta(f(n))$
$f(n) = O(f(n))$
$f(n) = \Omega(f(n))$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Proprietà simmetrica:

$f(n) = \Theta(g(n))$ se e soltanto se $g(n) = \Theta(f(n))$

Simmetria trasposta:

$f(n) = O(g(n))$ se e soltanto se $g(n) = \Omega(f(n))$
$f(n) = o(g(n))$ se e soltanto se $g(n) = \omega(f(n))$

Poiché queste proprietà sono valide per le notazioni asintotiche, è possibile definire un'analogia fra il confronto asintotico di due funzioni f e g e il confronto di due numeri reali a e b:

$f(n) = O(g(n))$ equivale a $a \leq b$
$f(n) = \Omega(g(n))$ equivale a $a \geq b$
$f(n) = \Theta(g(n))$ equivale a $a = b$
$f(n) = o(g(n))$ equivale a $a < b$
$f(n) = \omega(g(n))$ equivale a $a > b$

Diciamo che $f(n)$ è ***asintoticamente più piccola*** di $g(n)$ se $f(n) = o(g(n))$ e $f(n)$ è ***asintoticamente più grande*** di $g(n)$ se $f(n) = \omega(g(n))$. C'è una proprietà dei numeri reali che non può essere trasferita alla notazione asintotica:

Tricotomia: se a e b sono due numeri reali qualsiasi, deve essere valida una sola delle seguenti relazioni: $a < b, a = b, a > b$.

Sebbene sia possibile confrontare due numeri reali qualsiasi, non tutte le funzioni sono asintoticamente confrontabili; ovvero, date due funzioni $f(n)$ e $g(n)$, potrebbe accadere che non sia vero che $f(n) = O(g(n))$ né che $f(n) = \Omega(g(n))$. Per esempio, le funzioni n e $n^{1+\sin n}$ non possono essere confrontate mediante la notazione asintotica, in quanto il valore dell'esponente di $n^{1+\sin n}$ oscilla fra 0 e 2, assumendo tutti i valori intermedi.

Esercizi

3.1-1
Se $f(n)$ e $g(n)$ sono funzioni asintoticamente non negative, usate la definizione di base della notazione Θ per dimostrare che $\max(f(n), g(n)) = \Theta(f(n) + g(n))$.

3.1-2
Dimostrate che per qualsiasi costante reale a e b, con $b > 0$,

$$(n + a)^b = \Theta(n^b) \tag{3.2}$$

3.1-3
Spiegate perché l'asserzione "il tempo di esecuzione dell'algoritmo A è almeno $O(n^2)$" è priva di significato.

3.1-4
È vero che $2^{n+1} = O(2^n)$? È vero che $2^{2n} = O(2^n)$?

3.1-5
Dimostrate il Teorema 3.1.

3.1-6
Dimostrate che il tempo di esecuzione di un algoritmo è $\Theta(g(n))$ se e soltanto se il suo tempo di esecuzione nel caso peggiore è $O(g(n))$ e quello nel caso migliore è $\Omega(g(n))$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

3.1-7
Dimostrate che $o(g(n)) \cap \omega(g(n))$ è l'insieme vuoto.

3.1-8
Possiamo estendere la nostra notazione al caso di due parametri n e m che possono tendere indipendentemente all'infinito con tassi di crescita differenti. Per una data funzione $g(n, m)$, indichiamo con $O(g(n, m))$ l'insieme delle funzioni

$$O(g(n,m)) = \{f(n,m) : \text{esistono delle costanti positive } c, n_0 \text{ e } m_0 \text{ tali che } 0 \leq f(n,m) \leq cg(n,m) \text{ per ogni } n \geq n_0 \text{ o } m \geq m_0\}$$

Date le corrispondenti definizioni per $\Omega(g(n,m))$ e $\Theta(g(n,m))$.

3.2 Notazioni standard e funzioni comuni

Questo paragrafo presenta alcune funzioni e notazioni matematiche standard ed esamina le loro relazioni; descrive anche l'uso delle notazioni asintotiche.

Funzioni monotòne

Una funzione $f(n)$ è ***monotonicamente crescente*** se $m \leq n$ implica $f(m) \leq f(n)$; analogamente, $f(n)$ è ***monotonicamente decrescente*** se $m \leq n$ implica $f(m) \geq f(n)$. Una funzione $f(n)$ è ***strettamente crescente*** se $m < n$ implica $f(m) < f(n)$ e ***strettamente decrescente*** se $m < n$ implica $f(m) > f(n)$.

Floor e ceiling

Dato un numero reale x, indichiamo con $\lfloor x \rfloor$ (si legge "floor di x") l'intero più grande che è minore o uguale a x e con $\lceil x \rceil$ (si legge "ceiling di x") l'intero più piccolo che è maggiore o uguale a x. Per qualsiasi numero reale x

$$x - 1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x + 1 \tag{3.3}$$

Per qualsiasi numero intero n

$$\lceil n/2 \rceil + \lfloor n/2 \rfloor = n$$

Per qualsiasi numero reale $n \geq 0$ e due interi $a, b > 0$

$$\lceil \lceil n/a \rceil / b \rceil = \lceil n/ab \rceil \tag{3.4}$$

$$\lfloor \lfloor n/a \rfloor / b \rfloor = \lfloor n/ab \rfloor \tag{3.5}$$

$$\lceil a/b \rceil \leq (a + (b-1))/b \tag{3.6}$$

$$\lfloor a/b \rfloor \geq (a - (b-1))/b \tag{3.7}$$

La funzione floor $f(x) = \lfloor x \rfloor$ e la funzione ceiling $f(x) = \lceil x \rceil$ sono monotonicamente crescenti.

Aritmetica modulare

Per qualsiasi intero a e qualsiasi intero positivo n, il valore $a \bmod n$ è il ***resto*** (o ***residuo***) del quoziente a/n:

$$a \bmod n = a - \lfloor a/n \rfloor n \tag{3.8}$$

Ne segue che

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$0 \leq a \bmod n < n \tag{3.9}$$

Una volta definito il resto della divisione fra due numeri interi, è comodo utilizzare una notazione speciale per indicare l'uguaglianza dei resti. Se $(a \bmod n) = (b \bmod n)$, scriviamo $a \equiv b \pmod{n}$ e diciamo che a è ***congruo*** a b, modulo n. In altre parole, $a \equiv b \pmod{n}$ se a e b hanno lo stesso resto quando sono divisi per n. In modo equivalente, $a \equiv b \pmod{n}$ se e soltanto se n è un divisore di $b - a$. Scriviamo $a \not\equiv b \pmod{n}$ se a non è congruo a b, modulo n.

Polinomi

Dato un intero non negativo d, un ***polinomio in n di grado d*** è una funzione $p(n)$ della forma

$$p(n) = \sum_{i=0}^{d} a_i n^i$$

dove le costanti $a_0, a_1, \ldots, a_d$ sono i ***coefficienti*** del polinomio e $a_d \neq 0$. Un polinomio è asintoticamente positivo se e soltanto se $a_d > 0$. Per un polinomio asinoticamente positivo $p(n)$ di grado d, si ha $p(n) = \Theta(n^d)$. Per qualsiasi costante reale $a \geq 0$, la funzione n^a è monotonicamente crescente e per qualsiasi costante reale $a \leq 0$, la funzione n^a è monotonicamente decrescente. Si dice che una funzione $f(n)$ è ***polinomialmente limitata*** se $f(n) = O(n^k)$ per qualche costante k.

Esponenziali

Per $a > 0$, m e n reali qualsiasi, si hanno le seguenti identità:

$$\begin{aligned} a^0 &= 1 \\ a^1 &= a \\ a^{-1} &= 1/a \\ (a^m)^n &= a^{mn} \\ (a^m)^n &= (a^n)^m \\ a^m a^n &= a^{m+n} \end{aligned}$$

Per qualsiasi n e $a \geq 1$, la funzione a^n è monotonicamente crescente in n. Per comodità, assumeremo $0^0 = 1$.

Le velocità di crescita delle funzioni polinomiali ed esponenziali sono correlate dal seguente fatto. Per tutte le costanti a e b, con $a > 1$,

$$\lim_{n\to\infty} \frac{n^b}{a^n} = 0 \tag{3.10}$$

da cui possiamo concludere che

$$n^b = o(a^n)$$

Quindi, qualsiasi funzione esponenziale con una base strettamente maggiore di 1 cresce più rapidamente di qualsiasi funzione polinomiale.

Se usiamo e per indicare $2.71828\ldots$, la base dei logaritmi naturali, per qualsiasi valore reale x si ha

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots = \sum_{i=0}^{\infty} \frac{x^i}{i!} \tag{3.11}$$

Acquistato da Michele Michelotti il 2024-07-22 per uso personale (id:1095936, 1095936, 1095936) Copyright © 2022, McGraw-Hill Education (Italy)

Il simbolo "!" indica la funzione fattoriale (che verrà definita successivamente). Per ogni reale x, abbiamo la disuguaglianza

$$e^x \geq 1 + x \tag{3.12}$$

Qui l'uguaglianza vale soltanto se $x = 0$. Quando $|x| \leq 1$, abbiamo l'approssimazione

$$1 + x \leq e^x \leq 1 + x + x^2 \tag{3.13}$$

Quando $x \to 0$, l'approssimazione di e^x con $1 + x$ è abbastanza buona:

$$e^x = 1 + x + \Theta(x^2)$$

(In questa equazione la notazione asintotica è usata per descrivere il comportamento al limite per $x \to 0$, anziché per $x \to \infty$.) Per ogni x si ha

$$\lim_{n\to\infty} \left(1 + \frac{x}{n}\right)^n = e^x \tag{3.14}$$

Logaritmi

Adotteremo le seguenti notazioni:

$$\begin{aligned}
\lg n &= \log_2 n && \text{(logaritmo binario)} \\
\ln n &= \log_e n && \text{(logaritmo naturale)} \\
\lg^k n &= (\lg n)^k && \text{(elevamento a potenza)} \\
\lg \lg n &= \lg(\lg n) && \text{(composizione)}
\end{aligned}$$

Un'importante convenzione che adotteremo è che *le funzioni logaritmiche si applicano soltanto al termine successivo nella formula*, quindi $\lg n + k$ significa $(\lg n) + k$, non $\lg(n + k)$. Per $b > 1$ e $n > 0$, la funzione $\log_b n$ è strettamente crescente. Per qualsiasi reale $a > 0$, $b > 0$, $c > 0$ e n, si ha

$$\begin{aligned}
a &= b^{\log_b a} \\
\log_c(ab) &= \log_c a + \log_c b \\
\log_b a^n &= n \log_b a \\
\log_b a &= \frac{\log_c a}{\log_c b} && (3.15) \\
\log_b(1/a) &= -\log_b a \\
\log_b a &= \frac{1}{\log_a b} \\
a^{\log_b c} &= c^{\log_b a} && (3.16)
\end{aligned}$$

In tutte queste equazioni le basi dei logaritmi devono essere diverse da 1.

Per l'equazione (3.15), cambiando la base di un logaritmo da una costante all'altra, il valore del logaritmo cambia soltanto per un fattore costante, quindi useremo spesso la notazione "$\lg n$" quando i fattori costanti non sono importanti, come nella notazione O. Gli informatici ritengono che 2 sia la base più naturale dei logaritmi, perché molti algoritmi e strutture dati richiedono la suddivisione di un problema in due parti.

C'è un semplice sviluppo in serie di $\ln(1 + x)$ quando $|x| < 1$:

$$\ln(1 + x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \frac{x^5}{5} - \cdots$$

Abbiamo anche le seguenti disuguaglianze per $x > -1$:

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\frac{x}{1 + x} \le \ln(1 + x) \le x \tag{3.17}$$

L'uguaglianza vale soltanto se $x = 0$.

Una funzione $f(n)$ è detta ***polilogaritmicamente limitata*** se $f(n) = O(\lg^k n)$ per qualche costante k. Per correlare la crescita dei polinomi con quella dei polilogaritmi, sostituiamo nell'equazione (3.10) n con $\lg n$ e a con 2^a; otteniamo

$$\lim_{n\to\infty} \frac{\lg^b n}{(2^a)^{\lg n}} = \lim_{n\to\infty} \frac{\lg^b n}{n^a} = 0$$

Da questo limite, possiamo concludere che

$$\lg^b n = o(n^a)$$

per qualsiasi costante $a > 0$. Quindi, una funzione polinomiale positiva cresce più rapidamente di ogni funzione polilogaritmica.

Fattoriali

La notazione $n!$ (si legge "n fattoriale") è definita per i numeri interi $n \geq 0$:

$$n! = \begin{cases} 1 & \text{se } n = 0 \\ n \cdot (n-1)! & \text{se } n > 0 \end{cases}$$

Quindi, $n! = 1 \cdot 2 \cdot 3 \cdots n$.

Un limite superiore non stretto per la funzione fattoriale è $n! \leq n^n$, in quanto ciascuno degli n termini nel prodotto fattoriale è al massimo n. L'***approssimazione di Stirling***

$$n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right) \tag{3.18}$$

dove e è la base del logaritmo naturale, fornisce un limite superiore più stretto e anche un limite inferiore. È possibile dimostrare che (vedere l'Esercizio 3.2-3)

$$\begin{aligned} n! &= o(n^n) \\ n! &= \omega(2^n) \\ \lg(n!) &= \Theta(n \lg n) \end{aligned} \tag{3.19}$$

dove l'approssimazione di Stirling è utile per dimostrare l'equazione (3.19). Anche la seguente equazione vale per qualsiasi $n \geq 1$:

$$n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n e^{\alpha_n} \tag{3.20}$$

dove

$$\frac{1}{12n+1} < \alpha_n < \frac{1}{12n} \tag{3.21}$$

Iterazione di una funzione

Usiamo la notazione $f^{(i)}(n)$ per denotare la funzione $f(n)$ applicata iterativamente i volte a un valore iniziale di n. Formalmente, sia $f(n)$ una funzione definita sui reali. Per interi non negativi i, definiamo in modo ricorsivo

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$f^{(i)}(n) = \begin{cases} n & \text{se } i = 0 \\ f(f^{(i-1)}(n)) & \text{se } i > 0 \end{cases}$$

Per esempio, se $f(n) = 2n$, allora $f^{(i)}(n) = 2^i n$.

La funzione logaritmo iterato

Utilizziamo la notazione $\lg^* n$ (si legge "log stella di n") per denotare il logaritmo iterato, che è definito nel modo seguente. Sia $\lg^{(i)} n$ una funzione definita come nel precedente paragrafo, con $f(n) = \lg n$. Poiché il logaritmo di un numero non positivo non è definito, la funzione $\lg^{(i)} n$ è definita soltanto se $\lg^{(i-1)} n > 0$. Non bisogna confondere $\lg^{(i)} n$ (la funzione logaritmica applicata i volte in successione, a partire dall'argomento n) con $\lg^i n$ (il logaritmo di n elevato alla i-esima potenza). La funzione logaritmo iterato è definita così

$$\lg^* n = \min \left\{ i \geq 0 : \lg^{(i)} n \leq 1 \right\}$$

Il logaritmo iterato è una funzione che cresce *molto* lentamente:

$$\begin{aligned} \lg^* 2 &= 1 \\ \lg^* 4 &= 2 \\ \lg^* 16 &= 3 \\ \lg^* 65536 &= 4 \\ \lg^*(2^{65536}) &= 5 \end{aligned}$$

Poiché si stima che il numero di atomi nell'universo visibile sia pari a circa 10^{80}, che è molto più piccolo di 2^{65536}, difficilmente potremo incontrare un input di dimensione n tale che $\lg^* n > 5$.

Numeri di Fibonacci

I ***numeri di Fibonacci*** sono definiti dalla seguente ricorrenza:

$$\begin{aligned} F_0 &= 0 \\ F_1 &= 1 \\ F_i &= F_{i-1} + F_{i-2} \qquad \text{per } i \geq 2 \end{aligned} \tag{3.22}$$

Poiché ogni numero di Fibonacci è la somma dei due numeri precedenti, si ottiene la sequenza

$$0,\ 1,\ 1,\ 2,\ 3,\ 5,\ 8,\ 13,\ 21,\ 34,\ 55,\ \ldots$$

I numeri di Fibonacci sono correlati al ***rapporto aureo*** ϕ e al suo coniugato $\widehat{\phi}$, che sono le due radici dell'equazione

$$x^2 = x + 1 \tag{3.23}$$

e sono dati dalle seguenti formule (vedere l'Esercizio 3.2-6):

$$\phi = \frac{1 + \sqrt{5}}{2} = 1,61803\ldots \tag{3.24}$$

$$\widehat{\phi} = \frac{1 - \sqrt{5}}{2} = -0,61803\ldots$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 | Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Più precisamente, si ha

$$F_i = \frac{\phi^i - \widehat{\phi}^i}{\sqrt{5}}$$

Questa relazione può essere dimostrata per induzione (vedere l'Esercizio 3.2-7). Poiché $|\widehat{\phi}| < 1$, si ha

$$\begin{aligned} \frac{|\widehat{\phi}^i|}{\sqrt{5}} &< \frac{1}{\sqrt{5}} \\ &< \frac{1}{2} \end{aligned}$$

che implica

$$F_i = \left\lfloor \frac{\phi^i}{\sqrt{5}} + \frac{1}{2} \right\rfloor \tag{3.25}$$

Questa equazione indica che l'i-esimo numero di Fibonacci F_i è uguale a $\phi^i/\sqrt{5}$ arrotondato all'intero più vicino. Dunque, i numeri di Fibonacci crescono in modo esponenziale.

Esercizi

3.2-1

Dimostrate che, se $f(n)$ e $g(n)$ sono funzioni monotonicamente crescenti, allora lo sono anche le funzioni $f(n) + g(n)$ e $f(g(n))$; se $f(n)$ e $g(n)$ sono anche non negative, allora $f(n) \cdot g(n)$ è monotonicamente crescente.

3.2-2

Dimostrate l'equazione (3.16).

3.2-3

Dimostrate l'equazione (3.19). Dimostrate inoltre che $n! = \omega(2^n)$ e $n! = o(n^n)$.

3.2-4 ★

La funzione $\lceil \lg n \rceil !$ è polinomialmente limitata? La funzione $\lceil \lg \lg n \rceil !$ è polinomialmente limitata?

3.2-5 ★

Quale funzione è asintoticamente più grande: $\lg(\lg^* n)$ o $\lg^*(\lg n)$?

3.2-6

Dimostrate che il rapporto aureo ϕ e il suo coniugato $\widehat{\phi}$ soddisfano l'equazione $x^2 = x + 1$.

3.2-7

Dimostrate per induzione che l'i-esimo numero di Fibonacci soddisfa la seguente relazione (ϕ è il rapporto aureo e $\widehat{\phi}$ è il suo coniugato):

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$F_i = \frac{\phi^i - \widehat{\phi}^i}{\sqrt{5}}$$

3.2-8

Dimostrate che $k \ln k = \Theta(n)$ implica $k = \Theta(n / \ln n)$.

Problemi

3-1 Comportamento asintotico dei polinomi

Dato il seguente polinomio in n di grado d

$$p(n) = \sum_{i=0}^{d} a_i n^i$$

con $a_d > 0$ e una costante k, applicate le definizioni delle notazioni asintotiche per dimostrare le seguenti proprietà:

a. Se $k \geq d$, allora $p(n) = O(n^k)$.

b. Se $k \leq d$, allora $p(n) = \Omega(n^k)$.

c. Se $k = d$, allora $p(n) = \Theta(n^k)$.

d. Se $k > d$, allora $p(n) = o(n^k)$.

e. Se $k < d$, allora $p(n) = \omega(n^k)$.

3-2 Crescite asintotiche relative

Indicate, per ogni coppia di espressioni (A, B) della seguente tabella, se A è O, o, Ω, ω, o Θ di B. Supponete che $k \geq 1$, $\epsilon > 0$ e $c > 1$ siano costanti. Inserite le risposte ("sì" o "no") in ogni casella della tabella.

	A	B	O	o	Ω	ω	Θ
a.	$\lg^k n$	n^ϵ					
b.	n^k	c^n					
c.	$\sqrt{n}$	$n^{\sin n}$					
d.	2^n	$2^{n/2}$					
e.	$n^{\lg c}$	$c^{\lg n}$					
f.	$\lg(n!)$	$\lg(n^n)$					

3-3 Classificare le funzioni per tasso di crescita

a. Ordinate le seguenti funzioni per tasso di crescita; ovvero trovate una disposizione $g_1, g_2, \ldots, g_{30}$ delle funzioni che soddisfa le relazioni $g_1 = \Omega(g_2)$, $g_2 = \Omega(g_3), \ldots, g_{29} = \Omega(g_{30})$. Suddividete il vostro elenco in classi di equivalenza in modo tale che $f(n)$ e $g(n)$ si trovino nella stessa classe se e soltanto se $f(n) = \Theta(g(n))$.

$$\begin{array}{cccccc}
\lg(\lg^* n) & 2^{\lg^* n} & (\sqrt{2})^{\lg n} & n^2 & n! & (\lg n)! \\
(\frac{3}{2})^n & n^3 & \lg^2 n & \lg(n!) & 2^{2^n} & n^{1/\lg n} \\
\ln\ln n & \lg^* n & n \cdot 2^n & n^{\lg\lg n} & \ln n & 1 \\
2^{\lg n} & (\lg n)^{\lg n} & e^n & 4^{\lg n} & (n+1)! & \sqrt{\lg n} \\
\lg^*(\lg n) & 2^{\sqrt{2\lg n}} & n & 2^n & n \lg n & 2^{2^{n+1}}
\end{array}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

b. Trovate una funzione $f(n)$ non negativa che, per ogni funzione $g_i(n)$ del punto (a), non sia $O(g_i(n))$ né $\Omega(g_i(n))$.

3-4 Proprietà della notazione asintotica

Siano $f(n)$ e $g(n)$ due funzioni asintoticamente positive. Dimostrate la veridicità o falsità delle seguenti congetture.

a. $f(n) = O(g(n))$ implica $g(n) = O(f(n))$.

b. $f(n) + g(n) = \Theta(\min(f(n), g(n)))$.

c. $f(n) = O(g(n))$ implica $\lg(f(n)) = O(\lg(g(n)))$, dove $\lg(g(n)) \geq 1$ e $f(n) \geq 1$ per ogni n sufficientemente grande.

d. $f(n) = O(g(n))$ implica $2^{f(n)} = O\left(2^{g(n)}\right)$.

e. $f(n) = O\left((f(n))^2\right)$.

f. $f(n) = O(g(n))$ implica $g(n) = \Omega(f(n))$.

g. $f(n) = \Theta(f(n/2))$.

h. $f(n) + o(f(n)) = \Theta(f(n))$.

3-5 Varianti di O e Ω

Alcuni autori definiscono Ω in un modo un po' diverso dal nostro; usiamo $\overset{\infty}{\Omega}$ (si legge "Omega infinito") per questa definizione alternativa. Diciamo che $f(n) = \overset{\infty}{\Omega}(g(n))$ se esiste una costante positiva c tale che $f(n) \geq cg(n) \geq 0$ per un numero infinitamente grande di interi n.

a. Dimostrate che per ogni coppia di funzioni $f(n)$ e $g(n)$, asintoticamente non negative, vale almeno una delle relazioni $f(n) = O(g(n))$ e $f(n) = \overset{\infty}{\Omega}(g(n))$, mentre ciò non è vero se si usa Ω al posto di $\overset{\infty}{\Omega}$.

b. Descrivete i possibili vantaggi e svantaggi di usare $\overset{\infty}{\Omega}$, anziché Ω, per caratterizzare i tempi di esecuzione dei programmi.

Alcuni autori definiscono O in modo un po' diverso; usiamo O' per la definizione alternativa. Diciamo che $f(n) = O'(g(n))$ se e soltanto se $|f(n)| = O(g(n))$.

c. Che cosa accade per ciascuna direzione della clausola "se e soltanto se" nel Teorema 3.1, se sostituiamo O con O', senza cambiare Ω?

Alcuni autori definiscono $\widetilde{O}$ (si legge "O tilde") per indicare O con fattori logaritmici ignorati:

$$\widetilde{O}(g(n)) = \{f(n) : \text{ esistono delle costanti positive } c, k \text{ e } n_0 \text{ tali che } 0 \leq f(n) \leq cg(n)\lg^k(n) \text{ per ogni } n \geq n_0\}$$

d. Definite $\widetilde{\Omega}$ e $\widetilde{\Theta}$ in modo analogo. Dimostrate il corrispondente Teorema 3.1.

3-6 Funzioni iterate

L'operatore di iterazione * usato nella funzione $\lg^*$ può essere applicato a qualsiasi funzione monotonicamente crescente $f(n)$ nei numeri reali. Per una data costante $c \in \mathbb{R}$, definiamo la funzione iterata f_c^* in questo modo:

Acquistato da Michele Michelazzu Webster il 2023-07-07 23:13 Numero Ordine Libreria: 199503018 220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$f_c^*(n) = \min\left\{i \geq 0 : f^{(i)}(n) \leq c\right\}$$

Questa funzione non necessariamente è ben definita in tutti i casi. In altre parole, la quantità $f_c^*(n)$ è il numero di applicazioni ripetute della funzione f che sono necessarie per ridurre il suo argomento a un valore minore o uguale a c.

Per ciascuna delle seguenti funzioni $f(n)$ e costanti c, specificate il limite più stretto possibile per $f_c^*(n)$.

	$f(n)$	c	$f_c^*(n)$
a.	$n-1$	0	
b.	$\lg n$	1	
c.	$n/2$	1	
d.	$n/2$	2	
e.	$\sqrt{n}$	2	
f.	$\sqrt{n}$	1	
g.	$n^{1/3}$	2	
h.	$n/\lg n$	2	

Note

Secondo Knuth [210] l'origine della notazione O risale a un testo sulla teoria dei numeri scritto da P. Bachmann nel 1892. La notazione o è stata inventata da E. Landau nel 1909 per la sua trattazione della distribuzione dei numeri primi. Knuth [214] sostenne l'applicazione delle notazioni Ω e Θ per correggere la pratica diffusa, ma tecnicamente poco precisa, di usare la notazione O per entrambi i limiti superiore e inferiore. Molti continuano a usare la notazione O nei casi in cui la notazione Θ sarebbe tecnicamente più corretta. Per ulteriori informazioni sulla storia e lo sviluppo delle notazioni asintotiche, consultate Knuth [210, 214] e Brassard e Bratley [55].

Non tutti gli autori definiscono le notazioni asintotiche nello stesso modo, sebbene le varie definizioni concordino nella maggior parte delle situazioni più comuni. Alcune definizioni alternative includono funzioni che non sono asintoticamente non negative, nel qual caso sono appropriatamente limitati i loro valori assoluti.

L'equazione (3.20) è dovuta a Robbins [298]. Per altre proprietà delle funzioni matematiche di base, consultate un buon testo di matematica, come Abramowitz e Stegun [1] o Zwillinger [363], o un libro di calcolo, come Apostol [18] o Thomas et alias [335]. I testi di Knuth [210] e di Graham, Knuth e Patashnik [153] contengono materiale abbondante sulla matematica discreta applicata all'informatica.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Divide et impera 4

Nel Paragrafo 2.3.1 abbiamo visto come merge sort rappresenta un esempio del paradigma divide et impera. Ricordiamo che nel metodo divide et impera un problema viene risolto in modo ricorsivo, applicando tre passi a ogni livello di ricorsione:

Divide: questo passo divide il problema in un certo numero di sottoproblemi, che sono istanze più piccole dello stesso problema.

Impera: i sottoproblemi vengono risolti in modo ricorsivo. Comunque, quando i sottoproblemi hanno una dimensione sufficientemente piccola, essi vengono risolti direttamente.

Combina: le soluzioni dei sottoproblemi vengono combinate per generare la soluzione del problema originale.

Quando i sottoproblemi sono abbastanza grandi da essere risolti ricorsivamente, si ha il cosiddetto ***caso ricorsivo***. Una volta che i sottoproblemi diventano sufficientemente piccoli da non richiedere ricorsione, diciamo che la ricorsione ha "toccato il fondo" e che si è raggiunto il ***caso base***. A volte, oltre ai sottoproblemi che sono istanze più piccole dello stesso problema, dobbiamo anche risolvere dei sottoproblemi che non sono affatto uguali al problema originale. Noi consideriamo la soluzione di questi sottoproblemi come parte del passo combina.

In questo capitolo vedremo altri algoritmi basati sul metodo divide et impera. Il primo risolve il problema del massimo sottoarray: l'algoritmo riceve in input un array di numeri e determina il sottoarray di elementi contigui i cui valori hanno la somma massima. Poi vedremo due algoritmi divide et impera per moltiplicare delle matrici $n \times n$. Uno viene eseguito nel tempo $\Theta(n^3)$, che non è migliore del metodo semplice per moltiplicare le matrici quadrate. Mentre il secondo, l'algoritmo di Strassen, viene eseguito nel tempo $O(n^{2.81})$, che batte asintoticamente il metodo semplice.

Ricorrrenze

Le ricorrenze vanno mano nella mano con il paradigma divide et impera, in quanto ci offrono un modo naturale per caratterizzare i tempi di esecuzione degli algoritmi divide et impera. Una ***ricorrenza*** è un'equazione o disequazione che descrive una funzione in termini del suo valore con input più piccoli. Per esempio, come visto nel Paragrafo 2.3.2, il tempo di esecuzione $T(n)$ nel caso peggiore della procedura MERGE-SORT può essere descritto dalla ricorrenza

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ 2T(n/2) + \Theta(n) & \text{se } n > 1 \end{cases} \tag{4.1}$$

la cui soluzione abbiamo detto essere $T(n) = \Theta(n \lg n)$.

Acquistato da Michele Mule' (utente 1202.07.22.01) - Università degli Studi di Bari - 1955.01.16.227.01 - Copyright © 2022, The McGraw-Hill Education (Italy)

Le ricorrenze possono assumere varie forme. Per esempio, un algoritmo ricorsivo può suddividere i sottoproblemi in dimensioni differenti, come una ripartizione $2/3$ e $1/3$. Se i passi divide e combina richiedessero un tempo lineare, un algoritmo siffatto darebbe origine alla ricorrenza $T(n) = T(2n/3) + T(n/3) + \Theta(n)$.

I sottoproblemi non devono necessariamente essere una frazione costante della dimensione del problema originale. Per esempio, una versione ricorsiva della ricerca lineare (vedere l'Esercizio 2.1-3) creerebbe un solo sottoproblema contenente un solo elemento in meno rispetto al problema originale. Ogni chiamata ricorsiva impiegherebbe un tempo costante più il tempo per le chiamate ricorsive che essa fa, generando la ricorrenza $T(n) = T(n-1) + \Theta(1)$.

Questo capitolo presenta tre metodi per risolvere le ricorrenze – cioè per ottenere dei limiti asintotici "Θ" o "O" per la soluzione:

- Nel ***metodo di sostituzione***, ipotizziamo un limite e poi usiamo l'induzione matematica per dimostrare che la nostra ipotesi è corretta.
- Il ***metodo dell'albero di ricorsione*** converte la ricorrenza in un albero i cui nodi rappresentano i costi ai vari livelli della ricorsione; per risolvere la ricorrenza, adotteremo delle tecniche che limitano le sommatorie.
- Il ***metodo dell'esperto*** fornisce i limiti per ricorrenze nella forma

  $$T(n) = aT(n/b) + f(n) \tag{4.2}$$

  dove $a \geq 1$, $b > 1$ e $f(n)$ è una funzione data. Queste ricorrenze si presentano frequentemente. Una ricorrenza della forma (4.2) caratterizza un algoritmo divide et impera che crea a sottoproblemi, ciascuno dei quali ha una dimensione pari a $1/b$ quella del problema originale e in cui i passi divide e combina insieme richiedono un tempo $f(n)$.

  Per utilizzare il metodo dell'esperto occorre memorizzare tre casi, ma una volta fatto ciò, sarà facile determinare i limiti asintotici per molte ricorrenze semplici. Utilizzeremo il metodo dell'esperto per determinare i tempi di esecuzione degli algoritmi divide et impera per il problema del massimo sottoarray e per la moltiplicazione di matrici, come pure per altri algoritmi basati sul metodo divide et impera in altre parti di questo libro.

A volte vedremo ricorrenze che non sono uguaglianze, ma disuguaglianze, come $T(n) \leq 2T(n/2) + \Theta(n)$. Poiché questa ricorrenza stabilisce soltanto un limite superiore su $T(n)$, esprimeremo la sua soluzione utilizzando la notazione O, anziché la notazione Θ. Analogamente, se la disuguaglianza fosse invertita così $T(n) \geq 2T(n/2) + \Theta(n)$, poiché la ricorrenza fornisce soltanto un limite inferiore su $T(n)$, utilizzeremmo la notazione Ω nella sua soluzione.

Dettagli tecnici

In pratica, quando definiamo e risolviamo le ricorrenze, trascuriamo alcuni dettagli tecnici. Per esempio, se chiamiamo MERGE-SORT su n elementi quando n è dispari, finiamo con sottoproblemi di dimensione $\lfloor n/2 \rfloor$ e $\lceil n/2 \rceil$. In effetti, nessuna dimensione è $n/2$, perché $n/2$ non è un intero quando n è dispari. Tecnicamente, la ricorrenza che descrive il tempo di esecuzione nel caso peggiore di MERGE-SORT è effettivamente

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n) & \text{se } n > 1 \end{cases} \tag{4.3}$$

Acquistato da Michele Michele su Webster il 2022-07-23 12:13 Numero Ordine libreria: 1895016 242707 0 Mon Mar 16 2022 McGraw-Hill Education (Italy)

Le condizioni al contorno rappresentano un'altra classe di dettagli che tipicamente trascuriamo. Poiché il tempo di esecuzione di un algoritmo con un input di dimensione costante è una costante, le ricorrenze che derivano dai tempi di esecuzione degli algoritmi, generalmente, hanno $T(n) = \Theta(1)$ per valori sufficientemente piccoli di n. Per comodità, quindi, di solito ometteremo le definizioni delle condizioni al contorno delle ricorrenze e assumeremo che $T(n)$ sia costante per n piccolo. Per esempio, normalmente definiamo la ricorrenza (4.1) così

$$T(n) = 2T(n/2) + \Theta(n) \tag{4.4}$$

senza dare esplicitamente i valori per n piccolo. La ragione sta nel fatto che, sebbene una variazione di $T(1)$ cambi la soluzione esatta della ricorrenza, tuttavia la soluzione tipicamente non cambia per più di un fattore costante e quindi il tasso di crescita resta immutato.

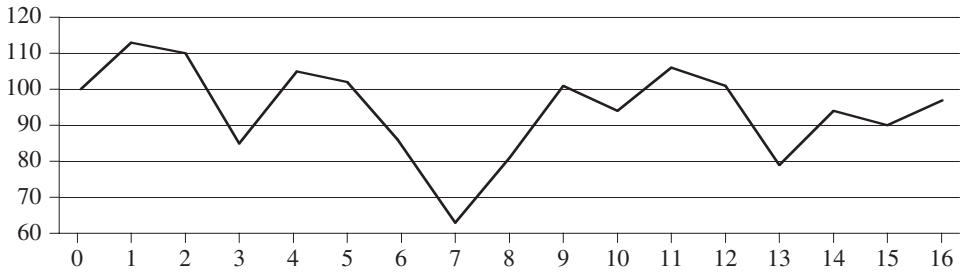
Quando definiamo e risolviamo le ricorrenze, spesso omettiamo le condizioni al contorno e le operazioni floor e ceiling. Procederemo senza questi dettagli e solo in seguito controlleremo se sono importanti oppure no; di solito non lo sono, tuttavia è bene sapere quando lo sono. L'esperienza aiuta, come pure alcuni teoremi che stabiliscono che questi dettagli non influiscono sui limiti asintotici di molte ricorrenze che si incontrano nell'analisi degli algoritmi (vedere il Teorema 4.1). In questo capitolo, tuttavia, ci occuperemo di alcuni di questi dettagli e chiariremo alcuni punti delicati dei metodi di risoluzione delle ricorrenze.

4.1 Il problema del massimo sottoarray

Supponiamo che ci sia stata offerta l'opportunità di investire nella Volatile Chemical Corporation. Il prezzo delle azioni della Volatile Chemical Corporation è molto volatile, come del resto lo sono i prodotti chimici che essa produce. Ci è consentito di comprare un blocco di azioni una sola volta e poi rivenderlo in una data successiva, comprando e vendendo dopo la chiusura delle contrattazioni del giorno. Per compensare questa restrizione, ci è permesso di sapere quale sarà il prezzo che avranno le azioni in futuro. Il nostro obiettivo è massimizzare il profitto. La Figura 4.1 mostra i prezzi delle azioni in un periodo di 17 giorni. Possiamo acquistare le azioni in qualsiasi momento, a partire dal giorno 0, quando il prezzo unitario delle azioni è 100 \$. Ovviamente, per massimizzare il profitto, vorremmo comprare al prezzo più basso possibile per poi vendere al prezzo più alto possibile ("buy low, sell high"). Purtroppo, potremmo non essere in grado di comprare al prezzo più basso e vendere al prezzo più alto in un determinato periodo. Nella Figura 4.1, il prezzo più basso si ha alla fine del giorno 7, che si presenta dopo il prezzo più alto, alla fine del giorno 1.

Potremmo pensare di poter massimizzare sempre il profitto comprando al prezzo più basso o vendendo al prezzo più alto. Per esempio, facendo riferimento alla Figura 4.1, potremmo massimizzare il profitto comprando al prezzo più basso, dopo il giorno 7. Se questa strategia funzionasse sempre, sarebbe facile determinare il massimo profitto: identificare il prezzo più alto e quello più basso; poi cercare il prezzo più basso a sinistra del prezzo più alto; cercare il prezzo più alto a destra del prezzo più basso e, infine, scegliere la coppia di prezzi con la differenza più grande. La Figura 4.2 mostra un semplice controesempio, che dimostra che il massimo profitto a volte si può ottenere senza bisogno di comprare al prezzo più basso o di vendere al prezzo più alto.

Acquistato da Michele Mo... Il materiale è protetto da copyright. Vietata la copia, anche parziale, senza autorizzazione. © McGraw-Hill Education (Italy)



Giorno	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Prezzo	100	113	110	85	105	102	86	63	81	101	94	106	101	79	94	90	97
Variazione		13	−3	−25	20	−3	−16	−23	18	20	−7	12	−5	−22	15	−4	7

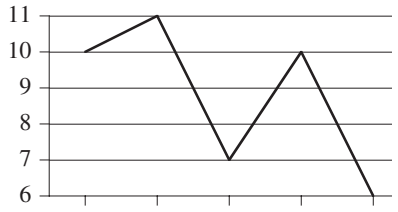
Figura 4.1 Informazioni sui prezzi delle azioni della Volatile Chemical Corporation dopo la chiusura delle contrattazioni in un periodo di 17 giorni. L'asse orizzontale del grafico indica il giorno; quello verticale indica il prezzo. La seconda riga della tabella fornisce la variazione del prezzo rispetto al giorno precedente.

Una soluzione a forza bruta

È facile trovare una soluzione a forza bruta per questo problema: basta provare tutte le possibili coppie di date compra-vendi in cui la data di acquisto precede quella di vendita. Un periodo di n giorni ha $\binom{n}{2}$ di tali coppie di date. Poiché $\binom{n}{2}$ è $\Theta(n^2)$, e la cosa migliore che possiamo sperare è valutare ciascuna coppia di date in un tempo costante, questo approccio richiederebbe un tempo $\Omega(n^2)$. Potremmo fare meglio?

Una trasformazione

Per progettare un algoritmo con un tempo di esecuzione $o(n^2)$, esamineremo l'input da un punto di vista un po' diverso. Vogliamo trovare una sequenza di giorni nella quale la variazione netta dal primo all'ultimo giorno sia massima. Anziché osservare i prezzi giornalieri, consideriamo la variazione giornaliera del prezzo, dove la variazione nel giorno i è la differenza tra i prezzi alla fine del giorno $i-1$ e alla fine del giorno i. La tabella nella Figura 4.1 riporta queste variazioni giornaliere nell'ultima riga. Se trattiamo questa riga come un array A, mostrato nella Figura 4.3, allora noi vogliamo trovare il sottoarray non vuoto di A i cui valo-



Giorno	0	1	2	3	4
Prezzo	10	11	7	10	6
Variazione		1	−4	3	−4

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995030164220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Figura 4.2 Un esempio che dimostra che il massimo profitto non sempre inizia con il prezzo più basso o termina con il prezzo più alto. Anche qui, l'asse orizzontale indica il giorno e quello verticale indica il prezzo. In questo caso, il massimo profitto di 3 $ per azione si otterrebbe comprando alla fine del giorno 2 e vendendo alla fine del giorno 3. Il prezzo di 7 $ alla fine del giorno 2 non è il più basso e il prezzo di 10 $ alla fine del giorno 3 non è il più alto.

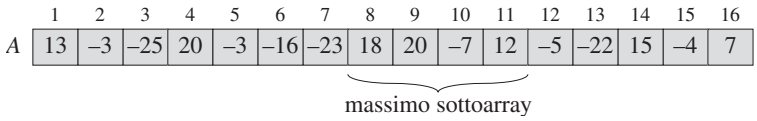


Figura 4.3 La variazione dei prezzi delle azioni come problema del massimo sottoarray. Qui il sottoarray $A[8\,..\,11]$, con somma 43, ha la differenza più grande di qualsiasi sottoarray di elementi contigui dell'array A.

ri hanno la somma massima. Chiameremo questo sottoarray di elementi contigui ***massimo sottoarray***. Per esempio, nell'array della Figura 4.3 il massimo sottoarray di $A[1\,..\,16]$ è $A[8\,..\,11]$, con la somma 43. Quindi, dovremmo comprare le azioni appena prima del giorno 8 (ovvero alla fine del giorno 7) e venderle alla fine del giorno 11, realizzando un profitto di 43 \$ per azione.

A prima vista, questa trasformazione non ci aiuta. Dobbiamo comunque controllare $\binom{n-1}{2} = \Theta(n^2)$ sottoarray per un periodo di n giorni. L'Esercizio 4.1-2 vi chiede di dimostrare che sebbene il calcolo del costo di un sottoarray potrebbe richiedere un tempo proporzionale alla lunghezza del sottoarray, quando calcoliamo tutte le somme dei $\Theta(n^2)$ sottoarray, possiamo organizzare i calcoli in modo che ciascun sottoarray richieda un tempo $O(1)$, noti i valori delle somme dei sottoarray precedentemente calcolate, cosicché la soluzione a forza bruta richiede un tempo $\Theta(n^2)$.

Studiamo allora una soluzione più efficiente del problema del massimo sottoarray. Nel fare questo, di solito parliamo di "un" massimo sottoarray, anziché "del" massimo sottoarray, in quanto potrebbero esserci più sottoarray con la somma massima.

Il problema del massimo sottoarray è interessante soltanto quando l'array contiene alcuni numeri negativi. Se tutti gli elementi dell'array fossero non negativi, il problema del massimo sottoarray sarebbe banale: la somma massima si ha con l'intero array.

Una soluzione utilizzando il metodo divide et impera

Vediamo come possiamo risolvere il problema del massimo sottoarray applicando il metodo divide et impera. Supponiamo di voler trovare un massimo sottoarray del sottoarray $A[low\,..\,high]$. Il metodo divide et impera suggerisce di suddividere il sottoarray in due sottoarray di dimensioni il più possibile uguali. Ovvero, troviamo il punto di mezzo del sottoarray, che chiamiamo *mid*, e consideriamo i sottoarray $A[low\,..\,mid]$ e $A[mid + 1\,..\,high]$. Come mostra la Figura 4.4(a), ogni sottoarray contiguo $A[i\,..\,j]$ di $A[low\,..\,high]$ deve trovarsi esattamente in una delle seguenti posizioni:

- Interamente nel sottoarray $A[low\,..\,mid]$, quindi $low \le i \le j \le mid$.
- Interamente nel sottoarray $A[mid + 1\,..\,high]$, quindi $mid < i \le j \le high$.
- Che passano per il punto di mezzo, quindi $low \le i \le mid < j \le high$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Quindi, un massimo sottoarray di $A[low\,..\,high]$ deve trovarsi in una di queste posizioni. Infatti, un massimo sottoarray di $A[low\,..\,high]$ deve avere la somma più grande rispetto a tutti gli altri sottoarray che si trovano interamente in $A[low\,..\,mid]$, interamente in $A[mid+1\,..\,high]$ o che passano per il punto di mezzo. Possiamo trovare i massimi sottoarray di $A[low\,..\,mid]$ e $A[mid + 1\,..\,high]$ in modo ricorsivo, in quanto questi due sottoproblemi sono istanze più piccole del

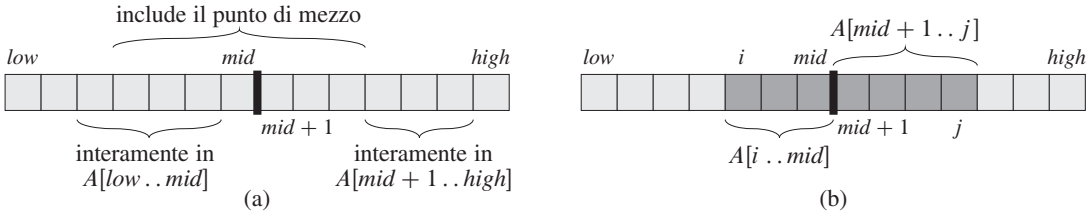


Figura 4.4 **(a)** Posizioni possibili dei sottoarray di $A[low\,..\,high]$: interamente in $A[low\,..\,mid]$, interamente in $A[mid + 1\,..\,high]$ o che passa per il punto di mezzo. **(b)** Qualsiasi sottoarray di $A[low\,..\,high]$ che include il punto di mezzo è formato da due sottoarray $A[i\,..\,mid]$ e $A[mid + 1\,..\,j]$, dove $low \le i \le mid$ e $mid < j \le high$.

problema di trovare un massimo sottoarray. Quindi, ciò che ci resta da fare è trovare un massimo sottoarray che passa per il punto di mezzo e, fra i tre sottoarray, prendere quello con somma maggiore.

Possiamo trovare facilmente un massimo sottoarray che include il punto di mezzo in un tempo lineare nella dimensione del sottoarray $A[low\,..\,high]$. Questo problema *non* è un'istanza più piccola del problema originale, in quanto ha il vincolo aggiuntivo che il sottoarray scelto deve passare per il punto di mezzo. Come mostra la Figura 4.4(b), qualsiasi sottoarray che passa per il punto di mezzo è formato da due sottoarray $A[i\,..\,mid]$ e $A[mid + 1\,..\,j]$, dove $low \le i \le mid$ e $mid < j \le high$. Quindi, basta trovare i massimi sottoarray della forma $A[i\,..\,mid]$ e $A[mid + 1\,..\,j]$ e poi combinarli. La procedura FIND-MAX-CROSSING-SUBARRAY prende come input l'array A e gli indici *low*, *mid* e *high*, e restituisce una tupla contenente gli indici che delimitano un massimo sottoarray che passa per il punto di mezzo, insieme alla somma dei valori in un massimo sottoarray.

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Questa procedura opera nel modo seguente. Le righe 1–7 trovano un massimo sottoarray della metà sinistra, $A[low\,..\,mid]$. Poiché questo sottoarray deve contenere $A[mid]$, il ciclo **for** delle righe 3–7 inizia con l'indice i a *mid* e procede per valori decrescenti dell'indice fino ad arrivare a *low*, cosicché ogni sottoarray che considera è della forma $A[i\,..\,mid]$. Le righe 1–2 inizializzano le variabili *left-sum*, che contiene la somma più grande finora trovata, e *sum*, che contiene la somma degli

elementi di $A[i\,..\,mid]$. Quando troviamo (riga 5) un sottoarray $A[i\,..\,mid]$ con una somma maggiore di *left-sum*, aggiorniamo *left-sum* con la somma di questo sottoarray nella riga 6, e nella riga 7 aggiorniamo la variabile *max-left* per memorizzare questo indice i. Le righe 8–14 operano in modo analogo per la metà destra, $A[mid + 1\,..\,high]$. Qui, il ciclo **for** delle righe 10–14 inizia con l'indice j a $mid + 1$ e procede per valori crescenti dell'indice fino ad arrivare ad *high*, cosicché ogni sottoarray che considera è della forma $A[mid + 1\,..\,j]$. Infine, la riga 15 restituisce gli indici *max-left* e *max-right*, che delimitano un massimo sottoarray che include il punto di mezzo, e la somma *left-sum* + *right-sum* dei valori del sottoarray $A[max\text{-}left\,..\,max\text{-}right]$.

Se il sottoarray $A[low\,..\,high]$ contiene n elementi ($n = high - low + 1$), possiamo affermare che la chiamata della procedura FIND-MAX-CROSSING-SUBARRAY$(A, low, mid, high)$ richiede un tempo $\Theta(n)$. Poiché ogni iterazione di ciascuno dei due cicli **for** richiede un tempo $\Theta(1)$, basta calcolare il numero totale di iterazioni. Il ciclo **for** delle righe 3–7 effettua $mid - low + 1$ iterazioni, e il ciclo **for** delle righe 10–14 effettua $high - mid$ iterazioni; quindi il numero totale di iterazioni è

$$\begin{aligned}(mid - low + 1) + (high - mid) &= high - low + 1\\ &= n\end{aligned}$$

Disponendo della procedura FIND-MAX-CROSSING-SUBARRAY che richiede tempo lineare, possiamo scrivere lo pseudocodice di un algoritmo divide et impera per risolvere il problema del massimo sottoarray:

```
FIND-MAXIMUM-SUBARRAY(A, low, high)
 1  if high == low
 2       return (low, high, A[low])          // caso base: un solo elemento
 3  else mid = ⌊(low + high)/2⌋
 4       (left-low, left-high, left-sum) =
              FIND-MAXIMUM-SUBARRAY(A, low, mid)
 5       (right-low, right-high, right-sum) =
              FIND-MAXIMUM-SUBARRAY(A, mid + 1, high)
 6       (cross-low, cross-high, cross-sum) =
              FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 7       if left-sum ≥ right-sum and left-sum ≥ cross-sum
 8            return (left-low, left-high, left-sum)
 9       elseif right-sum ≥ left-sum and right-sum ≥ cross-sum
10            return (right-low, right-high, right-sum)
11       else return (cross-low, cross-high, cross-sum)
```

La chiamata iniziale di FIND-MAXIMUM-SUBARRAY$(A, 1, A.length)$ troverà un massimo sottoarray di $A[1\,..\,n]$.

Come FIND-MAX-CROSSING-SUBARRAY, anche la procedura ricorsiva FIND-MAXIMUM-SUBARRAY restituisce una tupla contenente gli indici che delimitano un massimo sottoarray, insieme con la somma dei valori di un massimo sottoarray. La riga 1 verifica il caso base, dove il sottoarray ha un solo elemento. Un sottoarray con un solo elemento ha un unico sottoarray - sé stesso – e quindi la riga 2 restituisce una tupla con gli indici iniziale e finale dell'unico elemento, insieme con il suo valore. Le righe 3–11 gestiscono il caso ricorsivo. La riga 3

Acquistato da Michele Michelangeli (webeli@gmail.com) - Copyright © 1998-2016 The McGraw-Hill Education (Italy)

esegue la parte *divide*, calcolando l'indice *mid* del punto di mezzo. Chiamiamo $A[low\,..\,mid]$ ***sottoarray sinistro*** e $A[mid + 1\,..\,high]$ ***sottoarray destro***. Poiché adesso sappiamo che il sottoarray $A[low\,..\,high]$ contiene almeno due elementi, ciascuno dei sottoarray sinistro e destro deve avere almeno un elemento. Le righe 4 e 5 eseguono la parte *impera*, trovando ricorsivamente i massimi sottoarray all'interno dei sottoarray sinistro e destro, rispettivamente. Le righe 6–11 formano la parte *combina*. La riga 6 trova un massimo sottoarray che passa per il punto di mezzo. (Ricordiamo che noi attribuiamo la riga 6 alla parte combina perché risolve un sottoproblema che non è un'istanza più piccola del problema originale.) La riga 7 verifica se il sottoarray sinistro contiene un sottoarray con la somma massima, e la riga 8 restituisce tale sottoarray. Altrimenti, la riga 9 verifica se il sottoarray destro contiene un sottoarray con la somma massima, e la riga 10 restituisce tale sottoarray. Se nessuno dei due sottoarray, sinistro e destro, contiene un sottoarray con la somma massima, allora un massimo sottoarray dovrà passare per il punto di mezzo, e la riga 11 lo restituisce.

Analisi dell'algoritmo divide et impera

A questo punto, costruiamo una ricorrenza che descrive il tempo di esecuzione della procedura ricorsiva FIND-MAXIMUM-SUBARRAY. Come abbiamo fatto per analizzare merge sort nel Paragrafo 2.3.2, supponiamo per semplificare che la dimensione del problema originale sia una potenza di 2, cosicché le dimensioni di tutti i sottoproblemi sono numeri interi. Indichiamo con $T(n)$ il tempo di esecuzione di FIND-MAXIMUM-SUBARRAY su un sottoarray di n elementi. Per cominciare, la riga 1 richiede un tempo costante. Il caso base, quando $n = 1$, è semplice: la riga 2 richiede un tempo costante e quindi

$$T(1) = \Theta(1) \tag{4.5}$$

Il caso ricorsivo si verifica quando $n > 1$. Le righe 1 e 3 richiedono un tempo costante. Ciascun sottoproblema, risolto nelle righe 4 e 5, riguarda un sottoarray di $n/2$ elementi (la nostra ipotesi che la dimensione del problema originale sia una potenza di 2 assicura che $n/2$ sia un intero); quindi occorre un tempo $T(n/2)$ per risolvere ciascun sottoproblema. Poiché dobbiamo risolvere due sottoproblemi – per il sottoarray sinistro e per il sottoarray destro – il contributo al tempo di esecuzione delle righe 4 e 5 è pari a $2T(n/2)$. Come visto in precedenza, la chiamata di FIND-MAX-CROSSING-SUBARRAY nella riga 6 richiede un tempo $\Theta(n)$. Le righe 7–11 richiedono soltanto un tempo $\Theta(1)$. Dunque, per il caso ricorsivo abbiamo

$$\begin{aligned} T(n) &= \Theta(1) + 2T(n/2) + \Theta(n) + \Theta(1) \\ &= 2T(n/2) + \Theta(n) \end{aligned} \tag{4.6}$$

Combinando le equazioni (4.5) e (4.6), otteniamo una ricorrenza per il tempo di esecuzione $T(n)$ di FIND-MAXIMUM-SUBARRAY:

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ 2T(n/2) + \Theta(n) & \text{se } n > 1 \end{cases} \tag{4.7}$$

Questa ricorrenza è la stessa ricorrenza (4.1) di merge sort. Come vedremo con il metodo dell'esperto nel Paragrafo 4.5, questa ricorrenza ha la soluzione $T(n) = \Theta(n \lg n)$. Per capire perché la soluzione debba essere $T(n) = \Theta(n \lg n)$, potreste anche andare a rivedere l'albero di ricorsione nella Figura 2.5. Dunque,

il metodo divide et impera genera un algoritmo che è asintoticamente più veloce del metodo a forza bruta. Con merge sort e adesso con il problema del massimo sottoarray, iniziamo a farci un'idea di quanto potente possa essere il metodo divide et impera. A volte esso genera l'algoritmo asintoticamente più veloce per un problema; altre volte è possibile fare meglio. Come mostra l'Esercizio 4.1-5, esiste infatti un algoritmo in tempo lineare per il problema del massimo sottoarray che non usa il metodo divide et impera.

Esercizi

4.1-1

Che cosa restituisce FIND-MAXIMUM-SUBARRAY quando tutti gli elementi di A sono negativi?

4.1-2

Scrivete lo pseudocodice del metodo a forza bruta per risolvere il problema del massimo sottoarray. La vostra procedura dovrebbe richiedere tempo $\Theta(n^2)$.

4.1-3

Implementate gli algoritmi a forza bruta e ricorsivo per il problema del massimo sottoarray nel vostro computer. Qual è la dimensione n_0 del problema in corrispondenza della quale l'algoritmo ricorsivo incomincia a essere più veloce dell'algoritmo a forza bruta? Poi, modificate il caso base dell'algoritmo ricorsivo per utilizzare l'algoritmo a forza bruta quando la dimensione del problema è minore di n_0. Questo cambia la dimensione del problema in cui l'algoritmo ricorsivo comincia a essere più veloce di quello a forza bruta?

4.1-4

Supponiamo di cambiare la definizione del problema del massimo sottoarray per consentire che il risultato possa essere un sottoarray vuoto, la cui somma dei valori è 0. Come cambiereste gli algoritmi che non ammettono sottoarray vuoti per accettare come risultato un sottoarray vuoto?

4.1-5

Utilizzate le seguenti idee per sviluppare un algoritmo non ricorsivo in tempo lineare per risolvere il problema del massimo sottoarray. Iniziate dall'estremità sinistra dell'array e procedete verso destra, registrando il massimo sottoarray trovato fino a quel punto. Conoscendo un massimo sottoarray di $A[1\,..\,j]$, aggiornate la soluzione considerando il massimo sottoarray che termina con l'indice $j+1$ sulla base della seguente osservazione: un massimo sottoarray di $A[1\,..\,j+1]$ è un massimo sottoarray di $A[1\,..\,j]$ o un sottoarray $A[i\,..\,j+1]$, per $1 \le i \le j+1$. Determinate un massimo sottoarray della forma $A[i\,..\,j+1]$ in tempo costante, supponendo di conoscere un massimo sottoarray che termina con l'indice j.

4.2 Algoritmo di Strassen per il prodotto di matrici

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-9 Copyright © 2022 McGraw-Hill Education (Italy)

Se conoscete le matrici, probabilmente saprete come si moltiplicano. (Altrimenti, leggete il Paragrafo D.1 nell'Appendice D.) Se $A = (a_{ij})$ e $B = (b_{ij})$ sono matrici quadrate $n \times n$, allora nel prodotto $C = A \cdot B$, definiamo l'elemento c_{ij}, per $i, j = 1, 2, \ldots, n$, in questo modo

$$c_{ij} = \sum_{k=1}^{n} a_{ik} \cdot b_{kj} \tag{4.8}$$

Dobbiamo calcolare n^2 elementi della matrice, ciascuno dei quali è la somma di n valori. La seguente procedura prende le matrici A e B di dimensioni $n \times n$ e le moltiplica, restituendo il loro prodotto C che è una matrice $n \times n$. Supponiamo che ciascuna matrice abbia un attributo *rows*, che fornisce il numero di righe della matrice.

```
SQUARE-MATRIX-MULTIPLY(A, B)
1  n = A.rows
2  Sia C una nuova matrice n × n
3  for i = 1 to n
4      for j = 1 to n
5          c_ij = 0
6          for k = 1 to n
7              c_ij = c_ij + a_ik · b_kj
8  return C
```

La procedura SQUARE-MATRIX-MULTIPLY opera nel modo seguente. Il ciclo **for** (righe 3–7) calcola gli elementi di ciascuna riga i e, all'interno di una data riga i, il ciclo **for** (righe 4–7) calcola ciascuno degli elementi c_{ij}, per ogni colonna j. La riga 5 inizializza c_{ij} a 0 quando inizia il calcolo della somma che compare nell'equazione (4.8); ogni iterazione del ciclo **for** (righe 6–7) aggiunge un altro termine dell'equazione (4.8).

Poiché ciascuno dei tre cicli **for** annidati esegue esattamente n iterazioni, e ciascuna esecuzione della riga 7 richiede un tempo costante, la procedura SQUARE-MATRIX-MULTIPLY richiede un tempo $\Theta(n^3)$.

Qualcuno potrebbe pensare che qualsiasi algoritmo per moltiplicare le matrici richieda un tempo $\Omega(n^3)$, in quanto la definizione naturale di prodotto di matrici richiede tutte quelle moltiplicazioni. Ciò non è corretto: c'è un modo per moltiplicare le matrici nel tempo $o(n^3)$. In questo paragrafo esamineremo il notevole algoritmo ricorsivo di Strassen per moltiplicare matrici $n \times n$. Il suo tempo di esecuzione è $\Theta(n^{\lg 7})$, come vedremo nel Paragrafo 4.5. Poiché $\lg 7$ è compreso tra 2.80 e 2.81, l'algoritmo di Strassen viene eseguito nel tempo $O(n^{2.81})$, che è asintoticamente migliore della semplice procedura SQUARE-MATRIX-MULTIPLY.

Un semplice algoritmo divide et impera

Per semplificare le cose, quando utilizziamo un algoritmo divide et impera per calcolare il prodotto di matrici $C = A \cdot B$, supponiamo che n sia una potenza esatta di 2 in ciascuna delle matrici $n \times n$. Facciamo questa ipotesi perché in ogni passo *divide*, divideremo le matrici $n \times n$ in quattro matrici $n/2 \times n/2$, e supponendo che n sia una potenza esatta di 2, abbiamo la certezza che, finché $n \geq 2$, la dimensione $n/2$ è un intero.

Supponiamo di dividere ciascuna delle matrici A, B e C in quattro matrici $n/2 \times n/2$

Acquistato da Michele Michele su Webster il 2022-07-07 22:31:21 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} \tag{4.9}$$

L'equazione $C = A \cdot B$ può essere riscritta così:

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \cdot \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \tag{4.10}$$

L'equazione (4.10) corrisponde alle quattro equazioni

$$C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21} , \tag{4.11}$$
$$C_{12} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22} , \tag{4.12}$$
$$C_{21} = A_{21} \cdot B_{11} + A_{22} \cdot B_{21} , \tag{4.13}$$
$$C_{22} = A_{21} \cdot B_{12} + A_{22} \cdot B_{22} . \tag{4.14}$$

Ciascuna di queste quattro equazioni specifica due prodotti di matrici $n/2 \times n/2$ e la somma dei loro prodotti $n/2 \times n/2$. Possiamo utilizzare queste equazioni per creare un semplice algoritmo ricorsivo divide et impera:

SQUARE-MATRIX-MULTIPLY-RECURSIVE(A, B)

```
1  n = A.rows
2  Sia C una nuova matrice n × n
3  if n == 1
4      c11 = a11 · b11
5  else suddividi A, B e C secondo le equazioni (4.9)
6      C11 = SQUARE-MATRIX-MULTIPLY-RECURSIVE(A11, B11)
           + SQUARE-MATRIX-MULTIPLY-RECURSIVE(A12, B21)
7      C12 = SQUARE-MATRIX-MULTIPLY-RECURSIVE(A11, B12)
           + SQUARE-MATRIX-MULTIPLY-RECURSIVE(A12, B22)
8      C21 = SQUARE-MATRIX-MULTIPLY-RECURSIVE(A21, B11)
           + SQUARE-MATRIX-MULTIPLY-RECURSIVE(A22, B21)
9      C22 = SQUARE-MATRIX-MULTIPLY-RECURSIVE(A21, B12)
           + SQUARE-MATRIX-MULTIPLY-RECURSIVE(A22, B22)
10 return C
```

Questo pseudocodice sorvola su un sottile, ma importante, dettaglio di implementazione. Come dividiamo le matrici nella riga 5? Se dovessimo creare 12 nuove matrici $n/2 \times n/2$, impiegheremmo un tempo $\Theta(n^2)$ per copiarne gli elementi. In effetti, possiamo dividere le matrici senza copiare gli elementi. Il segreto sta nell'utilizzare il calcolo degli indici. Identifichiamo una sottomatrice mediante un intervallo di indici di riga e un intervallo di indici di colonna della matrice originale. Alla fine otterremo una sottomatrice in un modo un po' diverso da come rappresentiamo la matrice originale, e questo è il dettaglio tecnico sul quale sorvoliamo. Il vantaggio è che, potendo specificare le sottomatrici usando il calcolo degli indici, l'esecuzione della riga 5 richiede soltanto un tempo $\Theta(1)$ (anche se vedremo che asintoticamente non fa alcuna differenza sul tempo di esecuzione totale il fatto che copiamo o dividiamo sul posto).

Deriviamo ora una ricorrenza per caratterizzare il tempo di esecuzione di SQUARE-MATRIX-MULTIPLY-RECURSIVE. Indichiamo con $T(n)$ il tempo per moltiplicare due matrici $n \times n$ utilizzando questa procedura. Nel caso base, quando $n = 1$, eseguiamo l'unica moltiplicazione scalare nella riga 4, e quindi

$$T(1) = \Theta(1) \tag{4.15}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Il caso ricorsivo si ha per $n > 1$. Come detto in precedenza, la divisione delle matrici nella riga 5 richiede un tempo $\Theta(1)$, se utilizziamo il calcolo degli indici. Le righe 6–9 chiamano ricorsivamente SQUARE-MATRIX-MULTIPLY-RECURSIVE otto volte in totale. Poiché ogni chiamata ricorsiva moltiplica due matrici $n/2 \times n/2$, apportando un contributo di $T(n/2)$ al tempo di esecuzione totale, il tempo richiesto da tutte e otto le chiamate ricorsive è $8T(n/2)$. Dobbiamo mettere in conto anche le quattro somme di matrici nelle righe 6–9. Ciascuna

di queste matrici contiene $n^2/4$ elementi; quindi ciascuna delle quattro somme di matrici richiede un tempo $\Theta(n^2)$. Poiché il numero di somme di matrici è una costante, il tempo totale impiegato per sommare le matrici nelle righe 6–9 è $\Theta(n^2)$. (Anche qui utilizziamo il calcolo degli indici per inserire i risultati delle somme delle matrici nelle posizioni corrette della matrice C, con un costo aggiuntivo di un tempo $\Theta(1)$ per elemento.) Il tempo totale per il caso ricorsivo, quindi, è la somma del tempo per dividere le matrici, del tempo per eseguire tutte le chiamate ricorsive e del tempo per sommare le matrici risultanti dalle chiamate ricorsive:

$$\begin{aligned} T(n) &= \Theta(1) + 8T(n/2) + \Theta(n^2) \\ &= 8T(n/2) + \Theta(n^2) \end{aligned} \tag{4.16}$$

Notate che se implementassimo la divisione delle matrici copiando gli elementi (operazione che richiederebbe un tempo $\Theta(n^2)$), la ricorrenza non cambierebbe e, quindi, il tempo di esecuzione totale crescerebbe soltanto di un fattore costante.

Combinando le equazioni (4.15) e (4.16), si ottiene la ricorrenza per il tempo di esecuzione di SQUARE-MATRIX-MULTIPLY-RECURSIVE:

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ 8T(n/2) + \Theta(n^2) & \text{se } n > 1 \end{cases} \tag{4.17}$$

Come vedremo con il metodo dell'esperto nel Paragrafo 4.5, la ricorrenza (4.17) ha la soluzione $T(n) = \Theta(n^3)$. Quindi, questo semplice approccio divide et impera non è più veloce della semplice procedura SQUARE-MATRIX-MULTIPLY.

Prima di continuare a esaminare l'algoritmo di Strassen, vediamo da dove provengono le componenti dell'equazione (4.16). La divisione di ciascuna matrice $n \times n$ mediante il calcolo degli indici richiede un tempo $\Theta(1)$, e ci sono due matrici da dividere. Sebbene si possa dire che la divisione delle due matrici richieda un tempo $\Theta(2)$, tuttavia la costante 2 è inclusa nella notazione Θ. La somma di due matrici, ciascuna con k elementi, richiede un tempo $\Theta(k)$. Poiché le matrici che sommiamo hanno $n^2/4$ elementi ciascuna, si potrebbe dire che la somma di una coppia di matrici richieda un tempo $\Theta(n^2/4)$. Anche in questo caso, però, la notazione Θ include il fattore costante $1/4$, e si può affermare che la somma di due matrici $n/2 \times n/2$ richiede un tempo $\Theta(n^2)$. Ci sono quattro di tali somme di matrici, e ancora una volta, diciamo che richiedono un tempo $\Theta(n^2)$, anziché $\Theta(4n^2)$. (Ovviamente, qualcuno potrebbe osservare che avremmo potuto dire che le quattro somme di matrici richiedano un tempo $\Theta(4n^2/4)$, e che $4n^2/4 = n^2$; ma il punto qui è che la notazione Θ include i fattori costanti, qualsiasi essi siano.) Dunque, alla fine otteniamo due termini $\Theta(n^2)$, che possiamo combinare in un unico termine.

Tuttavia, quando consideriamo le otto chiamate ricorsive, non possiamo trascurare il fattore costante 8. In altre parole, dobbiamo dire che tutte insieme esse richiedono un tempo $8T(n/2)$, e non soltanto $T(n/2)$. Per capire questo, riesaminate l'albero di ricorsione della Figura 2.5 per la ricorrenza (2.1) (che è identica alla ricorrenza (4.7)), con il caso ricorsivo $T(n) = 2T(n/2) + \Theta(n)$. Il fattore 2 determinava quanti figli avesse ciascun nodo dell'albero, che a sua volta determinava quanti termini contribuissero alla somma in ciascun livello dell'albero. Se ignorassimo il fattore 8 nell'equazione (4.16) o il fattore 2 nella ricorrenza (4.1), l'albero di ricorsione sarebbe lineare, anziché "ramificato", e ciascun livello contribuirebbe con un solo termine alla somma.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995030167220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Ricordatevi quindi che, mentre la notazione asintotica include i fattori moltiplicativi costanti, una notazione ricorsiva come $T(n/2)$ non include tali fattori.

Metodo di Strassen

La chiave del metodo di Strassen consiste nel rendere un po' meno ramificato l'albero di ricorsione; ovvero, anziché eseguire otto moltiplicazioni ricorsive di matrici $n/2 \times n/2$, ne saranno eseguite soltanto sette. Il costo per eliminare una moltiplicazione di matrici sarà di parecchie nuove somme di matrici $n/2 \times n/2$, ma il numero di somme resterà costante. Questo numero di somme sarà incluso nella notazione Θ quando imposteremo l'equazione della ricorrenza per caratterizzare il tempo di esecuzione.

Il metodo di Strassen non è affatto ovvio. È composto da quattro passi:

1. Dividere le matrici di input A e B e la matrice di output C in sottomatrici $n/2 \times n/2$, come nell'equazione (4.9). Questo passo richiede un tempo $\Theta(1)$ per il calcolo degli indici, come nella procedura SQUARE-MATRIX-MULTIPLY-RECURSIVE.
2. Creare dieci matrici $S_1, S_2, \ldots, S_{10}$, ciascuna delle quali ha dimensione $n/2 \times n/2$ ed è la somma o differenza di due matrici create nel passo 1. Possiamo creare tutte e dieci le matrici nel tempo $\Theta(n^2)$.
3. Utilizzando le sottomatrici create nel passo 1 e le dieci matrici create nel passo 2, calcolare ricorsivamente sette matrici prodotto $P_1, P_2, \ldots, P_7$. Ciascuna matrice P_i ha dimensione $n/2 \times n/2$.
4. Calcolare le sottomatrici richieste $C_{11}, C_{12}, C_{21}, C_{22}$ della matrice C sommando e/o sottraendo varie combinazioni delle matrici P_i. È possibile calcolare tutte e quattro le sottomatrici nel tempo $\Theta(n^2)$.

Esamineremo i dettagli dei passi 2–4 più avanti, ma già da ora abbiamo le informazioni sufficienti per impostare una ricorrenza per il tempo di esecuzione del metodo di Strassen. Supponiamo che, quando la dimensione della matrice n si riduce a 1, eseguiamo un semplice prodotto scalare, come nella riga 4 della procedura SQUARE-MATRIX-MULTIPLY-RECURSIVE. Quando $n > 1$, i passi 1, 2 e 4 richiedono un tempo totale di $\Theta(n^2)$, e il passo 3 richiede di eseguire sette moltiplicazioni di matrici $n/2 \times n/2$. Quindi, otteniamo la seguente ricorrenza per il tempo di esecuzione $T(n)$ dell'algoritmo di Strassen:

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ 7T(n/2) + \Theta(n^2) & \text{se } n > 1 \end{cases} \tag{4.18}$$

Abbiamo scambiato una moltiplicare di matrici con un numero costante di somme di matrici. Se avete ben compreso le ricorrenze e la loro risoluzione potete rendervi conto che questo scambio effettivamente porta a un tempo di esecuzione asintotico più basso. Per il metodo dell'esperto descritto nel Paragrafo 4.5, la ricorrenza (4.18) ha la soluzione $T(n) = \Theta(n^{\lg 7})$.

Acquistato da Michele Michele la Monaca il 2023-07-07 21:13 Numero Ordine dell'Ente 1995021916 297707 © Copyright © 2024 McGraw-Hill Education (Italy)

Analizziamo adesso i dettagli. Nel passo 2 vengono create le seguenti dieci matrici:

$$\begin{aligned} S_1 &= B_{12} - B_{22} \\ S_2 &= A_{11} + A_{12} \\ S_3 &= A_{21} + A_{22} \end{aligned}$$

$$
\begin{aligned}
S_4 &= B_{21} - B_{11} \\
S_5 &= A_{11} + A_{22} \\
S_6 &= B_{11} + B_{22} \\
S_7 &= A_{12} - A_{22} \\
S_8 &= B_{21} + B_{22} \\
S_9 &= A_{11} - A_{21} \\
S_{10} &= B_{11} + B_{12}
\end{aligned}
$$

Poiché dobbiamo sommare o sottrarre dieci volte delle matrici $n/2 \times n/2$, questo passo richiede un tempo $\Theta(n^2)$.

Nel passo 3 moltiplichiamo ricorsivamente sette volte delle matrici $n/2 \times n/2$ per calcolare le seguenti matrici $n/2 \times n/2$, ciascuna delle quali è la somma o la differenza di prodotti di sottomatrici di A e B:

$$
\begin{aligned}
P_1 &= A_{11} \cdot S_1 &&= A_{11} \cdot B_{12} - A_{11} \cdot B_{22} \\
P_2 &= S_2 \cdot B_{22} &&= A_{11} \cdot B_{22} + A_{12} \cdot B_{22} \\
P_3 &= S_3 \cdot B_{11} &&= A_{21} \cdot B_{11} + A_{22} \cdot B_{11} \\
P_4 &= A_{22} \cdot S_4 &&= A_{22} \cdot B_{21} - A_{22} \cdot B_{11} \\
P_5 &= S_5 \cdot S_6 &&= A_{11} \cdot B_{11} + A_{11} \cdot B_{22} + A_{22} \cdot B_{11} + A_{22} \cdot B_{22} \\
P_6 &= S_7 \cdot S_8 &&= A_{12} \cdot B_{21} + A_{12} \cdot B_{22} - A_{22} \cdot B_{21} - A_{22} \cdot B_{22} \\
P_7 &= S_9 \cdot S_{10} &&= A_{11} \cdot B_{11} + A_{11} \cdot B_{12} - A_{21} \cdot B_{11} - A_{21} \cdot B_{12}
\end{aligned}
$$

Notate che le uniche moltiplicazioni da eseguire sono quelle che si trovano nella colonna centrale delle precedenti equazioni. La colonna a destra mostra semplicemente a che cosa sono uguali questi prodotti in funzione delle sottomatrici originali create nel passo 1.

Il passo 4 somma e sottrae le matrici P_i create nel passo 3 per costruire le quattro sottomatrici $n/2 \times n/2$ del prodotto C. Iniziamo con

$$C_{11} = P_5 + P_4 - P_2 + P_6$$

Espandendo il lato destro, mettendo le espansioni delle P_i in righe successive e allineando verticalmente i termini che si annullano, notiamo che C_{11} è uguale a

$$
\begin{array}{llll}
A_{11} \cdot B_{11} & + A_{11} \cdot B_{22} & + A_{22} \cdot B_{11} & + A_{22} \cdot B_{22} \\
 & - A_{22} \cdot B_{11} & & + A_{22} \cdot B_{21} \\
- A_{11} \cdot B_{22} & & & - A_{12} \cdot B_{22} \\
 & & - A_{22} \cdot B_{22} - A_{22} \cdot B_{21} + A_{12} \cdot B_{22} & + A_{12} \cdot B_{21} \\
\hline
A_{11} \cdot B_{11} & & & + A_{12} \cdot B_{21}
\end{array}
$$

che corrisponde all'equazione (4.11).

Analogamente, se consideriamo la sottomatrice

$$C_{12} = P_1 + P_2$$

essa sarà uguale a

$$
\begin{array}{ll}
A_{11} \cdot B_{12} & - A_{11} \cdot B_{22} \\
 & + A_{11} \cdot B_{22} + A_{12} \cdot B_{22} \\
\hline
A_{11} \cdot B_{12} & + A_{12} \cdot B_{22}
\end{array}
$$

che corrisponde all'equazione (4.12).

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Così

$$C_{21} = P_3 + P_4$$

sarà uguale a

$$\begin{array}{l} A_{21} \cdot B_{11} + A_{22} \cdot B_{11} \\ \qquad - A_{22} \cdot B_{11} + A_{22} \cdot B_{21} \\ \hline A_{21} \cdot B_{11} \qquad\qquad + A_{22} \cdot B_{21} \end{array}$$

che corrisponde all'equazione (4.13).

Infine,

$$C_{22} = P_5 + P_1 - P_3 - P_7$$

sarà uguale a

$$\begin{array}{l} A_{11} \cdot B_{11} + A_{11} \cdot B_{22} + A_{22} \cdot B_{11} + A_{22} \cdot B_{22} \\ \qquad - A_{11} \cdot B_{22} \qquad\qquad + A_{11} \cdot B_{12} \\ \qquad\qquad - A_{22} \cdot B_{11} \qquad\qquad - A_{21} \cdot B_{11} \\ - A_{11} \cdot B_{11} \qquad\qquad - A_{11} \cdot B_{12} + A_{21} \cdot B_{11} + A_{21} \cdot B_{12} \\ \hline \qquad\qquad A_{22} \cdot B_{22} \qquad\qquad + A_{21} \cdot B_{12} \end{array}$$

che corrisponde all'equazione (4.14). In totale, nel passo 4 sommiamo o sottraiamo otto volte delle matrici $n/2 \times n/2$; quindi, questo passo richiede effettivamente un tempo $\Theta(n^2)$.

Dunque, notiamo che l'algoritmo di Strassen, formato dai passi 1–4, fornisce il prodotto di matrici corretto e che la ricorrenza (4.18) caratterizza il suo tempo di esecuzione. Poiché, come vedremo nel Paragrafo 4.5, questa ricorrenza ha la soluzione $T(n) = \Theta(n^{\lg 7})$, il metodo di Strassen è asintoticamente più veloce della semplice procedura SQUARE-MATRIX-MULTIPLY. Le note alla fine di questo capitolo descrivono alcuni degli aspetti pratici dell'algoritmo di Strassen.

Esercizi

Nota: sebbene gli Esercizi 4.2-3, 4.2-4 e 4.2-5 riguardino alcune varianti dell'algoritmo di Strassen, dovreste leggere il Paragrafo 4.5 prima di provare a risolverli.

4.2-1
Utilizzate l'algoritmo di Strassen per calcolare il prodotto delle matrici

$$\begin{pmatrix} 1 & 3 \\ 7 & 5 \end{pmatrix} \begin{pmatrix} 6 & 8 \\ 4 & 2 \end{pmatrix}$$

Descrivete le operazioni.

Acquistato da Michele Mastrovito (mastrovito@gmail.com) 2023:3:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

4.2-2
Scrivete lo pseudocodice per l'algoritmo di Strassen.

4.2-3
Come dovrebbe essere modificato l'algoritmo di Strassen per moltiplicare delle matrici $n \times n$ quando n non è una potenza esatta di 2? Dimostrate che l'algoritmo risultante viene eseguito nel tempo $\Theta(n^{\lg 7})$.

4.2-4
Qual è il più grande valore di k tale che, se fosse possibile moltiplicare delle matrici 3×3 effettuando k moltiplicazioni (senza utilizzare la commutatività della moltiplicazione), allora sarebbe possibile moltiplicare delle matrici $n \times n$ nel tempo $o(n^{\lg 7})$? Quale dovrebbe essere il tempo di esecuzione di questo algoritmo?

4.2-5
V. Pan ha scoperto un modo di moltiplicare delle matrici 68×68 effettuando 132 464 moltiplicazioni, un modo di moltiplicare delle matrici 70×70 effettuando 143 640 moltiplicazioni e un modo di moltiplicare delle matrici 72×72 effettuando 155 424 moltiplicazioni. Quale metodo ha il miglior tempo di esecuzione asintotico quando viene utilizzato in un algoritmo divide et impera per moltiplicare le matrici? Confrontate questo metodo con l'algoritmo di Strassen.

4.2-6
Quanto velocemente è possibile moltiplicare una matrice $kn \times n$ per una matrice $n \times kn$, utilizzando l'algoritmo di Strassen come subroutine? Rispondete alla stessa domanda invertendo l'ordine delle matrici di input.

4.2-7
Spiegate come moltiplicare i numeri complessi $a+bi$ e $c+di$ effettuando soltanto tre moltiplicazioni di numeri reali. L'algoritmo dovrebbe ricevere a, b, c e d come input e produrre separatamente la parte reale $ac - bd$ e la parte immaginaria $ad + bc$.

4.3 Il metodo di sostituzione per risolvere le ricorrenze

Adesso che avete visto come le ricorrenze caratterizzano i tempi di esecuzione degli algoritmi divide et impera, imparerete a risolvere le ricorrenze. Iniziamo in questo paragrafo con il metodo di "sostituzione".

Il ***metodo di sostituzione*** per risolvere le ricorrenze richiede due passi:

1. Ipotizzare la forma della soluzione.
2. Usare l'induzione matematica per trovare le costanti e dimostrare che la soluzione funziona.

Il nome del metodo deriva dalla sostituzione della soluzione ipotizzata al posto della funzione quando l'ipotesi induttiva viene applicata a valori più piccoli. Questo metodo è potente, ma ovviamente può essere applicato soltanto nei casi in cui sia facile immaginare la forma della soluzione.

Il metodo di sostituzione può essere usato per determinare il limite inferiore o superiore di una ricorrenza. Come esempio, determiniamo un limite superiore per la ricorrenza

$$T(n) = 2T(\lfloor n/2 \rfloor) + n \tag{4.19}$$

Acquistato da Michele Michele su Webster il 2022-07-27 a uso esclusivo per fini personali. Non è consentita la distribuzione o la condivisione a terzi.

che è simile alle ricorrenze (4.3) e (4.4). Supponiamo che la soluzione sia $T(n) = O(n \lg n)$. Il nostro metodo consiste nel dimostrare che $T(n) \leq cn \lg n$ per una scelta appropriata della costante $c > 0$. Supponiamo, innanzi tutto, che questo limite sia valido per $\lfloor n/2 \rfloor$, ovvero che $T(\lfloor n/2 \rfloor) \leq c \lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)$. Facendo le opportune sostituzioni nella ricorrenza, si ha

$$\begin{aligned} T(n) &\leq 2(c \lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + n \\ &\leq cn \lg(n/2) + n \end{aligned}$$

$$
\begin{aligned}
&= cn \lg n - cn \lg 2 + n \\
&= cn \lg n - cn + n \\
&\leq cn \lg n
\end{aligned}
$$

L'ultimo passo è vero quando $c \geq 1$.

A questo punto, l'induzione matematica richiede di dimostrare che la nostra soluzione vale per le condizioni al contorno. Tipicamente, questo è fatto dimostrando che le condizioni al contorno sono appropriate come casi base della dimostrazione induttiva. Per la ricorrenza (4.19), dobbiamo dimostrare che è possibile scegliere una costante c sufficientemente grande in modo che il limite $T(n) \leq cn \lg n$ sia valido anche per le condizioni al contorno. Questa necessità a volte può creare dei problemi. Supponiamo per esempio che $T(1) = 1$ sia l'unica condizione al contorno della ricorrenza. Allora per $n = 1$, il limite $T(n) \leq cn \lg n$ diventa $T(1) \leq c1 \lg 1 = 0$, che è in contrasto con $T(1) = 1$. Di conseguenza, il caso base della nostra dimostrazione induttiva non risulta valido.

Questa difficoltà nel dimostrare un'ipotesi induttiva per una specifica condizione al contorno può essere facilmente superata. Per esempio, nella ricorrenza (4.19), sfruttiamo la notazione asintotica che ci richiede soltanto di provare che $T(n) \leq cn \lg n$ per $n \geq n_0$, dove n_0 è una costante *arbitrariamente scelta*. L'idea è quella di escludere la difficile condizione al contorno $T(1) = 1$ dalla dimostrazione induttiva. Osservate che per $n > 3$, la ricorrenza non dipende direttamente da $T(1)$. Pertanto, possiamo sostituire $T(1)$ con $T(2)$ e $T(3)$ come casi base della dimostrazione induttiva, ponendo $n_0 = 2$. Notate che facciamo una distinzione fra il caso base della ricorrenza ($n = 1$) e i casi base della dimostrazione induttiva ($n = 2$ e $n = 3$). Dalla ricorrenza otteniamo che $T(2) = 4$ e $T(3) = 5$. La dimostrazione induttiva che $T(n) \leq cn \lg n$ per qualche costante $c \geq 1$ adesso può essere completata scegliendo c sufficientemente grande in modo che $T(2) \leq c2 \lg 2$ e $T(3) \leq c3 \lg 3$. Come si può vedere, è sufficiente scegliere un valore $c \geq 2$ per rendere validi i casi base di $n = 2$ e $n = 3$. Per la maggior parte delle ricorrenze che esamineremo, è facile estendere le condizioni al contorno in modo che l'ipotesi induttiva sia valida per piccoli valori di n e per questo noi non tratteremo sempre esplicitamente tali dettagli.

Formulare una buona ipotesi

Purtroppo non esiste un metodo generale per indovinare la soluzione corretta di una ricorrenza. Per indovinare una soluzione bisogna avere esperienza e, a volte, creatività. Fortunatamente, esistono alcune euristiche che ci aiutano a diventare buoni indovini. Per formulare delle buone ipotesi, è anche possibile utilizzare gli alberi di ricorsione, che descriveremo nel Paragrafo 4.4.

Se una ricorrenza è simile a una che avete già visto, allora ha senso provare una soluzione analoga. Per esempio, considerate la ricorrenza

$$T(n) = 2T(\lfloor n/2 \rfloor + 17) + n$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

che sembra difficile per l'aggiunta del "17" nell'argomento di T. Intuitivamente, però, questo termine aggiuntivo non può influire in modo sostanziale sulla soluzione della ricorrenza. Quando n è grande, la differenza fra $T(\lfloor n/2 \rfloor)$ e $T(\lfloor n/2 \rfloor + 17)$ non è così grande: in entrambi i casi, n viene diviso all'incirca a metà. Di conseguenza, facciamo l'ipotesi che $T(n) = O(n \lg n)$, la cui validità può essere dimostrata applicando il metodo di sostituzione (vedere l'Esercizio 4.3-6).

Un altro modo per fare una buona ipotesi consiste nel dimostrare dei limiti superiori e inferiori laschi per la ricorrenza per poi ridurre progressivamente il grado di incertezza. Per esempio, potremmo iniziare con un limite inferiore di $T(n) = \Omega(n)$ per la ricorrenza (4.19), in quanto abbiamo il termine n nella ricorrenza, e provare un limite superiore iniziale pari a $T(n) = O(n^2)$. Poi, potremmo ridurre gradualmente il limite superiore e alzare quello inferiore fino a convergere alla soluzione corretta e asintoticamente stretta $T(n) = \Theta(n \lg n)$.

Finezze

Ci sono casi in cui è possibile ipotizzare correttamente un limite asintotico per la soluzione di una ricorrenza, ma in qualche modo sembra che i calcoli matematici non tornino nell'induzione. Di solito, il problema è che l'ipotesi induttiva non è abbastanza forte per dimostrare il limite esatto. Quando ci si imbatte in simili ostacoli, spesso basta correggere l'ipotesi sottraendo un termine di ordine inferiore per far tornare i conti. Consideriamo la seguente ricorrenza

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$$

Supponiamo che la soluzione sia $T(n) = O(n)$; proviamo a dimostrare che $T(n) \leq cn$ per una costante c appropriatamente scelta. Sostituendo la nostra ipotesi nella ricorrenza, otteniamo

$$\begin{aligned} T(n) &\leq c \lfloor n/2 \rfloor + c \lceil n/2 \rceil + 1 \\ &= cn + 1 \end{aligned}$$

che non implica $T(n) \leq cn$ qualunque sia il valore di c. Saremmo tentati di provare un'ipotesi più ampia, per esempio $T(n) = O(n^2)$, che può funzionare, ma in effetti la nostra ipotesi che la soluzione sia $T(n) = O(n)$ è corretta. Per provarlo, però, dobbiamo formulare un'ipotesi induttiva più forte.

Intuitivamente, la nostra ipotesi è quasi esatta: non vale soltanto a causa della costante 1, un termine di ordine inferiore. Nonostante questo, l'induzione matematica non funziona, a meno che non si usi la forma esatta dell'ipotesi induttiva. Superiamo questa difficoltà *sottraendo* un termine di ordine inferiore dalla precedente ipotesi. La nuova ipotesi è $T(n) \leq cn - d$, dove $d \geq 0$ è costante. Adesso abbiamo

$$\begin{aligned} T(n) &\leq (c \lfloor n/2 \rfloor - d) + (c \lceil n/2 \rceil - d) + 1 \\ &= cn - 2d + 1 \\ &\leq cn - d \end{aligned}$$

per ogni $d \geq 1$. Come prima, la costante c deve essere scelta sufficientemente grande perché siano soddisfatte le condizioni al contorno.

Potrebbe sembrare che l'idea di sottrarre un termine di ordine inferiore non sia intuitiva. Dopo tutto, se i calcoli matematici non tornano, non dovremmo aumentare la nostra ipotesi? Non necessariamente! Quando dimostriamo per induzione un limite superiore può risultare più difficile dimostrarne uno più grande perché dobbiamo usare tale limite più grande nella dimostrazione del passo induttivo. Nell'esempio corrente, in cui la ricorrenza ha più di un termine ricorsivo, sottraiamo il termine di ordine inferiore del limite proposto una volta per ogni termine ricorsivo. Nell'esempio precedente, abbiamo sottratto la costante d due volte, una

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

volta per il termine $T(\lfloor n/2 \rfloor)$ e una volta per il termine $T(\lceil n/2 \rceil)$. Alla fine abbiamo ottenuto la disuguaglianza $T(n) \leq cn - 2d + 1$, ed è stato facile trovare i valori di d che rendono $cn - 2d + 1$ minore o uguale a $cn - d$.

Evitare i tranelli

È facile sbagliare usando la notazione asintotica. Per esempio, nella ricorrenza (4.19) potremmo "dimostrare" erroneamente che $T(n) = O(n)$, supponendo che $T(n) \leq cn$ e poi deducendo che

$$\begin{aligned} T(n) &\leq 2(c\lfloor n/2 \rfloor) + n \\ &\leq cn + n \\ &= O(n) \qquad \Longleftarrow \textit{sbagliato!} \end{aligned}$$

in quanto c è una costante. L'errore sta nel fatto che non abbiamo dimostrato la *forma esatta* dell'ipotesi induttiva, ovvero che $T(n) \leq cn$. Quindi, dobbiamo dimostrare esplicitamente che $T(n) \leq cn$ quando vogliamo dimostrare che $T(n) = O(n)$.

Sostituzione di variabili

A volte, una piccola manipolazione algebrica può rendere una ricorrenza ignota simile a una che avete già visto. Per esempio, la seguente ricorrenza sembra difficile da risolvere

$$T(n) = 2T\left(\lfloor \sqrt{n} \rfloor\right) + \lg n$$

Tuttavia, è possibile semplificare questa ricorrenza con una sostituzione di variabili. Per comodità, ignoreremo l'arrotondamento agli interi di valori come $\sqrt{n}$. Ponendo $m = \lg n$ si ottiene

$$T(2^m) = 2T(2^{m/2}) + m$$

Adesso poniamo $S(m) = T(2^m)$ per ottenere la nuova ricorrenza

$$S(m) = 2S(m/2) + m$$

che è molto simile alla ricorrenza (4.19). In effetti, questa nuova ricorrenza ha la stessa soluzione: $S(m) = O(m \lg m)$. Ripristinando $T(n)$, otteniamo $T(n) = T(2^m) = S(m) = O(m \lg m) = O(\lg n \lg \lg n)$.

Esercizi

4.3-1
Dimostrate che la soluzione di $T(n) = T(n-1) + n$ è $O(n^2)$.

4.3-2
Dimostrate che la soluzione di $T(n) = T(\lceil n/2 \rceil) + 1$ è $O(\lg n)$.

4.3-3
Acquistato da Michele Michele su Webster il 2022-07-07 23:12 : Numera Ordine Libreria 19950306-320707-0 Copyright © 2022 McGraw-Hill Education (Italy)
Abbiamo visto che la soluzione di $T(n) = 2T(\lfloor n/2 \rfloor) + n$ è $O(n \lg n)$. Dimostrate che la soluzione di questa ricorrenza è anche $\Omega(n \lg n)$. In conclusione, la soluzione è $\Theta(n \lg n)$.

4.3-4
Dimostrate che, formulando una diversa ipotesi induttiva, è possibile superare la difficoltà della condizione al contorno $T(1) = 1$ per la ricorrenza (4.19), senza bisogno di modificare le condizioni al contorno per la dimostrazione induttiva.

4.3-5
Dimostrate che $\Theta(n \lg n)$ è la soluzione della ricorrenza (4.3) "esatta" per merge sort.

4.3-6
Dimostrate che la soluzione di $T(n) = 2T(\lfloor n/2 \rfloor + 17) + n$ è $O(n \lg n)$.

4.3-7
Utilizzando il metodo dell'esperto descritto nel Paragrafo 4.5, potete dimostrare che la soluzione della ricorrenza $T(n) = 4T(n/3) + n$ è $T(n) = \Theta(n^{\log_3 4})$. Dimostrate che la prova di sostituzione con l'ipotesi $T(n) \leq cn^{\log_3 4}$ non funziona. Poi mostrate come sottrarre un termine di ordine inferiore per far funzionare la prova di sostituzione.

4.3-8
Utilizzando il metodo dell'esperto descritto nel Paragrafo 4.5, potete dimostrare che la soluzione della ricorrenza $T(n) = 4T(n/2) + n^2$ è $T(n) = \Theta(n^2)$. Dimostrate che la prova di sostituzione con l'ipotesi $T(n) \leq cn^2$ non funziona. Poi mostrate come sottrarre un termine di ordine inferiore per far funzionare la prova di sostituzione.

4.3-9
Risolvete la ricorrenza $T(n) = 3T(\sqrt{n}) + \log n$ mediante una sostituzione di variabili. Dovreste ottenere una soluzione asintoticamente stretta. Non preoccupatevi se i valori non sono interi.

4.4 Il metodo dell'albero di ricorsione per risolvere le ricorrenze

Sebbene il metodo di sostituzione possa fornire una prova succinta della correttezza di una soluzione di una ricorrenza, a volte è difficile formulare una buona ipotesi per la soluzione. Disegnare un albero di ricorsione, come abbiamo fatto nella nostra analisi della ricorrenza di merge sort nel Paragrafo 2.3.2, è una tecnica semplice per ideare una buona ipotesi. In un ***albero di ricorsione*** ogni nodo rappresenta il costo di un singolo sottoproblema da qualche parte nell'insieme delle chiamate ricorsive di funzione. Sommiamo i costi all'interno di ogni livello dell'albero per ottenere un insieme di costi per livello; poi sommiamo tutti i costi per livello per determinare il costo totale di tutti i livelli della ricorsione.

Un albero di ricorsione è un ottimo metodo per ottenere una buona ipotesi, che poi viene verificata con il metodo di sostituzione. Quando si usa un albero di ricorsione per generare una buona ipotesi, spesso si tollera una certa dose di "approssimazione", in quanto l'ipotesi sarà verificata in un secondo momento. Tuttavia, se prestate particolare attenzione quando create l'albero di ricorsione e sommate i costi, potete usare l'albero di ricorsione come prova diretta di una soluzione della ricorrenza. In questo paragrafo, utilizzeremo l'albero di ricorsione per generare buone ipotesi; nel Paragrafo 4.6, utilizzeremo gli alberi di ricorsione direttamente per dimostrare il teorema che forma la base del metodo dell'esperto.

Acquistato da Michele Michele su Webster il 2022-01-07 23:12 Numero Ordine Libreria: 19503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

Per esempio, vediamo come un albero di ricorsione possa fornire una buona ipotesi per la ricorrenza $T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2)$. Iniziamo a ricercare un limite superiore per la soluzione. Poiché sappiamo che floor e ceiling di solito non influiscono sulla risoluzione delle ricorrenze (ecco un esempio di approssimazione che possiamo tollerare), creiamo un albero di ricorsione per la ricorrenza $T(n) = 3T(n/4) + cn^2$, ricordando che il coefficiente $c > 0$ è costante.

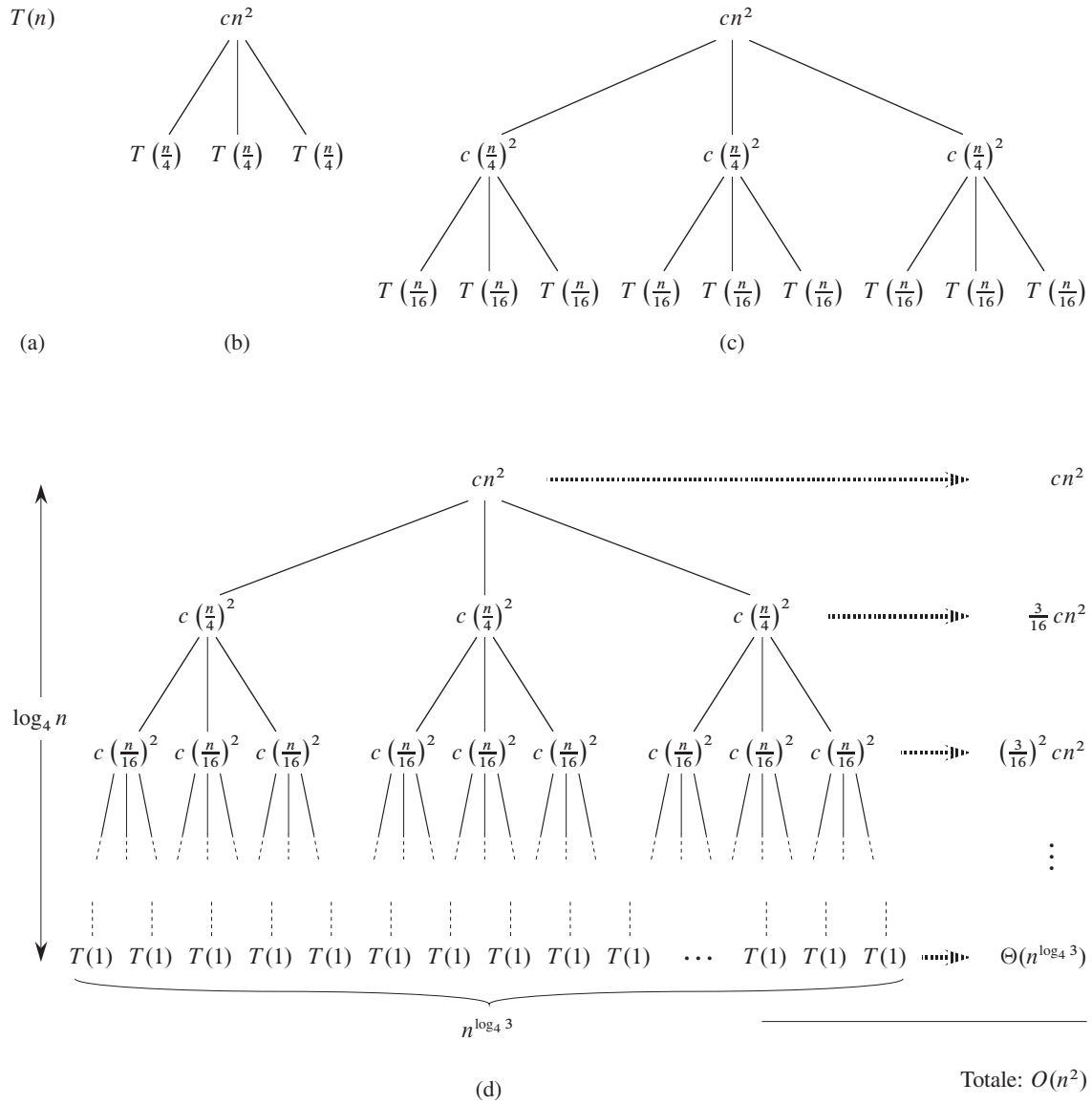


Figura 4.5 La costruzione di un albero di ricorsione per la ricorrenza $T(n) = 3T(n/4) + cn^2$. La parte **(a)** mostra $T(n)$, che viene progressivamente espanso nelle parti **(b)–(d)** per formare l'albero di ricorsione. L'albero completamente espanso nella parte (d) ha un'altezza $\log_4 n$ (con un numero di livelli pari a $\log_4 n + 1$).

Nella Figura 4.5 è illustrata la derivazione dell'albero di ricorsione per $T(n) = 3T(n/4) + cn^2$. Per comodità, supponiamo che n sia una potenza esatta di 4 (altro esempio di approssimazione tollerabile) in modo che le dimensioni di tutti i sottoproblemi siano intere. La parte (a) della figura mostra $T(n)$, che viene espanso nella parte (b) in un albero equivalente che rappresenta la ricorrenza. Il termine cn^2 nella radice rappresenta il costo al livello più alto della ricorsione; i tre sottoalberi della radice rappresentano i costi richiesti dai sottoproblemi di dimensione $n/4$. La parte (c) mostra il passo successivo di questo processo, dove vengono espansi i nodi di costo $T(n/4)$ della parte (b). Il costo di ogni nodo figlio della radice è $c(n/4)^2$. Continuiamo a espandere ogni nodo dell'albero, suddividendolo nelle sue parti costituenti, come stabilisce la ricorrenza.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

Poiché le dimensioni dei sottoproblemi diminuiscono di un fattore 4 ogni volta che scendiamo di un livello, alla fine dovremo raggiungere una condizione al contorno. A quale distanza dalla radice ne troveremo una? La dimensione del sottoproblema per un nodo alla profondità i è $n/4^i$. Quindi, la dimensione del sottoproblema diventa $n = 1$ quando $n/4^i = 1$ ovvero quando $i = \log_4 n$. Dunque, l'albero ha $\log_4 n + 1$ livelli (alle profondità $0, 1, 2, \ldots, \log_4 n$).

Adesso determiniamo il costo a ogni livello dell'albero. Ogni livello ha tre volte i nodi del livello precedente; quindi il numero di nodi alla profondità i è 3^i. Poiché le dimensioni dei sottoproblemi diminuiscono di un fattore 4 ogni volta che si scende di un livello rispetto alla radice, ogni nodo alla profondità i (per $i = 0, 1, 2, \ldots, \log_4 n - 1$) ha un costo di $c(n/4^i)^2$. Moltiplicando, vediamo che il costo totale di tutti i nodi alla profondità i (per $i = 0, 1, 2, \ldots, \log_4 n - 1$) è $3^i c(n/4^i)^2 = (3/16)^i cn^2$. L'ultimo livello, alla profondità $\log_4 n$, ha $3^{\log_4 n} = n^{\log_4 3}$ nodi, ciascuno con un costo $T(1)$, per un costo totale pari a $n^{\log_4 3} T(1)$, che è $\Theta(n^{\log_4 3})$, in quanto supponiamo che $T(1)$ sia una costante.

A questo punto, sommiamo i costi di tutti i livelli per determinare il costo dell'albero intero:

$$\begin{aligned}
T(n) &= cn^2 + \frac{3}{16}cn^2 + \left(\frac{3}{16}\right)^2 cn^2 + \cdots + \left(\frac{3}{16}\right)^{\log_4 n - 1} cn^2 + \Theta(n^{\log_4 3}) \\
&= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\
&= \frac{(3/16)^{\log_4 n} - 1}{(3/16) - 1} cn^2 + \Theta(n^{\log_4 3}) \qquad \text{(per l'equazione (A.5))}
\end{aligned}$$

Quest'ultima formula si presenta alquanto complicata, finché non realizziamo che possiamo di nuovo tollerare una piccola dose di approssimazione e usare come limite superiore una serie geometrica decrescente infinita. Facendo un passo indietro e applicando l'equazione (A.6), otteniamo

$$\begin{aligned}
T(n) &= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\
&< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\
&= \frac{1}{1 - (3/16)} cn^2 + \Theta(n^{\log_4 3}) \\
&= \frac{16}{13} cn^2 + \Theta(n^{\log_4 3}) \\
&= O(n^2)
\end{aligned}$$

Quindi, abbiamo ricavato l'ipotesi $T(n) = O(n^2)$ per la nostra ricorrenza originale $T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2)$. In questo esempio, i coefficienti di cn^2 formano una serie geometrica decrescente e, per l'equazione (A.6), la somma di questi coefficienti è limitata superiormente della costante $16/13$. Poiché il contributo della radice al costo totale è cn^2, la radice contribuisce con una frazione costante del costo totale. In altre parole, il costo totale dell'albero è dominato dal costo della radice.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria 1899303016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

In effetti, se $O(n^2)$ è davvero un limite superiore per la ricorrenza (come verificheremo subito), allora deve essere un limite stretto. Perché? Perché la prima chiamata ricorsiva contribuisce con un costo $\Theta(n^2)$, quindi $\Omega(n^2)$ deve essere un limite inferiore per la ricorrenza.

Adesso possiamo usare il metodo di sostituzione per verificare che la nostra ipotesi era corretta, ovvero $T(n) = O(n^2)$ è un limite superiore per la ricorrenza $T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2)$. Intendiamo dimostrare che $T(n) \leq dn^2$ per qualche costante $d > 0$. Utilizzando la stessa costante $c > 0$ di prima, otteniamo

$$\begin{aligned} T(n) &\leq 3T(\lfloor n/4 \rfloor) + cn^2 \\ &\leq 3d \lfloor n/4 \rfloor^2 + cn^2 \\ &\leq 3d(n/4)^2 + cn^2 \\ &= \frac{3}{16} dn^2 + cn^2 \\ &\leq dn^2 \end{aligned}$$

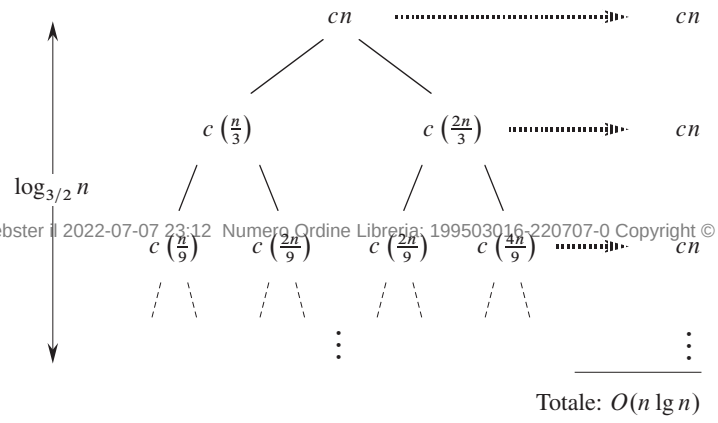
L'ultimo passaggio è vero quando $d \geq (16/13)c$.

Come altro esempio, più complicato, esaminate la Figura 4.6 che illustra l'albero di ricorsione per

$$T(n) = T(n/3) + T(2n/3) + O(n)$$

Anche qui, per semplificare, abbiamo omesso le funzioni floor e ceiling. Come in precedenza, c è il fattore costante nel termine $O(n)$. Quando sommiamo i valori nei singoli livelli dell'albero di ricorsione, otteniamo un valore pari a cn per ogni livello. Il più lungo cammino semplice dalla radice a una foglia è $n \to (2/3)n \to (2/3)^2 n \to \cdots \to 1$. Poiché $(2/3)^k n = 1$ quando $k = \log_{3/2} n$, l'altezza dell'albero è $\log_{3/2} n$.

Intuitivamente, prevediamo che la soluzione della ricorrenza sia al massimo pari al numero di livelli per il costo di ciascun livello, ovvero $O(cn \log_{3/2} n) = O(n \lg n)$. Tuttavia, la Figura 4.6 mostra soltanto i livelli superiori dell'albero di ricorsione, e non tutti i livelli dell'albero contribuiscono con un costo cn. Consideriamo il costo delle foglie. Se questo albero di ricorsione fosse un albero binario completo di altezza $\log_{3/2} n$, ci sarebbero $2^{\log_{3/2} n} = n^{\log_{3/2} 2}$ foglie. Poiché il costo di ogni foglia è una costante, il costo totale di tutte le foglie sarebbe $\Theta(n^{\log_{3/2} 2})$, che è $\omega(n \lg n)$, poiché $\log_{3/2} 2$ è una costante strettamente maggiore di 1. Tuttavia, questo albero di ricorsione non è un albero binario completo, pertanto ha



Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Figura 4.6 Un albero di ricorsione per la ricorrenza $T(n) = T(n/3) + T(2n/3) + cn$.

meno di $n^{\log_{3/2} 2}$ foglie. Inoltre, via via che si scende dalla radice, mancano sempre più nodi interni. Di conseguenza, non tutti i livelli contribuiscono esattamente con un costo cn; i livelli più bassi contribuiscono in misura minore. Potremmo calcolare con precisione tutti i costi, ma vi ricordiamo che stiamo semplicemente tentando di trovare un'ipotesi da utilizzare nel metodo di sostituzione. Accettiamo quindi l'approssimazione e proviamo a dimostrare che l'ipotesi $O(n \lg n)$ per il limite superiore è corretta. In effetti, possiamo applicare il metodo di sostituzione per verificare che $O(n \lg n)$ è un limite superiore per la soluzione della ricorrenza. Dimostriamo che $T(n) \leq dn \lg n$, dove d è un'opportuna costante positiva. Per $d \geq c/(\lg 3 - (2/3))$, abbiamo che

$$\begin{aligned}
T(n) &\leq T(n/3) + T(2n/3) + cn \\
&\leq d(n/3)\lg(n/3) + d(2n/3)\lg(2n/3) + cn \\
&= (d(n/3)\lg n - d(n/3)\lg 3) \\
&\qquad + (d(2n/3)\lg n - d(2n/3)\lg(3/2)) + cn \\
&= dn\lg n - d((n/3)\lg 3 + (2n/3)\lg(3/2)) + cn \\
&= dn\lg n - d((n/3)\lg 3 + (2n/3)\lg 3 - (2n/3)\lg 2) + cn \\
&= dn\lg n - dn(\lg 3 - 2/3) + cn \\
&\leq dn\lg n
\end{aligned}$$

Quindi, non è necessario svolgere un calcolo più accurato dei costi nell'albero di ricorsione.

Esercizi

4.4-1

Utilizzate un albero di ricorsione per determinare un buon limite superiore asintotico per la ricorrenza $T(n) = 3T(\lfloor n/2 \rfloor) + n$. Applicate il metodo di sostituzione per verificare la vostra soluzione.

4.4-2

Utilizzate un albero di ricorsione per determinare un buon limite superiore asintotico per la ricorrenza $T(n) = T(n/2) + n^2$. Applicate il metodo di sostituzione per verificare la vostra soluzione.

4.4-3

Utilizzate un albero di ricorsione per determinare un buon limite superiore asintotico per la ricorrenza $T(n) = 4T(n/2+2)+n$. Applicate il metodo di sostituzione per verificare la vostra soluzione.

4.4-4

Utilizzate un albero di ricorsione per determinare un buon limite superiore asintotico per la ricorrenza $T(n) = 2T(n-1) + 1$. Applicate il metodo di sostituzione per verificare la vostra soluzione.

Acquistato da Michele Michele su Webster il 2022-07-23 3:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

4.4-5

Utilizzate un albero di ricorsione per determinare un buon limite superiore asintotico per la ricorrenza $T(n) = T(n-1) + T(n/2) + n$. Applicate il metodo di sostituzione per verificare la vostra soluzione.

4.4-6

Utilizzando un albero di ricorsione, dimostrate che la soluzione della ricorrenza $T(n) = T(n/3) + T(2n/3) + cn$, dove c è una costante, è $\Omega(n \lg n)$.

4.4-7
Disegnate l'albero di ricorsione per $T(n) = 4T(\lfloor n/2 \rfloor) + cn$, dove c è una costante, e determinate un limite asintotico stretto per la sua soluzione. Verificate il limite ottenuto con il metodo di sostituzione.

4.4-8
Utilizzate un albero di ricorsione per trovare una soluzione asintoticamente stretta della ricorrenza $T(n) = T(n-a) + T(a) + cn$, dove $a \geq 1$ e $c > 0$ sono costanti.

4.4-9
Utilizzate un albero di ricorsione per trovare una soluzione asintoticamente stretta della ricorrenza $T(n) = T(\alpha n) + T((1-\alpha)n) + cn$, dove α è una costante nell'intervallo $0 < \alpha < 1$ e $c > 0$ è un'altra costante.

4.5 Il metodo dell'esperto per risolvere le ricorrenze

Il metodo dell'esperto rappresenta un "ricettario" per risolvere ricorrenze della forma

$$T(n) = aT(n/b) + f(n) \tag{4.20}$$

dove $a \geq 1$ e $b > 1$ sono costanti e $f(n)$ è una funzione asintoticamente positiva. Il metodo dell'esperto richiede la memorizzazione di tre casi, ma poi la soluzione di molte ricorrenze può essere facilmente determinata, spesso senza carta e penna.

La ricorrenza (4.20) descrive il tempo di esecuzione di un algoritmo che divide un problema di dimensione n in a sottoproblemi, ciascuno di dimensione n/b, dove a e b sono costanti positive. I sottoproblemi vengono risolti in modo ricorsivo, ciascuno nel tempo $T(n/b)$. La funzione $f(n)$ comprende il costo per dividere il problema e combinare i risultati dei sottoproblemi. Per esempio, la ricorrenza che risulta dall'algoritmo di Strassen ha $a = 7$, $b = 2$ e $f(n) = \Theta(n^2)$.

Tecnicamente parlando, la ricorrenza non è effettivamente ben definita perché n/b potrebbe non essere un intero. Tuttavia, la sostituzione di ciascuno degli a termini $T(n/b)$ con $T(\lfloor n/b \rfloor)$ o $T(\lceil n/b \rceil)$ non influisce sul comportamento asintotico della ricorrenza (questo sarà dimostrato nel prossimo paragrafo). Pertanto, di solito, omettiamo per comodità le funzioni floor e ceiling quando scriviamo ricorrenze di divide et impera di questa forma.

Il teorema dell'esperto

Il metodo dell'esperto dipende dal seguente teorema.

Teorema 4.1 (Teorema dell'esperto)
Date le costanti $a \geq 1$ e $b > 1$ e la funzione $f(n)$, sia $T(n)$ una funzione definita sugli interi non negativi dalla ricorrenza

$$T(n) = aT(n/b) + f(n)$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

dove n/b rappresenta $\lfloor n/b \rfloor$ o $\lceil n/b \rceil$. Allora $T(n)$ può essere asintoticamente limitata nei seguenti modi:

1. Se $f(n) = O(n^{\log_b a - \epsilon})$ per qualche costante $\epsilon > 0$, allora $T(n) = \Theta(n^{\log_b a})$.
2. Se $f(n) = \Theta(n^{\log_b a})$, allora $T(n) = \Theta(n^{\log_b a} \lg n)$.

3. Se $f(n) = \Omega(n^{\log_b a+\epsilon})$ per qualche costante $\epsilon > 0$ e se $af(n/b) \leq cf(n)$ per qualche costante $c < 1$ e per ogni n sufficientemente grande, allora $T(n) = \Theta(f(n))$. ■

Prima di applicare il teorema dell'esperto a qualche esempio, cerchiamo di capire che cosa dice. In ciascuno dei tre casi, confrontiamo la funzione $f(n)$ con la funzione $n^{\log_b a}$. Intuitivamente, la soluzione della ricorrenza è determinata dalla più grande delle due funzioni. Se, come nel caso 1, la funzione $n^{\log_b a}$ è la più grande, allora la soluzione è $T(n) = \Theta(n^{\log_b a})$. Se, come nel caso 3, la funzione $f(n)$ è la più grande, allora la soluzione è $T(n) = \Theta(f(n))$. Se, come nel caso 2, le due funzioni hanno la stessa dimensione, moltiplichiamo per un fattore logaritmico e la soluzione è $T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(f(n) \lg n)$.

Oltre a questo concetto intuitivo, ci sono alcuni dettagli tecnici da capire. Nel primo caso, $f(n)$ non soltanto deve essere più piccola di $n^{\log_b a}$, ma deve essere *polinomialmente* più piccola; ovvero, $f(n)$ deve essere asintoticamente più piccola di $n^{\log_b a}$ per un fattore n^{ϵ} per qualche costante $\epsilon > 0$. Nel terzo caso, $f(n)$ non soltanto deve essere più grande di $n^{\log_b a}$, ma deve essere polinomialmente più grande e soddisfare anche la condizione di "regolarità" $af(n/b) \leq cf(n)$. Questa condizione è soddisfatta dalla maggior parte delle funzioni polinomialmente limitate che incontreremo.

È importante capire che i tre casi non coprono tutte le funzioni possibili $f(n)$. C'è un intervallo fra i casi 1 e 2 in cui $f(n)$ è minore di $n^{\log_b a}$, ma non in modo polinomiale. Analogamente, c'è un intervallo fra i casi 2 e 3 in cui $f(n)$ è maggiore di $n^{\log_b a}$, ma non in modo polinomiale. Se la funzione $f(n)$ ricade in uno di questi intervalli o se la condizione di regolarità nel caso 3 non è soddisfatta, il metodo dell'esperto non può essere usato per risolvere la ricorrenza.

Applicazione del metodo dell'esperto

Per utilizzare il metodo dell'esperto, determiniamo semplicemente quale caso (se esiste) del teorema dell'esperto possiamo applicare e scriviamo la soluzione. Come primo esempio, consideriamo

$$T(n) = 9T(n/3) + n$$

Per questa ricorrenza abbiamo $a = 9$, $b = 3$, $f(n) = n$ e quindi $n^{\log_b a} = n^{\log_3 9} = \Theta(n^2)$. Poiché $f(n) = O(n^{\log_3 9-\epsilon})$, dove $\epsilon = 1$, possiamo applicare il caso 1 del teorema dell'esperto e concludere che la soluzione è $T(n) = \Theta(n^2)$.

Adesso consideriamo

$$T(n) = T(2n/3) + 1$$

dove $a = 1$, $b = 3/2$, $f(n) = 1$ e $n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$. Si applica il caso 2, in quanto $f(n) = \Theta(n^{\log_b a}) = \Theta(1)$ e, quindi, la soluzione della ricorrenza è $T(n) = \Theta(\lg n)$.

Nella ricorrenza

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$T(n) = 3T(n/4) + n \lg n$$

abbiamo $a = 3$, $b = 4$, $f(n) = n \lg n$ e $n^{\log_b a} = n^{\log_4 3} = O(n^{0,793})$. Poiché $f(n) = \Omega(n^{\log_4 3+\epsilon})$, dove $\epsilon \approx 0.2$, si applica il caso 3 se riusciamo a dimostrare che $f(n)$ soddisfa la condizione di regolarità. Per n sufficientemente grande, $af(n/b) = 3(n/4) \lg(n/4) \leq (3/4)n \lg n = cf(n)$ per $c = 3/4$. Di conseguenza, per il caso 3, la soluzione della ricorrenza è $T(n) = \Theta(n \lg n)$.

Il metodo dell'esperto non si applica alla ricorrenza

$$T(n) = 2T(n/2) + n \lg n$$

anche se ha la forma appropriata: $a = 2$, $b = 2$, $f(n) = n \lg n$ e $n^{\log_b a} = n$. Potremmo erroneamente ritenere che si possa applicare il caso 3, in quanto $f(n) = n \lg n$ è asintoticamente più grande di $n^{\log_b a} = n$; il problema è che non è *polinomialmente* più grande. Il rapporto $f(n)/n^{\log_b a} = (n \lg n)/n = \lg n$ è asintoticamente minore di n^ϵ per qualsiasi costante positiva ϵ. Quindi, la ricorrenza ricade nell'intervallo fra i casi 2 e 3 (vedere l'Esercizio 4.6-2 per una soluzione).

Utilizziamo ora il metodo dell'esperto per risolvere le ricorrenze incontrate nei Paragrafi 4.1 e 4.2.

La ricorrenza (4.7)

$$T(n) = 2T(n/2) + \Theta(n)$$

caratterizza i tempi di esecuzione dell'algoritmo divide et impera sia per il problema del massimo sottoarray sia per merge sort. (Secondo la nostra prassi abituale, omettiamo la definizione del caso base nella ricorrenza.) Qui abbiamo $a = 2$, $b = 2$, $f(n) = \Theta(n)$ e, quindi, $n^{\log_b a} = n^{\log_2 2} = n$. Si applica il caso 2, in quanto $f(n) = \Theta(n)$; pertanto la soluzione è $T(n) = \Theta(n \lg n)$.

La ricorrenza (4.17)

$$T(n) = 8T(n/2) + \Theta(n^2)$$

descrive il tempo di esecuzione del primo algoritmo divide et impera che abbiamo visto per il prodotto delle matrici. Adesso abbiamo $a = 8$, $b = 2$ e $f(n) = \Theta(n^2)$; quindi $n^{\log_b a} = n^{\log_2 8} = n^3$. Poiché n^3 è polinomialmente più grande di $f(n)$ (ovvero $f(n) = O(n^{3-\epsilon})$ per $\epsilon = 1$), si applica il caso 1, e $T(n) = \Theta(n^3)$.

Infine, consideriamo la ricorrenza (4.18)

$$T(n) = 7T(n/2) + \Theta(n^2)$$

che descrive il tempo di esecuzione dell'algoritmo di Strassen. Qui abbiamo $a = 7$, $b = 2$, $f(n) = \Theta(n^2)$ e, quindi, $n^{\log_b a} = n^{\log_2 7}$. Riscrivendo $\log_2 7$ nella forma $\lg 7$ e ricordando che $2.80 < \lg 7 < 2.81$, notiamo che $f(n) = O(n^{\lg 7 - \epsilon})$ per $\epsilon = 0.8$. Si applica ancora il caso 1, e la soluzione è $T(n) = \Theta(n^{\lg 7})$.

Esercizi

4.5-1

Applicare il metodo dell'esperto per determinare i limiti asintotici stretti per le seguenti ricorrenze:

a. $T(n) = 2T(n/4) + 1$

b. $T(n) = 2T(n/4) + \sqrt{n}$

c. $T(n) = 2T(n/4) + n$

d. $T(n) = 2T(n/4) + n^2$

Acquistato da Michele Michelangeli, Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

4.5-2

Il professor Caesar intende sviluppare un algoritmo per il prodotto di matrici che sia asintoticamente più veloce dell'algoritmo di Strassen. Il suo algoritmo utilizzerà il metodo divide et impera, dividendo ciascuna matrice in pezzi di dimensione $n/4 \times n/4$ e i passi divide e combina insieme richiederanno un tempo $\Theta(n^2)$.

Ha bisogno di determinare quanti sottoproblemi dovrà creare il suo algoritmo per battere l'algoritmo di Strassen. Se il suo algoritmo crea a sottoproblemi, allora la ricorrenza per il tempo di esecuzione $T(n)$ diventa $T(n) = aT(n/4) + \Theta(n^2)$. Qual è il più grande valore intero di a che rende l'algoritmo del professor Caesar asintoticamente più veloce dell'algoritmo di Strassen?

4.5-3
Applicate il metodo dell'esperto per dimostrare che la soluzione della ricorrenza $T(n) = T(n/2) + \Theta(1)$ della ricerca binaria è $T(n) = \Theta(\lg n)$ (la ricerca binaria è descritta nell'Esercizio 2.3-5).

4.5-4
Il metodo dell'esperto può essere applicato alla ricorrenza $T(n) = 4T(n/2) + n^2 \lg n$? Perché o perché no? Determinate un limite asintotico superiore per questa ricorrenza.

4.5-5 ⋆
Considerate la condizione di regolarità $af(n/b) \leq cf(n)$ per qualche costante $c < 1$, che è parte del caso 3 del teorema dell'esperto. Trovate due costanti $a \geq 1$ e $b > 1$ e una funzione $f(n)$ che soddisfa tutte le condizioni del caso 3 del teorema dell'esperto, tranne quella di regolarità.

⋆ 4.6 Dimostrazione del teorema dell'esperto

Questo paragrafo contiene una dimostrazione del teorema dell'esperto (Teorema 4.1). Non occorre comprendere la dimostrazione per applicare il teorema.

La dimostrazione si divide in due parti. La prima parte analizza la ricorrenza "principale" (4.20), sotto l'ipotesi semplificativa che $T(n)$ sia definita soltanto con potenze esatte di $b > 1$, ovvero per $n = 1, b, b^2, \ldots$. Questa parte fornisce tutta l'intuizione necessaria per capire perché il teorema dell'esperto è vero. La seconda parte mostra come l'analisi possa essere estesa a tutti gli interi positivi n; essa applica una tecnica matematica per trattare il problema della gestione di floor e ceiling.

In questo paragrafo, a volte, abuseremo un po' della nostra notazione asintotica utilizzandola per descrivere il comportamento di funzioni che sono definite soltanto con potenze esatte di b. Ricordiamo che le definizioni delle notazioni asintotiche richiedono che i limiti siano dimostrati per tutti i numeri sufficientemente grandi, non solo per quelli che sono potenze di b. Dal momento che potremmo definire nuove notazioni asintotiche che si applicano all'insieme $\{b^i : i = 0, 1, \ldots\}$, anziché agli interi non negativi, questo è un abuso minore.

Nonostante ciò, dobbiamo sempre stare in guardia quando utilizziamo la notazione asintotica su un dominio limitato per non incorrere in conclusioni improprie. Per esempio, dimostrare che $T(n) = O(n)$ quando n è una potenza esatta di 2 non garantisce che $T(n) = O(n)$. La funzione $T(n)$ potrebbe essere definita in questo modo

$$T(n) = \begin{cases} n & \text{se } n = 1, 2, 4, 8, \ldots \\ n^2 & \text{negli altri casi} \end{cases}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 - Numero Unico Libreria: 1995030016-2287077 - Copyright © 2022, McGraw-Hill Education (Italy)

nel qual caso il limite superiore migliore che possiamo dimostrare è $T(n) = O(n^2)$. A causa di questo genere di conseguenze drastiche, non useremo mai la notazione asintotica su un dominio limitato, a meno che non sia assolutamente chiaro dal contesto che lo stiamo facendo.

4.6.1 La dimostrazione per le potenze esatte

La prima parte della dimostrazione del teorema dell'esperto analizza la ricorrenza (4.20)

$$T(n) = aT(n/b) + f(n)$$

per il metodo dell'esperto, nell'ipotesi che n sia una potenza esatta di $b > 1$, dove b non deve essere necessariamente un intero. L'analisi è suddivisa in tre lemmi. Il primo lemma riduce il problema di risolvere la ricorrenza principale al problema di valutare un'espressione che contiene una sommatoria. Il secondo lemma determina alcuni limiti per questa sommatoria. Il terzo lemma riunisce i primi due per dimostrare una versione del teorema dell'esperto nel caso in cui n sia una potenza esatta di b.

Lemma 4.2
Siano $a \geq 1$ e $b > 1$ due costanti e $f(n)$ una funzione non negativa definita sulle potenze esatte di b. Se $T(n)$ è definita sulle potenze esatte di b dalla ricorrenza

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ aT(n/b) + f(n) & \text{se } n = b^i \end{cases}$$

dove i è un intero positivo, allora

$$T(n) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j) \tag{4.21}$$

Dimostrazione Utilizziamo l'albero di ricorsione nella Figura 4.7. La radice dell'albero ha costo $f(n)$ e ha a figli, ciascuno di costo $f(n/b)$. (È comodo pensare ad a come a un numero intero, specialmente quando rappresentiamo l'albero di ricorsione, anche se matematicamente ciò non sia richiesto.) Ciascuno di questi figli ha, a sua volta, a figli con un costo di $f(n/b^2)$; quindi ci sono a^2 nodi alla profondità 2. In generale, ci sono a^j nodi alla profondità j, ciascuno dei quali ha un costo di $f(n/b^j)$. Il costo di ogni foglia è $T(1) = \Theta(1)$ e ogni foglia si trova alla profondità $\log_b n$, in quanto $n/b^{\log_b n} = 1$. L'albero ha $a^{\log_b n} = n^{\log_b a}$ foglie.

Possiamo ottenere l'equazione (4.21) sommando i costi di ogni livello dell'albero, come illustra la figura. Il costo per un livello j di nodi interni è $a^j f(n/b^j)$; quindi il totale per tutti i livelli dei nodi interni è

Acquistato da Michele Michela su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\sum_{j=0}^{\log_b n - 1} a^j f(n/b^j)$$

Nel sottostante algoritmo divide et impera, questa somma rappresenta i costi per dividere i problemi in sottoproblemi e poi per ricombinare i sottoproblemi. Il costo di tutte le foglie, che è il costo per svolgere $n^{\log_b a}$ sottoproblemi di dimensione 1, è $\Theta(n^{\log_b a})$. ■

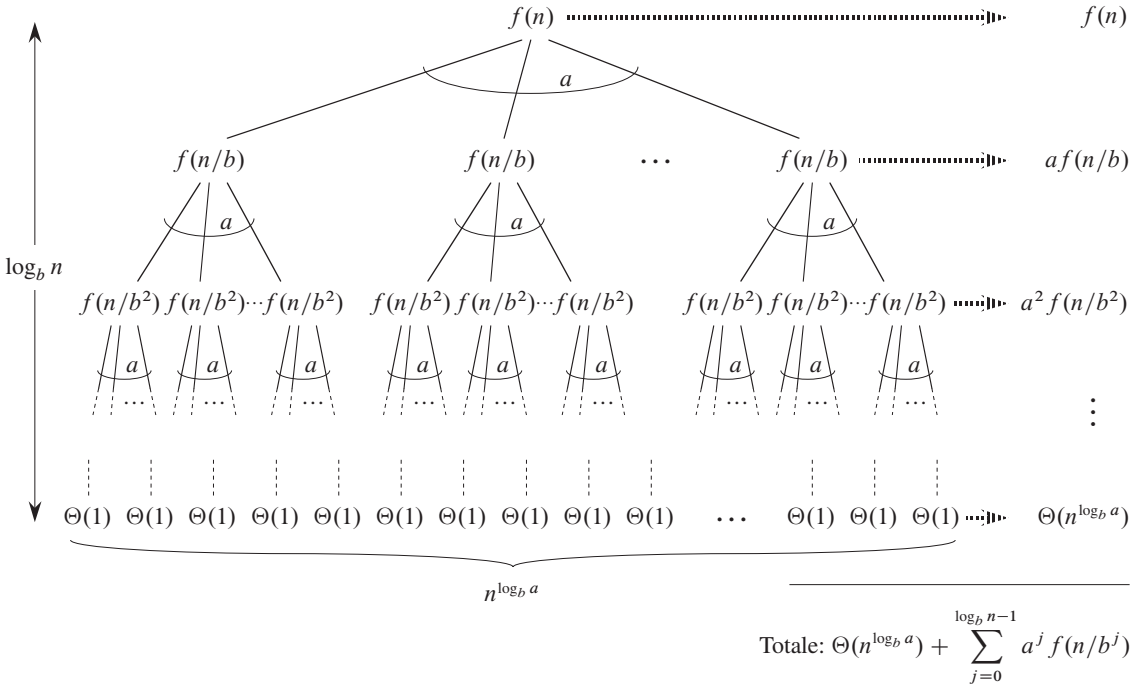


Figura 4.7 L'albero di ricorsione generato da $T(n) = aT(n/b) + f(n)$. È un albero a-ario completo con $n^{\log_b a}$ foglie e altezza $\log_b n$. Il costo dei nodi a ogni determinata profondità è indicato a destra e la loro somma è data dall'equazione (4.21).

Nei termini dell'albero di ricorsione, i tre casi del teorema dell'esperto corrispondono ai casi in cui il costo totale dell'albero è (1) dominato dai costi delle foglie, (2) equamente distribuito fra i livelli dell'albero o (3) dominato dal costo della radice.

La sommatoria nell'equazione (4.21) descrive il costo dei passi divide e combina del sottostante algoritmo divide et impera. Il prossimo lemma fornisce dei limiti asintotici sulla crescita della sommatoria.

Lemma 4.3
Siano $a \geq 1$ e $b > 1$ due costanti e $f(n)$ una funzione non negativa definita sulle potenze esatte di b. Una funzione $g(n)$ definita sulle potenze esatte di b da

$$g(n) = \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j) \tag{4.22}$$

ha i seguenti limiti asintotici per le potenze esatte di b.

1. Se $f(n) = O(n^{\log_b a - \epsilon})$ per qualche costante $\epsilon > 0$, allora $g(n) = O(n^{\log_b a})$.
2. Se $f(n) = \Theta(n^{\log_b a})$, allora $g(n) = \Theta(n^{\log_b a} \lg n)$.
3. Se $af(n/b) \leq cf(n)$ per qualche costante $c < 1$ e per ogni n sufficientemente grande, allora $g(n) = \Theta(f(n))$.

Acquistato da Michele Michele su Webster il 2022-07-07 Utente Ordine Libreria Piemme 2022-0000122005 Copyright © 2022, Ministero dell'Istruzione (MIM)

Dimostrazione Per il caso 1, abbiamo $f(n) = O(n^{\log_b a - \epsilon})$, che implica che $f(n/b^j) = O((n/b^j)^{\log_b a - \epsilon})$. Sostituendo nell'equazione (4.22), otteniamo

$$g(n) = O\left(\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \epsilon}\right) \tag{4.23}$$

Limitiamo la sommatoria all'interno della notazione O mettendo in evidenza i fattori comuni e semplificando; alla fine otteniamo una serie geometrica crescente:

$$\begin{aligned}\sum_{j=0}^{\log_b n-1} a^j \left(\frac{n}{b^j}\right)^{\log_b a-\epsilon} &= n^{\log_b a-\epsilon} \sum_{j=0}^{\log_b n-1} \left(\frac{ab^\epsilon}{b^{\log_b a}}\right)^j \\ &= n^{\log_b a-\epsilon} \sum_{j=0}^{\log_b n-1} (b^\epsilon)^j \\ &= n^{\log_b a-\epsilon} \left(\frac{b^{\epsilon \log_b n}-1}{b^\epsilon-1}\right) \\ &= n^{\log_b a-\epsilon} \left(\frac{n^\epsilon-1}{b^\epsilon-1}\right)\end{aligned}$$

Poiché b e ϵ sono costanti, possiamo riscrivere l'ultima espressione così:

$$n^{\log_b a-\epsilon} O(n^\epsilon) = O(n^{\log_b a})$$

Sostituendo la sommatoria dell'equazione (4.23) con questa espressione, si ha

$$g(n) = O(n^{\log_b a})$$

il che prova il caso 1.

Poiché il caso 2 assume $f(n) = \Theta(n^{\log_b a})$, si ha $f(n/b^j) = \Theta((n/b^j)^{\log_b a})$. Sostituendo nell'equazione (4.22), si ha

$$g(n) = \Theta\left(\sum_{j=0}^{\log_b n-1} a^j \left(\frac{n}{b^j}\right)^{\log_b a}\right) \tag{4.24}$$

Limitiamo la sommatoria all'interno della notazione Θ come nel caso 1; stavolta, però, non otteniamo una serie geometrica. Scopriamo invece che i termini della sommatoria sono tutti uguali:

$$\begin{aligned}\sum_{j=0}^{\log_b n-1} a^j \left(\frac{n}{b^j}\right)^{\log_b a} &= n^{\log_b a} \sum_{j=0}^{\log_b n-1} \left(\frac{a}{b^{\log_b a}}\right)^j \\ &= n^{\log_b a} \sum_{j=0}^{\log_b n-1} 1 \\ &= n^{\log_b a} \log_b n\end{aligned}$$

Sostituendo la sommatoria dell'equazione (4.24) con questa espressione, si ha

$$\begin{aligned}g(n) &= \Theta(n^{\log_b a} \log_b n) \\ &= \Theta(n^{\log_b a} \lg n)\end{aligned}$$

il che prova il caso 2.

Il caso 3 si dimostra in modo simile. Poiché $f(n)$ appare nella definizione (4.22) di $g(n)$ e tutti i termini di $g(n)$ sono non negativi, possiamo concludere che $g(n) = \Omega(f(n))$ per potenze esatte di b. Nell'enunciato del lemma abbiamo assunto $af(n/b) \le cf(n)$ per qualche costante $c < 1$ e per ogni n sufficientemente grande. Riscriviamo questa assunzione nella forma $f(n/b) \le (c/a) f(n)$ e la iteriamo j volte ottenendo $f(n/b^j) \le (c/a)^j f(n)$, ovvero $a^j f(n/b^j) \le c^j f(n)$, dove assumiamo che i valori su cui iteriamo siano sempre sufficientemente grandi. Siccome l'ultimo, e più piccolo, di questi valori è n/b^{j-1} basta assumere che n/b^{j-1} sia sufficientemente grande. Sostituendo nell'equazione (4.22)

Acquistato da Michele Michelassi (michelassi@gmail.com) Libraria, 1935020162a2107a0, Copyright © 2022 McGraw-Hill Education (Italy)

e semplificando, si ottiene una serie geometrica che, diversamente da quella del caso 1, è decrescente. Useremo un termine $O(1)$ per catturare i termini che non soddisfano l'ipotesi che n sia sufficientemente grande.

$$\begin{aligned} g(n) &= \sum_{j=0}^{\log_b n-1} a^j f(n/b^j) \\ &\leq \sum_{j=0}^{\log_b n-1} c^j f(n) + O(1) \\ &\leq f(n) \sum_{j=0}^{\infty} c^j + O(1) \\ &= f(n) \left(\frac{1}{1-c}\right) + O(1) \\ &= O(f(n)) \end{aligned}$$

L'ultimo passaggio è corretto in quanto c è costante. Pertanto possiamo concludere che $g(n) = \Theta(f(n))$ per potenze esatte di b. Il caso 3 è dimostrato e questo completa la dimostrazione del lemma. ■

Adesso possiamo dimostrare una versione del teorema dell'esperto per il caso in cui n è una potenza esatta di b.

Lemma 4.4
Siano $a \geq 1$ e $b > 1$ due costanti e $f(n)$ una funzione non negativa definita sulle potenze esatte di b. Se $T(n)$ è definita sulle potenze esatte di b dalla ricorrenza

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ aT(n/b) + f(n) & \text{se } n = b^i \end{cases}$$

dove i è un intero positivo, allora $T(n)$ ha i seguenti limiti asintotici per le potenze esatte di b.

1. Se $f(n) = O(n^{\log_b a - \epsilon})$ per qualche costante $\epsilon > 0$, allora $T(n) = \Theta(n^{\log_b a})$.
2. Se $f(n) = \Theta(n^{\log_b a})$, allora $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. Se $f(n) = \Omega(n^{\log_b a + \epsilon})$ per qualche costante $\epsilon > 0$ e se $af(n/b) \leq cf(n)$ per qualche costante $c < 1$ e per ogni n sufficientemente grande, allora $T(n) = \Theta(f(n))$.

Dimostrazione Usiamo il limiti del Lemma 4.3 per valutare la sommatoria (4.21) dal Lemma 4.2. Per il caso 1 abbiamo

$$\begin{aligned} T(n) &= \Theta(n^{\log_b a}) + O(n^{\log_b a}) \\ &= \Theta(n^{\log_b a}) \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Per il caso 2

$$\begin{aligned} T(n) &= \Theta(n^{\log_b a}) + \Theta(n^{\log_b a} \lg n) \\ &= \Theta(n^{\log_b a} \lg n) \end{aligned}$$

Per il caso 3

$$\begin{aligned} T(n) &= \Theta(n^{\log_b a}) + \Theta(f(n)) \\ &= \Theta(f(n)) \end{aligned}$$

in quanto $f(n) = \Omega(n^{\log_b a+\epsilon})$. ■

4.6.2 Floor e ceiling

Per completare la dimostrazione del teorema dell'esperto, a questo punto dobbiamo estendere la nostra analisi al caso in cui siano utilizzate le funzioni floor e ceiling nella ricorrenza del teorema dell'esperto, in modo che la ricorrenza sia definita per tutti i numeri interi, non soltanto per le potenze esatte di b. È semplice ottenere un limite inferiore per

$$T(n) = aT(\lceil n/b \rceil) + f(n) \tag{4.25}$$

e un limite superiore per

$$T(n) = aT(\lfloor n/b \rfloor) + f(n) \tag{4.26}$$

in quanto possiamo usare il limite $\lceil n/b \rceil \geq n/b$ nel primo caso per ottenere il risultato desiderato e il limite $\lfloor n/b \rfloor \leq n/b$ nel secondo caso. Per limitare inferiormente la ricorrenza (4.26) occorre pressoché la stessa tecnica per limitare superiormente la ricorrenza (4.25), quindi presenteremo soltanto quest'ultimo limite.

Modifichiamo l'albero di ricorsione della Figura 4.7 per generare l'albero di ricorsione illustrato nella Figura 4.8. Procedendo verso il basso nell'albero di ricorsione, otteniamo una sequenza di chiamate ricorsive con gli argomenti

$$\begin{aligned} &n \\ &\lceil n/b \rceil \\ &\lceil \lceil n/b \rceil /b \rceil \\ &\lceil \lceil \lceil n/b \rceil /b \rceil /b \rceil \\ &\quad\vdots \end{aligned}$$

Indichiamo con n_j il j-esimo elemento della sequenza, dove

$$n_j = \begin{cases} n & \text{se } j = 0 \\ \lceil n_{j-1}/b \rceil & \text{se } j > 0 \end{cases} \tag{4.27}$$

Il nostro primo obiettivo è determinare la profondità k alla quale n_k si riduce a una costante. Applicando la disuguaglianza $\lceil x \rceil \leq x + 1$, otteniamo

$$\begin{aligned} n_0 &\leq n \\ n_1 &\leq \frac{n}{b} + 1 \\ n_2 &\leq \frac{n}{b^2} + \frac{1}{b} + 1 \\ n_3 &\leq \frac{n}{b^3} + \frac{1}{b^2} + \frac{1}{b} + 1 \\ &\vdots \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

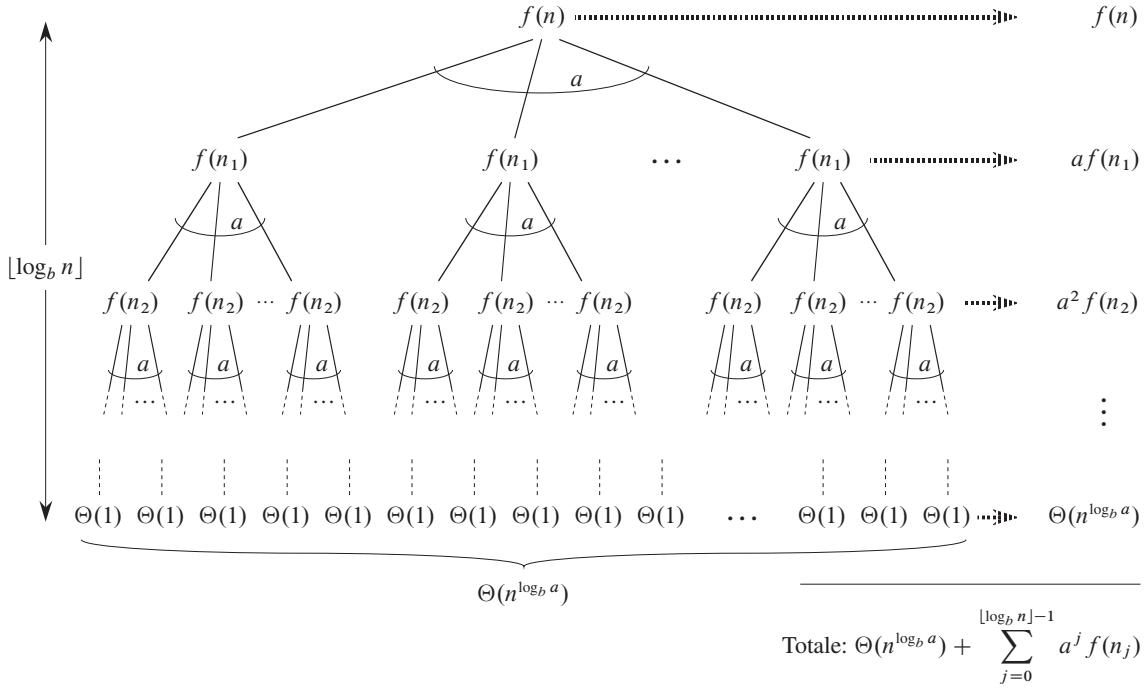


Figura 4.8 L'albero di ricorsione generato da $T(n) = aT(\lceil n/b \rceil) + f(n)$. L'argomento ricorsivo n_j è dato dall'equazione (4.27).

In generale,

$$\begin{aligned} n_j &\leq \frac{n}{b^j} + \sum_{i=0}^{j-1} \frac{1}{b^i} \\ &< \frac{n}{b^j} + \sum_{i=0}^{\infty} \frac{1}{b^i} \\ &= \frac{n}{b^j} + \frac{b}{b-1} \end{aligned}$$

Ponendo $j = \lfloor \log_b n \rfloor$, otteniamo

$$\begin{aligned} n_{\lfloor \log_b n \rfloor} &< \frac{n}{b^{\lfloor \log_b n \rfloor}} + \frac{b}{b-1} \\ &< \frac{n}{b^{\log_b n - 1}} + \frac{b}{b-1} \\ &= \frac{n}{n/b} + \frac{b}{b-1} \\ &= b + \frac{b}{b-1} \\ &= O(1) \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

e quindi alla profondità $\lfloor \log_b n \rfloor$, la dimensione del problema è al massimo una costante. Dalla Figura 4.8 otteniamo la relazione

$$T(n) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\lfloor \log_b n \rfloor - 1} a^j f(n_j) \tag{4.28}$$

che è pressoché uguale all'equazione (4.21), con la differenza che n è un numero intero arbitrario e non è vincolato a essere una potenza esatta di b.

Adesso possiamo calcolare la sommatoria

$$g(n) = \sum_{j=0}^{\lfloor \log_b n \rfloor - 1} a^j f(n_j) \tag{4.29}$$

dall'equazione (4.28) in modo analogo alla dimostrazione del Lemma 4.3. Iniziando dal caso 3, se $af(\lceil n/b \rceil) \le cf(n)$ per $n > b + b/(b-1)$, dove $c < 1$ è una costante, ne consegue che $a^j f(n_j) \le c^j f(n)$. Pertanto, la sommatoria nell'equazione (4.29) può essere calcolata proprio come nel Lemma 4.3. Nel caso 2 abbiamo $f(n) = \Theta(n^{\log_b a})$. Se dimostriamo che $f(n_j) = O(n^{\log_b a}/a^j) = O((n/b^j)^{\log_b a})$, allora potremo usare la dimostrazione del caso 2 del Lemma 4.3. Notate che $j \le \lfloor \log_b n \rfloor$ implica $b^j/n \le 1$. Il limite $f(n) = O(n^{\log_b a})$ implica che esiste una costante $c > 0$ tale che, per ogni n_j sufficientemente grande:

$$\begin{aligned}
f(n_j) &\le c\left(\frac{n}{b^j} + \frac{b}{b-1}\right)^{\log_b a} \\
&= c\left(\frac{n}{b^j}\left(1 + \frac{b^j}{n} \cdot \frac{b}{b-1}\right)\right)^{\log_b a} \\
&= c\left(\frac{n^{\log_b a}}{a^j}\right)\left(1 + \left(\frac{b^j}{n} \cdot \frac{b}{b-1}\right)\right)^{\log_b a} \\
&\le c\left(\frac{n^{\log_b a}}{a^j}\right)\left(1 + \frac{b}{b-1}\right)^{\log_b a} \\
&= O\left(\frac{n^{\log_b a}}{a^j}\right)
\end{aligned}$$

in quanto $c(1 + b/(b-1))^{\log_b a}$ è una costante. Quindi, il caso 2 è dimostrato. La dimostrazione del caso 1 è quasi identica. La chiave sta nel dimostrare il limite $f(n_j) = O(n^{\log_b a - \epsilon})$, che è simile alla corrispondente dimostrazione del caso 2, sebbene i calcoli algebrici siano più complicati.

Abbiamo così dimostrato i limiti superiori nel teorema dell'esperto per tutti i numeri interi n. La dimostrazione dei limiti inferiori è simile.

Esercizi

4.6-1 ⋆

Trovate un'espressione semplice ed esatta per n_j nell'equazione (4.27) per il caso in cui b sia un numero intero positivo, anziché un numero reale arbitrario.

4.6-2 ⋆

Dimostrate che, se $f(n) = \Theta(n^{\log_b a} \lg^k n)$, dove $k \ge 0$, la soluzione della ricorrenza del teorema dell'esperto è $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$. Per semplicità, limitate la vostra analisi alle potenze esatte di b.

4.6-3 ⋆

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Dimostrate che il caso 3 del teorema dell'esperto è sovradefinito, nel senso che la condizione di regolarità $af(n/b) \le cf(n)$ per qualche costante $c < 1$ implica che esiste una costante $\epsilon > 0$ tale che $f(n) = \Omega(n^{\log_b a + \epsilon})$.

Problemi

4-1 Esempi di ricorrenze

Trovate dei limiti superiori e inferiori per $T(n)$ in ciascuna delle seguenti ricorrenze. Supponete che $T(n)$ sia costante per $n \leq 2$. I limiti devono essere i più stretti possibili; spiegate le vostre soluzioni.

a. $T(n) = 2T(n/2) + n^4$

b. $T(n) = T(7n/10) + n$

c. $T(n) = 16T(n/4) + n^2$

d. $T(n) = 7T(n/3) + n^2$

e. $T(n) = 7T(n/2) + n^2$

f. $T(n) = 2T(n/4) + \sqrt{n}$

g. $T(n) = T(n-2) + n^2$

4-2 Il costo del passaggio dei parametri

In tutto il libro supporremo che il passaggio dei parametri durante le chiamate delle procedure richieda un tempo costante, anche quando viene passato un array di N elementi. Questa ipotesi è valida nella maggior parte dei sistemi, in quanto viene passato il puntatore all'array, non l'array. Questo problema esamina le implicazioni di tre strategie per passare i parametri:

1. Un array viene passato tramite un puntatore. Tempo $= \Theta(1)$.
2. Un array viene passato facendone una copia. Tempo $= \Theta(N)$, dove N è la dimensione dell'array.
3. Un array viene passato copiando soltanto la parte che potrebbe essere utilizzata dalla procedura chiamata. Tempo $= \Theta(q-p+1)$ se viene passato il sottoarray $A[p\,..\,q]$.

a. Considerate l'algoritmo ricorsivo di ricerca binaria per trovare un numero in un array ordinato (vedere l'Esercizio 2.3-5). Determinate le ricorrenze per i tempi di esecuzione nel caso peggiore della ricerca binaria quando gli array vengono passati utilizzando ciascuno dei tre metodi precedenti e trovate dei buoni limiti superiori per le soluzioni delle ricorrenze. Indicate con N la dimensione del problema originale e con n la dimensione di un sottoproblema.

b. Ripetete il punto (a) per l'algoritmo MERGE-SORT del Paragrafo 2.3.1.

4-3 Altri esempi di ricorrenze

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Trovate dei limiti asintotici superiori e inferiori per $T(n)$ in ciascuna delle seguenti ricorrenze. Supponete che $T(n)$ sia costante per n sufficientemente piccolo. I limiti devono essere i più stretti possibili; spiegate le vostre soluzioni.

a. $T(n) = 4T(n/3) + n \lg n$

b. $T(n) = 3T(n/3) + n/\lg n$

c. $T(n) = 4T(n/2) + n^2\sqrt{n}$

d. $T(n) = 3T(n/3 - 2) + n/2$

e. $T(n) = 2T(n/2) + n/\lg n$

f. $T(n) = T(n/2) + T(n/4) + T(n/8) + n$

g. $T(n) = T(n-1) + 1/n$

h. $T(n) = T(n-1) + \lg n$

i. $T(n) = T(n-2) + 1/\lg n$

j. $T(n) = \sqrt{n}T(\sqrt{n}) + n$

4-4 Numeri di Fibonacci

Questo problema sviluppa le proprietà dei numeri di Fibonacci, che sono definiti dalla ricorrenza (3.22). Utilizzeremo la tecnica delle funzioni generatrici per risolvere la ricorrenza di Fibonacci. La ***funzione generatrice*** (o ***serie di potenze formali***) $\mathcal{F}$ è definita in questo modo

$$\begin{aligned}\mathcal{F}(z) &= \sum_{i=0}^{\infty} F_i z^i \\ &= 0 + z + z^2 + 2z^3 + 3z^4 + 5z^5 + 8z^6 + 13z^7 + 21z^8 + \cdots\end{aligned}$$

Dove F_i è l'i-esimo numero di Fibonacci.

a. Dimostrate che $\mathcal{F}(z) = z + z\mathcal{F}(z) + z^2\mathcal{F}(z)$.

b. Dimostrate che

$$\begin{aligned}\mathcal{F}(z) &= \frac{z}{1 - z - z^2} \\ &= \frac{z}{(1 - \phi z)(1 - \hat{\phi} z)} \\ &= \frac{1}{\sqrt{5}}\left(\frac{1}{1 - \phi z} - \frac{1}{1 - \hat{\phi} z}\right)\end{aligned}$$

dove

$$\phi = \frac{1 + \sqrt{5}}{2} = 1,61803\ldots$$

e

$$\hat{\phi} = \frac{1 - \sqrt{5}}{2} = -0,61803\ldots$$

c. Dimostrate che

$$\mathcal{F}(z) = \sum_{i=0}^{\infty} \frac{1}{\sqrt{5}}(\phi^i - \hat{\phi}^i) z^i$$

d. Utilizzate il precedente punto per dimostrare che $F_i = \phi^i/\sqrt{5}$ per $i > 0$, arrotondato all'intero più vicino (*suggerimento:* notate che $|\hat{\phi}| < 1$).

Acquistato da Michele Michelangeli - Utilizzato il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

4-5 Collaudo dei chip

Il professor Diogene ha n chip di circuiti integrati, apparentemente identici, che in linea di principio sono capaci di collaudarsi a vicenda. L'apparecchiatura di prova può accogliere due chip alla volta. Dopo che l'apparecchiatura è stata avviata, ogni chip prova l'altro e indica se è buono o guasto. Un chip buono indica sempre con esattezza se l'altro chip è buono o guasto, ma l'indicazione di un chip guasto non è attendibile. Le quattro possibili indicazioni di un test sono le seguenti:

Chip A dice	Chip B dice	Conclusione
B è buono	A è buono	entrambi sono buoni o entrambi sono guasti
B è buono	A è guasto	almeno uno è guasto
B è guasto	A è buono	almeno uno è guasto
B è guasto	A è guasto	almeno uno è guasto

a. Dimostrate che se più di $n/2$ chip sono guasti, il professore non può determinare quali chip sono buoni applicando qualsiasi strategia basata su questo tipo di collaudo a coppie. Supponete che i chip guasti possano cospirare per ingannare il professore.

b. Considerate il problema di trovare un unico chip buono fra n chip, supponendo che più di $n/2$ chip siano buoni. Dimostrate che $\lfloor n/2 \rfloor$ collaudi a coppie sono sufficienti per ridurre il problema a uno di dimensione quasi dimezzata.

c. Dimostrate che i chip buoni possono essere identificati con $\Theta(n)$ collaudi a coppie, supponendo che più di $n/2$ chip siano buoni. Specificate e risolvete la ricorrenza che descrive il numero di collaudi.

4-6 Array di Monge

Un array A di $m \times n$ numeri reali è un ***array di Monge*** se, per ogni i, j, k e l tali che $1 \leq i < k \leq m$ e $1 \leq j < l \leq n$, si ha

$$A[i, j] + A[k, l] \leq A[i, l] + A[k, j]$$

In altre parole, ogni volta che scegliamo due righe e due colonne di un array di Monge e consideriamo i quattro elementi nelle intersezioni fra righe e colonne, la somma degli elementi superiore sinistro e inferiore destro è minore o uguale alla somma degli elementi inferiore sinistro e superiore destro. Ecco un esempio di array di Monge:

10	17	13	28	23
17	22	16	29	23
24	28	22	34	24
11	13	6	17	7
45	44	32	37	23
36	33	19	21	6
75	66	51	53	34

Acquistato da Michele Michele su Webster il 2022-07-07 Questo documento è protetto da copyright e ad uso esclusivo personale (vietata ogni forma di riproduzione e distribuzione)

a. Dimostrate che un array di Monge è tale se e soltanto se, per ogni $i = 1, 2, ..., m-1$ e $j = 1, 2, ..., n-1$, si ha

$$A[i, j] + A[i+1, j+1] \leq A[i, j+1] + A[i+1, j]$$

(*Suggerimento:* per la parte "se", usate l'induzione separatamente sulle righe e le colonne.)

b. Il seguente array non è un array di Monge; cambiate un elemento per trasformarlo in array di Monge (*suggerimento:* applicate il punto (a)).

37	23	22	32
21	6	7	10
53	34	30	31
32	13	9	6
43	21	15	8

c. Sia $f(i)$ l'indice della colonna che contiene il più piccolo elemento nella riga i (il più a sinistra se vi è più di un elemento minimo). Dimostrate che $f(1) \leq f(2) \leq \cdots \leq f(m)$ per qualsiasi array di Monge $m \times n$.

d. Ecco la descrizione di un algoritmo divide et impera che calcola il più piccolo elemento a sinistra in ogni riga di un array di Monge A di $m \times n$ elementi:

> Costruite una sottomatrice A' di A formata dalle righe di indice pari di A. Determinate ricorsivamente il più piccolo elemento a sinistra per ogni riga di A'. Poi calcolate il più piccolo elemento a sinistra nelle righe di indice dispari di A.

Spiegate come calcolare il più piccolo elemento a sinistra nelle righe di indice dispari di A (conoscendo il più piccolo elemento a sinistra nelle righe di indice pari) nel tempo $O(m + n)$.

e. Scrivete la ricorrenza che descrive il tempo di esecuzione dell'algoritmo presentato nel punto (d). Dimostrate che la sua soluzione è $O(m + n \log m)$.

Note

Divide et impera, come tecnica per progettare algoritmi, apparve per la prima volta in un articolo di Karatsuba e Ofman [195] del 1962. Tuttavia è probabile che sia stata utilizzata anche prima; secondo Heideman, Johnson e Burnus [164], C. F. Gauss scrisse nel 1805 il primo algoritmo della trasformata rapida di Fourier, e nella sua formulazione divideva il problema originale in sottoproblemi più piccoli le cui soluzioni venivano poi combinate assieme.

Il problema del massimo sottoarray, descritto nel Paragrafo 4.1, è una variante secondaria di un problema più generale studiato da Bentley [43, Capitolo 7].

L'algoritmo di Strassen [326] generò molto interesse quando venne pubblicato nel 1969. Prima di allora, pochi potevano immaginare che potesse esistere un algoritmo asintoticamente più veloce della procedura di base SQUARE-MATRIX-MULTIPLY. Da allora, il limite superiore asintotico per il prodotto di matrici è stato migliorato. Attualmente, l'algoritmo asintoticamente più efficiente per moltiplicare matrici $n \times n$, creato da Coppersmith e Winograd [79], ha un tempo di esecuzione di $O(n^{2.376})$. Il miglior limite inferiore è ovviamente $\Omega(n^2)$ (ovviamente perché occorre riempire n^2 elementi della matrice prodotto). Da un punto di vista pratico, l'algoritmo di Strassen non è il metodo migliore per moltiplicare le matrici, per le seguenti ragioni:

1. Il fattore costante nascosto nel tempo di esecuzione $\Theta(n^{\lg 7})$ dell'algoritmo di Strassen è più grande del fattore costante della procedura SQUARE-MATRIX-MULTIPLY con tempo $\Theta(n^3)$.
2. Quando le matrici sono sparse, i metodi appositamente progettati per queste matrici sono più rapidi.
3. L'algoritmo di Strassen non ha lo stesso livello di stabilità numerica del metodo semplice. In altre parole gli gli errori accumulati dall'algoritmo di Strassen, dovuti alla limitata precisione dell'aritmetica dei computer con valori non interi, sono maggiori di quelli accumulati da SQUARE-MATRIX-MULTIPLY.
4. Le sottomatrici che si formano nei livelli di ricorsione consumano spazio in memoria.

continua

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy).

Le ultime due ragioni sono state mitigate intorno al 1990. Higham [168] ha dimostrato che la differenza di stabilità numerica è stata sopravvalutata; sebbene l'algoritmo di Strassen sia numericamente troppo instabile per alcune applicazioni, tuttavia resta nei limiti accettabili per altre applicazioni. Bailey e altri [32] hanno descritto le tecniche per ridurre la quantità di memoria richiesta dall'algoritmo di Strassen. Nella pratica, le implementazioni veloci per moltiplicare matrici dense usano l'algoritmo di Strassen quando le dimensioni delle matrici sono oltre un livello di transizione (*crossover point*), poi ritornano al metodo semplice una volta che la dimensione dei sottoproblemi è scesa al di sotto del livello di transizione. Il valore esatto del livello di transizione dipende molto dal sistema. Le analisi che contano le operazioni, ma ignorano gli effetti della memoria cache e della pipeline, hanno prodotto bassi livelli di transizione, come $n = 8$ (Higham [168]) o $n = 12$ (Huss-Lederman e altri [187]).

D'Alberto e Nicolau [82] hanno sviluppato uno schema adattivo, che determina il punto di transizione eseguendo dei casi di prova (benchmark) quando viene installato il loro software. Hanno trovato il punto di transizione su vari sistemi che vanno da $n = 400$ a $n = 2150$; non sono riusciti a trovarlo soltanto per un paio di sistemi.

Le ricorrenze furono studiate già nel 1202 da L. Fibonacci, dal quale hanno preso il nome i numeri di Fibonacci. A. De Moivre introdusse il metodo delle funzioni generatrici (vedere il Problema 4-4) per risolvere le ricorrenze. Il metodo dell'esperto è stato adattato da Bentley, Haken e Saxe [44], che hanno descritto il metodo esteso presentato nell'Esercizio 4.6-2. Knuth [210] e Liu [238] spiegano come risolvere le ricorrenze lineari applicando il metodo delle funzioni generatrici. Purdom e Brown [288] e Graham, Knuth e Patashnik [153] trattano in modo esteso i metodi di risoluzione delle ricorrenze. Molti ricercatori, inclusi Akra e Bazzi [13], Roura [300], Verma [347] e Yap [361], hanno creato dei metodi per risolvere ricorrenze divide et impera più generali di quelle risolte dal metodo dell'esperto. Descriviamo qui il metodo ideato da Akra e Bazzi, nella versione modificata da Leighton [229]. Il metodo di Akra-Bazzi funziona per ricorrenze della forma

$$T(x) = \begin{cases} \Theta(1) & \text{se } 1 \le x \le x_0 \\ \sum_{i=1}^{k} a_i T(b_i x) + f(x) & \text{se } x > x_0 \end{cases} \tag{4.30}$$

dove

- $x \ge 1$ è un numero reale
- x_0 è una costante tale che $x_0 \ge 1/b_i$ e $x_0 \ge 1/(1 - b_i)$ per $i = 1, 2, \ldots, k$
- a_i è una costante positiva per $i = 1, 2, \ldots, k$
- b_i è una costante nell'intervallo $0 < b_i < 1$ per $i = 1, 2, \ldots, k$,
- $k \ge 1$ è una costante intera
- $f(x)$ e una funzione non negativa che soddisfa la ***condizione di crescita polinomiale***: esistono delle costanti positive c_1 e c_2 tali che per ogni $x \ge 1$, con $i = 1, 2, \ldots, k$, e per ogni u tale che $b_i x \le u \le x$, si ha $c_1 f(x) \le f(u) \le c_2 f(x)$. (Se $|f'(x)|$ è limitato superiormente da qualche polinomio in x, allora $f(x)$ soddisfa la condizione di crescita polinomiale. Per esempio, $f(x) = x^\alpha \lg^\beta x$ soddisfa questa condizione per qualsiasi costante reale α e β.)

Il metodo funziona anche su una ricorrenza quale $T(n) = T(\lfloor n/3 \rfloor) + T(\lfloor 2n/3 \rfloor) + O(n)$, a cui non si applica il metodo dell'esperto. Per risolvere la ricorrenza (4.30), troviamo prima il numero reale unico p tale che $\sum_{i=1}^{k} a_i b_i^p = 1$ (il valore di p esiste sempre). La soluzione della ricorrenza è quindi

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$T(n) = \Theta\left(x^p \left(1 + \int_1^x \frac{f(u)}{u^{p+1}}\, du\right)\right)$$

Il metodo di Akra-Bazzi potrebbe risultare difficile da usare, ma serve a risolvere le ricorrenze che modellano la divisione di un problema in sottoproblemi di dimensioni sostanzialmente differenti. Il metodo dell'esperto è più semplice da usare, ma si applica soltanto quando i sottoproblemi hanno la stessa dimensione.

Analisi probabilistica e algoritmi randomizzati

5

Questo capitolo introduce l'analisi probabilistica e gli algoritmi randomizzati. Se non avete dimestichezza con le basi della teoria delle probabilità, dovreste leggere l'Appendice C che tratta questi argomenti. L'analisi probabilistica e gli algoritmi randomizzati saranno utilizzati più volte in questo libro.

5.1 Il problema delle assunzioni

Supponete di dovere assumere un nuovo impiegato. Poiché i vostri precedenti tentativi di assumere un impiegato non hanno avuto successo, decidete di rivolgervi a un'agenzia di selezione del personale. L'agenzia vi invia un candidato al giorno. Venite a colloquio con questa persona e poi decidete se assumerla oppure no. Dovrete pagare all'agenzia un piccolo compenso per avere un colloquio con un candidato. L'assunzione effettiva di un candidato, invece, è un'operazione più costosa, perché dovrete licenziare l'attuale impiegato e pagare un consistente compenso all'agenzia. Intendete assumere la persona migliore possibile per il compito da svolgere. Di conseguenza, avete deciso che, dopo avere avuto un colloquio con un candidato, se questo è migliore dell'attuale impiegato, licenzierete l'attuale impiegato e assumerete il candidato. Siete intenzionati a pagare il prezzo derivante da questa strategia, ma volete stimare quale sarà questo prezzo.

La seguente procedura HIRE-ASSISTANT esprime questa strategia nella forma di pseudocodice. I candidati sono numerati da 1 a n. La procedura suppone che, dopo avere avuto un colloquio con il candidato i, voi siate in grado di determinare se questo candidato è il migliore fra quelli intervistati fino a quel momento. All'inizio, la procedura crea un candidato fittizio (con numero 0), che è meno qualificato di ogni altro candidato.

```
HIRE-ASSISTANT(n)
1  migliore = 0      // il candidato fittizio 0 è il meno qualificato
2  for i = 1 to n
3      colloquio con il candidato i
4      if il candidato i è migliore del candidato migliore
5          migliore = i
6          assumi il candidato i
```

Il modello dei costi per questo problema è diverso dal modello descritto nel Capitolo 2. Non siamo interessati al tempo di esecuzione di HIRE-ASSISTANT, ma bensì ai costi richiesti per il colloquio e l'assunzione. In apparenza, l'analisi dei costi di questo algoritmo potrebbe sembrare molto diversa dall'analisi del tempo di esecuzione, per esempio, di merge sort. Le tecniche analitiche adottate sono identiche sia quando valutiamo i costi sia quando valutiamo il tempo di esecuzione. In entrambi i casi, contiamo il numero di volte che vengono eseguite determinate operazioni elementari.

Acquistato da Michele Michelozzi il 2023-07-07 22:12 Numero Licenza: Licenza n. 6855034a1-2207d7 © Copyright 2022 McGraw-Hill Education (Italy)

Il colloquio ha un costo modesto (c_c), mentre l'assunzione ha un costo più alto (c_a). Se m è il numero totale di persone assunte, il costo totale associato a questo algoritmo è $O(nc_c + mc_a)$. Indipendentemente dal numero di persone assunte, dovremo sempre avere un colloquio con n candidati e, quindi, avremo sempre il costo nc_c associato ai colloqui. Quindi, concentriamo la nostra analisi sul costo di assunzione mc_a. Questa quantità varia ogni volta che viene eseguito l'algoritmo.

Questo scenario serve come modello per un tipico paradigma computazionale. Capita spesso di dover trovare il valore massimo o minimo in una sequenza, esaminando i singoli elementi della sequenza e conservando il "vincitore" corrente. Il problema delle assunzioni modella il numero di volte che noi cambiamo idea sull'elemento che consideriamo vincitore.

Analisi del caso peggiore

Nel caso peggiore, noi assumiamo ogni candidato con il quale abbiamo un colloquio. Questa situazione si verifica se i candidati si presentano in ordine strettamente crescente di qualità, nel qual caso effettuiamo n assunzioni, con un costo totale per le assunzioni pari a $O(nc_a)$.

Naturalmente, non sempre i candidati arrivano in ordine crescente di qualità. In effetti, non abbiamo alcuna idea sull'ordine in cui essi si presenteranno né abbiamo alcun controllo su tale ordine. Di conseguenza, è naturale chiedersi che cosa prevediamo che accada in un caso tipico o medio.

Analisi probabilistica

L'***analisi probabilistica*** è l'uso della probabilità nell'analisi dei problemi. Tipicamente, usiamo l'analisi probabilistica per analizzare il tempo di esecuzione di un algoritmo. A volte, la utilizziamo per analizzare altre grandezze, come il costo delle assunzioni nella procedura HIRE-ASSISTANT. Per svolgere un'analisi probabilistica, dobbiamo conoscere la distribuzione degli input o almeno fare delle ipotesi su tale distribuzione. Analizziamo quindi il nostro algoritmo, calcolando un tempo di esecuzione nel caso medio, dove la media è fatta sulla distribuzione degli input possibili. Quindi, in effetti, stiamo mediando il tempo di esecuzione su tutti gli input possibili. Questo tempo di esecuzione è detto ***tempo di esecuzione nel caso medio***.

La scelta della distribuzione degli input richiede molta attenzione. Per alcuni problemi, ha senso assumere alcune ipotesi sull'insieme di tutti i possibili input e applicare quindi l'analisi probabilistica come una tecnica per progettare algoritmi efficienti e come uno strumento di approfondimento della conoscenza del problema. Per altri problemi, invece, non è possibile ipotizzare una distribuzione di input accettabile; in questi casi, l'analisi probabilistica non può essere applicata.

Per il problema delle assunzioni, possiamo supporre che i candidati arrivino in ordine casuale. Che cosa significa ciò per questo problema? Noi supponiamo di poter confrontare due candidati qualsiasi e decidere quale dei due abbia i requisiti migliori; ovvero c'è un *ordine totale* nei candidati (vedere l'Appendice B per la definizione di ordine totale). Di conseguenza, possiamo classificare ogni candidato con un numero d'ordine unico da 1 a n, utilizzando $rango(i)$ per indicare il numero d'ordine (rango) del candidato i, e adottare la convenzione che a un rango più alto corrisponda un candidato più qualificato. La lista ordinata $\langle rango(1), rango(2), \ldots, rango(n)\rangle$ è una permutazione della lista $\langle 1, 2, \ldots, n\rangle$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Dire che i candidati si presentano in ordine casuale equivale a dire che questa lista di ranghi ha la stessa probabilità di essere una qualsiasi delle $n!$ permutazioni dei numeri da 1 a n. In alternativa, possiamo dire che i ranghi formano una ***permutazione casuale uniforme***, ovvero ciascuna delle $n!$ possibili permutazioni si presenta con uguale probabilità. Il Paragrafo 5.2 contiene un'analisi probabilistica del problema delle assunzioni.

Algoritmi randomizzati

Per potere utilizzare l'analisi probabilistica, dobbiamo sapere qualcosa sulla distribuzione degli input. In molti casi, sappiamo molto poco su questa distribuzione. Anche quando sappiamo qualcosa sulla distribuzione degli input, potremmo non essere in grado di modellare computazionalmente questa conoscenza. Eppure, spesso, è possibile utilizzare la probabilità e la casualità come strumento per progettare e analizzare gli algoritmi, rendendo casuale il comportamento di parte dell'algoritmo.

Nel problema delle assunzioni, sembra che i candidati si presentino in ordine casuale, tuttavia non abbiamo modo di sapere se ciò sia vero o no. Quindi, per sviluppare un algoritmo randomizzato per il problema delle assunzioni, dobbiamo avere un controllo maggiore sull'ordine in cui svolgiamo i colloqui con i candidati; per questo motivo, modificheremo leggermente il modello. Supponiamo che l'agenzia di selezione del personale abbia n candidati e che ci invii in anticipo una lista di candidati. Ogni giorno, scegliamo a caso il candidato con il quale avere un colloquio. Sebbene non sappiamo nulla sui candidati (a parte i loro nomi), abbiamo fatto una significativa modifica. Anziché fare affidamento sull'ipotesi che i candidati si presentino in ordine casuale, abbiamo ottenuto il controllo del processo e imposto un ordine casuale.

In generale, diciamo che un algoritmo è ***randomizzato*** se il suo comportamento è determinato non soltanto dal suo input, ma anche dai valori prodotti da un ***generatore di numeri casuali***. Supporremo di avere a disposizione un generatore di numeri casuali, che chiameremo RANDOM. Una chiamata di RANDOM(a, b) restituisce un numero intero compreso fra a e b (estremi inclusi); ciascuno di questi numeri interi ha la stessa probabilità di essere generato. Per esempio, RANDOM$(0, 1)$ genera il numero 0 con probabilità $1/2$ e il numero 1 con probabilità $1/2$. Una chiamata di RANDOM$(3, 7)$ restituisce uno dei numeri 3, 4, 5, 6 e 7, ciascuno con probabilità $1/5$. Ogni intero generato da RANDOM è indipendente dagli interi generati nelle precedenti chiamate. Potete immaginare che RANDOM lanci un dado con $(b - a + 1)$ facce per ottenere il suo output (in pratica, molti ambienti di programmazione hanno un ***generatore di numeri pseudocasuali***: un algoritmo deterministico che restituisce numeri che "sembrano" statisticamente casuali).

Quando analizziamo il tempo di esecuzione di un algoritmo randomizzato, consideriamo il valore atteso del tempo di esecuzione rispetto alla distribuzione dei valori restituiti dal generatore di numeri casuali. Distinguiamo questi algoritmi da quelli in cui l'input è casuale, chiamando il tempo di esecuzione di un algoritmo randomizzato ***tempo di esecuzione atteso***. In generale, consideriamo il tempo di esecuzione nel caso medio quando la distribuzione delle probabilità riguarda gli input dell'algoritmo, mentre consideriamo il tempo di esecuzione atteso quando l'algoritmo stesso effettua delle scelte casuali.

Acquistato da Michele Mastrolonardo (mastrolonardo@gmail.com) - Copyright © 2010 The McGraw-Hill Companies, S.r.l. - Tutti i diritti riservati - McGraw-Hill Education (Italy)

Esercizi

5.1-1

Dimostrate che l'ipotesi che siamo sempre in grado di determinare quale candidato sia migliore nella riga 4 della procedura HIRE-ASSISTANT implica che conosciamo un ordine totale sui ranghi dei candidati.

5.1-2 ⋆

Descrivete un'implementazione della procedura RANDOM(a, b) che effettua soltanto chiamate a RANDOM$(0, 1)$. Qual è il tempo di esecuzione previsto della vostra procedura in funzione di a e b?

5.1-3 ⋆

Supponete di voler generare il numero 0 con probabilità $1/2$ e 1 con probabilità $1/2$. Avete a disposizione una procedura BIASED-RANDOM che genera 0 o 1. Questa procedura genera 1 con probabilità p e 0 con probabilità $1 - p$, dove $0 < p < 1$, ma non conoscete il valore di p. Create un algoritmo che usa BIASED-RANDOM come subroutine e restituisce una soluzione imparziale, restituendo 0 con probabilità $1/2$ e 1 con probabilità $1/2$. Qual è il tempo di esecuzione previsto del vostro algoritmo in funzione di p?

5.2 Variabili casuali indicatrici

Per potere analizzare vari algoritmi, incluso il problema delle assunzioni, utilizzeremo le variabili casuali indicatrici. Queste variabili offrono un metodo comodo per la conversione tra probabilità e valori attesi. Supponiamo di avere lo spazio dei campioni S e un evento A. La ***variabile casuale indicatrice*** $\mathrm{I}\{A\}$ associata all'evento A è così definita

$$\mathrm{I}\{A\} = \begin{cases} 1 & \text{se si verifica l'evento } A \\ 0 & \text{se non si verifica l'evento } A \end{cases} \tag{5.1}$$

Come semplice esempio, supponiamo di lanciare in aria una moneta; vogliamo determinare il numero atteso di volte che otteniamo testa (T). Il nostro spazio dei campioni è $S = \{T, C\}$, con $\Pr\{T\} = \Pr\{C\} = 1/2$. Possiamo quindi definire una variabile casuale indicatrice X_T, associata al lancio di moneta che presenta testa, che è l'evento T. Questa variabile conta il numero di volte che si presenta testa in questo lancio; il suo valore è 1 se si presenta testa, altrimenti vale 0. Scriviamo

$$X_T = \mathrm{I}\{T\} = \begin{cases} 1 & \text{se si verifica } T \\ 0 & \text{se si verifica } C \end{cases}$$

Il numero atteso di T che si ottiene in un lancio della moneta è semplicemente il valore atteso della nostra variabile indicatrice X_T:

$$\begin{aligned} \mathrm{E}[X_T] &= \mathrm{E}[\mathrm{I}\{T\}] \\ &= 1 \cdot \Pr\{T\} + 0 \cdot \Pr\{C\} \\ &= 1 \cdot (1/2) + 0 \cdot (1/2) \\ &= 1/2 \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950316 220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Dunque, il numero atteso di T che si ottiene in un lancio di una moneta è $1/2$. Come dimostra il seguente lemma, il valore atteso di una variabile casuale indicatrice associata a un evento A è uguale alla probabilità che si verifichi A.

Lemma 5.1
Se S è lo spazio dei campioni e A è un evento nello spazio dei campioni S, ponendo $X_A = \mathrm{I}\{A\}$, si ha $\mathrm{E}[X_A] = \Pr\{A\}$.

Dimostrazione Per le definizioni del valore atteso – equazione (C.20) – e della variabile casuale indicatrice – equazione (5.1) – possiamo scrivere

$$\begin{aligned}\mathrm{E}[X_A] &= \mathrm{E}[\mathrm{I}\{A\}] \\ &= 1 \cdot \Pr\{A\} + 0 \cdot \Pr\{\overline{A}\} \\ &= \Pr\{A\}\end{aligned}$$

dove $\overline{A}$ indica $S - A$, il complemento di A. ■

Sebbene possa sembrare inutilmente complicato usare le variabili casuali indicatrici in applicazioni come quella di contare il numero atteso di volte che si presenta testa nel lancio di una moneta, tuttavia queste variabili sono utili per analizzare situazioni in cui effettuiamo ripetutamente delle prove casuali. Per esempio, le variabili casuali indicatrici ci offrono un metodo semplice per arrivare al risultato dell'equazione (C.37). In questa equazione, calcoliamo il numero di volte che otteniamo testa in n lanci della moneta, considerando separatamente la probabilità di ottenere 0 volte testa, 1 volta testa, 2 volte testa e così via. Tuttavia, il metodo più semplice proposto nell'equazione (C.38), in effetti, usa implicitamente le variabili casuali indicatrici. Per essere più espliciti, indichiamo con X_i la variabile casuale indicatrice associata all'evento in cui si presenta testa nell'i-esimo lancio della moneta: $X_i = \mathrm{I}\{\text{nell'}i\text{-esimo lancio si verifica l'evento } T\}$. Sia X la variabile casuale che indica il numero totale di T in n lanci:

$$X = \sum_{i=1}^{n} X_i$$

Per calcolare il numero atteso di T, prendiamo il valore atteso di entrambi i membri della precedente equazione, ottenendo

$$\mathrm{E}[X] = \mathrm{E}\left[\sum_{i=1}^{n} X_i\right]$$

Il membro sinistro di questa equazione è il valore atteso della somma di n variabili casuali. Per il Lemma 5.1, possiamo facilmente calcolare il valore atteso di ciascuna delle variabili casuali. Per l'equazione (C.21) – linearità del valore atteso – è facile calcolare il valore atteso della somma: è uguale alla somma dei valori attesi delle n variabili casuali. La linearità del valore atteso rende l'impiego delle variabili casuali indicatrici una potente tecnica analitica; si applica anche quando c'è dipendenza fra le variabili casuali. A questo punto possiamo calcolare facilmente il numero atteso di volte che si presenta testa:

$$\begin{aligned}\mathrm{E}[X] &= \mathrm{E}\left[\sum_{i=1}^{n} X_i\right] \\ &= \sum_{i=1}^{n} \mathrm{E}[X_i] \\ &= \sum_{i=1}^{n} 1/2 \\ &= n/2\end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Quindi, paragonate al metodo utilizzato nell'equazione (C.37), le variabili casuali indicatrici semplificano notevolmente i calcoli. In questo libro useremo sempre le variabili casuali.

Analisi del problema delle assunzioni con le variabili casuali indicatrici

Ritornando al problema delle assunzioni, adesso vogliamo calcolare il numero previsto di volte che assumiamo un nuovo impiegato. Per applicare l'analisi probabilistica, supponiamo che i candidati arrivino in ordine casuale, come descritto nel precedente paragrafo (vedremo nel Paragrafo 5.3 come rimuovere questa ipotesi). Sia X la variabile casuale il cui valore è uguale al numero di volte che assumiamo un nuovo impiegato. Applicando la definizione del valore atteso dell'equazione (C.20), otteniamo

$$\mathrm{E}[X] = \sum_{x=1}^{n} x \Pr\{X = x\}$$

Il calcolo di questa espressione non è semplice; utilizzeremo le variabili casuali indicatrici per semplificarlo notevolmente.

Per utilizzare le variabili casuali indicatrici, anziché calcolare $\mathrm{E}[X]$ definendo una variabile associata al numero di volte che assumiamo un nuovo impiegato, definiamo n variabili correlate al fatto che un candidato venga assunto oppure no. In particolare, indichiamo con X_i la variabile casuale indicatrice associata all'evento in cui l'i-esimo candidato sia assunto, ovvero

$$X_i = \mathrm{I}\{\text{il candidato } i \text{ è assunto}\} = \begin{cases} 1 & \text{se il candidato } i \text{ è assunto} \\ 0 & \text{se il candidato } i \text{ non è assunto} \end{cases}$$

e

$$X = X_1 + X_2 + \cdots + X_n \tag{5.2}$$

Per il Lemma 5.1, abbiamo che

$$\mathrm{E}[X_i] = \Pr\{\text{il candidato } i \text{ è assunto}\}$$

Quindi dobbiamo calcolare la probabilità che le righe 5–6 di HIRE-ASSISTANT siano eseguite.

Il candidato i è assunto, nella riga 6, esattamente quando il candidato i è migliore di tutti i candidati da 1 a $i-1$. Poiché abbiamo ipotizzato che i candidati arrivino in ordine casuale, i primi i candidati si sono presentati in ordine casuale. Uno qualsiasi dei primi i candidati ha la stessa possibilità di essere classificato come il migliore di tutti. Il candidato i ha la probabilità $1/i$ di essere qualificato migliore dei candidati da 1 a $i-1$ e, quindi, ha la probabilità $1/i$ di essere assunto. Per il Lemma 5.1, concludiamo che

$$\mathrm{E}[X_i] = 1/i \tag{5.3}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 1896030016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Adesso possiamo calcolare $\mathrm{E}[X]$:

$$\begin{aligned} \mathrm{E}[X] &= \mathrm{E}\left[\sum_{i=1}^{n} X_i\right] && \text{(per l'equazione (5.2))} \\ &= \sum_{i=1}^{n} \mathrm{E}[X_i] && \text{(per la linearità del valore atteso)} \end{aligned} \tag{5.4}$$

$$= \sum_{i=1}^{n} 1/i \qquad \text{(per l'equazione (5.3))}$$
$$= \ln n + O(1) \quad \text{(per l'equazione (A.7))} \tag{5.5}$$

Benché noi intervistiamo n candidati, ne assumiamo soltanto approssimativamente $\lg n$, in media. Sintetizziamo questo risultato nel seguente lemma.

Lemma 5.2
Supponendo che i candidati si presentino in ordine casuale, l'algoritmo HIRE-ASSISTANT ha un costo totale per le assunzioni pari a $O(c_a \ln n)$.

Dimostrazione Il limite si ricava immediatamente dalla nostra definizione di costo di assunzione e dall'equazione (5.5), che mostra che il numero previsto di assunzioni è approssimativamente $\ln n$. ■

Il costo per le assunzioni nel caso medio è un significativo miglioramento rispetto al costo $O(n c_a)$ per le assunzioni nel caso peggiore.

Esercizi

5.2-1
Nella procedura HIRE-ASSISTANT, supponendo che i candidati si presentino in ordine casuale, qual è la probabilità che venga effettuata esattamente una sola assunzione? Qual è la probabilità che vengano effettuate esattamente n assunzioni?

5.2-2
Nella procedura HIRE-ASSISTANT, supponendo che i candidati si presentino in ordine casuale, qual è la probabilità che vengano effettuate esattamente due assunzioni?

5.2-3
Utilizzate le variabili casuali indicatrici per calcolare il valore atteso della somma di n dadi.

5.2-4
Utilizzate le variabili casuali indicatrici per risolvere il seguente problema, che chiameremo ***problema dei cappelli***. Ognuno degli n clienti consegna il suo cappello a una persona che si trova all'ingresso di un ristorante. La persona restituisce i cappelli ai clienti in ordine casuale. Qual è il numero atteso di clienti che riceveranno il loro cappello?

5.2-5
Sia $A[1..n]$ un array di n numeri distinti. Se $i < j$ e $A[i] > A[j]$, allora la coppia (i, j) è detta ***inversione*** di A (vedere il Problema 2-4 per maggiori informazioni sulle inversioni). Supponete che gli elementi di A formino una permutazione casuale uniforme di $\langle 1, 2, \ldots, n \rangle$. Utilizzate le variabili casuali indicatrici per calcolare il numero atteso di inversioni.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

5.3 Algoritmi randomizzati

Nel precedente paragrafo abbiamo visto che la conoscenza di una distribuzione degli input ci aiuta ad analizzare il comportamento nel caso medio di un algoritmo. Molte volte, non abbiamo simili conoscenze e questo ci impedisce di svolgere

una analisi del caso medio. Come detto nel Paragrafo 5.1, potremmo utilizzare un algoritmo randomizzato. Per un problema come quello delle assunzioni, in cui è utile ipotizzare che tutte le permutazioni dell'input siano egualmente probabili, l'analisi probabilistica ci potrà guidare nello sviluppo di un algoritmo randomizzato. Anziché ipotizzare una distribuzione degli input, imponiamo noi una distribuzione. In pratica, prima di eseguire l'algoritmo, permutiamo casualmente i candidati per imporre la proprietà che ogni permutazione sia egualmente probabile. Benché l'algoritmo sia stato modificato, noi ci aspettiamo ancora di assumere un nuovo impiegato approssimativamente $\ln n$ volte. Ma ora ci attendiamo che questo accada per *ogni* input e non soltanto per quelli estratti da una particolare distribuzione.

Analizziamo meglio la differenza fra analisi probabilistica e algoritmi randomizzati. Nel Paragrafo 5.2 abbiamo visto che, supponendo che i candidati si presentino in ordine casuale, il numero previsto di volte che assumiamo un nuovo impiegato è pari a circa $\ln n$. Notate che l'algoritmo qui è deterministico; per un particolare input, il numero di volte che viene assunto un nuovo impiegato sarà sempre lo stesso. Inoltre, il numero di volte che assumiamo un nuovo impiegato è diverso se cambia l'input e dipende dai ranghi dei vari candidati. Poiché questo numero dipende soltanto dai ranghi dei candidati, possiamo rappresentare un particolare input elencando, in ordine, tali ranghi, ovvero $\langle rango(1), rango(2), \ldots, rango(n)\rangle$.

Con la lista dei ranghi $A_1 = \langle 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\rangle$, un nuovo impiegato sarà assunto 10 volte, in quanto ogni candidato è migliore del precedente; le righe 5–6 saranno eseguite in ogni iterazione dell'algoritmo. Con la lista dei ranghi $A_2 = \langle 10, 9, 8, 7, 6, 5, 4, 3, 2, 1\rangle$, un nuovo impiegato sarà assunto una sola volta, nella prima iterazione. Con la lista dei ranghi $A_3 = \langle 5, 2, 1, 8, 4, 7, 10, 9, 3, 6\rangle$, un nuovo impiegato sarà assunto tre volte, dopo i colloqui con i candidati di rango 5, 8 e 10. Ricordando che il costo del nostro algoritmo dipende dal numero di volte che viene assunto un nuovo impiegato, notiamo che ci sono input costosi, come A_1, input economici, come A_2, e input moderatamente costosi, come A_3.

Consideriamo, d'altra parte, l'algoritmo randomizzato che prima permuta i candidati e poi determina il miglior candidato. In questo caso, la randomizzazione è nell'algoritmo, non nella distribuzione degli input. Dato un particolare input (per esempio A_3), non possiamo dire quante volte sarà aggiornato il massimo, perché questa quantità è diversa ogni volta che viene eseguito l'algoritmo. La prima volta che eseguiamo l'algoritmo su A_3, potremmo ottenere la permutazione A_1 con 10 aggiornamenti, mentre la seconda volta che eseguiamo l'algoritmo, potremmo ottenere la permutazione A_2 con un solo aggiornamento. La terza volta, l'algoritmo potrebbe effettuare qualche altro numero di aggiornamenti. Ogni volta che eseguiamo l'algoritmo, l'esecuzione dipende dalle scelte casuali fatte ed è probabile che sia diversa dalle precedenti esecuzioni. Per questo algoritmo e per molti altri algoritmi randomizzati, *nessun input particolare determina il caso peggiore*. Neppure il vostro peggior nemico è in grado di produrre un brutto array di input, in quanto la permutazione casuale rende irrilevante l'ordine degli input. L'algoritmo randomizzato si comporta male soltanto se il generatore di numeri casuali produce una permutazione "sventurata".

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Per il problema delle assunzioni, l'unica modifica da apportare al codice è permutare casualmente l'array.

```
RANDOMIZED-HIRE-ASSISTANT(n)
1  permuta casualmente la lista dei candidati
2  migliore = 0      // il candidato fittizio 0 è il meno qualificato
3  for i = 1 to n
4      colloquio con il candidato i
5      if il candidato i è migliore del candidato migliore
6          migliore = i
7          assumi il candidato i
```

Con questa semplice modifica abbiamo creato un algoritmo randomizzato le cui prestazioni corrispondono a quelle ottenute supponendo che i candidati si presentino in ordine casuale.

Lemma 5.3
Il costo previsto per assumere nuovi impiegati nella procedura RANDOMIZED-HIRE-ASSISTANT è $O(c_a \ln n)$.

Dimostrazione Dopo avere permutato l'array di input, abbiamo una situazione identica a quella dell'analisi probabilistica di HIRE-ASSISTANT. ■

Il confronto fra i Lemmi 5.2 e 5.3 mette in risalto la differenza fra analisi probabilistica e algoritmi randomizzati. Nel Lemma 5.2 facciamo un'ipotesi sull'input. Nel Lemma 5.3 non facciamo alcuna ipotesi, sebbene la randomizzazione dell'input richieda un tempo aggiuntivo. Per essere coerenti con la nostra terminologia, abbiamo espresso il Lemma 5.2 in termini di costo delle assunzioni nel caso medio e il Lemma 5.3 in termini di costo atteso delle assunzioni. Nel prossimo paragrafo analizzeremo alcuni problemi relativi alla permutazione casuale dell'input.

Permutazione casuale degli array

Molti algoritmi randomizzati effettuano la randomizzazione dell'input permutando l'array di input (ci sono anche altri modi per usare la randomizzazione). Qui discuteremo due modi per fare ciò. Noi assumeremo di avere un array A che, senza perdere di generalità, contenga gli elementi da 1 a n. Il nostro obiettivo è produrre una permutazione casuale dell'array.

Un tipico metodo consiste nell'assegnare a ogni elemento $A[i]$ dell'array una priorità casuale $P[i]$ e, poi, nell'ordinare gli elementi di A in funzione di queste priorità. Per esempio, se il nostro array iniziale è $A = \langle 1, 2, 3, 4 \rangle$ e scegliamo le priorità casuali $P = \langle 36, 3, 97, 19 \rangle$, otterremo l'array $B = \langle 2, 4, 1, 3 \rangle$, in quanto la seconda priorità è la più piccola, seguita dalla quarta, poi dalla prima e, infine, dalla terza. Chiamiamo questa procedura PERMUTE-BY-SORTING:

```
PERMUTE-BY-SORTING(A)
1  n = A.length
2  sia P[1..n] un nuovo array
3  for i = 1 to n
4      P[i] = RANDOM(1, n^3)
5  ordina A, utilizzando P come chiavi di ordinamento
```

Acquistato da Michele Michelassi su Pearson+ il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

La riga 4 sceglie un numero casuale fra 1 e n^3. Utilizziamo un intervallo da 1 a n^3 per aumentare la possibilità che tutte le priorità di P siano uniche (l'Esercizio 5.3-5 chiede di dimostrare che la probabilità che tutte le priorità siano uniche è

almeno $1-1/n$; l'Esercizio 5.3-6 chiede come implementare l'algoritmo anche se due o più priorità sono identiche). Supponiamo che tutte le priorità siano uniche.

Il passo che consuma tempo in questa procedura è l'ordinamento nella riga 5. Come vedremo nel Capitolo 8, se utilizziamo un ordinamento per confronti, l'operazione di ordinamento richiede un tempo $\Omega(n \lg n)$. Possiamo raggiungere questo limite inferiore, perché abbiamo visto che merge sort richiede un tempo $\Theta(n \lg n)$. (Nella Parte II vedremo altri ordinamenti per confronti che richiedono un tempo $\Theta(n \lg n)$. L'Esercizio 8.3-4 chiede di risolvere, in tempo $O(n)$, il problema molto simile di ordinare dei numeri nell'intervallo da 0 a $n^3 - 1$.) Dopo l'ordinamento, se $P[i]$ è la j-esima priorità più piccola, allora $A[i]$ sarà nella posizione j dell'output. In questo modo otteniamo una permutazione. Resta da dimostrare che la procedura genera una ***permutazione casuale uniforme***, ovvero che ogni permutazione dei numeri da 1 a n ha la stessa probabilità di essere prodotta.

Lemma 5.4
Se tutte le priorità sono distinte, la procedura PERMUTE-BY-SORTING genera una permutazione casuale uniforme dell'input.

Dimostrazione Iniziamo a considerare la particolare permutazione in cui ogni elemento $A[i]$ riceve la i-esima priorità più piccola. Dimostreremo che questa permutazione si verifica con una probabilità esattamente pari a $1/n!$. Per $i = 1, 2, \ldots, n$, sia X_i l'evento in cui l'elemento $A[i]$ riceve la i-esima priorità più piccola. La probabilità che si verifichi l'evento X_i per qualsiasi i è data da

$$\Pr\{X_1 \cap X_2 \cap X_3 \cap \cdots \cap X_{n-1} \cap X_n\}$$

Applicando l'Esercizio C.2-5, questa probabilità è uguale a

$$\Pr\{X_1\} \cdot \Pr\{X_2 \mid X_1\} \cdot \Pr\{X_3 \mid X_2 \cap X_1\} \cdot \Pr\{X_4 \mid X_3 \cap X_2 \cap X_1\} \\ \cdots \Pr\{X_i \mid X_{i-1} \cap X_{i-2} \cap \cdots \cap X_1\} \cdots \Pr\{X_n \mid X_{n-1} \cap \cdots \cap X_1\}$$

Abbiamo che $\Pr\{X_1\} = 1/n$, perché questa è la probabilità che una priorità scelta a caso da un insieme di n elementi sia la più piccola. Poi, osserviamo che $\Pr\{X_2 \mid X_1\} = 1/(n-1)$, in quanto, dato che l'elemento $A[1]$ ha la priorità più piccola, ciascuno dei restanti $n-1$ elementi ha la stessa probabilità di avere la seconda priorità più piccola. In generale, per $i = 2, 3, \ldots, n$, abbiamo che $\Pr\{X_i \mid X_{i-1} \cap X_{i-2} \cap \cdots \cap X_1\} = 1/(n-i+1)$, in quanto, dato che gli elementi da $A[1]$ a $A[i-1]$ hanno le $i-1$ priorità più piccole (in ordine), ciascuno dei restanti $n-(i-1)$ elementi ha la stessa probabilità di avere la i-esima priorità più piccola; quindi, abbiamo

$$\Pr\{X_1 \cap X_2 \cap X_3 \cap \cdots \cap X_{n-1} \cap X_n\} = \left(\frac{1}{n}\right)\left(\frac{1}{n-1}\right)\cdots\left(\frac{1}{2}\right)\left(\frac{1}{1}\right) = \frac{1}{n!}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707 Copyright © 2022, McGraw-Hill Education (Italy)

Abbiamo dimostrato che la probabilità di ottenere la permutazione identità è $1/n!$.

Possiamo estendere questa dimostrazione a qualsiasi permutazione delle priorità. Consideriamo una permutazione fissa $\sigma = \langle \sigma(1), \sigma(2), \ldots, \sigma(n) \rangle$ dell'insieme $\{1, 2, \ldots, n\}$. Indichiamo con r_i il rango della priorità assegnata all'elemento $A[i]$, dove l'elemento con la j-esima priorità più piccola ha rango j. Se definiamo X_i l'evento in cui l'elemento $A[i]$ riceve la $\sigma(i)$-esima priorità più piccola,

ossia $r_i = \sigma(i)$, si può applicare ancora la stessa dimostrazione. Ne consegue che, per calcolare la probabilità di ottenere una permutazione qualsiasi, dobbiamo svolgere gli stessi calcoli; quindi la probabilità di ottenere questa permutazione è ancora $1/n!$. ■

Si potrebbe pensare che per dimostrare che una permutazione sia una permutazione casuale uniforme sia sufficiente dimostrare che, per ogni elemento $A[i]$, la probabilità che esso raggiunga la posizione j sia $1/n$. L'Esercizio 5.3-4 dimostra che questa condizione più debole, in effetti, non è sufficiente.

Un metodo migliore per generare una permutazione casuale consiste nel permutare l'array sul posto. La procedura RANDOMIZE-IN-PLACE lo fa nel tempo $O(n)$. Nell'iterazione i, l'elemento $A[i]$ viene scelto casualmente fra gli elementi compresi fra $A[i]$ e $A[n]$. Dopo l'iterazione i, $A[i]$ non viene più modificato.

```
RANDOMIZE-IN-PLACE(A)
1  n = A.length
2  for i = 1 to n
3      scambia A[i] ↔ A[RANDOM(i, n)]
```

Utilizzeremo una invariante di ciclo per dimostrare che questa procedura produce una permutazione casuale uniforme. Dato un insieme di n elementi, una k-permutazione è una sequenza che contiene k degli n elementi (vedere l'Appendice C). In un insieme di n elementi ci sono $n!/(n-k)!$ possibili k-permutazioni.

Lemma 5.5
La procedura RANDOMIZE-IN-PLACE genera una permutazione casuale uniforme.

Dimostrazione Utilizziamo la seguente invariante di ciclo:

> Appena prima della i-esima iterazione del ciclo **for**, righe 2–3, per ogni possibile $(i-1)$-permutazione, il sottoarray $A[1\,..\,i-1]$ contiene questa $(i-1)$-permutazione con probabilità $(n-i+1)!/n!$.

Dobbiamo dimostrare che questa invariante è vera prima della prima iterazione del ciclo, che ogni iterazione del ciclo conserva l'invariante e che l'invariante fornisce un'utile proprietà per dimostrare la correttezza quando il ciclo termina.

Inizializzazione: consideriamo la situazione appena prima della prima iterazione del ciclo, con $i = 1$. L'invariante di ciclo dice che per ogni possibile 0-permutazione, il sottoarray $A[1\,..\,0]$ contiene questa 0-permutazione con probabilità $(n-i+1)!/n! = n!/n! = 1$. Il sottoarray $A[1\,..\,0]$ è vuoto e una 0-permutazione non ha elementi. Quindi, $A[1\,..\,0]$ contiene qualsiasi 0-permutazione con probabilità 1 e l'invariante di ciclo è vera prima della prima iterazione.

Conservazione: supponiamo che, appena prima della i-esima iterazione, ogni possibile $(i-1)$-permutazione appaia nel sottoarray $A[1\,..\,i-1]$ con probabilità $(n-i+1)!/n!$; dimostreremo che, dopo l'i-esima iterazione, ogni possibile i-permutazione appare nel sottoarray $A[1\,..\,i]$ con probabilità $(n-i)!/n!$. L'incremento di i per la successiva iterazione manterrà l'invariante di ciclo.

Esaminiamo la i-esima iterazione. Consideriamo una particolare i-permutazione e indichiamo i suoi elementi con $\langle x_1, x_2, \ldots, x_i \rangle$. Questa permutazione è formata da una $(i-1)$-permutazione $\langle x_1, \ldots, x_{i-1} \rangle$ seguita dal valore x_i

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

che l'algoritmo pone in $A[i]$. Indichiamo con E_1 l'evento in cui le prime $i-1$ iterazioni hanno creato la particolare $(i-1)$-permutazione $\langle x_1, \ldots, x_{i-1}\rangle$ in $A[1\,..\,i-1]$. Per l'invariante di ciclo, $\Pr\{E_1\} = (n-i+1)!/n!$. Sia E_2 l'evento in cui la i-esima iterazione pone x_i nella posizione $A[i]$. La i-permutazione $\langle x_1, \ldots, x_i\rangle$ si forma in $A[1\,..\,i]$ precisamente quando si verificano entrambi gli eventi E_1 ed E_2, quindi calcoliamo $\Pr\{E_2 \cap E_1\}$. Applicando l'equazione (C.14), abbiamo

$$\Pr\{E_2 \cap E_1\} = \Pr\{E_2 \mid E_1\}\Pr\{E_1\}$$

La probabilità $\Pr\{E_2 \mid E_1\}$ è uguale a $1/(n-i+1)$, perché nella riga 3 l'algoritmo sceglie x_i a caso dagli $n-i+1$ valori nelle posizioni $A[i\,..\,n]$. Dunque, otteniamo

$$\begin{aligned}\Pr\{E_2 \cap E_1\} &= \Pr\{E_2 \mid E_1\}\Pr\{E_1\} \\ &= \frac{1}{n-i+1}\cdot\frac{(n-i+1)!}{n!} \\ &= \frac{(n-i)!}{n!}\end{aligned}$$

Conclusione: alla fine del ciclo, $i = n+1$ e il sottoarray $A[1\,..\,n]$ è una particolare n-permutazione con probabilità $(n-n)!/n! = 1/n!$.

Quindi, la procedura RANDOMIZE-IN-PLACE genera una permutazione casuale uniforme. ■

Un algoritmo randomizzato spesso è il metodo più semplice ed efficiente per risolvere un problema. In questo libro utilizzeremo talvolta degli algoritmi randomizzati.

Esercizi

5.3-1

Il professor Marceau non è d'accordo con l'invariante di ciclo utilizzata nella dimostrazione del Lemma 5.5. Egli contesta che sia vera prima della prima iterazione. Egli argomenta che si potrebbe altrettanto facilmente affermare che un sottoarray vuoto non contiene nessuna 0-permutazione. Di conseguenza, la probabilità che un sottoarray vuoto contenga una 0-permutazione dovrebbe essere 0, invalidando così l'invariante di ciclo prima della prima iterazione. Riscrivete la procedura RANDOMIZE-IN-PLACE in modo che l'invariante di ciclo si applichi a un sottoarray non vuoto prima della prima iterazione e modificate la dimostrazione del Lemma 5.5 per la vostra procedura.

5.3-2

Il professor Kelp decide di scrivere una procedura che produrrà casualmente qualsiasi permutazione tranne la permutazione identità; propone questa procedura:

Acquistato da Michele Michele su Webster il 2022-07-07 28:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

```
PERMUTE-WITHOUT-IDENTITY(A)
1  n = A.length
2  for i = 1 to n − 1
3      scambia A[i] con A[RANDOM(i + 1, n)]
```

Questo codice fa ciò che intende il professor Kelp?

5.3-3
Anziché scambiare l'elemento $A[i]$ con un elemento a caso del sottoarray $A[i \,..\, n]$, supponete di scambiarlo con un elemento a caso dell'array:

```
PERMUTE-WITH-ALL(A)
1  n = A.length
2  for i = 1 to n
3      scambia A[i] con A[RANDOM(1, n)]
```

Questo codice genera una permutazione casuale uniforme? Perché o perché no?

5.3-4
Il professor Armstrong suggerisce la seguente procedura per generare una permutazione casuale uniforme:

```
PERMUTE-BY-CYCLIC(A)
1  n = A.length
2  sia B[1..n] un nuovo array
3  offset = RANDOM(1, n)
4  for i = 1 to n
5      dest = i + offset
6      if dest > n
7          dest = dest − n
8      B[dest] = A[i]
9  return B
```

Dimostrate che ogni elemento $A[i]$ ha probabilità $1/n$ di trovarsi in una particolare posizione in B. Poi dimostrate che il professor Armstrong sbaglia, dimostrando che la permutazione risultante non è uniformemente casuale.

5.3-5 ⋆
Dimostrate che nell'array P della procedura PERMUTE-BY-SORTING la probabilità che tutti gli elementi siano unici è almeno $1 - 1/n$.

5.3-6
Spiegate come implementare l'algoritmo PERMUTE-BY-SORTING per gestire il caso in cui due o più priorità siano identiche; ovvero il vostro algoritmo dovrebbe produrre una permutazione casuale uniforme, anche se due o più priorità sono identiche.

5.3-7
Supponete di voler creare un ***campione casuale*** dell'insieme $\{1, 2, 3, \ldots, n\}$, ovvero un sottoinsieme S di m elementi, dove $0 \leq m \leq n$, tale che ciascun m-sottoinsieme abbia la stessa probabilità di essere creato. Un metodo potrebbe essere quello di impostare $A[i] = i$ per $i = 1, 2, 3, \ldots, n$, chiamare RANDOMIZE-IN-PLACE(A) e poi estrarre semplicemente i primi m elementi dell'array. Questo metodo farebbe n chiamate della procedura RANDOM. Se n è molto più grande di m, possiamo creare un campione casuale con un minor numero di chiamate di RANDOM. Dimostrate che la seguente procedura ricorsiva restituisce un sottoinsieme casuale S dell'insieme $\{1, 2, 3, \ldots, n\}$, in cui ciascun m-sottoinsieme ha la stessa probabilità, effettuando soltanto m chiamate di RANDOM:

Acquistato da Michele Mancini (mancini.michele@gmail.com) - Numero ordine interno: 2020-0 - Copyright © 2010 The McGraw-Hill Education (Italy)

```
RANDOM-SAMPLE(m, n)
1  if m == 0
2      return ∅
3  else S = RANDOM-SAMPLE(m − 1, n − 1)
4      i = RANDOM(1, n)
5      if i ∈ S
6          S = S ∪ {n}
7      else S = S ∪ {i}
8      return S
```

⋆ 5.4 Approfondimento dell'analisi probabilistica

Quest'ultima parte del capitolo approfondisce l'analisi probabilistica illustrando quattro esempi. Il primo esempio determina la probabilità che, in una stanza di k persone, due di esse siano nate nello stesso giorno dell'anno. Il secondo esempio studia il lancio casuale delle palline nei contenitori. Il terzo esempio analizza le sequenze di teste consecutive nel lancio di una moneta. L'ultimo esempio studia una variante del problema delle assunzioni, dove bisogna prendere delle decisioni senza intervistare tutti i candidati.

5.4.1 Il paradosso del compleanno

Il nostro primo esempio è il ***paradosso del compleanno***. Quante persone devono trovarsi in una stanza prima che ci sia una probabilità del 50% che due di esse siano nate nello stesso giorno dell'anno? Sorprendentemente, la risposta è: poche. Il paradosso è che, in effetti, ne bastano molte di meno del numero di giorni di un anno o anche della metà del numero di giorni di un anno, come vedremo.

Per rispondere a questa domanda, assegniamo a ogni persona un indice intero $1, 2, \ldots, k$, dove k è il numero delle persone che si trovano nella stanza. Ignoriamo il problema degli anni bisestili e supponiamo che tutti gli anni abbiano $n = 365$ giorni. Per $i = 1, 2, \ldots, k$, sia b_i il giorno dell'anno in cui è nata la persona i, con $1 \leq b_i \leq n$. Supponiamo inoltre che i compleanni siano uniformemente distribuiti negli n giorni dell'anno, quindi $\Pr\{b_i = r\} = 1/n$ per $i = 1, 2, \ldots, k$ e $r = 1, 2, \ldots, n$.

La probabilità che due persone, i e j, siano nate nello stesso giorno dell'anno dipende dal fatto che la selezione casuale dei compleanni sia indipendente. Nell'ipotesi che i compleanni siano indipendenti, la probabilità che il compleanno di i e il compleanno di j siano entrambi nel giorno r è

$$\begin{aligned}\Pr\{b_i = r \text{ e } b_j = r\} &= \Pr\{b_i = r\}\Pr\{b_j = r\} \\ &= 1/n^2\end{aligned}$$

Quindi, la probabilità che entrambi i compleanni siano nello stesso giorno è

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned}\Pr\{b_i = b_j\} &= \sum_{r=1}^{n} \Pr\{b_i = r \text{ e } b_j = r\} \\ &= \sum_{r=1}^{n} (1/n^2) \\ &= 1/n \qquad (5.6)\end{aligned}$$

Più intuitivamente, una volta scelto b_i, la probabilità che b_j sia scelto nello stesso giorno è $1/n$. Quindi, la probabilità che i e j abbiano lo stesso compleanno è la stessa probabilità che il compleanno di uno di essi sia in un dato giorno. Notate, tuttavia, che questa coincidenza dipende dall'ipotesi che i compleanni siano indipendenti. Possiamo analizzare la probabilità di almeno 2 su k persone che hanno lo stesso compleanno, osservando l'evento complementare. La probabilità che almeno due compleanni coincidano è 1 meno la probabilità che tutti i compleanni siano differenti. L'evento in cui k persone abbiano compleanni distinti è

$$B_k = \bigcap_{i=1}^{k} A_i$$

dove A_i è l'evento in cui il compleanno della persona i sia diverso da quello della persona j per ogni $j < i$. Poiché possiamo scrivere $B_k = A_k \cap B_{k-1}$, otteniamo dall'equazione (C.16) la ricorrenza

$$\Pr\{B_k\} = \Pr\{B_{k-1}\} \Pr\{A_k \mid B_{k-1}\} \tag{5.7}$$

dove consideriamo $\Pr\{B_1\} = \Pr\{A_1\} = 1$ come una condizione iniziale. In altre parole, la probabilità che $b_1, b_2, \ldots, b_k$ siano compleanni distinti è la probabilità che $b_1, b_2, \ldots, b_{k-1}$ siano compleanni distinti moltiplicata per la probabilità che $b_k \neq b_i$ per $i = 1, 2, \ldots, k-1$, dato che $b_1, b_2, \ldots, b_{k-1}$ sono distinti.

Se $b_1, b_2, \ldots, b_{k-1}$ sono distinti, la probabilità condizionata che $b_k \neq b_i$ per $i = 1, 2, \ldots, k-1$ è $\Pr\{A_k \mid B_{k-1}\} = (n-k+1)/n$, dal momento che, su n giorni, ci sono $n-(k-1)$ giorni non ancora assegnati. Applicando iterativamente la ricorrenza (5.7), otteniamo

$$\begin{aligned}
\Pr\{B_k\} &= \Pr\{B_{k-1}\} \Pr\{A_k \mid B_{k-1}\} \\
&= \Pr\{B_{k-2}\} \Pr\{A_{k-1} \mid B_{k-2}\} \Pr\{A_k \mid B_{k-1}\} \\
&\vdots \\
&= \Pr\{B_1\} \Pr\{A_2 \mid B_1\} \Pr\{A_3 \mid B_2\} \cdots \Pr\{A_k \mid B_{k-1}\} \\
&= 1 \cdot \left(\frac{n-1}{n}\right)\left(\frac{n-2}{n}\right) \cdots \left(\frac{n-k+1}{n}\right) \\
&= 1 \cdot \left(1-\frac{1}{n}\right)\left(1-\frac{2}{n}\right) \cdots \left(1-\frac{k-1}{n}\right)
\end{aligned}$$

Dalla disuguaglianza (3.12), $1 + x \leq e^x$, si ottiene

$$\begin{aligned}
\Pr\{B_k\} &\leq e^{-1/n} e^{-2/n} \cdots e^{-(k-1)/n} \\
&= e^{-\sum_{i=1}^{k-1} i/n} \\
&= e^{-k(k-1)/2n} \\
&\leq 1/2
\end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

quando $-k(k-1)/2n \leq \ln(1/2)$. La probabilità che tutti i k compleanni siano distinti è al massimo $1/2$ quando $k(k-1) \geq 2n \ln 2$ ovvero, risolvendo l'equazione quadratica, quando $k \geq (1 + \sqrt{1 + (8 \ln 2)n})/2$. Per $n = 365$, dobbiamo avere $k \geq 23$. Quindi, se nella stanza ci sono almeno 23 persone, la probabilità che almeno due di esse abbiano lo stesso compleanno è almeno $1/2$. Su Marte, un anno dura 669 giorni, quindi occorrono 31 marziani per ottenere lo stesso effetto.

Un'analisi tramite le variabili casuali indicatrici

Possiamo usare le variabili casuali indicatrici per fare un'analisi più semplice, ma approssimata, del paradosso del compleanno. Per ogni coppia (i, j) delle k persone che si trovano nella stanza, definiamo la variabile casuale indicatrice X_{ij}, con $1 \leq i < j \leq k$, in questo modo

$$\begin{aligned} X_{ij} &= \mathrm{I}\{\text{la persona } i \text{ e la persona } j \text{ hanno lo stesso compleanno}\} \\ &= \begin{cases} 1 & \text{se la persona } i \text{ e la persona } j \text{ hanno lo stesso compleanno} \\ 0 & \text{negli altri casi} \end{cases} \end{aligned}$$

Per l'equazione (5.6), la probabilità che due persone abbiano lo stesso compleanno è $1/n$, quindi per il Lemma 5.1, abbiamo

$$\begin{aligned} \mathrm{E}[X_{ij}] &= \Pr\{\text{la persona } i \text{ e la persona } j \text{ hanno lo stesso compleanno}\} \\ &= 1/n \end{aligned}$$

Indicando con X la variabile casuale che conta il numero di coppie di persone che hanno lo stesso compleanno, abbiamo

$$X = \sum_{i=1}^{k} \sum_{j=i+1}^{k} X_{ij}$$

Prendendo i valori attesi da entrambi i membri dell'equazione e applicando la linearità del valore atteso, otteniamo

$$\begin{aligned} \mathrm{E}[X] &= \mathrm{E}\left[\sum_{i=1}^{k} \sum_{j=i+1}^{k} X_{ij}\right] \\ &= \sum_{i=1}^{k} \sum_{j=i+1}^{k} \mathrm{E}[X_{ij}] \\ &= \binom{k}{2}\frac{1}{n} \\ &= \frac{k(k-1)}{2n} \end{aligned}$$

Se $k(k-1) \geq 2n$, il numero atteso di coppie di persone che hanno lo stesso compleanno è almeno 1. Dunque, se nella stanza ci sono almeno $\sqrt{2n} + 1$ persone, possiamo prevedere che almeno due persone siano nate nello stesso giorno dell'anno. Per $n = 365$, se $k = 28$, il numero atteso di coppie con lo stesso compleanno è $(28 \cdot 27)/(2 \cdot 365) \approx 1,0356$. Quindi, con almeno 28 persone, ci aspettiamo di trovare almeno una coppia con lo stesso compleanno. Su Marte, dove un anno dura 669 giorni, occorrono almeno 38 marziani.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 29503016.200707.0 Copyright © 2023, McGraw-Hill Education (Italy)

La prima analisi, che utilizzava soltanto le probabilità, ha determinato il numero di persone necessarie affinché la probabilità che ci sia una coppia con lo stesso compleanno superi $1/2$. La seconda analisi, che utilizzava le variabili casuali indicatrici, ha determinato un numero tale che il numero atteso di coppie di persone con lo stesso compleanno sia 1. Sebbene i numeri esatti di persone differiscano nei due casi, tuttavia essi sono asintoticamente uguali: $\Theta(\sqrt{n})$.

5.4.2 Lancio delle palline nei contenitori

Consideriamo il processo di lanciare a caso delle palline identiche in b contenitori, numerati $1, 2, \ldots, b$. I lanci sono indipendenti e in ogni lancio la pallina ha la stessa probabilità di finire in qualsiasi contenitore. La probabilità che una pallina lanciata finisca in un contenitore vuoto è $1/b$. Quindi, il lancio delle palline è una sequenza di prove ripetute di Bernoulli (vedere l'Appendice C.4) con una probabilità $1/b$ di successo, dove successo significa che la pallina finisce in un determinato contenitore. Questo modello è particolarmente utile per analizzare l'hashing (vedere il Capitolo 11) e noi possiamo rispondere a una serie di interessanti domande sul lancio delle palline (il Problema C-1 pone altre domande sul lancio delle palline).

Quante palline finiscono in un determinato contenitore? Il numero di palline che cadono in un determinato contenitore segue la distribuzione binomiale $b(k; n, 1/b)$. Se vengono lanciate n palline, l'equazione (C.37) ci dice che il numero atteso di palline che finiscono in un determinato contenitore è n/b.

Quante palline devono essere lanciate, in media, affinché una di esse finisca in un determinato contenitore? Il numero di lanci da effettuare affinché una pallina finisca in un determinato contenitore segue la distribuzione geometrica con probabilità $1/b$ e, per l'equazione (C.32), il numero atteso di lanci per fare centro in quel contenitore è $1/(1/b) = b$.

Quante palline devono essere lanciate affinché ogni contenitore abbia almeno una pallina? Chiamiamo "centro" un lancio in cui una pallina cade in un contenitore vuoto. Vogliamo determinare il numero atteso n di lanci necessari per fare b centri.

I centri possono essere utilizzati per ripartire gli n lanci in fasi. L'i-esima fase è formata dai lanci effettuati dopo l'$(i-1)$-esimo centro fino all'i-esimo centro. La prima fase è formata dal primo lancio, perché abbiamo la certezza di fare centro quando tutti i contenitori sono vuoti. Per ogni lancio effettuato durante l'i-esima fase, ci sono $i-1$ contenitori che contengono palline e $b-i+1$ contenitori vuoti. Quindi, per ogni lancio nell'i-esima fase, la probabilità di fare centro è $(b-i+1)/b$.

Se indichiamo con n_i il numero di lanci nell'i-esima fase, il numero di lanci necessari per fare b centri è $n = \sum_{i=1}^{b} n_i$. Ogni variabile casuale n_i ha una distribuzione geometrica con probabilità di successo $(b-i+1)/b$ e, per l'equazione (C.32), abbiamo

$$\mathrm{E}[n_i] = \frac{b}{b-i+1}$$

Applicando la linearità del valore atteso, otteniamo

$$\begin{aligned}
\mathrm{E}[n] &= \mathrm{E}\left[\sum_{i=1}^{b} n_i\right] \\
&= \sum_{i=1}^{b} \mathrm{E}[n_i] \\
&= \sum_{i=1}^{b} \frac{b}{b-i+1}
\end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$
\begin{aligned}
&= b \sum_{i=1}^{b} \frac{1}{i} \\
&= b(\ln b + O(1)) \quad \text{(per l'equazione (A.7))}
\end{aligned}
$$

Dunque, occorrono circa $b \ln b$ lanci prima che ci si possa aspettare che ogni contenitore abbia una pallina. Questo problema, detto anche ***problema del collezionista di figurine***, dice che una persona che tenta di collezionare b figurine differenti deve aspettarsi di dover raccogliere a caso circa $b \ln b$ figurine per raggiungere il suo obiettivo.

5.4.3 Serie dello stesso evento

Supponete di lanciare n volte una moneta. Qual è la serie più lunga di teste consecutive che prevedete di ottenere? La risposta è $\Theta(\lg n)$, come dimostra la seguente analisi.

Prima dimostriamo che la lunghezza prevista della serie più lunga di teste è $O(\lg n)$. La probabilità che in un singolo lancio si ottenga testa è pari a $1/2$. Indichiamo con A_{ik} l'evento in cui una serie di teste di lunghezza almeno uguale a k inizi con l'i-esimo lancio della moneta o, più precisamente, l'evento in cui per k lanci consecutivi $i, i+1, \ldots, i+k-1$ si presenti sempre testa, dove $1 \le k \le n$ e $1 \le i \le n-k+1$. Poiché i lanci della moneta sono mutuamente indipendenti, per un dato evento A_{ik}, la probabilità che in tutti i k lanci si ottenga testa è

$$\Pr\{A_{ik}\} = 1/2^k \tag{5.8}$$

Per $k = 2\lceil \lg n \rceil$, si ha

$$
\begin{aligned}
\Pr\{A_{i,2\lceil \lg n \rceil}\} &= 1/2^{2\lceil \lg n \rceil} \\
&\le 1/2^{2 \lg n} \\
&= 1/n^2
\end{aligned}
$$

Quindi la probabilità che una serie di teste di lunghezza almeno pari a $2\lceil \lg n \rceil$ inizi nella posizione i è molto piccola. Ci sono al massimo $n - 2\lceil \lg n \rceil + 1$ posizioni dove tale serie può iniziare. Pertanto, la probabilità che una serie di teste di lunghezza almeno pari a $2\lceil \lg n \rceil$ inizi in qualsiasi posizione è

$$
\begin{aligned}
\Pr\left\{ \bigcup_{i=1}^{n-2\lceil \lg n \rceil + 1} A_{i,2\lceil \lg n \rceil} \right\} &\le \sum_{i=1}^{n-2\lceil \lg n \rceil + 1} 1/n^2 \\
&< \sum_{i=1}^{n} 1/n^2 \\
&= 1/n
\end{aligned} \tag{5.9}
$$

in quanto, per la disuguaglianza di Boole (C.19), la probabilità dell'unione di più eventi è pari al massimo alla somma delle probabilità dei singoli eventi (notate che la disuguaglianza di Boole è valida anche per eventi come questi che non sono indipendenti).

Acquistato da Michele Michele su Webster il 2022-[illegible]

Adesso utilizziamo la disuguaglianza (5.9) per limitare la lunghezza della serie più lunga. Per $j = 0, 1, 2, \ldots, n$, indichiamo con L_j l'evento in cui la più lunga serie di teste abbia una lunghezza esattamente pari a j e con L la lunghezza della serie più lunga. Per la definizione del valore atteso, si ha

$$\mathrm{E}[L] = \sum_{j=0}^{n} j \Pr\{L_j\} \tag{5.10}$$

Potremmo provare a calcolare questa sommatoria utilizzando dei limiti superiori per ogni $\Pr\{L_j\}$ simili a quelli calcolati nella disuguaglianza (5.9). Purtroppo questo metodo genera limiti deboli. Tuttavia, possiamo sfruttare la precedente analisi per ottenere un buon limite. Informalmente, osserviamo che per nessun termine della sommatoria nell'equazione (5.10) i fattori j e $\Pr\{L_j\}$ sono entrambi grandi. Perché? Quando $j \geq 2\lceil \lg n \rceil$, allora $\Pr\{L_j\}$ è molto piccolo; quando $j < 2\lceil \lg n \rceil$, allora j è piuttosto piccolo. Più formalmente, notiamo che gli eventi L_j per $j = 0, 1, \ldots, n$ sono disgiunti; quindi la probabilità che una serie di teste di lunghezza almeno pari a $2\lceil \lg n \rceil$ inizi in qualsiasi posizione è $\sum_{j=2\lceil \lg n \rceil}^{n} \Pr\{L_j\}$. Per la disuguaglianza (5.9), abbiamo $\sum_{j=2\lceil \lg n \rceil}^{n} \Pr\{L_j\} < 1/n$. Inoltre, notando che $\sum_{j=0}^{n} \Pr\{L_j\} = 1$, allora $\sum_{j=0}^{2\lceil \lg n \rceil - 1} \Pr\{L_j\} \leq 1$. Quindi, otteniamo

$$\begin{aligned}
\mathrm{E}[L] &= \sum_{j=0}^{n} j \Pr\{L_j\} \\
&= \sum_{j=0}^{2\lceil \lg n \rceil - 1} j \Pr\{L_j\} + \sum_{j=2\lceil \lg n \rceil}^{n} j \Pr\{L_j\} \\
&< \sum_{j=0}^{2\lceil \lg n \rceil - 1} (2\lceil \lg n \rceil) \Pr\{L_j\} + \sum_{j=2\lceil \lg n \rceil}^{n} n \Pr\{L_j\} \\
&= 2\lceil \lg n \rceil \sum_{j=0}^{2\lceil \lg n \rceil - 1} \Pr\{L_j\} + n \sum_{j=2\lceil \lg n \rceil}^{n} \Pr\{L_j\} \\
&< 2\lceil \lg n \rceil \cdot 1 + n \cdot (1/n) \\
&= O(\lg n)
\end{aligned}$$

Le probabilità che una serie di teste superi $r\lceil \lg n \rceil$ lanci diminuiscono rapidamente con r. Per $r \geq 1$, la probabilità che una serie di $r\lceil \lg n \rceil$ teste inizi nella posizione i è

$$\begin{aligned}
\Pr\{A_{i, r\lceil \lg n \rceil}\} &= 1/2^{r\lceil \lg n \rceil} \\
&\leq 1/n^r
\end{aligned}$$

Quindi, la probabilità che la serie più lunga sia almeno $r\lceil \lg n \rceil$ è al massimo $n/n^r = 1/n^{r-1}$ ovvero la probabilità che la serie più lunga abbia una lunghezza minore di $r\lceil \lg n \rceil$ è almeno $1 - 1/n^{r-1}$.

Per esempio, se i lanci della moneta sono $n = 1000$, la probabilità di avere una serie di almeno $2\lceil \lg n \rceil = 20$ teste è al massimo $1/n = 1/1000$. Le probabilità di avere una serie più lunga di $3\lceil \lg n \rceil = 30$ teste è al massimo $1/n^2 = 1/1\,000\,000$.

Adesso dimostriamo un limite inferiore complementare: la lunghezza prevista della serie più lunga di teste in n lanci della moneta è $\Omega(\lg n)$. Per provare questo limite, cerchiamo le serie di lunghezza s suddividendo gli n lanci in circa n/s gruppi, ciascuno di s lanci. Se scegliamo $s = \lfloor (\lg n)/2 \rfloor$, possiamo dimostrare che è probabile che almeno uno di questi gruppi abbia soltanto teste e, quindi, è probabile che la serie più lunga abbia una lunghezza almeno pari a $s = \Omega(\lg n)$. Poi dimostreremo che la serie più lunga ha una lunghezza attesa pari a $\Omega(\lg n)$.

Acquistato da Michele Michelassi (michelassi@gmail.com) - Proprietà letteraria riservata - © 2010 The McGraw-Hill Education (Italy)

Suddividiamo gli n lanci della moneta in almeno $\lfloor n/\lfloor(\lg n)/2\rfloor\rfloor$ gruppi di $\lfloor(\lg n)/2\rfloor$ lanci consecutivi e limitiamo la probabilità che nessun gruppo possa avere soltanto teste. Per l'equazione (5.8), la probabilità che il gruppo che inizia nella posizione i abbia soltanto teste è

$$\begin{aligned}\Pr\{A_{i,\lfloor(\lg n)/2\rfloor}\} &= 1/2^{\lfloor(\lg n)/2\rfloor} \\ &\geq 1/\sqrt{n}\end{aligned}$$

La probabilità che nella posizione i non inizi una serie di teste di lunghezza almeno pari a $\lfloor(\lg n)/2\rfloor$ è, quindi, al massimo $1 - 1/\sqrt{n}$. Poiché gli $\lfloor n/\lfloor(\lg n)/2\rfloor\rfloor$ gruppi sono disgiunti e formati da lanci indipendenti, la probabilità che ciascuno di questi gruppi *non sia* una serie di lunghezza $\lfloor(\lg n)/2\rfloor$ è al massimo

$$\begin{aligned}\left(1 - 1/\sqrt{n}\right)^{\lfloor n/\lfloor(\lg n)/2\rfloor\rfloor} &\leq \left(1 - 1/\sqrt{n}\right)^{n/\lfloor(\lg n)/2\rfloor - 1} \\ &\leq \left(1 - 1/\sqrt{n}\right)^{2n/\lg n - 1} \\ &\leq e^{-(2n/\lg n - 1)/\sqrt{n}} \\ &= O(e^{-\lg n}) \\ &= O(1/n)\end{aligned}$$

Qui abbiamo utilizzato la disuguaglianza (3.12), $1 + x \leq e^x$, e ci siamo basati sul fatto (che dovreste verificare) che $(2n/\lg n - 1)/\sqrt{n} \geq \lg n$ per valori di n sufficientemente grandi. Ne consegue che la probabilità che la serie più lunga superi $\lfloor(\lg n)/2\rfloor$ è

$$\sum_{j=\lfloor(\lg n)/2\rfloor+1}^{n} \Pr\{L_j\} \geq 1 - O(1/n) \tag{5.11}$$

Adesso possiamo calcolare un limite inferiore per la lunghezza prevista della serie più lunga, iniziando con l'equazione (5.10) e procedendo in modo analogo all'analisi del limite superiore:

$$\begin{aligned}\mathrm{E}[L] &= \sum_{j=0}^{n} j \Pr\{L_j\} \\ &= \sum_{j=0}^{\lfloor(\lg n)/2\rfloor} j \Pr\{L_j\} + \sum_{j=\lfloor(\lg n)/2\rfloor+1}^{n} j \Pr\{L_j\} \\ &\geq \sum_{j=0}^{\lfloor(\lg n)/2\rfloor} 0 \cdot \Pr\{L_j\} + \sum_{j=\lfloor(\lg n)/2\rfloor+1}^{n} \lfloor(\lg n)/2\rfloor \Pr\{L_j\} \\ &= 0 \cdot \sum_{j=0}^{\lfloor(\lg n)/2\rfloor} \Pr\{L_j\} + \lfloor(\lg n)/2\rfloor \sum_{j=\lfloor(\lg n)/2\rfloor+1}^{n} \Pr\{L_j\} \\ &\geq 0 + \lfloor(\lg n)/2\rfloor\,(1 - O(1/n)) \qquad \text{(per la disuguaglianza (5.11))} \\ &= \Omega(\lg n)\end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Analogamente al paradosso del compleanno, possiamo svolgere un'analisi più semplice, ma approssimata, utilizzando le variabili casuali indicatrici. Sia $X_{ik} = \mathrm{I}\{A_{ik}\}$ la variabile casuale indicatrice associata alla serie di teste di lunghezza almeno pari a k che inizia con l'i-esimo lancio della moneta. Per contare il numero totale di tali sequenze, definiamo

$$X = \sum_{i=1}^{n-k+1} X_{ik}$$

Prendendo i valori attesi da entrambi i membri dell'equazione e applicando la linearità del valore atteso, otteniamo

$$\begin{aligned}
\mathrm{E}[X] &= \mathrm{E}\left[\sum_{i=1}^{n-k+1} X_{ik}\right] \\
&= \sum_{i=1}^{n-k+1} \mathrm{E}[X_{ik}] \\
&= \sum_{i=1}^{n-k+1} \Pr\{A_{ik}\} \\
&= \sum_{i=1}^{n-k+1} 1/2^k \\
&= \frac{n-k+1}{2^k}
\end{aligned}$$

Sostituendo i vari valori di k, possiamo calcolare il numero atteso di serie di lunghezza k. Se questo numero è grande (molto più grande di 1), allora si prevedono molte serie di lunghezza k e la probabilità che se ne verifichi una è alta. Se questo numero è piccolo (molto più piccolo di 1), allora si prevedono pochissime serie di lunghezza k e la probabilità che se ne verifichi una è bassa. Se $k = c \lg n$, per qualche costante positiva c, otteniamo

$$\begin{aligned}
\mathrm{E}[X] &= \frac{n - c\lg n + 1}{2^{c\lg n}} \\
&= \frac{n - c\lg n + 1}{n^c} \\
&= \frac{1}{n^{c-1}} - \frac{(c\lg n - 1)/n}{n^{c-1}} \\
&= \Theta(1/n^{c-1})
\end{aligned}$$

Se c è grande, il numero atteso di serie di lunghezza $c \lg n$ è molto piccolo e concludiamo che è poco probabile che tali serie si verifichino. D'altra parte, se $c < 1/2$, allora otteniamo $\mathrm{E}[X] = \Theta(1/n^{1/2-1}) = \Theta(n^{1/2})$ e prevediamo che ci sarà un numero elevato di serie di lunghezza $(1/2)\lg n$. Pertanto, è molto probabile che si verifichi una serie di tale lunghezza. Anche soltanto con queste stime approssimative possiamo concludere che la lunghezza prevista della serie più lunga è $\Theta(\lg n)$.

5.4.4 Il problema delle assunzioni online

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Come esempio finale, consideriamo una variante del problema delle assunzioni. Supponiamo adesso di non volere intervistare tutti i candidati per selezionare il migliore. Inoltre, non vogliamo assumere e licenziare ogni volta che troviamo un candidato migliore del precedente. In cambio, siamo disposti ad accontentarci di un candidato che sia soltanto vicino al migliore possibile, col vantaggio di fare esattamente una sola assunzione. Bisogna rispettare una regola dell'azienda:

dopo ogni colloquio, dobbiamo offrire immediatamente il lavoro al candidato o dobbiamo dirgli che non lo avrà. Qual è il miglior compromesso fra minimizzare il numero di colloqui e massimizzare la qualità del candidato assunto?

Possiamo modellare questo problema nel modo seguente. Dopo avere incontrato un candidato, siamo in grado di assegnargli un punteggio. Indichiamo con $score(i)$ il punteggio dato all'i-esimo candidato; due candidati non possono avere lo stesso punteggio. Dopo avere incontrato j candidati, conosciamo il candidato con il punteggio più alto, ma non sappiamo se uno dei restanti $n-j$ candidati avrà un punteggio più alto. Decidiamo di adottare la strategia di scegliere un numero intero positivo $k < n$, incontrare e scartare i primi k candidati e, in seguito, assumere il primo candidato che ha un punteggio più alto di tutti i candidati precedenti. Se scopriamo che il candidato più qualificato era fra i primi k precedentemente incontrati, allora assumeremo l'n-esimo candidato. Questa strategia è formalizzata nella seguente procedura ON-LINE-MAXIMUM(k, n), che restituisce l'indice del candidato che intendiamo assumere.

```
ON-LINE-MAXIMUM(k, n)
1  bestscore = −∞
2  for i = 1 to k
3       if score(i) > bestscore
4            bestscore = score(i)
5  for i = k + 1 to n
6       if score(i) > bestscore
7            return i
8  return n
```

Intendiamo determinare, per ogni possibile valore di k, la probabilità che abbiamo di assumere il candidato più qualificato. Poi sceglieremo il miglior valore possibile di k e implementeremo la strategia con questo valore. Per il momento, supponiamo che k sia fisso. Indichiamo con $M(j) = \max_{1 \le i \le j} \{score(i)\}$ il punteggio massimo fra i candidati da 1 a j. Sia S l'evento in cui riusciamo a selezionare il candidato più qualificato; sia S_i l'evento in cui ciò succede quando il più qualificato è l'i-esimo candidato che abbiamo incontrato. Poiché i vari eventi S_i sono disgiunti, abbiamo $\Pr\{S\} = \sum_{i=1}^{n} \Pr\{S_i\}$. Dal momento che non riusciremo mai a selezionare il candidato più qualificato quando questo è fra i primi k, abbiamo $\Pr\{S_i\} = 0$ per $i = 1, 2, \ldots, k$. Quindi, otteniamo

$$\Pr\{S\} = \sum_{i=k+1}^{n} \Pr\{S_i\} \tag{5.12}$$

Calcoliamo adesso $\Pr\{S_i\}$. Per riuscire a selezionare il candidato più qualificato quando questo è l'i-esimo candidato, devono accadere due cose. In primo luogo, il candidato più qualificato deve trovarsi nella posizione i, un evento che indicheremo con B_i. In secondo luogo, l'algoritmo non deve selezionare nessun candidato nelle posizioni da $k+1$ a $i-1$, questo si verifica soltanto se, per ogni j tale che $k+1 \le j \le i-1$, si ha che $score(j) < bestscore$ nella riga 6 (poiché i punteggi sono distinti, possiamo ignorare la possibilità che $score(j) = bestscore$). In altre parole, deve accadere che tutti i valori da $score(k+1)$ a $score(i-1)$ sono minori di $M(k)$; se, invece, un valore è maggiore di $M(k)$, sarà restituito l'indice del primo valore che è maggiore. Indichiamo con O_i l'evento in cui nessuno dei candidati nelle posizioni da $k+1$ a $i-1$ sia scelto. Fortunatamente, i due eventi

Acquistato da Michele Michele su Webster il 2022-[illegible]

B_i e O_i sono indipendenti. L'evento O_i dipende soltanto dall'ordinamento relativo dei valori nelle posizioni da 1 a $i-1$, mentre B_i dipende soltanto dal fatto che il valore nella posizione i sia maggiore dei valori in tutte le altre posizioni. L'ordinamento dei valori nelle posizioni da 1 a $i-1$ non dipende dal fatto che il valore nella posizione i sia maggiore di tutti questi valori e il valore nella posizione i non influisce sull'ordinamento dei valori nelle posizioni da 1 a $i-1$. Quindi, possiamo applicare l'equazione (C.15) per ottenere

$$\Pr\{S_i\} = \Pr\{B_i \cap O_i\} = \Pr\{B_i\}\Pr\{O_i\}$$

La probabilità $\Pr\{B_i\}$ è chiaramente $1/n$, in quanto il valore massimo ha uguale probabilità di trovarsi in una qualsiasi delle n posizioni. Affinché possa verificarsi l'evento O_i, il valore massimo delle posizioni da 1 a $i-1$ deve trovarsi in una delle prime k posizioni ed ha uguale probabilità di trovarsi in una di queste $i-1$ posizioni. Di conseguenza, $\Pr\{O_i\} = k/(i-1)$ e $\Pr\{S_i\} = k/(n(i-1))$. Applicando l'equazione (5.12), otteniamo

$$\begin{aligned}
\Pr\{S\} &= \sum_{i=k+1}^{n} \Pr\{S_i\} \\
&= \sum_{i=k+1}^{n} \frac{k}{n(i-1)} \\
&= \frac{k}{n} \sum_{i=k+1}^{n} \frac{1}{i-1} \\
&= \frac{k}{n} \sum_{i=k}^{n-1} \frac{1}{i}
\end{aligned}$$

Approssimiamo con gli integrali per limitare superiormente e inferiormente questa sommatoria. Per la disuguaglianza (A.12), abbiamo

$$\int_k^n \frac{1}{x}\,dx \le \sum_{i=k}^{n-1} \frac{1}{i} \le \int_{k-1}^{n-1} \frac{1}{x}\,dx$$

Calcolando questi integrali definiti, otteniamo

$$\frac{k}{n}(\ln n - \ln k) \le \Pr\{S\} \le \frac{k}{n}(\ln(n-1) - \ln(k-1))$$

che rappresentano dei limiti abbastanza stretti per $\Pr\{S\}$. Poiché intendiamo massimizzare la nostra probabilità di successo, concentriamo l'attenzione sulla scelta del valore k che massimizza il limite inferiore su $\Pr\{S\}$ (l'espressione del limite inferiore è più semplice da massimizzare dell'espressione del limite superiore). Derivando l'espressione $(k/n)(\ln n - \ln k)$ rispetto a k, otteniamo

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\frac{1}{n}(\ln n - \ln k - 1)$$

Ponendo questa derivata uguale a 0, notiamo che il limite inferiore sulla probabilità è massimizzato quando $\ln k = \ln n - 1 = \ln(n/e)$ ovvero quando $k = n/e$. Quindi, se implementiamo la nostra strategia con $k = n/e$, riusciremo ad assumere il candidato più qualificato con una probabilità pari almeno a $1/e$.

Esercizi

5.4-1

Quante persone devono trovarsi in una stanza prima che ci sia una probabilità almeno pari a $1/2$ che una di esse sia nata nel vostro stesso giorno dell'anno? Quante persone devono esserci prima che la probabilità che almeno due persone festeggino il compleanno il 4 luglio sia maggiore di $1/2$?

5.4-2

Supponete che delle palline siano lanciate in b contenitori fino a che qualche contenitore contenga due palline. Ogni lancio è indipendente e ogni pallina ha la stessa probabilità di finire in qualsiasi contenitore. Qual è il numero atteso di lanci?

5.4-3 ⋆

Per l'analisi del paradosso del compleanno, è necessario che i compleanni siano mutuamente indipendenti o è sufficiente che siano indipendenti a coppie? Spiegate la vostra risposta.

5.4-4 ⋆

Quante persone dovreste invitare a una festa per avere una buona probabilità di trovarne *tre* nate nello stesso giorno dell'anno?

5.4-5 ⋆

Qual è la probabilità che una stringa di lunghezza k su un insieme di dimensione n sia una k-permutazione? Qual è la relazione fra questa domanda e il paradosso del compleanno?

5.4-6 ⋆

Supponete che n palline siano lanciate in n contenitori; ogni lancio è indipendente e una pallina ha la stessa probabilità di finire in qualsiasi contenitore. Qual è il numero atteso di contenitori vuoti? Qual è il numero atteso di contenitori con una sola pallina?

5.4-7 ⋆

Affinate il limite inferiore sulla lunghezza di una serie dello stesso evento dimostrando che, in n lanci di una moneta, la probabilità che nessuna serie più lunga di $\lg n - 2\lg\lg n$ teste consecutive possa verificarsi è minore di $1/n$.

Problemi

5-1 Contatore probabilistico

Con un contatore ordinario a b bit è possibile contare soltanto fino a $2^b - 1$. Con il ***contatore probabilistico*** di R. Morris è possibile contare fino a un valore molto più grande, perdendo un po' di precisione.

Indichiamo con n_i il conteggio rappresentato con il valore i del contatore per $i = 0, 1, \ldots, 2^b - 1$, dove n_i forma una sequenza crescente di valori non negativi.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: ib9780262046305 01 Copyright © 2022, McGraw-Hill Education (Italy)

Supponiamo che il valore iniziale del contatore sia 0, che rappresenta il conteggio di $n_0 = 0$.

La procedura INCREMENT opera in modo probabilistico su di un contatore che contiene il valore i. Se $i = 2^b - 1$, allora viene segnalato un errore di overflow, altrimenti il contatore viene incrementato di 1 con probabilità $1/(n_{i+1} - n_i)$ e rimane inalterato con probabilità $1 - 1/(n_{i+1} - n_i)$. Se scegliamo $n_i = i$ per ogni

$i \geq 0$, allora il contatore è ordinario. Situazioni più interessanti si presentano se scegliamo, per esempio, $n_i = 2^{i-1}$ per $i > 0$ o $n_i = F_i$ (l'i-esimo numero di Fibonacci – vedere il Paragrafo 3.2).

Per questo problema, supponiamo che n_{2^b-1} sia sufficientemente grande da poter trascurare la probabilità di un errore di overflow.

a. Dimostrate che il valore atteso rappresentato dal contatore dopo n operazioni di INCREMENT è esattamente pari a n.

b. L'analisi della varianza del valore rappresentato dal contatore dipende dalla sequenza di n_i. Consideriamo un semplice caso: $n_i = 100i$ per ogni $i \geq 0$. Calcolate la varianza nel valore rappresentato dal contatore dopo n operazioni di INCREMENT.

5-2 Ricerca in un array non ordinato

Questo problema esamina tre algoritmi che ricercano un valore x in un array A di n elementi non ordinati.

Consideriamo la seguente strategia randomizzata: scegliamo un indice casuale i in A. Se $A[i] = x$, terminiamo la ricerca, altrimenti continuiamo a cercare scegliendo un nuovo indice casuale in A. Continuiamo a scegliere indici casuali in A finché non avremo trovato un indice j tale che $A[j] = x$ o finché non avremo esaminato tutti gli elementi di A. Notate che l'indice ogni volta viene scelto dall'intero insieme degli indici, quindi lo stesso elemento potrebbe essere esaminato più volte.

a. Scrivete la pseudocodifica per una procedura RANDOM-SEARCH che implementa la precedente strategia. Verificate che il vostro algoritmo termini dopo che sono stati selezionati tutti gli indici in A.

b. Supponete che ci sia un solo indice i tale che $A[i] = x$. Qual è il numero atteso di indici in A che devono essere selezionati prima che x venga trovato e la procedura RANDOM-SEARCH termini?

c. Generalizzando la soluzione che avete ottenuto nel punto (b), supponete che ci siano $k \geq 1$ indici i tali che $A[i] = x$. Qual è il numero atteso di indici in A che devono essere selezionati prima che x venga trovato e la procedura RANDOM-SEARCH termini? La risposta dovrebbe essere una funzione di n e k.

d. Supponete che non ci siano indici i tali che $A[i] = x$. Qual è il numero atteso di indici in A che devono essere selezionati prima che tutti gli elementi di A siano stati esaminati e la procedura RANDOM-SEARCH termini?

Consideriamo ora un algoritmo di ricerca lineare deterministica, che chiameremo DETERMINISTIC-SEARCH. L'algoritmo ricerca x in A, esaminando ordinatamente $A[1], A[2], A[3], \ldots, A[n]$ finché trova $A[i] = x$ o raggiunge la fine dell'array. Supponete che tutte le possibili permutazioni dell'array di input siano ugualmente probabili.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

e. Supponete che ci sia un solo indice i tale che $A[i] = x$. Qual è il tempo di esecuzione nel caso medio di DETERMINISTIC-SEARCH? Qual è il tempo di esecuzione nel caso peggiore di DETERMINISTIC-SEARCH?

f. Generalizzando la soluzione che avete ottenuto nel punto (e), supponete che ci siano $k \geq 1$ indici i tali che $A[i] = x$. Qual è il tempo di esecuzione nel

caso medio di DETERMINISTIC-SEARCH? Qual è il tempo di esecuzione nel caso peggiore di DETERMINISTIC-SEARCH? La risposta dovrebbe essere una funzione di n e k.

g. Supponete che non ci siano indici i tali che $A[i] = x$. Qual è il tempo di esecuzione nel caso medio di DETERMINISTIC-SEARCH? Qual è il tempo di esecuzione nel caso peggiore di DETERMINISTIC-SEARCH?

Infine, consideriamo l'algoritmo randomizzato SCRAMBLE-SEARCH che opera in questo modo: prima permuta casualmente l'array di input e poi svolge la ricerca lineare deterministica (precedentemente descritta) nell'array permutato.

h. Indicando con k il numero di indici i tali che $A[i] = x$, determinate il tempo di esecuzione nel caso peggiore e quello atteso di SCRAMBLE-SEARCH nei casi in cui $k = 0$ e $k = 1$. Generalizzate la vostra soluzione per includere il caso in cui $k \geq 1$.

i. Quale di questi tre algoritmi di ricerca utilizzereste? Perché?

Note

Bollobás [54], Hofri [175] e Spencer [322] descrivono numerose tecniche avanzate di analisi probabilistica. I vantaggi degli algoritmi randomizzati sono stati analizzati e classificati da Karp [201] e Rabin [289]. Il libro di testo di Motwani e Raghavan [263] tratta diffusamente gli algoritmi randomizzati.

Sono state studiate numerose varianti del problema delle assunzioni. Questo tipo di problema è più comunemente noto come "problema del segretario". Uno studio in quest'area è stato fatto da Ajtai, Meggido e Waarts [11].

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

II Ordinamento e statistiche d'ordine

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Introduzione

Questa parte presenta vari algoritmi che risolvono il seguente ***problema di ordinamento***:

Input: una sequenza di n numeri $\langle a_1, a_2, \ldots, a_n \rangle$.

Output: una permutazione (riarrangiamento) $\langle a'_1, a'_2, \ldots, a'_n \rangle$ della sequenza di input tale che $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

La sequenza di input di solito è un array di n elementi, anche se può essere rappresentata in qualche altra forma, come una lista.

La struttura dei dati

In pratica, i numeri da ordinare raramente sono valori isolati. Ogni numero di solito appartiene a una raccolta di dati detta ***record***. Ogni record contiene una ***chiave***, che è il valore da ordinare; la parte restante del record è composta da ***dati satelliti***, che di solito si muovono con la chiave. In pratica, quando un algoritmo di ordinamento permuta le chiavi, deve permutare anche i dati satelliti. Se ogni record include una grande quantità di dati satelliti, spesso permutiamo un array di puntatori ai record, anziché i record stessi, per minimizzare gli spostamenti dei dati.

In un certo senso, sono questi dettagli implementativi che distinguono un algoritmo da un programma vero e proprio. Ordinare singoli numeri o grandi record che contengono numeri è irrilevante per il *metodo* in base al quale una procedura di ordinamento determina l'ordine degli elementi. Pertanto, quando analizziamo un problema di ordinamento, tipicamente supponiamo che l'input sia composto soltanto da numeri. La conversione di un algoritmo che ordina numeri in un programma che ordina record è concettualmente semplice, sebbene in determinati casi alcuni dettagli tecnici potrebbero complicare la programmazione effettiva.

Acquistato da Michele Mastrolia (michele.mastrolia@studenti.unimi.it) - 10.15.80.5 - Copyright © 2010 The McGraw-Hill Education (Italy)

Perché ordinare?

Molti esperti di informatica considerano l'ordinamento come il problema fondamentale nello studio degli algoritmi. Le ragioni sono diverse:

- A volte l'esigenza di ordinare le informazioni è inerente a un'applicazione. Per esempio, per preparare l'estratto conto dei clienti, le banche hanno bisogno di ordinare gli assegni in base ai numeri.
- Gli algoritmi spesso usano l'ordinamento come una subroutine fondamentale. Per esempio, un programma che visualizza sullo schermo degli oggetti grafici che si sovrappongono uno sull'altro potrebbe avere bisogno di ordinare gli oggetti in base a una determinata relazione di "precedenza di rappresentazione", in modo che possa disegnare gli oggetti dall'ultimo al primo. In questo libro esamineremo molti algoritmi che usano l'ordinamento come subroutine.
- Esistono vari tipi di algoritmi di ordinamento, che impiegano un'ampia gamma di tecniche. In effetti, molte delle principali tecniche utilizzate nella progettazione degli algoritmi sono esemplificate negli algoritmi di ordinamento che sono stati sviluppati nel corso degli anni. Da questo punto di vista, l'ordinamento è anche un problema di interesse storico.
- L'ordinamento è un problema per il quale possiamo dimostrare un limite inferiore non banale (come vedremo nel Capitolo 8). I nostri migliori limiti superiori coincidono asintoticamente con il limite inferiore, quindi sappiamo che i nostri algoritmi di ordinamento sono asintoticamente ottimali. Inoltre, possiamo utilizzare il limite inferiore dell'ordinamento per dimostrare dei limiti inferiori per certi altri problemi.
- Molti problemi di ingegneria informatica emergono durante l'implementazione degli algoritmi di ordinamento. Quale sia il più veloce programma di ordinamento per una particolare situazione può dipendere da vari fattori, come informazioni preventive sulle chiavi e sui dati satelliti, la gerarchia della memoria (cache e memoria virtuale) del calcolatore usato e l'ambiente software. Molti di questi problemi si risolvono meglio a livello degli algoritmi, anziché "aggiustando" il codice.

Algoritmi di ordinamento

Nel Capitolo 2 abbiamo introdotto due algoritmi che ordinano n numeri reali. L'algoritmo insertion sort richiede un tempo $\Theta(n^2)$ nel caso peggiore. Tuttavia, poiché i suoi cicli interni sono compatti, è un algoritmo rapido di ordinamento sul posto per input di piccole dimensioni (ricordiamo che un algoritmo ordina ***sul posto*** soltanto se in ogni istante al più un numero costante di elementi dell'array di input sono registrati all'esterno dell'array). L'algoritmo merge sort ha un tempo di esecuzione asintotico migliore, $\Theta(n \lg n)$, ma la procedura MERGE che esso usa non opera sul posto.

In questa parte, introdurremo altri due algoritmi che ordinano numeri reali arbitrari. Heapsort, presentato nel Capitolo 6, ordina n numeri sul posto nel tempo $O(n \lg n)$; usa un'importante struttura dati, detta heap, con la quale è possibile implementare anche una coda di priorità.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995030016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Anche quicksort, presentato nel Capitolo 7, ordina n numeri sul posto, ma il suo tempo di esecuzione nel caso peggiore è $\Theta(n^2)$. Il suo tempo di esecuzione nel caso medio, però, è $\Theta(n \lg n)$ e, in generale, è migliore di heapsort. Come insertion sort, anche quicksort ha un codice compatto, quindi il fattore costante nascosto nel suo tempo di esecuzione è piccolo. È un algoritmo popolare per ordinare grandi array di input.

Insertion sort, merge sort, heapsort e quicksort sono tutti ordinamenti per confronti: determinano l'ordinamento di un array di input confrontando gli elementi. Il Capitolo 8 inizia presentando il modello computazionale dell'albero delle decisioni che consente di studiare le limitazioni delle prestazioni degli ordinamenti per confronti. Applicando questo modello, dimostreremo un limite inferiore pari a $\Omega(n \lg n)$ sul tempo di esecuzione nel caso peggiore di qualsiasi ordinamento per confronti di n input, dimostrando quindi che heapsort e merge sort sono ordinamenti per confronti asintoticamente ottimali.

Il Capitolo 8 continua dimostrando che possiamo battere il limite inferiore $\Omega(n \lg n)$, se riusciamo ad acquisire informazioni sull'ordinamento dell'input con un metodo che non è basato sui confronti degli elementi. L'algoritmo counting sort, per esempio, suppone che i numeri di input appartengano all'insieme $\{1, 2, \ldots, k\}$. Utilizzando l'indicizzazione dell'array come strumento per determinare l'ordine relativo degli elementi, counting sort può ordinare n numeri nel tempo $\Theta(k + n)$. Quindi, se $k = O(n)$, counting sort viene eseguito in un tempo che è lineare nella dimensione dell'array di input. È possibile utilizzare un algoritmo correlato, radix sort, per estendere il campo di applicazione di counting sort. Se ci sono n interi da ordinare, ogni intero ha d cifre e ogni cifra appartiene all'insieme $\{1, 2, \ldots, k\}$, allora radix sort può ordinare i numeri nel tempo $\Theta(d(n + k))$. Se d è una costante e k è $O(n)$, radix sort viene eseguito in tempo lineare. Un terzo algoritmo, bucket sort, richiede la conoscenza della distribuzione probabilistica dei numeri nell'array di input. Può ordinare n numeri reali uniformemente distribuiti nell'intervallo semiaperto $[0, 1)$ nel tempo $O(n)$ nel caso medio.

La seguente tabella riassume i tempi di esecuzione degli algoritmi di ordinamento descritti nei Capitoli 2 e 6–8. Come sempre, n indica il numero di elementi da ordinare. Per counting sort, gli elementi da ordinare sono interi nell'insieme $\{0, 1, \ldots, k\}$. Per radix sort, ogni elemento è un numero di d cifre, mentre ogni cifra può assumere k valori. Per bucket sort, supponiamo che le chiavi siano numeri reali uniformemente distribuiti nell'intervallo semiaperto $[0, 1)$. La colonna più a destra riporta il tempo di esecuzione atteso o il tempo di esecuzione nel caso medio, indicando quale dei due viene riportato quando differisce dal tempo di esecuzione nel caso peggiore. Omettiamo il tempo di esecuzione nel caso medio di heapsort perché non sarà analizzato in questo libro.

Algoritmo	Tempo di esecuzione nel caso peggiore	Tempo di esecuzione atteso/nel caso medio
Insertion sort	$\Theta(n^2)$	$\Theta(n^2)$
Merge sort	$\Theta(n \lg n)$	$\Theta(n \lg n)$
Heapsort	$O(n \lg n)$	—
Quicksort	$\Theta(n^2)$	$\Theta(n \lg n)$ (atteso)
Counting sort	$\Theta(k + n)$	$\Theta(k + n)$
Radix sort	$\Theta(d(n + k))$	$\Theta(d(n + k))$
Bucket sort	$\Theta(n^2)$	$\Theta(n)$ (caso medio)

Acquistato da Michele Michael su Webster il 2022-07-07 22:42 - Numero Ordine Libreria: 19[illegible]2207[illegible]0 Copyright © 2022, McGraw-Hill Education (Italy)

Statistiche d'ordine

L'i-esima statistica d'ordine di un insieme di n numeri è l'i-esimo numero più piccolo dell'insieme. Ovviamente, è possibile selezionare l'i-esima statistica d'ordine ordinando l'input e prendendo l'i-esimo elemento dell'output. Se non si fanno ipotesi sulla distribuzione dell'input, questo metodo viene eseguito nel tempo $\Omega(n \lg n)$, come dimostra il limite inferiore provato nel Capitolo 8.

Nel Capitolo 9 dimostreremo che è possibile trovare l'i-esimo elemento più piccolo nel tempo $O(n)$, anche quando gli elementi sono numeri reali arbitrari. Presenteremo un algoritmo con uno pseudocodice compatto che viene eseguito nel tempo $\Theta(n^2)$ nel caso peggiore, ma in tempo lineare nel caso medio. Descriveremo anche un algoritmo più complicato che viene eseguito nel tempo $O(n)$ nel caso peggiore.

Prerequisiti

Sebbene molti degli argomenti trattati in questa parte non richiedano calcoli matematici difficili, tuttavia in alcuni casi sono necessarie elaborazioni matematiche sofisticate. In particolare, l'analisi del caso medio di quicksort, bucket sort e l'algoritmo delle statistiche d'ordine applicano la teoria della probabilità, che è trattata nell'Appendice C, i concetti di analisi probabilistica e gli algoritmi randomizzati descritti nel Capitolo 5. L'analisi dell'algoritmo in tempo lineare nel caso peggiore per le statistiche d'ordine richiede passaggi matematici ancora più sofisticati delle altre analisi del caso peggiore che sono svolte in questa parte del libro.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Heapsort 6

In questo capitolo presentiamo un altro algoritmo di ordinamento. Come merge sort, ma diversamente da insertion sort, il tempo di esecuzione di heapsort è $O(n \lg n)$. Come insertion sort, ma diversamente da merge sort, heapsort effettua un ordinamento sul posto: in ogni istante soltanto un numero costante di elementi dell'array sono memorizzati all'esterno dell'array di input. Quindi, heapsort combina i migliori attributi dei due algoritmi di ordinamento che abbiamo già descritto.

Heapsort introduce anche un'altra tecnica di progettazione degli algoritmi: l'uso di una struttura dati, che in questo caso è detta "heap" (mucchio), per gestire le informazioni. Non soltanto la struttura heap è utile per l'heapsort, ma realizza anche un'efficiente coda di priorità. La struttura heap riapparirà negli algoritmi dei capitoli successivi.

Originariamente, il termine "heap" fu coniato nel contesto dell'heapsort, ma da allora è stato utilizzato per fare riferimento al magazzino di stoccaggio della memoria recuperata dai "garbage collector" (spazzini), come quelli forniti dai linguaggi di programmazione Lisp e Java. La nostra struttura heap *non* è magazzino di stoccaggio per il garbage collector e, ogni volta che faremo riferimento agli heap in questo libro, intenderemo una struttura dati e non un aspetto del recupero della memoria.

6.1 Heap

Un ***heap (binario)*** è una struttura dati composta da un array che possiamo considerare come un albero binario quasi completo (vedere il Paragrafo B.5.3), come illustra la Figura 6.1. Ogni nodo dell'albero corrisponde a un elemento dell'array. Tutti i livelli dell'albero sono completamente riempiti, tranne eventualmente l'ultimo che può essere riempito da sinistra fino a un certo punto. Un array A che rappresenta un heap è un oggetto con due attributi: $A.length$ indica il numero di elementi nell'array; $A.heap\text{-}size$ indica il numero degli elementi dell'heap che sono registrati nell'array A. Cioè, anche se ci possono essere dei numeri memorizzati in tutto l'array $A[1 \,..\, A.length]$, soltanto i numeri in $A[1 \,..\, A.heap\text{-}size]$, dove $0 \leq A.heap\text{-}size \leq A.length$, sono elementi validi dell'heap. La radice dell'albero è $A[1]$. Se i è l'indice di un nodo, gli indici di suo padre PARENT(i), del figlio sinistro LEFT(i) e del figlio destro RIGHT(i) possono essere facilmente calcolati:

Acquistato da Michele Mastrovito (mastrovitomichele@gmail.com) per uso esclusivo personale - Copyright © 2010 The McGraw-Hill Education (Italy)

```
PARENT(i)
1  return ⌊i/2⌋

LEFT(i)
1  return 2i

RIGHT(i)
1  return 2i + 1
```

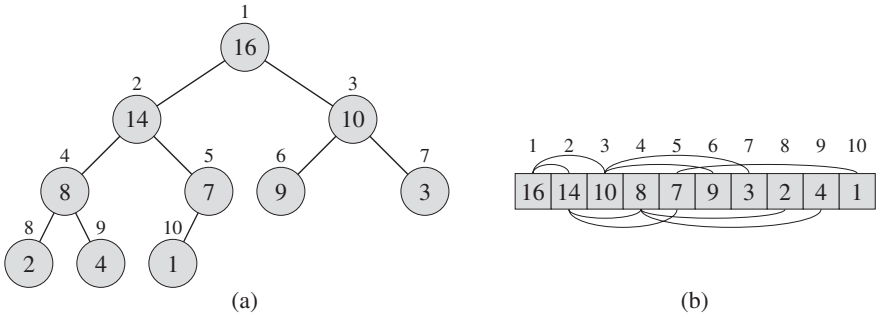


Figura 6.1 Un max-heap visto come **(a)** un albero binario e **(b)** un array. Il numero all'interno del cerchio di un nodo dell'albero è il valore registrato in quel nodo. Il numero sopra un nodo è il corrispondente indice dell'array. Sopra e sotto l'array ci sono delle linee che rappresentano le relazioni padre-figlio; i padri sono sempre a sinistra dei loro figli. L'albero ha altezza 3; il nodo con indice 4 e valore 8 ha altezza 1.

Nella maggior parte dei calcolatori, la procedura LEFT può calcolare $2i$ con una sola istruzione, facendo scorrere semplicemente di una posizione a sinistra la rappresentazione binaria di i. Analogamente, la procedura RIGHT può rapidamente calcolare $2i + 1$, facendo scorrere la rappresentazione binaria di i di una posizione a sinistra e aggiungendo 1 come bit meno significativo. La procedura PARENT può calcolare $\lfloor i/2 \rfloor$ con uno scorrimento di una posizione a destra della rappresentazione di i. In una buona implementazione di heapsort, queste tre procedure sono spesso realizzate come "macro" o procedure "in linea".

Ci sono due tipi di heap binari: max-heap e min-heap. In entrambi i tipi, i valori nei nodi soddisfano una ***proprietà dell'heap***, le cui caratteristiche dipendono dal tipo di heap. In un ***max-heap***, la ***proprietà del max-heap*** è che per ogni nodo i diverso dalla radice, si ha

$$A[\text{PARENT}(i)] \geq A[i]$$

ovvero il valore di un nodo è al massimo il valore di suo padre. Quindi, l'elemento più grande di un max-heap è memorizzato nella radice e il sottoalbero di un nodo contiene valori non maggiori di quello contenuto nel nodo stesso. Un ***min-heap*** è organizzato nel modo opposto; la ***proprietà del min-heap*** è che per ogni nodo i diverso dalla radice, si ha

$$A[\text{PARENT}(i)] \leq A[i]$$

Il più piccolo elemento in un min-heap è nella radice.

Per l'algoritmo heapsort utilizzeremo i max-heap. I min-heap sono talvolta utilizzati nelle code di priorità, che descriveremo nel Paragrafo 6.5. Dovremo specificare con precisione se abbiamo bisogno di un max-heap o di un min-heap per una particolare applicazione; quando le proprietà si applicano sia ai max-heap sia ai min-heap, utilizzeremo semplicemente il termine "heap".

Se osserviamo un heap nella forma di albero, definiamo ***altezza di un nodo*** il numero di archi nel cammino semplice più lungo che dal nodo scende fino a una foglia. Definiamo ***altezza di un heap*** l'altezza della sua radice. Poiché un heap di n elementi è basato su un albero binario completo, la sua altezza è $\Theta(\lg n)$ (vedere l'Esercizio 6.1-2). Vedremo che le operazioni fondamentali sugli heap vengono eseguite in un tempo che è al massimo proporzionale all'altezza dell'albero e, quindi, richiedono un tempo $O(\lg n)$. La parte restante di questo capitolo presenta alcune procedure di base e mostra come sono utilizzate in un algoritmo di ordinamento e in una coda di priorità.

Acquistato da Michele Michele su Webster il 2022-07-[illegible] Tutti i diritti riservati. Duplicazione vietata. Licenza n. 2022010100000[illegible] Copia ad uso esclusivo dell'acquirente.

- La procedura MAX-HEAPIFY, che è eseguita nel tempo $O(\lg n)$, è la chiave per conservare la proprietà del max-heap.
- La procedura BUILD-MAX-HEAP, che è eseguita in tempo lineare, genera un max-heap da un array di input non ordinato.
- La procedura HEAPSORT, che è eseguita nel tempo $O(n \lg n)$, ordina sul posto un array.
- Le quattro procedure MAX-HEAP-INSERT, HEAP-EXTRACT-MAX, HEAP-INCREASE-KEY e HEAP-MAXIMUM, che sono eseguite nel tempo $O(\lg n)$, consentono a un heap di essere utilizzato come una coda di priorità.

Esercizi

6.1-1
Quali sono il numero minimo e il numero massimo di elementi in un heap di altezza h?

6.1-2
Dimostrate che un heap di n elementi ha altezza $\lfloor \lg n \rfloor$.

6.1-3
Dimostrate che in qualsiasi sottoalbero di un max-heap, la radice del sottoalbero contiene il valore più grande fra tutti quelli che compaiono nel sottoalbero.

6.1-4
In un max-heap dove potrebbe risiedere il più piccolo elemento, supponendo che tutti gli elementi siano distinti?

6.1-5
Un array ordinato è un min-heap?

6.1-6
L'array $\langle 23, 17, 14, 6, 13, 10, 1, 5, 7, 12 \rangle$ è un max-heap?

6.1-7
Dimostrate che, nell'array che rappresenta un heap di n elementi, le foglie sono i nodi con indici $\lfloor n/2 \rfloor + 1, \lfloor n/2 \rfloor + 2, \ldots, n$.

6.2 Conservare la proprietà dell'heap

Per mantenere la proprietà di max-heap useremo la procedura MAX-HEAPIFY. I suoi input sono un array A e un indice i dell'array. Quando viene chiamata, MAX-HEAPIFY assume che gli alberi binari con radici in LEFT(i) e RIGHT(i) siano max-heap, ma che $A[i]$ possa essere più piccolo dei suoi figli, violando così la proprietà del max-heap. MAX-HEAPIFY fa "scendere" il valore $A[i]$ nel max-heap in modo che il sottoalbero con radice di indice i diventi un max-heap.

La Figura 6.2 illustra l'azione di MAX-HEAPIFY. A ogni passo, viene determinato il più grande tra gli elementi $A[i]$, $A[\text{LEFT}(i)]$ e $A[\text{RIGHT}(i)]$, il suo indice viene memorizzato in *massimo*. Se $A[i]$ è più grande, allora il sottoalbero con radice nel nodo i è un max-heap e la procedura termina. Altrimenti, uno dei due figli ha l'elemento più grande e $A[i]$ viene scambiato con $A[massimo]$; in questo modo, il nodo i e i suoi figli soddisfano la proprietà del max-heap. Il nodo con indice *massimo*, però, adesso ha il valore originale $A[i]$ e, quindi, il sottoalbero con radice in *massimo* potrebbe violare la proprietà del max-heap. Di conseguen-

Acquistato da Michele Mastrogiovanni (mastrogiovanni.michele@gmail.com) - Copia personale - Proprietà letteraria riservata - © 2023 McGraw-Hill Education (Italy)

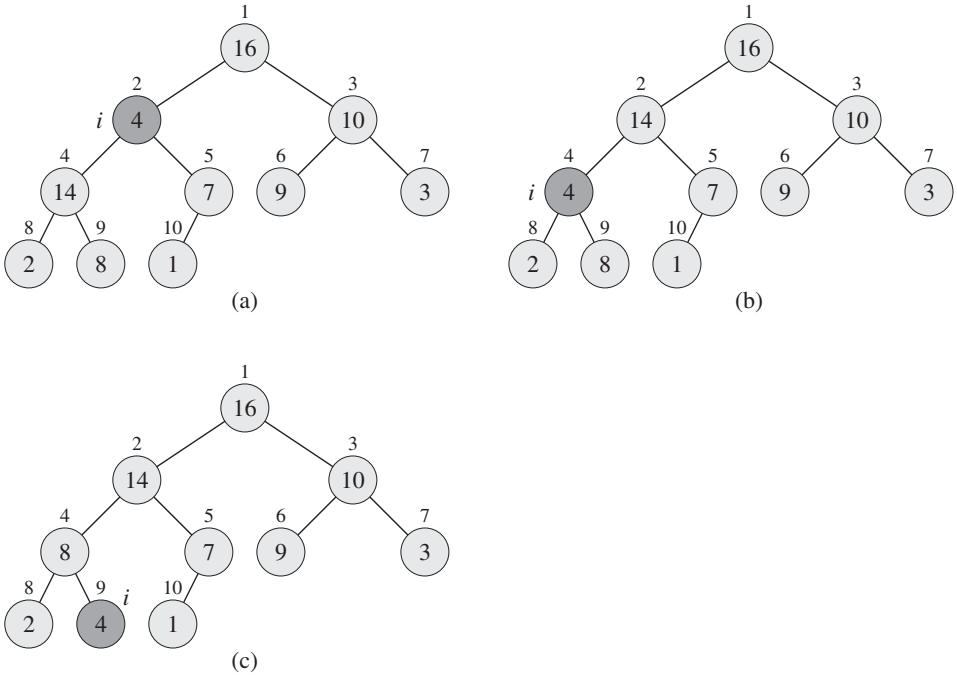


Figura 6.2 L'azione di MAX-HEAPIFY$(A, 2)$, dove $A.heap\text{-}size = 10$. **(a)** La configurazione iniziale, con $A[2]$ nel nodo $i = 2$ che viola la proprietà del max-heap, in quanto non è più grande di entrambi i figli. In **(b)** la proprietà del max-heap viene ripristinata nel nodo 2 scambiando $A[2]$ con $A[4]$; ma questo distrugge la proprietà del max-heap nel nodo 4. La chiamata ricorsiva MAX-HEAPIFY$(A, 4)$ adesso ha $i = 4$. Dopo avere scambiato $A[4]$ con $A[9]$, come illustra **(c)**, il nodo 4 è sistemato e la chiamata ricorsiva MAX-HEAPIFY$(A, 9)$ non apporta ulteriori modifiche alla struttura dati.

za, deve essere chiamata ricorsivamente la subroutine MAX-HEAPIFY per questo sottoalbero.

```
MAX-HEAPIFY(A, i)
 1  l = LEFT(i)
 2  r = RIGHT(i)
 3  if l ≤ A.heap-size and A[l] > A[i]
 4       massimo = l
 5  else massimo = i
 6  if r ≤ A.heap-size and A[r] > A[massimo]
 7       massimo = r
 8  if massimo ≠ i
 9       scambia A[i] con A[massimo]
10       MAX-HEAPIFY(A, massimo)
```

Il tempo di esecuzione di MAX-HEAPIFY in un sottoalbero di dimensione n con radice in un nodo i è pari al tempo $\Theta(1)$ per sistemare le relazioni fra gli elementi $A[i]$, $A[\text{LEFT}(i)]$ e $A[\text{RIGHT}(i)]$, più il tempo per eseguire MAX-HEAPIFY in un sottoalbero con radice in uno dei figli del nodo i (assumendo che venga effettuata la chiamata ricorsiva). I sottoalberi dei figli hanno ciascuno una dimensione che non supera $2n/3$ – il caso peggiore si verifica quando l'ultima riga dell'albero è piena esattamente a metà – e il tempo di esecuzione di MAX-HEAPIFY può quindi essere descritto dalla ricorrenza

Acquistato da Michele Michele su Webster il 2022-07-07 23:43:43 tramite Pearson Italia - 1035e3010-220107-Copyright © 2021 McGraw-Hill Education (Italy)

$$T(n) \leq T(2n/3) + \Theta(1)$$

La soluzione di questa ricorrenza, per il caso 2 del teorema dell'esperto (Teorema 4.1), è $T(n) = O(\lg n)$. In alternativa, possiamo indicare con $O(h)$ il tempo di esecuzione di MAX-HEAPIFY in un nodo di altezza h.

Esercizi

6.2-1

Illustrate l'azione di MAX-HEAPIFY$(A, 3)$ sull'array $A = \langle 27, 17, 3, 16, 13, 10, 1, 5, 7, 12, 4, 8, 9, 0\rangle$, utilizzando la Figura 6.2 come modello.

6.2-2

Usando la procedura MAX-HEAPIFY come modello, scrivete uno pseudocodice per la procedura MIN-HEAPIFY(A, i), che svolge le corrispondenti operazioni con un min-heap. Confrontate i tempi di esecuzione delle procedure MIN-HEAPIFY e MAX-HEAPIFY.

6.2-3

Qual è l'effetto di chiamare MAX-HEAPIFY(A, i) quando l'elemento $A[i]$ è maggiore dei suoi figli?

6.2-4

Qual è l'effetto di chiamare MAX-HEAPIFY(A, i) per $i > A.\textit{heap-size}/2$?

6.2-5

Il codice di MAX-HEAPIFY è molto efficiente in termini di fattori costanti, tranne eventualmente per la chiamata ricorsiva nella riga 10, che potrebbe indurre qualche compilatore a generare un codice inefficiente. Scrivete una procedura MAX-HEAPIFY efficiente che usa un costrutto di controllo iterativo (un ciclo), anziché la ricorsione.

6.2-6

Dimostrate che il tempo di esecuzione nel caso peggiore di MAX-HEAPIFY su un heap di dimensione n è $\Omega(\lg n)$ (*suggerimento:* per un heap con n nodi, assegnate i valori ai nodi in modo che MAX-HEAPIFY sia chiamata ricorsivamente in ogni nodo di un cammino semplice che scende dalla radice fino a una foglia).

6.3 Costruire un heap

Possiamo utilizzare la procedura MAX-HEAPIFY dal basso verso l'alto (bottom-up) per convertire un array $A[1\,..\,n]$, con $n = A.\textit{length}$, in un max-heap. Per l'Esercizio 6.1-7, tutti gli elementi nel sottoarray $A[(\lfloor n/2\rfloor + 1)\,..\,n]$ sono foglie dell'albero e quindi ciascuno di essi è un heap di un solo elemento che possiamo usare come punto di partenza. La procedura BUILD-MAX-HEAP attraversa i restanti nodi dell'albero ed esegue MAX-HEAPIFY in ciascuno di essi.

```
BUILD-MAX-HEAP(A)
1  A.heap-size = A.length
2  for i = ⌊A.length/2⌋ downto 1
3      MAX-HEAPIFY(A, i)
```

La Figura 6.3 illustra un esempio dell'azione di BUILD-MAX-HEAP.

Acquistato da Michele Mastrobuoni in data 02/03/2022 per uso esclusivamente personale - Copyright © 2022, McGraw-Hill Education (Italy)

Per verificare che BUILD-MAX-HEAP funziona correttamente, utilizziamo la seguente invariante di ciclo:

> All'inizio di ogni iterazione del ciclo **for**, righe 2–3, ogni nodo $i + 1$, $i + 2, \ldots, n$ è la radice di un max-heap.

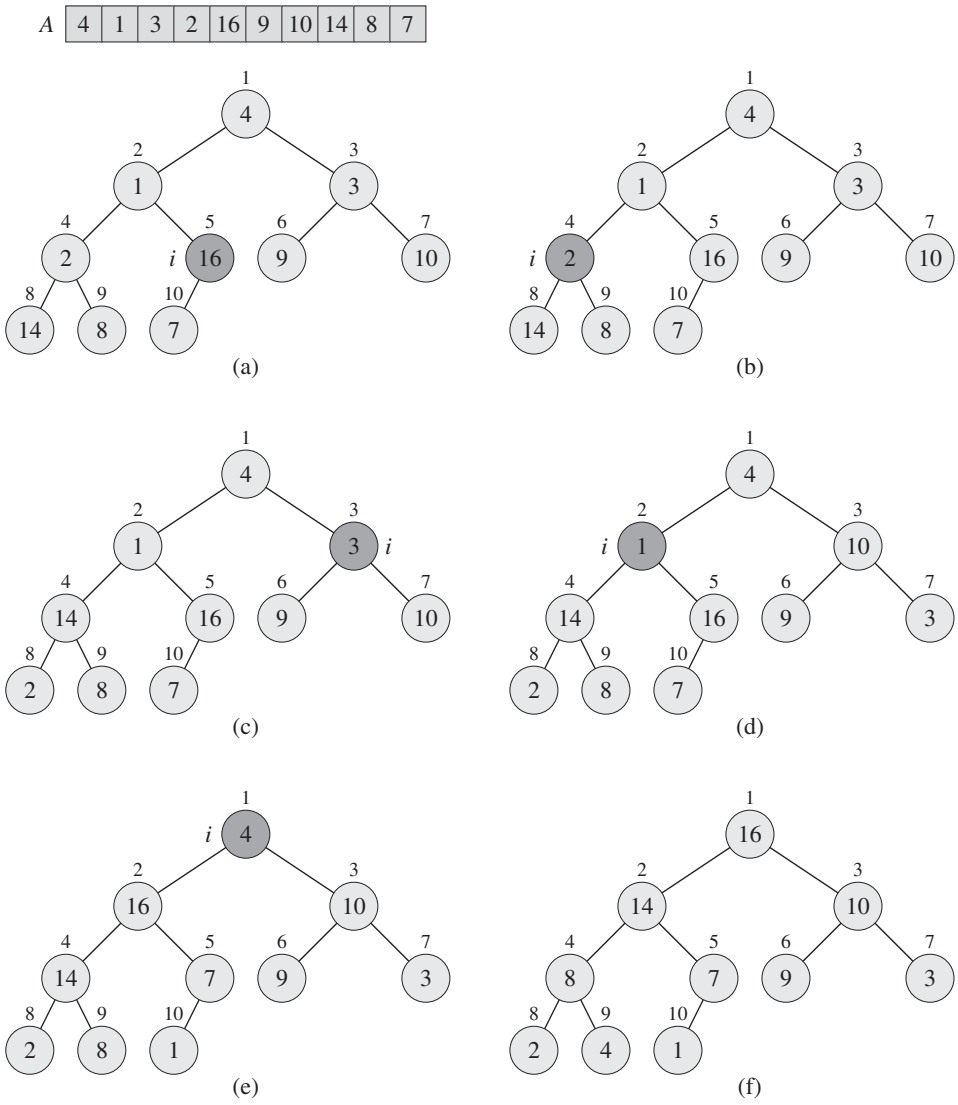


Figura 6.3 Il funzionamento di BUILD-MAX-HEAP illustrato mostrando la struttura dati prima della chiamata della procedura MAX-HEAPIFY nella riga 3 di BUILD-MAX-HEAP. **(a)** Un array di input di 10 elementi e l'albero binario che esso rappresenta. La figura mostra che l'indice i del ciclo fa riferimento al nodo 5 prima della chiamata della procedura MAX-HEAPIFY(A, i). **(b)** La struttura dati risultante. L'indice i del ciclo per la successiva iterazione fa riferimento al nodo 4. **(c)–(e)** Le successive iterazioni del ciclo **for** in BUILD-MAX-HEAP. Notate che ogni volta che viene chiamata la procedura MAX-HEAPIFY per un nodo, i due sottoalberi del nodo sono entrambi max-heap. **(f)** Il max-heap alla fine della procedura BUILD-MAX-HEAP.

Dobbiamo dimostrare che questa invariante è vera prima della prima iterazione del ciclo, che ogni iterazione del ciclo conserva l'invariante e che l'invariante fornisce un'utile proprietà per dimostrare la correttezza quando termina il ciclo.

Inizializzazione: prima della prima iterazione del ciclo, $i = \lfloor n/2 \rfloor$. Ogni nodo $\lfloor n/2 \rfloor + 1, \lfloor n/2 \rfloor + 2, \ldots, n$ è una foglia e, quindi, è la radice di un max-heap banale.

Conservazione: per verificare che ogni iterazione conserva l'invariante di ciclo, notiamo che i figli del nodo i hanno una numerazione più alta di i. Per l'invariante di ciclo, quindi, essi sono entrambi radici di max-heap. Questa è esattamente la condizione richiesta affinché la chiamata MAX-HEAPIFY(A, i) renda il nodo i la radice di un max-heap. Inoltre, la chiamata MAX-HEAPIFY preserva la proprietà che tutti i nodi $i + 1, i + 2, \ldots, n$ siano radici di max-heap. La diminuzione di i nell'aggiornamento del ciclo **for** ristabilisce l'invariante di ciclo per la successiva iterazione.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libraria: 1895032016-220707. Copyright © 2022, McGraw-Hill Education (Italy)

Conclusione: alla fine del ciclo, $i = 0$. Per l'invariante di ciclo, ogni nodo $1, 2, \ldots, n$ è la radice di un max-heap; in particolare, lo è il nodo 1.

Possiamo calcolare un semplice limite superiore sul tempo di esecuzione di BUILD-MAX-HEAP nel seguente modo. Ogni chiamata di MAX-HEAPIFY costa un tempo $O(\lg n)$ e ci sono $O(n)$ di queste chiamate. Quindi, il tempo di esecuzione è $O(n \lg n)$. Questo limite superiore, sebbene corretto, non è asintoticamente stretto.

Possiamo ottenere un limite più stretto osservando che il tempo per eseguire MAX-HEAPIFY in un nodo varia con l'altezza del nodo nell'albero, e le altezze della maggior parte dei nodi sono piccole. L'analisi più rigorosa si basa sulle proprietà che un heap di n elementi ha un'altezza $\lfloor \lg n \rfloor$ (vedere l'Esercizio 6.1-2) e, per ogni h, al massimo $\lceil n/2^{h+1} \rceil$ nodi di altezza h (vedere l'Esercizio 6.3-3).

Il tempo richiesto dalla procedura MAX-HEAPIFY quando viene chiamata per un nodo di altezza h è $O(h)$, quindi possiamo dire che il costo totale di BUILD-MAX-HEAP è limitato superiormente da

$$\sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h} \right)$$

L'ultima sommatoria può essere calcolata ponendo $x = 1/2$ nella Formula (A.8):

$$\begin{aligned} \sum_{h=0}^{\infty} \frac{h}{2^h} &= \frac{1/2}{(1-1/2)^2} \\ &= 2 \end{aligned}$$

Quindi, il tempo di esecuzione di BUILD-MAX-HEAP può essere limitato così

$$\begin{aligned} O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h} \right) &= O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h} \right) \\ &= O(n) \end{aligned}$$

Dunque, possiamo costruire un max-heap da un array non ordinato in tempo lineare.

Possiamo costruire un min-heap utilizzando la procedura BUILD-MIN-HEAP, che è uguale a BUILD-MAX-HEAP, con la differenza che la chiamata di MAX-HEAPIFY nella riga 3 è sostituita dalla chiamata di MIN-HEAPIFY (vedere l'Esercizio 6.2-2). BUILD-MIN-HEAP produce un min-heap da un array non ordinato in tempo lineare.

Esercizi

6.3-1

Utilizzando la Figura 6.3 come modello, illustrate l'azione di BUILD-MAX-HEAP sull'array $A = \langle 5, 3, 17, 10, 84, 19, 6, 22, 9 \rangle$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

6.3-2

Perché l'indice i nella riga 2 del ciclo di BUILD-MAX-HEAP deve diminuire da $\lfloor A.length/2 \rfloor$ a 1, anziché aumentare da 1 a $\lfloor A.length/2 \rfloor$?

6.3-3

Dimostrate che ci sono al massimo $\lceil n/2^{h+1} \rceil$ nodi di altezza h in qualsiasi heap di n elementi.

6.4 L'algoritmo heapsort

L'algoritmo heapsort inizia utilizzando BUILD-MAX-HEAP per costruire un max-heap nell'array di input $A[1\,..\,n]$, dove $n = A.length$. Poiché l'elemento più grande dell'array è memorizzato nella radice $A[1]$, esso può essere inserito nella sua posizione finale corretta scambiandolo con $A[n]$. Se adesso "togliamo" il nodo n dall'heap (diminuendo $A.heap\text{-}size$), notiamo che i figli della radice restano max-heap, ma la nuova radice potrebbe violare la proprietà del max-heap. Per ripristinare questa proprietà, tuttavia, basta una chiamata di MAX-HEAPIFY$(A, 1)$, che lascia un max-heap in $A[1\,..\,(n-1)]$. L'algoritmo heapsort poi ripete questo processo per il max-heap di dimensione $n-1$ e così via fino a un heap di dimensione 2 (vedere l'Esercizio 6.4-2 per una esatta invariante di ciclo).

```
HEAPSORT(A)
1  BUILD-MAX-HEAP(A)
2  for i = A.length downto 2
3      scambia A[1] con A[i]
4      A.heap-size = A.heap-size − 1
5      MAX-HEAPIFY(A, 1)
```

La Figura 6.4 illustra un esempio del funzionamento di HEAPSORT dopo che il max-heap è stato costruito. La figura mostra il max-heap prima della prima iterazione del ciclo **for** (righe 2–5) e dopo ciascuna iterazione.

La procedura HEAPSORT impiega un tempo $O(n \lg n)$, in quanto la chiamata di BUILD-MAX-HEAP impiega $O(n)$ e ciascuna delle $n-1$ chiamate di MAX-HEAPIFY impiega un tempo $O(\lg n)$.

Esercizi

6.4-1

Illustrate l'azione di HEAPSORT sull'array $A = \langle 5, 13, 2, 25, 7, 17, 20, 8, 4\rangle$, utilizzando la Figura 6.4 come modello.

6.4-2

Dimostrate la correttezza di HEAPSORT utilizzando la seguente invariante di ciclo:

> All'inizio di ogni iterazione del ciclo **for**, righe 2–5, il sottoaray $A[1\,..\,i]$ è un max-heap che contiene gli (i) elementi più piccoli di $A[1\,..\,n]$ e il sottoarray $A[i+1\,..\,n]$ contiene gli $(n-i)$ elementi più grandi di $A[1\,..\,n]$, ordinati.

6.4-3

Qual è il tempo di esecuzione di heapsort con un array A di lunghezza n che è già ordinato in senso crescente? E se l'array è ordinato in senso decrescente?

Acquistato da Michele Michele su Webster il 2022-0[illegible]3:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

6.4-4

Dimostrate che il tempo di esecuzione nel caso peggiore di heapsort è $\Omega(n \lg n)$.

6.4-5 ⋆

Dimostrate che, quando tutti gli elementi sono distinti, il tempo di esecuzione nel caso migliore di heapsort è $\Omega(n \lg n)$.

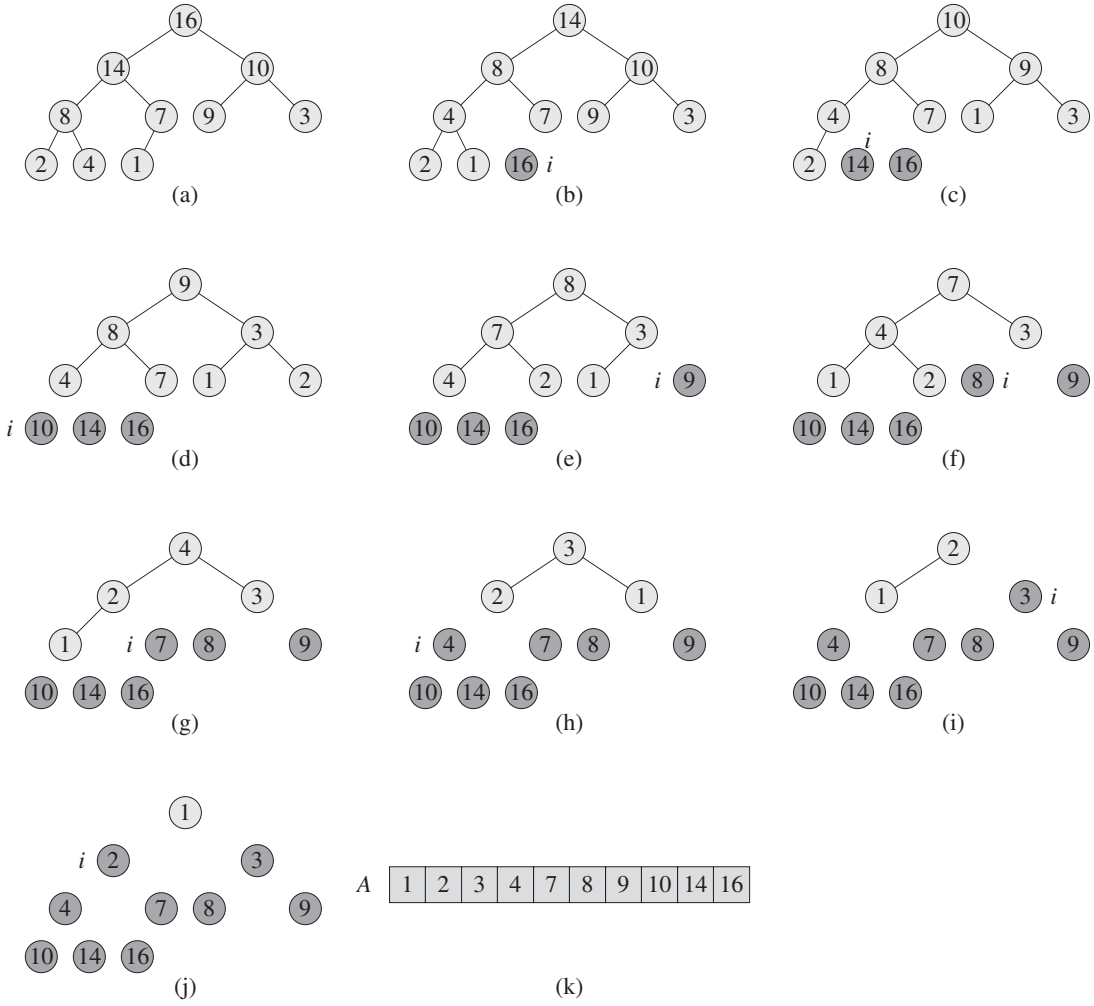


Figura 6.4 Il funzionamento di HEAPSORT. **(a)** La struttura max-heap subito dopo essere stata costruita da BUILD-MAX-HEAP. **(b)–(j)** La struttura max-heap subito dopo ciascuna chiamata di MAX-HEAPIFY nella riga 5 con indicato il valore di i in tale istante. Soltanto i nodi più chiari restano nell'heap. **(k)** L'array ordinato A risultante.

6.5 Code di priorità

Heapsort è un eccellente algoritmo, ma una buona implementazione di quicksort, che vedremo nel Capitolo 7, di solito batte heapsort. Pur tuttavia, la struttura dati heap ha molteplici usi. In questo paragrafo, presentiamo una delle applicazioni più diffuse dell'heap: un'efficiente coda di priorità. Analogamente agli heap, ci sono due tipi di code di priorità: code di max-priorità e code di min-priorità. Qui analizzeremo l'implementazione delle code di max-priorità che, a loro volta, si basano sui max-heap; l'Esercizio 6.5-3 chiede di scrivere le procedure per le code di min-priorità.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Una ***coda di priorità*** è una struttura dati che serve a mantenere un insieme S di elementi, ciascuno con un valore associato detto ***chiave***. Una ***coda di max-priorità*** supporta le seguenti operazioni.

INSERT(S, x) inserisce l'elemento x nell'insieme S, che equivale all'operazione: $S = S \cup \{x\}$.

MAXIMUM(S) restituisce l'elemento di S con la chiave più grande.

EXTRACT-MAX(S) rimuove e restituisce l'elemento di S con la chiave più grande.

INCREASE-KEY(S, x, k) aumenta il valore della chiave dell'elemento x al nuovo valore k, che si suppone sia almeno grande quanto il valore corrente della chiave dell'elemento x.

Tra le applicazioni delle code di max-priorità vi è quella di programmare i lavori su un computer condiviso. La coda di max-priority tiene traccia dei lavori da svolgere e delle loro relative priorità. Quando un lavoro è ultimato o interrotto, lo scheduler seleziona il lavoro con priorità più alta fra quelli in attesa chiamando EXTRACT-MAX. Lo scheduler può in ogni momento, aggiungere un nuovo lavoro alla coda chiamando INSERT.

In alternativa, una ***coda di min-priorità*** supporta le operazioni INSERT, MINIMUM, EXTRACT-MIN e DECREASE-KEY. Una coda di min-priorità può essere utilizzata in un simulatore controllato da eventi. Gli elementi della coda sono gli eventi da simulare. Ogni elemento della coda è associato al tempo in cui l'evento si può verificare; questo tempo serve da chiave dell'evento. Gli eventi devono essere simulati secondo l'ordine dei loro tempi, in quanto la simulazione di un evento può causare la simulazione di altri eventi futuri. Il programma di simulazione chiama EXTRACT-MIN a ogni passo per selezionare il successivo evento da simulare. Ogni volta che viene prodotto un nuovo evento, il simulatore lo inserisce nella coda di min-priorità chiamando INSERT. Nei Capitoli 23 e 24 vedremo altre applicazioni delle code di min-priorità, mettendo in evidenza l'operazione di DECREASE-KEY.

Come detto, possiamo utilizzare un heap per implementate una coda di priorità. In una data applicazione, come la programmazione dei lavori o la simulazione controllata da eventi, gli elementi di una coda di priorità corrispondono a oggetti dell'applicazione. Spesso è necessario determinare quale oggetto dell'applicazione corrisponde a un dato elemento della coda di priorità e viceversa. Pertanto, quando un heap viene utilizzato per implementare una coda di priorità, spesso occorre memorizzare in ogni elemento dell'heap un ***handle*** (aggancio) al corrispondente oggetto dell'applicazione. L'esatta realizzazione dell'handle (con un puntatore o con un intero) dipende dall'applicazione. Analogamente, occorre memorizzare un handle al corrispondente elemento dell'heap in ogni oggetto dell'applicazione. Qui, tipicamente, l'handle è un indice dell'array. Poiché gli elementi dell'heap cambiano posizione all'interno dell'array durante le operazioni con l'heap, un'implementazione reale, dopo avere spostato un elemento dell'heap, dovrebbe aggiornare anche l'indice dell'array nel corrispondente oggetto dell'applicazione. Poiché i dettagli per accedere agli oggetti dell'applicazione dipendono molto dall'applicazione e dalla sua implementazione, non li tratteremo qui, ma ci limiteremo a notare che, in pratica, questi handle hanno bisogno di essere mantenuti correttamente.

Acquistato da Michele Michele su Webster il 2022-09-07 23:10 Antonio Cordioli - ilmio.it - 10750303016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Adesso descriviamo come implementare le operazioni di una coda di max-priorità. La procedura HEAP-MAXIMUM implementa l'operazione MAXIMUM nel tempo $\Theta(1)$.

```
HEAP-MAXIMUM(A)
1  return A[1]
```

La procedura HEAP-EXTRACT-MAX implementa l'operazione EXTRACT-MAX. È simile al corpo del ciclo **for** (righe 3–5) della procedura HEAPSORT.

```
HEAP-EXTRACT-MAX(A)
1  if A.heap-size < 1
2       error "underflow dell'heap"
3  max = A[1]
4  A[1] = A[A.heap-size]
5  A.heap-size = A.heap-size − 1
6  MAX-HEAPIFY(A, 1)
7  return max
```

Il tempo di esecuzione di HEAP-EXTRACT-MAX è $O(\lg n)$, in quanto svolge soltanto una quantità costante di lavoro oltre al tempo $O(\lg n)$ di MAX-HEAPIFY.

La procedura HEAP-INCREASE-KEY implementa l'operazione INCREASE-KEY. L'elemento della coda di priorità la cui chiave deve essere aumentata è identificato da un indice i nell'array. Innanzitutto, la procedura aggiorna la chiave dell'elemento $A[i]$ con il suo nuovo valore. Successivamente, poiché l'aumento della chiave di $A[i]$ potrebbe violare la proprietà del max-heap, la procedura, in una maniera che ricorda il ciclo di inserimento (righe 5–7) di INSERTION-SORT nel Paragrafo 2.1, segue un cammino semplice da questo nodo verso la radice per trovare un posto appropriato alla nuova chiave. Durante questo attraversamento, HEAP-INCREASE-KEY confronta ripetutamente un elemento con suo padre e scambia le loro chiavi se la chiave dell'elemento è più grande; questa operazione termina se la chiave dell'elemento è più piccola, perché in questo caso la proprietà del max-heap è soddisfatta (vedere l'Esercizio 6.5-5 per una esatta invariante di ciclo).

```
HEAP-INCREASE-KEY(A, i, key)
1  if key < A[i]
2       error "la nuova chiave è più piccola di quella corrente"
3  A[i] = key
4  while i > 1 and A[PARENT(i)] < A[i]
5       scambia A[i] con A[PARENT(i)]
6       i = PARENT(i)
```

La Figura 6.5 illustra un esempio dell'operazione HEAP-INCREASE-KEY. Il tempo di esecuzione di HEAP-INCREASE-KEY con un heap di n elementi è $O(\lg n)$, in quanto il cammino verso la radice percorso dal nodo aggiornato nella riga 3 ha lunghezza $O(\lg n)$.

La procedura MAX-HEAP-INSERT implementa l'operazione INSERT. Prende come input la chiave del nuovo elemento da inserire nel max-heap A. La procedura prima espande il max-heap aggiungendo all'albero una nuova foglia la cui chiave è $-\infty$; poi chiama HEAP-INCREASE-KEY per impostare la chiave di questo nuovo nodo al suo valore corretto e mantenere la proprietà del max-heap.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

```
MAX-HEAP-INSERT(A, key)
1  A.heap-size = A.heap-size + 1
2  A[A.heap-size] = −∞
3  HEAP-INCREASE-KEY(A, A.heap-size, key)
```

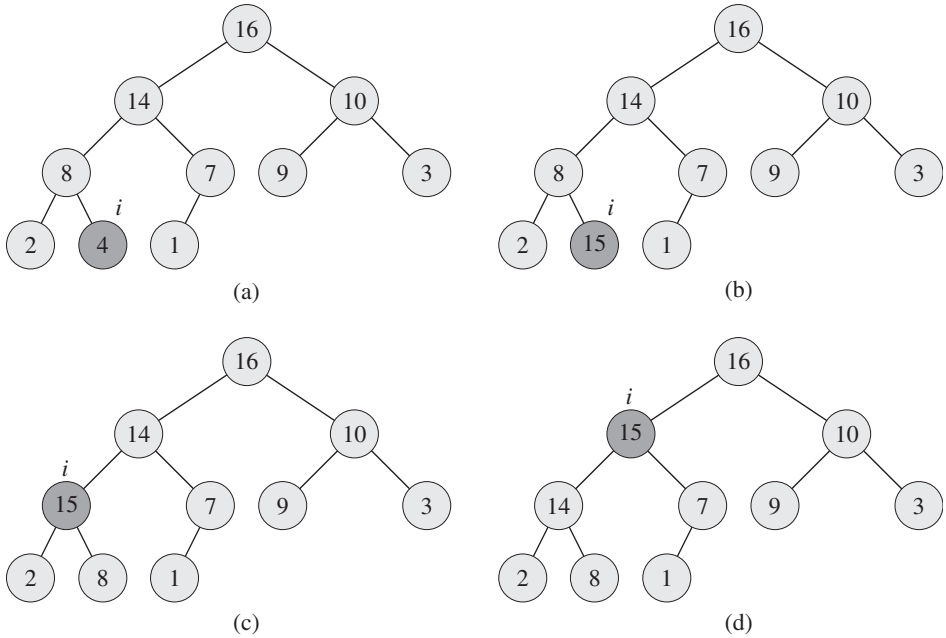


Figura 6.5 Il funzionamento di HEAP-INCREASE-KEY. **(a)** Il max-heap della Figura 6.4(a) con il nodo di indice i più scuro. **(b)** La chiave di questo nodo viene aumentata a 15. **(c)** Dopo un'iterazione del ciclo **while**, righe 4–6, il nodo e suo padre hanno le chiavi scambiate e l'indice i passa al padre. **(d)** Il max-heap dopo un'altra iterazione del ciclo **while**. A questo punto, $A[\text{PARENT}(i)] \geq A[i]$. La proprietà del max-heap adesso è soddisfatta e la procedura termina.

Il tempo di esecuzione della procedura MAX-HEAP-INSERT con un heap di n elementi è $O(\lg n)$. In sintesi, un heap può svolgere ciascuna operazione con le code di priorità nel tempo $O(\lg n)$ su un insieme di dimensione n.

Esercizi

6.5-1

Illustrate l'operazione di HEAP-EXTRACT-MAX sull'heap $A = \langle 15, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1 \rangle$.

6.5-2

Illustrate l'operazione di MAX-HEAP-INSERT$(A, 10)$ sull'heap $A = \langle 15, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1 \rangle$.

6.5-3

Scrivete lo pseudocodice per le procedure HEAP-MINIMUM, HEAP-EXTRACT-MIN, HEAP-DECREASE-KEY e MIN-HEAP-INSERT che implementano una coda di min-priorità con un min-heap.

6.5-4

Perché dobbiamo preoccuparci di impostare la chiave del nodo inserito a $-\infty$ nella riga 2 di MAX-HEAP-INSERT quando la successiva cosa che facciamo è aumentare la sua chiave al valore desiderato?

6.5-5

Dimostrate la correttezza di HEAP-INCREASE-KEY utilizzando la seguente invariante di ciclo:

> All'inizio di ogni iterazione del ciclo **while** (righe 4–6), l'array $A[1\,..\,A.\textit{heap-size}]$ soddisfa la proprietà del max-heap, tranne una possibile violazione: $A[i]$ potrebbe essere più grande di $A[\text{PARENT}(i)]$.

Potete supporre che il sottoarray $A[1\,..\,A.\textit{heap-size}]$ soddisfi la proprietà del max-heap nell'istante in cui viene chiamata la procedura HEAP-INCREASE-KEY.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

6.5-6
Ogni operazione di scambio nella riga 5 di HEAP-INCREASE-KEY, tipicamente, richiede tre assegnazioni. Applicate l'idea del ciclo interno di INSERTION-SORT per ridurre le tre assegnazioni a una sola assegnazione.

6.5-7
Spiegate come implementare una coda di tipo primo arrivato primo servito (FIFO: First-In, First-Out) con una coda di priorità. Spiegate come implementare uno stack con una coda di priorità (le code e gli stack sono definiti nel Paragrafo 10.1).

6.5-8
L'azione di HEAP-DELETE(A, i) cancella l'elemento nel nodo i dall'heap A. Implementate la procedura HEAP-DELETE in modo che il suo tempo di esecuzione sia $O(\lg n)$ per un max-heap di n elementi.

6.5-9
Descrivete un algoritmo con tempo $O(n \lg k)$ per fondere k liste ordinate in un'unica lista ordinata, dove n è il numero totale di elementi di tutte le liste di input (*suggerimento:* usate un min-heap per la fusione).

Problemi

6-1 Costruire un heap mediante inserimento
Possiamo costruire un heap chiamando ripetutamente la procedura MAX-HEAP-INSERT per inserire gli elementi nell'heap. Considerate la seguente variante della procedura BUILD-MAX-HEAP:

```
BUILD-MAX-HEAP'(A)
1  A.heap-size = 1
2  for i = 2 to A.length
3      MAX-HEAP-INSERT(A, A[i])
```

a. Le procedure BUILD-MAX-HEAP e BUILD-MAX-HEAP′ creano sempre lo stesso heap se vengono eseguite con lo stesso array di input? Dimostrate che lo fanno o fornite un esempio contrario.

b. Dimostrate che, nel caso peggiore, BUILD-MAX-HEAP′ richiede un tempo $\Theta(n \lg n)$ per costruire un heap di n elementi.

6-2 Analisi di un heap d-ario
Un ***heap d-ario*** è come un heap binario, in cui ogni nodo non-foglia ha d figli, anziché 2 figli (con una sola possibile eccezione).

a. Come rappresentereste un heap d-ario in un array?

b. Qual è l'altezza di un heap d-ario di n elementi espressa in funzione di n e d?

c. Realizzate un'implementazione efficiente di EXTRACT-MAX in un max-heap d-ario. Analizzate il suo tempo di esecuzione in funzione di d e n.

Acquistato da Michele Michela su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950316-220707-01 Copyright © 2022, McGraw-Hill Education (Italy)

d. Realizzate un'implementazione efficiente di INSERT in un max-heap d-ario. Analizzate il suo tempo di esecuzione in funzione di d e n.

e. Realizzate un'implementazione efficiente di INCREASE-KEY(A, i, k), che segnala un errore se $k < A[i]$, altrimenti imposta $A[i] = \max(A[i], k)$ e poi aggiorna appropriatamente la struttura del max-heap. Analizzate il suo tempo di esecuzione in funzione di d e n.

6-3 I tableau di Young

Un ***tableau di Young*** è una matrice $m \times n$ nella quale gli elementi delle righe sono ordinati da sinistra a destra e quelli delle colonne dall'alto verso il basso. Alcuni elementi possono essere ∞, che noi consideriamo elementi mancanti. Quindi, un tableau di Young può essere utilizzato per contenere $r \leq mn$ numeri finiti.

a. Disegnate un tableau di Young 4×4 che contiene gli elementi $\{9, 16, 3, 2, 4, 8, 5, 14, 12\}$.

b. Se Y è un tableau di Young $m \times n$, dimostrate che Y è vuoto se $Y[1, 1] = \infty$. Dimostrate che Y è pieno (contiene mn elementi) se $Y[m, n] < \infty$.

c. Create un algoritmo per eseguire l'operazione EXTRACT-MIN in un tableau di Young $m \times n$ non vuoto nel tempo $O(m + n)$. Il vostro algoritmo dovrebbe utilizzare una subroutine ricorsiva che risolve un problema $m \times n$ risolvendo ricorsivamente un sottoproblema $(m - 1) \times n$ o $m \times (n - 1)$ (*suggerimento:* pensate a MAX-HEAPIFY). Se $T(p)$, dove $p = m + n$, è il tempo massimo di esecuzione di EXTRACT-MIN con qualsiasi tableau di Young $m \times n$, scrivete e risolvete una ricorrenza per $T(p)$ che fornisce il limite sul tempo $O(m + n)$.

d. Spiegate come inserire un nuovo elemento in un tableau di Young $m \times n$ non pieno nel tempo $O(m + n)$.

e. Senza utilizzare alcun metodo di ordinamento come subroutine, spiegate come utilizzare un tableau di Young $n \times n$ per ordinare n^2 numeri nel tempo $O(n^3)$.

f. Indicate un algoritmo con tempo $O(m + n)$ per determinare se un particolare numero è memorizzato in un determinato tableau di Young $m \times n$.

Note

L'algoritmo heapsort è stato ideato da Williams [358], che ha descritto anche come implementare una coda di priorità con un heap. La procedura BUILD-MAX-HEAP è stata proposta da Floyd [107]. Nei Capitoli 16, 23 e 24 utilizzeremo i min-heap per implementare le code di min-priorità. Presenteremo anche implementazioni con limiti di tempo migliori; soltanto per certe operazioni nel Capitolo 19 e, nel Capitolo 20, sotto l'ipotesi che le chiavi siano scelte in un insieme limitato di numeri interi.

Se i dati sono interi di b bit e la memoria del calcolatore è costituita da parole indirizzabili di b bit, Fredman e Willard [116] hanno dimostrato come implementare MINIMUM nel tempo $O(1)$ e INSERT e EXTRACT-MIN nel tempo $O(\sqrt{\lg n})$. Thorup [338] ha migliorato il limite $O(\sqrt{\lg n})$ al tempo $O(\lg \lg n)$. Questo limite usa una quantità di spazio non limitato in n, ma può essere implementato con spazio lineare utilizzando l'hashing randomizzato.

Un importante caso speciale di code di priorità si verifica quando la sequenza delle operazioni EXTRACT-MIN è ***monotona***, ovvero i valori restituiti dalle successive operazioni EXTRACT-MIN sono monotonicamente crescenti nel tempo. Questo caso si presenta in molte applicazioni importanti, come l'algoritmo di Dijkstra per il problema dei cammini minimi da sorgente unica, che sarà descritto nel Capitolo 24, e nella simulazione di eventi discreti. Per l'algoritmo di Dijkstra è particolarmente importante che l'operazione DECREASE-KEY sia implementata in maniera efficiente. Nel caso della sequenza monotona, se i dati sono numeri interi nell'intervallo $1, 2, \ldots, C$, Ahuja, Mehlhorn, Orlin e Tarjan [8] hanno spiegato come implementare EXTRACT-MIN e INSERT nel tempo ammortizzato $O(\lg C)$ (vedere il Capitolo 17 per maggiori informazioni sull'analisi ammortizzata) e DECREASE-KEY nel tempo $O(1)$, utilizzando una struttura dati detta *radix heap*. Il limite $O(\lg C)$ può essere migliorato a $O(\sqrt{\lg C})$ utilizzando gli heap di Fibonacci (descritti nel Capitolo 19) in combinazione con le strutture radix heap. Cherkassky, Goldberg e Silverstein [66] hanno ulteriormente migliorato il limite al tempo atteso $O(\lg^{1/3+\epsilon} C)$ combinando la struttura di bucket a più livelli di Denardo e Fox [86] con l'heap di Thorup prima citato. Raman [292] ha ulteriormente perfezionato questi risultati ottenendo un limite di $O(\min(\lg^{1/4+\epsilon} C, \lg^{1/3+\epsilon} n))$, per qualsiasi costante $\epsilon > 0$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Quicksort 7

Quicksort è un algoritmo di ordinamento il cui tempo di esecuzione nel caso peggiore è $\Theta(n^2)$ con un array di input di n numeri. Nonostante questo tempo di esecuzione nel caso peggiore sia molto alto, quicksort spesso è la soluzione pratica migliore per effettuare un ordinamento, perché mediamente è molto efficiente: il suo tempo di esecuzione atteso è $\Theta(n \lg n)$ e i fattori costanti nascosti nella notazione $\Theta(n \lg n)$ sono molto piccoli. Inoltre ha il vantaggio di ordinare sul posto (come descritto a pagina 15) e funziona bene anche in ambienti con memoria virtuale. Il Paragrafo 7.1 descrive l'algoritmo e un'importante subroutine utilizzata da quicksort per il partizionamento. Poiché il comportamento di quicksort è complesso, inizieremo con una discussione intuitiva delle sue prestazioni nel Paragrafo 7.2 e rinvieremo l'analisi più formale alla fine del capitolo. Il Paragrafo 7.3 presenta una versione di quicksort che usa il campionamento casuale. Questo algoritmo ha un buon tempo di esecuzione nel caso medio e nessun particolare input provoca sistematicamente il comportamento nel caso peggiore dell'algoritmo. L'algoritmo randomizzato è analizzato nel Paragrafo 7.4, dove dimostriamo che viene eseguito nel tempo $\Theta(n^2)$ nel caso peggiore e, se gli elementi sono distinti, nel tempo atteso $O(n \lg n)$.

7.1 Descrizione di quicksort

Quicksort, come merge sort, è basato sul paradigma divide et impera presentato nel Paragrafo 2.3.1. Questi sono i tre passi del processo divide et impera per ordinare un generico sottoarray $A[p \,..\, r]$.

Divide: partizionare l'array $A[p \,..\, r]$ in due sottoarray $A[p \,..\, q-1]$ e $A[q+1 \,..\, r]$ (eventualmente vuoti) tali che ogni elemento di $A[p \,..\, q-1]$ sia minore o uguale ad $A[q]$ che, a sua volta, è minore o uguale a ogni elemento di $A[q+1 \,..\, r]$. Calcolare l'indice q come parte di questa procedura di partizionamento.

Impera: ordinare i due sottoarray $A[p \,..\, q-1]$ e $A[q+1 \,..\, r]$ chiamando ricorsivamente quicksort.

Combina: poiché i sottoarray sono già ordinati, non occorre alcun lavoro per combinarli: l'intero array $A[p \,..\, r]$ è ordinato.

La seguente procedura implementa quicksort.

Acquistato da Michele Mazzetti - Matricola 1022307 - 2182 - Università Cattolica - Licenza: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

```
QUICKSORT(A, p, r)
1  if p < r
2      q = PARTITION(A, p, r)
3      QUICKSORT(A, p, q − 1)
4      QUICKSORT(A, q + 1, r)
```

Per ordinare un intero array A, la chiamata iniziale è QUICKSORT$(A, 1, A.length)$.

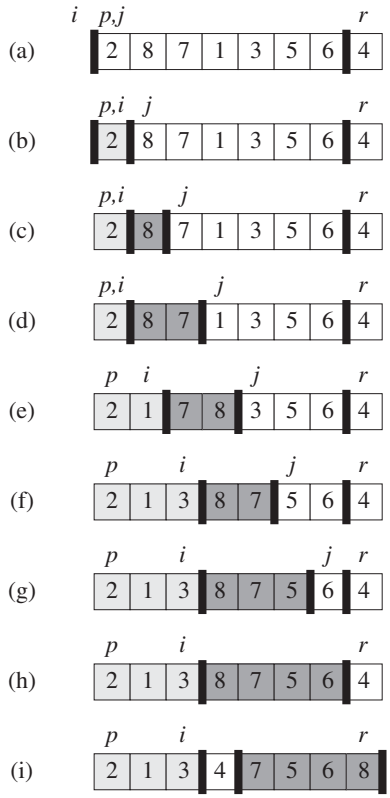


Figura 7.1 Il funzionamento di PARTITION su un array campione. L'elemento $A[r]$ dell'array diventa l'elemento pivot x. Gli elementi dell'array su sfondo grigio chiaro sono tutti nella prima partizione con valori non maggiori di x. Gli elementi su sfondo grigio scuro sono nella seconda partizione con valori maggiori di x. Gli elementi su sfondo bianco non sono stati ancora posti in una delle prime due partizioni; l'ultimo elemento su sfondo bianco è il *pivot*. **(a)** L'array iniziale e le impostazioni delle variabili. Nessuno degli elementi è stato posto in una delle prime due partizioni. **(b)** Il valore 2 viene "scambiato con sé stesso" e posto nella partizione dei valori più piccoli. **(c)–(d)** I valori 8 e 7 vengono inseriti nella partizione dei valori più grandi. **(e)** I valori 1 e 8 vengono scambiati e la prima partizione cresce. **(f)** I valori 3 e 7 vengono scambiati e la prima partizione cresce. **(g)–(h)** La seconda partizione acquisisce i valori 5 e 6 e il ciclo termina. **(i)** Le righe 7–8 scambiano il pivot inserendolo fra le due partizioni.

Partizionare l'array

L'elemento chiave dell'algoritmo è la procedura PARTITION, che riarrangia il sottoarray $A[p\,..\,r]$ sul posto.

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4       if A[j] ≤ x
5            i = i + 1
6            scambia A[i] con A[j]
7  scambia A[i + 1] con A[r]
8  return i + 1
```

La Figura 7.1 illustra il funzionamento di PARTITION su un array di 8 elementi. PARTITION seleziona sempre un elemento $x = A[r]$ come ***pivot*** intorno al quale partizionare il sottoarray $A[p\,..\,r]$. Durante l'esecuzione della procedura, l'array viene suddiviso in quattro regioni (eventualmente vuote). All'inizio di ogni iterazione del ciclo **for** (righe 3–6) ogni regione soddisfa alcune proprietà (Figura 7.2). Enunciamo queste proprietà nella forma di una invariante di ciclo:

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995080016-2280707-0 Copyright © 2022, McGraw-Hill Education (Italy)

> All'inizio di ogni iterazione del ciclo, righe 3–6, per qualsiasi indice k dell'array,
>
> 1. Se $p \le k \le i$, allora $A[k] \le x$.
> 2. Se $i + 1 \le k \le j - 1$, allora $A[k] > x$.
> 3. Se $k = r$, allora $A[k] = x$.

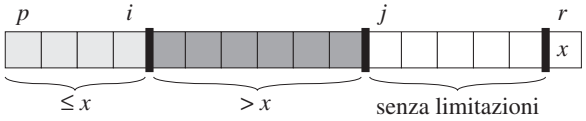


Figura 7.2 Le quattro regioni mantenute dalla procedura PARTITION in un sottoarray $A[p\,..\,r]$. I valori in $A[p\,..\,i]$ sono tutti minori o uguali a x, i valori in $A[i+1\,..\,j-1]$ sono tutti maggiori di x e $A[r] = x$. Gli elementi di $A[j\,..\,r-1]$ possono avere valori qualsiasi.

Gli indici fra j e $r-1$ non rientrano in alcuno di questi tre casi e i corrispondenti valori in queste posizioni non hanno una particolare relazione con il pivot x.

Dobbiamo dimostrare che questa invariante di ciclo è vera prima della prima iterazione, che ogni iterazione del ciclo conserva l'invariante e che l'invariante fornisce un'utile proprietà per dimostrare la correttezza quando il ciclo termina.

Inizializzazione: prima della prima iterazione del ciclo, $i = p-1$ e $j = p$. Non ci sono valori fra p e i né fra $i+1$ e $j-1$, quindi le prime due condizioni dell'invariante di ciclo sono soddisfatte. L'assegnazione nella riga 1 soddisfa la terza condizione.

Conservazione: come mostra la Figura 7.3, ci sono due casi da considerare, a seconda del risultato del test nella riga 4. La Figura 7.3(a) mostra che cosa accade quando $A[j] > x$; l'unica azione nel ciclo è incrementare j. Dopo l'incremento di j, la condizione 2 è soddisfatta per $A[j-1]$ e tutte le altre posizioni non cambiano. La Figura 7.3(b) mostra che cosa accade quando $A[j] \le x$; viene incrementato l'indice i, vengono scambiati $A[i]$ e $A[j]$ e, poi, viene incrementato l'indice j. In seguito allo scambio, adesso abbiamo $A[i] \le x$ e la condizione 1 è soddisfatta. Analogamente, abbiamo anche $A[j-1] > x$, in quanto l'elemento che è stato spostato in $A[j-1]$ è, per l'invariante di ciclo, più grande di x.

Conclusione: alla fine del ciclo, $j = r$. Pertanto, ogni posizione dell'array si trova in uno dei tre insiemi descritti dall'invariante e noi abbiamo ripartito i valori dell'array in tre insiemi: quelli minori o uguali a x, quelli maggiori di x e un insieme di un solo elemento che contiene x.

Le ultime due righe di PARTITION inseriscono il pivot al suo posto nel mezzo dell'array, scambiandolo con l'elemento più a sinistra che è maggiore di x, e poi restituiscono il nuovo indice del pivot. L'output di PARTITION adesso soddisfa le specifiche del passo *divide*. In effetti, esso soddisfa una condizione un po' più severa: dopo la riga 2 di QUICKSORT, $A[q]$ è strettamente minore di $A[q+1\,..\,r]$.

Il tempo di esecuzione di PARTITION con il sottoarray $A[p\,..\,r]$ è $\Theta(n)$, dove $n = r - p + 1$ (vedere l'Esercizio 7.1-3).

Esercizi

7.1-1

Utilizzando la Figura 7.1 come modello, illustrate l'operazione di PARTITION sull'array $A = \langle 13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11\rangle$.

7.1-2

Quale valore di q restituisce PARTITION quando tutti gli elementi di $A[p\,..\,r]$ hanno lo stesso valore? Modificate PARTITION in modo che $q = \lfloor (p+r)/2 \rfloor$ quando tutti gli elementi di $A[p\,..\,r]$ hanno lo stesso valore.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

7.1-3

Spiegate brevemente perché il tempo di esecuzione di PARTITION con un sottoarray di dimensione n è $\Theta(n)$.

7.1-4

Come modifichereste QUICKSORT per ordinare un array in senso non crescente?

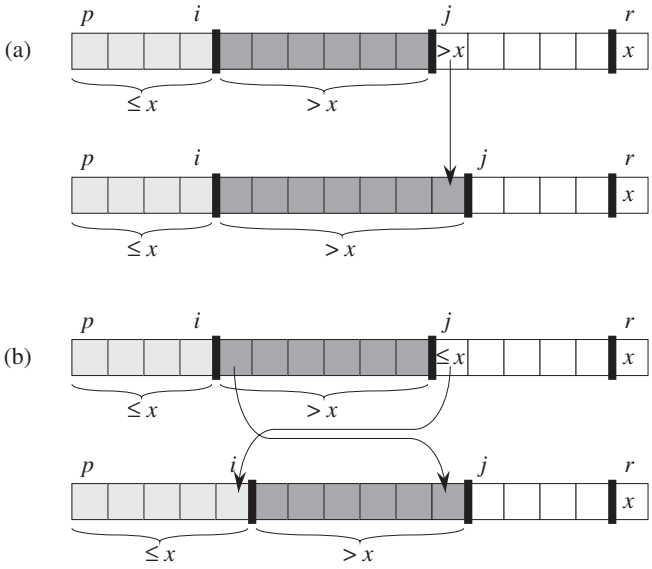


Figura 7.3 I due casi per una iterazione della procedura PARTITION. **(a)** Se $A[j] > x$, l'unica azione è incrementare j, che conserva l'invariante di ciclo. **(b)** Se $A[j] \leq x$, viene incrementato l'indice i, vengono scambiati $A[i]$ e $A[j]$; poi, viene incrementato l'indice j. Ancora una volta l'invariante di ciclo è conservata.

7.2 Prestazioni di quicksort

Il tempo di esecuzione di quicksort dipende dal fatto che il partizionamento sia bilanciato o sbilanciato e questo, a sua volta, dipende da quali elementi vengono utilizzati per il partizionamento. Se il partizionamento è bilanciato, l'algoritmo viene eseguito con la stessa velocità asintotica di merge sort. Se il partizionamento è sbilanciato, invece, quicksort può essere asintoticamente lento quanto insertion sort. In questo paragrafo analizzeremo, in modo informale, le prestazioni di quicksort nel caso di partizionamento bilanciato e sbilanciato.

Partizionamento nel caso peggiore

Il comportamento nel caso peggiore di quicksort si verifica quando la routine di partizionamento produce un sottoproblema con $n - 1$ elementi e uno con 0 elementi (dimostreremo questo fatto nel Paragrafo 7.4.1). Supponiamo che questo sbilanciamento si verifichi in ogni chiamata ricorsiva. Il partizionamento costa $\Theta(n)$ in termini di tempo. Poiché per una chiamata ricorsiva su un array vuoto $T(0) = \Theta(1)$, la ricorrenza per il tempo di esecuzione può essere espressa così:

$$\begin{aligned} T(n) &= T(n-1) + T(0) + \Theta(n) \\ &= T(n-1) + \Theta(n) \end{aligned}$$

Intuitivamente, se sommiamo i costi a ogni livello della ricorsione, otteniamo una serie aritmetica (equazione (A.2)), il cui valore è $\Theta(n^2)$. In effetti, è semplice applicare il metodo di sostituzione per dimostrare che la ricorrenza $T(n) = T(n-1) + \Theta(n)$ ha la soluzione $T(n) = \Theta(n^2)$ (vedere l'Esercizio 7.2-1). Quindi, se lo sbilanciamento delle due partizioni è massimo a ogni livello ricorsivo dell'algoritmo, il tempo di esecuzione è $\Theta(n^2)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

In definitiva, il tempo di esecuzione nel caso peggiore di quicksort non è migliore di quello di insertion sort. Inoltre, il tempo di esecuzione $\Theta(n^2)$ si ha quando l'array di input è già completamente ordinato – una situazione tipica in cui insertion sort è eseguito nel tempo $O(n)$.

Partizionamento nel caso migliore

Nel caso di bilanciamento massimo, PARTITION produce due sottoproblemi, ciascuno di dimensione non maggiore di $n/2$, in quanto uno ha dimensione $\lfloor n/2 \rfloor$ e l'altro ha dimensione $\lceil n/2 \rceil - 1$. In questo caso, quicksort viene eseguito molto più velocemente. La ricorrenza per il tempo di esecuzione è

$$T(n) \leq 2T(n/2) + \Theta(n)$$

che per il caso 2 del teorema dell'esperto (Teorema 4.1) ha la soluzione $T(n) = O(n \lg n)$. Dunque, il perfetto bilanciamento dei due lati della partizione a ogni livello di ricorsione produce un algoritmo asintoticamente più veloce.

Partizionamento bilanciato

Il tempo di esecuzione nel caso medio di quicksort è molto più vicino al caso migliore che al caso peggiore, come dimostrerà l'analisi svolta nel Paragrafo 7.4. Per capire perché, dobbiamo comprendere come il bilanciamento del partizionamento influisce sulla ricorrenza che descrive il tempo di esecuzione.

Supponiamo, per esempio, che l'algoritmo di partizionamento produca sempre una ripartizione proporzionale 9 a 1, che a prima vista potrebbe sembrare molto sbilanciata. In questo caso, otteniamo la ricorrenza

$$T(n) \leq T(9n/10) + T(n/10) + cn$$

sul tempo di esecuzione di quicksort, dove abbiamo esplicitamente incluso la costante c nascosta nel termine $\Theta(n)$. La Figura 7.4 illustra l'albero di ricorsione per questa ricorrenza. Notate che ogni livello dell'albero ha un costo cn, finché non viene raggiunta una condizione al contorno alla profondità $\log_{10} n = \Theta(\lg n)$, dopo la quale i livelli hanno al massimo un costo cn.

La ricorsione termina alla profondità $\log_{10/9} n = \Theta(\lg n)$. Il costo totale di quicksort è dunque $O(n \lg n)$. Pertanto, con una ripartizione proporzionale 9 a 1 a ogni livello di ricorsione, che intuitivamente sembra molto sbilanciata, quicksort viene eseguito nel tempo $O(n \lg n)$ – asintoticamente uguale a quello che si ha nel caso di ripartizione esattamente a metà.

In effetti, anche una ripartizione 99 a 1 determina un tempo di esecuzione pari a $O(n \lg n)$. La ragione è che qualsiasi ripartizione con proporzionalità *costante* produce un albero di ricorsione di profondità $\Theta(\lg n)$, dove il costo in ogni livello è $O(n)$. Il tempo di esecuzione è quindi $O(n \lg n)$ quando la ripartizione ha proporzionalità costante.

Alcune intuizioni sul caso medio

Per sviluppare una nozione chiara del caso medio per quicksort, dobbiamo fare un'ipotesi su quanto frequentemente prevediamo di incontrare i vari input. Il comportamento di quicksort è determinato dall'ordinamento relativo dei valori degli elementi dell'array che sono dati come input, non dai particolari valori dell'array. Analogamente a quanto fatto nell'analisi probabilistica del problema delle assunzioni (Paragrafo 5.2), ipotizzeremo per il momento che tutte le permutazioni dei numeri di input siano ugualmente probabili.

Acquistato da Michele Mo[illegible] - Proprietà McGraw-Hill Education (Italy) - Vietata la copia - Tutti i diritti riservati - © 2010 The McGraw-Hill Companies, S.r.l.

Quando eseguiamo quicksort su un array di input casuale, è poco probabile che il partizionamento avvenga sempre nello stesso modo a qualsiasi livello, come abbiamo ipotizzato nella nostra analisi informale. È logico supporre, invece, che

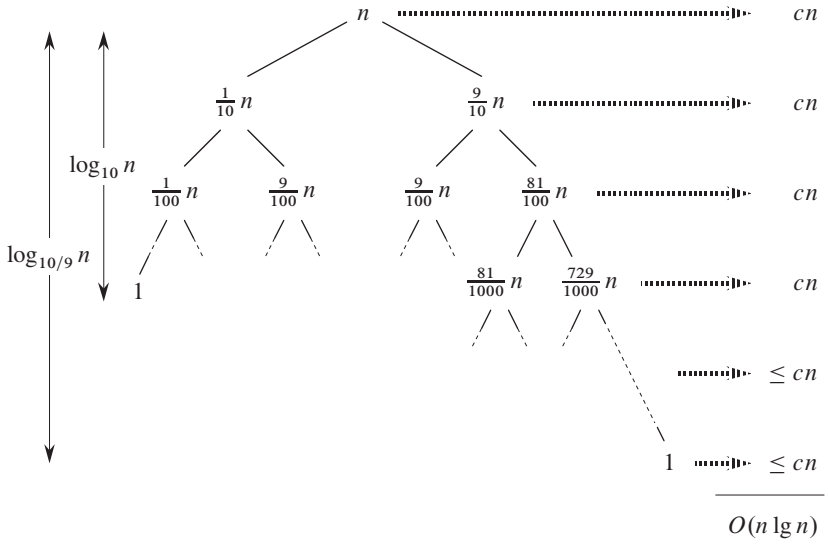


Figura 7.4 Un albero di ricorsione per QUICKSORT quando PARTITION genera sempre una ripartizione 9 a 1, determinando un tempo di esecuzione pari a $O(n \lg n)$. I nodi mostrano le dimensioni dei sottoproblemi, con i costi per livello a destra. Questi costi includono la costante c implicita nel termine $\Theta(n)$.

qualche ripartizione sarà ben bilanciata e qualche altra sarà molto sbilanciata. Per esempio, l'Esercizio 7.2-6 chiede di dimostrare che circa l'80% delle volte PARTITION produce una ripartizione che è più bilanciata di 9 a 1 e che circa il 20% delle volte produce una ripartizione che è meno bilanciata di 9 a 1.

Nel caso medio, PARTITION produce una combinazione di ripartizioni "buone" e "cattive". In un albero di ricorsione per l'esecuzione nel caso medio di PARTITION, le buone e le cattive ripartizioni sono distribuite a caso nell'albero. Supponiamo, ai fini dell'intuizione, che le ripartizioni buone e cattive si alternino nei vari livelli dell'albero e che quelle buone siano le ripartizioni nel caso migliore e quelle cattive siano le ripartizioni nel caso peggiore. La Figura 7.5(a) illustra le ripartizioni in due livelli consecutivi nell'albero di ricorsione. Nella radice dell'albero il costo del partizionamento è n e i sottoarray prodotti hanno dimensioni $n-1$ e 0: il caso peggiore. Nel livello successivo il sottoarray di dimensione $n-1$ è ripartito in due sottoarray di dimensioni $(n-1)/2-1$ e $(n-1)/2$: il caso migliore. Supponiamo che il costo della condizione al contorno sia 1 per il sottoarray di dimensione 0.

La combinazione della ripartizione cattiva seguita dalla ripartizione buona produce tre sottoarray di dimensioni 0, $(n-1)/2-1$ e $(n-1)/2$ con un costo di partizionamento complessivo pari a $\Theta(n) + \Theta(n-1) = \Theta(n)$. Certamente, questo caso non è peggiore di quello della Figura 7.5(b), ovvero un unico livello di partizionamento che genera due sottoarray di dimensione $(n-1)/2$, con un costo di $\Theta(n)$. Ma quest'ultimo caso è bilanciato! Intuitivamente, il costo $\Theta(n-1)$ della ripartizione cattiva può essere assorbito nel costo $\Theta(n)$ della ripartizione buona, quindi la ripartizione risultante è buona. In definitiva, il tempo di esecuzione di quicksort, quando i livelli si alternano fra buone e cattive ripartizioni, è come il tempo di esecuzione nel caso in cui le ripartizioni siano soltanto buone: ancora $O(n \lg n)$, ma con una costante un po' più grande nascosta dalla notazione O. Nel Paragrafo 7.4.2 faremo un'analisi più rigorosa del tempo di esecuzione atteso di una versione randomizzata di quicksort.

Acquistato da Michele Michele su Webster il 2022-07-07 23:13. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

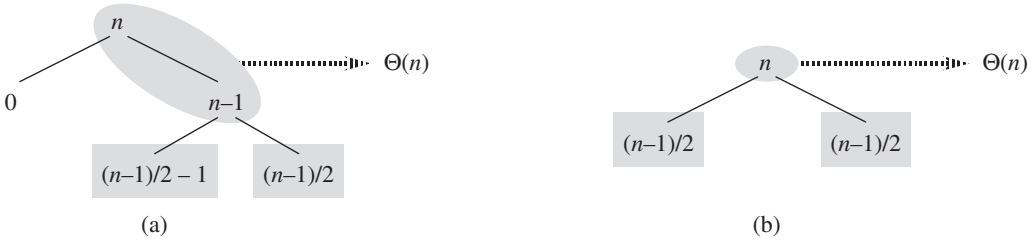


Figura 7.5 **(a)** Due livelli di un albero di ricorsione per quicksort. Il partizionamento nella radice costa n e produce una ripartizione "cattiva": due sottoarray di dimensioni 0 e $n-1$. Il partizionamento del sottoarray di dimensione $n-1$ costa $n-1$ e produce una ripartizione "buona": due sottoarray di dimensioni $(n-1)/2-1$ e $(n-1)/2$. **(b)** Un solo livello di un albero di ricorsione che è bilanciato molto bene. In entrambi i casi, il costo di partizionamento per i sottoproblemi è rappresentato da un'ellisse grigia e vale $\Theta(n)$. Inoltre i sottoproblemi che restano da risolvere nel caso (a), rappresentati da rettangoli grigi, non sono più grandi dei corrispondenti problemi che restano da risolvere nel caso (b).

Esercizi

7.2-1

Utilizzate il metodo di sostituzione per dimostrare che la ricorrenza $T(n) = T(n-1) + \Theta(n)$ ha la soluzione $T(n) = \Theta(n^2)$, come detto all'inizio del Paragrafo 7.2.

7.2-2

Qual è il tempo di esecuzione di QUICKSORT quando tutti gli elementi dell'array A hanno lo stesso valore?

7.2-3

Dimostrate che il tempo di esecuzione di QUICKSORT è $\Theta(n^2)$ quando l'array A contiene elementi distinti ed è ordinato in senso decrescente.

7.2-4

Le banche spesso registrano le transazioni in ordine cronologico, ma molte persone preferiscono ricevere l'estratto conto con gli assegni elencati in funzione dei numeri degli assegni. Le persone di solito emettono gli assegni in ordine numerico e i beneficiari di solito portano subito all'incasso gli assegni. Il problema di convertire l'ordine cronologico delle transazioni in ordine numerico degli assegni è dunque il problema di ordinare degli elementi di input che sono già quasi ordinati. Dimostrate che la procedura INSERTION-SORT tende a battere la procedura QUICKSORT in questo problema.

7.2-5

Supponete che le ripartizioni a qualsiasi livello di quicksort siano nella proporzione $1-\alpha$ ad α, dove $0 < \alpha \leq 1/2$ è una costante. Dimostrate che la profondità minima di una foglia nell'albero di ricorsione è approssimativamente $-\lg n / \lg \alpha$ e la profondità massima è approssimativamente $-\lg n / \lg(1-\alpha)$ (trascurate l'arrotondamento agli interi).

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

7.2-6 ★

Dimostrate che per qualsiasi costante $0 < \alpha \leq 1/2$, c'è la probabilità di circa $1-2\alpha$ che su un array di input casuale la procedura PARTITION produca una ripartizione più bilanciata della proporzione $1-\alpha$ ad α.

7.3 Una versione randomizzata di quicksort

Nell'analisi del comportamento di quicksort nel caso medio, abbiamo fatto l'ipotesi che tutte le permutazioni dei numeri di input fossero ugualmente probabili. Nella pratica, però, non possiamo aspettarci che questo sia sempre vero (vedere l'Esercizio 7.2-4). Come detto nel Paragrafo 5.3, a volte è possibile aggiungere la randomizzazione a un algoritmo per ottenere una buona prestazione attesa con tutti gli input. Molti considerano la versione randomizzata di quicksort come l'algoritmo di ordinamento di elezione per input sufficientemente grandi.

Nel Paragrafo 5.3, abbiamo randomizzato il nostro algoritmo permutando esplicitamente l'input. Potremmo fare la stessa cosa anche per quicksort; invece adotteremo un altro metodo di randomizzazione, detto ***campionamento casuale***, che ci consente di semplificare l'analisi. Anziché utilizzare sempre $A[r]$ come pivot, utilizzeremo un elemento scelto a caso dal sottoarray $A[p\,..\,r]$. Per fare questo, scambieremo l'elemento $A[r]$ con un elemento scelto a caso da $A[p\,..\,r]$. Questa modifica, con la quale campioniamo a caso l'intervallo $p, \ldots, r$, ci assicura che l'elemento pivot $x = A[r]$ avrà la stessa probabilità di essere uno qualsiasi degli $r - p + 1$ elementi del sottoarray. Poiché il pivot viene scelto a caso, ci aspettiamo che la ripartizione dell'array di input sia ben bilanciata in media.

Le modifiche da apportare a PARTITION e QUICKSORT sono modeste. Nella nuova procedura di partizionamento, implementiamo semplicemente lo scambio prima dell'effettivo partizionamento:

```
RANDOMIZED-PARTITION(A, p, r)
1  i = RANDOM(p, r)
2  scambia A[r] con A[i]
3  return PARTITION(A, p, r)
```

Il nuovo quicksort chiama RANDOMIZED-PARTITION, anziché PARTITION:

```
RANDOMIZED-QUICKSORT(A, p, r)
1  if p < r
2       q = RANDOMIZED-PARTITION(A, p, r)
3       RANDOMIZED-QUICKSORT(A, p, q − 1)
4       RANDOMIZED-QUICKSORT(A, q + 1, r)
```

Nel prossimo paragrafo analizzeremo questo algoritmo.

Esercizi

7.3-1

Perché dobbiamo analizzare il tempo di esecuzione atteso di un algoritmo randomizzato e non il tempo di esecuzione nel caso peggiore?

7.3-2

Durante l'esecuzione della procedura RANDOMIZED-QUICKSORT, quante chiamate del generatore di numeri casuali RANDOM vengono fatte nel caso peggiore? E nel caso migliore? Esprimete la vostra risposta usando la notazione Θ.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

7.4 Analisi di quicksort

Nel Paragrafo 7.2 abbiamo fornito l'intuizione sul comportamento nel caso peggiore di quicksort e sul perché ci aspettiamo che il suo funzionamento sia rapi-

do. In questo paragrafo analizziamo il comportamento di quicksort in modo più rigoroso. Iniziamo con l'analisi del caso peggiore, che si applica sia a QUICKSORT sia a RANDOMIZED-QUICKSORT, e concludiamo con l'analisi del tempo di esecuzione atteso di RANDOMIZED-QUICKSORT.

7.4.1 Analisi del caso peggiore

Nel Paragrafo 7.2 abbiamo visto che una ripartizione nel caso peggiore a qualsiasi livello di ricorsione in quicksort produce un tempo di esecuzione $\Theta(n^2)$ che, intuitivamente, è il tempo di esecuzione nel caso peggiore dell'algoritmo. Adesso dimostriamo questa asserzione.

Utilizzando il metodo di sostituzione (vedere il Paragrafo 4.3), possiamo dimostrare che il tempo di esecuzione di quicksort è $O(n^2)$. Sia $T(n)$ il tempo nel caso peggiore per la procedura QUICKSORT con un input di dimensione n. Otteniamo la ricorrenza

$$T(n) = \max_{0 \le q \le n-1} (T(q) + T(n-q-1)) + \Theta(n) \tag{7.1}$$

dove il parametro q varia da 0 a $n-1$, in quanto la procedura PARTITION genera due sottoproblemi con dimensione totale $n-1$. Supponiamo che la soluzione sia $T(n) \le cn^2$ per qualche costante c. Sostituendo questa soluzione nella ricorrenza (7.1), otteniamo

$$\begin{aligned} T(n) &\le \max_{0 \le q \le n-1} (cq^2 + c(n-q-1)^2) + \Theta(n) \\ &= c \cdot \max_{0 \le q \le n-1} (q^2 + (n-q-1)^2) + \Theta(n) \end{aligned}$$

L'espressione $q^2 + (n-q-1)^2$ raggiunge il massimo nei due estremi dell'intervallo $0 \le q \le n-1$ del parametro q; infatti, la derivata seconda dell'espressione rispetto a q è positiva (vedere l'Esercizio 7.4-3). Questa osservazione ci fornisce il limite $\max_{0 \le q \le n-1}(q^2 + (n-q-1)^2) \le (n-1)^2 = n^2 - 2n + 1$. Riprendendo l'espressione di $T(n)$, otteniamo

$$\begin{aligned} T(n) &\le cn^2 - c(2n-1) + \Theta(n) \\ &\le cn^2 \end{aligned}$$

perché possiamo assegnare alla costante c un valore sufficientemente grande affinché il termine $c(2n-1)$ prevalga sul termine $\Theta(n)$; quindi, $T(n) = O(n^2)$. Nel Paragrafo 7.2 abbiamo esaminato un caso specifico in cui quicksort impiega un tempo $\Omega(n^2)$: quando il partizionamento è sbilanciato. In alternativa, l'Esercizio 7.4-1 chiede di dimostrare che la ricorrenza (7.1) ha una soluzione $T(n) = \Omega(n^2)$. Quindi, il tempo di esecuzione (nel caso peggiore) di quicksort è $\Theta(n^2)$.

7.4.2 Tempo di esecuzione atteso

Abbiamo già dato una spiegazione intuitiva sul perché il tempo di esecuzione nel caso medio di RANDOMIZED-QUICKSORT sia $O(n \lg n)$: se, in ogni livello di ricorsione, la ripartizione indotta da RANDOMIZED-PARTITION pone una frazione costante qualsiasi degli elementi in un lato della partizione, allora l'albero di ricorsione ha profondità $\Theta(\lg n)$ e in ogni livello viene svolto un lavoro $O(n)$. Anche se aggiungiamo qualche nuovo livello con la ripartizione più sbilanciata possibile tra questi livelli, il tempo totale resta $O(n \lg n)$. Possiamo analizzare con precisione il tempo di esecuzione atteso di RANDOMIZED-QUICKSORT, spiegando prima come opera la procedura di partizionamento e poi sfruttando questa conoscenza per ricavare un limite $O(n \lg n)$ sul tempo di esecuzione atteso. Questo limite superiore sul tempo di esecuzione atteso, combinato con il limite $\Theta(n \lg n)$ nel caso

Acquistato da Michele Michetti su Webster 1 2023-07-17 21:10 Antonio Cadili (ibadili 1955[illegible] 2207210) con Vitalia © 2022 McGraw-Hill Education (Italy)

migliore che abbiamo visto nel Paragrafo 7.2, fornisce un tempo di esecuzione atteso pari a $\Theta(n \lg n)$. Nell'analisi assumeremo che gli elementi da ordinare siano tutti distinti.

Tempo di esecuzione e confronti

QUICKSORT e RANDOMIZED-QUICKSORT differiscono soltanto per il modo in cui scelgono il pivot; per il resto sono identiche. Possiamo quindi esporre la nostra analisi di RANDOMIZED-QUICKSORT discutendo le procedure QUICKSORT e PARTITION, ma facendo l'ipotesi che gli elementi pivot vengano scelti in modo casuale nel sottoarray che viene passato a RANDOMIZED-QUICKSORT. Il tempo di esecuzione di QUICKSORT è dominato dal tempo impiegato nella procedura PARTITION. Ogni volta che viene chiamata la procedura PARTITION, viene selezionato un elemento pivot; questo elemento non sarà mai incluso nelle successive chiamate ricorsive di QUICKSORT e PARTITION. Quindi, ci possono essere al massimo n chiamate di PARTITION durante l'intera esecuzione dell'algoritmo quicksort. Una chiamata di PARTITION impiega il tempo $O(1)$ più una quantità di tempo che è proporzionale al numero di iterazioni del ciclo **for** nelle righe 3–6. Ogni iterazione di questo ciclo **for** effettua un confronto fra l'elemento pivot e un altro elemento dell'array A (riga 4). Pertanto, se contiamo il numero totale di volte che la riga 4 viene eseguita, possiamo limitare il tempo totale impiegato nel ciclo **for** durante l'intera esecuzione di QUICKSORT.

Lemma 7.1
Se X è il numero di confronti svolti nella riga 4 di PARTITION nell'intera esecuzione di QUICKSORT su un array di n elementi, allora il tempo di esecuzione di QUICKSORT è $O(n + X)$.

Dimostrazione Per la precedente discussione, l'algoritmo effettua al massimo n chiamate di PARTITION, ciascuna delle quali svolge una quantità costante di lavoro e poi esegue il ciclo **for** un certo numero di volte. Ogni iterazione del ciclo **for** esegue la riga 4. ■

Il nostro obiettivo, quindi, è calcolare X, il numero totale di confronti svolti in tutte le chiamate di PARTITION. Non tenteremo di analizzare quanti confronti vengono effettuati in *ogni* chiamata di PARTITION. Piuttosto, deriveremo un limite globale sul numero totale di confronti. Per farlo, dobbiamo capire quando l'algoritmo confronta due elementi dell'array e quando non lo fa. Per semplificare l'analisi, rinominiamo $z_1, z_2, \ldots, z_n$ gli elementi dell'array A, dove z_i è l'i-esimo elemento più piccolo. Definiamo anche $Z_{ij} = \{z_i, z_{i+1}, \ldots, z_j\}$ l'insieme degli elementi compresi fra z_i e z_j, estremi inclusi.

Quando l'algoritmo confronta z_i e z_j? Per rispondere a questa domanda, osserviamo innanzitutto che ogni coppia di elementi viene confrontata al massimo una volta. Perché? Gli elementi sono confrontati soltanto con l'elemento pivot e, dopo che una particolare chiamata di PARTITION finisce, il pivot utilizzato in questa chiamata non viene più confrontato con nessun altro elemento.

Acquistato da Michele Michele su Webster il 2022-03-07 23:12. Numero Ordine Libreria: 1995020016-220707-0 Copyright © 2012, McGraw-Hill Education (Italy)

La nostra analisi usa le variabili casuali indicatrici (vedere il Paragrafo 5.2). Definiamo

$$X_{ij} = \mathrm{I}\{z_i \text{ è confrontato con } z_j\}$$

Qui stiamo considerando i confronti che vengono eseguiti in un istante qualsiasi durante l'esecuzione dell'algoritmo, non soltanto durante un'iterazione o una chiamata di PARTITION. Poiché ogni coppia viene confrontata al massimo

una volta, possiamo facilmente rappresentare il numero totale di confronti svolti dall'algoritmo in questo modo:

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$$

Prendendo i valori attesi da entrambi i lati e poi applicando la linearità del valore atteso e il Lemma 5.1, otteniamo

$$\begin{aligned} \mathrm{E}[X] &= \mathrm{E}\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}\right] \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathrm{E}[X_{ij}] \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \Pr\{z_i \text{ è confrontato con } z_j\} \end{aligned} \tag{7.2}$$

Resta da calcolare $\Pr\{z_i \text{ è confrontato con } z_j\}$. La nostra analisi suppone che ogni pivot sia scelto in modo casuale e indipendente. È utile riflettere su quando due elementi *non* sono confrontati. Considerate come input di quicksort i numeri da 1 a 10 (in qualsiasi ordine) e supponete che il primo pivot sia 7. La prima chiamata di PARTITION separa i numeri in due insiemi: $\{1, 2, 3, 4, 5, 6\}$ e $\{8, 9, 10\}$. In questo processo, il pivot 7 viene confrontato con tutti gli altri elementi, ma nessun numero del primo insieme (per esempio, 2) è o sarà mai confrontato con qualsiasi altro numero del secondo insieme (per esempio, 9).

In generale, poiché supponiamo che i valori degli elementi siano distinti, una volta che viene scelto un pivot x con $z_i < x < z_j$, sappiamo che z_i e z_j non potranno essere confrontati in un istante successivo. Se, d'altra parte, viene scelto z_i come pivot prima di qualsiasi altro elemento di Z_{ij}, allora z_i sarà confrontato con ogni elemento di Z_{ij}, tranne sé stesso. Analogamente, se viene scelto z_j come pivot prima di qualsiasi altro elemento di Z_{ij}, allora z_j sarà confrontato con ogni elemento di Z_{ij}, tranne sé stesso. Nell'esempio in esame, vengono confrontati i valori 7 e 9, perché 7 è il primo elemento di $Z_{7,9}$ che viene scelto come pivot. I valori 2 e 9, invece, non saranno mai confrontati perché il primo elemento pivot scelto in $Z_{2,9}$ è 7. Quindi, z_i e z_j vengono confrontati se e soltanto se il primo elemento che verrà scelto come pivot in Z_{ij} è z_i o z_j.

Calcoliamo adesso la probabilità che si verifichi questo evento. Prima del punto in cui un elemento di Z_{ij} viene scelto come pivot, l'intero insieme Z_{ij} si trova nella stessa partizione. Pertanto, qualsiasi elemento di Z_{ij} ha la stessa probabilità di essere scelto per primo come pivot. Poiché l'insieme Z_{ij} ha $j - i + 1$ elementi e poiché i pivot vengono scelti in modo casuale e indipendente, la probabilità che qualsiasi elemento sia il primo a essere scelto come pivot è $1/(j - i + 1)$. Quindi, abbiamo

Acquistato da Michele Michele su Webster il 2022-07-07 23:13 Numero Ordine Libreria 1995032016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned} \Pr\{z_i \text{ è confrontato con } z_j\} &= \Pr\{z_i \text{ o } z_j \text{ è il primo pivot scelto in } Z_{ij}\} \\ &= \Pr\{z_i \text{ è il primo pivot scelto in } Z_{ij}\} \\ &\quad + \Pr\{z_j \text{ è il primo pivot scelto in } Z_{ij}\} \\ &= \frac{1}{j - i + 1} + \frac{1}{j - i + 1} \\ &= \frac{2}{j - i + 1} \end{aligned} \tag{7.3}$$

La seconda riga si basa sul fatto che i due eventi si escludono a vicenda. Combinando le equazioni (7.2) e (7.3), otteniamo

$$\mathrm{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j-i+1}$$

Possiamo calcolare questa sommatoria effettuando un cambio di variabili ($k = j - i$) e applicando il limite sulle serie armoniche espresso dall'equazione (A.7):

$$\begin{aligned}
\mathrm{E}[X] &= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j-i+1} \\
&= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} \\
&< \sum_{i=1}^{n-1} \sum_{k=1}^{n} \frac{2}{k} \\
&= \sum_{i=1}^{n-1} O(\lg n) \\
&= O(n \lg n) \qquad (7.4)
\end{aligned}$$

Quindi concludiamo che, utilizzando RANDOMIZED-PARTITION, il tempo di esecuzione atteso di quicksort è $O(n \lg n)$ quando i valori degli elementi sono distinti.

Esercizi

7.4-1

Dimostrate che nella ricorrenza

$$T(n) = \max_{0 \le q \le n-1} (T(q) + T(n-q-1)) + \Theta(n) ,$$

$T(n) = \Omega(n^2)$.

7.4-2

Dimostrate che il tempo di esecuzione nel caso migliore di quicksort è $\Omega(n \lg n)$.

7.4-3

Dimostrate che $q^2 + (n-q-1)^2$ raggiunge un massimo sui valori $q = 0, 1, \ldots, n-1$ quando $q = 0$ o $q = n-1$.

7.4-4

Dimostrate che il tempo di esecuzione atteso di RANDOMIZED-QUICKSORT è $\Omega(n \lg n)$.

7.4-5

Il tempo di esecuzione di quicksort può essere migliorato in pratica sfruttando il fatto che insertion sort viene eseguito rapidamente se il suo input è "quasi" ordinato. Quando quicksort viene chiamato a operare su un sottoarray con meno di k elementi, lasciatelo terminare senza ordinare il sottoarray. Dopo che la chiamata principale di quicksort è completata, eseguite insertion sort sull'intero array per finire il processo di ordinamento. Dimostrate che questo algoritmo di ordinamento viene eseguito nel tempo atteso $O(nk + n \lg(n/k))$. Come dovrebbe essere scelto k in teoria e in pratica?

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

7.4-6 ⋆

Modificate la procedura PARTITION selezionando casualmente tre elementi dall'array A e partizionando rispetto alla loro mediana (il valore centrale dei tre elementi). Approssimate la probabilità di ottenere nel caso peggiore una ripartizione nella proporzione α a $(1 - \alpha)$ come una funzione di α nell'intervallo $0 < \alpha < 1$.

Problemi

7-1 Correttezza della partizione di Hoare

La versione di PARTITION presentata in questo capitolo non è l'algoritmo originale di partizionamento. La versione originale, che è stata ideata da C. A. R. Hoare, è questa:

HOARE-PARTITION(A, p, r)

```
 1  x = A[p]
 2  i = p − 1
 3  j = r + 1
 4  while TRUE
 5       repeat
 6            j = j − 1
 7       until A[j] ≤ x
 8       repeat
 9            i = i + 1
10       until A[i] ≥ x
11       if i < j
12            scambia A[i] con A[j]
13       else return j
```

a. Descrivete il funzionamento di HOARE-PARTITION sull'array $A = \langle 13, 19, 9, 5, 12, 8, 7, 4, 11, 2, 6, 21\rangle$, indicando i valori dell'array e i valori ausiliari dopo ogni iterazione del ciclo **while** nelle righe 4–13.

Le prossime tre domande richiedono un attento ragionamento sulla correttezza della procedura HOARE-PARTITION. Dimostrate che:

b. Gli indici i e j sono tali che non si accede mai a un elemento di A esterno al sottoarray $A[p\,..\,r]$.

c. Quando HOARE-PARTITION termina, restituisce un valore di j tale che $p \leq j < r$.

d. Ogni elemento di $A[p\,..\,j]$ è minore o uguale di qualsiasi elemento di $A[j + 1\,..\,r]$ quando HOARE-PARTITION termina.

La procedura PARTITION nel Paragrafo 7.1 separa il valore del pivot (originariamente in $A[r]$) dalle due partizioni che forma. La procedura HOARE-PARTITION, d'altra parte, inserisce sempre il valore del pivot (originariamente in $A[p]$) in una delle due partizioni $A[p\,..\,j]$ e $A[j + 1\,..\,r]$. Poiché $p \leq j < r$, questa ripartizione non è mai banale.

Acquistato da Michele Michele dal Web il 2023-07-07 23:37 Numero Ordine Libreria 1995030606237707 © Copyright © 2022 McGraw-Hill Education (Italy)

e. Riscrivete la procedura QUICKSORT per utilizzare HOARE-PARTITION.

7-2 Quicksort con elementi di valore uguale

L'analisi del tempo di esecuzione atteso descritta nel Paragrafo 7.4.2 suppone che tutti gli elementi abbiano valori distinti. In questo problema, esaminiamo che cosa accade quando tale ipotesi non è vera.

a. Supponiamo che tutti gli elementi abbiamo lo stesso valore. Qual è il tempo di esecuzione di quicksort randomizzato in questo caso?

b. La procedura PARTITION restituisce un indice q tale che ciascun elemento di $A[p\,..\,q-1]$ sia minore o uguale ad $A[q]$ e ogni elemento di $A[q+1\,..\,r]$ sia maggiore di $A[q]$. Modificate PARTITION per ottenere una procedura PARTITION$'(A, p, r)$ che permuta gli elementi di $A[p\,..\,r]$ e restituisce due indici q e t, con $p \leq q \leq t \leq r$, tali che

- tutti gli elementi di $A[q\,..\,t]$ siano uguali
- ogni elemento di $A[p\,..\,q-1]$ sia minore di $A[q]$
- ogni elemento di $A[t+1\,..\,r]$ sia maggiore di $A[q]$

Come PARTITION, anche PARTITION$'$ dovrà richiedere un tempo $\Theta(r-p)$.

c. Modificate RANDOMIZED-QUICKSORT per chiamare PARTITION$'$, e chiamate la nuova procedura RANDOMIZED-QUICKSORT$'$. Poi modificate QUICKSORT per ottenere una procedura QUICKSORT$'(p, r)$ che chiama RANDOMIZED-PARTITION$'$ ed esegue le ricorsioni soltanto sulle partizioni di cui non sappiamo già che tutti gli elementi sono uguali.

d. Utilizzando QUICKSORT$'$, come dovrebbe essere modificata l'analisi nel Paragrafo 7.4.2 per evitare l'ipotesi che tutti gli elementi abbiano valori diversi?

7-3 Analisi alternativa di quicksort

Un'analisi alternativa del tempo di esecuzione dell'algoritmo quicksort randomizzato si focalizza sul tempo di esecuzione atteso di ogni singola chiamata ricorsiva di RANDOMIZED-QUICKSORT, anziché sul numero di confronti effettuati.

a. Dimostrate che, dato un array di dimensione n, la probabilità che un particolare elemento sia scelto come pivot è $1/n$. Sulla base di questa dimostrazione definite le variabili casuali indicatrici $X_i = \mathrm{I}\{\text{l'}i\text{-esimo elemento più piccolo è scelto come pivot}\}$. Qual è il valore atteso $\mathrm{E}[X_i]$?

b. Sia $T(n)$ una variabile casuale che indica il tempo di esecuzione di quicksort con un array di dimensione n. Dimostrate che

$$\mathrm{E}[T(n)] = \mathrm{E}\left[\sum_{q=1}^{n} X_q \left(T(q-1) + T(n-q) + \Theta(n)\right)\right] \tag{7.5}$$

c. Dimostrate che l'equazione (7.5) può essere riscritta così:

$$\mathrm{E}[T(n)] = \frac{2}{n}\sum_{q=2}^{n-1} \mathrm{E}[T(q)] + \Theta(n) \tag{7.6}$$

d. Dimostrate che

$$\sum_{k=2}^{n-1} k \lg k \leq \frac{1}{2}n^2 \lg n - \frac{1}{8}n^2 \tag{7.7}$$

Acquistato da Michele Michele su Webster il 2022-07-07 22:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

(*Suggerimento:* scomponete la sommatoria in due parti, una per $k = 2, 3, \ldots, \lceil n/2 \rceil - 1$ e l'altra per $k = \lceil n/2 \rceil, \ldots, n-1$.)

e. Applicando il limite dell'equazione (7.7), dimostrate che la soluzione della ricorrenza nell'equazione (7.6) è $\mathrm{E}[T(n)] = \Theta(n \lg n)$ (*suggerimento:* dimostrate, per sostituzione, che $\mathrm{E}[T(n)] \leq an \lg n$ per valori di n sufficientemente grandi e per qualche costante positiva a).

7-4 Profondità dello stack per quicksort

L'algoritmo QUICKSORT descritto nel Paragrafo 7.1 contiene due chiamate ricorsive a sé stesso. Dopo che QUICKSORT ha chiamato PARTITION, essa ordina prima ricorsivamente il sottoarray sinistro e poi quello destro. La seconda chiamata ricorsiva in QUICKSORT non è veramente necessaria; può essere evitata utilizzando una struttura di controllo iterativa. Questa tecnica, detta ***ricorsione in coda***, è fornita automaticamente dai buoni compilatori. Considerate la seguente versione di quicksort, che simula la ricorsione in coda.

```
TAIL-RECURSIVE-QUICKSORT(A, p, r)
1  while p < r
2      // Partiziona e ordina il sottoarray sinistro
3      q = PARTITION(A, p, r)
4      TAIL-RECURSIVE-QUICKSORT(A, p, q − 1)
5      p = q + 1
```

a. Dimostrate che TAIL-RECURSIVE-QUICKSORT$(A, 1, A.length)$ ordina correttamente gli elementi dell'array A.

I compilatori di solito eseguono le procedure ricorsive utilizzando uno ***stack*** (o pila) che contiene informazioni pertinenti a ogni chiamata ricorsiva, inclusi i valori dei parametri. Le informazioni per la chiamata più recente si trovano in cima allo stack, mentre le informazioni per la chiamata iniziale si trovano in fondo allo stack. Quando una procedura viene chiamata, le sue informazioni vengono inserite (***push***) nello stack; quando termina, le sue informazioni vengono estratte (***pop***) dallo stack. Poiché si presuppone che i parametri di tipo array siano rappresentati da puntatori, le informazioni per ogni chiamata di procedura occupano uno spazio $O(1)$ nello stack. La ***profondità dello stack*** è la quantità massima di spazio utilizzato nello stack in un istante qualsiasi durante un processo.

b. Descrivete una situazione in cui la profondità dello stack di TAIL-RECURSIVE-QUICKSORT è $\Theta(n)$ con un array di input di n elementi.

c. Modificate il codice di TAIL-RECURSIVE-QUICKSORT in modo che lo stack nel caso peggiore sia $\Theta(\lg n)$, pur mantenendo il tempo di esecuzione atteso $O(n \lg n)$.

7-5 Partizione con la mediana-di-3

Un modo per migliorare la procedura RANDOMIZED-QUICKSORT consiste nell'effettuare il partizionamento scegliendo opportunamente (non casualmente) il pivot nel sottoarray. Un tipico approccio è il metodo della ***mediana-di-3***: scegliere come pivot la mediana (elemento centrale) di un insieme di 3 elementi selezionati a caso dal sottoarray (vedere l'Esercizio 7.4-6). Per questo problema, supponete che gli elementi nell'array di input $A[1 \,..\, n]$ siano distinti e che $n \geq 3$. Indicate con $A'[1 \,..\, n]$ l'array di output ordinato. Applicando il metodo della mediana-di-3 per scegliere il pivot x, definite $p_i = \Pr\{x = A'[i]\}$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

a. Scrivete una formula esatta di p_i in funzione di n e i per $i = 2, 3, \ldots, n-1$ (notate che $p_1 = p_n = 0$).

b. Di quanto abbiamo aumentato la probabilità di scegliere come pivot $x = A'[\lfloor (n+1)/2 \rfloor]$, la mediana di $A[1\,..\,n]$, rispetto all'implementazione normale? Supponete che $n \to \infty$ e calcolate il limite del rapporto fra queste probabilità.

c. Se definiamo "buona" una ripartizione in cui il pivot scelto è $x = A'[i]$, dove $n/3 \leq i \leq 2n/3$, di quanto abbiamo aumentato la probabilità di ottenere una buona ripartizione rispetto all'implementazione normale? (*Suggerimento:* approssimate la sommatoria con un integrale.)

d. Dedurre che il metodo della mediana-di-3 influisce soltanto sul fattore costante nel tempo di esecuzione $\Omega(n \lg n)$ di quicksort.

7-6 Ordinamento fuzzy degli intervalli

Considerate un problema di ordinamento in cui i numeri non sono noti con esattezza. Piuttosto, per ogni numero, conosciamo un intervallo nella retta reale cui appartiene il numero. In altre parole, conosciamo n intervalli chiusi nella forma $[a_i, b_i]$, dove $a_i \leq b_i$. L'obiettivo consiste nell'effettuare un ***ordinamento fuzzy*** di questi intervalli, cioè produrre una permutazione $\langle i_1, i_2, \ldots, i_n \rangle$ degli intervalli in modo che esistano valori $c_j \in [a_{i_j}, b_{i_j}]$ che soddisfano $c_1 \leq c_2 \leq \cdots \leq c_n$.

a. Progettate un algoritmo randomizzato che effettua un ordinamento fuzzy di n intervalli. L'algoritmo deve avere la struttura generale di un algoritmo che usa quicksort per ordinare rapidamente gli estremi sinistri degli intervalli (i valori a_i), ma deve sfruttare la sovrapposizione degli intervalli per migliorare il tempo di esecuzione (all'aumentare della sovrapposizione degli intervalli, il problema dell'ordinamento fuzzy degli intervalli diventa sempre più facile).

b. Dimostrate che, in generale, il tempo di esecuzione atteso del vostro algoritmo è $\Theta(n \lg n)$; questo tempo, però, diventa $\Theta(n)$ quando tutti gli intervalli si sovrappongono (cioè quando esiste un valore x tale che $x \in [a_i, b_i]$ per ogni i). Il vostro algoritmo non deve controllare esplicitamente questo caso speciale; piuttosto, le sue prestazioni dovrebbero migliorare naturalmente all'aumentare dell'entità della sovrapposizione degli intervalli.

Note

La procedura quicksort è stata inventata da Hoare [171]; la versione di Hoare è riportata nel Problema 7-1. La procedura PARTITION presentata nel Paragrafo 7.1 è stata ideata da N. Lomuto. L'analisi svolta nel Paragrafo 7.4 è stata sviluppata da Avrim Blum. Consultate Sedgewick [306] e Bentley [43] per informazioni dettagliate sulle tecniche di implementazione di queste procedure.

McIlroy [249] ha descritto come progettare un "avversario killer" che produce un array con il quale virtualmente qualsiasi implementazione di quicksort impiega un tempo $\Theta(n^2)$. Se l'implementazione è randomizzata, l'avversario killer genera l'array dopo avere esaminato le scelte casuali dell'algoritmo quicksort.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Ordinamento in tempo lineare 8

Abbiamo già introdotto parecchi algoritmi che possono ordinare n numeri nel tempo $O(n \lg n)$. Heapsort e merge sort raggiungono questo limite superiore nel caso peggiore; quicksort lo raggiunge nel caso medio. Inoltre, per ciascuno di questi algoritmi, possiamo produrre una sequenza di n numeri di input tale che l'algoritmo venga eseguito nel tempo $\Omega(n \lg n)$.

Questi algoritmi condividono un'interessante proprietà: *l'ordinamento che effettuano è basato soltanto su confronti fra gli elementi di input*. Questi algoritmi sono detti ***ordinamenti per confronti***. Tutti gli algoritmi di ordinamento presentati finora sono ordinamenti per confronti.

Nel Paragrafo 8.1 dimostreremo che qualsiasi ordinamento per confronti deve effettuare $\Omega(n \lg n)$ confronti nel caso peggiore per ordinare n elementi. Quindi, merge sort e heapsort sono asintoticamente ottimali e non esiste un ordinamento per confronti che sia più veloce per più di un fattore costante.

I Paragrafi 8.2, 8.3 e 8.4 esaminano tre algoritmi di ordinamento – counting sort, radix sort e bucket sort – che vengono eseguiti in tempo lineare. Ovviamente questi algoritmi usano operazioni diverse dai confronti per effettuare l'ordinamento. Di conseguenza, il limite inferiore $\Omega(n \lg n)$ non vale per questi algoritmi.

8.1 Limiti inferiori per l'ordinamento

In un ordinamento per confronti usiamo soltanto i confronti fra gli elementi per ottenere informazioni sull'ordinamento di una sequenza di input $\langle a_1, a_2, \ldots, a_n \rangle$. Ovvero, dati due elementi a_i e a_j, eseguiamo uno dei test $a_i < a_j$, $a_i \leq a_j$, $a_i = a_j$, $a_i \geq a_j$ o $a_i > a_j$ per determinare il loro ordine relativo. Non possiamo esaminare i valori degli elementi o ottenere informazioni sul loro ordine in altri modi.

In questo paragrafo supponiamo, senza perdere di generalità, che tutti gli elementi di input siano distinti. Fatta questa ipotesi, confronti della forma $a_i = a_j$ sono inutili, quindi possiamo supporre che non saranno fatti confronti di questo tipo. Inoltre notate che i confronti $a_i \leq a_j$, $a_i \geq a_j$, $a_i > a_j$ e $a_i < a_j$ sono tutti equivalenti, perché forniscono le stesse informazioni sull'ordine relativo di a_i e a_j. Quindi, supporremo che tutti i confronti abbiano la forma $a_i \leq a_j$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Il modello dell'albero di decisione

Gli ordinamenti per confronti possono essere visti astrattamente in termini di ***alberi di decisione***. Un albero di decisione è un albero binario pieno che rappresenta i confronti fra elementi che vengono effettuati da un particolare algoritmo di ordinamento che opera su un input di una data dimensione. Il controllo, lo

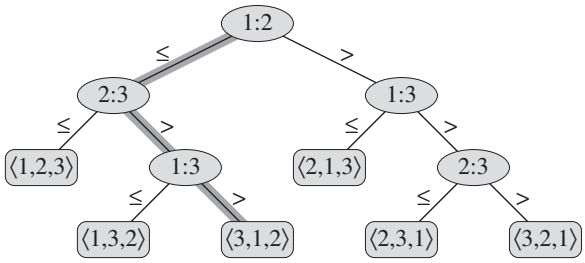


Figura 8.1 L'albero di decisione per l'algoritmo insertion sort che opera su tre elementi. Un nodo interno annotato con $i:j$ indica un confronto fra a_i e a_j. Una foglia annotata con la permutazione $\langle \pi(1), \pi(2), \ldots, \pi(n) \rangle$ indica l'ordinamento $a_{\pi(1)} \leq a_{\pi(2)} \leq \cdots \leq a_{\pi(n)}$. Il percorso ombreggiato indica le decisioni prese durante l'ordinamento della sequenza di input $\langle a_1 = 6, a_2 = 8, a_3 = 5 \rangle$; la permutazione $\langle 3, 1, 2 \rangle$ nella foglia indica che l'ordinamento è $a_3 = 5 \leq a_1 = 6 \leq a_2 = 8$. Ci sono $3! = 6$ permutazioni possibili degli elementi di input, quindi l'albero di decisione deve avere almeno 6 foglie.

spostamento dei dati e tutti gli altri aspetti dell'algoritmo vengono ignorati. La Figura 8.1 illustra l'albero di decisione che corrisponde all'algoritmo insertion sort (descritto nel Paragrafo 2.1) che opera su una sequenza di input di tre elementi.

In un albero di decisione, ogni nodo interno è annotato con $i:j$ per qualche i e j nell'intervallo $1 \leq i, j \leq n$, dove n è il numero di elementi nella sequenza di input. Ogni foglia è annotata con una permutazione $\langle \pi(1), \pi(2), \ldots, \pi(n) \rangle$ (consultare il Paragrafo C.1 per informazioni sulle permutazioni). L'esecuzione dell'algoritmo di ordinamento corrisponde a tracciare un cammino semplice dalla radice dell'albero di decisione fino a una foglia. Ogni nodo interno rappresenta un confronto $a_i \leq a_j$. Il sottoalbero sinistro detta i successivi confronti per $a_i \leq a_j$; il sottoalbero destro detta i successivi confronti per $a_i > a_j$.

Quando raggiunge una foglia, l'algoritmo ha stabilito l'ordinamento $a_{\pi(1)} \leq a_{\pi(2)} \leq \cdots \leq a_{\pi(n)}$. Poiché qualsiasi algoritmo di ordinamento corretto deve essere in grado di produrre ogni permutazione del suo input, una condizione necessaria affinché un ordinamento per confronti sia corretto è che ciascuna delle $n!$ permutazioni di n elementi appaia come una delle foglie dell'albero di decisione e che ciascuna di queste foglie sia raggiungibile dalla radice attraverso un percorso che corrisponde a una effettiva esecuzione dell'ordinamento per confronti (queste foglie saranno chiamate "raggiungibili"). Quindi, considereremo soltanto alberi di decisione in cui ogni permutazione compare in una foglia raggiungibile.

Un limite inferiore per il caso peggiore

La lunghezza del cammino semplice più lungo dalla radice di un albero di decisione a una delle sue foglie raggiungibili rappresenta il numero di confronti che svolge il corrispondente algoritmo di ordinamento nel caso peggiore. Di conseguenza, il numero di confronti nel caso peggiore per un dato algoritmo di ordinamento per confronti è uguale all'altezza del suo albero di decisione. Un limite inferiore sulle altezze di tutti gli alberi di decisione, dove ogni permutazione compare in una foglia raggiungibile, è pertanto un limite inferiore sul tempo di esecuzione di qualsiasi algoritmo di ordinamento per confronti. Il seguente teorema stabilisce questo limite inferiore.

Teorema 8.1

Qualsiasi algoritmo di ordinamento per confronti richiede $\Omega(n \lg n)$ confronti nel caso peggiore.

Acquistato da Michele Michele su Webster il 2022-07-15 12:12 - Napoli Online Libraria Tutti i diritti riservati - Copyright © 2022, Mondadori Education (Italy)

Dimostrazione Per quanto detto in precedenza, è sufficiente determinare l'altezza di un albero di decisione dove ogni permutazione appare come una foglia raggiungibile. Considerate un albero di decisione di altezza h con l foglie raggiungibili che corrisponde a un ordinamento per confronti di n elementi. Poiché

ciascuna delle $n!$ permutazioni dell'input compare in una foglia, si ha $n! \leq l$. Dal momento che un albero binario di altezza h non ha più di 2^h foglie, si ha

$n! \leq l \leq 2^h$

Prendendo i logaritmi, questa relazione implica che

$h \geq \lg(n!)$ (perché la funzione lg è monotonicamente crescente)
$= \Omega(n \lg n)$ (per l'equazione (3.19)) ■

Corollario 8.2
Heapsort e merge sort sono algoritmi di ordinamento per confronti asintoticamente ottimali.

Dimostrazione I limiti superiori $O(n \lg n)$ sui tempi di esecuzione per heapsort e merge sort corrispondono al limite inferiore $\Omega(n \lg n)$ nel caso peggiore del Teorema 8.1. ■

Esercizi

8.1-1
Qual è la profondità minima possibile di una foglia in un albero di decisione per un ordinamento per confronti?

8.1-2
Determinate i limiti asintoticamente stretti per $\lg(n!)$ senza applicare l'approssimazione di Stirling; valutate invece la sommatoria $\sum_{k=1}^{n} \lg k$ con le tecniche descritte nel Paragrafo A.2.

8.1-3
Dimostrate che non esiste un ordinamento per confronti il cui tempo di esecuzione è lineare per almeno metà degli $n!$ input di lunghezza n. Che cosa accade per una frazione $1/n$ degli input di lunghezza n? Che cosa accade per una frazione $1/2^n$?

8.1-4
Sia data una sequenza di n elementi da ordinare. La sequenza di input è formata da n/k sottosequenze, ciascuna di k elementi. Gli elementi di una sottosequenza sono tutti più piccoli degli elementi della successiva sottosequenza e più grandi degli elementi della precedente sottosequenza. Quindi, per ordinare l'intera sequenza di lunghezza n basta ordinare i k elementi in ciascuna delle n/k sottosequenze. Dimostrate che $\Omega(n \lg k)$ è un limite inferiore sul numero di confronti necessari a risolvere questa variante del problema di ordinamento (*suggerimento:* non è corretto combinare semplicemente i limiti inferiori delle singole sottosequenze).

8.2 Counting sort

Acquistato da Michele M [illegible] Copyright © 2005 Casa Editrice Ambrosiana - 2010 The McGraw-Hill Companies, S.r.l. - 2012 McGraw-Hill Education (Italy)

L'algoritmo ***counting sort*** suppone che ciascuno degli n elementi di input sia un numero intero compreso nell'intervallo da 0 a k, per qualche intero k. Quando $k = O(n)$, l'ordinamento viene effettuato nel tempo $\Theta(n)$.

Counting sort determina per ogni elemento di input x, il numero di elementi minori di x. Esso usa questa informazione per inserire l'elemento x direttamente nella sua posizione nell'array di output. Per esempio, se ci sono 17 elementi minori di x, allora x deve andare nella posizione di output 18. Questo schema

deve essere modificato leggermente per gestire il caso in cui più elementi hanno lo stesso valore, per evitare che siano inseriti nella stessa posizione.

Nel codice di counting sort, supponiamo che l'input sia un array $A[1..n]$, quindi $length[A] = n$. Occorrono altri due array: l'array $B[1..n]$ contiene l'output ordinato; l'array $C[0..k]$ fornisce la memoria temporanea di lavoro.

```
COUNTING-SORT(A, B, k)
 1  sia C[0..k] un nuovo array
 2  for i = 0 to k
 3      C[i] = 0
 4  for j = 1 to A.length
 5      C[A[j]] = C[A[j]] + 1
 6  // C[i] adesso contiene il numero di elementi uguali a i.
 7  for i = 1 to k
 8      C[i] = C[i] + C[i − 1]
 9  // C[i] adesso contiene il numero di elementi minori o uguali a i.
10  for j = A.length downto 1
11      B[C[A[j]]] = A[j]
12      C[A[j]] = C[A[j]] − 1
```

La Figura 8.2 illustra counting sort. Dopo che il ciclo **for** (righe 2–3) inizializza a zero tutti gli elementi dell'array C, ogni elemento di input viene esaminato nelle righe 4–5 del ciclo **for**. Se il valore di un elemento di input è i, incrementiamo $C[i]$. Quindi, dopo la riga 5, $C[i]$ contiene il numero degli elementi di input uguali a i per ogni intero $i = 0, 1, \ldots, k$. Le righe 7–8 determinano, per ogni $i = 0, 1, \ldots, k$, quanti elementi di input sono minori o uguali a i, mantenendo la somma corrente dell'array C.

Infine, le righe 10–12 del ciclo **for** inseriscono l'elemento $A[j]$ nella corretta posizione ordinata dell'array di output B. Se tutti gli n elementi sono distinti, quando viene eseguita per la prima volta la riga 10, per ogni $A[j]$, il valore $C[A[j]]$ rappresenta la posizione finale corretta di $A[j]$ nell'array di output, in quanto ci sono $C[A[j]]$ elementi minori o uguali ad $A[j]$. Poiché gli elementi potrebbero non essere distinti, $C[A[j]]$ viene ridotto ogni volta che viene inserito un valore $A[j]$ nell'array B. La riduzione di $C[A[j]]$ fa sì che il successivo elemento di input con un valore uguale ad $A[j]$, se esiste, venga inserito nella posizione immediatamente prima di $A[j]$ nell'array di output.

Quanto tempo richiede counting sort? Il ciclo **for** alle righe 2–3 impiega un tempo $\Theta(k)$, il ciclo **for** alle righe 4–5 impiega un tempo $\Theta(n)$, il ciclo **for** alle righe 7–8 impiega un tempo $\Theta(k)$ e il ciclo **for** alle righe 10–12 impiega un tempo $\Theta(n)$. Quindi, il tempo totale è $\Theta(k + n)$. Di solito counting sort viene utilizzato quando $k = O(n)$, nel qual caso il tempo di esecuzione è $\Theta(n)$.

Counting sort batte il limite inferiore di $\Omega(n \lg n)$ dimostrato nel Paragrafo 8.1 perché non è un ordinamento per confronti. Infatti, il codice non effettua alcun confronto fra gli elementi di input. Piuttosto, counting sort usa i valori effettivi degli elementi come indici di un array. Il limite inferiore $\Omega(n \lg n)$ non vale se ci allontaniamo dal modello di ordinamento per confronti.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995093016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

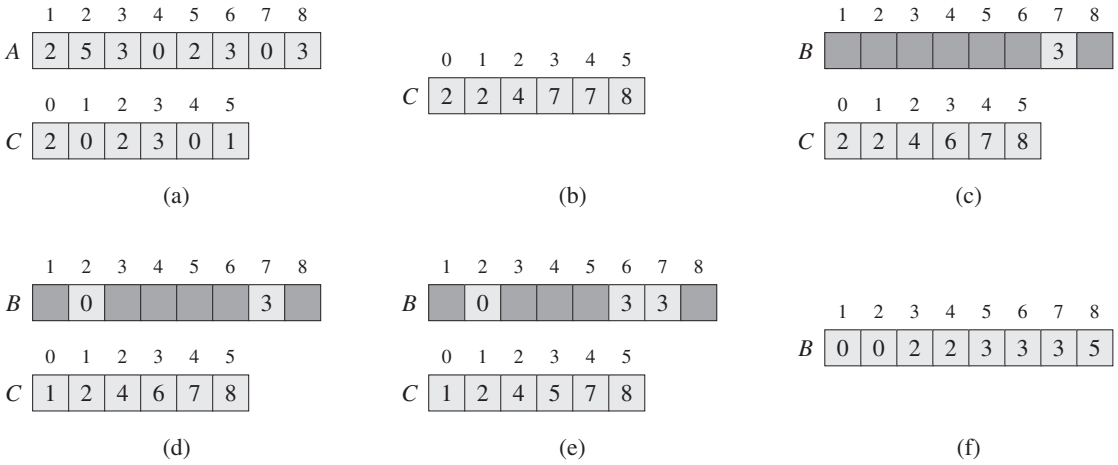


Figura 8.2 Il funzionamento di COUNTING-SORT su un array di input $A[1\,..\,8]$, dove ogni elemento di A è un intero non negativo non maggiore di $k = 5$. **(a)** L'array A e l'array ausiliario C dopo la riga 5. **(b)** L'array C dopo la riga 8. **(c)–(e)** L'array di output B e l'array ausiliario C, rispettivamente, dopo una, due e tre iterazioni del ciclo (righe 10–12). Le caselle di colore grigio chiaro nell'array B rappresentano gli elementi che sono stati inseriti. **(f)** L'array di output B è ordinato.

Un'importante proprietà di counting sort è la ***stabilità***: i numeri con lo stesso valore si presentano nell'array di output nello stesso ordine in cui si trovano nell'array di input. Ovvero, l'uguaglianza di due numeri viene risolta applicando la seguente regola: il numero che si presenta per primo nell'array di input sarà inserito per primo nell'array di output. Normalmente, la proprietà della stabilità è importante soltanto quando i dati satellite vengono spostati insieme con gli elementi da ordinare. La stabilità di counting sort è importante per un'altra ragione: counting sort viene spesso utilizzato come subroutine di radix sort. Come vedremo nel prossimo paragrafo, la stabilità di counting sort è cruciale per la correttezza di radix sort.

Esercizi

8.2-1

Utilizzando la Figura 8.2 come modello, illustrate il funzionamento di COUNTING-SORT sull'array $A = \langle 6, 0, 2, 0, 1, 3, 4, 6, 1, 3, 2 \rangle$.

8.2-2

Dimostrate che COUNTING-SORT è stabile.

8.2-3

Supponete che la riga 10 del ciclo **for** nella procedura COUNTING-SORT sia modificata così

10 **for** $j = 1$ **to** $A.length$

Dimostrate che l'algoritmo opera ancora correttamente. L'algoritmo modificato è stabile?

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1993d3016-22670750 Copyright © 2022, McGraw-Hill Education (Italy)

8.2-4

Descrivete un algoritmo che, dati n numeri interi compresi in un intervallo da 0 a k, svolga un'analisi preliminare del suo input e poi risponda nel tempo $O(1)$ a qualsiasi domanda su quanti degli n interi ricadono nell'intervallo $[a\,..\,b]$. Il vostro algoritmo dovrebbe impiegare un tempo $\Theta(n+k)$ per l'analisi preliminare.

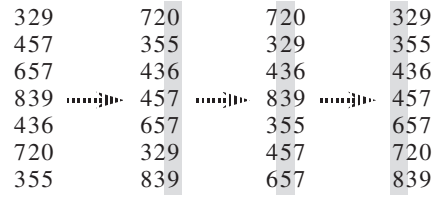


Figura 8.3 Il funzionamento di radix sort su una lista di sette numeri di 3 cifre. La prima colonna a sinistra è l'input. La altre colonne mostrano la lista dopo gli ordinamenti successivi rispetto alle cifre in posizioni di significatività crescente. L'ombreggiatura indica la posizione della cifra sulla quale è stato effettuato l'ordinamento per generare ciascuna lista dalla precedente.

8.3 Radix sort

Radix sort è l'algoritmo utilizzato dalle macchine per ordinare le schede perforate, che adesso si trovano soltanto nei musei di calcolatori. Le schede sono composte da 80 colonne e ogni colonna può essere perforata in una delle 12 posizioni disponibili. La macchina ordinatrice può essere meccanicamente "programmata" per esaminare una particolare colonna di ogni scheda di un mazzo e poi distribuire le schede in 12 contenitori in funzione della posizione perforata. Un operatore può così raccogliere le schede nei vari contenitori, in modo che le schede perforate nella prima posizione si trovino sopra quelle perforate nella seconda posizione e così via.

Per le cifre decimali, sono utilizzate soltanto 10 posizioni in ogni colonna (le altre due posizioni sono utilizzate per codificare i caratteri non numerici). Un numero di d cifre occupa un campo di d colonne. Poiché la macchina ordinatrice può esaminare una sola colonna alla volta, il problema di ordinare n schede rispetto a un numero di d cifre richiede un algoritmo di ordinamento.

Intuitivamente, vorremmo ordinare i numeri in base alla loro cifra *più significativa*, ordinare ciascuno dei contenitori risultanti in modo ricorsivo e, poi, combinare ordinatamente i mazzi delle schede. Purtroppo, poiché le schede in 9 dei 10 contenitori devono essere tenute da parte mentre ordiniamo un singolo contenitore, questa procedura genera molte pile intermedie di schede da gestire (vedere l'Esercizio 8.3-5).

Radix sort risolve il problema dell'ordinamento delle schede in una maniera contraria all'intuizione, ordinando prima le schede in base alla cifra *meno significativa*. Le schede vengono poi combinate in un unico mazzo: le schede del contenitore 0 precedono quelle del contenitore 1 che, a loro volta, precedono quelle del contenitore 2 e così via. Poi tutto il mazzo viene ordinato di nuovo in funzione della seconda cifra meno significativa e ricombinato in maniera analoga. Il processo continua finché le schede saranno ordinate rispetto a tutte le d cifre. È importante notare che a questo punto le schede sono completamente ordinate rispetto al numero di d cifre. Quindi, occorrono soltanto d passaggi attraverso il mazzo per completare l'ordinamento. La Figura 8.3 mostra come opera radix sort con un "mazzo" di sette numeri di 3 cifre.

È essenziale che gli ordinamenti delle cifre in questo algoritmo siano stabili. L'ordinamento svolto da una macchina ordinatrice di schede è stabile, ma l'operatore deve stare attento a non cambiare l'ordine delle schede mentre escono da un contenitore, anche se tutte le schede in un contenitore hanno la stessa cifra nella colonna scelta.

In un tipico calcolatore, che è una macchina sequenziale ad accesso casuale, radix sort viene talvolta utilizzato per ordinare record di informazioni con più campi chiave. Un tipico esempio è l'ordinamento delle date in base a tre chiavi:

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707 0 Copyright © 2022, McGraw-Hill Education (Italy)

anno, mese e giorno. Potremmo eseguire un algoritmo di ordinamento con una funzione di confronto che opera su due date così: confronta gli anni e, nel caso di pareggio, confronta i mesi; nel caso di un altro pareggio, confronta i giorni. In alternativa, potremmo ordinare le informazioni tre volte con un ordinamento stabile: prima in base al giorno, poi in base al mese e, infine, in base all'anno.

Il codice per radix sort è semplice. La seguente procedura suppone che ogni elemento nell'array A di n elementi abbia d cifre, dove la cifra 1 è quella di ordine più basso e la cifra d è quella di ordine più alto.

```
RADIX-SORT(A, d)
1  for i = 1 to d
2      usa un ordinamento stabile per ordinare l'array A sulla cifra i
```

Lemma 8.3
Dati n numeri di d cifre, dove ogni cifra può avere fino a k valori possibili, la procedura RADIX-SORT ordina correttamente i numeri nel tempo $\Theta(d(n + k))$, se l'ordinamento stabile utilizzato dalla procedura impiega un tempo $\Theta(n + k)$.

Dimostrazione La correttezza di radix sort si dimostra per induzione sulla colonna da ordinare (vedere l'Esercizio 8.3-3). L'analisi del tempo di esecuzione dipende dall'ordinamento stabile che viene utilizzato come algoritmo di ordinamento intermedio. Se ogni cifra si trova nell'intervallo da 0 a $k - 1$ (in modo che possa assumere k valori possibili) e k non è troppo grande, counting sort è la scelta ovvia da fare. Ogni passaggio su n numeri di d cifre richiede un tempo $\Theta(n+k)$. Poiché ci sono d passaggi, il tempo totale di radix sort è $\Theta(d(n+k))$. ■

Quando d è costante e $k = O(n)$, radix sort viene eseguito in tempo lineare. Più in generale, abbiamo una certa flessibilità sul modo in cui ripartire le singole chiavi in cifre.

Lemma 8.4
Dati n numeri di b bit e un intero positivo $r \leq b$, RADIX-SORT ordina correttamente questi numeri nel tempo $\Theta((b/r)(n + 2^r))$ se l'algoritmo di ordinamento stabile usato richiede tempo $\Theta(n + k)$ per input nell'intervallo da 0 a k.

Dimostrazione Per un valore $r \leq b$, consideriamo ogni chiave come se avesse $d = \lceil b/r \rceil$ cifre di r bit ciascuna. Ogni cifra è un numero intero compreso nell'intervallo da 0 a $2^r - 1$, quindi possiamo utilizzare counting sort con $k = 2^r - 1$ (per esempio, possiamo considerare una parola di 32 bit come se avesse 4 cifre di 8 bit; quindi $b = 32$, $r = 8$, $k = 2^r - 1 = 255$ e $d = b/r = 4$). Ogni passaggio di counting sort richiede il tempo $\Theta(n + k) = \Theta(n + 2^r)$; poiché ci sono d passaggi, il tempo di esecuzione totale è $\Theta(d(n + 2^r)) = \Theta((b/r)(n + 2^r))$. ■

Dati i valori di n e b, scegliamo il valore di r, con $r \leq b$, che rende minima l'espressione $(b/r)(n+2^r)$. Se $b < \lfloor \lg n \rfloor$, allora per qualsiasi valore di $r \leq b$, si ha $(n + 2^r) = \Theta(n)$. Quindi, scegliendo $r = b$, si ottiene un tempo di esecuzione $(b/b)(n + 2^b) = \Theta(n)$, che è asintoticamente ottimale. Se $b \geq \lfloor \lg n \rfloor$, allora scegliendo $r = \lfloor \lg n \rfloor$ si ottiene il tempo migliore a meno di un fattore costante, cosa che possiamo verificare nel modo seguente. Scegliendo $r = \lfloor \lg n \rfloor$, si ha un tempo di esecuzione pari a $\Theta(bn/\lg n)$. Aumentando r oltre $\lfloor \lg n \rfloor$, il termine 2^r nel numeratore cresce più rapidamente del termine r nel denominatore; quindi,

Acquistato da Michele Mostarda (2022/0007212) Numero Ordine Milano: 2145/2021/Q1 Copyright © 2022 McGraw-Hill Education (Italy)

aumentando r oltre $\lfloor \lg n \rfloor$, si ottiene un tempo di esecuzione pari a $\Omega(bn/\lg n)$. Se invece diminuiamo r sotto $\lfloor \lg n \rfloor$, il termine b/r cresce e il termine $n + 2^r$ resta a $\Theta(n)$.

Radix sort è preferibile a un algoritmo di ordinamento basato sui confronti, come quicksort? Se $b = O(\lg n)$, come spesso accade, e scegliamo $r \approx \lg n$, allora il tempo di esecuzione di radix sort è $\Theta(n)$, che sembra migliore di $\Theta(n \lg n)$, il tempo di esecuzione nel caso medio di quicksort. Notiamo però che i fattori costanti nascosti nella notazione Θ sono differenti. Sebbene radix sort richieda meno passaggi di quicksort sulle n chiavi, tuttavia ogni passaggio di radix sort potrebbe richiedere un tempo significativamente più lungo. La scelta dell'algoritmo di ordinamento ottimale dipende dalle caratteristiche delle implementazioni, della macchina (per esempio, quicksort spesso usa le cache hardware in modo più efficiente di radix sort) e dei dati di input. Inoltre, la versione di radix sort che usa counting sort come ordinamento stabile intermedio non effettua l'ordinamento sul posto, come invece fanno molti degli ordinamenti per confronti con tempo $\Theta(n \lg n)$. Quindi, se lo spazio nella memoria principale è limitato, potrebbe essere preferibile un algoritmo di ordinamento sul posto come quicksort.

Esercizi

8.3-1

Utilizzando la Figura 8.3 come modello, illustrate l'esecuzione di RADIX-SORT con la seguente lista di parole inglesi: COW, DOG, SEA, RUG, ROW, MOB, BOX, TAB, BAR, EAR, TAR, DIG, BIG, TEA, NOW, FOX.

8.3-2

Quali dei seguenti algoritmi di ordinamento sono stabili: insertion sort, merge sort, heapsort e quicksort? Trovate un semplice metodo per rendere stabile qualsiasi algoritmo di ordinamento. Quanto tempo e spazio in più richiede il vostro metodo?

8.3-3

Utilizzate l'induzione per dimostrare che radix sort funziona correttamente. In quale punto la vostra dimostrazione richiede l'ipotesi che l'ordinamento intermedio sia stabile?

8.3-4

Dimostrate come ordinare n numeri interi compresi nell'intervallo da 0 a $n^3 - 1$ nel tempo $O(n)$.

8.3-5 ⋆

Nel primo algoritmo di ordinamento delle schede presentato in questo paragrafo, esattamente quanti passaggi sono richiesti per ordinare i numeri di d cifre nel caso peggiore? Quante pile di schede dovrebbe gestire un operatore nel caso peggiore?

8.4 Bucket sort

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Bucket sort assume che l'input sia estratto da una distribuzione uniforme e ha tempo di esecuzione medio $\Theta(n)$. Come counting sort, bucket sort è veloce perché fa un'ipotesi sull'input. Mentre counting sort suppone che l'input sia formato da interi in un piccolo intervallo, bucket sort suppone che l'input sia generato da un processo casuale che distribuisce gli elementi uniformemente e indipendentemente nell'intervallo $[0, 1)$ (la distribuzione uniforme è definita nel Paragrafo C.2).

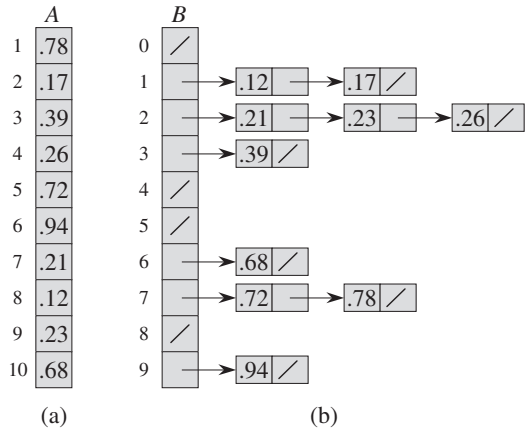


Figura 8.4
Il funzionamento di BUCKET-SORT con $n = 10$. **(a)** L'array di input $A[1 \,..\, 10]$. **(b)** L'array $B[0 \,..\, 9]$ delle liste ordinate (bucket) dopo l'esecuzione della riga 8 dell'algoritmo. Il bucket i contiene i valori nell'intervallo semiaperto $[i/10, (i + 1)/10)$. L'output ordinato è formato dalla concatenazione ordinata delle liste: $B[0], B[1], \ldots, B[9]$.

Bucket sort divide l'intervallo $[0, 1)$ in n sottointervalli della stessa dimensione, detti ***bucket***, e poi distribuisce gli n numeri di input nei bucket. Poiché gli input sono uniformemente e indipendentemente distribuiti nell'intervallo $[0, 1)$, non ci aspettiamo che molti numeri vadano a finire nello stesso bucket. Per produrre l'output, semplicemente ordiniamo i numeri in ogni bucket e poi scorriamo ordinatamente i bucket, elencando gli elementi in ciascuno di essi.

Il nostro codice per bucket sort suppone che l'input sia un array A di n elementi e che ogni elemento $A[i]$ dell'array soddisfi la relazione $0 \leq A[i] < 1$. Il codice richiede un array ausiliario $B[0 \,..\, n - 1]$ di liste concatenate (bucket) e suppone che ci sia un meccanismo per operare su tali liste (il Paragrafo 10.2 descrive come implementare le operazioni elementari con le liste concatenate).

```
BUCKET-SORT(A)
1  sia B[0 .. n − 1] un nuovo array
2  n = A.length
3  for i = 0 to n − 1
4       crea la lista vuota B[i]
5  for i = 1 to n
6       inserisce A[i] nella lista B[⌊n A[i]⌋]
7  for i = 0 to n − 1
8       ordina la lista B[i] con insertion sort
9  concatena ordinatamente le liste B[0], B[1], . . . , B[n − 1]
```

La Figura 8.4 illustra l'esecuzione di bucket sort su un array di input di 10 numeri.

Per dimostrare che questo algoritmo funziona correttamente, consideriamo due elementi $A[i]$ e $A[j]$. Supponiamo, senza perdere di generalità, che $A[i] \leq A[j]$. Poiché $\lfloor n A[i] \rfloor \leq \lfloor n A[j] \rfloor$, l'elemento $A[i]$ viene posto nello stesso bucket di $A[j]$ o in un bucket con un indice minore. Se $A[i]$ e $A[j]$ vengono posti nello stesso bucket, allora le righe 7–8 del ciclo **for** li inseriscono nell'ordine appropriato. Se $A[i]$ e $A[j]$ vengono posti in bucket differenti, allora la riga 9 li inserisce nell'ordine appropriato. Dunque, bucket sort funziona correttamente.

Acquistato da Michele Mo[illegible] - 2022-[illegible] - Proprietà letteraria riservata. © 2023 The McGraw-Hill Education (Italy)

Per analizzare il tempo di esecuzione, notiamo che tutte le righe, tranne la 8, richiedono un tempo $O(n)$ nel caso peggiore. Resta da valutare il tempo totale richiesto dalle n chiamate di insertion sort nella riga 8.

Per analizzare il costo delle chiamate di insertion sort, indichiamo con n_i la variabile casuale che rappresenta il numero degli elementi che vengono inseri-

ti nel bucket $B[i]$. Poiché insertion sort viene eseguito in un tempo quadratico (Paragrafo 2.2), il tempo di esecuzione di bucket sort è

$$T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)$$

Analizziamo adesso il tempo di esecuzione nel caso medio di bucket sort, calcolando il valore atteso del tempo di esecuzione, dove il valore atteso viene preso rispetto alla distribuzione dell'input. Prendendo i valori attesi da entrambi i lati e applicando la linearità del valore atteso, si ha

$$\begin{aligned}
\mathrm{E}[T(n)] &= \mathrm{E}\left[\Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)\right] \\
&= \Theta(n) + \sum_{i=0}^{n-1} \mathrm{E}\left[O(n_i^2)\right] \quad \text{(per la linearità del valore atteso)} \\
&= \Theta(n) + \sum_{i=0}^{n-1} O\left(\mathrm{E}\left[n_i^2\right]\right) \quad \text{(per l'equazione (C.22))} \qquad (8.1)
\end{aligned}$$

Noi affermiamo che

$$\mathrm{E}\left[n_i^2\right] = 2 - 1/n \qquad (8.2)$$

per $i = 0, 1, \ldots, n-1$. Non sorprende il fatto che ogni bucket i abbia lo stesso valore $\mathrm{E}[n_i^2]$, in quanto ogni valore nell'array di input A ha la stessa probabilità di ricadere in qualsiasi bucket. Per dimostrare l'equazione (8.2), definiamo le variabili casuali indicatrici

$X_{ij} = \mathrm{I}\{A[j] \text{ ricade nel bucket } i\}$

per $i = 0, 1, \ldots, n-1$ e $j = 1, 2, \ldots, n$. Quindi

$$n_i = \sum_{j=1}^{n} X_{ij}$$

Per calcolare $\mathrm{E}[n_i^2]$, espandiamo il quadrato e raggruppiamo i termini:

$$\begin{aligned}
\mathrm{E}\left[n_i^2\right] &= \mathrm{E}\left[\left(\sum_{j=1}^{n} X_{ij}\right)^2\right] \\
&= \mathrm{E}\left[\sum_{j=1}^{n}\sum_{k=1}^{n} X_{ij} X_{ik}\right] \\
&= \mathrm{E}\left[\sum_{j=1}^{n} X_{ij}^2 + \sum_{1\le j\le n}\ \sum_{\substack{1\le k\le n \\ k\ne j}} X_{ij} X_{ik}\right]
\end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016 220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$= \sum_{j=1}^{n} \mathrm{E}\left[X_{ij}^2\right] + \sum_{1\le j\le n}\ \sum_{\substack{1\le k\le n \\ k\ne j}} \mathrm{E}\left[X_{ij} X_{ik}\right] \qquad (8.3)$$

L'ultima riga deriva dalla linearità del valore atteso. Calcoliamo le due sommatorie separatamente. La variabile casuale indicatrice X_{ij} vale 1 con probabilità $1/n$ e 0 negli altri casi, quindi

$$\begin{aligned}\mathrm{E}\left[X_{ij}^2\right] &= 1 \cdot \frac{1}{n} + 0 \cdot \left(1 - \frac{1}{n}\right) \\ &= \frac{1}{n}\end{aligned}$$

Quando $k \neq j$, le variabili X_{ij} e X_{ik} sono indipendenti, quindi

$$\begin{aligned}\mathrm{E}\left[X_{ij} X_{ik}\right] &= \mathrm{E}\left[X_{ij}\right] \mathrm{E}\left[X_{ik}\right] \\ &= \frac{1}{n} \cdot \frac{1}{n} \\ &= \frac{1}{n^2}\end{aligned}$$

Sostituendo questi due valori attesi nell'equazione (8.3), si ha

$$\begin{aligned}\mathrm{E}\left[n_i^2\right] &= \sum_{j=1}^{n} \frac{1}{n} + \sum_{1 \le j \le n} \sum_{\substack{1 \le k \le n \\ k \neq j}} \frac{1}{n^2} \\ &= n \cdot \frac{1}{n} + n(n-1) \cdot \frac{1}{n^2} \\ &= 1 + \frac{n-1}{n} \\ &= 2 - \frac{1}{n}\end{aligned}$$

che dimostra l'equazione (8.2).

Utilizzando questo valore atteso nell'equazione (8.1), concludiamo che il tempo di esecuzione nel caso medio di bucket sort è $\Theta(n) + n \cdot O(2 - 1/n) = \Theta(n)$.

Bucket sort può richiedere tempo di esecuzione lineare anche con input che non provengono da una distribuzione uniforme. L'equazione (8.1) ci dice che bucket sort viene eseguito in tempo lineare purché l'input sia tale per cui la somma dei quadrati delle dimensioni dei bucket risulti lineare nel numero totale di elementi.

Esercizi

8.4-1

Utilizzando la Figura 8.4 come modello, illustrate l'esecuzione di BUCKET-SORT sull'array $A = \langle .79, .13, .16, .64, .39, .20, .89, .53, .71, .42 \rangle$.

8.4-2

Spiegate perché il tempo di esecuzione nel caso peggiore dell'algoritmo bucket sort è $\Theta(n^2)$. Quale semplice modifica dell'algoritmo preserva il suo tempo di esecuzione lineare nel caso medio e rende il suo tempo di esecuzione nel caso peggiore pari a $O(n \lg n)$?

8.4-3

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Sia X una variabile casuale che è uguale al numero di teste in due lanci di una moneta non truccata. Quanto vale $\mathrm{E}\left[X^2\right]$? Quanto vale $\mathrm{E}^2\left[X\right]$?

8.4-4 ⋆

Dati n punti nel cerchio unitario, $p_i = (x_i, y_i)$, tali che $0 < x_i^2 + y_i^2 \le 1$ per $i = 1, 2, \ldots, n$. Supponiamo che i punti siano uniformemente distribuiti, nel senso che la probabilità di trovare un punto in qualsiasi regione del cerchio sia proporzionale all'area di quella regione. Scrivete un algoritmo, con un tempo di

esecuzione nel caso medio pari a $\Theta(n)$, per ordinare gli n punti in funzione delle loro distanze $d_i = \sqrt{x_i^2 + y_i^2}$ dall'origine (*suggerimento:* scegliete le dimensioni dei bucket di BUCKET-SORT in modo da riflettere la distribuzione uniforme dei punti nel cerchio unitario).

8.4-5 ⋆

La ***distribuzione di probabilità*** $P(x)$ di una variabile casuale X è definita da $P(x) = \Pr\{X \leq x\}$. Supponete che una lista di n variabili casuali $X_1, X_2, \ldots, X_n$ venga estratta da una distribuzione di probabilità continua P che è calcolabile nel tempo $O(1)$. Scrivete un algoritmo per ordinare questi numeri in tempo lineare nel caso medio.

Problemi

8-1 Limiti inferiori probabilistici nell'ordinamento per confronti

In questo problema dimostriamo un limite inferiore probabilistico $\Omega(n \lg n)$ sul tempo di esecuzione di qualsiasi ordinamento per confronti deterministico o randomizzato con n elementi distinti di input. Iniziamo a esaminare un ordinamento per confronti deterministico A con albero di decisione T_A. Supponiamo che ogni permutazione degli input di A sia ugualmente probabile.

a. Supponiamo che ogni foglia di T_A sia etichettata con la probabilità di essere raggiunta, dato un input casuale. Dimostrate che esattamente $n!$ foglie sono etichettate con $1/n!$ e le restanti sono etichettate con 0.

b. Sia $D(T)$ la lunghezza del cammino esterno di un albero di decisione T; cioè $D(T)$ è la somma delle profondità di tutte le foglie di T. Sia T un albero di decisione con $k > 1$ foglie; siano LT e RT i sottoalberi sinistro e destro di T. Dimostrate che $D(T) = D(LT) + D(RT) + k$.

c. Sia $d(k)$ il valore minimo di $D(T)$ in tutti gli alberi di decisione T con $k > 1$ foglie. Dimostrate che $d(k) = \min_{1 \leq i \leq k-1} \{d(i) + d(k-i) + k\}$ (*suggerimento:* considerate un albero di decisione T con k foglie che raggiunge il minimo. Indicate con i_0 il numero di foglie in LT e con $k - i_0$ il numero di foglie in RT).

d. Dimostrate che, per un dato valore di $k > 1$ e i nell'intervallo $1 \leq i \leq k-1$, la funzione $i \lg i + (k-i) \lg(k-i)$ è minima per $i = k/2$. Concludete che $d(k) = \Omega(k \lg k)$.

e. Dimostrate che $D(T_A) = \Omega(n! \lg(n!))$ e concludete che il tempo nel caso medio per ordinare n elementi è $\Omega(n \lg n)$.

Adesso considerate un ordinamento per confronti *randomizzato* B. Possiamo estendere il modello dell'albero di decisione per gestire la randomizzazione incorporando due tipi di nodi: il nodo di confronto ordinario e il nodo di "randomizzazione". Un nodo di randomizzazione modella una scelta casuale della forma RANDOM$(1, r)$ fatta dall'algoritmo B; il nodo ha r figli, ciascuno dei quali ha la stessa probabilità di essere scelto durante un'esecuzione dell'algoritmo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 19503016-220707. © Copyright © 2022 McGraw-Hill Education (Italy)

f. Dimostrate che, per qualsiasi ordinamento per confronti randomizzato B, esiste un ordinamento per confronti deterministico A che, in media, non effettua più confronti di B.

8-2 Ordinamento sul posto in tempo lineare

Supponete di avere un array di n record di dati da ordinare e che la chiave di ogni record abbia il valore 0 o 1. Un algoritmo che ordina questo insieme di record potrebbe possedere alcune delle tre seguenti caratteristiche desiderabili:

1. L'algoritmo viene eseguito nel tempo $O(n)$.
2. L'algoritmo è stabile.
3. L'algoritmo ordina sul posto, utilizzando non più di una quantità costante di memoria oltre all'array originale.

a. Trovate un algoritmo che soddisfa i criteri 1 e 2.

b. Trovate un algoritmo che soddisfa i criteri 1 e 3.

c. Trovate un algoritmo che soddisfa i criteri 2 e 3.

d. Uno dei vostri algoritmi specificati nei punti (a)–(c) può essere utilizzato per ordinare n record con chiavi di b bit mediante radix sort nel tempo $O(bn)$? Spiegate come o perché no.

e. Supponete che gli n record abbiano le chiavi nell'intervallo da 1 a k. Spiegate come modificare counting sort in modo che i record possano essere ordinati sul posto nel tempo $O(n + k)$. Potete utilizzare una quantità $O(k)$ di memoria oltre all'array di input. Il vostro algoritmo è stabile? (*Suggerimento:* Come fareste con $k = 3$?)

8-3 Ordinare elementi di lunghezza variabile

a. Supponete di avere un array di interi, dove interi differenti possono avere numeri di cifre differenti, ma il numero totale di cifre di *tutti* gli interi dell'array è n. Spiegate come ordinare l'array nel tempo $O(n)$.

b. Supponete di avere un array di stringhe, dove stringhe differenti possono avere numeri differenti di caratteri, ma il numero totale di caratteri di tutte le stringhe è n. Spiegate come ordinare le stringhe nel tempo $O(n)$ (notate che l'ordine richiesto qui è quello alfabetico standard; per esempio, `a` $<$ `ab` $<$ `b`).

8-4 Il problema delle brocche d'acqua

Supponete di avere delle brocche per l'acqua, n rosse e n blu, tutte di forma e dimensione diverse. Tutte le brocche rosse contengono quantità d'acqua differenti, come pure le brocche blu. Inoltre, per ogni brocca rossa c'è una brocca blu che contiene la stessa quantità d'acqua e viceversa.

È vostro compito accoppiare una brocca rossa con una blu in modo che ogni coppia contenga la stessa quantità d'acqua. Per farlo, potete eseguire la seguente operazione: scegliere una coppia di brocche, di cui una rossa e l'altra blu, riempire la brocca rossa di acqua e, poi, versare l'acqua nella brocca blu. Questa operazione vi dirà se la brocca rossa (o quella blu) può contenere più acqua o se le due brocche hanno lo stesso volume. Supponete che questo confronto richieda un'unità di tempo. Il vostro obiettivo è trovare un algoritmo che effettua il numero minimo di confronti per accoppiare le brocche. Ricordatevi che non potete confrontare direttamente due brocche rosse o due brocche blu.

Acquistato da Michele Mirante su Webster il 2022-07-07 13:12 Numero Ordine Libreria 1195847216210 [...] Copyright © 2022, McGraw-Hill Education (Italy)

a. Descrivete un algoritmo deterministico che usa $\Theta(n^2)$ confronti per accoppiare le brocche d'acqua.

b. Dimostrate il limite inferiore $\Omega(n \lg n)$ per il numero di confronti che deve effettuare un algoritmo per risolvere questo problema.

c. Trovate un algoritmo randomizzato che abbia un numero atteso di confronti pari a $O(n \lg n)$; dimostrate che questo limite è corretto. Qual è il numero di confronti nel caso peggiore per il vostro algoritmo?

8-5 Ordinamento in media

Supponiamo che, anziché ordinare un array, sia richiesto semplicemente che gli elementi dell'array crescano in media. Più precisamente, chiamiamo ***k-ordinato*** un array A di n elementi se, per ogni $i = 1, 2, \ldots, n-k$, vale la seguente relazione:

$$\frac{\sum_{j=i}^{i+k-1} A[j]}{k} \leq \frac{\sum_{j=i+1}^{i+k} A[j]}{k}$$

a. Che cosa significa che un array è 1-ordinato?

b. Trovate una permutazione dei numeri $1, 2, \ldots, 10$ che sia 2-ordinata, ma non ordinata.

c. Dimostrate che un array di n elementi è k-ordinato se e soltanto se $A[i] \leq A[i+k]$ per ogni $i = 1, 2, \ldots, n-k$.

d. Trovate un algoritmo che trasformi in k-ordinato un array di n elementi nel tempo $O(n \lg(n/k))$.

È anche possibile dimostrare un limite inferiore sul tempo per produrre un array k-ordinato, quando k è una costante.

e. Dimostrate che un array k-ordinato di lunghezza n può essere ordinato nel tempo $O(n \lg k)$ (*suggerimento:* usate la soluzione dell'Esercizio 6.5-9).

f. Dimostrate che quando k è una costante, occorre un tempo $\Omega(n \lg n)$ per trasformare in k-ordinato un array di n elementi (*suggerimento:* usate la soluzione del precedente punto insieme con il limite inferiore sugli ordinamenti per confronti).

8-6 Limite inferiore sulla fusione di liste ordinate

Il problema di fondere due liste ordinate si presenta frequentemente. Nel Paragrafo 2.3.1 abbiamo visto una procedura per fare questo nella forma della subroutine MERGE. In questo problema, dimostreremo un limite inferiore di $2n - 1$ sul numero di confronti da effettuare nel caso peggiore per fondere due liste ordinate, ciascuna contenente n elementi.

Innanzi tutto, dimostriamo un limite inferiore di $2n - o(n)$ confronti utilizzando un albero di decisione.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

a. Dimostrate che, dati $2n$ numeri, ci sono $\binom{2n}{n}$ possibili modi di dividerli in due liste ordinate, ciascuna con n numeri.

b. Utilizzando un albero di decisione, dimostrate che qualsiasi algoritmo che fonde correttamente due liste ordinate usa almeno $2n - o(n)$ confronti.

Adesso dimostriamo un limite leggermente più stretto $2n - 1$.

c. Dimostrate che, se due elementi sono consecutivi nell'ordinamento e provengono da liste diverse, allora devono essere confrontati.

d. Utilizzate la risposta del precedente punto per dimostrare un limite inferiore di $2n - 1$ confronti per fondere due liste ordinate.

8-7 Il lemma dell'ordinamento 0-1 e l'algoritmo columnsort

Un'operazione ***confronta-scambia*** su due elementi di array $A[i]$ e $A[j]$, con $i < j$, ha la forma

```
COMPARE-EXCHANGE(A, i, j)
1  if A[i] > A[j]
2       scambia A[i] con A[j]
```

Dopo l'operazione confronta-scambia, sappiamo che $A[i] \leq A[j]$.

Un ***algoritmo confronta-scambia senza memoria*** opera soltanto tramite una sequenza di operazioni confronta-scambia predefinite. Gli indici delle posizioni confrontate nella sequenza devono essere determinati in anticipo e, sebbene possano dipendere dal numero di elementi da ordinare, tuttavia non possono dipendere dai valori da ordinare né dal risultato di una precedente operazione confronta-scambia. Per esempio, questo è insertion sort espresso nella forma di algoritmo confronta-scambia senza memoria:

```
INSERTION-SORT(A)
1  for j = 2 to A.length
2       for i = j − 1 downto 1
3            COMPARE-EXCHANGE(A, i, i + 1)
```

Il ***lemma dell'ordinamento 0-1*** fornisce un metodo efficace per dimostrare che un algoritmo confronta-scambia senza memoria produce un risultato ordinato. Esso stabilisce che se un algoritmo confronta-scambia senza memoria ordina correttamente tutte le sequenze di input formate soltanto da 0 e 1, allora esso ordina correttamente tutti gli input che contengono valori arbitrari.

Dimostrerete il lemma dell'ordinamento 0-1 dimostrando il suo contrapposto: se un algoritmo confronta-scambia senza memoria non riesce a ordinare un input di valori arbitrari, allora non riuscirà a ordinare un input di 0-1. Supponete che un algoritmo confronta-scambia senza memoria X non riesca a ordinare correttamente l'array $A[1\,..\,n]$. Indicate con $A[p]$ il valore più piccolo di A che l'algoritmo X inserisce nella posizione sbagliata, e con $A[q]$ il valore che l'algoritmo X sposta nella posizione in cui dovrebbe andare $A[p]$. Definite un array $B[1\,..\,n]$ di 0 e 1 in questo modo:

$$B[i] = \begin{cases} 0 & \text{se } A[i] \leq A[p] \\ 1 & \text{se } A[i] > A[p] \end{cases}$$

Acquistato da Michele Michele su Webster il 2022-07-07 22:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

a. Dimostrate che $A[q] > A[p]$, cosicché $B[p] = 0$ e $B[q] = 1$.

b. Per completare la dimostrazione del lemma dell'ordinamento 0-1, dimostrate che l'algoritmo X non riesce a ordinare correttamente l'array B.

Ora possiamo usare il lemma dell'ordinamento 0-1 per dimostrare che un particolare algoritmo di ordinamento funziona correttamente. L'algoritmo, ***columnsort***, opera su un array rettangolare di n elementi. L'array ha r righe e s colonne (quindi $n = rs$) ed è soggetto alle seguenti limitazioni:

- r deve essere pari
- s deve essere un divisore di r
- $r \geq 2s^2$

Quando columnsort termina, l'array è ***ordinato per colonne***: leggendo le colonne dall'alto verso il basso, e da sinistra a destra, gli elementi crescono monotonicamente.

L'algoritmo columnsort opera in otto passi, indipendentemente dal valore di n. I passi dispari sono uguali: ordina singolarmente ciascuna colonna. Ogni passo pari è una permutazione prefissata. I passi sono i seguenti:

1. Ordinare le singole colonne.
2. Trasporre l'array e modificarne la forma per riportarlo a r righe e s colonne. In altre parole, ripartire gli elementi della prima colonna a sinistra ordinatamente nelle prime r/s righe; ripartire gli elementi della seconda colonna ordinatamente nelle successive r/s righe; e così via.
3. Ordinare le singole colonne.
4. Effettuare l'inverso della permutazione del passo 2.
5. Ordinare le singole colonne.
6. Spostare la metà superiore di ciascuna colonna nella metà inferiore della stessa colonna, e la metà inferiore di ciascuna colonna nella metà superiore della successiva colonna a destra. Lasciare vuota la metà superiore della prima colonna a sinistra. Spostare la metà inferiore dell'ultima colonna nella metà superiore di una nuova colonna a destra, lasciando vuota la metà inferiore di questa nuova colonna.
7. Ordinare le singole colonne.
8. Effettuare l'inverso della permutazione del passo 6.

La Figura 8.5 mostra un esempio dei passi di columnsort con $r = 6$ e $s = 3$. (Sebbene questo esempio non soddisfi il requisito che $r \geq 2s^2$, tuttavia si dà il caso che funzioni ugualmente.)

c. Dimostrate che è possibile trattare columnsort come un algoritmo confronta-scambia senza memoria, anche se non si sa quale metodo di ordinamento utilizzano i passi dispari.

Sebbene possa sembrare difficile credere che columnsort ordini effettivamente l'array, si può usare il lemma di ordinamento 0-1 per dimostrarlo. Questo lemma si può applicare perché columnsort può essere considerato un algoritmo confronta-scambia senza memoria. Un paio di definizioni vi aiuteranno ad applicare il lemma di ordinamento 0-1. Diciamo che un'area di un array è ***pulita*** se contiene tutti 0 o tutti 1; altrimenti, l'area contiene sia 0 che 1, e in questo caso è detta ***sporca***. Da qui in poi, supponiamo che l'array di input contenga soltanto 0 e 1, e che possiamo trattarlo come un array con r righe e s colonne.

Acquistato da Michele Michele su Webster il 2022-07-27 13:10:07 Non è consentito il riutilizzo commerciale o la distribuzione in rete

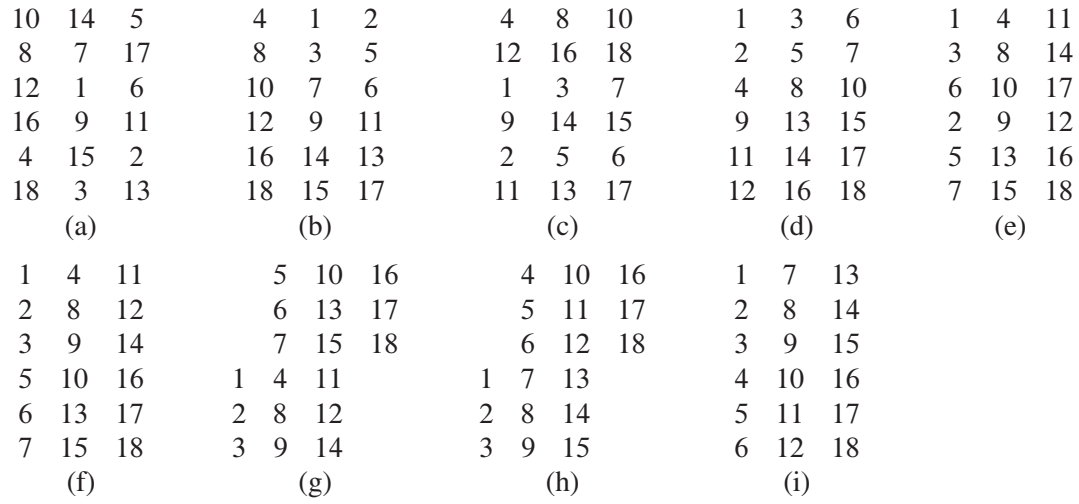


Figura 8.5 I passi di columnsort. **(a)** L'array di input con 6 righe e 3 colonne. **(b)** Dopo avere ordinato ciascuna colonna nel passo 1. **(c)** Dopo la trasposizione e la modifica della forma nel passo 2. **(d)** Dopo l'ordinamento delle singole colonne nel passo 3. **(e)** Dopo il passo 4, che inverte la permutazione del passo 2. **(f)** Dopo l'ordinamento delle singole colonne nel passo 5. **(g)** Dopo lo scorrimento di metà colonna nel passo 6. **(h)** Dopo l'ordinamento delle singole colonne nel passo 7. **(i)** Dopo il passo 8, che inverte la permutazione del passo 6. L'array adesso è ordinato per colonne.

d. Dimostrate che dopo i passi 1–3, l'array è formato da alcune righe pulite di 0 in cima, da alcune righe pulite di 1 in fondo, e al massimo da s righe sporche nel mezzo.

e. Dimostrate che dopo il passo 4, l'array, letto per colonne, inizia con un'area pulita di 0, termina con un'area pulita di 1, e ha un'area sporca al massimo di s^2 elementi nel mezzo.

f. Dimostrate che i passi 5–8 producono un output completamente ordinato di 0-1. Concludete che columnsort ordina correttamente tutti gli input che contengono valori arbitrari.

g. Adesso supponete che s non sia un divisore di r. Dimostrate che dopo i passi 1–3, l'array è formato da alcune righe pulite di 0 in cima, alcune righe pulite di 1 in fondo, e al massimo da $2s - 1$ righe sporche nel mezzo. Quanto deve essere grande r, rispetto a s, affinché columnsort ordini correttamente l'array quando s non è un divisore di r?

h. Suggerite una semplice modifica del passo 1 che consenta di mantenere il requisito che $r \geq 2s^2$, anche quando s non è un divisore di r, e dimostrate che con la vostra modifica, columnsort esegue correttamente l'ordinamento.

Acquistato da Michele Michelotti Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Note

Il modello dell'albero di decisione per studiare gli ordinamenti per confronti è stato introdotto da Ford e Johnson [111]. Knuth [212] ha trattato molte varianti del problema dell'ordinamento, incluso il limite inferiore teorico sulla complessità dell'ordinamento descritto in questo capitolo. I limiti inferiori per gli ordinamenti che usano generalizzazioni del modello dell'albero di decisione sono stati esaurientemente studiati da Ben-Or [39].

continua

Knuth attribuisce a H. H. Seward l'invenzione dell'algoritmo counting sort nel 1954 e anche l'idea di combinare counting sort con radix sort. Il processo radix sort che inizia con la cifra meno significativa sembra essere un algoritmo popolare ampiamente utilizzato dagli operatori di macchine meccaniche per ordinare le schede. Secondo Knuth, la prima pubblicazione su questo metodo è un documento del 1929 di L. J. Comrie che descrive un dispositivo per perforare le schede. Bucket sort è stato utilizzato fin dal 1956, quando l'idea di base fu proposta da E. J. Isaac e R. C. Singleton [189].

Munro e Raman [264] hanno creato un algoritmo di ordinamento stabile che richiede $O(n^{1+\epsilon})$ confronti nel caso peggiore, dove $0 < \epsilon \leq 1$ è una costante qualsiasi. Sebbene qualsiasi algoritmo con tempo $O(n \lg n)$ effettui meno confronti, tuttavia l'algoritmo di Munro e Raman sposta i dati soltanto $O(n)$ volte e opera sul posto.

Il caso di ordinare n interi di b bit nel tempo $o(n \lg n)$ è stato esaminato da molti ricercatori. Sono stati ottenuti diversi risultati positivi, ciascuno con ipotesi leggermente differenti sul modello di calcolo e sulle limitazioni imposte all'algoritmo. Tutti i risultati presuppongono che la memoria del calcolatore sia divisa in parole di b bit indirizzabili. Fredman e Willard [116] hanno introdotto la struttura dati detta *albero di fusione* che hanno utilizzato per ordinare n interi nel tempo $O(n \lg n / \lg \lg n)$. Successivamente, questo limite è stato migliorato a $O(n \sqrt{\lg n})$ da Andersson [16]. Questi algoritmi richiedono l'uso della moltiplicazione e varie costanti precalcolate. Andersson, Hagerup, Nilsson e Raman [17] hanno dimostrato come ordinare n interi nel tempo $O(n \lg \lg n)$ senza utilizzare la moltiplicazione, ma il loro metodo richiede uno spazio in memoria che potrebbe essere illimitato in termini di n. Utilizzando l'hashing moltiplicativo, è possibile ridurre lo spazio in memoria a $O(n)$, ma il limite $O(n \lg \lg n)$ nel caso peggiore sul tempo di esecuzione diventa un limite sul tempo atteso. Generalizzando l'albero di ricerca esponenziale di Andersson [16], Thorup [336] ha creato un algoritmo di ordinamento con tempo $O(n(\lg \lg n)^2)$ che non usa la moltiplicazione né la randomizzazione, e richiede spazio lineare. Combinando queste tecniche con alcune nuove idee, Han [159] ha migliorato il limite dell'ordinamento al tempo $O(n \lg \lg n \lg \lg \lg n)$. Sebbene questi algoritmi siano importanti scoperte teoriche, tuttavia sono tutti molto complicati e, attualmente, sembra improbabile che possano competere con gli algoritmi di ordinamento che vengono utilizzati nella programmazione pratica.

L'algoritmo columnsort nel Problema 8-7 è di Leighton [228].

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Mediane e statistiche d'ordine 9

L'i-esima ***statistica d'ordine*** di un insieme di n elementi è l'i-esimo elemento più piccolo. Per esempio, il ***minimo*** di un insieme di elementi è la prima statistica d'ordine ($i = 1$) e il ***massimo*** è l'n-esima statistica d'ordine ($i = n$). Informalmente, la ***mediana*** è il "punto di mezzo" dell'insieme. Se n è dispari, la mediana è unica, perché si verifica in corrispondenza di $i = (n + 1)/2$. Se n è pari, ci sono due mediane: una in corrispondenza di $i = n/2$ e l'altra in corrispondenza di $i = n/2 + 1$. Quindi, indipendentemente dalla parità di n, le mediane si hanno per $i = \lfloor (n + 1)/2 \rfloor$ (la ***mediana inferiore***) e per $i = \lceil (n + 1)/2 \rceil$ (la ***mediana superiore***). Per semplicità, in questo testo utilizzeremo sempre il termine "la mediana" per fare riferimento alla mediana inferiore.

Questo capitolo tratta il problema di selezionare l'i-esima statistica d'ordine da un insieme di n numeri distinti. Supponiamo, per comodità, che l'insieme contenga numeri distinti, sebbene tutto ciò che faremo potrà essere esteso al caso in cui l'insieme contenga valori ripetuti. Il ***problema della selezione*** può essere definito formalmente in questo modo:

Input: un insieme A di n numeri (distinti) e un intero i, con $1 \leq i \leq n$.

Output: l'elemento $x \in A$ che è maggiore esattamente di altri $i - 1$ elementi di A.

Il problema della selezione può essere risolto nel tempo $O(n \lg n)$, perché possiamo ordinare i numeri utilizzando heapsort o merge sort e, poi, indirizzare semplicemente l'i-esimo elemento nell'array di output. In questo capitolo vedremo degli algoritmi più veloci.

Nel Paragrafo 9.1 esamineremo il problema della selezione del minimo e del massimo in un insieme di elementi. Più interessante è il problema generale della selezione, che sarà analizzato nei due paragrafi successivi. Il Paragrafo 9.2 presenta un algoritmo randomizzato pratico che raggiunge un limite $O(n)$ sul tempo di esecuzione atteso, nell'ipotesi che gli elementi siano distinti. Il Paragrafo 9.3 descrive un algoritmo di interesse principalmente teorico che raggiunge un tempo di esecuzione $O(n)$ nel caso peggiore.

9.1 Minimo e massimo

Acquistato da Michele Mioduski Webster il 2022-07-07 22:12 Numero Ordine Libraria 1805010162307 Copyright © 2022 McGraw-Hill Education (Italy)

Quanti confronti sono necessari per determinare il minimo di un insieme di n elementi? Possiamo facilmente ottenere un limite superiore di $n - 1$ confronti: esaminiamo, uno alla volta, gli elementi dell'insieme e teniamo traccia dell'ultimo elemento più piccolo trovato. Nella seguente procedura supponiamo che l'insieme si trovi nell'array A, dove $A.length = n$.

```
MINIMUM(A)
1  min = A[1]
2  for i = 2 to A.length
3       if min > A[i]
4            min = A[i]
5  return min
```

Ovviamente, anche il massimo può essere trovato con $n - 1$ confronti.

È questo quanto di meglio possiamo fare? Sì, perché possiamo ottenere un limite inferiore di $n - 1$ confronti per determinare il minimo. Immaginate qualsiasi algoritmo che determina il minimo come a un torneo fra gli elementi. Ogni confronto è una partita del torneo dove vince il più piccolo fra i due elementi. Osservando che ogni elemento, tranne il vincitore, deve perdere almeno una partita, possiamo concludere che sono necessari $n - 1$ confronti per determinare il minimo. Dunque l'algoritmo MINIMUM è ottimale rispetto al numero di confronti effettuati.

Minimo e massimo simultanei

In alcune applicazioni occorre trovare il minimo e il massimo di un insieme di n elementi. Per esempio, un programma di grafica potrebbe richiedere di modificare la scala di un insieme di punti (x, y) per adattarli a uno schermo rettangolare o a un'altra unità grafica di output. Per farlo, il programma deve prima determinare il minimo e il massimo valore.

Non è difficile ideare un algoritmo, asintoticamente ottimale, in grado di trovare il minimo e il massimo di n elementi con $\Theta(n)$ confronti. Basta trovare il minimo e il massimo separatamente, effettuando $n - 1$ confronti per ciascuno di essi, per un totale di $2n - 2$ confronti. In realtà, possiamo trovare minimo e massimo con al massimo $3\lfloor n/2 \rfloor$ confronti. Possiamo farlo mantenendo aggiornati il minimo e il massimo trovati mentre scorriamo l'array. Anziché confrontare ogni elemento di input con i valori correnti del minimo e del massimo, a un costo di 2 confronti per elemento, elaboriamo gli elementi in coppia. Confrontiamo prima due elementi di input *l'uno con l'altro* e poi il più piccolo dei due con il minimo corrente e il più grande dei due con il massimo corrente, con un costo di 3 confronti per ogni 2 elementi.

L'impostazione dei valori iniziali del minimo e del massimo dipende dal fatto che n sia pari o dispari. Se n è dispari, assegniamo al minimo e al massimo il valore del primo elemento e poi elaboriamo i restanti elementi in coppia. Se n è pari, effettuiamo un confronto fra i primi 2 elementi per determinare i valori iniziali del minimo e del massimo; poi elaboriamo i restanti elementi in coppia, come nel caso di n dispari.

Analizziamo il numero totale di confronti. Se n è dispari, svolgiamo $3\lfloor n/2 \rfloor$ confronti. Se n è pari, svolgiamo un confronto iniziale seguito da $3(n - 2)/2$ confronti, per un totale di $3n/2 - 3$. Quindi, in entrambi i casi, il numero totale di confronti è al massimo $3\lfloor n/2 \rfloor$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Esercizi

9.1-1

Dimostrate che il secondo elemento più piccolo di n elementi può essere trovato con $n + \lceil \lg n \rceil - 2$ confronti nel caso peggiore (*suggerimento:* trovate anche l'elemento più piccolo).

9.1-2 ⋆

Dimostrate che sono necessari $\lceil 3n/2 \rceil - 2$ confronti nel caso peggiore per trovare il massimo e il minimo di n numeri (*suggerimento:* considerate quanti numeri sono potenzialmente massimo o minimo ed esaminate come un confronto influenza questi calcoli).

9.2 Selezione in tempo atteso lineare

Il problema generale della selezione sembra più difficile del semplice problema di trovare un minimo. Eppure, sorprendentemente, il tempo di esecuzione asintotico per entrambi i problemi è lo stesso: $\Theta(n)$. In questo paragrafo, presentiamo un algoritmo divide et impera per il problema della selezione. L'algoritmo RANDOMIZED-SELECT è modellato sull'algoritmo quicksort descritto nel Capitolo 7. Come in quicksort, l'idea di base è partizionare ricorsivamente l'array di input. Diversamente da quicksort, che elabora ricorsivamente entrambi i lati della partizione, RANDOMIZED-SELECT opera soltanto su un lato della partizione. Questa differenza appare evidente nell'analisi: mentre quicksort ha un tempo di esecuzione atteso pari a $\Theta(n \lg n)$, il tempo di esecuzione atteso di RANDOMIZED-SELECT è $\Theta(n)$, nell'ipotesi che tutti gli elementi siano distinti.

RANDOMIZED-SELECT usa la procedura RANDOMIZED-PARTITION introdotta nel Paragrafo 7.3; quindi, come RANDOMIZED-QUICKSORT, è un algoritmo randomizzato, in quanto il suo comportamento è determinato in parte dall'output di un generatore di numeri casuali. Il seguente codice per RANDOMIZED-SELECT restituisce l'i-esimo numero più piccolo dell'array $A[p \,..\, r]$.

```
RANDOMIZED-SELECT(A, p, r, i)
1  if p == r
2       return A[p]
3  q = RANDOMIZED-PARTITION(A, p, r)
4  k = q − p + 1
5  if i == k            // il valore del pivot è la soluzione
6       return A[q]
7  elseif i < k
8       return RANDOMIZED-SELECT(A, p, q − 1, i)
9  else return RANDOMIZED-SELECT(A, q + 1, r, i − k)
```

La procedura RANDOMIZED-SELECT opera in questo modo. La riga 1 verifica il caso base della ricorsione, in cui il sottoarray $A[p \,..\, r]$ è formato da un solo elemento. In questo caso, i deve essere uguale a 1, e viene restituito semplicemente $A[p]$ nella riga 2 come l'i-esimo elemento più piccolo. Altrimenti, la chiamata di RANDOMIZED-PARTITION nella riga 3 divide l'array $A[p \,..\, r]$ in due sottoarray (eventualmente vuoti) $A[p \,..\, q-1]$ e $A[q+1 \,..\, r]$, in modo che ciascun elemento di $A[p \,..\, q-1]$ sia minore o uguale ad $A[q]$ che, a sua volta, è minore di ciascun elemento di $A[q+1 \,..\, r]$. Come in quicksort, chiameremo $A[q]$ l'elemento ***pivot***. La riga 4 calcola il numero k di elementi nel sottoarray $A[p \,..\, q]$, ovvero il numero di elementi nel lato basso della partizione, più uno per l'elemento pivot. La riga 5 poi controlla se $A[q]$ è l'i-esimo elemento più piccolo. Se lo è, restituisce il valore di $A[q]$; altrimenti l'algoritmo determina in quale dei due sottoarray

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995693616 220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

$A[p\,..\,q-1]$ e $A[q+1\,..\,r]$ si trova l'i-esimo elemento più piccolo. Se $i < k$, l'elemento desiderato si trova nel lato basso della partizione e viene selezionato ricorsivamente dal sottoarray nella riga 8. Se $i > k$, l'elemento desiderato si trova nel lato alto della partizione. Poiché conosciamo già k valori che sono più piccoli dell'i-esimo elemento più piccolo di $A[p\,..\,r]$ – gli elementi in $A[p\,..\,q]$ – l'elemento desiderato è l'$(i-k)$-esimo elemento più piccolo di $A[q+1\,..\,r]$, che viene trovato ricorsivamente nella riga 9. Sembra che il codice consenta le chiamate ricorsive a sottoarray con 0 elementi, ma l'Esercizio 9.2-1 vi chiederà di dimostrare che questa situazione non potrà verificarsi.

Il tempo di esecuzione nel caso peggiore di RANDOMIZED-SELECT è $\Theta(n^2)$, anche per trovare il minimo, perché potremmo essere estremamente sfortunati, effettuando la partizione sempre attorno all'elemento più grande rimasto, e il partizionamento richiede un tempo $\Theta(n)$. Tuttavia, vedremo che l'algoritmo ha un tempo di esecuzione atteso lineare e, poiché è randomizzato, nessun input determina il caso peggiore nel comportamento dell'algoritmo.

Per analizzare il tempo di esecuzione atteso di RANDOMIZED-SELECT, indichiamo con $T(n)$ la variabile casuale che rappresenta il tempo di esecuzione su di un array $A[p\,..\,r]$ di n elementi e calcoliamo un limite superiore per $\mathrm{E}[T(n)]$ nel modo seguente. La procedura RANDOMIZED-PARTITION dà a qualsiasi elemento la stessa probabilità di essere selezionato come pivot. Quindi, per ogni k tale che $1 \le k \le n$, il sottoarray $A[p\,..\,q]$ ha k elementi (tutti minori o uguali al pivot) con probabilità $1/n$. Per $k = 1, 2, \ldots, n$, definiamo le variabili casuali indicatrici X_k dove

$$X_k = \mathrm{I}\{\text{il sottoarray } A[p\,..\,q] \text{ ha esattamente } k \text{ elementi}\}$$

Pertanto, supponendo che tutti gli elementi siano distinti, abbiamo

$$\mathrm{E}[X_k] = 1/n \tag{9.1}$$

Quando chiamiamo RANDOMIZED-SELECT e scegliamo $A[q]$ come pivot, non sappiamo, a priori, se termineremo immediatamente con la soluzione esatta o se ci sarà una ricorsione sul sottoarray $A[p\,..\,q-1]$ o sul sottoarray $A[q+1\,..\,r]$. Questa decisione dipende dalla posizione in cui si trova l'i-esimo elemento più piccolo rispetto ad $A[q]$. Supponendo che $T(n)$ sia una funzione monotonicamente crescente, possiamo limitare il tempo richiesto per la chiamata ricorsiva con il tempo richiesto per la chiamata ricorsiva sul massimo input possibile. In altre parole, supponiamo, per ottenere un limite superiore, che l'i-esimo elemento sia sempre nel lato della partizione con il maggior numero di elementi. Per una chiamata di RANDOMIZED-SELECT, la variabile casuale indicatrice X_k vale 1 per un solo valore di k e 0 per tutti gli altri valori di k. Quando $X_k = 1$, i due sottoarray sui quali potremmo effettuare la ricorsione hanno dimensioni $k-1$ e $n-k$. Quindi, abbiamo la ricorrenza

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned} T(n) &\le \sum_{k=1}^{n} X_k \cdot (T(\max(k-1, n-k)) + O(n)) \\ &= \sum_{k=1}^{n} X_k \cdot T(\max(k-1, n-k)) + O(n) \end{aligned}$$

Prendendo i valori attesi, si ha

$$\begin{aligned}
&\mathrm{E}[T(n)] \\
&\leq \mathrm{E}\left[\sum_{k=1}^{n} X_k \cdot T(\max(k-1, n-k)) + O(n)\right] \\
&= \sum_{k=1}^{n} \mathrm{E}[X_k \cdot T(\max(k-1, n-k))] + O(n) && \text{(per la linearità del valore atteso)} \\
&= \sum_{k=1}^{n} \mathrm{E}[X_k] \cdot \mathrm{E}[T(\max(k-1, n-k))] + O(n) && \text{(per l'equazione (C.24))} \\
&= \sum_{k=1}^{n} \frac{1}{n} \cdot \mathrm{E}[T(\max(k-1, n-k))] + O(n) && \text{(per l'equazione (9.1))}
\end{aligned}$$

Per applicare l'equazione (C.24), contiamo sul fatto che X_k e $T(\max(k-1, n-k))$ sono variabili casuali indipendenti. L'Esercizio 9.2-2 chiede di giustificare questa asserzione.

Consideriamo l'espressione $\max(k-1, n-k)$. Abbiamo

$$\max(k-1, n-k) = \begin{cases} k-1 & \text{se } k > \lceil n/2 \rceil \\ n-k & \text{se } k \leq \lceil n/2 \rceil \end{cases}$$

Se n è pari, ogni termine da $T(\lceil n/2 \rceil)$ fino a $T(n-1)$ appare esattamente due volte nella sommatoria e, se n è dispari, tutti questi termini appaiono due volte, mentre $T(\lfloor n/2 \rfloor)$ appare una volta soltanto. Dunque, abbiamo

$$\mathrm{E}[T(n)] \leq \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} \mathrm{E}[T(k)] + O(n)$$

Dimostriamo che $\mathrm{E}[T(n)] = O(n)$ per sostituzione. Supponiamo che $\mathrm{E}[T(n)] \leq cn$ per qualche costante c che soddisfa le condizioni iniziali della ricorrenza. Assumiamo che $T(n) = O(1)$ per n minore di qualche costante; sceglieremo questa costante successivamente. Scegliamo anche una costante a tale che la funzione descritta dal termine $O(n)$ (che descrive la componente non ricorsiva del tempo di esecuzione dell'algoritmo) sia limitata dall'alto da an per ogni $n > 0$. Utilizzando questa ipotesi induttiva, otteniamo

$$\begin{aligned}
\mathrm{E}[T(n)] &\leq \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} ck + an \\
&= \frac{2c}{n}\left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k\right) + an \\
&= \frac{2c}{n}\left(\frac{(n-1)n}{2} - \frac{(\lfloor n/2 \rfloor - 1)\lfloor n/2 \rfloor}{2}\right) + an \\
&\leq \frac{2c}{n}\left(\frac{(n-1)n}{2} - \frac{(n/2-2)(n/2-1)}{2}\right) + an \\
&= \frac{2c}{n}\left(\frac{n^2-n}{2} - \frac{n^2/4 - 3n/2 + 2}{2}\right) + an
\end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:13. Numero Ordine Libraria: 19950816-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$
\begin{aligned}
&= \frac{c}{n}\left(\frac{3n^2}{4} + \frac{n}{2} - 2\right) + an \\
&= c\left(\frac{3n}{4} + \frac{1}{2} - \frac{2}{n}\right) + an \\
&\leq \frac{3cn}{4} + \frac{c}{2} + an \\
&= cn - \left(\frac{cn}{4} - \frac{c}{2} - an\right)
\end{aligned}
$$

Per completare la dimostrazione, dobbiamo dimostrare che per n sufficientemente grande, quest'ultima espressione vale al massimo cn ovvero $cn/4 - c/2 - an \geq 0$. Se sommiamo $c/2$ a entrambi i lati e mettiamo in evidenza il fattore n, otteniamo $n(c/4 - a) \geq c/2$. Se scegliamo la costante c in modo che $c/4 - a > 0$, ovvero $c > 4a$, possiamo dividere entrambi i lati per $c/4 - a$, ottenendo

$$n \geq \frac{c/2}{c/4 - a} = \frac{2c}{c - 4a}$$

In definitiva, se supponiamo che $T(n) = O(1)$ per $n < 2c/(c - 4a)$, abbiamo $\mathrm{E}[T(n)] = O(n)$. Possiamo concludere che qualsiasi statistica d'ordine, in particolare la mediana, può essere determinata in tempo atteso lineare, nell'ipotesi che gli elementi siano distinti.

Esercizi

9.2-1

Dimostrate che nell'algoritmo RANDOMIZED-SELECT non viene mai effettuata una chiamata ricorsiva a un array di lunghezza 0.

9.2-2

Dimostrate che la variabile casuale indicatrice X_k e il valore $T(\max(k-1, n-k))$ sono indipendenti.

9.2-3

Scrivete una versione iterativa di RANDOMIZED-SELECT.

9.2-4

Supponete di utilizzare RANDOMIZED-SELECT per selezionare l'elemento minimo dell'array $A = \langle 3, 2, 9, 0, 7, 5, 4, 8, 6, 1\rangle$. Descrivete una sequenza di partizioni che determina il comportamento nel caso peggiore di RANDOMIZED-SELECT.

9.3 Selezione in tempo lineare nel caso peggiore

Esaminiamo adesso un algoritmo di selezione il cui tempo di esecuzione è $O(n)$ nel caso peggiore. Come RANDOMIZED-SELECT, anche l'algoritmo SELECT trova l'elemento desiderato partizionando ricorsivamente l'array di input. L'idea che sta alla base dell'algoritmo, tuttavia, è quella di *garantire* una buona ripartizione dell'array. SELECT usa l'algoritmo di partizionamento deterministico PARTITION, lo stesso che usa quicksort (vedere il Paragrafo 7.1), a parte una modifica che consente di accettare come parametro di input l'elemento attorno al quale effettuare il partizionamento.

Acquistato da Michele Michele su Webster il 2022-01-12 2013. Numero ordine singolo 1005000016 (2270572) Copyright © 2022 McGraw-Hill Education (Italy)

L'algoritmo SELECT determina l'i-esimo elemento più piccolo di un array di input di n elementi ($n > 1$), effettuando i seguenti passi (se $n = 1$, SELECT restituisce semplicemente il suo unico elemento di input come l'i-esimo valore più piccolo).

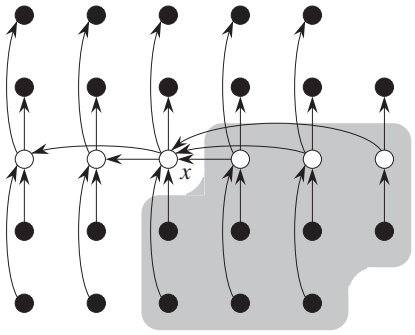


Figura 9.1 Analisi dell'algoritmo SELECT. Gli n elementi sono rappresentati da piccoli cerchi e ogni gruppo occupa una colonna. Le mediane dei gruppi sono rappresentate da cerchi bianchi; la mediana delle mediane è identificata dalla lettera x (per nostra convenzione, la mediana di un numero pari di elementi è la mediana inferiore). Le frecce vanno dagli elementi più grandi a quelli più piccoli; da questo si può notare che 3 elementi in ciascuno dei gruppi completi di 5 elementi a destra di x sono maggiori di x, e 3 elementi in ciascuno dei gruppi completi di 5 elementi a sinistra di x sono minori x. Gli elementi maggiori di x sono rappresentati su uno sfondo grigio.

1. Dividere gli n elementi dell'array di input in $\lfloor n/5 \rfloor$ gruppi di 5 elementi ciascuno e (al massimo) un gruppo con i restanti n mod 5 elementi.
2. Trovare la mediana di ciascuno degli $\lceil n/5 \rceil$ gruppi, effettuando prima un ordinamento per inserimento degli elementi di ogni gruppo (al massimo 5) e poi scegliendo la mediana dalla lista ordinata degli elementi di ogni gruppo.
3. Usare SELECT ricorsivamente per trovare la mediana x delle $\lceil n/5 \rceil$ mediane trovate nel passo 2 (se il numero di mediane è pari, allora per nostra convenzione x è la mediana inferiore).
4. Partizionare l'array di input attorno alla mediana delle mediane x utilizzando la versione modificata di PARTITION. Sia k il numero di elementi nel lato basso della partizione più 1 in modo che x è il k-esimo elemento più piccolo e il lato alto della partizione contiene $n - k$ elementi.
5. Se $i = k$, restituire x; altrimenti utilizzare SELECT ricorsivamente per trovare l'i-esimo elemento più piccolo nel lato basso se $i < k$ oppure l'$(i - k)$-esimo elemento più piccolo nel lato alto se $i > k$.

Per analizzare il tempo di esecuzione di SELECT, determiniamo prima un limite inferiore sul numero di elementi che sono maggiori dell'elemento di partizionamento x. La Figura 9.1 è utile per visualizzare i conteggi. Almeno metà delle mediane trovate nel passo 2 sono maggiori o uguali alla mediana delle mediane x.[1] Quindi, almeno metà degli $\lceil n/5 \rceil$ gruppi contribuisce con 3 elementi che sono maggiori di x, tranne quel gruppo che ha meno di 5 elementi (se n non è divisibile esattamente per 5) e quel gruppo che contiene x. Se escludiamo questi due gruppi, allora il numero di elementi maggiori di x è almeno

$$3\left(\left\lceil \frac{1}{2}\left\lceil \frac{n}{5}\right\rceil \right\rceil - 2\right) \geq \frac{3n}{10} - 6$$

Analogamente, il numero di elementi che sono minori di x è almeno $3n/10 - 6$. Quindi nel caso peggiore SELECT viene chiamata ricorsivamente al massimo su $7n/10 + 6$ elementi nel passo 5.

Acquistato da Michele Michele su Webster il 2022-06-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[1] Poiché abbiamo fatto l'ipotesi che i numeri siano distinti, tutte le mediane (tranne x) sono maggiori o minori di x.

Adesso possiamo sviluppare una ricorrenza per il tempo di esecuzione $T(n)$ nel caso peggiore dell'algoritmo SELECT. I passi 1, 2 e 4 impiegano un tempo $O(n)$ (il passo 2 è formato da $O(n)$ chiamate di insertion sort su insiemi di dimensione $O(1)$). Il passo 3 impiega un tempo $T(\lceil n/5 \rceil)$; il passo 5 impiega al massimo un tempo $T(7n/10 + 6)$, supponendo che T sia monotonicamente crescente. Facciamo l'ipotesi, che a prima vista può sembrare immotivata, che qualsiasi input con meno di 140 elementi richieda un tempo $O(1)$; l'origine della costante magica 140 sarà chiarita a breve. Possiamo ottenere la seguente ricorrenza:

$$T(n) \leq \begin{cases} O(1) & \text{se } n < 140 \\ T(\lceil n/5 \rceil) + T(7n/10 + 6) + O(n) & \text{se } n \geq 140 \end{cases}$$

Dimostreremo che il tempo di esecuzione è lineare, applicando il metodo di sostituzione. Più specificatamente, dimostreremo che $T(n) \leq cn$ per qualche costante c opportunamente grande e per ogni $n > 0$. Iniziamo supponendo che $T(n) \leq cn$ per qualche costante c opportunamente grande e per ogni $n < 140$; questa ipotesi è valida se c è abbastanza grande. Scegliamo anche una costante a tale che la funzione descritta dal precedente termine $O(n)$ (che descrive la componente non ricorsiva del tempo di esecuzione dell'algoritmo) sia limitata dall'alto da an per ogni $n > 0$. Sostituendo questa ipotesi induttiva nel lato destro della ricorrenza, si ottiene

$$\begin{aligned} T(n) &\leq c\lceil n/5 \rceil + c(7n/10 + 6) + an \\ &\leq cn/5 + c + 7cn/10 + 6c + an \\ &= 9cn/10 + 7c + an \\ &= cn + (-cn/10 + 7c + an) \end{aligned}$$

che vale al massimo cn, se

$$-cn/10 + 7c + an \leq 0 \tag{9.2}$$

La disequazione (9.2) è equivalente alla disequazione $c \geq 10a(n/(n-70))$ se $n > 70$. Poiché supponiamo che $n \geq 140$, allora $n/(n-70) \leq 2$; quindi scegliendo $c \geq 20a$ è soddisfatta la disequazione (9.2) (notate che la costante 140 non ha nulla di speciale; potremmo sostituirla con un altro numero intero strettamente maggiore di 70 e poi scegliere opportunamente c). Il tempo di esecuzione nel caso peggiore di SELECT è dunque lineare.

Come in un ordinamento per confronti (vedere il Paragrafo 8.1), SELECT e RANDOMIZED-SELECT determinano le informazioni sull'ordine relativo degli elementi esclusivamente tramite i confronti degli elementi. Ricordiamo dal Capitolo 8 che l'ordinamento richiede un tempo $\Omega(n \lg n)$ nel modello dei confronti, anche nel caso medio (vedere il Problema 8-1). Gli algoritmi di ordinamento in tempo lineare descritti nel Capitolo 8 fanno delle ipotesi sull'input. Gli algoritmi di selezione in tempo lineare presentati in questo capitolo, invece, non richiedono alcuna ipotesi sull'input. Essi non sono soggetti al limite inferiore $\Omega(n \lg n)$ perché riescono a risolvere il problema della selezione senza ordinare gli elementi. Pertanto il metodo di ordinamento e indicizzazione presentato nell'introduzione di questo capitolo è asintoticamente inefficiente.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

Esercizi

9.3-1

Nell'algoritmo SELECT gli elementi di input sono suddivisi in gruppi di 5. L'algoritmo opera in tempo lineare anche se gli elementi sono suddivisi in gruppi di 7? Dimostrate che SELECT non viene eseguito in tempo lineare se vengono utilizzati gruppi di 3 elementi.

9.3-2

Analizzate SELECT per dimostrare che, se $n \geq 140$, allora almeno $\lceil n/4 \rceil$ elementi sono maggiori della mediana delle mediane x e almeno $\lceil n/4 \rceil$ elementi sono minori di x.

9.3-3

Spiegate come quicksort possa essere eseguito nel tempo $O(n \lg n)$ nel caso peggiore, supponendo che tutti gli elementi siano distinti.

9.3-4 ⋆

Supponete che un algoritmo usi soltanto i confronti per trovare l'i-esimo elemento più piccolo in un insieme di n elementi. Dimostrate che l'algoritmo può trovare gli $i - 1$ elementi più piccoli e gli $n - i$ elementi più grandi senza svolgere confronti aggiuntivi.

9.3-5

Supponete di avere una subroutine "black-box" (scatola nera) che trova la mediana in tempo lineare nel caso peggiore. Create un semplice algoritmo che risolve in tempo lineare il problema della selezione per un'arbitraria statistica d'ordine.

9.3-6

I k-esimi ***quantili*** di un insieme di n elementi sono le $k - 1$ statistiche d'ordine che dividono l'insieme ordinato in k insiemi della stessa dimensione (a meno di 1). Create un algoritmo con tempo $O(n \lg k)$ che elenca i k-esimi quantili di un insieme.

9.3-7

Descrivete un algoritmo con tempo $O(n)$ che, dato un insieme S di n numeri distinti e un intero positivo $k \leq n$, trova gli i numeri in S che sono più vicini alla mediana di S.

9.3-8

Siano $X[1..n]$ e $Y[1..n]$ due array, ciascuno contenente n numeri già ordinati. Scrivete un algoritmo con tempo $O(\lg n)$ per trovare la mediana di tutti i $2n$ elementi degli array X e Y.

9.3-9

Il professor Olay è un consulente di una compagnia petrolifera, che sta progettando un importante oleodotto che attraversa da est a ovest una vasta area con n pozzi petroliferi. Da ogni pozzo parte una condotta che deve essere collegata direttamente alla condotta principale lungo il percorso minimo (da nord o sud), come illustra la Figura 9.2. Se sono note le coordinate x e y dei pozzi, in che modo il professor Olay può scegliere la posizione ottimale della condotta principale (quella che rende minima la lunghezza totale delle condotte verticali che collegano i pozzi alla condotta principale)? Dimostrate che la posizione ottimale può essere trovata in tempo lineare.

Acquistato da Michele Michele sul Webster il 2022-07-07 03:12 Numero Ordine Libreria 10853016 220707 | Copyright © 2022 McGraw-Hill Education (Italy)

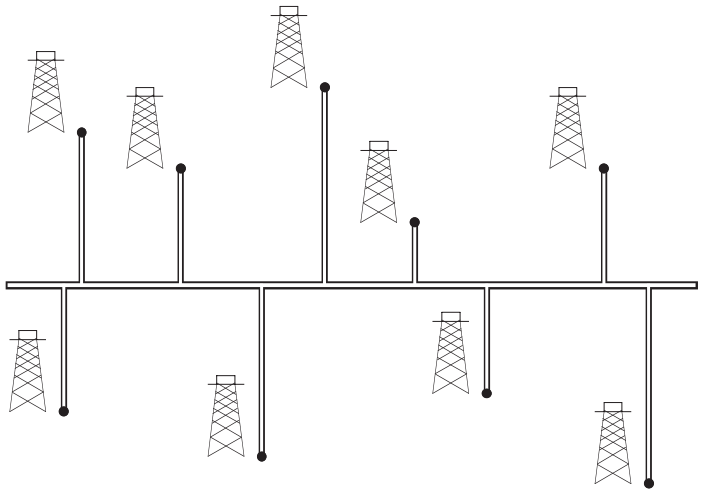


Figura 9.2 Il professor Olay deve determinare la posizione della condotta orizzontale che rende minima la lunghezza totale delle condotte verticali dai pozzi petroliferi.

Problemi

9-1 Trovare gli i numeri più grandi ordinati

Dato un insieme di n numeri, trovare gli i numeri più grandi ordinati utilizzando un algoritmo basato sui confronti. Trovate l'algoritmo che implementa ciascuno dei seguenti metodi con il miglior tempo di esecuzione asintotico nel caso peggiore; analizzate i tempi di esecuzione in funzione di n e i.

a. Ordinare i numeri ed elencare gli i numeri più grandi.

b. Costruire una coda di max-priorità dai numeri e chiamare i volte la procedura EXTRACT-MAX.

c. Utilizzare un algoritmo per statistiche d'ordine che trova l'i-esimo numero più grande, creare le partizioni attorno a questo numero e ordinare gli i numeri più grandi.

9-2 Mediana ponderata

Per n elementi distinti $x_1, x_2, \ldots, x_n$ con pesi positivi $w_1, w_2, \ldots, w_n$ tali che $\sum_{i=1}^{n} w_i = 1$, la ***mediana (inferiore) ponderata*** è l'elemento x_k che soddisfa le relazioni

$$\sum_{x_i < x_k} w_i < \frac{1}{2}$$

e

$$\sum_{x_i > x_k} w_i \leq \frac{1}{2}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Per esempio, se gli elementi sono $0.1, 0.35, 0.05, 0.1, 0.15, 0.05, 0.2$ e ciascun elemento è uguale al suo peso (ovvero, $w_i = x_i$ per $i = 1, 2, \ldots, 7$), allora la mediana è 0.1, ma la mediana ponderata è 0.2.

a. Dimostrate che la mediana di $x_1, x_2, \ldots, x_n$ è la mediana ponderata di x_i con pesi $w_i = 1/n$ per $i = 1, 2, \ldots, n$.

b. Come può essere calcolata la mediana ponderata di n elementi nel tempo $O(n \lg n)$ nel caso peggiore utilizzando un algoritmo di ordinamento?

c. Spiegate come calcolare la mediana ponderata nel tempo $\Theta(n)$ nel caso peggiore utilizzando un algoritmo in tempo lineare come SELECT, che è descritto nel Paragrafo 9.3.

Il ***problema della localizzazione dell'ufficio postale*** è definito nel seguente modo. Dati n punti $p_1, p_2, \ldots, p_n$ con i pesi associati $w_1, w_2, \ldots, w_n$, trovare un punto p (non necessariamente appartenente a uno dei punti di input) che rende minima la sommatoria $\sum_{i=1}^{n} w_i \, d(p, p_i)$, dove $d(a, b)$ è la distanza tra i punti a e b.

d. Dimostrate che la mediana ponderata è una soluzione ottima per il problema monodimensionale della localizzazione dell'ufficio postale, in cui i punti sono semplicemente numeri reali e la distanza tra i punti a e b è $d(a, b) = |a - b|$.

e. Trovate la soluzione ottima per il problema bidimensionale della localizzazione dell'ufficio postale, in cui i punti sono coppie di coordinate (x, y) e la distanza tra i punti $a = (x_1, y_1)$ e $b = (x_2, y_2)$ è la ***distanza di Manhattan*** data da $d(a, b) = |x_1 - x_2| + |y_1 - y_2|$.

9-3 Statistiche d'ordine piccole

Abbiamo dimostrato che il numero di confronti $T(n)$ nel caso peggiore utilizzato da SELECT per selezionare l'i-esima statistica d'ordine da n numeri soddisfa la relazione $T(n) = \Theta(n)$, ma la costante nascosta dalla notazione Θ è piuttosto grande. Se il valore di i è piccolo rispetto a n, possiamo implementare una procedura differente che usa SELECT come subroutine, ma effettua un minor numero di confronti nel caso peggiore.

a. Descrivete un algoritmo che usa $U_i(n)$ confronti per trovare l'i-esimo elemento più piccolo di n elementi, dove

$$U_i(n) = \begin{cases} T(n) & \text{se } i \geq n/2 \\ \lfloor n/2 \rfloor + U_i(\lceil n/2 \rceil) + T(2i) & \text{negli altri casi} \end{cases}$$

(*Suggerimento:* iniziate con $\lfloor n/2 \rfloor$ confronti di coppie disgiunte; poi eseguite una ricorsione sull'insieme che contiene l'elemento più piccolo di ogni coppia.)

b. Dimostrate che, se $i < n/2$, allora $U_i(n) = n + O(T(2i) \lg(n/i))$.

c. Dimostrate che, se i è una costante minore di $n/2$, allora $U_i(n) = n + O(\lg n)$.

d. Dimostrate che, se $i = n/k$ per $k \geq 2$, allora $U_i(n) = n + O(T(2n/k) \lg k)$.

9-4 Analisi alternativa della selezione randomizzata

Acquistato da Michele Michele su Webster il 2022-07-07T23:12. Numero Ordine Libraria: 1895036.220707.0 Copyright © 2022, McGraw-Hill Education (Italy)

In questo problema utilizzeremo variabili casuali indicatrici per analizzare la procedura RANDOMIZED-SELECT in modo simile a quanto fatto per la procedura RANDOMIZED-QUICKSORT nel Paragrafo 7.4.2. Come nell'analisi di quicksort, supporremo che tutti gli elementi siano distinti e cambieremo il nome degli elementi dell'array di input A in $z_1, z_2, \ldots, z_n$, dove z_i è l'i-esimo elemento più piccolo. Quindi, la chiamata RANDOMIZED-SELECT$(A, 1, n, k)$ restituisce z_k.

Per $1 \leq i < j \leq n$, sia

$X_{ijk} = \mathrm{I}\{z_i$ è confrontato con z_j durante l'esecuzione dell'algoritmo per trovare $z_k\}$

a. Trovate un'espressione esatta di $\mathrm{E}[X_{ijk}]$ (*suggerimento:* la vostra espressione potrebbe avere valori differenti, in base ai valori di i, j e k).

b. Indicate con X_k il numero totale di confronti tra gli elementi dell'array A necessari per trovare z_k. Dimostrate che

$$\mathrm{E}[X_k] \leq 2\left(\sum_{i=1}^{k}\sum_{j=k}^{n}\frac{1}{j-i+1} + \sum_{j=k+1}^{n}\frac{j-k-1}{j-k+1} + \sum_{i=1}^{k-2}\frac{k-i-1}{k-i+1}\right)$$

c. Dimostrate che $\mathrm{E}[X_k] \leq 4n$.

d. Concludete che, supponendo che tutti gli elementi dell'array A siano distinti, RANDOMIZED-SELECT viene eseguita nel tempo atteso $O(n)$.

Note

L'algoritmo che trova la mediana in tempo lineare nel caso peggiore è stato sviluppato da Blum, Floyd, Pratt, Rivest e Tarjan [50]. La versione randomizzata veloce è stata ideata da Hoare [170]. Floyd e Rivest [109] hanno sviluppato una versione randomizzata migliore che crea le partizioni attorno a un elemento selezionato ricorsivamente da un piccolo campione di elementi.

Non è ancora noto con esattezza il numero di confronti che sono necessari per determinare la mediana. Bent e John [41] hanno trovato un limite inferiore di $2n$ confronti per trovare la mediana. Schönhage, Paterson e Pippenger [303] hanno trovato un limite superiore di $3n$ confronti. Dor e Zwick hanno migliorato entrambi questi limiti; il loro limite superiore [94] è leggermente più piccolo di $2.95n$ e quello inferiore [95] è $(2+\epsilon)n$, con ϵ piccola costante positiva, migliorando così leggermente il lavoro di Dor et al. [93]. Paterson [273] descrive questi risultati e altri lavori correlati.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

III Strutture dati

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Introduzione

Gli insiemi sono fondamentali per l'informatica e la matematica. Mentre gli insiemi matematici sono immutabili, gli insiemi manipolati dagli algoritmi possono crescere, ridursi o cambiare nel tempo. Questi insiemi sono detti ***dinamici***. I prossimi cinque capitoli presentano alcune tecniche di base per rappresentare e manipolare gli insiemi dinamici finiti.

Gli algoritmi possono richiedere vari tipi di operazioni da svolgere sugli insiemi. Per esempio, molti algoritmi richiedono soltanto la capacità di inserire e cancellare degli elementi da un insieme e di verificare l'appartenenza di un elemento a un insieme. Un insieme dinamico che supporta queste operazioni è detto ***dizionario***. Altri algoritmi richiedono operazioni più complicate. Per esempio, le code di min-priorità, che sono state introdotte nel Capitolo 6 nell'ambito della struttura heap, supportano le operazioni per inserire un elemento in un insieme e per estrarre l'elemento più piccolo da un insieme. Il modo migliore di implementare un insieme dinamico dipende dalle operazioni che devono essere supportate.

Gli elementi di un insieme dinamico

In una tipica implementazione di un insieme dinamico, ogni elemento è rappresentato da un oggetto i cui attributi possono essere esaminati e manipolati se c'è un puntatore all'oggetto (il Paragrafo 10.3 descrive l'implementazione di oggetti e puntatori negli ambienti di programmazione che non li prevedono come tipi di dati di base). Per alcuni tipi di insiemi dinamici si suppone che uno degli attributi dell'oggetto sia una ***chiave*** di identificazione. Se le chiavi sono tutte diverse, possiamo pensare all'insieme dinamico come a un insieme di valori chiave. L'oggetto può contenere ***dati satelliti***, che vengono trasportati in altri attributi dell'oggetto, senza essere utilizzati in altro modo dall'implementazione dell'insieme. L'oggetto può anche includere attributi che vengono manipolati dalle operazioni svolte sull'insieme; questi attributi possono contenere dati o puntatori ad altri oggetti dell'insieme.

Acquistato da Michele Mule' (mmule@libero.it) - Copyright © 2022 McGraw-Hill Education (Italy)

Alcuni insiemi dinamici presuppongono che le chiavi siano estratte da un insieme totalmente ordinato, come i numeri reali o l'insieme di tutte le parole secondo il consueto ordine alfabetico (un insieme totalmente ordinato soddisfa la proprietà della tricotomia, definita a pagina 45). L'ordinamento totale ci consente di definire l'elemento minimo dell'insieme, per esempio, o di parlare del prossimo elemento più grande di un determinato elemento dell'insieme.

Operazioni sugli insiemi dinamici

Le operazioni su un insieme dinamico possono essere raggruppate in due categorie: le ***interrogazioni*** o ***query*** che restituiscono semplicemente informazioni sull'insieme; le ***operazioni di modifica*** che cambiano l'insieme. Elenchiamo qui di seguito una serie di operazioni tipiche. Un'applicazione reale di solito richiede l'implementazione di un numero limitato di queste operazioni.

SEARCH(S, k)
: Una query che, dato un insieme S e un valore chiave k, restituisce un puntatore x a un elemento di S tale che $key[x] = k$ oppure NIL se un elemento così non appartiene a S.

INSERT(S, x)
: Un'operazione di modifica che inserisce nell'insieme S l'elemento puntato da x. Di solito, si suppone che qualsiasi attributo dell'elemento x richiesto dall'implementazione dell'insieme sia stato già inizializzato.

DELETE(S, x)
: Un'operazione di modifica che, dato un puntatore x a un elemento dell'insieme S, rimuove x da S (notate che questa operazione usa un puntatore a un elemento x, non un valore chiave).

MINIMUM(S)
: Una query su un insieme totalmente ordinato S che restituisce un puntatore all'elemento di S con la chiave più piccola.

MAXIMUM(S)
: Una query su un insieme totalmente ordinato S che restituisce un puntatore all'elemento di S con la chiave più grande.

SUCCESSOR(S, x)
: Una query che, dato un elemento x la cui chiave appartiene a un insieme totalmente ordinato S, restituisce un puntatore al prossimo elemento più grande di S oppure NIL se x è l'elemento massimo.

PREDECESSOR(S, x)
: Una query che, dato un elemento x la cui chiave appartiene a un insieme totalmente ordinato S, restituisce un puntatore al prossimo elemento più piccolo di S oppure NIL se x è l'elemento minimo.

Le query SUCCESSOR e PREDECESSOR vengono spesso estese a insiemi con chiavi non distinte. Per un insieme con n chiavi, è lecito supporre che una chiamata di MINIMUM seguita da $n - 1$ chiamate di SUCCESSOR enumeri ordinatamente gli elementi dell'insieme.

Il tempo impiegato per eseguire un'operazione su un insieme, di solito, è misurato in funzione della dimensione dell'insieme, che viene specificata come uno degli argomenti dell'operazione. Per esempio, il Capitolo 13 descrive una struttura dati che è in grado di svolgere una qualsiasi delle operazioni precedentemente elencate su un insieme di dimensione n nel tempo $O(\lg n)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Sintesi della Parte III

I Capitoli 10–14 descrivono diverse strutture dati che possono essere utilizzate per implementare insiemi dinamici; molte di queste strutture saranno utilizzate successivamente per costruire algoritmi efficienti che risolvono vari tipi di problemi. Un'altra importante struttura dati – l'heap – è stata già presentata nel Capitolo 6.

Il Capitolo 10 illustra i concetti essenziali per operare con semplici strutture dati, come stack, code, liste concatenate e alberi radicati. In questo capitolo spiegheremo anche come implementare oggetti e puntatori in quegli ambienti di programmazione che non li includono fra i tipi di dati di base. Molti di questi argomenti dovrebbero essere familiari a chiunque abbia svolto un corso introduttivo di programmazione.

Il Capitolo 11 presenta le tavole hash, che supportano le operazioni INSERT, DELETE e SEARCH sui dizionari. Nel caso peggiore, l'hashing richiede un tempo $\Theta(n)$ per svolgere un'operazione SEARCH, ma il tempo atteso per le operazioni con le tavole hash è $O(1)$. L'analisi dell'hashing è basata sulla teoria della probabilità, ma la maggior parte del capitolo non richiede la conoscenza di questa materia.

Gli alberi binari di ricerca, che sono trattati nel Capitolo 12, supportano tutte le operazioni sugli insiemi dinamici precedentemente elencate. Nel caso peggiore, ogni operazione richiede un tempo $\Theta(n)$ su un albero con n elementi, ma su un albero binario di ricerca costruito in modo casuale, il tempo atteso per ogni operazione è $O(\lg n)$. Gli alberi binari di ricerca sono la base di molte altre strutture dati.

Gli alberi rosso-neri (red-black), una variante degli alberi binari di ricerca, sono presentati nel Capitolo 13. Diversamente dai normali alberi binari di ricerca, gli alberi rosso-neri garantiscono buone prestazioni: le operazioni richiedono un tempo $O(\lg n)$ nel caso peggiore. Un albero rosso-nero è un albero di ricerca bilanciato; il Capitolo 18 presenta un altro tipo di albero di ricerca bilanciato, detto B-albero. Sebbene i meccanismi degli alberi rosso-neri siano alquanto complicati, tuttavia nel capitolo potete racimolare gran parte delle informazioni sulle loro proprietà, senza bisogno di studiare dettagliatamente tali meccanismi. Nonostante questo, sarà molto istruttivo analizzare attentamente il codice.

Nel Capitolo 14 spieghiamo come estendere gli alberi rosso-neri per supportare operazioni diverse da quelle di base precedentemente elencate. La prima estensione ci consente di mantenere dinamicamente le statistiche d'ordine di un insieme di chiavi; la seconda estensione ci consente di mantenere degli intervalli di numeri reali.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

10 Strutture dati elementari

In questo capitolo esamineremo la rappresentazione di insiemi dinamici mediante semplici strutture dati che usano i puntatori. Sebbene molte strutture dati complesse possano essere modellate utilizzando i puntatori, presenteremo soltanto le strutture più semplici: stack (o pile), code, liste concatenate e alberi radicati. Presenteremo anche un metodo per rappresentare oggetti e puntatori mediante gli array.

10.1 Stack e code

Gli stack e le code sono insiemi dinamici dove l'elemento che viene rimosso dall'operazione DELETE è predeterminato. In uno ***stack*** l'elemento cancellato dall'insieme è quello inserito per ultimo: lo stack implementa lo schema ***LIFO*** (Last-In, First-Out). Analogamente, in una ***coda*** l'elemento cancellato è sempre quello che è rimasto nell'insieme per più tempo: la coda implementa lo schema ***FIFO*** (First-In, First-Out). Ci sono vari modi efficienti per implementare gli stack e le code in un calcolatore. In questo paragrafo spiegheremo come utilizzare un semplice array per implementare entrambe le strutture dati.

Stack

L'operazione INSERT su uno stack è detta PUSH, mentre l'operazione DELETE, che non ha un elemento come argomento, è detta POP. Questi nomi inglesi sono allusioni alle pile (*stack*) reali, come le pile di piatti caricate a molla che sono utilizzate nelle tavole calde. L'ordine in cui i piatti vengono rimossi (*pop*) dalla pila è inverso a quello in cui sono stati inseriti (*push*), in quanto è accessibile soltanto il piatto che è in cima alla pila.

Come illustra la Figura 10.1, possiamo implementare uno stack di al più n elementi con un array $S[1\,..\,n]$. L'array ha un attributo $S.top$ che è l'indice dell'elemento inserito più di recente. Lo stack è formato dagli elementi $S[1\,..\,S.top]$, dove $S[1]$ è l'elemento in fondo allo stack e $S[S.top]$ è l'elemento in cima.

Se $S.top = 0$, lo stack non contiene elementi ed è ***vuoto***. L'operazione STACK-EMPTY verifica se uno stack è vuoto. Se si tenta di estrarre un elemento (operazione POP) da uno stack vuoto, si ha un ***underflow*** dello stack, che di norma è un errore. Se $S.top$ supera n, si ha un ***overflow*** dello stack (nel nostro pseudocodice non ci preoccuperemo dell'overflow dello stack).

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Le operazioni sullo stack possono essere implementate con poche righe di codice.

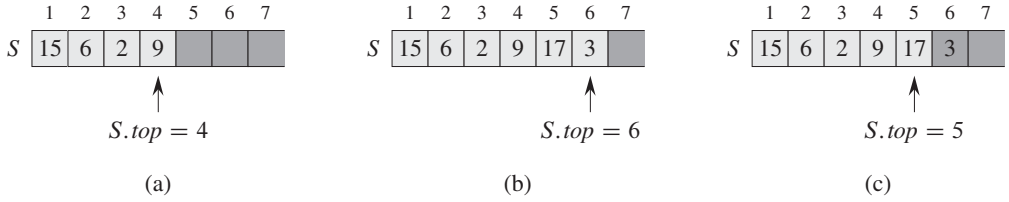


Figura 10.1 Implementazione di uno stack S con un array. Gli elementi dello stack appaiono soltanto nelle posizioni con sfondo grigio chiaro. **(a)** Lo stack S ha 4 elementi. L'elemento in cima allo stack è 9. **(b)** Lo stack S dopo le chiamate PUSH$(S, 17)$ e PUSH$(S, 3)$. **(c)** Lo stack S dopo la chiamata POP(S) ha ceduto l'elemento 3, che è l'elemento inserito più di recente. Sebbene l'elemento 3 appaia ancora nell'array, non appartiene più allo stack; l'elemento in cima allo stack è l'elemento 17.

```
STACK-EMPTY(S)
1  if S.top == 0
2       return TRUE
3  else return FALSE

PUSH(S, x)
1  S.top = S.top + 1
2  S[S.top] = x

POP(S)
1  if STACK-EMPTY(S)
2       error "underflow"
3  else S.top = S.top − 1
4       return S[S.top + 1]
```

La Figura 10.1 illustra gli effetti delle operazioni di modifica PUSH e POP. Ciascuna delle tre operazioni sullo stack richiede un tempo $O(1)$.

Code

Chiameremo ENQUEUE l'operazione INSERT su una coda. Chiameremo DEQUEUE l'operazione DELETE; come l'operazione POP su uno stack, anche DEQUEUE non richiede un elemento come argomento. La proprietà FIFO di una coda fa sì che essa funzioni come le file delle persone che a volte si formano davanti agli sportelli degli uffici.

La coda ha un inizio (*head*) e una fine (*tail*). Quando un elemento viene inserito nella coda, prende posto alla fine della coda, esattamente come l'ultima persona che arriva si mette in fondo alla fila. L'elemento rimosso è sempre quello che si trova all'inizio della coda, come la persona all'inizio della fila, che è quella rimasta in attesa più a lungo delle altre.

Acquistato da Michele Mirabella (michele.mirabella@uniba.it) - Numero ordine: Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

La Figura 10.2 illustra un modo di implementare una coda di $n - 1$ elementi al massimo, utilizzando un array $Q[1..n]$. L'attributo $Q.head$ indica (o punta) l'inizio della coda. L'attributo $Q.tail$ indica la prossima posizione in cui l'ultimo elemento che arriva sarà inserito nella coda. Gli elementi della coda occupano le posizioni $Q.head, Q.head + 1, \ldots, Q.tail - 1$. Alla fine dell'array si "va a capo" nel senso che la posizione 1 segue immediatamente la posizione n secondo un ordine circolare. Se $Q.head = Q.tail$, la coda è vuota. All'inizio $Q.head =$

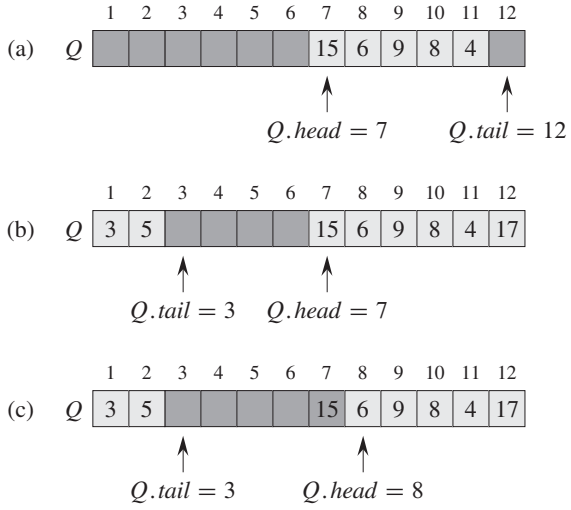


Figura 10.2 Una coda implementata con un array $Q[1\,..\,12]$. Gli elementi della coda appaiono soltanto nelle posizioni con sfondo grigio chiaro. **(a)** La coda ha 5 elementi nelle posizioni $Q[7\,..\,11]$. **(b)** La configurazione della coda dopo le chiamate ENQUEUE$(Q, 17)$, ENQUEUE$(Q, 3)$ e ENQUEUE$(Q, 5)$. **(c)** La configurazione della coda dopo che la chiamata DEQUEUE(Q) ha restituito il valore 15 della chiave che si trovava all'inizio della coda. Adesso la chiave all'inizio della coda è 6.

$Q.tail = 1$. Se la coda è vuota, il tentativo di rimuovere un elemento dalla coda provoca un underflow. Se $Q.head = Q.tail + 1$, la coda è piena e il tentativo di inserire un elemento provoca un overflow.

Nelle nostre procedure ENQUEUE e DEQUEUE abbiamo omesso i controlli degli errori di underflow e overflow. (L'Esercizio 10.1-4 vi chiederà di aggiungere il codice che controlla queste due condizioni di errore.) Lo pseudocodice suppone che $n = Q.length$.

```
ENQUEUE(Q, x)
1  Q[Q.tail] = x
2  if Q.tail == Q.length
3       Q.tail = 1
4  else Q.tail = Q.tail + 1
```

```
DEQUEUE(Q)
1  x = Q[Q.head]
2  if Q.head == Q.length
3       Q.head = 1
4  else Q.head = Q.head + 1
5  return x
```

La Figura 10.2 illustra gli effetti delle operazioni ENQUEUE e DEQUEUE. Ogni operazione richiede un tempo $O(1)$.

Esercizi

10.1-1

Utilizzando La Figura 10.1 come modello, illustrate il risultato di ogni operazione nella sequenza PUSH$(S, 4)$, PUSH$(S, 1)$, PUSH$(S, 3)$, POP(S), PUSH$(S, 8)$ e POP(S) su uno stack S inizialmente vuoto memorizzato nell'array $S[1\,..\,6]$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

10.1-2

Spiegate come implementare due stack in un array $A[1\,..\,n]$ in modo da non provocare overflow in nessuno dei due stack, a meno che il numero totale di elementi nei due stack non sia n. Le operazioni PUSH e POP dovrebbero essere eseguite nel tempo $O(1)$.

10.1-3
Utilizzando la Figura 10.2 come modello, illustrate il risultato di ogni operazione nella sequenza ENQUEUE$(Q, 4)$, ENQUEUE$(Q, 1)$, ENQUEUE$(Q, 3)$, DEQUEUE(Q), ENQUEUE$(Q, 8)$ e DEQUEUE(Q) su una coda Q inizialmente vuota memorizzata nell'array $Q[1 \,..\, 6]$.

10.1-4
Riscrivete ENQUEUE e DEQUEUE per rilevare le condizioni di underflow e overflow di una coda.

10.1-5
Mentre uno stack consente l'inserimento e la cancellazione di elementi in una sola estremità e una coda consente l'inserimento in una estremità e la cancellazione nell'altra estremità, una ***coda doppia*** o ***deque*** (double-ended queue) permette di inserire e cancellare elementi in entrambe le estremità. Scrivete quattro procedure con tempo $O(1)$ per inserire e cancellare elementi in entrambe le estremità di una coda doppia realizzata con un array.

10.1-6
Spiegate come implementare una coda utilizzando due stack. Analizzate il tempo di esecuzione delle operazioni sulla coda.

10.1-7
Spiegate come implementare uno stack utilizzando due code. Analizzate il tempo di esecuzione delle operazioni sullo stack.

10.2 Liste concatenate

Una ***lista concatenata*** è una struttura dati i cui oggetti sono disposti in ordine lineare. Diversamente da un array in cui l'ordine lineare è determinato dagli indici dell'array, l'ordine in una lista concatenata è determinato da un puntatore in ogni oggetto. Le liste concatenate sono una rappresentazione semplice e flessibile degli insiemi dinamici; supportano (anche se in modo non necessariamente efficiente) tutte le operazioni elencate a pagina 190.

Come illustra la Figura 10.3, ogni elemento di una ***lista doppiamente concatenata*** L è un oggetto con un attributo chiave *key* e altri due attributi puntatori: *next* e *prev*. L'oggetto può anche contenere altri dati satelliti. Dato un elemento x nella lista, $x.next$ punta al suo successore nella lista concatenata, mentre $x.prev$ punta al suo predecessore. Se $x.prev = \text{NIL}$, l'elemento x non ha un predecessore e quindi è il primo elemento della lista, detto ***testa*** o ***head*** della lista. Se $x.next = \text{NIL}$, l'elemento x non ha un successore e quindi è l'ultimo elemento della lista, che è detto ***coda*** o ***tail*** della lista. Un attributo $L.head$ punta al primo elemento della lista. Se $L.head = \text{NIL}$, la lista è vuota.

Una lista può avere varie forme: può essere singolarmente o doppiamente concatenata, ordinata oppure no, circolare oppure no. Se una lista è ***singolarmente concatenata***, omettiamo il puntatore *prev* in ogni elemento. Se una lista è ***ordinata***, l'ordine lineare della lista corrisponde all'ordine lineare delle chiavi memorizzate negli elementi della lista; l'elemento minimo è la testa della lista e l'elemento massimo è la coda della lista. Una lista può essere ***non ordinata***; gli elementi di questa lista possono presentarsi in qualsiasi ordine. In una lista ***circolare***, il puntatore *prev* della testa della lista punta alla coda e il puntatore *next* della coda della lista punta alla testa. Pertanto, la lista può essere vista come un anello di elementi.

Acquistato da Michele Michele su Webster il 2022-07-07 23:13. Numero Ordine Libreria: 19503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

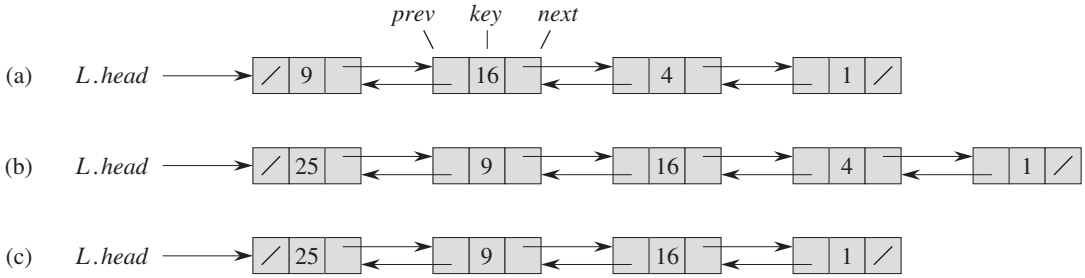


Figura 10.3 **(a)** Una lista doppiamente concatenata L che rappresenta l'insieme dinamico $\{1, 4, 9, 16\}$. Ogni elemento della lista è un oggetto con attributi per la chiave e per i puntatori (rappresentati da frecce) agli oggetti precedente e successivo. L'attributo *next* della coda e l'attributo *prev* della testa sono NIL (questo valore speciale è indicato da una barra inclinata). L'attributo $L.head$ punta al primo elemento della lista. **(b)** Dopo l'esecuzione di LIST-INSERT(L, x), dove $x.key = 25$, la lista concatenata ha un nuovo oggetto con chiave 25 come nuova testa. Questo nuovo oggetto punta alla vecchia testa con chiave 9. **(c)** Il risultato della successiva chiamata LIST-DELETE(L, x), dove x punta all'oggetto con chiave 4.

Nella parte restante di questo paragrafo faremo l'ipotesi che le liste con le quali operiamo siano non ordinate e doppiamente concatenate.

Ricerca in una lista concatenata

La procedura LIST-SEARCH(L, k) trova il primo elemento con la chiave k nella lista L mediante una semplice ricerca lineare, restituendo un puntatore a questo elemento. Se nessun oggetto con la chiave k è presente nella lista, allora viene restituito il valore NIL. Per la lista concatenata nella Figura 10.3(a), la chiamata LIST-SEARCH$(L, 4)$ restituisce un puntatore al terzo elemento e la chiamata LIST-SEARCH$(L, 7)$ restituisce NIL.

```
LIST-SEARCH(L, k)
1  x = L.head
2  while x ≠ NIL and x.key ≠ k
3       x = x.next
4  return x
```

Per effettuare una ricerca in una lista di n oggetti, la procedura LIST-SEARCH impiega il tempo $\Theta(n)$ nel caso peggiore, in quanto potrebbe essere necessario esaminare l'intera lista.

Inserire un elemento in una lista concatenata

Dato un elemento x il cui attributo *key* sia stato già impostato, la procedura LIST-INSERT inserisce x davanti alla lista concatenata, come illustra la Figura 10.3(b).

```
LIST-INSERT(L, x)
1  x.next = L.head
2  if L.head ≠ NIL
3       L.head.prev = x
4  L.head = x
5  x.prev = NIL
```

(Ricordiamo che la nostra notazione per gli attributi può essere usata in cascata, quindi $L.head.prev$ indica l'attributo *prev* dell'oggetto puntato da $L.head$.) Il tempo di esecuzione di LIST-INSERT con una lista di n elementi è $O(1)$.

Acquistato da Michele Michele su Webster il 2022-07-07 13:37:43 Numero ordine libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

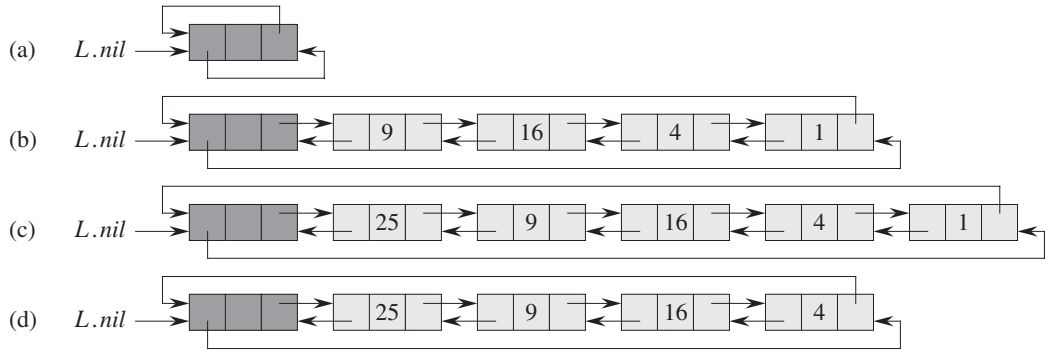


Figura 10.4 Una lista circolare doppiamente concatenata con una sentinella. La sentinella $L.nil$ è posta fra la testa e la coda. L'attributo $L.head$ non è più richiesto, in quanto possiamo accedere alla testa della lista tramite $L.nil.next$. **(a)** Una lista vuota. **(b)** La lista concatenata della Figura 10.3(a) con la chiave 9 in testa e la chiave 1 in coda. **(c)** La lista dopo l'esecuzione di LIST-INSERT$'(L, x)$, dove $x.key = 25$. Il nuovo oggetto diventa la testa della lista. **(d)** La lista dopo l'eliminazione dell'oggetto con chiave 1. La nuova coda è l'oggetto con chiave 4.

Cancellare un elemento da una lista concatenata

La procedura LIST-DELETE rimuove un elemento x da una lista concatenata L. Deve ricevere un puntatore a x; poi elimina x dalla lista aggiornando i puntatori. Per cancellare un elemento con una data chiave, dobbiamo prima chiamare LIST-SEARCH per ottenere un puntatore all'elemento.

```
LIST-DELETE(L, x)
1  if x.prev ≠ NIL
2      x.prev.next = x.next
3  else L.head = x.next
4  if x.next ≠ NIL
5      x.next.prev = x.prev
```

La Figura 10.3(c) mostra come viene cancellato un elemento da una lista concatenata. LIST-DELETE viene eseguita nel tempo $O(1)$, ma se vogliamo eliminare un elemento con una data chiave, occorre un tempo $\Theta(n)$ nel caso peggiore, perché dobbiamo prima chiamare LIST-SEARCH per trovare l'elemento.

Sentinelle

Il codice di LIST-DELETE sarebbe più semplice se potessimo ignorare le condizioni al contorno nella testa e nella coda della lista.

```
LIST-DELETE'(L, x)
1  x.prev.next = x.next
2  x.next.prev = x.prev
```

Una ***sentinella*** è un oggetto fittizio che ci consente di semplificare le condizioni al contorno. Per esempio, supponiamo di fornire alla lista L un oggetto $L.nil$ che rappresenta NIL, ma ha tutti gli attributi degli altri elementi della lista. Ogni volta che troviamo un riferimento a NIL nel codice della lista, sostituiamolo con un riferimento alla sentinella $L.nil$. Come illustra la Figura 10.4, questo trasforma una normale lista doppiamente concatenata in una ***lista circolare doppiamente concatenata con una sentinella***, dove la sentinella $L.nil$ è posta fra la testa e la coda; l'attributo $L.nil.next$ punta alla testa della lista, mentre $L.nil.prev$ punta alla coda. Analogamente, l'attributo *next* della coda e l'attributo *prev* della testa

Acquistato da Michele Michelozzi Webster ID 2022-07-07 23:12 Numero Ordine Libreria 1895201 Licenza 3207707 © Copyright © 2022 McGraw-Hill Education (Italy)

puntano entrambi a $L.nil$. Poiché $L.nil.next$ punta alla testa, possiamo eliminare del tutto l'attributo $L.head$, sostituendo ogni suo riferimento con un riferimento a $L.nil.next$. La Figura 10.4(a) mostra che una lista vuota è formata soltanto dalla sentinella, con $L.nil.next$ e $L.nil.prev$ impostati entrambi a $L.nil$.

Il codice di LIST-SEARCH è uguale al precedente, a parte la modifica dei riferimenti a NIL e $L.head$.

```
LIST-SEARCH′(L, k)
1  x = L.nil.next
2  while x ≠ L.nil and x.key ≠ k
3       x = x.next
4  return x
```

Utilizziamo la procedura di due righe LIST-DELETE′ vista precedentemente per cancellare un elemento dalla lista. La seguente procedura inserisce un elemento nella lista:

```
LIST-INSERT′(L, x)
1  x.next = L.nil.next
2  L.nil.next.prev = x
3  L.nil.next = x
4  x.prev = L.nil
```

La Figura 10.4 illustra gli effetti di LIST-INSERT′ e LIST-DELETE′ su una lista di questo tipo.

Le sentinelle raramente abbassano i limiti asintotici sui tempi delle operazioni con le strutture dati, ma possono ridurre i fattori costanti. Il vantaggio derivante dall'uso delle sentinelle all'interno dei cicli, di solito, riguarda la chiarezza del codice, più che la velocità; il codice per la lista concatenata, per esempio, si semplifica se si usano le sentinelle, ma si risparmia soltanto un tempo $O(1)$ nelle procedure LIST-INSERT′ e LIST-DELETE′. In altre situazione, tuttavia, l'uso delle sentinelle aiuta a compattare il codice in un ciclo, riducendo così il coefficiente dei termini in n o n^2 nel tempo di esecuzione.

Le sentinelle non dovrebbero essere utilizzate in modo indiscriminato. Se ci sono molte liste piccole, lo spazio extra in memoria utilizzato dalle loro sentinelle potrebbe costituire un notevole spreco di memoria. In questo libro utilizziamo le sentinelle soltanto se semplificano davvero il codice.

Esercizi

10.2-1

L'operazione INSERT per gli insiemi dinamici può essere implementata per una lista singolarmente concatenata nel tempo $O(1)$? E l'operazione DELETE?

10.2-2

Implementate uno stack utilizzando una lista singolarmente concatenata L. Le operazioni PUSH e POP dovrebbero richiedere ancora il tempo $O(1)$.

Acquistato da Michele Michele su Webster il 2022-07-07 20:12 Numero Ordine Libreria: 195303515 2207 07 © Copyright © 2022, McGraw-Hill Education (Italy)

10.2-3

Implementate una coda utilizzando una lista singolarmente concatenata L. Le operazioni ENQUEUE e DEQUEUE dovrebbero richiedere ancora il tempo $O(1)$.

10.2-4

Come detto in precedenza, ogni iterazione del ciclo nella procedura LIST-SEARCH′ richiede due test: uno per $x \neq L.nil$ e l'altro per $x.key \neq k$. Spiegate come eliminare il test per $x \neq L.nil$ in ogni iterazione.

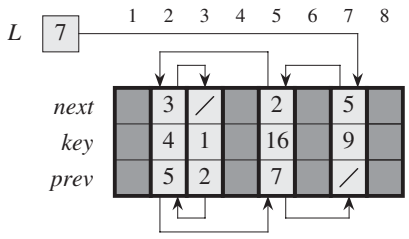


Figura 10.5 La lista della Figura 10.3(a) rappresentata dagli array *key*, *next* e *prev*. Ogni segmento verticale degli array rappresenta un singolo oggetto. I puntatori memorizzati corrispondono agli indici degli array indicati in alto; le frecce mostrano come interpretarli. Le posizioni degli oggetti su sfondo grigio chiaro contengono gli elementi della lista. La variabile L contiene l'indice della testa della lista.

10.2-5

Implementate le operazioni per i dizionari INSERT, DELETE e SEARCH utilizzando liste circolari singolarmente concatenate. Quali sono i tempi di esecuzione delle vostre procedure?

10.2-6

L'operazione UNION per gli insiemi dinamici richiede due insiemi disgiunti S_1 e S_2 come input e restituisce un insieme $S = S_1 \cup S_2$ che è formato da tutti gli elementi di S_1 e S_2. Gli insiemi S_1 e S_2, di solito, vengono distrutti dall'operazione. Spiegate come realizzare UNION nel tempo $O(1)$ utilizzando una lista appropriata come struttura dati.

10.2-7

Scrivete una procedura non ricorsiva con tempo $\Theta(n)$ che inverte una lista singolarmente concatenata di n elementi. La procedura dovrebbe utilizzare non più di una quantità costante di memoria oltre a quella richiesta per la lista stessa.

10.2-8 ★

Spiegate come implementare una lista doppiamente concatenata utilizzando un solo puntatore $x.np$ per ogni elemento, anziché i due puntatori *next* e *prev*. Supponete che tutti i valori del puntatore siano interpretati come interi di k bit e definite $x.np$ come $x.np = x.next$ XOR $x.prev$, l'operatore "or esclusivo" di k bit fra $x.next$ e $x.prev$ (il valore NIL è rappresentato da 0). Controllate di avere descritto le informazioni necessarie per accedere alla testa della lista. Spiegate come implementare le procedure SEARCH, INSERT e DELETE per tale lista. Spiegate inoltre come invertire la lista nel tempo $O(1)$.

10.3 Implementare puntatori e oggetti

Come possiamo implementare puntatori e oggetti in quei linguaggi che non li prevedono? In questo paragrafo descriveremo due tecniche per implementare alcune strutture dati concatenate senza utilizzare un esplicito tipo di dato puntatore. Sintetizzeremo oggetti e puntatori tramite array e indici di array.

Rappresentazione di oggetti con più array

È possibile rappresentare una collezione di oggetti che hanno gli stessi attributi utilizzando un array per ogni attributo. Per esempio, la Figura 10.5 illustra come possiamo implementare la lista concatenata della Figura 10.3(a) con tre array. L'array *key* contiene i valori delle chiavi che si trovano correntemente nell'insieme dinamico; i puntatori sono memorizzati negli array *next* e *prev*. Per un dato indice x dell'array, $key[x]$, $next[x]$ e $prev[x]$ rappresentano un oggetto della lista concatenata. Questo significa che un puntatore x è semplicemente un indice comune agli array *key*, *next* e *prev*.

Acquistato da Michele Michele su Webster # 2022-07-07 23:12 Numero Ordine Libreria: 2899503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

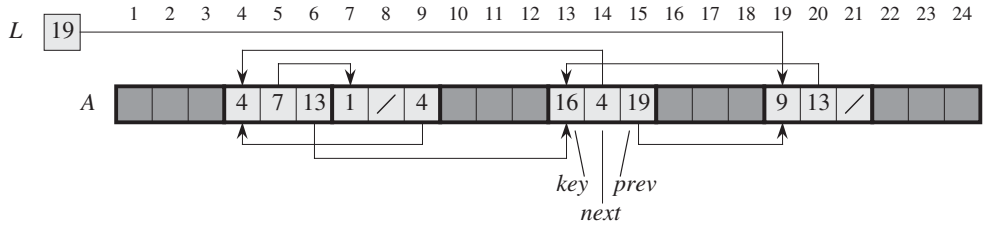


Figura 10.6 La lista concatenata delle Figure 10.3(a) e 10.5 rappresentata con un singolo array A. Ogni elemento della lista è un oggetto che occupa un sottoarray contiguo di lunghezza 3 all'interno dell'array. Il puntatore di un oggetto è l'indice del primo elemento dell'oggetto. I tre attributi *key*, *next* e *prev* corrispondono, rispettivamente, agli incrementi 0, 1 e 2 di tale indice. Gli oggetti che contengono gli elementi della lista sono illustrati con uno sfondo grigio chiaro; le frecce indicano l'ordinamento della lista.

Nella Figura 10.3(a) l'oggetto con chiave 4 segue l'oggetto con chiave 16 nella lista concatenata. Nella Figura 10.5 la chiave 4 appare in $key[2]$ e la chiave 16 appare in $key[5]$, quindi abbiamo $next[5] = 2$ e $prev[2] = 5$. La costante NIL che appare nell'attributo *next* della coda e nell'attributo *prev* della testa, di solito, viene rappresentata con un intero (come 0 o -1) che non può essere scambiato per un indice di array. Una variabile L contiene l'indice della testa della lista.

Rappresentazione di oggetti con un solo array

Le parole nella memoria di un calcolatore, tipicamente, sono indirizzate da numeri interi compresi fra 0 e $M - 1$, dove M è un intero sufficientemente grande. In molti linguaggi di programmazione un oggetto occupa un insieme contiguo di locazioni nella memoria del calcolatore. Un puntatore è semplicemente l'indirizzo della prima locazione di memoria dell'oggetto; le altre locazioni di memoria all'interno dell'oggetto possono essere indicizzate incrementando il puntatore di una quantità prefissata (offset).

Per implementare gli oggetti, possiamo adottare la stessa strategia anche negli ambienti di programmazione che non prevedono esplicitamente un tipo di dato puntatore. Per esempio, la Figura 10.6 mostra come un singolo array A possa essere utilizzato per memorizzare la lista concatenata delle Figure 10.3(a) e 10.5. Un oggetto occupa un sottoarray contiguo $A[j\,..\,k]$. L'indice j è il puntatore all'oggetto; ogni attributo dell'oggetto corrisponde a un incremento dell'indice nell'intervallo fra 0 e $k - j$. Nella Figura 10.6 gli offset che corrispondono a *key*, *next* e *prev* sono, rispettivamente, 0, 1 e 2. Per leggere il valore di $i.prev$, dato un puntatore i, aggiungiamo l'incremento 2 al valore i del puntatore, ottenendo $A[i + 2]$.

La rappresentazione con un solo array è flessibile perché consente di memorizzare nello stesso array oggetti di lunghezza diversa. Il problema di gestire tale collezione eterogenea di oggetti è più difficile del problema di gestire una collezione omogenea, dove tutti gli oggetti hanno gli stessi attributi. Poiché la maggior parte delle strutture dati che esamineremo sono composte da elementi omogenei, sarà sufficiente per i nostri scopi utilizzare la rappresentazione degli oggetti con più array.

Acquistato da Michele Michele su Webster il 2022-[illegible] Vietata la riproduzione, anche parziale, con qualsiasi mezzo effettuata (copia compresa).

Allocare e rilasciare gli oggetti

Per inserire una chiave in un insieme dinamico rappresentato da una lista doppiamente concatenata, dobbiamo allocare un puntatore a un oggetto correntemente inutilizzato nella rappresentazione della lista concatenata. Quindi, è utile gestire

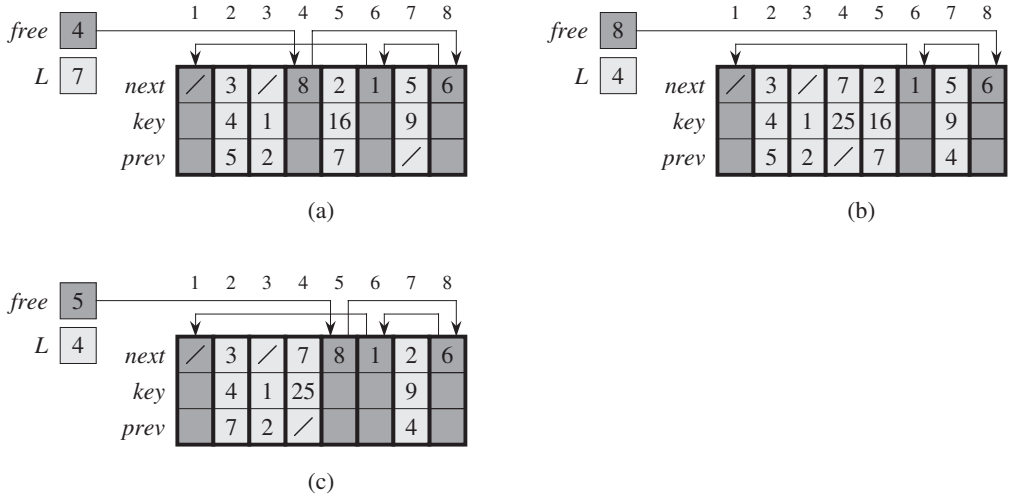


Figura 10.7 L'effetto delle procedure ALLOCATE-OBJECT e FREE-OBJECT. **(a)** La lista della Figura 10.5 (su sfondo chiaro) e una free list (su sfondo scuro). Le frecce indicano la struttura della free list. **(b)** Il risultato della chiamata ALLOCATE-OBJECT() (che restituisce l'indice 4), dell'assegnazione del valore 25 a *key*[4] e della chiamata LIST-INSERT($L, 4$). La nuova testa della free list è l'oggetto 8, che prima era *next*[4] nella free list. **(c)** Dopo l'esecuzione di LIST-DELETE($L, 5$), viene chiamata la procedura FREE-OBJECT(5). L'oggetto 5 diventa la nuova testa della free list e l'oggetto 8 lo segue nella free list.

lo spazio in memoria degli oggetti che non sono correntemente utilizzati nella rappresentazione della lista concatenata, in modo da potere allocare nuovi oggetti. In alcuni sistemi, esistono dei meccanismi automatici, detti ***garbage collector*** (spazzini della memoria), che hanno la responsabilità di determinare quali oggetti sono inutilizzati. Molte applicazioni, però, sono sufficientemente semplici da potersi assumere anche il compito di restituire un oggetto inutilizzato a un gestore della memoria. Esaminiamo adesso il problema di allocare e rilasciare (o deallocare) degli oggetti omogenei utilizzando una lista doppiamente concatenata rappresentata da più array.

Supponiamo che gli array nella rappresentazione con più array abbiano lunghezza m e che a un certo istante l'insieme dinamico contenga $n \leq m$ elementi. Allora n oggetti rappresentano gli elementi che si trovano correntemente nell'insieme dinamico e i restanti $m - n$ oggetti sono ***liberi***; gli oggetti liberi possono essere utilizzati per rappresentare gli elementi da inserire in futuro nell'insieme dinamico.

Manteniamo gli oggetti liberi in una lista singolarmente concatenata, che chiameremo ***free list***. Questa lista usa soltanto l'array *next*, che contiene i puntatori *next* all'interno della lista. La testa della free list è mantenuta nella variabile globale *free*. Quando l'insieme dinamico rappresentato dalla lista concatenata L non è vuoto, la free list può essere intrecciata con la lista L, come illustra la Figura 10.7. Notate che ogni oggetto nella rappresentazione sta o nella lista L o nella free list, ma non in entrambe le liste.

Acquistato da Michele Mostarda (michele.mostarda@gmail.com) - Vietata la copia - Copyright © 2010 The McGraw-Hill Education (Italy)

La free list si comporta come uno stack: il prossimo oggetto allocato è l'ultimo che è stato liberato. Possiamo adattare le operazioni su stack PUSH e POP per implementare, rispettivamente, le procedure per allocare e rilasciare gli oggetti. Supponiamo che la variabile globale *free* utilizzata nelle seguenti procedure punti al primo elemento della free list.

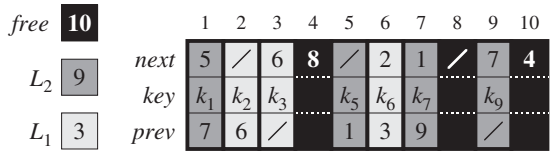


Figura 10.8 Due liste concatenate, L_1 (su sfondo chiaro) e L_2 (su sfondo scuro), intrecciate con una free list (su sfondo più scuro).

```
ALLOCATE-OBJECT()
1  if free == NIL
2       error "Spazio esaurito!"
3  else x = free
4       free = x.next
5       return x
```

```
FREE-OBJECT(x)
1  x.next = free
2  free = x
```

La free list inizialmente contiene tutti gli n oggetti non allocati. Quando lo spazio nella free list è esaurito, la procedura ALLOCATE-OBJECT segnala un errore. È possibile utilizzare un'unica free list a servizio di più liste concatenate. La Figura 10.8 illustra due liste concatenate e una free list intrecciate tramite gli array *key*, *next* e *prev*.

Le due procedure vengono eseguite nel tempo $O(1)$; questo rende molto conveniente il loro uso. Possono essere modificate per operare con qualsiasi collezione omogenea di oggetti, utilizzando uno degli attributi dell'oggetto come attributo *next* nella free list.

Esercizi

10.3-1

Disegnate uno schema della sequenza $\langle 13, 4, 8, 19, 5, 11 \rangle$ memorizzata come una lista doppiamente concatenata utilizzando la rappresentazione con più array. Fate lo stesso per la rappresentazione con un solo array.

10.3-2

Scrivete le procedure ALLOCATE-OBJECT e FREE-OBJECT per una collezione omogenea di oggetti implementata con la rappresentazione con un solo array.

10.3-3

Perché non occorre assegnare o riassegnare un valore agli attributi *prev* degli oggetti nell'implementazione delle procedure ALLOCATE-OBJECT e FREE-OBJECT?

10.3-4

Spesso è preferibile mantenere compatta la memorizzazione di tutti gli elementi di una lista doppiamente concatenata, utilizzando, per esempio, le prime m posizioni nella rappresentazione con più array (è questo il caso di un ambiente di calcolo con memoria virtuale paginata). Spiegate come implementare le procedure ALLOCATE-OBJECT e FREE-OBJECT in modo che la rappresentazione sia compatta. Supponete che non ci siano puntatori a elementi della lista concatenata all'esterno della lista stessa (*suggerimento:* usate l'implementazione di uno stack mediante array).

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

10.3-5
Sia L una lista doppiamente concatenata di lunghezza n memorizzata negli array *key*, *prev* e *next* di lunghezza m. Supponete che questi array siano gestiti dalle procedure ALLOCATE-OBJECT e FREE-OBJECT che mantengono una free list F doppiamente concatenata. Supponete inoltre che n degli m elementi siano nella lista L e $m - n$ elementi siano nella free list. Scrivete una procedura COMPACTIFY-LIST(L, F) che, date la lista L e la free list F, sposta gli elementi in L in modo che occupino le posizioni $1, 2, \ldots, n$ dell'array e aggiusta la free list F in modo che essa resti corretta, occupando le posizioni dell'array $n + 1, n + 2, \ldots, m$. Il tempo di esecuzione della procedura dovrebbe essere $\Theta(n)$; la procedura dovrebbe utilizzare soltanto una quantità costante di spazio extra. Dimostrate la correttezza della procedura.

10.4 Rappresentazione di alberi radicati

I metodi per rappresentare le liste che abbiamo descritto nel precedente paragrafo si estendono a qualsiasi struttura dati omogenea. In questo paragrafo, analizzeremo specificatamente il problema di rappresentare gli alberi radicati mediante strutture dati concatenate. Esamineremo prima gli alberi binari e poi un metodo per gli alberi radicati in cui i nodi possono avere un numero arbitrario di figli.

Rappresentiamo i singoli nodi di un albero con un oggetto. Analogamente alle liste concatenate, supponiamo che ogni nodo contenga un attributo *key*. I restanti attributi che ci interessano sono puntatori ad altri nodi, che variano a seconda del tipo di albero.

Alberi binari

Come illustra la Figura 10.9, utilizziamo gli attributi p, *left* e *right* per memorizzare i puntatori al padre, al figlio sinistro e al figlio destro di ogni nodo in un albero binario T. Se $x.p = \text{NIL}$, allora x è la radice. Se il nodo x non ha un figlio sinistro, allora $x.left = \text{NIL}$; lo stesso vale per il figlio destro. L'attributo $T.root$ punta alla radice dell'intero albero T. Se $T.root = \text{NIL}$, allora l'albero è vuoto.

Alberi con grado di ramificazione illimitato

Lo schema per rappresentare un albero binario può essere esteso a qualsiasi classe di alberi in cui il numero di figli di ogni nodo sia al massimo una qualche costante k: sostituiamo gli attributi *left* e *right* con $child_1, child_2, \ldots, child_k$. Questo schema non funziona più se il numero di figli di un nodo è illimitato, perché non sappiamo in anticipo quanti attributi (quanti array nella rappresentazione con più array) allocare. Inoltre, anche nel caso in cui il numero di figli k fosse limitato da una grande costante, ma la maggior parte dei nodi avesse un piccolo numero di figli, potremmo sprecare una notevole quantità di memoria.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: IB9503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

Fortunatamente, esiste uno schema geniale che usa gli alberi binari per rappresentare gli alberi con un numero arbitrario di figli. Questo schema ha il vantaggio di utilizzare soltanto uno spazio $O(n)$ per qualsiasi albero radicato di n nodi. La ***rappresentazione figlio-sinistro fratello-destro*** è illustrata nella Figura 10.10. Come prima, ogni nodo contiene un puntatore p al padre e $T.root$ punta alla radice

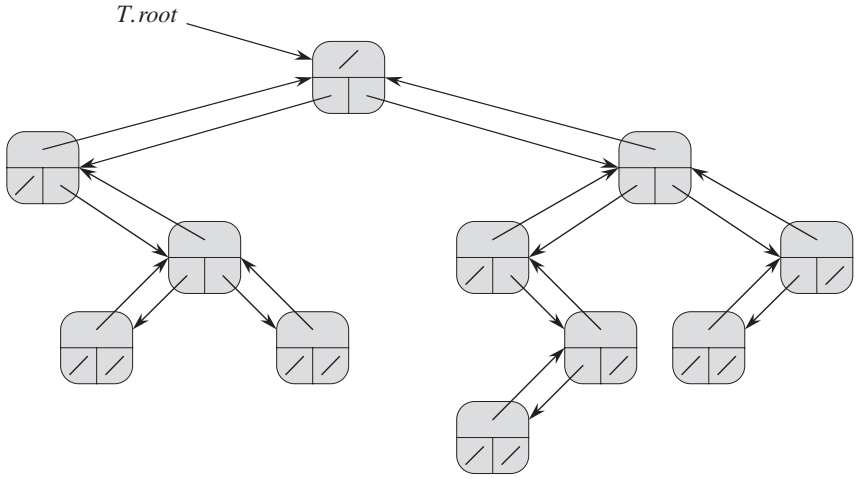
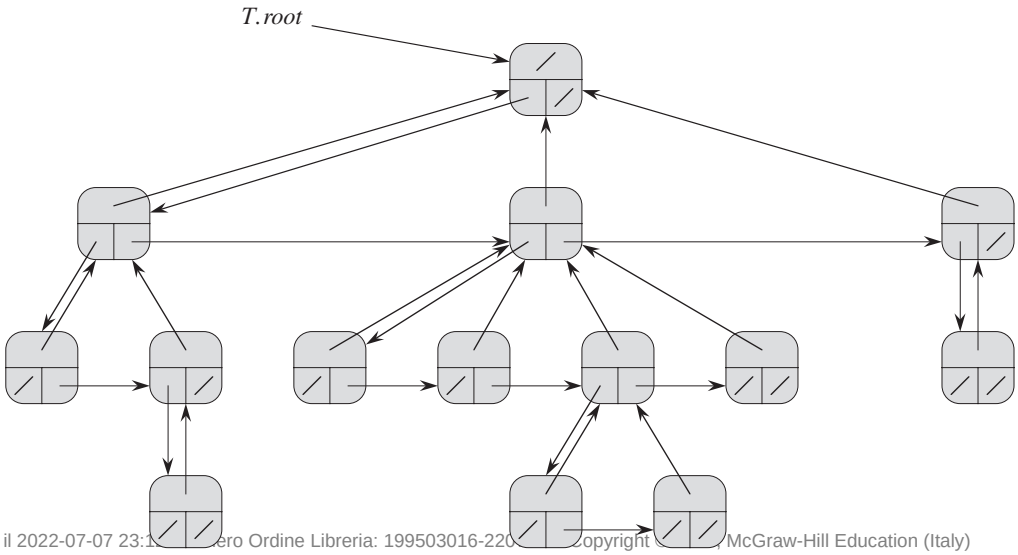


Figura 10.9 La rappresentazione di un albero binario T. Ogni nodo x ha gli attributi $p[x]$ (in alto), *left*$[x]$ (in basso a sinistra) e *right*$[x]$ (in basso a destra). Gli attributi *key* non sono indicati.

dell'albero T. Anziché avere un puntatore a ciascuno dei suoi figli, però, ogni nodo x ha soltanto due puntatori:

1. $x.$*left-child* punta al figlio più a sinistra del nodo x.
2. $x.$*right-sibling* punta al fratello di x immediatamente a destra.

Se il nodo x non ha figli, allora $x.$*left-child* $=$ NIL. Se il nodo x è il figlio più a destra di suo padre, allora $x.$*right-sibling* $=$ NIL.



Acquistato da Michele Michele su Webster il 2022-07-07 23:1 ero Ordine Libreria: 199503016-220 Copyright © McGraw-Hill Education (Italy)

Figura 10.10 La rappresentazione figlio-sinistro, fratello-destro di un albero T. Ogni nodo x ha gli attributi $p[x]$ (in alto), *left-child*$[x]$ (in basso a sinistra) e *right-sibling*$[x]$ (in basso a destra). Gli attributi *key* non sono indicati.

Altre rappresentazioni di alberi

Gli alberi radicati, a volte, vengono rappresentati in altri modi. Nel Capitolo 6, per esempio, abbiamo rappresentato un heap (che si basa su un albero binario completo) mediante un solo array più l'indice dell'ultimo elemento. Gli alberi presentati nel Capitolo 21 sono attraversati soltanto in direzione della radice, quindi sono presenti soltanto i puntatori al padre; mancano i puntatori ai figli. Esistono molti altri schemi. Lo schema migliore dipende dall'applicazione.

Esercizi

10.4-1

Disegnate l'albero binario con radice di indice 6 che è rappresentato dai seguenti attributi.

indice	*key*	*left*	*right*
1	12	7	3
2	15	8	NIL
3	4	10	NIL
4	10	5	9
5	2	NIL	NIL
6	18	1	4
7	7	NIL	NIL
8	14	6	2
9	21	NIL	NIL
10	5	NIL	NIL

10.4-2

Scrivete una procedura ricorsiva con tempo $O(n)$ che, dato un albero binario di n nodi, stampa la chiave di ogni nodo dell'albero.

10.4-3

Scrivete una procedura non ricorsiva con tempo $O(n)$ che, dato un albero binario di n nodi, stampa la chiave di ogni nodo dell'albero. Utilizzate uno stack come struttura dati ausiliaria.

10.4-4

Scrivete una procedura con tempo $O(n)$ che stampa tutte le chiavi di un albero radicato arbitrario di n nodi, quando l'albero è memorizzato utilizzando la rappresentazione figlio-sinistro fratello-destro.

10.4-5 ⋆

Scrivete una procedura non ricorsiva con tempo $O(n)$ che, dato un albero binario di n nodi, stampa la chiave di ogni nodo. La procedura non deve utilizzare più di una quantità costante di memoria all'esterno dell'albero stesso e non deve modificare l'albero, neanche solo temporaneamente, durante l'esecuzione.

10.4-6 ⋆

La rappresentazione figlio-sinistro fratello-destro di un albero radicato arbitrario usa tre puntatori in ogni nodo: *left-child*, *right-sibling* e *parent*. Partendo da un nodo qualsiasi, il padre di questo nodo può essere raggiunto e identificato in un tempo costante e tutti i suoi figli possono essere raggiunti e identificati in un tempo lineare nel numero dei figli. Spiegate come utilizzare soltanto due puntatori e un valore booleano in ogni nodo in modo che sia il padre di un nodo sia tutti i suoi figli possano essere raggiunti e identificati in un tempo lineare nel numero dei figli.

Acquistato da Michele Michele su Webster | 2022-07-07 23:12 | Numero Ordine Libreria: 1595050718 | 2021 | © Copyright © 2022, McGraw-Hill Education (Italy)

Problemi

10-1 Confronti fra liste

Per ciascuno dei quattro tipi di liste della seguente tabella, qual è il tempo di esecuzione asintotico nel caso peggiore per ciascuna delle operazioni su un insieme dinamico?

	Non ordinata, singolarmente concatenata	Ordinata, singolarmente concatenata	Non ordinata, doppiamente concatenata	Ordinata, doppiamente concatenata
SEARCH(L, k)				
INSERT(L, x)				
DELETE(L, x)				
SUCCESSOR(L, x)				
PREDECESSOR(L, x)				
MINIMUM(L)				
MAXIMUM(L)				

10-2 Heap riunibili con liste concatenate

Un ***heap riunibile*** (mergeable heap) supporta le seguenti operazioni: MAKE-HEAP (che crea un heap riunibile vuoto), INSERT, MINIMUM, EXTRACT-MIN e UNION.[1] Spiegate come implementare gli heap riunibili utilizzando le liste concatenate in ciascuno dei seguenti casi. Provate a rendere ogni operazione più efficiente possibile. Analizzate il tempo di esecuzione di ogni operazione in funzione della dimensione degli insiemi dinamici su cui agisce l'operazione.

a. Le liste sono ordinate.

b. Le liste non sono ordinate.

c. Le liste non sono ordinate e gli insiemi dinamici da riunire sono disgiunti.

10-3 Ricerca in una lista ordinata compatta

L'Esercizio 10.3-4 chiedeva come mantenere compatta una lista di n elementi nelle prime n posizioni di un array. Supponete che tutte le chiavi siano distinte e che la lista compatta sia anche ordinata, ovvero $key[i] < key[next[i]]$ per ogni $i = 1, 2, \ldots, n$ tale che $next[i] \neq$ NIL. Sotto questi ipotesi, dimostrate che il seguente algoritmo randomizzato può essere utilizzato per effettuare delle ricerche nella lista nel tempo atteso $O(\sqrt{n})$.

[1] Poiché, secondo la nostra definizione, un heap riunibile supporta le operazioni MINIMUM e EXTRACT-MIN, possiamo fare riferimento a questo heap anche con il termine ***min-heap riunibile***. D'altra parte, se un heap riunibile supportasse le operazioni MAXIMUM ed EXTRACT-MAX, potremmo chiamarlo ***max-heap riunibile***.

Acquistato da Michele Michele su Webster il 2022-07-20 12:12 Numero Ordine Inarcet 14253381 - 2022 The McGraw-Hill Companies, S.r.l. - Copyright © 2022 - McGraw-Hill Education (Italy)

```
COMPACT-LIST-SEARCH(L, n, k)
 1  i = L
 2  while i ≠ NIL and key[i] < k
 3       j = RANDOM(1, n)
 4       if key[i] < key[j] and key[j] ≤ k
 5            i = j
 6            if key[i] == k
 7                 return i
 8       i = next[i]
 9  if i == NIL or key[i] > k
10       return NIL
11  else return i
```

Se ignoriamo le righe 3–7 della procedura, abbiamo un normale algoritmo di ricerca in una lista concatenata ordinata, dove l'indice i punta, a turno, a ogni posizione della lista. La ricerca termina quando l'indice i "va oltre" la fine della lista o quando $key[i] \geq k$. Nel secondo caso, se $key[i] = k$, chiaramente abbiamo trovato una chiave con il valore k. Se, invece, $key[i] > k$, allora non troveremo mai una chiave con il valore k e, quindi, bisogna interrompere la ricerca.

Le righe 3–7 tentano di saltare su una posizione j scelta a caso. Tale salto è vantaggioso se $key[j]$ è maggiore di $key[i]$ e non maggiore di k; nel qual caso, j indica una posizione nella lista che i avrebbe raggiunto successivamente durante una normale ricerca. Poiché la lista è compatta, sappiamo che qualsiasi scelta di j tra 1 e n indica qualche oggetto della lista, non un elemento della free list.

Anziché analizzare direttamente le prestazioni di COMPACT-LIST-SEARCH, esamineremo un algoritmo simile, COMPACT-LIST-SEARCH$'$, che esegue due cicli distinti. Questo algoritmo richiede un parametro addizionale t, che determina un limite superiore sul numero di iterazioni del primo ciclo.

```
COMPACT-LIST-SEARCH'(L, n, k, t)
 1  i = L
 2  for q = 1 to t
 3       j = RANDOM(1, n)
 4       if key[i] < key[j] and key[j] ≤ k
 5            i = j
 6            if key[i] == k
 7                 return i
 8  while i ≠ NIL and key[i] < k
 9       i = next[i]
10  if i == NIL or key[i] > k
11       return NIL
12  else return i
```

Per confrontare l'esecuzione degli algoritmi COMPACT-LIST-SEARCH(L, n, k) e COMPACT-LIST-SEARCH$'(L, n, k, t)$, supponete che la sequenza degli interi restituita dalle chiamate di RANDOM$(1, n)$ sia la stessa per entrambi gli algoritmi.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

a. Supponete che COMPACT-LIST-SEARCH(L, n, k) richieda t iterazioni del ciclo **while** (righe 2–8). Dimostrate che COMPACT-LIST-SEARCH$'(L, n, k, t)$ fornisce lo stesso risultato e che il numero totale di iterazioni dei due cicli **for** e **while** all'interno di COMPACT-LIST-SEARCH$'$ è almeno t.

Nella chiamata COMPACT-LIST-SEARCH$'(L, n, k, t)$, sia X_t la variabile casuale che descrive la distanza nella lista concatenata (attraverso la catena dei puntatori *next*) dalla posizione i alla chiave desiderata k, dopo t iterazioni del ciclo **for** (righe 2–7).

b. Verificate che il tempo di esecuzione atteso di COMPACT-LIST-SEARCH$'(L, n, k, t)$ può essere espresso da $O(t + \mathrm{E}[X_t])$.

c. Dimostrate la relazione $\mathrm{E}[X_t] \leq \sum_{r=1}^{n}(1 - r/n)^t$ (*suggerimento:* applicate l'equazione (C.25)).

d. Dimostrate che $\sum_{r=0}^{n-1} r^t \leq n^{t+1}/(t + 1)$.

e. Verificate che $\mathrm{E}[X_t] \leq n/(t + 1)$.

f. Dimostrate che COMPACT-LIST-SEARCH$'(L, n, k, t)$ viene eseguito nel tempo atteso $O(t + n/t)$.

g. Concludete che il tempo di esecuzione atteso di COMPACT-LIST-SEARCH è $O(\sqrt{n})$.

h. Perché bisogna supporre che tutte le chiavi siano distinte in COMPACT-LIST-SEARCH? Dimostrate che i salti casuali non necessariamente aiutano asintoticamente quando la lista contiene chiavi ripetute.

Note

Aho, Hopcroft e Ullman [6] e Knuth [210] sono eccellenti testi di riferimento per le strutture dati elementari. Molti altri testi trattano sia le strutture dati di base sia la loro implementazione in un particolare linguaggio di programmazione. Fra questi libri di testo citiamo Goodrich e Tamassia [148], Main [242], Shaffer [312] e Weiss [353, 354, 355]. Gonnet [146] ha raccolto i dati sperimentali sulle prestazioni di molte operazioni con le strutture dati.

L'origine degli stack e delle code come strutture dati dell'informatica non è chiara, in quanto le corrispondenti nozioni esistevano già nella matematica e nella pratica aziendale basata su carta prima dell'introduzione dei calcolatori digitali. Knuth [210] cita A. M. Turing in merito allo sviluppo degli stack per il collegamento delle subroutine nel 1947.

Anche le strutture dati basate sui puntatori sembrano essere un'invenzione popolare. Secondo Knuth, sembra che i puntatori siano stati utilizzati nei primi calcolatori con memorie a tamburo. Il linguaggio A-1 sviluppato da G. M. Hopper nel 1951 rappresentava le formule algebriche come alberi binari. Knuth attribuisce al linguaggio IPL-II, sviluppato nel 1956 da A. Newell, J. C. Shaw e H. A. Simon, il riconoscimento dell'importanza e la diffusione dei puntatori. Il loro linguaggio IPL-III, sviluppato nel 1957, includeva esplicite operazioni con gli stack.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Hashing 11

Molte applicazioni richiedono un insieme dinamico che supporta soltanto le operazioni di dizionario INSERT, SEARCH e DELETE. Per esempio, il compilatore che traduce un linguaggio di programmazione mantiene una tabella di simboli, nella quale le chiavi degli elementi sono stringhe arbitrarie di caratteri che denotano identificatori nel linguaggio.

Una tavola hash è una struttura dati efficace per implementare i dizionari. Sebbene la ricerca di un elemento in una tavola hash richieda, nel caso peggiore, lo stesso tempo $\Theta(n)$ richiesto per ricercare un elemento in una lista concatenata, l'hashing si comporta molto bene nella pratica. Sotto ipotesi ragionevoli, il tempo medio per cercare un elemento in una tavola hash è $O(1)$.

Una tavola hash è una generalizzazione della nozione più semplice di array ordinario. L'indirizzamento diretto in un array ordinario sfrutta la possibilità di esaminare una posizione arbitraria in un array nel tempo $O(1)$. Il Paragrafo 11.1 descrive dettagliatamente l'indirizzamento diretto. Possiamo usare vantaggiosamente l'indirizzamento diretto quando possiamo permetterci di allocare un array che ha una posizione per ogni chiave possibile.

Quando il numero di chiavi effettivamente memorizzate è piccolo rispetto al numero totale di chiavi possibili, le tavole hash diventano una valida alternativa all'indirizzamento diretto di un array, in quanto una tavola hash tipicamente usa un array di dimensione proporzionale al numero di chiavi effettivamente memorizzate. Anziché utilizzare una chiave direttamente come un indice dell'array, la chiave viene usata per *calcolare* l'indice.

Il Paragrafo 11.2 presenta i concetti principali delle tavole hash, mettendo in particolare evidenza il *concatenamento* come un metodo per gestire le *collisioni* (si verifica una collisione quando più chiavi vengono associate a uno stesso indice di array). Il Paragrafo 11.3 spiega come calcolare gli indici di un array dalle chiavi utilizzando le funzioni hash. Presenteremo e analizzeremo alcune varianti sul tema. Il Paragrafo 11.4 descrive un altro metodo per gestire le collisioni: l'*indirizzamento aperto*. La conclusione è che l'hashing è una tecnica estremamente pratica ed efficace: le operazioni fondamentali sui dizionari in media richiedono soltanto un tempo $O(1)$. Il Paragrafo 11.5 descrive l'*hashing perfetto* che permette di effettuare ricerche nel tempo $O(1)$ nel *caso peggiore*, quando l'insieme delle chiavi da memorizzare è statico (cioè, quando l'insieme delle chiavi non cambia più, una volta che è stato memorizzato).

Acquistato da Michele Mo[illegible] 2022-7-27 Vietata la riproduzione di questo documento 978-88-386-2016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

11.1 Tavole a indirizzamento diretto

L'indirizzamento diretto è una tecnica semplice che funziona bene quando l'universo U delle chiavi è ragionevolmente piccolo. Supponiamo che un'applicazione

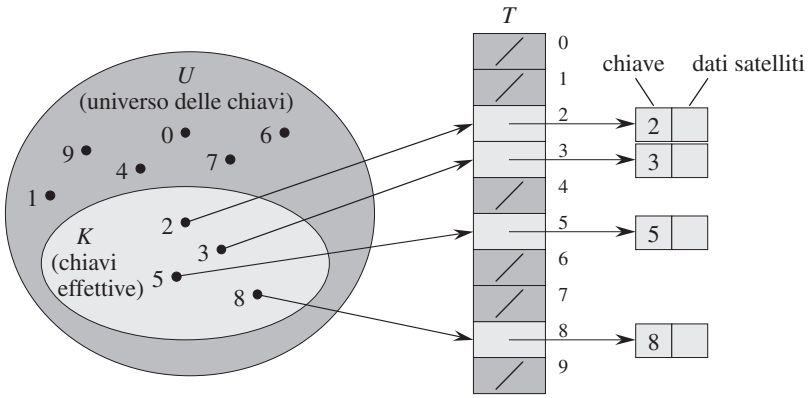


Figura 11.1
Implementazione di un insieme dinamico tramite la tavola a indirizzamento diretto T. Ogni chiave nell'universo $U = \{0, 1, \ldots, 9\}$ corrisponde a un indice della tavola. L'insieme $K = \{2, 3, 5, 8\}$ delle chiavi effettive determina le celle nella tavola che contengono i puntatori agli elementi. Le altre celle (su sfondo più scuro) contengono la costante NIL.

abbia bisogno di un insieme dinamico in cui ogni elemento ha una chiave estratta dall'universo $U = \{0, 1, \ldots, m-1\}$, dove m non è troppo grande. Supponiamo inoltre che due elementi non possano avere la stessa chiave.

Per rappresentare l'insieme dinamico, utilizziamo un array o ***tavola a indirizzamento diretto***, che indicheremo con $T[0\,..\,m-1]$, dove ogni posizione o ***cella*** corrisponde a una chiave nell'universo U. La Figura 11.1 illustra il metodo; la cella k punta a un elemento dell'insieme con chiave k. Se l'insieme non contiene l'elemento con chiave k, allora $T[k] = \text{NIL}$.

Le operazioni di dizionario sono semplici da implementare.

DIRECT-ADDRESS-SEARCH(T, k)
1 **return** $T[k]$

DIRECT-ADDRESS-INSERT(T, x)
1 $T[x.key] = x$

DIRECT-ADDRESS-DELETE(T, x)
1 $T[x.key] = \text{NIL}$

Ciascuna di queste operazioni richiede tempo $O(1)$.

Per alcune applicazioni, gli elementi dell'insieme dinamico possono essere memorizzati nella stessa tavola a indirizzamento diretto. Ovvero, anziché memorizzare la chiave e i dati satelliti di un elemento in un oggetto esterno alla tavola a indirizzamento diretto, con un puntatore da una cella della tavola all'oggetto, possiamo memorizzare l'oggetto nella cella stessa, risparmiando spazio. Possiamo usare una chiave speciale per indicare che una cella è vuota.

Inoltre, spesso non è necessario memorizzare la chiave dell'oggetto, in quanto se abbiamo l'indice di un oggetto nella tavola, abbiamo la sua chiave. Se non memorizziamo le chiavi, però, dobbiamo comunque prevedere un modo per indicare che una cella è vuota.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Esercizi

11.1-1

Supponete che un insieme dinamico S sia rappresentato da una tavola a indirizzamento diretto T di lunghezza m. Descrivete una procedura che trova l'elemento massimo di S. Qual è la prestazione della vostra procedura nel caso peggiore?

11.1-2
Un ***vettore di bit*** è semplicemente un array di bit (0 e 1). Un vettore di bit di lunghezza m occupa molto meno spazio di un array di m puntatori. Descrivete come utilizzare un vettore di bit per rappresentare un insieme dinamico di elementi distinti senza dati satelliti. Le operazioni di dizionario dovrebbero essere eseguite nel tempo $O(1)$.

11.1-3
Spiegate come implementare una tavola a indirizzamento diretto in cui le chiavi degli elementi memorizzati non hanno bisogno di essere distinte e gli elementi possono avere dati satelliti. Tutte e tre le operazioni di dizionario (INSERT, DELETE e SEARCH) dovrebbero essere eseguite nel tempo $O(1)$ (non dimenticate che DELETE richiede come argomento un puntatore a un oggetto da cancellare, non una chiave).

11.1-4 ⋆
Vogliamo implementare un dizionario utilizzando l'indirizzamento diretto su un array di enorme dimensione. All'inizio, gli elementi dell'array possono contenere dati non significativi ma sarebbe poco pratico inizializzare l'intero array, considerando la sua dimensione. Descrivete uno schema per implementare un dizionario a indirizzamento diretto su un array di tale dimensione. Ogni oggetto memorizzato dovrebbe occupare lo spazio $O(1)$; le operazioni SEARCH, INSERT e DELETE dovrebbero impiegare ciascuna un tempo $O(1)$; l'inizializzazione della struttura dati dovrebbe richiedere un tempo $O(1)$ (*suggerimento:* per capire se una voce dell'array è valida oppure no, usate un altro array, da trattare in modo simile a uno stack di dimensione pari al numero di chiavi effettivamente memorizzate nel dizionario).

11.2 Tavole hash

La difficoltà dell'indirizzamento diretto è ovvia: se l'universo delle chiavi U è troppo grande, memorizzare una tavola T di dimensione $|U|$ può essere impraticabile, o perfino impossibile, considerando la memoria disponibile in un normale calcolatore. Inoltre, l'insieme K delle chiavi *effettivamente memorizzate* può essere così piccolo rispetto a U che la maggior parte dello spazio allocato per la tavola T sarebbe sprecato.

Quando l'insieme K delle chiavi memorizzate in un dizionario è molto più piccolo dell'universo U di tutte le chiavi possibili, una tavola hash richiede molto meno spazio di una tavola a indirizzamento diretto. Lo spazio richiesto può essere ridotto a $\Theta(|K|)$, senza perdere il vantaggio di ricercare un elemento nella tavola hash nel tempo $O(1)$. L'unico problema è che questo limite vale per il *tempo medio*, mentre nell'indirizzamento diretto vale per il *tempo nel caso peggiore*.

Con l'indirizzamento diretto, un elemento con chiave k è memorizzato nella cella k. Con l'hashing, questo elemento è memorizzato nella cella $h(k)$; cioè, utilizziamo una ***funzione hash*** h per calcolare la cella dalla chiave k. Qui h associa l'universo U delle chiavi alle celle di una ***tavola hash*** $T[0\,..\,m-1]$:

Acquistato da Michele Michele su Webster il 2022-07-07 13:12 Numero Ordine Libreria: 1995030016-2207U7-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$h : U \to \{0, 1, \ldots, m-1\}$$

dove la dimensione m della tavola hash è generalmente molto più piccola di $|U|$. Diciamo che un elemento con chiave k viene mappato nella cella $h(k)$ o anche che $h(k)$ è il ***valore hash*** della chiave k. La Figura 11.2 illustra il concetto di base.

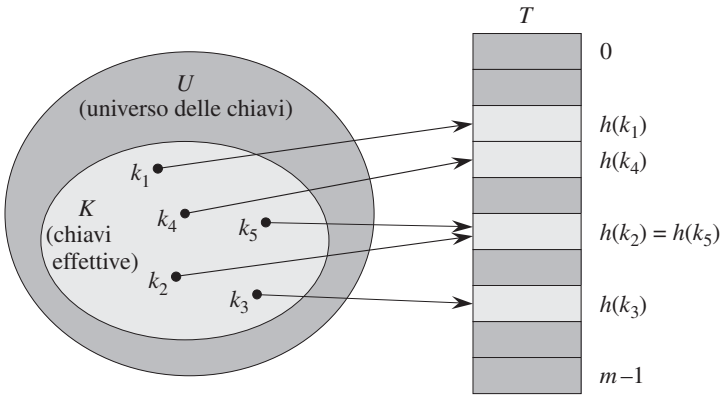


Figura 11.2 L'uso di una funzione hash h per associare le chiavi alle celle della tavola hash. Le chiavi k_2 e k_5 sono associate alla stessa cella, quindi sono in collisione.

Il compito della funzione hash è ridurre l'intervallo degli indici e di conseguenza la dimensione dell'array. L'array ha dimensione m invece di $|U|$.

C'è un problema: due chiavi possono essere mappate nella stessa cella. Questo evento si chiama ***collisione***. Fortunatamente, ci sono delle tecniche efficaci per risolvere i conflitti creati dalle collisioni.

Ovviamente, la soluzione ideale sarebbe quella di evitare qualsiasi collisione. Potremmo tentare di raggiungere questo obiettivo scegliendo un'opportuna funzione hash h. Un'idea potrebbe essere quella di costruire la funzione h in modo che essa sembri "casuale", evitando così le collisioni o almeno riducendo al minimo il loro numero. Il significato letterale del termine "hash" (polpettone fatto con avanzi di carne e verdure tritati) evoca l'immagine di rimescolamenti e spezzettamenti casuali, che rendono bene l'idea ispiratrice di questo approccio (naturalmente, una funzione hash deve essere deterministica, nel senso che un dato input k dovrebbe produrre sempre lo stesso output $h(k)$). Tuttavia, poiché $|U| > m$, ci devono essere almeno due chiavi che hanno lo stesso valore hash; evitare completamente le collisioni è quindi impossibile. In definitiva, anche se una funzione hash ben progettata e apparentemente "casuale" può ridurre al minimo il numero di collisioni, occorre comunque un metodo per risolvere le collisioni che si verificano.

Il resto di questo paragrafo descrive la tecnica più semplice per risolvere le collisioni: il concatenamento. Il Paragrafo 11.4 presenta un metodo alternativo per risolvere le collisioni: l'indirizzamento aperto.

Risoluzione delle collisioni mediante concatenamento

Nel ***concatenamento*** poniamo tutti gli elementi che sono associati alla stessa cella in una lista concatenata, come illustra la Figura 11.3. La cella j contiene un puntatore alla testa della lista di tutti gli elementi memorizzati che vengono mappati in j; se non ce ne sono, la cella j contiene la costante NIL.

Le operazioni di dizionario su una tavola hash T sono facili da implementare quando le collisioni sono risolte con il concatenamento.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

CHAINED-HASH-INSERT(T, x)
inserisce x in testa alla lista $T[h(x.key)]$

CHAINED-HASH-SEARCH(T, k)
ricerca un elemento con chiave k nella lista $T[h(k)]$

CHAINED-HASH-DELETE(T, x)
cancella x dalla lista $T[h(x.key)]$

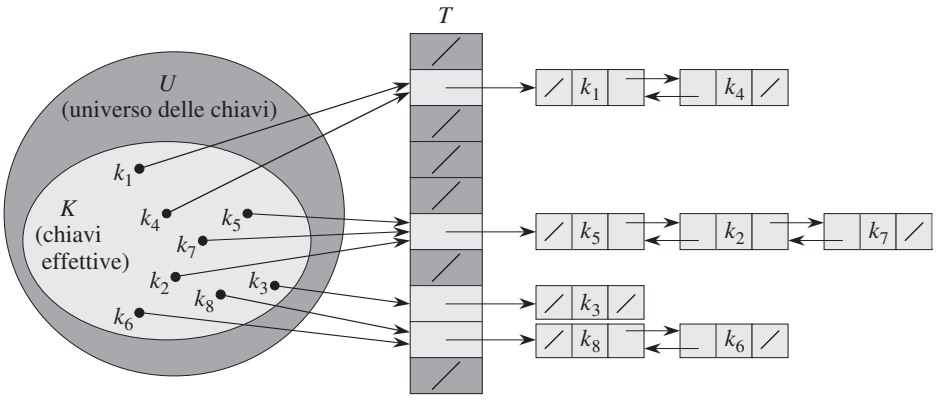


Figura 11.3 Risoluzione delle collisioni mediante concatenamento. Ogni cella $T[j]$ della tavola hash contiene una lista concatenata di tutte le chiavi il cui valore hash è j. Per esempio, $h(k_1) = h(k_4)$ e $h(k_5) = h(k_7) = h(k_2)$. La lista può essere singolarmente o doppiamente concatenata; abbiamo mostrato quella doppiamente concatenata perché così la cancellazione è più rapida.

Il tempo di esecuzione nel caso peggiore per l'inserimento è $O(1)$. La procedura di inserimento è veloce, anche perché si suppone che l'elemento x da inserire non sia presente nella tavola; se necessario, questa ipotesi può essere verificata (con un costo aggiuntivo) ricercando un elemento la cui chiave sia $x.key$, prima di effettuare l'inserimento. Per la ricerca, il tempo di esecuzione nel caso peggiore è proporzionale alla lunghezza della lista; in seguito analizzeremo questa operazione più dettagliatamente. La cancellazione di un elemento x può essere realizzata nel tempo $O(1)$ se le liste sono doppiamente concatenate, come mostra la Figura 11.3.

Notate che CHAINED-HASH-DELETE prende come input un elemento x, non la sua chiave k, quindi non occorre cercare prima x. Se la tavola hash supporta la cancellazione, allora le sue liste dovrebbero essere doppiamente concatenate in modo che la cancellazione di un elemento sia più rapida. Se le liste fossero singolarmente concatenate, per cancellare l'elemento x, dovremmo prima trovare x nella lista $T[h(x.key)]$ in modo da poter aggiornare l'attributo *next* del predecessore di x. Con le liste singolarmente concatenate, la cancellazione e la ricerca avrebbero essenzialmente lo stesso tempo di esecuzione asintotico.

Analisi dell'hashing con concatenamento

Come sono le prestazioni dell'hashing con il concatenamento? In particolare, quanto tempo occorre per ricercare un elemento con una data chiave?

Data una tavola hash T con m celle dove sono memorizzati n elementi, definiamo ***fattore di carico*** α della tavola T il rapporto n/m, ossia il numero medio di elementi memorizzati in una lista. La nostra analisi sarà fatta in funzione di α, che può essere minore, uguale o maggiore di 1.

Il comportamento nel caso peggiore dell'hashing con concatenamento è pessimo: tutte le n chiavi sono associate alla stessa cella, creando una lista di lunghezza n. Il tempo di esecuzione della ricerca è quindi $\Theta(n)$ più il tempo per calcolare la funzione hash – non migliore del tempo richiesto nel caso in cui avessimo utilizzato una lista concatenata per tutti gli elementi. Chiaramente, le tavole hash non sono utilizzate per le loro prestazioni nel caso peggiore (tuttavia, quando l'insieme delle chiavi è statico, si possono ottenere buone prestazioni nel caso peggiore usando l'hashing perfetto, descritto nel Paragrafo 11.5).

Acquistato da Michele Mahelin Vedatelli 2022-07-07 21:13 - Autore: Ordine Libraio 1805000116-230707 - Copyright © 2022 - McGraw-Hill Education (Italy)

Le prestazioni dell'hashing nel caso medio dipendono dal modo in cui la funzione hash h distribuisce mediamente l'insieme delle chiavi da memorizzare tra

le m celle. Il Paragrafo 11.3 tratta questi problemi; per adesso supponiamo che qualsiasi elemento abbia la stessa probabilità di essere mandato in una qualsiasi delle m celle, indipendentemente dalle celle in cui sono mandati gli altri elementi. Questa ipotesi è detta ***hashing uniforme semplice***.

Per $j = 0, 1, \ldots, m-1$, indicando con n_j la lunghezza della lista $T[j]$, avremo

$$n = n_0 + n_1 + \cdots + n_{m-1} \tag{11.1}$$

e il valore atteso di n_j sarà $\mathrm{E}[n_j] = \alpha = n/m$.

Supponiamo che basti un tempo $O(1)$ per calcolare il valore hash $h(k)$, in modo che il tempo richiesto per cercare un elemento con chiave k dipenda linearmente dalla lunghezza $n_{h(k)}$ della lista $T[h(k)]$. Mettendo assieme in un tempo $O(1)$ il tempo richiesto per calcolare la funzione hash e accedere alla cella $h(k)$, consideriamo il numero atteso di elementi esaminati dall'algoritmo di ricerca, ovvero il numero di elementi nella lista $T[h(k)]$ che vengono controllati per vedere se le loro chiavi sono uguali a k. Considereremo due casi. Nel primo caso, la ricerca non ha successo: nessun elemento nella tavola ha la chiave k. Nel secondo caso, la ricerca ha successo e viene trovato un elemento con chiave k.

Teorema 11.1
In una tavola hash le cui collisioni sono risolte con il concatenamento, una ricerca senza successo richiede un tempo $\Theta(1+\alpha)$ nel caso medio, nell'ipotesi di hashing uniforme semplice.

Dimostrazione Nell'ipotesi di hashing uniforme semplice, qualsiasi chiave k non ancora memorizzata nella tavola ha la stessa probabilità di essere associata a uno qualsiasi delle m celle. Il tempo atteso per ricercare senza successo una chiave k è il tempo atteso per svolgere le ricerche fino alla fine della lista $T[h(k)]$, che ha una lunghezza attesa pari a $\mathrm{E}[n_{h(k)}] = \alpha$. Quindi, il numero atteso di elementi esaminato in una ricerca senza successo è α e il tempo totale richiesto (incluso quello per calcolare $h(k)$) è $\Theta(1 + \alpha)$. ■

Il caso di una ricerca con successo è un po' differente, perché ogni lista non ha la stessa probabilità di essere oggetto delle ricerche. La probabilità che una lista sia oggetto delle ricerche è proporzionale al numero di elementi che contiene. Nonostante questo, il tempo atteso è ancora $\Theta(1 + \alpha)$.

Teorema 11.2
In una tavola hash le cui collisioni sono risolte con il concatenamento, una ricerca con successo richiede un tempo $\Theta(1 + \alpha)$ nel caso medio, nell'ipotesi di hashing uniforme semplice.

Dimostrazione Supponiamo che l'elemento da ricercare abbia la stessa probabilità di essere uno qualsiasi degli n elementi memorizzati nella tavola. Il numero di elementi esaminati durante una ricerca con successo di un elemento x è uno in più del numero di elementi che si trovano prima di x nella lista di x. Gli elementi che precedono x nella lista sono stati inseriti tutti dopo di x, perché i nuovi elementi vengono posti all'inizio della lista. Per trovare il numero atteso di elementi esaminati, prendiamo la media, sugli n elementi x nella tavola, di 1 più il numero atteso di elementi aggiunti alla lista di x dopo che x è stato aggiunto alla lista. Indichiamo con x_i l'i-esimo elemento inserito nella tavola, per $i = 1, 2, \ldots, n$, e sia $k_i = x_i.key$. Per le chiavi k_i e k_j, definiamo la variabile casuale indicatri-

Acquistato da Michele Michele su Webster il 2022-07-11 Copia personale non trasferibile © 2010 The McGraw-Hill Education (Italy)

ce $X_{ij} = \mathrm{I}\{h(k_i) = h(k_j)\}$. Nell'ipotesi di hashing uniforme semplice, abbiamo $\Pr\{h(k_i) = h(k_j)\} = 1/m$ e, quindi, per il Lemma 5.1, $\mathrm{E}[X_{ij}] = 1/m$. Dunque, il numero atteso di elementi esaminati in una ricerca con successo è

$$\begin{aligned}
&\mathrm{E}\left[\frac{1}{n}\sum_{i=1}^{n}\left(1+\sum_{j=i+1}^{n}X_{ij}\right)\right] \\
&\quad= \frac{1}{n}\sum_{i=1}^{n}\left(1+\sum_{j=i+1}^{n}\mathrm{E}[X_{ij}]\right) \quad \text{(per la linearità del valore atteso)} \\
&\quad= \frac{1}{n}\sum_{i=1}^{n}\left(1+\sum_{j=i+1}^{n}\frac{1}{m}\right) \\
&\quad= 1+\frac{1}{nm}\sum_{i=1}^{n}(n-i) \\
&\quad= 1+\frac{1}{nm}\left(\sum_{i=1}^{n}n-\sum_{i=1}^{n}i\right) \\
&\quad= 1+\frac{1}{nm}\left(n^2-\frac{n(n+1)}{2}\right) \quad \text{(per l'equazione (A.1))} \\
&\quad= 1+\frac{n-1}{2m} \\
&\quad= 1+\frac{\alpha}{2}-\frac{\alpha}{2n}
\end{aligned}$$

In conclusione, il tempo totale richiesto per una ricerca con successo (incluso il tempo per calcolare la funzione hash) è $\Theta(2+\alpha/2-\alpha/2n) = \Theta(1+\alpha)$. ■

Qual è il significato di questa analisi? Se il numero di celle della tavola hash è almeno proporzionale al numero di elementi della tavola, abbiamo $n = O(m)$ e, di conseguenza, $\alpha = n/m = O(m)/m = O(1)$. Pertanto, la ricerca richiede in media un tempo costante. Poiché l'inserimento richiede il tempo $O(1)$ nel caso peggiore e la cancellazione richiede il tempo $O(1)$ nel caso peggiore quando le liste sono doppiamente concatenate, tutte le operazioni di dizionario possono essere svolte, in media, nel tempo $O(1)$.

Esercizi

11.2-1

Supponete di usare una funzione hash h per mettere n chiavi distinte in un array T di lunghezza m. Nell'ipotesi di hashing uniforme semplice, qual è il numero atteso di collisioni? Più precisamente, qual è la cardinalità attesa di $\{\{k,l\} : k \neq l \text{ e } h(k) = h(l)\}$?

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

11.2-2

Mostrate cosa succede quando si inseriscono le chiavi $5, 28, 19, 15, 20, 33, 12, 17, 10$ in una tavola hash, risolvendo le collisioni mediante il concatenamento. Supponete che la tavola abbia 9 celle e che la funzione hash sia $h(k) = k \bmod 9$.

11.2-3
Il professor Marley ritiene che si possa ottenere un notevole miglioramento delle prestazioni modificando lo schema del concatenamento in modo che ogni lista sia mantenuta ordinata. Come influisce la modifica del professore sul tempo di esecuzione delle ricerche con successo, delle ricerche senza successo, degli inserimenti e delle cancellazioni?

11.2-4
Indicate come allocare e rilasciare lo spazio per gli elementi all'interno della stessa tavola hash concatenando in una free list tutte le celle di lista non utilizzate. Supponete che una cella contenga un bit di marcatura (flag), oltre a due puntatori o a un elemento e un puntatore. Tutte le operazioni del dizionario e della free list dovrebbero essere eseguite nel tempo atteso $O(1)$. La free list deve essere doppiamente concatenata o è sufficiente che sia singolarmente concatenata?

11.2-5
Supponete di memorizzare una serie di n chiavi in una tavola hash di dimensione m. Dimostrate che, se la chiavi vengono estratte da un universo U con $|U| > nm$, esiste un sottoinsieme di U di dimensione n formato da chiavi che vengono mandate tutte nella stessa cella e quindi il tempo di ricerca nel caso peggiore per l'hashing con concatenamento è $\Theta(n)$.

11.2-6
Supponete di memorizzare una serie di n chiavi in una tavola hash di dimensione m, con le collisioni risolte mediante concatenamento, e che sia nota la lunghezza di ciascuna lista, inclusa la lunghezza L della lista più lunga. Descrivete una procedura che seleziona casualmente una chiave nella tavola hash e la restituisca nel tempo atteso $O(L \cdot (1 + 1/\alpha))$.

11.3 Funzioni hash

In questo paragrafo, tratteremo alcuni problemi che riguardano il progetto di buone funzioni hash; poi presenteremo tre schemi per la loro realizzazione. Due degli schemi, l'hashing per divisione e l'hashing per moltiplicazione, sono di tipo euristico, mentre il terzo schema, l'hashing universale, usa la randomizzazione per fornire buone prestazioni dimostrabili.

Caratteristiche di una buona funzione hash

Una buona funzione hash soddisfa (approssimativamente) l'ipotesi dell'hashing uniforme semplice: ogni chiave ha la stessa probabilità di essere mandata in una qualsiasi delle m celle, indipendentemente dalla cella cui viene mandata qualsiasi altra chiave.

Purtroppo, di solito non è possibile verificare questa condizione, in quanto raramente è nota la distribuzione delle probabilità secondo la quale vengono estratte le chiavi. Inoltre le chiavi potrebbero non essere estratte in maniera indipendente. A volte tale distribuzione è nota. Per esempio, se le chiavi sono numeri reali casuali k distribuiti in modo indipendente e uniforme nell'intervallo $0 \leq k < 1$, la funzione hash

$$h(k) = \lfloor km \rfloor$$

soddisfa la condizione dell'hashing uniforme semplice.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Nella pratica, spesso è possibile utilizzare delle tecniche euristiche per realizzare funzioni hash con buone prestazioni. Le informazioni qualitative sulla distribuzione delle chiavi possono essere utili in questa fase di progettazione. Per esempio, considerate la tabella dei simboli di un compilatore, in cui le chiavi sono stringhe di caratteri che rappresentano gli identificatori di un programma. Simboli strettamente correlati, come `pt` e `pts`, si trovano spesso nello stesso programma. Una buona funzione hash dovrebbe ridurre al minimo la probabilità che tali varianti siano mappate nella stessa cella.

Un buon approccio consiste nel derivare il valore hash in modo che sia indipendente da qualsiasi regolarità che possa esistere nei dati. Per esempio, il "metodo della divisione" (descritto nel Paragrafo 11.3.1) calcola il valore hash come il resto della divisione fra la chiave e un determinato numero primo. Questo metodo spesso fornisce buoni risultati, purché il numero primo sia scelto in modo da non essere correlato a nessuna regolarità nella distribuzione delle chiavi.

Infine, notiamo che alcune applicazioni delle funzioni hash potrebbero richiedere proprietà più vincolanti di quelle richieste dall'hashing uniforme semplice. Per esempio, potremmo richiedere che le chiavi che sono "vicine" in qualche maniera forniscano valori hash che siano distanti (questa proprietà è particolarmente desiderabile quando si usa l'ispezione lineare, che è definita nel Paragrafo 11.4). L'hashing universale, descritto nel Paragrafo 11.3.3, spesso offre le proprietà richieste.

Interpretare le chiavi come numeri naturali

La maggior parte delle funzioni hash suppone che l'universo delle chiavi sia l'insieme dei numeri naturali $\mathbb{N} = \{0, 1, 2, \ldots\}$. Quindi, se le chiavi non sono numeri naturali, occorre un metodo per interpretarle come tali. Per esempio, una stringa di caratteri può essere interpretata come un numero intero espresso in una notazione posizionale di base opportuna. Quindi, l'identificatore `pt` può essere interpretato come la coppia di interi in notazione decimale $(112, 116)$, in quanto $\texttt{p} = 112$ e $\texttt{t} = 116$ nel codice dei caratteri ASCII; poi, rappresentato in notazione posizionale con base 128, `pt` diventa $(112 \cdot 128) + 116 = 14452$. In ogni applicazione è generalmente semplice trovare un metodo per interpretare ogni chiave come un numero naturale (eventualmente grande). Nei prossimi paragrafi supporremo che le chiavi siano numeri naturali.

11.3.1 Il metodo della divisione

Quando si applica il ***metodo della divisione*** per creare una funzione hash, una chiave k viene associata a una delle m celle prendendo il resto della divisione fra k e m; cioè la funzione hash è

$$h(k) = k \bmod m$$

Per esempio, se la tavola hash ha dimensione $m = 12$ e la chiave è $k = 100$, allora $h(k) = 4$. Questo metodo è molto veloce perché richiede una sola operazione di divisione.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950301 6-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Quando utilizziamo il metodo della divisione, di solito, evitiamo certi valori di m. Per esempio, m non dovrebbe essere una potenza di 2, perché se $m = 2^p$, allora $h(k)$ rappresenta proprio i p bit meno significativi di k. A meno che non sia noto che tutte le configurazioni dei p bit di ordine inferiore abbiano la stessa probabilità, è meglio rendere la funzione hash dipendente da tutti i bit della chiave.

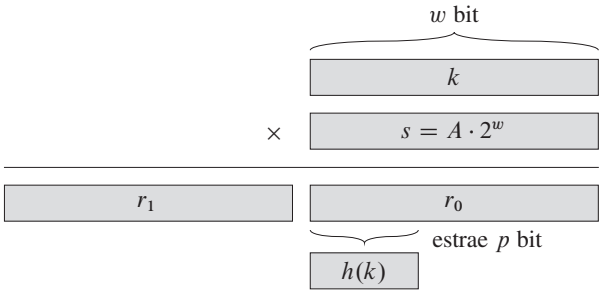


Figura 11.4 Il metodo della moltiplicazione per creare una funzione hash. Il valore della chiave k, rappresentato con w bit, viene moltiplicato per il valore $s = A \cdot 2^w$, anch'esso rappresentato con w bit. I p bit più significativi della metà meno significativa del prodotto formano il valore hash $h(k)$ desiderato.

Come vi sarà chiesto di dimostrare nell'Esercizio 11.3-3, scegliere $m = 2^p - 1$, quando k è una stringa di caratteri interpretata nella base 2^p, potrebbe essere una cattiva soluzione, perché la permutazione dei caratteri di k non cambia il suo valore hash.

Un numero primo non troppo vicino a una potenza esatta di 2 è spesso una buona scelta per m. Per esempio, supponiamo di allocare una tavola hash (risolvendo le collisioni mediante concatenamento) per contenere circa $n = 2000$ stringhe di caratteri, dove ogni carattere ha 8 bit. Poiché riteniamo accettabile esaminare in media 3 elementi in una ricerca senza successo, allochiamo una tavola hash di dimensione $m = 701$. Abbiamo scelto 701 perché è un numero primo vicino a $2000/3$, ma non a una potenza di 2. Trattando ogni chiave k come un numero intero, la nostra funzione hash diventa

$$h(k) = k \bmod 701$$

11.3.2 Il metodo della moltiplicazione

Il ***metodo della moltiplicazione*** per creare funzioni hash si svolge in due passi. Prima moltiplichiamo la chiave k per una costante A nell'intervallo $0 < A < 1$ ed estraiamo la parte frazionaria di kA. Poi moltiplichiamo questo valore per m e prendiamo la parte intera inferiore del risultato. In sintesi, la funzione hash è

$$h(k) = \lfloor m\,(k\,A \bmod 1) \rfloor$$

dove "$k\,A \bmod 1$" rappresenta la parte frazionaria di kA, cioè $kA - \lfloor kA \rfloor$.

Un vantaggio del metodo della moltiplicazione è che il valore di m non è critico. Tipicamente, lo scegliamo come una potenza di 2 ($m = 2^p$ per qualche intero p), il che rende semplice implementare la funzione hash nella maggior parte dei calcolatori nel modo seguente.

Supponiamo che la dimensione della parola della macchina sia w bit e che k entri in una sola parola. Come A prendiamo una frazione della forma $s/2^w$, dove s è un intero nell'intervallo $0 < s < 2^w$. Facendo riferimento alla Figura 11.4, moltiplichiamo prima k per l'intero di w bit $s = A \cdot 2^w$. Il risultato è un valore di $2w$ bit $r_1 2^w + r_0$, dove r_1 è la parte più significativa del prodotto e r_0 è la parte meno significativa del prodotto. Il valore hash desiderato di p bit è formato dai p bit più significativi di r_0.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Sebbene questo metodo funzioni con qualsiasi valore della costante A, tuttavia con qualche valore funziona meglio che con altri. La scelta ottimale dipende dalle caratteristiche dei dati da sottoporre all'hashing. Knuth [212] propone

$$A \approx (\sqrt{5} - 1)/2 = 0,6180339887\ldots \tag{11.2}$$

come valore che funziona ragionevolmente bene.

Come esempio, supponiamo di avere $k = 123456$, $p = 14$, $m = 2^{14} = 16384$ e $w = 32$. Adattando il valore proposto da Knuth, scegliamo come A la frazione della forma $s/2^{32}$ che è più vicina a $(\sqrt{5}-1)/2$, cosicché $A = 2654435769/2^{32}$. Quindi $k \cdot s = 327706022297664 = (76300 \cdot 2^{32}) + 17612864$ e così $r_1 = 76300$ e $r_0 = 17612864$. I 14 bit più significativi di r_0 forniscono il valore $h(k) = 67$.

⋆ 11.3.3 Hashing universale

Se un avversario sleale scegliesse le chiavi da elaborare con qualche funzione hash prefissata, potrebbe scegliere n chiavi che vengono mandate tutte alla stessa cella, generando un tempo medio di ricerca pari a $\Theta(n)$. Qualsiasi funzione hash è vulnerabile a un comportamento così inefficiente nel caso peggiore. L'unico sistema efficace per migliorare la situazione è scegliere *casualmente* la funzione hash in modo che sia *indipendente* dalle chiavi che devono essere effettivamente memorizzate. Questo approccio, detto ***hashing universale***, può permettere di ottenere buone prestazioni in media, indipendentemente da quali chiavi sceglie l'avversario.

Nell'hashing universale, all'inizio dell'esecuzione dell'algoritmo viene scelta casualmente la funzione hash in una classe di funzioni accuratamente progettata. Come nel caso di quicksort, la randomizzazione garantisce che nessun input possa provocare sistematicamente il comportamento nel caso peggiore dell'algoritmo. Siccome scegliamo casualmente la funzione hash, l'algoritmo può comportarsi in modo differente ogni volta che viene eseguito, anche con lo stesso input, garantendo buone prestazioni nel caso medio con qualsiasi input.

Riprendendo l'esempio della tabella dei simboli di un compilatore, scopriamo che la scelta degli identificatori effettuata dal programmatore adesso non può provocare prestazioni di hashing sistematicamente scadenti. Le prestazioni scadenti si verificano soltanto quando il compilatore sceglie una funzione hash casuale che si comporta in modo inefficiente su tale insieme di identificatori, ma la probabilità che si verifichi questo caso è piccola ed è la stessa per qualsiasi insieme di identificatori della stessa dimensione.

Sia $\mathcal{H}$ una collezione finita di funzioni hash che associano un dato universo U di chiavi all'intervallo $\{0, 1, \ldots, m-1\}$. Tale collezione è detta ***universale*** se, per ogni coppia di chiavi distinte $k, l \in U$, il numero di funzioni hash $h \in \mathcal{H}$ per le quali $h(k) = h(l)$ è al massimo $|\mathcal{H}|/m$. In altre parole, con una funzione hash scelta a caso da $\mathcal{H}$, la probabilità di una collisione fra due chiavi distinte k e l non è maggiore della probabilità $1/m$ di una collisione nel caso in cui $h(k)$ e $h(l)$ fossero scelte in modo casuale e indipendente dall'insieme $\{0, 1, \ldots, m-1\}$.

Il seguente teorema dimostra che una classe universale di funzioni hash garantisce un buon comportamento nel caso medio. Ricordiamo che n_i indica la lunghezza della lista $T[i]$.

Teorema 11.3

Supponiamo che una funzione hash h sia scelta casualmente da una collezione universale di funzioni hash e sia utilizzata per inserire n chiavi in una tavola T di dimensione m, applicando il concatenamento per risolvere le collisioni. Se la chiave k non è nella tavola, allora la lunghezza attesa $\mathrm{E}[n_{h(k)}]$ della lista dove viene inserita la chiave k è al massimo il fattore di carico $\alpha = n/m$. Se la chiave k è nella tavola, allora la lunghezza attesa $\mathrm{E}[n_{h(k)}]$ della lista che contiene la chiave k è al massimo $1 + \alpha$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995030016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Dimostrazione Notiamo che i valori attesi qui riguardano la scelta della funzione hash e non dipendono da nessuna ipotesi sulla distribuzione delle chiavi. Per ogni coppia k e l di chiavi distinte, definiamo la variabile casuale indicatrice $X_{kl} = \mathrm{I}\{h(k) = h(l)\}$. Poiché per la definizione di collezione universale di funzioni hash, una singola coppia di chiavi collide con probabilità al massimo $1/m$, abbiamo $\Pr\{h(k) = h(l)\} \leq 1/m$ e, quindi, il Lemma 5.1 implica che $\mathrm{E}[X_{kl}] \leq 1/m$.

Adesso definiamo, per ogni chiave k, la variabile casuale Y_k che è uguale al numero di chiavi diverse da k che sono associate alla stessa cella di k, quindi

$$Y_k = \sum_{\substack{l \in T \\ l \neq k}} X_{kl}$$

Ne consegue che

$$\begin{aligned} \mathrm{E}[Y_k] &= \mathrm{E}\left[\sum_{\substack{l \in T \\ l \neq k}} X_{kl}\right] \\ &= \sum_{\substack{l \in T \\ l \neq k}} \mathrm{E}[X_{kl}] \qquad \text{(per la linearità del valore atteso)} \\ &\leq \sum_{\substack{l \in T \\ l \neq k}} \frac{1}{m} \end{aligned}$$

Il resto della dimostrazione varia a seconda che la chiave k si trovi oppure no nella tavola T.

- Se $k \notin T$, allora $n_{h(k)} = Y_k$ e $|\{l : l \in T \text{ and } l \neq k\}| = n$. Quindi $\mathrm{E}[n_{h(k)}] = \mathrm{E}[Y_k] \leq n/m = \alpha$.
- Se $k \in T$, allora poiché la chiave k appare nella lista $T[h(k)]$ e il numero indicato da Y_k non include la chiave k, abbiamo $n_{h(k)} = Y_k + 1$ e $|\{l : l \in T \text{ e } l \neq k\}| = n-1$. Quindi $\mathrm{E}[n_{h(k)}] = \mathrm{E}[Y_k]+1 \leq (n-1)/m+1 = 1 + \alpha - 1/m < 1 + \alpha$. ■

Il seguente corollario dice che l'hashing universale offre il vantaggio desiderato: adesso è impossibile per un avversario scegliere una sequenza di operazioni che forzi il tempo di esecuzione nel caso peggiore. Randomizzando intelligentemente la scelta della funzione hash a tempo di esecuzione, abbiamo la garanzia che ogni sequenza di operazioni possa essere processata con un buon tempo di esecuzione nel caso medio.

Corollario 11.4
Utilizzando l'hashing universale e la risoluzione delle collisioni mediante concatenamento in una tavola con m celle inizialmente vuote, occorre il tempo atteso $\Theta(n)$ per eseguire qualsiasi sequenza di n operazioni INSERT, SEARCH e DELETE che contiene $O(m)$ operazioni INSERT.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Dimostrazione Poiché il numero di inserimenti è $O(m)$, si ha $n = O(m)$ e quindi $\alpha = O(1)$. Le operazioni INSERT e DELETE richiedono un tempo costante e, per il Teorema 11.3, il tempo atteso per ogni operazione SEARCH è $O(1)$. Per la linearità del valore atteso, quindi, il tempo atteso dell'intera sequenza di operazioni è $O(n)$. Poiché ogni operazione richiede il tempo $\Omega(1)$, è dimostrato il limite $\Theta(n)$. ■

Progettare una classe universale di funzioni hash

È abbastanza semplice progettare una classe universale di funzioni hash, come dimostreremo con l'aiuto di un po' di teoria dei numeri. Se non avete dimestichezza con la teoria dei numeri, consultate il Capitolo 31. Iniziamo scegliendo un numero primo p sufficientemente grande in modo che ogni possibile chiave k sia compresa nell'intervallo da 0 a $p-1$, estremi inclusi. Indichiamo con $\mathbb{Z}_p$ l'insieme $\{0, 1, \ldots, p-1\}$ e con $\mathbb{Z}_p^*$ l'insieme $\{1, 2, \ldots, p-1\}$. Essendo p un numero primo, possiamo risolvere le equazioni modulo p con i metodi descritti nel Capitolo 31. Poiché supponiamo che la dimensione dell'universo delle chiavi sia maggiore del numero di celle nella tavola hash, abbiamo $p > m$.

Adesso definiamo la funzione hash h_{ab} per ogni $a \in \mathbb{Z}_p^*$ e ogni $b \in \mathbb{Z}_p$ utilizzando una trasformazione lineare seguita da riduzioni modulo p e poi modulo m:

$$h_{ab}(k) = ((ak + b) \bmod p) \bmod m \tag{11.3}$$

Per esempio, con $p = 17$ e $m = 6$, abbiamo $h_{3,4}(8) = 5$. La famiglia di tutte queste funzioni hash è

$$\mathcal{H}_{pm} = \{h_{ab} : a \in \mathbb{Z}_p^* \text{ e } b \in \mathbb{Z}_p\} \tag{11.4}$$

Ogni funzione hash h_{ab} manda da $\mathbb{Z}_p$ a $\mathbb{Z}_m$. Questa classe di funzioni hash ha la bella proprietà che la dimensione m dell'intervallo di output è arbitraria – non necessariamente un numero primo – una caratteristica che utilizzeremo nel Paragrafo 11.5. Poiché le scelte possibili per a sono $p-1$ e le scelte possibili per b sono p, la collezione $\mathcal{H}_{pm}$ contiene $p(p-1)$ funzioni hash.

Teorema 11.5
La classe $\mathcal{H}_{pm}$ delle funzioni hash definita dalle equazioni (11.3) e (11.4) è universale.

Dimostrazione Considerate due chiavi distinte k e l dell'insieme $\mathbb{Z}_p$, con $k \neq l$. Per una data funzione hash h_{ab} poniamo

$$\begin{aligned} r &= (ak + b) \bmod p \\ s &= (al + b) \bmod p \end{aligned}$$

Notiamo intanto che $r \neq s$. Perché? Osserviamo che

$$r - s \equiv a(k - l) \pmod{p}$$

Ne consegue che $r \neq s$ perchè p è un numero primo e a e $(k-l)$ sono entrambi non nulli modulo p; quindi anche il loro prodotto deve essere non nullo modulo p per il Teorema 31.6. Pertanto, durante il calcolo di qualsiasi h_{ab} in $\mathcal{H}_{pm}$, input distinti k e l vengono associati a valori distinti r e s modulo p; al "livello mod p" non ci sono ancora collisioni. Inoltre, ciascuna delle $p(p-1)$ possibili scelte per la coppia (a, b) con $a \neq 0$ genera una coppia (r, s) *differente* con $r \neq s$, in quanto possiamo ricavare a e b, noti i valori r e s:

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995030016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned} a &= \big((r - s)((k - l)^{-1} \bmod p)\big) \bmod p \\ b &= (r - ak) \bmod p \end{aligned}$$

Dove $((k-l)^{-1} \bmod p)$ indica l'unico inverso moltiplicativo, modulo p, di $k-l$. Poiché ci sono soltanto $p(p-1)$ coppie possibili (r, s) con $r \neq s$, c'è una corrispondenza uno-a-uno fra le coppie (a, b) con $a \neq 0$ e le coppie (r, s) con $r \neq s$.

Quindi, per qualsiasi coppia di input k e l, se scegliamo (a, b) uniformemente a caso da $\mathbb{Z}_p^* \times \mathbb{Z}_p$, la coppia risultante (r, s) ha la stessa probabilità di essere qualsiasi coppia di valori distinti modulo p.

Ne consegue che la probabilità che le chiavi distinte k e l collidano è uguale alla probabilità che $r \equiv s \pmod{m}$ quando r e s sono scelte a caso come valori distinti modulo p. Per un dato valore di r, dei restanti $p - 1$ valori possibili di s, il numero di valori s tali che $s \neq r$ e $s \equiv r \pmod{m}$ è al massimo

$$\begin{aligned} \lceil p/m \rceil - 1 &\leq ((p + m - 1)/m) - 1 \quad \text{(per la disequazione (3.6))} \\ &= (p - 1)/m \end{aligned}$$

La probabilità che s collida con r quando viene ridotta modulo m è al massimo $((p - 1)/m)/(p - 1) = 1/m$. Di conseguenza, per qualsiasi coppia di valori distinti $k, l \in \mathbb{Z}_p$, si ha

$$\Pr\{h_{ab}(k) = h_{ab}(l)\} \leq 1/m$$

Quindi $\mathcal{H}_{pm}$ è davvero universale. ■

Esercizi

11.3-1

Supponete di effettuare una ricerca in una lista concatenata di lunghezza n, dove ogni elemento contiene una chiave k e un valore hash $h(k)$. Ogni chiave è una lunga stringa di caratteri. Come potreste trarre vantaggio dai valori hash quando ricercate nella lista un elemento con una data chiave?

11.3-2

Supponete che una stringa di r caratteri, trattata come numero in base 128, venga mappata in un insieme di m celle applicando il metodo della divisione. Il numero m è facilmente rappresentato come una parola di 32 bit nel calcolatore, ma la stringa di r caratteri, trattata come un numero con base 128, richiede molte parole. Come può essere applicato il metodo della divisione per calcolare il valore hash della stringa di caratteri senza utilizzare più di un numero costante di parole di memoria oltre a quelle usate per la stringa stessa?

11.3-3

Considerate una versione del metodo della divisione in cui $h(k) = k \bmod m$, dove $m = 2^p - 1$ e k è una stringa di caratteri interpretata come un numero con base 2^p. Dimostrate che, se la stringa x può essere ottenuta dalla stringa y permutando i suoi caratteri, allora le stringhe x e y sono mandate nello stesso valore. Indicate un'applicazione in cui non sarebbe desiderabile una funzione hash con questa proprietà.

11.3-4

Considerate una tavola hash di dimensione $m = 1000$ e una corrispondente funzione hash $h(k) = \lfloor m\,(k\,A \bmod 1) \rfloor$ per $A = (\sqrt{5} - 1)/2$. Calcolate le celle in cui saranno mandate le chiavi 61, 62, 63, 64 e 65.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 1995301&220767-2 Copyright © 2022, McGraw-Hill Education (Italy)

11.3-5 ★

Una famiglia $\mathcal{H}$ di funzioni hash che associano un insieme finito U a un insieme finito B si definisce ***ϵ-universale*** se, per ogni coppia di elementi distinti k e l di U, si ha

$$\Pr\{h(k) = h(l)\} \leq \epsilon$$

Dove la probabilità è calcolata sulle estrazioni casuali della funzione hash h dalla famiglia $\mathcal{H}$. Dimostrate che in ogni famiglia ϵ-universale di funzioni hash

$$\epsilon \geq \frac{1}{|B|} - \frac{1}{|U|}$$

11.3-6 ⋆

Indichiamo con U l'insieme delle n-tuple (una n-tupla è una serie di n valori) estratte da $\mathbb{Z}_p$; inoltre, sia $B = \mathbb{Z}_p$, dove p è un numero primo. Per ogni $b \in \mathbb{Z}_p$ definiamo la funzione hash $h_b : U \to B$ su di una n-tupla di input $\langle a_0, a_1, \ldots, a_{n-1}\rangle$ in U nel seguente modo:

$$h_b(\langle a_0, a_1, \ldots, a_{n-1}\rangle) = \sum_{j=0}^{n-1} a_j b^j$$

Sia $\mathcal{H} = \{h_b : b \in \mathbb{Z}_p\}$. Dimostrate che $\mathcal{H}$ è $((n-1)/p)$-universale secondo la definizione di ϵ-universale data nell'Esercizio 11.3-5 (*suggerimento:* vedere l'Esercizio 31.4-4).

11.4 Indirizzamento aperto

Nell'***indirizzamento aperto***, tutti gli elementi sono memorizzati nella tavola hash stessa; ovvero ogni cella della tavola contiene un elemento dell'insieme dinamico o la costante NIL. Quando cerchiamo un elemento, esaminiamo sistematicamente le celle della tavola finché non troviamo l'elemento desiderato o finché non ci accorgiamo che l'elemento non si trova nella tavola.

Diversamente dal concatenamento, non ci sono liste né elementi memorizzati all'esterno della tavola. Quindi, nell'indirizzamento aperto, la tavola hash può "riempirsi" al punto tale che non possono essere effettuati altri inserimenti; una conseguenza è che il fattore di carico α non supera mai 1. Ovviamente, potremmo memorizzare le liste per il concatenamento all'interno della tavola hash, nelle celle altrimenti inutilizzati della tavola hash (vedere l'Esercizio 11.2-4), ma il vantaggio dell'indirizzamento aperto sta nel fatto che esclude completamente i puntatori. Anziché seguire i puntatori, *calcoliamo* la sequenza delle celle da esaminare. La memoria extra liberata per non avere memorizzato i puntatori offre alla tavola hash un maggior numero di celle, a parità di memoria occupata, consentendo potenzialmente di ridurre il numero di collisioni e di accelerare le operazioni di ricerca.

Per effettuare un inserimento mediante l'indirizzamento aperto, esaminiamo in successione le posizioni della tavola hash (***ispezione***), finché non troviamo una cella vuota in cui inserire la chiave. Anziché seguire sempre lo stesso ordine $0, 1, \ldots, m-1$ (che richiede un tempo di ricerca $\Theta(n)$), la sequenza delle posizioni esaminate durante una ispezione *dipende dalla chiave da inserire*. Per determinare quali celle esaminare, estendiamo la funzione hash in modo da includere l'ordine di ispezione (a partire da 0) come secondo input. Quindi, la funzione hash diventa

$$h : U \times \{0, 1, \ldots, m-1\} \to \{0, 1, \ldots, m-1\}$$

Acquistato da Michele Mostarda il 07/07/2022 | Numero Licenza 1198833 | Copyright © 2022, McGraw-Hill Education (Italy)

Con l'indirizzamento aperto si richiede che, per ogni chiave k, la ***sequenza di ispezione***

$$\langle h(k,0), h(k,1), \ldots, h(k,m-1)\rangle$$

sia una permutazione di $\langle 0, 1, \ldots, m-1\rangle$, in modo che ogni posizione della tavola hash possa essere considerata come possibile cella in cui inserire una nuova chiave mentre la tavola si riempie.

Nel seguente pseudocodice supponiamo che gli elementi della tavola hash T siano chiavi senza dati satelliti; la chiave k è identica all'elemento che contiene la chiave k. Ogni cella contiene una chiave o la costante NIL (se la cella è vuota). La procedura HASH-INSERT ha in input una tavola hash T e una chiave k. Essa ritorna il numero della cella in cui ha memorizzato la chiave k oppure segnala un errore se la tavola era già piena.

```
HASH-INSERT(T, k)
1  i = 0
2  repeat
3       j = h(k, i)
4       if T[j] == NIL
5            T[j] = k
6            return j
7       else i = i + 1
8  until i == m
9  error "overflow della tavola hash"
```

L'algoritmo che ricerca la chiave k esamina la stessa sequenza di celle che ha esaminato l'algoritmo di inserimento quando ha inserito la chiave k. Quindi, la ricerca può terminare (senza successo) quando trova una cella vuota, perché la chiave k sarebbe stata inserita lì e non dopo nella sua sequenza di ispezione (questo ragionamento presuppone che le chiavi non vengano cancellate dalla tavola hash). La procedura HASH-SEARCH prende come input una tavola hash T e una chiave k; restituisce j se la cella j contiene la chiave k oppure NIL se la chiave k non si trova nella tavola T.

```
HASH-SEARCH(T, k)
1  i = 0
2  repeat
3       j = h(k, i)
4       if T[j] == k
5            return j
6       i = i + 1
7  until T[j] == NIL or i == m
8  return NIL
```

Acquistato da Michele Michele su Webster il 2022-07-20 12:12 Numero Ordine Libreria 1998088 - 2017 © Copyright © 2021 McGraw-Hill Education (Italy)

La cancellazione da una tavola hash a indirizzamento aperto è un'operazione difficile. Quando cancelliamo una chiave dalla cella i, non possiamo semplicemente marcare questa cella come vuota inserendovi la costante NIL. Così facendo, potrebbe essere impossibile ritrovare qualsiasi chiave k nel cui inserimento abbiamo esaminato la cella i e l'abbiamo trovata occupata. Una soluzione consiste nel marcare la cella registrandovi il valore speciale DELETED, anziché NIL. Dovremo quindi modificare la procedura HASH-INSERT per trattare tale cella come se fosse vuota, in modo da potere inserire una nuova chiave. Nessuna modifica è richiesta

per HASH-SEARCH, perché questa procedura ignora i valori DELETED durante la ricerca. Notate che, quando si usa il valore speciale DELETED, i tempi di ricerca non dipendono più dal fattore di carico α; per questo motivo, viene scelto di preferenza il concatenamento come tecnica di risoluzione delle collisioni quando le chiavi devono essere cancellate.

Nella nostra analisi facciamo l'ipotesi di ***hashing uniforme***: supponiamo che ogni chiave abbia la stessa probabilità di avere come sequenza di ispezione una delle $m!$ permutazioni di $\langle 0, 1, \ldots, m-1\rangle$. L'hashing uniforme estende il concetto di hashing uniforme semplice definito precedentemente al caso in cui la funzione hash produce, non un singolo numero, ma un'intera sequenza di ispezione. Poiché è difficile implementare il vero hashing uniforme, in pratica si usano delle approssimazioni accettabili (come il doppio hashing, definito più avanti).

Esamineremo tre tecniche comunemente utilizzate per calcolare le sequenze di ispezione richieste dall'indirizzamento aperto: ispezione lineare, ispezione quadratica e doppio hashing. Tutte e tre le tecniche garantiscono che $\langle h(k,0), h(k,1), \ldots, h(k,m-1)\rangle$ sia una permutazione di $\langle 0, 1, \ldots, m-1\rangle$ per ogni chiave k. Tuttavia, nessuna di queste tecniche soddisfa l'ipotesi di hashing uniforme, in quanto nessuna di esse è in grado di generare più di m^2 sequenze di ispezione differenti (anziché $m!$, come richiede l'hashing uniforme). Il doppio hashing ha il maggior numero di sequenze di ispezione e, come si può prevedere, fornisce i risultati migliori.

Ispezione lineare

Data una funzione hash ordinaria $h' : U \to \{0, 1, \ldots, m-1\}$, che chiameremo ***funzione hash ausiliaria***, il metodo dell'***ispezione lineare*** usa la funzione hash

$$h(k,i) = (h'(k) + i) \bmod m$$

per $i = 0, 1, \ldots, m-1$. Data la chiave k, la prima cella esaminata è $T[h'(k)]$, che è la cella data dalla funzione hash ausiliaria; la seconda cella esaminata è $T[h'(k)+1]$ e, così via, fino alla cella $T[m-1]$. Poi, l'ispezione riprende dalle celle $T[0], T[1], \ldots$ fino a $T[h'(k)-1]$. Poiché la prima cella ispezionata determina l'intera sequenza di ispezioni, ci sono soltanto m sequenze di ispezione distinte.

L'ispezione lineare è facile da implementare, ma presenta un problema noto come ***addensamento primario***: si formano lunghe file di celle occupate, che aumentano il tempo medio di ricerca. Gli addensamenti si formano perché una cella vuota preceduta da i celle piene ha la probabilità $(i+1)/m$ di essere la prossima a essere occupata. Le lunghe file di celle occupate tendono a diventare sempre più lunghe e il tempo di ricerca medio aumenta.

Ispezione quadratica

L'***ispezione quadratica*** usa una funzione hash della forma

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

$$h(k,i) = (h'(k) + c_1 i + c_2 i^2) \bmod m \tag{11.5}$$

dove h' è una funzione hash ausiliaria, c_1 e $c_2 \neq 0$ sono costanti ausiliarie e $i = 0, 1, \ldots, m-1$. La posizione iniziale esaminata è $T[h'(k)]$; le posizioni successivamente esaminate sono distanziate da quantità che dipendono in modo quadratico dal numero d'ordine di ispezione i. Questa tecnica funziona molto meglio dell'ispezione lineare, ma per fare pieno uso della tavola hash, i valori di c_1,

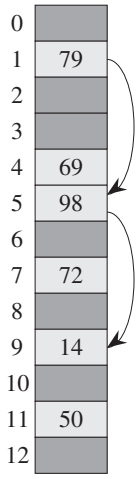


Figura 11.5
Inserimento con doppio hashing. La tavola hash ha dimensione 13 con $h_1(k) = k \bmod 13$ e $h_2(k) = 1 + (k \bmod 11)$. Poiché $14 \equiv 1 \pmod{13}$ e $14 \equiv 3 \pmod{11}$, la chiave 14 viene inserita nella cella vuota 9, dopo che le celle 1 e 5 sono state esaminate e trovate occupate.

c_2 ed m non si possono scegliere arbitrariamente. Il Problema 11-3 illustra un modo per selezionare questi parametri. Inoltre, se due chiavi hanno la stessa posizione iniziale di ispezione, allora le loro sequenze di ispezione sono identiche, perché $h(k_1, 0) = h(k_2, 0)$ implica $h(k_1, i) = h(k_2, i)$. Questa proprietà porta a una forma più lieve di addensamento, che chiameremo ***addensamento secondario***. Come nell'ispezione lineare, la prima posizione determina l'intera sequenza, quindi vengono utilizzate soltanto m sequenze di ispezione distinte.

Doppio hashing

Il doppio hashing è uno dei metodi migliori disponibili per l'indirizzamento aperto, perché le permutazioni prodotte hanno molte delle caratteristiche delle permutazioni scelte a caso. Il ***doppio hashing*** usa una funzione hash della forma

$$h(k, i) = (h_1(k) + i h_2(k)) \bmod m$$

dove h_1 e h_2 sono funzioni hash ausiliarie. L'ispezione inizia dalla posizione $T[h_1(k)]$; le successive posizioni sono distanziate dalle precedenti posizioni di una quantità $h_2(k)$, modulo m. Quindi, diversamente dal caso dell'ispezione lineare o quadratica, la sequenza di ispezione qui dipende in due modi dalla chiave k, perché possono variare sia la posizione iniziale di ispezione sia la distanza fra due posizioni successive di ispezione. La Figura 11.5 illustra un esempio di inserimento con doppio hashing.

Il valore $h_2(k)$ deve essere relativamente primo con la dimensione m della tavola hash perché venga ispezionata l'intera tavola hash (vedere l'Esercizio 11.4-4). Un modo pratico per garantire questa condizione è scegliere m potenza di 2 e definire h_2 in modo che produca sempre un numero dispari. Un altro modo è scegliere m primo e definire h_2 in modo che generi sempre un numero intero positivo minore di m. Per esempio, potremmo scegliere m primo e porre

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned} h_1(k) &= k \bmod m \\ h_2(k) &= 1 + (k \bmod m') \end{aligned}$$

dove m' deve essere scelto un po' più piccolo di m (come $m - 1$). Per esempio, se $k = 123456$, $m = 701$ e $m' = 700$, abbiamo $h_1(k) = 80$ e $h_2(k) = 257$, quindi l'ispezione inizia dalla posizione 80 e si ripete ogni 257 celle (modulo m) finché non sarà trovata la chiave o non saranno esaminate tutte le celle.

Quando m è primo oppure una potenza di due, il doppio hashing è migliore delle ispezioni lineari e quadratiche in quanto usa $\Theta(m^2)$ sequenze di ispezione, anziché $\Theta(m)$, perché ogni possibile coppia $(h_1(k), h_2(k))$ produce una distinta sequenza di ispezione. Di conseguenza, per questi valori di m, le prestazioni del doppio hashing risultano molto prossime a quelle dello schema "ideale" dell'hashing uniforme.

Sebbene, teoricamente, sia possibile utilizzare nel doppio hashing valori di m diversi dai numeri primi o dalle potenze di 2, nella pratica diventa più difficile generare con efficienza $h_2(k)$ in modo tale che sia relativamente primo con m, in parte perché la densità relativa $\phi(m)/m$ di tali numeri potrebbe essere piccola (vedere l'equazione (31.24)).

Analisi dell'hashing a indirizzamento aperto

La nostra analisi dell'indirizzamento aperto, come quella del concatenamento, è espressa in termini del fattore di carico $\alpha = n/m$ della tavola hash. Ovviamente, con l'indirizzamento aperto, abbiamo al massimo un elemento per cella, quindi $n \leq m$, e questo implica che $\alpha \leq 1$.

Supponiamo che venga applicato l'hashing uniforme. In questo schema teorico, la sequenza di ispezione $\langle h(k,0), h(k,1), \ldots, h(k,m-1)\rangle$ utilizzata per inserire o ricercare una chiave k ha la stessa probabilità di essere una qualsiasi permutazione di $\langle 0, 1, \ldots, m-1\rangle$. Ovviamente, una data chiave è associata a un'unica sequenza di ispezione costante; questo significa che, considerando la distribuzione delle probabilità nello spazio delle chiavi e il comportamento della funzione hash sulle chiavi, ogni possibile sequenza di ispezione è ugualmente probabile.

Adesso analizziamo il numero atteso di ispezioni dell'hashing con indirizzamento aperto nell'ipotesi di hashing uniforme, iniziando dall'analisi del numero di ispezioni fatte in una ricerca senza successo.

Teorema 11.6
Nell'ipotesi di hashing uniforme, data una tavola hash a indirizzamento aperto con un fattore di carico $\alpha = n/m < 1$, il numero atteso di ispezioni in una ricerca senza successo è al massimo $1/(1-\alpha)$.

Dimostrazione In una ricerca senza successo, ogni ispezione, tranne l'ultima, accede a una cella occupata che non contiene la chiave desiderata e l'ultima cella esaminata è vuota. Definiamo la variabile casuale X come il numero di ispezioni fatte in una ricerca senza successo; definiamo inoltre A_i (con $i = 1, 2, \ldots$) come l'evento in cui l'i-esima ispezione viene eseguita e trova una cella occupata. Allora l'evento $\{X \geq i\}$ è l'intersezione degli eventi $A_1 \cap A_2 \cap \cdots \cap A_{i-1}$. Limiteremo $\Pr\{X \geq i\}$ limitando $\Pr\{A_1 \cap A_2 \cap \cdots \cap A_{i-1}\}$. Per l'Esercizio C.2-5, si ha

$$\Pr\{A_1 \cap A_2 \cap \cdots \cap A_{i-1}\} = \Pr\{A_1\} \cdot \Pr\{A_2 \mid A_1\} \cdot \Pr\{A_3 \mid A_1 \cap A_2\} \cdots \Pr\{A_{i-1} \mid A_1 \cap A_2 \cap \cdots \cap A_{i-2}\}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Poiché ci sono n elementi ed m celle, $\Pr\{A_1\} = n/m$. Per $j > 1$, la probabilità che la j-esima ispezione trovi una cella occupata, dopo che le prime $j-1$ ispezioni hanno trovato celle occupate, è $(n-j+1)/(m-j+1)$. Questa probabilità deriva dal fatto che dovremmo trovare uno dei restanti $(n-(j-1))$ elementi in una delle $(m-(j-1))$ celle non ancora esaminate e, per l'ipotesi di hashing uniforme, la probabilità è il rapporto di queste quantità. Osservando che $n < m$ implica che $(n-j)/(m-j) \leq n/m$ per ogni j tale che $0 \leq j < m$, allora per

ogni i tale che $1 \leq i \leq m$, si ha

$$\begin{aligned}\Pr\{X \geq i\} &= \frac{n}{m} \cdot \frac{n-1}{m-1} \cdot \frac{n-2}{m-2} \cdots \frac{n-i+2}{m-i+2} \\ &\leq \left(\frac{n}{m}\right)^{i-1} \\ &= \alpha^{i-1}\end{aligned}$$

Adesso utilizziamo l'equazione (C.25) per limitare il numero atteso di ispezioni:

$$\begin{aligned}\mathrm{E}[X] &= \sum_{i=1}^{\infty} \Pr\{X \geq i\} \\ &\leq \sum_{i=1}^{\infty} \alpha^{i-1} \\ &= \sum_{i=0}^{\infty} \alpha^{i} \\ &= \frac{1}{1-\alpha}\end{aligned}$$

■

Questo limite per $1/(1-\alpha) = 1 + \alpha + \alpha^2 + \alpha^3 + \cdots$ ha un'interpretazione intuitiva. Una prima ispezione viene sempre effettuata. Con una probabilità approssimativamente pari ad α, la prima ispezione trova la cella occupata, quindi occorre effettuare una seconda ispezione. Con una probabilità approssimativamente pari ad α^2, le prime due celle sono occupate, quindi occorre effettuare una terza ispezione e così via.

Se α è una costante, il Teorema 11.6 indica che una ricerca senza successo viene eseguita nel tempo $O(1)$. Per esempio, se la tavola hash è piena a metà, il numero medio di ispezioni in una ricerca senza successo è al massimo $1/(1-0,5) = 2$. Se è piena al 90%, il numero medio di ispezioni è al massimo $1/(1-0,9) = 10$.

Il Teorema 11.6 fornisce le prestazioni della procedura HASH-INSERT in maniera quasi immediata.

Corollario 11.7

L'inserimento di un elemento in una tavola hash a indirizzamento aperto con un fattore di carico α richiede in media non più di $1/(1-\alpha)$ ispezioni, nell'ipotesi di hashing uniforme.

Dimostrazione Un elemento viene inserito soltanto se c'è spazio nella tavola e, quindi, $\alpha < 1$. L'inserimento di una chiave richiede una ricerca senza successo seguita dalla sistemazione della chiave nella prima cella vuota che viene trovata. Quindi, il numero atteso di ispezioni è al massimo $1/(1-\alpha)$. ■

Il calcolo del numero atteso di ispezioni per una ricerca con successo richiede un po' più di lavoro.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Teorema 11.8

Data una tavola hash a indirizzamento aperto con un fattore di carico $\alpha < 1$, il numero atteso di ispezioni in una ricerca con successo è al massimo

$$\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$$

supponendo che l'hashing sia uniforme e che ogni chiave nella tavola abbia la stessa probabilità di essere cercata.

Dimostrazione La ricerca di una chiave k segue la stessa sequenza di ispezione che è stata seguita quando è stato inserito l'elemento con chiave k. Per il Corollario 11.7, se k era la $(i+1)$-esima chiave inserita nella tavola hash, il numero atteso di ispezioni fatte in una ricerca di k è al massimo $1/(1-i/m) = m/(m-i)$. Calcolando la media su tutte le n chiavi della tavola hash, si ottiene il numero medio di ispezioni durante una ricerca con successo:

$$\begin{aligned}
\frac{1}{n}\sum_{i=0}^{n-1}\frac{m}{m-i} &= \frac{m}{n}\sum_{i=0}^{n-1}\frac{1}{m-i} \\
&= \frac{1}{\alpha}\sum_{k=m-n+1}^{m}\frac{1}{k} \\
&\leq \frac{1}{\alpha}\int_{m-n}^{m}(1/x)\,dx \quad \text{(per la disequazione (A.12))} \\
&= \frac{1}{\alpha}\ln\frac{m}{m-n} \\
&= \frac{1}{\alpha}\ln\frac{1}{1-\alpha}
\end{aligned}$$

■

Se la tavola hash è piena a metà, il numero atteso delle ispezioni in una ricerca con successo è minore di 1, 387. Se la tavola hash è piena al 90%, il numero atteso di ispezioni è minore di 2, 559.

Esercizi

11.4-1

Supponete di inserire le chiavi 10, 22, 31, 4, 15, 28, 17, 88, 59 in una tavola hash di lunghezza $m = 11$ utilizzando l'indirizzamento aperto con la funzione hash ausiliaria $h'(k) = k \bmod m$. Illustrate il risultato dell'inserimento di queste chiavi utilizzando l'ispezione lineare, l'ispezione quadratica con $c_1 = 1$ e $c_2 = 3$ e il doppio hashing con $h_2(k) = 1 + (k \bmod (m-1))$.

11.4-2

Scrivete uno pseudocodice per HASH-DELETE come delineata nel testo e modificate HASH-INSERT per gestire il valore speciale DELETED.

11.4-3

Considerate una tavola hash a indirizzamento aperto con hashing uniforme. Calcolate i limiti superiori sul numero atteso di ispezioni in una ricerca senza successo e sul numero atteso di ispezioni in una ricerca con successo quando il fattore di carico è $3/4$ e quando è $7/8$.

11.4-4 ⋆

Supponete di applicare la tecnica del doppio hashing per risolvere le collisioni, ovvero utilizzate la funzione hash $h(k,i) = (h_1(k) + i\,h_2(k)) \bmod m$. Dimostrate che, se m e $h_2(k)$ hanno il massimo comune divisore $d \geq 1$ per qualche chiave k, allora una ricerca senza successo della chiave k esamina soltanto una frazione $1/d$ della tavola hash prima di ritornare alla cella iniziale $h_1(k)$. Pertanto, se $d = 1$, e quindi m e $h_2(k)$ sono numeri relativamente primi, la ricerca esamina l'intera tavola hash (*suggerimento:* consultare il Capitolo 31).

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

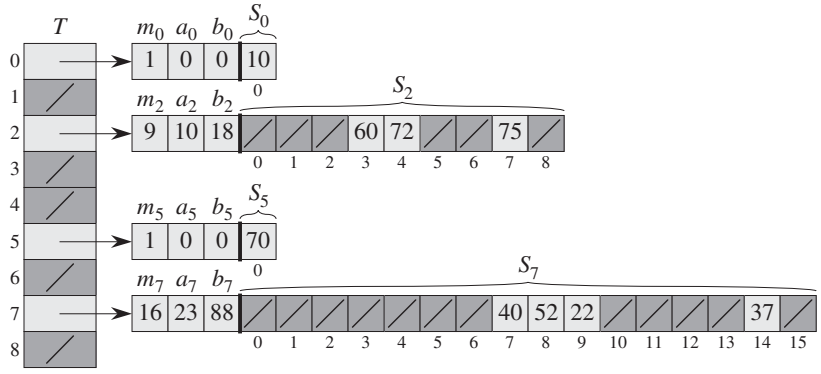


Figura 11.6
Applicazione dell'hashing perfetto per memorizzare l'insieme $K = \{10, 22, 37, 40, 52, 60, 70, 72, 75\}$. La funzione hash esterna è $h(k) = ((ak + b) \bmod p) \bmod m$, dove $a = 3$, $b = 42$, $p = 101$ e $m = 9$. Per esempio, $h(75) = 2$, quindi la chiave 75 viene mappata nella cella 2 della tavola T. Una tavola hash secondaria S_j memorizza tutte le chiavi che vengono associate alla cella j. La dimensione della tavola hash S_j è $m_j = n_j^2$ e la funzione hash associata è $h_j(k) = ((a_j k + b_j) \bmod p) \bmod m_j$. Poiché $h_2(75) = 7$, la chiave 75 viene memorizzata nella cella 7 della tavola hash secondaria S_2. Non ci sono collisioni in nessuna delle tavole hash secondarie, quindi la ricerca richiede un tempo costante nel caso peggiore.

11.4-5 ⋆

Considerate una tavola hash a indirizzamento aperto con un fattore di carico α. Trovate il valore non nullo α per cui il numero atteso di ispezioni in una ricerca senza successo è pari a due volte il numero atteso di ispezioni in una ricerca con successo. Utilizzate i limiti superiori dati dai Teoremi 11.6 e 11.8 per questi numeri attesi di ispezioni.

⋆ 11.5 Hashing perfetto

Sebbene sia utilizzato molto spesso per le sue ottime prestazioni nel caso medio, l'hashing può anche essere utilizzato per ottenere prestazioni eccellenti nel *caso peggiore* quando l'insieme delle chiavi è ***statico***: una volta che le chiavi sono memorizzate nella tavola, l'insieme delle chiavi non cambia più. Alcune applicazioni hanno insiemi di chiavi statici per natura: considerate l'insieme delle parole riservate di un linguaggio di programmazione oppure l'insieme dei nomi dei file registrati in un CD-ROM. Chiamiamo ***hashing perfetto*** una tecnica di hashing secondo la quale il numero di accessi in memoria richiesti per svolgere una ricerca è $O(1)$ nel caso peggiore.

Il concetto di base per creare uno schema di hashing perfetto è semplice. Utilizziamo uno schema di hashing a due livelli, con un hashing universale in ciascun livello. La Figura 11.6 illustra questo approccio.

Il primo livello è essenzialmente lo stesso dell'hashing con concatenamento: le n chiavi sono associate a m celle utilizzando una funzione hash h accuratamente selezionata da una famiglia universale di funzioni hash.

Tuttavia, anziché creare una lista delle chiavi associate alla cella j, utilizziamo una piccola ***tavola hash secondaria*** S_j con una funzione hash h_j associata. Scegliendo le funzioni hash h_j accuratamente, possiamo garantire che non ci siano collisioni al livello secondario.

Acquistato da Michele Michele su Webster il 2022-07-07 23:22 Numero Ordine Libreria: 1995003016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Per garantire che non ci siano collisioni al livello secondario, però, la dimensione m_j della tavola hash S_j dovrà avere il quadrato del numero n_j delle chiavi che si associano alla cella j. Sebbene possa sembrare probabile che tale dipendenza quadratica di m_j da n_j aumenti eccessivamente le esigenze complessive di spazio in memoria, tuttavia dimostreremo che, scegliendo opportunamente la funzione hash al primo livello, la quantità totale attesa di spazio utilizzato resterà $O(n)$.

Utilizziamo le funzioni hash scelte dalle classi universali delle funzioni hash descritte nel Paragrafo 11.3.3. La funzione hash del primo livello è scelta dalla classe $\mathcal{H}_{pm}$ dove, come nel Paragrafo 11.3.3, p è un numero primo maggiore del valore di qualsiasi chiave. Le chiavi che sono associate alla cella j vengono riassociate in una tavola hash secondaria S_j di dimensione m_j, utilizzando una funzione hash h_j scelta dalla classe $\mathcal{H}_{p,m_j}$.[1]

Procederemo in due passaggi. In primo luogo, determineremo come garantire che le tavole secondarie non abbiano collisioni. In secondo luogo, dimostreremo che la quantità attesa di memoria complessivamente utilizzata – per la tavola hash primaria e per tutte le tavole hash secondarie – è $O(n)$.

Teorema 11.9
Se memorizziamo n chiavi in una tavola hash di dimensione $m = n^2$ utilizzando una funzione hash h scelta a caso da una classe universale di funzioni hash, la probabilità che si verifichi una collisione è minore di $1/2$.

Dimostrazione Ci sono $\binom{n}{2}$ coppie di chiavi che possono collidere; ogni coppia collide con probabilità $1/m$, se h è scelta a caso da una famiglia universale $\mathcal{H}$ di funzioni hash. Sia X una variabile casuale che conta il numero di collisioni. Se $m = n^2$, il numero atteso di collisioni è

$$\begin{aligned} \mathrm{E}[X] &= \binom{n}{2} \cdot \frac{1}{n^2} \\ &= \frac{n^2 - n}{2} \cdot \frac{1}{n^2} \\ &< 1/2 \end{aligned}$$

Notate che questa analisi è simile a quella del paradosso del compleanno (Paragrafo 5.4.1). Applicando la disequazione di Markov (C.30), $\Pr\{X \geq t\} \leq \mathrm{E}[X]/t$, con $t = 1$, si conclude la dimostrazione. ■

Nella situazione descritta nel Teorema 11.9, in cui $m = n^2$, una funzione hash h scelta a caso in $\mathcal{H}$ è molto probabile che non abbia *nessuna* collisione. Dato l'insieme K con n chiavi da mappare nella tavola hash (ricordiamo che K è un insieme statico), è quindi facile trovare una funzione hash h esente da collisioni con pochi tentativi casuali.

Quando n è grande, però, una tavola hash di dimensione $m = n^2$ è eccessivamente grande. Pertanto, adottiamo un metodo di hashing a due livelli e utilizziamo l'approccio del Teorema 11.9 soltanto per le chiavi che vanno in una stessa cella. Per mandare le chiavi nelle $m = n$ celle viene utilizzata una funzione hash h esterna (di primo livello). Poi, se n_j chiavi vengono mandate nella cella j, viene utilizzata una tavola hash secondaria S_j di dimensione $m_j = n_j^2$ per effettuare una ricerca senza collisioni in un tempo costante.

Acquistato da Michele Michele su Webster il 2022-07-07 23:16 | Numero Ordine Libraria: 109503016 220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Adesso passiamo al problema di garantire che la memoria complessivamente utilizzata sia $O(n)$. Poiché la dimensione m_j della j-esima tavola hash secondaria

[1] Quando $n_j = m_j = 1$, non occorre necessariamente una funzione hash per la cella j; quando scegliamo una funzione hash $h_{ab}(k) = ((ak + b) \bmod p) \bmod m_j$ per tale cella, utilizziamo semplicemente $a = b = 0$.

cresce con il quadrato del numero n_j delle chiavi memorizzate, c'è il rischio che lo spazio totale occupato in memoria sia eccessivo.

Se la dimensione della tavola del primo livello è $m = n$, allora la quantità di memoria utilizzata è $O(n)$ per la tavola hash primaria, per lo spazio delle dimensioni m_j delle tavole hash secondarie e per lo spazio dei parametri a_j e b_j che definiscono le funzioni hash secondarie h_j estratte dalla classe $\mathcal{H}_{p,m_j}$ descritta nel Paragrafo 11.3.3 (tranne quando $n_j = 1$ e usiamo $a = b = 0$). Il seguente teorema e il successivo corollario forniscono un limite alla dimensione attesa complessiva di tutte le tavole hash secondarie. Un secondo corollario limita la probabilità che la dimensione complessiva di tutte le tavole hash secondarie sia superlineare (più precisamente, che sia maggiore o uguale di $4n$).

Teorema 11.10
Se memorizziamo n chiavi in una tavola hash di dimensione $m = n$ utilizzando una funzione hash h scelta a caso da una classe universale di funzioni hash, si ha

$$\mathrm{E}\left[\sum_{j=0}^{m-1} n_j^2\right] < 2n$$

dove n_j è il numero delle chiavi mandate nella cella j.

Dimostrazione Iniziamo con la seguente identità che vale per qualsiasi intero a non negativo:

$$a^2 = a + 2\binom{a}{2} \tag{11.6}$$

Abbiamo

$$\begin{aligned}
&\mathrm{E}\left[\sum_{j=0}^{m-1} n_j^2\right] \\
&\quad = \mathrm{E}\left[\sum_{j=0}^{m-1}\left(n_j + 2\binom{n_j}{2}\right)\right] && \text{(per l'equazione (11.6))} \\
&\quad = \mathrm{E}\left[\sum_{j=0}^{m-1} n_j\right] + 2\,\mathrm{E}\left[\sum_{j=0}^{m-1}\binom{n_j}{2}\right] && \text{(per la linearità del valore atteso)} \\
&\quad = \mathrm{E}[n] + 2\,\mathrm{E}\left[\sum_{j=0}^{m-1}\binom{n_j}{2}\right] && \text{(per l'equazione (11.1))} \\
&\quad = n + 2\,\mathrm{E}\left[\sum_{j=0}^{m-1}\binom{n_j}{2}\right] && \text{(perché } n \text{ non è una variabile casuale)}
\end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 19950311822070707-0 Copyright © 2022. McGraw-Hill Education (Italy)

Per calcolare la sommatoria $\sum_{j=0}^{m-1}\binom{n_j}{2}$, osserviamo che essa è esattamente il numero totale di coppie di chiavi che collidono nella tavola hash. Per la proprietà dell'hashing universale, il valore atteso di questa sommatoria è al massimo

$$\binom{n}{2}\frac{1}{m} = \frac{n(n-1)}{2m} = \frac{n-1}{2}$$

in quanto $m = n$. Quindi

$$\begin{aligned} \mathrm{E}\left[\sum_{j=0}^{m-1} n_j^2\right] &\leq n + 2\,\frac{n-1}{2} \\ &= 2n - 1 \\ &< 2n \end{aligned}$$

■

Corollario 11.11
Se memorizziamo n chiavi in una tavola hash di dimensione $m = n$ utilizzando una funzione hash h scelta a caso da una classe universale di funzioni hash e impostiamo la dimensione di ogni tavola hash secondaria a $m_j = n_j^2$ per $j = 0, 1, \ldots, m-1$, allora la quantità attesa di memoria richiesta per tutte le tavole hash secondarie in uno schema di hashing perfetto è minore di $2n$.

Dimostrazione Poiché $m_j = n_j^2$ per $j = 0, 1, \ldots, m-1$, dal Teorema 11.10 otteniamo

$$\mathrm{E}\left[\sum_{j=0}^{m-1} m_j\right] = \mathrm{E}\left[\sum_{j=0}^{m-1} n_j^2\right] < 2n \tag{11.7}$$

che completa la dimostrazione. ■

Corollario 11.12
Se memorizziamo n chiavi in una tavola hash di dimensione $m = n$ utilizzando una funzione hash h scelta a caso da una classe di funzioni hash e impostiamo la dimensione di ogni tavola hash secondaria a $m_j = n_j^2$ per $j = 0, 1, \ldots, m-1$, la probabilità che la memoria totale utilizzata per le tavole hash secondarie sia maggiore o uguale di $4n$ è minore di $1/2$.

Dimostrazione Applichiamo nuovamente la disequazione di Markov (C.30), $\Pr\{X \geq t\} \leq \mathrm{E}[X]/t$, questa volta alla disequazione (11.7) con $X = \sum_{j=0}^{m-1} m_j$ e $t = 4n$:

$$\begin{aligned} \Pr\left\{\sum_{j=0}^{m-1} m_j \geq 4n\right\} &\leq \frac{\mathrm{E}\left[\sum_{j=0}^{m-1} m_j\right]}{4n} \\ &< \frac{2n}{4n} \\ &= 1/2 \end{aligned}$$

■

Dal Corollario 11.12 osserviamo che, provando poche funzioni hash scelte a caso da una famiglia universale, se ne ottiene rapidamente una che usa una quantità di memoria accettabile.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Esercizi

11.5-1 ⋆

Supponete di inserire n chiavi in una tavola hash di dimensione m utilizzando l'indirizzamento aperto e l'hashing uniforme. Indicate con $p(n, m)$ la probabilità che non si verifichino collisioni. Dimostrate che $p(n, m) \leq e^{-n(n-1)/2m}$ (*suggerimento:* esaminate l'equazione (3.12)). Dimostrate che, quando n supera $\sqrt{m}$, la probabilità di evitare le collisioni tende rapidamente a zero.

Problemi

11-1 Limite sul numero di ispezioni nell'hashing

Una tavola hash con indirizzamento aperto di dimensione m è utilizzata per memorizzare n elementi, con $n \leq m/2$.

a. Nell'ipotesi di hashing uniforme, dimostrate che per $i = 1, 2, \ldots, n$, la probabilità che l'i-esimo inserimento richieda più di k ispezioni è al massimo 2^{-k}.

b. Dimostrate che per $i = 1, 2, \ldots, n$, la probabilità che l'i-esimo inserimento richieda più di $2 \lg n$ ispezioni è $O(1/n^2)$.

Indicate con la variabile casuale X_i il numero di ispezioni richieste dall'i-esimo inserimento. Avete dimostrato nel punto (b) che $\Pr\{X_i > 2 \lg n\} \leq 1/n^2$. Indicate con la variabile casuale $X = \max_{1 \leq i \leq n} X_i$ il numero massimo di ispezioni richieste da uno qualsiasi degli n inserimenti.

c. Dimostrate che $\Pr\{X > 2 \lg n\} \leq 1/n$.

d. Dimostrate che la lunghezza attesa $\mathrm{E}[X]$ della sequenza di ispezione più lunga è $O(\lg n)$.

11-2 Limite sulla dimensione delle celle nel concatenamento

Supponete di avere una tavola hash con n celle e di risolvere le collisioni con il concatenamento; supponete inoltre che n chiavi siano inserite nella tavola. Ogni chiave ha la stessa probabilità di essere associata a ciascuna cella. Indicate con M il numero massimo di chiavi in una cella dopo che tutte le chiavi sono state inserite. Il vostro compito è dimostrare un limite superiore $O(\lg n / \lg \lg n)$ su $\mathrm{E}[M]$, il valore atteso di M.

a. Dimostrate che la probabilità Q_k che k chiavi siano mandate in una particolare cella è data da

$$Q_k = \left(\frac{1}{n}\right)^k \left(1 - \frac{1}{n}\right)^{n-k} \binom{n}{k}$$

b. Sia P_k la probabilità che $M = k$, cioè la probabilità che la cella con il maggior numero di chiavi contenga k chiavi. Dimostrate che $P_k \leq n Q_k$.

c. Applicate la formula di approssimazione di Stirling (equazione (3.18)) per dimostrare che $Q_k < e^k / k^k$.

d. Dimostrate che esiste una costante $c > 1$ tale che $Q_{k_0} < 1/n^3$ per $k_0 = c \lg n / \lg \lg n$. Concludete che $P_k < 1/n^2$ per $k \geq k_0 = c \lg n / \lg \lg n$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

e. Dimostrate che

$$\mathrm{E}[M] \leq \Pr\left\{M > \frac{c \lg n}{\lg \lg n}\right\} \cdot n + \Pr\left\{M \leq \frac{c \lg n}{\lg \lg n}\right\} \cdot \frac{c \lg n}{\lg \lg n}$$

Concludete che $\mathrm{E}[M] = O(\lg n / \lg \lg n)$.

11-3 Ispezione quadratica

Supponete di avere una chiave k da cercare in una tavola hash con le posizioni $0, 1, \ldots, m-1$; supponete inoltre di avere una funzione hash h che manda dallo spazio delle chiavi all'insieme $\{0, 1, \ldots, m-1\}$. Lo schema di ricerca è il seguente.

1. Calcolate il valore $i = h(k)$ e ponete $j = 0$.
2. Cercate la chiave desiderata k nella posizione i. Se la trovate o se questa posizione è vuota, terminate la ricerca.
3. Ponete $i = i + 1$. Se i è diventato uguale a m la tavola è piena e in questo caso terminate la ricerca. Altrimenti ponete $j = (j + 1) \bmod m$ e ritornate al passo 2.

Supponete che m sia una potenza di 2.

a. Dimostrate che questo schema è un'istanza dello schema generale dell'"ispezione quadratica" mettendo in evidenza le costanti appropriate c_1 e c_2 per l'equazione (11.5).

b. Dimostrate che nel caso peggiore questo algoritmo esamina tutte le posizioni della tavola.

11-4 Hashing e autenticazione

Sia $\mathcal{H}$ una classe di funzioni hash in cui ogni funzione $h \in \mathcal{H}$ manda dall'universo U delle chiavi a $\{0, 1, \ldots, m-1\}$. Diciamo che $\mathcal{H}$ è ***k-universale*** se, per ogni sequenza costante di k chiavi distinte $\langle x^{(1)}, x^{(2)}, \ldots, x^{(k)} \rangle$ e per qualsiasi h scelta a caso da $\mathcal{H}$, la sequenza $\langle h(x^{(1)}), h(x^{(2)}), \ldots, h(x^{(k)}) \rangle$ ha la stessa probabilità di essere una delle m^k sequenze di lunghezza k con elementi estratti da $\{0, 1, \ldots, m-1\}$.

a. Dimostrate che, se la famiglia $\mathcal{H}$ delle funzioni hash è 2-universale, allora è universale.

b. Supponete che l'universo U sia l'insieme di n-tuple di valori estratti da $\mathbb{Z}_p = \{0, 1, \ldots, p-1\}$, dove p è un numero primo. Considerate un elemento $x = \langle x_0, x_1, \ldots, x_{n-1} \rangle \in U$. Per qualsiasi n-tupla $a = \langle a_0, a_1, \ldots, a_{n-1} \rangle \in U$, definite la funzione hash h_a in questo modo

$$h_a(x) = \left(\sum_{j=0}^{n-1} a_j x_j \right) \bmod p$$

e sia $\mathcal{H} = \{h_a\}$. Dimostrate che $\mathcal{H}$ è universale, ma non 2-universale. (*Suggerimento:* trovate una chiave per la quale tutte le funzioni hash in $\mathcal{H}$ producono lo stesso valore.)

c. Modificate leggermente $\mathcal{H}$ rispetto al punto (b): per qualsiasi $a \in U$ e per qualsiasi $b \in \mathbb{Z}_p$, definite

Acquistato da Michele Michelucci Michelucci il 2022-07-17 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$h'_{ab}(x) = \left(\sum_{j=0}^{n-1} a_j x_j + b \right) \bmod p$$

e $\mathcal{H}' = \{h'_{ab}\}$. Dimostrate che $\mathcal{H}'$ è 2-universale. (*Suggerimento:* considerate due prefissate n-tuple $x \in U$ e $y \in U$, con $x_i \neq y_i$ per qualche i. Che cosa accade a $h'_{ab}(x)$ e $h'_{ab}(y)$ quando a_i e b vanno oltre $\mathbb{Z}_p$?)

d. Alice e Bob si sono accordati in segreto su una funzione hash h di una famiglia $\mathcal{H}$ 2-universale di funzioni hash. Ogni $h \in \mathcal{H}$ manda da un universo di chiavi U a $\mathbb{Z}_p$, dove p è un numero primo. Successivamente, Alice invia un messaggio m a Bob tramite Internet, con $m \in U$. Alice autentica questo messaggio inviando anche un'etichetta di autenticazione $t = h(m)$; Bob verifica che la coppia (m, t) che ha ricevuto soddisfa davvero $t = h(m)$. Supponete che un avversario intercetti (m, t) durante la trasmissione e tenti di imbrogliare Bob sostituendo la coppia (m, t) con una coppia differente (m', t'). Dimostrate che la probabilità che l'avversario riesca a ingannare Bob facendogli accettare la coppia (m', t') è al massimo $1/p$, indipendentemente da quanta potenza di calcolo abbia a disposizione l'avversario e indipendentemente dal fatto che l'avversario conosca la famiglia $\mathcal{H}$ delle funzioni hash utilizzate.

Note

Knuth [212] e Gonnet [146] sono eccellenti testi di riferimento per l'analisi degli algoritmi di hashing. Knuth attribuisce a H. P. Luhn (1953) l'invenzione delle tavole hash e del metodo di concatenamento per risolvere le collisioni. All'incirca nello stesso periodo, G. M. Amdahl ideò l'indirizzamento aperto.

Nel 1979, Carter e Wegman svilupparono la nozione di classi universali di funzioni hash [59]. Fredman, Komlós e Szemerédi [113] progettarono lo schema dell'hashing perfetto per gli insiemi statici presentato nel Paragrafo 11.5. Dietzfelbinger e altri autori [87] hanno esteso questo metodo agli insiemi dinamici, riuscendo a gestire inserimenti e cancellazioni nel tempo atteso ammortizzato pari a $O(1)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Alberi binari di ricerca 12

Gli alberi di ricerca sono strutture dati che supportano molte operazioni sugli insiemi dinamici, fra le quali SEARCH, MINIMUM, MAXIMUM, PREDECESSOR, SUCCESSOR, INSERT e DELETE. Quindi, un albero di ricerca può essere utilizzato sia come dizionario sia come coda di priorità. Le operazioni di base su un albero binario di ricerca richiedono un tempo proporzionale all'altezza dell'albero. Per un albero binario completo con n nodi, tali operazioni sono eseguite nel tempo $\Theta(\lg n)$ nel caso peggiore. Se, invece, l'albero è una catena lineare di n nodi, le stesse operazioni richiedono un tempo $\Theta(n)$ nel caso peggiore. Come vedremo nel Paragrafo 12.4, l'altezza attesa di un albero binario di ricerca costruito in modo casuale è $O(\lg n)$, quindi le operazioni elementari degli insiemi dinamici svolte su questo tipo di albero richiedono in media il tempo $\Theta(\lg n)$. In pratica, non possiamo sempre garantire che gli alberi binari di ricerca possano essere costruiti in modo casuale, tuttavia esistono delle varianti di alberi binari di ricerca che assicurano buone prestazioni nel caso peggiore. Il Capitolo 13 presenta una di queste varianti: gli alberi rosso-neri, che hanno altezza $O(\lg n)$. Il Capitolo 18 introduce i B-alberi, che sono particolarmente adatti a mantenere i database nella memoria secondaria (disco).

Dopo avere presentato le proprietà fondamentali degli alberi binari di ricerca, i paragrafi successivi descriveranno come attraversare un albero binario di ricerca per visualizzarne ordinatamente i valori, come ricercare un valore in un albero binario di ricerca, come trovare l'elemento minimo o massimo, come trovare il predecessore o il successore di un elemento e come inserire o cancellare un elemento da un albero binario di ricerca. Le proprietà matematiche fondamentali degli alberi sono descritte nell'Appendice B.

12.1 Che cos'è un albero binario di ricerca?

Un albero binario di ricerca è organizzato, come suggerisce il nome, in un albero binario (Figura 12.1), che può essere rappresentato da una struttura dati concatenata in cui ogni nodo è un oggetto. Oltre a una chiave (*key*) e ai dati satelliti, ogni nodo dell'albero contiene gli attributi *left*, *right* e *p* che puntano ai nodi che corrispondono, rispettivamente, al figlio sinistro, al figlio destro e al padre del nodo. Se manca un figlio o il padre, i corrispondenti attributi contengono il valore NIL. Il nodo radice (*root*) è l'unico nodo nell'albero il cui attributo padre è NIL.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Le chiavi in un albero binario di ricerca sono sempre memorizzate in modo da soddisfare la ***proprietà degli alberi binari di ricerca***:

> Sia x un nodo in un albero binario di ricerca. Se y è un nodo nel sottoalbero sinistro di x, allora $y.key \leq x.key$. Se y è un nodo nel sottoalbero destro di x, allora $y.key \geq x.key$.

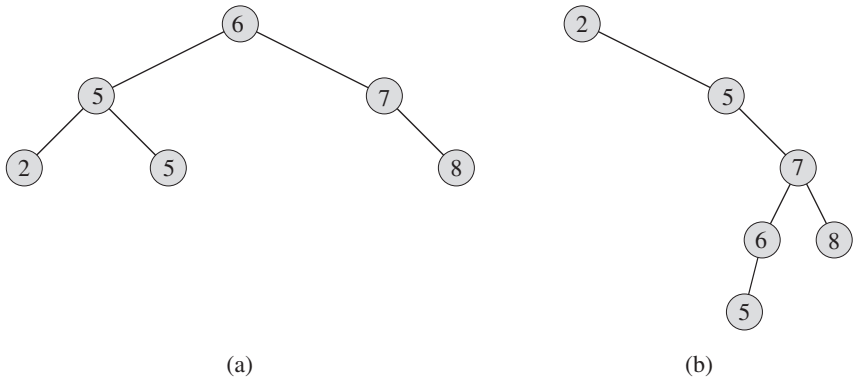


Figura 12.1 Alberi binari di ricerca. Per qualsiasi nodo x, le chiavi nel sottoalbero sinistro di x sono al massimo $x.key$ e le chiavi nel sottoalbero destro di x sono almeno $x.key$. Alberi binari di ricerca differenti possono rappresentare lo stesso insieme di valori. Il tempo di esecuzione nel caso peggiore per la maggior parte delle operazioni di ricerca in un albero è proporzionale all'altezza dell'albero. **(a)** Un albero binario di ricerca con 6 nodi e altezza 2. **(b)** Un albero binario di ricerca meno efficiente di altezza 4 che contiene le stesse chiavi.

Così, nella Figura 12.1(a), la chiave della radice è 5, le chiavi 2, 3 e 5 nel suo sottoalbero sinistro non sono maggiori di 5, e le chiavi 7 e 8 nel suo sottoalbero destro non sono minori di 5. La stessa proprietà vale per qualsiasi nodo dell'albero. Per esempio, la chiave 5 nella Figura 12.1(a) non è minore della chiave 2 nel suo sottoalbero sinistro e non è maggiore della chiave 5 nel suo sottoalbero destro.

La proprietà degli alberi binari di ricerca consente di elencare ordinatamente tutte le chiavi di un albero binario di ricerca con un semplice algoritmo ricorsivo di ***attraversamento simmetrico di un albero*** (inorder). Questo algoritmo è così chiamato perché la chiave della radice di un sottoalbero viene stampata nel mezzo tra la stampa dei valori nel sottoalbero sinistro e la stampa dei valori nel sottoalbero destro. Analogamente, un algoritmo di ***attraversamento anticipato di un albero*** (preorder) stampa la radice prima dei valori dei suoi sottoalberi e un algoritmo di ***attraversamento posticipato di un albero*** (postorder) stampa la radice dopo i valori dei suoi sottoalberi. Con la chiamata INORDER-TREE-WALK($T.root$) la seguente procedura stampa tutti gli elementi di un albero binario di ricerca T.

```
INORDER-TREE-WALK(x)
1  if x ≠ NIL
2      INORDER-TREE-WALK(x.left)
3      stampa x.key
4      INORDER-TREE-WALK(x.right)
```

Per esempio, l'attraversamento simmetrico stampa le chiavi in ciascuno dei due alberi binari di ricerca illustrati nella Figura 12.1 nel seguente ordine: $2, 3, 5, 5, 7, 8$. La correttezza dell'algoritmo si ricava direttamente per induzione dalla proprietà degli alberi binari di ricerca.

Occorre un tempo $\Theta(n)$ per attraversare un albero binario di ricerca di n nodi, perché, dopo la chiamata iniziale, la procedura viene chiamata ricorsivamente esattamente due volte per ogni nodo dell'albero – una volta per il figlio sinistro e una volta per il figlio destro. Il seguente teorema fornisce una prova più formale che occorre un tempo lineare per un attraversamento simmetrico di un albero.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Teorema 12.1

Se x è la radice di un sottoalbero di n nodi, la chiamata INORDER-TREE-WALK(x) richiede il tempo $\Theta(n)$.

Dimostrazione Sia $T(n)$ il tempo richiesto dalla procedura INORDER-TREE-WALK quando viene chiamata per la radice di un sottoalbero di n nodi. Poiché INORDER-TREE-WALK visita tutti gli n nodi del sottoalbero, si ha $T(n) = \Omega(n)$. Resta da dimostrare che $T(n) = O(n)$.

INORDER-TREE-WALK richiede una piccola quantità costante di tempo con un sottoalbero vuoto (per il test $x \neq \text{NIL}$), quindi $T(0) = c$ per qualche costante positiva c.

Per $n > 0$, supponete che INORDER-TREE-WALK sia chiamata per un nodo x il cui sottoalbero sinistro ha k nodi e il cui sottoalbero destro ha $n - k - 1$ nodi. Il tempo per eseguire INORDER-TREE-WALK(x) è $T(n) \leq T(k) + T(n - k - 1) + d$ per qualche costante positiva d che rappresenta un limite superiore per il tempo per eseguire INORDER-TREE-WALK(x), escludendo il tempo impiegato nelle chiamate ricorsive.

Applichiamo il metodo di sostituzione per provare che $T(n) = \Theta(n)$ dimostrando che $T(n) \leq (c+d)n+c$. Per $n = 0$, abbiamo $(c+d)\cdot 0+c = c = T(0)$. Per $n > 0$, abbiamo

$$\begin{aligned} T(n) &\leq T(k) + T(n-k-1) + d \\ &= ((c+d)k + c) + ((c+d)(n-k-1) + c) + d \\ &= (c+d)n + c - (c+d) + c + d \\ &= (c+d)n + c \end{aligned}$$

che completa la dimostrazione. ■

Esercizi

12.1-1

Disegnate degli alberi binari di ricerca di altezza 2, 3, 4, 5 e 6 per l'insieme delle chiavi $\{1, 4, 5, 10, 16, 17, 21\}$.

12.1-2

Qual è la differenza fra la proprietà degli alberi binari di ricerca e la proprietà del min-heap (vedere pagina 128)? È possibile utilizzare la proprietà del min-heap per elencare ordinatamente le chiavi di un albero di n nodi nel tempo $O(n)$? Spiegate come o perché no.

12.1-3

Scrivete un algoritmo non ricorsivo che effettua un attraversamento simmetrico di un albero (*suggerimento:* c'è una soluzione semplice che usa uno stack come struttura dati ausiliaria e una soluzione elegante, ma più complicata, che non usa lo stack, ma suppone che si possa controllare l'uguaglianza di due puntatori).

12.1-4

Scrivete gli algoritmi ricorsivi che svolgono gli attraversamenti anticipato e posticipato di un albero di n nodi nel tempo $\Theta(n)$.

12.1-5

Dimostrate che, poiché l'ordinamento per confronti di n elementi richiede il tempo $\Omega(n \lg n)$ nel caso peggiore, qualsiasi algoritmo basato sui confronti che viene utilizzato per costruire un albero binario di ricerca da una lista arbitraria di n elementi richiede il tempo $\Omega(n \lg n)$ nel caso peggiore.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

12.2 Interrogazione di un albero binario di ricerca

Una tipica operazione svolta su albero binario di ricerca è quella di cercare una chiave memorizzata nell'albero. Oltre all'operazione SEARCH, gli alberi binari di ricerca supportano interrogazioni (query) quali MINIMUM, MAXIMUM, SUC-

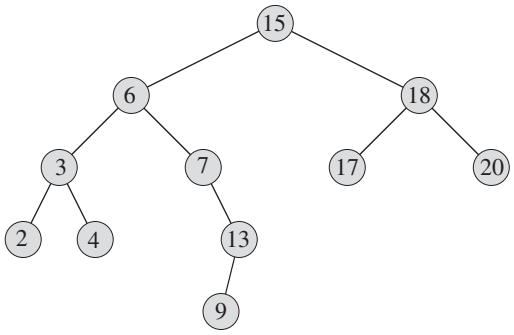


Figura 12.2
Interrogazioni in un albero binario di ricerca. Per cercare la chiave 13 nell'albero, seguiamo il percorso $15 \to 6 \to 7 \to 13$ dalla radice. La chiave minima è 2, che può essere trovata seguendo i puntatori *left* dalla radice. La chiave massima 20 si trova seguendo i puntatori *right* dalla radice. Il successore del nodo con la chiave 15 è il nodo con la chiave 17, perché è la chiave minima nel sottoalbero destro di 15. Il nodo con la chiave 13 non ha un sottoalbero destro, quindi il suo successore è il suo antenato più prossimo il cui figlio sinistro è anch'esso un suo antenato. In questo caso, il nodo con la chiave 15 è il suo successore.

CESSOR e PREDECESSOR. In questo paragrafo, esamineremo queste operazioni e dimostreremo che ciascuna di esse può essere eseguita nel tempo $O(h)$ in un albero binario di altezza h.

Ricerca

Utilizziamo la seguente procedura per cercare un nodo con una data chiave in un albero binario di ricerca. Dato un puntatore alla radice dell'albero e una chiave k, TREE-SEARCH restituisce un puntatore a un nodo con chiave k, se esiste, altrimenti restituisce il valore NIL.

```
TREE-SEARCH(x, k)
1  if x == NIL or k == x.key
2       return x
3  if k < x.key
4       return TREE-SEARCH(x.left, k)
5  else return TREE-SEARCH(x.right, k)
```

La procedura inizia la sua ricerca dalla radice e segue un cammino semplice verso il basso lungo l'albero, come illustra la Figura 12.2. Per ogni nodo x che incontra, confronta la chiave k con $x.key$. Se le due chiavi sono uguali, la ricerca termina. Se k è minore di $x.key$, la ricerca continua nel sottoalbero sinistro di x, in quanto la proprietà degli alberi binari di ricerca implica che k non può essere memorizzata nel sottoalbero destro. Simmetricamente, se k è maggiore di $x.key$, la ricerca continua nel sottoalbero destro. I nodi incontrati durante la ricorsione formano un cammino semplice verso il basso dalla radice dell'albero, quindi il tempo di esecuzione di TREE-SEARCH è $O(h)$, dove h è l'altezza dell'albero.

La stessa procedura può essere scritta in modo iterativo "srotolando" la ricorsione in un ciclo **while**. Questa versione è più efficiente nella maggior parte dei calcolatori.

```
ITERATIVE-TREE-SEARCH(x, k)
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

```
1  while x ≠ NIL and k ≠ x.key
2       if k < x.key
3            x = x.left
4       else x = x.right
5  return x
```

Minimo e massimo

Un elemento con chiave minima in un albero binario di ricerca può sempre essere trovato seguendo, a partire dalla radice, i puntatori *left* dei figli a sinistra, fino a quando non viene incontrato un valore NIL, come illustra la Figura 12.2. La seguente procedura restituisce un puntatore all'elemento minimo nel sottoalbero con radice in un nodo x, che supponiamo diverso da NIL.

```
TREE-MINIMUM(x)
1  while x.left ≠ NIL
2      x = x.left
3  return x
```

La proprietà degli alberi binari di ricerca garantisce la correttezza della procedura TREE-MINIMUM. Se un nodo x non ha un sottoalbero sinistro, allora poiché ogni chiave nel sottoalbero destro di x è almeno grande quanto $x.key$, la chiave minima nel sottoalbero con radice in x è $x.key$. Se il nodo x ha un sottoalbero sinistro, allora poiché nessuna chiave nel sottoalbero destro è minore di $x.key$ e ogni chiave nel sottoalbero sinistro non è maggiore di $x.key$, la chiave minima nel sottoalbero con radice in x può essere trovata nel sottoalbero con radice in $x.left$.

Lo pseudocodice per TREE-MAXIMUM è simmetrico.

```
TREE-MAXIMUM(x)
1  while x.right ≠ NIL
2      x = x.right
3  return x
```

Entrambe queste procedure vengono eseguite nel tempo $O(h)$ in un albero di altezza h perché, come in TREE-SEARCH, la sequenza dei nodi incontrati forma un cammino semplice che scende dalla radice.

Successore e predecessore

Dato un nodo in un albero binario di ricerca, a volte è importante trovare il suo successore nell'ordine stabilito da un attraversamento simmetrico. Se tutte le chiavi sono distinte, il successore di un nodo x è il nodo con la più piccola chiave che è maggiore di $x.key$. La struttura di un albero binario di ricerca ci consente di determinare il successore di un nodo senza mai confrontare le chiavi. La seguente procedura restituisce il successore di un nodo x in un albero binario di ricerca, se esiste, oppure NIL se x ha la chiave massima nell'albero.

```
TREE-SUCCESSOR(x)
1  if x.right ≠ NIL
2      return TREE-MINIMUM(x.right)
3  y = x.p
4  while y ≠ NIL and x == y.right
5      x = y
6      y = y.p
7  return y
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Il codice della procedura TREE-SUCCESSOR prevede due casi. Se il sottoalbero destro del nodo x non è vuoto, allora il successore di x è proprio il nodo più a sinistra nel sottoalbero destro, che viene trovato nella riga 2 chiamando TREE-MINIMUM($x.right$). Per esempio, il successore del nodo con la chiave 15 nella Figura 12.2 è il nodo con la chiave 17.

D'altra parte, come chiede di dimostrare l'Esercizio 12.2-6, se il sottoalbero destro del nodo x è vuoto e x ha un successore y, allora y è l'antenato più prossimo di x il cui figlio sinistro è anche antenato di x. Nella Figura 12.2, il successore del nodo con la chiave 13 è il nodo con la chiave 15. Per trovare y, semplicemente risaliamo l'albero partendo da x, finché incontriamo un nodo che è il figlio sinistro di suo padre; questa operazione è svolta dalle righe 3–7 di TREE-SUCCESSOR.

Il tempo di esecuzione di TREE-SUCCESSOR in un albero di altezza h è $O(h)$, perché seguiamo un cammino semplice che sale o uno che scende. Anche la procedura TREE-PREDECESSOR, che è simmetrica a TREE-SUCCESSOR, viene eseguita nel tempo $O(h)$.

Anche se le chiavi non sono distinte, definiamo successore e predecessore di qualsiasi nodo x quel nodo che viene restituito, rispettivamente, dalle chiamate di TREE-SUCCESSOR(x) e TREE-PREDECESSOR(x).

In sintesi, abbiamo dimostrato il seguente teorema.

Teorema 12.2

Le operazioni sugli insiemi dinamici SEARCH, MINIMUM, MAXIMUM, SUCCESSOR e PREDECESSOR possono essere svolte nel tempo $O(h)$ in un albero binario di ricerca di altezza h. ■

Esercizi

12.2-1

Supponete che un albero binario di ricerca contenga i numeri compresi fra 1 e 1000 e vogliate cercare il numero 363. Quale delle seguenti sequenze *non* può essere la sequenza dei nodi esaminata?

a. 2, 252, 401, 398, 330, 344, 397, 363.

b. 924, 220, 911, 244, 898, 258, 362, 363.

c. 925, 202, 911, 240, 912, 245, 363.

d. 2, 399, 387, 219, 266, 382, 381, 278, 363.

e. 935, 278, 347, 621, 299, 392, 358, 363.

12.2-2

Scrivete le versioni ricorsive di TREE-MINIMUM e TREE-MAXIMUM.

12.2-3

Scrivete la procedura TREE-PREDECESSOR.

12.2-4

Il professor Bunyan crede di avere scoperto un'importante proprietà degli alberi binari di ricerca. Supponete che la ricerca della chiave k in un albero binario di ricerca termini in una foglia. Considerate tre insiemi: A, le chiavi a sinistra del percorso di ricerca; B, le chiavi lungo il percorso di ricerca; C, le chiavi a destra del percorso di ricerca. Il professor Bunyan sostiene che tre chiavi qualsiasi $a \in A$, $b \in B$ e $c \in C$ devono soddisfare la relazione $a \leq b \leq c$. Trovate un controesempio più piccolo possibile alla tesi del professore.

Acquistato da Michele Michele su Webster il 2022-07-27 per uso esclusivo di Utenti Aziende, 1165-21dd-120d-0. Copyright © 2022, McGraw-Hill Education (Italy)

12.2-5

Dimostrate che se un nodo in un albero binario di ricerca ha due figli, allora il suo successore non ha un figlio sinistro e il suo predecessore non ha un figlio destro.

12.2-6
Considerate un albero binario di ricerca T le cui chiavi sono distinte. Dimostrate che se il sottoalbero destro di un nodo x in T è vuoto e x ha un successore y, allora y è l'antenato più prossimo di x il cui figlio sinistro è anche un antenato di x (ricordiamo che ogni nodo viene considerato antenato di sé stesso).

12.2-7
L'attraversamento simmetrico di un albero binario di ricerca di n nodi può essere implementato trovando l'elemento minimo nell'albero con la procedura TREE-MINIMUM e, poi, effettuando $n - 1$ chiamate di TREE-SUCCESSOR. Dimostrate che questo algoritmo viene eseguito nel tempo $\Theta(n)$.

12.2-8
Dimostrate che, in un albero binario di ricerca di altezza h, k successive chiamate di TREE-SUCCESSOR richiedono il tempo $O(k + h)$, indipendentemente dal nodo di partenza.

12.2-9
Siano T un albero binario di ricerca con chiavi distinte, x un nodo foglia e y suo padre. Dimostrate che $key[y]$ è la più piccola chiave di T che è maggiore di $x.key$ o la più grande chiave di T che è minore di $x.key$.

12.3 Inserimento e cancellazione

Le operazioni di inserimento e cancellazione modificano l'insieme dinamico rappresentato da un albero binario di ricerca. La struttura dati deve essere modificata per riflettere questa modifica, ma in modo tale che la proprietà degli alberi binari di ricerca resti valida. Come vedremo, modificare l'albero per inserire un nuovo elemento è relativamente semplice, ma gestire le cancellazioni è un poco più complicato.

Inserimento

Per inserire un nuovo valore v in un albero binario di ricerca T, utilizziamo la procedura TREE-INSERT. Questa procedura riceve un nodo z per il quale $z.key = v$, $z.left =$ NIL e $z.right =$ NIL; essa modifica T e qualche attributo di z in modo che z sia inserito in una posizione appropriata nell'albero.

La Figura 12.3 illustra il funzionamento di TREE-INSERT. Esattamente come le procedure TREE-SEARCH e ITERATIVE-TREE-SEARCH, TREE-INSERT inizia dalla radice dell'albero e il puntatore x traccia un cammino semplice in discesa cercando un NIL da sostituire con l'elemento di input z. La procedura mantiene anche un puntatore *inseguitore* y che punta al padre di x. Dopo l'inizializzazione, le righe 3–7 del ciclo **while** spostano questi due puntatori verso il basso, andando a sinistra o a destra a seconda dell'esito del confronto fra $z.key$ e $x.key$, finché a x non viene assegnato il valore NIL. Ci serve un puntatore inseguitore y perché, quando arriviamo a trovare il NIL dove mettere z, la ricerca è andata un passo oltre il nodo che deve essere modificato. Le righe 8–13 impostano i puntatori che servono a inserire z.

Acquistato da Michele Mannino (mannino.michele@gmail.com) - Numero Ordine Libreria 2095330 - Proprietà Copyright 2022 McGraw-Hill Education (Italy)

Analogamente alle altre operazioni elementari con gli alberi di ricerca, la procedura TREE-INSERT viene eseguita nel tempo $O(h)$ in un albero di altezza h.

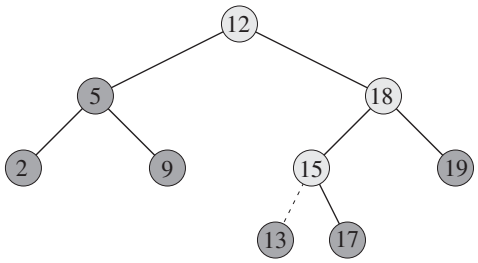


Figura 12.3 L'elemento con la chiave 13 viene inserito in un albero binario di ricerca. I nodi più chiari indicano il cammino semplice dalla radice verso la posizione dove l'elemento viene inserito. La linea tratteggiata indica il collegamento che è stato aggiunto per inserire l'elemento.

```
TREE-INSERT(T, z)
 1  y = NIL
 2  x = T.root
 3  while x ≠ NIL
 4       y = x
 5       if z.key < x.key
 6            x = x.left
 7       else x = x.right
 8  z.p = y
 9  if y == NIL
10       T.root = z          // L'albero T era vuoto
11  elseif z.key < y.key
12       y.left = z
13  else y.right = z
```

Cancellazione

La procedura per cancellare un nodo z da un albero binario di ricerca considera tre casi base ma, come vedremo, uno solo di essi è complicato.

- Se z non ha figli, modifichiamo suo padre $p[z]$ per sostituire z con NIL come suo figlio.
- Se il nodo z ha un solo figlio, eleviamo questo figlio in modo da occupare la posizione di z nell'albero, modificando il padre di z per sostituire z con il figlio di z.
- Se il nodo z ha due figli, troviamo il successore y di z – che deve trovarsi nel sottoalbero destro di z – e facciamo in modo che y assuma la posizione di z nell'albero. La parte restante del sottoalbero destro originale diventa il nuovo sottoalbero destro di y e il sottoalbero sinistro di z diventa il nuovo sottoalbero sinistro di y. Questo è il caso complicato perché, come vedremo, occorre considerare il caso in cui y è figlio destro di z.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

La procedura per cancellare un dato nodo z da un albero binario di ricerca T richiede come argomenti i puntatori a T e z. La procedura organizza i casi in modo leggermente diverso dai tre appena descritti e considera i quattro casi mostrati nella Figura 12.4.

- Se z non ha un figlio sinistro (parte (a) della figura), sostituiamo z con il suo figlio destro, che può essere NIL oppure no. Se il figlio destro di z è NIL, si ha il caso in cui z non ha figli. Se il figlio destro di z non è NIL, si ha il caso in cui z ha un solo figlio, che è il suo figlio destro.

- Se z ha un solo figlio, che è il suo figlio sinistro (parte (b) della figura), sostituiamo z con il suo figlio sinistro.
- Altrimenti, z ha un figlio sinistro e un figlio destro. Troviamo il successore y di z, che si trova nel sottoalbero destro di z e non ha un figlio sinistro (vedere l'Esercizio 12.2-5). Vogliamo staccare y dalla sua posizione corrente per metterlo al posto di z nell'albero.
  - Se y è il figlio destro di z (parte (c)), sostituiamo z con y, lasciando a y soltanto il figlio destro.
  - Altrimenti, y si trova nel sottoalbero destro di z, ma non è il figlio destro di z (parte (d)). In questo caso, sostituiamo prima y con il suo figlio destro; poi sostituiamo z con y.

Per poter spostare i sottoalberi all'interno dell'albero binario di ricerca, abbiamo definito una subroutine TRANSPLANT, che sostituisce un sottoalbero, come figlio di suo padre, con un altro sottoalbero. Quando TRANSPLANT sostituisce il sottoalbero con radice nel nodo u con il sottoalbero con radice nel nodo v, il padre del nodo u diventa il padre del nodo v, e il padre di u alla fine ha v come figlio.

```
TRANSPLANT(T, u, v)
1  if u.p == NIL
2      T.root = v
3  elseif u == u.p.left
4      u.p.left = v
5  else u.p.right = v
6  if v ≠ NIL
7      v.p = u.p
```

Le righe 1–2 gestiscono il caso in cui u è la radice di T. Altrimenti, u è un figlio sinistro o un figlio destro di suo padre. Le righe 3–4 aggiornano $u.p.left$ se u è un figlio sinistro; la riga 5 aggiorna $u.p.right$ se u è un figlio destro. Poiché v può essere NIL, le righe 6–7 aggiornano $v.p$ se v non è NIL. Si noti che TRANSPLANT non aggiorna $v.left$ e $v.right$; fare ciò, o non farlo, è compito della procedura che chiama TRANSPLANT.

Disponendo della procedura TRANSPLANT, ecco la procedura che cancella il nodo z dall'albero binario di ricerca T:

```
TREE-DELETE(T, z)
 1  if z.left == NIL
 2      TRANSPLANT(T, z, z.right)
 3  elseif z.right == NIL
 4      TRANSPLANT(T, z, z.left)
 5  else y = TREE-MINIMUM(z.right)
 6      if y.p ≠ z
 7          TRANSPLANT(T, y, y.right)
 8          y.right = z.right
 9          y.right.p = y
10      TRANSPLANT(T, z, y)
11      y.left = z.left
12      y.left.p = y
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

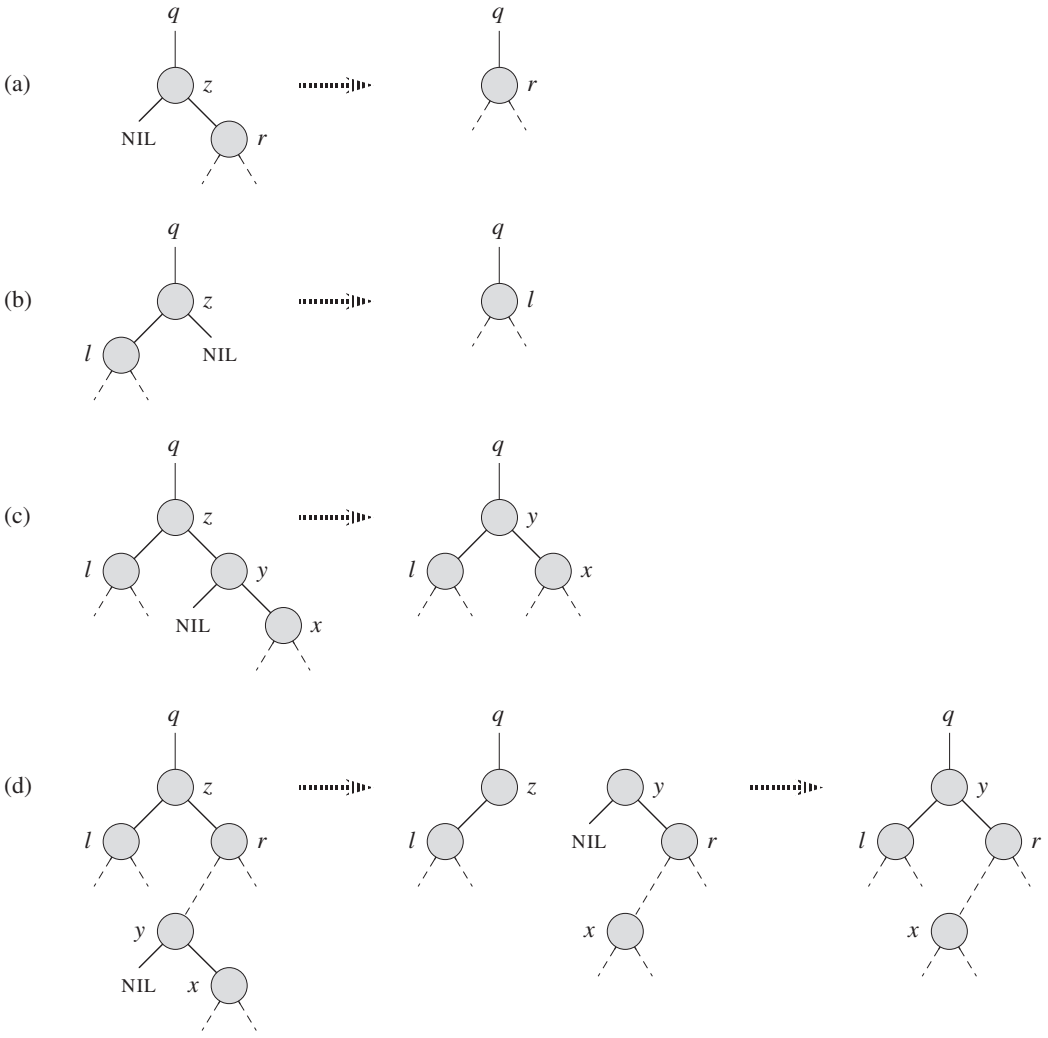


Figura 12.4 Cancellare un nodo z da un albero binario di ricerca. Il nodo z può essere la radice, un figlio sinistro del nodo q o un figlio destro di q. **(a)** Il nodo z non ha un figlio sinistro. Sostituiamo z con il suo figlio destro r, che può essere NIL oppure no. **(b)** Il nodo z ha un figlio sinistro l, ma non ha un figlio destro. Sostituiamo z con l. **(c)** Il nodo z ha due figli; il figlio sinistro è il nodo l, il figlio destro è il suo successore y, e il figlio destro di y è il nodo x. Sostituiamo z con y, modificando il figlio sinistro di y in modo che diventi l, ma lasciando x come figlio destro di y. **(d)** Il nodo z ha due figli (il figlio sinistro l e il figlio destro r), e il suo successore $y \neq r$ si trova nel sottoalbero con radice in r. Sostituiamo y con il suo figlio destro x e impostiamo y come padre di r. Poi, impostiamo y come figlio di q e padre di l.

La procedura TREE-DELETE esegue i quattro casi nel seguente modo. Le righe 1–2 gestiscono il caso in cui il nodo z non ha un figlio sinistro; le righe 3–4 gestiscono il caso in cui z ha un figlio sinistro, ma non un figlio destro. Le righe 5–12 trattano gli altri due casi, in cui z ha due figli. La riga 5 trova il nodo y, che è il successore di z. Poiché z ha un sottoalbero destro non vuoto, il suo successore deve essere il nodo di quel sottoalbero con la chiave più piccola; da qui la chiamata della procedura TREE-MINIMUM($z.right$). Come detto in precedenza, y non ha un figlio sinistro. Stacchiamo y dalla sua posizione corrente e mettiamolo al posto di z. Se y è il figlio destro di z, le righe 10–12 sostituiscono z, come figlio di suo

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 1995030I6-220707-0. Copyright © 2022 McGraw-Hill Education (Italy)

padre, con y; poi, sostituiscono il figlio sinistro di y con il figlio sinistro di z. Se y non è il figlio sinistro di z, le righe 7–9 sostituiscono y, come figlio di suo padre, con il figlio destro di y e cambiano il figlio destro di z nel figlio destro di y; le righe 10–12 sostituiscono z, come figlio di suo padre, con y e, infine, rimpiazzano il figlio sinistro di y con il figlio sinistro di z.

Ogni riga di TREE-DELETE, incluse le chiamate di TRANSPLANT, richiede un tempo costante, tranne la chiamata di TREE-MINIMUM nella riga 5. Quindi, la procedura TREE-DELETE viene eseguita nel tempo $O(h)$ in un albero di altezza h. In sintesi, abbiamo dimostrato il seguente teorema.

Teorema 12.3
Le operazioni sugli insiemi dinamici INSERT e DELETE possono essere svolte nel tempo $O(h)$ in un albero binario di ricerca di altezza h. ■

Esercizi

12.3-1
Scrivete una versione ricorsiva della procedura TREE-INSERT.

12.3-2
Supponete che un albero binario di ricerca sia costruito inserendo ripetutamente alcuni valori distinti nell'albero. Dimostrate che il numero di nodi esaminati durante la ricerca di un valore è pari a uno più il numero di nodi esaminati quando il valore fu inserito per la prima volta nell'albero.

12.3-3
Possiamo ordinare un dato insieme di n numeri costruendo prima un albero binario di ricerca che contiene questi numeri (usando ripetutamente TREE-INSERT per inserire i numeri uno alla volta) e stampando poi i numeri con un attraversamento simmetrico dell'albero. Quali sono i tempi di esecuzione nel caso peggiore e nel caso migliore per questo algoritmo di ordinamento?

12.3-4
L'operazione di cancellazione è "commutativa" nel senso che cancellare prima x e poi y da un albero binario di ricerca equivale a cancellare prima y e poi x? Spiegate perché sì oppure indicate un controesempio.

12.3-5
Supponete che ciascun nodo x, anziché includere l'attributo $x.p$, che punta al padre di x, includa $x.succ$, che punta al successore di x. Scrivete lo pseudocodice di SEARCH, INSERT e DELETE per un albero binario di ricerca T utilizzando questa rappresentazione. Queste procedure devono operare nel tempo $O(h)$, dove h è l'altezza dell'albero T (*suggerimento:* potreste implementare una subroutine che restituisce il padre di un nodo.)

12.3-6
Quando, nella procedura TREE-DELETE, il nodo z ha due figli possiamo scegliere il predecessore invece del successore come nodo y. Quali altre modifiche occorre apportare a TREE-DELETE se facciamo così? Alcuni ritengono che, con una strategia che scelga casualmente e con equità tra predecessore e successore, è possibile ottenere prestazioni empiriche migliori. Come dovrebbe essere modificata la procedura TREE-DELETE per implementare tale strategia?

Acquistato da Michele Mendola su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995059915 2207 © Copyright © 2022, McGraw-Hill Education (Italy)

⋆ 12.4 Alberi binari di ricerca costruiti in modo casuale

Abbiamo visto che tutte le operazioni elementari su un albero binario di ricerca vengono eseguite nel tempo $O(h)$, dove h è l'altezza dell'albero. Tuttavia, questa altezza varia quando gli elementi vengono inseriti o cancellati. Per esempio, se gli elementi vengono inseriti in ordine strettamente crescente, l'albero sarà una catena di altezza $n-1$. D'altra parte, l'Esercizio B.5-4 dimostra che $h \geq \lfloor \lg n \rfloor$. Analogamente a quicksort, possiamo dimostrare che il comportamento nel caso medio è molto più vicino al caso migliore che al caso peggiore.

Purtroppo, sappiamo poco sull'altezza media di un albero binario di ricerca quando utilizziamo sia l'inserimento sia la cancellazione per crearlo. Se l'albero è creato soltanto con gli inserimenti, l'analisi diventa più trattabile. Definiamo quindi un ***albero binario di ricerca costruito in modo casuale*** con n chiavi come quello che si ottiene inserendo in ordine casuale le chiavi in un albero inizialmente vuoto, dove ciascuna delle $n!$ permutazioni delle chiavi di input ha la stessa probabilità (l'Esercizio 12.4-3 chiede di dimostrare che questo concetto è diverso dal supporre che ogni albero binario di ricerca con n chiavi abbia la stessa probabilità). In questo paragrafo, dimostreremo il seguente teorema.

Teorema 12.4
L'altezza attesa di un albero binario di ricerca costruito in modo casuale con n chiavi distinte è $O(\lg n)$.

Dimostrazione Iniziamo definendo tre variabili casuali che aiutano a misurare l'altezza di un albero binario di ricerca costruito in modo casuale. Indichiamo con X_n l'altezza di un albero binario di ricerca costruito in modo casuale con n chiavi e definiamo l'***altezza esponenziale*** $Y_n = 2^{X_n}$. Quando costruiamo un albero binario di ricerca con n chiavi, scegliamo una chiave per la radice e indichiamo con R_n la variabile casuale che contiene il ***rango*** di questa chiave all'interno dell'insieme di n chiavi: ossia R_n è la posizione che tale chiave occuperebbe se l'insieme delle chiavi fosse ordinato. Il valore di R_n ha la stessa probabilità di essere un elemento qualsiasi dell'insieme $\{1, 2, \ldots, n\}$. Se $R_n = i$, allora il sottoalbero sinistro della radice è un albero binario di ricerca costruito in modo casuale con $i-1$ chiavi e il sottoalbero destro è un albero binario di ricerca costruito in modo casuale con $n-i$ chiavi. Poiché l'altezza di un albero binario è uno più la maggiore delle altezze dei due sottoalberi della radice, l'altezza esponenziale di un albero binario è due volte la più grande delle altezze esponenziali dei due sottoalberi della radice. Sapendo che $R_n = i$, abbiamo

$$Y_n = 2 \cdot \max(Y_{i-1}, Y_{n-i})$$

Come casi base, abbiamo $Y_1 = 1$, perché l'altezza esponenziale di un albero con un nodo è $2^0 = 1$ e, per comodità, definiamo $Y_0 = 0$.

Poi definiamo le variabili casuali indicatrici $Z_{n,1}, Z_{n,2}, \ldots, Z_{n,n}$, dove

$$Z_{n,i} = \mathrm{I}\{R_n = i\}$$

Acquistato da Michele Michele su Webster il 2022-07-07 25:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Poiché R_n ha la stessa probabilità di essere un elemento qualsiasi di $\{1, 2, \ldots, n\}$, abbiamo che $\Pr\{R_n = i\} = 1/n$ per $i = 1, 2, \ldots, n$, e quindi, per il Lemma 5.1,

$$\mathrm{E}[Z_{n,i}] = 1/n \tag{12.1}$$

per $i = 1, 2, \ldots, n$. Poiché un solo valore di $Z_{n,i}$ è 1 e tutti gli altri sono 0, abbiamo anche

$$Y_n = \sum_{i=1}^{n} Z_{n,i} \left(2 \cdot \max(Y_{i-1}, Y_{n-i})\right)$$

Dimostreremo che $\mathrm{E}[Y_n]$ è polinomiale in n, e questo in definitiva implica che $\mathrm{E}[X_n] = O(\lg n)$.

La variabile casuale indicatrice $Z_{n,i} = \mathrm{I}\{R_n = i\}$ è indipendente dai valori di Y_{i-1} e Y_{n-i}. Avendo scelto $R_n = i$, il sottoalbero sinistro, la cui altezza esponenziale è Y_{i-1}, è costruito in modo casuale con le $i-1$ chiavi i cui ranghi sono minori di i. Questo sottoalbero è proprio come qualsiasi albero binario di ricerca costruito in modo casuale con $i-1$ chiavi. Diversamente dal numero di chiavi che contiene, la struttura di questo sottoalbero non è affatto influenzata dalla scelta di $R_n = i$; quindi le variabili casuali Y_{i-1} e $Z_{n,i}$ sono indipendenti. Analogamente, il sottoalbero destro, la cui altezza esponenziale è Y_{n-i}, è costruito in modo casuale con le $n-i$ chiavi i cui ranghi sono maggiori di i. La sua struttura è indipendente dal valore di R_n, e quindi le variabili casuali Y_{n-i} e $Z_{n,i}$ sono indipendenti. Quindi

$$\begin{aligned}
\mathrm{E}[Y_n] &= \mathrm{E}\left[\sum_{i=1}^{n} Z_{n,i} \left(2 \cdot \max(Y_{i-1}, Y_{n-i})\right)\right] \\
&= \sum_{i=1}^{n} \mathrm{E}\left[Z_{n,i} \left(2 \cdot \max(Y_{i-1}, Y_{n-i})\right)\right] && \text{(per la linearità del valore atteso)} \\
&= \sum_{i=1}^{n} \mathrm{E}[Z_{n,i}]\,\mathrm{E}\left[2 \cdot \max(Y_{i-1}, Y_{n-i})\right] && \text{(per l'indipendenza)} \\
&= \sum_{i=1}^{n} \frac{1}{n} \cdot \mathrm{E}\left[2 \cdot \max(Y_{i-1}, Y_{n-i})\right] && \text{(per l'equazione (12.1))} \\
&= \frac{2}{n} \sum_{i=1}^{n} \mathrm{E}\left[\max(Y_{i-1}, Y_{n-i})\right] && \text{(per l'equazione (C.22))} \\
&\le \frac{2}{n} \sum_{i=1}^{n} \left(\mathrm{E}[Y_{i-1}] + \mathrm{E}[Y_{n-i}]\right) && \text{(per l'Esercizio C.3-4)}
\end{aligned}$$

Ogni termine $\mathrm{E}[Y_0], \mathrm{E}[Y_1], \ldots, \mathrm{E}[Y_{n-1}]$ appare due volte nell'ultima sommatoria, una volta come $\mathrm{E}[Y_{i-1}]$ e una volta come $\mathrm{E}[Y_{n-i}]$; quindi abbiamo la ricorrenza

$$\mathrm{E}[Y_n] \le \frac{4}{n} \sum_{i=0}^{n-1} \mathrm{E}[Y_i] \tag{12.2}$$

Applicando il metodo di sostituzione, dimostreremo che per qualsiasi intero positivo n, la ricorrenza (12.2) ha la soluzione

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\mathrm{E}[Y_n] \le \frac{1}{4}\binom{n+3}{3}$$

Per farlo, utilizzeremo l'identità

$$\sum_{i=0}^{n-1} \binom{i+3}{3} = \binom{n+3}{4} \tag{12.3}$$

L'Esercizio 12.4-1 chiede di dimostrare questa identità. Per i casi base, verifichiamo che i limiti

$$0 = Y_0 = \mathrm{E}[Y_0] \le \frac{1}{4}\binom{3}{3} = \frac{1}{4} \quad \text{e} \quad 1 = Y_1 = \mathrm{E}[Y_1] \le \frac{1}{4}\binom{1+3}{3} = 1$$

sono validi. Per il caso induttivo, si ha

$$\begin{aligned}
\mathrm{E}[Y_n] &\le \frac{4}{n}\sum_{i=0}^{n-1}\mathrm{E}[Y_i] \\
&\le \frac{4}{n}\sum_{i=0}^{n-1}\frac{1}{4}\binom{i+3}{3} && \text{(per l'ipotesi induttiva)} \\
&= \frac{1}{n}\sum_{i=0}^{n-1}\binom{i+3}{3} \\
&= \frac{1}{n}\binom{n+3}{4} && \text{(per l'equazione (12.3))} \\
&= \frac{1}{n}\cdot\frac{(n+3)!}{4!\,(n-1)!} \\
&= \frac{1}{4}\cdot\frac{(n+3)!}{3!\,n!} \\
&= \frac{1}{4}\binom{n+3}{3}
\end{aligned}$$

Abbiamo limitato $\mathrm{E}[Y_n]$, ma il nostro obiettivo finale è limitare $\mathrm{E}[X_n]$. Come chiede di dimostrare l'Esercizio 12.4-4, la funzione $f(x) = 2^x$ è convessa (vedere pagina 995); quindi, possiamo applicare la disequazione di Jensen (C.26)

$$2^{\mathrm{E}[X_n]} \le \mathrm{E}\left[2^{X_n}\right] = \mathrm{E}[Y_n]$$

per ottenere che

$$\begin{aligned}
2^{\mathrm{E}[X_n]} &\le \frac{1}{4}\binom{n+3}{3} \\
&= \frac{1}{4}\cdot\frac{(n+3)(n+2)(n+1)}{6} \\
&= \frac{n^3+6n^2+11n+6}{24}
\end{aligned}$$

Prendendo i logaritmi da entrambi i lati, si ha $\mathrm{E}[X_n] = O(\lg n)$.

■

Esercizi

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

12.4-1

Dimostrate l'equazione (12.3).

12.4-2

Descrivete un albero binario di ricerca di n nodi, dove la profondità media di un nodo nell'albero è $\Theta(\lg n)$, mentre l'altezza dell'albero è $\omega(\lg n)$. Calcolate un limite superiore asintotico sull'altezza di un albero binario di ricerca di n nodi in cui la profondità media di un nodo è $\Theta(\lg n)$.

12.4-3
Dimostrate che il concetto di albero binario di ricerca scelto in modo casuale con n chiavi, dove ogni albero binario di ricerca con n chiavi ha la stessa probabilità di essere scelto, è diverso dal concetto di albero binario di ricerca costruito in modo casuale che abbiamo descritto in questo paragrafo (*suggerimento:* elencate le possibilità quando $n = 3$).

12.4-4
Dimostrate che la funzione $f(x) = 2^x$ è convessa.

12.4-5 ⋆
Considerate la procedura RANDOMIZED-QUICKSORT che opera su una sequenza di n numeri distinti di input. Dimostrate che per qualsiasi costante $k > 0$, per tutte le $n!$ permutazioni dell'input, tranne $O(1/n^k)$ permutazioni, il tempo di esecuzione è $O(n \lg n)$.

Problemi

12-1 Alberi binari di ricerca con chiavi uguali
Le chiavi uguali sono un problema per l'implementazione degli alberi binari di ricerca.

a. Quali sono le prestazioni asintotiche della procedura TREE-INSERT quando viene utilizzata per inserire n elementi con chiavi identiche in un albero binario di ricerca inizialmente vuoto?

Vi proponiamo di migliorare TREE-INSERT verificando prima della riga 5 se $z.key = x.key$ e verificando prima della riga 11 se $z.key = y.key$. Quando valgono tali uguaglianze, implementate una delle strategie elencate sotto. Per ogni strategia, trovate le prestazioni asintotiche dell'inserimento di n elementi con chiavi identiche in un albero binario di ricerca inizialmente vuoto (le strategie sono descritte per la riga 5 che confronta le chiavi di z e x; sostituite x con y per le strategie relative alla riga 11).

b. Mantenete un flag booleano $b[x]$ nel nodo x e impostate x a $x.left$ o $x.right$ a seconda del valore di $b[x]$, che si alterna fra FALSE e TRUE ogni volta che il nodo x viene visitato durante l'inserimento di un nodo con la stessa chiave di x.

c. Mantenete una lista di nodi con chiavi uguali nel nodo x e inserite z nella lista.

d. Scegliete casualmente se assegnare $x.left$ o $x.right$ a x (indicate le prestazioni nel caso peggiore e ricavate in modo informale il tempo di esecuzione atteso).

12-2 Radix tree
Acquistato da Michele Mattioni (michele.mattioni@gmail.com) il 2022-07-07 22:12 Numero Ordine Libreria 1995001616 Copyright 2010 The McGraw-Hill Education (Italy)
Date due stringhe $a = a_0a_1 \ldots a_p$ e $b = b_0b_1 \ldots b_q$, dove a_i e b_j appartengono a un insieme ordinato di caratteri, diciamo che la stringa a è ***lessicograficamente minore*** della stringa b se è soddisfatta una delle seguenti condizioni:

1. Esiste un numero intero j, con $0 \leq j \leq \min(p, q)$, tale che $a_i = b_i$ per ogni $i = 0, 1, \ldots, j - 1$ e $a_j < b_j$.
2. $p < q$ e $a_i = b_i$ per ogni $i = 0, 1, \ldots, p$.

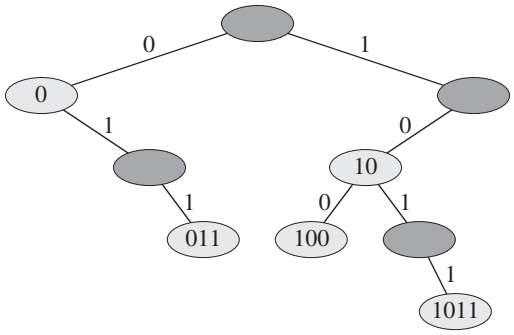


Figura 12.5 Un radix tree con le stringhe di bit 1011, 10, 011, 100 e 0. La chiave di un nodo qualsiasi può essere determinata seguendo il cammino semplice che va dalla radice al nodo. Non c'è bisogno, quindi, di memorizzare le chiavi nei nodi; le chiavi sono indicate soltanto per scopi illustrativi. I nodi hanno uno sfondo più scuro se le corrispondenti chiavi non si trovano nell'albero; tali nodi sono presenti soltanto per stabilire un percorso per gli altri nodi.

Per esempio, se a e b sono stringhe di bit, allora $10100 < 10110$ per la regola 1 (ponendo $j = 3$) e $10100 < 101000$ per la regola 2. Questo è simile all'ordinamento utilizzato nei dizionari.

La struttura dati ***radix tree*** illustrata nella Figura 12.5 contiene le stringhe di bit 1011, 10, 011, 100 e 0. Durante la ricerca di una chiave $a = a_0 a_1 \ldots a_p$, si va a sinistra in un nodo di profondità i se $a_i = 0$, a destra se $a_i = 1$. Sia S un insieme di stringhe binarie distinte, la cui somma delle lunghezze è n. Dimostrate come utilizzare un radix tree per ordinare lessicograficamente le stringhe di S nel tempo $\Theta(n)$. Per l'esempio illustrato nella Figura 12.5, l'output dell'ordinamento dovrebbe essere la sequenza 0, 011, 10, 100, 1011.

12-3 Profondità media di un nodo in un albero binario di ricerca costruito in modo casuale

In questo problema dimostreremo che la profondità media di un nodo in un albero binario di ricerca costruito in modo casuale con n nodi è $O(\lg n)$. Sebbene questo risultato sia meno stretto di quello del Teorema 12.4, la tecnica adottata rivela una sorprendente somiglianza fra la costruzione di un albero binario di ricerca e l'esecuzione della procedura RANDOMIZED-QUICKSORT, descritta nel Paragrafo 7.3. Definiamo ***lunghezza totale dei cammini*** $P(T)$ di un albero binario T la somma, per tutti i nodi x di T, della profondità del nodo x, che indichiamo con $d(x, T)$.

a. Deducete che la profondità media di un nodo in T è

$$\frac{1}{n} \sum_{x \in T} d(x, T) = \frac{1}{n} P(T)$$

Quindi, si vuole dimostrare che il valore atteso di $P(T)$ è $O(n \lg n)$.

b. Indicate con T_L e T_R, rispettivamente, i sottoalberi sinistro e destro dell'albero T. Dimostrate che, se T ha n nodi, allora

$$P(T) = P(T_L) + P(T_R) + n - 1$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria 2995316-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

c. Indicate con $P(n)$ il valore medio della lunghezza totale dei cammini di un albero binario di ricerca costruito in modo casuale con n nodi. Dimostrate che

$$P(n) = \frac{1}{n} \sum_{i=0}^{n-1} (P(i) + P(n - i - 1) + n - 1)$$

d. Dimostrate che è possibile riscrivere $P(n)$ in questo modo

$$P(n) = \frac{2}{n}\sum_{k=1}^{n-1} P(k) + \Theta(n)$$

e. Ricordando l'analisi alternativa della versione randomizzata di quicksort data nel Problema 7-3, concludete che $P(n) = O(n \lg n)$.

A ogni chiamata ricorsiva di quicksort, viene scelto un elemento pivot casuale per partizionare l'insieme degli elementi da ordinare. Ogni nodo di un albero binario di ricerca partiziona l'insieme degli elementi inseriti nel sottoalbero con radice in quel nodo.

f. Descrivete un'implementazione di quicksort dove i confronti per ordinare un insieme di elementi sono esattamente gli stessi confronti per inserire gli elementi in un albero binario di ricerca (l'ordine in cui avvengono i confronti può variare, ma devono essere fatti gli stessi confronti).

12-4 Numero di alberi binari differenti

Indichiamo con b_n il numero di alberi binari differenti con n nodi. In questo problema, troverete una formula per b_n e anche una stima asintotica.

a. Dimostrate che $b_0 = 1$ e che, per $n \geq 1$, si ha

$$b_n = \sum_{k=0}^{n-1} b_k b_{n-1-k}$$

b. Facendo riferimento al Problema 4-4 per la definizione di una funzione generatrice, sia $B(x)$ la funzione generatrice

$$B(x) = \sum_{n=0}^{\infty} b_n x^n$$

Dimostrate che $B(x) = xB(x)^2 + 1$, e quindi un modo per esprimere $B(x)$ in forma chiusa è

$$B(x) = \frac{1}{2x}\left(1 - \sqrt{1-4x}\right)$$

Lo sviluppo in ***serie di Taylor*** della funzione $f(x)$ intorno al punto $x = a$ è dato da

$$f(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(a)}{k!}(x-a)^k$$

dove $f^{(k)}(x)$ è la derivata di ordine k di f nel punto x.

c. Dimostrate che

$$b_n = \frac{1}{n+1}\binom{2n}{n}$$

Acquistato da Michele Michelucci Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

(l'n-esimo ***numero catalano***) utilizzando lo sviluppo in serie di Taylor di $\sqrt{1-4x}$ intorno al punto $x = 0$. (Se preferite, anziché utilizzare lo sviluppo in serie di Taylor, potete utilizzare la generalizzazione dello sviluppo in serie binomiale (C.4) a esponenti n non interi, dove per qualsiasi numero reale n e qualsiasi intero k, il coefficiente binomiale $\binom{n}{k}$ è interpretato come $n(n-1)\cdots(n-k+1)/k!$ se $k \geq 0$, 0 negli altri casi.)

d. Dimostrate che

$$b_n = \frac{4^n}{\sqrt{\pi} n^{3/2}} (1 + O(1/n))$$

Note

Knuth [212] descrive le forme elementari dell'albero binario di ricerca e numerose varianti. Pare che gli alberi binari di ricerca siano stati scoperti in maniera indipendente da molte persone alla fine degli anni cinquanta. I *radix tree* sono spesso chiamati *trie* (questo termine deriva dalle lettere centrali della parola "retrieval"). Anche questi alberi sono trattati da Knuth [212].

Molti testi, incluse le prime due edizioni di questo libro, descrivono un metodo un po' più semplice per cancellare un nodo da un albero binario di ricerca quando sono presenti entrambi i figli del nodo. Anziché sostituire il nodo z con il suo successore y, viene cancellato il nodo y, ma la sua chiave e i dati satelliti vengono copiati nel nodo z. Lo svantaggio di questo approccio è che il nodo effettivamente cancellato potrebbe non essere il nodo passato alla procedura di cancellazione. Se altre componenti di un programma utilizzano dei puntatori ai nodi nell'albero, esse potrebbero erroneamente trovarsi con puntatori "obsoleti" a nodi che sono stati cancellati. Sebbene il metodo di cancellazione presentato in questa edizione del libro sia un po' più complicato, esso garantisce che una chiamata della procedura per cancellare il nodo z cancella il nodo z e soltanto il nodo z.

Il Paragrafo 15.5 mostra come costruire un albero binario di ricerca ottimale quando le frequenze di ricerca sono note prima della costruzione dell'albero. Ovvero, date le frequenze di ricerca per ogni chiave e le frequenze di ricerca per i valori che ricadono fra due chiavi successive nell'albero, costruiamo un albero binario di ricerca per il quale un insieme di ricerche che rispetti queste frequenze esamina il numero minimo di nodi.

Aslam [24] è l'autore della dimostrazione (presentata nel Paragrafo 12.4) che limita l'altezza attesa di un albero binario di ricerca costruito in modo casuale. Martínez e Roura [244] descrivono degli algoritmi randomizzati per inserire e cancellare gli elementi da un albero binario di ricerca; il risultato di entrambe le operazioni è un albero binario di ricerca casuale. Tuttavia, la loro definizione di albero binario di ricerca casuale differisce leggermente da quella di albero binario di ricerca costruito in modo casuale che abbiamo presentato in questo capitolo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Alberi rosso-neri 13

Nel Capitolo 12 si è visto che un albero binario di ricerca di altezza h può implementare qualsiasi operazione elementare sugli insiemi dinamici – come SEARCH, PREDECESSOR, SUCCESSOR, MINIMUM, MAXIMUM, INSERT e DELETE – nel tempo $O(h)$. Queste operazioni sono, quindi, veloci se l'altezza dell'albero di ricerca è piccola; ma se l'altezza è grande, le loro prestazioni potrebbero non essere migliori di quelle di una lista concatenata. Gli alberi rosso-neri rappresentano uno dei tanti modi in cui gli alberi di ricerca vengono "bilanciati" per garantire che le operazioni elementari sugli insiemi dinamici richiedano un tempo $O(\lg n)$ nel caso peggiore.

13.1 Proprietà degli alberi rosso-neri

Un ***albero rosso-nero*** è un albero binario di ricerca con un bit aggiuntivo di memoria per ogni nodo: il ***colore*** del nodo, che può essere RED (rosso) o BLACK (nero). Assegnando dei vincoli al modo in cui i nodi possono essere colorati lungo qualsiasi cammino semplice che va dalla radice a una foglia, gli alberi rosso-neri garantiscono che nessuno di tali cammini sia lungo più del doppio di qualsiasi altro, quindi l'albero è approssimativamente ***bilanciato***.

Ogni nodo dell'albero adesso contiene gli attributi *color*, *key*, *left*, *right* e *p*. Se manca un figlio o il padre di un nodo, il corrispondente attributo puntatore del nodo contiene il valore NIL. Tratteremo questi valori NIL come puntatori a foglie (nodi esterni) dell'albero binario di ricerca e i nodi normali che contengono le chiavi come nodi interni dell'albero.

Un albero rosso-nero è un albero binario di ricerca che soddisfa le seguenti ***proprietà***:

1. Ogni nodo è rosso o nero.
2. La radice è nera.
3. Ogni foglia (NIL) è nera.
4. Se un nodo è rosso, allora entrambi i suoi figli sono neri.
5. Per ogni nodo, tutti i cammini semplici che vanno dal nodo alle foglie sue discendenti contengono lo stesso numero di nodi neri.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

La Figura 13.1(a) illustra un esempio di albero rosso-nero.

Per semplificare la gestione delle condizioni al contorno nel codice delle operazioni su di un albero rosso-nero, utilizzeremo un'unica sentinella per rappresentare NIL (vedere pagina 197). Per un albero rosso-nero T, la sentinella $T.nil$ è un oggetto con gli stessi attributi di un nodo ordinario dell'albero; il suo attributo *color* è BLACK e gli altri attributi – *p*, *left*, *right* e *key* – possono assumere valori

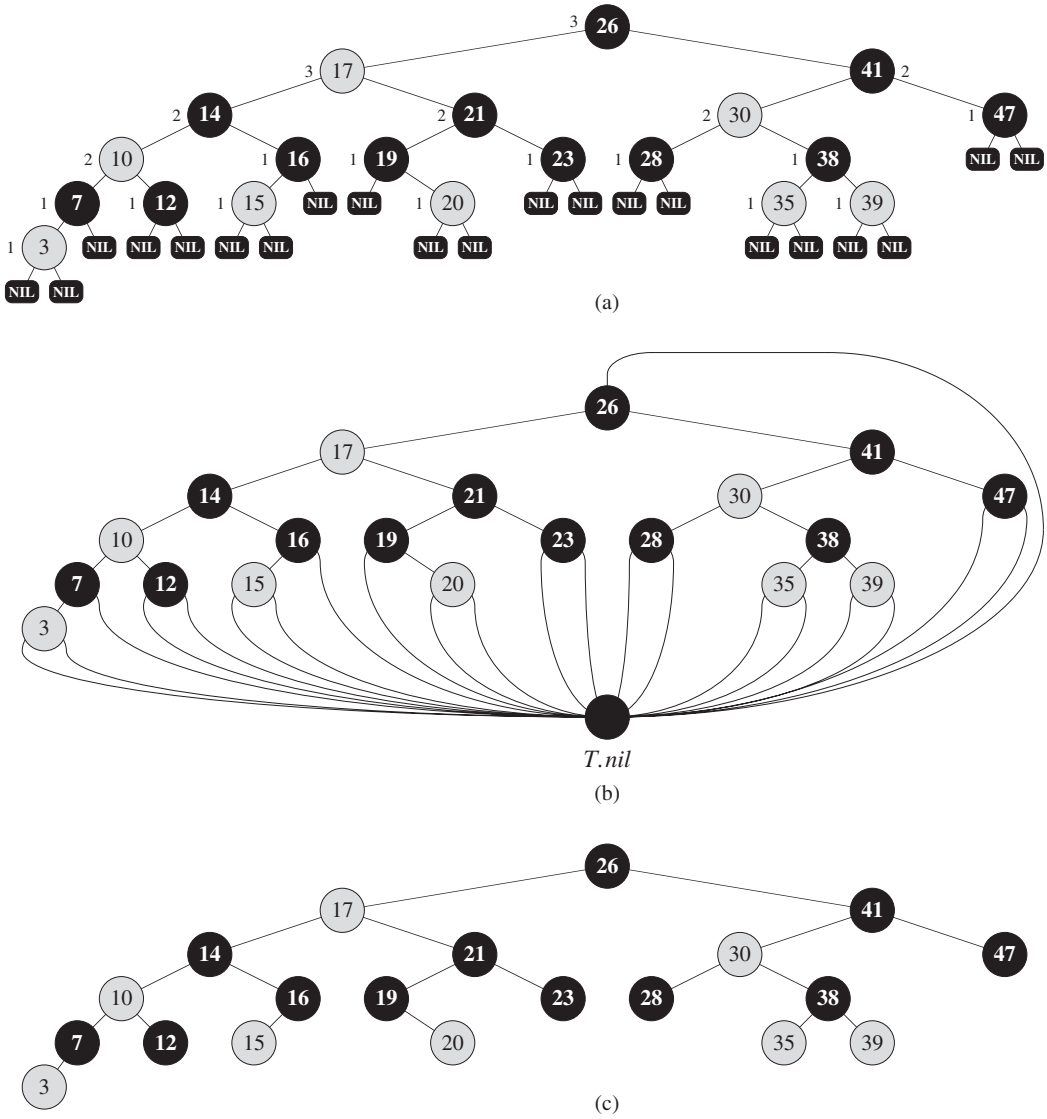


Figura 13.1 Un albero rosso-nero con i nodi neri su sfondo nero e i nodi rossi su sfondo grigio. Ogni nodo di un albero rosso-nero è rosso o nero; i figli di un nodo rosso sono entrambi neri; qualsiasi cammino semplice che va da un nodo a una foglia sua discendente contiene lo stesso numero di nodi neri. **(a)** Ogni foglia, rappresentata da NIL, è nera. Ogni nodo non-NIL è marcato con la sua altezza nera; i nodi NIL hanno l'altezza nera pari a 0. **(b)** Lo stesso albero rosso-nero dove tutti i nodi NIL sono stati sostituiti dall'unica sentinella *T.nil*, che è sempre nera (le altezze nere non sono indicate). La sentinella è anche padre della radice. **(c)** Lo stesso albero rosso-nero senza le foglie e il padre della radice. Utilizzeremo questo schema di rappresentazione nella parte restante di questo capitolo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1895303018-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

arbitrari. Come illustra la Figura 13.1(b), tutti i puntatori a NIL sono sostituiti con puntatori alla sentinella *T.nil*. L'uso della sentinella ci permette di trattare un figlio NIL di un nodo x come un nodo ordinario il cui padre è x. Avremmo potuto aggiungere un nodo sentinella distinto per ogni NIL nell'albero, in modo che il padre di ogni nodo NIL fosse ben definito; tuttavia questo approccio avrebbe sprecato dello spazio in memoria. Noi utilizziamo invece l'unica sentinella *T.nil* per rappresentare tutti i nodi NIL – tutte le foglie e il padre della radice. I valori degli

attributi p, *left*, *right* e *key* della sentinella sono irrilevanti, sebbene possa tornare comodo assegnare loro dei valori nel corso di una procedura.

In generale, concentriamo la nostra attenzione sui nodi interni di un albero rosso-nero, perché essi contengono i valori delle chiavi (*key*). Nella parte rimanente di questo capitolo, ometteremo le foglie nella rappresentazione degli alberi rosso-neri, come illustra la Figura 13.1(c).

Definiamo ***altezza nera*** di un nodo x, indicata da $\mathrm{bh}(x)$, il numero di nodi neri lungo un cammino semplice che inizia dal nodo x (ma non lo include) e finisce in una foglia. Per la proprietà 5, il concetto di altezza nera è ben definito, in quanto tutti i cammini semplici che scendono dal nodo hanno lo stesso numero di nodi neri. Per definizione, l'altezza nera di un albero rosso-nero è l'altezza nera della sua radice.

Il seguente lemma mostra perché gli alberi rosso-neri siano dei buoni alberi di ricerca.

Lemma 13.1
L'altezza massima di un albero rosso-nero con n nodi interni è $2\lg(n+1)$.

Dimostrazione Iniziamo dimostrando che il sottoalbero con radice in un nodo x qualsiasi contiene almeno $2^{\mathrm{bh}(x)} - 1$ nodi interni. Proveremo questa asserzione per induzione sull'altezza di x. Se l'altezza di x è 0, allora x deve essere una foglia ($T.nil$) e il sottoalbero con radice in x contiene realmente almeno $2^{\mathrm{bh}(x)} - 1 = 2^0 - 1 = 0$ nodi interni. Per il passo induttivo, consideriamo un nodo x che ha un'altezza positiva ed è quindi un nodo interno con due figli. Ogni figlio ha un'altezza nera pari a $\mathrm{bh}(x)$ o $\mathrm{bh}(x) - 1$, a seconda se il suo colore è, rispettivamente, rosso o nero. Poiché l'altezza di un figlio di x è minore dell'altezza di x, possiamo applicare l'ipotesi induttiva per concludere che ogni figlio ha almeno $2^{\mathrm{bh}(x)-1} - 1$ nodi interni. Quindi, il sottoalbero con radice in x contiene almeno $(2^{\mathrm{bh}(x)-1} - 1) + (2^{\mathrm{bh}(x)-1} - 1) + 1 = 2^{\mathrm{bh}(x)} - 1$ nodi interni; questo dimostra l'asserzione.

Per completare la dimostrazione del lemma, indichiamo con h l'altezza dell'albero. Per la proprietà 4, almeno metà dei nodi in qualsiasi cammino semplice dalla radice a una foglia (esclusa la radice) deve essere nera. Di conseguenza, l'altezza nera della radice deve essere almeno $h/2$; quindi, abbiamo

$$n \geq 2^{h/2} - 1$$

Spostando 1 nel lato sinistro e prendendo i logaritmi di entrambi i lati, otteniamo $\lg(n+1) \geq h/2$ ovvero $h \leq 2\lg(n+1)$. ■

Una immediata conseguenza di questo lemma è che le operazioni sugli insiemi dinamici SEARCH, MINIMUM, MAXIMUM, SUCCESSOR e PREDECESSOR possono essere implementate nel tempo $O(\lg n)$ negli alberi rosso-neri, perché possono essere eseguite nel tempo $O(h)$ in un albero binario di ricerca di altezza h (come dimostrato nel Capitolo 12) e qualsiasi albero rosso-nero di n nodi è un albero binario di ricerca di altezza $O(\lg n)$ (ovviamente, i riferimenti a NIL negli algoritmi del Capitolo 12 dovranno essere sostituiti con $T.nil$.) Sebbene gli algoritmi TREE-INSERT e TREE-DELETE del Capitolo 12 siano eseguiti nel tempo $O(\lg n)$ quando l'input è un albero rosso-nero, tuttavia essi non supportano direttamente le operazioni sugli insiemi dinamici INSERT e DELETE, in quanto non garantiscono che l'albero binario di ricerca modificato sarà ancora un albero rosso-nero. Tuttavia, nei Paragrafi 13.3 e 13.4 vedremo che queste due operazioni possono essere effettivamente eseguite nel tempo $O(\lg n)$.

Acquistato da Michele Mu[illegible] - [illegible] - [illegible] - [

Esercizi

13.1-1

Seguendo lo schema della Figura 13.1(a), disegnate l'albero binario di ricerca completo di altezza 3 con le chiavi $\{1, 2, \ldots, 15\}$. Aggiungete le foglie NIL e colorate i nodi in tre modi diversi in modo che le altezze nere degli alberi rosso-neri risultanti siano 2, 3 e 4.

13.1-2

Disegnate l'albero rosso-nero che si ottiene dopo la chiamata di TREE-INSERT con la chiave 36 per l'albero illustrato nella Figura 13.1. Se il nodo inserito è rosso, l'albero risultante è un albero rosso-nero? Che cosa cambia se il nodo inserito è nero?

13.1-3

Definiamo ***albero rosso-nero rilassato*** un albero binario di ricerca che soddisfa le proprietà degli alberi rosso-neri 1, 3, 4 e 5. In altre parole, la radice può essere rossa o nera. Considerate un albero rosso-nero rilassato T la cui radice è rossa. Se coloriamo di nero la radice di T, ma non apportiamo altre modifiche a T, l'albero risultante è un albero rosso-nero?

13.1-4

Supponete che ogni nodo rosso in un albero rosso-nero venga "assorbito" dal suo padre nero, in modo che i figli del nodo rosso diventino figli del padre nero (ignorate ciò che accade alle chiavi). Quali sono i possibili gradi di un nodo nero dopo che tutti i suoi figli rossi sono stati assorbiti? Che cosa potete dire sulle profondità delle foglie dell'albero risultante?

13.1-5

Dimostrate che, in un albero rosso-nero, il cammino semplice più lungo che va da un nodo x a una foglia sua discendente ha una lunghezza al massimo doppia di quella del cammino semplice più breve dal nodo x a una foglia sua discendente.

13.1-6

Qual è il numero massimo di nodi interni in un albero rosso-nero di altezza nera k? Qual è il numero minimo?

13.1-7

Descrivete un albero rosso-nero di n chiavi che realizza il massimo rapporto fra nodi interni rossi e nodi interni neri. Qual è questo rapporto? Quale albero ha il più piccolo rapporto e qual è il suo valore?

13.2 Rotazioni

Le operazioni TREE-INSERT e TREE-DELETE sugli alberi di ricerca, quando vengono eseguite in un albero rosso-nero di n chiavi, richiedono un tempo $O(\lg n)$. Poiché queste operazioni modificano l'albero, il risultato potrebbe violare le proprietà degli alberi rosso-neri elencate nel Paragrafo 13.1. Per ripristinare queste proprietà, dobbiamo modificare i colori di qualche nodo dell'albero e anche la struttura dei puntatori.

Acquistato da Michele Michele su Webster il 2022-07-07T23:43:42 Mimesis Edizioni Libreria (ID 1095132016, 220705101) Copyright © 2022 McGraw-Hill Education (Italy)

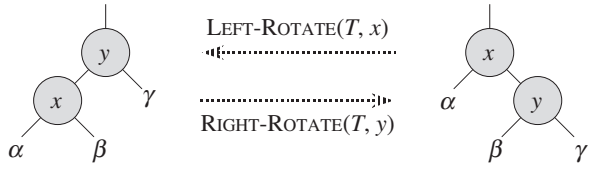


Figura 13.2 Le rotazioni in un albero binario di ricerca. L'operazione LEFT-ROTATE(T, x) trasforma la configurazione dei due nodi a destra nella configurazione a sinistra cambiando un numero costante di puntatori. La configurazione a sinistra può essere trasformata nella configurazione a destra con l'operazione inversa RIGHT-ROTATE(T, y). Le lettere α, β e γ rappresentano sottoalberi arbitrari. Una rotazione preserva la proprietà degli alberi binari di ricerca: le chiavi in α precedono $x.key$, che precede le chiavi in β, che precedono $y.key$, che precede le chiavi in γ.

La struttura dei puntatori viene modificata tramite una ***rotazione***: un'operazione locale in un albero di ricerca che preserva la proprietà degli alberi binari di ricerca. La Figura 13.2 illustra i due tipi di rotazioni: rotazione sinistra e rotazione destra. Quando eseguiamo una rotazione sinistra in un nodo x, supponiamo che il suo figlio destro y non sia $T.nil$; x può essere qualsiasi nodo il cui figlio destro non è $T.nil$. La rotazione sinistra "fa perno" sul collegamento tra x e y; il nodo y diventa la nuova radice del sottoalbero, con x come figlio sinistro di y e il figlio sinistro di y come figlio destro di x.

Lo pseudocodice per LEFT-ROTATE suppone che $x.right \neq T.nil$ e che il padre della radice sia $T.nil$.

```
LEFT-ROTATE(T, x)
 1  y = x.right                 // Imposta y
 2  x.right = y.left            // Sposta il sottoalbero sinistro di y
                                   nel sottoalbero destro di x.
 3  if y.left ≠ T.nil
 4       y.left.p = x
 5  y.p = x.p                   // Collega il padre di x a y.
 6  if x.p == T.nil
 7       T.root = y
 8  elseif x == x.p.left
 9       x.p.left = y
10  else x.p.right = y
11  y.left = x                  // Pone x a sinistra di y.
12  x.p = y
```

La Figura 13.3 illustra il funzionamento di LEFT-ROTATE. Il codice per RIGHT-ROTATE è simmetrico. Entrambe le procedure LEFT-ROTATE e RIGHT-ROTATE vengono eseguite nel tempo $O(1)$. Soltanto i puntatori vengono modificati da una rotazione; tutti gli altri attributi di un nodo non cambiano.

Esercizi

13.2-1

Scrivete lo pseudocodice per RIGHT-ROTATE.

13.2-2

Dimostrate che in ogni albero binario di ricerca di n nodi ci sono esattamente $n - 1$ rotazioni possibili.

Acquistato da Michele Michele su Webster il 2022-08-07 23:12 Numero Ordine Libreria 1993930010 220707 0 Copyright © 2022, McGraw-Hill Education (Italy)

13.2-3

Siano a, b e c dei nodi arbitrari, rispettivamente, nei sottoalberi α, β e γ dell'albero a destra nella Figura 13.2. Come cambiano le profondità di a, b e c quando viene effettuata una rotazione sinistra del nodo x?

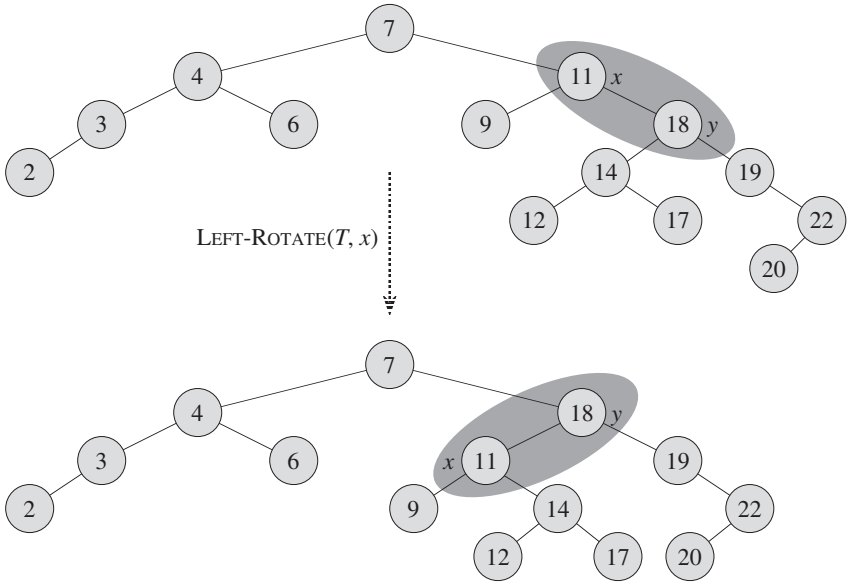


Figura 13.3 Esempio di come la procedura LEFT-ROTATE(T, x) modifica un albero binario di ricerca. Gli attraversamenti simmetrici dell'albero di input e dell'albero modificato producono la stessa lista dei valori delle chiavi.

13.2-4

Dimostrate che ogni albero binario di ricerca di n nodi può essere trasformato in qualsiasi altro albero binario di ricerca di n nodi effettuando $O(n)$ rotazioni (*suggerimento:* prima dimostrate che bastano al massimo $n - 1$ rotazioni destre per trasformare l'albero in una catena verso destra).

13.2-5 ⋆

Diciamo che un albero binario di ricerca T_1 può essere ***convertito a destra*** in un albero binario di ricerca T_2 se è possibile ottenere T_2 da T_1 attraverso una serie di chiamate di RIGHT-ROTATE. Indicate un esempio di due alberi T_1 e T_2 tali che T_1 non può essere convertito a destra in T_2. Poi dimostrate che se un albero T_1 può essere convertito a destra in T_2, allora può essere convertito a destra con $O(n^2)$ chiamate di RIGHT-ROTATE.

13.3 Inserimento

L'inserimento di un nodo in un albero rosso-nero di n nodi può essere effettuato nel tempo $O(\lg n)$. Utilizziamo una versione leggermente modificata della procedura TREE-INSERT (Paragrafo 12.3) per inserire un nodo z nell'albero T come se fosse un normale albero binario di ricerca e poi coloriamo z di rosso. (L'Esercizio 13.3-1 vi chiede di spiegare perché non abbiamo scelto di colorare di nero il nodo z.) Per garantire che le proprietà degli alberi rosso-neri siano preservate, chiamiamo una procedura ausiliaria RB-INSERT-FIXUP che ricolora i nodi ed effettua delle rotazioni. La chiamata RB-INSERT(T, z) inserisce il nodo z, il cui attributo *key* si suppone sia stato già riempito, nell'albero rosso-nero T.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 . Numero Ordine Libreria: 199503016-2207707-0 Copyright © 2022 . McGraw-Hill Education (Italy)

```
RB-INSERT(T, z)
 1  y = T.nil
 2  x = T.root
 3  while x ≠ T.nil
 4       y = x
 5       if z.key < x.key
 6            x = x.left
 7       else x = x.right
 8  z.p = y
 9  if y == T.nil
10       T.root = z
11  elseif z.key < y.key
12       y.left = z
13  else y.right = z
14  z.left = T.nil
15  z.right = T.nil
16  z.color = RED
17  RB-INSERT-FIXUP(T, z)
```

Ci sono quattro differenze fra le procedure TREE-INSERT e RB-INSERT. In primo luogo, tutte le istanze di NIL in TREE-INSERT sono sostituite con *T.nil*. In secondo luogo, assegniamo *T.nil* a *z.left* e *z.right* nelle righe 14–15 di RB-INSERT, per mantenere la struttura appropriata dell'albero. In terzo luogo, coloriamo z di rosso nella riga 16. In quarto luogo, poiché colorare z di rosso può causare una violazione di una delle proprietà degli alberi rosso-neri, chiamiamo RB-INSERT-FIXUP(T, z) nella riga 17 di RB-INSERT per ripristinare tali proprietà.

```
RB-INSERT-FIXUP(T, z)
 1  while z.p.color == RED
 2       if z.p == z.p.p.left
 3            y = z.p.p.right
 4            if y.color == RED
 5                 z.p.color = BLACK                    // Caso 1
 6                 y.color = BLACK                      // Caso 1
 7                 z.p.p.color = RED                    // Caso 1
 8                 z = z.p.p                            // Caso 1
 9            else if z == z.p.right
10                      z = z.p                         // Caso 2
11                      LEFT-ROTATE(T, z)               // Caso 2
12                 z.p.color = BLACK                    // Caso 3
13                 z.p.p.color = RED                    // Caso 3
14                 RIGHT-ROTATE(T, z.p.p)               // Caso 3
15       else (come la clausola then
                 con "right" e "left" scambiati)
16  T.root.color = BLACK
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Per capire come opera RB-INSERT-FIXUP, dovremo scindere l'analisi del codice in tre fasi principali. Nella prima fase, determineremo quali violazioni delle

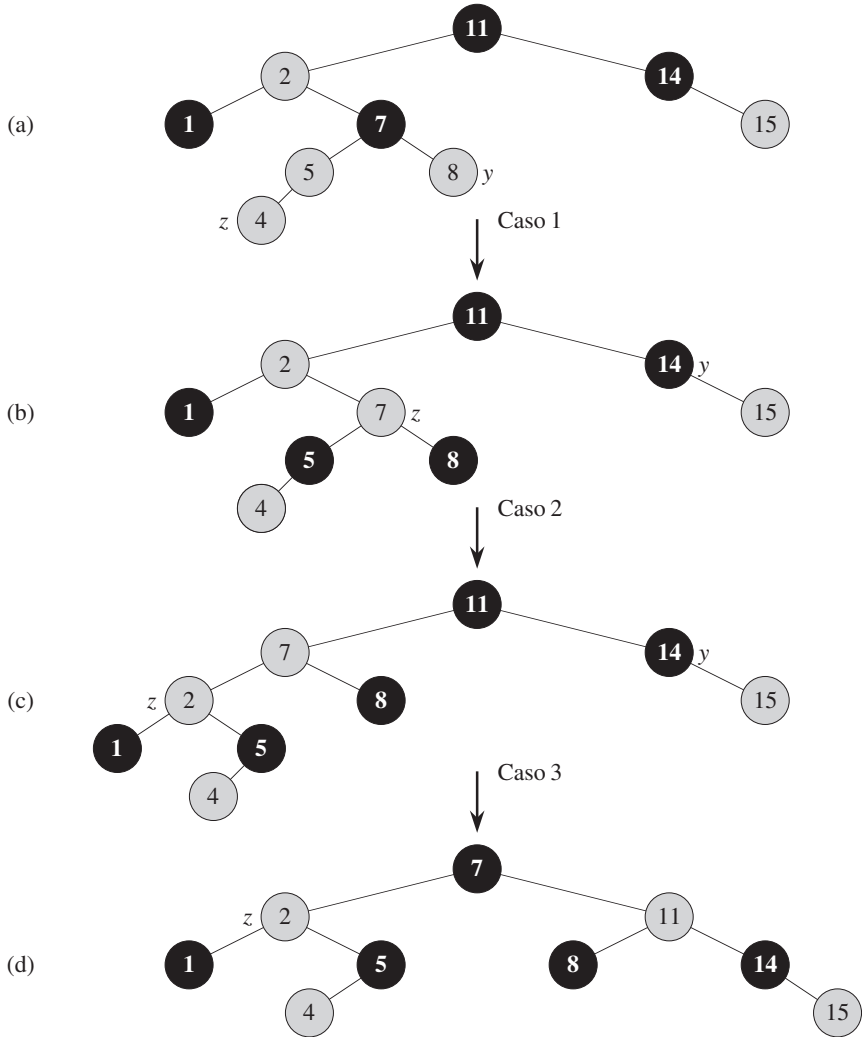


Figura 13.4
Il funzionamento di RB-INSERT-FIXUP. **(a)** Un nodo z dopo l'inserimento. Poiché z e suo padre $z.p$ sono entrambi rossi, si verifica una violazione della proprietà 4. Poiché lo zio y di z è rosso, si applica il caso 1 del codice. I nodi sono ricolorati e il puntatore z si sposta verso l'alto nell'albero, ottenendo l'albero illustrato in **(b)**. Ancora una volta, z e suo padre sono entrambi rossi, ma lo zio y di z è nero. Poiché z è il figlio destro di $z.p$, si applica il caso 2. Viene effettuata una rotazione sinistra; l'albero risultante è illustrato in **(c)**. Adesso z è il figlio sinistro di suo padre e si applica il caso 3. Una ricolorazione e una rotazione destra genera l'albero illustrato in **(d)**, che è un valido albero rosso-nero.

proprietà degli alberi rosso-neri sono introdotte da RB-INSERT quando il nodo z viene inserito e colorato di rosso. Nella seconda fase, esamineremo l'obiettivo globale del ciclo **while** (righe 1–15). Infine, analizzeremo i tre casi[1] in cui è suddiviso il ciclo **while** e vedremo come essi raggiungono l'obiettivo. La Figura 13.4 illustra come opera RB-INSERT-FIXUP in un particolare albero rosso-nero.

Quali proprietà degli alberi rosso-neri possono essere violate prima della chiamata di RB-INSERT-FIXUP? La proprietà 1 certamente continua a essere valida, come la proprietà 3, in quanto entrambi i figli del nuovo nodo rosso che è stato inserito sono la sentinella $T.nil$. Anche la proprietà 5, secondo la quale il numero di nodi neri è lo stesso in ogni cammino semplice da un dato nodo, è soddisfatta perché il nodo z sostituisce la sentinella (nera), e il nodo z è rosso con figli sentinella. Quindi, le uniche proprietà che potrebbero essere violate sono la proprietà 2, che richiede che la radice sia nera, e la proprietà 4, che dice che un nodo rosso non può avere un figlio rosso. Entrambe le possibili violazioni sono causate dal fatto che z viene colorato di rosso. La proprietà 2 è violata se z è la radice; la proprietà 4

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-230707-0 Copyright © 2022 McGraw-Hill Education (Italy)

[1] Il caso 2 ricade nel caso 3, quindi questi due casi non si escludono a vicenda.

è violata se il padre di z è rosso. La Figura 13.4(a) illustra una violazione della proprietà 4 dopo l'inserimento del nodo z. Il ciclo **while** (righe 1–15) mantiene vera, all'inizio di ogni iterazione del ciclo, la seguente invariante composta di tre parti:

All'inizio di ogni iterazione del ciclo:

a. Il nodo z è rosso.
b. Se $z.p$ è la radice, allora $z.p$ è nero.
c. Se ci sono violazioni delle proprietà degli alberi rosso-neri, al massimo ce n'è una e riguarda la proprietà 2 o la proprietà 4. Se c'è una violazione della proprietà 2, questa si verifica perché il nodo z è la radice ed è rosso. Se c'è una violazione della proprietà 4, essa si verifica perché z e $z.p$ sono entrambi rossi.

La parte (c), che tratta le violazioni delle proprietà degli alberi rosso-neri, è più importante delle parti (a) e (b) per dimostrare che RB-INSERT-FIXUP ripristina le proprietà degli alberi rosso-neri. Utilizzeremo le parti (a) e (b) per capire in quale stato ci troviamo durante l'esecuzione del codice. Poiché siamo interessati al nodo z e ai nodi vicini, è utile sapere dalla parte (a) che z è rosso. Utilizzeremo la parte (b) per verificare che il nodo $z.p.p$ esiste, quando faremo riferimento ad esso nelle righe 2, 3, 7, 8, 13 e 14.

Ricordiamo che bisogna dimostrare che un'invariante di ciclo è vera prima della prima iterazione del ciclo, che ogni iterazione conserva l'invariante e che l'invariante fornisce un'utile proprietà alla conclusione del ciclo.

Cominciamo con l'inizializzazione e la conclusione. Poi, quando esamineremo più dettagliatamente come funziona il corpo del ciclo, dimostreremo che il ciclo conserva l'invariante a ogni iterazione. Dimostreremo inoltre che ci sono due possibili risultati per ogni iterazione del ciclo: il puntatore z si sposta verso l'alto nell'albero oppure vengono effettuate delle rotazioni e il ciclo termina.

Inizializzazione: prima della prima iterazione del ciclo, partiamo da un albero rosso-nero senza violazioni a cui abbiamo aggiunto un nodo rosso z. Dimostriamo che tutte le parti dell'invariante sono vere nell'istante in cui viene chiamata la procedura RB-INSERT-FIXUP:

a. Quando viene chiamata RB-INSERT-FIXUP, z è il nodo rosso che è stato aggiunto.
b. Se $z.p$ è la radice, allora $z.p$ è inizialmente nero e non cambia colore prima della chiamata di RB-INSERT-FIXUP.
c. Abbiamo già visto che le proprietà 1, 3 e 5 sono valide quando viene chiamata la procedura RB-INSERT-FIXUP.

   Se c'è una violazione della proprietà 2, allora la radice rossa deve essere il nodo z appena inserito, che in tal caso è l'unico nodo interno nell'albero. Poiché il padre ed entrambi i figli di z sono la sentinella, che è nera, non c'è violazione della proprietà 4. Quindi, questa violazione della proprietà 2 è l'unica violazione delle proprietà degli alberi rosso-neri nell'intero albero.

   Se c'è una violazione della proprietà 4, allora poiché i figli del nodo z sono sentinelle nere e l'albero non aveva altre violazioni prima dell'inserimento di z, la violazione deve essere attribuita al fatto che z e $z.p$ sono entrambi rossi. Non ci sono altre violazioni delle proprietà degli alberi rosso-neri.

Acquistato da Michele Michele su [illegible] - Copyright © 2022 McGraw-Hill Education (Italy)

Conclusione: quando il ciclo termina, ciò accade perché $z.p$ è nero (se z è la radice, allora $z.p$ è la sentinella $T.nil$, che è nera). Quindi, non c'è violazione della proprietà 4 alla conclusione del ciclo. Per l'invariante di ciclo, l'unica proprietà che potrebbe essere violata è la proprietà 2. La riga 16 ripristina anche questa proprietà, cosicché quando RB-INSERT-FIXUP termina, tutte le proprietà degli alberi rosso-neri sono valide.

Conservazione: in effetti, ci sarebbero sei casi da considerare nel ciclo **while**, ma tre di essi sono simmetrici agli altri tre, a seconda che il padre $z.p$ di z sia un figlio sinistro o un figlio destro del nonno $z.p.p$ di z; ciò è determinato nella riga 2. Abbiamo riportato il codice soltanto per la situazione in cui $z.p$ è un figlio sinistro. Il nodo $z.p.p$ esiste, in quanto per la parte (b) dell'invariante di ciclo, se $z.p$ è la radice, allora $z.p$ è nero. Poiché entriamo in una iterazione del ciclo soltanto se $z.p$ è rosso, sappiamo che $z.p$ non può essere la radice. Quindi, $z.p.p$ esiste.

Il caso 1 si distingue dai casi 2 e 3 per il colore del fratello del padre di z (lo "zio" di z). La riga 3 fa sì che y punti allo zio $z.p.p.right$ di z e la riga 4 effettua un test. Se y è rosso, allora viene applicato il caso 1, altrimenti il controllo passa ai casi 2 e 3. In tutti e tre i casi, il nonno $z.p.p$ di z è nero, in quanto il padre $z.p$ è rosso, quindi la proprietà 4 è violata soltanto fra z e $z.p$.

Caso 1: lo zio y di z è rosso

La Figura 13.5 illustra la situazione per il caso 1 (righe 5–8). Questo caso viene eseguito quando $z.p$ e y sono entrambi rossi. Poiché $z.p.p$ è nero, possiamo colorare di nero $z.p$ e y, risolvendo così il problema che z e $z.p$ sono entrambi rossi; coloriamo di rosso $z.p.p$ per conservare la proprietà 5. Poi ripetiamo il ciclo **while** con $z.p.p$ come il nuovo nodo z. Il puntatore z si sposta di due livelli in alto nell'albero. Adesso dimostriamo che il caso 1 conserva l'invariante di ciclo all'inizio della successiva iterazione. Utilizziamo z per indicare il nodo z nell'iterazione corrente e $z' = z.p.p$ per indicare il nodo z nel test della riga 1 prima della successiva iterazione.

a. Poiché questa iterazione colora di rosso $z.p.p$, il nodo z' è rosso all'inizio della successiva iterazione.
b. Il nodo $z'.p$ è $z.p.p.p$ in questa iterazione e il colore di questo nodo non cambia. Se questo nodo è la radice, il suo colore era nero prima di questa iterazione e resta nero all'inizio della successiva iterazione.
c. Abbiamo già dimostrato che il caso 1 conserva la proprietà 5 e, chiaramente, non introduce una violazione delle proprietà 1 e 3.

   Se il nodo z' è la radice all'inizio della successiva iterazione, allora il caso 1 ha corretto l'unica violazione della proprietà 4 nella corrente iterazione. Poiché z' è rosso ed è la radice, la proprietà 2 diventa l'unica a essere violata e questa violazione è dovuta a z'.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 - Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

   Se il nodo z' non è la radice all'inizio della successiva iterazione, allora il caso 1 non ha creato una violazione della proprietà 2. Il caso 1 ha corretto l'unica violazione della proprietà 4 che esisteva all'inizio della corrente iterazione. Poi ha colorato di rosso z' e ha lasciato invariato $z'.p$. Se $z'.p$ era nero, non c'è violazione della proprietà 4. Se $z'.p$ era rosso, la colorazione di rosso di z' ha creato una violazione della proprietà 4 fra z' e $z'.p$.

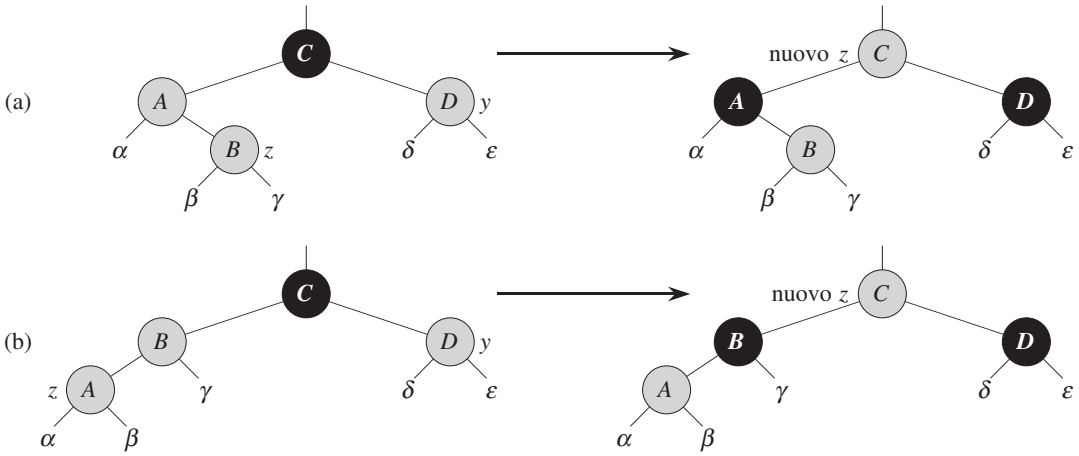


Figura 13.5 Il caso 1 della procedura RB-INSERT-FIXUP. La proprietà 4 è violata, in quanto z e suo padre $z.p$ sono entrambi rossi. La stessa azione viene svolta se **(a)** z è un figlio destro o **(b)** z è un figlio sinistro. I sottoalberi α, β, γ, δ e ε hanno la radice nera e la stessa altezza nera. Il codice per il caso 1 cambia i colori di qualche nodo, preservando la proprietà 5: tutti i cammini semplici che scendono da un nodo a una foglia hanno lo stesso numero di nodi neri. Il ciclo **while** continua con il nonno $z.p.p$ di z come il nuovo nodo z. Qualsiasi violazione della proprietà 4 adesso può verificarsi soltanto fra il nuovo nodo z, che è rosso, e suo padre, se anche questo nodo è rosso.

Caso 2: lo zio y di z è nero e z è un figlio destro
Caso 3: lo zio y di z è nero e z è un figlio sinistro

Nei casi 2 e 3, il colore dello zio y di z è nero. I due casi si distinguono a seconda che z sia un figlio destro o sinistro di $z.p$. Le righe 10–11 costituiscono il caso 2, che è illustrato nella Figura 13.6 insieme con il caso 3. Nel caso 2, il nodo z è un figlio destro di suo padre. Effettuiamo immediatamente una rotazione sinistra per trasformare la situazione nel caso 3 (righe 12–14), in cui il nodo z è un figlio sinistro. Poiché z e $z.p$ sono entrambi rossi, la rotazione non influisce né sull'altezza nera dei nodi né sulla proprietà 5. Sia che entriamo nel caso 3 direttamente o tramite il caso 2, lo zio y di z è nero, perché altrimenti avremmo eseguito il caso 1. In aggiunta, il nodo $z.p.p$ esiste, perché abbiamo dimostrato che questo nodo esisteva quando sono state eseguite le righe 2 e 3; inoltre, dopo avere spostato z di un livello in alto nella riga 10 e poi di un livello in basso nella riga 11, l'identità di $z.p.p$ resta invariata. Nel caso 3, cambiamo qualche colore ed effettuiamo una rotazione destra per preservare la proprietà 5; dopodiché, dal momento che non abbiamo più due nodi rossi consecutivi, abbiamo finito. Il corpo del ciclo **while** non viene eseguito un'altra volta, in quanto $z.p$ ora è nero.

Adesso dimostriamo che i casi 2 e 3 conservano l'invariante di ciclo (come appena dimostrato, $z.p$ sarà nero prima del successivo test nella riga 1 e il corpo del ciclo non sarà eseguito di nuovo).

Acquistato da Michele Michelo (Michelo_1202-27-07-2022) - Vietata la riproduzione non autorizzata - 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

a. Il caso 2 fa sì che z punti a $z.p$, che è rosso. Non ci sono altre modifiche di z o del suo colore nei casi 2 e 3.
b. Il caso 3 colora di nero $z.p$, per cui se $z.p$ è la radice, essa è nera all'inizio della successiva iterazione.
c. Come nel caso 1, anche nei casi 2 e 3 le proprietà 1, 3 e 5 si conservano.

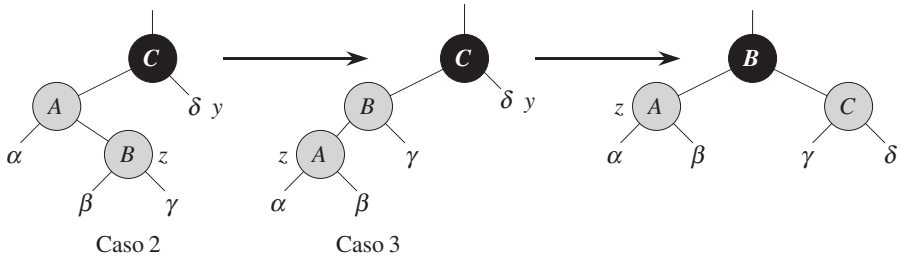


Figura 13.6 I casi 2 e 3 della procedura RB-INSERT-FIXUP. Come nel caso 1, la proprietà 4 è violata nel caso 2 e nel caso 3 perché z e suo padre $z.p$ sono entrambi rossi. I sottoalberi α, β, γ e δ hanno le radici nere (α, β e γ per la proprietà 4 e δ perché altrimenti saremmo nel caso 1) e la stessa altezza nera. Il caso 2 è trasformato nel caso 3 da una rotazione sinistra, che preserva la proprietà 5: ogni cammino semplice che scende da un nodo a una foglia ha lo stesso numero di nodi neri. Il caso 3 effettua qualche cambio di colore ed una rotazione destra che pure preservano la proprietà 5. Il ciclo **while** termina, perché la proprietà 4 è soddisfatta: non ci sono più due nodi rossi consecutivi.

Poiché nei casi 2 e 3 il nodo z non è la radice, sappiamo che non c'è violazione della proprietà 2. I casi 2 e 3 non introducono una violazione della proprietà 2, perché l'unico nodo che viene colorato rosso diventa figlio di un nodo nero per la rotazione effettuata nel caso 3.

I casi 2 e 3 correggono l'unica violazione della proprietà 4 e non introducono un'altra violazione.

Avendo dimostrato che ogni iterazione del ciclo conserva l'invariante, abbiamo verificato che la procedura RB-INSERT-FIXUP ripristina correttamente le proprietà degli alberi rosso-neri.

Analisi

Qual è il tempo di esecuzione di RB-INSERT? Poiché l'altezza di un albero rosso-nero di n nodi è $O(\lg n)$, le righe 1–16 di RB-INSERT richiedono il tempo $O(\lg n)$. Nella procedura RB-INSERT-FIXUP, il ciclo **while** viene ripetuto soltanto se viene eseguito il caso 1 e in tal caso il puntatore z si sposta di due livelli in alto nell'albero. Il numero totale di volte che può essere eseguito il ciclo **while** è quindi $O(\lg n)$. Di conseguenza, RB-INSERT richiede un tempo totale pari a $O(\lg n)$. È interessante notare che il ciclo **while** non effettua mai più di due rotazioni, perché termina se viene eseguito il caso 2 o il caso 3.

Esercizi

13.3-1

Nella riga 16 di RB-INSERT, coloriamo di rosso il nodo z appena inserito. Notate che se avessimo scelto di colorare di nero il nodo z, allora la proprietà 4 di un albero rosso-nero non sarebbe stata violata. Perché non abbiamo scelto di colorare di nero il nodo z?

13.3-2

Illustrate gli alberi rosso-neri che si ottengono dopo avere inserito in successione le chiavi 41, 38, 31, 12, 19, 8 in un albero rosso-nero inizialmente vuoto.

13.3-3

Supponete che l'altezza nera di ciascuno dei sottoalberi $\alpha, \beta, \gamma, \delta, \varepsilon$ nelle Figure 13.5 e 13.6 sia k. Etichettate i nodi in ogni figura con la loro altezza nera per verificare che la proprietà 5 è preservata dalla trasformazione indicata.

Acquistato da Michele Michele su Webster il 2022-07-30 23:11:1 Numero Ordine Libraria 1005033016 P20707.0 Copyright © 2022, McGraw-Hill Education (Italy)

13.3-4

Il professor Teach teme che RB-INSERT-FIXUP possa porre $T.nil.color$ a RED, nel qual caso il test nella riga 1 non farebbe terminare il ciclo quando z è la radice. Dimostrate che il timore del professore è infondato verificando che RB-INSERT-FIXUP non assegna mai RED a $T.nil.color$.

13.3-5
Considerate un albero rosso-nero che viene formato inserendo n nodi mediante la procedura RB-INSERT. Dimostrate che, se $n > 1$, l'albero ha almeno un nodo rosso.

13.3-6
Spiegate come implementare una procedura RB-INSERT efficiente se la rappresentazione degli alberi rosso-neri non prevede lo spazio per i puntatori ai nodi padre.

13.4 Cancellazione

Analogamente ad altre operazioni elementari su un albero rosso-nero di n nodi, la cancellazione di un nodo richiede un tempo $O(\lg n)$. La rimozione di un nodo da un albero rosso-nero è un'operazione soltanto un po' più complicata dell'inserimento di un nodo.

La procedura per cancellare un nodo da un albero rosso-nero si basa sulla procedura TREE-DELETE (descritta nel Paragrafo 12.3). Innanzi tutto, dobbiamo sistemare la subroutine TRANSPLANT, che è chiamata da TREE-DELETE, in modo che possa essere applicata a un albero rosso-nero:

```
RB-TRANSPLANT(T, u, v)
1  if u.p == T.nil
2      T.root = v
3  elseif u == u.p.left
4      u.p.left = v
5  else u.p.right = v
6  v.p = u.p
```

Ci sono due differenze fra le procedure RB-TRANSPLANT e TRANSPLANT. In primo luogo, la riga 1 fa riferimento alla sentinella $T.nil$, anziché a NIL. In secondo luogo, l'assegnazione a $v.p$ nella riga 6 si verifica in modo incondizionato: possiamo fare un'assegnazione a $v.p$ anche se v punta alla sentinella. In effetti, ci è permesso sfruttare la possibilità di fare un'assegnazione a $v.p$ quando $v = T.nil$.

La procedura RB-DELETE è simile alla procedura TREE-DELETE, a parte alcune righe aggiuntive di pseudocodice. Alcune delle righe addizionali tengono traccia di un nodo y che potrebbe causare delle violazioni delle proprietà degli alberi rosso-neri. Quando vogliamo cancellare il nodo z e z ha meno di due figli, z viene rimosso dall'albero, e y deve diventare z. Quando z ha due figli, y deve essere il successore di z, quindi viene spostato nella posizione di z nell'albero. Dobbiamo ricordarci del colore di y prima di rimuoverlo o di spostarlo all'interno dell'albero e tenere traccia del nodo x che si sposta nella posizione originale di y, perché anche il nodo x potrebbe causare delle violazioni delle proprietà degli alberi rosso-neri. Dopo aver cancellato il nodo z, RB-DELETE chiama una procedura ausiliaria RB-DELETE-FIXUP, che cambia i colori ed effettua delle rotazioni per ripristinare le proprietà degli alberi rosso-neri.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

```
RB-DELETE(T, z)
 1  y = z
 2  y-original-color = y.color
 3  if z.left == T.nil
 4       x = z.right
 5       RB-TRANSPLANT(T, z, z.right)
 6  elseif z.right == T.nil
 7       x = z.left
 8       RB-TRANSPLANT(T, z, z.left)
 9  else y = TREE-MINIMUM(z.right)
10       y-original-color = y.color
11       x = y.right
12       if y.p == z
13            x.p = y
14       else RB-TRANSPLANT(T, y, y.right)
15            y.right = z.right
16            y.right.p = y
17       RB-TRANSPLANT(T, z, y)
18       y.left = z.left
19       y.left.p = y
20       y.color = z.color
21  if y-original-color == BLACK
22       RB-DELETE-FIXUP(T, x)
```

Sebbene RB-DELETE contenga quasi il doppio delle righe di pseudocodice di TREE-DELETE, le due procedure hanno la stessa struttura di base. È possibile trovare le righe di TREE-DELETE all'interno di RB-DELETE (sostituendo NIL con *T.nil* e le chiamate di TRANSPLANT con le chiamate di RB-TRANSPLANT), eseguite sotto le stesse condizioni.

Ecco le altre differenze tra le due procedure:

- Manteniamo il nodo y come il nodo rimosso dall'albero o spostato all'interno dell'albero. La riga 1 imposta y in modo da puntare al nodo z, quando z ha meno di due figli; poi z viene rimosso. Quando z ha due figli, la riga 9 imposta y in modo da puntare al successore di z, proprio come nella procedura TREE-DELETE; y sarà spostato nella posizione di z all'interno dell'albero.
- Poiché il colore del nodo y potebbe cambiare, la variabile *y-original-color* memorizza il colore di y prima che avvenga qualche modifica. Le righe 2 e 10 impostano questa variabile subito dopo l'assegnazione di y. Quando z ha due figli, allora $y \neq z$ e il nodo y si sposta nella posizione originale del nodo z nell'albero rosso-nero; la riga 20 assegna a y lo stesso colore di z. Dobbiamo salvare il colore originale di y per controllarlo alla fine della procedura RB-DELETE: se fosse nero, la rimozione o lo spostamento di y potrebbe causare una violazione delle proprietà degli alberi rosso-neri.
- Come detto in precedenza, teniamo traccia del nodo x che si sposta nella posizione originale del nodo y. Le assegnazioni delle righe 4, 7 e 11 impostano x in modo da puntare all'unico figlio di y oppure, se y non ha figli, alla sentinella *T.nil* (ricordiamo dal Paragrafo 12.3 che y non ha un figlio sinistro.)

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

- Poiché il nodo x si sposta nella posizione del nodo y, l'attributo $x.p$ è sempre impostato per puntare alla posizione originale del padre di y, anche se x è, in effetti, la sentinella $T.nil$. A meno che z non sia il padre originale di y (questo accade soltanto se z ha due figli e il suo successore y è il figlio destro di z), l'assegnazione a $x.p$ avviene nella riga 6 della procedura RB-TRANSPLANT. (Si noti che, quando RB-TRANSPLANT viene chiamata nelle righe 5, 8 o 14, il secondo parametro passato è uguale a x.)

  Tuttavia, quando il padre originale di y è z, non vogliamo che $x.p$ punti al padre originale di y, perché stiamo eliminando quel nodo dall'albero. Poiché il nodo y si sposterà in alto per prendere la posizione di z, la riga 13 imposta $x.p$ a y in modo che $x.p$ punti alla posizione originale del padre di y, anche se $x = T.nil$.

- Infine, se il nodo y fosse nero, potremmo introdurre una o più violazioni delle proprietà degli alberi rosso-neri; quindi chiamiamo la procedura RB-DELETE-FIXUP nella riga 22 per ripristinare le proprietà. Se y fosse rosso, le proprietà degli alberi rosso-neri sarebbero ancora valide quando y viene rimosso o spostato, per le seguenti ragioni:

  1. Le altezze nere nell'albero non sono cambiate.
  2. Non sono stati creati nodi rossi consecutivi. Poiché y prende la posizione di z nell'albero, ma anche il colore di z, non possiamo avere due nodi rossi consecutivi nella nuova posizione di y. Inoltre, se y non è il figlio destro di z, l'originale figlio destro x di y sostituisce y nell'albero. Se y è rosso, x deve essere nero e, quindi, sostituendo y con x non si potrebbero creare due nodi rossi consecutivi.
  3. La radice resta nera perché, se il nodo y fosse stato rosso, non sarebbe potuto essere la radice.

Se il nodo y rimosso nella procedura RB-DELETE è nero, potrebbero verificarsi tre problemi, che saranno risolti dalla procedura RB-DELETE-FIXUP. In primo luogo, se y era la radice e un figlio rosso di y diventa la nuova radice, abbiamo violato la proprietà 2. In secondo luogo, se entrambi i nodi x e $x.p$ erano rossi, allora abbiamo violato la proprietà 4. In terzo luogo, spostando y all'interno dell'albero, qualsiasi cammino semplice che in precedenza conteneva y, adesso ha un nodo nero in meno; quindi, la proprietà 5 è violata da qualsiasi antenato di y nell'albero. Possiamo correggere la violazione della proprietà 5 dicendo che il nodo x, che ora occupa la posizione originale di y, ha un colore nero "extra". Ovvero, se aggiungiamo 1 al conteggio dei nodi neri in qualsiasi cammino semplice che contiene x, allora con questa interpretazione la proprietà 5 è valida. Quando eliminiamo o spostiamo il nodo nero y, "aggiungiamo" il suo colore al nodo x. Il problema è che adesso il nodo x non è né rosso né nero, violando così la proprietà 1. Il nodo x è "doppiamente nero" o "rosso e nero" e contribuisce, rispettivamente, con 2 o 1 al conteggio dei nodi neri nei cammini semplici che contengono x. L'attributo *color* di x sarà ancora RED (se x è rosso e nero) o BLACK (se x è doppiamente nero). In altre parole, il nero extra in un nodo è legato al fatto che si punta a tale nodo, non al valore dell'attributo *color*.

Acquistato da Michele Michele con licenza non trasferibile. Copyright © 2023 McGraw-Hill Education (Italy)

Adesso possiamo esaminare come la procedura RB-DELETE-FIXUP ripristina le proprietà degli alberi rosso-neri nell'albero di ricerca.

```
RB-DELETE-FIXUP(T, x)
 1  while x ≠ T.root and x.color == BLACK
 2      if x == x.p.left
 3          w = x.p.right
 4          if w.color == RED
 5              w.color = BLACK                          // Caso 1
 6              x.p.color = RED                          // Caso 1
 7              LEFT-ROTATE(T, x.p)                      // Caso 1
 8              w = x.p.right                            // Caso 1
 9          if w.left.color == BLACK and w.right.color == BLACK
10              w.color = RED                            // Caso 2
11              x = x.p                                  // Caso 2
12          else if w.right.color == BLACK
13                  w.left.color = BLACK                 // Caso 3
14                  w.color = RED                        // Caso 3
15                  RIGHT-ROTATE(T, w)                   // Caso 3
16                  w = x.p.right                        // Caso 3
17              w.color = x.p.color                      // Caso 4
18              x.p.color = BLACK                        // Caso 4
19              w.right.color = BLACK                    // Caso 4
20              LEFT-ROTATE(T, x.p)                      // Caso 4
21              x = T.root                               // Caso 4
22      else (come la clausola then con "right" e "left" scambiati)
23  x.color = BLACK
```

La procedura RB-DELETE-FIXUP ripristina le proprietà 1, 2 e 4. Gli esercizi 13.4-1 e 13.4-2 chiedono di dimostrare che la procedura ripristina le proprietà 2 e 4; quindi nella parte restante di questo paragrafo, concentreremo la nostra analisi sulla proprietà 1. L'obiettivo del ciclo **while** (righe 1–22) è spostare il nero extra in alto nell'albero finché

1. x punta a un nodo rosso e nero, nel qual caso coloriamo (singolarmente) di nero x nella riga 23,
2. x punta alla radice, nel qual caso il nero extra può essere semplicemente "rimosso" oppure
3. vengono effettuate opportune rotazioni e ricolorazioni.

All'interno del ciclo **while**, x punta sempre a un nodo doppiamente nero che non è la radice. Determiniamo nella riga 2 se x è un figlio sinistro o un figlio destro di suo padre $x.p$. (Abbiamo riportato il codice per la situazione in cui x è un figlio sinistro; la situazione in cui x è un figlio destro – riga 22 – è simmetrica.) Manteniamo un puntatore w al fratello di x. Poiché il nodo x è doppiamente nero, il nodo w non può essere $T.nil$, altrimenti il numero di nodi neri nel cammino semplice da $x.p$ alla foglia w (singolarmente nera) sarebbe più piccolo del numero di nodi neri nel cammino semplice da $x.p$ a x.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 1995032162207070. Copyright © 2022 McGraw-Hill Education (Italy)

I quattro casi[2] del codice sono illustrati nella Figura 13.7. Prima di esaminare in dettaglio i singoli casi, analizziamo più in generale come possiamo verificare

[2]Come nella procedura RB-INSERT-FIXUP, i casi di RB-DELETE-FIXUP non si escludono a vicenda.

che la trasformazione in ciascuno di questi casi preserva la proprietà 5. Il concetto base è che in ciascun caso il numero di nodi neri (incluso il nero extra di x) dalla radice (inclusa) del sottoalbero illustrato fino a ciascuno dei sottoalberi $\alpha, \beta, \ldots, \zeta$ è preservato dalla trasformazione. Quindi, se la proprietà 5 è valida prima della trasformazione, continua ad essere valida anche dopo. Per esempio, nella Figura 13.7(a), che illustra il caso 1, il numero di nodi neri dalla radice al sottoalbero α o β è 3, sia prima che dopo la trasformazione (ricordiamo che il nodo x contribuisce con un nero extra). Analogamente, il numero di nodi neri dalla radice a uno dei sottoalberi γ, δ, ε, e ζ è 2, sia prima che dopo la trasformazione. Nella Figura 13.7(b), il conteggio deve includere il valore c dell'attributo *color* della radice del sottoalbero illustrato, che può essere RED o BLACK. Se definiamo count(RED) $= 0$ e count(BLACK) $= 1$, allora il numero di nodi neri dalla radice ad α è $2 + \text{count}(c)$, sia prima che dopo la trasformazione. In questo caso, dopo la trasformazione, il nuovo nodo x ha l'attributo *color* uguale a c, ma questo nodo, in effetti, è rosso e nero (se $c =$ RED) o doppiamente nero (se $c =$ BLACK). Gli altri casi possono essere verificati in maniera analoga (vedere l'Esercizio 13.4-5).

Caso 1: il fratello w di x è rosso

Il caso 1 (righe 5–8 della procedura RB-DELETE-FIXUP e Figura 13.7(a)) si verifica quando il nodo w, il fratello del nodo x, è rosso. Poiché w deve avere i figli neri, possiamo scambiare i colori di w e $x.p$ e poi effettuare una rotazione sinistra di $x.p$, senza violare nessuna delle proprietà degli alberi rosso-neri. Il nuovo fratello di x, che era uno dei figli di w prima della rotazione, adesso è nero, e quindi abbiamo trasformato il caso 1 nel caso 2, 3 o 4.

I casi 2, 3 e 4 si verificano quando il nodo w è nero; si distinguono per i colori dei figli di w.

Caso 2: il fratello w di x è nero ed entrambi i figli di w sono neri

Nel caso 2 (righe 10–11 della procedura RB-DELETE-FIXUP e Figura 13.7(b)), entrambi i figli di w sono neri. Poiché anche w è nero, togliamo un nero sia da x sia da w, lasciando x con un solo nero e w rosso. Per compensare la rimozione di un nero da x e w, aggiungiamo un nero extra a $x.p$, che originariamente era rosso o nero. Per farlo, ripetiamo il ciclo **while** con $x.p$ come il nuovo nodo x. Notate che, se entriamo nel caso 2 attraverso il caso 1, il nuovo nodo x è rosso e nero, perché l'originale $x.p$ era rosso. Quindi, il valore c dell'attributo *color* del nuovo nodo x è RED; il ciclo termina con la verifica della condizione del ciclo. Il nuovo nodo x viene poi colorato (singolarmente) di nero nella riga 23.

Caso 3: il fratello w di x è nero, il figlio sinistro di w è rosso e il figlio destro di w è nero

Il caso 3 (righe 13–16 e Figura 13.7(c)) si verifica quando w è nero, suo figlio sinistro è rosso e suo figlio destro è nero. Possiamo scambiare i colori di w e di sui figlio sinistro $w.left$ e poi effettuare una rotazione destra di w, senza violare nessuna delle proprietà degli alberi rosso-neri. Il nuovo fratello w di x adesso è un nodo nero con un figlio destro rosso, quindi abbiamo trasformato il caso 3 nel caso 4.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

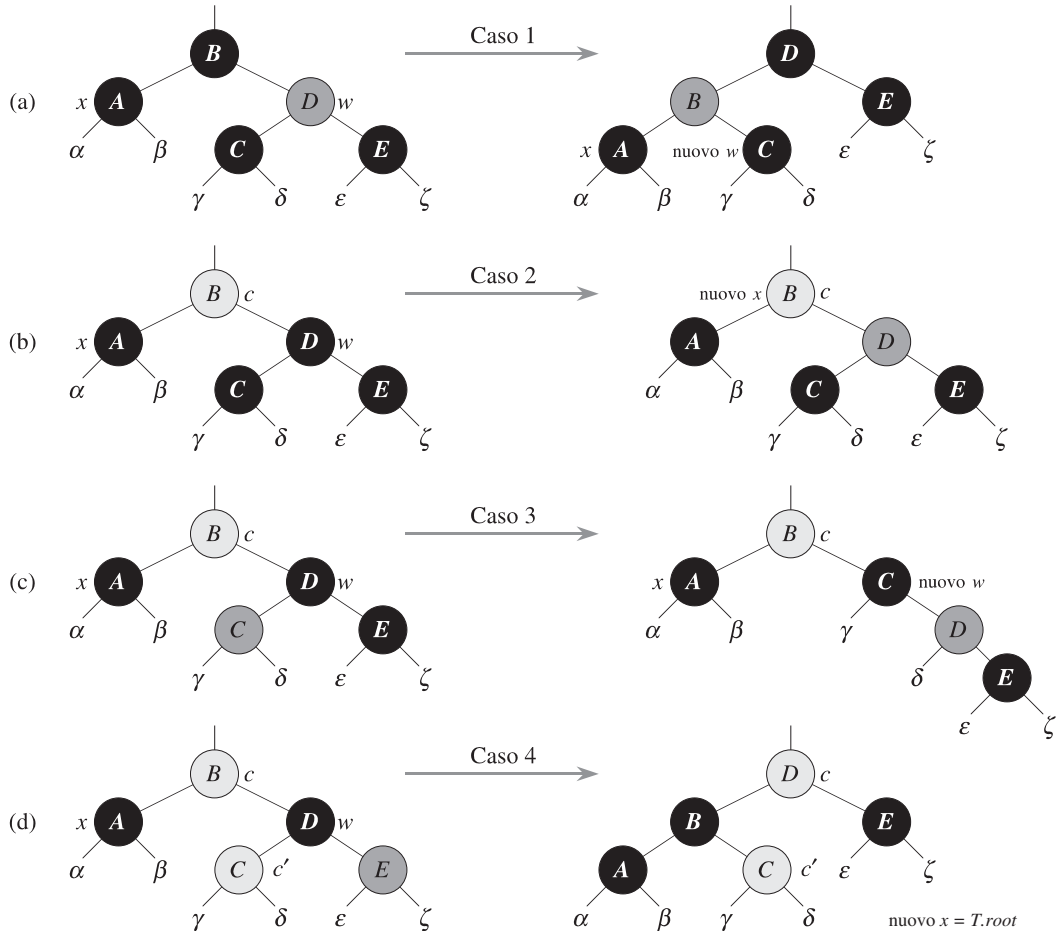


Figura 13.7 I casi nel ciclo **while** della procedura RB-DELETE-FIXUP. I nodi su sfondo nero hanno l'attributo *color* uguale a BLACK, quelli su sfondo grigio scuro hanno l'attributo *color* uguale a RED e quelli su sfondo grigio chiaro hanno gli attributi *color* rappresentati da c e c', che possono essere RED o BLACK. Le lettere $\alpha, \beta, \ldots, \zeta$ rappresentano sottoalberi arbitrari. In ogni caso, la configurazione a sinistra è trasformata nella configurazione a destra cambiando qualche colore e/o effettuando una rotazione. Qualsiasi nodo puntato da x ha un nero extra e può essere doppiamente nero o rosso e nero. L'unico caso che provoca la ripetizione del ciclo è il caso 2. **(a)** Il caso 1 è trasformato nel caso 2, 3 o 4 scambiando i colori dei nodi B e D ed effettuando una rotazione sinistra. **(b)** Nel caso 2, il nero extra associato al puntatore x viene spostato in alto nell'albero colorando di rosso il nodo D e puntando x al nodo B. Se entriamo nel caso 2 attraverso il caso 1, il ciclo **while** termina perché il nuovo nodo x è rosso e nero, pertanto il valore c del suo attributo *color* è RED. **(c)** Il caso 3 è trasformato nel caso 4 scambiando i colori dei nodi C e D ed effettuando una rotazione destra. **(d)** Nel caso 4, il nero extra associato a x può essere rimosso cambiando qualche colore ed effettuando una rotazione sinistra (senza violare le proprietà degli alberi rosso-neri); poi il ciclo termina.

Caso 4: il fratello w di x è nero e il figlio destro di w è rosso

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950816-230707-0 Copyright © 2022 McGraw-Hill Education (Italy)

Il caso 4 (righe 17–21 e Figura 13.7(d)) si verifica quando il fratello w del nodo x è nero e il figlio destro di w è rosso. Cambiando qualche colore ed effettuando una rotazione sinistra di $x.p$, possiamo rimuovere il nero extra da x, rendendolo singolarmente nero, senza violare nessuna delle proprietà degli alberi rosso-neri. Se alla fine x punta alla radice, il ciclo **while** termina con la verifica della condizione del ciclo.

Analisi

Qual è il tempo di esecuzione di RB-DELETE? Poiché l'altezza di un albero rosso-nero di n nodi è $O(\lg n)$, il costo totale della procedura senza la chiamata di RB-DELETE-FIXUP richiede il tempo $O(\lg n)$. All'interno di RB-DELETE-FIXUP, ciascuno dei casi 1, 3 e 4 termina dopo avere effettuato un numero costante di ricolorazioni e al massimo tre rotazioni. Il caso 2 è l'unico caso in cui il ciclo **while** può essere ripetuto; quindi il puntatore x si sposta verso l'alto nell'albero al massimo $O(\lg n)$ volte e non viene effettuata alcuna rotazione. Dunque, la procedura RB-DELETE-FIXUP richiede il tempo $O(\lg n)$ ed effettua al massimo tre rotazioni; in definitiva, anche il tempo totale di RB-DELETE è $O(\lg n)$.

Esercizi

13.4-1
Dimostrate che, dopo l'esecuzione della procedura RB-DELETE-FIXUP, la radice dell'albero deve essere nera.

13.4-2
Dimostrate che, se nella procedura RB-DELETE x e $x.p$ sono entrambi rossi, allora la proprietà 4 è ripristinata dalla chiamata RB-DELETE-FIXUP(T, x).

13.4-3
Nell'Esercizio 13.3-2 avete trovato l'albero rosso-nero che si ottiene inserendo in successione le chiavi 41, 38, 31, 12, 19, 8 in un albero inizialmente vuoto. Adesso trovate gli alberi rosso-neri che si ottengono cancellando in successione le chiavi 8, 12, 19, 31, 38, 41.

13.4-4
In quali righe del codice di RB-DELETE-FIXUP potremmo esaminare o modificare la sentinella $T.nil$?

13.4-5
Per ciascun caso della Figura 13.7 calcolate il numero di nodi neri dalla radice del sottoalbero illustrato fino a ciascuno dei sottoalberi $\alpha, \beta, \ldots, \zeta$; verificate che ogni conteggio ottenuto non cambia dopo la trasformazione. Quando un nodo ha l'attributo *color* pari a c o c', usate la notazione count(c) o count(c') simbolicamente nei vostri conteggi.

13.4-6
I professori Skelton e Baron temono che, all'inizio del caso 1 della procedura RB-DELETE-FIXUP, il nodo $x.p$ possa non essere nero. Se i professori avessero ragione, allora le righe 5–6 sarebbero sbagliate. Dimostrate che $x.p$ è nero all'inizio del caso 1 e, quindi, il timore dei professori è infondato.

13.4-7
Supponete che un nodo x venga inserito in un albero rosso-nero con la procedura RB-INSERT e, poi, immediatamente cancellato con la procedura RB-DELETE. L'albero rosso-nero risultante è lo stesso di quello iniziale? Spiegate la vostra risposta.

Acquistato da Michele Michelassi Webster il 2022-07-15 21:12 (Università Ortes Libreria 10251301) - ISBN: 9788838692975 (Copyright 2022 McGraw-Hill Education (Italy)

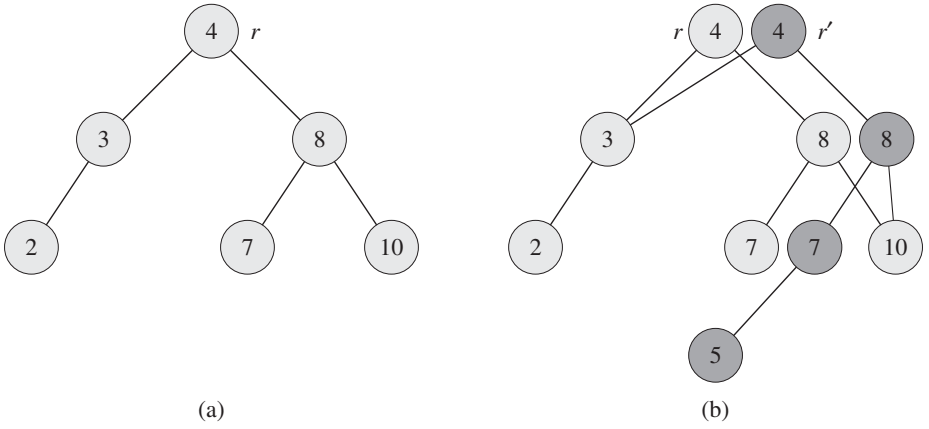


Figura 13.8 **(a)** Un albero binario di ricerca con le chiavi 2, 3, 4, 7, 8, 10. **(b)** L'albero binario di ricerca persistente che si ottiene inserendo la chiave 5. La versione più recente dell'insieme è formata dai nodi raggiungibili dalla radice r', mentre la precedente versione è formata dai nodi raggiungibili da r. I nodi su sfondo più scuro vengono aggiunti quando viene inserita la chiave 5.

Problemi

13-1 Insiemi dinamici persistenti

Durante l'esecuzione di un algoritmo, a volte potrebbe essere necessario conservare le versioni passate di un insieme dinamico mentre viene aggiornato; un insieme così è detto ***persistente***. Un modo per implementare un insieme persistente consiste nel copiare l'intero insieme ogni volta che viene modificato, ma questo approccio può rallentare un programma e anche consumare molto spazio in memoria. In alcuni casi, è possibile fare di meglio.

Considerate un insieme persistente S con le operazioni INSERT, DELETE e SEARCH, che implementiamo utilizzando gli alberi binari di ricerca come illustra la Figura 13.8(a). Manteniamo una radice distinta per ogni versione dell'insieme. Per inserire la chiave 5 nell'insieme, creiamo un nuovo nodo con la chiave 5. Questo nodo diventa il figlio sinistro di un nuovo nodo con la chiave 7, perché non possiamo modificare il nodo esistente con la chiave 7. Analogamente, il nuovo nodo con la chiave 7 diventa il figlio sinistro di un nuovo nodo con la chiave 8 il cui figlio destro è il nodo esistente con la chiave 10. Il nuovo nodo con la chiave 8 diventa, a sua volta, il figlio destro di una nuova radice r' con la chiave 4 il cui figlio sinistro è il nodo esistente con la chiave 3. Quindi, copiamo soltanto una parte dell'albero e condividiamo alcuni nodi con l'albero originale, come illustra la Figura 13.8(b).

Supponiamo che ogni nodo dell'albero abbia gli attributi *key*, *left* e *right*, ma non l'attributo p del padre del nodo (vedere anche l'Esercizio 13.3-6).

a. Per un generico albero binario di ricerca persistente, identificate i nodi che devono essere modificati per inserire una chiave k o cancellare un nodo y.

b. Scrivete una procedura PERSISTENT-TREE-INSERT che, dato un albero persistente T e una chiave k da inserire, restituisce un nuovo albero persistente T' che è il risultato dell'inserimento di k in T.

c. Se l'altezza dell'albero binario di ricerca persistente T è h, quanto spazio e tempo richiede la vostra implementazione di PERSISTENT-TREE-INSERT? (Lo spazio richiesto è proporzionale al numero dei nuovi nodi allocati.)

Acquistato da Michele Michele su Webster il 2022-07-07 Vietata la distribuzione, la copia e la vendita. Copyright © 2022 The McGraw-Hill Education (Italy)

d. Supponete di avere incluso l'attributo p (padre) in ogni nodo. In questo caso, PERSISTENT-TREE-INSERT dovrà svolgere delle operazioni di copia addizionali. Dimostrate che adesso PERSISTENT-TREE-INSERT richiede un tempo e uno spazio pari a $\Omega(n)$, dove n è il numero di nodi dell'albero.

e. Spiegate come utilizzare gli alberi rosso-neri per garantire che lo spazio e il tempo di esecuzione nel caso peggiore siano pari a $O(\lg n)$ per l'inserimento o la cancellazione.

13-2 Congiunzione di alberi rosso-neri

L'operazione di ***congiunzione*** (join) richiede due insiemi dinamici S_1 e S_2 e un elemento x tale che per ogni $x_1 \in S_1$ e $x_2 \in S_2$, si ha $x_1.key \leq x.key \leq x_2.key$. Il risultato è un insieme $S = S_1 \cup \{x\} \cup S_2$. In questo problema descriveremo come implementare l'operazione di congiunzione con gli alberi rosso-neri.

a. Dato un albero rosso-nero T, memorizzate la sua altezza nera nel nuovo attributo $T.bh$. Dimostrate che questo attributo può essere mantenuto da RB-INSERT e RB-DELETE senza richiedere uno spazio extra nei nodi dell'albero e senza aumentare i tempi di esecuzione asintotici. Dimostrate che discendendo lungo l'albero T, è possibile determinare l'altezza nera di ogni nodo nel tempo $O(1)$ per ogni nodo visitato.

Vogliamo implementare l'operazione RB-JOIN(T_1, x, T_2), che distrugge T_1 e T_2 e restituisce un albero rosso-nero $T = T_1 \cup \{x\} \cup T_2$. Sia n il numero totale dei nodi in T_1 e T_2.

b. Supponete che $T_1.bh \geq T_2.bh$. Descrivete un algoritmo con tempo $O(\lg n)$ che trova un nodo nero y in T_1 con la chiave più grande fra quei nodi la cui altezza nera è $T_2.bh$.

c. Sia T_y il sottoalbero con radice nel nodo y. Spiegate come $T_y \cup \{x\} \cup T_2$ può sostituire T_y nel tempo $O(1)$ senza distruggere la proprietà degli alberi binari di ricerca.

d. Di che colore dovrebbe essere colorato x per preservare le proprietà 1, 3 e 5 degli alberi rosso-neri? Spiegate come le proprietà 2 e 4 possano essere ripristinate nel tempo $O(\lg n)$.

e. Dimostrate che l'ipotesi del punto (b) non riduce la generalità del problema. Descrivete la situazione simmetrica che si verifica quando $T_1.bh \leq T_2.bh$.

f. Dimostrate che il tempo di esecuzione di RB-JOIN è $O(\lg n)$.

13-3 Alberi AVL

Un ***albero AVL*** è un albero binario di ricerca che è ***bilanciato in altezza***: per ogni nodo x, le altezze dei sottoalberi sinistro e destro di x differiscono al massimo di 1. Per implementare un albero AVL, bisogna mantenere un attributo extra in ogni nodo: $x.h$ che è l'altezza del nodo x. Come per qualsiasi albero binario di ricerca T, supponiamo che $T.root$ punti al nodo radice.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

a. Dimostrate che un albero AVL di n nodi ha un'altezza $O(\lg n)$ (*suggerimento:* dimostrate che in albero AVL di altezza h, ci sono almeno F_h nodi, dove F_h è l'h-esimo numero di Fibonacci).

b. Per inserire un nodo in un albero AVL, il nodo viene posto prima nella posizione appropriata nell'ordine di un albero binario di ricerca. Dopo questo inserimento, l'albero potrebbe non essere più bilanciato in altezza. Specificatamente, le altezze dei figli di qualche nodo potrebbero differire di 2. Descrivete una procedura BALANCE(x) che prende un sottoalbero con radice in x i cui figli sono bilanciati in altezza e hanno altezze che differiscono al massimo di 2 (cioè $|x.right.h - x.left.h| \leq 2$) e modifica il sottoalbero per bilanciarlo in altezza (*suggerimento:* usate le rotazioni).

c. Utilizzando la parte (b), descrivete una procedura ricorsiva AVL-INSERT(x, z) che prende un nodo x all'interno di un albero AVL e un nodo appena creato z (il cui attributo *key* sia già stato riempito) e aggiunge z al sottoalbero con radice in x, mantenendo la proprietà che x è la radice di un albero AVL. Analogamente alla procedura TREE-INSERT descritta nel Paragrafo 12.3, supponete che l'attributo $z.key$ sia stato già riempito e che $z.left = \text{NIL}$ e $z.right = \text{NIL}$; supponete inoltre che $z.h = 0$. Quindi, per inserire il nodo z nell'albero AVL T, chiamate AVL-INSERT$(T.root, z)$.

d. Dimostrate che la procedura AVL-INSERT, eseguita in un albero AVL di n nodi, impiega il tempo $O(\lg n)$ e svolge $O(1)$ rotazioni.

13-4 Treap

Se inseriamo un insieme di n elementi in un albero binario di ricerca, l'albero risultante potrebbe essere notevolmente sbilanciato, determinando lunghi tempi di ricerca. Tuttavia, come detto nel Paragrafo 12.4, gli alberi binari di ricerca costruiti in modo casuale tendono a essere bilanciati. Di conseguenza, una strategia che, in media, costruisce un albero bilanciato per un insieme fisso di elementi consiste nel permutare in modo casuale gli elementi e poi nell'inserire gli elementi in quell'ordine nell'albero.

Che cosa accade se non abbiamo tutti gli elementi contemporaneamente? Se riceviamo gli elementi uno alla volta, possiamo ancora costruire in modo casuale un albero binario di ricerca con tali elementi?

Esamineremo una struttura dati che risponde affermativamente a questa domanda. Un ***treap*** è un albero binario di ricerca che ha un modo diverso di ordinare i nodi. La Figura 13.9 illustra un esempio. Come al solito, ogni nodo x nell'albero ha un valore chiave $x.key$. In aggiunta, assegniamo una priorità $x.priority$, che è un numero casuale scelto in modo indipendente per ogni nodo. Supponiamo che tutte le priorità e tutte le chiavi siano distinte. I nodi del treap sono ordinati in modo che le chiavi rispettino la proprietà degli alberi binari di ricerca e le priorità rispettino la proprietà di ordinamento del min-heap:

- Se v è un figlio sinistro di u, allora $v.key < u.key$.
- Se v è un figlio destro di u, allora $v.key > u.key$.
- Se v è un figlio di u, allora $v.priority > u.priority$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995030016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Il nome "treap" (ottenuto dalle parole *tre*e e he*ap*) deriva da questa combinazione di proprietà; un treap ha le caratteristiche di un albero binario di ricerca e di un heap. Può essere d'aiuto pensare ai treap nel modo seguente. Supponiamo di inserire in un treap i nodi $x_1, x_2, \ldots, x_n$ con le chiavi associate. Il treap risultante è l'albero che si sarebbe formato se avessimo inserito i nodi in un normale albero binario di ricerca nell'ordine dato dalle loro priorità (scelte in modo casuale), ovvero $x_i.priority < x_j.priority$ significa che x_i è stato inserito prima di x_j.

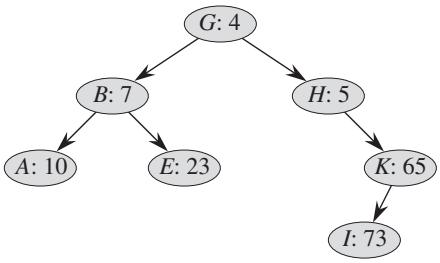


Figura 13.9 Un treap. Ogni nodo x è etichettato con $x.key : x.priority$. Per esempio, la radice ha chiave G e priorità 4.

a. Dimostrate che, dato un insieme di nodi $x_1, x_2, \ldots, x_n$ con relative chiavi e priorità (tutte distinte), esiste un unico treap associato a questi nodi.

b. Dimostrate che l'altezza attesa di un treap è $\Theta(\lg n)$ e, quindi, il tempo per cercare un valore nel treap è $\Theta(\lg n)$.

Vediamo come inserire un nuovo nodo in un treap esistente. La prima cosa da fare è assegnare al nuovo nodo una priorità casuale. Poi chiamiamo l'algoritmo di inserimento TREAP-INSERT, il cui funzionamento è illustrato nella Figura 13.10.

c. Spiegate come funziona TREAP-INSERT. Descrivete il processo in italiano e scrivete lo pseudocodice (*suggerimento:* eseguite la normale procedura di inserimento di un nodo in un albero binario di ricerca e poi effettuate le rotazioni per ripristinare la proprietà di ordinamento di un min-heap).

d. Dimostrate che il tempo di esecuzione atteso di TREAP-INSERT è $\Theta(\lg n)$.

TREAP-INSERT effettua una ricerca e poi una sequenza di rotazioni. Sebbene queste due operazioni abbiamo lo stesso tempo di esecuzione atteso, in pratica i loro costi sono differenti. Una ricerca legge le informazioni dal treap senza modificarle. Una rotazione, invece, cambia i puntatori dei padri e dei figli all'interno del treap. Nella maggior parte dei calcolatori, le operazioni di lettura sono molto più veloci di quelle di scrittura. Quindi, sarebbe preferibile che TREAP-INSERT svolgesse poche rotazioni. Dimostreremo che il numero atteso di rotazioni svolte è limitato da una costante.

Per farlo, ci servono alcune definizioni, che sono illustrate nella Figura 13.11. La ***dorsale sinistra*** di un albero binario di ricerca T è il cammino semplice dalla radice al nodo che ha la chiave più piccola. In altre parole, la dorsale sinistra è il cammino semplice dalla radice che è formato soltanto da archi sinistri. In modo simmetrico, la ***dorsale destra*** di T è il cammino semplice dalla radice che è formato soltanto da archi destri. La ***lunghezza*** di una dorsale è il numero di nodi che contiene.

e. Considerate il treap T subito dopo che TREAP-INSERT ha inserito x. Sia C la lunghezza della dorsale destra del sottoalbero sinistro di x. Sia D la lunghezza della dorsale sinistra del sottoalbero destro di x. Dimostrate che il numero totale di rotazioni che sono state effettuate durante l'inserimento di x è uguale a $C + D$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Adesso calcoliamo i valori attesi di C e D. Senza ridurre la generalità del problema, supponiamo che le chiavi siano $1, 2, \ldots, n$, in quanto le confrontiamo soltanto tra di loro.

Per i nodi x e y, con $y \neq x$, sia $k = x.key$ e $i = y.key$. Definiamo le variabili casuali indicatrici

$X_{i,k} = \mathrm{I}\{y$ è nella dorsale destra del sottoalbero sinistro di x (in $T)\}$

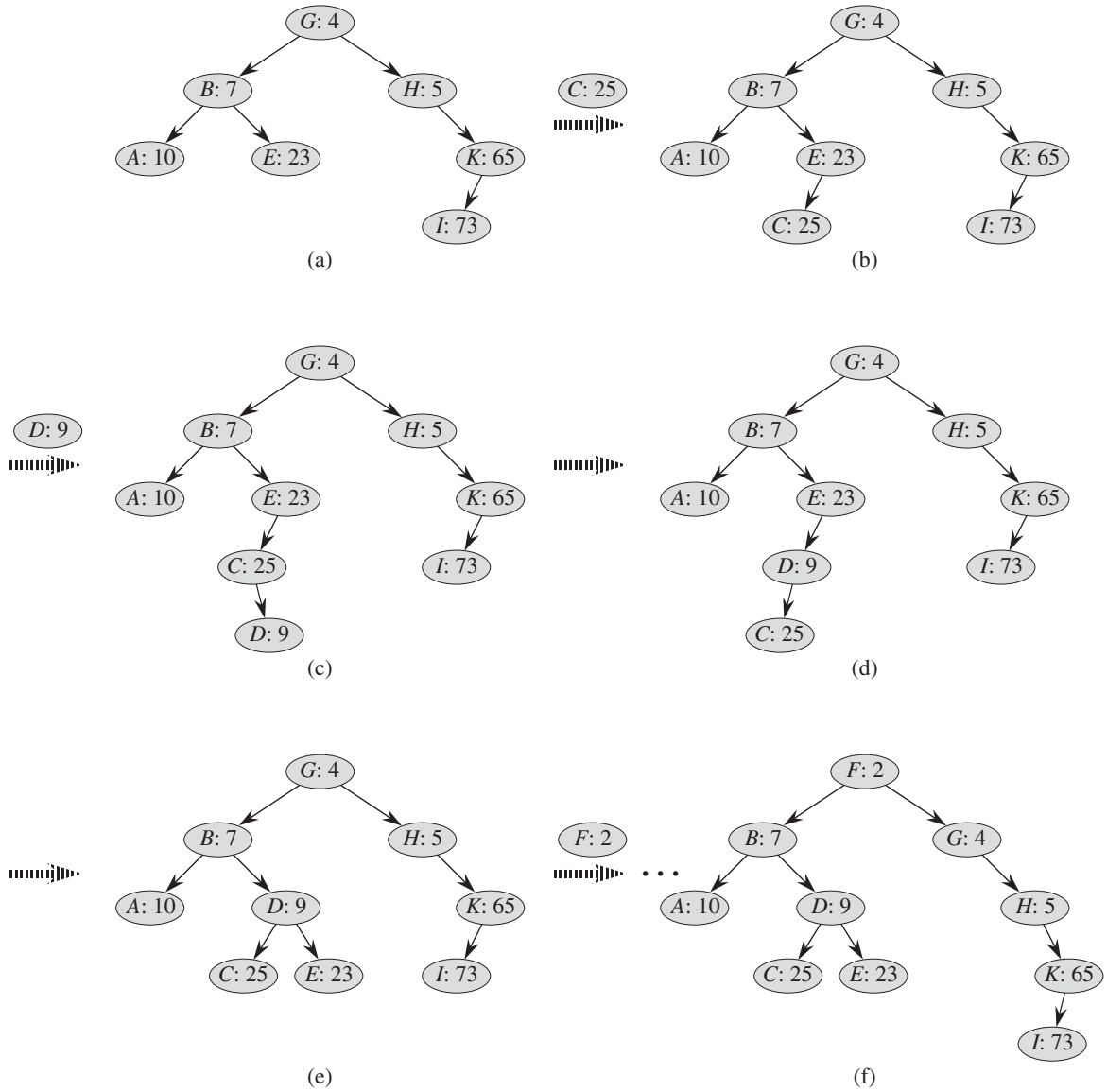


Figura 13.10 Il funzionamento di TREAP-INSERT. **(a)** Il treap originale, prima dell'inserimento del nodo. **(b)** Il treap dopo l'inserimento del nodo con chiave C e priorità 25. **(c)–(d)** Stadi intermedi durante l'inserimento del nodo con chiave D e priorità 9. **(e)** Il treap dopo l'inserimento illustrato nelle parti (c) e (d). **(f)** Il treap dopo l'inserimento del nodo con chiave F e priorità 2.

f. Dimostrate che $X_{i,k} = 1$, se e soltanto se $y.priority > x.priority$, $y.key < x.key$, e, per ogni z tale che $y.key < z.key < x.key$, si ha $y.priority < z.priority$.

g. Dimostrate che

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\Pr\{X_{i,k} = 1\} = \frac{(k-i-1)!}{(k-i+1)!} = \frac{1}{(k-i+1)(k-i)}$$

h. Dimostrate che

$$\mathrm{E}[C] = \sum_{j=1}^{k-1} \frac{1}{j(j+1)} = 1 - \frac{1}{k}$$

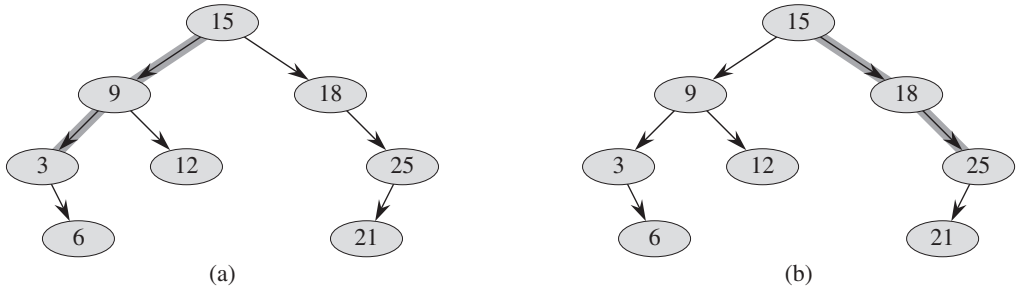


Figura 13.11 Le dorsali di un albero binario di ricerca. La dorsale sinistra è ombreggiata in **(a)**; la dorsale destra è ombreggiata in **(b)**.

i. Applicate la simmetria per dimostrare che

$$\mathrm{E}[D] = 1 - \frac{1}{n - k + 1}$$

j. Concludete che il numero atteso di rotazioni effettuate per l'inserimento di un nodo in un treap è minore di 2.

Note

L'idea di bilanciare un albero di ricerca è dovuta ad Adel'son-Vel'skiĭ e Landis [2], che nel 1962 hanno introdotto una classe di alberi di ricerca bilanciati detti "alberi AVL", che sono descritti nel Problema 13-3. Nel 1970 J. E. Hopcroft (in un manoscritto non pubblicato) introdusse un'altra classe di alberi di ricerca, detti "alberi 2-3". Il bilanciamento in un albero 2-3 è mantenuto manipolando i gradi dei nodi dell'albero. Il Capitolo 18 tratta i B-alberi, una generalizzazione degli alberi 2-3 sviluppata da Bayer e McCreight [35].

Gli alberi rosso-neri sono stati ideati da Bayer [34] con il nome di "B-alberi binari simmetrici". Guibas e Sedgewick [156] hanno studiato a lungo le loro proprietà e hanno introdotto la convenzione dei colori rosso e nero. Andersson [15] ha fornito una variante degli alberi rosso-neri più semplice da codificare. Weiss [352] ha chiamato questa variante albero AA. Questo tipo di albero è simile a un albero rosso-nero, con la differenza che i figli sinistri non possono essere rossi.

I treap sono stati ideati da Seidel e Aragon [310]. Sono l'implementazione standard di un dizionario in LEDA [254], che è una collezione bene implementata di strutture dati e algoritmi.

Esistono molte altre varianti di alberi binari bilanciati, come gli alberi bilanciati in peso [265], gli alberi k-neighbor [246] e gli alberi del capro espiatorio (scapegoat tree) [128]. Forse i più interessanti sono gli "alberi splay", introdotti da Sleator e Tarjan [321], che sono "auto-regolanti" (Tarjan [331] ha descritto molto bene questo tipi di alberi). Gli alberi splay mantengono il bilanciamento senza alcuna esplicita condizione di bilanciamento, come il colore. Piuttosto, le "operazioni splay" (che includono le rotazioni) sono svolte all'interno dell'albero ogni volta che si accede all'albero. Il costo ammortizzato (consultate il Capitolo 17) di ciascuna operazione in un albero di n nodi è $O(\lg n)$.

Acquistato da Michele Michele su Wednesday, 12 April 2023 12:23:12 | Numero Ordine Libreria: 2956390 | 2017 © Copyright © 2022 McGraw-Hill Education (Italy)

Le skip list [287] sono un'alternativa agli alberi binari bilanciati. Una skip list è una lista concatenata che viene ampliata con un certo numero di puntatori addizionali. Ogni operazione di dizionario viene eseguita nel tempo atteso $O(\lg n)$ in una skip list di n elementi.

14 Aumentare le strutture dati

Per alcuni problemi di ingegneria informatica è sufficiente una struttura dati elementare (fra quelle trattate in qualsiasi "libro di testo") – come una lista doppiamente concatenata, una tavola hash o un albero binario di ricerca – ma per molti altri problemi occorre un pizzico di creatività. Soltanto in rare situazioni avrete bisogno di creare un tipo di struttura dati completamente nuovo. Più frequentemente, sarà sufficiente aumentare una struttura dati elementare memorizzando in essa delle informazioni aggiuntive; poi, potrete programmare le nuove operazioni per la struttura dati per realizzare l'applicazione desiderata. Aumentare una struttura dati non è sempre semplice, in quanto le informazioni aggiuntive devono essere aggiornate e gestite dalle ordinarie operazioni sulla struttura dati.

Questo capitolo descrive due strutture dati che sono costruite aumentando gli alberi rosso-neri. Il Paragrafo 14.1 descrive un struttura dati che supporta le operazioni generali di statistica d'ordine su un insieme dinamico. Potremo così trovare rapidamente l'i-esimo numero più piccolo in un insieme o il rango di un elemento in un insieme ordinato di elementi. Il Paragrafo 14.2 generalizza il procedimento per aumentare una struttura dati e presenta un teorema che può semplificare l'aumento degli alberi rosso-neri. Il Paragrafo 14.3 applica questo teorema per agevolare la progettazione di una struttura dati che gestisce un insieme dinamico di intervalli, come gli intervalli temporali. Dato un intervallo di input, potremo rapidamente trovare un intervallo nell'insieme che si sovrappone ad esso.

14.1 Statistiche d'ordine dinamiche

Il Capitolo 9 ha introdotto il concetto di statistica d'ordine. Specificatamente, l'i-esima statistica d'ordine di un insieme di n elementi, con $i \in \{1, 2, \ldots, n\}$, è semplicemente l'elemento dell'insieme con l'i-esima chiave più piccola. Abbiamo visto che qualsiasi statistica d'ordine su di un insieme dinamico può essere ottenuta nel tempo $O(n)$ da un insieme non ordinato. In questo paragrafo, vedremo come modificare gli alberi rosso-neri in modo che qualsiasi statistica d'ordine su un insieme dinamico possa essere determinata nel tempo $O(\lg n)$. Vedremo che anche il ***rango*** di un elemento – la posizione che occupa nella sequenza ordinata degli elementi dell'insieme – può essere determinato nel tempo $O(\lg n)$.

Acquistato da Michele Michele su Webster il 2022-07-0[illegible] Figura Online Libraria 1265630[illegible] 220307 © Copyright © 2022 McGraw-Hill Education (Italy)

Nella Figura 14.1 è illustrata una struttura dati che è in grado di supportare operazioni rapide con le statistiche d'ordine. Un ***albero per statistiche d'ordine*** T è un albero rosso-nero con un'informazione aggiuntiva memorizzata in ogni nodo. In un nodo x di un albero rosso-nero, oltre agli attributi usuali $x.key$, $x.color$, $x.p$, $x.left$ e $x.right$, troviamo un altro attributo: $x.size$. Questo attributo contiene il numero di nodi (interni) nel sottoalbero con radice in x (incluso lo stesso x), cioè

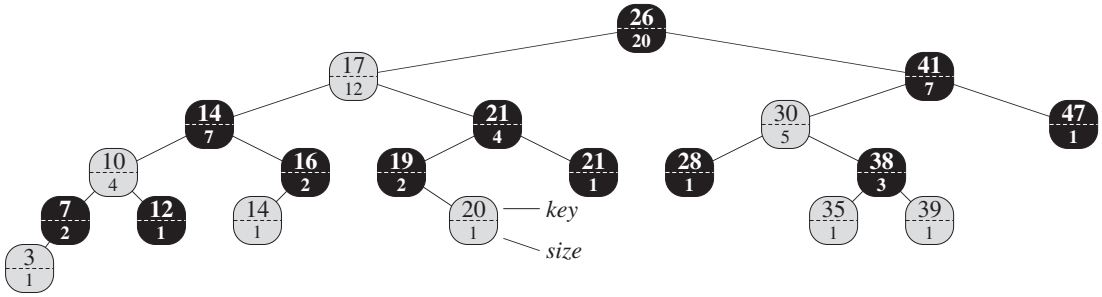


Figura 14.1 Un albero per statistiche d'ordine; è un albero rosso-nero aumentato. I nodi su sfondo grigio sono rossi; quelli su sfondo nero sono nodi neri. Oltre ai suoi attributi usuali, ogni nodo x ha un attributo $x.size$, che è il numero di nodi nel sottoalbero con radice in x.

la dimensione del sottoalbero. Se definiamo che la dimensione della sentinella è 0, ovvero impostiamo $T.nil.size$ a 0, allora abbiamo l'identità

$$x.size = x.left.size + x.right.size + 1$$

In un albero per statistiche d'ordine non è richiesto che le chiavi siano distinte (per esempio, l'albero nella Figura 14.1 ha due chiavi con valore 14 e due chiavi con valore 21). In presenza di chiavi uguali, la precedente nozione di rango non è ben definita. Eliminiamo questa ambiguità in un albero per statistiche d'ordine definendo il rango di un elemento come la posizione in cui l'elemento sarebbe elencato in un attraversamento simmetrico dell'albero. Per esempio, nella Figura 14.1 la chiave 14 memorizzata in un nodo nero ha rango 5 e la chiave 14 memorizzata in un nodo rosso ha rango 6.

Ricerca di un elemento con un dato rango

Prima di vedere come gestire le informazioni sulle dimensioni (*size*) durante l'inserimento e la cancellazione, esaminiamo l'implementazione di due operazioni di statistica d'ordine che usano questa informazione aggiuntiva. Iniziamo con un'operazione che trova un elemento con un dato rango. La procedura OS-SELECT(x, i) restituisce un puntatore al nodo che contiene l'i-esima chiave più piccola nel sottoalbero con radice in x. Per trovare il nodo con l'i-esima chiave più piccola in un albero per statistiche d'ordine T, chiamiamo OS-SELECT$(T.root, i)$.

```
OS-SELECT(x, i)
1  r = x.left.size + 1
2  if i == r
3        return x
4  elseif i < r
5        return OS-SELECT(x.left, i)
6  else return OS-SELECT(x.right, i − r)
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 - Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Nella riga 1 di OS-SELECT calcoliamo r, il rango del nodo x nel sottoalbero di radice x. Il valore di $x.left.size$ è il numero di nodi che precedono x in un attraversamento simmetrico del sottoalbero con radice in x. Quindi, $x.left.size+1$ è il rango di x all'interno del sottoalbero con radice in x. Se $i = r$, allora il nodo

x è l'i-esimo elemento più piccolo, quindi la riga 3 restituisce x. Se $i < r$, allora l'i-esimo elemento più piccolo è nel sottoalbero sinistro di x, quindi la riga 5 effettua una ricorsione su $x.left$. Se $i > r$, allora l'i-esimo elemento più piccolo è nel sottoalbero destro di x. Poiché ci sono r elementi nel sottoalbero con radice in x che precedono il sottoalbero destro di x in un attraversamento simmetrico, l'i-esimo elemento più piccolo nel sottoalbero con radice in x è l'$(i - r)$-esimo elemento più piccolo nel sottoalbero con radice in $x.right$. Questo elemento è determinato in modo ricorsivo nella riga 6.

Per vedere come opera OS-SELECT, consideriamo la ricerca del 17-esimo elemento più piccolo nell'albero per statistiche d'ordine della Figura 14.1. Iniziamo con x che punta alla radice, la cui chiave è 26, e con $i = 17$. Poiché la dimensione del sottoalbero sinistro di 26 è 12, il suo rango è 13. Quindi, sappiamo che il nodo con rango 17 è il quarto $(17 - 13 = 4)$ elemento più piccolo nel sottoalbero destro di 26. Dopo la chiamata ricorsiva, x è il nodo con chiave 41 e $i = 4$. Poiché la dimensione del sottoalbero sinistro di 41 è 5, il suo rango all'interno del suo sottoalbero è 6. Quindi, sappiamo che il nodo con rango 4 è il quarto elemento più piccolo nel sottoalbero sinistro di 41. Dopo la chiamata ricorsiva, x è il nodo con chiave 30 e il suo rango all'interno del suo sottoalbero è 2. Quindi, effettuiamo di nuovo la ricorsione per trovare il secondo $(4 - 2 = 2)$ elemento più piccolo nel sottoalbero con radice nel nodo con chiave 38. Adesso il suo sottoalbero sinistro ha dimensione 1; questo significa che esso è il secondo elemento più piccolo. Quindi, la procedura restituisce un puntatore al nodo con chiave 38.

Poiché per ogni chiamata ricorsiva si scende di un livello nell'albero per statistiche d'ordine, il tempo totale di OS-SELECT, nel caso peggiore, è proporzionale all'altezza dell'albero. Poiché l'albero è un albero rosso-nero, la sua altezza è $O(\lg n)$, dove n è il numero di nodi. Quindi, il tempo di esecuzione di OS-SELECT è $O(\lg n)$ per un insieme dinamico di n elementi.

Determinare il rango di un elemento

Dato un puntatore a un nodo x in un albero per statistiche d'ordine T, la procedura OS-RANK restituisce la posizione di x nell'ordinamento lineare determinato da un attraversamento simmetrico dell'albero T.

```
OS-RANK(T, x)
1  r = x.left.size + 1
2  y = x
3  while y ≠ T.root
4       if y == y.p.right
5            r = r + y.p.left.size + 1
6       y = y.p
7  return r
```

La procedura funziona nel modo seguente. Il rango di x può essere considerato come il numero di nodi che precedono x in un attraversamento simmetrico dell'albero, più 1 per x stesso. OS-RANK conserva la seguente invariante di ciclo:

> All'inizio di ogni iterazione del ciclo **while** (righe 3–6), r è il rango di $x.key$ nel sottoalbero con radice nel nodo y.

Utilizzeremo questa invariante di ciclo per dimostrare che OS-RANK opera correttamente:

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

Inizializzazione: prima della prima iterazione, la riga 1 assegna a r il rango di $x.key$ all'interno del sottoalbero con radice in x. L'assegnazione $y = x$ nella riga 2 rende l'invariante vera la prima volta che viene eseguito il test nella riga 3.

Conservazione: alla fine di ogni iterazione del ciclo **while**, poniamo $y = y.p$. Quindi dobbiamo dimostrare che, se r è il rango di $x.key$ nel sottoalbero con radice in y all'inizio del corpo del ciclo, allora r è il rango di $x.key$ nel sottoalbero con radice in $y.p$ alla fine del corpo del ciclo. In ogni iterazione del ciclo **while**, consideriamo il sottoalbero con radice in $y.p$. Abbiamo già contato il numero di nodi nel sottoalbero con radice nel nodo y che precedono x in un attraversamento simmetrico, quindi dobbiamo aggiungere i nodi nel sottoalbero con radice nel fratello di y che precedono x in un attraversamento simmetrico, più 1 per $y.p$, se anche questo nodo precede x. Se y è un figlio sinistro, allora né $y.p$ né altri nodi nel sottoalbero destro di $y.p$ precedono x, quindi lasciamo r invariato. Altrimenti, y è un figlio destro e tutti i nodi nel sottoalbero sinistro di $y.p$ precedono x, come pure lo stesso $y.p$. Quindi, nella riga 5, aggiungiamo $y.p.left.size + 1$ al valore corrente di r.

Conclusione: il ciclo termina quando $y = T.root$, sicché il sottoalbero con radice in y è l'intero albero. Dunque, il valore di r è il rango di $x.key$ nell'intero albero.

Per esempio, se eseguiamo OS-RANK nell'albero per statistiche d'ordine della Figura 14.1 per trovare il rango del nodo con chiave 38, otteniamo le seguente sequenza di valori per $y.key$ e r all'inizio del ciclo **while**:

iterazione	$y.key$	r
1	38	2
2	30	4
3	41	4
4	26	17

La procedura restituisce il rango 17.

Poiché ogni iterazione del ciclo **while** impiega il tempo $O(1)$ e y risale di un livello nell'albero a ogni iterazione, il tempo di esecuzione di OS-RANK, nel caso peggiore, è proporzionale all'altezza dell'albero: $O(\lg n)$ in un albero per statistiche d'ordine di n nodi.

Gestione delle dimensioni dei sottoalberi

Dato l'attributo *size* in ogni nodo, OS-SELECT e OS-RANK possono calcolare rapidamente le informazioni sulle statistiche d'ordine. Tuttavia, questo lavoro risulterebbe inutile se questi attributi non potessero essere gestiti con efficienza dalle operazioni di base che modificano gli alberi rosso-neri. Vediamo ora come gestire le dimensioni dei sottoalberi durante le operazioni di inserimento e cancellazione senza influire sul tempo di esecuzione asintotico di ciascuna operazione.

Acquistato da Michele Mirenda (mirenda.michele@gmail.com) il 22/10/2021 | Numero ordine: OP-ITA-0000010115827101 | Copyright © 2022, McGraw-Hill Education (Italy)

Come detto nel Paragrafo 13.3, l'inserimento in un albero rosso-nero si svolge in due fasi. Nella prima fase, si discende dalla radice dell'albero, inserendo il nuovo nodo come figlio di un nodo esistente. Nella seconda fase si risale verso la radice, cambiando i colori ed effettuando qualche rotazione per conservare le proprietà degli alberi rosso-neri.

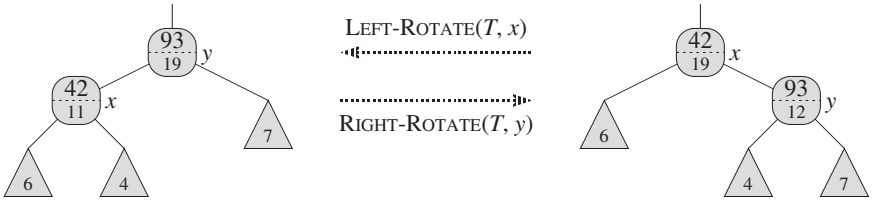


Figura 14.2
Aggiornamento delle dimensioni dei sottoalberi durante le rotazioni. I due nodi i cui attributi *size* devono essere aggiornati sono quelli uniti dal collegamento attorno ai quali si effettua la rotazione. Gli aggiornamenti sono locali, richiedendo soltanto le informazioni *size* memorizzate in x, y e nelle radici dei sottoalberi rappresentati da triangoli.

Per gestire le dimensioni dei sottoalberi nella prima fase, incrementiamo semplicemente $x.size$ per ogni nodo x nel cammino semplice attraversato dalla radice fino alle foglie. Il nuovo nodo che viene aggiunto ha l'attributo *size* pari a 1. Poiché ci sono $O(\lg n)$ nodi lungo il cammino percorso, il costo addizionale per gestire gli attributi *size* è $O(\lg n)$.

Nella seconda fase, le uniche modifiche strutturali dell'albero rosso-nero di base sono provocate dalle rotazioni, che sono al massimo due. Inoltre, una rotazione è un'operazione locale: soltanto due nodi hanno gli attributi *size* invalidati, i due nodi uniti dal collegamento attorno ai quali viene effettuata la rotazione. Facendo riferimento al codice della procedura LEFT-ROTATE(T, x) (Paragrafo 13.2), aggiungiamo le seguenti righe:

13 $y.size = x.size$
14 $x.size = x.left.size + x.right.size + 1$

L'aggiornamento degli attributi è illustrato nella Figura 14.2. La modifica di RIGHT-ROTATE è simmetrica.

Poiché vengono effettuate al massimo due rotazioni durante l'inserimento in un albero rosso-nero, occorre soltanto un tempo addizionale $O(1)$ per aggiornare gli attributi *size* nella seconda fase. Quindi, il tempo totale per completare l'inserimento in un albero per statistiche d'ordine di n nodi è $O(\lg n)$, che è asintoticamente uguale a quello di un normale albero rosso-nero.

Anche la cancellazione da un albero rosso-nero è formata da due fasi: la prima opera sull'albero di ricerca sottostante; la seconda provoca al massimo tre rotazioni, senza altre modifiche strutturali (vedere il Paragrafo 13.4). La prima fase o rimuove un nodo y oppure lo sposta più in alto nell'albero. Per aggiornare le dimensioni dei sottoalberi, seguiamo un cammino semplice dal nodo y (partendo dalla sua posizione originale nell'albero) fino alla radice, riducendo il valore dell'attributo *size* per ogni nodo che incontriamo. Poiché questo cammino semplice ha una lunghezza $O(\lg n)$ in un albero rosso-nero di n nodi, il tempo aggiuntivo che viene impiegato per gestire gli attributi *size* nella prima fase è $O(\lg n)$. Le $O(1)$ rotazioni nella seconda fase della cancellazione possono essere gestite come è stato fatto nell'inserimento. Quindi, le operazioni di inserimento e cancellazione, inclusa la gestione degli attributi *size*, richiedono un tempo $O(\lg n)$ su di un albero per statistiche d'ordine di n nodi.

Esercizi

14.1-1
Spiegate come opera OS-SELECT$(T.root, 10)$ sull'albero rosso-nero T della Figura 14.1.

14.1-2
Spiegate come opera OS-RANK(T, x) sull'albero rosso-nero T della Figura 14.1 se il nodo x ha $x.key = 35$.

14.1-3
Scrivete una versione non ricorsiva di OS-SELECT.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

14.1-4
Scrivete una procedura ricorsiva di OS-KEY-RANK(T, k) che riceve in input un albero per statistiche d'ordine T e una chiave k e restituisce il rango di k nell'insieme dinamico rappresentato da T. Supponete che le chiavi di T siano distinte.

14.1-5
Dato un elemento x in un albero per statistiche d'ordine di n nodi e un numero naturale i, come può essere determinato nel tempo $O(\lg n)$ l'i-esimo successore di x nell'ordinamento lineare dell'albero?

14.1-6
Notate che ogni volta che c'è un riferimento all'attributo *size* di un nodo nelle procedure OS-SELECT e OS-RANK, l'attributo è utilizzato soltanto per calcolare il rango del nodo nel sottoalbero con radice in quel nodo. Conformemente, supponete di registrare in ogni nodo il suo rango nel sottoalbero di cui esso è la radice. Spiegate come questa informazione può essere gestita durante l'inserimento e la cancellazione (ricordiamo che queste due operazioni possono provocare delle rotazioni).

14.1-7
Spiegate come utilizzare un albero per statistiche d'ordine per contare nel tempo $O(n \lg n)$ il numero di inversioni (vedere il Problema 2-4) in un array di dimensione n.

14.1-8 ⋆
Considerate n corde in un cerchio, ciascuna definita dai suoi estremi. Descrivete un algoritmo con tempo $O(n \lg n)$ per determinare il numero di coppie di corde che si intersecano all'interno del cerchio (per esempio, se le n corde sono diametri che si intersecano al centro del cerchio, allora la soluzione corretta è $\binom{n}{2}$). Supponete che due corde non possano avere un estremo in comune.

14.2 Come aumentare una struttura dati

Il procedimento per aumentare una struttura dati elementare in modo da supportare nuove funzionalità si usa spesso nella progettazione degli algoritmi. Questo procedimento sarà applicato di nuovo nel prossimo paragrafo per progettare una struttura dati che supporta le operazioni sugli intervalli. In questo paragrafo, esamineremo i passi che costituiscono tale procedimento. Dimostreremo anche un teorema che, in molti casi, permette di aumentare facilmente gli alberi rosso-neri.

Il procedimento per aumentare una struttura dati può essere suddiviso in quattro passi:

1. Scegliere una struttura dati di base.
2. Determinare le informazioni aggiuntive da gestire nella struttura dati di base.
3. Verificare che le informazioni aggiuntive possono essere gestite dalle operazioni elementari sulla struttura dati di base che la modificano.
4. Sviluppare nuove operazioni.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Come in tutti i metodi di progettazione, non occorre seguire alla cieca questi passi nell'ordine in cui sono elencati. Spesso la progettazione include una fase in cui si procede per tentativi e il progresso nei vari passi, di solito, avviene in parallelo. Per esempio, non ha senso determinare le informazioni aggiuntive o sviluppare le nuove operazioni (passi 2 e 4) se non siamo in grado di gestire con efficienza le informazioni aggiuntive. Ciononostante, questo metodo in quattro passi rappresenta

uno strumento valido per focalizzare i nostri sforzi sul processo di aumento di una struttura dati ed è anche un buon sistema per organizzare la documentazione di una struttura dati aumentata.

Abbiamo seguito questi passi nel Paragrafo 14.1 per progettare i nostri alberi per statistiche d'ordine. Per il passo 1, abbiamo scelto gli alberi rosso-neri come struttura dati di base. Un segnale sull'idoneità degli alberi rosso-neri proviene dal loro efficiente supporto ad altre operazioni sugli insiemi dinamici con ordinamento totale, come MINIMUM, MAXIMUM, SUCCESSOR e PREDECESSOR.

Per il passo 2, abbiamo aggiunto l'attributo *size*, dove ogni nodo x memorizza la dimensione del sottoalbero con radice in x. In generale, le informazioni aggiuntive rendono le operazioni più efficienti. Per esempio, avremmo potuto implementare le procedure OS-SELECT e OS-RANK utilizzando soltanto le chiavi memorizzate nell'albero, ma tali procedure non sarebbero state eseguite nel tempo $O(\lg n)$. A volte, le informazioni aggiuntive sono informazioni sui puntatori anziché sui dati, come nell'Esercizio 14.2-1.

Per il passo 3, abbiamo garantito che le operazioni di inserimento e cancellazione possano gestire correttamente gli attributi *size*, continuando a essere eseguite nel tempo $O(\lg n)$. La situazione ideale sarebbe che si debbano aggiornare soltanto pochi elementi della struttura dati per gestire le informazioni aggiuntive. Per esempio, se registrassimo semplicemente in ogni nodo il suo rango nell'albero, le procedure OS-SELECT e OS-RANK sarebbero eseguite rapidamente, ma l'inserimento di un nuovo elemento minimo modificherebbe questa informazione in tutti i nodi dell'albero. Se, invece, memorizziamo le dimensioni dei sottoalberi, l'inserimento di un nuovo elemento modifica le informazioni soltanto in $O(\lg n)$ nodi.

Per il passo 4, abbiamo sviluppato le operazioni OS-SELECT e OS-RANK. Dopo tutto, l'esigenza di svolgere nuove operazioni è il motivo principale per cui aumentiamo una struttura dati. A volte, anziché sviluppare nuove operazioni, utilizziamo le informazioni aggiuntive per accelerare quelle esistenti, come nell'Esercizio 14.2-1.

Aumentare gli alberi rosso-neri

Quando una struttura dati aumentata si basa sugli alberi rosso-neri, possiamo provare che certi tipi di informazioni aggiuntive possono essere gestiti in maniera efficiente nelle operazioni di inserimento e cancellazione, semplificando notevolmente il passo 3. La dimostrazione del seguente teorema è simile al ragionamento fatto nel Paragrafo 14.1 per spiegare che l'attributo *size* può essere correttamente gestito negli alberi per statistiche d'ordine.

Teorema 14.1 (Aumento di un albero rosso-nero)
Sia f un attributo che aumenta un albero rosso-nero T di n nodi; supponiamo che il valore di f per un nodo x possa essere calcolato utilizzando soltanto le informazioni nei nodi x, $x.left$ e $x.right$, inclusi $x.left.f$ e $x.right.f$. Allora, è possibile gestire i valori di f in tutti i nodi di T durante l'inserimento e la cancellazione, senza influire asintoticamente sulla prestazione $O(\lg n)$ di queste operazioni.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022. McGraw-Hill Education (Italy)

Dimostrazione Il concetto che sta alla base della dimostrazione è che la modifica di un attributo f in un nodo x si propaga soltanto agli antenati di x nell'albero. In altre parole, la modifica di $x.f$ potrebbe richiedere l'aggiornamento di $x.p.f$, ma niente altro; l'aggiornamento di $x.p.f$ potrebbe richiedere l'aggiornamento di

$x.p.p.f$, ma niente altro; e così via risalendo l'albero. Quando viene aggiornato $T.root.f$, nessun altro nodo dipende da questo nuovo valore, quindi il processo termina. Poiché l'altezza di un albero rosso-nero è $O(\lg n)$, la modifica di un attributo f in un nodo richiede un tempo $O(\lg n)$ per aggiornare i nodi che dipendono da tale modifica.

L'inserimento di un nodo x nell'albero T si compone di due fasi (vedere il Paragrafo 13.3). Durante la prima fase, x viene inserito come figlio di un nodo esistente $x.p$. Il valore di $x.f$ può essere calcolato nel tempo $O(1)$ perché, per ipotesi, dipende soltanto dalle informazioni negli altri attributi dello stesso x e dalle informazioni dei figli di x, ma i figli di x sono entrambi la sentinella $T.nil$. Una volta calcolato $x.f$, la modifica si propaga verso l'alto nell'albero. Quindi, il tempo totale per la prima fase dell'inserimento è $O(\lg n)$. Durante la seconda fase, le uniche modifiche strutturali dell'albero derivano dalle rotazioni. Poiché in una rotazione cambiano soltanto due nodi, il tempo totale per aggiornare gli attributi f è $O(\lg n)$ per rotazione. Dal momento che in un inserimento ci sono al massimo due rotazioni, il tempo totale per l'inserimento è $O(\lg n)$.

Come l'inserimento, anche la cancellazione si svolge in due fasi (vedere il Paragrafo 13.4). Nella prima fase, le modifiche dell'albero si verificano quando il nodo cancellato viene effettivamente rimosso dall'albero. Se il nodo cancellato aveva due figli il suo successore viene messo al suo posto e quindi il nodo effettivamente rimosso è il successore. La propagazione degli aggiornamenti di f indotti da queste modifiche costa al massimo $O(\lg n)$, in quanto le modifiche cambiano localmente l'albero. La sistemazione dell'albero rosso-nero durante la seconda fase richiede al massimo tre rotazioni, ciascuna delle quali impiega al massimo il tempo $O(\lg n)$ per propagare gli aggiornamenti di f. Quindi, come per l'inserimento, il tempo totale per la cancellazione è $O(\lg n)$. ■

In molti casi, come la gestione degli attributi *size* negli alberi per statistiche d'ordine, il costo di aggiornamento dopo una rotazione è $O(1)$, anziché $O(\lg n)$ che abbiamo ottenuto nella dimostrazione del Teorema 14.1 (un esempio è descritto nell'Esercizio 14.2-3).

Esercizi

14.2-1

Spiegate come, aggiungendo dei puntatori ai nodi, ciascuna delle operazioni MINIMUM, MAXIMUM, SUCCESSOR e PREDECESSOR possa essere eseguita nel tempo $O(1)$ nel caso peggiore in un albero per statistiche d'ordine aumentato, senza influenzare le prestazioni asintotiche delle altre operazioni.

14.2-2

È possibile gestire le altezze nere dei nodi in un albero rosso-nero come attributi dei nodi dell'albero senza influire sulle prestazioni asintotiche delle altre operazioni con l'albero rosso-nero? Spiegate come o perché no. E riguardo alla possibilità di gestire le profondità dei nodi?

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

14.2-3 ★

Indichiamo con $\otimes$ un operatore binario associativo; sia a un attributo aggiuntivo in ciascun nodo di un albero rosso-nero. Supponete di volere includere in ogni nodo x un attributo addizionale f tale che $x.f = x_1.a \otimes x_2.a \otimes \cdots \otimes x_m.a$, dove $x_1, x_2, \ldots, x_m$ è l'elenco dei nodi di un attraversamento simmetrico del sottoalbero con radice in x. Dimostrate che gli attributi f possono essere appropriatamente aggiornati nel tempo $O(1)$ dopo una rotazione. Modificate un po' il vostro ragionamento per applicarlo agli attributi *size* negli alberi per statistiche d'ordine.

14.2-4 ⋆

Vogliamo aumentare gli alberi rosso-neri con un'operazione RB-ENUMERATE(x, a, b) che genera in output tutte le chiavi k tali che $a \leq k \leq b$ in un albero rosso-nero con radice in x. Spiegate come implementare RB-ENUMERATE nel tempo $\Theta(m + \lg n)$, dove m è il numero di chiavi in output e n è il numero di nodi interni dell'albero (*suggerimento:* non occorre aggiungere nuovi attributi all'albero rosso-nero).

14.3 Alberi di intervalli

In questo paragrafo estenderemo gli alberi rosso-neri per supportare le operazioni con gli insiemi dinamici di intervalli. Un ***intervallo chiuso*** è una coppia ordinata di numeri reali $[t_1, t_2]$, con $t_1 \leq t_2$. L'intervallo $[t_1, t_2]$ rappresenta l'insieme $\{t \in \mathbb{R} : t_1 \leq t \leq t_2\}$. Gli ***intervalli semiaperti*** e ***aperti*** omettono, rispettivamente, uno o entrambi gli estremi dell'insieme. In questo paragrafo, supporremo che gli intervalli siano chiusi; estendere i risultati agli intervalli aperti e semiaperti è concettualmente semplice.

Gli intervalli sono comodi per rappresentare gli eventi che si svolgono in un periodo continuo di tempo. Per esempio, potrebbe essere necessario interrogare una base di dati di intervalli temporali per trovare quali eventi si sono verificati durante un determinato intervallo di tempo. La struttura dati di questo paragrafo fornisce uno strumento efficiente per gestire tali basi di dati di intervalli.

Possiamo rappresentare un intervallo $[t_1, t_2]$ come un oggetto i, con gli attributi $i.low = t_1$ (***estremo inferiore***) e $i.high = t_2$ (***estremo superiore***). Diciamo che gli intervalli i e i' ***si sovrappongono*** se $i \cap i' \neq \emptyset$, ovvero se $i.low \leq i'.high$ e $i'.low \leq i.high$. Come mostra la Figura 14.3, due intervalli qualsiasi i e i' soddisfano la ***tricotomia degli intervalli***; ovvero una sola delle seguenti proprietà può essere vera:

a. i e i' si sovrappongono.

b. i è a sinistra di i' (cioè $i.high < i'.low$).

c. i è a destra di i' (cioè $i'.high < i.low$).

Un ***albero di intervalli*** è un albero rosso-nero che gestisce un insieme dinamico di elementi, in cui ogni elemento x contiene un intervallo $x.int$. Gli alberi di intervalli supportano le seguenti operazioni.

INTERVAL-INSERT(T, x) aggiunge l'elemento x, il cui attributo *int* si suppone contenga un intervallo, all'albero di intervalli T.

INTERVAL-DELETE(T, x) rimuove l'elemento x dall'albero di intervalli T.

INTERVAL-SEARCH(T, i) restituisce un puntatore a un elemento x nell'albero di intervalli T tale che $x.int$ si sovrappone all'intervallo i; restituisce un puntatore alla sentinella $T.nil$ se non esiste tale elemento nell'insieme.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-2297070 Copyright © 2022, McGraw-Hill Education (Italy)

La Figura 14.4 illustra come un albero di intervalli rappresenta un insieme di intervalli. Seguiremo i quattro passi del metodo descritto nel Paragrafo 14.2 per illustrare il progetto di un albero di intervalli e le operazioni che vengono svolte su di esso.

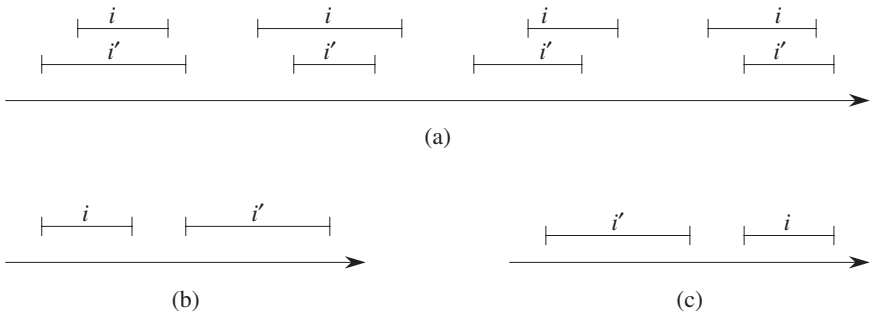


Figura 14.3 Tricotomia degli intervalli per due intervalli chiusi i e i'. **(a)** Se i e i' si sovrappongono, si hanno quattro casi; in tutti e quattro, $i.low \leq i'.high$ e $i'.low \leq i.high$. **(b)** Gli intervalli non si sovrappongono e $i.high < i'.low$. **(c)** Gli intervalli non si sovrappongono e $i'.high < i.low$.

Passo 1: struttura dati di base

Scegliamo un albero rosso-nero in cui ogni nodo x contiene un intervallo $x.int$ e la chiave di x è l'estremo inferiore, $x.int.low$, dell'intervallo. Quindi, un attraversamento simmetrico della struttura dati elenca ordinatamente gli intervalli in funzione dell'estremo inferiore.

Passo 2: informazioni aggiuntive

Oltre agli intervalli, ogni nodo x contiene un valore $x.max$, che è il massimo tra tutti gli estremi degli intervalli memorizzati nel sottoalbero con radice in x.

Passo 3: gestione delle informazioni

Dobbiamo verificare che le operazioni di inserimento e cancellazione possono essere svolte nel tempo $O(\lg n)$ in un albero di intervalli di n nodi. Se conosciamo l'intervallo $x.int$ e i valori max dei figli del nodo x, possiamo determinare $x.max$:

$$x.max = \max(x.int.high, x.left.max, x.right.max)$$

Per il Teorema 14.1, le operazioni di inserimento e cancellazione vengono eseguite nel tempo $O(\lg n)$. In realtà, l'aggiornamento degli attributi max dopo una rotazione può essere eseguito nel tempo $O(1)$, come dimostrano gli Esercizi 14.2-3 e 14.3-1.

Passo 4: sviluppare le nuove operazioni

L'unica operazione nuova da implementare è INTERVAL-SEARCH(T, i), che trova un nodo nell'albero T il cui intervallo si sovrappone all'intervallo i. Se non c'è un intervallo che si sovrappone a i nell'albero, viene restituito un puntatore alla sentinella $T.nil$.

```
INTERVAL-SEARCH(T, i)
1  x = T.root
2  while x ≠ T.nil e i non si sovrappone a x.int
3      if x.left ≠ T.nil e x.left.max ≥ i.low
4          x = x.left
5      else x = x.right
6  return x
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

La ricerca di un intervallo che si sovrappone a i inizia con x nella radice dell'albero e prosegue verso il basso. Termina quando viene trovato un intervallo che si sovrappone a i o quando x punta alla sentinella $T.nil$. Poiché ogni iterazione del ciclo di base impiega il tempo $O(1)$ e poiché l'altezza di un albero rosso-nero di n nodi è $O(\lg n)$, la procedura INTERVAL-SEARCH impiega il tempo $O(\lg n)$.

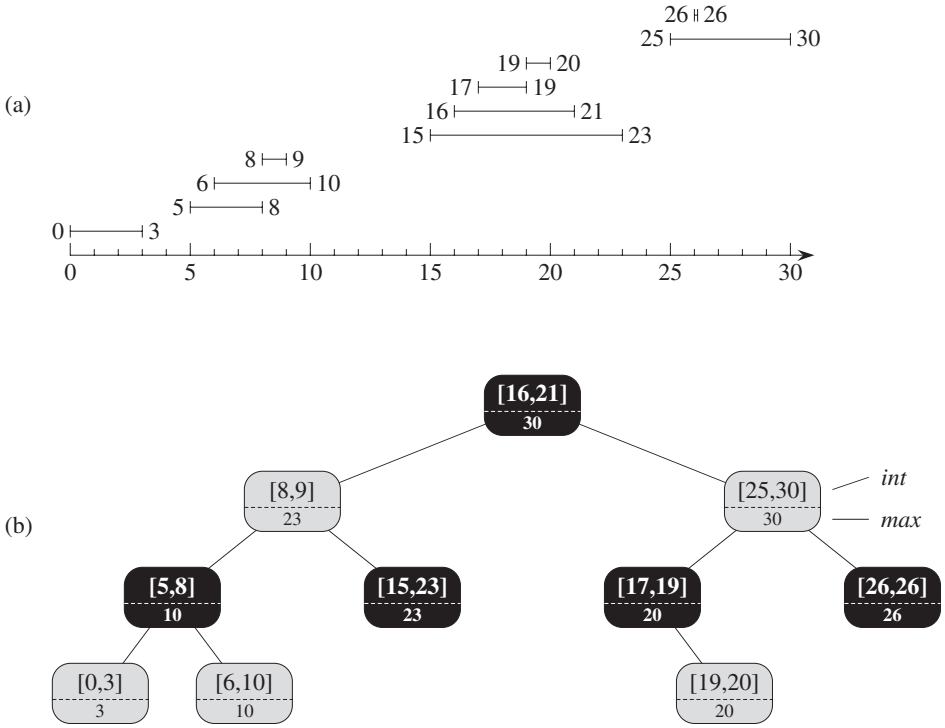


Figura 14.4 Un albero di intervalli. **(a)** Un insieme di 10 intervalli, ordinati dal basso verso l'alto in funzione dell'estremo sinistro degli intervalli. **(b)** L'albero di intervalli che li rappresenta. Ogni nodo x contiene un intervallo (sopra la linea tratteggiata) e il massimo valore degli estremi degli intervalli nel sottoalbero di radice x (sotto la linea tratteggiata). Un attraversamento simmetrico dell'albero elenca ordinatamente i nodi in funzione dell'estremo sinistro.

Prima di spiegare perché l'operazione INTERVAL-SEARCH è corretta, analizziamo il suo funzionamento con l'albero di intervalli illustrato nella Figura 14.4. Supponiamo di volere trovare un intervallo che si sovrappone all'intervallo $i = [22, 25]$. Iniziamo con x nella radice, che contiene $[16, 21]$ e non si sovrappone a i. Poiché $x.left.max = 23$ è maggiore di $i.low = 22$, il ciclo continua con x come figlio sinistro della radice – il nodo che contiene $[8, 9]$; anche questo intervallo non si sovrappone a i. Stavolta $x.left.max = 10$ è minore di $i.low = 22$, quindi il ciclo continua con il figlio destro di x come il nuovo x. L'intervallo $[15, 23]$ memorizzato in questo nodo si sovrappone a i, quindi la procedura restituisce questo nodo.

Come esempio di ricerca senza successo, supponiamo di cercare un intervallo che si sovrappone a $i = [11, 14]$ nell'albero di intervalli illustrato nella Figura 14.4. Iniziamo ancora una volta con x come radice. Poiché l'intervallo $[16, 21]$ della radice non si sovrappone a i e poiché $x.left.max = 23$ è maggiore di $i.low = 11$, andiamo a sinistra nel nodo che contiene $[8, 9]$. L'intervallo $[8, 9]$ non si sovrappone a i e $x.left.max = 10$ è minore di $i.low = 11$, quindi andiamo a destra (notate che nessun intervallo nel sottoalbero sinistro si sovrappone a i). L'intervallo $[15, 23]$ non si sovrappone a i e suo figlio sinistro è $T.nil$, quindi andiamo a destra, il ciclo termina e viene restituita la sentinella $T.nil$.

Acquistato da Michele Michele su Webster il 2022-07-0[illegible]

Per spiegare perché la procedura INTERVAL-SEARCH è corretta, bisogna capire perché è sufficiente esaminare un solo percorso dalla radice. Il concetto di base è che in qualsiasi nodo x, se $x.int$ non si sovrappone a i, la ricerca procede sempre in una direzione sicura: alla fine sarà trovato un intervallo che si sovrappone a quello dato, se ce n'è uno nell'albero. Il seguente teorema definisce in maniera più precisa questa proprietà.

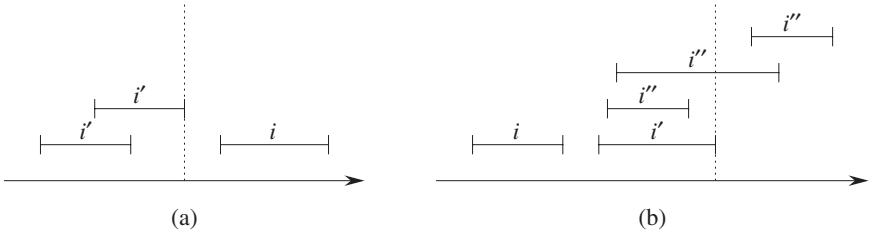


Figura 14.5 Gli intervalli nella dimostrazione del Teorema 14.2. Il valore di $x.left.max$ è illustrato in entrambi i casi come una linea tratteggiata. **(a)** La ricerca va a destra. Nessun intervallo i' nel sottoalbero sinistro di x può sovrapporsi a i. **(b)** La ricerca va a sinistra. Il sottoalbero sinistro di x contiene un intervallo che si sovrappone a i (caso non illustrato) oppure c'è un intervallo i' nel sottoalbero sinistro di x tale che $i'.high = x.left.max$. Poiché i non si sovrappone a i', non può sovrapporsi a un intervallo i'' nel sottoalbero destro di x, perché $i'.low \leq i''.low$.

Teorema 14.2

La procedura INTERVAL-SEARCH(T, i) restituisce un nodo il cui intervallo si sovrappone a i oppure restituisce $T.nil$ se l'albero T non contiene alcun nodo il cui intervallo si sovrappone a i.

Dimostrazione Il ciclo **while** (righe 2–5) termina quando $x = T.nil$ o quando i si sovrappone a $x.int$. Nel secondo caso, è certamente corretto restituire x. Quindi, concentriamo la nostra attenzione sul primo caso, in cui il ciclo **while** termina perché $x = T.nil$.

Utilizziamo la seguente invariante per il ciclo **while** (righe 2–5):

> Se l'albero T contiene un intervallo che si sovrappone a i, allora esiste un tale intervallo nel sottoalbero con radice in x.

Applichiamo questa invariante di ciclo nel modo seguente:

Inizializzazione: prima della prima iterazione, la riga 1 imposta x come radice di T, quindi l'invariante è vera.

Conservazione: in ogni iterazione del ciclo **while**, viene eseguita la riga 4 o la riga 5. Dimostreremo che l'invariante di ciclo si conserva nei due casi.

Se viene eseguita la riga 5, allora per la condizione di diramazione nella riga 3, abbiamo $x.left = T.nil$ o $x.left.max < i.low$. Se $x.left = T.nil$, il sottoalbero con radice in $x.left$ chiaramente non contiene un intervallo che si sovrappone a i; quindi, impostando x a $x.right$, si conserva l'invariante. Supponete, allora, che $x.left \neq T.nil$ e $x.left.max < i.low$. Come illustra la Figura 14.5(a), per ogni intervallo i' nel sottoalbero sinistro di x, abbiamo

$$\begin{aligned} i'.high &\leq x.left.max \\ &< i.low \end{aligned}$$

Per la tricotomia degli intervalli, i' e i non si sovrappongono. Allora il sottoalbero sinistro di x non contiene intervalli che si sovrappongono a i; quindi, impostando x a $x.right$, si conserva l'invariante.

Se, invece, viene eseguita la riga 4, allora dimostreremo che è vera la versione contrapposta dell'invariante di ciclo. Ovvero che se non c'è un intervallo che si sovrappone a i nel sottoalbero con radice in $x.left$, allora non c'è un intervallo che si sovrappone a i in qualsiasi punto dell'albero. Poiché viene eseguita la riga 4, allora per la condizione di diramazione nella riga 3, abbiamo $x.left.max \geq i.low$. Inoltre, per la definizione dell'attributo max, ci deve essere un intervallo i' nel sottoalbero sinistro di x tale che

$$\begin{aligned} i'.high &= x.left.max \\ &\geq i.low \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-230707-91.Copyright © 2022, McGraw-Hill Education (Italy)

La Figura 14.5(b) illustra la situazione. Poiché i e i' non si sovrappongono e poiché non è vero che $i'.high < i.low$, allora per la tricotomia degli intervalli si ha $i.high < i'.low$. Gli alberi di intervalli usano come chiavi gli estremi inferiori degli intervalli, quindi la proprietà dell'albero di ricerca implica che, per qualsiasi intervallo i'' nel sottoalbero destro di x, si abbia

$$\begin{aligned} i.high &< i'.low \\ &\leq i''.low \end{aligned}$$

Per la tricotomia degli intervalli, i e i'' non si sovrappongono. Concludiamo che, indipendentemente dal fatto che un intervallo nel sottoalbero sinistro di x si sovrapponga oppure no a i, impostando x a $x.left$, si conserva l'invariante.

Conclusione: se il ciclo termina quando $x = T.nil$, non c'è un intervallo che si sovrappone a i nel sottoalbero con radice in x. La versione contrapposta dell'invariante di ciclo implica che T non contiene un intervallo che si sovrappone a i. Quindi è corretto restituire $x = T.nil$. ■

In conclusione, la procedura INTERVAL-SEARCH funziona correttamente.

Esercizi

14.3-1

Scrivete lo pseudocodice per LEFT-ROTATE che opera sui nodi di un albero di intervalli e aggiorna gli attributi *max* nel tempo $O(1)$.

14.3-2

Riscrivete il codice per INTERVAL-SEARCH in modo che funzioni correttamente nell'ipotesi che tutti gli intervalli siano aperti.

14.3-3

Descrivete un algoritmo efficiente che, dato un intervallo i, restituisce un intervallo che si sovrappone a i e che ha l'estremo inferiore minimo, oppure restituisce $T.nil$, se tale intervallo non esiste.

14.3-4

Dato un albero di intervalli T e un intervallo i, spiegate come tutti gli intervalli in T che si sovrappongono a i possono essere elencati nel tempo $O(\min(n, k \lg n))$, dove k è il numero di intervalli nella lista di output (*suggerimento:* un metodo semplice effettua successive interrogazioni, modificando l'albero tra una interrogazione e l'altra. Un metodo un po' più complicato non modifica l'albero).

14.3-5

Descrivete le modifiche da apportare alle procedure degli alberi di intervalli per supportare la nuova operazione INTERVAL-SEARCH-EXACTLY(T, i) che restituisce un puntatore a un nodo x in un albero di intervalli T tale che $x.int.low = i.low$ e $x.int.high = i.high$, oppure $T.nil$ se T non contiene tale nodo. Tutte le operazioni, inclusa INTERVAL-SEARCH-EXACTLY, dovrebbero essere eseguite nel tempo $O(\lg n)$ in un albero di intervalli di n nodi.

Acquistato da Michele Michele su Webster il 2022-07-07 23:11 Numero Ordine Libreria: 1995309010-2267070-0 Copyright © 2022, McGraw-Hill Education (Italy)

14.3-6

Spiegate come gestire un insieme dinamico Q di numeri che supporta l'operazione MIN-GAP; questa operazione fornisce il valore della differenza tra i due numeri più vicini in Q. Per esempio, se $Q = \{1, 5, 9, 15, 18, 22\}$, allora MIN-GAP$(Q)$ restituisce $18 - 15 = 3$, in quanto 15 e 18 sono i due numeri più vicini in Q. Le operazioni INSERT, DELETE, SEARCH e MIN-GAP devono essere le più efficienti possibili; analizzate i loro tempi di esecuzione.

14.3-7 ⋆

Tipicamente, la basi di dati di VLSI rappresentano un circuito integrato come una lista di rettangoli. Supponete che ogni rettangolo abbia i lati paralleli agli assi x e y, in modo che possa essere rappresentato dalle coordinate x e y minime e massime. Create un algoritmo con tempo $O(n \lg n)$ per determinare se un insieme di rettangoli così rappresentati contenga due rettangoli che si sovrappongono. Il vostro algoritmo non deve elencare tutte le coppie di rettangoli che si intersecano, ma deve indicare che esiste una sovrapposizione anche quando un rettangolo si sovrappone interamente a un altro e le linee di contorno non si intersecano (*suggerimento:* fate scorrere una retta di "scansione" attraverso l'insieme dei rettangoli).

Problemi

14-1 Punto di massima sovrapposizione

Supponete di voler trovare un ***punto di massima sovrapposizione*** in un insieme di intervalli – un punto dove si sovrappone il maggior numero di intervalli del database.

a. Dimostrate che c'è sempre un punto di massima sovrapposizione che è un estremo di uno dei segmenti.

b. Progettate una struttura dati che supporta efficientemente le operazioni INTERVAL-INSERT, INTERVAL-DELETE e FIND-POM; quest'ultima operazione restituisce un punto di massima sovrapposizione. (*Suggerimento:* create un albero rosso-nero con tutti gli estremi. Associate il valore $+1$ a ogni estremo sinistro e il valore -1 a ogni estremo destro. Aumentate i nodi dell'albero con un'informazione aggiuntiva per gestire il punto di massima sovrapposizione.)

14-2 La permutazione di Josephus

Il ***problema di Josephus*** è definito nel modo seguente. Supponete che n persone siano disposte in cerchio; sia dato un numero intero positivo $m \leq n$. Iniziando da una persona, si procede intorno al cerchio allontanando ogni m-esima persona. Dopo avere allontanato una persona, il conteggio prosegue con le persone rimaste. Il processo continua finché non saranno allontanate tutte le n persone. L'ordine in cui le persone vengono allontanate dal cerchio definisce la ***(n, m)-permutazione di Josephus*** degli interi $1, 2, \ldots, n$. Per esempio, la $(7, 3)$-permutazione di Josephus è $\langle 3, 6, 2, 7, 5, 1, 4 \rangle$.

a. Supponete che m sia una costante. Descrivete un algoritmo con tempo $O(n)$ che, dato un intero n, generi come output la (n, m)-permutazione di Josephus.

b. Supponete che m non sia una costante. Descrivete un algoritmo con tempo $O(n \lg n)$ che, dati gli interi n e m, generi in output la (n, m)-permutazione di Josephus.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Note

Preparata e Shamos [283] descrivono diversi alberi di intervalli che si trovano nella letteratura, facendo riferimento agli studi di H. Edelsbrunner (1980) ed E. M. McCreight (1981). Il libro esamina dettagliatamente un albero di intervalli per il quale, data una base di dati statica di n intervalli, si possono enumerare tutti i k intervalli che si sovrappongono a un dato intervallo nel tempo $O(k + \lg n)$.

IV Tecniche avanzate di progettazione e di analisi

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Introduzione

Questa parte tratta tre importanti tecniche per la progettazione e l'analisi di algoritmi efficienti: la programmazione dinamica (Capitolo 15), gli algoritmi golosi (Capitolo 16) e l'analisi ammortizzata (Capitolo 17). Nelle parti precedenti di questo libro abbiamo presentato alcune tecniche molto comuni, come il metodo divide et impera, la randomizzazione e la soluzione delle ricorrenze. Le nuove tecniche presentate in questa parte sono più sofisticate, ma utili per affrontare e risolvere in modo efficiente molti problemi computazionali. Gli argomenti trattati in questa parte saranno utilizzati nelle parti successive del libro.

Tipicamente, la programmazione dinamica viene applicata per risolvere problemi di ottimizzazione, nei quali si arriva a una soluzione ottima effettuando una serie di scelte. Nell'eseguire tale serie di scelte, talvolta si ripresentano gli stessi sottoproblemi da risolvere. La programmazione dinamica è efficace quando lo stesso sottoproblema può comparire più volte; il concetto chiave della programmazione dinamica consiste nel memorizzare le soluzioni dei sottoproblemi per poterle usare nel caso in cui si dovesse ripresentare uno di questi sottoproblemi. Il Capitolo 15 spiega come questo semplice concetto, a volte, consente di trasformare algoritmi con tempi esponenziali in algoritmi con tempi polinomiali.

Analogamente agli algoritmi della programmazione dinamica, gli algoritmi golosi tipicamente sono utilizzati per risolvere problemi di ottimizzazione nei quali occorre fare una serie di scelte per arrivare a una soluzione ottima. L'idea che sta alla base di un algoritmo goloso è quella di effettuare le singole scelte in modo che siano localmente ottime. Un semplice esempio è il problema del resto in monete: per minimizzare il numero di monete necessarie per formare un determinato resto, è sufficiente selezionare ripetutamente la moneta di taglio più grande che non supera l'importo ancora dovuto. Ci sono vari problemi come questo per i quali un algoritmo goloso fornisce una soluzione ottima più rapidamente di quanto sarebbe possibile con un metodo di programmazione dinamica. Tuttavia, non sempre è facile dire se una tecnica golosa sarà applicabile. Il Capitolo 16 introduce la teoria dei matroidi, che può essere utile in molti casi per valutare l'applicabilità della tecnica golosa.

Acquistato da Michele Michele su Webster il 2022-07-07 23:13. Numero Ordine Libreria 1995020916-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

L'analisi ammortizzata è uno strumento per analizzare gli algoritmi che svolgono una sequenza di operazioni simili. Anziché limitare il costo della sequenza delle operazioni limitando il costo di ciascuna operazione separatamente, l'analisi

ammortizzata può essere utilizzata per fornire un limite del costo effettivo dell'intera sequenza di operazioni. Un motivo per cui questa idea può risultare efficace è che è poco probabile che, in una sequenza di operazioni, ogni singola operazione venga eseguita nel tempo del suo caso peggiore. Alcune operazioni potranno essere costose, ma molte altre saranno più economiche. L'analisi ammortizzata, tuttavia, non è un semplice strumento di analisi; essa permette anche di ragionare sul progetto degli algoritmi, in quanto la progettazione di un algoritmo e l'analisi del suo tempo di esecuzione sono strettamente interconnesse. Il Capitolo 17 introduce tre metodi per svolgere l'analisi ammortizzata degli algoritmi.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Programmazione dinamica 15

La programmazione dinamica, come il metodo divide et impera, risolve i problemi combinando le soluzioni dei sottoproblemi (in questo contesto, con il termine "programmazione" facciamo riferimento a un metodo tabulare, non alla scrittura del codice per un calcolatore). Come detto nei Capitoli 2 e 4, gli algoritmi divide et impera suddividono un problema in sottoproblemi indipendenti, risolvono in modo ricorsivo i sottoproblemi e, poi, combinano le loro soluzioni per risolvere il problema originale. La programmazione dinamica, invece, può essere applicata quando i sottoproblemi non sono indipendenti, ovvero quando i sottoproblemi hanno in comune dei sottosottoproblemi. In questo contesto, un algoritmo divide et impera svolge molto più lavoro del necessario, risolvendo ripetutamente i sottosottoproblemi comuni. Un algoritmo di programmazione dinamica risolve ciascun sottosottoproblema una sola volta e salva la sua soluzione in una tabella, evitando così il lavoro di ricalcolare la soluzione ogni volta che si presenta il sottosottoproblema.

La programmazione dinamica, tipicamente, si applica ai ***problemi di ottimizzazione***. Per questi problemi ci possono essere molte soluzioni possibili. Ogni soluzione ha un valore e si vuole trovare una soluzione con il valore ottimo (minimo o massimo). Precisiamo che abbiamo detto *una* soluzione ottima del problema, non *la* soluzione ottima, perché ci possono essere più soluzioni che raggiungono il valore ottimo.

Il processo di sviluppo di un algoritmo di programmazione dinamica può essere suddiviso in una sequenza di quattro fasi.

1. Caratterizzare la struttura di una soluzione ottima.
2. Definire in modo ricorsivo il valore di una soluzione ottima.
3. Calcolare il valore di una soluzione ottima, di solito con uno schema bottom-up (dal basso verso l'alto).
4. Costruire una soluzione ottima dalle informazioni calcolate.

Le fasi 1–3 formano la base per risolvere un problema applicando la programmazione dinamica. La fase 4 può essere omessa se è richiesto soltanto il valore di una soluzione ottima. Quando dobbiamo eseguire la fase 4, a volte memorizziamo delle informazioni aggiuntive durante il calcolo della fase 3 per semplificare la costruzione di una soluzione ottima.

Acquistato da Michele Michelangeli (Webstore) il 2022-07-17 23:13 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

I prossimi paragrafi applicano il metodo della programmazione dinamica per risolvere alcuni problemi di ottimizzazione. Il Paragrafo 15.1 esamina il problema di tagliare un'asta in parti di lunghezza più piccola in modo da massimizzare il loro valore totale. Il Paragrafo 15.2 spiega come moltiplicare una sequenza di matrici in modo da eseguire complessivamente il minor numero di prodotti scalari. Dopo questi esempi di programmazione dinamica, il Paragrafo 15.3 descrive due caratteristiche fondamentali che un problema deve avere per essere risolto

con la tecnica della programmazione dinamica. Il Paragrafo 15.4 spiega come trovare la più lunga sottosequenza comune a due sequenze usando la programmazione dinamica. Infine, il Paragrafo 15.5 applica la programmazione dinamica per costruire alberi binari di ricerca che hanno prestazioni ottime per una prefissata distribuzione di probabilità delle chiavi da ricercare.

15.1 Taglio delle aste

Il nostro primo esempio usa la programmazione dinamica per risolvere il problema di decidere dove tagliare delle aste di acciaio. La Serling Enterprises acquista delle lunghe aste di acciaio e le taglia in aste più corte, che poi vende. Ogni taglio è liberamente effettuabile. La direzione della Serling Enterprises vuole sapere qual è il modo migliore di tagliare le aste.

Supponiamo di conoscere, per $i = 1, 2, \ldots$, il prezzo p_i in dollari che la Serling Enterprises fa pagare un'asta di lunghezza i pollici. Le lunghezze delle aste sono sempre un numero intero di pollici. La Figura 15.1 riporta una tabella di prezzi campione.

Il ***problema del taglio delle aste*** può essere definito nel modo seguente. Data un'asta di lunghezza n pollici e una tabella di prezzi p_i, per $i = 1, 2, \ldots, n$, determinare il ricavo massimo r_n che si può ottenere tagliando l'asta e vendendone i pezzi. Si noti che, se il prezzo p_n di un'asta di lunghezza n è sufficientemente grande, la soluzione ottima potrebbe essere quella di non effettuare alcun taglio.

Consideriamo il caso in cui $n = 4$. La Figura 15.2 mostra tutti i modi in cui può essere tagliata un'asta lunga 4 pollici, incluso quello senza alcun taglio. Notiamo che tagliare un'asta di 4 pollici in due pezzi da 2 pollici fornisce un ricavo di $p_2 + p_2 = 5 + 5 = 10$, che è la soluzione ottima.

Un'asta di lunghezza n può essere tagliata in 2^{n-1} modi differenti, in quanto si ha un'opzione indipendente di tagliare, o non tagliare, alla distanza di i pollici dall'estremità sinistra, per $i = 1, 2, \ldots, n - 1$.[1] Denotiamo una decomposizione in pezzi utilizzando la normale notazione additiva, cosicché $7 = 2 + 2 + 3$ indica che un'asta di lunghezza 7 viene tagliata in tre pezzi – due di lunghezza 2 e uno di lunghezza 3. Se una soluzione ottima prevede il taglio dell'asta in k pezzi, per $1 \leq k \leq n$, allora una decomposizione ottima

$$n = i_1 + i_2 + \cdots + i_k$$

dell'asta in pezzi di lunghezze $i_1, i_2, \ldots, i_k$ fornisce il ricavo massimo corrispondente

$$r_n = p_{i_1} + p_{i_2} + \cdots + p_{i_k}$$

Per il problema in esame, possiamo determinare, per ispezione, i ricavi ottimi r_i, per $i = 1, 2, \ldots, 10$ e le corrispondenti decomposizioni ottime

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[1] Se dovessimo tagliare i pezzi in ordine non decrescente, ci sarebbero meno modi da considerare. Per $n = 4$, dovremmo considerare soltanto 5 modi: parti (a), (b), (c), (e) e (h) nella Figura 15.2. Il numero di modi è detto ***funzione di partizione***, che è approssimativamente uguale a $e^{\pi\sqrt{2n/3}}/4n\sqrt{3}$. Questa quantità è minore di 2^{n-1}, ma è maggiore di qualsiasi polinomio in n. Non approfondiremo ulteriormente questo argomento, tuttavia.

lunghezza i	1	2	3	4	5	6	7	8	9	10
prezzo p_i	1	5	8	9	10	17	17	20	24	30

Figura 15.1 Una tabella di prezzi campione delle aste. Un'asta di lunghezza i pollici viene venduta al prezzo di p_i dollari.

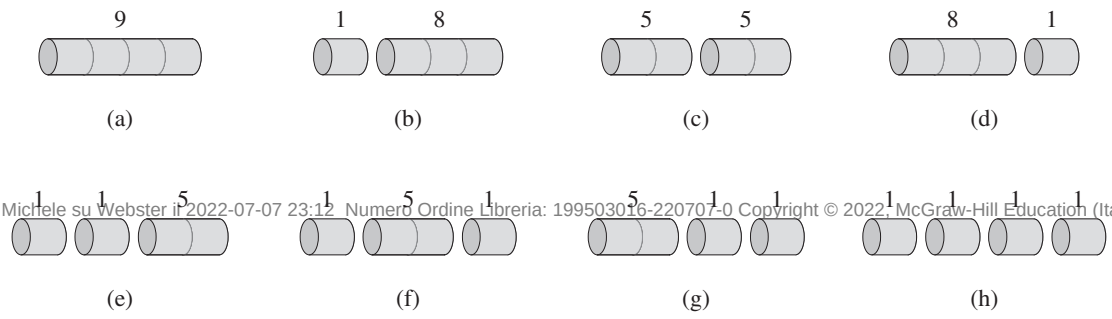
$r_1 = 1$ dalla soluzione $1 = 1$ (nessun taglio)
$r_2 = 5$ dalla soluzione $2 = 2$ (nessun taglio)
$r_3 = 8$ dalla soluzione $3 = 3$ (nessun taglio)
$r_4 = 10$ dalla soluzione $4 = 2 + 2$
$r_5 = 13$ dalla soluzione $5 = 2 + 3$
$r_6 = 17$ dalla soluzione $6 = 6$ (nessun taglio)
$r_7 = 18$ dalla soluzione $7 = 1 + 6$ o $7 = 2 + 2 + 3$
$r_8 = 22$ dalla soluzione $8 = 2 + 6$
$r_9 = 25$ dalla soluzione $9 = 3 + 6$
$r_{10} = 30$ dalla soluzione $10 = 10$ (nessun taglio)

Più in generale, possiamo esprimere i valori r_n per $n \geq 1$ in funzione dei ricavi ottimi delle aste più corte:

$$r_n = \max(p_n, r_1 + r_{n-1}, r_2 + r_{n-2}, \ldots, r_{n-1} + r_1) \tag{15.1}$$

Il primo argomento, p_n, corrisponde alla vendita dell'asta di lunghezza n senza tagli. Gli altri $n - 1$ argomenti corrispondono al ricavo massimo ottenuto facendo un taglio iniziale dell'asta in due pezzi di dimensione i e $n - i$, per $i = 1, 2, \ldots, n - 1$, e poi tagliando in modo ottimale gli ulteriori pezzi, ottenendo i ricavi r_i e r_{n-i} da questi due pezzi. Poiché non sappiamo a priori quale valore di i ottimizza i ricavi, dobbiamo considerare tutti i possibili valori di i e selezionare quello che massimizza i ricavi. Abbiamo anche l'opzione di non scegliere alcun valore di i, se possiamo ottenere il massimo ricavo vendendo le aste senza tagliarle.

Notate che per risolvere il problema originale di dimensione n, noi risolviamo problemi più piccoli dello stesso tipo, ma di dimensioni inferiori. Una volta effettuato il primo taglio, possiamo considerare i due pezzi come istanze indipendenti del problema del taglio delle aste. La soluzione ottima complessiva incorpora le soluzioni ottime dei due sottoproblemi correlati che massimizzano i ricavi di ciascuno dei due pezzi. Diciamo che il problema del taglio delle aste presenta una ***sottostruttura ottima***: le soluzioni ottime di un problema incorporano le



Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

Figura 15.2 Gli 8 modi possibili di tagliare un'asta di lunghezza 4. Sopra ogni pezzo c'è il valore del pezzo, secondo la tabella dei prezzi della Figura 15.1. La soluzione migliore è la parte (c) – tagliare l'asta in due pezzi di lunghezza 2 – che fornisce un valore totale di 10.

soluzioni ottime dei sottoproblemi correlati, che possono essere risolti in modo indipendente.

C'è un modo un po' più semplice di definire una struttura ricorsiva per il problema del taglio delle aste: consideriamo la decomposizione formata da un primo pezzo di lunghezza i tagliato dall'estremità sinistra e dal pezzo restante di destra di lunghezza $n - i$. Soltanto il pezzo restante di destra (non il primo pezzo) potrà essere ulteriormente tagliato. Possiamo vedere ciascuna decomposizione di un'asta di lunghezza n in questo modo: un primo pezzo seguito da un'eventuale decomposizione del pezzo restante. Così facendo, possiamo esprimere la soluzione senza alcun taglio dicendo che il primo pezzo ha dimensione $i = n$ e ricavo p_n e che il pezzo restante ha dimensione 0 con ricavo $r_0 = 0$. Otteniamo così la seguente versione semplificata dell'equazione (15.1):

$$r_n = \max_{1 \le i \le n} (p_i + r_{n-i}) \tag{15.2}$$

Secondo questa formulazione, una soluzione ottima incorpora la soluzione di *un solo* sottoproblema – il pezzo restante – anziché due.

Algoritmo ricorsivo top-down

La seguente procedura implementa il calcolo implicito nell'equazione (15.2) in modo ricorsivo top-down.

```
CUT-ROD(p, n)
1  if n == 0
2      return 0
3  q = -∞
4  for i = 1 to n
5      q = max(q, p[i] + CUT-ROD(p, n - i))
6  return q
```

La procedura CUT-ROD riceve come input un array $p[1..n]$ di prezzi e un intero n, e restituisce il massimo ricavo possibile per un'asta di lunghezza n. Se $n = 0$, nessun ricavo è possibile; quindi CUT-ROD restituisce 0 nella riga 2. La riga 3 inizializza il ricavo massimo q a $-\infty$, in modo che il ciclo **for**, righe 4–5, calcola correttamente $q = \max_{1 \le i \le n}(p_i + \text{CUT-ROD}(p, n - i))$; la riga 6 poi restituisce questo valore. Una semplice induzione su n dimostra che questa risposta è uguale alla risposta desiderata r_n, utilizzando l'equazione (15.2).

Se codificate CUT-ROD nel vostro linguaggio di programmazione preferito, vedrete che quando la dimensione dell'input diventa moderatamente grande, il vostro programma richiede un tempo relativamente lungo. Per $n = 40$, troverete che il vostro programma richiede molto più di qualche minuto e molto probabilmente anche più di un'ora. Infatti, ogni volta che incrementate n di 1, il tempo di esecuzione del vostro programma quasi raddoppia.

Acquistato da Michele Michele su Webster il 2022-07-0[illegible] Proprietà letteraria riservata. © 2022 McGraw-Hill Education (Italy)

Perché CUT-ROD è così inefficiente? Il problema è che la procedura CUT-ROD chiama più e più volte sé stessa in modo ricorsivo con gli stessi valori dei parametri; risolve ripetutamente gli stessi sottoproblemi. La Figura 15.3 illustra che cosa accade per $n = 4$: CUT-ROD(p, n) chiama CUT-ROD$(p, n - i)$ per $i = 1, 2, \ldots, n$. Ossia CUT-ROD(p, n) chiama CUT-ROD(p, j) per $j = 0, 1, \ldots, n - 1$. Se questo processo si svolge in modo ricorsivo, la quantità di lavoro svolto, come funzione di n, cresce in modo esplosivo.

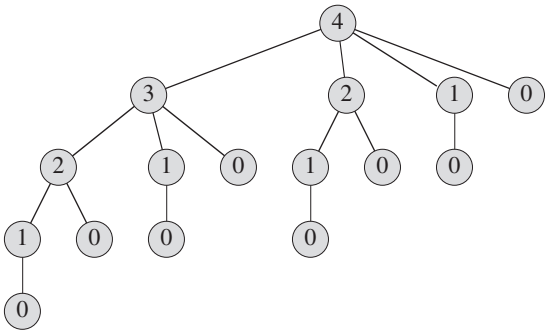


Figura 15.3 L'albero di ricorsione mostra le chiamate ricorsive che derivano da una chiamata di CUT-ROD(p, n) per $n = 4$. Le etichette all'interno dei nodi indicano la dimensione n del corrispondente sottoproblema, in modo che un arco da un padre con etichetta s a un figlio con etichetta t equivale a tagliare un pezzo iniziale di dimensione $s - t$ e a lasciare un sottoproblema di dimensione t. Un cammino dalla radice a una foglia corrisponde a uno dei 2^{n-1} modi di tagliare un'asta di lunghezza n. In generale, questo albero di ricorsione ha 2^n nodi e 2^{n-1} foglie.

Per analizzare il tempo di esecuzione di CUT-ROD, indichiamo con $T(n)$ il numero totale di chiamate di CUT-ROD quando la chiamata viene effettuata con il secondo parametro uguale a n. Questa espressione è uguale al numero di nodi in un sottoalbero la cui radice ha l'etichetta n nell'albero di ricorsione. Il conteggio include la chiamata iniziale alla radice. Quindi, $T(0) = 1$ e

$$T(n) = 1 + \sum_{j=0}^{n-1} T(j) \tag{15.3}$$

Il valore iniziale 1 riguarda la chiamata della radice, e il termine $T(j)$ conta il numero di chiamate (incluse le chiamate ricorsive) dovute alla chiamata di CUT-ROD$(p, n - i)$, dove $j = n - i$. Come l'Esercizio 15.1-1 vi chiede di dimostrare,

$$T(n) = 2^n \tag{15.4}$$

e quindi il tempo di esecuzione di CUT-ROD è esponenziale in n.

Questo tempo di esecuzione esponenziale non è poi così sorprendente. La procedura CUT-ROD considera esplicitamente tutti i 2^{n-1} modi possibili di tagliare un'asta di lunghezza n. L'albero delle chiamate ricorsive ha 2^{n-1} foglie, una per ogni modo possibile di tagliare l'asta. Le etichette nel cammino semplice dalla radice a una foglia forniscono le dimensioni di ciascun pezzo destro prima di effettuare un taglio; ovvero le etichette forniscono i corrispondenti punti di taglio, misurati dall'estremità destra dell'asta.

Applicare la programmazione dinamica al taglio ottimale delle aste

Vediamo adesso come sia possibile convertire CUT-ROD in un algoritmo efficiente utilizzando la programmazione dinamica.

Il metodo della programmazione dinamica funziona in questo modo. Avendo osservato che una semplice soluzione ricorsiva non è efficiente perché risolve ripetutamente gli stessi sottoproblemi, facciamo in modo che ogni sottoproblema sia risolto *una volta* soltanto, salvando la sua soluzione. Se avremo bisogno di nuovo della soluzione di questo sottoproblema, potremo riaverla immediatamente, senza bisogno di ricalcolarla. La programmazione dinamica richiede una memoria extra per ridurre il tempo di esecuzione; si tratta di un tipico esempio di ***compromesso tempo-memoria***. Il risparmio di tempo ottenibile può essere notevole: una soluzione con tempo esponenziale può essere trasformata in una soluzione con tempo polinomiale. Un metodo di programmazione dinamica viene eseguito in tempo polinomiale quando il numero di sottoproblemi *distinti* richiesti è polinomiale nella dimensione dell'input e ciascun sottoproblema può essere risolto in un tempo polinomiale.

Acquistato da Michele Michele su Webster il 2023-07-07 23:12 Numero Ordine Libreria: 1995030162307070 Copyright © 2022 McGraw-Hill Education (Italy)

Di solito, ci sono due modi equivalenti per implementare la programmazione dinamica. Illustreremo entrambi con il nostro esempio del taglio delle aste.

Il primo approccio è il ***metodo top-down con annotazione*** (memoization). In questo approccio, si scrive la procedura ricorsiva in modo naturale, modificandola per salvare il risultato di ciascun sottoproblema (di solito, in un array o in una tavola hash). La procedura prima verifica se ha risolto precedentemente questo sottoproblema. In caso affermativo, restituisce il valore salvato, risparmiando gli ulteriori calcoli a quel livello; altrimenti la procedura calcola il valore nel modo usuale. Diciamo che la procedura ricorsiva è stata dotata di un ***blocco note***, nel senso che "ricorda" quali risultati sono stati calcolati in precedenza.

Il secondo approccio è il ***metodo bottom-up***. Tipicamente, questo approccio dipende da un concetto naturale della "dimensione" di un sottoproblema, in modo tale che la risoluzione di un particolare sottoproblema dipenda soltanto dalla risoluzione di sottoproblemi "più piccoli". Ordiniamo i sottoproblemi per dimensione e poi li risolviamo ordinatamente a partire dal più piccolo. Quando risolviamo un particolare sottoproblema, abbiamo già risolto tutti i sottoproblemi più piccoli da cui dipende la sua soluzione, e abbiamo salvato le loro soluzioni. Ogni sottoproblema viene risolto una sola volta e, quando lo incontriamo, abbiamo già risolto tutti i suoi sottoproblemi.

Questi due approcci generano algoritmi con lo stesso tempo di esecuzione asintotico, tranne in casi eccezionali quando l'approccio top-down in effetti non esamina ricorsivamente tutti i possibili sottoproblemi. L'approccio bottom-up spesso ha fattori costanti molto migliori, in quanto ha meno costi per le chiamate di procedura.

Ecco la pseudocodifica della procedura top-down CUT-ROD, con l'aggiunta del blocco note:

```
MEMOIZED-CUT-ROD(p, n)
1  Sia r[0..n] un nuovo array
2  for i = 0 to n
3      r[i] = −∞
4  return MEMOIZED-CUT-ROD-AUX(p, n, r)

MEMOIZED-CUT-ROD-AUX(p, n, r)
1  if r[n] ≥ 0
2      return r[n]
3  if n == 0
4      q = 0
5  else q = −∞
6      for i = 1 to n
7          q = max(q, p[i] + MEMOIZED-CUT-ROD-AUX(p, n − i, r))
8  r[n] = q
9  return q
```

Acquistato da Michele Michele su Webster il 2022-07-07 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Qui la procedura principale MEMOIZED-CUT-ROD inizializza un nuovo array ausiliario $r[0..n]$ con il valore $-\infty$, una scelta comoda per indicare i valori "incogniti" (i valori dei ricavi noti sono sempre non negativi). Poi chiama la sua routine ausiliaria MEMOIZED-CUT-ROD-AUX.

La procedura MEMOIZED-CUT-ROD-AUX non è altro che la versione dotata di blocco note della precedente procedura CUT-ROD. La riga 1 controlla innanzi tutto se il valore desiderato è già noto; in questo caso, la riga 2 restituisce tale valore. Altrimenti, le righe 3–7 calcolano il valore desiderato q nel modo usuale, la riga 8 lo salva in $r[n]$ e la riga 9 lo restituisce.

La versione bottom-up è ancora più semplice:

```
BOTTOM-UP-CUT-ROD(p, n)
1  Sia r[0..n] un nuovo array
2  r[0] = 0
3  for j = 1 to n
4       q = -∞
5       for i = 1 to j
6            q = max(q, p[i] + r[j - i])
7       r[j] = q
8  return r[n]
```

Per l'approccio bottom-up della programmazione dinamica, BOTTOM-UP-CUT-ROD segue l'ordine naturale dei sottoproblemi: un problema di dimensione i è "più piccolo" di un sottoproblema di dimensione j, se $i < j$. Quindi, la procedura risolve i sottoproblemi di dimensioni $j = 0, 1, \ldots, n$, in quest'ordine.

La riga 1 della procedura BOTTOM-UP-CUT-ROD crea un nuovo array $r[0..n]$ in cui salvare i risultati dei sottoproblemi; la riga 2 inizializza $r[0]$ a 0, in quanto un'asta di lunghezza 0 non genera alcun ricavo. Le righe 3–6 risolvono ciascun sottoproblema di dimensione j, per $j = 1, 2, \ldots, n$, nell'ordine delle dimensioni crescenti. L'approccio utilizzato per risolvere un problema di una particolare dimensione j è uguale a quello utilizzato da CUT-ROD, ad eccezione della riga 6 che adesso fa riferimento direttamente all'elemento $r[j - i]$ dell'array, anziché fare una chiamata ricorsiva per risolvere il sottoproblema di dimensione $j - i$. La riga 7 salva in $r[j]$ la soluzione del sottoproblema di dimensione j. Infine, la riga 8 restituisce $r[n]$, che è uguale al valore ottimo r_n.

Le versioni bottom-up e top-down hanno lo stesso tempo di esecuzione asintotico. Il tempo di esecuzione della procedura BOTTOM-UP-CUT-ROD è $\Theta(n^2)$, a causa della doppia struttura annidata del suo ciclo. Il numero di iterazioni del suo ciclo più interno **for**, nelle righe 5–6, forma una serie aritmetica. Anche il tempo di esecuzione della sua controparte top-down, MEMOIZED-CUT-ROD, è $\Theta(n^2)$, sebbene questo tempo di esecuzione sia un po' più difficile da spiegare. Poiché una chiamata ricorsiva per risolvere un sottoproblema precedentemente risolto termina immediatamente, MEMOIZED-CUT-ROD risolve ciascun sottoproblema una sola volta. La procedura risolve i sottoproblemi di dimensione $0, 1, \ldots, n$. Per risolvere un sottoproblema di dimensione n, il ciclo **for**, righe 6–7, effettua n iterazioni. Quindi, il numero totale di iterazioni di questo ciclo **for**, per tutte le chiamate ricorsive di MEMOIZED-CUT-ROD, forma una serie aritmetica, per un totale di $\Theta(n^2)$ iterazioni, esattamente come il ciclo interno **for** di BOTTOM-UP-CUT-ROD. (In effetti qui stiamo utilizzando una forma di analisi aggregata. Vedremo più dettagliatamente questo tipo di analisi nel Paragrafo 17.1.)

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995U3016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

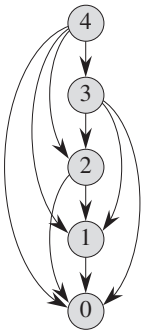


Figura 15.4 Il grafo dei sottoproblemi per il problema del taglio delle aste con $n = 4$. Le etichette nei vertici del grafo indicano le dimensioni dei sottoproblemi corrispondenti. Un arco orientato (x, y) indica che occorre una soluzione del sottoproblema y per risolvere il sottoproblema x. Questo grafo è una versione ridotta dell'albero della Figura 15.3, in cui tutti i nodi con la stessa etichetta sono raccolti in un unico vertice e tutti gli archi vanno dal padre al figlio.

Grafo dei sottoproblemi

Quando si affronta un problema di programmazione dinamica, è importante capire l'insieme dei sottoproblemi e come i sottoproblemi dipendano l'uno dall'altro.

Il ***grafo dei sottoproblemi*** di un problema incorpora esattamente queste informazioni. La Figura 15.4 mostra il grafo dei sottoproblemi per il problema del taglio delle aste con $n = 4$. È un grafo orientato, che contiene un vertice per ogni sottoproblema distinto. Il grafo dei sottoproblemi ha un arco orientato dal vertice del sottoproblema x al vertice del sottoproblema y, se per determinare una soluzione ottima per il sottoproblema x occorre considerare direttamente una soluzione ottima per il sottoproblema y. Per esempio, il grafo dei sottoproblemi contiene un arco da x a y se una procedura ricorsiva top-down per risolvere x chiama direttamente se stessa per risolvere y. Possiamo pensare al grafo dei sottoproblemi come a una versione "ridotta" o "compatta" dell'albero di ricorsione per il metodo ricorsivo top-down, in cui tutti i nodi relativi allo stesso sottoproblema vengono fusi in un unico vertice e tutti gli archi sono orientati dal padre al figlio.

Il metodo bottom-up della programmazione dinamica considera i vertici del grafo dei sottoproblemi in un ordine tale da risolvere i sottoproblemi y adiacenti a un dato sottoproblema x, prima di risolvere il sottoproblema x. (Ricordiamo dal Paragrafo B.4 che la relazione di adiacenza non è necessariamente simmetrica.) Utilizzando la terminologia del Capitolo 22, in un algoritmo bottom-up della programmazione dinamica, i vertici del grafo dei sottoproblemi vengono considerati secondo un "ordinamento topologico inverso" o un "ordinamento topologico della versione trasposta" del grafo dei sottoproblemi (vedere il Paragrafo 22.4). In altre parole, nessun sottoproblema viene considerato finché non sono stati risolti tutti i sottoproblemi da cui esso dipende. Analogamente, applicando i concetti dello stesso capitolo, possiamo immaginare il metodo top-down (con annotazione) della programmazione dinamica come una "visita in profondità" del grafo dei sottoproblemi (vedere il Paragrafo 22.3).

La dimensione del grafo dei sottoproblemi $G = (V, E)$ ci può aiutare a determinare il tempo di esecuzione dell'algoritmo di programmazione dinamica. Poiché ogni sottoproblema viene risolto una sola volta, il tempo di esecuzione è la somma dei tempi di esecuzione necessari per risolvere ciascun sottoproblema. Tipicamente, il tempo per calcolare la soluzione di un sottoproblema è proporzionale al grado (numero di archi uscenti) del vertice corrispondente nel grafo dei sottoproblemi, e il numero di sottoproblemi è uguale al numero dei vertici del grafo dei sottoproblemi. In questo caso comune, il tempo di esecuzione della programmazione dinamica è lineare nel numero di vertici e archi.

Acquistato da Michele Michele su Webster il 2022-07-07 Non distribuire. Copyright © 2022, The MIT Press (Italy)

Ricostruire una soluzione

La nostra soluzione con la programmazione dinamica del problema del taglio delle aste fornisce il valore di una soluzione ottima, non la soluzione effettiva: una lista di dimensioni dei pezzi. Possiamo estendere l'approccio della programmazione dinamica per memorizzare non soltanto il *valore* ottimo calcolato per ciascun sottoproblema, ma anche una *scelta* che determina il valore ottimo. Con questa informazione, siamo in grado di stampare facilmente una soluzione ottima.

Ecco una versione estesa di BOTTOM-UP-CUT-ROD che calcola, per ogni dimensione j dell'asta, non soltanto il ricavo massimo r_j, ma anche s_j, la dimensione ottima del primo pezzo da tagliare:

```
EXTENDED-BOTTOM-UP-CUT-ROD(p, n)
 1  Siano r[0..n] ed s[0..n] due nuovi array
 2  r[0] = 0
 3  for j = 1 to n
 4      q = -∞
 5      for i = 1 to j
 6          if q < p[i] + r[j - i]
 7              q = p[i] + r[j - i]
 8              s[j] = i
 9      r[j] = q
10  return r ed s
```

Questa procedura è simile a BOTTOM-UP-CUT-ROD, con la differenza che crea l'array s nella riga 1, e aggiorna $s[j]$ nella riga 8 per conservare la dimensione ottima i del primo pezzo da tagliare quando viene risolto un sottoproblema di dimensione j.

La seguente procedura riceve una tabella di prezzi p e una dimensione n dell'asta; poi chiama EXTENDED-BOTTOM-UP-CUT-ROD per calcolare l'array $s[1..n]$ delle dimensioni ottime dei primi pezzi e stampa la lista completa delle dimensioni dei pezzi per una decomposizione ottima di un'asta di lunghezza n:

```
PRINT-CUT-ROD-SOLUTION(p, n)
1  (r, s) = EXTENDED-BOTTOM-UP-CUT-ROD(p, n)
2  while n > 0
3      print s[n]
4      n = n - s[n]
```

Nel nostro esempio del taglio delle aste, la chiamata EXTENDED-BOTTOM-UP-CUT-ROD$(p, 10)$ restituisce i seguenti array:

i	0	1	2	3	4	5	6	7	8	9	10
$r[i]$	0	1	5	8	10	13	17	18	22	25	30
$s[i]$	0	1	2	3	2	2	6	1	2	3	10

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Una chiamata di PRINT-CUT-ROD-SOLUTION$(p, 10)$ stamperebbe soltanto 10, mentre una chiamata con $n = 7$ stamperebbe i tagli 1 e 6, che corrispondono alla prima decomposizione ottima per r_7, data in precedenza.

Esercizi

15.1-1

Dimostrate che l'equazione (15.4) deriva dall'equazione (15.3) con la condizione iniziale $T(0) = 1$.

15.1-2

Dimostrate, per mezzo di un controesempio, che la seguente strategia "golosa" non sempre determina un modo ottimale di tagliare le aste. Definite la ***densità*** di un'asta di lunghezza i come p_i/i, ovvero il suo valore per pollice. La strategia golosa per un'asta di lunghezza n taglia un primo pezzo di lunghezza i (con $1 \leq i \leq n$) che ha la massima densità. Poi continua ad applicare il metodo goloso al pezzo restante di lunghezza $n - i$.

15.1-3

Considerate una variante del problema del taglio delle aste in cui, oltre al prezzo p_i di ciascuna asta, ogni taglio ha un costo fisso c. Il ricavo associato a una soluzione adesso è la somma dei prezzi di vendita meno i costi dei tagli. Trovate un algoritmo di programmazione dinamica per risolvere questa variante del problema.

15.1-4

Modificate la procedura MEMOIZED-CUT-ROD in modo che restituisca non soltanto il valore, ma anche la soluzione effettiva.

15.1-5

I numeri di Fibonacci sono definiti dalla ricorrenza (3.22). Trovate un algoritmo di programmazione dinamica con tempo $O(n)$ per calcolare l'n-esimo numero di Fibonacci. Disegnate il grafo dei sottoproblemi. Quanti vertici e archi ci sono nel grafo?

15.2 Moltiplicare una sequenza di matrici

Il prossimo esempio di programmazione dinamica è un algoritmo che risolve il problema della moltiplicazione di una sequenza di matrici. Data una sequenza di n matrici $\langle A_1, A_2, \ldots, A_n \rangle$, vogliamo calcolare il prodotto

$$A_1 A_2 \cdots A_n \tag{15.5}$$

Possiamo calcolare l'espressione (15.5) utilizzando come subroutine l'algoritmo standard per moltiplicare una coppia di matrici, dopo che abbiamo posto le opportune parentesi per eliminare qualsiasi ambiguità sul modo in cui devono essere moltiplicate le matrici. La moltiplicazione delle matrici è associativa, quindi tutte le parentesizzazioni forniscono lo stesso prodotto. Un prodotto di matrici è ***completamente parentesizzato*** se è una singola matrice oppure è il prodotto, racchiuso tra parentesi, di due prodotti di matrici completamente parentesizzati. Per esempio, se la sequenza delle matrici è $\langle A_1, A_2, A_3, A_4 \rangle$, il prodotto $A_1 A_2 A_3 A_4$ può essere completamente parentesizzato in cinque modi distinti:

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$(A_1(A_2(A_3A_4)))$
$(A_1((A_2A_3)A_4))$
$((A_1A_2)(A_3A_4))$
$((A_1(A_2A_3))A_4)$
$(((A_1A_2)A_3)A_4)$

Il modo in cui parentesizziamo una sequenza di matrici può avere un impatto notevole sul costo per calcolare il prodotto. Consideriamo prima il costo per moltiplicare due matrici. L'algoritmo standard è dato dal seguente pseudocodice, che generalizza la procedura SQUARE-MATRIX-MULTIPLY descritta nel Paragrafo 4.2. Gli attributi *rows* e *columns* sono i numeri di righe e di colonne di una matrice.

```
MATRIX-MULTIPLY(A, B)
1  if A.columns ≠ B.rows
2       error "dimensioni non compatibili"
3  else Sia C una nuova matrice A.rows × B.columns
4       for i = 1 to A.rows
5            for j = 1 to B.columns
6                 c_ij = 0
7                 for k = 1 to A.columns
8                      c_ij = c_ij + a_ik · b_kj
9       return C
```

Possiamo moltiplicare due matrici A e B soltanto se sono ***compatibili***: il numero di colonne di A deve essere uguale al numero di righe di B. Se A è una matrice $p \times q$ e B è una matrice $q \times r$, la matrice risultante C è una matrice $p \times r$. Il tempo per calcolare C è dominato dal numero di prodotti scalari nella riga 8, che è pqr. In seguito esprimeremo i costi in funzione del numero di prodotti scalari.

Per mostrare come il costo per moltiplicare le matrici dipenda dallo schema di parentesizzazione, consideriamo il problema di moltiplicare una sequenza di tre matrici $\langle A_1, A_2, A_3\rangle$. Supponiamo che le dimensioni delle matrici siano, rispettivamente, 10×100, 100×5 e 5×50. Se moltiplichiamo secondo lo schema di parentesizzazione $((A_1A_2)A_3)$, eseguiamo $10 \cdot 100 \cdot 5 = 5000$ prodotti scalari per calcolare la matrice 10×5 risultante dal prodotto A_1A_2, più altri $10 \cdot 5 \cdot 50 = 2500$ prodotti scalari per moltiplicare questa matrice per A_3, per un totale di 7500 prodotti scalari. Se, invece, moltiplichiamo secondo lo schema di parentesizzazione $(A_1(A_2A_3))$, eseguiamo $100 \cdot 5 \cdot 50 = 25000$ prodotti scalari per calcolare la matrice 100×50 risultante dal prodotto A_2A_3, più altri $10 \cdot 100 \cdot 50 = 50000$ prodotti scalari per moltiplicare A_1 per questa matrice, per un totale di 75000 prodotti scalari. Quindi, il calcolo della moltiplicazione delle matrici è 10 volte più rapido con il primo schema di parentesizzazione.

Il ***problema della moltiplicazione di una sequenza di matrici*** può essere definito in questo modo: data una sequenza di n matrici $\langle A_1, A_2, \ldots, A_n\rangle$, dove la matrice A_i ha dimensioni $p_{i-1} \times p_i$ per $i = 1, 2, \ldots, n$, determinare lo schema di parentesizzazione completa del prodotto $A_1A_2 \cdots A_n$ che minimizza il numero di prodotti scalari.

È importante notare che, nel problema della moltiplicazione di una sequenza di matrici, non vengono effettivamente moltiplicate le matrici. Il nostro obbiettivo è soltanto quello di determinare un ordine di moltiplicazione delle matrici che ha il costo minimo. Tipicamente, il tempo impiegato per determinare quest'ordine ottimo è più che ripagato dal tempo risparmiato successivamente per eseguire effettivamente i prodotti delle matrici (per esempio, eseguire soltanto 7500 prodotti scalari, anziché 75000).

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995036016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Contare il numero di parentesizzazioni

Prima di risolvere il problema della moltiplicazione di una sequenza di matrici con la programmazione dinamica, vogliamo dimostrare che un controllo esaustivo di

tutti i possibili schemi di parentesizzazione non ci consente di ottenere un algoritmo efficiente. Indichiamo con $P(n)$ il numero di parentesizzazioni alternative di una sequenza di n matrici. Quando $n = 1$, c'è una sola matrice e, quindi, un solo schema di parentesizzazione. Quando $n \geq 2$, un prodotto di matrici completamente parentesizzato è il prodotto di due sottoprodotti di matrici completamente parentesizzati e la suddivisione fra i due sottoprodotti può avvenire fra la k-esima e la $(k+1)$-esima matrice per qualsiasi $k = 1, 2, \ldots, n-1$. Quindi, otteniamo la ricorrenza

$$P(n) = \begin{cases} 1 & \text{se } n = 1 \\ \displaystyle\sum_{k=1}^{n-1} P(k)P(n-k) & \text{se } n \geq 2 \end{cases} \tag{15.6}$$

Il Problema 12-4 richiedeva di dimostrare che la soluzione di una ricorrenza simile è la sequenza dei ***numeri catalani***, che cresce come $\Omega(4^n/n^{3/2})$. Un esercizio più semplice (vedere l'Esercizio 15.2-3) consiste nel dimostrare che la soluzione della ricorrenza (15.6) è $\Omega(2^n)$. Il numero di soluzioni è quindi esponenziale in n; pertanto la tecnica a forza bruta di ricercare tutti i possibili schemi di parentesizzazione è una strategia inadeguata per determinare la parentesizzazione ottima di una sequenza di matrici.

Applicare la programmazione dinamica

Utilizzeremo il metodo della programmazione dinamica per determinare la parentesizzazione ottima di una sequenza di matrici. Per fare questo, seguiremo un procedimento di quattro fasi che abbiamo definito all'inizio di questo capitolo:

1. Caratterizzare la struttura di una soluzione ottima.
2. Definire in modo ricorsivo il valore di una soluzione ottima.
3. Calcolare il valore di una soluzione ottima.
4. Costruire una soluzione ottima dalle informazioni calcolate.

Esamineremo ordinatamente queste fasi per spiegare con chiarezza come applicarle al problema.

Fase 1: struttura di una parentesizzazione ottima

La prima fase del paradigma della programmazione dinamica consiste nel trovare la sottostruttura ottima e poi utilizzare questa sottostruttura per costruire una soluzione ottima del problema dalle soluzioni ottime dei sottoproblemi. Per il problema della moltiplicazione di una sequenza di matrici, possiamo svolgere tale compito nel modo seguente. Per comodità, adottiamo la notazione $A_{i..j}$, dove $i \leq j$, per la matrice che si ottiene calcolando il prodotto $A_i A_{i+1} \cdots A_j$. Notate che, se il problema non è banale, cioè $i < j$, allora qualsiasi parentesizzazione del prodotto $A_i A_{i+1} \cdots A_j$ deve suddividere il prodotto fra A_k e A_{k+1} per qualche intero k nell'intervallo $i \leq k < j$; ovvero, per qualche valore di k, prima calcoliamo le matrici $A_{i..k}$ e $A_{k+1..j}$ e, poi, le moltiplichiamo per ottenere il prodotto finale $A_{i..j}$. Il costo di questa parentesizzazione è, quindi, il costo per calcolare la matrice $A_{i..k}$, più il costo per calcolare la matrice $A_{k+1..j}$, più il costo per moltiplicare queste due matrici.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995030I6-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

La sottostruttura ottima di questo problema è la seguente. Supponiamo che una parentesizzazione ottima di $A_i A_{i+1} \cdots A_j$ suddivida il prodotto fra A_k e A_{k+1}. Allora la parentesizzazione della prima sottosequenza $A_i A_{i+1} \cdots A_k$ all'interno di questa parentesizzazione ottima di $A_i A_{i+1} \cdots A_j$ deve essere una parentesizzazione ottima di $A_i A_{i+1} \cdots A_k$. Perché? Se ci fosse un modo meno costoso di parentesizzare $A_i A_{i+1} \cdots A_k$, sostituendo questa parentesizzazione in quella ottima di $A_i A_{i+1} \cdots A_j$ otterremmo un'altra parentesizzazione di $A_i A_{i+1} \cdots A_j$ il cui costo sarebbe minore di quella ottima: una contraddizione. Un'osservazione analoga vale per la parentesizzazione della sottosequenza $A_{k+1} A_{k+2} \cdots A_j$ nella parentesizzazione ottima di $A_i A_{i+1} \cdots A_j$: deve essere una parentesizzazione ottima di $A_{k+1} A_{k+2} \cdots A_j$.

Adesso utilizziamo la nostra sottostruttura ottima per dimostrare che possiamo costruire una soluzione ottima del problema dalle soluzioni ottime dei sottoproblemi. Abbiamo visto che qualsiasi soluzione di un'istanza non banale del problema della moltiplicazione di una sequenza di matrici richiede che il prodotto venga suddiviso in due sottoprodotti; inoltre qualsiasi soluzione ottima contiene al suo interno soluzioni ottime delle istanze dei sottoproblemi. Quindi, possiamo costruire una soluzione ottima di un'istanza del problema della moltiplicazione di una sequenza di matrici suddividendo il problema in due sottoproblemi (quelli della parentesizzazione ottima di $A_i A_{i+1} \cdots A_k$ e $A_{k+1} A_{k+2} \cdots A_j$), trovando le soluzioni ottime delle istanze dei sottoproblemi e, infine, combinando le soluzioni ottime dei sottoproblemi. Quando cerchiamo il punto esatto in cui suddividere il prodotto, dobbiamo considerare tutti i possibili punti per avere la certezza di avere esaminato il punto ottimo.

Fase 2: una soluzione ricorsiva

Adesso definiamo il costo di una soluzione ottima ricorsivamente in funzione delle soluzioni ottime dei sottoproblemi. Per il problema della moltiplicazione di una sequenza di matrici, scegliamo come sottoproblemi i problemi di determinare il costo minimo di una parentesizzazione di $A_i A_{i+1} \cdots A_j$ per $1 \leq i \leq j \leq n$. Sia $m[i, j]$ il numero minimo di prodotti scalari richiesti per calcolare la matrice $A_{i..j}$; per il problema principale, il costo del metodo più economico per calcolare $A_{1..n}$ sarà quindi $m[1, n]$.

Possiamo definire $m[i, j]$ ricorsivamente in questo modo. Se $i = j$, il problema è banale; la sequenza è formata da una matrice $A_{i..i} = A_i$, quindi non occorre eseguire alcun prodotto scalare. Allora, $m[i, i] = 0$ per $i = 1, 2, \ldots, n$. Per calcolare $m[i, j]$ quando $i < j$, sfruttiamo la struttura di una soluzione ottima ottenuta nella fase 1. Supponiamo che la parentesizzazione ottima suddivida il prodotto $A_i A_{i+1} \cdots A_j$ fra A_k e A_{k+1}, dove $i \leq k < j$. Quindi, $m[i, j]$ è uguale al costo minimo per calcolare i sottoprodotti $A_{i..k}$ e $A_{k+1..j}$, più il costo per moltiplicare queste due matrici. Ricordando che ogni matrice A_i è $p_{i-1} \times p_i$, il calcolo del prodotto delle matrici $A_{i..k} A_{k+1..j}$ richiede $p_{i-1} p_k p_j$ prodotti scalari. Quindi, otteniamo

Acquistato da Michele Mostarda (mostarda) il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$m[i, j] = m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j$$

Questa equazione ricorsiva suppone che sia noto il valore di k, che invece non conosciamo. Notiamo, tuttavia, che ci sono soltanto $j - i$ valori possibili per k, ovvero $k = i, i + 1, \ldots, j - 1$. Poiché la parentesizzazione ottima deve utilizzare uno di questi valori di k, dobbiamo semplicemente controllarli tutti per trova-

re il migliore. Quindi, la nostra definizione ricorsiva per il costo minimo di una parentesizzazione del prodotto $A_i A_{i+1} \cdots A_j$ diventa

$$m[i, j] = \begin{cases} 0 & \text{se } i = j \\ \min\limits_{i \le k < j} \{m[i, k] + m[k+1, j] + p_{i-1} p_k p_j\} & \text{se } i < j \end{cases} \tag{15.7}$$

I valori $m[i, j]$ sono i costi delle soluzioni ottime dei sottoproblemi, ma essi non ci forniscono tutte le informazioni necessarie a ricostruire la soluzione ottima. Per poterlo fare definiamo $s[i, j]$ come il valore di k in cui è stato suddiviso il prodotto $A_i A_{i+1} \cdots A_j$ per ottenere una parentesizzazione ottima. Ovvero, $s[i, j]$ è uguale a un valore k tale che $m[i, j] = m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j$.

Fase 3: calcolo dei costi ottimi

A questo punto, è semplice scrivere un algoritmo ricorsivo basato sulla ricorrenza (15.7) per calcolare il costo minimo $m[1, n]$ del prodotto $A_1 A_2 \cdots A_n$. Tuttavia, come vedremo nel Paragrafo 15.3, questo algoritmo ha un tempo esponenziale, che non è migliore della tecnica a forza bruta di controllare tutti i possibili schemi di parentesizzazione del prodotto.

Osserviamo che ci sono relativamente pochi problemi distinti: un problema per ogni possibile scelta di i e j, con $1 \le i \le j \le n$, per un totale di $\binom{n}{2} + n = \Theta(n^2)$. Un algoritmo ricorsivo può incontrare ciascun sottoproblema più volte nelle varie diramazioni del suo albero di ricorsione. Questa proprietà dei sottoproblemi che si ripresentano è la seconda caratteristica peculiare dell'applicabilità della programmazione dinamica (la prima è la sottostruttura ottima).

Anziché calcolare la soluzione della ricorrenza (15.7) ricorsivamente, calcoliamo il costo ottimale applicando un metodo tabulare bottom-up (nel Paragrafo 15.3 presenteremo il corrispondente approccio top-down con annotazione). Implementeremo il metodo bottom-up tabulare con la procedura MATRIX-CHAIN-ORDER riportata più sotto. Questa procedura assume che la matrice A_i abbia dimensioni $p_{i-1} \times p_i$ per $i = 1, 2, \ldots, n$. L'input è una sequenza $p = \langle p_0, p_1, \ldots, p_n \rangle$, dove $p.length = n + 1$. La procedura usa una tabella ausiliaria $m[1..n, 1..n]$ per memorizzare i costi $m[i, j]$ e una tabella ausiliaria $s[1..n, 1..n]$ che registra l'indice k cui corrisponde il costo ottimo nel calcolo di $m[i, j]$. La tabella s sarà poi usata per costruire una soluzione ottima.

Per implementare correttamente il metodo bottom-up, dobbiamo determinare quali posizioni nella tabella sono utilizzate nel calcolo di $m[i, j]$. L'Equazione (15.7) indica che il costo $m[i, j]$ per calcolare il prodotto di $j - i + 1$ matrici dipende soltanto dai costi per calcolare prodotti di sequenze di meno di $j - i + 1$ matrici. Ovvero, per $k = i, i + 1, \ldots, j - 1$, la matrice $A_{i..k}$ è un prodotto di $k - i + 1 < j - i + 1$ matrici e la matrice $A_{k+1..j}$ è un prodotto di $j - k < j - i + 1$ matrici. Quindi, l'algoritmo dovrebbe riempire la tabella m in modo da risolvere il problema della parentesizzazione di sequenze di matrici di lunghezza crescente. Per il sottoproblema della parentesizzazione ottima della sequenza di matrici $A_i A_{i+1} \cdots A_j$ assumiamo come dimensione del problema la lunghezza $j - i + 1$ della sequenza.

Acquistato da Michele Michele su Webster il 2022-07-07 21:11 Numero Ordine: [illegible] Licenza non trasferibile e non riproducibile - Tutti i diritti riservati

```
MATRIX-CHAIN-ORDER(p)
 1  n = p.length − 1
 2  Siano m[1..n, 1..n] ed s[1..n − 1, 2..n] due nuove tabelle
 3  for i = 1 to n
 4      m[i, i] = 0
 5  for l = 2 to n                    // l è la lunghezza della sequenza
 6      for i = 1 to n − l + 1
 7          j = i + l − 1
 8          m[i, j] = ∞
 9          for k = i to j − 1
10              q = m[i, k] + m[k + 1, j] + p_{i−1} p_k p_j
11              if q < m[i, j]
12                  m[i, j] = q
13                  s[i, j] = k
14  return m ed s
```

L'algoritmo prima calcola $m[i,i] = 0$ per $i = 1,2,\ldots,n$ (i costi minimi per sequenze di lunghezza $l = 1$) nelle righe 3–4. Poi usa la ricorrenza (15.7) per calcolare $m[i,i+1]$ per $i = 1,2,\ldots,n-1$ (i costi minimi per sequenze di lunghezza $l = 2$) durante la prima esecuzione del ciclo **for** nelle righe 5–13. Nella seconda iterazione del ciclo, l'algoritmo $m[i,i+2]$ per $i = 1,2,\ldots,n-2$ (i costi minimi per le catene di lunghezza $l = 3$) e così via. In ciascun passo, il costo calcolato $m[i,j]$ nelle righe 10–13 dipende soltanto dagli elementi della tabella $m[i,k]$ e $m[k+1,j]$ già calcolati.

La Figura 15.5 illustra questa procedura con una sequenza di $n = 6$ matrici. Poiché abbiamo definito $m[i,j]$ soltanto per $i \le j$, viene utilizzata solamente la porzione della tabella m che si trova subito sopra la diagonale principale. La figura mostra la tabella ruotata in modo che la diagonale principale risulti orizzontale. La sequenza delle matrici è elencata sotto la figura. Utilizzando questo schema, il costo minimo $m[i,j]$ per moltiplicare una sottosequenza di matrici $A_i A_{i+1} \cdots A_j$ può essere trovato nel punto di intersezione delle linee con direzione nord-est da A_i e con direzione nord-ovest da A_j. Ogni riga orizzontale nella tabella contiene le posizioni per sequenze di matrici della stessa lunghezza. MATRIX-CHAIN-ORDER calcola le righe dal basso verso l'alto e da sinistra a destra all'interno di ogni riga. Una posizione $m[i,j]$ è calcolata utilizzando i prodotti $p_{i-1} p_k p_j$ per $k = i, i+1, \ldots, j-1$ e tutte le posizioni con direzione sud-ovest e sud-est da $m[i,j]$.

Da un semplice esame della struttura annidata dei cicli della procedura MATRIX-CHAIN-ORDER si deduce che il tempo di esecuzione dell'algoritmo è pari a $O(n^3)$. I cicli hanno tre livelli di annidamento e ogni indice di ciclo (l, i e k) assume al massimo $n-1$ valori. L'Esercizio 15.2-5 chiede di dimostrare che il tempo di esecuzione di questo algoritmo, in effetti, è anche $\Omega(n^3)$. L'algoritmo richiede lo spazio $\Theta(n^2)$ per memorizzare le tabelle m e s. Quindi, la procedura MATRIX-CHAIN-ORDER è molto più efficiente del metodo con tempo esponenziale che elenca tutte le possibili parentesizzazioni controllandole una per una.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995080016-2267070 Copyright © 2022, McGraw-Hill Education (Italy)

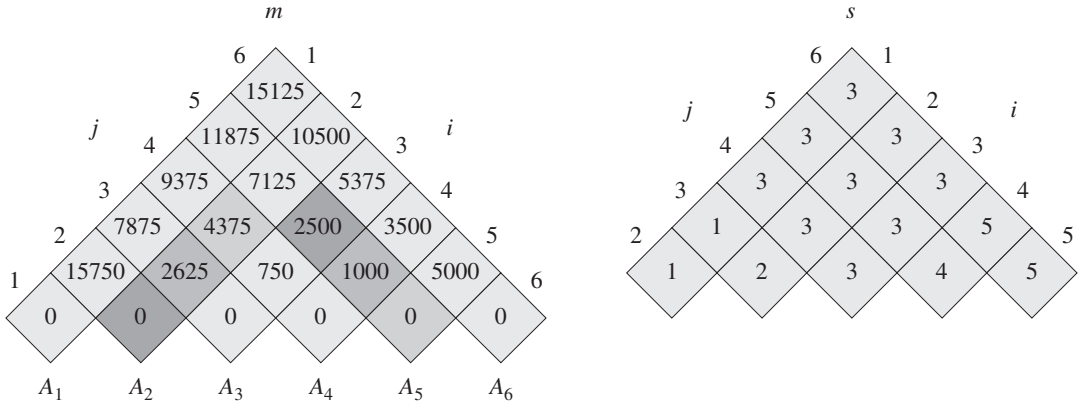


Figura 15.5 Le tabelle m e s calcolate da MATRIX-CHAIN-ORDER per $n = 6$ matrici con le seguenti dimensioni:

matrice	A_1	A_2	A_3	A_4	A_5	A_6
dimensione	30×35	35×15	15×5	5×10	10×20	20×25

Le tabelle sono ruotate in modo che la diagonale principale sia orizzontale. Della tabella m sono utilizzati soltanto la diagonale principale e il triangolo superiore. Della tabella s è utilizzato soltanto il triangolo superiore. Il numero minimo di prodotti scalari per moltiplicare le 6 matrici è $m[1, 6] = 15125$. Fra le posizioni con sfondo più scuro, le coppie che hanno la stessa gradazione di grigio sono utilizzate insieme nella riga 10 durante i calcoli

$$m[2,5] = \min \begin{cases} m[2,2] + m[3,5] + p_1 p_2 p_5 = 0 + 2500 + 35 \cdot 15 \cdot 20 & = 13000 \\ m[2,3] + m[4,5] + p_1 p_3 p_5 = 2625 + 1000 + 35 \cdot 5 \cdot 20 & = 7125 \\ m[2,4] + m[5,5] + p_1 p_4 p_5 = 4375 + 0 + 35 \cdot 10 \cdot 20 & = 11375 \end{cases}$$
$$= 7125$$

Fase 4: costruire una soluzione ottima

La procedura MATRIX-CHAIN-ORDER determina il numero ottimo di prodotti scalari richiesti per moltiplicare una sequenza di matrici, ma non mostra direttamente come moltiplicare le matrici. La tabella $s[1\,..\,n, 1\,..\,n]$ ci fornisce le informazioni per farlo. Ogni posizione $s[i, j]$ registra quel valore di k per il quale la parentesizzazione ottima di $A_i A_{i+1} \cdots A_j$ suddivide il prodotto fra A_k e A_{k+1}. Quindi, sappiamo che il prodotto finale delle matrici nel calcolo ottimale di $A_{1..n}$ è $A_{1..s[1,n]} A_{s[1,n]+1..n}$. I prodotti precedenti possono essere calcolati ricorsivamente, perché $s[1, s[1, n]]$ determina l'ultimo prodotto nel calcolo di $A_{1..s[1,n]}$ e $s[s[1, n]+1, n]$ determina l'ultimo prodotto nel calcolo di $A_{s[1,n]+1..n}$. La seguente procedura ricorsiva produce una parentesizzazione ottima di $\langle A_i, A_{i+1}, \ldots, A_j \rangle$, dati gli indici i e j e la tabella s calcolata da MATRIX-CHAIN-ORDER. La chiamata iniziale di PRINT-OPTIMAL-PARENS$(s, 1, n)$ produce una parentesizzazione ottima di $\langle A_1, A_2, \ldots, A_n \rangle$.

PRINT-OPTIMAL-PARENS(s, i, j)

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995303026-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

```
1  if i == j
2      stampa "A"_i
3  else stampa "("
4      PRINT-OPTIMAL-PARENS(s, i, s[i, j])
5      PRINT-OPTIMAL-PARENS(s, s[i, j] + 1, j)
6      stampa ")"
```

Nell'esempio della Figura 15.5, la chiamata PRINT-OPTIMAL-PARENS$(s, 1, 6)$ produce la parentesizzazione $((A_1(A_2A_3))((A_4A_5)A_6))$.

Esercizi

15.2-1

Trovate una parentesizzazione ottima del prodotto di una sequenza di matrici le cui dimensioni sono $\langle 5, 10, 3, 12, 5, 50, 6\rangle$.

15.2-2

Create un algoritmo ricorsivo MATRIX-CHAIN-MULTIPLY(A, s, i, j) che esegue in modo ottimo il prodotto di una sequenza di matrici, dati gli indici i e j, la sequenza delle matrici $\langle A_1, A_2, \ldots, A_n\rangle$ e la tabella s calcolata da MATRIX-CHAIN-ORDER (la chiamata iniziale sarà MATRIX-CHAIN-MULTIPLY$(A, s, 1, n)$).

15.2-3

Utilizzate il metodo di sostituzione per dimostrare che la soluzione della ricorrenza (15.6) è $\Omega(2^n)$.

15.2-4

Descrivete il grafo dei sottoproblemi per la moltiplicazione di una sequenza di matrici con una sequenza di input di lunghezza n. Quanti vertici avrà il grafo? Quanti archi avrà e quali sono questi archi?

15.2-5

Sia $R(i, j)$ il numero di volte che viene fatto riferimento alla posizione $m[i, j]$ della tabella m per calcolare le altre posizioni della tabella in una chiamata di MATRIX-CHAIN-ORDER. Dimostrate che il numero totale di riferimenti per l'intera tabella è il seguente

$$\sum_{i=1}^{n}\sum_{j=i}^{n} R(i, j) = \frac{n^3 - n}{3}$$

(*Suggerimento:* potrebbe essere utile l'equazione (A.3).)

15.2-6

Dimostrate che una parentesizzazione completa di un'espressione di n elementi ha esattamente $n - 1$ coppie di parentesi.

15.3 Elementi della programmazione dinamica

Sebbene abbiamo appena visto due esempi di utilizzo del metodo della programmazione dinamica, tuttavia qualcuno potrebbe chiedersi quando può applicare questo metodo. Da un punto di vista ingegneristico, quando dovremmo cercare di risolvere un problema con la programmazione dinamica? In questo paragrafo, esamineremo i due principali ingredienti che deve avere un problema di ottimizzazione affinché possa essere applicata la programmazione dinamica: la sottostruttura ottima e i sottoproblemi ripetuti. Riesamineremo inoltre in modo più dettagliato come la *tecnica di annotazione* (memoization) può aiutarci a sfruttare la proprietà dei sottoproblemi ripetuti in un approccio ricorsivo top-down.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Sottostruttura ottima

La prima fase del processo di risoluzione di un problema di ottimizzazione mediante la programmazione dinamica consiste nel caratterizzare la struttura di una soluzione ottima. Ricordiamo che un problema presenta una ***sottostruttura ottima*** se una soluzione ottima del problema contiene al suo interno le soluzioni ottime dei sottoproblemi. Quando un problema presenta una sottostruttura ottima, ciò potrebbe essere un buon indizio dell'applicabilità della programmazione dinamica (ma potrebbe anche indicare che è possibile applicare una strategia golosa, come vedremo nel Capitolo 16). Nella programmazione dinamica, costruiamo una soluzione ottima del problema dalle soluzioni ottime dei sottoproblemi. Di conseguenza, dobbiamo essere sicuri che l'insieme dei sottoproblemi considerati includa quelli utilizzati in una soluzione ottima.

Abbiamo identificato una sottostruttura ottima in entrambi i problemi esaminati in questo capitolo. Nel Paragrafo 15.1, abbiamo osservato che il modo migliore di tagliare un'asta di lunghezza n (se effettuiamo dei tagli) implica il taglio ottimale dei due pezzi risultanti dal primo taglio. Nel Paragrafo 15.2, abbiamo osservato che una parentesizzazione ottima di $A_i A_{i+1} \cdots A_j$ che suddivide il prodotto fra A_k e A_{k+1} contiene al suo interno soluzioni ottime dei problemi di parentesizzazione di $A_i A_{i+1} \cdots A_k$ e $A_{k+1} A_{k+2} \cdots A_j$.

Vi accorgerete che, quando si cerca di scoprire la sottostruttura ottima di un problema, si segue sempre uno schema comune:

1. Dimostrate che una soluzione del problema consiste nel fare una scelta (per esempio, scegliere il taglio iniziale di un'asta o un indice in corrispondenza del quale suddividere la sequenza delle matrici). Questa scelta lascia uno o più sottoproblemi da risolvere.
2. Per un dato problema, supponete di conoscere la scelta che porta a una soluzione ottima. Non vi interessa sapere come sia stata determinata questa scelta. Semplicemente, supponete di conoscere tale scelta.
3. Fatta la scelta, determinate quali sottoproblemi ne derivano e quale sia il modo migliore per caratterizzare lo spazio di sottoproblemi risultante.
4. Dimostrate che le soluzioni dei sottoproblemi che avete utilizzato all'interno della soluzione ottima del problema devono essere necessariamente ottime, adottando una tecnica "taglia e incolla". Per fare questo, prima supponete che ciascuna delle soluzioni dei sottoproblemi non sia ottima e, poi, arrivate a una contraddizione. In particolare, "tagliando" la soluzione non ottima di un sottoproblema e "incollando" quella ottima, dimostrate che potete ottenere una soluzione migliore del problema originale, contraddicendo l'ipotesi di avere già una soluzione ottima. Se in una soluzione ottima ci sono più sottoproblemi, normalmente, essi sono così simili che la tecnica "taglia e incolla" applicata a un sottoproblema può essere facilmente adattata anche agli altri sottoproblemi.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Per caratterizzare lo spazio dei sottoproblemi, una buona regola consiste nel cercare di mantenere tale spazio quanto più semplice possibile, per poi espanderlo, se necessario. Per esempio, lo spazio dei sottoproblemi che abbiamo considerato nel problema del taglio delle aste conteneva i problemi dei tagli ottimali di un'asta di lunghezza i per ogni dimensione i. Questo spazio di sottoproblemi ha

funzionato bene e non c'è stato bisogno di provare uno spazio più generale. Supponiamo, invece, di limitare lo spazio dei sottoproblemi per la moltiplicazione di una sequenza di matrici a quei prodotti matriciali della forma $A_1 A_2 \cdots A_j$. Come nel caso esaminato in precedenza, una parentesizzazione ottima deve suddividere questo prodotto fra A_k e A_{k+1} per qualche $1 \leq k < j$. A meno che non garantiamo che k sia sempre uguale a $j-1$, troveremo che i sottoproblemi hanno la forma $A_1 A_2 \cdots A_k$ e $A_{k+1} A_{k+2} \cdots A_j$ e che quest'ultimo sottoproblema non ha la forma $A_1 A_2 \cdots A_j$. Ecco perché, per questo problema, è stato necessario consentire ai sottoproblemi di variare in "entrambe le estremità", ovvero consentire sia i che a j di variare nel sottoproblema $A_i A_{i+1} \cdots A_j$.

La sottostruttura ottima varia in funzione del tipo di problema in due modi:

1. per il numero di sottoproblemi che sono utilizzati in una soluzione ottima del problema originale;
2. per il numero di scelte che possiamo fare per determinare quale sottoproblema (o quali sottoproblemi) utilizzare in una soluzione ottima.

Nel problema del taglio delle aste, una soluzione ottima per tagliare un'asta di dimensione n usa un solo sottoproblema (di dimensione $n - i$), ma dobbiamo considerare n scelte per i per determinare quale di esse fornisce una soluzione ottima. La moltiplicazione di una sequenza di matrici relativa alla sottosequenza $A_i A_{i+1} \cdots A_j$ è un esempio di un caso in cui ci sono due sottoproblemi e $j - i$ scelte. Per una data matrice A_k, in corrispondenza della quale suddividiamo il prodotto, abbiamo due sottoproblemi – parentesizzazione di $A_i A_{i+1} \cdots A_k$ e parentesizzazione di $A_{k+1} A_{k+2} \cdots A_j$ – e dobbiamo trovare le soluzioni ottime per *entrambi* i sottoproblemi. Una volta trovate queste soluzioni, scegliamo l'indice k fra $j - i$ candidati.

Informalmente, il tempo di esecuzione di un algoritmo di programmazione dinamica dipende dal prodotto di due fattori: il numero di sottoproblemi da risolvere complessivamente e il numero di scelte da considerare per ogni sottoproblema. Nel problema del taglio delle aste, avevamo $\Theta(n)$ sottoproblemi complessivamente e al massimo n scelte da esaminare per ogni sottoproblema, con un tempo di esecuzione pari a $O(n^2)$. Nella moltiplicazione di una sequenza di matrici c'erano $\Theta(n^2)$ sottoproblemi complessivamente e per ciascuno di essi avevamo al massimo $n - 1$ scelte, con un tempo di esecuzione pari a $O(n^3)$ (in effetti, il tempo di esecuzione è $\Theta(n^3)$ per l'Esercizio 15.2-5).

Di solito, il grafo dei sottoproblemi fornisce un modo alternativo per eseguire la stessa analisi. Ogni vertice corrisponde a un sottoproblema, e le scelte di un sottoproblema sono gli archi che incidono nel vertice di quel sottoproblema. Ricordiamo che nel taglio delle aste, il grafo dei sottoproblemi aveva n vertici e al massimo n archi per vertice, che generavano un tempo di esecuzione pari a $O(n^2)$. Per la moltiplicazione di una sequenza di matrici, se disegnassimo il grafo dei sottoproblemi, esso avrebbe $\Theta(n^2)$ vertici e ciascun vertice avrebbe un grado al massimo pari a $n - 1$, per un totale di $O(n^3)$ vertici e archi.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199505016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Spesso la programmazione dinamica usa la sottostruttura ottima secondo uno schema *bottom-up* (dal basso verso l'alto); ovvero, prima vengono trovate le soluzioni ottime dei sottoproblemi e, dopo avere risolto i sottoproblemi, viene trovata una soluzione ottima del problema. Trovare una soluzione ottima del problema significa scegliere i sottoproblemi da utilizzare per risolvere il problema. Il costo

della soluzione del problema, di solito, è pari ai costi per risolvere i sottoproblemi più un costo che è direttamente imputabile alla scelta stessa. Per esempio, nel taglio delle aste, prima risolviamo i sottoproblemi per trovare i modi ottimali di tagliare le aste di lunghezza i per $i = 0, 1, \ldots, n-1$, e poi determiniamo quale sottoproblema fornisce una soluzione ottima per un'asta di lunghezza n, utilizzando l'equazione (15.2). Il costo imputabile alla scelta è il termine p_i nell'equazione (15.2). Nella moltiplicazione di una sequenza di matrici, prima determiniamo le parentesizzazioni ottime delle sottosequenze di $A_i A_{i+1} \cdots A_j$; poi scegliamo la matrice A_k, in corrispondenza della quale suddividere il prodotto. Il costo imputabile alla scelta è il termine $p_{i-1} p_k p_j$.

Nel Capitolo 16 esamineremo gli "algoritmi golosi", che hanno molte affinità con la programmazione dinamica. In particolare, i problemi ai quali si applicano gli algoritmi golosi hanno una sottostruttura ottima. Una differenza importante fra gli algoritmi golosi e la programmazione dinamica è che negli algoritmi golosi utilizziamo la sottostruttura ottima secondo uno schema *top-down* (dall'alto verso il basso). Anziché trovare prima le soluzioni ottime dei sottoproblemi e poi fare una scelta, gli algoritmi golosi prima fanno una scelta – quella che sembra ottima in quel momento – e poi risolvono un sottoproblema risultante, senza preoccuparsi di risolvere anche tutti gli altri sottoproblemi più piccoli collegati. Sorprendentemente, talvolta questa strategia funziona!

Finezze

Occorre prestare attenzione a non assumere l'esistenza della sottostruttura ottima anche quando non è possibile farlo. Consideriamo i seguenti due problemi in cui sono dati un grafo orientato $G = (V, A)$ e i vertici $u, v \in V$.

Cammino minimo in un grafo non pesato:[2] trovare un cammino da u a v formato dal minor numero di archi. Tale cammino deve essere semplice, perché eliminando un ciclo da un cammino si ottiene un cammino con un minor numero di archi.

Cammino semplice massimo in un grafo non pesato: trovare un cammino semplice da u a v che è formato dal maggior numero di archi. Dobbiamo richiedere che il cammino sia semplice, perché altrimenti potremmo attraversare un ciclo un numero indefinito di volte per creare cammini con un numero arbitrariamente grande di archi.

Il problema del cammino minimo in un grafo non pesato presenta una sottostruttura ottima, come si può vedere nel modo seguente. Supponiamo che $u \neq v$, in modo che il problema non sia banale; allora qualsiasi cammino p da u a v deve contenere un vertice intermedio w (notate che w può essere u o v). Quindi possiamo scomporre il cammino $u \overset{p}{\rightsquigarrow} v$ nei sottocammini $u \overset{p_1}{\rightsquigarrow} w \overset{p_2}{\rightsquigarrow} v$. Chiaramente, il numero di archi in p è uguale alla somma del numero di archi in p_1 e del numero di archi in p_2. Noi asseriamo che, se p è un cammino ottimo (cioè minimo) da u a v, allora p_1 deve essere un cammino minimo da u a w. Perché? Applichiamo la tecnica "taglia e incolla": se ci fosse un altro cammino, p'_1, da u a w con un

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

[2]Utilizziamo il termine "non pesato" per distinguere questo problema da quello della ricerca dei cammini minimi con archi pesati, che esamineremo nei Capitoli 24 e 25. Il problema del cammino minimo in un grafo non pesato si può risolvere con la tecnica della ricerca in ampiezza (breadth-first search) descritta nel Capitolo 22.

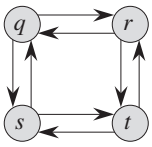


Figura 15.6 Un grafo orientato non pesato per il quale il problema di trovare un cammino semplice massimo non ha una sottostruttura ottima. Il cammino $q \to r \to t$ è un cammino semplice massimo da q a t, ma il sottocammino $q \to r$ non è un cammino semplice massimo da q a r, né il sottocammino $r \to t$ è un cammino semplice massimo da r a t.

numero minore di archi rispetto a p_1, allora potremmo tagliare p_1 e incollare p'_1 per ottenere un cammino $u \overset{p'_1}{\rightsquigarrow} w \overset{p_2}{\rightsquigarrow} v$ cha ha meno archi di p, contraddicendo l'ipotesi che p sia un cammino ottimo. In maniera simmetrica, p_2 deve essere un cammino minimo da w a v. Quindi, possiamo trovare un cammino minimo da u a v considerando tutti i vertici intermedi w, trovando un cammino minimo da u a w e un cammino minimo da w a v, e scegliendo un vertice intermedio w che produce il cammino minimo totale. Nel Paragrafo 25.3 utilizzeremo una variante di questa osservazione della sottostruttura ottima per trovare un cammino minimo fra ogni coppia di vertici in un grafo orientato pesato.

Saremmo tentati di supporre che anche il problema di trovare un cammino semplice più lungo in un grafo non pesato presenti una sottostruttura ottima. Dopo tutto, se scomponiamo un cammino semplice più lungo $u \overset{p}{\rightsquigarrow} v$ nei sottocammini $u \overset{p_1}{\rightsquigarrow} w \overset{p_2}{\rightsquigarrow} v$, perché p_1 non dovrebbe essere un cammino semplice massimo da u a w, e p_2 non dovrebbe essere un cammino semplice massimo da w a v? La risposta è no! La Figura 15.6 illustra un esempio. Consideriamo il cammino $q \to r \to t$, che è un cammino semplice massimo da q a t. Il cammino $q \to r$ è un cammino semplice massimo da q a r? No, perché il cammino $q \to s \to t \to r$ è un cammino semplice più lungo. Il cammino $r \to t$ è un cammino semplice massimo da r a t? Ancora no, perché il cammino $r \to q \to s \to t$ è un cammino semplice più lungo.

Questo esempio dimostra che per i cammini semplici massimi, non soltanto manca una sottostruttura ottima, ma non è possibile assemblare una soluzione "valida" del problema dalle soluzioni dei sottoproblemi. Se combiniamo i cammini semplici massimi $q \to s \to t \to r$ e $r \to q \to s \to t$, otteniamo il cammino $q \to s \to t \to r \to q \to s \to t$, che non è semplice. Il problema di trovare un cammino semplice massimo in un grafo non pesato sembra davvero non avere alcuna sorta di sottostruttura ottima. Non è stato ancora trovato un algoritmo efficiente di programmazione dinamica per questo problema. In effetti, si tratta di un problema NP-completo, che – come vedremo nel Capitolo 34 – significa che è improbabile che possa essere risolto in un tempo polinomiale.

Che cosa rende la sottostruttura di un cammino semplice massimo così diversa da quella di un cammino minimo? Sebbene in entrambi i casi si siano individuati due sottoproblemi nella soluzione di un problema, i sottoproblemi per trovare il cammino semplice massimo non sono ***indipendenti***, mentre lo sono per i cammini minimi. Che cosa significa che i sottoproblemi sono indipendenti? Significa che la soluzione di un sottoproblema non influisce sulla soluzione di un altro sottoproblema dello stesso problema. Nell'esempio della Figura 15.6, per il problema di trovare un cammino semplice massimo da q a t abbiamo due sottoproblemi: trovare i cammini semplici massimi da q a r e da r a t. Per il primo di questi sottoproblemi, scegliamo il cammino $q \to s \to t \to r$; quindi abbiamo utilizzato anche i vertici s e t. Non possiamo più utilizzare questi vertici nel secondo sottoproblema, perché la combinazione delle due soluzioni dei sottoproblemi produrrebbe un cammino che non è semplice. Se non possiamo più utilizzare il vertice t, allora il secondo sottoproblema è irrisolvibile, perché t appartiene al cammino che

Acquistato da Michele Mastrolonardo (2 copie) - Proprietà riservata ai sensi della Legge 633/1941 - Copyright © 2010 The McGraw-Hill Education (Italy)

cerchiamo e non è il vertice in cui si congiungono le soluzioni dei sottoproblemi (questo vertice è r). Aver utilizzato i vertici s e t nella soluzione di un sottoproblema ci impedisce di utilizzarli nella soluzione dell'altro sottoproblema. Tuttavia, dobbiamo utilizzare almeno uno di questi vertici per risolvere l'altro sottoproblema e dobbiamo utilizzarli entrambi per trovare la soluzione ottima di questo sottoproblema. Quindi, diciamo che questi sottoproblemi non sono indipendenti. In altre parole, il nostro utilizzo delle risorse nel risolvere un sottoproblema (le risorse sono i vertici) le ha rese indisponibili per l'altro sottoproblema.

Perché, allora, i sottoproblemi per trovare un cammino minimo sono indipendenti? La risposta è che, per loro natura, i sottoproblemi non condividono le risorse. Noi asseriamo che se un vertice w si trova in un cammino minimo p da u a v, allora possiamo congiungere *qualsiasi* cammino minimo $u \overset{p_1}{\rightsquigarrow} w$ con *qualsiasi* cammino minimo $w \overset{p_2}{\rightsquigarrow} v$ per produrre un cammino minimo da u a v. Abbiamo la certezza che, tranne w, nessun vertice può apparire in entrambi i percorsi p_1 e p_2. Perché? Supponiamo che qualche vertice $x \neq w$ appaia in entrambi i cammini p_1 e p_2, in modo che possiamo scomporre p_1 in $u \overset{p_{ux}}{\rightsquigarrow} x \rightsquigarrow w$ e p_2 in $w \rightsquigarrow x \overset{p_{xv}}{\rightsquigarrow} v$. Per la sottostruttura ottima di questo problema, il cammino p ha tanti archi quanti ne hanno i due cammini p_1 e p_2 assieme; diciamo che p ha e archi. Adesso costruiamo un cammino $p' = u \overset{p_{ux}}{\rightsquigarrow} x \overset{p_{xv}}{\rightsquigarrow} v$ da u a v. Siccome abbiamo tolto i cammini da x a w e da w a x, ciascuno dei quali contiene almeno un arco, il cammino p' ha al massimo $e - 2$ archi, che contraddice l'ipotesi che p sia una cammino minimo. Quindi, abbiamo la certezza che i sottoproblemi per il problema del cammino minimo sono indipendenti.

Entrambi i problemi esaminati nei Paragrafi 15.1 e 15.2 hanno sottoproblemi indipendenti. Nella moltiplicazione di una sequenza di matrici, i sottoproblemi sono i prodotti delle sottosequenze $A_i A_{i+1} \cdots A_k$ e $A_{k+1} A_{k+2} \cdots A_j$. Queste sottosequenze sono disgiunte, quindi nessuna matrice potrebbe essere inclusa in entrambe. Nel problema del taglio delle aste, per trovare il modo migliore di tagliare un'asta di lunghezza n, abbiamo esaminato i modi migliori di tagliare aste di lunghezza i per $i = 0, 1, \ldots, n - 1$. Poiché una soluzione ottima per un'asta di lunghezza n include soltanto una delle soluzioni dei sottoproblemi (quelli che rimangono dopo il taglio del primo pezzo), l'indipendenza dei sottoproblemi è evidente.

Ripetizione dei sottoproblemi

Il secondo ingrediente che un problema di ottimizzazione deve avere affinché la programmazione dinamica possa essere applicata è che lo spazio dei sottoproblemi deve essere "piccolo", nel senso che un algoritmo ricorsivo per il problema risolve ripetutamente gli stessi sottoproblemi, anziché generare sempre nuovi sottoproblemi. Tipicamente, il numero totale di sottoproblemi distinti è un polinomio nella dimensione dell'input. Quando un algoritmo ricorsivo rivisita più volte lo stesso problema, diciamo che il problema di ottimizzazione ha dei ***sottoproblemi ripetuti***.[3] D'altra parte, un problema per il quale sia appropriato un approccio

Acquistato da Michele Michele su Webster il 2022-07-07 23:12, Numero Ordine Libreria: 1995036016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[3] Potrebbe sembrare strano che la programmazione dinamica si affidi a sottoproblemi che sono indipendenti e ripetuti. Sebbene questi requisiti possano apparire contraddittori, tuttavia esprimono due concetti differenti. Due sottoproblemi dello stesso problema sono indipendenti se non condividono le stesse risorse. Due sottoproblemi sono ripetuti se sono esattamente lo stesso sottoproblema che si ripresenta come sottoproblema di problemi differenti.

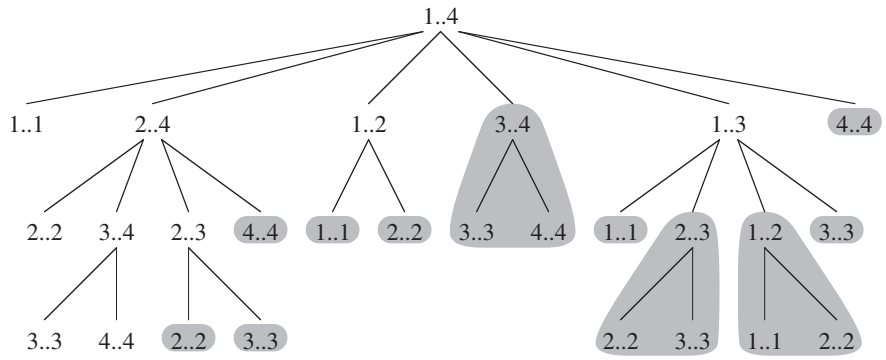


Figura 15.7 L'albero di ricorsione per il calcolo di RECURSIVE-MATRIX-CHAIN$(p, 1, 4)$. Ogni nodo contiene i parametri i e j. I calcoli svolti in un sottoalbero con sfondo grigio sono sostituiti da una singola ricerca in tabella nella chiamata MEMOIZED-MATRIX-CHAIN.

divide et impera, dovrebbe generare problemi nuovi di zecca a ogni passo della ricorsione. Gli algoritmi di programmazione dinamica tipicamente sfruttano i sottoproblemi ripetuti risolvendo ciascun sottoproblema una sola volta e, poi, memorizzando la soluzione in una tabella da cui può essere recuperata quando serve, impiegando un tempo costante per la ricerca.

Nel Paragrafo 15.1 abbiamo visto che una soluzione ricorsiva per il taglio delle aste effettua un numero esponenziale di chiamate ricorsive per trovare le soluzioni dei problemi più piccoli. La nostra soluzione tabulare riduce il tempo esponenziale di un algoritmo ricorsivo a un tempo lineare.

Per illustrare più dettagliatamente la proprietà dei sottoproblemi ripetuti, riesaminiamo il problema della moltiplicazione di una sequenza di matrici. Facendo riferimento alla Figura 15.5, notiamo che MATRIX-CHAIN-ORDER cerca ripetutamente la soluzione dei sottoproblemi nelle righe inferiori quando risolve i sottoproblemi nelle righe superiori. Per esempio, la posizione $m[3, 4]$ viene usata 4 volte: durante il calcolo di $m[2, 4]$, $m[1, 4]$, $m[3, 5]$ e $m[3, 6]$. Se il valore di $m[3, 4]$ fosse ricalcolato ogni volta, anziché soltanto cercato, il tempo di esecuzione aumenterebbe enormemente. Per vedere questo, consideriamo la seguente procedura ricorsiva (inefficiente) che calcola $m[i, j]$, il numero minimo di prodotti scalari richiesti per calcolare il prodotto di una sequenza di matrici $A_{i..j} = A_i A_{i+1} \cdots A_j$. La procedura si basa direttamente sulla ricorrenza (15.7).

```
RECURSIVE-MATRIX-CHAIN(p, i, j)
1  if i == j
2      return 0
3  m[i, j] = ∞
4  for k = i to j − 1
5      q = RECURSIVE-MATRIX-CHAIN(p, i, k)
             + RECURSIVE-MATRIX-CHAIN(p, k + 1, j)
             + p_{i−1} p_k p_j
6      if q < m[i, j]
7          m[i, j] = q
8  return m[i, j]
```

Acquistato da Michele Michele su Websterit 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

La Figura 15.7 illustra l'albero di ricorsione prodotto dalla chiamata RECURSIVE-MATRIX-CHAIN$(p, 1, 4)$. Ogni nodo è etichettato dai valori dei parametri i e j. Notate che alcune coppie di valori si presentano più volte.

In effetti, possiamo dimostrare che il tempo per calcolare $m[1, n]$ con questa procedura ricorsiva è almeno esponenziale in n. Sia $T(n)$ il tempo impiegato

da RECURSIVE-MATRIX-CHAIN per calcolare una parentesizzazione ottima di una sequenza di n matrici. Se supponiamo che per ogni esecuzione delle righe 1–2 e delle righe 6–7 occorre almeno un'unità di tempo, allora l'ispezione della procedura genera la ricorrenza

$$\begin{aligned} T(1) &\geq 1 \\ T(n) &\geq 1 + \sum_{k=1}^{n-1} (T(k) + T(n-k) + 1) \qquad \text{per } n > 1 \end{aligned}$$

Notate che per $i = 1, 2, \ldots, n-1$, ogni termine $T(i)$ appare una volta come $T(k)$ e una volta come $T(n-k)$; raccogliendo gli $n-1$ addendi uguali a 1 nella sommatoria insieme con l'1 iniziale, possiamo riscrivere la ricorrenza così

$$T(n) \geq 2 \sum_{i=1}^{n-1} T(i) + n \tag{15.8}$$

Dimostreremo che $T(n) = \Omega(2^n)$ utilizzando il metodo di sostituzione. Più precisamente, dimostreremo che $T(n) \geq 2^{n-1}$ per ogni $n \geq 1$. La base è semplice, in quanto $T(1) \geq 1 = 2^0$. Induttivamente, per $n \geq 2$ abbiamo

$$\begin{aligned} T(n) &\geq 2 \sum_{i=1}^{n-1} 2^{i-1} + n \\ &= 2 \sum_{i=0}^{n-2} 2^i + n \\ &= 2(2^{n-1} - 1) + n \\ &= (2^n - 2) + n \\ &\geq 2^{n-1} \end{aligned}$$

che completa la dimostrazione. Quindi, la quantità totale di lavoro svolto dalla chiamata RECURSIVE-MATRIX-CHAIN$(p, 1, n)$ è almeno esponenziale in n.

Confrontate questo algoritmo ricorsivo top-down (senza annotazione) con l'algoritmo bottom-up di programmazione dinamica. Quest'ultimo è più efficiente perché sfrutta la proprietà dei sottoproblemi ripetuti. Ci sono soltanto $\Theta(n^2)$ sottoproblemi differenti e l'algoritmo di programmazione dinamica risolve ciascun sottoproblema una sola volta. L'algoritmo ricorsivo, invece, deve risolvere ripetutamente ciascun sottoproblema, ogni volta che un sottoproblema si ripresenta nell'albero di ricorsione. Quando un albero di ricorsione per la soluzione ricorsiva naturale di un problema contiene più volte lo stesso sottoproblema e il numero totale di sottoproblemi differenti è piccolo, la programmazione dinamica può migliorare l'efficienza e, talvolta, in modo eclatante.

Ricostruire una soluzione ottima

Spesso memorizziamo in una tabella la scelta fatta in ciascun sottoproblema, per evitare di dovere ricostruire questa informazione dai costi che abbiamo memorizzato.

Acquistato da Michele Michele su Webster il 2022-[illegible]

Per la moltiplicazione di una sequenza di matrici, la tabella $s[i, j]$ permette di risparmiare una notevole quantità di lavoro per ricostruire una soluzione ottima. Supponiamo di non avere utilizzato la tabella $s[i, j]$, avendo riempito soltanto la tabella $m[i, j]$ che contiene i costi ottimi dei sottoproblemi. Ci sono $j - i$ scelte per determinare quali sottoproblemi utilizzare in una soluzione ottima per parentesizzare $A_i A_{i+1} \cdots A_j$; $j - i$ non è una costante. Quindi, occorrerebbe il tempo

$\Theta(j-i) = \omega(1)$ per ricostruire i sottoproblemi scelti per una soluzione di un determinato problema. Memorizzando in $s[i, j]$ l'indice della matrice in corrispondenza della quale suddividiamo il prodotto $A_i A_{i+1} \cdots A_j$, possiamo ricostruire ciascuna scelta nel tempo $O(1)$.

Annotazione

Come abbiamo visto per il problema del taglio delle aste, vi è un approccio alternativo alla programmazione dinamica che, pur conservando la strategia top-down, spesso offre la stessa efficienza dell'usuale approccio bottom-up alla programmazione dinamica. Il concetto che sta alla base di questa tecnica consiste nel *dotare di un blocco note* il naturale, ma inefficiente, algoritmo ricorsivo. Come nell'approccio bottom-up, viene utilizzata una tabella con le soluzioni dei sottoproblemi, ma la struttura di controllo per riempire la tabella è più simile all'algoritmo ricorsivo.

Un algoritmo ricorsivo con annotazione utilizza una posizione di una tabella (il blocco note) per la soluzione di ciascun sottoproblema. Ogni posizione della tabella inizialmente contiene un valore speciale per indicare che la posizione non è stata ancora riempita. La prima volta che si presenta il sottoproblema durante l'esecuzione dell'algoritmo ricorsivo, viene calcolata la soluzione del sottoproblema che, poi, viene annotata nella tabella. Successivamente, ogni volta che si ripresenta questo sottoproblema, l'algoritmo ricerca e restituisce il corrispondente valore memorizzato nella tabella.[4] Riportiamo qui di seguito una versione con annotazione di RECURSIVE-MATRIX-CHAIN. Notate come essa assomigli alla versione con annotazione per il problema del taglio delle aste.

```
MEMOIZED-MATRIX-CHAIN(p)
1  n = p.length − 1
2  Sia m[1..n, 1..n] una nuova tabella
3  for i = 1 to n
4      for j = i to n
5          m[i, j] = ∞
6  return LOOKUP-CHAIN(m, p, 1, n)

LOOKUP-CHAIN(m, p, i, j)
1  if m[i, j] < ∞
2      return m[i, j]
3  if i == j
4      m[i, j] = 0
5  else for k = i to j − 1
6          q = LOOKUP-CHAIN(m, p, i, k)
                 + LOOKUP-CHAIN(m, p, k + 1, j) + p_{i−1} p_k p_j
7          if q < m[i, j]
8              m[i, j] = q
9  return m[i, j]
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[4] Questo approccio presuppone che sia noto l'insieme di tutti i parametri dei sottoproblemi e che sia definita la relazione fra le posizioni della tabella e i sottoproblemi. Un altro approccio consiste nell'annotare le soluzioni utilizzando l'hashing con i parametri dei sottoproblemi come chiavi.

MEMOIZED-MATRIX-CHAIN, come MATRIX-CHAIN-ORDER, utilizza una tabella $m[1\,..\,n, 1\,..\,n]$ per i valori calcolati di $m[i, j]$, il numero minimo di prodotti scalari richiesti per calcolare la matrice $A_{i..j}$. Inizialmente, ogni posizione della tabella contiene il valore ∞ per indicare che non è stato ancora inserito un valore in tale posizione. Quando viene eseguita la chiamata LOOKUP-CHAIN(p, i, j), se $m[i, j] < \infty$ nella riga 1, la procedura restituisce semplicemente il costo precedentemente calcolato $m[i, j]$ (riga 2). Altrimenti, il costo viene calcolato come in RECURSIVE-MATRIX-CHAIN, memorizzato in $m[i, j]$ e poi restituito. Quindi, LOOKUP-CHAIN(p, i, j) restituisce sempre il valore di $m[i, j]$, ma lo calcola soltanto se è la prima volta che viene chiamata la procedura LOOKUP-CHAIN con i parametri i e j.

La Figura 15.7 illustra come la procedura MEMOIZED-MATRIX-CHAIN consente di risparmiare tempo rispetto a RECURSIVE-MATRIX-CHAIN. I sottoalberi con sfondo grigio rappresentano i valori che vengono cercati, anziché calcolati.

Come l'algoritmo bottom-up di programmazione dinamica MATRIX-CHAIN-ORDER, anche la procedura MEMOIZED-MATRIX-CHAIN viene eseguita nel tempo $O(n^3)$. La riga 5 di MEMOIZED-MATRIX-CHAIN viene eseguita $\Theta(n^2)$ volte. Possiamo classificare le chiamate di LOOKUP-CHAIN in due tipi:

1. le chiamate in cui $m[i, j] = \infty$; vengono eseguite le righe 3–9.
2. le chiamate in cui $m[i, j] < \infty$; LOOKUP-CHAIN termina nella riga 2.

Ci sono $\Theta(n^2)$ chiamate del primo tipo, una per ogni posizione della tabella. Tutte le chiamate del secondo tipo sono fatte come chiamate ricorsive da chiamate del primo tipo. Ogni volta che una chiamata di LOOKUP-CHAIN fa delle chiamate ricorsive, ne fa $O(n)$. Quindi, in totale ci sono $O(n^3)$ chiamate del secondo tipo. Ogni chiamata del secondo tipo impiega un tempo $O(1)$; ogni chiamata del primo tipo impiega un tempo $O(n)$ più il tempo speso nelle sue chiamate ricorsive. Il tempo totale è, dunque, $O(n^3)$. In conclusione, la tecnica di annotazione trasforma un algoritmo con tempo $\Omega(2^n)$ in un algoritmo con tempo $O(n^3)$.

In sintesi, il problema della moltiplicazione di una sequenza di matrici può essere risolto nel tempo $O(n^3)$ sia da un algoritmo top-down con annotazione sia da un algoritmo bottom-up di programmazione dinamica. Entrambi i metodi sfruttano la proprietà dei sottoproblemi ripetuti. Ci sono soltanto $\Theta(n^2)$ sottoproblemi differenti in totale e ciascuno di questi metodi calcola la soluzione di ogni sottoproblema una sola volta. Senza il processo di annotazione, l'algoritmo ricorsivo naturale viene eseguito in un tempo esponenziale, perché i sottoproblemi da risolvere vengono ripetutamente risolti.

In generale, se tutti i sottoproblemi devono essere risolti almeno una volta, un algoritmo bottom-up di programmazione dinamica, di solito, supera le prestazioni di un algoritmo top-down con annotazione per un fattore costante, perché non ci sono i costi per la ricorsione e i costi per la gestione della tabella sono minori. Inoltre, ci sono problemi per i quali è possibile sfruttare alcune regolarità negli accessi alla tabella nell'algoritmo di programmazione dinamica per ridurre ulteriormente le esigenze di tempo o spazio. In alternativa, se alcuni sottoproblemi nello spazio dei sottoproblemi non richiedono affatto di essere risolti, la soluzione con annotazione ha il vantaggio di risolvere soltanto quei sottoproblemi che devono essere sicuramente risolti.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria 1993000010222703 © Copyright © 2022, McGraw-Hill Education (Italy)

Esercizi

15.3-1

Qual è il metodo più efficiente per determinare il numero ottimo di prodotti matriciali nel problema della moltiplicazione di una sequenza di matrici: enumerare tutti gli schemi di parentesizzazione della moltiplicazione e calcolare il numero di prodotti per ogni schema oppure eseguire RECURSIVE-MATRIX-CHAIN? Spiegate la vostra risposta.

15.3-2

Disegnate l'albero di ricorsione per la procedura MERGE-SORT descritta nel Paragrafo 2.3.1 per un array di 16 elementi. Spiegate perché la tecnica di annotazione non è efficace per accelerare un buon algoritmo divide et impera come MERGE-SORT.

15.3-3

Considerate una variante del problema della moltiplicazione di una sequenza di matrici in cui l'obiettivo è la parentesizzazione della sequenza delle matrici per massimizzare, anziché minimizzare, il numero dei prodotti scalari. Questo problema presenta una sottostruttura ottima?

15.3-4

Come detto in precedenza, nella programmazione dinamica prima risolviamo i sottoproblemi e poi scegliamo quali sottoproblemi utilizzare in una soluzione ottima del problema. Il professor Capulet sostiene che non sempre è necessario risolvere tutti i sottoproblemi per trovare una soluzione ottima; ritiene che una soluzione ottima del problema della moltiplicazione di una sequenza di matrici può essere trovata scegliendo, *prima* di risolvere i sottoproblemi, la matrice A_k in corrispondenza della quale suddividere il sottoprodotto $A_i A_{i+1} \cdots A_j$ (scegliendo k per minimizzare la quantità $p_{i-1} p_k p_j$). Trovare un'istanza del problema della moltiplicazione di una sequenza di matrici per la quale questo approccio goloso produce una soluzione non ottima.

15.3-5

Supponete che nel problema del taglio delle aste, descritto nel Paragrafo 15.1, ci sia anche un limite l_i sul numero di pezzi di lunghezza i che è possibile produrre, per $i = 1, 2, \ldots, n$. Dimostrate che la proprietà della sottostruttura ottima descritta nel Paragrafo 15.1 non è più valida.

15.3-6

Immaginate di voler cambiare una valuta con un'altra. Vi accorgete che, anziché cambiare direttamente la vostra valuta con l'altra, potrebbe essere più conveniente fare una serie di cambi intermedi con altre valute fino ad arrivare alla valuta desiderata. Supponete di poter fare il cambio tra n differenti valute, numerate con $1, 2, \ldots, n$, dove 1 rappresenta la vostra valuta iniziale ed n quella finale. Per ogni coppia di valute i e j, conoscete il tasso di cambio r_{ij}; questo significa che, se partite con d unità della valuta i, potete scambiarle con $d r_{ij}$ unità della valuta j. Una sequenza di cambi potrebbe richiedere una commissione, il cui valore dipende dal numero di cambi effettuati. Sia c_k la commissione che viene applicata quando effettuate k cambi. Dimostrate che, se $c_k = 0$ per ogni $k = 1, 2, \ldots, n$, allora il problema di trovare la migliore sequenza di cambi dalla valuta 1 alla valuta n presenta una sottostruttura ottima. Dimostrate inoltre che se le commissioni c_k hanno valori arbitrari, il problema di trovare la migliore sequenza di cambi dalla valuta 1 alla valuta n non necessariamente presenta una sottostruttura ottima.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503916-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

15.4 La più lunga sottosequenza comune (LCS)

Nelle applicazioni biologiche spesso si confronta il DNA di due (o più) organismi differenti. Una sequenza di DNA è costituita da una stringa di molecole chiamate ***basi***; le possibili basi sono l'adenina, la citosina, la guanina e la timina. Rappresentando ciascuna di queste basi con le loro lettere iniziali, una sequenza di DNA può essere rappresentata con una stringa su un insieme finito {A, C, G, T} (la stringa è definita nell'Appendice C). Per esempio, il DNA di un organismo potrebbe essere $S_1 =$ ACCGGTCGAGTGCGCGGAAGCCGGCCGAA, mentre il DNA di un altro organismo potrebbe essere $S_2 =$ GTCGTTCGGAATGCCGTTGCTCTGTAAA. Una ragione per confrontare due sequenze di DNA è determinare quanto "simili" siano le due sequenze, come misura di quanto siano strettamente correlati i due organismi. La somiglianza può ed è definita in vari modi. Per esempio, potremmo dire che due sequenze di DNA si somigliano se una è una sottostringa dell'altra (il Capitolo 32 descrive gli algoritmi che risolvono questo problema). Nell'esempio in esame, S_1 non è una sottostringa di S_2 né S_2 è una sottostringa di S_1. In alternativa, potremmo dire che due molecole di DNA si somigliano se il numero di modifiche richieste per trasformare una molecola nell'altra è piccolo (vedere il Problema 15-5). Un altro modo per misurare la somiglianza delle stringhe S_1 e S_2 consiste nel trovare una terza stringa S_3 le cui basi si presentano in ciascuna delle stringhe S_1 e S_2; queste basi devono presentarsi nello stesso ordine, senza essere necessariamente consecutive. Quanto più lunga possiamo trovare la stringa S_3, tanto più S_1 ed S_2 si somigliano. Nel nostro esempio, la più lunga stringa S_3 è GTCGTCGGAAGCCGGCCGAA.

Formalizziamo quest'ultimo concetto di somiglianza come il problema della più lunga sottosequenza comune. Una sottosequenza di una data sequenza è la sequenza stessa alla quale sono stati tolti zero o più elementi. Formalmente, data una sequenza $X = \langle x_1, x_2, \ldots, x_m \rangle$, un'altra sequenza $Z = \langle z_1, z_2, \ldots, z_k \rangle$ è una ***sottosequenza*** di X se esiste una sequenza strettamente crescente $\langle i_1, i_2, \ldots, i_k \rangle$ di indici di X tale che per ogni $j = 1, 2, \ldots, k$, si ha $x_{i_j} = z_j$. Per esempio, $Z = \langle B, C, D, B \rangle$ è una sottosequenza di $X = \langle A, B, C, B, D, A, B \rangle$ con la corrispondente sequenza di indici $\langle 2, 3, 5, 7 \rangle$.

Date due sequenze X e Y, diciamo che una sequenza Z è una ***sottosequenza comune*** di X e Y se Z è una sottosequenza di entrambe le sequenze X e Y. Per esempio, se $X = \langle A, B, C, B, D, A, B \rangle$ e $Y = \langle B, D, C, A, B, A \rangle$, la sequenza $\langle B, C, A \rangle$ è una sottosequenza comune di X e Y. Tuttavia, la sequenza $\langle B, C, A \rangle$ non è una *più lunga* sottosequenza comune (Longest Common Subsequence o LCS) di X e Y, perché ha lunghezza 3 e la sequenza $\langle B, C, B, A \rangle$, che è anche comune a X e Y, ha lunghezza 4. La sequenza $\langle B, C, B, A \rangle$ è una LCS di X e Y, come pure la sequenza $\langle B, D, A, B \rangle$, perché non esiste una sottosequenza comune di lunghezza 5 o più.

Nel ***problema della più lunga sottosequenza comune*** sono date due sequenze $X = \langle x_1, x_2, \ldots, x_m \rangle$ e $Y = \langle y_1, y_2, \ldots, y_n \rangle$ e si vuole trovare una sottosequenza di lunghezza massima che è comune a X e Y. Questo paragrafo dimostra che il problema della LCS può essere risolto in modo efficiente applicando la programmazione dinamica.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022. McGraw-Hill Education (Italy)

Fase 1: caratterizzare la più lunga sottosequenza comune

Una tecnica a forza bruta per risolvere il problema della più lunga sottosequenza comune consiste nell'enumerare tutte le sottosequenze di X e controllare le singole sottosequenze per vedere se sono anche sottosequenze di Y, tenendo traccia della più lunga sottosequenza trovata. Ogni sottosequenza di X corrisponde a un sottoinsieme degli indici $\{1, 2, \ldots, m\}$ di X. Ci sono 2^m sottosequenze di X, quindi questo approccio richiede un tempo esponenziale, il che lo rende inutilizzabile per le sequenze lunghe.

Tuttavia, il problema della LCS gode della proprietà della sottostruttura ottima, come dimostra il seguente teorema. Come vedremo, le classi naturali di sottoproblemi corrispondono a coppie di "prefissi" delle due sequenze di input. Più precisamente, data una sequenza $X = \langle x_1, x_2, \ldots, x_m \rangle$, definiamo $X_i = \langle x_1, x_2, \ldots, x_i \rangle$ l'i-esimo ***prefisso*** di X, per $i = 0, 1, \ldots, m$. Per esempio, se $X = \langle A, B, C, B, D, A, B \rangle$, allora $X_4 = \langle A, B, C, B \rangle$ e X_0 è la sequenza vuota.

Teorema 15.1 (Sottostruttura ottima di una LCS)
Siano $X = \langle x_1, x_2, \ldots, x_m \rangle$ e $Y = \langle y_1, y_2, \ldots, y_n \rangle$ le sequenze; sia $Z = \langle z_1, z_2, \ldots, z_k \rangle$ una qualsiasi LCS di X e Y.

1. Se $x_m = y_n$, allora $z_k = x_m = y_n$ e Z_{k-1} è una LCS di X_{m-1} e Y_{n-1}.
2. Se $x_m \neq y_n$, allora $z_k \neq x_m$ implica che Z è una LCS di X_{m-1} e Y.
3. Se $x_m \neq y_n$, allora $z_k \neq y_n$ implica che Z è una LCS di X e Y_{n-1}.

Dimostrazione (1) Se $z_k \neq x_m$, allora potremmo accodare $x_m = y_n$ a Z per ottenere una sottosequenza comune di X e Y di lunghezza $k + 1$, contraddicendo l'ipotesi che Z sia una *più lunga* sottosequenza comune di X e Y. Quindi, deve essere $z_k = x_m = y_n$. Ora, il prefisso Z_{k-1} è una sottosequenza comune di X_{m-1} e Y_{n-1} di lunghezza $k - 1$. Vogliamo dimostrare che questo prefisso è una LCS. Supponiamo per assurdo che ci sia una sottosequenza comune W di X_{m-1} e Y_{n-1} di lunghezza maggiore di $k - 1$. Allora, accodando $x_m = y_n$ a W si ottiene una sottosequenza comune di X e Y la cui lunghezza è maggiore di k, che è una contraddizione.

(2) Se $z_k \neq x_m$, allora Z è una sottosequenza comune di X_{m-1} e Y. Se esistesse una sottosequenza comune W di X_{m-1} e Y di lunghezza maggiore di k, allora W sarebbe anche una sottosequenza comune di X_m e Y, contraddicendo l'ipotesi che Z sia una LCS di X e Y.

(3) La dimostrazione è simmetrica a quella del punto (2). ■

La caratterizzazione del Teorema 15.1 dimostra che una LCS di due sequenze contiene al suo interno una LCS di prefissi delle due sequenze. Quindi, il problema della più lunga sottosequenza comune gode della proprietà della sottostruttura ottima. Una soluzione ricorsiva gode anche della proprietà dei sottoproblemi ripetuti, come vedremo qui di seguito.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503015-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Fase 2: una soluzione ricorsiva

Il Teorema 15.1 implica che ci sono uno o due sottoproblemi da esaminare per trovare una LCS di $X = \langle x_1, x_2, \ldots, x_m \rangle$ e $Y = \langle y_1, y_2, \ldots, y_n \rangle$. Se $x_m = y_n$, dobbiamo trovare una LCS di X_{m-1} e Y_{n-1}. Accodando $x_m = y_n$ a questa LCS,

si ottiene una LCS di X e Y. Se $x_m \neq y_n$, allora dobbiamo risolvere due sottoproblemi: trovare una LCS di X_{m-1} e Y e trovare una LCS di X e Y_{n-1}. La più lunga di queste due LCS è una LCS di X e Y. Poiché questi casi esauriscono tutte le possibilità, sappiamo che all'interno di una LCS di X e Y deve essere utilizzata una delle soluzioni ottime dei sottoproblemi.

Possiamo verificare facilmente la proprietà dei sottoproblemi ripetuti nel problema della più lunga sottosequenza comune. Per trovare una LCS di X e Y, potrebbe essere necessario trovare sia una LCS di X e Y_{n-1} sia una LCS di X_{m-1} e Y. Tuttavia, ciascuno di questi sottoproblemi ha il sottosottoproblema di trovare la LCS di X_{m-1} e Y_{n-1}. Molti altri sottoproblemi condividono sottosottoproblemi.

Come nel problema della moltiplicazione di una sequenza di matrici, la nostra soluzione ricorsiva del problema della più lunga sottosequenza comune richiede la definizione di una ricorrenza per il valore di una soluzione ottima. Definiamo $c[i, j]$ come la lunghezza di una LCS delle sequenze X_i e Y_j. Se $i = 0$ o $j = 0$, una delle sequenze ha lunghezza 0, quindi la LCS ha lunghezza 0. La sottostruttura ottima del problema della LCS consente di scrivere la formula ricorsiva

$$c[i, j] = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0 \\ c[i-1, j-1] + 1 & \text{se } i, j > 0 \text{ e } x_i = y_j \\ \max(c[i, j-1], c[i-1, j]) & \text{se } i, j > 0 \text{ e } x_i \neq y_j \end{cases} \tag{15.9}$$

È importante notare che in questa formulazione ricorsiva una condizione del problema riduce il numero di sottoproblemi che possiamo considerare. Quando $x_i = y_j$, possiamo e dobbiamo considerare soltanto il sottoproblema di trovare la LCS di X_{i-1} e Y_{j-1}. Altrimenti, dobbiamo considerare soltanto i due sottoproblemi di trovare la LCS di X_i e Y_{j-1} e la LCS di X_{i-1} e Y_j. Nei precedenti algoritmi di programmazione dinamica che abbiamo esaminato – per il taglio delle aste e per la moltiplicazione di una sequenza di matrici – nessun sottoproblema è stato escluso a causa delle condizioni del problema. Trovare la LCS non è l'unico algoritmo di programmazione dinamica che esclude dei sottoproblemi in base alle condizioni del problema. Per esempio, anche il problema della distanza di editing (vedere il Problema 15-5) ha questa caratteristica.

Fase 3: calcolare la lunghezza di una LCS

Utilizzando l'equazione (15.9) potremmo scrivere facilmente un algoritmo ricorsivo con tempo esponenziale per calcolare la lunghezza di una LCS di due sequenze. Tuttavia, poiché ci sono soltanto $\Theta(mn)$ sottoproblemi distinti, possiamo utilizzare la programmazione dinamica per calcolare le soluzioni con un metodo bottom-up. La procedura LCS-LENGTH riceve come input due sequenze $X = \langle x_1, x_2, \ldots, x_m \rangle$ e $Y = \langle y_1, y_2, \ldots, y_n \rangle$ e memorizza i valori $c[i, j]$ in una tabella $c[0..m, 0..n]$, le cui posizioni sono calcolate secondo l'ordine delle righe (cioè, vengono inseriti i valori nella prima riga di c da sinistra a destra, poi vengono inseriti i valori nella seconda riga e così via).

Acquistato da Michele Michele su Webster il 2022-0[illegible] Copyright 2022, McGraw-Hill Education (Italy)

La procedura utilizza anche la tabella $b[1..m, 1..n]$ per semplificare la costruzione di una soluzione ottima. Intuitivamente, $b[i, j]$ punta alla posizione della tabella che corrisponde alla soluzione ottima del sottoproblema che è stata scelta per calcolare $c[i, j]$. La procedura restituisce le tabelle b e c; la posizione $c[m, n]$ contiene la lunghezza di una LCS di X e Y.

```
LCS-LENGTH(X, Y)
 1  m = X.length
 2  n = Y.length
 3  Siano b[1..m, 1..n] e c[0..m, 0..n] due nuove tabelle
 4  for i = 1 to m
 5      c[i, 0] = 0
 6  for j = 0 to n
 7      c[0, j] = 0
 8  for i = 1 to m
 9      for j = 1 to n
10          if x_i == y_j
11              c[i, j] = c[i − 1, j − 1] + 1
12              b[i, j] = "↖"
13          elseif c[i − 1, j] ≥ c[i, j − 1]
14              c[i, j] = c[i − 1, j]
15              b[i, j] = "↑"
16          else c[i, j] = c[i, j − 1]
17              b[i, j] = "←"
18  return c e b
```

La Figura 15.8 illustra le tabelle prodotte da LCS-LENGTH con le sequenze $X = \langle A, B, C, B, D, A, B\rangle$ e $Y = \langle B, D, C, A, B, A\rangle$. Il tempo di esecuzione della procedura è $\Theta(mn)$, perché il calcolo di ogni posizione della tabella richiede un tempo $\Theta(1)$.

Fase 4: costruire una LCS

La tabella b restituita dalla procedura LCS-LENGTH può essere utilizzata per costruire rapidamente una LCS delle sequenze $X = \langle x_1, x_2, \ldots, x_m\rangle$ e $Y = \langle y_1, y_2, \ldots, y_n\rangle$. Iniziamo semplicemente da $b[m, n]$ e attraversiamo la tabella seguendo le frecce. Ogni volta che incontriamo una freccia "↖" nella posizione $b[i, j]$, significa che $x_i = y_j$ è un elemento della LCS trovata da LCS-LENGTH. In questo modo gli elementi della LCS si incontrano in ordine inverso. La seguente procedura ricorsiva stampa una LCS di X e Y nell'ordine corretto. La chiamata iniziale è PRINT-LCS$(b, X, X.length, Y.length)$.

```
PRINT-LCS(b, X, i, j)
1  if i == 0 or j == 0
2      return
3  if b[i, j] == "↖"
4      PRINT-LCS(b, X, i − 1, j − 1)
5      print x_i
6  elseif b[i, j] == "↑"
7      PRINT-LCS(b, X, i − 1, j)
8  else PRINT-LCS(b, X, i, j − 1)
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Per la tabella b illustrata nella Figura 15.8, questa procedura stampa $BCBA$. La procedura impiega un tempo $O(m + n)$, perché a ogni chiamata ricorsiva essa decrementa almeno uno dei valori i e j.

i		j	0	1	2	3	4	5	6
		y_j	B	D	C	A	B	A
0	x_i		0	0	0	0	0	0	0
1	A		0	↑ 0	↑ 0	↑ 0	↖ 1	← 1	↖ 1
2	B		0	↖ 1	← 1	← 1	↑ 1	↖ 2	← 2
3	C		0	↑ 1	↑ 1	↖ 2	← 2	↑ 2	↑ 2
4	B		0	↖ 1	↑ 1	↑ 2	↑ 2	↖ 3	← 3
5	D		0	↑ 1	↖ 2	↑ 2	↑ 2	↑ 3	↑ 3
6	A		0	↑ 1	↑ 2	↑ 2	↖ 3	↑ 3	↖ 4
7	B		0	↖ 1	↑ 2	↑ 2	↑ 3	↖ 4	↑ 4

Figura 15.8 Le tabelle c e b calcolate da LCS-LENGTH con le sequenze $X = \langle A, B, C, B, D, A, B\rangle$ e $Y = \langle B, D, C, A, B, A\rangle$. La casella nella riga i e colonna j contiene il valore di $c[i, j]$ e la freccia appropriata come valore di $b[i, j]$. Il valore 4 in $c[7, 6]$ – l'angolo inferiore destro della tabella – è la lunghezza di una LCS $\langle B, C, B, A\rangle$ di X e Y. Per $i, j > 0$, la posizione $c[i, j]$ dipende soltanto dal test se $x_i = y_j$ e dai valori $c[i - 1, j]$, $c[i, j - 1]$ e $c[i - 1, j - 1]$, che sono calcolati prima di $c[i, j]$. Per ricostruire gli elementi di una LCS, basta seguire le frecce $b[i, j]$ partendo dall'angolo inferiore destro della tabella; il cammino è indicato dallo sfondo grigio. Ogni freccia "↖" sul cammino corrisponde a una posizione (evidenziata) per la quale $x_i = y_j$ è un elemento di una LCS.

Migliorare il codice

Dopo avere sviluppato un algoritmo, spesso ci accorgiamo che è possibile migliorare il tempo di esecuzione o la quantità di memoria utilizzata dall'algoritmo. Alcune modifiche possono semplificare il codice dell'algoritmo e migliorare i fattori costanti, ma non producono miglioramenti asintotici delle prestazioni. Altre modifiche possono portare a sostanziali risparmi asintotici di tempo e memoria.

Per esempio, nell'algoritmo della LCS potremmo eliminare completamente la tabella b. Ogni posizione $c[i, j]$ dipende soltanto da altre tre posizioni della tabella c: $c[i - 1, j - 1]$, $c[i - 1, j]$ e $c[i, j - 1]$. Dato il valore di $c[i, j]$, possiamo determinare nel tempo $O(1)$ quale di questi tre valori è stato utilizzato per calcolare $c[i, j]$, senza ispezionare la tabella b. Quindi, possiamo ricostruire una LCS nel tempo $O(m + n)$ utilizzando una procedura simile a PRINT-LCS (l'Esercizio 15.4-2 chiede di scrivere lo pseudocodice). Sebbene questo metodo permetta di risparmiare uno spazio $\Theta(mn)$ in memoria, tuttavia lo spazio ausiliario richiesto per calcolare una LCS non diminuisce asintoticamente, perché occorre comunque uno spazio $\Theta(mn)$ per la tabella c.

Possiamo, però, ridurre il fabbisogno asintotico di memoria per LCS-LENGTH, perché questa procedura usa soltanto due righe alla volta della tabella c: la riga da calcolare e la riga precedente (in effetti, possiamo utilizzare uno spazio soltanto un po' più grande di quello richiesto da una riga di c per calcolare la lunghezza di una LCS. Vedere l'Esercizio 15.4-4). Questo miglioramento funziona se occorre calcolare soltanto la lunghezza di una LCS; se vogliamo ricostruire gli elementi di una LCS, la tabella più piccola non può contenere le informazioni necessarie per rifare il percorso inverso nel tempo $O(m + n)$.

Esercizi

Acquistato da Michele Michele su Webster il 2022-03-21 12:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

15.4-1

Trovate una LCS delle sequenze $\langle 1, 0, 0, 1, 0, 1, 0, 1\rangle$ e $\langle 0, 1, 0, 1, 1, 0, 1, 1, 0\rangle$.

15.4-2

Scrivete lo pseudocodice per ricostruire nel tempo $O(m+n)$ una LCS dalla tabella completa c e dalle sequenze originali $X = \langle x_1, x_2, \ldots, x_m\rangle$ e $Y = \langle y_1, y_2, \ldots, y_n\rangle$, senza utilizzare la tabella b.

15.4-3
Create una versione della procedura LCS-LENGTH che usa la tecnica di annotazione e che viene eseguita nel tempo $O(mn)$.

15.4-4
Spiegate come calcolare la lunghezza di una LCS utilizzando soltanto $2 \cdot \min(m, n)$ posizioni nella tabella c più uno spazio $O(1)$ aggiuntivo. Risolvete lo stesso problema utilizzando $\min(m, n)$ posizioni più uno spazio $O(1)$ aggiuntivo.

15.4-5
Create un algoritmo con tempo $O(n^2)$ per trovare la più lunga sottosequenza monotonicamente crescente di una sequenza di n numeri.

15.4-6 ⋆
Create un algoritmo con tempo $O(n \lg n)$ per trovare la più lunga sottosequenza monotonicamente crescente di una sequenza di n numeri (*suggerimento:* notate che l'ultimo elemento di una sottosequenza candidata di lunghezza i è grande almeno quanto l'ultimo elemento di una sottosequenza candidata di lunghezza $i - 1$. Memorizzate le sottosequenze candidate aggiungendo dei puntatori alla sequenza di input).

15.5 Alberi binari di ricerca ottimi

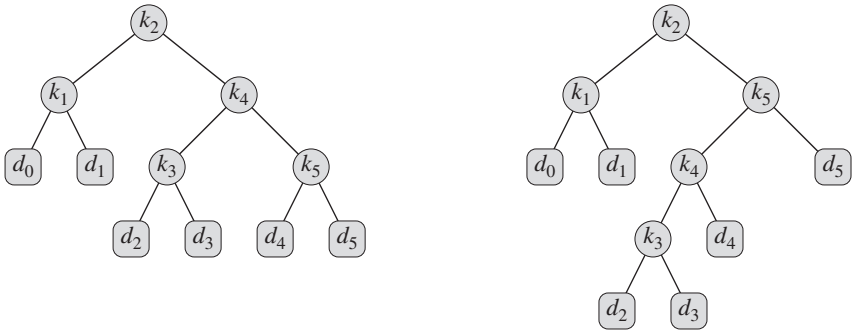
Supponete di progettare un programma di traduzione di un testo dall'inglese in italiano. Per ogni ricorrenza di una parola inglese nel testo, bisogna cercare l'equivalente parola italiana. Un modo per fare queste operazioni di ricerca è costruire un albero binario di ricerca con n parole inglesi come chiavi e le corrispondenti parole italiane come dati satelliti. Poiché ogni parola del testo dovrà essere cercata nell'albero, bisogna ridurre al minimo il tempo totale impiegato nelle ricerche. Potremmo garantire un tempo $O(\lg n)$ per ogni ricerca, utilizzando un albero rosso-nero o qualsiasi altro albero binario di ricerca bilanciato. Le parole, però, si presentano con frequenze differenti e potrebbe accadere che una parola frequentemente utilizzata, come "the" si trovi lontana dalla radice, mentre una parola raramente utilizzata, come "machicolation", si trovi vicino alla radice. Tale organizzazione rallenterebbe la traduzione del testo, in quanto il numero di nodi visitati durante la ricerca di una chiave in un albero binario di ricerca è pari a uno più la profondità del nodo che contiene la chiave. Noi vogliamo che le parole più frequenti nel testo occupino posizioni più vicine alla radice.[5] Inoltre, potrebbero esserci parole nel testo inglese per le quali non esistono le corrispondenti parole italiane;[6] queste parole potrebbero non apparire affatto nell'albero binario di ricerca. Come possiamo organizzare un albero binario di ricerca per minimizzare il numero di nodi visitati in tutte le ricerche, conoscendo quanto spesso si presenta ogni singola parola?

Ciò di cui abbiamo bisogno è un ***albero binario di ricerca ottimo***. Formalmente, data una sequenza $K = \langle k_1, k_2, \ldots, k_n \rangle$ di n chiavi distinte e ordinate (con $k_1 < k_2 < \cdots < k_n$), vogliamo costruire un albero binario di ricerca con queste chiavi. Per ogni chiave k_i, abbiamo una probabilità p_i che una ricerca riguar-

Acquistato da Michele Mele (michele.mele@unina.it) - 07/23 - Vietata la distribuzione - Copyright © 2010 The McGraw-Hill Education (Italy)

[5] Se il testo da tradurre riguarda l'architettura dei castelli, sarebbe preferibile avere la parola "machicolation" vicino alla radice.

[6] Sì, c'è la traduzione in italiano di *machicolation*: è *caditoio*.



(a) (b)

Figura 15.9 Due alberi binari di ricerca per un insieme di $n = 5$ chiavi con le seguenti probabilità:

i	0	1	2	3	4	5
p_i		0,15	0,10	0,05	0,10	0,20
q_i	0,05	0,10	0,05	0,05	0,05	0,10

(a) Un albero binario di ricerca con un costo atteso di ricerca pari a 2,80. **(b)** Un albero binario di ricerca con un costo atteso di ricerca pari a 2,75. Questo albero è ottimo.

derà k_i. Alcune ricerche potrebbero riguardare valori che non si trovano in K, quindi abbiamo anche $n + 1$ *chiavi fittizie* (o *chiavi dummy*) $d_0, d_1, d_2, \ldots, d_n$ che rappresentano valori che non appartengono a K. In particolare, d_0 rappresenta tutti i valori minori di k_1, d_n rappresenta tutti i valori maggiori di k_n e, per $i = 1, 2, \ldots, n-1$, la chiave fittizia d_i rappresenta tutti i valori fra k_i e k_{i+1}. Per ogni chiave fittizia d_i, abbiamo una probabilità q_i che una ricerca corrisponderà a d_i. La Figura 15.9 illustra due alberi binari di ricerca di un insieme di $n = 5$ chiavi. Ogni chiave k_i è un nodo interno; ogni chiave fittizia d_i è una foglia. Una ricerca può riuscire (viene trovata una chiave k_i) o fallire (viene trovata una chiave fittizia d_i), quindi abbiamo

$$\sum_{i=1}^{n} p_i + \sum_{i=0}^{n} q_i = 1 \tag{15.10}$$

Poiché conosciamo le probabilità delle ricerche per ogni chiave e per ogni chiave fittizia, possiamo determinare il costo atteso di una ricerca in un determinato albero binario di ricerca T. Supponiamo che il costo effettivo di una ricerca sia il numero di nodi esaminati, ovvero la profondità del nodo trovato dalla ricerca in T, più 1. Allora, il costo atteso di una ricerca in T è

$$\begin{aligned}
\mathrm{E}[\text{costo di una ricerca in } T] &= \sum_{i=1}^{n} (\text{profondità}_T(k_i) + 1) \cdot p_i + \\
&\quad \sum_{i=0}^{n} (\text{profondità}_T(d_i) + 1) \cdot q_i \\
&= 1 + \sum_{i=1}^{n} \text{profondità}_T(k_i) \cdot p_i + \\
&\quad \sum_{i=0}^{n} \text{profondità}_T(d_i) \cdot q_i
\end{aligned} \tag{15.11}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016,220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

dove profondità$_T$ indica la profondità di un nodo nell'albero T. L'ultima uguaglianza deriva dall'equazione (15.10). Nella Figura 15.9(a) possiamo calcolare il costo atteso di ricerca nodo per nodo:

nodo	profondità	probabilità	contributo
k_1	1	0,15	0,30
k_2	0	0,10	0,10
k_3	2	0,05	0,15
k_4	1	0,10	0,20
k_5	2	0,20	0,60
d_0	2	0,05	0,15
d_1	2	0,10	0,30
d_2	3	0,05	0,20
d_3	3	0,05	0,20
d_4	3	0,05	0,20
d_5	3	0,10	0,40
Totale			2,80

Per un dato insieme di probabilità, il nostro obiettivo è costruire un albero binario di ricerca il cui costo atteso di ricerca è minimo. Questo albero è detto ***albero binario di ricerca ottimo***. La Figura 15.9(b) illustra un albero binario di ricerca ottimo per le probabilità elencate nella didascalia della figura; il suo costo atteso è 2,75. Questo esempio dimostra che un albero binario di ricerca ottimo non è necessariamente un albero la cui altezza totale è minima, né possiamo necessariamente costruire un albero binario di ricerca ottimo ponendo sempre nella radice la chiave con la probabilità massima. Qui, la chiave k_5 ha la probabilità di ricerca più grande di qualsiasi chiave e la radice dell'albero binario di ricerca ottimo è invece k_2 (il costo minimo atteso di qualsiasi albero binario di ricerca con k_5 nella radice è 2,85).

Come nella moltiplicazione di una sequenza di matrici, il controllo esaustivo di tutte le possibilità non riesce a produrre un algoritmo efficiente. Possiamo etichettare i nodi di qualsiasi albero binario di n nodi con le chiavi $k_1, k_2, \ldots, k_n$ per costruire un albero binario di ricerca e, poi, aggiungere le chiavi fittizie come foglie. Nel Problema 12-4 abbiamo visto che il numero di alberi binari con n nodi è $\Omega(4^n/n^{3/2})$; quindi, in una ricerca esaustiva, dovremmo esaminare un numero esponenziale di alberi binari di ricerca. Nessuna sorpresa, quindi, se risolveremo questo problema con la programmazione dinamica.

Fase 1: la struttura di un albero binario di ricerca ottimo

Per caratterizzare la sottostruttura ottima degli alberi binari di ricerca ottimi, iniziamo con una osservazione sui sottoalberi. Consideriamo un sottoalbero qualsiasi di un albero binario di ricerca: le sue chiavi devono essere in un intervallo contiguo $k_i, \ldots, k_j$, per qualche $1 \le i \le j \le n$. Inoltre, un sottoalbero che contiene le chiavi $k_i, \ldots, k_j$ deve anche avere come foglie le chiavi fittizie $d_{i-1}, \ldots, d_j$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Adesso possiamo definire la sottostruttura ottima: se un albero binario di ricerca ottimo T ha un sottoalbero T' che contiene le chiavi $k_i, \ldots, k_j$, allora questo sottoalbero T' deve essere ottimo anche per il sottoproblema con chiavi $k_i, \ldots, k_j$ e chiavi fittizie $d_{i-1}, \ldots, d_j$. È possibile applicare la tecnica taglia e incolla. Se

ci fosse un sottoalbero T'' il cui costo atteso è minore di quello di T', allora potremmo tagliare T' da T e incollare T'', ottenendo un albero binario di ricerca con un costo atteso minore di quello di T, ma questo sarebbe in contraddizione con l'ipotesi che T sia un albero binario di ricerca ottimo.

Bisogna utilizzare la sottostruttura ottima per dimostrare che è possibile costruire una soluzione ottima del problema dalle soluzioni ottime dei sottoproblemi. Date le chiavi $k_i, \ldots, k_j$, una di queste chiavi, per esempio k_r ($i \le r \le j$), sarà la radice di un sottoalbero ottimo che contiene queste chiavi. Il sottoalbero sinistro della radice k_r conterrà le chiavi $k_i, \ldots, k_{r-1}$ (e le chiavi fittizie $d_{i-1}, \ldots, d_{r-1}$) e il sottoalbero destro conterrà le chiavi $k_{r+1}, \ldots, k_j$ (e le chiavi fittizie $d_r, \ldots, d_j$). Se esaminiamo tutte le radici candidate k_r, con $i \le r \le j$, e determiniamo tutti gli alberi binari di ricerca ottimi che contengono $k_i, \ldots, k_{r-1}$ e quelli che contengono $k_{r+1}, \ldots, k_j$, avremo la garanzia di trovare un albero binario di ricerca ottimo.

C'è un dettaglio importante da esaminare sui sottoalberi "vuoti". Supponiamo di scegliere k_i come radice di un sottoalbero con chiavi $k_i, \ldots, k_j$. In base al precedente ragionamento, il sottoalbero sinistro di k_i contiene le chiavi $k_i, \ldots, k_{i-1}$. È naturale dedurre che questa sequenza non contiene chiavi. Ricordiamo, però, che i sottoalberi contengono anche le chiavi fittizie. Adottiamo la convenzione che un sottoalbero che contiene le chiavi $k_i, \ldots, k_{i-1}$ non ha chiavi reali, ma contiene l'unica chiave fittizia d_{i-1}. In modo simmetrico, se selezioniamo k_j come radice, allora il sottoalbero destro di k_j contiene le chiavi $k_{j+1}, \ldots, k_j$; questo sottoalbero destro non contiene chiavi reali, ma contiene la chiave fittizia d_j.

Fase 2: una soluzione ricorsiva

A questo punto possiamo definire il valore di una soluzione ottima in modo ricorsivo. Il nostro dominio dei sottoproblemi è trovare un albero binario di ricerca ottimo che contiene le chiavi $k_i, \ldots, k_j$, dove $i \ge 1$, $j \le n$ e $j \ge i - 1$ (quando $j = i - 1$, non ci sono chiavi reali e c'è l'unica chiave fittizia d_{i-1}). Definiamo $e[i, j]$ come il costo di ricerca atteso in un albero binario di ricerca ottimo che contiene le chiavi $k_i, \ldots, k_j$. In ultima analisi, vogliamo calcolare $e[1, n]$.

Il caso semplice si verifica quando $j = i - 1$; c'è una sola chiave fittizia: d_{i-1}. Il costo atteso di ricerca è $e[i, i - 1] = q_{i-1}$.[7]

Quando $j \ge i$, bisogna scegliere una radice k_r fra $k_i, \ldots, k_j$ e poi creare, come suo sottoalbero sinistro, un albero binario di ricerca ottimo con le chiavi $k_i, \ldots, k_{r-1}$ e, come suo sottoalbero destro, un albero binario di ricerca ottimo con le chiavi $k_{r+1}, \ldots, k_j$. Che cosa accade al costo atteso di ricerca di un sottoalbero quando questo diventa un sottoalbero di un nodo? La profondità di ogni nodo nel sottoalbero aumenta di 1. Per l'equazione (15.11), il costo atteso di ricerca di questo sottoalbero aumenta della somma di tutte le probabilità nel sottoalbero. Per

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

[7] Nota del curatore: in realtà il costo atteso è 1 in quanto il sottoalbero contiene un solo nodo che viene visitato in ogni ricerca. Il testo assume implicitamente, come costo da minimizzare per i sottoalberi, il numero atteso di nodi visitati in una ricerca di una chiave appartenente all'insieme delle chiavi (reali o fittizie) che compaiono nel sottoalbero stesso moltiplicato per la probabilità che una chiave appartenga a tale insieme. Questo rende vera la definizione (15.11) del costo anche per i sottoproblemi, quando la sommatoria è limitata ai nodi del sottoalbero e la somma delle probabilità è minore di 1.

un sottoalbero con chiavi $k_i, \ldots, k_j$, indichiamo questa somma di probabilità con la seguente espressione

$$w(i, j) = \sum_{l=i}^{j} p_l + \sum_{l=i-1}^{j} q_l \tag{15.12}$$

Quindi, se k_r è la radice di un sottoalbero ottimo che contiene le chiavi $k_i, \ldots, k_j$, abbiamo

$$e[i, j] = p_r + (e[i, r-1] + w(i, r-1)) + (e[r+1, j] + w(r+1, j))$$

Osservando che

$$w(i, j) = w(i, r-1) + p_r + w(r+1, j)$$

possiamo riscrivere $e[i, j]$ in questo modo

$$e[i, j] = e[i, r-1] + e[r+1, j] + w(i, j) \tag{15.13}$$

L'equazione ricorsiva (15.13) suppone che sia noto il nodo k_r da utilizzare come radice. Scegliendo la radice che ha il costo atteso di ricerca minimo, otteniamo la formula ricorsiva finale:

$$e[i, j] = \begin{cases} q_{i-1} & \text{se } j = i-1 \\ \min\limits_{i \le r \le j} \{e[i, r-1] + e[r+1, j] + w(i, j)\} & \text{se } i \le j \end{cases} \tag{15.14}$$

I valori $e[i, j]$ rappresentano i costi attesi di ricerca negli alberi binari di ricerca ottimi. Per tenere traccia della struttura degli alberi binari di ricerca ottimi, definiamo $root[i, j]$, per $1 \le i \le j \le n$, come l'indice r per il quale k_r è la radice di un albero binario di ricerca ottimo che contiene le chiavi $k_i, \ldots, k_j$. Spiegheremo come calcolare i valori di $root[i, j]$ e lasceremo al lettore il compito di costruire l'albero binario di ricerca ottimo da questi valori (l'Esercizio 15.5-1).

Fase 3: calcolare il costo di ricerca atteso in un albero binario di ricerca ottimo

A questo punto, qualcuno avrà notato qualche analogia fra la caratterizzazione degli alberi binari di ricerca ottimi e la caratterizzazione della moltiplicazione di una sequenza di matrici. Per entrambi i domini dei problemi, i sottoproblemi sono formati da sottointervalli di indici contigui. Una implementazione ricorsiva diretta dell'equazione (15.14) potrebbe risultare inefficiente come l'algoritmo ricorsivo diretto della moltiplicazione di una sequenza di matrici. Invece, memorizziamo i valori $e[i, j]$ in una tabella $e[1..n+1, 0..n]$. Il primo indice deve arrivare fino a $n+1$ (anziché n) perché, per ottenere un sottoalbero che contiene soltanto la chiave fittizia d_n, dobbiamo calcolare e memorizzare $e[n+1, n]$. Il secondo indice deve iniziare da 0 perché, per ottenere un sottoalbero che contiene soltanto la chiave fittizia d_0, dobbiamo calcolare e memorizzare $e[1, 0]$. Utilizzeremo soltanto le posizioni $e[i, j]$ per le quali $j \ge i - 1$. Utilizzeremo anche una tabella $root[i, j]$ per memorizzare la radice del sottoalbero che contiene le chiavi $k_i, \ldots, k_j$. Questa tabella usa soltanto le posizioni per le quali $1 \le i \le j \le n$. Per migliorare l'efficienza, utilizzeremo un'altra tabella. Anziché ricominciare da zero il calcolo di $w(i, j)$ ogni volta che calcoliamo $e[i, j]$ – il che richiederebbe $\Theta(j - i)$ addizioni – memorizziamo questi valori in una tabella $w[1..n+1, 0..n]$. Per il caso base, calcoliamo $w[i, i-1] = q_{i-1}$ per $1 \le i \le n+1$. Per $j \ge i$, calcoliamo

Acquistato da Michele Mostarda (mostarda@gmail.com) in data 20/07/2022 17:07:00 tramite Numero Ordine Utente: 2022-0720170700 - Copyright © 2022 McGraw-Hill Education (Italy)

$$w[i, j] = w[i, j-1] + p_j + q_j \tag{15.15}$$

Quindi, possiamo calcolare ciascuno dei $\Theta(n^2)$ valori di $w[i, j]$ nel tempo $\Theta(1)$. Il seguente pseudocodice riceve come input le probabilità $p_1, \ldots, p_n$ e $q_0, \ldots, q_n$ e la dimensione n e restituisce le tabelle e e *root*.

```
OPTIMAL-BST(p, q, n)
 1  Siano e[1..n+1, 0..n], w[1..n+1, 0..n]
          e root[1..n, 1..n] tre nuove tabelle
 2  for i = 1 to n + 1
 3      e[i, i-1] = q_{i-1}
 4      w[i, i-1] = q_{i-1}
 5  for l = 1 to n
 6      for i = 1 to n - l + 1
 7          j = i + l - 1
 8          e[i, j] = ∞
 9          w[i, j] = w[i, j-1] + p_j + q_j
10          for r = i to j
11              t = e[i, r-1] + e[r+1, j] + w[i, j]
12              if t < e[i, j]
13                  e[i, j] = t
14                  root[i, j] = r
15  return e e root
```

Dalla precedente descrizione e per l'analogia con la procedura MATRIX-CHAIN-ORDER (Paragrafo 15.2), il funzionamento di OPTIMAL-BST dovrebbe essere abbastanza chiaro. Il primo ciclo **for** (righe 2–4) inizializza i valori di $e[i, i-1]$ e $w[i, i-1]$. Il ciclo **for** (righe 5–14) usa le ricorrenze (15.14) e (15.15) per calcolare $e[i, j]$ e $w[i, j]$ per ogni $1 \le i \le j \le n$. Nella prima iterazione, quando $l = 1$, il ciclo calcola $e[i, i]$ e $w[i, i]$ per $i = 1, 2, \ldots, n$. Nella seconda iterazione, con $l = 2$, il ciclo calcola $e[i, i+1]$ e $w[i, i+1]$ per $i = 1, 2, \ldots, n-1$, e così via. Il ciclo **for** più interno (righe 10–14) prova ciascun indice r candidato per determinare quale chiave k_r utilizzare come radice di un albero binario di ricerca ottimo che contiene le chiavi $k_i, \ldots, k_j$. Questo ciclo **for** salva il valore corrente dell'indice r nella posizione $root[i, j]$, ogni volta che trova una chiave migliore da utilizzare come radice.

La Figura 15.10 illustra le tabelle $e[i, j]$, $w[i, j]$ e $root[i, j]$ calcolate dalla procedura OPTIMAL-BST con la distribuzione delle chiavi indicata nella Figura 15.9. Come nell'esempio della moltiplicazione di una sequenza di matrici (Figura 15.5), le tabelle sono ruotate per rappresentare orizzontalmente le diagonali. La procedura OPTIMAL-BST calcola le righe dal basso verso l'alto e ciascuna riga da sinistra a destra. La procedura OPTIMAL-BST impiega un tempo $\Theta(n^3)$, esattamente come MATRIX-CHAIN-ORDER. È facile capire che il tempo di esecuzione è $O(n^3)$, perché i cicli **for** hanno tre livelli di annidamento e ogni indice di ciclo assume al massimo n valori. Gli indici di ciclo in OPTIMAL-BST non hanno esattamente gli stessi limiti di quelli di MATRIX-CHAIN-ORDER, ma differiscono al massimo di 1 in più o in meno. Quindi, come MATRIX-CHAIN-ORDER, la procedura OPTIMAL-BST impiega il tempo $\Omega(n^3)$.

Acquistato da Michele Michele su Webster il 2022-07-07 22:13 Numero Ordine Libreria: 1995039616-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

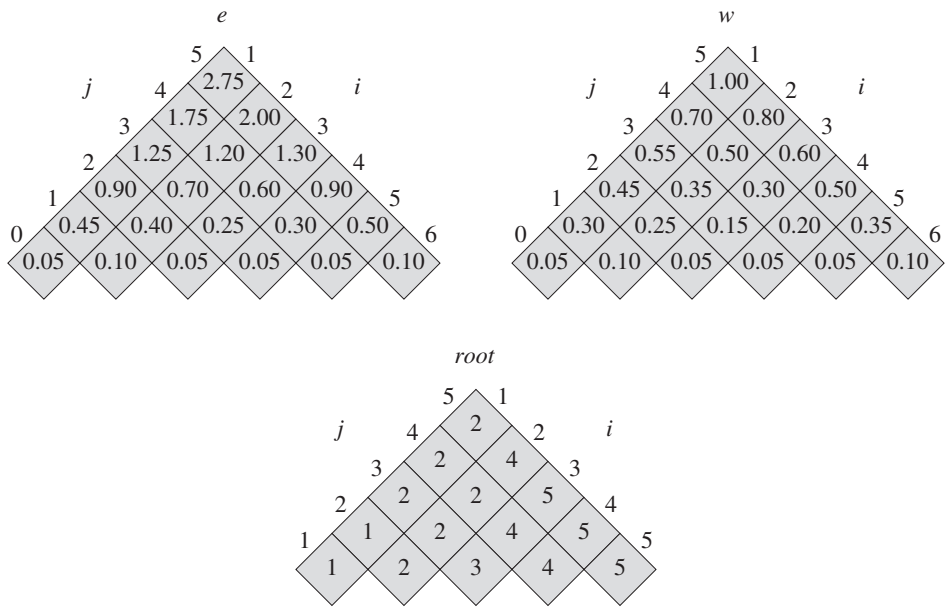


Figura 15.10 Le tabelle $e[i, j]$, $w[i, j]$ e $root[i, j]$ calcolate da OPTIMAL-BST con la distribuzione delle chiavi indicata nella Figura 15.9. Le tabelle sono ruotate in modo che le diagonali siano orizzontali.

Esercizi

15.5-1

Scrivete lo pseudocodice per la procedura CONSTRUCT-OPTIMAL-BST(*root*) che, data la tabella *root*, genera in output la struttura di un albero binario di ricerca ottimo. Per l'esempio illustrato nella Figura 15.10, la vostra procedura dovrebbe visualizzare la struttura

k_2 è la radice
k_1 è il figlio sinistro di k_2
d_0 è il figlio sinistro di k_1
d_1 è il figlio destro di k_1
k_5 è il figlio destro di k_2
k_4 è il figlio sinistro di k_5
k_3 è il figlio sinistro di k_4
d_2 è il figlio sinistro di k_3
d_3 è il figlio destro di k_3
d_4 è il figlio destro di k_4
d_5 è il figlio destro di k_5

che corrisponde all'albero binario di ricerca ottimo illustrato nella Figura 15.9(b).

15.5-2

Acquistato da Michele Michels su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016. 220707. Copyright © 2022, McGraw-Hill Education (Italy)

Determinate il costo e la struttura di un albero binario di ricerca ottimo per un insieme di $n = 7$ chiavi con le seguenti probabilità:

i	0	1	2	3	4	5	6	7
p_i		0,04	0,06	0,08	0,02	0,10	0,12	0,14
q_i	0,06	0,06	0,06	0,06	0,05	0,05	0,05	0,05

15.5-3
Anziché mantenere la tabella $w[i, j]$, supponiamo di calcolare il valore di $w(i, j)$ direttamente dall'equazione (15.12) nella riga 9 di OPTIMAL-BST e di utilizzare il valore risultante nella riga 11. Come influisce tutto questo sul tempo di esecuzione asintotico di OPTIMAL-BST?

15.5-4 ⋆
Knuth [213] ha dimostrato che ci sono sempre delle radici di sottoalberi ottimi tali che $root[i, j-1] \le root[i, j] \le root[i+1, j]$ per ogni $1 \le i < j \le n$. Utilizzate questo fatto per modificare la procedura OPTIMAL-BST in modo che sia eseguita nel tempo $\Theta(n^2)$.

Problemi

15-1 Cammino semplice più lungo in un grafo orientato aciclico
Un grafo orientato aciclico $G = (V, E)$, i cui archi hanno come pesi dei numeri reali, ha due vertici distinti s e t. Descrivete un approccio di programmazione dinamica per trovare un cammino semplice più lungo da s a t. Disegnate il grafo dei sottoproblemi. Qual è l'efficienza del vostro algoritmo?

15-2 Sottosequenza palindroma più lunga
Un ***palindromo*** è una stringa alfabetica non vuota che può essere letta da sinistra a destra e viceversa. Esempi di polindromi sono tutte le stringhe di lunghezza 1, `radar`, `anilina` e `aibofobia` (paura dei palindromi).

Scrivete un algoritmo efficiente per trovare il più lungo palindromo che è sottosequenza di una stringa di input. Per esempio, data la stringa di input `character`, l'algoritmo dovrebbe fornire `carac`. Qual è il tempo di esecuzione del vostro algoritmo?

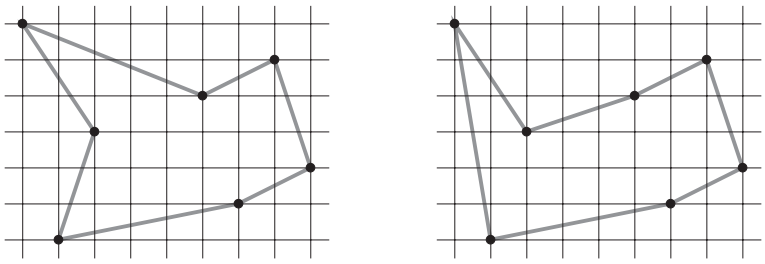
15-3 Problema del commesso viaggiatore euclideo e bitonico
Il ***problema del commesso viaggiatore euclideo*** consiste nel determinare il cammino chiuso minimo che collega un dato insieme di n punti del piano. La Figura 15.11(a) illustra la soluzione di un problema con un insieme di 7 punti. Il problema generale è NP-difficile, pertanto la sua soluzione richiede un tempo più che polinomiale (vedere il Capitolo 34).

J. L. Bentley suggerisce di semplificare il problema considerando soltanto i ***cammini bitonici***, ovvero quei cammini che iniziano dal punto più a sinistra, vanno sempre da sinistra a destra fino a raggiungere il punto più a destra e poi sempre da destra a sinistra fino a ritornare al punto di partenza. La Figura 15.11(b) illustra il cammino bitonico minimo per lo stesso insieme di 7 punti. In questo caso, è possibile definire un algoritmo con tempo polinomiale.

Descrivete un algoritmo con tempo $O(n^2)$ per determinare un cammino bitonico ottimo. Potete supporre che due punti non possano avere la stessa coordinata x (*suggerimento:* fate una scansione dei punti da sinistra a destra, mantenendo le possibilità ottime per le due parti del cammino).

Acquistato da Michele Michele su Webster il 2022-07-07 29.12 - Numero Ordine Libraria: 1995090115-2207077 © Copyright © 2022, McGraw-Hill Education (Italy)

15-4 Una stampa accurata
Considerate il problema di stampare in modo accurato un paragrafo di testo usando caratteri di larghezza fissa. Il testo di input è una sequenza di n parole di lunghezza $l_1, l_2, \ldots, l_n$; la lunghezza di una parola è espressa dal numero di caratteri.



(a) (b)

Figura 15.11 Sette punti di un piano illustrati su una griglia con quadrati di lato unitario. **(a)** Il cammino chiuso minimo, con lunghezza approssimativamente pari a 24,89. Questo cammino non è bitonico. **(b)** Il cammino bitonico minimo per lo stesso insieme di punti. La sua lunghezza è approssimativamente pari a 25,58.

Vogliamo stampare questo paragrafo in modo accurato su un certo numero di righe, ciascuna delle quali contiene al massimo M caratteri. Il nostro criterio di "stampa accurata" è il seguente. Se una data riga contiene le parole da i a j, con $i \leq j$, e lasciamo esattamente uno spazio fra le parole, il numero di spazi extra alla fine della riga è $M - j + i - \sum_{k=i}^{j} l_k$, che deve essere non negativo, in modo che le parole possano adattarsi alla riga. Vogliamo rendere minima la sommatoria, per tutte le righe tranne l'ultima, dei cubi dei numeri di spazi extra che restano alla fine di ogni riga. Descrivete un algoritmo di programmazione dinamica per stampare in modo accurato un paragrafo di n parole. Analizzate il tempo di esecuzione e la quantità di memoria richiesta dal vostro algoritmo.

15-5 Distanza di editing

Per trasformare una stringa di input $x[1..m]$ in una stringa di output $y[1..n]$, possiamo svolgere varie operazioni. Date le stringhe x e y, il nostro obiettivo è effettuare una serie di trasformazioni che cambiano x in y. Utilizziamo un array z per memorizzare i risultati intermedi (supponendo che l'array abbia una dimensione sufficiente a contenere tutti i caratteri che servono). Inizialmente, z è vuoto e, alla fine, dovremmo avere $z[j] = y[j]$ per $j = 1, 2, \ldots, n$. Manteniamo gli indici correnti i in x e j in z; le operazioni possono modificare z e questi indici. Inizialmente, $i = j = 1$. Dobbiamo esaminare tutti i caratteri di x durante la trasformazione; questo significa che, alla fine della sequenza delle operazioni, dobbiamo avere $i = m + 1$.

Ci sono sei operazioni di trasformazione:

Copia un carattere da x a z, impostando $z[j] = x[i]$ e poi incrementando i e j. Questa operazione esamina $x[i]$.

Sostituisci un carattere di x con un altro carattere c, impostando $z[j] = c$ e poi incrementando i e j. Questa operazione esamina $x[i]$.

Cancella un carattere di x, incrementando i, senza modificare j. Questa operazione esamina $x[i]$.

Inserisci il carattere c in z, impostando $z[j] = c$ e poi incrementando j, senza modificare i. Questa operazione non esamina i caratteri di x.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Scambia i prossimi due caratteri copiandoli da x a z, ma in ordine inverso; per fare questo, impostiamo prima $z[j] = x[i + 1]$ e $z[j + 1] = x[i]$ e, poi, $i = i + 2$ e $j = j + 2$. Questa operazione esamina $x[i]$ e $x[i + 1]$.

Distruggi la parte restante di x, impostando $i = m + 1$. Questa operazione esamina tutti i caratteri di x che non sono stati ancora esaminati. Se questa operazione viene svolta, deve essere l'ultima.

Per esempio, un modo per trasformare la stringa di input `algorithm` nella stringa di output `altruistic` consiste nell'utilizzare questa sequenza di operazioni (i caratteri sottolineati sono $x[i]$ e $z[j]$ dopo ogni operazione):

Operazione	x	z
stringhe iniziali	a̲lgorithm	_
copia	al̲gorithm	a_
copia	alg̲orithm	al_
sostituisci con t	algo̲rithm	alt_
cancella	algor̲ithm	alt_
copia	algori̲thm	altr_
inserisci u	algori̲thm	altru_
inserisci i	algori̲thm	altrui_
inserisci s	algori̲thm	altruis_
scambia	algorit̲hm	altruisti_
inserisci c	algorit̲hm	altruistic_
distruggi	algorithm_	altruistic_

Notate che ci sono molte altre sequenze di operazioni che possono trasformare `algorithm` in `altruistic`.

Ciascuna delle operazioni di trasformazione ha un costo associato. Il costo di un'operazione dipende dalla specifica applicazione, ma supponiamo che il costo di ciascuna operazione sia una costante nota. Supponiamo inoltre che i singoli costi delle operazioni di copia e sostituzione siano minori dei costi combinati delle operazioni di cancellazione e inserimento, altrimenti le operazioni di copia e sostituzione non sarebbero utilizzate. Il costo di una data sequenza di operazioni di trasformazione è la somma dei costi delle singole operazioni nella sequenza. Per la precedente sequenza, il costo per trasformare `algorithm` in `altruistic` è

$$(3 \cdot \text{costo(copia)}) + \text{costo(sostituisci)} + \text{costo(cancella)} + (4 \cdot \text{costo(inserisci)}) + \text{costo(scambia)} + \text{costo(distruggi)}$$

a. Date due sequenze $x[1..m]$ e $y[1..n]$ e un insieme di costi delle operazioni di trasformazione, la ***distanza di editing*** tra x e y è il costo della sequenza di operazioni più economica che trasforma x in y. Descrivete un algoritmo di programmazione dinamica che trova la distanza di editing tra $x[1..m]$ e $y[1..n]$ e stampa una sequenza di operazioni ottima. Analizzate il tempo di esecuzione e la quantità di memoria richiesta dal vostro algoritmo.

Il problema della distanza di editing è una generalizzazione del problema dell'allineamento di due sequenze di DNA (vedere, per esempio, Setubal e Meidanis [311, Paragrafo 3.2]). Ci sono vari metodi per misurare la somiglianza di due sequenze di DNA. Uno di questi metodi consiste nell'allineare due sequenze x e y inserendo degli spazi in posizioni arbitrarie delle due sequenze (incluse le due estremità), in modo che le sequenze risultanti x' e y' abbiano la stessa lunghezza, ma non abbiano uno spazio nella stessa posizione (ovvero, per nessuna posizione j, gli elementi $x'[j]$ e $y'[j]$ possono essere entrambi uno spazio). Poi assegniamo un "punteggio" a ogni posizione. La posizione j riceve i seguenti punteggi:

Acquistato da Michele Michele su Webster il 2022-07-07 20:13 (utente Ordine Libreria 1804820116 - 220707) © Cambridge © 2022 McGraw-Hill Education (Italy)

- $+1$ se $x'[j] = y'[j]$ e nessuno dei due elementi è uno spazio
- -1 se $x'[j] \neq y'[j]$ e nessuno dei due elementi è uno spazio
- -2 se $x'[j]$ o $y'[j]$ è uno spazio

Il punteggio dell'allineamento è la somma dei punteggi delle singole posizioni. Per esempio, date le sequenze $x =$ GATCGGCAT e $y =$ CAATGTGAATC, un allineamento è

```
G ATCG GCAT
CAAT GTGAATC
-*++++++++*+
```

Un segno più (+) sotto una posizione indica un punteggio $+1$ per quella posizione; un segno meno (−) indica un punteggio -1 e l'asterisco ($*$) indica un punteggio -2; quindi, questo allineamento ha un punteggio totale pari a $6 \cdot 1 - 2 \cdot 1 - 4 \cdot 2 = -4$.

b. Spiegate come trasformare il problema di trovare un allineamento ottimo in un problema di ricerca della distanza di editing, utilizzando un sottoinsieme delle operazioni copia, sostituisci, cancella, inserisci, scambia e distruggi.

15-6 Programmare una festa aziendale

Il professor Stewart è un consulente del presidente di una grande azienda che sta programmando una festa per i dipendenti. L'azienda ha una struttura gerarchica, nella quale la relazione di dipendenza forma un albero che ha la radice nel presidente. L'ufficio del personale ha classificato ogni dipendente con un grado di giovialità, espresso da un numero reale. Affinché la festa possa essere piacevole per tutti i partecipanti, il presidente non vuole che alla festa siano presenti un dipendente e il suo diretto superiore. Il professor Stewart ha l'albero che descrive la struttura gerarchica dell'azienda mediante una rappresentazione figlio-sinistro fratello-destro (vedere il Paragrafo 10.4). Ogni nodo dell'albero contiene, oltre ai puntatori, il nome di un dipendente e il suo grado di giovialità. Descrivete un algoritmo che prepara la lista degli invitati, massimizzando la somma dei gradi di giovialità degli ospiti. Analizzate il tempo di esecuzione dell'algoritmo.

15-7 Algoritmo di Viterbi

Possiamo applicare la programmazione dinamica a un grafo orientato $G = (V, A)$ per il riconoscimento della voce. Ogni arco $(u, v) \in A$ è etichettato con un suono $\sigma(u, v)$ che appartiene a un insieme finito Σ di suoni. Il grafo etichettato è un modello formale di una persona che parla una lingua molto semplice. Ogni cammino nel grafo che inizia da un vertice distinto $v_0 \in V$ corrisponde a una possibile sequenza di suoni prodotti dal modello. L'etichetta di un cammino orientato è definita come la concatenazione delle etichette degli archi in quel cammino.

a. Supponete di avere un grafo etichettato G con un vertice distinto v_0 e una sequenza $s = \langle \sigma_1, \sigma_2, \ldots, \sigma_k \rangle$ di caratteri dell'insieme Σ. Descrivete un algoritmo efficiente che restituisce un cammino in G che inizia da v_0 e ha s come sua etichetta, se tale cammino esiste. Altrimenti, l'algoritmo dovrà restituire CAMMINO-INESISTENTE. Analizzate il tempo di esecuzione dell'algoritmo (*suggerimento:* potrebbe essere utile consultare il Capitolo 22).

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Adesso, supponete che a ogni arco $(u, v) \in A$ sia associata una probabilità non negativa $p(u, v)$ di attraversare l'arco (u, v) a partire dal vertice u, producendo così il corrispondente suono. La somma delle probabilità degli archi che escono da un vertice qualsiasi è pari a 1. La probabilità di un cammino è definita come il prodotto delle probabilità dei suoi archi. Possiamo considerare la probabilità di un cammino che inizia da v_0 come la probabilità che un "percorso casuale" che

inizia da v_0 segua il cammino specificato, dove la scelta di quale arco prendere in un vertice u viene fatta in maniera casuale, secondo le probabilità degli archi disponibili che partono da u.

b. Estendete l'algoritmo descritto nel punto (a) in modo che, se viene restituito un cammino, questo sia il *più probabile cammino* che parte da v_0 e che ha l'etichetta s. Analizzate il tempo di esecuzione dell'algoritmo.

15-8 Compressione delle immagini con il metodo seam carving

Sia data un'immagine a colori formata da un array $m \times n$ di $A[1\,..\,m, 1\,..\,n]$ pixel, dove ogni pixel specifica tre intensità di colori RGB (red, green e blue). Si vuole comprimere leggermente questa immagine. In particolare, si vuole eliminare un pixel da ciascuna delle m righe, in modo che l'intera immagine si restringa di un pixel. Tuttavia, per evitare sgradevoli effetti visivi, occorre che i pixel rimossi in due righe adiacenti siano nelle stesse colonne o in colonne adiacenti; i pixel rimossi formano una linea ("seam") che va dalla riga superiore a quella inferiore; i pixel successivi di questa seam sono adiacenti verticalmente o diagonalmente.

a. Dimostrate che il numero di tali seam cresce almeno in modo esponenziale in m, supponendo che $n > 1$.

b. Supponete adesso che per ciascun pixel $A[i, j]$ sia stato calcolato un valore di discontinuità $d[i, j]$, che indica il grado di discontinuità che si avrebbe se si eliminasse il pixel $A[i, j]$. In altre parole, quanto più è basso il grado di discontinuità di un pixel, tanto più questo pixel è simile ai suoi pixel adiacenti. Supponete inoltre di definire il grado di discontinuità di una seam come la somma dei gradi di discontinuità dei suoi pixel.

Scrivete un algoritmo per trovare una seam con il minimo grado di discontinuità. Qual è l'efficienza del vostro algoritmo?

15-9 Taglio di una stringa

Un certo linguaggio per l'elaborazione delle stringhe consente a un programmatore di tagliare una stringa in due parti. Poiché questa operazione richiede che la stringa venga copiata, il taglio di una stringa di n caratteri in due parti costa n unità di tempo. Supponete che un programmatore voglia tagliare una stringa in più parti. L'ordine in cui vengono fatti i tagli può influire sul tempo totale impiegato. Per esempio, supponete che il programmatore voglia tagliare una stringa di 20 caratteri dopo 2, 8 e 10 caratteri (la numerazione dei caratteri è in ordine crescente a partire dall'estremità sinistra, che è 1). Se i tagli sono programmati in modo che vengano effettuati da sinistra a destra, il primo taglio costa 20 unità di tempo, il secondo taglio costa 18 unità di tempo (tagliando la stringa dai caratteri 3 al 20 nel carattere 8), e il terzo taglio costa 12 unità di tempo, per un totale di 50 unità di tempo. Se i tagli sono programmati per essere effettuati da destra a sinistra, invece, il primo taglio costa 20 unità di tempo, il secondo taglio costa 10 unità di tempo e il terzo taglio costa 8 unità di tempo, per un totale di 38 unità di tempo. In un altro ordine ancora, il primo taglio viene effettuato nel carattere 8 (costo 20), poi si taglia la parte sinistra nel carattere 2 (costo 8) e, infine, si taglia la parte destra nel carattere 10 (costo 12), per un totale di 40 unità di tempo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 - Numero Unico Libreria: 1483938516-220707 - Copyright © 2022, McGraw-Hill Education (Italy)

Scrivete un algoritmo che, dati i numeri di caratteri dopo i quali tagliare, determina il modo più economico per effettuare la sequenza dei tagli. Più formalmente, data una stringa S di n caratteri e un array $L[1..m]$ che contiene i punti di taglio, calcolare il costo minimo di una sequenza di tagli e la sequenza di tagli che consente di ottenere tale costo.

15-10 Pianificazione di una strategia di investimenti

La vostra conoscenza degli algoritmi vi ha permesso di ottenere un interessante lavoro presso l'Acme Computer Company, insieme a un bonus iniziale di 10000 $. Decidete di investire questa somma con l'obiettivo di massimizzarne il rendimento dopo 10 anni. Affidate la gestione di questa somma all'Amalgamated Investment Company, la quale chiede il rispetto delle seguenti regole. La società offre n investimenti differenti, numerati da 1 a n. Per ogni anno j, l'investimento i fornisce un tasso di rendimento di r_{ij}. In altre parole, se investite d dollari nell'investimento i nell'anno j, alla fine dell'anno j, avrete $d r_{ij}$ dollari. I tassi di rendimento sono garantiti, ovvero avrete tutti i rendimenti per i prossimi 10 anni per ciascun investimento. Le decisioni sugli investimenti possono essere prese soltanto una volta all'anno. Alla fine di ogni anno potete lasciare negli stessi investimenti il denaro ricavato nell'anno precedente, oppure spostare il denaro su altri investimenti, sia spostando il denaro tra gli investimenti attuali, sia spostandolo su nuovi investimenti. Se non spostate il denaro tra due anni consecutivi pagate una tassa di f_1 dollari, mentre se spostate il denaro pagate una tassa di f_2 dollari, con $f_2 > f_1$.

a. Il problema, come è stato definito, vi consente di distribuire i vostri soldi in più investimenti in ciascun anno. Dimostrate che esiste una strategia di investimenti ottima che pone, in ciascun anno, tutti i soldi in un unico investimento. (Ricordiamo che una strategia di investimenti ottima massimizza la quantità di denaro dopo 10 anni e non considera altri obiettivi, quali minimizzare il rischio.)

b. Dimostrate che il problema di pianificare la strategia di investimenti ottima presenta una sottostruttura ottima.

c. Scrivete un algoritmo che pianifica la strategia di investimenti ottima. Qual è il tempo di esecuzione di questo algoritmo?

d. Supponte che l'Amalgamated Investments Company imponga un'ulteriore restrizione: in qualsiasi momento non è possibile avere più di 15000 $ in un investimento qualsiasi. Dimostrate che il problema di massimizzare il rendimento alla fine dei 10 anni non presenta più una sottostruttura ottima.

15-11 Programmazione delle scorte

La Rinky Dink Company produce macchine che ripristinano le piste di pattinaggio. La domanda per queste macchine varia di mese in mese, e quindi la società ha bisogno di sviluppare una strategia per pianificare la sua produzione tenendo conto di una domanda fluttuante, ma prevedibile. La società vuole preparare un piano di produzione per i prossimi n mesi. Per ogni mese i, la società conosce la domanda d_i, ovvero il numero di macchine che venderà. Sia $D = \sum_{i=1}^{n} d_i$ la domanda totale per i prossimi n mesi. La società può disporre di una manodopera

Acquistato da Michele Mariele su webasset il 2021-07-01 2:0:22 Numero Ordine Libreria: 1955858 19-2207.07 © Copyright © 2022, McGraw-Hill Education (Italy)

a tempo pieno per produrre m macchine al mese. Se avesse bisogno di produrre più di m macchine in un dato mese, potrà utilizzare una manodopera part-time a un costo che incide per c dollari per macchina. Inoltre, se alla fine di un mese la società non riuscisse a vendere tutte le macchine, dovrà sostenere un costo per le scorte. Il costo delle scorte per j macchine è espresso da una funzione $h(j)$ per $j = 1, 2, \ldots, D$, dove $h(j) \geq 0$ per $1 \leq j \leq D$ e $h(j) \leq h(j+1)$ per $1 \leq j \leq D-1$.

Scrivete un algoritmo che calcola un piano che minimizza i costi per la società soddisfacendo tutta la domanda. Il tempo di esecuzione dovrà essere polinomiale in n e D.

15-12 Arruolare giocatori free-agent per una squadra di baseball

Il general manager di una squadra di baseball, nel fuori stagione, vuole arruolare alcuni giocatori free-agent. Il proprietario della squadra ha autorizzato il general manager a spendere una somma di X \$ per arruolare questi giocatori. Il general manager sarà licenziato se spende più di X \$.

Il general manager sta valutando N ruoli differenti e, per ciascun ruolo, sono disponibili P giocatori free-agent che giocano in tale ruolo.[8] Poiché il nostro general manager non vuole sovraccaricare la tabella dei turni con troppi giocatori nei singoli ruoli, ha previsto di arruolare un solo giocatore free-agent per ciascun ruolo. (Se il general manager non arruola alcun giocatore free-agent in un particolare ruolo, utilizzerà quelli che giocano già in quel ruolo.)

Per determinare il valore di un giocatore, il general manager decide di utilizzare una statistica sabermetrica[9] detta "VORP", che sta per "value over replacement player". Un giocatore con un VORP più alto vale più di uno che ha un VORP più basso. Arruolare un giocatore con un VORP più alto non costa necessariamente di più di un giocatore con un VORP più basso, perché ci sono altri fattori che determinano il costo di arruolamento.

Per ciascun giocatore free-agent, il general manager dispone di tre tipi di informazioni:

- il ruolo del giocatore
- il costo di arruolamento
- il valore di VORP

Scrivete un algoritmo che massimizza il VORP totale dei giocatori arruolati, senza superare la spesa complessiva di X \$. Potreste supporre che ciascun giocatore firmi il contratto per un multiplo di 100000 \$. L'algoritmo dovrebbe fornire il VORP totale dei giocatori arruolati, il costo totale e un elenco dei giocatori arruolati. Analizzate il tempo di esecuzione e lo spazio richiesto dall'algoritmo.

---

[8] Sebbene ci siano 9 ruoli in una squadra di baseball, N non è necessariamente uguale a 9 perché alcuni general manager hanno idee diverse sul modo di concepire i ruoli. Per esempio, un general manager potrebbe considerare "ruoli" distinti il lanciatore mancino e il lanciatore destrimano, come pure il primo lanciatore, il lanciatore di un solo inning e il lanciatore di più inning.

[9] ***Sabermetrics*** è l'applicazione di analisi statistica che elabora i dati sui giocatori di baseball. Offre vari modi di confrontare i valori relativi dei singoli giocatori.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995O3016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Note

Lo studio sistematico della programmazione dinamica è stato avviato da R. Bellman nel 1955. In questo ambito e anche nella programmazione lineare, la parola "programmazione" fa riferimento all'impiego di un metodo di risoluzione dei problemi basato sulle tabelle. Sebbene fossero già note alcune tecniche di ottimizzazione che incorporavano elementi della programmazione dinamica, Bellman ha introdotto in questa materia una solida teoria matematica [37].

Galil e Park [126] classificano gli algoritmi di programmazione dinamica in base alla dimensione della tabella e al numero di altri elementi da cui dipende ciascun elemento della tabella. Essi chiamano un algoritmo di programmazione dinamica tD/eD se la dimensione della sua tabella è $O(n^t)$ e ciascun elemento dipende da $O(n^e)$ altri elementi. Per esempio, l'algoritmo per moltiplicare una sequenza di matrici (Paragrafo 15.2) sarebbe $2D/1D$, e l'algoritmo per trovare la più lunga sottosequenza comune (Paragrafo 15.4) sarebbe $2D/0D$.

Hu e Shing [183, 184] hanno creato un algoritmo con tempo $O(n \lg n)$ per risolvere il problema della moltiplicazione di una sequenza di matrici.

L'algoritmo con tempo $O(mn)$ per risolvere il problema della più lunga sottosequenza comune ha suscitato l'interesse di molti studiosi. Knuth [71] si è domandato se esistessero algoritmi subquadratici per il problema della LCS. Masek e Paterson [245] hanno risposto affermativamente a questa domanda, fornendo un algoritmo che viene eseguito nel tempo $O(mn/\lg n)$, dove $n \leq m$ e le sequenze sono estratte da un insieme di dimensione limitata. Nel caso speciale in cui nessun elemento appaia più di una volta in una sequenza di input, Szymanski [327] ha dimostrato che il problema può essere risolto nel tempo $O((n+m)\lg(n+m))$. Molti di questi risultati possono essere estesi al problema del calcolo della distanza di editing delle stringhe (Problema 15-5).

Un vecchio articolo di Gilbert e Moore [134] sulla codifica binaria a lunghezza variabile descrive delle applicazioni per costruire alberi binari di ricerca ottimi per il caso in cui tutte le probabilità p_i siano 0; questo articolo contiene un algoritmo con tempo $O(n^3)$. Aho, Hopcroft e Ullman [5] hanno sviluppato l'algoritmo descritto nel Paragrafo 15.5. L'Esercizio 15.5-4 è tratto da un articolo di Knuth [213]. Hu e Tucker [185] hanno ideato un algoritmo per il caso in cui tutte le probabilità p_i siano 0, che impiega un tempo $O(n^2)$ e uno spazio $O(n)$; successivamente, Knuth [212] ha ridotto il tempo a $O(n \lg n)$.

Il Problema 15-8 è stato ideato da Avidan e Shamir [27], che hanno pubblicato sul web un video molto interessante che illustra questa tecnica di compressione delle immagini.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

16 Algoritmi golosi

Gli algoritmi per i problemi di ottimizzazione, tipicamente, eseguono una sequenza di passi, con una serie di scelte a ogni passo. Per molti problemi di ottimizzazione è uno spreco applicare le tecniche della programmazione dinamica per effettuare le scelte migliori; è preferibile utilizzare algoritmi più semplici ed efficienti. Un ***algoritmo goloso*** (greedy) fa sempre la scelta che sembra ottima in un determinato momento, ovvero fa una scelta *localmente ottima*, nella speranza che tale scelta porterà a una soluzione globalmente ottima. Questo capitolo esamina i problemi di ottimizzazione che possono essere risolti con gli algoritmi golosi. Prima di leggere questo capitolo, vi consigliamo di consultare il Capitolo 15 (in particolare il Paragrafo 15.3) che tratta la programmazione dinamica.

Gli algoritmi golosi non sempre riescono a trovare le soluzioni ottime, ma per molti problemi sono in grado di farlo. Nel Paragrafo 16.1 esamineremo un problema semplice, ma non banale – il problema della selezione di attività – per il quale un algoritmo goloso calcola in modo efficiente una soluzione. Arriveremo all'algoritmo goloso considerando prima una soluzione mediante la programmazione dinamica e poi dimostrando che è sempre possibile fare delle scelte golose per arrivare a una soluzione ottima.

Il Paragrafo 16.2 tratta gli elementi fondamentali della strategia golosa, fornendo un approccio diretto per provare la correttezza degli algoritmi golosi. Il Paragrafo 16.3 presenta un'importante applicazione delle tecniche golose: il progetto dei codici per la compressione dei dati (codici di Huffman). Nel Paragrafo 16.4 descriveremo alcuni concetti teorici che stanno alla base dei "matroidi", che sono particolari strutture combinatorie per le quali un algoritmo goloso fornisce sempre una soluzione ottima. Infine, il Paragrafo 16.5 illustra un'applicazione dei matroidi per risolvere un problema di programmazione di lavori di durata unitaria con scadenze e penalità.

Il metodo goloso è molto potente e può essere applicato a una vasta gamma di problemi. I successivi capitoli presenteranno molti algoritmi che possono essere visti come applicazioni del metodo goloso, inclusi gli algoritmi per gli alberi di connessione minimi (Capitolo 23), l'algoritmo di Dijkstra per il problema dei cammini minimi da sorgente unica (Capitolo 24) e l'euristica golosa di Chvátal per la copertura di insiemi (Capitolo 35). Gli algoritmi per gli alberi di connessione minimi sono applicazioni classiche del metodo goloso. Sebbene questo capitolo e il Capitolo 23 possano essere letti indipendentemente l'uno dall'altro, potrebbe essere utile leggerli insieme.

Acquistato da Michele Michele su Webster il 2022-0[illegible]

16.1 Problema della selezione di attività

Il nostro primo esempio è il problema della programmazione di più attività in competizione che richiedono l'uso esclusivo di una risorsa comune, con l'obiettivo di selezionare il più grande insieme di attività mutuamente compatibili. Supponiamo di avere un insieme $S = \{a_1, a_2, \ldots, a_n\}$ di n ***attività*** che devono utilizzare la stessa risorsa, per esempio un'aula universitaria, che può essere utilizzata per svolgere una sola attività alla volta. Ogni attività a_i ha un ***tempo di inizio*** s_i e un ***tempo di fine*** f_i, con $0 \leq s_i < f_i < \infty$. Quando viene selezionata, l'attività a_i si svolge durante l'intervallo semiaperto $[s_i, f_i)$. Le attività a_i e a_j sono ***compatibili*** se gli intervalli $[s_i, f_i)$ e $[s_j, f_j)$ non si sovrappongono (ovvero a_i e a_j sono compatibili se $s_i \geq f_j$ o $s_j \geq f_i$). Il ***problema della selezione di attività*** consiste nel selezionare il sottoinsieme che contiene il maggior numero di attività mutuamente compatibili. Supponiamo che le attività siano ordinate in modo monotonicamente crescente rispetto ai tempi di fine (capirete presto perché è conveniente considerare le attività ordinate in questo modo):

$$f_1 \leq f_2 \leq f_3 \leq \cdots \leq f_{n-1} \leq f_n \,. \tag{16.1}$$

Per esempio, consideriamo il seguente insieme S di attività:

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

Per questo esempio, il sottoinsieme $\{a_3, a_9, a_{11}\}$ è formato da attività mutuamente compatibili, ma non è il sottoinsieme di dimensione massima, perché il sottoinsieme $\{a_1, a_4, a_8, a_{11}\}$ è più grande. In effetti, $\{a_1, a_4, a_8, a_{11}\}$ è un sottoinsieme massimo di attività mutuamente compatibili; un altro sottoinsieme massimo è $\{a_2, a_4, a_9, a_{11}\}$.

Il problema sarà risolto in vari passaggi. Iniziamo pensando ad una soluzione con la programmazione dinamica in cui consideriamo varie scelte per determinare quali sottoproblemi utilizzare in una soluzione ottima. Poi, vedremo che sarà sufficiente considerare una sola scelta – la scelta golosa – e che, facendo questa scelta golosa, rimane un solo sottoproblema da risolvere. Sulla base di queste considerazioni, svilupperemo un algoritmo goloso ricorsivo per risolvere il problema della programmazione delle attività. Completeremo il processo di sviluppo di una soluzione golosa trasformando l'algoritmo ricorsivo in uno iterativo. Sebbene i passaggi che descriveremo in questo paragrafo siano più complicati del necessario per sviluppare un algoritmo goloso, tuttavia essi illustrano la relazione tra algoritmi golosi e programmazione dinamica.

La sottostruttura ottima del problema della selezione di attività

Possiamo facilmente verificare che il problema della selezione di attività presenta una sottostruttura ottima. Indichiamo con S_{ij} l'insieme delle attività che iniziano dopo la fine dell'attività a_i e finiscono prima dell'inizio dell'attività a_j. Supponiamo di voler trovare un insieme massimo di attività mutuamente compatibili in S_{ij}; supponiamo inoltre che A_{ij} sia un insieme massimo che include una certa attività a_k. Includendo a_k in una soluzione ottima, restano due sottoproblemi: trovare le attività mutuamente compatibili nell'insieme S_{ik} (le attività che iniziano dopo la fine di a_i e finiscono prima dell'inizio di a_k) e trovare le attività mutua-

Acquistato da Michele Mule' (mule.michele@gmail.com) in data 11/02/2022 alle 23:13:32 con licenza 2858 - Copyright © 2010 The McGraw-Hill Education (Italy)

mente compatibili nell'insieme S_{kj} (le attività che iniziano dopo la fine di a_k e finiscono prima dell'inizio di a_j). Siano $A_{ik} = A_{ij} \cap S_{ik}$ e $A_{kj} = A_{ij} \cap S_{kj}$, in modo che A_{ik} contenga le attività in A_{ij} che finiscono prima dell'inizio di a_k e che A_{kj} contenga le attività in A_{ij} che iniziano dopo la fine di a_k. Ne deriva che $A_{ij} = A_{ik} \cup \{a_k\} \cup A_{kj}$ e, quindi, l'insieme massimo A_{ij} di attività mutuamente compatibili in S_{ij} è formato da $|A_{ij}| = |A_{ik}| + |A_{kj}| + 1$ attività.

Applicando il ragionamento "taglia e incolla", si ha che la soluzione ottima A_{ij} deve includere anche le soluzioni ottime dei due sottoproblemi per S_{ik} e S_{kj}. Se trovassimo un insieme A'_{kj} di attività mutuamente compatibili in S_{kj} tale che $|A'_{kj}| > |A_{kj}|$, allora potremmo utilizzare A'_{kj}, anziché A_{kj}, in una soluzione del sottoproblema per S_{ij}. Avremmo potuto costruire un insieme di $|A_{ik}| + |A'_{kj}| + 1 > |A_{ik}| + |A_{kj}| + 1 = |A_{ij}|$ attività mutuamente compatibili, che contraddice l'ipotesi che A_{ij} sia una soluzione ottima. Un ragionamento simmetrico si applica alle attività in S_{ik}.

Questo modo di caratterizzare la sottostruttura ottima ci suggerisce di risolvere il problema della selezione di attività tramite la programmazione dinamica. Se indichiamo con $c[i, j]$ la dimensione di una soluzione ottima per l'insieme S_{ij}, otteniamo la ricorrenza

$$c[i, j] = c[i, k] + c[k, j] + 1$$

Ovviamente, se non sapessimo che una soluzione ottima per l'insieme S_{ij} include l'attività a_k, dovremmo esaminare tutte le attività in S_{ij} per trovare quella da scegliere, pertanto

$$c[i, j] = \begin{cases} 0 & \text{se } S_{ij} = \emptyset \\ \max\limits_{a_k \in S_{ij}} \{c[i, k] + c[k, j] + 1\} & \text{se } S_{ij} \neq \emptyset \end{cases} \tag{16.2}$$

A questo punto, potremmo sviluppare una algoritmo ricorsivo con annotazione oppure potremmo applicare il metodo bottom-up e riempire una tabella man mano che procediamo. Tuttavia, vogliamo prendere in esame un'altra importante caratteristica del problema della selezione di attività che potremo utilizzare con grande vantaggio.

Fare la scelta golosa

Che cosa accadrebbe se potessimo scegliere l'attività da aggiungere alla nostra soluzione ottima senza dover risolvere prima tutti i sottoproblemi? Ciò ci eviterebbe di dover considerare tutte le scelte previste dalla ricorrenza (16.2). In effetti, per il problema della selezione di attività, occorre considerare una sola scelta: la scelta golosa.

Che cosa intendiamo con scelta golosa nel problema della selezione di attività? Ora, tra le attività che possiamo scegliere, ce ne deve essere una che finisce per prima. L'intuito ci dice, quindi, di scegliere l'attività in S che finisce per prima, perché così la risorsa resterebbe disponibile per il maggior numero possibile di attività successive. (Se più attività in S hanno il primo tempo di fine, scegliamo una qualsiasi di queste attività.) In altre parole, poiché le attività sono ordinate in modo monotonicamente crescente rispetto ai tempi di fine, la scelta golosa è l'attività a_1. Scegliere l'attività che finisce per prima non è l'unico modo di fare una scelta golosa per questo problema; l'Esercizio 16.1-3 chiede di analizzare altri modi.

Acquistato da Michele Michele su Webster il 2022-07-21 - Copia non autorizzata - Licenza non trasferibile - Diritti riservati

Se facciamo la scelta golosa, ci resta un solo sottoproblema da risolvere: trovare le attività che iniziano dopo la fine di a_1. Perché non dobbiamo considerare le attività che finiscono prima dell'inizio di a_1? Poiché $s_1 < f_1$ ed f_1 è il primo tempo di fine di qualsiasi attività, allora nessuna attività può avere un tempo di fine minore o uguale a s_1. Quindi, tutte le attività che sono compatibili con l'attività a_1 devono iniziare dopo la fine di a_1.

Inoltre, abbiamo già stabilito che il problema della selezione di attività presenta una sottostruttura ottima. Sia $S_k = \{a_i \in S : s_i \geq f_k\}$ l'insieme delle attività che iniziano dopo la fine dell'attività a_k. Se facciamo la scelta golosa dell'attività a_1, S_1 resta l'unico sottoproblema da risolvere.[1] La sottostruttura ottima ci dice che se a_1 appartiene alla soluzione ottima, allora una soluzione ottima del problema originale è formata dall'attività a_1 e da tutte le attività in una soluzione ottima del sottoproblema S_1.

Resta un'importante domanda: la nostra intuizione è corretta? La scelta golosa (nella quale abbiamo scelto l'attività che finisce per prima) fa sempre parte di qualche soluzione ottima? Il seguente teorema dimostra di sì.

Teorema 16.1
Consideriamo un sottoproblema non vuoto S_k e sia a_m l'attività in S_k che ha il primo tempo di fine; allora l'attività a_m è inclusa in qualche sottoinsieme massimo di attività mutuamente compatibili di S_k.

Dimostrazione Supponiamo che A_k sia un sottoinsieme massimo di attività mutuamente compatibili di S_k e sia a_j un'attività in A_k con il più piccolo tempo di fine. Se $a_j = a_m$, abbiamo finito, perché abbiamo dimostrato che l'attività a_m è utilizzata in qualche sottoinsieme massimo di attività mutuamente compatibili di S_k. Se $a_j \neq a_m$, sia $A'_k = A_k - \{a_j\} \cup \{a_m\}$ l'insieme A_k con la sostituzione di a_m con a_j. Le attività in A'_k sono disgiunte, perché lo sono le attività in A_k, a_j è l'attività che finisce per prima in A_k ed $f_m \leq f_j$. Poiché $|A'_k| = |A_k|$, concludiamo che A'_k è un sottoinsieme massimo di attività mutuamente compatibili di S_k che include a_m. ■

Notiamo quindi che, sebbene sia possibile risolvere il problema della selezione di attività con la programmazione dinamica, non è necessario farlo (fra l'altro, non abbiamo ancora esaminato se questo problema include dei sottoproblemi ripetuti). Possiamo invece selezionare ripetutamente l'attività che finisce per prima, mantenendo soltanto le attività compatibili con questa attività, e ripetendo il procedimento finché non resteranno altre attività. Inoltre, poiché scegliamo sempre l'attività con il primo tempo di fine, i tempi di fine delle attività che scegliamo devono essere strettamente crescenti. Possiamo considerare ciascuna attività una sola volta, se le attività sono monotonicamente crescenti rispetto ai tempi di fine.

Un algoritmo per risolvere il problema della selezione di attività non ha bisogno di operare dal basso verso l'alto (bottom-up) come un algoritmo di programmazione dinamica basato su tabelle. Esso può operare dall'alto verso il basso (top-down) scegliendo un'attività da inserire nella soluzione ottima e poi risolvendo il sotto-

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995063016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[1] A volte parleremo di insiemi S_k come sottoproblemi, anziché come insiemi di attività. Sarà sempre chiaro dal contesto se ci stiamo riferendo a S_k come a un insieme di attività o al sottoproblema il cui input è tale insieme.

problema di scegliere le attività tra quelle che sono compatibili con le attività già scelte. Tipicamente, gli algoritmi golosi seguono questo schema top-down: fare una scelta e risolvere un sottoproblema, diversamente dalla tecnica bottom-up che risolve i sottoproblemi prima di fare una scelta.

Un algoritmo goloso ricorsivo

Dopo aver visto come evitare l'approccio della programmazione dinamica e utilizzare invece un algoritmo goloso top-down, siamo pronti a scrivere una semplice procedura ricorsiva per risolvere il problema della selezione di attività. La procedura RECURSIVE-ACTIVITY-SELECTOR riceve i tempi di inizio e fine delle attività, rappresentati come array s ed f,[2] l'indice k che definisce il sottoproblema S_k da risolvere e la dimensione n del problema originale. Restituisce un insieme massimo di attività mutuamente compatibili in S_k. Supponiamo che le n attività di input siano già ordinate in modo monotonicamente crescente rispetto ai tempi di fine, secondo l'equazione (16.1). Se non lo fossero, possiamo ordinarle in tal modo nel tempo $O(n \lg n)$, risolvendo in maniera arbitraria il caso di due tempi di fine uguali. Per iniziare, aggiungiamo l'attività fittizia a_0 con $f_0 = 0$, in modo che il sottoproblema S_0 rappresenti tutto l'insieme delle attività S. La chiamata iniziale, che risolve il problema intero, è RECURSIVE-ACTIVITY-SELECTOR$(s, f, 0, n)$.

```
RECURSIVE-ACTIVITY-SELECTOR(s, f, k, n)
1  m = k + 1
2  while m ≤ n and s[m] < f[k]        // Trova la prima attività in S_k
3       m = m + 1
4  if m ≤ n
5       return {a_m} ∪ RECURSIVE-ACTIVITY-SELECTOR(s, f, m, n)
6  else return ∅
```

La Figura 16.1 illustra il funzionamento dell'algoritmo. In una chiamata ricorsiva di RECURSIVE-ACTIVITY-SELECTOR(s, f, k, n), il ciclo **while** (righe 2–3) cerca la prima attività in S_k. Il ciclo esamina $a_{k+1}, a_{k+2}, \ldots, a_n$, finché non trova la prima attività a_m che è compatibile con a_k; tale attività ha $s_m \geq f_k$. Se il ciclo termina perché ha trovato tale attività, la procedura restituisce (riga 5) l'unione di $\{a_m\}$ e del sottoinsieme massimo di S_m restituito dalla chiamata ricorsiva di RECURSIVE-ACTIVITY-SELECTOR(s, f, m, n). In alternativa, il ciclo può terminare perché $m > n$, nel qual caso abbiamo esaminato tutte le attività in S_k senza trovarne una compatibile con a_k. In questo caso $S_k = \emptyset$ e, quindi, la procedura restituisce $\emptyset$ (riga 6).

Supponendo che le attività siano già ordinate rispetto ai loro tempi di fine, il tempo di esecuzione di RECURSIVE-ACTIVITY-SELECTOR$(s, f, 0, n)$ è $\Theta(n)$, come si vede facilmente. In tutte le chiamate ricorsive, ogni attività viene esaminata esattamente una sola volta nel test del ciclo **while** (riga 2). In particolare, l'attività a_i viene esaminata nell'ultima chiamata in cui $k < i$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[2] Poiché lo pseudocodice riceve s ed f come array, i loro indici vengono rappresentati tra parentesi quadre.

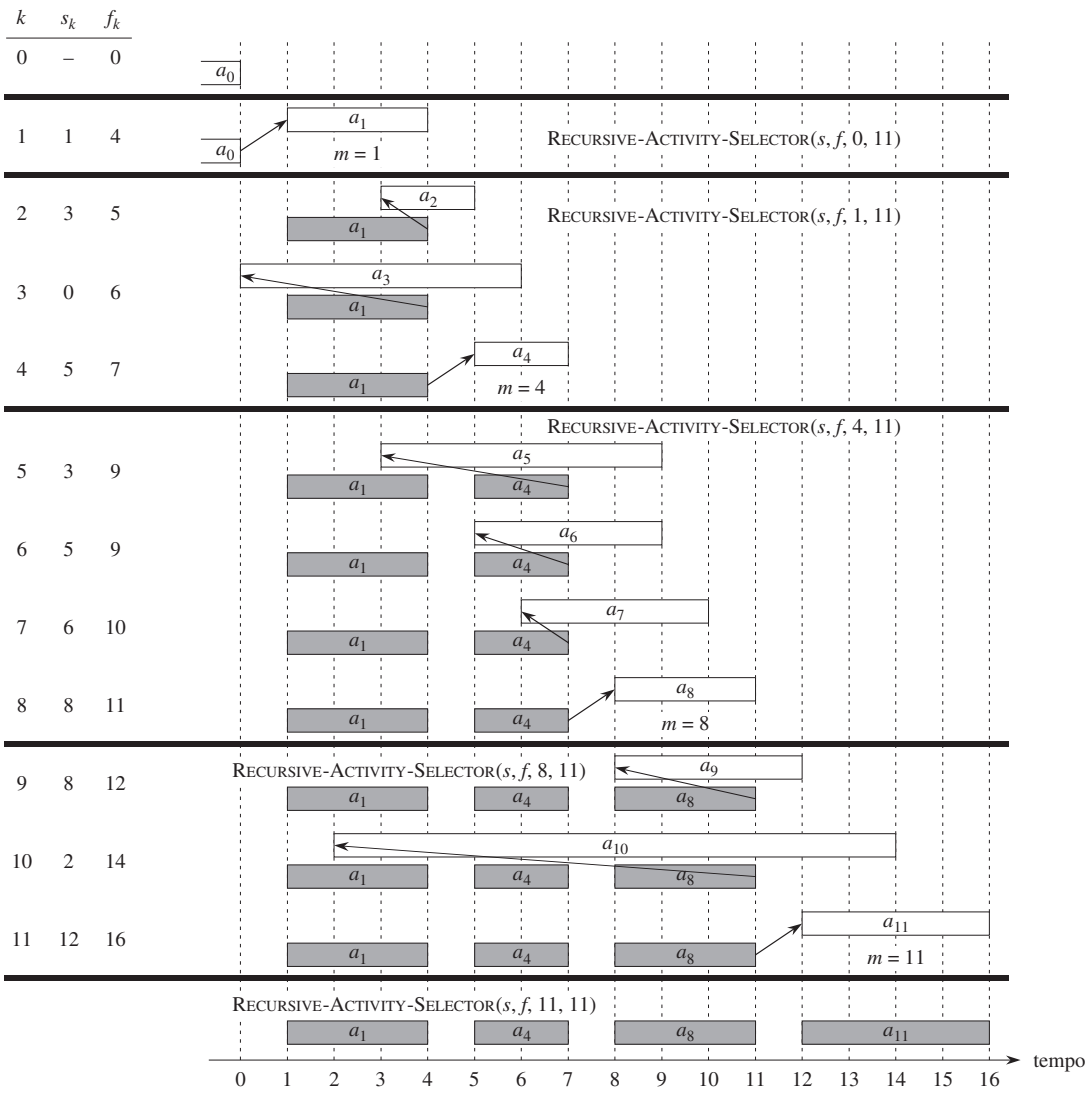


Figura 16.1 Il funzionamento di RECURSIVE-ACTIVITY-SELECTOR con le 11 attività date in precedenza. Le attività considerate in ciascuna chiamata ricorsiva sono rappresentate fra le righe orizzontali. L'attività fittizia a_0 finisce nel tempo 0 e, nella chiamata iniziale, RECURSIVE-ACTIVITY-SELECTOR(s, f, 0, 11), viene selezionata l'attività a_1. In ogni chiamata ricorsiva, le attività già selezionate sono su sfondo grigio; l'attività su sfondo bianco è quella in esame. Se un'attività ha il tempo di inizio che precede il tempo di fine dell'ultima attività che è stata inserita (la freccia fra le attività punta a sinistra) viene scartata, altrimenti (la freccia punta direttamente in alto o a destra) viene selezionata. L'ultima chiamata ricorsiva, RECURSIVE-ACTIVITY-SELECTOR(s, f, 11, 11), restituisce $\emptyset$. L'insieme risultante delle attività selezionate è $\{a_1, a_4, a_8, a_{11}\}$.

Un algoritmo goloso iterativo

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Possiamo facilmente convertire la nostra procedura ricorsiva in una iterativa. La procedura RECURSIVE-ACTIVITY-SELECTOR è quasi "ricorsiva in coda" (vedere il Problema 7-4): termina con una chiamata ricorsiva a sé stessa seguita da un'operazione di unione. Di solito, è semplice trasformare una procedura con ricorsione in coda in una forma iterativa; in effetti, i compilatori di alcuni linguaggi di programmazione effettuano questa trasformazione automaticamente. Come detto in

precedenza, la procedura RECURSIVE-ACTIVITY-SELECTOR elabora i sottoproblemi S_k, ovvero i sottoproblemi che sono formati dalle attività che finiscono per ultime.

La procedura GREEDY-ACTIVITY-SELECTOR è una versione iterativa della procedura RECURSIVE-ACTIVITY-SELECTOR. Si suppone che le attività di input siano ordinate in modo monotonicamente crescente rispetto ai tempi di fine. La procedura raccoglie le attività selezionate in un insieme A e restituisce questo insieme quando ha finito.

```
GREEDY-ACTIVITY-SELECTOR(s, f)
1  n = s.length
2  A = {a_1}
3  k = 1
4  for m = 2 to n
5      if s[m] ≥ f[k]
6          A = A ∪ {a_m}
7          k = m
8  return A
```

La procedura funziona in questo modo. La variabile k indicizza l'ultimo inserimento in A, che corrisponde all'attività a_k nella versione ricorsiva. Poiché le attività vengono considerate in base all'ordine monotonicamente crescente dei loro tempi di fine, f_k è sempre il massimo tempo di fine di qualsiasi attività in A; ovvero

$$f_k = \max\{f_i : a_i \in A\} \tag{16.3}$$

Le righe 2–3 selezionano l'attività a_1, inizializzano A in modo che contenga proprio questa attività e inizializzano k per indicizzare questa attività. Il ciclo **for** (righe 4–7) trova l'attività che finisce per prima in S_k. Il ciclo esamina a turno ogni attività a_m e inserisce a_m in A, se è compatibile con tutte le attività precedentemente selezionate; questa attività è quella che finisce per prima in S_k. Per vedere se l'attività a_m è compatibile con tutte le attività che si trovano correntemente in A, in base all'equazione (16.3) basta controllare (riga 5) che il suo tempo di inizio s_m non preceda il tempo di fine f_k dell'ultima attività che è stata inserita in A. Se l'attività a_m è compatibile, allora le righe 6–7 inseriscono l'attività a_m in A e impostano k a m. L'insieme A restituito dalla chiamata GREEDY-ACTIVITY-SELECTOR(s, f) è esattamente l'insieme restituito dalla chiamata RECURSIVE-ACTIVITY-SELECTOR$(s, f, 0, n)$.

Analogamente alla versione ricorsiva, GREEDY-ACTIVITY-SELECTOR programma un insieme di n attività nel tempo $\Theta(n)$, nell'ipotesi che le attività siano inizialmente ordinate rispetto ai loro tempi di fine.

Esercizi

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

16.1-1

Descrivete un algoritmo di programmazione dinamica per il problema della selezione di attività, facendo riferimento alla ricorrenza (16.2). L'algoritmo dovrebbe calcolare le dimensioni $c[i, j]$, definite in precedenza, e produrre il sottoinsieme massimo A delle attività. Supponete che gli input siano stati ordinati secondo l'equazione (16.1). Confrontate il tempo di esecuzione della vostra soluzione con il tempo di esecuzione di GREEDY-ACTIVITY-SELECTOR.

16.1-2
Anziché selezionare sempre l'attività che finisce per prima, supponete di selezionare quella che inizia per ultima tra quelle compatibili con tutte le attività precedentemente selezionate. Spiegate perché questo approccio è un algoritmo goloso e dimostrate che esso fornisce una soluzione ottima.

16.1-3
Non sempre un approccio goloso al problema della selezione di attività genera un insieme massimo di attività mutuamente compatibili. Descrivete un esempio per dimostrare che la tecnica che sceglie l'attività di durata minima fra quelle che sono compatibili con le attività precedentemente selezionate non funziona. Fate la stessa cosa per le tecniche che scelgono sempre l'attività compatibile che si sovrappone al minor numero di attività restanti o che scelgono sempre l'attività compatibile che inizia per prima.

16.1-4
Supponete di avere un insieme di attività che devono essere svolte in un certo numero di aule universitarie. Bisogna programmare tutte le attività utilizzando il minor numero possibile di aule. Descrivete un algoritmo goloso efficiente che determina come associare le attività alle aule.

Questo problema è noto come il ***problema della colorazione di un grafo di intervalli***. È possibile creare un grafo di intervalli i cui vertici sono le attività date e i cui archi collegano le attività incompatibili. Trovare il numero minimo di colori necessari per colorare ogni vertice in modo che due vertici adiacenti non possano avere lo stesso colore equivale a trovare il numero minimo di aule universitarie necessarie a programmare le attività.

16.1-5
Considerate una modifica del problema della selezione di attività in cui ciascuna attività a_i ha, oltre ai tempi di inizio e fine, un valore v_i. L'obiettivo non è più massimizzare il numero, ma il valore totale delle attività programmate. Ovvero, bisogna scegliere un insieme A di attività compatibili tale che $\sum_{a_k \in A} v_k$ sia massimizzato. Scrivete un algoritmo con tempo polinomiale per questo problema.

16.2 Elementi della strategia golosa

Un algoritmo goloso produce una soluzione ottima di un problema effettuando una sequenza di scelte. Ad ogni punto di decisione l'algoritmo fa la scelta che in quel momento sembra la migliore. Questa strategia euristica non sempre produce una soluzione ottima ma, come abbiamo visto nel problema della selezione di attività, a volte ci riesce. Questo paragrafo descrive alcune delle proprietà generali dei metodi golosi.

Il procedimento che abbiamo seguito nel Paragrafo 16.1 per sviluppare un algoritmo goloso è stato un po' più complesso del consueto. Abbiamo compiuto i seguenti passi:

Acquistato da Michele Mastrangelo - Vivadue il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

1. Determinare la sottostruttura ottima del problema.
2. Sviluppare una soluzione ricorsiva (per il problema della selezione di attività abbiamo formulato la ricorrenza (16.2), evitando però di sviluppare un algoritmo ricorsivo basato su questa ricorrenza.)
3. Dimostrare che se facciamo la scelta golosa, resta un solo sottoproblema.

4. Dimostrare che la scelta golosa è sempre sicura.
5. Sviluppare un algoritmo ricorsivo che implementa la strategia golosa.
6. Convertire l'algoritmo ricorsivo in un algoritmo iterativo.

Nel compiere questi passi, abbiamo esaminato dettagliatamente gli elementi della programmazione dinamica su cui si basa un algoritmo goloso. Per esempio, nel problema della selezione di attività, prima abbiamo definito i sottoproblemi S_{ij}, dove variavano entrambi gli indici i e j. Poi, abbiamo scoperto che, se avessimo fatto sempre la scelta golosa, avremmo potuto limitare i sottoproblemi alla forma S_k.

In alternativa, avremmo potuto modellare la nostra sottostruttura ottima avendo in mente una scelta golosa, in modo che la scelta lasci un solo sottoproblema da risolvere. Nel problema della selezione di attività, avremmo potuto scartare il secondo indice e definire i sottoproblemi della forma S_k. Poi, avremmo potuto dimostrare che una scelta golosa (l'attività a_m che finisce per prima in S_k), combinata con una soluzione ottima dell'insieme residuo S_m di attività compatibili, produce una soluzione ottima di S_k. Più in generale, progettiamo gli algoritmi golosi secondo questa sequenza di passi:

1. Esprimere il problema di ottimizzazione in una forma in cui, fatta una scelta, resta un solo sottoproblema da risolvere.
2. Dimostrare che esiste sempre una soluzione ottima del problema originale che fa la scelta golosa, quindi la scelta golosa è sempre sicura.
3. Dimostrare la sottostruttura ottima verificando che, dopo aver fatto la scelta golosa, ciò che resta è un sottoproblema con la proprietà che, se combiniamo una soluzione ottima del sottoproblema con la scelta golosa che abbiamo fatto, arriviamo a una soluzione ottima del problema originale.

Utilizzeremo questo procedimento più diretto nei paragrafi successivi di questo capitolo. Nonostante ciò, sotto ogni algoritmo goloso c'è quasi sempre una soluzione più onerosa di programmazione dinamica.

Come si fa a dire se un algoritmo goloso risolverà un particolare problema di ottimizzazione? Non c'è un metodo generale, ma la proprietà della scelta golosa e la sottostruttura ottima sono i due ingredienti chiave. Se dimostriamo che un problema soddisfa queste proprietà, allora siamo sulla strada buona per sviluppare un algoritmo goloso per questo problema.

La proprietà della scelta gelosa

Il primo ingrediente chiave è la ***proprietà della scelta golosa***: una soluzione globalmente ottima può essere ottenuta facendo una scelta (greedy) localmente ottima. In altre parole, quando valutiamo la scelta da fare, facciamo la scelta che sembra migliore per il problema corrente, senza considerare le soluzioni dei sottoproblemi.

Acquistato da Michele Michele su Webster il 2022-07-07 22:12 - Numero licenza: 1000002818220707 - Copyright 2022, McGraw-Hill Education (Italy)

È qui che gli algoritmi golosi differiscono dalla programmazione dinamica. Nella programmazione dinamica, facciamo una scelta a ogni passo, ma la scelta di solito dipende dalle soluzioni dei sottoproblemi. Di conseguenza, tipicamente risolviamo i problemi di programmazione dinamica secondo uno schema bottom-up, elaborando prima i sottoproblemi più piccoli per arrivare a quelli più grandi.

(In alternativa, possiamo risolverli secondo uno schema top-down, ma con annotazione. Ovviamente, anche se il codice opera dall'alto verso il basso, dobbiamo ugualmente risolvere i sottoproblemi prima di fare una scelta.) In un algoritmo goloso, facciamo la scelta che sembra migliore in un determinato momento e, poi, risolviamo il sottoproblema che deriva da tale scelta. La scelta fatta da un algoritmo goloso può dipendere dalle precedenti scelte, ma non può dipendere dalle scelte future o dalle soluzioni dei sottoproblemi. Quindi, diversamente dalla programmazione dinamica, che risolve i sottoproblemi prima di fare la prima scelta, un algoritmo goloso fa la sua prima scelta prima di risolvere qualsiasi sottoproblema. Un algoritmo di programmazione dinamica procede dal basso verso l'alto, mentre una strategia golosa di solito procede dall'alto verso il basso, facendo una scelta golosa dopo l'altra e riducendo ogni istanza del problema a una più piccola.

Ovviamente, dobbiamo dimostrare che facendo la scelta golosa a ogni passo si arriva a una soluzione globalmente ottima. Generalmente, come nel Teorema 16.1, la dimostrazione considera una soluzione globalmente ottima di un sottoproblema. Poi si mostra come la soluzione ottima possa essere modificata sostituendo la scelta golosa al posto di un'altra scelta, ottenendo un sottoproblema simile, ma più piccolo.

Di solito la scelta golosa si effettua in modo più efficiente che se dovessimo prendere in considerazione un numero maggiore di scelte. Per esempio, nel problema della selezione di attività, supponendo che le attività siano già ordinate in modo monotonicamente crescente rispetto ai tempi di fine, dobbiamo esaminare ciascuna attività soltanto una volta. Spesso, grazie a un'elaborazione preliminare dell'input o all'impiego di una struttura dati appropriata (spesso una coda di priorità), riusciamo a fare rapidamente delle scelte golose, ottenendo così un algoritmo efficiente.

Sottostruttura ottima

Un problema ha una ***sottostruttura ottima*** se una soluzione ottima del problema contiene al suo interno soluzioni ottime dei sottoproblemi. Questa proprietà è un ingrediente chiave per l'applicabilità della programmazione dinamica e degli algoritmi golosi.

Come esempio di sottostruttura ottima, ricordiamo come abbiamo dimostrato nel Paragrafo 16.1 che, se una soluzione ottima del sottoproblema S_{ij} include un'attività a_k, allora essa deve contenere anche soluzioni ottime dei sottoproblemi S_{ik} e S_{kj}. Data questa sottostruttura ottima, abbiamo concluso che, se conoscessimo quale attività utilizzare come a_k, potremmo costruire una soluzione ottima di S_{ij}, selezionando a_k e tutte le attività nelle soluzioni ottime dei sottoproblemi S_{ik} e S_{kj}. Sulla base di questa osservazione sulla sottostruttura ottima, siamo stati in grado di concepire la ricorrenza (16.2) che descrive il valore di una soluzione ottima.

Di solito, utilizziamo un approccio più diretto alla sottostruttura ottima quando la applichiamo agli algoritmi golosi. Come detto in precedenza, ci concediamo il lusso di assumere di arrivare a un sottoproblema facendo la scelta golosa nel problema originale. Tutto ciò che dobbiamo fare è dedurre che una soluzione ottima del sottoproblema, combinata con la scelta golosa già fatta, genera una soluzione ottima del problema originale. Questo schema applica implicitamente l'induzione ai sottoproblemi per dimostrare che, facendo la scelta golosa a ogni passo, si ottiene una soluzione ottima.

Acquistato da Michele Mahabir (mahabir.2010@libero.it) - Non condivisibile - Licenza n. 2021/0/0/0

Confronto fra strategia golosa e programmazione dinamica

Poiché la proprietà della sottostruttura ottima è sfruttata sia dalla strategia golosa sia dalla programmazione dinamica, qualcuno potrebbe essere tentato di ottenere la soluzione di un problema con la programmazione dinamica, quando è sufficiente una soluzione golosa; qualcun altro, invece, potrebbe erroneamente pensare di trovare una soluzione golosa, quando in effetti è richiesta una soluzione di programmazione dinamica. Per illustrare le sottili differenze tra le due tecniche, analizziamo due varianti di un classico problema di ottimizzazione.

Il ***problema dello zaino 0-1*** può essere formulato nel modo seguente. Un ladro entra in un magazzino e trova n oggetti; l'i-esimo oggetto vale v_i dollari e pesa w_i libbre, dove v_i e w_i sono numeri interi. Il ladro vorrebbe realizzare il furto di maggior valore, ma il suo zaino non può sopportare un peso maggiore di W libbre (W è un intero). Quali oggetti dovrà prendere? (Questo problema è detto "problema dello zaino 0-1" perché ogni oggetto deve essere preso o lasciato; il ladro non può prendere una frazione di un oggetto né più volte lo stesso oggetto.)

Nel ***problema dello zaino frazionario***, le condizioni sono le stesse, con la differenza che il ladro può prendere frazioni di oggetti, anziché fare una scelta binaria (0-1) per ogni oggetto. Nel problema dello zaino 0-1, un oggetto può essere paragonato a un lingotto d'oro; nel problema dello zaino frazionario, invece, un oggetto può essere paragonato alla polvere d'oro.

Entrambi i problemi dello zaino presentano la proprietà della sottostruttura ottima. Per il problema 0-1, consideriamo il carico più prezioso che pesa al più W libbre. Se togliamo l'oggetto j da questo carico, la parte restante deve essere il carico più prezioso, il cui peso non supera $W - w_j$ libbre, che il ladro può realizzare con gli $n - 1$ oggetti originali, escludendo l'oggetto j. Per il problema dello zaino frazionario, consideriamo che, se togliamo dal carico ottimo un peso w di una parte dell'oggetto j, il restante carico deve essere il carico più prezioso, il cui peso non supera $W - w$, che il ladro può realizzare con gli $n - 1$ oggetti originali più $w_j - w$ libbre dell'oggetto j.

Sebbene i problemi siano simili, il problema dello zaino frazionario può essere risolto con una strategia golosa, il problema dello zaino 0-1 no. Per risolvere il problema frazionario, calcoliamo innanzi tutto il valore per unità di peso o *valore specifico* di ciascun oggetto: v_i / w_i. Adottando una strategia golosa, il ladro inizia a prendere la maggiore quantità possibile dell'oggetto che ha il più grande valore specifico. Se la scorta di questo oggetto si esaurisce e il ladro può ancora mettere qualcosa nello zaino, prende la maggiore quantità possibile dell'oggetto che ha il secondo più grande valore specifico e così via, finché non potrà più mettere altro nello zaino. Quindi, ordinando gli oggetti rispetto al valore specifico, l'algoritmo goloso viene eseguito nel tempo $O(n \lg n)$. L'Esercizio 16.2-1 vi chiederà di dimostrare che il problema dello zaino frazionario ha la proprietà della scelta golosa.

Per capire perché questa strategia non funziona con il problema dello zaino 0-1, esaminate l'esempio illustrato nella Figura 16.2(a). Ci sono 3 oggetti e lo zaino può contenere al massimo 50 libbre. L'oggetto 1 pesa 10 libbre e vale 60 dollari. L'oggetto 2 pesa 20 libbre e vale 100 dollari. L'oggetto 3 pesa 30 libbre e vale 120 dollari. Quindi, il valore specifico dell'oggetto 1 è 6 dollari/libbra, che è maggiore del valore specifico dell'oggetto 2 (5 dollari/libbra) e dell'oggetto 3

Acquistato da Michele Michele su Webster il 2022-01-17 2

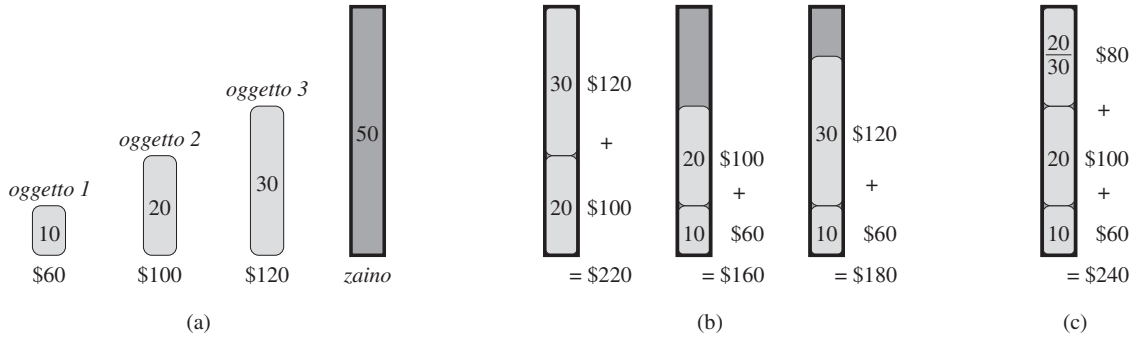


Figura 16.2 La strategia golosa non funziona per il problema dello zaino 0-1. **(a)** Il ladro deve selezionare un sottoinsieme dei tre oggetti illustrati il cui peso non può superare 50 libbre. **(b)** Il sottoinsieme ottimo include gli oggetti 2 e 3. Qualsiasi soluzione con l'oggetto 1 non è ottima, anche se questo oggetto ha il più grande valore specifico. **(c)** Per il problema dello zaino frazionario, si ottiene una soluzione ottima, selezionando gli oggetti in base al più grande valore specifico.

(4 dollari/libbra). Secondo la strategia golosa, quindi, il ladro dovrebbe prendere prima l'oggetto 1. Come potete vedere dall'analisi dei casi nella Figura 16.2(b), però, la soluzione ottima include gli oggetti 2 e 3, non l'oggetto 1. Le due possibili soluzioni che includono l'oggetto 1 non sono ottime.

Nel problema dello zaino frazionario, invece, la strategia golosa, che prevede che il ladro prenda prima l'oggetto 1, fornisce una soluzione ottima, come illustra la Figura 16.2(c). Nel problema dello zaino 0-1, se il ladro prende l'oggetto 1 non riesce a riempire completamente il suo zaino e lo spazio vuoto abbassa il valore specifico reale della refurtiva. Nel problema dello zaino 0-1, quando consideriamo un oggetto da includere nello zaino, dobbiamo confrontare la soluzione del sottoproblema in cui l'oggetto viene incluso con la soluzione del sottoproblema in cui l'oggetto viene escluso, prima di potere fare la scelta. Il problema formulato in questo modo genera molti sottoproblemi ripetuti – una caratteristica peculiare della programmazione dinamica e, in effetti, il problema dello zaino 0-1 può essere risolto con la programmazione dinamica (vedere l'Esercizio 16.2-2).

Esercizi

16.2-1

Dimostrate che il problema dello zaino frazionario ha la proprietà della scelta golosa.

16.2-2

Descrivete un algoritmo di programmazione dinamica per trovare nel tempo $O(n\,W)$ una soluzione del problema dello zaino 0-1, dove n è il numero di oggetti e W è il peso massimo degli oggetti che il ladro può mettere nel suo zaino.

16.2-3

Supponete che nel problema dello zaino 0-1 l'ordine degli oggetti, quando vengono ordinati per pesi crescenti, sia lo stesso che si ottiene ordinandoli per valori decrescenti. Descrivete un algoritmo efficiente per trovare una soluzione ottima di questa variante del problema dello zaino; dimostrate che il vostro algoritmo è corretto.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-2287707-0 Copyright © 2022, McGraw-Hill Education (Italy)

16.2-4
Il professor Gecko ha sempre sognato di attraversare il North Dakota con i pattini in linea. Ha previsto di percorrere la strada statale U.S. 2, che parte da Grand Forks, al confine orientale con il Minnesota, e arriva a Williston, vicino al confine occidentale con il Montana. Il professore può portare due litri di acqua e può pattinare per m miglia prima di finire l'acqua. (Poiché il North Dakota è relativamente piano, il professore non deve preoccuparsi di bere più acqua nelle frazioni in salita che in quelle in discesa o in pianura.) Il professore partirà da Grand Forks con due litri di acqua. La sua mappa del North Dakota indica tutte le stazioni di servizio lungo la U.S. 2 in cui potrà rifornirsi di acqua e le distanze tra queste stazioni.

L'obiettivo del professore è minimizzare il numero di rifornimenti di acqua lungo la strada. Scrivete un metodo efficiente che consenta al professore di sapere in quali stazioni dovrà fare rifornimento di acqua. Dimostrate che la vostra strategia genera una soluzione ottima e specificate il suo tempo di esecuzione.

16.2-5
Descrivete un algoritmo efficiente che, dato un insieme $\{x_1, x_2, \ldots, x_n\}$ di punti sull'asse reale, determina il più piccolo insieme di intervalli chiusi di lunghezza unitaria che contiene tutti i punti dati. Dimostrate che il vostro algoritmo è corretto.

16.2-6 ⋆
Spiegate come risolvere il problema dello zaino frazionario nel tempo $O(n)$.

16.2-7
Supponete di avere due insiemi A e B, ciascuno contenente n interi positivi. Potete riordinare gli insiemi come preferite. Dopo averli riordinati, indicate con a_i l'i-esimo elemento dell'insieme A e con b_i l'i-esimo elemento dell'insieme B. Otterrete un compenso pari a $\prod_{i=1}^{n} {a_i}^{b_i}$. Descrivete un algoritmo che massimizza questo compenso. Dimostrate che il vostro algoritmo massimizza il compenso e calcolate il suo tempo di esecuzione.

16.3 I codici di Huffman

I codici di Huffman sono una tecnica molto diffusa ed efficace per comprimere i dati; risparmi dal 20% al 90% sono tipici, a seconda delle caratteristiche dei dati da comprimere. I dati vengono considerati come una sequenza di caratteri. L'algoritmo goloso di Huffman usa una tabella che contiene quante volte compare ciascun carattere (la sua frequenza) per realizzare un metodo ottimo per rappresentare ciascun carattere con una stringa binaria.

Supponiamo di avere un file di dati di 100 000 caratteri che vogliamo comprimere. Sappiamo che i caratteri nel file si presentano con le frequenze riportate nella Figura 16.3; ovvero nel file ci sono soltanto sei caratteri differenti e il carattere `a` si presenta 45 000 volte. Ci sono vari modi di rappresentare questo tipo di file di dati. Consideriamo il problema di progettare un ***codice binario di caratteri*** (o soltanto ***codice*** per semplicità) dove ciascun carattere è rappresentato da una stringa binaria unica, detta ***parola in codice***. Se utilizziamo un ***codice a lunghezza fissa***, occorrono 3 bit per rappresentare sei caratteri: `a` = 000, `b` = 001, ..., `f` = 101. Questo metodo richiede 300 000 bit per codificare tutto il file. C'è un metodo migliore?

Acquistato da Michele Michele su Webster il 2022-0[illegible]

	a	b	c	d	e	f
Frequenza (in migliaia)	45	13	12	16	9	5
Parola in codice a lunghezza fissa	000	001	010	011	100	101
Parola in codice a lunghezza variabile	0	101	100	111	1101	1100

Figura 16.3 Un problema di codifica dei caratteri. Un file di dati di 100 000 caratteri contiene soltanto i caratteri `a`–`f`, con le frequenze indicate. Se a ciascun carattere è assegnata una parola in codice di 3 bit, il file può essere codificato con 300 000 bit. Utilizzando il codice a lunghezza variabile indicato nella figura, il file può essere codificato con 224 000 bit.

Un ***codice a lunghezza variabile*** può fare molto meglio di un codice a lunghezza fissa, assegnando ai caratteri più frequenti delle parole in codice corte e ai caratteri meno frequenti delle parole in codice lunghe. La Figura 16.3 illustra questo tipo di codice; qui la stringa 0 (lunga 1 bit) rappresenta il carattere `a` e la stringa 1100 (lunga 4 bit) rappresenta il carattere `f`. Questo codice richiede

$$(45 \cdot 1 \ + \ 13 \cdot 3 \ + \ 12 \cdot 3 \ + \ 16 \cdot 3 \ + \ 9 \cdot 4 \ + \ 5 \cdot 4) \cdot 1000 = 224\,000 \text{ bit}$$

per rappresentare il file – un risparmio del 25% circa. In effetti, questo è un codice di caratteri ottimo per questo file, come vedremo più avanti.

Codici prefissi

In questo paragrafo consideriamo soltanto i codici in cui nessuna parola in codice è anche un prefisso di qualche altra parola in codice. Tali codici sono detti ***codici prefissi***.[3] È possibile dimostrare (anche se non lo faremo qui) che la compressione ottima dei dati che può essere realizzata con un codice di caratteri qualsiasi può essere sempre realizzata con un codice prefisso, quindi non perdiamo in generalità se limitiamo la nostra attenzione ai codici prefissi.

La codifica è sempre semplice per qualsiasi codice binario di caratteri; basta concatenare le parole in codice che rappresentano i singoli caratteri del file. Per esempio, con il codice prefisso a lunghezza variabile della Figura 16.3, la codifica del file con i 3 caratteri `abc` è $0 \cdot 101 \cdot 100 = 0101100$, dove il simbolo "$\cdot$" indica la concatenazione.

I codici prefissi sono convenienti perché semplificano la decodifica. Poiché nessuna parola in codice è prefisso di un'altra, la prima parola in codice di un file codificato è inequivocabile. Possiamo semplicemente identificare la prima parola in codice, riconvertirla nel carattere originale e ripetere il processo di decodifica per la parte restante del file codificato. Nell'esempio in esame, la stringa 001011101 viene interpretata univocamente come $0 \cdot 0 \cdot 101 \cdot 1101$, che viene decodificata come `aabe`.

Il processo di decodifica richiede una rappresentazione pratica del codice prefisso, in modo che la prima parola in codice possa essere facilmente staccata. Un albero binario le cui foglie sono i caratteri dati fornisce tale rappresentazione. Interpretiamo la parola in codice per un carattere come il cammino semplice dalla radice a quel carattere, dove 0 significa "raggiungi il figlio sinistro" e 1 signifi-

Acquistato da Michele Mastrolonardo il 2023-07-20 16:12:16 - Vietata la copia - Copyright © 2022 McGraw-Hill Education (Italy)

[3] Forse sarebbe più appropriato parlare di "codici senza prefissi"; tuttavia il termine "codici prefissi" è standard nella letteratura.

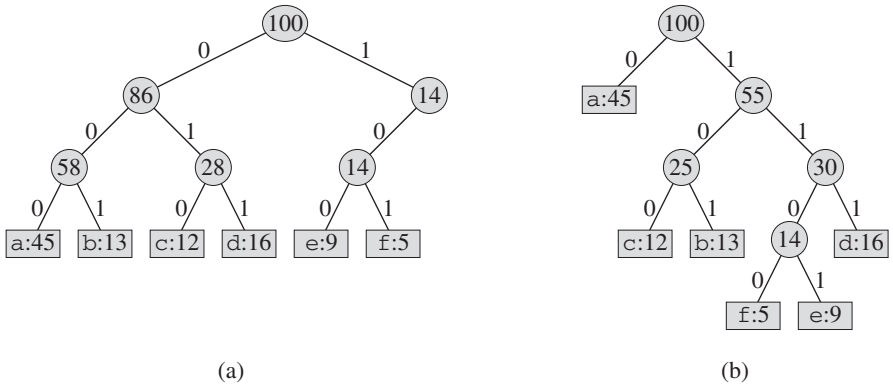


Figura 16.4 Gli alberi che corrispondono agli schemi di codifica della Figura 16.3. Ogni foglia è etichettata con un carattere e la sua frequenza di occorrenza. Ogni nodo interno è etichettato con la somma delle frequenze delle foglie nel suo sottoalbero. **(a)** L'albero che corrisponde al codice a lunghezza fissa `a` = 000, . . . , `f` = 101. **(b)** L'albero che corrisponde al codice prefisso ottimo `a` = 0, `b` = 101, . . . , `f` = 1100.

ca "raggiungi il figlio destro". La Figura 16.4 illustra gli alberi per i due codici del nostro esempio. Notate che questi non sono alberi binari di ricerca, perché le foglie non sono necessariamente ordinate e i nodi interni non hanno dei caratteri come chiave.

Un codice ottimo per un file è rappresentato sempre da un albero binario *pieno*, dove ogni nodo (tranne le foglie) ha due figli (vedere l'Esercizio 16.3-2). Il codice a lunghezza fissa nel nostro esempio non è ottimo, perché il suo albero, illustrato nella Figura 16.4(a), non è un albero binario pieno: ci sono parole in codice che iniziano con 10. . . , ma nessuna che inizia con 11. . . . Poiché adesso possiamo limitare l'analisi agli alberi binari pieni, possiamo dire che se C è l'alfabeto dal quale vengono estratti i caratteri e tutte le frequenze dei caratteri sono positive, allora l'albero per un codice prefisso ottimo ha esattamente $|C|$ foglie, una per ogni lettera dell'alfabeto, ed esattamente $|C| - 1$ nodi interni (vedere l'Esercizio B.5-3).

Dato un albero T che corrisponde a un codice prefisso, è semplice calcolare il numero di bit richiesti per codificare un file. Per ogni carattere c dell'alfabeto C, indichiamo con $c.freq$ la frequenza di c nel file e con $d_T(c)$ la profondità della foglia di c nell'albero. Notate che $d_T(c)$ è anche la lunghezza della parola in codice per il carattere c. Il numero di bit richiesti per codificare un file è quindi

$$B(T) = \sum_{c \in C} c.freq \cdot d_T(c) \tag{16.4}$$

che definiamo come il ***costo*** dell'albero T.

Costruire un codice di Huffman

Huffman ha ideato un algoritmo goloso che costruisce un codice prefisso ottimo detto ***codice di Huffman***. Conformemente alle osservazioni fatte nel Paragrafo 16.2, la dimostrazione della correttezza di questo algoritmo si basa sulla proprietà della scelta golosa e sulla sottostruttura ottima. Anziché dimostrare che queste proprietà sono soddisfatte e poi sviluppare uno pseudocodice, presentiamo prima lo pseudocodice. In questo modo, sarà più facile spiegare come l'algoritmo fa le scelte golose.

Acquistato da Michele Michele su Webster il 2022-[illegible] Non è consentito il riutilizzo, la copia o la distribuzione [illegible]

Nel seguente pseudocodice supponiamo che C sia un insieme di n caratteri e che ogni carattere $c \in C$ sia un oggetto con un attributo $c.freq$ che definisce la sua frequenza. L'algoritmo costruisce l'albero T che corrisponde al codice ottimo

secondo uno schema bottom-up. Inizia con un insieme di $|C|$ foglie ed esegue una sequenza di $|C| - 1$ "fusioni" per creare l'albero finale. Una coda di min-priorità Q, con chiave l'attributo $freq$, è utilizzata per identificare i due oggetti con frequenze minime da fondere insieme. Il risultato della fusione dei due oggetti è un nuovo oggetto la cui frequenza è la somma delle frequenze dei due oggetti originali.

```
HUFFMAN(C)
1  n = |C|
2  Q = C
3  for i = 1 to n − 1
4       Alloca un nuovo nodo z
5       z.left = x = EXTRACT-MIN(Q)
6       z.right = y = EXTRACT-MIN(Q)
7       z.freq = x.freq + y.freq
8       INSERT(Q, z)
9  return EXTRACT-MIN(Q)      // Restituisce la radice dell'albero
```

Nell'esempio in esame, l'algoritmo di Huffman procede come illustra la Figura 16.5. Poiché ci sono 6 lettere nell'alfabeto, la dimensione iniziale della coda è $n = 6$ e occorrono 5 fusioni per costruire l'albero. L'albero finale rappresenta il codice prefisso ottimo. La parola in codice per una lettera è la sequenza delle etichette degli archi che si incontrano lungo il cammino semplice che va dalla radice alla lettera.

La riga 2 inizializza la coda di min-priorità Q con i caratteri in C. Il ciclo **for** (righe 3–8) estrae ripetutamente i due nodi x e y di frequenza minima dalla coda e li sostituisce nella coda con un nuovo nodo z che rappresenta il risultato della loro fusione. La frequenza di z è calcolata come la somma delle frequenze di x e y nella riga 7. Il nodo z ha x come figlio sinistro e y come figlio destro (quest'ordine è arbitrario; scambiando i figli destro e sinistro di un nodo qualsiasi si ottiene un codice diverso, ma con lo stesso costo). Dopo $n - 1$ fusioni, l'unico nodo rimasto nella coda – la radice dell'albero del codice – viene restituito nella riga 9.

Anche se l'algoritmo produrrebbe lo stesso risultato se escludessimo le variabili x e y – facendo un'assegnazione diretta di $z.left$ e $z.right$ nelle righe 5 e 6, e modificando la riga 7 in $z.freq = z.left.freq + z.right.freq$ – noi utilizzeremo i nomi dei nodi x e y nella dimostrazione della correttezza. Quindi, per noi è più comodo lasciare x e y.

L'analisi del tempo di esecuzione dell'algoritmo di Huffman suppone che la coda Q sia implementata come un min-heap binario (vedere il Capitolo 6). Per un insieme C di n caratteri, l'inizializzazione di Q nella riga 2 può essere effettuata nel tempo $O(n)$ utilizzando la procedura BUILD-MIN-HEAP descritta nel Paragrafo 6.3. Il ciclo **for** (righe 3–8) viene eseguito esattamente $n - 1$ volte; considerando che ogni operazione con l'heap richiede un tempo $O(\lg n)$, il ciclo contribuisce con $O(n \lg n)$ al tempo di esecuzione. Quindi, il tempo totale di esecuzione della procedura HUFFMAN con un insieme di n caratteri è $O(n \lg n)$. Possiamo ridurre il tempo di esecuzione a $O(n \lg \lg n)$ sostituendo il min-heap binario con un albero di van Emde Boas (Capitolo 20).

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950301 6-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

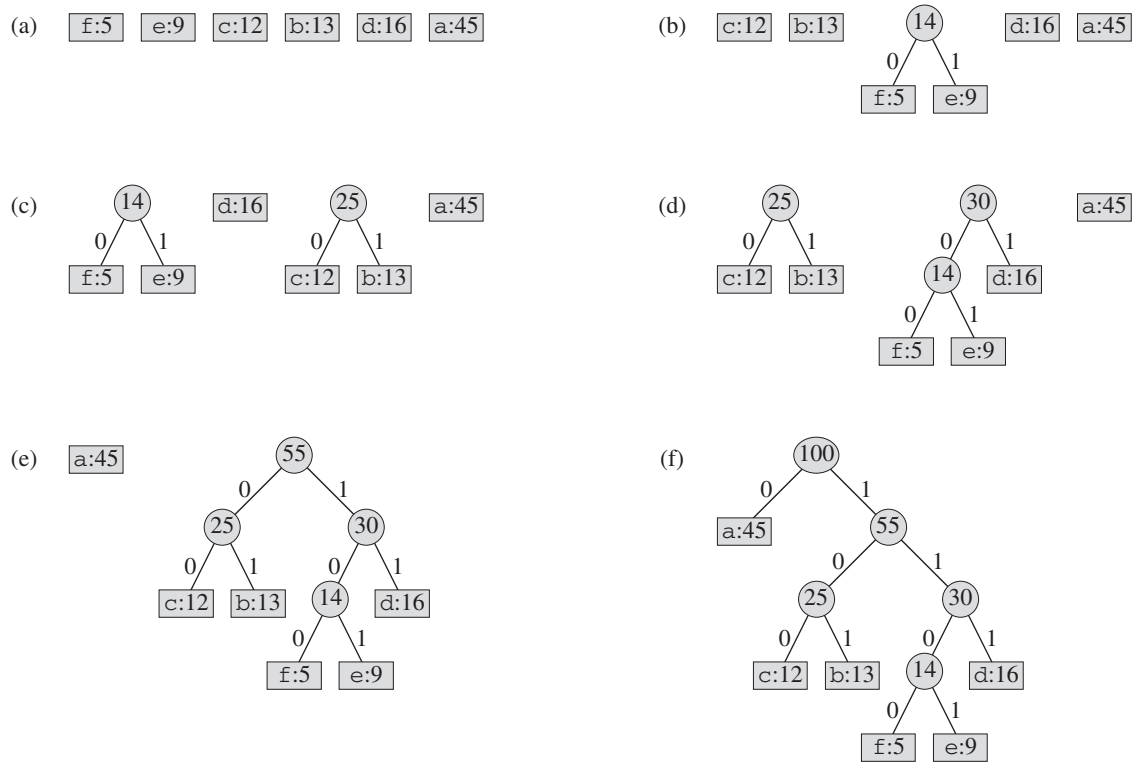


Figura 16.5 I passi dell'algoritmo di Huffman per le frequenze date nella Figura 16.3. Ogni parte illustra i contenuti della coda ordinati per frequenze crescenti. A ogni passo, i due alberi con le frequenze più piccole vengono fusi. Le foglie sono rappresentate da rettangoli che contengono un carattere e la sua frequenza. I nodi interni sono rappresentati da cerchi che contengono la somma delle frequenze dei loro figli. Un arco che collega un nodo interno con i suoi figli ha l'etichetta 0 se è un arco che congiunge un figlio sinistro, ha l'etichetta 1 se è un arco che congiunge un figlio destro. La parola in codice per una lettera è la sequenza delle etichette degli archi che collegano la radice alla foglia di quella lettera. **(a)** L'insieme iniziale di $n = 6$ nodi, uno per ogni lettera. **(b)–(e)** Stadi intermedi. **(f)** L'albero finale.

Correttezza dell'algoritmo di Huffman

Per dimostrare che l'algoritmo goloso HUFFMAN è corretto, dimostriamo che il problema di determinare un codice prefisso ottimo ha le proprietà della scelta golosa e della sottostruttura ottima. Il prossimo lemma dimostra che la proprietà della scelta golosa è soddisfatta.

Lemma 16.2
Sia C un alfabeto dove ogni carattere $c \in C$ ha frequenza $c.freq$. Siano x e y due caratteri in C che hanno le frequenze più piccole. Allora esiste un codice prefisso ottimo per C in cui le parole in codice per x e y hanno la stessa lunghezza e differiscono soltanto nell'ultimo bit.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Dimostrazione La dimostrazione si basa sull'idea di prendere l'albero T che rappresenta un arbitrario codice prefisso ottimo e di modificarlo per creare un albero che rappresenta un altro codice prefisso ottimo, in modo che i caratteri x e y appaiano come foglie sorelle con profondità massima nel nuovo albero. Se questa trasformazione sarà possibile, allora le parole in codice per x e y avranno la stessa lunghezza e saranno diverse soltanto nell'ultimo bit.

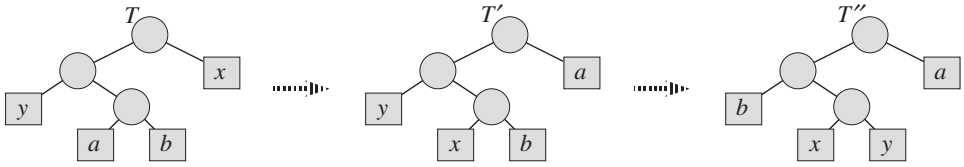


Figura 16.6 Una rappresentazione dei passaggi chiave nella dimostrazione del Lemma 16.2. Nell'albero ottimo T, le foglie a e b sono due foglie sorelle a profondità massima. Le foglie x e y sono le due foglie che l'algoritmo di Huffman fonde per prime; esse appaiono in posizioni arbitrarie nell'albero T. Le foglie a e x vengono scambiate per ottenere l'albero T'. Poi, vengono scambiate le foglie b e y per ottenere l'albero T''. Poiché ogni scambio non aumenta il costo, anche l'albero risultante T'' è un albero ottimo.

Siano a e b due caratteri che sono foglie sorelle con profondità massima in T. Senza perdere in generalità, supponiamo che $a.freq \leq b.freq$ e $x.freq \leq y.freq$. Poiché $x.freq$ e $y.freq$ sono, in ordine, le due frequenze minime delle foglie e poiché $a.freq$ e $b.freq$ sono due frequenze arbitrarie pure esse in ordine, si ha $x.freq \leq a.freq$ e $y.freq \leq b.freq$.

Nella parte restante della dimostrazione, è possible avere $x.freq = a.freq$ o $y.freq = b.freq$. Tuttavia, se avessimo $x.freq = b.freq$, avremmo anche $a.freq = b.freq = x.freq = y.freq$ (vedere l'Esercizio 16.3-1), e il lemma sarebbe banalmente vero. Quindi supporremo che $x.freq \neq b.freq$, e dunque $x \neq b$.

Come illustra la Figura 16.6, scambiamo le posizioni di a e x in T per ottenere un albero T'; poi scambiamo le posizioni di b e y in T' per ottenere un albero T'' in cui x e y sono foglie sorelle con profondità massima. (Notate che se $x = b$, ma $y \neq a$, allora l'albero T'' non ha x e y come foglie sorelle con profondità massima. Poiché supponiamo che $x \neq b$, questa situazione non può verificarsi.) Per l'equazione (16.4), la differenza di costo tra T e T' è

$$\begin{aligned}
&B(T) - B(T') \\
&\quad = \sum_{c \in C} c.freq \cdot d_T(c) - \sum_{c \in C} c.freq \cdot d_{T'}(c) \\
&\quad = x.freq \cdot d_T(x) + a.freq \cdot d_T(a) - x.freq \cdot d_{T'}(x) - a.freq \cdot d_{T'}(a) \\
&\quad = x.freq \cdot d_T(x) + a.freq \cdot d_T(a) - x.freq \cdot d_T(a) - a.freq \cdot d_T(x) \\
&\quad = (a.freq - x.freq)(d_T(a) - d_T(x)) \\
&\quad \geq 0
\end{aligned}$$

perché entrambi i valori $a.freq - x.freq$ e $d_T(a) - d_T(x)$ sono non negativi. Più precisamente, $a.freq - x.freq$ è non negativo perché x è una foglia con frequenza minima; $d_T(a) - d_T(x)$ è non negativo perché a è una foglia con profondità massima in T. Analogamente, lo scambio di y e b non aumenta il costo, quindi il valore $B(T') - B(T'')$ è non negativo. Ne consegue che $B(T'') \leq B(T)$ e, poiché T è un albero ottimo, $B(T) \leq B(T'')$, e questo implica che $B(T'') = B(T)$. Dunque, T'' è un albero ottimo dove x e y figurano come foglie sorelle con profondità massima; questo dimostra il lemma. ■

Acquistato da Michele Mahele su Webster il 2022-07-05 23:12 Numero Ordine Libreria: 1995052016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Il Lemma 16.2 implica che il processo di costruzione di un albero ottimo mediante fusioni può iniziare, senza perdere in generalità, con la scelta golosa di

fondere i due caratteri che hanno frequenza minima. Perché questa è una scelta golosa? Il costo di una fusione può essere visto come la somma delle frequenze dei due elementi da fondere. L'Esercizio 16.3-4 dimostra che il costo totale dell'albero costruito è la somma dei costi delle fusioni. Fra tutte le possibili operazioni di fusione a ogni passo, la procedura HUFFMAN sceglie quella che costa di meno.

Il prossimo lemma dimostra che il problema di costruire i codici prefissi ottimi ha la proprietà della sottostruttura ottima.

Lemma 16.3
Sia C un alfabeto con la frequenza $c.freq$ definita per ogni carattere $c \in C$. Siano x e y due caratteri in C con frequenza minima. Sia C' l'alfabeto C con i caratteri x, y rimossi e il (nuovo) carattere aggiunto z, tale che $C' = C - \{x, y\} \cup \{z\}$. Definiamo f per C' come per C, con la differenza che $z.freq = x.freq + y.freq$. Sia T' un albero qualsiasi che rappresenta un codice prefisso ottimo per l'alfabeto C'. Allora l'albero T, ottenuto da T' sostituendo il nodo foglia z con un nodo interno che ha x e y come figli, rappresenta un codice prefisso ottimo per l'alfabeto C.

Dimostrazione Prima dimostriamo che il costo $B(T)$ dell'albero T può essere espresso in funzione del costo $B(T')$ dell'albero T' considerando le componenti di costo dell'equazione (16.4). Per ogni $c \in C - \{x, y\}$, si ha $d_T(c) = d_{T'}(c)$ e, quindi, $c.freq \cdot d_T(c) = c.freq \cdot d_{T'}(c)$. Poiché $d_T(x) = d_T(y) = d_{T'}(z) + 1$, otteniamo la relazione

$$\begin{aligned} x.freq \cdot d_T(x) + y.freq \cdot d_T(y) &= (x.freq + y.freq)(d_{T'}(z) + 1) \\ &= z.freq \cdot d_{T'}(z) + (x.freq + y.freq) \end{aligned}$$

dalla quale concludiamo che

$$B(T) = B(T') + x.freq + y.freq$$

ovvero

$$B(T') = B(T) - x.freq - y.freq$$

Dimostriamo il lemma per assurdo. Supponiamo che T non rappresenti un codice prefisso ottimo per C. Allora esiste un albero T'' tale che $B(T'') < B(T)$. Senza perdere in generalità (per il Lemma 16.2), l'albero T'' ha x e y come foglie sorelle. Sia T''' l'albero T'' con il padre comune di x e y sostituito da una foglia z con frequenza $z.freq = x.freq + y.freq$. Allora

$$\begin{aligned} B(T''') &= B(T'') - x.freq - y.freq \\ &< B(T) - x.freq - y.freq \\ &= B(T') \end{aligned}$$

che contraddice l'ipotesi che T' rappresenti un codice prefisso ottimo per C'. Quindi, T deve rappresentare un codice prefisso ottimo per l'alfabeto C. ■

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Teorema 16.4
La procedura HUFFMAN produce un codice prefisso ottimo.

Dimostrazione È una conseguenza immediata dei Lemmi 16.2 e 16.3. ■

Esercizi

16.3-1

Spiegate perché, nella dimostrazione del Lemma 16.2, se $x.freq = b.freq$, allora deve essere $a.freq = b.freq = x.freq = y.freq$.

16.3-2

Dimostrate che un albero binario che non è pieno non può rappresentare un codice prefisso ottimo.

16.3-3

Qual è il codice di Huffman ottimo per il seguente insieme di frequenze basato sui primi 8 numeri di Fibonacci?

`a:1 b:1 c:2 d:3 e:5 f:8 g:13 h:21`

Sapreste generalizzare la vostra risposta per trovare il codice ottimo quando le frequenze sono i primi n numeri di Fibonacci?

16.3-4

Dimostrate che il costo totale di un albero per un codice può essere calcolato anche come la somma, estesa a tutti i nodi interni, delle frequenze complessive dei due figli del nodo.

16.3-5

Se i caratteri di un alfabeto vengono ordinati in modo che le loro frequenze siano monotonicamente decrescenti, dimostrate che esiste un codice ottimo le cui parole in codice hanno lunghezze monotonicamente crescenti.

16.3-6

Supponete di avere un codice prefisso ottimo per un insieme $C = \{0, 1, \ldots, n-1\}$ di caratteri e di volere trasmettere questo codice utilizzando il minor numero possibile di bit. Spiegate come rappresentare un codice prefisso ottimo per C utilizzando soltanto $2n - 1 + n\lceil \lg n \rceil$ bit (*suggerimento:* utilizzate $2n - 1$ bit per specificare la struttura dell'albero, come risulterebbe da un attraversamento dell'albero).

16.3-7

Generalizzate l'algoritmo di Huffman utilizzando parole in un codice ternario (cioè le parole in codice usano i simboli 0, 1 e 2); dimostrate che l'algoritmo genera codici ternari ottimi.

16.3-8

Supponete che un file di dati contenga una sequenza di caratteri di 8 bit, i cui 256 caratteri hanno frequenze simili: la frequenza massima è meno del doppio della frequenza minima. Dimostrate che il codice di Huffman in questo caso non è più efficiente di un normale codice a lunghezza fissa di 8 bit.

16.3-9

Dimostrate che nessuno schema di compressione è in grado di comprimere un file di caratteri di 8 bit scelti a caso, neanche di un solo bit (*suggerimento*: confrontate il numero dei file con il numero dei possibili file codificati).

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950301 6-22670-0 Copyright © 2022, McGraw-Hill Education (Italy)

⋆ 16.4 Matroidi e metodi golosi

Esiste un'interessante teoria sugli algoritmi golosi, che presenteremo sinteticamente in questo paragrafo. Questa teoria è utile per determinare quando il metodo goloso genera soluzioni ottime. Usa delle particolari strutture combinatorie dette "matroidi". Sebbene questa teoria non tratti tutti i casi ai quali è possibile applicare un metodo goloso (per esempio, essa non si applica al problema della selezione di attività del Paragrafo 16.1 o al problema della codifica di Huffman del Paragrafo 16.3), tuttavia consente di analizzare molti casi di interesse pratico. Inoltre essa è stata estesa per permettere di trattare molte altre applicazioni; consultate le note alla fine di questo capitolo per i riferimenti bibliografici.

Matroidi

Un ***matroide*** è una coppia ordinata $M = (S, \mathcal{I})$ che soddisfa le seguenti condizioni.

1. S è un insieme finito.
2. $\mathcal{I}$ è una famiglia non vuota di sottoinsiemi di S, detti ***sottoinsiemi indipendenti*** di S, tali che se $B \in \mathcal{I}$ e $A \subseteq B$, allora $A \in \mathcal{I}$. Diciamo che la famiglia $\mathcal{I}$ è ***ereditaria*** se soddisfa questa proprietà. Notate che l'insieme vuoto $\emptyset$ è necessariamente un membro di $\mathcal{I}$.
3. Se $A \in \mathcal{I}$, $B \in \mathcal{I}$ e $|A| < |B|$, allora esiste un elemento $x \in B - A$ tale che $A \cup \{x\} \in \mathcal{I}$. Diciamo che M soddisfa la ***proprietà di scambio***.

La parola "matroide" è stata coniata da Hassler Whitney, mentre studiava i ***matroidi matriciali***, in cui gli elementi di S sono le righe di una data matrice e un insieme di righe è indipendente se le righe sono linearmente indipendenti nel senso convenzionale del termine. È facile dimostrare che questa struttura definisce un matroide (vedere l'Esercizio 16.4-2).

Come altro esempio di matroide, consideriamo il ***matroide grafico*** $M_G = (S_G, \mathcal{I}_G)$ definito su di un grafo non orientato $G = (V, E)$ nel seguente modo.

- L'insieme S_G è l'insieme E degli archi di G.
- Se A è un sottoinsieme di E, allora $A \in \mathcal{I}_G$, se e soltanto se A è aciclico; ovvero un insieme di archi A è indipendente, se e soltanto se il sottografo $G_A = (V, A)$ forma una foresta.

Il matroide grafico M_G è in stretta relazione con il problema dell'albero di connessione minimo, che è descritto dettagliatamente nel Capitolo 23.

Teorema 16.5
Se $G = (V, E)$ è un grafo non orientato, allora $M_G = (S_G, \mathcal{I}_G)$ è un matroide.

Acquistato da Michele Michele su Webster il 2022-[illegible] Ordine [illegible] 0316-220702-0 Copyright [illegible] Inc. All rights reserved. Unauthorized distribution (Italy)

Dimostrazione Chiaramente, $S_G = E$ è un insieme finito. Inoltre, $\mathcal{I}_G$ è una famiglia ereditaria, perché un sottoinsieme di una foresta è una foresta. In altri termini, togliendo degli archi da un insieme aciclico di archi non è possibile creare cicli.

Quindi, resta da dimostrare che M_G soddisfa la proprietà di scambio. Supponiamo che $G_A = (V, A)$ e $G_B = (V, B)$ siano foreste di G e che $|B| > |A|$; ovvero A e B sono insiemi aciclici di archi e B contiene più archi di A.

Possiamo affermare che una foresta $F = (V_F, E_F)$ contiene esattamente $|V_F| - |E_F|$ alberi. Per capire perché, supponete che F sia formato da t alberi, dove l'i-esimo albero contiene v_i vertici ed e_i archi. Quindi, si ha

$$\begin{aligned} |E_F| &= \sum_{i=1}^{t} e_i \\ &= \sum_{i=1}^{t} (v_i - 1) \quad \text{(dal Teorema B.2)} \\ &= \sum_{i=1}^{t} v_i - t \\ &= |V_F| - t \end{aligned}$$

che implica che $t = |V_F| - |E_F|$. Quindi, la foresta G_A contiene $|V| - |A|$ alberi, e la foresta G_B contiene $|V| - |B|$ alberi.

Poiché la foresta G_B ha meno alberi della foresta G_A, la foresta G_B deve contenere qualche albero T i cui vertici sono in due differenti alberi nella foresta G_A. Inoltre, poiché T è connesso, deve contenere un arco (u, v) tale che i vertici u e v si trovino in alberi differenti nella foresta G_A. Poiché l'arco (u, v) collega i vertici in due alberi differenti nella foresta G_A, l'arco (u, v) può essere aggiunto alla foresta G_A senza creare un ciclo. Dunque, M_G soddisfa la proprietà di scambio, completando la dimostrazione che M_G è un matroide. ■

Dato un matroide $M = (S, \mathcal{I})$, un elemento $x \notin A$ è una ***estensione*** di $A \in \mathcal{I}$ se x può essere aggiunto ad A preservando l'indipendenza; ovvero x è un'estensione di A se $A \cup \{x\} \in \mathcal{I}$. Come esempio, consideriamo un matroide grafico M_G. Se A è un insieme indipendente di archi, allora l'arco e è un'estensione di A, se e soltanto se e non è in A e l'aggiunta di e ad A non crea un ciclo.

Se A è un sottoinsieme indipendente in un matroide M, diciamo che A è ***massimale*** se non ha estensioni; ovvero A è massimale se non è contenuto in un sottoinsieme indipendente di M più grande di A. La seguente proprietà è utile in molti casi.

Teorema 16.6
Tutti i sottoinsiemi indipendenti massimali in un matroide hanno la stessa dimensione.

Dimostrazione Supponiamo per assurdo che A sia un sottoinsieme indipendente massimale di M ed esista un altro sottoinsieme B di M, indipendente e massimale, che è più grande di A. Allora, la proprietà di scambio implica che A è estendibile a un insieme indipendente più grande $A \cup \{x\}$ per qualche $x \in B - A$, contraddicendo l'ipotesi che A sia massimale. ■

Acquistato da Michele Michelangeli (vedi pagina 2027 di 2027) - Proprietà letteraria riservata - Copyright © 2023 McGraw-Hill Education (Italy)

Come esempio di questo teorema, consideriamo un matroide grafico M_G per un grafo G non orientato e connesso. Ogni sottoinsieme indipendente massimale di M_G deve essere un albero libero con esattamente $|V| - 1$ archi che collegano tutti i vertici di G. Tale albero è detto ***albero di connessione*** di G.

Si dice che un matroide $M = (S, \mathcal{I})$ è ***pesato*** se esiste una funzione peso w che assegna un peso $w(x)$ strettamente positivo a ogni elemento $x \in S$. La funzione peso w si estende ai sottoinsiemi di S mediante sommatoria

$$w(A) = \sum_{x \in A} w(x)$$

per qualsiasi $A \subseteq S$. Per esempio, se indichiamo con $w(e)$ la lunghezza di un arco e in un matroide grafico M_G, allora $w(A)$ è la lunghezza totale degli archi nell'insieme di archi A.

Algoritmi golosi in un matroide pesato

Molti problemi per i quali un metodo goloso fornisce soluzioni ottime possono essere così formulati: trovare un sottoinsieme indipendente di peso massimo in un matroide pesato. In altre parole, dato un matroide pesato $M = (S, \mathcal{I})$, vogliamo trovare un insieme indipendente $A \in \mathcal{I}$ tale che $w(A)$ sia massimizzato. Tale sottoinsieme, che è indipendente e ha il peso più grande possibile, è detto ***sottoinsieme ottimo*** del matroide. Poiché il peso $w(x)$ di qualsiasi elemento $x \in S$ è positivo, un sottoinsieme ottimo è sempre un sottoinsieme indipendente massimale – è sempre vantaggioso rendere A più grande possibile.

Per esempio, nel ***problema dell'albero di connessione minimo*** abbiamo un grafo $G = (V, E)$ non orientato e connesso e una funzione lunghezza w tale che $w(e)$ è la lunghezza (positiva) di un arco e (in questo contesto utilizziamo il termine "lunghezza" per fare riferimento ai pesi originali degli archi del grafo, riservando il termine "peso" per fare riferimento ai pesi nel matroide associato). Bisogna trovare un sottoinsieme di archi che colleghi tutti i vertici assieme e che abbia la minima lunghezza totale. Per vedere questo come il problema di trovare un sottoinsieme ottimo di un matroide, consideriamo il matroide pesato M_G con la funzione peso w', dove $w'(e) = w_0 - w(e)$ e w_0 è più grande della lunghezza massima di qualsiasi arco. In questo matroide pesato, tutti i pesi sono positivi e un sottoinsieme ottimo è un albero di connessione con la minima lunghezza totale nel grafo originale. Più precisamente, ogni sottoinsieme A indipendente e massimale corrisponde a un albero di connessione con $|V| - 1$ archi, e poiché

$$\begin{aligned} w'(A) &= \sum_{e \in A} w'(e) \\ &= \sum_{e \in A} (w_0 - w(e)) \\ &= (|V| - 1)w_0 - \sum_{e \in A} w(e) \\ &= (|V| - 1)w_0 - w(A) \end{aligned}$$

per qualsiasi sottoinsieme A indipendente e massimale, un sottoinsieme indipendente che massimizza la quantità $w'(A)$ deve minimizzare $w(A)$. Quindi, qualsiasi algoritmo che è in grado di trovare un sottoinsieme ottimo A in un matroide arbitrario può risolvere il problema dell'albero di connessione minimo.

Il Capitolo 23 descrive gli algoritmi per risolvere il problema dell'albero di connessione minimo, mentre qui presentiamo un algoritmo goloso che funziona con qualsiasi matroide pesato. L'algoritmo riceve come input un matroide pesato $M = (S, \mathcal{I})$ con la relativa funzione peso w positiva e restituisce un sottoinsieme ottimo A. Nel nostro pseudocodice, indichiamo i componenti di M con $M.S$ e $M.\mathcal{I}$ e la funzione peso con w. L'algoritmo è goloso perché esamina, uno dopo l'altro in ordine monotonicamente decrescente di peso, ogni elemento $x \in S$ e lo aggiunge immediatamente all'insieme A, se $A \cup \{x\}$ è indipendente.

Acquistato da Michele Michele su Webster il 2022-[illegible]

```
GREEDY(M, w)
1  A = ∅
2  Ordina M.S in modo monotonicamente decrescente rispetto al peso w
3  for ogni elemento x ∈ M.S, preso in ordine
     monotonicamente decrescente rispetto al peso w(x)
4      if A ∪ {x} ∈ M.I
5          A = A ∪ {x}
6  return A
```

La riga 4 controlla se, aggiungendo un elemento x ad A, l'insieme A resta indipendente. Se A resta indipendente, allora la riga 5 aggiunge x ad A, altrimenti x viene scartato. Poiché l'insieme vuoto è indipendente e poiché ciascuna iterazione del ciclo **for** mantiene l'indipendenza di A, il sottoinsieme A è sempre indipendente, per induzione. Di conseguenza, GREEDY restituisce sempre un sottoinsieme indipendente A. Vedremo presto che A è un sottoinsieme con il massimo peso possibile, quindi A è un sottoinsieme ottimo.

Il tempo di esecuzione di GREEDY è facile da analizzare. Indichiamo $|S|$ con n. La fase di ordinamento di GREEDY richiede un tempo $O(n \lg n)$. La riga 4 viene eseguita esattamente n volte, una volta per ogni elemento di S. Ogni volta che viene eseguita la riga 4 bisogna controllare se l'insieme $A \cup \{x\}$ è indipendente oppure no. Se ciascuno di questi controlli richiede un tempo $O(f(n))$, l'intero algoritmo viene eseguito nel tempo $O(n \lg n + nf(n))$.

Dimostriamo adesso che GREEDY restituisce un sottoinsieme ottimo.

Lemma 16.7 (I matroidi hanno la proprietà della scelta golosa)
Supponiamo che $M = (S, \mathcal{I})$ sia un matroide pesato con la funzione peso w e che S sia ordinato in modo monotonicamente decrescente rispetto al peso. Sia x il primo elemento di S tale che $\{x\}$ sia indipendente, se esiste tale elemento. Se x esiste, allora esiste un sottoinsieme ottimo A di S che contiene x.

Dimostrazione Se l'elemento x non esiste, allora l'unico sottoinsieme indipendente è l'insieme vuoto e la dimostrazione termina. Altrimenti, sia B un sottoinsieme ottimo non vuoto. Supponiamo che $x \notin B$; altrimenti poniamo $A = B$ e la dimostrazione termina. Nessun elemento di B ha un peso maggiore di $w(x)$. Per spiegare questo, osserviamo che se $y \in B$, allora $\{y\}$ è indipendente, perché $B \in \mathcal{I}$ e $\mathcal{I}$ è una famiglia ereditaria. La nostra scelta di x, quindi, garantisce che $w(x) \geq w(y)$ per qualsiasi $y \in B$.

Costruiamo l'insieme A nel modo seguente. Iniziamo con $A = \{x\}$. Per la scelta di x, A è indipendente. Applicando la proprietà di scambio, troviamo ripetutamente un nuovo elemento di B che può essere aggiunto ad A finché $|A| = |B|$, preservando l'indipendenza di A. A questo punto A e B sono uguali, eccetto che A contiene x mentre B contiene un qualche altro elemento y. Dunque, $A = B - \{y\} \cup \{x\}$ per qualche $y \in B$, e quindi

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned} w(A) &= w(B) - w(y) + w(x) \\ &\geq w(B) \end{aligned}$$

Poiché B è ottimo, anche A deve essere ottimo, e poiché $x \in A$, il lemma è dimostrato. ■

Adesso dimostriamo che se un elemento inizialmente non è un'opzione possibile, non potrà esserlo successivamente.

Lemma 16.8
Sia $M = (S, \mathcal{I})$ un matroide. Se x è un elemento di S che è un'estensione di qualche sottoinsieme indipendente A di S, allora x è anche un'estensione di $\emptyset$.

Dimostrazione Poiché x è un'estensione di A, allora $A \cup \{x\}$ è indipendente. Poiché $\mathcal{I}$ è una famiglia ereditaria, $\{x\}$ deve essere indipendente. Quindi, x è un'estensione di $\emptyset$. ■

Corollario 16.9
Sia $M = (S, \mathcal{I})$ un matroide. Se x è un elemento di S tale che x non sia un'estensione di $\emptyset$, allora x non è un'estensione di qualsiasi sottoinsieme indipendente A di S.

Dimostrazione Questo corollario è il contrapposto del Lemma 16.8. ■

Il Corollario 16.9 dice che qualsiasi elemento che non può essere utilizzato subito non potrà più essere utilizzato. Quindi, la procedura GREEDY non può commettere un errore se ignora gli elementi iniziali di S che non sono un'estensione di $\emptyset$, in quanto tali elementi non potranno mai essere utilizzati.

Lemma 16.10 (I matroidi hanno la proprietà della sottostruttura ottima)
Sia x il primo elemento di S scelto dalla procedura GREEDY con il matroide pesato $M = (S, \mathcal{I})$. Il problema rimanente di trovare un sottoinsieme indipendente di peso massimo che contiene x si riduce a trovare un sottoinsieme indipendente di peso massimo del matroide pesato $M' = (S', \mathcal{I}')$, dove

$$\begin{aligned} S' &= \{y \in S : \{x, y\} \in \mathcal{I}\} \\ \mathcal{I}' &= \{B \subseteq S - \{x\} : B \cup \{x\} \in \mathcal{I}\} \end{aligned}$$

e la funzione peso per M' è la funzione peso per M, ristretta a S' (chiamiamo M' la ***contrazione*** di M per l'elemento x).

Dimostrazione Se A è un sottoinsieme indipendente di peso massimo di M che contiene x, allora $A' = A - \{x\}$ è un sottoinsieme indipendente di M'. Viceversa, qualsiasi sottoinsieme indipendente A' di M' produce un sottoinsieme indipendente $A = A' \cup \{x\}$ di M. Poiché in entrambi i casi $w(A) = w(A') + w(x)$, una soluzione di peso massimo in M che contiene x genera una soluzione di peso massimo in M' e viceversa. ■

Teorema 16.11 (Correttezza dell'algoritmo goloso con i matroidi)
Se $M = (S, \mathcal{I})$ è un matroide pesato con la funzione peso w, allora la procedura GREEDY(M, w) restituisce un sottoinsieme ottimo.

Dimostrazione Per il Corollario 16.9, tutti gli elementi che vengono inizialmente ignorati perché non sono estensioni di $\emptyset$ possono essere dimenticati, perché non potranno più essere utili. Una volta che il primo elemento x è stato selezionato, il Lemma 16.7 implica che GREEDY non può commettere un errore se aggiunge x ad A, perché esiste un sottoinsieme ottimo che contiene x. Infine, il Lemma 16.10 implica che il problema che resta è quello di trovare un sottoinsieme ottimo nel matroide M' che è la contrazione di M per x. Dopo che la procedura GREEDY ha impostato A a $\{x\}$, tutti i suoi restanti passaggi possono essere interpretati come operazioni sul matroide $M' = (S', \mathcal{I}')$, perché B è indipendente in M' se e

Acquistato da Michele Michele su Webster il 2022-[illegible] Vietata la copia, la riproduzione e la rivendita (Italy)

soltanto se $B \cup \{x\}$ è indipendente in M, per qualsiasi insieme $B \in \mathcal{I}'$. Quindi, la successiva operazione di GREEDY troverà un sottoinsieme indipendente di peso massimo per M' e il risultato finale di GREEDY sarà trovare un sottoinsieme indipendente di peso massimo per M. ■

Esercizi

16.4-1

Dimostrate che $(S, \mathcal{I}_k)$ è un matroide, dove S è un insieme finito qualsiasi e $\mathcal{I}_k$ è l'insieme di tutti i sottoinsiemi di S di dimensione al massimo k, dove $k \leq |S|$.

16.4-2 ⋆

Data una matrice T $m \times n$ definita in qualche campo (i numeri reali, per esempio), dimostrate che $(S, \mathcal{I})$ è un matroide, dove S è l'insieme di colonne di T e $A \in \mathcal{I}$ se e soltanto se le colonne in A sono linearmente indipendenti.

16.4-3 ⋆

Dimostrate che, se $(S, \mathcal{I})$ è un matroide, allora $(S, \mathcal{I}')$ è un matroide, dove

$\mathcal{I}' = \{A' : S - A' \text{ contiene qualche massimale } A \in \mathcal{I}\}$

Ovvero gli insiemi indipendenti massimali di $(S, \mathcal{I}')$ sono proprio i complementi degli insiemi indipendenti massimali di $(S, \mathcal{I})$.

16.4-4 ⋆

Sia S un insieme finito e sia $S_1, S_2, \ldots, S_k$ una partizione di S in sottoinsiemi disgiunti non vuoti. Definite la struttura $(S, \mathcal{I})$ con la condizione che $\mathcal{I} = \{A : |A \cap S_i| \leq 1 \text{ per } i = 1, 2, \ldots, k\}$. Dimostrate che $(S, \mathcal{I})$ è un matroide. Ovvero l'insieme di tutti gli insiemi A che contengono al massimo un elemento di ogni sottoinsieme della partizione determina gli insiemi indipendenti di un matroide.

16.4-5

Spiegate come dovrebbe essere trasformata la funzione peso di un problema di matroide pesato, quando la soluzione ottima desiderata è un sottoinsieme indipendente massimale di *peso minimo*, per renderlo un problema standard dei matroidi pesati. Dimostrate che la vostra trasformazione è corretta.

⋆ 16.5 Un problema di programmazione dei lavori

Un interessante problema che può essere risolto utilizzando i matroidi è quello di ottimizzare la programmazione dei lavori di durata unitaria in un singolo processore, dove ogni lavoro ha una scadenza e una penalità che deve essere pagata se non viene rispettata la scadenza. Il problema appare complicato, ma può essere risolto in modo sorprendentemente semplice formulandolo come matroide e utilizzando un algoritmo goloso.

Acquistato da Michele Michelucci (michelucci@gmail.com) - Copia non autorizzata - Copyright © 2023 McGraw-Hill Education (Italy)

Un ***lavoro di durata unitaria*** è un compito, come un programma da eseguire in un calcolatore, che richiede una sola unità di tempo per essere completato. Dato un insieme finito S di lavori di durata unitaria, un ***piano di programmazione*** per l'insieme S è una permutazione di S che specifica l'ordine in cui questi lavori devono essere eseguiti. Il primo lavoro nel piano di programmazione inizia al tempo 0 e termina al tempo 1, il secondo lavoro inizia al tempo 1 e termina al tempo 2 e così via.

Il problema della ***programmazione dei lavori di durata unitaria con scadenze e penalità per un singolo processore*** ha i seguenti input:

- un insieme $S = \{a_1, a_2, \ldots, a_n\}$ di n lavori di durata unitaria.
- un insieme di n interi che rappresentano le ***scadenze*** $d_1, d_2, \ldots, d_n$; la generica scadenza d_i è tale che $1 \leq d_i \leq n$. Si suppone inoltre che il generico lavoro a_i termini al tempo d_i.
- un insieme di n pesi non negativi o ***penalità*** $w_1, w_2, \ldots, w_n$; si paga la penalità w_i, se il lavoro a_i non termina entro il tempo d_i, mentre non c'è alcuna penalità se un lavoro termina entro la sua scadenza.

Il problema è trovare un piano di programmazione per S che minimizza le penalità totali da pagare per le scadenze non rispettate.

Consideriamo un determinato piano di programmazione. Diciamo che un lavoro è ***in ritardo*** se termina dopo la scadenza prevista nel piano di programmazione, altrimenti il lavoro è ***in anticipo***. Un arbitrario piano di programmazione può essere sempre organizzato in modo che il lavori in anticipo precedano i lavori in ritardo. Per spiegare questo, notiamo che se un lavoro in anticipo a_i segue un lavoro in ritardo a_j, allora possiamo scambiare le posizioni di a_i e a_j, in modo che a_i resti un lavoro in anticipo e a_j resti un lavoro in ritardo.

In modo analogo, possiamo affermare che un arbitrario piano di programmazione può essere sempre posto nella ***forma canonica***, in cui i lavori in anticipo precedono i lavori in ritardo e quelli in anticipo sono programmati in ordine monotonicamente crescente rispetto alle scadenze. Per farlo, organizziamo innanzi tutto il piano di programmazione in modo che i lavori in anticipo precedano quelli in ritardo. Poi, se ci sono due lavori in anticipo a_i e a_j che terminano rispettivamente ai tempi k e $k+1$ previsti dal piano, con $d_j < d_i$, scambiamo le posizioni di a_i e a_j. Poiché a_j è in anticipo prima dello scambio, deve essere $k+1 \leq d_j$. Allora $k+1 < d_i$ e, quindi, a_i è ancora un lavoro in anticipo dopo lo scambio. Il lavoro a_j viene anticipato nel piano di programmazione, quindi è ancora un lavoro in anticipo dopo lo scambio delle posizioni.

Il problema di ottimizzare un piano di programmazione si riduce così a trovare un insieme A di lavori che devono essere in anticipo nel piano di programmazione ottimo. Una volta che A è determinato, possiamo creare il piano effettivo elencando gli elementi di A in ordine monotonicamente crescente rispetto alle scadenze e, di seguito, i lavori in ritardo (cioè $S - A$) in qualsiasi ordine, generando un ordinamento canonico del piano di programmazione ottimo.

Diciamo che un insieme A di lavori è ***indipendente*** se esiste un piano di programmazione per questi lavori in cui nessun lavoro è in ritardo. Chiaramente, l'insieme dei lavori in anticipo per un piano di programmazione forma un insieme indipendente di lavori. Sia $\mathcal{I}$ l'insieme di tutti gli insiemi indipendenti di lavori.

Consideriamo il problema di determinare se un dato insieme A di lavori è indipendente. Per $t = 0, 1, 2, \ldots, n$, indichiamo con $N_t(A)$ il numero di lavori in A la cui scadenza è t o un tempo precedente. Notate che $N_0(A) = 0$ per qualsiasi insieme A.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Lemma 16.12

Per qualsiasi insieme di lavori A, le seguenti asserzioni sono equivalenti.

1. L'insieme A è indipendente.

2. Per $t = 0, 1, 2, \ldots, n$, si ha $N_t(A) \leq t$.
3. Se i lavori in A sono programmati in ordine monotonicamente crescente rispetto alle scadenze, allora nessun lavoro è in ritardo.

Dimostrazione Per dimostrare che (1) implica (2), dimostriamo il contrapposto: se $N_t(A) > t$ per qualche t, allora non è possibile creare un piano di programmazione senza lavori in ritardo per l'insieme A, perché ci sono più di t lavori da finire prima del tempo t. Di conseguenza, l'asserzione (1) implica la (2). Se la (2) è vera, allora anche la (3) deve essere vera: non c'è modo di "incepparsi" quando programmiamo i lavori in ordine monotonicamente crescente rispetto alle scadenze, in quanto la (2) implica che l'i-esima scadenza più grande è al massimo i. Infine, la (3) implica banalmente la (1). ■

Applicando la proprietà 2 del Lemma 16.12, possiamo facilmente stabilire se un dato insieme di lavori è indipendente oppure no (vedere l'Esercizio 16.5-2).

Il problema di minimizzare la somma delle penalità dei lavori in ritardo è equivalente al problema di massimizzare la somma delle penalità dei lavori in anticipo. Il seguente teorema garantisce che possiamo utilizzare l'algoritmo goloso per trovare un insieme indipendente A di lavori con la massima penalità totale.

Teorema 16.13
Se S è un insieme di lavori di durata unitaria con scadenze e $\mathcal{I}$ è l'insieme di tutti gli insiemi indipendenti di lavori, allora il corrispondente sistema $(S, \mathcal{I})$ è un matroide.

Dimostrazione Ogni sottoinsieme di un insieme indipendente di lavori è certamente indipendente. Per provare la proprietà di scambio, supponiamo che B e A siano insiemi indipendenti di lavori e che $|B| > |A|$. Sia k il più grande t tale che $N_t(B) \leq N_t(A)$ (questo valore di t esiste, in quanto $N_0(A) = N_0(B) = 0$). Poiché $N_n(B) = |B|$ e $N_n(A) = |A|$, ma $|B| > |A|$, deve essere $k < n$ e $N_j(B) > N_j(A)$ per ogni j nell'intervallo $k + 1 \leq j \leq n$. Quindi, l'insieme B contiene più lavori con scadenza $k + 1$ dell'insieme A. Sia a_i un lavoro in $B - A$ con scadenza $k + 1$. Sia $A' = A \cup \{a_i\}$.

Adesso dimostriamo che A' deve essere indipendente applicando la proprietà 2 del Lemma 16.12. Per $0 \leq t \leq k$, abbiamo $N_t(A') = N_t(A) \leq t$, perché A è indipendente. Per $k < t \leq n$, abbiamo $N_t(A') \leq N_t(B) \leq t$, perché B è indipendente. Dunque A' è indipendente, e questo completa la dimostrazione che $(S, \mathcal{I})$ è un matroide. ■

Per il Teorema 16.11, possiamo utilizzare un algoritmo goloso per trovare un insieme di lavori A indipendente di peso massimo. Poi, possiamo creare un piano di programmazione ottimo che ha i lavori in A come suoi lavori in anticipo. Questo metodo è un algoritmo efficiente per programmare lavori di durata unitaria con scadenze e penalità per un singolo processore. Il tempo di esecuzione è $O(n^2)$ se si usa GREEDY, in quanto ciascuno degli $O(n)$ controlli sull'indipendenza fatti da tale algoritmo richiede un tempo $O(n)$ (vedere l'Esercizio 16.5-2). Una implementazione più rapida è descritta nel Problema 16-4.

Acquistato da Michele Michele su Webster il 2022-07-07 20:42 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022: McGraw-Hill Education (Italy)

La Figura 16.7 illustra un esempio di un problema di programmazione di lavori di durata unitaria con scadenze e penalità per un singolo processore. In questo

	Lavoro						
a_i	1	2	3	4	5	6	7
d_i	4	2	4	3	1	4	6
w_i	70	60	50	40	30	20	10

Figura 16.7 Un'istanza del problema della programmazione di lavori di durata unitaria con scadenze e penalità per un singolo processore.

esempio, l'algoritmo goloso seleziona i lavori a_1, a_2, a_3 e a_4, poi scarta a_5 e a_6 e, infine, accetta a_7. La programmazione ottima finale è

$\langle a_2, a_4, a_1, a_3, a_7, a_5, a_6 \rangle$

che ha un totale di penalità pari a $w_5 + w_6 = 50$.

Esercizi

16.5-1

Risolvete l'istanza del problema di programmazione illustrato nella Figura 16.7, sostituendo ciascuna penalità w_i con $80 - w_i$.

16.5-2

Mostrate come utilizzare la proprietà 2 del Lemma 16.12 per determinare nel tempo $O(|A|)$ se un dato insieme A di lavori è indipendente oppure no.

Problemi

16-1 Resto in monete

Considerate il problema di formare il resto di n centesimi utilizzando il minor numero di monete. Supponete che il valore di ciascuna moneta sia un intero.

a. Descrivete un algoritmo goloso per formare il resto con monete da 1, 5, 10 e 25 centesimi. Dimostrate che il vostro algoritmo fornisce una soluzione ottima.

b. Supponete che i valori delle monete disponibili siano potenze di c, cioè i valori siano $c^0, c^1, \ldots, c^k$ per due interi $c > 1$ e $k \geq 1$. Dimostrate che l'algoritmo goloso produce sempre una soluzione ottima.

c. Trovate un insieme di valori delle monete per il quale l'algoritmo goloso non produce una soluzione ottima. L'insieme dovrebbe includere il centesimo, in modo che ci sia una soluzione per qualsiasi valore di n.

d. Descrivete un algoritmo con tempo $O(nk)$ che calcola il resto per qualsiasi insieme di k valori di monete differenti, supponendo che una delle monete sia il centesimo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

16-2 Minimizzare il tempo medio di completamento dei lavori

Supponete di avere un insieme $S = \{a_1, a_2, \ldots, a_n\}$ di lavori, dove il lavoro a_i richiede p_i unità di tempo di elaborazione per essere completato, dopo che è iniziato. Questi lavori saranno eseguiti, uno alla volta, in un calcolatore. Sia c_i il ***tempo di completamento*** del lavoro a_i, ovvero l'istante in cui termina l'elaborazione del lavoro a_i. Il vostro obiettivo è ridurre al minimo il tempo medio di

completamento, cioè minimizzare $(1/n)\sum_{i=1}^{n} c_i$. Per esempio, supponete che ci siano due lavori, a_1 e a_2, con $p_1 = 3$ e $p_2 = 5$; considerate il piano di programmazione in cui viene eseguito prima a_2 e poi a_1. Allora $c_2 = 5$, $c_1 = 8$ e il tempo medio di completamento è $(5 + 8)/2 = 6{,}5$. Se, invece, viene eseguito prima a_1 e poi a_2, allora $c_1 = 3$ e $c_2 = 8$, e il tempo medio di completamento è $(3 + 8)/2 = 5{,}5$.

a. Scrivete un algoritmo che programma i lavori in modo da minimizzare il tempo medio di completamento. Ciascun lavoro deve essere eseguito senza considerare diritti di precedenza, cioè, una volta che il lavoro a_i è iniziato, deve essere eseguito senza interruzioni fino alla fine per p_i unità di tempo. Dimostrate che il vostro algoritmo minimizza il tempo medio di completamento e determinate il tempo di esecuzione del vostro algoritmo.

b. Supponete adesso che i lavori non siano tutti immediatamente disponibili; ovvero ciascun lavoro ha un ***tempo di rilascio*** r_i prima del quale non può essere elaborato. Supponete inoltre che un lavoro possa essere eseguito rispettando le precedenze, ovvero un lavoro può essere sospeso e riavviato in un tempo successivo. Per esempio, un lavoro a_i con tempo di elaborazione $p_i = 6$ può iniziare al tempo 1 ed essere sospeso al tempo 4. Poi, può essere ripreso al tempo 10, di nuovo sospeso al tempo 11 e, infine, ripreso al tempo 13 e completato al tempo 15. Il lavoro a_i viene eseguito per un totale di 6 unità di tempo, ma il suo tempo di esecuzione è stato diviso in tre parti. Diciamo che il tempo di completamento di a_i è 15. Descrivete un algoritmo che programma i lavori in modo da minimizzare il tempo medio di completamento in questo nuovo scenario. Dimostrate che l'algoritmo minimizza il tempo medio di completamento e determinate il tempo di esecuzione del vostro algoritmo.

16-3 Sottografi aciclici

a. La ***matrice di incidenza*** di un grafo non orientato $G = (V, E)$ è una matrice M $|V| \times |E|$ tale che $M_{ve} = 1$ se l'arco e è incidente sul vertice v, altrimenti $M_{ve} = 0$. Dimostrate che un insieme di colonne di M è linearmente indipendente nel campo degli interi modulo 2, se e soltanto se il corrispondente insieme di archi è aciclico. Poi, utilizzate il risultato dell'Esercizio 16.4-2 per fornire una dimostrazione alternativa del Teorema 16.5.

b. Supponete che un peso $w(e)$ non negativo sia associato a ciascun arco di un grafo non orientato $G = (V, E)$. Descrivete un algoritmo efficiente per trovare un sottoinsieme aciclico di E con il massimo peso totale.

c. Sia $G(V, E)$ un arbitrario grafo orientato. Supponiamo che $(E, \mathcal{I})$ sia definito in modo che $A \in \mathcal{I}$ se e soltanto se A non contiene alcun ciclo orientato. Trovate un esempio di un grafo orientato G tale che il sistema associato $(E, \mathcal{I})$ non sia un matroide. Specificate quale condizione della definizione di matroide non è soddisfatta.

Acquistato da Michele Michele su Webstar il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

d. La ***matrice di incidenza*** di un grafo orientato $G = (V, E)$ privo di cappi è una matrice M $|V| \times |E|$ tale che $M_{ve} = -1$ se l'arco e esce dal vertice v, $M_{ve} = 1$ se l'arco e entra nel vertice v, altrimenti $M_{ve} = 0$. Dimostrate che se un insieme di colonne di M è linearmente indipendente, allora il corrispondente insieme di archi non contiene un ciclo orientato.

e. L'Esercizio 16.4-2 dice che l'insieme degli insiemi linearmente indipendenti delle colonne di qualsiasi matrice M forma un matroide. Spiegate perché i risultati dei punti (c) e (d) non sono in contraddizione. In che modo può mancare la perfetta corrispondenza fra il concetto di aciclicità di un insieme di archi e il concetto di indipendenza lineare dell'insieme di colonne della matrice di incidenza associato all'insieme di archi?

16-4 Varianti di programmazione

Considerate il seguente algoritmo per il problema della programmazione di lavori di durata unitaria con scadenze e penalità (descritto nel Paragrafo 16.5). Supponete che tutte le n porzioni di tempo siano inizialmente vuote; la generica porzione di tempo i è un intervallo di durata unitaria che finisce al tempo i. Considerate i lavori in ordine monotonicamente decrescente rispetto alle penalità. Quando considerate il lavoro a_j, se esiste una porzione di tempo ancora vuota in corrispondenza o prima della scadenza d_j di a_j, assegnate a_j all'ultima di tali porzioni, occupandola interamente. Se tale porzione non esiste, assegnate il lavoro a_j all'ultima delle porzioni non ancora occupate.

a. Dimostrate che questo algoritmo fornisce sempre una soluzione ottima.

b. Utilizzate la foresta degli insiemi disgiunti presentata nel Paragrafo 21.3 per implementare l'algoritmo in modo efficiente. Supponete che l'insieme dei lavori di input sia già ordinato in modo monotonicamente decrescente rispetto alle penalità. Analizzate il tempo di esecuzione della vostra implementazione.

16-5 Caching off-line

I moderni computer usano una cache per memorizzare una piccola quantità di dati in una memoria veloce. Anche se un programma può accedere a grandi quantità di dati, memorizzando una piccola parte della memoria principale nella ***cache*** (una memoria piccola, ma più veloce), il tempo di accesso può ridursi notevolmente.

Quando viene eseguito, un programma effettua una sequenza $\langle r_1, r_2, \ldots, r_n \rangle$ di n richieste di memoria, ciascuna delle quali riguarda un particolare dato. Per esempio, un programma che accede a 4 elementi distinti $\{a, b, c, d\}$ potrebbe effettuare la sequenza di richieste $\langle d, b, d, b, d, a, c, d, b, a, c, b \rangle$. Sia k la dimensione della cache. Quando la cache contiene k elementi e il programma richiede il $(k + 1)$-esimo elemento, il sistema deve decidere, per questa e ogni successiva richiesta, quali k elementi deve mantenere nella cache. Più precisamente, per ogni richiesta r_i, l'algoritmo di gestione della cache verifica se l'elemento r_i è già nella cache. Se c'è, si ha un ***successo della cache***, altrimenti si ha un ***fallimento della cache***. Dopo un fallimento della cache, il sistema recupera r_i dalla memoria principale, e l'algoritmo che gestisce la cache deve decidere se mantenere r_i nella cache. Se decide di mantenere r_i e la cache contiene già k elementi, deve cancellare un elemento per fare posto a r_i. L'algoritmo di gestione della cache cancella il dato con l'obiettivo di minimizzare il numero di fallimenti della cache per l'intera sequenza di richieste.

Acquistato da Michele Michele su Webster il 2022-01-27 23:13 Numero Edizione Libraria 1935203046-3207107-0-Copyright © 2022, McGraw-Hill Education (Italy)

Il caching è un tipico problema on-line, nel senso che dobbiamo prendere delle decisioni su quali dati mantenere nella cache, senza conoscere le richieste future. Qui, tuttavia, noi consideriamo la versione off-line di questo problema, in cui conosciamo in anticipo l'intera sequenza di n richieste e la dimensione della cache k, e vogliamo minimizzare il numero totale di fallimenti della cache.

Possiamo risolvere questo problema off-line tramite una strategia golosa detta *più lontano nel futuro* (furthest in future), che cancella il dato nella cache il cui prossimo accesso nella sequenza delle richieste è il più lontano nel futuro.

a. Scrivete lo pseudocodice per un manager della cache che usa la strategia *più lontano nel futuro*. L'input deve essere una sequenza di richieste $\langle r_1, r_2, \ldots, r_n \rangle$ e una dimensione della cache k; l'output deve essere una sequenza di decisioni sui dati da cancellare dopo ogni richiesta. Qual è il tempo di esecuzione del vostro algoritmo?

b. Dimostrate che il problema del *caching off-line* presenta una sottostruttura ottima.

c. Dimostrate che la strategia *più lontano nel futuro* genera il minor numero possibile di fallimenti della cache.

Note

Per ulteriori informazioni sugli algoritmi golosi e i matroidi, consultate Lawler [225] e Papadimitriou e Steiglitz [272]. L'algoritmo goloso è apparso la prima volta nella letteratura dell'ottimizzazione combinatoria in un articolo del 1971 scritto da Edmonds [102], sebbene la teoria dei matroidi risalga a un lavoro di Whitney [356] del 1935.

La nostra dimostrazione della correttezza dell'algoritmo goloso per il problema della selezione di attività è basata su quella di Gavril [132]. Il problema della programmazione dei lavori è stato studiato da Lawler [225], da Horowitz, Sahni e Rajasekaran [182] e da Brassard e Bratley [55].

I codici di Huffman furono inventati nel 1952 [186]; Lelewer e Hirschberg [232] hanno raccolto le tecniche di compressione dei dati che erano note fino al 1987.

Un'estensione della teoria dei matroidi alla teoria dei greedoidi è stata ideata da Korte e Lovász [217, 218, 219, 220], che hanno generalizzato molto la teoria presentata in questo capitolo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

17 Analisi ammortizzata

Nell'***analisi ammortizzata*** il tempo richiesto per eseguire una sequenza di operazioni su una struttura dati viene calcolato come media dei tempi di tutte le operazioni eseguite. L'analisi ammortizzata può essere utilizzata per dimostrare che il costo medio di un'operazione è piccolo, se si considera la media di una sequenza di operazioni, anche se una singola operazione all'interno della sequenza può essere costosa. L'analisi ammortizzata differisce dall'analisi del caso medio perché non applica la teoria della probabilità; l'analisi ammortizzata valuta le *prestazioni medie di ciascuna operazione nel caso peggiore*.

I primi tre paragrafi di questo capitolo trattano i tre metodi più utilizzati nell'analisi ammortizzata. Il Paragrafo 17.1 inizia con il metodo dell'aggregazione, che richiede di determinare un limite superiore $T(n)$ al costo totale di una sequenza di n operazioni. Il costo medio per operazione è dato da $T(n)/n$. Consideriamo il costo medio come il costo ammortizzato di ciascuna operazione, in modo che tutte le operazioni abbiano lo stesso costo ammortizzato.

Il Paragrafo 17.2 tratta il metodo degli accantonamenti (o metodo del credito), in cui viene determinato un costo ammortizzato di ciascuna operazione. Quando ci sono più tipi di operazioni, ciascun tipo di operazione può avere un costo ammortizzato differente. Il metodo degli accantonamenti attribuisce un costo aggiuntivo ad alcune operazioni iniziali della sequenza, memorizzando questo costo sotto forma di "credito prepagato" su specifici oggetti nella struttura dati. Il credito viene successivamente utilizzato nella sequenza per pagare alcune operazioni cui è attribuito un costo minore del loro costo effettivo.

Il Paragrafo 17.3 descrive il metodo del potenziale, che è simile a quello degli accantonamenti perché viene determinato il costo ammortizzato di ciascuna operazione ed è possibile sovraccaricare le prime operazioni per poter alleggerire in seguito eventuali operazioni molto costose. Il metodo del potenziale mantiene il credito sotto forma di "energia potenziale" della struttura dati nel suo insieme, anziché associare il credito a singoli oggetti all'interno della struttura dati.

Esamineremo due esempi per spiegare questi tre metodi. Nel primo esempio sarà utilizzato uno stack con la nuova operazione MULTIPOP, che elimina più oggetti contemporaneamente. Nel secondo esempio sarà utilizzato un contatore binario che conta, a partire da 0, con una sola operazione INCREMENT.

Mentre leggete questo capitolo, ricordatevi che i costi assegnati durante l'analisi ammortizzata servono esclusivamente all'analisi, nel senso che non devono apparire nel codice. Se, per esempio, viene assegnato un credito a un oggetto x quando viene utilizzato il metodo degli accantonamenti, non occorre assegnare un valore corrispondente a qualche attributo $x.credito$ nel codice.

Le informazioni su una particolare struttura dati che si ottengono dall'analisi ammortizzata possono servire a ottimizzare il progetto degli algoritmi. Per esempio, nel Paragrafo 17.4 utilizzeremo il metodo del potenziale per analizzare una tavola che si espande e si contrae dinamicamente.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995030l6-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

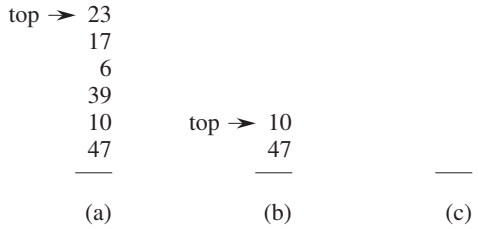


Figura 17.1 L'effetto dell'operazione MULTIPOP sullo stack S, il cui contenuto iniziale è illustrato in **(a)**. I primi 4 oggetti vengono eliminati da MULTIPOP$(S, 4)$; il risultato è illustrato in **(b)**. La successiva operazione è MULTIPOP$(S, 7)$, che svuota lo stack – illustrato in **(c)** – perché sono rimasti meno di 7 oggetti.

17.1 Il metodo dell'aggregazione

L'analisi basata sul ***metodo dell'aggregazione*** dimostra che, per ogni n, una sequenza di n operazioni impiega nel *caso peggiore* un tempo totale $T(n)$. Nel caso peggiore, il costo medio o ***costo ammortizzato*** per operazione è quindi $T(n)/n$. Notate che questo costo ammortizzato si applica a ciascuna operazione, anche quando ci sono più tipi di operazioni nella sequenza. Gli altri due metodi analizzati in questo capitolo, il metodo degli accantonamenti e il metodo del potenziale, potrebbero attribuire costi ammortizzati differenti ai diversi tipi di operazioni.

Operazioni su stack

Come primo esempio del metodo dell'aggregazione analizzeremo gli stack che includono una nuova operazione. Il Paragrafo 10.1 ha presentato le due operazioni fondamentali sugli stack, ciascuna delle quali impiega un tempo $O(1)$:

PUSH(S, x) inserisce l'oggetto x nello stack S.

POP(S) elimina l'oggetto in cima allo stack S e restituisce l'oggetto eliminato. Se si chiama POP su uno stack vuoto, si genera un errore.

Poiché ciascuna di queste operazioni è eseguita nel tempo $O(1)$, assumiamo che il costo di ciascuna di esse sia 1. Quindi, il costo totale di una sequenza di n operazioni PUSH e POP è n, e il tempo di esecuzione effettivo di n operazioni è $\Theta(n)$.

Adesso aggiungiamo l'operazione MULTIPOP(S, k), che elimina i primi k oggetti dalla cima dello stack S o svuota l'intero stack se questo contiene meno di k oggetti. Naturalmente assumiamo che k sia positivo; altrimenti l'operazione MULTIPOP lascia lo stack invariato. Nel seguente pseudocodice, l'operazione STACK-EMPTY restituisce TRUE se non ci sono oggetti nello stack, altrimenti restituisce FALSE.

```
MULTIPOP(S, k)
1  while not STACK-EMPTY(S) and k > 0
2      POP(S)
3      k = k − 1
```

Acquistato da Michele Michelassi (Account: c2022-07-05.2.12.X) Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

La Figura 17.1 illustra un esempio dell'operazione MULTIPOP.

Qual è il tempo di esecuzione di MULTIPOP(S, k) in uno stack di s oggetti? Il tempo di esecuzione effettivo è lineare nel numero di operazioni POP effettivamente eseguite; quindi è sufficiente analizzare MULTIPOP in funzione del costo

astratto 1 per PUSH e per POP. Il numero di iterazioni del ciclo **while** è il numero $\min(s, k)$ di oggetti eliminati dallo stack. Per ogni iterazione del ciclo viene effettuata una chiamata POP nella riga 2. Quindi, il costo totale di MULTIPOP è $\min(s, k)$ e il tempo di esecuzione effettivo è una funzione lineare di questo costo.

Analizziamo una sequenza di n operazioni PUSH, POP e MULTIPOP su uno stack inizialmente vuoto. Il costo nel caso peggiore di un'operazione MULTIPOP nella sequenza è $O(n)$, in quanto la dimensione dello stack è al massimo n. Il tempo nel caso peggiore di qualsiasi operazione sullo stack è quindi $O(n)$; pertanto una sequenza di n operazioni costa $O(n^2)$, perché potremmo avere $O(n)$ operazioni MULTIPOP che costano $O(n)$ ciascuna. Sebbene questa analisi sia corretta, il risultato $O(n^2)$, che abbiamo ottenuto considerando il costo nel caso peggiore di ogni singola operazione, non è stretto.

Applicando il metodo dell'aggregazione, possiamo ottenere un limite superiore più stretto che considera l'intera sequenza di n operazioni. In effetti, sebbene una singola operazione MULTIPOP possa essere costosa, qualsiasi sequenza di n operazioni PUSH, POP e MULTIPOP su uno stack inizialmente vuoto può costare al più $O(n)$. Perché? Ogni oggetto può essere eliminato al massimo una volta per ogni volta che viene inserito. Di conseguenza, il numero di volte che l'operazione POP può essere chiamata su uno stack non vuoto, incluse le chiamate all'interno di MULTIPOP, è al massimo il numero di operazioni PUSH, che è al più n. Per ogni n, qualsiasi sequenza di n operazioni PUSH, POP e MULTIPOP impiega complessivamente un tempo $O(n)$. Il costo medio di un'operazione è $O(n)/n = O(1)$. Nel metodo dell'aggregazione attribuiamo il costo medio quale costo ammortizzato di ogni operazione. In questo esempio, quindi, tutte e tre le operazioni sullo stack hanno un costo ammortizzato pari a $O(1)$.

Ripetiamo che, sebbene abbiamo appena dimostrato che il costo medio, e quindi il tempo di esecuzione, di un'operazione sullo stack è $O(1)$, non abbiamo applicato alcun concetto della teoria della probabilità. In effetti, abbiamo dimostrato un limite $O(n)$ nel *caso peggiore* per una sequenza di n operazioni. Dividendo questo costo totale per n abbiamo ottenuto il costo medio per operazione o costo ammortizzato.

Incrementare un contatore binario

Come altro esempio del metodo dell'aggregazione, consideriamo il problema di implementare un contatore binario di k bit che conta a partire da 0. Come contatore utilizziamo un array $A[0..k-1]$ di bit, dove $A.length = k$. Un numero binario x che è memorizzato nel contatore ha il bit meno significativo in $A[0]$ e il bit più significativo in $A[k-1]$, cosicché $x = \sum_{i=0}^{k-1} A[i] \cdot 2^i$. Inizialmente, $x = 0$ e, quindi, $A[i] = 0$ per $i = 0, 1, \ldots, k-1$. Per sommare 1 (modulo 2^k) al valore del contatore, utilizziamo la seguente procedura.

```
INCREMENT(A)
1  i = 0
2  while i < A.length and A[i] == 1
3       A[i] = 0
4       i = i + 1
5  if i < A.length
6       A[i] = 1
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Valore del contatore	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	Costo totale
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	3
3	0	0	0	0	0	0	1	1	4
4	0	0	0	0	0	1	0	0	7
5	0	0	0	0	0	1	0	1	8
6	0	0	0	0	0	1	1	0	10
7	0	0	0	0	0	1	1	1	11
8	0	0	0	0	1	0	0	0	15
9	0	0	0	0	1	0	0	1	16
10	0	0	0	0	1	0	1	0	18
11	0	0	0	0	1	0	1	1	19
12	0	0	0	0	1	1	0	0	22
13	0	0	0	0	1	1	0	1	23
14	0	0	0	0	1	1	1	0	25
15	0	0	0	0	1	1	1	1	26
16	0	0	0	1	0	0	0	0	31

Figura 17.2 Un contatore binario di 8 bit mentre il suo valore passa da 0 a 16 per effetto di una sequenza di 16 operazioni INCREMENT. I bit che cambiano per formare il valore successivo sono su sfondo grigio. Il costo cumulativo dei bit che cambiano è illustrato a destra. Notate che il costo totale non supera mai il doppio del numero totale di operazioni INCREMENT.

La Figura 17.2 illustra che cosa accade a un contatore binario quando viene incrementato 16 volte: dal valore iniziale 0 al valore finale 16. All'inizio di ogni iterazione del ciclo **while** (righe 2–4) vogliamo aggiungere 1 nella posizione i. Se $A[i] = 1$, aggiungendo 1 nella posizione i, il bit 1 diventa 0 con un riporto di 1, che sarà aggiunto nella posizione $i + 1$ nella successiva iterazione del ciclo. Altrimenti il ciclo termina, e allora, se $i < k$, sappiamo che $A[i] = 0$; in questo caso, la riga 6 aggiunge 1 nella posizione i, cambiando 0 in 1. Il costo di ciascuna operazione INCREMENT è lineare nel numero di bit che cambiano valore.

Come nell'esempio dello stack, un'analisi superficiale produce un limite che è corretto, ma non stretto. Una singola esecuzione di INCREMENT richiede un tempo $\Theta(k)$ nel caso peggiore, che si verifica quando tutti gli elementi dell'array A valgono 1. Quindi, una sequenza di n operazioni INCREMENT su un contatore inizialmente a zero richiede un tempo $O(nk)$ nel caso peggiore.

Possiamo affinare la nostra analisi per ottenere un costo $O(n)$ nel caso peggiore per una sequenza di n operazioni INCREMENT, osservando che non tutti i bit cambiano ogni volta che viene chiamata la procedura INCREMENT. Come illustra la Figura 17.2, $A[0]$ cambia ogni volta che INCREMENT viene chiamata. Il successivo bit, $A[1]$, cambia una volta sì e una volta no: una sequenza di n operazioni INCREMENT su un contatore inizialmente a zero fa sì che $A[1]$ cambi $\lfloor n/2 \rfloor$ volte. Analogamente, il bit $A[2]$ cambia soltanto ogni quattro volte, ovvero $\lfloor n/4 \rfloor$ volte in una sequenza di n operazioni INCREMENT. In generale, per $i = 0, 1, \ldots, k-1$, il bit $A[i]$ cambia $\lfloor n/2^i \rfloor$ volte in una sequenza di n operazioni INCREMENT su un contatore inizialmente a zero. Per $i \geq k$, il bit $A[i]$ non esiste e quindi non può cambiare. Il numero totale di cambi nella sequenza è quindi

$$\begin{aligned}\sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor &< n \sum_{i=0}^{\infty} \frac{1}{2^i} \\ &= 2n\end{aligned}$$

per l'equazione (A.6). Il tempo nel caso peggiore per una sequenza di n operazioni INCREMENT su un contatore inizialmente nullo è dunque $O(n)$. Il costo medio di ciascuna operazione – e quindi il costo ammortizzato per operazione – è pari a $O(n)/n = O(1)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Esercizi

17.1-1

Se l'insieme delle operazioni sullo stack includesse un'operazione MULTIPUSH, che inserisce k elementi nello stack, il limite $O(1)$ sul costo ammortizzato delle operazioni sullo stack sarebbe ancora valido?

17.1-2

Dimostrate che, se un'operazione DECREMENT fosse inclusa nell'esempio del contatore di k bit, n operazioni potrebbero richiedere un tempo $\Theta(nk)$.

17.1-3

Una sequenza di n operazioni viene eseguita su una struttura dati. La i-esima operazione costa i se i è una potenza esatta di 2, altrimenti costa 1. Applicate il metodo dell'aggregazione per determinare il costo ammortizzato per operazione.

17.2 Il metodo degli accantonamenti

Nel ***metodo degli accantonamenti*** dell'analisi ammortizzata vengono assegnati costi variabili a operazioni differenti; qualche operazione potrebbe essere associata a un costo maggiore o minore del suo costo effettivo. Il costo che viene imputato a un'operazione è detto ***costo ammortizzato***. Quando il costo ammortizzato di un'operazione supera il costo effettivo, la differenza viene assegnata a specifici oggetti nella struttura dati sotto forma di ***credito***. Il credito potrà essere successivamente utilizzato per contribuire a pagare le operazioni il cui costo ammortizzato è minore del costo effettivo. Quindi, il costo ammortizzato di un'operazione può essere visto come se fosse suddiviso fra il costo effettivo e il credito, che può essere depositato o prelevato. Questo metodo è molto diverso dal metodo dell'aggregazione, dove tutte le operazioni hanno lo stesso costo ammortizzato.

I costi ammortizzati delle operazioni devono essere scelti con molta attenzione. Se vogliamo che l'analisi con i costi ammortizzati dimostri che nel caso peggiore il costo medio per operazione è piccolo, il costo ammortizzato totale di una sequenza di operazioni deve essere un limite superiore per il costo effettivo totale della sequenza. Inoltre, come nel metodo dell'aggregazione, questa relazione deve restare valida per tutte le sequenze di operazioni. Se indichiamo con c_i il costo effettivo della i-esima operazione e con $\widehat{c}_i$ il costo ammortizzato della i-esima operazione, deve essere

$$\sum_{i=1}^{n} \widehat{c}_i \geq \sum_{i=1}^{n} c_i \tag{17.1}$$

per ogni sequenza di n operazioni. Il credito totale memorizzato nella struttura dati è la differenza fra il costo ammortizzato totale e il costo effettivo totale, ovvero $\sum_{i=1}^{n} \widehat{c}_i - \sum_{i=1}^{n} c_i$. Per la disequazione (17.1), il credito totale associato alla struttura dati deve essere sempre non negativo. Se il credito totale potesse diventare negativo (sarebbe il risultato di attribuire un costo sottostimato alle prime operazioni con la promessa di saldare il loro debito successivamente), allora i costi ammortizzati totali sostenuti fino a quel momento sarebbero inferiori ai costi effettivi totali; per la sequenza delle operazioni eseguite fino a quel momento, il costo ammortizzato totale non potrebbe essere un limite superiore per il costo effettivo totale. Quindi, dobbiamo controllare attentamente che il credito totale nella struttura dati non diventi mai negativo.

Acquistato da Michele Michele su Webster il 2022-0[illegible]

Operazioni su stack

Per illustrare il metodo degli accantonamenti dell'analisi ammortizzata, riprendiamo l'esempio dello stack. Ricordiamo che i costi effettivi delle operazioni erano

PUSH	1
POP	1
MULTIPOP	$\min(k, s)$

dove k è l'argomento passato a MULTIPOP e s è la dimensione dello stack quando viene chiamata l'operazione MULTIPOP. Assegniamo i seguenti costi ammortizzati:

PUSH	2
POP	0
MULTIPOP	0

Notate che il costo ammortizzato di MULTIPOP è costante (0), mentre il costo effettivo è variabile. Qui, tutti e tre i costi ammortizzati sono $O(1)$, anche se in generale i costi ammortizzati delle operazioni in considerazione possono differire asintoticamente.

Adesso dimostriamo che qualsiasi sequenza di operazioni su stack può essere pagata utilizzando i costi ammortizzati. Supponiamo di utilizzare un conto in dollari per rappresentare ciascuna unità di costo. Iniziamo da uno stack vuoto. Ricordiamo l'analogia fra uno stack e una pila di piatti di una tavola calda (Paragrafo 10.1). Quando aggiungiamo un piatto alla pila (operazione PUSH), spendiamo un dollaro per pagare il costo effettivo di questa operazione e ci resta un credito di un dollaro (dei due dollari del costo ammortizzato), che lasciamo sul piatto. In qualsiasi momento, ogni piatto della pila ha un dollaro di credito su di esso.

Il dollaro lasciato sul piatto è il credito prepagato per il costo richiesto per toglierlo dalla pila (operazione POP). Quando eseguiamo un'operazione POP, non imputiamo un costo ammortizzato all'operazione e paghiamo il suo costo effettivo utilizzando il credito rimasto nella pila. Per togliere un piatto dalla pila, prendiamo il dollaro di credito che è sul piatto e lo utilizziamo per pagare il costo effettivo dell'operazione. Quindi, assegnando un costo ammortizzato un po' alto all'operazione PUSH, non abbiamo bisogno di attribuire un costo ammortizzato all'operazione POP.

Non occorre attribuire un costo ammortizzato neanche all'operazione MULTIPOP. Per togliere il primo piatto, prendiamo il dollaro di credito sul piatto e lo utilizziamo per pagare il costo effettivo di un'operazione POP. Per togliere il secondo piatto, abbiamo ancora un dollaro di credito sul piatto per pagare l'operazione POP e così via. Quindi, abbiamo sempre un credito sufficiente per pagare le operazioni MULTIPOP.

In altre parole, poiché ogni piatto nella pila ha un dollaro di credito su di esso e la pila ha sempre un numero non negativo di piatti, abbiamo la garanzia che l'ammontare del credito sarà sempre non negativo. Quindi, per *qualsiasi* sequenza di n operazioni PUSH, POP e MULTIPOP, il costo ammortizzato totale è un limite superiore per il costo effettivo totale. Poiché il costo ammortizzato totale è $O(n)$, anche il costo effettivo totale sarà $O(n)$.

Acquistato da Michele Michele Vertes il 2022-07-07 20:08 Numero Ordine Interno: 1095211.2 Copyright © 2022 McGraw-Hill Education (Italy)

Incrementare un contatore binario

Come altro esempio del metodo degli accantonamenti, analizziamo l'operazione INCREMENT su un contatore binario che inizia da zero. Come abbiamo osservato in precedenza, il tempo di esecuzione di questa operazione è proporzionale al numero di bit che cambiano valore, che utilizzeremo come costo per questo esempio. Utilizziamo ancora una volta un conto in dollari per rappresentare ciascuna unità di costo (il cambio del valore di un bit in questo esempio).

Per l'analisi ammortizzata, imputiamo un costo ammortizzato di due dollari per impostare un bit al valore 1. Dopo che un bit è stato impostato a 1, utilizziamo un dollaro (dei due dollari del costo ammortizzato) per pagare l'impostazione effettiva del bit e lasciamo sul bit l'altro dollaro come credito da utilizzare successivamente quando riporteremo il bit a 0. In qualsiasi momento, ogni bit 1 nel contatore ha un dollaro di credito su di esso e, quindi, non occorre imputare un costo ammortizzato per riportare un bit a 0; semplicemente paghiamo l'operazione per ripristinare il valore 0 con il dollaro di credito sul bit.

Ora possiamo calcolare il costo ammortizzato dell'operazione INCREMENT. Il costo per riportare i bit a 0 all'interno del ciclo **while** viene pagato dai dollari sui bit che passano da 1 a 0. Al massimo viene impostato un bit a 1 (riga 6 di INCREMENT) e quindi il costo ammortizzato di un'operazione INCREMENT è al più due dollari. Il numero dei bit che hanno valore 1 non è mai negativo, quindi l'ammontare del credito è sempre non negativo. Dunque, per n operazioni INCREMENT, il costo ammortizzato totale è $O(n)$, che limita il costo effettivo totale.

Esercizi

17.2-1

Una sequenza di operazioni su stack viene eseguita su uno stack la cui dimensione non supera mai k. Dopo ogni k operazioni, viene fatta una copia di backup dell'intero stack per motivi di sicurezza. Dimostrate che il costo di n operazioni su stack, inclusa la copia dello stack, è $O(n)$ assegnando dei costi ammortizzati appropriati alle varie operazioni.

17.2-2

Ripetete l'Esercizio 17.1-3 utilizzando il metodo degli accantonamenti.

17.2-3

Supponete non soltanto di incrementare un contatore, ma anche di riportarlo a zero (cioè, impostando tutti i suoi bit a 0). Spiegate come implementare un contatore come un array di bit in modo che qualsiasi sequenza di n operazioni INCREMENT e RESET impieghi un tempo $O(n)$ con un contatore inizialmente a zero (*suggerimento:* utilizzate un puntatore al bit 1 più significativo).

17.3 Il metodo del potenziale

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Anziché rappresentare il lavoro prepagato come credito memorizzato con specifici oggetti nella struttura dati, il ***metodo del potenziale*** dell'analisi ammortizzata rappresenta il lavoro prepagato come "energia potenziale" (o semplicemente "potenziale") che può essere liberata per pagare operazioni future. Il potenziale è associato alla struttura dati nel suo insieme, anziché a specifici oggetti all'interno della struttura dati.

Il metodo del potenziale funziona nel seguente modo. Si parte da una struttura dati iniziale D_0 sulla quale vengono eseguite n operazioni. Per ogni $i = 1, 2, \ldots, n$, indichiamo con c_i il costo effettivo della i-esima operazione e con D_i la struttura dati che si ottiene dopo avere applicato la i-esima operazione alla struttura dati D_{i-1}. Una ***funzione potenziale*** Φ associa ciascuna struttura dati D_i a un numero reale $\Phi(D_i)$, che è il ***potenziale*** della struttura dati D_i. Il ***costo ammortizzato*** $\widehat{c}_i$ della i-esima operazione rispetto alla funzione potenziale Φ è definito dalla seguente equazione:

$$\widehat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) \tag{17.2}$$

Il costo ammortizzato di ciascuna operazione è quindi il suo costo effettivo più l'incremento di potenziale dovuto all'operazione. Per l'equazione (17.2), il costo ammortizzato totale delle n operazioni è

$$\begin{aligned}\sum_{i=1}^{n} \widehat{c}_i &= \sum_{i=1}^{n} (c_i + \Phi(D_i) - \Phi(D_{i-1})) \\ &= \sum_{i=1}^{n} c_i + \Phi(D_n) - \Phi(D_0) \end{aligned} \tag{17.3}$$

La seconda uguaglianza deriva dall'equazione (A.9), perché i termini $\Phi(D_i)$ intermedi si annullano a due a due.

Se definiamo una funzione potenziale Φ tale che $\Phi(D_n) \geq \Phi(D_0)$, allora il costo ammortizzato totale $\sum_{i=1}^{n} \widehat{c}_i$ è un limite superiore per il costo effettivo totale $\sum_{i=1}^{n} c_i$. In pratica, non sempre sappiamo quante operazioni potrebbero essere eseguite. Quindi, se imponiamo che $\Phi(D_i) \geq \Phi(D_0)$ per ogni i, allora abbiamo la garanzia, come nel metodo degli accantonamenti, che paghiamo in anticipo. Spesso è comodo porre $\Phi(D_0)$ a 0 e poi dimostrare che $\Phi(D_i) \geq 0$ per ogni i (l'Esercizio 17.3-1 presenta un semplice modo di gestire i casi in cui $\Phi(D_0) \neq 0$).

Intuitivamente, se la differenza di potenziale $\Phi(D_i) - \Phi(D_{i-1})$ della i-esima operazione è positiva, allora il costo ammortizzato $\widehat{c}_i$ rappresenta un valore sovrastimato per la i-esima operazione, e il potenziale della struttura dati aumenta. Se la differenza di potenziale è negativa, allora il costo ammortizzato rappresenta un valore sottostimato per la i-esima operazione, e il costo effettivo dell'operazione è pagato dalla riduzione del potenziale. I costi ammortizzati definiti dalle equazioni (17.2) e (17.3) dipendono dalla scelta della funzione potenziale Φ. Differenti funzioni potenziale possono produrre costi ammortizzati differenti, che sono comunque limiti superiori per i costi effettivi. Spesso la scelta delle funzione potenziale è il risultato di qualche compromesso; la funzione potenziale migliore da utilizzare dipende dai limiti di tempo desiderati.

Operazioni su stack

Per applicare il metodo del potenziale, ritorniamo ancora una volta all'esempio dello stack e alle operazioni PUSH, POP e MULTIPOP. Definiamo la funzione potenziale Φ per uno stack come il numero di oggetti nello stack. Per uno stack vuoto D_0, dal quale iniziamo, abbiamo $\Phi(D_0) = 0$. Poiché il numero di oggetti nello stack non è mai negativo, lo stack D_i che si ottiene dopo la i-esima operazione ha un potenziale non negativo e quindi

$$\begin{aligned}\Phi(D_i) &\geq 0 \\ &= \Phi(D_0)\end{aligned}$$

Acquistato da Michele Mastrogiovanni (id 2023-07-27 21:41 Purchase Order: Libreria Universitaria 6 Dicembre 2023) per uso personale - © 2023 McGraw-Hill Education (Italy)

Il costo ammortizzato totale di n operazioni rispetto a Φ, quindi, rappresenta un limite superiore per il costo effettivo.

Calcoliamo adesso i costi ammortizzati delle varie operazioni sullo stack. Se la i-esima operazione su uno stack che contiene s oggetti è un'operazione PUSH, allora la differenza di potenziale è

$$\begin{aligned}\Phi(D_i) - \Phi(D_{i-1}) &= (s+1) - s \\ &= 1\end{aligned}$$

Per l'equazione (17.2), il costo ammortizzato di questa operazione PUSH è

$$\begin{aligned}\widehat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= 1 + 1 \\ &= 2\end{aligned}$$

Supponiamo che la i-esima operazione sullo stack sia MULTIPOP(S,k) e che $k' = \min(k,s)$ oggetti vengano eliminati dallo stack. Il costo effettivo dell'operazione è k', e la differenza di potenziale e

$$\Phi(D_i) - \Phi(D_{i-1}) = -k'$$

Quindi, il costo ammortizzato dell'operazione MULTIPOP è

$$\begin{aligned}\widehat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= k' - k' \\ &= 0\end{aligned}$$

Analogamente, il costo ammortizzato di un'ordinaria operazione POP è 0.

Il costo ammortizzato di ciascuna delle tre operazioni è $O(1)$, quindi il costo ammortizzato totale di una sequenza di n operazioni è $O(n)$. Avendo già dimostrato che $\Phi(D_i) \geq \Phi(D_0)$, il costo totale ammortizzato di n operazioni è un limite superiore per il costo totale effettivo. Il costo nel caso peggiore di n operazioni è quindi $O(n)$.

Incrementare un contatore binario

Come altro esempio del metodo del potenziale, esaminiamo di nuovo il problema di incrementare un contatore binario. Stavolta il potenziale del contatore dopo la i-esima operazione INCREMENT è definito dal valore b_i, il numero di bit 1 nel contatore dopo la i-esima operazione.

Calcoliamo il costo ammortizzato di un'operazione INCREMENT. Supponiamo che la i-esima operazione INCREMENT ripristini t_i bit a 0. Il costo effettivo dell'operazione è quindi al massimo $t_i + 1$, perché oltre a ripristinare t_i bit a 0, l'operazione imposta al massimo un bit a 1. Se $b_i = 0$, allora la i-esima operazione ripristina tutti i k bit a 0, e quindi $b_{i-1} = t_i = k$. Se $b_i > 0$, allora $b_i = b_{i-1} - t_i + 1$. In entrambi i casi, $b_i \leq b_{i-1} - t_i + 1$ e la differenza di potenziale è

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned}\Phi(D_i) - \Phi(D_{i-1}) &\leq (b_{i-1} - t_i + 1) - b_{i-1} \\ &= 1 - t_i\end{aligned}$$

Il costo ammortizzato è quindi

$$\begin{aligned}\widehat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &\leq (t_i + 1) + (1 - t_i) \\ &= 2\end{aligned}$$

Se il contatore inizia da zero, allora $\Phi(D_0) = 0$. Poiché $\Phi(D_i) \geq 0$ per ogni i, il costo ammortizzato totale di una sequenza di n operazioni INCREMENT è un limite superiore per il costo effettivo totale, e quindi il costo di n operazioni INCREMENT nel caso peggiore è $O(n)$.

Il metodo del potenziale è un semplice strumento per analizzare il contatore, anche quando il conteggio non inizia da zero. Ci sono inizialmente b_0 bit 1 e, dopo n operazioni INCREMENT, ci sono b_n bit 1, dove $0 \leq b_0, b_n \leq k$ (ricordiamo che k è il numero di bit nel contatore). Possiamo riscrivere l'equazione (17.3) in questo modo

$$\sum_{i=1}^{n} c_i = \sum_{i=1}^{n} \hat{c}_i - \Phi(D_n) + \Phi(D_0) \tag{17.4}$$

Abbiamo $\hat{c}_i \leq 2$ per ogni $1 \leq i \leq n$. Poiché $\Phi(D_0) = b_0$ e $\Phi(D_n) = b_n$, il costo effettivo totale di n operazioni INCREMENT è

$$\begin{aligned} \sum_{i=1}^{n} c_i &\leq \sum_{i=1}^{n} 2 - b_n + b_0 \\ &= 2n - b_n + b_0 \end{aligned}$$

Notate in particolare che, essendo $b_0 \leq k$, quando $k = O(n)$, il costo effettivo totale è $O(n)$. In altre parole, se eseguiamo almeno $n = \Omega(k)$ operazioni INCREMENT, il costo effettivo totale è $O(n)$, indipendentemente dal valore iniziale del contatore.

Esercizi

17.3-1

Supponete di avere una funzione potenziale Φ tale che $\Phi(D_i) \geq \Phi(D_0)$ per ogni i, ma $\Phi(D_0) \neq 0$. Dimostrate che esiste una funzione potenziale Φ' tale che $\Phi'(D_0) = 0$, $\Phi'(D_i) \geq 0$ per ogni $i \geq 1$, e che i costi ammortizzati con Φ' sono uguali ai costi ammortizzati con Φ.

17.3-2

Ripetete l'Esercizio 17.1-3 applicando il metodo del potenziale per l'analisi.

17.3-3

Supponete di avere un normale min-heap binario con n elementi che consente di eseguire le istruzioni INSERT e EXTRACT-MIN nel tempo $O(\lg n)$ nel caso peggiore. Definite una funzione potenziale Φ tale che il costo ammortizzato di INSERT sia $O(\lg n)$ e il costo ammortizzato di EXTRACT-MIN sia $O(1)$; dimostrate che la funzione opera correttamente.

17.3-4

Qual è il costo totale per eseguire n delle operazioni su stack PUSH, POP e MULTIPOP, supponendo che lo stack inizi con s_0 oggetti e finisca con s_n oggetti?

Acquistato da Michele Morabito su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

17.3-5

Supponete che un contatore inizi da un numero con b bit 1 nella sua rappresentazione binaria, anziché da 0. Dimostrate che il costo per eseguire n operazioni INCREMENT è $O(n)$ se $n = \Omega(b)$ (senza assumere che b sia costante).

17.3-6

Spiegate come implementare una coda con due stack ordinari (Esercizio 10.1-6) in modo che il costo ammortizzato di ciascuna operazione ENQUEUE e DEQUEUE sia $O(1)$.

17.3-7

Progettate una struttura dati per supportare le seguenti operazioni per un multiinsieme (insieme con elementi duplicati) dinamico S di interi:

INSERT(S, x) inserisce x in S.

DELETE-LARGER-HALF(S) cancella gli $\lceil |S|/2 \rceil$ elementi più grandi da S.

Spiegate come implementare questa struttura dati in modo che qualsiasi sequenza di m operazioni INSERT e DELETE-LARGER-HALF venga eseguita nel tempo $O(m)$. La vostra implementazione dovrebbe includere anche la possibilità di stampare gli elementi di S nel tempo $O(|S|)$.

17.4 Tavole dinamiche

In alcune applicazioni non sappiamo in anticipo quanti oggetti saranno memorizzati in una tavola. Potremmo allocare lo spazio in memoria per la tavola, per poi scoprire che non è sufficiente. In tal caso, la tavola deve essere riallocata con una dimensione maggiore e tutti gli oggetti memorizzati nella tavola originale dovranno essere copiati nella nuova tavola più grande. Analogamente, se sono stati cancellati molti oggetti dalla tavola, potrebbe essere conveniente riallocare la tavola con una dimensione più piccola. In questo paragrafo, analizzeremo questo problema di aumentare e diminuire dinamicamente la dimensione di una tavola. Applicando l'analisi ammortizzata, dimostreremo che il costo ammortizzato per eseguire un inserimento o una cancellazione è soltanto $O(1)$, anche se il costo effettivo di un'operazione è grande quando viene eseguita un'espansione o una contrazione della tavola. Inoltre, vedremo come garantire che lo spazio inutilizzato in una tavola dinamica non superi mai una frazione costante dello spazio totale.

Si suppone che la tavola dinamica supporti le operazioni TABLE-INSERT e TABLE-DELETE. TABLE-INSERT inserisce nella tavola un elemento che occupa una singola ***cella***, ovvero uno spazio per un elemento. Analogamente, l'operazione TABLE-DELETE può essere vista come l'eliminazione di un elemento dalla tavola, con conseguente liberazione della corrispondente cella. I dettagli del metodo utilizzato per organizzare la tavola non sono importanti; potremmo utilizzare uno stack (Paragrafo 10.1), un heap (Capitolo 6) o una tavola hash (Capitolo 11). Potremmo anche utilizzare un array o una collezione di array per implementare lo spazio in cui memorizzare gli oggetti, come abbiamo fatto nel Paragrafo 10.3.

Sarà utile applicare un concetto introdotto nell'analisi del processo di hashing (Capitolo 11). Definiamo ***fattore di carico*** $\alpha(T)$ di una tavola non vuota T il numero di elementi memorizzati nella tavola diviso per la dimensione (numero di celle) della tavola. Per definizione, una tavola vuota (senza elementi) ha dimensione 0 e il suo fattore di carico è 1. Se il fattore di carico di una tavola dinamica è limitato inferiormente da una costante, lo spazio inutilizzato nella tavola non supera mai una frazione costante della quantità totale di spazio.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707:0 Copyright © 2022, McGraw-Hill Education (Italy)

Iniziamo analizzando una tavola dinamica nella quale effettueremo soltanto l'operazione di inserimento; poi, considereremo il caso più generale in cui sono ammesse entrambe le operazioni di inserimento e cancellazione.

17.4.1 Espansione di una tavola

Supponiamo che lo spazio in memoria per una tavola sia allocato come un array di celle. Una tavola si riempie quando tutte le celle sono state utilizzate ovvero quando il suo fattore di carico è 1.[1] In alcuni ambienti software, se si tenta di inserire un elemento in una tavola piena, non c'è altra alternativa che terminare prematuramente l'operazione con un errore. Noi, invece, supponiamo che il nostro ambiente software, come molti di quelli moderni, disponga di un sistema di gestione della memoria che è in grado di allocare e rilasciare i blocchi di memoria su richiesta. Quindi, se un elemento viene inserito in una tavola piena, possiamo ***espandere*** la tavola allocando una nuova tavola che ha più celle della vecchia tavola. Poiché vogliamo che la tavola risieda in spazi di memoria contigui, dobbiamo allocare un nuovo array per la tavola più grande e poi copiare gli elementi della vecchia tavola in quella nuova.

Un tipico metodo euristico consiste nell'allocare una nuova tavola che ha il doppio di celle della vecchia tavola. Se effettuiamo soltanto inserimenti, il fattore di carico di una tavola è sempre almeno $1/2$ e, quindi, la quantità di spazio sprecato non supera mai la metà dello spazio totale nella tavola.

Nel seguente pseudocodice, supponiamo che T sia un oggetto che rappresenta la tavola. L'attributo $T.table$ contiene un puntatore al blocco di memoria che rappresenta la tavola. L'attributo $T.num$ contiene il numero di elementi della tavola; l'attributo $T.size$ è il numero totale di celle nella tavola. Inizialmente, la tavola è vuota: $T.num = T.size = 0$.

```
TABLE-INSERT(T, x)
 1  if T.size == 0
 2       alloca T.table con una cella
 3       T.size = 1
 4  if T.num == T.size
 5       alloca new-table con 2 · T.size celle
 6       inserisce tutti gli elementi di T.table in new-table
 7       rilascia T.table
 8       T.table = new-table
 9       T.size = 2 · T.size
10  inserisce x in T.table
11  T.num = T.num + 1
```

Notate che ci sono due procedure di "inserimento": la stessa procedura TABLE-INSERT e l'***inserimento elementare*** in una tavola nelle righe 6 e 10. Possiamo analizzare il tempo di esecuzione di TABLE-INSERT in base al numero di inserimenti elementari, assegnando un costo 1 a ogni inserimento elementare. Supponiamo che il tempo di esecuzione effettivo di TABLE-INSERT sia lineare nel tempo per inserire i singoli elementi e che quindi il costo aggiuntivo per allocare una tavola iniziale nella riga 2 sia costante e il costo aggiuntivo per allocare e rilasciare lo spazio nelle righe 5 e 7 sia dominato dal costo di trasferimento de-

Acquistato da Michele su Webster 1/2022 07/07/2022 Numero Ordine Libreria: 1995.3819 Copyright © 2022, McGraw-Hill Education (Italy)

[1] In alcuni casi, come nelle tavole hash a indirizzamento aperto, è preferibile considerare piena una tavola quando il suo fattore di carico è uguale a qualche costante strettamente minore di 1 (vedere l'Esercizio 17.4-1).

gli elementi nella riga 6. Chiamiamo ***espansione*** l'evento che si verifica quando viene eseguita la clausola **then** nelle righe 5–9.

Analizziamo una sequenza di n operazioni TABLE-INSERT su una tavola inizialmente vuota. Qual è il costo c_i della i-esima operazione? Se c'è spazio nella tavola corrente (o se questa è la prima operazione), allora $c_i = 1$, perché dobbiamo eseguire l'unico inserimento elementare nella riga 10. Se, invece, la tavola corrente è piena e si verifica un'espansione, allora $c_i = i$: il costo è 1 per l'inserimento elementare nella riga 10 più $i - 1$ per gli elementi che devono essere copiati dalla vecchia tavola in quella nuova (riga 6). Se vengono eseguite n operazioni, il costo nel caso peggiore di un'operazione è $O(n)$, che determina un limite superiore pari a $O(n^2)$ per il tempo di esecuzione totale di n operazioni.

Questo limite non è stretto, perché la tavola viene espansa raramente durante l'esecuzione di n operazioni TABLE-INSERT. Più precisamente, la i-esima operazione provoca un'espansione soltanto se $i - 1$ è una potenza esatta di 2. Infatti, il costo ammortizzato di un'operazione è $O(1)$, come possiamo dimostrare applicando il metodo dell'aggregazione. Il costo della i-esima operazione è

$$c_i = \begin{cases} i & \text{se } i-1 \text{ è una potenza esatta di 2} \\ 1 & \text{negli altri casi} \end{cases}$$

Il costo totale di n operazioni TABLE-INSERT è quindi

$$\begin{aligned} \sum_{i=1}^{n} c_i &\leq n + \sum_{j=0}^{\lfloor \lg n \rfloor} 2^j \\ &< n + 2n \\ &= 3n \end{aligned}$$

perché ci sono al massimo n operazioni che costano 1 e i costi delle restanti operazioni formano una serie geometrica. Poiché il costo totale di n operazioni TABLE-INSERT è $3n$, il costo ammortizzato di una singola operazione è 3.

Applicando il metodo degli accantonamenti, possiamo capire perché il costo ammortizzato di un'operazione TABLE-INSERT deve essere 3. Intuitivamente, ogni elemento paga per 3 inserimenti elementari: inserire sé stesso nella tavola corrente, spostare sé stesso quando la tavola viene espansa e spostare un altro elemento, che è già stato spostato una volta, quando la tavola viene espansa. Per esempio, supponiamo che la dimensione della tavola sia m immediatamente dopo un'espansione. Allora, il numero di elementi nella tavola è $m/2$ e la tavola non contiene alcun credito. Addebitiamo un costo di 3 dollari per ogni inserimento. L'inserimento elementare che viene effettuato subito costa 1 dollaro. Un altro dollaro viene posto come credito sull'elemento inserito. Il terzo dollaro è posto come credito su uno degli $m/2$ elementi già presenti nella tavola. Il riempimento della tavola richiede altri $m/2-1$ inserimenti; pertanto, quando la tavola contiene m elementi ed è piena, ogni elemento ha un dollaro per pagare il suo reinserimento durante l'espansione.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Anche il metodo del potenziale può essere utilizzato per analizzare una sequenza di n operazioni TABLE-INSERT e lo utilizzeremo nel Paragrafo 17.4.2 per progettare un'operazione TABLE-DELETE anch'essa con un costo ammortizzato $O(1)$. Iniziamo definendo una funzione potenziale Φ che è 0 subito dopo un'espansione, ma raggiunge la dimensione della tavola quando la tavola è piena, in

modo che la successiva espansione possa essere pagata dal potenziale. La funzione potenziale potrebbe essere questa:

$$\Phi(T) = 2 \cdot T.num - T.size \tag{17.5}$$

Subito dopo un'espansione, abbiamo $T.num = T.size/2$ e quindi $\Phi(T) = 0$, come volevamo. Immediatamente prima di un'espansione, abbiamo $T.num = T.size$ e quindi $\Phi(T) = T.num$, come volevamo. Il valore iniziale del potenziale è 0 e, poiché la tavola è almeno piena a metà, $T.num \geq T.size/2$, e questo implica che $\Phi(T)$ è sempre non negativo. Quindi, la somma dei costi ammortizzati di n operazioni TABLE-INSERT è un limite superiore per la somma dei costi effettivi.

Per analizzare il costo ammortizzato della i-esima operazione TABLE-INSERT, indichiamo con num_i il numero di elementi memorizzati nella tavola dopo la i-esima operazione, con $size_i$ la dimensione totale della tavola dopo la i-esima operazione e con Φ_i il potenziale dopo la i-esima operazione. Inizialmente, abbiamo $num_0 = 0$, $size_0 = 0$ e $\Phi_0 = 0$.

Se la i-esima operazione TABLE-INSERT non provoca un'espansione, allora abbiamo $size_i = size_{i-1}$ e il costo ammortizzato dell'operazione è

$$\begin{aligned}
\widehat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\
&= 1 + (2 \cdot num_i - size_i) - (2 \cdot num_{i-1} - size_{i-1}) \\
&= 1 + (2 \cdot num_i - size_i) - (2(num_i - 1) - size_i) \\
&= 3
\end{aligned}$$

Se la i-esima operazione provoca un'espansione, allora abbiamo $size_i = 2 \cdot size_{i-1}$ e $size_{i-1} = num_{i-1} = num_i - 1$; questo implica che $size_i = 2 \cdot (num_i - 1)$. Dunque, il costo ammortizzato dell'operazione è

$$\begin{aligned}
\widehat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\
&= num_i + (2 \cdot num_i - size_i) - (2 \cdot num_{i-1} - size_{i-1}) \\
&= num_i + (2 \cdot num_i - 2 \cdot (num_i - 1)) - (2(num_i - 1) - (num_i - 1)) \\
&= num_i + 2 - (num_i - 1) \\
&= 3
\end{aligned}$$

La Figura 17.3 rappresenta i valori di num_i, $size_i$ e Φ_i in funzione di i. Notate come il potenziale cresce per pagare l'espansione della tavola.

17.4.2 Espansione e contrazione di una tavola

Per implementare un'operazione TABLE-DELETE, è sufficiente eliminare l'elemento specificato dalla tavola. Tuttavia, spesso è preferibile ***contrarre*** la tavola quando il fattore di carico della tavola diventa troppo piccolo, in modo che lo spazio sprecato non sia eccessivo. La contrazione della tavola è simile all'espansione: quando il numero di elementi nella tavola diventa troppo piccolo, allochiamo una nuova tavola più piccola e poi copiamo gli elementi della vecchia tavola in quella nuova. Lo spazio in memoria per la vecchia tavola può essere poi rilasciato restituendolo al sistema di gestione della memoria. L'ideale sarebbe che fossero preservate due proprietà:

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

- il fattore di carico della tavola dinamica è limitato inferiormente da una costante.

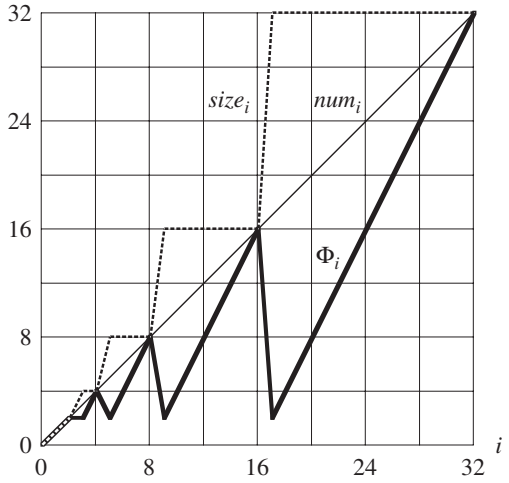


Figura 17.3 L'effetto di una sequenza di n operazioni TABLE-INSERT sul numero num_i di elementi della tavola, sul numero $size_i$ di celle della tavola e sul potenziale $\Phi_i = 2 \cdot num_i - size_i$; ciascuno di questi valori è misurato dopo la i-esima operazione. La linea sottile rappresenta num_i, la linea tratteggiata rappresenta $size_i$ e la linea più spessa rappresenta Φ_i. Notate che immediatamente prima di un'espansione, il potenziale è cresciuto fino a raggiungere il numero di elementi della tavola e, quindi, può pagare il trasferimento di tutti gli elementi nella nuova tavola. In seguito, il potenziale scende a 0, ma aumenta immediatamente di 2 quando l'elemento che ha causato l'espansione viene inserito nella tavola.

- il costo ammortizzato di un'operazione sulla tavola è limitato superiormente da una costante.

Supponiamo che il costo possa essere misurato in base al numero di inserimenti e cancellazioni elementari.

Potremmo ritenere che si debba raddoppiare la dimensione della tavola quando un elemento viene inserito in una tavola piena e dimezzare la dimensione quando una cancellazione fa sì che la tavola sia piena per meno della metà. Questa strategia assicura che il fattore di carico della tavola non sia mai minore di $1/2$, purtroppo però può comportare un aumento eccessivo del costo ammortizzato di un'operazione. Consideriamo il seguente scenario. Eseguiamo n operazioni su una tavola T, dove n è una potenza esatta di 2. Le prime $n/2$ operazioni sono inserimenti che, in base all'analisi precedente, hanno un costo totale pari a $\Theta(n)$. Alla fine di questa sequenza di inserimenti, $T.num = T.size = n/2$. Per le altre $n/2$ operazioni, eseguiamo la seguente sequenza:

I, D, D, I, I, D, D, I, I, . . .

dove I sta per inserimento e D sta per cancellazione. Il primo inserimento provoca un'espansione della tavola fino alla dimensione n. Le due successive cancellazioni provocano una contrazione della tavola, riportando la dimensione a $n/2$. I due successivi inserimenti provocano un'altra espansione, e così via. Il costo di ciascuna espansione e contrazione è $\Theta(n)$, e ce ne sono $\Theta(n)$ di queste operazioni. Quindi, il costo totale di n operazioni è $\Theta(n^2)$ e il costo ammortizzato di un'operazione è $\Theta(n)$.

Il difetto di questa strategia è ovvio: dopo un'espansione, non eseguiamo un numero sufficiente di cancellazioni per pagare una contrazione. Analogamente, dopo una contrazione, non eseguiamo un numero sufficiente di inserimenti per pagare un'espansione.

Acquistato da Michele Michele su Webster il 2022-07-25 12 - Numero Grande Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

È possibile migliorare questa strategia consentendo al fattore di carico della tavola di scendere sotto $1/2$. Specificatamente, continuiamo a raddoppiare la dimensione della tavola quando un elemento viene inserito in una tavola piena, ma dimezziamo la dimensione della tavola quando una cancellazione porta la tavola a essere piena per meno di $1/4$, anziché per meno di $1/2$. Il fattore di carico della tavola è quindi limitato inferiormente dalla costante $1/4$.

Intuitivamente, noi consideriamo come ideale il fattore di carico $1/2$, nel qual caso il potenziale della tavola sarà 0. Quando il fattore di carico si discosta da $1/2$, il potenziale aumenta in modo che, quando la tavola si espande o contrae, essa ha acquisito un potenziale sufficiente per pagare la copia di tutti gli elementi nella tavola appena allocata. Quindi, occorre una funzione potenziale che cresca fino a $T.num$ quando il fattore di carico aumenta fino a 1 o diminuisce fino a $1/4$. Dopo un'espansione o una contrazione della tabella, il fattore di carico ritorna a $1/2$ e il potenziale della tabella ritorna a 0.

Omettiamo il codice della procedura TABLE-DELETE, perché è simile a quello di TABLE-INSERT. Tuttavia, per l'analisi è utile supporre che, se il numero di elementi nella tavola scende a 0, lo spazio in memoria della tavola venga rilasciato; ovvero, se $T.num = 0$, allora $T.size = 0$.

Adesso possiamo utilizzare il metodo del potenziale per analizzare il costo di una sequenza di n operazioni TABLE-INSERT e TABLE-DELETE. Iniziamo definendo una funzione potenziale Φ che è 0 subito dopo un'espansione o contrazione e cresce quando il fattore di carico aumenta fino a 1 o diminuisce fino a $1/4$. Indichiamo con $\alpha(T) = T.num/T.size$ il fattore di carico di una tavola non vuota T. Poiché per una tavola vuota, $T.num = T.size = 0$ e $\alpha(T) = 1$, si ha sempre $T.num = \alpha(T) \cdot T.size$, indipendentemente dal fatto che la tavola sia vuota oppure no. Nella nostra analisi utilizzeremo la seguente funzione potenziale

$$\Phi(T) = \begin{cases} 2 \cdot T.num - T.size & \text{se } \alpha(T) \geq 1/2 \\ T.size/2 - T.num & \text{se } \alpha(T) < 1/2 \end{cases} \tag{17.6}$$

Notate che il potenziale di una tavola vuota è 0 e che il potenziale non è mai negativo. Quindi, il costo ammortizzato totale di una sequenza di operazioni rispetto a Φ è un limite superiore per il costo effettivo della sequenza.

Prima di procedere con l'analisi, è importante esaminare alcune proprietà della funzione potenziale, come illustra la Figura 17.4. Notate che quando il fattore di carico è $1/2$, il potenziale è 0. Quando il fattore di carico è 1, si ha $T.size = T.num$, che implica $\Phi(T) = T.num$; quindi il potenziale può pagare un'espansione se viene inserito un elemento. Quando il fattore di carico è $1/4$, si ha $T.size = 4 \cdot T.num$, che implica $\Phi(T) = T.num$; quindi il potenziale può pagare una contrazione se viene cancellato un elemento.

Per analizzare una sequenza di n operazioni TABLE-INSERT e TABLE-DELETE, indichiamo con c_i il costo effettivo della i-esima operazione, con $\hat{c}_i$ il suo costo ammortizzato rispetto a Φ, con num_i il numero di elementi memorizzati nella tavola dopo la i-esima operazione, con $size_i$ la dimensione totale della tavola dopo la i-esima operazione, con α_i il fattore di carico della tavola dopo la i-esima operazione e con Φ_i il potenziale dopo la i-esima operazione. Inizialmente, $num_0 = 0$, $size_0 = 0$, $\alpha_0 = 1$ e $\Phi_0 = 0$.

Esaminiamo prima il caso in cui la i-esima operazione è TABLE-INSERT. Se $\alpha_{i-1} \geq 1/2$, l'analisi è identica a quella dell'espansione della tavola (Paragrafo 17.4.1). Indipendentemente dal fatto che la tavola si espanda oppure no, il costo ammortizzato $\hat{c}_i$ dell'operazione è al più 3. Se $\alpha_{i-1} < 1/2$, la tavola non può espandersi per effetto dell'operazione, perché l'espansione si verifica solo quando $\alpha_{i-1} = 1$. Se anche $\alpha_i < 1/2$, allora il costo ammortizzato della i-esima operazione è

$$\begin{aligned} \hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= 1 + (size_i/2 - num_i) - (size_{i-1}/2 - num_{i-1}) \end{aligned}$$

Acquistato da Michele Mele - Vietata la distribuzione e la copia non autorizzata - Copyright © 2022, McGraw-Hill Education (Italy)

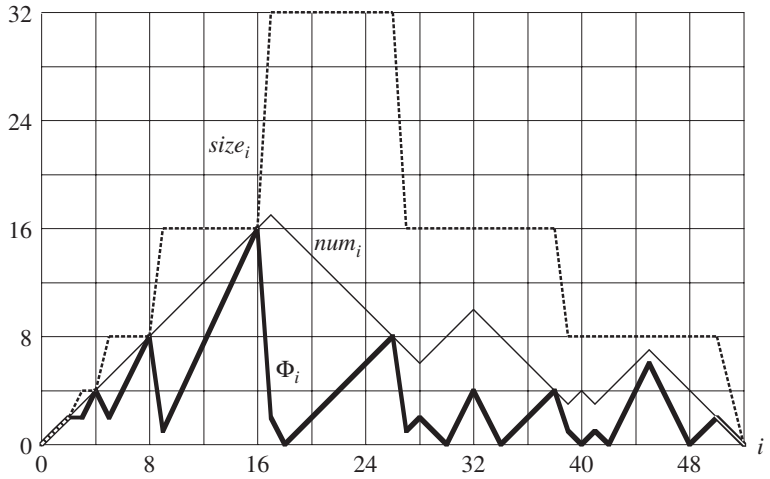


Figura 17.4 L'effetto di una sequenza di n operazioni TABLE-INSERT e TABLE-DELETE sul numero num_i di elementi della tavola, sul numero $size_i$ di celle della tavola e sul potenziale

$$\Phi_i = \begin{cases} 2 \cdot num_i - size_i & \text{se } \alpha_i \geq 1/2 \\ size_i/2 - num_i & \text{se } \alpha_i < 1/2 \end{cases}$$

Ciascuno di questi valori è misurato dopo la i-esima operazione. La linea sottile rappresenta num_i, la linea tratteggiata rappresenta $size_i$ e la linea più spessa rappresenta Φ_i. Notate che immediatamente prima di un'espansione, il potenziale è cresciuto fino a raggiungere il numero di elementi della tavola e, quindi, può pagare il trasferimento di tutti gli elementi nella nuova tavola. Analogamente, immediatamente prima di una contrazione, il potenziale è cresciuto fino a raggiungere il numero di elementi della tavola.

$$\begin{aligned} &= 1 + (size_i/2 - num_i) - (size_i/2 - (num_i - 1)) \\ &= 0 \end{aligned}$$

Se $\alpha_{i-1} < 1/2$, ma $\alpha_i \geq 1/2$, allora

$$\begin{aligned} \widehat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= 1 + (2 \cdot num_i - size_i) - (size_{i-1}/2 - num_{i-1}) \\ &= 1 + (2(num_{i-1} + 1) - size_{i-1}) - (size_{i-1}/2 - num_{i-1}) \\ &= 3 \cdot num_{i-1} - \frac{3}{2} size_{i-1} + 3 \\ &= 3\alpha_{i-1} size_{i-1} - \frac{3}{2} size_{i-1} + 3 \\ &< \frac{3}{2} size_{i-1} - \frac{3}{2} size_{i-1} + 3 \\ &= 3 \end{aligned}$$

Dunque, il costo ammortizzato di un'operazione TABLE-INSERT è al più 3.

Acquistato da Michele Michele su Webster il 2022-07-0[illegible] Copyright © 2022 Addison-Wesley (Italy)

Ritorniamo al caso in cui la i-esima operazione è TABLE-DELETE. In questo caso, $num_i = num_{i-1} - 1$. Se $\alpha_{i-1} < 1/2$, allora dobbiamo vedere se l'operazione provoca una contrazione. Se non la provoca, allora $size_i = size_{i-1}$ e il costo ammortizzato dell'operazione è

$$\begin{aligned} \widehat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= 1 + (size_i/2 - num_i) - (size_{i-1}/2 - num_{i-1}) \end{aligned}$$

$$
\begin{aligned}
&= 1 + (size_i/2 - num_i) - (size_i/2 - (num_i + 1)) \\
&= 2
\end{aligned}
$$

Se $\alpha_{i-1} < 1/2$ e la i-esima operazione provoca una contrazione, allora il costo effettivo dell'operazione è $c_i = num_i + 1$, perché viene cancellato un elemento e vengono spostati num_i elementi. Abbiamo $size_i/2 = size_{i-1}/4 = num_{i-1} = num_i + 1$ e il costo ammortizzato dell'operazione è

$$
\begin{aligned}
\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\
&= (num_i + 1) + (size_i/2 - num_i) - (size_{i-1}/2 - num_{i-1}) \\
&= (num_i + 1) + ((num_i + 1) - num_i) - ((2 \cdot num_i + 2) - (num_i + 1)) \\
&= 1
\end{aligned}
$$

Quando la i-esima operazione è TABLE-DELETE e $\alpha_{i-1} \geq 1/2$, anche il costo ammortizzato è limitato superiormente da una costante. Lasciamo al lettore questa analisi (vedere l'Esercizio 17.4-2).

In sintesi, poiché il costo ammortizzato di ciascuna operazione è limitato superiormente da una costante, il tempo effettivo per qualsiasi sequenza di n operazioni su una tavola dinamica è $O(n)$.

Esercizi

17.4-1

Supponete di volere implementare una tavola hash dinamica a indirizzamento aperto. Perché potremmo voler considerare piena la tavola quando il suo fattore di carico raggiunge un valore α che è strettamente minore di 1? Descrivete brevemente come effettuare un inserimento in una tavola hash dinamica a indirizzamento aperto in modo tale che il valore atteso del costo ammortizzato per l'inserimento sia $O(1)$. Perché il valore atteso del costo effettivo per l'inserimento non è necessariamente $O(1)$ per tutti gli inserimenti?

17.4-2

Dimostrate che, se $\alpha_{i-1} \geq 1/2$ e la i-esima operazione su una tavola dinamica è TABLE-DELETE, il costo ammortizzato dell'operazione rispetto alla funzione potenziale (17.6) è limitato superiormente da una costante.

17.4-3

Anziché contrarre una tavola dimezzando la sua dimensione quando il fattore di carico scende sotto $1/4$, supponete di contrarre la tavola moltiplicando la sua dimensione per $2/3$ quando il fattore di carico scende sotto $1/3$. Utilizzando la funzione potenziale

$$\Phi(T) = |2 \cdot T.num - T.size|$$

dimostrate che il costo ammortizzato di un'operazione TABLE-DELETE che usa questa strategia è limitato superiormente da una costante.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Problemi

17-1 Contatore binario a bit invertiti

Il Capitolo 30 esamina un importante algoritmo: Fast Fourier Transform (trasformata rapida di Fourier) o FFT. Il primo passo di questo algoritmo esegue una ***permutazione per inversione dei bit*** su un array di input $A[0\,..\,n-1]$, la cui lun-

ghezza è $n = 2^k$ per qualche intero k non negativo. Questa permutazione scambia gli elementi i cui indici hanno rappresentazioni binarie che sono una l'inverso dell'altra.

Possiamo esprimere ciascun indice a come una sequenza di k bit $\langle a_{k-1}, a_{k-2}, \ldots, a_0 \rangle$, dove $a = \sum_{i=0}^{k-1} a_i \, 2^i$. Definiamo

$$\text{rev}_k(\langle a_{k-1}, a_{k-2}, \ldots, a_0 \rangle) = \langle a_0, a_1, \ldots, a_{k-1} \rangle$$

quindi

$$\text{rev}_k(a) = \sum_{i=0}^{k-1} a_{k-i-1} 2^i$$

Per esempio, se $n = 16$ (ovvero $k = 4$), allora $\text{rev}_k(3) = 12$, in quanto la rappresentazione a 4 bit di 3 è 0011 che, quando viene invertita, diventa 1100, che è la rappresentazione a 4 bit di 12.

a. Data una funzione rev_k che viene eseguita nel tempo $\Theta(k)$, scrivete un algoritmo per effettuare la permutazione per inversione dei bit su un array di lunghezza $n = 2^k$ nel tempo $O(nk)$.

Possiamo utilizzare un algoritmo basato sull'analisi ammortizzata per migliorare il tempo di esecuzione della permutazione per inversione dei bit. Utilizziamo un "contatore a bit invertiti" e una procedura BIT-REVERSED-INCREMENT che, dato un contatore a bit invertiti di valore a, produce $\text{rev}_k(\text{rev}_k(a) + 1)$. Per esempio, se $k = 4$ e il contatore a bit invertiti inizia da 0, allora le successive chiamate di BIT-REVERSED-INCREMENT producono la sequenza

$$0000, 1000, 0100, 1100, 0010, 1010, \ldots = 0, 8, 4, 12, 2, 10, \ldots$$

b. Supponete che le parole nel vostro calcolatore memorizzino valori di k bit e che il calcolatore sia in grado di elaborare nell'unità di tempo i valori binari con operazioni quali lo scorrimento a sinistra o a destra di quantità arbitrarie, AND bit a bit, OR bit a bit e così via. Descrivete un'implementazione della procedura BIT-REVERSED-INCREMENT che consente di eseguire la permutazione per inversione dei bit su un array di n elementi nel tempo totale $O(n)$.

c. Supponete che sia possibile fare scorrere una parola a sinistra o a destra di un solo bit nell'unità di tempo. È ancora possibile implementare una permutazione per inversione dei bit nel tempo $O(n)$?

17-2 Effettuare una ricerca binaria dinamica

La ricerca binaria in un array ordinato viene eseguita in tempo logaritmico, mentre il tempo per inserire un nuovo elemento è lineare nella dimensione dell'array. È possibile migliorare il tempo di inserimento utilizzando più array ordinati.

Acquistato da Michele Michele su Webster il 2022-07-07 Copyright © 2022, McGraw-Hill Education (Italy)

Più precisamente, vogliamo eseguire le operazioni SEARCH e INSERT in un insieme di n elementi. Sia $k = \lceil \lg(n+1) \rceil$ e sia $\langle n_{k-1}, n_{k-2}, \ldots, n_0 \rangle$ la rappresentazione binaria di n. Abbiamo k array ordinati $A_0, A_1, \ldots, A_{k-1}$; la lunghezza dell'array A_i è 2^i, per $i = 0, 1, \ldots, k-1$. Un array è pieno se $n_i = 1$, è vuoto se $n_i = 0$. Il numero totale di elementi contenuti in tutti i k array è quindi $\sum_{i=0}^{k-1} n_i \, 2^i = n$. Sebbene ciascun array sia ordinato, non c'è una particolare relazione fra gli elementi dei vari array.

a. Descrivete come eseguire l'operazione SEARCH per questa struttura dati. Analizzate il suo tempo di esecuzione nel caso peggiore.

b. Descrivete come inserire un nuovo elemento in questa struttura dati. Analizzate il tempo di esecuzione nel caso peggiore e il tempo di esecuzione ammortizzato.

c. Descrivete come implementare DELETE.

17-3 Alberi ammortizzati bilanciati in peso

Considerate un ordinario albero binario di ricerca che è stato aumentato aggiungendo a ciascun nodo x l'attributo $x.size$, che contiene il numero di chiavi memorizzate nel sottoalbero con radice in x. Sia α una costante nell'intervallo $1/2 \leq \alpha < 1$. Diciamo che un dato nodo x è ***α-bilanciato*** se $x.left.size \leq \alpha \cdot x.size$ e $x.right.size \leq \alpha \cdot x.size$. L'albero è globalmente α-bilanciato se ogni suo nodo è α-bilanciato. Il seguente approccio ammortizzato per mantenere gli alberi bilanciati in peso è stato proposto da G. Varghese.

a. Un albero $1/2$-bilanciato è, in un certo senso, il più bilanciato possibile. Dato un nodo x in un arbitrario albero binario di ricerca, spiegate come ricostruire il sottoalbero con radice in x in modo che diventi $1/2$-bilanciato. Il vostro algoritmo dovrebbe essere eseguito nel tempo $\Theta(x.size)$ e può utilizzare uno spazio ausiliario di memoria pari a $O(x.size)$.

b. Dimostrate che, per svolgere una ricerca in un albero binario di ricerca α-bilanciato di n nodi, occorre un tempo $O(\lg n)$ nel caso peggiore.

Per la parte restante di questo problema, supponete che la costante α sia strettamente più grande di $1/2$. Supponete che le operazioni INSERT e DELETE siano implementate, come al solito, per un albero binario di ricerca di n nodi, con la differenza che, dopo avere eseguito ciascuna di tali operazioni, se alcuni nodi nell'albero non sono più α-bilanciati, allora il sottoalbero con radice nel più alto di tali nodi viene "ricostruito" in modo che diventi $1/2$-bilanciato.

Analizzeremo questo schema di ricostruzione utilizzando il metodo del potenziale. Per un nodo x in un albero binario di ricerca T, definiamo

$$\Delta(x) = |x.left.size - x.right.size|$$

e definiamo il potenziale di T come

$$\Phi(T) = c \sum_{x \in T : \Delta(x) \geq 2} \Delta(x)$$

dove c è una costante sufficientemente grande che dipende da α.

c. Dimostrate che qualsiasi albero binario di ricerca ha potenziale non negativo e che un albero $1/2$-bilanciato ha potenziale 0.

Acquistato da Michele Mi[illegible] il 2022-07-07 22:12 (unported) Proprietà letteraria riservata 10850318 220707 0 Copyright © 2022 McGraw-Hill Education (Italy)

d. Supponete che m unità di potenziale possano pagare la ricostruzione di un sottoalbero di m nodi. Quanto deve essere grande c (in termini di α) in modo che la ricostruzione di un sottoalbero che non è α-bilanciato possa impiegare un tempo ammortizzato $O(1)$?

e. Dimostrate che l'inserimento o la cancellazione di un nodo in un albero α-bilanciato di n nodi ha un tempo ammortizzato pari a $O(\lg n)$.

17-4 Il costo di ristrutturazione degli alberi rosso-neri

Ci sono quattro operazioni fondamentali degli alberi rosso-neri che effettuano delle ***modifiche strutturali***: inserimento di nodi, cancellazione di nodi, rotazione e ricolorazione. Abbiamo visto che RB-INSERT e RB-DELETE usano soltanto $O(1)$ rotazioni, inserimenti e cancellazioni di nodi per mantenere le proprietà degli alberi rosso-neri, ma potrebbero fare molte più ricolorazioni.

a. Descrivete un valido albero rosso-nero con n nodi tale che, chiamando RB-INSERT per aggiungere l'$(n+1)$-esimo nodo, si abbiano $\Omega(\lg n)$ ricolorazioni. Poi descrivete un valido albero rosso-nero con n nodi tale che, chiamando RB-DELETE per un particolare nodo, si abbiano $\Omega(\lg n)$ ricolorazioni.

Anche se nel caso peggiore il numero di ricolorazioni per operazione può essere logaritmico, dovremo dimostrare che qualsiasi sequenza di m operazioni RB-INSERT e RB-DELETE con un albero rosso-nero inizialmente vuoto provoca $O(m)$ modifiche strutturali nel caso peggiore. Notate che ogni ricolorazione di un nodo è conteggiata come modifica strutturale.

b. Alcuni dei casi gestiti dal ciclo principale del codice di entrambe le operazioni RB-INSERT-FIXUP e RB-DELETE-FIXUP sono ***conclusivi***: una volta incontrati, provocano la fine del ciclo dopo un numero costante di operazioni aggiuntive. Per ciascuno dei casi di RB-INSERT-FIXUP e RB-DELETE-FIXUP, specificate quali sono conclusivi e quali no (*suggerimento:* esaminate le Figure 13.5, 13.6 e 13.7).

Innanzi tutto dobbiamo analizzare le modifiche strutturali quando sono eseguiti soltanto gli inserimenti. Sia T un albero rosso-nero e indichiamo con $\Phi(T)$ il numero di nodi rossi in T. Supponiamo che una unità di potenziale possa pagare le modifiche strutturali effettuate da uno qualsiasi dei casi di RB-INSERT-FIXUP.

c. Sia T' l'albero che si ottiene applicando il Caso 1 di RB-INSERT-FIXUP a T. Dimostrate che $\Phi(T') = \Phi(T) - 1$.

d. L'inserimento di un nodo in un albero rosso-nero mediante RB-INSERT può essere suddiviso in tre parti. Elencate le modifiche strutturali e le variazioni del potenziale causate dalle righe 1–16 di RB-INSERT, dai casi non conclusivi di RB-INSERT-FIXUP e dai casi conclusivi di RB-INSERT-FIXUP.

e. Utilizzando il punto (d), dimostrate che il numero ammortizzato di modifiche strutturali effettuate da una chiamata qualsiasi di RB-INSERT è $O(1)$.

Adesso vogliamo provare che si verificano $O(m)$ modifiche strutturali quando ci sono entrambe le operazioni di inserimento e cancellazione. Definiamo per ogni nodo x

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$w(x) = \begin{cases} 0 & \text{se } x \text{ è rosso} \\ 1 & \text{se } x \text{ è nero e non ha figli rossi} \\ 0 & \text{se } x \text{ è nero e ha un figlio rosso} \\ 2 & \text{se } x \text{ è nero e ha due figli rossi} \end{cases}$$

Adesso ridefiniamo il potenziale di un albero rosso-nero T in questo modo:

$$\Phi(T) = \sum_{x \in T} w(x)$$

Indichiamo con T' l'albero che si ottiene applicando a T qualsiasi caso non conclusivo di RB-INSERT-FIXUP o RB-DELETE-FIXUP.

f. Provate che $\Phi(T') \leq \Phi(T) - 1$ per ogni caso non conclusivo di RB-INSERT-FIXUP. Deducete che il numero ammortizzato di modifiche strutturali effettuate da una chiamata qualsiasi di RB-INSERT-FIXUP è $O(1)$.

g. Provate che $\Phi(T') \leq \Phi(T) - 1$ per ogni caso non conclusivo di RB-DELETE-FIXUP. Deducete che il numero ammortizzato di modifiche strutturali effettuate da una chiamata qualsiasi di RB-DELETE-FIXUP è $O(1)$.

h. Completate la dimostrazione che, nel caso peggiore, qualsiasi sequenza di m operazioni RB-INSERT e RB-DELETE effettua $O(m)$ modifiche strutturali.

17-5 Analisi competitiva di liste auto-organizzanti con l'euristica move-to-front

Una ***lista auto-organizzante*** è una lista concatenata di n elementi, in cui ciascun elemento ha una chiave unica. Quando ricerchiamo un elemento nella lista, conosciamo la chiave, e vogliamo trovare l'elemento che ha questa chiave.

Una lista auto-organizzante ha due importanti proprietà:

1. Per trovare un elemento nella lista, nota la sua chiave, occorre attraversare la lista dall'inizio finché non viene trovato l'elemento con la chiave data. Se l'elemento occupa la k-esima posizione dall'inizio della lista, il costo per trovarlo è k.
2. È possibile riordinare gli elementi della lista dopo un'operazione qualsiasi, secondo una determinata regola con un dato costo. Possiamo scegliere l'euristica che preferiamo per riordinare la lista.

Supponiamo di iniziare da una lista di n elementi e di avere una sequenza $\sigma = \langle \sigma_1, \sigma_2, \ldots, \sigma_m \rangle$ di accesso a tale lista, dove $\sigma_1, \sigma_2, \ldots, \sigma_m$ sono chiavi da trovare nell'ordine specificato. Il costo della sequenza è la somma dei costi dei singoli accessi della sequenza.

Tra i tanti modi possibili di riordinare la lista dopo un'operazione, questo problema usa la trasposizione di elementi adiacenti – scambiando le loro posizioni nella lista – con un costo unitario per ogni operazione di trasposizione. Dimostreremo, mediante una funzione potenziale, che una particolare euristica per riordinare la lista, *move-to-front*, ha un costo totale non maggiore di 4 volte quello di qualsiasi altra euristica per mantenere ordinata la lista – anche se quest'altra euristica conosce in anticipo la sequenza di accesso! Questo tipo di analisi è detto ***analisi competitiva***.

Acquistato da Michele Michele su Webster il 2022-07-07 13:12 Numero Ordine Libreria: 1993038210-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Per un'euristica H e un ordinamento iniziale della lista, indichiamo con $C_H(\sigma)$ il costo di accesso della sequenza σ. Sia m il numero di accessi in σ.

a. Dimostrate che, se l'euristica H non conosce la sequenza di accesso in anticipo, allora il costo nel caso peggiore per H con una sequenza di accesso σ è $C_H(\sigma) = \Omega(mn)$.

Con l'euristica ***move-to-front***, subito dopo la ricerca di un elemento x, spostiamo x nella prima posizione della lista (per questo l'euristica è detta *move-to-front*).

Indichiamo con $\text{rank}_L(x)$ il rango dell'elemento x nella lista L, ovvero la posizione di x nella lista L. Per esempio, se x è il quarto elemento in L, allora $\text{rank}_L(x) = 4$. Indichiamo con c_i il costo dell'accesso σ_i utilizzando l'euristica move-to-front, che include il costo per trovare l'elemento nella lista e il costo per spostarlo all'inizio della lista mediante una serie di trasposizioni di elementi adiacenti nella lista.

b. Dimostrate che se σ_i accede all'elemento x nella lista L utilizzando l'euristica move-to-front, allora $c_i = 2 \cdot \text{rank}_L(x) - 1$.

Adesso confrontiamo move-to-front con un'altra euristica H che elabora una sequenza di accesso secondo le due precedenti proprietà. L'euristica H può trasporre gli elementi della lista in qualsiasi modo e può anche conoscere l'intera sequenza di accesso in anticipo.

Sia L_i la lista dopo l'accesso σ_i mediante l'euristica move-to-front; sia L_i^* la lista dopo l'accesso σ_i mediante l'euristica H. Indichiamo con c_i il costo dell'accesso σ_i per l'euristica move-to-front e con c_i^* il costo per l'euristica H. Supponiamo che l'euristica H effettui t_i^* trasposizioni durante l'accesso σ_i.

c. Nel punto (b) avete dimostrato che $c_i = 2 \cdot \text{rank}_{L_{i-1}}(x) - 1$. Adesso dimostrate che $c_i^* = \text{rank}_{L_{i-1}^*}(x) + t_i^*$.

Definiamo ***inversione*** nella lista L_i una coppia di elementi y e z tale che y precede z in L_i e z precede y in L_i^*. Supponiamo che la lista L_i abbia q_i inversioni dopo l'elaborazione della sequenza di accesso $\langle \sigma_1, \sigma_2, \ldots, \sigma_i \rangle$. Definiamo quindi una funzione potenziale Φ che associa L_i a un numero reale tramite $\Phi(L_i) = 2q_i$. Per esempio, se L_i ha gli elementi $\langle e, c, a, d, b \rangle$ e L_i^* ha gli elementi $\langle c, a, b, d, e \rangle$, allora L_i ha 5 inversioni $((e,c),(e,a),(e,d),(e,b),(d,b))$, e quindi $\Phi(L_i) = 10$. Notate che $\Phi(L_i) \geq 0$ per ogni i e che, se le euristiche move-to-front e H iniziano con la stessa lista L_0, allora $\Phi(L_0) = 0$.

d. Dimostrate che una trasposizione aumenta il potenziale di 2 o lo riduce di 2.

Supponiamo che l'accesso σ_i trovi l'elemento x. Per capire come il potenziale cambi a causa di σ_i, suddividiamo gli elementi diversi da x in quattro insiemi, in funzione della posizione che occupavano prima dell'i-esimo accesso:

- L'insieme A è formato dagli elementi che precedono x in L_{i-1} e L_{i-1}^*.
- L'insieme B è formato dagli elementi che precedono x in L_{i-1} e che seguono x in L_{i-1}^*.
- L'insieme C è formato dagli elementi che seguono x in L_{i-1} e che precedono x in L_{i-1}^*.
- L'insieme D è formato dagli elementi che seguono x in L_{i-1} e L_{i-1}^*.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

e. Dimostrate che $\text{rank}_{L_{i-1}}(x) = |A| + |B| + 1$ e $\text{rank}_{L_{i-1}^*}(x) = |A| + |C| + 1$.

f. Dimostrate che l'accesso σ_i provoca una variazione di potenziale pari a

$$\Phi(L_i) - \Phi(L_{i-1}) \leq 2(|A| - |B| + t_i^*)$$

dove, come prima, l'euristica H effettua t_i^* trasposizioni durante l'accesso σ_i.

Indicate con $\widehat{c}_i = c_i + \Phi(L_i) - \Phi(L_{i-1})$ il costo ammortizzato $\widehat{c}_i$ dell'accesso σ_i.

g. Dimostrate che il costo ammortizzato $\widehat{c}_i$ dell'accesso σ_i è limitato superiormente da $4c_i^*$.

h. Concludete che il costo $C_{\text{MTF}}(\sigma)$ della sequenza di accesso σ con l'euristica move-to-front è al massimo 4 volte il costo $C_H(\sigma)$ di σ con qualsiasi euristica H, supponendo che entrambe le euristiche inizino dalla stessa lista.

Note

Aho, Hopcroft e Ullman [5] hanno usato l'analisi aggregata per determinare il tempo di esecuzione delle operazioni su di una foresta di insiemi disgiunti; nel Capitolo 21 analizzeremo questa struttura dati con il metodo del potenziale. Tarjan [332] descrive i metodi degli accantonamenti e del potenziale dell'analisi ammortizzata e presenta numerose applicazioni. Attribuisce il metodo degli accantonamenti a più autori, fra i quali citiamo M. R. Brown, R. E. Tarjan, S. Huddleston e K. Mehlhorn. Secondo Tarjan, il metodo del potenziale è da attribuire a D. D. Sleator. Il termine "ammortizzato" è dovuto a D. D. Sleator e R. E. Tarjan.

Le funzioni potenziale sono utili anche per dimostrare i limiti inferiori di particolari tipi di problemi. Per ogni configurazione del problema, definiamo una funzione potenziale che associa la configurazione a un numero reale. Poi determiniamo il potenziale Φ_{init} della configurazione iniziale, il potenziale Φ_{final} della configurazione finale e la variazione massima del potenziale $\Delta\Phi_{\text{max}}$ dovuta a un passo qualsiasi. Quindi, il numero di passi deve essere almeno $|\Phi_{\text{final}} - \Phi_{\text{init}}| \,/\, |\Delta\Phi_{\text{max}}|$. Cormen, Sundquist e Wisniewski [80], Floyd [108] e Aggarwal e Vitter [3] hanno descritto alcune applicazioni delle funzioni potenziale per dimostrare i limiti inferiori nella complessità di I/O. Krumme, Cybenko e Venkataraman [222] hanno applicato le funzioni potenziale per dimostrare i limiti inferiori nel ***gossiping*** (pettegolezzo): comunicare un singolo dato da ciascun vertice di un grafo a tutti gli altri vertici.

L'euristica move-to-front del Problema 17-5 funziona molto bene nei casi pratici. Inoltre, se ravvisiamo che quando troviamo un elemento, è possibile staccarlo dalla sua posizione corrente e riposizionarlo all'inizio della lista in un tempo costante, possiamo dimostrare che il costo dell'euristica move-to-front è al massimo 2 volte il costo di altre euristiche, incluse quelle che conoscono in anticipo la sequenza di accesso.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

V Strutture dati avanzate

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Introduzione

Questa parte riprende l'esame delle strutture dati che supportano le operazioni con gli insiemi dinamici, ma a un livello un po' più avanzato rispetto alla Parte III. Per esempio, due capitoli fanno ampio uso delle tecniche dell'analisi ammortizzata che abbiamo descritto nel Capitolo 17.

Il Capitolo 18 presenta i B-alberi, che sono alberi di ricerca bilanciati appositamente progettati per essere memorizzati sui dischi. Poiché questi dischi sono molto più lenti della memoria ad accesso casuale (RAM), le prestazioni dei B-alberi vengono misurate non soltanto in base al tempo computazionale richiesto per eseguire le operazioni sugli insiemi dinamici, ma anche in base al numero di accessi al disco. Per ogni operazione con un B-albero, il numero di accessi al disco aumenta con l'altezza del B-albero, che è mantenuta bassa dalle operazioni del B-albero.

Il Capitolo 19 descrive l'implementazione di un heap riunibile, che supporta le operazioni INSERT, MINIMUM, EXTRACT-MIN e UNION.[1] L'operazione UNION unisce due heap. Gli heap di Fibonacci – descritti nel Capitolo 19 – supportano anche le operazioni DELETE e DECREASE-KEY. Utilizziamo i limiti sui tempi ammortizzati per misurare le prestazioni degli heap di Fibonacci. Le operazioni INSERT, MINIMUM e UNION richiedono un tempo effettivo e ammortizzato pari a $O(1)$ con gli heap di Fibonacci; le operazioni EXTRACT-MIN e DELETE richiedono un tempo ammortizzato pari a $O(\lg n)$. Il vantaggio più significativo degli heap di Fibonacci, tuttavia, è che DECREASE-KEY richiede un tempo ammortizzato pari a $O(1)$. Il piccolo tempo ammortizzato dell'operazione DECREASE-KEY spiega perché gli heap di Fibonacci sono i componenti chiave di alcuni fra gli algoritmi asintoticamente più veloci per i problemi dei grafi.

Osservando che è possibile battere il limite inferiore $\Omega(n \lg n)$ nelle operazioni di ordinamento quando le chiavi sono numeri interi in un intervallo ristretto, il Capitolo 20 chiede se sia possibile progettare una struttura dati che supporta le

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[1] Come detto nel Problema 10-2, poiché un heap riunibile supporta le operazioni MINIMUM e EXTRACT-MIN, possiamo fare riferimento a questo heap anche con il termine ***min-heap riunibile***. D'altra parte, se un heap riunibile supportasse le operazioni MAXIMUM ed EXTRACT-MAX, potrebbe essere chiamato ***max-heap riunibile***. A meno che non sia diversamente specificato, gli heap riunibili dovranno essere considerati min-heap riunibili.

operazioni sugli insiemi dinamici SEARCH, INSERT, DELETE, MINIMUM, MAXIMUM, SUCCESSOR e PREDECESSOR nel tempo $o(\lg n)$ quando le chiavi sono numeri interi in un intervallo ristretto. La risposta è che ciò è possibile se si utilizza una struttura dati ricorsiva nota come albero di van Emde Boas. Se le chiavi sono numeri interi unici appartenenti all'insieme $\{0, 1, 2, \ldots, u-1\}$, dove u è una potenza esatta di 2, allora gli alberi di van Emde Boas supportano le predette operazioni nel tempo $O(\lg \lg u)$.

Infine, il Capitolo 21 presenta le strutture dati per gli insiemi disgiunti. Abbiamo un universo di n elementi che sono raggruppati in insiemi dinamici. Inizialmente, ciascun elemento costituisce un insieme di un solo elemento (singoletto o singleton). L'operazione UNION unisce due insiemi e l'interrogazione FIND-SET identifica l'insieme in cui si trova un particolare elemento in quel momento. Rappresentando ciascun insieme con un semplice albero radicato, otteniamo delle operazioni sorprendentemente veloci: una sequenza di m operazioni viene eseguita nel tempo $O(m\,\alpha(n))$, dove $\alpha(n)$ è una funzione crescente incredibilmente lenta – $\alpha(n)$ è al più 4 in qualsiasi applicazione immaginabile. L'analisi ammortizzata che dimostra questo limite di tempo è tanto complessa quanto è semplice la struttura dati.

Gli argomenti trattati in questa parte non sono gli unici esempi di strutture dati "avanzate". Fra le altre strutture avanzate figurano:

- Gli ***alberi dinamici***, ideati da Sleator e Tarjan [320] e descritti da Tarjan [331], mantengono una foresta di alberi radicati disgiunti. A ogni arco di ciascun albero è associato un numero reale come costo. Gli alberi dinamici supportano le interrogazioni per trovare i padri, le radici e i costi degli archi e il costo minimo degli archi in un cammino da un nodo fino a una radice. Gli alberi possono essere manipolati tagliando degli archi, aggiornando tutti i costi degli archi in un cammino da un nodo a una radice, appendendo una radice a un altro albero e rendendo un nodo radice dell'albero in cui si trova. Una implementazione degli alberi dinamici fornisce un limite $O(\lg n)$ per il tempo ammortizzato di ogni operazione; una implementazione più complessa fornisce un limite di tempo $O(\lg n)$ nel caso peggiore. Gli alberi dinamici sono utilizzati in alcuni degli algoritmi asintoticamente più veloci per le reti di flusso.
- Gli ***alberi splay***, ideati da Sleator e Tarjan [321] e descritti da Tarjan [331], sono forme di alberi binari di ricerca nei quali le operazioni di ricerca standard sono eseguite nel tempo ammortizzato $O(\lg n)$. L'uso degli alberi splay semplifica gli alberi dinamici.
- Le strutture dati ***persistenti*** consentono di eseguire delle interrogazioni, e a volte anche degli aggiornamenti, sulle precedenti versioni di una struttura dati. Driscoll, Sarnak, Sleator e Tarjan [98] hanno descritto alcune tecniche che permettono con un piccolo costo di tempo e spazio di rendere persistenti le strutture dati concatenate. Il Problema 13-1 presenta un semplice esempio di insieme dinamico persistente.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

- Come vedremo nel Capitolo 20, molte strutture dati consentono un'implementazione più rapida delle operazioni di dizionario (INSERT, DELETE e SEARCH) per un universo limitato di chiavi. Sfruttando queste limitazioni, tali strutture sono in grado di ottenere tempi di esecuzione asintotici nel caso peggiore più rapidi rispetto alle strutture dati basate sui confronti. Fredman e Willard hanno sviluppato l'***albero di fusione*** [116], che è stata la prima struttura dati a per-

mettere di realizzare operazioni di dizionario più rapide quando l'universo è limitato agli interi. Hanno spiegato come implementare queste operazioni nel tempo $O(\lg n / \lg \lg n)$. Molte altre strutture dati successive, inclusi gli ***alberi di ricerca esponenziali*** [16], hanno migliorato i limiti per alcune o tutte le operazioni di dizionario e sono citate nelle note in fondo ai capitoli di questo libro.

- Le strutture dati per i ***grafi dinamici*** supportano varie interrogazioni, consentendo anche di modificare la struttura di un grafo tramite le operazioni che inseriscono o cancellano vertici o archi. Fra gli esempi delle interrogazioni che sono supportate figurano la connettività dei vertici [167], la connettività degli archi, gli alberi di connessione minimi [166], la biconnettività e la chiusura transitiva [165].

Le note alla fine dei capitoli di questo libro includono riferimenti ad altre strutture dati.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

18 B-alberi

18.1 Introduzione

I B-alberi sono alberi di ricerca bilanciati progettati per operare bene sui dischi o altre unità di memoria secondaria ad accesso diretto. I B-alberi sono simili agli alberi rosso-neri (Capitolo 13), ma sono più efficienti nel minimizzare le operazioni di input/output sul disco. Molti sistemi di basi di dati usano i B-alberi, o loro varianti, per memorizzare le informazioni.

I B-alberi differiscono dagli alberi rosso-neri per il fatto che i nodi di un B-albero possono avere molti figli, da poche unità a diverse centinaia. Ovvero, il "grado di ramificazione" di un B-albero può essere molto grande, ed è di solito determinato dalle caratteristiche dell'unità a disco utilizzata. I B-alberi sono simili agli alberi rosso-neri per il fatto che ogni B-albero di n nodi ha altezza $O(\lg n)$, ma l'altezza di un B-albero può essere considerevolmente più piccola di quella di un albero rosso-nero, considerando che il suo grado di ramificazione, e quindi la base del logaritmo che esprime la sua altezza, può essere molto più grande. Quindi, anche i B-alberi possono essere utilizzati per implementare molte operazioni sugli insiemi dinamici nel tempo $O(\lg n)$.

I B-alberi sono una generalizzazione naturale degli alberi binari di ricerca. La Figura 18.1 illustra un semplice B-albero. Se un nodo interno x di un B-albero contiene $x.n$ chiavi, allora x ha $x.n + 1$ figli. Le chiavi nel nodo x sono utilizzate come punti di separazione che suddividono l'intervallo delle chiavi gestite da x in $x.n + 1$ sottointervalli, ciascuno gestito da un figlio di x. Quando cerchiamo una chiave in un B-albero, prendiamo una decisione fra $x.n + 1$ possibili scelte, sulla base dei confronti con le $x.n$ chiavi memorizzate nel nodo x. La struttura dei nodi foglia è diversa da quella dei nodi interni; esamineremo queste differenze nel Paragrafo 18.2.

Il Paragrafo 18.2 dà una definizione precisa di B-albero e dimostra che l'altezza di un B-albero cresce soltanto logaritmicamente con il numero di nodi che contiene. Il Paragrafo 18.3 descrive come ricercare una chiave e inserire una chiave in un B-albero; il Paragrafo 18.4 descrive la cancellazione. Prima di continuare, bisogna capire perché le strutture dati progettate per operare in un disco debbano essere valutate differentemente da quelle progettate per operare nella memoria principale ad accesso casuale.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-9 Copyright © 2022 McGraw-Hill Education (Italy)

Strutture dati nella memoria secondaria

Esistono varie tecnologie che consentono di dare una capacità di memoria ai calcolatori. La ***memoria primaria*** (o ***memoria principale***) di un calcolatore normalmente è formata da chip di memoria al silicio. Questa tecnologia, tipicamente,

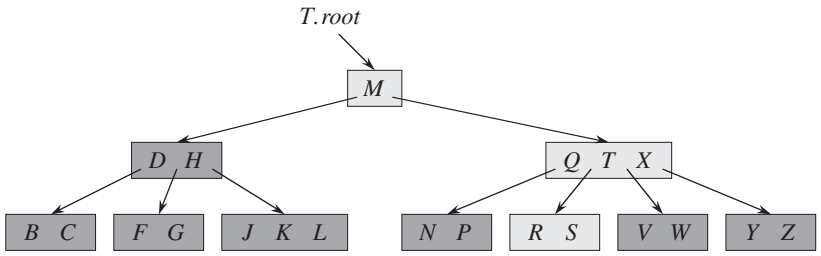


Figura 18.1 Un B-albero le cui chiavi sono le consonanti dell'alfabeto. Un nodo interno x contenente $x.n$ chiavi ha $x.n + 1$ figli. Tutte le foglie si trovano alla stessa profondità nell'albero. I nodi su sfondo chiaro vengono esaminati durante una ricerca della lettera R.

è per più di un ordine di grandezza più costosa per unità di memoria della tecnologia basata sui supporti magnetici, come i nastri o i dischi. La maggior parte dei calcolatori hanno anche una ***memoria secondaria*** che è realizzata con dischi magnetici; la quantità di questa memoria secondaria spesso supera la quantità di memoria primaria di almeno due ordini di grandezza.

La Figura 18.2 illustra una tipica unità a disco. L'unità è composta da uno o più ***piatti***, che ruotano a velocità costante attorno a un ***asse*** comune. La superficie di ogni piatto è ricoperta con un materiale magnetizzabile. Ogni piatto è letto o scritto da una ***testina*** posta all'estremità di un ***braccio***. I bracci possono spostare le loro testine avvicinandosi o allontanandosi dall'asse di rotazione. Se una testina è ferma, la superficie che passa sotto di essa è detta ***traccia***. Più piatti aumentano soltanto la capacità del disco, non le sue prestazioni.

Sebbene i dischi siano più economici e abbiano capacità più grandi della memoria principale, tuttavia sono molto più lenti, perché hanno delle parti meccaniche in movimento.[1] Ci sono due componenti nel movimento meccanico: la rotazione del piatto e lo spostamento del braccio. Attualmente, i dischi che si trovano in commercio hanno velocità di rotazione di 5400–15000 giri al minuto (RPM: Revolutions Per Minute); la velocità di rotazione più comune è 7200 RPM. Sebbene 7200 RPM possa sembrare una velocità alta, tuttavia una rotazione completa dura 8,33 millisecondi; questo tempo è quasi 5 ordini di grandezza più lungo dei tempi di accesso di 100 nanosecondi che troviamo nelle memorie al silicio. In altre parole, se dobbiamo aspettare che venga completata un'intera rotazione prima che un particolare elemento si trovi sotto la testina di lettura/scrittura, potremmo accedere alla memoria principale quasi 100 000 volte durante questo intervallo di tempo! In media dobbiamo aspettare soltanto per metà rotazione, ma la differenza fra i tempi di accesso della memoria al silicio e del disco resta enorme. Anche il movimento dei bracci richiede qualche tempo. Attualmente, i tempi di accesso medio dei dischi in commercio sono compresi tra 8 e 11 millisecondi.

Per ammortizzare il tempo impiegato ad aspettare i movimenti meccanici, i dischi non accedono a un solo elemento alla volta, ma a più elementi contemporaneamente. Le informazioni vengono suddivise in un certo numero di ***pagine*** della stessa dimensione, che sono consecutive nelle tracce, e ogni operazione di lettura o scrittura sul disco è di una o più pagine intere. Per un tipico disco, una pagina può avere una lunghezza da 2^{11} a 2^{14} byte. Una volta che la testina di lettura/scrittura è posizionata correttamente e il disco ha ruotato raggiungendo l'inizio della pagina appropriata, la lettura o la scrittura di un disco magnetico è intera-

Acquistato da Michele Mi[illegible] il 2022-07-17 22:12:12 tramite Ordine Direct n. 2951016 261027 - © da vinci 2021 McGraw-Hill Education (Italy)

[1] Recentemente sono state immesse sul mercato unità di memoria allo stato solido. Sebbene siano più veloci dei dischi meccanici, le unità di memoria allo stato solido hanno un costo per gigabyte più alto e capacità più piccole rispetto ai dischi meccanici.

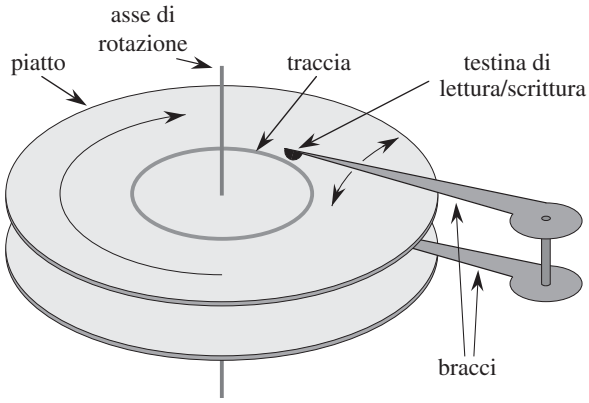


Figura 18.2 Una tipica unità a disco. È formata da uno o più piatti che ruotano attorno a un asse (qui sono mostrati due piatti). Ogni piatto viene letto o scritto con una testina che è posta all'estremità di un braccio. I bracci ruotano assieme attorno a un perno comune. Una traccia è la superficie che passa sotto la testina di lettura/scrittura quando questa è ferma.

mente elettronica (a parte la rotazione del disco) e questo consente di leggere o scrivere rapidamente grandi quantità di dati.

Di solito, il tempo per accedere a una pagina di informazioni su un disco e leggerla è più lungo del tempo che impiega il computer per esaminare tutte le informazioni lette. Per questo motivo, in questo capitolo esamineremo separatamente i due componenti principali del tempo di esecuzione:

- il numero di accessi al disco
- il tempo (di elaborazione) della CPU

Il numero di accessi al disco è misurato dal numero di pagine di informazioni che devono essere lette o scritte sul disco. È importante notare che il tempo di accesso al disco non è costante – dipende dalla distanza fra la traccia corrente e la traccia desiderata e anche dallo stato rotazionale iniziale del disco. Ciononostante, utilizzeremo il numero di pagine lette o scritte come un'approssimazione di primo grado del tempo totale impiegato per accedere al disco.

In una tipica applicazione dei B-alberi, la quantità di dati elaborati è talmente grande che tutti i dati non possono stare contemporaneamente nella memoria principale. Gli algoritmi dei B-alberi copiano le pagine selezionate dal disco nella memoria principale, quando occorre, e riscrivono sul disco le pagine che sono state modificate. Gli algoritmi dei B-alberi sono progettati in modo che, in qualsiasi istante, nella memoria principale sia presente soltanto un numero costante di pagine; quindi, la dimensione della memoria principale non limita la dimensione dei B-alberi che possono essere gestiti.

Nel nostro pseudocodice abbiamo modellato le operazioni su disco nel modo seguente. Sia x un puntatore a un oggetto. Se l'oggetto si trova correntemente nella memoria principale del calcolatore, allora possiamo fare riferimento agli attributi dell'oggetto nel modo consueto: $x.key$, per esempio. Se invece l'oggetto identificato da x si trova sul disco, allora dobbiamo eseguire l'operazione DISK-READ(x) per copiare l'oggetto x nella memoria principale, prima di potere fare riferimento ai suoi attributi (si suppone che, se x è già nella memoria principale, allora DISK-READ(x) non richiede alcun accesso al disco; si tratta di una "istruzione nulla"). Analogamente, l'operazione DISK-WRITE(x) è utilizzata per salvare su disco le modifiche che sono state fatte negli attributi dell'oggetto x. Il tipico modello per operare con un oggetto è il seguente:

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0. Copyright © 2022, McGraw-Hill Education (Italy).

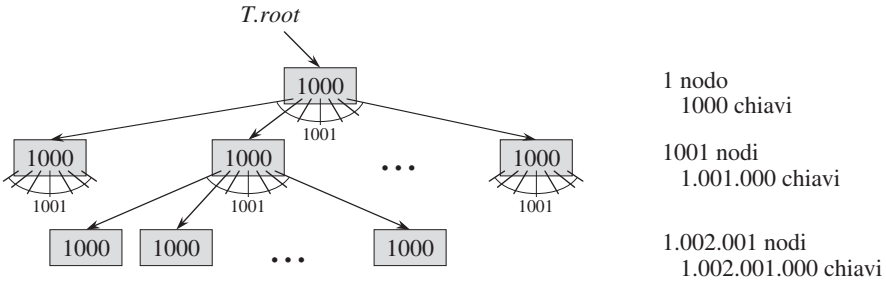


Figura 18.3 Un B-albero di altezza 2 che contiene oltre un miliardo di chiavi. All'interno di ogni nodo x è indicato il valore di $x.n$, il numero di chiavi in x. Ciascun nodo interno e ciascuna foglia contiene 1000 chiavi. In questo B-albero ci sono 1001 nodi alla profondità 1 e oltre un milione di foglie alla profondità 2.

x = un puntatore a qualche oggetto
DISK-READ(x)
operazioni che accedono e/o modificano gli attributi di x
DISK-WRITE(x) // Omesso se nessun attributo di x è stato modificato.
Altre operazioni che accedono agli attributi di x, senza modificarli.

In ogni istante il sistema può mantenere soltanto un numero limitato di pagine nella memoria principale. Si suppone che il sistema elimini dalla memoria principale le pagine che non sono più utilizzate; i nostri algoritmi per i B-alberi ignoreranno questo problema.

Poiché in molti sistemi il tempo di esecuzione di un algoritmo per un B-albero è determinato principalmente dal numero di operazioni DISK-READ e DISK-WRITE che sono eseguite, è logico utilizzare queste operazioni con efficienza facendo in modo che leggano o scrivano il maggior numero di informazioni possibile. Quindi, un nodo di un B-albero di solito è grande quanto un'intera pagina del disco. Il numero di figli che può avere un nodo di un B-albero è pertanto limitato dalla dimensione di una pagina del disco.

Per un grande B-albero memorizzato su un disco, si usano gradi di ramificazione tra 50 e 2000, a seconda della dimensione di una chiave rispetto alla dimensione di una pagina. Un grado di ramificazione elevato riduce considerevolmente sia l'altezza dell'albero sia il numero di accessi al disco richiesti per trovare una chiave qualsiasi. La Figura 18.3 illustra un B-albero con un grado di ramificazione 1001 e altezza 2 che può memorizzare oltre un miliardo di chiavi; ciononostante, poiché il nodo radice può essere tenuto permanentemente nella memoria principale, sono richiesti al più soltanto *due* accessi al disco per trovare una chiave qualsiasi in questo albero!

18.2 Definizione dei B-alberi

Per semplificare le cose, supponiamo, come abbiamo fatto con gli alberi binari di ricerca e gli alberi rosso-neri, che tutte le "informazioni satelliti" associate a una chiave siano memorizzate nello stesso nodo della chiave. In pratica, potremmo memorizzare assieme a ciascuna chiave un puntatore a un'altra pagina del disco che contiene le informazioni satelliti di quella chiave. Lo pseudocodice in questo capitolo suppone implicitamente che le informazioni satelliti associate a una chiave, o il puntatore a tali informazioni satelliti, viaggino con la chiave quando questa si sposta da un nodo all'altro.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Una tipica variante del B-albero, detta ***B*$^+$*-albero***, memorizza tutte le informazioni satelliti nelle foglie, mentre le chiavi e i puntatori ai figli sono memorizzati nei nodi interni, massimizzando così il grado di ramificazione dei nodi interni.

Un ***B-albero*** T è un albero radicato (la cui radice è $T.root$) che ha le seguenti proprietà:

1. Ogni nodo x ha i seguenti attributi:
   a. $x.n$, il numero di chiavi correntemente memorizzate nel nodo x.
   b. Le $x.n$ chiavi stesse, $x.key_1, x.key_2, \ldots, x.key_{x.n}$, memorizzate in ordine non decrescente, in modo che $x.key_1 \leq x.key_2 \leq \cdots \leq x.key_{x.n}$.
   c. $x.leaf$, un valore booleano che è TRUE se x è una foglia e FALSE se x è un nodo interno.
2. Ogni nodo interno x contiene anche $x.n+1$ puntatori $x.c_1, x.c_2, \ldots, x.c_{x.n+1}$ ai suoi figli. I nodi foglia non hanno figli, quindi i loro attributi c_i non sono definiti.
3. Le chiavi $x.key_i$ separano gli intervalli delle chiavi memorizzate in ciascun sottoalbero: se k_i è una chiave qualsiasi memorizzata nel sottoalbero con radice $c_i[x]$, allora

   $$k_1 \leq x.key_1 \leq k_2 \leq x.key_2 \leq \cdots \leq x.key_{x.n} \leq k_{x.n+1}$$

4. Tutte le foglie hanno la stessa profondità, che è l'altezza h dell'albero.
5. Ci sono limiti superiori e inferiori per il numero di chiavi che un nodo può contenere. Questi limiti possono essere espressi in termini di un intero $t \geq 2$ chiamato ***grado minimo*** del B-albero:
   a. Ogni nodo, tranne la radice, deve avere almeno $t - 1$ chiavi. Ogni nodo interno, tranne la radice, quindi ha almeno t figli. Se l'albero non è vuoto, la radice deve avere almeno una chiave.
   b. Ogni nodo può contenere al massimo $2t - 1$ chiavi. Quindi, un nodo interno può avere al massimo $2t$ figli. Diciamo che un nodo è ***pieno*** se contiene esattamente $2t - 1$ chiavi.[2]

Il B-albero più semplice si ha quando $t = 2$. Ogni nodo interno ha 2, 3 o 4 figli, e si ha un ***albero 2-3-4***. Di solito, però, vengono utilizzati valori di t molto più grandi.

Altezza di un B-albero

Il numero di accessi al disco richiesti dalla maggior parte delle operazioni su un B-albero è proporzionale all'altezza del B-albero. Analizziamo ora l'altezza nel caso peggiore di un B-albero.

Teorema 18.1
Se $n \geq 1$, allora per qualsiasi B-albero T di n chiavi, con altezza h e grado minimo $t \geq 2$, si ha

Acquistato da Michele Michele su Webster il 2022-07-27 23:57 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$h \leq \log_t \frac{n+1}{2}$$

[2] Un'altra tipica variante del B-albero, detta ***B*-albero***, richiede che ogni nodo interno sia pieno almeno per $2/3$, anziché per metà, come richiede un B-albero.

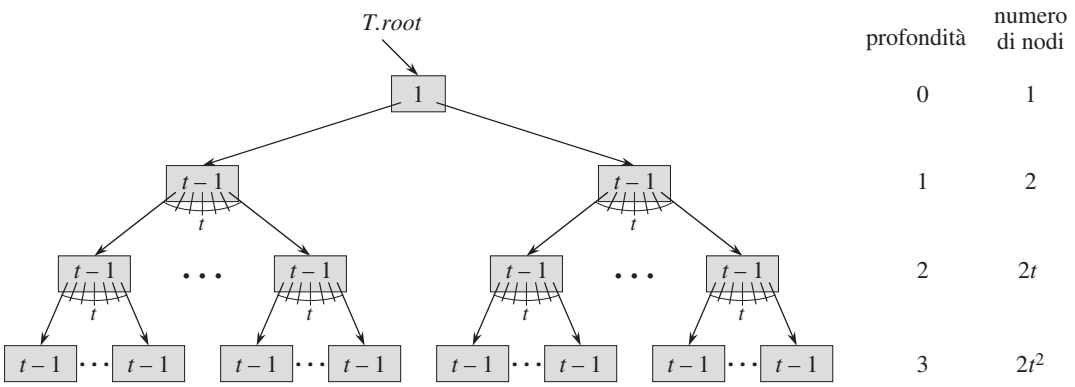


Figura 18.4 Un B-albero di altezza 3 che contiene il minor numero possibile di chiavi. All'interno di ogni nodo x è indicato il valore $x.n$.

Dimostrazione La radice di un B-albero contiene almeno una chiave e tutti gli altri nodi contengono almeno $t - 1$ chiavi. Quindi T, che ha altezza h, ha almeno 2 nodi alla profondità 1, almeno $2t$ nodi alla profondità 2, almeno $2t^2$ nodi alla profondità 3, e così via, fino alla profondità h, dove ci sono almeno $2t^{h-1}$ nodi. La Figura 18.4 illustra tale albero per $h = 3$. Quindi, il numero n di chiavi soddisfa la seguente relazione

$$\begin{aligned} n &\geq 1 + (t-1)\sum_{i=1}^{h} 2t^{i-1} \\ &= 1 + 2(t-1)\left(\frac{t^h - 1}{t-1}\right) \\ &= 2t^h - 1 \end{aligned}$$

Dalla quale è facile ottenere che $t^h \leq (n+1)/2$. Prendendo i logaritmi in base t da entrambi i lati, si ottiene ciò che si voleva dimostrare. ■

Qui si vede la potenza dei B-alberi rispetto agli alberi rosso-neri. Sebbene l'altezza dell'albero cresca come $O(\lg n)$ in entrambi i casi (ricordiamo che t è una costante), tuttavia per i B-alberi la base del logaritmo può essere molte volte più grande. Quindi, rispetto agli alberi rosso-neri, i B-alberi consentono un risparmio di un fattore di circa $\lg t$ nel numero di nodi esaminati nella maggior parte delle operazioni sugli alberi. Poiché l'esame di un nodo arbitrario di un albero, di solito, richiede un accesso al disco, il numero di accessi al disco viene significativamente ridotto.

Esercizi

Acquistato da Michele Miceli su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

18.2-1

Perché non è consentito avere un grado minimo $t = 1$?

18.2-2

Per quali valori di t l'albero della Figura 18.1 è un valido B-albero?

18.2-3

Trovate tutti i B-alberi validi di grado minimo 2 che rappresentano $\{1, 2, 3, 4, 5\}$.

18.2-4
Qual è il numero massimo di chiavi che possono essere memorizzate in un B-albero di altezza h? Esprimete questo numero in funzione del grado minimo t.

18.2-5
Descrivete la struttura dati che si otterrebbe se ogni nodo nero in un albero rosso-nero dovesse assorbire i suoi figli rossi, incorporando come propri anche i figli dei nodi rossi.

18.3 Operazioni fondamentali con i B-alberi

Questo paragrafo descrive dettagliatamente le operazioni B-TREE-SEARCH, B-TREE-CREATE e B-TREE-INSERT. In queste procedure adottiamo due convenzioni:

- La radice del B-albero si trova sempre nella memoria principale, quindi non è mai necessario eseguire l'operazione DISK-READ sulla radice; occorre, invece, eseguire un'operazione DISK-WRITE sulla radice, ogni volta che la radice viene modificata.
- Prima di passare un nodo come parametro a una procedura, occorre eseguire un'operazione DISK-READ su tale nodo.

Tutte le procedure che presentiamo sono algoritmi a "singolo passaggio", che procedono verso il basso partendo dalla radice dell'albero, senza tornare indietro.

Ricerca in un B-albero

La ricerca in un B-albero è molto simile alla ricerca in un albero binario di ricerca, con la differenza che anziché scegliere tra due alternative in ogni nodo, bisogna scegliere fra più alternative a seconda del numero di figli che ha ciascun nodo. Più precisamente, in ogni nodo interno x, possiamo scegliere fra $x.n + 1$ alternative possibili.

La procedura B-TREE-SEARCH è una semplice generalizzazione della procedura TREE-SEARCH definita per gli alberi binari di ricerca. La procedura B-TREE-SEARCH riceve come input un puntatore al nodo radice x di un sottoalbero e una chiave k da cercare nel sottoalbero. La chiamata di livello più alto ha quindi la forma B-TREE-SEARCH$(T.root, k)$. Se k si trova nel B-albero, la procedura B-TREE-SEARCH restituisce la coppia ordinata (y, i) che è formata da un nodo y e da un indice i, tali che $y.key_i = k$, altrimenti restituisce il valore NIL.

```
B-TREE-SEARCH(x, k)
1  i = 1
2  while i ≤ x.n and k > x.key_i
3       i = i + 1
4  if i ≤ x.n and k == x.key_i
5       return (x, i)
6  elseif x.leaf
7       return NIL
8  else DISK-READ(x.c_i)
9       return B-TREE-SEARCH(x.c_i, k)
```

Acquistato da Michele Michele su Webster il 2022-07-07 13:12 - Numero ordine Libreria 1409503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Utilizzando una procedura di ricerca lineare, le righe 1–3 trovano il più piccolo indice i tale che $k \leq x.key_i$, altrimenti assegnano $x.n + 1$ a i. Le righe 4–5 controllano se è stata trovata la chiave; se la ricerca ha successo, restituiscono la chiave. Le righe 6–9 terminano la ricerca senza successo (se x è una foglia) oppure effettuano ricorsivamente la ricerca nel sottoalbero appropriato di x, dopo avere eseguito l'operazione obbligatoria DISK-READ su tale nodo figlio.

La precedente Figura 18.1 illustra il funzionamento di B-TREE-SEARCH; i nodi su sfondo chiaro vengono esaminati durante una ricerca della chiave R.

Come nella procedura TREE-SEARCH per gli alberi binari di ricerca, i nodi incontrati durante la ricorsione formano un cammino semplice in discesa a partire dalla radice dell'albero. Il numero di pagine del disco cui accede la procedura B-TREE-SEARCH è quindi $O(h) = O(\log_t n)$, dove h è l'altezza del B-albero ed n è il numero di chiavi nel B-albero. Poiché $x.n < 2t$, il tempo impiegato dal ciclo **while** (righe 2–3) all'interno di ogni nodo è $O(t)$ e il tempo totale della CPU è $O(th) = O(t \log_t n)$.

Creare un B-albero vuoto

Per costruire un B-albero T, prima utilizziamo B-TREE-CREATE per creare una radice vuota e poi chiamiamo B-TREE-INSERT per aggiungere nuove chiavi. Entrambe queste procedure usano una procedura ausiliaria ALLOCATE-NODE, che alloca una pagina del disco da utilizzare come nuovo nodo nel tempo $O(1)$. Supponiamo che un nodo creato da ALLOCATE-NODE non richieda l'operazione DISK-READ, in quanto non sono state ancora memorizzate informazioni utili sul disco per questo nodo.

```
B-TREE-CREATE(T)
1  x = ALLOCATE-NODE()
2  x.leaf = TRUE
3  x.n = 0
4  DISK-WRITE(x)
5  T.root = x
```

La procedura B-TREE-CREATE richiede $O(1)$ operazioni su disco e un tempo di CPU pari a $O(1)$.

Inserire una chiave in un B-albero

L'inserimento di una chiave in un B-albero è un'operazione molto più complicata dell'inserimento di una chiave in un albero binario di ricerca. Come abbiamo fatto con gli alberi binari di ricerca, cerchiamo la posizione della foglia in cui inserire la nuova chiave. Con un B-albero, tuttavia, non possiamo semplicemente creare un nuovo nodo foglia e inserire la nuova chiave, perché l'albero risultante non sarebbe più un valido B-albero. Inseriamo, invece, la nuova chiave in un nodo foglia esistente. Poiché non possiamo inserire una chiave in un nodo foglia che è pieno, introduciamo un'operazione che ***divide*** un nodo pieno y (che ha $2t - 1$ chiavi) in corrispondenza della sua ***chiave mediana*** $y.key_t$ in due nodi che hanno $t - 1$ chiavi ciascuno. La chiave mediana si sposta in alto nel nodo padre di y per identificare il punto di divisione fra i due nuovi alberi. Tuttavia, se anche il padre di y è pieno, bisogna dividere questo nodo prima che la nuova chiave possa essere inserita; questa necessità di dividere i nodi pieni può propagarsi verso l'alto per tutto l'albero.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

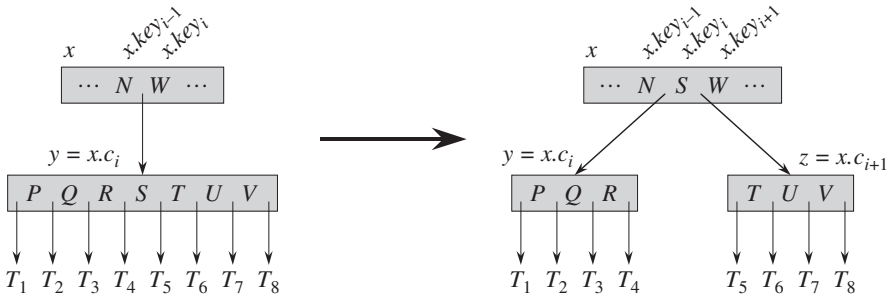


Figura 18.5 Divisione di un nodo con $t = 4$. Il nodo $y = x.c_i$ viene diviso in due nodi, y e z, e la chiave mediana S di y viene spostata in alto nel nodo padre di y.

Come in un albero binario di ricerca, possiamo inserire una chiave in un B-albero in un singolo passaggio, scendendo dalla radice fino a una foglia. Per farlo, non è necessario attendere di sapere se bisogna dividere un nodo pieno per effettuare l'inserimento. Invece, mentre attraversiamo l'albero cercando la posizione della nuova chiave, dividiamo ogni nodo pieno che incontriamo lungo il cammino (inclusa la stessa foglia). Quindi, ogni volta che vogliamo dividere un nodo pieno y, siamo sicuri che suo padre non è pieno.

Dividere un nodo in un B-albero

La procedura B-TREE-SPLIT-CHILD riceve come input un nodo interno x *non pieno* (si suppone che si trovi nella memoria principale), un indice i tale che $y = x.c_i$ (anche questo si suppone che si trovi nella memoria principale) sia un nodo figlio *pieno* di x. La procedura poi divide questo figlio in due e modifica x in modo che abbia un altro figlio (per dividere una radice piena, prima bisogna trasformare la radice in un figlio di un nuovo nodo radice vuoto, in modo da potere utilizzare la procedura B-TREE-SPLIT-CHILD. Quindi, l'altezza dell'albero cresce di uno; la divisione è l'unico modo in cui l'albero può crescere).

La Figura 18.5 illustra questo processo. Il nodo pieno $y = x.c_i$ viene diviso in prossimità della sua chiave mediana S, che si sposta in alto nel nodo x, che è il padre di y. Le chiavi in y che sono più grandi della chiave mediana vengono poste in un nuovo nodo z, che diventa un nuovo figlio di x.

La procedura B-TREE-SPLIT-CHILD esegue una semplice operazione "taglia e incolla". Nel caso in esame, y è l'i-esimo figlio di x ed è il nodo che viene diviso. Originariamente, il nodo y ha $2t$ figli ($2t - 1$ chiavi), ma questa operazione riduce il numero dei suoi figli a t ($t - 1$ chiavi). Il nodo z "adotta" i t figli maggiori di y ($t - 1$ chiavi). Il nodo z diventa un nuovo figlio di x, che si trova subito dopo y nella tabella dei figli di x. La chiave mediana di y si sposta in alto per diventare la chiave in x che separa y e z. Le righe 1–9 creano il nodo z e assegnano a questo nodo le $t - 1$ chiavi più grandi e i corrispondenti t figli di y. La riga 10 aggiorna il conteggio delle chiavi di y. Infine, le righe 11–17 inseriscono z come figlio di x, spostano la chiave mediana da y a x, in modo da separare y da z e aggiornano il conteggio delle chiavi di x. Le righe 18–20 scrivono sul disco tutte le pagine che sono state modificate. Il tempo della CPU utilizzato da B-TREE-SPLIT-CHILD è $\Theta(t)$, a causa dei cicli delle righe 5–6 e 8–9 (gli altri cicli eseguono $O(t)$ iterazioni). La procedura esegue $O(1)$ operazioni su disco.

Acquistato da Michele Michele su Webster il 2022-07-27 23:12 Numero Ordine Libreria 4005020162270707 0 Copyright © 2013 McGraw-Hill Education (Italy)

```
B-TREE-SPLIT-CHILD(x, i)
 1  z = ALLOCATE-NODE()
 2  y = x.c_i
 3  z.leaf = y.leaf
 4  z.n = t − 1
 5  for j = 1 to t − 1
 6       z.key_j = y.key_{j+t}
 7  if not y.leaf
 8       for j = 1 to t
 9            z.c_j = y.c_{j+t}
10  y.n = t − 1
11  for j = x.n + 1 downto i + 1
12       x.c_{j+1} = x.c_j
13  x.c_{i+1} = z
14  for j = x.n downto i
15       x.key_{j+1} = x.key_j
16  x.key_i = y.key_t
17  x.n = x.n + 1
18  DISK-WRITE(y)
19  DISK-WRITE(z)
20  DISK-WRITE(x)
```

Inserire una chiave in un B-albero con un singolo passaggio nell'albero
Inseriamo una chiave k in un B-albero T di altezza h con un singolo passaggio in discesa nell'albero, richiedendo $O(h)$ accessi al disco. Il tempo di CPU richiesto è $O(th) = O(t \log_t n)$. La procedura B-TREE-INSERT usa B-TREE-SPLIT-CHILD per garantire che la ricorsione non arrivi mai a un nodo pieno.

```
B-TREE-INSERT(T, k)
 1  r = T.root
 2  if r.n == 2t − 1
 3       s = ALLOCATE-NODE()
 4       T.root = s
 5       s.leaf = FALSE
 6       s.n = 0
 7       s.c_1 = r
 8       B-TREE-SPLIT-CHILD(s, 1)
 9       B-TREE-INSERT-NONFULL(s, k)
10  else B-TREE-INSERT-NONFULL(r, k)
```

Le righe 3–9 gestiscono il caso in cui il nodo radice r è pieno: la radice viene divisa e un nuovo nodo s (che ha due figli) diventa la radice. La divisione della radice è l'unico modo per fare crescere l'altezza di un B-albero. La Figura 18.6 illustra questo caso. Diversamente da un albero binario di ricerca, un B-albero cresce in altezza nella radice, non nelle foglie.

Acquistato da Michele Cuviello (cuvmic@libero.it) - 2022-07-12 - Numero Licenza 2245031048 - 27401 - Copyright © 2014, McGraw-Hill Education (Italy)

La procedura finisce chiamando B-TREE-INSERT-NONFULL per inserire la chiave k nell'albero che ha un nodo radice non pieno. La procedura B-TREE-INSERT-NONFULL effettua la ricorsione quante volte serve per discendere l'al-

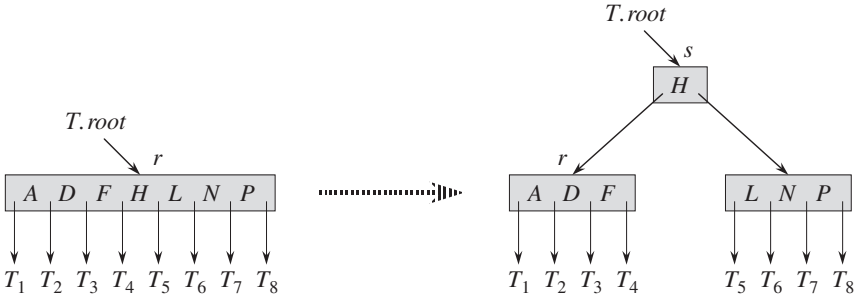


Figura 18.6 Divisione della radice con $t = 4$. Il nodo radice r viene diviso in due e viene creato un nuovo nodo radice s. La nuova radice contiene la chiave mediana di r e ha le due metà di r come figli. Dopo la divisione della radice, l'altezza del B-albero cresce di uno.

bero assicurandosi che il nodo in cui effettua la ricorsione non sia mai pieno, chiamando B-TREE-SPLIT-CHILD se necessario.

La procedura ricorsiva ausiliaria B-TREE-INSERT-NONFULL inserisce la chiave k nel nodo x, che si suppone non sia pieno quando viene chiamata la procedura. Il funzionamento di B-TREE-INSERT e il funzionamento ricorsivo di B-TREE-INSERT-NONFULL garantiscono che questa ipotesi sia vera.

```
B-TREE-INSERT-NONFULL(x, k)
 1  i = x.n
 2  if x.leaf
 3       while i ≥ 1 and k < x.key_i
 4            x.key_{i+1} = x.key_i
 5            i = i − 1
 6       x.key_{i+1} = k
 7       x.n = x.n + 1
 8       DISK-WRITE(x)
 9  else while i ≥ 1 and k < x.key_i
10            i = i − 1
11       i = i + 1
12       DISK-READ(x.c_i)
13       if x.c_i.n == 2t − 1
14            B-TREE-SPLIT-CHILD(x, i)
15            if k > x.key_i
16                 i = i + 1
17       B-TREE-INSERT-NONFULL(x.c_i, k)
```

La procedura B-TREE-INSERT-NONFULL funziona nel modo seguente. Le righe 3–8 gestiscono il caso in cui x sia un nodo foglia, inserendo la chiave k in x. Se x non è un nodo foglia, allora dobbiamo inserire k nel nodo foglia appropriato del sottoalbero con radice nel nodo interno x. In questo caso, le righe 9–11 determinano il figlio di x nel quale deve scendere la ricorsione. La riga 13 rileva se la ricorsione deve andare in un nodo figlio pieno, nel qual caso la riga 14 usa B-TREE-SPLIT-CHILD per dividere questo nodo in due figli non pieni e le righe 15–16 determinano quale di questi due figli è quello su cui scendere (notate che non è necessaria l'operazione DISK-READ$(x.c_i)$ dopo che la riga 16 incrementa i, in quanto la ricorsione raggiungerà in questo caso un nodo figlio che è stato appena creato da B-TREE-SPLIT-CHILD). L'effetto complessivo delle righe 13–16 è quindi garantire che la procedura non effettui la ricorsione in un nodo

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 19950306-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

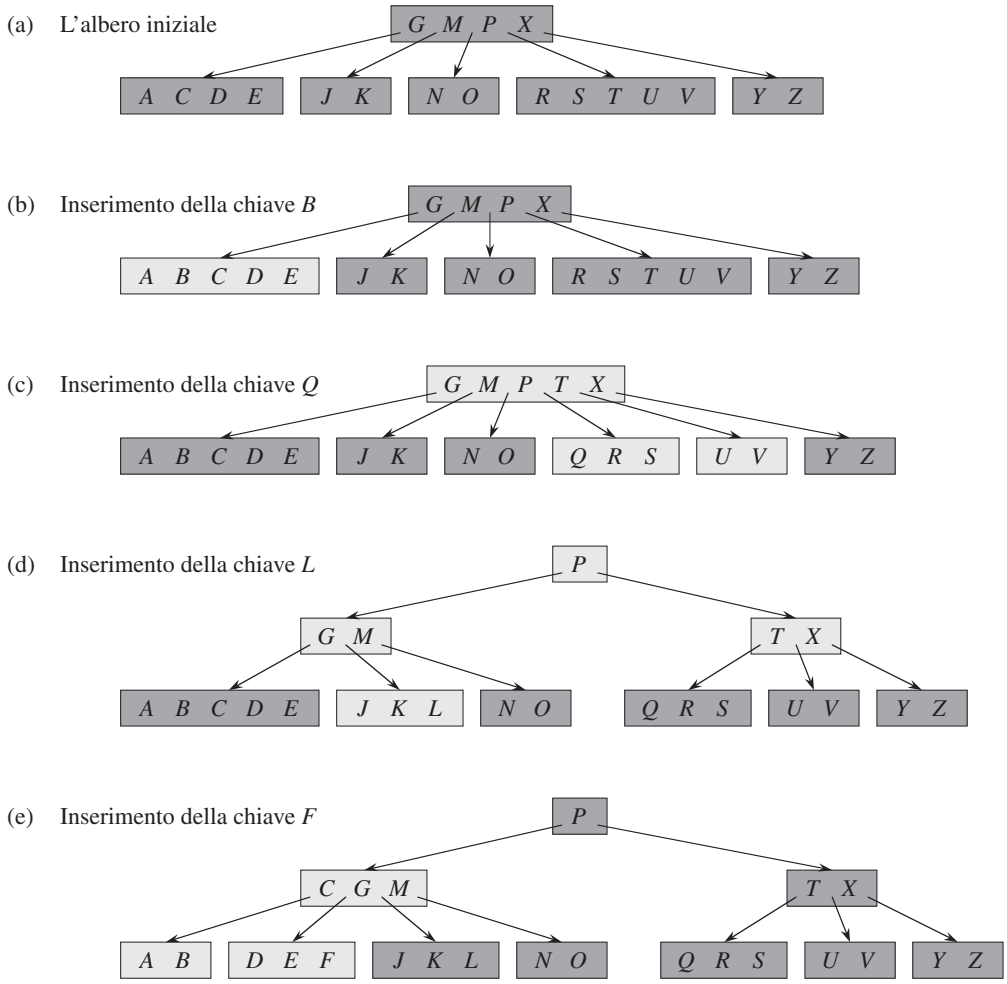


Figura 18.7 Inserire le chiavi in un B-albero. Il grado minimo t per questo B-albero è 3, quindi un nodo può contenere al massimo 5 chiavi. I nodi che sono modificati dal processo di inserimento hanno uno sfondo chiaro. **(a)** L'albero iniziale per questo esempio. **(b)** Il risultato dell'inserimento della chiave B nell'albero iniziale; questo è un semplice inserimento in un nodo foglia. **(c)** Il risultato dell'inserimento della chiave Q nell'albero precedente. Il nodo $RSTUV$ è diviso in due nodi che contengono RS e UV; la chiave T viene spostata in alto nella radice; la chiave Q viene inserita nella metà a sinistra (il nodo RS). **(d)** Il risultato dell'inserimento della chiave L nell'albero precedente. La radice viene immediatamente divisa, perché è piena, e l'altezza del B-albero cresce di uno. Poi viene inserita la chiave L nella foglia che contiene JK. **(e)** Il risultato dell'inserimento della chiave F nell'albero precedente. Il nodo $ABCDE$ viene diviso prima che la chiave F venga inserita nella metà a destra (il nodo DE).

pieno. La riga 17 effettua la ricorsione per inserire k nel sottoalbero appropriato. La Figura 18.7 illustra i vari casi di inserimento di una chiave in un B-albero.

Acquistato da Michele Michele su Webster il 2022-07-07 23:13. Numero Ordine Libreria 1955903016-220707-0.Copyright © 2022 McGraw-Hill Education (Italy)

Il numero di accessi al disco effettuati da B-TREE-INSERT è $O(h)$ per un B-albero di altezza h, in quanto sono eseguite soltanto $O(1)$ operazioni DISK-READ e DISK-WRITE tra due chiamate consecutive di B-TREE-INSERT-NONFULL. Il tempo totale della CPU è $O(th) = O(t \log_t n)$. Poiché la procedura B-TREE-INSERT-NONFULL è ricorsiva in coda, in alternativa potrebbe essere implementata come un ciclo **while**, dimostrando che il numero di pagine che devono essere sempre presenti nella memoria principale è $O(1)$.

Esercizi

18.3-1

Illustrate i risultati che si ottengono inserendo ordinatamente le seguenti chiavi

$F, S, Q, K, C, L, H, T, V, W, M, R, N, P, A, B, X, Y, D, Z, E$

in un B-albero vuoto con grado minimo 2. Disegnate soltanto le configurazioni dell'albero che si hanno appena prima di effettuare la divisione di qualche nodo; disegnate anche la configurazione finale.

18.3-2

Spiegate in quali circostanze, se pur ce ne sono, vengono eseguite delle operazioni ridondanti DISK-READ o DISK-WRITE durante l'esecuzione della procedura B-TREE-INSERT. (Un'operazione DISK-READ è ridondante se riguarda una pagina che è già presente in memoria. Un'operazione DISK-WRITE è ridondante se scrive sul disco un pagina che è uguale a quella già scritta sul disco.)

18.3-3

Spiegate come trovare la chiave minima memorizzata in un B-albero e il predecessore di una data chiave memorizzata in un B-albero.

18.3-4 ⋆

Supponete che le chiavi $\{1, 2, \ldots, n\}$ vengano inserite in un B-albero vuoto con grado minimo 2. Quanti nodi ha il B-albero finale?

18.3-5

Poiché i nodi foglia non richiedono puntatori ai figli, potrebbero utilizzare un valore t diverso (più grande) dai nodi interni per la stessa dimensione della pagina sul disco. Spiegate come modificare le procedure di creazione e inserimento di un B-albero per realizzare questa variante.

18.3-6

Supponete che la procedura B-TREE-SEARCH sia implementata utilizzando la ricerca binaria, anziché la ricerca lineare, all'interno di ogni nodo. Dimostrate che questa modifica fa sì che il tempo della CPU sia $O(\lg n)$, indipendentemente da come venga scelto t come una funzione di n.

18.3-7

Supponete che l'hardware del disco consenta di scegliere arbitrariamente la dimensione di una pagina del disco, ma che il tempo richiesto per leggere la pagina sia $a + bt$, dove a e b sono costanti prefissate e t sia il grado minimo di un B-albero che usa le pagine della dimensione scelta. Descrivete come scegliere t in modo da minimizzare (approssimativamente) il tempo di ricerca nel B-albero. Indicate un valore ottimo di t per il caso in cui $a = 5$ millisecondi e $b = 10$ microsecondi.

18.4 Cancellare una chiave da un B-albero

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

La cancellazione di una chiave da un B-albero è analoga all'inserimento, ma un po' più complicata, perché una chiave può essere cancellata da un nodo qualsiasi – non soltanto da una foglia – e l'eliminazione di una chiave da un nodo interno richiede che i figli del nodo vengano riorganizzati. Come nell'inserimento, dobbiamo stare attenti che la cancellazione non generi un albero la cui struttura viola le proprietà di un B-albero.

Esattamente come dovevamo garantire che un nodo non diventasse troppo grande a causa di un inserimento, così dobbiamo garantire che un nodo non diventi troppo piccolo durante la cancellazione (fa eccezione la radice che può avere meno chiavi del numero minimo consentito $t - 1$). Proprio come un semplice algoritmo di inserimento poteva avere bisogno di tornare indietro se incontrava un nodo pieno lungo il cammino semplice che portava al punto in cui doveva essere inserita la chiave, così una semplice procedura di cancellazione potrebbe avere bisogno di tornare indietro se un nodo (diverso dalla radice), lungo il cammino semplice che porta al punto in cui la chiave deve essere cancellata, ha il numero minimo di chiavi.

La procedura B-TREE-DELETE cancella la chiave k dal sottoalbero con radice in x. Questa procedura è strutturata per garantire che, ogni volta che viene chiamata ricorsivamente B-TREE-DELETE su un nodo x, il numero di chiavi in x sia almeno pari al grado minimo t. Notate che per soddisfare questa condizione occorre una chiave in più del minimo richiesto dalle usuali condizioni dei B-alberi, quindi a volte potrebbe essere necessario spostare una chiave in un nodo figlio, prima che la ricorsione raggiunga questo nodo. Questa ulteriore condizione ci permette di cancellare una chiave in un singolo passaggio in discesa dell'albero, senza dover "tornare indietro" (con una eccezione, di cui diremo più avanti). La seguente specifica per la cancellazione di una chiave da un B-albero deve essere interpretata con l'intesa che, qualora il nodo radice x diventasse un nodo interno senza chiavi (questa situazione potrebbe presentarsi nei casi 2c e 3b, descritti più avanti), il nodo x verrebbe cancellato e l'unico figlio di x, $x.c_1$, diventerebbe la nuova radice dell'albero. In questo modo, l'altezza dell'albero si riduce di uno e viene preservata la proprietà che la radice dell'albero contiene almeno una chiave (a meno che l'albero non sia vuoto).

Anziché presentare lo pseudocodice, descriveremo il funzionamento dell'operazione di cancellazione. La Figura 18.8 illustra i vari casi di cancellazione delle chiavi da un B-albero.

1. Se la chiave k si trova nel nodo x e x è una foglia, cancellare la chiave k da x.
2. Se la chiave k si trova nel nodo x e x è un nodo interno, eseguire le seguenti operazioni:
   a. Se il figlio y che precede k nel nodo x ha almeno t chiavi, trovare il predecessore k' di k nel sottoalbero con radice in y. Cancellare ricorsivamente k' e sostituire k con k' in x (le operazioni per trovare e cancellare k' possono essere eseguite in un singolo passaggio in discesa).
   b. Simmetricamente, se il figlio z che segue k nel nodo x ha almeno t chiavi, trovare il successore k' di k nel sottoalbero con radice in z. Cancellare ricorsivamente k' e sostituire k con k' in x (le operazioni per trovare e cancellare k' possono essere eseguite in un singolo passaggio in discesa).
   c. Altrimenti, se entrambi i nodi y e z hanno soltanto $t - 1$ chiavi, fondere k e tutto z nel nodo y, in modo che x perda sia k sia il puntatore a z, adesso y contiene $2t - 1$ chiavi. Poi, rilasciare z e cancellare ricorsivamente k da y.
3. Se la chiave k non è presente nel nodo interno x, determinare la radice $x.c_i$ del sottoalbero appropriato che deve contenere k, se k è davvero presente nell'albero. Se $x.c_i$ ha soltanto $t - 1$ chiavi, eseguire il passo 3a o 3b, se necessario, per avere la garanzia di arrivare a un nodo che contiene almeno t chiavi. Poi, finire effettuando la ricorsione sul figlio appropriato di x.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-230707-0 Copyright © 2022, McGraw-Hill Education (Italy)

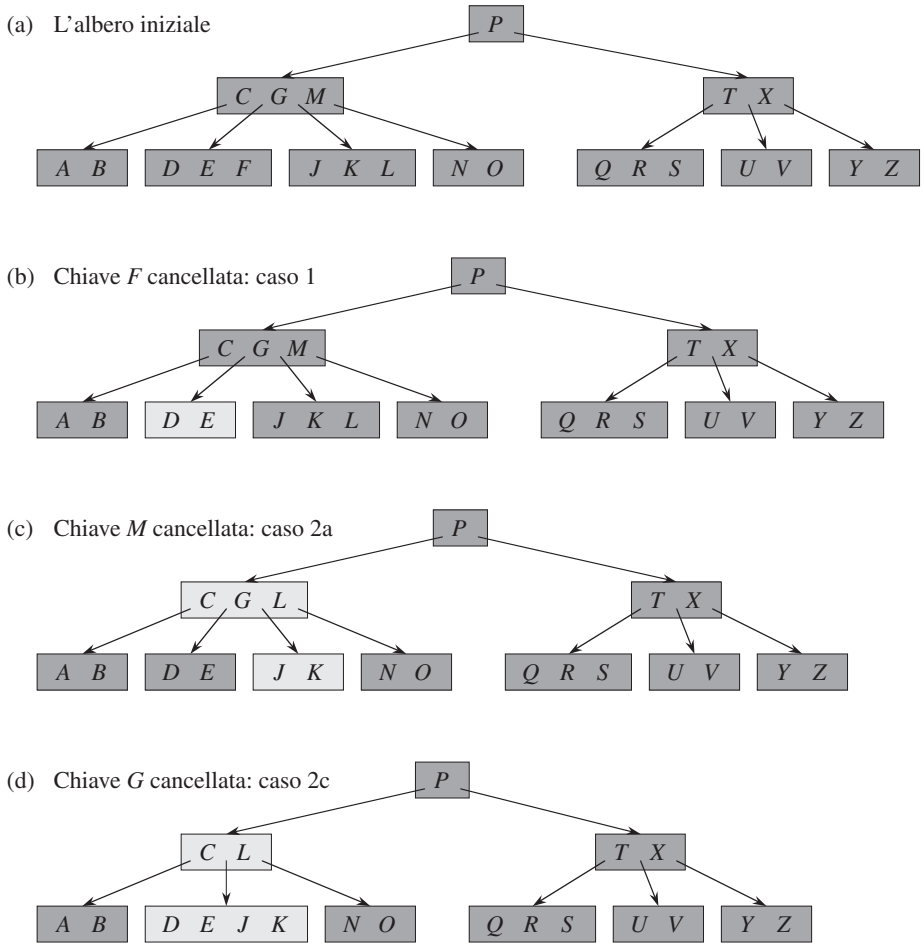


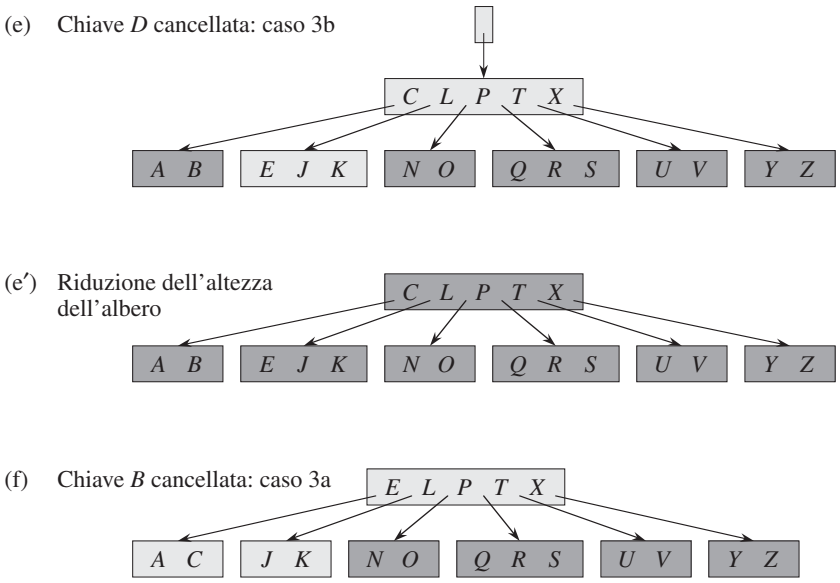
Figura 18.8 Cancellare le chiavi da un B-albero. Il grado minimo per questo B-albero è $t = 3$, quindi un nodo (diverso dalla radice) non può avere meno di 2 chiavi. I nodi che sono modificati hanno lo sfondo chiaro. **(a)** Il B-albero della Figura 18.7(e). **(b)** Cancellazione della chiave F. Questo è il caso 1: una semplice cancellazione da una foglia. **(c)** Cancellazione della chiave M. Questo è il caso 2a: il predecessore L di M viene spostato in alto per occupare la posizione di M. **(d)** Cancellazione della chiave G. Questo è il caso 2c: la chiave G viene spinta in basso per creare il nodo $DEGJK$, poi viene cancellata da questa foglia (caso 1). **(e)** Cancellazione della chiave D. Questo è il caso 3b: la ricorsione non può raggiungere il nodo CL perché ha soltanto 2 chiavi, quindi la chiave P viene spinta in basso e fusa con CL e TX per formare $CLPTX$; poi, la chiave D viene cancellata da una foglia (caso 1). **(e′)** Dopo (e), la radice viene cancellata e l'altezza dell'albero si riduce di uno. **(f)** Cancellazione della chiave B. Questo è il caso 3a: la chiave C viene spostata per occupare la posizione di B; la chiave E viene spostata per occupare la posizione di C.

a. Se $x.c_i$ ha soltanto $t - 1$ chiavi, ma ha un fratello adiacente con almeno t chiavi, assegnare a $x.c_i$ una chiave extra, spostando in basso una chiave da x in $x.c_i$, spostando in alto una chiave dal fratello sinistro o destro di $x.c_i$ in x e, infine, spostando in $x.c_i$ un puntatore al figlio del fratello (quello adiacente alla chiave spostata).
b. Se $x.c_i$ ed entrambi i fratelli adiacenti di $x.c_i$ hanno $t - 1$ chiavi, fondere $x.c_i$ con un fratello in un nuovo nodo; questo richiede che una chiave venga spostata in basso da x nel nuovo nodo per diventare la chiave mediana di questo nodo.

Poiché la maggior parte delle chiavi in un B-albero si trovano nelle foglie, possiamo prevedere che in pratica le operazioni di cancellazione siano molto spesso utilizzate per eliminare le chiavi dalle foglie. La procedura B-TREE-DELETE quindi opera effettuando un singolo passaggio in discesa dell'albero, senza bisogno di tornare indietro. Quando cancelliamo una chiave in un nodo interno, però, la procedura effettua un passaggio in discesa dell'albero, ma potrebbe avere bisogno di ritornare nel nodo dal quale la chiave è stata cancellata per sostituire la chiave con il suo predecessore o successore (casi 2a e 2b).

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Sebbene questa procedura possa sembrare complicata, tuttavia richiede soltanto $O(h)$ operazioni su disco per un B-albero di altezza h, perché vengono effettua-



te soltanto $O(1)$ chiamate delle operazioni DISK-READ e DISK-WRITE tra due invocazioni ricorsive della procedura. Il tempo della CPU è $O(th) = O(t \log_t n)$.

Esercizi

18.4-1

Illustrate i risultati che si ottengono cancellando le chiavi C, P e V, ordinatamente, dall'albero della Figura 18.8(f).

18.4-2

Scrivete lo pseudocodice per B-TREE-DELETE.

Problemi

18-1 Stack nella memoria secondaria

Supponiamo di implementare uno stack in un calcolatore che ha una quantità relativamente piccola di memoria primaria veloce e una quantità relativamente grande di memoria meno veloce su un disco. Le operazioni PUSH e POP operano con valori di una singola parola. Lo stack può crescere al punto da non potere essere interamente contenuto nella memoria principale, quindi la maggior parte di esso sarà memorizzata sul disco.

Una semplice, ma inefficiente, implementazione dello stack tiene l'intero stack sul disco. Manteniamo in memoria un puntatore allo stack, che è l'indirizzo sul disco dell'elemento in cima allo stack. Se il puntatore ha valore p, l'elemento in cima allo stack è la $(p \bmod m)$-esima parola nella pagina $\lfloor p/m \rfloor$ del disco, dove m è il numero di parole per pagina.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995080-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Per implementare l'operazione PUSH (inserimento), aumentiamo il puntatore dello stack, leggiamo la pagina appropriata sul disco caricandola nella memoria principale, copiamo l'elemento da inserire nella parola appropriata della pagina della memoria principale e, infine, scriviamo la pagina modificata sul disco. L'operazione POP (cancellazione) è simile. Diminuiamo il puntatore dello stack,

leggiamo la pagina appropriata sul disco caricandola nella memoria principale e restituiamo l'elemento in cima allo stack. Non occorre scrivere la pagina sul disco, perché non è stata modificata.

Dal momento che le operazioni su disco sono relativamente costose, per qualsiasi implementazione occorre tenere in conto due costi: il numero totale di accessi al disco e il tempo totale della CPU. Nell'esempio in esame, ciascun accesso a una pagina di m parole ha due costi: un accesso al disco e un tempo di CPU $\Theta(m)$.

a. Asintoticamente, qual è nel caso peggiore il numero di accessi al disco per n operazioni con lo stack, se si usa questa semplice implementazione? Qual è il tempo della CPU per n operazioni con lo stack? (Esprimete le vostre soluzioni in funzione di m e n sia per questo punto che per i successivi.)

Consideriamo adesso un'implementazione dello stack in cui manteniamo nella memoria principale una pagina dello stack (utilizziamo anche una piccola quantità di questa memoria per tenere traccia della pagina che si trova correntemente in memoria). Possiamo eseguire un'operazione con lo stack soltanto se la pagina del disco sulla quale operare si trova nella memoria principale. Se necessario, la pagina correntemente in memoria può essere scritta sul disco e la nuova pagina può essere letta dal disco e caricata in memoria. Se la pagina sulla quale operare si trova già in memoria, non occorre accedere al disco.

b. Qual è, nel caso peggiore, il numero di accessi al disco richiesti per eseguire n operazioni PUSH? Qual è il tempo della CPU?

c. Qual è, nel caso peggiore, il numero di accessi al disco richiesti per eseguire n operazioni sullo stack? Qual è il tempo della CPU?

Supponiamo adesso di implementare lo stack tenendo due pagine nella memoria principale (oltre a un piccolo numero di parole per la contabilità).

d. Spiegate come gestire le pagine dello stack in modo che il numero ammortizzato di accessi al disco per qualsiasi operazione con lo stack sia $O(1/m)$ e il tempo ammortizzato della CPU per qualsiasi operazione con lo stack sia $O(1)$.

18-2 Operazioni di congiunzione e spartizione negli alberi 2-3-4

L'operazione di ***congiunzione*** (join) prende due insiemi dinamici S' e S'' e un elemento x tale che, per qualsiasi $x' \in S'$ e $x'' \in S''$, si abbia $x'.key < x.key < x''.key$ e restituisce un insieme $S = S' \cup \{x\} \cup S''$. La ***spartizione*** (split) corrisponde all'operazione "inversa" della congiunzione: dato un insieme dinamico S e un elemento $x \in S$, la spartizione crea un insieme S' che è formato da tutti gli elementi di $S - \{x\}$ le cui chiavi sono più piccole di $x.key$ e un insieme S'' formato da tutti gli elementi di $S - \{x\}$ le cui chiavi sono più grandi di $x.key$. In questo problema, vogliamo esaminare come implementare queste operazioni con gli alberi 2-3-4. Supponiamo per comodità che gli elementi siano formati soltanto da chiavi e che tutti i valori delle chiavi siano distinti.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12; Numero Ordine Libreria: 199503016-2207072; Copyright © 2022, McGraw-Hill Education (Italy)

a. Spiegate come mantenere, per ogni nodo x di un albero 2-3-4, l'altezza del sottoalbero con radice in x come un attributo $x.height$. Verificate che la vostra implementazione non influisca sui tempi di esecuzione asintotici delle operazioni di ricerca, inserimento e cancellazione.

b. Spiegate come implementare l'operazione di congiunzione. Dati due alberi 2-3-4 T' e T'' e una chiave k, la congiunzione dovrebbe essere eseguita nel tempo $O(1 + |h' - h''|)$, dove h' e h'' sono, rispettivamente, le altezze degli alberi T' e T''.

c. Considerate il cammino semplice p dalla radice di un albero 2-3-4 T a una data chiave k, l'insieme S' delle chiavi in T che sono minori di k e l'insieme S'' delle chiavi in T che sono maggiori di k. Dimostrate che p divide S' in un insieme di alberi $\{T'_0, T'_1, \ldots, T'_m\}$ e in un insieme di chiavi $\{k'_1, k'_2, \ldots, k'_m\}$ dove, per $i = 1, 2, \ldots, m$, si ha $y < k'_i < z$ per qualsiasi chiave $y \in T'_{i-1}$ e $z \in T'_i$. Qual è la relazione fra le altezze di T'_{i-1} e T'_i? Descrivete come p divide S'' in un insieme di alberi e in un insieme di chiavi.

d. Spiegate come implementare l'operazione di spartizione con l'albero T. Utilizzate l'operazione di congiunzione per assemblare le chiavi di S' in un unico albero 2-3-4 T' e le chiavi di S'' in un unico albero 2-3-4 T''. Il tempo di esecuzione dell'operazione di spartizione dovrebbe essere $O(\lg n)$, dove n è il numero di chiavi di T (*suggerimento:* i costi delle congiunzioni dovrebbero combinarsi).

Note

Knuth [212], Sedgewick [307], Aho, Hopcroft e Ullman [5] hanno analizzato dettagliatamente i B-alberi e vari schemi di alberi bilanciati. Comer [75] presenta una panoramica completa dei B-alberi. Guibas e Sedgewick [156] descrivono le relazioni fra i vari tipi di schemi di alberi bilanciati, inclusi gli alberi rosso-neri e gli alberi 2-3-4.

Nel 1970, J. E. Hopcroft ha inventato gli alberi 2-3 (i precursori dei B-alberi e degli alberi 2-3-4) nei quali ogni nodo interno ha due o tre figli. Nel 1972, Bayer e McCreight hanno presentato i B-alberi [35]; non hanno spiegato la scelta del nome per questi alberi.

Bender, Demaine e Farach-Colton [40] hanno studiato come utilizzare in modo efficiente i B-alberi in presenza di effetti connessi alla struttura gerarchica della memoria. I loro algoritmi ***cache-oblivious*** operano con efficienza senza conoscere esplicitamente le quantità di dati trasferiti all'interno della struttura gerarchica della memoria.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

19 Heap di Fibonacci

Gli heap di Fibonacci sono strutture dati che svolgono un duplice compito. In primo luogo, esse supportano un insieme di operazioni che formano il cosiddetto "heap riunibile". In secondo luogo, molte operazioni degli heap di Fibonacci vengono eseguite in un tempo ammortizzato costante, che rende questa struttura dati molto adatta alle applicazioni che utilizzano spesso queste operazioni.

Heap riunibili

Un ***heap riunibile*** è una struttura dati che supporta le seguenti operazioni, in cui ciascun elemento ha una chiave (*key*):

MAKE-HEAP() crea e restituisce un nuovo heap senza elementi.

INSERT(H, x) inserisce nell'heap H un elemento x, la cui chiave è già riempita.

MINIMUM(H) restituisce un puntatore all'elemento dell'heap la cui chiave è minima.

EXTRACT-MIN(H) toglie dall'heap H l'elemento con chiave minima, restituendo un puntatore all'elemento.

UNION(H_1, H_2) crea e restituisce un nuovo heap che contiene tutti gli elementi degli heap H_1 e H_2. Gli heap H_1 e H_2 vengono "distrutti" da questa operazione.

Oltre alle precedenti operazioni degli heap riunibili, gli heap di Fibonacci supportano anche le seguenti operazioni.

DECREASE-KEY(H, x, k) assegna all'elemento x all'interno dell'heap H il nuovo valore della chiave k, che supponiamo non sia maggiore del suo valore corrente.[1]

DELETE(H, x) cancella l'elemento x dall'heap H.

Come indica la tabella della Figura 19.1, se non è richiesta l'operazione UNION, i normali heap binari (definiti nel Capitolo 6) funzionano abbastanza bene. Le operazioni, tranne UNION, vengono eseguite nel caso peggiore nel tempo $O(\lg n)$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[1] Come detto nell'introduzione alla Parte V, gli heap riunibili sono, per nostra definizione, min-heap riunibili, quindi è possibile applicare le operazioni MINIMUM, EXTRACT-MIN e DECREASE-KEY. In alternativa, potremmo definire un ***max-heap riunibile*** con le operazioni MAXIMUM, EXTRACT-MAX e INCREASE-KEY.

Procedura	Heap binario (caso peggiore)	Heap di Fibonacci (ammortizzato)
MAKE-HEAP	$\Theta(1)$	$\Theta(1)$
INSERT	$\Theta(\lg n)$	$\Theta(1)$
MINIMUM	$\Theta(1)$	$\Theta(1)$
EXTRACT-MIN	$\Theta(\lg n)$	$O(\lg n)$
UNION	$\Theta(n)$	$\Theta(1)$
DECREASE-KEY	$\Theta(\lg n)$	$\Theta(1)$
DELETE	$\Theta(\lg n)$	$O(\lg n)$

Figura 19.1 I tempi di esecuzione delle operazioni con due implementazioni di heap riunibili. Il numero di elementi nell'heap durante l'esecuzione di un'operazione è indicato da n.

con un heap binario. Se, invece, deve essere supportata l'operazione UNION, allora le prestazioni degli heap binari sono scadenti. Concatenando i due array che contengono gli heap binari da fondere e, poi, eseguendo BUILD-MIN-HEAP (Paragrafo 6.3), l'operazione UNION impiega un tempo $\Theta(n)$ nel caso peggiore.

Gli heap di Fibonacci, d'altra parte, hanno limiti di tempo asintotici migliori degli heap binari per le operazioni INSERT, UNION e DECREASE-KEY, e hanno gli stessi tempi di esecuzione asintotici per le altre operazioni. Notate, tuttavia, che i tempi di esecuzione per gli heap di Fibonacci nella Figura 19.1 sono limiti sui tempi ammortizzati, non sui tempi nel caso peggiore. L'operazione UNION richiede un tempo ammortizzato costante in un heap di Fibonacci, che è significativamente migliore del tempo lineare nel caso peggiore richiesto in un heap binario (supponendo, naturalmente, che sia sufficiente un limite sul tempo ammortizzato).

Gli heap di Fibonacci nella teoria e nella pratica

Da un punto di vista pratico, però, i fattori costanti e la complessità di programmazione degli heap di Fibonacci rendono questi heap meno interessanti degli ordinari heap binari (o k-ari) nella maggior parte delle applicazioni. Quindi, gli heap di Fibonacci restano interessanti principalmente da un punto di vista teorico. Se venisse sviluppata una struttura dati molto più semplice e con gli stessi limiti sui tempi ammortizzati degli heap di Fibonacci, essa avrebbe anche un grande interesse pratico.

Gli heap binari e gli heap di Fibonacci non eseguono in modo efficiente l'operazione SEARCH, in quanto possono impiegare un tempo relativamente lungo per trovare un elemento con una data chiave. Per questo motivo, operazioni come DECREASE-KEY e DELETE, che fanno riferimento a un particolare elemento, richiedono un puntatore all'elemento fra i loro parametri di input. Come visto nella nostra analisi delle code di priorità (Paragrafo 6.5), quando utilizziamo un heap riunibile in un'applicazione, spesso memorizziamo in ogni elemento dell'heap riunibile un riferimento al corrispondente oggetto dell'applicazione; inoltre, memorizziamo in ogni oggetto dell'applicazione un riferimento al corrispondente elemento dell'heap riunibile. Il tipo di questi riferimenti dipende dall'applicazione e dalla sua implementazione.

Acquistato da Michele Michelo su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Come molte altre strutture dati finora viste, anche gli heap di Fibonacci si basano su alberi radicati. Rappresenteremo ciascun elemento con un nodo all'interno di un albero, e ciascun nodo avrà un attributo *key* (chiave). Nella parte restante

di questo capitolo utilizzeremo il termine "nodo", anziché "elemento". Ignoreremo inoltre i problemi relativi all'allocazione dei nodi prima dell'inserimento e al rilascio dei nodi dopo la cancellazione, supponendo che il codice che chiama le procedure degli heap tratti questi dettagli.

Il Paragrafo 19.1 definisce gli heap di Fibonacci, ne descrive la rappresentazione e illustra la funzione potenziale utilizzata nell'analisi ammortizzata. Il Paragrafo 19.2 spiega come implementare le operazioni con gli heap riunibili e calcolare i limiti sui tempi ammortizzati della Figura 19.1. Le due restanti operazioni, DECREASE-KEY e DELETE, sono descritte nel Paragrafo 19.3. Infine, il Paragrafo 19.4 completa una parte fondamentale dell'analisi e spiega anche l'origine del nome assegnato a questa struttura dati.

19.1 Struttura degli heap di Fibonacci

Un ***heap di Fibonacci*** è un insieme di alberi radicati che sono ***min-heap ordinati***, ovvero ciascun albero gode della ***proprietà dei min-heap***: la chiave di un nodo è maggiore o uguale alla chiave di suo padre. La Figura 19.2(a) illustra un esempio di heap di Fibonacci.

Come illustra la Figura 19.2(b), ogni nodo x contiene un puntatore $x.p$ a suo padre e un puntatore $x.child$ a uno dei suoi figli. I figli di x sono collegati insieme in una lista circolare doppiamente concatenata, che è detta ***lista dei figli*** di x. Ogni figlio y in una lista di figli ha i puntatori $y.left$ e $y.right$ che puntano, rispettivamente, al fratello sinistro e al fratello destro di y. Se il nodo y è figlio unico, allora $y.left = y.right = y$. L'ordine in cui i fratelli appaiono in una lista di figli è arbitrario.

Le liste circolari doppiamente concatenate (vedere il Paragrafo 10.2) offrono due vantaggi quando vengono utilizzate negli heap di Fibonacci. In primo luogo, ci permettono di eliminare un nodo da una lista circolare doppiamente concatenata nel tempo $O(1)$. In secondo luogo, date due liste, possiamo collegarle in un'unica lista circolare doppiamente concatenata nel tempo $O(1)$. Nelle descrizioni delle operazioni con gli heap di Fibonacci, faremo riferimento a queste operazioni in modo informale, lasciando al lettore il compito di occuparsi dei dettagli delle loro implementazioni.

Utilizzeremo altri due attributi in ciascun nodo. Il numero di figli nella lista dei figli del nodo x è memorizzato in $x.degree$. L'attributo booleano $x.mark$ indica se il nodo x ha perso un figlio dall'ultima volta che è diventato figlio di un altro nodo. I nodi appena creati non sono marcati. Un nodo x perde la marcatura ogni volta che diventa figlio di un altro nodo. Finché non esamineremo l'operazione DECREASE-KEY nel Paragrafo 19.3, tutti gli attributi *mark* saranno impostati a FALSE.

È possibile accedere a un dato heap di Fibonacci H tramite un puntatore $H.min$ alla radice di un albero che contiene una chiave minima; questo nodo speciale è detto ***nodo minimo*** dell'heap di Fibonacci. Se un heap di Fibonacci H è vuoto, allora $H.min = \text{NIL}$.

Acquistato da Michele Michele su Webster il 2022-0[illegible] Copyright © 2023 The McGraw-Hill Education (Italy)

Le radici di tutti gli alberi in un heap di Fibonacci sono collegate insieme tramite i loro puntatori *left* e *right* in una lista circolare doppiamente concatenata, che è detta ***lista delle radici*** dell'heap di Fibonacci. Il puntatore $H.min$ quindi punta al nodo che ha la chiave minima nella lista delle radici. L'ordine degli alberi all'interno di una lista di radici è arbitrario.

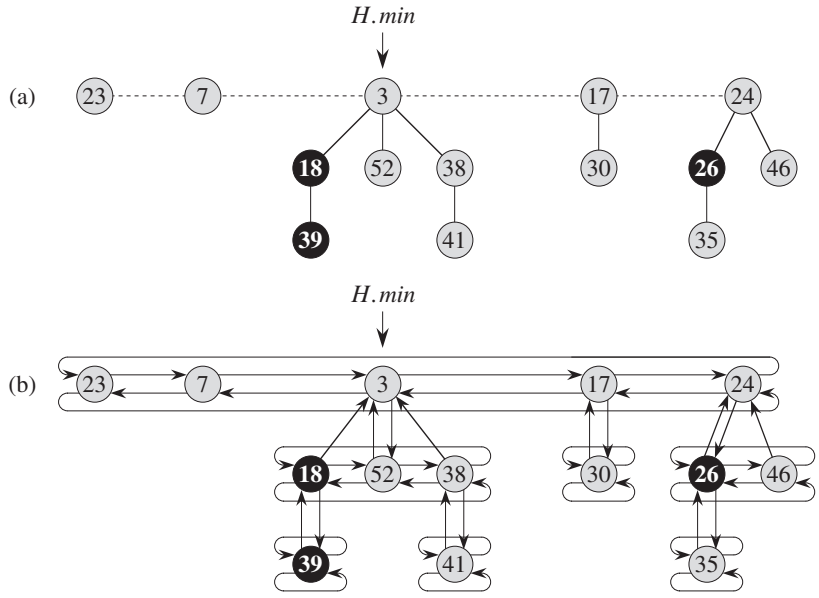


Figura 19.2 **(a)** Un heap di Fibonacci formato da cinque alberi ordinati come min-heap e 14 nodi. La linea tratteggiata indica la lista delle radici. Il nodo minimo dell'heap è quello che contiene la chiave 3. I tre nodi marcati hanno sfondo nero. Il potenziale di questo particolare heap di Fibonacci è $5 + 2 \cdot 3 = 11$. **(b)** Una rappresentazione più completa che mostra i puntatori p (frecce rivolte in alto), *child* (frecce rivolte in basso) e *left* e *right* (frecce laterali). Questi dettagli sono omessi nelle restanti figure di questo capitolo, in quanto tutte le informazioni illustrate possono essere determinate da ciò che appare nella parte (a).

Utilizzeremo un altro attributo per un heap di Fibonacci H: il numero dei nodi che si trovano correntemente in H è memorizzato in $H.n$.

Funzione potenziale

Come accennato in precedenza, utilizzeremo il metodo del potenziale del Paragrafo 17.3 per analizzare le prestazioni delle operazioni con gli heap di Fibonacci. Per un dato heap di Fibonacci H, indichiamo con $t(H)$ il numero di alberi nella lista delle radici di H e con $m(H)$ il numero di nodi marcati in H. Il potenziale dell'heap di Fibonacci H è definito da

$$\Phi(H) = t(H) + 2\,m(H) \tag{19.1}$$

(Nel Paragrafo 19.3 vedremo quale sia l'intuizione per la scelta di questa funzione potenziale.) Per esempio, il potenziale dell'heap di Fibonacci illustrato nella Figura 19.2 è $5 + 2 \cdot 3 = 11$. Il potenziale di un insieme di heap di Fibonacci è la somma dei potenziali degli heap di Fibonacci di cui è costituito. Supporremo che un'unità di potenziale possa pagare una quantità costante di lavoro; questa quantità costante è sufficientemente grande da coprire il costo di una qualsiasi delle componenti di lavoro con tempo costante prefissato che potremo incontrare.

Assumiamo che un'applicazione con heap di Fibonacci inizi senza heap. Il potenziale iniziale, quindi, è 0 e, per l'equazione (19.1), resterà non negativo nei successivi periodi. Dall'equazione (17.3) deriva che un limite superiore al costo totale ammortizzato è anche un limite superiore al costo totale effettivo per la sequenza delle operazioni.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Grado massimo

L'analisi ammortizzata che faremo nei restanti paragrafi di questo capitolo suppone che ci sia un limite superiore noto $D(n)$ per il grado massimo di un nodo qualsiasi in un heap di Fibonacci di n nodi. Non lo dimostreremo, ma quando sono supportate soltanto le operazioni con heap riunibili, allora $D(n) \leq \lfloor \lg n \rfloor$ (il

Problema 19-2(d) vi chiede di dimostrare tale proprietà). Nei Paragrafi 19.3 e 19.4 dimostreremo che, quando sono supportate anche le operazioni DECREASE-KEY e DELETE, allora $D(n) = O(\lg n)$.

19.2 Operazioni con heap riunibili

L'idea chiave delle operazioni degli heap riunibili eseguite con gli heap di Fibonacci consiste nel ritardare il lavoro quanto più è possibile. C'è un bilanciamento di prestazioni fra le implementazioni delle varie operazioni. Per esempio, inseriamo un nodo aggiungendolo alla lista delle radici; ciò richiede un tempo costante. Se iniziassimo da un heap di Fibonacci vuoto e poi inserissimo k nodi, l'heap di Fibonacci sarebbe formato da una lista di k nodi. Il bilanciamento consiste nel fatto che, quando eseguiamo un'operazione EXTRACT-MIN con l'heap di Fibonacci H, dopo avere rimosso il nodo cui punta $H.min$, dovremo ricercare tra i restanti $k - 1$ nodi nella lista delle radici per trovare il nuovo nodo minimo. Mentre effettuiamo le ricerche nella lista delle radici durante l'operazione EXTRACT-MIN, consolidiamo anche i nodi in alberi min-heap ordinati per ridurre la dimensione della lista delle radici. Vedremo che, indipendentemente da come si presenta la lista delle radici prima di un'operazione EXTRACT-MIN, successivamente ciascun nodo avrà un grado che è unico all'interno della lista delle radici; questo porta ad avere una dimensione della lista delle radici non maggiore di $D(n) + 1$.

Creare un nuovo heap di Fibonacci

Per creare un heap di Fibonacci vuoto, la procedura MAKE-FIB-HEAP alloca e restituisce l'heap di Fibonacci H, dove $H.n = 0$ e $H.min =$ NIL; non ci sono alberi in H. Poiché $t(H) = 0$ e $m(H) = 0$, il potenziale dell'heap di Fibonacci vuoto è $\Phi(H) = 0$. Il costo ammortizzato di MAKE-FIB-HEAP è quindi uguale al suo costo effettivo $O(1)$.

Inserire un nodo

La seguente procedura inserisce un nodo x nell'heap di Fibonacci H, supponendo che il nodo sia stato già allocato e $x.key$ sia stato già riempito.

```
FIB-HEAP-INSERT(H, x)
 1  x.degree = 0
 2  x.p = NIL
 3  x.child = NIL
 4  x.mark = FALSE
 5  if H.min == NIL
 6       crea una lista delle radici per H che contiene solo x
 7       H.min = x
 8  else inserisce x nella lista delle radici di H
 9       if x.key < H.min.key
10            H.min = x
11  H.n = H.n + 1
```

Acquistato da Michele Michele su Webster il 2022-07-07 22:12 [Numero Ordine Libreria: 1895020.11_220707_0] Copyright © 2022, McGraw-Hill Education (Italy)

Le righe 1–4 inizializzano alcuni degli attributi strutturali del nodo x. La riga 5 verifica se l'heap di Fibonacci H è vuoto. Se lo è, le righe 6–7 rendono x l'uni-

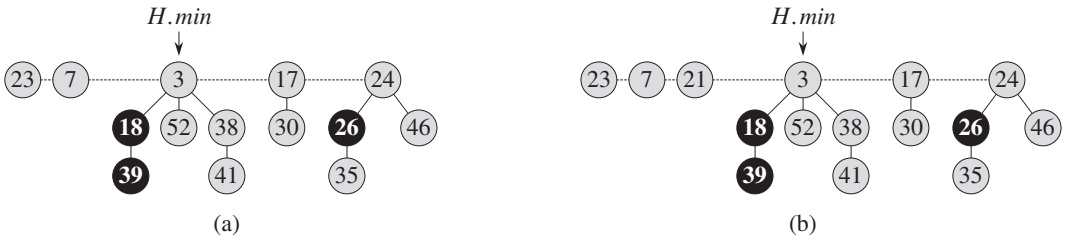


Figura 19.3 Inserimento di un nodo in un heap di Fibonacci. **(a)** Un heap di Fibonacci H. **(b)** L'heap di Fibonacci H dopo l'inserimento del nodo con la chiave 21. Il nodo diventa un albero ordinato come min-heap e poi viene aggiunto alla lista delle radici, diventando il fratello sinistro della radice.

co nodo della lista delle radici di H e impostano $H.min$ in modo che punti a x. Altrimenti, le righe 8–10 inseriscono x nella lista delle radici di H e aggiornano $H.min$, se necessario. Infine, la riga 11 incrementa $H.n$ per tenere conto dell'inserimento del nuovo nodo. La Figura 19.3 mostra un nodo con la chiave 21 inserito nell'heap di Fibonacci della Figura 19.2.

Per determinare il costo ammortizzato di FIB-HEAP-INSERT, indichiamo con H l'heap di Fibonacci di input e con H' l'heap di Fibonacci risultante. Allora, $t(H') = t(H) + 1$ e $m(H') = m(H)$; l'aumento del potenziale è

$$((t(H) + 1) + 2\,m(H)) - (t(H) + 2\,m(H)) = 1$$

Poiché il costo effettivo è $O(1)$, il costo ammortizzato è $O(1) + 1 = O(1)$.

Trovare il nodo minimo

Il nodo minimo di un heap di Fibonacci H è dato dal puntatore $H.min$, quindi possiamo trovare il nodo minimo nel tempo effettivo $O(1)$. Poiché il potenziale di H non cambia, il costo ammortizzato di questa operazione è eguale al suo costo effettivo $O(1)$.

Unire due heap di Fibonacci

La seguente procedura unisce gli heap di Fibonacci H_1 e H_2, distruggendo H_1 e H_2. Concatena le liste delle radici di H_1 e H_2 e poi determina il nodo minimo. Dopodiché, gli oggetti che rappresentano H_1 e H_2 non saranno più utilizzabili.

FIB-HEAP-UNION(H_1, H_2)
1 H = MAKE-FIB-HEAP()
2 $H.min = H_1.min$
3 concatena la lista delle radici di H_2 con la lista delle radici di H
4 **if** ($H_1.min$ == NIL) or ($H_2.min \neq$ NIL and $H_2.min.key < H_1.min.key$)
5 $H.min = H_2.min$
6 $H.n = H_1.n + H_2.n$
7 **return** H

Acquistato da Michele Michelutti Webster il 2022-07-07 22:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Le righe 1–3 concatenano le liste delle radici di H_1 e H_2 in una nuova lista di radici H. Le righe 2, 4 e 5 impostano il nodo minimo di H; la riga 6 assegna a $H.n$ il numero totale di nodi. La riga 7 restituisce l'heap di Fibonacci H risultante. Come nella procedura FIB-HEAP-INSERT, non si ha alcun consolidamento degli alberi.

La variazione del potenziale è

$$\begin{aligned}\Phi(H) - (\Phi(H_1) + \Phi(H_2)) \\ &= (t(H) + 2m(H)) - ((t(H_1) + 2m(H_1)) + (t(H_2) + 2m(H_2))) \\ &= 0\end{aligned}$$

in quanto $t(H) = t(H_1) + t(H_2)$ e $m(H) = m(H_1) + m(H_2)$. Il costo ammortizzato di FIB-HEAP-UNION è quindi uguale al suo costo effettivo $O(1)$.

Estrarre il nodo minimo

L'estrazione del nodo minimo è l'operazione più complicata fra quelle presentate in questo paragrafo; è anche quella in cui viene finalmente svolto il lavoro, fin qui rinviato, di consolidamento degli alberi nella lista delle radici. Il seguente pseudocodice estrae il nodo minimo. Per comodità, si suppone che, quando un nodo viene rimosso da una lista concatenata, i puntatori che restano nella lista vengano aggiornati; i puntatori che si trovano nel nodo estratto restano invariati. È utilizzata anche la procedura ausiliaria CONSOLIDATE, che è descritta più avanti.

```
FIB-HEAP-EXTRACT-MIN(H)
 1  z = H.min
 2  if z ≠ NIL
 3      for ciascun figlio x di z
 4          aggiungi x alla lista delle radici di H
 5          x.p = NIL
 6      rimuovi z dalla lista delle radici di H
 7      if z == z.right
 8          H.min = NIL
 9      else H.min = z.right
10          CONSOLIDATE(H)
11      H.n = H.n − 1
12  return z
```

Come illustra la Figura 19.4, FIB-HEAP-EXTRACT-MIN opera trasformando in radice ciascuno dei figli del nodo minimo ed eliminando il nodo minimo dalla lista delle radici. Poi consolida la lista delle radici collegando le radici di pari grado finché non resterà al più una radice per ciascun grado. La procedura inizia nella riga 1 salvando un puntatore z al nodo minimo; questo puntatore viene restituito alla fine. Se z = NIL, allora l'heap di Fibonacci H è già vuoto e abbiamo finito. Altrimenti cancelliamo il nodo z da H trasformando in radici di H tutti i figli di z nelle righe 3–5 (mettendoli nella lista delle radici) ed eliminando z dalla lista delle radici nella riga 6. Se z coincide con il suo figlio destro dopo la riga 6, allora z era l'unico nodo nella lista delle radici e non aveva figli, quindi tutto ciò che resta da fare è svuotare l'heap di Fibonacci nella riga 8 prima di restituire z. Altrimenti, impostiamo il puntatore $H.min$ nella lista delle radici in modo che punti a un nodo diverso da z (in questo caso, il figlio destro di z), che non sarà necessariamente il nuovo nodo minimo quando FIB-HEAP-EXTRACT-MIN avrà terminato. La Figura 19.4(b) illustra l'heap di Fibonacci della Figura 19.4(a) dopo che è stata eseguita la riga 9.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Utente Libreria: 1983035015 2207074 (Copyright © 2022, McGraw-Hill Education (Italy))

Il passo successivo, in cui viene ridotto il numero di alberi nell'heap di Fibonacci, consiste nel ***consolidare*** la lista delle radici di H; questa operazione è svolta

dalla chiamata di CONSOLIDATE(H). Il consolidamento della lista delle radici consiste nell'eseguire ripetutamente i seguenti passi finché ogni radice nella lista delle radici non avrà un *grado* distinto.

1. Trovare due radici x e y nella lista delle radici con lo stesso grado. Senza perdita di generalità assumiamo $x.key \leq y.key$.
2. ***Collegare*** y a x: eliminare y dalla lista delle radici e trasformare y in un figlio di x. Questa operazione è svolta dalla procedura FIB-HEAP-LINK. L'attributo $x.degree$ viene incrementato e la marcatura di y, se c'è, viene eliminata.

La procedura CONSOLIDATE usa un array ausiliario $A[0\,..\,D(H.n)]$ per tenere traccia delle radici in funzione dei loro gradi. Se $A[i] = y$, allora y è correntemente una radice con $y.degree = i$. Ovviamente, per allocare l'array dobbiamo saper calcolare il limite superiore $D(H.n)$ sul grado massimo; nel Paragrafo 19.4 vedremo come fare.

```
CONSOLIDATE(H)
 1  Sia A[0..D(H.n)] un nuovo array
 2  for i = 0 to D(H.n)
 3      A[i] = NIL
 4  for ciascun nodo w nella lista delle radici di H
 5      x = w
 6      d = x.degree
 7      while A[d] ≠ NIL
 8          y = A[d]          // Un altro nodo con lo stesso grado di x
 9          if x.key > y.key
10              scambia x con y
11          FIB-HEAP-LINK(H, y, x)
12          A[d] = NIL
13          d = d + 1
14      A[d] = x
15  H.min = NIL
16  for i = 0 to D(H.n)
17      if A[i] ≠ NIL
18          if H.min == NIL
19              crea una lista di radici per H che contiene solo A[i]
20              H.min = A[i]
21          else inserisce A[i] nella lista delle radici di H
22              if A[i].key < H.min.key
23                  H.min = A[i]
```

```
FIB-HEAP-LINK(H, y, x)
1  Rimuove y dalla lista delle radici di H
2  Trasforma y in un figlio di x, incrementando x.degree
3  y.mark = FALSE
```

Acquistato da Michele Michelangeli il 2022-07-07 22:13:13 Numero Ordine Libreria 1095038/0 220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

In dettaglio, la procedura CONSOLIDATE opera nel seguente modo. Le righe 1–3 allocano e inizializzano A ponendo ogni elemento a NIL. Il ciclo **for** (righe 4–14) elabora ciascuna radice w nella lista delle radici. Quando le radici vengono collegate insieme, w potrebbe essere collegata a qualche altro nodo e non essere più

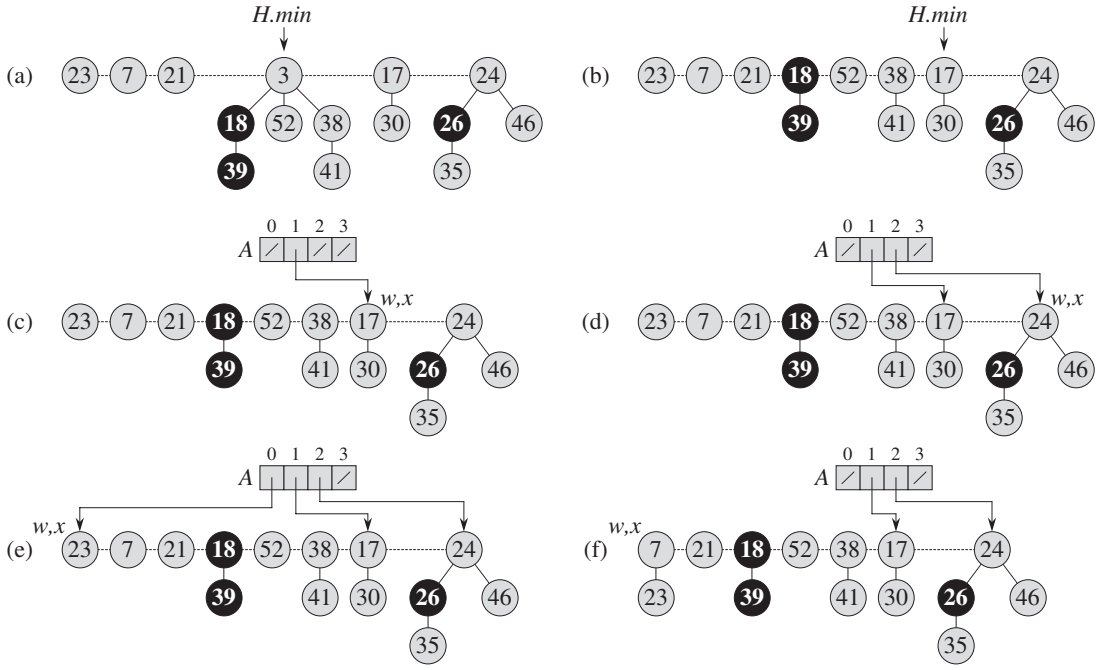
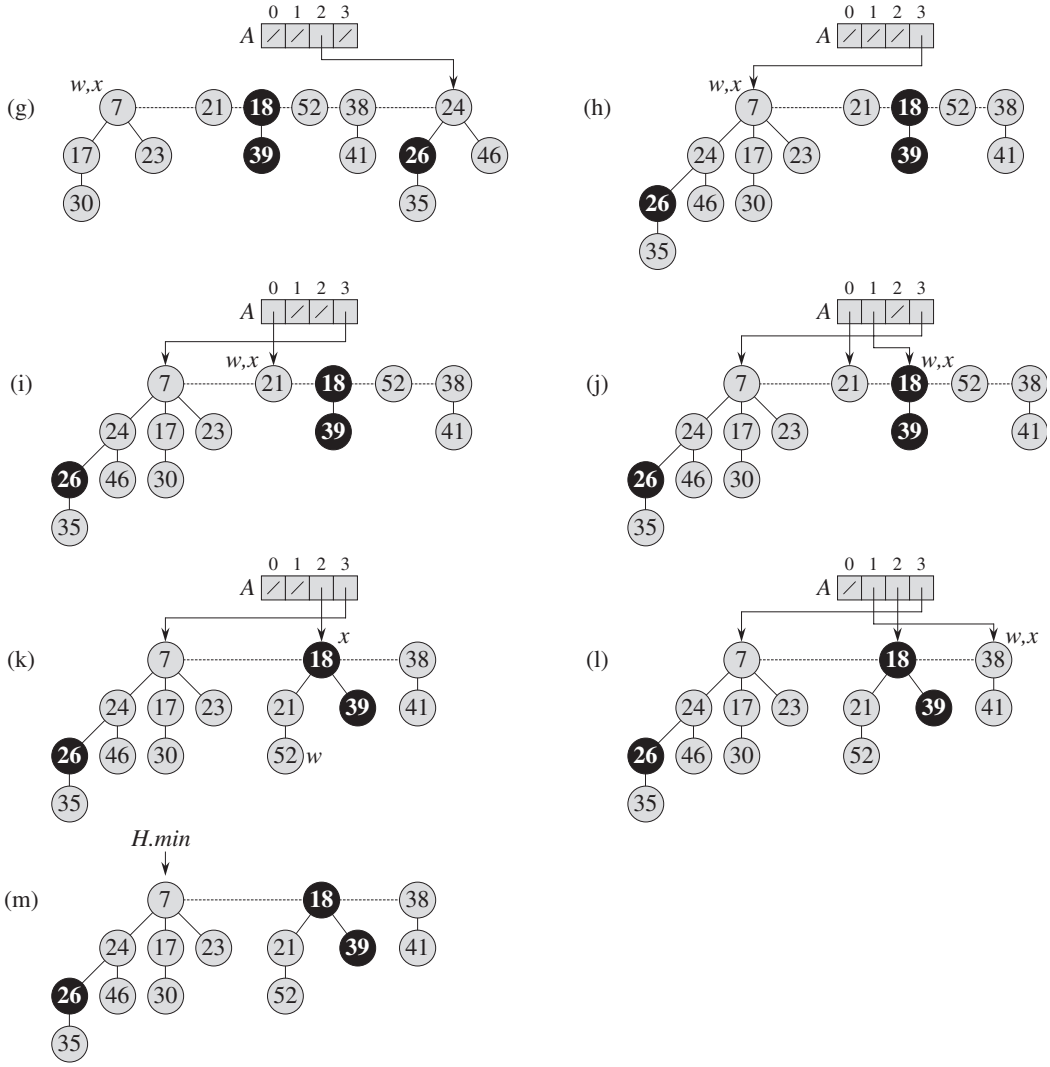


Figura 19.4 L'azione di FIB-HEAP-EXTRACT-MIN. **(a)** Un heap di Fibonacci H. **(b)** La situazione dopo che il nodo minimo z è stato eliminato dalla lista delle radici e i suoi figli sono stati aggiunti alla lista delle radici. **(c)–(e)** L'array A e gli alberi dopo ciascuna delle prime tre iterazioni del ciclo **for** (righe 4–14) della procedura CONSOLIDATE. La lista delle radici è elaborata iniziando dal nodo puntato da $H.min$ e seguendo i puntatori *right*. Ogni parte mostra i valori di w e x alla fine di un'iterazione. **(f)–(h)** La successiva iterazione del ciclo **for**, con i valori di w e x illustrati alla fine di ciascuna iterazione del ciclo **while** (righe 7–13). La parte (f) illustra la situazione dopo il primo passaggio del ciclo **while**. Il nodo con la chiave 23 è stato collegato al nodo con la chiave 7, che adesso è puntato da x. Nella parte (g) il nodo con la chiave 17 è stato collegato al nodo con la chiave 7, che è ancora puntato da x. Nella parte (h) il nodo con la chiave 24 è stato collegato al nodo con la chiave 7. Poiché nessun nodo era stato precedentemente puntato da $A[3]$, alla fine della iterazione del ciclo **for**, $A[3]$ è impostato per puntare alla radice dell'albero risultante. **(i)–(l)** La situazione dopo ciascuna delle quattro iterazioni successive del ciclo **for**. **(m)** L'heap di Fibonacci H dopo la ricostruzione della lista delle radici in base all'array A e la determinazione del nuovo puntatore $H.min$.

una radice. Nonostante ciò, w è sempre in un albero radicato in qualche nodo x, che può essere oppure no lo stesso w. Poiché vogliamo al massimo una radice per ogni grado, esaminiamo l'array A per vedere se contiene una radice y con lo stesso grado di x. In caso affermativo, colleghiamo le radici x e y, garantendo che x resti una radice dopo il collegamento; ovvero, colleghiamo y a x dopo lo scambio dei puntatori alle due radici se la chiave di y è più piccola della chiave di x. Una volta collegato y a x, il grado di x aumenta di 1, e quindi continuiamo questo processo, collegando x e un'altra radice il cui grado è uguale al nuovo grado di x, finché non ci saranno altre radici che abbiamo elaborato con lo stesso grado di x. Poi impostiamo l'elemento appropriato di A in modo che punti a x; così facendo, quando elaboreremo successivamente le radici, sappiamo che x è l'unica radice del suo grado che abbiamo già elaborato. Alla fine del ciclo **for**, resterà al più una radice per ogni grado e l'array A punterà a ciascuna di queste radici.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Unico Licenza: 1995969010220707 © Copyright © 2022, McGraw-Hill Education (Italy)

Il ciclo **while** (righe 7–13) collega ripetutamente la radice x dell'albero che contiene il nodo w con un altro albero la cui radice ha lo stesso grado di x, finché



nessun'altra radice avrà lo stesso grado. Questo ciclo **while** mantiene la seguente invariante:

All'inizio di ogni iterazione del ciclo **while**, $d = x.degree$.

Utilizziamo questa invariante di ciclo nel seguente modo:

Inizializzazione: la riga 6 garantisce che l'invariante di ciclo sia vera la prima volta che viene eseguito il ciclo.

Conservazione: in ciascuna iterazione del ciclo **while**, $A[d]$ punta a qualche radice y. Poiché $d = x.degree = y.degree$, dobbiamo collegare x e y. Con questa operazione di collegamento, chi ha la chiave più piccola fra x e y diventa padre dell'altro, per questo le righe 9–10 scambiano i puntatori di x e y se necessario. Poi, colleghiamo y a x chiamando FIB-HEAP-LINK(H, y, x) nella riga 11. Questa chiamata incrementa $x.degree$, ma lascia $y.degree$ a d. Poiché il nodo y non è più una radice, il suo puntatore nell'array A viene rimosso nella riga 12. Poiché la chiamata di FIB-HEAP-LINK incrementa il valore di $x.degree$, la riga 13 ripristina l'invariante $d = x.degree$.

Acquistato da Michele Michelucci (michelucci@libero.it) - Copia personale, vietata la riproduzione a fini commerciali - © 2022 McGraw-Hill Education (Italy)

Conclusione: ripetiamo il ciclo **while** finché $A[d] = \text{NIL}$, nel qual caso non c'è un'altra radice con lo stesso grado di x.

Dopo che il ciclo **while** è completato, impostiamo $A[d]$ a x nella riga 14 ed eseguiamo la successiva iterazione del ciclo **for**.

Le Figure 19.4(c)–(e) illustrano l'array A e gli alberi risultanti dopo le prime tre iterazioni del ciclo **for** (righe 4–14). Nella successiva iterazione del ciclo **for** vengono effettuati tre collegamenti, i cui risultati sono illustrati nelle Figure 19.4(f)–(h). Le Figure 19.4(i)–(l) illustrano l'effetto delle quattro iterazioni successive del ciclo **for**.

A questo punto bisogna fare pulizia. Una volta che il ciclo **for** (righe 4–14) è completato, la riga 15 svuota la lista delle radici e le righe 16–23 la ricostruiscono dall'array A. L'heap di Fibonacci risultante è illustrato nella Figura 19.4(m). Dopo avere consolidato la lista delle radici, FIB-HEAP-EXTRACT-MIN termina diminuendo il valore di $H.n$ nella riga 11 e restituendo un puntatore al nodo cancellato z nella riga 12.

Adesso siamo pronti a dimostrare che il costo ammortizzato per estrarre il nodo minimo di un heap di Fibonacci di n nodi è $O(D(n))$. Indichiamo con H l'heap di Fibonacci appena prima di eseguire l'operazione FIB-HEAP-EXTRACT-MIN.

Il costo effettivo per estrarre il nodo minimo può essere calcolato nel seguente modo. Un contributo pari a $O(D(n))$ è imputabile al fatto che ci sono al più $D(n)$ figli del nodo minimo che vengono elaborati in FIB-HEAP-EXTRACT-MIN e al lavoro che viene svolto nelle righe 2–3 e 16–23 di CONSOLIDATE. Resta da analizzare il contributo del ciclo **for** (righe 4–14), per il quale utilizzeremo un'analisi aggregata. La dimensione della lista delle radici dopo la chiamata di CONSOLIDATE è al più $D(n) + t(H) - 1$, perché è costituita dai $t(H)$ nodi originali della lista delle radici, meno il nodo radice estratto, più i figli del nodo estratto, il cui numero è al più $D(n)$. All'interno di una data iterazione del ciclo **for** (righe 4–14), il numero di iterazioni del ciclo **while** (righe 7–13) dipende dalla lista delle radici. Ma noi sappiamo che, ogni volta che viene eseguito il ciclo **while**, una delle radici viene collegata a un'altra radice, pertanto la quantità totale di lavoro svolto nel ciclo **for** è al più proporzionale a $D(n) + t(H)$. Dunque, il lavoro totale effettivo per estrarre il nodo minimo è $O(D(n) + t(H))$.

Il potenziale prima di estrarre il nodo minimo è $t(H) + 2\,m(H)$ e il potenziale dopo l'estrazione è al più $(D(n) + 1) + 2\,m(H)$, perché restano al più $D(n) + 1$ radici e nessun nodo viene marcato durante l'operazione. Il costo ammortizzato è quindi al più

$$\begin{aligned} & O(D(n) + t(H)) + ((D(n) + 1) + 2\,m(H)) - (t(H) + 2\,m(H)) \\ &\quad = \quad O(D(n)) + O(t(H)) - t(H) \\ &\quad = \quad O(D(n)) \end{aligned}$$

perché possiamo scegliere opportunamente le unità di potenziale per dominare la costante nascosta in $O(t(H))$. Intuitivamente, il costo per eseguire ciascun collegamento è pagato dalla riduzione del potenziale dovuta al fatto che nel collegamento si riduce di una unità il numero delle radici. Vedremo nel Paragrafo 19.4 che $D(n) = O(\lg n)$, quindi il costo ammortizzato per estrarre il nodo minimo è $O(\lg n)$.

Acquistato da Michele Michele su Webster il 2022-07-07 20:42:42. Numero Unico Licenza: 1995030040 227407 © Copyright © 2022, McGraw-Hill Education (Italy)

Esercizi

19.2-1

Rappresentate l'heap di Fibonacci che si ottiene eseguendo la procedura FIB-HEAP-EXTRACT-MIN sull'heap di Fibonacci illustrato nella Figura 19.4(m).

19.3 Diminuire il valore di una chiave e cancellare un nodo

In questo paragrafo, spiegheremo come diminuire il valore della chiave di un nodo in un heap di Fibonacci nel tempo ammortizzato $O(1)$ e come cancellare un nodo qualsiasi da un heap di Fibonacci di n nodi nel tempo ammortizzato $O(D(n))$. Nel Paragrafo 19.4 dimostreremo che il grado massimo $D(n)$ è $O(\lg n)$, e questo implica che le procedure FIB-HEAP-EXTRACT-MIN e FIB-HEAP-DELETE sono eseguite nel tempo ammortizzato $O(\lg n)$.

Diminuire il valore di una chiave

Nel seguente pseudocodice per l'operazione FIB-HEAP-DECREASE-KEY, supponiamo come in precedenza che l'eliminazione di un nodo da una lista concatenata non implichi alcuna modifica degli attributi strutturali nel nodo rimosso.

```
FIB-HEAP-DECREASE-KEY(H, x, k)
1  if k > x.key
2       error "la nuova chiave è maggiore di quella corrente"
3  x.key = k
4  y = x.p
5  if y ≠ NIL and x.key < y.key
6       CUT(H, x, y)
7       CASCADING-CUT(H, y)
8  if x.key < H.min.key
9       H.min = x
```

```
CUT(H, x, y)
1  Rimuove x dalla lista dei figli di y, decrementando y.degree
2  Aggiunge x alla lista delle radici di H
3  x.p = NIL
4  x.mark = FALSE
```

```
CASCADING-CUT(H, y)
1  z = y.p
2  if z ≠ NIL
3       if y.mark == FALSE
4            y.mark = TRUE
5       else CUT(H, y, z)
6            CASCADING-CUT(H, z)
```

Acquistato da Michele Michele su Webster | 2022-07-27 22:11 | Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

La procedura FIB-HEAP-DECREASE-KEY opera nel seguente modo. Le righe 1–3 verificano che la nuova chiave non sia maggiore della chiave corrente di x; poi assegnano la nuova chiave a x. Se x è una radice oppure se $x.key \geq y.key$, dove y è il padre di x, allora non è necessaria alcuna modifica strutturale, perché non è stato violato l'ordinamento dei min-heap. Le righe 4–5 verificano questa condizione.

Se l'ordinamento dei min-heap è stato violato, possono verificarsi molti cambiamenti. Iniziamo ***tagliando*** x nella riga 6. La procedura CUT "taglia" il collegamento fra x e suo padre y, trasformando x in una radice.

Utilizziamo gli attributi *mark* per ottenere i limiti di tempo desiderati. Questi attributi registrano una piccola parte della storia di ciascun nodo. Supponiamo che la storia del nodo x sia questa:

1. In un certo istante, il nodo x era una radice.
2. Successivamente, x viene collegato a (diventa figlio di) un altro nodo.
3. Infine, due figli di x vengono rimossi con dei tagli.

Non appena viene perso il secondo figlio, tagliamo x da suo padre, trasformandolo in una nuova radice. L'attributo $x.mark$ è TRUE se sono stati eseguiti i passi 1 e 2 ed è stato tagliato un figlio di x. La procedura CUT, quindi, cancella $x.mark$ nella riga 4, perché esegue il passo 1. (Adesso si può capire perché la riga 3 di FIB-HEAP-LINK cancella $y.mark$: il nodo y viene collegato a un altro nodo, quindi si sta svolgendo il passo 2. La volta successiva che un figlio di y sarà tagliato, $y.mark$ sarà impostato a TRUE.)

Non abbiamo ancora finito, perché x potrebbe essere il secondo figlio tagliato dal padre y dal momento in cui y è stato collegato a un altro nodo. Quindi, la riga 7 di FIB-HEAP-DECREASE-KEY esegue un'operazione di ***taglio in cascata*** su y. Se y è una radice, allora il test nella riga 2 di CASCADING-CUT fa semplicemente terminare la procedura. Se y non è marcato, la procedura lo marca nella riga 4, perché il suo primo figlio è stato appena tagliato, poi termina. Se, invece, y è marcato, allora ha appena perso il suo secondo figlio; y viene tagliato nella riga 5 e la procedura CASCADING-CUT chiama se stessa in modo ricorsivo (riga 6) sul padre z di y. Questa procedura effettua la ricorsione risalendo l'albero finché non trova una radice o un nodo non marcato.

Una volta che tutti i tagli in cascata sono stati fatti, le righe 8–9 di FIB-HEAP-DECREASE-KEY terminano aggiornando $H.min$ se necessario. L'unico nodo la cui chiave è cambiata è x (il valore della chiave è diminuito). Quindi, il nuovo nodo minimo è il nodo minimo originale o il nodo x.

La Figura 19.5 illustra l'esecuzione delle due chiamate della procedura FIB-HEAP-DECREASE-KEY, iniziando dall'heap di Fibonacci illustrato nella Figura 19.5(a). La prima chiamata, illustrata nella Figura 19.5(b), non richiede tagli in cascata. La seconda chiamata, illustrata nelle Figure 19.5(c)–(e), richiede due tagli in cascata.

Dimostriamo adesso che il costo ammortizzato di FIB-HEAP-DECREASE-KEY è soltanto $O(1)$. Iniziamo determinando il suo costo effettivo. La procedura FIB-HEAP-DECREASE-KEY impiega un tempo $O(1)$, più il tempo per effettuare i tagli in cascata. Supponete che CASCADING-CUT venga chiamata ricorsivamente c volte da una data invocazione di FIB-HEAP-DECREASE-KEY. Ogni chiamata di CASCADING-CUT richiede un tempo $O(1)$, escluse le chiamate ricorsive. Quindi, il costo effettivo di FIB-HEAP-DECREASE-KEY, incluse tutte le chiamate ricorsive, è $O(c)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950301 6-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Calcoliamo adesso la variazione di potenziale. Indichiamo con H l'heap di Fibonacci appena prima dell'operazione FIB-HEAP-DECREASE-KEY. La chiamata a CUT nella riga 6 di FIB-HEAP-DECREASE-KEY crea un nuovo albero radicato in x e toglie l'eventuale marcatura di x (poteva anche non essere marcato). Ogni chiamata ricorsiva di CASCADING-CUT, tranne l'ultima, taglia un nodo marcato e cancella il bit di marcatura. Dopodiché, ci sono $t(H) + c$ alberi (i $t(H)$ alberi

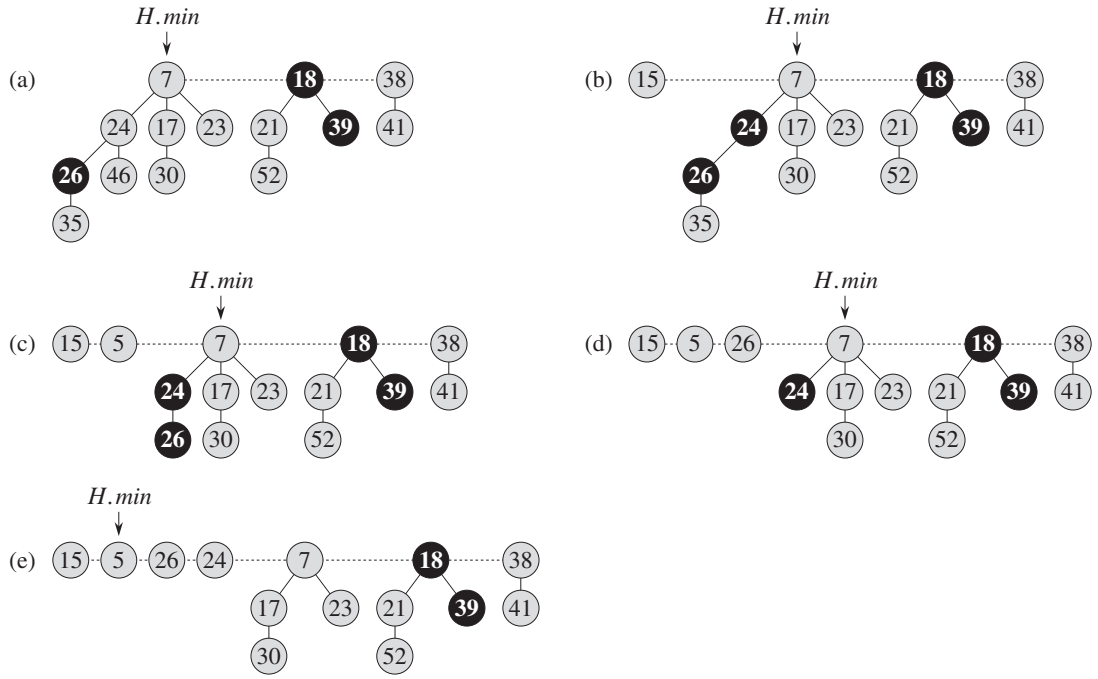


Figura 19.5 Due chiamate della procedura FIB-HEAP-DECREASE-KEY. **(a)** L'heap di Fibonacci iniziale. **(b)** La chiave 46 viene ridotta a 15; il nodo con questa chiave diventa radice e suo padre (con la chiave 24) viene marcato. **(c)–(e)** La chiave 35 viene ridotta a 5. Nella parte (c) il nodo, che adesso ha la chiave 5, è diventato radice. Suo padre, con la chiave 26, è marcato, quindi si verifica un taglio in cascata. Il nodo con la chiave 26 viene tagliato da suo padre e diventa una radice non marcata in (d). Si verifica un altro taglio in cascata, perché anche il nodo con la chiave 24 è marcato. Questo nodo viene tagliato da suo padre e diventa una radice non marcata nella parte (e). A questo punto, i tagli in cascata finiscono, perché il nodo con la chiave 7 è una radice (anche se questo nodo non fosse una radice, i tagli in cascata finirebbero lo stesso, perché il nodo non è marcato). Il risultato dell'operazione FIB-HEAP-DECREASE-KEY è illustrato nella parte (e), con $H.min$ che punta al nuovo nodo minimo.

originali, $c - 1$ alberi prodotti dai tagli in cascata e l'albero con radice in x) e al più $m(H) - c + 2$ nodi marcati ($c - 1$ sono stati smarcati dai tagli in cascata e l'ultima chiamata di CASCADING-CUT potrebbe avere marcato un nodo). La variazione del potenziale è quindi al più

$$((t(H) + c) + 2(m(H) - c + 2)) - (t(H) + 2\,m(H)) = 4 - c$$

Dunque, il costo ammortizzato di FIB-HEAP-DECREASE-KEY è al più

$$O(c) + 4 - c = O(1)$$

perché possiamo scegliere opportunamente le unità di potenziale per dominare la costante nascosta in $O(c)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Adesso si può capire perché la funzione potenziale è stata definita per includere un termine che è due volte il numero dei nodi marcati. Quando un nodo marcato y viene tagliato in cascata, il suo bit di marcatura viene cancellato, quindi il potenziale viene ridotto di 2 unità. Una unità di potenziale paga il taglio e la cancellazione del bit di marcatura; l'altra unità compensa l'incremento unitario di potenziale dovuto al fatto che il nodo y diventa una radice.

Cancellare un nodo

È facile cancellare un nodo da un heap di Fibonacci di n nodi nel tempo ammortizzato $O(D(n))$, come dimostra il seguente pseudocodice. Supponiamo che non sia già presente una chiave di valore $-\infty$ nell'heap di Fibonacci.

FIB-HEAP-DELETE(H, x)
1 FIB-HEAP-DECREASE-KEY$(H, x, -\infty)$
2 FIB-HEAP-EXTRACT-MIN(H)

La procedura FIB-HEAP-DELETE trasforma x nel nodo minimo dell'heap di Fibonacci assegnando ad esso una chiave minima unica $-\infty$. Il nodo x viene poi rimosso dall'heap di Fibonacci dalla procedura FIB-HEAP-EXTRACT-MIN. Il tempo ammortizzato di FIB-HEAP-DELETE è la somma del tempo ammortizzato $O(1)$ di FIB-HEAP-DECREASE-KEY e del tempo ammortizzato $O(D(n))$ di FIB-HEAP-EXTRACT-MIN. Poiché $D(n) = O(\lg n)$, come vedremo nel Paragrafo 19.4, il tempo ammortizzato di FIB-HEAP-DELETE è $O(\lg n)$.

Esercizi

19.3-1
Supponete che una radice x in un heap di Fibonacci sia marcata. Spiegate come x è diventata una radice marcata. Dimostrate che, ai fini dell'analisi, non è importante che la radice x sia marcata, anche se non è una radice che è stata prima collegata a un altro nodo e poi ha perso un figlio.

19.3-2
Giustificate il tempo ammortizzato $O(1)$ di FIB-HEAP-DECREASE-KEY come costo medio per operazione utilizzando il metodo dell'aggregazione.

19.4 Limitare il grado massimo

Per dimostrare che il tempo ammortizzato di FIB-HEAP-EXTRACT-MIN e FIB-HEAP-DELETE è $O(\lg n)$, dobbiamo provare che il limite superiore $D(n)$ per il grado di un nodo qualsiasi di un heap di Fibonacci di n nodi è $O(\lg n)$. In particolare, dimostreremo che $D(n) \leq \lfloor \log_\phi n \rfloor$, dove ϕ è il rapporto aureo, definito nell'equazione (3.24) come

$$\phi = (1 + \sqrt{5})/2 = 1.61803\ldots$$

Essenzialmente, l'analisi si sviluppa nel seguente modo. Per ogni nodo x all'interno di un heap di Fibonacci, la ***dimensione*** del nodo x, indicata con size(x), è il numero di nodi, incluso lo stesso x, nel sottoalbero con radice in x (notate che x non deve necessariamente trovarsi nella lista delle radici – può essere un nodo qualsiasi). Dimostreremo che size(x) è esponenziale in $x.degree$. Ricordiamo che $x.degree$ contiene sempre un conteggio accurato del grado di x.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-2207074-0 Copyright © 2022, McGraw-Hill Education (Italy)

Lemma 19.1
Sia x un nodo qualsiasi di un heap di Fibonacci e supponiamo che $x.degree = k$. Indichiamo con $y_1, y_2, \ldots, y_k$ i figli di x nell'ordine in cui sono stati collegati a x, dal meno recente al più recente. Allora, $y_1.degree \geq 0$ e $y_i.degree \geq i - 2$ per $i = 2, 3, \ldots, k$.

Dimostrazione Ovviamente, $y_1.degree \geq 0$.

Per $i \geq 2$, notiamo che, quando y_i è stato collegato a x, $y_1, y_2, \ldots, y_{i-1}$ erano tutti figli di x, quindi doveva essere $x.degree \geq i-1$. Il nodo y_i viene collegato a x soltanto se $x.degree = y_i.degree$, quindi doveva essere anche $y_i.degree \geq i-1$ in quel momento. Da allora, il nodo y_i ha perso al più un figlio, perché se avesse perso due figli, sarebbe stato tagliato da x. Dunque, possiamo concludere che $y_i.degree \geq i-2$. ■

Infine, siamo arrivati alla parte dell'analisi che spiega il nome "heap di Fibonacci". Ricordiamo dal Paragrafo 3.2 che per $k = 0, 1, 2, \ldots$, il k-esimo numero di Fibonacci è definito dalla ricorrenza

$$F_k = \begin{cases} 0 & \text{se } k = 0 \\ 1 & \text{se } k = 1 \\ F_{k-1} + F_{k-2} & \text{se } k \geq 2 \end{cases}$$

Il seguente lemma offre un altro modo di esprimere F_k.

Lemma 19.2
Per qualsiasi intero $k \geq 0$,

$$F_{k+2} = 1 + \sum_{i=0}^{k} F_i$$

Dimostrazione La dimostrazione si svolge per induzione su k. Se $k = 0$,

$$\begin{aligned} 1 + \sum_{i=0}^{0} F_i &= 1 + F_0 \\ &= 1 + 0 \\ &= 1 \\ &= F_2 \end{aligned}$$

Adesso, se facciamo l'ipotesi induttiva che $F_{k+1} = 1 + \sum_{i=0}^{k-1} F_i$, otteniamo

$$\begin{aligned} F_{k+2} &= F_k + F_{k+1} \\ &= F_k + \left(1 + \sum_{i=0}^{k-1} F_i\right) \\ &= 1 + \sum_{i=0}^{k} F_i \end{aligned}$$

■

Lemma 19.3

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Per qualsiasi intero $k \geq 0$, il $(k+2)$-esimo numero di Fibonacci soddisfa la relazione $F_{k+2} \geq \phi^k$.

Dimostrazione La dimostrazione si svolge per induzione su k. I casi base sono per $k = 0$ e $k = 1$. Quando $k = 0$, si ha $F_2 = 1 = \phi^0$, e quando $k = 1$, si ha $F_3 = 2 > 1.619 > \phi^1$. Il passo induttivo è per $k \geq 2$; supponiamo

che $F_{i+2} > \phi^i$ per $i = 0, 1, \ldots, k-1$. Ricordiamo che ϕ è la radice positiva dell'equazione (3.23), $x^2 = x + 1$. Quindi otteniamo

$$\begin{aligned} F_{k+2} &= F_{k+1} + F_k \\ &\geq \phi^{k-1} + \phi^{k-2} && \text{(per l'ipotesi induttiva)} \\ &= \phi^{k-2}(\phi + 1) \\ &= \phi^{k-2} \cdot \phi^2 && \text{(per l'equazione (3.23))} \\ &= \phi^k \end{aligned}$$

■

Il seguente lemma e il suo corollario completano l'analisi.

Lemma 19.4
Sia x un nodo qualsiasi in un heap di Fibonacci e sia $k = x.degree$. Allora, $\text{size}(x) \geq F_{k+2} \geq \phi^k$, dove $\phi = (1 + \sqrt{5})/2$.

Dimostrazione Indichiamo con s_k la dimensione più piccola possibile di un nodo qualsiasi di grado k in un heap di Fibonacci. Banalmente, $s_0 = 1$ e $s_1 = 2$. Il numero s_k è al più $\text{size}(x)$ e, poiché la dimensione di un nodo non si riduce se vengono aggiunti dei figli al nodo, il valore di s_k aumenta monotonicamente con k. Consideriamo un nodo z, in un heap di Fibonacci, tale che $z.degree = k$ e $\text{size}(z) = s_k$. Poiché $s_k \leq \text{size}(x)$, calcoliamo un limite inferiore per $\text{size}(x)$ trovando un limite inferiore per s_k. Come nel Lemma 19.1, indichiamo con $y_1, y_2, \ldots, y_k$ i figli di z nell'ordine in cui sono stati collegati a z. Per calcolare il limite di s_k, conteggiamo 1 per z stesso e 1 per il primo figlio y_1 (per il quale $\text{size}(y_1) \geq 1$), ottenendo

$$\begin{aligned} \text{size}(x) &\geq s_k \\ &\geq 2 + \sum_{i=2}^{k} s_{y_i.degree} \\ &\geq 2 + \sum_{i=2}^{k} s_{i-2} \end{aligned}$$

dove l'ultima riga deriva dal Lemma 19.1 (per cui $y_i.degree \geq i - 2$) e dalla monotonicità di s_k (per cui $s_{y_i.degree} \geq s_{i-2}$).

Adesso dimostriamo per induzione su k che $s_k \geq F_{k+2}$ per qualsiasi intero k non negativo. I casi base, per $k = 0$ e $k = 1$, sono banali. Per il passo induttivo, supponiamo che $k \geq 2$ e che $s_i \geq F_{i+2}$ per $i = 0, 1, \ldots, k-1$. Abbiamo

$$\begin{aligned} s_k &\geq 2 + \sum_{i=2}^{k} s_{i-2} \\ &\geq 2 + \sum_{i=2}^{k} F_i \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned} &= 1 + \sum_{i=0}^{k} F_i \\ &= F_{k+2} && \text{(per il Lemma 19.2)} \\ &\geq \phi^k && \text{(per il Lemma 19.3)} \end{aligned}$$

Quindi, abbiamo dimostrato che $\text{size}(x) \geq s_k \geq F_{k+2} \geq \phi^k$. ■

Corollario 19.5
Il grado massimo $D(n)$ di un nodo qualsiasi in un heap di Fibonacci di n nodi è $O(\lg n)$.

Dimostrazione Sia x un nodo qualsiasi in un heap di Fibonacci di n nodi e sia $k = x.degree$. Per il Lemma 19.4, si ha $n \geq \text{size}(x) \geq \phi^k$. Prendendo i logaritmi in base ϕ, otteniamo $k \leq \log_\phi n$ (in effetti, poiché k è un intero, $k \leq \lfloor \log_\phi n \rfloor$). Il grado massimo $D(n)$ di un nodo qualsiasi è dunque $O(\lg n)$. ■

Esercizi

19.4-1
Il professor Pinocchio sostiene che l'altezza di un heap di Fibonacci di n nodi sia $O(\lg n)$. Per dimostrare che il professore si sbaglia, trovate una sequenza di operazioni degli heap di Fibonacci che crea un heap di Fibonacci formato da un solo albero che è una catena lineare di n nodi (dove n è un intero positivo).

19.4-2
Supponiamo di generalizzare la regola del taglio in cascata in modo da tagliare un nodo x dal padre non appena il nodo perde il suo k-esimo figlio, per qualche costante intera k (la regola nel Paragrafo 19.3 usa $k = 2$). Per quali valori di k si ha $D(n) = O(\lg n)$?

Problemi

19-1 Implementazione alternativa della cancellazione
Il professor Pisano ha proposto la seguente variante della procedura FIB-HEAP-DELETE, sostenendo che è più veloce quando il nodo da cancellare non è il nodo puntato da $H.min$.

```
PISANO-DELETE(H, x)
1  if x == H.min
2       FIB-HEAP-EXTRACT-MIN(H)
3  else y = x.p
4       if y ≠ NIL
5            CUT(H, x, y)
6            CASCADING-CUT(H, y)
7       aggiunge la lista dei figli di x alla lista delle radici di H
8       elimina x dalla lista delle radici di H
```

a. L'asserzione del professore che questa procedura venga eseguita più velocemente si basa in parte sull'ipotesi che la riga 7 possa essere eseguita nel tempo effettivo $O(1)$. Che cosa c'è di sbagliato in questa ipotesi?

b. Trovate un buon limite superiore per il tempo effettivo della procedura PISANO-DELETE quando x non è puntato da $H.min$. Il vostro limite dovrebbe essere espresso in funzione di $x.degree$ e del numero c di chiamate della procedura CASCADING-CUT.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0. Copyright © 2022, McGraw-Hill Education (Italy)

c. Supponete di chiamare la procedura PISANO-DELETE(H, x) e di indicare con H' l'heap di Fibonacci risultante. Nell'ipotesi che il nodo x non sia una radice, trovate un limite al potenziale di H' esprimendolo in funzione di $x.degree$, c, $t(H)$ e $m(H)$.

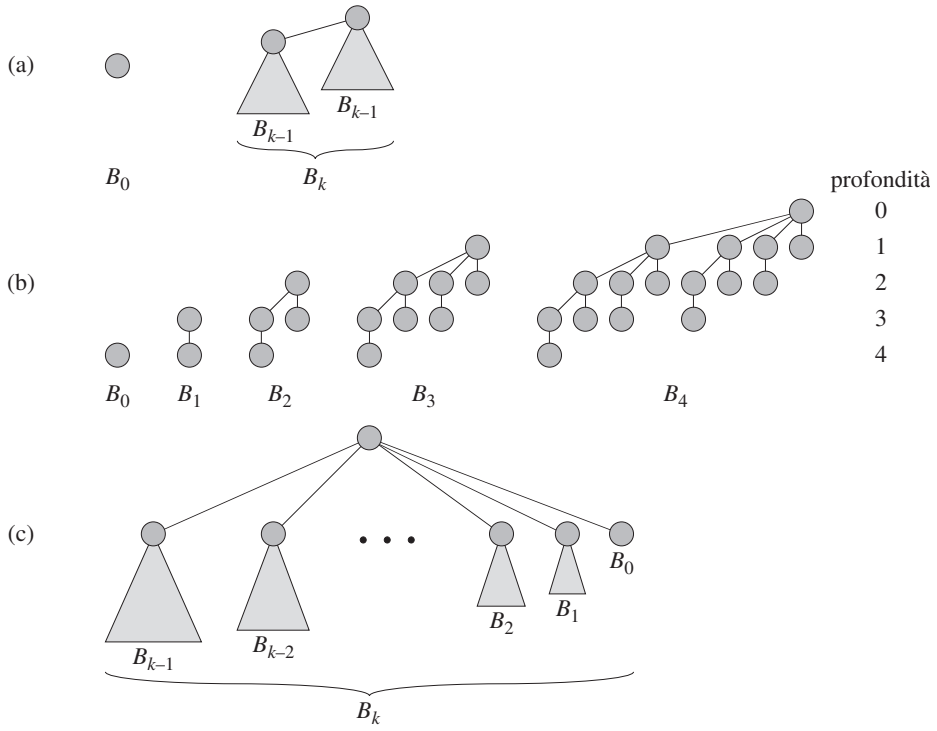


Figura 19.6
(a) La definizione ricorsiva dell'albero binomiale B_k. I triangoli rappresentano sottoalberi radicati. **(b)** Gli alberi binomiali da B_0 a B_4. Sono indicate le profondità dei nodi dell'albero B_4. **(c)** Un altro modo di rappresentare l'albero binomiale B_k.

d. Concludete che il tempo ammortizzato per PISANO-DELETE non è asintoticamente migliore di FIB-HEAP-DELETE, anche quando $x \neq min[H]$.

19-2 Alberi binomiali ed heap binomiali

Un ***albero binomiale*** B_k è un albero ordinato (Paragrafo B.5.2) definito in modo ricorsivo. Come illustra la Figura 19.6(a), l'albero binomiale B_0 è formato da un solo nodo. L'albero binomiale B_k è formato da due alberi binomiali B_{k-1} che sono ***collegati*** insieme: la radice di un albero è il figlio più a sinistra della radice dell'altro albero. La Figura 19.6(b) illustra gli alberi binomiali da B_0 a B_4.

a. Dimostrate che per un albero binomiale B_k:

1. Ci sono 2^k nodi.
2. L'altezza dell'albero è k.
3. Ci sono esattamente $\binom{k}{i}$ nodi alla profondità i per $i = 0, 1, \ldots, k$.
4. La radice ha grado k, che è maggiore del grado di qualsiasi altro nodo; inoltre, come mostra la Figura 19.6(c), se i figli della radice sono numerati da sinistra a destra con $k-1, k-2, \ldots, 0$, il figlio i è la radice di un sottoalbero B_i.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Un ***heap binomiale*** H è un insieme di alberi binomiali che soddisfa le seguenti proprietà:

1. Ogni nodo ha una chiave (come un heap di Fibonacci).
2. Ogni albero binomiale in H gode della proprietà dei min-heap.
3. Per qualsiasi intero k non negativo, c'è al più un albero binomiale in H la cui radice ha grado k.

b. Supponete che un heap binomiale H abbia n nodi. Spiegate la relazione tra gli alberi binomiali contenuti da H e la rappresentazione binaria di n. Concludete che H è formato al più da $\lfloor \lg n \rfloor + 1$ alberi binomiali.

Supponiamo di rappresentare un heap binomiale nel modo seguente. Usiamo lo schema figlio-sinistro fratello-destro descritto nel Paragrafo 10.4 per rappresentare ciascun albero binomiale all'interno di un heap binomiale. Ogni nodo contiene la sua chiave, il puntatore al proprio padre, il puntatore al figlio più a sinistra, il puntatore al fratello immediatamente a destra (questi puntatori sono NIL ove appropriato) e il proprio grado (ovvero il numero di figli, come negli heap di Fibonacci). Le radici formano una lista singolarmente concatenata, ordinata in funzione dei gradi delle radici (dal più piccolo al più grande); l'accesso all'heap binomiale avviene tramite un puntatore al primo nodo della lista delle radici.

c. Completate la descrizione della rappresentazione di un heap binomiale (ovvero il nome degli attributi, spiegate quando gli attributi hanno il valore NIL e definite come è organizzata la lista delle radici) e descrivete come implementare con gli heap binomiali le sette operazioni che sono state implementate con gli heap di Fibonacci in questo capitolo. Ogni operazione deve essere eseguita nel tempo $O(\lg n)$ nel caso peggiore, dove n è il numero di nodi dell'heap binomiale (o, nel caso di un'operazione UNION, nei due heap binomiali da unire). L'operazione MAKE-HEAP deve essere eseguita in un tempo costante.

d. Supponete di dover implementare soltanto le operazioni degli heap riunibili con un heap di Fibonacci (ovvero non implementate le operazioni DECREASE-KEY o DELETE). Quali saranno le analogie tra gli alberi di un heap di Fibonacci e quelli di un heap binomiale? Quali saranno le loro differenze? Dimostrate che il grado massimo in un heap di Fibonacci di n nodi è al più $\lfloor \lg n \rfloor$.

e. Il professor McGee ha ideato una nuova struttura dati basata sugli heap di Fibonacci. Un heap di McGee ha la stessa struttura di un heap di Fibonacci e supporta le operazioni degli heap riunibili. Le implementazioni delle operazioni sono le stesse degli heap di Fibonacci, con la differenza che l'inserimento e l'unione effettuano il consolidamento come loro ultimo passo. Quali sono i tempi di esecuzione nel caso peggiore delle operazioni con gli heap di McGee?

19-3 Altre operazioni con gli heap di Fibonacci

Vogliamo estendere un heap di Fibonacci H in modo da supportare due nuove operazioni, senza cambiare il tempo di esecuzione ammortizzato delle altre operazioni degli heap di Fibonacci.

a. L'operazione FIB-HEAP-CHANGE-KEY(H, x, k) cambia la chiave di x assegnandole il valore k. Sviluppate un'implementazione efficiente di FIB-HEAP-CHANGE-KEY e analizzate il tempo di esecuzione ammortizzato di questa implementazione nei casi in cui k sia maggiore, minore o uguale a $x.key$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 , Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

b. Sviluppate un'implementazione efficiente di FIB-HEAP-PRUNE(H, r) che cancella $q = \min(r, H.n)$ nodi da H. I nodi da cancellare dovrebbero essere arbitrari. Analizzate il tempo di esecuzione ammortizzato della vostra implementazione (*suggerimento:* potrebbe essere necessario modificare la struttura dati e la funzione potenziale).

19-4 Heap 2-3-4

Il Capitolo 18 ha introdotto gli alberi 2-3-4, dove ogni nodo interno (tranne la radice) ha due, tre o quattro figli e tutte le foglie hanno la stessa profondità. In questo problema vogliamo implementare un ***heap 2-3-4*** che supporta le operazioni degli heap riunibili.

Gli heap 2-3-4 differiscono dagli alberi 2-3-4 per i seguenti motivi. Negli heap 2-3-4, soltanto le foglie possono memorizzare le chiavi e ogni foglia x memorizza una sola chiave nell'attributo $x.key$. Non c'è un particolare ordinamento delle chiavi nelle foglie. Ogni nodo interno x contiene un valore $x.small$ che è uguale alla chiave più piccola memorizzata in una foglia qualsiasi nel sottoalbero con radice in x. La radice r contiene un attributo $r.height$ che è l'altezza dell'albero. Infine, gli heap 2-3-4 sono progettati per essere mantenuti nella memoria principale, quindi non occorre eseguire operazioni di lettura e scrittura su disco.

Implementate le seguenti operazioni con gli heap 2-3-4. Ciascuna delle operazioni nei punti (a)–(e) deve essere eseguita nel tempo $O(\lg n)$ con un heap 2-3-4 di n elementi. L'operazione UNION nel punto (f) deve essere eseguita nel tempo $O(\lg n)$, dove n è il numero di elementi nei due heap di input.

a. MINIMUM: restituisce un puntatore alla foglia che ha la chiave più piccola.

b. DECREASE-KEY: diminuisce la chiave di una foglia x, assegnando alla chiave un valore $k \leq x.key$.

c. INSERT: inserisce una foglia x con chiave k.

d. DELETE: cancella una foglia x.

e. EXTRACT-MIN: estrae la foglia che ha la chiave più piccola.

f. UNION: unisce due heap 2-3-4, restituendo un unico heap 2-3-4 e distruggendo gli heap di input.

Note

Gli heap di Fibonacci sono stati introdotti da Fredman e Tarjan [115]. Il loro articolo descrive anche l'applicazione degli heap di Fibonacci ai problemi dei cammini minimi da sorgente unica, dei cammini minimi fra tutte le coppie, di matching pesato su grafo bipartito e dell'albero di connessione minimo.

Successivamente, Driscoll, Gabow, Shrairman e Tarjan [97] hanno sviluppato gli "heap rilassati" come alternativa agli heap di Fibonacci. Ci sono due tipi di heap rilassati. Uno ha gli stessi limiti sui tempi ammortizzati degli heap di Fibonacci. L'altro tipo consente di eseguire DECREASE-KEY nel tempo $O(1)$ nel caso peggiore (non ammortizzato) e EXTRACT-MIN e DELETE nel tempo $O(\lg n)$ nel caso peggiore. Gli heap rilassati offrono altri vantaggi rispetto agli heap di Fibonacci negli algoritmi paralleli.

Consultate anche le note del Capitolo 6 per altre strutture dati che supportano operazioni DECREASE-KEY veloci quando la sequenza dei valori restituiti dalle chiamate di EXTRACT-MIN è monotonicamente crescente nel tempo e i dati sono numeri interi in un determinato intervallo.

Acquistato da Michele Vitolo (michele.vitolo@gmail.com) il 2023-07-05 22:21 tramite Ordine Professionisti 1955 (2023-07-03) con licenza Information Disclosure Statement Evaluation (Italy)

Alberi di van Emde Boas 20

Nei precedenti capitoli abbiamo visto le strutture dati che supportano le operazioni di una coda di priorità – gli heap binari nel Capitolo 6, gli alberi rosso-neri nel Capitolo 13,[1] e gli heap di Fibonacci nel Capitolo 19. In ciascuna di queste strutture dati, almeno un'operazione importante richiede un tempo $O(\lg n)$, nel caso peggiore o in quello ammortizzato. Infatti, poiché ciascuna di queste strutture dati basa le sue decisioni sul confronto di chiavi, il limite inferiore $\Omega(n \lg n)$ sull'ordinamento visto nel Paragrafo 8.1 ci dice che almeno un'operazione deve richiedere un tempo $\Omega(\lg n)$. Perché? Se potessimo eseguire le operazioni INSERT ed EXTRACT-MIN nel tempo $o(\lg n)$, allora potremmo ordinare n chiavi nel tempo $o(n \lg n)$ eseguendo prima n operazioni INSERT e poi n operazioni EXTRACT-MIN.

Nel Capitolo 8 abbiamo visto, però, che a volte è possibile sfruttare delle informazioni aggiuntive sulle chiavi per eseguire l'ordinamento nel tempo $o(n \lg n)$. In particolare, con l'algoritmo counting sort è possibile ordinare n chiavi (ciascuna delle quali è un intero nell'intervallo da 0 a k) nel tempo $\Theta(n + k)$, che è $\Theta(n)$ quando $k = O(n)$.

Poiché possiamo eludere il limite inferiore $\Omega(n \lg n)$ sull'ordinamento quando le chiavi sono numeri interi in un intervallo limitato, vorremmo sapere se sia possibile eseguire ciascuna delle operazioni di una coda di priorità nel tempo $o(\lg n)$ in una simile situazione. In questo capitolo, vedremo che ciò è possibile: gli alberi di van Emde Boas supportano le operazioni della coda di priorità, e poche altre ancora, ciascuna nel tempo $O(\lg \lg n)$ nel caso peggiore. La limitazione è che le chiavi devono essere numeri interi nell'intervallo da 0 a $n - 1$ e non sono ammessi duplicati.

Più precisamente, gli alberi di van Emde Boas supportano le operazioni sugli insiemi dinamici elencate a pagina 190 – SEARCH, INSERT, DELETE, MINIMUM, MAXIMUM, SUCCESSOR e PREDECESSOR – nel tempo $O(\lg \lg n)$. In questo capitolo, ometteremo la discussione sui dati satelliti, concentrandoci soltanto sulla memorizzazione delle chiavi. Dato che consideriamo soltanto le chiavi e che non è ammesso memorizzare chiavi duplicate, anziché descrivere l'operazione SEARCH, implementeremo l'operazione più semplice MEMBER(S, x), che restituisce un valore booleano che indica se x si trova nell'insieme dinamico S.

Finora, abbiamo utilizzato il parametro n per due scopi distinti: il numero di elementi nell'insieme dinamico e l'intervallo dei valori ammissibili. Per evitare possibili confusioni, da questo momento utilizzeremo n per indicare il numero

Acquistato da Michele Mastrolia (mastrolia@gmail.com) - Distribuito da Pearson Italia - Tutti i diritti riservati - © 2022 McGraw-Hill Education (Italy)

[1] Il Capitolo 13 non tratta esplicitamente come implementare EXTRACT-MIN e DECREASE-KEY, ma possiamo facilmente costruire queste operazioni per qualsiasi struttura dati che supporta le operazioni MINIMUM, DELETE e INSERT.

di elementi che si trovano nell'insieme e u per l'intervallo dei valori ammissibili, quindi ciascuna operazione dell'albero di van Emde Boas viene eseguita nel tempo $O(\lg \lg u)$. Chiamiamo ***universo*** l'insieme $\{0, 1, 2, \ldots, u-1\}$ dei valori che possono essere memorizzati e u la ***dimensione dell'universo***. Supporremo in questo capitolo che u sia una potenza esatta di 2, ovvero $u = 2^k$ per qualche intero $k \geq 1$.

Il Paragrafo 20.1 presenta alcuni semplici metodi che ci metteranno sulla direzione giusta. Perfezioneremo questi metodi nel Paragrafo 20.2, presentando le protostrutture di van Emde Boas, che sono ricorsive, ma non raggiungono l'obiettivo di eseguire le operazioni nel tempo $O(\lg \lg u)$. Il Paragrafo 20.3 modifica le protostrutture di van Emde Boas per sviluppare gli alberi di van Emde Boas, e mostra come implementare ciascuna operazione nel tempo $O(\lg \lg u)$.

20.1 Metodi preliminari

In questo paragrafo esamineremo vari metodi per memorizzare un insieme dinamico. Sebbene nessuno di essi raggiunga il limite di tempo $O(\lg \lg u)$ che desideriamo, tuttavia essi saranno la base per capire gli alberi di van Emde Boas quando li descriveremo in questo capitolo.

Indirizzamento diretto

L'indirizzamento diretto, come visto nel Capitolo 11.1, fornisce il metodo più semplice per memorizzare un insieme dinamico. Poiché in questo capitolo siamo interessati soltanto alla memorizzazione delle chiavi, possiamo semplificare il metodo dell'indirizzamento diretto per memorizzare l'insieme dinamico come un vettore di bit (descritto nell'Esercizio 11.1-2). Per memorizzare un insieme dinamico di valori dell'universo $\{0, 1, 2, \ldots, u-1\}$, manteniamo un array $A[0\,..\,u-1]$ di u bit. L'elemento $A[x]$ è 1 se il valore x appartiene all'insieme dinamico, altrimenti è 0. Sebbene possiamo eseguire ciascuna delle operazioni INSERT, DELETE e MEMBER nel tempo $O(1)$ con un vettore di bit, ciascuna delle restanti operazioni – MINIMUM, MAXIMUM, SUCCESSOR e PREDECESSOR – richiede un tempo $\Theta(u)$ nel caso peggiore, perché potrebbe essere necessario analizzare $\Theta(u)$ elementi.[2] Per esempio, se un insieme dinamico contiene soltanto i valori 0 e $u-1$, allora per trovare il successore di 0, dovremmo analizzare gli elementi da 1 a $u-2$ prima di trovare un 1 in $A[u-1]$.

Sovrapporre un albero binario

È possibile abbreviare le lunghe scansioni nel vettore di bit sovrapponendogli un albero binario di bit. La Figura 20.1 illustra un esempio. Gli elementi del vettore di bit formano le foglie dell'albero binario e ogni nodo interno contiene un 1 se e soltanto se qualche foglia nel suo sottoalbero contiene un 1. In altre parole, il bit memorizzato in un nodo interno è l'OR logico dei suoi due figli.

Acquistato da Michele Michele su Webster il 2022-0[illegible] Copyright © 2010 The McGraw-Hill Companies, Srl (Italy) - McGraw-Hill Education (Italy)

[2] In questo capitolo supponiamo che MINIMUM e MAXIMUM restituiscano NIL se l'insieme dinamico è vuoto e che SUCCESSOR e PREDECESSOR restituiscano NIL se l'elemento che queste procedure ricevono non hanno, rispettivamente, un successore o un predecessore.

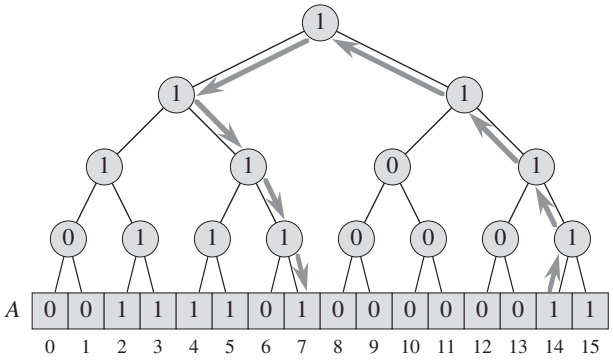


Figura 20.1 Un albero binario di bit sovrapposto a un vettore di bit che rappresenta l'insieme $\{2, 3, 4, 5, 7, 14, 15\}$ quando $u = 16$. Ogni nodo interno contiene un 1 se e soltanto se qualche foglia nel suo sottoalbero contiene un 1. Le frecce mostrano il cammino seguito per determinare il predecessore di 14 nell'insieme.

Le operazioni che richiedevano un tempo $\Theta(u)$ nel caso peggiore con un semplice vettore di bit adesso usano la struttura ad albero:

- Per trovare il valore minimo nell'insieme, iniziamo dalla radice e scendiamo verso le foglie, prendendo sempre il nodo più a sinistra che contiene un 1.
- Per trovare il valore massimo nell'insieme, iniziamo dalla radice e scendiamo verso le foglie, prendendo sempre il nodo più a destra che contiene un 1.
- Per trovare il successore di x, iniziamo dalla foglia con indice x e risaliamo verso la radice finché non entreremo da sinistra in un nodo che ha un 1 nel suo figlio destro z. Poi scendiamo dal nodo z, prendendo sempre il nodo più a sinistra che contiene un 1 (ovvero troviamo il valore minimo nel sottoalbero con radice nel figlio destro z).
- Per trovare il predecessore di x, iniziamo dalla foglia con indice x e risaliamo verso la radice finché non entreremo da destra in un nodo che ha un 1 nel suo figlio sinistro z. Poi scendiamo dal nodo z, prendendo sempre il nodo più a destra che contiene un 1 (ovvero troviamo il valore massimo nel sottoalbero con radice nel figlio sinistro z).

La Figura 20.1 mostra il cammnino seguito per trovare il predecessore, 7, del valore 14.

Aumentiamo in modo appropriato anche le operazioni INSERT e DELETE. Quando inseriamo un valore, memorizziamo un 1 in ciascun nodo sul cammino semplice che va dalla foglia appropriata fino alla radice. Quando cancelliamo un valore, partiamo dalla foglia appropriata e raggiungiamo la radice, ricalcolando il bit in ogni nodo interno del cammino come l'OR logico dei suoi due figli.

Poiché l'altezza dell'albero è $\lg u$ e ciascuna delle precedenti operazioni compie al più un attraversamento verso l'alto dell'albero e al più un attraversamento verso il basso, ciascuna operazione richiede un tempo $O(\lg u)$ nel caso peggiore.

Questo approccio è soltanto marginalmente migliore di quello che usa semplicemente un albero rosso-nero. Possiamo ugualmente eseguire l'operazione MEMBER nel tempo $O(1)$, mentre le ricerche in un albero rosso-nero richiedono un tempo $O(\lg n)$. Quindi, se il numero n di elementi memorizzati fosse molto più piccolo della dimensione u dell'universo, un albero rosso-nero sarebbe più veloce per tutte le altre operazioni.

Acquistato da Michele Mazzucco (mazzucco@gmail.com) - Vietata la riproduzione, anche parziale, non autorizzata - Copyright © 2010 McGraw-Hill Education (Italy)

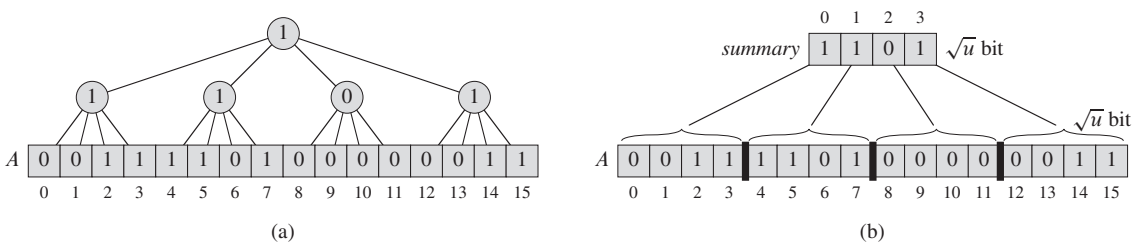


Figura 20.2 **(a)** Un albero di grado $\sqrt{u}$ sovrapposto allo stesso vettore di bit della Figura 20.1. Ogni nodo interno memorizza l'OR logico dei bit nel suo sottoalbero. **(b)** Una vista della stessa struttura, ma con i nodi interni alla profondità 1 trattati come un array $summary[0\,..\,\sqrt{u}-1]$, dove $summary[i]$ è l'OR logico del sottoarray $A[i\sqrt{u}\,..\,(i+1)\sqrt{u}-1]$.

Sovrapporre un albero di altezza costante

Che cosa accade se sovrapponiamo un albero con un grado maggiore? Supponiamo che la dimensione dell'universo sia $u = 2^{2k}$ per qualche intero k, in modo che $\sqrt{u}$ sia un intero. Anziché sovrapporre un albero binario al vettore di bit, sovrapponiamo un albero di grado $\sqrt{u}$. La Figura 20.2(a) mostra tale albero per lo stesso vettore di bit della Figura 20.1. L'altezza dell'albero risultante è sempre 2.

Come prima, ogni nodo interno memorizza l'OR logico dei bit nel suo sottoalbero, cosicché i $\sqrt{u}$ nodi interni alla profondità 1 riassumono ciascun gruppo di $\sqrt{u}$ valori. Come mostra la Figura 20.2(b), possiamo pensare a questi nodi come un array $summary[0\,..\,\sqrt{u}-1]$, dove $summary[i]$ contiene un 1 se e soltanto se il sottoarray $A[i\sqrt{u}\,..\,(i+1)\sqrt{u}-1]$ contiene un 1. Questo sottoarray di A di $\sqrt{u}$ bit è detto i-esimo ***cluster***. Per un dato valore di x, il bit $A[x]$ appare nel cluster numero $\lfloor x/\sqrt{u} \rfloor$. Adesso l'operazione INSERT viene eseguita nel tempo $O(1)$: per inserire x, $A[x]$ e $summary[\lfloor x/\sqrt{u} \rfloor]$ vengono impostati a 1. Possiamo utilizzare l'array *summary* per eseguire ciascuna delle operazioni MINIMUM, MAXIMUM, SUCCESSOR, PREDECESSOR e DELETE nel tempo $O(\sqrt{u})$:

- Per trovare il valore minimo (massimo), cerchiamo l'elemento più a sinistra (destra) in *summary* che contiene un 1, diciamo sia $summary[i]$, e poi effettuiamo una ricerca lineare all'interno dell'i-esimo cluster per trovare l'1 più a sinistra (destra).
- Per trovare il successore (predecessore) di x, prima cerchiamo a destra (sinistra) nel suo cluster. Se troviamo un 1, questa posizione è il risultato; altrimenti, poniamo $i = \lfloor x/\sqrt{u} \rfloor$ e ricerchiamo a destra (sinistra) nell'array *summary* partendo dall'indice i. La prima posizione che contiene un 1 fornisce l'indice di un cluster. Ricerchiamo in questo cluster l'1 più a sinistra (destra); la corrispondente posizione rappresenta il successore (predecessore).
- Per cancellare il valore x, poniamo $i = \lfloor x/\sqrt{u} \rfloor$. Impostiamo prima $A[x]$ a 0 e poi $summary[i]$ all'OR logico dei bit nell'i-esimo cluster.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

In ciascuna delle precedenti operazioni, le ricerche avvengono al più in due cluster di $\sqrt{u}$ bit e nell'array *summary*; quindi ciascuna operazione richiede un tempo $O(\sqrt{u})$.

A prima vista, sembra che abbiamo fatto un progresso all'incontrario. Sovrapponendo un albero binario, abbiamo ottenuto delle operazioni con tempo $O(\lg u)$, che sono asintoticamente più veloci di quelle con tempo $O(\sqrt{u})$. Tuttavia, l'u-

tilizzo di un albero di grado $\sqrt{u}$ è l'idea chiave degli alberi di van Emde Boas. Riprenderemo questo concetto nel prossimo paragrafo.

Esercizi

20.1-1
Modificate le strutture dati di questo paragrafo per supportare le chiavi duplicate.

20.1-2
Modificate le strutture dati di questo paragrafo per supportare le chiavi che sono associate a dati satelliti.

20.1-3
Si noti che, utilizzando le strutture di questo paragrafo, il modo in cui vengono trovati il successore e il predecessore di un valore x non dipende dal fatto che x si trovi nell'insieme. Spiegate come trovare il successore di x in un albero binario di ricerca quando x non è memorizzato nell'albero.

20.1-4
Si supponga che, al posto di un albero di grado $\sqrt{u}$, si voglia sovrapporre un albero di grado $u^{1/k}$, dove $k > 1$ è una costante. Quale sarà l'altezza di tale albero? Quale sarà la durata di ciascuna operazione?

20.2 Una struttura ricorsiva

In questo paragrafo, modificheremo il concetto di sovrapporre un albero di grado $\sqrt{u}$ a un vettore di bit. Nel precedente paragrafo, abbiamo utilizzato una struttura di sintesi (summary) di dimensione $\sqrt{u}$, con ciascun elemento che punta a un'altra struttura (cluster) di dimensione $\sqrt{u}$. Adesso, rendiamo la struttura ricorsiva, riducendo la dimensione dell'universo alla sua radice quadrata a ogni livello di ricorsione. Partendo da un universo di dimensione u, creiamo delle strutture che contengono $\sqrt{u} = u^{1/2}$ elementi, che a loro volta contengono strutture di $u^{1/4}$ elementi, che a loro volta contengono strutture di $u^{1/8}$ elementi, e così via, fino alla dimensione base di 2.

Per semplicità, in questo paragrafo, supponiamo che $u = 2^{2^k}$ per qualche intero k, in modo che $u, u^{1/2}, u^{1/4}, \ldots$ siano interi. Questa ipotesi potrebbe risultare molto restrittiva nella pratica, ammettendo soltanto quei valori di u che appartengono alla sequenza $2, 4, 16, 256, 65536, \ldots$. Vedremo nel prossimo paragrafo come attenuare questa ipotesi restrittiva, supponendo soltanto che $u = 2^k$ per qualche intero k. Poiché la struttura che esaminiamo in questo paragrafo è soltanto un'anticipazione della struttura ad albero di van Emde Boas, tollereremo questa restrizione per agevolare la nostra spiegazione.

Ricordando che il nostro obiettivo è ottenere operazioni con tempi di esecuzione di $O(\lg \lg u)$, vediamo come possiamo raggiungere questo obiettivo. Alla fine del Paragrafo 4.3, abbiamo visto che cambiando le variabili, possiamo dimostrare che la ricorrenza

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$T(n) = 2T\left(\lfloor \sqrt{n} \rfloor\right) + \lg n \tag{20.1}$$

ha la soluzione $T(n) = O(\lg n \lg \lg n)$. Consideriamo una ricorrenza simile, ma più semplice:

$$T(u) = T(\sqrt{u}) + O(1) \tag{20.2}$$

Se utilizziamo la stessa tecnica, cambiando le variabili, possiamo dimostrare che la ricorrenza (20.2) ha la soluzione $T(u) = O(\lg \lg u)$. Ponendo $m = \lg u$, ovvero $u = 2^m$, abbiamo

$$T(2^m) = T(2^{m/2}) + O(1)$$

Adesso, se poniamo $S(m) = T(2^m)$, otteniamo la nuova ricorrenza

$$S(m) = S(m/2) + O(1)$$

Per il caso 2 del metodo dell'esperto, questa ricorrenza ha la soluzione $S(m) = O(\lg m)$. Ritornando da $S(m)$ a $T(u)$, otteniamo $T(u) = T(2^m) = S(m) = O(\lg m) = O(\lg \lg u)$.

La ricorrenza (20.2) guiderà la nostra ricerca di una struttura dati. Progetteremo una struttura dati ricorsiva che si riduce di un fattore $\sqrt{u}$ a ogni livello della sua ricorsione. Quando un'operazione attraversa questa struttura dati, impiegherà una quantità costante di tempo a ogni livello prima di passare al livello successivo. La ricorrenza (20.2) caratterizzerà quindi il tempo di esecuzione dell'operazione.

Ecco un altro modo di vedere come si arrivi al termine $\lg \lg u$ nella soluzione della ricorrenza (20.2). Se consideriamo la dimensione dell'universo a ogni livello della struttura dati ricorsiva, abbiamo la sequenza $u, u^{1/2}, u^{1/4}, u^{1/8}, \ldots$. Se consideriamo quanti bit occorrono per memorizzare la dimensione dell'universo a ogni livello, occorrono $\lg u$ bit al primo livello, e ogni livello richiede metà bit del precedente. In generale, se iniziamo con b bit e dimezziamo il numero di bit a ogni livello, allora dopo $\lg b$ livelli, resterà un solo bit. Poiché $b = \lg u$, vediamo che dopo $\lg \lg u$ livelli, abbiamo una dimensione dell'universo pari a 2.

Riesaminando la struttura dati della Figura 20.2, vediamo che un determinato valore x si trova nel cluster numero $\lfloor x/\sqrt{u} \rfloor$. Se consideriamo x come un intero binario di $\lg u$ bit, quel numero di cluster, $\lfloor x/\sqrt{u} \rfloor$, è dato dai $(\lg u)/2$ bit più significativi di x. All'interno del suo cluster, x si trova nella posizione $x \bmod \sqrt{u}$, che è data dai $(\lg u)/2$ bit meno significativi di x. Poiché conviene utilizzare questo tipo di indicizzazione, definiamo alcune funzioni che ci aiuteranno a farlo:

$$\begin{aligned} \text{high}(x) &= \lfloor x/\sqrt{u} \rfloor \\ \text{low}(x) &= x \bmod \sqrt{u} \\ \text{index}(x, y) &= x\sqrt{u} + y \end{aligned}$$

La funzione $\text{high}(x)$ fornisce i $(\lg u)/2$ bit più significativi di x che rappresentano il numero di cluster di x. La funzione $\text{low}(x)$ fornisce i $(\lg u)/2$ bit meno significativi di x e che rappresentano la posizione di x all'interno del suo cluster. La funzione $\text{index}(x, y)$ costruisce il numero di un elemento da x e y, trattando x come i $(\lg u)/2$ bit più significativi del numero dell'elemento e y come i $(\lg u)/2$ bit meno significativi. Abbiamo l'identità $x = \text{index}(\text{high}(x), \text{low}(x))$. Il valore di u utilizzato da ciascuna di queste funzioni sarà sempre la dimensione dell'universo della struttura dati in cui chiamiamo la funzione, che cambia mentre scendiamo nella struttura ricorsiva.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

20.2.1 Protostrutture di van Emde Boas

Prendendo spunto dalla ricorrenza (20.2), progettiamo una struttura dati ricorsiva che supporta le operazioni. Sebbene questa struttura dati non ci permetta di raggiungere l'obiettivo del tempo $O(\lg \lg u)$ per alcune operazioni, essa servirà da base per la struttura ad albero di van Emde Boas che esamineremo nel Paragrafo 20.3.

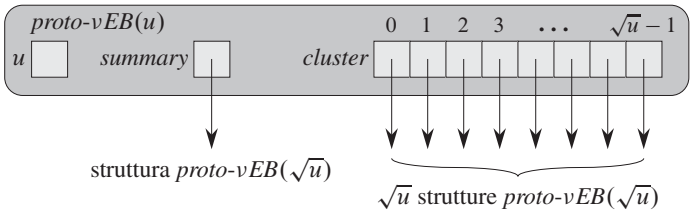


Figura 20.3 Le informazioni in una struttura *proto-vEB*(u) quando $u \geq 4$. La struttura contiene la dimensione dell'universo u, un puntatore *summary* a una struttura *proto-vEB*$(\sqrt{u})$ e un array *cluster*$[0\,..\,\sqrt{u}-1]$ di $\sqrt{u}$ puntatori alle strutture *proto-vEB*$(\sqrt{u})$.

Per l'universo $\{0, 1, 2, \ldots, u-1\}$, definiamo una ***protostruttura di van Emde Boas*** o ***protostruttura vEB***, che indichiamo con *proto-vEB*(u), in modo ricorsivo come segue. Ogni struttura *proto-vEB*(u) contiene un attributo u che fornisce la dimensione del suo universo. Essa contiene anche le seguenti informazioni:

- Se $u = 2$, allora questa è la dimensione base, e contiene un array $A[0\,..\,1]$ di due bit.
- Altrimenti, $u = 2^{2^k}$ per qualche intero $k \geq 1$, per cui $u \geq 4$. Oltre alla dimensione dell'universo u, la struttura *proto-vEB*(u) contiene i seguenti attributi, illustrati nella Figura 20.3:
  - un puntatore chiamato *summary* a una struttura *proto-vEB*$(\sqrt{u})$
  - un array *cluster*$[0\,..\,\sqrt{u}-1]$ di $\sqrt{u}$ puntatori, ciascuno a una struttura *proto-vEB*$(\sqrt{u})$.

L'elemento x, dove $0 \leq x < u$, viene memorizzato in modo ricorsivo come elemento low(x) nel cluster numero high(x).

Nella struttura a due livelli del precedente paragrafo, ciascun nodo memorizza un array di dimensione $\sqrt{u}$, in cui ogni elemento contiene un bit. Dall'indice di ciascun elemento, possiamo calcolare l'indice iniziale del sottoarray di dimensione $\sqrt{u}$ che il bit sintetizza. Nella struttura proto-vEB, utilizziamo puntatori espliciti, anziché indici calcolati. L'array *summary* contiene i bit di sintesi memorizzati in modo ricorsivo in una struttura proto-vEB, e l'array *cluster* contiene $\sqrt{u}$ puntatori.

La Figura 20.4 mostra una struttura *proto-vEB*(16) completamente espansa, che rappresenta l'insieme $\{2, 3, 4, 5, 7, 14, 15\}$. Se il valore i si trova nella struttura proto-vEB puntata da *summary*, allora l'i-esimo cluster contiene qualche valore dell'insieme rappresentato. Come nell'albero ad altezza costante, *cluster*$[i]$ rappresenta i valori da $i\sqrt{u}$ a $(i+1)\sqrt{u}-1$, che formamo l'i-esimo cluster.

Al livello base, gli elementi degli insiemi dinamici sono memorizzati in qualche struttura *proto-vEB*(2); nelle restanti strutture *proto-vEB*(2) sono memorizzati i bit di sintesi. Sotto ciascuna delle strutture base non-summary, la figura indica quali bit memorizza. Per esempio, la struttura *proto-vEB*(2) indicata con "elementi 6, 7" memorizza il bit 6 (0, perché l'elemento 6 non è nell'insieme) nel suo $A[0]$ e il bit 7 (1, perché l'elemento 7 è nell'insieme) nel suo $A[1]$.

Come i cluster, ogni summary è un insieme dinamico con la dimensione dell'universo $\sqrt{u}$; quindi possiamo rappresentare ciascun summary come una struttura *proto-vEB*$(\sqrt{u})$. I quattro bit di sintesi per la struttura principale *proto-vEB*(16) si trovano nella struttura *proto-vEB*(4) più a sinistra e alla fine appaiono in due strutture *proto-vEB*(2). Per esempio, la struttura *proto-vEB*(2) indicata con "cluster 2, 3" ha $A[0] = 0$, che significa che il cluster 2 della struttura *proto-vEB*(16) (che contiene gli elementi $8, 9, 10, 11$) è formato da soli 0, e $A[1] = 1$, che significa che il cluster 3 (che contiene gli elementi $12, 13, 14, 15$) ha almeno un 1.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

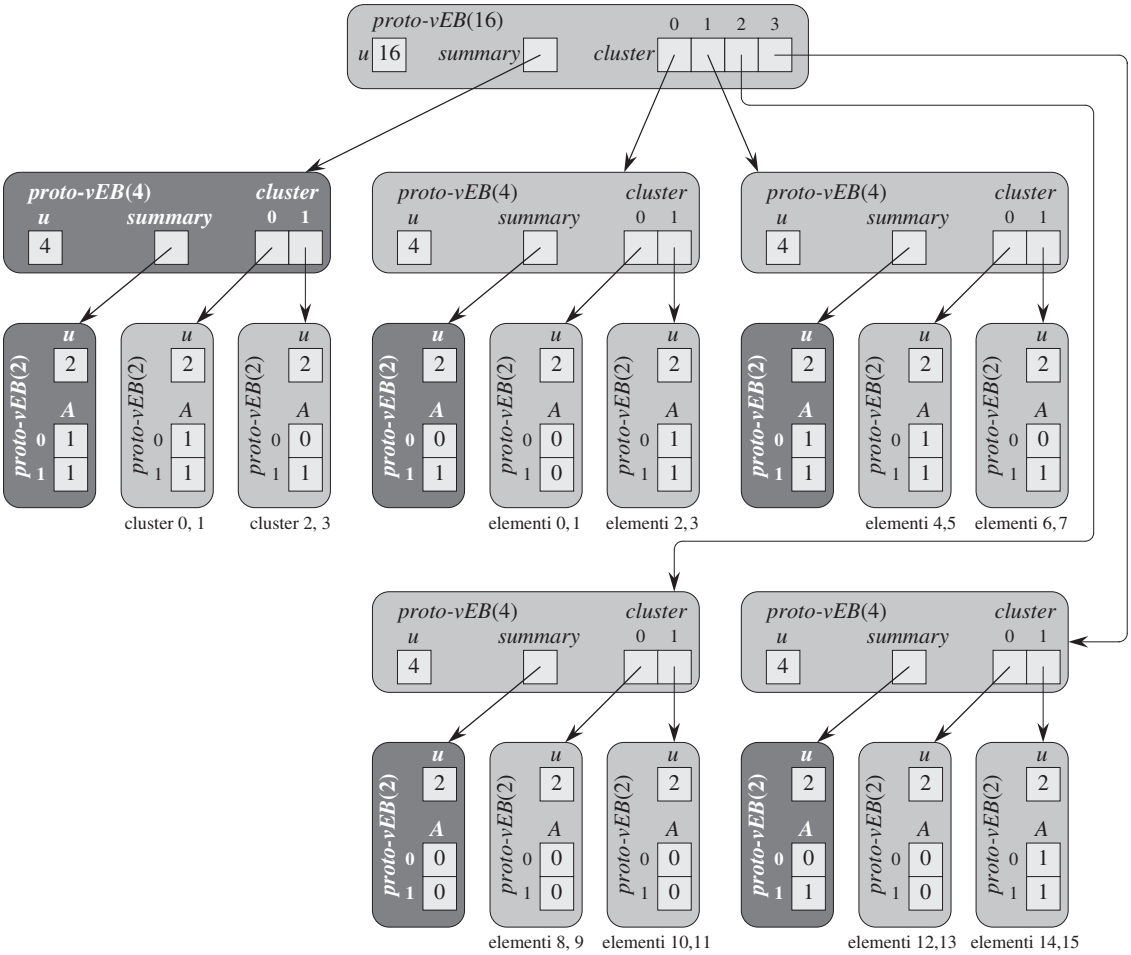


Figura 20.4 Una struttura *proto-vEB*(16) che rappresenta l'insieme $\{2, 3, 4, 5, 7, 14, 15\}$. Essa punta a quattro strutture *proto-vEB*(4) nei *cluster*$[0\,..\,3]$ e a una struttura *summary*, che è anche una struttura *proto-vEB*(4). Ciascuna struttura *proto-vEB*(4) punta a due strutture *proto-vEB*(2) nei *cluster*$[0\,..\,1]$ e a una struttura *summary*, che è anche *proto-vEB*(2). Ciascuna struttura *proto-vEB*(2) contiene un array $A[0\,..\,1]$ di due bit. Le strutture *proto-vEB*(2) sopra gli "elementi i, j" memorizzano i bit i e j dell'insieme dinamico; le strutture *proto-vEB*(2) sopra i "cluster i, j" memorizzano i bit della struttura *summary* per i cluster i e j nella struttura *proto-vEB*(16) di livello superiore. Per chiarezza, il colore grigio scuro indica il livello superiore di una struttura proto-vEB che memorizza le informazioni di *summary* per la sua struttura madre; per il resto, tale struttura proto-vEB è identica a qualsiasi altra struttura proto-vEB avente la stessa dimensione dell'universo.

Ciascuna struttura *proto-vEB*(4) punta al suo summary, che è memorizzato come una struttura *proto-vEB*(2). Per esempio, osservate la struttura *proto-vEB*(2) a sinistra di quella indicata con "elementi 0, 1". Poiché il suo $A[0]$ è 0, la struttura "elementi 0, 1" è formata da soli 0, e poiché il suo $A[1]$ è 1, sappiamo che la struttura "elementi 2, 3" contiene almeno un 1.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

20.2.2 Operazioni con una protostruttura di van Emde Boas

Adesso descriviamo come eseguire le operazioni con una struttura proto-vEB. Esaminiamo prima le operazioni di interrogazione – MEMBER, MINIMUM, MAXIMUM e SUCCESSOR – che non cambiano la struttura proto-vEB. Poi esaminere-

mo le operazioni INSERT e DELETE. L'Esercizio 20.2-1 vi chiederà di scrivere lo pseudocodice di MAXIMUM e PREDECESSOR, che sono operazioni simmetriche, rispettivamente, a MINIMUM e SUCCESSOR.

Ciascuna delle operazioni MEMBER, SUCCESSOR, PREDECESSOR, INSERT e DELETE riceve un parametro x e una struttura proto-vEB V. In ciascuna di queste operazioni si suppone che $0 \le x < V.u$.

Determinare se un valore è nell'insieme

Per eseguire MEMBER(x), occorre trovare il bit corrispondente a x all'interno della struttura *proto-vEB*(2) appropriata. Possiamo farlo nel tempo $O(\lg \lg u)$, senza usare le strutture *summary*. La seguente procedura riceve una struttura proto-vEB V e un valore x, e restituisce un bit che indica se x si trova nell'insieme dinamico.

```
PROTO-VEB-MEMBER(V, x)
1  if V.u == 2
2        return V.A[x]
3  else return PROTO-VEB-MEMBER(V.cluster[high(x)], low(x))
```

La procedura PROTO-VEB-MEMBER funziona nel modo seguente. La riga 1 verifica se siamo nel caso base, dove V è una struttura *proto-vEB*(2). La riga 2 gestisce il caso base, restituendo semplicemente il bit appropriato dell'array A. La riga 3 tratta il caso ricorsivo, esaminando una struttura proto-vEB più piccola. Il valore high(x) indica quale struttura *proto-vEB*$(\sqrt{u})$ visitiamo, e low(x) determina quale elemento stiamo cercando all'interno della struttura *proto-vEB*$(\sqrt{u})$.

Vediamo che cosa accade quando chiamiamo PROTO-VEB-MEMBER$(V, 6)$ con la struttura *proto-vEB*(16) della Figura 20.4. Poiché high$(6) = 1$ quando $u = 16$, la ricorsione interessa la struttura *proto-vEB*(4) che si trova più in alto a destra e viene ricercato l'elemento low$(6) = 2$ di tale struttura. Poiché in questa chiamata ricorsiva $u = 4$, la ricorsione si ripete. Con $u = 4$, si ha high$(2) = 1$ e low$(2) = 0$; quindi viene ricercato l'elemento 0 della struttura *proto-vEB*(2) che si trova più in alto a destra. Poiché questa chiamata ricorsiva è un caso base, essa restituisce $A[0] = 0$ attraverso la catena delle chiamate ricorsive. Dunque, il risultato è che PROTO-VEB-MEMBER$(V, 6)$ restituisce 0; ciò indica che 6 non è nell'insieme.

Per determinare il tempo di esecuzione di PROTO-VEB-MEMBER, indichiamo con $T(u)$ il suo tempo di esecuzione con una struttura *proto-vEB*(u). Ciascuna chiamata ricorsiva richiede un tempo costante, escludendo il tempo impiegato dalle chiamate ricorsive che essa fa. Quando PROTO-VEB-MEMBER fa una chiamata ricorsiva, la chiamata riguarda una struttura *proto-vEB*$(\sqrt{u})$. Quindi, possiamo caratterizzare il tempo di esecuzione tramite la ricorrenza $T(u) = T(\sqrt{u}) + O(1)$, che abbiamo già visto come ricorrenza (20.2). La sua soluzione è $T(u) = O(\lg \lg u)$; quindi concludiamo che PROTO-VEB-MEMBER viene eseguita nel tempo $O(\lg \lg u)$.

Acquistato da Michele Videlesa Webster il 2012-07-07 13:30:01 (unlimited) Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Trovare l'elemento minimo

Vediamo adesso come possiamo eseguire l'operazione MINIMUM. La procedura PROTO-VEB-MINIMUM(V) restituisce l'elemento minimo nella struttura proto-vEB V oppure NIL se V è un insieme vuoto.

```
PROTO-VEB-MINIMUM(V)
 1  if V.u == 2
 2       if V.A[0] == 1
 3            return 0
 4       elseif V.A[1] == 1
 5            return 1
 6       else return NIL
 7  else min-cluster = PROTO-VEB-MINIMUM(V.summary)
 8       if min-cluster == NIL
 9            return NIL
10       else offset = PROTO-VEB-MINIMUM(V.cluster[min-cluster])
11            return index(min-cluster, offset)
```

Questa procedura opera nel modo seguente. La riga 1 verifica il caso base, che le righe 2–6 gestiscono tramite un metodo a forza bruta. Le righe 7–11 gestiscono il caso ricorsivo. La riga 7 trova il numero del primo cluster che contiene un elemento dell'insieme. Per fare questo, chiama in modo ricorsivo la procedura PROTO-VEB-MINIMUM su $V.summary$, che è una struttura $proto\text{-}vEB(\sqrt{u})$. La riga 7 assegna questo numero di cluster alla variabile *min-cluster*. Se l'insieme è vuoto, la chiamata ricorsiva ha restituito NIL, e la riga 9 restituisce NIL. Altrimenti, l'elemento minimo dell'insieme è da qualche parte nel cluster numero *min-cluster*. La chiamata ricorsiva nella riga 10 trova l'offset all'interno del cluster dell'elemento minimo. Infine, la riga 11 ricava il valore dell'elemento minimo dal numero del cluster e dall'offset, e restituisce tale valore.

Sebbene le informazioni di summary ci consentano di trovare rapidamente il cluster che contiene l'elemento minimo, poiché questa procedura effettua due chiamate ricorsive sulle strutture $proto\text{-}vEB(\sqrt{u})$, essa non viene eseguita nel tempo $O(\lg \lg u)$ nel caso peggiore. Indicando con $T(u)$ il tempo nel caso peggiore della procedura PROTO-VEB-MINIMUM su una struttura $proto\text{-}vEB(u)$, otteniamo la ricorrenza

$$T(u) = 2T(\sqrt{u}) + O(1) \tag{20.3}$$

Anche qui, cambiamo le variabili per risolvere questa ricorrenza, ponendo $m = \lg u$, che fornisce

$$T(2^m) = 2T(2^{m/2}) + O(1)$$

Ponendo $S(m) = T(2^m)$, si ha

$$S(m) = 2S(m/2) + O(1)$$

la cui soluzione, per il caso 1 del metodo dell'esperto, è $S(m) = \Theta(m)$. Cambiando di nuovo le variabili da $S(m)$ a $T(u)$, si ha $T(u) = T(2^m) = S(m) = \Theta(m) = \Theta(\lg u)$. Quindi, notiamo che, a causa della seconda chiamata ricorsiva, PROTO-VEB-MINIMUM viene eseguita nel tempo $\Theta(\lg u)$, anziché nel tempo desiderato $O(\lg \lg u)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Trovare il successore

L'operazione SUCCESSOR è ancora peggio; infatti, essa effettua due chiamate ricorsive e una chiamata di PROTO-VEB-MINIMUM nel caso peggiore. La procedura PROTO-VEB-SUCCESSOR(V, x) restituisce l'elemento più piccolo nella

struttura proto-vEB V che è maggiore di x, oppure NIL se nessun elemento in V è maggiore di x. Non richiede che x sia un elemento dell'insieme, ma suppone che $0 \leq x < V.u$.

```
PROTO-VEB-SUCCESSOR(V, x)
 1  if V.u == 2
 2       if x == 0 and V.A[1] == 1
 3            return 1
 4       else return NIL
 5  else offset = PROTO-VEB-SUCCESSOR(V.cluster[high(x)], low(x))
 6       if offset ≠ NIL
 7            return index(high(x), offset)
 8       else succ-cluster = PROTO-VEB-SUCCESSOR(V.summary, high(x))
 9            if succ-cluster == NIL
10                 return NIL
11            else offset = PROTO-VEB-MINIMUM(V.cluster[succ-cluster])
12                 return index(succ-cluster, offset)
```

La procedura PROTO-VEB-SUCCESSOR opera nel modo seguente. La riga 1 verifica il caso base, che le righe 2–4 gestiscono tramite un metodo a forza bruta: l'unico caso in cui x può avere un successore all'interno di una struttura *proto-vEB*(2) è quando $x = 0$ e $A[1] = 1$. Le righe 5–12 gestiscono il caso ricorsivo. La riga 5 ricerca un successore di x nel cluster di x, assegnando il risultato a *offset*. La riga 6 controlla se x ha un successore all'interno del suo cluster; se trova il successore, la riga 7 calcola e restituisce il valore di questo successore. Altrimenti, le ricerche devono essere effettuate in altri cluster. La riga 8 assegna a *succ-cluster* il numero del successivo cluster non vuoto, utilizzando le informazioni di summary per trovarlo. La riga 9 controlla se *succ-cluster* è NIL e la riga 10 restituisce NIL se tutti i successivi cluster sono vuoti. Se *succ-cluster* non è NIL, la riga 11 assegna a *offset* il primo elemento all'interno di tale cluster e la riga 12 calcola e restituisce l'elemento minimo di questo cluster.

Nel caso peggiore, PROTO-VEB-SUCCESSOR chiama se stessa in modo ricorsivo due volte sulle strutture *proto-vEB*$(\sqrt{u})$, ed effettua una chiamata di PROTO-VEB-MINIMUM su una struttura *proto-vEB*$(\sqrt{u})$. Quindi, la ricorrenza per il tempo di esecuzione $T(u)$ nel caso peggiore di PROTO-VEB-SUCCESSOR è

$$\begin{aligned} T(u) &= 2T(\sqrt{u}) + \Theta(\lg \sqrt{u}) \\ &= 2T(\sqrt{u}) + \Theta(\lg u) \end{aligned}$$

Possiamo applicare la stessa tecnica che abbiamo utilizzato per la ricorrenza (20.1) per dimostrare che questa ricorrenza ha la soluzione $T(u) = \Theta(\lg u \lg \lg u)$. Quindi, la procedura PROTO-VEB-SUCCESSOR è asintoticamente più lenta di PROTO-VEB-MINIMUM.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Inserire un elemento

L'inserimento di un elemento deve avvenire nel cluster appropriato; inoltre il bit di summary per tale cluster deve essere impostato a 1. La procedura PROTO-VEB-INSERT(V, x) inserisce il valore x nella struttura proto-vEB V.

```
PROTO-VEB-INSERT(V, x)
1  if V.u == 2
2        V.A[x] = 1
3  else PROTO-VEB-INSERT(V.cluster[high(x)], low(x))
4        PROTO-VEB-INSERT(V.summary, high(x))
```

Nel caso base, la riga 2 imposta a 1 il bit appropriato nell'array A. Nel caso ricorsivo, la chiamata ricorsiva nella riga 3 inserisce x nel cluster appropriato, e la riga 4 imposta a 1 il bit di summary per tale cluster.

Poiché PROTO-VEB-INSERT effettua due chiamate ricorsive nel caso peggiore, la ricorrenza (20.3) caratterizza il suo tempo di esecuzione. Quindi, la procedura PROTO-VEB-INSERT viene eseguita nel tempo $\Theta(\lg u)$.

Cancellare un elemento

L'operazione DELETE è più complessa dell'inserimento. Sebbene sia sempre possibile impostare a 1 un bit di summary durante l'inserimento, non è sempre possibile riportare a 0 lo stesso bit durante la cancellazione. Occorre determinare se qualche bit nel cluster appropriato è 1. Per come abbiamo definito le strutture proto-vEB, dovremmo esaminare tutti i $\sqrt{u}$ bit all'interno di un cluster per determinare quanti di essi sono 1. In alternativa, potremmo aggiungere un attributo n alla struttura proto-vEB, contando gli elementi che essa contiene. Gli Esercizi 20.2-2 e 20.2-3 vi chiederà di scrivere lo pseudocodice di PROTO-VEB-DELETE.

Chiaramente, occorre modificare la struttura proto-vEB per fare in modo che ciascuna operazione effettui al più una chiamata ricorsiva. Nel prossimo paragrafo vedremo come fare.

Esercizi

20.2-1

Scrivete lo pseudocodice per le procedure PROTO-VEB-MAXIMUM e PROTO-VEB-PREDECESSOR.

20.2-2

Scrivete lo pseudocodice per la procedura PROTO-VEB-DELETE. Esso dovrebbe aggiornare l'appropriato bit di summary esaminando i relativi bit del cluster. Qual è il tempo di esecuzione nel caso peggiore della vostra procedura?

20.2-3

Aggiungete l'attributo n a ciascuna struttura proto-vEB, assegnandogli il numero di elementi correntemente presenti nell'insieme che rappresenta; scrivete lo pseudocodice di PROTO-VEB-DELETE che usa l'attributo n per decidere quando riportare a 0 i bit di summary. Qual è il tempo di esecuzione nel caso peggiore della vostra procedura? Quali altre procedure devono essere modificate a causa di questo nuovo attributo? Le modifiche influiscono sui loro tempi di esecuzione?

Acquistato da Michele Michele su Webster il 2022-[illegible]

20.2-4

Modificate la struttura proto-vEB per supportare le chiavi duplicate.

20.2-5

Modificate la struttura proto-vEB per supportare chiavi a cui sono associati dei dati satelliti.

20.2-6
Scrivete lo pseudocodice per una procedura che crea una struttura *proto-vEB*(u).

20.2-7
Dimostrate che se viene eseguita la riga 9 di PROTO-VEB-MINIMUM, la struttura proto-vEB è vuota.

20.2-8
Supponete di progettare una struttura proto-vEB nella quale ogni array *cluster* ha soltanto $u^{1/4}$ elementi. Qual è il tempo di esecuzione di ciascuna operazione?

20.3 L'albero di van Emde Boas

La struttura proto-vEB descritta nel precedente paragrafo è vicina a quella che ci serve per ottenere tempi di esecuzione pari a $O(\lg \lg u)$, ma non raggiunge lo scopo per il numero troppo elevato di ricorsioni necessarie nella maggior parte delle operazioni. In questo paragrafo, progetteremo una struttura dati che è simile alla struttura proto-vEB, ma memorizza qualche informazione in più, eliminando così qualche ricorsione.

Nel Paragrafo 20.2 abbiamo osservato che l'ipotesi fatta sulla dimensione dell'universo – ovvero $u = 2^{2^k}$ per qualche intero k – è eccessivamente restrittiva, perché confina i possibili valori di u a un insieme troppo sparso. Da ora in poi, quindi, ammetteremo che la dimensione dell'universo u sia una potenza esatta di 2, e quando $\sqrt{u}$ non è un intero – ovvero, se u è una potenza dispari di 2 ($u = 2^{2k+1}$ per qualche intero $k \geq 0$) – allora divideremo i $\lg u$ bit di un numero nei $\lceil (\lg u)/2 \rceil$ bit più significativi e nei $\lfloor (\lg u)/2 \rfloor$ bit meno significativi. Per comodità, indichiamo $2^{\lceil (\lg u)/2 \rceil}$ (la "radice quadrata superiore" di u) con $\uparrow\sqrt{u}$ e $2^{\lfloor (\lg u)/2 \rfloor}$ (la "radice quadrata inferiore" di u) con $\downarrow\sqrt{u}$, in modo che $u = \uparrow\sqrt{u} \cdot \downarrow\sqrt{u}$ e, quando u è una potenza pari di 2 ($u = 2^{2k}$ per qualche intero k), $\uparrow\sqrt{u} = \downarrow\sqrt{u} = \sqrt{u}$. Poiché adesso ammettiamo che u possa essere una potenza dispari di 2, dobbiamo ridefinire le funzioni descritte nel Paragrafo 20.2:

$$\begin{aligned} \text{high}(x) &= \left\lfloor x / \downarrow\sqrt{u} \right\rfloor \\ \text{low}(x) &= x \bmod \downarrow\sqrt{u} \\ \text{index}(x, y) &= x \downarrow\sqrt{u} + y \end{aligned}$$

20.3.1 Alberi di van Emde Boas

L'***albero di van Emde Boas*** o ***albero vEB*** modifica la struttura proto-vEB. Denoteremo con $vEB(u)$ un albero vEB con una dimensione dell'universo pari a u e, a meno che u non sia uguale alla dimensione base 2, l'attributo *summary* punta a un albero $vEB(\uparrow\sqrt{u})$ e l'array $cluster[0 \,..\, \uparrow\sqrt{u} - 1]$ punta ai $\uparrow\sqrt{u}$ alberi $vEB(\downarrow\sqrt{u})$. Come mostra la Figura 20.5, un albero vEB contiene due attributi non presenti nelle strutture proto-vEB:

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950301 6-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

- *min* memorizza l'elemento minimo nell'albero vEB
- *max* memorizza l'elemento massimo nell'albero vEB

Inoltre, l'elemento memorizzato in *min* non appare in nessuno degli alberi ricorsivi $vEB(\downarrow\sqrt{u})$ cui punta l'array *cluster*. Gli elementi memorizzati in un albero $vEB(u)$ V, quindi, sono $V.min$ più tutti gli elementi memorizzati in modo ricorsivo negli alberi $vEB(\downarrow\sqrt{u})$ cui punta $V.cluster[0 \,..\, \uparrow\sqrt{u} - 1]$. Notate che, quando

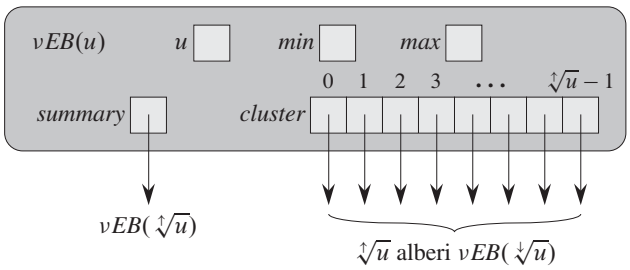


Figura 20.5 Le informazioni in un albero $vEB(u)$ quando $u > 2$. La struttura contiene la dimensione dell'universo u, gli elementi *min* e *max*, un puntatore *summary* a un albero $vEB(\sqrt[\uparrow]{u})$ e un array $cluster[0\,..\,\sqrt[\uparrow]{u} - 1]$ di $\sqrt[\uparrow]{u}$ puntatori agli alberi $vEB(\sqrt[\downarrow]{u})$.

un albero vEB contiene due o più elementi, noi trattiamo *min* e *max* in modo differente: l'elemento memorizzato in *min* non appare in alcun cluster, diversamente dall'elemento memorizzato in *max*.

Poiché la dimensione base è 2, un albero $vEB(2)$ non richiede l'array A che ha la corrispondente struttura *proto-*$vEB(2)$. Piuttosto, possiamo determinare i suoi elementi dagli attributi *min* e *max*. In un albero vEB senza elementi, indipendentemente dalla dimensione u del suo universo, *min* e *max* sono NIL.

La Figura 20.6 mostra un albero $vEB(16)$ V che contiene l'insieme $\{2, 3, 4, 5, 7, 14, 15\}$. Poiché il più piccolo elemento è 2, $V.min$ è uguale a 2, e anche se $\text{high}(2) = 0$, l'elemento 2 non appare nell'albero $vEB(4)$ cui punta $V.cluster[0]$: notate che $V.cluster[0].min$ è uguale a 3, e quindi 2 non è in questo albero vEB. Analogamente, poiché $V.cluster[0].min$ è uguale a 3, e 2 e 3 sono gli unici elementi in $V.cluster[0]$, i cluster $vEB(2)$ all'interno di $V.cluster[0]$ sono vuoti.

Gli attributi *min* e *max* svolgono un ruolo chiave per ridurre il numero di chiamate ricorsive all'interno delle operazioni con gli alberi vEB. Questi attributi ci aiuteranno in quattro modi:

1. Le operazioni MINIMUM e MAXIMUM non richiedono ricorsione, in quanto possono restituire semplicemente i valori di *min* e *max*.
2. L'operazione SUCCESSOR può evitare una chiamata ricorsiva per determinare se il successore di un valore x si trova all'interno di $\text{high}(x)$. Questo perché il successore di x si trova all'interno del suo cluster se e soltanto se x è strettamente minore dell'attributo *max* del suo cluster. Una considerazione analoga vale per PREDECESSOR e *min*.
3. Dai valori di *min* e *max* possiamo dire in un tempo costante se un albero vEB non ha elementi, se ha un solo elemento o almeno due elementi. Questo ci aiuterà nelle operazioni INSERT e DELETE. Se *min* e *max* sono entrambi NIL, allora l'albero vEB non ha elementi. Se *min* e *max* non sono NIL, ma sono uguali, allora l'albero vEB ha esattamente un elemento. Altrimenti, *min* e *max* non sono NIL, ma sono diversi, e l'albero vEB ha due o più elementi.
4. Se sappiamo che un albero vEB è vuoto, possiamo inserire un elemento in esso aggiornando soltanto i suoi attributi *min* e *max*. Quindi, l'inserimento in un albero vEB vuoto può essere fatto in un tempo costante. Analogamente, se sappiamo che un albero vEB ha un solo elemento, possiamo cancellare l'elemento in un tempo costante aggiornando soltanto *min* e *max*. Queste proprietà ci consentiranno di abbreviare la catena delle chiamate ricorsive.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Anche se la dimensione dell'universo u è una potenza dispari di 2, la differenza tra le dimensioni dell'albero vEB del summary e dei cluster non influirà sui tempi di esecuzione asintotici delle operazioni con l'albero vEB. Le procedure ricorsive

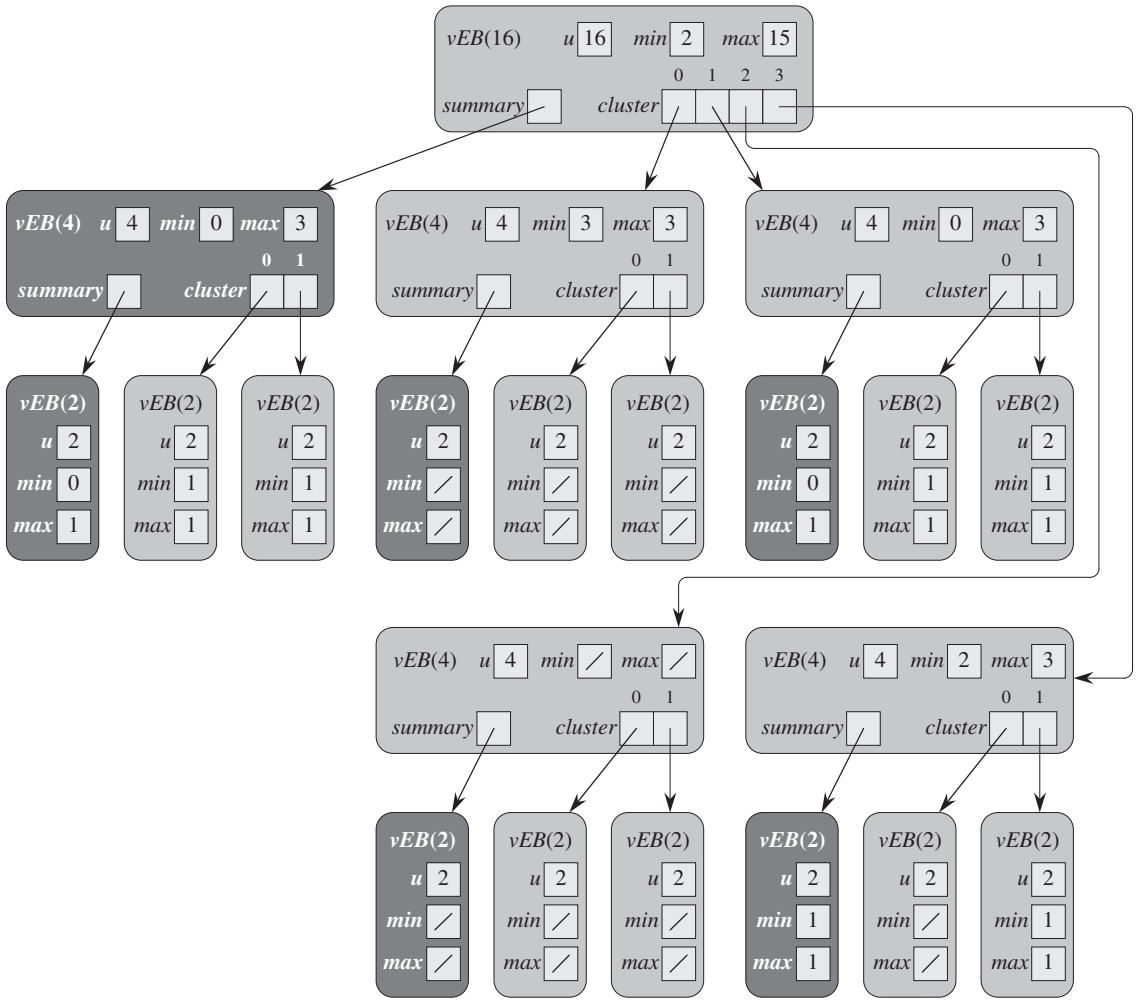


Figura 20.6 Un albero $vEB(16)$ che corrisponde all'albero proto-vEB della Figura 20.4. Esso memorizza l'insieme $\{2, 3, 4, 5, 7, 14, 15\}$. Il simbolo / indica il valore NIL. Il valore memorizzato nell'attributo *min* di un albero vEB non appare in nessuno dei suoi cluster. Il colore grigio scuro svolge lo stesso compito descritto nella Figura 20.4.

che implementano le operazioni con l'albero vEB avranno tempi di esecuzione caratterizzati dalla ricorrenza

$$T(u) \le T(\sqrt[\uparrow]{u}) + O(1) \tag{20.4}$$

Questa ricorrenza somiglia alla ricorrenza (20.2), e la risolveremo in modo simile. Ponendo $m = \lg u$, possiamo riscriverla così

$$T(2^m) \le T(2^{\lceil m/2 \rceil}) + O(1)$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Poiché $\lceil m/2 \rceil \le 2m/3$ per ogni $m \ge 2$, si ha

$$T(2^m) \le T(2^{2m/3}) + O(1)$$

Ponendo $S(m) = T(2^m)$, riscriviamo quest'ultima ricorrenza così

$$S(m) \le S(2m/3) + O(1)$$

che, per il caso 2 del metodo dell'esperto, ha la soluzione $S(m) = O(\lg m)$. (In termini di soluzione asintotica, la frazione $2/3$ non è diversa dalla frazione $1/2$, perché quando si applica il metodo dell'esperto, si ha $\log_{3/2} 1 = \log_2 1 = 0$.) Dunque, abbiamo $T(u) = T(2^m) = S(m) = O(\lg m) = O(\lg \lg u)$.

Prima di utilizzare un albero di van Emde Boas, dobbiamo conoscere la dimensione dell'universo u, in modo da poter creare un albero di van Emde Boas della dimensione appropriata, che inizialmente rappresenta un insieme vuoto. Come il Problema 20-1 vi chiede di dimostrare, lo spazio totale richiesto da un albero di van Emde Boas è $O(u)$, ed è semplice creare un albero vuoto nel tempo $O(u)$. D'altra parte, possiamo creare un albero rosso-nero vuoto in un tempo costante. Quindi, non dovremmo utilizzare un albero di van Emde Boas quando eseguiamo un piccolo numero di operazioni, in quanto il tempo per creare la struttura dati supererebbe il tempo risparmiato nelle singole operazioni. Questo inconveniente di solito non è significativo, in quanto tipicamente utilizziamo una struttura dati semplice, come un array o una lista concatenata, per rappresentare un insieme che contiene soltanto pochi elementi.

20.3.2 Operazioni con un albero di van Emde Boas

A questo punto possiamo vedere come eseguire le operazioni con un albero di van Emde Boas. Come abbiamo fatto con la protostruttura di van Emde Boas, considereremo prima le operazioni di interrogazione, e poi INSERT e DELETE. A causa della piccola asimmetria tra gli elementi minimo e massimo in un albero vEB – quando un albero vEB contiene almeno due elementi, l'elemento minimo non appare all'interno di un cluster, diversamente dall'elemento massimo – forniremo lo pseudocodice per tutte le operazioni di query. Come nelle operazioni con le protostrutture di van Emde Boas, anche qui nelle operazioni che ricevono i parametri V e x, dove V è un albero di van Emde Boas e x è un elemento, si suppone che $0 \le x < V.u$.

Trovare gli elementi minimo e massimo

Poiché il minimo e il massimo sono memorizzati negli attributi *min* e *max*, due delle operazioni sono molto semplici e richiedono un tempo costante:

```
VEB-TREE-MINIMUM(V)
1  return V.min

VEB-TREE-MAXIMUM(V)
1  return V.max
```

Determinare se un valore è nell'insieme

La procedura VEB-TREE-MEMBER(V, x) ha un caso ricorsivo come quello di PROTO-VEB-MEMBER, ma il caso base è un po' diverso. Verifichiamo direttamente se x è uguale al minimo o al massimo. Poiché un albero vEB non memorizza i bit come una protostruttura vEB, progettiamo VEB-TREE-MEMBER per restituire TRUE o FALSE, anziché 1 o 0.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

```
VEB-TREE-MEMBER(V, x)
1  if x == V.min or x == V.max
2       return TRUE
3  elseif V.u == 2
4       return FALSE
5  else return VEB-TREE-MEMBER(V.cluster[high(x)], low(x))
```

La riga 1 verifica se x è uguale all'elemento minimo o massimo. Se lo è, la riga 2 restituisce TRUE; altrimenti, la riga 3 verifica il caso base. Poiché un albero $vEB(2)$ non ha elementi diversi da quelli memorizzati in *min* e *max*, se questo è il caso base, la riga 4 restituisce FALSE. L'altra possibilità – non è un caso base è x è diverso da *min* e da *max* – è gestita dalla chiamata ricorsiva nella riga 5.

La ricorrenza (20.4) caratterizza il tempo di esecuzione della procedura VEB-TREE-MEMBER, e quindi questa procedura richiede un tempo $O(\lg \lg u)$.

Trovare il successore e il predecessore

Adesso vediamo come implementare l'operazione SUCCESSOR. Ricordiamo che la procedura PROTO-VEB-SUCCESSOR(V, x) potrebbe fare due chiamate ricorsive: la prima per determinare se il successore di x si trova nello stesso cluster di x e, in caso contrario, la seconda per trovare il cluster che contiene il successore di x. Poiché è possibile accedere rapidamente al valore massimo in un albero vEB, possiamo evitare di fare due chiamate ricorsive, facendone una sola sul cluster o sul summary, ma non su entrambi.

```
VEB-TREE-SUCCESSOR(V, x)
 1  if V.u == 2
 2       if x == 0 and V.max == 1
 3            return 1
 4       else return NIL
 5  elseif V.min ≠ NIL and x < V.min
 6       return V.min
 7  else max-low = VEB-TREE-MAXIMUM(V.cluster[high(x)])
 8       if max-low ≠ NIL and low(x) < max-low
 9            offset = VEB-TREE-SUCCESSOR(V.cluster[high(x)], low(x))
10            return index(high(x), offset)
11       else succ-cluster = VEB-TREE-SUCCESSOR(V.summary, high(x))
12            if succ-cluster == NIL
13                 return NIL
14            else offset = VEB-TREE-MINIMUM(V.cluster[succ-cluster])
15                 return index(succ-cluster, offset)
```

Questa procedura ha sei istruzioni **return** e diversi casi. Iniziamo dal caso base nelle righe 2–4, che restituisce 1 nella riga 3 se stiamo cercando il successore di 0 e 1 si trova nell'insieme di 2 elementi; altrimenti, il caso base restituisce NIL nella riga 4.

Acquistato da Michele Michels su Webster il 2022-07-07 23:12 Numero Ordine Libraria: 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

Se non ci troviamo nel caso base, la riga 5 verifica se x è strettamente minore dell'elemento minimo; se ciò è vero, la riga 6 restituisce semplicemente l'elemento minimo.

Se arriviamo alla riga 7, allora sappiamo che non ci troviamo in un caso base e che x è maggiore o uguale al valore minimo nell'albero vEB V. La riga 7 assegna a *max-low* l'elemento massimo nel cluster di x. Se il cluster di x contiene qualche elemento che è maggiore di x, allora sappiamo che il successore di x si trova da qualche parte nel cluster di x. La riga 8 verifica questa condizione. Se il successore di x è all'interno del cluster di x, allora la riga 9 determina dove si trova esattamente, e la riga 10 restituisce il successore nello stesso modo della riga 7 di PROTO-VEB-SUCCESSOR.

Arriviamo alla riga 11 se x è maggiore o uguale all'elemento massimo del suo cluster. In questo caso, le righe 11–15 trovano il successore di x nello stesso modo delle righe 8–12 di PROTO-VEB-SUCCESSOR.

È facile vedere come la ricorrenza (20.4) caratterizza il tempo di esecuzione di VEB-TREE-SUCCESSOR. In base al risultato della verifica della riga 7, la procedura chiama se stessa in modo ricorsivo nella riga 9 (su un albero vEB con dimensione dell'universo $\sqrt[\downarrow]{u}$) o nella riga 11 (su un albero vEB con dimensione dell'universo $\sqrt[\uparrow]{u}$). In ogni caso, la chiamata ricorsiva riguarda un albero vEB con dimensione dell'universo al più $\sqrt[\uparrow]{u}$. La parte restante della procedura, incluse le chiamate di VEB-TREE-MINIMUM e VEB-TREE-MAXIMUM, richiede un tempo $O(1)$. Quindi, VEB-TREE-SUCCESSOR viene eseguita nel tempo $O(\lg \lg u)$ nel caso peggiore.

La procedura VEB-TREE-PREDECESSOR è simmetrica alla procedura VEB-TREE-SUCCESSOR, ma con un caso aggiuntivo:

```
VEB-TREE-PREDECESSOR(V, x)
 1  if V.u == 2
 2      if x == 1 and V.min == 0
 3          return 0
 4      else return NIL
 5  elseif V.max ≠ NIL and x > V.max
 6      return V.max
 7  else min-low = VEB-TREE-MINIMUM(V.cluster[high(x)])
 8      if min-low ≠ NIL and low(x) > min-low
 9          offset = VEB-TREE-PREDECESSOR(V.cluster[high(x)], low(x))
10          return index(high(x), offset)
11      else pred-cluster = VEB-TREE-PREDECESSOR(V.summary, high(x))
12          if pred-cluster == NIL
13              if V.min ≠ NIL and x > V.min
14                  return V.min
15              else return NIL
16          else offset = VEB-TREE-MAXIMUM(V.cluster[pred-cluster])
17              return index(pred-cluster, offset)
```

Le righe 13–14 formano il caso aggiuntivo. Questo caso si verifica quando il predecessore di x, se esiste, non si trova nel cluster di x. La procedura VEB-TREE-SUCCESSOR garantiva che se il successore di x fosse stato all'esterno del cluster di x, allora esso doveva essere in un cluster con un numero più grande. Ma se il predecessore di x è il valore minimo nell'albero vEB V, allora il predecessore non si trova in alcun cluster. La riga 13 verifica questa condizione, e la riga 14 restituisce il valore minimo appropriato.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995030-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Questo caso extra non influisce sul tempo di esecuzione asintotico di VEB-TREE-PREDECESSOR se confrontato con VEB-TREE-SUCCESSOR, e quindi VEB-TREE-PREDECESSOR viene eseguita nel tempo $O(\lg \lg u)$ nel caso peggiore.

Inserire un elemento

Vediamo adesso come inserire un elemento in un albero vEB. Ricordiamo che PROTO-VEB-INSERT effettuava due chiamate ricorsive: una per inserire l'elemento e una per inserire il numero del cluster dell'elemento nel summary. La procedura VEB-TREE-INSERT farà una sola chiamata ricorsiva. Come potremo cavarcela con una sola chiamata? Quando inseriamo un elemento, il cluster dove va inserito l'elemento può contenere già un elemento oppure no. Se il cluster ha già un altro elemento, allora il numero del cluster è già nel summary, e così non c'è bisogno di fare quella chiamata ricorsiva. Se il cluster non ha un altro elemento, allora l'elemento da inserire diventa l'unico elemento nel cluster, e non è necessaria la ricorsione per inserire un elemento in un albero vEB vuoto:

```
VEB-EMPTY-TREE-INSERT(V, x)
1  V.min = x
2  V.max = x
```

Disponendo di questa procedura, possiamo scrivere il seguente pseudocodice per VEB-TREE-INSERT(V, x), supponendo che x non sia già un elemento nell'insieme rappresentato dall'albero vEB V:

```
VEB-TREE-INSERT(V, x)
 1  if V.min == NIL
 2      VEB-EMPTY-TREE-INSERT(V, x)
 3  else if x < V.min
 4          exchange x with V.min
 5      if V.u > 2
 6          if VEB-TREE-MINIMUM(V.cluster[high(x)]) == NIL
 7              VEB-TREE-INSERT(V.summary, high(x))
 8              VEB-EMPTY-TREE-INSERT(V.cluster[high(x)], low(x))
 9          else VEB-TREE-INSERT(V.cluster[high(x)], low(x))
10      if x > V.max
11          V.max = x
```

Questa procedura opera nel modo seguente. La riga 1 verifica se V è un albero vEB vuoto; se lo è, la riga 2 gestisce questo semplice caso. Le righe 3–11 suppongono che V non sia vuoto, e quindi qualche elemento sarà inserito in uno dei cluster di V. Ma questo elemento non necessariamente è l'elemento x passato a VEB-TREE-INSERT. Se $x < min$ (la cui verifica avviene nella riga 3), allora x deve diventare il nuovo *min*. Tuttavia, non vogliamo perdere il valore di *min* originale, e quindi occorre inserirlo in uno dei cluster di V. In questo caso, la riga 4 scambia x con *min*, in modo da inserire *min* originale in uno dei cluster di V.

Acquistato da Michele Mastrolonardo - ISBN 9788838673132 - Numero di licenza non disponibile - Copyright © 2010 The McGraw-Hill Education (Italy)

Le righe 6–9 vengono eseguite soltanto se V non è un albero vEB del caso base. La riga 6 determina se il cluster dove andrà x è vuoto; se lo è, la riga 7 inserisce il numero di cluster di x nel summary e la riga 8 gestisce il semplice

caso dell'inserimento di x in un cluster vuoto. Se il cluster di x non è vuoto, allora la riga 9 inserisce x nel suo cluster. In questo caso, non occorre aggiornare il summary, in quanto il numero del cluster di x è già un membro del summary.

Infine, le righe 10–11 hanno il compito di aggiornare *max* se $x > max$. Si noti che, se V è un albero vEB del caso base che non è vuoto, le righe 3–4 e 10–11 aggiornano appropriatamente *min* e *max*.

Ancora una volta, possiamo facilmente vedere come la ricorrenza (20.4) caratterizza il tempo di esecuzione. In base al risultato della verifica nella riga 6, viene eseguita la chiamata ricorsiva nella riga 7 (su un albero vEB con dimensione dell'universo $\uparrow\sqrt{u}$) o la chiamata ricorsiva nella riga 9 (su un albero vEB con dimensione dell'universo $\downarrow\sqrt{u}$). In ogni caso, la chiamata ricorsiva riguarda un albero vEB con dimensione dell'universo al più $\uparrow\sqrt{u}$. Poiché la parte restante della procedura VEB-TREE-INSERT richiede un tempo $O(1)$, si applica la ricorrenza (20.4), e quindi il tempo di esecuzione è $O(\lg \lg u)$.

Cancellare un elemento

Infine, vediamo come cancellare un elemento da un albero vEB. La procedura VEB-TREE-DELETE(V, x) suppone che x sia un elemento dell'insieme rappresentato dall'albero vEB V.

```
VEB-TREE-DELETE(V, x)
 1  if V.min == V.max
 2      V.min = NIL
 3      V.max = NIL
 4  elseif V.u == 2
 5      if x == 0
 6          V.min = 1
 7      else V.min = 0
 8      V.max = V.min
 9  else if x == V.min
10          first-cluster = VEB-TREE-MINIMUM(V.summary)
11          x = index(first-cluster,
                VEB-TREE-MINIMUM(V.cluster[first-cluster]))
12          V.min = x
13      VEB-TREE-DELETE(V.cluster[high(x)], low(x))
14      if VEB-TREE-MINIMUM(V.cluster[high(x)]) == NIL
15          VEB-TREE-DELETE(V.summary, high(x))
16          if x == V.max
17              summary-max = VEB-TREE-MAXIMUM(V.summary)
18              if summary-max == NIL
19                  V.max = V.min
20              else V.max = index(summary-max,
                        VEB-TREE-MAXIMUM(V.cluster[summary-max]))
21      elseif x == V.max
22          V.max = index(high(x),
                VEB-TREE-MAXIMUM(V.cluster[high(x)]))
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

La procedura VEB-TREE-DELETE opera nel modo seguente. Se l'albero vEB V contiene un solo elemento, allora la sua cancellazione è altrettanto semplice come inserire un elemento in un albero vEB vuoto: basta impostare *min* e *max* a NIL. Le righe 1–3 gestiscono questo caso. Altrimenti, V ha almeno due elementi. La

riga 4 verifica se V è un albero vEB del caso base; se lo è, le righe 5–8 impostano *min* e *max* all'elemento rimasto.

Le righe 9–22 suppongono che V abbia due o più elementi e che $u \geq 4$. In questo caso, dovremo cancellare un elemento da un cluster. Tuttavia, l'elemento che cancelliamo da un cluster potrebbe non essere x, perché se x è uguale a *min*, allora una volta cancellato x, qualche altro elemento all'interno di uno dei cluster di V diventa il nuovo *min*, e dovremo cancellare tale elemento dal suo cluster. Se il test nella riga 9 rivela che ci troviamo in questo caso, allora la riga 10 imposta *first-cluster* al numero del cluster che contiene il più piccolo elemento diverso da *min*, e la riga 11 imposta x al valore del più piccolo elemento in tale cluster. Questo elemento diventa il nuovo *min* nella riga 12 e, poiché impostiamo x al valore di tale elemento, esso è l'elemento che sarà cancellato dal suo cluster.

Quando arriviamo alla riga 13, sappiamo che dobbiamo cancellare l'elemento x dal suo cluster, sia che x sia il valore originariamente passato a vEB-TREE-DELETE sia che x sia l'elemento che deve diventare il nuovo minimo. La riga 13 cancella x dal suo cluster. Questo cluster adesso potrebbe essere vuoto; la verifica di questo avviene nella riga 14. Se il cluster è vuoto, occorre rimuovere il numero del cluster di x dal summary; questo compito è svolto dalla riga 15. Dopo avere aggiornato il summary, occorre aggiornare *max*. La riga 16 verifica se stiamo cancellando l'elemento massimo in V e, in caso affermativo, la riga 17 imposta *summary-max* al numero del cluster non vuoto con il numero più grande. (La chiamata vEB-TREE-MAXIMUM($V.summary$) funziona perché abbiamo già chiamato ricorsivamente vEB-TREE-DELETE su $V.summary$, e quindi $V.summary.max$ è già stato opportunamente aggiornato.) Se tutti i cluster di V sono vuoti, allora l'unico elemento rimasto in V è *min*; la riga 18 verifica questo caso, e la riga 19 aggiorna *max* in modo appropriato. Altrimenti, la riga 20 imposta *max* al massimo elemento nel cluster con il numero più grande. (Se questo è il cluster in cui abbiamo cancellato l'elemento, ci affidiamo di nuovo alla chiamata ricorsiva nella riga 13 che ha già corretto l'attributo *max* di tale cluster.)

Infine, dobbiamo gestire il caso in cui il cluster di x non sia vuoto dopo la cancellazione di x. Sebbene non occorra aggiornare il summary in questo caso, potrebbe essere necessario aggiornare *max*. La riga 21 verifica questo caso, e se occorre aggiornare *max*, ciò avviene nella riga 22 (anche qui ci affidiamo alla chiamata ricorsiva che ha corretto *max* nel cluster).

Adesso dimostriamo che vEB-TREE-DELETE viene eseguita nel tempo $O(\lg \lg u)$ nel caso peggiore. A prima vista potrebbe sembrare che la ricorrenza (20.4) non si possa applicare sempre, perché una singola chiamata di vEB-TREE-DELETE può effettuare due chiamate ricorsive: una nella riga 13 e una nella riga 15. Sebbene la procedura possa effettuare entrambe le chiamate ricorsive, vediamo che cosa accade in tal caso. Affinché venga effettuata la chiamata ricorsiva nella riga 15, il test nella riga 14 deve indicare che il cluster di x è vuoto. Tale cluster può essere vuoto soltanto se x era l'unico elemento nel suo cluster quando è stata fatta la chiamata ricorsiva nella riga 13. Ma se x era l'unico elemento nel suo cluster, allora quella chiamata ricorsiva ha richiesto un tempo $O(1)$, perché ha eseguito soltanto le righe 1–3. Dunque, abbiamo due possibilità mutuamente esclusive:

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 . Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

- La chiamata ricorsiva nella riga 13 ha richiesto un tempo costante.
- La chiamata ricorsiva nella riga 15 non è stata effettuata.

In ogni caso, la ricorrenza (20.4) caratterizza il tempo di esecuzione di vEB-TREE-DELETE, e quindi il suo tempo di esecuzione nel caso peggiore è $O(\lg \lg u)$.

Esercizi

20.3-1

Modificate gli alberi vEB per supportare le chiavi duplicate.

20.3-2

Modificate gli alberi vEB per supportare chiavi a cui sono associati dei dati satelliti.

20.3-3

Scrivete lo pseudocodice per una procedura che crea un albero di van Emde Boas vuoto.

20.3-4

Che cosa accade se chiamate VEB-TREE-INSERT con un elemento che è già nell'albero vEB? Che cosa accade se chiamate VEB-TREE-DELETE con un elemento che non è nell'albero vEB? Spiegate il comportamento delle procedure. Mostrate come modificare gli alberi vEB e le loro operazioni in modo da poter verificare in un tempo costante se è presente un elemento.

20.3-5

Supponete che, anziché $\uparrow\sqrt{u}$ cluster, ciascuno con dimensione dell'universo $\downarrow\sqrt{u}$, avessimo costruito alberi vEB per avere $u^{1/k}$ cluster, ciascuno con dimensione dell'universo $u^{1-1/k}$, dove $k > 1$ è una costante. Se dovessimo modificare le operazioni in modo appropriato, quali sarebbero i loro tempi di esecuzione? Ai fini dell'analisi, supponete che $u^{1/k}$ e $u^{1-1/k}$ siano sempre numeri interi.

20.3-6

La creazione di un albero vEB con dimensione dell'universo u richiede un tempo $O(u)$. Supponete di voler tener conto esplicitamente di tale tempo. Qual è il numero minimo di operazioni n per cui il tempo ammortizzato di ciascuna operazione in un albero vEB è $O(\lg\lg u)$?

Problemi

20-1 Spazio richiesto dagli alberi di van Emde Boas

Questo problema serve ad esaminare lo spazio richiesto dagli alberi di van Emde Boas e suggerisce un metodo per modificare la struttura dati in modo che il suo spazio richiesto dipenda dal numero di elementi n effettivamente memorizzati nell'albero, anziché dalla dimensione dell'universo u. Per semplicità, supponete che $\sqrt{u}$ sia sempre un intero.

a. Spiegate perché la seguente ricorrenza caratterizza lo spazio richiesto $P(u)$ di un albero di van Emde Boas con dimensione dell'universo u:

$$P(u) = (\sqrt{u} + 1)P(\sqrt{u}) + \Theta(\sqrt{u}) \tag{20.5}$$

b. Dimostrate che la ricorrenza (20.5) ha la soluzione $P(u) = O(u)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Per ridurre lo spazio richiesto, definiamo un ***albero di van Emde Boas con spazio ridotto***, o ***albero RS-vEB***, come un albero vEB V, ma con le seguenti modifiche:

- L'attributo $V.cluster$, anziché essere memorizzato come un semplice array di puntatori agli alberi vEB con dimensione dell'universo $\sqrt{u}$, è una tavola hash (vedere il Capitolo 11) memorizzata come una tavola dinamica (vedere il Paragrafo 17.4). Come nella versione con array di $V.cluster$, la tavola hash memorizza dei puntatori ad alberi RS-vEB con dimensione dell'universo $\sqrt{u}$. Per

trovare l'i-esimo cluster, cerchiamo la chiave i nella tavola hash, in modo da poter trovare l'i-esimo cluster tramite una sola ricerca nella tavola hash.

- La tavola hash memorizza soltanto i puntatori ai cluster non vuoti. Se la ricerca di un cluster nella tavola hash restituisce NIL, significa che il cluster è vuoto.
- L'attributo $V.summary$ è NIL se tutti i cluster sono vuoti. Altrimenti, $V.summary$ punta a un albero RS-vEB con dimensione dell'universo $\sqrt{u}$.

Poiché la tavola hash viene implementata con una tavola dinamica, lo spazio richiesto è proporzionale al numero di cluster non vuoti.

Per inserire un elemento in un albero RS-vEB non vuoto, creiamo l'albero RS-vEB chiamando la seguente procedura, dove il parametro u è la dimensione dell'universo dell'albero RS-vEB:

```
CREATE-NEW-RS-VEB-TREE(u)
1  Alloca un nuovo albero vEB V
2  V.u = u
3  V.min = NIL
4  V.max = NIL
5  V.summary = NIL
6  Crea V.cluster come una tavola hash dinamica vuota
7  return V
```

c. Modificate la procedura VEB-TREE-INSERT per ottenere lo pseudocodice della procedura RS-VEB-TREE-INSERT(V, x), che inserisce x nell'albero RS-vEB V, chiamando CREATE-NEW-RS-VEB-TREE quando necessario.

d. Modificate la procedura VEB-TREE-SUCCESSOR per ottenere lo pseudocodice della procedura RS-VEB-TREE-SUCCESSOR(V, x), che restituisce il successore di x nell'albero RS-vEB V, o NIL se x non ha un successore in V.

e. Dimostrate che, nell'ipotesi di hashing uniforme semplice, le vostre procedure RS-VEB-TREE-INSERT e RS-VEB-TREE-SUCCESSOR vengono eseguite nel tempo atteso $O(\lg \lg u)$.

f. Supponendo che gli elementi non vengano mai cancellati da un albero vEB, dimostrate che lo spazio richiesto da una struttura ad albero RS-vEB è $O(n)$, dove n è il numero di elementi effettivamente memorizzati nell'albero RS-vEB.

g. Gli alberi RS-vEB hanno un altro vantaggio sugli alberi vEB: la loro creazione richiede meno tempo. Quanto tempo occorre per creare un albero RS-vEB vuoto?

20-2 y-fast trie

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950301 6-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Questo problema esamina gli alberi di D. Willard "y-fast trie" che, come quelli di van Emde Boas, eseguono ciascuna delle operazioni MEMBER, MINIMUM, MAXIMUM, PREDECESSOR e SUCCESSOR sugli elementi estratti da un universo con dimensione u nel tempo $O(\lg \lg u)$ nel caso peggiore. Le operazioni INSERT e DELETE richiedono un tempo ammortizzato $O(\lg \lg u)$. Come gli alberi di van Emde Boas con spazio ridotto (vedere il Problema 20-1), gli alberi y-fast trie

utilizzano soltanto uno spazio $O(n)$ per memorizzare n elementi. Il progetto degli alberi y-fast trie si basa sull'hashing perfetto (vedere il Paragrafo 11.5).

Come struttura preliminare, supponiamo di creare una tavola hash perfetta che contiene non soltanto tutti gli elementi nell'insieme dinamico, ma anche tutti i prefissi della rappresentazione binaria di ogni elemento dell'insieme. Per esempio, se $u = 16$, ovvero $\lg u = 4$, e $x = 13$ è nell'insieme, allora poiché la rappresentazione binaria di 13 è 1101, la tavola hash perfetta contiene le stringhe 1, 11, 110 e 1101. Oltre alla tavola hash, creiamo una lista doppiamente concatenata degli elementi dell'insieme, in ordine crescente.

a. Quanto spazio richiede questa struttura?

b. Dimostrate come eseguire le operazioni MINIMUM e MAXIMUM nel tempo $O(1)$; le operazioni MEMBER, PREDECESSOR e SUCCESSOR nel tempo $O(\lg \lg u)$; e le operazioni INSERT e DELETE nel tempo $O(\lg u)$.

Per ridurre lo spazio richiesto a $O(n)$, apportiamo le seguenti modifiche alla struttura dati:

- Organizziamo gli n elementi in $n / \lg u$ gruppi di dimensione $\lg u$. (Supponete per adesso che $\lg u$ divide n.) Il primo gruppo è formato dai $\lg u$ elementi più piccoli dell'insieme, il secondo gruppo è formato dai successivi $\lg u$ elementi più piccoli, e così via.
- Scegliamo un valore "rappresentativo" per ciascun gruppo. Il valore rappresentativo dell'i-esimo gruppo è grande almeno quanto il più grande elemento dell'i-esimo gruppo, ed è più piccolo di qualsiasi elemento dell'$(i + 1)$-esimo gruppo. (Il valore rappresentativo dell'ultimo gruppo potrebbe essere l'elemento più grande possibile $u - 1$.) Notate che un valore rappresentativo potrebbe essere un valore che non si trova correntemente nell'insieme.
- Memorizziamo i $\lg u$ elementi di ciascun gruppo in un albero di ricerca binario bilanciato, come un albero rosso-nero. Ogni valore rappresentativo punta all'albero di ricerca binario bilanciato del suo gruppo, e ciascun albero di ricerca binario bilanciato punta al valore rappresentativo del suo gruppo.
- La tavola hash perfetta memorizza soltanto i valori rappresentativi, anch'essi memorizzati in una lista doppiamente concatenata in ordine crescente.

Chiamiamo questa struttura albero ***y-fast trie***.

c. Dimostrate che un albero y-fast trie richiede soltanto uno spazio $O(n)$ per memorizzare n elementi.

d. Dimostrate come eseguire le operazioni MINIMUM e MAXIMUM nel tempo $O(\lg \lg u)$ con un albero y-fast trie.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

e. Dimostrate come eseguire l'operazione MEMBER nel tempo $O(\lg \lg u)$.

f. Dimostrate come eseguire le operazioni PREDECESSOR e SUCCESSOR nel tempo $O(\lg \lg u)$.

g. Spiegate perché le operazioni INSERT e DELETE richiedono un tempo $\Omega(\lg \lg u)$.

h. Dimostrate come attenuare l'esigenza che ciascun gruppo in un albero y-fast trie abbia esattamente $\lg u$ elementi per consentire alle operazioni INSERT e DELETE di essere eseguite nel tempo ammortizzato $O(\lg \lg u)$, senza influire sul tempo di esecuzione asintotico delle altre operazioni.

Note

La struttura dati presentata in questo capitolo ha il nome di P. van Emde Boas, che ne descrisse una prima forma nel 1975 [340]. Successivi articoli di van Emde Boas [341] e van Emde Boas, Kaas e Zijlstra [342] hanno affinato l'idea originaria e l'esposizione della struttura dati. Mehlhorn e Näher [253] successivamente hanno ampliato il concetto per applicarlo a dimensioni dell'universo che sono numeri primi. Il libro di Mehlhorn [250] contiene una trattazione degli alberi di van Emde Boas leggermente diversa da quella presentata in questo capitolo.

Applicando il concetto di albero di van Emde Boas, Dementiev et al. [84] hanno sviluppato un albero di ricerca non ricorsivo su tre livelli che è risultato più veloce dell'albero di van Emde Boas nei loro esperimenti.

Wang e Lin [348] hanno progettato una versione speciale (*hardware-pipelined*) degli alberi di van Emde Boas, che richiede un tempo ammortizzato costante per operazione utilizzando $O(\lg \lg u)$ stadi nella pipeline.

Un limite inferiore di Pătraşcu e Thorup [274, 275] per trovare il predecessore dimostra che gli alberi di van Emde Boas sono ottimali per questa operazione, anche se permettiamo la randomizzazione.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

21 Strutture dati per insiemi disgiunti

Alcune applicazioni richiedono di raggruppare n elementi distinti in una collezione di insiemi disgiunti. Queste applicazioni spesso richiedono l'esecuzione di due particolari operazioni: trovare l'unico insieme che contiene un determinato elemento e unire due insiemi. Questo capitolo esamina i metodi per mantenere una struttura dati che supporta queste operazioni.

Il Paragrafo 21.1 descrive le operazioni supportate da una struttura dati per insiemi disgiunti e presenta una semplice applicazione. Il Paragrafo 21.2 presenta una semplice implementazione degli insiemi disgiunti che utilizza le liste concatenate. Nel Paragrafo 21.3 esamineremo una rappresentazione più efficiente che usa gli alberi radicati. Se utilizziamo la rappresentazione con gli alberi radicati, il tempo di esecuzione è teoricamente superlineare ma lineare ai fini pratici.

Il Paragrafo 21.4 definisce e analizza una funzione che cresce molto rapidamente; la sua funzione inversa, che cresce molto lentamente, compare nell'espressione del tempo di esecuzione delle operazioni delle implementazioni basate sugli alberi. Applicheremo l'analisi ammortizzata per dimostrare un limite superiore sul tempo di esecuzione che è appena un po' superlineare.

21.1 Operazioni con gli insiemi disgiunti

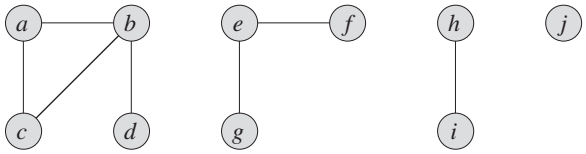
Una ***struttura dati per insiemi disgiunti*** mantiene una collezione $\mathcal{S} = \{S_1, S_2, \ldots, S_k\}$ di insiemi dinamici disgiunti. Ciascun insieme è identificato da un ***rappresentante***, che è un elemento dell'insieme. In alcune applicazioni non è importante quale elemento sarà utilizzato come rappresentante; l'unica condizione che imponiamo è che, se richiediamo due volte il rappresentante di un insieme dinamico senza modificare l'insieme fra le due richieste, dobbiamo ottenere la stessa risposta entrambe le volte. In altre applicazioni ci potrebbe essere una regola prestabilita per scegliere il rappresentante, per esempio si potrebbe scegliere l'elemento più piccolo dell'insieme (supponendo, ovviamente, che gli elementi possano essere ordinati).

Come in altre implementazioni degli insiemi dinamici finora analizzate, ogni elemento di un insieme è rappresentato da un oggetto. Indicando con x un oggetto, vogliamo supportare le seguenti operazioni:

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

MAKE-SET(x) crea un nuovo insieme il cui unico elemento (e rappresentante) è x. Poiché gli insiemi sono disgiunti, x non può trovarsi in qualche altro insieme.

UNION(x, y) unisce gli insiemi dinamici che contengono x e y, per esempio S_x e S_y, in un nuovo insieme che è l'unione di questi due insiemi. Si suppone che i due insiemi siano disgiunti prima dell'operazione. Il rappresentante dell'in-



(a)

Arco elaborato	Collezione di insiemi disgiunti									
insiemi iniziali	$\{a\}$	$\{b\}$	$\{c\}$	$\{d\}$	$\{e\}$	$\{f\}$	$\{g\}$	$\{h\}$	$\{i\}$	$\{j\}$
(b,d)	$\{a\}$	$\{b,d\}$	$\{c\}$		$\{e\}$	$\{f\}$	$\{g\}$	$\{h\}$	$\{i\}$	$\{j\}$
(e,g)	$\{a\}$	$\{b,d\}$	$\{c\}$		$\{e,g\}$	$\{f\}$		$\{h\}$	$\{i\}$	$\{j\}$
(a,c)	$\{a,c\}$	$\{b,d\}$			$\{e,g\}$	$\{f\}$		$\{h\}$	$\{i\}$	$\{j\}$
(h,i)	$\{a,c\}$	$\{b,d\}$			$\{e,g\}$	$\{f\}$		$\{h,i\}$		$\{j\}$
(a,b)	$\{a,b,c,d\}$				$\{e,g\}$	$\{f\}$		$\{h,i\}$		$\{j\}$
(e,f)	$\{a,b,c,d\}$				$\{e,f,g\}$			$\{h,i\}$		$\{j\}$
(b,c)	$\{a,b,c,d\}$				$\{e,f,g\}$			$\{h,i\}$		$\{j\}$

(b)

Figura 21.1 **(a)** Un grafo con quattro componenti connesse: $\{a, b, c, d\}$, $\{e, f, g\}$, $\{h, i\}$ e $\{j\}$. **(b)** La collezione degli insiemi disgiunti dopo che ogni arco è stato elaborato.

sieme risultante è un elemento qualsiasi di $S_x \cup S_y$, sebbene molte implementazioni di UNION scelgano specificamente il rappresentante di S_x o quello di S_y come nuovo rappresentante. Poiché richiediamo che gli insiemi nella collezione siano disgiunti, "distruggiamo" gli insiemi S_x e S_y, eliminandoli dalla collezione $\mathcal{S}$. Nella pratica, spesso gli elementi di uno degli insiemi vengono assorbiti dall'altro insieme.

FIND-SET(x) restituisce un puntatore al rappresentante dell'insieme (unico) che contiene x.

In questo capitolo analizzeremo i tempi di esecuzione delle strutture dati per gli insiemi disgiunti in funzione di due parametri: n, il numero di operazioni MAKE-SET, ed m, il numero totale di operazioni MAKE-SET, UNION e FIND-SET. Poiché gli insiemi sono disgiunti, ciascuna operazione UNION riduce di un'unità il numero degli insiemi. Dopo $n - 1$ operazioni UNION, quindi, resta un solo insieme. Ne consegue che il numero di operazioni UNION è al più $n - 1$. Poiché le operazioni MAKE-SET sono incluse nel numero totale di operazioni m, allora $m \geq n$. Si suppone che le n operazioni MAKE-SET siano le prime n operazioni eseguite.

Un'applicazione delle strutture dati per insiemi disgiunti

Una delle tante applicazioni delle strutture dati per insiemi disgiunti consiste nel determinare le componenti connesse di un grafo non orientato (vedere il Paragrafo B.4). La Figura 21.1(a), per esempio, illustra un grafo con quattro componenti connesse.

Acquistato da Michele Mirabile su LaWeb.it il 2022-01-07 23:12 Numero Ordine Libreria 1005084/220707-0Quattrini © 2022 - McGraw-Hill Education (Italy)

La seguente procedura CONNECTED-COMPONENTS usa le operazioni degli insiemi disgiunti per calcolare le componenti connesse di un grafo. Una volta che CONNECTED-COMPONENTS ha preelaborato il grafo, la procedura SAME-COMPONENT è in grado di determinare se due vertici sono nella stessa compo-

nente connessa.[1] (L'insieme dei vertici di un grafo G è indicato da $G.V$; l'insieme degli archi è indicato da $G.E$.)

```
CONNECTED-COMPONENTS(G)
1  for ogni vertice v ∈ G.V
2       MAKE-SET(v)
3  for ogni arco (u, v) ∈ G.E
4       if FIND-SET(u) ≠ FIND-SET(v)
5            UNION(u, v)
```

```
SAME-COMPONENT(u, v)
1  if FIND-SET(u) == FIND-SET(v)
2       return TRUE
3  else return FALSE
```

Inizialmente, la procedura CONNECTED-COMPONENTS pone ciascun vertice v nel proprio insieme. Poi, per ogni arco (u, v), unisce gli insiemi che contengono u e v. Per l'Esercizio 21.1-2, dopo che tutti gli archi sono stati elaborati, due vertici si trovano nella stessa componente connessa, se e soltanto se i corrispondenti oggetti si trovano nello stesso insieme. Dunque, CONNECTED-COMPONENTS calcola gli insiemi in modo tale che la procedura SAME-COMPONENT possa determinare se due vertici si trovano nella stessa componente connessa. La Figura 21.1(b) illustra come vengono calcolati gli insiemi disgiunti da CONNECTED-COMPONENTS.

In una implementazione reale di questo algoritmo per le componenti connesse, le rappresentazioni del grafo e della struttura dati degli insiemi disgiunti dovrebbero avere dei riferimenti incrociati. Ovvero, un oggetto che rappresenta un vertice dovrà avere un puntatore al corrispondente oggetto di un insieme disgiunto, e viceversa. Questi dettagli di programmazione dipendono dal linguaggio di programmazione, quindi non li tratteremo qui.

Esercizi

21.1-1

Supponete che la procedura CONNECTED-COMPONENTS venga eseguita sul grafo non orientato $G = (V, E)$, dove $V = \{a, b, c, d, e, f, g, h, i, j, k\}$ e che gli archi di E siano elaborati nel seguente ordine: $(d, i), (f, k), (g, i), (b, g), (a, h), (i, j), (d, k), (b, j), (d, f), (g, j), (a, e), (i, d)$. Elencate i vertici in ciascuna componente connessa dopo ogni iterazione delle righe 3–5.

21.1-2

Dimostrate che, dopo che tutti gli archi sono stati elaborati da CONNECTED-COMPONENTS, due vertici si trovano nella stessa componente connessa, se e soltanto se appartengono allo stesso insieme.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[1] Se gli archi del grafo sono "statici" – non cambiano nel tempo – le componenti connesse possono essere calcolate più velocemente effettuando una visita in profondità del grafo (Esercizio 22.3-12). A volte, però, gli archi vengono aggiunti "dinamicamente" e occorre aggiornare le componenti connesse ogni volta che viene aggiunto un nuovo arco. In questo caso, può essere più efficiente l'implementazione presentata in questo paragrafo, anziché eseguire una nuova ricerca in profondità per ogni nodo che viene aggiunto.

21.1-3
Durante l'esecuzione di CONNECTED-COMPONENTS su un grafo non orientato $G = (V, E)$ con k componenti connesse, quante volte viene chiamata l'operazione FIND-SET? Quante volte viene chiamata UNION? Esprimete le vostre risposte in funzione di $|V|$, $|E|$ e k.

21.2 Rappresentazione di insiemi disgiunti tramite liste concatenate

La Figura 21.2(a) mostra un semplice modo di implementare una struttura dati per gli insiemi disgiunti: ciascun insieme è rappresentato dalla sua lista concatenata. L'oggetto di ciascun insieme ha gli attributi *head*, che punta al primo oggetto della lista, e *tail*, che punta all'ultimo oggetto. Ogni oggetto nella lista contiene un elemento dell'insieme, un puntatore al successivo oggetto della lista e un puntatore che ritorna all'oggetto dell'insieme. All'interno di ciascuna lista concatenata, gli oggetti possono apparire in qualsiasi ordine. Il rappresentante è l'elemento dell'insieme nel primo oggetto della lista.

Con questa rappresentazione mediante liste concatenate, entrambe le operazioni MAKE-SET e FIND-SET sono semplici da realizzare e richiedono un tempo $O(1)$. Per realizzare l'operazione MAKE-SET(x), creiamo una nuova lista concatenata il cui unico oggetto è x. Per l'operazione FIND-SET(x), basta seguire il puntatore da x per arrivare all'oggetto del suo insieme e poi ritornare all'elemento nell'oggetto cui punta *head*. Per esempio, nella Figura 21.2(a), la chiamata FIND-SET(g) restituirebbe f.

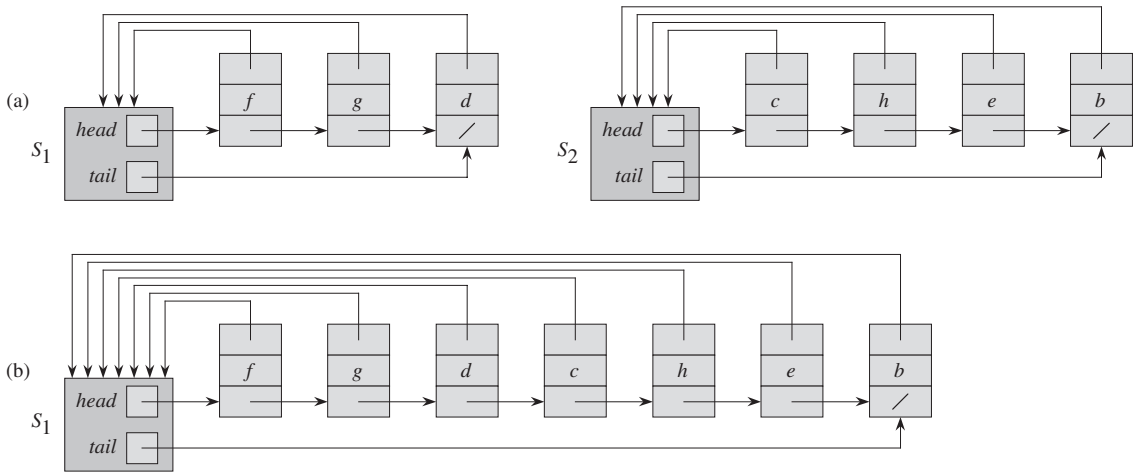


Figura 21.2 **(a)** Rappresentazioni con liste concatenate di due insiemi. L'insieme S_1 contiene gli elementi d, f e g, con f come rappresentante. L'insieme S_2 contiene gli elementi b, c, e e h, con c come rappresentante. Ogni oggetto nella lista contiene un elemento dell'insieme, un puntatore al successivo oggetto nella lista e un puntatore che ritorna all'oggetto dell'insieme. Ogni oggetto dell'insieme ha i puntatori *head* e *tail* rispettivamente al primo e all'ultimo oggetto. **(b)** Il risultato dell'operazione UNION(g, e), che aggiunge la lista concatenata che contiene e alla lista concatenata che contiene g. Il rappresentante dell'insieme risultante è f. L'oggetto dell'insieme per la lista S_2 di e viene eliminato.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016 220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Operazione	Numero di oggetti aggiornati
MAKE-SET(x_1)	1
MAKE-SET(x_2)	1
⋮	⋮
MAKE-SET(x_n)	1
UNION(x_2, x_1)	1
UNION(x_3, x_2)	2
UNION(x_4, x_3)	3
⋮	⋮
UNION(x_n, x_{n-1})	$n-1$

Figura 21.3 Una sequenza di $2n-1$ operazioni su n oggetti che richiede un tempo $\Theta(n^2)$, o in media un tempo $\Theta(n)$ per operazione, utilizzando la rappresentazione degli insiemi tramite liste concatenate e la semplice implementazione di UNION.

Una semplice implementazione dell'operazione di unione

La più semplice implementazione dell'operazione UNION che usa la rappresentazione degli insiemi mediante liste concatenate richiede molto più tempo rispetto all'operazione MAKE-SET o FIND-SET. Come illustra la Figura 21.2(b), noi eseguiamo UNION(x, y) aggiungendo la lista di y alla fine della lista di x. Il rappresentante della lista di x diventa il rappresentante dell'insieme risultante. Purtroppo, dobbiamo aggiornare il puntatore all'oggetto dell'insieme per ogni oggetto che originariamente si trovava nella lista di y; questo richiede un tempo lineare nella lunghezza della lista di y. Nella Figura 21.2, per esempio, l'operazione UNION(g, e) fa sì che i puntatori siano aggiornati negli oggetti di b, c, e e h.

In effetti, non è difficile trovare una sequenza di m operazioni su n oggetti che richiede un tempo $\Theta(n^2)$. Supponiamo di avere gli oggetti $x_1, x_2, \ldots, x_n$. Eseguiamo la sequenza di n operazioni MAKE-SET seguite dalle $n-1$ operazioni UNION elencate nella Figura 21.3, quindi $m = 2n-1$. Impieghiamo un tempo $\Theta(n)$ per eseguire le n operazioni MAKE-SET. Poiché l'i-esima operazione UNION aggiorna i oggetti, il numero totale di oggetti aggiornati da tutte le $n-1$ operazioni UNION è

$$\sum_{i=1}^{n-1} i = \Theta(n^2)$$

Il numero totale di operazioni è $2n-1$, pertanto ciascuna operazione richiede in media un tempo $\Theta(n)$. Dunque, il tempo ammortizzato di un'operazione è $\Theta(n)$.

Euristica dell'unione pesata

Nel caso peggiore, la precedente implementazione della procedura UNION richiede un tempo medio $\Theta(n)$ per chiamata, perché potremmo appendere una lista più lunga a una più corta; dobbiamo aggiornare il puntatore all'oggetto dell'insieme per ogni elemento della lista più lunga. Supponiamo invece che ogni lista includa anche la lunghezza della lista (che è facile da mantenere); supponiamo inoltre di appendere sempre la lista più piccola a quella più lunga, risolvendo in modo arbitrario i casi di liste aventi la stessa lunghezza. Con questa semplice ***euristica dell'unione pesata***, una singola operazione UNION può ancora impiegare un tempo $\Omega(n)$, se entrambi gli insiemi hanno $\Omega(n)$ elementi. Tuttavia, come illustra il seguente teorema, una sequenza di m operazioni MAKE-SET, UNION e FIND-SET, n delle quali sono operazioni MAKE-SET, impiega un tempo $O(m + n \lg n)$.

Acquistato da Michele Michele su Webster il 2022-07-07 11:21:11 Edizione Studenti - Copia non cedibile a terzi (Copyright Pearson Italia, Milano-Torino)

Teorema 21.1
Utilizzando la rappresentazione degli insiemi disgiunti tramite liste concatenate e l'euristica dell'unione pesata, una sequenza di m operazioni MAKE-SET, UNION e FIND-SET, n delle quali sono operazioni MAKE-SET, impiega un tempo $O(m + n \lg n)$.

Dimostrazione Poiché ogni operazione UNION unisce due insiemi disgiunti, vengono eseguite al più $n - 1$ operazioni UNION. Vediamo adesso il limite di tempo complessivamente impiegato da queste operazioni UNION. Iniziamo calcolando, per ogni oggetto, un limite superiore al numero di volte che viene aggiornato il puntatore di un oggetto all'oggetto del suo insieme. Consideriamo un particolare oggetto x. Sappiamo che ogni volta che il puntatore di x viene aggiornato, x deve trovarsi nell'insieme più piccolo. Quindi, la prima volta che il puntatore di x viene aggiornato, l'insieme risultante deve avere almeno 2 elementi. Analogamente, la seconda volta che il puntatore di x viene aggiornato, l'insieme risultante deve avere almeno 4 elementi. Procedendo così, osserviamo che per ogni $k \leq n$, dopo che il puntatore di x viene aggiornato $\lceil \lg k \rceil$ volte, l'insieme risultante deve avere almeno k elementi. Dato che l'insieme più grande ha al più n elementi, il puntatore in ciascun oggetto è stato aggiornato al più $\lceil \lg n \rceil$ volte in tutte le operazioni UNION. Quindi il tempo totale speso per aggiornare i puntatori degli oggetti durante tutte le operazioni UNION è $O(n \lg n)$. Dobbiamo tenere conto anche degli aggiornamenti dei puntatori *head* e *tail* e delle lunghezze delle liste, che richiedono soltanto un tempo $\Theta(1)$ per ogni operazione UNION. Il tempo totale impiegato per eseguire tutte le operazioni di UNION è dunque $O(n \lg n)$.

Il tempo per l'intera sequenza di m operazioni si calcola facilmente così: ciascuna operazione MAKE-SET e FIND-SET impiega un tempo $O(1)$ e ci sono $O(m)$ di queste operazioni; il tempo totale per l'intera sequenza è quindi $O(m + n \lg n)$. ■

Esercizi

21.2-1
Scrivete lo pseudocodice per MAKE-SET, FIND-SET e UNION utilizzando la rappresentazione con le liste concatenate e l'euristica dell'unione pesata. Ricordatevi di specificare gli attributi che avete pensato di assegnare agli oggetti degli insiemi e agli oggetti delle liste.

21.2-2
Illustrate la struttura dati e i risultati forniti dalle operazioni FIND-SET del seguente programma. Utilizzate la rappresentazione con le liste concatenate e l'euristica dell'unione pesata.

```
1  for i = 1 to 16
2      MAKE-SET(x_i)
3  for i = 1 to 15 by 2
4      UNION(x_i, x_{i+1})
5  for i = 1 to 13 by 4
6      UNION(x_i, x_{i+2})
7  UNION(x_1, x_5)
8  UNION(x_11, x_13)
9  UNION(x_1, x_10)
10 FIND-SET(x_2)
11 FIND-SET(x_9)
```

Acquistato da Michele Michele su Webster il 2022-07-17 22:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Supponete che, se gli insiemi che contengono x_i e x_j hanno la stessa dimensione, allora l'operazione UNION(x_i, x_j) appende la lista di x_j alla lista di x_i.

21.2-3
Utilizzando la rappresentazione con le liste concatenate e l'euristica dell'unione pesata, adattate la dimostrazione del Teorema 21.1 per ottenere i seguenti limiti dei tempi ammortizzati: $O(1)$ per le operazioni MAKE-SET e FIND-SET e $O(\lg n)$ per l'operazione UNION.

21.2-4
Trovate un limite asintotico stretto per il tempo di esecuzione della sequenza delle operazioni illustrate nella Figura 21.3, supponendo di utilizzare la rappresentazione con le liste concatenate e l'euristica dell'unione pesata.

21.2-5
Il professor Gompers ha il sospetto che potrebbe bastare un solo puntatore per ciascun oggetto di insieme, anziché due (*head* e *tail*), mantenendo due puntatori per ogni elemento di lista. Dimostrate che il sospetto del professore è ben fondato, descrivendo come rappresentare ciascun insieme tramite una lista concatenata tale che ogni operazione abbia lo stesso tempo di esecuzione delle operazioni descritte in questo paragrafo. Descrivete inoltre il funzionamento delle operazioni. Il vostro schema dovrebbe tener conto dell'euristica dell'unione pesata, con lo stesso effetto descritto in questo paragrafo (*suggerimento:* utilizzate l'ultimo elemento di una lista concatenata come rappresentante del suo insieme.)

21.2-6
Suggerite una semplice modifica della procedura UNION per la rappresentazione con le liste concatenate, al fine di escludere la necessità di tenere il puntatore *tail* all'ultimo oggetto in ciascuna lista. Indipendentemente dal fatto che venga utilizzata l'euristica dell'unione pesata, la vostra modifica non dovrà cambiare il tempo di esecuzione asintotico della procedura UNION (*suggerimento:* anziché aggiungere una lista in coda all'altra, inseritela frammezzo).

21.3 Foreste di insiemi disgiunti

In una implementazione più veloce degli insiemi disgiunti, rappresentiamo gli insiemi con alberi radicati, dove ogni nodo contiene un elemento e ogni albero rappresenta un insieme. In una ***foresta di insiemi disgiunti***, illustrata nella Figura 21.4(a), ogni elemento punta soltanto a suo padre. Il nodo radice di ogni albero contiene il rappresentante ed è padre di sé stesso. Come vedremo più avanti, sebbene gli algoritmi semplici che usano questa rappresentazione non siano più veloci di quelli che usano la rappresentazione con le liste concatenate, introducendo due euristiche – "unione per rango" e "compressione del cammino" – possiamo ottenere una struttura dati per insiemi disgiunti asintoticamente ottimale.

Realizziamo le tre operazioni degli insiemi disgiunti nel seguente modo. Un'operazione MAKE-SET crea semplicemente un albero con un solo nodo. Un'operazione FIND-SET segue i puntatori ai padri finché non trova la radice dell'albero. I nodi visitati in questo cammino semplice verso la radice costituiscono il ***cammino di ricerca***. Un'operazione UNION, illustrata nella Figura 21.4(b), fa sì che la radice di un albero punti alla radice dell'altro albero.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950316-2207070 Copyright © 2022 McGraw-Hill Education (Italy)

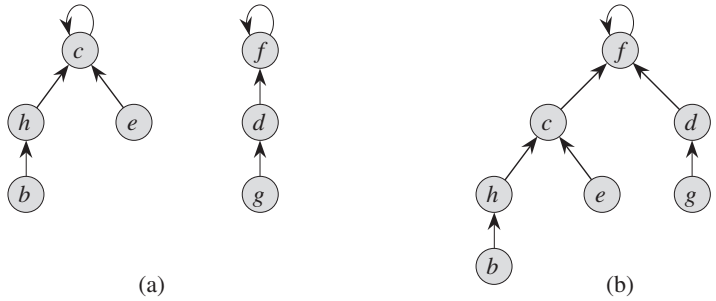


Figura 21.4 Una foresta di insiemi disgiunti. **(a)** Due alberi che rappresentano i due insiemi della Figura 21.2. L'albero a sinistra rappresenta l'insieme $\{b, c, e, h\}$, con c come rappresentante dell'insieme; l'albero a destra rappresenta l'insieme $\{d, f, g\}$, con f come rappresentante dell'insieme. **(b)** Il risultato dell'operazione UNION(e, g).

Euristiche per migliorare il tempo di esecuzione

Finora, non abbiamo migliorato rispetto all'implementazione con le liste concatenate. Una sequenza di $n - 1$ operazioni UNION può creare un albero che è una catena lineare di n nodi. Utilizzando due euristiche, però, possiamo ottenere un tempo di esecuzione che è quasi lineare nel numero totale di operazioni m.

La prima euristica, ***unione per rango***, è simile all'euristica dell'unione pesata che abbiamo utilizzato con la rappresentazione delle liste concatenate. L'idea consiste nel fare in modo che la radice dell'albero con meno nodi punti alla radice dell'albero con più nodi. Anziché tenere esplicitamente traccia della dimensione del sottoalbero radicato in ciascun nodo, adotteremo un approccio che semplifica l'analisi. Per ogni nodo manteniamo un ***rango*** che è un limite superiore per l'altezza del nodo. Nell'unione per rango, la radice con il rango più piccolo viene fatta puntare alla radice con il rango più grande durante un'operazione UNION.

La seconda euristica, ***compressione del cammino***, è molto semplice ed efficace. Come illustra la Figura 21.5, questa euristica viene utilizzata durante le operazioni FIND-SET per fare in modo che ciascun nodo nel cammino di ricerca punti direttamente alla radice. La compressione del cammino non cambia i ranghi.

Pseudocodice per foreste di insiemi disgiunti

Per implementare una foresta di insiemi disgiunti con l'euristica dell'unione per rango, bisogna tenere traccia dei ranghi. Per ogni nodo x manteniamo il valore intero $x.rank$, che è un limite superiore per l'altezza di x (il numero di archi nel cammino semplice più lungo fra x e una foglia sua discendente). Quando l'operazione MAKE-SET crea un insieme con un solo elemento, il rango iniziale dell'unico nodo nel corrispondente albero è 0. Ogni operazione FIND-SET lascia tutti i ranghi inalterati. Quando applichiamo UNION a due alberi, si presentano due casi, a seconda che le radici abbiamo oppure no lo stesso rango. Se le radici non hanno lo stesso rango, trasformiamo la radice di rango più alto nel padre della radice di rango più basso, ma i ranghi restano inalterati. Se, invece, le radici hanno lo stesso rango, trasformiamo arbitrariamente una delle radici in padre e ne incrementiamo il rango.

Scriviamo questo metodo con il nostro pseudocodice. Indichiamo con $x.p$ il padre del nodo x. La procedura LINK, una subroutine chiamata da UNION, riceve i puntatori a due radici come input.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libraria: 1995030016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

```
MAKE-SET(x)
1  x.p = x
2  x.rank = 0
```

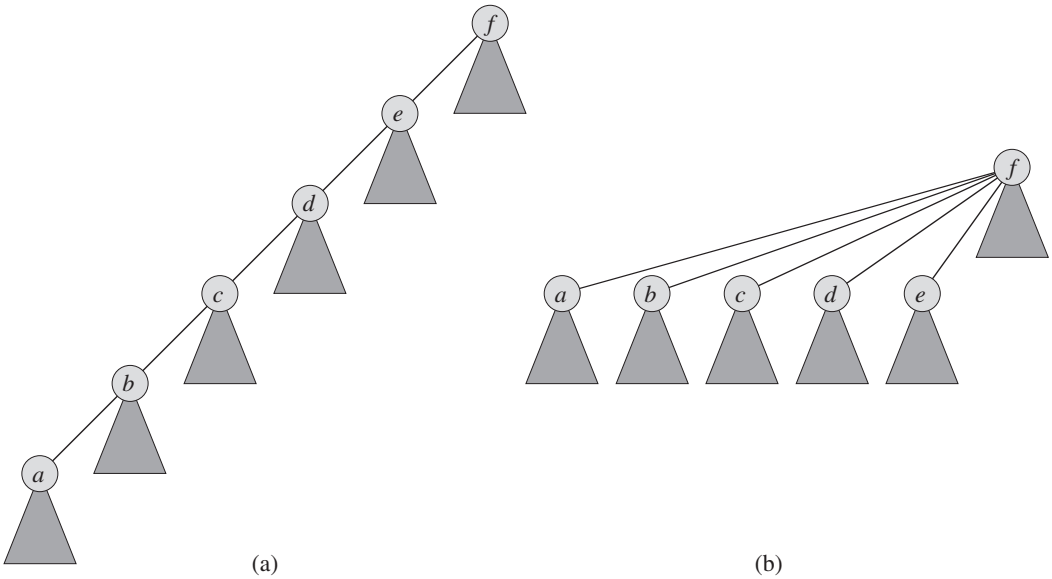


Figura 21.5 La compressione del cammino durante l'operazione FIND-SET. Le frecce e i cappi delle radici sono stati omessi. **(a)** Un albero che rappresenta un insieme prima di eseguire FIND-SET(a). I triangoli rappresentano i sottoalberi le cui radici sono i nodi indicati nella figura. Ogni nodo ha un puntatore a suo padre. **(b)** Lo stesso insieme dopo l'esecuzione di FIND-SET(a). Ogni nodo lungo il cammino di ricerca adesso punta direttamente alla radice.

```
UNION(x, y)
1  LINK(FIND-SET(x), FIND-SET(y))
```

```
LINK(x, y)
1  if x.rank > y.rank
2       y.p = x
3  else x.p = y
4       if x.rank == y.rank
5            y.rank = y.rank + 1
```

La procedura FIND-SET con la compressione del cammino è molto semplice.

```
FIND-SET(x)
1  if x ≠ x.p
2       x.p = FIND-SET(x.p)
3  return x.p
```

La procedura FIND-SET è un ***metodo a doppio passaggio***: durante il primo passaggio, risale il cammino di ricerca per trovare la radice; durante il secondo passaggio, discende il cammino di ricerca per aggiornare i nodi in modo che puntino direttamente alla radice. Ogni chiamata di FIND-SET(x) restituisce $x.p$ nella riga 3. Se x è la radice, allora la riga 2 non viene eseguita e viene restituito $x.p$ che è uguale a x. Questo è il caso in cui la ricorsione tocca il fondo. Altrimenti, viene eseguita la riga 2 e la chiamata ricorsiva con il parametro $x.p$ restituisce un puntatore alla radice. La riga 2 aggiorna il nodo x in modo che punti direttamente alla radice; la riga 3 restituisce questo puntatore.

Acquistato da Michele Michele su Webster il 2022-[illegible]

Effetto delle euristiche sul tempo di esecuzione

Sia l'unione per rango sia la compressione del cammino usate separatamente migliorano il tempo di esecuzione delle operazioni con le foreste di insiemi disgiunti; il miglioramento è ancora più grande se le due euristiche sono utilizzate insieme. Con la sola unione per rango si ottiene un tempo di esecuzione pari a $O(m \lg n)$ (vedere l'Esercizio 21.4-4) e questo limite è stretto (vedere l'Esercizio 21.3-3). Anche se non lo dimostreremo qui, tuttavia se ci sono n operazioni MAKE-SET (e quindi al più $n - 1$ operazioni UNION) ed f operazioni FIND-SET, con la sola euristica della compressione del cammino si ottiene un tempo di esecuzione nel caso peggiore pari a $\Theta(n + f \cdot (1 + \log_{2+f/n} n))$.

Quando utilizziamo sia l'unione per rango sia la compressione del cammino, il tempo di esecuzione nel caso peggiore è $O(m\, \alpha(n))$, dove $\alpha(n)$ è una funzione che cresce *molto* lentamente (sarà definita nel Paragrafo 21.4). In qualsiasi applicazione di una struttura dati per insiemi disgiunti, $\alpha(n) \leq 4$; quindi, possiamo considerare il tempo di esecuzione lineare in m in tutti casi pratici (anche se, a rigor di termini, esso è superlineare). Nel Paragrafo 21.4 dimostreremo questo limite superiore.

Esercizi

21.3-1

Svolgete l'Esercizio 21.2-2 utilizzando una foresta di insiemi disgiunti con l'unione per rango e la compressione del cammino.

21.3-2

Scrivete una versione non ricorsiva dell'operazione FIND-SET con la compressione del cammino.

21.3-3

Trovate una sequenza di m operazioni MAKE-SET, UNION e FIND-SET, n delle quali sono operazioni MAKE-SET, che impiega un tempo $\Omega(m \lg n)$ se viene utilizzata soltanto l'unione per rango.

21.3-4

Supponete di dovere aggiungere l'operazione PRINT-SET(x), che riceve un nodo x e stampa tutti gli elementi dell'insieme di x, in un ordine qualsiasi. Descrivete come aggiungere un singolo attributo a ciascun nodo in una foresta di insiemi disgiunti in modo che PRINT-SET(x) richieda un tempo lineare nel numero di elementi dell'insieme x e i tempi di esecuzione asintotici delle altre operazioni restino inalterati. Supponete di poter stampare ciascun elemento dell'insieme nel tempo $O(1)$.

21.3-5 ⋆

Dimostrate che qualsiasi sequenza di m operazioni MAKE-SET, FIND-SET e LINK, dove tutte le operazioni LINK figurano prima di qualsiasi operazione FIND-SET, impiega soltanto un tempo $O(m)$ se vengono utilizzate sia la compressione del cammino sia l'unione per rango. Che cosa accade nella stessa situazione se viene utilizzata soltanto l'euristica della compressione del cammino?

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950316-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

⋆ 21.4 Unione per rango con compressione del cammino

Come abbiamo notato nel Paragrafo 21.3, il tempo di esecuzione risultante dalla combinazione delle euristiche dell'unione per rango e della compressione del cammino è $O(m\, \alpha(n))$ per m operazioni di insiemi disgiunti su n elementi. In

questo paragrafo, esamineremo la funzione α per mostrare quanto lentamente essa cresca. Poi proveremo questo tempo di esecuzione utilizzando il metodo del potenziale dell'analisi ammortizzata.

Una funzione che cresce molto rapidamente e la sua inversa che cresce molto lentamente

Per $k \geq 0$ e $j \geq 1$ interi qualsiasi, definiamo la funzione $A_k(j)$ in questo modo

$$A_k(j) = \begin{cases} j+1 & \text{se } k = 0 \\ A_{k-1}^{(j+1)}(j) & \text{se } k \geq 1 \end{cases}$$

dove l'espressione $A_{k-1}^{(j+1)}(j)$ usa la notazione dell'iterazione di una funzione descritta nel Paragrafo 3.2.

Più precisamente, $A_{k-1}^{(0)}(j) = j$ e $A_{k-1}^{(i)}(j) = A_{k-1}(A_{k-1}^{(i-1)}(j))$ per $i \geq 1$. Il parametro k sarà chiamato ***livello*** della funzione A.

La funzione $A_k(j)$ cresce in senso stretto con j e k. Per capire quanto rapidamente cresca questa funzione, ricaviamo prima delle espressioni in forma chiusa per $A_1(j)$ e $A_2(j)$.

Lemma 21.2
Per qualsiasi intero $j \geq 1$, si ha $A_1(j) = 2j + 1$.

Dimostrazione Utilizziamo prima l'induzione su i per dimostrare che $A_0^{(i)}(j) = j + i$. Per il caso base, abbiamo $A_0^{(0)}(j) = j = j + 0$. Per il passo induttivo, supponiamo che $A_0^{(i-1)}(j) = j + (i-1)$. Allora $A_0^{(i)}(j) = A_0(A_0^{(i-1)}(j)) = (j+(i-1))+1 = j+i$. Infine, notiamo che $A_1(j) = A_0^{(j+1)}(j) = j+(j+1) = 2j+1$. ■

Lemma 21.3
Per qualsiasi intero $j \geq 1$, si ha $A_2(j) = 2^{j+1}(j+1) - 1$.

Dimostrazione Utilizziamo prima l'induzione su i per dimostrare che $A_1^{(i)}(j) = 2^i(j+1) - 1$. Per il caso base, abbiamo $A_1^{(0)}(j) = j = 2^0(j+1) - 1$. Per il passo induttivo, supponiamo che $A_1^{(i-1)}(j) = 2^{i-1}(j+1) - 1$. Allora $A_1^{(i)}(j) = A_1(A_1^{(i-1)}(j)) = A_1(2^{i-1}(j+1) - 1) = 2\cdot(2^{i-1}(j+1)-1)+1 = 2^i(j+1)-2+1 = 2^i(j+1)-1$. Infine, notiamo che $A_2(j) = A_1^{(j+1)}(j) = 2^{j+1}(j+1)-1$. ■

Adesso è possibile capire quanto la funzione $A_k(j)$ cresca rapidamente, esaminando semplicemente $A_k(1)$ per i livelli $k = 0, 1, 2, 3, 4$. Dalla definizione di $A_0(k)$ e dai precedenti lemmi, si ha $A_0(1) = 1 + 1 = 2$, $A_1(1) = 2 \cdot 1 + 1 = 3$ e $A_2(1) = 2^{1+1} \cdot (1+1) - 1 = 7$. Abbiamo anche

$$\begin{aligned} A_3(1) &= A_2^{(2)}(1) \\ &= A_2(A_2(1)) \\ &= A_2(7) \\ &= 2^8 \cdot 8 - 1 \\ &= 2^{11} - 1 \\ &= 2047 \end{aligned}$$

e

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned}
A_4(1) &= A_3^{(2)}(1) \\
&= A_3(A_3(1)) \\
&= A_3(2047) \\
&= A_2^{(2048)}(2047) \\
&\gg A_2(2047) \\
&= 2^{2048} \cdot 2048 - 1 \\
&> 2^{2048} \\
&= (2^4)^{512} \\
&= 16^{512} \\
&\gg 10^{80}
\end{aligned}$$

che è il numero stimato di atomi nell'universo osservabile. (Il simbolo "$\gg$" indica la relazione "molto maggiore di".)

Definiamo l'inverso della funzione $A_k(n)$, per qualsiasi intero $n \geq 0$, in questo modo

$$\alpha(n) = \min\{k : A_k(1) \geq n\}$$

Ovvero $\alpha(n)$ è il livello k più basso per il quale $A_k(1)$ è almeno pari a n. Dai precedenti valori di $A_k(1)$, abbiamo che

$$\alpha(n) = \begin{cases} 0 & \text{per } 0 \leq n \leq 2 \\ 1 & \text{per } n = 3 \\ 2 & \text{per } 4 \leq n \leq 7 \\ 3 & \text{per } 8 \leq n \leq 2047 \\ 4 & \text{per } 2048 \leq n \leq A_4(1) \end{cases}$$

È soltanto per quei valori eccezionalmente grandi di n (maggiori di $A_4(1)$, un numero enorme) che $\alpha(n) > 4$, quindi possiamo ritenere che $\alpha(n) \leq 4$ in tutte le applicazioni pratiche.

Proprietà dei ranghi

Nella parte restante di questo paragrafo, dimostreremo un limite $O(m\,\alpha(n))$ per il tempo di esecuzione delle operazioni degli insiemi disgiunti con l'unione per rango e la compressione del cammino. Per dimostrare questo limite, proviamo prima alcune semplici proprietà dei ranghi.

Lemma 21.4

Per tutti i nodi x, si ha $x.rank \leq x.p.rank$, dove la disuguaglianza è stretta se $x \neq x.p$. Il valore di $x.rank$ inizialmente è 0 e può aumentare nel tempo, ma soltanto finché $x = x.p$; non appena $x \neq x.p$ il valore di $x.rank$ non cambia più. Il valore di $x.p.rank$ cresce monotonicamente nel tempo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Dimostrazione La dimostrazione si ottiene per semplice induzione sul numero di operazioni, utilizzando le implementazioni di MAKE-SET, UNION e FIND-SET che abbiamo descritto nel Paragrafo 21.3. Lasciamo al lettore il compito di dimostrare questo lemma (Esercizio 21.4-1). ■

Corollario 21.5
Seguendo il cammino semplice da un nodo qualsiasi verso la radice, i ranghi dei nodi crescono in senso stretto. ■

Lemma 21.6
Il rango di qualsiasi nodo è al più $n - 1$.

Dimostrazione Il rango di ogni nodo inizia da 0 e aumenta soltanto dopo le operazioni LINK. Poiché ci sono al più $n - 1$ operazioni UNION, allora ci sono al più $n - 1$ operazioni LINK. Poiché ogni operazione LINK lascia tutti i ranghi inalterati oppure aumenta di un'unità il rango di qualche nodo, tutti i ranghi sono al più $n - 1$. ■

Il Lemma 21.6 fornisce un limite non stretto sui ranghi. In effetti, ogni nodo ha rango al più $\lfloor \lg n \rfloor$ (vedere l'Esercizio 21.4-2). Tuttavia, il limite meno stretto del Lemma 21.6 sarà sufficiente per i nostri scopi.

Dimostrazione del limite di tempo

Utilizzeremo il metodo del potenziale dell'analisi ammortizzata (vedere il Paragrafo 17.3) per dimostrare il limite di tempo $O(m\,\alpha(n))$. Per svolgere l'analisi ammortizzata, è preferibile supporre di utilizzare l'operazione LINK, anziché l'operazione UNION. Ovvero, poiché i parametri della procedura LINK sono puntatori alle due radici, supponiamo che le appropriate operazioni FIND-SET siano eseguite separatamente. Il seguente lemma dimostra che, anche se conteggiamo le operazioni extra FIND-SET indotte dalle chiamate di UNION, il tempo di esecuzione asintotico non cambia.

Lemma 21.7
Supponiamo di convertire una sequenza S' di m' operazioni MAKE-SET, UNION e FIND-SET in una sequenza S di m operazioni MAKE-SET, LINK e FIND-SET, trasformando ciascuna operazione UNION in due operazioni FIND-SET seguite da un'operazione LINK. Allora, se la sequenza S viene eseguita nel tempo $O(m\,\alpha(n))$, la sequenza S' viene eseguita nel tempo $O(m'\,\alpha(n))$.

Dimostrazione Poiché ogni operazione UNION nella sequenza S' viene trasformata in tre operazioni in S, si ha $m' \leq m \leq 3m'$. Poiché $m = O(m')$, un limite di tempo $O(m\,\alpha(n))$ per la sequenza convertita S implica un limite di tempo $O(m'\,\alpha(n))$ per la sequenza originale S'. ■

Nella parte restante di questo paragrafo supporremo che la sequenza iniziale di m' operazioni MAKE-SET, UNION e FIND-SET sia stata convertita in una sequenza di m operazioni MAKE-SET, LINK e FIND-SET. Dimostreremo un limite di tempo pari a $O(m\,\alpha(n))$ per la sequenza convertita e utilizzeremo il Lemma 21.7 per dimostrare che la sequenza originale di m' operazioni ha un tempo di esecuzione pari a $O(m'\,\alpha(n))$.

Acquistato da Michele Michele su Webster il 2022-0[illegible]

La funzione potenziale

La funzione potenziale che utilizziamo assegna un potenziale $\phi_q(x)$ a ciascun nodo x nella foresta degli insiemi disgiunti dopo q operazioni. Sommiamo i potenziali dei nodi per ottenere il potenziale dell'intera foresta: $\Phi_q = \sum_x \phi_q(x)$,

dove Φ_q indica il potenziale della foresta dopo q operazioni. Poiché la foresta è vuota prima della prima operazione, poniamo arbitrariamente $\Phi_0 = 0$. Nessun potenziale Φ_q sarà negativo.

Il valore di $\phi_q(x)$ dipende dal fatto che x sia la radice di un albero dopo la q-esima operazione. Se lo è o se $x.rank = 0$, allora $\phi_q(x) = \alpha(n) \cdot x.rank$.

Adesso supponiamo che, dopo la q-esima operazione, x non sia una radice e che $x.rank \geq 1$. Bisogna definire due funzioni ausiliarie di x prima di poter definire $\phi_q(x)$. In primo luogo definiamo

$$\text{level}(x) = \max\{k : x.p.rank \geq A_k(x.rank)\}$$

Ovvero level(x) è il massimo livello k per il quale la funzione A_k, applicata al rango di x, non è più grande del rango del padre di x.

Asseriamo che

$$0 \leq \text{level}(x) < \alpha(n) \tag{21.1}$$

che possiamo dimostrare nel seguente modo. Abbiamo

$$\begin{aligned} x.p.rank &\geq x.rank + 1 && \text{(per il Lemma 21.4)} \\ &= A_0(x.rank) && \text{(per definizione di } A_0(j)) \end{aligned}$$

che implica che level$(x) \geq 0$; inoltre abbiamo

$$\begin{aligned} A_{\alpha(n)}(x.rank) &\geq A_{\alpha(n)}(1) && \text{(perché } A_k(j) \text{ è crescente in senso stretto)} \\ &\geq n && \text{(per la definizione di } \alpha(n)) \\ &> x.p.rank && \text{(per il Lemma 21.6)} \end{aligned}$$

che implica che level$(x) < \alpha(n)$. Notate che, poiché $x.p.rank$ cresce monotonicamente nel tempo, anche level(x) cresce monotonicamente nel tempo.

La seconda funzione ausiliaria si applica quando $x.rank \geq 1$:

$$\text{iter}(x) = \max\left\{i : x.p.rank \geq A^{(i)}_{\text{level}(x)}(x.rank)\right\}$$

Ovvero iter(x) è il numero massimo di volte che possiamo applicare iterativamente $A_{\text{level}(x)}$, applicata inizialmente al rango di x, prima di ottenere un valore più grande del rango del padre di x.

Asseriamo che quando $x.rank \geq 1$, abbiamo

$$1 \leq \text{iter}(x) \leq x.rank \tag{21.2}$$

che possiamo dimostrare nel seguente modo. Abbiamo

$$\begin{aligned} x.p.rank &\geq A_{\text{level}(x)}(x.rank) && \text{(per la definizione di level}(x)) \\ &= A^{(1)}_{\text{level}(x)}(x.rank) && \text{(per la definizione di iterazione di una funzione)} \end{aligned}$$

che implica che iter$(x) \geq 1$; inoltre abbiamo

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned} A^{(x.rank+1)}_{\text{level}(x)}(x.rank) &= A_{\text{level}(x)+1}(x.rank) && \text{(per definizione di } A_k(j)) \\ &> x.p.rank && \text{(per definizione di level}(x)) \end{aligned}$$

che implica che iter$(x) \leq x.rank$. Notate che, poiché $x.p.rank$ cresce monotonicamente nel tempo, affinché iter(x) possa diminuire, level(x) deve crescere. Se level(x) non cambia, iter(x) deve crescere o restare invariata.

Una volta definite queste funzioni ausiliarie, possiamo definire il potenziale del nodo x dopo q operazioni:

$$\phi_q(x) = \begin{cases} \alpha(n) \cdot x.rank & \text{se } x \text{ è una radice o} \\ & x.rank = 0 \\ (\alpha(n) - \text{level}(x)) \cdot x.rank - \text{iter}(x) & \text{se } x \text{ non è una radice e} \\ & x.rank \geq 1 \end{cases}$$

Esaminiamo qualche altra utile proprietà dei potenziali dei nodi.

Lemma 21.8
Per ogni nodo x e per qualsiasi numero di operazioni q, si ha

$$0 \leq \phi_q(x) \leq \alpha(n) \cdot x.rank$$

Dimostrazione Se x è una radice o $x.rank = 0$, allora $\phi_q(x) = \alpha(n) \cdot x.rank$ per definizione. Supponiamo adesso che x non sia una radice e che $x.rank \geq 1$. Otteniamo un limite inferiore per $\phi_q(x)$, massimizzando $\text{level}(x)$ e $\text{iter}(x)$. Per il limite (21.1), $\text{level}(x) \leq \alpha(n) - 1$ e, per il limite (21.2), $\text{iter}(x) \leq x.rank$; quindi

$$\begin{aligned} \phi_q(x) &= (\alpha(n) - \text{level}(x)) \cdot x.rank - \text{iter}(x) \\ &\geq (\alpha(n) - (\alpha(n) - 1)) \cdot x.rank - x.rank \\ &= x.rank - x.rank \\ &= 0 \end{aligned}$$

Analogamente, otteniamo un limite superiore per $\phi_q(x)$ minimizzando $\text{level}(x)$ e $\text{iter}(x)$. Per il limite (21.1), $\text{level}(x) \geq 0$ e, per il limite (21.2), $\text{iter}(x) \geq 1$. Quindi,

$$\begin{aligned} \phi_q(x) &\leq (\alpha(n) - 0) \cdot x.rank - 1 \\ &= \alpha(n) \cdot x.rank - 1 \\ &< \alpha(n) \cdot x.rank \end{aligned}$$

■

Corollario 21.9
Se il nodo x non è una radice e $x.rank > 0$, allora $\phi_q(x) < \alpha(n) \cdot x.rank$. ■

Variazioni di potenziale e costi ammortizzati delle operazioni

A questo punto siamo in grado di analizzare come le operazioni degli insiemi disgiunti influiscono sui potenziali dei nodi. Conoscendo la variazione del potenziale provocata da ciascuna operazione, possiamo determinare il costo ammortizzato di ogni operazione.

Lemma 21.10
Indichiamo con x un nodo che non è una radice e supponiamo che la q-esima operazione sia LINK o FIND-SET. Allora, dopo la q-esima operazione, $\phi_q(x) \leq \phi_{q-1}(x)$. Inoltre, se $x.rank \geq 1$ e il valore di $\text{level}(x)$ o di $\text{iter}(x)$ cambia a causa della q-esima operazione, allora $\phi_q(x) \leq \phi_{q-1}(x) - 1$. Ovvero il potenziale di x non può aumentare e, se x ha rango positivo e il valore di $\text{level}(x)$ o di $\text{iter}(x)$ cambia, allora il potenziale di x diminuisce di almeno un'unità.

Acquistato da Michele Michele su Webster il 2022-0[illegible]

Dimostrazione Poiché x non è una radice, la q-esima operazione non cambia $x.rank$ e, poiché n non varia dopo le n operazioni MAKE-SET iniziali, anche $\alpha(n)$ non varia. Quindi, queste componenti della formula del potenziale di x restano inalterate dopo la q-esima operazione. Se $x.rank = 0$, allora $\phi_q(x) = \phi_{q-1}(x) = 0$. Supponiamo adesso che $x.rank \geq 1$.

Ricordiamo che level(x) cresce monotonicamente nel tempo. Se la q-esima operazione non cambia il valore di level(x), allora il valore di iter(x) aumenta o resta inalterato. Se entrambe le funzioni level(x) e iter(x) restano inalterate, allora $\phi_q(x) = \phi_{q-1}(x)$. Se level$(x)$ non cambia e iter(x) aumenta, allora il valore di iter(x) cresce di almeno un'unità e, quindi, $\phi_q(x) \leq \phi_{q-1}(x) - 1$.

Infine, se la q-esima operazione aumenta level(x), questa funzione cresce di almeno un'unità, quindi il valore del termine $(\alpha(n) - \text{level}(x)) \cdot x.rank$ si riduce di almeno $x.rank$. Poiché il valore di level(x) è aumentato, iter(x) potrebbe diminuire, ma per il limite (21.2), tale diminuzione è al più $x.rank - 1$. Quindi, l'aumento del potenziale dovuto alla variazione di iter(x) è minore della riduzione del potenziale provocata dalla variazione di level(x); concludiamo che $\phi_q(x) \leq \phi_{q-1}(x) - 1$. ■

I tre ultimi lemmi dimostrano che il costo ammortizzato di ciascuna operazione MAKE-SET, LINK e FIND-SET è $O(\alpha(n))$. Ricordiamo dall'equazione (17.2) che il costo ammortizzato di ciascuna operazione è pari al suo costo effettivo più l'aumento del potenziale dovuto all'operazione.

Lemma 21.11
Il costo ammortizzato di ciascuna operazione MAKE-SET è $O(1)$.

Dimostrazione Supponiamo che la q-esima operazione sia MAKE-SET(x). Questa operazione crea il nodo x con rango 0, quindi $\phi_q(x) = 0$. Non ci sono variazioni di ranghi o potenziali, pertanto $\Phi_q = \Phi_{q-1}$. Osservando che il costo effettivo dell'operazione MAKE-SET è $O(1)$, la dimostrazione è completa. ■

Lemma 21.12
Il costo ammortizzato di ciascuna operazione LINK è $O(\alpha(n))$.

Dimostrazione Supponiamo che la q-esima operazione sia LINK(x, y). Il costo effettivo dell'operazione LINK è $O(1)$. Senza perdere in generalità, supponiamo che LINK trasformi y nel padre di x.

Per determinare la variazione di potenziale dovuta a LINK, notiamo che gli unici nodi il cui potenziale può variare sono x, y e i figli di y appena prima dell'operazione. Dimostreremo che l'unico nodo il cui potenziale può aumentare a causa di LINK è y e che questo aumento è al più $\alpha(n)$:

- Per il Lemma 21.10, qualsiasi nodo che è figlio di y appena prima dell'operazione LINK non può aumentare il suo potenziale a causa di LINK.
- Per la definizione di $\phi_q(x)$, notiamo che, poiché x era una radice appena prima della q-esima operazione, $\phi_{q-1}(x) = \alpha(n) \cdot x.rank$. Se $x.rank = 0$, allora $\phi_q(x) = \phi_{q-1}(x) = 0$, altrimenti

  $$\begin{aligned} \phi_q(x) &< \alpha(n) \cdot x.rank \quad \text{(per il Corollario 21.9)} \\ &= \phi_{q-1}(x) \end{aligned}$$

  Quindi il potenziale di x diminuisce.

Acquistato da Michele Michele su Ateneo in 12/2017 © The McGraw-Hill Companies, S.r.l. - Copyright Apparati Privati McGraw-Hill Education (Italy)

- Poiché y è una radice prima di eseguire l'operazione LINK, allora $\phi_{q-1}(y) = \alpha(n) \cdot rank[y]$. L'operazione LINK lascia y come radice, non cambia il rango di y oppure lo aumenta di un'unità; ne consegue che $\phi_q(y) = \phi_{q-1}(y)$ oppure $\phi_q(y) = \phi_{q-1}(y) + \alpha(n)$.

L'aumento di potenziale dovuto all'operazione LINK, quindi, è al più $\alpha(n)$. Il costo ammortizzato dell'operazione LINK è $O(1) + \alpha(n) = O(\alpha(n))$. ■

Lemma 21.13
Il costo ammortizzato di ciascuna operazione FIND-SET è $O(\alpha(n))$.

Dimostrazione Supponiamo che la q-esima operazione sia FIND-SET e che il cammino semplice di ricerca contenga s nodi. Il costo effettivo dell'operazione FIND-SET è $O(s)$. Dimostreremo che l'operazione FIND-SET non provoca alcun aumento di potenziale in nessun nodo e che almeno $\max(0, s - (\alpha(n) + 2))$ nodi nel cammino di ricerca riducono il loro potenziale di almeno un'unità.

Per provare che nessun nodo aumenta il suo potenziale, applichiamo il Lemma 21.10 a tutti i nodi, tranne alla radice. Se x è la radice, allora il suo potenziale è $\alpha(n) \cdot x.rank$, che non varia.

Adesso proviamo che almeno $\max(0, s - (\alpha(n) + 2))$ nodi riducono il loro potenziale di almeno un'unità. Indichiamo con x un nodo nel cammino di ricerca tale che $x.rank > 0$ e x sia seguito da un altro nodo y che non è una radice, dove $\text{level}(y) = \text{level}(x)$ appena prima dell'operazione FIND-SET (non è necessario che il nodo y segua *immediatamente* x nel cammino di ricerca). Tutti i nodi (tranne al più $\alpha(n) + 2$ nodi) nel cammino di ricerca soddisfano queste limitazioni su x. Quelli che non le soddisfano sono il primo nodo nel cammino di ricerca (se ha rango 0), l'ultimo nodo nel cammino (cioè la radice) e l'ultimo nodo w nel cammino per il quale $\text{level}(w) = k$, per ogni $k = 0, 1, 2, \ldots, \alpha(n) - 1$.

Per tale nodo x dimostreremo che il suo potenziale diminuisce di almeno un'unità. Sia $k = \text{level}(x) = \text{level}(y)$. Appena prima della compressione del cammino provocata da FIND-SET, si ha

$$\begin{aligned}
x.p.rank &\geq A_k^{(\text{iter}(x))}(x.rank) && \text{(per definizione di iter}(x)\text{)} \\
y.p.rank &\geq A_k(y.rank) && \text{(per definizione di level}(y)\text{)} \\
y.rank &\geq x.p.rank && \text{(per il Corollario 21.5 e perché } y \text{ segue } x \text{ nel cammino di ricerca)}
\end{aligned}$$

Raggruppando queste disequazioni e indicando con i il valore di $\text{iter}(x)$ prima della compressione del cammino, si ha

$$\begin{aligned}
y.p.rank &\geq A_k(y.rank) \\
&\geq A_k(x.p.rank) && \text{(perché } A_k(j) \text{ cresce in senso stretto)} \\
&\geq A_k(A_k^{(\text{iter}(x))}(x.rank)) \\
&= A_k^{(i+1)}(x.rank)
\end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Poiché la compressione del cammino fa sì che x e y abbiano lo stesso padre, sappiamo che, dopo la compressione del cammino, $x.p.rank = y.p.rank$, e che la compressione del cammino non riduce $y.p.rank$. Poiché $x.rank$ non varia, dopo la compressione del cammino si ha che $x.p.rank \geq A_k^{(i+1)}(x.rank)$. Quindi, la

compressione del cammino fa sì che aumenti iter(x) (almeno fino a $i + 1$) o level(x) (questo accade se iter(x) aumenta almeno fino a $x.rank + 1$). In entrambi i casi, per il Lemma 21.10, si ha $\phi_q(x) \leq \phi_{q-1}(x) - 1$. Quindi, il potenziale di x diminuisce di almeno un'unità.

Il costo ammortizzato dell'operazione FIND-SET è pari al costo effettivo più la variazione di potenziale. Il costo effettivo è $O(s)$ e abbiamo dimostrato che il potenziale totale diminuisce di almeno $\max(0, s-(\alpha(n)+2))$. Il costo ammortizzato, quindi, è al massimo $O(s) - (s - (\alpha(n) + 2)) = O(s) - s + O(\alpha(n)) = O(\alpha(n))$, perché possiamo scegliere opportunamente le unità di potenziale per dominare la costante nascosta in $O(s)$. ■

Raggruppando i precedenti lemmi, si ottiene il seguente teorema.

Teorema 21.14
Una sequenza di m operazioni MAKE-SET, UNION e FIND-SET, n delle quali sono operazioni MAKE-SET, può essere eseguita, nel caso peggiore, in una foresta di insiemi disgiunti con l'unione per rango e la compressione del cammino nel tempo $O(m\,\alpha(n))$.

Dimostrazione Immediata dai Lemmi 21.7, 21.11, 21.12 e 21.13. ■

Esercizi

21.4-1
Dimostrate il Lemma 21.4.

21.4-2
Dimostrate che ogni nodo ha rango al più $\lfloor \lg n \rfloor$.

21.4-3
Alla luce dell'Esercizio 21.4-2, quanti bit sono necessari per memorizzare $x.rank$ per ciascun nodo x?

21.4-4
Utilizzando l'Esercizio 21.4-2, dimostrate che le operazioni in una foresta di insiemi disgiunti con l'unione per rango, ma senza la compressione del cammino, vengono eseguite nel tempo $O(m \lg n)$.

21.4-5
Il professor Dante ritiene che, poiché i ranghi dei nodi crescono in senso stretto lungo un cammino verso la radice, i livelli dei nodi devono crescere monotonicamente lungo il cammino. In altre parole, se $x.rank > 0$ e $x.p$ non è una radice, allora level(x) $\leq$ level($x.p$). Il professore ha ragione?

21.4-6 ★
Consideriamo la funzione $\alpha'(n) = \min\{k : A_k(1) \geq \lg(n + 1)\}$. Dimostrate che $\alpha'(n) \leq 3$ per tutti i valori possibili di n e, utilizzando l'Esercizio 21.4-2, spiegate come modificare l'argomento della funzione potenziale per provare che una sequenza di m operazioni MAKE-SET, UNION e FIND-SET, n delle quali sono operazioni MAKE-SET, può essere eseguita, nel caso peggiore, in una foresta di insiemi disgiunti con l'unione per rango e la compressione del cammino nel tempo $O(m\,\alpha'(n))$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Problemi

21-1 Mimimo off-line

Il ***problema del minimo off-line*** richiede di mantenere un insieme dinamico T di elementi del dominio $\{1, 2, \ldots, n\}$ con le operazioni INSERT ed EXTRACT-MIN. Data una sequenza S di n chiamate INSERT e m chiamate EXTRACT-MIN, dove ogni chiave in $\{1, 2, \ldots, n\}$ viene inserita una sola volta, vogliamo determinare quale chiave viene restituita da ciascuna chiamata EXTRACT-MIN. Specificatamente, vogliamo riempire un array $extracted[1..m]$ dove, per $i = 1, 2, \ldots, m$, $extracted[i]$ è la chiave restituita dalla i-esima chiamata EXTRACT-MIN. Il problema è "off-line" nel senso che ci è consentito elaborare l'intera sequenza S prima di determinare una qualsiasi delle chiavi restituite.

a. Nella seguente istanza del problema del minimo off-line, ogni operazione INSERT è rappresentata da un numero e ogni operazione EXTRACT-MIN è rappresentata dalla lettera E:

4, 8, E, 3, E, 9, 2, 6, E, E, E, 1, 7, E, 5

Inserite i valori corretti nell'array *extracted*.

Per sviluppare un algoritmo che risolve questo problema, suddividiamo la sequenza S in sottosequenze omogenee; ovvero rappresentiamo S con

$$\mathrm{I}_1, \mathrm{E}, \mathrm{I}_2, \mathrm{E}, \mathrm{I}_3, \ldots, \mathrm{I}_m, \mathrm{E}, \mathrm{I}_{m+1}$$

dove E rappresenta una singola chiamata EXTRACT-MIN e I_j rappresenta una sequenza (eventualmente vuota) di chiamate INSERT. Per ciascuna sottosequenza I_j, inizialmente poniamo le chiavi inserite da queste operazioni in un insieme K_j, che è vuoto se I_j è vuoto. Poi, eseguiamo il seguente pseudocodice.

```
OFF-LINE-MINIMUM(m, n)
1  for i = 1 to n
2      determina j tale che i ∈ K_j
3      if j ≠ m + 1
4          extracted[j] = i
5          sia l il più piccolo valore che è maggiore di j
               per il quale esiste l'insieme K_l
6          K_l = K_j ∪ K_l, distruggendo K_j
7  return extracted
```

b. Dimostrate che l'array *extracted* restituito da OFF-LINE-MINIMUM è corretto.

c. Descrivete come implementare OFF-LINE-MINIMUM in modo efficiente con una struttura dati di insiemi disgiunti. Specificate un limite stretto per il tempo di esecuzione nel caso peggiore della vostra implementazione.

Acquistato da Michele Michele su Webster il 2022-07-07 15:32 Numero Ordine Libreria 2022-0016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

21-2 Determinare la profondità

Nel ***problema di determinare la profondità*** manteniamo una foresta $\mathcal{F} = \{T_i\}$ di alberi radicati con tre operazioni:

MAKE-TREE(v) crea un albero il cui unico nodo è v.

FIND-DEPTH(v) restituisce la profondità del nodo v all'interno del suo albero.

GRAFT(r, v) trasforma il nodo r, che si suppone essere la radice di un albero, nel figlio del nodo v, che si suppone appartenere a un albero diverso da r, ma che non necessariamente ne è la radice.

a. Supponete di utilizzare una rappresentazione con alberi simile a una foresta di insiemi disgiunti: $v.p$ è il padre del nodo v, tranne che $v.p = v$ se v è una radice. Supponete inoltre di implementare l'operazione GRAFT(r, v) impostando $r.p = v$ e l'operazione FIND-DEPTH(v) in modo che risalga il cammino di ricerca fino alla radice restituendo il totale di tutti i nodi incontrati escluso v. Dimostrate che il tempo di esecuzione nel caso peggiore di una sequenza di m operazioni MAKE-TREE, FIND-DEPTH e GRAFT è $\Theta(m^2)$.

Utilizzando le euristiche dell'unione per rango e della compressione del cammino, è possibile ridurre il tempo di esecuzione nel caso peggiore. Utilizziamo la foresta degli insiemi disgiunti $\mathcal{S} = \{S_i\}$, dove a ciascun insieme S_i (che è esso stesso un albero) viene associato un albero T_i nella foresta $\mathcal{F}$. La struttura dell'albero di un insieme S_i, tuttavia, non corrisponde necessariamente a quella di T_i. In effetti, l'implementazione di S_i non registra l'esatta relazione padre-figlio in T_i, ma ciononostante consente di determinare la profondità di un nodo qualsiasi in T_i.

L'idea chiave consiste nel mantenere in ciascun nodo v una "pseudodistanza" $v.d$, che è definita in maniera tale che la somma delle pseudodistanze lungo il cammino semplice da v alla radice del suo insieme S_i sia uguale alla profondità di v in T_i. Ovvero, se il cammino semplice da v alla sua radice in S_i è $v_0, v_1, \ldots, v_k$, dove $v_0 = v$ e v_k è la radice di S_i, allora la profondità di v in T_i è $\sum_{j=0}^{k} v_j.d$.

b. Implementate l'operazione MAKE-TREE.

c. Spiegate come modificare FIND-SET per implementare FIND-DEPTH. La vostra implementazione dovrebbe effettuare la compressione del cammino e il suo tempo di esecuzione dovrebbe essere lineare nella lunghezza del cammino di ricerca. Verificate che la vostra implementazione aggiorni correttamente le pseudodistanze.

d. Spiegate come implementare l'operazione GRAFT(r, v), che combina gli insiemi che contengono r e v, modificando le procedure UNION e LINK. Verificate che la vostra implementazione aggiorni correttamente le pseudodistanze. Notate che la radice di un insieme S_i non è necessariamente la radice del corrispondente albero T_i.

e. Specificate un limite stretto per il tempo di esecuzione nel caso peggiore di una sequenza di m operazioni MAKE-TREE, FIND-DEPTH e GRAFT, n delle quali sono operazioni MAKE-TREE.

21-3 L'algoritmo di Tarjan per i minimi comuni antenati off-line

Il ***minimo comune antenato*** di due nodi u e v in un albero radicato T è il nodo w che è un antenato di entrambi i nodi u e v e che ha la massima profondità in T.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Il ***problema dei minimi comuni antenati off-line*** è così definito: dati un albero radicato T e un insieme arbitrario $P = \{\{u, v\}\}$ di coppie non ordinate di nodi in T, si vuole determinare il minimo comune antenato di ciascuna coppia in P.

Per risolvere il problema dei minimi comuni antenati off-line, la seguente procedura effettua un attraversamento dell'albero T con la chiamata iniziale LCA$(T.root)$. Si suppone che ciascun nodo sia di colore WHITE prima dell'attraversamento.

```
LCA(u)
 1  MAKE-SET(u)
 2  FIND-SET(u).ancestor = u
 3  for ogni figlio v di u in T
 4      LCA(v)
 5      UNION(u, v)
 6      FIND-SET(u).ancestor = u
 7  u.color = BLACK
 8  for ogni nodo v tale che {u, v} ∈ P
 9      if v.color == BLACK
10          stampa "Il minimo comune antenato di"
                u "e" v "è" FIND-SET(v).ancestor
```

a. Dimostrate che la riga 10 viene eseguita esattamente una volta per ogni coppia $\{u, v\} \in P$.

b. Dimostrate che, quando viene chiamata LCA(u), il numero di insiemi nella struttura dati degli insiemi disgiunti è uguale alla profondità di u in T.

c. Dimostrate che la procedura LCA stampa correttamente il minimo comune antenato di u e v per ogni coppia $\{u, v\} \in P$.

d. Analizzate il tempo di esecuzione della procedura LCA, supponendo di utilizzare l'implementazione della struttura dati per gli insiemi disgiunti descritta nel Paragrafo 21.3.

Note

Molti risultati importanti sulle strutture dati per gli insiemi disgiunti sono dovuti almeno in parte a R. E. Tarjan. Applicando il metodo dell'aggregazione, Tarjan [329, 331] ha fornito il primo limite superiore stretto in termini della funzione inversa $\widehat{\alpha}(m, n)$ della funzione di Ackermann. (La funzione $A_k(j)$ descritta nel Paragrafo 21.4 è simile alla funzione di Ackermann e la funzione $\alpha(n)$ è simile alla funzione inversa. Entrambe le funzioni $\alpha(n)$ e $\widehat{\alpha}(m, n)$ sono al più 4 per tutti i valori possibili di m ed n.) Un limite superiore pari a $O(m \lg^* n)$ era stato dimostrato in precedenza da Hopcroft e Ullman [5, 180]. La descrizione fatta nel Paragrafo 21.4 è un adattamento di una successiva analisi di Tarjan [333] che, a sua volta, si basa su un'analisi di Kozen [221]. Harfst e Reingold [162] hanno sviluppato una versione basata sul potenziale del precedente limite di Tarjan.

Tarjan e van Leeuwen [334] hanno analizzato alcune varianti dell'euristica della compressione del cammino, inclusi i "metodi a singolo passaggio", che a volte offrono fattori costanti migliori nelle loro prestazioni rispetto ai metodi a doppio passaggio. Come le prime analisi di Tarjan della normale euristica della compressione del cammino, le analisi di Tarjan e van Leeuwen si basano sul metodo dell'aggregazione. Successivamente, Harfst e Reingold [162] hanno mostrato come modificare la funzione potenziale per adattare la loro analisi della compressione del cammino a queste varianti a singolo passaggio. Gabow e Tarjan [122] hanno dimostrato che in certe applicazioni, le operazioni con gli insiemi disgiunti possono essere eseguite nel tempo $O(m)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Tarjan [330] ha dimostrato che occorre un limite inferiore di tempo $\Omega(m\, \widehat{\alpha}(m, n))$ per eseguire le operazioni con qualsiasi struttura dati per insiemi disgiunti che soddisfa determinate condizioni tecniche. Questo limite inferiore è stato successivamente generalizzato da Fredman e Saks [114], che hanno dimostrato che, nel caso peggiore, bisogna accedere a $\Omega(m\, \widehat{\alpha}(m, n))$ parole di memoria di $\lg n$ bit.

VI Algoritmi per grafi

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Introduzione

I grafi sono strutture dati molto comuni in informatica e gli algoritmi che operano con essi sono di fondamentale importanza in questo campo. Ci sono centinaia di interessanti problemi computazionali che sono definiti in termini di grafi. In questa parte ne esamineremo alcuni dei più significativi.

Il Capitolo 22 spiega come rappresentare un grafo in un computer; poi esamina gli algoritmi che effettuano le ricerche in un grafo utilizzando una visita in ampiezza (breadth-first search) o una visita in profondità (depth-first search). Saranno descritte due applicazioni della visita in profondità: l'ordinamento topologico di un grafo orientato aciclico e la scomposizione di un grafo orientato nelle sue componenti fortemente connesse.

Il Capitolo 23 descrive come calcolare un albero di connessione di peso minimo per un grafo. Tale albero è definito come il modo meno pesante di collegare tutti i vertici del grafo quando a ciascun arco è associato un peso. Gli algoritmi per calcolare gli alberi di connessione minimi sono buoni esempi di algoritmi golosi (vedere il Capitolo 16).

I Capitoli 24 e 25 considerano il problema di calcolare i cammini minimi fra i vertici quando ciascun arco è associato a una lunghezza o "peso". Il Capitolo 24 considera il calcolo dei cammini minimi da un dato vertice sorgente a tutti gli altri vertici; il Capitolo 25 considera il calcolo dei cammini minimi fra tutte le coppie di vertici.

Infine, il Capitolo 26 spiega come calcolare un flusso massimo di materiale in una rete (grafo orientato) che ha una specifica sorgente di materiale, uno specifico pozzo e specifiche capacità per la quantità di materiale che può attraversare ciascun arco orientato. Questo problema generale si presenta in numerose forme e un buon algoritmo che calcola i flussi massimi può essere utilizzato per risolvere con efficienza vari problemi correlati.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Nel descrivere il tempo di esecuzione di un algoritmo per un dato grafo $G = (V, E)$, di solito, esprimiamo la dimensione dell'input in funzione del numero di vertici $|V|$ e del numero di archi $|E|$ del grafo. Ovvero, ci sono due parametri importanti che descrivono la dimensione dell'input, non uno solo. Adotteremo una notazione comune per questi parametri. Nella notazione asintotica (come la notazione O o la notazione Θ) e *soltanto* all'interno di questa notazione, il simbolo V indica $|V|$ e il simbolo E indica $|E|$. Per esempio, se scriviamo "l'algoritmo viene

eseguito nel tempo $O(VE)$", intendiamo che l'algoritmo viene eseguito nel tempo $O(|V||E|)$. Questa convenzione semplifica la lettura delle formule dei tempi di esecuzione, senza rischi di ambiguità.

Un'altra convenzione che adottiamo riguarda la pseudocodifica. Indichiamo con $G.V$ l'insieme dei vertici di un grafo G e con $G.E$ l'insieme degli archi del grafo. In altre parole, lo pseudocodice vede gli insiemi dei vertici e degli archi come attributi di un grafo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Algoritmi elementari per grafi 22

Questo capitolo descrive i metodi per rappresentare un grafo e per effettuare delle ricerche in un grafo. Effettuare delle ricerche in un grafo significa seguire sistematicamente gli archi del grafo in modo da visitare i vertici del grafo. Un algoritmo di ricerca in un grafo può scoprire molte cose sulla struttura di un grafo. Molti algoritmi iniziano ricercando nel loro grafo di input per ottenere queste informazioni strutturali. Altri algoritmi per grafi sono organizzati come semplici elaborazioni di algoritmi di ricerca di base. Le tecniche di ricerca in un grafo sono il cuore degli algoritmi che operano con i grafi.

Il Paragrafo 22.1 descrive le due più comuni rappresentazioni computazionali dei grafi: le liste di adiacenza e le matrici di adiacenza. Il Paragrafo 22.2 presenta un semplice algoritmo di ricerca per grafi, detto visita in ampiezza (breadth-first search), e spiega come creare l'albero risultante da una visita in ampiezza. Il Paragrafo 22.3 descrive il processo di ricerca in profondità e prova alcuni risultati standard sull'ordine in cui la ricerca in profondità visita i vertici. Il Paragrafo 22.4 descrive la prima applicazione reale della visita in profondità: l'ordinamento topologico di un grafo orientato aciclico. Una seconda applicazione della visita in profondità – trovare le componenti fortemente connesse di un grafo orientato – è descritta nel Paragrafo 22.5.

22.1 Rappresentazione dei grafi

Ci sono due metodi standard per rappresentare un grafo $G = (V, E)$: come una collezione di liste di adiacenza o come una matrice di adiacenza. Entrambi i metodi possono essere applicati sia ai grafi orientati che a quelli non orientati. Di solito, si preferisce la rappresentazione con liste di adiacenza, perché permette di rappresentare in modo compatto i grafi ***sparsi*** – quelli in cui $|E|$ è molto più piccolo di $|V|^2$. Per la maggior parte degli algoritmi per grafi descritti in questo libro si suppone che un grafo di input sia rappresentato tramite liste di adiacenza. Tuttavia, potrebbe essere preferibile una rappresentazione con matrice di adiacenza, quando il grafo è ***denso*** – $|E|$ è prossimo a $|V|^2$ – o quando dobbiamo essere in grado di dire rapidamente se c'è un arco che collega due vertici particolari. Per esempio, due degli algoritmi per cammini minimi fra tutte le coppie di vertici, descritti nel Capitolo 25, suppongono che i loro grafi di input siano rappresentati da matrici di adiacenza.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

La ***rappresentazione con liste di adiacenza*** di un grafo $G = (V, E)$ consiste in un array Adj di $|V|$ liste, una per ogni vertice in V. Per ogni $u \in V$, la lista di adiacenza $Adj[u]$ contiene tutti i vertici v tali che esista un arco $(u, v) \in E$. Ovvero $Adj[u]$ include tutti i vertici adiacenti a u in G (in alternativa, potrebbe contenere i puntatori a questi vertici). Poiché le liste di adiacenza rappresentano gli archi di un grafo, nello pseudocodice trattiamo l'array Adj come un attributo

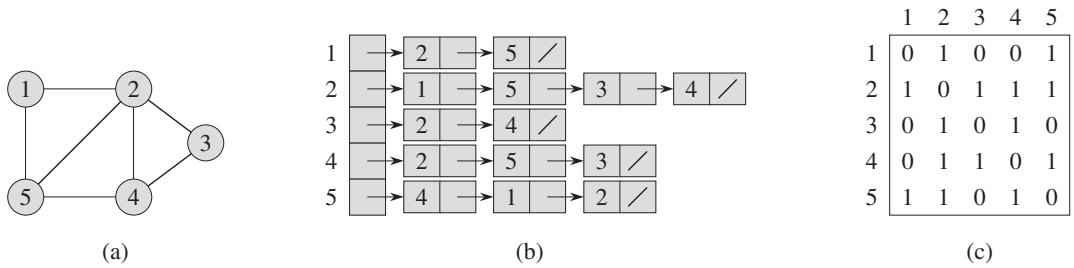


Figura 22.1 Due rappresentazioni di un grafo non orientato. **(a)** Un grafo non orientato G con cinque vertici e sette archi. **(b)** Una rappresentazione con liste di adiacenza di G. **(c)** La rappresentazione con matrice di adiacenza di G.

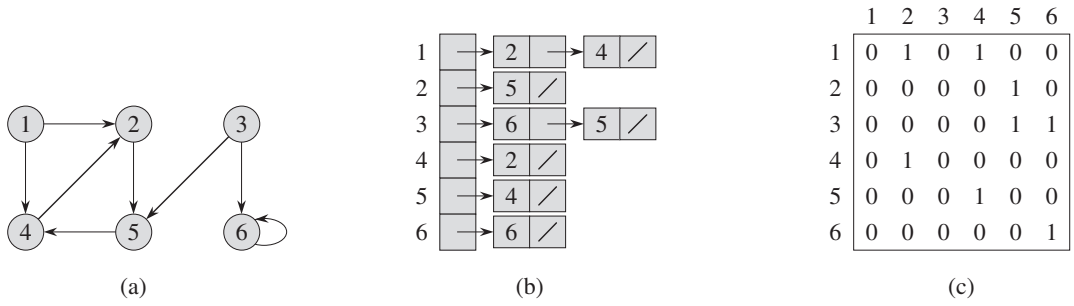


Figura 22.2 Due rappresentazioni di un grafo orientato. **(a)** Un grafo orientato G con sei vertici e otto archi. **(b)** Una rappresentazione con liste di adiacenza di G. **(c)** La rappresentazione con matrice di adiacenza di G.

del grafo, come trattiamo l'insieme degli archi E. Nello pseudocodice, quindi, adotteremo la notazione $G.Adj[u]$. La Figura 22.1(b) è una rappresentazione con liste di adiacenza del grafo non orientato della Figura 22.1(a). Analogamente, la Figura 22.2(b) è una rappresentazione con liste di adiacenza del grafo orientato della Figura 22.2(a).

Se G è un grafo orientato, la somma delle lunghezze di tutte le liste di adiacenza è $|E|$, perché un arco della forma (u, v) è rappresentato inserendo v in $Adj[u]$. Se G è un grafo non orientato, la somma delle lunghezze di tutte le liste di adiacenza è $2|E|$, perché se (u, v) è un arco non orientato, allora u appare nella lista di adiacenza di v e viceversa. Per i grafi orientati e non orientati, la rappresentazione con liste di adiacenza ha l'interessante proprietà che la quantità di memoria richiesta è $\Theta(V + E)$.

Le liste di adiacenza possono essere facilmente adattate per rappresentare i ***grafi pesati***, cioè, i grafi per i quali ogni arco ha un ***peso*** associato, tipicamente dato da una ***funzione peso*** $w : E \to \mathbb{R}$. Per esempio, sia $G = (V, E)$ un grafo pesato con la funzione peso w. Il peso $w(u, v)$ dell'arco $(u, v) \in E$ viene memorizzato semplicemente assieme al vertice v nella lista di adiacenza di u. La rappresentazione con le liste di adiacenza è molto robusta, nel senso che può essere modificata per supportare molte altre varianti di grafi.

Acquistato da Michele Michele su Webster il 2022-07-07 per uso esclusivamente personale. Copyright © 2022, McGraw-Hill Education (Italy)

Uno svantaggio potenziale della rappresentazione con liste di adiacenza è che non c'è un modo più veloce per determinare se un particolare arco (u, v) è presente nel grafo che cercare v nella lista di adiacenza $Adj[u]$. Per porre un rimedio a questo svantaggio, si può rappresentare il grafo con una matrice di adiacenza, al costo di usare asintoticamente una maggiore quantità di memoria (l'Esercizio 22.1-8 include alcuni suggerimenti su come modificare le liste di adiacenza per accelerare le ricerche degli archi).

Per la ***rappresentazione con matrice di adiacenza*** di un grafo $G = (V, E)$ si suppone che i vertici siano numerati $1, 2, \ldots, |V|$ in modo arbitrario. La rappresentazione con matrice di adiacenza di un grafo G consiste in una matrice $A = (a_{ij})$ di dimensioni $|V| \times |V|$ tale che

$$a_{ij} = \begin{cases} 1 & \text{se } (i, j) \in E \\ 0 & \text{negli altri casi} \end{cases}$$

Le Figure 22.1(c) e 22.2(c) sono le matrici di adiacenza del grafo non orientato e del grafo orientato illustrati, rispettivamente, nelle Figure 22.1(a) e 22.2(a). La matrice di adiacenza di un grafo richiede una memoria $\Theta(V^2)$, indipendentemente dal numero di archi nel grafo.

Osservate la simmetria rispetto alla diagonale principale della matrice di adiacenza nella Figura 22.1(c). Poiché (u, v) e (v, u) rappresentano lo stesso arco in un grafo non orientato, la matrice di adiacenza A di un grafo non orientato è uguale alla sua trasposta: $A = A^{\mathrm{T}}$. In alcune applicazioni conviene memorizzare soltanto gli elementi che si trovano sopra e lungo la diagonale della matrice di adiacenza, riducendo così la memoria richiesta per memorizzare il grafo quasi della metà.

Come la rappresentazione con liste di adiacenza di un grafo, anche la rappresentazione con matrice di adiacenza può essere utilizzata per i grafi pesati. Per esempio, se $G = (V, E)$ è un grafo pesato con la funzione peso w, il peso $w(u, v)$ dell'arco $(u, v) \in E$ viene semplicemente memorizzato come l'elemento nella riga u e nella colonna v della matrice di adiacenza. Se un arco non esiste, si può memorizzare un valore NIL nel corrispondente elemento nella matrice, sebbene per molti problemi sia preferibile utilizzare un valore come 0 o ∞.

Sebbene la rappresentazione con liste di adiacenza sia asintoticamente efficiente almeno quanto la rappresentazione con matrice di adiacenza, tuttavia quando i grafi sono abbastanza piccoli potrebbe essere preferita la matrice di adiacenza per la sua semplicità. Inoltre, se il grafo non è pesato, c'è un ulteriore vantaggio per la rappresentazione con matrice di adiacenza che riguarda la memoria richiesta. Anziché utilizzare una parola della memoria del calcolatore per ogni elemento della matrice di adiacenza, basta utilizzare un solo bit per ogni elemento.

Rappresentazione degli attributi

La maggior parte degli algoritmi che operano sui grafi devono mantenere degli attributi dei vertici e/o degli archi. Indichiamo questi attributi utilizzando la nostra solita notazione, ovvero $v.d$ per un attributo d di un vertice v. Quando indichiamo gli archi come coppie di vertici, utilizziamo lo stesso stile di notazione. Per esempio, se gli archi hanno un attributo f, allora indichiamo con $(u, v).f$ questo attributo per l'arco (u, v). La nostra notazione degli attributi è sufficiente per presentare e capire gli algoritmi.

L'implementazione degli attributi dei vertici e degli archi nei programmi reali può essere completamente differente. Non esiste un unico modo migliore per memorizzare e accedere agli attributi dei vertici e degli archi. Per un caso specifico, la vostra decisione potrebbe dipende dal linguaggio di programmazione che state utilizzando, dall'algoritmo che state implementando e dal modo in cui il vostro programma usa il grafo. Se rappresentate un grafo mediante le liste di adiacenza, potreste rappresentare gli attributi dei vertici in array addizionali, per esempio un array $d[1 \ldots |V|]$ che si affianca all'array Adj. Se i vertici adiacenti a u si trovano in $Adj[u]$, allora ciò che noi chiamiamo attributo $u.d$ verrebbe effettivamente memo-

Acquistato da Michele Michele su Webster il 2022-07-07 25:12 Numero Ordine Libreria: 19930301 6-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

rizzato nell'elemento dell'array $d[u]$. Ci sono molti altri modi di implementare gli attributi. Per esempio, in un linguaggio di programmazione orientato agli oggetti, gli attributi dei vertici potrebbero essere rappresentati come variabili di istanza all'interno di una sottoclasse di una classe `Vertex`.

Esercizi

22.1-1

Data una rappresentazione con liste di adiacenza di un grafo orientato, quanto tempo occorre per calcolare il grado uscente di ogni vertice? Quanto tempo occorre per calcolare il grado entrante di ogni vertice?

22.1-2

Utilizzate le liste di adiacenza per rappresentare un albero binario completo con 7 vertici. Create la rappresentazione equivalente con la matrice di adiacenza. Supponete che i vertici siano numerati da 1 a 7 come in un heap binario.

22.1-3

Il ***trasposto*** di un grafo orientato $G = (V, E)$ è il grafo $G^{\mathrm{T}} = (V, E^{\mathrm{T}})$, dove $E^{\mathrm{T}} = \{(v, u) \in V \times V : (u, v) \in E\}$. Quindi, G^{T} è G con tutti i suoi archi invertiti. Descrivete degli algoritmi efficienti per calcolare G^{T} da G, rappresentando G sia con le liste di adiacenza sia con la matrice di adiacenza. Analizzate i tempi di esecuzione dei vostri algoritmi.

22.1-4

Data una rappresentazione con liste di adiacenza di un multigrafo $G = (V, E)$, descrivete un algoritmo con tempo $O(V + E)$ per calcolare la rappresentazione con liste di adiacenza del grafo non orientato "equivalente" $G' = (V, E')$, dove E' è formato dagli archi in E con tutti gli archi multipli fra due vertici sostituiti da un singolo arco e con tutti i cappi rimossi.

22.1-5

Il ***quadrato*** di un grafo orientato $G = (V, E)$ è il grafo $G^2 = (V, E^2)$ tale che $(u, w) \in E^2$ se e soltanto se, per qualche $v \in V$, si abbia $(u, v) \in E$ e $(v, w) \in E$. Ovvero G^2 contiene un arco fra u e w, se G contiene un cammino con due soli archi fra u e w. Descrivete degli algoritmi efficienti per calcolare G^2 da G, rappresentando G sia con le liste di adiacenza sia con la matrice di adiacenza. Analizzate i tempi di esecuzione dei vostri algoritmi.

22.1-6

Quando viene utilizzata una rappresentazione con matrice di adiacenza, la maggior parte degli algoritmi per grafi richiede un tempo $\Omega(V^2)$, ma ci sono alcune eccezioni. Dimostrate che per determinare se un grafo orientato G contiene un ***pozzo universale*** – un vertice con grado entrante $|V| - 1$ e grado uscente 0 – basta un tempo $O(V)$, data una matrice di adiacenza per G.

22.1-7

La ***matrice di incidenza*** di un grafo orientato $G = (V, E)$ privo di cappi è una matrice $B = (b_{ij})$ di dimensioni $|V| \times |E|$ tale che

$$b_{ij} = \begin{cases} -1 & \text{se l'arco } j \text{ esce dal vertice } i \\ 1 & \text{se l'arco } j \text{ entra nel vertice } i \\ 0 & \text{negli altri casi} \end{cases}$$

Descrivete che cosa rappresentano gli elementi del prodotto matriciale BB^{T}, dove B^{T} è la matrice trasposta di B.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

22.1-8
Supponete che, anziché una lista concatenata, ogni elemento dell'array $Adj[u]$ sia una tavola hash che contiene i vertici v per i quali $(u, v) \in E$. Se tutte le ricerche degli archi sono equamente probabili, qual è il tempo atteso per determinare se un arco si trova nel grafo? Quali svantaggi ha questo schema? Suggerite una struttura dati alternativa per le liste di archi che risolve questi problemi. La vostra struttura dati alternativa ha degli svantaggi rispetto alla tavola hash?

22.2 Visita in ampiezza

La ***visita in ampiezza*** (breadth-first search) è uno dei più semplici algoritmi di ricerca nei grafi e l'archetipo per molti algoritmi importanti che operano con i grafi. L'algoritmo di Prim per l'albero di connessione minimo (Paragrafo 23.2) e l'algoritmo di Dijkstra per i cammini minimi da sorgente unica (Paragrafo 24.4) si basano su concetti simili a quelli della visita in ampiezza.

Dato un grafo $G = (V, E)$ e un vertice distinto s, detto ***sorgente***, la visita in ampiezza ispeziona sistematicamente gli archi di G per "scoprire" tutti i vertici che sono raggiungibili da s. Calcola la distanza (il minimo numero di archi) da s a ciascun vertice raggiungibile. Genera anche un albero BF (breadth-first tree) con radice s che contiene tutti i vertici raggiungibili. Per ogni vertice v raggiungibile da s, il cammino semplice nell'albero BF che va da s a v corrisponde a un "cammino minimo" da s a v in G, cioè un cammino che contiene il minor numero di archi. L'algoritmo opera sui grafi orientati e non orientati.

La visita in ampiezza è chiamata così perché espande la frontiera fra i vertici scoperti e quelli da scoprire in maniera uniforme lungo l'ampiezza della frontiera. Ovvero l'algoritmo scopre tutti i vertici che si trovano a distanza k da s, prima di scoprire i vertici a distanza $k + 1$.

Per tenere traccia del lavoro svolto, la visita in ampiezza colora i vertici di bianco, di grigio o di nero. Inizialmente tutti i vertici sono bianchi; dopo possono diventare grigi e poi neri. Un vertice viene ***scoperto*** quando viene incontrato per la prima volta durante la visita; in quel momento cessa di essere un vertice bianco. I vertici grigi e neri, quindi, sono vertici che sono stati scoperti, ma l'algoritmo li mantiene distinti per fare in modo che la visita proceda in ampiezza.[1] Se $(u, v) \in E$ e il vertice u è nero, allora il vertice v è grigio oppure nero; ovvero tutti i vertici adiacenti ai vertici neri sono stati scoperti. I vertici grigi possono avere qualche vertice bianco adiacente; essi rappresentano la frontiera fra i vertici scoperti e quelli da scoprire.

La visita in ampiezza costruisce l'albero BF che, inizialmente contiene soltanto la sua radice, che è il vertice sorgente s. Quando un vertice bianco v viene scoperto durante l'ispezione della lista di adiacenza di un vertice u già scoperto, il vertice v e l'arco (u, v) vengono aggiunti all'albero. Il vertice u è detto ***predecessore*** o ***padre*** di v nell'albero BF. Poiché un vertice viene scoperto una sola volta, esso ha al più un padre. Le relazioni di antenato e discendente nell'albero BF sono definite rispetto alla radice s come di consueto: se u è lungo il cammino che va dalla radice s al vertice v, allora u è un antenato di v e v è un discendente di u.

Acquistato da Michele Mi[illegible] il 2022-07-07 22:12:21 Numero Ordine Interno: 1981882 [illegible] © 2010 The McGraw-Hill Education (Italy)

[1] Abbiamo distinto i vertici grigi da quelli neri soltanto per aiutarci a spiegare come funziona la visita in ampiezza. Infatti, come dimostra l'Esercizio 22.2-3, avremmo ottenuto lo stesso risultato anche se non avessimo fatto una distinzione tra vertici grigi e vertici neri.

La seguente procedura BFS per la visita in ampiezza suppone che il grafo di input $G = (V, E)$ sia rappresentato con le liste di adiacenza; mantiene diverse altre strutture dati addizionali per ogni vertice nel grafo. Il colore di ogni vertice $u \in V$ è memorizzato nell'attributo $u.color$ e il predecessore di u è memorizzato nell'attributo $u.\pi$. Se u non ha un predecessore (per esempio, se $u = s$ o u non è stato ancora scoperto), allora $u.\pi =$ NIL. La distanza dalla sorgente s al vertice u calcolata dall'algoritmo è memorizzata nell'attributo $u.d$. L'algoritmo usa anche una coda Q con schema FIFO (vedere il Paragrafo 10.1) per gestire l'insieme dei vertici grigi.

```
BFS(G, s)
 1  for ogni vertice u ∈ G.V − {s}
 2       u.color = WHITE
 3       u.d = ∞
 4       u.π = NIL
 5  s.color = GRAY
 6  s.d = 0
 7  s.π = NIL
 8  Q = ∅
 9  ENQUEUE(Q, s)
10  while Q ≠ ∅
11       u = DEQUEUE(Q)
12       for ogni v ∈ G.Adj[u]
13            if v.color == WHITE
14                 v.color = GRAY
15                 v.d = u.d + 1
16                 v.π = u
17                 ENQUEUE(Q, v)
18       u.color = BLACK
```

La Figura 22.3 illustra varie fasi della procedura BFS con un grafo campione.

La procedura BFS opera nel seguente modo. Con l'eccezione del vertice sorgente s, le righe 1–4 colorano di bianco tutti i vertici, assegnano all'attributo $u.d$ il valore infinito per ogni vertice u e assegnano al padre di ogni vertice il valore NIL. La riga 5 colora s di grigio, perché questo vertice è considerato scoperto quando inizia la procedura. La riga 6 inizializza $s.d$ a 0; la riga 7 assegna al predecessore della sorgente il valore NIL. Le righe 8–9 inizializzano Q con la coda che contiene il solo vertice s.

Il ciclo **while** (righe 10–18) si ripete finché restano dei vertici grigi, che sono vertici scoperti le cui liste di adiacenza non sono state ancora completamente esaminate. Questo ciclo **while** conserva la seguente invariante:

> Quando viene eseguito il test della riga 10, la coda Q è formata dall'insieme dei vertici grigi.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Sebbene non vogliamo utilizzare questa invariante di ciclo per provare la correttezza, tuttavia è facile capire che l'invariante è vera prima della prima iterazione e che ogni iterazione del ciclo conserva l'invariante. Prima della prima iterazione, l'unico vertice grigio, e l'unico vertice in Q, è il vertice sorgente s. La riga 11 determina il vertice grigio u che si trova in testa alla coda Q e lo elimina da Q. Il ciclo **for** (righe 12–17) esamina ciascun vertice v nella lista di adiacenza di u.

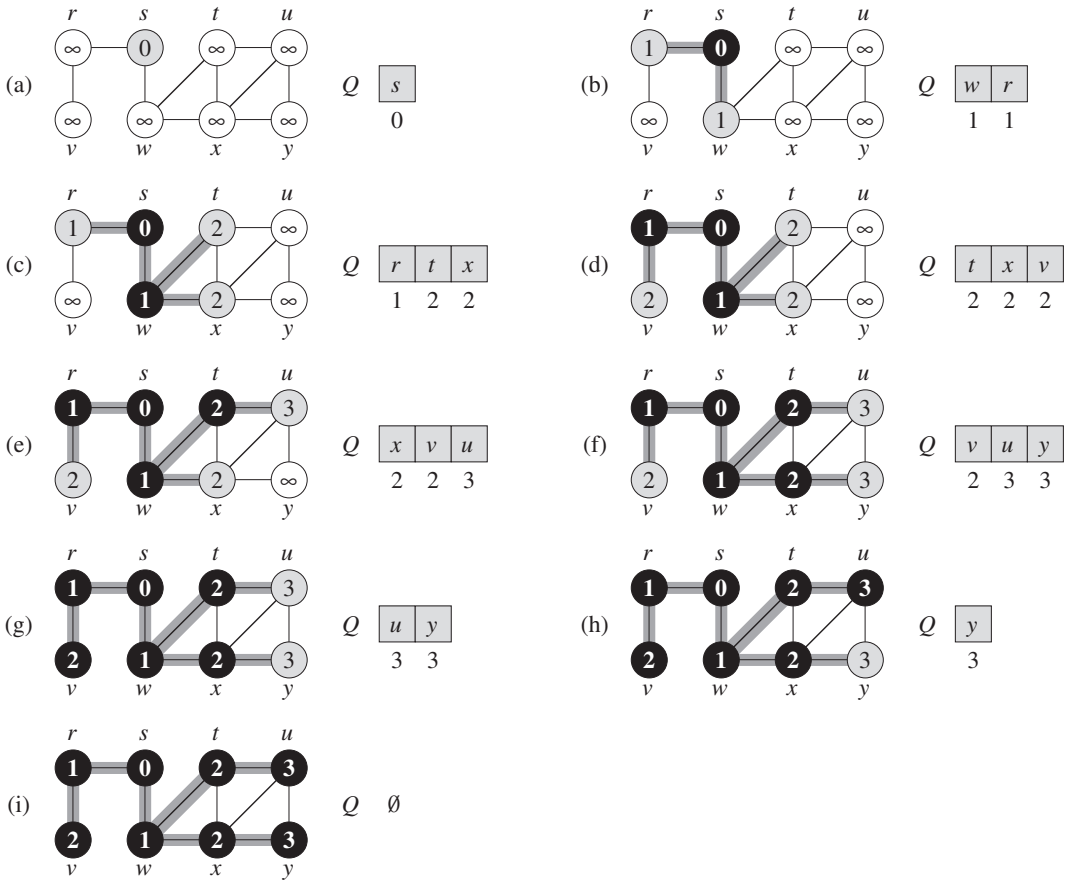


Figura 22.3 Il funzionamento della procedura BFS con un grafo non orientato. Gli archi sono rappresentati in grigio quando vengono prodotti da BFS. All'interno di ciascun vertice u è indicato il valore di $d[u]$. La coda Q è rappresentata all'inizio di ogni iterazione del ciclo **while** (righe 10–18). In corrispondenza dei vertici nella coda sono annotate le distanze dei vertici.

Se v è bianco, allora non è stato ancora scoperto; l'algoritmo lo scopre eseguendo le righe 14–17. La procedura colora di grigio il vertice v, imposta a $u.d + 1$ la sua distanza $v.d$, memorizza u come padre in $v.\pi$ e infine mette v in fondo alla coda Q. Dopo che tutti i vertici nella lista di adiacenza di u sono stati esaminati, u viene colorato di nero nella riga 18. L'invariante di ciclo si conserva perché, quando un vertice viene colorato di grigio (riga 14), viene anche accodato (riga 17) e, quando un vertice viene eliminato dalla coda (riga 11), viene anche colorato di nero (riga 18).

I risultati della visita in ampiezza possono dipendere dall'ordine in cui i vicini di un dato vertice vengono visitati nella riga 12: l'albero BF può variare, ma le distanze d calcolate dall'algoritmo sono le stesse (Esercizio 22.2-5).

Acquistato da Michele Mastrangelo Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Analisi

Prima di dimostrare le varie proprietà della visita in ampiezza, vogliamo analizzare il suo tempo di esecuzione con un grafo di input $G = (V, E)$. Utilizziamo il metodo dell'aggregazione, che abbiamo descritto nel Paragrafo 17.1. Dopo l'inizializzazione, nessun vertice sarà colorato di bianco, quindi il test nella riga 13 garantisce che ciascun vertice venga accodato al più una volta e, di conseguenza, venga eliminato dalla coda al più una volta.

Le operazioni di inserimento e cancellazione dalla coda richiedono un tempo $O(1)$, quindi il tempo totale dedicato alle operazioni con la coda è $O(V)$. Poiché la lista di adiacenza di ciascun vertice viene ispezionata soltanto quando il vertice viene rimosso dalla coda, ogni lista di adiacenza viene ispezionata al più una volta. Poiché la somma delle lunghezze di tutte le liste di adiacenza è $\Theta(E)$, il tempo totale impiegato per ispezionare le liste di adiacenza è $O(E)$. Il costo aggiuntivo di inizializzazione è $O(V)$, quindi il tempo di esecuzione totale di BFS è $O(V+E)$. In conclusione, la visita in ampiezza viene eseguita in un tempo che è lineare nella dimensione della rappresentazione con liste di adiacenza del grafo G.

Cammini minimi

Come detto in precedenza, la visita in ampiezza trova la distanza di ciascun vertice raggiungibile in un grafo $G = (V, E)$ da un dato vertice sorgente $s \in V$. Definiamo la ***distanza di cammino minimo*** $\delta(s, v)$ da s a v come il numero minimo di archi in un cammino qualsiasi che va dal vertice s al vertice v; se non c'è un cammino che va da s a v, allora $\delta(s, v) = \infty$. Un cammino di lunghezza $\delta(s, v)$ da s a v è detto ***cammino minimo***[2] da s a v. Prima di dimostrare che la visita in ampiezza calcola effettivamente le distanze di cammino minimo, analizziamo una importante proprietà di queste distanze.

Lemma 22.1
Se $G = (V, E)$ è un grafo orientato o non orientato e $s \in V$ è un vertice arbitrario, allora per qualsiasi arco $(u, v) \in E$ si ha

$$\delta(s, v) \leq \delta(s, u) + 1$$

Dimostrazione Se u è un vertice raggiungibile da s, allora lo è anche v. In questo caso, il cammino minimo da s a v non può essere più lungo del cammino minimo da s a u seguito dall'arco (u, v), quindi la disuguaglianza è valida. Se u non è raggiungibile da s, allora $\delta(s, u) = \infty$ e la disuguaglianza è valida. ■

Vogliamo dimostrare che BFS calcola correttamente $v.d = \delta(s, v)$ per ogni vertice $v \in V$. Prima dimostriamo che $v.d$ è un limite superiore per $\delta(s, v)$.

Lemma 22.2
Sia $G = (V, E)$ un grafo orientato o non orientato e supponiamo che BFS venga eseguita sul grafo G da un dato vertice sorgente $s \in V$. Allora, al termine della procedura, per ogni vertice $v \in V$, il valore $v.d$ calcolato da BFS soddisfa la relazione $v.d \geq \delta(s, v)$.

Dimostrazione Applichiamo l'induzione sul numero di operazioni ENQUEUE. La nostra ipotesi induttiva è che $v.d \geq \delta(s, v)$ per ogni $v \in V$.

Il caso base dell'induzione è la situazione che si ha subito dopo che il vertice s è stato inserito nella coda (riga 9 di BFS). L'ipotesi induttiva è vera qui, perché $s.d = 0 = \delta(s, s)$ e $v.d = \infty \geq \delta(s, v)$ per ogni $v \in V - \{s\}$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[2] Nei Capitoli 24 e 25 generalizzeremo il nostro studio dei cammini minimi ai grafi pesati, in cui ogni arco ha un peso effettivo e il peso di un cammino è la somma dei pesi degli archi che lo compongono. I grafi considerati nel presente capitolo non sono pesati, ovvero tutti gli archi hanno peso unitario.

Per il passo induttivo, consideriamo un vertice bianco v che viene scoperto durante la visita a partire da un vertice u. L'ipotesi induttiva implica che $u.d \geq \delta(s, u)$. Per l'assegnazione eseguita nella riga 15 e per il Lemma 22.1, si ha

$$\begin{aligned} v.d &= u.d + 1 \\ &\geq \delta(s, u) + 1 \\ &\geq \delta(s, v) \end{aligned}$$

Il vertice v viene inserito nella coda e non sarà mai più reinserito perché viene anche colorato di grigio e la clausola **then** (righe 14–17) viene eseguita soltanto per i vertici bianchi. Quindi, il valore di $v.d$ non cambierà più e l'ipotesi induttiva resta valida. ■

Per dimostrare che $v.d = \delta(s, v)$, dobbiamo prima vedere più dettagliatamente come opera la coda Q durante l'esecuzione di BFS. Il prossimo lemma dimostra che in qualsiasi istante ci sono al più due distinti valori d nella coda.

Lemma 22.3
Supponiamo che durante l'esecuzione di BFS su un grafo $G = (V, E)$, la coda Q contenga i vertici $\langle v_1, v_2, \ldots, v_r \rangle$, dove v_1 è l'inizio della coda Q e v_r è la fine. Allora, $v_r.d \leq v_1.d + 1$ e $v_i.d \leq v_{i+1}.d$ per $i = 1, 2, \ldots, r - 1$.

Dimostrazione La dimostrazione è per induzione sul numero di operazioni eseguite con la coda. Inizialmente, quando la coda contiene soltanto s, il lemma è certamente valido.

Per il passo induttivo, dobbiamo provare che il lemma è valido dopo l'inserimento e la cancellazione di un vertice nella coda. Se il vertice v_1 viene eliminato dalla coda, v_2 diventa il nuovo inizio della coda (se la coda si svuota, allora il lemma è banalmente valido). Per l'ipotesi induttiva, $v_1.d \leq v_2.d$; ma allora abbiamo $v_r.d \leq v_1.d + 1 \leq v_2.d + 1$ e le restanti disuguaglianze restano inalterate. Dunque, il lemma è dimostrato con v_2 all'inizio della coda.

L'inserimento di un vertice nella coda richiede un esame più attento del codice. Quando inseriamo nella coda un vertice v (riga 17 di BFS), esso diventa v_{r+1}. In quel momento, abbiamo già eliminato dalla coda Q il vertice u, la cui lista di adiacenza è quella correntemente ispezionata e, per l'ipotesi induttiva, il nuovo inizio della coda v_1 ha $v_1.d \geq u.d$. Pertanto, $v_{r+1}.d = v.d = u.d + 1 \leq v_1.d + 1$. Per l'ipotesi induttiva, abbiamo anche $v_r.d \leq u.d + 1$, ovvero $v_r.d \leq u.d + 1 = v.d = v_{r+1}.d$, e le restanti disuguaglianze restano inalterate. Dunque, il lemma è dimostrato quando v viene inserito nella coda. ■

Il seguente corollario dimostra che i valori d dei vertici quando vengono inseriti nella coda sono monotonicamente crescenti nel tempo.

Corollario 22.4
Supponiamo che i vertici v_i e v_j siano inseriti nella coda durante l'esecuzione di BFS e che v_i sia inserito nella coda prima di v_j. Allora $v_i.d \leq v_j.d$ nell'istante in cui v_j viene inserito nella coda.

Acquistato da Michele Mostarda (mostarda@gmail.com) il 22/03/2013. Tutti i diritti riservati. Materiale protetto da copyright © The McGraw-Hill Education (Italy)

Dimostrazione È una conseguenza immediata del Lemma 22.3 e della proprietà che ogni vertice riceve un valore finito d al più una volta durante l'esecuzione di BFS. ■

Adesso possiamo dimostrare che la visita in ampiezza calcola correttamente le distanze dei cammini minimi.

Teorema 22.5 (Correttezza della visita in ampiezza)
Sia $G = (V, E)$ un grafo orientato o non orientato e supponiamo che la procedura BFS venga eseguita sul grafo G da un dato vertice sorgente $s \in V$. Allora, durante la sua esecuzione, BFS scopre tutti i vertici $v \in V$ che sono raggiungibili dalla sorgente s e, alla fine dell'esecuzione, $v.d = \delta(s, v)$ per ogni $v \in V$. Inoltre, per qualsiasi vertice $v \neq s$ che è raggiungibile da s, uno dei cammini minimi da s a v è un cammino minimo da s a $v.\pi$ seguito dall'arco $(v.\pi, v)$.

Dimostrazione Supponiamo, per assurdo, che qualche vertice riceva un valore d che non è uguale alla distanza del suo cammino minimo. Sia v il vertice con il minimo $\delta(s, v)$ che riceve questo valore errato d; chiaramente $v \neq s$. Per il Lemma 22.2, $v.d \geq \delta(s, v)$, e quindi $v.d > \delta(s, v)$. Il vertice v deve essere raggiungibile da s, perché se non lo fosse, allora $\delta(s, v) = \infty \geq v.d$. Sia u il vertice che precede immediatamente v in un cammino minimo da s a v, cosicché $\delta(s, v) = \delta(s, u) + 1$. Poiché $\delta(s, u) < \delta(s, v)$ e per come abbiamo scelto v, deve essere $u.d = \delta(s, u)$. Ponendo insieme queste proprietà, si ha

$$v.d > \delta(s, v) = \delta(s, u) + 1 = u.d + 1 \tag{22.1}$$

Adesso consideriamo il momento in cui BFS sceglie di eliminare il vertice u dalla coda Q (riga 11). In quel momento, il vertice v può essere bianco, grigio o nero. Dimostreremo che in ciascuno di questi casi, si arriva a una contraddizione della disuguaglianza (22.1). Se il vertice v è bianco, allora la riga 15 imposta $v.d = u.d + 1$, che contraddice la disuguaglianza (22.1). Se il vertice v è nero, allora era stato già rimosso dalla coda e, per il Corollario 22.4, si ha $v.d \leq u.d$, che contraddice ancora la disuguaglianza (22.1). Se il vertice v è grigio, allora era stato colorato di grigio dopo la cancellazione dalla coda di qualche vertice w, che era stato cancellato da Q prima di u e per il quale $v.d = w.d + 1$. Per il Corollario 22.4, però, $w.d \leq u.d$, e quindi si ha $v.d = w.d + 1 \leq u.d + 1$, che contraddice ancora la disuguaglianza (22.1). Dunque, possiamo concludere che $v.d = \delta(s, v)$ per ogni $v \in V$. Tutti i vertici raggiungibili da s devono essere scoperti, perché se non lo fossero, avrebbero $\infty = v.d > \delta(s, v)$. Per concludere la dimostrazione del teorema, osserviamo che, se $v.\pi = u$, allora $v.d = u.d + 1$. Quindi, possiamo ottenere un cammino minimo da s a v prendendo un cammino minimo da s a $v.\pi$ e poi attraversando l'arco $(v.\pi, v)$. ■

Alberi di visita in ampiezza

La procedura BFS costruisce un albero BF mentre visita il grafo, come illustra la Figura 22.3. L'albero è rappresentato dagli attributi π. Più formalmente, per un grafo $G = (V, E)$ con sorgente s, definiamo il ***sottografo dei predecessori*** di G come $G_\pi = (V_\pi, E_\pi)$, dove

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$V_\pi = \{v \in V : v.\pi \neq \text{NIL}\} \cup \{s\}$$

e

$$E_\pi = \{(v.\pi, v) : v \in V_\pi - \{s\}\}$$

Il sottografo dei predecessori G_π è un ***albero BF*** (o albero di visita in ampiezza) se V_π è formato dai vertici raggiungibili da s e, per ogni $v \in V_\pi$, c'è un solo

cammino semplice da s a v in G_π che è anche un cammino minimo da s a v in G. Un albero BF è effettivamente un albero, perché è connesso e $|E_\pi| = |V_\pi| - 1$ (vedere il Teorema B.2). Gli archi in E_π sono detti ***archi d'albero***.

Il seguente lemma dimostra che il sottografo dei predecessori è un albero BF.

Lemma 22.6
Quando viene applicata a un grafo $G = (V, E)$, orientato o non orientato, la procedura BFS costruisce π in modo tale the il sottografo dei predecessori $G_\pi = (V_\pi, E_\pi)$ sia un albero BF.

Dimostrazione La riga 16 della procedura BFS imposta $v.\pi = u$ se e soltanto se $(u, v) \in E$ e $\delta(s, v) < \infty$ – ovvero, se v è raggiungibile da s – e quindi V_π è formato dai vertici in V che sono raggiungibili da s. Poiché G_π forma un albero, per il Teorema B.2, esso contiene un unico cammino semplice da s a ciascun vertice in V_π. Applicando il Teorema 22.5 in modo induttivo, concludiamo che ciascuno di questi cammini è un cammino minimo. ■

La seguente procedura stampa i vertici di un cammino minimo da s a v, supponendo che la procedura BFS sia già stata eseguita per calcolare l'albero dei cammini minimi.

```
PRINT-PATH(G, s, v)
1  if v == s
2       stampa s
3  elseif v.π == NIL
4       stampa "non ci sono cammini da" s "a" v
5  else PRINT-PATH(G, s, v.π)
6       stampa v
```

Questa procedura viene eseguita in un tempo lineare nel numero di vertici del cammino stampato, perché ogni chiamata ricorsiva riguarda un cammino che ha un vertice in meno.

Esercizi

22.2-1
Calcolate i valori d e π che si ottengono effettuando una visita in ampiezza del grafo orientato della Figura 22.2(a), utilizzando il vertice 3 come sorgente.

22.2-2
Calcolate i valori d e π che si ottengono effettuando una visita in ampiezza del grafo non orientato della Figura 22.3, utilizzando il vertice u come sorgente.

22.2-3
Dimostrate che è sufficiente utilizzare un singolo bit per memorizzare il colore di ciascun vertice verificando che la procedura BFS produrrebbe lo stesso risultato se le righe 5 e 14 fossero eliminate.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995083016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

22.2-4
Qual è il tempo di esecuzione della procedura BFS se il suo grafo di input è rappresentato da una matrice di adiacenza e l'algoritmo viene modificato per gestire questa forma di input?

22.2-5
Dimostrate che in una visita in ampiezza il valore $u.d$ assegnato a un vertice u è indipendente dall'ordine in cui si trovano i vertici in ogni lista di adiacenza. Utilizzando la Figura 22.3 come esempio, dimostrate che l'albero BF calcolato dalla procedura BFS può dipendere dall'ordinamento delle liste di adiacenza.

22.2-6
Indicate un esempio di un grafo orientato $G = (V, E)$, un vertice sorgente $s \in V$ e un insieme di archi $E_\pi \subseteq E$ tali che, per ogni vertice $v \in V$, l'unico cammino semplice da s a v nel grafo (V, E_π) sia un cammino minimo in G eppure l'insieme degli archi E_π non può essere prodotto eseguendo la procedura BFS sul grafo G, indipendentemente da come siano ordinati i vertici all'interno di ciascuna lista di adiacenza.

22.2-7
Ci sono due tipi di lottatori professionisti: "buoni" e "cattivi". Fra una coppia qualsiasi di lottatori professionisti ci può essere o no rivalità. Supponete di avere n lottatori e una lista di r coppie di lottatori fra i quali c'è rivalità. Create un algoritmo con tempo $O(n + r)$ che determina se sia possibile designare alcuni lottatori come buoni e tutti gli altri come cattivi, in modo che la rivalità sia sempre fra un lottatore buono e uno cattivo. Se tale designazione è possibile, il vostro algoritmo dovrebbe effettuarla.

22.2-8 ⋆
Il ***diametro*** di un albero $T = (V, E)$ è dato da $\max_{u,v \in V} \delta(u, v)$, ovvero è la più grande fra tutte le distanze di cammino minimo nell'albero. Create un algoritmo efficiente per calcolare il diametro di un albero e analizzate il tempo di esecuzione del vostro algoritmo.

22.2-9
Sia $G = (V, E)$ un grafo non orientato e connesso. Create un algoritmo con tempo $O(V + E)$ per calcolare un cammino in G che attraversa ciascun arco in E esattamente un volta in ogni direzione. Spiegate come potete trovare l'uscita in un labirinto, avendo a disposizione una grande quantità di monetine da un centesimo.

22.3 Visita in profondità

La strategia adottata dalla visita in profondità (depth-first search) consiste, come è implicito nel suo nome, nel visitare il grafo sempre più "in profondità" se possibile. Nella visita in profondità, gli archi vengono ispezionati a partire dall'ultimo vertice scoperto v che ha ancora archi non ispezionati che escono da esso. Quando tutti gli archi di v sono stati ispezionati, la visita "fa marcia indietro" per ispezionare gli archi che escono dal vertice dal quale v era stato scoperto. Questo processo continua finché non saranno stati scoperti tutti i vertici che sono raggiungibili dal vertice sorgente originale. Se restano dei vertici non scoperti, allora uno di essi viene selezionato come nuovo vertice sorgente e la visita riparte da questa sorgente. L'intero processo viene ripetuto finché non saranno scoperti tutti i vertici del grafo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

Come nella visita in ampiezza, quando un vertice v viene scoperto durante un'ispezione della lista di adiacenza di un vertice u già scoperto, la visita in profondità registra questo evento assegnando u all'attributo $v.\pi$ di v (il predecessore). Diversamente dalla visita in ampiezza, il cui sottografo dei predecessori forma un

albero, il sottografo dei predecessori prodotto da una visita in profondità può essere formato da più alberi, perché la visita può essere ripetuta da più sorgenti.[3] Il ***sottografo dei predecessori*** di una visita in profondità è quindi definito in modo un po' diverso da quello di una visita in ampiezza: poniamo $G_\pi = (V, E_\pi)$, dove

$$E_\pi = \{(v.\pi, v) : v \in V \text{ e } v.\pi \neq \text{NIL}\}$$

Il sottografo dei predecessori di una visita in profondità forma una ***foresta DF*** (depth-first forest) composta da vari ***alberi DF*** (depth-first tree). Gli archi in E_π sono detti ***archi d'albero***.

Come nella visita in ampiezza, i vertici vengono colorati durante la visita in profondità per indicare il loro stato. Inizialmente, tutti i vertici sono bianchi. Un vertice diventa grigio quando viene ***scoperto*** durante la visita; diventa nero quando viene ***completato***, ovvero quando la sua lista di adiacenza è stata completamente ispezionata. Questa tecnica garantisce che ogni vertice vada a finire in un solo albero DF, in modo che questi alberi siano disgiunti.

Oltre a creare una foresta DF, la visita in profondità associa anche a ciascun vertice delle informazioni temporali. Ogni vertice v ha due informazioni temporali: la prima $v.d$ registra il momento in cui il vertice v viene scoperto (e colorato di grigio); la seconda $v.f$ registra il momento in cui la visita completa l'ispezione della lista di adiacenza del vertice v (che diventa nero). Queste informazioni temporali sono utilizzate in molti algoritmi che operano con i grafi e, in generale, agevolano l'analisi del comportamento della visita in profondità.

La seguente procedura DFS registra nell'attributo $u.d$ il momento in cui scopre il vertice u e nell'attributo $u.f$ il momento in cui completa la visita del vertice u. Queste informazioni temporali sono numeri interi compresi fra 1 e $2|V|$, perché ciascuno dei $|V|$ vertici può essere scoperto una sola volta e la sua visita può essere completata una sola volta. Per ogni vertice u si ha

$$u.d < u.f \tag{22.2}$$

Il vertice u è WHITE prima del tempo $u.d$, GRAY fra il tempo $u.d$ e il tempo $u.f$, e BLACK successivamente.

Il seguente pseudocodice è l'algoritmo di base che effettua una visita in profondità. Il grafo di input G può essere orientato o non orientato. La variabile *time* è una variabile globale che utilizziamo per registrare le informazioni temporali.

```
DFS(G)
1  for ogni vertice u ∈ G.V
2      u.color = WHITE
3      u.π = NIL
4  time = 0
5  for ogni vertice u ∈ G.V
6      if u.color == WHITE
7          DFS-VISIT(G, u)
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[3] Potrebbe sembrare arbitrario che una visita in ampiezza sia limitata a una sola sorgente, mentre una visita in profondità può effettuare la visita da più sorgenti. Sebbene, in teoria, una visita in ampiezza possa procedere da più sorgenti e una visita in profondità possa essere limitata a una sola sorgente, tuttavia il nostro approccio riflette il modo in cui vengono tipicamente utilizzati i risultati di queste visite. La visita in ampiezza, di solito, è utilizzata per trovare le distanze dei cammini minimi (e il corrispondente sottografo dei predecessori) da una data sorgente. La visita in profondità, di solito, è una subroutine in un altro algoritmo, come vedremo più avanti in questo capitolo.

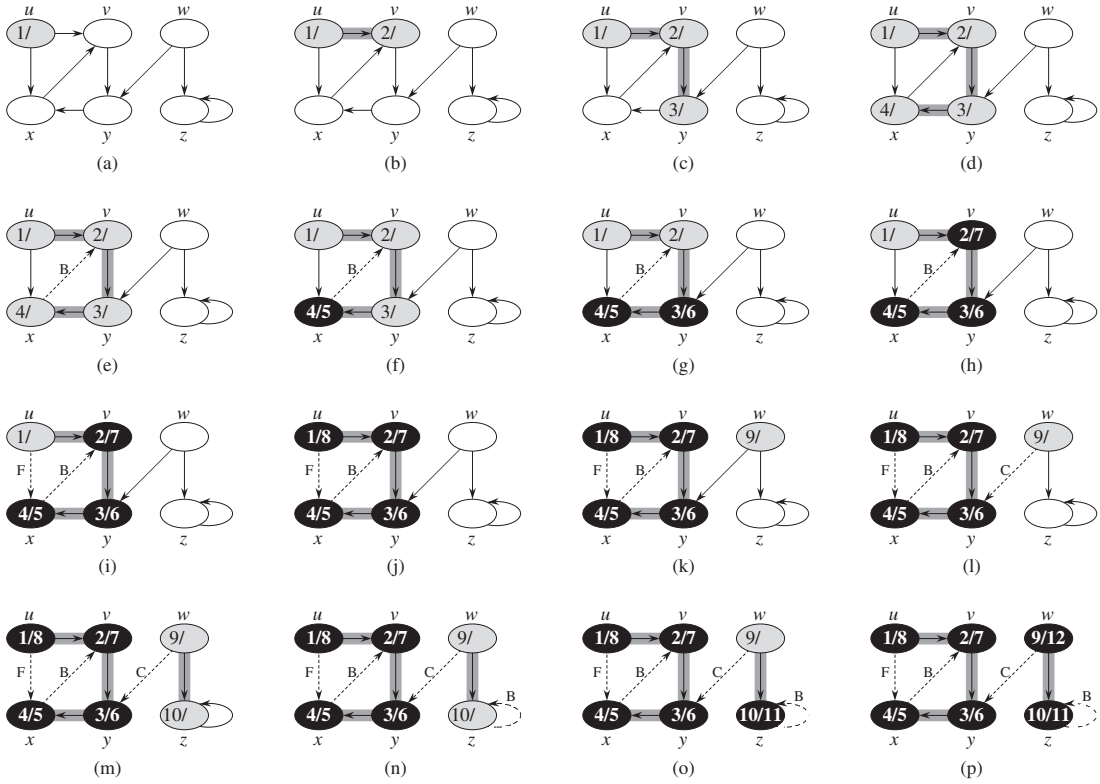


Figura 22.4 Le fasi dell'algoritmo DFS di visita in profondità con un grafo orientato. Dopo che gli archi sono stati ispezionati dall'algoritmo, vengono rappresentati su uno sfondo grigio (se sono archi d'albero) o tratteggiati (negli altri casi). Quelli che non sono archi d'albero sono etichettati con le lettere B, C o F a seconda che siano archi all'indietro, trasversali o in avanti. All'interno dei vertici sono riportate le informazioni temporali (tempo di scoperta/tempo di completamento).

```
DFS-VISIT(G, u)
 1  time = time + 1          // Il vertice bianco u è stato appena scoperto.
 2  u.d = time
 3  u.color = GRAY
 4  for ogni v ∈ G.Adj[u]    // Ispeziona l'arco (u, v)
 5      if v.color == WHITE
 6          v.π = u
 7          DFS-VISIT(G, v)
 8  u.color = BLACK          // Colora di nero u; visita completata.
 9  time = time + 1
10  u.f = time
```

La Figura 22.4 illustra le varie fasi della procedura DFS applicata al grafo illustrato nella Figura 22.2.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

La procedura DFS opera nel seguente modo. Le righe 1–3 colorano di bianco tutti i vertici e inizializzano i loro attributi π a NIL. La riga 4 azzera il contatore globale del tempo. Le righe 5–7 controllano, uno alla volta, tutti i vertici in V e, quando trovano un vertice bianco, lo visitano utilizzando la procedura DFS-VISIT. Ogni volta che viene chiamata la procedura DFS-VISIT(u) nella riga 7, il vertice u diventa la radice di un nuovo albero della foresta DF. Quando la proce-

dura DFS termina, a ogni vertice u è stato assegnato un ***tempo di scoperta*** $u.d$ e un ***tempo di completamento*** $u.f$.

In ogni chiamata di DFS-VISIT(u), il vertice u è inizialmente bianco. La riga 1 incrementa la variabile globale *time*, la riga 2 registra il nuovo valore di *time* come il tempo di scoperta $u.d$ e la riga 3 colora di grigio u. Le righe 4–7 ispezionano ogni vertice v adiacente a u e visitano in modo ricorsivo il vertice v, se è bianco. Quando un vertice $v \in Adj[u]$ viene esaminato nella riga 4, diciamo che l'arco (u, v) è stato ***ispezionato*** dalla visita in profondità. Infine, dopo che tutti i nodi che escono da u sono stati ispezionati, le righe 8–10 colorano di nero u, incrementano *time* e registrano in $u.f$ il tempo di completamento della visita.

Notate che i risultati della visita in profondità potrebbero dipendere dall'ordine in cui i vertici sono ispezionati nella riga 5 della procedura DFS e dall'ordine in cui i vicini di un vertice vengono visitati nella riga 4 della procedura DFS-VISIT. In pratica, queste differenze nell'ordine in cui vengono effettuate le visite non causano problemi, in quanto *qualsiasi* risultato della visita in profondità di solito può essere efficacemente utilizzato, perché i risultati ottenuti sono essenzialmente equivalenti.

Qual è il tempo di esecuzione della procedura DFS? I cicli nelle righe 1–3 e nelle righe 5–7 di DFS impiegano un tempo $\Theta(V)$, escluso il tempo per eseguire le chiamate di DFS-VISIT. Come abbiamo fatto per la visita in ampiezza, applichiamo il metodo dell'aggregazione. La procedura DFS-VISIT è chiamata esattamente una volta per ogni vertice $v \in V$, perché DFS-VISIT viene invocata soltanto se un vertice è bianco e la prima cosa che fa è colorare di grigio il vertice. Durante un'esecuzione di DFS-VISIT(v), il ciclo nelle righe 4–7 viene eseguito $|Adj[v]|$ volte. Poiché

$$\sum_{v \in V} |Adj[v]| = \Theta(E)$$

il costo totale per eseguire le righe 4–7 di DFS-VISIT è $\Theta(E)$. Il tempo di esecuzione di DFS è dunque $\Theta(V + E)$.

Proprietà della visita in profondità

La visita in profondità fornisce informazioni preziose sulla struttura di un grafo. Forse la più importante proprietà della visita in profondità è che il sottografo dei predecessori G_π forma effettivamente una foresta di alberi, in quanto la struttura degli alberi DF rispecchia esattamente la struttura delle chiamate ricorsive di DFS-VISIT. Ovvero, $u = v.\pi$ se e soltanto se DFS-VISIT(v) è stata chiamata durante una visita della lista di adiacenza di u. In aggiunta, il vertice v è un discendente del vertice u nella foresta DF se e soltanto se v viene scoperto durante il periodo in cui u è grigio.

Un'altra importante proprietà della visita in profondità è che i tempi di scoperta e di completamento hanno una ***struttura di parentesi***. Se rappresentiamo la scoperta del vertice u con una parentesi aperta "$(u$" e il suo completamento con una parentesi chiusa "$u)$", allora la storia delle scoperte e dei completamenti produce un'espressione ben formata, nel senso che le parentesi sono opportunamente annidate. Per esempio, la visita in profondità della Figura 22.5(a) corrisponde alla parentesizzazione illustrata nella Figura 22.5(b). Il seguente teorema consente di definire la proprietà della struttura di parentesi in un altro modo.

Acquistato da Michele Michela su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 19950316. 220707 © Copyright © 2022. McGraw-Hill Education (Italy)

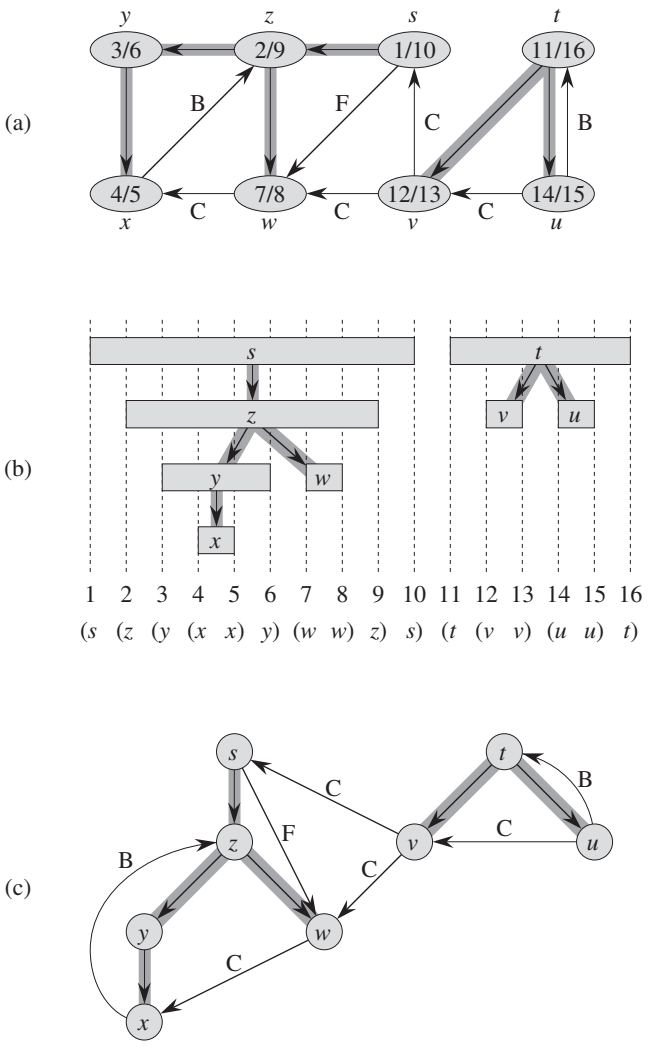


Figura 22.5 Proprietà della visita in profondità. **(a)** Il risultato di una visita in profondità di un grafo orientato. I vertici includono le informazioni temporali e i tipi di archi sono indicati come nella Figura 22.4. **(b)** Gli intervalli fra il tempo di scoperta e il tempo di completamento di ciascun vertice corrispondono alla parentesizzazione illustrata. Ogni rettangolo si estende nell'intervallo definito dai tempi di scoperta e di completamento del corrispondente vertice. Sono illustrati gli archi d'albero. Se due intervalli si sovrappongono, allora uno viene annidato all'interno dell'altro e il vertice che corrisponde all'intervallo più piccolo è un discendente del vertice che corrisponde all'intervallo più grande. **(c)** Il grafo della parte (a) ridisegnato con tutti gli archi d'albero e in avanti che scendono in un albero DF e con tutti gli archi all'indietro che salgono da un discendente a un antenato.

Teorema 22.7 (Teorema delle parentesi)
In qualsiasi visita in profondità di un grafo $G = (V, E)$ (orientato o non orientato), per ogni coppia di vertici u e v, è soddisfatta una sola delle seguenti tre condizioni:

- Gli intervalli $[u.d, u.f]$ e $[v.d, v.f]$ sono completamente disgiunti; inoltre u e v non sono discendenti l'uno dell'altro nella foresta DF.
- L'intervallo $[u.d, u.f]$ è interamente contenuto nell'intervallo $[v.d, v.f]$; inoltre u è un discendente di v in un albero DF.
- L'intervallo $[v.d, v.f]$ è interamente contenuto nell'intervallo $[u.d, u.f]$; inoltre v è un discendente di u in un albero DF.

Acquistato da Michele Michele su Webster il 2022-07-07 12:12:12 Vietata la distribuzione a terzi. Copyright © 2022, McGraw-Hill Education (Italy)

Dimostrazione Iniziamo con il caso in cui $u.d < v.d$. Ci sono due sottocasi da considerare, a seconda che $v.d < u.f$ oppure no. Il primo sottocaso si verifica quando $v.d < u.f$, quindi v è stato scoperto mentre u era ancora grigio. Questo implica che v è un discendente di u. Inoltre, poiché v è stato scoperto più recentemente di u, vengono ispezionati tutti i suoi archi uscenti e l'ispezione di v viene

completata prima che la visita riprenda da u e venga completata l'ispezione di u. In questo caso, quindi, l'intervallo $[v.d, v.f]$ è interamente contenuto nell'intervallo $[u.d, u.f]$. Nel secondo sottocaso, $u.f < v.d$ e, per la disuguaglianza (22.2), $u.d < u.f < v.d < v.f$; quindi gli intervalli $[u.d, u.f]$ e $[v.d, v.f]$ sono disgiunti. Poiché gli intervalli sono disgiunti, nessuno dei due vertici è stato scoperto mentre l'altro era grigio, quindi nessuno dei due vertici è un discendente dell'altro.

Il caso in cui $v.d < u.d$ è simile, perché basta invertire i ruoli di u e v nella precedente discussione.

■

Corollario 22.8 (Annidamento degli intervalli dei discendenti)
Il vertice v è un discendente proprio del vertice u nella foresta DF per un grafo G (orientato o non orientato) se e soltanto se $u.d < v.d < v.f < u.f$.

Dimostrazione È una conseguenza immediata del Teorema 22.7. ■

Il prossimo teorema fornisce un'altra caratterizzazione importante di quando un vertice è un discendente di un altro vertice nella foresta DF.

Teorema 22.9 (Teorema del cammino bianco)
In una foresta DF di un grafo $G = (V, E)$ (orientato o non orientato), il vertice v è un discendente del vertice u se e soltanto se, al tempo $u.d$ in cui viene scoperto u, il vertice v può essere raggiunto da u lungo un cammino che è formato esclusivamente da vertici bianchi.

Dimostrazione $\Rightarrow$: se $v = u$, allora il cammino da u a v contiene soltanto il vertice u, che è ancora bianco quando impostiamo il valore di $u.d$. Adesso, supponiamo che v sia un discendente proprio di u nella foresta DF. Per il Corollario 22.8, $u.d < v.d$, e quindi v è bianco nel tempo $u.d$. Poiché v può essere un discendente qualsiasi di u, tutti i vertici lungo l'unico cammino semplice da u a v nella foresta DF sono bianchi al tempo $u.d$.

$\Leftarrow$: supponiamo che ci sia un cammino di vertici bianchi da u a v al tempo $u.d$ e che v non diventi un discendente di u nell'albero DF. Senza perdere in generalità, supponiamo che tutti gli altri vertici diversi da v lungo il cammino diventino discendenti di u (altrimenti, indichiamo con v il vertice più vicino a u lungo il cammino che non diventa un discendente di u). Sia w il predecessore di v lungo il cammino, cosicché w è un discendente di u (in effetti, w e u potrebbero essere lo stesso vertice) e, per il Corollario 22.8, si ha $w.f \leq u.f$. Poiché v deve essere scoperto dopo u, ma prima che sia completata la visita di w, si ha $u.d < v.d < w.f \leq u.f$. Il Teorema 22.7 allora implica che l'intervallo $[v.d, v.f]$ sia interamente contenuto nell'intervallo $[u.d, u.f]$. Per il Corollario 22.8, v deve essere un discendente di u. ■

Classificazione degli archi

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Un'altra interessante proprietà della visita in profondità è che la visita può essere utilizzata per classificare gli archi del grafo di input $G = (V, E)$. Questa classificazione degli archi può essere utilizzata per raccogliere informazioni importanti su un grafo. Per esempio, nel prossimo paragrafo, vedremo che un grafo orientato è aciclico se e soltanto se una visita in profondità non genera archi "all'indietro" (Lemma 22.11).

Possiamo definire quattro tipi di archi in base alla foresta DF G_π prodotta da una visita in profondità del grafo G.

1. ***Archi d'albero***: sono gli archi nella foresta DF G_π. L'arco (u, v) è un arco d'albero se v viene scoperto la prima volta durante l'esplorazione di (u, v).
2. ***Archi all'indietro***: sono quegli archi (u, v) che collegano un vertice u a un antenato v in un albero DF. I cappi, che possono presentarsi nei grafi orientati, sono considerati archi all'indietro.
3. ***Archi in avanti***: sono gli archi (u, v) (diversi dagli archi d'albero) che collegano un vertice u a un discendente v in un albero DF.
4. ***Archi trasversali***: tutti gli altri archi. Possono connettere i vertici nello stesso albero DF, purché un vertice non sia un antenato dell'altro, oppure possono connettere vertici di alberi DF differenti.

Nelle Figure 22.4 e 22.5 gli archi sono etichettati per indicare il loro tipo. La Figura 22.5(c) mostra anche come il grafo della Figura 22.5(a) può essere ridisegnato in modo che tutti gli archi d'albero e in avanti puntino in basso in un albero DF e tutti gli archi all'indietro puntino in alto. Qualsiasi grafo può essere ridisegnato in questo modo.

L'algoritmo DFS possiede le informazioni necessarie per classificare gli archi che incontra. L'idea chiave è che ogni arco (u, v) può essere classificato in base al colore del vertice v che viene raggiunto quando l'arco viene ispezionato per la prima volta:

1. WHITE indica un arco d'albero.
2. GRAY indica un arco all'indietro.
3. BLACK indica un arco in avanti o trasversale.

Il primo caso è immediato dalla specifica dell'algoritmo. Per il secondo caso, notate che i vertici grigi formano sempre una catena lineare di discendenti che corrisponde allo stack delle chiamate attive di DFS-VISIT; il numero di vertici grigi è uno più della profondità dell'ultimo vertice scoperto nella foresta DF. L'ispezione procede sempre a partire dal vertice grigio più profondo, quindi un arco che raggiunge un altro vertice grigio raggiunge un antenato. Il terzo caso gestisce l'ultima possibilità; si può dimostrare che tale arco (u, v) è un arco in avanti se $u.d < v.d$ e un arco trasversale se $u.d > v.d$ (Esercizio 22.3-5).

In un grafo non orientato potrebbe esserci qualche ambiguità nella classificazione degli archi, perché (u, v) e (v, u) in effetti sono lo stesso arco. In tale caso, l'arco viene classificato con il *primo* tipo della lista di classificazione che può essere applicato. In modo equivalente (vedere l'Esercizio 22.3-6), l'arco viene classificato a seconda se si incontra prima (u, v) o (v, u) durante l'esecuzione dell'algoritmo.

Acquistato da Michele Michele su Webster il 2022-07-0[illegible] - Copyright © 2010 The McGraw-Hill Companies S.r.l.

Dimostriamo adesso che in una visita in profondità di un grafo non orientato non si presentano mai archi in avanti e trasversali.

Teorema 22.10

In una visita in profondità di un grafo non orientato G, gli archi di G possono essere archi d'albero o archi all'indietro.

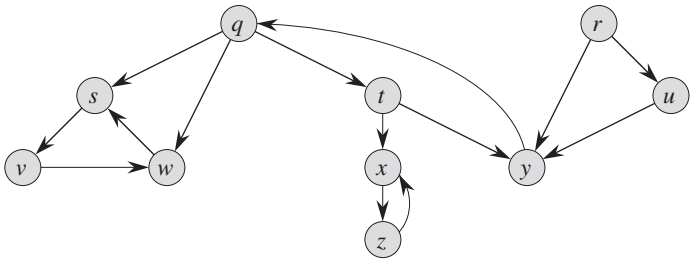


Figura 22.6 Un grafo orientato per gli Esercizi 22.3-2 e 22.5-2.

Dimostrazione Sia (u, v) un arco arbitrario di G e, senza perdere in generalità, supponiamo che $u.d < v.d$. Allora, il vertice v deve essere scoperto e completato prima che sia completato u (mentre u è grigio), perché v si trova nella lista di adiacenza di u. Se l'arco (u, v) viene ispezionato prima nella direzione da u a v, allora v è un vertice non scoperto (bianco) fino a quel momento, perché altrimenti avremmo già ispezionato questo arco nella direzione da v a u. Quindi, (u, v) diventa un arco d'albero. Se (u, v) viene ispezionato prima nella direzione da v a u, allora (u, v) è un arco all'indietro, perché u è ancora grigio quando l'arco viene esplorato per la prima volta. ■

Nei prossimi paragrafi vedremo varie applicazioni di questi teoremi.

Esercizi

22.3-1

Create una tabella 3×3 con le etichette WHITE, GRAY e BLACK per le righe e le colonne. In ogni cella (i, j) indicate se, in qualsiasi punto durante una visita in profondità di un grafo orientato, ci può essere un arco che va da un vertice di colore i a un vertice di colore j. Per ciascuno degli archi possibili indicate il tipo di arco. Create una tabella analoga per la visita in profondità di un grafo non orientato.

22.3-2

Illustrate come funziona una visita in profondità con il grafo della Figura 22.6. Supponete che il ciclo **for** (righe 5–7) della procedura DFS consideri i vertici in ordine alfabetico e che ogni lista di adiacenza sia ordinata alfabeticamente. Indicate i tempi di scoperta e di completamento di ciascun vertice e la classificazione di ciascun arco.

22.3-3

Illustrate la struttura delle parentesi della visita in profondità del grafo illustrato nella Figura 22.4.

22.3-4

Dimostrate che basta un solo bit per memorizzare il colore di ciascun vertice verificando che la procedura DFS produrrebbe lo stesso risultato se la riga 3 di DFS-VISIT venisse eliminata.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

22.3-5

Dimostrate che l'arco (u, v) è

a. Un arco d'albero o un arco in avanti se e soltanto se $u.d < v.d < v.f < u.f$.

b. Un arco all'indietro se e soltanto se $v.d \leq u.d < u.f \leq v.f$.

c. Un arco trasversale se e soltanto se $v.d < v.f < u.d < u.f$.

22.3-6
Dimostrate che in un grafo non orientato classificare un arco (u, v) come arco d'albero o arco all'indietro, a seconda se viene incontrato prima (u, v) o (v, u) durante la visita in profondità, equivale a classificare l'arco secondo la priorità dei tipi nello schema di classificazione.

22.3-7
Riscrivete la procedura DFS utilizzando uno stack per eliminare la ricorsione.

22.3-8
Trovate un controesempio alla congettura che, se esiste un cammino da u a v in un grafo orientato G e se $u.d < v.d$ in una visita in profondità di G, allora v è un discendente di u nella foresta DF risultante.

22.3-9
Trovate un controesempio alla congettura che, se esiste un cammino da u a v in un grafo orientato G, allora per qualsiasi visita in profondità deve risultare $v.d \leq u.f$.

22.3-10
Modificate lo pseudocodice per la visita in profondità in modo che stampi tutti gli archi del grafo orientato G insieme ai loro tipi. Descrivete quali eventuali modifiche devono essere apportate se G è un grafo non orientato.

22.3-11
Spiegate come un vertice u di un grafo orientato possa finire in un albero DF che contiene soltanto u, anche se u ha degli archi entranti e uscenti nel grafo G.

22.3-12
Dimostrate che una visita in profondità di un grafo non orientato G può essere utilizzata per identificare le componenti connesse di G e che la foresta DF contiene tanti alberi quante sono le componenti connesse di G. Più precisamente, spiegate come modificare la visita in profondità in modo che a ogni vertice v sia assegnata un'etichetta $v.cc$ di numeri interi compresi fra 1 e k, dove k è il numero di componenti connesse di G, tale che $u.cc = v.cc$ se e soltanto se u e v si trovano nella stessa componente connessa.

22.3-13 ⋆
Un grafo orientato $G = (V, E)$ è ***singolarmente connesso*** se $u \rightsquigarrow v$ implica che ci sia al più un cammino semplice da u a v per tutti i vertici $u, v \in V$. Create un algoritmo efficiente per determinare se un grafo orientato è singolarmente connesso.

22.4 Ordinamento topologico

Questo paragrafo spiega come utilizzare la visita in profondità per eseguire l'ordinamento topologico di un grafo orientato aciclico o semplicemente "dag" (dall'inglese *directed acyclic graph*). Un ***ordinamento topologico*** di un dag $G = (V, E)$ è un ordinamento lineare di tutti i suoi vertici tale che, se G contiene un arco (u, v), allora u appare prima di v nell'ordinamento (se il grafo non è aciclico, allora non è possibile effettuare alcun ordinamento lineare). Un ordinamento topologico di un grafo può essere visto come un ordinamento dei suoi vertici lungo una linea orizzontale in modo che tutti gli archi orientati siano diretti da sinistra a destra. L'ordinamento topologico è quindi diverso dal tipo usuale di "ordinamento" studiato nella Parte II.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12:11 Numero Ordine Libraria: 1995086 16-230717-0 Copyright © 2022 McGraw-Hill Education (Italy)

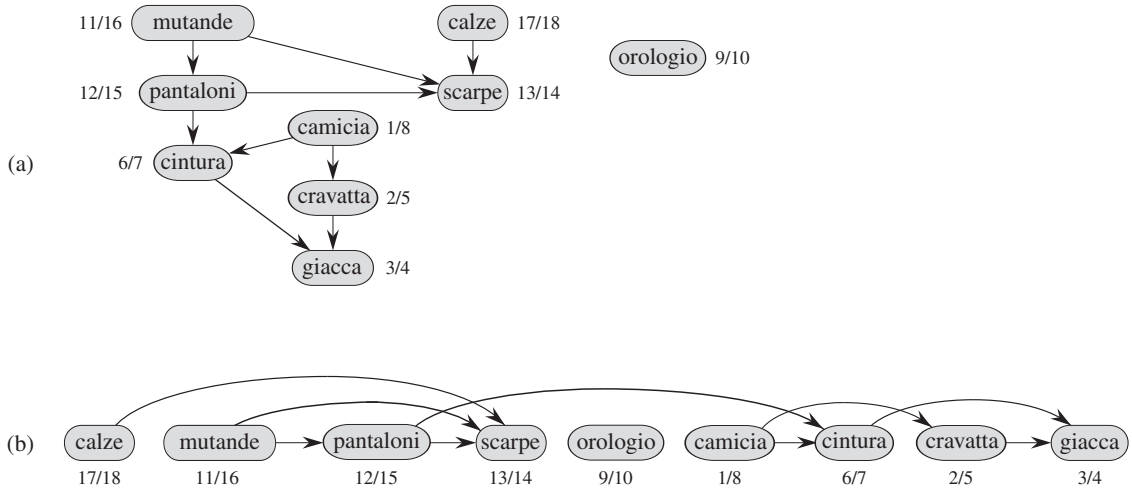


Figura 22.7 **(a)** Il professor Bumstead ordina topologicamente i sui indumenti quando si veste. Ogni arco orientato (u, v) significa che l'indumento u deve essere indossato prima dell'indumento v. I tempi di scoperta e di completamento di una visita in profondità sono illustrati accanto a ciascun vertice. **(b)** Lo stesso grafo rappresentato secondo l'ordinamento topologico. I suoi vertici sono ordinati da sinistra a destra in senso decrescente rispetto ai tempi di completamento. Notate che tutti gli archi orientati sono diretti da sinistra a destra.

I grafi orientati aciclici sono utilizzati in molte applicazioni per indicare le precedenze fra gli eventi. La Figura 22.7 illustra un caso che si verifica tutte le mattine quando il professor Bumstead si veste. Il professore deve indossare determinati indumenti prima di altri (per esempio, le calze prima delle scarpe). Altri indumenti possono essere indossati in qualsiasi ordine (per esempio, le calze e i pantaloni). Un arco orientato (u, v) nel dag della Figura 22.7(a) indica che l'indumento u deve essere indossato prima dell'indumento v. Un ordinamento topologico di questo dag, quindi, determina la sequenza ordinata in cui indossare gli indumenti per vestirsi. La Figura 22.7(b) illustra il dag ordinato topologicamente come un sequenza ordinata di vertici lungo una linea orizzontale tale che tutti gli archi orientati siano diretti da sinistra a destra.

Il seguente semplice algoritmo ordina topologicamente un dag.

TOPOLOGICAL-SORT(G)

1 Chiama DFS(G) per calcolare i tempi di completamento $v.f$
  per ciascun vertice v
2 Una volta completata l'ispezione di un vertice,
  inserisce il vertice in testa a una lista concatenata
3 **return** la lista concatenata dei vertici

La Figura 22.7(b) illustra come si presentano i vertici ordinati topologicamente in senso inverso rispetto ai loro tempi di completamento.

Acquistato da Michele Michele su Website il 2022-07-07 23:12. Numero Ordine Libreria 10050316 220707 0. Copyright © 2022, McGraw-Hill Education (Italy)

È possibile eseguire un ordinamento topologico nel tempo $\Theta(V + E)$, perché la visita in profondità impiega un tempo $\Theta(V + E)$ e occorre un tempo $O(1)$ per inserire ciascuno dei $|V|$ vertici in testa alla lista concatenata.

Dimostriamo la correttezza di questo algoritmo utilizzando il seguente lemma chiave che caratterizza i grafi orientati aciclici.

Lemma 22.11
Un grafo orientato G è aciclico se e soltanto se una visita in profondità di G non genera archi all'indietro.

Dimostrazione $\Rightarrow$: supponiamo che ci sia un arco all'indietro (u, v). Allora, il vertice v è un antenato del vertice u nella foresta DF. Quindi, esiste un cammino che va da v a u nel grafo G e l'arco all'indietro (u, v) completa un ciclo.

$\Leftarrow$: supponiamo che il grafo G contenga un ciclo c. Dimostriamo che una visita in profondità di G genera un arco all'indietro. Sia v il primo vertice che viene scoperto in c e sia (u, v) l'arco precedente in c. Al tempo $v.d$, i vertici di c formano un cammino di vertici bianchi da v a u. Per il teorema del cammino bianco, il vertice u diventa un discendente di v nella foresta DF. Dunque, (u, v) è un arco all'indietro. ■

Teorema 22.12
TOPOLOGICAL-SORT(G) produce un ordinamento topologico di un grafo orientato aciclico G.

Dimostrazione Supponiamo che la procedura DFS venga eseguita su un dato dag $G = (V, E)$ per determinare i tempi di completamento dei suoi vertici. È sufficiente dimostrare che per una coppia qualsiasi di vertici distinti $u, v \in V$, se esiste un arco in G che va da u a v, allora $v.f < u.f$. Consideriamo un arco qualsiasi (u, v) ispezionato da DFS(G). Quando questo arco viene ispezionato, il vertice v non può essere grigio, perché altrimenti v sarebbe un antenato di u e (u, v) sarebbe un arco all'indietro, contraddicendo il Lemma 22.11. Quindi, il vertice v deve essere bianco o nero. Se v è bianco, diventa un discendente di u e quindi $v.f < u.f$. Se v è nero, la sua ispezione è stata già completata, quindi il valore di $v.f$ è già stato impostato. Poiché stiamo ancora ispezionando dal vertice u, dobbiamo ancora assegnare un'informazione temporale a $u.f$ e, quando lo faremo, avremo ancora $v.f < u.f$. Quindi, per qualsiasi arco (u, v) nel dag, si ha $v.f < u.f$, e questo dimostra il teorema. ■

Esercizi

22.4-1
Illustrate come vengono ordinati i vertici quando la procedura TOPOLOGICAL-SORT viene eseguita sul grafo orientato aciclico della Figura 22.8, con le ipotesi fatte nell'Esercizio 22.3-2.

22.4-2
Descrivete un algoritmo con tempo lineare che riceve come input un grafo orientato aciclico $G = (V, E)$ e due vertici s e t, e restituisce il numero di cammini semplici da s a t nel grafo G. Per esempio, nel grafo orientato aciclico della Figura 22.8 ci sono esattamente quattro cammini semplici dal vertice p al vertice v: pov, $poryv$, $posryv$ e $psryv$ (l'algoritmo dovrà soltanto contare i cammini, senza elencarli).

Acquistato da Michele Michele su Webster il 2022-07-07 23:11. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

22.4-3
Descrivete un algoritmo che determina se un grafo non orientato $G = (V, E)$ contiene oppure no un ciclo. Il vostro algoritmo dovrebbe essere eseguito nel tempo $O(V)$, indipendentemente da $|E|$.

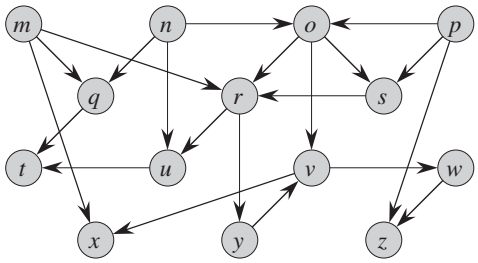


Figura 22.8 Un grafo orientato aciclico per l'ordinamento topologico.

22.4-4

Dimostrare o confutare: se un grafo orientato G contiene dei cicli, allora la procedura TOPOLOGICAL-SORT(G) produce un ordinamento di vertici che minimizza il numero di archi "cattivi" che non sono coerenti con l'ordinamento ottenuto.

22.4-5

Un altro modo di eseguire un ordinamento topologico su un grafo orientato aciclico $G = (V, E)$ consiste nel trovare ripetutamente un vertice di grado entrante pari a 0, stamparlo e quindi rimuoverlo dal grafo insieme a tutti i suoi archi uscenti. Spiegate come implementare questa idea in modo che l'algoritmo di ordinamento possa essere eseguito nel tempo $O(V + E)$. Che cosa accade a questo algoritmo se il grafo G ha dei cicli?

22.5 Componenti fortemente connesse

Consideriamo adesso una classica applicazione della visita in profondità: scomporre un grafo orientato nelle sue componenti fortemente connesse. Questo paragrafo spiega come effettuare questa scomposizione utilizzando due visite in profondità. Molti algoritmi che operano con i grafi orientati iniziano da questa scomposizione. Una volta effettuata la scomposizione, questi algoritmi elaborano separatamente ciascuna componente fortemente connessa e poi combinano le soluzioni in accordo con la struttura delle connessioni fra le componenti.

Ricordiamo dall'Appendice B che una componente fortemente connessa di un grafo orientato $G = (V, E)$ è un insieme massimale di vertici $C \subseteq V$ tale che per ogni coppia di vertici u e v in C, si ha $u \rightsquigarrow v$ e $v \rightsquigarrow u$; ovvero i vertici u e v sono raggiungibili l'uno dall'altro. La Figura 22.9 illustra un esempio.

Il nostro algoritmo per trovare le componenti fortemente connesse di un grafo $G = (V, E)$ utilizza il grafo trasposto di G, che è definito nell'Esercizio 22.1-3 come il grafo $G^{\mathrm{T}} = (V, E^{\mathrm{T}})$, dove $E^{\mathrm{T}} = \{(u, v) : (v, u) \in E\}$. Ovvero E^{T} è formato dagli archi di G con le direzioni invertite. Data una rappresentazione con liste di adiacenza di G, il tempo richiesto per creare G^{T} è $O(V + E)$. È interessante osservare che G e G^{T} hanno esattamente le stesse componenti fortemente connesse: i vertici u e v sono raggiungibili l'uno dall'altro in G, se e soltanto se sono raggiungibili l'uno dall'altro in G^{T}. La Figura 22.9(b) illustra il grafo trasposto del grafo della Figura 22.9(a), con le componenti fortemente connesse su uno sfondo grigio.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Il seguente algoritmo con tempo lineare (cioè con tempo $\Theta(V + E)$) calcola le componenti fortemente connesse di un grafo orientato $G = (V, E)$ utilizzando due visite in profondità, una su G e una su G^{T}.

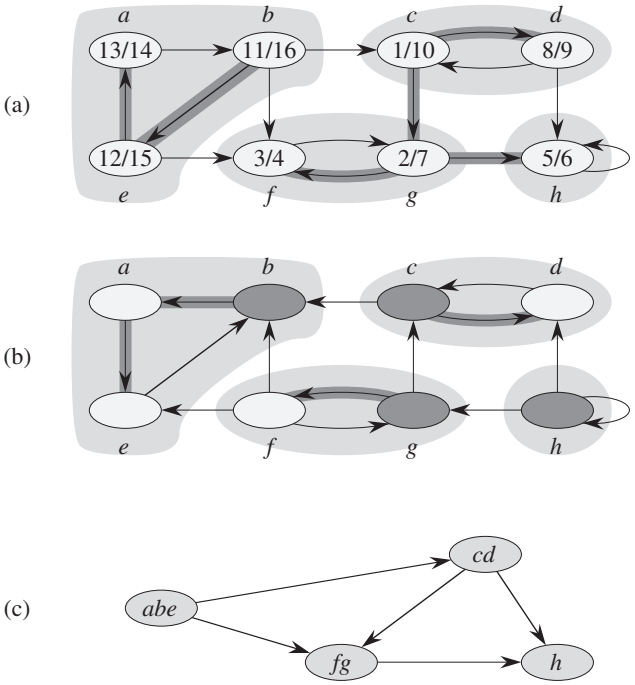


Figura 22.9
(a) Un grafo orientato G. Le componenti fortemente connesse di G sono illustrate come aree ombreggiate. Ogni vertice è etichettato con i suoi tempi di scoperta e di completamento. Gli archi d'albero hanno uno sfondo grigio scuro.
(b) Il grafo G^{T} (trasposto di G). È illustrata la foresta DF calcolata nella riga 3 della procedura STRONGLY-CONNECTED-COMPONENTS con gli archi d'albero su sfondo grigio scuro. Ciascuna componente fortemente connessa corrisponde a un albero DF. I vertici b, c, g e h, anch'essi rappresentati su sfondo grigio scuro, sono le radici degli alberi DF prodotti dalla visita in profondità di G^{T}.
(c) Il grafo aciclico delle componenti G^{SCC} ottenuto contraendo tutti gli archi all'interno di ciascuna componente fortemente connessa di G in modo che resti un solo vertice in ciascuna componente.

STRONGLY-CONNECTED-COMPONENTS(G)

1 Chiama DFS(G) per calcolare i tempi di completamento $u.f$ per ciascun vertice u
2 Calcola G^{T}
3 Chiama DFS(G^{T}), ma nel ciclo principale di DFS, considera i vertici in ordine decrescente rispetto ai tempi $u.f$ (calcolati nella riga 1)
4 Genera l'output dei vertici di ciascun albero della foresta DF che è stata prodotta nella riga 3 come una singola componente fortemente connessa

L'idea che sta alla base dell'algoritmo deriva da una proprietà fondamentale del ***grafo delle componenti*** $G^{\mathrm{SCC}} = (V^{\mathrm{SCC}}, E^{\mathrm{SCC}})$, che è definito nel seguente modo. Supponiamo che G abbia le componenti fortemente connesse $C_1, C_2, \ldots, C_k$. L'insieme dei vertici V^{SCC} è $\{v_1, v_2, \ldots, v_k\}$ e contiene un vertice v_i per ogni componente fortemente connessa C_i di G. Esiste un arco $(v_i, v_j) \in E^{\mathrm{SCC}}$ se G contiene un arco orientato (x, y) per qualche $x \in C_i$ e qualche $y \in C_j$. In altri termini, contraendo tutti gli archi i cui vertici incidenti sono all'interno della stessa componente fortemente connessa di G, si ottiene il grafo G^{SCC}. La Figura 22.9(c) illustra il grafo delle componenti del grafo della Figura 22.9(a).

La proprietà fondamentale è che il grafo delle componenti è un grafo orientato aciclico, che è implicata dal seguente lemma.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Lemma 22.13
Siano C e C' due componenti fortemente connesse distinte nel grafo orientato $G = (V, E)$. Se $u, v \in C$ e $u', v' \in C'$ e supponendo che ci sia un cammino $u \rightsquigarrow u'$ in G, allora non può esistere anche un cammino $v' \rightsquigarrow v$ in G.

Dimostrazione Se esiste un cammino $v' \rightsquigarrow v$ in G, allora esistono i cammini $u \rightsquigarrow u' \rightsquigarrow v'$ e $v' \rightsquigarrow v \rightsquigarrow u$ in G. Quindi, u e v' sono raggiungibili l'uno dall'altro, contraddicendo così l'ipotesi che C e C' siano componenti fortemente connesse e distinte. ■

Vedremo che, esaminando i vertici nella seconda visita in profondità in ordine decrescente rispetto ai tempi di completamento che sono stati calcolati nella prima visita in profondità, in essenza, stiamo visitando i vertici del grafo delle componenti (ciascuno dei quali corrisponde a una componente fortemente connessa di G) secondo un ordinamento topologico.

Poiché la procedura STRONGLY-CONNECTED-COMPONENTS effettua due visite in profondità, corriamo il rischio di essere ambigui quando parliamo di $u.d$ o $u.f$. In questo paragrafo, questi valori fanno sempre riferimento ai tempi di scoperta e di completamento che sono stati calcolati dalla prima chiamata di DFS nella riga 1.

Estendiamo la notazione dei tempi di scoperta e di completamento agli insiemi dei vertici. Se $U \subseteq V$, allora definiamo $d(U) = \min_{u \in U} \{u.d\}$ e $f(U) = \max_{u \in U} \{u.f\}$. Ovvero $d(U)$ e $f(U)$ sono, rispettivamente, il primo tempo di scoperta e l'ultimo tempo di completamento di un vertice qualsiasi in U.

Il seguente lemma e il suo corollario definiscono una proprietà fondamentale che mette in relazione le componenti fortemente connesse e i tempi di completamento nella prima visita in profondità.

Lemma 22.14
Siano C e C' delle componenti fortemente connesse e distinte nel grafo orientato $G = (V, E)$. Supponiamo che esista un arco $(u, v) \in E$, dove $u \in C$ e $v \in C'$. Allora $f(C) > f(C')$.

Dimostrazione Ci sono due casi, a seconda di quale componente fortemente connessa, C o C', contiene il primo vertice che viene scoperto durante la visita in profondità.

Se $d(C) < d(C')$, indichiamo con x il primo vertice scoperto in C. Al tempo $x.d$, tutti i vertici in C e C' sono bianchi. Esiste un cammino in G da x a ciascun vertice in C che è formato soltanto da vertici bianchi. Poiché $(u, v) \in E$, per un vertice qualsiasi $w \in C'$, al tempo $x.d$ esiste anche un cammino da x a w in G che è formato soltanto da vertici bianchi: $x \rightsquigarrow u \rightarrow v \rightsquigarrow w$. Per il teorema del cammino bianco, tutti i vertici in C e C' diventano discendenti di x nell'albero DF. Per il Corollario 22.8, $x.f = f(C) > f(C')$.

Se, invece, $d(C) > d(C')$, indichiamo con y il primo vertice scoperto in C'. Al tempo $y.d$, tutti i vertici in C' sono bianchi ed esiste un cammino in G da y a ciascun vertice in C' che è formato soltanto da vertici bianchi. Per il teorema del cammino bianco, tutti i vertici in C' diventano discendenti di y nell'albero DF e, per il Corollario 22.8, $y.f = f(C')$. Al tempo $y.d$, tutti i vertici in C sono bianchi. Poiché esiste un arco (u, v) da C a C', il Lemma 22.13 implica che non può esistere un cammino da C' a C. Pertanto, nessun vertice in C è raggiungibile da y. Al tempo $y.f$, quindi, tutti i vertici in C sono ancora bianchi. Dunque, per un vertice qualsiasi $w \in C$, si ha $w.f > y.f$, e questo implica che $f(C) > f(C')$. ■

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 1995035016-2207077-0 Copyright © 2022, McGraw-Hill Education (Italy)

Il seguente corollario ci dice che ogni arco in G^T che congiunge differenti componenti fortemente connesse va da una componente con un tempo di completamento antecedente (nella prima visita in profondità) a una componente con un tempo di completamento successivo.

Corollario 22.15
Siano C e C' delle componenti fortemente connesse e distinte nel grafo orientato $G = (V, E)$. Supponiamo che esista un arco $(u, v) \in E^T$, dove $u \in C$ e $v \in C'$. Allora $f(C) < f(C')$.

Dimostrazione Poiché $(u, v) \in E^T$, si ha $(v, u) \in E$. Poiché le componenti fortemente connesse di G e G^T sono le stesse, il Lemma 22.14 implica che $f(C) < f(C')$. ■

Il Corollario 22.15 è fondamentale per capire perché funziona la procedura STRONGLY-CONNECTED-COMPONENTS. Esaminiamo che cosa accade quando eseguiamo la seconda visita in profondità, che si svolge su G^T. Iniziamo con la componente fortemente connessa C il cui tempo di completamento $f(C)$ è massimo. La visita inizia da qualche vertice $x \in C$ e prosegue su tutti i vertici in C. Per il Corollario 22.15, non ci sono archi in G^T che vanno da C a un'altra componente fortemente connessa, quindi la visita da x non passerà per i vertici di un'altra componente. Pertanto, l'albero radicato in x contiene esattamente i vertici di C.

Una volta completata la visita di tutti i vertici in C, la riga 3 seleziona come radice un vertice di qualche altra componente fortemente connessa C', il cui tempo di completamento $f(C')$ è massimo rispetto a tutte le altre componenti, tranne C. Saranno visitati tutti i vertici in C', ma per il Corollario 22.15 gli unici archi in G^T che vanno da C' a un'altra componente devono essere diretti verso C, che è già stata visitata. In generale, quando la visita in profondità di G^T nella riga 3 si svolge su una componente fortemente connessa, tutti gli archi che escono da tale componente devono essere diretti verso componenti già visitate. Ciascun albero DF, quindi, rappresenterà esattamente una sola componente fortemente connessa. Il seguente teorema formalizza questo ragionamento.

Teorema 22.16
La procedura STRONGLY-CONNECTED-COMPONENTS(G) calcola correttamente le componenti fortemente connesse di un grafo orientato G.

Dimostrazione Dimostriamo per induzione sul numero di alberi DF trovati nella visita in profondità di G^T nella riga 3 che i vertici di ciascun albero formano una componente fortemente connessa. L'ipotesi induttiva è che i primi k alberi prodotti nella riga 3 siano componenti fortemente connesse. Il caso base dell'induzione, quando $k = 0$, è banale.

Nel passo induttivo, supponiamo che ciascuno dei primi k alberi DF prodotti nella riga 3 sia una componente fortemente connessa e prendiamo in considerazione il $(k+1)$-esimo albero prodotto. Supponiamo inoltre che la radice di questo albero sia il vertice u e che u si trovi nella componente fortemente connessa C.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 (Numero Ordine Libreria 1995202616220737) Copyright © 2023 McGraw-Hill Education (Italy)

Considerando il modo in cui abbiamo scelto le radici nella visita in profondità nella riga 3, abbiamo $u.f = f(C) > f(C')$ per qualsiasi componente fortemente connessa C' diversa da C che non è stata ancora visitata. Per l'ipotesi induttiva, nel momento in cui viene visitato il vertice u, tutti gli altri vertici di C sono bianchi. Per il teorema del cammino bianco, quindi, tutti gli altri vertici di C sono

discendenti del vertice u nel suo albero DF. Inoltre, per l'ipotesi induttiva e per il Corollario 22.15, tutti gli archi in G^{T} che escono da C devono essere diretti verso componenti fortemente connesse che sono state già visitate. Quindi, nessun vertice in qualsiasi componente fortemente connessa diversa da C sarà un discendente di u durante la visita in profondità di G^{T}. Dunque, i vertici dell'albero DF in G^{T} che è radicato in u formano esattamente una sola componente fortemente connessa; questo completa il passo induttivo e la dimostrazione del teorema. ■

Ecco un altro modo di vedere come opera la seconda visita in profondità. Consideriamo il grafo delle componenti $(G^{\mathrm{T}})^{\mathrm{SCC}}$ di G^{T}. Se associamo ciascuna componente fortemente connessa (che è stata ispezionata nella seconda visita in profondità) a un vertice di $(G^{\mathrm{T}})^{\mathrm{SCC}}$, i vertici di $(G^{\mathrm{T}})^{\mathrm{SCC}}$ vengono visitati nel senso inverso di un ordinamento topologico. Se invertiamo gli archi di $(G^{\mathrm{T}})^{\mathrm{SCC}}$, otteniamo il grafo $((G^{\mathrm{T}})^{\mathrm{SCC}})^{\mathrm{T}}$. Poiché $((G^{\mathrm{T}})^{\mathrm{SCC}})^{\mathrm{T}} = G^{\mathrm{SCC}}$ (vedere l'Esercizio 22.5-4), la seconda visita in profondità esamina i vertici di G^{SCC} nel senso diretto di un ordinamento topologico.

Esercizi

22.5-1
Come cambia il numero delle componenti fortemente connesse di un grafo se viene aggiunto un nuovo arco?

22.5-2
Spiegate come opera la procedura STRONGLY-CONNECTED-COMPONENTS con il grafo della Figura 22.6. Specificatamente, indicate i tempi di completamento calcolati nella riga 1 e illustrate la foresta prodotta nella riga 3. Supponete che il ciclo nelle righe 5–7 della procedura DFS consideri i vertici in ordine alfabetico e che le liste di adiacenza siano in ordine alfabetico.

22.5-3
Il professor Bacon ritiene che l'algoritmo per le componenti fortemente connesse possa essere semplificato utilizzando il grafo originale (anziché il grafo trasposto) nella seconda visita in profondità e ispezionando i vertici in ordine *crescente* rispetto ai tempi di completamento. Il professore ha ragione?

22.5-4
Dimostrate che, per qualsiasi grafo orientato G, si ha $((G^{\mathrm{T}})^{\mathrm{SCC}})^{\mathrm{T}} = G^{\mathrm{SCC}}$. Ovvero il grafo trasposto del grafo delle componenti di G^{T} è uguale al grafo delle componenti di G.

22.5-5
Create un algoritmo con tempo $O(V + E)$ per calcolare il grafo delle componenti di un grafo orientato $G = (V, E)$. Verificate che ci sia al più un arco fra due vertici nel grafo delle componenti prodotto dal vostro algoritmo.

Acquistato da Michele Mele (michele.mele@studenti.unipd.it) - Prodotto con licenza 9788838670707-0 Copyright © 2022, McGraw-Hill Education (Italy)

22.5-6
Dato un grafo orientato $G = (V, E)$, spiegate come creare un altro grafo $G' = (V, E')$ tale che (a) G' abbia le stesse componenti fortemente connesse di G, (b) G' abbia lo stesso grafo delle componenti di G e (c) E' sia più piccolo possibile. Descrivete un algoritmo veloce per calcolare G'.

22.5-7
Un grafo orientato $G = (V, E)$ è detto ***semiconnesso*** se, per qualsiasi coppia di vertici $u, v \in V$, si ha $u \rightsquigarrow v$ o $v \rightsquigarrow u$. Create un algoritmo efficiente per determinare se G è semiconnesso. Dimostrate che il vostro algoritmo è corretto e analizzate il suo tempo di esecuzione.

Problemi

22-1 Classificare gli archi con una visita in ampiezza
Una foresta DF classifica gli archi di un grafo in archi d'albero, archi all'indietro, archi in avanti e archi trasversali. Anche un albero BF può essere utilizzato per classificare nelle stesse quattro categorie gli archi raggiungibili dal vertice sorgente della visita.

a. Dimostrate che, nella visita in ampiezza di un grafo non orientato, valgono le seguenti proprietà:

1. Non ci sono archi all'indietro né archi in avanti.
2. Per ogni arco d'albero (u, v), si ha $d[v] = d[u] + 1$.
3. Per ogni arco trasversale (u, v), si ha $v.d = u.d$ oppure $v.d = u.d + 1$.

b. Dimostrate che, nella visita in ampiezza di un grafo orientato, valgono le seguenti proprietà:

1. Non ci sono archi in avanti.
2. Per ogni arco d'albero (u, v), si ha $v.d = u.d + 1$.
3. Per ogni arco trasversale (u, v), si ha $v.d \leq u.d + 1$.
4. Per ogni arco all'indietro (u, v), si ha $0 \leq v.d \leq u.d$.

22-2 Punti di articolazione, ponti e componenti biconnesse
Sia $G = (V, E)$ un grafo connesso non orientato. Un ***punto di articolazione*** di G è un vertice la cui rimozione fa sì che G non sia più un grafo connesso. Un ***ponte*** di G è un arco la cui rimozione fa sì che G non sia più un grafo connesso. Una ***componente biconnessa*** di G è un insieme massimale di archi tale che due suoi archi qualsiasi giacciono in un ciclo semplice comune. La Figura 22.10 illustra gli elementi di queste definizioni. È possibile determinare i punti di articolazione, i ponti e le componenti biconnesse utilizzando una visita in profondità. Sia $G_\pi = (V, E_\pi)$ un albero DF del grafo G.

a. Dimostrate che la radice di G_π è un punto di articolazione di G se e soltanto se ha almeno due figli in G_π.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12, Numero Ordine Libreria: 1805032016-200707-0, Copyright © 2022, McGraw-Hill Education (Italy)

b. Sia v un vertice di G_π diverso dalla radice. Dimostrate che v è un punto di articolazione di G se e soltanto se v ha un figlio s per il quale non esiste alcun arco all'indietro che va da s o da un discendente qualsiasi di s verso un antenato proprio di v.

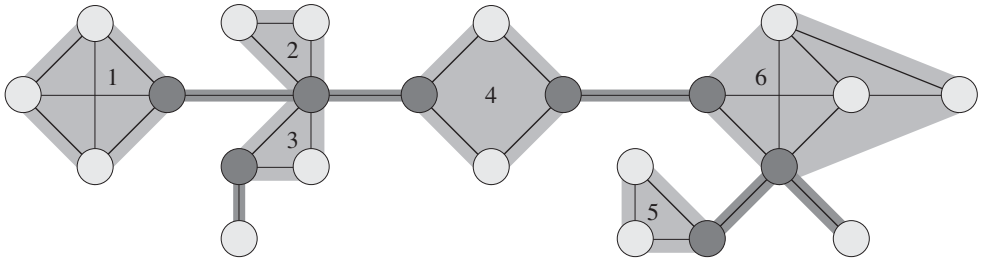


Figura 22.10 I punti di articolazione, i ponti e le componenti biconnesse del grafo connesso non orientato da utilizzare nel Problem 22-2. I punti di articolazione sono i vertici di colore grigio scuro, i ponti sono gli archi di colore grigio scuro e le componenti biconnesse sono gli archi nelle aree ombreggiate, in cui è indicata la numerazione *bcc*.

c. Sia

$$v.low = \min \begin{cases} v.d \\ w.d : (u, w) \text{ è un arco all'indietro} \\ \quad \text{per qualche discendente } u \text{ di } v \end{cases}$$

Spiegate come calcolare $v.low$ per tutti i vertici $v \in V$ nel tempo $O(E)$.

d. Spiegate come calcolare tutti i punti di articolazione nel tempo $O(E)$.

e. Dimostrate che un arco di G è un ponte se e soltanto se esso non giace su un ciclo semplice di G.

f. Spiegate come calcolare tutti i ponti di G nel tempo $O(E)$.

g. Dimostrate che le componenti biconnesse di G partizionano gli archi di G che non sono ponti.

h. Create un algoritmo con tempo $O(E)$ per etichettare ciascun arco e di G con un numero intero positivo $e.bcc$ tale che $e.bcc = e'.bcc$ se e soltanto se e ed e' sono nella stessa componente biconnessa.

22-3 Ciclo euleriano

Un ***ciclo euleriano*** di un grafo orientato fortemente connesso $G = (V, E)$ è un ciclo che attraversa ciascun arco di G una sola volta, sebbene possa visitare un vertice più di una volta.

a. Dimostrate che G ha un ciclo euleriano se e soltanto se in-degree(v) = out-degree(v) per ogni vertice $v \in V$, dove in-degree(v) e out-degree(v) sono, rispettivamente, il grado entrante e il grado uscente di v.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

b. Descrivete un algoritmo con tempo $O(E)$ per trovare un ciclo euleriano del grafo G, se esiste (*suggerimento:* riunite i cicli disgiunti sugli archi).

22-4 Raggiungibilità

Sia $G = (V, E)$ un grafo orientato dove ciascun vertice $u \in V$ è etichettato con un numero intero unico $L(u)$ nell'insieme $\{1, 2, \ldots, |V|\}$. Per ogni vertice $u \in V$, sia $R(u) = \{v \in V : u \rightsquigarrow v\}$ l'insieme dei vertici che sono raggiungibili da u. Definiamo $\min(u)$ come il vertice in $R(u)$ la cui etichetta è minima, ovvero $\min(u)$ è il vertice v tale che $L(v) = \min\{L(w) : w \in R(u)\}$. Descrivete un algoritmo con tempo $O(V + E)$ che calcola $\min(u)$ per tutti i vertici $u \in V$.

Note

Even [104] e Tarjan [331] sono testi eccellenti sugli algoritmi per grafi. La visita in ampiezza è stata scoperta da Moore [261] nel contesto della ricerca di percorsi nei labirinti. Lee [227] ha scoperto, in modo autonomo, lo stesso algoritmo nel contesto delle connessioni nei circuiti stampati.

Hopcroft e Tarjan [179] hanno sostenuto la superiorità delle liste di adiacenza sulle matrici di adiacenza nella rappresentazione dei grafi sparsi; sono stati i primi a riconoscere l'importanza algoritmica della visita in profondità. La visita in profondità è stata ampiamente utilizzata a partire dagli ultimi anni '50, specialmente nei programmi di intelligenza artificiale.

Tarjan [328] ha creato un algoritmo in tempo lineare per trovare le componenti fortemente connesse. L'algoritmo per le componenti fortemente connesse descritto nel Paragrafo 22.5 è stato adattato da Aho, Hopcroft e Ullman [6], che ne attribuiscono la paternità a S. R. Kosaraju (nessuna pubblicazione) e M. Sharir [315]. Anche Gabow [120] ha sviluppato un algoritmo per le componenti fortemente connesse che si basa sulla contrazione dei cicli e usa due stack per ottenere un tempo di esecuzione lineare dell'algoritmo. Knuth [210] è stato il primo a creare un algoritmo in tempo lineare per l'ordinamento topologico.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Alberi di connessione minimi 23

Nella progettazione dei circuiti elettronici spesso è necessario rendere elettricamente equivalenti i pin di più componenti collegandoli insieme. Per interconnettere un insieme di n pin possiamo utilizzare un sistema di $n-1$ fili conduttori, ciascuno dei quali collega due pin. Fra tutti questi sistemi, di solito, viene preferito quello che usa la minor quantità di filo conduttore.

Possiamo modellare questo problema di cablaggio tramite un grafo connesso non orientato $G = (V, E)$, dove V è l'insieme dei pin ed E è l'insieme delle possibili interconnessioni fra le coppie di pin; inoltre a ogni arco $(u, v) \in E$ è dato un peso $w(u, v)$ che specifica il costo (quantità necessaria di filo conduttore) per collegare u e v. Vogliamo trovare un sottoinsieme aciclico $T \subseteq E$ che collega tutti i vertici, il cui peso totale

$$w(T) = \sum_{(u,v)\in T} w(u, v)$$

sia minimo. Poiché T è aciclico e collega tutti i vertici, deve anche formare un albero, che è detto ***albero di connessione*** perché "connette" il grafo G. Il problema di trovare l'albero T è detto ***problema dell'albero di connessione minimo***.[1] La Figura 23.1 illustra un esempio di grafo connesso e il suo albero di connessione minimo.

In questo capitolo, esamineremo due algoritmi per risolvere il problema dell'albero di connessione minimo: l'algoritmo di Kruskal e l'algoritmo di Prim. Entrambi gli algoritmi possono essere eseguiti nel tempo $O(E \lg V)$ utilizzando dei normali heap binari. Utilizzando gli heap di Fibonacci, l'algoritmo di Prim può essere eseguito più rapidamente nel tempo $O(E + V \lg V)$, che è un miglioramento se $|V|$ è molto più piccolo di $|E|$.

I due algoritmi sono algoritmi golosi (descritti nel Capitolo 16). A ogni passo di un algoritmo goloso deve essere fatta una delle tante scelte possibili. La strategia golosa prevede di fare la scelta che in quel particolare momento è ritenuta la migliore. In generale, questa strategia non garantisce che vengano trovate le soluzioni globalmente ottime per i problemi. Tuttavia, per il problema dell'albero di connessione minimo, è possibile dimostrare che alcune strategie golose forniscono un albero di connessione con peso minimo. Sebbene il presente capitolo possa essere letto indipendentemente dal Capitolo 16, tuttavia i metodi golosi che ci accingiamo a presentare sono un'applicazione classica dei concetti teorici introdotti nel Capitolo 16.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria 1995030[illegible]-220707-0. Copyright © 2022, McGraw-Hill Education (Italy)

[1] Il termine "albero di connessione minimo" è un'abbreviazione di "albero di connessione di peso minimo." Non stiamo, per esempio, riducendo al minimo il numero di archi in T, perché tutti gli alberi di connessione hanno esattamente $|V| - 1$ archi per il Teorema B.2.

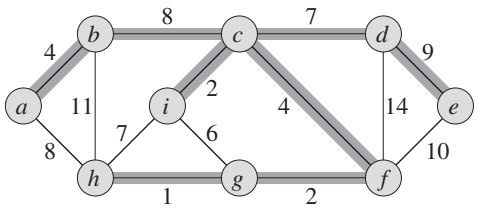


Figura 23.1 Un albero di connessione minimo per un grafo connesso. Sono indicati i pesi degli archi. Gli archi di un albero di connessione minimo sono rappresentati su sfondo grigio. Il peso totale dell'albero è 37. Questo albero di connessione minimo non è unico: eliminando l'arco (b, c) e sostituendolo con l'arco (a, h) si ottiene un altro albero di connessione con peso 37.

Il Paragrafo 23.1 presenta un "metodo generico" per l'albero di connessione minimo che fa crescere un albero di connessione aggiungendo un arco alla volta. Il Paragrafo 23.2 descrive due algoritmi che implementano questo metodo generico. Il primo algoritmo, sviluppato da Kruskal, è simile all'algoritmo per le componenti connesse descritto nel Paragrafo 21.1. Il secondo algoritmo, dovuto a Prim, è simile all'algoritmo di Dijkstra per i cammini minimi (Paragrafo 24.4).

Poiché un albero è un tipo di grafo, per essere precisi dobbiamo definire un albero in termini non soltanto dei suoi archi, ma anche dei suoi vertici. Sebbene questo capitolo tratti gli alberi in base ai loro archi, noi opereremo sapendo che i vertici di un albero T sono quelli nei quali incide qualche arco di T.

23.1 Creare un albero di connessione minimo

Supponiamo di avere un grafo connesso non orientato $G = (V, E)$ con una funzione peso $w : E \to \mathbb{R}$ e di volere trovare un albero di connessione minimo per il grafo G. Entrambi gli algoritmi che consideriamo in questo capitolo usano un approccio goloso per risolvere il problema, ma applicano in modo differente tale approccio.

Questa strategia golosa è sintetizzata nel seguente "metodo generico", che fa crescere l'albero di connessione minimo di un arco alla volta. Il metodo generico gestisce un insieme di archi A, conservando la seguente invariante di ciclo:

> Prima di ogni iterazione, A è un sottoinsieme di qualche albero di connessione minimo.

A ogni passo, determiniamo un arco (u, v) che può essere aggiunto ad A senza violare questa invariante, nel senso che $A \cup \{(u, v)\}$ è anche un sottoinsieme di un albero di connessione minimo. Tale arco è detto ***arco sicuro*** per A, perché può essere tranquillamente aggiunto ad A preservando l'invariante.

GENERIC-MST(G, w)

```
1  A = ∅
2  while A non forma un albero di connessione
3      trova un arco (u, v) che è sicuro per A
4      A = A ∪ {(u, v)}
5  return A
```

Utilizziamo l'invariante di ciclo nel modo seguente:

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Inizializzazione: dopo la riga 1, l'insieme A soddisfa banalmente l'invariante di ciclo.

Conservazione: il ciclo nelle righe 2–4 conserva l'invariante perché aggiunge soltanto archi sicuri.

Conclusione: tutti gli archi aggiunti ad A si trovano in un albero di connessione minimo, quindi l'insieme A restituito nella riga 5 deve essere un albero di connessione minimo.

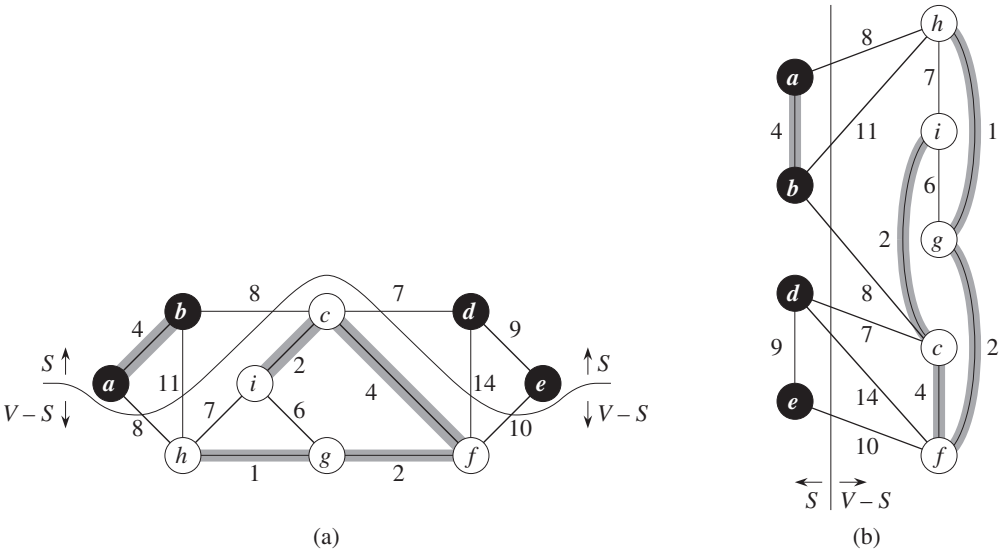


Figura 23.2 Due modi di vedere un taglio $(S, V - S)$ del grafo illustrato nella Figura 23.1. **(a)** I vertici nell'insieme S sono rappresentati in nero, quelli nell'insieme $V - S$ sono rappresentati in bianco. Gli archi che attraversano il taglio sono quelli che collegano i vertici bianchi con i vertici neri. L'arco (d, c) è l'unico arco leggero che attraversa il taglio. Un sottoinsieme A degli archi è rappresentato con uno sfondo grigio; notate che il taglio $(S, V - S)$ rispetta A, perché nessun arco di A attraversa il taglio. **(b)** Lo stesso grafo con i vertici nell'insieme S a sinistra e i vertici nell'insieme $V - S$ a destra. Un arco attraversa il taglio se collega un vertice a sinistra con un vertice a destra.

La parte più difficile è, ovviamente, trovare un arco sicuro nella riga 3. Un arco sicuro deve esistere, perché quando viene eseguita la riga 3, l'invariante impone che ci sia un albero di connessione T tale che $A \subseteq T$. All'interno del corpo del ciclo **while**, A deve essere un sottoinsieme proprio di T, quindi deve esistere un arco $(u, v) \in T$ tale che $(u, v) \notin A$ e (u, v) è un arco sicuro per A.

Nella parte restante di questo paragrafo, definiremo una regola (Teorema 23.1) per riconoscere gli archi sicuri. Il prossimo paragrafo descrive due algoritmi che applicano questa regola per trovare in maniera efficiente gli archi sicuri.

Innanzi tutto abbiamo bisogno di alcune definizioni. Un ***taglio*** $(S, V - S)$ di un grafo non orientato $G = (V, E)$ è una partizione di V. La Figura 23.2 illustra questo concetto. Si dice che un arco $(u, v) \in E$ ***attraversa*** il taglio $(S, V - S)$ se una delle sue estremità si trova in S e l'altra in $V - S$.

Si dice che un taglio ***rispetta*** un insieme A di archi se nessun arco di A attraversa il taglio. Un arco è un ***arco leggero*** per un taglio se il suo peso è il minimo fra i pesi degli altri archi che attraversano il taglio. Notate che ci possono essere più archi leggeri che attraversano un taglio nel caso di pesi uguali. Più in generale, un arco è un ***arco leggero*** per una data proprietà se il suo peso è il minimo fra tutti gli archi che soddisfa tale proprietà.

La nostra regola per riconoscere gli archi sicuri è definita dal seguente teorema.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Teorema 23.1

Sia $G = (V, E)$ un grafo connesso non orientato con una funzione peso w a valori reali definita in E. Sia A un sottoinsieme di E che è contenuto in qualche albero di connessione minimo per G, sia $(S, V - S)$ un taglio qualsiasi di G che rispetta A e sia (u, v) un arco leggero che attraversa $(S, V - S)$. Allora, l'arco (u, v) è sicuro per A.

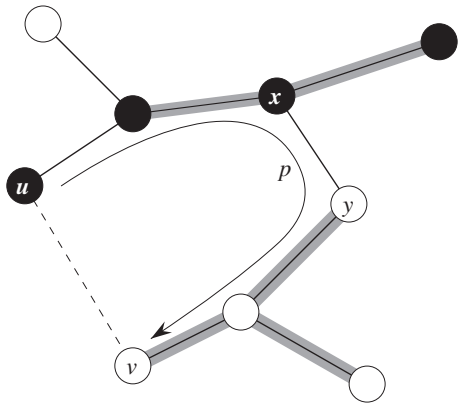


Figura 23.3 La dimostrazione del Teorema 23.1. I vertici in S sono neri; i vertici in $V - S$ sono bianchi. Sono rappresentati gli archi nell'albero di connessione minimo T, mentre gli archi nel grafo G non sono rappresentati. Gli archi in A hanno uno sfondo grigio; (u, v) è un arco leggero che attraversa il taglio $(S, V - S)$. L'arco (x, y) è un arco nell'unico cammino semplice p che va da u a v in T. Un albero di connessione minimo T' che contiene (u, v) si ottiene eliminando l'arco (x, y) da T e aggiungendo l'arco (u, v).

Dimostrazione Sia T un albero di connessione minimo che contiene A e supponiamo che T non contenga l'arco leggero (u, v), perché se lo contenesse, il teorema sarebbe dimostrato. Costruiremo un altro albero di connessione minimo T' che include $A \cup \{(u, v)\}$ utilizzando una tecnica "taglia e incolla", dimostrando così che (u, v) è un arco sicuro per A.

L'arco (u, v) forma un ciclo con gli archi nel cammino semplice p che va da u a v in T, come è illustrato nella Figura 23.3. Poiché u e v si trovano su lati opposti del taglio $(S, V - S)$, c'è almeno un altro arco in T che appartiene al cammino semplice p e che attraversa il taglio. Sia (x, y) uno di questi archi. L'arco (x, y) non appartiene ad A, perché il taglio rispetta A. Poiché (x, y) si trova nel cammino semplice unico da u a v in T, eliminando (x, y), l'albero T si spezza in due componenti. Aggiungendo (u, v), le due componenti si ricongiungono per formare un nuovo albero di connessione $T' = T - \{(x, y)\} \cup \{(u, v)\}$.

Dimostriamo adesso che T' è un albero di connessione minimo. Poiché (u, v) è un arco leggero che attraversa $(S, V - S)$ e anche l'arco (x, y) attraversa questo taglio, allora $w(u, v) \leq w(x, y)$. Quindi, si ha

$$\begin{aligned} w(T') &= w(T) - w(x, y) + w(u, v) \\ &\leq w(T) \end{aligned}$$

Ma T è un albero di connessione minimo, quindi $w(T) \leq w(T')$; di conseguenza, anche T' deve essere un albero di connessione minimo.

Resta da dimostrare che (u, v) è effettivamente un arco sicuro per A. Sappiamo che $A \subseteq T'$, in quanto $A \subseteq T$ e $(x, y) \notin A$; quindi $A \cup \{(u, v)\} \subseteq T'$. Di conseguenza, poiché T' è un albero di connessione minimo, (u, v) è un arco sicuro per A. ■

Il Teorema 23.1 ci consente di capire meglio il funzionamento del metodo GENERIC-MST con un grafo connesso $G = (V, E)$. Durante l'esecuzione del metodo, l'insieme A è sempre aciclico; altrimenti un albero di connessione minimo che include A conterrebbe un ciclo, e ciò sarebbe una contraddizione. In qualsiasi momento dell'esecuzione, il grafo $G_A = (V, A)$ è una foresta e ciascuna delle componenti connesse di G_A è un albero. (Alcuni alberi possono contenere un solo vertice, come per esempio quando inizia l'algoritmo: A è vuoto e la foresta contiene $|V|$ alberi, uno per ogni vertice.) Inoltre, qualsiasi arco sicuro (u, v) per A collega componenti distinte di G_A, perché $A \cup \{(u, v)\}$ deve essere aciclico.

Acquistato da Michele Michele su Webster il 2022-07-07 23:43. Numero Ordine Libraria 1000310. Copyright © 2010 McGraw-Hill Education (Italy)

Il ciclo **while** nelle righe 2–4 di GENERIC-MST viene eseguito $|V| - 1$ volte, in quanto i $|V| - 1$ archi di un albero di connessione minimo vengono determinati uno dopo l'altro. Inizialmente, quando $A = \emptyset$, ci sono $|V|$ alberi in G_A e ogni iterazione riduce questo numero di 1. Quando la foresta contiene un albero soltanto, il metodo termina.

I due algoritmi descritti nel Paragrafo 23.2 usano il seguente corollario del Teorema 23.1.

Corollario 23.2
Sia $G = (V, E)$ un grafo connesso non orientato con una funzione peso w a valori reali definita in E. Sia A un sottoinsieme di E che è contenuto in qualche albero di connessione minimo per G e sia $C = (V_C, E_C)$ una componente connessa (un albero) nella foresta $G_A = (V, A)$. Se (u, v) è un arco leggero che collega C a qualche altra componente in G_A, allora (u, v) è sicuro per A.

Dimostrazione Il taglio $(V_C, V - V_C)$ rispetta A e (u, v) è un arco leggero per questo taglio. Quindi, (u, v) è sicuro per A. ■

Esercizi

23.1-1
Sia (u, v) un arco di peso minimo in un grafo connesso G. Dimostrate che (u, v) appartiene a qualche albero di connessione minimo di G.

23.1-2
Il professor Sabatier propone il seguente teorema come inverso del Teorema 23.1. Sia $G = (V, E)$ un grafo connesso non orientato con una funzione peso w a valori reali definita in E. Sia A un sottoinsieme di E che è contenuto in qualche albero di connessione minimo per G, sia $(S, V - S)$ un taglio qualsiasi di G che rispetta A e sia (u, v) un arco sicuro per A che attraversa $(S, V - S)$. Allora, (u, v) è un arco leggero per il taglio. Dimostrate che il professore si sbaglia fornendo un controesempio.

23.1-3
Dimostrate che, se un arco (u, v) è contenuto in qualche albero di connessione minimo, allora esso è un arco leggero che attraversa qualche taglio del grafo.

23.1-4
Fornite un semplice esempio di grafo connesso tale che l'insieme degli archi $\{(u, v) :$ esiste un taglio $(S, V - S)$ tale che (u, v) sia un arco leggero che attraversa $(S, V - S)\}$ non formi un albero di connessione minimo.

23.1-5
Sia e un arco di peso massimo in qualche ciclo del grafo connesso $G = (V, E)$. Dimostrate che esiste un albero di connessione minimo di $G' = (V, E - \{e\})$ che è anche un albero di connessione minimo di G. Ovvero esiste un albero di connessione minimo di G che non include e.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

23.1-6
Dimostrate che un grafo ha un unico albero di connessione minimo se, per ogni taglio del grafo, esiste un unico arco leggero che attraversa il taglio. Dimostrate che il contrario non è vero fornendo un controesempio.

23.1-7
Dimostrate che, se tutti i pesi degli archi di un grafo sono positivi, allora qualsiasi sottoinsieme di archi che collega tutti i vertici e ha un peso totale minimo deve essere un albero. Trovate un esempio per dimostrare che la stessa conclusione non vale se si ammette che qualche peso possa essere non positivo.

23.1-8
Sia T un albero di connessione minimo di un grafo G e sia L la lista ordinata dei pesi degli archi di T. Dimostrate che per qualsiasi altro albero di connessione minimo T' di G, la lista L è anche la lista ordinata dei pesi degli archi di T'.

23.1-9
Sia T un albero di connessione minimo di un grafo $G = (V, E)$ e sia V' un sottoinsieme di V. Sia T' il sottografo di T indotto da V' e sia G' il sottografo di G indotto da V'. Dimostrate che, se T' è connesso, allora T' è un albero di connessione minimo di G'.

23.1-10
Dato un grafo G e un albero di connessione minimo T, supponete di ridurre il peso di uno degli archi in T. Dimostrate che T è ancora un albero di connessione minimo per G. Più formalmente, sia T un albero di connessione minimo per G con i pesi degli archi dati dalla funzione peso w. Scegliete un arco $(x, y) \in T$ e un numero positivo k e definite la funzione peso w' in questo modo:

$$w'(u, v) = \begin{cases} w(u, v) & \text{se } (u, v) \neq (x, y) \\ w(x, y) - k & \text{se } (u, v) = (x, y) \end{cases}$$

Dimostrate che T è un albero di connessione minimo per G con i pesi degli archi dati da w'.

23.1-11 ⋆
Dato un grafo G e un albero di connessione minimo T, supponete di ridurre il peso di uno degli archi che non appartiene a T. Descrivete un algoritmo per trovare l'albero di connessione minimo nel grafo modificato.

23.2 Gli algoritmi di Kruskal e Prim

I due algoritmi per gli alberi di connessione minimi descritti in questo paragrafo sono elaborazioni del metodo generico. Ciascuno di essi usa una regola specifica per determinare un arco sicuro nella riga 3 di GENERIC-MST. Nell'algoritmo di Kruskal l'insieme A è una foresta i cui vertici sono tutti quelli del grafo. L'arco sicuro aggiunto ad A è sempre un arco di peso minimo nel grafo che collega due componenti distinte. Nell'algoritmo di Prim l'insieme A forma un albero singolo. L'arco sicuro aggiunto ad A è sempre un arco di peso minimo che collega l'albero con un vertice che non appartiene all'albero.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria 1995035610-2287070 © Copyright © 2022, McGraw-Hill Education (Italy)

Algoritmo di Kruskal

L'algoritmo di Kruskal trova un arco sicuro da aggiungere alla foresta in costruzione scegliendo, fra tutti gli archi che collegano due alberi qualsiasi nella foresta, un arco (u, v) di peso minimo. Indichiamo con C_1 e C_2 i due alberi che sono collegati da (u, v). Poiché (u, v) deve essere un arco leggero che collega C_1 a qualche

altro albero, il Corollario 23.2 implica che (u, v) è un arco sicuro per C_1. L'algoritmo di Kruskal è un algoritmo goloso, perché a ogni passo aggiunge alla foresta un arco con il minor peso possibile.

La nostra implementazione dell'algoritmo di Kruskal è simile all'algoritmo che calcola le componenti connesse che abbiamo descritto nel Paragrafo 21.1. Usa una struttura dati per insiemi disgiunti per mantenere vari insiemi disgiunti di elementi. Ogni insieme contiene i vertici di un albero della foresta corrente. L'operazione FIND-SET(u) restituisce un rappresentante dell'insieme che contiene u. Quindi, possiamo determinare se due vertici u e v appartengono allo stesso albero verificando se FIND-SET(u) è uguale a FIND-SET(v). L'unione degli alberi è effettuata dalla procedura UNION.

L'algoritmo di Kruskal opera come illustra la Figura 23.4. Le righe 1–3 inizializzano l'insieme A come un insieme vuoto e creano $|V|$ alberi, uno per ogni vertice. Il ciclo **for** nelle righe 5–8 esamina gli archi nell'ordine dal più leggero al più pesante. Il ciclo verifica, per ogni arco (u, v), se le estremità u e v appartengono allo stesso albero; in caso affermativo, l'arco (u, v) non può essere aggiunto alla foresta senza generare un ciclo, quindi l'arco viene scartato. Altrimenti i due vertici appartengono ad alberi differenti. In questo caso, l'arco (u, v) viene aggiunto ad A nella riga 7 e i vertici dei due alberi vengono fusi nella riga 8.

```
MST-KRUSKAL(G, w)
1  A = ∅
2  for ogni vertice v ∈ G.V
3       MAKE-SET(v)
4  ordina gli archi di G.E in senso non decrescente rispetto al peso w
5  for ogni arco (u, v) ∈ G.E, preso in ordine di peso non decrescente
6       if FIND-SET(u) ≠ FIND-SET(v)
7            A = A ∪ {(u, v)}
8            UNION(u, v)
9  return A
```

Il tempo di esecuzione dell'algoritmo di Kruskal per un grafo $G = (V, E)$ dipende dall'implementazione della struttura dati per gli insiemi disgiunti. Utilizzeremo l'implementazione della foresta di insiemi disgiunti descritta nel Paragrafo 21.3 con le euristiche dell'unione per rango e della compressione dei cammini, perché è l'implementazione asintoticamente più veloce fra quelle conosciute. L'inizializzazione dell'insieme A nella riga 1 richiede un tempo $O(1)$; il tempo per ordinare gli archi nella riga 4 è $O(E \lg E)$ (valuteremo tra poco il costo delle $|V|$ operazioni MAKE-SET del ciclo **for** nelle righe 2–3). Il ciclo **for** nelle righe 5–8 esegue $O(E)$ operazioni FIND-SET e UNION sulla foresta degli insiemi disgiunti. Tenendo conto anche delle $|V|$ operazioni MAKE-SET, si ottiene un tempo totale pari a $O((V + E)\,\alpha(V))$, dove α è la funzione (con crescita molto lenta) definita nel Paragrafo 21.4. Poiché supponiamo che G sia un grafo connesso, abbiamo $|E| \geq |V| - 1$, e quindi le operazioni con gli insiemi disgiunti richiedono un tempo $O(E\,\alpha(V))$. Inoltre, poiché $\alpha(|V|) = O(\lg V) = O(\lg E)$, il tempo di esecuzione totale dell'algoritmo di Kruskal è $O(E \lg E)$. Osservando che $|E| < |V|^2$, si ha $\lg |E| = O(\lg V)$; quindi possiamo ridefinire il tempo di esecuzione dell'algoritmo di Kruskal come $O(E \lg V)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995003016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

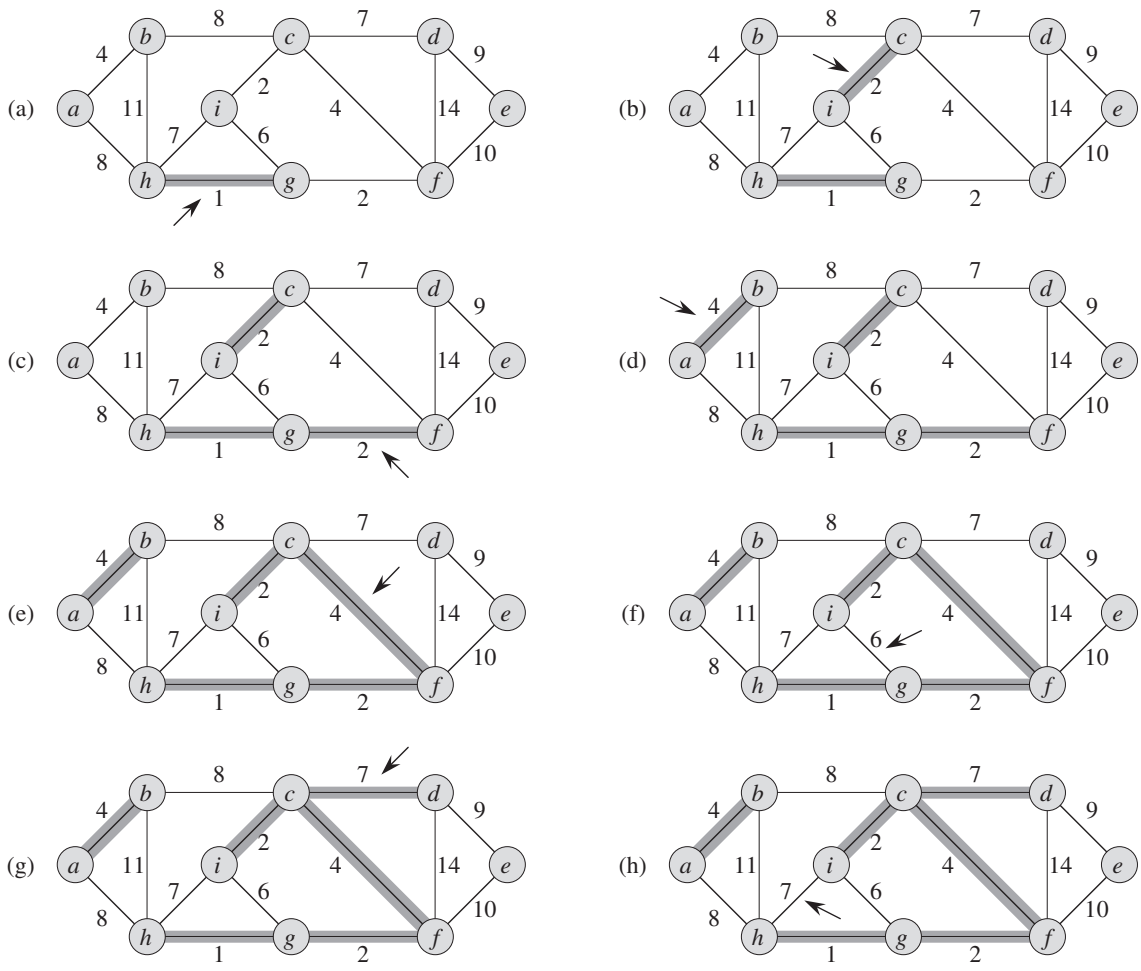
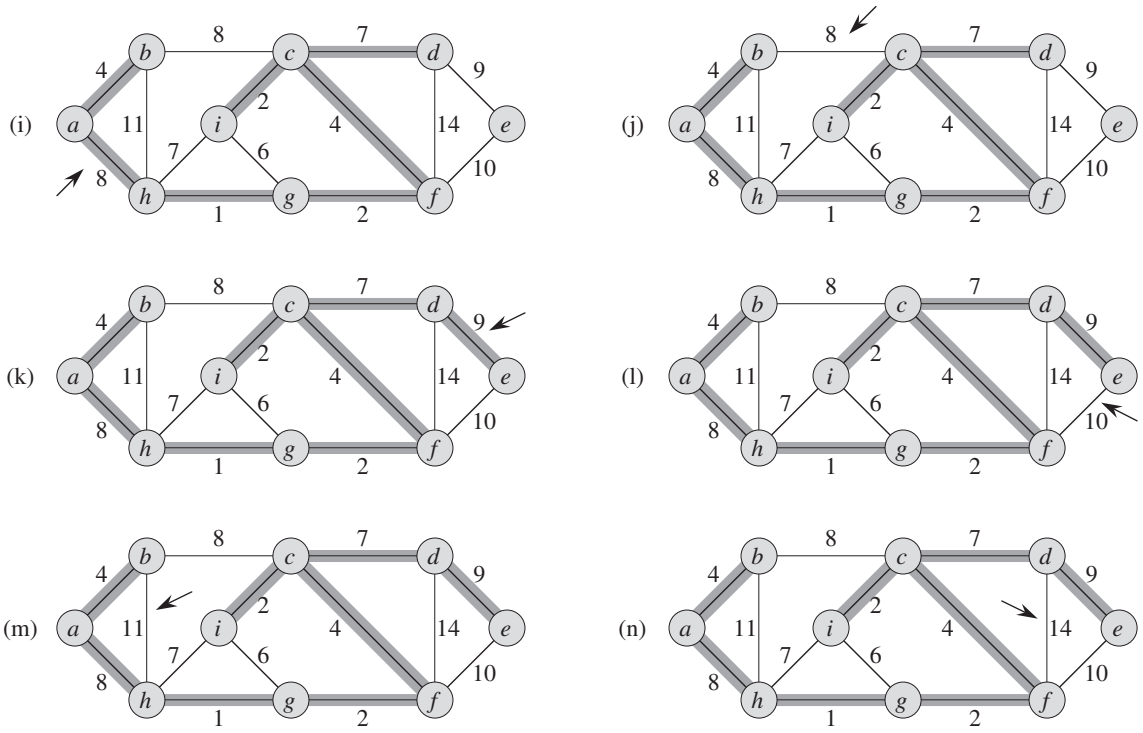


Figura 23.4 L'esecuzione dell'algoritmo di Kruskal con il grafo illustrato nella Figura 23.1. Gli archi su sfondo grigio appartengono alla foresta A che è in costruzione. Gli archi sono considerati dall'algoritmo in ordine di peso crescente. Una freccia indica l'arco che è correntemente considerato nel passo dell'algoritmo. Se l'arco unisce due alberi distinti nella foresta, esso viene aggiunto alla foresta, fondendo così i due alberi in un unico albero.

Algoritmo di Prim

Come l'algoritmo di Kruskal, anche l'algoritmo di Prim è un caso speciale del metodo generico per gli alberi di connessione minimi descritto nel Paragrafo 23.1. L'algoritmo di Prim opera in modo molto simile all'algoritmo di Dijkstra per trovare i cammini minimi in un grafo, come vedremo nel Paragrafo 24.4. L'algoritmo di Prim ha la proprietà che gli archi nell'insieme A formano sempre un albero singolo. Come illustra la Figura 23.5, l'albero inizia da un arbitrario vertice radice r e si sviluppa fino a coprire tutti i vertici in V. A ogni passo viene aggiunto all'albero A un arco leggero che collega A con un vertice isolato – un vertice che non sia estremo di qualche arco in A. Per il Corollario 23.2, questa regola aggiunge soltanto gli archi che sono sicuri per A; cosicché, quando l'algoritmo termina, gli archi in A formano un albero di connessione minimo. Questa strategia è golosa perché l'albero cresce includendo a ogni passo un arco che contribuisce con la quantità più piccola possibile a formare il peso dell'albero.

Acquistato da Michele Michele su Webster il 2022-07-27 23:13:11 Numero Ordine Interno: 1205-2018-2-207207 Copyright © 2022 McGraw-Hill Education (Italy)



La chiave per implementare con efficienza l'algoritmo di Prim consiste nel trovare un modo veloce per scegliere un nuovo arco da aggiungere all'albero formato dagli archi in A. Nel seguente pseudocodice, il grafo connesso G e la radice r dell'albero di connessione minimo da costruire sono gli input per l'algoritmo. Durante l'esecuzione dell'algoritmo, tutti i vertici che *non* si trovano nell'albero risiedono in una coda di min-priorità Q basata su un campo *key*. Per ogni vertice v, l'attributo $v.key$ è il peso minimo di un arco qualsiasi che collega v a un vertice nell'albero; per convenzione, $v.key = \infty$ se tale arco non esiste. L'attributo $v.\pi$ indica il padre di v nell'albero. Durante l'esecuzione dell'algoritmo, l'insieme A di GENERIC-MST è mantenuto implicitamente come

$$A = \{(v, v.\pi) : v \in V - \{r\} - Q\}$$

Quando l'algoritmo termina, la coda di min-priorità Q è vuota; l'albero di connessione minimo A per G è quindi

$$A = \{(v, v.\pi) : v \in V - \{r\}\}$$

```
MST-PRIM(G, w, r)
 1  for ogni u ∈ G.V
 2      u.key = ∞
 3      u.π = NIL
 4  r.key = 0
 5  Q = G.V
 6  while Q ≠ ∅
 7      u = EXTRACT-MIN(Q)
 8      for ogni v ∈ G.Adj[u]
 9          if v ∈ Q and w(u, v) < v.key
10              v.π = u
11              v.key = w(u, v)
```

Acquistato da Michele Michele su Kyobo il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

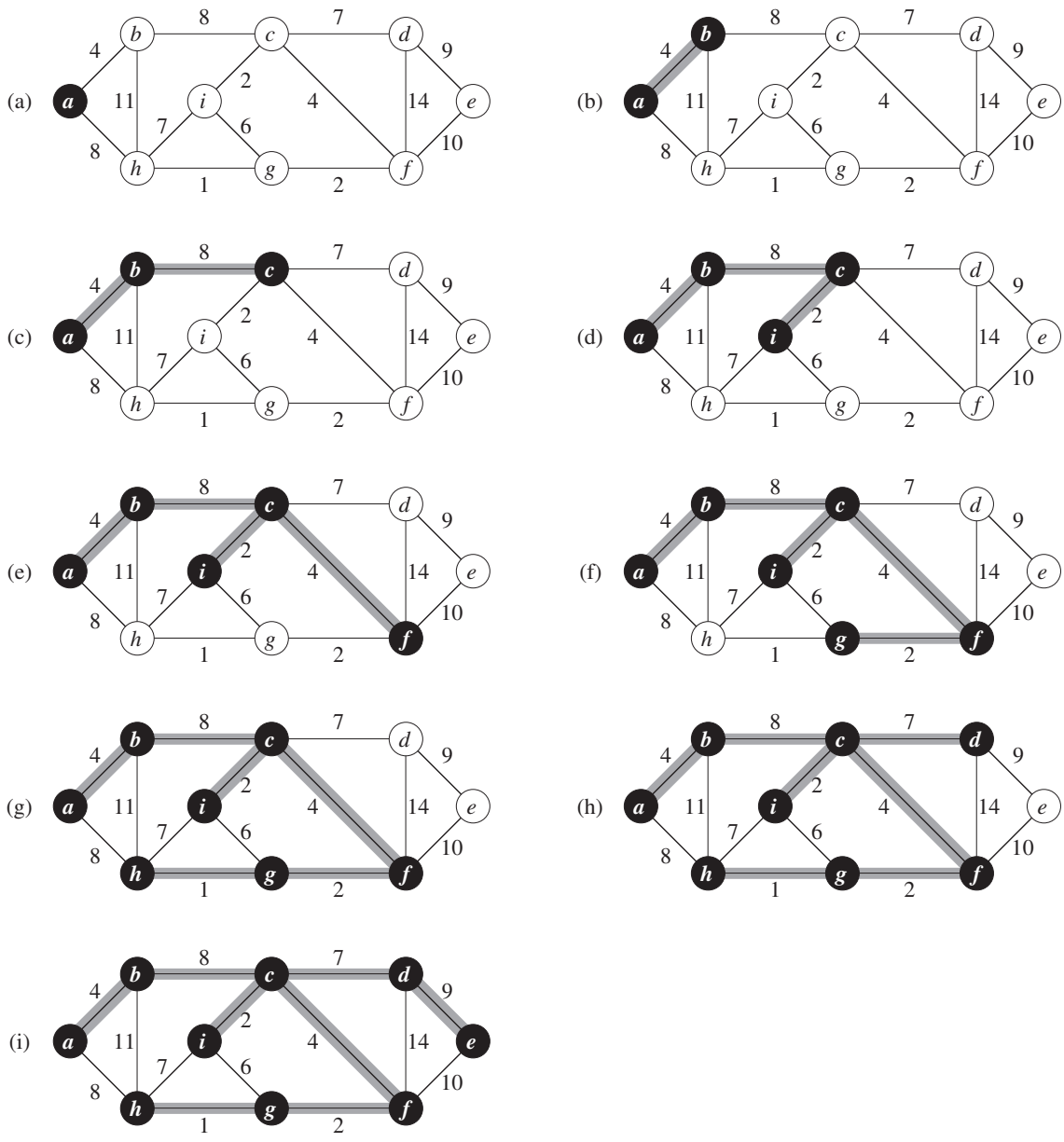


Figura 23.5 L'esecuzione dell'algoritmo di Prim con il grafo illustrato nella Figura 23.1. Il vertice radice è a. Gli archi con sfondo grigio appartengono all'albero in costruzione; i vertici dell'albero sono di colore nero. A ogni passo dell'algoritmo, i vertici dell'albero determinano un taglio del grafo e un arco leggero che attraversa il taglio viene aggiunto all'albero. Nel secondo passo, per esempio, l'algoritmo può scegliere di aggiungere all'albero l'arco (b, c) o l'arco (a, h), in quanto entrambi sono archi leggeri che attraversano il taglio.

Acquistato da Michele Michele su Webster il 2022-07-07 07:07:12 Numero Ordine Libreria 1005501015 2207070 © Copyright © 2022, McGraw-Hill Education (Italy)

L'algoritmo di Prim opera come illustra la Figura 23.5. Le righe 1–5 impostano la chiave di ciascun vertice a ∞ (ad eccezione della radice r, la cui chiave è impostata a 0; in questo modo la radice sarà il primo vertice ad essere elaborato), assegnano al padre di ciascun vertice il valore NIL e inizializzano la coda di min-priorità Q in modo che contenga tutti i vertici. L'algoritmo conserva la seguente invariante di ciclo composta da tre parti:

Prima di ogni iterazione del ciclo **while** (righe 6–11):

1. $A = \{(v, v.\pi) : v \in V - \{r\} - Q\}$.
2. I vertici già inseriti nell'albero di connessione minimo sono quelli che appartengono a $V - Q$.
3. Per ogni vertice $v \in Q$, se $v.\pi \neq \text{NIL}$, allora $v.key < \infty$ e $v.key$ è il peso di un arco leggero $(v, v.\pi)$ che collega v a qualche vertice che si trova già nell'albero di connessione minimo.

La riga 7 identifica un vertice $u \in Q$ incidente su un arco leggero che attraversa il taglio $(V - Q, Q)$ (ad eccezione della prima iterazione, nella quale $u = r$ per la riga 4). Quando il vertice u viene eliminato dall'insieme Q, viene aggiunto all'insieme $V - Q$ dei vertici dell'albero, quindi $(u, u.\pi)$ viene aggiunto ad A. Il ciclo **for** nelle righe 8–11 aggiorna i campi *key* e π di qualsiasi vertice v adiacente a u, ma che non appartiene all'albero. L'aggiornamento conserva la terza parte dell'invariante di ciclo.

Le prestazioni dell'algoritmo di Prim dipendono dal modo in cui viene implementata la coda di min-priorità Q. Se Q è implementata come un min-heap binario (vedere il Capitolo 6), possiamo utilizzare la procedura BUILD-MIN-HEAP per eseguire nel tempo $O(V)$ l'inizializzazione nelle righe 1–5. Il corpo del ciclo **while** viene eseguito $|V|$ volte e, poiché ogni operazione EXTRACT-MIN richiede un tempo $O(\lg V)$, il tempo totale per tutte le chiamate di EXTRACT-MIN è $O(V \lg V)$. Il ciclo **for** (righe 8–11) viene eseguito $O(E)$ volte complessivamente, in quanto la somma delle lunghezze di tutte le liste di adiacenza è $2|E|$. All'interno del ciclo **for**, il test che verifica l'appartenenza a Q nella riga 9 può essere implementato in tempo costante mantenendo un bit per ciascun vertice che indica se un vertice appartiene o no a Q, e aggiornando il bit quando il vertice viene eliminato da Q. L'assegnazione nella riga 11 richiede implicitamente un'operazione DECREASE-KEY sul min-heap, che può essere implementata con un min-heap binario nel tempo $O(\lg V)$. Quindi, il tempo totale dell'algoritmo di Prim è $O(V \lg V + E \lg V) = O(E \lg V)$, che è asintoticamente uguale a quello della nostra implementazione dell'algoritmo di Kruskal.

Il tempo di esecuzione asintotico dell'algoritmo di Prim può essere migliorato utilizzando gli heap di Fibonacci. Nel Capitolo 19 abbiamo visto che, se $|V|$ elementi sono organizzati in un heap di Fibonacci, è possibile eseguire un'operazione EXTRACT-MIN nel tempo ammortizzato $O(\lg V)$ e un'operazione DECREASE-KEY (per implementare la riga 11) nel tempo ammortizzato $O(1)$. Quindi, se utilizziamo un heap di Fibonacci per implementare la coda di min-priorità Q, il tempo di esecuzione dell'algoritmo di Prim migliora diventando $O(E + V \lg V)$.

Esercizi

23.2-1

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

L'algoritmo di Kruskal può restituire alberi di connessione differenti per lo stesso grafo di input G, a seconda di come vengono gestiti gli archi di peso uguale durante l'ordinamento degli archi. Dimostrate che per ogni albero di connessione minimo T di G, c'è un modo di ordinare gli archi di G nell'algoritmo di Kruskal che permette all'algoritmo di restituire T.

23.2-2
Supponiamo che il grafo $G = (V, E)$ sia rappresentato come una matrice di adiacenza. Sviluppate per questo caso una semplice implementazione dell'algoritmo di Prim con tempo $O(V^2)$.

23.2-3
L'implementazione dell'algoritmo di Prim con un heap di Fibonacci è asintoticamente più veloce dell'implementazione con un heap binario per un grafo sparso $G = (V, E)$, dove $|E| = \Theta(V)$? Che cosa accade nel caso di grafo denso, dove $|E| = \Theta(V^2)$? Che relazione deve esserci fra $|E|$ e $|V|$ affinché l'implementazione con un heap di Fibonacci sia asintoticamente più veloce dell'implementazione con un heap binario?

23.2-4
Supponete che tutti i pesi degli archi in un grafo siano numeri interi nell'intervallo da 1 a $|V|$. Qual è la velocità di esecuzione dell'algoritmo di Kruskal? Che cosa accade se i pesi degli archi sono numeri interi nell'intervallo da 1 a W, per qualche costante W?

23.2-5
Supponete che tutti i pesi degli archi in un grafo siano numeri interi nell'intervallo da 1 a $|V|$. Qual è la velocità di esecuzione dell'algoritmo di Prim? Che cosa accade se i pesi degli archi sono numeri interi nell'intervallo da 1 a W, per qualche costante W?

23.2-6 ⋆
Supponete che i pesi degli archi in un grafo siano uniformemente distribuiti nell'intervallo semiaperto $[0, 1)$. Quale algoritmo può essere eseguito più velocemente, quello di Kruskal o quello di Prim?

23.2-7 ⋆
Supponete di avere già calcolato un albero di connessione minimo per un grafo G. Quanto velocemente può essere aggiornato l'albero di connessione minimo se vengono aggiunti a G un nuovo vertice e nuovi archi incidenti?

23.2-8
Il professor Borden propone un nuovo algoritmo divide et impera per calcolare gli alberi di connessione minimi, che opera nel modo seguente. Dato un grafo $G = (V, E)$, l'algoritmo partiziona l'insieme V dei vertici in due insiemi V_1 e V_2 tali che $|V_1|$ e $|V_2|$ differiscono al massimo di 1. Sia E_1 l'insieme degli archi che sono incidenti soltanto sui vertici in V_1 e sia E_2 l'insieme degli archi che sono incidenti soltanto sui vertici in V_2. Mostrare che V_1 e V_2 possono essere scelti in modo che i due sottografi (V_1, E_1) e (V_2, E_2) siano entrambi connessi. L'algoritmo risolve ricorsivamente il problema di trovare un albero di connessione minimo in ciascuno dei due sottografi $G_1 = (V_1, E_1)$ e $G_2 = (V_2, E_2)$. Infine, seleziona l'arco di peso minimo in E che attraversa il taglio (V_1, V_2) e usa questo arco per unire i due alberi di connessione minimi risultanti in un unico albero di connessione.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Dimostrate che l'algoritmo calcola correttamente un albero di connessione minimo di G oppure trovate un esempio in cui l'algoritmo fallisce.

Problemi

23-1 Secondo migliore albero di connessione minimo

Sia $G = (V, E)$ un grafo connesso non orientato con la funzione peso $w : E \to \mathbb{R}$. Supponete che $|E| \geq |V|$ e che tutti i pesi degli archi siano distinti.

Un secondo migliore albero di connessione minimo è definito nel modo seguente. Sia $\mathcal{T}$ l'insieme di tutti gli alberi di connessione di G e sia T' un albero di connessione minimo di G. Allora un ***secondo migliore albero di connessione minimo*** è un albero di connessione T tale che $w(T) = \min_{T'' \in \mathcal{T} - \{T'\}} \{w(T'')\}$.

a. Dimostrate che l'albero di connessione minimo è unico, ma che il secondo migliore albero di connessione minimo non è necessariamente unico.

b. Sia T un albero di connessione minimo di G. Dimostrate che esistono degli archi $(u, v) \in T$ e $(x, y) \notin T$ tali che $T - \{(u, v)\} \cup \{(x, y)\}$ sia un secondo migliore albero di connessione minimo di G.

c. Sia T un albero di connessione di G e, per due vertici qualsiasi $u, v \in V$, sia $max[u, v]$ un arco di peso massimo nell'unico cammino semplice fra u e v in T. Descrivete un algoritmo con tempo $O(V^2)$ che, dato T, calcola $max[u, v]$ per ogni $u, v \in V$.

d. Descrivete un algoritmo efficiente per calcolare il secondo migliore albero di connessione minimo di G.

23-2 Albero di connessione minimo nei grafi sparsi

Per un grafo connesso $G = (V, E)$ molto sparso, è possibile migliorare ulteriormente il tempo di esecuzione $O(E + V \lg V)$ dell'algoritmo di Prim con gli heap di Fibonacci "pre-elaborando" G per ridurre il numero di vertici prima di eseguire l'algoritmo.

In particolare, scegliamo, per ogni vertice u, l'arco di peso minimo (u, v) incidente su u e poniamo l'arco (u, v) nell'albero di connessione minimo in costruzione; poi contraiamo tutti gli archi scelti (vedere il Paragrafo B.4). Anziché contrarre questi archi uno alla volta, identifichiamo prima gli insiemi dei vertici che sono uniti nello stesso nuovo vertice. Poi creiamo il grafo che avremmo ottenuto se avessimo contratto questi archi uno alla volta, ma lo facciamo "rinominando" gli archi in base agli insiemi in cui si trovavano le loro estremità. Molti archi del grafo originale potrebbero essere rinominati con lo stesso nome. In tal caso, si ottiene un solo arco, e il suo peso è il minimo fra i pesi dei corrispondenti archi originali.

Inizialmente, impostiamo come un albero vuoto l'albero di connessione minimo T che vogliamo costruire; per ogni arco $(u, v) \in E$, poniamo $(u, v).orig = (u, v)$ e $(u, v).c = w(u, v)$. Utilizziamo l'attributo *orig* per fare riferimento all'arco del grafo iniziale che è associato a un arco nel grafo contratto. L'attributo c memorizza il peso di un arco e, mentre gli archi vengono contratti, esso viene aggiornato secondo il precedente schema per scegliere i pesi degli archi. La procedura MST-REDUCE riceve in input G, *orig*, c, e T e restituisce un grafo contratto G' e gli attributi aggiornati $orig'$ e c' per il grafo G'. La procedura accumula anche gli archi di G nell'albero di connessione minimo T.

Acquistato da Michele Mi [illegible] 2025 [illegible] Proprietà letteraria [illegible] © 2023 McGraw-Hill Education (Italy)

```
MST-REDUCE(G, T)
 1  for ogni v ∈ G.V
 2      v.mark = FALSE
 3      MAKE-SET(v)
 4  for ogni u ∈ G.V
 5      if u.mark == FALSE
 6          sceglie v ∈ G.Adj[u] in modo che (u, v).c sia minimizzato
 7          UNION(u, v)
 8          T = T ∪ {(u, v).orig}
 9          u.mark = v.mark = TRUE
10  G'.V = {FIND-SET(v) : v ∈ G.V}
11  G'.E = ∅
12  for ogni (x, y) ∈ G.E
13      u = FIND-SET(x)
14      v = FIND-SET(y)
15      if (u, v) ∉ G'.E
16          G'.E = G'.E ∪ {(u, v)}
17          (u, v).orig' = (x, y).orig
18          (u, v).c' = (x, y).c
19      else if (x, y).c < (u, v).c'
20              (u, v).orig' = (x, y).orig
21              (u, v).c' = (x, y).c
22  costruisce le liste di adiacenza G'.Adj per G'
23  return G' e T
```

a. Sia T l'insieme degli archi restituito dalla procedura MST-REDUCE e sia A l'albero di connessione minimo del grafo G' formato dalla chiamata MST-PRIM(G', c', r), dove c' è il peso degli archi di $G'.E$ ed r è un vertice qualsiasi in $G'.V$. Dimostrate che $T \cup \{(x, y).orig' : (x, y) \in A\}$ è un albero di connessione minimo di G.

b. Dimostrate che $|G'.V| \leq |V|/2$.

c. Spiegate come implementare la procedura MST-REDUCE in modo che venga eseguita nel tempo $O(E)$ (*suggerimento:* usate strutture dati semplici).

d. Supponete di eseguire k fasi di MST-REDUCE, utilizzando gli output G', $orig'$ e c' prodotti da una fase come input G, $orig$ e c della successiva fase e accumulando gli archi in T. Dimostrate che il tempo di esecuzione complessivo delle k fasi è $O(kE)$.

e. Dopo avere eseguito k fasi di MST-REDUCE, come descritto nel punto (d), supponete di eseguire l'algoritmo di Prim chiamando MST-PRIM(G', c', r), dove G' e il peso c' sono restituiti dall'ultima fase ed r è un vertice qualsiasi in $G'.V$. Spiegate come scegliere k in modo che il tempo di esecuzione totale sia $O(E \lg \lg V)$. Dimostrate che la vostra scelta di k minimizza il tempo di esecuzione asintotico totale.

Acquistato da Michele Michele su Webster il 2022-07-07 23:27. Numero Ordine Libreria 18050716-2247707-0 Copyright © 2022 McGraw-Hill Education (Italy)

f. Per quali valori di $|E|$ (in funzione di $|V|$) l'algoritmo di Prim con pre-elaborazione è asintoticamente più efficiente dell'algoritmo di Prim senza pre-elaborazione?

23-3 Albero di connessione collo di bottiglia

Un ***albero di connessione collo di bottiglia*** T di un grafo non orientato G è un albero di connessione di G il cui peso d'arco massimo è il minimo fra tutti gli alberi di connessione di G. Il peso dell'arco di peso massimo in T è detto valore dell'albero di connessione collo di bottiglia.

a. Dimostrate che un albero di connessione minimo è un albero di connessione collo di bottiglia.

Dal punto (a) si deduce che trovare un albero di connessione collo di bottiglia non è più difficile di trovare un albero di connessione minimo. Nei punti che seguono vedremo che un albero di connessione collo di bottiglia può essere trovato in tempo lineare.

b. Descrivete un algoritmo con tempo lineare che, dato un grafo G e un intero b, determina se il valore dell'albero di connessione collo di bottiglia è al massimo b.

c. Utilizzate l'algoritmo del punto (b) come una subroutine in un algoritmo con tempo lineare per il problema dell'albero di connessione collo di bottiglia (*suggerimento:* potreste utilizzare una subroutine che contrae gli insiemi degli archi, come nella procedura MST-REDUCE descritta nel Problema 23-2).

23-4 Algoritmi alternativi per l'albero di connessione minimo

In questo problema forniamo lo pseudocodice per tre diversi algoritmi, ciascuno dei quali riceve un grafo come input e restituisce un insieme di archi T. Per ogni algoritmo, dimostrate che T è un albero di connessione minimo oppure provate che non lo è. Inoltre descrivete l'implementazione più efficiente per ciascun algoritmo, indipendentemente dal fatto che l'algoritmo calcoli oppure no un albero di connessione minimo.

a. MAYBE-MST-A(G, w)

```
1  ordina gli archi in senso non crescente rispetto ai loro pesi w
2  T = E
3  for ogni arco e preso in ordine di peso non crescente
4      if T − {e} è un grafo connesso
5          T = T − {e}
6  return T
```

b. MAYBE-MST-B(G, w)

```
1  T = ∅
2  for ogni arco e preso in ordine arbitrario
3      if T ∪ {e} non ha cicli
4          T = T ∪ {e}
5  return T
```

c. MAYBE-MST-C(G, w)

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

```
1  T = ∅
2  for ogni arco e preso in ordine arbitrario
3      T = T ∪ {e}
4      if T ha un ciclo c
5          sia e′ un arco di peso massimo in c
6          T = T − {e′}
7  return T
```

Note

Tarjan [331] ha svolto un'indagine generale sul problema dell'albero di connessione minimo, fornendo del materiale d'avanguardia eccellente. Graham e Hell [152] hanno scritto la storia del problema dell'albero di connessione minimo. Tarjan attribuisce il primo algoritmo per gli alberi di connessione minimi a un articolo del 1926 di O. Borůvka. L'algoritmo di Borůvka consiste nell'eseguire $O(\lg V)$ iterazioni della procedura MST-REDUCE descritta nel Problema 23-2. L'algoritmo di Kruskal fu descritto da Kruskal [223] nel 1956. L'algoritmo, comunemente noto come algoritmo di Prim, è stato veramente inventato da Prim [286], ma era stato anche ideato precedentemente da V. Jarník nel 1930.

La ragione per la quale gli algoritmi golosi sono efficienti nel trovare gli alberi di connessione minimi è che l'insieme delle foreste di un grafo forma un matroide grafico (vedere il Paragrafo 16.4).

Quando $|E| = \Omega(V \lg V)$, l'algoritmo di Prim, implementato con gli heap di Fibonacci viene eseguito nel tempo $O(E)$. Per grafi più sparsi, combinando i concetti di base dell'algoritmo di Prim, dell'algoritmo di Kruskal e dell'algoritmo di Borůvka con alcune strutture dati avanzate, Fredman e Tarjan [115] hanno creato un algoritmo che viene eseguito nel tempo $O(E \lg^* V)$. Gabow, Galil, Spencer e Tarjan [121] hanno migliorato questo algoritmo portando il tempo di esecuzione a $O(E \lg \lg^* V)$. Chazelle [61] ha creato un algoritmo che viene eseguito nel tempo $O(E\, \hat{\alpha}(E, V))$, dove $\hat{\alpha}(E, V)$ è l'inversa della funzione di Ackermann (consultate le note in fondo al Capitolo 21 per una breve descrizione della funzione di Ackermann e dalla sua inversa). Diversamente dai precedenti algoritmi per l'albero di connessione minimo, l'algoritmo di Chazelle non applica il metodo goloso.

Un problema analogo è la ***verifica dell'albero di connessione*** in cui, dati un grafo $G = (V, E)$ e un albero $T \subseteq E$, occorre determinare se T è un albero di connessione minimo di G. King [204] ha sviluppato un algoritmo con tempo lineare per la verifica dell'albero di connessione, basandosi su un precedente studio di Komlós [216] e Dixon, Rauch e Tarjan [91].

I precedenti algoritmi sono tutti deterministici e appartengono al modello basato sui confronti che abbiamo descritto nel Capitolo 8. Karger, Klein e Tarjan [196] hanno sviluppato un algoritmo randomizzato per l'albero di connessione minimo che viene eseguito nel tempo atteso $O(V + E)$. Questo algoritmo usa la ricorsione in modo simile all'algoritmo per la selezione in tempo lineare descritto nel Paragrafo 9.3: una chiamata ricorsiva in un problema ausiliario identifica un sottoinsieme degli archi E' che non possono trovarsi in nessun albero di connessione minimo. Successivamente, un'altra chiamata ricorsiva su $E - E'$ trova l'albero di connessione minimo. L'algoritmo applica anche i concetti dell'algoritmo di Borůvka e dell'algoritmo di King per la verifica dell'albero di connessione.

Fredman e Willard [117] hanno dimostrato come trovare un albero di connessione minimo nel tempo $O(V + E)$ utilizzando un algoritmo deterministico che non è basato sui confronti. Il loro algoritmo suppone che i dati siano numeri interi di b bit e che la memoria del calcolatore sia formata da parole indirizzabili di b bit.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Cammini minimi da sorgente unica 24

Il professor Patrick vuole trovare la strada più corta possibile da Phoenix a Indianapolis. Avendo a disposizione una carta stradale degli Stati Uniti sulla quale sono indicate le distanze fra le coppie di incroci stradali adiacenti, come potrà il professore determinare tale strada? Una soluzione possibile consiste nell'enumerare tutte le strade da Phoenix a Indianapolis, sommare le distanze di ciascuna strada e selezionare la più corta. È facile intuire che, anche se escludiamo le strade che contengono dei cicli, ci sono milioni di possibilità, la maggior parte delle quali non vale la pena considerare. Per esempio, il percorso da Phoenix a Indianapolis che passa per Seattle è ovviamente da scartare, perché Seattle è quasi mille miglia fuori mano.

24.1 Definizioni e proprietà dei cammini mimini

In questo capitolo e nel Capitolo 25 spiegheremo come risolvere con efficienza questo tipo di problemi. In un ***problema dei cammini minimi*** è dato un grafo orientato pesato $G = (V, E)$, con una funzione peso $w : E \to \mathbb{R}$ che associa agli archi dei pesi di valore reale. Il ***peso*** $w(p)$ del cammino $p = \langle v_0, v_1, \ldots, v_k \rangle$ è la somma dei pesi degli archi che lo compongono:

$$w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$$

Il ***peso di un cammino minimo*** $\delta(u, v)$ da u a v è definito in questo modo:

$$\delta(u, v) = \begin{cases} \min\{w(p) : u \overset{p}{\rightsquigarrow} v\} & \text{se esiste un cammino da } u \text{ a } v \\ \infty & \text{negli altri casi} \end{cases}$$

Un ***cammino minimo*** dal vertice u al vertice v è definito come un cammino qualsiasi p con peso $w(p) = \delta(u, v)$.

Nell'esempio della strada da Phoenix a Indianapolis, possiamo modellare la carta stradale come un grafo: i vertici rappresentano gli incroci stradali, gli archi rappresentano i tratti di strada fra gli incroci e i pesi degli archi rappresentano le distanze stradali. Lo scopo è trovare un cammino minimo da un dato incrocio di Phoenix a un dato incrocio di Indianapolis.

Acquistato da Michele Mastrogiovanni (mastrogiovanni.michele@gmail.com) - Numero licenza libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

I pesi degli archi possono essere interpretati come grandezze diverse dalle distanze. Spesso sono utilizzati per rappresentare tempi, costi, penalità, perdite o altre quantità che si accumulano linearmente lungo un cammino e che occorre minimizzare.

L'algoritmo di visita in ampiezza descritto nel Paragrafo 22.2 è un algoritmo per cammini minimi che opera con i grafi non pesati, cioè grafi in cui ogni arco può

essere considerato di peso unitario. Poiché molti dei concetti relativi alla visita in ampiezza servono nell'analisi dei cammini minimi nei grafi pesati, vi consigliamo di rivedere il Paragrafo 22.2 prima di continuare.

Varianti

In questo capitolo approfondiremo l'analisi del ***problema dei cammini minimi da sorgente unica***: dato un grafo $G = (V, E)$, vogliamo trovare un cammino minimo che va da un dato vertice ***sorgente*** $s \in V$ a ciascun vertice $v \in V$. Molti altri problemi possono essere risolti con l'algoritmo per i cammini minimi da sorgente unica, incluse le seguenti varianti.

Problema dei cammini minimi con destinazione unica: trovare un cammino minimo da ciascun vertice v a un dato vertice ***destinazione*** t. Invertendo la direzione di ciascun arco nel grafo, possiamo ricondurre questo problema al problema dei cammini minimi da sorgente unica.

Problema del cammino minimo per una coppia di vertici: trovare un cammino minimo da u a v, noti i vertici u e v. Se risolviamo il problema della sorgente unica con il vertice sorgente u, abbiamo risolto anche questo problema. Inoltre, tutti gli algoritmi noti per questo problema hanno lo stesso tempo di esecuzione asintotico nel caso peggiore dei migliori algoritmi per sorgente unica.

Problema dei cammini minimi fra tutte le coppie di vertici: trovare un cammino minimo da u a v per ogni coppia di vertici u e v. Sebbene questo problema possa essere risolto eseguendo un algoritmo per sorgente unica una volta per ogni vertice, tuttavia esiste un metodo più veloce per risolverlo; inoltre la sua struttura è di per sé interessante. Il Capitolo 25 tratta dettagliatamente il problema dei cammini minimi fra tutte le coppie di vertici.

Sottostruttura ottima di un cammino minimo

Tipicamente, gli algoritmi per i cammini minimi si basano sulla proprietà che un cammino minimo fra due vertici contiene altri cammini minimi al suo interno (anche l'algoritmo di Edmonds-Karp per i problemi di flusso massimo descritto nel Capitolo 26 si basa su questa proprietà). La proprietà della sottostruttura ottima è uno degli indicatori principali per l'applicabilità della programmazione dinamica (Capitolo 15) e del metodo goloso (Capitolo 16). L'algoritmo di Dijkstra, che descriveremo nel Paragrafo 24.4, è un algoritmo goloso e l'algoritmo di Floyd-Warshall, che trova i cammini minimi fra tutte le coppie di vertici (Capitolo 25), è un algoritmo di programmazione dinamica. Il seguente lemma definisce in modo più preciso la proprietà della sottostruttura ottima dei cammini minimi.

Lemma 24.1 (I sottocammini di cammini minimi sono cammini minimi)
Dato un grafo orientato pesato $G = (V, E)$ con la funzione peso $w : E \to \mathbb{R}$, sia $p = \langle v_0, v_1, \ldots, v_k \rangle$ un cammino minimo dal vertice v_0 al vertice v_k e, per qualsiasi i e j tali che $0 \le i \le j \le k$, sia $p_{ij} = \langle v_i, v_{i+1}, \ldots, v_j \rangle$ il sottocammino di p dal vertice v_i al vertice v_j. Allora p_{ij} è un cammino minimo da v_i a v_j.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 - Numero Ordine Libreria 10050819 230707-0 Copyright © 2022 McGraw-Hill Education (Italy)

Dimostrazione Se scomponiamo il cammino p in $v_0 \overset{p_{0i}}{\rightsquigarrow} v_i \overset{p_{ij}}{\rightsquigarrow} v_j \overset{p_{jk}}{\rightsquigarrow} v_k$, abbiamo $w(p) = w(p_{0i}) + w(p_{ij}) + w(p_{jk})$. Supponiamo adesso che ci sia un

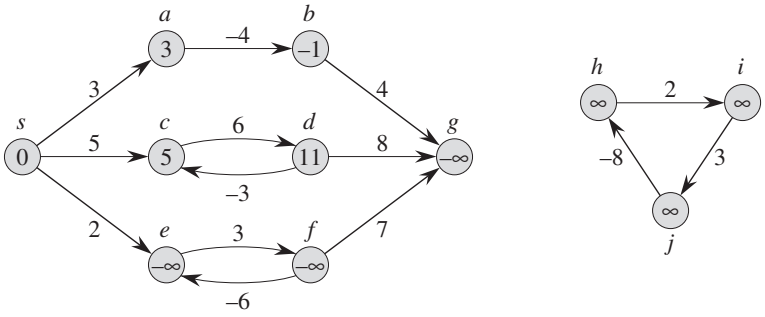


Figura 24.1 Archi con pesi negativi in un grafo orientato. All'interno di ciascun vertice è indicato il peso del suo cammino minimo dalla sorgente s. Poiché i vertici e ed f formano un ciclo di peso negativo che è raggiungibile da s, i pesi dei loro cammini minimi sono $-\infty$. Poiché il vertice g è raggiungibile da un vertice il cui peso di cammino minimo è $-\infty$, anche il peso del suo cammino minimo è $-\infty$. Vertici come h, i e j non possono essere raggiunti da s, pertanto i pesi dei loro cammini minimi sono ∞, anche se giacciono su un ciclo di peso negativo.

cammino p'_{ij} da v_i a v_j con peso $w(p'_{ij}) < w(p_{ij})$. Allora $v_0 \overset{p_{0i}}{\rightsquigarrow} v_i \overset{p'_{ij}}{\rightsquigarrow} v_j \overset{p_{jk}}{\rightsquigarrow} v_k$ è un cammino da v_0 a v_k il cui peso $w(p_{0i}) + w(p'_{ij}) + w(p_{jk})$ è minore di $w(p)$, che contraddice l'ipotesi che p sia un cammino minimo da v_0 a v_k. ■

Archi di peso negativo

In alcuni casi del problema dei cammini minimi da sorgente unica possono presentarsi degli archi i cui pesi sono negativi. Se il grafo $G = (V, E)$ non contiene cicli di peso negativo che sono raggiungibili dalla sorgente s, allora per ogni $v \in V$, il peso del cammino minimo $\delta(s, v)$ resta ben definito, anche se ha un valore negativo. Tuttavia, se esiste un ciclo di peso negativo che è raggiungibile da s, i pesi dei cammini minimi non sono ben definiti. Nessun cammino da s a un vertice del ciclo può essere un cammino minimo – è sempre possibile trovare un cammino di peso minore che segue il cammino "minimo" proposto e poi attraversa il ciclo di peso negativo. Se esiste un ciclo di peso negativo in qualche cammino da s a v, definiamo $\delta(s, v) = -\infty$.

La Figura 24.1 illustra l'effetto dei pesi negativi e dei cicli di peso negativo sui pesi dei cammini minimi. Poiché c'è un solo cammino da s ad a (il cammino $\langle s, a\rangle$), $\delta(s, a) = w(s, a) = 3$. Analogamente, c'è un solo cammino da s a b, e quindi $\delta(s, b) = w(s, a) + w(a, b) = 3 + (-4) = -1$. Esiste un numero molto grande di cammini da s a c: $\langle s, c\rangle$, $\langle s, c, d, c\rangle$, $\langle s, c, d, c, d, c\rangle$ e così via. Poiché il ciclo $\langle c, d, c\rangle$ ha peso $6 + (-3) = 3 > 0$, il cammino minimo da s a c è $\langle s, c\rangle$, con peso $\delta(s, c) = 5$. Il cammino minimo da s a d è $\langle s, c, d\rangle$, con peso $\delta(s, d) = w(s, c) + w(c, d) = 11$. Analogamente, esiste un numero molto grande di cammini da s a e: $\langle s, e\rangle$, $\langle s, e, f, e\rangle$, $\langle s, e, f, e, f, e\rangle$ e così via. Tuttavia, poiché il ciclo $\langle e, f, e\rangle$ ha peso $3 + (-6) = -3 < 0$, non esiste un cammino minimo da s a e. Attraversando il ciclo di peso negativo $\langle e, f, e\rangle$ un numero arbitrario di volte, è possibile trovare dei cammini da s a e con pesi negativi arbitrariamente grandi, quindi $\delta(s, e) = -\infty$. Analogamente, $\delta(s, f) = -\infty$. Poiché g è raggiungibile da f, è anche possibile trovare cammini con pesi negativi arbitrariamente grandi da s a g, quindi $\delta(s, g) = -\infty$. Anche i vertici h, i e j formano un ciclo di peso negativo. Questi vertici però non sono raggiungibili da s, quindi $\delta(s, h) = \delta(s, i) = \delta(s, j) = \infty$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Alcuni algoritmi per cammini minimi, come l'algoritmo di Dijkstra, suppongono che tutti i pesi degli archi nel grafo di input siano non negativi, come nell'esempio della carta stradale. Altri, come l'algoritmo di Bellman-Ford, accettano archi di peso negativo nel grafo di input e forniscono una soluzione corretta se non ci sono cicli di peso negativo che possono essere raggiunti dalla sorgente. Tipica-

mente, se esiste un ciclo di peso negativo di questo tipo, l'algoritmo è in grado di rilevarne e segnalarne l'esistenza.

I cicli

Un cammino minimo può contenere un ciclo? Come abbiamo appena visto, un cammino minimo non può contenere un ciclo di peso negativo. Non può contenere neanche un ciclo di peso positivo, perché eliminando il ciclo dal cammino si ottiene un cammino con la stessa sorgente, la stessa destinazione e un peso più piccolo. Ovvero, se $p = \langle v_0, v_1, \ldots, v_k \rangle$ è un cammino e $c = \langle v_i, v_{i+1}, \ldots, v_j \rangle$ è un ciclo di peso positivo in questo cammino (cosicché $v_i = v_j$ e $w(c) > 0$), allora il cammino $p' = \langle v_0, v_1, \ldots, v_i, v_{j+1}, v_{j+2}, \ldots, v_k \rangle$ ha peso $w(p') = w(p) - w(c) < w(p)$, e quindi p non può essere un cammino minimo da v_0 a v_k.

Restano così soltanto i cicli di peso 0. Possiamo eliminare un ciclo di peso 0 da un cammino qualsiasi per ottenere un altro cammino dello stesso peso. Quindi, se c'è un cammino minimo da una sorgente s a una destinazione v che contiene un ciclo di peso 0, allora esiste un altro cammino minimo da s a v senza questo ciclo. Finché un cammino minimo ha cicli di peso 0, è possibile eliminare ripetutamente questi cicli dal cammino fino a ottenere un cammino minimo privo di cicli. Pertanto, senza perdere in generalità, possiamo supporre che nelle nostre ricerche i cammini minimi non abbiano cicli, ossia siano cammini semplici. Poiché un cammino aciclico qualsiasi in un grafo $G = (V, E)$ contiene al più $|V|$ vertici distinti, esso contiene al più $|V| - 1$ archi. Quindi, possiamo limitare la nostra analisi ai cammini minimi che hanno al più $|V| - 1$ archi.

Rappresentazione dei cammini minimi

Spesso è necessario calcolare non soltanto i pesi dei cammini minimi, ma anche i vertici in questi cammini. La rappresentazione che utilizziamo per i cammini minimi è simile a quella utilizzata per gli alberi delle visite in ampiezza (Paragrafo 22.2). Dato un grafo $G = (V, E)$, manteniamo per ogni vertice $v \in V$ un ***predecessore*** $v.\pi$ che può essere un altro vertice o il valore NIL. Gli algoritmi per i cammini minimi di questo capitolo impostano gli attributi π in modo che la catena dei predecessori che inizia da un vertice v percorra all'indietro un cammino minimo da s a v. Quindi, dato un vertice v per il quale $v.\pi \neq \text{NIL}$, la procedura PRINT-PATH(G, s, v) del Paragrafo 22.2 può essere utilizzata per stampare un cammino minimo da s a v.

Durante l'esecuzione di un algoritmo per cammini minimi, tuttavia, i valori π non devono necessariamente indicare i cammini minimi. Come nella visita in ampiezza, saremo interessati al ***sottografo dei predecessori*** $G_\pi = (V_\pi, E_\pi)$ indotto dai valori π. Anche qui, definiamo l'insieme dei vertici V_π come l'insieme dei vertici di G con predecessori non NIL, più la sorgente s:

$$V_\pi = \{v \in V : v.\pi \neq \text{NIL}\} \cup \{s\}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

L'insieme degli archi orientati E_π è l'insieme degli archi indotto dai valori π per i vertici in V_π:

$$E_\pi = \{(v.\pi, v) \in E : v \in V_\pi - \{s\}\}$$

Dimostreremo che i valori π prodotti dagli algoritmi di questo capitolo hanno la proprietà che al termine della loro esecuzione G_π è un "albero di cammini

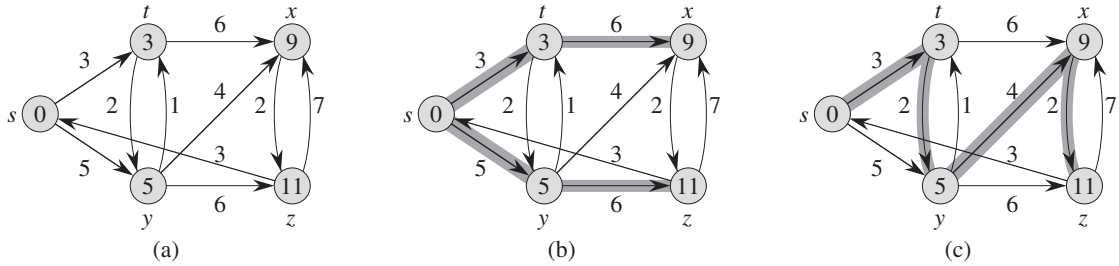


Figura 24.2 **(a)** Un grafo orientato pesato con i pesi dei cammini minimi dalla sorgente s. **(b)** Gli archi ombreggiati formano un albero di cammini minimi con radice nella sorgente s. **(c)** Un altro albero di cammini minimi con la stessa radice.

minimi" – informalmente, un albero radicato che contiene un cammino minimo dalla sorgente s a ogni vertice che è raggiungibile da s. Un albero di cammini minimi è simile all'albero di visita in ampiezza descritto nel Paragrafo 22.2, con la differenza che contiene i cammini dalla sorgente che sono definiti minimi in base ai pesi degli archi, anziché al numero degli archi. Per essere più precisi, sia $G = (V, E)$ un grafo orientato pesato con la funzione peso $w : E \to \mathbb{R}$; supponiamo che G non contenga cicli di peso negativo raggiungibili dalla sorgente $s \in V$, in modo che i cammini minimi siano ben definiti. Un ***albero di cammini minimi*** radicato in s è un sottografo orientato $G' = (V', E')$, dove $V' \subseteq V$ e $E' \subseteq E$, tale che

1. V' è l'insieme dei vertici raggiungibili da s in G
2. G' forma un albero con radice s
3. per ogni $v \in V'$, l'unico cammino semplice da s a v in G' è un cammino minimo da s a v in G.

I cammini minimi non sono necessariamente unici né lo sono gli alberi dei cammini minimi. Per esempio, la Figura 24.2 illustra un grafo orientato pesato e due alberi di cammini minimi con la stessa radice.

Rilassamento

Gli algoritmi descritti in questo capitolo usano la tecnica del ***rilassamento***. Per ogni vertice $v \in V$, manteniamo un attributo $v.d$, che è un limite superiore per il peso di un cammino minimo dalla sorgente s a v. L'attributo $v.d$ è detto ***stima del cammino minimo***. La seguente procedura con tempo $\Theta(V)$ inizializza le stime dei cammini minimi e i predecessori.

```
INITIALIZE-SINGLE-SOURCE(G, s)
1  for ogni vertice v ∈ G.V
2      v.d = ∞
3      v.π = NIL
4  s.d = 0
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Dopo l'inizializzazione, $v.\pi = \text{NIL}$ per ogni $v \in V$, $s.d = 0$ e $v.d = \infty$ per $v \in V - \{s\}$.

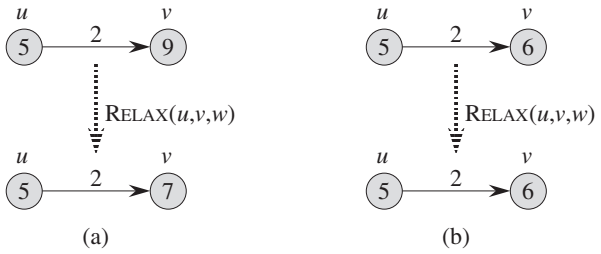


Figura 24.3
Rilassamento di un arco (u, v) con peso $w(u, v) = 2$. La stima del cammino minimo di ciascun vertice è illustrata all'interno del vertice. **(a)** Poiché $v.d > u.d + w(u, v)$ prima del rilassamento, il valore di $v.d$ diminuisce. **(b)** Qui, $v.d \leq u.d + w(u, v)$ prima del rilassamento, quindi il rilassamento non cambia $v.d$.

Il processo di ***rilassamento*** di un arco (u, v) consiste nel verificare se, passando per u, è possibile migliorare il cammino minimo per v precedentemente trovato e, in caso affermativo, nell'aggiornare $v.d$ e $v.\pi$. Un passo di rilassamento[1] può ridurre il valore della stima del cammino minimo $v.d$ e aggiornare il campo $v.\pi$ del predecessore di v. Il seguente codice effettua un passo di rilassamento sull'arco (u, v) in tempo $O(1)$.

```
RELAX(u, v, w)
1  if v.d > u.d + w(u, v)
2        v.d = u.d + w(u, v)
3        v.π = u
```

La Figura 24.3 illustra due esempi di rilassamento di un arco, uno in cui la stima di un cammino minimo diminuisce e uno in cui la stima non cambia.

Ogni algoritmo di questo capitolo chiama INITIALIZE-SINGLE-SOURCE e poi rilassa ripetutamente gli archi. Inoltre, il rilassamento è l'unico modo per modificare i predecessori e le stime dei cammini minimi. Gli algoritmi di questo capitolo differiscono per il numero di volte in cui rilassano ciascun arco e per l'ordine in cui rilassano gli archi. Nell'algoritmo di Dijkstra e nell'algoritmo per i cammini minimi dei grafi orientati aciclici, ogni arco viene rilassato una sola volta. Nell'algoritmo di Bellman-Ford ogni arco viene rilassato $|V| - 1$ volte.

Proprietà dei cammini minimi e del rilassamento

Per provare la correttezza degli algoritmi descritti in questo capitolo, faremo ricorso a diverse proprietà dei cammini minimi e del rilassamento. Definiamo qui di seguito queste proprietà; il Paragrafo 24.6 le proverà formalmente. Per agevolare la lettura, ogni proprietà qui definita include il numero del relativo lemma o corollario del Paragrafo 24.6. Le ultime cinque di queste di proprietà, che fanno riferimento alle stime dei cammini minimi o al sottografo dei predecessori, suppongono implicitamente che il grafo sia inizializzato con una chiamata della procedura INITIALIZE-SINGLE-SOURCE(G, s) e che l'unico modo in cui le stime dei cammini minimi e il sottografo dei predecessori possono cambiare è tramite una sequenza di passi di rilassamento.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[1] Può sembrare strano che il termine "rilassamento" sia utilizzato per un'operazione che riduce un limite superiore. L'uso di questo termine appartiene alla tradizione. L'effetto di un passo di rilassamento può essere visto come un rilassamento del vincolo $v.d \leq u.d + w(u, v)$ che, per la disuguaglianza triangolare (Lemma 24.10), deve essere soddisfatto se $u.d = \delta(s, u)$ e $v.d = \delta(s, v)$. Ovvero, se $v.d \leq u.d + w(u, v)$, non occorre esercitare alcuna "pressione" perché questo vincolo sia rispettato, quindi il vincolo è "rilassato".

Disuguaglianza triangolare (Lemma 24.10)
Per qualsiasi arco $(u, v) \in E$, si ha $\delta(s, v) \leq \delta(s, u) + w(u, v)$.

Proprietà del limite superiore (Lemma 24.11)
Per tutti i vertici $v \in V$, si ha sempre $v.d \geq \delta(s, v)$ e, una volta che il limite superiore $v.d$ assume il valore $\delta(s, v)$, esso non cambia più.

Proprietà dell'assenza di un cammino (Corollario 24.12)
Se non c'è un cammino da s a v, allora si ha sempre $v.d = \delta(s, v) = \infty$.

Proprietà della convergenza (Lemma 24.14)
Se $s \rightsquigarrow u \rightarrow v$ è un cammino minimo in G per qualche $u, v \in V$ e se $u.d = \delta(s, u)$ in un istante qualsiasi prima del rilassamento dell'arco (u, v), allora $v.d = \delta(s, v)$ in tutti gli istanti successivi.

Proprietà del rilassamento del cammino (Lemma 24.15)
Se $p = \langle v_0, v_1, \ldots, v_k \rangle$ è un cammino minimo da $s = v_0$ a v_k e gli archi di p vengono rilassati nell'ordine $(v_0, v_1), (v_1, v_2), \ldots, (v_{k-1}, v_k)$, allora $v_k.d = \delta(s, v_k)$. Questa proprietà è soddisfatta indipendentemente da altri passi di rilassamento che vengono effettuati, anche se sono interposti fra i rilassamenti degli archi di p.

Proprietà del sottografo dei predecessori (Lemma 24.17)
Una volta che $v.d = \delta(s, v)$ per ogni $v \in V$, il sottografo dei predecessori è un albero di cammini minimi radicato in s.

Organizzazione del capitolo

Il Paragrafo 24.2 presenta l'algoritmo di Bellman-Ford, che risolve il problema dei cammini minimi da sorgente unica nel caso generale in cui gli archi possono avere peso negativo. L'algoritmo di Bellman-Ford è notevolmente semplice; inoltre ha l'ulteriore vantaggio di rilevare se un ciclo di peso negativo è raggiungibile dalla sorgente. Il Paragrafo 24.3 presenta un algoritmo con tempo lineare per calcolare i cammini minimi da una sorgente unica in un grafo orientato aciclico. Il Paragrafo 24.4 tratta l'algoritmo di Dijkstra, che ha un tempo di esecuzione minore rispetto all'algoritmo di Bellman-Ford, ma richiede che i pesi degli archi non siano negativi. Il Paragrafo 24.5 spiega come utilizzare l'algoritmo di Bellman-Ford per risolvere un caso speciale di "programmazione lineare". Infine, il Paragrafo 24.6 dimostra le proprietà dei cammini minimi e del rilassamento precedentemente definite.

La nostra analisi richiede alcune convenzioni per eseguire operazioni aritmetiche con grandezze infinite. Assumeremo che, per qualsiasi numero reale $a \neq -\infty$, si abbia $a + \infty = \infty + a = \infty$. Inoltre, affinché le nostre dimostrazioni siano valide in presenza di cicli di peso negativo, assumeremo che, per qualsiasi numero reale $a \neq \infty$, si abbia $a + (-\infty) = (-\infty) + a = -\infty$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Per tutti gli algoritmi presentati in questo capitolo si suppone che il grafo orientato G sia memorizzato secondo la rappresentazione con liste di adiacenza. Inoltre, assieme a ciascun arco è memorizzato il suo peso, in modo che attraversando ciascuna lista di adiacenza, sia possibile determinare i pesi degli archi in un tempo $O(1)$ per ogni arco.

24.2 L'algoritmo di Bellman-Ford

L'***algoritmo di Bellman-Ford*** risolve il problema dei cammini minimi da sorgente unica nel caso generale in cui i pesi degli archi possono essere negativi. Dato un grafo orientato pesato $G = (V, E)$ con sorgente s e funzione peso $w : E \to \mathbb{R}$, l'algoritmo di Bellman-Ford restituisce un valore booleano che indica se esiste oppure no un ciclo di peso negativo che è raggiungibile dalla sorgente. Se un tale ciclo esiste, l'algoritmo indica che il problema non ha soluzione. Se un tale ciclo non esiste, l'algoritmo fornisce i cammini minimi e i loro pesi.

L'algoritmo usa il rilassamento, riducendo progressivamente il valore stimato $v.d$ per il peso di un cammino minimo dalla sorgente s a ciascun vertice $v \in V$, fino a raggiungere il peso effettivo $\delta(s, v)$ di un cammino minimo. L'algoritmo restituisce TRUE se e soltanto se il grafo non contiene cicli di peso negativo che sono raggiungibili dalla sorgente.

```
BELLMAN-FORD(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  for i = 1 to |G.V| − 1
3       for ogni arco (u, v) ∈ G.E
4            RELAX(u, v, w)
5  for ogni arco (u, v) ∈ G.E
6       if v.d > u.d + w(u, v)
7            return FALSE
8  return TRUE
```

La Figura 24.4 illustra l'esecuzione dell'algoritmo di Bellman-Ford con un grafo di 5 vertici. Dopo l'inizializzazione dei valori d e π di tutti i vertici nella riga 1, l'algoritmo effettua $|V| - 1$ passaggi sugli archi del grafo. Ogni passaggio è un'iterazione del ciclo **for** delle righe 2–4 e consiste nell'effettuare un rilassamento per ciascun arco del grafo. Le Figure 24.4(b)–(e) illustrano lo stato dell'algoritmo dopo ciascuno dei quattro passaggi sugli archi. Dopo avere effettuato $|V| - 1$ passaggi, le righe 5–8 controllano se esiste un ciclo di peso negativo e restituiscono il valore booleano appropriato (più avanti chiariremo perché funziona questo controllo).

L'algoritmo di Bellman-Ford viene eseguito nel tempo $O(VE)$, perché l'inizializzazione nella riga 1 richiede un tempo $\Theta(V)$, ciascuno dei $|V| - 1$ passaggi sugli archi nelle righe 2–4 richiede un tempo $\Theta(E)$ e il ciclo **for** nelle righe 5–7 richiede un tempo $O(E)$.

Per provare la correttezza dell'algoritmo di Bellman-Ford, iniziamo dimostrando che, se non ci sono cicli di peso negativo, l'algoritmo calcola correttamente i pesi dei cammini minimi per tutti i vertici raggiungibili dalla sorgente.

Acquistato da Michele Michele su Webster il 2022-07-22 12:24:42 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Lemma 24.2

Sia $G = (V, E)$ un grafo orientato pesato con sorgente s e funzione peso $w : E \to \mathbb{R}$; supponiamo che G non contenga cicli di peso negativo che sono raggiungibili da s. Allora, dopo $|V| - 1$ iterazioni del ciclo **for** delle righe 2–4 di BELLMAN-FORD, si ha $v.d = \delta(s, v)$ per tutti i vertici v che sono raggiungibili da s.

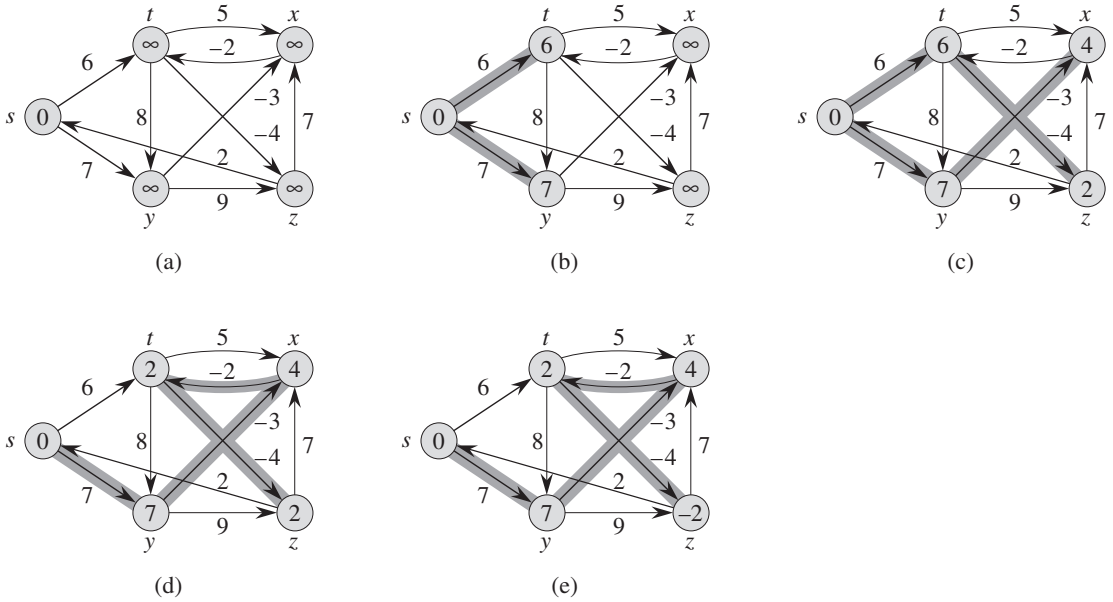


Figura 24.4 L'esecuzione dell'algoritmo di Bellman-Ford. La sorgente è il vertice s. I valori d sono annotati all'interno dei vertici; gli archi ombreggiati indicano i valori dei predecessori: se l'arco (u, v) è ombreggiato, allora $v.\pi = u$. In questo esempio specifico, ogni passaggio rilassa gli archi nell'ordine $(t, x), (t, y), (t, z), (x, t), (y, x), (y, z), (z, x), (z, s), (s, t), (s, y)$. **(a)** La situazione appena prima del primo passaggio sugli archi. **(b)–(e)** La situazione dopo successivi passaggi sugli archi. I valori d e π nella parte (e) sono i valori finali. L'algoritmo di Bellman-Ford restituisce TRUE in questo esempio.

Dimostrazione Dimostriamo il lemma applicando la proprietà del rilassamento del cammino. Consideriamo un vertice qualsiasi v che è raggiungibile da s e indichiamo con $p = \langle v_0, v_1, \ldots, v_k \rangle$, dove $v_0 = s$ e $v_k = v$, un cammino minimo aciclico da s a v. Il cammino p ha al più $|V| - 1$ archi, quindi $k \leq |V| - 1$. Ciascuna delle $|V| - 1$ iterazioni del ciclo **for** delle righe 2–4 rilassa tutti gli $|E|$ archi. Fra gli archi rilassati nella i-esima iterazione, per $i = 1, 2, \ldots, k$, c'è l'arco (v_{i-1}, v_i). Per la proprietà del rilassamento del cammino, quindi $v.d = v_k.d = \delta(s, v_k) = \delta(s, v)$. ■

Corollario 24.3

Sia $G = (V, E)$ un grafo orientato pesato con sorgente s e funzione peso $w : E \to \mathbb{R}$. Allora, per ogni vertice $v \in V$, esiste un cammino da s a v se e soltanto se l'algoritmo BELLMAN-FORD termina con $v.d < \infty$ quando viene eseguito sul grafo G.

Dimostrazione Lasciamo al lettore il compito di dimostrare questo corollario (Esercizio 24.2-2). ■

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Teorema 24.4 (Correttezza dell'algoritmo di Bellman-Ford)

Supponiamo di eseguire l'algoritmo di BELLMAN-FORD su un grafo orientato pesato $G = (V, E)$ con sorgente s e funzione peso $w : E \to \mathbb{R}$. Se G non contiene cicli di peso negativo che sono raggiungibili da s, allora l'algoritmo restituisce TRUE, si ha $v.d = \delta(s, v)$ per tutti i vertici $v \in V$ e il sottografo dei predecessori G_π è un albero di cammini minimi radicato in s. Se G contiene un ciclo di peso negativo che è raggiungibile da s, allora l'algoritmo restituisce FALSE.

Dimostrazione Supponiamo che il grafo G non contenga cicli di peso negativo che sono raggiungibili dalla sorgente s. Dimostriamo prima l'asserzione che al termine dell'algoritmo, $v.d = \delta(s, v)$ per tutti i vertici $v \in V$. Se il vertice v è raggiungibile da s, allora il Lemma 24.2 dimostra questa asserzione. Se v non è raggiungibile da s, allora l'asserzione è vera per la proprietà dell'assenza di un cammino. Dunque, l'asserzione è dimostrata. La proprietà del sottografo dei predecessori, insieme a questa asserzione, implica che G_π è un albero di cammini minimi. Adesso utilizziamo l'asserzione per dimostrare che l'algoritmo di BELLMAN-FORD restituisce TRUE. Al termine dell'algoritmo, per tutti gli archi $(u, v) \in E$, si ha

$$\begin{aligned} v.d &= \delta(s, v) \\ &\le \delta(s, u) + w(u, v) \quad \text{(per la disuguaglianza triangolare)} \\ &= u.d + w(u, v) \end{aligned}$$

Quindi nessuno dei controlli nella riga 6 fa sì che l'algoritmo di BELLMAN-FORD restituisca FALSE; quindi l'algoritmo restituisce TRUE.

Supponiamo adesso che il grafo G contenga un ciclo di peso negativo che è raggiungibile dalla sorgente s; indichiamo questo ciclo con $c = \langle v_0, v_1, \ldots, v_k \rangle$, dove $v_0 = v_k$; allora

$$\sum_{i=1}^{k} w(v_{i-1}, v_i) < 0 \tag{24.1}$$

Supponiamo per assurdo che l'algoritmo di Bellman-Ford restituisca TRUE. Quindi, $v_i.d \le v_{i-1}.d + w(v_{i-1}, v_i)$ per $i = 1, 2, \ldots, k$. Sommando le disuguaglianze lungo il ciclo c si ottiene

$$\begin{aligned} \sum_{i=1}^{k} v_i.d &\le \sum_{i=1}^{k} (v_{i-1}.d + w(v_{i-1}, v_i)) \\ &= \sum_{i=1}^{k} v_{i-1}.d + \sum_{i=1}^{k} w(v_{i-1}, v_i) \end{aligned}$$

Poiché $v_0 = v_k$, ogni vertice di c appare una sola volta in ciascuna delle sommatorie $\sum_{i=1}^{k} v_i.d$ e $\sum_{i=1}^{k} v_{i-1}.d$, quindi

$$\sum_{i=1}^{k} v_i.d = \sum_{i=1}^{k} v_{i-1}.d$$

Inoltre, per il Corollario 24.3, $v_i.d$ è finito per $i = 1, 2, \ldots, k$; pertanto

$$0 \le \sum_{i=1}^{k} w(v_{i-1}, v_i)$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

che contraddice la disuguaglianza (24.1). Concludiamo che l'algoritmo di Bellman-Ford restituisce TRUE se il grafo G non contiene cicli di peso negativo che sono raggiungibili dalla sorgente, altrimenti restituisce FALSE. ■

Esercizi

24.2-1

Eseguite l'algoritmo di Bellman-Ford sul grafo orientato illustrato nella Figura 24.4, utilizzando il vertice z come sorgente. In ogni passaggio rilassate gli archi nello stesso ordine della figura e indicate i valori di d e π dopo ogni passaggio; poi assegnate all'arco (z, x) un peso 4 ed eseguite di nuovo l'algoritmo utilizzando s come sorgente.

24.2-2

Dimostrate il Corollario 24.3.

24.2-3

Dato un grafo orientato pesato $G = (V, E)$ senza cicli di peso negativo, indichiamo con m il massimo, per ogni vertice $v \in V$, del numero minimo di archi in un cammino minimo dalla sorgente s a v (qui il cammino minimo è riferito al peso, non al numero di archi). Suggerite una semplice modifica da apportare all'algoritmo di Bellman-Ford in modo che l'algoritmo possa terminare in $m + 1$ passaggi, anche se m non è noto a priori.

24.2-4

Modificate l'algoritmo di Bellman-Ford in modo che $v.d$ venga posto a $-\infty$ per tutti i vertici v per i quali esiste un ciclo di peso negativo in qualche cammino dalla sorgente a v.

24.2-5 ★

Sia $G = (V, E)$ un grafo orientato pesato con funzione peso $w : E \to \mathbb{R}$. Create un algoritmo con tempo $O(VE)$ che calcola, per ogni vertice $v \in V$, il valore $\delta^*(v) = \min_{u \in V} \{\delta(u, v)\}$.

24.2-6 ★

Supponete che un grafo orientato pesato $G = (V, E)$ abbia un ciclo di peso negativo. Create un algoritmo efficiente per elencare i vertici di un tale ciclo. Dimostrate che il vostro algoritmo è corretto.

24.3 Cammini minimi da sorgente unica nei grafi orientati aciclici

Rilassando gli archi di un dag (grafo orientato aciclico) pesato $G = (V, E)$ secondo un ordine topologico dei suoi vertici, è possibile calcolare i cammini minimi da una sorgente unica nel tempo $\Theta(V + E)$. I cammini minimi sono sempre ben definiti in un dag, perché anche se ci possono essere archi di peso negativo, non possono esistere cicli di peso negativo.

L'algoritmo inizia ordinando topologicamente il dag (vedere il Paragrafo 22.4) per imporre un ordinamento lineare ai vertici. Se esiste un cammino dal vertice u al vertice v, allora u precede v nell'ordine topologico. Effettuiamo un passaggio sui vertici secondo l'ordine topologico. Durante l'elaborazione di un vertice, vengono rilassati tutti gli archi che escono dal vertice.

Acquistato da Michele Michele su Webster il 2022-07-06 23:12 Numero Ordine Libreria: 1993585016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

```
DAG-SHORTEST-PATHS(G, w, s)
1  dispone in ordine topologico i vertici di G
2  INITIALIZE-SINGLE-SOURCE(G, s)
3  for ogni vertice u preso in ordine topologico
4      for ogni vertice v ∈ G.Adj[u]
5          RELAX(u, v, w)
```

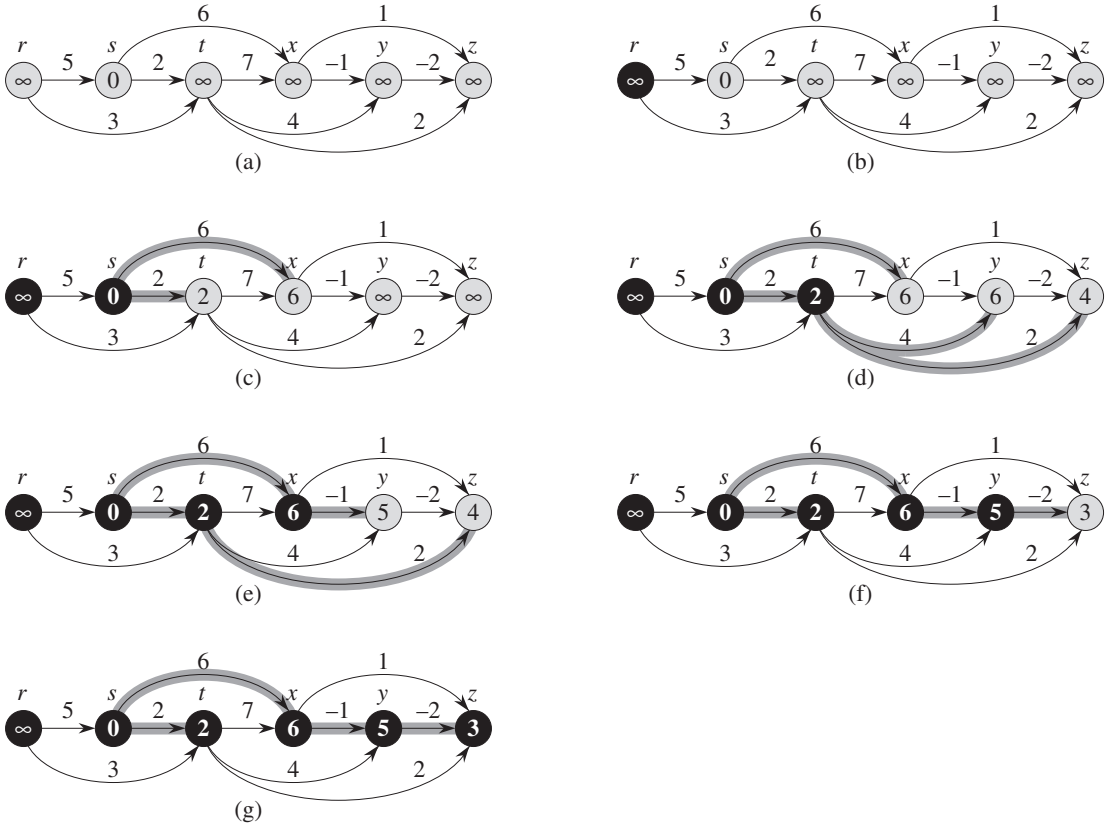


Figura 24.5 L'esecuzione dell'algoritmo per i cammini minimi in un grafo orientato aciclico. I vertici sono in ordine topologico da sinistra a destra. Il vertice sorgente è s. I valori d sono indicati all'interno dei vertici; gli archi ombreggiati indicano i valori π. **(a)** La situazione prima della prima iterazione del ciclo **for** delle righe 3–5. **(b)–(g)** Le situazioni dopo successive iterazioni del ciclo **for** delle righe 3–5. Il nuovo vertice che viene colorato di nero in una iterazione è quello che è stato utilizzato come vertice u in tale iterazione. I valori indicati nella parte (g) sono quelli finali.

La Figura 24.5 illustra l'esecuzione di questo algoritmo. Il tempo di esecuzione è facile da analizzare. Come abbiamo visto nel Paragrafo 22.4, l'ordinamento topologico della riga 1 può essere effettuato nel tempo $\Theta(V + E)$. La chiamata della procedura INITIALIZE-SINGLE-SOURCE nella riga 2 richiede un tempo $\Theta(V)$. Il ciclo **for** delle righe 3–5 effettua un'iterazione per ciascun vertice. Complessivamente, il ciclo **for** delle righe 4–5 rilassa ciascun arco una sola volta (qui abbiamo usato il metodo di aggregazione). Poiché ciascuna iterazione del ciclo **for** interno richiede un tempo $\Theta(1)$, il tempo di esecuzione totale è $\Theta(V + E)$, che è lineare nella dimensione di una rappresentazione con liste di adiacenza del grafo.

Il seguente teorema dimostra che la procedura DAG-SHORTEST-PATHS calcola correttamente i cammini minimi.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Teorema 24.5

Se un grafo orientato pesato $G = (V, E)$ ha sorgente s e nessun ciclo, allora al termine della procedura DAG-SHORTEST-PATHS, $v.d = \delta(s, v)$ per tutti i vertici $v \in V$ e il sottografo dei predecessori G_π è un albero di cammini minimi.

Dimostrazione Dimostriamo prima che $v.d = \delta(s, v)$ per tutti i vertici $v \in V$ al termine della procedura. Se v non è raggiungibile da s, allora $v.d = \delta(s, v) = \infty$

per la proprietà dell'assenza di un cammino. Se invece v è raggiungibile da s, allora esiste un cammino minimo $p = \langle v_0, v_1, \ldots, v_k \rangle$, dove $v_0 = s$ e $v_k = v$. Poiché i vertici vengono elaborati in ordine topologico, gli archi di p vengono rilassati nell'ordine $(v_0, v_1), (v_1, v_2), \ldots, (v_{k-1}, v_k)$. La proprietà del rilassamento del cammino implica che $v_i.d = \delta(s, v_i)$ al termine della procedura per $i = 0, 1, \ldots, k$. Infine, per la proprietà del sottografo dei predecessori, G_π è un albero di cammini minimi. ■

Un'applicazione interessante di questo algoritmo consiste nel determinare i cammini critici nell'analisi dei ***diagrammi PERT***.[2] Gli archi rappresentano i lavori da eseguire; i pesi degli archi rappresentano i tempi richiesti per eseguire specifici lavori. Se l'arco (u, v) entra nel vertice v e l'arco (v, x) esce da v, allora il lavoro (u, v) deve essere svolto prima del lavoro (v, x). Un cammino attraverso questo dag rappresenta una sequenza di lavori che devono essere eseguiti in un particolare ordine. Un ***cammino critico*** è un cammino *massimo* attraverso il dag, che corrisponde al tempo massimo per eseguire una sequenza ordinata di lavori. Il peso di un cammino critico è un limite inferiore per il tempo totale richiesto per eseguire tutti i lavori. È possibile trovare un cammino critico in uno dei seguenti modi:

- Cambiando il segno dei pesi degli archi ed eseguendo DAG-SHORTEST-PATHS.
- Eseguendo DAG-SHORTEST-PATHS, dopo avere sostituito "∞" con "$-\infty$" nella riga 2 di INITIALIZE-SINGLE-SOURCE e "$>$" con "$<$" nella procedura RELAX.

Esercizi

24.3-1
Eseguite DAG-SHORTEST-PATHS sul grafo orientato illustrato nella Figura 24.5, utilizzando il vertice r come sorgente.

24.3-2
Supponete di modificare la riga 3 di DAG-SHORTEST-PATHS in questo modo:
3 **for** i primi $|V| - 1$ vertici presi in ordine topologico
La procedura resta corretta?

24.3-3
La precedente formulazione dei diagrammi PERT non è molto naturale. Sarebbe più naturale se i vertici rappresentassero i lavori e gli archi rappresentassero i vincoli sulla successione dei lavori; ovvero l'arco (u, v) dovrebbe indicare che il lavoro u deve essere eseguito prima del lavoro v. In questo modo, i pesi sarebbero assegnati ai vertici, non agli archi. Modificate la procedura DAG-SHORTEST-PATHS in modo che trovi in tempo lineare un cammino massimo in un grafo orientato aciclico con i vertici pesati.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

24.3-4
Create un algoritmo efficiente per contare il numero totale di cammini in un grafo orientato aciclico. Analizzate il vostro algoritmo.

[2] "PERT" è l'acronimo di "Program Evaluation and Review Technique".

24.4 Algoritmo di Dijkstra

L'algoritmo di Dijkstra risolve il problema dei cammini minimi da sorgente unica in un grafo orientato pesato $G = (V, E)$ nel caso in cui tutti i pesi degli archi non siano negativi. In questo paragrafo, quindi, supponiamo che $w(u, v) \geq 0$ per ogni arco $(u, v) \in E$. Come vedremo più avanti, con una buona implementazione, il tempo di esecuzione dell'algoritmo di Dijkstra è minore di quello dell'algoritmo di Bellman-Ford.

L'algoritmo di Dijkstra mantiene un insieme S di vertici i cui pesi finali dei cammini minimi dalla sorgente s sono stati già determinati. L'algoritmo seleziona ripetutamente il vertice $u \in V - S$ con la stima minima del cammino minimo, aggiunge u a S e rilassa tutti gli archi che escono da u. Nella seguente implementazione, manteniamo una coda di min-priorità Q di vertici, utilizzando come chiavi i loro valori d.

```
DIJKSTRA(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  S = ∅
3  Q = G.V
4  while Q ≠ ∅
5      u = EXTRACT-MIN(Q)
6      S = S ∪ {u}
7      for ogni vertice v ∈ G.Adj[u]
8          RELAX(u, v, w)
```

L'algoritmo di Dijkstra rilassa gli archi come illustra la Figura 24.6. La riga 1 effettua la consueta inizializzazione dei valori d e π; la riga 2 inizializza l'insieme S come l'insieme vuoto. L'algoritmo mantiene l'invariante che $Q = V - S$ all'inizio di ogni iterazione del ciclo **while** delle righe 4–8. La riga 3 inizializza la coda di min-priorità Q in modo che questa contenga tutti i vertici in V; poiché $S = \emptyset$ in quel momento, l'invariante è vera dopo la riga 3. Ogni volta che viene eseguito il ciclo **while** delle righe 4–8, un vertice u viene estratto da $Q = V - S$ e aggiunto all'insieme S, conservando così l'invariante (la prima volta che viene eseguito il ciclo, $u = s$). Il vertice u, quindi, ha la stima minima del cammino minimo di tutti i vertici in $V - S$. Le righe 7–8 rilassano ogni arco (u, v) che esce da u e aggiornano la stima $v.d$ e il predecessore $v.\pi$, se il cammino minimo che arriva a v può essere migliorato passando per u. Notate che i vertici non vengono mai inseriti in Q dopo la riga 3 e che ogni vertice viene estratto da Q e aggiunto a S una sola volta, quindi il ciclo **while** delle righe 4–8 viene ripetuto esattamente $|V|$ volte.

Poiché l'algoritmo di Dijkstra sceglie sempre il vertice "più leggero" o "più vicino" in $V - S$ da aggiungere all'insieme S, diciamo che applica una strategia golosa. Le strategie golose sono descritte dettagliatamente nel Capitolo 16, ma non è necessario leggere quel capitolo per capire l'algoritmo di Dijkstra. In generale, le strategie golose non forniscono sempre i risultati ottimali, ma come dimostrano il seguente teorema e il suo corollario, l'algoritmo di Dijkstra calcola davvero i cammini minimi. Il punto chiave consiste nel dimostrare che ogni volta che un vertice u viene aggiunto all'insieme S, si ha $u.d = \delta(s, u)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

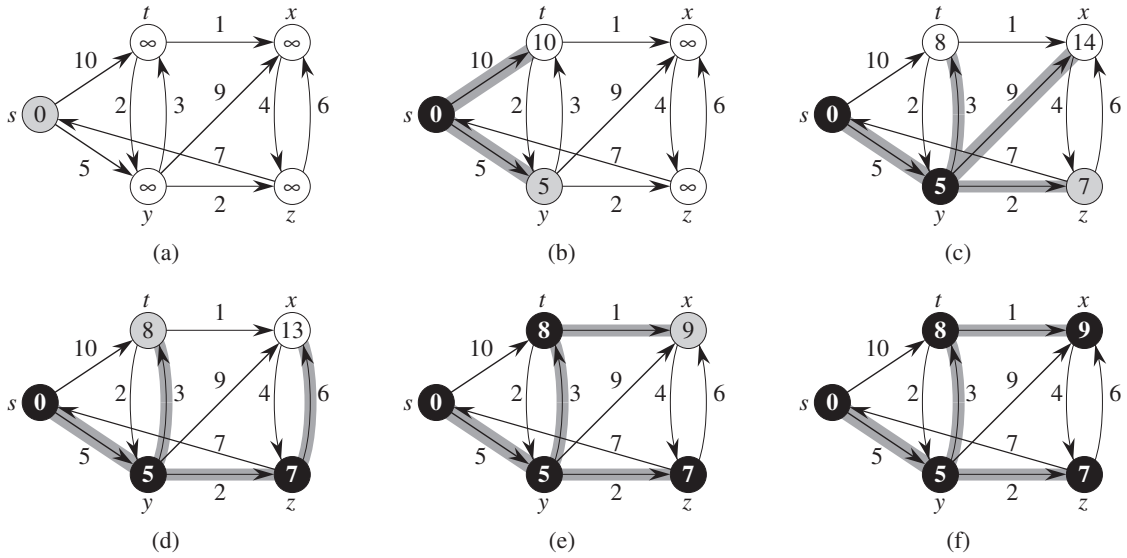


Figura 24.6 L'esecuzione dell'algoritmo di Dijkstra. La sorgente s è il vertice più a sinistra. Le stime dei cammini minimi sono illustrate all'interno dei vertici; gli archi ombreggiati indicano i valori dei predecessori. I vertici neri si trovano nell'insieme S e i vertici bianchi si trovano nella coda di min-priorità $Q = V - S$. **(a)** La situazione appena prima della prima iterazione del ciclo **while** delle righe 4–8. Il vertice ombreggiato ha il valore minimo d ed è scelto come vertice u nella riga 5. **(b)–(f)** Le situazioni dopo successive iterazioni del ciclo **while**. Il vertice ombreggiato in una parte viene scelto nella riga 5 come vertice u per la successiva iterazione. I valori d e π illustrati nella parte (f) sono i valori finali.

Teorema 24.6 (Correttezza dell'algoritmo di Dijkstra)
L'algoritmo di Dijkstra, eseguito su un grafo orientato pesato $G = (V, E)$ con funzione peso non negativa w e sorgente s, termina con $u.d = \delta(s, u)$ per tutti i vertici $u \in V$.

Dimostrazione Utilizziamo la seguente invariante di ciclo:

> All'inizio di ogni iterazione del ciclo **while** delle righe 4–8, $v.d = \delta(s, v)$ per ogni vertice $v \in S$.

È sufficiente dimostrare che, per ogni vertice $u \in V$, $u.d = \delta(s, u)$ nell'istante in cui u viene aggiunto all'insieme S. Una volta dimostrato che $u.d = \delta(s, u)$, applichiamo la proprietà del limite superiore per dimostrare che l'uguaglianza resta valida in tutti gli istanti successivi.

Inizializzazione: Inizialmente, $S = \emptyset$, quindi l'invariante è banalmente vera.

Conservazione: Vogliamo dimostrare che in ogni iterazione, $u.d = \delta(s, u)$ per il vertice aggiunto all'insieme S. Supponiamo per assurdo che u sia il primo vertice per il quale $u.d \neq \delta(s, u)$ quando esso viene aggiunto all'insieme S. Focalizzeremo l'attenzione sulla situazione all'inizio dell'iterazione del ciclo **while** in cui u viene aggiunto a S e otterremo la contraddizione che $u.d = \delta(s, u)$ in quell'istante, esaminando un cammino minimo da s a u. Deve essere $u \neq s$, perché s è il primo vertice aggiunto all'insieme S e $s.d = \delta(s, s) = 0$ in quell'istante. Poiché $u \neq s$, è anche vero che $S \neq \emptyset$ appena prima che u venga aggiunto a S. Deve esistere qualche cammino da s a u, perché altrimenti

Acquistato da Michele Michele su Webster 1,2022-07-07 23:12 Numero Ordine Libreria: 1995030162220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

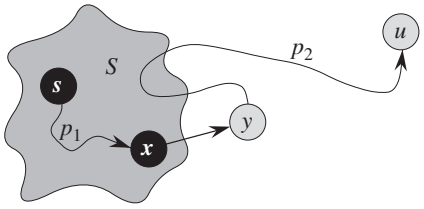


Figura 24.7 La dimostrazione del Teorema 24.6. L'insieme S non è vuoto appena prima che il vertice u sia aggiunto ad esso. Un cammino minimo p dalla sorgente s al vertice u può essere scomposto in $s \overset{p_1}{\rightsquigarrow} x \rightarrow y \overset{p_2}{\rightsquigarrow} u$, dove y è il primo vertice nel cammino che non si trova in S e $x \in S$ precede immediatamente y. I vertici x e y sono distinti, ma possiamo avere $s = x$ o $y = u$. Il cammino p_2 può rientrare oppure no nell'insieme S.

$u.d = \delta(s,u) = \infty$ per la proprietà dell'assenza di un cammino, e questo violerebbe la nostra ipotesi che $u.d \neq \delta(s,u)$. Poiché esiste almeno un cammino, allora deve esistere un cammino minimo p da s a u. Prima di aggiungere u all'insieme S, il cammino p collega un vertice in S, cioè s, a un vertice in $V - S$, cioè u. Consideriamo il primo vertice y lungo p tale che $y \in V - S$ e sia $x \in S$ il predecessore di y. Allora, come illustra la Figura 24.7, il cammino p può essere scomposto in $s \overset{p_1}{\rightsquigarrow} x \rightarrow y \overset{p_2}{\rightsquigarrow} u$ (ciascuno dei cammini p_1 e p_2 potrebbe non avere archi).

Asseriamo che $y.d = \delta(s,y)$ quando u viene aggiunto a S. Per dimostrare ciò, osserviamo che $x \in S$. Allora, poiché u è scelto come il primo vertice per il quale $u.d \neq \delta(s,u)$ quando esso viene aggiunto a S, era vero che $x.d = \delta(s,x)$ quando x è stato aggiunto a S. L'arco (x,y) è stato rilassato in quell'istante, quindi l'asserzione è dimostrata per la proprietà della convergenza.

Adesso possiamo ottenere una contraddizione per dimostrare che $u.d = \delta(s,u)$. Poiché y precede u in un cammino minimo da s a u e tutti i pesi degli archi non sono negativi (in particolare quelli sul cammino p_2), si ha $\delta(s,y) \leq \delta(s,u)$, e quindi

$$\begin{aligned} y.d &= \delta(s,y) \\ &\leq \delta(s,u) \qquad (24.2) \\ &\leq u.d \qquad \text{(per la proprietà del limite superiore)} \end{aligned}$$

Tuttavia, poiché entrambi i vertici u e y si trovavano in $V - S$ quando u venne scelto nella riga 5, si ha $u.d \leq y.d$. Quindi, le due disuguaglianze in (24.2) sono in effetti delle uguaglianze:

$$y.d = \delta(s,y) = \delta(s,u) = u.d$$

Di conseguenza, $u.d = \delta(s,u)$, che contraddice la nostra scelta di u. Concludiamo che $u.d = \delta(s,u)$ quando u viene aggiunto a S, e che questa uguaglianza permane in tutti gli istanti successivi.

Conclusione: Al termine, $Q = \emptyset$ che, insieme con la nostra precedente invariante che $Q = V - S$, implica che $S = V$. Dunque, $u.d = \delta(s,u)$ per tutti i vertici $u \in V$. ■

Acquistato da Michele Michele su Webster il 2022-07-07 21:43 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Corollario 24.7

Se eseguiamo l'algoritmo di Dijkstra su un grafo orientato pesato $G = (V,E)$ con una funzione peso non negativa w e sorgente s, allora al termine dell'algoritmo, il sottografo dei predecessori G_π è un albero di cammini minimi radicato in s.

Dimostrazione È una conseguenza immediata del Teorema 24.6 e della proprietà del sottografo dei predecessori. ■

Analisi

Quanto è veloce l'algoritmo di Dijkstra? Esso mantiene la coda di min-priorità Q chiamando tre operazioni delle code di priorità: INSERT (implicita nella riga 3), EXTRACT-MIN (riga 5) e DECREASE-KEY (implicita in RELAX, che è chiamata nella riga 8). L'operazione INSERT viene chiamata una sola volta per ogni vertice, come EXTRACT-MIN. Poiché ogni vertice $u \in V$ viene aggiunto all'insieme S esattamente una volta, ogni arco nella lista di adiacenza $Adj[u]$ viene esaminato nel ciclo **for** delle righe 7–8 esattamente una volta durante l'esecuzione dell'algoritmo. Poiché il numero totale di archi in tutte le liste di adiacenza è $|E|$, ci sono in totale $|E|$ iterazioni di questo ciclo **for**, e quindi in totale l'algoritmo chiama DECREASE-KEY al più $|E|$ volte (facciamo presente ancora una volta che stiamo utilizzando il metodo di aggregazione).

Il tempo di esecuzione dell'algoritmo di Dijkstra dipende dal modo in cui viene implementata la coda di min-priorità. Consideriamo prima il caso in cui manteniamo la coda di min-priorità sfruttando il fatto che i vertici sono numerati da 1 a $|V|$. Memorizziamo semplicemente $v.d$ nell'elemento v-esimo di un array. Ciascuna operazione INSERT e DECREASE-KEY impiega un tempo $O(1)$ e ciascuna operazione EXTRACT-MIN impiega un tempo $O(V)$ (perché dobbiamo ispezionare l'intero array), per un tempo totale pari a $O(V^2 + E) = O(V^2)$.

Se il grafo è sufficientemente sparso – in particolare, $E = o(V^2/\lg V)$ – conviene implementare la coda di min-priorità con un min-heap binario (come abbiamo descritto nel Paragrafo 6.5, un importante dettaglio di implementazione è che i vertici devono mantenere dei puntatori ai corrispondenti elementi dell'heap e viceversa). Ogni operazione EXTRACT-MIN richiede un tempo $O(\lg V)$. Come prima, ci sono $|V|$ operazioni come questa. Il tempo per costruire un min-heap binario è $O(V)$. Ogni operazione DECREASE-KEY richiede un tempo $O(\lg V)$, e ci sono al più $|E|$ operazioni come questa. Il tempo totale di esecuzione è dunque $O((V + E) \lg V)$, che è $O(E \lg V)$ se tutti i vertici sono raggiungibili dalla sorgente. Questo tempo di esecuzione è un miglioramento rispetto alla semplice implementazione con tempo $O(V^2)$ se $E = o(V^2/\lg V)$.

In effetti, è possibile ottenere un tempo di esecuzione pari a $O(V \lg V + E)$ implementando la coda di min-priorità con un heap di Fibonacci (vedere il Capitolo 19). Il costo ammortizzato di ciascuna delle $|V|$ operazioni EXTRACT-MIN è $O(\lg V)$ e ogni chiamata a DECREASE-KEY richiede soltanto un tempo ammortizzato $O(1)$; queste chiamate in totale non sono più di $|E|$. Storicamente, lo sviluppo degli heap di Fibonacci è stato motivato dall'osservazione che nell'algoritmo di Dijkstra ci sono, tipicamente, molte più chiamate a DECREASE-KEY che a EXTRACT-MIN, quindi qualsiasi metodo per ridurre il tempo ammortizzato dell'operazione DECREASE-KEY a $o(\lg V)$, senza aumentare il tempo ammortizzato di EXTRACT-MIN, avrebbe generato un'implementazione asintoticamente più veloce rispetto agli heap binari.

L'algoritmo di Dijkstra ha delle analogie sia con la visita in ampiezza (vedere il Paragrafo 22.2) sia con l'algoritmo di Prim per calcolare gli alberi di copertura minimi (vedere il Paragrafo 23.2). È simile alla visita in ampiezza in quanto l'insieme S corrisponde all'insieme dei vertici neri in una visita in ampiezza; proprio come i vertici in S hanno i loro pesi finali dei cammini minimi, così i vertici neri hanno le loro distanze corrette in una visita in ampiezza. L'algoritmo di Dijkstra è simile a quello di Prim in quanto entrambi gli algoritmi utilizzano una coda di min-priorità per trovare il vertice "più leggero" al di fuori di un dato insieme

Acquistato da Michele Mi[illegible] - 2023-07-28 12:[illegible] - Copia limitata - ISBN 9788838693[illegible] - © Edizioni 2022 McGraw-Hill Education (Italy)

(l'insieme S nell'algoritmo di Dijkstra e l'albero in costruzione nell'algoritmo di Prim), aggiungono questo vertice all'insieme e regolano opportunamente i pesi dei rimanenti vertici che non appartengono all'insieme.

Esercizi

24.4-1

Eseguite l'algoritmo di Dijkstra sul grafo orientato illustrato nella Figura 24.2, utilizzando come sorgente prima il vertice s e poi il vertice z. Nello stile della Figura 24.6, indicate i valori d e π e i vertici nell'insieme S dopo ogni iterazione del ciclo **while**.

24.4-2

Trovate un semplice esempio di un grafo orientato con archi di peso negativo per il quale l'algoritmo di Dijkstra fornisce risultati errati. Perché la dimostrazione del Teorema 24.6 non è applicabile se si ammettono archi con peso negativo?

24.4-3

Supponete di modificare la riga 4 dell'algoritmo di Dijkstra in questo modo:

4 **while** $|Q| > 1$

Questa modifica fa sì che il ciclo **while** venga eseguito $|V| - 1$ volte, anziché $|V|$ volte. L'algoritmo proposto è corretto?

24.4-4

Il professor Gaedel ha scritto un programma per implementare l'algoritmo di Dijkstra. Il programma genera $v.d$ e $v.\pi$ per ogni vertice $v \in V$. Scrivete un algoritmo con tempo $O(V + E)$ per controllare l'output del programma del professore. L'algoritmo deve determinare se gli attributi d e π corrispondono a quelli dell'albero dei cammini minimi. Potete supporre che tutti i pesi degli archi siano non negativi.

24.4-5

Il professor Newman sostiene di aver trovato un metodo più semplice per verificare la correttezza dell'algoritmo di Dijkstra. Egli afferma che l'algoritmo di Dijkstra rilassa gli archi di ciascun cammino minimo nel grafo nell'ordine in cui appaiono nel cammino, e quindi applica la proprietà del rilassamento del cammino a ciascun vertice raggiungibile dalla sorgente. Dimostrate che il professore si sbaglia costruendo un grafo orientato per il quale l'algoritmo di Dijkstra potrebbe rilassare gli archi di un cammino minimo fuori sequenza.

24.4-6

Dato un grafo orientato $G = (V, E)$ nel quale ogni arco $(u, v) \in E$ ha un valore associato $r(u, v)$, che è un numero reale nell'intervallo $0 \leq r(u, v) \leq 1$ che rappresenta l'affidabilità di un canale di comunicazione dal vertice u al vertice v. Interpretiamo $r(u, v)$ come la probabilità che il canale da u a v non fallisca e supponiamo che queste probabilità siano indipendenti. Create un algoritmo efficiente per trovare il cammino più affidabile fra due vertici dati.

Acquistato da Michele Michele su Webster il 2022-02-07 17:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

24.4-7

Sia $G = (V, E)$ un grafo orientato pesato con funzione peso positiva $w : E \to \{1, 2, \ldots, W\}$, dove W è un intero positivo; supponiamo che due vertici non possano avere gli stessi pesi di un cammino minimo dal vertice sorgente s. Supponiamo adesso di definire un grafo orientato non pesato $G' = (V \cup V', E')$ sostituendo ciascun arco $(u, v) \in E$ con $w(u, v)$ archi di peso unitario in serie. Quanti vertici ha il grafo G'? Supponiamo adesso di effettuare una visita in am-

piezza nel grafo G'. Dimostrate che l'ordine in cui i vertici di V vengono colorati di nero durante la visita in ampiezza di G' è lo stesso ordine in cui i vertici di V vengono estratti dalla coda di priorità nella riga 5 dell'algoritmo di Dijkstra quando questo viene eseguito sul grafo G.

24.4-8

Sia $G = (V, E)$ un grafo orientato pesato con funzione peso non negativa $w : E \to \{0, 1, \ldots, W\}$, dove W è un intero non negativo. Modificate l'algoritmo di Dijkstra per calcolare i cammini minimi da una data sorgente s nel tempo $O(WV + E)$.

24.4-9

Modificate l'algoritmo dell'Esercizio 24.4-8 in modo che sia eseguito nel tempo $O((V + E) \lg W)$. (*Suggerimento:* quante stime distinte di cammini minimi ci possono essere in $V - S$ in un istante qualsiasi?)

24.4-10

Supponete di avere un grafo orientato pesato $G = (V, E)$ in cui gli archi che escono dalla sorgente s possono avere pesi negativi, tutti gli altri pesi degli archi non sono negativi e non ci sono cicli di peso negativo. Dimostrate che l'algoritmo di Dijkstra trova correttamente i cammini minimi da s in questo grafo.

24.5 Vincoli sulle differenze e cammini minimi

Il Capitolo 29 studia il problema generale di programmazione lineare, in cui occorre ottimizzare una funzione lineare soggetta a un insieme di disuguaglianze lineari. In questo paragrafo, esamineremo un caso speciale di programmazione lineare che può essere ridotto alla ricerca di cammini minimi da una sorgente unica. Il problema dei cammini minimi da sorgente unica che si ottiene può essere poi risolto utilizzando l'algoritmo di Bellman-Ford, che così risolve anche il problema di programmazione lineare.

Programmazione lineare

Nel classico ***problema di programmazione lineare*** sono dati una matrice A di m righe ed n colonne, un vettore b di m elementi e un vettore c di n elementi. Vogliamo trovare un vettore x di n elementi che massimizza la ***funzione obiettivo*** $\sum_{i=1}^{n} c_i x_i$ soggetta agli m vincoli dati da $Ax \leq b$.

Sebbene l'algoritmo del simplesso, che è l'argomento principale del Capitolo 29, non sempre venga eseguito in tempo polinomiale nella dimensione del suo input, ci sono altri algoritmi per la programmazione lineare che vengono eseguiti in tempo polinomiale. Sono varie le ragioni per le quali è importante capire come sono strutturati i problemi di programmazione lineare. In primo luogo, sapere che un determinato problema può essere trattato come un problema di programmazione lineare di dimensione polinomiale significa immediatamente che esiste un algoritmo con tempo polinomiale per il problema. In secondo luogo, ci sono molti casi speciali di programmazione lineare per i quali esistono algoritmi più veloci. Per esempio, il problema dei cammini minimi da sorgente unica (Esercizio 24.5-4) e il problema di flusso massimo (Esercizio 26.1-5) sono casi speciali di programmazione lineare.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

A volte non si è interessati alla funzione obiettivo, ma si vuole semplicemente trovare una ***soluzione ammissibile***, cioè un vettore x che soddisfa la relazione $Ax \leq b$, oppure determinare che non esiste una soluzione ammissibile. Analizzeremo uno di questi ***problemi di ammissibilità***.

Sistemi di vincoli sulle differenze

In un ***sistema di vincoli sulle differenze***, ogni riga della matrice A della programmazione lineare contiene un 1 e un -1; tutti gli altri elementi della matrice sono 0. Quindi, i vincoli dati da $Ax \leq b$ sono un insieme di m ***vincoli sulle differenze*** con n incognite, in cui ciascun vincolo è una semplice disuguaglianza lineare della forma

$$x_j - x_i \leq b_k$$

dove $1 \leq i, j \leq n, i \neq j$ e $1 \leq k \leq m$.

Per esempio, consideriamo il problema di trovare il vettore $x = (x_i)$ di 5 elementi che soddisfa la relazione

$$\begin{pmatrix} 1 & -1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 & -1 \\ -1 & 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 1 & 0 \\ 0 & 0 & -1 & 0 & 1 \\ 0 & 0 & 0 & -1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{pmatrix} \leq \begin{pmatrix} 0 \\ -1 \\ 1 \\ 5 \\ 4 \\ -1 \\ -3 \\ -3 \end{pmatrix}$$

Questo problema equivale a trovare i valori delle incognite x_1, x_2, x_3, x_4, x_5, tali che siano soddisfatti i seguenti 8 vincoli sulle differenze:

$$x_1 - x_2 \leq 0 \tag{24.3}$$
$$x_1 - x_5 \leq -1 \tag{24.4}$$
$$x_2 - x_5 \leq 1 \tag{24.5}$$
$$x_3 - x_1 \leq 5 \tag{24.6}$$
$$x_4 - x_1 \leq 4 \tag{24.7}$$
$$x_4 - x_3 \leq -1 \tag{24.8}$$
$$x_5 - x_3 \leq -3 \tag{24.9}$$
$$x_5 - x_4 \leq -3 \tag{24.10}$$

Una soluzione di questo problema è $x = (-5, -3, 0, -1, -4)$, come può essere verificato direttamente controllando le singole disuguaglianze. In effetti, questo problema ammette più di una soluzione. Un'altra soluzione è $x' = (0, 2, 5, 4, 1)$. Queste due soluzioni sono correlate: ogni componente di x' è più grande di 5 della corrispondente componente di x. Questo fatto non è una mera coincidenza.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Lemma 24.8

Sia $x = (x_1, x_2, \ldots, x_n)$ una soluzione di un sistema $Ax \leq b$ di vincoli sulle differenze e sia d una costante qualsiasi. Allora anche $x + d = (x_1 + d, x_2 + d, \ldots, x_n + d)$ è una soluzione di $Ax \leq b$.

Dimostrazione Per ogni x_i e x_j, si ha $(x_j + d) - (x_i + d) = x_j - x_i$. Quindi, se x soddisfa la relazione $Ax \leq b$, anche $x + d$ la soddisfa. ■

I sistemi di vincoli sulle differenze si presentano in varie applicazioni. Per esempio, le incognite x_i potrebbero essere gli istanti di tempo in cui si verificano determinati eventi. In questo caso, ciascun vincolo potrebbe imporre che fra due eventi consecutivi debba trascorre una quantità minima o massima di tempo. Gli eventi potrebbero rappresentare i lavori da eseguire durante l'assemblaggio di un prodotto. Se al tempo x_1 applichiamo un adesivo che richiede di aspettare 2 ore per asciugare e dobbiamo aspettare che esso sia asciutto per incollare un nuovo pezzo all'istante x_2, allora abbiamo il vincolo che $x_2 \geq x_1 + 2$ ovvero che $x_1 - x_2 \leq -2$. In alternativa, potremmo richiedere che il nuovo pezzo possa essere incollato dopo che è stato applicato l'adesivo ma non più tardi di quando l'adesivo si sia asciugato a metà. In questo caso, otteniamo la coppia di vincoli $x_2 \geq x_1$ e $x_2 \leq x_1 + 1$ ovvero $x_1 - x_2 \leq 0$ e $x_2 - x_1 \leq 1$.

Grafo dei vincoli

È conveniente interpretare i sistemi dei vincoli sulle differenze da un punto di vista della teoria dei grafi. In un sistema $Ax \leq b$ di vincoli alle differenze, possiamo interpretare A, la matrice $m \times n$ della programmazione lineare, come la matrice trasposta di una matrice di incidenza (vedere l'Esercizio 22.1-7) per un grafo con n vertici ed m archi. Ogni vertice v_i nel grafo, per $i = 1, 2, \ldots, n$, corrisponde a una delle n variabili incognite x_i. Ogni arco orientato nel grafo corrisponde a una delle m disuguaglianze a due incognite.

Più formalmente, dato un sistema $Ax \leq b$ di vincoli sulle differenze, il corrispondente ***grafo dei vincoli*** è un grafo orientato pesato $G = (V, E)$, dove

$$V = \{v_0, v_1, \ldots, v_n\}$$

e

$$E = \{(v_i, v_j) : x_j - x_i \leq b_k \text{ è un vincolo}\} \cup \{(v_0, v_1), (v_0, v_2), (v_0, v_3), \ldots, (v_0, v_n)\}$$

Il vertice addizionale v_0 viene aggiunto, come vedremo in seguito, per garantire l'esistenza di un vertice da cui ogni altro vertice è raggiungibile. Quindi, l'insieme dei vertici V è formato da un vertice v_i per ogni incognita x_i, più un vertice addizionale v_0. L'insieme degli archi E contiene un arco per ogni vincolo, più un arco (v_0, v_i) per ogni incognita x_i. Se $x_j - x_i \leq b_k$ è un vincolo, allora il peso dell'arco (v_i, v_j) è $w(v_i, v_j) = b_k$. Il peso di ogni arco che esce da v_0 è 0. La Figura 24.8 illustra il grafo dei vincoli per il sistema di vincoli sulle differenze (24.3)–(24.10).

Il seguente teorema dimostra che è possibile trovare una soluzione di un sistema di vincoli sulle differenze trovando i pesi di cammino minimo nel corrispondente grafo dei vincoli.

Teorema 24.9

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Dato un sistema $Ax \leq b$ di vincoli sulle differenze, sia $G = (V, E)$ il corrispondente grafo dei vincoli. Se G non contiene cicli di peso negativo, allora

$$x = (\delta(v_0, v_1), \delta(v_0, v_2), \delta(v_0, v_3), \ldots, \delta(v_0, v_n)) \tag{24.11}$$

è una soluzione ammissibile per il sistema. Se G contiene un ciclo di peso negativo, allora non esiste una soluzione ammissibile per il sistema.

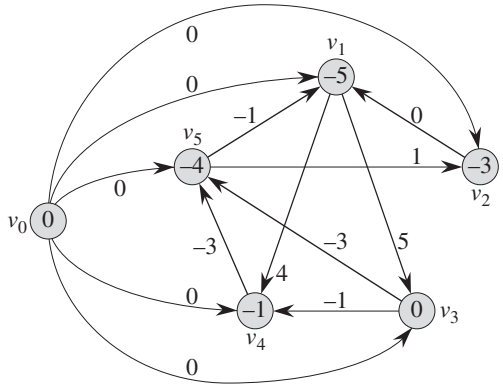


Figura 24.8 Il grafo dei vincoli che corrisponde al sistema di vincoli sulle differenze (24.3)–(24.10). All'interno di ogni vertice v_i è indicato il valore di $\delta(v_0, v_i)$. Una soluzione ammissibile del sistema è $x = (-5, -3, 0, -1, -4)$.

Dimostrazione Dimostriamo prima che, se il grafo dei vincoli non contiene cicli di peso negativo, allora l'equazione (24.11) fornisce una soluzione ammissibile. Consideriamo un arco qualsiasi $(v_i, v_j) \in E$. Per la disuguaglianza triangolare, $\delta(v_0, v_j) \leq \delta(v_0, v_i) + w(v_i, v_j)$ ovvero $\delta(v_0, v_j) - \delta(v_0, v_i) \leq w(v_i, v_j)$. Dunque, ponendo $x_i = \delta(v_0, v_i)$ e $x_j = \delta(v_0, v_j)$, il vincolo $x_j - x_i \leq w(v_i, v_j)$ che corrisponde all'arco (v_i, v_j) è soddisfatto.

Adesso dimostriamo che, se il grafo dei vincoli contiene un ciclo di peso negativo, allora il sistema di vincoli sulle differenze non ha una soluzione ammissibile. Senza perdere in generalità, supponiamo che il ciclo di peso negativo sia $c = \langle v_1, v_2, \ldots, v_k \rangle$, dove $v_1 = v_k$ (il vertice v_0 non può trovarsi sul ciclo c, perché non ha archi entranti). Il ciclo c corrisponde ai seguenti vincoli sulle differenze:

$$\begin{aligned} x_2 - x_1 &\leq w(v_1, v_2) \\ x_3 - x_2 &\leq w(v_2, v_3) \\ &\vdots \\ x_{k-1} - x_{k-2} &\leq w(v_{k-2}, v_{k-1}) \\ x_k - x_{k-1} &\leq w(v_{k-1}, v_k) \end{aligned}$$

Supponiamo che esista una soluzione per x che soddisfa ciascuna di queste k disuguaglianze. Questa soluzione deve anche soddisfare la disuguaglianza che si ottiene sommando tutte le k disuguaglianze. Se sommiamo gli elementi a sinistra, ciascuna incognita x_i viene sommata una volta e sottratta una volta (ricordiamo che $v_1 = v_k$ implica $x_1 = x_k$), quindi la somma degli elementi a sinistra è 0. La somma degli elementi a destra è $w(c)$, quindi otteniamo $0 \leq w(c)$. Tuttavia, poiché c è un ciclo di peso negativo, $w(c) < 0$; questo porta alla contraddizione che $0 \leq w(c) < 0$. ■

Risoluzione dei sistemi di vincoli sulle differenze

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Il Teorema 24.9 ci dice che possiamo utilizzare l'algoritmo di Bellman-Ford per risolvere un sistema di vincoli sulle differenze. Poiché ci cono archi che dal vertice sorgente v_0 raggiungono tutti gli altri vertici nel grafo dei vincoli, qualsiasi ciclo di peso negativo nel grafo dei vincoli è raggiungibile da v_0. Se l'algoritmo di Bellman-Ford restituisce TRUE, allora i pesi di cammino minimo sono una soluzione ammissibile per il sistema. Nella Figura 24.8, per esempio, i pesi di cam-

mino minimo forniscono la soluzione ammissibile $x = (-5, -3, 0, -1, -4)$ e, per il Lemma 24.8, anche $x = (d-5, d-3, d, d-1, d-4)$ è una soluzione ammissibile per qualsiasi costante d. Se l'algoritmo di Bellman-Ford restituisce FALSE, non esiste una soluzione ammissibile per il sistema di vincoli sulle differenze.

Un sistema di vincoli sulle differenze con m vincoli in n incognite produce un grafo con $n+1$ vertici e $n+m$ archi. Quindi, utilizzando l'algoritmo di Bellman-Ford, è possibile risolvere il sistema nel tempo $O((n+1)(n+m)) = O(n^2+nm)$. L'Esercizio 24.5-5 chiede di modificare l'algoritmo in modo che possa essere eseguito nel tempo $O(nm)$, anche se m è molto più piccolo di n.

Esercizi

24.5-1

Trovate una soluzione ammissibile oppure determinate che non esiste una soluzione ammissibile per il seguente sistema di vincoli sulle differenze:

$$\begin{aligned}
x_1 - x_2 &\le 1\\
x_1 - x_4 &\le -4\\
x_2 - x_3 &\le 2\\
x_2 - x_5 &\le 7\\
x_2 - x_6 &\le 5\\
x_3 - x_6 &\le 10\\
x_4 - x_2 &\le 2\\
x_5 - x_1 &\le -1\\
x_5 - x_4 &\le 3\\
x_6 - x_3 &\le -8
\end{aligned}$$

24.5-2

Trovate una soluzione ammissibile oppure determinate che non esiste una soluzione ammissibile per il seguente sistema di vincoli sulle differenze:

$$\begin{aligned}
x_1 - x_2 &\le 4\\
x_1 - x_5 &\le 5\\
x_2 - x_4 &\le -6\\
x_3 - x_2 &\le 1\\
x_4 - x_1 &\le 3\\
x_4 - x_3 &\le 5\\
x_4 - x_5 &\le 10\\
x_5 - x_3 &\le -4\\
x_5 - x_4 &\le -8
\end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

24.5-3

In un grafo di vincoli può un cammino minimo che inizia nel nuovo vertice v_0 avere peso positivo? Spiegate la risposta.

24.5-4

Descrivete il problema del cammino minimo per una singola coppia di vertici come un programma lineare.

24.5-5
Illustrate una piccola modifica da apportare all'algoritmo di Bellman-Ford in modo che, quando l'algoritmo viene utilizzato per risolvere un sistema di vincoli sulle differenze con m disuguaglianze in n incognite, il tempo di esecuzione sia $O(nm)$.

24.5-6
Supponete che, oltre a un sistema di vincoli sulle differenze, vogliate gestire anche i ***vincoli di uguaglianza*** della forma $x_i = x_j + b_k$. Spiegate come adattare l'algoritmo di Bellman-Ford per risolvere questa variante del sistema di vincoli.

24.5-7
Spiegate come risolvere un sistema di vincoli sulle differenze utilizzando un algoritmo simile a quello di Bellman-Ford che viene eseguito su un grafo di vincoli senza il vertice extra v_0.

24.5-8 ★
Sia $Ax \leq b$ un sistema di m vincoli sulle differenze in n incognite. Dimostrate che l'algoritmo di Bellman-Ford, quando viene eseguito sul corrispondente grafo dei vincoli, massimizza la sommatoria $\sum_{i=1}^{n} x_i$ soggetta ai vincoli $Ax \leq b$ e $x_i \leq 0$ per ogni x_i.

24.5-9 ★
Dimostrate che l'algoritmo di Bellman-Ford, quando viene eseguito sul grafo dei vincoli per un sistema $Ax \leq b$ di vincoli sulle differenze, minimizza la quantità $(\max\{x_i\} - \min\{x_i\})$ soggetta a $Ax \leq b$. Spiegate come possa essere sfruttato questo fatto utilizzando l'algoritmo per programmare i lavori da eseguire nei processi di fabbricazione.

24.5-10
Supponete che ogni riga della matrice A di un programma lineare $Ax \leq b$ corrisponda a un vincolo sulla differenza, a un vincolo su una sola variabile della forma $x_i \leq b_k$ o a un vincolo su una sola variabile della forma $-x_i \leq b_k$. Spiegate come adattare l'algoritmo di Bellman-Ford per risolvere questa variante del sistema di vincoli.

24.5-11
Create un algoritmo efficiente per risolvere un sistema $Ax \leq b$ di vincoli sulle differenze quando tutti gli elementi di b sono valori reali e tutte le incognite x_i devono essere numeri interi.

24.5-12 ★
Create un algoritmo efficiente per risolvere un sistema $Ax \leq b$ di vincoli sulle differenze quando tutti gli elementi di b sono valori reali e alcune incognite x_i (non necessariamente tutte) devono essere numeri interi.

24.6 Dimostrazione delle proprietà dei cammini minimi

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Nel corso di questo capitolo le nostre argomentazioni sulla correttezza si sono basate sulla disuguaglianza triangolare, sulla proprietà del limite superiore, sulla proprietà dell'assenza di un cammino, sulla proprietà della convergenza, sulla proprietà del rilassamento del cammino e sulla proprietà del sottografo dei predecessori. Abbiamo definito queste proprietà all'inizio del capitolo, senza dimostrarle; lo faremo in questo paragrafo.

La disuguaglianza triangolare

Analizzando la visita in ampiezza (Paragrafo 22.2), abbiamo dimostrato nel Lemma 22.1 una semplice proprietà delle distanze minime nei grafi non pesati. La disuguaglianza triangolare generalizza tale proprietà per i grafi pesati.

Lemma 24.10 (Disuguaglianza triangolare)
Sia $G = (V, E)$ un grafo orientato pesato con funzione peso $w : E \to \mathbb{R}$ e sorgente s. Allora, per tutti gli archi $(u, v) \in E$, si ha

$$\delta(s, v) \leq \delta(s, u) + w(u, v)$$

Dimostrazione Supponiamo che ci sia un cammino minimo p dalla sorgente s al vertice v. Allora p non pesa più di qualsiasi altro cammino da s a v. Specificatamente, il cammino p non pesa più del particolare percorso che è formato da un cammino minimo dalla sorgente s al vertice u e poi dall'arco (u, v). L'Esercizio 24.6-3 chiede di gestire il caso in cui non ci sia un cammino minimo da s a v. ■

Effetti del rilassamento sulle stime dei cammini minimi

Il prossimo gruppo di lemmi descrive gli effetti sulle stime dei cammini minimi di una sequenza di passi di rilassamento sugli archi di un grafo orientato pesato che è stato inizializzato dalla procedura INITIALIZE-SINGLE-SOURCE.

Lemma 24.11 (Proprietà del limite superiore)
Sia $G = (V, E)$ un grafo orientato pesato con funzione peso $w : E \to \mathbb{R}$. Sia $s \in V$ il vertice sorgente e supponiamo che il grafo sia inizializzato dalla procedura INITIALIZE-SINGLE-SOURCE(G, s). Allora, $v.d \geq \delta(s, v)$ per ogni $v \in V$ e questa invariante si conserva per qualsiasi sequenza di passi di rilassamento sugli archi di G. Inoltre, una volta che $v.d$ raggiunge il suo limite inferiore $\delta(s, v)$, non cambia più.

Dimostrazione Dimostriamo l'invariante $v.d \geq \delta(s, v)$ per tutti i vertici $v \in V$ per induzione sul numero di passi di rilassamento.

Per il caso base, la relazione $v.d \geq \delta(s, v)$ è certamente vera dopo l'inizializzazione, perché $s.d = 0 \geq \delta(s, s)$ (notate che $\delta(s, s)$ vale $-\infty$ se s si trova in un ciclo di peso negativo, altrimenti vale 0) e $v.d = \infty$ implica $v.d \geq \delta(s, v)$ per ogni $v \in V - \{s\}$.

Per il passaggio induttivo, consideriamo il rilassamento di un arco (u, v). Per l'ipotesi induttiva, $x.d \geq \delta(s, x)$ per ogni $x \in V$ prima del rilassamento. L'unico valore d che può cambiare è $v.d$. Se questo valore cambia, si ha

$$\begin{aligned} v.d &= u.d + w(u, v) \\ &\geq \delta(s, u) + w(u, v) && \text{(per l'ipotesi induttiva)} \\ &\geq \delta(s, v) && \text{(per la disuguaglianza triangolare)} \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Quindi l'invariante si è conservata.

Per capire perché il valore di $v.d$ non cambia più dopo che $v.d = \delta(s, v)$, notate che, avendo raggiunto il suo limite inferiore, $v.d$ non può diminuire perché abbiamo appena dimostrato che $v.d \geq \delta(s, v)$ né può aumentare perché i passi di rilassamento non aumentano i valori d. ■

Corollario 24.12 (Proprietà dell'assenza di un cammino)
Supponiamo che in un grafo orientato pesato $G = (V, E)$ con funzione peso $w : E \to \mathbb{R}$, non esista un cammino che collega un vertice sorgente $s \in V$ a un dato vertice $v \in V$. Allora, dopo che il grafo è stato inizializzato da INITIALIZE-SINGLE-SOURCE(G, s), si ha $v.d = \delta(s, v) = \infty$; questa uguaglianza si mantiene come un'invariante per qualsiasi sequenza di passi di rilassamento sugli archi del grafo G.

Dimostrazione Per la proprietà del limite superiore, si ha sempre $\infty = \delta(s, v) \leq v.d$, e quindi $v.d = \infty = \delta(s, v)$. ■

Lemma 24.13
Sia $G = (V, E)$ un grafo orientato pesato con funzione peso $w : E \to \mathbb{R}$ e sia $(u, v) \in E$. Allora, immediatamente dopo il rilassamento dell'arco (u, v) effettuato da RELAX(u, v, w), si ha $v.d \leq u.d + w(u, v)$.

Dimostrazione Se, prima del rilassamento dell'arco (u, v), si ha $v.d > u.d + w(u, v)$, successivamente si avrà $v.d = u.d + w(u, v)$. Se, invece, $v.d \leq u.d + w(u, v)$ prima del rilassamento, allora non cambia né $u.d$ né $v.d$, e quindi successivamente si avrà $v.d \leq u.d + w(u, v)$. ■

Lemma 24.14 (Proprietà della convergenza)
Sia $G = (V, E)$ un grafo orientato pesato con funzione peso $w : E \to \mathbb{R}$. Sia $s \in V$ un vertice sorgente e sia $s \rightsquigarrow u \to v$ un cammino minimo in G per alcuni vertici $u, v \in V$. Supponiamo che G sia inizializzato da INITIALIZE-SINGLE-SOURCE(G, s) e che poi sia eseguita sugli archi di G una sequenza di passi di rilassamento che include la chiamata RELAX(u, v, w). Se $u.d = \delta(s, u)$ in un istante qualsiasi prima della chiamata, allora sarà sempre $v.d = \delta(s, v)$ dopo la chiamata.

Dimostrazione Per la proprietà del limite superiore, se $u.d = \delta(s, u)$ in un istante qualsiasi prima del rilassamento dell'arco (u, v), allora questa uguaglianza si mantiene successivamente. In particolare, dopo il rilassamento dell'arco (u, v), si ha

$$\begin{aligned} v.d &\leq u.d + w(u, v) && \text{(per il Lemma 24.13)} \\ &= \delta(s, u) + w(u, v) \\ &= \delta(s, v) && \text{(per il Lemma 24.1)} \end{aligned}$$

Per la proprietà del limite superiore, $v.d \geq \delta(s, v)$, da cui si deduce che $v.d = \delta(s, v)$, e questa uguaglianza si mantiene successivamente. ■

Lemma 24.15 (Proprietà del rilassamento del cammino)
Sia $G = (V, E)$ un grafo orientato pesato con funzione peso $w : E \to \mathbb{R}$ e sia $s \in V$ un vertice sorgente. Consideriamo un cammino minimo $p = \langle v_0, v_1, \ldots, v_k \rangle$ da $s = v_0$ a v_k. Se il grafo G è inizializzato da INITIALIZE-SINGLE-SOURCE(G, s) e poi viene eseguita una sequenza di passi di rilassamento che include, nell'ordine, il rilassamento degli archi $(v_0, v_1), (v_1, v_2), \ldots, (v_{k-1}, v_k)$, allora sarà $v_k.d = \delta(s, v_k)$ dopo questi rilassamenti e anche successivamente.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine: Area3-3565234-220707. © Copyright © 2022 McGraw-Hill Education (Italy)

Questa proprietà è soddisfatta indipendentemente da quali altri rilassamenti di archi siano effettuati, inclusi quei rilassamenti che sono interposti fra i rilassamenti degli archi di p.

Dimostrazione Dimostriamo per induzione che, dopo il rilassamento dell'i-esimo arco del cammino p, si ha $v_i.d = \delta(s, v_i)$. Per il caso base, $i = 0$, e prima di rilassare qualsiasi arco di p, si ha dall'inizializzazione che $v_0.d = s.d = 0 = \delta(s, s)$. Per la proprietà del limite superiore, il valore di $s.d$ non cambia più dopo l'inizializzazione.

Per il passaggio induttivo, supponiamo che $v_{i-1}.d = \delta(s, v_{i-1})$ ed esaminiamo il rilassamento dell'arco (v_{i-1}, v_i). Per la proprietà della convergenza, dopo questo rilassamento, si ha $v_i.d = \delta(s, v_i)$, e questa uguaglianza si mantiene successivamente. ■

Rilassamento e alberi di cammini minimi

Adesso dimostriamo che, dopo che una sequenza di rilassamenti ha portato le stime dei cammini minimi a convergere ai pesi dei cammini minimi, il sottografo dei predecessori G_π indotto dai valori π risultanti è un albero di cammini minimi per G. Iniziamo con il seguente lemma, che dimostra che il sottografo dei predecessori forma sempre un albero la cui radice è la sorgente.

Lemma 24.16
Sia $G = (V, E)$ un grafo orientato pesato con funzione peso $w : E \to \mathbb{R}$; sia $s \in V$ un vertice sorgente e supponiamo che G non contenga cicli di peso negativo che sono raggiungibili da s. Allora, dopo che il grafo è stato inizializzato da INITIALIZE-SINGLE-SOURCE(G, s), il sottografo dei predecessori G_π forma un albero con radice s e una sequenza qualsiasi di passi di rilassamento sugli archi di G mantiene questa proprietà come invariante.

Dimostrazione Inizialmente, l'unico vertice in G_π è il vertice sorgente, quindi il lemma è banalmente vero. Consideriamo un sottografo di predecessori G_π che si forma dopo una sequenza di passi di rilassamento. Dimostriamo prima che G_π è aciclico. Supponiamo per assurdo che qualche passo di rilassamento crei un ciclo nel grafo G_π, che indichiamo con $c = \langle v_0, v_1, \ldots, v_k \rangle$, dove $v_k = v_0$. Allora, $v_i.\pi = v_{i-1}$ per $i = 1, 2, \ldots, k$ e, senza perdere in generalità, possiamo supporre che sia stato il rilassamento dell'arco (v_{k-1}, v_k) a generare il ciclo in G_π.

Asseriamo che tutti i vertici nel ciclo c sono raggiungibili dalla sorgente s. Perché? Ogni vertice in c ha un predecessore diverso da NIL, quindi a ciascun vertice in c è stato assegnato un valore finito della stima del cammino minimo quando al suo attributo π è stato assegnato un valore diverso da NIL. Per la proprietà del limite superiore, ogni vertice nel ciclo c ha un peso di cammino minimo di valore finito; questo implica che ciascun vertice è raggiungibile da s.

Esamineremo le stime dei cammini minimi nel ciclo c appena prima di chiamare RELAX(v_{k-1}, v_k, w) e dimostreremo che c è un ciclo di peso negativo, contraddicendo l'ipotesi che G non contenga cicli di peso negativo che sono raggiungibili dalla sorgente. Appena prima della chiamata, si ha $v_i.\pi = v_{i-1}$ per $i = 1, 2, \ldots, k - 1$. Quindi, per $i = 1, 2, \ldots, k - 1$, l'ultimo aggiornamento di $v_i.d$ è stato fatto dall'assegnazione $v_i.d = v_{i-1}.d + w(v_{i-1}, v_i)$. Se il valore di $v_{i-1}.d$ cambiasse da allora, potrebbe soltanto diminuire. Quindi, appena prima della chiamata RELAX(v_{k-1}, v_k, w), si ha

$$v_i.d \geq v_{i-1}.d + w(v_{i-1}, v_i) \qquad \text{per ogni } i = 1, 2, \ldots, k - 1 \tag{24.12}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995030162 20707 © Copyright © 2022, McGraw-Hill Education (Italy)

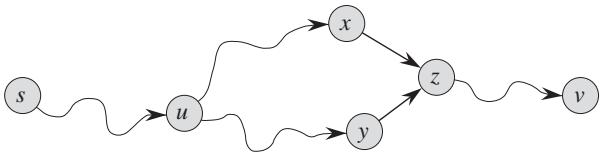


Figura 24.9
La dimostrazione che un cammino in G_π dalla sorgente s al vertice v è unico. Se ci fossero due cammini p_1 ($s \rightsquigarrow u \rightsquigarrow x \rightarrow z \rightsquigarrow v$) e p_2 ($s \rightsquigarrow u \rightsquigarrow y \rightarrow z \rightsquigarrow v$), con $x \neq y$, allora $z.\pi = x$ e $z.\pi = y$: una contraddizione.

Poiché la chiamata modifica $v_k.\pi$, immediatamente prima abbiamo anche la disuguaglianza stretta

$$v_k.d > v_{k-1}.d + w(v_{k-1}, v_k)$$

Sommando questa disuguaglianza stretta con le $k - 1$ disuguaglianze (24.12), otteniamo la somma delle stime dei cammini minimi lungo il ciclo c:

$$\begin{aligned}\sum_{i=1}^{k} v_i.d &> \sum_{i=1}^{k}(v_{i-1}.d + w(v_{i-1}, v_i)) \\ &= \sum_{i=1}^{k} v_{i-1}.d + \sum_{i=1}^{k} w(v_{i-1}, v_i)\end{aligned}$$

Ma

$$\sum_{i=1}^{k} v_i.d = \sum_{i=1}^{k} v_{i-1}.d$$

perché ogni vertice nel ciclo c appare esattamente una volta in ciascuna sommatoria. Questa uguaglianza implica

$$0 > \sum_{i=1}^{k} w(v_{i-1}, v_i)$$

Quindi, la somma dei pesi lungo il ciclo c è negativa, e questo rappresenta la contraddizione richiesta.

A questo punto, abbiamo dimostrato che G_π è un grafo orientato aciclico. Per dimostrare che forma un albero con radice s, basta dimostrare che per ogni vertice $v \in V_\pi$ esiste un cammino unico da s a v in G_π (vedere l'Esercizio B.5-2).

Prima dobbiamo dimostrare che esiste un cammino che da s raggiunge qualsiasi vertice in V_π. I vertici in V_π sono quelli con valori π diversi da NIL, più il vertice s. Si può dimostrare per induzione che esiste un cammino da s a ogni vertice in V_π. Lasciamo al lettore i dettagli di questa dimostrazione (Esercizio 24.6-6).

Per completare la dimostrazione del lemma, dobbiamo provare che, per qualsiasi vertice $v \in V_\pi$, esiste al più un cammino da s a v nel grafo G_π. Supponiamo che non sia così. Ovvero supponiamo che ci siano due cammini semplici da s a un vertice v: p_1, che può essere scomposto in $s \rightsquigarrow u \rightsquigarrow x \rightarrow z \rightsquigarrow v$, e p_2, che può essere scomposto in $s \rightsquigarrow u \rightsquigarrow y \rightarrow z \rightsquigarrow v$, con $x \neq y$ (Figura 24.9). Ma allora $z.\pi = x$ e $z.\pi = y$; questo implica $x = y$, che è una contraddizione. Concludiamo che esiste un cammino semplice unico in G_π che va da s a v, e quindi G_π forma un albero con radice s. ■

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Adesso possiamo dimostrare che se, dopo avere eseguito una sequenza di passi di rilassamento, a tutti i vertici sono stati assegnati i valori esatti dei pesi dei cammini minimi, allora il sottografo dei predecessori G_π è un albero di cammini minimi.

Lemma 24.17 (Proprietà del sottografo dei predecessori)
Sia $G = (V, E)$ un grafo orientato pesato con funzione peso $w : E \to \mathbb{R}$; sia $s \in V$ un vertice sorgente e supponiamo che G non contenga cicli di peso negativo che sono raggiungibili da s. Chiamiamo la procedura INITIALIZE-SINGLE-SOURCE(G, s) e poi eseguiamo una sequenza qualsiasi di passi di rilassamento sugli archi di G che produce $v.d = \delta(s, v)$ per ogni $v \in V$. Allora, il sottografo dei predecessori G_π è un albero di cammini minimi radicato in s.

Dimostrazione Dobbiamo dimostrare che le tre proprietà degli alberi dei cammini minimi descritte a pagina 543 sono soddisfatte per G_π. Per dimostrare la prima proprietà, dobbiamo provare che V_π è l'insieme dei vertici raggiungibili da s. Per definizione, il peso di un cammino minimo $\delta(s, v)$ è finito se e soltanto se v è raggiungibile da s, e quindi i vertici che sono raggiungibili da s sono esattamente quelli con valori finiti di d. Ma a un vertice $v \in V - \{s\}$ è assegnato un valore finito per $v.d$ se e soltanto se $v.\pi \neq$ NIL. Quindi, i vertici in V_π sono esattamente quelli che sono raggiungibili da s.

La seconda proprietà deriva direttamente dal Lemma 24.16. Resta, quindi, da dimostrare l'ultima proprietà degli alberi dei cammini minimi: per ogni vertice $v \in V_\pi$, l'unico cammino semplice $s \overset{p}{\rightsquigarrow} v$ in G_π è un cammino minimo da s a v in G. Sia $p = \langle v_0, v_1, \ldots, v_k \rangle$, dove $v_0 = s$ e $v_k = v$. Per $i = 1, 2, \ldots, k$, si ha $v_i.d = \delta(s, v_i)$ e $v_i.d \geq v_{i-1}.d + w(v_{i-1}, v_i)$, per cui concludiamo che $w(v_{i-1}, v_i) \leq \delta(s, v_i) - \delta(s, v_{i-1})$.

Sommando i pesi lungo il cammino p, si ha

$$\begin{aligned}
w(p) &= \sum_{i=1}^{k} w(v_{i-1}, v_i) \\
&\leq \sum_{i=1}^{k} (\delta(s, v_i) - \delta(s, v_{i-1})) \\
&= \delta(s, v_k) - \delta(s, v_0) && \text{(perché gli altri termini della sommatoria si annullano a vicenda)} \\
&= \delta(s, v_k) && \text{(perché } \delta(s, v_0) = \delta(s, s) = 0\text{)}
\end{aligned}$$

Quindi, $w(p) \leq \delta(s, v_k)$. Poiché $\delta(s, v_k)$ è un limite inferiore per il peso di un cammino qualsiasi da s a v_k, concludiamo che $w(p) = \delta(s, v_k)$, e quindi p è un cammino minimo da s a $v = v_k$. ■

Esercizi

24.6-1
Trovate due alberi di cammini minimi per il grafo orientato illustrato nella Figura 24.2 (a pagina 543) diversi da quelli indicati.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

24.6-2
Trovate un esempio di un grafo orientato pesato $G = (V, E)$ con funzione peso $w : E \to \mathbb{R}$ e sorgente s tale che G soddisfi la seguente proprietà: per ogni arco $(u, v) \in E$, esiste un albero di cammini minimi radicato in s che contiene (u, v) e un altro albero di cammini minimi radicato in s che non contiene (u, v).

24.6-3
Completate la dimostrazione del Lemma 24.10 per gestire i casi in cui i pesi di cammino minimo siano ∞ o $-\infty$.

24.6-4
Sia $G = (V, E)$ un grafo orientato pesato con sorgente s e supponete che G sia inizializzato da INITIALIZE-SINGLE-SOURCE(G, s). Dimostrate che, se una sequenza di passi di rilassamento assegna a $s.\pi$ un valore diverso da NIL, allora G contiene un ciclo di peso negativo.

24.6-5
Sia $G = (V, E)$ un grafo orientato pesato senza archi di peso negativo e sia $s \in V$ il vertice sorgente. Supponete che $v.\pi$ sia il predecessore di v in un cammino minimo *qualsiasi* che porta a v dalla sorgente s se $v \in V - \{s\}$ è raggiungibile da s, o NIL negli altri casi. Trovate un esempio di un tale grafo G e un'assegnazione dei valori π che produce un ciclo in G_π. (Per il Lemma 24.16, tale assegnazione non può essere prodotta da una sequenza di passi di rilassamento.)

24.6-6
Sia $G = (V, E)$ un grafo orientato pesato con funzione peso $w : E \to \mathbb{R}$ e senza cicli di peso negativo. Sia $s \in V$ il vertice sorgente e supponete che il grafo G sia inizializzato dalla procedura INITIALIZE-SINGLE-SOURCE(G, s). Dimostrate che per ogni vertice $v \in V_\pi$, esiste un cammino da s a v in G_π e che questa proprietà si conserva come un'invariante per qualsiasi sequenza di rilassamenti.

24.6-7
Sia $G = (V, E)$ un grafo orientato pesato che non contiene cicli di peso negativo. Sia $s \in V$ il vertice sorgente e supponete che il grafo G sia inizializzato dalla procedura INITIALIZE-SINGLE-SOURCE(G, s). Dimostrate che esiste una sequenza di $|V| - 1$ passi di rilassamento che produce $v.d = \delta(s, v)$ per ogni $v \in V$.

24.6-8
Sia G un arbitrario grafo orientato pesato con un ciclo di peso negativo che è raggiungibile dalla sorgente s. Dimostrate che è sempre possibile costruire una sequenza infinita di rilassamenti degli archi di G in modo che ogni rilassamento provochi una variazione della stima del cammino minimo.

Problemi

24-1 Algoritmo di Bellman-Ford migliorato da Yen
Supponiamo di ordinare i rilassamenti degli archi in ciascun passaggio dell'algoritmo di Bellman-Ford nel modo seguente. Prima del primo passaggio, assegniamo un arbitrario ordinamento lineare $v_1, v_2, \ldots, v_{|V|}$ ai vertici del grafo di input $G = (V, E)$. Poi, partizioniamo l'insieme degli archi E in $E_f \cup E_b$, dove $E_f = \{(v_i, v_j) \in E : i < j\}$ ed $E_b = \{(v_i, v_j) \in E : i > j\}$. (Supponiamo che G non contenga cappi, in modo che ogni arco si trovi in E_f o in E_b.) Definiamo $G_f = (V, E_f)$ e $G_b = (V, E_b)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 (Numero Ordine Libreria: 1995901242267070) Copyright © 2022, McGraw-Hill Education (Italy)

a. Dimostrate che G_f è aciclico con ordinamento topologico $\langle v_1, v_2, \ldots, v_{|V|} \rangle$ e che G_b è aciclico con ordinamento topologico $\langle v_{|V|}, v_{|V|-1}, \ldots, v_1 \rangle$.

Supponiamo di implementare ciascun passaggio dell'algoritmo di Bellman-Ford nel modo seguente. Visitiamo ciascun vertice nell'ordine $v_1, v_2, \ldots, v_{|V|}$, rilassando gli archi di E_f che escono dal vertice. Poi visitiamo ciascun vertice nell'ordine $v_{|V|}, v_{|V|-1}, \ldots, v_1$, rilassando gli archi di E_b che escono dal vertice.

b. Dimostrate che con questo schema, se G non contiene cicli di peso negativo che sono raggiungibili dalla sorgente s, allora dopo soltanto $\lceil |V|/2 \rceil$ passaggi sugli archi, $v.d = \delta(s, v)$ per tutti i vertici $v \in V$.

c. Questo schema migliora il tempo di esecuzione asintotico dell'algoritmo di Bellman-Ford?

24-2 Annidamento di scatole

Una scatola d-dimensionale con dimensioni $(x_1, x_2, \ldots, x_d)$ ***si annida*** all'interno di un'altra scatola con dimensioni $(y_1, y_2, \ldots, y_d)$, se esiste una permutazione π su $\{1, 2, \ldots, d\}$ tale che $x_{\pi(1)} < y_1, x_{\pi(2)} < y_2, \ldots, x_{\pi(d)} < y_d$.

a. Dimostrate che la relazione di annidamento è transitiva.

b. Descrivete un metodo efficiente per determinare se una scatola d-dimensionale si annida all'interno di un'altra oppure no.

c. Supponete di avere un insieme di n scatole d-dimensionali $\{B_1, B_2, \ldots, B_n\}$. Descrivete un algoritmo efficiente per determinare la sequenza più lunga $\langle B_{i_1}, B_{i_2}, \ldots, B_{i_k} \rangle$ di scatole tale che B_{i_j} si annidi all'interno di $B_{i_{j+1}}$ per $j = 1, 2, \ldots, k-1$. Esprimete il tempo di esecuzione dell'algoritmo in funzione di n e d.

24-3 Arbitraggio

L'***arbitraggio*** consiste nell'utilizzare le differenze fra i tassi di cambio delle valute per trasformare una unità di una valuta in un po' più di una unità della stessa valuta. Per esempio, supponete che con un dollaro USA sia possibile acquistare 46,4 rupie indiane, che con una rupia sia possibile acquistare 2,5 yen giapponesi e che con uno yen sia possibile acquistare 0,0091 dollari USA. Allora, convertendo le valute, un commerciante può iniziare con un dollaro USA e acquistare $46{,}4 \times 2{,}5 \times 0{,}0091 = 1{,}0556$ dollari USA, realizzando un profitto del 5,56%.

Supponete di avere n valute $c_1, c_2, \ldots, c_n$ e una tabella R di tassi di scambio di dimensioni $n \times n$, tale che con una unità della valuta c_i sia possibile acquistare $R[i, j]$ unità della valuta c_j.

a. Create un algoritmo efficiente per determinare se esiste oppure no una sequenza di valute $\langle c_{i_1}, c_{i_2}, \ldots, c_{i_k} \rangle$ tale che

$$R[i_1, i_2] \cdot R[i_2, i_3] \cdots R[i_{k-1}, i_k] \cdot R[i_k, i_1] > 1$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Analizzate il tempo di esecuzione del vostro algoritmo.

b. Create un algoritmo efficiente per stampare una tale sequenza, se esiste. Analizzate il tempo di esecuzione del vostro algoritmo.

24-4 Algoritmo progressivo di Gabow per i cammini minimi da sorgente unica

Un ***algoritmo progressivo*** risolve un problema considerando inizialmente soltanto i bit più significativi di ciascun valore di input (per esempio, il peso di un arco). Poi affina la soluzione iniziale esaminando i due bit più significativi; prosegue esaminando un numero progressivamente crescente di bit più significativi, affinando ogni volta la soluzione, finché non avrà considerato tutti i bit e non avrà calcolato la soluzione esatta.

In questo problema descriveremo un algoritmo che calcola i cammini minimi da una sorgente unica esaminando progressivamente i pesi degli archi. Dato un grafo orientato $G = (V, E)$ con pesi d'arco w interi non negativi. Sia $W = \max_{(u,v)\in E}\{w(u, v)\}$. Il nostro obiettivo è sviluppare un algoritmo che viene eseguito nel tempo $O(E \lg W)$. Supponiamo che tutti i vertici siano raggiungibili dalla sorgente.

L'algoritmo esamina i bit nella rappresentazione binaria dei pesi degli archi uno alla volta, dal bit più significativo a quello meno significativo. Specificatamente, sia $k = \lceil \lg(W + 1) \rceil$ il numero di bit nella rappresentazione binaria di W e, per $i = 1, 2, \ldots, k$, sia $w_i(u, v) = \lfloor w(u, v)/2^{k-i} \rfloor$. Ovvero, $w_i(u, v)$ è la versione "in scala ridotta" di $w(u, v)$ data dagli i bit più significativi di $w(u, v)$ (quindi, $w_k(u, v) = w(u, v)$ per ogni $(u, v) \in E$). Per esempio, se $k = 5$ e $w(u, v) = 25$, che ha la rappresentazione binaria $\langle 11001 \rangle$, allora $w_3(u, v) = \langle 110 \rangle = 6$. Come altro esempio con $k = 5$, se $w(u, v) = \langle 00100 \rangle = 4$, allora $w_3(u, v) = \langle 001 \rangle = 1$. Definiamo $\delta_i(u, v)$ come il peso del cammino minimo dal vertice u al vertice v utilizzando la funzione peso w_i. Quindi, $\delta_k(u, v) = \delta(u, v)$ per ogni $u, v \in V$. Per un dato vertice sorgente s, l'algoritmo scalare prima calcola i pesi dei cammini minimi $\delta_1(s, v)$ per ogni $v \in V$, poi calcola $\delta_2(s, v)$ per ogni $v \in V$, e così via fino a calcolare $\delta_k(s, v)$ per ogni $v \in V$. Supponendo sempre che $|E| \geq |V| - 1$, vedremo che per calcolare δ_i da δ_{i-1} occorre un tempo $O(E)$, quindi l'intero algoritmo richiede un tempo $O(kE) = O(E \lg W)$.

a. Supponete che, per ogni vertice $v \in V$, si abbia $\delta(s, v) \leq |E|$. Dimostrate che è possibile calcolare $\delta(s, v)$ per ogni $v \in V$ nel tempo $O(E)$.

b. Dimostrate che è possibile calcolare $\delta_1(s, v)$ per ogni $v \in V$ nel tempo $O(E)$.

Adesso bisogna calcolare δ_i da δ_{i-1}.

c. Dimostrate che, per $i = 2, 3, \ldots, k$, si ha $w_i(u, v) = 2w_{i-1}(u, v)$ oppure $w_i(u, v) = 2w_{i-1}(u, v) + 1$. Poi, dimostrate che

$$2\delta_{i-1}(s, v) \leq \delta_i(s, v) \leq 2\delta_{i-1}(s, v) + |V| - 1$$

per ogni $v \in V$.

d. Per $i = 2, 3, \ldots, k$ e per ogni $(u, v) \in E$, definite

$$\widehat{w}_i(u, v) = w_i(u, v) + 2\delta_{i-1}(s, u) - 2\delta_{i-1}(s, v)$$

Dimostrate che, per $i = 2, 3, \ldots, k$ e per ogni $u, v \in V$, il valore "ripesato" $\widehat{w}_i(u, v)$ dell'arco (u, v) è un intero non negativo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 1995032016-220707-0. Copyright © 2022 McGraw-Hill Education (Italy)

e. Adesso definite $\widehat{\delta}_i(s, v)$ come il peso del cammino minimo da s a v utilizzando la funzione peso $\widehat{w}_i$. Dimostrate che per $i = 2, 3, \ldots, k$ e per ogni $v \in V$

$$\delta_i(s, v) = \widehat{\delta}_i(s, v) + 2\delta_{i-1}(s, v)$$

e che $\widehat{\delta}_i(s, v) \leq |E|$.

f. Spiegate come calcolare $\delta_i(s, v)$ da $\delta_{i-1}(s, v)$ per ogni $v \in V$ nel tempo $O(E)$ e concludete che $\delta(s, v)$ può essere calcolato per ogni $v \in V$ nel tempo $O(E \lg W)$.

24-5 Algoritmo di Karp per il ciclo di peso medio minimo

Sia $G = (V, E)$ un grafo orientato con funzione peso $w : E \to \mathbb{R}$ e sia $n = |V|$. Definiamo il ***peso medio*** di un ciclo $c = \langle e_1, e_2, \ldots, e_k \rangle$ di archi in E come

$$\mu(c) = \frac{1}{k} \sum_{i=1}^{k} w(e_i)$$

Sia $\mu^* = \min_c \mu(c)$, dove c varia su tutti i cicli orientati in G. Un ciclo c per il quale $\mu(c) = \mu^*$ è detto ***ciclo di peso medio minimo***. In questo problema esamineremo un algoritmo efficiente per calcolare μ^*.

Supponiamo, senza perdere in generalità, che ogni vertice $v \in V$ sia raggiungibile da un vertice sorgente $s \in V$. Sia $\delta(s, v)$ il peso di un cammino minimo da s a v e sia $\delta_k(s, v)$ il peso di un cammino minimo da s a v formato *esattamente* da k archi. Se non c'è un cammino da s a v con esattamente k archi, allora $\delta_k(s, v) = \infty$.

a. Dimostrate che, se $\mu^* = 0$, allora G non contiene cicli di peso negativo e $\delta(s, v) = \min_{0 \le k \le n-1} \delta_k(s, v)$ per tutti i vertici $v \in V$.

b. Dimostrate che, se $\mu^* = 0$, allora

$$\max_{0 \le k \le n-1} \frac{\delta_n(s, v) - \delta_k(s, v)}{n - k} \ge 0$$

per tutti i vertici $v \in V$ (*suggerimento:* applicate entrambe le proprietà del punto (a)).

c. Sia c un ciclo di peso 0 e siano u e v due vertici qualsiasi di c. Supponete che $\mu^* = 0$ e che il peso del cammino semplice da u a v lungo il ciclo sia x. Dimostrate che $\delta(s, v) = \delta(s, u) + x$ (*suggerimento:* il peso del cammino da v a u lungo il ciclo è $-x$).

d. Dimostrate che, se $\mu^* = 0$, allora in ogni ciclo di peso medio minimo c'è un vertice v tale che

$$\max_{0 \le k \le n-1} \frac{\delta_n(s, v) - \delta_k(s, v)}{n - k} = 0$$

(*Suggerimento:* dimostrate che un cammino minimo che arriva in un vertice qualsiasi di un ciclo di peso medio minimo può essere esteso lungo il ciclo per creare un cammino minimo che arriva nel vertice successivo del ciclo.)

Acquistato da Michele Michele Sarrebeter (1622-01-07 12:12:12 Numero Ordine Libreria 1000000000220781) Copyright © 2022, McGraw-Hill Education (Italy)

e. Dimostrate che, se $\mu^* = 0$, allora

$$\min_{v \in V} \max_{0 \le k \le n-1} \frac{\delta_n(s, v) - \delta_k(s, v)}{n - k} = 0$$

f. Dimostrate che, se si aggiunge una costante t al peso di ogni arco di G, allora μ^* aumenta di t. Utilizzate questo fatto per provare che

$$\mu^* = \min_{v \in V} \max_{0 \le k \le n-1} \frac{\delta_n(s, v) - \delta_k(s, v)}{n - k}$$

g. Descrivete un algoritmo con tempo $O(VE)$ per calcolare μ^*.

24-6 Cammini minimi bitonici

Una sequenza è ***bitonica*** se cresce monotonicamente e poi diminuisce monotonicamente oppure se, dopo uno scorrimento circolare, cresce monotonicamente e poi diminuisce monotonicamente. Per esempio, le sequenze $\langle 1, 4, 6, 8, 3, -2 \rangle$, $\langle 9, 2, -4, -10, -5 \rangle$ e $\langle 1, 2, 3, 4 \rangle$ sono bitoniche, mentre la sequenza $\langle 1, 3, 12, 4, 2, 10 \rangle$ non è bitonica (consultate il Problema 15-3 del commesso viaggiatore euclideo e bitonico.) Supponiamo di avere un grafo orientato $G = (V, E)$ con funzione peso $w : E \to \mathbb{R}$, dove tutti i pesi degli archi sono unici; vogliamo trovare i cammini minimi dalla sorgente unica s. Abbiamo un'ulteriore informazione: per ogni vertice $v \in V$, i pesi degli archi lungo un cammino minimo qualsiasi da s a v formano una sequenza bitonica.

Create l'algoritmo più efficiente possibile per risolvere questo problema e analizzate il suo tempo di esecuzione.

Note

L'algoritmo di Dijkstra [89] apparve nel 1959, ma non faceva alcun cenno alle code di priorità. L'algoritmo di Bellman-Ford è basato su due algoritmi distinti: quello di Bellman [38] e quello di Ford [110]. Bellman descrive la relazione fra i cammini minimi e i vincoli sulle differenze. Lawler [225] descrive l'algoritmo con tempo lineare per i cammini minimi in un grafo orientato aciclico, che egli considera parte del folclore.

Quando i pesi degli archi sono numeri interi relativamente piccoli, è possibile utilizzare algoritmi più efficienti per risolvere il problema dei cammini minimi da sorgente unica. La sequenza dei valori restituiti dalle chiamate di EXTRACT-MIN nell'algoritmo di Dijkstra è monotonicamente crescente nel tempo. Come descritto nelle note in fondo al Capitolo 6, in questo caso ci sono diverse strutture dati che possono implementare le varie operazioni delle code di priorità in modo più efficiente di un heap binario o di un heap di Fibonacci. Ahuja, Mehlhorn, Orlin e Tarjan [8] hanno sviluppato un algoritmo che viene eseguito nel tempo $O(E + V\sqrt{\lg W})$ su grafi con archi di peso non negativo, dove W è il massimo fra i pesi degli archi del grafo. I limiti migliori sono quelli di Thorup [338], che ha creato un algoritmo con tempo $O(E \lg \lg V)$, e di Raman [292], che ha creato un algoritmo con tempo $O\left(E + V \min\left\{(\lg V)^{1/3+\epsilon}, (\lg W)^{1/4+\epsilon}\right\}\right)$. Questi due algoritmi usano una quantità di memoria che dipende dalla dimensione della parola del calcolatore in cui vengono eseguiti. Sebbene la quantità di memoria utilizzata possa essere illimitata nella dimensione dell'input, essa può essere ridotta a essere lineare nella dimensione dell'input applicando l'hashing randomizzato.

Per i grafi non orientati con pesi interi, Thorup [337] ha creato un algoritmo con tempo $O(V + E)$ per trovare i cammini minimi da sorgente unica. Diversamente dagli algoritmi precedentemente citati, questo algoritmo non è una implementazione dell'algoritmo di Dijkstra, in quanto la sequenza dei valori restituiti dalle chiamate di EXTRACT-MIN non è monotonicamente crescente nel tempo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Per i grafi con archi di peso negativo, un algoritmo creato da Gabow e Tarjan [123] viene eseguito nel tempo $O(\sqrt{V}E \lg(VW))$ e uno sviluppato da Goldberg [138] viene eseguito nel tempo $O(\sqrt{V}E \lg W)$, dove $W = \max_{(u,v) \in E} \{|w(u, v)|\}$.

Cherkassky, Goldberg e Radzik [65] hanno effettuato numerosi esperimenti per confrontare vari algoritmi per cammini minimi.

Cammini minimi fra tutte le coppie 25

25.1 Introduzione

In questo capitolo analizzeremo il problema di trovare i cammini minimi fra tutte le coppie di vertici in un grafo. Questo problema si presenta, per esempio, quando si vuole creare la tabella con le distanze fra le varie coppie delle città di un atlante stradale. Come nel Capitolo 24, è dato un grafo orientato pesato $G = (V, E)$ con una funzione peso $w : E \to \mathbb{R}$ che associa agli archi dei pesi di valore reale. Vogliamo trovare, per ogni coppia di vertici $u, v \in V$, un cammino (di peso) minimo da u a v, dove il peso di un cammino è la somma dei pesi degli archi che lo compongono. Tipicamente, vogliamo ottenere l'output in forma tabulare: l'elemento nella riga u e nella colonna v rappresenta il peso di un cammino minimo dal vertice u al vertice v.

Per risolvere un problema di cammini minimi fra tutte le coppie, possiamo eseguire un algoritmo per cammini minimi da sorgente unica $|V|$ volte, una volta per ogni vertice, che è utilizzato come sorgente. Se i pesi degli archi sono tutti non negativi, possiamo utilizzare l'algoritmo di Dijkstra. Se implementiamo la coda di min-priorità con un array lineare, il tempo di esecuzione è $O(V^3 + VE) = O(V^3)$. L'implementazione della coda di min-priorità con un min-heap binario consente di ottenere un tempo di esecuzione pari a $O(VE \lg V)$, che è un miglioramento se il grafo è sparso. In alternativa, possiamo implementare la coda di min-priorità con un heap di Fibonacci, ottenendo un tempo di esecuzione pari a $O(V^2 \lg V + VE)$.

Se il grafo ha archi di peso negativo, l'algoritmo di Dijkstra non può essere utilizzato. In questo caso, bisogna eseguire l'algoritmo più lento di Bellman-Ford, una volta per ogni vertice. Il tempo di esecuzione risultante è $O(V^2E)$, che per un grafo denso è $O(V^4)$. In questo capitolo vedremo che è possibile fare meglio. Esamineremo anche la relazione tra il problema dei cammini minimi fra tutte le coppie e la moltiplicazione di matrici e ne studieremo la struttura algebrica.

Diversamente dagli algoritmi per sorgente unica, che presuppongono che il grafo sia rappresentato con le liste di adiacenza, la maggior parte degli algoritmi in questo capitolo usa una rappresentazione con la matrice di adiacenza (l'algoritmo di Johnson per i grafi sparsi, nel Paragrafo 25.4, usa le liste di adiacenza). Per comodità, supponiamo che i vertici abbiano la numerazione $1, 2, \ldots, |V|$; in questo modo, l'input è una matrice W di dimensioni $n \times n$ che rappresenta i pesi degli archi di un grafo orientato $G = (V, E)$ di n vertici. Ovvero $W = (w_{ij})$, dove

Acquistato da Michele Michele su Webster il 2022-07-04 23:12 Numero Ordine Libreria: 1995050615220704 © Copyright © 2021, McGraw-Hill Education (Italy)

$$w_{ij} = \begin{cases} 0 & \text{se } i = j \\ \text{il peso dell'arco orientato } (i, j) & \text{se } i \neq j \text{ e } (i, j) \in E \\ \infty & \text{se } i \neq j \text{ e } (i, j) \notin E \end{cases} \tag{25.1}$$

Sono ammessi gli archi di peso negativo, ma per adesso supponiamo che il grafo di input non contenga cicli di peso negativo. L'output di forma tabulare degli algoritmi per cammini minimi fra tutte le coppie presentato in questo capitolo è una matrice $D = (d_{ij})$, di dimensioni $n \times n$, dove l'elemento d_{ij} contiene il peso di un cammino minimo dal vertice i al vertice j. Ovvero, se indichiamo con $\delta(i, j)$ il peso del cammino minimo dal vertice i al vertice j (come nel Capitolo 24), allora alla fine si avrà $d_{ij} = \delta(i, j)$.

Per risolvere il problema dei cammini minimi fra tutte le coppie su una matrice di adiacenza di input, bisogna calcolare non soltanto i pesi dei cammini minimi, ma anche la ***matrice dei predecessori*** $\Pi = (\pi_{ij})$, dove π_{ij} è NIL se $i = j$ o se non esiste un cammino da i a j, altrimenti π_{ij} è il predecessore di j in qualche cammino minimo da i. Proprio come il sottografo dei predecessori G_π (descritto nel Capitolo 24) è un albero di cammini minimi per un dato vertice sorgente, così il sottografo indotto dalla i-esima riga della matrice Π sarà un albero di cammini minimi con radice i. Per ogni vertice $i \in V$, definiamo il ***sottografo dei predecessori*** di G per i come $G_{\pi,i} = (V_{\pi,i}, E_{\pi,i})$, dove

$$V_{\pi,i} = \{j \in V : \pi_{ij} \neq \text{NIL}\} \cup \{i\}$$

e

$$E_{\pi,i} = \{(\pi_{ij}, j) : j \in V_{\pi,i} - \{i\}\}$$

Se $G_{\pi,i}$ è un albero di cammini minimi, allora la seguente procedura, che è una versione modificata della procedura PRINT-PATH descritta nel Capitolo 22, stampa un cammino minimo dal vertice i al vertice j.

```
PRINT-ALL-PAIRS-SHORTEST-PATH(Π, i, j)
1  if i == j
2      stampa i
3  elseif π_ij == NIL
4      stampa "non esiste un cammino da" i "a" j
5  else PRINT-ALL-PAIRS-SHORTEST-PATH(Π, i, π_ij)
6      stampa j
```

Al fine di mettere in risalto le caratteristiche essenziali degli algoritmi per i cammini minimi fra tutte le coppie, in questo capitolo non tratteremo la creazione e le proprietà delle matrici dei predecessori così approfonditamente come abbiamo fatto con i sottografi dei predecessori nel Capitolo 24. I concetti fondamentali sono trattati in alcuni esercizi.

Organizzazione del capitolo

Il Paragrafo 25.2 presenta un algoritmo di programmazione dinamica che si basa sulla moltiplicazione delle matrici per risolvere il problema dei cammini minimi fra tutte le coppie. Applicando la tecnica dell'*elevazione al quadrato ripetuta*, questo algoritmo può essere eseguito nel tempo $\Theta(V^3 \lg V)$. Nel Paragrafo 25.3 è descritto un altro algoritmo di programmazione dinamica: l'algoritmo di Floyd-Warshall, che viene eseguito nel tempo $\Theta(V^3)$. Il Paragrafo 25.3 tratta anche il problema di trovare la chiusura transitiva di un grafo orientato, che è in relazione con il problema dei cammini minimi fra tutte le coppie. Infine, il Paragrafo 25.4

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria 189503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

presenta l'algoritmo di Johnson. Diversamente dagli altri algoritmi descritti in questo capitolo, l'algoritmo di Johnson usa la rappresentazione con liste di adiacenza di un grafo. Risolve il problema dei cammini minimi fra tutte le coppie nel tempo $O(V^2 \lg V + VE)$, e questo lo rende un buon algoritmo per i grafi sparsi molto grandi.

Prima di continuare, dobbiamo stabilire alcune convenzioni per le rappresentazioni con le matrici di adiacenza. In primo luogo, assumeremo che in generale il grafo di input $G = (V, E)$ abbia n vertici, quindi $n = |V|$. In secondo luogo, adotteremo la convenzione di indicare le matrici con lettere maiuscole, come W, L o D, e i loro elementi con lettere minuscole con pedici, come w_{ij}, l_{ij} o d_{ij}. Per indicare le iterazioni, alcune matrici hanno gli apici fra parentesi, come $L^{(m)} = \left(l_{ij}^{(m)}\right)$ o $D^{(m)} = \left(d_{ij}^{(m)}\right)$. Infine, se A è una matrice $n \times n$, assumeremo che il valore di n sia memorizzato nell'attributo $A.rows$.

25.2 Cammini minimi e moltiplicazione di matrici

Questo paragrafo presenta un algoritmo di programmazione dinamica per il problema dei cammini minimi fra tutte le coppie di vertici in un grafo orientato $G = (V, E)$. Ogni volta che viene eseguito il ciclo principale del programma dinamico viene chiamata un'operazione che è molto simile alla moltiplicazione delle matrici, quindi l'algoritmo simula la ripetizione del prodotto fra matrici. Inizieremo sviluppando un algoritmo con tempo $\Theta(V^4)$ per il problema dei cammini minimi fra tutte le coppie e poi miglioreremo il suo tempo di esecuzione a $\Theta(V^3 \lg V)$.

Prima di procedere, riassumiamo brevemente le fasi descritte nel Capitolo 15 per sviluppare un algoritmo di programmazione dinamica.

1. Caratterizzare la struttura di una soluzione ottima.
2. Definire in modo ricorsivo il valore di una soluzione ottima.
3. Calcolare il valore di una soluzione ottima secondo uno schema bottom-up (dal basso verso l'alto).

La quarta fase – costruire una soluzione ottima dalle informazioni calcolate – sarà trattata negli esercizi.

La struttura di un cammino minimo

Iniziamo caratterizzando la struttura di una soluzione ottima. Per il problema dei cammini minimi fra tutte le coppie in un grafo $G = (V, E)$, abbiamo dimostrato (Lemma 24.1) che tutti i sottocammini di un cammino minimo sono cammini minimi. Supponiamo che il grafo sia rappresentato da una matrice di adiacenza $W = (w_{ij})$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Consideriamo un cammino minimo p dal vertice i al vertice j e supponiamo che p contenga al più m archi. Supponendo che non ci siano cicli di peso negativo, m è un valore finito. Se $i = j$, allora p ha peso 0 e non ha archi. Se i vertici i e j sono distinti, allora scomponiamo il cammino p in $i \overset{p'}{\rightsquigarrow} k \rightarrow j$, dove il cammino p' adesso contiene al più $m-1$ archi. Per il Lemma 24.1, p' è un cammino minimo da i a k, quindi $\delta(i, j) = \delta(i, k) + w_{kj}$.

Una soluzione ricorsiva per il problema dei cammini minimi fra tutte le coppie

Adesso, sia $l_{ij}^{(m)}$ il peso minimo di un cammino qualsiasi dal vertice i al vertice j che contiene al più m archi. Quando $m = 0$, esiste un cammino minimo da i a j senza archi, se e soltanto se $i = j$; quindi

$$l_{ij}^{(0)} = \begin{cases} 0 & \text{se } i = j \\ \infty & \text{se } i \neq j \end{cases}$$

Per $m \geq 1$, calcoliamo $l_{ij}^{(m)}$ come il minimo tra $l_{ij}^{(m-1)}$ (il peso del cammino minimo da i a j che è formato al più da $m-1$ archi) e il peso minimo di un cammino qualsiasi da i a j che è formato al più da m archi, che si ottiene considerando tutti i possibili predecessori k di j. Quindi, la definizione ricorsiva è la seguente:

$$\begin{aligned} l_{ij}^{(m)} &= \min\left(l_{ij}^{(m-1)}, \min_{1 \leq k \leq n}\left\{l_{ik}^{(m-1)} + w_{kj}\right\}\right) \\ &= \min_{1 \leq k \leq n}\left\{l_{ik}^{(m-1)} + w_{kj}\right\} \end{aligned} \tag{25.2}$$

L'ultima uguaglianza deriva dal fatto che $w_{jj} = 0$ per ogni j.

Quali sono i pesi effettivi dei cammini minimi $\delta(i, j)$? Se il grafo non contiene cicli di peso negativo, allora per ogni coppia di vertici i e j per cui $\delta(i, j) < \infty$, c'è un cammino minimo da i a j che è semplice e quindi contiene al più $n-1$ archi. Un cammino dal vertice i al vertice j con più di $n-1$ archi non può avere un peso minore di un cammino minimo da i a j. Pertanto i pesi effettivi dei cammini minimi sono dati da

$$\delta(i, j) = l_{ij}^{(n-1)} = l_{ij}^{(n)} = l_{ij}^{(n+1)} = \cdots \tag{25.3}$$

Calcolo dei pesi dei cammini minimi secondo lo schema bottom-up

Prendendo come input la matrice $W = (w_{ij})$, calcoliamo ora una serie di matrici $L^{(1)}, L^{(2)}, \ldots, L^{(n-1)}$, dove $L^{(m)} = \left(l_{ij}^{(m)}\right)$ per $m = 1, 2, \ldots, n-1$. La matrice finale $L^{(n-1)}$ contiene i pesi effettivi dei cammini minimi. Notate che $l_{ij}^{(1)} = w_{ij}$ per tutti i vertici $i, j \in V$, quindi $L^{(1)} = W$.

Il cuore dell'algoritmo è la seguente procedura che, date le matrici $L^{(m-1)}$ e W, restituisce la matrice $L^{(m)}$; ovvero la procedura aggiunge un arco ai cammini minimi calcolati fino a quel momento.

```
EXTEND-SHORTEST-PATHS(L, W)
1  n = L.rows
2  Sia L' = (l'_ij) una nuova matrice n × n
3  for i = 1 to n
4      for j = 1 to n
5          l'_ij = ∞
6          for k = 1 to n
7              l'_ij = min(l'_ij, l_ik + w_kj)
8  return L'
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

La procedura calcola una matrice $L' = (l'_{ij})$ e la restituisce alla fine. Ciò viene fatto calcolando l'equazione (25.2) per ogni i e j, utilizzando L al posto di $L^{(m-1)}$ e L' al posto di $L^{(m)}$ (abbiamo omesso gli apici per rendere le matrici di input e output indipendenti da m). Il suo tempo di esecuzione è $\Theta(n^3)$ a causa dei tre cicli **for** annidati.

Adesso possiamo esaminare la relazione con la moltiplicazione di matrici. Supponiamo di volere calcolare il prodotto $C = A \cdot B$ delle due matrici A e B di dimensioni $n \times n$. Allora, per $i, j = 1, 2, \ldots, n$, calcoliamo

$$c_{ij} = \sum_{k=1}^{n} a_{ik} \cdot b_{kj} \tag{25.4}$$

Notate che se effettuiamo le sostituzioni

$$\begin{array}{rcl} l^{(m-1)} & \to & a \\ w & \to & b \\ l^{(m)} & \to & c \\ \min & \to & + \\ + & \to & \cdot \end{array}$$

nell'equazione (25.2), otteniamo l'equazione (25.4). Quindi, se apportiamo queste modifiche alla procedura EXTEND-SHORTEST-PATHS e sostituiamo ∞ (l'identità per min) con 0 (l'identità per $+$), otteniamo la stessa procedura con tempo $\Theta(n^3)$ per moltiplicare le matrici che abbiamo visto nel Paragrafo 4.2:

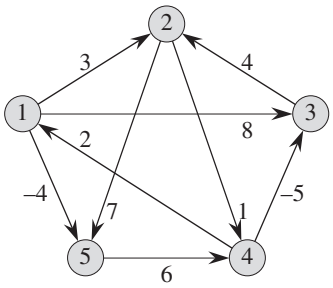
```
SQUARE-MATRIX-MULTIPLY(A, B)
1  n = A.rows
2  Sia C una nuova matrice n × n
3  for i = 1 to n
4      for j = 1 to n
5          c_ij = 0
6          for k = 1 to n
7              c_ij = c_ij + a_ik · b_kj
8  return C
```

Ritornando al problema dei cammini minimi fra tutte le coppie, calcoliamo i pesi dei cammini minimi estendendo i cammini di un arco alla volta. Indicando con $A \cdot B$ il "prodotto" delle matrici restituito da EXTEND-SHORTEST-PATHS(A, B), calcoliamo la sequenza di $n - 1$ matrici

$$\begin{array}{lcl} L^{(1)} & = & L^{(0)} \cdot W = W \\ L^{(2)} & = & L^{(1)} \cdot W = W^2 \\ L^{(3)} & = & L^{(2)} \cdot W = W^3 \\ & \vdots & \\ L^{(n-1)} & = & L^{(n-2)} \cdot W = W^{n-1} \end{array}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Come detto in precedenza, la matrice $L^{(n-1)} = W^{n-1}$ contiene i pesi dei cammini minimi. La seguente procedura calcola questa sequenza nel tempo $\Theta(n^4)$.



$$L^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad L^{(2)} = \begin{pmatrix} 0 & 3 & 8 & 2 & -4 \\ 3 & 0 & -4 & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & \infty & 1 & 6 & 0 \end{pmatrix}$$

$$L^{(3)} = \begin{pmatrix} 0 & 3 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} \quad L^{(4)} = \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix}$$

Figura 25.1 Un grafo orientato e la sequenza delle matrici $L^{(m)}$ calcolate da SLOW-ALL-PAIRS-SHORTEST-PATHS. Il lettore può verificare che $L^{(5)} = L^{(4)} \cdot W$ è uguale a $L^{(4)}$, quindi $L^{(m)} = L^{(4)}$ per ogni $m \geq 4$.

SLOW-ALL-PAIRS-SHORTEST-PATHS(W)

```
1  n = W.rows
2  L^(1) = W
3  for m = 2 to n − 1
4       Sia L^(m) una nuova matrice n × n
5       L^(m) = EXTEND-SHORTEST-PATHS(L^(m−1), W)
6  return L^(n−1)
```

La Figura 25.1 illustra un grafo e le matrici $L^{(m)}$ calcolate dalla procedura SLOW-ALL-PAIRS-SHORTEST-PATHS.

Migliorare il tempo di esecuzione

Il nostro obiettivo, tuttavia, non è calcolare *tutte* le matrici $L^{(m)}$: siamo interessati soltanto alla matrice $L^{(n-1)}$. Ricordiamo che, in assenza di cicli di peso negativo, l'equazione (25.3) implica $L^{(m)} = L^{(n-1)}$ per ogni intero $m \geq n-1$. Come la classica moltiplicazione di matrici, anche la moltiplicazione definita dalla procedura EXTEND-SHORTEST-PATHS è associativa (vedere l'Esercizio 25.2-4). Quindi, possiamo calcolare $L^{(n-1)}$ con soli $\lceil \lg(n-1) \rceil$ prodotti di matrici, calcolando la sequenza

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 (Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned} L^{(1)} &= W^1 = W \\ L^{(2)} &= W^2 = W \cdot W \\ L^{(4)} &= W^4 = W^2 \cdot W^2 \\ L^{(8)} &= W^8 = W^4 \cdot W^4 \\ &\vdots \\ L^{(2^{\lceil \lg(n-1) \rceil})} &= W^{2^{\lceil \lg(n-1) \rceil}} = W^{2^{\lceil \lg(n-1) \rceil - 1}} \cdot W^{2^{\lceil \lg(n-1) \rceil - 1}} \end{aligned}$$

Poiché $2^{\lceil \lg(n-1) \rceil} \geq n-1$, il prodotto finale $L^{(2^{\lceil \lg(n-1) \rceil})}$ è uguale a $L^{(n-1)}$.

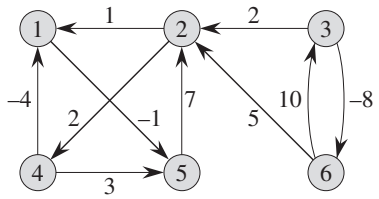


Figura 25.2 Il grafo orientato pesato da utilizzare negli Esercizi 25.2-1, 25.3-1 e 25.4-1.

La seguente procedura calcola la precedente sequenza di matrici applicando questa tecnica dell'***elevazione al quadrato ripetuta***.

```
FASTER-ALL-PAIRS-SHORTEST-PATHS(W)
1  n = W.rows
2  L^(1) = W
3  m = 1
4  while m < n − 1
5      Sia L^(2m) una nuova matrice n × n
6      L^(2m) = EXTEND-SHORTEST-PATHS(L^(m), L^(m))
7      m = 2m
8  return L^(m)
```

In ogni iterazione del ciclo **while** delle righe 4–7, calcoliamo $L^{(2m)} = \left(L^{(m)}\right)^2$, iniziando con $m = 1$. Alla fine di ogni iterazione, raddoppiamo il valore di m. L'ultima iterazione calcola $L^{(n-1)}$, calcolando in effetti $L^{(2m)}$ per qualche $n - 1 \leq 2m < 2n - 2$. Per l'equazione (25.3), si ha $L^{(2m)} = L^{(n-1)}$. La prossima volta che viene eseguito il test nella riga 4, il valore di m è raddoppiato e allora $m \geq n - 1$, quindi il test fallisce e la procedura restituisce l'ultima matrice calcolata.

Il tempo di esecuzione di FASTER-ALL-PAIRS-SHORTEST-PATHS è $\Theta(n^3 \lg n)$ perché ciascuno dei $\lceil \lg(n-1) \rceil$ prodotti di matrici richiede un tempo $\Theta(n^3)$. Notate che il codice è compatto, perché non contiene strutture dati complesse, quindi la costante nascosta nella notazione Θ è piccola.

Esercizi

25.2-1

Eseguite la procedura SLOW-ALL-PAIRS-SHORTEST-PATHS sul grafo orientato pesato della Figura 25.2, indicando le matrici che si ottengono per ogni iterazione del ciclo. Fate lo stesso con la procedura FASTER-ALL-PAIRS-SHORTEST-PATHS.

25.2-2

Perché si richiede che $w_{ii} = 0$ per ogni $1 \leq i \leq n$?

25.2-3

Nella normale moltiplicazione di matrici, a che cosa corrisponde la matrice

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$L^{(0)} = \begin{pmatrix} 0 & \infty & \infty & \cdots & \infty \\ \infty & 0 & \infty & \cdots & \infty \\ \infty & \infty & 0 & \cdots & \infty \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \infty & \infty & \infty & \cdots & 0 \end{pmatrix}$$

utilizzata negli algoritmi per i cammini minimi?

25.2-4
Dimostrate che la moltiplicazione delle matrici definita dalla procedura EXTEND-SHORTEST-PATHS è associativa.

25.2-5
Esprimete il problema dei cammini minimi da sorgente unica come un prodotto tra matrici e un vettore. Spiegate come la valutazione di questo prodotto corrisponda a un algoritmo simile a quello di Bellman-Ford (vedere il Paragrafo 24.2).

25.2-6
Supponete di volere calcolare i vertici dei cammini minimi negli algoritmi di questo paragrafo. Spiegate come calcolare la matrice dei predecessori Π dalla matrice completa L dei pesi dei cammini minimi nel tempo $O(n^3)$.

25.2-7
I vertici dei cammini minimi possono essere calcolati anche contemporaneamente ai pesi. Definite $\pi_{ij}^{(m)}$ come il predecessore del vertice j in un cammino di peso minimo da i a j che contiene al più m archi. Modificate EXTEND-SHORTEST-PATHS e SLOW-ALL-PAIRS-SHORTEST-PATHS per calcolare le matrici $\Pi^{(1)}$, $\Pi^{(2)}, \ldots, \Pi^{(n-1)}$ mentre vengono calcolate le matrici $L^{(1)}, L^{(2)}, \ldots, L^{(n-1)}$.

25.2-8
La procedura FASTER-ALL-PAIRS-SHORTEST-PATHS, così com'è scritta, richiede che siano memorizzate $\lceil \lg(n-1) \rceil$ matrici, ciascuna con n^2 elementi, per uno spazio totale in memoria pari a $\Theta(n^2 \lg n)$. Modificate la procedura in modo che lo spazio in memoria sia ridotto a $\Theta(n^2)$, utilizzando soltanto due matrici $n \times n$.

25.2-9
Modificate la procedura FASTER-ALL-PAIRS-SHORTEST-PATHS in modo che possa rilevare la presenza di un ciclo di peso negativo.

25.2-10
Create un algoritmo efficiente per trovare la lunghezza (numero di archi) di un ciclo di peso negativo di lunghezza minima all'interno di un grafo.

25.3 Algoritmo di Floyd-Warshall

In questo paragrafo utilizzeremo una formulazione differente della programmazione dinamica per risolvere il problema dei cammini minimi fra tutte le coppie in un grafo orientato $G = (V, E)$. L'algoritmo risultante, noto come ***algoritmo di Floyd-Warshall***, viene eseguito nel tempo $\Theta(V^3)$. Come prima, possono essere presenti archi di peso negativo, ma si suppone che non ci siano cicli di peso negativo. Come nel Paragrafo 25.2, seguiremo il processo della programmazione dinamica per sviluppare l'algoritmo. Dopo avere studiato l'algoritmo risultante, presenteremo un metodo analogo per trovare la chiusura transitiva di un grafo orientato.

Acquistato da Michele Michele su Webster il 2022-07-07 28:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

La struttura di un cammino minimo

Nell'algoritmo di Floyd-Warshall utilizziamo una caratterizzazione della struttura di un cammino minimo diversa da quella utilizzata nel Paragrafo 25.2. L'algoritmo considera i vertici "intermedi" di un cammino minimo, dove un ***vertice intermedio***

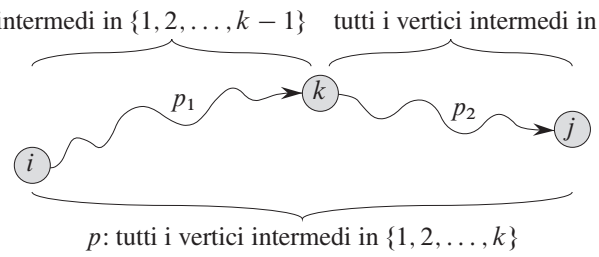


Figura 25.3 Il cammino p è un cammino minimo dal vertice i al vertice j; k è il vertice intermedio di p con la numerazione più grande. Il cammino p_1, la porzione del cammino p dal vertice i al vertice k, ha tutti i vertici intermedi nell'insieme $\{1, 2, \ldots, k-1\}$. Lo stesso vale per il cammino p_2 dal vertice k al vertice j.

di un cammino semplice $p = \langle v_1, v_2, \ldots, v_l \rangle$ è un vertice qualsiasi di p diverso da v_1 e v_l, ovvero un vertice qualsiasi dell'insieme $\{v_2, v_3, \ldots, v_{l-1}\}$.

L'algoritmo di Floyd-Warshall si basa sulla seguente osservazione. Supponendo che i vertici di G siano $V = \{1, 2, \ldots, n\}$, consideriamo un sottoinsieme $\{1, 2, \ldots, k\}$ di vertici per un generico k. Per una coppia qualsiasi di vertici $i, j \in V$, consideriamo tutti i cammini da i a j i cui vertici intermedi sono tutti in $\{1, 2, \ldots, k\}$ e sia p un cammino di peso minimo fra di essi (il cammino p è semplice). L'algoritmo di Floyd-Warshall sfrutta una relazione fra il cammino p e i cammini minimi da i a j i cui i vertici intermedi appartengono tutti all'insieme $\{1, 2, \ldots, k-1\}$. Tale relazione dipende dal fatto che k sia un vertice intermedio del cammino p oppure no.

- Se k non è un vertice intermedio del cammino p, allora tutti i vertici intermedi del cammino p sono nell'insieme $\{1, 2, \ldots, k-1\}$. Quindi, un cammino minimo dal vertice i al vertice j con tutti i vertici intermedi nell'insieme $\{1, 2, \ldots, k-1\}$ è anche un cammino minimo da i a j con tutti i vertici intermedi nell'insieme $\{1, 2, \ldots, k\}$.
- Se k è un vertice intermedio del cammino p, allora spezziamo p in $i \overset{p_1}{\rightsquigarrow} k \overset{p_2}{\rightsquigarrow} j$ come illustra la Figura 25.3. Per il Lemma 24.1, p_1 è un cammino minimo da i a k con tutti i vertici intermedi nell'insieme $\{1, 2, \ldots, k\}$. Poiché il vertice k non è un vertice intermedio del cammino p_1, allora p_1 è un cammino minimo da i a k con tutti i vertici intermedi nell'insieme $\{1, 2, \ldots, k-1\}$. Analogamente, p_2 è un cammino minimo dal vertice k al vertice j con tutti i vertici intermedi nell'insieme $\{1, 2, \ldots, k-1\}$.

Una soluzione ricorsiva per il problema dei cammini minimi fra tutte le coppie

Sulla base delle precedenti osservazioni, formuliamo una definizione ricorsiva delle stime dei cammini minimi che è diversa da quella del Paragrafo 25.2. Sia $d_{ij}^{(k)}$ il peso di un cammino minimo dal vertice i al vertice j, i cui vertici intermedi si trovano tutti nell'insieme $\{1, 2, \ldots, k\}$. Se $k = 0$, un cammino dal vertice i al vertice j, che non include vertici intermedi con numerazione maggiore di 0, non ha alcun vertice intermedio. Un tale cammino ha al massimo un arco, e quindi $d_{ij}^{(0)} = w_{ij}$. Una definizione ricorsiva basata sulla precedente considerazione è questa:

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950301 6-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{se } k = 0 \\ \min\left(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\right) & \text{se } k \geq 1 \end{cases} \tag{25.5}$$

Poiché per un cammino qualsiasi tutti i vertici intermedi si trovano nell'insieme $\{1, 2, \ldots, n\}$, la matrice $D^{(n)} = \left(d_{ij}^{(n)}\right)$ fornisce la soluzione finale: $d_{ij}^{(n)} = \delta(i, j)$ per ogni $i, j \in V$.

Calcolo dei pesi dei cammini minimi secondo lo schema bottom-up

Applicando la ricorrenza (25.5), la seguente procedura bottom-up può essere utilizzata per calcolare i valori $d_{ij}^{(k)}$ per valori crescenti di k. Il suo input è una matrice W $n \times n$ definita secondo l'equazione (25.1). La procedura restituisce la matrice $D^{(n)}$ dei pesi dei cammini minimi.

```
FLOYD-WARSHALL(W)
1  n = W.rows
2  D^(0) = W
3  for k = 1 to n
4      Sia D^(k) = (d_ij^(k)) una nuova matrice n × n
5      for i = 1 to n
6          for j = 1 to n
7              d_ij^(k) = min(d_ij^(k-1), d_ik^(k-1) + d_kj^(k-1))
8  return D^(n)
```

La Figura 25.4 illustra le matrici $D^{(k)}$ calcolate dall'algoritmo di Floyd-Warshall per il grafo della Figura 25.1.

Il tempo di esecuzione dell'algoritmo di Floyd-Warshall è determinato dai tre cicli **for** annidati nelle righe 3–7. Poiché ogni esecuzione della riga 7 richiede un tempo $O(1)$, l'algoritmo viene eseguito nel tempo $\Theta(n^3)$. Come nell'ultimo algoritmo del Paragrafo 25.2, il codice è compatto, perché è privo di strutture dati complesse, e quindi la costante nascosta nella notazione Θ è piccola. Ne consegue che l'algoritmo di Floyd-Warshall è molto utile anche per grafi di input di discrete dimensioni.

Costruzione di un cammino minimo

Ci sono vari metodi per costruire i cammini minimi nell'algoritmo di Floyd-Warshall. Uno consiste nel calcolare la matrice D dei pesi dei cammini minimi e poi nel costruire la matrice dei predecessori Π dalla matrice D. Questo metodo può essere implementato con un tempo di esecuzione pari a $O(n^3)$ (Esercizio 25.2-6). Data la matrice dei predecessori Π, la procedura PRINT-ALL-PAIRS-SHORTEST-PATH può essere utilizzata per stampare i vertici di un determinato cammino minimo.

Possiamo anche calcolare la matrice dei predecessori Π mano a mano che l'algoritmo calcola le matrici $D^{(k)}$. Specificatamente, calcoliamo una sequenza di matrici $\Pi^{(0)}, \Pi^{(1)}, \ldots, \Pi^{(n)}$, dove $\Pi = \Pi^{(n)}$ e $\pi_{ij}^{(k)}$ è definito come il predecessore del vertice j in un cammino minimo dal vertice i, i cui vertici intermedi si trovano tutti nell'insieme $\{1, 2, \ldots, k\}$.

Acquistato da Michele Michele su Webster il 2022-[illegible] Vietata la riproduzione non autorizzata

Possiamo formulare una definizione ricorsiva di $\pi_{ij}^{(k)}$. Quando $k = 0$, un cammino minimo da i a j non ha alcun vertice intermedio; quindi

$$\pi_{ij}^{(0)} = \begin{cases} \text{NIL} & \text{se } i = j \text{ o } w_{ij} = \infty \\ i & \text{se } i \neq j \text{ e } w_{ij} < \infty \end{cases} \tag{25.6}$$

$$D^{(0)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad \Pi^{(0)} = \begin{pmatrix} \text{NIL} & 1 & 1 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & \text{NIL} & \text{NIL} \\ 4 & \text{NIL} & 4 & \text{NIL} & \text{NIL} \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad \Pi^{(1)} = \begin{pmatrix} \text{NIL} & 1 & 1 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & \text{NIL} & \text{NIL} \\ 4 & 1 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(2)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad \Pi^{(2)} = \begin{pmatrix} \text{NIL} & 1 & 1 & 2 & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & 2 & 2 \\ 4 & 1 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(3)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad \Pi^{(3)} = \begin{pmatrix} \text{NIL} & 1 & 1 & 2 & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & 2 & 2 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(4)} = \begin{pmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} \quad \Pi^{(4)} = \begin{pmatrix} \text{NIL} & 1 & 4 & 2 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(5)} = \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} \quad \Pi^{(5)} = \begin{pmatrix} \text{NIL} & 3 & 4 & 5 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix}$$

Figura 25.4 La sequenza delle matrici $D^{(k)}$ e $\Pi^{(k)}$ calcolate dall'algoritmo di Floyd-Warshall per il grafo della Figura 25.1.

Per $k \geq 1$, se prendiamo il cammino $i \rightsquigarrow k \rightsquigarrow j$, dove $k \neq j$, allora il predecessore di j che scegliamo è uguale al predecessore di j che avevamo scelto in un cammino minimo da k con tutti i vertici intermedi nell'insieme $\{1, 2, \ldots, k-1\}$. Altrimenti, scegliamo lo stesso predecessore di j che avevamo scelto in un cammino minimo da i con tutti i vertici intermedi nell'insieme $\{1, 2, \ldots, k-1\}$. Formalmente, per $k \geq 1$,

$$\pi_{ij}^{(k)} = \begin{cases} \pi_{ij}^{(k-1)} & \text{se } d_{ij}^{(k-1)} \leq d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \\ \pi_{kj}^{(k-1)} & \text{se } d_{ij}^{(k-1)} > d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \end{cases} \tag{25.7}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Lasciamo al lettore il compito di incorporare il calcolo delle matrici $\Pi^{(k)}$ nella procedura FLOYD-WARSHALL (Esercizio 25.3-3). La Figura 25.4 illustra la sequenza delle matrici $\Pi^{(k)}$ che l'algoritmo risultante calcola per il grafo della Figura 25.1. L'esercizio chiede anche di svolgere il compito più difficile di dimostrare che il sottografo dei predecessori $G_{\pi,i}$ è un albero di cammini minimi con

radice i. L'Esercizio 25.3-7 propone un altro metodo per ricostruire i cammini minimi.

Chiusura transitiva di un grafo orientato

Dato un grafo orientato $G = (V, E)$ con l'insieme dei vertici $V = \{1, 2, \ldots, n\}$, vogliamo determinare, per ciascuna coppia di vertici $i, j \in V$, se esiste un cammino da i a j in G. La ***chiusura transitiva*** di G è definita come il grafo $G^* = (V, E^*)$, dove

$$E^* = \{(i, j) : \text{esiste un cammino dal vertice } i \text{ al vertice } j \text{ in } G\}$$

Un modo per calcolare la chiusura transitiva di un grafo nel tempo $\Theta(n^3)$ consiste nell'assegnare un peso 1 a ogni arco di E e nell'eseguire l'algoritmo di Floyd-Warshall. Se esiste un cammino dal vertice i al vertice j, si ha $d_{ij} < n$, altrimenti $d_{ij} = \infty$.

C'è un altro metodo simile per calcolare la chiusura transitiva di G nel tempo $\Theta(n^3)$, che consente di risparmiare tempo e spazio. Questo metodo consiste nel sostituire le operazioni aritmetiche min e + dell'algoritmo di Floyd-Warshall con le operazioni logiche $\vee$ (OR logico) e $\wedge$ (AND logico). Per $i, j, k = 1, 2, \ldots, n$, definiamo $t_{ij}^{(k)}$ pari a 1, se esiste un cammino nel grafo G dal vertice i al vertice j con tutti i vertici intermedi nell'insieme $\{1, 2, \ldots, k\}$, pari a 0 negli altri casi. Costruiamo la chiusura transitiva $G^* = (V, E^*)$ ponendo l'arco (i, j) in E^* se e soltanto se $t_{ij}^{(n)} = 1$. Una definizione ricorsiva di $t_{ij}^{(k)}$, analoga alla ricorrenza (25.5), è la seguente:

$$t_{ij}^{(0)} = \begin{cases} 0 & \text{se } i \neq j \text{ e } (i, j) \notin E \\ 1 & \text{se } i = j \text{ o } (i, j) \in E \end{cases}$$

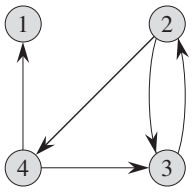
e per $k \geq 1$:

$$t_{ij}^{(k)} = t_{ij}^{(k-1)} \vee \left(t_{ik}^{(k-1)} \wedge t_{kj}^{(k-1)}\right) \tag{25.8}$$

Come nell'algoritmo di Floyd-Warshall, calcoliamo le matrici $T^{(k)} = \left(t_{ij}^{(k)}\right)$ per valori crescenti di k.

```
TRANSITIVE-CLOSURE(G)
 1  n = |G.V|
 2  Sia T^(0) = (t_ij^(0)) una nuova matrice n × n
 3  for i = 1 to n
 4      for j = 1 to n
 5          if i == j or (i, j) ∈ G.E
 6              t_ij^(0) = 1
 7          else t_ij^(0) = 0
 8  for k = 1 to n
 9      Sia T^(k) = (t_ij^(k)) una nuova matrice n × n
10      for i = 1 to n
11          for j = 1 to n
12              t_ij^(k) = t_ij^(k-1) ∨ (t_ik^(k-1) ∧ t_kj^(k-1))
13  return T^(n)
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)



$$T^{(0)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{pmatrix} \quad T^{(1)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{pmatrix} \quad T^{(2)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{pmatrix}$$

$$T^{(3)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix} \quad T^{(4)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}$$

Figura 25.5 Un grafo orientato e le matrici $T^{(k)}$ calcolate dall'algoritmo per la chiusura transitiva.

La Figura 25.5 illustra le matrici $T^{(k)}$ calcolate dalla procedura TRANSITIVE-CLOSURE per un grafo campione. La procedura TRANSITIVE-CLOSURE, come l'algoritmo di Floyd-Warshall, viene eseguita nel tempo $\Theta(n^3)$. In alcuni calcolatori, tuttavia, le operazioni logiche su valori di un singolo bit vengono eseguite più rapidamente delle operazioni aritmetiche su parole di interi. Inoltre, poiché l'algoritmo per la chiusura transitiva usa soltanto valori booleani, anziché interi, lo spazio che richiede è minore di quello richiesto dall'algoritmo di Floyd-Warshall per un fattore che corrisponde alla dimensione di una parola di memoria del calcolatore.

Esercizi

25.3-1

Eseguite l'algoritmo di Floyd-Warshall sul grafo orientato pesato della Figura 25.2. Mostrate la matrice $D^{(k)}$ che si ottiene in ogni iterazione del ciclo esterno.

25.3-2

Spiegate come calcolare la chiusura transitiva utilizzando la tecnica descritta nel Paragrafo 25.2.

25.3-3

Modificate la procedura FLOYD-WARSHALL per includere il calcolo delle matrici $\Pi^{(k)}$ secondo le equazioni (25.6) e (25.7). Dimostrate che per ogni $i \in V$, il sottografo dei predecessori $G_{\pi,i}$ è un albero di cammini minimi con radice i (*suggerimento:* per dimostrare che $G_{\pi,i}$ è aciclico, prima dimostrate che $\pi_{ij}^{(k)} = l$ implica $d_{ij}^{(k)} \geq d_{il}^{(k)} + w_{lj}$, secondo la definizione di $\pi_{ij}^{(k)}$. Poi, adattate la dimostrazione del Lemma 24.16).

Acquistato da Michele Miccele su Weinstein 2022-07-07 23:12 Numero Ordine Libreria: 1.1895030115.220707 © Copyright © 2022, McGraw-Hill Education (Italy)

25.3-4

Per quanto detto in precedenza, l'algoritmo di Floyd-Warshall richiede uno spazio $\Theta(n^3)$, perché calcoliamo $d_{ij}^{(k)}$ per $i, j, k = 1, 2, \ldots, n$. Dimostrate che la seguente procedura, che elimina semplicemente tutti gli apici, è corretta, e quindi occorre soltanto uno spazio $\Theta(n^2)$.

```
FLOYD-WARSHALL'(W)
1  n = W.rows
2  D = W
3  for k = 1 to n
4      for i = 1 to n
5          for j = 1 to n
6              d_ij = min(d_ij, d_ik + d_kj)
7  return D
```

25.3-5
Supponete di modificare il modo in cui viene gestita l'uguaglianza nell'equazione (25.7):

$$\pi_{ij}^{(k)} = \begin{cases} \pi_{ij}^{(k-1)} & \text{se } d_{ij}^{(k-1)} < d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \\ \pi_{kj}^{(k-1)} & \text{se } d_{ij}^{(k-1)} \geq d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \end{cases}$$

È corretta questa definizione alternativa della matrice dei predecessori Π?

25.3-6
Come può essere utilizzato l'output dell'algoritmo di Floyd-Warshall per rilevare la presenza di un ciclo di peso negativo?

25.3-7
Un altro metodo per ricostruire i cammini minimi nell'algoritmo di Floyd-Warshall consiste nell'utilizzare i valori $\phi_{ij}^{(k)}$ per $i, j, k = 1, 2, \ldots, n$, dove $\phi_{ij}^{(k)}$ è il vertice intermedio con numerazione più grande di un cammino minimo da i a j, in cui tutti i vertici intermedi si trovano nell'insieme $\{1, 2, \ldots, k\}$. Formulate una definizione ricorsiva per $\phi_{ij}^{(k)}$, modificate la procedura FLOYD-WARSHALL per calcolare i valori $\phi_{ij}^{(k)}$ e riscrivete la procedura PRINT-ALL-PAIRS-SHORTEST-PATH per prendere la matrice $\Phi = \left(\phi_{ij}^{(n)}\right)$ come input. C'è qualche analogia fra la matrice Φ e la tabella s che è stata utilizzata per risolvere il problema della moltiplicazione di una sequenza di matrici nel Paragrafo 15.2?

25.3-8
Create un algoritmo con tempo $O(VE)$ per calcolare la chiusura transitiva di un grafo orientato $G = (V, E)$.

25.3-9
Supponete che la chiusura transitiva di un grafo orientato aciclico possa essere calcolata nel tempo $f(|V|, |E|)$, dove f è una funzione monotonicamente crescente di $|V|$ e $|E|$. Dimostrate che il tempo per calcolare la chiusura transitiva $G^* = (V, E^*)$ di un generico grafo orientato $G = (V, E)$ è $f(|V|, |E|) + O(V + E^*)$.

Acquistato da Michele Michele su Webster il 2022-07-27 23:12. Numero Ordine Libraria 190503016.230707.0. Copyright © 2022, McGraw-Hill Education (Italy)

25.4 Algoritmo di Johnson per i grafi sparsi

L'algoritmo di Johnson trova i cammini minimi fra tutte le coppie nel tempo $O(V^2 \lg V + VE)$. Per i grafi sparsi è asintoticamente migliore sia della tecnica dell'elevazione al quadrato ripetuta sia dell'algoritmo di Floyd-Warshall. L'algoritmo restituisce una matrice di pesi di cammini minimi per tutte le coppie di vertici oppure segnala la presenza di un ciclo negativo nel grafo di input. L'algo-

ritmo di Johnson usa come subroutine sia l'algoritmo di Dijkstra sia l'algoritmo di Bellman-Ford, che sono descritti nel Capitolo 24.

L'algoritmo di Johnson usa la tecnica del ***ricalcolo dei pesi***, che funziona nel modo seguente. Se tutti i pesi degli archi w in un grafo $G = (V, E)$ non sono negativi, possiamo trovare i cammini minimi fra tutte le coppie di vertici eseguendo l'algoritmo di Dijkstra una volta per ogni vertice; se la coda di min-priorità è implementata con un heap di Fibonacci, il tempo di esecuzione di questo algoritmo per tutte le coppie è $O(V^2 \lg V + VE)$. Se G ha archi di peso negativo, ma è privo di cicli di peso negativo, calcoliamo semplicemente un nuovo insieme di pesi di archi non negativi che ci consente di utilizzare lo stesso metodo. Questo nuovo insieme di pesi $\hat{w}$ deve soddisfare due importanti proprietà.

1. Per ogni coppia di vertici $u, v \in V$, un cammino p è un cammino minimo da u a v con la funzione peso w, se e soltanto se p è anche un cammino minimo da u a v con la funzione peso $\hat{w}$.
2. Per ogni arco (u, v), il nuovo peso $\hat{w}(u, v)$ non è negativo.

Come vedremo più avanti, l'elaborazione preliminare di G per determinare la nuova funzione peso $\hat{w}$ può essere eseguita nel tempo $O(VE)$.

Preservare i cammini minimi con il ricalcolo dei pesi

Come dimostra il seguente lemma, è facile proporre una tecnica di ricalcolo dei pesi degli archi che soddisfa la prima proprietà. Indichiamo con δ i pesi dei cammini minimi derivati dalla funzione peso w e con $\hat{\delta}$ i pesi dei cammini minimi derivati dalla funzione peso $\hat{w}$.

Lemma 25.1 (Il ricalcolo dei pesi non cambia i cammini minimi)
Dato un grafo orientato pesato $G = (V, E)$ con funzione peso $w : E \to \mathbb{R}$, sia $h : V \to \mathbb{R}$ una funzione qualsiasi che associa i vertici a numeri reali. Per ogni arco $(u, v) \in E$, definiamo

$$\hat{w}(u, v) = w(u, v) + h(u) - h(v) \tag{25.9}$$

Sia $p = \langle v_0, v_1, \ldots, v_k \rangle$ un cammino qualsiasi dal vertice v_0 al vertice v_k. Allora p è un cammino minimo da v_0 a v_k con la funzione peso w, se e soltanto se p è anche un cammino minimo con la funzione peso $\hat{w}$. Ovvero, $w(p) = \delta(v_0, v_k)$ se e soltanto se $\hat{w}(p) = \hat{\delta}(v_0, v_k)$. Inoltre, G ha un ciclo di peso negativo con la funzione peso w, se e soltanto se G ha un ciclo di peso negativo con la funzione peso $\hat{w}$.

Dimostrazione Iniziamo dimostrando che

$$\hat{w}(p) = w(p) + h(v_0) - h(v_k) \tag{25.10}$$

Abbiamo

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned}
\hat{w}(p) &= \sum_{i=1}^{k} \hat{w}(v_{i-1}, v_i) \\
&= \sum_{i=1}^{k} (w(v_{i-1}, v_i) + h(v_{i-1}) - h(v_i))
\end{aligned}$$

$$\begin{aligned} &= \sum_{i=1}^{k} w(v_{i-1}, v_i) + h(v_0) - h(v_k) && \text{(perché gli altri termini della sommatoria si annullano)} \\ &= w(p) + h(v_0) - h(v_k) \end{aligned}$$

Quindi, un cammino qualsiasi p da v_0 a v_k ha $\widehat{w}(p) = w(p) + h(v_0) - h(v_k)$. Siccome $h(v_0)$ e $h(v_k)$ non dipendono dal cammino, se un cammino da v_0 a v_k è più corto di un altro con la funzione peso w, allora è anche più corto se si usa la funzione peso $\widehat{w}$. Quindi, $w(p) = \delta(v_0, v_k)$ se e soltanto se $\widehat{w}(p) = \widehat{\delta}(v_0, v_k)$.

Infine, dimostriamo che G ha un ciclo di peso negativo con la funzione peso w, se e soltanto se G ha un ciclo di peso negativo con la funzione peso $\widehat{w}$. Consideriamo un ciclo qualsiasi $c = \langle v_0, v_1, \ldots, v_k \rangle$, dove $v_0 = v_k$. Per l'equazione (25.10), si ha

$$\begin{aligned} \widehat{w}(c) &= w(c) + h(v_0) - h(v_k) \\ &= w(c) \end{aligned}$$

Quindi, il ciclo c ha peso negativo con la funzione peso w, se e soltanto se ha peso negativo con la funzione peso $\widehat{w}$. ■

Produrre pesi non negativi con la tecnica del ricalcolo dei pesi

L'obiettivo successivo è garantire che la seconda proprietà sia valida: vogliamo che il peso $\widehat{w}(u, v)$ non sia negativo per ogni arco $(u, v) \in E$. Dato un grafo orientato pesato $G = (V, E)$ con funzione peso $w : E \to \mathbb{R}$, creiamo un nuovo grafo $G' = (V', E')$, dove $V' = V \cup \{s\}$ per un nuovo vertice $s \notin V$ ed $E' = E \cup \{(s, v) : v \in V\}$. Estendiamo la funzione peso w in modo che $w(s, v) = 0$ per ogni $v \in V$. Notate che, poiché s non ha archi entranti, nessun cammino minimo in G' (tranne i cammini minimi con sorgente s) contiene s. Inoltre, G' non ha cicli di peso negativo, se e soltanto se G non ha cicli di peso negativo. La Figura 25.6(a) illustra il grafo G' che corrisponde al grafo G della Figura 25.1.

Adesso supponiamo che G e G' non abbiano cicli di peso negativo. Definiamo $h(v) = \delta(s, v)$ per ogni $v \in V'$. Per la disuguaglianza triangolare (Lemma 24.10), abbiamo $h(v) \leq h(u) + w(u, v)$ per tutti gli archi $(u, v) \in E'$. Quindi, se definiamo i nuovi pesi $\widehat{w}$ secondo l'equazione (25.9), abbiamo $\widehat{w}(u, v) = w(u, v) + h(u) - h(v) \geq 0$, e la seconda proprietà è soddisfatta. La Figura 25.6(b) illustra il grafo G' della Figura 25.6(a) con i pesi degli archi ricalcolati.

Calcolo dei cammini minimi fra tutte le coppie

Per calcolare i cammini minimi fra tutte le coppie, l'algoritmo di Johnson usa come subroutine l'algoritmo di Bellman-Ford (Paragrafo 24.2) e l'algoritmo di Dijkstra (Paragrafo 24.4). Suppone che gli archi siano memorizzati in liste di adiacenza. L'algoritmo restituisce la solita matrice $D = d_{ij}$, di dimensioni $|V| \times |V|$, dove $d_{ij} = \delta(i, j)$, oppure segnala che il grafo di input contiene un ciclo di peso negativo. Analogamente agli altri algoritmi per cammini minimi fra tutte le coppie, supponiamo che i vertici siano numerati da 1 a $|V|$.

Acquistato da Michele Michele su Webster il 2022-0[illegible]

La seguente procedura JOHNSON svolge semplicemente le azioni che abbiamo specificato in precedenza. La riga 1 produce G'. La riga 2 esegue l'algoritmo di Bellman-Ford su G' con la funzione peso w e il vertice sorgente s. Se G', e quindi G, contiene un ciclo di peso negativo, la riga 3 segnala il problema. Le righe 4–12 assumono che G' non contenga cicli di peso negativo. Le righe 4–5

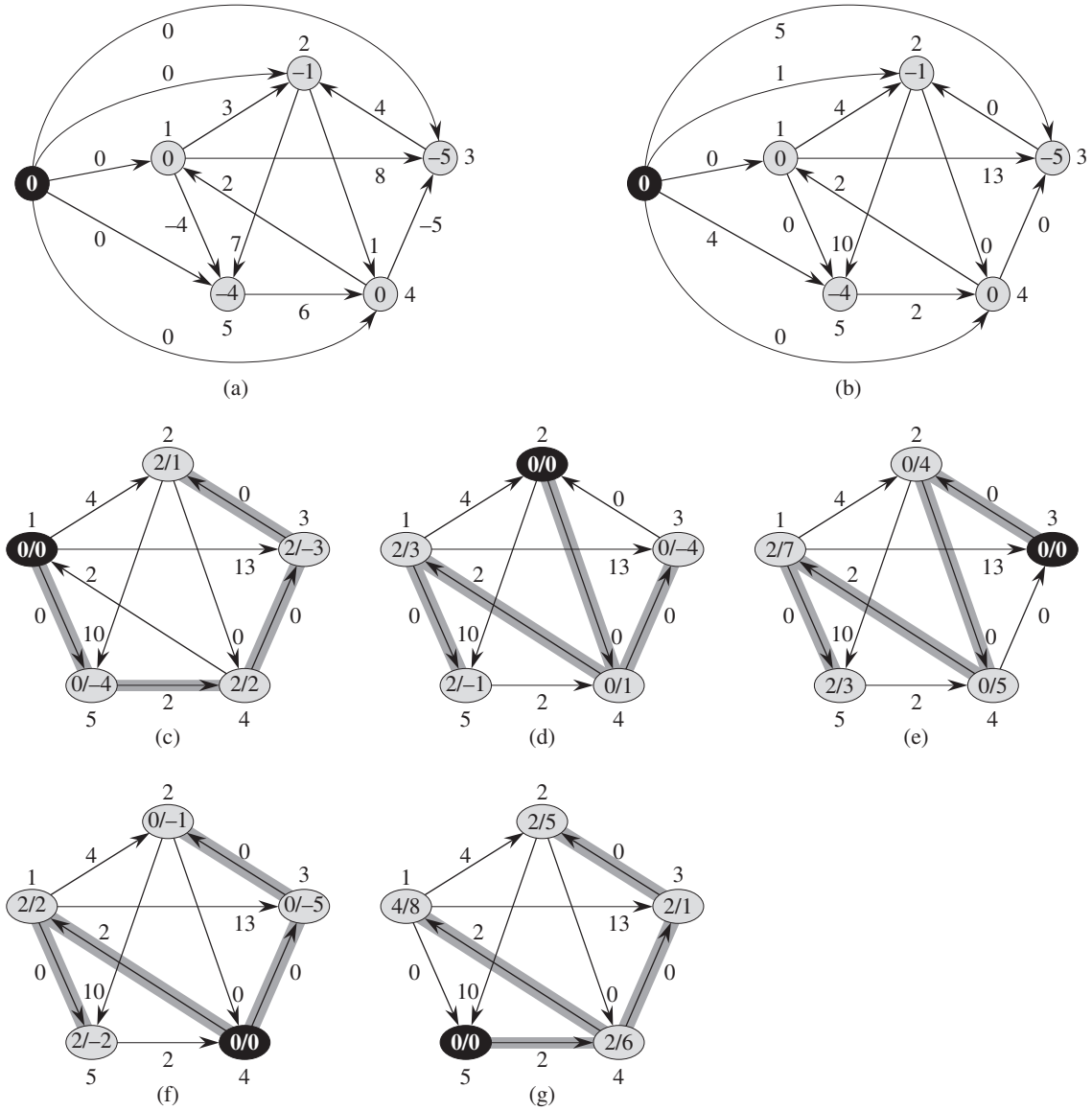


Figura 25.6 L'algoritmo di Johnson per i cammini minimi fra tutte le coppie viene eseguito sul grafo della Figura 25.1. **(a)** Il grafo G' con la funzione peso originale w. Il nuovo vertice s è nero. All'interno di ogni vertice v è indicato il valore $h(v) = \delta(s, v)$. **(b)** Il peso di ogni arco (u, v) viene ricalcolato con la funzione peso $\widehat{w}(u, v) = w(u, v) + h(u) - h(v)$. **(c)–(g)** Il risultato dell'algoritmo di Dijkstra applicato a ciascun vertice di G con la funzione peso $\widehat{w}$. In ogni parte, il vertice sorgente u è nero e gli archi ombreggiati sono gli alberi dei cammini minimi calcolati dall'algoritmo. All'interno di ogni vertice v sono indicati i valori $\widehat{\delta}(u, v)$ e $\delta(u, v)$, separati dal simbolo /. Il valore $d_{uv} = \delta(u, v)$ è uguale a $\widehat{\delta}(u, v) + h(v) - h(u)$.

assegnano a $h(v)$ il peso del cammino minimo $\delta(s, v)$ calcolato dall'algoritmo di Bellman-Ford per ogni $v \in V'$. Le righe 6–7 calcolano i nuovi pesi $\widehat{w}$. Per ogni coppia di vertici $u, v \in V$, il ciclo **for** nelle righe 9–12 calcola il peso del cammino minimo $\widehat{\delta}(u, v)$ chiamando l'algoritmo di Dijkstra una volta per ogni vertice in V. La riga 12 memorizza nella posizione d_{uv} della matrice il peso corretto del cammino minimo $\delta(u, v)$, calcolato mediante l'equazione (25.10). Infine, la riga 13 restituisce la matrice D completa. La Figura 25.6 illustra l'esecuzione dell'algoritmo di Johnson.

Acquistato da Michele Michelessi Webster ti 2022-07-07 23:12 Numero Ordine Libreria: 1995080016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Se la coda di min-priorità nell'algoritmo di Dijkstra è implementata con un heap di Fibonacci, il tempo di esecuzione dell'algoritmo di Johnson è $O(V^2 \lg V + VE)$. L'implementazione più semplice con un min-heap binario genera un tempo di esecuzione di $O(VE \lg V)$, che è ancora asintoticamente più veloce dell'algoritmo di Floyd-Warshall se il grafo è sparso.

```
JOHNSON(G, w)
 1  Calcola G', dove G'.V = G.V ∪ {s},
        G'.E = G.E ∪ {(s, v) : v ∈ G.V} e
        w(s, v) = 0 per ogni v ∈ G.V
 2  if BELLMAN-FORD(G', w, s) == FALSE
 3      stampa "il grafo di input contiene un ciclo di peso negativo"
 4  else for ogni vertice v ∈ G'.V
 5           assegna a h(v) il valore di δ(s, v)
                  calcolato dall'algoritmo di Bellman-Ford
 6       for ogni arco (u, v) ∈ G'.E
 7           ŵ(u, v) = w(u, v) + h(u) − h(v)
 8       Sia D = (d_uv) una nuova matrice n × n
 9       for ogni vertice u ∈ G.V
10           esegue DIJKSTRA(G, ŵ, u) per calcolare δ̂(u, v) per ogni v ∈ G.V
11           for ogni vertice v ∈ G.V
12               d_uv = δ̂(u, v) + h(v) − h(u)
13       return D
```

Esercizi

25.4-1

Utilizzate l'algoritmo di Johnson per trovare i cammini minimi fra tutte le coppie di vertici nel grafo della Figura 25.2. Elencate i valori di h e $\widehat{w}$ calcolati dall'algoritmo.

25.4-2

Qual è lo scopo di aggiungere il nuovo vertice s a V, ottenendo V'?

25.4-3

Supponete che $w(u, v) \geq 0$ per ogni arco $(u, v) \in E$. Qual è la relazione fra le funzioni peso w e $\widehat{w}$?

25.4-4

Il professor Greenstreet ritiene che ci sia un metodo più semplice per ricalcolare i pesi degli archi di quello utilizzato nell'algoritmo di Johnson. Ponendo $w^* = \min_{(u,v)\in E}\{w(u, v)\}$, definite $\widehat{w}(u, v) = w(u, v) - w^*$ per tutti gli archi $(u, v) \in E$. Che cosa c'è di sbagliato nel metodo di ricalcolo del professore?

25.4-5

Supponete di eseguire l'algoritmo di Johnson su un grafo orientato G con funzione peso w. Dimostrate che, se G contiene un ciclo c di peso 0, allora $\widehat{w}(u, v) = 0$ per ogni arco (u, v) in c.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

25.4-6

Il professor Michener ritiene che non sia necessario creare un nuovo vertice sorgente nella riga 1 della procedura JOHNSON. Secondo il professore basta soltanto porre $G' = G$ e considerare s come un vertice qualsiasi in $V[G]$. Trovate un esempio di grafo orientato pesato G per dimostrare che, implementando l'idea del

professore nella procedura JOHNSON, si ottengono risultati errati. Poi dimostrate che, se G è un grafo fortemente connesso (ogni vertice è raggiungibile da qualsiasi altro vertice), i risultati restituiti da JOHNSON con la modifica del professore sono corretti.

Problemi

25-1 Chiusura transitiva di un grafo dinamico

Supponete di volere mantenere la chiusura transitiva di un grafo orientato $G = (V, E)$ durante l'inserimento degli archi in E. Ovvero, dopo l'inserimento di un arco, bisogna aggiornare la chiusura transitiva degli archi che sono stati inseriti fino a quel momento. Supponete che il grafo G inizialmente non abbia archi e che la chiusura transitiva debba essere rappresentata come una matrice booleana.

a. Spiegate come la chiusura transitiva $G^* = (V, E^*)$ di un grafo $G = (V, E)$ possa essere aggiornata nel tempo $O(V^2)$ quando un nuovo arco viene aggiunto al grafo G.

b. Trovate un esempio di un grafo G e di un arco e tali che sia richiesto un tempo $\Omega(V^2)$ per aggiornare la chiusura transitiva dopo l'inserimento di e in G, indipendentemente da quale algoritmo venga utilizzato.

c. Descrivete un algoritmo efficiente per aggiornare la chiusura transitiva durante l'inserimento degli archi nel grafo. Per una sequenza qualsiasi di n inserimenti, il vostro algoritmo dovrebbe essere eseguito nel tempo totale $\sum_{i=1}^{n} t_i = O(V^3)$, dove t_i è il tempo per aggiornare la chiusura transitiva quando viene inserito l'i-esimo arco. Dimostrate che l'algoritmo raggiunge questo limite di tempo.

25-2 Cammini minimi nei grafi ϵ-densi

Un grafo $G = (V, E)$ è ***ϵ-denso*** se $|E| = \Theta(V^{1+\epsilon})$ per qualche costante ϵ nell'intervallo $0 < \epsilon \leq 1$. Utilizzando i min-heap d-ari (vedere il Problema 6-2) negli algoritmi per cammini minimi nei grafi ϵ-densi, possiamo competere con i tempi di esecuzione degli algoritmi basati sugli heap di Fibonacci senza ricorrere a una struttura dati così complessa.

a. Quali sono i tempi di esecuzione asintotici di INSERT, EXTRACT-MIN e DECREASE-KEY in funzione di d e del numero n di elementi in un min-heap d-ario? Quali sono questi tempi di esecuzione se scegliamo $d = \Theta(n^\alpha)$ per qualche costante $0 < \alpha \leq 1$? Confrontate questi tempi di esecuzione con i costi ammortizzati di queste operazioni per un heap di Fibonacci.

b. Spiegate come calcolare nel tempo $O(E)$ i cammini minimi da una sorgente singola in un grafo orientato ϵ-denso $G = (V, E)$ senza archi di peso negativo (*suggerimento:* scegliete d come una funzione di ϵ).

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950301G-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

c. Spiegate come risolvere nel tempo $O(VE)$ il problema dei cammini minimi fra tutte le coppie in un grafo orientato ϵ-denso $G = (V, E)$ senza archi di peso negativo.

d. Spiegate come risolvere nel tempo $O(VE)$ il problema dei cammini minimi fra tutte le coppie in un grafo orientato ϵ-denso $G = (V, E)$ che può avere archi di peso negativo, ma non cicli di peso negativo.

Note

Lawler [225] ha svolto un'interessante analisi del problema dei cammini minimi fra tutte le coppie, sebbene non abbia esaminato le soluzioni per i grafi sparsi. Egli attribuisce al folclore l'algoritmo basato sulla moltiplicazione di matrici. Floyd [106] ha sviluppato l'algoritmo di Floyd-Warshall, basandosi su un teorema di Warshall [350] che descrive come calcolare la chiusura transitiva delle matrici booleane. L'algoritmo di Johnson è preso da [193].

Numerosi ricercatori hanno migliorato gli algoritmi per calcolare i cammini minimi tramite la moltiplicazione di matrici. Fredman [112] ha dimostrato che il problema dei cammini minimi fra tutte le coppie può essere risolto effettuando $O(V^{5/2})$ confronti fra somme di pesi di archi; ha ottenuto un algoritmo che viene eseguito nel tempo $O(V^3(\lg\lg V/\lg V)^{1/3})$, che è leggermente migliore del tempo di esecuzione dell'algoritmo di Floyd-Warshall. Han [160] ha ridotto il tempo di esecuzione a $O(V^3(\lg\lg V/\lg V)^{5/4})$. Un altro gruppo di ricercatori ha dimostrato che gli algoritmi per la moltiplicazione rapida delle matrici (vedere le note in fondo al Capitolo 28) possono essere applicati al problema dei cammini minimi fra tutte le coppie. Sia $O(n^\omega)$ il tempo di esecuzione dell'algoritmo più veloce per moltiplicare le matrici $n \times n$; attualmente $\omega < 2,376$ [79]. Galil e Margalit [124, 125] e Seidel [309] hanno progettato degli algoritmi che risolvono il problema dei cammini minimi fra tutte le coppie nei grafi non orientati e non pesati nel tempo $(V^\omega p(V))$, dove $p(n)$ indica una particolare funzione che è polilogaritmicamente limitata in n. Nei grafi densi questi algoritmi hanno tempi più piccoli del tempo $O(VE)$ richiesto per effettuare $|V|$ visite in ampiezza. Molti ricercatori hanno esteso questi risultati per ottenere algoritmi che risolvono il problema dei cammini minimi fra tutte le coppie nei grafi non orientati, i cui pesi degli archi sono numeri interi nell'intervallo $\{1, 2, \ldots, W\}$. Fra questi algoritmi, il più veloce asintoticamente è quello di Shoshan e Zwick [317], che viene eseguito nel tempo $O(WV^\omega p(VW))$.

Karger, Koller e Phillips [197] e, in maniera autonoma, McGeoch [248] hanno dato un limite di tempo che dipende da E^*, l'insieme degli archi in E che figurano in qualche cammino minimo. Dato un grafo con archi di peso non negativo, i loro algoritmi vengono eseguiti nel tempo $O(VE^* + V^2 \lg V)$ e rappresentano un miglioramento rispetto al tempo di esecuzione dell'algoritmo di Dijkstra di $|V|$ volte, quando $|E^*| = o(E)$.

Baswana, Hariharan e Sen [33] hanno esaminato gli algoritmi decrementali per mantenere le informazioni sui cammini minimi fra le coppie e la chiusura transitiva. Questi algoritmi ammettono una sequenza mista di cancellazioni e query; per esempio, il Problema 25-1, in cui vengono inseriti gli archi, richiede un algoritmo decrementale. Gli algoritmi di Baswana, Hariharan e Sen sono randomizzati e, quando esiste un cammino, il loro algoritmo di chiusura transitiva potrebbe non riuscire a segnalarlo con una probabilità di $1/n^c$, con $c > 0$ arbitrario. I tempi di query sono $O(1)$ con probabilità elevata. Per la chiusura transitiva, il tempo ammortizzato per ciascun aggiornamento è $O(V^{4/3}\lg^{1/3} V)$. Per i cammini minimi fra le coppie, i tempi di aggiornamento dipendono dalle query. Per le query che forniscono soltanto i pesi dei cammini minimi, il tempo ammortizzato per aggiornamento è $O(V^3/E\lg^2 V)$. Per segnalare il cammino minimo effettivo, il tempo di aggiornamento ammortizzato è $\min(O(V^{3/2}\sqrt{\lg V}), O(V^3/E\lg^2 V))$. Demetrescu e Italiano [85] hanno mostrato come gestire le operazioni di aggiornamento e le query quando gli archi vengono inseriti e cancellati, purché ciascun arco abbia un intervallo limitato di valori possibili appartenenti ai numeri reali.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Aho, Hopcroft e Ullman [5] hanno definito una struttura algebrica nota come "semianello chiuso", che opera come uno schema generale per risolvere i problemi dei cammini nei grafi orientati. Sia l'algoritmo di Floyd-Warshall sia quello per la chiusura transitiva, descritti nel Paragrafo 25.3, sono casi particolari di algoritmi per cammini minimi fra tutte le coppie che si basano sui semianelli chiusi. Maggs e Plotkin [241] hanno spiegato come trovare gli alberi di connessione minimi utilizzando un semianello chiuso.

Flusso massimo 26

Nello stesso modo in cui è possibile modellare una carta stradale come un grafo orientato al fine di trovare il cammino minimo da una città all'altra, così è possibile interpretare un grafo orientato come una "rete di flusso" e utilizzarlo per rispondere alle domande sui flussi dei materiali. Immaginate un materiale che scorre attraverso un sistema da una sorgente, dove il materiale è prodotto, fino a raggiungere un pozzo (o destinazione) dove viene consumato. La sorgente produce il materiale a una certa velocità; il pozzo consuma il materiale alla stessa velocità. Il "flusso" del materiale in un punto qualsiasi del sistema è intuitivamente la velocità con la quale si sposta il materiale. Le reti di flusso possono essere utilizzate per modellare i liquidi che scorrono nelle tubazioni, i componenti che attraversano le catene di montaggio, la corrente nelle reti elettriche, le informazioni nelle reti di comunicazione e così via.

Ogni arco orientato in una rete di flusso può essere considerato come un condotto per il materiale. Ogni condotto ha una capacità prefissata, che è espressa come la quantità massima di materiale che può fluire nel condotto nell'unità di tempo, per esempio 200 litri di liquido all'ora in una tubazione o 20 ampere di corrente elettrica in un conduttore. I vertici sono i giunti tra i condotti e il materiale scorre nei vertici (tranne la sorgente e il pozzo) senza accumularsi. In altre parole, la velocità con la quale il materiale entra in un vertice deve essere uguale alla velocità con la quale esce dal vertice. Questa proprietà è detta "conservazione del flusso" ed è equivalente alla legge di Kirchhoff delle correnti quando il materiale è la corrente elettrica.

Nel problema del flusso massimo, vogliamo calcolare la velocità massima con la quale il materiale può essere trasportato dalla sorgente al pozzo senza violare alcun vincolo sulla capacità. È uno tra i problemi più semplici che riguardano le reti di flusso e, come vedremo in questo capitolo, può essere risolto con algoritmi efficienti. Inoltre, le tecniche di base utilizzate negli algoritmi di flusso massimo possono essere adattate per risolvere altri problemi di flusso su rete.

Questo capitolo presenta due metodi generali per risolvere il problema del flusso massimo. Il Paragrafo 26.1 formalizza i concetti di flusso e reti di flusso, definendo formalmente il problema del flusso massimo. Il Paragrafo 26.2 descrive il metodo classico di Ford e Fulkerson per trovare i flussi massimi. Un'applicazione di questo metodo – trovare un abbinamento massimo in un grafo non orientato bipartito – è descritta nel Paragrafo 26.3. Il Paragrafo 26.4 presenta il metodo "push-relabel", che è alla base di molti dei più veloci algoritmi per i problemi di flusso su rete. Il Paragrafo 26.5 tratta l'algoritmo "relabel-to-front", una particolare implementazione del metodo push-relabel che viene eseguito nel tempo $O(V^3)$. Sebbene questo algoritmo non sia il più veloce tra gli algoritmi conosciuti, esso consente di illustrare alcune delle tecniche utilizzate negli algoritmi asintoticamente più veloci e, nella pratica, è abbastanza efficiente.

Acquistato da Michele Mo... Weber ... Mon Jul 11 ... 2022 ... 2022 ... McGraw-Hill Education (Italy)

26.1 Reti di flusso

In questo paragrafo definiremo le reti di flusso secondo la teoria dei grafi, descriveremo le loro proprietà e formuleremo con esattezza il problema del flusso massimo. Presenteremo anche qualche utile notazione.

Reti di flusso e flussi

Una ***rete di flusso*** $G = (V, E)$ è un grafo orientato in cui ogni arco $(u, v) \in E$ ha una ***capacità*** non negativa $c(u, v) \geq 0$. Si richiede inoltre che se E contiene un arco (u, v), allora non esiste un arco (v, u) nella direzione opposta (vedremo più avanti come trattare questa limitazione.) Se $(u, v) \notin E$, allora per comodità definiamo $c(u, v) = 0$, e non ammettiamo cappi. Distinguiamo due vertici in una rete di flusso: una ***sorgente*** s e un ***pozzo*** t. Per comodità, supponiamo che ogni vertice giaccia in qualche cammino dalla sorgente al pozzo. Ovvero, per ogni vertice $v \in V$, esiste un cammino $s \rightsquigarrow v \rightsquigarrow t$. Il grafo è quindi connesso e, poiché ogni vertice diverso da s ha almeno un arco entrante, $|E| \geq |V| - 1$. La Figura 26.1 illustra un esempio di rete di flusso.

Adesso possiamo definire i flussi in modo più formale. Sia $G = (V, E)$ una rete di flusso con una funzione capacità c. Sia s la sorgente della rete e sia t il pozzo. Un ***flusso*** in G è una funzione a valori reali $f : V \times V \to \mathbb{R}$ che soddisfa le seguenti due proprietà:

Vincolo sulla capacità: per ogni $u, v \in V$, si richiede che $f(u, v) \leq c(u, v)$.

Conservazione del flusso: per ogni $u \in V - \{s, t\}$, si richiede che

$$\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v)$$

Quando $(u, v) \notin E$, non ci può essere flusso da u a v, e $f(u, v) = 0$.

La quantità $f(u, v)$ è detta ***flusso*** dal vertice u al vertice v. Il ***valore*** $|f|$ di un flusso f è definito come

$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) \tag{26.1}$$

ovvero il flusso totale che esce dalla sorgente meno il flusso che entra nella sorgente. (Qui, la notazione $|\cdot|$ indica il valore del flusso, non il valore assoluto o la cardinalità.) Normalmente una rete di flusso non ha archi entranti nella sorgente, e il flusso che entra nella sorgente, dato da $\sum_{v \in V} f(v, s)$, sarà 0; lo abbiamo incluso comunque perché, quando presenteremo le reti residue più avanti in questo capitolo, il flusso che entra nella sorgente diventerà significativo. Nel ***problema del flusso massimo***, è data una rete di flusso G con una sorgente s e un pozzo t e si vuole trovare un flusso di valore massimo.

Acquistato da Michele Michele su Webster il 2022-07-0[illegible] Copyright The McGraw-Hill Companies, Inc. Permission required for reproduction or display.

Prima di vedere un esempio di un problema di flusso su rete, esaminiamo brevemente la definizione di flusso e le due proprietà del flusso. Il vincolo sulla capacità dice semplicemente che il flusso da un vertice all'altro deve essere non negativo e non deve superare la capacità prefissata. La proprietà della conservazione del flusso dice che il flusso totale che entra in un vertice, diverso dalla sorgente o dal pozzo, deve essere uguale al flusso totale che esce da quel vertice – informalmente si può dire che "il flusso entrante è uguale al flusso uscente".

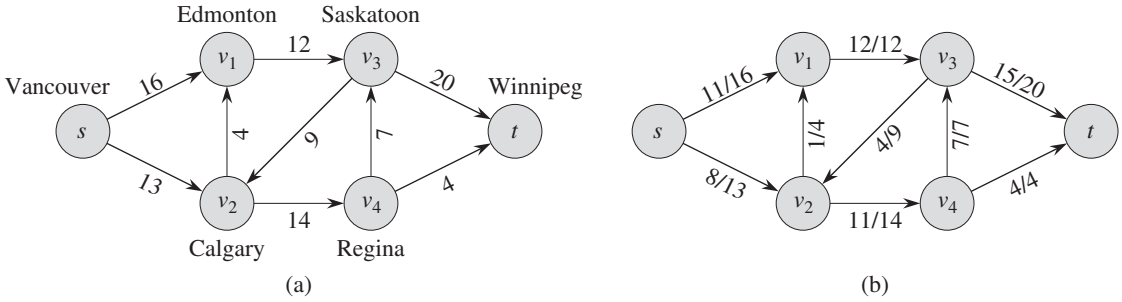


Figura 26.1 **(a)** Una rete di flusso $G = (V, E)$ per il problema del trasporto su autocarro della Lucky Puck Company. La fabbrica di Vancouver è la sorgente s e il magazzino di Winnipeg è il pozzo t. I dischi da hockey (puck) vengono spediti attraverso città intermedie, ma soltanto $c(u, v)$ casse al giorno possono essere trasportate dalla città u alla città v. Ogni arco è etichettato con la sua capacità. **(b)** Un flusso f in G con valore $|f| = 19$. Ogni arco (u, v) è etichettato con $f(u, v)/c(u, v)$. Il simbolo / è utilizzato semplicemente per separare il flusso dalla capacità; non indica l'operazione di divisione.

Un esempio di flusso

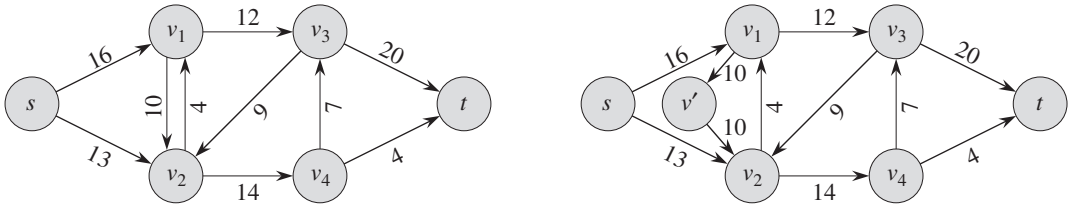
Una rete di flusso può essere utilizzata per modellare il problema del trasporto su autocarro illustrato nella Figura 26.1(a). La Lucky Puck Company ha una fabbrica a Vancouver (la sorgente s) dove produce dischi per hockey su ghiaccio e ha un magazzino (il pozzo t) a Winnipeg. La Lucky Puck affitta lo spazio sugli autocarri di un'altra azienda per spedire i dischi dalla fabbrica al magazzino. Poiché gli autocarri percorrono determinate strade (gli archi) tra le città (i vertici) e hanno una capacità limitata, la Lucky Puck può trasportare al massimo $c(u, v)$ casse al giorno fra ogni coppia di città u e v, come illustra la Figura 26.1(a). La Lucky Puck non ha alcun controllo su queste strade e capacità né può modificare la rete di flusso illustrata nella Figura 26.1(a). L'obiettivo della Lucky Puck è determinare il numero massimo p di casse al giorno che possono essere spedite e, poi, produrre questa quantità, perché non c'è motivo di produrre più dischi di quanti ne possano essere spediti al magazzino. La Lucky Puck non si preoccupa del tempo che impiega un determinato disco per andare dalla fabbrica al magazzino; deve preoccuparsi soltanto del fatto che p casse al giorno lascino la fabbrica e p casse al giorno arrivino al magazzino.

È appropriato modellare il "flusso" delle spedizioni con un flusso in questa rete perché il numero di casse che vengono spedite ogni giorno da una città all'altra è soggetto a un vincolo sulla capacità. Inoltre, deve essere rispettato il principio della conservazione del flusso per cui, a regime, la velocità con la quale i dischi entrano in una città intermedia deve essere uguale alla velocità con la quale escono dalla città, altrimenti le casse si accumulerebbero nelle città intermedie.

Modellare i problemi con archi antiparalleli

Acquistato da Michele Michele su Webster il 2022-07-07 23:12:1 Numero Ordine Libraria 109503016 239705 01 Copyright © 2022 McGraw-Hill Education (Italy)

Supponiamo che l'azienda di trasporto offra alla Lucky Puck l'opportunità di affittare lo spazio per 10 casse negli autocarri che vanno da Edmonton a Calgary. Potrebbe sembrare naturale aggiungere questa opportunità al nostro esempio e formare la rete illustrata nella Figura 26.2(a). Tuttavia, questa rete ha un problema: viola la nostra ipotesi iniziale che, se un arco $(v_1, v_2) \in E$, allora $(v_2, v_1) \notin E$. I due archi (v_1, v_2) e (v_2, v_1) sono detti ***antiparalleli***. Quindi, se vogliamo modellare un problema di flusso con archi antiparalleli, dobbiamo trasformare la rete



(a) (b)

Figura 26.2 Convertire una rete con archi antiparalleli in una rete equivalente senza archi antiparalleli. **(a)** Una rete di flusso contenente entrambi gli archi (v_1, v_2) e (v_2, v_1). **(b)** Una rete equivalente senza archi antiparalleli. Aggiungiamo il nuovo vertice v' e sostituiamo l'arco (v_1, v_2) con la coppia di archi (v_1, v') e (v', v_2), entrambi con la stessa capacità di (v_1, v_2).

in una rete equivalente che non contiene archi antiparalleli. La Figura 26.2(b) mostra questa rete equivalente. Scegliamo uno degli archi antiparalleli, in questo caso (v_1, v_2), e sostituiamolo con la coppia di archi (v_1, v') e (v', v_2) aggiungendo un nuovo vertice v'. Assegniamo ai due nuovi archi la stessa capacità dell'arco originale. La rete risultante soddisfa la proprietà che, se un arco appartiene alla rete, non esiste l'arco opposto. L'Esercizio 26.1-1 chiede di dimostrare che la rete risultante è equivalente a quella originale.

Dunque, notiamo che un problema pratico di flusso può essere modellato più naturalmente da una rete con archi antiparalleli. Tuttavia, è opportuno escludere gli archi antiparalleli; per questo abbiamo descritto un semplice metodo per trasformare una rete contenente archi antiparallelli in una rete equivalente senza archi antiparalleli.

Reti con più sorgenti e pozzi

Un problema di flusso massimo può avere più sorgenti e pozzi, anziché una sola sorgente e un solo pozzo. La Lucky Puck Company, per esempio, potrebbe avere un insieme di m fabbriche $\{s_1, s_2, \ldots, s_m\}$ e un insieme di n magazzini $\{t_1, t_2, \ldots, t_n\}$, come illustra la Figura 26.3(a). Fortunatamente, questo problema non è più difficile di un normale problema di flusso massimo.

Possiamo ridurre il problema di determinare un flusso massimo in una rete con più sorgenti e pozzi a un normale problema di flusso massimo. La Figura 26.3(b) mostra come la rete (a) può essere trasformata in una normale rete di flusso con una sola sorgente e un solo pozzo. Aggiungiamo una ***supersorgente*** s e un arco orientato (s, s_i) con capacità $c(s, s_i) = \infty$ per ogni $i = 1, 2, \ldots, m$. Abbiamo creato anche un ***superpozzo*** t e abbiamo aggiunto un arco orientato (t_i, t) con capacità $c(t_i, t) = \infty$ per ogni $i = 1, 2, \ldots, n$. Intuitivamente, un flusso qualsiasi nella rete (a) corrisponde a un flusso nella rete (b) e viceversa. La sorgente singola s fornisce semplicemente tutto il flusso richiesto dalle varie sorgenti s_i e, analogamente, il pozzo singolo t riceve tutto il flusso consumato dai vari pozzi t_i. L'Esercizio 26.1-2 chiede di dimostrare formalmente che i due problemi sono equivalenti.

Acquistato da Michele Michele su Webster il 2022-07-25 21:10:20 Numero Ordine Interno: 2055810.2007267 - Copyright © 2022 The McGraw-Hill Education (Italy)

Esercizi

26.1-1

Dimostrate che spezzando un arco in una rete di flusso si ottiene una rete equivalente. Più formalmente, supponete che la rete di flusso G contenga l'arco (u, v) e

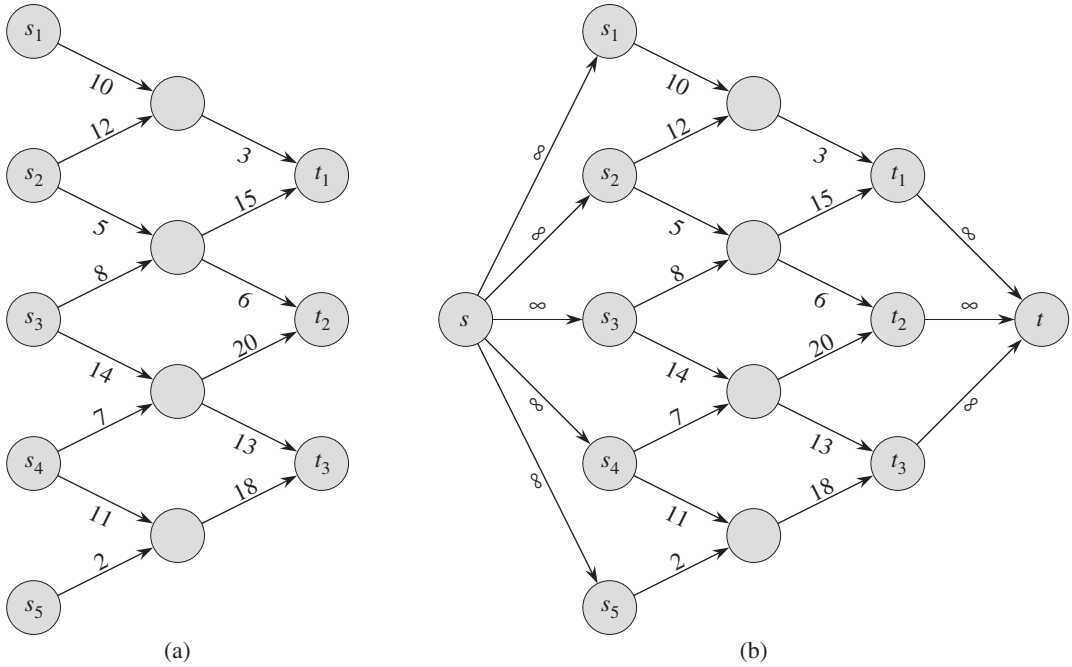


Figura 26.3 Trasformare un problema di flusso massimo con più sorgenti e pozzi in un problema con una sola sorgente e un solo pozzo. **(a)** Una rete di flusso con cinque sorgenti $S = \{s_1, s_2, s_3, s_4, s_5\}$ e tre pozzi $T = \{t_1, t_2, t_3\}$. **(b)** Una rete di flusso equivalente con una sola sorgente e un solo pozzo. Abbiamo aggiunto una supersorgente s e un arco con capacità infinita che collega s a ciascuna delle varie sorgenti. Abbiamo anche aggiunto un superpozzo t e un arco con capacità infinita che collega ciascuno dei vari pozzi a t.

creiate una nuova rete di flusso G' aggiungendo un nuovo vertice x e sostituendo (u, v) con i nuovi archi (u, x) e (x, v) di capacità $c(u, x) = c(x, v) = c(u, v)$. Dimostrate che un flusso massimo in G' ha lo stesso valore di un flusso massimo in G.

26.1-2

Estendete le definizioni e le proprietà dei flussi al problema delle reti con sorgenti e pozzi multipli. Dimostrate che un flusso qualsiasi in una rete con sorgenti e pozzi multipli corrisponde a un flusso di valore identico nella rete con una sola sorgente e un solo pozzo ottenuta aggiungendo una supersorgente e un superpozzo e viceversa.

26.1-3

Supponete che una rete di flusso $G = (V, E)$ violi l'ipotesi che la rete contenga un cammino $s \rightsquigarrow v \rightsquigarrow t$ per ogni vertice $v \in V$. Sia u un vertice dal quale non esiste un cammino $s \rightsquigarrow u \rightsquigarrow t$. Dimostrate che deve esistere un flusso massimo f in G tale che $f(u, v) = f(v, u) = 0$ per ogni vertice $v \in V$.

26.1-4

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Sia f un flusso in una rete e sia α un numero reale. Il ***prodotto scalare del flusso***, indicato con αf, è una funzione da $V \times V$ a $\mathbb{R}$ definita da

$$(\alpha f)(u, v) = \alpha \cdot f(u, v)$$

Dimostrate che i flussi in una rete formano un ***insieme convesso***. Ovvero dimostrate che, se f_1 ed f_2 sono flussi, allora anche $\alpha f_1 + (1 - \alpha) f_2$ è un flusso per ogni α nell'intervallo $0 \leq \alpha \leq 1$.

26.1-5
Definite il problema del flusso massimo come un problema di programmazione lineare.

26.1-6
Il professor Adam ha due figli che, purtroppo, si detestano. Il problema è così grave che non soltanto i due fratelli rifiutano di andare a scuola insieme, ma ciascuno di essi rifiuta perfino di percorrere la stessa strada che quel giorno ha percorso l'altro. I due fratelli non hanno problemi a incrociare i loro percorsi agli angoli delle strade. Fortunatamente, sia la casa del professore sia la scuola si trovano in edifici ad angolo, ma oltre a questo non è sicuro se sarà possibile mandare entrambi i figli alla stessa scuola. Il professore ha una mappa della sua città. Spiegate come può essere formulato il problema di determinare se entrambi i figli del professore potranno andare alla stessa scuola come un problema di flusso massimo.

26.1-7
Supponete che in una rete di flusso, oltre agli archi, anche i vertici abbiano delle capacità, ovvero che ciascun vertice v abbia un limite $l(v)$ sulla quantità di flusso che lo può attraversare. Spiegate come trasformare una rete di flusso $G = (V, E)$ con capacità nei vertici in una rete di flusso equivalente $G' = (V', E')$ senza capacità nei vertici, in modo tale che un flusso massimo in G' abbia lo stesso valore di un flusso massimo in G. Quanti vertici e archi avrà G'?

26.2 Il metodo di Ford-Fulkerson

Questo paragrafo presenta il metodo di Ford-Fulkerson per risolvere il problema del flusso massimo. Lo abbiamo chiamato "metodo", anziché "algoritmo", perché esso ammette varie implementazioni con tempi di esecuzione differenti. Il metodo di Ford-Fulkerson dipende da tre importanti idee che trascendono il metodo stesso e sono rilevanti per molti algoritmi e problemi di flusso: reti residue, cammini aumentanti e tagli. Queste idee sono fondamentali per l'importante teorema del "flusso massimo/taglio minimo" (Teorema 26.6), che caratterizza il valore di un flusso massimo in funzione dei tagli della rete di flusso. Alla fine di questo paragrafo presenteremo una specifica implementazione del metodo di Ford-Fulkerson e analizzeremo il suo tempo di esecuzione.

Il metodo di Ford-Fulkerson aumenta in modo iterativo il valore del flusso. Iniziamo con $f(u, v) = 0$ per ogni $u, v \in V$, assegnando al flusso iniziale un valore 0. A ogni iterazione, incrementiamo il valore del flusso in G cercando un *cammino aumentante* in una "rete residua" associata G_f. Una volta che conosciamo gli archi di un cammino aumentante in G_f, possiamo facilmente identificare degli archi specifici in G ai quali possiamo modificare il flusso in modo tale da aumentare il valore $|f|$ del flusso. Sebbene ogni iterazione del metodo di Ford-Fulkerson faccia aumentare il valore del flusso, vedremo che il flusso in un particolare arco di G può sia aumentare che diminuire; la diminuzione del flusso in alcuni archi potrebbe essere necessaria per consentire a un algoritmo di inviare più flusso dalla sorgente al pozzo. Aumentiamo ripetutamente il flusso finché nella rete residua non ci saranno altri cammini aumentanti. Il teorema del flusso massimo/taglio minimo dimostrerà che, alla fine, questo processo genera un flusso massimo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

FORD-FULKERSON-METHOD(G, s, t)
1 inizializza il flusso f a 0
2 **while** esiste un cammino aumentante p nella rete residua G_f
3 aumenta il flusso f lungo p
4 **return** f

Per implementare e analizzare il metodo di Ford-Fulkerson, è necessario introdurre altri concetti.

Reti residue

Intuitivamente, data una rete di flusso G e un flusso f, la rete residua G_f è composta da archi con capacità che rappresentano come possiamo modificare il flusso negli archi di G. Un arco della rete di flusso può accettare una quantità di flusso aggiuntivo pari alla capacità dell'arco meno il flusso su tale arco. Se tale valore è positivo, inseriamo questo arco in G_f con una "capacità residua" pari a $c_f(u, v) = c(u, v) - f(u, v)$. Gli unici archi di G che si trovano in G_f sono quelli che possono accettare più flusso. Gli archi (u, v) il cui flusso è uguale alla loro capacità hanno $c_f(u, v) = 0$; questi archi non si trovano in G_f.

La rete residua G_f può contenere archi che non si trovano in G. Quando un algoritmo manipola il flusso, con l'obiettivo di aumentare il flusso totale, potrebbe essere necessario ridurre il flusso in un particolare arco. Per rappresentare una possibile riduzione di un flusso positivo $f(u, v)$ in un arco G, poniamo un arco (v, u) in G_f con capacità residua $c_f(v, u) = f(u, v)$ – ovvero un arco che può accettare un flusso nella direzione opposta a (u, v), che al più può annullare il flusso in (u, v). Questi archi opposti nella rete residua consentono a un algoritmo di inviare un flusso opposto a quello che aveva già inviato in un arco. Inviare un flusso opposto equivale a *ridurre* il flusso nell'arco, che è un'operazione necessaria in molti algoritmi.

Più formalmente, supponiamo di avere una rete di flusso $G = (V, E)$ con una sorgente s e un pozzo t. Sia f un flusso in G; consideriamo una coppia di vertici $u, v \in V$. Definiamo ***capacità residua*** $c_f(u, v)$ in questo modo

$$c_f(u, v) = \begin{cases} c(u, v) - f(u, v) & \text{se } (u, v) \in E \\ f(v, u) & \text{se } (v, u) \in E \\ 0 & \text{negli altri casi} \end{cases} \tag{26.2}$$

Poiché l'ipotesi $(u, v) \in E$ implica che $(v, u) \notin E$, un solo caso nell'equazione (26.2) può essere applicato a ciascuna coppia di vertici.

Come esempio dell'equazione (26.2), se $c(u, v) = 16$ e $f(u, v) = 11$, allora possiamo aumentare $f(u, v)$ fino a $c_f(u, v) = 5$ unità prima di superare il vincolo della capacità nell'arco (u, v). Vogliamo anche consentire a un algoritmo di restituire fino a 11 unità di flusso da v a u, e quindi $c_f(v, u) = 11$.

Data una rete di flusso $G = (V, E)$ con un flusso f, la ***rete residua*** di G indotta da f è $G_f = (V, E_f)$, dove

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$E_f = \{(u, v) \in V \times V : c_f(u, v) > 0\} \tag{26.3}$$

Ovvero, come detto in precedenza, ogni arco della rete residua, o ***arco residuo***, può ammettere un flusso che è maggiore di 0. La Figura 26.4(a) riporta la rete di flusso G e il flusso f della Figura 26.1(b); la Figura 26.4(b) illustra la corrispondente rete residua G_f.

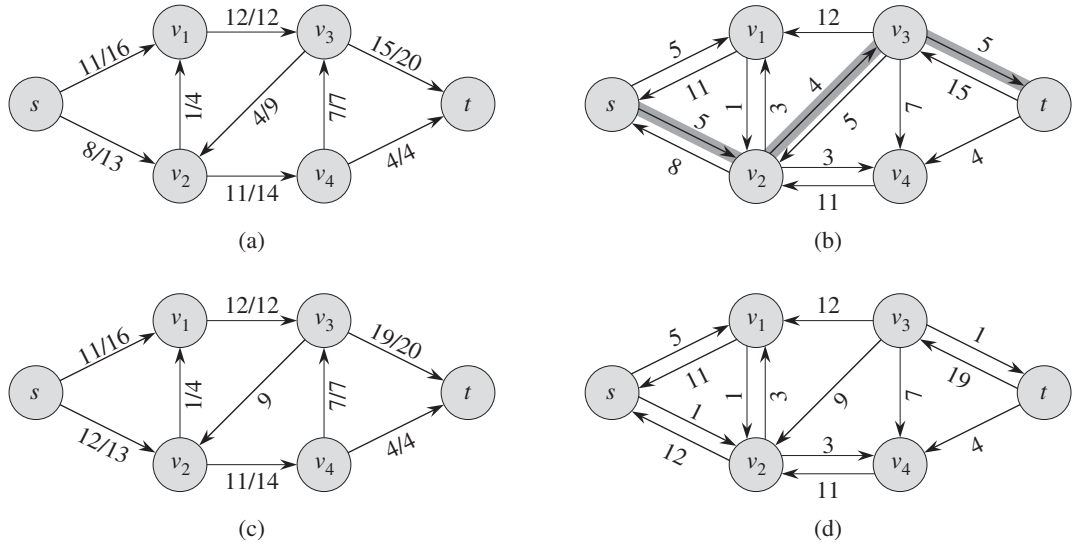


Figura 26.4 **(a)** La rete di flusso G e il flusso f della Figura 26.1(b). **(b)** La rete residua G_f con il cammino aumentante p ombreggiato; la sua capacità residua è $c_f(p) = c_f(v_2, v_3) = 4$. Gli archi con capacità residua pari a 0, come (v_1, v_3), non sono indicati; è questa una convezione che adotteremo in tutto il paragrafo. **(c)** Il flusso in G che si ottiene aumentando il flusso lungo il cammino p della sua capacità residua 4. Gli archi senza flusso, come (v_3, v_2), sono indicati solo con le loro capacità; questa è un'altra convenzione che adotteremo. **(d)** La rete residua indotta dal flusso in (c).

Gli archi in E_f o sono archi di E o sono i loro opposti, e quindi

$$|E_f| \leq 2\,|E|$$

Osservate che la rete residua G_f è simile a una rete di flusso con capacità date da c_f. Non soddisfa la nostra definizione di rete di flusso perché può contenere sia l'arco (u, v) sia il suo opposto (v, u). A parte questa differenza, una rete residua ha le stesse proprietà di una rete di flusso, e possiamo definire un flusso nella rete residua come quello che soddisfa la definizione di flusso, ma rispetto alle capacità c_f nella rete G_f.

Un flusso in una rete residua rappresenta una guida per aggiungere flusso alla rete originale. Se f è un flusso in G ed f' è un flusso nella corrispondente rete residua G_f, definiamo $f \uparrow f'$, l'***aumento*** di f' del flusso f, come una funzione da $V \times V$ a $\mathbb{R}$ in questo modo

$$(f \uparrow f')(u, v) = \begin{cases} f(u, v) + f'(u, v) - f'(v, u) & \text{se } (u, v) \in E \\ 0 & \text{negli altri casi} \end{cases} \tag{26.4}$$

Il concetto che sta alla base di questa definizione deriva dalla definizione di rete residua. Noi aumentiamo il flusso in (u, v) di $f'(u, v)$, ma lo riduciamo di $f'(v, u)$, perché immettere un flusso nell'arco opposto della rete residua significa diminuire il flusso nella rete originale. L'immissione di flusso nell'arco opposto della rete residua è detta anche ***cancellazione***. Per esempio, inviare 5 casse di dischi per hockey da u a v e 2 casse da v a u equivale (ai fini del risultato finale) a inviare 3 casse da u a v e nessuna cassa da v a u. Una cancellazione di questo tipo è fondamentale per qualsiasi algoritmo di flusso massimo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Lemma 26.1

Sia $G = (V, E)$ una rete di flusso con sorgente s e pozzo t, e sia f un flusso in G. Sia G_f la rete residua di G indotta da f e sia f' un flusso in G_f. Allora la funzione $f \uparrow f'$ definita dall'equazione (26.4) è un flusso in G con valore $|f \uparrow f'| = |f| + |f'|$.

Dimostrazione Verifichiamo prima che la funzione $f \uparrow f'$ rispetti il vincolo sulla capacità per ogni vertice in E e la conservazione del flusso in ogni vertice in $V - \{s, t\}$.

Per il vincolo sulla capacità, notiamo intanto che se $(u, v) \in E$, allora $c_f(v, u) = f(u, v)$. Quindi, si ha $f'(v, u) \leq c_f(v, u) = f(u, v)$, ovvero

$$\begin{aligned}
(f \uparrow f')(u, v) &= f(u, v) + f'(u, v) - f'(v, u) && \text{(per l'equazione (26.4))} \\
&\geq f(u, v) + f'(u, v) - f(u, v) && \text{(perché } f'(v, u) \leq f(u, v)\text{)} \\
&= f'(u, v) \\
&\geq 0
\end{aligned}$$

Inoltre

$$\begin{aligned}
&(f \uparrow f')(u, v) \\
&\qquad = f(u, v) + f'(u, v) - f'(v, u) && \text{(per l'equazione (26.4))} \\
&\qquad \leq f(u, v) + f'(u, v) && \text{(perché i flussi sono non negativi)} \\
&\qquad \leq f(u, v) + c_f(u, v) && \text{(vincolo sulla capacità)} \\
&\qquad = f(u, v) + c(u, v) - f(u, v) && \text{(definizione di } c_f\text{)} \\
&\qquad = c(u, v)
\end{aligned}$$

Per la conservazione del flusso, poiché f e f' rispettano la legge della conservazione del flusso, per ogni $u \in V - \{s, t\}$ si ha

$$\begin{aligned}
\sum_{v \in V} (f \uparrow f')(u, v) &= \sum_{v \in V} (f(u, v) + f'(u, v) - f'(v, u)) \\
&= \sum_{v \in V} f(u, v) + \sum_{v \in V} f'(u, v) - \sum_{v \in V} f'(v, u) \\
&= \sum_{v \in V} f(v, u) + \sum_{v \in V} f'(v, u) - \sum_{v \in V} f'(u, v) \\
&= \sum_{v \in V} (f(v, u) + f'(v, u) - f'(u, v)) \\
&= \sum_{v \in V} (f \uparrow f')(v, u)
\end{aligned}$$

La terza riga deriva dalla seconda per la conservazione del flusso.

Infine, calcoliamo il valore di $f \uparrow f'$. Ricordiamo che non sono ammessi archi antiparalleli in G (mentre in G_f lo sono), e quindi per ogni vertice $v \in V$, sappiamo che c'è un arco (s, v) o (v, s), ma mai entrambi. Definiamo $V_1 = \{v : (s, v) \in E\}$ come l'insieme dei vertici con archi che escono da s, e $V_2 = \{v : (v, s) \in E\}$ come l'insieme dei vertici con archi che entrano in s. Abbiamo $V_1 \cup V_2 \subseteq V$ e, poiché non ammettiamo archi antiparalleli, $V_1 \cap V_2 = \emptyset$.

Acquistato da Michele Mo[illegible] 2012-07-02 17:[illegible] numero [illegible] 1[illegible] 2012, McGraw-Hill Education (Italy)

Calcoliamo adesso

$$\begin{aligned} |f \uparrow f'| &= \sum_{v \in V} (f \uparrow f')(s, v) - \sum_{v \in V} (f \uparrow f')(v, s) \\ &= \sum_{v \in V_1} (f \uparrow f')(s, v) - \sum_{v \in V_2} (f \uparrow f')(v, s) \end{aligned} \tag{26.5}$$

La seconda riga deriva dal fatto che $(f \uparrow f')(w, x)$ è 0 se $(w, x) \notin E$. Applichiamo adesso la definizione di $f \uparrow f'$ all'equazione (26.5) e poi riordiniamo e raggruppiamo i termini per ottenere

$$\begin{aligned} &|f \uparrow f'| \\ &= \sum_{v \in V_1} (f(s, v) + f'(s, v) - f'(v, s)) - \sum_{v \in V_2} (f(v, s) + f'(v, s) - f'(s, v)) \\ &= \sum_{v \in V_1} f(s, v) + \sum_{v \in V_1} f'(s, v) - \sum_{v \in V_1} f'(v, s) \\ &\qquad - \sum_{v \in V_2} f(v, s) - \sum_{v \in V_2} f'(v, s) + \sum_{v \in V_2} f'(s, v) \\ &= \sum_{v \in V_1} f(s, v) - \sum_{v \in V_2} f(v, s) \\ &\qquad + \sum_{v \in V_1} f'(s, v) + \sum_{v \in V_2} f'(s, v) - \sum_{v \in V_1} f'(v, s) - \sum_{v \in V_2} f'(v, s) \\ &= \sum_{v \in V_1} f(s, v) - \sum_{v \in V_2} f(v, s) + \sum_{v \in V_1 \cup V_2} f'(s, v) - \sum_{v \in V_1 \cup V_2} f'(v, s) \end{aligned} \tag{26.6}$$

Nell'equazione (26.6) possiamo riferire tutte e quattro le sommatorie a V, in quanto ogni termine aggiuntivo ha valore 0 (l'Esercizio 26.2-1 vi chiede di dimostrare questo formalmente). Quindi, si ha

$$\begin{aligned} |f \uparrow f'| &= \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) + \sum_{v \in V} f'(s, v) - \sum_{v \in V} f'(v, s) \\ &= |f| + |f'| \end{aligned} \tag{26.7}$$

■

Cammini aumentanti

Data una rete di flusso $G = (V, E)$ con un flusso f, un ***cammino aumentante*** p è un cammino semplice da s a t nella rete residua G_f. Per la definizione di rete residua, possiamo aumentare il flusso di un arco (u, v) in un cammino aumentante fino a $c_f(u, v)$, senza violare il vincolo sulla capacità qualunque sia l'arco (u, v) o (v, u) che si trovi nella rete di flusso originale G.

Il cammino ombreggiato nella Figura 26.4(b) è un cammino aumentante. Trattando la rete residua G_f della figura come una rete di flusso, possiamo incrementare al massimo di 4 unità il flusso che attraversa ciascun arco di questo cammino, senza violare nessun vincolo sulla capacità, perché la capacità residua minima lungo questo cammino è $c_f(v_2, v_3) = 4$. La quantità massima di cui può crescere il flusso in ogni arco di un cammino aumentante p è detta ***capacità residua*** di p ed è espressa da

$$c_f(p) = \min \{c_f(u, v) : (u, v) \text{ è in } p\}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950016-220707-0 Copyright.© 2022, McGraw-Hill Education (Italy).

Il seguente lemma (l'Esercizio 26.2-7 vi chiede di dimostrarlo) rende più precise le precedenti considerazioni.

Lemma 26.2
Sia $G = (V, E)$ una rete di flusso, sia f un flusso in G e sia p un cammino aumentante in G_f. Definiamo una funzione $f_p : V \times V \to \mathbb{R}$ come

$$f_p(u, v) = \begin{cases} c_f(p) & \text{se } (u, v) \text{ è in } p \\ 0 & \text{negli altri casi} \end{cases} \tag{26.8}$$

Allora, f_p è un flusso in G_f con valore $|f_p| = c_f(p) > 0$. ■

Il seguente corollario dimostra che, se aggiungiamo f_p a f, otteniamo un altro flusso in G il cui valore è più vicino al massimo. La Figura 26.4(c) illustra il risultato che si ottiene aggiungendo f_p della Figura 26.4(b) a f della Figura 26.4(a); la Figura 26.4(d) mostra la rete residua risultante.

Corollario 26.3
Sia $G = (V, E)$ una rete di flusso, sia f un flusso in G e sia p un cammino aumentante in G_f. Sia f_p una funzione definita secondo l'equazione (26.8); supponiamo inoltre di aumentare f di f_p. Allora la funzione $f \uparrow f_p$ è un flusso in G con valore $|f \uparrow f_p| = |f| + |f_p| > |f|$.

Dimostrazione È una conseguenza immediata dei Lemmi 26.1 e 26.2. ■

Tagli delle reti di flusso

Il metodo di Ford-Fulkerson incrementa ripetutamente il flusso lungo i cammini aumentanti, finché non viene trovato un flusso massimo. Come facciamo a sapere di aver trovato un flusso massimo quando l'algoritmo termina? Il teorema del flusso massimo/taglio minimo, che dimostreremo più avanti, ci dice che un flusso è massimo se e soltanto se la sua rete residua non contiene cammini aumentanti. Per dimostrare questo teorema, dobbiamo prima illustrare il concetto di taglio di una rete di flusso.

Un ***taglio*** (S, T) della rete di flusso $G = (V, E)$ è una partizione di V in S e $T = V - S$ tale che $s \in S$ e $t \in T$ (questa definizione è simile alla definizione di "taglio" che abbiamo utilizzato per gli alberi di connessione minimi nel Capitolo 23, con la differenza che qui stiamo tagliando un grafo orientato, anziché un grafo non orientato, e vogliamo che $s \in S$ e $t \in T$). Se f è un flusso, allora il ***flusso netto*** $f(S, T)$ attraverso il taglio (S, T) è definito come

$$f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u) \tag{26.9}$$

La ***capacità*** del taglio (S, T) è

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$c(S, T) = \sum_{u \in S} \sum_{v \in T} c(u, v) . \tag{26.10}$$

Un ***taglio minimo*** di una rete è un taglio la cui capacità è minima rispetto a tutti i tagli della rete.

L'asimmetria tra le definizioni di flusso e capacità di un taglio è intenzionale e importante. Per quanto riguarda le capacità, noi consideriamo soltanto le capacità

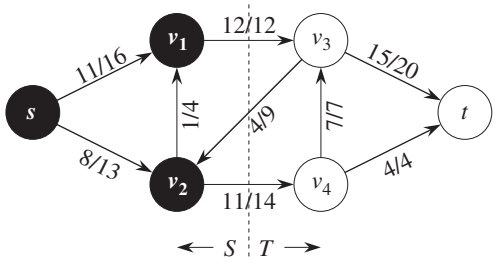


Figura 26.5 Un taglio (S, T) nella rete di flusso della Figura 26.1(b), dove $S = \{s, v_1, v_2\}$ e $T = \{v_3, v_4, t\}$. I vertici in S sono neri; i vertici in T sono bianchi. Il flusso netto attraverso (S, T) è $f(S, T) = 19$ e la capacità è $c(S, T) = 26$.

degli archi che vanno da S a T, ignorando gli archi nella direzione opposta. Per quanto riguarda il flusso, noi consideriamo il flusso che va da S a T meno quello che va nella direzione opposta da T a S. Il motivo di questa differenza sarà chiarito più avanti in questo paragrafo.

La Figura 26.5 illustra il taglio $(\{s, v_1, v_2\}, \{v_3, v_4, t\})$ nella rete di flusso della Figura 26.1(b). Il flusso netto attraverso questo taglio è

$$\begin{aligned} f(v_1, v_3) + f(v_2, v_4) - f(v_3, v_2) &= 12 + 11 - 4 \\ &= 19 \end{aligned}$$

e la sua capacità è

$$\begin{aligned} c(v_1, v_3) + c(v_2, v_4) &= 12 + 14 \\ &= 26 \end{aligned}$$

Il seguente lemma dimostra che, per un dato flusso f, il flusso netto attraverso un taglio qualsiasi è lo stesso ed è uguale a $|f|$, il valore del flusso.

Lemma 26.4
Sia f un flusso in una rete di flusso G con sorgente s e pozzo t; sia (S, T) un taglio di G. Allora il flusso netto attraverso (S, T) è $f(S, T) = |f|$.

Dimostrazione Possiamo riscrivere la condizione della conservazione del flusso per un nodo qualsiasi $u \in V - \{s, t\}$ in questo modo

$$\sum_{v \in V} f(u, v) - \sum_{v \in V} f(v, u) = 0 \tag{26.11}$$

Prendendo la definizione di $|f|$ dall'equazione (26.1) e aggiungendo il membro sinistro dell'equazione (26.11), che è uguale a 0, sommando per tutti i vertici in $S - \{s\}$, si ottiene

$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) + \sum_{u \in S - \{s\}} \left(\sum_{v \in V} f(u, v) - \sum_{v \in V} f(v, u) \right)$$

Sviluppando la sommatoria destra e riorganizzando i termini, si ha

$$\begin{aligned} |f| &= \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) + \sum_{u \in S - \{s\}} \sum_{v \in V} f(u, v) - \sum_{u \in S - \{s\}} \sum_{v \in V} f(v, u) \\ &= \sum_{v \in V} \left(f(s, v) + \sum_{u \in S - \{s\}} f(u, v) \right) - \sum_{v \in V} \left(f(v, s) + \sum_{u \in S - \{s\}} f(v, u) \right) \\ &= \sum_{v \in V} \sum_{u \in S} f(u, v) - \sum_{v \in V} \sum_{u \in S} f(v, u) \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995303019-2207070 Copyright © 2022, McGraw-Hill Education (Italy)

Poiché $V = S \cup T$ e $S \cap T = \emptyset$, possiamo scomporre ciascuna sommatoria su V in sommatorie su S e T, ottenendo

$$\begin{aligned}
|f| &= \sum_{v \in S}\sum_{u \in S} f(u,v) + \sum_{v \in T}\sum_{u \in S} f(u,v) - \sum_{v \in S}\sum_{u \in S} f(v,u) - \sum_{v \in T}\sum_{u \in S} f(v,u) \\
&= \sum_{v \in T}\sum_{u \in S} f(u,v) - \sum_{v \in T}\sum_{u \in S} f(v,u) \\
&\qquad + \left(\sum_{v \in S}\sum_{u \in S} f(u,v) - \sum_{v \in S}\sum_{u \in S} f(v,u) \right)
\end{aligned}$$

Le due sommatorie all'interno delle parentesi sono uguali, perché per ogni vertice $x, y \in V$, il termine $f(x,y)$ appare una sola volta in ciascuna sommatoria. Dunque, queste sommatorie si annullano e si ha

$$\begin{aligned}
|f| &= \sum_{u \in S}\sum_{v \in T} f(u,v) - \sum_{u \in S}\sum_{v \in T} f(v,u) \\
&= f(S,T)
\end{aligned}$$

■

Un corollario del Lemma 26.4 mostra come utilizzare le capacità per limitare il valore di un flusso.

Corollario 26.5
Il valore di un flusso qualsiasi f in una rete di flusso G è limitato superiormente dalla capacità di un taglio qualsiasi di G.

Dimostrazione Sia (S,T) un taglio qualsiasi di G e sia f un flusso qualsiasi. Per il Lemma 26.4 e i vincoli sulla capacità, si ha

$$\begin{aligned}
|f| &= f(S,T) \\
&= \sum_{u \in S}\sum_{v \in T} f(u,v) - \sum_{u \in S}\sum_{v \in T} f(v,u) \\
&\le \sum_{u \in S}\sum_{v \in T} f(u,v) \\
&\le \sum_{u \in S}\sum_{v \in T} c(u,v) \\
&= c(S,T)
\end{aligned}$$

■

Una conseguenza immediata del Corollario 26.5 è che il flusso massimo in una rete è limitato superiormente dalla capacità di un taglio minimo della rete. L'importante teorema del flusso massimo/taglio minimo, che adesso definiamo e dimostriamo, dice che il valore di un flusso massimo in effetti è uguale alla capacità di un taglio minimo.

Acquistato da Michele Michele su Pearson+ il 2022-07-07 23:12 Numero Ordine Libreria: 2023010210207-0 Copyright © 2022, McGraw-Hill Education (Italy)

Teorema 26.6 (Teorema del flusso massimo/taglio minimo)
Se f è un flusso in una rete di flusso $G = (V,E)$ con sorgente s e pozzo t, allora le seguenti condizioni sono equivalenti:

1. f è un flusso massimo in G.
2. La rete residua G_f non contiene cammini aumentanti.
3. $|f| = c(S,T)$ per qualche taglio (S,T) di G.

Dimostrazione (1) $\Rightarrow$ (2): supponiamo per assurdo che f sia un flusso massimo in G, ma che G_f abbia un cammino aumentante p. Allora, per il Corollario 26.3, la somma dei flussi $f + f_p$, dove f_p è dato dall'equazione (26.8), è un flusso in G con valore strettamente maggiore di $|f|$, contraddicendo l'ipotesi che f sia un flusso massimo.

(2) $\Rightarrow$ (3): supponiamo che G_f non abbia cammini aumentanti, ovvero che G_f non contenga un cammino da s a t. Definiamo

$$S = \{v \in V : \text{esiste un cammino da } s \text{ a } v \text{ in } G_f\}$$

e $T = V - S$. La partizione (S, T) è un taglio: banalmente si ha $s \in S$ e $t \notin S$ perché non esiste un cammino da s a t in G_f. Consideriamo due vertici $u \in S$ e $v \in T$. Se $(u, v) \in E$ deve essere $f(u, v) = c(u, v)$ altrimenti $(u, v) \in E_f$ e quindi v dovrebbe stare in S. Se $(v, u) \in E$, dobbiamo avere $f(v, u) = 0$, perché altrimenti $c_f(u, v) = f(v, u)$ sarebbe positivo e si avrebbe $(u, v) \in E_f$, e quindi v dovrebbe stare in S. Ovviamente, se né (u, v) né (v, u) si trovano in E, allora $f(u, v) = f(v, u) = 0$; quindi, si ha

$$\begin{aligned} f(S, T) &= \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{v \in T} \sum_{u \in S} f(v, u) \\ &= \sum_{u \in S} \sum_{v \in T} c(u, v) - \sum_{v \in T} \sum_{u \in S} 0 \\ &= c(S, T) \end{aligned}$$

Dunque, per il Lemma 26.4, si ha $|f| = f(S, T) = c(S, T)$.

(3) $\Rightarrow$ (1): per il Corollario 26.5, $|f| \leq c(S, T)$ per ogni taglio (S, T). La condizione $|f| = c(S, T)$ quindi implica che f è un flusso massimo. ■

L'algoritmo di base di Ford-Fulkerson

In ogni iterazione del metodo di Ford-Fulkerson, troviamo *qualche* cammino aumentante p e utilizziamo p per modificare il flusso f. Come suggeriscono il Lemma 26.2 e il Corollario 26.3, sostituiamo f con $f \uparrow f_p$, ottenendo un nuovo flusso il cui valore è $|f| + |f_p|$. La seguente implementazione del metodo calcola il flusso massimo in una rete di flusso $G = (V, E)$ aggiornando l'attributo del flusso $(u, v).f$ per ogni arco $(u, v) \in E$.[1] Se $(u, v) \notin E$, assumiamo implicitamente che $(u, v).f = 0$. Supponiamo inoltre che le capacità $c(u, v)$ vengano fornite con la rete di flusso e che $c(u, v) = 0$ se $(u, v) \notin E$. La capacità residua $c_f(u, v)$ è calcolata secondo la formula (26.2). L'espressione $c_f(p)$ nel codice in effetti è una variabile temporanea che memorizza la capacità residua del cammino p.

```
FORD-FULKERSON(G, s, t)
1  for ogni arco (u, v) ∈ G.E
2      (u, v).f = 0
3  while esiste un cammino p da s a t nella rete residua G_f
4      c_f(p) = min {c_f(u, v) : (u, v) è in p}
5      for ogni arco (u, v) in p
6          if (u, v) ∈ E
7              (u, v).f = (u, v).f + c_f(p)
8          else (v, u).f = (v, u).f − c_f(p)
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950316-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[1] Come detto nel Paragrafo 22.1, un attributo f per l'arco (u, v) è rappresentato con la stessa notazione – $(u, v).f$ – che abbiamo adottato per l'attributo di un oggetto qualsiasi.

L'algoritmo FORD-FULKERSON espande semplicemente lo pseudocodice FORD-FULKERSON-METHOD precedentemente descritto. La Figura 26.6 illustra il risultato di ciascuna iterazione durante un'esecuzione di prova. La righe 1–2 inizializzano il flusso f a 0. Il ciclo **while** delle righe 3–8 trova ripetutamente un cammino aumentante p in G_f e incrementa il flusso f lungo p della capacità residua $c_f(p)$. Ogni arco residuo nel cammino p può essere un arco della rete originale o l'opposto di un arco della rete originale. Le righe 6–8 aggiornano appropriatamente il flusso in ciascun caso, sommando il flusso quando l'arco residuo è un arco originale e sottraendo il flusso nell'altro caso. Quando non esistono più cammini aumentanti, il flusso f è un flusso massimo.

Analisi dell'algoritmo di Ford-Fulkerson

Il tempo di esecuzione dell'algoritmo FORD-FULKERSON dipende dal modo in cui viene trovato il cammino aumentante p nella riga 3. Se il cammino non viene scelto bene, l'algoritmo potrebbe anche non terminare: il valore del flusso crescerà per i successivi aumenti, ma non convergerà necessariamente al valore del flusso massimo.[2] Tuttavia, se il cammino aumentante viene scelto utilizzando una visita in ampiezza (descritta nel Paragrafo 22.2), l'algoritmo viene eseguito in tempo polinomiale. Prima di dimostrare questo risultato, però, troviamo un semplice limite per il caso in cui il cammino aumentante sia scelto arbitrariamente e tutte le capacità siano numeri interi.

Molto spesso nella pratica, il problema del flusso massimo si presenta con capacità espresse da numeri interi. Se le capacità sono numeri razionali, è possibile effettuare un'appropriata conversione di scala per trasformarle tutte in numeri interi. Se f^* indica un flusso massimo nella rete trasformata, allora un'implementazione semplice di FORD-FULKERSON esegue il ciclo **while** (righe 3–8) al massimo $|f^*|$ volte, perché il valore del flusso aumenta di almeno un'unità in ogni iterazione.

Il lavoro all'interno del ciclo **while** può essere svolto in modo efficiente se implementiamo la rete di flusso $G = (V, E)$ con la struttura dati appropriata e troviamo un cammino aumentante con un algoritmo che richiede tempo lineare. Supponiamo di mantenere una struttura dati che corrisponde a un grafo orientato $G' = (V, E')$, dove $E' = \{(u, v) : (u, v) \in E \text{ o } (v, u) \in E\}$. Gli archi nella rete G sono anche archi in G', pertanto è facile mantenere capacità e flussi in questa struttura dati.

Dato un flusso f su G, gli archi nella rete residua G_f sono costituiti da tutti gli archi (u, v) di G' tali che $c_f(u, v) > 0$, dove c_f è conforme all'equazione (26.2). Il tempo per trovare un cammino in una rete residua è quindi $O(V + E') = O(E)$ utilizzando sia una visita in profondità che una visita in ampiezza. Quindi, ogni iterazione del ciclo **while** richiede un tempo $O(E)$; ne consegue che il tempo totale di esecuzione di FORD-FULKERSON è $O(E\ |f^*|)$.

Quando le capacità sono numeri interi e il valore del flusso ottimo $|f^*|$ è piccolo, il tempo di esecuzione dell'algoritmo di Ford-Fulkerson è buono. La Figura 26.7(a) illustra un esempio di ciò che può accadere in una semplice rete di flusso se $|f^*|$ è grande. Un flusso massimo in questa rete ha valore 2 000 000: un milione di unità di flusso attraversano il cammino $s \to u \to t$ e un milione di unità di flusso attraversano il cammino $s \to v \to t$. Se il primo cammino aumentante

Acquistato da Michele Mancini (mancini.michele@gmail.com) - Non riproducibile - Copyright © 2010 The McGraw-Hill Education (Italy)

[2] Il metodo di Ford-Fulkerson potrebbe non terminare soltanto se le capacità degli archi sono numeri irrazionali.

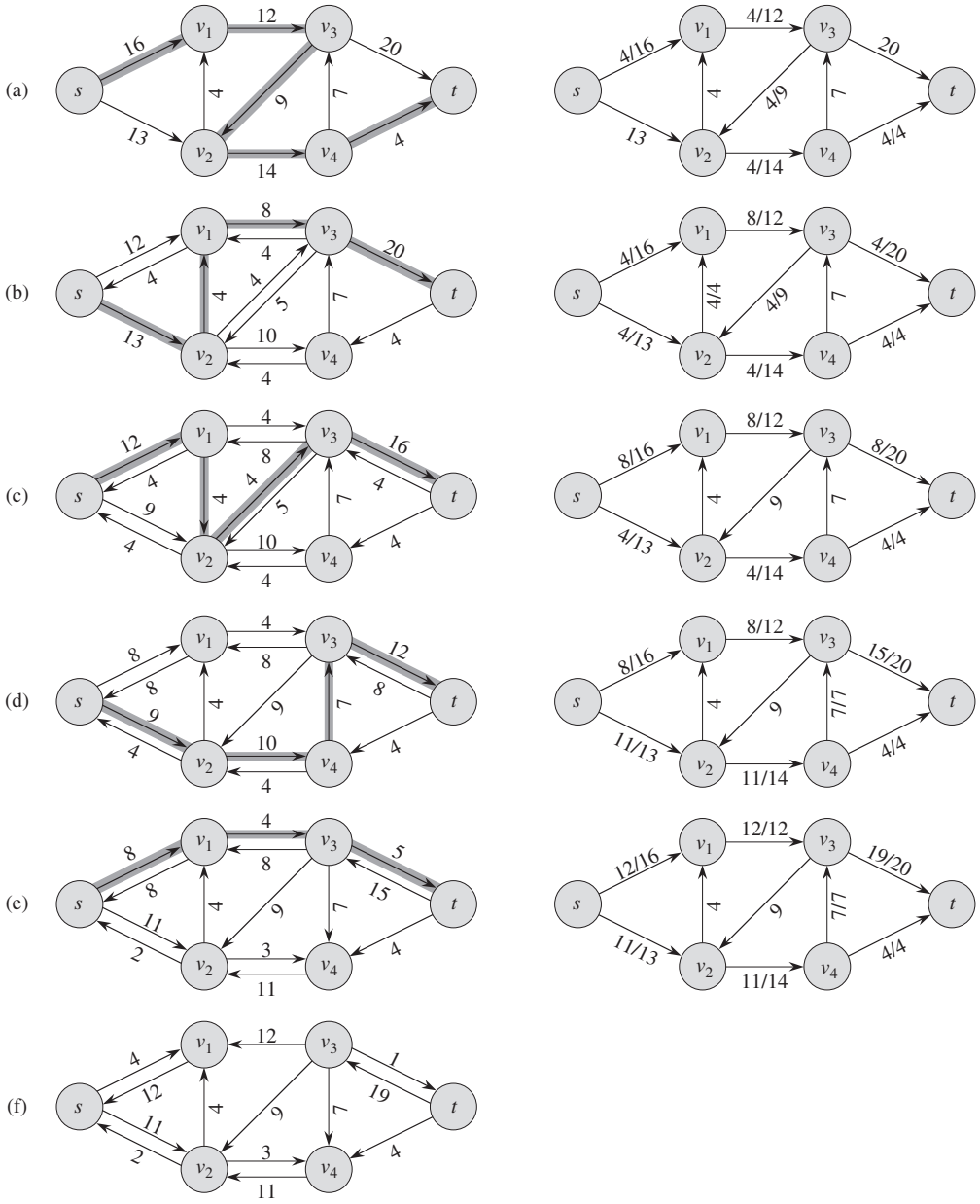


Figura 26.6 L'esecuzione dell'algoritmo di base di Ford-Fulkerson. **(a)–(e)** Le successive iterazioni del ciclo **while**. Il lato sinistro di ciascuna parte mostra la rete residua G_f della riga 3 con un cammino aumentante p ombreggiato. Il lato destro di ciascuna parte mostra il nuovo flusso f che si ottiene sommando f_p a f. La rete residua in (a) è la rete di input G. **(f)** La rete residua quando viene eseguito l'ultimo test del ciclo **while**. La rete non ha cammini aumentanti, quindi il flusso f illustrato in (e) è un flusso massimo. Il valore del flusso massimo trovato è 23.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

trovato da FORD-FULKERSON è $s \to u \to v \to t$, illustrato nella Figura 26.7(a), il flusso ha valore 1 dopo la prima iterazione. La rete residua risultante è illustrata nella Figura 26.7(b). Se la seconda iterazione trova il cammino aumentante $s \to v \to u \to t$, illustrato nella Figura 26.7(b), il flusso ha valore 2. La Figura 26.7(c) illustra la rete residua risultante. Possiamo continuare, scegliendo il

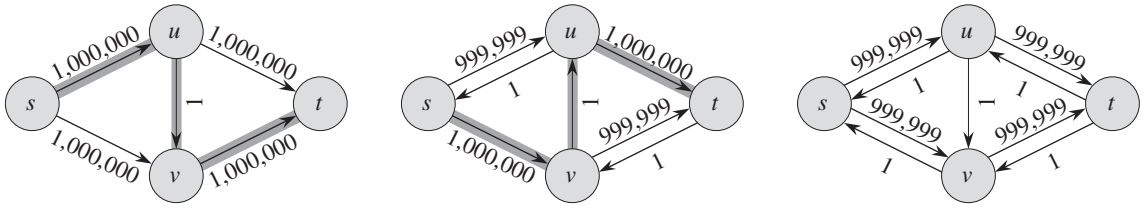


Figura 26.7 **(a)** Una rete di flusso per la quale l'algoritmo di FORD-FULKERSON può richiedere un tempo $\Theta(E\,|f^*|)$, dove f^* è un flusso massimo, qui indicato con $|f^*| = 2\,000\,000$. Il cammino ombreggiato è un cammino aumentante con capacità residua 1. **(b)** La rete residua risultante con un altro cammino aumentante con capacità residua 1. **(c)** La rete residua risultante.

cammino aumentante $s \to u \to v \to t$ nelle iterazioni dispari e il cammino aumentante $s \to v \to u \to t$ nelle iterazioni pari. Effettueremmo un totale di due milioni di aumenti, con un incremento di una sola unità del valore del flusso in ciascun aumento.

Algoritmo di Edmonds-Karp

Il limite per la procedura FORD-FULKERSON può essere migliorato se implementiamo il calcolo del cammino aumentante p nella riga 3 con una visita in ampiezza, cioè se il cammino aumentante è un *cammino minimo* da s a t nella rete residua, dove ogni arco ha distanza (peso) di valore unitario. Chiameremo il metodo di Ford-Fulkerson così implementato ***algoritmo di Edmonds-Karp***. Adesso dimostriamo che l'algoritmo di Edmonds-Karp viene eseguito nel tempo $O(VE^2)$.

L'analisi dipende dalle distanze dei vertici nella rete residua G_f. Il seguente lemma usa la notazione $\delta_f(u, v)$ per la distanza del cammino minimo da u a v in G_f, dove ogni arco ha distanza unitaria.

Lemma 26.7
Se l'algoritmo di Edmonds-Karp viene eseguito su una rete di flusso $G = (V, E)$ con sorgente s e pozzo t, allora per ogni vertice $v \in V - \{s, t\}$, la distanza del cammino minimo $\delta_f(s, v)$ nella rete residua G_f aumenta monotonicamente per ogni aumento di flusso.

Dimostrazione Supponiamo per assurdo che per qualche vertice $v \in V - \{s, t\}$ ci sia un aumento di flusso che provoca una diminuzione della distanza del cammino minimo da s a v. Sia f il flusso appena prima del primo aumento che riduce una distanza del cammino minimo; sia f' il flusso subito dopo. Se v è il vertice con il minimo $\delta_{f'}(s, v)$ la cui distanza è stata ridotta dall'aumento, allora $\delta_{f'}(s, v) < \delta_f(s, v)$. Se $p = s \rightsquigarrow u \to v$ è un cammino minimo da s a v in $G_{f'}$, allora $(u, v) \in E_{f'}$ e

$$\delta_{f'}(s, u) = \delta_{f'}(s, v) - 1 \tag{26.12}$$

Acquistato da Michele Miffre sul Website il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

Per il modo in cui abbiamo scelto v, sappiamo che la distanza del vertice u dalla sorgente s non è aumentata, ovvero

$$\delta_{f'}(s, u) \geq \delta_f(s, u) \tag{26.13}$$

Noi asseriamo che $(u, v) \notin E_f$. Perché? Se avessimo $(u, v) \in E_f$, allora dovremmo avere anche

$$\begin{aligned}
\delta_f(s, v) &\leq \delta_f(s, u) + 1 && \text{(per il Lemma 24.10, la disuguaglianza triangolare)} \\
&\leq \delta_{f'}(s, u) + 1 && \text{(per la disequazione (26.13))} \\
&= \delta_{f'}(s, v) && \text{(per l'equazione (26.12))}
\end{aligned}$$

Questo contraddice l'ipotesi che $\delta_{f'}(s, v) < \delta_f(s, v)$.

Com'è possibile avere $(u, v) \notin E_f$ e $(u, v) \in E_{f'}$? L'aumento deve avere incrementato il flusso da v a u. L'algoritmo di Edmonds-Karp aumenta sempre il flusso lungo i cammini minimi, e quindi il cammino minimo da s a u in G_f ha (v, u) come suo ultimo arco; pertanto, si ha

$$\begin{aligned}
\delta_f(s, v) &= \delta_f(s, u) - 1 \\
&\leq \delta_{f'}(s, u) - 1 && \text{(per la disequazione (26.13))} \\
&= \delta_{f'}(s, v) - 2 && \text{(per l'equazione (26.12))}
\end{aligned}$$

Questo contraddice l'ipotesi che $\delta_{f'}(s, v) < \delta_f(s, v)$. Concludiamo che l'ipotesi che un tale vertice v esista non è corretta.

■

Il prossimo teorema limita il numero di iterazioni dell'algoritmo di Edmonds-Karp.

Teorema 26.8
Se l'algoritmo di Edmonds-Karp viene eseguito su una rete di flusso $G = (V, E)$ con sorgente s e pozzo t, allora il numero totale di aumenti di flusso effettuati dall'algoritmo è $O(VE)$.

Dimostrazione Diciamo che un arco (u, v) di una rete residua G_f è ***critico*** in un cammino aumentante p se la capacità residua di p è la capacità residua di (u, v), cioè se $c_f(p) = c_f(u, v)$. Dopo avere aumentato il flusso lungo un cammino aumentante, ogni arco critico lungo il cammino scompare dalla rete residua. Inoltre, in ogni cammino aumentante ci deve essere almeno un arco critico. Dimostreremo che ciascuno degli $|E|$ archi può diventare critico al più $|V|/2$ volte.

Siano u e v due vertici in V che sono collegati da un arco in E. Poiché i cammini aumentanti sono cammini minimi, quando (u, v) diventa critico per la prima volta, si ha

$$\delta_f(s, v) = \delta_f(s, u) + 1$$

Una volta che il flusso viene aumentato, l'arco (u, v) scompare dalla rete residua; non potrà riapparire successivamente in un altro cammino aumentante finché il flusso da u a v non diminuirà, e questo accade soltanto se (v, u) appare in un cammino aumentante. Se f' è il flusso in G quando si verifica questo evento, allora si ha

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\delta_{f'}(s, u) = \delta_{f'}(s, v) + 1$$

Poiché $\delta_f(s, v) \leq \delta_{f'}(s, v)$ per il Lemma 26.7, si ha

$$\begin{aligned}
\delta_{f'}(s, u) &= \delta_{f'}(s, v) + 1 \\
&\geq \delta_f(s, v) + 1 \\
&= \delta_f(s, u) + 2
\end{aligned}$$

Di conseguenza, dall'istante in cui (u, v) diventa critico all'istante in cui diventa di nuovo critico, la distanza di u dalla sorgente aumenta di almeno 2 unità. La distanza di u dalla sorgente, inizialmente, è almeno pari a 0. I vertici intermedi in un cammino minimo da s a u non possono contenere s, u o t (perché (u, v) nel cammino critico implica che $u \neq t$). Pertanto, fino al momento in cui u diventa irraggiungibile dalla sorgente, se mai questo si verifica, la sua distanza è al massimo $|V| - 2$. Quindi, dopo la prima volta che (u, v) diventa critico esso può ridiventarlo al più altre $(|V| - 2)/2 = |V|/2 - 1$ volte, per un totale di al più $|V|/2$ volte. Poiché ci sono $O(E)$ coppie di vertici che possono essere connessi da un arco in un grafo residuo, il numero totale di archi critici durante l'intera esecuzione dell'algoritmo di Edmonds-Karp è $O(VE)$. Ogni cammino aumentante ha almeno un arco critico, e quindi il teorema è dimostrato. ■

Poiché ogni iterazione di FORD-FULKERSON può essere implementata con tempo $O(E)$ quando il cammino aumentante viene trovato con una visita in ampiezza, il tempo totale di esecuzione dell'algoritmo di Edmonds-Karp è $O(VE^2)$. Vedremo che con gli algoritmi push-relabel si possono ottenere limiti migliori. L'algoritmo del Paragrafo 26.4 fornisce un metodo per raggiungere un tempo di esecuzione $O(V^2E)$, che costituisce la base per l'algoritmo con tempo $O(V^3)$ del Paragrafo 26.5.

Esercizi

26.2-1

Dimostrate che le sommatorie nell'equazione (26.6) sono uguali alle sommatorie nell'equazione (26.7).

26.2-2

Qual è il flusso attraverso il taglio $(\{s, v_2, v_4\}, \{v_1, v_3, t\})$ nella Figura 26.1(b)? Qual è la capacità di questo taglio?

26.2-3

Illustrate l'esecuzione dell'algoritmo di Edmonds-Karp sulla rete di flusso della Figura 26.1(a).

26.2-4

Nell'esempio della Figura 26.6, qual è il taglio minimo che corrisponde al flusso massimo indicato nella figura? Tra i cammini aumentanti che figurano nell'esempio, quali cancellano del flusso?

26.2-5

Ricordiamo che la costruzione descritta nel Paragrafo 26.1, che trasforma una rete di flusso con più sorgenti e pozzi in una rete di flusso con una sola sorgente e un solo pozzo, aggiunge degli archi con capacità infinita. Dimostrate che un flusso qualsiasi nella rete risultante ha un valore finito se gli archi della rete originale con più sorgenti e pozzi hanno capacità finita.

Acquistato da Michele Mi[illegible] Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

26.2-6

Supponete che ogni sorgente s_i in una rete con più sorgenti e pozzi produca esattamente p_i unità di flusso, cosicché $\sum_{v \in V} f(s_i, v) = p_i$. Supponete inoltre che ogni pozzo t_j consumi esattamente q_j unità, cosicché $\sum_{v \in V} f(v, t_j) = q_j$, dove $\sum_i p_i = \sum_j q_j$. Spiegate come trasformare il problema di trovare un flusso f che rispetta questi vincoli addizionali nel problema di trovare un flusso massimo in una rete di flusso con una sola sorgente e un solo pozzo.

26.2-7
Dimostrate il Lemma 26.2.

26.2-8
Supponete di ridefinire la rete residua eliminando gli archi che entrano in s. Dimostrate che la procedura FORD-FULKERSON calcola ancora correttamente un flusso massimo.

26.2-9
Supponendo che f e f' siano entrambi flussi in una rete G, si vuole calcolare il flusso $f \uparrow f'$. Il flusso risultante soddisfa la proprietà della conservazione del flusso? Soddisfa il vincolo sulla capacità?

26.2-10
Dimostrate che un flusso massimo in una rete $G = (V, E)$ può essere sempre trovato con una sequenza di non più di $|E|$ cammini aumentanti (*suggerimento:* scegliete i cammini *dopo* avere trovato il flusso massimo).

26.2-11
L'***arcoconnettività*** di un grafo non orientato è il minimo numero k di archi che devono essere rimossi per rendere il grafo non connesso. Per esempio, l'arcoconnettività di un albero è 1 e l'arcoconnettività di una catena ciclica di vertici è 2. Spiegate come determinare l'arcoconnettività di un grafo non orientato $G = (V, E)$ eseguendo un algoritmo di flusso massimo su al più $|V|$ reti di flusso, ciascuna con $O(V)$ vertici e $O(E)$ archi.

26.2-12
Supponete che una rete di flusso G abbia degli archi che entrano nella sorgente s e che le capacità degli archi siano intere. Sia f un flusso in G dove uno degli archi (v, s) che entra nella sorgente ha $f(v, s) = 1$. Dimostrate che deve esistere un altro flusso f' con $f'(v, s) = 0$ tale che $|f| = |f'|$. Scrivete un algoritmo che calcola f', noto f, nel tempo $O(E)$.

26.2-13
Supponete di voler trovare, fra tutti i tagli minimi in una rete di flusso G con capacità intere, il taglio che contiene il minor numero di archi. Spiegate come modificare le capacità di G per creare una nuova rete di flusso G' dove un taglio minimo qualsiasi in G' è un taglio minimo con il minor numero di archi in G.

26.3 Abbinamento massimo nei grafi bipartiti

Alcuni problemi combinatoriali possono essere facilmente trattati come problemi di flusso massimo. Ne è un esempio il problema del flusso massimo con più sorgenti e pozzi che abbiamo descritto nel Paragrafo 26.1. Ci sono altri problemi combinatoriali che in apparenza hanno poco a che fare con le reti di flusso, ma che in effetti possono essere ricondotti a problemi di flusso massimo. Questo paragrafo presenta uno di tali problemi: trovare un abbinamento massimo in un grafo bipartito. Per risolvere questo problema, sfrutteremo una proprietà di integralità fornita dal metodo di Ford-Fulkerson. Vedremo anche che il metodo di Ford-Fulkerson può essere utilizzato per risolvere il problema dell'abbinamento massimo in un grafo $G = (V, E)$ nel tempo $O(VE)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 1995030016-226707-0. Copyright © 2022. McGraw-Hill Education (Italy).

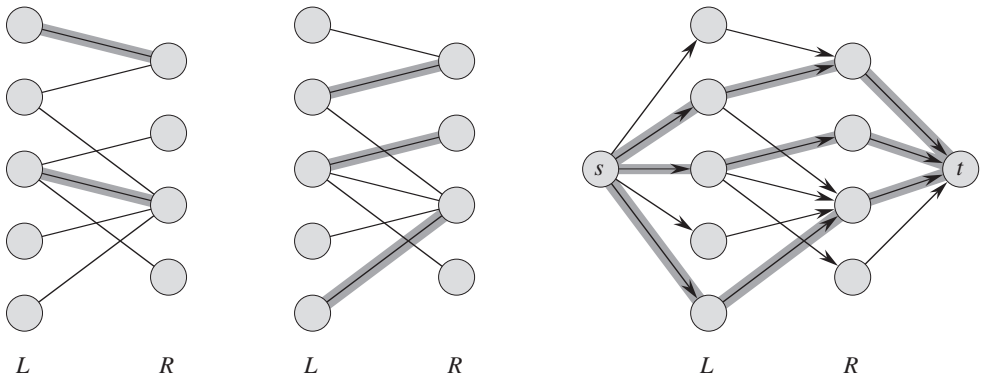


Figura 26.8 Un grafo bipartito $G = (V, E)$ con partizione dei vertici $V = L \cup R$. **(a)** Un abbinamento con cardinalità 2, indicato dagli archi ombreggiati. **(b)** Un abbinamento massimo con cardinalità 3. **(c)** La rete di flusso corrispondente G', dove è indicato un flusso massimo. Ogni arco ha capacità 1. Gli archi ombreggiati hanno un flusso 1; tutti gli altri non hanno flusso. Gli archi ombreggiati da L a R corrispondono a quelli dell'abbinamento massimo di (b).

Il problema dell'abbinamento massimo nei grafi bipartiti

Dato un grafo non orientato $G = (V, E)$, un ***abbinamento*** (matching) è un sottoinsieme di archi $M \subseteq E$ tale che, per ogni vertice $v \in V$, al massimo un arco di M sia incidente su v. Diciamo che un vertice $v \in V$ è ***accoppiato*** dall'abbinamento M se qualche arco in M è incidente su v; altrimenti, v è un vertice ***non accoppiato***. Un ***abbinamento massimo*** è un abbinamento con cardinalità massima, cioè un abbinamento M tale che, per qualsiasi abbinamento M', si abbia $|M| \geq |M'|$. In questo paragrafo limiteremo la nostra analisi alla ricerca degli abbinamenti massimi nei grafi bipartiti: grafi in cui l'insieme dei vertici può essere partizionato in $V = L \cup R$, dove L ed R sono insiemi disgiunti e ogni arco in E collega L ed R. Supponiamo inoltre che ogni vertice in V abbia almeno un arco incidente. La Figura 26.8 illustra il concetto di abbinamento in un grafo bipartito.

Il problema di trovare un abbinamento massimo in un grafo bipartito ha molte applicazioni pratiche. Consideriamo, per esempio, il problema di abbinare un insieme L di macchine a un insieme R di processi da eseguire contemporaneamente. La presenza dell'arco (u, v) in E significa che una particolare macchina $u \in L$ è in grado di eseguire un particolare processo $v \in R$. Un abbinamento massimo fornisce lavoro al maggior numero possibile di macchine.

Trovare un abbinamento massimo nei grafi bipartiti

Possiamo utilizzare il metodo di Ford-Fulkerson per trovare un abbinamento massimo in un grafo $G = (V, E)$ non orientato bipartito in un tempo polinomiale in $|V|$ e $|E|$. La tecnica consiste nel costruire una rete di flusso i cui flussi corrispondono agli abbinamenti, come illustra la Figura 26.8(c). Definiamo la ***rete di flusso associata*** $G' = (V', E')$ al grafo bipartito G nel modo seguente. Siano la sorgente s e il pozzo t due nuovi vertici (che non appartengono a V), e poniamo $V' = V \cup \{s, t\}$. Se la partizione dei vertici di G è $V = L \cup R$, gli archi orientati di G' sono gli archi di E, orientati da L a R, assieme a $|V|$ nuovi archi:

$$E' = \{(s, u) : u \in L\} \cup \{(u, v) : (u, v) \in E\} \cup \{(v, t) : v \in R\}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Per completare la costruzione, assegniamo una capacità unitaria a ciascun arco in E'. Poiché ogni vertice in V ha almeno un arco incidente, allora $|E| \geq |V|/2$. Quindi $|E| \leq |E'| = |E| + |V| \leq 3|E|$, e dunque $|E'| = \Theta(E)$.

Il seguente lemma dimostra che un abbinamento in G corrisponde direttamente a un flusso nella rete di flusso G' associata al grafo G. Diciamo che un flusso f in una rete di flusso $G = (V, E)$ è ***a valori interi*** se $f(u, v)$ è un intero per ogni $(u, v) \in V \times V$.

Lemma 26.9
Sia $G = (V, E)$ un grafo bipartito con partizione dei vertici $V = L \cup R$; sia $G' = (V', E')$ la sua rete di flusso associata. Se M è un abbinamento in G, allora esiste un flusso a valori interi f in G' con valore $|f| = |M|$. Viceversa, se f è un flusso a valori interi in G', allora c'è un abbinamento M in G con cardinalità $|M| = |f|$.

Dimostrazione Dimostriamo prima che un abbinamento M in G corrisponde a un flusso f a valori interi in G'. Definiamo f nel modo seguente. Se $(u, v) \in M$, allora $f(s, u) = f(u, v) = f(v, t) = 1$. Per tutti gli altri archi $(u, v) \in E'$, definiamo $f(u, v) = 0$. È facile verificare che f soddisfa i vincoli sulla capacità e la conservazione del flusso.

Intuitivamente, ogni arco $(u, v) \in M$ corrisponde a un'unità di flusso in G' che attraversa il cammino $s \to u \to v \to t$. Inoltre, i cammini indotti dagli archi in M sono disgiunti sui vertici, tranne che per s e t. Il flusso netto attraverso il taglio $(L \cup \{s\}, R \cup \{t\})$ è uguale a $|M|$; quindi, per il Lemma 26.4, il valore del flusso è $|f| = |M|$.

Per dimostrare l'opposto, sia f un flusso a valori interi in G' e sia

$$M = \{(u, v) : u \in L,\ v \in R,\ \text{and } f(u, v) > 0\}$$

Ogni vertice $u \in L$ ha un solo arco entrante, che è (s, u), la cui capacità è 1. Quindi, ogni $u \in L$ ha al più un'unità di flusso positivo entrante e, se entra un'unità di flusso positivo, per la conservazione del flusso, deve uscire un'unità di flusso positivo. Inoltre, poiché f è a valori interi, per ogni $u \in L$, l'unità di flusso può entrare al più in un arco e può uscire al più in un arco. Quindi, un'unità di flusso positivo entra in u se e soltanto se c'è un solo vertice $v \in R$ tale che $f(u, v) = 1$; inoltre c'è al più un arco uscente da ciascun vertice $u \in L$ che ha un flusso positivo. Un ragionamento simmetrico può essere fatto per ogni $v \in R$. L'insieme M è quindi un abbinamento.

Per verificare che $|M| = |f|$, osserviamo che per ogni vertice accoppiato $u \in L$, si ha $f(s, u) = 1$, e per ogni arco $(u, v) \in E - M$, si ha $f(u, v) = 0$; pertanto $f(L \cup \{s\}, R \cup \{t\})$, il flusso netto attraverso il taglio $(L \cup \{s\}, R \cup \{t\})$, è uguale a $|M|$. Applicando il Lemma 26.4, si ha $|f| = f(L \cup \{s\}, R \cup \{t\}) = |M|$. ■

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Unico Libreria 1000000000 2200107 © Copyright © 2022, McGraw-Hill Education (Italy)

In base al Lemma 26.9, possiamo concludere che un abbinamento massimo in un grafo bipartito G corrisponde a un flusso massimo nella sua rete di flusso associata G'; quindi possiamo calcolare un abbinamento massimo in G eseguendo su G' un algoritmo di flusso massimo. L'unico problema in questo ragionamento è che l'algoritmo di flusso massimo potrebbe restituire un flusso in G' per il quale qualche valore di $f(u, v)$ non è un intero, nonostante il flusso $|f|$ debba avere valori interi. Il seguente teorema dimostra che, se utilizziamo il metodo di Ford-Fulkerson, questo problema non si presenta.

Teorema 26.10 (Teorema dell'integralità)
Se la funzione capacità c assume soltanto valori interi, allora il flusso massimo f prodotto dal metodo di Ford-Fulkerson ha la proprietà che $|f|$ è un valore intero. Inoltre, per tutti i vertici u e v, il valore di $f(u, v)$ è un intero.

Dimostrazione La dimostrazione è per induzione sul numero delle iterazioni. Lasciamo al lettore il compito di dimostrare questo teorema (Esercizio 26.3-2). ■

Adesso possiamo dimostrare il seguente corollario del Lemma 26.9.

Corollario 26.11
La cardinalità di un abbinamento massimo M in grafo bipartito G è uguale al valore di un flusso massimo f nella rete di flusso associata G'.

Dimostrazione Utilizziamo la nomenclatura del Lemma 26.9. Supponiamo che M sia un abbinamento massimo in G e che il corrispondente flusso f in G' non sia massimo. Allora esiste un flusso massimo f' in G' tale che $|f'| > |f|$. Poiché le capacità in G' sono valori interi, per il Teorema 26.10, possiamo assumere che f' abbia valori interi. Quindi, f' corrisponde a un abbinamento M' in G con cardinalità $|M'| = |f'| > |f| = |M|$, e questo contraddice l'ipotesi che M sia un abbinamento massimo. In modo analogo, possiamo dimostrare che, se f è un flusso massimo in G', il suo corrispondente abbinamento è un abbinamento massimo in G. ■

Quindi, dato un grafo G non orientato bipartito, possiamo trovare un abbinamento massimo creando la rete di flusso G', eseguendo il metodo di Ford-Fulkerson e ottenendo direttamente un abbinamento massimo M dal flusso massimo f a valori interi che è stato trovato. Poiché qualsiasi abbinamento in un grafo bipartito ha cardinalità al più $\min(L, R) = O(V)$, il valore del flusso massimo in G' è $O(V)$. Di conseguenza, possiamo trovare un abbinamento massimo in un grafo bipartito nel tempo $O(VE') = O(VE)$, in quanto $|E'| = \Theta(E)$.

Esercizi

26.3-1
Eseguite l'algoritmo di Ford-Fulkerson sulla rete di flusso della Figura 26.8(c) e rappresentate la rete residua dopo ogni aumento di flusso. Numerate, dall'alto verso il basso, i vertici in L da 1 a 5 e i vertici in R da 6 a 9. Per ogni iterazione, selezionate il cammino aumentante che è lessicograficamente più piccolo.

26.3-2
Dimostrate il Teorema 26.10.

26.3-3
Sia $G = (V, E)$ un grafo bipartito con partizione dei vertici $V = L \cup R$; sia G' la sua rete di flusso associata. Trovate un buon limite superiore per la lunghezza di un cammino aumentante qualsiasi che viene trovato in G' durante l'esecuzione di FORD-FULKERSON.

Acquistato da Michele Mainela su Webster il 2022-07-07 22:12. Nuova Circolo Libreria 1005032162 - 2017 © Copyright © 2022 McGraw-Hill Education (Italy)

26.3-4 ⋆
Un ***abbinamento perfetto*** è un abbinamento in cui ogni vertice è accoppiato. Sia $G = (V, E)$ un grafo non orientato bipartito con partizione dei vertici $V = L \cup R$, dove $|L| = |R|$. Per ogni $X \subseteq V$, definite l'***intorno*** di X come

$$N(X) = \{y \in V : (x, y) \in E \text{ per qualche } x \in X\}$$

ovvero l'insieme dei vertici adiacenti a qualche elemento di X. Dimostrate il ***teorema di Hall***: esiste un abbinamento perfetto in G se e soltanto se $|A| \leq |N(A)|$ per ogni sottoinsieme $A \subseteq L$.

26.3-5 ⋆

Un grafo bipartito $G = (V, E)$, dove $V = L \cup R$, è ***d-regolare*** se ogni vertice $v \in V$ ha un grado esattamente pari a d. Ogni grafo bipartito d-regolare ha $|L| = |R|$. Dimostrate che ogni grafo bipartito d-regolare ha un abbinamento con cardinalità $|L|$ provando che un taglio minimo della rete di flusso associata ha capacità $|L|$.

⋆ 26.4 Algoritmi push-relabel

In questo paragrafo presentiamo l'approccio "push-relabel" per calcolare i flussi massimi. Attualmente, molti degli algoritmi di flusso massimo asintoticamente più veloci sono algoritmi push-relabel e le implementazioni più veloci degli algoritmi di flusso massimo si basano sul metodo push-relabel. Altri problemi di flusso, come quello del flusso a costo minimo, possono essere risolti in modo efficiente applicando i metodi push-relabel. Questo paragrafo presenta il "generico" algoritmo di Goldberg per il flusso massimo, che ha una semplice implementazione con tempo di esecuzione $O(V^2E)$, che migliora il limite $O(VE^2)$ dell'algoritmo di Edmonds-Karp. Nel Paragrafo 26.5 affineremo questo algoritmo generico per ottenere un altro algoritmo push-relabel che viene eseguito nel tempo $O(V^3)$.

Gli algoritmi push-relabel operano in modo più localizzato rispetto al metodo di Ford-Fulkerson. Anziché esaminare l'intera rete residua per trovare un cammino aumentante, gli algoritmi push-relabel operano su un vertice alla volta, considerando soltanto l'intorno di un vertice nella rete residua. Inoltre, diversamente dal metodo di Ford-Fulkerson, gli algoritmi push-relabel non mantengono la proprietà della conservazione del flusso durante la loro esecuzione. Tuttavia, mantengono un ***preflusso***, che è una funzione $f : V \times V \to \mathbb{R}$ che soddisfa il vincolo sulla capacità e il seguente rilassamento della proprietà della conservazione del flusso:

$$\sum_{v \in V} f(v, u) - \sum_{v \in V} f(u, v) \geq 0$$

per ogni vertice $u \in V - \{s\}$; ovvero il flusso entrante in un vertice può superare quello uscente. Questa quantità è detta ***flusso in eccesso*** nel vertice u ed è data da

$$e(u) = \sum_{v \in V} f(v, u) - \sum_{v \in V} f(u, v) \tag{26.14}$$

Diciamo che un vertice $u \in V - \{s, t\}$ è ***traboccante*** se $e(u) > 0$.

Inizieremo questo paragrafo descrivendo l'intuizione che sta alla base del metodo push-relabel. Poi esamineremo le due operazioni utilizzate dal metodo: l'operazione "push" (inviare il preflusso) e l'operazione "relabel" (cambiare l'etichetta di un vertice). Infine, presenteremo un generico algoritmo push-relabel e ne analizzeremo la correttezza e il tempo di esecuzione.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707 © Copyright © 2022, McGraw-Hill Education (Italy)

Intuizione

L'intuizione che sta alla base del metodo push-relabel può essere più facilmente descritta in termini di flussi dei fluidi: consideriamo una rete di flusso $G = (V, E)$ come un sistema di tubi interconnessi di capacità prefissate. Applicando questa

analogia al metodo di Ford-Fulkerson, potremmo dire che ogni cammino aumentante nella rete genera un flusso addizionale di fluido, senza punti di diramazione, che scorre dalla sorgente al pozzo. Il metodo di Ford-Fulkerson aggiunge in modo iterativo nuove linee di flusso finché non sarà più possibile aggiungerne altre.

Il generico algoritmo push-relabel si basa su un concetto molto diverso. Come prima, gli archi orientati corrispondono ai tubi. I vertici, che rappresentano le giunzioni dei tubi, hanno due interessanti proprietà. La prima è che, per accogliere il flusso in eccesso, ogni vertice ha un tubo di scarico che porta a un serbatoio arbitrariamente grande dove è possibile accumulare il fluido. La seconda proprietà è che ogni vertice, il suo serbatoio e tutte le sue connessioni con i tubi si trovano su una piattaforma la cui altezza aumenta con il procedere dell'algoritmo.

Le altezze dei vertici determinano il modo in cui il flusso viene inviato: il flusso può essere inviato soltanto in discesa, cioè da un vertice più alto a uno più basso. Ci può essere un flusso positivo da un vertice più basso a uno più alto, ma le operazioni push inviano il flusso soltanto in discesa. L'altezza della sorgente è fissata a $|V|$ e l'altezza del pozzo è fissata a 0. Le altezze di tutti gli altri vertici partono da 0 e aumentano col tempo. L'algoritmo prima invia in basso la maggior parte possibile di flusso, dalla sorgente verso il pozzo. La quantità di flusso che invia è quella esatta per riempire fino alla massima capacità tutti i tubi che escono dalla sorgente; ovvero l'algoritmo invia un flusso pari alla capacità del taglio $(s, V - s)$. Quando il flusso raggiunge per la prima volta un vertice intermedio, si raccoglie nel serbatoio del vertice. Da qui, sarà poi inviato verso il basso.

Potrebbe accadere che gli unici tubi che escono da un vertice u e che non sono ancora saturi di flusso siano collegati a vertici che si trovano allo stesso livello di u o a un livello superiore. In questo caso, per eliminare il flusso in eccesso da un vertice traboccante u, bisogna aumentare la sua altezza – questa operazione è detta "relabel" o "innalzamento" del vertice u. L'altezza del vertice viene aumentata di un'unità rispetto all'altezza del più basso tra i suoi vicini, con i quali il vertice è collegato tramite tubi non saturi. Dopo che un vertice è stato innalzato, quindi, c'è almeno un tubo che esce dal vertice attraverso il quale può essere inviato altro flusso.

Al termine, tutto il flusso che può attraversare la rete raggiunge il pozzo. Non può arrivare altro flusso, perché i tubi rispettano i vincoli sulla capacità; la quantità di flusso che attraversa un taglio qualsiasi è ancora limitata dalla capacità del taglio. Per rendere il preflusso un "flusso legale", l'algoritmo rimanda alla sorgente l'eccesso accumulato nei serbatoi dei vertici traboccanti, continuando a innalzare i vertici oltre l'altezza $|V|$ della sorgente. Come vedremo, una volta che tutti i serbatoi sono stati svuotati, il preflusso non solo è un "flusso legale", ma è anche un flusso massimo.

Le operazioni di base

Come risulta dalla precedente discussione, ci sono due operazioni di base che vengono eseguite da un algoritmo push-relabel: inviare il flusso in eccesso da un vertice a uno dei suoi vertici adiacenti (operazione *push*) e aumentare l'altezza di un vertice (operazione *relabel*). L'applicabilità di queste operazioni dipende dalle altezze dei vertici, che adesso definiamo con maggiore precisione.

Acquistato da Michele Mastrogiovanni - Vietata la vendita, la riproduzione, la modifica, la condivisione e la cessione a terzi. © 2023 McGraw-Hill Education (Italy)

Sia $G = (V, E)$ una rete di flusso con sorgente s e pozzo t e sia f un preflusso in G. Una funzione $h : V \to \mathbb{N}$ è una ***funzione altezza***[3] se $h(s) = |V|$, $h(t) = 0$ e

$$h(u) \leq h(v) + 1$$

per ogni arco residuo $(u, v) \in E_f$. Si ottiene immediatamente il seguente lemma.

Lemma 26.12
Sia $G = (V, E)$ una rete di flusso, sia f un preflusso in G e sia h una funzione altezza in V. Per due vertici qualsiasi $u, v \in V$, se $h(u) > h(v) + 1$, allora (u, v) non è un arco del grafo residuo. ■

L'operazione push
L'operazione di base PUSH(u, v) può essere applicata se u è un vertice traboccante, $c_f(u, v) > 0$ e $h(u) = h(v)+1$. Il seguente pseudocodice aggiorna il preflusso f in una rete $G = (V, E)$. Si suppone che la capacità residua $c_f(u, v)$ possa essere calcolata in tempo costante, essendo noti c ed f. Il flusso in eccesso in un vertice u è memorizzato nell'attributo $u.e$; l'altezza di u è memorizzata nell'attributo $u.h$. L'espressione $\Delta_f(u, v)$ è una variabile temporanea che memorizza la quantità di flusso che può essere inviata da u a v.

```
PUSH(u, v)
1  // Si applica quando: u è traboccante, c_f(u, v) > 0 e u.h = v.h + 1.
2  // Azione: invia Δ_f(u, v) = min(u.e, c_f(u, v)) unità di flusso da u a v.
3  Δ_f(u, v) = min(u.e, c_f(u, v))
4  if (u, v) ∈ E
5       (u, v).f = (u, v).f + Δ_f(u, v)
6  else (v, u).f = (v, u).f − Δ_f(u, v)
7  u.e = u.e − Δ_f(u, v)
8  v.e = v.e + Δ_f(u, v)
```

Il codice di PUSH opera nel modo seguente. Poiché il vertice u ha un flusso in eccesso $u.e$ positivo e la capacità residua di (u, v) è positiva, è possibile aumentare il flusso da u a v di $\Delta_f(u, v) = \min(u.e, c_f(u, v))$ senza rendere negativo $u.e$ o superare la capacità $c(u, v)$. La riga 3 calcola il valore $\Delta_f(u, v)$ e le righe 4–6 aggiornano f. La riga 5 aumenta il flusso nell'arco (u, v), perché stiamo immettendo del flusso in un arco residuo che è anche un arco originale. La riga 6 riduce il flusso nell'arco (v, u), perché l'arco originale è l'opposto di un arco nella rete originale. Infine, le righe 7–8 aggiornano il flusso in eccesso nei vertici u e v. Quindi, se f è un preflusso prima della chiamata di PUSH, esso resta un preflusso anche dopo.

Notate che nulla nel codice di PUSH dipende dalle altezze di u e v, e tuttavia impediamo che questa operazione sia eseguita a meno che non sia $u.h = v.h + 1$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[3]Nella letteratura, una funzione altezza di solito è detta "funzione distanza" e l'altezza di un vertice è detta "etichetta o label della distanza". Utilizziamo il termine "altezza" perché rende meglio l'idea che sta alla base dell'algoritmo. Manteniamo l'uso del termine "relabel" per fare riferimento all'operazione che aumenta l'altezza di un vertice. L'altezza di un vertice è in relazione alla sua distanza dal pozzo t, la distanza che si ottiene con una visita in ampiezza del grafo trasposto G^T.

Quindi, il flusso in eccesso viene inviato verso il basso soltanto se c'è una differenza di altezza pari a 1. Per il Lemma 26.12, non esistono archi residui tra due vertici se la differenza tra le loro altezze è maggiore di 1, e quindi, purché l'attributo h sia davvero una funzione altezza, non c'è nulla da guadagnare consentendo al flusso di essere spinto in basso da una differenza di altezza maggiore di 1.

Quando viene eseguita la procedura PUSH(u, v), diremo che viene effettuato un ***invio*** di flusso da u a v. Se un'operazione push si applica a qualche arco (u, v) che esce dal vertice u, diremo anche che l'operazione push si applica a u. Avremo un ***invio saturante*** se l'arco (u, v) diventa ***saturo*** (dopodiché $c_f(u, v) = 0$); altrimenti, avremo un ***invio non saturante***. Quando un arco diventa saturo, esso sparisce dalla rete residua. Un semplice lemma caratterizza il risultato di un invio non saturante.

Lemma 26.13
Dopo un invio non saturante da u a v, il vertice u non è più traboccante.

Dimostrazione Poiché l'invio non era saturante, la quantità di flusso $\Delta_f(u, v)$ effettivamente inviata deve essere uguale al valore di $u.e$ prima dell'invio. Poiché $u.e$ viene ridotto di tale quantità, l'eccesso di flusso si annulla dopo l'invio. ■

L'operazione relabel
L'operazione di base RELABEL(u) si applica se u è traboccante e se $u.h \leq v.h$ per ogni arco $(u, v) \in E_f$. In altre parole, possiamo innalzare un vertice traboccante u se, per ogni vertice v per il quale c'è una capacità residua da u a v, il flusso non può essere inviato da u a v perché v non è più basso di u (ricordiamo che, per definizione, né la sorgente s né il pozzo t possono essere traboccanti, quindi né s né t possono essere innalzati).

RELABEL(u)
1 **// Si applica quando:** u è traboccante e, per ogni $v \in V$ tale che $(u, v) \in E_f$, si ha $u.h \leq v.h$.
2 **// Azione:** aumenta l'altezza di u.
3 $u.h = 1 + \min\{v.h : (u, v) \in E_f\}$

Quando viene eseguita la procedura RELABEL(u), diremo che il vertice u è ***innalzato***. Notate che, quando il vertice u viene innalzato, bisogna che E_f contenga almeno un arco che esce da u, in modo che la ricerca del minimo venga eseguita su un insieme non vuoto. Questa condizione è garantita dall'ipotesi che u sia traboccante che, a sua volta, implica che

$$u.e = \sum_{v \in V} f(v, u) - \sum_{v \in V} f(u, v) > 0$$

Poiché tutti i flussi sono non negativi, ci deve essere almeno un vertice v tale che $(v, u).f > 0$. Ma allora $c_f(u, v) > 0$, che implica che $(u, v) \in E_f$. L'operazione RELABEL(u) quindi attribuisce a u la massima altezza consentita dai vincoli sulle funzioni altezza.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

L'algoritmo generico

Il generico algoritmo push-relabel usa la seguente subroutine per creare un preflusso iniziale nella rete di flusso.

```
INITIALIZE-PREFLOW(G, s)
 1  for ogni vertice v ∈ G.V
 2      v.h = 0
 3      v.e = 0
 4  for ogni arco (u, v) ∈ G.E
 5      (u, v).f = 0
 6  s.h = |G.V|
 7  for ogni vertice v ∈ s.Adj
 8      (s, v).f = c(s, v)
 9      v.e = c(s, v)
10      s.e = s.e − c(s, v)
```

INITIALIZE-PREFLOW crea un preflusso iniziale f definito da

$$(u,v).f = \begin{cases} c(u,v) & \text{se } u = s \\ 0 & \text{negli altri casi} \end{cases} \tag{26.15}$$

Ovvero, ogni arco che esce dalla sorgente s viene riempito fino alla massima capacità, mentre tutti gli altri archi sono privi di flusso. Per ogni vertice v adiacente alla sorgente, inizialmente si ha $v.e = c(s,v)$ e l'attributo $s.e$ viene inizializzato con il valore negativo della somma di queste capacità. Inoltre, l'algoritmo generico inizia con una funzione altezza h data da

$$u.h = \begin{cases} |V| & \text{se } u = s \\ 0 & \text{negli altri casi} \end{cases} \tag{26.16}$$

L'equazione (26.16) definisce una funzione altezza perché gli unici archi (u,v) per i quali $u.h > v.h + 1$ sono quelli per i quali $u = s$; poiché questi archi sono saturi, essi non si trovano nella rete residua.

L'inizializzazione, seguita da una serie di operazioni push e relabel eseguite senza un ordine particolare, produce l'algoritmo GENERIC-PUSH-RELABEL:

```
GENERIC-PUSH-RELABEL(G)
1  INITIALIZE-PREFLOW(G, s)
2  while esiste un'operazione push o relabel
3      sceglie un'operazione push o relabel applicabile e la esegue
```

Il seguente lemma ci dice che, finché esiste un vertice traboccante, è possibile applicare almeno una delle due operazioni di base.

Lemma 26.14 (A un vertice traboccante è applicabile un'operazione push o un'operazione relabel)
Sia $G = (V,E)$ una rete di flusso con sorgente s e pozzo t; sia f un preflusso e sia h un funzione altezza per f. Se u è un vertice traboccante, allora è possibile applicare a questo vertice un'operazione push o un'operazione relabel.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022. McGraw-Hill Education (Italy)

Dimostrazione Per qualsiasi arco residuo (u,v), si ha $h(u) \le h(v) + 1$ perché h è una funzione altezza. Se un'operazione push non può essere applicata a u, allora per qualsiasi arco residuo (u,v) deve essere $h(u) < h(v) + 1$, che implica $h(u) \le h(v)$. Quindi, un'operazione relabel può essere applicata a u. ■

Correttezza del metodo push-relabel

Per dimostrare che il generico algoritmo push-relabel risolve il problema del flusso massimo, dobbiamo prima dimostrare che, se esso termina, il preflusso f è un flusso massimo. Poi dimostreremo che esso termina. Iniziamo con alcune osservazioni sulla funzione altezza h.

Lemma 26.15 (Le altezze dei vertici non diminuiscono mai)
Durante l'esecuzione della procedura GENERIC-PUSH-RELABEL su una rete di flusso $G = (V, E)$, per ogni vertice $u \in V$, l'altezza $u.h$ non diminuisce mai. Inoltre, quando un'operazione relabel viene applicata a un vertice u, l'altezza $u.h$ aumenta almeno di 1.

Dimostrazione Poiché le altezze dei vertici cambiano soltanto durante le operazioni relabel, è sufficiente provare la seconda asserzione del lemma. Se il vertice u sta per essere innalzato, allora per ogni vertice v tale che $(u, v) \in E_f$, si ha $u.h \leq v.h$. Quindi $u.h < 1 + \min\{v.h : (u, v) \in E_f\}$ e, di conseguenza, l'operazione deve aumentare $u.h$. ■

Lemma 26.16
Sia $G = (V, E)$ una rete di flusso con sorgente s e pozzo t. Durante l'esecuzione di GENERIC-PUSH-RELABEL su G, l'attributo h è mantenuto come una funzione altezza.

Dimostrazione La dimostrazione è per induzione sul numero di operazioni di base che vengono eseguite. Inizialmente, h è una funzione altezza, come abbiamo precedentemente osservato. Asseriamo che, se h è una funzione altezza, allora dopo un'operazione RELABEL(u), h sarà ancora una funzione altezza. Se consideriamo un arco residuo $(u, v) \in E_f$ che esce da u, allora l'operazione RELABEL(u) garantisce che anche dopo l'operazione sarà $u.h \leq v.h + 1$. Adesso consideriamo un arco residuo (w, u) che entra in u. Per il Lemma 26.15, $w.h \leq u.h + 1$ prima dell'operazione RELABEL(u) implica $w.h < u.h + 1$ dopo l'operazione. Quindi, dopo l'operazione RELABEL(u), h resta una funzione altezza.

Adesso, consideriamo un'operazione PUSH(u, v). Questa operazione può aggiungere l'arco (v, u) a E_f e può rimuovere (u, v) da E_f. Nel primo caso, si ha $v.h = u.h - 1 < u.h + 1$, e quindi h resta una funzione altezza. Nel secondo caso, la rimozione di (u, v) dalla rete residua elimina il corrispondente vincolo, e quindi h resta una funzione altezza. ■

Il seguente lemma definisce un'importante proprietà delle funzioni altezza.

Lemma 26.17
Sia $G = (V, E)$ una rete di flusso con sorgente s e pozzo t; sia f un preflusso in G e sia h una funzione altezza in V. Allora non esiste alcun cammino dalla sorgente s al pozzo t nella rete residua G_f.

Acquistato da Michele Mastrostefano il 2025-07-22 12:12 Numero di serie del libro: 9788863216220 Copyright © 2023 McGraw-Hill Education (Italy)

Dimostrazione Supponiamo per assurdo che ci sia un cammino $p = \langle v_0, v_1, \ldots, v_k \rangle$ da s a t in G_f, dove $v_0 = s$ e $v_k = t$. Senza perdere in generalità, supponiamo che p sia un cammino semplice, e quindi $k < |V|$. Per $i = 0, 1, \ldots, k - 1$, l'arco $(v_i, v_{i+1}) \in E_f$. Poiché h è una funzione altezza, $h(v_i) \leq h(v_{i+1}) + 1$ per $i = 0, 1, \ldots, k - 1$. Combinando queste disuguaglianze lungo il cammino p, si ottiene $h(s) \leq h(t) + k$. Tuttavia, poiché $h(t) = 0$, si ha $h(s) \leq k < |V|$, che contraddice il requisito che $h(s) = |V|$ in una funzione altezza. ■

Adesso siamo pronti a dimostrare che, se il generico algoritmo push-relabel termina, il preflusso che esso calcola è un flusso massimo.

Teorema 26.18 (Correttezza del generico algoritmo push-relabel)
Se l'algoritmo GENERIC-PUSH-RELABEL termina quando viene eseguito su una rete di flusso $G = (V, E)$ con sorgente s e pozzo t, allora il preflusso f che esso calcola è un flusso massimo per G.

Dimostrazione Utilizziamo la seguente invariante di ciclo:

Ogni volta che viene eseguito il test del ciclo **while** nella riga 2 in GENERIC-PUSH-RELABEL, f è un preflusso.

Inizializzazione: INITIALIZE-PREFLOW rende f un preflusso.

Conservazione: le uniche operazioni all'interno del ciclo **while** (righe 2–3) sono push e relabel. Le operazioni relabel agiscono soltanto sulle altezze, non sui valori dei flussi; quindi se f è un preflusso, resta tale anche dopo un'operazione relabel. Come detto a pagina 618, se f è un preflusso prima di un'operazione push, resta un preflusso anche dopo tale operazione.

Conclusione: al termine, ogni vertice in $V - \{s, t\}$ deve avere un flusso in eccesso nullo perché, per il Lemma 26.14 e per l'invariante che f è sempre un preflusso, non ci sono vertici traboccanti. Ne consegue che f è un flusso. Per il Lemma 26.16, h è una funzione altezza; quindi il Lemma 26.17 dice che non ci sono cammini da s a t nella rete residua G_f. Per il teorema del flusso massimo/taglio minimo (Teorema 26.6), quindi, f è un flusso massimo. ■

Analisi del metodo push-relabel

Per dimostrare che il generico algoritmo push-relabel termina davvero, dobbiamo porre un limite superiore al numero di operazioni che svolge. Ciascuno dei tre tipi di operazioni – innalzamenti, invii saturanti e invii non saturanti – ha un proprio limite. Conoscendo questi limiti, sarà semplice costruire un algoritmo che viene eseguito nel tempo $O(V^2E)$. Prima di iniziare l'analisi, tuttavia, dimostriamo un importante lemma. Ricordiamo che nella rete residua sono ammessi anche archi che entrano nella sorgente.

Lemma 26.19
Sia $G = (V, E)$ una rete di flusso con sorgente s e pozzo t; sia f un preflusso in G. Allora, per ogni vertice traboccante x, esiste un cammino semplice da x a s nella rete residua G_f.

Dimostrazione Per un vertice traboccante x, sia $U = \{v : \text{esiste un cammino semplice da } x \text{ a } v \text{ in } G_f\}$, e supponiamo per assurdo che $s \notin U$. Sia $\overline{U} = V - U$.

Considerando la definizione di eccesso dell'equazione (26.14) (eseguendo le sommatorie su tutti i vertici in U) e notando che $V = U \cup \overline{U}$, si ha

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned}
&\sum_{u \in U} e(u) \\
&\quad = \sum_{u \in U} \left(\sum_{v \in V} f(v, u) - \sum_{v \in V} f(u, v) \right) \\
&\quad = \sum_{u \in U} \left(\left(\sum_{v \in U} f(v, u) + \sum_{v \in \overline{U}} f(v, u) \right) - \left(\sum_{v \in U} f(u, v) + \sum_{v \in \overline{U}} f(u, v) \right) \right)
\end{aligned}$$

$$
\begin{aligned}
&= \sum_{u\in U}\sum_{v\in U} f(v,u) + \sum_{u\in U}\sum_{v\in \overline{U}} f(v,u) - \sum_{u\in U}\sum_{v\in U} f(u,v) - \sum_{u\in U}\sum_{v\in \overline{U}} f(u,v) \\
&= \sum_{u\in U}\sum_{v\in \overline{U}} f(v,u) - \sum_{u\in U}\sum_{v\in \overline{U}} f(u,v)
\end{aligned}
$$

Sappiamo che la quantità $\sum_{u\in U} e(u)$ deve essere positiva perché $e(x) > 0$, $x \in U$, tutti i vertici diversi da s hanno flussi in eccesso non negativi e, per ipotesi, $s \notin U$. Quindi, si ha

$$\sum_{u\in U}\sum_{v\in \overline{U}} f(v,u) - \sum_{u\in U}\sum_{v\in \overline{U}} f(u,v) > 0 \tag{26.17}$$

Tutti i flussi negli archi sono non negativi e, quindi, per l'equazione (26.17), deve essere $\sum_{u\in U}\sum_{v\in \overline{U}} f(v,u) > 0$. Dunque, ci deve essere almeno una coppia di vertici $u' \in U$ e $v' \in \overline{U}$ con $f(v',u') > 0$. Tuttavia, se $f(v',u') > 0$, ci deve essere un arco residuo (u',v'); ciò significa che c'è un cammino semplice da x a v' (il cammino $x \rightsquigarrow u' \to v'$), e questo contraddice la definizione di U. ■

Il prossimo lemma definisce un limite per le altezze dei vertici; il suo corollario definisce il limite per il numero totale di operazioni relabel che sono eseguite.

Lemma 26.20
Sia $G = (V,E)$ una rete di flusso con sorgente s e pozzo t. In qualsiasi istante durante l'esecuzione di GENERIC-PUSH-RELABEL su G, si ha $u.h \le 2|V| - 1$ per ogni vertice $u \in V$.

Dimostrazione Le altezze della sorgente s del pozzo t non cambiano mai perché questi vertici per definizione non sono traboccanti. Quindi, si ha sempre $s.h = |V|$ e $t.h = 0$, entrambe queste altezze non superano $2|V| - 1$.

Adesso consideriamo un vertice qualsiasi $u \in V - \{s,t\}$. Inizialmente, $u.h = 0 \le 2|V| - 1$. Dimostreremo che dopo ogni operazione relabel, si ha ancora $u.h \le 2|V| - 1$. Quando il vertice u viene innalzato, esso è traboccante, e il Lemma 26.19 dice che esiste un cammino semplice p da u a s in G_f. Sia $p = \langle v_0, v_1, \ldots, v_k\rangle$, dove $v_0 = u$, $v_k = s$, e $k \le |V| - 1$ perché p è semplice. Per $i = 0, 1, \ldots, k-1$, si ha $(v_i, v_{i+1}) \in E_f$, e quindi, per il Lemma 26.16, $v_i.h \le v_{i+1}.h + 1$. Espandendo queste disuguaglianze lungo il cammino p, si ottiene $u.h = v_0.h \le v_k.h + k \le s.h + (|V| - 1) = 2|V| - 1$. ■

Corollario 26.21 (Limite per operazioni relabel)
Sia $G = (V,E)$ una rete di flusso con sorgente s e pozzo t. Allora, durante l'esecuzione di GENERIC-PUSH-RELABEL su G, il numero di operazioni relabel è al massimo $2|V| - 1$ per ogni vertice e al massimo $(2|V| - 1)(|V| - 2) < 2|V|^2$ in totale.

Acquistato da Michele Michele su Webster il 2022-07-07 23:10. Numero Ordine Libreria 199503016.220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

Dimostrazione Soltanto $|V| - 2$ vertici in $V - \{s,t\}$ possono essere innalzati. Sia $u \in V - \{s,t\}$. L'operazione RELABEL(u) aumenta $u.h$. Il valore di $u.h$ è inizialmente 0 e per il Lemma 26.20 cresce al massimo fino a $2|V| - 1$. Quindi, ogni vertice $u \in V - \{s,t\}$ è innalzato al massimo $2|V| - 1$ volte e il numero totale di operazioni relabel che vengono eseguite è al massimo $(2|V| - 1)(|V| - 2) < 2|V|^2$. ■

Il Lemma 26.20 ci aiuta anche a trovare il limite per il numero di invii saturanti.

Lemma 26.22 (Limite per gli invii saturanti)
Durante l'esecuzione di GENERIC-PUSH-RELABEL su una rete di flusso $G = (V, E)$, il numero di invii saturanti è minore di $2|V||E|$.

Dimostrazione Per ogni coppia di vertici $u, v \in V$, conteremo insieme gli invii saturanti da u a v e da v a u, chiamandoli invii saturanti tra u e v. Se tali invii esistono, almeno uno degli archi (u, v) e (v, u) è effettivamente un arco in E. Adesso, supponiamo che ci sia stato un invio saturante da u a v. In quel momento, $v.h = u.h - 1$. Affinché ci possa essere un altro invio da u a v, l'algoritmo deve prima inviare del flusso da v a u, e questo non potrà accadere finché non sarà $v.h = u.h + 1$. Poiché $u.h$ non diminuisce mai, affinché possa essere $v.h = u.h + 1$, il valore di $v.h$ deve aumentare almeno di 2. Analogamente, $u.h$ deve aumentare almeno di 2 tra gli invii saturanti da v a u. Le altezze partono da 0 e, per il Lemma 26.20, non superano mai $2|V| - 1$; questo implica che il numero di volte che l'altezza di un vertice qualsiasi può aumentare di 2 è minore di $|V|$. Poiché almeno una delle altezze $u.h$ e $v.h$ deve crescere di 2 tra due invii saturanti qualsiasi tra u e v, ci sono meno di $2|V|$ invii saturanti tra u e v. Moltiplicando per il numero di archi si ottiene un limite minore di $2|V||E|$ per il numero totale di invii saturanti. ■

Il seguente lemma limita il numero di invii non saturanti nel generico algoritmo push-relabel.

Lemma 26.23 (Limite per gli invii non saturanti)
Durante l'esecuzione di GENERIC-PUSH-RELABEL su una rete di flusso $G = (V, E)$, il numero di invii non saturanti è minore di $4|V|^2(|V| + |E|)$.

Dimostrazione Definiamo una funzione potenziale $\Phi = \sum_{v:e(v)>0} v.h$. Inizialmente $\Phi = 0$; il valore di Φ può cambiare dopo ogni innalzamento, invio saturante e invio non saturante. Troveremo un limite all'incidenza che hanno gli invii saturanti e gli innalzamenti sull'aumento di Φ. Poi dimostreremo che ogni invio non saturante deve ridurre Φ almeno di 1 e utilizzeremo questi limiti per ottenere un limite superiore per il numero di invii non saturanti.

Esaminiamo i due modi in cui Φ può aumentare. In primo luogo, l'innalzamento di un vertice u aumenta Φ di una quantità minore di $2|V|$, perché l'insieme sul quale viene calcolata la sommatoria rimane lo stesso e l'innalzamento non può aumentare l'altezza di u oltre la sua altezza massima, che, per il Lemma 26.20, è al più $2|V| - 1$. In secondo luogo, un invio saturante da un vertice u a un vertice v aumenta Φ di una quantità minore di $2|V|$, perché nessuna altezza cambia e soltanto il vertice v, la cui altezza è al massimo $2|V| - 1$, può eventualmente diventare traboccante.

Adesso dimostriamo che un invio non saturante da u a v riduce Φ almeno di 1. Perché? Prima dell'invio non saturante, u era traboccante, mentre v poteva essere traboccante oppure no. Per il Lemma 26.13, u non è più traboccante dopo l'invio. Inoltre, v deve essere traboccante dopo l'invio, a meno che non sia la sorgente. Quindi, la funzione potenziale Φ è diminuita esattamente di $u.h$ ed è aumentata di 0 o $v.h$. Poiché $u.h - v.h = 1$, l'effetto netto è che la funzione potenziale è diminuita almeno di 1.

Acquistato da Michele Michele su Webster il 2022-07-15 12:12 Proprietà della Libreria 18953816 220707 0 Copyright © 2022 McGraw-Hill Education (Italy)

Quindi, durante l'esecuzione dell'algoritmo, l'aumento totale di Φ è dovuto agli innalzamenti e agli invii saturanti ed è limitato dal Corollario 26.21 e dal Lemma 26.22 a essere minore di $(2|V|)(2|V|^2) + (2|V|)(2|V||E|) = 4|V|^2(|V| + |E|)$. Poiché $\Phi \geq 0$, la riduzione totale e, quindi, il numero totale di invii non saturanti, è minore di $4|V|^2(|V| + |E|)$. ■

Una volta definiti i limiti per il numero di innalzamenti, di invii saturanti e di invii non saturanti, abbiamo le informazioni necessarie per analizzare la procedura GENERIC-PUSH-RELABEL e, quindi, qualsiasi algoritmo basato sul metodo push-relabel.

Teorema 26.24
Durante l'esecuzione di GENERIC-PUSH-RELABEL su una rete di flusso $G = (V, E)$, il numero di operazioni di base è $O(V^2E)$.

Dimostrazione È una conseguenza immediata del Corollario 26.21 e dei Lemmi 26.22 e 26.23. ■

Quindi, l'algoritmo termina dopo $O(V^2E)$ operazioni. A questo punto, tutto ciò che rimane è definire un metodo efficiente per implementare ciascuna delle operazioni e per scegliere l'operazione appropriata da eseguire.

Corollario 26.25
Esiste un'implementazione del generico algoritmo push-relabel che viene eseguito nel tempo $O(V^2E)$ su una rete di flusso $G = (V, E)$.

Dimostrazione L'Esercizio 26.4-2 chiede di spiegare come può essere implementato il generico algoritmo push-relabel con un costo $O(V)$ per ogni operazione relabel e un costo $O(1)$ per ogni operazione push. L'esercizio chiede anche di progettare una struttura dati che consenta di selezionare nel tempo $O(1)$ un'operazione applicabile. Da questo si dimostra il corollario. ■

Esercizi

26.4-1
Dimostrate che, dopo l'esecuzione della procedura INITIALIZE-PREFLOW(G, s), si ha $s.e \leq -|f^*|$, dove f^* è un flusso massimo per G.

26.4-2
Spiegate come può essere implementato il generico algoritmo push-relabel con un tempo $O(V)$ per ogni operazione relabel, un tempo $O(1)$ per ogni operazione push e un tempo $O(1)$ per selezionare un'operazione applicabile, per un tempo totale pari a $O(V^2E)$.

26.4-3
Dimostrate che il generico algoritmo push-relabel impiega un tempo totale pari a $O(VE)$ per eseguire tutte le $O(V^2)$ operazioni relabel.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

26.4-4
Supponete che sia stato trovato un flusso massimo in una rete di flusso $G = (V, E)$ utilizzando un algoritmo push-relabel. Create un algoritmo veloce per trovare un taglio minimo in G.

26.4-5
Create un algoritmo push-relabel efficiente per trovare un abbinamento massimo in un grafo bipartito. Analizzate l'algoritmo.

26.4-6
Supponete che tutte le capacità degli archi in una rete di flusso $G = (V, E)$ siano nell'insieme $\{1, 2, \ldots, k\}$. Analizzate il tempo di esecuzione del generico algoritmo push-relabel in termini di $|V|$, $|E|$ e k. (*Suggerimento:* quante volte ciascun arco può ricevere un invio non saturante prima di diventare saturo?)

26.4-7
Dimostrate che la riga 6 di INITIALIZE-PREFLOW può essere cambiata in questo modo

6 $\quad s.h = |G.V| - 2$

senza influire sulla correttezza o sulle prestazioni asintotiche del generico algoritmo push-relabel.

26.4-8
Sia $\delta_f(u, v)$ la distanza (numero di archi) da u a v nella rete residua G_f. Dimostrate che GENERIC-PUSH-RELABEL mantiene la proprietà che $u.h < |V|$ implica $u.h \leq \delta_f(u, t)$ e che $u.h \geq |V|$ implica $u.h - |V| \leq \delta_f(u, s)$.

26.4-9 ⋆
Come nel precedente esercizio, sia $\delta_f(u, v)$ la distanza da u a v nella rete residua G_f. Spiegate come può essere modificato il generico algoritmo push-relabel per mantenere la proprietà che $u.h < |V|$ implica $u.h = \delta_f(u, t)$ e che $u.h \geq |V|$ implica $u.h - |V| = \delta_f(u, s)$. Il tempo totale che la vostra implementazione dedica al mantenimento di questa proprietà dovrebbe essere $O(VE)$.

26.4-10
Dimostrate che il numero di invii non saturanti eseguiti da GENERIC-PUSH-RELABEL su una rete di flusso $G = (V, E)$ è al più $4|V|^2|E|$ per $|V| \geq 4$.

⋆ 26.5 Algoritmo relabel-to-front

Il metodo push-relabel ci consente di applicare le operazioni di base in qualsiasi ordine. Tuttavia, se scegliamo con attenzione l'ordine e gestiamo con efficienza la struttura dati della rete, possiamo risolvere il problema del flusso massimo in un tempo minore del limite $O(V^2E)$ dato dal Corollario 26.25. Esaminiamo adesso l'algoritmo relabel-to-front, un algoritmo push-relabel il cui tempo di esecuzione è $O(V^3)$, che asintoticamente è buono almeno quanto $O(V^2E)$, ma è migliore per le reti dense.

L'algoritmo relabel-to-front mantiene una lista con i vertici della rete. A partire dalla testa della lista, l'algoritmo ispeziona la lista, selezionando ripetutamente un vertice traboccante u per "scaricarlo", ovvero continua a eseguire le operazioni push e relabel finché u non avrà più un flusso in eccesso positivo. Quando un vertice viene innalzato (operazione relabel), viene spostato in testa alla lista (da qui il nome "relabel-to-front") e l'algoritmo ricomincia a ispezionare la lista.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libraria: 1995030162220707.0. Copyright © 2022, McGraw-Hill Education (Italy)

La correttezza e l'analisi dell'algoritmo relabel-to-front dipendono dal concetto di "archi ammissibili": quegli archi nella rete residua nei quali può essere inviato del flusso. Dopo avere dimostrato alcune proprietà della rete di archi ammissibili, esamineremo l'operazione che scarica un vertice traboccante e poi presenteremo e analizzeremo l'algoritmo relabel-to-front.

Archi e reti ammissibili

Se $G = (V, E)$ è una rete di flusso con sorgente s e pozzo t, f è un preflusso in G e h è una funzione altezza, allora diciamo che (u, v) è un ***arco ammissibile*** se $c_f(u, v) > 0$ e $h(u) = h(v) + 1$; altrimenti (u, v) è un arco ***inammissibile***. La ***rete ammissibile*** è $G_{f,h} = (V, E_{f,h})$, dove $E_{f,h}$ è l'insieme degli archi ammissibili.

La rete ammissibile è formata da quegli archi nei quali può essere inviato del flusso. Il seguente lemma dimostra che questa rete è un grafo orientato aciclico (dag).

Lemma 26.26 (La rete ammissibile è aciclica)
Se $G = (V, E)$ è una rete di flusso, f è un preflusso in G e h è una funzione altezza in G, allora la rete ammissibile $G_{f,h} = (V, E_{f,h})$ è aciclica.

Dimostrazione La dimostrazione è per assurdo. Supponiamo che $G_{f,h}$ contenga un ciclo $p = \langle v_0, v_1, \ldots, v_k \rangle$, dove $v_0 = v_k$ e $k > 0$. Poiché ogni arco in p è ammissibile, si ha $h(v_{i-1}) = h(v_i) + 1$ per $i = 1, 2, \ldots, k$. Sommando intorno al ciclo, si ottiene

$$\begin{aligned} \sum_{i=1}^{k} h(v_{i-1}) &= \sum_{i=1}^{k} (h(v_i) + 1) \\ &= \sum_{i=1}^{k} h(v_i) + k \end{aligned}$$

Poiché ogni vertice nel ciclo p appare una volta in ciascuna delle sommatorie, si ha l'assurdo che $0 = k$. ■

I prossimi due lemmi mostrano in che modo le operazioni push e relabel cambiano la rete ammissibile.

Lemma 26.27
Sia $G = (V, E)$ una rete di flusso, sia f un preflusso in G e supponiamo che l'attributo h sia una funzione altezza. Se un vertice u è traboccante e (u, v) è un arco ammissibile, allora si applica l'operazione PUSH(u, v). Questa operazione non crea nuovi archi ammissibili, ma potrebbe far diventare inammissibile l'arco (u, v).

Dimostrazione Per la definizione di arco ammissibile, il flusso può essere inviato da u a v. Poiché u è traboccante, si applica l'operazione PUSH(u, v). L'unico arco residuo nuovo che può essere creato inviando del flusso da u a v è l'arco (v, u). Poiché $v.h = u.h - 1$, l'arco (v, u) non può diventare ammissibile. Se l'operazione è un invio saturante, allora $c_f(u, v) = 0$ dopo l'operazione, e l'arco (u, v) diventa inammissibile. ■

Acquistato da Michele Mazzucco il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Lemma 26.28
Sia $G = (V, E)$ una rete di flusso, sia f un preflusso in G e supponiamo che l'attributo h sia una funzione altezza. Se un vertice u è traboccante e non ci sono archi ammissibili che escono da u, allora si applica l'operazione RELABEL(u). Dopo questa operazione, c'è almeno un arco ammissibile che esce da u, ma non ci sono archi ammissibili che entrano in u.

Dimostrazione Se il vertice u è traboccante, allora per il Lemma 26.14, si applica un'operazione push o relabel al vertice. Se non ci sono archi ammissibili che escono da u, allora nessun flusso può essere inviato da u e quindi si può applicare l'operazione RELABEL(u). Dopo questa operazione, $u.h = 1 + \min\{v.h : (u, v) \in E_f\}$. Quindi, se v è un vertice che realizza il minimo in questo insieme, l'arco (u, v) diventa ammissibile. Pertanto, dopo l'innalzamento, c'è almeno un arco ammissibile che esce da u.

Per dimostrare che non ci sono archi ammissibili che entrano in u dopo un'operazione relabel, supponiamo che esista un vertice v tale che (v, u) sia ammissibile. Allora, $v.h = u.h + 1$ dopo l'innalzamento, e quindi $v.h > u.h + 1$ appena prima dell'innalzamento. Ma per il Lemma 26.12, non ci sono archi residui tra i vertici le cui altezze hanno una differenza maggiore di 1. Inoltre, l'innalzamento di un vertice non cambia la rete residua. Quindi (v, u) non è nella rete residua e, dunque, non può essere nella rete ammissibile. ■

Liste dei vicini

Nell'algoritmo relabel-to-front gli archi sono organizzati in "liste di vicini". Data una rete di flusso $G = (V, E)$, la ***lista dei vicini*** $u.N$ per un vertice $u \in V$ è una lista singolarmente concatenata dei vertici vicini a u in G. Quindi, il vertice v appare nella lista $u.N$ se $(u, v) \in E$ o $(v, u) \in E$. La lista dei vicini $u.N$ contiene esattamente quei vertici v per i quali ci può essere un arco residuo (u, v). L'attributo $u.N.head$ punta al primo vertice in $u.N$, e $v.next\text{-}neighbor$ punta al vertice v che segue nella lista dei vicini; questo puntatore è NIL se v è l'ultimo vertice nella lista dei vicini. L'algoritmo relabel-to-front ispeziona ciclicamente ogni lista dei vicini in un ordine arbitrario, che resta costante per tutta l'esecuzione dell'algoritmo. Per ogni vertice u, l'attributo $u.current$ punta al vertice correntemente esaminato nella lista $u.N$. Inizialmente, $u.current$ ha il valore $u.N.head$.

Scaricare un vertice traboccante

Un vertice traboccante u viene ***scaricato*** inviando ai vertici vicini tutto il suo flusso in eccesso che attraversa gli archi ammissibili, innalzando u se necessario per fare in modo che gli archi che escono da u diventino ammissibili. Lo pseudocodice è il seguente.

```
DISCHARGE(u)
1  while u.e > 0
2      v = u.current
3      if v == NIL
4          RELABEL(u)
5          u.current = u.N.head
6      elseif c_f(u, v) > 0 and u.h == v.h + 1
7          PUSH(u, v)
8      else u.current = v.next-neighbor
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

La Figura 26.9 illustra diverse iterazioni del ciclo **while** (righe 1–8), che viene eseguito finché il vertice u ha un flusso in eccesso positivo. Ogni iterazione esegue una sola delle seguenti tre azioni, a seconda del vertice v che è correntemente esaminato nella lista dei vicini $u.N$.

1. Se v è NIL, allora abbiamo oltrepassato la fine della lista $u.N$. La riga 4 innalza il vertice u; poi la riga 5 sceglie come vertice vicino al vertice u corrente il primo elemento della lista dei vicini $u.N$ (il successivo Lemma 26.29 stabilisce che in questo caso si applica l'operazione relabel).
2. Se v non è NIL e (u, v) è un arco ammissibile (determinato dal test nella riga 6), allora la riga 7 invia al vertice v un po' o eventualmente tutto il flusso in eccesso di u.
3. Se v non è NIL, ma (u, v) è inammissibile, allora la riga 8 fa avanzare $u.current$ di una posizione nella lista dei vicini $u.N$.

Notate che, se la procedura DISCHARGE viene chiamata su un vertice u traboccante, allora l'ultima operazione svolta da DISCHARGE deve essere un invio da u. Perché? La procedura termina soltanto quando $u.e$ diventa zero, e il valore di $u.e$ non viene modificato né dall'operazione relabel né dall'avanzamento del puntatore $u.current$.

Dobbiamo essere sicuri che, quando DISCHARGE chiama PUSH o RELABEL, tali operazioni siano applicabili. Il prossimo lemma dimostra questo fatto.

Lemma 26.29
Se DISCHARGE chiama PUSH(u, v) nella riga 7, allora è applicabile un'operazione push a (u, v). Se DISCHARGE chiama RELABEL(u) nella riga 4, allora è applicabile un'operazione relabel a u.

Dimostrazione I test nelle righe 1 e 6 garantiscono che un'operazione push si verifica soltanto se l'operazione è applicabile, e questo dimostra la prima asserzione del lemma.

Per provare la seconda asserzione, secondo il test nella riga 1 e il Lemma 26.28, dobbiamo soltanto dimostrare che tutti gli archi che escono da u sono inammissibili. Se una chiamata a DISCHARGE(u) inizia con $u.current$ che punta all'inizio della lista dei vicini di u e termina con tale puntatore che punta alla fine della lista allora tutti gli archi che escono da u sono inamissibili e si può applicare l'operazione relabel. D'altra parte è possibile che durante l'esecuzione di DISCHARGE(u) il puntatore $u.current$ percorra soltanto una parte della lista prima che la procedura termini. A questo punto possono esserci delle chiamate a DISCHARGE su altri vertici, ma $u.current$ continuerà a muoversi lungo la lista quando verrà eseguita una successiva chiamata DISCHARGE(u) sullo stesso vertice u. Consideriamo cosa succede durante un intero passaggio lungo la lista che inizi dalla cima di $u.N$ e termini con $u.current = \text{NIL}$. Quando $u.current$ raggiunge la fine della lista, la procedura innalza il vertice u e ricomincia un nuovo passaggio. Affinché il puntatore $u.current$ possa oltrepassare un vertice $v \in u.N$ durante un passaggio, l'arco (u, v) deve essere considerato inammissibile dal test nella riga 6. Pertanto, quando il passaggio viene completato, ogni arco che esce da u è stato ritenuto inammissibile in qualche istante durante il passaggio. L'osservazione principale è che alla fine del passaggio, ogni arco che esce da u è ancora inammissibile. Perché? Per il Lemma 26.27, gli invii non possono creare archi ammissibili, qualunque sia il vertice da cui viene inviato il flusso. Quindi, qualsiasi arco ammissibile deve essere creato da un'operazione relabel. Ma il vertice u non viene innalzato durante il passaggio (per effetto di una chiamata DISCHARGE(v)) e, per il Lemma 26.28, qualsiasi altro vertice v che viene innalzato durante il passaggio non ha archi ammissibili entranti dopo l'innalzamento. Quindi, al termine del passaggio, tutti gli archi che escono da u restano inammissibili, e il lemma è dimostrato. ■

Acquistato da Michele Michele su Weneta Il 2022-07-07 09:12 Numero Ordine Libreria 1995/3016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

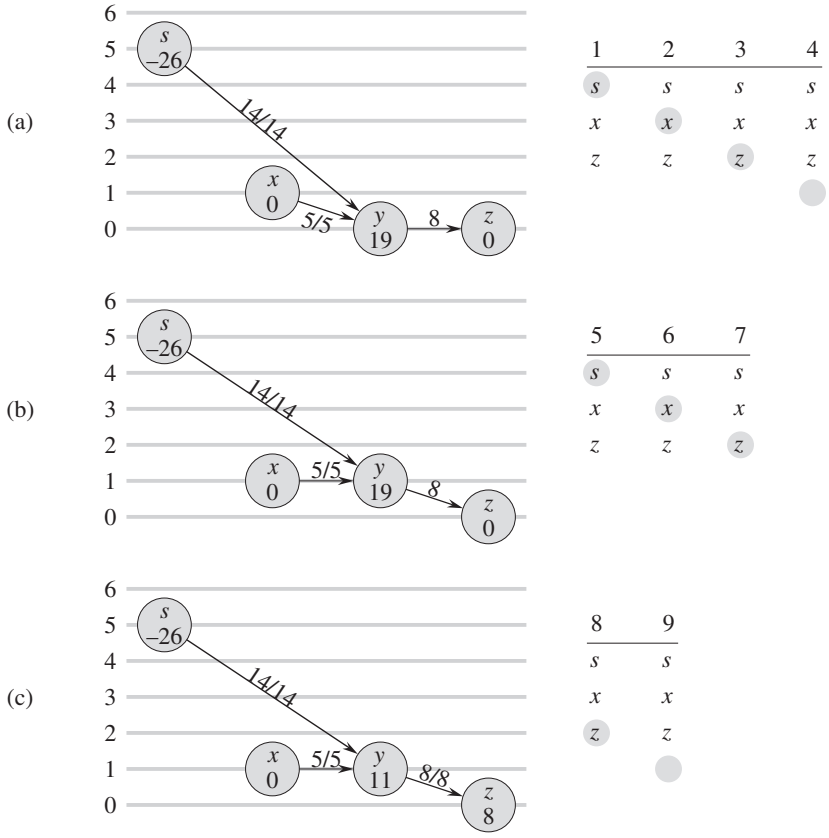
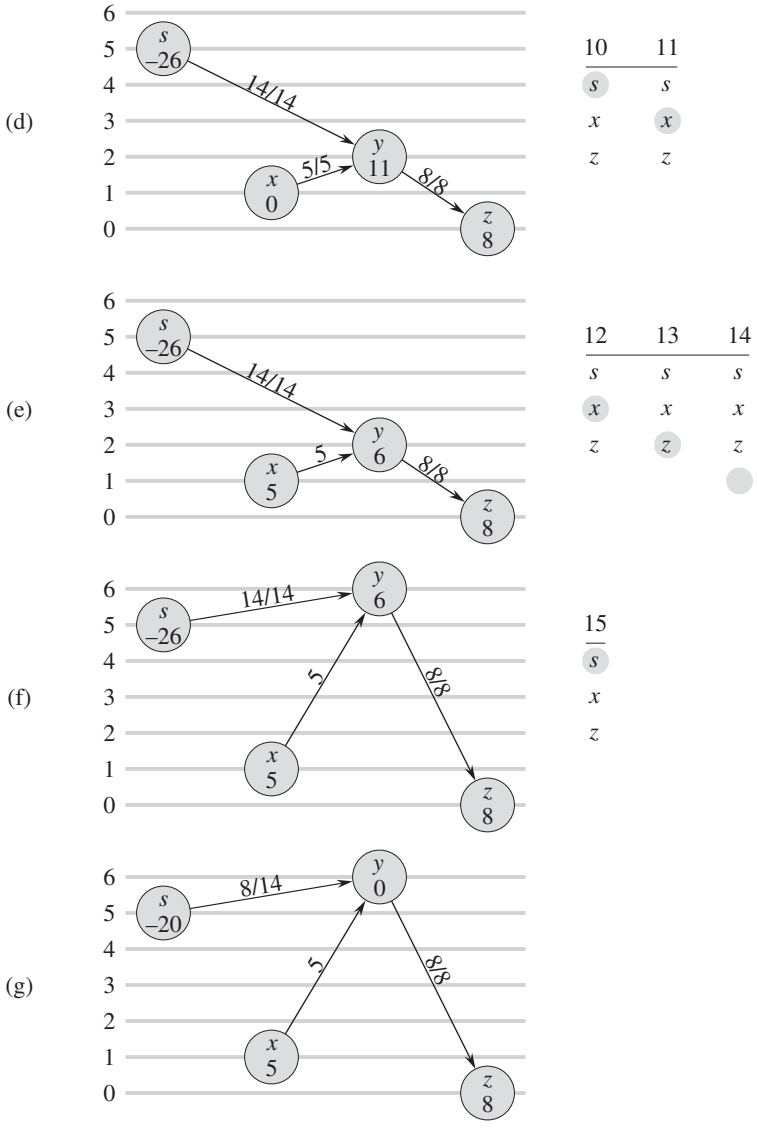


Figura 26.9 Processo di scarica del vertice y. Occorrono 15 iterazioni del ciclo **while** della procedura DISCHARGE per inviare tutto il flusso in eccesso da y. Sono rappresentati soltanto i vertici vicini a y e gli archi della rete di flusso che entrano o escono da y. In ciascuna parte della figura, il numero all'interno di ciascun vertice indica il suo flusso in eccesso all'inizio della prima iterazione che è rappresentata in quella parte; ogni vertice è posto all'altezza che gli compete per tutta la durata di quella parte. A destra è illustrata la lista dei vicini $y.N$ all'inizio di ogni iterazione, con il numero di iterazione in alto. Il vicino ombreggiato è $y.current$. **(a)** Inizialmente, ci sono 19 unità di flusso in eccesso da inviare da y, e $y.current = s$. Le iterazioni 1, 2 e 3 fanno soltanto avanzare $y.current$, perché non ci sono archi ammissibili che escono da y. Nell'iterazione 4, $y.current = \text{NIL}$ (come indica il cerchio ombreggiato sotto la lista dei vicini), quindi y viene innalzato e $y.current$ viene riportato a puntare al primo elemento della lista dei vicini. **(b)** Dopo l'innalzamento, il vertice y ha altezza 1. Nelle iterazioni 5 e 6, gli archi (y, s) e (y, x) risultano inammissibili, ma 8 unità di flusso in eccesso vengono inviate da y a z nell'iterazione 7. A causa dell'invio, $y.current$ non avanza in questa iterazione. **(c)** Poiché l'invio nell'iterazione 7 ha saturato (y, z), quest'arco è considerato inammissibile nell'iterazione 8. Nell'iterazione 9 si ha $y.current = \text{NIL}$, quindi il vertice y viene innalzato di nuovo e $y.current$ è riportato a puntare al primo elemento di $y.N$. **(d)** Nell'iterazione 10, l'arco (y, s) è inammissibile, ma 5 unità di flusso in eccesso vengono inviate da y a x nell'iterazione 11. **(e)** Poiché $y.current$ non è stato incrementato nell'iterazione 11, l'iterazione 12 considera inammissibile l'arco (y, x). L'iterazione 13 considera inammissibile l'arco (y, z), e l'iterazione 14 innalza il vertice y e riporta $y.current$ a puntare al primo elemento di $y.N$. **(f)** L'iterazione 15 invia 6 unità di flusso in eccesso da y a s. **(g)** Il vertice y adesso non ha flusso in eccesso, quindi la procedura DISCHARGE termina. In questo esempio, DISCHARGE inizia e termina con $y.current$ che punta alla testa della lista dei vicini, ma in generale non è necessariamente così.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995089016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)



L'algoritmo relabel-to-front

Nell'algoritmo relabel-to-front manteniamo una lista concatenata L che è formata da tutti i vertici in $V - \{s, t\}$. Una proprietà fondamentale è che i vertici in L sono ordinati topologicamente rispetto alla rete ammissibile, come vedremo nella successiva invariante di ciclo (ricordiamo dal Lemma 26.26 che la rete ammissibile è un grafo orientato aciclico).

Nello pseudocodice per l'algoritmo relabel-to-front si suppone che le liste dei vicini $u.N$ siano già state create per ogni vertice u. Si suppone inoltre che $u.next$ punti al vertice che segue u nella lista L e che, come di consueto, $u.next = \text{NIL}$, se u è l'ultimo vertice della lista.

Acquistato da Michele Michelis su Webeta il 2022-07-07 23:12 Author/Original Library: 1995030162726707 © Copyright © 2022 McGraw-Hill Education (Italy)

```
RELABEL-TO-FRONT(G, s, t)
 1  INITIALIZE-PREFLOW(G, s)
 2  L = G.V − {s, t}, in qualsiasi ordine
 3  for ogni vertice u ∈ G.V − {s, t}
 4      u.current = u.N.head
 5  u = L.head
 6  while u ≠ NIL
 7      old-height = u.h
 8      DISCHARGE(u)
 9      if u.h > old-height
10          sposta u in testa alla lista L
11      u = u.next
```

L'algoritmo relabel-to-front opera nel modo seguente. La riga 1 inizializza il preflusso e le altezze con gli stessi valori del generico algoritmo push-relabel. La riga 2 inizializza la lista L per contenere tutti i vertici potenzialmente traboccanti, in un ordine qualsiasi. Le righe 3–4 inizializzano il puntatore *current* di ciascun vertice u con il primo vertice nella lista dei vicini di u.

Come illustra la Figura 26.10, il ciclo **while** delle righe 6–11 attraversa la lista L, scaricando i vertici. La riga 5 fa iniziare il ciclo dal primo vertice della lista. A ogni esecuzione del ciclo, viene scaricato un vertice u nella riga 8. Se il vertice u viene innalzato dalla procedura DISCHARGE, la riga 10 lo sposta in testa alla lista L. Possiamo scoprire se il vertice u è stato innalzato confrontando l'altezza che esso aveva prima di essere scaricato, e che è stata salvata nella variabile *old-height*, con la sua altezza dopo l'operazione (riga 9). La riga 11 fa in modo che la successiva iterazione del ciclo **while** usi il vertice che segue u nella lista L. Se u è stato spostato in testa alla lista, il vertice utilizzato nella successiva iterazione è quello che segue u nella sua nuova posizione nella lista.

Per dimostrare che la procedura RELABEL-TO-FRONT calcola un flusso massimo, proveremo che essa è un'implementazione del generico algoritmo push-relabel. Innanzi tutto, notiamo che essa esegue le operazioni push e relabel soltanto quando queste sono applicabili, perché il Lemma 26.29 garantisce che DISCHARGE esegue tali operazioni soltanto quando sono applicabili. Resta da provare che, quando RELABEL-TO-FRONT termina, non è applicabile nessuna operazione di base. La parte restante dell'esame della correttezza dell'algoritmo si basa sulla seguente invariante di ciclo:

> In ogni test della riga 6 di RELABEL-TO-FRONT, la lista L è un ordinamento topologico dei vertici nella rete ammissibile $G_{f,h} = (V, E_{f,h})$ e nessun vertice che precede u nella lista ha flusso in eccesso.

Inizializzazione: subito dopo l'esecuzione di INITIALIZE-PREFLOW, $s.h = |V|$ e $v.h = 0$ per ogni $v \in V - \{s\}$. Poiché $|V| \geq 2$ (perché V contiene almeno s e t), nessun arco può essere ammissibile. Quindi, $E_{f,h} = \emptyset$ e qualsiasi ordinamento di $V - \{s, t\}$ è un ordinamento topologico di $G_{f,h}$. Poiché u inizialmente è la testa della lista L, non ci sono vertici prima di esso e quindi nessun vertice prima di u può avere flusso in eccesso.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-9 Copyright © 2022, McGraw-Hill Education (Italy).

Conservazione: per verificare che l'ordinamento topologico viene mantenuto in ogni iterazione del ciclo **while**, iniziamo osservando che la rete ammissibile viene modificata soltanto da operazioni push e relabel. Per il Lemma 26.27, le

operazioni push non producono archi ammissibili. Quindi, gli archi ammissibili possono essere creati soltanto da operazioni relabel. Dopo che un vertice u è stato innalzato, tuttavia, il Lemma 26.28 stabilisce che non ci sono archi ammissibili che entrano in u, ma possono esserci archi ammissibili che escono da u. Quindi, spostando u in testa alla lista L, l'algoritmo garantisce che qualsiasi arco ammissibile che esce da u soddisfa l'ordinamento topologico.

Per verificare che nessun vertice che precede u in L ha flusso in eccesso, indichiamo con u' il vertice che diventerà u nella successiva iterazione. I vertici che precederanno u' nella successiva iterazione includono il vertice corrente u (per la riga 11) e o nessun altro vertice (se u viene innalzato) o gli stessi vertici di prima (se u non viene innalzato). Poiché il vertice u viene scaricato, dopo non avrà flusso in eccesso. Quindi, se u viene innalzato mentre viene scaricato, nessun vertice che precede u' avrà flusso in eccesso. Se u non viene innalzato mentre viene scaricato, nessun vertice che lo precede nella lista ha accumulato flusso in eccesso durante tale scaricamento, perché la lista L resta sempre topologicamente ordinata durante lo scaricamento (come detto in precedenza, gli archi ammissibili sono creati soltanto da operazioni relabel, non da operazioni push), e quindi ciascuna operazione push fa sì che il flusso in eccesso si sposti soltanto sui vertici più in basso nella lista (oppure su s o t). Anche in questo caso, nessun vertice che precede u' avrà flusso in eccesso.

Conclusione: quando il ciclo termina, u ha appena superato la fine della lista L, e quindi l'invariante di ciclo garantisce che il flusso in eccesso di ogni vertice è 0. Dunque, non è applicabile nessuna operazione di base.

Analisi

Dimostriamo adesso che la procedura RELABEL-TO-FRONT viene eseguita nel tempo $O(V^3)$ su una qualsiasi rete di flusso $G = (V, E)$. Poiché l'algoritmo relabel-to-front è un'implementazione del generico algoritmo push-relabel, utilizzeremo il Corollario 26.21 che fornisce un limite $O(V)$ per il numero di operazioni relabel eseguite per ciascun vertice e un limite $O(V^2)$ per il numero totale di operazioni relabel complessivamente eseguite. Inoltre, l'Esercizio 26.4-3 fornisce un limite $O(VE)$ per il tempo totale impiegato per eseguire le operazioni relabel e il Lemma 26.22 fornisce un limite $O(VE)$ per il numero totale di invii saturanti.

Teorema 26.30
Il tempo di esecuzione di RELABEL-TO-FRONT su una qualsiasi rete di flusso $G = (V, E)$ è $O(V^3)$.

Dimostrazione Consideriamo una "fase" dell'algoritmo relabel-to-front come il tempo che passa tra due operazioni relabel consecutive. Ci sono $O(V^2)$ fasi, perché ci sono $O(V^2)$ operazioni relabel. Ogni fase è formata al massimo da $|V|$ chiamate di DISCHARGE; questo può essere dimostrato nel modo seguente. Se DISCHARGE non esegue un'operazione relabel, allora la successiva chiamata di DISCHARGE interesserà un vertice successivo nella lista L, e la lunghezza di L è minore di $|V|$. Se DISCHARGE esegue un'operazione relabel, la successiva chiamata di DISCHARGE apparterrà a una fase differente. Poiché ogni fase contiene al massimo $|V|$ chiamate di DISCHARGE e ci sono $O(V^2)$ fasi, il numero di volte che viene chiamata DISCHARGE nella riga 8 di RELABEL-TO-FRONT è $O(V^3)$. Quindi, il lavoro totale svolto dal ciclo **while** in RELABEL-TO-FRONT, escludendo il lavoro svolto all'interno di DISCHARGE, è al massimo $O(V^3)$.

Acquistato da Michele Mastrodonato (Matricola 2022-07-27 22:12, Numero Ordine Libreria 1855139) per uso personale. © 2024 McGraw-Hill Education (Italy)

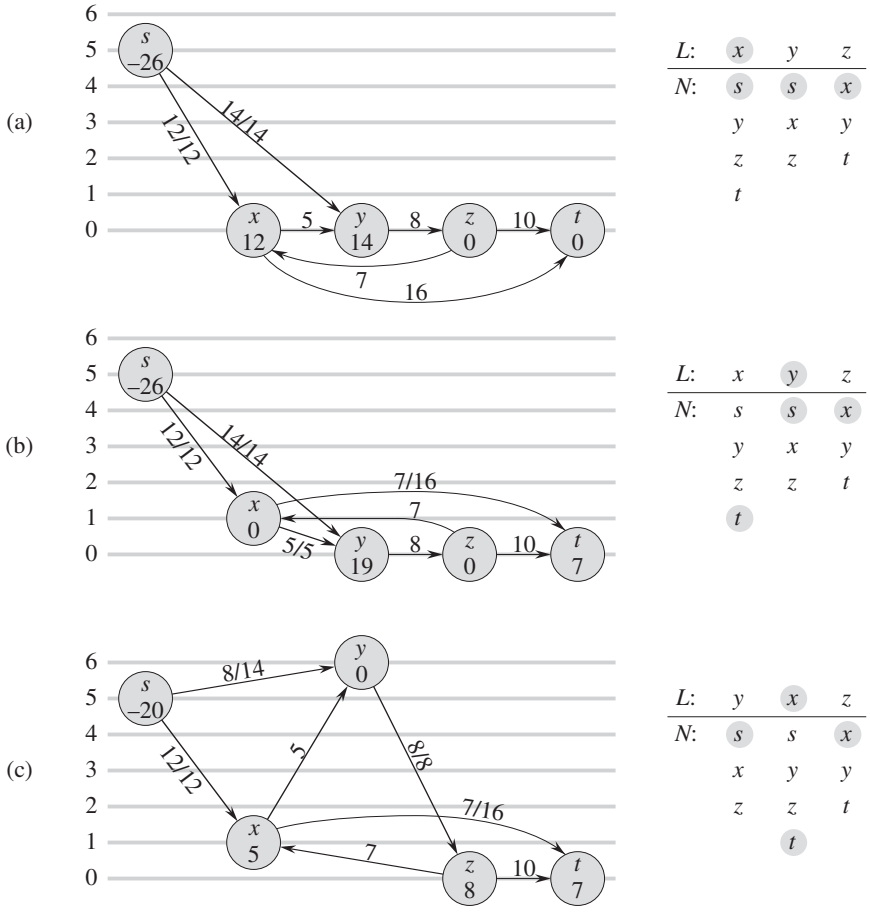
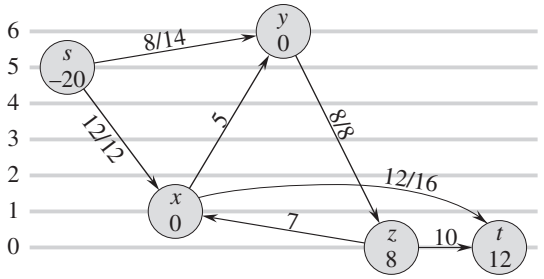


Figura 26.10 Il funzionamento di RELABEL-TO-FRONT. **(a)** Una rete di flusso appena prima della prima iterazione del ciclo **while**. Inizialmente, 26 unità di flusso lasciano la sorgente s. A destra è indicata la lista iniziale $L = \langle x, y, z \rangle$, dove inizialmente $u = x$. Sotto ciascun vertice della lista L è indicata la sua lista dei vicini (il vicino corrente è ombreggiato). Il vertice x è scaricato; viene portato all'altezza 1; 5 unità di flusso in eccesso vengono inviate a y e le restanti 7 unità di flusso in eccesso vengono inviate al pozzo t. Poiché il vertice x viene innalzato, viene spostato in testa alla lista L, la cui struttura in questo caso non cambia. **(b)** Dopo x, il vertice successivo in L che viene scaricato è y. La Figura 26.9 illustra i dettagli dell'operazione di scarico del vertice y in questa situazione. Poiché il vertice y viene innalzato, esso viene spostato in testa alla lista L. **(c)** Il vertice x adesso segue y nella lista L, e quindi viene scaricato di nuovo, inviando tutte le 5 unità di flusso in eccesso a t. Poiché il vertice x non viene innalzato in questa operazione di scarica, esso resta al suo posto nella lista L. **(d)** Il vertice z viene scaricato perché segue il vertice x nella lista L; viene portato all'altezza 1 e tutte le 8 unità di flusso in eccesso vengono inviate a t. Poiché z viene innalzato, viene spostato in testa alla lista L. **(e)** Il vertice y adesso segue il vertice z in L e quindi viene scaricato. Tuttavia, poiché y non ha flusso in eccesso, la procedura DISCHARGE termina immediatamente e y resta al suo posto in L. Poi viene scaricato il vertice x. Poiché anche questo vertice non ha flusso in eccesso, la DISCHARGE termina e x resta al suo posto in L. RELABEL-TO-FRONT ha raggiunto la fine della lista L e termina. Non ci sono vertici traboccanti e il preflusso è un flusso massimo.

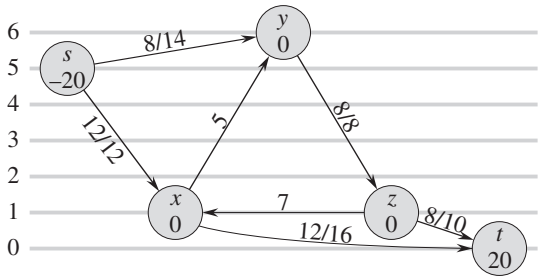
Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libraria 1995303016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

(d)



L:	y	x	z
N:	s	s	x
	x	y	y
	z	z	t
		t	

(e)



L:	z	y	x
N:	x	s	s
	y	x	y
	t	z	z
			t

A questo punto, dobbiamo trovare un limite al lavoro svolto all'interno di DISCHARGE durante l'esecuzione dell'algoritmo. Ogni iterazione del ciclo **while** all'interno di DISCHARGE esegue una delle tre azioni possibili. Analizzeremo la quantità totale di lavoro svolto per ciascuna di queste azioni.

Iniziamo con le operazioni relabel (righe 4–5). L'Esercizio 26.4-3 fornisce un limite $O(VE)$ per tutte le $O(V^2)$ operazioni relabel che vengono eseguite.

Adesso, supponiamo che l'azione aggiorni il puntatore $u.current$ nella riga 8. Questa azione si verifica $O(\text{degree}(u))$ volte, per ogni volta che il vertice u viene innalzato, e complessivamente $O(V \cdot \text{degree}(u))$ volte per tale vertice. Per tutti i vertici, quindi, la quantità totale di lavoro svolto per fare avanzare i puntatori nelle liste dei vicini è $O(VE)$, per il lemma della stretta di mano (Esercizio B.4-1).

Il terzo tipo di azione che esegue DISCHARGE è l'operazione push (riga 7). Sappiamo già che il numero totale di invii saturanti è $O(VE)$. Notiamo che, se viene eseguito un invio non saturante, DISCHARGE termina immediatamente, in quanto l'invio annulla il flusso in eccesso. Quindi, ci può essere al massimo un invio non saturante per ogni chiamata di DISCHARGE. Come già osservato, la procedura DISCHARGE viene chiamata $O(V^3)$ volte, e quindi il tempo totale impiegato per eseguire gli invii non saturanti è $O(V^3)$.

Il tempo di esecuzione di RELABEL-TO-FRONT è dunque $O(V^3 + VE)$, che è pari a $O(V^3)$. ■

Esercizi

26.5-1

Illustrate l'esecuzione di RELABEL-TO-FRONT secondo la rappresentazione della Figura 26.10 per la rete di flusso della Figura 26.1(a). Supponete che l'ordinamento iniziale dei vertici in L sia $\langle v_1, v_2, v_3, v_4 \rangle$ e che le liste dei vicini siano

$$
\begin{aligned}
v_1.N &= \langle s, v_2, v_3 \rangle \\
v_2.N &= \langle s, v_1, v_3, v_4 \rangle \\
v_3.N &= \langle v_1, v_2, v_4, t \rangle \\
v_4.N &= \langle v_2, v_3, t \rangle
\end{aligned}
$$

Acquistato da Michele Mi[illegible] - ISBN 9788838693[illegible] - Copyright © 2023 McGraw-Hill Education (Italy)

26.5-2 ⋆
Implementate un algoritmo push-relabel con una coda FIFO di vertici traboccanti. L'algoritmo scarica ripetutamente il vertice che si trova all'inizio alla coda; i vertici che non erano traboccanti prima dell'operazione di scarica, ma che lo diventano dopo, vengono posti in fondo alla coda. Dopo che il vertice all'inizio della coda è stato scaricato, viene rimosso dalla coda. Quando la coda è vuota, l'algoritmo termina. Dimostrate che questo algoritmo può essere implementato per calcolare un flusso massimo nel tempo $O(V^3)$.

26.5-3
Dimostrate che il generico algoritmo push-relabel continua a funzionare se RELABEL aggiorna $u.h$ calcolando semplicemente $u.h = u.h + 1$. In che modo questa modifica influisce sull'analisi di RELABEL-TO-FRONT?

26.5-4 ⋆
Dimostrate che, se viene scaricato sempre un vertice traboccante di altezza massima, il metodo push-relabel può essere eseguito nel tempo $O(V^3)$.

26.5-5
Supponete che, a un certo punto dell'esecuzione di un algoritmo push-relabel, esista un intero $0 < k \leq |V|-1$ per il quale nessun vertice ha $v.h = k$. Dimostrate che tutti i vertici con $v.h > k$ si trovano sul lato della sorgente rispetto a un taglio minimo. Se un tale valore di k esiste, l'***euristica del gap*** aggiorna ogni vertice $v \in V - s$ per il quale $v.h > k$ ponendo $v.h = \max(v.h, |V| + 1)$. Dimostrate che l'attributo h risultante è una funzione altezza (l'euristica del gap è cruciale per ottenere nella pratica buone prestazioni delle implementazioni del metodo push-relabel).

Problemi

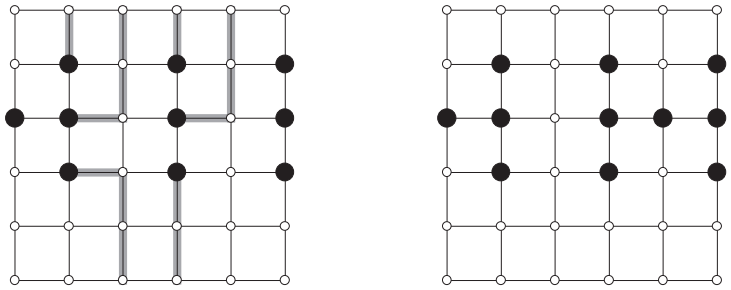
26-1 Il problema della fuga
Una ***griglia*** $n \times n$ è un grafo non orientato che è formata da n righe ed n colonne di vertici, come illustra la Figura 26.11. Indichiamo con (i, j) il vertice che si trova nella i-esima riga e nella j-esima colonna. Tutti i vertici in una griglia hanno esattamente quattro vicini, tranne i vertici che si trovano lungo il perimetro, cioè i punti (i, j) per i quali $i = 1$, $i = n$, $j = 1$ o $j = n$.

Dati $m \leq n^2$ punti di partenza $(x_1, y_1), (x_2, y_2), \ldots, (x_m, y_m)$ nella griglia, il ***problema della fuga*** consiste nel determinare se esistono m cammini disgiunti sui vertici che congiungono i punti di partenza con m punti distinti sul perimetro della griglia. Per esempio, la griglia nella Figura 26.11(a) ha una fuga, mentre quella nella Figura 26.11(b) non ha una fuga.

a. Considerate una rete di flusso in cui anche i vertici, oltre agli archi, hanno una capacità. Ovvero, il flusso totale positivo che entra in un dato vertice è soggetto a un vincolo sulla capacità. Dimostrate che il problema di trovare il flusso massimo in una rete i cui archi e vertici hanno capacità può essere ricondotto a un ordinario problema di flusso massimo in una rete di flusso di dimensione comparabile.

Acquistato da Michele Michele su Webster il 2022-07-07 [illegible]

b. Descrivete un algoritmo efficiente per risolvere il problema della fuga e analizzate il suo tempo di esecuzione.



(a) (b)

Figura 26.11 Le griglie per il problema della fuga. I punti di partenza sono neri; gli altri vertici della griglia sono bianchi. **(a)** Una griglia con una fuga, indicata dai cammini ombreggiati. **(b)** Una griglia senza fuga.

26-2 Copertura minima di cammini

Una ***copertura di cammini*** di un grafo orientato $G = (V, E)$ è un insieme P di cammini disgiunti sui vertici tale che ogni vertice in V è incluso in un solo cammino in P. I cammini possono iniziare e terminare in qualsiasi punto e possono avere qualsiasi lunghezza, anche 0. Una ***copertura minima di cammini*** di G è una copertura di cammini che contiene il minor numero possibile di cammini.

a. Create un algoritmo efficiente per trovare una copertura minima di cammini di un grafo orientato aciclico $G = (V, E)$. (*Suggerimento:* supponendo che $V = \{1, 2, \ldots, n\}$, costruite il grafo $G' = (V', E')$, dove

$$\begin{aligned} V' &= \{x_0, x_1, \ldots, x_n\} \cup \{y_0, y_1, \ldots, y_n\} \\ E' &= \{(x_0, x_i) : i \in V\} \cup \{(y_i, y_0) : i \in V\} \cup \{(x_i, y_j) : (i, j) \in E\} \end{aligned}$$

ed eseguite un algoritmo di flusso massimo.)

b. Il vostro algoritmo funziona con i grafi orientati che contengono cicli? Spiegate la risposta.

26-3 Consulenza algoritmica

Il professor Gore vuole creare una società di consulenza algoritmica. Ha identificato n classi importanti di algoritmi (che corrispondono approssimativamente alle diverse parti di questo libro) e le ha rappresentate con un insieme $A = \{A_1, A_2, \ldots, A_n\}$. Per ciascuna classe A_k, può assumere un esperto in tale area per c_k dollari. La società di consulenza ha preparato un insieme $J = \{J_1, J_2, \ldots, J_m\}$ di possibili progetti. Per realizzare il progetto J_i, la società ha bisogno di assumere gli esperti in un sottoinsieme $R_i \subseteq A$ di classi. Ogni esperto può partecipare a più progetti contemporaneamente. Se la società decide di realizzare il progetto J_i, deve assumere gli esperti in tutte le classi di R_i, e avrà un ricavo di p_i dollari.

Il compito del professor Gore è determinare le classi per le quali assumere gli esperti e quali progetti selezionare per massimizzare il guadagno netto, che è dato dal ricavato totale dei progetti selezionati meno il costo totale per i compensi degli esperti.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Considerate la rete di flusso G che contiene una sorgente s, i vertici $A_1, A_2, \ldots, A_n$, i vertici $J_1, J_2, \ldots, J_m$ e un pozzo t. Per $k = 1, 2 \ldots, n$, la rete di flusso contiene un arco (s, A_k) di capacità $c(s, A_k) = c_k$ e, per $i = 1, 2, \ldots, m$, esiste un arco (J_i, t) di capacità $c(J_i, t) = p_i$. Per $k = 1, 2, \ldots, n$ e $i = 1, 2, \ldots, m$, se $A_k \in R_i$, allora G contiene un arco (A_k, J_i) di capacità $c(A_k, J_i) = \infty$.

a. Dimostrate che, se $J_i \in T$ per un taglio di capacità finita (S, T) di G, allora $A_k \in T$ per ogni $A_k \in R_i$.

b. Spiegate come ottenere il guadagno netto massimo dalla capacità del taglio minimo di G e dai valori dati di p_i.

c. Create un algoritmo efficiente per determinare quali progetti realizzare e quali esperti assumere. Analizzate il tempo di esecuzione del vostro algoritmo in funzione di m, n ed $r = \sum_{i=1}^{m} |R_i|$.

26-4 Aggiornamento del flusso massimo

Sia $G = (V, E)$ una rete di flusso con sorgente s, pozzo t e capacità intere. Supponete di conoscere un flusso massimo in G.

a. Supponete che la capacità di un singolo arco $(u, v) \in E$ aumenti di una unità. Create un algoritmo con tempo $O(V + E)$ per aggiornare il flusso massimo.

b. Supponete che la capacità di un singolo arco $(u, v) \in E$ diminuisca di una unità. Create un algoritmo con tempo $O(V + E)$ per aggiornare il flusso massimo.

26-5 Algoritmo progressivo per il flusso massimo

Sia $G = (V, E)$ una rete di flusso con sorgente s, pozzo t e una capacità intera $c(u, v)$ in ogni arco $(u, v) \in E$. Sia $C = \max_{(u,v)\in E} c(u, v)$.

a. Dimostrate che un taglio minimo di G ha capacità al più $C\,|E|$.

b. Per un dato numero K, dimostrate che è possibile trovare nel tempo $O(E)$ un cammino aumentante di capacità almeno pari a K, se un tale cammino esiste.

La seguente variante di FORD-FULKERSON-METHOD può essere utilizzata per calcolare un flusso massimo in G.

```
MAX-FLOW-BY-SCALING(G, s, t)
1  C = max_{(u,v)∈E} c(u, v)
2  inizializza il flusso f a 0
3  K = 2^⌊lg C⌋
4  while K ≥ 1
5       while esiste un cammino aumentante p di capacità almeno pari a K
6            aumenta il flusso f lungo p
7       K = K/2
8  return f
```

c. Dimostrate che MAX-FLOW-BY-SCALING restituisce un flusso massimo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995303016-220707 © Copyright © 2022, McGraw-Hill Education (Italy)

d. Dimostrate che la capacità di un taglio minimo del grafo residuo G_f vale al più $2K\,|E|$ ogni volta che viene eseguita la riga 4.

e. Dimostrate che il ciclo **while** interno (righe 5–6) viene eseguito $O(E)$ volte per ogni valore di K.

f. Concludete che la procedura MAX-FLOW-BY-SCALING può essere implementata con un tempo di esecuzione pari a $O(E^2 \lg C)$.

26-6 Algoritmo di Hopcroft-Karp per l'abbinamento nei grafi bipartiti

In questo problema descriviamo un algoritmo più veloce, sviluppato da Hopcroft e Karp, per trovare un abbinamento massimo in un grafo bipartito. L'algoritmo viene eseguito nel tempo $O(\sqrt{V}E)$. Dato un grafo non orientato bipartito $G = (V, E)$, dove $V = L \cup R$ e tutti gli archi hanno una sola estremità in L, sia M un abbinamento in G. Diciamo che un cammino semplice P in G è un ***cammino aumentante*** rispetto a M se inizia in un vertice non accoppiato in L, termina in un vertice non accoppiato in R e i suoi archi appartengono alternativamente a M e a $E - M$ (questa definizione di cammino aumentante, anche se differente, è in relazione con un cammino aumentante in una rete di flusso). In questo problema, trattiamo un cammino come una sequenza di archi, anziché come una sequenza di vertici. Un cammino minimo aumentante rispetto a un abbinamento M è un cammino aumentante con un numero minimo di archi. Dati due insiemi A e B, la ***differenza simmetrica*** $A \oplus B$ è definita come $(A - B) \cup (B - A)$, che identifica gli elementi che si trovano in uno solo dei due insiemi.

a. Dimostrate che, se M è un abbinamento e P è un cammino aumentante rispetto a M, allora la differenza simmetrica $M \oplus P$ è un abbinamento e $|M \oplus P| = |M| + 1$. Dimostrate che, se $P_1, P_2, \ldots, P_k$ sono cammini aumentanti disgiunti sui vertici rispetto a M, allora la differenza simmetrica $M \oplus (P_1 \cup P_2 \cup \cdots \cup P_k)$ è un abbinamento con cardinalità $|M| + k$.

La struttura generale del nostro algoritmo è la seguente:

```
HOPCROFT-KARP(G)
1  M = ∅
2  repeat
3       sia P = {P_1, P_2, ..., P_k} un insieme massimale di cammini minimi
            aumentanti disgiunti sui vertici rispetto a M
4       M = M ⊕ (P_1 ∪ P_2 ∪ ··· ∪ P_k)
5  until P == ∅
6  return M
```

La parte restante di questo problema chiede di analizzare il numero di iterazioni nell'algoritmo (ovvero il numero di iterazioni nel ciclo **repeat**) e di descrivere un'implementazione della riga 3.

b. Dati due abbinamenti M ed M^* in G, dimostrate che ogni vertice nel grafo $G' = (V, M \oplus M^*)$ ha grado al più 2. Concludete che G' è una unione disgiunta di cammini semplici o cicli. Dimostrate che gli archi in ciascuno di tali cammini semplici o cicli appartengono alternativamente a M o a M^*. Dimostrate che, se $|M| \leq |M^*|$, allora $M \oplus M^*$ contiene almeno $|M^*| - |M|$ cammini aumentanti disgiunti sui vertici rispetto a M.

Acquistato da Michele Michelangeli (2207747) in data 23/2/2022 | Numero di Licenza: 1450307 | Copyright © 2022, McGraw-Hill Education (Italy)

Sia l la lunghezza di un cammino minimo aumentante rispetto a un abbinamento M; sia $P_1, P_2, \ldots, P_k$ un insieme massimale di cammini aumentanti disgiunti sui vertici di lunghezza l rispetto a M. Sia $M' = M \oplus (P_1 \cup \cdots \cup P_k)$ e supponete che P sia un cammino minimo aumentante rispetto a M'.

c. Dimostrate che, se P è un cammino disgiunto sui vertici da $P_1, P_2, \ldots, P_k$, allora P ha più di l archi.

d. Adesso supponete che P non sia un cammino disgiunto sui vertici da P_1, $P_2, \ldots, P_k$. Sia A l'insieme degli archi $(M \oplus M') \oplus P$. Dimostrate che $A = (P_1 \cup P_2 \cup \cdots \cup P_k) \oplus P$ e che $|A| \geq (k+1)l$. Concludete che P ha più di l archi.

e. Dimostrate che, se un cammino minimo aumentante rispetto a M ha l archi, la dimensione dell'abbinamento massimo è al più $|M| + |V|/(l+1)$.

f. Dimostrate che il numero di iterazioni del ciclo **repeat** nell'algoritmo è al massimo $2\sqrt{|V|}$. (*Suggerimento:* di quanto può crescere M dopo l'iterazione numero $\sqrt{|V|}$?)

g. Create un algoritmo con tempo $O(E)$ per trovare un insieme massimale di cammini minimi aumentanti disgiunti sui vertici $P_1, P_2, \ldots, P_k$ per un dato abbinamento M. Concludete che il tempo totale di esecuzione di HOPCROFT-KARP è $O(\sqrt{V}E)$.

Note

Ahuja, Magnanti e Orlin [7], Even [104], Lawler [225], Papadimitriou e Steiglitz [272] e Tarjan [331] sono buoni testi di riferimento per le reti di flusso e relativi algoritmi. Goldberg, Tardos e Tarjan [140] presentano una bella rassegna di algoritmi per problemi di flusso su rete; Schrijver [305] ha scritto un'interessante analisi degli sviluppi storici nel campo dei flussi su rete.

Il metodo di Ford-Fulkerson è attribuito a Ford e Fulkerson [110], che hanno iniziato lo studio formale di molti dei problemi nell'area del flusso su rete, inclusi i problemi del flusso massimo e dell'abbinamento su grafi bipartiti. Molte tra le prime implementazioni del metodo di Ford-Fulkerson trovavano i cammini aumentanti utilizzando la visita in ampiezza; Edmonds e Karp [103] e, in maniera autonoma, Dinic [90] hanno dimostrato che questa strategia produce un algoritmo con tempo polinomiale. Un concetto analogo, basato sui "flussi bloccanti" (blocking flows), fu sviluppato per primo da Dinic [90]. Karzanov [203] ha sviluppato per primo il concetto di preflusso. Il metodo push-relabel è stato ideato da Goldberg [137] e da Goldberg e Tarjan [141]. Goldberg e Tarjan hanno creato un algoritmo con tempo $O(V^3)$ che usa una coda per mantenere l'insieme dei vertici traboccanti, e un algoritmo che usa gli alberi dinamici per ottenere un tempo di esecuzione pari a $O(VE \lg(V^2/E+2))$.

Molti altri ricercatori hanno sviluppato algoritmi push-relabel per il problema del flusso massimo. Ahuja e Orlin [9] e Ahuja, Orlin e Tarjan [10] hanno sviluppato algoritmi basati sulla progressione. Cheriyan e Maheshwari [63] hanno proposto l'operazione di invio del flusso dal vertice traboccante di altezza massima. Cheriyan e Hagerup [62] hanno suggerito di permutare casualmente le liste dei vicini; molti ricercatori [14, 205, 277] hanno sviluppato delle brillanti tecniche di derandomizzazione di questa idea, ottenendo una serie di algoritmi più veloci. L'algoritmo di King, Rao e Tarjan [205] è il più veloce di tali algoritmi; il suo tempo di esecuzione è $O(VE \log_{E/(V \lg V)} V)$.

Acquistato da Michele Michele su Webstar il 2021-07-13. Il numero di copie autorizzate è 2. Copyright © 2021, McGraw-Hill Education (Italy)

continua

Attualmente, l'algoritmo asintoticamente più veloce per il problema del flusso massimo è attribuito a Goldberg e Rao [139]; il tempo di esecuzione dell'algoritmo è $O(\min(V^{2/3}, E^{1/2})E \lg(V^2/E + 2) \lg C)$, dove $C = \max_{(u,v)\in E} c(u, v)$. Questo algoritmo non usa il metodo push-relabel, ma si basa sulla ricerca dei flussi bloccanti. Tutti i precedenti algoritmi di flusso massimo, inclusi quelli descritti in questo capitolo, usano una certa nozione di distanza (gli algoritmi push-relabel usano la nozione analoga di altezza), con una lunghezza unitaria assegnata implicitamente a ciascun arco. Questo nuovo algoritmo adotta un approccio differente e assegna una lunghezza 0 agli archi di alta capacità e una lunghezza 1 agli archi di bassa capacità. Informalmente, riguardo a queste lunghezze, i cammini minimi dalla sorgente al pozzo tendono ad avere un'elevata capacità; questo significa che è sufficiente eseguire un minor numero di iterazioni.

Nella pratica, gli algoritmi push-relabel attualmente prevalgono sugli algoritmi basati sui cammini aumentanti o sulla programmazione lineare per il problema del flusso massimo. Uno studio di Cherkassky e Goldberg [64] sottolinea l'importanza dell'uso di due euristiche per implementare un algoritmo push-relabel. La prima euristica è quella di eseguire periodicamente una visita in ampiezza del grafo residuo per ottenere valori più precisi delle altezze. La seconda euristica è quella del gap, descritta nell'Esercizio 26.5-5. Essi concludono che la scelta migliore tra le varianti push-relabel è quella di scaricare il vertice traboccante con l'altezza massima.

Attualmente, l'algoritmo migliore per trovare l'abbinamento massimo sui grafi bipartiti, scoperto da Hopcroft e Karp [177], viene eseguito nel tempo $O(\sqrt{V}E)$ ed è descritto nel Problema 26-6. Il libro di Lovász e Plummer [240] è un eccellente testo di consultazione per i problemi di abbinamento.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

VII Argomenti scelti

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Introduzione

Questa parte tratta una serie di argomenti scelti sugli algoritmi con l'obiettivo di estendere e integrare i concetti precedentemente descritti in questo libro. Alcuni capitoli introducono nuovi modelli di calcolo, come i circuiti combinatori o i calcolatori paralleli; altri capitoli sono dedicati a materie specifiche, come la geometria computazionale o la teoria dei numeri. Gli ultimi due capitoli descrivono alcune limitazioni nella progettazione di algoritmi efficienti e introducono le tecniche per aggirare tali limitazioni.

Il Capitolo 27 presenta un modello algoritmico per il calcolo parallelo basato sul multithreading dinamico. Il capitolo illustra i concetti di base del modello, spiegando come quantificare il parallelismo in termini di *lavoro* (*work*) e *durata* (*span*); poi esamina alcuni interessanti algoritmi multithread, inclusi gli algoritmi merge sort e quelli che eseguono il prodotto tra matrici.

Il Capitolo 28 descrive alcuni algoritmi efficienti per operare con le matrici. Presenta due metodi generali – la scomposizione LU e la scomposizione LUP – per risolvere nel tempo $O(n^3)$ le equazioni lineari con il metodo di eliminazione di Gauss. Vedremo che anche le operazioni di inversione e moltiplicazione delle matrici possono essere eseguite altrettanto rapidamente. Il capitolo si conclude mostrando come sia possibile calcolare una soluzione approssimata con il metodo dei minimi quadrati quando un insieme di equazioni lineari non ha una soluzione esatta.

Il Capitolo 29 tratta la programmazione lineare, nella quale si vuole massimizzare o minimizzare un obiettivo, avendo a disposizione risorse limitate e rispettando contemporaneamente più vincoli. La programmazione lineare ha numerose applicazioni pratiche. Questo capitolo tratta la formulazione e la soluzione di programmi lineari. Il metodo di risoluzione descritto è l'algoritmo del simplesso, che è il più vecchio algoritmo di programmazione lineare. Diversamente dalla maggior parte degli algoritmi presentati in questo libro, l'algoritmo del simplesso non viene eseguito in un tempo polinomiale nel caso peggiore, ma è abbastanza efficiente e ampiamente utilizzato nella pratica.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Il Capitolo 30 studia le operazioni sui polinomi e spiega come una ben nota tecnica di elaborazione dei segnali – la trasformata rapida di Fourier o FFT (Fast Fourier Transform) – possa essere utilizzata per moltiplicare due polinomi di grado n nel tempo $O(n \lg n)$. Il capitolo esamina anche alcune implementazioni efficienti della FFT, incluso un circuito parallelo.

Il Capitolo 31 è dedicato agli algoritmi basati sulla teoria dei numeri. Dopo avere illustrato i concetti fondamentali della teoria dei numeri, il capitolo presenta l'algoritmo di Euclide per calcolare il massimo comun divisore; poi, descrive gli algoritmi per risolvere le equazioni lineari modulari e per elevare un numero a una potenza modulo un altro numero. Viene descritta anche un'importante applicazione degli algoritmi basati sulla teoria dei numeri: la crittografia a chiave pubblica RSA. Questo sistema di crittografia può essere utilizzato non soltanto per scrivere messaggi in codice, in modo che non possano essere letti dagli avversari, ma anche per generare firme digitali. Il capitolo poi presenta la versione randomizzata del test di primalità di Miller-Rabin, che può essere utilizzata per trovare in modo efficiente i numeri primi grandi – un requisito fondamentale per il sistema RSA. Infine, il capitolo tratta l'euristica del "rho" di Pollard per fattorizzare i numeri interi e descrive lo stato dell'arte della fattorizzazione dei numeri interi.

Il Capitolo 32 studia il problema di pattern matching su stringhe, che consiste nel trovare tutte le occorrenze di una stinga di caratteri all'interno di un'altra stringa; questo problema si presenta spesso nei programmi di elaborazione dei testi. Dopo avere esaminato un metodo semplice, il capitolo ne illustra uno più elegante, quello sviluppato da Rabin e Karp. Poi descrive una soluzione efficiente basata sugli automi a stati finiti e, infine, presenta l'algoritmo di Knuth-Morris-Pratt, che raggiunge l'efficienza grazie a un'intelligente preelaborazione della stringa da cercare.

La geometria computazionale è l'argomento del Capitolo 33. Dopo avere illustrato i concetti fondamentali della geometria computazionale, il capitolo presenta un metodo di "scansione" in grado di determinare in modo efficiente se un insieme di segmenti di retta contiene delle intersezioni. Due intelligenti algoritmi per trovare l'involucro convesso di un insieme di punti – l'algoritmo di Graham e l'algoritmo di Jarvis – dimostrano la potenza dei metodi di scansione. Il capitolo termina con un algoritmo efficiente per trovare la coppia più vicina in un dato insieme di punti nel piano.

Il Capitolo 34 è dedicato ai problemi NP-completi. Molti interessanti problemi computazionali sono NP-completi, ma attualmente non conosciamo alcun algoritmo con tempo polinomiale che sia in grado di risolverli. Questo capitolo presenta le tecniche per determinare quando un problema è NP-completo. Si è dimostrato che molti dei problemi classici sono NP-completi: determinare se un grafo ha un ciclo hamiltoniano, determinare se una formula booleana è soddisfattibile e determinare se un dato insieme di numeri ha un sottoinsieme i cui elementi hanno per somma un determinato valore. Il capitolo dimostra inoltre che il noto problema del commesso viaggiatore è NP-completo.

Il Capitolo 35 spiega come sia possibile utilizzare gli algoritmi approssimati per trovare in modo efficiente soluzioni approssimate di problemi NP-completi. Per alcuni problemi NP-completi, è facile trovare soluzioni approssimate che sono quasi ottime, ma per altri problemi perfino i migliori algoritmi approssimati operano sempre peggio al crescere della dimensione del problema. Poi, ci sono alcuni problemi per i quali è possibile impiegare quantità crescenti di tempo di calcolo per ottenere soluzioni approssimate sempre migliori. Il capitolo illustra queste possibilità con il problema della copertura dei vertici (versioni pesata e non pesata), una versione di ottimizzazione della soddisfattibilità 3-CNF, il problema del commesso viaggiatore, il problema della copertura di insiemi e il problema della somma dei sottoinsiemi.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Algoritmi multithread 27

La grande maggioranza degli algoritmi di questo libro sono ***algoritmi seriali*** adatti per essere eseguiti in un computer con un solo processore, che può eseguire una sola istruzione alla volta. In questo capitolo, estenderemo il nostro modello algoritmico agli ***algoritmi paralleli***, che possono essere eseguiti in un computer multiprocessore che consente di eseguire più istruzioni contemporaneamente. In particolare, esamineremo l'elegante modello degli algoritmi multithread dinamici, che si prestano alla progettazione e all'analisi algoritmica, come pure a un'efficiente implementazione pratica.

I computer paralleli – computer con più unità di elaborazione – sono diventati molto comuni e sono commercializzati in un'ampia gamma di prezzi e prestazioni. I ***chip multiprocessore*** per computer desktop e portatili sono relativamente poco costosi; sono formati da un unico ***chip multicore*** che ospita più unità di elaborazione, ciascuna delle quali è un processore che può accedere a una memoria comune. A un livello intermedio di prezzi/prestazioni ci sono i *cluster* derivati da singoli computer – spesso semplici PC – con una rete di interconnessione a loro dedicata. Le macchine più costose sono i supercomputer, che spesso utilizzano una combinazione di architetture e reti studiate appositamente per ottenere le massime prestazioni in termini di istruzioni eseguite al secondo.

I computer multiprocessore esistono, in una forma o nell'altra, da decenni. Sebbene fin dall'inizio della storia dell'informatica sia stato scelto il modello di macchina ad accesso casuale per l'elaborazione seriale, nessun modello singolo per l'elaborazione parallela ha ottenuto un consenso così grande. Una delle principali ragioni è che i produttori non sono stati d'accordo su un singolo modello architetturale per i computer paralleli. Per esempio, alcuni computer paralleli dispongono della ***memoria condivisa***, alle cui locazioni ciascun processore può accedere direttamente. Altri computer paralleli utilizzano la ***memoria distribuita***, che è riservata a ciascun processore, e deve essere inviato un esplicito messaggio tra i processori affinché un processore possa accedere alla memoria di un altro processore. Con l'avvento della tecnologia multicore, invece, i nuovi computer desktop e portatili sono computer paralleli con memoria condivisa, e il trend sembra orientato verso i sistemi multiprocessore con memoria condivisa. Sebbene sarà il tempo a confermarlo, questo sarà l'approccio che adotteremo in questo capitolo.

Uno dei modi più comuni di programmare i chip multiprocessore e altri computer paralleli con memoria condivisa consiste nell'utilizzare il ***threading statico***, che rappresenta un'astrazione software per dei "processori virtuali" o ***thread***, che condividono una memoria comune. Ogni thread mantiene un contatore di programma e può eseguire il codice in modo indipendente dagli altri thread. Il sistema operativo carica un thread in un processore per eseguirlo e lo esclude quando deve eseguire un altro thread. Sebbene il sistema operativo consenta ai programmatori di creare e distruggere i thread, queste operazioni sono relativamente lente.

Acquistato da Michele Mastrangelo (mastrangelo.michele@gmail.com) - Copia non autorizzata - 2023/2024 - © McGraw-Hill Education (Italy)

Quindi, per la maggior parte delle applicazioni, i thread restano per tutta la durata di una elaborazione, e per questo sono detti "thread statici".

Purtroppo, programmare un computer parallelo con memoria condivisa utilizzando direttamente i thread statici è un compito difficile che può causare errori. Una ragione è che suddividere dinamicamente il lavoro fra i thread in modo che ciascun thread riceva approssimativamente lo stesso carico è un'impresa complicata. Tranne per applicazioni molto semplici, il programmatore deve utilizzare protocolli di comunicazione complessi per implementare uno scheduler (*pianificatore del lavoro*) per bilanciare i carichi di lavoro. Tutto questo ha portato alla creazione delle ***piattaforme concorrenti***, che forniscono uno strato di software che coordina, pianifica e gestisce le risorse dell'elaborazione parallela. Alcune piattaforme concorrenti sono realizzate come librerie runtime, mentre altre forniscono dei veri e propri linguaggi di programmazione paralleli con compilatori e supporti runtime.

La programmazione multithread dinamica

Una classe importante di piattaforme concorrenti utilizza il ***multithreading dinamico***, che è il modello che adotteremo in questo capitolo. Il multithreading dinamico consente ai programmatori di specificare il parallelismo nelle applicazioni senza preoccuparsi dei protocolli di comunicazione, del bilanciamento dei carichi e di altre bizzarrie della programmazione con il threading statico. Una piattaforma concorrente contiene uno scheduler, che bilancia automaticamente i carichi dell'elaborazione, semplificando così notevolmente il lavoro del programmatore. Sebbene le funzionalità degli ambienti per il multithreading dinamico siano ancora in evoluzione, quasi tutti supportano due funzionalità: parallelismo annidato e cicli paralleli. Il parallelismo annidato consente a una subroutine di essere "generata", consentendo alla procedura chiamante di procedere mentre la subroutine generata elabora il suo risultato. Un ciclo parallelo è simile a un normale ciclo **for**, con la differenza che le iterazioni del ciclo possono essere eseguite contemporaneamente.

Queste due funzionalità formano la base del modello del multithreading dinamico che analizzeremo in questo capitolo. Un aspetto chiave di questo modello è che il programmatore ha bisogno di specificare soltanto il parallelismo logico all'interno di una elaborazione, mentre i thread all'interno della piattaforma concorrente sottostante pianificano e bilanciano i carichi di elaborazione tra di loro. Esamineremo gli algoritmi multithread scritti per questo modello e anche il modo in cui la piattaforma concorrente può pianificare con efficienza le elaborazioni.

Il nostro modello di multithreading dinamico offre alcuni importanti vantaggi:

- È una semplice estensione del nostro modello di programmazione seriale. Possiamo descrivere un algoritmo multithread aggiungendo al nostro pseudocodice tre parole chiave della "concorrenza": **parallel**, **spawn** e **sync**. Inoltre, se eliminiamo queste parole chiave dallo pseudocodice multithread, il testo risultante è uno pseudocodice seriale per lo stesso problema che noi chiamiamo "serializzazione" dell'algoritmo multithread.
- Offre un modo teoricamente chiaro di quantificare il parallelismo in base a due concetti: "lavoro" (work) e "durata" (span).
- Molti algoritmi multithread che applicano il parallelismo annidato sono una derivazione naturale del paradigma divide et impera. Inoltre, gli algoritmi multithread, come gli algoritmi divide et impera, si prestano bene all'analisi mediante la soluzione di ricorrenze.

Acquistato da Michele Michele su Webster il 2022-07-07 15:12 Numero Prime Libreria: 1993301612 2p07 © Copyright © 2022, McGraw-Hill Education (Italy)

- Il modello rispecchia fedelmente il modo in cui si svolge praticamente l'elaborazione parallela. Un numero crescente di piattaforme concorrenti supporta una delle tante varianti del multithreading dinamico, comprese le piattaforme Cilk [51, 119], Cilk++ [72], OpenMP [60], Task Parallel Library [231] e Threading Building Blocks [293].

Il Paragrafo 27.1 presenta il modello del multithreading dinamico e le tre metriche *lavoro*, *durata* e *parallelismo*, che utilizzeremo per analizzare gli algoritmi multithread. Il Paragrafo 27.2 spiega come moltiplicare le matrici con il multithreading; il Paragrafo 27.3 affronta il problema complesso del merge sort tramite algoritmi multithread.

27.1 Fondamenti del multithreading dinamico

Iniziamo la nostra presentazione del multithreading dinamico utilizzando l'esempio del calcolo ricorsivo dei numeri di Fibonacci. Ricordiamo che i numeri di Fibonacci sono definiti dalla ricorsione (3.22):

$$\begin{aligned} F_0 &= 0 \\ F_1 &= 1 \\ F_i &= F_{i-1} + F_{i-2} \qquad \text{per } i \geq 2 \end{aligned}$$

Ecco un semplice algoritmo seriale ricorsivo che calcola l'n-esimo numero di Fibonacci:

```
FIB(n)
1  if n ≤ 1
2      return n
3  else x = FIB(n − 1)
4       y = FIB(n − 2)
5       return x + y
```

In pratica non conviene calcolare i grandi numeri di Fibonacci in questo modo, perché questo algoritmo esegue molti calcoli ripetitivi. La Figura 27.1 mostra l'albero delle istanze della procedura ricorsiva che sono create quando si calcola F_6. Per esempio, una chiamata di FIB(6) chiama ricorsivamente FIB(5) e poi FIB(4). Ma la chiamata di FIB(5) determina anche una chiamata di FIB(4). Entrambe le istanze di FIB(4) forniscono lo stesso risultato ($F_4 = 3$). Poiché la procedura FIB non applica la tecnica di annotazione (*memoization*), la seconda chiamata di FIB(4) replica il lavoro della prima chiamata.

Indichiamo con $T(n)$ il tempo di esecuzione di FIB(n). Poiché FIB(n) contiene due chiamate ricorsive più una quantità costante di lavoro extra, si ricava la seguente ricorsione

$$T(n) = T(n-1) + T(n-2) + \Theta(1)$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 100503016-220707-0 Copyright © 2022. McGraw-Hill Education (Italy)

Questa ricorsione ha la soluzione $T(n) = \Theta(F_n)$, che possiamo dimostrare utilizzando il metodo di sostituzione. Per ipotesi induttiva, supponiamo che $T(n) \leq aF_n - b$, dove $a > 1$ e $b > 0$ sono costanti. Facendo le opportune sostituzioni e se si sceglie un valore di b sufficientemente grande rispetto alla costante in $\Theta(1)$, si ottiene

$$\begin{aligned} T(n) &\leq (aF_{n-1} - b) + (aF_{n-2} - b) + \Theta(1) \\ &= a(F_{n-1} + F_{n-2}) - 2b + \Theta(1) \end{aligned}$$

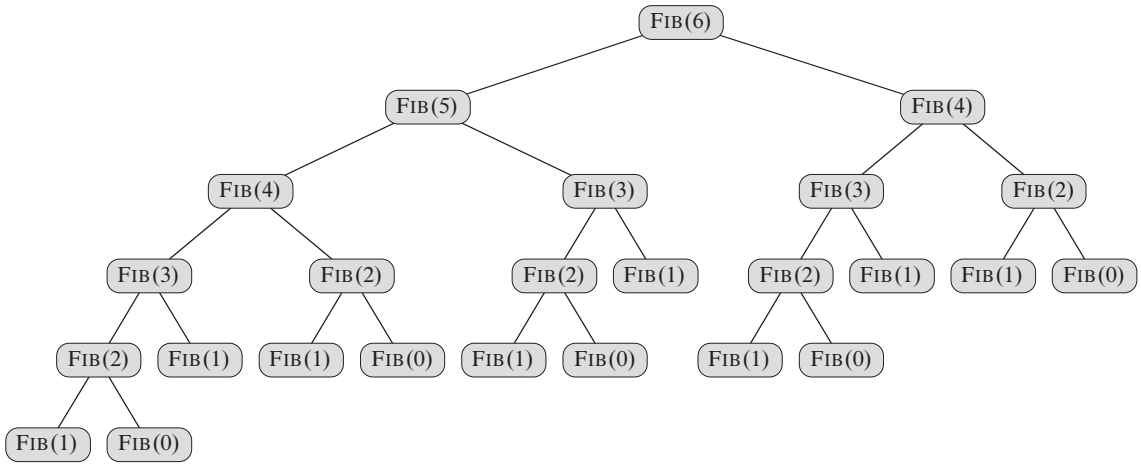


Figura 27.1 Albero delle istanze della procedura ricorsiva che vengono create per calcolare FIB(6). Ogni istanza di FIB con lo stesso argomento svolge lo stesso lavoro per ottenere lo stesso risultato, fornendo un modo poco efficiente, ma interessante, di calcolare i numeri di Fibonacci.

$$\begin{aligned} &= aF_n - b - (b - \Theta(1)) \\ &\leq aF_n - b \end{aligned}$$

Poi, possiamo scegliere un valore di a abbastanza grande da soddisfare la condizione iniziale. Infine, dall'equazione (3.25), si ottiene il limite analitico

$$T(n) = \Theta(\phi^n) \tag{27.1}$$

dove $\phi = (1 + \sqrt{5})/2$ è il rapporto aureo. Poiché F_n cresce esponenzialmente in n, questa procedura è un metodo particolarmente lento di calcolare i numeri di Fibonacci. (Vedere il Problema 31-3 per metodi molto più veloci.)

Sebbene la procedura FIB sia un metodo inefficiente di calcolare i numeri di Fibonacci, tuttavia essa fornisce un buon esempio per illustrare i concetti chiave dell'analisi degli algoritmi multithread. Notate che all'interno di FIB(n), le due chiamate ricorsive di FIB($n-1$) e FIB($n-2$) (rispettivamente, nelle righe 3 e 4) sono indipendenti l'una dall'altra: esse potrebbero essere chiamate in qualsiasi ordine, e il calcolo eseguito da una procedura non influisce sull'altra. Quindi, le due chiamate ricorsive possono essere eseguite in parallelo.

Ampliamo il nostro pseudocodice con il parallelismo aggiungendo le ***parole chiave della concorrenza*** **spawn** (*genera*) e **sync** (*sincronizza*). Ecco come può essere riscritta la procedura FIB per utilizzare il multithreading dinamico:

```
P-FIB(n)
1  if n ≤ 1
2       return n
3  else x = spawn P-FIB(n − 1)
4       y = P-FIB(n − 2)
5       sync
6       return x + y
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Notate che se cancelliamo le parole chiave della concorrenza **spawn** e **sync** dalla procedura P-FIB, il testo dello pseudocodice risultante è identico a FIB (a parte il nome della procedura nell'intestazione e nelle due chiamate ricorsive). Definiamo ***serializzazione*** di un algoritmo multithread l'algoritmo seriale che si ottiene cancellando le parole chiave del multithreading: **spawn**, **sync** e **parallel** (*esegui in*

parallelo) che useremo per i cicli paralleli. Il nostro pseudocodice multithread ha l'interessante proprietà che una serializzazione è sempre un normale pseudocodice seriale che risolve lo stesso problema.

Il ***parallelismo annidato*** si ha quando la parola chiave **spawn** precede una chiamata di procedura, come nella riga 3. La semantica di spawn differisce da una normale chiamata di procedura per il fatto che l'istanza della procedura che esegue lo spawn (la procedura *madre*) può continuare ad essere eseguita in parallelo con la subroutine generata (la procedura *figlia*), anziché aspettare che la subroutine figlia venga completata, come accade in una esecuzione seriale. In questo caso, mentre la subroutine generata sta calcolando P-FIB$(n-1)$, la procedura madre potrebbe passare al calcolo di P-FIB$(n-2)$ nella riga 4 in parallelo con la subroutine figlia. Poiché la procedura P-FIB è ricorsiva, queste due chiamate di subroutine creano, a loro volta, un parallelismo annidato, come fanno pure le loro figlie, creando così un albero potenzialmente enorme di sottocalcoli, tutti eseguiti in parallelo.

La parola chiave **spawn** non significa, tuttavia, che una procedura *deve* essere eseguita contemporaneamente con la sua subroutine figlia, ma soltanto che *può* esserlo. Le parole chiave della concorrenza esprimono il ***parallelismo logico*** dell'elaborazione, indicando quali parti del calcolo possono essere eseguite in parallelo. Durante l'esecuzione, è compito dello ***scheduler*** determinare quali sottocalcoli eseguire contemporaneamente, assegnandoli ai processori disponibili mentre si svolge l'elaborazione. Descriveremo più avanti la teoria che sta alla base degli scheduler.

Una procedura non può utilizzare in modo sicuro i valori restituiti dalle sue subroutine figlie finché non avrà eseguito un'istruzione **sync**, come nella riga 5. La parola chiave **sync** indica che la procedura deve aspettare, se necessario, che tutte le sue subroutine figlie siano completate prima di passare all'istruzione dopo **sync**. Nella procedura P-FIB, è necessaria un'istruzione **sync** prima dell'istruzione **return** nella riga 6 per evitare l'anomalia che si verificherebbe se x e y fossero sommate prima che x sia stata calcolata. Oltre alla sincronizzazione esplicita fornita dall'istruzione **sync**, ciascuna procedura esegue un'istruzione **sync** implicitamente prima di finire, garantendo così che tutte le sue subroutine figlie siano completate prima che ciò accada.

Un modello per l'esecuzione multithread

Può essere utile immaginare il ***calcolo multithread*** – l'insieme delle istruzioni eseguite da un processore per conto di un programma multithread – come un grafo orientato aciclico ovvero un *dag* (directed acyclic graph) $G = (V, E)$, detto ***dag di calcolo***. Nell'esempio della Figura 27.2 è riportato il dag di calcolo che si ottiene per calcolare P-FIB(4). In teoria, i vertici di V rappresentano le istruzioni e gli archi di E rappresentano le dipendenze tra le istruzioni, dove $(u, v) \in E$ significa che l'istruzione u deve essere eseguita prima dell'istruzione v. Per comodità, tuttavia, se una catena di istruzioni non contiene un controllo parallelo (nessuna istruzione **spawn**, **sync** o **return** da una subroutine figlia – tramite un'istruzione **return** esplicita o la **return** che viene implicitamente eseguita alla fine di una procedura), possiamo raggruppare le sue istruzioni in un unico ***strand*** (trefolo), che è quindi la concatenazione di una o più istruzioni. Le istruzioni che riguardano il controllo parallelo non sono incluse negli strand, ma sono rappresentate nella struttura del dag. Per esempio, se uno strand ha due successori, uno di essi deve

Acquistato da Michele Mastrolonardo (mastrolonardo@libero.it) - Copyright © 2022 - 2023 McGraw-Hill Education (Italy)

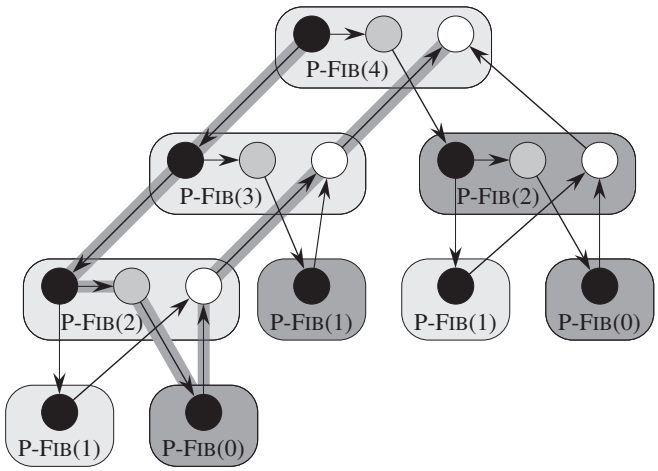


Figura 27.2 Un dag che rappresenta il calcolo di P-FIB(4). Ogni cerchio rappresenta uno strand: i cerchi neri rappresentano i casi base o la parte di una istanza della procedura fino alla generazione di P-FIB($n-1$) nella riga 3; i cerchi grigi rappresentano la parte della procedura che chiama P-FIB($n-2$) nella riga 4 fino all'istruzione **sync** nella riga 5, dove essa si ferma ad aspettare che termini la subroutine generata P-FIB($n-1$); i cerchi bianchi rappresentano la parte della procedura dopo **sync**, dal punto in cui somma x e y fino al punto in cui restituisce il risultato. I gruppi di strand che appartengono alla stessa procedura sono inclusi in un rettangolo a spigoli arrotondati, in grigio chiaro per le procedure generate e in grigio scuro per le procedure chiamate. Gli archi di generazione e gli archi di chiamata puntano verso il basso, gli archi di continuazione puntano orizzontalmente verso destra e gli archi di ritorno puntano verso l'alto. Supponendo che ciascuno strand richieda un'unità di tempo, la metrica *lavoro* vale 17 unità di tempo, perché ci sono 17 strand, e la metrica *durata* vale 8 unità di tempo, perché il cammino critico – indicato con archi su sfondo grigio scuro – contiene 8 strand.

essere stato generato da un'istruzione **spawn**, e uno strand con più predecessori indica che i predecessori si sono ricongiunti a causa di un'istruzione **sync**. Dunque, nel caso generale, l'insieme V forma l'insieme degli strand, mentre l'insieme E degli archi orientati rappresenta le dipendenze tra gli strand indotte dal controllo parallelo. Se G ha un cammino orientato dallo strand u allo strand v, diciamo che i due strand sono ***(logicamente) in serie***. Altrimenti, gli strand u e v sono ***(logicamente) in parallelo***.

È possibile rappresentare un calcolo multithread con un dag di strand inseriti in un albero di istanze di procedure. Per esempio, la Figura 27.1 mostra l'albero delle istanze di procedura per P-FIB(6) senza indicare la struttura dettagliata degli strand. La Figura 27.2 espande una sezione di tale albero per mostrare gli strand che formano ciascuna procedura. Tutti gli archi orientati che collegano gli strand si trovano all'interno di una procedura o lungo gli archi non orientati dell'albero delle procedure.

È possibile classificare gli archi di un dag di calcolo per indicare il tipo di dipendenza tra i vari strand. Un ***arco di continuazione*** (u, u'), tracciato orizzontalmente nella Figura 27.2, collega uno strand u con il suo successore u' all'interno della stessa istanza di procedura. Quando uno strand u genera uno strand v, il dag contiene un ***arco di generazione*** (u, v), che punta verso il basso nella figura. Anche gli ***archi di chiamata***, che rappresentano le normali chiamate di procedura, puntano verso il basso. Lo strand u che genera lo strand v si differenzia dallo strand u che chiama v in quanto un'istruzione spawn implica un arco di continuazione orizzontale da u allo strand u' dopo lo strand u nella sua procedura, indicando che u' è libero di essere eseguito contemporaneamente a v, mentre una chiamata non implica tale arco. Quando uno strand u ritorna alla sua procedura chiamante e x è lo strand immediatamente dopo la successiva istruzione **sync** nella procedura chiamante, il dag di calcolo contiene un ***arco di ritorno*** (u, x), che punta verso l'alto. Un calcolo inizia con uno ***strand iniziale*** – il vertice nero nella procedura P-FIB(4) della Figura 27.2 – e termina con uno ***strand finale*** – il vertice bianco nella procedura P-FIB(4).

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

Studieremo l'esecuzione di algoritmo multithread in un ***computer parallelo ideale***, che è formato da più processori e da una memoria condivisa ***sequenzialmente coerente***; "sequenzialmente coerente" significa che la memoria condivisa, che in realtà potrebbe eseguire contemporaneamente molte operazioni di lettu-

ra e scrittura richieste dai processori, genera gli stessi risultati come se, a ogni passo, venisse eseguita una singola istruzione da uno solo dei processori. In altri termini, la memoria si comporta come se le istruzioni venissero eseguite in sequenza, secondo un certo ordine lineare globale che preserva l'ordine in cui ciascun processore esegue le sue istruzioni. Per i calcoli multithread dinamici, che sono pianificati automaticamente nei processori dalla piattaforma concorrente, la memoria condivisa si comporta come se le istruzioni del calcolo multithread venissero intercalate per produrre un ordine lineare che preserva l'ordine parziale del dag di calcolo. A seconda della pianificazione effettuata dallo scheduler, l'ordine potrebbe essere diverso da un'esecuzione all'altra di un programma, ma il comportamento di qualsiasi esecuzione può essere capito supponendo che le istruzioni siano eseguite in qualche ordine lineare coerente con il dag di calcolo.

Oltre alle ipotesi sulla semantica, il modello del computer parallelo ideale richiede delle ipotesi sulle prestazioni. In particolare, si suppone che ciascun processore nella macchina abbia la stessa potenza di calcolo; inoltre viene trascurato il costo della pianificazione. Sebbene quest'ultima ipotesi possa sembrare ottimistica, tuttavia si è visto che in pratica, per gli algoritmi con sufficiente "parallelismo" (un termine che definiremo meglio in seguito), i costi aggiuntivi per la pianificazione sono generalmente minimi.

Misura delle prestazioni

È possibile misurare l'efficienza teorica di un algoritmo multithread tramite due metriche: "lavoro" e "durata". La metrica ***lavoro*** di un calcolo multithread è il tempo totale per eseguire un calcolo intero in un processore. In altre parole, il lavoro è la somma dei tempi impiegati da ciascuno degli strand che formano un calcolo completo. Per un dag di calcolo, in cui ciascuno strand richiede un'unità di tempo, il lavoro è semplicemente il numero dei vertici nel dag. La metrica ***durata*** è il tempo più lungo per eseguire gli strand lungo qualsiasi cammino nel dag. Anche in questo caso, per un dag in cui ogni strand richiede un'unità di tempo, la durata è pari al numero di vertici sul ***cammino critico*** nel dag. (Ricordiamo dal Paragrafo 24.3 che è possibile trovare un cammino critico in un dag $G = (V, E)$ nel tempo $\Theta(V+E)$.) Per esempio, il dag di calcolo della Figura 27.2 ha 17 vertici in tutto e 8 vertici nel suo cammino critico, quindi se ciascuno strand richiede un'unità di tempo, il suo lavoro vale 17 unità di tempo e la sua durata vale 8 unità di tempo.

Il tempo di esecuzione effettivo di un calcolo multithread dipende non soltanto dal suo lavoro e dalla sua durata, ma anche da quanti processori sono disponibili e dal modo in cui lo scheduler alloca gli strand ai processori. Per indicare il tempo di esecuzione di un calcolo multithread su P processori, metteremo l'indice P a pedice. Per esempio, indicheremo con T_P il tempo di esecuzione di un algoritmo su P processori. Il lavoro è il tempo di esecuzione su un singolo processore, ossia T_1. La durata è il tempo di esecuzione che otterremmo se ciascuno strand fosse eseguito in un suo proprio processore – in altre parole, se avessimo un numero illimitato di processori – e quindi indichiamo la durata con T_∞.

Acquistato da Michele Mastrolia (mastrolia@libero.it) il 2022-07-12 22:12:12 per uso personale - Vietata la distribuzione - ISBN 9788838693013 - © 2023 McGraw-Hill Education (Italy)

Il lavoro e la durata forniscono dei limiti inferiori sul tempo di esecuzione T_P di un calcolo multithread su P processori:

- In un passo, un computer parallelo ideale con P processori può eseguire al più P unità di lavoro; quindi, nel tempo T_P, può eseguire al più una quantità di lavoro pari a PT_P. Poiché il lavoro totale da svolgere è T_1, abbiamo $PT_P \geq T_1$.

Dividendo per P si ottiene la ***legge del lavoro***:

$$T_P \geq T_1/P \tag{27.2}$$

- Un computer parallelo ideale con P processori non può essere più veloce di una macchina con un numero illimitato di processori. In altri termini, una macchina con un numero illimitato di processori può emulare una macchina con P processori utilizzando soltanto P dei suoi processori. Quindi, si ha la seguente ***legge della durata***:

$$T_P \geq T_\infty \tag{27.3}$$

Definiamo ***speedup*** (accelerazione) di un calcolo su P processori il rapporto T_1/T_P, che indica quante volte più veloce è il calcolo su P processori rispetto a un solo processore. Per la legge del lavoro, abbiamo $T_P \geq T_1/P$, che implica che $T_1/T_P \leq P$. Quindi, lo speedup su P processori può essere al più P. Quando lo speedup è lineare nel numero dei processori, ovvero quando $T_1/T_P = \Theta(P)$, il calcolo presenta uno ***speedup lineare***, e quando $T_1/T_P = P$ si ha uno ***speedup lineare perfetto***.

Il rapporto T_1/T_∞ tra lavoro e durata indica il ***parallelismo*** del calcolo multithread. Possiamo vedere il parallelismo da tre prospettive. Come rapporto, il parallelismo indica la quantità media di lavoro che può essere eseguito in parallelo in ogni passo lungo il cammino critico. Come limite superiore, il parallelismo indica lo speedup massimo che può essere ottenuto con un numero qualsiasi di processori. Infine, e questa è la cosa forse più importante, il parallelismo rappresenta un limite alla possibilità di ottenere uno speedup lineare perfetto. Più precisamente, quando il numero di processori supera il parallelismo, il calcolo non può ottenere uno speedup lineare perfetto. Per capire quest'ultimo punto, supponete che $P > T_1/T_\infty$, nel qual caso la legge della durata implica che lo speedup soddisfi la relazione $T_1/T_P \leq T_1/T_\infty < P$. Inoltre, se il numero P di processori nel computer parallelo ideale supera di molto il parallelismo – ovvero se $P \gg T_1/T_\infty$ – allora $T_1/T_P \ll P$, cosicché lo speedup è molto più piccolo del numero di processori. In altre parole, quanto più processori vengono utilizzati oltre il parallelismo, tanto meno perfetto sarà lo speedup.

Come esempio consideriamo il calcolo P-FIB(4) nella Figura 27.2; supponiamo che ciascuno strand richieda un'unità di tempo. Poiché $T_1 = 17$ e $T_\infty = 8$, il parallelismo è $T_1/T_\infty = 17/8 = 2.125$. Di conseguenza, è impossibile raggiungere uno speedup molto più che doppio, indipendentemente dal numero di processori utilizzati per eseguire il calcolo. Per dimensioni di input più grandi, tuttavia, vedremo che P-FIB(n) presenta un sostanziale parallelismo.

Definiamo come ***lasco*** (slackness) di parallelismo di un calcolo multithread eseguito su un computer parallelo ideale con P processori il rapporto $(T_1/T_\infty)/P = T_1/(PT_\infty)$, che è il fattore per il quale il parallelismo del calcolo supera il numero di processori nella machina. Quindi, se abbiamo un lasco minore di 1, non possiamo sperare di ottenere uno speedup lineare perfetto, perché $T_1/(PT_\infty) < 1$ e la legge della durata implica che lo speedup su P processori soddisfi la relazione $T_1/T_P \leq T_1/T_\infty < P$. Quando il lasco diminuisce da 1 a 0, lo speedup del calcolo si allontana sempre più da quello lineare perfetto. Se il lasco è maggiore di 1, invece, il vincolo che limita lo speedup è la quantità di lavoro per processore. Come vedremo, da 1 in avanti quanto più aumenta il lasco tanto più un buon scheduler può avvicinarsi allo speedup lineare perfetto.

Acquistato da Michele Michele su Webster il 2022-0[illegible] Vietata la riproduzione non autorizzata (Art. 171 L.A.)

Pianificazione

Le buone prestazioni non si ottengono semplicemente minimizzando i valori di lavoro e durata; occorre anche pianificare efficientemente gli strand da eseguire nei processori della macchina parallela. Il nostro modello di programmazione multithread non consente di specificare quali strand debbano essere eseguiti in ciascun processore. Esso, invece, si affida allo scheduler della piattaforma concorrente per mappare dinamicamente il calcolo nei singoli processori. In pratica, lo scheduler mappa gli strand nei thread statici e il sistema operativo pianifica i thread sui processori stessi, ma questo livello extra di indirezione non è necessario per la nostra comprensione della pianificazione. Possiamo semplicemente immaginare che lo scheduler della piattaforma concorrente mappi gli strand direttamente nei processori.

Lo scheduler multithread deve pianificare il calcolo senza sapere in anticipo quando saranno generati gli strand o quando saranno completati – esso deve operare ***on-line***. Inoltre, un buon scheduler opera in modo distribuito, con i tread che costituiscono lo scheduler che cooperano per bilanciare il carico del calcolo. Esistono ottimi scheduler on-line distribuiti, ma la loro analisi è complessa.

Per semplificare la nostra analisi, invece, esamineremo uno scheduler on-line ***centralizzato***, che, in ogni determinato istante, conosce lo stato globale del calcolo. In particolare, analizzeremo gli ***scheduler golosi***, che assegnano in ogni passo il maggior numero possibile di strand ai processori. Se almeno P strand sono pronti per essere eseguiti durante un passo, diciamo che il passo è un ***passo completo***, e uno scheduler goloso assegna ai processori P qualsiasi degli strand pronti. Altrimenti, sono pronti meno di P strand da eseguire, nel qual caso diciamo che il passo è un ***passo incompleto***, e lo scheduler assegna ciascuno degli strand pronti ad un processore.

Per la legge del lavoro, il miglior tempo di esecuzione che possiamo sperare con P processori è $T_P = T_1/P$ e, per la legge della durata, il miglior tempo di esecuzione che possiamo sperare è $T_P = T_\infty$. Il seguente teorema dimostra che la programmazione golosa è indubbiamente buona in quanto ha come limite superiore la somma di questi due limiti inferiori.

Teorema 27.1
In un computer parallelo ideale con P processori, uno scheduler goloso esegue un calcolo multithread con lavoro T_1 e durata T_∞ nel tempo

$$T_P \leq T_1/P + T_\infty \tag{27.4}$$

Dimostrazione Iniziamo considerando i passi completi. In ogni passo completo, i P processori insieme eseguono una quantità totale di lavoro pari a P. Supponete per assurdo che il numero di passi completi sia strettamente più grande di $\lfloor T_1/P \rfloor$. Allora, il lavoro totale dei passi completi è almeno

$$\begin{aligned} P \cdot (\lfloor T_1/P \rfloor + 1) &= P \lfloor T_1/P \rfloor + P \\ &= T_1 - (T_1 \bmod P) + P && \text{(per l'equazione (3.8))} \\ &> T_1 && \text{(per la disuguaglianza (3.9))} \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 22:12 Numero Ordine Libreria: 1995023016 220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Quindi, arriviamo alla contraddizione che P processori eseguono più lavoro di quello richiesto dal calcolo, e questo ci consente di concludere che il numero di passi completi è al più $\lfloor T_1/P \rfloor$.

Adesso, consideriamo un passo incompleto. Sia G il dag che rappresenta tutto il calcolo e, senza perdere di generalità, supponiamo che ciascuno strand richieda

un'unità di tempo. (Possiamo sostituire ogni strand più lungo con una catena di strand con tempo unitario.) Sia G' il sottografo di G che deve essere eseguito all'inizio del passo incompleto, e sia G'' il sottografo che resta da eseguire dopo il passo incompleto. Un cammino più lungo in un dag deve iniziare necessariamente da un vertice con grado entrante 0. Poiché un passo incompleto di uno scheduler goloso esegue tutti gli strand con grado entrante 0 in G', un cammino più lungo in G'' deve avere una lunghezza minore di un'unità di quella di un cammino più lungo in G'. In altre parole, un passo incompleto riduce di 1 la durata del dag ancora da eseguire. Dunque, il numero dei passi incompleti è al più T_∞.

Poiché ogni passo può essere completo o incompleto, il teorema è dimostrato. ■

Il seguente corollario al Teorema 27.1 dimostra che uno scheduler goloso ha sempre buone prestazioni.

Corollario 27.2
Il tempo di esecuzione T_P di un calcolo multithread pianificato da uno scheduler goloso in un computer parallelo ideale con P processori differisce per al più un fattore 2 da quello ottimale.

Dimostrazione Sia T_P^* il tempo di esecuzione prodotto da uno scheduler ottimale in una macchina con P processori, e siano T_1 e T_∞, rispettivamente, il lavoro e la durata del calcolo. Poiché le leggi del lavoro e della durata – disuguaglianze (27.2) e (27.3) – ci dicono che $T_P^* \geq \max(T_1/P, T_\infty)$, il Teorema 27.1 implica che

$$\begin{aligned} T_P &\leq T_1/P + T_\infty \\ &\leq 2 \cdot \max(T_1/P, T_\infty) \\ &\leq 2T_P^* \end{aligned}$$

■

Il prossimo corollario indica che, in effetti, uno scheduler goloso ottiene uno speedup lineare quasi perfetto con qualsiasi calcolo multithread al crescere del lasco.

Corollario 27.3
Sia T_P il tempo di esecuzione di un calcolo multithread prodotto da uno scheduler goloso in un computer parallelo ideale con P processori, e siano T_1 e T_∞, rispettivamente, il lavoro e la durata del calcolo. Allora, se $P \ll T_1/T_\infty$, abbiamo $T_P \approx T_1/P$, ovvero uno speedup pari approssimativamente a P.

Dimostrazione Se supponiamo che $P \ll T_1/T_\infty$, allora è anche $T_\infty \ll T_1/P$, e quindi il Teorema 27.1 ci indica che $T_P \leq T_1/P + T_\infty \approx T_1/P$. Poiché, per la legge del lavoro (27.2), $T_P \geq T_1/P$, concludiamo che $T_P \approx T_1/P$, ovvero lo speedup è $T_1/T_P \approx P$. ■

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Il simbolo $\ll$ indica la relazione "molto minore", ma di quanto è "molto minore"? Come regola generale, un lasco di almeno 10 – ovvero un parallelismo 10 volte più dei processori – normalmente è sufficiente per ottenere un buon valore di speedup. Quindi, il termine durata nel limite goloso (disuguaglianza (27.4)) è minore del 10% del termine lavoro-per-processore, che è un rapporto sufficientemente buono per molte applicazioni ingegneristiche. Per esempio, se un calcolo viene eseguito su 10 o 100 processori soltanto, non ha senso confrontare un parallelismo di 1 000 000 con un parallelismo di 10 000, anche se differiscono per

Lavoro: $T_1(A \cup B) = T_1(A) + T_1(B)$
Durata: $T_\infty(A \cup B) = T_\infty(A) + T_\infty(B)$

(a)

Lavoro: $T_1(A \cup B) = T_1(A) + T_1(B)$
Durata: $T_\infty(A \cup B) = \max(T_\infty(A), T_\infty(B))$

(b)

Figura 27.3 Lavoro e durata della composizione di due sottocalcoli. **(a)** Quando due sottocalcoli sono uniti in serie, il lavoro della composizione è la somma dei loro lavori, e la durata della composizione è la somma delle loro durate. **(b)** Quando due sottocalcoli sono uniti in parallelo, il lavoro della composizione resta uguale alla somma dei loro lavori, ma la durata della composizione è soltanto il massimo delle loro durate.

un fattore 100. Come dimostra il Problema 27-2, a volte riducendo un enorme parallelismo, possiamo ottenere algoritmi migliori sotto altri aspetti e che pure rispondono molto bene alla variazione di un numero ragionevole di processori.

Analisi degli algoritmi multithread

Adesso abbiamo tutti gli strumenti che servono per analizzare gli algoritmi multithread e trovare dei buoni limiti sui loro tempi di esecuzione con più processori. Analizzare il lavoro è relativamente semplice, in quanto basta analizzare il tempo di esecuzione di un normale algoritmo seriale – ovvero la serializzazione dell'algoritmo multithread – che ormai dovreste essere in grado di fare, dal momento che la maggior parte di questo libro è dedicata a questo argomento. L'analisi della durata è più interessante, ma generalmente non più difficile, una volta che avrete imparato a farla. Descriveremo i concetti base utilizzando il programma P-FIB.

L'analisi del lavoro $T_1(n)$ di P-FIB(n) non presenta ostacoli, in quanto l'abbiamo già fatta. La procedura FIB originale è essenzialmente la serializzazione di P-FIB, e quindi $T_1(n) = T(n) = \Theta(\phi^n)$ dall'equazione (27.1).

La Figura 27.3 illustra come analizzare la durata. Se i due sottocalcoli sono uniti in serie, le loro durate si sommano per formare la durata della loro composizione, mentre se vengono uniti in parallelo, la durata della loro composizione è il massimo delle durate dei due sottocalcoli. Per P-FIB(n), la chiamata di P-FIB$(n-1)$ nella riga 3 viene eseguita in parallelo con la chiamata di P-FIB$(n-2)$ nella riga 4. Quindi, possiamo esprimere la durata di P-FIB(n) con la seguente ricorsione

$$\begin{aligned} T_\infty(n) &= \max(T_\infty(n-1), T_\infty(n-2)) + \Theta(1) \\ &= T_\infty(n-1) + \Theta(1) \end{aligned}$$

che ha la soluzione $T_\infty(n) = \Theta(n)$.

Il parallelismo di P-FIB(n) è $T_1(n)/T_\infty(n) = \Theta(\phi^n/n)$, che cresce significativamente all'aumentare di n. Quindi, perfino nei computer paralleli più grandi, un modesto valore di n è sufficiente per ottenere uno speedup lineare quasi perfetto per P-FIB(n), in quanto questa procedura presenta un considerevole lasco di parallelismo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Cicli paralleli

Molti algoritmi contengono cicli le cui iterazioni possono operare in parallelo. Come vedremo, è possibile parallelizzare tali cicli utilizzando le parole chiave **spawn** e **sync**, ma è molto più conveniente specificare direttamente che le iterazioni di tali cicli possono essere eseguite contemporaneamente. Il nostro pseudocodice offre questa funzionalità utilizzando la parola chiave della concorrenza **parallel**, che precede la parola chiave **for** nell'istruzione di un ciclo **for**.

Come esempio, consideriamo il problema di moltiplicare una matrice $A = (a_{ij})$, di dimensioni $n \times n$, per un vettore $x = (x_j)$, di dimensione n. Il vettore risultante $y = (y_i)$, di dimensione n, è dato dall'equazione

$$y_i = \sum_{j=1}^{n} a_{ij} x_j$$

per $i = 1, 2, \ldots, n$. Possiamo eseguire la moltiplicazione matrice-vettore calcolando tutti gli elementi di y in parallelo, nel modo seguente:

```
MAT-VEC(A, x)
1  n = A.rows
2  sia y un nuovo vettore di dimensione n
3  parallel for i = 1 to n
4       y_i = 0
5  parallel for i = 1 to n
6       for j = 1 to n
7            y_i = y_i + a_ij x_j
8  return y
```

In questo codice, le parole chiave **parallel for** nelle righe 3 e 5 indicano che le iterazioni dei rispettivi cicli possono essere eseguite contemporaneamente. Un compilatore può implementare ciascun ciclo **parallel for** come una subroutine divide et impera utilizzando il parallelismo annidato. Per esempio, il ciclo **parallel for** nelle righe 5–7 può essere implementato con la chiamata MAT-VEC-MAIN-LOOP$(A, x, y, n, 1, n)$, dove il compilatore produce la subroutine ausiliaria MAT-VEC-MAIN-LOOP, nel modo seguente:

```
MAT-VEC-MAIN-LOOP(A, x, y, n, i, i')
1  if i == i'
2       for j = 1 to n
3            y_i = y_i + a_ij x_j
4  else mid = ⌊(i + i')/2⌋
5       spawn MAT-VEC-MAIN-LOOP(A, x, y, n, i, mid)
6       MAT-VEC-MAIN-LOOP(A, x, y, n, mid + 1, i')
7       sync
```

Questo codice genera in modo ricorsivo la prima metà delle iterazioni del ciclo da eseguire in parallelo con la seconda metà delle iterazioni; poi esegue un'istruzione **sync**, creando così un albero binario di esecuzione, dove le foglie sono singole iterazioni del ciclo, come mostra la Figura 27.4.

Per calcolare il lavoro $T_1(n)$ di MAT-VEC su una matrice $n \times n$, calcoliamo semplicemente il tempo di esecuzione della sua serializzazione, che si ottiene sostituendo i cicli **parallel for** con normali cicli **for**. Quindi, abbiamo $T_1(n) = \Theta(n^2)$, in quanto prevale il tempo di esecuzione quadratico dei cicli doppiamente annidati nelle righe 5–7. Questa analisi, tuttavia, sembra ignorare il costo aggiuntivo dovuto alla generazione ricorsiva nell'implementazione dei cicli paralleli. In effetti, il costo aggiuntivo della generazione ricorsiva aumenta il lavoro di un ciclo parallelo rispetto a quello della sua serializzazione, ma non in modo asintotico. Per capire perché, notate che, dal momento che l'albero delle istanze delle procedure ricorsive è un albero binario pieno, il numero di nodi interni è pari al numero di foglie meno 1 (vedere l'Esercizio B.5-3). Ogni nodo interno svolge un lavoro costante

Acquistato da Michele Michele su Webster il 2022-07-07 23:19 Nessuna parte di questa pubblicazione può essere riprodotta © 2022 - MIT Press (Massachusetts Institute of Technology)

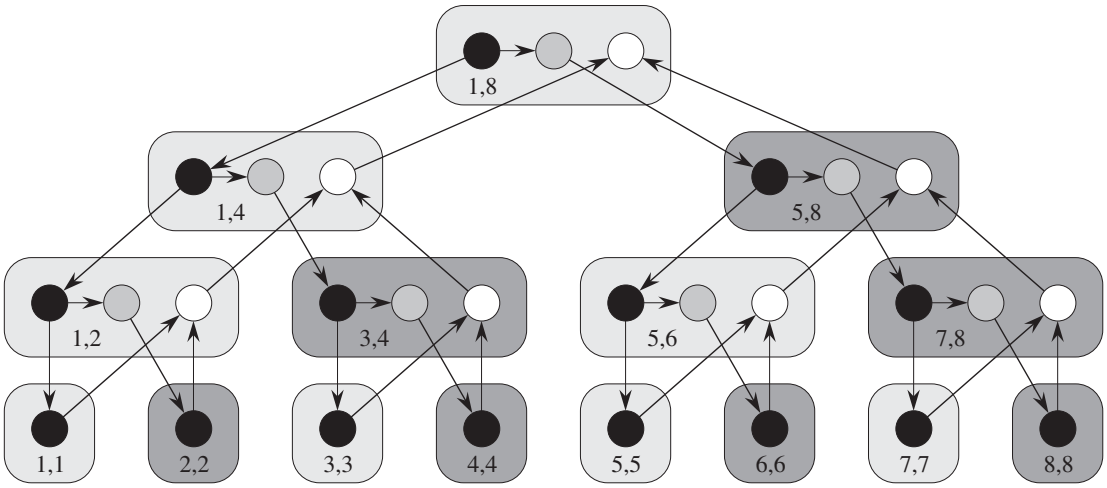


Figura 27.4 Un dag che rappresenta il calcolo di MAT-VEC-MAIN-LOOP$(A, x, y, 8, 1, 8)$. I due numeri all'interno di ciascun rettangolo a spigoli arrotondati sono i valori degli ultimi due parametri (i e i' nell'intestazione della procedura) nella chiamata della procedura. I cerchi neri rappresentano gli strand che corrispondono al caso base o alla parte della procedura fino all'istruzione **spawn** di MAT-VEC-MAIN-LOOP nella riga 5; i cerchi grigi rappresentano gli strand che corrispondono alla parte della procedura che chiama MAT-VEC-MAIN-LOOP nella riga 6 fino all'istruzione **sync** nella riga 7, dove viene sospesa finché non termina la subroutine generata nella riga 5; i cerchi bianchi rappresentano gli strand che corrispondono alla parte (trascurabile) della procedura dopo **sync** e fino al punto in cui termina.

per dividere l'intervallo di iterazione, e ogni foglia corrisponde a un'iterazione del ciclo, che richiede almeno un tempo costante ($\Theta(n)$ in questo caso). Quindi, possiamo ammortizzare il costo aggiuntivo della generazione ricorsiva con il lavoro delle iterazioni, contribuendo al più per un fattore costante al lavoro totale.

Nella pratica, le piattaforme concorrenti del multithreading dinamico a volte ***ingrossano*** le foglie della ricorsione eseguendo più iterazioni in una sola foglia, sia automaticamente sia sotto il controllo del programmatore, riducendo così il costo aggiuntivo della generazione ricorsiva. Questo modo di ridurre il costo aggiuntivo, però, riduce anche il parallelismo, ma se il calcolo ha un sufficiente lasco di parallelismo, questo non sacrifica uno speedup lineare quasi perfetto.

Dobbiamo tenere conto anche del costo aggiuntivo della generazione ricorsiva quando analizziamo la durata del costrutto di ciclo parallelo. Poiché la profondità della chiamata ricorsiva è logaritmica nel numero delle iterazioni, la durata di un ciclo parallelo con n iterazioni, in cui la i-esima iterazione ha una durata pari a $iter_\infty(i)$, è data da

$$T_\infty(n) = \Theta(\lg n) + \max_{1 \le i \le n} iter_\infty(i)$$

Per esempio, per MAT-VEC su una matrice $n \times n$, la durata del ciclo parallelo di inizializzazione (righe 3–4) è $\Theta(\lg n)$, perché la generazione ricorsiva prevale sul lavoro con tempo costante di ciascuna iterazione. La durata del ciclo doppiamente annidato nelle righe 5–7 è $\Theta(n)$, in quanto ciascuna iterazione del ciclo **parallel for** esterno contiene n iterazioni del ciclo **for** (seriale) interno. La durata del codice restante nella procedura è costante e, quindi, i cicli doppiamente annidati prevalgono sulla durata, determinando una durata totale pari a $\Theta(n)$ per l'intera procedura. Poiché il lavoro vale $\Theta(n^2)$, il parallelismo è $\Theta(n^2)/\Theta(n) = \Theta(n)$. (L'Esercizio 27.1-6 chiede di fornire un'implementazione con un parallelismo ancora maggiore.)

Acquistato da Michele Michele su Webster il 2022-07-07 23:14 Numero Ordine Libraria 1095030016 220707 | Copyright © 2022, McGraw-Hill Education (Italy)

Situazioni di conflitto

Un algoritmo multithread è ***deterministico*** se fa sempre le stesse cose con lo stesso input, indipendentemente da come vengono pianificate le sue istruzioni sul computer multicore. È ***non deterministico*** se il suo comportamento può variare da un'esecuzione all'altra. Spesso, un algoritmo multithreaded che è stato progettato come deterministico non si comporta come tale, perché contiene un "conflitto di determinatezza".

Le situazioni di conflitto sono la rovina della concorrenza. Tra i più famosi bachi da conflitto ricordiamo la macchina Therac-25 per terapia radioattiva, che uccise tre persone e ne ferì molte altre, e il blackout del 2003 nell'America del nord, che lasciò senza energia elettrica oltre 50 milioni di persone. Questi pericolosi bachi sono noti per essere difficili da scoprire. Potreste eseguire dei test in laboratorio per vari giorni senza riscontrare malfunzionamenti e poi scoprire che il vostro software sporadicamente fallisce sul campo.

Un ***conflitto di determinatezza*** si verifica quando due istruzioni logicamente in parallelo accedono alla stessa locazione di memoria e almeno una di esse esegue un'operazione di scrittura. La seguente procedura illustra una situazione di conflitto:

```
RACE-EXAMPLE( )
1  x = 0
2  parallel for i = 1 to 2
3       x = x + 1
4  stampa x
```

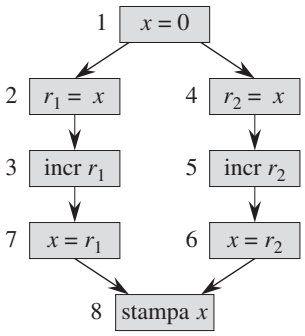
Dopo l'inizializzazione di x a 0 nella riga 1, RACE-EXAMPLE crea due strand paralleli, ciascuno dei quali aumenta x nella riga 3. Sebbene possa sembrare che questa procedura debba sempre stampare il valore 2 (come farebbe sicuramente la sua serializzazione), essa invece potrebbe stampare il valore 1. Vediamo come potrebbe verificarsi questa anomalia.

Quando un processore aumenta x, l'operazione non è indivisibile, ma è composta da una sequenza di istruzioni:

1. Leggi il valore di x nella memoria e scrivilo in uno dei registri del processore.
2. Aumenta il valore nel registro.
3. Scrivi il valore del registro nella variabile x in memoria.

La Figura 27.5(a) illustra un dag di calcolo che rappresenta l'esecuzione di RACE-EXAMPLE, con gli strand suddivisi nelle singole istruzioni. Ricordiamo che, poiché un computer parallelo ideale supporta la coerenza sequenziale, possiamo considerare l'esecuzione parallela di un algoritmo multithread come una intercalazione delle istruzioni che rispetta le dipendenze nel dag. La parte (b) della figura mostra i valori di un'esecuzione del calcolo che presenta l'anomalia. Il valore di x è registrato nella memoria; r_1 ed r_2 sono i registri dei processori. Nel passo 1, uno dei processori imposta x a 0. Nei passi 2 e 3, il processore 1 legge x nella memoria, lo scrive nel suo registro r_1 e lo incrementa, generando il valore 1 in r_1. A questo punto, entra in scena il processore 2 eseguendo le istruzioni 4–6. Il processore 2 legge x nella memoria, lo scrive nel registro r_2 e lo incrementa, generando il valore 1 in r_2; poi, memorizza questo valore in x, impostando x a 1. A questo punto, il processore 1 riprende l'esecuzione dal passo 7, memorizzando il valore 1 di r_1 in x, il che lascia inalterato il valore di x. Dunque, il passo 8 stampa il valore 1, anziché 2, come avrebbe fatto la serializzazione.

Acquistato da Michele Michele su Webster il 2022-07-07 20:12 Numero Ordine Libreria: 1955830 6-No 2707 0 Copyright 2022 McGraw-Hill Education (Italy)



(a)

passo	x	r_1	r_2
1	0	–	–
2	0	0	–
3	0	1	–
4	0	1	0
5	0	1	1
6	1	1	1
7	1	1	1

(b)

Figura 27.5
Illustrazione del conflitto di determinatezza nella procedura RACE-EXAMPLE. **(a)** Un dag di calcolo che mostra le dipendenze tra le singole istruzioni. I registri dei processori sono r_1 ed r_2. Sono omesse le istruzioni che non sono correlate al conflitto, come l'implementazione del ciclo di controllo. **(b)** Una sequenza di esecuzioni che mette in evidenza il baco, mostrando i valori di x in memoria e il contenuto dei registri r_1 ed r_2 in ogni passo della sequenza.

Vediamo che cosa è accaduto. Se, per effetto dell'esecuzione parallela, il processore 1 eseguisse tutte le sue istruzioni prima del processore 2, sarebbe stampato il valore 2. D'altra parte, se l'effetto dell'esecuzione parallela fosse che il processore 2 eseguisse tutte le sue istruzioni prima del processore 1, sarebbe stampato ancora il valore 2. Se, invece, le istruzioni dei due processori vengono eseguite contemporaneamente, è possibile, come in questo esempio, che uno degli aggiornamenti di x venga perso.

Ovviamente, ci sono molte esecuzioni che non evidenziano il baco. Per esempio, se l'esecuzione fosse $\langle 1, 2, 3, 7, 4, 5, 6, 8\rangle$ o $\langle 1, 4, 5, 6, 2, 3, 7, 8\rangle$, si avrebbe il risultato corretto. È questo il problema che si ha con i conflitti di decisioni. In generale, molti ordinamenti producono risultati corretti – come quelli in cui le istruzioni a sinistra vengono eseguite prima di quelle a destra, o viceversa. Ma alcuni ordinamenti generano risultati errati quando le istruzioni sono intercalate. Di conseguenza, le situazioni di conflitto possono essere estremamente difficili da verificare. Potreste eseguire dei test per giorni senza scoprire il baco, e poi assistere a un catastrofico crash del sistema sul campo quando il risultato è critico.

Sebbene sia possibile gestire questi conflitti in vari modi, incluso l'impiego di blocchi di mutua esclusione e di altri metodi di sincronizzazione, per i nostri scopi, garantiremo semplicemente che gli strand che operano in parallelo siano ***indipendenti***: non hanno conflitti di decisioni fra di loro. Quindi, in un costrutto **parallel for**, tutte le iterazioni saranno indipendenti. Tra un'istruzione **spawn** e la corrispondente **sync**, il codice della subroutine generata sarà indipendente dal codice del padre, incluso il codice eseguito da altre subroutine generate o da figli chiamati. Si noti gli argomenti passati a una subroutine generata vengono valutati nel codice del padre prima che la subroutine venga effettivamente generata; quindi, la valutazione degli argomenti di una subroutine generata è in serie con qualsiasi accesso a tali argomenti dopo la generazione della subroutine.

Come esempio di come sia facile generare un codice con conflitti, ecco un'implementazione errata della moltiplicazione matrice-vettore che ottiene una durata di $\Theta(\lg n)$ parallelizzando il ciclo **for** interno:

```
MAT-VEC-WRONG(A, x)
1  n = A.rows
2  sia y un nuovo vettore di dimensione n
3  parallel for i = 1 to n
4      y_i = 0
5  parallel for i = 1 to n
6      parallel for j = 1 to n
7          y_i = y_i + a_ij x_j
8  return y
```

Acquistato da Michele Michela su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Questa procedura, purtroppo, non è corretta a causa dei conflitti per aggiornare y_i nella riga 7, che viene eseguita contemporaneamente per tutti i valori n di j. L'Esercizio 27.1-6 chiede di scrivere un'implementazione corretta con una durata pari a $\Theta(\lg n)$.

Un algoritmo multithread con conflitti in alcuni casi può essere corretto. Per esempio, due thread paralleli potrebbero memorizzare lo stesso valore in una variabile condivisa; non sarebbe importante quale dei due abbia registrato per primo il valore. In generale, però, considereremo illegale un codice che contiene dei conflitti.

Una lezione di scacchi

Chiudiamo questo paragrafo con un fatto vero che è avvenuto durante lo sviluppo di ⋆Socrates [81], un programma multithread per il gioco degli scacchi. Per semplicità di esposizione, abbiamo semplificato i tempi. Il prototipo del programma fu realizzato in un computer con 32 processori, ma alla fine fu eseguito in un supercomputer con 512 processori. A un certo punto, gli sviluppatori aggiunsero una procedura di ottimizzazione nel programma per ridurre il suo tempo di esecuzione per un importante benchmark sulla macchina a 32 processori da $T_{32} = 65$ secondi a $T'_{32} = 40$ secondi. Tuttavia, gli sviluppatori, dopo avere valutato le prestazioni tramite le metriche lavoro e durata, conclusero che la versione ottimizzata, che era più veloce con 32 processori, in effetti sarebbe stata più lenta con 512 processori. Di conseguenza, abbandonarono la procedura di "ottimizzazione".

Questa fu la loro analisi. La versione originale del programma aveva un lavoro pari a $T_1 = 2048$ secondi e una durata pari a $T_\infty = 1$ secondo. Se trattiamo la disuguaglianza (27.4) come un'equazione, $T_P = T_1/P + T_\infty$, e la utilizziamo come un'approssimazione del tempo di esecuzione su P processori, vediamo che effettivamente $T_{32} = 2048/32 + 1 = 65$. Con l'ottimizzazione, il lavoro diventa $T'_1 = 1024$ secondi e la durata diventa $T'_\infty = 8$ secondi. Utilizzando ancora la nostra approssimazione, otteniamo $T'_{32} = 1024/32 + 8 = 40$.

Le velocità relative delle due versioni cambiano quando calcoliamo i tempi di esecuzione su 512 processori. In particolare, abbiamo $T_{512} = 2048/512 + 1 = 5$ secondi e $T'_{512} = 1024/512 + 8 = 10$ secondi. L'ottimizzazione che accelera il programma su 32 processori avrebbe reso il programma due volte più lento su 512 processori! La durata di 8 della versione ottimizzata, che non era il termine predominante nel tempo di esecuzione su 32 processori, divenne il termine predominante su 512 processori, vanificando il vantaggio di utilizzare più processori. La morale di questa storia è che il lavoro e la durata possono essere strumenti migliori per estrapolare le prestazioni della semplice misura dei tempi di esecuzione.

Esercizi

27.1-1

Supponte di sostituire la semplice chiamata di P-FIB$(n-2)$ nella riga 4 della procedura P-FIB con l'istruzione **spawn** P-FIB$(n-2)$. Qual è l'impatto asintotico sul lavoro, sulla durata e sul parallelismo?

Acquistato da Michele Michele su Webster il 2022-0[illegible] Numero Unico Licenza: [illegible] Copyright © 2022 McGraw-Hill Education (Italy)

27.1-2

Disegnate il dag di calcolo che si ottiene eseguendo P-FIB(5). Supponendo che ogni strand nel calcolo richieda un'unità di tempo, quali sono i valori del lavoro, della durata e del parallelismo del calcolo? Descrivete come pianificare il dag su 3 processori utilizzando lo scheduler goloso ed etichettando ogni strand con il passo temporale in cui viene eseguito.

27.1-3
Dimostrate che uno scheduler goloso ottiene il seguente limite sul tempo, che è leggermente più forte di quello dimostrato nel Teorema 27.1:

$$T_P \leq \frac{T_1 - T_\infty}{P} + T_\infty \tag{27.5}$$

27.1-4
Costruite un dag di calcolo per il quale una esecuzione di uno scheduler goloso può richiedere quasi il doppio del tempo di un'altra esecuzione di uno scheduler goloso con lo stesso numero di processori. Descrivete come dovrebbero procedere le due esecuzioni.

27.1-5
Il professor Karan misura il suo algoritmo multithread deterministico su 4, 10 e 64 processori di un computer parallelo ideale utilizzando uno scheduler goloso. Egli sostiene che le tre esecuzioni hanno fornito $T_4 = 80$ secondi, $T_{10} = 42$ secondi e $T_{64} = 10$ secondi. Dimostrate che il professore è un bugiardo o un incompetente (*suggerimento:* utilizzate la legge del lavoro (27.2), la legge della durata (27.3) e la disuguaglianza (27.5) dell'Esercizio 27.1-3).

27.1-6
Create un algoritmo multithread per moltiplicare una matrice $n \times n$ per un vettore di dimensione n che realizza un parallelismo $\Theta(n^2/\lg n)$, mantenendo un lavoro pari a $\Theta(n^2)$.

27.1-7
Considerate il seguente pseudocodice multithread che traspone una matrice A di dimensioni $n \times n$:

```
P-TRANSPOSE(A)
1  n = A.rows
2  parallel for j = 2 to n
3      parallel for i = 1 to j − 1
4          scambia a_ij con a_ji
```

Analizzate il lavoro, la durata e il parallelismo di questo algoritmo.

27.1-8
Supponete di sostituire il ciclo **parallel for** nella riga 3 di P-TRANSPOSE (vedere l'Esercizio 27.1-7) con un normale ciclo **for**. Analizzate il lavoro, la durata e il parallelismo dell'algoritmo risultante.

27.1-9
Per quanti processori le due versioni del programma degli scacchi hanno la stessa velocità di esecuzione, supponendo che $T_P = T_1/P + T_\infty$?

Acquistato da Michele Michele su Webster il 2022-07-07 23.12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

27.2 Moltiplicazione multithread delle matrici

In questo paragrafo vedremo come eseguire la moltiplicazione multithread delle matrici, un problema il cui tempo di esecuzione seriale è stato analizzato nel Paragrafo 4.2. Esamineremo gli algoritmi multithread che si basano sui normali tre cicli annidati, come pure gli algoritmi divide et impera.

Algoritmi multithread per moltiplicare le matrici

Il primo algoritmo che studieremo è un semplice algoritmo che si basa sulla parallelizzazione dei cicli nella procedura SQUARE-MATRIX-MULTIPLY a pagina 64:

```
P-SQUARE-MATRIX-MULTIPLY(A, B)
1  n = A.rows
2  sia C una nuova matrice n × n
3  parallel for i = 1 to n
4      parallel for j = 1 to n
5          c_ij = 0
6          for k = 1 to n
7              c_ij = c_ij + a_ik · b_kj
8  return C
```

Per analizzare questo algoritmo, notate che, dal momento che la serializzazione dell'algoritmo è semplicemente SQUARE-MATRIX-MULTIPLY, il lavoro è $T_1(n) = \Theta(n^3)$, che è pari al tempo di esecuzione di SQUARE-MATRIX-MULTIPLY. La durata è $T_\infty(n) = \Theta(n)$, perché segue un cammino verso il basso nell'albero di ricorsione per il ciclo **parallel for** a partire dalla riga 3, poi un cammino verso il basso lungo l'albero di ricorsione per il ciclo **parallel for** nella riga 4; infine esegue tutte le n iterazioni del ciclo **for** ordinario a partire dalla riga 6, con una durata totale pari a $\Theta(\lg n) + \Theta(\lg n) + \Theta(n) = \Theta(n)$. Quindi, il parallelismo è $\Theta(n^3)/\Theta(n) = \Theta(n^2)$. L'Esercizio 27.2-4 vi chiede di parallelizzare il ciclo interno per ottenere un parallelismo pari a $\Theta(n^3/\lg n)$, che non potete ottenere utilizzando semplicemente il ciclo **parallel for**, perché creereste dei conflitti.

Un algoritmo multithread divide et impera per moltiplicare le matrici

Come detto nel Paragrafo 4.2, è possibile moltiplicare in modo seriale due matrici $n \times n$ nel tempo $\Theta(n^{\lg 7}) = O(n^{2.81})$ utilizzando il metodo divide et impera di Strassen; questo ci spinge ad applicare il multithreading a tale algoritmo. Come abbiamo fatto nel Paragrafo 4.2, iniziamo ad applicare il multithreading a un algoritmo divide et impera più semplice.

Ricordiamo dalla pagina 65 che la procedura SQUARE-MATRIX-MULTIPLY-RECURSIVE, che moltiplica due matrici A e B, di dimensioni $n \times n$, per produrre la matrice C, $n \times n$, si basa sulla suddivisione di ciascuna delle tre matrici in quattro sottomatrici $n/2 \times n/2$:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

Quindi, il prodotto delle matrici può essere scritto così

$$\begin{aligned} \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} &= \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \\ &= \begin{pmatrix} A_{11}B_{11} & A_{11}B_{12} \\ A_{21}B_{11} & A_{21}B_{12} \end{pmatrix} + \begin{pmatrix} A_{12}B_{21} & A_{12}B_{22} \\ A_{22}B_{21} & A_{22}B_{22} \end{pmatrix} \end{aligned} \tag{27.6}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria 1999030151227/12-0 Copyright © 2021 McGraw-Hill Education (Italy)

Dunque, per moltiplicare due matrici $n \times n$, eseguiamo otto moltiplicazioni di matrici $n/2 \times n/2$ e una somma di matrici $n \times n$. Il seguente pseudocodice implementa questo metodo divide et impera utilizzando il parallelismo annidato. Diversamente dalla procedura SQUARE-MATRIX-MULTIPLY-RECURSIVE sulla quale si basa, la procedura P-MATRIX-MULTIPLY-RECURSIVE riceve la matrice di output come parametro per evitare di allocare inutilmente delle matrici.

```
P-MATRIX-MULTIPLY-RECURSIVE(C, A, B)
 1  n = A.rows
 2  if n == 1
 3      c_11 = a_11 b_11
 4  else sia T una nuova matrice n × n
 5      suddividi A, B, C e T nelle sottomatrici n/2 × n/2
            A_11, A_12, A_21, A_22; B_11, B_12, B_21, B_22; C_11, C_12, C_21, C_22;
            e T_11, T_12, T_21, T_22; rispettivamente
 6      spawn P-MATRIX-MULTIPLY-RECURSIVE(C_11, A_11, B_11)
 7      spawn P-MATRIX-MULTIPLY-RECURSIVE(C_12, A_11, B_12)
 8      spawn P-MATRIX-MULTIPLY-RECURSIVE(C_21, A_21, B_11)
 9      spawn P-MATRIX-MULTIPLY-RECURSIVE(C_22, A_21, B_12)
10      spawn P-MATRIX-MULTIPLY-RECURSIVE(T_11, A_12, B_21)
11      spawn P-MATRIX-MULTIPLY-RECURSIVE(T_12, A_12, B_22)
12      spawn P-MATRIX-MULTIPLY-RECURSIVE(T_21, A_22, B_21)
13      P-MATRIX-MULTIPLY-RECURSIVE(T_22, A_22, B_22)
14      sync
15      parallel for i = 1 to n
16          parallel for j = 1 to n
17              c_ij = c_ij + t_ij
```

La riga 3 gestisce il caso base, dove vengono moltiplicate le matrici 1×1. Le righe 4–17 gestiscono il caso ricorsivo. La riga 4 alloca una matrice temporanea T; la riga 5 suddivide ciascuna delle matrici A, B, C e T in sottomatrici $n/2 \times n/2$. (Come nella procedura SQUARE-MATRIX-MULTIPLY-RECURSIVE a pagina 65, sorvoliamo sul dettaglio secondario di come utilizzare il calcolo degli indici per rappresentare le sezioni delle sottomatrici di una matrice.) La chiamata ricorsiva nella riga 6 assegna alla sottomatrice C_{11} il prodotto delle sottomatrici $A_{11}B_{11}$, cosicché C_{11} è uguale al primo dei due termini di cui esso è somma nell'equazione (27.6). In modo simile, le righe 7–9 assegnano a C_{12}, C_{21} e C_{22} il primo dei due termini di cui essi sono somme nell'equazione (27.6). La riga 10 assegna alla sottomatrice T_{11} il prodotto delle sottomatrici $A_{12}B_{21}$, in modo che T_{11} sia uguale al secondo dei due termini di cui è somma C_{11}. Le righe 11–13 assegnano a T_{12}, T_{21} e T_{22} il secondo dei due termini di cui, rispettivamente, sono somma C_{12}, C_{21} e C_{22}. Le prime sette chiamate ricorsive sono generate, e l'ultima viene eseguita nello strand principale. L'istruzione **sync** nella riga 14 garantisce che tutti i prodotti delle sottomatrici nelle righe 6–13 siano stati calcolati, dopodiché vengono sommati i prodotti in T ai prodotti in C, utilizzando i cicli **parallel for** doppiamente annidati nelle righe 15–17.

Analizziamo prima il lavoro $M_1(n)$ della procedura P-MATRIX-MULTIPLY-RECURSIVE, ripetendo l'analisi del tempo di esecuzione seriale della sua progenitrice SQUARE-MATRIX-MULTIPLY-RECURSIVE. Nel caso ricorsivo, suddividiamo le matrici nel tempo $\Theta(1)$, eseguiamo otto moltiplicazioni ricorsive di matrici $n/2 \times n/2$, e finiamo con il lavoro $\Theta(n^2)$ per sommare due matrici $n \times n$. Quindi, la ricorsione per il lavoro $M_1(n)$ è

Acquistato da Michele Mattioni - Vietata la riproduzione non autorizzata - Copyright © 2022 McGraw-Hill Education (Italy)

$$\begin{aligned} M_1(n) &= 8M_1(n/2) + \Theta(n^2) \\ &= \Theta(n^3) \end{aligned}$$

per il caso 1 del teorema dell'esperto. In altre parole, il lavoro del nostro algoritmo multithread è asintoticamente uguale al tempo di esecuzione della procedura SQUARE-MATRIX-MULTIPLY del Paragrafo 4.2, con i suoi tre cicli annidati.

Per determinare la durata $M_\infty(n)$ di P-MATRIX-MULTIPLY-RECURSIVE, notiamo innanzi tutto che la durata per suddividere le matrici è $\Theta(1)$, che è dominato dalla durata $\Theta(\lg n)$ dei cicli **parallel for** doppiamente annidati nelle righe15–17. Poiché le otto chiamate ricorsive parallele vengono eseguite su matrici della stessa dimensione, la durata massima di una chiamata ricorsiva è quella di una chiamata qualsiasi. Dunque, la ricorsione per la durata $M_\infty(n)$ di P-MATRIX-MULTIPLY-RECURSIVE è

$$M_\infty(n) = M_\infty(n/2) + \Theta(\lg n) \tag{27.7}$$

Questa ricorsione non rientra in nessuno dei casi del teorema dell'esperto, ma soddisfa la condizione dell'Esercizio 4.6-2. Per l'Esercizio 4.6-2, quindi, la soluzione della ricorsione (27.7) è $M_\infty(n) = \Theta(\lg^2 n)$.

Conoscendo il lavoro e la durata di P-MATRIX-MULTIPLY-RECURSIVE, possiamo calcolare il suo parallelismo come $M_1(n)/M_\infty(n) = \Theta(n^3/\lg^2 n)$, che è molto alto.

Metodo multithreading di Strassen

Per applicare il multithreading all'algoritmo di Strassen, seguiamo lo stesso procedimento generale descritto a pagina 67, ma utilizzando il parallelismo annidato:

1. Suddividiamo le matrici di input A e B e la matrice di output C in sottomatrici $n/2 \times n/2$, come nell'equazione (27.6). Questo passo richiede un lavoro e una durata $\Theta(1)$ per il calcolo degli indici.
2. Creiamo dieci matrici $S_1, S_2, \ldots, S_{10}$, ciascuna delle quali ha dimensioni $n/2 \times n/2$ ed è la somma o la differenza delle due matrici create nel passo 1. Possiamo creare tutte e dieci le matrici con un lavoro $\Theta(n^2)$ e una durata $\Theta(\lg n)$, utilizzando i cicli **parallel for** doppiamente annidati.
3. Utilizzando le sottomatrici create nel passo 1 e le dieci matrici create nel passo 2, utilizziamo un'istruzione **spawn** per calcolare in modo ricorsivo le sette matrici prodotto $P_1, P_2, \ldots, P_7$, di dimensioni $n/2 \times n/2$.
4. Calcoliamo le sottomatrici $C_{11}, C_{12}, C_{21}, C_{22}$ della matrice risultante C sommando e sottraendo varie combinazioni delle matrici P_i, ancora una volta utilizzando i cicli **parallel for** doppiamente annidati. Possiamo calcolare tutte e quattro le sottomatrici con un lavoro $\Theta(n^2)$ e una durata $\Theta(\lg n)$.

Per analizzare questo algoritmo, osserviamo innanzi tutto che, dal momento che la serializzazione è uguale all'algoritmo seriale originale, il lavoro è semplicemente il tempo di esecuzione della serializzazione, ovvero $\Theta(n^{\lg 7})$. Come per la procedura P-MATRIX-MULTIPLY-RECURSIVE, possiamo trovare una ricorsione per la durata. In questo caso, sette chiamate ricorsive vengono eseguite in parallelo, ma poiché operano tutte su matrici della stessa dimensione, otteniamo la stessa ricorsione (27.7), come per la procedura P-MATRIX-MULTIPLY-RECURSIVE, che ha la soluzione $\Theta(\lg^2 n)$. Quindi, il parallelismo del metodo multithread di Strassen è $\Theta(n^{\lg 7}/\lg^2 n)$, che è alto, sebbene leggermente minore del parallelismo di P-MATRIX-MULTIPLY-RECURSIVE.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Esercizi

27.2-1

Disegnate il dag di calcolo per calcolare P-SQUARE-MATRIX-MULTIPLY su matrici 2×2, indicando come i vertici del vostro diagramma corrispondono agli

strand nell'esecuzione dell'algoritmo. Utilizzate la convenzione che gli archi generati e quelli delle chiamate puntano verso il basso, gli archi di continuazione puntano orizzontalmente verso destra e gli archi di ritorno puntano verso l'alto.

27.2-2
Ripetete l'Esercizio 27.2-1 per la procedura P-MATRIX-MULTIPLY-RECURSIVE.

27.2-3
Scrivete lo pseudocodice per un algoritmo multithread che moltiplica una matrice A di dimensioni $n \times n$ per un vettore x di dimensione n con lavoro $\Theta(n^2)$ ma con durata soltanto $\Theta(\lg n)$. Analizzate il vostro algoritmo.

27.2-4
Scrivete lo pseudocodice per un algoritmo multithread che moltiplica due matrici $n \times n$ con un lavoro di $\Theta(n^3)$, ma una durata di appena $\Theta(\lg n)$. Analizzate il vostro algoritmo.

27.2-5
Scrivete lo pseudocodice per un algoritmo multithread efficiente che moltiplica una matrice $p \times q$ per una matrice $q \times r$. Il vostro algoritmo dovrebbe avere un parallelismo alto anche quando qualcuna delle dimensioni p, q ed r si riduce a 1. Analizzate il vostro algoritmo.

27.2-6
Scrivete lo pseudocodice per un algoritmo multithread efficiente che traspone sul posto una matrice $n \times n$ utilizzando il metodo divide et impera. Analizzate il vostro algoritmo.

27.2-7
Scrivete lo pseudocodice per un'implementazione multithread efficiente dell'algoritmo di Floyd-Warshall (vedere il Paragrafo 25.3), che calcola i cammini minimi tra tutte le coppie di vertici in un grafo pesato. Analizzate il vostro algoritmo.

27.3 Algoritmi multithread per merge sort

Abbiamo esaminato l'algoritmo merge sort seriale nel Paragrafo 2.3.1; nel Paragrafo 2.3.2 abbiamo analizzato il suo tempo di esecuzione, che vale $\Theta(n \lg n)$. Poiché l'algoritmo merge sort utilizza già il paradigma divide et impera, esso è un candidato ideale al multithreading. Possiamo facilmente modificare lo pseudocodice in modo che la prima chiamata ricorsiva sia effettuata da un'istruzione spawn:

```
MERGE-SORT'(A, p, r)
1  if p < r
2      q = ⌊(p + r)/2⌋
3      spawn MERGE-SORT'(A, p, q)
4      MERGE-SORT'(A, q + 1, r)
5      sync
6      MERGE(A, p, q, r)
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Come la sua controparte seriale, MERGE-SORT′ ordina il sottoarray $A[p \,..\, r]$. Dopo che le due subroutine ricorsive nelle righe 3 e 4 sono state completate – cosa che è garantita dall'istruzione **sync** nella riga 5 – MERGE-SORT′ chiama la stessa procedura MERGE di pagina 27.

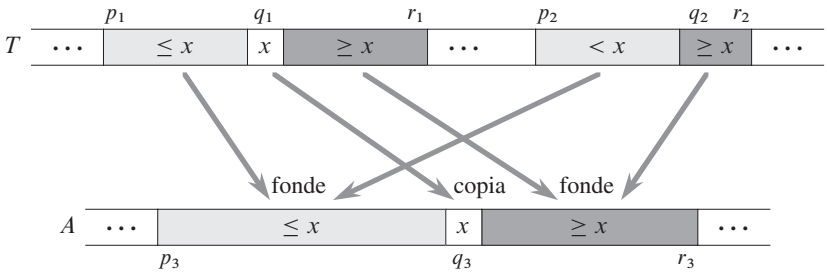


Figura 27.6 Rappresentazione dell'idea che sta alla base della fusione multithread di due sottoarray ordinati $T[p_1 \,..\, r_1]$ e $T[p_2 \,..\, r_2]$ nel sottoarray $A[p_3 \,..\, r_3]$. Se $x = T[q_1]$ è la mediana di $T[p_1 \,..\, r_1]$ e q_2 è il posto in $T[p_2 \,..\, r_2]$ tale che x ricada tra $T[q_2 - 1]$ e $T[q_2]$, ogni elemento nei sottoarray $T[p_1 \,..\, q_1 - 1]$ e $T[p_2 \,..\, q_2 - 1]$ (grigio chiaro) è minore o uguale a x, e ogni elemento nei sottoarray $T[q_1 + 1 \,..\, r_1]$ e $T[q_2 + 1 \,..\, r_2]$ (grigio scuro) è almeno x. Per la fusione, calcoliamo l'indice q_3 dove x deve stare in $A[p_3 \,..\, r_3]$, copiamo x in $A[q_3]$ e poi fondiamo ricorsivamente $T[p_1 \,..\, q_1 - 1]$ con $T[p_2 \,..\, q_2 - 1]$ in $A[p_3 \,..\, q_3 - 1]$ e $T[q_1 + 1 \,..\, r_1]$ con $T[q_2 \,..\, r_2]$ in $A[q_3 + 1 \,..\, r_3]$.

Analizziamo MERGE-SORT′. Per farlo, dobbiamo prima analizzare MERGE. Ricordiamo che il suo tempo di esecuzione seriale per ordinare n elementi è $\Theta(n)$. Poiché MERGE è seriale, il suo lavoro e la sua durata sono $\Theta(n)$. Quindi, la seguente ricorsione caratterizza il lavoro $MS'_1(n)$ di MERGE-SORT′ su n elementi

$$\begin{aligned} MS'_1(n) &= 2\,MS'_1(n/2) + \Theta(n) \\ &= \Theta(n \lg n) \end{aligned}$$

che è uguale al tempo di esecuzione seriale di merge sort. Poiché le due chiamate ricorsive di MERGE-SORT′ possono essere eseguite in parallelo, la durata MS'_∞ è data dalla ricorsione

$$\begin{aligned} MS'_\infty(n) &= MS'_\infty(n/2) + \Theta(n) \\ &= \Theta(n) \end{aligned}$$

Dunque, il parallelismo di MERGE-SORT′ è pari a $MS'_1(n)/MS'_\infty(n) = \Theta(\lg n)$, che non è molto elevato. Per esempio, per ordinare 10 milioni di elementi, potrebbe raggiungere uno speedup lineare con pochi processori, ma non si comporterebbe altrettanto bene con centinaia di processori.

Probabilmente avrete già individuato dove si trova il collo di bottiglia del parallelismo in questo algoritmo multithread di merge sort: la procedura seriale MERGE. Sebbene a prima vista la fusione possa sembrare intrinsecamente seriale, in effetti è possibile creare una versione multithreaded per essa applicando il parallelismo annidato.

La nostra strategia divide et impera per la fusione multithreaded, che è illustrata nella Figura 27.6, opera su sottoarray dell'array T. Supponiamo di dover fondere due sottoarray ordinati $T[p_1 \,..\, r_1]$ di lunghezza $n_1 = r_1 - p_1 + 1$ e $T[p_2 \,..\, r_2]$ di lunghezza $n_2 = r_2 - p_2 + 1$ in un altro sottoarray $A[p_3 \,..\, r_3]$ di lunghezza $n_3 = r_3 - p_3 + 1 = n_1 + n_2$. Senza perdere in generalità, facciamo l'ipotesi semplificativa che $n_1 \geq n_2$.

Troviamo prima l'elemento di mezzo $x = T[q_1]$ del sottoarray $T[p_1 \,..\, r_1]$, dove $q_1 = \lfloor (p_1 + r_1)/2 \rfloor$. Poiché il sottoarray è ordinato, x è una mediana di $T[p_1 \,..\, r_1]$: ogni elemento in $T[p_1 \,..\, q_1 - 1]$ non è maggiore di x, e ogni elemento in $T[q_1 + 1 \,..\, r_1]$ non è minore di x. Poi utilizziamo la ricerca binaria per trovare l'indice q_2 nel sottoarray $T[p_2 \,..\, r_2]$ tale che il sottoarray rimarrebbe ordinato inserendo x tra $T[q_2 - 1]$ e $T[q_2]$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria 1995034216 220707 Copyright © 2022, McGraw-Hill Education (Italy)

Poi fondiamo i sottoarray originali $T[p_1 \,..\, r_1]$ e $T[p_2 \,..\, r_2]$ in $A[p_3 \,..\, r_3]$ nel modo seguente:

1. Poniamo $q_3 = p_3 + (q_1 - p_1) + (q_2 - p_2)$.
2. Copiamo x in $A[q_3]$.

3. Fondiamo ricorsivamente $T[p_1 \,..\, q_1 - 1]$ con $T[p_2 \,..\, q_2 - 1]$ e inseriamo il risultato nel sottoarray $A[p_3 \,..\, q_3 - 1]$.
4. Fondiamo ricorsivamente $T[q_1 + 1 \,..\, r_1]$ con $T[q_2 \,..\, r_2]$ e inseriamo il risultato nel sottoarray $A[q_3 + 1 \,..\, r_3]$.

Quando calcoliamo q_3, la quantità $q_1 - p_1$ è il numero di elementi nel sottoarray $T[p_1 \,..\, q_1 - 1]$, e la quantità $q_2 - p_2$ è il numero di elementi nel sottoarray $T[p_2 \,..\, q_2 - 1]$. Quindi, la loro somma è il numero di elementi che precedono x nel sottoarray $A[p_3 \,..\, r_3]$.

Il caso base si ha quando $n_1 = n_2 = 0$, e qui non c'è lavoro da fare per fondere due sottoarray vuoti. Poiché abbiamo supposto che il sottoarray $T[p_1 \,..\, r_1]$ abbia almeno la stessa lunghezza di $T[p_2 \,..\, r_2]$, ovvero $n_1 \geq n_2$, possiamo verificare il caso base semplicemente verificando che $n_1 = 0$. Dobbiamo anche assicurarci che la ricorsione gestisca appropriatamente il caso in cui uno solo dei due sottoarray sia vuoto; tale sottoarray, per la nostra ipotesi che $n_1 \geq n_2$, deve essere $T[p_2 \,..\, r_2]$.

Adesso traduciamo tutte queste idee nello pseudocodice. Iniziamo con la ricerca binaria, che esprimiamo in modo seriale. La procedura BINARY-SEARCH(x, T, p, r) riceve una chiave x e un sottoarray $T[p \,..\, r]$, e restituisce uno dei seguenti indici:

- Se $T[p \,..\, r]$ è vuoto ($r < p$), restituisce l'indice p.
- Se $x \leq T[p]$ e quindi minore o uguale di tutti gli elementi di $T[p \,..\, r]$, restituisce l'indice p.
- Se $x > T[p]$, restituisce il massimo indice q nell'intervallo $p < q \leq r + 1$ tale che $T[q - 1] < x$.

Questo è lo pseudocodice:

```
BINARY-SEARCH(x, T, p, r)
1  low = p
2  high = max(p, r + 1)
3  while low < high
4       mid = ⌊(low + high)/2⌋
5       if x ≤ T[mid]
6            high = mid
7       else low = mid + 1
8  return high
```

La chiamata BINARY-SEARCH(x, T, p, r) richiede un tempo seriale $\Theta(\lg n)$ nel caso peggiore, dove $n = r - p + 1$ è la dimensione del sottoarray che viene elaborato. (Vedere l'Esercizio 2.3-5.) Poiché BINARY-SEARCH è una procedura seriale, il lavoro e la durata sono entrambi $\Theta(\lg n)$.

A questo punto possiamo scrivere lo pseudocodice per la procedura di fusione multithread. Come la procedura MERGE a pagina 27, anche P-MERGE suppone che i due sottoarray da fondere si trovino all'interno dello stesso array. Diversamente da MERGE, però, P-MERGE non suppone che i due sottoarray da fondere siano adiacenti all'interno dell'array (ovvero P-MERGE non richiede che $p_2 = r_1 + 1$). Un'altra differenza tra MERGE e P-MERGE è che P-MERGE riceve come argomento un sottoarray di output A nel quale devono essere memorizzati i valori fusi. La chiamata P-MERGE$(T, p_1, r_1, p_2, r_2, A, p_3)$ fonde i sottoarray ordinati $T[p_1 \,..\, r_1]$ e $T[p_2 \,..\, r_2]$ nel sottoarray $A[p_3 \,..\, r_3]$, dove $r_3 =$

Acquistato da Michele Mule (mule.michele@gmail.com) il 21/07/2022 per uso interno e non commerciale - Copyright © 2022 McGraw-Hill Education (Italy)

$p_3 + (r_1 - p_1 + 1) + (r_2 - p_2 + 1) - 1 \quad = \quad p_3 + (r_1 - p_1) + (r_2 - p_2) + 1,$
valore che non viene fornito in input.

```
P-MERGE(T, p1, r1, p2, r2, A, p3)
 1  n1 = r1 − p1 + 1
 2  n2 = r2 − p2 + 1
 3  if n1 < n2                    // assicura che n1 ≥ n2
 4       scambia p1 con p2
 5       scambia r1 con r2
 6       scambia n1 con n2
 7  if n1 == 0                    // entrambi vuoti?
 8       return
 9  else q1 = ⌊(p1 + r1)/2⌋
10       q2 = BINARY-SEARCH(T[q1], T, p2, r2)
11       q3 = p3 + (q1 − p1) + (q2 − p2)
12       A[q3] = T[q1]
13       spawn P-MERGE(T, p1, q1 − 1, p2, q2 − 1, A, p3)
14       P-MERGE(T, q1 + 1, r1, q2, r2, A, q3 + 1)
15       sync
```

La procedura P-MERGE opera nel seguente modo. Le righe 1–2 calcolano le lunghezze n_1 e n_2, rispettivamente, dei sottoarray $T[p_1 \,..\, r_1]$ e $T[p_2 \,..\, r_2]$. Le righe 3–6 garantiscono l'ipotesi che $n_1 \geq n_2$. La riga 7 verifica il caso base in cui il sottoarray $T[p_1 \,..\, r_1]$ è vuoto (e quindi anche $T[p_2 \,..\, r_2]$ è vuoto); in questo caso la procedura termina. Le righe 9–15 implementano il metodo divide et impera. La riga 9 calcola il punto di mezzo di $T[p_1 \,..\, r_1]$; la riga 10 trova il punto q_2 in $T[p_2 \,..\, r_2]$ tale che tutti elementi in $T[p_2 \,..\, q_2 - 1]$ siano minori di $T[q_1]$ (che corrisponde a x) e tutti gli elementi in $T[q_2 \,..\, p_2]$ siano almeno uguali a $T[q_1]$. La riga 11 calcola l'indice q_3 dell'elemento che divide il sottoarray di output $A[p_3 \,..\, r_3]$ in $A[p_3 \,..\, q_3 - 1]$ e $A[q_3 + 1 \,..\, r_3]$; poi la riga 12 copia $T[q_1]$ direttamente in $A[q_3]$.

Poi, applichiamo la ricorsione utilizzando il parallelismo annidato. La riga 13 genera il primo sottoproblema, mentre la riga 14 chiama il secondo sottoproblema in parallelo. L'istruzione **sync** nella riga 15 garantisce che i sottoproblemi siano stati completati prima che la procedura termini. (Poiché ogni procedura implicitamente esegue un'istruzione **sync** prima di terminare, avremmo potuto omettere l'istruzione **sync** nella riga 15, tuttavia includerla è una buona pratica di codificazione.) Vi è una certa ingegnosità nel codice per garantire che la procedura funzioni correttamente anche quando il sottoarray $T[p_2 \,..\, r_2]$ è vuoto. Essa consiste nel fatto che, ad ogni chiamata ricorsiva, un elemento mediano di $T[p_1 \,..\, r_1]$ viene copiato nell'array di output finché l'array $T[p_1 \,..\, r_1]$ non diventi vuoto, che è la condizione per il caso base.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Analisi della fusione multithread

Deriviamo una ricorsione per la durata $PM_\infty(n)$ di P-MERGE, dove i due sottoarray contengono un totale di $n = n_1 + n_2$ elementi. Poiché la generazione nella riga 13 e la chiamata nella riga 14 operano logicamente in parallelo, è necessario esaminare soltanto la più costosa delle due chiamate. Per questo, occorre notare che nel caso peggiore, il numero massimo di elementi nelle chiamate ricorsive può

essere al più $3n/4$, che possiamo spiegare nel seguente modo. Poiché le righe 3–6 garantiscono che $n_2 \le n_1$, si ha che $n_2 = 2n_2/2 \le (n_1 + n_2)/2 = n/2$. Nel caso peggiore, una delle due chiamate ricorsive fonde $\lfloor n_1/2 \rfloor$ elementi di $T[p_1 \,..\, r_1]$ con tutti gli n_2 elementi di $T[p_2 \,..\, r_2]$, e quindi il numero di elementi interessati nella chiamata è

$$\begin{aligned} \lfloor n_1/2 \rfloor + n_2 &\le n_1/2 + n_2/2 + n_2/2 \\ &= (n_1 + n_2)/2 + n_2/2 \\ &\le n/2 + n/4 \\ &= 3n/4 \end{aligned}$$

Aggiungendo il costo $\Theta(\lg n)$ della chiamata di BINARY-SEARCH nella riga 10, otteniamo la seguente ricorsione per la durata nel caso peggiore:

$$PM_\infty(n) = PM_\infty(3n/4) + \Theta(\lg n) \tag{27.8}$$

(Per il caso base, la durata è $\Theta(1)$, in quando le righe 1–8 vengono eseguite in un tempo costante.) Questa ricorsione non rientra in nessuno dei casi del teorema dell'esperto, ma soddisfa la condizione dell'Esercizio 4.6-2. Quindi, la soluzione della ricorsione (27.8) è $PM_\infty(n) = \Theta(\lg^2 n)$.

Analizziamo adesso il lavoro $PM_1(n)$ di P-MERGE su n elementi, che vedremo essere pari a $\Theta(n)$. Poiché ciascuno degli n elementi deve essere copiato dall'array T all'array A, abbiamo $PM_1(n) = \Omega(n)$. Quindi, resta da dimostrare soltanto che $PM_1(n) = O(n)$.

Deriveremo innanzi tutto una ricorsione per il lavoro nel caso peggiore. La ricerca binaria nella riga 10 costa $\Theta(\lg n)$ nel caso peggiore e prevale sull'altro lavoro fuori dalle chiamate ricorsive. Notiamo che, sebbene le chiamate ricorsive nelle righe 13 e 14 possano fondere numeri diversi di elementi, le due chiamate ricorsive insieme fondono al più n elementi (in effetti $n - 1$ elementi, in quanto $T[q_1]$ non partecipa alle chiamate ricorsive). Inoltre, come abbiamo visto nell'analisi della durata, una chiamata ricorsiva opera su al più $3n/4$ elementi. Dunque, otteniamo la seguente ricorsione

$$PM_1(n) = PM_1(\alpha n) + PM_1((1 - \alpha)n) + O(\lg n) \tag{27.9}$$

dove α è all'interno dell'intervallo $1/4 \le \alpha \le 3/4$. Notate inoltre che il valore effettivo di α può variare in ogni livello di ricorsione.

Dimostriamo che la ricorsione (27.9) ha la soluzione $PM_1 = O(n)$ utilizzando il metodo della sostituzione. Supponiamo che $PM_1(n) \le c_1 n - c_2 \lg n$ per qualche costante positiva c_1 e c_2. Facendo le opportune sostituzioni, si ha

$$\begin{aligned} PM_1(n) &\le (c_1\alpha n - c_2 \lg(\alpha n)) + (c_1(1 - \alpha)n - c_2 \lg((1 - \alpha)n)) + \Theta(\lg n) \\ &= c_1(\alpha + (1 - \alpha))n - c_2(\lg(\alpha n) + \lg((1 - \alpha)n)) + \Theta(\lg n) \\ &= c_1 n - c_2(\lg \alpha + \lg n + \lg(1 - \alpha) + \lg n) + \Theta(\lg n) \\ &= c_1 n - c_2 \lg n - (c_2(\lg n + \lg(\alpha(1 - \alpha))) - \Theta(\lg n)) \\ &\le c_1 n - c_2 \lg n \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

in quanto possiamo scegliere c_2 sufficientemente grande in modo che $c_2(\lg n + \lg(\alpha(1 - \alpha)))$ prevalga sul termine $\Theta(\lg n)$. Inoltre, possiamo scegliere c_1 sufficientemente grande per soddisfare le condizioni di base della ricorsione. Poiché il lavoro $PM_1(n)$ di P-MERGE è sia $\Omega(n)$ che $O(n)$, si ha $PM_1(n) = \Theta(n)$.

Il parallelismo di P-MERGE è $PM_1(n)/PM_\infty(n) = \Theta(n/\lg^2 n)$.

Merge sort multithread

Adesso che abbiamo una procedura di fusione multithread ben parallelizzata, possiamo incorporarla in un algoritmo multithread di merge sort. Questa versione di merge sort è simile alla procedura MERGE-SORT′ descritta in precedenza, ma diversamente da MERGE-SORT′, riceve come argomento un sottoarray di output B, che conterrà il risultato ordinato. In particolare, la chiamata P-MERGE-SORT(A, p, r, B, s) ordina gli elementi in $A[p\,..\,r]$ e li memorizza in $B[s\,..\,s + r - p]$.

```
P-MERGE-SORT(A, p, r, B, s)
 1  n = r − p + 1
 2  if n == 1
 3       B[s] = A[p]
 4  else sia T[1..n] un nuovo array
 5       q = ⌊(p + r)/2⌋
 6       q′ = q − p + 1
 7       spawn P-MERGE-SORT(A, p, q, T, 1)
 8       P-MERGE-SORT(A, q + 1, r, T, q′ + 1)
 9       sync
10       P-MERGE(T, 1, q′, q′ + 1, n, B, s)
```

Dopo che la riga 1 ha calcolato il numero di elementi n nel sottoarray di input $A[p\,..\,r]$, le righe 2–3 gestiscono il caso base quando l'array ha soltanto un elemento. Le righe 4–6 preparano la generazione ricorsiva nella riga 7 e la chiamata ricorsiva nella riga 8, che operano in parallelo. In particolare, la riga 4 alloca un array temporaneo T con n elementi per memorizzare i risultati del merge sort ricorsivo. La riga 5 calcola l'indice q di $A[p\,..\,r]$ per dividere gli elementi nei due sottoarray $A[p\,..\,q]$ e $A[q + 1\,..\,r]$ che saranno ordinati in modo ricorsivo; la riga 6 calcola il numero di elementi q' nel primo sottoarray $A[p\,..\,q]$, che la riga 8 usa per determinare l'indice in T da cui iniziare a memorizzare il risultato ordinato di $A[q + 1\,..\,r]$. A questo punto, vengono eseguite la generazione e la chiamata ricorsiva, seguite dall'istruzione **sync** nella riga 9, che forza la procedura ad aspettare finché la subroutine generata non sia terminata. Infine, la riga 10 chiama P-MERGE per fondere i sottoarray ordinati $T[1\,..\,q']$ e $T[q' + 1\,..\,n]$ nel sottoarray di output $B[s\,..\,s + r - p]$.

Analisi del merge sort multithread

Iniziamo con l'analisi del lavoro $PMS_1(n)$ di P-MERGE-SORT, che è considerevolmente più facile dell'analisi del lavoro di P-MERGE. Il lavoro è dato dalla ricorsione

$$\begin{aligned} PMS_1(n) &= 2\,PMS_1(n/2) + PM_1(n) \\ &= 2\,PMS_1(n/2) + \Theta(n) \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero ordine: 3951300 (20707-0 Copyright © 2022, McGraw-Hill Education (Italy)

che è uguale alla ricorsione (4.4) per la procedura MERGE-SORT normale descritta nel Paragrafo 2.3.1 e la cui soluzione è $PMS_1(n) = \Theta(n \lg n)$ per il caso 2 del teorema dell'esperto.

Deriviamo e analizziamo una ricorsione per la durata $PMS_\infty(n)$ nel caso peggiore. Poiché le due chiamate ricorsive di P-MERGE-SORT nelle righe 7 and 8 operano logicamente in parallelo, possiamo ignorarne una di esse, ottenendo la ricorsione

$$\begin{aligned} PMS_\infty(n) &= PMS_\infty(n/2) + PM_\infty(n) \\ &= PMS_\infty(n/2) + \Theta(\lg^2 n) \end{aligned} \tag{27.10}$$

Come per la ricorsione (27.8), non può essere applicato il teorema dell'esperto alla ricorsione (27.10), ma si può applicare l'Esercizio 4.6-2. La soluzione è $PMS_\infty(n) = \Theta(\lg^3 n)$, e quindi la durata di P-MERGE-SORT è $\Theta(\lg^3 n)$.

La fusione parallela conferisce a P-MERGE-SORT un notevole vantaggio di parallelismo rispetto a MERGE-SORT′. Ricordiamo che il parallelismo di MERGE-SORT′, che chiama la procedura seriale MERGE, è soltanto $\Theta(\lg n)$. Per P-MERGE-SORT, il parallelismo è

$$\begin{aligned} PMS_1(n)/PMS_\infty(n) &= \Theta(n \lg n)/\Theta(\lg^3 n) \\ &= \Theta(n/\lg^2 n) \end{aligned}$$

che è molto migliore sia in teoria che nella pratica. Una buona implementazione pratica potrebbe sacrificare un po' il parallelismo ingrossando il caso base per ridurre le costanti nascoste dalla notazione asintotica. Un modo semplice di ingrossare il caso base consiste nell'utilizzare un ordinamento seriale normale, per esempio quicksort, quando la dimensione dell'array è sufficientemente piccola.

Esercizi

27.3-1
Spiegate come ingrossare il caso base di P-MERGE.

27.3-2
Anziché trovare un elemento mediano nel sottoarray più grande, come fa la procedura P-MERGE, considerate una variante che trova un elemento mediano di tutti gli elementi nei due sottoarray ordinati utilizzando il risultato dell'Esercizio 9.3-8. Scrivete lo pseudocodice per una procedura di fusione multithreaded efficiente che utilizza questa variante. Analizzate il vostro algoritmo.

27.3-3
Scrivete un algoritmo multithreaded efficiente per suddividere un array intorno a un pivot, come fa la procedura PARTITION a pagina 142. Non è necessario suddividere l'array sul posto. Rendete il vostro algoritmo quanto più parallelo possibile. Analizzate il vostro algoritmo. (*Suggerimento:* potrebbe essere necessario utilizzare un array ausiliario ed effettuare più passaggi sugli elementi di input.)

27.3-4
Scrivete una versione multithreaded della procedura RECURSIVE-FFT descritta a pagina 761. Rendete la vostra implementazione quanto più parallela possibile. Analizzate il vostro algoritmo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016.220707-0 Copyright © 2022. McGraw-Hill Education (Italy)

27.3-5 ⋆
Scrivete una versione multithreaded della procedura RANDOMIZED-SELECT descritta a pagina 177. Rendete la vostra implementazione quanto più parallela possibile. Analizzate il vostro algoritmo. (*Suggerimento:* utilizzate l'algoritmo creato nell'Esercizio 27.3-3.)

27.3-6 ⋆
Spiegate come creare una versione multithread della procedura SELECT descritta nel Paragrafo 9.3. Rendete la vostra implementazione quanto più parallela possibile. Analizzate il vostro algoritmo.

Problemi

27-1 Implementare i cicli paralleli utilizzando il parallelismo annidato
Considerate il seguente algoritmo multithread per sommare a due a due gli elementi degli array $A[1..n]$ e $B[1..n]$, di dimensione n, scrivendo i risultati in $C[1..n]$:

```
SUM-ARRAYS(A, B, C)
1  parallel for i = 1 to A.length
2      C[i] = A[i] + B[i]
```

a. Riscrivete il ciclo parallelo in SUM-ARRAYS utilizzando il parallelismo annidato (**spawn** e **sync**) come nella procedura MAT-VEC-MAIN-LOOP. Analizzate il parallelismo della vostra implementazione.

Considerate la seguente implementazione alternativa del ciclo parallelo, che contiene un valore *grain-size* da specificare:

```
SUM-ARRAYS'(A, B, C)
1  n = A.length
2  grain-size = ?              // da determinare
3  r = ⌈n/grain-size⌉
4  for k = 0 to r − 1
5      spawn ADD-SUBARRAY(A, B, C, k · grain-size + 1,
                          min((k + 1) · grain-size, n))
6  sync

ADD-SUBARRAY(A, B, C, i, j)
1  for k = i to j
2      C[k] = A[k] + B[k]
```

b. Supponete di porre *grain-size* = 1. Qual è il parallelismo di questa implementazione?

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

c. Scrivete una formula per la durata di SUM-ARRAYS′ in funzione di n e *grain-size*. Derivate il miglior valore di *grain-size* per massimizzare il parallelismo.

27-2 Risparmiare spazio nella moltiplicazione delle matrici

La procedura P-MATRIX-MULTIPLY-RECURSIVE ha lo svantaggio che deve allocare una matrice temporanea T di dimensione $n \times n$, che può influire negativamente sulle costanti nascoste dalla notazione Θ. Tuttavia, la procedura P-MATRIX-MULTIPLY-RECURSIVE ha un parallelismo elevato. Per esempio, trascurando le costanti nella notazione Θ, il parallelismo per moltiplicare matrici 1000×1000 è approssimativamente $1000^3/10^2 = 10^7$, in quanto $\lg 1000 \approx 10$. La maggior parte dei computer paralleli hanno molto meno di 10 milioni di processori.

a. Descrivete un algoritmo multithread ricorsivo che non richiede la matrice temporanea T al costo di aumentare la durata a $\Theta(n)$ (*suggerimento:* calcolate $C = C + AB$ seguendo la strategia generale di P-MATRIX-MULTIPLY-RECURSIVE, ma inizializzate C in parallelo e inserite un'istruzione **sync** in una posizione opportunamente scelta).

b. Specificate e risolvete le ricorsioni per il lavoro e la durata della vostra implementazione.

c. Analizzate il parallelismo della vostra implementazione. Trascurando lo costanti nella notazione Θ, calcolate il parallelismo su matrici 1000×1000. Confrontate il parallelismo della vostra implementazione con quello della procedura P-MATRIX-MULTIPLY-RECURSIVE.

27-3 Algoritmi multithreaded per matrici

a. Parallelizzate la procedura LU-DECOMPOSITION descritta a pagina 686, scrivendo lo pseudocodice per una versione multithreaded di questo algoritmo. Rendete la vostra implementazione quanto più parallela possibile; analizzate il lavoro, la durata e il parallelismo.

b. Fate lo stesso per la procedura LUP-DECOMPOSITION a pagina 688.

c. Fate lo stesso per la procedura LUP-SOLVE a pagina 683.

d. Fate lo stesso per un algoritmo multithread che si basa sull'equazione (28.13) per invertire una matrice simmetrica definita positiva.

27-4 Riduzioni multithread e calcoli dei prefissi

Una ***⊗-riduzione*** di un array $x[1..n]$, dove $\otimes$ è un operatore associativo, è il valore

$$y = x[1] \otimes x[2] \otimes \cdots \otimes x[n]$$

La seguente procedura calcola in modo seriale la $\otimes$-riduzione di un sottoarray $x[i..j]$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

```
REDUCE(x, i, j)
1  y = x[i]
2  for k = i + 1 to j
3       y = y ⊗ x[k]
4  return y
```

a. Utilizzate il parallelismo annidato per implementare un algoritmo multithread P-REDUCE, che svolge la stessa funzione con un lavoro $\Theta(n)$ e una durata $\Theta(\lg n)$. Analizzate il vostro algoritmo.

Un problema correlato è quello del calcolo dei ***⊗-prefissi*** – operazione detta anche ***⊗-scansione*** – su un array $x[1..n]$, dove $\otimes$ è un operatore associativo. La $\otimes$-scansione genera il seguente array $y[1..n]$

$$\begin{aligned} y[1] &= x[1] \\ y[2] &= x[1] \otimes x[2] \\ y[3] &= x[1] \otimes x[2] \otimes x[3] \\ &\vdots \\ y[n] &= x[1] \otimes x[2] \otimes x[3] \otimes \cdots \otimes x[n] \end{aligned}$$

che è formato da tutti i prefissi dell'array x "sommati" utilizzando l'operatore $\otimes$. La seguente procedura seriale SCAN calcola i $\otimes$-prefissi:

```
SCAN(x)
1  n = x.length
2  sia y[1..n] un nuovo array
3  y[1] = x[1]
4  for i = 2 to n
5       y[i] = y[i − 1] ⊗ x[i]
6  return y
```

Purtroppo, rendere multithread la procedura SCAN non è semplice. Per esempio, cambiando il ciclo **for** con un ciclo **parallel for**, si creano dei conflitti, in quanto ogni iterazione del corpo del ciclo dipende dalla precedente iterazione. La seguente procedura P-SCAN-1 esegue il calcolo dei $\otimes$-prefissi in parallelo, benché in modo inefficiente:

```
P-SCAN-1(x)
1  n = x.length
2  sia y[1..n] un nuovo array
3  P-SCAN-1-AUX(x, y, 1, n)
4  return y

P-SCAN-1-AUX(x, y, i, j)
1  parallel for l = i to j
2       y[l] = P-REDUCE(x, 1, l)
```

b. Analizzate il lavoro, la durata e il parallelismo di P-SCAN-1.

Utilizzando il parallelismo annidato, potete ottenere un calcolo più efficiente dei $\otimes$-prefissi.

Acquistato da Michele Michele su Webster il 2022-01-17 08:13 - Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

```
P-SCAN-2(x)
1  n = x.length
2  sia y[1..n] un nuovo array
3  P-SCAN-2-AUX(x, y, 1, n)
4  return y
```

```
P-SCAN-2-AUX(x, y, i, j)
1  if i == j
2       y[i] = x[i]
3  else k = ⌊(i + j)/2⌋
4       spawn P-SCAN-2-AUX(x, y, i, k)
5       P-SCAN-2-AUX(x, y, k + 1, j)
6       sync
7       parallel for l = k + 1 to j
8            y[l] = y[k] ⊗ y[l]
```

c. Dimostrate che la procedura P-SCAN-2 è corretta; analizzate il lavoro, la durata e il parallelismo.

È possibile migliorare sia P-SCAN-1 sia P-SCAN-2 eseguendo il calcolo dei $\otimes$-prefissi in due passi distinti. Nel primo passo, raccogliamo in un array temporaneo t i valori delle $\otimes$-riduzioni di alcuni sottoarray contigui di x; nel secondo passo, utilizziamo i valori in t per calcolare il risultato finale y. Il seguente pseudocodice implementa questa strategia, ma alcune espressioni sono state omesse:

```
P-SCAN-3(x)
1  n = x.length
2  siano y[1..n] e t[1..n] due nuovi array
3  y[1] = x[1]
4  if n > 1
5       P-SCAN-UP(x, t, 2, n)
6       P-SCAN-DOWN(x[1], x, t, y, 2, n)
7  return y

P-SCAN-UP(x, t, i, j)
1  if i == j
2       return x[i]
3  else
4       k = ⌊(i + j)/2⌋
5       t[k] = spawn P-SCAN-UP(x, t, i, k)
6       right = P-SCAN-UP(x, t, k + 1, j)
7       sync
8       return _____          // completate l'espressione

P-SCAN-DOWN(v, x, t, y, i, j)
1  if i == j
2       y[i] = v ⊗ x[i]
3  else
4       k = ⌊(i + j)/2⌋
5       spawn P-SCAN-DOWN(_____, x, t, y, i, k)    // completate l'espressione
6       P-SCAN-DOWN(_____, x, t, y, k + 1, j)      // completate l'espressione
7       sync
```

Acquistato da Michele Michelon Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

d. Completate le tre espressioni mancanti nella riga 8 di P-SCAN-UP e nelle righe 5 e 6 di P-SCAN-DOWN. Dimostrate che, con le espressioni che avete inserito, la procedura P-SCAN-3 è corretta (*suggerimento:* dimostrate che il valore v passato a P-SCAN-DOWN(v, x, t, y, i, j) soddisfa la relazione $v = x[1] \otimes x[2] \otimes \cdots \otimes x[i-1]$).

e. Analizzate il lavoro, la durata e il parallelismo di P-SCAN-3.

27-5 Un algoritmo multithread per un semplice calcolo con lo stampino

La scienza del calcolo è piena di algoritmi che richiedono di riempire un array con i valori che dipendono dai valori di certi elementi vicini già calcolati, insieme ad altre informazioni che non cambiano durante il corso del calcolo. Lo schema degli elementi vicini non cambia durante il calcolo e viene detto ***stampino*** (*stencil)*. Per esempio, il Paragrafo 15.4 presenta un algoritmo a stampino per calcolare la più lunga sottosequenza comune, dove il valore dell'elemento $c[i, j]$ dipende soltanto dai valori di $c[i-1, j]$, $c[i, j-1]$ e $c[i-1, j-1]$, come pure dagli elementi x_i e y_j all'interno delle due sequenze di input. Le sequenze di input non variano, ma l'algoritmo riempie l'array bidimensionale c in modo da calcolare l'elemento $c[i, j]$ dopo aver calcolato i tre elementi $c[i-1, j]$, $c[i, j-1]$ e $c[i-1, j-1]$.

In questo problema, vediamo come applicare il parallelismo annidato per rendere multitread un semplice calcolo con stampino su un array A, di dimensioni $n \times n$, nel quale, dei valori di A, quello posto nell'elemento $A[i, j]$ dipende soltanto dai valori di $A[i', j']$, dove $i' \le i$ e $j' \le j$ (e, ovviamente, $i' \ne i$ o $j' \ne j$). In altre parole, il valore di un elemento dipende soltanto dai valori degli elementi che si trovano sopra di esso e/o alla sua sinistra, insieme alle informazioni statiche all'esterno dell'array. Supponiamo inoltre che, una volta che sono stati riempiti gli elementi dai quali dipende $A[i, j]$, possiamo riempire $A[i, j]$ nel tempo $\Theta(1)$ (come nella procedura LCS-LENGTH descritta nel Paragrafo 15.4).

Possiamo suddividere l'array A, di dimensioni $n \times n$, in quattro sottoarray di dimensioni $n/2 \times n/2$, nel modo seguente:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \tag{27.11}$$

Notate che è possibile riempire ricorsivamente il sottoarray A_{11}, in quanto esso non dipende dagli elementi degli altri tre sottoarray. Una volta riempito A_{11}, possiamo continuare a riempire ricorsivamente A_{12} e A_{21} in parallelo perché, sebbene entrambi dipendano da A_{11}, essi sono indipendenti l'uno dall'altro. Infine, possiamo riempire ricorsivamente A_{22}.

a. Sulla base delle precedenti considerazioni e applicando la scomposizione (27.11), scrivete lo pseudocodice multithread che esegue questo semplice calcolo con stampino utilizzando un algoritmo divide et impera SIMPLE-STENCIL. (Non preoccupatevi dei dettagli del caso base, che dipende dal particolare stampino.) Specificate e risolvete le ricorsioni per il lavoro e la durata del vostro algoritmo in funzione di n. Qual è il parallelismo?

Acquistato da Michele Michele su Webster il 2022-07-07 23:10. Numero Ordine Libreria: 1985030l6-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

b. Modificate la vostra soluzione del punto (a) per dividere un array $n \times n$ in nove sottoarray $n/3 \times n/3$, utilizzando la ricorsione con il massimo parallelismo possibile. Analizzate questo algoritmo. Quanto parallelismo in più o in meno ha questo algoritmo rispetto a quello del punto (a)?

c. Generalizzate le vostre soluzioni dei punti (a) e (b) nel modo seguente. Scegliete un numero intero $b \geq 2$. Dividete un array $n \times n$ in b^2 sottoarray, ciascuno di dimensioni $n/b \times n/b$, utilizzando la ricorsione con il massimo parallelismo possibile. Esprimete il lavoro, la durata e il parallelismo del vostro algoritmo in funzione di n e b. Dimostrate che, utilizzando questo approccio, il parallelismo deve essere $o(n)$ per qualsiasi $b \geq 2$. (*Suggerimento:* per quest'ultimo argomento, dimostrate che l'esponente di n nel parallelismo è strettamente minore di 1 per qualsiasi $b \geq 2$.)

d. Scrivete lo pseudocodice di un algoritmo multithread per questo semplice calcolo con stampino raggiungendo un parallelismo pari a $\Theta(n/\lg n)$. Dimostrate, utilizzando il lavoro e la durata, che il problema in effetti ha un parallelismo intrinseco pari a $\Theta(n)$. Di conseguenza, la natura divide et impera del nostro pseudocodice multithread non ci consente di raggiungere questo parallelismo massimo.

27-6 Algoritmi multithread randomizzati

Come per i normali algoritmi seriali, a volte vogliamo implementare algoritmi multithread randomizzati. Questo problema esamina come adattare le varie misure delle prestazioni per gestire il comportamento atteso di tali algoritmi. Il problema chiede anche di progettare e analizzare un algoritmo multithread per un quicksort randomizzato.

a. Spiegate come modificare la legge del lavoro (27.2), la legge della durata (27.3) e il limite dello scheduler goloso (27.4) per gestire i comportamenti attesi quando T_P, T_1 e T_∞ sono tutte variabili casuali.

b. Considerate un algoritmo multithread randomizzato per il quale per l'1% del tempo si ha $T_1 = 10^4$ e $T_{10,000} = 1$, ma per il 99% del tempo si ha $T_1 = T_{10,000} = 10^9$. Dimostrate che lo ***speedup*** di un algoritmo multithread randomizzato deve essere definito come $\mathrm{E}[T_1]/\mathrm{E}[T_P]$, anziché $\mathrm{E}[T_1/T_P]$.

c. Dimostrate che il ***parallelismo*** di un algoritmo multithread randomizzato deve essere definito come il rapporto $\mathrm{E}[T_1]/\mathrm{E}[T_\infty]$.

d. Applicate il multithreading all'algoritmo RANDOMIZED-QUICKSORT descritto a pagina 148 utilizzando il parallelismo annidato. (Non parallelizzate RANDOMIZED-PARTITION.) Scrivete lo pseudocodice per il vostro algoritmo P-RANDOMIZED-QUICKSORT.

e. Analizzate il vostro algoritmo multithread per il quicksort randomizzato (*suggerimento:* rivedete l'analisi di RANDOMIZED-SELECT a pagina 177).

Note

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

I computer paralleli, i modelli per i computer paralleli e i modelli algoritmici per la programmazione parallela si sono diffusi sotto varie forme per anni. Le edizioni precedenti di questo libro hanno trattato le reti di ordinamento e il modello PRAM (Parallel Random-Access Machine). Il modello data-parallel [48, 169] è un altro noto modello di programmazione algoritmica, che presenta le operazioni sui vettori e sulle matrici come primitive.

continua

Graham [150] e Brent [56] hanno dimostrato che esistono scheduler che raggiungono il limite del Teorema 27.1. Eager, Zahorian e Lazowska [99] hanno dimostrato che qualsiasi scheduler goloso raggiunge tale limite e hanno proposto la metodologia basata sul lavoro e la durata (benché con altro nome) per analizzare gli algoritmi paralleli. Blelloch [47] ha sviluppato un modello di programmazione algoritmica basato sul lavoro e la durata (che ha chiamato la "profondità" del calcolo) per la programmazione data-parallel. Blumofe e Leiserson [53] hanno creato un algoritmo di pianificazione distribuito per il multithreading dinamico basato sul "work-stealing" (sottrazione del lavoro) randomizzato e hanno dimostrato che l'algoritmo raggiunge il limite $\mathrm{E}[T_P] \leq T_1/P + O(T_\infty)$. Arora, Blumofe e Plaxton [19] e Blelloch, Gibbons e Matias [49] hanno scritto ottimi algoritmi per la pianificazione del multithreading dinamico.

Lo pseudocodice multithread e il modello di programmazione sono stati influenzati dal progetto Cilk [51, 119] del MIT e dalle estensioni Cilk++ [72] al C++ distribuite da Cilk Arts, Inc. Molti degli algoritmi multithread di questo capitolo sono apparsi nelle note non pubblicate di C. E. Leiserson e H. Prokop e sono stati implementati in Cilk o Cilk++. L'algoritmo multithread per merge sort è stato ispirato da un algoritmo di Akl [12].

Il concetto di coerenza sequenziale è dovuto a Lamport [224].

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Operazioni con le matrici 28

Le operazioni con le matrici sono essenziali nel calcolo scientifico. Algoritmi efficienti per operare con le matrici sono quindi di notevole interesse pratico. Questo capitolo tratta le operazioni con le matrici, in particolare la moltiplicazione delle matrici e la risoluzione dei sistemi di equazioni lineari. L'Appendice D tratta i concetti fondamentali delle matrici.

Il Paragrafo 28.1 spiega come risolvere un sistema di equazioni lineari utilizzando le fattorizzazioni LUP. Poi, il Paragrafo 28.2 esamina la stretta relazione tra il problema di moltiplicare le matrici e il problema di invertire una matrice. Infine, il Paragrafo 28.3 descrive l'importante classe delle matrici simmetriche e definite positive e spiega come queste matrici possono essere utilizzate per trovare una soluzione ai minimi quadrati di un sistema sovradeterminato di equazioni lineari.

Un importante problema che si presenta nella pratica è la ***stabilità numerica***. A causa della limitata precisione delle rappresentazioni in virgola mobile degli attuali calcolatori, gli errori di arrotondamento nei calcoli numerici possono essere amplificati durante un procedimento di calcolo, generando risultati inesatti; tali calcoli sono ***numericamente instabili***. Anche se in qualche occasione considereremo brevemente la stabilità numerica, tuttavia essa non è un argomento che sarà approfondito in questo capitolo. Consigliamo al lettore l'eccellente libro di Golub e Van Loan [145] per un'analisi completa dei problemi di stabilità numerica.

28.1 Risoluzione dei sistemi di equazioni lineari

Risolvere un sistema di equazioni lineari è un problema fondamentale che si presenta in diverse applicazioni. Un sistema lineare può essere espresso come un'equazione matriciale in cui ogni elemento della matrice o del vettore appartiene a un campo, tipicamente i numeri reali $\mathbb{R}$. Questo paragrafo spiega come risolvere un sistema di equazioni lineari utilizzando un metodo chiamato fattorizzazione LUP.

Iniziamo da un sistema di equazioni lineari in n incognite $x_1, x_2, \ldots, x_n$:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 , \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 , \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n \end{aligned} \tag{28.1}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Un insieme di valori per $x_1, x_2, \ldots, x_n$ che soddisfano contemporaneamente tutte le equazioni (28.1) è detto ***soluzione*** di queste equazioni. In questo paragrafo, trattiamo soltanto il caso in cui ci sono esattamente n equazioni in n incognite.

Per comodità riscriviamo le equazioni (28.1) come equazione matrice-vettore

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}$$

ovvero, ponendo $A = (a_{ij})$, $x = (x_i)$ e $b = (b_i)$, nella forma

$$Ax = b \tag{28.2}$$

Se A non è singolare, ha la matrice inversa A^{-1}, e

$$x = A^{-1}b \tag{28.3}$$

è il vettore soluzione. Possiamo dimostrare che x è l'unica soluzione dell'equazione (28.2) nel modo seguente. Se ci fossero due soluzioni, x e x', allora $Ax = Ax' = b$ e, indicando con I la matrice identità, si ha

$$\begin{aligned} x &= Ix \\ &= (A^{-1}A)x \\ &= A^{-1}(Ax) \\ &= A^{-1}(Ax') \\ &= (A^{-1}A)x' \\ &= x' \end{aligned}$$

In questo paragrafo, ci occuperemo principalmente del caso in cui A non è singolare ovvero, per il Teorema D.1, il rango di A è uguale al numero n di incognite. Tuttavia, ci sono altre possibilità che meritano una breve discussione. Se il numero di equazioni è minore del numero n di incognite – o, più in generale, se il rango di A è minore di n – allora il sistema è ***indeterminato***. Tipicamente, un sistema indeterminato ha infinite soluzioni, ma potrebbe non ammettere alcuna soluzione se le equazioni non sono coerenti. Se il numero di equazioni supera il numero n di incognite, il sistema è ***sovradeterminato***, e potrebbe non esistere alcuna soluzione. La ricerca di buone soluzioni approssimate per i sistemi di equazioni lineari sovradeterminati è un problema importante che sarà trattato nel Paragrafo 28.3.

Ritorniamo al problema di risolvere il sistema $Ax = b$ di n equazioni in n incognite. Un metodo potrebbe essere quello di calcolare A^{-1} e poi, utilizzando l'equazione (28.3), moltiplicare entrambi i lati per A^{-1}, ottenendo $x = A^{-1}b$. Questo metodo, nella pratica, ha il problema della instabilità numerica. Fortunatamente, esiste un altro metodo – la fattorizzazione LUP – che è numericamente stabile e ha il vantaggio ulteriore di essere più veloce nella pratica.

Descrizione della fattorizzazione LUP

L'idea che sta alla base della fattorizzazione LUP consiste nel trovare tre matrici L, U e P, di dimensioni $n \times n$, tali che

Acquistato da Michele Michele su Webster il 2022-07-01 23:12 Numero Ordine Libreria: 1999033015-2207-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$PA = LU \tag{28.4}$$

dove

- L è una matrice triangolare inferiore unitaria
- U è una matrice triangolare superiore
- P è una matrice di permutazione

Le matrici L, U e P che soddisfano l'equazione (28.4) sono una ***fattorizzazione LUP*** della matrice A. Vedremo che ogni matrice A non singolare possiede tale fattorizzazione. Il vantaggio di calcolare una fattorizzazione LUP della matrice A è che i sistemi lineari possono essere risolti più rapidamente quando sono triangolari, come nel caso di entrambe le matrici L e U. Avendo trovato una fattorizzazione LUP di A, possiamo risolvere l'equazione (28.2) $Ax = b$ risolvendo soltanto i sistemi lineari triangolari, nel modo seguente. Moltiplicando entrambi i lati di $Ax = b$ per P si ottiene l'equazione equivalente $PAx = Pb$, che per l'Esercizio D.2-3 equivale a permutare le equazioni (28.1). Applicando la fattorizzazione (28.4), si ottiene

$$LUx = Pb$$

Adesso possiamo risolvere questa equazione risolvendo due sistemi lineari triangolari. Definiamo $y = Ux$, dove x è il vettore soluzione cercato. Innanzi tutto, risolviamo il sistema triangolare inferiore

$$Ly = Pb \tag{28.5}$$

per trovare il vettore incognito y, applicando il cosiddetto "metodo delle sostituzioni in avanti". Avendo trovato y, possiamo risolvere il sistema triangolare superiore

$$Ux = y \tag{28.6}$$

per trovare l'incognita x, applicando il cosiddetto "metodo delle sostituzioni all'indietro". Dal momento che la matrice di permutazione P è invertibile (Esercizio D.2-3), moltiplicando entrambi i membri dell'equazione (28.4) per P^{-1}, si ha $P^{-1}PA = P^{-1}LU$, ovvero

$$A = P^{-1}LU \tag{28.7}$$

Dunque, il vettore x è la soluzione di $Ax = b$:

$$\begin{aligned} Ax &= P^{-1}LUx && \text{(per l'equazione (28.7))} \\ &= P^{-1}Ly && \text{(per l'equazione (28.6))} \\ &= P^{-1}Pb && \text{(per l'equazione (28.5))} \\ &= b \end{aligned}$$

Adesso dobbiamo spiegare come funzionano i metodi delle sostituzioni in avanti e all'indietro; poi affronteremo il problema di calcolare la fattorizzazione LUP.

Sostituzioni in avanti e all'indietro

Il metodo delle ***sostituzioni in avanti*** può risolvere il sistema triangolare inferiore (28.5) nel tempo $\Theta(n^2)$, essendo noti L, P e b. Per comodità, rappresentiamo la matrice di permutazione P in modo compatto con un array $\pi[1..n]$. Per $i = 1, 2, \ldots, n$, l'elemento $\pi[i]$ indica che $P_{i,\pi[i]} = 1$ e $P_{ij} = 0$ per $j \neq \pi[i]$. Quindi, PA ha $a_{\pi[i],j}$ nella riga i e nella colonna j, e Pb ha $b_{\pi[i]}$ come suo i-esimo elemento. Poiché L è una matrice triangolare inferiore unitaria, l'equazione (28.5) può essere riscritta così:

Acquistato da Michele Mariotti ed Vodafone p. 2022 04 21 2022 Numero Ordine Licenza: 1095901 del 21/04/2022 Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned}
y_1 &= b_{\pi[1]} \\
l_{21}y_1 + y_2 &= b_{\pi[2]} \\
l_{31}y_1 + l_{32}y_2 + y_3 &= b_{\pi[3]} \\
&\vdots \\
l_{n1}y_1 + l_{n2}y_2 + l_{n3}y_3 + \cdots + y_n &= b_{\pi[n]}
\end{aligned}$$

Possiamo trovare direttamente y_1, perché la prima equazione ci dice che $y_1 = b_{\pi[1]}$. Avendo trovato y_1, possiamo sostituirlo nella seconda equazione, ottenendo

$$y_2 = b_{\pi[2]} - l_{21}y_1$$

Adesso, possiamo sostituire y_1 e y_2 nella terza equazione, ottenendo

$$y_3 = b_{\pi[3]} - (l_{31}y_1 + l_{32}y_2)$$

In generale, sostituiamo $y_1, y_2, \ldots, y_{i-1}$ "in avanti" nella i-esima equazione per trovare y_i:

$$y_i = b_{\pi[i]} - \sum_{j=1}^{i-1} l_{ij}y_j$$

Il metodo delle ***sostituzioni all'indietro*** è simile a quello delle sostituzioni in avanti. Noti U e y, risolviamo prima la n-esima equazione e poi procediamo all'indietro fino alla prima equazione. Analogamente alle sostituzioni in avanti, questo processo viene eseguito nel tempo $\Theta(n^2)$. Poiché U è una matrice triangolare superiore, possiamo riscrivere il sistema (28.6) così:

$$\begin{aligned}
u_{11}x_1 + u_{12}x_2 + \cdots + u_{1,n-2}x_{n-2} + u_{1,n-1}x_{n-1} + u_{1n}x_n &= y_1 \\
u_{22}x_2 + \cdots + u_{2,n-2}x_{n-2} + u_{2,n-1}x_{n-1} + u_{2n}x_n &= y_2 \\
&\vdots \\
u_{n-2,n-2}x_{n-2} + u_{n-2,n-1}x_{n-1} + u_{n-2,n}x_n &= y_{n-2} \\
u_{n-1,n-1}x_{n-1} + u_{n-1,n}x_n &= y_{n-1} \\
u_{n,n}x_n &= y_n
\end{aligned}$$

Quindi, possiamo trovare $x_n, x_{n-1}, \ldots, x_1$ in successione:

$$\begin{aligned}
x_n &= y_n/u_{n,n} \\
x_{n-1} &= (y_{n-1} - u_{n-1,n}x_n)/u_{n-1,n-1} \\
x_{n-2} &= (y_{n-2} - (u_{n-2,n-1}x_{n-1} + u_{n-2,n}x_n))/u_{n-2,n-2} \\
&\vdots
\end{aligned}$$

o in generale

$$x_i = \left(y_i - \sum_{j=i+1}^{n} u_{ij}x_j\right)/u_{ii}$$

Acquistato da Michele Michele su Webster il 2022-07-23 13:12 Numero Ordine Licenza: 00503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Noti P, L, U e b, la procedura LUP-SOLVE trova x combinando i metodi delle sostituzioni in avanti e all'indietro. Lo pseudocodice suppone che la dimensione n appaia nell'attributo $L.rows$ e che la matrice di permutazione P sia rappresentata dall'array π.

LUP-SOLVE(L, U, π, b)
1 $n = L.rows$
2 Sia x un nuovo vettore di lunghezza n
3 **for** $i = 1$ **to** n
4 $\quad y_i = b_{\pi[i]} - \sum_{j=1}^{i-1} l_{ij} y_j$
5 **for** $i = n$ **downto** 1
6 $\quad x_i = \left(y_i - \sum_{j=i+1}^{n} u_{ij} x_j\right) / u_{ii}$
7 **return** x

La procedura LUP-SOLVE trova y effettuando le sostituzioni in avanti nelle righe 3–4; poi trova x effettuando le sostituzioni all'indietro nelle righe 5–6. Poiché c'è un ciclo implicito nelle sommatorie all'interno di ciascuno dei cicli **for**, il tempo di esecuzione è $\Theta(n^2)$.

Come esempio di questi metodi, consideriamo il sistema di equazioni lineari definito in questo modo:

$$\begin{pmatrix} 1 & 2 & 0 \\ 3 & 4 & 4 \\ 5 & 6 & 3 \end{pmatrix} x = \begin{pmatrix} 3 \\ 7 \\ 8 \end{pmatrix}$$

dove

$$A = \begin{pmatrix} 1 & 2 & 0 \\ 3 & 4 & 4 \\ 5 & 6 & 3 \end{pmatrix}$$

$$b = \begin{pmatrix} 3 \\ 7 \\ 8 \end{pmatrix}$$

Vogliamo trovare l'incognita x. La fattorizzazione LUP è

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 0.2 & 1 & 0 \\ 0.6 & 0.5 & 1 \end{pmatrix}$$

$$U = \begin{pmatrix} 5 & 6 & 3 \\ 0 & 0.8 & -0.6 \\ 0 & 0 & 2.5 \end{pmatrix}$$

$$P = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

(Il lettore può verificare che $PA = LU$.) Utilizzando il metodo delle sostituzioni in avanti, risolviamo $Ly = Pb$ rispetto a y

$$\begin{pmatrix} 1 & 0 & 0 \\ 0.2 & 1 & 0 \\ 0.6 & 0.5 & 1 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 8 \\ 3 \\ 7 \end{pmatrix}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Otteniamo

$$y = \begin{pmatrix} 8 \\ 1.4 \\ 1.5 \end{pmatrix}$$

calcolando prima y_1, poi y_2 e infine y_3.

Effettuando le sostituzioni all'indietro, risolviamo $Ux = y$ rispetto a x

$$\begin{pmatrix} 5 & 6 & 3 \\ 0 & 0.8 & -0.6 \\ 0 & 0 & 2.5 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 8 \\ 1.4 \\ 1.5 \end{pmatrix}$$

Otteniamo la soluzione cercata

$$x = \begin{pmatrix} -1.4 \\ 2.2 \\ 0.6 \end{pmatrix}$$

calcolando prima x_3, poi x_2 e infine x_1.

Calcolo di una fattorizzazione LU

Abbiamo visto che, se una fattorizzazione LUP può essere calcolata per una matrice A non singolare, è possibile utilizzare le sostituzioni in avanti e all'indietro per risolvere il sistema di equazioni lineari $Ax = b$. Adesso dobbiamo spiegare come trovare in maniera efficiente una fattorizzazione LUP della matrice A. Iniziamo dal caso in cui A sia una matrice $n \times n$ non singolare e P sia assente (ovvero $P = I_n$). In questo caso, dobbiamo trovare una fattorizzazione $A = LU$. Le due matrici L e U sono una ***fattorizzazione LU*** di A.

Il processo con il quale eseguiamo la fattorizzazione LU è detto ***eliminazione di Gauss***. Iniziamo sottraendo dei multipli della prima equazione dalle altre equazioni, in modo che la prima variabile sia rimossa dalle altre equazioni. Poi, sottraiamo dei multipli della seconda equazione dalla terza e dalle successive equazioni, in modo che la prima e la seconda variabile siano rimosse da tali equazioni. Continuiamo questo processo finché il sistema restante non avrà una forma triangolare superiore – in effetti, è la matrice U. La matrice L è composta dai moltiplicatori di riga che provocano l'eliminazione delle variabili.

L'algoritmo per implementare questa strategia è ricorsivo. Vogliamo ottenere una fattorizzazione LU per una matrice A non singolare di dimensioni $n \times n$. Se $n = 1$, allora abbiamo finito, perché possiamo scegliere $L = I_1$ e $U = A$. Per $n > 1$, dividiamo A in quattro parti

$$\begin{aligned} A &= \left(\begin{array}{c|ccc} a_{11} & a_{12} & \cdots & a_{1n} \\ \hline a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{array}\right) \\ &= \begin{pmatrix} a_{11} & w^{\mathrm{T}} \\ v & A' \end{pmatrix} \end{aligned}$$

dove v è un vettore colonna di dimensione $(n-1)$, w^{T} è un vettore riga di dimensione $(n-1)$ e A' è una matrice $(n-1) \times (n-1)$. Poi, applicando l'algebra delle matrici (verificate le equazioni effettuando semplicemente le moltiplicazioni), possiamo fattorizzare A in questo modo:

Acquistato da Michele Michele su Webster il 2022-07-07 23:42. Non vendibile. Utente: 1025020 - 6540707. © Copyright © 2022 McGraw-Hill Education (Italy)

$$\begin{aligned} A &= \begin{pmatrix} a_{11} & w^{\mathrm{T}} \\ v & A' \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ v/a_{11} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{11} & w^{\mathrm{T}} \\ 0 & A' - vw^{\mathrm{T}}/a_{11} \end{pmatrix} \end{aligned} \tag{28.8}$$

Gli zeri nella prima e nella seconda matrice dell'equazione (28.8) sono, rispettivamente, il vettore riga e il vettore colonna di dimensione $n-1$. Il termine vw^{T}/a_{11}, formato prendendo il prodotto esterno di v e w e dividendo ciascun elemento del risultato per a_{11}, è una matrice $(n-1) \times (n-1)$, che è conforme nella dimensione alla matrice A' dalla quale è stata sottratta. La matrice $(n-1) \times (n-1)$ risultante

$$A' - vw^{\mathrm{T}}/a_{11} \tag{28.9}$$

è detta ***complemento di Schur*** della matrice A rispetto all'elemento a_{11}.

Asseriamo che, se A non è singolare, allora anche il complemento di Schur non è singolare. Perché? Supponiamo che il complemento di Schur, che ha dimensione $(n-1) \times (n-1)$, sia singolare. Allora, per il Teorema D.1, esso ha rango di riga strettamente minore di $n-1$. Poiché gli ultimi $n-1$ elementi nella prima colonna della matrice

$$\begin{pmatrix} a_{11} & w^{\mathrm{T}} \\ 0 & A' - vw^{\mathrm{T}}/a_{11} \end{pmatrix}$$

sono tutti 0, le ultime $n-1$ righe di questa matrice devono avere rango di riga strettamente minore di $n-1$. Il rango di riga dell'intera matrice, quindi, è strettamente minore di n. Applicando l'Esercizio D.2-8 all'equazione (28.8), il rango della matrice A è strettamente minore di n e, per il Teorema D.1, si perviene alla contraddizione che A è singolare.

Poiché il complemento di Schur non è singolare, adesso possiamo trovare ricorsivamente una fattorizzazione LU per esso. Poniamo

$$A' - vw^{\mathrm{T}}/a_{11} = L'U'$$

dove L' è una matrice triangolare inferiore unitaria e U' è una matrice triangolare superiore. Applicando l'algebra delle matrici, si ha

$$\begin{aligned} A &= \begin{pmatrix} 1 & 0 \\ v/a_{11} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{11} & w^{\mathrm{T}} \\ 0 & A' - vw^{\mathrm{T}}/a_{11} \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ v/a_{11} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{11} & w^{\mathrm{T}} \\ 0 & L'U' \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ v/a_{11} & L' \end{pmatrix} \begin{pmatrix} a_{11} & w^{\mathrm{T}} \\ 0 & U' \end{pmatrix} \\ &= LU \end{aligned}$$

che è la fattorizzazione LU (notate che, poiché L' è una matrice triangolare inferiore unitaria, anche L lo è, e poiché U' è una matrice triangolare superiore, anche U lo è).

Ovviamente, se $a_{11} = 0$, questo metodo non funziona, perché ci sarebbe una divisione per 0. Esso non funziona neanche quando il primo elemento in alto a sinistra del complemento di Schur $A' - vw^{\mathrm{T}}/a_{11}$ è 0, perché il successivo passo di ricorsione prevede una divisione per tale elemento. Gli elementi per i quali si divide durante la fattorizzazione LU sono detti ***pivot*** e occupano le posizioni lungo la diagonale della matrice U. La ragione per la quale includiamo una matrice di permutazione P durante la fattorizzazione LUP è che essa ci consente di evitare di dividere per zero gli elementi. L'uso delle permutazioni per evitare la divisione per 0 (o per numeri piccoli, il che può provocare instabilità numerica) è detto ***pivoting***.

Acquistato da Michele Mo... per uso esclusivo... Copyright © 2022 McGraw-Hill Education (Italy)

Una classe importante di matrici per cui la fattorizzazione LU funziona sempre correttamente è la classe delle matrici simmetriche e definite positive. Tali matrici non richiedono pivoting e, quindi, la strategia ricorsiva precedentemente descritta può essere impiegata senza temere di dividere per 0. Proveremo questo risultato e altri ancora nel Paragrafo 28.3.

Il codice per la fattorizzazione LU di una matrice A si ricava dalla strategia ricorsiva, con la differenza che un ciclo di iterazione sostituisce la ricorsione (questa trasformazione è un'ottimizzazione standard per una procedura "ricorsiva in coda" – una procedura la cui ultima operazione è una chiamata ricorsiva a sé stessa. Si veda il Problema 7-4). Si suppone che la dimensione di A sia memorizzata nell'attributo $A.rows$. Inizializziamo la matrice U con elementi 0 sotto la diagonale e la matrice L con elementi 1 nella diagonale e con elementi 0 sopra la diagonale.

```
LU-DECOMPOSITION(A)
 1  n = A.rows
 2  Siano L e U due nuove matrici n × n
 3  Inizializza U con elementi 0 sotto la diagonale
 4  Inizializza L con elementi 1 nella diagonale
        e con elementi 0 sopra la diagonale
 5  for k = 1 to n
 6      u_kk = a_kk
 7      for i = k + 1 to n
 8          l_ik = a_ik/u_kk          // l_ik contiene v_i
 9          u_ki = a_ki               // u_ki contiene w_i^T
10      for i = k + 1 to n
11          for j = k + 1 to n
12              a_ij = a_ij − l_ik u_kj
13  return L e U
```

Il ciclo **for** esterno, che inizia nella riga 5, si ripete una volta per ogni passo ricorsivo. All'interno di questo ciclo, il pivot è calcolato come $u_{kk} = a_{kk}$ nella riga 6. All'interno del ciclo **for** nelle righe 7–9 (che non vengono eseguite quando $k = n$), i vettori v e w^{T} sono utilizzati per aggiornare L e U. Gli elementi del vettore v sono calcolati nella riga 8, dove v_i è memorizzato in l_{ik}; gli elementi del vettore w^{T} sono calcolati nella riga 9, dove w_i^{T} è memorizzato in u_{ki}. Infine, gli elementi del complemento di Schur sono calcolati nelle righe 10–12 e memorizzati di nuovo nella matrice A. (Non occorre dividere per a_{kk} nella riga 12 perché lo abbiamo già fatto quando abbiamo calcolato l_{ik} nella riga 8.) Poiché la riga 12 ha tre livelli di annidamento, la procedura LU-DECOMPOSITION viene eseguita nel tempo $\Theta(n^3)$.

La Figura 28.1 illustra il funzionamento di LU-DECOMPOSITION; notate un'ottimizzazione standard della procedura in cui gli elementi significativi di L e U sono memorizzati "sul posto" nella matrice A. Ovvero, possiamo stabilire una corrispondenza tra ogni elemento a_{ij} e l_{ij} (se $i > j$) o u_{ij} (se $i \leq j$) e aggiornare la matrice A in modo che contenga sia L che U quando la procedura termina. Lo pseudocodice per questa ottimizzazione è ottenuto dal precedente pseudocodice sostituendo semplicemente i riferimenti a l e u con a; non è difficile verificare che questa trasformazione preserva la correttezza.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950316-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

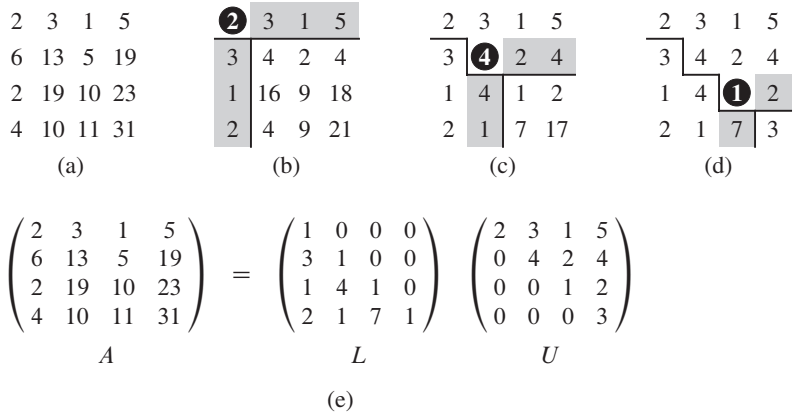


Figura 28.1 Il funzionamento di LU-DECOMPOSITION. **(a)** La matrice A. **(b)** L'elemento $a_{11} = 2$ nel cerchio nero è il pivot, la colonna ombreggiata è v/a_{11} e la riga ombreggiata è w^{T}. Gli elementi di U finora calcolati si trovano sopra la linea orizzontale e gli elementi di L sono a sinistra della linea verticale. Il complemento di Schur $A' - vw^{\mathrm{T}}/a_{11}$ si trova in basso a destra. **(c)** Qui operiamo sul complemento di Schur prodotto nella parte (b). L'elemento $a_{22} = 4$ nel cerchio nero è il pivot e la riga e la colonna ombreggiate sono, rispettivamente, v/a_{22} e w^{T} (nel partizionamento del complemento di Schur). Le linee dividono la matrice tra gli elementi di U finora calcolati (in alto), gli elementi di L finora calcolati (a sinistra) e il nuovo complemento di Schur (in basso a destra). **(d)** Il passo successivo completa la fattorizzazione (l'elemento 3 nel nuovo complemento di Schur diventa parte di U quando termina la ricorsione). **(e)** La fattorizzazione $A = LU$.

Calcolo di una fattorizzazione LUP

In generale, quando si risolve un sistema di equazioni lineari $Ax = b$, dobbiamo utilizzare come pivot anche degli elementi che non appartengono alla diagonale di A per evitare di dividere per 0. Non soltanto bisogna evitare di dividere per 0, ma anche di dividere per un numero troppo piccolo, anche se A non è singolare, perché potrebbe verificarsi il problema dell'instabilità numerica durante i calcoli. Per questo motivo, cercheremo di utilizzare come pivot un valore grande.

Le operazioni matematiche per calcolare la fattorizzazione LUP sono simili a quelle della fattorizzazione LU. Ricordiamo che, data una matrice non singolare A di dimensioni $n \times n$, vogliamo trovare una matrice di permutazione P, una matrice triangolare inferiore unitaria L e una matrice triangolare superiore U tali che $PA = LU$. Prima di partizionare la matrice A, come abbiamo fatto per la fattorizzazione LU, spostiamo un elemento non nullo, per esempio a_{k1}, da una posizione nella prima colonna alla posizione $(1, 1)$ della matrice. (Se la prima colonna contiene soltanto zeri, allora A è singolare, perché il suo determinante è zero, per i Teoremi D.4 e D.5.) Per preservare il sistema di equazioni, scambiamo la riga 1 con la riga k; questo equivale a moltiplicare A per una matrice di permutazione Q a sinistra (Esercizio D.2-3). Quindi, possiamo scrivere QA così

$$QA = \begin{pmatrix} a_{k1} & w^{\mathrm{T}} \\ v & A' \end{pmatrix}$$

dove $v = (a_{21}, a_{31}, \ldots, a_{n1})^{\mathrm{T}}$, tranne che a_{11} sostituisce a_{k1}; $w^{\mathrm{T}} = (a_{k2}, a_{k3}, \ldots, a_{kn})$; inoltre A' è una matrice $(n-1) \times (n-1)$. Poiché $a_{k1} \neq 0$, adesso possiamo eseguire le stesse operazioni algebriche che abbiamo eseguito per la fattorizzazione LU, avendo però la garanzia di non dividere per 0:

$$\begin{aligned} QA &= \begin{pmatrix} a_{k1} & w^{\mathrm{T}} \\ v & A' \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^{\mathrm{T}} \\ 0 & A' - vw^{\mathrm{T}}/a_{k1} \end{pmatrix} \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Come visto per la fattorizzazione LU, se A non è singolare, allora anche il complemento di Schur $A' - vw^{\mathrm{T}}/a_{k1}$ non è singolare. Pertanto, possiamo trovare induttivamente una fattorizzazione LUP per esso, con una matrice triangolare inferiore unitaria L', una matrice triangolare superiore U' e una matrice di permutazione P', tali che

$$P'(A' - vw^{\mathrm{T}}/a_{k1}) = L'U'$$

Definiamo

$$P = \begin{pmatrix} 1 & 0 \\ 0 & P' \end{pmatrix} Q$$

che è una matrice di permutazione, perché è il prodotto di due matrici di permutazione (Esercizio D.2-3). Adesso abbiamo

$$\begin{aligned}
PA &= \begin{pmatrix} 1 & 0 \\ 0 & P' \end{pmatrix} QA \\
&= \begin{pmatrix} 1 & 0 \\ 0 & P' \end{pmatrix} \begin{pmatrix} 1 & 0 \\ v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^{\mathrm{T}} \\ 0 & A' - vw^{\mathrm{T}}/a_{k1} \end{pmatrix} \\
&= \begin{pmatrix} 1 & 0 \\ P'v/a_{k1} & P' \end{pmatrix} \begin{pmatrix} a_{k1} & w^{\mathrm{T}} \\ 0 & A' - vw^{\mathrm{T}}/a_{k1} \end{pmatrix} \\
&= \begin{pmatrix} 1 & 0 \\ P'v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^{\mathrm{T}} \\ 0 & P'(A' - vw^{\mathrm{T}}/a_{k1}) \end{pmatrix} \\
&= \begin{pmatrix} 1 & 0 \\ P'v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^{\mathrm{T}} \\ 0 & L'U' \end{pmatrix} \\
&= \begin{pmatrix} 1 & 0 \\ P'v/a_{k1} & L' \end{pmatrix} \begin{pmatrix} a_{k1} & w^{\mathrm{T}} \\ 0 & U' \end{pmatrix} \\
&= LU
\end{aligned}$$

che è la fattorizzazione LUP. Poiché L' è una matrice triangolare inferiore unitaria, anche L lo è, e poiché U' è una matrice triangolare superiore, anche U lo è.

Notate che in questa derivazione, diversamente da quella per la fattorizzazione LU, sia il vettore colonna v/a_{k1} sia il complemento di Schur $A' - vw^{\mathrm{T}}/a_{k1}$ devono essere moltiplicati per la matrice di permutazione P'. Questo è lo pseudocodice per la fattorizzazione LUP:

```
LUP-DECOMPOSITION(A)
 1  n = A.rows
 2  Sia π[1..n] un nuovo array
 3  for i = 1 to n
 4      π[i] = i
 5  for k = 1 to n
 6      p = 0
 7      for i = k to n
 8          if |a_ik| > p
 9              p = |a_ik|
10              k' = i
11      if p == 0
12          error "matrice singolare"
13      scambia π[k] con π[k']
14      for i = 1 to n
15          scambia a_ki con a_k'i
16      for i = k + 1 to n
17          a_ik = a_ik/a_kk
18          for j = k + 1 to n
19              a_ij = a_ij − a_ik a_kj
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

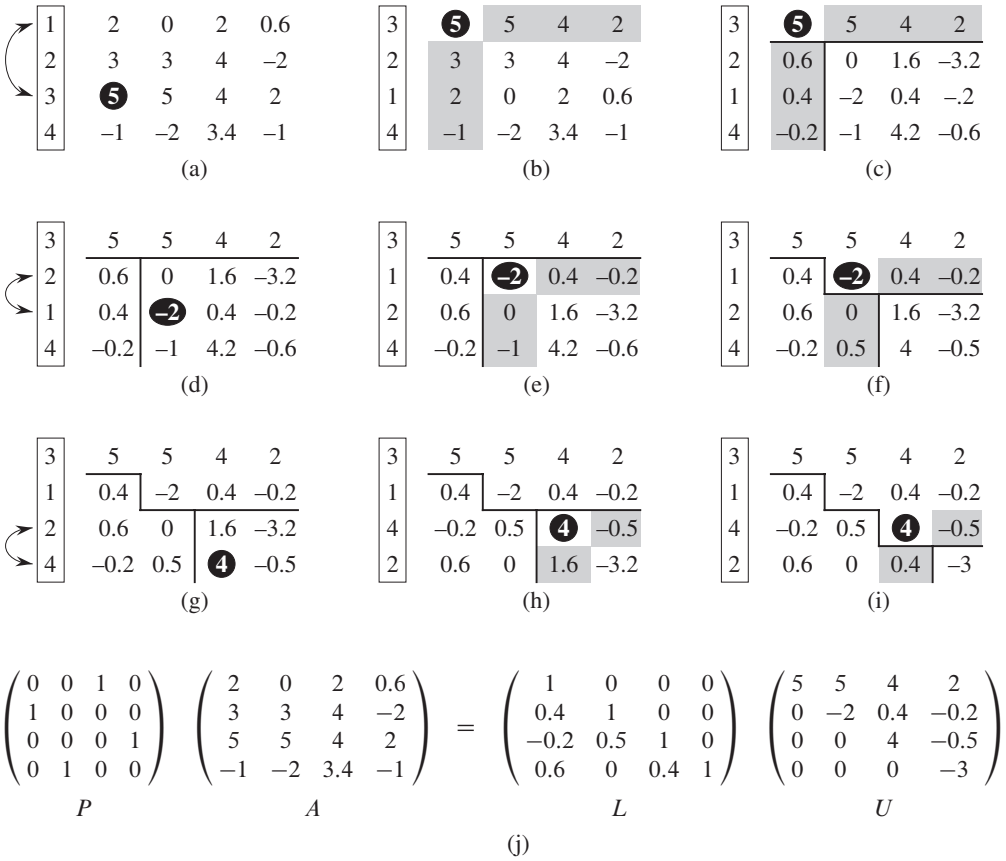


Figura 28.2 Il funzionamento di LUP-DECOMPOSITION. **(a)** La matrice di input A; a sinistra è indicata la permutazione identità delle righe. Il primo passo dell'algoritmo determina che l'elemento 5 dentro il cerchio nero nella terza riga è il pivot per la prima colonna. **(b)** Le righe 1 e 3 vengono scambiate e la permutazione viene aggiornata. La colonna e la riga ombreggiate rappresentano v e w^{T}. **(c)** Il vettore v è sostituito da $v/5$ e la parte in basso a destra della matrice viene aggiornata con il complemento di Schur. Le linee dividono la matrice in tre regioni: gli elementi di U (in alto), gli elementi di L (a sinistra) e gli elementi del complemento di Schur (in basso a destra). **(d)–(f)** Il secondo passo. **(g)–(i)** Il terzo passo. Nessun'altra modifica avviene nel quarto e ultimo passo. **(j)** La fattorizzazione LUP $PA = LU$.

Analogamente a LU-DECOMPOSITION, lo pseudocodice per la fattorizzazione LUP sostituisce la ricorsione con un ciclo di iterazione. Per migliorare l'implementazione diretta della ricorsione, manteniamo dinamicamente la matrice di permutazione P come un array π, dove $\pi[i] = j$ significa che l'i-esima riga di P contiene 1 nella colonna j. Implementiamo anche il codice per calcolare L e U "sul posto" nella matrice A. Pertanto, quando la procedura termina, si ha

$$a_{ij} = \begin{cases} l_{ij} & \text{se } i > j \\ u_{ij} & \text{se } i \leq j \end{cases}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

La Figura 28.2 illustra come LUP-DECOMPOSITION fattorizza una matrice. L'array π viene inizializzato dalle righe 3–4 per rappresentare la permutazione identità. Il ciclo **for** esterno, che inizia nella riga 5, implementa la ricorsione. A ogni esecuzione del ciclo esterno, le righe 6–10 determinano l'elemento $a_{k'k}$ con il valore assoluto più grande tra gli elementi che si trovano nella prima colonna corrente (la colonna k) della matrice $(n-k+1) \times (n-k+1)$, di cui stiamo cercando

la fattorizzazione LU. Se tutti gli elementi nella prima riga corrente sono nulli, le righe 11–12 segnalano che la matrice è singolare. Per scegliere il pivot, scambiamo $\pi[k']$ con $\pi[k]$ nella riga 13 e scambiamo le righe k e k' della matrice A nelle righe 14–15, e questo seleziona l'elemento a_{kk} come pivot. (Vengono scambiate le righe intere, perché nella precedente derivazione del metodo, oltre al complemento di Schur $A' - vw^{\mathrm{T}}/a_{k1}$, anche il vettore colonna v/a_{k1} viene moltiplicato per P'.) Infine, il complemento di Schur viene calcolato dalle righe 16–19 in un modo molto simile a quello delle righe 7–12 di LU-DECOMPOSITION, con la differenza che qui l'operazione è scritta per funzionare "sul posto".

Per la presenza di una struttura con tre cicli annidati, la procedura LUP-DECOMPOSITION ha un tempo di esecuzione pari a $\Theta(n^3)$, che è lo stesso di LU-DECOMPOSITION. Dunque, il pivoting incrementa il tempo di esecuzione al massimo di un fattore costante.

Esercizi

28.1-1

Risolvete l'equazione

$$\begin{pmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ -6 & 5 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 3 \\ 14 \\ -7 \end{pmatrix}$$

applicando il metodo delle sostituzioni in avanti.

28.1-2

Calcolate una fattorizzazione LU della matrice

$$\begin{pmatrix} 4 & -5 & 6 \\ 8 & -6 & 7 \\ 12 & -7 & 12 \end{pmatrix}$$

28.1-3

Risolvete l'equazione

$$\begin{pmatrix} 1 & 5 & 4 \\ 2 & 0 & 3 \\ 5 & 8 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 12 \\ 9 \\ 5 \end{pmatrix}$$

utilizzando una fattorizzazione LUP.

28.1-4

Descrivete la fattorizzazione LUP di una matrice diagonale.

28.1-5

Descrivete la fattorizzazione LUP di una matrice di permutazione A e dimostrate che essa è unica.

28.1-6

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Librerja: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Dimostrate che, per ogni $n \geq 1$, esiste una matrice singolare $n \times n$ che ha una fattorizzazione LU.

28.1-7

Nella procedura LU-DECOMPOSITION è necessario eseguire l'iterazione del ciclo **for** più esterno quando $k = n$? E nella procedura LUP-DECOMPOSITION?

28.2 Inversione di matrici

Sebbene nella pratica le matrici inverse di solito non siano utilizzate per risolvere i sistemi di equazioni lineari, preferendo invece utilizzare delle tecniche numericamente più stabili, come la fattorizzazione LUP, tuttavia in alcuni casi è necessario calcolare le matrici inverse. In questo paragrafo, mostreremo come la fattorizzazione LUP possa essere utilizzata per calcolare una matrice inversa. Inoltre, dimostreremo che la moltiplicazione delle matrici e il calcolo di una matrice inversa sono problemi ugualmente difficili, nel senso che (sotto determinate condizioni tecniche) possiamo utilizzare un algoritmo che risolve un problema per risolvere anche l'altro nello stesso tempo di esecuzione asintotico. Quindi, per invertire una matrice possiamo utilizzare l'algoritmo di Strassen (si veda il Paragrafo 4.2) che moltiplica le matrici. In effetti, il documento originale di Strassen aveva come obiettivo il problema di dimostrare che un sistema di equazioni lineari poteva essere risolto con un metodo più rapido di quello usuale.

Calcolo di una matrice inversa da una fattorizzazione LUP

Supponiamo di avere una fattorizzazione LUP di una matrice A nella forma di tre matrici L, U e P tali che $PA = LU$. Utilizzando la procedura LUP-SOLVE, possiamo risolvere un'equazione della forma $Ax = b$ nel tempo $\Theta(n^2)$. Poiché la fattorizzazione LUP dipende da A, ma non da b, possiamo eseguire LUP-SOLVE su un secondo sistema di equazioni della forma $Ax = b'$ con un tempo aggiuntivo $\Theta(n^2)$. In generale, una volta che abbiamo la fattorizzazione LUP di A, possiamo risolvere, nel tempo $\Theta(kn^2)$, k versioni dell'equazione $Ax = b$ che differiscono soltanto per b.

L'equazione

$$AX = I_n \tag{28.10}$$

può essere vista come un sistema di n equazioni distinte della forma $Ax = b$. Queste equazioni definiscono la matrice X come l'inversa di A. Per essere precisi, indichiamo con X_i l'i-esima colonna di X e ricordiamo che il vettore unità e_i è l'i-esima colonna di I_n. L'equazione (28.10) può essere risolta rispetto a X utilizzando la fattorizzazione LUP di A per risolvere ciascuna delle equazioni

$$AX_i = e_i$$

separatamente rispetto a X_i. Ciascuna delle n colonne X_i può essere calcolata nel tempo $\Theta(n^2)$, e così il calcolo di X dalla fattorizzazione LUP di A richiede un tempo $\Theta(n^3)$. Poiché la fattorizzazione LUP di A può essere calcolata nel tempo $\Theta(n^3)$, l'inversa A^{-1} di una matrice A può essere calcolata nel tempo $\Theta(n^3)$.

Moltiplicazione di matrici e inversione di matrici

Acquistato da Michele Mi[illegible] per uso esclusivo [illegible] © 2010 The McGraw-Hill Education (Italy)

Dimostriamo adesso che l'accelerazione teorica che si ottiene nella moltiplicazione delle matrici si traduce in un'accelerazione nell'inversione delle matrici. In effetti, dimostriamo qualcosa di più forte: l'inversione delle matrici è equivalente alla moltiplicazione delle matrici, nel seguente senso. Se $M(n)$ indica il tempo per moltiplicare due matrici $n \times n$, allora c'è un modo per invertire una matrice $n \times n$ nel tempo $O(M(n))$. Inoltre, se $I(n)$ indica il tempo per invertire una matrice non singolare di dimensioni $n \times n$, allora c'è un modo per moltiplicare due matrici $n \times n$ nel tempo $O(I(n))$. Dimostriamo questi risultati con due teoremi distinti.

Teorema 28.1 (La moltiplicazione non è più difficile dell'inversione)
Se invertiamo una matrice $n \times n$ nel tempo $I(n)$, dove $I(n) = \Omega(n^2)$ e $I(n)$ soddisfa la condizione di regolarità $I(3n) = O(I(n))$, allora possiamo moltiplicare due matrici $n \times n$ nel tempo $O(I(n))$.

Dimostrazione Siano A e B due matrici $n \times n$ di cui vogliamo calcolare il prodotto C. Definiamo la matrice D di dimensioni $3n \times 3n$ come

$$D = \begin{pmatrix} I_n & A & 0 \\ 0 & I_n & B \\ 0 & 0 & I_n \end{pmatrix}$$

La matrice inversa di D è

$$D^{-1} = \begin{pmatrix} I_n & -A & AB \\ 0 & I_n & -B \\ 0 & 0 & I_n \end{pmatrix}$$

Quindi possiamo calcolare il prodotto AB prendendo la sottomatrice $n \times n$ in alto a destra di D^{-1}. Possiamo creare la matrice D nel tempo $\Theta(n^2) = O(I(n))$ e possiamo invertire D nel tempo $O(I(3n)) = O(I(n))$, per la condizione di regolarità su $I(n)$. Dunque abbiamo $M(n) = O(I(n))$. ■

Notate che $I(n)$ soddisfa la condizione di regolarità se $I(n) = \Theta(n^c \lg^d n)$, con $c > 0$ e $d \geq 0$ costanti qualsiasi.

La dimostrazione che l'inversione delle matrici non è più difficile della moltiplicazione delle matrici si basa su alcune proprietà delle matrici simmetriche e definite positive che saranno provate nel Paragrafo 28.3.

Teorema 28.2 (L'inversione non è più difficile della moltiplicazione)
Supponiamo di poter moltiplicare due matrici reali $n \times n$ nel tempo $M(n)$, dove $M(n) = \Omega(n^2)$ ed $M(n)$ soddisfa le due condizioni di regolarità $M(n + k) = O(M(n))$ per ogni k nell'intervallo $0 \leq k \leq n$ e $M(n/2) \leq cM(n)$ per qualche costante $c < 1/2$. Allora è possibile calcolare nel tempo $O(M(n))$ l'inversa di qualsiasi matrice reale non singolare di dimensioni $n \times n$.

Dimostrazione Dimostriamo il teorema per le matrici reali. L'Esercizio 28.2-6 chiede di generalizzare la dimostrazione per le matrici i cui elementi sono numeri complessi. Possiamo assumere che n sia una potenza esatta di 2, in quanto si ha

$$\begin{pmatrix} A & 0 \\ 0 & I_k \end{pmatrix}^{-1} = \begin{pmatrix} A^{-1} & 0 \\ 0 & I_k \end{pmatrix}$$

per qualsiasi $k > 0$. Quindi, scegliendo k in modo che $n + k$ sia una potenza di 2, allarghiamo la matrice a una dimensione che è la successiva potenza di 2 e otteniamo la soluzione cercata A^{-1} dalla soluzione del problema allargato. La prima condizione di regolarità su $M(n)$ garantisce che questo allargamento non provoca un incremento del tempo di esecuzione di oltre un fattore costante.

Acquistato da Michele Michele su Webster il 2022-01-[illegible] Copyright © 2022, Questo materiale è protetto (ed)

Per il momento supponiamo che la matrice A, $n \times n$, sia simmetrica e definita positiva. Dividiamo A in quattro sottomatrici $n/2 \times n/2$:

$$A = \begin{pmatrix} B & C^{\mathrm{T}} \\ C & D \end{pmatrix} \quad \text{e} \quad A^{-1} = \begin{pmatrix} R & T \\ U & V \end{pmatrix} \tag{28.11}$$

Poi, se indichiamo con

$$S = D - CB^{-1}C^{\mathrm{T}} \tag{28.12}$$

il complemento di Schur di A rispetto a B (diremo di più su questa forma del complemento di Schur nel Paragrafo 28.3), otteniamo

$$A^{-1} = \begin{pmatrix} B^{-1} + B^{-1}C^{\mathrm{T}}S^{-1}CB^{-1} & -B^{-1}C^{\mathrm{T}}S^{-1} \\ -S^{-1}CB^{-1} & S^{-1} \end{pmatrix} \tag{28.13}$$

in quanto $AA^{-1} = I_n$, come può essere verificato eseguendo la moltiplicazione delle matrici. Siccome A è simmetrica e definita positiva, per i Lemmi 28.4 e 28.5 (Paragrafo 28.3), anche B ed S sono simmetriche e definite positive. Quindi, per il Lemma 28.3 (Paragrafo 28.3), esistono le inverse B^{-1} ed S^{-1} e, per l'Esercizio D.2-6, B^{-1} ed S^{-1} sono simmetriche di modo che $(B^{-1})^{\mathrm{T}} = B^{-1}$ ed $(S^{-1})^{\mathrm{T}} = S^{-1}$. Possiamo quindi calcolare le sottomatrici R, T, U e V di A^{-1} nel modo seguente, in cui tutte le matrici sono $n/2 \times n/2$:

1. Forma le sottomatrici B, C, C^{T} e D di A.
2. Calcola in modo ricorsivo l'inversa B^{-1} di B.
3. Calcola il prodotto di matrici $W = CB^{-1}$ e poi la sua trasposta W^{T}, che è uguale a $B^{-1}C^{\mathrm{T}}$ (per l'Esercizio D.1-2 e perché $(B^{-1})^{\mathrm{T}} = B^{-1}$).
4. Calcola il prodotto di matrici $X = WC^{\mathrm{T}}$, che è uguale a $CB^{-1}C^{\mathrm{T}}$, e poi la matrice $S = D - X = D - CB^{-1}C^{\mathrm{T}}$.
5. Calcola in modo ricorsivo l'inversa S^{-1} di S e pone V uguale a S^{-1}.
6. Calcola il prodotto di matrici $Y = S^{-1}W$, che è uguale a $S^{-1}CB^{-1}$, e la sua trasposta Y^{T}, che è uguale a $B^{-1}C^{\mathrm{T}}S^{-1}$ (per l'Esercizio D.1-2 e perché $(B^{-1})^{\mathrm{T}} = B^{-1}$ e $(S^{-1})^{\mathrm{T}} = S^{-1}$). Pone T uguale a $-Y^{\mathrm{T}}$ e U uguale a $-Y$.
7. Calcola il prodotto di matrici $Z = W^{\mathrm{T}}Y$, che è uguale a $B^{-1}C^{\mathrm{T}}S^{-1}CB^{-1}$, e pone R uguale a $B^{-1} + Z$.

Dunque, è possibile invertire una matrice $n \times n$ simmetrica e definita positiva invertendo due matrici $n/2 \times n/2$ (B ed S) nei passi 2 e 5, eseguendo quattro moltiplicazioni di matrici $n/2 \times n/2$ nei passi 3, 4, 6 e 7, più un costo aggiuntivo di $O(n^2)$ per estrarre le sottomatrici da A ed eseguire un numero costante di somme, sottrazioni e trasposizioni con queste matrici $n/2 \times n/2$. Si ottiene la seguente ricorrenza

$$\begin{aligned} I(n) &\leq 2I(n/2) + 4M(n/2) + O(n^2) \\ &= 2I(n/2) + \Theta(M(n)) \\ &= O(M(n)) \end{aligned}$$

La seconda riga è vera perché la seconda condizione di regolarità nella definizione del teorema implica che $4M(n/2) \leq 2M(n)$ e abbiamo assunto che $M(n) = \Omega(n^2)$. La terza riga è vera perché la seconda condizione di regolarità ci consente di applicare il caso 3 del teorema dell'esperto (Teorema 4.1).

Acquistato da Michele Mirizzi (michele.mirizzi@gmail.com) - Copia non cedibile - Proprietà letteraria riservata - © 2022 The McGraw-Hill Education (Italy)

Resta da dimostrare che il tempo di esecuzione asintotico per moltiplicare le matrici può essere ottenuto per invertire le matrici quando A è una matrice invertibile, ma non è simmetrica e definita positiva. L'idea di base è che, per qualsiasi matrice non singolare A, la matrice $A^{\mathrm{T}}A$ è simmetrica (per l'Esercizio D.1-2) e

definita positiva (per il Teorema D.6). Pertanto, il trucco consiste nel ridurre il problema di invertire A nel problema di invertire $A^{\mathrm{T}}A$.

La riduzione si basa sull'osservazione che quando A è una matrice non singolare $n \times n$, si ha

$$A^{-1} = (A^{\mathrm{T}}A)^{-1}A^{\mathrm{T}}$$

perché $((A^{\mathrm{T}}A)^{-1}A^{\mathrm{T}})A = (A^{\mathrm{T}}A)^{-1}(A^{\mathrm{T}}A) = I_n$ e perché una matrice inversa è unica. Quindi, possiamo calcolare A^{-1} moltiplicando prima A^{T} per A per ottenere $A^{\mathrm{T}}A$, poi invertendo la matrice simmetrica e definita positiva $A^{\mathrm{T}}A$ utilizzando il precedente algoritmo divide et impera, e infine moltiplicando il risultato per A^{T}. Ciascuno di questi tre passi richiede un tempo $O(M(n))$ e, quindi, qualsiasi matrice non singolare con elementi reali può essere invertita nel tempo $O(M(n))$. ■

La dimostrazione del Teorema 28.2 suggerisce un metodo per risolvere l'equazione $Ax = b$ utilizzando la fattorizzazione LU senza pivoting, se la matrice A non è singolare. Moltiplichiamo entrambi i lati dell'equazione per A^{T}, ottenendo $(A^{\mathrm{T}}A)x = A^{\mathrm{T}}b$. Questa trasformazione non influisce sulla soluzione x, perché A^{T} è invertibile; quindi possiamo fattorizzare la matrice $A^{\mathrm{T}}A$ simmetrica e definita positiva calcolando una fattorizzazione LU. Poi utilizziamo le sostituzioni in avanti e all'indietro per trovare x con il lato destro $A^{\mathrm{T}}b$.

Sebbene questo metodo sia teoricamente corretto, in pratica la procedura LUP-DECOMPOSITION funziona molto meglio. La fattorizzazione LUP richiede un minor numero di operazioni aritmetiche per un fattore costante e ha proprietà numeriche migliori.

Esercizi

28.2-1

Sia $M(n)$ il tempo richiesto per moltiplicare delle matrici $n \times n$ e sia $S(n)$ il tempo richiesto per elevare al quadrato una matrice $n \times n$. Dimostrate che la moltiplicazione e l'elevazione al quadrato delle matrici hanno essenzialmente la stessa difficoltà: un algoritmo che moltiplica le matrici nel tempo $M(n)$ implica un algoritmo che eleva al quadrato una matrice nel tempo $O(M(n))$, e un algoritmo che eleva al quadrato una matrice nel tempo $S(n)$ implica un algoritmo che moltiplica le matrici nel tempo $O(S(n))$.

28.2-2

Sia $M(n)$ il tempo richiesto per moltiplicare delle matrici $n \times n$ e sia $L(n)$ il tempo per calcolare la fattorizzazione LUP di una matrice $n \times n$. Dimostrate che la moltiplicazione delle matrici e il calcolo della fattorizzazione LUP hanno essenzialmente la stessa difficoltà: un algoritmo che moltiplica le matrici nel tempo $M(n)$ implica un algoritmo che calcola la fattorizzazione LUP nel tempo $O(M(n))$, e un algoritmo che calcola la fattorizzazione LUP nel tempo $L(n)$ implica un algoritmo che moltiplica le matrici nel tempo $O(L(n))$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

28.2-3

Sia $M(n)$ il tempo richiesto per moltiplicare delle matrici $n \times n$ e sia $D(n)$ il tempo richiesto per calcolare il determinante di una matrice $n \times n$. Dimostrate che la moltiplicazione delle matrici e il calcolo del determinante hanno essenzialmente la stessa difficoltà: un algoritmo che moltiplica le matrici nel tempo $M(n)$ implica

un algoritmo che calcola il determinante nel tempo $O(M(n))$, e un algoritmo che calcola il determinante nel tempo $D(n)$ implica un algoritmo che moltiplica le matrici nel tempo $O(D(n))$.

28.2-4
Sia $M(n)$ il tempo richiesto per moltiplicare delle matrici booleane $n \times n$ e sia $T(n)$ il tempo per calcolare la chiusura transitiva delle matrici booleane $n \times n$ (vedere il Paragrafo 25.3). Dimostrate che un algoritmo che moltiplica le matrici booleane nel tempo $M(n)$ implica un algoritmo che calcola la chiusura transitiva nel tempo $O(M(n) \lg n)$, e un algoritmo che calcola la chiusura transitiva nel tempo $T(n)$ implica un algoritmo che moltiplica le matrici nel tempo $O(T(n))$.

28.2-5
L'algoritmo di inversione delle matrici basato sul Teorema 28.2 funziona quando gli elementi delle matrici appartengono al campo dei numeri interi modulo 2? Spiegate la risposta.

28.2-6 ⋆
Generalizzate l'algoritmo di inversione delle matrici del Teorema 28.2 per gestire le matrici di numeri complessi; dimostrate che la vostra generalizzazione funziona correttamente. (*Suggerimento:* anziché la trasposta di A, utilizzate la ***matrice trasposta coniugata*** A^*, che si ottiene dalla trasposta di A sostituendo ogni elemento con il suo complesso coniugato. Anziché le matrici simmetriche, considerate le ***matrici hermitiane***, che sono matrici A tali che $A = A^*$.)

28.3 Matrici simmetriche e definite positive e minimi quadrati

Le matrici simmetriche e definite positive hanno molte proprietà interessanti; per esempio, non sono singolari ed è possibile calcolarne la fattorizzazione LU senza temere di dividere per 0. In questo paragrafo, definiremo molte altre importanti proprietà delle matrici simmetriche e definite positive e presenteremo come applicazione il calcolo di una curva di approssimazione ai minimi quadrati.

La prima proprietà che dimostriamo è, probabilmente, la più importante.

Lemma 28.3
Ogni matrice definita positiva è non singolare.

Dimostrazione Supponiamo che una matrice A sia singolare. Allora, per il Corollario D.3, esiste un vettore x non nullo tale che $Ax = 0$. Quindi, $x^{\mathrm{T}}Ax = 0$, e allora A non può essere una matrice definita positiva. ■

La dimostrazione che la fattorizzazione LU viene eseguita su una matrice A simmetrica e definita positiva senza dividere per 0 è più complessa. Iniziamo dimostrando le proprietà di particolari sottomatrici di A. Definiamo ***sottomatrice principale*** di A di ordine k la matrice A_k che è formata dall'intersezione delle prime k righe e delle prime k colonne di A.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950301 6-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Lemma 28.4
Se A è una matrice simmetrica e definita positiva, allora ogni sottomatrice principale di A è simmetrica e definita positiva.

Dimostrazione È ovvio che ogni sottomatrice principale A_k è simmetrica. Per dimostrare che A_k è definita positiva, supponiamo che non lo sia, poi arriveremo a una contraddizione. Se A_k non è definita positiva, allora esiste un vettore $x_k \neq 0$ di dimensione k tale che $x_k^{\mathrm{T}} A x_k \leq 0$. Se A è una matrice $n \times n$ e

$$A = \begin{pmatrix} A_k & B^{\mathrm{T}} \\ B & C \end{pmatrix} \tag{28.14}$$

per le sottomatrici B (di dimensione $(n-k) \times k$) e C (di dimensione $(n-k) \times (n-k)$), definiamo il vettore $x = (\, x_k^{\mathrm{T}} \quad 0 \,)^{\mathrm{T}}$ di dimensione n, dove ci sono $n-k$ elementi 0 che seguono x_k. Allora, si ha

$$\begin{aligned} x^{\mathrm{T}} A x &= (\, x_k^{\mathrm{T}} \quad 0 \,) \begin{pmatrix} A_k & B^{\mathrm{T}} \\ B & C \end{pmatrix} \begin{pmatrix} x_k \\ 0 \end{pmatrix} \\ &= (\, x_k^{\mathrm{T}} \quad 0 \,) \begin{pmatrix} A_k x_k \\ B x_k \end{pmatrix} \\ &= x_k^{\mathrm{T}} A_k x_k \\ &\leq 0 \end{aligned}$$

Questo contraddice l'ipotesi che A sia una matrice definita positiva. ■

Adesso esaminiamo alcune proprietà essenziali del complemento di Schur. Sia A una matrice simmetrica e definita positiva; sia A_k una sottomatrice principale di A di dimensione $k \times k$. Dividiamo A secondo l'equazione (28.14). Generalizziamo l'equazione (28.9) per definire il ***complemento di Schur*** di A rispetto ad A_k in questo modo:

$$S = C - B A_k^{-1} B^{\mathrm{T}} \tag{28.15}$$

(Per il Lemma 28.4, A_k è simmetrica e definita positiva; quindi A_k^{-1} esiste per il Lemma 28.3 e il complemento S è ben definito.) Notate che la precedente definizione (28.9) del complemento di Schur è coerente con la definizione (28.15), se si pone $k = 1$.

Il prossimo lemma dimostra che i complementi di Schur di matrici simmetriche e definite positive sono matrici simmetriche e definite positive. Questo risultato è stato utilizzato nel Teorema 28.2 e il suo corollario serve a dimostrare la correttezza della fattorizzazione LU per matrici simmetriche e definite positive.

Lemma 28.5 (Lemma del complemento di Schur)
Se A è una matrice simmetrica e definita positiva e A_k è una sottomatrice principale di A di ordine k, allora il complemento di Schur di A rispetto ad A_k è una matrice simmetrica e definita positiva.

Dimostrazione Poiché A è simmetrica, anche la sottomatrice C è simmetrica. Per l'Esercizio D.2-6, il prodotto $B A_k^{-1} B^{\mathrm{T}}$ è una matrice simmetrica e, per l'Esercizio D.1-1, S è una matrice simmetrica.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 - Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Resta da dimostrare che S è una matrice definita positiva. Consideriamo la partizione di A data nell'equazione (28.14). Per qualsiasi vettore x non nullo, si ha $x^{\mathrm{T}} A x > 0$ per l'ipotesi che A sia una matrice definita positiva. Dividiamo x in due sottovettori y e z compatibili, rispettivamente, con A_k e C. Poiché A_k^{-1} esiste, si ha:

$$\begin{aligned} x^{\mathrm{T}}Ax &= \begin{pmatrix} y^{\mathrm{T}} & z^{\mathrm{T}} \end{pmatrix} \begin{pmatrix} A_k & B^{\mathrm{T}} \\ B & C \end{pmatrix} \begin{pmatrix} y \\ z \end{pmatrix} \\ &= \begin{pmatrix} y^{\mathrm{T}} & z^{\mathrm{T}} \end{pmatrix} \begin{pmatrix} A_k y + B^{\mathrm{T}} z \\ By + Cz \end{pmatrix} \\ &= y^{\mathrm{T}} A_k y + y^{\mathrm{T}} B^{\mathrm{T}} z + z^{\mathrm{T}} By + z^{\mathrm{T}} Cz \\ &= (y + A_k^{-1} B^{\mathrm{T}} z)^{\mathrm{T}} A_k (y + A_k^{-1} B^{\mathrm{T}} z) + z^{\mathrm{T}} (C - BA_k^{-1} B^{\mathrm{T}}) z \end{aligned} \tag{28.16}$$

per magia delle matrici (eseguite le moltiplicazioni per verifica). Quest'ultima equazione equivale a "completare il quadrato" della forma quadratica (vedere l'Esercizio 28.3-2).

Poiché $x^{\mathrm{T}}Ax > 0$ è vera per qualsiasi vettore x non nullo, prendiamo un sottovettore z non nullo e poi scegliamo $y = -A_k^{-1} B^{\mathrm{T}} z$, che elimina il primo termine dell'equazione (28.16), lasciando il solo termine

$$z^{\mathrm{T}} (C - BA_k^{-1} B^{\mathrm{T}}) z = z^{\mathrm{T}} Sz$$

come valore dell'espressione. Per ogni $z \neq 0$, si ha quindi $z^{\mathrm{T}} Sz = x^{\mathrm{T}} Ax > 0$, e pertanto S è una matrice definita positiva. ■

Corollario 28.6
La fattorizzazione LU di una matrice simmetrica e definita positiva non provoca divisioni per 0.

Dimostrazione Sia A una matrice simmetrica e definita positiva. Dimostreremo qualcosa di più forte dell'asserzione del corollario: ogni pivot è strettamente positivo. Il primo pivot è a_{11}. Sia e_1 il primo vettore unità, dal quale si ottiene $a_{11} = e_1^{\mathrm{T}} Ae_1 > 0$. Poiché il primo passo della fattorizzazione LU produce il complemento di Schur di A rispetto ad $A_1 = (a_{11})$, il Lemma 28.5 implica che tutti i pivot siano positivi per induzione. ■

Approssimazione ai minimi quadrati

Trovare le curve che approssimano un dato insieme di punti è un'applicazione importante delle matrici simmetriche e definite positive. Supponiamo di avere un insieme di m punti

$$(x_1, y_1), (x_2, y_2), \ldots, (x_m, y_m)$$

Sappiamo che i valori y_i sono soggetti a errori di misura. Vogliamo determinare una funzione $F(x)$ tale che

$$y_i = F(x_i) + \eta_i \tag{28.17}$$

per $i = 1, 2, \ldots, m$, dove gli errori di approssimazione η_i sono piccoli. La forma della funzione F dipende dal problema che si sta esaminando. Qui assumiamo che questa funzione abbia la forma di una sommatoria linearmente ponderata:

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$F(x) = \sum_{j=1}^{n} c_j f_j(x)$$

dove il numero degli addendi n e le specifiche ***funzioni di base*** f_j vengono scelti basandosi sulla conoscenza del problema in esame. Una tipica scelta è $f_j(x) = x^{j-1}$, che significa che

$$F(x) = c_1 + c_2x + c_3x^2 + \cdots + c_nx^{n-1}$$

è un polinomio di grado $n - 1$ in x. Quindi, dati m punti $(x_1, y_1), (x_2, y_2), \ldots, (x_m, y_m)$, vogliamo calcolare n coefficienti $c_1, c_2, \ldots, c_n$ che minimizzano gli errori di approssimazione $\eta_1, \eta_2, \ldots, \eta_m$.

Scegliendo $n = m$, possiamo calcolare *esattamente* ciascun termine y_i nell'equazione (28.17). Tuttavia, una funzione F di grado così elevato "interpola il rumore" oltre ai dati e, in generale, fornisce risultati insoddisfacenti quando è utilizzata per prevedere y per valori di x diversi da quelli dati. Di solito è meglio scegliere n molto più piccolo di m e sperare che, scegliendo bene i coefficienti c_j, riusciamo a ottenere una funzione F che catturi le regolarità significative dei punti trascurando il rumore. Esistono alcuni principi teorici per scegliere n, tuttavia la loro trattazione esula dagli obiettivi di questo libro. In ogni caso, una volta scelto n, si perviene a un sistema sovradeterminato di equazioni di cui vogliamo approssimare la soluzione. Vediamo come tutto questo può essere realizzato. Indichiamo con

$$A = \begin{pmatrix} f_1(x_1) & f_2(x_1) & \ldots & f_n(x_1) \\ f_1(x_2) & f_2(x_2) & \ldots & f_n(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ f_1(x_m) & f_2(x_m) & \ldots & f_n(x_m) \end{pmatrix}$$

la matrice dei valori delle funzioni di base nei punti dati; ovvero $a_{ij} = f_j(x_i)$. Sia $c = (c_k)$ il vettore, di dimensione n, dei coefficienti che cerchiamo. Allora

$$Ac = \begin{pmatrix} f_1(x_1) & f_2(x_1) & \ldots & f_n(x_1) \\ f_1(x_2) & f_2(x_2) & \ldots & f_n(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ f_1(x_m) & f_2(x_m) & \ldots & f_n(x_m) \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{pmatrix} =$$

$$= \begin{pmatrix} F(x_1) \\ F(x_2) \\ \vdots \\ F(x_m) \end{pmatrix}$$

è il vettore, di dimensione m, dei "valori previsti" per y. Quindi

$$\eta = Ac - y$$

è il vettore, di dimensione m, degli ***errori di approssimazione***.

Per minimizzare gli errori di approssimazione, scegliamo di minimizzare la norma del vettore degli errori η, che fornisce una ***soluzione ai minimi quadrati***, in quanto

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\|\eta\| = \left(\sum_{i=1}^{m} \eta_i^2 \right)^{1/2}$$

Poiché

$$\|\eta\|^2 = \|Ac - y\|^2 = \sum_{i=1}^{m} \left(\sum_{j=1}^{n} a_{ij}c_j - y_i \right)^2$$

possiamo minimizzare $\|\eta\|$ derivando $\|\eta\|^2$ rispetto a ciascun c_k e poi ponendo il risultato a 0:

$$\frac{d\ \|\eta\|^2}{dc_k} = \sum_{i=1}^{m} 2\left(\sum_{j=1}^{n} a_{ij}c_j - y_i\right) a_{ik} = 0 \tag{28.18}$$

Le n equazioni (28.18) per $k = 1, 2, \ldots, n$ sono equivalenti alla singola equazione matriciale

$$(Ac - y)^{\mathrm{T}} A = 0$$

ovvero, per l'Esercizio D.1-2, a

$$A^{\mathrm{T}}(Ac - y) = 0$$

che implica

$$A^{\mathrm{T}}Ac = A^{\mathrm{T}}y \tag{28.19}$$

In statistica questa è detta ***equazione normale***. La matrice $A^{\mathrm{T}}A$ è simmetrica per l'Esercizio D.1-2; inoltre, se A ha rango di colonna massimo, allora per il Teorema D.6, $A^{\mathrm{T}}A$ è anche definita positiva. Quindi, $(A^{\mathrm{T}}A)^{-1}$ esiste e la soluzione dell'equazione (28.19) è

$$\begin{aligned} c &= \left((A^{\mathrm{T}}A)^{-1}A^{\mathrm{T}}\right) y \\ &= A^{+}y \end{aligned} \tag{28.20}$$

dove la matrice $A^{+} = ((A^{\mathrm{T}}A)^{-1}A^{\mathrm{T}})$ è detta ***pseudoinversa*** della matrice A. La pseudoinversa è una naturale generalizzazione del concetto di matrice inversa al caso in cui A non sia quadrata (confrontate l'equazione (28.20) come soluzione approssimata di $Ac = y$ con la soluzione $A^{-1}b$ come soluzione esatta di $Ax = b$).

Come esempio di approssimazione ai minimi quadrati, supponiamo di avere cinque punti nel piano

$$\begin{aligned} (x_1, y_1) &= (-1, 2) \\ (x_2, y_2) &= (1, 1) \\ (x_3, y_3) &= (2, 1) \\ (x_4, y_4) &= (3, 0) \\ (x_5, y_5) &= (5, 3) \end{aligned}$$

illustrati dai punti neri nella Figura 28.3. Vogliamo approssimare questi punti con un polinomio di secondo grado

$$F(x) = c_1 + c_2 x + c_3 x^2$$

Iniziamo dalla matrice dei valori delle funzioni di base

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$A = \begin{pmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \\ 1 & x_4 & x_4^2 \\ 1 & x_5 & x_5^2 \end{pmatrix} = \begin{pmatrix} 1 & -1 & 1 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 3 & 9 \\ 1 & 5 & 25 \end{pmatrix}$$

la cui pseudoinversa è

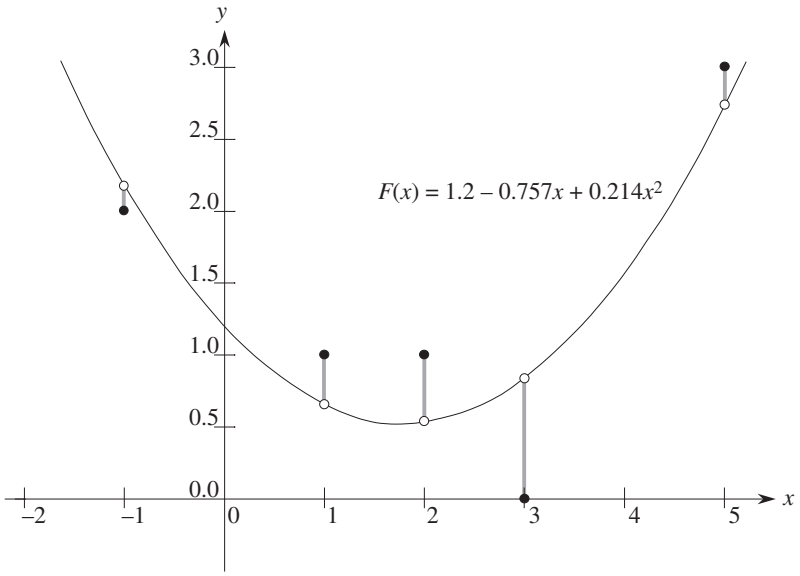


Figura 28.3
Approssimazione ai minimi quadrati di un insieme di cinque punti $\{(-1, 2), (1, 1), (2, 1), (3, 0), (5, 3)\}$ con un polinomio di secondo grado. I cerchi neri rappresentano i punti noti; i cerchi bianchi rappresentano i corrispondenti valori stimati con il polinomio $F(x) = 1.2 - 0.757x + 0.214x^2$, il polinomio di secondo grado che minimizza la somma degli errori quadratici. L'errore per ogni punto è indicato da un segmento grigio.

$$A^+ = \begin{pmatrix} 0.500 & 0.300 & 0.200 & 0.100 & -0.100 \\ -0.388 & 0.093 & 0.190 & 0.193 & -0.088 \\ 0.060 & -0.036 & -0.048 & -0.036 & 0.060 \end{pmatrix}$$

Moltiplicando y per A^+, otteniamo il vettore dei coefficienti

$$c = \begin{pmatrix} 1.200 \\ -0.757 \\ 0.214 \end{pmatrix}$$

che corrisponde al polinomio di secondo grado

$$F(x) = 1.200 - 0.757x + 0.214x^2$$

Questa funzione rappresenta la migliore approssimazione quadratica dei punti dati, nel senso dei minimi quadrati.

Nella pratica, l'equazione normale (28.19) si risolve moltiplicando y per A^{T} e poi trovando una fattorizzazione LU di $A^{\mathrm{T}}A$. Se A ha rango massimo, si ha la garanzia che la matrice $A^{\mathrm{T}}A$ non è singolare, perché è simmetrica e definita positiva (vedere l'Esercizio D.1-2 e il Teorema D.6).

Esercizi

28.3-1
Dimostrate che ogni elemento diagonale di una matrice simmetrica e definita positiva è positivo.

28.3-2
Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)
Sia $A = \begin{pmatrix} a & b \\ b & c \end{pmatrix}$ una matrice 2×2 simmetrica e definita positiva. Dimostrate che il suo determinante $ac - b^2$ è positivo "completando il quadrato" in modo analogo a quanto fatto nella dimostrazione del Lemma 28.5.

28.3-3
Dimostrate che l'elemento massimo di una matrice simmetrica e definita positiva si trova nella diagonale.

28.3-4
Dimostrate che il determinante di qualsiasi sottomatrice principale di una matrice simmetrica e definita positiva è positivo.

28.3-5
Indicate con A_k la sottomatrice principale di ordine k di una matrice A simmetrica e definita positiva. Dimostrate che $\det(A_k)/\det(A_{k-1})$ è il k-esimo pivot durante la fattorizzazione LU, dove per convenzione $\det(A_0) = 1$.

28.3-6
Trovate la funzione della forma

$$F(x) = c_1 + c_2 x \lg x + c_3 e^x$$

che sia la migliore approssimazione ai minimi quadrati dei punti

$$(1, 1), (2, 1), (3, 3), (4, 8)$$

28.3-7
Dimostrate che la pseudoinversa A^+ soddisfa le seguenti equazioni:

$$\begin{aligned} AA^+A &= A \\ A^+AA^+ &= A^+ \\ (AA^+)^{\mathrm{T}} &= AA^+ \\ (A^+A)^{\mathrm{T}} &= A^+A \end{aligned}$$

Problemi

28-1 Sistemi tridiagonali di equazioni lineari
Considerate la seguente matrice tridiagonale:

$$A = \begin{pmatrix} 1 & -1 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & 2 \end{pmatrix}$$

a. Calcolate una fattorizzazione LU di A.

b. Risolvete l'equazione $Ax = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \end{pmatrix}^{\mathrm{T}}$ utilizzando il metodo delle sostituzioni in avanti e all'indietro.

c. Calcolate la matrice inversa di A.

d. Dimostrate che per qualsiasi matrice A, $n \times n$, tridiagonale, simmetrica e definita positiva e per qualsiasi vettore b di dimensione n, l'equazione $Ax = b$ può essere risolta nel tempo $O(n)$ eseguendo una fattorizzazione LU. Dimostrate che qualsiasi metodo basato sul calcolo di A^{-1} è asintoticamente più costoso nel caso peggiore.

Acquistato da Michele Michelo Vellucci il 2025-07-31 10:07:17 (Indirizzo IP: 95.250.16.47) 2025 McGraw-Hill Education (Italy)

e. Dimostrate che per qualsiasi matrice A, $n \times n$, non singolare e tridiagonale e per qualsiasi vettore b di dimensione n, l'equazione $Ax = b$ può essere risolta nel tempo $O(n)$ eseguendo una fattorizzazione LUP.

28-2 Interpolazione con curve spline

Un metodo pratico per interpolare un insieme di punti con una curva consiste nell'utilizzare le ***spline cubiche***. Dato un insieme $\{(x_i, y_i) : i = 0, 1, \ldots, n\}$ di $n + 1$ coppie di coordinate di punti, dove $x_0 < x_1 < \cdots < x_n$, vogliamo adattare ai punti una curva $f(x)$ formata da porzioni consecutive di curve cubiche (spline). Ovvero la curva $f(x)$ è composta da n polinomi cubici $f_i(x) = a_i + b_i x + c_i x^2 + d_i x^3$ per $i = 0, 1, \ldots, n-1$, dove se x è compreso nell'intervallo $x_i \le x \le x_{i+1}$, allora il valore della curva è dato da $f(x) = f_i(x - x_i)$. I punti x_i in cui i polinomi cubici "si incollano" sono detti ***nodi***. Per semplicità, assumeremo che $x_i = i$ per $i = 0, 1, \ldots, n$.

Per garantire la continuità della funzione $f(x)$, richiediamo che

$$\begin{aligned} f(x_i) &= f_i(0) = y_i \\ f(x_{i+1}) &= f_i(1) = y_{i+1} \end{aligned}$$

per $i = 0, 1, \ldots, n-1$. Per garantire che $f(x)$ abbia un andamento sufficientemente regolare, richiediamo anche la continuità della derivata prima in ogni nodo:

$$f'(x_{i+1}) = f_i'(1) = f_{i+1}'(0)$$

per $i = 0, 1, \ldots, n-1$.

a. Supponete di conoscere non soltanto le coppie delle coordinate dei punti $\{(x_i, y_i)\}$, per $i = 0, 1, \ldots, n$, ma anche la derivata prima $D_i = f'(x_i)$ in ciascun nodo. Esprimete ciascun coefficiente a_i, b_i, c_i e d_i in funzione dei valori y_i, y_{i+1}, D_i e D_{i+1} (ricordiamo che $x_i = i$). Con quale velocità possono essere calcolati i $4n$ coefficienti dalle coppie di coordinate e dalle derivate prime?

Resta il problema di come scegliere le derivate prime di $f(x)$ nei nodi. Un metodo è quello di richiedere che le derivate seconde siano continue nei nodi:

$$f''(x_{i+1}) = f_i''(1) = f_{i+1}''(0)$$

per $i = 0, 1, \ldots, n-1$. Per il primo e l'ultimo nodo, supponiamo che $f''(x_0) = f_0''(0) = 0$ e $f''(x_n) = f_{n-1}''(1) = 0$; queste ipotesi rendono $f(x)$ una ***spline cubica naturale***.

b. Utilizzate i vincoli di continuità sulla derivata seconda per dimostrare che, per $i = 1, 2, \ldots, n-1$, si ha

$$D_{i-1} + 4D_i + D_{i+1} = 3(y_{i+1} - y_{i-1}) \tag{28.21}$$

c. Dimostrate che

$$2D_0 + D_1 = 3(y_1 - y_0) \tag{28.22}$$

$$D_{n-1} + 2D_n = 3(y_n - y_{n-1}) \tag{28.23}$$

d. Riscrivete le equazioni (28.21)–(28.23) come un'equazione matriciale con il vettore delle incognite $D = \langle D_0, D_1, \ldots, D_n \rangle$. Quali caratteristiche ha la matrice nella vostra equazione?

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950316-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

e. Dimostrate che un insieme di $n + 1$ coppie di coordinate di punti può essere interpolato con una spline cubica naturale nel tempo $O(n)$ (vedere il Problema 28-1).

f. Spiegate come calcolare una spline cubica naturale che interpola un insieme di $n + 1$ punti (x_i, y_i) che soddisfano $x_0 < x_1 < \cdots < x_n$, anche quando x_i non è necessariamente uguale a i. Quale equazione matriciale deve essere risolta e qual è la velocità di esecuzione del vostro algoritmo?

Note

Ci sono molti testi eccellenti che trattano il calcolo numerico e scientifico in modo molto più dettagliato di quanto è stato fatto in questo capitolo. Fra i più interessanti citiamo: George e Liu [133], Golub e Van Loan [145], Press, Teukolsky, Vetterling e Flannery [284, 285] e Strang [324, 325].

Golub e Van Loan [145] esaminano il problema della stabilità numerica. Spiegano perché $\det(A)$ non è necessariamente un buon indicatore della stabilità di una matrice A; in alternativa, propongono di utilizzare $\|A\|_\infty \|A^{-1}\|_\infty$, dove $\|A\|_\infty = \max_{1 \le i \le n} \sum_{j=1}^{n} |a_{ij}|$. Essi spiegano anche come trovare questo valore senza calcolare effettivamente A^{-1}.

Il processo di eliminazione di Gauss, sul quale si basano le fattorizzazioni LU e LUP, è stato il primo metodo sistematico per risolvere i sistemi di equazioni lineari. È stato anche uno dei primi algoritmi numerici. Sebbene fosse già conosciuto, la sua scoperta è comunemente attribuita a C. F. Gauss (1777–1855). Nel suo famoso articolo [326], Strassen dimostrò anche che una matrice $n \times n$ può essere invertita nel tempo $O(n^{\lg 7})$. Winograd [359] è stato il primo a dimostrare che la moltiplicazione delle matrici non è più difficile dell'inversione delle matrici; la dimostrazione inversa è dovuta ad Aho, Hopcroft e Ullman [5].

Un altro importante metodo di fattorizzazione delle matrici è il ***metodo SVD*** (Singular Value Decomposition) o ***fattorizzazione agli autovalori***. In questo metodo, una matrice A, di dimensioni $m \times n$, viene fattorizzata in $A = Q_1 \Sigma Q_2^{\mathrm{T}}$, dove Σ è una matrice $m \times n$ con valori non nulli soltanto nella diagonale, Q_1 è matrice $m \times m$ con colonne mutuamente ortonormali e Q_2 è una matrice $n \times n$, anch'essa con colonne mutuamente ortonormali. Due vettori sono ***ortonormali*** se il loro prodotto interno è 0 e ciascun vettore ha norma 1. Il metodo SVD è trattato bene nei libri di Strang [324, 325] e Golub e Van Loan [145].

Strang [325] ha fatto un'eccellente presentazione delle matrici simmetriche e definite positive e dell'algebra lineare in generale.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

29 Programmazione lineare

29.1 Introduzione

Molti problemi possono essere formulati come processi per massimizzare o minimizzare un obiettivo, avendo a disposizione risorse limitate e rispettando contemporaneamente più vincoli. Se l'obiettivo può essere definito come una funzione lineare di alcune variabili e se i vincoli sulle risorse possono essere specificati come relazioni di uguaglianza o disuguaglianza di tali variabili, allora abbiamo un ***problema di programmazione lineare***. I programmi lineari si presentano in numerose applicazioni pratiche. Iniziamo studiando un problema di strategie elettorali.

Un problema politico

Supponete di candidarvi alle prossime elezioni politiche. La vostra circoscrizione elettorale ha tre tipi di aree: urbana, periferica e rurale. Queste aree hanno, rispettivamente, 100 000, 200 000 e 50 000 elettori. Per poter governare efficacemente, vorreste ottenere la maggioranza dei voti in ciascuna di queste aree. La vostra onorabilità non vi consente di supportare programmi politici in cui non credete. Tuttavia, sapete che determinate questioni sono particolarmente efficaci per ottenere consensi in alcune zone. I punti principali del vostro programma politico sono la costruzione di nuove strade, il controllo delle armi, i contributi alle aziende agricole e una tassa sui carburanti con lo scopo di migliorare il traffico cittadino. Secondo una ricerca del vostro staff pubblicitario, è possibile stimare quanti voti potete guadagnare o perdere in ogni segmento di popolazione spendendo 1000 euro per pubblicizzare ciascuno di questi punti. Queste informazioni sono raccolte nella tabella della Figura 29.1, dove i valori indicano le migliaia di voti urbani, periferici o rurali che potreste ottenere investendo 1000 euro per pubblicizzare ciascun punto del programma politico; i valori negativi indicano i voti che perdereste. Il vostro compito consiste nel trovare la quantità minima di denaro da investire per ottenere 50 000 voti urbani, 100 000 voti periferici e 25 000 voti rurali.

Procedendo per tentativi, è possibile definire una strategia che consente di ottenere il numero di voti richiesti, ma una tale strategia potrebbe non essere la più economica. Per esempio, supponete di investire 20 000 euro per pubblicizzare la costruzione delle strade, 0 euro per il controllo delle armi, 4 000 euro per i contributi alle aziende agricole e 9 000 euro per la tassa sui carburanti. In questo caso, otterreste $20(-2) + 0(8) + 4(0) + 9(10) = 50$ mila voti urbani, $20(5) + 0(2) + 4(0) + 9(0) = 100$ mila voti periferici e $20(3)+0(-5)+4(10)+9(-2) = 82$ mila voti rurali. Otterreste il numero esatto dei voti richiesti per l'area urbana e la periferia e più voti di quelli richiesti nell'area rurale. (In effetti,

Acquistato da Michele Michele su Webster il 2022-07-27 22:01:10. Opera protetta da diritto d'autore e non riproducibile in alcuna forma senza autorizzazione.

Punti del programma politico	Area: urbana	periferica	rurale
Costruire strade	−2	5	3
Controllo delle armi	8	2	−5
Contributi alle aziende agricole	0	0	10
Tassa sui carburanti	10	0	−2

Figura 29.1 Gli effetti dei vari punti del programma politico sugli elettori. Ogni valore della tabella indica le migliaia di voti urbani, periferici o rurali che si otterrebbero investendo 1000 euro per pubblicizzare ciascun punto del programma politico. I valori negativi indicano i voti che si perderebbero.

nell'area rurale, otterreste più voti dei votanti!) Per ottenere questi voti, avreste investito in pubblicità $20 + 0 + 4 + 9 = 33$ mila euro.

Naturalmente, vi chiederete se tale strategia sia la migliore possibile. Ovvero, avreste potuto raggiungere gli obiettivi spendendo meno in pubblicità? Facendo altri tentativi, potreste ottenere la risposta a questa domanda, ma sarebbe meglio disporre di un metodo sistematico per rispondere a simili domande. A tal fine, formuliamo matematicamente questa domanda. Introduciamo 4 variabili:

- x_1 rappresenta la spesa (in migliaia di euro) per pubblicizzare la costruzione delle strade.
- x_2 rappresenta la spesa (in migliaia di euro) per pubblicizzare il controllo delle armi.
- x_3 rappresenta la spesa (in migliaia di euro) per pubblicizzare i contributi alle aziende agricole.
- x_4 rappresenta la spesa (in migliaia di euro) per pubblicizzare la tassa sui carburanti.

Possiamo scrivere il requisito che vogliamo ottenere almeno 50 000 voti urbani così:

$$-2x_1 + 8x_2 + 0x_3 + 10x_4 \geq 50 \tag{29.1}$$

Analogamente, possiamo scrivere i requisiti che vogliamo ottenere almeno 100 000 voti periferici e 25 000 voti rurali così:

$$5x_1 + 2x_2 + 0x_3 + 0x_4 \geq 100 \tag{29.2}$$

e

$$3x_1 - 5x_2 + 10x_3 - 2x_4 \geq 25 \tag{29.3}$$

Ogni valore delle variabili x_1, x_2, x_3, x_4 che soddisfa le disuguaglianze (29.1)–(29.3) costituisce una strategia che consentirà di ottenere un numero sufficiente di voti per ciascuno dei tre tipi di aree elettorali. Per ridurre i costi al livello più basso possibile, bisogna minimizzare l'importo della spesa pubblicitaria, ovvero bisogna minimizzare l'espressione

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$x_1 + x_2 + x_3 + x_4 \tag{29.4}$$

Sebbene la pubblicità negativa sia un evento possibile nelle campagne elettorali, tuttavia non esiste una pubblicità di costo negativo. Di conseguenza, deve essere

$$x_1 \geq 0,\ x_2 \geq 0,\ x_3 \geq 0 \text{ e } x_4 \geq 0 \tag{29.5}$$

Combinando le disuguaglianze (29.1)–(29.3) e (29.5) con l'obiettivo di minimizzare l'espressione (29.4), otteniamo il cosiddetto "programma lineare". Esprimiamo questo problema nella seguente forma:

$$\text{minimizzare} \quad x_1 + x_2 + x_3 + x_4 \tag{29.6}$$

con le condizioni

$$-2x_1 + 8x_2 + 0x_3 + 10x_4 \geq 50 \tag{29.7}$$

$$5x_1 + 2x_2 + 0x_3 + 0x_4 \geq 100 \tag{29.8}$$

$$3x_1 - 5x_2 + 10x_3 - 2x_4 \geq 25 \tag{29.9}$$

$$x_1, x_2, x_3, x_4 \geq 0 \tag{29.10}$$

La soluzione di questo programma lineare offre al politico una strategia per la sua campagna elettorale.

Programmi lineari generali

Nel problema della programmazione lineare generale, vogliamo ottimizzare una funzione lineare soggetta a un sistema di disuguaglianze lineari. Dati un insieme di numeri reali $a_1, a_2, \ldots, a_n$ e un insieme di variabili $x_1, x_2, \ldots, x_n$, una ***funzione lineare*** f di queste variabili è così definita:

$$f(x_1, x_2, \ldots, x_n) = a_1x_1 + a_2x_2 + \cdots + a_nx_n = \sum_{j=1}^{n} a_jx_j$$

Se b è un numero reale ed f è una funzione lineare, allora l'equazione

$$f(x_1, x_2, \ldots, x_n) = b$$

è una ***uguaglianza lineare*** e le disuguaglianze

$$f(x_1, x_2, \ldots, x_n) \leq b$$

e

$$f(x_1, x_2, \ldots, x_n) \geq b$$

sono ***disuguaglianze lineari***. Utilizziamo il termine ***vincoli lineari*** per indicare sia le uguaglianze lineari sia le disuguaglianze lineari. Nella programmazione lineare non sono ammesse le disuguaglianze strette. Formalmente un ***problema di programmazione lineare*** consiste nel minimizzare o massimizzare una funzione lineare soggetta a un insieme finito di vincoli lineari. Se stiamo minimizzando una funzione, allora il programma lineare è detto ***programma lineare di minimizzazione***; se stiamo massimizzando una funzione, allora il programma lineare è detto ***programma lineare di massimizzazione***.

La parte restante di questo capitolo tratta la formulazione e la risoluzione di programmi lineari. Sebbene ci siano numerosi algoritmi di programmazione lineare con tempo polinomiale, non li analizzeremo in questo capitolo. Studieremo invece l'algoritmo del simplesso, che è il più vecchio algoritmo di programmazione lineare. L'algoritmo del simplesso non viene eseguito in un tempo polinomiale nel caso peggiore, ma è abbastanza efficiente e ampiamente utilizzato nella pratica.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Panoramica sulla programmazione lineare

Per descrivere le proprietà e gli algoritmi dei programmi lineari, è comodo avere delle forme particolari in cui esprimerle. In questo capitolo utilizzeremo due for-

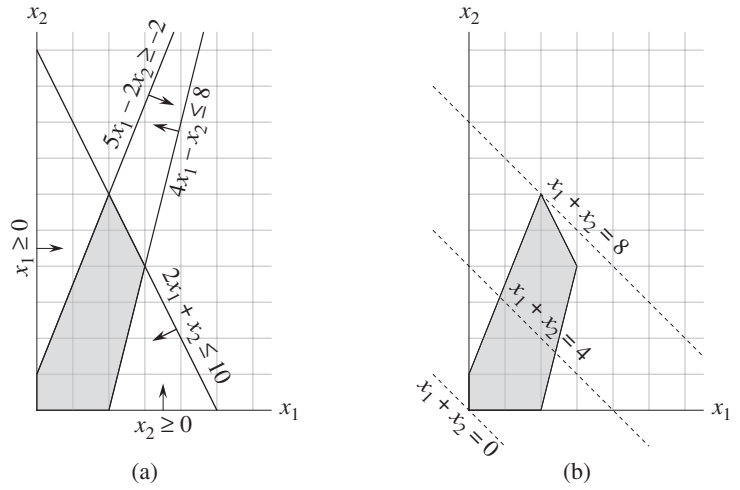


Figura 29.2
(a) Il programma lineare con i vincoli dati da (29.12)–(29.15). Ogni vincolo è rappresentato da una retta e da una direzione. L'intersezione dei vincoli forma la regione ammissibile, che è ombreggiata.
(b) Le linee tratteggiate indicano, rispettivamente, i punti per i quali il valore obiettivo è 0, 4 e 8. La soluzione ottima del programma lineare è $x_1 = 2$ e $x_2 = 6$ con valore obiettivo 8.

me: la ***canonica*** e la ***standard*** (*slack*). Queste forme saranno definite con precisione nel Paragrafo 29.2. Informalmente, un programma lineare in forma canonica è la massimizzazione di una funzione lineare soggetta a *disuguaglianze* lineari, mentre un programma lineare nella forma standard è la massimizzazione di una funzione lineare soggetta a *uguaglianze* lineari. Di solito utilizziamo la forma canonica per esprimere i programmi lineari, ma è più comodo utilizzare la forma standard per descrivere i dettagli dell'algoritmo del simplesso. Per adesso, focalizziamo l'attenzione sulla massimizzazione di una funzione lineare di n variabili soggetta a un sistema di m disuguaglianze lineari.

Consideriamo prima il seguente programma lineare con due variabili:

$$\text{massimizzare} \quad x_1 + x_2 \tag{29.11}$$

con le condizioni

$$4x_1 - x_2 \leq 8 \tag{29.12}$$
$$2x_1 + x_2 \leq 10 \tag{29.13}$$
$$5x_1 - 2x_2 \geq -2 \tag{29.14}$$
$$x_1, x_2 \geq 0 \tag{29.15}$$

Un'assegnazione qualsiasi di valori alle variabili x_1 e x_2 che soddisfa tutti i vincoli (29.12)–(29.15) è una ***soluzione ammissibile*** per il programma lineare. Se rappresentiamo graficamente i vincoli nel piano di coordinate cartesiane (x_1, x_2), come illustra la Figura 29.2(a), notiamo che l'insieme delle soluzioni ammissibili (l'area ombreggiata nella figura) forma una regione convessa[1] nello spazio bidimensionale. Questa regione convessa è detta ***regione ammissibile***. La funzione che vogliamo massimizzare è detta ***funzione obiettivo***. Teoricamente, potremmo calcolare la funzione obiettivo $x_1 + x_2$ in ciascun punto della regione ammissibile; chiameremo ***valore obiettivo*** il valore della funzione obiettivo in un particolare punto. Potremmo quindi identificare, come soluzione ottima, un punto con valore obiettivo massimo. Per questo esempio (e per la maggior parte dei programmi lineari), la regione ammissibile contiene un numero infinito di punti; ci serve quindi

Acquistato da Michele Michele su Vitalsource il 2022-07-07T23:13 Mirafiera Ordine Libreria 19750316 9780707 0 Copyright © 2022 McGraw-Hill Education (Italy)

[1] Una definizione intuitiva di regione convessa è che essa soddisfa la condizione che, dati due punti qualsiasi nella regione convessa, tutti i punti di un segmento di retta che congiunge questi due punti appartengono alla regione convessa.

un metodo efficiente per trovare un punto con il massimo valore obiettivo, senza dover calcolare esplicitamente la funzione obiettivo in ogni punto della regione ammissibile.

Nello spazio a due dimensioni, possiamo trovare una soluzione ottima usando una procedura grafica. L'insieme dei punti per i quali $x_1 + x_2 = z$, per ogni z, è una retta con pendenza -1. Se rappresentiamo graficamente $x_1 + x_2 = 0$, otteniamo la retta con pendenza -1 che passa per l'origine, come illustra la Figura 29.2(b). L'intersezione di questa retta con la regione ammissibile è l'insieme delle soluzioni ammissibili che hanno valore obiettivo pari a 0. In questo caso, l'intersezione della retta con la regione ammissibile è il punto $(0, 0)$. Più in generale, per ogni z, l'intersezione della retta $x_1 + x_2 = z$ con la regione ammissibile è l'insieme delle soluzioni ammissibili che hanno valore obiettivo pari a z. La Figura 29.2(b) illustra le rette $x_1 + x_2 = 0$, $x_1 + x_2 = 4$ e $x_1 + x_2 = 8$. Poiché la regione ammissibile nella Figura 29.2 è limitata, deve esistere qualche valore massimo z per il quale l'intersezione della retta $x_1 + x_2 = z$ con la regione ammissibile non sia vuota. Un punto qualsiasi in cui si verifica questo è una soluzione ottima del programma lineare, che in questo caso è il punto $x_1 = 2$ e $x_2 = 6$ con valore obiettivo 8. Non è un caso se una soluzione ottima per il programma lineare si trova in corrispondenza di un vertice della regione ammissibile. Il valore massimo di z per il quale la retta $x_1 + x_2 = z$ interseca la regione ammissibile deve essere sul confine della regione ammissibile, e quindi l'intersezione di questa retta con il confine della regione ammissibile o è un vertice o è un segmento di retta. Se l'intersezione è un vertice, allora c'è una sola soluzione ottima, che è un vertice. Se l'intersezione è un segmento di retta, tutti i punti di tale segmento devono avere lo stesso valore obiettivo; in particolare, entrambi gli estremi del segmento sono soluzioni ottime. Poiché ciascun estremo di un segmento di retta è un vertice, anche in questo caso esiste una soluzione ottima in un vertice.

Sebbene non sia facile rappresentare graficamente i programmi lineari con più di due variabili, tuttavia è possibile applicare lo stesso processo intuitivo. Se un programma lineare ha tre variabili, allora ogni vincolo è descritto da un semispazio nello spazio tridimensionale. L'intersezione di questi semispazi forma la regione ammissibile. L'insieme dei punti per i quali la funzione obiettivo ha valore z adesso è un piano (escludendo i casi degeneri). Se tutti i coefficienti della funzione obiettivo non sono negativi e se l'origine è una soluzione ammissibile per il programma lineare, allora quando allontaniamo questo piano dall'origine, in direzione normale alla funzione obiettivo, troviamo punti con valore obiettivo crescente. (Se l'origine non è una soluzione ammissibile o se qualche coefficiente nella funzione obiettivo è negativo, la rappresentazione intuitiva diventa un po' più complicata.) Come nel caso bidimensionale, poiché la regione ammissibile è convessa, l'insieme dei punti che raggiungono il valore obiettivo ottimo deve includere un vertice della regione ammissibile. Analogamente, se abbiamo n variabili, ciascun vincolo definisce un semispazio nello spazio a n dimensioni. La regione ammissibile formata dall'intersezione di questi semispazi è detta ***simplesso***. La funzione obiettivo adesso è un iperpiano e, a causa delle convessità, una soluzione ottima sarà ancora un vertice del simplesso.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995l03016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

L'***algoritmo del simplesso*** riceve come input un programma lineare e restituisce una soluzione ottima. Inizia da un vertice del simplesso ed esegue una sequenza di iterazioni. In ogni iterazione si sposta lungo un lato del simplesso dal vertice corrente a un vertice adiacente il cui valore obiettivo non è più piccolo di quello

del vertice corrente (di solito è più grande). L'algoritmo del simplesso termina quando raggiunge un massimo locale, che è un vertice i cui vertici adiacenti hanno tutti un valore obiettivo più piccolo. Poiché la regione ammissibile è convessa e la funzione obiettivo è lineare, questo ottimo locale è, in effetti, un ottimo globale. Nel Paragrafo 29.5, applicheremo il concetto di "dualità" per dimostrare che la soluzione restituita dall'algoritmo del simplesso è davvero ottima.

Sebbene la rappresentazione geometrica offra una buona visione intuitiva del funzionamento dell'algoritmo del simplesso, non faremo esplicito riferimento a tale rappresentazione quando descriveremo i dettagli dell'algoritmo del simplesso nel Paragrafo 29.4. Utilizzeremo invece una rappresentazione algebrica. Scriveremo prima il programma lineare nella forma standard, che è un sistema di uguaglianze lineari. Queste uguaglianze lineari esprimono alcune variabili, dette "variabili di base", in funzione di altre variabili, dette "variabili non di base". Lo spostamento da un vertice all'altro sarà effettuato trasformando una variabile di base in una variabile non di base, e una variabile non di base in una variabile di base. Questa operazione è detta "pivoting" (rotazione attorno a un perno, il pivot) e, da un punto di vista algebrico, non è che una riscrittura del programma lineare in una forma standard equivalente.

L'esempio con due variabili descritto precedentemente era particolarmente semplice. Sarà necessario considerare molti altri dettagli in questo capitolo, come i programmi lineari che non hanno soluzioni, i programmi lineari che non hanno una soluzione ottima finita e i programmi lineari per i quali l'origine non è una soluzione ammissibile.

Applicazioni della programmazione lineare

La programmazione lineare ha numerosissime applicazioni. Qualsiasi libro di testo sulla ricerca operativa è pieno di esempi di programmazione lineare. Attualmente, la programmazione lineare è uno strumento standard che viene insegnato nella maggior parte delle università di economia e commercio. Il caso delle elezioni politiche è un tipico esempio. Due altri esempi di programmazione lineare sono i seguenti:

- Una compagnia aerea vuole programmare i turni di lavoro del suo personale di volo. Tale programmazione deve soddisfare le disposizioni governative (i vincoli) che impongono, fra le altre cose, che ogni componente di un equipaggio non può lavorare per più di un certo numero di ore consecutive e che ogni equipaggio deve lavorare su un solo modello di aereo per almeno un mese. La compagnia aerea vuole programmare il turni degli equipaggi per tutti i suoi voli utilizzando il minor numero possibile di personale.
- Una compagnia petrolifera deve decidere dove cercare il petrolio. L'installazione di un pozzo petrolifero in una particolare posizione ha un costo associato e, sulla base di ricerche geologiche, un ricavo previsto di un certo numero di barili di petrolio. La compagnia ha un budget limitato per installare nuovi pozzi e vuole massimizzare la quantità di petrolio che prevede di estrarre, rispettando questo budget.

Acquistato da Michele Michelucci (michelucci@gmail.com) - Licenza Creative Commons - Copyright © 2022 McGraw-Hill Education (Italy)

I programmi lineari sono utili anche per modellare e risolvere problemi di grafi e di ottimizzazione combinatoria, come quelli descritti in questo libro. Abbiamo già visto un caso particolare di programmazione lineare per risolvere i sistemi di

vincoli sulle differenze nel Paragrafo 24.5. Nel Paragrafo 29.3 studieremo come formulare vari problemi di grafi e di flusso su rete come programmi lineari. Nel Paragrafo 35.4 utilizzeremo la programmazione lineare come uno strumento per trovare una soluzione approssimata di un altro problema di grafi.

Algoritmi per la programmazione lineare

Questo capitolo studia l'algoritmo del simplesso. Se implementato in modo appropriato, questo algoritmo è in grado di risolvere rapidamente dei programmi lineari generali. Tuttavia, con particolari input artificialmente costruiti, l'algoritmo del simplesso può richiedere un tempo esponenziale. Il primo algoritmo con tempo polinomiale per la programmazione lineare è stato l'***algoritmo dell'ellissoide***, che in pratica risulta molto lento. Una seconda classe di algoritmi con tempo polinomiale include i cosiddetti ***metodi dei punti interni***. Diversamente dall'algoritmo del simplesso, che si sposta lungo la parte esterna della regione ammissibile e mantiene una soluzione ammissibile che è un vertice del simplesso in ogni iterazione, questi algoritmi si spostano all'interno della regione ammissibile. Le soluzioni intermedie, anche se ammissibili, non sono necessariamente vertici del simplesso, ma la soluzione finale è un vertice. Per grandi input, gli algoritmi dei punti interni possono risultare veloci quanto quello del simplesso, e talvolta anche più veloci.

Se aggiungiamo a un programma lineare un'ulteriore condizione che tutte le variabili devono assumere valori interi, abbiamo un ***programma lineare intero***. L'Esercizio 34.6-3 chiede di dimostrare che trovare una soluzione ammissibile per questo problema è un problema NP-difficile; poiché non si conoscono algoritmi con tempo polinomiale per qualsiasi problema NP-difficile, non esiste un algoritmo con tempo polinomiale per la programmazione lineare intera. D'altra parte, un problema di programmazione lineare generale può essere risolto in tempo polinomiale.

In questo capitolo, se abbiamo un programma lineare con variabili $x = (x_1, x_2, \ldots, x_n)$ e vogliamo fare riferimento a un particolare valore delle variabili, utilizzeremo la notazione $\bar{x} = (\bar{x}_1, \bar{x}_2, \ldots, \bar{x}_n)$.

29.2 Le forme canoniche e standard

Questo paragrafo descrive due forme, canonica e standard, che saranno utili per specificare e operare con i programmi lineari. Nella forma canonica, tutti i vincoli sono espressi da disuguaglianze, mentre nella forma standard, i vincoli sono espressi da uguaglianze.

La forma canonica

Nella ***forma canonica*** sono noti n numeri reali $c_1, c_2, \ldots, c_n$, m numeri reali $b_1, b_2, \ldots, b_m$ ed mn numeri reali a_{ij} per $i = 1, 2, \ldots, m$ e $j = 1, 2, \ldots, n$. Vogliamo trovare n numeri reali $x_1, x_2, \ldots, x_n$ che

massimizzano $$\sum_{j=1}^{n} c_j x_j \tag{29.16}$$

con le condizioni

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero ordine libro 5158-2-2707. Copyright © 2022, McGraw-Hill Education (Italy)

$$\sum_{j=1}^{n} a_{ij} x_j \leq b_i \quad \text{per } i = 1, 2, \ldots, m \tag{29.17}$$

$$x_j \geq 0 \quad \text{per } j = 1, 2, \ldots, n \tag{29.18}$$

Generalizzando la terminologia introdotta per il programma lineare con due variabili, chiamiamo ***funzione obiettivo*** l'espressione (29.16) e ***vincoli*** le $n + m$ disuguaglianze (29.17) e (29.18). Gli n vincoli (29.18) sono detti ***vincoli di non negatività***. Un programma lineare arbitrario non richiede vincoli di non negatività, mentre la forma canonica li richiede. A volte è comodo esprimere un programma lineare in una forma più compatta. Se creiamo una matrice $A = (a_{ij})$ di dimensioni $m \times n$, un vettore $b = (b_i)$ di dimensione m, un vettore $c = (c_j)$ di dimensione n e un vettore $x = (x_j)$ di dimensione n, allora possiamo riscrivere il programma lineare definito dalle espressioni (29.16)–(29.18) nel seguente modo:

massimizzare $c^{\mathrm{T}} x$ (29.19)

con le condizioni

$$Ax \leq b \tag{29.20}$$

$$x \geq 0 \tag{29.21}$$

Nell'espressione (29.19) il termine $c^{\mathrm{T}} x$ è il prodotto interno di due vettori. Nell'espressione (29.20) Ax è un prodotto matrice-vettore; nell'espressione (29.21) $x \geq 0$ significa che ogni elemento del vettore x è non negativo. Vedremo che è possibile specificare un programma lineare nella forma canonica mediante una tupla (A, b, c); inoltre adotteremo la convenzione che A, b e c avranno sempre le dimensioni precedentemente indicate.

Adesso introduciamo la terminologia per descrivere le soluzioni dei programmi lineari. Parte di questa terminologia è stata utilizzata nel precedente esempio del programma lineare con due variabili. Un valore $\bar{x}$ delle variabili che soddisfa tutti i vincoli è detta ***soluzione ammissibile***, mentre un valore $\bar{x}$ delle variabili che non soddisfa almeno un vincolo è detta ***soluzione inammissibile***. Diciamo che una soluzione $\bar{x}$ ha ***valore obiettivo*** $c^{\mathrm{T}} \bar{x}$. Una soluzione ammissibile $\bar{x}$ il cui valore obiettivo è il più grande tra tutte le soluzioni ammissibili è una ***soluzione ottima*** e il suo valore obiettivo $c^{\mathrm{T}} \bar{x}$ è detto ***valore obiettivo ottimo***. Un programma lineare che non ha soluzioni ammissibili è detto ***inammissibile***, altrimenti è detto ***ammissibile***. Un programma lineare che ha qualche soluzione ammissibile, ma non ha un valore obiettivo ottimo finito è detto ***illimitato***. L'Esercizio 29.2-9 chiede di dimostrare che un programma lineare può avere un valore obiettivo ottimo finito anche se la regione ammissibile non è limitata.

Trasformare i programmi lineari nella forma canonica

È sempre possibile convertire nella forma canonica un programma lineare che rappresenta la minimizzazione o massimizzazione di una funzione lineare soggetta a vincoli lineari. Un programma lineare potrebbe non essere in forma canonica per una delle seguenti quattro ragioni:

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0. Copyright © 2022 McGraw-Hill Education (Italy)

1. La funzione obiettivo può essere una minimizzazione, anziché una massimizzazione.
2. Ci possono essere variabili senza vincoli di non negatività.

3. Ci possono essere ***vincoli di uguaglianza*** che però hanno il segno uguale, anziché il segno minore-o-uguale.
4. Ci possono essere ***vincoli di disuguaglianza*** che hanno il segno maggiore-o-uguale, anziché il segno minore-o-uguale.

Quando un programma lineare L viene trasformato in un altro programma lineare L', deve valere la proprietà che da una soluzione ottima per L' si può ottenere una soluzione ottima per L. Per spiegare questo concetto, diciamo che due programmi lineari di massimizzazione L e L' sono ***equivalenti*** se per ogni soluzione ammissibile $\bar{x}$ per L con valore obiettivo z, esiste una soluzione ammissibile corrispondente $\bar{x}'$ per L' con valore obiettivo z, e per ogni soluzione ammissibile $\bar{x}'$ per L' con valore obiettivo z, esiste una soluzione ammissibile corrispondente $\bar{x}$ per L con valore obiettivo z (questa definizione non implica una corrispondenza uno-a-uno tra le soluzioni ammissibili). Un programma lineare di minimizzazione L e un programma lineare di massimizzazione L' sono equivalenti se, per ogni soluzione ammissibile $\bar{x}$ per L con valore obiettivo z, esiste una soluzione ammissibile corrispondente $\bar{x}'$ per L' con valore obiettivo $-z$, e per ogni soluzione ammissibile $\bar{x}'$ per L' con valore obiettivo z, esiste una soluzione ammissibile corrispondente $\bar{x}$ per L con valore obiettivo $-z$.

Adesso spieghiamo come eliminare, uno per uno, ciascuno dei possibili problemi precedentemente elencati. Dopo avere eliminato un problema, dimostreremo che il nuovo programma lineare è equivalente a quello vecchio.

Per trasformare un programma lineare di minimazzazione L in un equivalente programma lineare di massimizzazione L', basta cambiare il segno dei coefficienti nella funzione obiettivo. Poiché L ed L' hanno lo stesso insieme di soluzioni ammissibili e, per qualsiasi soluzione ammissibile, il valore obiettivo in L è l'opposto del valore obiettivo in L', questi due programmi lineari sono equivalenti. Per esempio, consideriamo il programma lineare

$$\begin{aligned} &\text{minimizzare} && -2x_1 + 3x_2 \\ &\text{con le condizioni} \\ & && x_1 + x_2 = 7 \\ & && x_1 - 2x_2 \leq 4 \\ & && x_1 \geq 0 \end{aligned}$$

Se cambiamo il segno dei coefficienti della funzione obiettivo, otteniamo

$$\begin{aligned} &\text{massimizzare} && 2x_1 - 3x_2 \\ &\text{con le condizioni} \\ & && x_1 + x_2 = 7 \\ & && x_1 - 2x_2 \leq 4 \\ & && x_1 \geq 0 \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Adesso vediamo come trasformare un programma lineare in cui alcune variabili non hanno vincoli di non negatività in un programma in cui ogni variabile ha un vincolo di non negatività. Supponiamo che qualche variabile x_j non abbia un vincolo di non negatività. Sostituiamo ogni occorrenza di x_j con $x'_j - x''_j$ e aggiungiamo i vincoli di non negatività $x'_j \geq 0$ e $x''_j \geq 0$. Quindi, se la funzione obiettivo ha un termine $c_j x_j$, sostituiamo questo termine con $c_j x'_j - c_j x''_j$; inoltre se un

vincolo i ha un termine $a_{ij}x_j$, questo termine viene sostituito da $a_{ij}x'_j - a_{ij}x''_j$. Qualsiasi soluzione ammissibile $\hat{x}$ per il nuovo programma lineare corrisponde a una soluzione ammissibile $\bar{x}$ per il programma lineare originale con $\bar{x}_j = \hat{x}'_j - \hat{x}''_j$ e con lo stesso valore obiettivo. Inoltre, qualsiasi soluzione ammissibile $\bar{x}$ per il programma lineare originale corrisponde a una soluzione ammissibile $\hat{x}$ per il nuovo programma lineare con $\hat{x}'_j = \bar{x}_j$ e $\hat{x}''_j = 0$ se $\bar{x}_j \geq 0$, oppure con $\hat{x}''_j = \bar{x}_j$ e $\hat{x}'_j = 0$ se $\bar{x}_j < 0$. I due programmi lineari hanno lo stesso valore obiettivo indipendentemente dal segno di $\bar{x}_j$. Quindi i due programmi lineari sono equivalenti. Applichiamo questo schema di trasformazione a ogni variabile che non ha un vincolo di non negatività per ottenere un programma lineare equivalente in cui tutte le variabili hanno vincoli di non negatività.

Riprendendo l'esempio precedente, vogliamo garantire che ogni variabile abbia un corrispondente vincolo di non negatività. La variabile x_1 ha tale vincolo, ma la variabile x_2 no. Quindi, sostituiamo x_2 con le due variabili x'_2 e x''_2 e modifichiamo il programma lineare in questo modo:

massimizzare $\quad 2x_1 - 3x'_2 + 3x''_2$

con le condizioni

$$\begin{aligned} x_1 + x'_2 - x''_2 &= 7 \\ x_1 - 2x'_2 + 2x''_2 &\leq 4 \\ x_1, x'_2, x''_2 &\geq 0 \end{aligned} \tag{29.22}$$

Adesso trasformiamo i vincoli di uguaglianza in vincoli di disuguaglianza. Supponiamo che un programma lineare abbia un vincolo di uguaglianza $f(x_1, x_2, \ldots, x_n) = b$. Poiché $x = y$ se e soltanto se $x \geq y$ e $x \leq y$, possiamo sostituire questo vincolo di uguaglianza con la coppia di vincoli di disuguaglianza $f(x_1, x_2, \ldots, x_n) \leq b$ e $f(x_1, x_2, \ldots, x_n) \geq b$. Ripetendo questa trasformazione per ogni vincolo di uguaglianza, otteniamo un programma lineare in cui tutti i vincoli sono disuguaglianze.

Infine, possiamo trasformare i vincoli maggiore-o-uguale in vincoli minore-o-uguale moltiplicando questi vincoli per -1. Ovvero, qualsiasi disuguaglianza della forma

$$\sum_{j=1}^{n} a_{ij}x_j \geq b_i$$

è equivalente a

$$\sum_{j=1}^{n} -a_{ij}x_j \leq -b_i$$

Quindi, sostituendo ogni coefficiente a_{ij} con $-a_{ij}$ e ogni valore b_i con $-b_i$, otteniamo un vincolo minore-o-uguale equivalente. Per completare l'esempio, sostituiamo l'uguaglianza nel vincolo (29.22) con due disuguaglianze, ottenendo

Acquistato da Michele Michele su Vitalsource il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

massimizzare $\quad 2x_1 - 3x'_2 + 3x''_2$

con le condizioni

$$\begin{aligned} x_1 + x'_2 - x''_2 &\leq 7 \\ x_1 + x'_2 - x''_2 &\geq 7 \\ x_1 - 2x'_2 + 2x''_2 &\leq 4 \\ x_1, x'_2, x''_2 &\geq 0 \end{aligned} \tag{29.23}$$

Infine, cambiamo il segno ai coefficienti del vincolo (29.23). Per coerenza tra i nomi delle variabili, cambiamo il nome x_2' in x_2 e x_2'' in x_3, ottenendo la seguente forma canonica:

$$\text{massimizzare} \quad 2x_1 - 3x_2 + 3x_3 \tag{29.24}$$

con le condizioni

$$x_1 + x_2 - x_3 \leq 7 \tag{29.25}$$

$$-x_1 - x_2 + x_3 \leq -7 \tag{29.26}$$

$$x_1 - 2x_2 + 2x_3 \leq 4 \tag{29.27}$$

$$x_1, x_2, x_3 \geq 0 \tag{29.28}$$

Trasformare i programmi lineari nella forma standard

Per risolvere in modo efficiente un programma lineare con l'algoritmo del simplesso, preferiamo esprimerlo in una forma in cui alcuni dei vincoli sono vincoli di uguaglianza. Più precisamente, lo trasformeremo in una forma in cui i vincoli di non negatività sono gli unici vincoli di disuguaglianza e i restanti vincoli sono uguaglianze. Sia

$$\sum_{j=1}^{n} a_{ij} x_j \leq b_i \tag{29.29}$$

un vincolo di disuguaglianza. Introduciamo una nuova variabile s e riscriviamo la disuguaglianza (29.29) con i seguenti due vincoli

$$s = b_i - \sum_{j=1}^{n} a_{ij} x_j \tag{29.30}$$

$$s \geq 0 \tag{29.31}$$

La variabile s è detta ***variabile scarto*** (*slack*) perché misura lo scarto tra il primo e il secondo membro dell'equazione (29.29). (Vedremo fra poco perché è opportuno scrivere le equazioni con le sole variabili scarto alla sinistra.) Poiché la disuguaglianza (29.29) è vera se e soltanto se sono vere entrambe le espressioni (29.30) e (29.31), possiamo applicare questa trasformazione a ogni vincolo di disuguaglianza di un programma lineare, ottenendo un programma lineare equivalente in cui gli unici vincoli di disuguaglianza sono i vincoli di non negatività. Quando effettuiamo una trasformazione dalla forma canonica alla forma standard, utilizziamo x_{n+i} (anziché s) per indicare la variabile scarto associata alla i-esima disuguaglianza. L'i-esimo vincolo è quindi

$$x_{n+i} = b_i - \sum_{j=1}^{n} a_{ij} x_j \tag{29.32}$$

assieme al vincolo di non negatività $x_{n+i} \geq 0$.

Acquistato da Michele Michele su Webster il 2022-07-01 23:12. Numero Ordine Libraio: 1995021616 228767 01 Copyright © 2022 McGraw-Hill Education (Italy)

Applicando questa trasformazione a ogni vincolo di un programma lineare nella forma canonica, otteniamo un programma lineare in una forma differente. Per esempio, se utilizziamo le variabili scarto x_4, x_5 e x_6 nel programma lineare descritto dalle espressioni (29.24)–(29.28), otteniamo

$$\text{massimizzare} \quad 2x_1 - 3x_2 + 3x_3 \tag{29.33}$$

con le condizioni

$$x_4 = 7 - x_1 - x_2 + x_3 \tag{29.34}$$

$$x_5 = -7 + x_1 + x_2 - x_3 \tag{29.35}$$
$$x_6 = 4 - x_1 + 2x_2 - 2x_3 \tag{29.36}$$
$$x_1, x_2, x_3, x_4, x_5, x_6 \geq 0 \tag{29.37}$$

In questo programma lineare, tutti i vincoli, tranne quelli di non negatività, sono uguaglianze e ogni variabile è soggetta a un vincolo di non negatività. Esprimiamo ciascun vincolo di uguaglianza ponendo una delle variabili nel lato sinistro dell'uguaglianza e tutte le altre nel lato destro. Inoltre, ogni equazione ha lo stesso insieme di variabili nel lato destro, e queste variabili sono anche le uniche che appaiono nella funzione obiettivo. Le variabili nel lato sinistro delle uguaglianze sono dette ***variabili di base*** e quelle nel lato destro sono dette ***variabili non di base***.

Per i programmi lineari che soddisfano queste condizioni, a volte ometteremo sia i vincoli espliciti di non negatività che le parole "massimizzare" e "con le condizioni". Inoltre utilizzeremo la variabile z per indicare il valore della funzione obiettivo. La forma risultante è detta ***forma standard***. Se scriviamo il programma lineare definito dalle espressioni (29.33)–(29.37) nella forma standard, otteniamo

$$z = 2x_1 - 3x_2 + 3x_3 \tag{29.38}$$
$$x_4 = 7 - x_1 - x_2 + x_3 \tag{29.39}$$
$$x_5 = -7 + x_1 + x_2 - x_3 \tag{29.40}$$
$$x_6 = 4 - x_1 + 2x_2 - 2x_3 \tag{29.41}$$

Come nella forma canonica, sarà comodo avere una notazione più concisa per descrivere una forma standard. Come vedremo nel Paragrafo 29.4, gli insiemi delle variabili di base e non di base cambiano durante l'esecuzione dell'algoritmo del simplesso. Indicheremo con N l'insieme degli indici delle variabili non di base e con B l'insieme degli indici delle variabili di base. Avremo sempre che $|N| = n$, $|B| = m$ ed $N \cup B = \{1, 2, \ldots, n + m\}$. Le equazioni saranno indicizzate con gli elementi di B; le variabili nel lato destro saranno indicizzate con gli elementi di N. Come nella forma canonica, utilizzeremo b_i, c_j e a_{ij} per indicare i termini costanti e i coefficienti. Utilizzeremo anche v per indicare un termine costante facoltativo nella funzione obiettivo. Quindi possiamo definire concisamente una forma standard con una tupla (N, B, A, b, c, v), che denota la forma standard

$$z = v + \sum_{j \in N} c_j x_j \tag{29.42}$$
$$x_i = b_i - \sum_{j \in N} a_{ij} x_j \quad \text{per } i \in B \tag{29.43}$$

dove tutte le variabili x hanno il vincolo di non essere negative. Poiché sottraiamo la sommatoria $\sum_{j \in N} a_{ij} x_j$ nell'espressione (29.43), i valori a_{ij} in effetti sono gli opposti dei coefficienti che "appaiono" nella forma standard.

Per esempio, nella forma standard

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$z = 28 - \frac{x_3}{6} - \frac{x_5}{6} - \frac{2x_6}{3}$$
$$x_1 = 8 + \frac{x_3}{6} + \frac{x_5}{6} - \frac{x_6}{3}$$
$$x_2 = 4 - \frac{8x_3}{3} - \frac{2x_5}{3} + \frac{x_6}{3}$$

$$x_4 = 18 - \frac{x_3}{2} + \frac{x_5}{2}$$

abbiamo $B = \{1, 2, 4\}$, $N = \{3, 5, 6\}$,

$$A = \begin{pmatrix} a_{13} & a_{15} & a_{16} \\ a_{23} & a_{25} & a_{26} \\ a_{43} & a_{45} & a_{46} \end{pmatrix} = \begin{pmatrix} -1/6 & -1/6 & 1/3 \\ 8/3 & 2/3 & -1/3 \\ 1/2 & -1/2 & 0 \end{pmatrix}$$

$$b = \begin{pmatrix} b_1 \\ b_2 \\ b_4 \end{pmatrix} = \begin{pmatrix} 8 \\ 4 \\ 18 \end{pmatrix}$$

$c = \begin{pmatrix} c_3 & c_5 & c_6 \end{pmatrix}^{\mathrm{T}} = \begin{pmatrix} -1/6 & -1/6 & -2/3 \end{pmatrix}^{\mathrm{T}}$ e $v = 28$. Notate che gli indici in A, b e c non sono necessariamente insiemi di interi contigui; essi dipendono dagli insiemi degli indici B ed N.

Come esempio del fatto che gli elementi di A sono gli opposti dei coefficienti che appaiono nella forma standard, osservate che l'equazione di x_1 include il termine $x_3/6$, mentre il coefficiente a_{13} è $-1/6$, anziché $+1/6$.

Esercizi

29.2-1

Se esprimiamo il programma lineare (29.24)–(29.28) secondo la notazione compatta (29.19)–(29.21), quali sono i valori di n, m, A, b e c?

29.2-2

Trovate tre soluzioni ammissibili per il programma lineare descritto dalle espressioni (29.24)–(29.28). Qual è il valore obiettivo di ciascuna soluzione?

29.2-3

Per la forma standard (29.38)–(29.41), quali sono i valori di N, B, A, b, c e v?

29.2-4

Trasformate il seguente programma lineare nella forma canonica:

$$\begin{aligned} &\text{minimizzare} && 2x_1 + 7x_2 + x_3 \\ &\text{con le condizioni} \\ & && x_1 - x_3 = 7 \\ & && 3x_1 + x_2 \geq 24 \\ & && x_2 \geq 0 \\ & && x_3 \leq 0 \end{aligned}$$

29.2-5

Trasformate il seguente programma lineare nella forma standard:

$$\begin{aligned} &\text{massimizzare} && 2x_1 - 6x_3 \\ &\text{con le condizioni} \\ & && x_1 + x_2 - x_3 \leq 7 \\ & && 3x_1 - x_2 \geq 8 \\ & && -x_1 + 2x_2 + 2x_3 \geq 0 \\ & && x_1, x_2, x_3 \geq 0 \end{aligned}$$

Quali sono la variabili di base e non di base?

Acquistato da Michele Michele su Webster il 2022-07-07 13:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

29.2-6
Dimostrate che il seguente programma lineare è inammissibile:

$$\begin{aligned}
&\text{massimizzare} && 3x_1 - 2x_2 \\
&\text{con le condizioni} \\
& && x_1 + x_2 \leq 2 \\
& && -2x_1 - 2x_2 \leq -10 \\
& && x_1, x_2 \geq 0
\end{aligned}$$

29.2-7
Dimostrate che il seguente programma lineare è illimitato:

$$\begin{aligned}
&\text{massimizzare} && x_1 - x_2 \\
&\text{con le condizioni} \\
& && -2x_1 + x_2 \leq -1 \\
& && -x_1 - 2x_2 \leq -2 \\
& && x_1, x_2 \geq 0
\end{aligned}$$

29.2-8
Supponete di avere un programma lineare generale con n variabili ed m vincoli e di trasformarlo nella forma canonica. Trovate un limite superiore per il numero di variabili e vincoli nel programma lineare risultante.

29.2-9
Trovate un esempio di programma lineare per il quale la regione ammissibile non è limitata, ma il valore obiettivo ottimo è finito.

29.3 Formulare i problemi come programmi lineari

Anche se questo capitolo è incentrato principalmente sull'analisi dell'algoritmo del simplesso, tuttavia è anche importante sapere riconoscere quando un problema può essere formulato come un programma lineare. Una volta che un problema è formulato come un programma lineare di dimensione polinomiale, può essere risolto in tempo polinomiale tramite gli algoritmi dell'ellissoide o dei punti interni. Esistono vari pacchetti software per la programmazione lineare che possono essere utilizzati per risolvere i problemi con efficienza, quindi una volta che un problema è stato formulato come un programma lineare, potrà essere risolto praticamente da uno di questi pacchetti software.

Esamineremo numerosi esempi concreti di problemi di programmazione lineare. Inizieremo con due problemi che abbiamo già studiato: il problema dei cammini minimi da sorgente unica (vedere il Capitolo 24) e il problema del flusso massimo (vedere il Capitolo 26). Poi descriveremo il problema del flusso di costo minimo. Esiste un algoritmo con tempo polinomiale che non si basa sulla programmazione lineare per risolvere il problema del flusso di costo minimo, ma non lo esamineremo. Infine, descriveremo il problema del flusso di più merci (multicommodity flow), per il quale l'unico algoritmo noto con tempo polinomiale si basa sulla programmazione lineare.

Risolvendo i problemi dei grafi nella Parte VI, abbiamo utilizzato la notazione degli attributi, come $v.d$ e $(u, v).f$. I programmi lineari, invece, di solito utiliz-

Acquistato da Michele Mazzucco - Vedi http://www.mhe.it/2009-algoritmi - Numero licenza: 1006516 - Copyright © 2010 The McGraw-Hill Education (Italy)

zano variabili con indici, anziché oggetti con attributi associati. Pertanto, quando rappresentiamo le variabili nei programmi lineari, indichiamo i vertici e gli archi tramite indici. Per esempio, indichiamo il peso del cammino minimo per il vertice v con d_v, anziché $v.d$. Analogamente, indichiamo il flusso dal vertice u al vertice v con f_{uv}, anziché $(u, v).f$. Per quantità che sono date come input dei problemi, quali i pesi o le capacità degli archi, continueremo a utilizzare notazioni come $w(u, v)$ e $c(u.v)$.

Cammini minimi

Il problema dei cammini minimi da sorgente unica può essere formulato come un programma lineare. In questo paragrafo descriveremo la formulazione del problema del cammino minimo tra una coppia di vertici, lasciando al lettore il compito di estendere tale formulazione al problema più generale dei cammini minimi da sorgente unica (Esercizio 29.3-3).

Nel problema del cammino minimo tra una coppia di vertici, abbiamo un grafo orientato pesato $G = (V, E)$ con la funzione peso $w : E \to \mathbb{R}$ che associa agli archi dei pesi di valore reale, un vertice sorgente s e un vertice destinazione t. Vogliamo calcolare il valore d_t, che è il peso di un cammino minimo da s a t. Per esprimere questo problema come un programma lineare, bisogna determinare un insieme di variabili e vincoli che definiscono quando si ha un cammino minimo da s a t. Fortunatamente, l'algoritmo di Bellman-Ford fa proprio questo. Quando l'algoritmo di Bellman-Ford termina, ha calcolato, per ogni vertice v, un valore d_v tale che per ogni arco $(u, v) \in E$, si ha $d_v \leq d_u + w(u, v)$. Il vertice sorgente inizialmente riceve un valore $d_s = 0$, che non viene mai modificato. Quindi otteniamo il seguente programma lineare per calcolare il peso del cammino minimo da s a t:

$$\text{massimizzare} \quad d_t \tag{29.44}$$

con le condizioni

$$d_v \leq d_u + w(u, v) \quad \text{per ogni arco } (u, v) \in E \tag{29.45}$$

$$d_s = 0 \tag{29.46}$$

Potrebbe sembrare sorprendente che questo programma lineare massimizzi una funzione obiettivo quando si suppone che esso calcoli i cammini minimi. Non vogliamo minimizzare la funzione obiettivo, in quanto ponendo $\bar{d}_v = 0$ per ogni $v \in V$ si otterrebbe una soluzione ottima del programma lineare senza risolvere il problema dei cammini minimi. Noi massimizziamo perché una soluzione ottima per il problema dei cammini minimi imposta ciascun $\bar{d}_v$ a $\min_{u:(u,v)\in E} \{\bar{d}_u + w(u, v)\}$, cosicché $\bar{d}_v$ è il valore più grande che è minore o uguale a tutti i valori dell'insieme $\{\bar{d}_u + w(u, v)\}$. Vogliamo massimizzare d_v per tutti i vertici v che si trovano in un cammino minimo da s a t con questi vincoli su tutti i vertici v, e massimizzando d_t si raggiunge tale obiettivo.

In questo programma lineare, ci sono $|V|$ variabili d_v, una per ogni vertice $v \in V$. Ci sono $|E| + 1$ vincoli, uno per ogni arco più il vincolo aggiuntivo per il vertice sorgente, che ha sempre il valore 0.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Flusso massimo

Anche il problema del flusso massimo può essere espresso come un programma lineare. Ricordiamo che in un problema di flusso massimo sono dati un grafo

orientato $G = (V, E)$, dove ogni arco $(u, v) \in E$ ha una capacità non negativa $c(u, v) \geq 0$, e due vertici distinti, una sorgente s e un pozzo t. Secondo la definizione data nel Paragrafo 26.1, un flusso è una funzione a valori reali $f : V \times V \to \mathbb{R}$ che soddisfa i vincoli sulla capacità e la conservazione del flusso. Un flusso massimo è un flusso che soddisfa questi vincoli e massimizza il valore del flusso, che è il flusso totale che esce dalla sorgente. Un flusso, quindi, soddisfa dei vincoli lineari e il valore di un flusso è una funzione lineare. Ricordando inoltre che $c(u, v) = 0$ se $(u, v) \notin E$ e non ci sono archi antiparalleli, possiamo esprimere il problema di flusso massimo come un programma lineare:

$$\text{massimizzare} \quad \sum_{v \in V} f_{sv} - \sum_{v \in V} f_{vs} \tag{29.47}$$

con le condizioni

$$f_{uv} \leq c(u, v) \quad \text{per ogni } u, v \in V \tag{29.48}$$

$$\sum_{v \in V} f_{vu} = \sum_{v \in V} f_{uv} \quad \text{per ogni } u \in V - \{s, t\} \tag{29.49}$$

$$f_{uv} \geq 0 \quad \text{per ogni } u, v \in V \tag{29.50}$$

Questo programma lineare ha $|V|^2$ variabili, che corrispondono al flusso tra ogni coppia di vertici, e $2|V|^2 + |V| - 2$ vincoli.

Di solito è più efficiente risolvere un programma lineare di dimensioni più piccole. Il programma lineare definito dalle espressioni (29.47)–(29.50) ha, per semplificare la notazione, un flusso e una capacità pari a 0 per ogni coppia di vertici u, v con $(u, v) \notin E$. Sarebbe più efficiente riscrivere il programma lineare in modo che abbia $O(V + E)$ vincoli. L'Esercizio 29.3-5 chiede di fare così.

Flusso di costo minimo

In questo paragrafo abbiamo utilizzato la programmazione lineare per risolvere i problemi per i quali conoscevamo già degli algoritmi efficienti. In effetti, un algoritmo efficiente appositamente progettato per un problema, come l'algoritmo di Dijkstra per il problema dei cammini minimi da sorgente unica o il metodo push-relabel per il flusso massimo, spesso sono più efficienti della programmazione lineare, sia in teoria che in pratica.

La vera potenza della programmazione lineare sta nella capacità di risolvere nuovi problemi. Ricordiamo il problema affrontato dai candidati politici all'inizio di questo capitolo. Il problema di ottenere un numero sufficiente di voti, senza spendere troppi soldi, non è risolto da nessuno degli algoritmi finora esaminati in questo libro, ma è risolto dalla programmazione lineare. I libri abbondano di esempi di questi problemi reali che possono essere risolti dalla programmazione lineare. La programmazione lineare è anche particolarmente utile per risolvere varianti di problemi per i quali non si conoscono ancora algoritmi efficienti.

Consideriamo, per esempio, la seguente generalizzazione del problema di flusso massimo. Supponiamo che ogni arco (u, v) abbia, oltre a una capacità $c(u, v)$, un costo di valore reale $a(u, v)$. Come nel problema del flusso massimo, supponiamo che $c(u, v) = 0$ se $(u, v) \notin E$ e non ci siano archi antiparalleli. Se inviamo f_{uv} unità di flusso nell'arco (u, v), sosteniamo il costo $a(u, v) f_{uv}$. Un altro dato del problema è il flusso richiesto d. Vogliamo inviare d unità di flusso da s a t in modo che il costo totale del flusso, $\sum_{(u,v) \in E} a(u, v) f_{uv}$, sia minimo. Questo problema è detto ***problema del flusso di costo minimo***.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

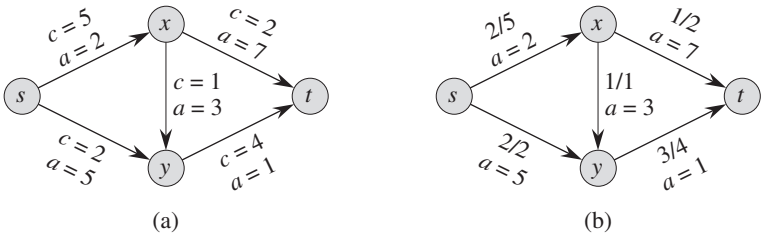


Figura 29.3
(a) Un esempio del problema del flusso di costo minimo. Indichiamo le capacità con c e i costi con a. Il vertice s è la sorgente e il vertice t è il pozzo. Vogliamo inviare 4 unità di flusso da s a t. **(b)** Una soluzione del problema del flusso di costo minimo in cui 4 unità di flusso vengono inviate da s a t. Per ogni arco, il flusso e la capacità sono indicati come flusso/capacità.

La Figura 29.3(a) illustra un esempio del problema del flusso di costo minimo. Vogliamo inviare 4 unità di flusso da s a t con il costo totale minimo. Un flusso legale qualsiasi, ovvero una funzione f che soddisfa i vincoli (29.48)–(29.49), richiede un costo totale di $\sum_{(u,v)\in E} a(u,v)\,f_{uv}$. Vogliamo trovare quel particolare flusso di 4 unità che minimizza questo costo. Una soluzione ottima è data nella Figura 29.3(b), il cui costo totale è $\sum_{(u,v)\in E} a(u,v)\,f_{uv} = (2\cdot 2) + (5\cdot 2) + (3\cdot 1) + (7\cdot 1) + (1\cdot 3) = 27$.

Ci sono algoritmi con tempo polinomiale appositamente ideati per il problema del flusso di costo minimo, ma la loro analisi esula dagli scopi di questo libro. Tuttavia, possiamo esprimere il problema del flusso di costo minimo come un programma lineare. Il programma lineare somiglia a quello per il problema del flusso massimo con il vincolo aggiuntivo che il valore del flusso deve essere di d unità e con la nuova funzione obiettivo di minimizzare il costo:

$$\text{minimizzare} \quad \sum_{(u,v)\in E} a(u,v)\,f_{uv} \tag{29.51}$$

con le condizioni

$$\begin{aligned}
f_{uv} &\le c(u,v) && \text{per ogni } u,v \in V \\
\sum_{v\in V} f_{vu} - \sum_{v\in V} f_{uv} &= 0 && \text{per ogni } u \in V - \{s,t\} \\
\sum_{v\in V} f_{sv} - \sum_{v\in V} f_{vs} &= d \\
f_{uv} &\ge 0 && \text{per ogni } u,v \in V
\end{aligned} \tag{29.52}$$

Flusso di più merci

Come ultimo esempio, consideriamo un altro problema di flusso. Supponiamo che la Lucky Puck Company (Paragrafo 26.1) decida di diversificare la sua linea di prodotti e di spedire non soltanto dischi da hockey, ma anche caschi e mazze da hockey. Ogni pezzo dell'equipaggiamento da hockey viene prodotto in un'apposita fabbrica, ha il suo magazzino e deve essere spedito ogni giorno dalla fabbrica al magazzino. Le mazze sono prodotte a Vancouver e devono essere spedite a Saskatoon; i caschi sono prodotti a Edmonton e devono essere spediti a Regina. La capacità della rete di spedizione non cambia e i vari pezzi dell'equipaggiamento (o ***merci***) devono condividere la stessa rete.

Acquistato da Michele Michele su Webster il 2022-07-07 22:13 - Nessuna Online Libreria 1998-2016 - 2007 © McGraw-Hill Education (Italy)

Questo esempio è un caso del ***problema del flusso di più merci***. In questo problema è dato un grafo orientato $G = (V, E)$ in cui ogni arco $(u, v) \in E$ ha una capacità $c(u, v) \ge 0$. Come nel problema del flusso massimo, implicitamente supponiamo che $c(u, v) = 0$ per $(u, v) \notin E$ e che il grafo non abbia archi antiparalleli. Inoltre, sono date k merci differenti, $K_1, K_2, \ldots, K_k$, dove la merce i è specificata dalla tripla $K_i = (s_i, t_i, d_i)$. Qui, s_i è la sorgente della merce i, t_i è il pozzo della merce i, e d_i è la domanda, che è il valore del flusso da s_i a t_i richiesto per la merce i.

Definiamo un flusso per la merce i – indicato con f_i (cosicché f_{iuv} è il flusso della merce i dal vertice u al vertice v) – come una funzione a valori reali che soddisfa la conservazione del flusso e i vincoli sulla capacità. Adesso definiamo f_{uv}, il ***flusso aggregato***, come la somma dei flussi delle varie merci, cosicché $f_{uv} = \sum_{i=1}^{k} f_{iuv}$. Il flusso aggregato in un arco (u, v) non deve superare la capacità dell'arco (u, v). Per il modo in cui abbiamo descritto questo problema, non c'è nulla da minimizzare; dobbiamo soltanto determinare se è possibile trovare un tale flusso. Quindi, scriviamo un programma lineare con una funzione obiettivo "nulla":

$$
\begin{aligned}
&\text{minimizzare} && 0 && \\
&\text{con le condizioni} && && \\
& && \sum_{i=1}^{k} f_{iuv} \le c(u,v) && \text{per ogni } u, v \in V \\
& \sum_{v \in V} f_{iuv} - \sum_{v \in V} f_{ivu} = 0 && && \text{per ogni } i = 1, 2, \ldots, k \text{ e} \\
& && && \text{per ogni } u \in V - \{s_i, t_i\} \\
& \sum_{v \in V} f_{i,s_i,v} - \sum_{v \in V} f_{i,v,s_i} = d_i && && \text{per ogni } i = 1, 2, \ldots, k \\
& && f_{iuv} \ge 0 && \text{per ogni } u, v \in V \text{ e} \\
& && && \text{per ogni } i = 1, 2, \ldots, k
\end{aligned}
$$

L'unico algoritmo noto con tempo polinomiale per questo problema consiste nell'esprimere il problema come un programma lineare e poi nel risolverlo con un algoritmo di programmazione lineare con tempo polinomiale.

Esercizi

29.3-1

Scrivete nella forma canonica il programma lineare del cammino minimo tra una coppia di vertici definito dalle espressioni (29.44)–(29.46).

29.3-2

Scrivete esplicitamente il programma lineare che equivale a trovare il cammino minimo dal nodo s al nodo y nella Figura 24.2(a).

29.3-3

Nel problema dei cammini minimi da sorgente unica bisogna trovare i pesi dei cammini minimi da un vertice sorgente s a tutti i vertici $v \in V$. Dato un grafo G, scrivete un programma lineare per il quale la soluzione ha la proprietà che d_v è il peso del cammino minimo da s a v per ogni vertice $v \in V$.

29.3-4

Scrivete esplicitamente il programma lineare che equivale a trovare il flusso massimo nella Figura 26.1(a).

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

29.3-5

Riscrivete il programma lineare per il flusso massimo (29.47)–(29.50) in modo che usi soltanto $O(V + E)$ vincoli.

29.3-6

Scrivete un programma lineare che, dato un grafo bipartito $G = (V, E)$, risolve il problema dell'abbinamento massimo nel grafo bipartito.

29.3-7

Nel ***problema del flusso di più merci di costo minimo*** è dato un grafo orientato $G = (V, E)$ in cui ogni arco $(u, v) \in E$ ha una capacità $c(u, v) \geq 0$ e un costo $a(u, v)$. Come nel problema del flusso di più merci, sono date k merci differenti, $K_1, K_2, \ldots, K_k$, dove la merce i è specificata dalla tripla $K_i = (s_i, t_i, d_i)$.

Definiamo il flusso f_i per la merce i e il flusso aggregato f_{uv} nell'arco (u, v) come nel problema del flusso di più merci. Un flusso ammissibile è quello in cui il flusso aggregato in ciascun arco (u, v) non supera la capacità dell'arco (u, v). Il costo di un flusso è $\sum_{u,v \in V} a(u, v) f_{uv}$ e l'obiettivo è trovare il flusso ammissibile di costo minimo. Esprimete questo problema come un programma lineare.

29.4 L'algoritmo del simplesso

L'algoritmo del simplesso è il classico metodo per risolvere i programmi lineari. Diversamente dalla maggior parte degli altri algoritmi esaminati in questo libro, il suo tempo di esecuzione non è polinomiale nel caso peggiore. Tuttavia, permette di capire i programmi lineari e spesso è estremamente rapido nelle applicazioni pratiche. Oltre ad avere un'interpretazione geometrica, descritta all'inizio di questo capitolo, l'algoritmo del simplesso presenta qualche analogia con il processo di eliminazione di Gauss, che abbiamo esaminato nel Paragrafo 28.1. Il processo di Gauss inizia con un sistema di uguaglianze lineari la cui soluzione non è nota. In ogni iterazione, riscriviamo questo sistema in una forma equivalente con qualche caratteristica strutturale aggiuntiva. Dopo un certo numero di iterazioni, abbiamo riscritto il sistema in modo tale che la soluzione è semplice da ottenere. L'algoritmo del simplesso procede in una maniera analoga, e possiamo considerarlo come il processo di eliminazione di Gauss per le disuguaglianze.

Descriviamo l'idea principale che sta dietro un'iterazione dell'algoritmo del simplesso. A ogni iterazione è associata una "soluzione di base", che si ottiene facilmente dalla forma standard del programma lineare: poniamo a zero tutte le variabili non di base e calcoliamo i valori delle variabili di base usando i vincoli di uguaglianza. Una soluzione di base corrisponde sempre a un vertice del simplesso. Algebricamente, un'iterazione converte una forma standard in una forma standard equivalente. Il valore obiettivo della soluzione di base ammissibile associata non sarà minore di quello della precedente iterazione (di solito è maggiore). Per realizzare questo incremento del valore obiettivo, scegliamo una variabile non di base tale che, se aumentiamo il valore di questa variabile da zero, allora aumenta anche il valore obiettivo. L'entità dell'incremento della variabile è limitata dagli altri vincoli. In particolare, aumentiamo la variabile finché qualche variabile di base diventa zero. Poi riscriviamo la forma standard, scambiando i ruoli di quella variabile di base e della variabile non di base che abbiamo scelto. Sebbene abbiamo utilizzato un particolare valore delle variabili per guidare l'algoritmo, e lo utilizzeremo nelle nostre dimostrazioni, l'algoritmo non mantiene esplicitamente questa soluzione. Esso semplicemente riscrive il programma lineare finché la soluzione ottima diventa "ovvia".

Acquistato da Michele Michele su Webster il 2022-07-27 23:10. Numero edizione libera: 1805288162207357. Copyright © 2012, McGraw-Hill Education (Italia)

Un esempio dell'algoritmo del simplesso

Iniziamo con un esempio completo. Consideriamo il seguente programma lineare nella forma canonica:

$$\text{massimizzare} \quad 3x_1 + x_2 + 2x_3 \tag{29.53}$$

con le condizioni

$$x_1 + x_2 + 3x_3 \leq 30 \tag{29.54}$$

$$2x_1 + 2x_2 + 5x_3 \leq 24 \tag{29.55}$$

$$4x_1 + x_2 + 2x_3 \leq 36 \tag{29.56}$$

$$x_1, x_2, x_3 \geq 0 \tag{29.57}$$

Per potere utilizzare l'algoritmo del simplesso, dobbiamo convertire il programma lineare nella forma standard; abbiamo spiegato come effettuare tale trasformazione nel Paragrafo 29.2. Oltre a indicare una trasformazione algebrica, lo scarto è anche un utile concetto algoritmico. Ricordando dal Paragrafo 29.2 che ogni variabile ha un corrispondente vincolo di non negatività, diciamo che un vincolo di uguaglianza è ***stretto*** per un particolare valore delle sue variabili non di base se tale valore fa diventare zero la variabile di base del vincolo. Analogamente, un valore delle variabili non di base che fa diventare negativa una variabile di base ***viola*** quel vincolo. Quindi, le variabili scarto indicano esplicitamente quanto lontano sia ogni vincolo dall'essere stretto, e così ci aiutano a determinare di quanto possiamo incrementare i valori delle variabili non di base senza violare alcun vincolo.

Associando le variabili scarto x_4, x_5 e x_6, rispettivamente, alle disuguaglianze (29.54)–(29.56) e ponendo il programma lineare nella forma standard, otteniamo

$$z = 3x_1 + x_2 + 2x_3 \tag{29.58}$$

$$x_4 = 30 - x_1 - x_2 - 3x_3 \tag{29.59}$$

$$x_5 = 24 - 2x_1 - 2x_2 - 5x_3 \tag{29.60}$$

$$x_6 = 36 - 4x_1 - x_2 - 2x_3 \tag{29.61}$$

Il sistema dei vincoli (29.59)–(29.61) ha 3 equazioni e 6 variabili. Qualsiasi valore delle variabili x_1, x_2 e x_3 definisce i valori per x_4, x_5 e x_6; ci sono dunque infinite soluzioni per questo sistema di equazioni. Una soluzione è ammissibile se tutte le variabili $x_1, x_2, \ldots, x_6$ sono non negative; possono esistere anche infinite soluzioni ammissibili. Il numero infinito di soluzioni possibili per un sistema come questo sarà utile nelle successive dimostrazioni. Concentreremo la nostra attenzione sulla ***soluzione di base***: poniamo a zero tutte le variabili (non di base) nel lato destro e poi calcoliamo i valori delle variabili (di base) sul lato sinistro. In questo esempio, la soluzione di base è $(\bar{x_1}, \bar{x_2}, \ldots, \bar{x_6}) = (0, 0, 0, 30, 24, 36)$, il cui valore obiettivo è $z = (3 \cdot 0) + (1 \cdot 0) + (2 \cdot 0) = 0$. Notate che questa soluzione di base pone $\bar{x}_i = b_i$ per ogni $i \in B$. Un'iterazione dell'algoritmo del simplesso riscriverà il sistema delle equazioni e la funzione obiettivo mettendo un insieme diverso di variabili nel lato destro. Quindi, ci sarà una soluzione di base differente associata al problema riscritto.

Ribadiamo il concetto che riscrivere il programma lineare non cambia in alcun modo il problema di programmazione lineare sottostante; il problema in un'iterazione ha lo stesso insieme di soluzioni ammissibili che aveva nella precedente iterazione. Il problema, però, ha una soluzione di base diversa da quella della precedente iterazione.

Acquistato da Michele Mastrolonardo (mastrolonardo@gmail.com) - Copyright © 2010 The McGraw-Hill Education (Italy)

Se una soluzione di base è anche ammissibile, la chiameremo ***soluzione di base ammissibile***. Durante l'esecuzione dell'algoritmo del simplesso, la soluzione di base sarà quasi sempre una soluzione di base ammissibile. Tuttavia, come vedremo nel Paragrafo 29.6, per le prime iterazioni dell'algoritmo del simplesso la soluzione di base potrebbe non essere ammissibile.

Il nostro obiettivo, in ogni iterazione, è riformulare il programma lineare in modo che la soluzione di base abbia un valore obiettivo più grande. Selezioniamo una variabile non di base x_e il cui coefficiente nella funzione obiettivo è positivo; incrementiamo il valore di x_e quanto più possibile, senza violare alcun vincolo. La variabile x_e diventa variabile di base e qualche altra variabile x_l diventa variabile non di base. Anche i valori di altre variabili di base e quello della funzione obiettivo possono cambiare.

Per continuare l'esempio, esaminiamo l'incremento del valore di x_1. Quando aumentiamo x_1, i valori di x_4, x_5 e x_6 diminuiscono tutti. Poiché abbiamo un vincolo di non negatività per ogni variabile, non possiamo consentire a nessuna di esse di diventare negativa. Se x_1 cresce oltre il valore 30, allora x_4 diventa negativa, mentre x_5 e x_6 diventano negative quando x_1 supera, rispettivamente, 12 e 9. Il terzo vincolo (29.61) è quello più stretto e limita l'entità dell'incremento di x_1. Quindi, scambiamo i ruoli di x_1 e x_6. Risolvendo l'equazione (29.61) rispetto a x_1, otteniamo

$$x_1 = 9 - \frac{x_2}{4} - \frac{x_3}{2} - \frac{x_6}{4} \tag{29.62}$$

Per riscrivere le altre equazioni con x_6 nel lato destro, sostituiamo x_1 utilizzando l'equazione (29.62). Facendo così nell'equazione (29.59), otteniamo

$$\begin{aligned} x_4 &= 30 - x_1 - x_2 - 3x_3 \\ &= 30 - \left(9 - \frac{x_2}{4} - \frac{x_3}{2} - \frac{x_6}{4}\right) - x_2 - 3x_3 \\ &= 21 - \frac{3x_2}{4} - \frac{5x_3}{2} + \frac{x_6}{4} \end{aligned} \tag{29.63}$$

Analogamente, possiamo combinare l'equazione (29.62) con il vincolo (29.60) e la funzione obiettivo (29.58) per riscrivere il programma lineare nella seguente forma:

$$z = 27 + \frac{x_2}{4} + \frac{x_3}{2} - \frac{3x_6}{4} \tag{29.64}$$

$$x_1 = 9 - \frac{x_2}{4} - \frac{x_3}{2} - \frac{x_6}{4} \tag{29.65}$$

$$x_4 = 21 - \frac{3x_2}{4} - \frac{5x_3}{2} + \frac{x_6}{4} \tag{29.66}$$

$$x_5 = 6 - \frac{3x_2}{2} - 4x_3 + \frac{x_6}{2} \tag{29.67}$$

Questa operazione è detta ***pivoting*** (rotazione attorno a un perno: il pivot). Come visto in precedenza, un'operazione di pivoting sceglie una variabile non di base x_e, detta ***variabile entrante***, e una variabile di base x_l, detta ***variabile uscente***, e scambia i loro ruoli.

Il programma lineare descritto dalle equazioni (29.64)–(29.67) è equivalente al programma lineare descritto dalle equazioni (29.58)–(29.61). Le operazioni che eseguiamo nell'algoritmo del simplesso sono due: riscrivere le equazioni, spostando le variabili tra il lato sinistro e il lato destro, e sostituire un'equazione in un'altra. La prima operazione crea banalmente un problema equivalente; anche la seconda operazione, per l'algebra lineare elementare, crea un problema equivalente (vedere l'Esercizio 29.4-3).

Acquistato da Michele Michele su Webster il 2022-[illegible] Copyright [illegible] 2015 MIT Press - Vietata la vendita - All rights reserved - Italy

Per dimostrare questa equivalenza, notiamo che la soluzione di base originale $(0, 0, 0, 30, 24, 36)$ soddisfa le nuove equazioni (29.65)–(29.67) e ha il va-

lore obiettivo $27 + (1/4) \cdot 0 + (1/2) \cdot 0 - (3/4) \cdot 36 = 0$. La soluzione di base associata al nuovo programma lineare pone a zero le variabili non di base ed è $(9, 0, 0, 21, 6, 0)$, con valore obiettivo $z = 27$. Con semplici passaggi aritmetici possiamo verificare che questa soluzione soddisfa anche le equazioni (29.59)–(29.61) e, quando viene inserita nella funzione obiettivo (29.58), ha valore obiettivo $(3 \cdot 9) + (1 \cdot 0) + (2 \cdot 0) = 27$.

Continuando l'esempio, vogliamo trovare una nuova variabile di cui vogliamo incrementare il valore. Non vogliamo aumentare x_6 perché, quando il suo valore aumenta, il valore obiettivo diminuisce. Possiamo tentare di aumentare x_2 o x_3; scegliamo x_3. Di quanto possiamo aumentare x_3 senza violare alcun vincolo? Il vincolo (29.65) limita l'aumento a 18, il vincolo (29.66) lo limita a $42/5$ e il vincolo (29.67) lo limita a $3/2$. Il terzo vincolo è di nuovo il più stretto, pertanto lo riscriviamo in modo che x_3 si trovi nel lato sinistro e x_5 si trovi nel lato destro. Poi sostituiamo questa nuova equazione, $x_3 = 3/2 - 3x_2/8 - x_5/4 + x_6/8$, nelle equazioni (29.64)–(29.66) per ottenere il nuovo, ma equivalente, sistema

$$z = \frac{111}{4} + \frac{x_2}{16} - \frac{x_5}{8} - \frac{11x_6}{16} \tag{29.68}$$

$$x_1 = \frac{33}{4} - \frac{x_2}{16} + \frac{x_5}{8} - \frac{5x_6}{16} \tag{29.69}$$

$$x_3 = \frac{3}{2} - \frac{3x_2}{8} - \frac{x_5}{4} + \frac{x_6}{8} \tag{29.70}$$

$$x_4 = \frac{69}{4} + \frac{3x_2}{16} + \frac{5x_5}{8} - \frac{x_6}{16} \tag{29.71}$$

Questo sistema ha la soluzione di base associata $(33/4, 0, 3/2, 69/4, 0, 0)$, con valore obiettivo $111/4$. Adesso l'unico modo per incrementare il valore obiettivo è aumentare x_2. Ai tre vincoli corrispondono, rispettivamente, i limiti superiori 132, 4 e ∞. (Il limite superiore ∞ dato dal vincolo (29.71) deriva dal fatto che, quando aumentiamo x_2, aumenta anche il valore della variabile di base x_4. Questo vincolo, quindi, non pone alcuna restrizione sull'entità dell'incremento di x_2.) Aumentiamo x_2 a 4; x_2 diventa una variabile non di base. Poi risolviamo l'equazione (29.70) rispetto a x_2 e sostituiamo nelle altre equazioni per ottenere

$$z = 28 - \frac{x_3}{6} - \frac{x_5}{6} - \frac{2x_6}{3} \tag{29.72}$$

$$x_1 = 8 + \frac{x_3}{6} + \frac{x_5}{6} - \frac{x_6}{3} \tag{29.73}$$

$$x_2 = 4 - \frac{8x_3}{3} - \frac{2x_5}{3} + \frac{x_6}{3} \tag{29.74}$$

$$x_4 = 18 - \frac{x_3}{2} + \frac{x_5}{2} \tag{29.75}$$

A questo punto, tutti i coefficienti nella funzione obiettivo sono negativi. Come vedremo più avanti in questo capitolo, questa situazione si verifica soltanto quando avremo riscritto il programma lineare in modo che la soluzione di base sia una soluzione ottima. Quindi, per questo problema, la soluzione $(8, 4, 0, 18, 0, 0)$, con valore obiettivo 28, è ottima. Adesso possiamo ritornare al nostro programma lineare originale dato dalle equazioni (29.53)–(29.57). Le uniche variabili nel programma lineare originale sono x_1, x_2 e x_3, e quindi la nostra soluzione è $x_1 = 8$, $x_2 = 4$ e $x_3 = 0$, con valore obiettivo $(3 \cdot 8) + (1 \cdot 4) + (2 \cdot 0) = 28$. Notate che i valori delle variabili scarto nell'ultima soluzione misurano l'entità dello scar-

Acquistato da Michele Monaci (michele.monaci@unibo.it) - Proprietà letteraria riservata - © 2023 The McGraw-Hill Education (Italy)

to in ciascuna disuguaglianza. La variabile scarto x_4 è 18, e il lato sinistro della disuguaglianza (29.54), che ha valore $8 + 4 + 0 = 12$, differisce di 18 rispetto al lato destro che vale 30. Le variabili scarto x_5 e x_6 sono 0 e così, nelle disuguaglianze (29.55) e (29.56), il lato sinistro è uguale al lato destro. Notate inoltre che, anche se i coefficienti nella forma standard originale sono interi, i coefficienti negli altri programmi lineari non sono necessariamente interi, e le soluzioni intermedie non sono necessariamente intere. Inoltre, l'ultima soluzione di un programma lineare non è necessariamente intera; è soltanto una pura coincidenza se questo esempio ha una soluzione intera.

Pivoting

Adesso formalizziamo la procedura per l'operazione di pivoting. La procedura PIVOT riceve come input una forma standard, data dalla tupla (N, B, A, b, c, v), l'indice l della variabile uscente x_l e l'indice e della variabile entrante x_e. Restituisce la tupla $(\widehat{N}, \widehat{B}, \widehat{A}, \widehat{b}, \widehat{c}, \widehat{v})$ che descrive la nuova forma standard. (Ricordiamo di nuovo che gli elementi delle matrici A e $\widehat{A}$ sono gli opposti dei coefficienti che appaiono nella forma standard.)

```
PIVOT(N, B, A, b, c, v, l, e)
 1  // Calcola i coefficienti dell'equazione per la nuova variabile di base x_e.
 2  Sia Â una nuova matrice m × n
 3  b̂_e = b_l / a_le
 4  for ogni j ∈ N − {e}
 5      â_ej = a_lj / a_le
 6  â_el = 1 / a_le
 7  // Calcola i coefficienti degli altri vincoli.
 8  for ogni i ∈ B − {l}
 9      b̂_i = b_i − a_ie b̂_e
10      for ogni j ∈ N − {e}
11          â_ij = a_ij − a_ie â_ej
12      â_il = −a_ie â_el
13  // Calcola la funzione obiettivo.
14  v̂ = v + c_e b̂_e
15  for ogni j ∈ N − {e}
16      ĉ_j = c_j − c_e â_ej
17  ĉ_l = −c_e â_el
18  // Calcola i nuovi insiemi di variabili di base e non di base.
19  N̂ = N − {e} ∪ {l}
20  B̂ = B − {l} ∪ {e}
21  return (N̂, B̂, Â, b̂, ĉ, v̂)
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

PIVOT opera nel modo seguente. Le righe 3–6 calcolano i coefficienti della nuova equazione per la variabile x_e, riscrivendo l'equazione che ha x_l nel lato sinistro perché abbia invece x_e nel lato sinistro. Le righe 8–12 aggiornano le restanti equazioni sostituendo ogni occorrenza di x_e con il lato destro di questa nuova equazione. Le righe 14–17 effettuano la stessa sostituzione per la funzione obiettivo; le righe 19 e 20 aggiornano gli insiemi delle variabili di base e non di base. La riga 21 restituisce la nuova forma standard. Se $a_{le} = 0$, così com'è scritta, la

procedura PIVOT causerebbe un errore dividendo per 0; tuttavia, come vedremo nella dimostrazione dei Lemmi 29.2 e 29.12, la procedura PIVOT viene chiamata soltanto se $a_{le} \neq 0$.

Adesso sintetizziamo l'effetto della procedura PIVOT sui valori delle variabili nella soluzione di base.

Lemma 29.1
Consideriamo una chiamata di PIVOT(N, B, A, b, c, v, l, e) in cui $a_{le} \neq 0$. Siano $(\widehat{N}, \widehat{B}, \widehat{A}, \widehat{b}, \widehat{c}, \widehat{v})$ i valori restituiti dalla chiamata e indichiamo con $\bar{x}$ la soluzione di base dopo la chiamata. Allora

1. $\bar{x}_j = 0$ per ogni $j \in \widehat{N}$.
2. $\bar{x}_e = b_l / a_{le}$.
3. $\bar{x}_i = b_i - a_{ie}\widehat{b}_e$ per ogni $i \in \widehat{B} - \{e\}$.

Dimostrazione La prima asserzione è vera perché la soluzione di base pone sempre a zero tutte le variabili non di base. Quando poniamo a zero ciascuna variabile non di base in un vincolo

$$x_i = \widehat{b}_i - \sum_{j \in \widehat{N}} \widehat{a}_{ij} x_j$$

abbiamo che $\bar{x}_i = \widehat{b}_i$ per ogni $i \in \widehat{B}$. Poiché $e \in \widehat{B}$, per la riga 3 di PIVOT abbiamo

$$\bar{x}_e = \widehat{b}_e = b_l / a_{le}$$

Questo dimostra la seconda asserzione. Analogamente, utilizzando la riga 9 per ogni $i \in \widehat{B} - \{e\}$, abbiamo

$$\bar{x}_i = \widehat{b}_i = b_i - a_{ie}\widehat{b}_e$$

Questo dimostra la terza asserzione. ■

L'algoritmo formale del simplesso

Adesso possiamo formalizzare l'algoritmo del simplesso, che abbiamo illustrato con un esempio. Quell'esempio era particolarmente interessante, ma ci sarebbero molte altre questioni da considerare:

- Come facciamo a determinare se un programma lineare è ammissibile?
- Che cosa facciamo se il programma lineare è ammissibile, ma la soluzione di base iniziale non è ammissibile?
- Come facciamo a determinare se un programma lineare è illimitato?
- Come facciamo a scegliere le variabili entranti e uscenti?

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Nel Paragrafo 29.6 spiegheremo come determinare se un problema è ammissibile e, nel caso in cui lo sia, come trovare una forma standard nella quale la soluzione di base iniziale è ammissibile. Pertanto, supponiamo di avere una procedura INITIALIZE-SIMPLEX(A, b, c) che riceve come input un programma lineare nella forma canonica, ovvero una matrice $A = (a_{ij})$ di dimensioni $m \times n$, un vettore

$b = (b_i)$ di dimensione m e un vettore $c = (c_j)$ di dimensione n. Se il problema è inammissibile, la procedura restituisce un messaggio per segnalare che il programma è inammissibile e poi termina; altrimenti restituisce una forma standard per la quale la soluzione di base iniziale è ammissibile.

La procedura SIMPLEX riceve come input un programma lineare nella forma canonica, come abbiamo appena descritto. Restituisce un vettore $\bar{x} = (\bar{x}_j)$ di dimensione n che è una soluzione ottima per il programma lineare descritto dalle equazioni (29.19)–(29.21).

```
SIMPLEX(A, b, c)
 1  (N, B, A, b, c, v) = INITIALIZE-SIMPLEX(A, b, c)
 2  Sia Δ un nuovo vettore di dimensione n
 3  while qualche indice j ∈ N ha c_j > 0
 4      sceglie un indice e ∈ N per il quale c_e > 0
 5      for ogni indice i ∈ B
 6          if a_ie > 0
 7              Δ_i = b_i / a_ie
 8          else Δ_i = ∞
 9      sceglie un indice l ∈ B che minimizza Δ_i
10      if Δ_l == ∞
11          return "illimitato"
12      else (N, B, A, b, c, v) = PIVOT(N, B, A, b, c, v, l, e)
13  for i = 1 to n
14      if i ∈ B
15          x̄_i = b_i
16      else x̄_i = 0
17  return (x̄_1, x̄_2, ..., x̄_n)
```

La procedura SIMPLEX opera nel modo seguente. La riga 1 chiama la procedura INITIALIZE-SIMPLEX(A, b, c), precedentemente descritta, che decide se il programma lineare è ammissibile oppure restituisce una forma standard per la quale la soluzione di base è ammissibile.

La parte principale dell'algoritmo è il ciclo **while** (righe 3–12). Se tutti i coefficienti nella funzione obiettivo sono negativi, il ciclo **while** termina; altrimenti la riga 4 seleziona come variabile entrante una variabile x_e il cui coefficiente nella funzione obiettivo è positivo. Sebbene siamo liberi di scegliere come variabile entrante qualsiasi variabile di tale tipo, tuttavia supponiamo di utilizzare qualche regola deterministica prestabilita. Poi, le righe 5–9 controllano ogni vincolo e selezionano quello che limita maggiormente l'entità di incremento di x_e senza violare alcun vincolo di non negatività; la variabile di base associata a questo vincolo è x_l. Anche qui, siamo liberi di scegliere una delle tante variabili come variabile uscente, tuttavia supponiamo di utilizzare qualche regola deterministica prestabilita. Se nessuno dei vincoli limita l'entità di incremento della variabile entrante, l'algoritmo restituisce "illimitato" nella riga 11, altrimenti la riga 12 scambia i ruoli delle variabili entrante e uscente chiamando la subroutine PIVOT(N, B, A, b, c, v, l, e), precedentemente descritta. Le righe 13–16 calcolano una soluzione per le variabili originali della programmazione lineare $\bar{x}_1, \bar{x}_2, \ldots, \bar{x}_n$, ponendo a zero tutte le variabili non di base e a b_i ogni variabile di base $\bar{x}_i$; la riga 17 restituisce questi valori.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Per dimostrare che la procedura SIMPLEX è corretta, prima dimostriamo che, se SIMPLEX ha una soluzione ammissibile iniziale e poi termina, allora restituisce una soluzione ammissibile oppure determina che il programma lineare è illimitato. Poi, proveremo che SIMPLEX termina. Infine, nel Paragrafo 29.5 (Teorema 29.10) dimostreremo che la soluzione restituita è ottima.

Lemma 29.2
Dato un programma lineare (A, b, c), supponiamo che la chiamata di INITIALIZE-SIMPLEX nella riga 1 di SIMPLEX restituisca una forma standard per la quale la soluzione di base è ammissibile. Allora, se SIMPLEX restituisce una soluzione nella riga 17, questa soluzione è ammissibile per il programma lineare. Se SIMPLEX restituisce "illimitato" nella riga 11, il programma lineare è illimitato.

Dimostrazione Utilizziamo la seguente invariante di ciclo composta da tre parti:

All'inizio di ogni iterazione del ciclo **while** (righe 3–12):

1. La forma standard è equivalente alla forma standard restituita dalla chiamata di INITIALIZE-SIMPLEX.
2. Per ogni $i \in B$, si ha $b_i \geq 0$.
3. La soluzione di base associata alla forma standard è ammissibile.

Inizializzazione: l'equivalenza delle forme standard è banale per la prima iterazione. Nell'asserzione del lemma, noi supponiamo che la chiamata di INITIALIZE-SIMPLEX nella riga 1 di SIMPLEX restituisca una forma standard per la quale la soluzione di base è ammissibile. Quindi, la terza parte dell'invariante è vera. Inoltre, poiché ogni variabile di base x_i è impostata a b_i nella soluzione di base, e l'ammissibilità della soluzione di base implica che ogni variabile di base x_i non è negativa, si ha che $b_i \geq 0$. Dunque, la seconda parte dell'invariante è vera.

Conservazione: dimostreremo che l'invariante di ciclo si conserva, supponendo che l'istruzione **return** nella riga 11 non sia eseguita. Analizzeremo il caso in cui viene eseguita la riga 11 quando parleremo della conclusione del ciclo.

Un'iterazione del ciclo **while** scambia il ruolo di una variabile di base con quello di una variabile non di base, chiamando la procedura PIVOT. Per l'Esercizio 29.4-3, la forma standard è equivalente a quella della precedente iterazione che, per l'invariante di ciclo, è equivalente alla forma standard iniziale.

Adesso dimostriamo la seconda parte dell'invariante di ciclo. Supponiamo che, all'inizio di ogni iterazione del ciclo **while**, $b_i \geq 0$ per ogni $i \in B$; dimostreremo che queste disuguaglianze restano vere dopo la chiamata di PIVOT nella riga 12. È sufficiente dimostrare che la riga 12 conserva questa parte dell'invariante, poiché le uniche modifiche delle variabili b_i e dell'insieme B delle variabili di base si verificano con questa assegnazione. Indichiamo con b_i, a_{ij} e B i valori prima della chiamata di PIVOT e con $\widehat{b}_i$ i valori restituiti da PIVOT.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1998036716-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Innanzi tutto notiamo che $\widehat{b}_e \geq 0$ perché $b_l \geq 0$ per l'invariante di ciclo, $a_{le} > 0$ per le righe 6 e 9 di SIMPLEX e $\widehat{b}_e = b_l / a_{le}$ per la riga 3 di PIVOT. Per i restanti indici $i \in B - \{l\}$, si ha

$$\begin{aligned} \widehat{b}_i &= b_i - a_{ie}\widehat{b}_e && \text{(per la riga 9 di PIVOT)} \\ &= b_i - a_{ie}(b_l / a_{le}) && \text{(per la riga 3 di PIVOT)} \end{aligned} \tag{29.76}$$

Abbiamo due casi da considerare, a seconda che $a_{ie} > 0$ o $a_{ie} \leq 0$. Se $a_{ie} > 0$, allora poiché scegliamo l tale che

$$b_l/a_{le} \leq b_i/a_{ie} \quad \text{per ogni } i \in B \tag{29.77}$$

abbiamo

$$\begin{aligned}
\widehat{b}_i &= b_i - a_{ie}(b_l/a_{le}) \quad \text{(per l'equazione (29.76))} \\
&\geq b_i - a_{ie}(b_i/a_{ie}) \quad \text{(per la disuguaglianza (29.77))} \\
&= b_i - b_i \\
&= 0
\end{aligned}$$

Ne consegue che $\widehat{b}_i \geq 0$. Se $a_{ie} \leq 0$, allora poiché a_{le}, b_i e b_l sono tutti valori non negativi, l'equazione (29.76) implica che anche $\widehat{b}_i$ deve essere non negativo. Adesso dimostriamo che la soluzione di base è ammissibile, ovvero che tutte le variabili hanno valori non negativi. Le variabili non di base sono poste a zero e quindi non sono negative. Ogni variabile di base x_i è definita dall'equazione

$$x_i = b_i - \sum_{j \in N} a_{ij} x_j$$

La soluzione di base pone $\bar{x}_i = b_i$. Utilizzando la seconda parte dell'invariante di ciclo, concludiamo che ogni variabile di base $\bar{x}_i$ non è negativa.

Conclusione: il ciclo **while** può terminare in uno dei due modi possibili. Se termina per la condizione nella riga 3, allora la soluzione di base corrente è ammissibile e questa soluzione è restituita nella riga 17. Il ciclo può anche terminare restituendo "illimitato" nella riga 11. In questo caso, per ogni iterazione del ciclo **for** (righe 5–8), quando viene eseguita la riga 6, si ha $a_{ie} \leq 0$. Consideriamo la soluzione $\bar{x}$ definita come

$$\bar{x}_i = \begin{cases} \infty & \text{se } i = e \\ 0 & \text{se } i \in N - \{e\} \\ b_i - \sum_{j \in N} a_{ij} \bar{x}_j & \text{se } i \in B \end{cases}$$

Adesso dimostriamo che questa soluzione è ammissibile, cioè che tutte le variabili sono non negative. Le variabili non di base diverse da $\bar{x}_e$ sono nulle e $\bar{x}_e = \infty > 0$; quindi tutte le variabili non di base sono non negative. Per ogni variabile di base $\bar{x}_i$, abbiamo

$$\begin{aligned}
\bar{x}_i &= b_i - \sum_{j \in N} a_{ij} \bar{x}_j \\
&= b_i - a_{ie} \bar{x}_e
\end{aligned}$$

L'invariante di ciclo implica che $b_i \geq 0$ e abbiamo $a_{ie} \leq 0$ e $\bar{x}_e = \infty > 0$. Quindi, $\bar{x}_i \geq 0$. Adesso dimostriamo che il valore obiettivo per la soluzione $\bar{x}$ è illimitato. Dall'equazione (29.42) il valore obiettivo è

$$\begin{aligned}
z &= v + \sum_{j \in N} c_j \bar{x}_j \\
&= v + c_e \bar{x}_e
\end{aligned}$$

Poiché $c_e > 0$ (per la riga 4) e $\bar{x}_e = \infty$, il valore obiettivo è ∞, e quindi il programma lineare è illimitato. ■

Acquistato da Michele Michele su Webster il 2022-07-07 a 12:13 (Numero Ordine Libreria 1475208616 2207707 1 Copyright © 2022 McGraw-Hill Education (Italy))

Resta da dimostrare che la procedura SIMPLEX termina, e quando termina, la soluzione restituita è ottima. Il Paragrafo 29.5 esaminerà l'ottimalità. Adesso dimostriamo che SIMPLEX termina.

Terminazione della procedura

Nell'esempio descritto all'inizio di questo paragrafo, ogni iterazione dell'algoritmo del simplesso aumentava il valore obiettivo associato alla soluzione di base. Come l'Esercizio 29.4-2 chiede di dimostrare, nessuna iterazione di SIMPLEX può ridurre il valore obiettivo associato alla soluzione di base. Purtroppo, è possibile che un'iterazione lasci inalterato il valore obiettivo. Questo fenomeno è detto ***degenerazione***, che adesso analizziamo dettagliatamente.

Il valore obiettivo viene modificato dall'assegnazione $\hat{v} = v + c_e\hat{b}_e$ nella riga 14 di PIVOT. Poiché SIMPLEX chiama PIVOT soltanto quando $c_e > 0$, l'unico modo in cui il valore obiettivo può restare inalterato (cioè $\hat{v} = v$) è che $\hat{b}_e$ sia zero. Questo valore viene assegnato come $\hat{b}_e = b_l/a_{le}$ nella riga 3 di PIVOT. Poiché chiamiamo sempre PIVOT con $a_{le} \neq 0$, perché $\hat{b}_e$ sia zero e, quindi, il valore obiettivo resti inalterato, deve essere $b_l = 0$.

Questo evento può verificarsi davvero. Consideriamo il programma lineare

$$\begin{aligned} z &= x_1 + x_2 + x_3 \\ x_4 &= 8 - x_1 - x_2 \\ x_5 &= x_2 - x_3 \end{aligned}$$

Supponiamo di scegliere x_1 come variabile entrante e x_4 come variabile uscente. Dopo il pivoting, otteniamo

$$\begin{aligned} z &= 8 + x_3 - x_4 \\ x_1 &= 8 - x_2 - x_4 \\ x_5 &= x_2 - x_3 \end{aligned}$$

A questo punto, l'unica scelta è il pivoting con x_3 entrante e x_5 uscente. Poiché $b_5 = 0$, il valore obiettivo 8 resta inalterato dopo il pivoting:

$$\begin{aligned} z &= 8 + x_2 - x_4 - x_5 \\ x_1 &= 8 - x_2 - x_4 \\ x_3 &= x_2 - x_5 \end{aligned}$$

Il valore obiettivo non è cambiato, ma è cambiata la nostra forma standard. Fortunatamente, se ripetiamo il pivoting, con x_2 entrante e x_1 uscente, il valore obiettivo aumenterà (a 16), e l'algoritmo del simplesso potrà continuare.

La degenerazione può impedire all'algoritmo del simplesso di terminare, perché potrebbe portare a un fenomeno detto ***ciclicità***: le forme standard in due diverse iterazioni di SIMPLEX sono identiche. A causa della degenerazione, SIMPLEX potrebbe scegliere una sequenza di operazioni di pivoting che lasciano invariato il valore obiettivo, ma fanno ritornare a una stessa forma standard. Essendo SIMPLEX un algoritmo deterministico, se si genera una ciclicità, esso elaborerà per sempre la stessa serie di forme standard, senza mai terminare.

La ciclicità è l'unica ragione per la quale SIMPLEX potrebbe non terminare. Per dimostrare questo fatto, dobbiamo prima sviluppare qualche altro strumento di analisi.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

A ogni iterazione, SIMPLEX mantiene A, b, c e v oltre agli insiemi N e B. Sebbene mantenere esplicitamente A, b, c e v sia essenziale per un'implementazione efficiente dell'algoritmo del simplesso, tuttavia non è strettamente necessario. In altre parole, la forma standard è unicamente determinata dagli insiemi delle variabili di base e non di base. Prima di provare questo fatto, dimostriamo un utile lemma algebrico.

Lemma 29.3
Sia I un insieme di indici. Per ogni $i \in I$, siano α_i e β_i numeri reali e sia x_i una variabile reale. Sia γ un numero reale qualsiasi. Supponiamo che per qualsiasi valore di x_i si abbia

$$\sum_{i \in I} \alpha_i x_i = \gamma + \sum_{i \in I} \beta_i x_i \tag{29.78}$$

Allora $\alpha_i = \beta_i$ per ogni $i \in I$ e $\gamma = 0$.

Dimostrazione Poiché l'equazione (29.78) è vera per qualsiasi valore di x_i, possiamo utilizzare dei valori particolari per trarre delle conclusioni su α, β e γ. Se poniamo $x_i = 0$ per ogni $i \in I$, concludiamo che $\gamma = 0$. Adesso scegliamo un indice arbitrario $i \in I$ e poniamo $x_i = 1$ e $x_k = 0$ per ogni $k \neq i$. Si ottiene che deve essere $\alpha_i = \beta_i$. Poiché abbiamo scelto i come un indice qualsiasi in I, concludiamo che $\alpha_i = \beta_i$ per ogni $i \in I$. ■

Un particolare programma lineare ha varie forme standard; ricordiamo che ogni forma standard ha lo stesso insieme di soluzioni ammissibili e ottime del programma lineare originale. Adesso dimostriamo che la forma standard di un programma lineare è determinata unicamente dall'insieme delle variabili di base. Ovvero, dato un insieme di variabili di base, a tali variabili è associata un'unica forma standard (un unico insieme di coefficienti e lati destri).

Lemma 29.4
Sia (A, b, c) un programma lineare nella forma canonica. Dato un insieme B di variabili di base, la forma standard associata è univocamente determinata.

Dimostrazione Supponiamo per assurdo che ci siano due forme standard differenti con lo stesso insieme B di variabili di base. Le forme standard devono avere anche gli stessi insiemi $N = \{1, 2, \ldots, n + m\} - B$ di variabili non di base. Scriviamo la prima forma standard così

$$z = v + \sum_{j \in N} c_j x_j \tag{29.79}$$

$$x_i = b_i - \sum_{j \in N} a_{ij} x_j \quad \text{for } i \in B \tag{29.80}$$

e la seconda così

$$z = v' + \sum_{j \in N} c'_j x_j \tag{29.81}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$x_i = b'_i - \sum_{j \in N} a'_{ij} x_j \quad \text{for } i \in B \tag{29.82}$$

Consideriamo il sistema delle equazioni che si ottiene sottraendo ciascuna equazione (29.82) dalla corrispondente equazione (29.80). Il sistema risultante è

$$0 = (b_i - b'_i) - \sum_{j \in N} (a_{ij} - a'_{ij}) x_j \quad \text{per } i \in B$$

ovvero

$$\sum_{j \in N} a_{ij} x_j = (b_i - b'_i) + \sum_{j \in N} a'_{ij} x_j \quad \text{per } i \in B$$

Adesso, per ogni $i \in B$, applichiamo il Lemma 29.3 con $\alpha_i = a_{ij}$, $\beta_i = a'_{ij}$ e $\gamma = b_i - b'_i$. Poiché $\alpha_i = \beta_i$, si ha $a_{ij} = a'_{ij}$ per ogni $j \in N$ e, poiché $\gamma = 0$, si ha $b_i = b'_i$. Quindi, per le due forme standard, A e b sono uguali ad A' e b'. Con un ragionamento analogo, l'Esercizio 29.4-1 dimostra che deve essere anche $c = c'$ e $\nu = \nu'$, e quindi le due forme standard devono essere identiche. ■

Adesso possiamo dimostrare che la ciclicità è l'unica ragione per la quale SIMPLEX potrebbe non terminare.

Lemma 29.5
Se la procedura SIMPLEX non termina entro $\binom{n+m}{m}$ iterazioni, allora entra in un ciclo infinito.

Dimostrazione Per il Lemma 29.4, l'insieme B delle variabili di base determina univocamente una forma standard. Poiché ci sono $n + m$ variabili e $|B| = m$, ci sono $\binom{n+m}{m}$ modi di scegliere B. Quindi, ci sono soltanto $\binom{n+m}{m}$ forme standard distinte. Di conseguenza, se la procedura SIMPLEX esegue più di $\binom{n+m}{m}$ iterazioni, deve essere entrata in un ciclo infinito. ■

La ciclicità è un fenomeno teoricamente possibile, ma estremamente raro. Può essere evitato scegliendo con molta attenzione le variabili entranti e uscenti. Una tecnica consiste nel perturbare leggermente l'input in modo che sia impossibile avere due soluzioni con lo stesso valore obiettivo. Un'altra tecnica consiste nel risolvere i casi di uguaglianza scegliendo sempre la variabile con l'indice più piccolo. Quest'ultima strategia è nota come ***regola di Bland***. Omettiamo la dimostrazione che queste strategie evitano la ciclicità.

Lemma 29.6
Se nelle righe 4 e 9 della procedura SIMPLEX i casi di uguaglianza sono risolti sempre scegliendo la variabile con l'indice minimo, allora SIMPLEX deve terminare. ■

Concludiamo questo paragrafo con il seguente lemma.

Lemma 29.7
Supponendo che la procedura INITIALIZE-SIMPLEX restituisca una forma standard per la quale la soluzione di base è ammissibile, la procedura SIMPLEX segnala che un programma lineare è illimitato oppure termina con una soluzione ammissibile entro $\binom{n+m}{m}$ iterazioni.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Dimostrazione I Lemmi 29.2 e 29.6 dimostrano che, se INITIALIZE-SIMPLEX restituisce una forma standard per la quale la soluzione di base è ammissibile, la procedura SIMPLEX segnala che un programma lineare è illimitato oppure termina con una soluzione ammissibile. Per contrapposizione del Lemma 29.5, se la procedura SIMPLEX termina con una soluzione ammissibile, allora termina entro $\binom{n+m}{m}$ iterazioni. ■

Esercizi

29.4-1

Completate la dimostrazione del Lemma 29.4 dimostrando che deve essere $c = c'$ e $v = v'$.

29.4-2

Dimostrate che la chiamata della procedura PIVOT nella riga 12 di SIMPLEX non ridurrà mai il valore di v.

29.4-3

Dimostrate che la forma standard passata alla procedura PIVOT e la forma standard che la procedura restituisce sono equivalenti.

29.4-4

Supponete di trasformare un programma lineare (A, b, c) in forma canonica nella forma standard. Dimostrate che la soluzione di base è ammissibile se e soltanto se $b_i \geq 0$ per $i = 1, 2, \ldots, m$.

29.4-5

Risolvete il seguente programma lineare utilizzando la procedura SIMPLEX:

$$\begin{aligned} &\text{massimizzare} && 18x_1 + 12,5x_2 \\ &\text{con le condizioni} \\ & && x_1 + x_2 \leq 20 \\ & && x_1 \leq 12 \\ & && x_2 \leq 16 \\ & && x_1, x_2 \geq 0 \end{aligned}$$

29.4-6

Risolvete il seguente programma lineare utilizzando la procedura SIMPLEX:

$$\begin{aligned} &\text{massimizzare} && -5x_1 - 3x_2 \\ &\text{con le condizioni} \\ & && x_1 - x_2 \leq 1 \\ & && 2x_1 + x_2 \leq 2 \\ & && x_1, x_2 \geq 0 \end{aligned}$$

29.4-7

Risolvete il seguente programma lineare utilizzando la procedura SIMPLEX:

$$\begin{aligned} &\text{minimizzare} && x_1 + x_2 + x_3 \\ &\text{con le condizioni} \\ & && 2x_1 + 7,5x_2 + 3x_3 \geq 10\,000 \\ & && 20x_1 + 5x_2 + 10x_3 \geq 30\,000 \\ & && x_1, x_2, x_3 \geq 0 \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

29.4-8

Dimostrando il Lemma 29.5, si è detto che ci sono al più $\binom{m+n}{n}$ modi di scegliere un insieme B di variabili di base. Trovate un esempio di programma lineare in cui il numero di modi di scegliere l'insieme B è strettamente minore di $\binom{m+n}{n}$.

29.5 Dualità

Abbiamo dimostrato che, sotto certe ipotesi, la procedura SIMPLEX termina. Tuttavia, non abbiamo ancora dimostrato che questa procedura trova effettivamente la soluzione ottima per un programma lineare. Per farlo, introduciamo un concetto potente: ***la dualità nella programmazione lineare***.

La dualità ci consente di dimostrare che una soluzione è ottima. Abbiamo visto un esempio di dualità nel Capitolo 26 con il Teorema 26.6 del "flusso-massimo/taglio-minimo". Per esempio, data un'istanza di un problema di flusso massimo, supponiamo di trovare un flusso f di valore $|f|$. Come facciamo a sapere se f è un flusso massimo? Per il teorema del flusso-massimo/taglio-minimo, se possiamo trovare un taglio il cui valore è anch'esso $|f|$, allora abbiamo verificato che f è davvero un flusso massimo. Questo è un esempio di dualità: dato un problema di massimizzazione, definiamo un corrispondente problema di minimizzazione in modo tale che i due problemi abbiano lo stesso valore obiettivo ottimo.

Dato un programma lineare in cui deve essere massimizzato l'obiettivo, spiegheremo come formulare un programma lineare ***duale*** in cui deve essere minimizzato l'obiettivo e il cui valore ottimo è identico a quello del programma lineare originale. Quando facciamo riferimento ai programmi lineari duali, il programma lineare originale è detto ***primale***.

Dato un programma lineare primale in forma canonica, secondo le espressioni (29.16)–(29.18), definiamo il programma lineare duale in questo modo:

$$\text{minimizzare} \qquad \sum_{i=1}^{m} b_i y_i \tag{29.83}$$

con le condizioni

$$\sum_{i=1}^{m} a_{ij} y_i \geq c_j \quad \text{per } j = 1, 2, \ldots, n \tag{29.84}$$

$$y_i \geq 0 \quad \text{per } i = 1, 2, \ldots, m \tag{29.85}$$

Per formare il duale, cambiamo l'obiettivo di massimizzare nell'obiettivo di minimizzare, scambiamo i ruoli dei coefficienti nei lati destri dei vincoli con i coefficienti della funzione obiettivo e sostituiamo il segno minore-o-uguale con il segno maggiore-o-uguale. Ciascuno degli m vincoli nel primale ha una variabile associata y_i nel duale e ciascuno degli n vincoli nel duale ha una variabile associata x_j nel primale. Per esempio, consideriamo il programma lineare (29.53)–(29.57). Il duale di questo programma lineare è

$$\text{minimizzare} \qquad 30y_1 + 24y_2 + 36y_3 \tag{29.86}$$

con le condizioni

$$y_1 + 2y_2 + 4y_3 \geq 3 \tag{29.87}$$

$$y_1 + 2y_2 + y_3 \geq 1 \tag{29.88}$$

$$3y_1 + 5y_2 + 2y_3 \geq 2 \tag{29.89}$$

$$y_1, y_2, y_3 \geq 0 \tag{29.90}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

Nel Teorema 29.10 dimostreremo che il valore ottimo del programma lineare duale è sempre uguale al valore ottimo del programma lineare primale. Inoltre, l'algoritmo del simplesso in effetti risolve implicitamente i programmi lineari primale e duale nello stesso tempo, fornendo così la prova dell'ottimalità.

Iniziamo dimostrando la ***dualità debole***, che stabilisce che qualsiasi soluzione ammissibile per il programma lineare primale ha un valore non maggiore di quello di qualsiasi soluzione ammissibile per il programma lineare duale.

Lemma 29.8 (Dualità debole nella programmazione lineare)
Sia $\bar{x}$ una soluzione ammissibile qualsiasi per il programma lineare primale (29.16)–(29.18) e sia $\bar{y}$ una soluzione ammissibile qualsiasi per il programma lineare duale (29.83)–(29.85). Allora

$$\sum_{j=1}^{n} c_j \bar{x}_j \leq \sum_{i=1}^{m} b_i \bar{y}_i$$

Dimostrazione Abbiamo

$$\begin{aligned}
\sum_{j=1}^{n} c_j \bar{x}_j &\leq \sum_{j=1}^{n} \left(\sum_{i=1}^{m} a_{ij} \bar{y}_i \right) \bar{x}_j && \text{(per l'espressione (29.84))} \\
&= \sum_{i=1}^{m} \left(\sum_{j=1}^{n} a_{ij} \bar{x}_j \right) \bar{y}_i \\
&\leq \sum_{i=1}^{m} b_i \bar{y}_i && \text{(per l'espressione (29.17))}
\end{aligned}$$

■

Corollario 29.9
Sia $\bar{x}$ una soluzione ammissibile per un programma lineare primale (A, b, c) e sia $\bar{y}$ una soluzione ammissibile per il corrispondente programma lineare duale. Se

$$\sum_{j=1}^{n} c_j \bar{x}_j = \sum_{i=1}^{m} b_i \bar{y}_i$$

allora $\bar{x}$ e $\bar{y}$ sono soluzioni ottime, rispettivamente, per il primale e il duale.

Dimostrazione Per il Lemma 29.8, il valore obiettivo di una soluzione ammissibile per il primale non può superare quello di una soluzione ammissibile per il duale. Il programma lineare primale è un problema di massimizzazione e il duale è un problema di minimizzazione. Quindi, se le soluzioni ammissibili $\bar{x}$ e $\bar{y}$ hanno lo stesso valore obiettivo, nessuna delle due potrà essere migliorata. ■

Prima di dimostrare che esiste sempre una soluzione duale il cui valore è uguale a quello di una soluzione primale ottima, descriviamo come trovare una tale soluzione. Quando abbiamo applicato l'algoritmo del simplesso al programma lineare (29.53)–(29.57), l'ultima iterazione ha fornito la forma standard (29.72)–(29.75) con funzione obiettivo $z = 28 - x_3/6 - x_5/6 - 2x_6/3$, $B = \{1, 2, 4\}$ ed $N = \{3, 5, 6\}$. Come vedremo più avanti, la soluzione di base associata all'ultima forma standard è una soluzione ottima per il programma lineare; di conseguenza, una soluzione ottima per il programma lineare (29.53)–(29.57) è $(\bar{x}_1, \bar{x}_2, \bar{x}_3) = (8, 4, 0)$, con valore obiettivo $(3 \cdot 8) + (1 \cdot 4) + (2 \cdot 0) = 28$. Come vedremo dopo, possiamo ricavare una soluzione duale ottima: gli opposti dei coefficienti della funzione obiettivo primale sono i valori delle variabili duali. Più precisamente, supponiamo che l'ultima forma standard del primale sia

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 10950301:220707:0. Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned}
z &= v' + \sum_{j \in N} c'_j x_j \\
x_i &= b'_i - \sum_{j \in N} a'_{ij} x_j \quad \text{per } i \in B
\end{aligned}$$

Allora una soluzione duale ottima può essere ottenuta ponendo

$$\bar{y}_i = \begin{cases} -c'_{n+i} & \text{se } (n+i) \in N \\ 0 & \text{negli altri casi} \end{cases} \tag{29.91}$$

Quindi, una soluzione ottima per il programma lineare duale definito dalle espressioni (29.86)–(29.90) è $\bar{y}_1 = 0$ (perché $n + 1 = 4 \in B$), $\bar{y}_2 = -c'_5 = 1/6$ e $\bar{y}_3 = -c'_6 = 2/3$. Calcolando la funzione obiettivo duale (29.86), otteniamo un valore obiettivo pari a $(30 \cdot 0) + (24 \cdot (1/6)) + (36 \cdot (2/3)) = 28$, che conferma che il valore obiettivo del primale è davvero uguale al valore obiettivo del duale. Combinando questi calcoli con il Lemma 29.8, abbiamo una prova che il valore obiettivo ottimo del programma lineare primale è 28. Dimostriamo adesso che, in generale, una soluzione ottima per il duale e una prova dell'ottimalità di una soluzione per il primale, possono essere ottenute in questo modo.

Teorema 29.10 (Dualità nella programmazione lineare)
Supponiamo che la procedura SIMPLEX restituisca i valori $\bar{x} = (\bar{x}_1, \bar{x}_2, \ldots, \bar{x}_n)$ per il programma lineare primale (A, b, c). Indichiamo con N e B le variabili di base e non di base per l'ultima forma standard; indichiamo con c' i coefficienti nell'ultima forma standard e supponiamo che i valori $\bar{y} = (\bar{y}_1, \bar{y}_2, \ldots, \bar{y}_m)$ siano definiti dall'equazione (29.91). Allora $\bar{x}$ è una soluzione ottima per il programma lineare primale, $\bar{y}$ è una soluzione ottima per il programma lineare duale e

$$\sum_{j=1}^{n} c_j \bar{x}_j = \sum_{i=1}^{m} b_i \bar{y}_i \tag{29.92}$$

Dimostrazione Per il Corollario 29.9, se possiamo trovare delle soluzioni ammissibili $\bar{x}$ e $\bar{y}$ che soddisfano l'equazione (29.92), allora $\bar{x}$ deve essere una soluzione primale ottima e $\bar{y}$ deve essere una soluzione duale ottima. Dimostriamo che le soluzioni $\bar{x}$ e $\bar{y}$ descritte nella definizione del teorema soddisfano l'equazione (29.92). Supponiamo di applicare l'algoritmo SIMPLEX a un programma lineare primale, definito dalle espressioni (29.16)–(29.18). L'algoritmo passa attraverso una serie di forme standard per poi terminare con una forma standard finale con la funzione obiettivo

$$z = v' + \sum_{j \in N} c'_j x_j \tag{29.93}$$

Poiché SIMPLEX termina con una soluzione, per la condizione nella riga 3 sappiamo che

$$c'_j \le 0 \quad \text{per ogni } j \in N \tag{29.94}$$

Se definiamo

$$c'_j = 0 \quad \text{per ogni } j \in B \tag{29.95}$$

possiamo riscrivere l'equazione (29.93) così

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned} z &= v' + \sum_{j \in N} c'_j x_j \\ &= v' + \sum_{j \in N} c'_j x_j + \sum_{j \in B} c'_j x_j \quad (\text{perché } c'_j = 0 \text{ se } j \in B) \\ &= v' + \sum_{j=1}^{n+m} c'_j x_j \quad (\text{perché } N \cup B = \{1, 2, \ldots, n+m\}) \end{aligned} \tag{29.96}$$

Per la soluzione di base $\bar{x}$ associata a questa forma standard finale, $\bar{x}_j = 0$ per ogni $j \in N$, e $z = \nu'$. Poiché tutte le forme standard sono equivalenti, se calcoliamo la funzione obiettivo originale in $\bar{x}$, dobbiamo ottenere lo stesso valore obiettivo, cioè

$$\begin{aligned}
\sum_{j=1}^{n} c_j \bar{x}_j &= \nu' + \sum_{j=1}^{n+m} c'_j \bar{x}_j && (29.97)\\
&= \nu' + \sum_{j \in N} c'_j \bar{x}_j + \sum_{j \in B} c'_j \bar{x}_j \\
&= \nu' + \sum_{j \in N} (c'_j \cdot 0) + \sum_{j \in B} (0 \cdot \bar{x}_j) && (29.98)\\
&= \nu'
\end{aligned}$$

Adesso dimostriamo che la soluzione $\bar{y}$, definita dall'equazione (29.91), è ammissibile per il programma lineare duale e che il suo valore obiettivo $\sum_{i=1}^{m} b_i \bar{y}_i$ è uguale a $\sum_{j=1}^{n} c_j \bar{x}_j$. L'equazione (29.97) dice che la prima e l'ultima forma standard, valutate in $\bar{x}$, sono uguali. Più in generale, l'equivalenza di tutte le forme standard implica che per *qualsiasi* insieme di valori $x = (x_1, x_2, \ldots, x_n)$, abbiamo

$$\sum_{j=1}^{n} c_j x_j = \nu' + \sum_{j=1}^{n+m} c'_j x_j$$

Quindi, per ogni prefissato insieme di valori $\bar{x} = (\bar{x}_1, \bar{x}_2, \ldots, \bar{x}_n)$, abbiamo

$$\begin{aligned}
&\sum_{j=1}^{n} c_j \bar{x}_j \\
&= \nu' + \sum_{j=1}^{n+m} c'_j \bar{x}_j \\
&= \nu' + \sum_{j=1}^{n} c'_j \bar{x}_j + \sum_{j=n+1}^{n+m} c'_j \bar{x}_j \\
&= \nu' + \sum_{j=1}^{n} c'_j \bar{x}_j + \sum_{i=1}^{m} c'_{n+i} \bar{x}_{n+i} \\
&= \nu' + \sum_{j=1}^{n} c'_j \bar{x}_j + \sum_{i=1}^{m} (-\bar{y}_i)\, \bar{x}_{n+i} && \text{(per le equazioni (29.91) e (29.95))} \\
&= \nu' + \sum_{j=1}^{n} c'_j \bar{x}_j + \sum_{i=1}^{m} (-\bar{y}_i) \left(b_i - \sum_{j=1}^{n} a_{ij} \bar{x}_j \right) && \text{(per l'equazione (29.32))} \\
&= \nu' + \sum_{j=1}^{n} c'_j \bar{x}_j - \sum_{i=1}^{m} b_i \bar{y}_i + \sum_{i=1}^{m} \sum_{j=1}^{n} (a_{ij} \bar{x}_j)\, \bar{y}_i \\
&= \nu' + \sum_{j=1}^{n} c'_j \bar{x}_j - \sum_{i=1}^{m} b_i \bar{y}_i + \sum_{j=1}^{n} \sum_{i=1}^{m} (a_{ij} \bar{y}_i)\, \bar{x}_j \\
&= \left(\nu' - \sum_{i=1}^{m} b_i \bar{y}_i \right) + \sum_{j=1}^{n} \left(c'_j + \sum_{i=1}^{m} a_{ij} \bar{y}_i \right) \bar{x}_j
\end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 9788893413016-220707-0000 Copyright © 2022, McGraw-Hill Education (Italy)

quindi

$$\sum_{j=1}^{n} c_j \bar{x}_j = \left(\nu' - \sum_{i=1}^{m} b_i \bar{y}_i\right) + \sum_{j=1}^{n} \left(c'_j + \sum_{i=1}^{m} a_{ij} \bar{y}_i\right) \bar{x}_j \tag{29.99}$$

Applicando il Lemma 29.3 all'equazione (29.99), otteniamo

$$\nu' - \sum_{i=1}^{m} b_i \bar{y}_i = 0 \tag{29.100}$$

$$c'_j + \sum_{i=1}^{m} a_{ij} \bar{y}_i = c_j \quad \text{per } j = 1, 2, \ldots, n \tag{29.101}$$

Per l'equazione (29.100), abbiamo $\sum_{i=1}^{m} b_i \bar{y}_i = \nu'$, e quindi il valore obiettivo del duale $\left(\sum_{i=1}^{m} b_i \bar{y}_i\right)$ è uguale a quello del primale (ν'). Resta da dimostrare che la soluzione $\bar{y}$ è ammissibile per il problema duale. Dalle equazioni (29.94) e (29.95), abbiamo che $c'_j \leq 0$ per ogni $j = 1, 2, \ldots, n + m$. Quindi, per ogni $i = 1, 2, \ldots, m$, l'equazione (29.101) implica che

$$\begin{aligned} c_j &= c'_j + \sum_{i=1}^{m} a_{ij} \bar{y}_i \\ &\leq \sum_{i=1}^{m} a_{ij} \bar{y}_i \end{aligned}$$

che soddisfa i vincoli (29.84) del duale. Infine, poiché $c'_j \leq 0$ per ogni $j \in N \cup B$, quando impostiamo $\bar{y}$ secondo l'equazione (29.91), otteniamo che ogni $\bar{y}_i \geq 0$, e quindi anche i vincoli di non negatività sono soddisfatti. ■

Abbiamo dimostrato che, dato un programma lineare ammissibile, se la procedura INITIALIZE-SIMPLEX restituisce una soluzione ammissibile e se SIMPLEX termina senza restituire "illimitato", allora la soluzione restituita è effettivamente una soluzione ottima. Abbiamo spiegato anche come creare una soluzione ottima per il programma lineare duale.

Esercizi

29.5-1

Formulate il duale del programma lineare dato nell'Esercizio 29.4-5.

29.5-2

Supponete di avere un programma lineare che non è in forma canonica. Potreste ottenere il duale, trasformando dapprima il programma nella forma canonica e poi prendendo il duale. Tuttavia, sarebbe più comodo ottenere direttamente il duale. Spiegate come, dato un programma lineare arbitrario, sia possibile ottenere direttamente il duale del programma lineare.

Acquistato da Michele Michele su Webster | 2022-07-07 23:12 | Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

29.5-3

Scrivete il duale del programma lineare del flusso massimo, dato dalle espressioni (29.47)–(29.49) a pagina 719. Spiegate come interpretare questa formulazione come un problema di taglio minimo.

29.5-4
Scrivete il duale del programma lineare del flusso di costo minimo, dato dalle espressioni (29.51)–(29.52) a pagina 720. Spiegate come interpretare questo problema in termini di grafi e flussi.

29.5-5
Dimostrate che il duale del duale di un programma lineare è il programma lineare primale.

29.5-6
Quale risultato nel Capitolo 26 può essere interpretato come dualità debole per il problema del flusso massimo?

29.6 La soluzione di base iniziale ammissibile

In questo paragrafo spiegheremo come verificare se un programma lineare è ammissibile e, se lo è, come creare una forma standard per la quale la soluzione di base è ammissibile. Concluderemo dimostrando il teorema fondamentale della programmazione lineare, che dice che la procedura SIMPLEX fornisce sempre il risultato corretto.

Trovare una soluzione iniziale

Nel Paragrafo 29.4 abbiamo supposto di avere una procedura INITIALIZE-SIMPLEX che determina se un programma lineare ha soluzioni ammissibili e, se ce le ha, fornisce una forma standard per la quale la soluzione di base è ammissibile. Adesso descriviamo questa procedura.

Un programma lineare può essere ammissibile, ma la soluzione di base iniziale può non essere ammissibile. Consideriamo, per esempio, il seguente programma lineare:

$$\text{massimizzare} \quad 2x_1 - x_2 \tag{29.102}$$

con le condizioni

$$2x_1 - x_2 \leq 2 \tag{29.103}$$

$$x_1 - 5x_2 \leq -4 \tag{29.104}$$

$$x_1, x_2 \geq 0 \tag{29.105}$$

Se dovessimo trasformare questo programma lineare nella forma standard, la soluzione di base porrebbe $x_1 = 0$ e $x_2 = 0$. Questa soluzione viola il vincolo (29.104), e quindi non è una soluzione ammissibile. Di conseguenza, INITIALIZE-SIMPLEX non può restituire semplicemente la forma standard ovvia. Per verificare se questo programma lineare abbia delle soluzioni ammissibili, possiamo formulare un ***programma lineare ausiliario***. Per questo programma lineare ausiliario saremo in grado di trovare (dopo un po' di lavoro) una forma standard per la quale la soluzione di base è ammissibile. Inoltre, la soluzione di questo programma lineare ausiliario consentirà di determinare se il programma lineare iniziale è ammissibile e, se lo è, fornirà una soluzione ammissibile con la quale possiamo inizializzare SIMPLEX.

Acquistato da Michele Michele su Webster il 2022-07-27 Copia personale. Vietata la distribuzione. Copyright © 2022, Mc Graw Hill Education (Italy)

Lemma 29.11
Sia L un programma lineare nella forma canonica, definito secondo le equazioni (29.16)–(29.18). Sia L_{aux} il seguente programma lineare con $n + 1$ variabili:

massimizzare $-x_0$ (29.106)

con le condizioni

$$\sum_{j=1}^{n} a_{ij} x_j - x_0 \leq b_i \quad \text{per } i = 1, 2, \ldots, m \tag{29.107}$$

$$x_j \geq 0 \quad \text{per } j = 0, 1, \ldots, n \tag{29.108}$$

Allora L è ammissibile se e soltanto se il valore obiettivo ottimo di L_{aux} è 0.

Dimostrazione Supponiamo che il programma lineare L abbia una soluzione ammissibile $\bar{x} = (\bar{x}_1, \bar{x}_2, \ldots, \bar{x}_n)$. Allora la soluzione $\bar{x}_0 = 0$ combinata con $\bar{x}$ è una soluzione ammissibile per L_{aux} con valore obiettivo 0. Poiché $x_0 \geq 0$ è un vincolo di L_{aux} e la funzione obiettivo è massimizzare $-x_0$, questa soluzione deve essere ottima per L_{aux}.

Viceversa, supponiamo che il valore obiettivo ottimo di L_{aux} sia 0. Allora $\bar{x}_0 = 0$ e i valori delle restanti variabili $\bar{x}$ soddisfano i vincoli di L. ■

Adesso descriviamo la strategia per trovare una soluzione di base iniziale ammissibile per un programma lineare L nella forma canonica:

```
INITIALIZE-SIMPLEX(A, b, c)
 1  Sia k l'indice del b_i minimo
 2  if b_k ≥ 0                    // La soluzione di base iniziale è ammissibile?
 3      return ({1, 2, ..., n}, {n + 1, n + 2, ..., n + m}, A, b, c, 0)
 4  Forma L_aux aggiungendo −x_0 al lato sinistro di ogni equazione
        e ponendo la funzione obiettivo pari a −x_0
 5  Sia (N, B, A, b, c, v) la forma standard risultante per L_aux
 6  l = n + k
 7  // L_aux ha n + 1 variabili non di base ed m variabili di base
 8  (N, B, A, b, c, v) = PIVOT(N, B, A, b, c, v, l, 0)
 9  // La soluzione di base adesso è ammissibile per L_aux.
10  Ripete il ciclo while (righe 3–12) di SIMPLEX finché non trova
        una soluzione ottima per L_aux
11  if la soluzione ottima per L_aux pone x̄_0 = 0
12      if x̄_0 è una variabile di base
13          esegue un'operazione di pivoting per renderla non di base
14      Dalla forma standard finale di L_aux, elimina x_0 dai vincoli e ripristina
            la funzione obiettivo originale di L, sostituendo
            ciascuna variabile di base in questa funzione obiettivo
            con il lato destro del suo vincolo associato
15      return la forma standard finale modificata
16  else return "inammissibile"
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 89503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

La procedura INITIALIZE-SIMPLEX opera nel modo seguente. Nelle righe 1–3 controlliamo implicitamente la soluzione di base per la forma standard iniziale di L data da $N = \{1, 2, \ldots, n\}$, $B = \{n + 1, n + 2, \ldots, n + m\}$, $\bar{x}_i = b_i$ per ogni $i \in B$, e $\bar{x}_j = 0$ per ogni $j \in N$. (Creare la forma standard non richiede un lavoro esplicito, in quanto i valori di A, b e c sono uguali nelle due forme

standard e canonica.) Se la riga 2 trova che questa soluzione di base è ammissibile – ovvero $\bar{x}_i \geq 0$ per ogni $i \in N \cup B$ – allora la riga 3 restituisce la forma standard; altrimenti la riga 4 forma il programma lineare ausiliario L_{aux} secondo il Lemma 29.11.

Poiché la soluzione di base iniziale per L non è ammissibile, non sarà ammissibile neanche la soluzione di base iniziale per la forma standard di L_{aux}. Per trovare una soluzione ammissibile, eseguiamo una singola operazione di pivoting. La riga 6 seleziona $l = n + k$ come indice della variabile di base che sarà la variabile uscente nell'operazione di pivoting. Poiché le variabili di base sono $x_{n+1}, x_{n+2}, \ldots, x_{n+m}$, la variabile uscente x_l sarà quella con il valore più negativo. La riga 8 esegue una chiamata di PIVOT, con x_0 entrante e x_l uscente. Come vedremo fra poco, la soluzione di base che si ottiene con questa chiamata di PIVOT è ammissibile.

Adesso che abbiamo una forma standard per la quale la soluzione di base è ammissibile, possiamo chiamare ripetutamente PIVOT (riga 10) per risolvere completamente il programma lineare ausiliario. Come mostra il test nella riga 11, se troviamo una soluzione ottima per L_{aux} con valore obiettivo 0, allora le righe 12–14 creano una forma standard di L per la quale la soluzione di base è ammissibile. Per farlo, le righe 12–13, gestiscono il caso degenere in cui x_0 è ancora una variabile di base con valore $\bar{x}_0 = 0$. In questo caso, eseguiamo una singola operazione di pivoting per eliminare x_0 dalle variabili di base, utilizzando qualsiasi $e \in N$ tale che $a_{0e} \neq 0$ come variabile entrante. La nuova soluzione di base resta ammissibile; l'operazione di pivoting non cambia il valore delle variabili. Poi, cancelliamo tutti i termini x_0 dai vincoli e ripristiniamo la funzione obiettivo originale di L. La funzione obiettivo originale può contenere sia le variabili di base sia le variabili non di base. Quindi, sostituiamo nella funzione obiettivo ciascuna variabile di base con il lato destro del suo vincolo associato. La riga 15 restituisce questa forma standard modificata. D'altra parte, se nella riga 11 scopriamo che il programma lineare originale L è inammissibile, restituiamo questa informazione nella riga 16.

Esaminiamo adesso il funzionamento di INITIALIZE-SIMPLEX con il programma lineare (29.102)–(29.105). Questo programma lineare è ammissibile se possiamo trovare valori non negativi per x_1 e x_2 che soddisfano le disuguaglianze (29.103) e (29.104). Applicando il Lemma 29.11, formuliamo il programma lineare ausiliario in questo modo:

$$\text{massimizzare} \quad -x_0 \tag{29.109}$$

con le condizioni

$$2x_1 - x_2 - x_0 \leq 2 \tag{29.110}$$

$$x_1 - 5x_2 - x_0 \leq -4 \tag{29.111}$$

$$x_1, x_2, x_0 \geq 0$$

Per il Lemma 29.11, se il valore obiettivo ottimo di questo programma lineare ausiliario è 0, allora il programma lineare originale ha una soluzione ammissibile. Se il valore obiettivo ottimo di questo programma lineare ausiliario è negativo, allora il programma lineare originale non ha una soluzione ammissibile.

Acquistato da Michele Michele su Webster il 2022-[illegible] Non [illegible] [illegible] [illegible]

Se scriviamo questo programma lineare nella forma standard, otteniamo

$$\begin{aligned} z &= -x_0 \\ x_3 &= 2 - 2x_1 + x_2 + x_0 \\ x_4 &= -4 - x_1 + 5x_2 + x_0 \end{aligned}$$

Non siamo ancora fuori dai guai, perché la soluzione di base, che porrebbe $x_4 = -4$, non è ammissibile per questo programma lineare ausiliario. Tuttavia, con una chiamata di PIVOT possiamo convertire questa forma standard in una forma nella quale la soluzione di base è ammissibile. Come indica la riga 8, scegliamo x_0 come variabile entrante. Nella riga 6 scegliamo x_4 come variabile uscente, che è la variabile di base il cui valore nella soluzione di base è il più negativo. Dopo il pivoting, abbiamo la forma standard

$$\begin{aligned} z &= -4 - x_1 + 5x_2 - x_4 \\ x_0 &= 4 + x_1 - 5x_2 + x_4 \\ x_3 &= 6 - x_1 - 4x_2 + x_4 \end{aligned}$$

La soluzione di base associata è $(x_0, x_1, x_2, x_3, x_4) = (4, 0, 0, 6, 0)$, che è ammissibile. Adesso chiamiamo ripetutamente PIVOT fino a ottenere una soluzione ottima per L_{aux}. In questo caso, una chiamata di PIVOT con x_2 entrante e x_0 uscente fornisce

$$\begin{aligned} z &= - x_0 \\ x_2 &= \frac{4}{5} - \frac{x_0}{5} + \frac{x_1}{5} + \frac{x_4}{5} \\ x_3 &= \frac{14}{5} + \frac{4x_0}{5} - \frac{9x_1}{5} + \frac{x_4}{5} \end{aligned}$$

Questa forma standard è la soluzione finale del problema ausiliario. Poiché questa soluzione ha $x_0 = 0$, sappiamo che il nostro problema iniziale era ammissibile. Inoltre, poiché $x_0 = 0$, possiamo eliminare x_0 dal sistema dei vincoli. Poi possiamo utilizzare la funzione obiettivo originale, con le appropriate sostituzioni fatte per includere soltanto le variabili non di base. Nell'esempio in esame, otteniamo la funzione obiettivo

$$2x_1 - x_2 = 2x_1 - \left(\frac{4}{5} - \frac{x_0}{5} + \frac{x_1}{5} + \frac{x_4}{5}\right)$$

Ponendo $x_0 = 0$ e semplificando, otteniamo la funzione obiettivo

$$-\frac{4}{5} + \frac{9x_1}{5} - \frac{x_4}{5}$$

e la forma standard

$$\begin{aligned} z &= -\frac{4}{5} + \frac{9x_1}{5} - \frac{x_4}{5} \\ x_2 &= \frac{4}{5} + \frac{x_1}{5} + \frac{x_4}{5} \\ x_3 &= \frac{14}{5} - \frac{9x_1}{5} + \frac{x_4}{5} \end{aligned}$$

Questa forma standard ha una soluzione di base ammissibile, che possiamo restituire alla procedura SIMPLEX.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Adesso dimostriamo formalmente la correttezza di INITIALIZE-SIMPLEX.

Lemma 29.12

Se un programma lineare L non ha una soluzione ammissibile, allora INITIALIZE-SIMPLEX restituisce "inammissibile", altrimenti restituisce una forma standard valida per la quale la soluzione di base è ammissibile.

Dimostrazione Supponiamo innanzi tutto che il programma lineare L non abbia soluzioni ammissibili. Allora per il Lemma 29.11, il valore obiettivo ottimo di L_{aux}, definito da (29.106)–(29.108), è diverso da zero e, per il vincolo di non negatività su x_0, una soluzione ottima deve avere un valore obiettivo negativo. Inoltre, questo valore obiettivo deve essere finito, perché ponendo $x_i = 0$, per $i = 1, 2, \ldots, n$, e $x_0 = |\min_{i=1}^{m} \{b_i\}|$ si ha una soluzione ammissibile, e questa soluzione ha valore obiettivo $-|\min_{i=1}^{m} \{b_i\}|$. Quindi, la riga 10 di INITIALIZE-SIMPLEX troverà una soluzione con un valore obiettivo negativo. Sia $\bar{x}$ la soluzione di base associata alla forma standard finale. Non possiamo avere $\bar{x}_0 = 0$, perché allora L_{aux} avrebbe valore obiettivo 0, contraddicendo il fatto che il valore obiettivo è negativo. Quindi il risultato del test nella riga 11 è che la riga 16 restituisce "inammissibile".

Supponiamo adesso che il programma lineare L abbia una soluzione ammissibile. Dall'Esercizio 29.4-4, sappiamo che, se $b_i \geq 0$ per $i = 1, 2, \ldots, m$, allora la soluzione di base associata alla forma standard iniziale è ammissibile. In questo caso, le righe 2–3 restituiranno la forma standard associata all'input. (Non c'è molto da fare per convertire la forma canonica nella forma standard, perché A, b e c sono uguali in entrambe le forme.)

Nella parte restante della dimostrazione, analizzeremo il caso in cui il programma lineare è ammissibile, ma non ritorniamo alla riga 3. Dimostriamo che in questo caso le righe 4–10 trovano una soluzione ammissibile per L_{aux} con valore obiettivo 0. Innanzi tutto, per le righe 1–2, dobbiamo avere

$$b_k < 0$$

e

$$b_k \leq b_i \quad \text{per ogni } i \in B \tag{29.112}$$

Nella riga 8, eseguiamo un'operazione di pivoting in cui la variabile uscente x_l (ricordiamo che $l = n + k$, per cui $b_l < 0$) è il lato sinistro dell'equazione con b_i minimo e la variabile entrante è x_0, la variabile extra che è stata aggiunta. Adesso dimostriamo che dopo questo pivoting, tutti gli elementi di b non sono negativi, e quindi la soluzione di base per L_{aux} è ammissibile. Indicando con $\bar{x}$ la soluzione di base dopo la chiamata di PIVOT e con $\widehat{b}$ e $\widehat{B}$ i valori restituiti da PIVOT, il Lemma 29.1 implica che

$$\bar{x}_i = \begin{cases} b_i - a_{ie}\widehat{b}_e & \text{se } i \in \widehat{B} - \{e\} \\ b_l / a_{le} & \text{se } i = e \end{cases} \tag{29.113}$$

La chiamata di PIVOT nella riga 8 ha $e = 0$. Se riscriviamo le espressioni (29.107) per includere i coefficienti a_{i0}, otteniamo

$$\sum_{j=0}^{n} a_{ij} x_j \leq b_i \quad \text{per } i = 1, 2, \ldots, m \tag{29.114}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

quindi

$$a_{i0} = a_{ie} = -1 \quad \text{per ogni } i \in B \tag{29.115}$$

(Notate che a_{i0} è il coefficiente di x_0 come appare nell'espressione (29.114), non l'opposto del coefficiente, perché L_{aux} è in forma canonica, non in forma standard.) Poiché $l \in B$, abbiamo anche $a_{le} = -1$. Quindi $b_l / a_{le} > 0$, e così $\bar{x}_e > 0$.

Per le restanti variabili di base, abbiamo

$$\begin{aligned}
\bar{x}_i &= b_i - a_{ie}\hat{b}_e && \text{(per l'equazione (29.113))} \\
&= b_i - a_{ie}(b_l/a_{le}) && \text{(per la riga 3 of PIVOT)} \\
&= b_i - b_l && \text{(per l'equazione (29.115) e } a_{le} = -1\text{)} \\
&\geq 0 && \text{(per la disuguaglianza (29.112))}
\end{aligned}$$

che implica che ogni variabile di base adesso è non negativa. Quindi la soluzione di base dopo la chiamata di PIVOT nella riga 8 è ammissibile. Poi viene eseguita la riga 10, che risolve L_{aux}. Poiché abbiamo assunto che L abbia una soluzione ammissibile, allora per il Lemma 29.11 L_{aux} ha una soluzione ottima con valore obiettivo 0. Poiché tutte le forme standard sono equivalenti, la soluzione di base finale per L_{aux} deve avere $\bar{x}_0 = 0$, e dopo avere rimosso x_0 dal programma lineare, si ottiene una forma standard che è ammissibile per L. Questa forma standard viene poi restituita nella riga 15. ■

Teorema fondamentale della programmazione lineare

Concludiamo questo capitolo dimostrando che la procedura SIMPLEX funziona. In particolare, un programma lineare può essere inammissibile o illimitato oppure può avere una soluzione ottima con un valore obiettivo finito e, in ogni caso, SIMPLEX opera in modo appropriato.

Teorema 29.13 (Teorema fondamentale della programmazione lineare)
Un programma lineare L espresso in forma canonica

1. ha una soluzione ottima con un valore obiettivo finito oppure
2. è inammissibile oppure
3. è illimitato.

Se L è inammissibile, SIMPLEX restituisce "inammissibile". Se L è illimitato, SIMPLEX restituisce "illimitato". Altrimenti, SIMPLEX restituisce una soluzione ottima con un valore obiettivo finito.

Dimostrazione Per il Lemma 29.12, se il programma lineare L è inammissibile, allora SIMPLEX restituisce "inammissibile". Adesso supponiamo che il programma lineare L sia ammissibile. Per il Lemma 29.12, INITIALIZE-SIMPLEX restituisce una forma standard per la quale la soluzione di base è ammissibile. Per il Lemma 29.7, quindi, SIMPLEX restituisce "illimitato" oppure termina con una soluzione ammissibile. Se termina con una soluzione finita, allora il Teorema 29.10 ci dice che questa soluzione è ottima. D'altra parte, se SIMPLEX restituisce "illimitato", il Lemma 29.2 ci dice che il programma lineare L è veramente illimitato. Poiché SIMPLEX termina sempre in uno di questi modi, la dimostrazione è completa. ■

Acquistato da Michele Mastrocinque Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Esercizi

29.6-1
Scrivete uno pseudocodice dettagliato per implementare le righe 5 e 14 della procedura INITIALIZE-SIMPLEX.

29.6-2
Dimostrate che quando la procedura INITIALIZE-SIMPLEX esegue il ciclo principale di SIMPLEX, non restituisce mai il messaggio "illimitato".

29.6-3

Supponete di avere un programma lineare L in forma canonica e che, per L e per il duale di L, le soluzioni di base associate alle forme standard iniziali siano ammissibili. Dimostrate che il valore obiettivo ottimo di L è 0.

29.6-4

Supponete che in un programma lineare siano ammesse le disuguaglianze strette. Dimostrate che in questo caso il teorema fondamentale della programmazione lineare non è valido.

29.6-5

Risolvete il seguente programma lineare utilizzando SIMPLEX:

$$\begin{aligned}
&\text{massimizzare} && x_1 + 3x_2 \\
&\text{con le condizioni} \\
& && x_1 - x_2 \le 8 \\
& && -x_1 - x_2 \le -3 \\
& && -x_1 + 4x_2 \le 2 \\
& && x_1, x_2 \ge 0
\end{aligned}$$

29.6-6

Risolvete il seguente programma lineare utilizzando SIMPLEX:

$$\begin{aligned}
&\text{massimizzare} && x_1 - 2x_2 \\
&\text{con le condizioni} \\
& && x_1 + 2x_2 \le 4 \\
& && -2x_1 - 6x_2 \le -12 \\
& && x_2 \le 1 \\
& && x_1, x_2 \ge 0
\end{aligned}$$

29.6-7

Risolvete il seguente programma lineare utilizzando SIMPLEX:

$$\begin{aligned}
&\text{massimizzare} && x_1 + 3x_2 \\
&\text{con le condizioni} \\
& && -x_1 + x_2 \le -1 \\
& && -x_1 - x_2 \le -3 \\
& && -x_1 + 4x_2 \le 2 \\
& && x_1, x_2 \ge 0
\end{aligned}$$

29.6-8

Risolvete il programma lineare definito dalle espressioni (29.6)–(29.10).

29.6-9

Considerate il seguente programma lineare P a una sola variabile:

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned}
&\text{massimizzare} && tx \\
&\text{con le condizioni} \\
& && rx \le s \\
& && x \ge 0
\end{aligned}$$

dove r, s e t sono numeri reali arbitrari. Sia D il duale di P.

Trovate i valori di r, s e t per i quali potete asserire che

1. Entrambi i programmi P e D hanno soluzioni ottime con valori obiettivo finiti.
2. P è ammissibile, ma D è inammissibile.
3. D è ammissibile, ma P è inammissibile.
4. Né P né D sono ammissibili.

Problemi

29-1 Ammissibilità delle disuguaglianze lineari

Dato un sistema di m disuguaglianze lineari in n variabili $x_1, x_2, \ldots, x_n$, il ***problema dell'ammissibilità delle disuguaglianze lineari*** consiste nel verificare se esiste una combinazione di valori delle variabili che soddisfa contemporaneamente ciascuna delle disuguaglianze.

a. Dimostrate che, se avete un algoritmo per la programmazione lineare, potete utilizzarlo per risolvere un problema di ammissibilità delle disuguaglianze lineari. Il numero di variabili e vincoli che utilizzate nel problema della programmazione lineare dovrebbe essere polinomiale in n ed m.

b. Dimostrate che, se avete un algoritmo per il problema dell'ammissibilità delle disuguaglianze lineari, potete utilizzarlo per risolvere un problema di programmazione lineare. Il numero di variabili e disuguaglianze lineari che utilizzate nel problema di ammissibilità delle disuguaglianze lineari dovrebbe essere polinomiale in n ed m, il numero di variabili e vincoli nel programma lineare.

29-2 Proprietà degli scarti complementari

La ***proprietà degli scarti complementari*** descrive una relazione tra i valori delle variabili primali e i vincoli duali e tra i valori delle variabili duali e i vincoli primali. Sia $\bar{x}$ una soluzione ammissibile per il programma lineare primale definito da (29.16)–(29.18) e sia $\bar{y}$ una soluzione ammissibile per il programma lineare duale definito da (29.83)–(29.85). La proprietà degli scarti complementari stabilisce che le seguenti condizioni sono necessarie e sufficienti affinché $\bar{x}$ e $\bar{y}$ siano soluzioni ottime:

$$\sum_{i=1}^{m} a_{ij}\bar{y}_i = c_j \text{ o } \bar{x}_j = 0 \text{ per } j = 1, 2, \ldots, n$$

e

$$\sum_{j=1}^{n} a_{ij}\bar{x}_j = b_i \text{ o } \bar{y}_i = 0 \text{ per } i = 1, 2, \ldots, m$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

a. Verificate che la proprietà degli scarti complementari vale per il programma lineare definito da (29.53)–(29.57).

b. Dimostrate che la proprietà degli scarti complementari vale per qualsiasi programma lineare primale e il suo corrispondente duale.

c. Dimostrate che una soluzione ammissibile $\bar{x}$ per un programma lineare primale definito da (29.16)–(29.18) è ottima se e soltanto se esistono dei valori $\bar{y} = (\bar{y}_1, \bar{y}_2, \ldots, \bar{y}_m)$ che soddisfano queste tre condizioni:

1. $\bar{y}$ è una soluzione ammissibile per il programma lineare duale definito da (29.83)–(29.85)
2. $\sum_{i=1}^{m} a_{ij}\bar{y}_i = c_j$ per ogni j tale che $\bar{x}_j > 0$
3. $\bar{y}_i = 0$ per ogni i tale che $\sum_{j=1}^{n} a_{ij}\bar{x}_j < b_i$

29-3 Programmazione lineare intera

Un ***problema di programmazione lineare intera*** è un problema di programmazione lineare con il vincolo aggiuntivo che le variabili x devono assumere valori interi. L'Esercizio 34.6-3 dimostra che determinare se un programma lineare intero ha una soluzione ammissibile è un problema NP-difficile; questo significa che non si conosce nessun algoritmo con tempo polinomiale per questo problema.

a. Dimostrate che la dualità debole (Lemma 29.8) vale per un programma lineare intero.

b. Dimostrate che la dualità (Teorema 29.10) non vale sempre per un programma lineare intero.

c. Dato un programma lineare primale nella forma canonica, indicate con P il valore obiettivo ottimo per il programma lineare primale, con D il valore obiettivo ottimo per il suo duale, con IP il valore obiettivo ottimo per la versione intera del primale (ovvero il primale con il vincolo aggiuntivo che le variabili devono assumere valori interi) e con ID il valore obiettivo ottimo per la versione intera del duale. Supponendo che il programma intero primale e il programma intero duale siano entrambi ammissibili e limitati, dimostrate che

$$IP \leq P = D \leq ID$$

29-4 Lemma di Farkas

Siano A una matrice $m \times n$ e c un vettore di dimensione n. Allora il lemma di Farkas stabilisce che uno solo dei sistemi

$$\begin{aligned} Ax &\leq 0 \\ c^{\mathrm{T}}x &> 0 \end{aligned}$$

e

$$\begin{aligned} A^{\mathrm{T}}y &= c \\ y &\geq 0 \end{aligned}$$

è risolvibile, dove x è un vettore di dimensione n e y è un vettore di dimensione m. Dimostrate il lemma di Farkas.

Acquistato da Michele Michele su Webster il 2022-07-23:1 Vietata la riproduzione, libreria 1985-2022 0-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

29-5 Circolazione a costo minimo

In questo problema, consideriamo una variante del problema del flusso di costo minimo, descritto nel Paragrafo 29.3, nel caso in cui non siano noti la domanda, la sorgente e il pozzo. Come prima, conosciamo una rete di flusso e i costi degli archi $a(u, v)$. Un flusso è ammissibile se soddisfa il vincolo sulla capacità su ogni arco e la conservazione del flusso in *ogni* vertice. L'obiettivo è trovare, fra tutti i flussi ammissibili, quello di costo minimo. Questo problema è detto ***problema della circolazione a costo minimo.***

a. Formulate il problema della circolazione a costo minimo come un programma lineare.

b. Supponete che per ogni arco $(u, v) \in E$, si abbia $a(u, v) > 0$. Caratterizzate una soluzione ottima per il problema della circolazione a costo minimo.

c. Formulate il problema di flusso massimo come un programma lineare del problema della circolazione a costo minimo. Ovvero, data un'istanza del problema di flusso massimo $G = (V, E)$ con sorgente s, pozzo t e capacità degli archi c, create un problema di circolazione a costo minimo determinando una rete $G' = (V', E')$ con capacità degli archi c' e costi degli archi a' in modo tale che sia possibile ottenere una soluzione del problema del flusso massimo da una soluzione del problema della circolazione a costo minimo.

d. Formulate il problema dei cammini minimi da sorgente unica come un programma lineare del problema della circolazione a costo minimo.

Note

Questo capitolo è soltanto un'introduzione allo studio della vasta materia della programmazione lineare. Alcuni libri sono dedicati esclusivamente alla programmazione lineare, fra i quali citiamo Chvátal [70], Gass [131], Karloff [198], Schrijver [304] e Vanderbei [345]. Molti altri libri fanno una buona trattazione della programmazione lineare, fra i quali figurano Papadimitriou e Steiglitz [272] e Ahuja, Magnanti e Orlin [7]. Gli argomenti trattati in questo capitolo si basano sulla metodologia di Chvátal.

L'algoritmo del simplesso per la programmazione lineare è stato ideato da G. Dantzig nel 1947. Subito dopo, si scoprì che un certo numero di problemi in vari campi potevano essere formulati come programmi lineari e risolti con l'algoritmo del simplesso. Questa scoperta portò a un crescente impiego della programmazione lineare, assieme a molti algoritmi. Le varianti dell'algoritmo del simplesso restano i metodi più popolari per risolvere i problemi della programmazione lineare. Questa storia è narrata in vari testi, incluse le note nei libri di Chvátal [70] e Karloff [198].

L'algoritmo dell'ellissoide è stato il primo algoritmo con tempo polinomiale per la programmazione lineare. Ideato da L. G. Khachian nel 1979, l'algoritmo dell'ellissoide si basava su un precedente lavoro di N. Z. Shor, D. B. Judin e A. S. Nemirovskii. L'uso di questo algoritmo per risolvere una varietà di problemi nell'ottimizzazione combinatoria è descritto da Grötschel, Lovász e Schrijver [155]. Attualmente, l'algoritmo dell'ellissoide non sembra competitivo con l'algoritmo del simplesso nelle applicazioni pratiche. L'articolo di Karmarkar [199] include una descrizione del primo algoritmo dei punti interni. Successivamente, molti ricercatori progettarono algoritmi di questo tipo. Uno studio interessante di questo argomento si trova nell'articolo di Goldfarb e Todd [142] e nel libro di Ye [362].

L'analisi dell'algoritmo del simplesso è un'area attiva di ricerca. Klee e Minty hanno realizzato un esempio nel quale l'algoritmo del simplesso esegue $2^n - 1$ iterazioni. L'algoritmo del simplesso di solito ha buone prestazioni nella pratica e molti ricercatori hanno tentato di fornire delle spiegazioni teoriche per questa osservazione empirica. Una ricerca iniziata da Borgwardt e continuata da molti altri indica che, sotto certe ipotesi probabilistiche sull'input, l'algoritmo del simplesso converge in tempo atteso polinomiale. In quest'area sono stati fatti recenti progressi grazie al lavoro di Spielman e Teng [323], che hanno introdotto l'analisi lisciata (smoothed analysis) degli algoritmi e l'hanno applicata all'algoritmo del simplesso. È noto che l'algoritmo del simplesso è più efficiente in alcuni casi speciali. Particolarmente importante è l'algoritmo del simplesso per le reti, che è l'algoritmo del simplesso specializzato alla risoluzione dei problemi di flusso sulle reti. Per alcuni problemi delle reti, inclusi i problemi dei cammini minimi, del flusso massimo e del flusso di costo minimo, alcune varianti dell'algoritmo del simplesso per le reti vengono eseguite in tempo polinomiale. Consultate, per esempio, l'articolo di Orlin [269] e le citazioni contenute.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995030016 220707-0 Copyright © 2022: McGraw-Hill Education (Italy)

30 Polinomi e FFT

Il metodo semplice per sommare due polinomi di grado n richiede un tempo $\Theta(n)$, mentre il metodo semplice per moltiplicarli richiede un tempo $\Theta(n^2)$. In questo capitolo, spiegheremo come la trasformata rapida di Fourier o FFT (Fast Fourier Transform) può ridurre il tempo per moltiplicare i polinomi a $\Theta(n \lg n)$.

L'uso più comune delle trasformate di Fourier, e quindi della FFT, si ha nell'elaborazione dei segnali. Un segnale è dato nel ***dominio del tempo***: come una funzione che associa al tempo un'ampiezza. L'analisi di Fourier consente di esprimere il segnale come una sommatoria pesata di sinusoidi sfasate e di frequenze diverse. I pesi e le fasi associate alle frequenze caratterizzano il segnale nel ***dominio delle frequenze***. Tra le applicazioni più comuni della FFT ricordiamo le tecniche di compressione utilizzate per codificare le informazioni audio e video, inclusi i file MP3. L'elaborazione dei segnali è una materia complessa sulla quale sono stati scritti molti libri interessanti; le note in fondo al capitolo ne citano alcuni.

Polinomi

Un ***polinomio*** nella variabile x in un campo algebrico F è una rappresentazione di una funzione $A(x)$ come una sommatoria formale:

$$A(x) = \sum_{j=0}^{n-1} a_j x^j$$

I valori $a_0, a_1, \ldots, a_{n-1}$ sono detti ***coefficienti*** del polinomio. I coefficienti appartengono al campo F, tipicamente l'insieme dei numeri complessi $\mathbb{C}$. Un polinomio $A(x)$ è detto di ***grado*** k se il suo più grande coefficiente diverso da zero è a_k. Qualsiasi intero strettamente più grande del grado di un polinomio è un ***grado limite*** del polinomio. Quindi, il grado di un polinomio di grado limite n può essere qualsiasi intero tra 0 ed $n - 1$, incluso.

Ci sono varie operazioni che vogliamo definire per i polinomi. Per quanto riguarda l'***addizione di polinomi***, se $A(x)$ e $B(x)$ sono polinomi di grado limite n, diciamo che la loro ***somma*** è un polinomio $C(x)$, anch'esso di grado limite n, tale che $C(x) = A(x) + B(x)$ per ogni x nel campo sottostante. Ovvero, se

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$A(x) = \sum_{j=0}^{n-1} a_j x^j$$

e

$$B(x) = \sum_{j=0}^{n-1} b_j x^j$$

allora

$$C(x) = \sum_{j=0}^{n-1} c_j x^j$$

dove $c_j = a_j + b_j$ per $j = 0, 1, \ldots, n-1$. Per esempio, se abbiamo i polinomi $A(x) = 6x^3 + 7x^2 - 10x + 9$ e $B(x) = -2x^3 + 4x - 5$, allora $C(x) = 4x^3 + 7x^2 - 6x + 4$.

Per quanto riguarda la ***moltiplicazione di polinomi***, se $A(x)$ e $B(x)$ sono polinomi di grado limite n, il loro ***prodotto*** $C(x)$ è un polinomio di grado limite $2n-1$ tale che $C(x) = A(x)B(x)$ per ogni x nel campo sottostante. Probabilmente, avrete già moltiplicato dei polinomi, moltiplicando ciascun termine di $A(x)$ per ciascun termine di $B(x)$ e combinando i termini con la stessa potenza. Per esempio, possiamo moltiplicare $A(x) = 6x^3 + 7x^2 - 10x + 9$ e $B(x) = -2x^3 + 4x - 5$ in questo modo:

$$\begin{array}{rrrrrrr}
 & & 6x^3 & + 7x^2 & - 10x & + 9 \\
 & - 2x^3 & & + 4x & - 5 & \\
\hline
 & - 30x^3 & - 35x^2 & + 50x & - 45 & \\
24x^4 & + 28x^3 & - 40x^2 & + 36x & & \\
- 12x^6 & - 14x^5 & + 20x^4 & - 18x^3 & & \\
\hline
- 12x^6 & - 14x^5 & + 44x^4 & - 20x^3 & - 75x^2 & + 86x & - 45
\end{array}$$

Un altro modo per esprimere il prodotto $C(x)$ è

$$C(x) = \sum_{j=0}^{2n-2} c_j x^j \tag{30.1}$$

dove

$$c_j = \sum_{k=0}^{j} a_k b_{j-k} \tag{30.2}$$

Notate che grado(C) = grado(A) + grado(B); questo implica che, se A è un polinomio di grado limite n_a e B è un polinomio di grado limite n_b, allora C è un polinomio di grado limite $n_a + n_b - 1$. Poiché un polinomio di grado limite k è anche un polinomio di grado limite $k+1$, di solito diremo che il prodotto C è un polinomio di grado limite $n_a + n_b$.

Organizzazione del capitolo

Il Paragrafo 30.1 descrive due modi di rappresentare i polinomi: la rappresentazione per coefficienti e la rappresentazione per punti. I metodi semplici per moltiplicare i polinomi – equazioni (30.1) e (30.2) – richiedono un tempo $\Theta(n^2)$ quando i polinomi sono rappresentati per coefficienti, ma soltanto un tempo $\Theta(n)$ quando sono rappresentati per punti. Tuttavia, è possibile moltiplicare nel tempo $\Theta(n \lg n)$ i polinomi rappresentati per coefficienti effettuando le conversioni fra le due rappresentazioni. Per spiegare questo metodo, dobbiamo prima esaminare le radici complesse dell'unità (Paragrafo 30.2). Poi, utilizziamo la FFT e la sua inversa (Paragrafo 30.2) per effettuare le conversioni. Il Paragrafo 30.3 spiega come implementare la FFT rapidamente in entrambi i modelli seriale e parallelo.

Questo capitolo usa spesso i numeri complessi, quindi il simbolo i sarà utilizzato esclusivamente per indicare $\sqrt{-1}$.

Acquistato da Michele Mo[illegible] per uso esclusivamente personale - Tutti i diritti riservati - © 2010 The McGraw-Hill Education (Italy)

30.1 Rappresentazione dei polinomi

La rappresentazione per coefficienti e la rappresentazione per punti sono in qualche modo equivalenti; ovvero, un polinomio nella forma per punti ha una rappresentazione unica nella forma per coefficienti. In questo paragrafo, introduciamo le due rappresentazioni e spieghiamo come possono essere combinate per moltiplicare nel tempo $\Theta(n \lg n)$ due polinomi di grado limite n.

Rappresentazione per coefficienti

Una ***rappresentazione per coefficienti*** di un polinomio $A(x) = \sum_{j=0}^{n-1} a_j x^j$ di grado limite n è un vettore di coefficienti $a = (a_0, a_1, \ldots, a_{n-1})$. Nelle equazioni matriciali di questo capitolo, di solito, un vettore sarà trattato come vettore colonna.

La rappresentazione per coefficienti è comoda per eseguire alcune operazioni con i polinomi. Per esempio, l'operazione di ***valutazione del polinomio*** $A(x)$ in un dato punto x_0 consiste nel calcolare il valore di $A(x_0)$. La valutazione richiede un tempo $\Theta(n)$ utilizzando la ***regola di Horner***:

$$A(x_0) = a_0 + x_0(a_1 + x_0(a_2 + \cdots + x_0(a_{n-2} + x_0(a_{n-1}))\cdots))$$

Analogamente, la somma di due polinomi rappresentati dai vettori dei coefficienti $a = (a_0, a_1, \ldots, a_{n-1})$ e $b = (b_0, b_1, \ldots, b_{n-1})$ richiede un tempo $\Theta(n)$: produciamo semplicemente il vettore dei coefficienti $c = (c_0, c_1, \ldots, c_{n-1})$, dove $c_j = a_j + b_j$ per $j = 0, 1, \ldots, n-1$.

Adesso consideriamo la moltiplicazione di due polinomi $A(x)$ e $B(x)$, di grado limite n, rappresentati nella forma per coefficienti. Se utilizziamo il metodo descritto dalle equazioni (30.1) e (30.2), il prodotto dei polinomi richiede un tempo $\Theta(n^2)$, perché ciascun coefficiente nel vettore a deve essere moltiplicato per ciascun coefficiente nel vettore b. La moltiplicazione dei polinomi rappresentati per coefficienti sembra essere notevolmente più difficile della valutazione di un polinomio o della somma di due polinomi. Il vettore dei coefficienti c risultante, dato dall'equazione (30.2), è anche detto ***convoluzione*** dei vettori di input a e b. La convoluzione è indicata da $c = a \otimes b$. Poiché il prodotto dei polinomi e il calcolo delle convoluzioni sono problemi computazionali fondamentali di considerevole importanza pratica, questo capitolo descrive gli algoritmi efficienti per questi problemi.

Rappresentazione per punti

Una ***rappresentazione per punti*** di un polinomio $A(x)$ di grado limite n è un insieme di n ***coppie di valori***

$$\{(x_0, y_0), (x_1, y_1), \ldots, (x_{n-1}, y_{n-1})\}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

tali che tutti i valori x_k sono distinti e

$$y_k = A(x_k) \tag{30.3}$$

per $k = 0, 1, \ldots, n-1$. Un polinomio ha diverse rappresentazioni per punti, in quanto un insieme qualsiasi di n punti distinti $x_0, x_1, \ldots, x_{n-1}$ può essere utilizzato come base per la rappresentazione.

In teoria, è semplice calcolare una rappresentazione per punti di un polinomio dato nella forma per coefficienti, perché basta selezionare n punti distinti $x_0, x_1, \ldots, x_{n-1}$ e poi valutare $A(x_k)$ per $k = 0, 1, \ldots, n-1$. Con il metodo di Horner, questa valutazione di n punti richiede un tempo $\Theta(n^2)$. Vedremo più avanti che, se scegliamo i punti x_k intelligentemente, questo calcolo può essere accelerato per essere eseguito in un tempo $\Theta(n \lg n)$.

L'operazione inversa della valutazione – determinare la rappresentazione per coefficienti di un polinomio da una rappresentazione per punti – è detta ***interpolazione***. Il seguente teorema dimostra che l'interpolazione è ben definita quando il polinomio interpolante richiesto ha un grado limite pari al numero di coppie di valori della rappresentazione per punti.

Teorema 30.1 (Unicità del polinomio interpolante)
Per un insieme qualsiasi $\{(x_0, y_0), (x_1, y_1), \ldots, (x_{n-1}, y_{n-1})\}$ di n punti tali che tutti i valori x_k siano distinti, esiste un unico polinomio $A(x)$ di grado limite n tale che $y_k = A(x_k)$ per $k = 0, 1, \ldots, n-1$.

Dimostrazione La dimostrazione si basa sull'esistenza dell'inversa di una certa matrice. L'Equazione (30.3) è equivalente all'equazione matriciale

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{pmatrix} \tag{30.4}$$

La matrice a sinistra è indicata con $V(x_0, x_1, \ldots, x_{n-1})$ ed è detta matrice di Vandermonde. Per il Problema D-1, il determinante di questa matrice è

$$\prod_{0 \le j < k \le n-1} (x_k - x_j)$$

Quindi, per il Teorema D.5, la matrice è invertibile (non è singolare) se i valori x_k sono distinti. Di conseguenza, i coefficienti a_j possono essere unicamente determinati dalla rappresentazione per punti:

$$a = V(x_0, x_1, \ldots, x_{n-1})^{-1} y$$

■

La dimostrazione del Teorema 30.1 descrive un algoritmo di interpolazione basato sulla risoluzione del sistema di equazioni lineari (30.4). Utilizzando gli algoritmi di fattorizzazione LU descritti nel Capitolo 28, possiamo risolvere queste equazioni nel tempo $O(n^3)$.

Un algoritmo più veloce per l'interpolazione di n punti si basa sulla ***formula di Lagrange***:

$$A(x) = \sum_{k=0}^{n-1} y_k \frac{\prod_{j \neq k} (x - x_j)}{\prod_{j \neq k} (x_k - x_j)} \tag{30.5}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Potreste verificare che il lato destro di questa equazione è un polinomio di grado limite n che soddisfa $A(x_k) = y_k$ per ogni k. L'Esercizio 30.1-5 chiede come calcolare nel tempo $\Theta(n^2)$ i coefficienti di A utilizzando la formula di Lagrange.

Quindi, la valutazione e l'interpolazione di n punti sono operazioni inverse ben definite che effettuano le conversioni tra la rappresentazione per coefficienti di un polinomio e una rappresentazione per punti.[1] Gli algoritmi prima descritti per questi problemi richiedono un tempo $\Theta(n^2)$.

La rappresentazione per punti è molto comoda per molte operazioni con i polinomi. Per l'addizione, se $C(x) = A(x) + B(x)$, allora $C(x_k) = A(x_k) + B(x_k)$ per ogni punto x_k. Più precisamente, se abbiamo una rappresentazione per punti di A

$$\{(x_0, y_0), (x_1, y_1), \ldots, (x_{n-1}, y_{n-1})\}$$

e una rappresentazione per punti di B

$$\{(x_0, y'_0), (x_1, y'_1), \ldots, (x_{n-1}, y'_{n-1})\}$$

(notate che A e B sono valutati negli *stessi* n punti), allora una rappresentazione per punti di C è

$$\{(x_0, y_0 + y'_0), (x_1, y_1 + y'_1), \ldots, (x_{n-1}, y_{n-1} + y'_{n-1})\}$$

Quindi, il tempo per sommare due polinomi di grado limite n nella rappresentazione per punti è $\Theta(n)$.

Analogamente, la rappresentazione per punti è comoda per moltiplicare i polinomi. Se $C(x) = A(x)B(x)$, allora $C(x_k) = A(x_k)B(x_k)$ per ogni punto x_k; quindi è possibile moltiplicare punto per punto una rappresentazione per punti di A per una rappresentazione per punti di B per ottenere una rappresentazione per punti di C. Tuttavia, dobbiamo risolvere il problema che grado(C) = grado(A) + grado(B); se A e B sono di grado limite n, allora C è di grado limite $2n$. Una normale rappresentazione per punti di A e B è formata da n coppie di valori per ogni polinomio; moltiplicandole si ottengono n coppie di valori, ma occorrono $2n$ coppie per interpolare un polinomio unico C di grado limite $2n$ (vedere l'Esercizio 30.1-4). Quindi, dobbiamo iniziare con delle rappresentazioni per punti "estese" di A e B, ciascuna delle quali è formata da $2n$ coppie di valori. Data una rappresentazione per punti estesa di A

$$\{(x_0, y_0), (x_1, y_1), \ldots, (x_{2n-1}, y_{2n-1})\}$$

e una corrispondente rappresentazione per punti estesa di B

$$\{(x_0, y'_0), (x_1, y'_1), \ldots, (x_{2n-1}, y'_{2n-1})\}$$

allora una rappresentazione per punti di C è

$$\{(x_0, y_0 y'_0), (x_1, y_1 y'_1), \ldots, (x_{2n-1}, y_{2n-1} y'_{2n-1})\}$$

Dati due polinomi di input nella rappresentazione per punti estesa, notiamo che il tempo per moltiplicarli per ottenere la forma per punti del risultato è $\Theta(n)$, molto meno del tempo richiesto per moltiplicare i polinomi nella forma per coefficienti.

Infine, dobbiamo spiegare come valutare un polinomio, dato nella forma per punti, in un nuovo punto. Per questo problema, non sembra esserci un approccio più semplice che convertire prima il polinomio nella forma per coefficienti e poi nel valutarlo nel nuovo punto.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016; 220797-0 Copyright © 2022 McGraw-Hill Education (Italy)

[1] È noto che l'interpolazione è un problema complesso dal punto di vista della stabilità numerica. Sebbene gli approcci qui descritti siano matematicamente corretti, piccole differenze negli input o errori di arrotondamento durante i calcoli possono causare grandi differenze nei risultati.

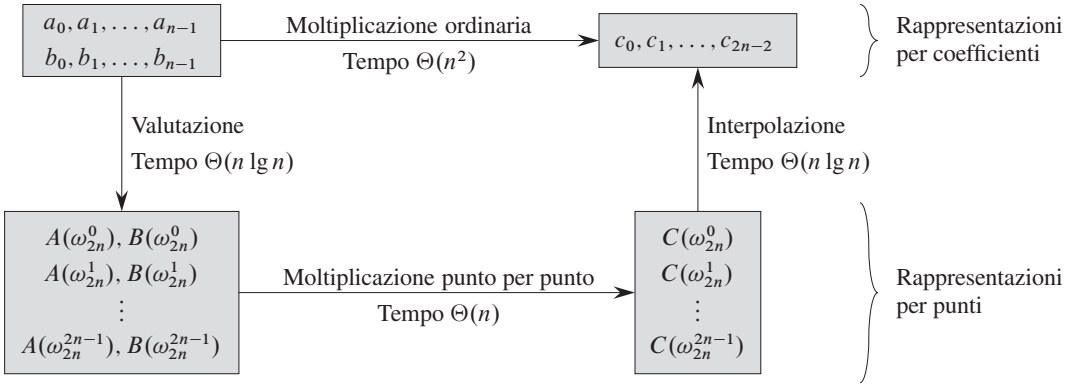


Figura 30.1 Schema grafico di un efficiente processo di moltiplicazione di polinomi. Le rappresentazioni in alto sono nella forma per coefficienti, mentre quelle in basso sono nella forma per punti. Le frecce da sinistra a destra corrispondono all'operazione di moltiplicazione. I termini ω_{2n} sono le radici $(2n)$-esime complesse dell'unità.

Moltiplicazione rapida di polinomi nella forma per coefficienti

Possiamo utilizzare il metodo della moltiplicazione con tempo lineare per i polinomi nella forma per punti per accelerare la moltiplicazione dei polinomi nella forma per coefficienti? La risposta dipende dalla nostra capacità di convertire rapidamente un polinomio dalla forma per coefficienti alla forma per punti (valutazione) e viceversa (interpolazione).

Possiamo utilizzare i punti che vogliamo come punti di valutazione, tuttavia se scegliamo con attenzione questi punti, possiamo effettuare le conversioni tra le rappresentazioni in un tempo di appena $\Theta(n \lg n)$. Come vedremo nel Paragrafo 30.2, se scegliamo "le radici complesse dell'unità" come punti di valutazione, possiamo ottenere una rappresentazione per punti prendendo la trasformata discreta di Fourier o DFT (Discrete Fourier Transform) di un vettore di coefficienti. L'operazione inversa, l'interpolazione, può essere eseguita prendendo la "DFT inversa" delle coppie di valori, ottenendo un vettore di coefficienti. Nel Paragrafo 30.2 dimostreremo come la FFT calcola nel tempo $\Theta(n \lg n)$ la DFT e la DFT inversa.

La Figura 30.1 rappresenta graficamente questa strategia. C'è un piccolo dettaglio che riguarda il grado limite. Il prodotto di due polinomi di grado limite n è un polinomio di grado limite $2n$. Prima di valutare i polinomi di input A e B, quindi, dobbiamo raddoppiare il loro grado limite a $2n$, aggiungendo n coefficienti 0 di ordine superiore. Poiché i vettori hanno $2n$ elementi, utilizziamo "le radici $(2n)$-esime complesse dell'unità", che sono indicate con i termini ω_{2n} nella Figura 30.1.

Data la FFT, abbiamo la seguente procedura con tempo $\Theta(n \lg n)$ per moltiplicare due polinomi $A(x)$ e $B(x)$ di grado limite n, dove le rappresentazioni dell'input e dell'output sono nella forma per coefficienti. Supponiamo che n sia una potenza di 2; questa ipotesi può essere sempre soddisfatta aggiungendo dei coefficienti 0 di ordine superiore.

Acquistato da Michele Michele su Webster | 2022-07-07 23:12 | Numero Ordine Libreria: 1995803016-2207-0 Copyright © 2022 McGraw-Hill Education (Italy)

1. *Raddoppiare il grado limite:* creare le rappresentazioni per coefficienti di $A(x)$ e $B(x)$ come polinomi di grado limite $2n$ aggiungendo a ciascuna rappresentazione n coefficienti 0 di ordine superiore.

2. *Valutazione:* calcolare le rappresentazioni per punti di $A(x)$ e $B(x)$ di lunghezza $2n$ applicando due volte la FFT di ordine $2n$. Queste rappresentazioni contengono i valori dei due polinomi nelle radici $(2n)$-esime dell'unità.
3. *Moltiplicazione punto per punto:* calcolare una rappresentazione per punti del polinomio $C(x) = A(x)B(x)$ moltiplicando questi valori punto per punto. La rappresentazione risultante contiene il valore di $C(x)$ in ciascuna radice $(2n)$-esima dell'unità.
4. *Interpolazione:* creare la rappresentazione per coefficienti del polinomio $C(x)$ applicando una sola volta la FFT a $2n$ coppie di valori per calcolare la DFT inversa.

I passi (1) e (3) richiedono un tempo $\Theta(n)$; i passi (2) e (4) richiedono un tempo $\Theta(n \lg n)$. Quindi, dopo avere spiegato come utilizzare la FFT, avremo dimostrato il seguente teorema.

Teorema 30.2

Il prodotto di due polinomi di grado limite n può essere calcolato nel tempo $\Theta(n \lg n)$, con entrambe le rappresentazioni dell'input e dell'output nella forma per coefficienti. ■

Esercizi

30.1-1

Moltiplicate i polinomi $A(x) = 7x^3 - x^2 + x - 10$ e $B(x) = 8x^3 - 6x + 3$ utilizzando le equazioni (30.1) e (30.2).

30.1-2

La valutazione di un polinomio $A(x)$ di grado limite n in un dato punto x_0 può essere fatta anche dividendo $A(x)$ per il polinomio $(x - x_0)$ per ottenere un polinomio quoziente $q(x)$ di grado limite $n - 1$ e un resto r, tali che

$$A(x) = q(x)(x - x_0) + r$$

Chiaramente, $A(x_0) = r$. Spiegate come calcolare il resto r e i coefficienti di $q(x)$ nel tempo $\Theta(n)$ conoscendo x_0 e i coefficienti di A.

30.1-3

Derivate una rappresentazione per punti di $A^{\text{rev}}(x) = \sum_{j=0}^{n-1} a_{n-1-j} x^j$ da una rappresentazione per punti di $A(x) = \sum_{j=0}^{n-1} a_j x^j$, supponendo che nessun punto sia 0.

30.1-4

Dimostrate che occorrono n punti distinti per specificare unicamente un polinomio di grado limite n, ovvero, se i punti noti sono meno di n, essi non sono in grado di identificare un unico polinomio di grado limite n. (*Suggerimento:* utilizzando il Teorema 30.1, che cosa potreste dire su un insieme di $n - 1$ punti al quale aggiungete un altro punto arbitrariamente scelto?)

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 1995032016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

30.1-5

Spiegate come utilizzare l'equazione (30.5) per calcolare l'interpolazione nel tempo $\Theta(n^2)$. (*Suggerimento:* prima calcolate la rappresentazione per coefficienti del polinomio $\prod_j (x - x_j)$ e poi dividete per $(x - x_k)$ come richiede il numeratore

di ciascun termine; usate l'Esercizio 30.1-2. Ciascuno degli n denominatori può essere calcolato nel tempo $O(n)$.)

30.1-6
Spiegate perché è sbagliato l'approccio "ovvio" alla divisione dei polinomi che usa la rappresentazione per punti, dividendo cioè i valori y corrispondenti. Esaminate separatamente il caso in cui la divisione ottenuta sia esatta e il caso in cui non lo sia.

30.1-7
Considerate due insiemi A e B, ciascuno contenente n interi nell'intervallo da 0 a $10n$. Bisogna calcolare la ***somma cartesiana*** di A e B, definita da

$$C = \{x + y : x \in A \text{ e } y \in B\}$$

Notate che gli interi in C si trovano all'intervallo da 0 a $20n$. Bisogna trovare gli elementi di C e il numero di volte che ciascun elemento di C è realizzato come una somma di elementi in A e B. Dimostrate che il problema può essere risolto nel tempo $O(n \lg n)$ (*suggerimento:* rappresentate A e B come polinomi di grado al più $10n$).

30.2 DFT e FFT

Nel Paragrafo 30.1 abbiamo detto che, se utilizziamo le radici complesse dell'unità, possiamo valutare e interpolare i polinomi nel tempo $\Theta(n \lg n)$. In questo paragrafo definiamo le radici complesse dell'unità e studiamo le loro proprietà, definiamo la DFT e poi spieghiamo come la FFT calcola la DFT e la sua inversa in un tempo $\Theta(n \lg n)$.

Radici complesse dell'unità

Una ***radice n-esima complessa dell'unità*** è un numero complesso ω tale che

$$\omega^n = 1$$

Esistono esattamente n radici n-esime complesse dell'unità: $e^{2\pi ik/n}$ per $k = 0, 1, \ldots, n-1$. Per interpretare questa formula, utilizziamo la definizione dell'esponenziale di un numero complesso:

$$e^{iu} = \cos(u) + i \sin(u)$$

La Figura 30.2 illustra che le n radici complesse dell'unità sono equamente distanziate nel cerchio di raggio unitario con centro nell'origine del piano complesso. Il valore

$$\omega_n = e^{2\pi i/n} \tag{30.6}$$

è detto ***radice n-esima principale dell'unità***; tutte le altre radici n-esime complesse dell'unità sono potenze di ω_n.[2]

Acquistato da Michele Machetti - Wed Jan 25 2023 07:27:28 - Licenza Individuale - Vietata la copia - © 2022 McGraw-Hill Education (Italy)

[2]Molti altri autori definiscono ω_n in modo differente: $\omega_n = e^{-2\pi i/n}$. Questa definizione alternativa di solito viene utilizzata nelle applicazioni per l'elaborazione dei segnali. La matematica sottostante è sostanzialmente la stessa con entrambe le definizioni di ω_n.

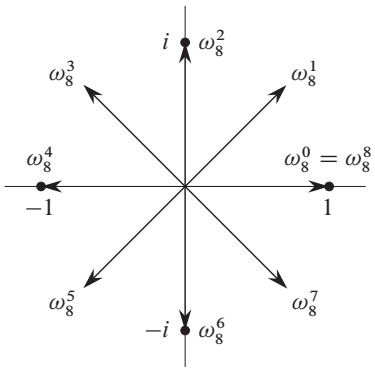


Figura 30.2 I valori di $\omega_8^0, \omega_8^1, \ldots, \omega_8^7$ nel piano complesso, dove $\omega_8 = e^{2\pi i/8}$ è la radice ottava principale dell'unità.

Le n radici n-esime complesse dell'unità

$$\omega_n^0, \omega_n^1, \ldots, \omega_n^{n-1}$$

formano un gruppo rispetto alla moltiplicazione (vedere il Paragrafo 31.3). Questo gruppo ha la stessa struttura del gruppo additivo $(\mathbb{Z}_n, +)$ modulo n, perché $\omega_n^n = \omega_n^0 = 1$ implica che $\omega_n^j \omega_n^k = \omega_n^{j+k} = \omega_n^{(j+k) \bmod n}$. Analogamente, $\omega_n^{-1} = \omega_n^{n-1}$. Le proprietà essenziali delle radici n-esime complesse dell'unità sono definite nei seguenti lemmi.

Lemma 30.3 (Lemma della cancellazione)

Per $n \geq 0$, $k \geq 0$ e $d > 0$ interi qualsiasi, si ha

$$\omega_{dn}^{dk} = \omega_n^k \tag{30.7}$$

Dimostrazione Il lemma segue direttamente dall'equazione (30.6), perché

$$\begin{aligned} \omega_{dn}^{dk} &= \left(e^{2\pi i/dn}\right)^{dk} \\ &= \left(e^{2\pi i/n}\right)^k \\ &= \omega_n^k \end{aligned}$$

■

Corollario 30.4

Se $n > 0$ è un numero intero pari, si ha

$$\omega_n^{n/2} = \omega_2 = -1$$

Dimostrazione Lasciamo al lettore il compito di dimostrare questo corollario (Esercizio 30.2-1). ■

Acquistato da Michele Michele su Webster il 2022-07-07 22:12:12 Utente Ordine Libreria 20250216.220707 Copyright © 2022, McGraw-Hill Education (Italy)

Lemma 30.5 (Lemma del dimezzamento)

Se $n > 0$ è pari, allora i quadrati delle n radici n-esime complesse dell'unità sono le $n/2$ radici $(n/2)$-esime complesse dell'unità.

Dimostrazione Per il lemma della cancellazione, abbiamo $(\omega_n^k)^2 = \omega_{n/2}^k$, con k intero qualsiasi non negativo. Notate che, elevando al quadrato tutte le radici

n-esime complesse dell'unità, ciascuna radice $(n/2)$-esima dell'unità è ottenuta due volte, perché

$$\begin{aligned}(\omega_n^{k+n/2})^2 &= \omega_n^{2k+n} \\ &= \omega_n^{2k}\omega_n^n \\ &= \omega_n^{2k} \\ &= (\omega_n^k)^2\end{aligned}$$

Quindi, ω_n^k e $\omega_n^{k+n/2}$ hanno lo stesso quadrato. Questa proprietà può anche essere dimostrata utilizzando il Corollario 30.4, in quanto $\omega_n^{n/2} = -1$ implica $\omega_n^{k+n/2} = -\omega_n^k$, e quindi $(\omega_n^{k+n/2})^2 = (\omega_n^k)^2$. ■

Come vedremo, il lemma del dimezzamento è essenziale al nostro approccio divide et impera per effettuare le trasformazioni tra le rappresentazioni per punti e per coefficienti dei polinomi, perché garantisce che la dimensione dei sottoproblemi ricorsivi sia soltanto la metà del problema originale.

Lemma 30.6 (Lemma della somma)
Per $n \geq 1$ intero qualsiasi e k intero non negativo e non divisibile per n, si ha

$$\sum_{j=0}^{n-1} \left(\omega_n^k\right)^j = 0$$

Dimostrazione L'Equazione (A.5) si applica sia ai valori complessi sia ai valori reali, quindi abbiamo

$$\begin{aligned}\sum_{j=0}^{n-1} \left(\omega_n^k\right)^j &= \frac{(\omega_n^k)^n - 1}{\omega_n^k - 1} \\ &= \frac{(\omega_n^n)^k - 1}{\omega_n^k - 1} \\ &= \frac{(1)^k - 1}{\omega_n^k - 1} \\ &= 0\end{aligned}$$

Richiedendo che k non sia divisibile per n, si ha la garanzia che il denominatore non sia 0, perché $\omega_n^k = 1$ soltanto se k è divisibile per n. ■

Trasformata discreta di Fourier (DFT)

Ricordiamo che vogliamo valutare un polinomio

$$A(x) = \sum_{j=0}^{n-1} a_j x^j$$

di grado limite n in $\omega_n^0, \omega_n^1, \omega_n^2, \ldots, \omega_n^{n-1}$ (ovvero nelle n radici n-esime complesse dell'unità).[3] Supponiamo che il polinomio A sia dato nella forma per coeffi-

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 19950316-220707-0. Copyright © 2022. McGraw-Hill Education (Italy)

[3]La lunghezza n in effetti è ciò che abbiamo chiamato $2n$ nel Paragrafo 30.1, perché abbiamo raddoppiato il grado limite dei polinomi prima della loro valutazione. Quindi, nel contesto della moltiplicazione dei polinomi, in pratica operiamo con le radici $(2n)$-esime complesse dell'unità.

cienti: $a = (a_0, a_1, \ldots, a_{n-1})$. Definiamo i risultati y_k, per $k = 0, 1, \ldots, n-1$, così:

$$\begin{aligned} y_k &= A(\omega_n^k) \\ &= \sum_{j=0}^{n-1} a_j \omega_n^{kj} \end{aligned} \tag{30.8}$$

Il vettore $y = (y_0, y_1, \ldots, y_{n-1})$ è la ***trasformata discreta di Fourier*** o ***DFT*** (Discrete Fourier Transform) del vettore dei coefficienti $a = (a_0, a_1, \ldots, a_{n-1})$, che si scrive anche $y = \mathrm{DFT}_n(a)$.

Trasformata rapida di Fourier (FFT)

Utilizzando un metodo detto ***trasformata rapida di Fourier*** o ***FFT*** (Fast Fourier Transform) che sfrutta le proprietà speciali delle radici complesse dell'unità, è possibile calcolare $\mathrm{DFT}_n(a)$ nel tempo $\Theta(n \lg n)$, mentre il metodo semplice richiede un tempo $\Theta(n^2)$. Noi supponiamo che n sia una potenza esatta di 2. Sebbene esistano delle strategie per trattare valori di n che non sono potenze di 2, esse esulano dagli obiettivi di questo libro.

Il metodo FFT adotta una strategia divide et impera, che usa separatamente i coefficienti di indice pari e di indice dispari del polinomio $A(x)$ per definire i due nuovi polinomi $A^{[0]}(x)$ e $A^{[1]}(x)$ di grado limite $n/2$:

$$\begin{aligned} A^{[0]}(x) &= a_0 + a_2 x + a_4 x^2 + \cdots + a_{n-2} x^{n/2-1} \\ A^{[1]}(x) &= a_1 + a_3 x + a_5 x^2 + \cdots + a_{n-1} x^{n/2-1} \end{aligned}$$

Notate che $A^{[0]}$ contiene tutti i coefficienti di indice pari di A (la rappresentazione binaria dell'indice termina con 0) e $A^{[1]}$ contiene tutti i coefficienti di indice dispari (la rappresentazione binaria dell'indice termina con 1). Ne segue che

$$A(x) = A^{[0]}(x^2) + x A^{[1]}(x^2) \tag{30.9}$$

e quindi il problema di valutare $A(x)$ in $\omega_n^0, \omega_n^1, \ldots, \omega_n^{n-1}$ si riduce a

1. calcolare i polinomi $A^{[0]}(x)$ e $A^{[1]}(x)$ di grado limite $n/2$ nei punti

   $$(\omega_n^0)^2, (\omega_n^1)^2, \ldots, (\omega_n^{n-1})^2 \tag{30.10}$$

   e poi
2. combinare i risultati secondo l'equazione (30.9).

Per il lemma del dimezzamento, la lista dei valori (30.10) non è composta da n valori distinti, ma soltanto dalle $n/2$ radici $(n/2)$-esime complesse dell'unità, dove ciascuna radice si presenta esattamente due volte. Quindi, i polinomi $A^{[0]}$ e $A^{[1]}$ di grado limite $n/2$ sono calcolati ricorsivamente nelle $n/2$ radici $(n/2)$-esime complesse dell'unità. Questi sottoproblemi hanno esattamente la stessa forma del problema originale, ma la loro dimensione è dimezzata. Abbiamo diviso con successo il calcolo di una DFT_n di n elementi nel calcolo di due $\mathrm{DFT}_{n/2}$ di $n/2$ elementi. Questa scomposizione è la base per il successivo algoritmo FFT ricorsivo, che calcola la DFT di un vettore $a = (a_0, a_1, \ldots, a_{n-1})$ di n elementi, dove n è una potenza di 2.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

```
RECURSIVE-FFT(a)
 1  n = a.length              // n è una potenza di 2
 2  if n == 1
 3      return a
 4  ω_n = e^{2πi/n}
 5  ω = 1
 6  a^[0] = (a_0, a_2, ..., a_{n-2})
 7  a^[1] = (a_1, a_3, ..., a_{n-1})
 8  y^[0] = RECURSIVE-FFT(a^[0])
 9  y^[1] = RECURSIVE-FFT(a^[1])
10  for k = 0 to n/2 − 1
11      y_k = y_k^[0] + ω y_k^[1]
12      y_{k+(n/2)} = y_k^[0] − ω y_k^[1]
13      ω = ω ω_n
14  return y                  // Si suppone che y sia un vettore colonna.
```

La procedura RECURSIVE-FFT opera nel modo seguente. Le righe 2–3 rappresentano la base della ricorsione; la DFT di un elemento è l'elemento stesso, perché in questo caso

$$\begin{aligned} y_0 &= a_0\,\omega_1^0 \\ &= a_0 \cdot 1 \\ &= a_0 \end{aligned}$$

Le righe 6–7 definiscono i vettori dei coefficienti per i polinomi $A^{[0]}$ e $A^{[1]}$. Le righe 4, 5 e 13 garantiscono che il valore di ω sia opportunamente aggiornato in modo che, quando vengono eseguite le righe 11–12, si abbia $\omega = \omega_n^k$. (Mantenendo il valore corrente di ω da un'iterazione all'altra, è possibile risparmiare tempo sul calcolo di ω_n^k, evitando di calcolarlo dall'inizio ad ogni iterazione del ciclo **for**.) Le righe 8–9 eseguono i calcoli ricorsivi di $\mathrm{DFT}_{n/2}$, ponendo, per $k = 0, 1, \ldots, n/2 - 1$

$$\begin{aligned} y_k^{[0]} &= A^{[0]}(\omega_{n/2}^k) \\ y_k^{[1]} &= A^{[1]}(\omega_{n/2}^k) \end{aligned}$$

ovvero, poiché $\omega_{n/2}^k = \omega_n^{2k}$ per il lemma della cancellazione,

$$\begin{aligned} y_k^{[0]} &= A^{[0]}(\omega_n^{2k}) \\ y_k^{[1]} &= A^{[1]}(\omega_n^{2k}) \end{aligned}$$

Le righe 11–12 combinano i risultati dei calcoli ricorsivi di $\mathrm{DFT}_{n/2}$. Per $y_0, y_1, \ldots, y_{n/2-1}$, la riga 11 fornisce

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned} y_k &= y_k^{[0]} + \omega_n^k y_k^{[1]} \\ &= A^{[0]}(\omega_n^{2k}) + \omega_n^k A^{[1]}(\omega_n^{2k}) \\ &= A(\omega_n^k) \qquad \text{(per l'equazione (30.9))} \end{aligned}$$

Per $y_{n/2}, y_{n/2+1}, \ldots, y_{n-1}$, ponendo $k = 0, 1, \ldots, n/2 - 1$, la riga 12 fornisce

$$
\begin{aligned}
y_{k+(n/2)} &= y_k^{[0]} - \omega_n^k y_k^{[1]} \\
&= y_k^{[0]} + \omega_n^{k+(n/2)} y_k^{[1]} && \text{(perché } \omega_n^{k+(n/2)} = -\omega_n^k\text{)} \\
&= A^{[0]}(\omega_n^{2k}) + \omega_n^{k+(n/2)} A^{[1]}(\omega_n^{2k}) \\
&= A^{[0]}(\omega_n^{2k+n}) + \omega_n^{k+(n/2)} A^{[1]}(\omega_n^{2k+n}) && \text{(perché } \omega_n^{2k+n} = \omega_n^{2k}\text{)} \\
&= A(\omega_n^{k+(n/2)}) && \text{(per l'equazione (30.9))}
\end{aligned}
$$

Quindi, il vettore y restituito da RECURSIVE-FFT è effettivamente la DFT del vettore di input a.

Le righe 11 e 12 moltiplicano ciascun valore $y_k^{[1]}$ per ω_n^k, per $k = 0, 1, \ldots, n/2 - 1$. La riga 11 somma questo prodotto a $y_k^{[0]}$ e la riga 12 lo sottrae. Poiché ogni fattore ω_n^k è utilizzato sia nella forma positiva sia nella forma negativa, i fattori ω_n^k sono detti ***fattori di rotazione*** (*twiddle factor*).

Per determinare il tempo di esecuzione della procedura RECURSIVE-FFT, notiamo che, escludendo le chiamate ricorsive, ogni invocazione richiede un tempo $\Theta(n)$, dove n è la lunghezza del vettore di input. La ricorrenza per il tempo di esecuzione è quindi

$$
\begin{aligned}
T(n) &= 2T(n/2) + \Theta(n) \\
&= \Theta(n \lg n)
\end{aligned}
$$

Dunque, la trasformata rapida di Fourier consente di valutare nel tempo $\Theta(n \lg n)$ un polinomio di grado limite n nelle radici n-esime complesse dell'unità.

Interpolazione delle radici complesse dell'unità

Adesso completiamo lo schema di moltiplicazione dei polinomi spiegando come interpolare le radici complesse dell'unità con un polinomio, il che ci permette di riconvertire una forma per punti in una forma per coefficienti. Interpoliamo scrivendo la DFT come un'equazione matriciale e poi esaminiamo la forma della matrice inversa.

In base all'equazione (30.4) possiamo scrivere la DFT come il prodotto matriciale $y = V_n a$, dove V_n è una matrice di Vandermonde che contiene le potenze appropriate di ω_n:

$$
\begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega_n & \omega_n^2 & \omega_n^3 & \cdots & \omega_n^{n-1} \\ 1 & \omega_n^2 & \omega_n^4 & \omega_n^6 & \cdots & \omega_n^{2(n-1)} \\ 1 & \omega_n^3 & \omega_n^6 & \omega_n^9 & \cdots & \omega_n^{3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_n^{n-1} & \omega_n^{2(n-1)} & \omega_n^{3(n-1)} & \cdots & \omega_n^{(n-1)(n-1)} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{n-1} \end{pmatrix}
$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995030016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

L'elemento (k, j) di V_n è ω_n^{kj}, per $j, k = 0, 1, \ldots, n - 1$; gli esponenti degli elementi di V_n formano una tavola pitagorica.

Per l'operazione inversa, che scriviamo come $a = \text{DFT}_n^{-1}(y)$, procediamo moltiplicando y per la matrice V_n^{-1}, l'inversa di V_n.

Teorema 30.7
Per $j, k = 0, 1, \ldots, n-1$, l'elemento (j, k) di V_n^{-1} è ω_n^{-kj}/n.

Dimostrazione Dimostriamo che $V_n^{-1}V_n = I_n$, la matrice identità $n \times n$. Consideriamo l'elemento (j, j') di $V_n^{-1}V_n$:

$$\begin{aligned}[V_n^{-1}V_n]_{jj'} &= \sum_{k=0}^{n-1}(\omega_n^{-kj}/n)(\omega_n^{kj'}) \\ &= \sum_{k=0}^{n-1}\omega_n^{k(j'-j)}/n\end{aligned}$$

Questa sommatoria è 1 se $j' = j$, altrimenti è 0 per il lemma della somma (Lemma 30.6). Notate che, per potere applicare il lemma della somma, ci basiamo sul fatto che $-(n-1) \le j' - j \le n-1$, e quindi $j' - j$ non è divisibile per n. ■

Data la matrice inversa V_n^{-1}, abbiamo che la $\mathrm{DFT}_n^{-1}(y)$ è data da

$$a_j = \frac{1}{n}\sum_{k=0}^{n-1} y_k \omega_n^{-kj} \tag{30.11}$$

per $j = 0, 1, \ldots, n-1$. Confrontando le equazioni (30.8) e (30.11), notiamo che, se modifichiamo l'algoritmo FFT scambiando i ruoli di a e y, sostituendo ω_n con ω_n^{-1} e dividendo ciascun elemento del risultato per n, calcoliamo la DFT inversa (vedere l'Esercizio 30.2-4). Quindi, anche la DFT_n^{-1} può essere calcolata nel tempo $\Theta(n \lg n)$.

Quindi, utilizzando la FFT e la FFT inversa, possiamo convertire un polinomio di grado limite n dalla rappresentazione per coefficienti alla rappresentazione per punti e viceversa nel tempo $\Theta(n \lg n)$. Nel contesto della moltiplicazione dei polinomi, abbiamo dimostrato il seguente teorema.

Teorema 30.8 (Teorema della convoluzione)
Per due vettori qualsiasi a e b di lunghezza n, dove n è una potenza di 2, si ha

$$a \otimes b = \mathrm{DFT}_{2n}^{-1}(\mathrm{DFT}_{2n}(a) \cdot \mathrm{DFT}_{2n}(b))$$

Qui i vettori a e b vengono riempiti di 0 fino a raggiungere la lunghezza $2n$ e il simbolo $\cdot$ indica il prodotto elemento per elemento di due vettori di $2n$ elementi. ■

Esercizi

30.2-1
Dimostrate il Corollario 30.4.

30.2-2
Calcolate la DFT del vettore $(0, 1, 2, 3)$.

30.2-3
Svolgete l'Esercizio 30.1-1 utilizzando lo schema con tempo $\Theta(n \lg n)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

30.2-4
Scrivete lo pseudocodice per calcolare la DFT_n^{-1} nel tempo $\Theta(n \lg n)$.

30.2-5
Descrivete la generalizzazione della procedura FFT al caso in cui n sia una potenza di 3. Scrivete un'equazione di ricorrenza per il tempo di esecuzione e risolvetela.

30.2-6 ⋆

Anziché eseguire una FFT di n elementi nel campo dei numeri complessi (dove n è pari), supponete di utilizzare l'anello $\mathbb{Z}_m$ di interi modulo m, dove $m = 2^{tn/2} + 1$ e t è un intero positivo arbitrario. Utilizzate $\omega = 2^t$, anziché ω_n, come radice n-esima principale dell'unità, modulo m. Dimostrate che la DFT e la DFT inversa sono ben definite in questo sistema.

30.2-7

Data una lista di valori $z_0, z_1, \ldots, z_{n-1}$ (eventualmente con ripetizioni), spiegate come trovare i coefficienti di un polinomio $P(x)$ di grado limite $n+1$ che ha zeri soltanto in $z_0, z_1, \ldots, z_{n-1}$ (eventualmente con ripetizioni). La vostra procedura dovrebbe essere eseguita nel tempo $O(n \lg^2 n)$. (*Suggerimento:* il polinomio $P(x)$ ha uno zero in z_j se e soltanto se $P(x)$ è un multiplo di $(x - z_j)$.)

30.2-8 ⋆

La ***trasformata chirp*** di un vettore $a = (a_0, a_1, \ldots, a_{n-1})$ è il vettore $y = (y_0, y_1, \ldots, y_{n-1})$, dove $y_k = \sum_{j=0}^{n-1} a_j z^{kj}$ e z è un numero complesso qualsiasi. La DFT è quindi un caso speciale della trasformata chirp, ottenuta prendendo $z = \omega_n$. Dimostrate che la trasformata chirp può essere valutata nel tempo $O(n \lg n)$ per qualsiasi numero complesso z. (*Suggerimento:* utilizzate l'equazione

$$y_k = z^{k^2/2} \sum_{j=0}^{n-1} \left(a_j z^{j^2/2}\right) \left(z^{-(k-j)^2/2}\right)$$

per vedere la trasformata chirp come una convoluzione.)

30.3 Implementazioni efficienti della FFT

Poiché le applicazioni pratiche della DFT, come l'elaborazione dei segnali, richiedono la massima velocità, questo paragrafo esamina due implementazioni efficienti della FFT. Prima esamineremo una versione iterativa dell'algoritmo FFT che viene eseguito nel tempo $\Theta(n \lg n)$, ma ha una costante nascosta nella notazione Θ più piccola di quella dell'implementazione ricorsiva descritta nel Paragrafo 30.2. (Se implementata opportunamente, la versione ricorsiva potrebbe usare in modo più efficiente la cache hardware.) Poi utilizzeremo le intuizioni che ci hanno portato all'implementazione iterativa per progettare un efficiente circuito parallelo per la FFT.

Implementazione iterativa della FFT

Notiamo innanzi tutto che il ciclo **for** (righe 10–13) di RECURSIVE-FFT calcola due volte il valore $\omega_n^k y_k^{[1]}$. Nella terminologia dei compilatori, questo valore è detto ***sottoespressione comune***. Possiamo modificare il ciclo per calcolare tale valore una volta soltanto, memorizzandolo in una variabile temporanea t.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

```
for k = 0 to n/2 − 1
    t = ω y_k^[1]
    y_k = y_k^[0] + t
    y_{k+(n/2)} = y_k^[0] − t
    ω = ω ω_n
```

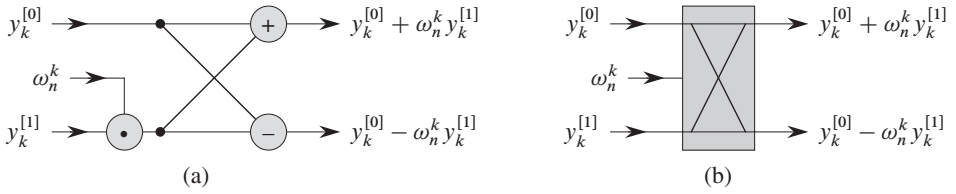


Figura 30.3
Operazione butterfly. **(a)** I due valori di input entrano da sinistra, il fattore di twiddle ω_n^k è moltiplicato per $y_k^{[1]}$; la somma e la differenza sono gli output a destra. **(b)** Schema semplificato di un'operazione butterfly. Utilizzeremo questa rappresentazione in un circuito parallelo per la FFT.

Chiameremo ***operazione butterfly*** la serie delle operazioni eseguite in questo ciclo: moltiplicare il fattore di rotazione $\omega = \omega_n^k$ per $y_k^{[1]}$, memorizzare il prodotto in t e sommare e sottrarre t da $y_k^{[0]}$. La Figura 30.3 illustra schematicamente l'operazione butterfly.

Dimostriamo adesso come rendere iterativa (anziché ricorsiva) la struttura dell'algoritmo FFT. Nella Figura 30.4 abbiamo disposto in una struttura ad albero i vettori di input per le chiamate ricorsive di una invocazione di RECURSIVE-FFT, dove la chiamata iniziale è per $n = 8$. L'albero ha un nodo per ogni chiamata della procedura, etichettato con il corrispondente vettore di input. Ogni invocazione di RECURSIVE-FFT fa due chiamate ricorsive, a meno che non abbia ricevuto un vettore con un solo elemento. La prima chiamata diventa il figlio sinistro e la seconda chiamata diventa il figlio destro.

Osservando l'albero, notiamo che, se disponessimo gli elementi del vettore iniziale a nell'ordine in cui essi appaiono nelle foglie, potremmo effettuare l'esecuzione della procedura RECURSIVE-FFT partendo dal basso, invece che dall'alto. Innanzi tutto, prendiamo gli elementi in coppie, calcoliamo la DFT di ogni coppia utilizzando un'operazione butterfly, e sostituiamo la coppia con la sua DFT. Adesso il vettore contiene $n/2$ DFT di 2 elementi. Poi, prendiamo queste $n/2$ DFT in coppie e calcoliamo le DFT di quattro elementi che si ottengono eseguendo due operazioni butterfly, sostituendo due DFT di 2 elementi con una DFT di 4 elementi. Il vettore adesso contiene $n/4$ DFT di 4 elementi. Continuiamo in questo modo finché il vettore conterrà due DFT di $n/2$ elementi, che, con $n/2$ operazioni butterfly, possiamo riunire per formare la DFT finale di n elementi.

Per tradurre queste considerazioni in codice, utilizziamo un array $A[0\,..\,n-1]$ che inizialmente contiene gli elementi del vettore di input a nell'ordine in cui essi appaiono nelle foglie dell'albero della Figura 30.4. (Vedremo più avanti come determinare questo ordine, che è detto permutazione per inversione di bit.) Poiché la combinazione deve essere fatta a ogni livello dell'albero, introduciamo una variabile s per contare i livelli, cha vanno da 1 (in basso, quando combiniamo le coppie per formare le DFT di 2 elementi) a $\lg n$ (in alto, quando combiniamo due DFT di $n/2$ elementi per produrre il risultato finale). L'algoritmo quindi ha la seguente struttura:

```
1  for s = 1 to lg n
2      for k = 0 to n − 1 by 2^s
3          combina le due DFT di 2^{s−1} elementi in
           A[k .. k + 2^{s−1} − 1] e A[k + 2^{s−1} .. k + 2^s − 1]
           in una DFT di 2^s elementi in A[k .. k + 2^s − 1]
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Possiamo esprimere il corpo del ciclo (riga 3) con uno pseudocodice più preciso. Basta copiare il ciclo **for** dalla procedura RECURSIVE-FFT, identificando $y^{[0]}$ con $A[k\,..\,k + 2^{s-1} - 1]$ e $y^{[1]}$ con $A[k + 2^{s-1}\,..\,k + 2^s - 1]$. Il fattore di rotazione utilizzato in ogni operazione butterfly dipende dal valore di s; è una potenza

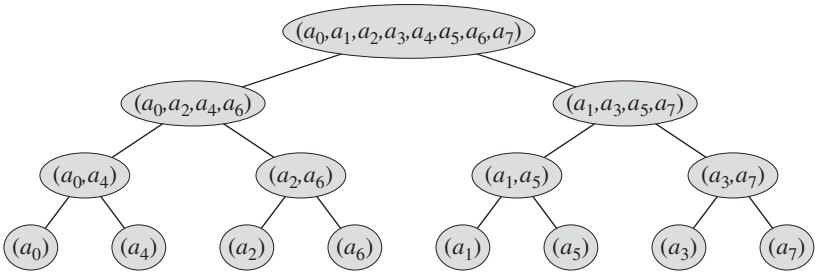


Figura 30.4 L'albero dei vettori di input per le chiamate ricorsive della procedura RECURSIVE-FFT. L'invocazione iniziale è per $n = 8$.

di ω_m, dove $m = 2^s$ (introduciamo la variabile m soltanto per migliorare la leggibilità). Introduciamo un'altra variabile temporanea u che ci consente di eseguire sul posto l'operazione butterfly. Quando sostituiamo la riga 3 della struttura generale con il corpo del ciclo, otteniamo il seguente pseudocodice, che forma la base dell'implementazione parallela che descriveremo più avanti. Il codice prima chiama la procedura ausiliaria BIT-REVERSE-COPY(a, A) per copiare il vettore a nell'array A nell'ordine iniziale in cui ci servono i valori.

```
ITERATIVE-FFT(a)
 1  BIT-REVERSE-COPY(a, A)
 2  n = a.length              // n è una potenza di 2
 3  for s = 1 to lg n
 4      m = 2^s
 5      ω_m = e^{2πi/m}
 6      for k = 0 to n − 1 by m
 7          ω = 1
 8          for j = 0 to m/2 − 1
 9              t = ω A[k + j + m/2]
10              u = A[k + j]
11              A[k + j] = u + t
12              A[k + j + m/2] = u − t
13              ω = ω ω_m
14  return A
```

Come fa la procedura BIT-REVERSE-COPY a disporre gli elementi del vettore di input a nell'array A nell'ordine desiderato? L'ordine in cui appaiono le foglie nella Figura 30.4 è una ***permutazione per inversione di bit***. Cioè, se indichiamo con $\text{rev}(k)$ il numero intero di $\lg n$ bit formato invertendo l'ordine dei bit della rappresentazione binaria di k, vogliamo inserire l'elemento a_k del vettore nella posizione $A[\text{rev}(k)]$ dell'array. Nella Figura 30.4, per esempio, le foglie appaiono nell'ordine $0, 4, 2, 6, 1, 5, 3, 7$; questa sequenza in forma binaria è $000, 100, 010, 110, 001, 101, 011, 111$, e quando invertiamo i bit di ciascun valore, otteniamo la sequenza $000, 001, 010, 011, 100, 101, 110, 111$. Per capire che vogliamo una permutazione per inversione di bit in generale, notate che al livello superiore dell'albero gli indici il cui bit meno significativo è 0 sono posti nel sottoalbero sinistro e gli indici il cui bit meno significativo è 1 sono posti nel sottoalbero destro. Escludendo il bit meno significativo in ogni livello, continuiamo questo processo scendendo l'albero, finché non otterremo l'ordine dato dalla permutazione per inversione di bit nelle foglie.

Acquistato da Michele Michele su Webster il 2022-0[illegible]

Poiché la funzione $\text{rev}(k)$ è facile da calcolare, la procedura BIT-REVERSE-COPY può essere scritta nel modo seguente.

```
BIT-REVERSE-COPY(a, A)
1  n = a.length
2  for k = 0 to n − 1
3      A[rev(k)] = a_k
```

L'implementazione iterativa della FFT viene eseguita nel tempo $\Theta(n \lg n)$. La chiamata a BIT-REVERSE-COPY(a, A) viene sicuramente eseguita nel tempo $O(n \lg n)$, perché si ripete n volte e si può invertire nel tempo $O(\lg n)$ un intero di $\lg n$ bit compreso tra 0 ed $n - 1$. (In pratica, di solito conosciamo in anticipo il valore iniziale di n, quindi potrebbe essere opportuno includere nel codice una tabella che associa k a $\text{rev}(k)$; questo consentirebbe di eseguire BIT-REVERSE-COPY nel tempo $\Theta(n)$ con una piccola costante nascosta. In alternativa, potremmo utilizzare lo schema intelligente del contatore binario a bit invertiti che abbiamo descritto nel Problema 17-1.) Per completare la dimostrazione che la procedura ITERATIVE-FFT viene eseguita nel tempo $\Theta(n \lg n)$, proviamo che $L(n)$ – il numero di volte che il corpo del ciclo interno (righe 8–13) viene eseguito – è $\Theta(n \lg n)$. Il ciclo **for** (righe 6–13) viene eseguito $n/m = n/2^s$ volte per ciascun valore di s, e il ciclo più interno (righe 8–13) viene eseguito $m/2 = 2^{s-1}$ volte; quindi

$$\begin{aligned} L(n) &= \sum_{s=1}^{\lg n} \frac{n}{2^s} \cdot 2^{s-1} \\ &= \sum_{s=1}^{\lg n} \frac{n}{2} \\ &= \Theta(n \lg n) \end{aligned}$$

Un circuito parallelo per la FFT

Possiamo utilizzare molte delle proprietà che ci hanno permesso di implementare un efficiente algoritmo FFT iterativo per produrre un efficiente algoritmo parallelo per calcolare la FFT. Rappresenteremo l'algoritmo FFT parallelo come un circuito. Il circuito parallelo che calcola la FFT con n input è illustrato nella Figura 30.5 per $n = 8$; la prima operazione è una permutazione per inversione di bit degli input, seguita da $\lg n$ stadi, ciascuno dei quali è formato da $n/2$ operazioni butterfly eseguite in parallelo. La profondità del circuito è quindi $\Theta(\lg n)$.

La parte più a sinistra del circuito FFT parallelo esegue la permutazione per inversione di bit e la parte restante simula la procedura iterativa ITERATIVE-FFT. Poiché ogni iterazione del ciclo **for** più esterno esegue $n/2$ operazioni butterfly indipendenti, il circuito le esegue in parallelo. Il valore di s in ogni iterazione all'interno di ITERATIVE-FFT corrisponde a uno stadio di operazioni butterfly, come indica la Figura 30.5. All'interno dello stadio s, per $s = 1, 2, \ldots, \lg n$, ci sono $n/2^s$ gruppi di operazioni butterfly (che corrispondono a ciascun valore di k in ITERATIVE-FFT), con 2^{s-1} operazioni butterfly per gruppo (che corrispondono a ciascun valore di j in ITERATIVE-FFT). Le operazioni butterfly illustrate nella Figura 30.5 corrispondono alle operazioni butterfly del ciclo più interno (righe 9–12 di ITERATIVE-FFT). Notate inoltre che i fattori di rotazione utilizzati nelle operazioni butterfly corrispondono a quelli utilizzati in ITERATIVE-FFT: nello stadio s, utilizziamo $\omega_m^0, \omega_m^1, \ldots, \omega_m^{m/2-1}$, dove $m = 2^s$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

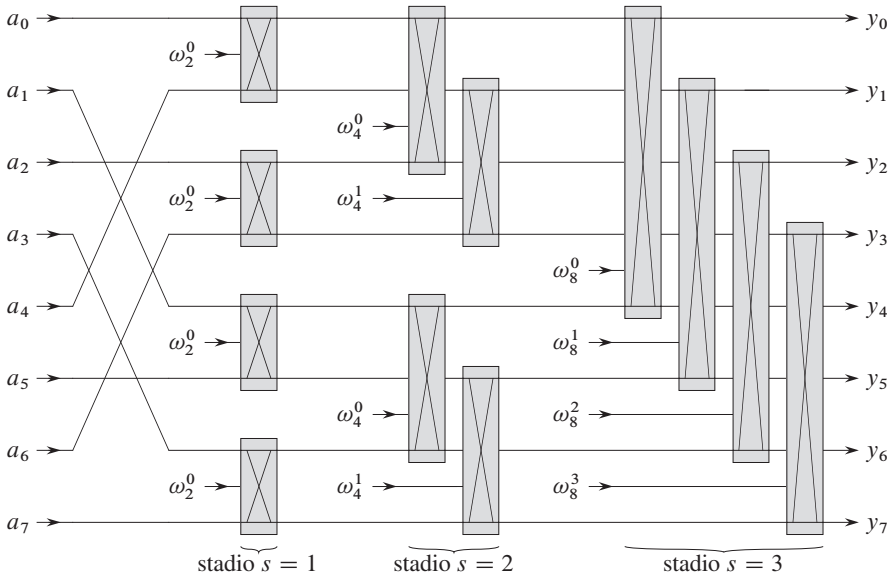


Figura 30.5 Un circuito che calcola la FFT in parallelo, qui illustrato con $n = 8$ input. Ogni operazione butterfly riceve come input i valori su due fili, assieme a un fattore di twiddle, e produce come output i valori in due fili. Gli stadi delle operazioni butterfly sono etichettati in modo da indicare le iterazioni del ciclo più esterno della procedura ITERATIVE-FFT. Soltanto i fili in alto e in basso che attraversano un'operazione butterfly interagiscono con questa operazione; i fili che passano in mezzo a un'operazione butterfly non influiscono su questa operazione né i loro valori vengono modificati da questa operazione. Per esempio, la prima operazione butterfly in alto nello stadio 2 non ha nulla a che fare con il filo 1 (il filo il cui output è etichettato y_1); i suoi input e output sono soltanto nei fili 0 e 2 (etichettati rispettivamente y_0 e y_2). Il circuito ha profondità $\Theta(\lg n)$ ed esegue in tutto $\Theta(n \lg n)$ operazioni butterfly.

Esercizi

30.3-1

Spiegate come la procedura ITERATIVE-FFT calcola la DFT del vettore di input $(0, 2, 3, -1, 4, 5, 7, 9)$.

30.3-2

Spiegate come implementare un algoritmo FFT con la permutazione per inversione di bit che viene eseguita alla fine, anziché all'inizio, del calcolo (*suggerimento:* considerate la DFT inversa).

30.3-3

Quante volte la procedura ITERATIVE-FFT calcola i fattori di rotazione in ogni stadio? Riscrivete ITERATIVE-FFT per calcolare i fattori di rotazione soltanto 2^{s-1} volte nello stadio s.

30.3-4 ⋆

Supponete che i sommatori all'interno delle operazioni butterfly del circuito FFT a volte si guastino producendo sempre un output nullo, indipendentemente dai loro input. Supponete che si guasti un solo sommatore, ma non sapete quale. Spiegate come identificare il sommatore guasto fornendo gli input all'intero circuito FFT e osservando gli output. Quanto è efficiente il vostro metodo?

Problemi

30-1 Moltiplicazione di polinomi con il metodo divide et impera

a. Spiegate come moltiplicare due polinomi lineari $ax + b$ e $cx + d$ utilizzando soltanto tre moltiplicazioni (*suggerimento:* una delle moltiplicazioni è $(a + b) \cdot (c + d)$).

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-2207070 Copyright © 2022, McGraw-Hill Education (Italy)

b. Create due algoritmi divide et impera per moltiplicare due polinomi di grado limite n nel tempo $\Theta(n^{\lg 3})$. Il primo algoritmo dovrebbe dividere i coefficienti del polinomio di input in una metà superiore e in una metà inferiore; il secondo algoritmo dovrebbe dividerli a seconda che i loro indici siano pari o dispari.

c. Dimostrate che due interi di n bit possono essere moltiplicati in $O(n^{\lg 3})$ passi, dove ogni passo opera al più su un numero costante di valori di un bit.

30-2 Matrici di Toeplitz

Una ***matrice di Toeplitz*** è una matrice $A = (a_{ij})$, $n \times n$, tale che $a_{ij} = a_{i-1,j-1}$ per $i = 2, 3, \ldots, n$ e $j = 2, 3, \ldots, n$.

a. La somma di due matrici di Toeplitz è necessariamente una matrice di Toeplitz? E il prodotto?

b. Descrivete come rappresentare una matrice di Toeplitz in modo che due matrici Toeplitz, $n \times n$, possano essere sommate nel tempo $O(n)$.

c. Descrivete un algoritmo con tempo $O(n \lg n)$ per moltiplicare una matrice di Toeplitz, $n \times n$, per un vettore di dimensione n. Utilizzate la rappresentazione che avere creato nel punto (b).

d. Create un algoritmo efficiente per moltiplicare due matrici di Toeplitz $n \times n$. Analizzate il suo tempo di esecuzione.

30-3 Trasformata rapida di Fourier a più dimensioni

È possibile generalizzare la trasformata discreta di Fourier monodimensionale definita dall'equazione (30.8) a d dimensioni. L'input è un array $A = (a_{i_1,i_2,\ldots,i_d})$, le cui d dimensioni sono $n_1, n_2, \ldots, n_d$, dove $n_1 n_2 \cdots n_d = n$. La seguente equazione definisce la trasformata discreta di Fourier a d dimensioni:

$$y_{k_1,k_2,\ldots,k_d} = \sum_{j_1=0}^{n_1-1} \sum_{j_2=0}^{n_2-1} \cdots \sum_{j_d=0}^{n_d-1} a_{j_1,j_2,\ldots,j_d} \omega_{n_1}^{j_1 k_1} \omega_{n_2}^{j_2 k_2} \cdots \omega_{n_d}^{j_d k_d}$$

per $0 \le k_1 < n_1, 0 \le k_2 < n_2, \ldots, 0 \le k_d < n_d$.

a. Dimostrate che è possibile calcolare una DFT a d dimensioni calcolando in sequenza le DFT monodimensionali in ciascuna dimensione. Ovvero, prima si calcolano n/n_1 DFT monodimensionali distinte lungo la dimensione 1. Poi, utilizzando come input il risultato delle DFT lungo la dimensione 1, si calcolano le n/n_2 DFT monodimensionali distinte lungo la dimensione 2. Utilizzando questo risultato come input, si calcolano n/n_3 DFT monodimensionali distinte lungo la dimensione 3, e così via, fino alla dimensione d.

b. Dimostrate che l'ordine delle dimensioni non è importante, quindi è possibile calcolare una DFT a d dimensioni calcolando le DFT monodimensionali in qualsiasi ordine delle d dimensioni.

c. Dimostrate che, se ciascuna delle DFT monodimensionali viene calcolata tramite la FFT, il tempo totale per calcolare una DFT a d dimensioni è $O(n \lg n)$, che non dipende da d.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

30-4 Calcolo di tutte le derivate di un polinomio in un punto

Dato un polinomio $A(x)$ di grado limite n, la sua derivata t-esima è definita da

$$A^{(t)}(x) = \begin{cases} A(x) & \text{se } t = 0 \\ \frac{d}{dx} A^{(t-1)}(x) & \text{se } 1 \le t \le n-1 \\ 0 & \text{se } t \ge n \end{cases}$$

Conoscendo la rappresentazione per coefficienti $(a_0, a_1, \ldots, a_{n-1})$ di $A(x)$ e un punto x_0, determinate $A^{(t)}(x_0)$ per $t = 0, 1, \ldots, n-1$.

a. Dati i coefficienti $b_0, b_1, \ldots, b_{n-1}$ tali che

$$A(x) = \sum_{j=0}^{n-1} b_j (x - x_0)^j$$

spiegate come calcolare $A^{(t)}(x_0)$, per $t = 0, 1, \ldots, n-1$, nel tempo $O(n)$.

b. Spiegate come trovare $b_0, b_1, \ldots, b_{n-1}$ nel tempo $O(n \lg n)$, dato $A(x_0 + \omega_n^k)$ per $k = 0, 1, \ldots, n-1$.

c. Dimostrate che

$$A(x_0 + \omega_n^k) = \sum_{r=0}^{n-1} \left(\frac{\omega_n^{kr}}{r!} \sum_{j=0}^{n-1} f(j) g(r-j) \right)$$

dove $f(j) = a_j \cdot j!$ e

$$g(l) = \begin{cases} x_0^{-l}/(-l)! & \text{se } -(n-1) \le l \le 0 \\ 0 & \text{se } 1 \le l \le n-1 \end{cases}$$

d. Spiegate come calcolare $A(x_0 + \omega_n^k)$ per $k = 0, 1, \ldots, n-1$ nel tempo $O(n \lg n)$. Concludete che tutte le derivate non banali di $A(x)$ possono essere calcolate in x_0 nel tempo $O(n \lg n)$.

30-5 Valutazione di un polinomio in più punti

Abbiamo osservato che il problema di valutare un polinomio di grado limite n in un punto singolo può essere risolto nel tempo $O(n)$ utilizzando la regola di Horner. Abbiamo anche scoperto che un tale polinomio può essere valutato in tutte le n radici complesse dell'unità nel tempo $O(n \lg n)$ utilizzando la FFT. Vediamo adesso come valutare un polinomio di grado limite n in n punti arbitrari nel tempo $O(n \lg^2 n)$. A tal fine, utilizzeremo il fatto che possiamo calcolare nel tempo $O(n \lg n)$ il resto della divisione di due polinomi, un risultato che riteniamo vero senza dimostrarlo. Per esempio, il resto della divisione di $3x^3 + x^2 - 3x + 1$ per $x^2 + x + 2$ è

$$(3x^3 + x^2 - 3x + 1) \bmod (x^2 + x + 2) = -7x + 5$$

Conoscendo la rappresentazione per coefficienti di un polinomio $A(x) = \sum_{k=0}^{n-1} a_k x^k$ ed n punti $x_0, x_1, \ldots, x_{n-1}$, vogliamo calcolare gli n valori $A(x_0), A(x_1), \ldots, A(x_{n-1})$. Per $0 \le i \le j \le n-1$, definiamo i polinomi $P_{ij}(x) = \prod_{k=i}^{j} (x - x_k)$ e $Q_{ij}(x) = A(x) \bmod P_{ij}(x)$. Notate che $Q_{ij}(x)$ ha grado al più $j - i$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

a. Dimostrate che $A(x) \bmod (x - z) = A(z)$ per ogni punto z.

b. Dimostrate che $Q_{kk}(x) = A(x_k)$ e $Q_{0,n-1}(x) = A(x)$.

c. Dimostrate che, per $i \leq k \leq j$, si ha $Q_{ik}(x) = Q_{ij}(x) \bmod P_{ik}(x)$ e $Q_{kj}(x) = Q_{ij}(x) \bmod P_{kj}(x)$.

d. Create un algoritmo con tempo $O(n \lg^2 n)$ per valutare $A(x_0)$, $A(x_1)$, ..., $A(x_{n-1})$.

30-6 FFT tramite l'aritmetica modulare

Secondo la definizione, la trasformata rapida di Fourier richiede l'uso dei numeri complessi, che possono determinare la perdita di precisione a causa degli errori di arrotondamento. Per alcuni problemi, si sa che la soluzione contiene soltanto numeri interi ed è preferibile utilizzare una variante della FFT basata sull'aritmetica modulare per garantire che la soluzione sia calcolata con esattezza. Un esempio di tale problema è la moltiplicazione di due polinomi con coefficienti interi. L'Esercizio 30.2-6 descrive un approccio che usa un modulo di lunghezza $\Omega(n)$ bit per gestire una DFT in n punti. Questo problema descrive un altro approccio che usa un modulo con la lunghezza più ragionevole $O(\lg n)$; è richiesta la conoscenza degli argomenti trattati nel Capitolo 31. Sia n una potenza di 2.

a. Supponete di cercare il più piccolo valore di k tale che $p = kn + 1$ sia un numero primo. Trovate una semplice ragione euristica per cui è logico aspettarsi che k sia approssimativamente pari a $\lg n$. (Il valore di k potrebbe essere molto più grande o più piccolo, ma è ragionevole prevedere di esaminare in media $O(\lg n)$ valori candidati di k.) Qual è la lunghezza attesa di p in funzione della lunghezza di n?

Sia g un generatore di $\mathbb{Z}_p^*$ e sia $w = g^k \bmod p$.

b. Dimostrate che la DFT e la DFT inversa sono operazioni modulo p inverse ben definite, dove w è utilizzata come radice n-esima principale dell'unità.

c. Dimostrate che la FFT e la sua inversa possono essere modificate per lavorare modulo p con tempo $O(n \lg n)$ assumendo che le operazioni sulle parole di $O(\lg n)$ bit richiedano un tempo unitario. Supponete che l'algoritmo riceva p e w.

d. Calcolate la DFT modulo $p = 17$ del vettore $(0, 5, 3, 7, 7, 2, 1, 6)$. Notate che $g = 3$ è un generatore di $\mathbb{Z}_{17}^*$.

Note

Il libro di Van Loan [344] tratta in modo completo e metodico la trasformata rapida di Fourier. Press, Teukolsky, Vetterling e Flannery [284, 285] hanno descritto molto bene la FFT e le sue applicazioni. Per una eccellente introduzione all'elaborazione dei segnali – una tipica area applicativa della FFT – consultate i testi di Oppenheim e Schafer [267] e Oppenheim e Willsky [268]. Il libro di Oppenheim e Schafer descrive anche come gestire i casi in cui n non sia una potenza intera di 2.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

L'analisi di Fourier non è limitata ai dati monodimensionali. È ampiamente utilizzata nell'elaborazione delle immagini per analizzare i dati in 2 o più dimensioni. I libri di Gonzalez e Woods [147] e Pratt [282] descrivono le trasformate di Fourier a più dimensioni e il loro impiego nell'elaborazione delle immagini; i libri di Tolimieri, An e Lu [339] e Van Loan [344] trattano gli aspetti matematici delle trasformate di Fourier a più dimensioni.

continua

A Cooley e Tukey [77] è attribuita l'invenzione della FFT negli anni sessanta. In effetti, la FFT è stata scoperta molto tempo prima, ma la sua importanza non è stata pienamente apprezzata prima dell'avvento dei moderni calcolatori digitali. Sebbene Press, Flannery, Teukolsky e Vetterling attribuiscano le origini del metodo a Runge e König nel 1924, un articolo di Heideman, Johnson e Burrus [164] traccia la storia della FFT risalendo fino a C. F. Gauss nel 1805.

Frigo e Johnson [118] hanno sviluppato un'implementazione rapida e flessibile della FFT, detta FFTW ("fastest Fourier transform in the West"). FFTW è stata progettata per quei casi che richiedono molti calcoli della DFT per problemi della stessa dimensione. Prima di calcolare le DFT, la FFTW esegue una "pianificazione" che determina, tramite una serie di tentativi, come ripartire efficientemente il calcolo della FFT nella macchina host per una particolare dimensione del problema. La FFTW si adatta per un uso efficiente della cache hardware e, una volta che i sottoproblemi sono abbastanza piccoli, li risolve con un codice lineare ottimizzato. In aggiunta, la FFTW ha l'insolito vantaggio di impiegare un tempo $\Theta(n \lg n)$ per qualsiasi dimensione del problema n, anche quando n è un numero primo grande.

Sebbene la trasformata di Fourier standard supponga che l'input rappresenti punti che sono uniformemente distribuiti nel dominio del tempo, altre tecniche possono approssimare la FFT per dati non uniformemente distribuiti. L'articolo di Ware [349] fornisce una panoramica di queste tecniche.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Algoritmi di teoria dei numeri 31

La teoria dei numeri una volta era considerata un argomento interessante, ma essenzialmente inutile, della matematica pura. Oggi gli algoritmi basati sulla teoria dei numeri sono ampiamente utilizzati, in parte grazie all'invenzione degli schemi di crittografia basati su numeri primi grandi. La realizzabilità di questi schemi si fonda sulla nostra capacità di trovare facilmente numeri primi grandi, mentre la sicurezza di questi schemi si fonda sul fatto che non conosciamo un modo efficiente per fattorizzare il prodotto di numeri primi grandi (o risolvere problemi correlati quale il calcolo del logaritmo discreto). Questo capitolo presenta alcuni concetti della teoria dei numeri e gli algoritmi associati che sono alla base di tali applicazioni.

Il Paragrafo 31.1 introduce i concetti elementari della teoria dei numeri, come la divisibilità, la congruenza modulare e l'unicità della fattorizzazione. Il Paragrafo 31.2 studia uno dei più antichi algoritmi del mondo: l'algoritmo di Euclide per calcolare il massimo comun divisore di due numeri interi. Il Paragrafo 31.3 illustra i concetti dell'aritmetica modulare. Il Paragrafo 31.4 studia l'insieme dei multipli di un dato numero a, modulo n, e spiega come trovare tutte le soluzioni dell'equazione $ax \equiv b \pmod{n}$ mediante l'algoritmo di Euclide. Il teorema cinese del resto è descritto nel Paragrafo 31.5. Il Paragrafo 31.6 esamina le potenze di un dato numero a, modulo n, e presenta un algoritmo che applica la tecnica dell'elevazione al quadrato ripetuta per calcolare in modo efficiente $a^b \bmod n$, dati a, b ed n. Questa operazione è fondamentale nei test di primalità e in molti sistemi moderni di crittografia. Il Paragrafo 31.7 descrive il sistema di crittografia a chiave pubblica RSA. Il Paragrafo 31.8 esamina un test di primalità randomizzato che può essere utilizzato per trovare in modo efficiente numeri primi grandi, un compito essenziale nella creazione di chiavi per il sistema di crittografia RSA. Infine, il Paragrafo 31.9 esamina una semplice, ma efficiente, euristica per fattorizzare i numeri interi piccoli. È curioso il fatto che la fattorizzazione sia un problema che vorremmo rimanesse intrattabile, dal momento che la sicurezza del sistema RSA dipende dalla difficoltà di fattorizzare i numeri interi grandi.

Dimensione dell'input e costo dei calcoli aritmetici

Poiché opereremo con numeri interi grandi, dobbiamo rivedere i concetti di dimensione dell'input e di costo delle operazioni aritmetiche elementari.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

In questo capitolo, un "grande input" tipicamente significa un input che contiene "numeri interi grandi", anziché un input che contiene "molti numeri interi" (come nell'ordinamento). Quindi, misureremo la dimensione di un input in funzione del *numero dei bit* richiesti per rappresentare l'input, non del numero di interi presenti nell'input. Un algoritmo con input interi $a_1, a_2, \ldots, a_k$ è un ***algoritmo con tempo polinomiale*** se viene eseguito in un tempo polinomiale in $\lg a_1$, $\lg a_2, \ldots, \lg a_k$, cioè, polinomiale nelle lunghezze dei suoi input in codice binario.

In questo libro, generalmente, abbiamo ipotizzato che le operazioni aritmetiche elementari (moltiplicazione, divisione o calcolo del resto) fossero operazioni primitive con tempo di esecuzione unitario. Contando il numero di tali operazioni aritmetiche che vengono eseguite da un algoritmo, abbiamo la base per fare una stima ragionevole del tempo di esecuzione effettivo dell'algoritmo in un calcolatore. Tuttavia, le operazioni elementari possono richiedere molto tempo, se i loro input sono grandi. Pertanto, è preferibile misurare quante ***operazioni sui bit*** richiede un algoritmo di teoria dei numeri. In questo modello, una moltiplicazione di due interi di β bit con il metodo ordinario usa $\Theta(\beta^2)$ operazioni sui bit. Analogamente, la divisione di un intero di β bit per un intero più corto o il calcolo del resto di tale divisione sono operazioni che possono essere eseguite nel tempo $\Theta(\beta^2)$ da semplici algoritmi (vedere l'Esercizio 31.1-12). Esistono metodi più veloci. Per esempio, un semplice metodo divide et impera per moltiplicare due interi di β bit ha un tempo di esecuzione $\Theta(\beta^{\lg 3})$ e il metodo più veloce che si conosce ha un tempo di esecuzione $\Theta(\beta \lg \beta \lg\lg \beta)$. Ai fini pratici, tuttavia, l'algoritmo con tempo $\Theta(\beta^2)$ spesso è il migliore, quindi utilizzeremo questo limite come base delle nostre analisi.

In questo capitolo, gli algoritmi generalmente sono analizzati in funzione del numero di operazioni aritmetiche e del numero di operazioni sui bit che essi richiedono.

31.1 Concetti elementari di teoria dei numeri

Questo paragrafo descrive brevemente i concetti fondamentali della teoria dei numeri che riguardano l'insieme $\mathbb{Z} = \{\ldots, -2, -1, 0, 1, 2, \ldots\}$ dei numeri interi e l'insieme $\mathbb{N} = \{0, 1, 2, \ldots\}$ dei numeri naturali.

Divisibilità e divisori

Il concetto di numero intero divisibile per un altro numero intero è fondamentale nella teoria dei numeri. La notazione $d \mid a$ (si legge "d ***divide*** a") significa che $a = kd$ per qualche intero k. Ogni numero intero divide 0. Se $a > 0$ e $d \mid a$, allora $|d| \leq |a|$. Se $d \mid a$, allora si dice anche che a è un ***multiplo*** di d. Se d non divide a, scriviamo $d \nmid a$.

Se $d \mid a$ e $d \geq 0$, diciamo che d è un ***divisore*** di a. Notate che $d \mid a$ se e soltanto se $-d \mid a$, quindi non perdiamo in generalità definendo i divisori come numeri non negativi, restando sottinteso che anche il negativo di un divisore qualsiasi di a divide a. Un divisore di un intero a diverso da zero è almeno pari a 1, ma non è maggiore di $|a|$. Per esempio, i divisori di 24 sono 1, 2, 3, 4, 6, 8, 12 e 24.

Ogni intero a è divisibile per i ***divisori banali*** 1 e a. I divisori non banali di a sono anche detti ***fattori*** di a. Per esempio, i fattori di 20 sono 2, 4, 5 e 10.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Numeri primi e numeri composti

Un intero $a > 1$ i cui unici divisori sono i divisori banali 1 e a è detto ***numero primo*** o, semplicemente, ***primo***. I numeri primi hanno molte proprietà speciali e svolgono un ruolo molto importante nella teoria dei numeri. I primi venti numeri primi sono:

2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71

L'Esercizio 31.1-2 chiede di dimostrare che esistono infiniti numeri primi. Un intero $a > 1$ che non è primo è detto ***numero composto*** o, semplicemente, ***composto***. Per esempio, 39 è composto perché $3 \mid 39$. L'intero 1 è detto ***unità*** e non è né primo né composto. Analogamente, l'intero 0 e tutti i numeri interi negativi non sono né primi né composti.

Teorema della divisione, resto e congruenza modulare

Dato un intero n, gli interi possono essere suddivisi in due parti: multipli di n e non multipli di n. Gran parte della teoria dei numeri si basa sull'affinamento di questa partizione classificando i non multipli di n a seconda del resto che si ottiene quando vengono divisi per n. Il seguente teorema è la base per effettuare questo affinamento. La dimostrazione del teorema non sarà data (potete consultare, per esempio, Niven e Zuckerman [266]).

Teorema 31.1 (Teorema della divisione)
Per a intero qualsiasi ed n intero positivo, esistono due interi unici q ed r tali che $0 \leq r < n$ e $a = qn + r$. ■

Il valore $q = \lfloor a/n \rfloor$ è il ***quoziente*** della divisione. Il valore $r = a \bmod n$ è il ***resto*** (o ***residuo***) della divisione. Si ha che $n \mid a$ se e soltanto se $a \bmod n = 0$.

Gli interi possono essere suddivisi in n classi di congruenza a seconda dei loro resti modulo n. La ***classe di congruenza modulo n*** che contiene un intero a è

$$[a]_n = \{a + kn : k \in \mathbb{Z}\}$$

Per esempio, $[3]_7 = \{\ldots, -11, -4, 3, 10, 17, \ldots\}$; altri modi di indicare questo insieme sono $[-4]_7$ e $[10]_7$. Utilizzando la notazione definita a pagina 46, possiamo dire che scrivere $a \in [b]_n$ equivale a scrivere $a \equiv b \pmod{n}$. L'insieme di tutte queste classi di congruenza è

$$\mathbb{Z}_n = \{[a]_n : 0 \leq a \leq n - 1\} \tag{31.1}$$

Spesso vediamo la definizione

$$\mathbb{Z}_n = \{0, 1, \ldots, n - 1\} \tag{31.2}$$

che dovrebbe essere considerata equivalente all'equazione (31.1), restando sottinteso che 0 rappresenta $[0]_n$, 1 rappresenta $[1]_n$ e così via; ogni classe è rappresentata dal suo elemento non negativo minimo. Bisogna comunque ricordarsi delle classi di congruenza sottostanti. Per esempio, un riferimento a -1 come a un elemento di $\mathbb{Z}_n$ è un riferimento alla classe $[n-1]_n$, perché $-1 \equiv n-1 \pmod{n}$.

Divisore comune e massimo comun divisore

Se d è un divisore di a e d è anche un divisore di b, allora d è un ***divisore comune*** di a e b. Per esempio, i divisori di 30 sono 1, 2, 3, 5, 6, 10, 15 e 30, e così i divisori comuni di 24 e 30 sono 1, 2, 3 e 6. Notate che 1 è un divisore comune di due interi qualsiasi.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Una proprietà importante dei divisori comuni è che

$$d \mid a \quad \text{e} \quad d \mid b \quad \text{implica} \quad d \mid (a + b) \quad \text{e} \quad d \mid (a - b) \tag{31.3}$$

Più in generale, si ha che

$$d \mid a \quad \text{e} \quad d \mid b \quad \text{implica} \quad d \mid (ax + by) \tag{31.4}$$

per x e y interi qualsiasi. Inoltre, se $a \mid b$, allora $|a| \leq |b|$ oppure $b = 0$; questo implica che

$$a \mid b \quad \text{e} \quad b \mid a \quad \text{implica} \quad a = \pm b \tag{31.5}$$

Il ***massimo comun divisore*** di due interi a e b, non entrambi nulli, è il più grande dei divisori comuni di a e b; sarà indicato da $\text{MCD}(a, b)$. Per esempio, $\text{MCD}(24, 30) = 6$, $\text{MCD}(5, 7) = 1$ e $\text{MCD}(0, 9) = 9$. Se a e b sono entrambi non nulli, allora $\text{MCD}(a, b)$ è un intero tra 1 e $\min(|a|, |b|)$. Definiamo $\text{MCD}(0, 0) = 0$; questa definizione è necessaria per rendere universalmente valide le proprietà standard della funzione MCD (come la successiva equazione (31.9)).

Elenchiamo le proprietà elementari della funzione MCD:

$$\text{MCD}(a, b) = \text{MCD}(b, a) \tag{31.6}$$
$$\text{MCD}(a, b) = \text{MCD}(-a, b) \tag{31.7}$$
$$\text{MCD}(a, b) = \text{MCD}(|a|, |b|) \tag{31.8}$$
$$\text{MCD}(a, 0) = |a| \tag{31.9}$$
$$\text{MCD}(a, ka) = |a| \qquad \text{per ogni } k \in \mathbb{Z} \tag{31.10}$$

Il seguente teorema offre un'utile caratterizzazione alternativa di $\text{MCD}(a, b)$.

Teorema 31.2
Se a e b sono interi qualsiasi, non entrambi nulli, allora $\text{MCD}(a, b)$ è il più piccolo elemento positivo dell'insieme $\{ax + by : x, y \in \mathbb{Z}\}$ delle combinazioni lineari di a e b.

Dimostrazione Sia s la più piccola combinazione lineare positiva di a e b e sia $s = ax + by$ per qualche $x, y \in \mathbb{Z}$. Sia $q = \lfloor a/s \rfloor$. L'equazione (3.8) allora implica

$$\begin{aligned} a \bmod s &= a - qs \\ &= a - q(ax + by) \\ &= a\,(1 - qx) + b\,(-qy) \end{aligned}$$

e quindi anche $a \bmod s$ è una combinazione lineare di a e b. Tuttavia, poiché $0 \leq a \bmod s < s$, si ha che $a \bmod s = 0$, in quanto s è la più piccola combinazione lineare positiva. Pertanto, $s \mid a$ e, per un ragionamento analogo, $s \mid b$. Quindi, s è un divisore comune di a e b, e così $\text{MCD}(a, b) \geq s$. L'equazione (31.4) implica che $\text{MCD}(a, b) \mid s$, in quanto $\text{MCD}(a, b)$ divide sia a che b ed s è una combinazione lineare di a e b. Ma $\text{MCD}(a, b) \mid s$ ed $s > 0$ implicano che $\text{MCD}(a, b) \leq s$. Combinando $\text{MCD}(a, b) \geq s$ e $\text{MCD}(a, b) \leq s$ si ottiene $\text{MCD}(a, b) = s$; concludiamo che s è il massimo comun divisore di a e b. ■

Corollario 31.3
Per a e b interi qualsiasi, se $d \mid a$ e $d \mid b$, allora $d \mid \text{MCD}(a, b)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Quinte Licenza: 1995030810-2287070 Copyright © 2021, McGraw-Hill Education (Italy)

Dimostrazione Questo corollario segue dall'equazione (31.4), perché $\text{MCD}(a, b)$ è una combinazione lineare di a e b per il Teorema 31.2. ■

Corollario 31.4
Per a e b interi qualsiasi ed n intero non negativo, si ha

$$\text{MCD}(an, bn) = n\,\text{MCD}(a, b)$$

Dimostrazione Se $n = 0$, il corollario è banale. Se $n > 0$, allora $\text{MCD}(an, bn)$ è il più piccolo elemento positivo dell'insieme $\{anx + bny\}$, che è n volte il più piccolo elemento positivo dell'insieme $\{ax + by\}$. ■

Corollario 31.5
Per n, a e b interi qualsiasi, se $n \mid ab$ e $\text{MCD}(a, n) = 1$, allora $n \mid b$.

Dimostrazione Lasciamo al lettore il compito di dimostrare questo corollario (Esercizio 31.1-5). ■

Numeri primi tra loro

Due interi a e b sono detti ***primi tra loro*** o ***relativamente primi*** se il loro unico divisore comune è 1, cioè, se $\text{MCD}(a, b) = 1$. Per esempio, 8 e 15 sono relativamente primi, perché i divisori di 8 sono 1, 2, 4 e 8, mentre i divisori di 15 sono 1, 3, 5 e 15. Il seguente teorema stabilisce che, se due interi sono entrambi relativamente primi con lo stesso intero p, allora il loro prodotto è pure relativamente primo con p.

Teorema 31.6
Per a, b e p interi qualsiasi, se $\text{MCD}(a, p) = 1$ e $\text{MCD}(b, p) = 1$, allora $\text{MCD}(ab, p) = 1$.

Dimostrazione Dal Teorema 31.2 si deduce che esistono degli interi x, y, x' e y' tali che

$$\begin{aligned} ax + py &= 1 \\ bx' + py' &= 1 \end{aligned}$$

Moltiplicando queste equazioni e riordinando i termini, si ha

$$ab(xx') + p(ybx' + y'ax + pyy') = 1$$

Di conseguenza, poiché 1 è una combinazione lineare positiva di ab e p, l'applicazione del Teorema 31.2 completa la dimostrazione. ■

Gli interi $n_1, n_2, \ldots, n_k$ sono detti ***relativamente primi a coppie*** se, per $i \neq j$, si ha $\text{MCD}(n_i, n_j) = 1$.

Unicità della scomposizione in fattori

Il seguente teorema asserisce un fatto elementare, ma importante, sulla divisibilità dei numeri interi per i numeri primi.

Teorema 31.7
Per p numero primo qualsiasi e a e b numeri interi qualsiasi, se $p \mid ab$, allora $p \mid a$ e\o $p \mid b$.

Dimostrazione Supponiamo per assurdo che $p \mid ab$, ma che $p \nmid a$ e $p \nmid b$. Quindi, $\text{MCD}(a, p) = 1$ e $\text{MCD}(b, p) = 1$, perché gli unici divisori di p sono 1 e p, e per ipotesi p non divide né a né b. Il Teorema 31.6 implica che $\text{MCD}(ab, p) = 1$, contraddicendo l'ipotesi che $p \mid ab$, perché $p \mid ab$ implica $\text{MCD}(ab, p) = p$. Questa contraddizione completa la dimostrazione. ■

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Una conseguenza del Teorema 31.7 è che un intero ha una scomposizione unica in fattori primi.

Teorema 31.8 (Unicità della scomposizione in fattori primi)
Un numero intero composto a può essere scritto in un unico modo come prodotto della seguente forma

$$a = p_1^{e_1} p_2^{e_2} \cdots p_r^{e_r}$$

Dove i termini p_i sono numeri primi, $p_1 < p_2 < \cdots < p_r$, e i termini e_i sono interi positivi.

Dimostrazione Lasciamo al lettore il compito di dimostrare questo teorema (Esercizio 31.1-11). ■

Per esempio, il numero 6000 può essere scomposto in fattori primi soltanto così: $2^4 \cdot 3 \cdot 5^3$.

Esercizi

31.1-1
Dimostrate che se $a > b > 0$ e $c = a + b$, allora $c \bmod a = b$.

31.1-2
Dimostrate che esistono infiniti numeri primi (*suggerimento:* dimostrate che nessuno dei numeri primi $p_1, p_2, \ldots, p_k$ divide $(p_1 p_2 \cdots p_k) + 1$).

31.1-3
Dimostrate che, se $a \mid b$ e $b \mid c$, allora $a \mid c$.

31.1-4
Dimostrate che, se p è primo e $0 < k < p$, allora $\text{MCD}(k, p) = 1$.

31.1-5
Dimostrate il Corollario 31.5.

31.1-6
Dimostrate che, se p è primo e $0 < k < p$, allora $p \mid \binom{p}{k}$. Concludete che per a e b interi qualsiasi e p primo qualsiasi, si ha

$$(a + b)^p \equiv a^p + b^p \pmod{p}$$

31.1-7
Dimostrate che, se a e b sono interi positivi tali che $a \mid b$, allora

$$(x \bmod b) \bmod a = x \bmod a$$

per qualsiasi x. Dimostrate, sotto le stesse ipotesi, che

$$x \equiv y \pmod{b} \text{ implica } x \equiv y \pmod{a}$$

per x e y interi qualsiasi.

31.1-8
Acquistato da Michele Michele su Webster il 2022-07-07 23:13 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)
Per qualsiasi intero $k > 0$, un intero n è una ***k-esima potenza*** se esiste un intero a tale che $a^k = n$. Un numero intero $n > 1$ è una ***potenza non banale*** se è una k-esima potenza per qualche intero $k > 1$. Spiegate come determinare se un intero n di β bit è una potenza non banale in un tempo polinomiale in β.

31.1-9
Dimostrate le equazioni (31.6)–(31.10).

31.1-10
Dimostrate che l'operatore MCD è associativo; ovvero dimostrate che, per a, b e c interi qualsiasi, si ha

$$\text{MCD}(a, \text{MCD}(b, c)) = \text{MCD}(\text{MCD}(a, b), c)$$

31.1-11 ⋆
Dimostrate il Teorema 31.8.

31.1-12
Create degli algoritmi efficienti per dividere un intero di β bit per un intero più corto e per calcolare il resto di tale divisione. I vostri algoritmi dovrebbero essere eseguiti nel tempo $O(\beta^2)$.

31.1-13
Create un algoritmo efficiente per convertire un intero di β bit (binario) nella rappresentazione decimale. Dimostrate che, se la moltiplicazione o la divisione di interi la cui lunghezza è al più β richiede un tempo $M(\beta)$, allora la conversione da binario a decimale può essere eseguita nel tempo $\Theta(M(\beta) \lg \beta)$ (*suggerimento:* utilizzate un metodo divide et impera, ottenendo le metà superiore e inferiore del risultato con ricorsioni separate).

31.2 Massimo comun divisore

In questo paragrafo descriviamo l'algoritmo di Euclide per calcolare in modo efficiente il massimo comun divisore di due interi. L'analisi del tempo di esecuzione porta a una sorprendente connessione con i numeri di Fibonacci, che rappresentano l'input nel caso peggiore per l'algoritmo di Euclide. In questo paragrafo limiteremo l'analisi ai numeri interi non negativi. Questa limitazione è giustificata dall'equazione (31.8), che stabilisce che $\text{MCD}(a, b) = \text{MCD}(|a|, |b|)$.

In linea di principio, è possibile calcolare $\text{MCD}(a, b)$ per i numeri interi positivi a e b dalle scomposizioni in fattori primi di a e b. In effetti, se

$$a = p_1^{e_1} p_2^{e_2} \cdots p_r^{e_r} \tag{31.11}$$

$$b = p_1^{f_1} p_2^{f_2} \cdots p_r^{f_r} \tag{31.12}$$

utilizzando esponenti zero per ottenere lo stesso insieme di numeri primi $p_1\, p_2 \ldots\, p_r$ per a e b, allora, come chiede di dimostrare l'Esercizio 31.2-1, si ha

$$\text{MCD}(a, b) = p_1^{\min(e_1, f_1)} p_2^{\min(e_2, f_2)} \cdots p_r^{\min(e_r, f_r)} \tag{31.13}$$

Tuttavia, come vedremo nel Paragrafo 31.9, attualmente i migliori algoritmi di fattorizzazione non vengono eseguiti in tempo polinomiale. Quindi, appare improbabile che questo approccio per calcolare il massimo comun divisore possa produrre un algoritmo efficiente.

Acquistato da Michele Michele su Webster | 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

L'algoritmo di Euclide per calcolare il massimo comun divisore si basa sul seguente teorema.

Teorema 31.9 (Teorema della ricorsione del MCD)
Per a intero non negativo e b intero positivo, si ha

$$\text{MCD}(a, b) = \text{MCD}(b, a \bmod b)$$

Dimostrazione Dimostreremo che $\text{MCD}(a, b)$ e $\text{MCD}(b, a \bmod b)$ si dividono a vicenda, quindi per l'equazione (31.5) devono essere uguali (perché entrambi non sono negativi). Dimostriamo prima che $\text{MCD}(a, b) \mid \text{MCD}(b, a \bmod b)$. Se poniamo $d = \text{MCD}(a, b)$, allora $d \mid a$ e $d \mid b$. Per l'equazione (3.8), $(a \bmod b) = a - qb$, dove $q = \lfloor a/b \rfloor$. Pertanto, poiché $(a \bmod b)$ è una combinazione lineare di a e b, l'equazione (31.4) implica che $d \mid (a \bmod b)$. Allora, poiché $d \mid b$ e $d \mid (a \bmod b)$, il Corollario 31.3 implica che $d \mid \text{MCD}(b, a \bmod b)$ ovvero

$$\text{MCD}(a, b) \mid \text{MCD}(b, a \bmod b) \tag{31.14}$$

In modo simile, si può dimostrare che $\text{MCD}(b, a \bmod b) \mid \text{MCD}(a, b)$. Se poniamo $d = \text{MCD}(b, a \bmod b)$, allora $d \mid b$ e $d \mid (a \bmod b)$. Poiché $a = qb + (a \bmod b)$, dove $q = \lfloor a/b \rfloor$, allora a è una combinazione lineare di b e $(a \bmod b)$. Per l'equazione (31.4), concludiamo che $d \mid a$. Poiché $d \mid b$ e $d \mid a$, si ha che $d \mid \text{MCD}(a, b)$ per il Corollario 31.3 ovvero

$$\text{MCD}(b, a \bmod b) \mid \text{MCD}(a, b) \tag{31.15}$$

Utilizzando l'equazione (31.5) per combinare le equazioni (31.14) e (31.15), si completa la dimostrazione. ■

Algoritmo di Euclide

Il seguente algoritmo per il MCD è descritto negli *Elementi* di Euclide (300 a.C. circa), ma potrebbe avere origini ancora più antiche. L'algoritmo di Euclide è scritto come un programma ricorsivo, basato direttamente sul Teorema 31.9. Gli input a e b sono interi arbitrari non negativi.

```
EUCLID(a, b)
1  if b = 0
2       return a
3  else return EUCLID(b, a mod b)
```

Come esempio di esecuzione dell'algoritmo EUCLID, consideriamo il calcolo di $\text{MCD}(30, 21)$:

$$\begin{aligned} \text{EUCLID}(30, 21) &= \text{EUCLID}(21, 9) \\ &= \text{EUCLID}(9, 3) \\ &= \text{EUCLID}(3, 0) \\ &= 3 \end{aligned}$$

In questo calcolo, ci sono tre chiamate ricorsive di EUCLID.

La correttezza dell'algoritmo EUCLID segue dal Teorema 31.9 e dal fatto che, se l'algoritmo restituisce a nella riga 2, allora $b = 0$, quindi l'equazione (31.9) implica che $\text{MCD}(a, b) = \text{MCD}(a, 0) = a$. L'algoritmo non può effettuare la ricorsione all'infinito, perché il secondo argomento diminuisce in senso stretto in ogni chiamata ricorsiva ed è sempre non negativo. Dunque, EUCLID termina sempre con la soluzione corretta.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995U3016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Tempo di esecuzione dell'algoritmo di Euclide

Analizziamo il tempo di esecuzione nel caso peggiore di EUCLID come una funzione della dimensione di a e b. Supponiamo, senza perdere in generalità, che $a > b \geq 0$. Questa ipotesi può essere giustificata dall'osservazione che se $b > a \geq 0$, allora $\text{EUCLID}(a, b)$ effettua immediatamente la chiamata ricorsiva $\text{EUCLID}(b, a)$. Ovvero, se il primo argomento è minore del secondo, EUCLID

spende una chiamata ricorsiva per scambiare i suoi argomenti e poi procede. Analogamente, se $b = a > 0$, la procedura termina dopo una chiamata ricorsiva, perché $a \bmod b = 0$.

Il tempo di esecuzione totale di EUCLID è proporzionale al numero di chiamate ricorsive che fa. La nostra analisi usa i numeri di Fibonacci F_k definiti dalla ricorrenza (3.22).

Lemma 31.10
Se $a > b \geq 1$ e la chiamata EUCLID(a, b) esegue $k \geq 1$ chiamate ricorsive, allora $a \geq F_{k+2}$ e $b \geq F_{k+1}$.

Dimostrazione La dimostrazione è per induzione su k. Per il caso base dell'induzione, sia $k = 1$. Allora $b \geq 1 = F_2$ e, poiché $a > b$, deve essere $a \geq 2 = F_3$. Poiché $b > (a \bmod b)$, in ogni chiamata ricorsiva il primo argomento è strettamente più grande del secondo; quindi, l'ipotesi che $a > b$ è vera per ogni chiamata ricorsiva.

Supponiamo, per induzione, che il lemma sia vero se vengono fatte $k - 1$ chiamate ricorsive; dovremo poi provare che è vero per k chiamate ricorsive. Poiché $k > 0$, si ha $b > 0$, ed EUCLID(a, b) chiama ricorsivamente EUCLID$(b, a \bmod b)$ che, a sua volta, effettua $k - 1$ chiamate ricorsive. L'ipotesi induttiva poi implica che $b \geq F_{k+1}$ (dimostrando così una parte del lemma) e $(a \bmod b) \geq F_k$. Abbiamo

$$\begin{aligned} b + (a \bmod b) &= b + (a - \lfloor a/b \rfloor\, b) \\ &\leq a \end{aligned}$$

perché $a > b > 0$ implica $\lfloor a/b \rfloor \geq 1$. Quindi

$$\begin{aligned} a &\geq b + (a \bmod b) \\ &\geq F_{k+1} + F_k \\ &= F_{k+2} \end{aligned}$$

■

Il seguente teorema è un corollario immediato di questo lemma.

Teorema 31.11 (Teorema di Lamé)
Per $k \geq 1$ intero qualsiasi, se $a > b \geq 1$ e $b < F_{k+1}$, allora la chiamata EUCLID(a, b) effettua meno di k chiamate ricorsive. ■

Possiamo dimostrare che il limite superiore del Teorema 31.11 è il migliore possibile. I numeri consecutivi di Fibonacci sono gli input nel caso peggiore per l'algoritmo EUCLID. Poiché EUCLID(F_3, F_2) effettua esattamente una chiamata ricorsiva e poiché per $k > 2$ si ha $F_{k+1} \bmod F_k = F_{k-1}$, allora si ha anche

$$\begin{aligned} \text{MCD}(F_{k+1}, F_k) &= \text{MCD}(F_k, (F_{k+1} \bmod F_k)) \\ &= \text{MCD}(F_k, F_{k-1}) \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Quindi EUCLID(F_{k+1}, F_k) esegue la ricorsione *esattamente* $k - 1$ volte, rispettando il limite superiore del Teorema 31.11.

Poiché F_k è all'incirca $\phi^k / \sqrt{5}$, dove ϕ è il rapporto aureo $(1 + \sqrt{5})/2$ definito dall'equazione (3.24), il numero di chiamate ricorsive in EUCLID è $O(\lg b)$ (vedere l'Esercizio 31.2-5 per un limite più stretto). Segue che, se l'algoritmo EUCLID viene applicato a due numeri di β bit, allora eseguirà $O(\beta)$ operazioni aritmetiche e $O(\beta^3)$ operazioni sui bit (supponendo che la moltiplicazione e la divisione di

a	b	$\lfloor a/b \rfloor$	d	x	y
99	78	1	3	−11	14
78	21	3	3	3	−11
21	15	1	3	−2	3
15	6	2	3	1	−2
6	3	2	3	0	1
3	0	—	3	1	0

Figura 31.1 Un esempio dell'esecuzione di EXTENDED-EUCLID con gli input 99 e 78. Ogni riga mostra un livello della ricorsione: i valori degli input a e b, il valore calcolato $\lfloor a/b \rfloor$ e i valori restituiti d, x e y. La tripla (d, x, y) restituita diventa la tripla (d', x', y') utilizzata nel calcolo al precedente livello della ricorsione. La chiamata EXTENDED-EUCLID$(99, 78)$ restituisce $(3, -11, 14)$, cosicché MCD$(99, 78) = 3 = 99 \cdot (-11) + 78 \cdot 14$.

numeri di β bit richiedano $O(\beta^2)$ operazioni sui bit). Il Problema 31-2 chiede di dimostrare un limite $O(\beta^2)$ per il numero di operazioni sui bit.

Forma estesa dell'algoritmo di Euclide

Riscriviamo l'algoritmo di Euclide per calcolare altri dati utili. Specificatamente, estendiamo l'algoritmo per calcolare i coefficienti interi x e y tali che

$$d = \text{MCD}(a, b) = ax + by \tag{31.16}$$

Notate che i coefficienti x e y possono essere nulli o negativi. Questi coefficienti saranno utili più avanti per calcolare gli inversi moltiplicativi modulari. La procedura EXTENDED-EUCLID riceve come input una coppia di interi non negativi e restituisce una tripla nella forma (d, x, y) che soddisfa l'equazione (31.16).

```
EXTENDED-EUCLID(a, b)
1  if b == 0
2      return (a, 1, 0)
3  else (d', x', y') = EXTENDED-EUCLID(b, a mod b)
4      (d, x, y) = (d', y', x' − ⌊a/b⌋ y')
5      return (d, x, y)
```

La Figura 31.1 illustra l'esecuzione di EXTENDED-EUCLID con il calcolo di MCD$(99, 78)$.

La procedura EXTENDED-EUCLID è una variante della procedura EUCLID. La riga 1 è equivalente al test "$b == 0$" nella riga 1 di EUCLID. Se $b = 0$, allora EXTENDED-EUCLID restituisce non soltanto $d = a$ nella riga 2, ma anche i coefficienti $x = 1$ e $y = 0$, cosicché $a = ax + by$. Se $b \neq 0$, EXTENDED-EUCLID prima calcola (d', x', y') in modo che $d' = \text{MCD}(b, a \bmod b)$ e

$$d' = bx' + (a \bmod b)y' \tag{31.17}$$

Come per l'algoritmo EUCLID, in questo caso abbiamo $d = \text{MCD}(a, b) = d' = \text{MCD}(b, a \bmod b)$. Per ottenere x e y tali che $d = ax + by$, iniziamo riscrivendo l'equazione (31.17) utilizzando l'equazione $d = d'$ e l'equazione (3.8):

Acquistato da Michele Michele su Webster il 2022-07-07 20:12 - Numero Ordine Libreria: 1995030116-2207077-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned} d &= bx' + (a - \lfloor a/b \rfloor b)y' \\ &= ay' + b(x' - \lfloor a/b \rfloor y') \end{aligned}$$

Quindi, scegliendo $x = y'$ e $y = x' - \lfloor a/b \rfloor y'$, l'equazione $d = ax + by$ è soddisfatta, provando la correttezza di EXTENDED-EUCLID.

Poiché il numero delle chiamate ricorsive fatte in EUCLID è uguale al numero delle chiamate ricorsive fatte in EXTENDED-EUCLID, i tempi di esecuzione di EUCLID ed EXTENDED-EUCLID sono gli stessi, a meno di un fattore costante. Cioè, per $a > b > 0$, il numero delle chiamate ricorsive è $O(\lg b)$.

Esercizi

31.2-1
Dimostrate che le equazioni (31.11) e (31.12) implicano l'equazione (31.13).

31.2-2
Calcolate i valori (d, x, y) restituiti da EXTENDED-EUCLID$(899, 493)$.

31.2-3
Dimostrate che per a, k ed n interi qualsiasi, si ha

$$\text{MCD}(a, n) = \text{MCD}(a + kn, n)$$

31.2-4
Riscrivete EUCLID in una forma iterativa che usa soltanto una quantità costante di memoria (cioè memorizza soltanto un numero costante di valori interi).

31.2-5
Se $a > b \geq 0$, dimostrate che la chiamata EUCLID(a, b) effettua al più $1 + \log_\phi b$ chiamate ricorsive. Migliorate questo limite portandolo a $1 + \log_\phi(b/\text{MCD}(a, b))$.

31.2-6
Che cosa restituisce la chiamata EXTENDED-EUCLID(F_{k+1}, F_k)? Provate che la vostra risposta è corretta.

31.2-7
Definite la funzione MCD per più di due argomenti con l'equazione ricorsiva $\text{MCD}(a_0, a_1, \ldots, a_n) = \text{MCD}(a_0, \text{MCD}(a_1, a_2, \ldots, a_n))$. Dimostrate che la funzione MCD restituisce la stessa soluzione indipendentemente dall'ordine in cui i suoi argomenti sono specificati. Spiegate anche come trovare gli interi $x_0, x_1, \ldots, x_n$ tali che $\text{MCD}(a_0, a_1, \ldots, a_n) = a_0x_0 + a_1x_1 + \cdots + a_nx_n$. Dimostrate che il numero di divisioni eseguite dal vostro algoritmo è $O(n + \lg(\max\{a_0, a_1, \ldots, a_n\}))$.

31.2-8
Definite $\text{mcm}(a_1, a_2, \ldots, a_n)$ come il ***minimo comune multiplo*** degli n interi $a_1, a_2, \ldots, a_n$, cioè il più piccolo intero non negativo che è multiplo di ogni a_i. Spiegate come calcolare $\text{mcm}(a_1, a_2, \ldots, a_n)$ in modo efficiente utilizzando l'operazione MCD (con due argomenti) come subroutine.

31.2-9
Dimostrate che n_1, n_2, n_3 ed n_4 sono relativamente primi a coppie se e soltanto se

$$\text{MCD}(n_1n_2, n_3n_4) = \text{MCD}(n_1n_3, n_2n_4) = 1$$

Provate più in generale che $n_1, n_2, \ldots, n_k$ sono relativamente primi a coppie, se e soltanto se gli elementi di un insieme di $\lceil \lg k \rceil$ coppie di numeri derivati dagli interi n_i sono relativamente primi.

Acquistato da Michele Mostarda (Attestato 2022-07-07 23:12) Numero Ordine Libreria 1805030153 2707070 Copyright © 2012 McGraw-Hill Education (Italy)

31.3 Aritmetica modulare

Informalmente, possiamo pensare all'aritmetica modulare come all'aritmetica tradizionale dei numeri interi, con la differenza che le operazioni sono modulo n, quindi ogni risultato x è sostituito con l'elemento di $\{0, 1, \ldots, n-1\}$ che è congruo a x, modulo n (cioè x è sostituito con $x \bmod n$). Questo modello informale è sufficiente se ci limitiamo alle operazioni di addizione, sottrazione e moltiplicazione. Un modello più formale per l'aritmetica modulare, che descriveremo ora, si inquadra meglio nel contesto della teoria dei gruppi.

Gruppi finiti

Un ***gruppo*** $(S, \oplus)$ è un insieme S con un'operazione binaria $\oplus$ definita in S per la quale valgono le seguenti proprietà.

1. **Chiusura:** per ogni $a, b \in S$, si ha $a \oplus b \in S$.
2. **Identità:** esiste un elemento $e \in S$, detto ***identità*** del gruppo, tale che $e \oplus a = a \oplus e = a$ per ogni $a \in S$.
3. **Associatività:** per ogni $a, b, c \in S$, si ha $(a \oplus b) \oplus c = a \oplus (b \oplus c)$.
4. **Inverso:** per ogni $a \in S$, esiste un elemento unico $b \in S$, detto ***inverso*** di a, tale che $a \oplus b = b \oplus a = e$.

Per esempio, consideriamo il gruppo familiare $(\mathbb{Z}, +)$ dei numeri interi $\mathbb{Z}$ con l'operazione di addizione: 0 è l'elemento identità e l'inverso di a è $-a$. Se un gruppo $(S, \oplus)$ soddisfa la ***proprietà commutativa*** $a \oplus b = b \oplus a$ per ogni $a, b \in S$, allora è un ***gruppo abeliano***. Se un gruppo $(S, \oplus)$ soddisfa la condizione $|S| < \infty$, allora è un ***gruppo finito***.

I gruppi definiti dall'addizione e dalla moltiplicazione modulari

È possibile formare due gruppi abeliani finiti utilizzando l'addizione e la moltiplicazione modulo n, dove n è un intero positivo. Questi gruppi si basano sulle classi di congruenza degli interi modulo n, definiti nel Paragrafo 31.1.

Per definire un gruppo in $\mathbb{Z}_n$, occorrono delle operazioni binarie appropriate, che possiamo ottenere ridefinendo le operazioni ordinarie di addizione e moltiplicazione. È semplice definire le operazioni di addizione e moltiplicazione per $\mathbb{Z}_n$, perché la classe di congruenza di due interi determina unicamente la classe di congruenza della loro somma o del loro prodotto. Ovvero, se $a \equiv a' \pmod{n}$ e $b \equiv b' \pmod{n}$, allora

$$\begin{aligned} a + b &\equiv a' + b' \pmod{n} \\ ab &\equiv a'b' \pmod{n} \end{aligned}$$

Quindi, definiamo l'addizione e la moltiplicazione modulo n indicandole, rispettivamente, con $+_n$ e $\cdot_n$ nel modo seguente:

$$\begin{aligned} [a]_n +_n [b]_n &= [a + b]_n \\ [a]_n \cdot_n [b]_n &= [ab]_n \end{aligned} \tag{31.18}$$

(In modo analogo, è possibile definire la sottrazione in $\mathbb{Z}_n$ come $[a]_n -_n [b]_n = [a-b]_n$, mentre la divisione è più complicata, come vedremo.) Questi fatti giustificano la prassi comune e comoda di utilizzare il più piccolo elemento non negativo

Acquistato da Michele Michele su Webster il 2022-0[illegible] Mondadori Education Libreria - 1[illegible] Copyright © 2022, McGraw-Hill Education (Italy)

$+_6$	0	1	2	3	4	5
0	0	1	2	3	4	5
1	1	2	3	4	5	0
2	2	3	4	5	0	1
3	3	4	5	0	1	2
4	4	5	0	1	2	3
5	5	0	1	2	3	4

(a)

$\cdot_{15}$	1	2	4	7	8	11	13	14
1	1	2	4	7	8	11	13	14
2	2	4	8	14	1	7	11	13
4	4	8	1	13	2	14	7	11
7	7	14	13	4	11	2	1	8
8	8	1	2	11	4	13	14	7
11	11	7	14	2	13	1	8	4
13	13	11	7	1	14	8	4	2
14	14	13	11	8	7	4	2	1

(b)

Figura 31.2 Due gruppi finiti. Le classi di congruenza sono indicate dai loro elementi rappresentativi. **(a)** Il gruppo $(\mathbb{Z}_6, +_6)$. **(b)** Il gruppo $(\mathbb{Z}_{15}^*, \cdot_{15})$.

di ciascuna classe di congruenza come rappresentante di classe quando vengono eseguiti i calcoli in $\mathbb{Z}_n$. L'addizione, la sottrazione e la moltiplicazione sono eseguite di solito sui rappresentanti, ma ciascun risultato x è sostituito con il rappresentante della sua classe (cioè con $x \bmod n$). Utilizzando questa definizione di addizione modulo n, definiamo il ***gruppo additivo modulo n*** come $(\mathbb{Z}_n, +_n)$. La dimensione del gruppo additivo modulo n è $|\mathbb{Z}_n| = n$. La Figura 31.2(a) fornisce la tabella delle operazioni per il gruppo $(\mathbb{Z}_6, +_6)$.

Teorema 31.12
Il sistema $(\mathbb{Z}_n, +_n)$ è un gruppo abeliano finito.

Dimostrazione L'equazione (31.18) dimostra che il gruppo $(\mathbb{Z}_n, +_n)$ è chiuso. L'associatività e la commutatività di $+_n$ seguono dall'associatività e dalla commutatività di $+$:

$$\begin{aligned}
([a]_n +_n [b]_n) +_n [c]_n &= [a + b]_n +_n [c]_n \\
&= [(a + b) + c]_n \\
&= [a + (b + c)]_n \\
&= [a]_n +_n [b + c]_n \\
&= [a]_n +_n ([b]_n +_n [c]_n)
\end{aligned}$$

$$\begin{aligned}
[a]_n +_n [b]_n &= [a + b]_n \\
&= [b + a]_n \\
&= [b]_n +_n [a]_n
\end{aligned}$$

L'elemento identità di $(\mathbb{Z}_n, +_n)$ è 0 (cioè $[0]_n$). L'inverso (additivo) di un elemento a (cioè $[a]_n$) è l'elemento $-a$ (cioè $[-a]_n$ o $[n - a]_n$), perché $[a]_n +_n [-a]_n = [a - a]_n = [0]_n$. ■

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Utilizzando la definizione della moltiplicazione modulo n, definiamo il ***gruppo moltiplicativo modulo n*** come $(\mathbb{Z}_n^*, \cdot_n)$. Gli elementi di questo gruppo sono l'insieme $\mathbb{Z}_n^*$ degli elementi in $\mathbb{Z}_n$ che sono relativamente primi con n:

$$\mathbb{Z}_n^* = \{[a]_n \in \mathbb{Z}_n : \text{MCD}(a, n) = 1\}$$

Per vedere che $\mathbb{Z}_n^*$ è ben definito, notiamo che per $0 \le a < n$, si ha $a \equiv (a + kn) \pmod{n}$ per ogni intero k. Per l'Esercizio 31.2-3, quindi, $\text{MCD}(a, n) = 1$ implica $\text{MCD}(a + kn, n) = 1$ per ogni intero k. Poiché $[a]_n = \{a + kn : k \in \mathbb{Z}\}$, l'insieme $\mathbb{Z}_n^*$ è ben definito. Un esempio di tale gruppo è

$$\mathbb{Z}_{15}^* = \{1, 2, 4, 7, 8, 11, 13, 14\}$$

dove l'operazione del gruppo è la moltiplicazione modulo 15. (Qui indichiamo un elemento $[a]_{15}$ con a; per esempio, indichiamo $[7]_{15}$ con 7.) La Figura 31.2(b) illustra il gruppo $(\mathbb{Z}_{15}^*, \cdot_{15})$. Per esempio, $8 \cdot 11 \equiv 13 \pmod{15}$, se operiamo in $\mathbb{Z}_{15}^*$. L'elemento identità per questo gruppo è 1.

Teorema 31.13
Il sistema $(\mathbb{Z}_n^*, \cdot_n)$ è un gruppo abeliano finito.

Dimostrazione Il Teorema 31.6 implica che $(\mathbb{Z}_n^*, \cdot_n)$ è chiuso. L'associatività e la commutatività possono essere provate per $\cdot_n$ come è stato fatto per $+_n$ nella dimostrazione del Teorema 31.12. L'elemento identità è $[1]_n$. Per dimostrare l'esistenza degli inversi, indichiamo con a un elemento di $\mathbb{Z}_n^*$ e con (d, x, y) l'output di $\text{EXTENDED-EUCLID}(a, n)$. Allora $d = 1$, perché $a \in \mathbb{Z}_n^*$, e

$$ax + ny = 1 \tag{31.19}$$

ovvero

$$ax \equiv 1 \pmod{n}$$

Quindi, $[x]_n$ è un inverso moltiplicativo di $[a]_n$, modulo n.

Inoltre, asseriamo che $[x]_n \in \mathbb{Z}_n^*$. Per capire perché, l'equazione (31.19) dimostra che la più piccola combinazione lineare positiva di x ed n deve essere 1. Dunque, il Teorema 31.2 implica che $\text{MCD}(x, n) = 1$. La dimostrazione che gli inversi sono unicamente definiti è rimandata al Corollario 31.26. ■

Come esempio di calcolo di inversi moltiplicativi, supponiamo che $a = 5$ ed $n = 11$. Allora $\text{EXTENDED-EUCLID}(a, n)$ restituisce $(d, x, y) = (1, -2, 1)$, quindi $1 = 5 \cdot (-2) + 11 \cdot 1$. Dunque, $[-2]_{11}$ (cioè $[9]_{11}$) è l'inverso moltiplicativo di $[5]_{11}$.

Quando opereremo con i gruppi $(\mathbb{Z}_n, +_n)$ e $(\mathbb{Z}_n^*, \cdot_n)$ nella parte restante di questo capitolo, seguiremo la comoda prassi di indicare le classi di congruenza con i loro elementi rappresentativi e di indicare le operazioni $+_n$ e $\cdot_n$, rispettivamente, con le consuete notazioni aritmetiche $+$ e $\cdot$ (o giustapposizione, cosicché $ab = a \cdot b$). Inoltre, le congruenze modulo n possono essere interpretate come equazioni in $\mathbb{Z}_n$. Per esempio, la due affermazioni seguenti sono equivalenti:

$$\begin{aligned} ax &\equiv b \pmod{n} \\ [a]_n \cdot_n [x]_n &= [b]_n \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-0[illegible] Copyright The McGraw-Hill Companies (2022)

Per comodità, a volte faremo riferimento a un gruppo $(S, \oplus)$ semplicemente con S, quando l'operazione è evidente dal contesto. Potremo così fare riferimento ai gruppi $(\mathbb{Z}_n, +_n)$ e $(\mathbb{Z}_n^*, \cdot_n)$, rispettivamente, con $\mathbb{Z}_n$ e $\mathbb{Z}_n^*$.

L'inverso (moltiplicativo) di un elemento a è indicato con $(a^{-1} \bmod n)$. La divisione in $\mathbb{Z}_n^*$ è definita dall'equazione $a/b \equiv ab^{-1} \pmod{n}$. Per esempio, in $\mathbb{Z}_{15}^*$ abbiamo che $7^{-1} \equiv 13 \pmod{15}$, perché $7 \cdot 13 \equiv 91 \equiv 1 \pmod{15}$, quindi $4/7 \equiv 4 \cdot 13 \equiv 7 \pmod{15}$.

La dimensione di $\mathbb{Z}_n^*$ è indicata con $\phi(n)$. Questa funzione, detta ***funzione fi di Eulero*** soddisfa la seguente equazione:

$$\phi(n) = n \prod_{p \,:\, p \text{ è primo e } p \mid n} \left(1 - \frac{1}{p}\right) \tag{31.20}$$

Dove p varia su tutti i numeri primi che dividono n (incluso lo stesso n, se n è primo). Non dimostreremo questa formula. Intuitivamente, si parte dalla lista degli n resti $\{0, 1, \ldots, n-1\}$ e poi, per ogni primo p che divide n, si elimina ogni multiplo di p nella lista. Per esempio, poiché i divisori primi di 45 sono 3 e 5, si ha

$$\begin{aligned}\phi(45) &= 45\left(1 - \frac{1}{3}\right)\left(1 - \frac{1}{5}\right) \\ &= 45\left(\frac{2}{3}\right)\left(\frac{4}{5}\right) \\ &= 24\end{aligned}$$

Se p è primo, allora $\mathbb{Z}_p^* = \{1, 2, \ldots, p-1\}$, e

$$\begin{aligned}\phi(p) &= p\left(1 - \frac{1}{p}\right) \\ &= p - 1\end{aligned} \tag{31.21}$$

Se n è composto, allora $\phi(n) < n - 1$, sebbene si possa dimostrare che

$$\phi(n) > \frac{n}{e^{\gamma} \ln\ln n + \frac{3}{\ln\ln n}} \tag{31.22}$$

per $n \geq 3$, dove $\gamma = 0.5772156649\ldots$ è la ***costante di Eulero***. Un limite inferiore più semplice (ma meno stretto) per $n > 5$ è

$$\phi(n) > \frac{n}{6 \ln\ln n} \tag{31.23}$$

Il limite inferiore (31.22) è essenzialmente il migliore possibile, in quanto

$$\liminf_{n \to \infty} \frac{\phi(n)}{n / \ln\ln n} = e^{-\gamma} \tag{31.24}$$

Sottogruppi

Se $(S, \oplus)$ è un gruppo, $S' \subseteq S$ e anche $(S', \oplus)$ è un gruppo, allora $(S', \oplus)$ è detto ***sottogruppo*** di $(S, \oplus)$. Per esempio, gli interi pari formano un sottogruppo degli interi rispetto all'addizione. Il seguente teorema fornisce un utile strumento per il riconoscimento dei sottogruppi.

Acquistato da Michele Mastrolia (mastrolia.michele@gmail.com) - 2022-07-31 20:22:22 - Numero Ordine Libreria 1900000026 - 2022 - Copyright © 2022, McGraw-Hill Education (Italy)

Teorema 31.14 (Un sottoinsieme chiuso non vuoto di un gruppo finito è un sottogruppo)
Se $(S, \oplus)$ è un gruppo finito ed S' è un sottoinsieme non vuoto di S tale che $a \oplus b \in S'$ per ogni $a, b \in S'$, allora $(S', \oplus)$ è un sottogruppo di $(S, \oplus)$.

Dimostrazione Lasciamo al lettore il compito di dimostrare questo teorema (Esercizio 31.3-3). ■

Per esempio, l'insieme $\{0, 2, 4, 6\}$ forma un sottogruppo di $\mathbb{Z}_8$, perché non è vuoto ed è chiuso rispetto all'operazione $+$ (cioè è chiuso rispetto a $+_8$).

Il seguente teorema fornisce un vincolo estremamente utile per la dimensione di un sottogruppo; omettiamo la dimostrazione.

Teorema 31.15 (Teorema di Lagrange)
Se $(S, \oplus)$ è un gruppo finito ed $(S', \oplus)$ è un sottogruppo di $(S, \oplus)$, allora $|S'|$ è un divisore di $|S|$. ■

Un sottogruppo S' di un gruppo S è detto ***sottogruppo proprio*** se $S' \neq S$. Il seguente corollario sarà utilizzato nella nostra analisi del test di primalità di Miller-Rabin nel Paragrafo 31.8.

Corollario 31.16
Se S' è un sottogruppo proprio di un gruppo finito S, allora $|S'| \leq |S|/2$. ■

Sottogruppi generati da un elemento

Il Teorema 31.14 fornisce un metodo interessante per produrre un sottogruppo di un gruppo finito $(S, \oplus)$: scegliamo un elemento a e prendiamo tutti gli elementi che possono essere generati da a utilizzando l'operazione del gruppo. Specificatamente, definiamo $a^{(k)}$ per $k \geq 1$ come

$$a^{(k)} = \bigoplus_{i=1}^{k} a = \underbrace{a \oplus a \oplus \cdots \oplus a}_{k}$$

Per esempio, se prendiamo $a = 2$ nel gruppo $\mathbb{Z}_6$, la sequenza $a^{(1)}, a^{(2)}, \ldots$ è

$$2, 4, 0, 2, 4, 0, 2, 4, 0, \ldots$$

Nel gruppo $\mathbb{Z}_n$ abbiamo $a^{(k)} = ka \bmod n$ e nel gruppo $\mathbb{Z}_n^*$ abbiamo $a^{(k)} = a^k \bmod n$. Il ***sottogruppo generato da a***, indicato con $\langle a \rangle$ o $(\langle a \rangle, \oplus)$, è definito da

$$\langle a \rangle = \{a^{(k)} : k \geq 1\}$$

Diciamo che a ***genera*** il sottogruppo $\langle a \rangle$ o che a è un ***generatore*** di $\langle a \rangle$. Poiché S è finito, $\langle a \rangle$ è un sottoinsieme di S, eventualmente comprendente tutto S. Poiché l'associatività di $\oplus$ implica

$$a^{(i)} \oplus a^{(j)} = a^{(i+j)}$$

$\langle a \rangle$ è chiuso e quindi, per il Teorema 31.14, $\langle a \rangle$ è un sottogruppo di S. Per esempio, in $\mathbb{Z}_6$ si ha

$$\begin{aligned} \langle 0 \rangle &= \{0\} \\ \langle 1 \rangle &= \{0, 1, 2, 3, 4, 5\} \\ \langle 2 \rangle &= \{0, 2, 4\} \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Analogamente, in $\mathbb{Z}_7^*$ si ha

$$\begin{aligned} \langle 1 \rangle &= \{1\} \\ \langle 2 \rangle &= \{1, 2, 4\} \\ \langle 3 \rangle &= \{1, 2, 3, 4, 5, 6\} \end{aligned}$$

L'***ordine*** di a (nel gruppo S), indicato da $\text{ord}(a)$, è definito come il più piccolo intero positivo t tale che $a^{(t)} = e$.

Teorema 31.17
Per qualsiasi gruppo finito $(S, \oplus)$ e qualsiasi $a \in S$, l'ordine di un elemento è uguale alla dimensione del sottogruppo che genera, ovvero $\text{ord}(a) = |\langle a \rangle|$.

Dimostrazione Sia $t = \text{ord}(a)$. Poiché $a^{(t)} = e$ e $a^{(t+k)} = a^{(t)} \oplus a^{(k)} = a^{(k)}$ per $k \geq 1$, se $i > t$, allora $a^{(i)} = a^{(j)}$ per qualche $j < i$. Quindi, non ci sono nuovi elementi dopo $a^{(t)}$, cosicché $\langle a \rangle = \{a^{(1)}, a^{(2)}, \ldots, a^{(t)}\}$ e $|\langle a \rangle| \leq t$. Per dimostrare che $|\langle a \rangle| \geq t$, supponiamo per assurdo che $a^{(i)} = a^{(j)}$ per qualche i e j che soddisfano la relazione $1 \leq i < j \leq t$. Allora $a^{(i+k)} = a^{(j+k)}$ per $k \geq 0$. Ma questo implica che $a^{(i+(t-j))} = a^{(j+(t-j))} = e$, che è una contraddizione, perché $i + (t - j) < t$, ma t è il minimo valore positivo tale che $a^{(t)} = e$. Di conseguenza, ogni elemento della sequenza $a^{(1)}, a^{(2)}, \ldots, a^{(t)}$ è distinto e $|\langle a \rangle| \geq t$. Concludiamo che $\text{ord}(a) = |\langle a \rangle|$. ■

Corollario 31.18
La sequenza $a^{(1)}, a^{(2)}, \ldots$ è periodica con periodo $t = \text{ord}(a)$; cioè $a^{(i)} = a^{(j)}$ se e soltanto se $i \equiv j \pmod{t}$. ■

Coerentemente con il precedente corollario, possiamo definire $a^{(0)}$ come e e $a^{(i)}$ come $a^{(i \bmod t)}$, dove $t = \text{ord}(a)$, per ogni intero i.

Corollario 31.19
Se $(S, \oplus)$ è un gruppo finito con l'elemento identità e, allora per ogni $a \in S$, si ha

$$a^{(|S|)} = e$$

Dimostrazione Il teorema di Lagrange (Teorema 31.15) implica che $\text{ord}(a) \mid |S|$, e quindi $|S| \equiv 0 \pmod{t}$, dove $t = \text{ord}(a)$. Pertanto, $a^{(|S|)} = a^{(0)} = e$. ■

Esercizi

31.3-1
Create le tabelle delle operazioni per i gruppi $(\mathbb{Z}_4, +_4)$ e $(\mathbb{Z}_5^*, \cdot_5)$. Dimostrate che questi gruppi sono isomorfi esibendo una corrispondenza biunivoca α tra i loro elementi tale che $a + b \equiv c \pmod{4}$ se e soltanto se $\alpha(a) \cdot \alpha(b) \equiv \alpha(c) \pmod{5}$.

31.3-2
Elencate tutti i sottogruppi di $\mathbb{Z}_9$ e di $\mathbb{Z}_{13}^*$.

31.3-3
Dimostrate il Teorema 31.14.

31.3-4
Dimostrate che, se p è primo ed e è un intero positivo, allora

$$\phi(p^e) = p^{e-1}(p - 1)$$

31.3-5
Dimostrate che, per ogni $n > 1$ e per ogni $a \in \mathbb{Z}_n^*$, la funzione $f_a : \mathbb{Z}_n^* \to \mathbb{Z}_n^*$ definita da $f_a(x) = ax \bmod n$ è una permutazione di $\mathbb{Z}_n^*$.

Acquistato da Michele Mundo su Wattpad il 2022-07-27 20:13. Nuova Ordine Libreria 1681/2016-21707-0 Copyright © 2022, McGraw-Hill Education (Italy)

31.4 Risolvere le equazioni lineari modulari

Consideriamo adesso il problema di trovare le soluzioni dell'equazione

$$ax \equiv b \pmod{n} \tag{31.25}$$

dove $a > 0$ ed $n > 0$. Ci sono varie applicazioni per questo problema; per esempio, lo utilizzeremo come parte della procedura per trovare le chiavi nel sistema di crittografia a chiave pubblica RSA nel Paragrafo 31.7. Supponendo di conoscere i valori di a, b ed n, vogliamo trovare tutti i valori di x, modulo n, che soddisfano l'equazione (31.25). Ci possono essere zero, una o più soluzioni.

Indichiamo con $\langle a \rangle$ il sottogruppo di $\mathbb{Z}_n$ generato da a. Poiché $\langle a \rangle = \{a^{(x)} : x > 0\} = \{ax \bmod n : x > 0\}$, l'equazione (31.25) ha una soluzione se e soltanto se $b \in \langle a \rangle$. Il teorema di Lagrange (Teorema 31.15) ci dice che $|\langle a \rangle|$ deve essere un divisore di n. Il seguente teorema offre una caratterizzazione esatta di $\langle a \rangle$.

Teorema 31.20
Per a ed n interi positivi qualsiasi, se $d = \text{MCD}(a, n)$, allora

$$\langle a \rangle = \langle d \rangle = \{0, d, 2d, \ldots, ((n/d) - 1)d\} \tag{31.26}$$

in $\mathbb{Z}_n$, e quindi

$$|\langle a \rangle| = n/d$$

Dimostrazione Iniziamo dimostrando che $d \in \langle a \rangle$. Ricordiamo che la procedura EXTENDED-EUCLID(a, n) produce interi x' e y' tali che $ax' + ny' = d$. Quindi $ax' \equiv d \pmod{n}$, e allora $d \in \langle a \rangle$. In altre parole, d è un multiplo di a in $\mathbb{Z}_n$.

Poiché $d \in \langle a \rangle$, segue che ogni multiplo di d appartiene ad $\langle a \rangle$, perché qualsiasi multiplo di un multiplo di a è esso stesso un multiplo di a. Quindi $\langle a \rangle$ contiene tutti gli elementi in $\{0, d, 2d, \ldots, ((n/d) - 1)d\}$. Cioè $\langle d \rangle \subseteq \langle a \rangle$.

Adesso dimostriamo che $\langle a \rangle \subseteq \langle d \rangle$. Se $m \in \langle a \rangle$, allora $m = ax \bmod n$ per qualche intero x, e così $m = ax + ny$ per qualche intero y. Poiché $d \mid a$ e $d \mid n$, allora $d \mid m$ per l'equazione (31.4). Dunque $m \in \langle d \rangle$.

Combinando questi risultati, si ha che $\langle a \rangle = \langle d \rangle$. Per vedere che $|\langle a \rangle| = n/d$, osserviamo che ci sono esattamente n/d multipli di d tra 0 ed $n - 1$, estremi inclusi. ■

Corollario 31.21
L'equazione $ax \equiv b \pmod{n}$ è risolvibile rispetto all'incognita x se e soltanto se $d \mid b$, dove $d = \text{MCD}(a, n)$. ■

Dimostrazione L'equazione $ax \equiv b \pmod{n}$ è risolvibile se e soltanto se $[b] \in \langle a \rangle$, che equivale a dire

Acquistato da Michele Michele su Webster il 2022-07-23 12:17 Numero Ordine Libraria: 109503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$(b \bmod n) \in \{0, d, 2d, \ldots, ((n/d) - 1)d\}$$

per il Teorema 31.20. Se $0 \leq b < n$, allora $b \in \langle a \rangle$ se e soltanto se $d \mid b$, in quanto gli elementi di $\langle a \rangle$ sono multipli di d. Se $b < 0$ o $b \geq n$, il corollario deriva dall'osservazione che $d \mid b$ se e soltanto se $d \mid (b \bmod n)$, in quanto b e $b \bmod n$ differiscono per un multiplo di n, che è un multiplo di d. ■

Corollario 31.22
L'equazione $ax \equiv b \pmod{n}$ ha d soluzioni distinte modulo n, dove $d = \text{MCD}(a, n)$, oppure non ha soluzioni.

Dimostrazione Se $ax \equiv b \pmod{n}$ ha una soluzione, allora $b \in \langle a \rangle$. Per il Teorema 31.17, $\text{ord}(a) = |\langle a \rangle|$, e così il Corollario 31.18 e il Teorema 31.20 implicano che la sequenza $ai \bmod n$, per $i = 0, 1, \ldots$, è periodica con periodo $|\langle a \rangle| = n/d$. Se $b \in \langle a \rangle$, allora b appare esattamente d volte nella sequenza $ai \bmod n$, per $i = 0, 1, \ldots, n-1$, perché il blocco dei valori $\langle a \rangle$ avente lunghezza n/d viene ripetuto esattamente d volte quando i cresce da 0 a $n - 1$. Gli indici x delle d posizioni per cui $ax \bmod n = b$ sono le soluzioni dell'equazione $ax \equiv b \pmod{n}$. ■

Teorema 31.23
Sia $d = \text{MCD}(a, n)$ e supponiamo che $d = ax' + ny'$, con x' e y' interi (per esempio, quelli calcolati da EXTENDED-EUCLID). Se $d \mid b$, allora una delle soluzioni dell'equazione $ax \equiv b \pmod{n}$ ha il valore x_0, dove

$$x_0 = x'(b/d) \bmod n$$

Dimostrazione Abbiamo

$$\begin{aligned} ax_0 &\equiv ax'(b/d) \pmod{n} \\ &\equiv d(b/d) \pmod{n} \qquad (\text{perché } ax' \equiv d \pmod{n}) \\ &\equiv b \pmod{n} \end{aligned}$$

e quindi x_0 è una soluzione di $ax \equiv b \pmod{n}$. ■

Teorema 31.24
Supponiamo che l'equazione $ax \equiv b \pmod{n}$ sia risolvibile (cioè $d \mid b$, dove $d = \text{MCD}(a, n)$) e che x_0 sia una soluzione di questa equazione. Allora l'equazione ha esattamente d soluzioni distinte, modulo n, date da $x_i = x_0 + i(n/d)$ per $i = 0, 1, \ldots, d - 1$.

Dimostrazione Poiché $n/d > 0$ e $0 \le i(n/d) < n$ per $i = 0, 1, \ldots, d - 1$, i valori $x_0, x_1, \ldots, x_{d-1}$ sono tutti distinti, modulo n. Poiché x_0 è una soluzione di $ax \equiv b \pmod{n}$, si ha $ax_0 \bmod n = b$. Quindi, per $i = 0, 1, \ldots, d - 1$, si ha

$$\begin{aligned} ax_i \bmod n &= a(x_0 + in/d) \bmod n \\ &= (ax_0 + ain/d) \bmod n \\ &= ax_0 \bmod n \quad (\text{perché } d \mid a \text{ implica che } ain/d \text{ è un multiplo di } n) \\ &\equiv b \pmod{n} \end{aligned}$$

e quindi $ax_i \equiv b \pmod{n}$, che implica che anche x_i è una soluzione. Per il Corollario 31.22, l'equazione $ax \equiv b \pmod{n}$ ha esattamente d soluzioni, e quindi $x_0, x_1, \ldots, x_{d-1}$ devono essere tutte le soluzioni. ■

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1996039016-2207077 © Copyright© 2022, McGraw-Hill Education (Italy)

Abbiamo sviluppato tutta la matematica necessaria per risolvere l'equazione $ax \equiv b \pmod{n}$; il seguente algoritmo stampa tutte le soluzioni di questa equazione. Gli input a ed n sono interi positivi arbitrari e b è un intero arbitrario.

```
MODULAR-LINEAR-EQUATION-SOLVER(a, b, n)
1  (d, x', y') = EXTENDED-EUCLID(a, n)
2  if d | b
3       x_0 = x'(b/d) mod n
4       for i = 0 to d − 1
5            stampa (x_0 + i(n/d)) mod n
6  else stampa "nessuna soluzione"
```

Per provare questa procedura, consideriamo l'equazione $14x \equiv 30 \pmod{100}$ (qui, $a = 14$, $b = 30$ ed $n = 100$). Chiamando EXTENDED-EUCLID nella riga 1, otteniamo $(d, x, y) = (2, -7, 1)$. Poiché $2 \mid 30$, vengono eseguite le righe 3–5. Nella riga 3, calcoliamo $x_0 = (-7)(15) \bmod 100 = 95$. Il ciclo nelle righe 4–5 stampa le due soluzioni 95 e 45.

La procedura MODULAR-LINEAR-EQUATION-SOLVER opera nel modo seguente. La riga 1 calcola $d = \text{MCD}(a, n)$ e due valori x' e y' tali che $d = ax' + ny'$, dimostrando che x' è una soluzione dell'equazione $ax' \equiv d \pmod{n}$. Se d non divide b, allora l'equazione $ax \equiv b \pmod{n}$ non ha soluzione, per il Corollario 31.21. La riga 2 verifica se $d \mid b$; se ciò non è vero, la riga 6 segnala che non ci sono soluzioni; altrimenti la riga 3 calcola una soluzione x_0 di $ax \equiv b \pmod{n}$, secondo il Teorema 31.23. Data una soluzione, il Teorema 31.24 stabilisce che le altre $d - 1$ soluzioni possono essere ottenute aggiungendo multipli di (n/d), modulo n. Il ciclo **for** (righe 4–5) stampa tutte le d soluzioni, partendo da x_0 e distanziando le altre soluzioni di (n/d), modulo n.

L'algoritmo MODULAR-LINEAR-EQUATION-SOLVER esegue $O(\lg n + \text{MCD}(a, n))$ operazioni aritmetiche, perché EXTENDED-EUCLID esegue $O(\lg n)$ operazioni aritmetiche e ogni iterazione del ciclo **for** (righe 4–5) esegue un numero costante di operazioni aritmetiche.

I seguenti corollari sono delle specializzazioni particolarmente interessanti del Teorema 31.24.

Corollario 31.25
Per ogni $n > 1$, se $\text{MCD}(a, n) = 1$, allora l'equazione $ax \equiv b \pmod{n}$ ha una soluzione unica, modulo n. ■

Se $b = 1$, un caso comune di notevole interesse, la soluzione x che stiamo cercando è un ***inverso moltiplicativo*** di a, modulo n.

Corollario 31.26
Per ogni $n > 1$, se $\text{MCD}(a, n) = 1$, allora l'equazione $ax \equiv 1 \pmod{n}$ ha una soluzione unica, modulo n; altrimenti, non ha soluzione. ■

Il Corollario 31.26 ci consente di utilizzare la notazione $(a^{-1} \bmod n)$ per fare riferimento all'inverso moltiplicativo di a, modulo n, quando a e n sono primi tra loro. Se $\text{MCD}(a, n) = 1$, allora una soluzione dell'equazione $ax \equiv 1 \pmod{n}$ è l'intero x restituito da EXTENDED-EUCLID, perché l'equazione

$$\text{MCD}(a, n) = 1 = ax + ny$$

implica $ax \equiv 1 \pmod{n}$. Quindi, possiamo calcolare $(a^{-1} \bmod n)$ in modo efficiente utilizzando EXTENDED-EUCLID.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Esercizi

31.4-1

Trovate tutte le soluzioni dell'equazione $35x \equiv 10 \pmod{50}$.

31.4-2

Dimostrate che l'equazione $ax \equiv ay \pmod{n}$ implica $x \equiv y \pmod{n}$ se $\text{MCD}(a, n) = 1$. Dimostrate che la condizione $\text{MCD}(a, n) = 1$ è necessaria trovando un controesempio con $\text{MCD}(a, n) > 1$.

31.4-3

Considerate la seguente modifica della riga 3 della procedura MODULAR-LINEAR-EQUATION-SOLVER:

3 $\quad x_0 = x'(b/d) \bmod (n/d)$

Funzionerà? Spiegate la risposta.

31.4-4 ⋆

Sia $f(x) \equiv f_0 + f_1 x + \cdots + f_t x^t \pmod{p}$ un polinomio di grado t, con i coefficienti f_i presi da $\mathbb{Z}_p$, dove p è un numero primo. Diciamo che $a \in \mathbb{Z}_p$ è uno ***zero*** di f se $f(a) \equiv 0 \pmod{p}$. Dimostrate che, se a è uno zero di f, allora $f(x) \equiv (x - a)g(x) \pmod{p}$ per qualche polinomio $g(x)$ di grado $t - 1$. Dimostrate per induzione su t che un polinomio $f(x)$ di grado t può avere al più t zeri distinti modulo un primo p.

31.5 Il teorema cinese del resto

Intorno al 100 d.C. il matematico cinese Sun-Tsŭ risolse il problema di trovare quegli interi x che danno come resto 2, 3 e 2 quando vengono divisi, rispettivamente, per 3, 5 e 7. Una di tali soluzioni è $x = 23$; tutte le soluzioni hanno la forma $23 + 105k$ per k intero arbitrario. Il "teorema cinese del resto" fornisce una corrispondenza tra un sistema di equazioni modulo un insieme di moduli primi tra loro a coppie (per esempio, 3, 5 e 7) e un'equazione modulo il loro prodotto (per esempio, 105).

Il teorema cinese del resto ha due impieghi principali. Supponiamo che il numero intero n sia scomponibile nei fattori $n = n_1 n_2 \cdots n_k$, con i fattori n_i relativamente primi a coppie. In primo luogo, il teorema cinese del resto è un "teorema descrittivo di struttura" perché descrive la struttura di $\mathbb{Z}_n$ come identica a quella del prodotto cartesiano $\mathbb{Z}_{n_1} \times \mathbb{Z}_{n_2} \times \cdots \times \mathbb{Z}_{n_k}$ con l'addizione e la moltiplicazione componente per componente modulo n_i nella i-esima componente. In secondo luogo, questa descrizione spesso può essere utilizzata per creare algoritmi efficienti, perché operare in ciascuno dei sistemi $\mathbb{Z}_{n_i}$ può essere più efficiente (in termini di operazioni sui bit) che eseguire operazioni modulo n.

Teorema 31.27 (Teorema cinese del resto)

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 . Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Sia $n = n_1 n_2 \cdots n_k$, dove gli n_i sono relativamente primi a coppie. Consideriamo la corrispondenza

$$a \leftrightarrow (a_1, a_2, \ldots, a_k) \tag{31.27}$$

dove $a \in \mathbb{Z}_n$, $a_i \in \mathbb{Z}_{n_i}$ e

$a_i = a \bmod n_i$

per $i = 1, 2, \ldots, k$. Allora, la relazione (31.27) esprime una corrispondenza biunivoca (biiezione) tra $\mathbb{Z}_n$ e il prodotto cartesiano $\mathbb{Z}_{n_1} \times \mathbb{Z}_{n_2} \times \cdots \times \mathbb{Z}_{n_k}$. Le operazioni eseguite sugli elementi di $\mathbb{Z}_n$ possono essere eseguite in modo equivalente sulle corrispondenti k-tuple eseguendo le operazioni separatamente in ciascuna posizione coordinata e usando il sistema appropriato. Ovvero, se

$$\begin{aligned} a &\leftrightarrow (a_1, a_2, \ldots, a_k) \\ b &\leftrightarrow (b_1, b_2, \ldots, b_k) \end{aligned}$$

allora

$$(a + b) \bmod n \leftrightarrow ((a_1 + b_1) \bmod n_1, \ldots, (a_k + b_k) \bmod n_k) \tag{31.28}$$

$$(a - b) \bmod n \leftrightarrow ((a_1 - b_1) \bmod n_1, \ldots, (a_k - b_k) \bmod n_k) \tag{31.29}$$

$$(ab) \bmod n \leftrightarrow (a_1 b_1 \bmod n_1, \ldots, a_k b_k \bmod n_k) \tag{31.30}$$

Dimostrazione La trasformazione tra le due rappresentazioni è abbastanza semplice. Il passaggio da a a $(a_1, a_2, \ldots, a_k)$ è molto semplice e richiede soltanto k operazioni "mod". Il calcolo di a dagli input $(a_1, a_2, \ldots, a_k)$ è un po' più complicato e può essere eseguito nel modo seguente. Iniziamo definendo $m_i = n/n_i$ per $i = 1, 2, \ldots, k$; quindi m_i è il prodotto di tutti gli n_j diversi da n_i: $m_i = n_1 n_2 \cdots n_{i-1} n_{i+1} \cdots n_k$. Poi definiamo

$$c_i = m_i (m_i^{-1} \bmod n_i) \tag{31.31}$$

per $i = 1, 2, \ldots, k$. L'equazione (31.31) è sempre ben definita: poiché m_i ed n_i sono primi tra loro (per il Teorema 31.6), il Corollario 31.26 assicura che $(m_i^{-1} \bmod n_i)$ esiste. Infine, possiamo calcolare a come una funzione di a_1, a_2, $\ldots$, a_k nel modo seguente:

$$a \equiv (a_1 c_1 + a_2 c_2 + \cdots + a_k c_k) \pmod{n} \tag{31.32}$$

Dimostriamo adesso che l'equazione (31.32) garantisce che $a \equiv a_i \pmod{n_i}$ per $i = 1, 2, \ldots, k$. Notate che se $j \neq i$, allora $m_j \equiv 0 \pmod{n_i}$, e questo implica che $c_j \equiv m_j \equiv 0 \pmod{n_i}$. Notate inoltre che $c_i \equiv 1 \pmod{n_i}$, per l'equazione (31.31). Abbiamo così l'interessante e utile corrispondenza

$$c_i \leftrightarrow (0, 0, \ldots, 0, 1, 0, \ldots, 0)$$

un vettore che ha 0 in tutte le posizioni, tranne nella i-esima posizione, dove ha un 1. I termini c_i quindi formano, in un certo senso, una "base" per la rappresentazione. Pertanto, per ogni i abbiamo

$$\begin{aligned} a &\equiv a_i c_i && \pmod{n_i} \\ &\equiv a_i m_i (m_i^{-1} \bmod n_i) && \pmod{n_i} \\ &\equiv a_i && \pmod{n_i} \end{aligned}$$

che è quanto volevamo dimostrare: il nostro metodo di calcolare a dai termini a_i produce un risultato a che soddisfa i vincoli $a \equiv a_i \pmod{n_i}$ per $i = 1, 2, \ldots, k$. La corrispondenza è biunivoca, perché possiamo eseguire la trasformazione in entrambe le direzioni. Infine, le equazioni (31.28)–(31.30) seguono direttamente dall'Esercizio 31.1-7, perché $x \bmod n_i = (x \bmod n) \bmod n_i$ per ogni x e $i = 1, 2, \ldots, k$. ■

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950316-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

I seguenti corollari saranno utilizzati più avanti in questo capitolo.

	0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	40	15	55	30	5	45	20	60	35	10	50	25
1	26	1	41	16	56	31	6	46	21	61	36	11	51
2	52	27	2	42	17	57	32	7	47	22	62	37	12
3	13	53	28	3	43	18	58	33	8	48	23	63	38
4	39	14	54	29	4	44	19	59	34	9	49	24	64

Figura 31.3 Una illustrazione del teorema cinese del resto per $n_1 = 5$ e $n_2 = 13$. Per questo esempio, $c_1 = 26$ e $c_2 = 40$. Nella riga i, colonna j è illustrato il valore di a, modulo 65, tale che $(a \bmod 5) = i$ e $(a \bmod 13) = j$. Notate che la riga 0, colonna 0 contiene uno zero. Analogamente, la riga 4, colonna 12 contiene 64 (equivalente a -1). Poiché $c_1 = 26$, uno spostamento in basso di una riga aumenta a di 26. Analogamente, $c_2 = 40$ significa che spostandosi a destra di una colonna, il valore di a aumenta di 40. Aumentare a di 1 corrisponde a uno spostamento in diagonale verso il basso e a destra, continuando nella prima riga in alto, quando si raggiunge l'ultima riga, o nella prima colonna a sinistra, quando si raggiunge l'ultima colonna a destra.

Corollario 31.28

Se $n_1, n_2, \ldots, n_k$ sono relativamente primi a coppie ed $n = n_1 n_2 \cdots n_k$, allora per $a_1, a_2, \ldots, a_k$ interi qualsiasi, il sistema di equazioni

$$x \equiv a_i \pmod{n_i}$$

per $i = 1, 2, \ldots, k$, ha un'unica soluzione modulo n nell'incognita x. ■

Corollario 31.29

Se $n_1, n_2, \ldots, n_k$ sono relativamente primi a coppie ed $n = n_1 n_2 \cdots n_k$, allora per x e a interi qualsiasi, si ha

$$x \equiv a \pmod{n_i}$$

per $i = 1, 2, \ldots, k$ se e soltanto se

$$x \equiv a \pmod{n}$$ ■

Come esempio applicativo del teorema cinese del resto, supponiamo di avere queste due equazioni:

$$\begin{aligned} a &\equiv 2 \pmod 5 \\ a &\equiv 3 \pmod{13} \end{aligned}$$

per cui $a_1 = 2$, $n_1 = m_2 = 5$, $a_2 = 3$ ed $n_2 = m_1 = 13$; vogliamo calcolare a mod 65, in quanto $n = 65$. Poiché $13^{-1} \equiv 2 \pmod 5$ e $5^{-1} \equiv 8 \pmod{13}$, abbiamo

$$\begin{aligned} c_1 &= 13(2 \bmod 5) = 26 \\ c_2 &= 5(8 \bmod 13) = 40 \end{aligned}$$

e

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned} a &\equiv 2 \cdot 26 + 3 \cdot 40 \pmod{65} \\ &\equiv 52 + 120 \pmod{65} \\ &\equiv 42 \pmod{65} \end{aligned}$$

Osservate la Figura 31.3 per un'illustrazione del teorema cinese del resto, modulo 65.

In definitiva, a seconda della convenienza, possiamo operare sia direttamente modulo n sia usando la rappresentazione trasformata ed eseguendo separatamente i calcoli modulo n_i. I calcoli sono del tutto equivalenti.

Esercizi

31.5-1

Trovate tutte le soluzioni delle equazioni $x \equiv 4 \pmod 5$ e $x \equiv 5 \pmod{11}$.

31.5-2

Trovate tutti gli interi x che, quando sono divisi per 9, 8 e 7, danno come resto, rispettivamente, 1, 2 e 3.

31.5-3

Utilizzando le definizioni del Teorema 31.27, dimostrate che, se $\text{MCD}(a, n) = 1$, allora

$$(a^{-1} \bmod n) \leftrightarrow ((a_1^{-1} \bmod n_1), (a_2^{-1} \bmod n_2), \ldots, (a_k^{-1} \bmod n_k))$$

31.5-4

Utilizzando le definizioni del Teorema 31.27, dimostrate che, per ogni polinomio f, il numero di radici dell'equazione $f(x) \equiv 0 \pmod n$ è uguale al prodotto del numero di radici di ciascuna delle equazioni $f(x) \equiv 0 \pmod{n_1}$, $f(x) \equiv 0 \pmod{n_2}$, $\ldots$, $f(x) \equiv 0 \pmod{n_k}$.

31.6 Potenze di un elemento

Proprio come è naturale considerare i multipli di un dato elemento a, modulo n, spesso è naturale considerare la sequenza delle potenze di a, modulo n, dove $a \in \mathbb{Z}_n^*$:

$$a^0, a^1, a^2, a^3, \ldots \tag{31.33}$$

modulo n. Indicizzando a partire da 0, il valore con indice 0 di questa sequenza è $a^0 \bmod n = 1$ e il valore con indice i è $a^i \bmod n$. Per esempio, le potenze di 3 modulo 7 sono

i	0	1	2	3	4	5	6	7	8	9	10	11	$\cdots$
$3^i \bmod 7$	1	3	2	6	4	5	1	3	2	6	4	5	$\cdots$

mentre le potenze di 2 modulo 7 sono

i	0	1	2	3	4	5	6	7	8	9	10	11	$\cdots$
$2^i \bmod 7$	1	2	4	1	2	4	1	2	4	1	2	4	$\cdots$

In questo paragrafo, indichiamo con $\langle a \rangle$ il sottogruppo di $\mathbb{Z}_n^*$ generato da a ripetendo la moltiplicazione e con $\text{ord}_n(a)$ (l'"ordine di a, modulo n") l'ordine di a in $\mathbb{Z}_n^*$. Per esempio, $\langle 2 \rangle = \{1, 2, 4\}$ in $\mathbb{Z}_7^*$ e $\text{ord}_7(2) = 3$. Utilizzando la definizione della funzione $\phi(n)$ di Eulero come dimensione di $\mathbb{Z}_n^*$ (vedere il Paragrafo 31.3), adesso possiamo esprimere il Corollario 31.19 nella notazione di $\mathbb{Z}_n^*$ per ottenere il teorema di Eulero e specializzarlo a $\mathbb{Z}_p^*$, dove p è un numero primo, per ottenere il teorema di Fermat.

Acquistato da Michele Michele su Webster il 2022-05-25 12:20:22 (Vedere-Illeati.it (ita))

Teorema 31.30 (Teorema di Eulero)

Per ogni intero $n > 1$, si ha

$$a^{\phi(n)} \equiv 1 \pmod n \text{ per ogni } a \in \mathbb{Z}_n^* \qquad \blacksquare$$

Teorema 31.31 (Teorema di Fermat)
Se p è un numero primo, allora

$$a^{p-1} \equiv 1 \pmod{p} \text{ per ogni } a \in \mathbb{Z}_p^*$$

Dimostrazione Per l'equazione (31.21), $\phi(p) = p - 1$ se p è primo. ■

Il teorema di Fermat si applica a ogni elemento in $\mathbb{Z}_p$ tranne 0, perché $0 \notin \mathbb{Z}_p^*$. Inoltre, per ogni $a \in \mathbb{Z}_p$, si ha $a^p \equiv a \pmod{p}$ se p è primo.

Se $\text{ord}_n(g) = |\mathbb{Z}_n^*|$, allora ogni elemento in $\mathbb{Z}_n^*$ è una potenza di g, modulo n, e diciamo che g è una ***radice primitiva*** o un ***generatore*** di $\mathbb{Z}_n^*$. Per esempio, 3 è una radice primitiva, modulo 7, mentre 2 non è una radice primitiva, modulo 7. Se $\mathbb{Z}_n^*$ possiede una radice primitiva, il gruppo $\mathbb{Z}_n^*$ è detto ***ciclico***. Omettiamo la dimostrazione del seguente teorema, che è stato dimostrato da Niven e Zuckerman [266].

Teorema 31.32
I valori di $n > 1$ per i quali il gruppo $\mathbb{Z}_n^*$ è ciclico sono 2, 4, p^e e $2p^e$, per tutti i primi $p > 2$ e per tutti gli interi positivi e. ■

Se g è una radice primitiva di $\mathbb{Z}_n^*$ e a è un elemento qualsiasi di $\mathbb{Z}_n^*$, allora esiste un elemento z tale che $g^z \equiv a \pmod{n}$. Questo elemento z è detto ***logaritmo discreto*** o ***indice*** di a, modulo n, in base g; indichiamo questo valore con $\text{ind}_{n,g}(a)$.

Teorema 31.33 (Teorema del logaritmo discreto)
Se g è una radice primitiva di $\mathbb{Z}_n^*$, allora l'equazione $g^x \equiv g^y \pmod{n}$ è vera se e soltanto se è vera l'equazione $x \equiv y \pmod{\phi(n)}$.

Dimostrazione Supponiamo prima che $x \equiv y \pmod{\phi(n)}$. Allora $x = y + k\phi(n)$ per qualche intero k. Quindi, si ha

$$\begin{aligned}
g^x &\equiv g^{y+k\phi(n)} \pmod{n} \\
&\equiv g^y \cdot (g^{\phi(n)})^k \pmod{n} \\
&\equiv g^y \cdot 1^k \pmod{n} \qquad \text{(per il teorema di Eulero)} \\
&\equiv g^y \pmod{n}
\end{aligned}$$

Viceversa, supponiamo che $g^x \equiv g^y \pmod{n}$. Poiché la sequenza delle potenze di g genera ogni elemento di $\langle g \rangle$ e $|\langle g \rangle| = \phi(n)$, il Corollario 31.18 implica che la sequenza delle potenze di g è periodica con periodo $\phi(n)$. Pertanto, se $g^x \equiv g^y \pmod{n}$, allora deve essere $x \equiv y \pmod{\phi(n)}$. ■

Esaminiamo adesso le radici quadrate di 1, modulo una potenza di un numero primo. Il seguente teorema sarà utile per sviluppare un algoritmo per il test di primalità descritto nel Paragrafo 31.8.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Teorema 31.34
Se p è un numero primo dispari ed $e \geq 1$, allora l'equazione

$$x^2 \equiv 1 \pmod{p^e} \tag{31.34}$$

ha due sole soluzioni: $x = 1$ e $x = -1$.

Dimostrazione L'equazione (31.34) è equivalente a

$$p^e \mid (x-1)(x+1)$$

Poiché $p > 2$, possiamo avere $p \mid (x-1)$ o $p \mid (x+1)$, ma non entrambi (altrimenti, per la proprietà (31.3), p dividerebbe anche la loro differenza $(x+1)-(x-1)=2$). Se $p \nmid (x-1)$, allora $\text{MCD}(p^e, x-1) = 1$ e, per il Corollario 31.5, si avrebbe $p^e \mid (x+1)$. Ovvero $x \equiv -1 \pmod{p^e}$. Analogamente, se $p \nmid (x+1)$, allora $\text{MCD}(p^e, x+1) = 1$, e il Corollario 31.5 implica che $p^e \mid (x-1)$, cosicché $x \equiv 1 \pmod{p^e}$. Dunque, si ha $x \equiv -1 \pmod{p^e}$ o $x \equiv 1 \pmod{p^e}$. ■

Un numero x è una ***una radice quadrata non banale di 1, modulo n***, se soddisfa l'equazione $x^2 \equiv 1 \pmod{n}$, ma x non è equivalente a una delle due radici quadrate "banali": 1 e -1, modulo n. Per esempio, 6 è una radice quadrata non banale di 1, modulo 35. Il seguente corollario al Teorema 31.34 sarà utilizzato nella dimostrazione della correttezza della procedura per il test di primalità di Miller-Rabin (Paragrafo 31.8).

Corollario 31.35
Se esiste una radice quadrata non banale di 1, modulo n, allora il numero n è composto.

Dimostrazione Per contrapposizione del Teorema 31.34, se esiste una radice quadrata non banale di 1, modulo n, allora n non può essere un numero primo dispari o una potenza di un numero primo dispari. Se $x^2 \equiv 1 \pmod{2}$, allora $x \equiv 1 \pmod{2}$, e quindi tutte le radici quadrate di 1, modulo 2, sono banali. Quindi, n non può essere primo. Infine, deve essere $n > 1$ affinché possa esistere una radice quadrata non banale di 1. Dunque n deve essere composto. ■

Calcolo delle potenze tramite elevazione al quadrato ripetuta

Un'operazione che si presenta spesso nei calcoli della teoria dei numeri è l'elevazione di un numero a una potenza modulo un altro numero, detta anche ***elevazione a potenza modulare***. Più precisamente, vogliamo un metodo efficiente per calcolare $a^b \bmod n$, dove a e b sono interi non negativi ed n è un intero positivo. L'elevazione a potenza modulare è un'operazione fondamentale in molte routine di test della primalità e nel sistema di crittografia a chiave pubblica RSA. La tecnica dell'***elevazione al quadrato ripetuta*** risolve questo problema in modo efficiente utilizzando la rappresentazione binaria di b.

Sia $\langle b_k, b_{k-1}, \ldots, b_1, b_0 \rangle$ la rappresentazione binaria di b (la rappresentazione binaria è lunga $k+1$ bit, b_k è il bit più significativo e b_0 è il bit meno significativo). La procedura MODULAR-EXPONENTIATION calcola $a^c \bmod n$, aumentando c da 0 a b mediante raddoppi e incrementi.

Il calcolo del quadrato nella riga 6 in ogni iterazione spiega il nome "elevazione al quadrato ripetuta". Per esempio, con $a = 7$, $b = 560$ ed $n = 561$, l'algoritmo calcola la sequenza dei valori modulo 561 indicati nella Figura 31.4; gli esponenti utilizzati sono indicati nella riga c della tabella.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

i	9	8	7	6	5	4	3	2	1	0
b_i	1	0	0	0	1	1	0	0	0	0
c	1	2	4	8	17	35	70	140	280	560
d	7	49	157	526	160	241	298	166	67	1

Figura 31.4 I risultati della procedura MODULAR-EXPONENTIATION quando si calcola $a^b \pmod n$, dove $a = 7$, $b = 560 = \langle 1000110000 \rangle$ ed $n = 561$. I valori sono illustrati dopo ogni esecuzione del ciclo **for**. Il risultato finale è 1.

```
MODULAR-EXPONENTIATION(a, b, n)
 1  c = 0
 2  d = 1
 3  Sia ⟨b_k, b_{k−1}, ..., b_0⟩ la rappresentazione binaria di b
 4  for i = k downto 0
 5      c = 2c
 6      d = (d · d) mod n
 7      if b_i == 1
 8          c = c + 1
 9          d = (d · a) mod n
10  return d
```

La variabile c non è effettivamente richiesta dall'algoritmo, ma è inclusa per semplificare la spiegazione; l'algoritmo mantiene la seguente invariante di ciclo composta da due parti:

Appena prima di ogni iterazione del ciclo **for** (righe 4–9), si ha

1. Il valore di c è uguale al prefisso $\langle b_k, b_{k-1}, \ldots, b_{i+1} \rangle$ della rappresentazione binaria di b;
2. $d = a^c \bmod n$.

Utilizziamo l'invariante di ciclo nel modo seguente:

Inizializzazione: inizialmente $i = k$, quindi il prefisso $\langle b_k, b_{k-1}, \ldots, b_{i+1} \rangle$ è vuoto; questo corrisponde a $c = 0$. Inoltre $d = 1 = a^0 \bmod n$.

Conservazione: indichiamo con c' e d' i valori di c e d alla fine di un'iterazione del ciclo **for** e, quindi, i valori prima della successiva iterazione. Ogni iterazione aggiorna $c' = 2c$ (se $b_i = 0$) o $c' = 2c + 1$ (se $b_i = 1$), quindi c sarà corretto prima della successiva iterazione. Se $b_i = 0$, allora $d' = d^2 \bmod n = (a^c)^2 \bmod n = a^{2c} \bmod n = a^{c'} \bmod n$. Se $b_i = 1$, allora $d' = d^2 a \bmod n = (a^c)^2 a \bmod n = a^{2c+1} \bmod n = a^{c'} \bmod n$. In entrambi i casi, $d = a^c \bmod n$ prima della successiva iterazione.

Conclusione: alla fine $i = -1$. Quindi $c = b$, perché c ha il valore del prefisso $\langle b_k, b_{k-1}, \ldots, b_0 \rangle$ della rappresentazione binaria di b. Dunque $d = a^c \bmod n = a^b \bmod n$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Se gli input a, b ed n sono numeri di β bit, allora il numero totale di operazioni aritmetiche richieste è $O(\beta)$ e il numero totale di operazioni sui bit è $O(\beta^3)$.

Esercizi

31.6-1

Create una tabella che illustra l'ordine di ogni elemento in $\mathbb{Z}_{11}^*$. Selezionate la più piccola radice primitiva g e calcolate una tabella dei valori di $\text{ind}_{11,g}(x)$ per ogni $x \in \mathbb{Z}_{11}^*$.

31.6-2

Create un algoritmo di elevazione a potenza modulare che esamina i bit di b da destra a sinistra, anziché da sinistra a destra.

31.6-3

Se è noto $\phi(n)$, spiegate come calcolare $a^{-1} \bmod n$ per ogni $a \in \mathbb{Z}_n^*$ utilizzando la procedura MODULAR-EXPONENTIATION.

31.7 Crittografia a chiave pubblica RSA

Un sistema di crittografia a chiave pubblica può essere utilizzato per cifrare i messaggi trasmessi tra due parti che comunicano in modo tale che una spia che intercetta i messaggi cifrati non sia in grado di decifrarli. Un sistema di crittografia a chiave pubblica consente anche a una parte di aggiungere alla fine del messaggio elettronico una "firma digitale" che non può essere contraffatta. Tale firma è la versione elettronica di una firma apposta manualmente su un documento di carta. Può essere facilmente controllata da chiunque, non può essere contraffatta da nessuno e perde la sua validità se qualche bit del messaggio viene alterato. La firma digitale, pertanto, fornisce un'autenticazione sia dell'identità del firmatario sia del contenuto del messaggio firmato. È lo strumento perfetto per i contratti d'affari firmati elettronicamente, gli assegni elettronici, gli ordini di acquisto elettronici e altre comunicazioni elettroniche che devono essere autenticate.

Il sistema di crittografia a chiave pubblica RSA si basa sull'enorme differenza tra la facilità di trovare numeri primi grandi e la difficoltà di scomporre in fattori il prodotto di due numeri primi grandi. Il Paragrafo 31.8 descrive una procedura efficiente per trovare numeri primi grandi; il Paragrafo 31.9 discute il problema della fattorizzazione di numeri interi grandi.

Sistemi di crittografia a chiave pubblica

In un sistema di crittografia a chiave pubblica, ciascun partecipante ha una ***chiave pubblica*** e una ***chiave segreta***. Ogni chiave è un pezzo di informazione. Per esempio, nel sistema di crittografia RSA, ogni chiave è formata da una coppia di interi. Tradizionalmente, negli esempi di crittografia sono utilizzati i partecipanti "Alice" e "Bob"; indicheremo le loro chiavi pubbliche e segrete con P_A, S_A per Alice e con P_B, S_B per Bob.

Ciascun partecipante crea la sua chiave pubblica e la sua chiave segreta; mantiene segreta la sua chiave segreta, ma può rivelare la sua chiave pubblica a qualcuno o perfino renderla nota a tutti. In effetti, spesso è comodo assumere che tutte le chiavi pubbliche siano disponibili in una cartella pubblica, in modo che qualsiasi partecipante possa facilmente ottenere la chiave pubblica di qualunque altro partecipante. Le chiavi pubbliche e segrete specificano funzioni che possono essere applicate a qualsiasi messaggio. Indichiamo con $\mathcal{D}$ l'insieme dei messaggi ammissibili. Per esempio, $\mathcal{D}$ potrebbe essere l'insieme di tutte le sequenze di bit di lunghezza finita. Nella formulazione più semplice e originale della crittografia a

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

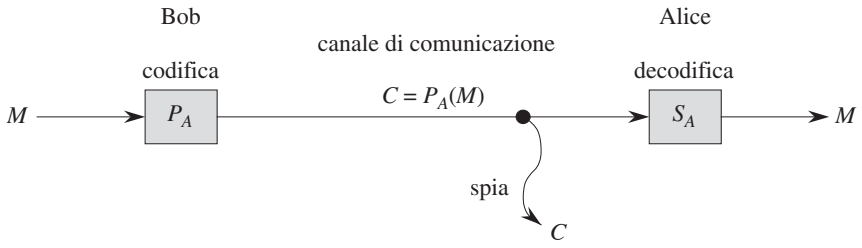


Figura 31.5 Codifica in un sistema a chiave pubblica. Bob codifica il messaggio M utilizzando la chiave pubblica P_A di Alice e trasmette il testo cifrato risultante $C = P_A(M)$ ad Alice. Una spia che intercetta il testo cifrato trasmesso non riesce a ottenere alcuna informazione sul contenuto di M. Alice riceve C e lo decifra utilizzando la sua chiave segreta per ottenere il messaggio originale $M = S_A(C)$.

chiave pubblica, si richiede che le chiavi pubbliche e segrete specifichino funzioni biunivoche da $\mathcal{D}$ a $\mathcal{D}$. La funzione che corrisponde alla chiave pubblica P_A di Alice è indicata con $P_A()$ e la funzione che corrisponde alla sua chiave segreta S_A è indicata con $S_A()$. Le funzioni $P_A()$ ed $S_A()$ sono quindi permutazioni di $\mathcal{D}$. Supponiamo che le funzioni $P_A()$ ed $S_A()$ siano calcolabili in modo efficiente dalle corrispondenti chiavi P_A ed S_A.

Le chiavi pubblica e segreta di un partecipante qualsiasi sono "accoppiate", nel senso che specificano funzioni che sono una l'inversa dell'altra:

$$M = S_A(P_A(M)) \tag{31.35}$$

$$M = P_A(S_A(M)) \tag{31.36}$$

per qualsiasi messaggio $M \in \mathcal{D}$. Trasformando M con le due chiavi P_A ed S_A successivamente, in entrambe le direzioni, si ottiene il messaggio originale M.

In un sistema di crittografia a chiave pubblica, è essenziale che nessuno, tranne Alice, sia in grado di calcolare la funzione $S_A()$ entro una qualsiasi quantità di tempo praticabile. La segretezza della posta che viene codificata e spedita ad Alice e l'autenticità delle firme digitali di Alice si basano sull'ipotesi che soltanto Alice sia in grado di calcolare $S_A()$. Questo requisito è il motivo per cui Alice mantiene segreta la chiave S_A; se non lo facesse, perderebbe la sua unicità e il sistema di crittografia non potrebbe fornirle funzionalità uniche. L'ipotesi che soltanto Alice possa calcolare $S_A()$ deve essere valida anche quando tutti conoscono P_A e possono calcolare con efficienza $P_A()$, la funzione inversa di $S_A()$. La principale difficoltà nella progettazione di un sistema reale di crittografia a chiave pubblica sta nel riuscire a capire come creare un sistema in cui sia possibile rivelare una trasformazione $P_A()$, senza svelare il procedimento per calcolare la corrispondente trasformazione inversa $S_A()$.

In un sistema di crittografia a chiave pubblica, la codifica opera come illustra la Figura 31.5. Supponiamo che Bob voglia inviare ad Alice un messaggio cifrato M, in modo che esso sia incomprensibile per una spia. Lo scenario per inviare il messaggio è il seguente.

- Bob ottiene la chiave pubblica P_A di Alice (da una cartella pubblica o direttamente da Alice).
- Bob calcola il ***testo cifrato*** $C = P_A(M)$ che corrisponde al messaggio M e trasmette C ad Alice.

Acquistato da Michele Michele su Webster / 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

- Quando Alice riceve il testo cifrato C, applica la sua chiave segreta S_A per ottenere il messaggio originale: $S_A(C) = S_A(P_A(M)) = M$.

Poiché $S_A()$ e $P_A()$ sono funzioni inverse, Alice può ottenere M da C. Poiché soltanto Alice è in grado di calcolare $S_A()$, soltanto lei può ottenere M da C. La codifica di M tramite $P_A()$ ha protetto il messaggio M, impedendo che potesse essere letto da altre persone, tranne Alice.

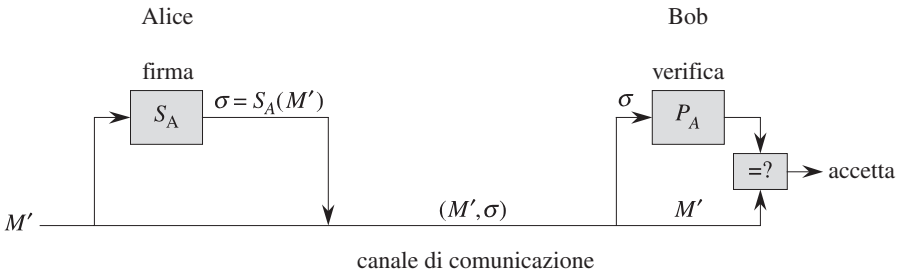


Figura 31.6 Le firme digitali in un sistema a chiave pubblica. Alice firma il messaggio M' aggiungendovi la sua firma digitale $\sigma = S_A(M')$. Trasmette la coppia messaggio/firma (M', σ) a Bob, che la verifica controllando l'equazione $M' = P_A(\sigma)$. Se l'equazione è soddisfatta, accetta (M', σ) come un messaggio che è stato firmato da Alice.

Le firme digitali possono essere facilmente implementate in modo analogo all'interno della nostra formulazione di sistema di crittografia a chiave pubblica (ci sono altre tecniche per realizzare le firme digitali, che non saranno trattate qui). Supponiamo adesso che Alice desideri trasmettere a Bob una risposta M' con una firma digitale. Lo scenario della firma digitale è illustrato nella Figura 31.6.

- Alice calcola la sua ***firma digitale*** σ per il messaggio M' utilizzando la sua chiave segreta S_A e l'equazione $\sigma = S_A(M')$.
- Alice invia la coppia messaggio/firma (M', σ) a Bob.
- Quando Bob riceve la coppia (M', σ), può verificare che essa proviene da Alice utilizzando la chiave pubblica di Alice per verificare se l'equazione $M' = P_A(\sigma)$ è soddisfatta. (Presumibilmente, M' contiene il nome di Alice, quindi Bob conosce quale chiave pubblica utilizzare.) Se l'equazione è soddisfatta, allora Bob conclude che il messaggio M' è stato effettivamente firmato da Alice. Se l'equazione non è soddisfatta, Bob conclude che il messaggio M' o la firma digitale σ sono stati danneggiati da errori di trasmissione o che la coppia (M', σ) è stata contraffatta.

Poiché una firma digitale fornisce sia l'autenticazione dell'identità del firmatario sia l'autenticazione del contenuto del messaggio firmato, essa è analoga a una firma apposta manualmente in fondo a un documento di carta.

Una proprietà importante della firma digitale è che essa è verificabile da chiunque abbia accesso alla chiave pubblica del firmatario. Un messaggio firmato può essere verificato da un partecipante e poi trasmesso ad altri partecipanti che, a loro volta, possono verificare la firma. Per esempio, il messaggio potrebbe essere un assegno elettronico trasmesso da Alice a Bob. Dopo che Bob ha verificato la firma di Alice sull'assegno, può consegnare l'assegno alla sua banca che, a sua volta, può verificare l'autenticità della firma ed effettuare il trasferimento appropriato dei fondi.

È importante notare che un messaggio firmato non è necessariamente cifrato; il messaggio può essere "in chiaro" e quindi non protetto contro lettori non autorizzati. Componendo i precedenti protocolli per la codifica e la firma, possiamo creare messaggi che sono firmati e codificati. Il firmatario prima aggiunge la sua firma digitale al messaggio e poi codifica la coppia messaggio/firma risultante con la chiave pubblica del destinatario; quest'ultimo decifra il messaggio ricevuto con la sua chiave segreta per ottenere sia il messaggio originale sia la firma digitale. Poi, può verificare la firma utilizzando la chiave pubblica del firmatario. Il corrispondente processo combinato che usa i tradizionali sistemi cartacei consiste nel firmare il documento e poi nel sigillarlo all'interno di una busta di carta che viene aperta soltanto dal destinatario indicato nella busta.

Acquistato da Michele Michele su Webster il 2022-[illegible]

Il sistema di crittografia RSA

Nel ***sistema di crittografia a chiave pubblica RSA***, un partecipante crea la chiave pubblica e la chiave segreta con la seguente procedura.

1. Seleziona a caso due numeri primi grandi p e q tali che $p \neq q$. I primi p e q potrebbero essere, per esempio, di 1024 bit ciascuno.
2. Calcola $n = pq$.
3. Seleziona un piccolo numero intero dispari e che sia relativamente primo con $\phi(n)$, che, per l'equazione (31.20), è uguale a $(p-1)(q-1)$.
4. Calcola d come l'inverso moltiplicativo di e, modulo $\phi(n)$. (Il Corollario 31.26 garantisce che d esiste ed è unicamente definito. Possiamo utilizzare la tecnica del Paragrafo 31.4 per calcolare d, noti e e $\phi(n)$.)
5. Pubblica la coppia $P = (e, n)$ come la sua ***chiave pubblica RSA***.
6. Tiene segreta la coppia $S = (d, n)$ come la sua ***chiave segreta RSA***.

Per questo schema, il dominio $\mathcal{D}$ è l'insieme $\mathbb{Z}_n$. La trasformazione di un messaggio M associato a una chiave pubblica $P = (e, n)$ è

$$P(M) = M^e \bmod n \tag{31.37}$$

La trasformazione di un testo cifrato C associato a una chiave segreta $S = (d, n)$ è

$$S(C) = C^d \bmod n \tag{31.38}$$

Queste equazioni si applicano sia alla codifica sia alle firme. Per creare una firma, il firmatario applica la sua chiave segreta al messaggio da firmare, anziché a un testo cifrato. Per verificare una firma, la chiave pubblica del firmatario viene applicata a questa, anziché a un messaggio da codificare.

Le operazioni della chiave pubblica e della chiave segreta possono essere implementate utilizzando la procedura MODULAR-EXPONENTIATION descritta nel Paragrafo 31.6. Per analizzare il tempo di esecuzione di queste operazioni, supponiamo che la chiave pubblica (e, n) e la chiave segreta (d, n) soddisfino le relazioni $\lg e = O(1)$, $\lg d \leq \beta$ e $\lg n \leq \beta$. Allora, l'applicazione di una chiave pubblica richiede $O(1)$ moltiplicazioni modulari e usa $O(\beta^2)$ operazioni sui bit. L'applicazione di una chiave segreta richiede $O(\beta)$ moltiplicazioni modulari, utilizzando $O(\beta^3)$ operazioni sui bit.

Teorema 31.36 (Correttezza del sistema RSA)
Le equazioni (31.37) e (31.38) del sistema RSA definiscono le trasformazioni inverse di $\mathbb{Z}_n$ che soddisfano le equazioni (31.35) e (31.36).

Dimostrazione Dalle equazioni (31.37) e (31.38), per qualsiasi $M \in \mathbb{Z}_n$, si ha

$$P(S(M)) = S(P(M)) = M^{ed} \pmod{n}$$

Poiché e e d sono inversi moltiplicativi modulo $\phi(n) = (p-1)(q-1)$, si ha

$$ed = 1 + k(p-1)(q-1)$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

per qualche intero k. Ma allora, se $M \not\equiv 0 \pmod{p}$, si ha

$$\begin{aligned}
M^{ed} &\equiv M(M^{p-1})^{k(q-1)} && \pmod{p} \\
&\equiv M((M \bmod p)^{p-1})^{k(q-1)} && \pmod{p} \\
&\equiv M(1)^{k(q-1)} && \pmod{p} \qquad \text{(per il Teorema 31.31)} \\
&\equiv M && \pmod{p}
\end{aligned}$$

Inoltre, $M^{ed} \equiv M \pmod{p}$ se $M \equiv 0 \pmod{p}$. Quindi

$$M^{ed} \equiv M \pmod{p}$$

per ogni M. Analogamente, si ha

$$M^{ed} \equiv M \pmod{q}$$

per ogni M. Quindi, per il Corollario 31.29 al teorema cinese del resto, si ha

$$M^{ed} \equiv M \pmod{n}$$

per ogni M. ■

La sicurezza del sistema di crittografia RSA si basa in gran parte sulla difficoltà di fattorizzare i numeri interi grandi. Se un avversario può fattorizzare il modulo n in una chiave pubblica, allora può anche derivare la chiave segreta dalla chiave pubblica, utilizzando la conoscenza dei fattori p e q nello stesso modo in cui li ha utilizzati il creatore della chiave pubblica. Così se la fattorizzazione di numeri interi grandi è facile, allora anche la violazione del sistema RSA è facile. L'asserzione inversa che, se la fattorizzazione di numeri interi grandi è difficile, allora la violazione del sistema RSA è difficile, non è dimostrata.

Dopo un ventennio di ricerche, tuttavia, non è stato ancora trovato un metodo più semplice per violare il sistema di crittografia a chiave pubblica RSA che quello di fattorizzare il modulo n; e come vedremo nel Paragrafo 31.9, la scomposizione in fattori di numeri interi grandi è sorprendentemente difficile. Selezionando a caso due numeri primi di 512 bit e moltiplicandoli, si può creare una chiave pubblica che non può essere "violata" in una quantità di tempo praticabile con l'attuale tecnologia. In assenza di progressi sostanziali nella progettazione degli algoritmi di teoria dei numeri, se implementato con cura applicando gli standard in vigore, il sistema di crittografia RSA è in grado di fornire un elevato livello di sicurezza alle applicazioni.

Al fine di garantire la sicurezza con il sistema di crittografia RSA, tuttavia, è consigliabile operare con numeri interi che sono lunghi centinaia bit, per resistere agli eventuali progressi nell'arte della fattorizzazione. Quando abbiamo scritto questo libro (2001), tipicamente i moduli RSA erano compresi nell'intervallo da 768 a 2048 bit. Per creare moduli di tali dimensioni, dobbiamo essere in grado di trovare con efficienza numeri primi grandi. Il Paragrafo 31.8 tratta questo problema.

Per ragioni di efficienza, il sistema RSA spesso viene utilizzato in una modalità "ibrida" o "a gestione di chiavi" con sistemi veloci di crittografia a chiave non pubblica. Con un tale sistema, le chiavi di codifica e decodifica sono identiche. Se Alice vuole trasmettere privatamente un lungo messaggio M a Bob, seleziona una chiave casuale K per il sistema di crittografia a chiave non pubblica e codifica M tramite K, ottenendo un testo cifrato C. Qui, C ha la stessa lunghezza di M, ma K è una chiave molto piccola. Quindi Alice codifica K utilizzando la chiave pubblica di Bob del sistema RSA. Poiché la chiave K è piccola, il calcolo di $P_B(K)$ è veloce (molto più veloce del calcolo di $P_B(M)$). Alice trasmette $(C, P_B(K))$ a Bob, che decodifica $P_B(K)$ per ottenere K e poi usa K per decodificare C, ottenendo M.

Acquistato da Michele Michele su Webster il 2022-[illegible]

Un analogo approccio ibrido spesso viene utilizzato per creare firme digitali in modo efficiente. In questo approccio, il sistema RSA è combinato con una

funzione hash pubblica anticollisioni h – una funzione che è facile da calcolare, ma per la quale è computazionalmente impraticabile trovare due messaggi M ed M' tali che $h(M) = h(M')$. Il valore $h(M)$ è una piccola (160 bit, per esempio) "impronta digitale" del messaggio M. Se Alice vuole firmare un messaggio M, prima applica h a M per ottenere l'impronta digitale $h(M)$, che poi codifica con la sua chiave segreta; trasmette $(M, S_A(h(M)))$ a Bob come versione firmata del messaggio M. Bob può verificare la firma calcolando $h(M)$ e verificando che applicando la chiave P_A a $S_A(h(M))$ si ottenga $h(M)$. Poiché nessuno può creare due messaggi con la stessa impronta digitale, è computazionalmente impraticabile modificare un messaggio firmato e preservare la validità della firma.

Infine, notiamo che l'uso dei ***certificati*** semplifica notevolmente la distribuzione delle chiavi pubbliche. Per esempio, supponiamo che ci sia un'"autorità affidabile" T la cui chiave pubblica sia nota a tutti. Alice può ottenere da T un messaggio firmato (il suo certificato) che stabilisce che "la chiave pubblica di Alice è P_A". Questo certificato "si autentifica da solo" perché tutti conoscono P_T. Alice può includere il suo certificato nei messaggi firmati, in modo che il destinatario possa disporre immediatamente della chiave pubblica di Alice per verificare la sua firma. Poiché la chiave di Alice è stata firmata da T, il destinatario sa che la chiave di Alice è proprio di Alice.

Esercizi

31.7-1

Considerate un insieme di chiavi RSA con $p = 11$, $q = 29$, $n = 319$ ed $e = 3$. Quale valore di d dovrebbe essere utilizzato nella chiave segreta? Qual è la codifica del messaggio $M = 100$?

31.7-2

Dimostrate che, se l'esponente pubblico e di Alice è 3 e un avversario ottiene l'esponente segreto d di Alice, dove $0 < d < \phi(n)$, allora l'avversario può fattorizzare il modulo n di Alice in tempo polinomiale nel numero di bit di n. (Sebbene non sia richiesta la dimostrazione, tuttavia è interessante sapere che questo risultato resta valido anche se la condizione $e = 3$ viene rimossa. Consultate Miller [256].)

31.7-3 ⋆

Dimostrate che il sistema RSA è moltiplicativo nel senso che

$$P_A(M_1)P_A(M_2) \equiv P_A(M_1M_2) \pmod{n}$$

Utilizzate questo fatto per dimostrare che, se un avversario avesse una procedura in grado di decodificare in modo efficiente l'1% dei messaggi in $\mathbb{Z}_n$ codificati con P_A, allora potrebbe impiegare un algoritmo probabilistico per decodificare tutti i messaggi codificati con P_A con una probabilità elevata.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022. McGraw-Hill Education (Italy)

⋆ 31.8 Test di primalità

In questo paragrafo, analizzeremo il problema di trovare i numeri primi grandi. Inizieremo con una discussione sulla densità dei numeri primi, poi esamineremo un approccio plausibile (ma incompleto) al test di primalità e, infine, presenteremo un effettivo test randomizzato di primalità dovuto a Miller e Rabin.

La densità dei numeri primi

Per molte applicazioni (come la crittografia) è necessario trovare numeri primi grandi "casuali". Fortunatamente, i numeri primi grandi non sono troppo rari, cosicché non occorre troppo tempo per verificare dei numeri interi casuali della dimensione appropriata fino a trovare un numero primo. La ***funzione di distribuzione dei numeri primi*** $\pi(n)$ specifica quanti numeri primi sono minori o uguali a n. Per esempio, $\pi(10) = 4$, perché ci sono 4 numeri primi minori o uguali a 10, ovvero 2, 3, 5 e 7. Il teorema dei numeri primi offre un'utile approssimazione di $\pi(n)$.

Teorema 31.37 (Teorema dei numeri primi)

$$\lim_{n\to\infty} \frac{\pi(n)}{n/\ln n} = 1$$ ■

L'approssimazione $n/\ln n$ fornisce una stima ragionevolmente accurata di $\pi(n)$ anche per piccoli valori di n. Per esempio, c'è uno scarto minore del 6% per $n = 10^9$, con $\pi(n) = 50\,847\,534$ ed $n/\ln n \approx 48\,254\,942$. (Per un teorico dei numeri, 10^9 è un numero piccolo.)

Possiamo considerare il processo di scegliere a caso un intero n e di determinare se esso è un numero primo come una prova ripetuta di Bernoulli (vedere il Paragrafo C.4). Per il teorema dei numeri primi, la probabilità che un intero n scelto a caso sia un numero primo può essere stimata pari a $1/\ln n$. La distribuzione geometrica ci dice quante prove occorrono per ottenere un successo e, per l'equazione (C.32), il numero previsto di prove è approssimativamente $\ln n$. Quindi, dovremmo esaminare approssimativamente $\ln n$ interi scelti a caso prossimi a n per poter trovare un numero primo che abbia la stessa lunghezza di n. Per esempio, trovare un numero primo di 1024 bit potrebbe richiedere la verifica di primalità per circa $\ln 2^{1024} \approx 710$ numeri di 1024 bit scelti a caso (questo numero potrebbe essere dimezzato se scegliessimo soltanto i numeri interi dispari).

Nel resto di questo paragrafo esamineremo il problema di determinare se un grande numero intero dispari n sia primo oppure no. Per comodità di notazione, supponiamo che n abbia la seguente scomposizione in fattori:

$$n = p_1^{e_1} p_2^{e_2} \cdots p_r^{e_r} \tag{31.39}$$

Dove $r \geq 1$, $p_1, p_2, \ldots, p_r$ sono i fattori primi di n ed $e_1, e_2, \ldots, e_r$ sono interi positivi. Naturalmente, n è primo se e soltanto se $r = 1$ ed $e_1 = 1$.

Un semplice approccio al problema della verifica della primalità è la ***divisione di prova***. Proviamo a dividere n per ciascun intero $2, 3, \ldots, \lfloor\sqrt{n}\rfloor$ (gli interi pari maggiori di 2 possono essere ignorati). È facile capire che n è primo se e soltanto se nessuno dei divisori di prova divide n. Supponendo che ciascuna divisione di prova richieda un tempo costante, il tempo di esecuzione nel caso peggiore è $\Theta(\sqrt{n})$, che è esponenziale nella lunghezza di n. (Ricordiamo che se n è codificato in binario tramite β bit, allora $\beta = \lceil \lg(n+1) \rceil$, e quindi $\sqrt{n} = \Theta(2^{\beta/2})$.)

Acquistato da Michele Michele su Webster il 2022-05-20 21:21:11 Copyright © 2022 McGraw-Hill Education

Di conseguenza, la divisione di prova funziona bene soltanto se n è molto piccolo o ha un fattore primo piccolo. Quando funziona, la divisione di prova offre il vantaggio non soltanto di determinare se n è primo o composto, ma anche di determinare uno dei fattori primi di n, se n è composto.

In questo paragrafo siamo interessati soltanto a sapere se un dato numero n è primo; se n è composto, non siamo interessati a scomporlo in fattori primi. Come

vedremo nel Paragrafo 31.9, scomporre un numero in fattori primi è un'operazione computazionalmente costosa. Forse sorprenderà il fatto che è molto più semplice dire se un dato numero è primo che determinare la scomposizione in fattori primi del numero, se non è primo.

Test di pseudoprimalità

Consideriamo adesso un metodo di verifica della primalità che è "quasi esatto", in quanto è sufficientemente buono per molte applicazioni pratiche. Più avanti presenteremo un affinamento di questo metodo che ne elimina i piccoli difetti. Indichiamo con $\mathbb{Z}_n^+$ gli elementi non nulli di $\mathbb{Z}_n$:

$$\mathbb{Z}_n^+ = \{1, 2, \ldots, n-1\}$$

Se n è primo, allora $\mathbb{Z}_n^+ = \mathbb{Z}_n^*$.

Diciamo che n è uno ***pseudoprimo in base a*** se n è composto e

$$a^{n-1} \equiv 1 \pmod{n} \tag{31.40}$$

Il teorema di Fermat (Teorema 31.31) implica che se n è primo, allora n soddisfa l'equazione (31.40) per ogni a in $\mathbb{Z}_n^+$. Quindi, se possiamo trovare qualche $a \in \mathbb{Z}_n^+$ tale che n *non* soddisfi l'equazione (31.40), allora n è certamente composto. Sorprendentemente, il viceversa è *quasi* vero, cosicché questo criterio costituisce un test di primalità quasi perfetto. Il test serve a stabilire se n soddisfa l'equazione (31.40) per $a = 2$. Se non la soddisfa, dichiariamo n composto; altrimenti, proviamo a supporre che n sia primo (quando, in effetti, tutto ciò che sappiamo è che n o è primo o è uno pseudoprimo in base 2).

La seguente procedura simula in questo modo la verifica della primalità di n; usa la procedura MODULAR-EXPONENTIATION descritta nel Paragrafo 31.6. Si suppone che l'input n sia un intero dispari maggiore di 2.

```
PSEUDOPRIME(n)
1  if MODULAR-EXPONENTIATION(2, n − 1, n) ≢ 1 (mod n)
2        return COMPOSTO        // Esatto.
3  else return PRIMO            // Forse.
```

Questa procedura può commettere errori, ma di un solo tipo. Ovvero, se la procedura dice che n è composto, allora ciò è sempre esatto. Se invece dice che n è primo, allora commette un errore soltanto se n è uno pseudoprimo in base 2.

Quante volte sbaglia questa procedura? Raramente. Ci sono soltanto 22 valori di n minori di 10 000 per i quali sbaglia; i primi quattro di tali valori sono 341, 561, 645 e 1105. È possibile dimostrare che la probabilità che questo programma commetta un errore con un numero scelto a caso di β bit tende a zero per $\beta \to \infty$. Utilizzando una stima più precisa (dovuta a Pomerance [280]) del numero di pseudoprimi in base 2 di una data dimensione, è possibile prevedere che un numero di 512 bit scelto a caso, che è dichiarato primo dalla precedente procedura, ha meno di una probabilità su 10^{20} di essere uno pseudoprimo in base 2, e che un numero di 1024 bit scelto a caso, che è dichiarato primo, ha meno di una probabilità su 10^{41} di essere uno pseudoprimo in base 2.

Acquistato da Michele Mastrogiovanni - Vietata la riproduzione - Copyright © 2022, McGraw-Hill Education (Italy)

Pertanto, se state semplicemente tentando di trovare un numero primo grande per qualche applicazione, in pratica non sbaglierete quasi mai se sceglierete numeri grandi a caso, finché uno di essi non indurrà la procedura PSEUDOPRIME a restituire PRIMO come output. Ma se i numeri che sono oggetto del test di

primalità non sono scelti a caso, occorre un approccio migliore per verificare la primalità. Come vedremo, con un po' più di intelligenza e una certa dose di randomizzazione, saremo in grado di realizzare una routine per il test di primalità che funziona bene con tutti gli input.

Sfortunatamente, non possiamo eliminare completamente tutti gli errori controllando semplicemente l'equazione (31.40) per un secondo numero di base, per esempio $a = 3$, perché ci sono numeri interi composti n che soddisfano l'equazione (31.40) per *ogni* $a \in \mathbb{Z}_n^*$. Questi interi sono detti ***numeri di Carmichael***. I primi tre numeri di Carmichael sono 561, 1105 e 1729. I numeri di Carmichael sono estremamente rari; per esempio, ce ne sono soltanto 255 minori di 100 000 000. L'Esercizio 31.8-2 aiuta a capire perché questi numeri sono così rari.

Adesso spieghiamo come migliorare il nostro test di primalità in modo che non sia tratto in inganno dai numeri di Carmichael.

Test randomizzato di primalità di Miller-Rabin

Il test di primalità di Miller-Rabin supera i problemi del semplice test PSEUDOPRIME grazie a due modifiche:

- Prova diversi valori scelti a caso per la base a, anziché un solo valore.
- Mentre calcola ciascuna potenza modulare, rileva la presenza di una radice quadrata non banale di 1, modulo n, durante le ultime elevazioni al quadrato. Se tale radice esiste, la procedura si ferma e restituisce COMPOSTO come output. Il Corollario 31.35 (Paragrafo 31.6) giustifica questa tecnica di identificazione dei numeri composti.

Lo pseudocodice per il test di primalità di Miller-Rabin è riportato più avanti. L'input $n > 2$ è il numero dispari sottoposto al test di primalità ed s è il numero dei valori scelti a caso in $\mathbb{Z}_n^+$ che saranno provati come base. Il codice usa il generatore di numeri casuali RANDOM descritto a pagina 97: RANDOM$(1, n-1)$ restituisce un intero a scelto a caso che soddisfa la relazione $1 \le a \le n-1$. Il codice usa una procedura ausiliaria WITNESS tale che WITNESS(a, n) è TRUE se e soltanto se a è un "testimone" del fatto che n è un numero composto – ovvero, se è possibile utilizzare a per provare (in un modo che vedremo) che n è composto. Il test WITNESS(a, n) è un'estensione più efficace del test

$$a^{n-1} \not\equiv 1 \pmod{n}$$

che aveva formato la base (con $a = 2$) per la procedura PSEUDOPRIME. Innanzi tutto, presenteremo e giustificheremo la struttura della procedura WITNESS; poi spiegheremo come viene utilizzata nel test di primalità di Miller-Rabin.

```
WITNESS(a, n)
1  Siano t e u tali che t ≥ 1, u è dispari ed n − 1 = 2^t u
2  x_0 = MODULAR-EXPONENTIATION(a, u, n)
3  for i = 1 to t
4      x_i = x_{i−1}^2 mod n
5      if x_i == 1 and x_{i−1} ≠ 1 and x_{i−1} ≠ n − 1
6          return TRUE
7  if x_t ≠ 1
8      return TRUE
9  return FALSE
```

Acquistato da Michele Michele su Webster il 2022-07-07 21:12. Numero ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Sia $n - 1 = 2^t u$, con $t \geq 1$ e u dispari; ovvero la rappresentazione binaria di $n - 1$ è la rappresentazione binaria del numero intero dispari u seguita esattamente da t zeri. Pertanto $a^{n-1} \equiv (a^u)^{2^t} \pmod{n}$, cosicché possiamo calcolare $a^{n-1} \bmod n$ calcolando prima $a^u \bmod n$ e poi elevando al quadrato il risultato t volte consecutive.

Lo pseudocodice di WITNESS calcola $a^{n-1} \bmod n$, determinando prima il valore $x_0 = a^u \bmod n$ nella riga 2 e poi elevando al quadrato il risultato t volte in una riga del ciclo **for** (righe 3–6). Per induzione su i, la sequenza $x_0, x_1, \ldots, x_t$ dei valori calcolati soddisfa l'equazione $x_i \equiv a^{2^i u} \pmod{n}$ per $i = 0, 1, \ldots, t$, cosicché in particolare $x_t \equiv a^{n-1} \pmod{n}$. Ogni volta che viene eseguito un passo di elevazione al quadrato nella riga 4, però, il ciclo potrebbe terminare prematuramente se le righe 5–6 rilevano che è stata scoperta una radice quadrata non banale di 1. (Spiegheremo tra poco questo test.) In questo caso, l'algoritmo finisce restituendo TRUE. Le righe 7–8 restituiscono TRUE se il valore calcolato per $x_t \equiv a^{n-1} \pmod{n}$ è diverso da 1, esattamente come la procedura PSEUDOPRIME restituisce COMPOSTO in questo caso. La riga 9 restituisce FALSE se la riga 6 o la riga 8 non ha restituito TRUE.

Adesso dimostriamo che, se WITNESS(a, n) restituisce TRUE, allora una prova che n è un numero composto può essere costruita utilizzando a.

Se la procedura WITNESS restituisce TRUE nella riga 8, allora ha scoperto che $x_t = a^{n-1} \bmod n \neq 1$. Se n è primo, però, per il teorema di Fermat (Teorema 31.31) si ha che $a^{n-1} \equiv 1 \pmod{n}$ per ogni $a \in \mathbb{Z}_n^+$. Quindi n non può essere primo e l'equazione $a^{n-1} \bmod n \neq 1$ è una prova di questo fatto.

Se la procedura WITNESS restituisce TRUE nella riga 6, allora ha scoperto che x_{i-1} è una radice quadrata non banale di $x_i = 1$, modulo n, perché si ha $x_{i-1} \not\equiv \pm 1 \pmod{n}$ e $x_i \equiv x_{i-1}^2 \equiv 1 \pmod{n}$. Il Corollario 31.35 stabilisce che soltanto se n è un numero composto ci può essere una radice quadrata non banale di 1 modulo n, quindi una dimostrazione che x_{i-1} non è una radice quadrata banale di 1 modulo n è una prova che n è composto.

Questo completa la dimostrazione della correttezza della procedura WITNESS. Se la chiamata WITNESS(a, n) genera TRUE come output, allora n è sicuramente composto; una prova che n è composto può essere facilmente determinata da a ed n.

A questo punto presentiamo brevemente una descrizione alternativa del comportamento di WITNESS come una funzione della sequenza $X = \langle x_0, x_1, \ldots, x_t \rangle$, che potrà risultare utile più avanti, quando analizzeremo l'efficienza del test di primalità di Miller-Rabin. Notate che, se $x_i = 1$ per qualche $0 \leq i < t$, la procedura WITNESS potrebbe non calcolare la parte restante della sequenza. Se però lo facesse tutti i valori $x_{i+1}, x_{i+2}, \ldots, x_t$ sarebbero uguali a 1; nella sequenza X noi consideriamo anche tutti questi elementi uguali a 1. Si hanno quattro casi:

1. $X = \langle \ldots, d \rangle$, dove $d \neq 1$: la sequenza X non finisce in 1. La procedura restituisce TRUE; a è un testimone che n è un numero composto (per il teorema di Fermat).

Acquistato da Michele Michela su Webstar il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

2. $X = \langle 1, 1, \ldots, 1 \rangle$: tutti gli elementi della sequenza X sono 1. La procedura restituisce FALSE; a non è un testimone che n è un numero composto.
3. $X = \langle \ldots, -1, 1, \ldots, 1 \rangle$: la sequenza X finisce in 1 e l'ultimo elemento diverso da 1 è uguale a -1. La procedura restituisce FALSE; a non è un testimone che n è un numero composto.

4. $X = \langle \ldots, d, 1, \ldots, 1 \rangle$, dove $d \neq \pm 1$: la sequenza X finisce in 1, ma l'ultimo elemento diverso da 1 non è -1. La procedura restituisce TRUE; a è un testimone che n è un numero composto, perché d è una radice quadrata non banale di 1.

Adesso esaminiamo il test di primalità di Miller-Rabin che si basa sulla procedura WITNESS. Ancora una volta supponiamo che n sia un numero intero dispari maggiore di 2.

```
MILLER-RABIN(n, s)
1  for j = 1 to s
2      a = RANDOM(1, n − 1)
3      if WITNESS(a, n)
4          return COMPOSTO          // Esatto.
5  return PRIMO                     // Quasi certamente.
```

La procedura MILLER-RABIN è una ricerca probabilistica di una prova che n sia un numero composto. Il ciclo principale (che inizia nella riga 1) seleziona s valori casuali di a da $\mathbb{Z}_n^+$ (riga 2). Se uno dei valori a selezionati testimonia che n è composto, allora MILLER-RABIN restituisce come output COMPOSTO nella riga 4. Tale output è sempre esatto, per la correttezza di WITNESS. Se non viene trovato alcun testimone durante le s prove, la procedura MILLER-RABIN presume che ciò sia dovuto al fatto che non ci sono testimoni da trovare e, quindi, suppone che n sia primo. Vedremo che questo output probabilmente è corretto se s è sufficientemente grande, ma esiste una piccola probabilità che la procedura possa essere sfortunata quando sceglie i valori di a e che i testimoni esistano anche quando non vengono trovati.

Per descrivere il funzionamento di MILLER-RABIN, indichiamo con n il numero di Carmichael 561, cosicché $n - 1 = 560 = 2^4 \cdot 35$. Supponendo che $a = 7$ sia scelto come base, la Figura 31.4 mostra che WITNESS calcola $x_0 \equiv a^{35} \equiv 241 \pmod{561}$ e quindi calcola la sequenza $X = \langle 241, 298, 166, 67, 1 \rangle$. Pertanto, una radice quadrata non banale di 1 viene scoperta nell'ultimo passo di elevazione al quadrato, in quanto $a^{280} \equiv 67 \pmod{n}$ e $a^{560} \equiv 1 \pmod{n}$. Di conseguenza, $a = 7$ è un testimone che n è un numero composto, WITNESS$(7, n)$ restituisce TRUE e MILLER-RABIN restituisce COMPOSTO.

Se n è un numero di β bit, la procedura MILLER-RABIN richiede $O(s\beta)$ operazioni aritmetiche e $O(s\beta^3)$ operazioni sui bit, perché asintoticamente non richiede più lavoro di s elevazioni a potenza modulare.

Tasso di errore del test di primalità di Miller-Rabin

Se la procedura MILLER-RABIN restituisce PRIMO come output, allora c'è una piccola probabilità che abbia commesso un errore. Tuttavia, diversamente dalla procedura PSEUDOPRIME, la probabilità di errore non dipende da n: non ci sono input cattivi per questa procedura. Tale probabilità, invece, dipende dalla dimensione di s e dalla "fortuna" quando vengono scelti i valori a per la base. Inoltre, poiché ogni test è più rigoroso di un semplice controllo dell'equazione (31.40), possiamo prevedere che in linea di principio il tasso di errore sia piccolo per numeri interi n scelti a caso. Il seguente teorema presenta un'argomentazione più precisa.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1994030010220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Teorema 31.38
Se n è un numero composto dispari, allora il numero di testimoni del fatto che n è composto è almeno pari a $(n-1)/2$.

Dimostrazione Per dimostrare il teorema, proveremo che il numero di non testimoni è al più $(n-1)/2$.

Iniziamo osservando che qualsiasi non testimone deve essere un elemento di $\mathbb{Z}_n^*$. Perché? Consideriamo un non testimone qualsiasi a; deve soddisfare la relazione $a^{n-1} \equiv 1 \pmod{n}$ ovvero $a \cdot a^{n-2} \equiv 1 \pmod{n}$. Quindi esiste una soluzione dell'equazione $ax \equiv 1 \pmod{n}$, per esempio a^{n-2}. Per il Corollario 31.21, $\text{MCD}(a, n) \mid 1$, che a sua volta implica che $\text{MCD}(a, n) = 1$. Di conseguenza, a è un elemento di $\mathbb{Z}_n^*$; tutti i non testimoni appartengono a $\mathbb{Z}_n^*$.

Per completare la dimostrazione, proviamo che non soltanto tutti i non testimoni sono contenuti in $\mathbb{Z}_n^*$, ma che essi sono tutti contenuti in un sottogruppo proprio B di $\mathbb{Z}_n^*$ (ricordiamo che B è un *sottogruppo proprio* di $\mathbb{Z}_n^*$ quando B è un sottogruppo di $\mathbb{Z}_n^*$, ma è diverso da $\mathbb{Z}_n^*$). Per il Corollario 31.16, si ha $|B| \leq |\mathbb{Z}_n^*|/2$. Poiché $|\mathbb{Z}_n^*| \leq n-1$, otteniamo $|B| \leq (n-1)/2$. Pertanto il numero di non testimoni è al più $(n-1)/2$, e quindi il numero di testimoni è almeno $(n-1)/2$.

Vediamo ora come trovare un sottogruppo proprio B di $\mathbb{Z}_n^*$ che contiene tutti i non testimoni. Consideriamo due casi.

Caso 1: esiste un elemento $x \in \mathbb{Z}_n^*$ tale che

$$x^{n-1} \not\equiv 1 \pmod{n}$$

In altre parole, n non è un numero di Carmichael. Come detto in precedenza, poiché i numeri di Carmichael sono estremamente rari, il caso 1 è il caso principale che si presenta "nella pratica" (per esempio, quando il numero n viene scelto a caso e deve essere verificata la sua primalità).

Sia $B = \{b \in \mathbb{Z}_n^* : b^{n-1} \equiv 1 \pmod{n}\}$. Chiaramente, B non è vuoto, perché $1 \in B$. Poiché B è chiuso rispetto alla moltiplicazione modulo n, si ha che B è un sottogruppo di $\mathbb{Z}_n^*$ per il Teorema 31.14. Notate che tutti i non testimoni appartengono a B, perché un non testimone a soddisfa la relazione $a^{n-1} \equiv 1 \pmod{n}$. Poiché $x \in \mathbb{Z}_n^* - B$, si ha che B è un sottogruppo proprio di $\mathbb{Z}_n^*$.

Caso 2: per ogni $x \in \mathbb{Z}_n^*$, si ha

$$x^{n-1} \equiv 1 \pmod{n} \tag{31.41}$$

In altre parole, n è un numero di Carmichael. Questo caso è estremamente raro nella pratica. Tuttavia, il test di Miller-Rabin (diversamente da un test di pseudoprimalità) può determinare con efficienza che i numeri di Carmichael sono composti, come vedremo qui di seguito.

In questo caso, n non può essere la potenza di un numero primo. Per capire perché, supponiamo per assurdo che $n = p^e$, dove p è un numero primo ed $e > 1$. Otterremo una contraddizione. Poiché n per ipotesi è dispari, anche p deve essere dispari. Il Teorema 31.32 implica che $\mathbb{Z}_n^*$ è un gruppo ciclico: contiene un generatore g tale che $\text{ord}_n(g) = |\mathbb{Z}_n^*| = \phi(n) = p^e(1 - 1/p) = (p-1)p^{e-1}$. Per l'equazione (31.41), si ha $g^{n-1} \equiv 1 \pmod{n}$. Allora il teorema del logaritmo discreto (Teorema 31.33, prendendo $y = 0$) implica che $n - 1 \equiv 0 \pmod{\phi(n)}$, ovvero

$$(p-1)p^{e-1} \mid p^e - 1$$

Acquistato da Michele Michelazzo (Website) il 2022-07-07 22:12:13 Non-commercial Online Library License #9950016.220707.0 Copyright © 2022 McGraw-Hill Education (Italy)

Questa è una contraddizione per $e > 1$, perché $(p-1)p^{e-1}$ è divisibile per il numero primo p, mentre $p^e - 1$ non lo è. Dunque, n non è una potenza di un numero primo.

Poiché il numero primo dispari n non è una potenza di un numero primo, scomponiamolo in un prodotto $n_1 n_2$, dove n_1 ed n_2 sono numeri dispari maggiori di 1 che sono primi tra loro. (Ci sono vari modi per fare questo e non è importante quale si sceglie. Per esempio, se $n = p_1^{e_1} p_2^{e_2} \cdots p_r^{e_r}$, allora si può scegliere $n_1 = p_1^{e_1}$ ed $n_2 = p_2^{e_2} p_3^{e_3} \cdots p_r^{e_r}$.)

Ricordiamo che abbiamo definito t e u in modo che $n - 1 = 2^t u$, con $t \geq 1$ e u dispari, e che per un input a, la procedura WITNESS calcola la sequenza

$$X = \langle a^u, a^{2u}, a^{2^2 u}, \ldots, a^{2^t u} \rangle$$

(tutti i calcoli sono operazioni modulo n).

Una coppia (v, j) di interi è detta ***accettabile*** se $v \in \mathbb{Z}_n^*$, $j \in \{0, 1, \ldots, t\}$, e

$$v^{2^j u} \equiv -1 \pmod{n}$$

Le coppie accettabili certamente esistono perché u è dispari; possiamo scegliere $v = n - 1$ e $j = 0$, in modo che $(n-1, 0)$ sia una coppia accettabile. Adesso selezioniamo il numero più grande possibile j tale che esista una coppia accettabile (v, j); fissiamo v in modo che (v, j) sia una coppia accettabile. Sia

$$B = \{x \in \mathbb{Z}_n^* : x^{2^j u} \equiv \pm 1 \pmod{n}\}$$

Poiché B è chiuso rispetto alla moltiplicazione modulo n, esso è un sottogruppo di $\mathbb{Z}_n^*$. Quindi, per il Corollario 31.16, $|B|$ divide $|\mathbb{Z}_n^*|$. Ogni non testimone deve essere un elemento di B, perché la sequenza X prodotta da un non testimone deve essere composta da elementi tutti pari a 1 oppure deve contenere un elemento -1 non oltre la j-esima posizione, perché j è massimo. (Se (a, j') è una coppia accettabile, dove a non è un testimone, deve essere $j' \leq j$, per come abbiamo scelto j.)

Utilizziamo adesso l'esistenza di v per dimostrare che esiste un elemento $w \in \mathbb{Z}_n^* - B$. Poiché $v^{2^j u} \equiv -1 \pmod{n}$, si ha $v^{2^j u} \equiv -1 \pmod{n_1}$ per il Corollario 31.29 al teorema cinese del resto. Per il Corollario 31.28, esiste un elemento w che soddisfa contemporaneamente le equazioni

$$\begin{aligned} w &\equiv v \pmod{n_1} \\ w &\equiv 1 \pmod{n_2} \end{aligned}$$

Di conseguenza, si ha

$$\begin{aligned} w^{2^j u} &\equiv -1 \pmod{n_1} \\ w^{2^j u} &\equiv 1 \pmod{n_2} \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-27 18:12:00 Copyright © 2010 The McGraw-Hill Companies, Inc. All rights reserved.

Per il Corollario 31.29, $w^{2^j u} \not\equiv 1 \pmod{n_1}$ implica $w^{2^j u} \not\equiv 1 \pmod{n}$ e $w^{2^j u} \not\equiv -1 \pmod{n_2}$ implica $w^{2^j u} \not\equiv -1 \pmod{n}$. Allora $w^{2^j u} \not\equiv \pm 1 \pmod{n}$ e quindi $w \notin B$.

Resta da dimostrare che $w \in \mathbb{Z}_n^*$; lo faremo operando prima con modulo n_1 e poi con modulo n_2. Operando con modulo n_1, osserviamo che, poiché $v \in \mathbb{Z}_n^*$, si ha $\text{MCD}(v, n) = 1$, e così anche $\text{MCD}(v, n_1) = 1$; se v non ha alcun divisore comune con n, non ha certamente alcun divisore comune con n_1. Poiché $w \equiv v \pmod{n_1}$, notiamo che $\text{MCD}(w, n_1) = 1$. Operando con modulo n_2,

osserviamo che $w \equiv 1 \pmod{n_2}$ implica $\text{MCD}(w, n_2) = 1$. Per combinare questi risultati, utilizziamo il Teorema 31.6, che implica che $\text{MCD}(w, n_1 n_2) = \text{MCD}(w, n) = 1$; cioè $w \in \mathbb{Z}_n^*$.

Pertanto $w \in \mathbb{Z}_n^* - B$; concludiamo il caso 2 dicendo che B è un sottogruppo proprio di $\mathbb{Z}_n^*$.

In entrambi i casi, notiamo che il numero di testimoni del fatto che n è composto è almeno pari a $(n-1)/2$. ■

Teorema 31.39

Per $n > 2$ intero dispari qualsiasi ed s intero positivo, la probabilità che MILLER-RABIN(n, s) commetta un errore è al più 2^{-s}.

Dimostrazione Utilizzando il Teorema 31.38, notiamo che se n è composto, allora ogni esecuzione del ciclo **for** (righe 1–4) ha una probabilità almeno pari a $1/2$ di identificare un testimone x che n è composto. La procedura MILLER-RABIN commette un errore soltanto se è così sfortunata da non riuscire a scoprire un testimone in una delle s iterazioni del ciclo principale. La probabilità di una tale sequenza di insuccessi è al più 2^{-s}. ■

Se n è un numero primo, MILLER-RABIN restituisce sempre PRIMO, e se n è composto, la probabilità che MILLER-RABIN restituisca PRIMO è al più 2^{-s}.

Tuttavia, quando applichiamo la procedura MILLER-RABIN a un numero intero n scelto a caso, dobbiamo considerare anche la probabilità che n sia primo, per potere interpretare correttamente il risultato della procedura MILLER-RABIN. Supponiamo di fissare una lunghezza di bit β e di scegliere a caso un numero intero n di lunghezza β bit per verificarne la primalità. Indichiamo con A l'evento che n sia primo. Per il teorema dei numeri primi (Teorema 31.37), la probabilità che n sia primo è approssimativamente

$$\begin{aligned} \Pr\{A\} &\approx 1/\ln n \\ &\approx 1.443/\beta \end{aligned}$$

Adesso indichiamo con B l'evento che MILLER-RABIN restituisca PRIMO. Abbiamo $\Pr\{\overline{B} \mid A\} = 0$ (ovvero $\Pr\{B \mid A\} = 1$) e $\Pr\{B \mid \overline{A}\} \leq 2^{-s}$ (ovvero $\Pr\{\overline{B} \mid \overline{A}\} > 1 - 2^{-s}$).

Ma quanto vale $\Pr\{A \mid B\}$, la probabilità che n sia primo, supponendo che MILLER-RABIN restituisca PRIMO? Per la forma alternativa del teorema di Bayes (equazione (C.18)), si ha

$$\begin{aligned} \Pr\{A \mid B\} &= \frac{\Pr\{A\}\Pr\{B \mid A\}}{\Pr\{A\}\Pr\{B \mid A\} + \Pr\{\overline{A}\}\Pr\{B \mid \overline{A}\}} \\ &\approx \frac{1}{1 + 2^{-s}(\ln n - 1)} \end{aligned}$$

Questa probabilità non supera $1/2$ finché s è maggiore di $\lg(\ln n - 1)$. Intuitivamente, tutte queste prove iniziali sono richieste semplicemente perché la fiducia che deriva dal non riuscire a trovare un testimone del fatto che n sia composto possa screditare l'ipotesi iniziale che n sia composto. Per un numero con $\beta = 1024$ bit, questo test iniziale richiede approssimativamente un numero di prove pari a

$$\begin{aligned} \lg(\ln n - 1) &\approx \lg(\beta/1.443) \\ &\approx 9 \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 :Numero Ordine Libreria: 199503016:220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

In ogni caso, la scelta di s pari a 50 dovrebbe essere sufficiente per quasi tutte le applicazioni immaginabili.

In effetti, la situazione è molto migliore. Se stiamo tentando di trovare numeri primi grandi applicando MILLER-RABIN a numeri interi dispari grandi *scelti a caso*, allora scegliendo un piccolo valore di s (3, per esempio), è molto improbabile ottenere risultati errati (anche se non lo dimostreremo qui). In altre parole, se n è un intero composto dispari scelto a caso, il numero atteso di non testimoni del fatto che n sia composto è probabile che sia molto più piccolo di $(n-1)/2$. Se l'intero n non è scelto a caso, però, il meglio che possiamo dimostrare è che il numero di non testimoni è al più $(n-1)/4$, utilizzando una versione migliorata del Teorema 31.38. Inoltre, esistono interi n per i quali il numero di non testimoni è $(n-1)/4$.

Esercizi

31.8-1

Dimostrate che, se un intero dispari $n > 1$ non è un numero primo o la potenza di un numero primo, allora esiste una radice quadrata non banale di 1 modulo n.

31.8-2 ⋆

È possibile rafforzare un po' il teorema di Eulero nella forma seguente:

$a^{\lambda(n)} \equiv 1 \pmod{n}$ per ogni $a \in \mathbb{Z}_n^*$

dove $n = p_1^{e_1} \cdots p_r^{e_r}$ e

$$\lambda(n) = \text{mcm}(\phi(p_1^{e_1}), \ldots, \phi(p_r^{e_r})) \tag{31.42}$$

Dimostrate che $\lambda(n) \mid \phi(n)$. Un numero composto n è un numero di Carmichael se $\lambda(n) \mid n-1$. Il più piccolo numero di Carmichael è $561 = 3 \cdot 11 \cdot 17$; qui, $\lambda(n) = \text{mcm}(2, 10, 16) = 80$, che divide 560. Dimostrate che i numeri di Carmichael devono soddisfare due condizioni: non devono essere divisibili per il quadrato di un numero primo qualsiasi e devono essere il prodotto di almeno tre numeri primi. Per questo motivo, non sono molto comuni.

31.8-3

Dimostrate che, se x è una radice quadrata non banale di 1, modulo n, allora $\text{MCD}(x-1, n)$ e $\text{MCD}(x+1, n)$ sono entrambi divisori non banali di n.

⋆ 31.9 Scomposizione di un numero intero in fattori primi

Supponiamo di avere un numero intero n che vogliamo ***fattorizzare***, cioè scomporre in un prodotto di numeri primi. Il test di primalità del precedente paragrafo ci potrebbe dire che n è composto, ma di solito non fornisce i fattori primi di n. Fattorizzare un intero n grande sembra essere molto più difficile che determinare semplicemente se n è un numero primo o composto. Attualmente è impossibile con i moderni supercalcolatori e i migliori algoritmi fattorizzare un numero arbitrario di 1024 bit.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995U3016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Euristica del rho di Pollard

La divisione di prova per tutti gli interi fino a B garantisce di fattorizzare completamente qualsiasi numero fino a B^2. Con la stessa quantità di lavoro, la seguente procedura fattorizza qualsiasi numero fino a B^4 (a meno che non siamo sfortuna-

ti). Poiché la procedura è soltanto un'euristica, non sono garantiti né il suo tempo di esecuzione né il suo successo, tuttavia la procedura è molto efficace nella pratica. Un altro vantaggio della procedura POLLARD-RHO è che usa soltanto un numero costante di locazioni di memoria. (È facile implementare POLLARD-RHO su una calcolatrice programmabile per trovare i fattori di piccoli numeri.)

```
POLLARD-RHO(n)
 1  i = 1
 2  x_1 = RANDOM(0, n − 1)
 3  y = x_1
 4  k = 2
 5  while TRUE
 6      i = i + 1
 7      x_i = (x_{i−1}^2 − 1) mod n
 8      d = MCD(y − x_i, n)
 9      if d ≠ 1 and d ≠ n
10          stampa d
11      if i == k
12          y = x_i
13          k = 2k
```

La procedura funziona nel modo seguente. Le righe 1–2 inizializzano i con 1 e x_1 con un valore scelto a caso in $\mathbb{Z}_n$. Il ciclo **while**, che inizia nella riga 5, si ripete all'infinito per cercare i fattori di n. Durante ogni iterazione del ciclo **while**, viene utilizzata la ricorrenza

$$x_i = (x_{i-1}^2 - 1) \bmod n \tag{31.43}$$

nella riga 7 per produrre il successivo valore di x_i nella sequenza infinita

$$x_1, x_2, x_3, x_4, \ldots \tag{31.44}$$

Il valore di i viene corrispondentemente aumentato nella riga 6. Il codice è scritto utilizzando variabili con pedici x_i per chiarezza, ma il programma funziona lo stesso se vengono eliminati tutti i pedici, perché soltanto il più recente valore di x_i deve essere mantenuto. Con questa modifica, la procedura usa soltanto un numero costante di locazioni di memoria.

Ogni tanto il programma salva l'ultimo valore generato x_i nella variabile y. Più precisamente, i valori salvati sono quelli i cui pedici sono potenze di 2:

$$x_1, x_2, x_4, x_8, x_{16}, \ldots$$

La riga 3 salva il valore x_1; la riga 12 salva x_k se i è uguale a k. La variabile k è inizializzata a 2 nella riga 4; il valore di k viene raddoppiato nella riga 13 se il valore di y viene aggiornato. Quindi, k segue la sequenza $1, 2, 4, 8, \ldots$ e fornisce sempre il pedice del successivo valore x_k da salvare in y.

Acquistato da Michele Michelone Webster il 2022-07-07 22:13 Numero Ordine Libraria 1395563 0-320707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Le righe 8–10 tentano di trovare un fattore di n, utilizzando il valore salvato di y e il valore corrente di x_i. In particolare, la riga 8 calcola il massimo comun divisore $d = \text{MCD}(y - x_i, n)$. Se d è un divisore non banale di n (il controllo avviene nella riga 9), allora la riga 10 stampa d.

Questa procedura per trovare un fattore, inizialmente, può sembrare misteriosa. Notate però che POLLARD-RHO non stampa mai una risposta sbagliata; ogni numero che stampa è un divisore non banale di n. Ma POLLARD-RHO potrebbe

non stampare nulla; non ci sono garanzie che produrrà qualche risultato. Tuttavia, vedremo che c'è una buona ragione per aspettarsi che POLLARD-RHO stampi un fattore p di n dopo $\Theta(\sqrt{p})$ iterazioni del ciclo **while**. Quindi, se n è composto, possiamo prevedere che questa procedura scopra un numero di divisori sufficiente per fattorizzare completamente n dopo circa $n^{1/4}$ aggiornamenti, in quanto ogni fattore primo p di n, tranne eventualmente il più grande, è minore di $\sqrt{n}$.

Iniziamo la nostra analisi del comportamento di questa procedura studiando il tempo che impiega una sequenza casuale modulo n per ripetere un valore. Poiché $\mathbb{Z}_n$ è finito e poiché ogni valore nella sequenza (31.44) dipende soltanto dal precedente valore, la sequenza (31.44) a un certo punto si ripete. Una volta raggiunto un elemento x_i tale che $x_i = x_j$ per qualche $j < i$, ci troviamo in un ciclo, perché $x_{i+1} = x_{j+1}$, $x_{i+2} = x_{j+2}$ e così via. La ragione del nome "euristica del rho" è che, come illustra la Figura 31.7, la sequenza $x_1, x_2, \ldots, x_{j-1}$ può essere disegnata come la "coda" del simbolo ρ (rho) e il ciclo $x_j, x_{j+1}, \ldots, x_i$ come il "corpo" di ρ.

Consideriamo la questione del tempo che occorre alla sequenza di x_i per ripetersi. Questo non è esattamente ciò che ci serve, ma vedremo più avanti come modificare l'argomento della discussione.

Per effettuare questa stima, supponiamo che la funzione

$$f_n(x) = (x^2 - 1) \bmod n$$

si comporti come una funzione "casuale". Ovviamente, non si tratta di una funzione effettivamente casuale, ma questa ipotesi ci consente di ottenere risultati coerenti con il comportamento osservato della procedura POLLARD-RHO. Quindi possiamo considerare che ciascun elemento x_i sia stato scelto in modo indipendente da $\mathbb{Z}_n$ secondo una distribuzione uniforme in $\mathbb{Z}_n$. Dall'analisi del paradosso del compleanno (Paragrafo 5.4.1) si deduce che il numero atteso di passi eseguiti prima che la sequenza entri in ciclo è $\Theta(\sqrt{n})$.

Vediamo ora la modifica richiesta. Sia p un fattore non banale di n tale che $\text{MCD}(p, n/p) = 1$. Per esempio, se n ha la fattorizzazione $n = p_1^{e_1} p_2^{e_2} \cdots p_r^{e_r}$, allora possiamo assumere p pari a $p_1^{e_1}$. (Se $e_1 = 1$, allora p è proprio il più piccolo fattore primo di n, un buon esempio da ricordare.)

La sequenza $\langle x_i \rangle$ induce una sequenza corrispondente $\langle x_i' \rangle$ modulo p, dove

$$x_i' = x_i \bmod p$$

per ogni i.

Inoltre, poiché f_n è definita utilizzando soltanto operazioni aritmetiche (elevazione al quadrato e sottrazione) modulo n, vedremo che è possibile calcolare x_{i+1}' da x_i'; la vista "modulo p" della sequenza è una versione ridotta di ciò che sta accadendo con il modulo n:

$$\begin{aligned}
x_{i+1}' &= x_{i+1} \bmod p \\
&= f_n(x_i) \bmod p \\
&= ((x_i^2 - 1) \bmod n) \bmod p \\
&= (x_i^2 - 1) \bmod p && \text{(per l'Esercizio 31.1-7)} \\
&= ((x_i \bmod p)^2 - 1) \bmod p \\
&= ((x_i')^2 - 1) \bmod p \\
&= f_p(x_i')
\end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Licenza: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

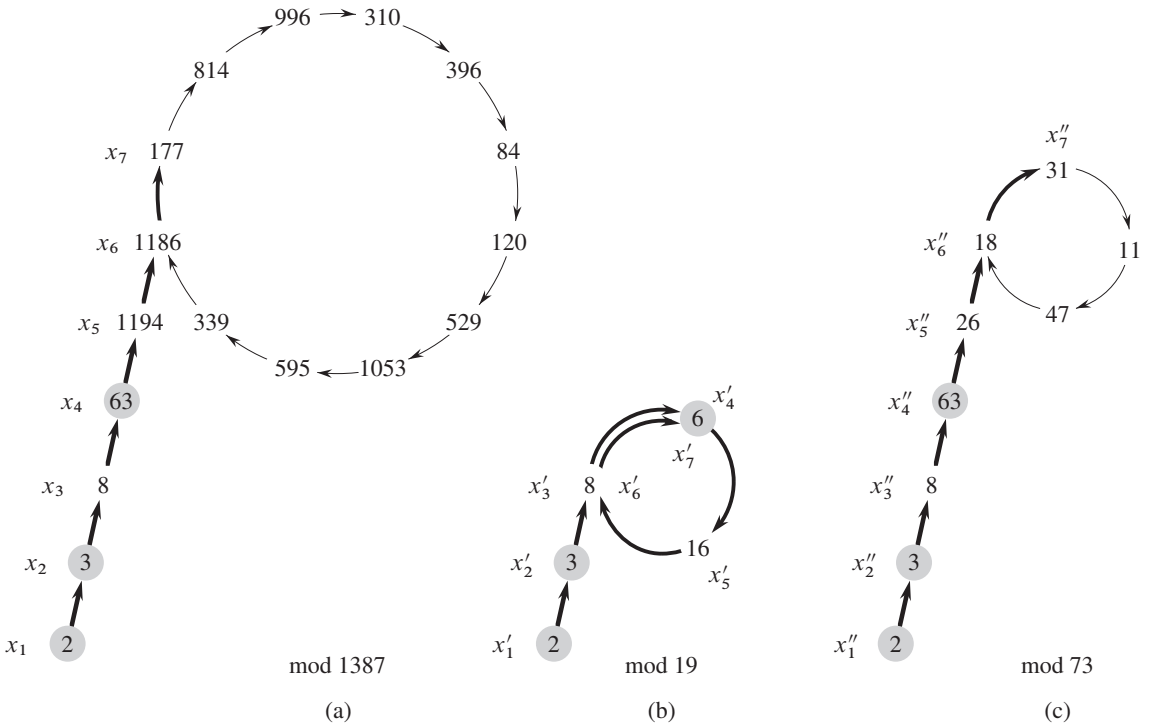


Figura 31.7 Euristica del rho di Pollard. **(a)** I valori prodotti dalla ricorrenza $x_{i+1} = (x_i^2 - 1) \bmod 1387$, a partire da $x_1 = 2$. La scomposizione in fattori primi di 1387 è $19 \cdot 73$. Le frecce più spesse indicano i passi di iterazione che sono eseguiti prima che sia scoperto il fattore 19. Le frecce meno spesse puntano a valori non raggiunti nell'iterazione, per illustrare la forma del simbolo ρ (rho). I valori ombreggiati sono i valori y memorizzati da POLLARD-RHO. Il fattore 19 viene scoperto dopo che è stato raggiunto $x_7 = 177$, quando viene calcolato $\text{MCD}(63 - 177, 1387) = 19$. Il primo valore x che verrebbe ripetuto è 1186, ma il fattore 19 viene scoperto prima che tale valore venga ripetuto. **(b)** I valori prodotti dalla stessa ricorrenza, modulo 19. Ogni valore x_i dato nella parte (a) è congruo, modulo 19, al valore x'_i qui illustrato. Per esempio, $x_4 = 63$ e $x_7 = 177$ sono entrambi congrui a 6, modulo 19. **(c)** I valori prodotti dalla stessa ricorrenza, modulo 73. Ogni valore x_i dato nella parte (a) è congruo, modulo 73, al valore x''_i qui illustrato. Per il teorema cinese del resto, ogni nodo nella parte (a) corrisponde a una coppia di nodi, uno nella parte (b) e l'altro nella parte (c).

Quindi, sebbene non stiamo calcolando esplicitamente la sequenza $\langle x'_i \rangle$, questa sequenza è ben definita e soddisfa la stessa ricorrenza della sequenza $\langle x_i \rangle$.

Ragionando come prima, notiamo che il numero atteso di passi prima che la sequenza $\langle x'_i \rangle$ si ripeta è $\Theta(\sqrt{p})$. Se p è piccolo rispetto a n, la sequenza $\langle x'_i \rangle$ può ripetersi molto più rapidamente della sequenza $\langle x_i \rangle$. Infatti, la sequenza $\langle x'_i \rangle$ si ripete non appena due elementi della sequenza $\langle x_i \rangle$ sono semplicemente congrui modulo p, anziché modulo n. Un esempio è illustrato nelle parti (b) e (c) della Figura 31.7.

Acquistato da Michele Michele su Wiley | 2022-07-01 14:12 | Numero Utente Wiley: 1192-0120-2207-0 | Copyright © 2022, McGraw-Hill Education (Italy)

Indichiamo con t l'indice del primo valore ripetuto nella sequenza $\langle x'_i \rangle$ e con $u > 0$ la lunghezza del ciclo che è stato prodotto in questo modo. Ovvero, t e $u > 0$ sono i più piccoli valori tali che $x'_{t+i} = x'_{t+u+i}$ per ogni $i \geq 0$. In base alle precedenti argomentazioni, i valori attesi di t e u sono entrambi $\Theta(\sqrt{p})$. Notate che se $x'_{t+i} = x'_{t+u+i}$, allora $p \mid (x_{t+u+i} - x_{t+i})$. Quindi, $\text{MCD}(x_{t+u+i} - x_{t+i}, n) > 1$.

Di conseguenza, una volta che POLLARD-RHO ha salvato in y un valore x_k tale che $k \geq t$, allora $y \bmod p$ è sempre nel ciclo modulo p (se viene salvato un nuovo valore in y, anche questo valore è nel ciclo modulo p). Alla fine, k assume un valore che è maggiore di u e la procedura quindi effettua un giro completo attorno al ciclo modulo p senza cambiare il valore di y. Viene scoperto un fattore di n quando x_i "incontra" il valore precedentemente memorizzato di y, modulo p, cioè quando $x_i \equiv y \pmod{p}$.

Presumibilmente, il fattore trovato è p, sebbene saltuariamente possa accadere che venga scoperto un multiplo di p. Poiché i valori attesi di t e u sono $\Theta(\sqrt{p})$, il numero atteso di passi richiesti per produrre il fattore p è $\Theta(\sqrt{p})$.

Ci sono due ragioni per le quali questo algoritmo potrebbe non comportarsi come previsto. In primo luogo, l'analisi euristica del tempo di esecuzione non è rigorosa ed è possibile che il ciclo di valori, modulo p, possa essere molto più grande di $\sqrt{p}$. In questo caso, l'algoritmo funziona correttamente, ma molto più lentamente di quanto richiesto. Nella pratica l'effetto di questo problema appare dubbio. In secondo luogo, i divisori di n prodotti da questo algoritmo potrebbero essere sempre uguali a uno dei fattori banali 1 e n. Per esempio, supponiamo che $n = pq$, dove p e q sono primi. Potrebbe accadere che i valori di t e u per p siano uguali ai valori di t e u per q, e quindi il fattore p viene sempre scoperto nella stessa operazione MCD che scopre il fattore q. Poiché entrambi i fattori vengono identificati contemporaneamente, viene scoperto il fattore banale $pq = n$, che è inutile. Anche questo problema sembra essere insignificante nella pratica. Se necessario, l'euristica può essere riavviata con una diversa ricorrenza della forma $x_{i+1} = (x_i^2 - c) \bmod n$ (i valori $c = 0$ e $c = 2$ devono essere evitati per motivi che non possiamo spiegare qui, ma altri valori vanno bene).

Ovviamente, questa analisi è euristica e non rigorosa, perché la ricorrenza non è effettivamente "casuale". Ciononostante, la procedura si comporta bene in pratica e sembra avere la stessa efficienza indicata dall'analisi euristica. È il metodo che viene scelto per trovare piccoli fattori primi di un numero grande. Per fattorizzare completamente un numero n composto di β bit, bisogna trovare soltanto tutti i fattori primi minori di $\lfloor n^{1/2} \rfloor$, quindi prevediamo che POLLARD-RHO richieda al più $n^{1/4} = 2^{\beta/4}$ operazioni aritmetiche e al più $n^{1/4}\beta^2 = 2^{\beta/4}\beta^2$ operazioni sui bit. La capacità di POLLARD-RHO di trovare un piccolo fattore p di n con un numero atteso $\Theta(\sqrt{p})$ di operazioni aritmetiche spesso è la sua caratteristica più attraente.

Esercizi

31.9-1

Facendo riferimento alla storia dell'esecuzione della procedura POLLARD-RHO illustrata nella Figura 31.7(a), quando viene stampato il fattore 73 di 1387?

31.9-2

Supponendo di conoscere una funzione $f : \mathbb{Z}_n \to \mathbb{Z}_n$ e un valore iniziale $x_0 \in \mathbb{Z}_n$, definite $x_i = f(x_{i-1})$ per $i = 1, 2, \ldots$. Siano t e $u > 0$ i più piccoli valori tali che $x_{t+i} = x_{t+u+i}$ per $i = 0, 1, \ldots$. Nella terminologia dell'algoritmo del rho di Pollard, t è la lunghezza della coda del rho e u è la lunghezza del ciclo del rho. Create un algoritmo efficiente per calcolare t e u in modo esatto e analizzate il suo tempo di esecuzione.

Acquistato da Michele Michele su Webster il 2022-07-12 12:22 Numero Ordine Libreria: 1995036 - 2207-01 Copyright © 2022, McGraw-Hill Education (Italy)

31.9-3

Quanti passi prevedete che debba eseguire la procedura POLLARD-RHO per scoprire un fattore della forma p^e, dove p è primo ed $e > 1$?

31.9-4 ⋆

Uno svantaggio della procedura POLLARD-RHO, così come è stata scritta, è che richiede un calcolo del MCD per ogni passo della ricorrenza. Qualcuno ha suggerito che tutti i calcoli del MCD potrebbero essere raggruppati, calcolando il prodotto di più valori x_i consecutivi e poi utilizzando questo prodotto al posto di x_i nel calcolo del MCD. Descrivete attentamente come potrebbe essere implementata questa idea, spiegate perché potrebbe funzionare e indicate quale dimensione dei gruppi scegliereste come la più efficiente per operare con un numero n di β bit.

Problemi

31-1 Algoritmo del MCD binario

In molti calcolatori le operazioni di sottrazione, controllo della parità (pari o dispari) di un intero binario e divisione per 2 possono essere eseguite più rapidamente del calcolo dei resti. Questo problema esamina l'***algoritmo del MCD binario***, che evita i calcoli dei resti utilizzati nell'algoritmo di Euclide.

a. Dimostrate che, se a e b sono pari, allora $\text{MCD}(a, b) = 2\,\text{MCD}(a/2, b/2)$.

b. Dimostrate che, se a è dispari e b è pari, allora $\text{MCD}(a, b) = \text{MCD}(a, b/2)$.

c. Dimostrate che, se a e b sono dispari, allora $\text{MCD}(a, b) = \text{MCD}((a-b)/2, b)$.

d. Progettate un algoritmo efficiente che calcola il MCD binario dei numeri interi a e b, dove $a \geq b$, nel tempo $O(\lg a)$. Supponete che ogni operazione di sottrazione, controllo di parità e divisione per 2 possa essere eseguita nell'unità di tempo.

31-2 Analisi delle operazioni sui bit nell'algoritmo di Euclide

a. Considerate il tipico algoritmo "carta e penna" per eseguire la divisione lunga: dividere a per b, ottenendo un quoziente q e un resto r. Dimostrate che questo metodo richiede $O((1 + \lg q) \lg b)$ operazioni sui bit.

b. Definite $\mu(a, b) = (1 + \lg a)(1 + \lg b)$. Dimostrate che il numero di operazioni sui bit eseguite dall'algoritmo EUCLID per ridurre il problema del calcolo di $\text{MCD}(a, b)$ a quello del calcolo di $\text{MCD}(b, a \bmod b)$ è al più $c(\mu(a, b) - \mu(b, a \bmod b))$ per qualche costante $c > 0$ sufficientemente grande.

c. Dimostrate che $\text{EUCLID}(a, b)$ richiede $O(\mu(a, b))$ operazioni sui bit in generale e $O(\beta^2)$ operazioni sui bit quando opera su due input di β bit.

31-3 Tre algoritmi per i numeri di Fibonacci

Questo problema confronta l'efficienza di tre metodi per calcolare l'n-esimo numero di Fibonacci F_n, essendo noto n. Supponete che il costo per sommare, sottrarre o moltiplicare due numeri sia $O(1)$, indipendentemente dalla dimensione dei numeri.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

a. Dimostrate che il tempo di esecuzione del metodo ricorsivo semplice per calcolare F_n basato sulla ricorrenza (3.22) è esponenziale in n.

b. Spiegate come calcolare F_n nel tempo $O(n)$ annotando i valori calcolati.

c. Spiegate come calcolare F_n nel tempo $O(\lg n)$ utilizzando soltanto l'addizione e la moltiplicazione di interi. (*Suggerimento:* considerate la matrice

$$\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$$

e le sue potenze.)

d. Supponete adesso che sommare due numeri di β bit richieda un tempo $\Theta(\beta)$ e che moltiplicare due numeri di β bit richieda un tempo $\Theta(\beta^2)$. Qual è il tempo di esecuzione di questi tre metodi con questa misura più ragionevole dei costi delle operazioni aritmetiche elementari?

31-4 Residuo quadratico

Sia p un numero primo dispari. Un numero $a \in Z_p^*$ è un ***residuo quadratico*** se l'equazione $x^2 = a \pmod{p}$ ha una soluzione per l'incognita x.

a. Dimostrate che ci sono esattamente $(p-1)/2$ residui quadratici, modulo p.

b. Se p è un numero primo, definiamo il ***simbolo di Legendre*** $(\frac{a}{p})$, per $a \in \mathbb{Z}_p^*$, pari a 1 se a è un residuo quadratico modulo p, pari a -1 negli altri casi. Dimostrate che, se $a \in \mathbb{Z}_p^*$, allora

$$\left(\frac{a}{p}\right) \equiv a^{(p-1)/2} \pmod{p}$$

Create un algoritmo efficiente per determinare se un dato numero a è un residuo quadratico modulo p. Analizzate l'efficienza del vostro algoritmo.

c. Dimostrate che, se p è un numero primo della forma $4k + 3$ e a è un residuo quadratico in $\mathbb{Z}_p^*$, allora $a^{k+1} \bmod p$ è una radice quadrata di a, modulo p. Quanto tempo occorre per calcolare la radice quadrata di un residuo quadratico a modulo p?

d. Descrivete un algoritmo randomizzato efficiente per calcolare un residuo non quadratico, modulo un primo arbitrario p, cioè un elemento di $\mathbb{Z}_p^*$ che non è un residuo quadratico. Quante operazioni aritmetiche dovrà eseguire in media il vostro algoritmo?

Note

Niven e Zuckerman [266] hanno fatto un'eccellente introduzione alla teoria elementare dei numeri. Knuth [211] include una buona analisi degli algoritmi che calcolano il massimo comun divisore e di altri importanti algoritmi della teoria dei numeri. Bach [30] e Riesel [296] presentano gli studi più recenti sulla teoria computazionale dei numeri. Dixon [92] fornisce una panoramica delle tecniche di fattorizzazione e dei test di primalità. Gli atti della conferenza pubblicati da Pomerance [281] contengono numerosi articoli interessanti sulla teoria dei numeri. Più recentemente, Bach e Shallit [31] hanno realizzato un'eccellente sintesi dei concetti fondamentali della teoria computazionale dei numeri.

continua

Acquistato da Michele Bellia (michele.bellia@gmail.com) - Copyright © The McGraw-Hill Companies, S.r.l.

Knuth [211] discute le origini dell'algoritmo di Euclide, che appare nel Libro 7, Proposizioni 1 e 2, degli *Elementi* del matematico greco Euclide, scritto intorno al 300 a.C. La descrizione di Euclide potrebbe essere stata presa da un algoritmo attribuito a Eudosso intorno al 375 a.C. L'algoritmo di Euclide potrebbe avere l'onore di essere il più antico algoritmo non banale; il suo unico rivale è un algoritmo per la moltiplicazione che era conosciuto all'epoca degli antichi egizi. Shallit [313] ha narrato la storia dell'analisi dell'algoritmo di Euclide.

Knuth attribuisce un caso speciale del teorema cinese del resto (Teorema 31.27) al matematico cinese Sun-Tsŭ, che visse tra il 200 a.C. e il 200 d.C. – la data è molto incerta. Lo stesso caso speciale fu descritto dal matematico greco Nicomaco intorno al 100 d.C. Fu generalizzato da Chhin Chiu-Shao nel 1247. Infine, il teorema cinese del resto fu definito e dimostrato nella sua completa generalità da L. Eulero nel 1734.

L'algoritmo randomizzato per il test di primalità che abbiamo presentato in questo capitolo è stato ideato da Miller [256] e Rabin [290]; attualmente è l'algoritmo più veloce, a meno di fattori costanti. La dimostrazione del Teorema 31.39 è un piccola variante di quella suggerita da Bach [29]. Una dimostrazione di un risultato più forte per MILLER-RABIN è stata fatta da Monier [259, 260]. Per molti anni il test di primalità è stato un esempio classico in cui la randomizzazione sembrava necessaria per ottenere un algoritmo efficiente (con tempo polinomiale). Nel 2002, Agrawal, Kayal e Saxema [4] hanno sorpreso tutti con il loro algoritmo deterministico in tempo polinomiale. Fino a quel momento il miglior algoritmo deterministico conosciuto per il test di primalità, dovuto a Coen e Lenstra [74], richiedeva tempo $(\lg n)^{O(\lg\lg\lg n)}$ sull'input n, che è appena un po' superpolinomiale. Nondimeno, ai fini pratici gli algoritmi randomizzati per il test di primalità risultano più efficienti e quindi da preferire.

Il problema di trovare numeri primi grandi "a caso" è descritto molto bene in un articolo di Beauchemin, Brassard, Crépeau, Goutier e Pomerance [36].

Il concetto di sistema di crittografia a chiave pubblica è attribuito a Diffie e Hellman [88]. Il sistema di crittografia RSA fu proposto nel 1977 da Rivest, Shamir e Adleman [297]. Da allora, il campo della crittografia ha iniziato a fiorire. La conoscenza del sistema RSA è progressivamente migliorata e le moderne implementazioni hanno affinato notevolmente le tecniche di base presentate in questo capitolo. Inoltre, sono state sviluppate molte nuove tecniche per provare la sicurezza dei sistemi di crittografia. Per esempio, Goldwasser e Micali [143] hanno dimostrato che la randomizzazione può essere uno strumento efficace per progettare schemi sicuri di codifica a chiave pubblica. Per quanto riguarda le firme digitali, Goldwasser, Micali e Rivest [144] hanno presentato uno schema di firme digitali per cui ogni tipo di contraffazione concepibile si è dimostrato altrettanto difficile quanto la fattorizzazione. Menezes e altri autori [255] hanno fatto una panoramica sulle applicazioni della crittografia.

L'euristica del rho per la fattorizzazione degli interi è stata inventata da Pollard [278]. La versione presentata in questo capitolo è una variante proposta da Brent [57].

Gli algoritmi migliori per fattorizzare i numeri grandi hanno un tempo di esecuzione che cresce quasi esponenzialmente con la radice cubica della lunghezza del numero n da fattorizzare. L'algoritmo di fattorizzazione con il crivello generale per campi numerici, sviluppato da Buhler e altri autori [58] come un'estensione delle idee nell'algoritmo di fattorizzazione con il crivello per campi numerici di Pollard [279] e Lenstra e altri [233] e affinato da Coppersmith [78] e altri, è forse in generale il più efficiente di tali algoritmi per input grandi. Sebbene sia difficile fare un'analisi rigorosa di questo algoritmo, sotto ragionevoli ipotesi, possiamo stimare un tempo di esecuzione pari a $L(1/3, n)^{1.902+o(1)}$, dove $L(\alpha, n) = e^{(\ln n)^{\alpha}(\ln\ln n)^{1-\alpha}}$.

Acquistato da Michele Michelozzi Webster (2022-07-07 22:41 Numero Ordine Interno 183503216, 220707) Condivisione © 2022 McGraw-Hill Education (Italy)

Il metodo delle curve ellittiche dovuto a Lenstra [234] può essere più efficiente per qualche input rispetto al metodo del crivello per campi numerici, perché, come il metodo del rho di Pollard, è in grado di trovare molto rapidamente un piccolo fattore primo p. Con questo metodo, il tempo per trovare p è stimato pari a $L(1/2, p)^{\sqrt{2}+o(1)}$.

32 String matching

32.1 Introduzione

Trovare tutte le occorrenze di una stringa di caratteri all'interno di un testo è un problema che si presenta spesso nelle applicazioni di elaborazione dei testi. Tipicamente, il testo è un documento da modificare e la stringa cercata (o *pattern*) è una particolare parola fornita dall'utente. Algoritmi efficienti per questo problema – detto "string matching" – possono migliorare significativamente la reattività dei programmi di elaborazione dei testi. Tra le loro molte applicazioni gli algoritmi di string matching vengono usati anche per cercare particolari pattern nelle sequenze del DNA. Anche i motori di ricerca li usano per cercare le pagine web rilevanti per le interrogazioni.

Formalizziamo il ***problema dello string matching*** nel modo seguente. Supponiamo che il testo sia un array $T[1..n]$ di lunghezza n e che il pattern sia un array $P[1..m]$ di lunghezza $m \leq n$. Supponiamo inoltre che gli elementi di P e T siano caratteri appartenenti a un alfabeto finito Σ. Per esempio, potremmo avere $\Sigma = \{0, 1\}$ o $\Sigma = \{a, b, \ldots, z\}$. Gli array di caratteri P e T sono spesso chiamati ***stringhe*** di caratteri.

Facendo riferimento alla Figura 32.1, diremo che il pattern P ***occorre con uno spostamento s*** nel testo T (ovvero il pattern P ***si trova a partire dalla posizione s + 1*** nel testo T) se $0 \leq s \leq n - m$ e $T[s + 1..s + m] = P[1..m]$ (cioè se $T[s + j] = P[j]$, per $1 \leq j \leq m$). Se P occorre con spostamento s in T, allora s è uno ***spostamento valido***, altrimenti s non è uno spostamento valido. Il problema dello string matching consiste nel trovare tutti gli spostamenti validi con i quali un dato pattern P occorre in un dato testo T.

Ad eccezione dell'algoritmo ingenuo di forza bruta, che esamineremo nel Paragrafo 32.2, tutti gli algoritmi di string matching descritti in questo capitolo eseguono qualche preelaborazione basata sul pattern e poi trovano tutti gli spostamenti validi; quest'ultima fase è detta "matching". La Figura 32.2 indica i tempi di preelaborazione e di matching per ciascuno degli algoritmi di questo capitolo. Il tempo di esecuzione totale di ogni algoritmo è la somma del tempo di preelaborazione e del tempo di matching. Il Paragrafo 32.3 presenta un interessante algoritmo di string matching, dovuto a Rabin e Karp. Sebbene il tempo di esecuzione $\Theta((n - m + 1)m)$ nel caso peggiore dell'algoritmo di Rabin-Karp non sia migliore di quello del metodo ingenuo, questo algoritmo funziona molto meglio nel caso medio e nella pratica. Inoltre l'algoritmo di Rabin-Karp può essere appropriatamente generalizzato per risolvere altri problemi di pattern matching su stringhe.

Acquistato da Michele Michele su Webster il 2022-02-23 (Numero ordine: primeria) - ISBN 9788891221234 - Copyright © 2021 The MIT Press (Italy)

Il Paragrafo 32.4 descrive un algoritmo di string matching che inizia costruendo un automa a stati finiti appositamente progettato per ricercare le occorrenze di un dato pattern P in un testo. Questo algoritmo ha un tempo di preelabora-

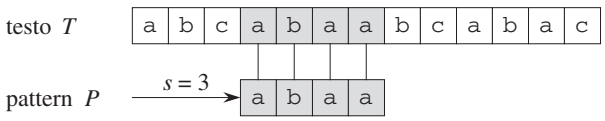


Figura 32.1 Il problema dello string matching. L'obiettivo è trovare tutte le occorrenze del pattern $P = \texttt{abaa}$ nel testo $T = \texttt{abcabaabcabac}$. Il pattern occorre soltanto una volta nel testo con spostamento $s = 3$, che è uno spostamento valido. Ogni carattere del pattern è collegato con una linea verticale al corrispondente carattere nel testo; tutti i caratteri corrispondenti sono su sfondo grigio.

zione di $O(m\,|\Sigma|)$, ma un tempo di matching di appena $\Theta(n)$. L'analogo, ma più intelligente, algoritmo di Knuth-Morris-Pratt (o algoritmo KMP) è descritto nel Paragrafo 32.5; l'algoritmo KMP ha lo stesso tempo di matching $\Theta(n)$, ma riduce il tempo di preelaborazione ad appena $\Theta(m)$.

Notazione e terminologia

Indicheremo con Σ^* (si legge "sigma stella") l'insieme di tutte le stringhe di lunghezza finita formate utilizzando i caratteri dell'alfabeto Σ. In questo capitolo, considereremo soltanto stringhe di lunghezza finita. Anche la stringa di lunghezza zero, detta ***stringa nulla*** e indicata con ε, appartiene a Σ^*. La lunghezza di una stringa x è indicata con $|x|$. La ***concatenazione*** di due stringhe x e y, indicata con xy, ha lunghezza $|x| + |y|$ ed è composta dai caratteri di x seguiti dai caratteri di y.

Diremo che una stringa w è un ***prefisso*** di una stringa x, che indicheremo con $w \sqsubset x$, se $x = wy$ per qualche stringa $y \in \Sigma^*$. Notate che, se $w \sqsubset x$, allora $|w| \leq |x|$. Analogamente, diremo che una stringa w è un ***suffisso*** di una stringa x, che indicheremo con $w \sqsupset x$, se $x = yw$ per qualche $y \in \Sigma^*$. Da $w \sqsupset x$ segue che $|w| \leq |x|$. La stringa nulla ε è sia suffisso sia prefisso di qualsiasi stringa. Per esempio, si ha $\texttt{ab} \sqsubset \texttt{abcca}$ e $\texttt{cca} \sqsupset \texttt{abcca}$. È utile notare che per due stringhe qualsiasi x e y e per un carattere qualsiasi a, si ha $x \sqsupset y$ se e soltanto se $xa \sqsupset ya$. Notate inoltre che $\sqsubset$ e $\sqsupset$ sono relazioni transitive. Il seguente lemma sarà utile più avanti.

Lemma 32.1 (Lemma della sovrapposizione dei suffissi)
Supponiamo che x, y e z siano stringhe tali che $x \sqsupset z$ e $y \sqsupset z$. Se $|x| \leq |y|$, allora $x \sqsupset y$. Se $|x| \geq |y|$, allora $y \sqsupset x$. Se $|x| = |y|$, allora $x = y$.

Dimostrazione La dimostrazione grafica è illustrata nella Figura 32.3. ■

Per brevità di notazione, indicheremo con P_k il prefisso $P[1\,..\,k]$ di k caratteri del pattern $P[1\,..\,m]$. Quindi, $P_0 = \varepsilon$ e $P_m = P = P[1\,..\,m]$. Analogamente, indichiamo con T_k il prefisso di k caratteri del testo T. Utilizzando questa notazione, possiamo definire il problema dello string matching come quello di trovare tutti gli spostamenti s nell'intervallo $0 \leq s \leq n - m$ tali che $P \sqsupset T_{s+m}$.

Algoritmo	Tempo di preelaborazione	Tempo di matching
Ingenuo	0	$O((n-m+1)m)$
Rabin-Karp	$\Theta(m)$	$O((n-m+1)m)$
Automa a stati finiti	$O(m\,\|\Sigma\|)$	$\Theta(n)$
Knuth-Morris-Pratt	$\Theta(m)$	$\Theta(n)$

Figura 32.2 Gli algoritmi di string matching descritti in questo capitolo e i loro tempi di preelaborazione e matching.

Acquistato da Michele Miccoli su Webster il 2022-07-07 23:12 Numero Ordine Libreria 0995003016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

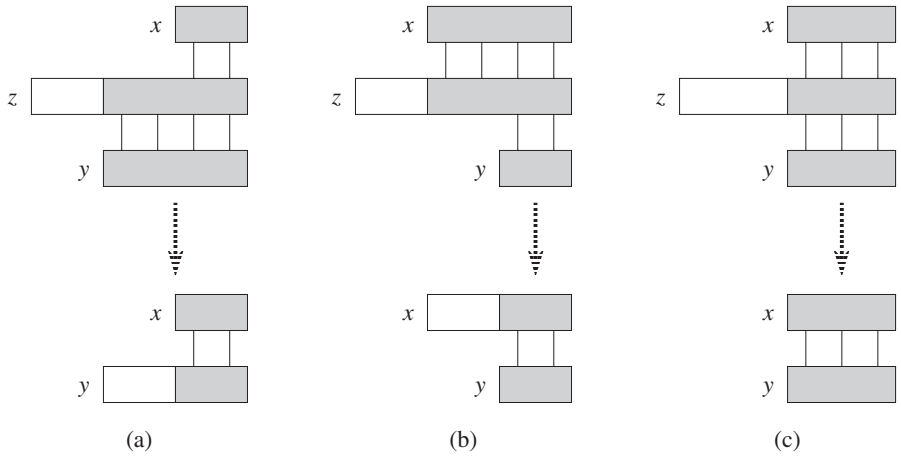


Figura 32.3
Una dimostrazione grafica del Lemma 32.1. Supponiamo che $x \sqsupset z$ e $y \sqsupset z$. Le tre parti della figura illustrano i tre casi del lemma. Le linee verticali collegano le zone di matching (in grigio) delle stringhe. **(a)** Se $|x| \leq |y|$, allora $x \sqsupset y$. **(b)** Se $|x| \geq |y|$, allora $y \sqsupset x$. **(c)** Se $|x| = |y|$, allora $x = y$.

Nel nostro pseudocodice, useremo come operazione primitiva il confronto di due stringhe della stessa lunghezza per controllare se sono uguali. Se le stringhe vengono confrontate da sinistra a destra e il confronto s'interrompe quando vengono incontrati due caratteri diversi (*mismatch*), assumiamo che il tempo impiegato da tale controllo sia una funzione lineare del numero di caratteri uguali (*match*) che sono stati scoperti. Più precisamente, si suppone che il test "$x == y$" richieda un tempo $\Theta(t + 1)$, dove t è la lunghezza della più lunga stringa z tale che $z \sqsubset x$ e $z \sqsubset y$. (Scriviamo $\Theta(t + 1)$, anziché $\Theta(t)$, per gestire il caso in cui $t = 0$; i primi caratteri confrontati sono diversi, ma occorre una quantità di tempo positiva per eseguire questo confronto.)

32.2 L'algoritmo ingenuo di string matching

L'algoritmo ingenuo (*naive*) trova tutti gli spostamenti validi utilizzando un ciclo che verifica la condizione $P[1\,..\,m] = T[s + 1\,..\,s + m]$ per ciascuno degli $n - m + 1$ valori possibili di s.

```
NAIVE-STRING-MATCHER(T, P)
1  n = T.length
2  m = P.length
3  for s = 0 to n − m
4       if P[1..m] == T[s + 1..s + m]
5            stampa "Occorrenza del pattern con spostamento" s
```

Come illustra la Figura 32.4, la procedura ingenua di string matching può essere interpretata graficamente come se si facesse scivolare sul testo una "sagoma" contenente il pattern, osservando per quali spostamenti tutti i caratteri sulla sagoma sono uguali ai corrispondenti caratteri nel testo. Il ciclo **for**, che inizia nella riga 3, considera esplicitamente ogni possibile spostamento. Il controllo nella riga 4 determina se lo spostamento corrente è valido oppure no; questo controllo richiede un ciclo implicito per confrontare le posizioni dei caratteri corrispondenti finché tutte le posizioni coincidono o si verifica un mismatch. La riga 5 stampa ogni spostamento valido s.

La procedura NAIVE-STRING-MATCHER richiede un tempo $O((n-m+1)m)$ e questo limite è stretto nel caso peggiore. Per esempio, consideriamo la stringa di

Acquistato da Michele Michele su Webster il 2022-0[illegible] Nicola Maria Caterina [illegible] Copyright © 2022, McGraw-Hill Education (Italy)

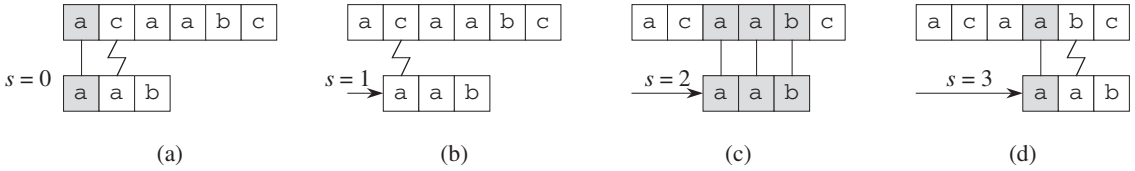


Figura 32.4 Il funzionamento della procedura NAIVE-STRING-MATCHER per il pattern $P = \texttt{aab}$ e il testo $T = \texttt{acaabc}$. Possiamo immaginare il pattern P come una "sagoma" che facciamo scivolare accanto al testo. **(a)–(d)** I quattro successivi allineamenti tentati dalla procedura. Le linee verticali collegano le zone dove si ha un match (su sfondo grigio); la linea seghettata collega il primo carattere di mismatch che viene trovato, se esiste. La parte (c) illustra una occorrenza del pattern, con spostamento $s = 2$.

testo $\texttt{a}^n$ (una stringa di n a) e il pattern $\texttt{a}^m$. Per ciascuno degli $n - m + 1$ possibili valori dello spostamento s, il ciclo implicito nella riga 4, che confronta i caratteri corrispondenti, deve essere eseguito m volte per stabilire se lo spostamento è valido. Il tempo di esecuzione nel caso peggiore è quindi $\Theta((n - m + 1)m)$, che è $\Theta(n^2)$ se $m = \lfloor n/2 \rfloor$. Il tempo di esecuzione di NAIVE-STRING-MATCHER è uguale al suo tempo di matching, perché non c'è preelaborazione.

Come vedremo, NAIVE-STRING-MATCHER non è una procedura ottima per questo problema. In effetti, in questo capitolo esamineremo un algoritmo con un tempo di preelaborazione pari a $\Theta(m)$ nel caso peggiore e un tempo di matching pari a $\Theta(n)$ nel caso peggiore. NAIVE-STRING-MATCHER non è efficiente perché le informazioni ottenute sul testo per un valore di s vengono completamente ignorate quando si considerano altri valori di s. Tali informazioni, invece, possono essere molto preziose. Per esempio, se $P = \texttt{aaab}$ e scopriamo che $s = 0$ è valido, allora nessuno degli spostamenti 1, 2 e 3 è valido, perché $T[4] = \texttt{b}$. Nei prossimi paragrafi esamineremo vari metodi per rendere efficiente l'uso di questo genere di informazioni.

Esercizi

32.2-1

Descrivete i confronti tra il pattern $P = \texttt{0001}$ e il testo $T = \texttt{000010001010001}$ effettuati dalla procedura NAIVE-STRING-MATCHER.

32.2-2

Supponete che tutti i caratteri nel pattern P siano differenti. Spiegate come accelerare la procedura NAIVE-STRING-MATCHER in modo che venga eseguita nel tempo $O(n)$ con un testo T di n caratteri.

32.2-3

Supponete che il pattern P e il testo T siano stringhe di lunghezza m ed n, rispettivamente, scelte *a caso* dall'alfabeto d-ario $\Sigma_d = \{0, 1, \ldots, d - 1\}$, dove $d \geq 2$. Dimostrate che il numero *atteso* di confronti carattere per carattere effettuati dal ciclo implicito nella riga 4 dell'algoritmo ingenuo è

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$(n - m + 1)\frac{1 - d^{-m}}{1 - d^{-1}} \leq 2(n - m + 1)$$

per tutte le esecuzioni di questo ciclo. (Supponete che l'algoritmo ingenuo smetta di confrontare i caratteri per un dato spostamento quando si verifica un mismatch o quando si verifica un match dell'intero pattern.) Quindi, per stringhe scelte a caso, l'algoritmo ingenuo è abbastanza efficiente.

32.2-4

Supponete che il pattern P contenga le occorrenze di un ***carattere jolly*** $\diamond$ che può sostituire una stringa *arbitraria* di caratteri (anche una stringa di lunghezza zero). Per esempio, il pattern `ab◇ba◇c` occorre nel testo `cabccbacbacab` come

$$\texttt{c}\underbrace{\texttt{ab}}_{\texttt{ab}}\underbrace{\texttt{cc}}_{\diamond}\underbrace{\texttt{ba}}_{\texttt{ba}}\underbrace{\texttt{cba}}_{\diamond}\underbrace{\texttt{c}}_{\texttt{c}}\texttt{ab}$$

e come

$$\texttt{c}\underbrace{\texttt{ab}}_{\texttt{ab}}\underbrace{\texttt{ccbac}}_{\diamond}\underbrace{\texttt{ba}}_{\texttt{ba}}\underbrace{}_{\diamond}\underbrace{\texttt{c}}_{\texttt{c}}\texttt{ab}$$

Notate che il carattere jolly può ripetersi un numero arbitrario di volte nel pattern, ma si suppone che non compaia affatto all'interno del testo. Create un algoritmo con tempo polinomiale per determinare se un tale pattern P sia presente in un dato testo T; analizzate il tempo di esecuzione del vostro algoritmo.

32.3 Algoritmo di Rabin-Karp

Rabin e Karp hanno proposto un algoritmo di string matching che ha buone prestazioni nella pratica e che può essere generalizzato ad altri algoritmi per risolvere problemi analoghi, come il matching con un pattern bidimensionale. L'algoritmo di Rabin-Karp effettua una preelaborazione con tempo $\Theta(m)$ e ha un tempo di esecuzione di $\Theta((n-m+1)m)$ nel caso peggiore. Sotto certe ipotesi, tuttavia, il suo tempo di esecuzione nel caso medio è migliore.

Questo algoritmo usa i concetti fondamentali della teoria dei numeri, come la congruenza di due numeri modulo un terzo numero. Consultate il Paragrafo 31.1 per le definizioni pertinenti.

Per semplificare l'esposizione, supponiamo che $\Sigma = \{\texttt{0}, \texttt{1}, \texttt{2}, \ldots, \texttt{9}\}$, in modo che ciascun carattere sia una cifra decimale. (Nel caso generale, possiamo supporre che ciascun carattere sia una cifra nella rappresentazione in base d, dove $d = |\Sigma|$.) Così facendo, possiamo considerare una stringa di k caratteri consecutivi come la rappresentazione di un numero decimale di lunghezza k. La stringa di caratteri `31415` quindi corrisponde al numero decimale 31415. Data la duplice interpretazione dei caratteri di input come simboli grafici e come cifre, in questo paragrafo è comodo indicare tali caratteri come se fossero cifre, nel formato tipografico standard usato nel testo.

Dato un pattern $P[1\,..\,m]$, indichiamo con p il corrispondente valore decimale. Analogamente, dato un testo $T[1\,..\,n]$, indichiamo con t_s il valore decimale della sottostringa $T[s+1\,..\,s+m]$ di lunghezza m, per $s = 0, 1, \ldots, n-m$. Certamente, $t_s = p$ se e soltanto se $T[s+1\,..\,s+m] = P[1\,..\,m]$; quindi, s è uno spostamento valido se e soltanto se $t_s = p$. Se potessimo calcolare p nel tempo $\Theta(m)$ e tutti i valori t_s nel tempo totale $\Theta(n-m+1)$,[1] allora potremmo determinare tutti gli spostamenti validi s nel tempo $\Theta(m) + \Theta(n-m+1) = \Theta(n)$ confrontando p con ciascun valore t_s. (Per il momento, non preoccupiamoci della possibilità che i valori p e t_s possano essere numeri molto grandi.)

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 19950316-220767-0 Copyright © 2022. McGraw-Hill Education (Italy)

[1] Scriviamo $\Theta(n-m+1)$, anziché $\Theta(n-m)$, perché s può assumere $n-m+1$ valori differenti. Il termine "+1" è significativo in senso asintotico perché, quando $m = n$, calcolare l'unico valore t_s richiede un tempo $\Theta(1)$, non $\Theta(0)$.

Possiamo calcolare p nel tempo $\Theta(m)$ utilizzando la regola di Horner (vedere il Paragrafo 30.1):

$$p = P[m] + 10\,(P[m-1] + 10(P[m-2] + \cdots + 10(P[2] + 10P[1])\cdots))$$

Il valore t_0 può essere calcolato in modo analogo da $T[1\,..\,m]$ nel tempo $\Theta(m)$.

Per calcolare i restanti valori $t_1, t_2, \ldots, t_{n-m}$ nel tempo $\Theta(n-m)$, basta osservare che t_{s+1} può essere calcolato da t_s in tempo costante, in quanto

$$t_{s+1} = 10(t_s - 10^{m-1}T[s+1]) + T[s+m+1] \tag{32.1}$$

Sottraendo $10^{m-1}T[s+1]$ si elimina la cifra più significativa da t_s; moltiplicando il risultato per 10 si fa scorrere il numero di una posizione a sinistra; sommando $T[s+m+1]$ si inserisce la cifra meno significativa appropriata. Per esempio, se $m = 5$ e $t_s = 31415$, vogliamo eliminare la cifra più significativa $T[s+1] = 3$ e inserire una nuova cifra meno significativa (supponiamo che sia $T[s+5+1] = 2$) per ottenere

$$t_{s+1} = 10(31415 - 10000 \cdot 3) + 2 = 14152$$

Se la costante 10^{m-1} viene precalcolata (questo può essere fatto nel tempo $O(\lg m)$ utilizzando le tecniche del Paragrafo 31.6, sebbene per questa applicazione sia più adeguato un metodo semplice con tempo $O(m)$), allora ogni esecuzione dell'equazione (32.1) richiede un numero costante di operazioni aritmetiche. Quindi, è possibile calcolare p nel tempo $\Theta(m)$ e $t_0, t_1, \ldots, t_{n-m}$ nel tempo $\Theta(n-m+1)$; è possibile trovare tutte le occorrenze del pattern $P[1\,..\,m]$ nel testo $T[1\,..\,n]$ con un tempo di preelaborazione $\Theta(m)$ e un tempo di matching $\Theta(n-m+1)$.

Finora abbiamo intenzionalmente trascurato un problema: p e t_s potrebbero essere troppo grandi per lavorarci in modo conveniente. Se P contiene m caratteri, non possiamo ragionevolmente assumere che ogni operazione aritmetica con p (che è lungo m cifre) richieda un "tempo costante". Fortunatamente, c'è un semplice rimedio per questo problema, come illustra la Figura 32.5: calcolare p e i t_s modulo un appropriato intero q. Poiché il calcolo di p e t_0 e della ricorrenza (32.1) può essere eseguito con modulo q, è possibile calcolare p modulo q nel tempo $\Theta(m)$ e tutti i t_s modulo q nel tempo $\Theta(n-m+1)$. Tipicamente, il modulo q è scelto come un numero primo tale che $10q$ sia rappresentabile con una parola del calcolatore, in modo che si possano eseguire tutti i calcoli necessari in singola precisione. In generale, con un alfabeto d-ario $\{0, 1, \ldots, d-1\}$, scegliamo q in modo che dq possa essere rappresentato con una parola del calcolatore e adattiamo l'equazione della ricorrenza (32.1) per operare con modulo q, in questo modo

$$t_{s+1} = (d(t_s - T[s+1]h) + T[s+m+1]) \bmod q \tag{32.2}$$

Il termine $h \equiv d^{m-1} \pmod q$ è il valore della cifra "1" nella posizione più significativa di una finestra di testo di m cifre.

Acquistato da Michele Michelucci (mic71832@unet.univie.ac.at) - Licenza per uso personale - Vietata la copia - Copyright © 2022 McGraw-Hill Education (Italy)

Notiamo, però, che la soluzione di operare modulo q non è perfetta, perché $t_s \equiv p \pmod q$ non implica che $t_s = p$. D'altra parte, se $t_s \not\equiv p \pmod q$, allora sicuramente si ha $t_s \neq p$, quindi lo spostamento s non è valido. Allora possiamo utilizzare il controllo $t_s \equiv p \pmod q$ come un rapido controllo euristico per escludere gli spostamenti s non validi. Qualsiasi spostamento s per il quale $t_s \equiv p \pmod q$ deve essere ulteriormente controllato per verificare se s è effettivamente valido o se abbiamo soltanto un ***falso successo*** (*spurious hit*). Questo controllo può essere fatto esplicitamente verificando la condizione

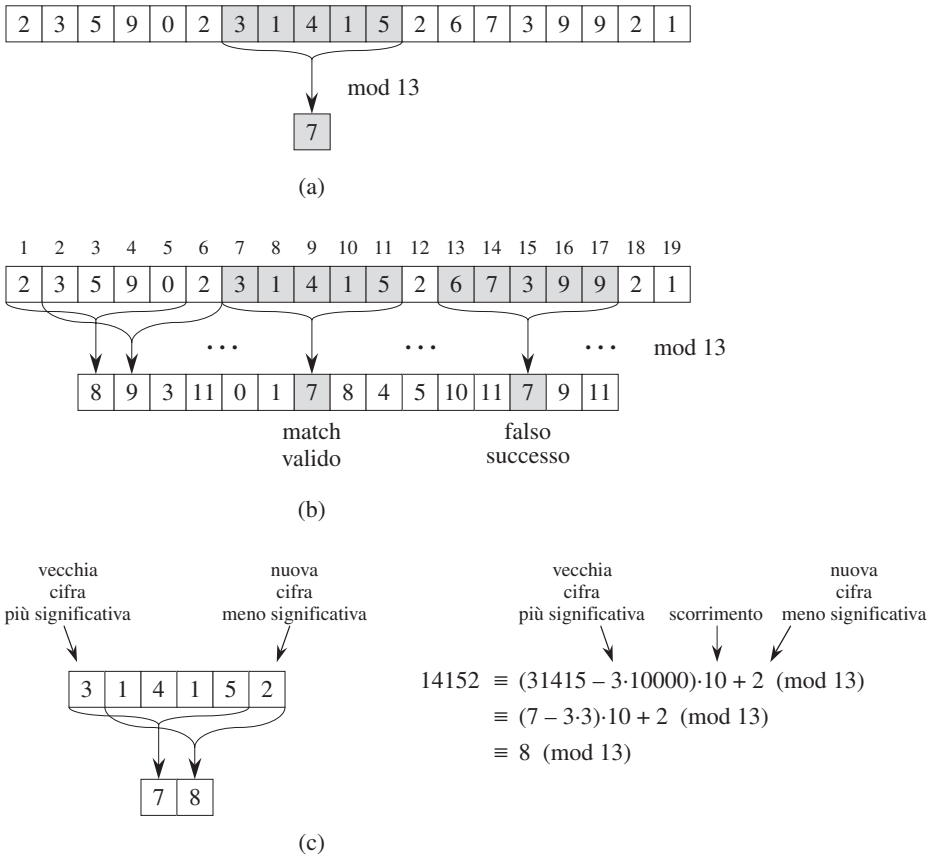


Figura 32.5 L'algoritmo di Rabin-Karp. Ogni carattere è una cifra decimale e noi calcoliamo i valori modulo 13. **(a)** Una stringa di testo. Una finestra di testo di lunghezza 5 è rappresentata in grigio. Il valore del numero in grigio è calcolato modulo 13, quindi si ottiene il valore 7. **(b)** La stessa stringa di testo con i valori calcolati modulo 13 per ogni posizione possibile di una finestra di testo di lunghezza 5. Supponendo che il pattern sia $P = 31415$, l'algoritmo cerca le finestre il cui valore modulo 13 sia 7, perché $31415 \equiv 7 \pmod{13}$. Trova due di tali finestre, rappresentate in grigio nella figura. La prima, che inizia nella posizione 7, è davvero una occorrenza del pattern, mentre la seconda, che inizia nella posizione 13, è un falso successo. **(c)** Calcolo del valore di una finestra in tempo costante, dato il valore della finestra precedente. La prima finestra ha valore 31415. Eliminando la cifra più significativa (3), effettuando uno scorrimento a sinistra (moltiplicando per 10) e infine sommando la cifra meno significativa (2), si ottiene il nuovo valore 14152. Poiché tutti i calcoli sono eseguiti con modulo 13, il valore della prima finestra è 7 e il valore calcolato della nuova finestra è 8.

$P[1\,..\,m] = T[s+1\,..\,s+m]$. Se q è sufficientemente grande, allora possiamo sperare che i falsi successi si verifichino così raramente che il costo del controllo extra possa mantenersi basso.

La seguente procedura precisa questi concetti. Gli input della procedura sono il testo T, il pattern P, la base d (che di solito è $|\Sigma|$) e il numero primo q.

```
RABIN-KARP-MATCHER(T, P, d, q)
 1  n = T.length
 2  m = P.length
 3  h = d^(m−1) mod q
 4  p = 0
 5  t_0 = 0
 6  for i = 1 to m                    // Preelaborazione.
 7      p = (dp + P[i]) mod q
 8      t_0 = (dt_0 + T[i]) mod q
 9  for s = 0 to n − m                // Matching.
10      if p == t_s
11          if P[1..m] == T[s + 1..s + m]
12              stampa "Occorrenza del pattern con spostamento" s
13      if s < n − m
14          t_{s+1} = (d(t_s − T[s + 1]h) + T[s + m + 1]) mod q
```

La procedura RABIN-KARP-MATCHER opera nel modo seguente. Tutti i caratteri sono interpretati come cifre in base d. I pedici di t sono indicati soltanto per

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950316-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

migliorare la leggibilità del codice; il programma funziona correttamente anche se tutti i pedici vengono cancellati. La riga 3 inizializza h al valore della cifra più significativa di una finestra di m cifre. Le righe 4–8 calcolano p come il valore di $P[1\,..\,m]$ mod q e t_0 come il valore di $T[1\,..\,m]$ mod q. Il ciclo **for** (righe 9–14) si ripete per tutti i possibili spostamenti s, conservando la seguente invariante:

Quando viene eseguita la riga 10, $t_s = T[s + 1\,..\,s + m] \bmod q$.

Se $p = t_s$ nella riga 10 si verifica un ***successo***; in questo caso, verifichiamo se $P[1\,..\,m] = T[s + 1\,..\,s + m]$ nella riga 11 per escludere la possibilità di un falso successo. La riga 12 segnala tutti gli spostamenti validi che vengono trovati. Se $s < n - m$ (controllo effettuato nella riga 13), allora il ciclo **for** deve essere eseguito almeno un'altra volta, e viene quindi prima eseguita la riga 14 per garantire che l'invariante di ciclo sia vera quando viene raggiunta di nuovo la riga 10. La riga 14 calcola il valore di t_{s+1} mod q dal valore di t_s mod q in tempo costante utilizzando direttamente l'equazione (32.2).

La procedura RABIN-KARP-MATCHER ha un tempo di preelaborazione pari a $\Theta(m)$ e un tempo di matching pari a $\Theta((n - m + 1)m)$ nel caso peggiore, in quanto (come l'algoritmo ingenuo di string matching) l'algoritmo di Rabin-Karp verifica esplicitamente tutti gli spostamenti validi. Se $P = \mathtt{a}^m$ e $T = \mathtt{a}^n$, allora le verifiche richiedono un tempo $\Theta((n-m+1)m)$, perché ciascuno degli $n-m+1$ spostamenti possibili è valido.

In molte applicazioni sono previsti pochi spostamenti validi (talvolta soltanto un numero costante c di essi); in queste applicazioni il tempo di matching atteso dell'algoritmo è soltanto $O((n - m + 1) + cm) = O(n+m)$, più il tempo richiesto per elaborare i falsi successi. È possibile basare un'analisi euristica sull'ipotesi che la riduzione dei valori modulo q operi come una corrispondenza casuale da Σ^* a $\mathbb{Z}_q$. (Consultate il Paragrafo 11.3.1 sull'uso del metodo della divisione nell'hashing. È difficile formalizzare e provare una tale ipotesi, sebbene ci sia un approccio praticabile che consiste nell'ipotizzare che q venga scelto a caso tra i numeri interi di dimensione appropriata. Qui non proseguiremo in questo tipo di formalizzazione.) Possiamo dunque prevedere che il numero di falsi successi sia $O(n/q)$, perché la probabilità che un valore t_s arbitrario sia equivalente a p, modulo q, può essere stimata pari a $1/q$. Poiché ci sono $O(n)$ posizioni in cui il controllo della riga 10 fallisce e noi impieghiamo un tempo $O(m)$ per ogni successo, il tempo di matching atteso che richiese l'algoritmo di Rabin-Karp è

$$O(n) + O(m(v + n/q))$$

Il termine v rappresenta il numero di spostamenti validi. Questo tempo di esecuzione è $O(n)$ se $v = O(1)$ e scegliamo $q \geq m$. Ovvero, se il numero atteso di spostamenti validi è piccolo ($O(1)$) e il numero primo q è scelto in modo da essere più grande della lunghezza del pattern, allora possiamo prevedere che l'algoritmo di Rabin-Karp richieda soltanto il tempo di matching $O(n + m)$. Poiché $m \leq n$, questo tempo di matching atteso è $O(n)$.

Acquistato da Michele Mu... Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Esercizi

32.3-1

Operando con modulo $q = 11$, quanti falsi successi si verificano nella procedura RABIN-KARP-MATCHER durante la ricerca del pattern $P = 26$ nel testo $T = 3141592653589793$?

32.3-2

Come estendereste il metodo di Rabin-Karp al problema di cercare in una stringa di testo l'occorrenza di un pattern qualsiasi di un dato insieme di k pattern? Iniziate supponendo che tutti i k pattern abbiano la stessa lunghezza. Poi generalizzate la vostra soluzione per accettare pattern di lunghezza differente.

32.3-3

Spiegate come estendere il metodo di Rabin-Karp per gestire il problema di cercare un dato pattern $m \times m$ in un array di caratteri $n \times n$. (Il pattern può essere spostato verticalmente e orizzontalmente, ma non può essere ruotato.)

32.3-4

Alice ha una copia di un file $A = \langle a_{n-1}, a_{n-2}, \ldots, a_0 \rangle$ lungo n bit; anche Bob ha un file $B = \langle b_{n-1}, b_{n-2}, \ldots, b_0 \rangle$ di n bit. Alice e Bob vogliono sapere se i loro file sono uguali. Per evitare di trasmettere tutto il file A o B, utilizzano il seguente controllo probabilistico veloce. Insieme scelgono un numero primo $q > 1000n$ e selezionano a caso un numero intero x da $\{0, 1, \ldots, q-1\}$. Poi Alice valuta

$$A(x) = \left(\sum_{i=0}^{n-1} a_i x^i \right) \bmod q$$

e Bob valuta in modo simile $B(x)$. Dimostrate che se $A \neq B$, c'è al più una probabilità su 1000 che $A(x) = B(x)$, mentre se i due file sono uguali, $A(x)$ è necessariamente uguale a $B(x)$ (*suggerimento:* vedere l'Esercizio 31.4-4).

32.4 String matching con automi a stati finiti

Molti algoritmi di string matching costruiscono un automa a stati finiti che ispeziona la stringa di testo T per trovare tutte le occorrenze del pattern P. Questo paragrafo presenta un metodo per costruire un tale automa. Gli automi di string matching sono molto efficienti: esaminano ciascun carattere del testo *una sola volta*, impiegando un tempo costante per ogni carattere. Il tempo di matching impiegato – dopo il tempo di preelaborazione del pattern per costruire l'automa – è quindi $\Theta(n)$. Il tempo per costruire l'automa, tuttavia, può essere grande, se Σ è grande. Il Paragrafo 32.5 descrive un metodo più intelligente per risolvere questo problema.

Inizieremo questo paragrafo con la definizione di automa a stati finiti. Poi esamineremo uno speciale automa di string matching e spiegheremo come utilizzare l'automa per trovare le occorrenze di un pattern all'interno di un testo. Questa discussione include i dettagli su come simulare il comportamento di un automa di string matching in un dato testo. Infine, descriveremo come costruire l'automa di string matching per un dato pattern di input.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Automa a stati finiti

Un ***automa a stati finiti*** M, illustrato nella Figura 32.6, è una 5-tupla $(Q, q_0, A, \Sigma, \delta)$, dove

- Q è un insieme finito di ***stati***
- $q_0 \in Q$ è lo ***stato iniziale***
- $A \subseteq Q$ è un insieme distinto di ***stati accettanti***

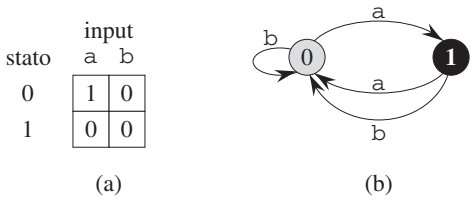


Figura 32.6 Un semplice automa a due stati finiti con l'insieme degli stati $Q = \{0, 1\}$, lo stato iniziale $q_0 = 0$ e l'alfabeto di input $\Sigma = \{\texttt{a}, \texttt{b}\}$. **(a)** Una rappresentazione tabulare della funzione di transizione δ. **(b)** Un diagramma equivalente degli stati di transizione. Lo stato 1 è l'unico stato accettante (di colore nero). Gli archi orientati rappresentano le transizioni. Per esempio, l'arco dallo stato 1 allo stato 0 etichettato b indica $\delta(1, \texttt{b}) = 0$. Questo automa accetta quelle stringhe che terminano con un numero dispari di lettere a. Più precisamente, una stringa x è accettata se e soltanto se $x = yz$, dove $y = \varepsilon$, o y termina con una b, e $z = \texttt{a}^k$, dove k è dispari. Per esempio, la sequenza degli stati di questo automa per l'input abaaa (incluso lo stato iniziale) è $\langle 0, 1, 0, 1, 0, 1\rangle$, quindi l'automa accetta questo input. Per l'input abbaa, la sequenza degli stati è $\langle 0, 1, 0, 0, 1, 0\rangle$, quindi l'automa rifiuta questo input.

- Σ è un ***alfabeto di input*** finito
- δ è una funzione da $Q \times \Sigma$ in Q, detta ***funzione di transizione*** di M

L'automa a stati finiti inizia nello stato q_0 e legge i caratteri della sua stringa di input uno alla volta. Se l'automa si trova nello stato q e legge un carattere di input a, passa ("effettua una transizione") dallo stato q allo stato $\delta(q, a)$. Se lo stato corrente q è un elemento di A, si dice che la macchina M ha ***accettato*** la stringa finora letta. Se un input non viene accettato, diremo che è ***rifiutato***. La Figura 32.6 illustra queste definizioni con un semplice automa a due stati.

Un automa a stati finiti M induce una funzione ϕ, detta ***funzione stato finale***, da Σ^* a Q tale che $\phi(w)$ è lo stato in cui finisce M dopo avere ispezionato la stringa w. Quindi, M accetta una stringa w se e soltanto se $\phi(w) \in A$. La funzione ϕ è definita dalla relazione ricorsiva

$$\begin{aligned} \phi(\varepsilon) &= q_0 \\ \phi(wa) &= \delta(\phi(w), a) \quad \text{per} \quad w \in \Sigma^*, a \in \Sigma \end{aligned}$$

Automa di string matching

Esiste un automa di string matching per ogni pattern P; questo automa deve essere costruito dal pattern in un passo di preelaborazione prima che esso possa essere utilizzato per effettuare le ricerche nella stringa di testo. La Figura 32.7 illustra questa costruzione per il pattern $P = \texttt{ababaca}$. Da qui in avanti, assumeremo che P sia un pattern prefissato; per brevità, non indicheremo la dipendenza da P nella nostra notazione.

Per specificare l'automa di string matching che corrisponde a un dato pattern $P[1\,..\,m]$, definiamo prima una funzione ausiliaria σ, detta ***funzione suffisso***, associata a P. La funzione σ è una corrispondenza da Σ^* a $\{0, 1, \ldots, m\}$ tale che $\sigma(x)$ è la lunghezza del prefisso più lungo di P che è un suffisso di x:

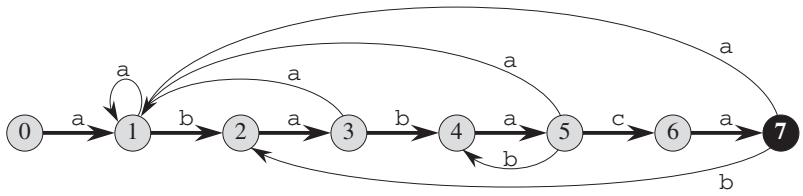
$$\sigma(x) = \max\{k : P_k \sqsupset x\} \tag{32.3}$$

La funzione suffisso σ è ben definita perché la stringa nulla $P_0 = \varepsilon$ è un suffisso di qualsiasi stringa. Per esempio, per il pattern $P = \texttt{ab}$, si ha $\sigma(\varepsilon) = 0$, $\sigma(\texttt{ccaca}) = 1$ e $\sigma(\texttt{ccab}) = 2$. Per un pattern P di lunghezza m, si ha $\sigma(x) = m$ se e soltanto se $P \sqsupset x$. Dalla definizione della funzione suffisso segue che, se $x \sqsupset y$, allora $\sigma(x) \le \sigma(y)$. Definiamo l'automa di string matching che corrisponde a un dato pattern $P[1\,..\,m]$ nel modo seguente.

Acquistato da Michele Michele su Webster il 2022-07-04 23:12 Numero Ordine Libreria: 1995036816-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

- L'insieme degli stati Q è $\{0, 1, \ldots, m\}$. Lo stato iniziale q_0 è lo stato 0; lo stato m è l'unico stato accettante.
- La funzione di transizione δ è definita dalla seguente equazione, per ogni stato q e carattere a:

$$\delta(q, a) = \sigma(P_q a) \tag{32.4}$$



(a)

stato	input a	b	c	P
0	1	0	0	a
1	1	2	0	b
2	3	0	0	a
3	1	4	0	b
4	5	0	0	a
5	1	4	6	c
6	7	0	0	a
7	1	2	0	

(b)

i	—	1	2	3	4	5	6	7	8	9	10	11
$T[i]$	—	a	b	a	b	a	b	a	c	a	b	a
stato $\phi(T_i)$	0	1	2	3	4	5	4	5	6	**7**	2	3

(c)

Figura 32.7
(a) Un diagramma degli stati di transizione per l'automa di string matching che accetta tutte le stringhe che terminano con la stringa `ababaca`. Lo stato 0 è lo stato iniziale; lo stato 7 (di colore nero) è l'unico stato accettante. Un arco orientato dallo stato i allo stato j, etichettato con la lettera a, rappresenta $\delta(i, a) = j$. Gli archi diretti a destra che formano la "spina dorsale" dell'automa (indicati con un tratto più spesso nella figura), corrispondono ai match tra il pattern e i caratteri di input. Gli archi diretti a sinistra corrispondono ai mismatch. Alcuni archi di mismatch non sono illustrati; per convenzione, se uno stato i non ha un arco uscente con l'etichetta a per qualche $a \in \Sigma$, allora $\delta(i, a) = 0$.
(b) La corrispondente funzione di transizione δ e il pattern $P =$ `ababaca`. Gli elementi corrispondenti ai match tra il pattern e i caratteri di input sono rappresentati su uno sfondo grigio.
(c) Il funzionamento dell'automa con il testo $T =$ `abababacaba`. Sotto ogni carattere di testo $T[i]$ è indicato lo stato $\phi(T_i)$ in cui si trova l'automa dopo l'elaborazione del prefisso T_i. Viene trovata una occorrenza del pattern, che termina nella posizione 9.

Definiamo $\delta(q, a) = \sigma(P_q a)$ perché vogliamo conservare la traccia del più lungo prefisso del pattern P che fino a questo momento coincide con la stringa di testo T. Consideriamo i caratteri di T letti più di recente. Affinché una sottostringa di T – per esempio la sottostringa che termina in $T[i]$ – coincida con qualche prefisso P_j di P, questo prefisso P_j deve essere un suffisso di T_i. Supponiamo che $q = \phi(T_i)$, in modo che dopo aver letto T_i, l'automa sia nello stato q. Progettiamo la funzione di transizione δ in modo che questo numero di stato, q, ci indichi la lunghezza del più lungo prefisso di P che coincide con un suffisso di T_i. Ovvero, nello stato q, $P_q \sqsupset T_i$ e $q = \sigma(T_i)$ (quando $q = m$, tutti gli m caratteri di P coincidono con un suffisso di T_i, e quindi abbiamo trovato un match). Dunque, poiché $\phi(T_i)$ e $\sigma(T_i)$ sono entrambi uguali a q, vedremo (nel successivo Teorema 32.4) che l'automa conserva la seguente invariante:

$$\phi(T_i) = \sigma(T_i) \tag{32.5}$$

Se l'automa è nello stato q e legge il successivo carattere $T[i + 1] = a$, allora vogliamo che la transizione porti allo stato che corrisponde al più lungo prefisso di P che è un suffisso di $T_i a$, e che lo stato sia $\sigma(T_i a)$. Poiché P_q è il più lungo prefisso di P che è un suffisso di T_i, il più lungo prefisso di P che è un suffisso di $T_i a$ non è soltanto $\sigma(T_i a)$, ma anche $\sigma(P_q a)$ (il Lemma 32.3, a pagina 834, dimostra che $\sigma(T_i a) = \sigma(P_q a)$). Dunque, quando l'automa è nello stato q, vogliamo che la funzione di transizione nel carattere a porti l'automa nello stato $\sigma(P_q a)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:10 Numero Ordine Libreria 1803016222 2022 © Copyright © 2022 McGraw-Hill Education (Italy)

Ci sono due casi da considerare. Nel primo caso, $a = P[q + 1]$, quindi il carattere a continua a coincidere con il pattern; in questo caso, poiché $\delta(q, a) = q + 1$, la transizione continua a seguire la "spina dorsale" dell'automa (gli archi più spessi nella Figura 32.7). Nel secondo caso, $a \neq P[q + 1]$; quindi a non coincide con il pattern. Qui dobbiamo trovare un prefisso di P più corto che sia anche un suffisso di T_i. Poiché il passo di preelaborazione confronta il pattern con sé stesso quando crea l'automa di string matching, la funzione di transizione identifica subito il maggiore tra tali prefissi più corti.

Esaminiamo un esempio. L'automa di string matching della Figura 32.7 ha $\delta(5, \mathtt{c}) = 6$, che rappresenta il primo caso, in cui il match continua. Per illustrare il secondo caso, notiamo che l'automa della Figura 32.7 ha $\delta(5, \mathtt{b}) = 4$. Effettuiamo questa transizione perché se l'automa legge una `b` nello stato $q = 5$, allora $P_q\mathtt{b} = \mathtt{ababab}$, e il più lungo prefisso di P, che è anche un suffisso di `ababab`, è $P_4 = \mathtt{abab}$.

Per spiegare il funzionamento di un automa di string matching, presentiamo un semplice, ma efficiente, programma per simulare il comportamento di un tale automa (rappresentato dalla sua funzione di transizione δ) nel trovare le occorrenze di un pattern P di lunghezza m in un testo di input $T[1\,..\,n]$. Come per qualsiasi automa di string matching per un pattern di lunghezza m, l'insieme degli stati Q è $\{0, 1, \ldots, m\}$, lo stato iniziale è 0 e l'unico stato accettante è lo stato m.

```
FINITE-AUTOMATON-MATCHER(T, δ, m)
1  n = T.length
2  q = 0
3  for i = 1 to n
4       q = δ(q, T[i])
5       if q == m
6            stampa "Occorrenza del pattern con spostamento" i − m
```

La semplice struttura del ciclo di FINITE-AUTOMATON-MATCHER implica che il suo tempo di matching con una stringa di testo di lunghezza n è $\Theta(n)$. Questo tempo di matching, però, non include il tempo di preelaborazione richiesto per calcolare la funzione di transizione δ. Tratteremo questo problema più avanti, dopo avere dimostrato che la procedura FINITE-AUTOMATON-MATCHER funziona correttamente.

Consideriamo il funzionamento dell'automa con un testo di input $T[1\,..\,n]$. Proveremo che l'automa si trova nello stato $\sigma(T_i)$ dopo avere ispezionato il carattere $T[i]$. Poiché $\sigma(T_i) = m$ se e soltanto se $P \sqsupset T_i$, la macchina si trova nello stato accettante m se e soltanto se è stato appena ispezionato l'intero pattern P. Per provare questo, utilizziamo i seguenti lemmi sulla funzione suffisso σ.

Lemma 32.2 (Disuguaglianza della funzione suffisso)
Per ogni stringa x e carattere a, si ha $\sigma(xa) \le \sigma(x) + 1$.

Dimostrazione Facendo riferimento alla Figura 32.8, poniamo $r = \sigma(xa)$. Se $r = 0$, allora la condizione $\sigma(xa) = r \le \sigma(x) + 1$ è banalmente soddisfatta, perché il valore di $\sigma(x)$ non è mai negativo. Quindi supponiamo che $r > 0$. Adesso $P_r \sqsupset xa$, per la definizione di σ. Quindi $P_{r-1} \sqsupset x$, escludendo la a alla fine di P_r e alla fine di xa. Ne consegue che $r - 1 \le \sigma(x)$, perché $\sigma(x)$ è il valore massimo di k tale $P_k \sqsupset x$, e $\sigma(xa) = r \le \sigma(x) + 1$. ∎

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

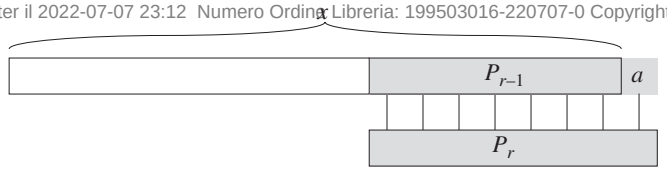


Figura 32.8 Illustrazione della dimostrazione del Lemma 32.2. La figura indica che $r \le \sigma(x) + 1$, dove $r = \sigma(xa)$.

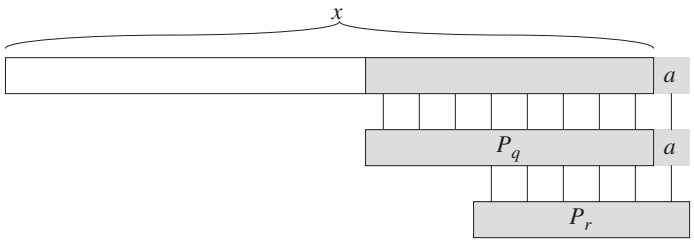


Figura 32.9 Illustrazione della dimostrazione del Lemma 32.3. La figura indica che $r = \sigma(P_q a)$, dove $q = \sigma(x)$ e $r = \sigma(xa)$.

Lemma 32.3 (Ricorsione della funzione suffisso)
Per ogni stringa x e carattere a, se $q = \sigma(x)$, allora $\sigma(xa) = \sigma(P_q a)$.

Dimostrazione Dalla definizione di σ, si ha $P_q \sqsupset x$. Come illustra la Figura 32.9, si ha anche $P_q a \sqsupset xa$. Se poniamo $r = \sigma(xa)$, allora $r \leq q + 1$ per il Lemma 32.2. Poiché $P_q a \sqsupset xa$, $P_r \sqsupset xa$ e $|P_r| \leq |P_q a|$, il Lemma 32.1 implica che $P_r \sqsupset P_q a$. Quindi $r \leq \sigma(P_q a)$, cioè $\sigma(xa) \leq \sigma(P_q a)$; ma si ha anche $\sigma(P_q a) \leq \sigma(xa)$, perché $P_q a \sqsupset xa$. Dunque $\sigma(xa) = \sigma(P_q a)$. ■

Adesso possiamo dimostrare il teorema principale che caratterizza il comportamento di un automa di string matching con un dato testo di input. Come detto in precedenza, questo teorema dimostra che l'automa tiene semplicemente traccia, a ogni passo, del più lungo prefisso del pattern che è un suffisso di quanto è stato letto fino a quel momento. In altre parole, l'automa conserva l'invariante (32.5).

Teorema 32.4
Se ϕ è la funzione dello stato finale di un automa di string matching per un dato pattern P e $T[1\,..\,n]$ è un testo di input per l'automa, allora

$$\phi(T_i) = \sigma(T_i)$$

per $i = 0, 1, \ldots, n$.

Dimostrazione La dimostrazione è per induzione su i. Per $i = 0$, il teorema è banalmente vero, perché $T_0 = \varepsilon$. Quindi, $\phi(T_0) = 0 = \sigma(T_0)$.

Adesso supponiamo che $\phi(T_i) = \sigma(T_i)$ e proviamo che $\phi(T_{i+1}) = \sigma(T_{i+1})$. Indichiamo $\phi(T_i)$ con q e $T[i + 1]$ con a. Allora si ha

$$\begin{aligned}
\phi(T_{i+1}) &= \phi(T_i a) && \text{(per le definizioni di } T_{i+1} \text{ e } a) \\
&= \delta(\phi(T_i), a) && \text{(per la definizione di } \phi) \\
&= \delta(q, a) && \text{(per la definizione di } q) \\
&= \sigma(P_q a) && \text{(per la definizione (32.4) di } \delta) \\
&= \sigma(T_i a) && \text{(per il Lemma 32.3 e per induzione)} \\
&= \sigma(T_{i+1}) && \text{(per la definizione di } T_{i+1})
\end{aligned}$$

■

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Per il Teorema 32.4, se la macchina passa nello stato q nella riga 4, allora q è il più grande valore tale che $P_q \sqsupset T_i$. Quindi, si ha $q = m$ nella riga 5 se e soltanto se è stata appena ispezionata una occorrenza del pattern P. Concludiamo che FINITE-AUTOMATON-MATCHER funziona correttamente.

Calcolare la funzione di transizione

La seguente procedura calcola la funzione di transizione δ da un dato pattern $P[1..m]$.

COMPUTE-TRANSITION-FUNCTION(P, Σ)

```
1  m = P.length
2  for q = 0 to m
3      for ogni carattere a ∈ Σ
4          k = min(m + 1, q + 2)
5          repeat
6              k = k − 1
7          until P_k ⊐ P_q a
8          δ(q, a) = k
9  return δ
```

Questa procedura calcola $\delta(q, a)$ in modo semplice secondo la sua definizione nell'equazione (32.4). I cicli annidati che iniziano nelle righe 2 e 3 considerano tutti gli stati q e i caratteri a; le righe 4–8 assegnano a $\delta(q, a)$ il più grande valore di k tale che $P_k \sqsupset P_q a$. Il codice inizia con il massimo valore possibile di k, che è $\min(m, q + 1)$, e riducono il valore di k finché $P_k \sqsupset P_q a$. Il tempo di esecuzione di COMPUTE-TRANSITION-FUNCTION è $O(m^3 |\Sigma|)$, perché i cicli esterni contribuiscono con un fattore $m|\Sigma|$, il ciclo **repeat** interno può essere eseguito al più $m + 1$ volte e il controllo $P_k \sqsupset P_q a$ nella riga 7 può richiedere di confrontare fino a m caratteri.

Esistono procedure molto più veloci; il tempo richiesto per calcolare δ da P può essere ridotto a $O(m|\Sigma|)$, utilizzando qualche informazione calcolata intelligentemente sul pattern P (vedere l'Esercizio 32.5-8). Con questa procedura migliorata per calcolare δ, è possibile trovare tutte le occorrenze di un pattern di lunghezza m in un testo di lunghezza n con un alfabeto Σ in un tempo di preelaborazione $O(m|\Sigma|)$ e un tempo di matching $\Theta(n)$.

Esercizi

32.4-1

Costruite l'automa di string matching per il pattern $P = \texttt{aabab}$ e descrivete il suo funzionamento con la stringa di testo $T = \texttt{aaababaabaababaab}$.

32.4-2

Disegnate un diagramma degli stati di transizione per un automa di string matching per il pattern $\texttt{ababbabbababbababbabb}$ con l'alfabeto $\Sigma = \{\texttt{a}, \texttt{b}\}$.

32.4-3

Un pattern P è detto ***non sovrapponibile*** se $P_k \sqsupset P_q$ implica $k = 0$ o $k = q$. Descrivete il diagramma degli stati di transizione dell'automa di string matching per un pattern non sovrapponibile.

32.4-4 ★

Dati due pattern P e P', descrivete come costruire un automa a stati finiti che determina tutte le occorrenze di *entrambi* i pattern. Provate a minimizzare il numero degli stati del vostro automa.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

32.4-5

Dato un pattern P che contiene i caratteri jolly (vedere l'Esercizio 32.2-4), descrivete come costruire un automa a stati finiti che può trovare una occorrenza di P in un testo T con un tempo di matching $O(n)$, dove $n = |T|$.

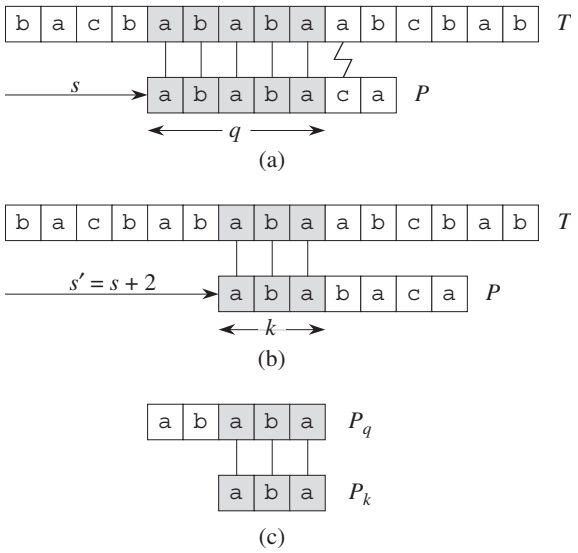


Figura 32.10 La funzione prefisso π. **(a)** Il pattern $P = \texttt{ababaca}$ è allineato con un testo T in modo che i primi $q = 5$ caratteri coincidono. I caratteri coincidenti, su sfondo grigio, sono collegati da linee verticali. **(b)** Sapendo soltanto che i primi 5 caratteri coincidono, possiamo dedurre che uno spostamento $s + 1$ non è valido, mentre uno spostamento $s' = s + 2$ è coerente con tutto ciò che sappiamo sul testo e, quindi, è potenzialmente valido. **(c)** Le informazioni utili per tali deduzioni possono essere precalcolate confrontando il pattern con sé stesso. Qui vediamo che il prefisso più lungo di P, che è anche un suffisso proprio di P_5, è P_3. Questa informazione è precalcolata e rappresentata nell'array π, quindi $\pi[5] = 3$. Dato che q caratteri coincidono quando lo spostamento è s, il successivo spostamento potenzialmente valido sarà $s' = s + (q - \pi[q])$.

⋆ 32.5 Algoritmo di Knuth-Morris-Pratt

Presentiamo adesso l'algoritmo di string matching con tempo lineare sviluppato da Knuth, Morris e Pratt. Questo algoritmo evita completamente il calcolo della funzione di transizione δ e il suo tempo di matching è $\Theta(n)$, utilizzando soltanto una funzione ausiliaria π precalcolata in base al pattern nel tempo $\Theta(m)$ e che viene memorizzata in un array $\pi[1\,..\,m]$. L'array π consente alla funzione di transizione δ di essere calcolata con efficienza (in senso ammortizzato) "al volo" quando serve. In un certo senso, per ogni stato $q = 0, 1, \ldots, m$ e ogni carattere $a \in \Sigma$, il valore $\pi[q]$ contiene quelle informazioni richieste per calcolare $\delta(q, a)$ che sono indipendenti da a (questa osservazione sarà chiarita più avanti). Poiché l'array π ha soltanto m elementi, mentre δ ha $\Theta(m\,|\Sigma|)$ elementi, si risparmia un fattore $|\Sigma|$ nel tempo di preelaborazione calcolando π, anziché δ.

La funzione prefisso per un pattern

La funzione prefisso π per un pattern racchiude le informazioni su come il pattern si confronta con i suoi stessi spostamenti. Tali informazioni possono essere utilizzate per evitare di controllare spostamenti inutili nell'algoritmo ingenuo di pattern matching o per evitare di precalcolare l'intera funzione di transizione δ per un automa di string matching.

Consideriamo il funzionamento dell'algoritmo ingenuo di string matching. La Figura 32.10(a) illustra un particolare spostamento s di una sagoma che contiene il pattern $P = \texttt{ababaca}$ a confronto con un testo T. Per questo esempio, $q = 5$ caratteri coincidono, ma il sesto carattere del pattern non coincide con il corrispondente carattere del testo T. L'informazione che q caratteri coincidono permette di identificare i corrispondenti caratteri del testo. Conoscendo questi q caratteri del testo, è possibile determinare immediatamente che certi spostamenti non sono validi. Nell'esempio della figura, lo spostamento $s + 1$ necessariamente non è valido, perché il primo carattere del pattern (a) sarebbe allineato con un carattere del testo che, come sappiamo, coincide con il secondo carattere del pattern (b). Lo spostamento $s' = s + 2$ illustrato nella parte (b) della figura, invece, allinea i primi tre caratteri del pattern con i tre caratteri del testo che devono ne-

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

cessariamente coincidere. In generale, è utile conoscere la risposta alla seguente domanda:

> Sapendo che i caratteri del pattern $P[1..q]$ coincidono con i caratteri del testo $T[s+1..s+q]$, qual è lo spostamento minimo $s' > s$ tale che
>
> $$P[1..k] = T[s'+1..s'+k] \tag{32.6}$$
>
> con $s'+k = s+q$?

In altre parole, sapendo che $P_q \sqsupset T_{s+q}$, vogliamo trovare il più lungo prefisso proprio P_k di P_q che è anche un suffisso di T_{s+q}. (Poiché $s'+k = s+q$, dati s e q, trovare il più piccolo spostamento s' è equivalente a trovare la lunghezza k del più lungo prefisso.) Sommiamo la differenza $q-k$ delle lunghezze di questi prefissi di P allo spostamento s per ottenere il nuovo spostamento s' in modo che $s' = s+(q-k)$. Nel caso migliore $k = 0$ per cui $s' = s+q$ ed eliminiamo immediatamente tutti gli spostamenti $s+1, s+2, \ldots, s+q-1$. In ogni caso, nel nuovo spostamento s' non sarà necessario confrontare i primi k caratteri di P con i corrispondenti caratteri di T, perché abbiamo la garanzia che essi coincidono per l'equazione (32.6). L'informazione richiesta può essere precalcolata confrontando il pattern con sé stesso, come illustra la Figura 32.10(c). Poiché $T[s'+1..s'+k]$ è una parte della porzione nota del testo, esso è un suffisso della stringa P_q. L'equazione (32.6) può quindi essere interpretata come una richiesta del più grande $k < q$ tale che $P_k \sqsupset P_q$. Allora, $s' = s+(q-k)$ è il prossimo spostamento potenzialmente valido. Vedremo che conviene memorizzare il numero k di caratteri coincidenti nel nuovo spostamento s', anziché memorizzare $s'-s$.

Formalizziamo il precalcolo richiesto nel modo seguente. Dato un pattern $P[1..m]$, la ***funzione prefisso*** per il pattern P è la funzione $\pi : \{1,2,\ldots,m\} \to \{0,1,\ldots,m-1\}$ tale che

$$\pi[q] = \max\{k : k < q \quad \text{e} \quad P_k \sqsupset P_q\}$$

Ovvero, $\pi[q]$ è la lunghezza del prefisso più lungo di P che è un suffisso proprio di P_q. Come altro esempio, la Figura 32.11(a) indica la funzione prefisso completa π per il pattern `abababababca`.

L'algoritmo di Knuth-Morris-Pratt è incluso nel seguente pseudocodice come procedura KMP-MATCHER. Come vedremo, è costruito principalmente sul modello della procedura FINITE-AUTOMATON-MATCHER. La procedura KMP-MATCHER chiama la procedura ausiliaria COMPUTE-PREFIX-FUNCTION per calcolare π.

```
KMP-MATCHER(T, P)
 1  n = T.length
 2  m = P.length
 3  π = COMPUTE-PREFIX-FUNCTION(P)
 4  q = 0                              // Numero di caratteri coincidenti.
 5  for i = 1 to n                     // Ispeziona il testo da sinistra a destra.
 6      while q > 0 and P[q + 1] ≠ T[i]
 7          q = π[q]                   // Il carattere successivo non coincide.
 8      if P[q + 1] == T[i]
 9          q = q + 1                  // Il carattere successivo coincide.
10      if q == m                      // Il pattern P coincide tutto?
11          stampa "Occorrenza del pattern con spostamento" i − m
12          q = π[q]                   // Cerca il prossimo match.
```

Acquistato da Michele Michele Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19[illegible] Copyright 2010, The McGraw-Hill Education (Italy)

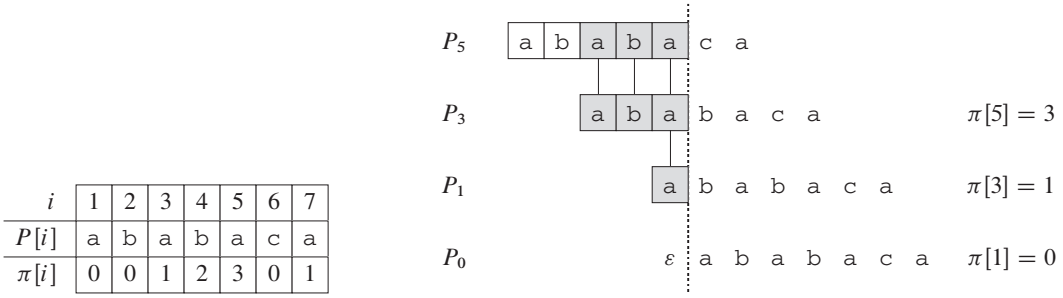


Figura 32.11 Illustrazione del Lemma 32.5 per il pattern $P = \texttt{ababaca}$ e $q = 5$. **(a)** La funzione π per il pattern dato. Poiché $\pi[5] = 3$, $\pi[3] = 1$ e $\pi[1] = 0$, iterando π si ottiene $\pi^*[5] = \{3, 1, 0\}$. **(b)** Facciamo scorrere verso destra la sagoma contenente il pattern P e osserviamo quando qualche prefisso P_k di P coincide con qualche suffisso proprio di P_5; ciò accade per $k = 3$, 1 e 0. Nella figura la prima riga indica P e la linea verticale punteggiata è tracciata esattamente dopo P_5. Le righe successive indicano tutti gli spostamenti di P per i quali qualche prefisso P_k di P coincide con qualche suffisso di P_5. I caratteri che coincidono sono rappresentati su sfondo grigio. Le linee verticali collegano i caratteri allineati che coincidono. Quindi, $\{k : k < 5 \text{ e } P_k \sqsupset P_5\} = \{3, 1, 0\}$. Il Lemma 32.5 asserisce che $\pi^*[q] = \{k : k < q \text{ e } P_k \sqsupset P_q\}$ per ogni q.

```
COMPUTE-PREFIX-FUNCTION(P)
 1  m = P.length
 2  Sia π[1..m] un nuovo array
 3  π[1] = 0
 4  k = 0
 5  for q = 2 to m
 6       while k > 0 and P[k + 1] ≠ P[q]
 7            k = π[k]
 8       if P[k + 1] == P[q]
 9            k = k + 1
10       π[q] = k
11  return π
```

Queste due procedure hanno molto in comune, in quanto entrambe confrontano una stringa con il pattern P: KMP-MATCHER confronta il testo T con P; COMPUTE-PREFIX-FUNCTION confronta P con sé stesso.

Iniziamo con l'analisi del tempo di esecuzione di queste procedure. Dimostrare la loro correttezza sarà più complicato.

Analisi del tempo di esecuzione

Il tempo di esecuzione di COMPUTE-PREFIX-FUNCTION è $\Theta(m)$, utilizzando il metodo del potenziale dell'analisi ammortizzata (vedere il Paragrafo 17.1). L'unica cosa difficile da dimostrare è che il ciclo **while** (righe 6–7) viene eseguito $O(m)$ volte complessivamente. Dimostreremo che esso esegue al più $m - 1$ iterazioni. Iniziamo facendo qualche osservazione su k. In primo luogo, la riga 4 pone k a 0, e l'unica altra riga che può modificare k è la riga 9, che aumenta k al più di un'unità durante ciascuna esecuzione del corpo del ciclo **for** (righe 5–10). Quindi, l'incremento totale di k è al più $m - 1$. In secondo luogo, poiché $k < q$ quando inizia il ciclo **for** e poiché q aumenta in ogni iterazione del corpo del ciclo **for**, la

Acquistato da Michele Michele su Webster il 2022-[illegible]

relazione $k < q$ è sempre soddisfatta. Ne consegue che le assegnazioni nelle righe 3 e 10 assicurano che $\pi[q] < q$ per ogni $q = 1, 2, \ldots, m$; questo significa che ogni iterazione del ciclo **while** riduce k. In terzo luogo, il valore di k non diventerà mai negativo. Sulla base di queste considerazioni, notiamo che la riduzione totale di k nel ciclo **while** è limitata superiormente dall'aumento totale di k in tutte le iterazioni del ciclo **for**, che è $m - 1$. Dunque, il ciclo **while** viene eseguito al più $m-1$ volte complessivamente, e la procedura COMPUTE-PREFIX-FUNCTION viene eseguita nel tempo $\Theta(m)$.

L'Esercizio 32.5-4 chiede di dimostrare, eseguendo un'analisi aggregata, che il tempo di matching della procedura KMP-MATCHER è $\Theta(n)$.

Rispetto alla procedura FINITE-AUTOMATON-MATCHER, utilizzando π anziché δ, abbiamo ridotto il tempo di preelaborazione del pattern da $O(m\,|\Sigma|)$ a $\Theta(m)$, mantenendo il limite del tempo effettivo di matching a $\Theta(n)$.

Correttezza del calcolo della funzione prefisso

Vedremo in seguito che la funzione prefisso π ci aiuterà a simulare la funzione di transizione δ in un automa di string matching. Ma prima dobbiamo dimostrare che la procedura COMPUTE-PREFIX-FUNCTION calcola correttamente la funzione prefisso. Per questo, dobbiamo trovare tutti i prefissi P_k che sono suffissi propri di un dato prefisso P_q. Il valore di $\pi[q]$ fornisce il prefisso più lungo, ma il seguente lemma, illustrato nella Figura 32.11, dimostra che, iterando la funzione prefisso π, è possibile enumerare tutti i prefissi P_k che sono suffissi propri di un dato prefisso P_q. Sia

$$\pi^*[q] = \{\pi[q], \pi^{(2)}[q], \pi^{(3)}[q], \ldots, \pi^{(t)}[q]\}$$

dove $\pi^{(i)}[q]$ è definito in termini di iterazione funzionale, cosicché $\pi^{(0)}[q] = q$ e $\pi^{(i)}[q] = \pi[\pi^{(i-1)}[q]]$ per $i \geq 1$, e dove è sottinteso che la sequenza in $\pi^*[q]$ termina quando viene raggiunto $\pi^{(t)}[q] = 0$.

Lemma 32.5 (Iterazione della funzione prefisso)
Sia P un pattern di lunghezza m con la funzione prefisso π. Allora, per $q = 1, 2, \ldots, m$, si ha $\pi^*[q] = \{k : k < q \text{ e } P_k \sqsupset P_q\}$.

Dimostrazione Dimostriamo prima che $\pi^*[q] \subseteq \{k : k < q \text{ e } P_k \sqsupset P_q\}$ ovvero

$$i \in \pi^*[q] \text{ implica } P_i \sqsupset P_q \tag{32.7}$$

Se $i \in \pi^*[q]$, allora $i = \pi^{(u)}[q]$ per qualche $u > 0$. Dimostriamo l'equazione (32.7) per induzione su u. Per $u = 1$, si ha $i = \pi[q]$, e l'asserzione è provata perché $i < q$ e $P_{\pi[q]} \sqsupset P_q$ per la definizione di π. Utilizzando le relazioni $\pi[i] < i$ e $P_{\pi[i]} \sqsupset P_i$ e la transitività di $<$ e $\sqsupset$, si dimostra l'asserzione per ogni i in $\pi^*[q]$. Pertanto, $\pi^*[q] \subseteq \{k : k < q \text{ e } P_k \sqsupset P_q\}$.

Dimostriamo adesso che $\{k : k < q \text{ e } P_k \sqsupset P_q\} \subseteq \pi^*[q]$. Supponiamo per assurdo che esistano degli interi nell'insieme $\{k : k < q \text{ e } P_k \sqsupset P_q\} - \pi^*[q]$; sia j il più grande di tali valori. Poiché $\pi[q]$ è il massimo valore in $\{k : k < q \text{ e } P_k \sqsupset P_q\}$ e $\pi[q] \in \pi^*[q]$, deve essere $j < \pi[q]$; quindi indichiamo con j' il più piccolo intero in $\pi^*[q]$ che è maggiore di j (possiamo scegliere $j' = \pi[q]$ se non c'è un altro numero in $\pi^*[q]$ che è maggiore di j). Si ha $P_j \sqsupset P_q$ perché $j \in \{k : k < q \text{ e } P_k \sqsupset P_q\}$; si ha anche $P_{j'} \sqsupset P_q$ perché $j' \in \pi^*[q]$ e per l'equazione (32.7). Quindi, $P_j \sqsupset P_{j'}$ per il Lemma 32.1 e j

Acquistato da Michele Michele su Webster il 2022-01-07 15:12 Numero Ordine Libreria 1253981016220767-0 Copyright © 2022 McGraw-Hill Education (Italy)

è il massimo valore che è minore di j' con questa proprietà. Dunque deve essere $\pi[j'] = j$ e, poiché $j' \in \pi^*[q]$, deve anche essere $j \in \pi^*[q]$. Questa contraddizione dimostra il lemma. ■

L'algoritmo COMPUTE-PREFIX-FUNCTION calcola $\pi[q]$ ordinatamente per $q = 1, 2, \ldots, m$. Il calcolo di $\pi[1] = 0$ nella riga 3 della procedura COMPUTE-PREFIX-FUNCTION è certamente corretto, perché $\pi[q] < q$ per ogni q. Il seguente lemma e il suo corollario saranno utilizzati per dimostrare che COMPUTE-PREFIX-FUNCTION calcola $\pi[q]$ correttamente per $q > 1$.

Lemma 32.6
Sia P un pattern di lunghezza m e sia π la funzione prefisso per P. Per $q = 1, 2, \ldots, m$, se $\pi[q] > 0$, allora $\pi[q] - 1 \in \pi^*[q-1]$.

Dimostrazione Se $r = \pi[q] > 0$, allora $r < q$ e $P_r \sqsupset P_q$; quindi $r - 1 < q - 1$ e $P_{r-1} \sqsupset P_{q-1}$ (escludendo l'ultimo carattere di P_r e P_q). Di conseguenza, per il Lemma 32.5, si ha $\pi[q] - 1 = r - 1 \in \pi^*[q-1]$. ■

Per $q = 2, 3, \ldots, m$, definiamo il sottoinsieme $E_{q-1} \subseteq \pi^*[q-1]$ come

$$\begin{aligned} E_{q-1} &= \{k \in \pi^*[q-1] : P[k+1] = P[q]\} \\ &= \{k : k < q-1 \text{ e } P_k \sqsupset P_{q-1} \text{ e } P[k+1] = P[q]\} \text{ (per il Lemma 32.5)} \\ &= \{k : k < q-1 \text{ e } P_{k+1} \sqsupset P_q\} \end{aligned}$$

L'insieme E_{q-1} è composto dai valori $k < q - 1$ per i quali $P_k \sqsupset P_{q-1}$ e $P_{k+1} \sqsupset P_q$, perché $P[k+1] = P[q]$. Quindi, E_{q-1} è composto da quei valori $k \in \pi^*[q-1]$ per i quali è possibile estendere P_k a P_{k+1} e ottenere un suffisso proprio di P_q.

Corollario 32.7
Sia P un pattern di lunghezza m e sia π la funzione prefisso per P. Per $q = 2, 3, \ldots, m$

$$\pi[q] = \begin{cases} 0 & \text{se } E_{q-1} = \emptyset \\ 1 + \max\{k \in E_{q-1}\} & \text{se } E_{q-1} \neq \emptyset \end{cases}$$

Dimostrazione Se E_{q-1} è vuoto, non esiste un valore $k \in \pi^*[q-1]$ (incluso $k = 0$) per il quale è possibile estendere P_k a P_{k+1} e ottenere un suffisso proprio di P_q. Quindi $\pi[q] = 0$.

Se E_{q-1} non è vuoto, allora per ogni $k \in E_{q-1}$ si ha $k + 1 < q$ e $P_{k+1} \sqsupset P_q$. Quindi, dalla definizione di $\pi[q]$, si ha

$$\pi[q] \geq 1 + \max\{k \in E_{q-1}\} \tag{32.8}$$

Notate che $\pi[q] > 0$. Sia $r = \pi[q] - 1$, cosicché $r + 1 = \pi[q]$. Poiché $r + 1 > 0$, si ha $P[r+1] = P[q]$. Inoltre, per il Lemma 32.6, si ha $r \in \pi^*[q-1]$. Di conseguenza $r \in E_{q-1}$, e così $r \leq \max\{k \in E_{q-1}\}$ ovvero

Acquistato da Michele Michele su Webster il 2022-07-07 20:12 Numero Ordine Libreria: 1035838162 2207070 Copyright © 2022 McGraw-Hill Education (Italy)

$$\pi[q] \leq 1 + \max\{k \in E_{q-1}\} \tag{32.9}$$

Combinando le equazioni (32.8) e (32.9) completiamo la dimostrazione. ■

Adesso completiamo la dimostrazione che COMPUTE-PREFIX-FUNCTION calcola correttamente π. Nella procedura COMPUTE-PREFIX-FUNCTION, all'inizio

di ogni iterazione del ciclo **for** (righe 5–10), si ha che $k = \pi[q-1]$. Questa condizione è forzata dalle righe 3 e 4 quando il ciclo viene eseguito per la prima volta e resta vera in ogni successiva iterazione del ciclo a causa della riga 10. Le righe 6–9 elaborano k in modo tale che assuma il valore corretto di $\pi[q]$. Il ciclo nelle righe 6–7 esamina tutti i valori $k \in \pi^*[q-1]$ finché non trova quello per cui $P[k+1] = P[q]$; a questo punto k è il più grande valore nell'insieme E_{q-1}, quindi, per il Corollario 32.7, possiamo assegnare a $\pi[q]$ il valore $k+1$. Se un tale valore di k non viene trovato, $k = 0$ nella riga 8. Se $P[1] = P[q]$, allora dovremmo assegnare il valore 1 a k e a $\pi[q]$; altrimenti dovremmo lasciare inalterato k e assegnare il valore 0 a $\pi[q]$. Le righe 8–10 assegnano correttamente i valori di k e $\pi[q]$ in entrambi i casi. Questo completa la dimostrazione della correttezza di COMPUTE-PREFIX-FUNCTION.

Correttezza dell'algoritmo KMP

La procedura KMP-MATCHER può essere vista come una nuova implementazione della procedura FINITE-AUTOMATON-MATCHER, utilizzando però la funzione prefisso π per calcolare le transizioni di stato. Specificatamente, dimostreremo che nella i-esima iterazione dei cicli **for** di entrambe le procedure KMP-MATCHER e FINITE-AUTOMATON-MATCHER, lo stato q ha lo stesso valore quando verifichiamo l'uguaglianza con m (nella riga 10 di KMP-MATCHER e nella riga 5 di FINITE-AUTOMATON-MATCHER). Una volta dimostrato che KMP-MATCHER simula il comportamento di FINITE-AUTOMATON-MATCHER, la correttezza di KMP-MATCHER segue dalla correttezza di FINITE-AUTOMATON-MATCHER (spiegheremo più avanti perché è necessaria la riga 12 in KMP-MATCHER).

Prima di dimostrare formalmente che KMP-MATCHER simula correttamente FINITE-AUTOMATON-MATCHER, è necessario capire come la funzione prefisso π rimpiazza la funzione di transizione δ. Ricordiamo che, quando un automa di string matching è nello stato q e ispeziona un carattere $a = T[i]$, esso passa in un nuovo stato $\delta(q, a)$. Se $a = P[q+1]$, a continua a coincidere con il pattern, e quindi $\delta(q, a) = q + 1$. Altrimenti, se $a \neq P[q+1]$, a non coincide con il pattern, e quindi $0 \leq \delta(q, a) \leq q$. Nel primo caso, quando a continua a coincidere con il pattern, KMP-MATCHER passa nello stato $q+1$ senza fare riferimento alla funzione π: il test del ciclo **while** nella riga 6 risulta falso la prima volta, il test nella riga 8 risulta vero, e la riga 9 aumenta q.

La funzione π entra in gioco quando il carattere a non coincide con il pattern, cosicché il nuovo stato $\delta(q, a)$ può essere q o a sinistra di q lungo la spina dorsale dell'automa. Il ciclo **while** (righe 6–7) di KMP-MATCHER itera gli stati in $\pi^*[q]$, fermandosi quando arriva in uno stato, per esempio q', tale che a coincide con $P[q'+1]$ oppure q' ha raggiunto il valore 0. Se a coincide con $P[q'+1]$, allora la riga 9 imposta il nuovo stato a $q'+1$, che deve essere uguale a $\delta(q, a)$ affinché la simulazione funzioni correttamente. In altre parole, il nuovo stato $\delta(q, a)$ può essere lo stato 0 o il successivo di qualche stato in $\pi^*[q]$.

Acquistato da Michele Mastrogiovanni - Copia personale non cedibile - ISBN 9788838670121 - Copyright © 2022, McGraw-Hill Education (Italy)

Esaminiamo l'esempio delle Figure 32.7 e 32.11, che riguardano il pattern $P = \texttt{ababaca}$. Supponiamo che l'automa sia nello stato $q = 5$; gli stati in $\pi^*[5]$ sono, in ordine decrescente, 3, 1 e 0. Se il successivo carattere ispezionato è `c`, allora possiamo facilmente notare che l'automa passa allo stato $\delta(5, \texttt{c}) = 6$ in entrambe le procedure FINITE-AUTOMATON-MATCHER e KMP-MATCHER. Adesso supponiamo che il successivo carattere ispezionato sia `b`; in questo caso, l'automa passa allo stato $\delta(5, \texttt{b}) = 4$. Il ciclo **while** di KMP-MATCHER termina

avendo eseguito la riga 7 una sola volta, e arriva nello stato $q' = \pi[5] = 3$. Poiché $P[q'+1] = P[4] = \texttt{b}$, il test nella riga 8 risulta vero, e KMP-MATCHER passa al nuovo stato $q' + 1 = 4 = \delta(5, \texttt{b})$. Infine, supponiamo che il successivo carattere ispezionato sia a; in questo caso, l'automa passa allo stato $\delta(5, \texttt{a}) = 1$. Le prime tre volte che viene eseguito il test nella riga 6, il test risulta vero. La prima volta si ha $P[6] = \texttt{c} \neq \texttt{a}$, e KMP-MATCHER passa allo stato $\pi[5] = 3$ (il primo stato in $\pi^*[5]$). La seconda volta, si ha $P[4] = \texttt{b} \neq \texttt{a}$ e si passa allo stato $\pi[3] = 1$ (il secondo stato in $\pi^*[5]$). La terza volta si ha $P[2] = \texttt{b} \neq \texttt{a}$ e si passa allo stato $\pi[1] = 0$ (l'ultimo stato in $\pi^*[5]$). Il ciclo **while** termina quando arriva nello stato $q' = 0$. Adesso, la riga 8 trova che $P[q' + 1] = P[1] = \texttt{a}$, e la riga 9 muove l'automa nel nuovo stato $q' + 1 = 1 = \delta(5, \texttt{a})$.

Quindi, si deduce che KMP-MATCHER itera gli stati in $\pi^*[q]$ in ordine decrescente, fermandosi in qualche stato q' e poi passando eventualmente allo stato $q' + 1$. Sebbene questo possa sembrare un lavoro complesso solo per simulare il calcolo di $\delta(q, a)$, ricordiamo che, asintoticamente, la procedura KMP-MATCHER non è più lenta di FINITE-AUTOMATON-MATCHER.

A questo punto possiamo dimostrare formalmente la correttezza dell'algoritmo di Knuth-Morris-Pratt. Per il Teorema 32.4, si ha che $q = \sigma(T_i)$ ogni volta che viene eseguita la riga 4 di FINITE-AUTOMATON-MATCHER. Quindi, basta dimostrare che la stessa proprietà è valida anche per il ciclo **for** di KMP-MATCHER. La dimostrazione si svolge per induzione sul numero di iterazioni del ciclo. Inizialmente, entrambe le procedure impostano q a 0 quando eseguono per la prima volta il loro ciclo **for**. Consideriamo l'iterazione i del ciclo **for** di KMP-MATCHER e supponiamo che q' sia lo stato all'inizio di questa iterazione. Per l'ipotesi induttiva, si ha $q' = \sigma(T_{i-1})$. Dobbiamo dimostrare che $q = \sigma(T_i)$ nella riga 10. (Anche qui dobbiamo trattare a parte la riga 12.)

Quando consideriamo il carattere $T[i]$, il più lungo prefisso di P che è un suffisso di T_i è $P_{q'+1}$ (se $P[q' + 1] = T[i]$) o qualche prefisso (non necessariamente proprio, e forse vuoto) di $P_{q'}$. Esaminiamo separatamente i tre casi in cui $\sigma(T_i) = 0$, $\sigma(T_i) = q' + 1$ e $0 < \sigma(T_i) \leq q'$.

- Se $\sigma(T_i) = 0$, allora $P_0 = \varepsilon$ è l'unico prefisso di P che è un suffisso di T_i. Il ciclo **while** (righe 6–7) itera i valori in $\pi^*[q']$, ma sebbene $P_q \sqsupset T_i$ per ogni $q \in \pi^*[q']$, il ciclo non trova mai uno stato q tale che $P[q + 1] = T[i]$. Il ciclo termina quando q raggiunge il valore 0, e ovviamente la riga 9 non viene eseguita. Quindi, $q = 0$ nella riga 10, e quindi $q = \sigma(T_i)$.
- Se $\sigma(T_i) = q' + 1$, allora $P[q' + 1] = T[i]$, e il test del ciclo **while** nella riga 6 fallisce la prima volta. Viene eseguita la riga 9 che incrementa q, cosicché dopo si ha $q = q' + 1 = \sigma(T_i)$.
- Se $0 < \sigma(T_i) \leq q'$, allora il ciclo **while** (righe 6–7) viene eseguito almeno una volta, controllando in ordine decrescente ciascun valore $q \in \pi^*[q']$; poi sarà interrotto per qualche $q < q'$. Quindi, P_q è il più lungo prefisso di $P_{q'}$ per il quale $P[q + 1] = T[i]$, cosicché quando il ciclo **while** termina, $q + 1 = \sigma(P_{q'}T[i])$. Poiché $q' = \sigma(T_{i-1})$, il Lemma 32.3 implica che $\sigma(T_{i-1}T[i]) = \sigma(P_{q'}T[i])$. Dunque, alla fine del ciclo **while** si ha

$$\begin{aligned} q + 1 &= \sigma(P_{q'}T[i]) \\ &= \sigma(T_{i-1}T[i]) \\ &= \sigma(T_i) \end{aligned}$$

Dopo che la riga 9 ha incrementato q, si ha $q = \sigma(T_i)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995030l6-220707 0 Copyright © 2022, McGraw-Hill Education (Italy)

La riga 12 è necessaria in KMP-MATCHER per evitare un possibile riferimento a $P[m+1]$ nella riga 6, dopo che è stata trovata un'occorrenza di P. (La prova che $q = \sigma(T_{i-1})$ continua a valere nella successiva esecuzione della riga 6 deriva dal suggerimento dato nell'Esercizio 32.5-8: $\delta(m, a) = \delta(\pi[m], a)$ ovvero $\sigma(Pa) = \sigma(P_{\pi[m]}a)$ per qualsiasi $a \in \Sigma$.) Le altre considerazioni per concludere sulla correttezza dell'algoritmo di Knuth-Morris-Pratt derivano dalla correttezza di FINITE-AUTOMATON-MATCHER, perché abbiamo appena visto che KMP-MATCHER simula il comportamento di FINITE-AUTOMATON-MATCHER.

Esercizi

32.5-1

Calcolate la funzione prefisso π per il pattern `ababbabbabbababbabb`.

32.5-2

Trovate un limite superiore per la dimensione di $\pi^*[q]$ come una funzione di q. Dimostrate con un esempio che il vostro limite è stretto.

32.5-3

Spiegate come determinare le occorrenze del pattern P nel testo T esaminando la funzione π per la stringa PT (la stringa di lunghezza $m + n$ che è la concatenazione di P e T).

32.5-4

Svolgete un'analisi aggregata per dimostrare che il tempo di esecuzione di KMP-MATCHER è $\Theta(n)$.

32.5-5

Utilizzate una funzione potenziale per dimostrare che il tempo di esecuzione di KMP-MATCHER è $\Theta(n)$.

32.5-6

Spiegate come migliorare KMP-MATCHER sostituendo l'occorrenza di π nella riga 7 (non nella riga 12) con π', dove π' è definito ricorsivamente per $q = 1, 2, \ldots, m$ dall'equazione

$$\pi'[q] = \begin{cases} 0 & \text{se } \pi[q] = 0 \\ \pi'[\pi[q]] & \text{se } \pi[q] \neq 0 \text{ e } P[\pi[q]+1] = P[q+1] \\ \pi[q] & \text{se } \pi[q] \neq 0 \text{ e } P[\pi[q]+1] \neq P[q+1] \end{cases}$$

Spiegate perché l'algoritmo modificato è corretto e perché questa modifica costituisce un miglioramento.

32.5-7

Create un algoritmo con tempo lineare per determinare se un testo T è una rotazione ciclica di un'altra stringa T'. Per esempio, le stringhe `arc` e `car` sono rotazioni cicliche l'una dell'altra.

Acquistato da Michele Mazzucco Worcester il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

32.5-8 ★

Create un algoritmo efficiente per calcolare la funzione di transizione δ per l'automa di string matching che corrisponde a un dato pattern P. L'algoritmo dovrebbe essere eseguito nel tempo $O(m\,|\Sigma|)$ (*suggerimento:* dimostrate che $\delta(q, a) = \delta(\pi[q], a)$ se $q = m$ o $P[q+1] \neq a$).

Problemi

32-1 String matching basato sui fattori di ripetizione

Indichiamo con y^i la concatenazione della stringa y con sé stessa, ripetuta i volte. Per esempio, $(\texttt{ab})^3 = \texttt{ababab}$. Si dice che una stringa $x \in \Sigma^*$ ha un ***fattore di ripetizione*** r se $x = y^r$ per qualche stringa $y \in \Sigma^*$ e qualche $r > 0$. Indichiamo con $\rho(x)$ il più grande valore di r tale che x abbia un fattore di ripetizione r.

a. Create un algoritmo efficiente che riceve come input un pattern $P[1\,..\,m]$ e calcola il valore $\rho(P_i)$ per $i = 1, 2, \ldots, m$. Qual è il tempo di esecuzione dell'algoritmo?

b. Per ogni pattern $P[1\,..\,m]$, definite $\rho^*(P)$ come $\max_{1 \le i \le m} \rho(P_i)$. Dimostrate che se il pattern P è scelto a caso dall'insieme di tutte le stringhe binarie di lunghezza m, allora il valore atteso di $\rho^*(P)$ è $O(1)$.

c. Dimostrate che il seguente algoritmo trova correttamente tutte le occorrenze del pattern P in un testo $T[1\,..\,n]$ nel tempo $O(\rho^*(P)n + m)$.

```
REPETITION-MATCHER(P, T)
 1  m = P.length
 2  n = T.length
 3  k = 1 + ρ*(P)
 4  q = 0
 5  s = 0
 6  while s ≤ n − m
 7      if T[s + q + 1] == P[q + 1]
 8          q = q + 1
 9          if q == m
10              stampa "Occorrenza del pattern
                                con spostamento" s
11      if q == m or T[s + q + 1] ≠ P[q + 1]
12          s = s + max(1, ⌈q/k⌉)
13          q = 0
```

L'algoritmo è stato creato da Galil e Seiferas. Estendendo ulteriormente queste idee, hanno ottenuto un algoritmo di string matching con tempo lineare che richiede una quantità di memoria di appena $O(1)$, oltre alla memoria richiesta per P e T.

Note

La relazione tra string matching e teoria degli automi a stati finiti è stata descritta da Aho, Hopcroft e Ullman [5]. L'algoritmo di Knuth-Morris-Pratt [215] è stato scoperto in maniera autonoma da Knuth e Pratt e da Morris, che poi hanno pubblicato congiuntamente il loro lavoro. Reingold, Urban e Gries [295] hanno fornito una versione alternativa dell'algoritmo di Knuth-Morris-Pratt. L'algoritmo di Rabin-Karp è stato ideato da Rabin e Karp [202]. Galil e Seiferas [127] descrivono un interessante algoritmo di string matching deterministico con tempo lineare che usa una quantità di memoria di appena $O(1)$, oltre alla memoria richiesta per memorizzare il pattern e il testo.

Acquistato da Michele Mirabella su Virgilio (10.0.2.2) il 2021-02-12 19:49:00 UTC - Copia concessa in licenza a 1 utente. Copia non cedibile a terzi. Copyright The McGraw-Hill Companies (Italy)

Geometria computazionale 33

La geometria computazionale è la branca dell'informatica che studia gli algoritmi per risolvere i problemi geometrici; trova applicazioni in vari campi dell'ingegneria e della matematica moderne, quali la grafica computerizzata, la robotica, la progettazione VLSI, la progettazione CAD, la modellistica molecolare, la metallurgia, la manifattura, il progetto di tessuti, la silvicoltura e la statistica. L'input di un problema di geometria computazionale tipicamente è la descrizione di un insieme di oggetti geometrici, come un insieme di punti, un insieme di segmenti di retta o i vertici di un poligono in ordine antiorario. L'output di solito è una risposta a una domanda sugli oggetti geometrici, per esempio se qualche retta interseca un'altra retta, oppure potrebbe essere un nuovo oggetto geometrico, come l'involucro convesso di un insieme di punti (che è il più piccolo poligono convesso che include i punti).

In questo capitolo, esamineremo alcuni algoritmi di geometria computazionale in due dimensioni, cioè nel piano. Ogni oggetto di input è rappresentato come un insieme di punti $\{p_1, p_2, p_3, \ldots\}$, dove ogni $p_i = (x_i, y_i)$ e $x_i, y_i \in \mathbb{R}$. Per esempio, un poligono P di n vertici è rappresentato dalla sequenza $\langle p_0, p_1, p_2, \ldots, p_{n-1}\rangle$ dei suoi vertici nell'ordine in cui appaiono lungo il perimetro di P. La geometria computazionale può anche occuparsi di problemi nello spazio tridimensionale e perfino in spazi di dimensione maggiore, ma tali problemi e le loro soluzioni potrebbero essere molto difficili da visualizzare. Comunque, anche limitandoci al piano, potremo vedere alcune importanti tecniche di geometria computazionale.

Il Paragrafo 33.1 spiega come rispondere in modo efficiente e preciso ad alcune domande elementari sui segmenti di retta: se un segmento è ruotato in senso orario o antiorario rispetto a un altro segmento con cui ha un estremo in comune, in quale direzione svoltiamo quando percorriamo due segmenti consecutivi e se due segmenti si intersecano. Il Paragrafo 33.2 presenta una tecnica detta "sweeping" (spazzolamento) che utilizzeremo per sviluppare un algoritmo con tempo $O(n \lg n)$ che determina se ci sono intersezioni in un insieme di n segmenti di retta. Il Paragrafo 33.3 descrive due algoritmi con "sweeping rotazionale" che calcolano l'involucro convesso di un insieme di n punti: l'algoritmo di Graham, che viene eseguito nel tempo $O(n \lg n)$, e l'algoritmo di Jarvis, che richiede un tempo $O(nh)$, dove h è il numero di vertici dell'involucro convesso. Infine, il Paragrafo 33.4 descrive un algoritmo divide et impera con tempo $O(n \lg n)$ che trova la coppia di punti più vicini in un insieme di n punti nel piano.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

33.1 Proprietà dei segmenti di retta

Molti degli algoritmi di geometria computazionale descritti in questo capitolo richiedono delle risposte a domande sulle proprietà dei segmenti di retta. Una ***combinazione convessa*** di due punti distinti $p_1 = (x_1, y_1)$ e $p_2 = (x_2, y_2)$ è un punto

qualsiasi $p_3 = (x_3, y_3)$ tale che, per qualche α nell'intervallo $0 \leq \alpha \leq 1$, si abbia $x_3 = \alpha x_1 + (1-\alpha)x_2$ e $y_3 = \alpha y_1 + (1-\alpha)y_2$. Scriveremo anche che $p_3 = \alpha p_1 + (1-\alpha)p_2$. Intuitivamente, p_3 è un punto qualsiasi che appartiene alla retta che passa per p_1 e p_2 e può coincidere con i punti p_1 e p_2 o essere compreso tra questi due punti. Dati due punti distinti p_1 e p_2, il ***segmento di retta*** $\overline{p_1 p_2}$ è l'insieme delle combinazioni convesse di p_1 e p_2. I punti p_1 e p_2 sono gli ***estremi*** del segmento $\overline{p_1 p_2}$. Se è importante l'ordine di successione degli estremi p_1 e p_2, allora parleremo di ***segmento orientato*** $\overrightarrow{p_1 p_2}$. Se p_1 è l'***origine*** $(0, 0)$ del piano cartesiano, possiamo considerare il segmento orientato $\overrightarrow{p_1 p_2}$ come il ***vettore*** p_2.

In questo paragrafo, daremo le risposte alle seguenti domande:

1. Dati due segmenti orientati $\overrightarrow{p_0 p_1}$ e $\overrightarrow{p_0 p_2}$, il segmento $\overrightarrow{p_0 p_1}$ segue in senso orario $\overrightarrow{p_0 p_2}$ nella rotazione attorno al loro estremo comune p_0?
2. Dati due segmenti di retta $\overline{p_0 p_1}$ e $\overline{p_1 p_2}$, se percorriamo $\overline{p_0 p_1}$ e poi $\overline{p_1 p_2}$, effettuiamo una svolta a sinistra nel punto p_1?
3. I segmenti di retta $\overline{p_1 p_2}$ e $\overline{p_3 p_4}$ si intersecano?

Non ci sono restrizioni sui punti dati.

È possibile rispondere a ogni domanda nel tempo $O(1)$; questo non dovrebbe sorprendere in quanto la dimensione dell'input di ogni domanda è $O(1)$. Inoltre i nostri metodi utilizzeranno soltanto addizioni, sottrazioni, moltiplicazioni e confronti. Non avremo bisogno né di divisioni né di funzioni trigonometriche, perché il loro calcolo è un'operazione computazionalmente costosa e potrebbe causare errori di arrotondamento. Per esempio, il metodo "semplice" per determinare se due segmenti si intersecano – calcolare l'equazione della retta nella forma $y = mx + b$ per ciascun segmento (m è il coefficiente angolare o pendenza della retta e b è l'intercetta dell'asse y), trovare il punto di intersezione delle rette e verificare se questo punto si trova su entrambi i segmenti – usa la divisione per trovare il punto di intersezione. Quando i segmenti sono quasi paralleli, questo metodo è molto sensibile alla precisione dell'operazione di divisione nei calcolatori reali. Il metodo in questo paragrafo, che evita l'operazione di divisione, è molto più preciso.

Prodotto vettoriale

Il calcolo del prodotto vettoriale è fondamentale nei metodi di geometria computazionale che elaborano i segmenti di retta. Consideriamo i vettori p_1 e p_2, illustrati nella Figura 33.1(a). Il ***prodotto vettoriale*** $p_1 \times p_2$ può essere interpretato come l'area con segno del parallelogramma formato dai punti $(0, 0)$, p_1, p_2 e $p_1 + p_2 = (x_1 + x_2, y_1 + y_2)$. Un'equivalente, ma più utile, definizione indica il prodotto vettoriale come il determinante di una matrice:[1]

$$\begin{aligned} p_1 \times p_2 &= \det\begin{pmatrix} x_1 & x_2 \\ y_1 & y_2 \end{pmatrix} \\ &= x_1 y_2 - x_2 y_1 \\ &= -p_2 \times p_1 \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[1] In effetti, il prodotto vettoriale è un concetto tridimensionale. È un vettore che è perpendicolare sia a p_1 sia a p_2 secondo la "regola della mano destra" e la cui ampiezza è $|x_1 y_2 - x_2 y_1|$. Tuttavia, in questo capitolo considereremo, per comodità, il prodotto vettoriale semplicemente come il valore $x_1 y_2 - x_2 y_1$.

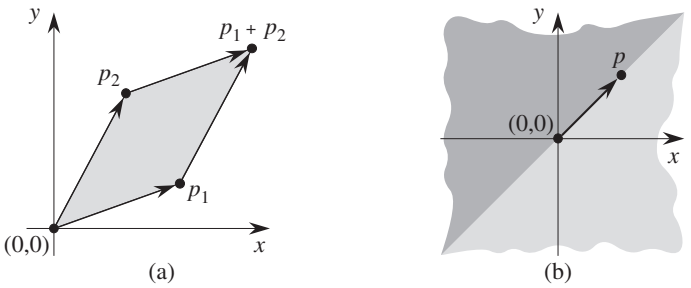


Figura 33.1
(a) Il prodotto vettoriale dei vettori p_1 e p_2 è l'area con segno del parallelogramma. **(b)** La zona di colore grigio chiaro contiene i vettori che seguono in senso orario il vettore p. La zona di colore grigio scuro contiene i vettori seguono in senso antiorario il vettore p.

Se il prodotto vettoriale $p_1 \times p_2$ è positivo, allora p_1 segue in senso orario p_2 nella rotazione attorno all'origine $(0, 0)$; se questo prodotto vettoriale è negativo, allora p_1 segue in senso antiorario p_2 (vedere l'Esercizio 33.1-1). La Figura 33.1(b) illustra le zone orarie e antiorarie riferite a un vettore p. Un caso limite si verifica quando il prodotto vettoriale è 0; in tal caso, i vettori sono ***collineari***, in quanto puntano nella stessa direzione o in direzioni opposte.

Per determinare se un segmento orientato $\overrightarrow{p_0p_1}$ segue in senso orario un segmento orientato $\overrightarrow{p_0p_2}$ nella rotazione attorno al loro estremo comune p_0, effettuiamo semplicemente una traslazione in modo che p_0 diventi l'origine degli assi cartesiani. Ovvero indichiamo con $p_1 - p_0$ il vettore $p_1' = (x_1', y_1')$, dove $x_1' = x_1 - x_0$ e $y_1' = y_1 - y_0$, e definiamo $p_2 - p_0$ in maniera analoga. Poi calcoliamo il prodotto vettoriale

$$(p_1 - p_0) \times (p_2 - p_0) = (x_1 - x_0)(y_2 - y_0) - (x_2 - x_0)(y_1 - y_0)$$

Se questo prodotto vettoriale è positivo, allora $\overrightarrow{p_0p_1}$ segue in senso orario $\overrightarrow{p_0p_2}$; se il prodotto è negativo, $\overrightarrow{p_0p_1}$ segue in senso antiorario $\overrightarrow{p_0p_2}$.

Determinare se due segmenti consecutivi svoltano a destra o a sinistra

La prossima domanda è se due segmenti consecutivi $\overline{p_0p_1}$ e $\overline{p_1p_2}$ svoltano a sinistra o a destra nel punto p_1. Ovvero occorre un metodo per determinare in quale senso ruota un dato angolo $\angle p_0p_1p_2$. I prodotti vettoriali consentono di rispondere a questa domanda senza calcolare l'angolo. Come illustra la Figura 33.2, semplicemente verifichiamo se il segmento orientato $\overrightarrow{p_0p_2}$ è ruotato in senso orario o antiorario rispetto al segmento orientato $\overrightarrow{p_0p_1}$. Per fare questo, calcoliamo il prodotto vettoriale $(p_2 - p_0) \times (p_1 - p_0)$. Se il segno di questo prodotto vettoriale è negativo, allora $\overrightarrow{p_0p_2}$ è antiorario rispetto a $\overrightarrow{p_0p_1}$, e quindi si ha una svolta a sinistra nel punto p_1. Un prodotto vettoriale positivo indica una rotazione in senso orario e, quindi, una svolta a destra. Un prodotto vettoriale nullo significa che i punti p_0, p_1 e p_2 sono collineari.

Determinare se due segmenti si intersecano

Per determinare se due segmenti di retta si intersecano, verifichiamo se ciascuno dei due segmenti taglia la retta che contiene l'altro segmento. Un segmento $\overline{p_1p_2}$ ***taglia*** una retta se il punto p_1 giace da una parte della retta e il punto p_2 giace dall'altra parte. Un caso limite si verifica quando p_1 o p_2 giacciono entrambi sulla retta. Due segmenti di retta si interescano se e soltanto se una o entrambe le seguenti condizioni sono soddisfatte:

1. Ogni segmento taglia la retta che contiene l'altro segmento.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

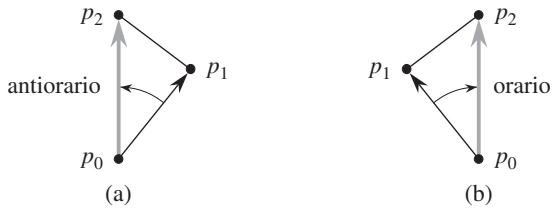


Figura 33.2 Utilizzare il prodotto vettoriale per determinare in quale direzione svoltano due segmenti consecutivi $\overline{p_0p_1}$ e $\overline{p_1p_2}$ nel punto p_1. Verifichiamo se il segmento orientato $\overrightarrow{p_0p_2}$ segue in senso orario o antiorario il segmento orientato $\overrightarrow{p_0p_1}$. **(a)** Se il senso è antiorario, si ha una svolta a sinistra nel punto p_1. **(b)** Se il senso è orario, si ha una svolta a destra nel punto p_1.

2. Un estremo di un segmento giace sull'altro segmento (questa condizione segue dal caso limite).

Le seguenti procedure implementano queste idee. SEGMENTS-INTERSECT restituisce TRUE se i segmenti $\overline{p_1p_2}$ e $\overline{p_3p_4}$ si intersecano, altrimenti restituisce FALSE. Chiama le subroutine DIRECTION, che calcola la direzione relativa (oraria o antioraria) utilizzando il metodo del prodotto vettoriale precedentemente descritto, e ON-SEGMENT, che verifica se un punto, che si suppone collineare con un segmento, giace su questo segmento.

```
SEGMENTS-INTERSECT(p1, p2, p3, p4)
 1  d1 = DIRECTION(p3, p4, p1)
 2  d2 = DIRECTION(p3, p4, p2)
 3  d3 = DIRECTION(p1, p2, p3)
 4  d4 = DIRECTION(p1, p2, p4)
 5  if ((d1 > 0 and d2 < 0) or (d1 < 0 and d2 > 0)) and
        ((d3 > 0 and d4 < 0) or (d3 < 0 and d4 > 0))
 6      return TRUE
 7  elseif d1 == 0 and ON-SEGMENT(p3, p4, p1)
 8      return TRUE
 9  elseif d2 == 0 and ON-SEGMENT(p3, p4, p2)
10      return TRUE
11  elseif d3 == 0 and ON-SEGMENT(p1, p2, p3)
12      return TRUE
13  elseif d4 == 0 and ON-SEGMENT(p1, p2, p4)
14      return TRUE
15  else return FALSE

DIRECTION(pi, pj, pk)
1  return (pk − pi) × (pj − pi)

ON-SEGMENT(pi, pj, pk)
1  if min(xi, xj) ≤ xk ≤ max(xi, xj) and min(yi, yj) ≤ yk ≤ max(yi, yj)
2      return TRUE
3  else return FALSE
```

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

La procedura SEGMENTS-INTERSECT funziona nel modo seguente. Le righe 1–4 calcolano la direzione relativa d_i di ciascun estremo p_i rispetto all'altro segmento. Se tutte le direzioni relative sono diverse da zero, allora possiamo facilmente determinare se i segmenti $\overline{p_1p_2}$ e $\overline{p_3p_4}$ si intersecano, nel modo seguente. Il segmento $\overline{p_1p_2}$ taglia la retta che contiene il segmento $\overline{p_3p_4}$ se i segmenti orientati $\overrightarrow{p_3p_1}$ e $\overrightarrow{p_3p_2}$ hanno direzioni opposte rispetto a $\overrightarrow{p_3p_4}$. In questo caso, i segni di

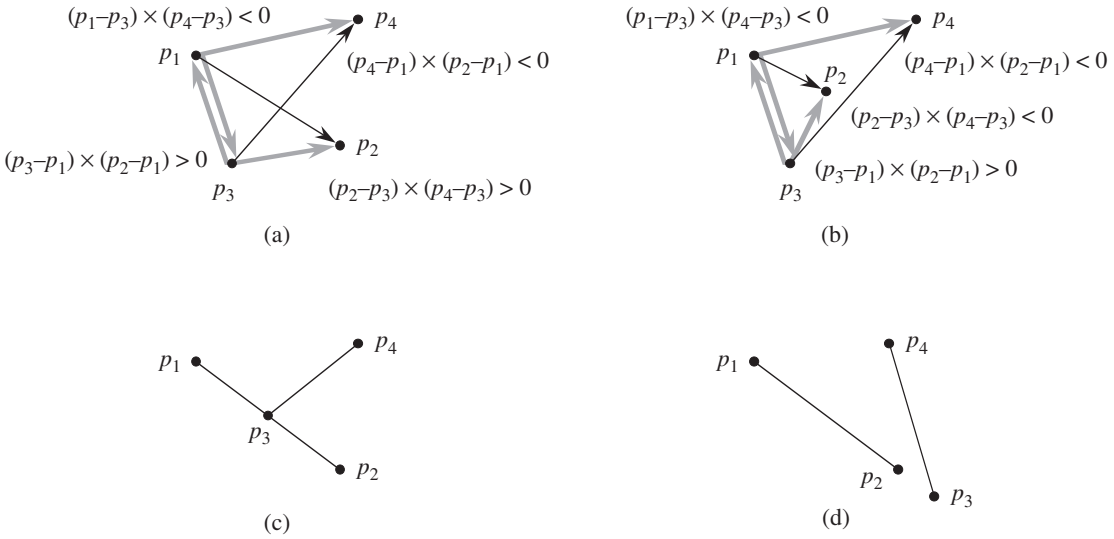


Figura 33.3 I casi previsti dalla procedura SEGMENTS-INTERSECT. **(a)** I segmenti $\overline{p_1 p_2}$ e $\overline{p_3 p_4}$ tagliano l'uno la retta dell'altro. Poiché il $\overline{p_3 p_4}$ taglia la retta che contiene il $\overline{p_1 p_2}$, i segni dei prodotti vettoriali $(p_3 - p_1) \times (p_2 - p_1)$ e $(p_4 - p_1) \times (p_2 - p_1)$ sono diversi. Poiché $\overline{p_1 p_2}$ taglia la retta che contiene $\overline{p_3 p_4}$, i segni dei prodotti vettoriali $(p_1-p_3)\times(p_4-p_3)$ e $(p_2-p_3)\times(p_4-p_3)$ sono diversi. **(b)** Il segmento $\overline{p_3 p_4}$ taglia la retta che contiene $\overline{p_1 p_2}$, ma $\overline{p_1 p_2}$ non taglia la retta che contiene $\overline{p_3 p_4}$. I segni dei prodotti vettoriali $(p_1 - p_3) \times (p_4 - p_3)$ e $(p_2 - p_3) \times (p_4 - p_3)$ sono uguali. **(c)** Il punto p_3 è collineare con $\overline{p_1 p_2}$ ed è compreso tra p_1 e p_2. **(d)** Il punto p_3 è collineare con $\overline{p_1 p_2}$, ma non è compreso tra p_1 e p_2. I segmenti non si intersecano.

d_1 e d_2 sono diversi. Analogamente, $\overline{p_3 p_4}$ taglia la retta che contiene $\overline{p_1 p_2}$ se i segni di d_3 e d_4 sono diversi. Se il test nella riga 5 risulta vero, allora i segmenti si tagliano, e SEGMENTS-INTERSECT restituisce TRUE. La Figura 33.3(a) illustra questo caso. Negli altri casi un segmento non taglia la retta dell'altro segmento, anche se si può verificare un caso limite. Se tutte le direzioni relative sono diverse da zero, non si può verificare il caso limite. Allora tutti i test $d_i = 0$ nelle righe 7–13 falliscono e SEGMENTS-INTERSECT restituisce FALSE nella riga 15. La Figura 33.3(b) illustra questo caso.

Un caso limite si verifica se una direzione relativa qualsiasi d_k è 0. Qui sappiamo che p_k è collineare con l'altro segmento. Giace sull'altro segmento se e soltanto se è compreso tra gli estremi di tale segmento. La procedura ON-SEGMENT indica se p_k è compreso tra gli estremi del segmento $\overline{p_i p_j}$, che sarà l'altro segmento se la procedura viene chiamata nelle righe 7–13; la procedura assume che p_k sia collineare con il segmento $\overline{p_i p_j}$. Le Figure 33.3(c) e (d) illustrano i casi con i punti collineari. Nella Figura 33.3(c), p_3 si trova su $\overline{p_1 p_2}$, e quindi SEGMENTS-INTERSECT restituisce TRUE nella riga 12. Poiché nella Figura 33.3(d) non ci sono estremi di un segmento su un altro segmento, la procedura SEGMENTS-INTERSECT restituisce FALSE nella riga 15.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Altre applicazioni dei prodotti vettoriali

Nei successivi paragrafi di questo capitolo presenteremo altri impieghi dei prodotti vettoriali. Nel Paragrafo 33.3 avremo bisogno di ordinare un insieme di punti secondo i loro angoli polari rispetto a una data origine. Come chiede di dimostrare

l'Esercizio 33.1-3, i prodotti vettoriali possono essere utilizzati per eseguire dei confronti nella procedura di ordinamento. Nel Paragrafo 33.2 utilizzeremo gli alberi rosso-neri per mantenere l'ordinamento verticale di un insieme di segmenti di retta. Anziché mantenere esplicitamente i valori delle chiavi, sostituiremo ogni confronto tra chiavi nel codice dell'albero rosso-nero con il calcolo di un prodotto vettoriale per determinare quale dei due segmenti che interseca una data retta verticale si trova sopra l'altro segmento.

Esercizi

33.1-1

Dimostrate che, se il prodotto $p_1 \times p_2$ è positivo, allora il vettore p_1 segue in senso orario il vettore p_2 rispetto all'origine $(0, 0)$ e che, se questo prodotto vettoriale è negativo, allora p_1 segue in senso antiorario p_2.

33.1-2

Il professor van Pelt ritiene che è sufficiente controllare soltanto la dimensione x nella riga 1 della procedura ON-SEGMENT. Spiegate perché il professore sbaglia.

33.1-3

L'***angolo polare*** di un punto p_1 rispetto a un'origine p_0 è l'angolo del vettore $p_1 - p_0$ nel consueto sistema di coordinate polari. Per esempio, l'angolo polare di $(3, 5)$ rispetto a $(2, 4)$ è l'angolo del vettore $(1, 1)$, che è 45 gradi o $\pi/4$ radianti. L'angolo polare di $(3, 3)$ rispetto a $(2, 4)$ è l'angolo del vettore $(1, -1)$, che è 315 gradi o $7\pi/4$ radianti. Scrivete lo pseudocodice per ordinare una sequenza $\langle p_1, p_2, \ldots, p_n \rangle$ di n punti secondo i loro angoli polari rispetto a una data origine p_0. La vostra procedura dovrebbe impiegare un tempo $O(n \lg n)$ e utilizzare i prodotti vettoriali per confrontare gli angoli.

33.1-4

Spiegate come determinare nel tempo $O(n^2 \lg n)$ se un insieme di n punti contiene tre punti collineari.

33.1-5

Un ***poligono*** è una linea spezzata chiusa nel piano; ovvero è una linea formata da una sequenza di segmenti di retta, detti ***lati*** del poligono, che inizia e termina nello stesso punto. Un punto che unisce due lati consecutivi è detto ***vertice*** del poligono. Se il poligono è ***semplice***, come di solito si suppone, non ha intersezioni con se stesso. L'insieme dei punti nel piano racchiusi da un poligono semplice forma la ***parte interna*** del poligono, l'insieme dei punti sul poligono stesso forma il ***perimetro***, e l'insieme dei punti che circondano il poligono forma la ***parte esterna***. Un poligono semplice è ***convesso*** se, dati due punti qualsiasi nel suo perimetro o nella sua parte interna, tutti i punti nel segmento di retta che passa per questi due punti si trovano nel perimetro o nella parte interna del poligono. Richiediamo inoltre che i vertici di un poligono convesso non siano mai combinazione convessa di due punti distinti sul perimetro o all'interno del poligono.

Acquistato da Michele Michele su Webster il 2022-07-01 2 Professor Amundsen suggerisce il seguente metodo per determinare se una

sequenza $\langle p_0, p_1, \ldots, p_{n-1} \rangle$ di n punti forma i vertici consecutivi di un poligono convesso. L'output è "sì" se l'insieme $\{\angle p_i p_{i+1} p_{i+2} : i = 0, 1, \ldots, n-1\}$, dove la somma dei pedici è eseguita modulo n, non contiene svolte né a sinistra né a destra; altrimenti l'output è "no". Dimostrate che, sebbene questo metodo venga eseguito in tempo lineare, non sempre produce la soluzione esatta. Modificate il metodo del professore in modo che produca sempre la soluzione esatta in tempo lineare.

33.1-6
Dato un punto $p_0 = (x_0, y_0)$, la ***semiretta orizzontale destra*** da p_0 è l'insieme dei punti $\{p_i = (x_i, y_i) : x_i \geq x_0 \text{ and } y_i = y_0\}$, ovvero è l'insieme dei punti che si trovano a destra di p_0, incluso p_0. Spiegate come determinare nel tempo $O(1)$ se una data semiretta orizzontale destra da p_0 interseca un segmento di retta $\overline{p_1 p_2}$ riducendo il problema a quello di determinare se due segmenti di retta si intersecano.

33.1-7
Un modo per determinare se un punto p_0 si trova nella parte interna di un poligono P semplice, ma non necessariamente convesso, consiste nel considerare una semiretta qualsiasi da p_0 e nel verificare che essa interseca il perimetro di P un numero dispari di volte, ma che p_0 stesso non appartiene al perimetro di P. Spiegate come determinare nel tempo $\Theta(n)$ se un punto p_0 è nella parte interna di un poligono P di n vertici. (*Suggerimento:* utilizzate l'Esercizio 33.1-6. Controllate che il vostro algoritmo operi correttamente quando la semiretta interseca il perimetro del poligono in un vertice e quando si sovrappone a un lato del poligono.)

33.1-8
Spiegate come calcolare l'area di un poligono semplice, ma non necessariamente convesso, di n vertici nel tempo $\Theta(n)$ (vedere l'Esercizio 33.1-5 per le definizioni sui poligoni).

33.2 Verificare se qualche coppia di segmenti si interseca

Questo paragrafo presenta un algoritmo per determinare se un insieme di segmenti contiene due segmenti che si intersecano. L'algoritmo usa una tecnica detta "sweeping" (spazzolamento), che è comune a molti algoritmi di geometria computazionale. Come dimostrano gli esercizi alla fine di questo paragrafo, l'algoritmo (o alcune sue semplici varianti) può essere utilizzato anche per risolvere altri problemi di geometria computazionale.

L'algoritmo viene eseguito nel tempo $O(n \lg n)$, dove n è il numero di segmenti. Determina soltanto se esiste qualche intersezione; non stampa tutte le intersezioni. (Per l'Esercizio 33.2-1, occorre un tempo $\Omega(n^2)$ nel caso peggiore per trovare *tutte* le intersezioni in un insieme di n segmenti di retta.)

Nello ***sweeping***, un'immaginaria retta verticale, detta ***retta di sweeping***, passa attraverso l'insieme degli oggetti geometrici, di solito da sinistra a destra. La dimensione spaziale che attraversa la retta di sweeping, in questo caso la dimensione x, è trattata come una dimensione temporale. Lo sweeping fornisce un metodo per ordinare gli oggetti geometrici, di solito memorizzandoli in una struttura dati dinamica, e per sfruttare le relazioni tra di essi. L'algoritmo di intersezione dei segmenti descritto in questo paragrafo considera tutti gli estremi dei segmenti ordinatamente da sinistra a destra e controlla se c'è un'intersezione ogni volta che incontra un estremo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Per descrivere e provare la correttezza dell'algoritmo che determina se due degli n segmenti di retta si intersecano, faremo due ipotesi semplificative. In primo luogo, supporremo che nessun segmento di input sia verticale. In secondo luogo, supporremo che non esistano tre segmenti di input che si intersecano nello stesso punto. Gli Esercizi 33.2-8 e 33.2-9 chiedono di dimostrare che l'algoritmo è abbastanza robusto da richiedere soltanto una piccola modifica per funzionare an-

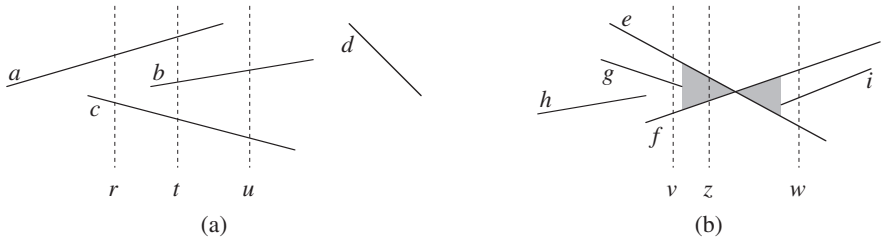


Figura 33.4
Ordinamento dei segmenti con varie rette di sweeping verticali. **(a)** Si ha $a \succcurlyeq_r c$, $a \succcurlyeq_t b$, $b \succcurlyeq_t c$, $a \succcurlyeq_t c$ e $b \succcurlyeq_u c$. Il segmento d non è confrontabile con gli altri segmenti illustrati nella figura. **(b)** Dopo che i segmenti e ed f si intersecano, il loro ordine s'inverte: si ha $e \succcurlyeq_v f$, ma $f \succcurlyeq_w e$. Qualsiasi retta di sweeping (per esempio, z) che attraversa la zona ombreggiata ha i segmenti e ed f consecutivi nel suo preordine totale.

che quando queste ipotesi non sono soddisfatte. In effetti, l'eliminazione di tali ipotesi semplificative e la gestione dei casi limite sono spesso la parte più difficile della programmazione degli algoritmi di geometria computazionale e della dimostrazione della loro correttezza.

Ordinare i segmenti

Poiché per ipotesi non ci sono segmenti verticali, qualsiasi segmento di input che interseca una data retta di sweeping verticale la interseca in un solo punto. Quindi, possiamo ordinare i segmenti che intersecano una retta di sweeping verticale in funzione delle coordinate y dei punti di intersezione.

Più precisamente, consideriamo due segmenti s_1 ed s_2. Diciamo che questi segmenti sono ***confrontabili*** in x se la retta di sweeping verticale con coordinata x interseca entrambi i segmenti. Diciamo che s_1 è ***sopra*** s_2 in x, che indichiamo con $s_1 \succcurlyeq_x s_2$, se s_1 ed s_2 sono confrontabili in x e l'intersezione di s_1 con la retta di sweeping in x è più alta dell'intersezione di s_2 con la stessa retta di sweeping. Nella Figura 33.4(a), per esempio, abbiamo le relazioni $a \succcurlyeq_r c$, $a \succcurlyeq_t b$, $b \succcurlyeq_t c$, $a \succcurlyeq_t c$ e $b \succcurlyeq_u c$. Il segmento d non è confrontabile con altri segmenti.

Per ogni x, la relazione "$\succcurlyeq_x$" è un preordine totale (vedere il Paragrafo B.2) nei segmenti che intersecano la retta di sweeping in x. Ovvero, la relazione è transitiva, e se i segmenti s_1 e s_2 intersecano la retta di sweeping in x, allora è vero che $s_1 \succcurlyeq_x s_2$ oppure $s_2 \succcurlyeq_x s_1$, o entrambe le cose (se s_1 e s_2 si intersecano sulla retta di sweeping). (La relazione $\succcurlyeq_x$ è anche riflessiva, ma non simmetrica né antisimmetrica.) Tuttavia, il preordine totale può essere diverso per valori differenti di x, in quanto i segmenti entrano ed escono dall'ordinamento. Un segmento entra nell'ordinamento quando la retta di sweeping incontra il suo estremo sinistro ed esce dall'ordinamento quando la retta di sweeping incontra il suo estremo destro.

Che cosa accade quando la retta di sweeping supera il punto di intersezione di due segmenti? Come illustra la Figura 33.4(b), le posizioni dei segmenti nel preordine totale si invertono. Le rette di sweeping v e w sono, rispettivamente, a sinistra e a destra del punto di intersezione dei segmenti e ed f; si ha $e \succcurlyeq_v f$ ed $f \succcurlyeq_w e$. Poiché per ipotesi non esistono tre segmenti che si intersecano nello stesso punto, deve esistere qualche retta di sweeping verticale x per la quale i segmenti e ed f, che si intersecano, sono *consecutivi* nel preordine totale $\succcurlyeq_x$. Qualsiasi retta di sweeping che attraversa la zona ombreggiata della Figura 33.4(b), per esempio z, ha i segmenti e ed f consecutivi nel suo preordine totale.

Acquistato da Michele Michele su Webster il 2022-07-05 21:55:40 Ordine Libreria 1000001641 2022 (Solo Consultazione) © 2010 The McGraw-Hill Education (Italy)

Spostare la retta di sweeping

Gli algoritmi di sweeping tipicamente gestiscono due insiemi di dati:

1. Lo ***stato della retta di sweeping*** fornisce le relazioni tra gli oggetti intersecati dalla retta di sweeping.

2. La ***lista dei punti evento*** è una sequenza di coordinate x, ordinate da sinistra a destra, che definisce le posizioni di arresto della retta di sweeping. Chiameremo ***punto evento*** ciascuna di tali posizioni di arresto. Lo stato della retta di sweeping può cambiare soltanto in un punto evento.

Per alcuni algoritmi (come quello richiesto nell'Esercizio 33.2-7), la lista dei punti evento viene determinata dinamicamente durante l'esecuzione dell'algoritmo. L'algoritmo che descriveremo, invece, determina i punti evento prima di effettuare lo sweeping, basandosi soltanto su semplici proprietà dei dati di input. In particolare, ogni estremo di un segmento è un punto evento. Ordiniamo gli estremi dei segmenti per coordinata x crescente e procediamo da sinistra a destra. (Se due o più estremi sono ***coverticali***, cioè hanno la stessa coordinata x, risolviamo l'uguaglianza inserendo tutti gli estremi sinistri coverticali prima degli estremi destri coverticali. All'interno di un insieme di estremi sinistri coverticali, inseriamo prima quelli con coordinate y più piccole; facciamo lo stesso all'interno di un insieme di estremi destri coverticali.) Inseriamo un segmento nello stato della retta di sweeping quando viene incontrato il suo estremo sinistro; cancelliamo un segmento dallo stato della retta di sweeping quando viene incontrato il suo estremo destro. Quando due segmenti diventano consecutivi nel preordine totale, verifichiamo se si intersecano. Lo stato della retta di sweeping è un preordine totale T, per il quale richiediamo le seguenti operazioni:

- INSERT(T, s): inserisce il segmento s in T.
- DELETE(T, s): cancella il segmento s da T.
- ABOVE(T, s): restituisce il segmento che si trova immediatamente sopra il segmento s in T.
- BELOW(T, s): restituisce il segmento che si trova immediatamente sotto il segmento s in T.

È possibile che i segmenti s_1 ed s_2 siano mutualmente uno sopra l'altro nel preordine totale T; questo caso può presentarsi se s_1 ed s_2 si intersecano sulla retta di sweeping il cui preordine totale è dato da T. In questo caso, i due segmenti potrebbero apparire in qualsiasi ordine in T. Se ci sono n segmenti nell'input, possiamo eseguire ciascuna di queste operazioni nel tempo $O(\lg n)$ utilizzando gli alberi rosso-neri. Ricordiamo che le operazioni con gli alberi rosso-neri (Capitolo 13) richiedono il confronto tra chiavi. Possiamo sostituire i confronti tra chiavi con i confronti che usano i prodotti vettoriali per determinare l'ordinamento relativo di due segmenti (vedere l'Esercizio 33.2-2).

Pseudocodice per l'intersezione dei segmenti

Il seguente algoritmo ANY-SEGMENTS-INTERSECT riceve come input un insieme S di n segmenti di retta e restituisce il valore booleano TRUE se due segmenti in S si intersecano, altrimenti restituisce FALSE. Il preordine totale T è implementato con un albero rosso-nero.

Acquistato da Michele Mi[illegible] Vettorato il 2022-07-07 12:12:12 - Rosso Cartiere Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

La Figura 33.5 illustra l'esecuzione dell'algoritmo. La riga 1 inizializza il preordine totale con l'insieme vuoto. La riga 2 determina la lista dei punti evento ordinando i $2n$ estremi dei segmenti da sinistra a destra, risolvendo le uguaglianze secondo le modalità precedentemente descritte. Notate che la riga 2 può essere eseguita ordinando lessicograficamente gli estremi rispetto alla terna (x, e, y), dove x e y sono le consuete coordinate, $e = 0$ per un estremo sinistro ed $e = 1$ per un estremo destro.

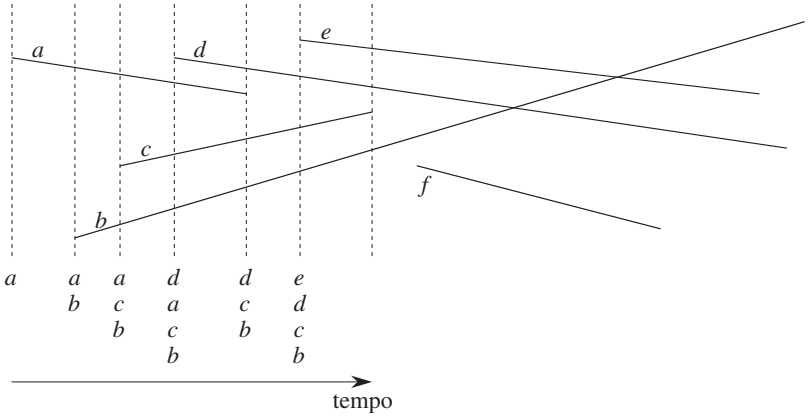


Figura 33.5
L'esecuzione di ANY-SEGMENTS-INTERSECT. Ogni retta tratteggiata rappresenta la retta di sweeping in un punto evento. Ad eccezione dell'ultima retta di sweeping a destra, la sequenza ordinata dei nomi dei segmenti sotto ogni retta di sweeping rappresenta il preordine totale T alla fine del ciclo **for** in cui viene elaborato il corrispondente punto evento. La retta di sweeping più a destra si presenta quando viene elaborato l'estremo destro del segmento c; poiché i segmenti d e b circondano c e si intersecano fra di loro, la procedura restituisce TRUE.

```
ANY-SEGMENTS-INTERSECT(S)
 1  T = ∅
 2  Ordina gli estremi dei segmenti in S da sinistra a destra, così:
        risolve l'uguaglianza delle coordinate x inserendo
                gli estremi sinistri prima degli estremi destri;
        risolve ulteriori uguaglianze inserendo prima i punti
                con coordinate y più piccole.
 3  for ogni punto p nella lista ordinata degli estremi
 4      if p è l'estremo sinistro di un segmento s
 5          INSERT(T, s)
 6          if (ABOVE(T, s) esiste e interseca s)
                o (BELOW(T, s) esiste e interseca s)
 7              return TRUE
 8      if p è l'estremo destro di un segmento s
 9          if esistono i due segmenti ABOVE(T, s) e BELOW(T, s)
                e ABOVE(T, s) interseca BELOW(T, s)
10              return TRUE
11          DELETE(T, s)
12  return FALSE
```

Ogni iterazione del ciclo **for** (righe 3–11) elabora un punto evento p. Se p è l'estremo sinistro di un segmento s, la riga 5 aggiunge s al preordine totale e le righe 6–7 restituiscono TRUE se s interseca uno dei suoi segmenti consecutivi nel preordine totale definito dalla retta di sweeping che passa per p. (Un caso limite si verifica se p giace su un altro segmento s'. In questo caso, richiediamo soltanto che s ed s' siano inseriti consecutivamente in T.) Se p è l'estremo destro di un segmento s, allora s deve essere cancellato dal preordine totale. Le righe 9–10 restituiscono TRUE se c'è un'intersezione tra i due segmenti adiacenti a s nel preordine totale definito dalla retta di sweeping che passa per p; questi segmenti diventeranno consecutivi nel preordine totale quando s sarà cancellato. Se questi segmenti non si intersecano, la riga 11 cancella il segmento s dal preordine totale. Se i segmenti che circondano il segmento s si intersecano, essi sarebbero diventati consecutivi dopo la rimozione di s se l'istruzione **return** nella riga 10 non avesse impedito di eseguire la riga 11. Il prossimo paragrafo sulla correttezza dell'algoritmo chiarirà perché è sufficiente controllare i segmenti che circondano s. Infine, se non vengono trovate intersezioni durante l'elaborazione di tutti i $2n$ punti eventi, la riga 12 restituisce FALSE.

Acquistato da Michele Michele su Webster il 2022-07-27 13:41. Vietata ogni copia, diffusione e distribuzione. Copyright © 2022 McGraw-Hill Education (Italy)

Correttezza

Per provare che l'algoritmo ANY-SEGMENTS-INTERSECT è corretto, dimostreremo che la chiamata ANY-SEGMENTS-INTERSECT(S) restituisce TRUE se e soltanto se esiste un'intersezione tra i segmenti in S.

È facile capire che ANY-SEGMENTS-INTERSECT restituisce TRUE (nelle righe 7 e 10) soltanto se trova un'intersezione tra due segmenti di input. Quindi, se restituisce TRUE, c'è un'intersezione.

Dobbiamo dimostrare anche il contrario: se c'è un'intersezione, allora ANY-SEGMENTS-INTERSECT restituisce TRUE. Supponiamo che ci sia almeno un'intersezione. Sia p il punto di intersezione più a sinistra (per risolvere le uguaglianze, scegliamo il punto che ha la coordinata y più piccola) e siano a e b i segmenti che si intersecano in p. Poiché non c'è alcuna intersezione a sinistra di p, l'ordine dato da T è corretto in tutti i punti a sinistra di p. Poiché non è possibile che tre segmenti si intersechino nello stesso punto, esiste una retta di sweeping z per la quale a e b diventano consecutivi nel preordine totale.[2] Inoltre, z è a sinistra di p o passa per p. Sulla retta di sweeping z esiste un estremo q che è il punto evento in cui a e b diventano consecutivi nel preordine totale. Se p è sulla retta di sweeping z, allora $q = p$. Se p non è sulla retta di sweeping z, allora q è a sinistra di p. In ogni caso, l'ordine dato da T è corretto appena prima che venga incontrato il punto q. (Ecco dove utilizziamo l'ordine lessicografico in cui l'algoritmo elabora i punti eventi. Poiché p è il più basso dei punti di intersezione a sinistra, anche se p è sulla retta di sweeping z e c'è un altro punto di intersezione p' su z, il punto evento $q = p$ viene elaborato prima che l'altro punto di intersezione p' possa interferire con il preordine totale T. Inoltre, anche se p è l'estremo sinistro di un segmento, per esempio a, e l'estremo destro dell'altro segmento b, poiché gli eventi degli estremi sinistri si verificano prima degli eventi degli estremi destri, il segmento b è in T quando il segmento a viene incontrato per la prima volta.) Il punto evento q o viene elaborato da ANY-SEGMENTS-INTERSECT o non viene elaborato.

Se q viene elaborato da ANY-SEGMENTS-INTERSECT, ci sono soltanto due possibilità per l'azione svolta:

1. Uno dei due segmenti a o b viene inserito in T e l'altro segmento si trova immediatamente sopra o sotto di esso nel preordine totale. Le righe 4–7 gestiscono questo caso.
2. I segmenti a e b sono già in T e viene cancellato un segmento tra di essi nel preordine totale, cosicché a e b diventano consecutivi. Le righe 8–11 gestiscono questo caso.

In entrambi i casi, viene trovata l'intersezione p e ANY-SEGMENTS-INTERSECT restituisce TRUE.

Se il punto evento q non viene elaborato da ANY-SEGMENTS-INTERSECT, la procedura deve essere terminata prima che fossero elaborati tutti i punti eventi. Questa situazione si può verificare soltanto se ANY-SEGMENTS-INTERSECT ha già trovato un'intersezione e ha restituito TRUE.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[2] Se consentiamo che tre segmenti si intersechino nello stesso punto, potrebbe esserci un segmento interposto c che interseca sia a che b nel punto p. Ovvero, potremmo avere $a \succcurlyeq_w c$ e $c \succcurlyeq_w b$ per qualsiasi retta di sweeping w a sinistra di p per la quale $a \succcurlyeq_w b$. L'Esercizio 33.2-8 chiede di dimostrare che l'algoritmo ANY-SEGMENTS-INTERSECT è corretto anche se tre segmenti si intersecano nello stesso punto.

Quindi, se c'è un'intersezione, ANY-SEGMENTS-INTERSECT restituisce TRUE. Come visto in precedenza, se ANY-SEGMENTS-INTERSECT restituisce TRUE, c'è un'intersezione. Di conseguenza, ANY-SEGMENTS-INTERSECT restituisce sempre una risposta corretta.

Tempo di esecuzione

Se ci sono n segmenti nell'insieme S, allora ANY-SEGMENTS-INTERSECT viene eseguita nel tempo $O(n \lg n)$. La riga 1 richiede un tempo $O(1)$. La riga 2 richiede un tempo $O(n \lg n)$, se si usa merge sort o heapsort. Poiché ci sono $2n$ punti eventi, il ciclo **for** (righe 3–11) viene ripetuto al più $2n$ volte. Ogni iterazione richiede un tempo $O(\lg n)$, perché ogni operazione con l'albero rosso-nero richiede un tempo $O(\lg n)$ e, utilizzando il metodo del Paragrafo 33.1, ogni test di intersezione richiede un tempo $O(1)$. Il tempo totale è dunque $O(n \lg n)$.

Esercizi

33.2-1
Dimostrate che ci possono essere $\Theta(n^2)$ intersezioni in un insieme di n segmenti di retta.

33.2-2
Dati due segmenti a e b che sono confrontabili in x, spiegate come determinare nel tempo $O(1)$ quale di queste relazioni è vera: $a \succcurlyeq_x b$ o $b \succcurlyeq_x a$. Supponete che nessuno dei due segmenti sia verticale. (*Suggerimento:* se a e b non si intersecano, potete utilizzare i prodotti vettoriali. Se a e b si intersecano – cosa che potete determinare usando soltanto prodotti vettoriali – potete comunque utilizzare soltanto l'addizione, la sottrazione e la moltiplicazione, evitando la divisione. Ovviamente, nell'applicazione in cui abbiamo utilizzato la relazione $\succcurlyeq_x$, se a e b si intersecano, possiamo fermarci e dichiarare che abbiamo trovato un'intersezione.)

33.2-3
Il professor Mason propone di modificare ANY-SEGMENTS-INTERSECT in modo che, anziché terminare dopo avere trovato un'intersezione, stampi i segmenti che si intersecano e continui con la successiva iterazione del ciclo **for**. Il professore ha chiamato la procedura risultante PRINT-INTERSECTING-SEGMENTS e afferma che essa stampa tutte le intersezioni, da sinistra a destra, via via che si verificano nell'insieme dei segmenti. Dimostrate che il professore ha torto su due punti, trovando come controesempio un insieme di segmenti per cui la prima intersezione trovata da PRINT-INTERSECTING-SEGMENTS non è quella più a sinistra e un insieme di segmenti per cui PRINT-INTERSECTING-SEGMENTS non riesce a trovare tutte le intersezioni.

33.2-4
Create un algoritmo con tempo $O(n \lg n)$ per determinare se un poligono di n vertici è semplice.

Acquistato da Michele Michele su Webster il 2022-03-23 12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

33.2-5
Create un algoritmo con tempo $O(n \lg n)$ per determinare se due poligoni semplici con un totale di n vertici si intersecano.

33.2-6
Un ***disco*** è formato da un cerchio più la sua parte interna ed è rappresentato da un centro e un raggio. Due dischi si intersecano se hanno qualche punto in comune. Create un algoritmo con tempo $O(n \lg n)$ per determinare se due dischi qualsiasi di un insieme di n dischi si intersecano.

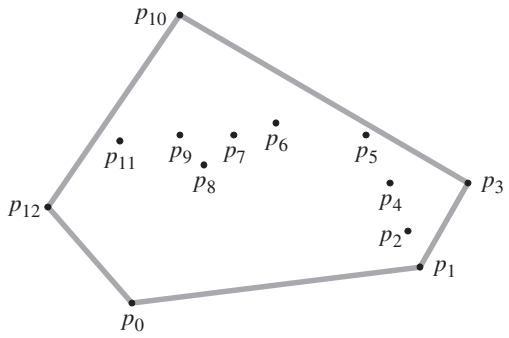


Figura 33.6
Un insieme di punti $Q = \{p_0, p_1, \ldots, p_{12}\}$ con il suo involucro convesso $\mathrm{CH}(Q)$ in grigio.

33.2-7
Dato un insieme di n segmenti di retta contenente un totale di k intersezioni, spiegate come produrre in output tutte le k intersezioni nel tempo $O((n+k)\lg n)$.

33.2-8
Dimostrate che ANY-SEGMENTS-INTERSECT funziona correttamente anche se tre o più segmenti si intersecano nello stesso punto.

33.2-9
Dimostrate che ANY-SEGMENTS-INTERSECT funziona correttamente in presenza di segmenti verticali se l'estremo inferiore di un segmento verticale viene elaborato come se fosse un estremo sinistro e l'estremo superiore viene elaborato come se fosse un estremo destro. Come cambia la vostra risposta all'Esercizio 33.2-2 se sono ammessi i segmenti verticali?

33.3 Trovare l'involucro convesso

L'***involucro convesso*** di un insieme di punti Q, che indicheremo con $\mathrm{CH}(Q)$, è il più piccolo poligono convesso P tale che ogni punto di Q si trova sul perimetro di P o nella sua parte interna (vedere l'Esercizio 33.1-5 per una definizione più precisa di poligono convesso). Assumiamo implicitamente che i punti in Q sono distinti e che Q contiene almeno tre punti non collineari. Intuitivamente, possiamo immaginare ciascun punto di Q come un chiodo che sporge da una tavola. L'involucro convesso è la forma che assume un elastico che circonda tutti i chiodi. La Figura 33.6 illustra un insieme di punti e il suo involucro convesso.

In questo paragrafo presenteremo due algoritmi che calcolano l'involucro convesso di un insieme di n punti. Entrambi gli algoritmi forniscono come output i vertici dell'involucro convesso in ordine antiorario. Il primo è l'algoritmo di Graham, che viene eseguito nel tempo $O(n \lg n)$. Il secondo è l'algoritmo di Jarvis, che viene eseguito nel tempo $O(nh)$, dove h è il numero di vertici dell'involucro convesso. Come potete notare dalla Figura 33.6, ogni vertice di $\mathrm{CH}(Q)$ è un punto dell'insieme Q. Entrambi gli algoritmi utilizzano questa proprietà, decidendo quali vertici di Q conservare come vertici dell'involucro convesso e quali vertici di Q scartare.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria 1995053016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

Esistono, in effetti, vari metodi che calcolano gli involucri convessi nel tempo $O(n \lg n)$. Sia l'algoritmo di Graham sia l'algoritmo di Jarvis adottano una tecnica, detta "sweeping rotazionale", che elabora i vertici nell'ordine degli angoli polari che essi formano con un vertice di riferimento. Fra gli altri metodi figurano i seguenti.

- Il ***metodo incrementale*** ordina i punti da sinistra a destra, producendo una sequenza $\langle p_1, p_2, \ldots, p_n \rangle$. All'$i$-esimo stadio, l'involucro convesso $\text{CH}(\{p_1, p_2, \ldots, p_{i-1}\})$ degli $i - 1$ punti più a sinistra viene aggiornato in base all'i-esimo punto da sinistra, formando così $\text{CH}(\{p_1, p_2, \ldots, p_i\})$. Come chiede di dimostrare l'Esercizio 33.3-6, questo metodo può essere implementato in modo da richiedere un tempo totale pari a $O(n \lg n)$.
- Il ***metodo divide et impera*** – che divide nel tempo $\Theta(n)$ l'insieme di n punti in due sottoinsiemi, uno contenente gli $\lceil n/2 \rceil$ punti più a sinistra e l'altro contenente gli $\lfloor n/2 \rfloor$ punti più a destra – calcola gli involucri convessi dei sottoinsiemi in modo ricorsivo; poi usa un metodo intelligente per combinare gli involucri nel tempo $O(n)$. Il tempo di esecuzione è descritto dalla nota ricorrenza $T(n) = 2T(n/2) + O(n)$, e quindi il metodo divide et impera viene eseguito nel tempo $O(n \lg n)$.
- Il ***metodo prune-and-search*** è simile all'algoritmo, descritto nel Paragrafo 9.3, che calcola la mediana in tempo lineare nel caso peggiore. Trova la porzione superiore (o "catena superiore") dell'involucro convesso eliminando ripetutamente una frazione costante dei punti restanti, finché resta soltanto la catena superiore dell'involucro convesso. Poi esegue la stessa operazione per la catena inferiore. Questo metodo è asintoticamente il più veloce: se l'involucro convesso contiene h vertici, viene eseguito in un tempo di appena $O(n \lg h)$.

Il calcolo dell'involucro convesso di un insieme di punti è un problema interessante per conto proprio. Inoltre, gli algoritmi per risolvere altri problemi di geometria computazionale iniziano calcolando un involucro convesso. Consideriamo, per esempio, il problema bidimensionale della ***coppia di punti più lontani***: dato un insieme di n punti nel piano, vogliamo trovare i due punti che si trovano alla distanza massima. Come chiede di dimostrare l'Esercizio 33.3-3, questi due punti devono essere vertici dell'involucro convesso. Anche se non lo dimostreremo qui, la coppia dei vertici più lontani di un poligono convesso di n vertici può essere trovata nel tempo $O(n)$. Quindi, calcolando l'involucro convesso degli n punti di input nel tempo $O(n \lg n)$ e poi trovando la coppia dei vertici più lontani del poligono convesso risultante, possiamo trovare la coppia dei punti più lontani in un insieme qualsiasi di n punti nel tempo $O(n \lg n)$.

Algoritmo di Graham

L'***algoritmo di Graham*** risolve il problema dell'involucro convesso mantenendo uno stack S dei punti candidati. Ogni punto dell'insieme di input Q viene inserito una sola volta nello stack; i punti che non sono vertici di $\text{CH}(Q)$ alla fine vengono rimossi dallo stack. Quando l'algoritmo termina, lo stack S contiene esattamente i vertici di $\text{CH}(Q)$, in ordine antiorario rispetto alla loro posizione sul perimetro.

La procedura GRAHAM-SCAN riceve come input un insieme di punti Q, dove $|Q| \geq 3$. Utilizza la funzione TOP(S), che restituisce il punto in cima allo stack S senza modificare S, e la funzione NEXT-TO-TOP(S), che restituisce il punto che occupa una posizione sotto la cima dello stack S senza modificare S. Come dimostreremo tra breve, lo stack S restituito da GRAHAM-SCAN contiene, dal basso verso l'alto, esattamente i vertici di $\text{CH}(Q)$ in ordine antiorario.

Acquistato da Michele Michele su Webster il 2022-0[illegible] Copyright © 2021 Mondadori Education S.p.A. Milano - Vietata la copia in rete

La Figura 33.7 illustra l'esecuzione di GRAHAM-SCAN. La riga 1 sceglie p_0 come il punto con la coordinata y minima, selezionando il punto più a sinistra se più punti hanno la stessa y minima. Poiché non c'è un punto di Q che sta sotto

p_0 e tutti gli altri punti con la stessa coordinata y sono alla sua destra, p_0 è un vertice di CH(Q). La riga 2 ordina i restanti punti di Q in funzione degli angoli polari rispetto a p_0, utilizzando lo stesso metodo descritto nell'Esercizio 33.1-3: il confronto dei prodotti vettoriali. Se due o più punti hanno lo stesso angolo polare rispetto a p_0, tutti tranne il più lontano di tali punti sono combinazioni convesse di p_0 e del punto più lontano, e quindi li possiamo escludere direttamente. Indichiamo con m il numero dei punti restanti, diversi da p_0. L'angolo polare, misurato in radianti, di ciascun punto di Q rispetto a p_0 è compreso nell'intervallo semiaperto $[0, \pi)$. Poiché i punti sono ordinati in funzione degli angoli polari, essi risultano ordinati in senso antiorario rispetto a p_0. Indichiamo questa sequenza ordinata di punti con $\langle p_1, p_2, \ldots, p_m \rangle$. Notate che i punti p_1 e p_m sono vertici di CH(Q) (vedere l'Esercizio 33.3-1). La Figura 33.7(a) illustra i punti della Figura 33.6 numerati per valori crescenti degli angoli polari rispetto a p_0.

GRAHAM-SCAN(Q)

```
1  Sia p_0 il punto di Q con la coordinata y minima o
       il punto più a sinistra se più punti hanno la stessa y minima
2  Siano ⟨p_1, p_2, ..., p_m⟩ i restanti punti di Q
       ordinati rispetto agli angoli polari in senso antiorario attorno a p_0
       (avendo rimosso tutti i punti che hanno lo stesso angolo,
       tranne quello che è più lontano da p_0)
3  Sia S uno stack vuoto
4  PUSH(p_0, S)
5  PUSH(p_1, S)
6  PUSH(p_2, S)
7  for i = 3 to m
8      while l'angolo formato dai punti NEXT-TO-TOP(S), TOP(S),
               e p_i effettua una svolta non a sinistra
9          POP(S)
10     PUSH(p_i, S)
11 return S
```

La parte restante della procedura usa lo stack S. Le righe 3–6 inizializzano lo stack inserendo, dal basso verso l'alto, i primi tre punti p_0, p_1 e p_2. La Figura 33.7(a) illustra lo stack iniziale S. Il ciclo **for** (righe 7–10) viene eseguito una volta per ogni punto della sottosequenza $\langle p_3, p_4, \ldots, p_m \rangle$. L'obiettivo è che, dopo l'elaborazione del punto p_i, lo stack S contenga, dal basso verso l'alto, i vertici di CH($\{p_0, p_1, \ldots, p_i\}$) in ordine antiorario. Il ciclo **while** (8–9) elimina dallo stack quei punti che non sono vertici dell'involucro convesso. Quando attraversiamo l'involucro convesso in senso antiorario, dovremmo effettuare una svolta a sinistra in ogni vertice. Quindi, ogni volta che il ciclo **while** trova un vertice in cui effettuiamo una svolta non a sinistra, il vertice viene rimosso dallo stack. (Controllando una svolta non a sinistra, anziché semplicemente una svolta a destra, questo test preclude la possibilità di un angolo piatto in un vertice dell'involucro convesso risultante. Non vogliamo angoli piatti, perché nessun vertice di un poligono convesso può essere una combinazione convessa di altri vertici del poligono.) Dopo avere rimosso tutti i vertici che hanno svolte non a sinistra quando ci dirigiamo verso il punto p_i, inseriamo p_i nello stack. Le Figure 33.7(b)–(k) illustrano lo stato dello stack S dopo ogni iterazione del ciclo **for**. Infine, GRAHAM-SCAN restituisce lo stack S nella riga 11. La Figura 33.7(l) illustra il corrispondente involucro convesso.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1955096162207077 Copyright © 2022, McGraw-Hill Education (Italy)

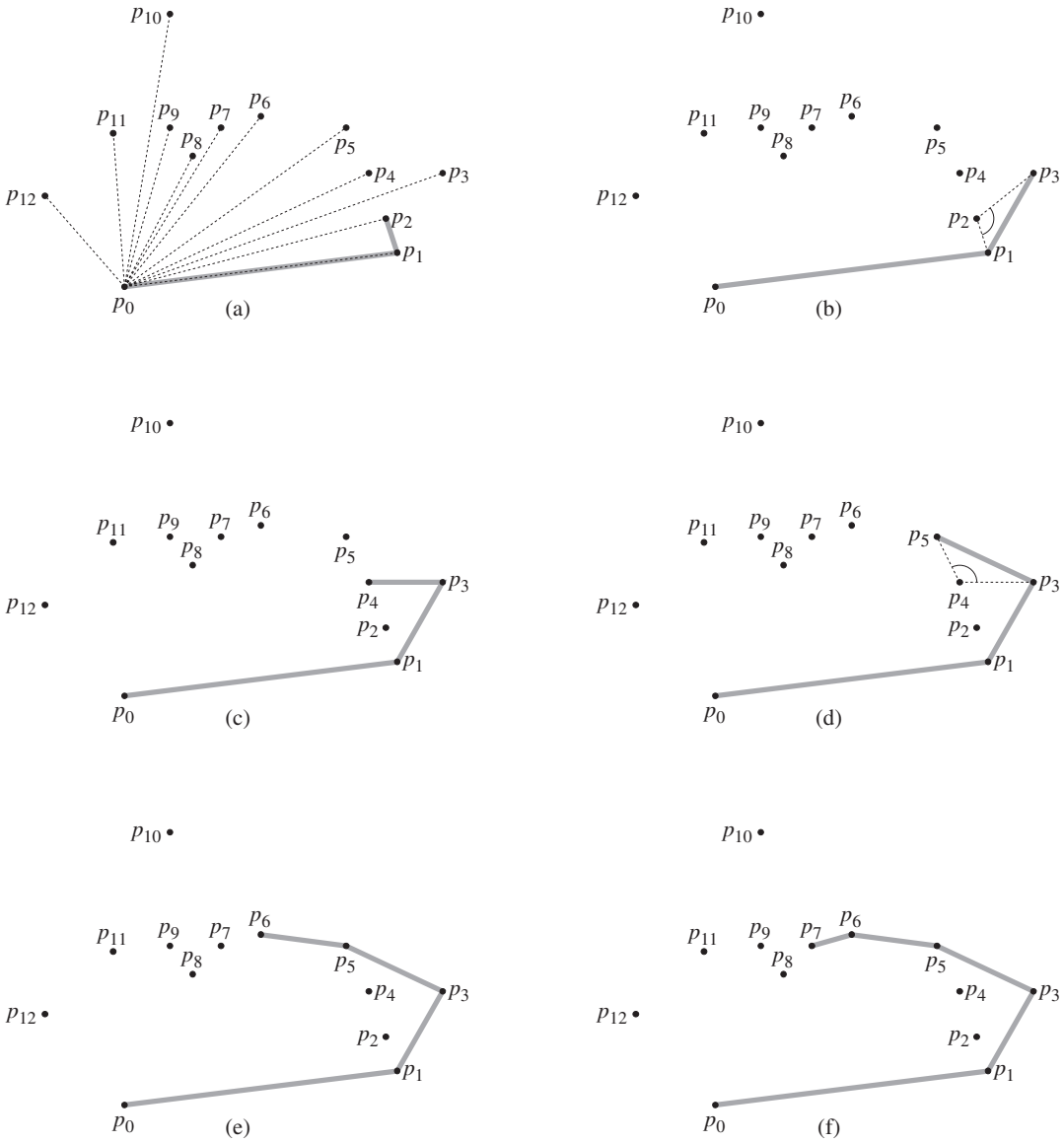
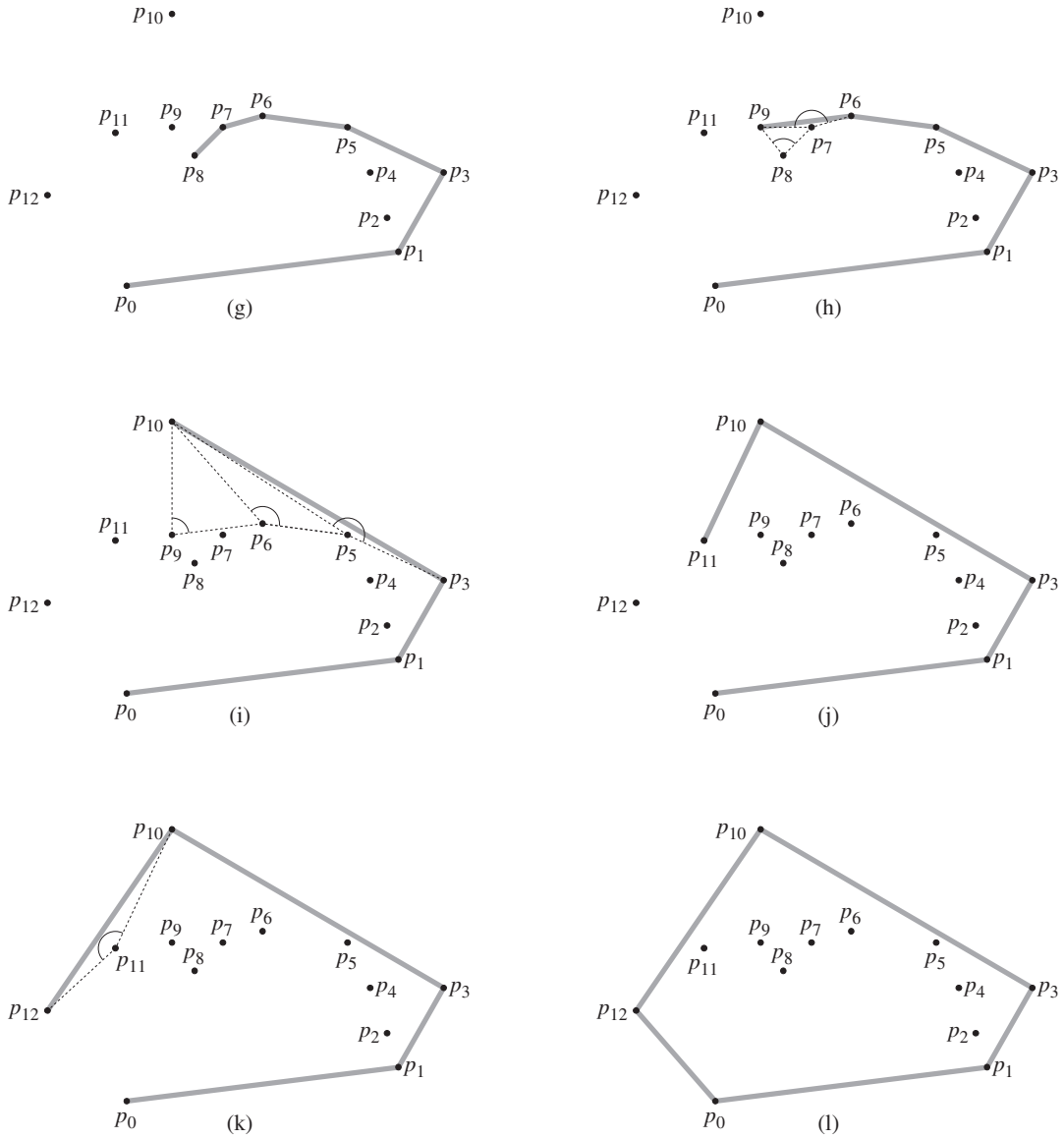


Figura 33.7 L'esecuzione di GRAHAM-SCAN sull'insieme Q della Figura 33.6. L'involucro convesso corrente contenuto nello stack S è illustrato in grigio a ogni passo. **(a)** La sequenza $\langle p_1, p_2, \ldots, p_{12} \rangle$ dei punti numerati per valori crescenti degli angoli polari rispetto a p_0; lo stack iniziale S contiene p_0, p_1 e p_2. **(b)–(k)** Lo stack S dopo ogni iterazione del ciclo **for** (righe 7–10). Le linee tratteggiate illustrano le svolte non a sinistra, che provocano l'estrazione dei punti dallo stack. Nella parte (h), per esempio, la svolta a destra nell'angolo $\angle p_7 p_8 p_9$ provoca la rimozione di p_8; poi la svolta a destra nell'angolo $\angle p_6 p_7 p_9$ provoca la rimozione di p_7. **(l)** L'involucro convesso restituito dalla procedura, che corrisponde a quello della Figura 33.6.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022. McGraw-Hill Education (Italy)

Il seguente teorema dimostra formalmente la correttezza di GRAHAM-SCAN.

Teorema 33.1 (Correttezza dell'algoritmo di Graham)
Se l'algoritmo GRAHAM-SCAN viene eseguito su un insieme di punti Q, dove $|Q| \geq 3$, alla fine lo stack S conterrà, dal basso verso l'alto, esattamente i vertici di $\mathrm{CH}(Q)$ in ordine antiorario.



Dimostrazione Dopo la riga 2, abbiamo la sequenza dei punti $\langle p_1, p_2, \ldots, p_m \rangle$. Definiamo il sottoinsieme di punti $Q_i = \{p_0, p_1, \ldots, p_i\}$, per $i = 2, 3, \ldots, m$. I punti in $Q - Q_m$ sono quelli che sono stati rimossi perché avevano lo stesso angolo polare rispetto a p_0 di qualche punto in Q_m; questi punti non si trovano in $\text{CH}(Q)$, quindi $\text{CH}(Q_m) = \text{CH}(Q)$. Allora basta dimostrare che, quando l'algoritmo GRAHAM-SCAN termina, lo stack S è formato dai vertici di $\text{CH}(Q_m)$ disposti in ordine antiorario, dal basso verso l'alto. Notate che proprio come p_0, p_1 e p_m sono vertici di $\text{CH}(Q)$, così i punti p_0, p_1 e p_i sono tutti vertici di $\text{CH}(Q_i)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995030018-2207072 Copyright © 2022 McGraw-Hill Education (Italy)

La dimostrazione usa la seguente invariante di ciclo:

> All'inizio di ogni iterazione del ciclo **for** (righe 7–10), lo stack S contiene, dal basso verso l'alto, esattamente i vertici di $\text{CH}(Q_{i-1})$ in ordine antiorario.

Inizializzazione: l'invariante è vera la prima volta che viene eseguita la riga 7, perché in quel momento lo stack S contiene esattamente i vertici di $Q_2 = Q_{i-1}$, e questo insieme di tre vertici forma il proprio involucro convesso; inoltre, questi vertici appaiono in ordine antiorario, dal basso verso l'alto.

Conservazione: quando inizia un'iterazione del ciclo **for**, il punto in cima allo stack S è p_{i-1}, che era stato inserito alla fine della precedente iterazione (o prima della prima iterazione, quando $i = 3$). Sia p_j il punto in cima allo stack S dopo che è stato eseguito il ciclo **while** (righe 8–9), ma prima che la riga 10 inserisca p_i, e sia p_k il punto appena sotto p_j in S. Quando p_j è il punto in cima allo stack S e non è stato ancora inserito p_i, lo stack S contiene esattamente gli stessi punti che conteneva dopo l'iterazione j del ciclo **for**. Quindi, per l'invariante di ciclo, in quel momento S contiene esattamente i vertici di $\text{CH}(Q_j)$, che appaiono in ordine antiorario, dal basso verso l'alto.

Restiamo ancora su questo momento prima che p_i sia inserito nello stack. Facendo riferimento alla Figura 33.8(a), poiché l'angolo di p_i rispetto a p_0 è maggiore dell'angolo polare di p_j e poiché l'angolo $\angle p_k p_j p_i$ effettua una svolta a sinistra (altrimenti avremmo dovuto rimuovere p_j), notiamo che, poiché S contiene esattamente i vertici di $\text{CH}(Q_j)$, una volta che inseriamo p_i, lo stack S conterrà esattamente i vertici di $\text{CH}(Q_j \cup \{p_i\})$, ancora in ordine antiorario, dal basso verso l'alto.

Dimostriamo ora che $\text{CH}(Q_j \cup \{p_i\})$ è lo stesso insieme di punti di $\text{CH}(Q_i)$. Consideriamo un punto qualsiasi p_t che è stato rimosso durante l'iterazione i del ciclo **for**; sia p_r il punto appena sotto p_t nello stack S nel momento in cui p_t viene rimosso (p_r potrebbe essere p_j). L'angolo $\angle p_r p_t p_i$ effettua una svolta non a sinistra e l'angolo polare di p_t rispetto a p_0 è maggiore dell'angolo polare di p_r. Come illustra la Figura 33.8(b), p_t deve trovarsi o nella parte interna del triangolo formato da p_0, p_r e p_i o su un lato di questo triangolo (ma non è un vertice del triangolo). Chiaramente, poiché p_t è all'interno di un triangolo formato da tre altri punti di Q_i, non può essere un vertice di $\text{CH}(Q_i)$. Poiché p_t non è un vertice di $\text{CH}(Q_i)$, si ha

$$\text{CH}(Q_i - \{p_t\}) = \text{CH}(Q_i) \tag{33.1}$$

Sia P_i l'insieme dei punti che sono stati rimossi durante l'iterazione i del ciclo **for**. Poiché l'uguaglianza (33.1) si applica a tutti i punti di P_i, possiamo applicarla ripetutamente per dimostrare che $\text{CH}(Q_i - P_i) = \text{CH}(Q_i)$. Ma $Q_i - P_i = Q_j \cup \{p_i\}$, e quindi concludiamo che $\text{CH}(Q_j \cup \{p_i\}) = \text{CH}(Q_i - P_i) = \text{CH}(Q_i)$.

Abbiamo dimostrato che, una volta inserito p_i, lo stack S contiene esattamente i vertici di $\text{CH}(Q_i)$ in ordine antiorario, dal basso verso l'alto. L'incremento di i farà sì che l'invariante di ciclo sarà valida per la successiva iterazione.

Conclusione: quando il ciclo termina, si ha $i = m + 1$, e quindi l'invariante di ciclo implica che lo stack S contiene, dal basso verso l'alto, esattamente i vertici, in ordine antiorario, di $\text{CH}(Q_m)$, che è $\text{CH}(Q)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503046-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Questo completa la dimostrazione. ■

Dimostriamo ora che il tempo di esecuzione di GRAHAM-SCAN è $O(n \lg n)$, dove $n = |Q|$. La riga 1 richiede un tempo $\Theta(n)$. La riga 2 richiede un tempo $O(n \lg n)$, se si usa merge sort o heapsort per ordinare gli angoli polari e il

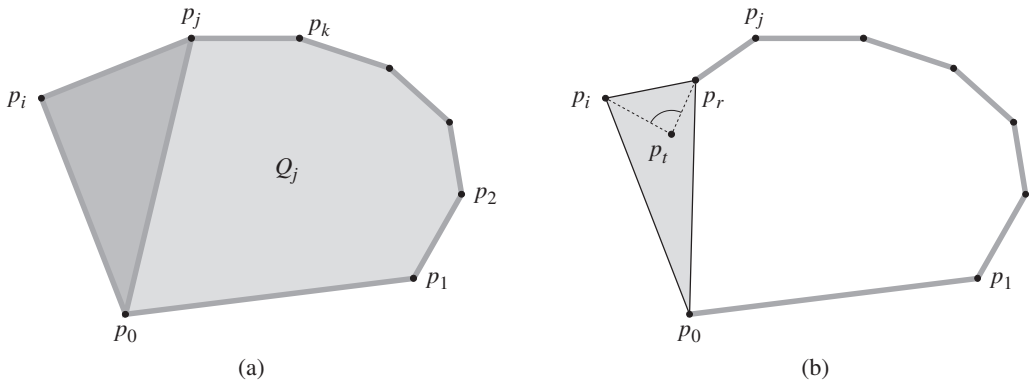


Figura 33.8 Dimostrazione della correttezza di GRAHAM-SCAN. **(a)** Poiché l'angolo polare di p_i rispetto a p_0 è maggiore dell'angolo polare di p_j e poiché l'angolo $\angle p_k p_j p_i$ effettua una svolta a sinistra, aggiungendo p_i a $\text{CH}(Q_j)$ si ottengono esattamente i vertici di $\text{CH}(Q_j \cup \{p_i\})$. **(b)** Se l'angolo $\angle p_r p_t p_i$ effettua una svolta non a sinistra, allora p_t si trova o nella parte interna del triangolo formato da p_0, p_r, e p_i o su un lato del triangolo, e non può essere un vertice di $\text{CH}(Q_i)$.

metodo dei prodotti vettoriali descritto nel Paragrafo 33.1 per confrontare gli angoli (la rimozione di tutti i punti che hanno lo stesso angolo polare, tranne il punto più lontano, può essere effettuata in un tempo totale pari a $O(n)$). Le righe 3–6 richiedono un tempo $O(1)$. Poiché $m \leq n - 1$, il ciclo **for** (righe 7–10) viene eseguito al più $n - 3$ volte. Poiché PUSH richiede un tempo $O(1)$, ogni iterazione richiede un tempo $O(1)$, escludendo il tempo impiegato nel ciclo **while** delle righe 8–9, e quindi il ciclo **for** richiede complessivamente un tempo $O(n)$, escluso il ciclo **while** annidato.

Utilizziamo il metodo dell'aggregazione per dimostrare che il ciclo **while** richiede complessivamente un tempo $O(n)$. Per $i = 0, 1, \ldots, m$, ogni punto p_i viene inserito nello stack S una sola volta. Come nell'analisi della procedura MULTIPOP fatta nel Paragrafo 17.1, osserviamo che c'è al più un'operazione POP per ogni operazione PUSH. Almeno tre punti – p_0, p_1 e p_m – non vengono mai rimossi dallo stack, quindi in effetti vengono eseguite complessivamente al più $m-2$ operazioni POP. Ogni iterazione del ciclo **while** esegue un'operazione POP, e quindi in totale ci sono al più $m-2$ iterazioni del ciclo **while**. Poiché il test nella riga 8 richiede un tempo $O(1)$, ogni chiamata di POP richiede un tempo $O(1)$, e poiché $m \leq n - 1$, il tempo totale impiegato dal ciclo **while** è $O(n)$. Dunque, il tempo di esecuzione di GRAHAM-SCAN è $O(n \lg n)$.

Algoritmo di Jarvis

L'***algoritmo di Jarvis*** calcola l'involucro convesso di un insieme di punti Q adottando una tecnica detta ***impacchettamento*** o ***confezionamento dei regali***. L'algoritmo viene eseguito nel tempo $O(nh)$, dove h è il numero di vertici di $\text{CH}(Q)$. Quando h è $o(\lg n)$, l'algoritmo di Jarvis è asintoticamente più veloce dell'algoritmo di Graham.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Intuitivamente, l'algoritmo di Jarvis simula l'operazione di impacchettare con un foglio di carta l'insieme dei punti Q. Iniziamo ancorando l'estremità del foglio di carta al punto più basso dell'insieme, che è lo stesso punto p_0 dal quale inizia l'algoritmo di Graham. Questo punto è un vertice dell'involucro convesso. Tiriamo la carta verso destra per tenderla; poi la tiriamo verso l'alto finché

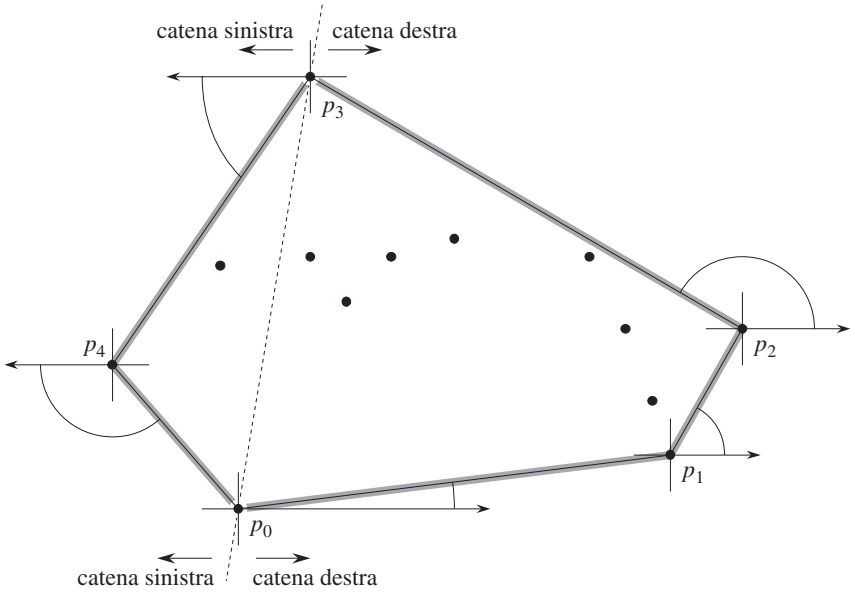


Figura 33.9
Il funzionamento dell'algoritmo di Jarvis. Il primo vertice scelto è il punto più basso p_0. Il vertice successivo, p_1, è quello che ha il più piccolo angolo polare rispetto a p_0 tra tutti gli altri vertici. Segue il vertice p_2 che ha il più piccolo angolo polare rispetto a p_1. La catena destra sale fino al punto più alto p_3. Poi, viene costruita la catena sinistra trovando gli angoli polari minimi riferiti all'asse x negativo.

non tocchiamo un punto dell'insieme. Questo punto deve essere anche un vertice dell'involucro convesso. Mantenendo tesa la carta, continuiamo a impacchettare l'insieme dei vertici finché non ritorniamo al punto originale p_0.

Più formalmente, l'algoritmo di Jarvis costruisce una sequenza $H = \langle p_0, p_1, \ldots, p_{h-1} \rangle$ dei vertici di $\mathrm{CH}(Q)$. Iniziamo da p_0. Come illustra la Figura 33.9, il successivo vertice dell'involucro convesso p_1 ha il più piccolo angolo polare rispetto a p_0 (se più punti hanno lo stesso angolo polare, scegliamo il punto più lontano da p_0). Analogamente, p_2 ha il più piccolo angolo polare rispetto a p_1, e così via. Quando raggiungiamo il vertice più alto, per esempio p_k (se ci sono più vertici alla massima altezza, scegliamo quello più lontano), abbiamo costruito la ***catena destra*** di $\mathrm{CH}(Q)$, come illustra la Figura 33.9. Per costruire la ***catena sinistra***, iniziamo da p_k e scegliamo p_{k+1} come il punto con il più piccolo angolo polare rispetto a p_k, facendo riferimento stavolta *all'asse x negativo*. Continuiamo a formare la catena sinistra riferendo gli angoli polari all'asse x negativo, finché non ritorniamo al vertice originale p_0.

Potremmo implementare l'algoritmo di Jarvis con uno sweeping concettuale attorno all'involucro convesso, ovvero, senza costruire separatamente la catena destra e la catena sinistra. Tipicamente, tali implementazioni tengono traccia dell'angolo dell'ultimo lato scelto dell'involucro convesso e richiedono che la sequenza degli angoli dei lati dell'involucro sia strettamente crescente (nell'intervallo da 0 a 2π radianti). Il vantaggio di costruire le catene separate è che non occorre calcolare esplicitamente gli angoli; le tecniche del Paragrafo 33.1 sono sufficienti per confrontare gli angoli.

Se implementato appropriatamente, l'algoritmo di Jarvis ha un tempo di esecuzione pari a $O(nh)$. Per ciascuno degli h vertici di $\mathrm{CH}(Q)$, troviamo il vertice con l'angolo polare minimo. Ogni confronto tra gli angoli polari richiede un tempo $O(1)$, utilizzando le tecniche del Paragrafo 33.1. Come abbiamo descritto nel Paragrafo 9.1, è possibile calcolare il minimo di n valori nel tempo $O(n)$, se ogni confronto richiede un tempo $O(1)$. Quindi, l'algoritmo di Jarvis richiede un tempo $O(nh)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria 1655036016 2220710 Copyright © 2022 CLEUP (McGraw-Hill Education Italy)

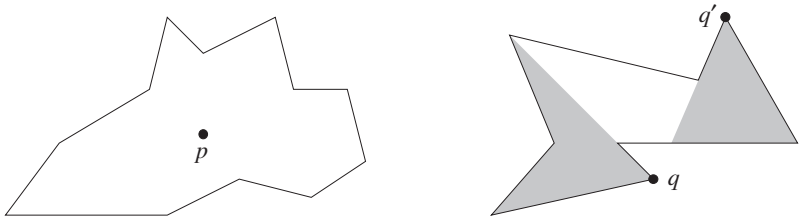


Figura 33.10
La definizione di un poligono a forma di stella, da utilizzare nell'Esercizio 33.3-4. **(a)** Un poligono a forma di stella. Il segmento che ha per estremi il punto p e un punto qualsiasi q del perimetro interseca il perimetro soltanto in q. **(b)** Un poligono non a forma di stella. La regione grigia a sinistra è l'ombra del punto q; la regione grigia a destra è l'ombra del punto q'. Poiché queste regioni sono disgiunte, il nucleo è vuoto.

Esercizi

33.3-1
Dimostrate che nella procedura GRAHAM-SCAN i punti p_1 e p_m devono essere vertici di CH(Q).

33.3-2
Considerate un modello di calcolo che supporta l'addizione, il confronto e la moltiplicazione e per il quale esiste un limite inferiore $\Omega(n \lg n)$ per ordinare n numeri. Dimostrate che, in un tale modello, $\Omega(n \lg n)$ è un limite inferiore per calcolare, ordinatamente, i vertici dell'involucro convesso di un insieme di n punti.

33.3-3
Dato un insieme di punti Q, dimostrate che i due punti che si trovano alla massima distanza devono essere vertici di CH(Q).

33.3-4
Se P è un poligono e q è un punto sul suo perimetro, l'***ombra*** del punto q è l'insieme dei punti r tali che il segmento $\overline{qr}$ si trova interamente sul perimetro o nella parte interna del poligono. Un poligono P è ***a forma di stella*** se esiste un punto p nella parte interna di P che è nell'ombra di qualsiasi punto del perimetro di P. L'insieme di tutti questi punti p è detto ***nucleo*** di P (vedere la Figura 33.10). Dato un poligono P di n vertici, a forma di stella, specificato dai suoi vertici in ordine antiorario, spiegate come calcolare CH(P) nel tempo $O(n)$.

33.3-5
Nel ***problema dell'involucro convesso on-line*** gli n punti dell'insieme Q vengono dati uno alla volta. Man mano che arrivano i punti, si calcola l'involucro convesso dei punti noti. Ovviamente, è possibile eseguire l'algoritmo di Graham una volta per ogni punto, con un tempo totale di esecuzione $O(n^2 \lg n)$. Spiegate come risolvere il problema dell'involucro convesso on-line in un tempo totale $O(n^2)$.

33.3-6 ★
Spiegate come implementare il metodo incrementale per calcolare l'involucro convesso di n punti in modo che possa essere eseguito nel tempo $O(n \lg n)$.

33.4 Trovare la coppia di punti più vicini

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Consideriamo adesso il problema di trovare la coppia di punti più vicini in un insieme Q di $n \geq 2$ punti. L'aggettivo "vicini" è da intendere nel senso della consueta distanza euclidea: la distanza tra i punti $p_1 = (x_1, y_1)$ e $p_2 = (x_2, y_2)$ è $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. Due punti in un insieme Q possono essere coincidenti, nel qual caso la loro distanza è zero. Questo problema ha applicazioni, per

esempio, nei sistemi di controllo del traffico. Un sistema per controllare il traffico aereo o marittimo potrebbe richiedere di sapere quali sono i due mezzi più vicini al fine di rilevare potenziali collisioni.

Un algoritmo a forza bruta per trovare le coppie di punti più vicini controlla semplicemente tutte le $\binom{n}{2} = \Theta(n^2)$ coppie di punti. In questo paragrafo, descriveremo un algoritmo divide et impera per questo problema, il cui tempo di esecuzione è descritto dalla nota ricorrenza $T(n) = 2T(n/2) + O(n)$. Quindi, questo algoritmo richiede soltanto un tempo $O(n \lg n)$.

L'algoritmo divide et impera

Ogni chiamata ricorsiva dell'algoritmo riceve come input un sottoinsieme $P \subseteq Q$ e gli array X e Y, ciascuno dei quali contiene tutti i punti del sottoinsieme di input P. I punti nell'array X sono ordinati in modo che le loro coordinate x siano monotonicamente crescenti. Analogamente, l'array Y è ordinato per valori monotonicamente crescenti delle coordinate y. Notate che per restare entro il limite di tempo $O(n \lg n)$, non possiamo permetterci di eseguire un ordinamento in ogni chiamata ricorsiva; se lo facessimo, la ricorrenza per il tempo di esecuzione sarebbe $T(n) = 2T(n/2) + O(n \lg n)$, la cui soluzione è $T(n) = O(n \lg^2 n)$. (Utilizzate la versione del metodo dell'esperto descritta nell'Esercizio 4.6-2.) Vedremo più avanti come utilizzare il "preordinamento" per mantenere questa proprietà senza eseguire effettivamente un ordinamento in ogni chiamata ricorsiva.

Una chiamata ricorsiva con input P, X e Y controlla innanzi tutto se $|P| \leq 3$. Se ciò è vero, la chiamata esegue semplicemente il metodo a forza bruta precedentemente descritto: controllare tutte le $\binom{|P|}{2}$ coppie di punti e restituire la coppia dei punti più vicini. Se $|P| > 3$, la chiamata ricorsiva applica il paradigma divide et impera nel modo seguente.

Divide: trova una retta verticale l che biseca l'insieme di punti P in due insiemi P_L e P_R tali che $|P_L| = \lceil |P|/2 \rceil$, $|P_R| = \lfloor |P|/2 \rfloor$, tutti i punti in P_L sono sulla retta l o alla sua sinistra e tutti i punti in P_R sono sulla retta l o alla sua destra. L'array X è suddiviso negli array X_L e X_R, che contengono, rispettivamente, i punti di P_L e P_R ordinati per valori monotonicamente crescenti delle coordinate x. Analogamente, l'array Y è suddiviso negli array Y_L e Y_R, che contengono, rispettivamente, i punti di P_L e P_R ordinati per valori monotonicamente crescenti delle coordinate y.

Impera: avendo diviso l'insieme P negli insiemi P_L e P_R, effettua due chiamate ricorsive, una per trovare la coppia dei punti più vicini in P_L e l'altra per trovare la coppia dei punti più vicini in P_R. Gli input per la prima chiamata sono il sottoinsieme P_L e gli array X_L e Y_L; la seconda chiamata riceve gli input P_R, X_R e Y_R. Siano δ_L e δ_R, rispettivamente, le distanze dei punti più vicini restituite per P_L e P_R e sia $\delta = \min(\delta_L, \delta_R)$.

Combina: la coppia dei punti più vicini o è la coppia con la distanza δ trovata da una delle chiamate ricorsive o è una coppia di punti con un punto in P_L e l'altro in P_R. L'algoritmo determina se esiste una coppia con un punto in P_L e l'altro in P_R la cui distanza è minore di δ. Osservate che se esiste una coppia di punti la cui distanza è minore di δ, entrambi i punti della coppia devono trovarsi entro δ unità dalla retta l. Quindi, come illustra la Figura 33.11(a), entrambi i punti devono trovarsi nella banda verticale larga 2δ centrata sulla retta l. Per trovare una tale coppia, se esiste, l'algoritmo svolge le seguenti azioni.

Acquistato da Michele Michele su Webster il 2022-07-07 Studio Vitale Materiale protetto da copyright © 2010, The MIT Press Licenziato (Italy)

1. Crea un array Y', che è l'array Y dal quale sono stati eliminati i punti che non si trovano nella banda verticale larga 2δ. L'array Y' è ordinato rispetto alle coordinate y, esattamente come Y.
2. Per ogni punto p dell'array Y', l'algoritmo tenta di trovare i punti in Y' che si trovano entro δ unità da p. Come vedremo più avanti, devono essere considerati soltanto i 7 punti di Y' che seguono p. L'algoritmo calcola la distanza tra p e ciascuno di questi 7 punti e tiene traccia della distanza δ' della coppia dei punti più vicini che è stata trovata considerando tutte le coppie di punti di Y'.
3. Se $\delta' < \delta$, allora la banda verticale contiene una coppia di punti che sono più vicini di quelli trovati dalle chiamate ricorsive. L'algoritmo restituisce questa coppia di punti e la distanza δ'; altrimenti restituisce la coppia dei punti e la distanza δ trovate dalle chiamate ricorsive.

La precedente descrizione omette alcuni dettagli implementativi che sono necessari per ottenere il tempo di esecuzione $O(n \lg n)$. Dopo avere provato la correttezza dell'algoritmo, dimostreremo come implementare l'algoritmo per conseguire il limite di tempo desiderato.

Correttezza

La correttezza di questo algoritmo per trovare la coppia di punti più vicini è ovvia, con l'eccezione di due aspetti. In primo luogo, escludendo la ricorsione quando $|P| \leq 3$, abbiamo la garanzia di non tentare di risolvere un sottoproblema che è formato da un solo punto. Il secondo aspetto è che dobbiamo controllare soltanto i 7 punti che seguono ciascun punto p nell'array Y'; dimostriamo adesso questa proprietà.

Supponiamo che, a qualche livello della ricorsione, la coppia dei punti più vicini sia $p_L \in P_L$ e $p_R \in P_R$. Quindi, la distanza δ' tra p_L e p_R è strettamente minore di δ. Il punto p_L deve trovarsi sulla retta l o alla sua sinistra, a meno di δ unità. Analogamente, p_R si trova sulla retta l o alla sua destra, a meno di δ unità. Inoltre, la distanza verticale tra p_L e p_R è minore di δ unità. Quindi, come illustra la Figura 33.11(a), p_L e p_R si trovano all'interno di un rettangolo $\delta \times 2\delta$ centrato sulla retta l (all'interno di questo rettangolo possono esserci altri punti).

Dimostriamo ora che al più 8 punti di P possono trovarsi all'interno di questo rettangolo $\delta \times 2\delta$. Consideriamo il quadrato $\delta \times \delta$ che forma la metà sinistra di questo rettangolo. Poiché tutti i punti di P_L sono distanti almeno δ unità, al più 4 punti possono trovarsi all'interno di questo quadrato, come illustra la Figura 33.11(b). Analogamente, al più 4 punti di P_R possono trovarsi all'interno del quadrato $\delta \times \delta$ che forma la metà destra del rettangolo. Quindi, al più 8 punti di P possono trovarsi all'interno del rettangolo $\delta \times 2\delta$. (Notate che, poiché i punti sulla retta l possono appartenere sia a P_L sia a P_R, ci possono essere fino a 4 punti su l. Questo limite è raggiunto se ci sono due coppie di punti coincidenti tali che ogni coppia sia formata da un punto di P_L e da un punto di P_R: una coppia è dove l interseca il lato superiore del rettangolo e l'altra coppia è dove l interseca il lato inferiore del rettangolo.)

Acquistato da Michele Nicolai via Webster il 2023-07-07 23:12. Mondo Online Libreria 1995061122207-0 Copyright © 2022 McGraw-Hill Education (Italy)

Avendo dimostrato che al più 8 punti di P possono trovarsi all'interno del rettangolo, si può dedurre che basta controllare soltanto i 7 punti che seguono ciascun punto nell'array Y'. Sempre supponendo che la coppia dei punti più vicini sia p_L e p_R, possiamo anche supporre, senza perdere in generalità, che p_L preceda p_R nell'array Y'. Allora, anche se p_L si presenta il più presto possibile in Y' e p_R

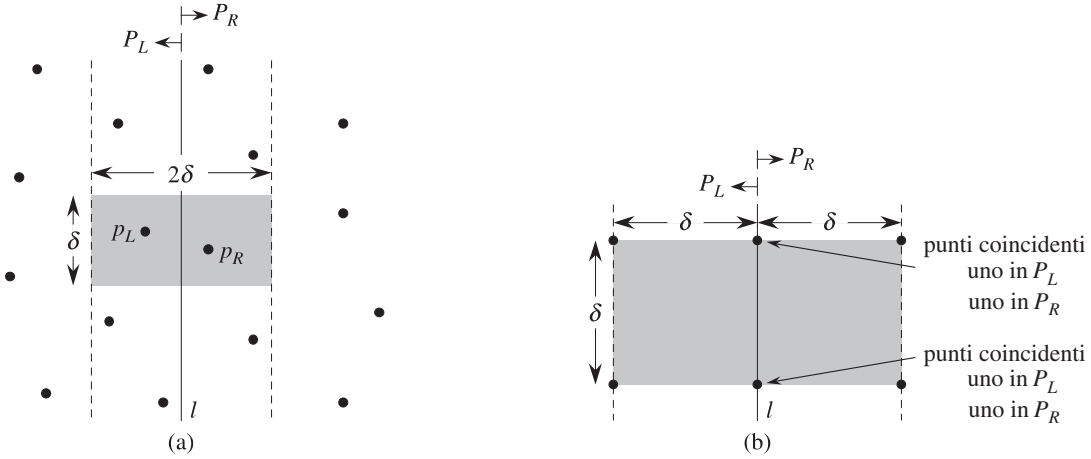


Figura 33.11 Concetti fondamentali della dimostrazione che l'algoritmo per trovare la coppia di punti più vicini ha bisogno di controllare soltanto i 7 punti che seguono ciascun punto nell'array Y'. **(a)** Se $p_L \in P_L$ e $p_R \in P_R$ si trovano a una distanza minore di δ unità, devono essere all'interno di un rettangolo $\delta \times 2\delta$ centrato sulla retta l. **(b)** Ecco come 4 punti, che sono distanti a due a due almeno δ unità, possono trovarsi all'interno di un quadrato $\delta \times \delta$. A sinistra ci sono 4 punti di P_L; a destra ci sono 4 punti di P_R. Possono esserci 8 punti nel rettangolo $\delta \times 2\delta$, se i punti indicati sulla retta l sono effettivamente coppie di punti coincidenti con un punto in P_L e uno in P_R.

si presenta il più tardi possibile, p_R è in una delle 7 posizioni che seguono p_L. Quindi, abbiamo dimostrato la correttezza dell'algoritmo per la coppia dei punti più vicini.

Implementazione e tempo di esecuzione

Come detto in precedenza, il nostro obiettivo è ottenere la seguente ricorrenza per il tempo di esecuzione: $T(n) = 2T(n/2) + O(n)$, dove $T(n)$ è il tempo di esecuzione per un insieme di n punti. La principale difficoltà sta nel garantire che gli array X_L, X_R, Y_L e Y_R, che vengono passati alle chiamate ricorsive, siano ordinati rispetto alla coordinata appropriata e inoltre che l'array Y' sia ordinato rispetto alla coordinata y. (Notate che se l'array X, che viene ricevuto da una chiamata ricorsiva, è già ordinato, allora la suddivisione dell'insieme P in P_L e P_R viene facilmente realizzata in tempo lineare.)

L'osservazione fondamentale è che in ogni chiamata vogliamo formare un sottoinsieme ordinato di un array ordinato. Per esempio, una particolare chiamata riceve il sottoinsieme P e l'array Y, ordinato rispetto alla coordinata y. Avendo suddiviso P in P_L e P_R, occorre formare gli array Y_L e Y_R, che sono ordinati rispetto alla coordinata y. Inoltre, questi array devono essere formati in tempo lineare. Il metodo può essere visto come l'opposto della procedura MERGE descritta nel Paragrafo 2.3.1: stiamo dividendo un array ordinato in due array ordinati. Il seguente pseudocodice può essere un buon punto di partenza.

```
1  Siano YL[1 .. Y.length] e YR[1 .. Y.length] due nuovi array
2  YL.length = YR.length = 0
3  for i = 1 to Y.length
4      if Y[i] ∈ PL
5          YL.length = YL.length + 1
6          YL[YL.length] = Y[i]
7      else YR.length = YR.length + 1
8          YR[YR.length] = Y[i]
```

Acquistato da Michele Michele su Webster il 2022-07-07 19:12 - Numero Ordine Libreria 1.995.2016-220707 - Copyright © 2022, McGraw-Hill Education (Italy)

Semplicemente esaminiamo in ordine i punti dell'array Y. Se un punto $Y[i]$ si trova in P_L, lo aggiungiamo alla fine dell'array Y_L, altrimenti lo aggiungiamo alla fine dell'array Y_R. Un analogo pseudocodice può essere utilizzato per formare gli array X_L, X_R e Y'.

L'ultimo problema rimasto è come avere inizialmente i punti ordinati. Questo problema può essere risolto semplicemente ***preordinando*** gli array; ovvero ordiniamo gli array una volta per tutte, *prima* della prima chiamata ricorsiva. Questi array ordinati vengono passati nella prima chiamata ricorsiva e da qui vengono ridotti per effettuare le chiamate ricorsive, quando necessario. Il preordinamento aggiunge un tempo $O(n \lg n)$ al tempo di esecuzione, ma adesso ogni passo della ricorsione richiede un tempo lineare, escludendo le chiamate ricorsive. Quindi, se indichiamo con $T(n)$ il tempo di esecuzione di ciascun passo ricorsivo e con $T'(n)$ il tempo di esecuzione dell'intero algoritmo, abbiamo $T'(n) = T(n) + O(n \lg n)$ e

$$T(n) = \begin{cases} 2T(n/2) + O(n) & \text{se } n > 3 \\ O(1) & \text{se } n \leq 3 \end{cases}$$

Quindi, $T(n) = O(n \lg n)$ e $T'(n) = O(n \lg n)$.

Esercizi

33.4-1

Il professor Williams ha ideato uno schema che consente all'algoritmo per la coppia dei punti più vicini di controllare soltanto i 5 punti che seguono ciascun punto nell'array Y'. L'idea consiste nell'inserire i punti sulla retta l sempre nell'insieme P_L. In tal modo, non possono esserci coppie di punti coincidenti sulla retta l con un punto in P_L e uno in P_R. Quindi, al più 6 punti possono trovarsi nel rettangolo $\delta \times 2\delta$. Qual è l'errore dello schema del professore?

33.4-2

Dimostrate che basta controllare soltanto i punti nelle 5 posizioni successive di ciascun punto nell'array Y'.

33.4-3

Oltre alla distanza euclidea, ci sono altri modi in cui può essere definita la distanza tra due punti. Nel piano, la ***L_m-distanza*** tra i punti p_1 e p_2 è data dall'espressione $(|x_1 - x_2|^m + |y_1 - y_2|^m)^{1/m}$. La distanza euclidea, quindi, è la L_2-distanza. Modificate l'algoritmo per la coppia dei punti più vicini per utilizzare la L_1-distanza, che è nota anche come ***distanza di Manhattan***.

33.4-4

Dati due punti p_1 e p_2 nel piano, la L_∞-distanza tra di essi è data da $\max(|x_1 - x_2|, |y_1 - y_2|)$. Modificate l'algoritmo per la coppia dei punti più vicini per utilizzare la L_∞-distanza.

Acquistato da Michele Mastrangelo Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

33.4-5

Supponete che $\Omega(n)$ dei punti dati nell'algoritmo della coppia dei punti più vicini siano coverticali. Spiegate come determinare gli insiemi P_L e P_R e come stabilire se ciascun punto di Y è in P_L o P_R, mantenendo il tempo di esecuzione dell'algoritmo pari a $O(n \lg n)$.

33.4-6
Suggerite una modifica dell'algoritmo per la coppia dei punti più vicini per evitare di preordinare l'array Y, mantenendo il tempo di esecuzione $O(n \lg n)$ (*suggerimento:* unite gli array ordinati Y_L e Y_R per formare l'array ordinato Y).

Problemi

33-1 Strati convessi
Dato un insieme di punti Q nel piano, definiamo gli ***strati convessi*** di Q in modo induttivo. Il primo strato convesso di Q è formato da quei punti di Q che sono vertici di $\mathrm{CH}(Q)$. Per $i > 1$, sia Q_i l'insieme formato dai punti dell'insieme Q dal quale sono stati rimossi tutti i punti degli strati convessi $1, 2, \ldots, i-1$. Allora, l'i-esimo strato convesso di Q è $\mathrm{CH}(Q_i)$ se $Q_i \neq \emptyset$, altrimenti è indefinito.

a. Create un algoritmo con tempo $O(n^2)$ per trovare gli strati convessi di un insieme di n punti.

b. Dimostrate che occorre un tempo $\Omega(n \lg n)$ per calcolare gli strati convessi di un insieme di n punti con un modello di calcolo qualsiasi che richiede un tempo $\Omega(n \lg n)$ per ordinare n numeri reali.

33-2 Strati massimali
Sia Q un insieme di n punti nel piano. Diciamo che il punto (x, y) ***domina*** il punto (x', y') se $x \geq x'$ e $y \geq y'$. Un punto di Q che non è dominato da altri punti di Q è detto ***massimale***. Notate che Q può contenere molti punti massimali, che possono essere organizzati in ***strati massimali*** nel modo seguente. Il primo strato massimale L_1 è l'insieme dei punti massimali di Q. Per $i > 1$, l'i-esimo strato massimale L_i è l'insieme dei punti massimali di $Q - \bigcup_{j=1}^{i-1} L_j$.

Supponete che Q abbia k strati massimali non vuoti; sia y_i la coordinata y del punto più a sinistra in L_i per $i = 1, 2, \ldots, k$. Per adesso supponete che due punti di Q non possano avere la stessa coordinata x o y.

a. Dimostrate che $y_1 > y_2 > \cdots > y_k$.

Considerate un punto (x, y) che sia a sinistra di qualsiasi punto di Q e la cui coordinata y sia diversa dalla coordinata y di qualsiasi altro punto di Q. Sia $Q' = Q \cup \{(x, y)\}$.

b. Sia j l'indice minimo tale che $y_j < y$, a meno che $y < y_k$, nel qual caso poniamo $j = k + 1$. Dimostrate che gli strati massimali di Q' sono i seguenti.

- Se $j \leq k$, allora gli strati massimali di Q' sono gli stessi di Q, con la differenza che L_j include anche (x, y) come suo nuovo punto più a sinistra.
- Se $j = k + 1$, allora i primi k strati massimali di Q' sono gli stessi di Q, ma in aggiunta Q' ha un $(k+1)$-esimo strato massimale non vuoto: $L_{k+1} = \{(x, y)\}$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:32 Numero Ordine Libraria 1995030362207070 Copyright © 2022 McGraw-Hill Education (Italy)

c. Descrivete un algoritmo con tempo $O(n \lg n)$ per calcolare gli strati massimali di un insieme Q di n punti (*suggerimento:* spostate una retta di sweeping da destra a sinistra).

d. Quali problemi potrebbero sorgere se consentissimo ai punti di input di avere la stessa coordinata x o y? Suggerite un modo per risolvere questi problemi.

33-3 Fantasmi e acchiappafantasmi

Un gruppo di n acchiappafantasmi sta combattendo contro n fantasmi. Ogni acchiappafantasma è armato con un fucile che spara raggi protonici in grado di eliminare i fantasmi. Un raggio protonico segue una linea retta e termina quando colpisce il fantasma. Gli acchiappafantasmi decidono di adottare la seguente strategia. Ciascuno di essi sarà abbinato a un fantasma, formando così n coppie fantasma-acchiappafantasma; poi contemporaneamente ogni acchiappafantasma sparerà un raggio protonico al proprio fantasma. Come saprete, è *molto* pericoloso incrociare i raggi protonici, e così gli acchiappafantasmi devono scegliere opportunamente gli accoppiamenti per evitare di incrociare i raggi protonici. Supponete che la posizione di ciascun acchiappafantasma e di ciascun fantasma sia un punto fisso nel piano e che non esistano tre posizioni collineari.

a. Dimostrate che esiste una retta che passa per un acchiappafantasma e un fantasma tale che il numero di acchiappafantasmi da un lato della retta sia uguale al numero di fantasmi nello stesso lato. Descrivete come trovare tale retta nel tempo $O(n \lg n)$.

b. Create un algoritmo con tempo $O(n^2 \lg n)$ per accoppiare acchiappafantasmi e fantasmi in modo tale che i raggi protonici non si incrocino.

33-4 Il gioco Shangai

Il professor Charon ha n bastoncini, che sono disposti uno sopra l'altro in una certa configurazione. Ogni bastoncino è identificato dai suoi estremi e ogni estremo è una tripla ordinata di coordinate (x, y, z). Nessun bastoncino è verticale. Il professore vuole raccogliere tutti i bastoncini, uno alla volta, con la condizione che un bastoncino può essere raccolto soltanto se non ci sono altri bastoncini su di esso.

a. Scrivete una procedura che riceve in input due bastoncini a e b e indica se a è sopra o sotto b o non ha alcuna relazione con b.

b. Create un algoritmo efficiente che determina se è possibile raccogliere tutti i bastoncini, e in caso affermativo, fornisce una sequenza valida in cui possono essere raccolti i bastoncini.

33-5 Distribuzioni di punti a involucro sparso

Consideriamo il problema di calcolare l'involucro convesso di un insieme di punti nel piano che sono stati selezionati secondo qualche distribuzione casuale nota. A volte, il numero di punti (o dimensione) dell'involucro convesso di n punti selezionati da una tale distribuzione ha un valore atteso di $O(n^{1-\epsilon})$ per qualche costante $\epsilon > 0$. Questa ***distribuzione*** è detta ***a involucro sparso*** (sparse hulled). Fra le distribuzioni a involucro sparso figurano le seguenti:

Acquistato da Michele Mastrogiovanni - Copia concessa da The McGraw-Hill Education (Italy) - (Copyright The McGraw-Hill Education (Italy)

- I punti selezionati uniformemente da un disco di raggio unitario. L'involucro convesso ha una dimensione attesa di $\Theta(n^{1/3})$.

- I punti selezionati uniformemente dalla parte interna di un poligono convesso con k lati, per qualsiasi costante k. L'involucro convesso ha una dimensione attesa di $\Theta(\lg n)$.
- I punti selezionati secondo una distribuzione normale bidimensionale. L'involucro convesso ha una dimensione attesa di $\Theta(\sqrt{\lg n})$.

a. Dati due poligoni convessi, rispettivamente, con n_1 e n_2 vertici, spiegate come calcolare l'involucro convesso di tutti gli $n_1 + n_2$ punti nel tempo $O(n_1 + n_2)$ (i poligoni possono sovrapporsi).

b. Dimostrate che l'involucro convesso di un insieme di n punti selezionati in modo arbitrario da una distribuzione sparse hulled può essere calcolato nel tempo atteso $O(n)$ (*suggerimento:* trovate ricorsivamente gli involucri convessi dei primi $n/2$ punti e dei secondi $n/2$ punti, poi combinate i risultati).

Note

Questo capitolo è solo una breve introduzione agli algoritmi e alle tecniche della geometria computazionale. Fra i libri di geometria computazionale citiamo quelli di Preparata e Shamos [283], Edelsbrunner [100] e O'Rourke [270].

Sebbene la geometria sia stata studiata fin dall'antichità, lo sviluppo degli algoritmi che risolvono i problemi geometrici è relativamente recente. Preparata e Shamos notano che la prima nozione di complessità di un problema fu data nel 1902 da E. Lemoine che, mentre studiava le costruzioni euclidee (quelle con riga e compasso), identificò un insieme di cinque primitive: posizionare un'asticella del compasso in un dato punto, posizionare un'asticella su una data retta, disegnare un cerchio, posizionare il filo del righello in un dato punto e tracciare una retta. Lemoine era interessato al numero di primitive richieste per realizzare una data costruzione; chiamò questa quantità "semplicità della costruzione".

L'algoritmo del Paragrafo 33.2, che determina se due segmenti si intersecano, è stato ideato da Shamos e Hoey [314].

La versione originale dell'algoritmo di Graham è stata descritta da Graham [151]. L'algoritmo basato sulla tecnica dell'impacchettamento è stato sviluppato da Jarvis [190]. Utilizzando un modello di calcolo basato sull'albero di decisione, Yao [360] ha dimostrato un limite inferiore pari a $\Omega(n \lg n)$ per il tempo di esecuzione di qualsiasi algoritmo che calcola l'involucro convesso. Se viene considerato il numero di vertici h dell'involucro convesso, l'algoritmo *prune-and-search* di Kirkpatrick e Seidel [207], che richiede un tempo $O(n \lg h)$, è asintoticamente ottimo.

L'algoritmo divide et impera con tempo $O(n \lg n)$ per trovare la coppia di punti più vicini è stato ideato da Shamos ed è descritto da Preparata e Shamos [283]. Preparata e Shamos dimostrano inoltre che l'algoritmo è asintoticamente ottimo in un modello basato sull'albero di decisione.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

NP-Completezza 34

34.1 Introduzione

Quasi tutti gli algoritmi finora analizzati sono ***algoritmi con tempo polinomiale***: con input di dimensione n, il loro tempo di esecuzione nel caso peggiore è $O(n^k)$ per qualche costante k. È naturale chiedersi se *tutti* i problemi possano essere risolti in tempo polinomiale. La risposta è no. Per esempio, ci sono problemi, come il noto "problema della terminazione" di Turing, che non può essere risolto da nessun calcolatore, indipendentemente dal tempo a disposizione. Inoltre, ci sono problemi che possono essere risolti, ma non nel tempo $O(n^k)$ per qualsiasi costante k. In generale, i problemi che sono risolvibili con algoritmi in tempo polinomiale sono considerati problemi trattabili o facili, mentre i problemi che richiedono un tempo superpolinomiale sono considerati problemi intrattabili o difficili.

Questo capitolo tratta una interessante classe di problemi, detti "problemi NP-completi", il cui stato non è noto. Sebbene non sia stato ancora scoperto un algoritmo con tempo polinomiale per un problema NP-completo, tuttavia nessuno ha finora dimostrato che non possa esistere un algoritmo con tempo polinomiale per uno di questi problemi. Stiamo parlando della cosiddetta "questione P $\neq$ NP" che, fin dalla sua formalizzazione nel 1971, è stato uno dei problemi più complessi e studiati dai ricercatori informatici.

Un aspetto particolarmente interessante dei problemi NP-completi è che molti di essi superficialmente sembrano simili a dei problemi per i quali conosciamo algoritmi con tempo polinomiale. In ciascuna delle seguenti coppie di problemi, uno è risolvibile in tempo polinomiale e l'altro è un problema NP-completo, ma la differenza tra i problemi sembra minima:

Cammini minimi e cammini massimi: nel Capitolo 24 abbiamo visto che anche con archi di peso negativo è possibile trovare i cammini *minimi* da una sorgente unica in un grafo orientato $G = (V, E)$ nel tempo $O(VE)$. È invece difficile trovare il cammino semplice *massimo* tra due vertici. Anche solo determinare se un grafo contiene un cammino semplice con almeno un dato numero di archi è un problema NP-completo.

Acquistato da Michele Mo... - Copyright © 2022 The McGraw-Hill Education (Italy)

Ciclo euleriano e ciclo hamiltoniano: un ***ciclo euleriano*** di un grafo orientato connesso $G = (V, E)$ è un ciclo che attraversa ciascun *arco* di G una sola volta, sebbene possa visitare un vertice più di una volta. Per il Problema 22-3 possiamo determinare se un grafo ha un ciclo euleriano in un tempo di appena $O(E)$ e, in effetti, possiamo trovare gli archi del ciclo euleriano nel tempo $O(E)$. Un ***ciclo hamiltoniano*** di un grafo orientato $G = (V, E)$ è un ciclo semplice che contiene ciascun *vertice* in V. Determinare se un grafo orientato

ha un ciclo hamiltoniano è un problema NP-completo. (Più avanti in questo capitolo, dimostreremo che determinare se un grafo *non orientato* ha un ciclo hamiltoniano è un problema NP-completo.)

Soddisfattibilità di formule 2-CNF e soddisfattibilità di formule 3-CNF: una formula booleana contiene variabili i cui valori sono 0 o 1, connettivi booleani come $\wedge$ (AND), $\vee$ (OR) e $\neg$ (NOT), e parentesi. Una formula booleana è ***soddisfattibile*** se esiste un'assegnazione di valori 0 e 1 per le sue variabili che la rende pari a 1. In seguito definiremo i termini in modo più formale; per il momento, diciamo informalmente che una formula booleana è nella ***forma normale k-congiuntiva*** o k-CNF, se è espressa come l'AND di clausole OR di k variabili o delle loro negazioni. Per esempio, la formula booleana $(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3)$ è nella forma 2-CNF (ha anche l'assegnazione di soddisfattibilità $x_1 = 1, x_2 = 0, x_3 = 1$). Esiste un algoritmo con tempo polinomiale per determinare se una formula 2-CNF è soddisfattibile, ma come vedremo più avanti in questo capitolo, determinare se una formula 3-CNF è soddisfattibile è un problema NP-completo.

NP-completezza e classi P ed NP

In questo capitolo, faremo riferimento a tre classi di problemi: P, NP ed NPC; a quest'ultima classe appartengono i problemi NP-completi. Descriviamo adesso queste classi in modo informale; successivamente le definiremo in modo più formale.

La classe P è formata da quei problemi che sono risolvibili in tempo polinomiale. Più specificatamente, sono problemi che possono essere risolti nel tempo $O(n^k)$ per qualche costante k, dove n è la dimensione dell'input del problema. Molti dei problemi esaminati nei precedenti capitoli sono in classe P.

La classe NP è formata da quei problemi che sono "verificabili" in tempo polinomiale. Con questo intendiamo che se avessimo ricevuto in qualche maniera un "certificato" di una soluzione, allora potremmo verificare che il certificato è corretto in tempo polinomiale nella dimensione dell'input del problema. Per esempio, nel problema del ciclo hamiltoniano, dato un grafo orientato $G = (V, E)$, un certificato sarebbe una sequenza $\langle v_1, v_2, v_3, \ldots, v_{|V|} \rangle$ di $|V|$ vertici. È facile verificare in tempo polinomiale che $(v_i, v_{i+1}) \in E$ per $i = 1, 2, 3, \ldots, |V| - 1$ e che $(v_{|V|}, v_1) \in E$. Come altro esempio, per la soddisfattibilità delle formule 3-CNF, un certificato sarebbe un'assegnazione di valori alle variabili. Possiamo facilmente verificare in tempo polinomiale che questa assegnazione soddisfa la formula booleana.

Un problema qualsiasi in classe P è anche in classe NP, perché se un problema è in P allora possiamo risolverlo in tempo polinomiale senza avere un certificato. Formalizzeremo questo concetto più avanti nel capitolo; per adesso, possiamo pensare che P $\subseteq$ NP. La questione aperta è se P sia oppure no un sottoinsieme proprio di NP.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

Informalmente, un problema appartiene alla classe NPC – e diremo che è ***NP-completo*** – se è in classe NP ed è "difficile" da risolvere come qualsiasi problema in classe NP. Più avanti nel capitolo definiremo formalmente che cosa intendiamo con "problema difficile da risolvere come qualsiasi problema in classe NP". Per il momento, asseriamo senza dimostrarlo che se un *qualsiasi* problema NP-completo può essere risolto in tempo polinomiale, allora *tutti* i problemi in NP

hanno un algoritmo con tempo polinomiale. La maggior parte degli scienziati informatici ritiene che i problemi NP-completi siano intrattabili perché, considerando l'ampia gamma di problemi NP-completi che sono stati finora studiati – senza che nessuno sia riuscito a trovare una soluzione con tempo polinomiale per uno qualsiasi di questi problemi – sarebbe davvero sbalorditivo se tutti potessero essere risolti in tempo polinomiale. D'altra parte, considerando gli sforzi finora compiuti per dimostrare che i problemi NP-completi sono intrattabili – senza un risultato conclusivo – non possiamo escludere la possibilità che i problemi NP-completi siano effettivamente risolvibili in tempo polinomiale.

Per diventare buoni progettisti di algoritmi, bisogna capire i concetti fondamentali della teoria della NP-completezza. Se si può stabilire che un problema è NP-completo, allora si è in grado di fornire una buona prova della sua intrattabilità. In questo caso, un buon progettista farebbe meglio a dedicare il proprio tempo allo sviluppo di un algoritmo di approssimazione (vedere il Capitolo 35) o alla soluzione di un caso speciale trattabile, anziché ricercare un algoritmo rapido che risolve con esattezza il problema. Inoltre, molti interessanti problemi naturali che superficialmente non sembrano più difficili dei problemi di ordinamento, di ricerca in un grafo o di flusso su rete, in effetti, sono problemi NP-completi. Dunque, è importante acquisire dimestichezza con questa speciale classe di problemi.

Tecniche per identificare i problemi NP-completi

Le tecniche utilizzate per provare che un particolare problema è NP-completo differiscono da quelle finora utilizzate nella maggior parte di questo libro per progettare e analizzare gli algoritmi. C'è una ragione fondamentale che spiega questa differenza: per dimostrare che un problema è NP-completo, definiamo quanto difficile sia il problema (o almeno quanto noi crediamo che sia difficile), non quanto sia facile. Non stiamo tentando di provare l'esistenza di un algoritmo efficiente, ma piuttosto che è poco probabile che esista un algoritmo efficiente. In questo modo, le dimostrazioni della NP-completezza sono in qualche modo simili alla dimostrazione di un limite inferiore $\Omega(n \lg n)$ per il tempo di esecuzione di qualsiasi algoritmo di ordinamento per confronti (Paragrafo 8.1); tuttavia, le tecniche specifiche utilizzate per provare la NP-completezza differiscono dal metodo dell'albero di decisione utilizzato nel Paragrafo 8.1.

La dimostrazione che un problema è NP-completo si basa su tre concetti fondamentali, qui di seguito descritti.

Problemi di decisione e problemi di ottimizzazione

Molti fra i problemi più interessanti sono ***problemi di ottimizzazione***, in cui ogni soluzione ammissibile (cioè "legale") ha un valore associato, e noi vogliamo trovare la soluzione ammissibile con il valore migliore. Per esempio, in un problema che chiamiamo SHORTEST-PATH, abbiamo un grafo non orientato G e i vertici u e v e vogliamo trovare un cammino da u a v che usa il minor numero di archi (in altre parole, SHORTEST-PATH è il problema del cammino minimo per una coppia di vertici in un grafo non orientato e non pesato). Notate, tuttavia, che la NP-completezza non si applica direttamente ai problemi di ottimizzazione, ma ai ***problemi di decisione***, in cui la risposta è semplicemente "sì" o "no" (o, più formalmente, "1" o "0").

Acquistato da Michele Mirenda - Vietata la riproduzione non autorizzata - Copyright © 2023 McGraw-Hill Education (Italy)

Sebbene dimostrare che un problema è NP-completo ci confini nel settore dei problemi di decisione, esiste una comoda relazione tra i problemi di ottimizza-

zione e i problemi di decisione. Di solito, è possibile trattare un dato problema di ottimizzazione con un problema di decisione associato, imponendo un limite al valore da ottimizzare. Per SHORTEST-PATH, ad esempio, un problema di decisione associato, che chiamiamo PATH, consiste nel determinare se, conoscendo un grafo orientato G, i vertici u e v e un intero k, esiste un cammino da u a v formato al più da k archi.

La relazione tra un problema di ottimizzazione e il problema di decisione associato opera in nostro favore quando tentiamo di dimostrare che il problema di ottimizzazione è "difficile". Questo perché il problema di decisione è in un certo senso "più facile" o almeno "non più difficile". Come esempio specifico, possiamo risolvere PATH risolvendo SHORTEST-PATH e poi confrontando il numero di archi nel cammino minimo trovato con il valore del parametro k del problema di decisione. In altre parole, se un problema di ottimizzazione è facile, anche il problema di decisione associato è facile. Esprimendoci in modo più attinente alla NP-completezza, se possiamo fornire una prova che un problema di decisione è difficile, possiamo anche fornire una prova che il problema di ottimizzazione associato è difficile. Quindi, anche se la teoria della NP-completezza restringe l'attenzione ai problemi di decisione, spesso ha implicazioni anche nei problemi di ottimizzazione.

Riduzioni

Il precedente concetto di dimostrare che un problema non è più difficile o più semplice di un altro si applica anche quando entrambi i problemi sono problemi di decisione. Sfrutteremo questo concetto in quasi tutte le dimostrazioni della NP-completezza nel modo seguente. Consideriamo un problema di decisione, per esempio A, che vorremmo risolvere in tempo polinomiale. Chiamiamo l'input di un particolare problema ***istanza*** del problema; per esempio, in PATH un'istanza sarebbe formata da un particolare grafo G, da particolari vertici u e v di G e da un particolare intero k. Adesso supponiamo che ci sia un altro problema di decisione, per esempio B, che sappiamo risolvere in tempo polinomiale. Infine, supponiamo di avere una procedura che trasforma un'istanza α di A in qualche istanza β di B con le seguenti caratteristiche:

1. La trasformazione richiede un tempo polinomiale.
2. Le soluzioni sono le stesse. Ovvero la soluzione per α è "sì" se e soltanto se anche la soluzione per β è "sì".

Una tale procedura si chiama ***algoritmo di riduzione*** con tempo polinomiale e, come illustra la Figura 34.1, fornisce un modo per risolvere il problema A in tempo polinomiale:

1. Data un'istanza α del problema A, utilizziamo un algoritmo di riduzione con tempo polinomiale per trasformarla in un'istanza β del problema B.
2. Eseguiamo sull'istanza β l'algoritmo di decisione con tempo polinomiale per il problema B.
3. Utilizziamo la soluzione per β come soluzione per α.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995030316-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Se ciascuno di questi passi impiega un tempo polinomiale, tutti e tre i passi impiegheranno un tempo polinomiale, e quindi abbiamo un modo per decidere su α in tempo polinomiale. In altre parole, "riducendo" la risoluzione del problema A

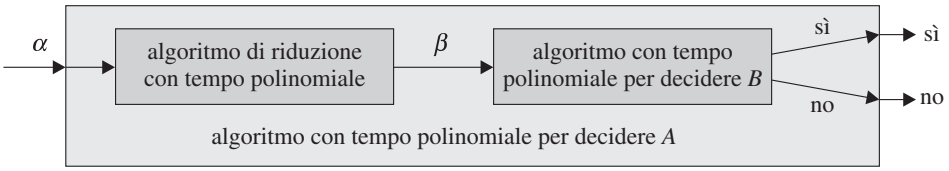


Figura 34.1 Usare un algoritmo di riduzione con tempo polinomiale per risolvere un problema di decisione A in tempo polinomiale, dato un algoritmo di decisione con tempo polinomiale per un altro problema B. Trasformiamo in tempo polinomiale un'istanza α di A in un'istanza β di B, risolviamo B in tempo polinomiale e utilizziamo la soluzione per β come soluzione per α.

alla risoluzione del problema B, utilizziamo la "facilità" di B per provare la "facilità" di A. Ricordando che la NP-completezza consiste nel dimostrare quanto difficile sia un problema, non quanto sia facile, utilizziamo le riduzioni in tempo polinomiale all'inverso per dimostrare che un problema è NP-completo. Estendendo ulteriormente questo concetto, potremmo utilizzare le riduzioni in tempo polinomiale per provare che non esiste un algoritmo con tempo polinomiale per un particolare problema B. Supponiamo di avere un problema di decisione A per il quale sappiamo già che non può esistere un algoritmo con tempo polinomiale (per adesso non ci interessa sapere come trovare un tale problema A). Supponiamo inoltre di avere una riduzione in tempo polinomiale che trasforma le istanze di A in istanze di B. Adesso possiamo utilizzare una semplice dimostrazione per assurdo per provare che non può esistere un algoritmo con tempo polinomiale per B. Supponiamo l'opposto e cioè che B abbia un algoritmo con tempo polinomiale. Allora, utilizzando il metodo illustrato nella Figura 34.1, avremmo un modo per risolvere il problema A in tempo polinomiale, e questo contraddice l'ipotesi che non esiste un algoritmo con tempo polinomiale per A.

Per la NP-completezza, non possiamo supporre che non esista assolutamente un algoritmo con tempo polinomiale per il problema A. La metodologia della dimostrazione è analoga, in quanto proviamo che il problema B è NP-completo in base all'ipotesi che anche il problema A sia NP-completo.

Un primo problema NP-completo

Poiché la tecnica di riduzione si basa sul fatto di conoscere un problema NP-completo per potere dimostrare che un altro problema è NP-completo, occorre un "primo" problema NP-completo. Il problema che utilizzeremo è quello della soddisfattibilità dei circuiti, in cui abbiamo un circuito combinatorio booleano composto da porte AND, OR e NOT, e vogliamo conoscere se esiste un insieme di input booleani per questo circuito che fa sì che il suo output sia 1. Dimostreremo che questo primo problema è NP-completo nel Paragrafo 34.4.

Organizzazione del capitolo

Questo capitolo studia gli aspetti della NP-completezza che riguardano più direttamente l'analisi degli algoritmi. Nel Paragrafo 34.2 formalizzeremo il concetto di "problema" e definiremo la classe di complessità P dei problemi di decisione risolvibili in tempo polinomiale. Vedremo anche come questi concetti si adattino al contesto della teoria dei linguaggi formali. Il Paragrafo 34.3 definisce la classe NP dei problemi di decisione le cui soluzioni possono essere verificate in tempo polinomiale. Viene anche posta formalmente la questione $\mathrm{P} \neq \mathrm{NP}$.

Acquistato da Michele Michele su Webster il 2022-07-07 20:12 Tomeo Strade Libreria. 1955031E0220707.0 Copyright © 2022, McGraw-Hill Education (Italy)

Il Paragrafo 34.4 spiega come utilizzare le "riduzioni" in tempo polinomiale per studiare le relazioni tra i problemi. Definisce la NP-completezza e riporta una dimostrazione schematica che un problema, detto "soddisfattibilità dei circuiti", è NP-completo. Dopo avere trovato un problema NP-completo, nel Paragrafo 34.5

spieghiamo come dimostrare che altri problemi sono NP-completi in modo più semplice con la tecnica delle riduzioni. La metodologia è illustrata dimostrando che due problemi di soddisfattibilità delle formule sono NP-completi. Il Paragrafo 34.6 presenta altri problemi di cui è possibile provare la NP-completezza.

34.2 Tempo polinomiale

Iniziamo lo studio della NP-completezza formalizzando il concetto dei problemi risolvibili in tempo polinomiale. Questi problemi, in generale, sono considerati trattabili per ragioni filosofiche, non matematiche. A sostegno di questo possiamo addurre tre argomenti.

In primo luogo, sebbene sia ragionevole considerare intrattabile un problema che richiede un tempo $\Theta(n^{100})$, ci sono pochissimi problemi che nella pratica richiedono un tempo polinomiale di grado così elevato. Tipicamente, i problemi calcolabili in tempo polinomiale che si incontrano nella pratica richiedono molto meno tempo. L'esperienza ha dimostrato che, una volta che viene scoperto un algoritmo con tempo polinomiale per un problema, spesso seguono altri algoritmi più efficienti. Anche se attualmente il migliore algoritmo per un problema ha un tempo di esecuzione di $\Theta(n^{100})$, è probabile che presto sarà scoperto un algoritmo con un tempo di esecuzione molto migliore.

In secondo luogo, per molti modelli di calcolo, un problema che può essere risolto in tempo polinomiale in un modello può essere risolto sempre in tempo polinomiale in un altro modello. Per esempio, la classe dei problemi risolvibili in tempo polinomiale dalla macchina seriale ad accesso casuale, utilizzata in gran parte di questo libro, è la stessa classe dei problemi risolvibili in tempo polinomiale nelle macchine astratte di Turing.[1] È anche la stessa classe dei problemi risolvibili in tempo polinomiale in un calcolatore parallelo quando il numero di processori cresce polinomialmente con la dimensione dell'input.

In terzo luogo, la classe dei problemi risolvibili in tempo polinomiale ha delle interessanti proprietà di chiusura, in quanto i polinomi sono chiusi rispetto all'addizione, alla moltiplicazione e alla composizione. Per esempio, se l'output di un algoritmo con tempo polinomiale è utilizzato come input di un altro, l'algoritmo composto è polinomiale. L'Esercizio 34.2-5 vi chiede di dimostrare che se un algoritmo effettua un numero costante di chiamate a una subroutine che richiede tempo polinomiale ed effettua una quantità di lavoro addizionale che richiede tempo polinomiale, allora il tempo calcolo richiesto dall'algoritmo composto è anch'esso polinomiale.

Problemi astratti

Per capire la classe dei problemi risolvibili in tempo polinomiale, dobbiamo avere innanzi tutto una nozione formale di ciò che è un "problema". Definiamo un ***problema astratto*** Q come una relazione binaria tra un insieme I di ***istanze*** del problema e un insieme S di ***soluzioni*** del problema. Per esempio, un istanza per SHORTEST-PATH è una tripla formata da un grafo e due vertici. Una soluzione è una sequenza di vertici nel grafo, con eventualmente la sequenza vuota a indicare che non esiste alcun cammino. Il problema SHORTEST-PATH è quindi la relazio-

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995038016-2297070 Copyright © 2022 McGraw-Hill Education (Italy)

[1] Per una trattazione completa del modello della macchina di Turing, consultate Hopcroft e Ullman [181] o Lewis e Papadimitriou [237].

ne che associa ogni istanza di un grafo e due vertici a un cammino minimo che collega i due vertici. Poiché i cammini minimi non sono necessariamente unici, una determinata istanza del problema può avere più di una soluzione.

Questa formulazione di problema astratto è più generale di quella richiesta per i nostri scopi. Come detto in precedenza, la teoria della NP-completezza restringe l'attenzione ai ***problemi di decisione***: quelli che hanno una soluzione sì/no. In questo caso, possiamo considerare un problema di decisione astratto come una funzione che mappa l'insieme delle istanze I nell'insieme delle soluzioni $\{0, 1\}$. Per esempio, un problema di decisione associato a SHORTEST-PATH è il problema PATH che abbiamo visto in precedenza. Se $i = \langle G, u, v, k\rangle$ è un'istanza del problema di decisione PATH, allora $\text{PATH}(i) = 1$ (sì) se un cammino minimo da u a v ha al più k archi, altrimenti $\text{PATH}(i) = 0$ (no). Molti problemi astratti non sono problemi di decisione, ma ***problemi di ottimizzazione***, dove qualche valore deve essere minimizzato o massimizzato. Tuttavia, come abbiamo detto precedentemente, di solito è semplice trasformare un problema di ottimizzazione in un problema di decisione che non è più difficile da risolvere.

Codifica

Se un programma per calcolatore deve risolvere un problema astratto, le istanze del problema devono essere rappresentate in modo che possano essere capite dal programma. Una ***codifica*** di un insieme S di oggetti astratti è una funzione e che mappa S nell'insieme delle stringhe binarie.[2] Per esempio, abbiamo tutti una certa dimestichezza con la codifica dei numeri naturali $\mathbb{N} = \{0, 1, 2, 3, 4, \ldots\}$ come stringhe $\{0, 1, 10, 11, 100, \ldots\}$. Utilizzando questa codifica, $e(17) = 10001$. Chi ha esaminato le rappresentazioni dei caratteri della tastiera in un calcolatore avrà dimestichezza con il codice ASCII dove, per esempio, la codifica di `A` è 1000001. Anche un oggetto composto può essere codificato come una stringa binaria, combinando le rappresentazioni delle parti che lo compongono. I poligoni, i grafi, le funzioni, le coppie ordinate, i programmi – tutto può essere codificato sotto forma di stringhe binarie. Quindi, un algoritmo che "risolve" qualche problema di decisione astratto, in effetti, riceve come input la codifica di un'istanza del problema.

Chiameremo ***problema concreto*** un problema il cui insieme di istanze è l'insieme delle stringhe binarie. Diremo che un algoritmo ***risolve*** un problema concreto nel tempo $O(T(n))$ se, quando riceve un'istanza i del problema di lunghezza $n = |i|$, l'algoritmo è capace di produrre la soluzione nel tempo $O(T(n))$.[3] Pertanto, un problema concreto è ***risolvibile in tempo polinomiale***, se esiste un algoritmo che lo risolve nel tempo $O(n^k)$ per qualche costante k.

Adesso possiamo definire formalmente la ***classe di complessità*** **P** come l'insieme dei problemi di decisione concreti che sono risolvibili in tempo polinomiale.

Possiamo utilizzare la codifica per mappare un problema astratto in un problema concreto. Dato un problema di decisione astratto Q che mappa un insieme di istanze I nell'insieme $\{0, 1\}$, è possibile utilizzare una codifica $e : I \to \{0, 1\}^*$

Acquistato da Michele Mistrangelo Vietata la distribuzione non autorizzata © 2022 McGraw-Hill Education (Italy)

[2] Il codominio della funzione e non deve essere necessariamente formato da stringhe *binarie*; è accettabile un insieme qualsiasi di stringhe su un alfabeto finito che ha almeno 2 simboli.

[3] Si suppone che l'output dell'algoritmo sia disgiunto dal suo input. Poiché occorre almeno un intervallo temporale per produrre ciascun bit dell'output e ci sono $O(T(n))$ intervalli temporali, la dimensione dell'output è $O(T(n))$.

per generare un problema di decisione concreto, che indichiamo con $e(Q)$.[4] Se la soluzione di un'istanza del problema astratto $i \in I$ è $Q(i) \in \{0,1\}$, allora anche la soluzione dell'istanza del problema concreto $e(i) \in \{0,1\}^*$ è $Q(i)$. Tecnicamente, ci possono essere stringhe binarie che non rappresentano istanze significative del problema astratto. Per comodità, assumeremo che queste stringhe siano arbitrariamente mappate a 0. Quindi, il problema concreto produce le stesse soluzioni del problema astratto con istanze di stringhe binarie che rappresentano le codifiche di istanze del problema astratto.

Vogliamo estendere la definizione di risolvibilità in tempo polinomiale dai problemi concreti ai problemi astratti utilizzando la codifica come ponte, ma vorremmo che la definizione sia indipendente da qualsiasi tipo di codifica. Ovvero, l'efficienza della risoluzione di un problema non deve dipendere dal modo in cui il problema è codificato. Purtroppo, essa dipende fortemente dalla codifica. Per esempio, supponiamo che un intero k debba essere passato come unico input a un algoritmo; supponiamo inoltre che il tempo di esecuzione dell'algoritmo sia $\Theta(k)$. Se l'intero k è passato nel formato ***unario*** – una stringa di k 1 – allora il tempo di esecuzione dell'algoritmo è $O(n)$ con gli input di lunghezza n, che è un tempo polinomiale. Se invece utilizziamo la più naturale rappresentazione binaria dell'intero k, allora la lunghezza dell'input è $n = \lfloor \lg k \rfloor + 1$. In questo caso, il tempo di esecuzione dell'algoritmo è $\Theta(k) = \Theta(2^n)$, che è esponenziale nella dimensione dell'input. Quindi, a seconda della codifica, l'algoritmo viene eseguito in tempo polinomiale o in tempo superpolinomiale.

La codifica di un problema astratto è quindi molto importante per capire la nozione di tempo polinomiale. In effetti, non possiamo parlare di risoluzione di un problema astratto se non specifichiamo prima una codifica. Ciò nonostante, se escludiamo le codifiche "costose", come quella unaria, l'effettiva codifica di un problema praticamente incide poco sul fatto che il problema possa essere risolto in tempo polinomiale. Per esempio, rappresentare gli interi in base 3, anziché in base 2, non ha alcun effetto sul fatto che un problema sia risolvibile in tempo polinomiale, perché un intero in base 3 può essere trasformato in un intero in base 2 in un tempo polinomiale.

Diciamo che una funzione $f : \{0,1\}^* \to \{0,1\}^*$ è ***calcolabile in tempo polinomiale*** se esiste un algoritmo A con tempo polinomiale che, dato un input $x \in \{0,1\}^*$, produce come output $f(x)$. Per qualche insieme di istanze I del problema, diciamo che due codifiche e_1 ed e_2 sono ***polinomialmente correlate*** se esistono due funzioni calcolabili in tempo polinomiale f_{12} ed f_{21} tali che, per qualsiasi $i \in I$, si abbia $f_{12}(e_1(i)) = e_2(i)$ ed $f_{21}(e_2(i)) = e_1(i)$.[5] Ovvero, la codifica $e_2(i)$ può essere calcolata dalla codifica $e_1(i)$ mediante un algoritmo con tempo polinomiale, e viceversa. Se due codifiche e_1 ed e_2 di un problema astratto sono polinomialmente correlate, il fatto che il problema sia risolvibile in tempo polinomiale oppure no non dipende dal tipo di codifica utilizzato, come dimostra il seguente lemma.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[4] Come vedremo più avanti, $\{0,1\}^*$ indica l'insieme di tutte le stringhe composte da simboli dell'insieme $\{0,1\}$.

[5] Tecnicamente, richiediamo anche che le funzioni f_{12} ed f_{21} "mappino non-istanze in non-istanze". Una ***non-istanza*** di una codifica e è una stringa $x \in \{0,1\}^*$ tale che non esiste un'istanza i per la quale $e(i) = x$. Richiediamo che $f_{12}(x) = y$ per ogni non-istanza x della codifica e_1, dove y è qualche non-istanza di e_2, e che $f_{21}(x') = y'$ per ogni non-istanza x' di e_2, dove y' è qualche non-istanza di e_1.

Lemma 34.1
Sia Q un problema di decisione astratto in un insieme di istanze I e siano e_1 ed e_2 due codifiche polinomialmente correlate in I. Allora, $e_1(Q) \in \text{P}$ se e soltanto se $e_2(Q) \in \text{P}$.

Dimostrazione È sufficiente dimostrare il lemma soltanto nel senso diretto, in quanto il senso inverso è simmetrico. Supponiamo, quindi, che $e_1(Q)$ possa essere risolto nel tempo $O(n^k)$ per qualche costante k. Inoltre, supponiamo che per qualsiasi istanza i del problema, la codifica $e_1(i)$ possa essere calcolata dalla codifica $e_2(i)$ nel tempo $O(n^c)$ per qualche costante c, dove $n = |e_2(i)|$. Per risolvere il problema $e_2(Q)$ con l'input $e_2(i)$, prima calcoliamo $e_1(i)$ e poi eseguiamo l'algoritmo per $e_1(Q)$ con l'input $e_1(i)$. Quanto tempo occorre? La conversione delle codifiche richiede un tempo $O(n^c)$, e quindi $|e_1(i)| = O(n^c)$, perché l'output di un calcolatore seriale non può essere più lungo del suo tempo di esecuzione. Per risolvere il problema con $e_1(i)$ occorre un tempo $O(|e_1(i)|^k) = O(n^{ck})$, che è polinomiale perché c e k sono costanti. ■

Quindi, il fatto che un problema astratto abbia le sue istanze codificate in base 2 o in base 3 non influisce sulla sua "complessità", cioè sul fatto che sia risolvibile in tempo polinomiale oppure no, ma se le istanze sono codificate in unario, la complessità può cambiare. Per poter parlare in maniera indipendente dalla codifica, in generale si suppone che le istanze del problema siano codificate secondo qualche schema valido e conciso, a meno che non sia indicato diversamente. Più precisamente, assumeremo che la codifica di un intero sia polinomialmente correlata alla sua rappresentazione binaria e che la codifica di un insieme finito sia polinomialmente correlata alla sua codifica come una lista dei suoi elementi, racchiusi fra parentesi graffe e separati da virgole. (ASCII è uno di questi schemi di codifica.) Con un tale schema di codifica "standard" è possibile derivare delle valide codifiche di altri oggetti matematici, come tuple, grafi e formule. Per indicare la codifica standard di un oggetto, racchiuderemo l'oggetto tra parentesi angolari. Quindi, $\langle G \rangle$ indica la codifica standard di un grafo G.

Finché utilizziamo implicitamente una codifica che è polinomialmente correlata a questa codifica standard, possiamo parlare direttamente di problemi astratti senza fare riferimento a qualche particolare codifica, sapendo che la scelta della codifica non influisce sulla risolvibilità in tempo polinomiale del problema astratto. D'ora in avanti, assumeremo in generale che tutte le istanze del problema siano stringhe binarie codificate secondo la codifica standard, a meno che non specifichiamo esplicitamente un'altra codifica. Inoltre ignoreremo la distinzione tra problemi astratti e problemi concreti. Tuttavia, il lettore dovrebbe valutare attentamente i problemi che si presentano nella pratica, quando la codifica standard non è ovvia e il tipo di codifica influisce sulla complessità degli algoritmi.

Struttura dei linguaggi formali

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Uno dei vantaggi di focalizzare l'attenzione sui problemi di decisione è che ad essi è facile applicare i meccanismi della teoria dei linguaggi formali. È utile a questo punto rivedere qualche definizione di questa teoria. Un ***alfabeto*** Σ è un insieme finito di simboli. Un ***linguaggio*** L su Σ è un insieme qualsiasi di stringhe formato da simboli di Σ. Per esempio, se $\Sigma = \{0, 1\}$, l'insieme $L = \{10, 11, 101, 111, 1011, 1101, 10001, \ldots\}$ è il linguaggio delle rappresentazioni binarie dei numeri primi. Indichiamo con ε la ***stringa vuota*** e con $\emptyset$ il

linguaggio vuoto. Il linguaggio di tutte le stringhe su Σ è indicato con Σ^*. Per esempio, se $\Sigma = \{0, 1\}$, allora $\Sigma^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, \ldots\}$ è l'insieme di tutte le stringhe binarie. Ogni linguaggio L su Σ è un sottoinsieme di Σ^*.

È possibile eseguire varie operazioni con i linguaggi. Le operazioni sugli insiemi, come l'***unione*** e l'***intersezione***, derivano direttamente dalle definizioni della teoria degli insiemi. Definiamo ***complemento*** del linguaggio L come $\overline{L} = \Sigma^* - L$. La ***concatenazione*** di due linguaggi L_1 e L_2 è il linguaggio

$$L = \{x_1 x_2 : x_1 \in L_1 \text{ e } x_2 \in L_2\}$$

La ***chiusura*** o ***stella di Kleene*** di un linguaggio L è il linguaggio

$$L^* = \{\varepsilon\} \cup L \cup L^2 \cup L^3 \cup \cdots$$

dove L^k è il linguaggio ottenuto concatenando L con sé stesso k volte.

Dal punto di vista della teoria dei linguaggi, l'insieme delle istanze per un problema qualsiasi di decisione Q è semplicemente l'insieme Σ^*, dove $\Sigma = \{0, 1\}$. Poiché Q è interamente caratterizzato da quelle istanze del problema che producono una risposta 1 (sì), possiamo considerare Q come un linguaggio L su $\Sigma = \{0, 1\}$, dove

$$L = \{x \in \Sigma^* : Q(x) = 1\}$$

Per esempio, il problema di decisione PATH ha il linguaggio corrispondente

$$\begin{aligned}\text{PATH} = \{\langle G, u, v, k\rangle : \ & G = (V, E) \text{ è un grafo non orientato} \\ & u, v \in V \\ & k \geq 0 \text{ è un intero ed} \\ & \text{esiste un cammino da } u \text{ a } v \text{ in } G \\ & \text{formato al più da } k \text{ archi}\}\end{aligned}$$

(A volte, per comodità, utilizzeremo lo stesso nome – PATH in questo caso – per fare riferimento sia a un problema di decisione sia al suo linguaggio corrispondente.)

La struttura dei linguaggi formali ci permette di esprimere in modo conciso la relazione tra i problemi di decisione e gli algoritmi che li risolvono. Diremo che un algoritmo A ***accetta*** una stringa $x \in \{0, 1\}^*$ se, dato l'input x, l'output dell'algoritmo $A(x)$ è 1. Il linguaggio ***accettato*** da un algoritmo A è l'insieme delle stringhe $L = \{x \in \{0, 1\}^* : A(x) = 1\}$, cioè l'insieme delle stringhe che l'algoritmo accetta. Un algoritmo A ***rifiuta*** una stringa x se $A(x) = 0$.

Anche se il linguaggio L è accettato da un algoritmo A, l'algoritmo non necessariamente rifiuterà una stringa $x \notin L$ che riceve come input. Per esempio, l'algoritmo potrebbe ciclare indefinitamente. Un linguaggio L è ***deciso*** da un algoritmo A se ogni stringa binaria in L è accettata da A e ogni stringa binaria che non appartiene a L è rifiutata da A.

Un linguaggio L è ***accettato in tempo polinomiale*** da un algoritmo A se è accettato da A e se inoltre esiste una costante k tale che per qualsiasi stringa $x \in L$ di lunghezza n, l'algoritmo A accetta x nel tempo $O(n^k)$. Un linguaggio L è ***deciso in tempo polinomiale*** da un algoritmo A se esiste una costante k tale che per qualsiasi stringa $x \in \{0, 1\}^*$ di lunghezza n, l'algoritmo decide correttamente se $x \in L$ nel tempo $O(n^k)$. Quindi, per accettare un linguaggio, un algoritmo deve occuparsi soltanto delle stringhe in L, ma per decidere un linguaggio, deve accettare o rifiutare correttamente ogni stringa in $\{0, 1\}^*$.

Acquistato da Michele Michele su Webster il 2022-[illegible] Vietata la copia e la distribuzione a terzi per qualsiasi scopo (Copyright Mondadori Education S.p.A.)

Per esempio, il linguaggio PATH può essere accettato in tempo polinomiale. Un algoritmo di accettazione con tempo polinomiale verifica se G codifica un grafo non orientato, verifica se u e v sono vertici in G, usa la visita in ampiezza per trovare un cammino minimo da u a v in G e poi confronta il numero di archi nel cammino minimo ottenuto con k. Se G codifica un grafo non orientato e il cammino da u a v ha al più k archi, l'algoritmo restituisce l'output 1 e termina, altrimenti continua la sua esecuzione all'infinito. Questo algoritmo non decide PATH, perché non restituisce esplicitamente l'output 0 per le istanze in cui un cammino minimo ha più di k archi. Un algoritmo di decisione per PATH deve rifiutare esplicitamente le stringhe binarie che non appartengono a PATH. Per un problema di decisione come PATH, è facile progettare un tale algoritmo di decisione: anziché eseguire un ciclo all'infinito quando non esiste un cammino da u a v con al più k archi, l'algoritmo restituisce l'output 0 e termina. Per altri problemi, come il problema della terminazione di Turing, esiste un algoritmo di accettazione, ma non esiste un algoritmo di decisione.

Possiamo definire informalmente una ***classe di complessità*** come un insieme di linguaggi, per i quali l'appartenenza alla classe è stabilita in base a una ***misura di complessità***, come il tempo di esecuzione, di un algoritmo che determina se una data stringa x appartiene al linguaggio L. La definizione effettiva di classe di complessità è molto più tecnica.[6]

Utilizzando questa struttura di linguaggi formali, possiamo fornire una definizione alternativa della classe di complessità P:

$$\mathrm{P} = \{L \subseteq \{0,1\}^* : \text{esiste un algoritmo } A \text{ che decide } L \text{ in tempo polinomiale}\}$$

In effetti, P è anche la classe dei linguaggi che possono essere accettati in tempo polinomiale.

Teorema 34.2
$\mathrm{P} = \{L : L \text{ è accettato da un algoritmo con tempo polinomiale}\}$

Dimostrazione Poiché la classe dei linguaggi decisi dagli algoritmi con tempo polinomiale è un sottoinsieme della classe dei linguaggi accettati dagli algoritmi con tempo polinomiale, occorre dimostrare soltanto che, se L è accettato da un algoritmo con tempo polinomiale, è deciso da un algoritmo con tempo polinomiale. Sia L il linguaggio accettato da qualche algoritmo con tempo polinomiale A. Utilizzeremo un metodo classico di "simulazione" per costruire un altro algoritmo con tempo polinomiale A' che decide L. Poiché A accetta L nel tempo $O(n^k)$ per qualche costante k, esiste anche una costante c tale che A accetta L al più in $T = cn^k$ passi. Per una stringa qualsiasi di input x, l'algoritmo A' simula l'azione di A per il tempo T. Trascorso il tempo T, l'algoritmo A' esamina il comportamento di A. Se A ha accettato x, allora A' accetta x restituendo l'output 1. Se A non ha accettato x, allora A' rifiuta x restituendo l'output 0. Il tempo extra richiesto da A' per simulare A non aumenta il tempo di esecuzione per più di un fattore polinomiale, e quindi A' è un algoritmo con tempo polinomiale che decide L. ■

Acquistato da Michele Michelazzo Webster il 2022-07-07 23:13 Numero Ordine Libreria 1858016 220707 0 Copyright © 2022 McGraw-Hill Education (Italy)

[6] Per maggiori informazioni sulle classi di complessità, consultare il documento di Hartmanis e Stearns [163].

Notiamo che la dimostrazione del Teorema 34.2 non è costruttiva. Per un dato linguaggio $L \in$ P, in effetti potremmo non conoscere un limite per il tempo di esecuzione dell'algoritmo A che accetta L. Ciò nonostante, sappiamo che tale limite esiste e, quindi, esiste un algoritmo A' che può verificare questo limite, anche se non sarà facile trovare l'algoritmo A'.

Esercizi

34.2-1

Definite il problema di ottimizzazione LONGEST-PATH-LENGTH come la relazione che associa ciascuna istanza di un grafo non orientato e due vertici al numero di archi in un cammino semplice più lungo tra i due vertici. Definite il problema di decisione LONGEST-PATH $= \{\langle G, u, v, k\rangle : G = (V, E)$ è un grafo non orientato, $u, v \in V, k \geq 0$ è un intero ed esiste un cammino semplice da u a v in G formato da almeno k archi$\}$. Dimostrate che il problema di ottimizzazione LONGEST-PATH-LENGTH può essere risolto in tempo polinomiale se e soltanto se LONGEST-PATH $\in$ P.

34.2-2

Date una definizione formale per il problema di trovare il ciclo semplice più lungo in un grafo non orientato. Definite il problema di decisione associato. Specificate il linguaggio corrispondente al problema di decisione.

34.2-3

Definite una codifica formale dei grafi orientati come stringhe binarie utilizzando una rappresentazione con matrice di adiacenza. Fate la stessa cosa utilizzando una rappresentazione con liste di adiacenza. Dimostrate che le due rappresentazioni sono polinomialmente correlate.

34.2-4

L'algoritmo di programmazione dinamica per il problema dello zaino 0-1 richiesto nell'Esercizio 16.2-2 è un algoritmo con tempo polinomiale? Spiegate la vostra risposta.

34.2-5

Dimostrate che se un algoritmo effettua al più un numero costante di chiamate a delle subroutine con tempo polinomiale e richiede un lavoro ulteriore che è pure polinomiale, allora il suo tempo di esecuzione totale è polinomiale. Dimostrate inoltre che un numero polinomiale di chiamate a subroutine con tempo polinomiale può generare un algoritmo con tempo esponenziale.

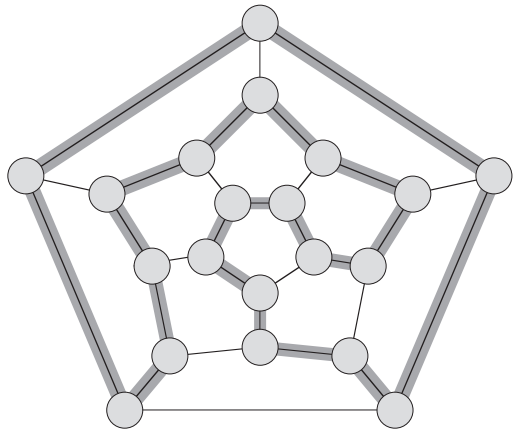
34.2-6

Dimostrate che la classe P, vista come un insieme di linguaggi, è chiusa rispetto all'unione, all'intersezione, alla concatenazione, al complemento e alla stella di Kleene. Ovvero, se $L_1, L_2 \in$ P, allora $L_1 \cup L_2 \in$ P, $L_1 \cap L_2 \in$ P, $L_1 L_2 \in$ P, $\overline{L}_1 \in$ P, and $L_1^* \in$ P.

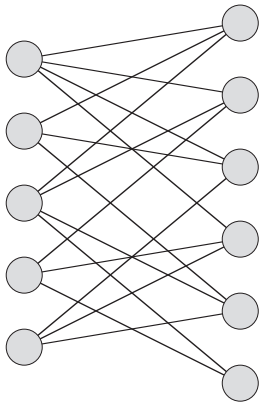
Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

34.3 Verifica in tempo polinomiale

Esaminiamo adesso gli algoritmi che "verificano" l'appartenenza ai linguaggi. Per esempio, supponiamo che per una data istanza $\langle G, u, v, k\rangle$ del problema di decisione PATH sia dato anche un cammino p da u a v. Possiamo facilmente verificare se la lunghezza di p è al più k e, in caso affermativo, possiamo considerare p come un "certificato" che l'istanza appartiene davvero a PATH. Per il problema



(a)



(b)

Figura 34.2 **(a)** Un grafo che rappresenta i vertici, gli archi e le facce di un dodecaedro, con un ciclo hamiltoniano indicato dagli archi ombreggiati. **(b)** Un grafo bipartito con un numero dispari di vertici. Un grafo di questo tipo è non hamiltoniano.

di decisione PATH, questo certificato non sembra molto conveniente. Dopo tutto, PATH appartiene a P – infatti, PATH può essere risolto in tempo lineare – e quindi la verifica dell'appartenenza tramite un dato certificato richiede lo stesso tempo della risoluzione del problema partendo da zero. Esamineremo adesso un problema per il quale non si conosce ancora alcun algoritmo di decisione con tempo polinomiale, ma la cui verifica è semplice, se è dato un certificato.

Cicli hamiltoniani

Il problema di trovare un ciclo hamiltoniano in un grafo non orientato è stato studiato per oltre un secolo. Formalmente, un ***ciclo hamiltoniano*** di un grafo non orientato $G = (V, E)$ è un ciclo semplice che contiene ogni vertice di V. Un grafo che contiene un ciclo hamiltoniano è detto ***hamiltoniano***, altrimenti è ***non hamiltoniano***. Il nome rende omaggio a W. R. Hamilton che ha descritto un gioco matematico su un dodecaedro (Figura 34.2(a)) in cui un giocatore inserisce cinque spilli in cinque vertici consecutivi e l'altro giocatore deve completare il cammino per formare un ciclo contenente tutti i vertici.[7] Il dodecaedro è hamiltoniano; la Figura 34.2(a) illustra un ciclo hamiltoniano. Tuttavia, non tutti i grafi sono hamiltoniani. Per esempio, la Figura 34.2(b) illustra un grafo bipartito con un numero dispari di vertici; l'Esercizio 34.3-2 chiede di dimostrare che tutti i grafi di questo tipo sono non hamiltoniani.

Possiamo definire il ***problema del ciclo hamiltoniano*** "Un grafo G ha un ciclo hamiltoniano?" come un linguaggio formale:

$$\text{HAM-CYCLE} = \{\langle G\rangle : G \text{ è un grafo hamiltoniano}\}$$

In che modo un algoritmo può decidere il linguaggio HAM-CYCLE? Data un'istanza del problema $\langle G\rangle$, un possibile algoritmo di decisione elenca tutte le permutazioni dei vertici di G e poi verifica ciascuna permutazione per vedere se è

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[7] In una lettera indirizzata il 17 ottobre 1856 all'amico John T. Graves, Hamilton [158, p. 624] scrisse: "Ho scoperto che alcuni giovani si divertono moltissimo con un nuovo gioco matematico, in cui un giocatore inserisce cinque spilli in cinque punti consecutivi ... e l'altro giocatore deve inserire quindici spilli in successione ciclica (dalla teoria descritta in questa lettera ciò è sempre possibile), in modo da coprire tutti gli altri punti, finendo nell'immediata vicinanza dello spillo dal quale ha iniziato il primo giocatore."

un cammino hamiltoniano. Qual è il tempo di esecuzione di questo algoritmo? Se utilizziamo la codifica "ragionevole" di un grafo come la sua matrice di adiacenza, il numero di vertici m nel grafo è $\Omega(\sqrt{n})$, dove $n = |\langle G \rangle|$ è la lunghezza della codifica di G. Ci sono $m!$ possibili permutazioni dei vertici, e quindi il tempo di esecuzione è $\Omega(m!) = \Omega(\sqrt{n}\,!) = \Omega(2^{\sqrt{n}})$, che non è $O(n^k)$ per qualsiasi costante k. Quindi, questo algoritmo ingenuo non viene eseguito in tempo polinomiale. In effetti, il problema del ciclo hamiltoniano è NP-completo, come dimostreremo nel Paragrafo 34.6.

Algoritmi di verifica

Consideriamo un problema un po' più semplice. Supponiamo che un amico ci dica che un dato grafo G è hamiltoniano e che, per dimostrarlo, elenchi ordinatamente i vertici di un ciclo hamiltoniano. Sarebbe davvero semplice verificare la sua dimostrazione: controlliamo semplicemente se il ciclo fornito è hamiltoniano, verificando se è una permutazione dei vertici di V e se ciascuno degli archi consecutivi lungo il ciclo appartiene al grafo. Questo algoritmo di verifica può essere implementato per essere eseguito nel tempo $O(n^2)$, dove n è la lunghezza della codifica di G. Quindi, una dimostrazione che un ciclo hamiltoniano esiste in un grafo può essere verificata in tempo polinomiale.

Definiamo ***algoritmo di verifica*** un algoritmo A con due argomenti, dove un argomento è una normale stringa di input x e l'altro argomento è una stringa binaria y detta ***certificato***. Diremo che un algoritmo A con due argomenti ***verifica*** una stringa di input x se esiste un certificato y tale che $A(x, y) = 1$. Il ***linguaggio verificato*** da un algoritmo di verifica A è

$$L = \{x \in \{0,1\}^* : \text{ esiste } y \in \{0,1\}^* \text{ tale che } A(x, y) = 1\}$$

Intuitivamente, un algoritmo A verifica un linguaggio L se, per ogni stringa $x \in L$, esiste un certificato y che A può utilizzare per dimostrare che $x \in L$. Inoltre, per ogni stringa $x \notin L$, non deve esistere un certificato che prova che $x \in L$. Per esempio, nel problema del ciclo hamiltoniano, il certificato è la lista dei vertici in qualche ciclo hamiltoniano. Se un grafo è hamiltoniano, il ciclo hamiltoniano stesso offre sufficienti informazioni per verificare questo fatto. Viceversa, se un grafo non è hamiltoniano, non esiste una lista di vertici che possa fare credere all'algoritmo di verifica che il grafo è hamiltoniano, perché l'algoritmo di verifica, per sicurezza, controlla attentamente il "ciclo".

La classe di complessità NP

La ***classe di complessità*** **NP** è la classe dei linguaggi che possono essere verificati da un algoritmo polinomiale.[8] Più precisamente, un linguaggio L appartiene a NP se e soltanto se esistono un algoritmo A con due input e con tempo polinomiale e una costante c tale che

$$L = \{x \in \{0,1\}^* : \text{ esiste un certificato } y \text{ con } |y| = O(|x|^c) \text{ tale che } A(x, y) = 1\}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine: Thesis 1985(2016) 2207010 Copyright © 2022, McGraw-Hill Education (Italy)

Diremo che l'algoritmo A ***verifica*** il linguaggio L ***in tempo polinomiale***.

[8] Il nome "NP" sta per "nondeterministic polynomial time" (tempo polinomiale non deterministico). Originariamente, la classe NP fu studiata nel contesto del non determinismo, ma questo libro usa una più semplice, ma equivalente, nozione di verifica. Hopcroft e Ullman [181] forniscono una buona rappresentazione della NP-completezza in termini di modelli di calcolo non deterministici.

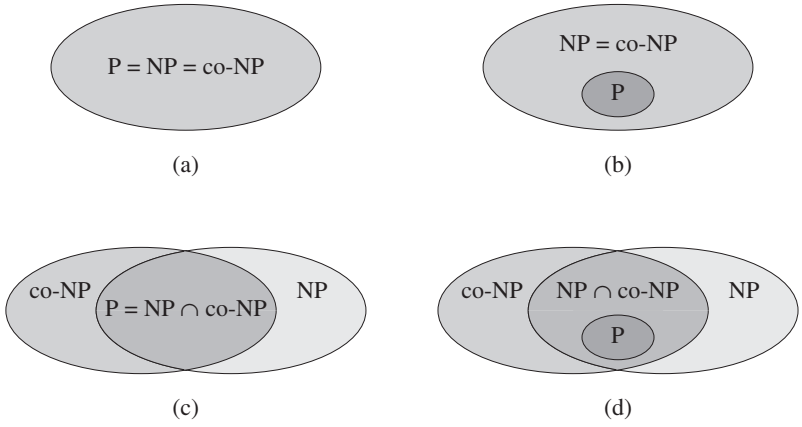


Figura 34.3 Le quattro possibili soluzioni per le relazioni tra le classi di complessità. In ogni diagramma, una regione che racchiude un'altra regione indica una relazione di sottoinsieme proprio. **(a)** $\mathrm{P} = \mathrm{NP} = \text{co-NP}$. Molti ricercatori considerano questa soluzione come la meno probabile. **(b)** Se NP è chiusa rispetto al complemento, allora $\mathrm{NP} = \text{co-NP}$, ma non necessariamente $\mathrm{P} = \mathrm{NP}$. **(c)** $\mathrm{P} = \mathrm{NP} \cap \text{co-NP}$, ma NP non è chiusa rispetto al complemento. **(d)** $\mathrm{NP} \neq \text{co-NP}$ e $\mathrm{P} \neq \mathrm{NP} \cap \text{co-NP}$. Molti ricercatori considerano questa soluzione come la più probabile.

Dalla precedente analisi del problema del ciclo hamiltoniano, segue che HAM-CYCLE $\in$ NP (è sempre piacevole sapere che un insieme importante non sia vuoto). Inoltre, se $L \in \mathrm{P}$, allora $L \in \mathrm{NP}$, perché se esiste un algoritmo con tempo polinomiale che decide L, l'algoritmo può essere facilmente convertito in un algoritmo di verifica con due argomenti che ignora semplicemente qualsiasi certificato e accetta esattamente quelle stringhe di input che determina appartenere a L. Quindi, $\mathrm{P} \subseteq \mathrm{NP}$.

Non sappiamo se $\mathrm{P} = \mathrm{NP}$, ma molti ricercatori credono che P ed NP non siano la stessa classe. Intuitivamente, la classe P è composta da problemi che possono essere rapidamente risolti. La classe NP è composta da problemi per i quali è possibile verificare rapidamente una soluzione. Avrete imparato dall'esperienza che spesso è più difficile risolvere un problema dall'inizio che verificare una soluzione presentata in modo chiaro, specialmente quando il tempo è limitato. Gli scienziati informatici, in generale, credono che questa analogia possa essere estesa alle classi P ed NP, e che quindi NP includa linguaggi che non sono in P.

C'è una prova convincente, ma non definitiva, che $\mathrm{P} \neq \mathrm{NP}$ – l'esistenza di linguaggi che sono "NP-completi". Studieremo questa classe nel Paragrafo 34.4.

Oltre alla questione $\mathrm{P} \neq \mathrm{NP}$ restano irrisolte molte altre questioni fondamentali. Malgrado il gran lavoro di molti ricercatori, nessuno sa se la classe NP sia chiusa rispetto al complemento. Ovvero, $L \in \mathrm{NP}$ implica $\overline{L} \in \mathrm{NP}$? Possiamo definire la ***classe di complessità*** **co-NP** come l'insieme dei linguaggi L tali che $\overline{L} \in \mathrm{NP}$. La questione se NP sia chiusa rispetto al complemento può essere riformulata così: $\mathrm{NP} = \text{co-NP}$? Poiché P è chiusa rispetto al complemento (Esercizio 34.2-6), segue che $\mathrm{P} \subseteq \mathrm{NP} \cap \text{co-NP}$. Ancora una volta, però, non sappiamo se $\mathrm{P} = \mathrm{NP} \cap \text{co-NP}$ o se esista qualche linguaggio in $\mathrm{NP} \cap \text{co-NP} - \mathrm{P}$. La Figura 34.3 illustra i quattro scenari possibili.

Quindi, la nostra conoscenza dell'esatta relazione tra P e NP purtroppo è incompleta. Ciò nonostante, anche se potremmo non essere in grado di dimostrare che un certo problema è intrattabile, se dimostriamo che esso è NP-completo abbiamo ugualmente ottenuto molte utili informazioni su di esso.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Esercizi

34.3-1

Considerate il linguaggio GRAPH-ISOMORPHISM $= \{\langle G_1, G_2\rangle : G_1 \text{ e } G_2 \text{ sono grafi isomorfi}\}$. Dimostrate che GRAPH-ISOMORPHISM $\in$ NP descrivendo un algoritmo con tempo polinomiale per verificare il linguaggio.

34.3-2
Dimostrate che se G è un grafo non orientato bipartito con un numero dispari di vertici, allora G è un grafo non hamiltoniano.

34.3-3
Dimostrate che, se HAM-CYCLE $\in$ P, allora il problema di elencare ordinatamente i vertici di un ciclo hamiltoniano è risolvibile in tempo polinomiale.

34.3-4
Dimostrate che la classe dei linguaggi NP è chiusa rispetto all'unione, all'intersezione, alla concatenazione e alla stella di Kleene. Discutete la chiusura di NP rispetto al complemento.

34.3-5
Dimostrate che qualsiasi linguaggio in NP può essere deciso da un algoritmo che viene eseguito nel tempo $2^{O(n^k)}$ per qualche costante k.

34.3-6
Un ***cammino hamiltoniano*** in un grafo è un cammino semplice che visita una sola volta tutti i vertici. Dimostrate che il linguaggio HAM-PATH $= \{\langle G, u, v\rangle$: esiste un cammino hamiltoniano da u a v nel grafo $G\}$ appartiene a NP.

34.3-7
Dimostrate che il problema del cammino hamiltoniano può essere risolto in tempo polinomiale nei grafi orientati aciclici. Create un algoritmo efficiente per questo problema.

34.3-8
Sia ϕ una formula booleana composta dalle variabili booleane di input $x_1, x_2, \ldots, x_k$, dalle negazioni ($\neg$), dalle funzioni AND ($\wedge$) e OR ($\vee$) e da parentesi. La formula ϕ è una ***tautologia*** se vale 1 per ogni assegnazione di 1 e 0 alle sue variabili di input. Definite TAUTOLOGY come il linguaggio di formule booleane che sono tautologie. Dimostrate che TAUTOLOGY $\in$ co-NP.

34.3-9
Dimostrate che P $\subseteq$ co-NP.

34.3-10
Dimostrate che, se NP $\neq$ co-NP, allora P $\neq$ NP.

34.3-11
Sia G un grafo connesso non orientato con almeno 3 vertici e sia G^3 il grafo ottenuto collegando tutte le coppie di vertici che sono collegate da un cammino in G di lunghezza al più 3. Dimostrate che G^3 è un grafo hamiltoniano (*suggerimento:* costruite un albero di connessione per G e utilizzate una dimostrazione induttiva).

34.4 NP-completezza e riducibilità

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 2995033018 220707 © Copyright © 2022, McGraw-Hill Education (Italy)

Probabilmente la ragione più convincente per cui gli scienziati informatici credono che P $\neq$ NP è l'esistenza della classe di problemi "NP-completi". Questa classe ha la sorprendente proprietà che, se un *qualsiasi* problema NP-completo può essere risolto in tempo polinomiale, allora *tutti* i problemi in NP hanno un algoritmo con tempo polinomiale, cioè P $=$ NP. Nonostante gli anni di studio, tuttavia, non è mai stato scoperto un algoritmo con tempo polinomiale per un qualsiasi problema NP-completo.

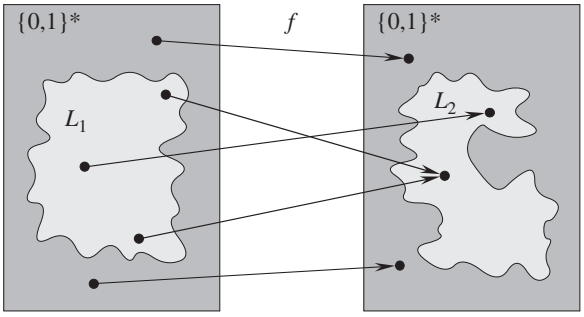


Figura 34.4 Illustrazione di una riduzione in tempo polinomiale da un linguaggio L_1 a un linguaggio L_2 tramite una funzione di riduzione f. Per qualsiasi input $x \in \{0,1\}^*$, la questione $x \in L_1$ ha la stessa soluzione della questione $f(x) \in L_2$.

Il linguaggio HAM-CYCLE è un problema NP-completo. Se decidessimo HAM-CYCLE in tempo polinomiale, allora potremmo risolvere tutti i problemi in NP in tempo polinomiale. In effetti, se NP − P dovesse risultare non vuoto, potremmo dire con certezza che HAM-CYCLE ∈ NP − P.

I linguaggi NP-completi sono, in un certo senso, i linguaggi "più difficili" in NP. In questo paragrafo, spiegheremo come confrontare la "difficoltà" relativa dei linguaggi utilizzando una nozione precisa detta "riducibilità in tempo polinomiale". Poi definiremo formalmente i linguaggi NP-completi e concluderemo abbozzando una dimostrazione che uno di tali linguaggi, detto CIRCUIT-SAT, è NP-completo. Nei Paragrafi 34.5 e 34.6 utilizzeremo la nozione di riducibilità per dimostrare che molti altri problemi sono NP-completi.

Riducibilità

Intuitivamente, un problema Q può essere ridotto a un altro problema Q' se un'istanza qualsiasi di Q può essere "facilmente riformulata" come un'istanza di Q', la cui soluzione fornisce una soluzione all'istanza di Q. Per esempio, il problema di risolvere le equazioni lineari in un'incognita x si riduce al problema di risolvere le equazioni quadratiche. Data un'istanza $ax + b = 0$, trasformiamola in $0x^2 + ax + b = 0$, la cui soluzione fornisce una soluzione per $ax + b = 0$. Quindi, se un problema Q si riduce a un altro problema Q', allora Q in un certo senso "non è più difficile da risolvere" di Q'.

Tornando alla struttura dei linguaggi formali per i problemi di decisione, diremo che un linguaggio L_1 è ***riducibile in tempo polinomiale*** a un linguaggio L_2, e scriveremo $L_1 \leq_P L_2$, se esiste una funzione calcolabile in tempo polinomiale $f : \{0,1\}^* \to \{0,1\}^*$ tale che per ogni $x \in \{0,1\}^*$

$$x \in L_1 \text{ se e soltanto se } f(x) \in L_2 \tag{34.1}$$

La funzione f è detta ***funzione di riduzione***; un algoritmo con tempo polinomiale F che calcola f è detto ***algoritmo di riduzione***.

La Figura 34.4 illustra il concetto di riduzione in tempo polinomiale da un linguaggio L_1 a un altro linguaggio L_2. Ciascun linguaggio è un sottoinsieme di $\{0,1\}^*$. La funzione di riduzione f fornisce una mappatura in tempo polinomiale tale che, se $x \in L_1$, allora $f(x) \in L_2$. Inoltre, se $x \notin L_1$, allora $f(x) \notin L_2$. Quindi, la funzione di riduzione mappa qualsiasi istanza x del problema di decisione rappresentato dal linguaggio L_1 in un'istanza $f(x)$ del problema rappresentato da L_2. Fornendo una soluzione alla questione $f(x) \in L_2$, si fornisce direttamente la soluzione alla questione $x \in L_1$.

Le riduzioni in tempo polinomiale sono un potente strumento per provare che i vari linguaggi appartengono a P.

Acquistato da Michele Mule su Vitalsource | ID 9788838677... | Licenza Ordinaria | 09/05/2022 | 2022 McGraw-Hill Education (Italy)

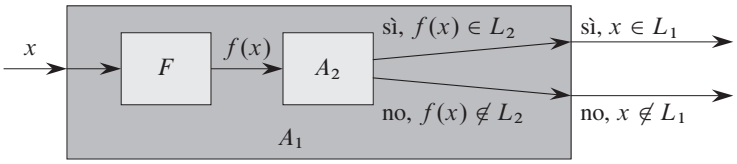


Figura 34.5
Dimostrazione del Lemma 34.3. L'algoritmo F è un algoritmo di riduzione che calcola la funzione di riduzione f da L_1 a L_2 in tempo polinomiale; A_2 è un algoritmo con tempo polinomiale che decide L_2. È illustrato un algoritmo A_1 che decide se $x \in L_1$, utilizzando prima F per trasformare un input qualsiasi x in $f(x)$ e poi A_2 per decidere se $f(x) \in L_2$.

Lemma 34.3
Se $L_1, L_2 \subseteq \{0,1\}^*$ sono linguaggi tali che $L_1 \leq_P L_2$, allora $L_2 \in$ P implica $L_1 \in$ P.

Dimostrazione Sia A_2 un algoritmo con tempo polinomiale che decide L_2 e sia F un algoritmo di riduzione con tempo polinomiale che calcola la funzione di riduzione f. Realizzeremo un algoritmo con tempo polinomiale A_1 che decide L_1.

La Figura 34.5 illustra la realizzazione di A_1. Per un dato input $x \in \{0,1\}^*$, l'algoritmo A_1 usa F per trasformare x in $f(x)$; poi usa A_2 per verificare se $f(x) \in L_2$. L'output di A_2 è il valore fornito come output di A_1.

La correttezza di A_1 segue dalla condizione (34.1). L'algoritmo viene eseguito in tempo polinomiale, perché entrambi gli algoritmi F e A_2 vengono eseguiti in tempo polinomiale (vedere l'Esercizio 34.2-5). ■

NP-completezza

Le riduzioni in tempo polinomiale sono uno strumento formale per dimostrare che un problema è difficile almeno quanto un altro, a meno di un fattore di tempo polinomiale. Ovvero, se $L_1 \leq_P L_2$, allora L_1 non è più difficile di L_2, a meno di un fattore polinomiale, e questo spiega la presenza del simbolo "minore o uguale" nella notazione della riduzione. Adesso possiamo definire l'insieme dei linguaggi NP-completi, che sono i problemi più difficili in NP.

Un linguaggio $L \subseteq \{0,1\}^*$ è ***NP-completo*** se

1. $L \in$ NP e
2. $L' \leq_P L$ per ogni $L' \in$ NP

Se un linguaggio L soddisfa la proprietà 2, ma non necessariamente la proprietà 1, diremo che L è ***NP-difficile***. Definiamo anche NPC come la classe dei linguaggi NP-completi.

Come dimostra il seguente teorema, la NP-completezza è cruciale per decidere se P è effettivamente uguale a NP.

Teorema 34.4
Se un qualsiasi problema NP-completo è risolvibile in tempo polinomiale, allora P = NP. In modo equivalente, se un qualsiasi problema in NP non è risolvibile in tempo polinomiale, allora nessun problema NP-completo è risolvibile in tempo polinomiale.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12; Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Dimostrazione Supponete che $L \in$ P e che $L \in$ NPC. Per qualsiasi $L' \in$ NP, si ha $L' \leq_P L$ per la proprietà 2 della definizione della NP-completezza. Quindi, per il Lemma 34.3, si ha anche $L' \in$ P, e questo dimostra la prima asserzione del teorema. Per dimostrare la seconda asserzione, notiamo che è la contrapposizione della prima asserzione. ■

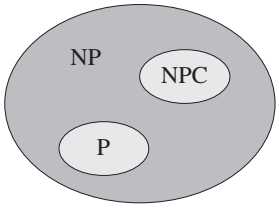


Figura 34.6 Come la maggior parte degli scienziati informatici vede le relazioni tra P, NP e NPC. Entrambe le classi P e NPC sono interamente contenute all'interno di NP; inoltre $P \cap NPC = \emptyset$.

È per questa ragione che la ricerca sulla questione $P \neq NP$ è incentrata sui problemi NP-completi. La maggior parte degli scienziati informatici ritiene che $P \neq NP$, e questo implica le relazioni tra P, NP e NPC illustrate nella Figura 34.6. Per quel che ne sappiamo, però, qualcuno potrebbe scoprire un algoritmo con tempo polinomiale per un problema NP-completo, dimostrando così che $P = NP$. Ciò nonostante, poiché non è stato ancora scoperto alcun algoritmo con tempo polinomiale per un qualsiasi problema NP-completo, la dimostrazione che un problema è NP-completo rappresenta una prova eccellente della sua intrattabilità.

Soddisfattibilità dei circuiti

Abbiamo definito il concetto di problema NP-completo, ma non abbiamo ancora provato l'esistenza di un qualsiasi problema NP-completo. Una volta dimostrato che almeno un problema è NP-completo, possiamo utilizzare la riducibilità in tempo polinomiale come uno strumento per provare la NP-completezza di altri problemi. Quindi, concentriamoci sulla dimostrazione dell'esistenza di un problema NP-completo: il problema della soddisfattibilità dei circuiti.

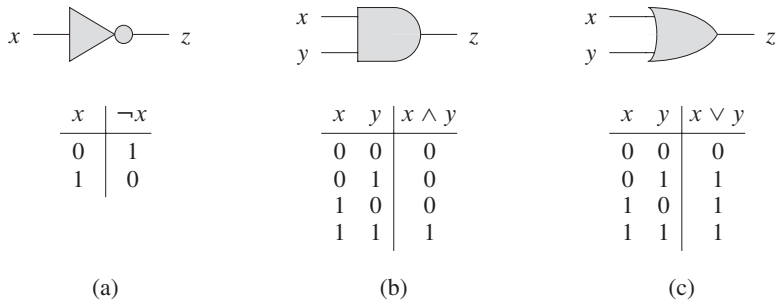
Purtroppo, la dimostrazione formale che il problema della soddisfattibilità dei circuiti è NP-completo richiede dettagli tecnici che esulano dagli obiettivi di questo libro. Invece, descriveremo in modo informale una dimostrazione che si basa sulla conoscenza dei circuiti combinatori booleani.

I circuiti combinatori booleani sono formati da elementi combinatori booleani che sono interconnessi tramite fili. Un ***elemento combinatorio booleano*** è un elemento qualsiasi del circuito che ha un numero costante di input e output booleani e che calcola una funzione ben definita. I valori booleani sono presi dall'insieme $\{0, 1\}$, dove 0 rappresenta FALSE e 1 rappresenta TRUE.

Gli elementi combinatori booleani che utilizziamo nel problema della soddisfattibilità dei circuiti calcolano una semplice funzione booleana e sono detti ***porte logiche***. La Figura 34.7 illustra le tre porte logiche elementari che saranno utilizzate nel problema della soddisfattibilità dei circuiti: la ***porta NOT*** (o ***invertitore***), la ***porta AND*** e la ***porta OR***. La porta NOT riceve un unico ***input*** binario x, il cui valore può essere 0 o 1, e produce un ***output*** binario z, il cui valore è l'opposto del valore di input. Ciascuna delle altre due porte riceve due input binari x e y e produce un unico output binario z.

Il funzionamento di ogni porta e di qualsiasi elemento combinatorio booleano può essere descritto con una ***tavola di verità***, illustrata sotto ogni porta nella Figura 34.7. Una tavola di verità fornisce gli output dell'elemento combinatorio per ogni possibile combinazione degli input. Per esempio, la tavola di verità per la porta OR indica che, quando gli input sono $x = 0$ e $y = 1$, il valore dell'output è $z = 1$. Utilizziamo i seguenti simboli: $\neg$ per indicare la funzione NOT, $\wedge$ per indicare la funzione AND e $\vee$ per indicare la funzione OR. Quindi, per esempio, $0 \vee 1 = 1$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)



x	$\neg x$
0	1
1	0

(a)

x	y	$x \wedge y$
0	0	0
0	1	0
1	0	0
1	1	1

(b)

x	y	$x \vee y$
0	0	0
0	1	1
1	0	1
1	1	1

(c)

Figura 34.7 Tre porte logiche elementari, con input e output binari. Sotto ogni porta è riportata la tavola di verità che descrive il funzionamento della porta. **(a)** La porta NOT. **(b)** La porta AND. **(c)** La porta OR.

Possiamo generalizzare le porte AND e OR per elaborare più di due input. L'output di una porta AND è 1 se tutti gli input sono 1, altrimenti è 0. L'output di una porta OR è 1 se uno degli input è 1, altrimenti è 0.

Un ***circuito combinatorio booleano*** è formato da uno o più elementi combinatori booleani interconnessi tramite ***fili***. Un filo può connettere l'output di un elemento all'input di un altro elemento; in questo modo, il valore di output del primo elemento diventa il valore di input del secondo elemento. La Figura 34.8 illustra due circuiti combinatori booleani simili; differiscono soltanto per una porta. La parte (a) della figura illustra anche i valori nei singoli fili, dato l'input $\langle x_1 = 1, x_2 = 1, x_3 = 0\rangle$. Un singolo filo non può essere collegato all'output di più elementi combinatori, ma può alimentare l'input di più elementi combinatori. Il numero di input alimentati da un filo è detto ***grado di diramazione*** o ***fan-out*** del filo. Un filo che non è collegato a nessun output è detto ***input del circuito***; riceve i valori di input da una sorgente esterna. Un filo che non è collegato a nessun input è detto ***output del circuito***; fornisce i risultati calcolati dal circuito al mondo esterno. (Un filo interno può anche essere collegato all'output di un circuito.) Ai fini della definizione del problema della soddisfattibilità dei circuiti, limitiamo il numero di output di un circuito a 1, anche se nei progetti di unità hardware reali un circuito combinatorio booleano può avere più output.

I circuiti combinatori booleani non contengono cicli. In altre parole, supponiamo di creare un grafo orientato $G = (V, E)$ con un vertice per ogni elemento combinatorio e con k archi orientati per ogni filo il cui grado di diramazione è k; esiste un arco orientato (u, v) se un filo collega l'output dell'elemento u a un input dell'elemento v. Allora il grafo G deve essere aciclico.

Un'***assegnazione di verità*** per un circuito combinatorio booleano è un insieme di valori booleani di input. Diremo che un circuito combinatorio booleano con un solo output è ***soddisfattibile*** se ha un'***assegnazione*** che lo soddisfa: un'assegnazione di verità per la quale l'output del circuito assume il valore 1. Per esempio, il circuito nella Figura 34.8(a) ha l'assegnazione $\langle x_1 = 1, x_2 = 1, x_3 = 0\rangle$ che lo soddisfa; quindi è soddisfattibile. Come l'Esercizio 34.4-1 chiede di dimostrare, nessuna assegnazione di valori a x_1, x_2 e x_3 fa sì che il circuito nella Figura 34.8(b) produca un output pari a 1; poiché l'output è sempre 0, il circuito non è soddisfattibile.

Acquistato da Michele Michele su Webster il 2022-07-01 2022 The MIT Press, Cambridge, Massachusetts 02142 Copyright © 2022. The MIT Press. All rights reserved. (Italy)

Il ***problema della soddisfattibilità dei circuiti*** è: "Un circuito combinatorio booleano composto dalle porte AND, OR e NOT è soddisfattibile?" Tuttavia, per porre questa domanda in modo formale, dobbiamo adottare una codifica standard per i circuiti. La ***dimensione*** di un circuito combinatorio booleano è il numero di elementi combinatori booleani più il numero di fili nel circuito. Potremmo realizzare una codifica basata sui grafi che mappa un dato circuito C in una stringa binaria $\langle C\rangle$ la cui lunghezza è polinomiale nella dimensione del circuito stesso.

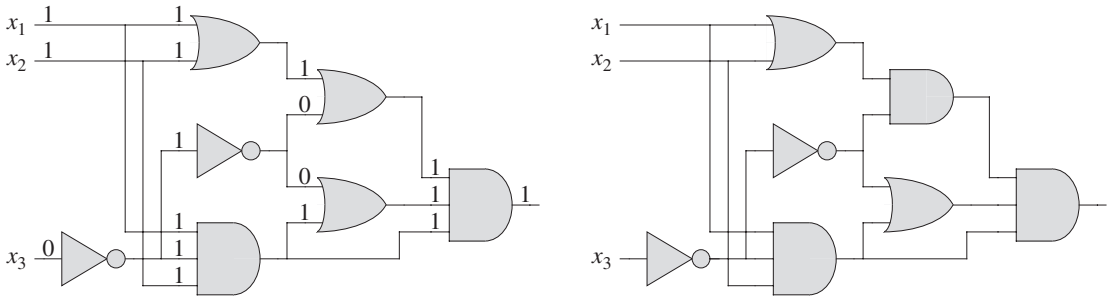


Figura 34.8 Due istanze del problema della soddisfattibilità dei circuiti. **(a)** L'assegnazione $\langle x_1 = 1, x_2 = 1, x_3 = 0\rangle$ agli input di questo circuito fa sì che l'output del circuito sia 1. Il circuito è quindi soddisfattibile. **(b)** Nessuna assegnazione agli input di questo circuito può fare assumere il valore 1 all'output del circuito. Il circuito quindi non è soddisfattibile.

In termini di linguaggio formale, possiamo quindi definire

CIRCUIT-SAT =
$\{\langle C\rangle : C$ è un circuito combinatorio booleano soddisfattibile$\}$

Il problema della soddisfattibilità dei circuiti si presenta nell'area dell'ottimizzazione dell'hardware assistita da calcolatore. Se un sottocircuito produce sempre 0, questo sottocircuito può essere sostituito con un sottocircuito più semplice che omette tutte le porte logiche e fornisce il valore costante 0 come output. Potrebbe essere utile avere un algoritmo con tempo polinomiale per questo problema.

Dato un circuito C, potremmo tentare di determinare se è soddisfattibile controllando semplicemente tutte le possibili assegnazioni agli input. Purtroppo, se ci sono k input, ci sono 2^k possibili assegnazioni. Quando la dimensione di C è polinomiale in k, il controllo di tutte le assegnazioni richiede un tempo $\Omega(2^k)$, che è superpolinomiale nella dimensione del circuito.[9] In effetti, come è stato detto, ci sono prove evidenti che non esiste alcun algoritmo con tempo polinomiale che risolve il problema della soddisfattibilità dei circuiti, perché la soddisfattibilità dei circuiti è un problema NP-completo. Dividiamo la dimostrazione di questo fatto in due parti, basate sulle due parti della definizione della NP-completezza.

Lemma 34.5
Il problema della soddisfattibilità dei circuiti appartiene alla classe NP.

Dimostrazione Forniremo un algoritmo A con due input e con tempo polinomiale che può verificare CIRCUIT-SAT. Uno degli input di A è (una codifica standard di) un circuito combinatorio booleano C. L'altro input è un certificato che corrisponde a un'assegnazione di valori booleani ai fili di C (consultate l'Esercizio 34.4-4 per un certificato più piccolo).

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[9] D'altra parte, se la dimensione del circuito C è $\Theta(2^k)$, allora un algoritmo il cui tempo di esecuzione è $O(2^k)$ ha un tempo di esecuzione che è polinomiale nella dimensione del circuito. Anche se P $\neq$ NP, questa situazione non sarebbe in contraddizione con il fatto che il problema è NP-completo; l'esistenza di un algoritmo con tempo polinomiale per un caso speciale non implica che ci sia un algoritmo con tempo polinomiale per tutti i casi.

L'algoritmo A è realizzato nel modo seguente. Per ogni porta logica del circuito, l'algoritmo controlla che il valore fornito dal certificato sul filo di output sia calcolato correttamente come una funzione dei valori sui fili di input. Poi, se l'output dell'intero circuito è 1, l'algoritmo restituisce 1 come output, perché i valori assegnati agli input di C sono un'assegnazione che soddisfa il circuito C, altrimenti restituisce 0.

Quando l'algoritmo A riceve come input un circuito soddisfattibile C, esiste un certificato, la cui lunghezza è polinomiale nella dimensione di C, che fa sì che A restituisca 1 come output. Quando l'algoritmo A riceve come input un circuito non soddisfattibile, nessun certificato può far credere ad A che il circuito sia soddisfattibile. L'algoritmo A viene eseguito in tempo polinomiale: con una buona implementazione, può bastare un tempo lineare. Quindi, CIRCUIT-SAT può essere verificato in tempo polinomiale e CIRCUIT-SAT $\in$ NP. ■

La seconda parte della dimostrazione che CIRCUIT-SAT è NP-completo consiste nel dimostrare che il linguaggio è NP-difficile. Ovvero dobbiamo dimostrare che ogni linguaggio in NP è riducibile in tempo polinomiale a CIRCUIT-SAT. La dimostrazione effettiva di questo fatto è piena di complicati dettagli tecnici, quindi ci accontenteremo di abbozzare la dimostrazione basandoci su alcune conoscenze del funzionamento dell'hardware di un calcolatore.

Un programma per calcolatore viene registrato nella memoria del calcolatore come una sequenza di istruzioni. Una tipica istruzione codifica un'operazione da eseguire, gli indirizzi degli operandi nella memoria e un indirizzo in cui deve essere memorizzato il risultato. Una speciale locazione della memoria, detta ***contatore del programma***, tiene traccia della successiva istruzione da eseguire. Il contatore del programma viene automaticamente incrementato ogni volta che viene letta un'istruzione; in questo modo le istruzioni vengono eseguite in sequenza dal calcolatore. Tuttavia, un'istruzione potrebbe scrivere un valore nel contatore del programma e, quindi, la normale esecuzione sequenziale delle istruzioni potrebbe essere modificata, consentendo al calcolatore di eseguire cicli e salti condizionati.

In un punto qualsiasi, durante l'esecuzione di un programma, l'intero stato dei calcoli è rappresentato nella memoria del calcolatore. (La memoria viene riservata per contenere il programma stesso, il contatore del programma, l'area di lavoro e alcuni bit di stato che il calcolatore usa per la sua contabilità.) Chiameremo ***configurazione*** un particolare stato della memoria del calcolatore. L'esecuzione di un'istruzione può essere vista come la mappatura di una configurazione in un'altra configurazione. L'hardware del calcolatore che realizza questa mappatura può essere implementato come un circuito combinatorio booleano, che indicheremo con M nella dimostrazione del seguente lemma.

Lemma 34.6
Il problema della soddisfattibilità dei circuiti è NP-difficile.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 195303822201070 Copyright © 2022, McGraw-Hill Education (Italy)

Dimostrazione Sia L un linguaggio qualsiasi in NP. Descriveremo un algoritmo F con tempo polinomiale che calcola una funzione di riduzione f che mappa ogni stringa binaria x in un circuito $C = f(x)$ tale che $x \in L$, se e soltanto se $C \in$ CIRCUIT-SAT.

Poiché $L \in$ NP, deve esistere un algoritmo A che verifica L in tempo polinomiale. L'algoritmo F utilizza l'algoritmo A con due input per calcolare la funzione di riduzione f.

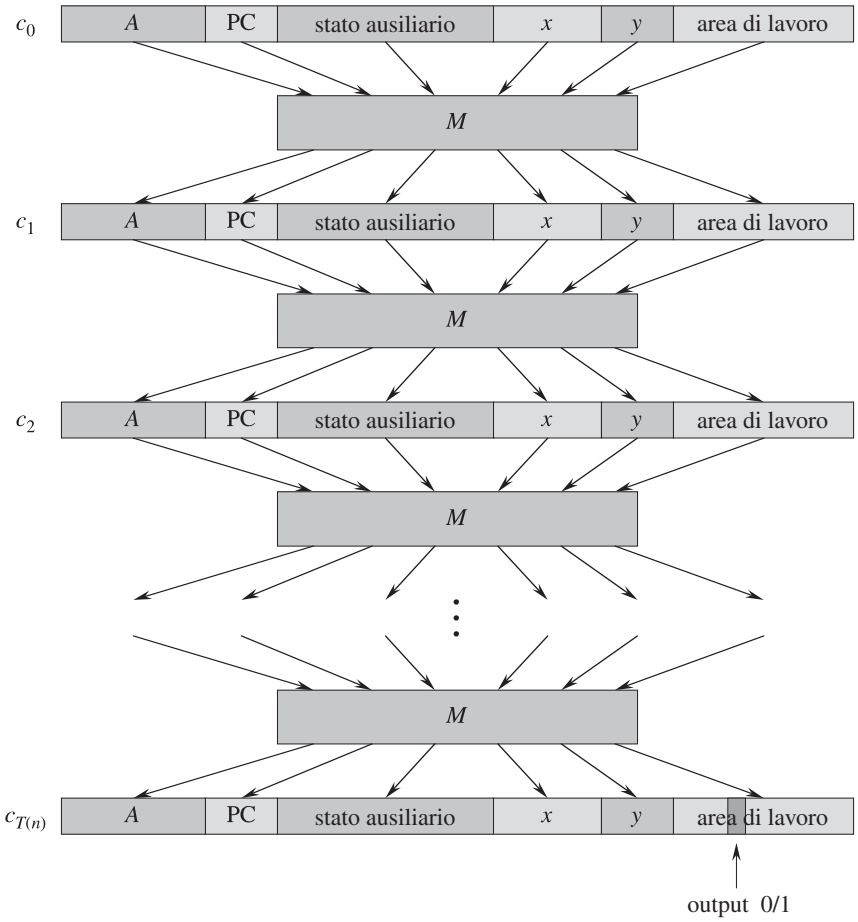


Figura 34.9 La sequenza delle configurazioni prodotte da un algoritmo A che viene eseguito con un input x e un certificato y. Ogni configurazione rappresenta lo stato del calcolatore per un passo di calcolo e, oltre ad A, x e y, include il contatore del programma (PC), lo stato ausiliario della macchina e l'area di lavoro. Ad eccezione del certificato y, la configurazione iniziale c_0 è costante. Ogni configurazione viene mappata nella successiva configurazione da un circuito combinatorio booleano M. L'output è un bit distinto nell'area di lavoro.

Sia $T(n)$ il tempo di esecuzione nel caso peggiore dell'algoritmo A con stringhe di input di lunghezza n; sia $k \geq 1$ una costante tale che $T(n) = O(n^k)$ e la lunghezza del certificato sia $O(n^k)$. (Il tempo di esecuzione di A è effettivamente polinomiale nella dimensione totale dell'input, che include sia una stringa di input che un certificato, ma poiché la lunghezza del certificato è polinomiale nella lunghezza n della stringa di input, il tempo di esecuzione è polinomiale in n.)

L'idea di base della dimostrazione è rappresentare il calcolo di A come una sequenza di configurazioni. Come illustra la Figura 34.9, ogni configurazione può essere suddivisa in parti che includono il programma per A, il contatore del programma, lo stato ausiliario della macchina, l'input x, il certificato y e l'area di lavoro. Partendo da una configurazione iniziale c_0, ogni configurazione c_i viene mappata nella successiva configurazione c_{i+1} dal circuito combinatorio M che implementa l'hardware del calcolatore. L'output dell'algoritmo A (0 o 1) viene scritto in qualche locazione specifica dell'area di lavoro quando termina l'esecuzione di A e, se supponiamo che da quel momento in poi l'algoritmo A resterà fermo, tale valore non cambierà mai. Quindi, se l'algoritmo viene eseguito al più per $T(n)$ passi, l'output si presenta come uno dei bit in $c_{T(n)}$.

Acquistato da Michele Malvolo su Webster | 2022-07-07 23:18 | Numero Ordine Libraria: 1995010162207070 | Copyright © 2022, McGraw-Hill Education (Italy)

L'algoritmo di riduzione F realizza un singolo circuito combinatorio che calcola tutte le configurazioni prodotte da una data configurazione iniziale. L'idea è quella di mettere insieme $T(n)$ copie del circuito M. L'output dell'i-esimo circuito, che produce la configurazione c_i, alimenta direttamente l'input dell'$(i + 1)$-esimo circuito. Quindi, le configurazioni, anziché essere registrate nella memoria

del computer, si trovano semplicemente come valori nei fili che collegano le copie di M.

Ricordiamo che cosa deve fare l'algoritmo di riduzione con tempo polinomiale F. Dato un input x, F deve calcolare un circuito $C = f(x)$ che è soddisfattibile se e soltanto se esiste un certificato y tale che $A(x, y) = 1$. Quando F riceve un input x, prima calcola $n = |x|$ e realizza un circuito combinatorio C' formato da $T(n)$ copie di M. L'input di C' è una configurazione iniziale che corrisponde a un calcolo su $A(x, y)$; l'output è la configurazione $c_{T(n)}$.

Il circuito $C = f(x)$ calcolato da F è ottenuto modificando leggermente C'. In primo luogo, gli input di C' che corrispondono al programma per A, al contatore iniziale del programma, all'input x e allo stato iniziale della memoria vengono collegati direttamente a questi valori noti. Quindi, i soli input che restano per il circuito corrispondono al certificato y. In secondo luogo, tutti gli output del circuito sono ignorati, tranne il bit di $c_{T(n)}$ che corrisponde all'output di A. Questo circuito C, così realizzato, calcola $C(y) = A(x, y)$ per ogni input y di lunghezza $O(n^k)$. L'algoritmo di riduzione F, quando riceve una stringa di input x, calcola un tale circuito C e lo restituisce in output.

Restano da dimostrare due proprietà. In primo luogo, dobbiamo dimostrare che F calcola correttamente una funzione di riduzione f. Ovvero dobbiamo dimostrare che C è soddisfattibile se e soltanto se esiste un certificato y tale che $A(x, y) = 1$. In secondo luogo, dobbiamo dimostrare che l'algoritmo F viene eseguito in tempo polinomiale.

Per dimostrare che F calcola correttamente una funzione di riduzione, supponiamo che esista un certificato y di lunghezza $O(n^k)$ tale che $A(x, y) = 1$. Allora, se applichiamo i bit di y agli input di C, l'output di C è $C(y) = A(x, y) = 1$. Quindi, se esiste un certificato, allora C è soddisfattibile. Per dimostrare il viceversa, supponiamo che C sia soddisfattibile. Allora esiste un input y di C tale che $C(y) = 1$, per cui concludiamo che $A(x, y) = 1$. Quindi, F calcola correttamente una funzione di riduzione.

Per completare la dimostrazione, dobbiamo soltanto dimostrare che F viene eseguito in tempo polinomiale in $n = |x|$. La prima osservazione è che il numero di bit richiesti per rappresentare una configurazione è polinomiale in n. Il programma per A ha dimensione costante, indipendentemente dalla lunghezza del suo input x. La lunghezza dell'input x è n e la lunghezza del certificato y è $O(n^k)$. Poiché l'algoritmo viene eseguito al più per $O(n^k)$ passi, anche la quantità di area di lavoro richiesta da A è polinomiale in n. (Si suppone che quest'area di memoria sia contigua; l'Esercizio 34.4-5 chiede di estendere l'analisi al caso in cui le locazioni cui accede A siano distribuite su una regione di memoria molto più vasta e il particolare schema di distribuzione possa variare per ogni input x.)

Il circuito combinatorio M che implementa l'hardware del calcolatore ha dimensione polinomiale nella lunghezza di una configurazione, che è polinomiale in $O(n^k)$ e quindi in n (la maggior parte di questo circuito implementa la logica del sistema di memoria). Il circuito C è formato al più da $t = O(n^k)$ copie di M; quindi ha una dimensione polinomiale in n. L'algoritmo di riduzione F può realizzare C da x in tempo polinomiale, perché ogni passo della realizzazione richiede un tempo polinomiale. ■

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Dunque, il linguaggio CIRCUIT-SAT è difficile almeno quanto un linguaggio qualsiasi in NP e, poiché appartiene a NP, è NP-completo.

Teorema 34.7
Il problema della soddisfattibilità dei circuiti è NP-completo.

Dimostrazione È una conseguenza immediata dei Lemmi 34.5 e 34.6 e della definizione della NP-completezza. ■

Esercizi

34.4-1
Verificate che il circuito nella Figura 34.8(b) non è soddisfattibile.

34.4-2
Dimostrate che la relazione $\leq_P$ è una relazione transitiva per i linguaggi. Ovvero dimostrate che, se $L_1 \leq_P L_2$ e $L_2 \leq_P L_3$, allora $L_1 \leq_P L_3$.

34.4-3
Dimostrate che $L \leq_P \overline{L}$ se e soltanto se $\overline{L} \leq_P L$.

34.4-4
Dimostrate che come certificato in una dimostrazione alternativa del Lemma 34.5 si può utilizzare un'assegnazione che soddisfa il circuito. Quale certificato rende più semplice la dimostrazione?

34.4-5
La dimostrazione del Lemma 34.6 assume che l'area di lavoro per l'algoritmo A occupi una regione contigua di dimensione polinomiale. In quale punto della dimostrazione viene utilizzata questa ipotesi? Dimostrate che questa ipotesi non riduce la generalità del Lemma.

34.4-6
Un linguaggio L è ***completo*** per una classe di linguaggi C rispetto alle riduzioni in tempo polinomiale se $L \in C$ e $L' \leq_P L$ per ogni $L' \in C$. Dimostrate che $\emptyset$ e $\{0,1\}^*$ sono gli unici linguaggi in P che non sono completi per P rispetto alle riduzioni in tempo polinomiale.

34.4-7
Dimostrate che L è completo per NP se e soltanto se $\overline{L}$ è completo per co-NP.

34.4-8
L'algoritmo di riduzione F nella dimostrazione del Lemma 34.6 realizza il circuito $C = f(x)$ basandosi sulla conoscenza di x, A e k. Il professor Sartre osserva che la stringa x è un input di F, ma questo algoritmo conosce soltanto l'esistenza di A, k e del fattore costante implicito nel tempo di esecuzione $O(n^k)$ (perché il linguaggio L appartiene a NP), non i loro valori effettivi. Quindi, il professore conclude che F non è in grado di realizzare il circuito C e che il linguaggio CIRCUIT-SAT non è necessariamente NP-difficile. Spiegate l'errore nel ragionamento del professore.

34.5 Dimostrazioni della NP-completezza

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

La NP-completezza del problema della soddisfattibilità dei circuiti si basa su una dimostrazione diretta che $L \leq_P$ CIRCUIT-SAT per ogni linguaggio $L \in$ NP. In questo paragrafo spiegheremo come dimostrare che dei linguaggi sono NP-completi senza ridurre direttamente *ogni* linguaggio in NP a tali linguaggi. Descriveremo questa metodologia dimostrando che vari problemi di soddisfattibilità delle formule sono NP-completi. Il Paragrafo 34.6 presenta molti altri esempi di questa metodologia.

Il seguente lemma è la base del nostro metodo per dimostrare che un linguaggio è NP-completo.

Lemma 34.8
Se L è un linguaggio tale che $L' \leq_P L$ per qualche $L' \in$ NPC, allora L è NP-difficile. Inoltre, se $L \in$ NP, allora $L \in$ NPC.

Dimostrazione Poiché L' è NP-completo, per ogni $L'' \in$ NP, si ha $L'' \leq_P L'$. Per ipotesi $L' \leq_P L$; quindi per la transitività (Esercizio 34.4-2), si ha $L'' \leq_P L$, che dimostra che L è NP-difficile. Se $L \in$ NP, si ha anche $L \in$ NPC. ■

In altre parole, riducendo a L un linguaggio L' che sappiamo essere NP-completo, implicitamente riduciamo ogni linguaggio in NP a L. Quindi, il Lemma 34.8 ci offre un metodo per dimostrare che un linguaggio L è NP-completo:

1. Dimostrare che $L \in$ NP.
2. Selezionare un noto linguaggio NP-completo L'.
3. Descrivere un algoritmo che calcola una funzione f che mappa ogni istanza $x \in \{0,1\}^*$ di L' in un'istanza $f(x)$ di L.
4. Dimostrare che la funzione f soddisfa $x \in L'$ se e soltanto se $f(x) \in L$ per ogni $x \in \{0,1\}^*$.
5. Dimostrare che l'algoritmo che calcola f viene eseguito in tempo polinomiale.

(I punti 2–5 dimostrano che L è NP-difficile.) Questa metodologia di riduzione da un noto linguaggio NP-completo è di gran lunga più semplice del procedimento che dimostra direttamente come effettuare le riduzioni da ogni linguaggio in NP. Provare che CIRCUIT-SAT $\in$ NPC ci ha permesso di "aprirci la strada". Poiché adesso sappiamo che il problema della soddisfattibilità dei circuiti è NP-completo, possiamo dimostrare molto più facilmente che altri problemi sono NP-completi. Inoltre, arricchendo il nostro catalogo di nuovi problemi NP-completi, avremo più possibilità di scelta per il linguaggio dal quale effettuare la riduzione.

Soddisfattibilità delle formule

Illustriamo la metodologia della riduzione fornendo una dimostrazione della NP-completezza per il problema di determinare se una formula booleana, non un circuito, è soddisfattibile. Questo problema ha l'onore storico di essere il primo problema a essere stato dimostrato NP-completo.

Formalizziamo il problema della ***soddisfattibilità (delle formule)*** in funzione del linguaggio SAT nel modo seguente. Un'istanza di SAT è una formula booleana ϕ formata dai seguenti elementi:

1. n variabili booleane: $x_1, x_2, \ldots, x_n$.
2. m connettivi booleani: una funzione booleana qualsiasi con uno o due input e un output, come $\wedge$ (AND), $\vee$ (OR), $\neg$ (NOT), $\rightarrow$ (implicazione) e $\leftrightarrow$ (se e soltanto se).
3. Parentesi (senza perdere in generalità, supponiamo che non ci siano parentesi ridondanti, cioè ci sia al più una coppia di parentesi per ogni connettivo booleano).

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 1995020016-220707 0 Copyright © 2022. McGraw-Hill Education (Italy)

È semplice codificare una formula booleana ϕ in una lunghezza che è polinomiale in $n + m$. Come nei circuiti combinatori booleani, un'***assegnazione di verità*** per una formula booleana ϕ è un insieme di valori per le variabili di ϕ e un'***assegnazione*** che la soddisfa è un'assegnazione di verità per la quale ϕ vale 1. Una formula con un'assegnazione che la soddisfa è una ***formula soddisfattibile***. Il problema della soddisfattibilità chiede se una data formula booleana è soddisfattibile; in termini di linguaggio formale, si ha

$$\text{SAT} = \{\langle\phi\rangle : \phi \text{ è una formula booleana soddisfattibile}\}$$

Per esempio, la formula

$$\phi = ((x_1 \to x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$$

ha l'assegnazione $\langle x_1 = 0, x_2 = 0, x_3 = 1, x_4 = 1\rangle$ che la soddisfa, in quanto

$$\begin{aligned} \phi &= ((0 \to 0) \vee \neg((\neg 0 \leftrightarrow 1) \vee 1)) \wedge \neg 0 \\ &= (1 \vee \neg(1 \vee 1)) \wedge 1 \\ &= (1 \vee 0) \wedge 1 \\ &= 1 \end{aligned} \tag{34.2}$$

Quindi, questa formula ϕ appartiene a SAT.

L'algoritmo ingenuo che determina se una formula booleana arbitraria è soddisfattibile non viene eseguito in tempo polinomiale. Ci sono 2^n possibili assegnazioni in una formula ϕ con n variabili. Se la lunghezza di $\langle\phi\rangle$ è polinomiale in n, allora controllare ogni assegnazione richiede un tempo $\Omega(2^n)$, che è superpolinomiale nella lunghezza di $\langle\phi\rangle$. Come dimostra il seguente teorema, è poco probabile che esista un algoritmo con tempo polinomiale.

Teorema 34.9
La soddisfattibilità delle formule booleane è un problema NP-completo.

Dimostrazione Iniziamo dimostrando che SAT $\in$ NP. Poi proveremo che SAT è NP-difficile dimostrando che CIRCUIT-SAT $\leq_P$ SAT; per il Lemma 34.8, questo proverà il teorema.

Per dimostrare che SAT appartiene a NP, dimostriamo che un certificato formato da un'assegnazione che soddisfa una formula di input ϕ può essere verificato in tempo polinomiale. L'algoritmo di verifica sostituisce semplicemente ciascuna variabile nella formula con il suo valore corrispondente e poi calcola l'espressione, come abbiamo fatto nella precedente equazione (34.2). Questo compito è svolto facilmente in tempo polinomiale. Se l'espressione risulta pari a 1, la formula è soddisfattibile. Quindi, la prima condizione del Lemma 34.8 per la NP-completezza è soddisfatta.

Per provare che SAT è NP-difficile, dimostriamo che CIRCUIT-SAT $\leq_P$ SAT. In altre parole, qualsiasi istanza della soddisfattibilità dei circuiti può essere ridotta in tempo polinomiale a un'istanza di soddisfattibilità delle formule. È possibile utilizzare l'induzione per esprimere qualsiasi circuito combinatorio booleano come una formula booleana. Consideriamo semplicemente la porta che produce l'output del circuito ed esprimiamo induttivamente ciascuno degli input della porta come una formula. La formula per il circuito viene ottenuta scrivendo un'espressione che applica la funzione della porta alle formule dei suoi input.

Acquistato da Michele Mi... Vietata la riproduzione non autorizzata ... McGraw-Hill Education (Italy)

Purtroppo, questo semplice metodo non effettua una riduzione in tempo polinomiale. Come l'Esercizio 34.5-1 chiede di provare, le sottoformule condivise

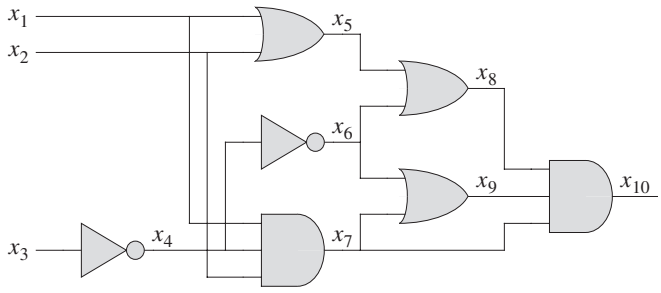


Figura 34.10 Riduzione della soddisfattibilità dei circuiti alla soddisfattibilità delle formule. La formula prodotta dall'algoritmo di riduzione ha una variabile per ogni filo del circuito.

(che si hanno nelle porte i cui fili di output hanno gradi di diramazione maggiori o uguali a 2) possono provocare una crescita esponenziale della dimensione della formula generata. Quindi, l'algoritmo di riduzione deve essere molto più intelligente.

La Figura 34.10 illustra l'idea di base della riduzione da CIRCUIT-SAT a SAT per il circuito della Figura 34.8(a). Per ogni filo x_i nel circuito C, la formula ϕ ha una variabile x_i. L'operazione di una porta adesso può essere espressa come una formula con le variabili dei suoi fili incidenti. Per esempio, l'operazione della porta AND di output è $x_{10} \leftrightarrow (x_7 \wedge x_8 \wedge x_9)$.

La formula ϕ prodotta dall'algoritmo di riduzione è l'operazione AND della variabile dell'output del circuito con la congiunzione delle clausole che descrivono le operazioni di ogni porta. Per il circuito illustrato nella figura, la formula è

$$\begin{aligned}\phi \;=\; & x_{10} \wedge (x_4 \leftrightarrow \neg x_3) \\ & \wedge (x_5 \leftrightarrow (x_1 \vee x_2)) \\ & \wedge (x_6 \leftrightarrow \neg x_4) \\ & \wedge (x_7 \leftrightarrow (x_1 \wedge x_2 \wedge x_4)) \\ & \wedge (x_8 \leftrightarrow (x_5 \vee x_6)) \\ & \wedge (x_9 \leftrightarrow (x_6 \vee x_7)) \\ & \wedge (x_{10} \leftrightarrow (x_7 \wedge x_8 \wedge x_9))\end{aligned}$$

Dato un circuito C, è semplice produrre una tale formula ϕ in tempo polinomiale.

Perché il circuito C è soddisfattibile esattamente quando lo è la formula ϕ? Se C ha un'assegnazione di soddisfattibilità, ogni filo del circuito ha un valore ben definito e l'output del circuito è 1. Pertanto, l'assegnazione dei valori dei fili alle variabili di ϕ fa sì che ciascuna clausola di ϕ valga 1, e quindi la congiunzione di tutte le clausole vale 1. Viceversa, se esiste un'assegnazione che fa sì che ϕ valga 1, il circuito C è soddisfattibile per un ragionamento analogo. Quindi, abbiamo dimostrato che CIRCUIT-SAT $\leq_P$ SAT, e questo completa la dimostrazione. ■

Soddisfattibilità delle formule 3-CNF

È possibile dimostrare che molti problemi sono NP-completi per riduzione dalla soddisfattibilità delle formule. L'algoritmo di riduzione, però, deve gestire qualsiasi formula di input, e questo potrebbe richiedere di considerare un enorme numero di casi. Di conseguenza, spesso è preferibile effettuare la riduzione da un linguaggio ristretto di formule booleane, in modo da ridurre il numero di casi da considerare. Naturalmente, non dobbiamo restringere il linguaggio così tanto da renderlo risolvibile in tempo polinomiale. Un comodo linguaggio è la soddisfattibilità delle formule 3-CNF o 3-CNF-SAT.

Acquistato da Michele Michele su Webster il 2022-[illegible] Copyright 2002, The McGraw-Hill Companies (Italia)

Definiamo la soddisfattibilità delle formule 3-CNF utilizzando i seguenti termini. Un ***letterale*** in una formula booleana è una occorrenza di una variabile o della sua negazione. Una formula booleana è nella ***forma normale congiuntiva*** o ***CNF***, se è espressa come un AND di ***clausole***, ciascuna delle quali è l'OR di uno o più letterali. Una formula booleana è nella ***forma normale 3-congiuntiva*** o ***3-CNF***, se ciascuna clausola ha esattamente tre letterali distinti.

Per esempio, la formula booleana

$$(x_1 \vee \neg x_1 \vee \neg x_2) \wedge (x_3 \vee x_2 \vee x_4) \wedge (\neg x_1 \vee \neg x_3 \vee \neg x_4)$$

è nella forma 3-CNF. La prima delle sue tre clausole è $(x_1 \vee \neg x_1 \vee \neg x_2)$, che contiene i tre letterali x_1, $\neg x_1$ e $\neg x_2$.

Nel linguaggio 3-CNF-SAT, ci chiediamo se una data formula booleana ϕ in 3-CNF è soddisfattibile. Il seguente teorema dimostra che è poco probabile che esista un algoritmo con tempo polinomiale in grado di determinare la soddisfattibilità delle formule booleane, anche se sono espresse in questa semplice forma normale.

Teorema 34.10
La soddisfattibilità delle formule booleane nella forma 3-CNF è un problema NP-completo.

Dimostrazione Il ragionamento fatto nella dimostrazione del Teorema 34.9 per provare che SAT $\in$ NP può essere applicato qui per provare che 3-CNF-SAT $\in$ NP. Quindi, per il Lemma 34.8, dobbiamo dimostrare soltanto che SAT $\leq_P$ 3-CNF-SAT.

L'algoritmo di riduzione può essere suddiviso in tre fasi. Ogni fase trasforma progressivamente la formula di input ϕ avvicinandola sempre di più alla forma 3-CNF desiderata. La prima fase è simile a quella utilizzata per dimostrare che CIRCUIT-SAT $\leq_P$ SAT nel Teorema 34.9. Innanzi tutto, realizziamo un "albero binario di parsing" per la formula di input ϕ, con i letterali come foglie e i connettivi come nodi interni. La Figura 34.11 illustra l'albero di parsing per la formula

$$\phi = ((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2 \tag{34.3}$$

Se la formula di input contiene una clausola come l'OR di più letterali, l'associatività può essere utilizzata per parentesizzare completamente l'espressione in modo che ogni nodo interno nell'albero risultante abbia 1 o 2 figli. L'albero binario di parsing adesso può essere visto come un circuito per calcolare la funzione.

Effettuando una riduzione analoga a quella della dimostrazione del Teorema 34.9, introduciamo una variabile y_i per l'output di ogni nodo interno. Poi, riscriviamo la formula originale ϕ come l'AND tra la variabile della radice e una congiunzione di clausole che descrivono l'operazione di ciascun nodo. Per la formula (34.3), l'espressione risultante è

$$\begin{aligned}\phi' = {} & y_1 \\ & \wedge (y_1 \leftrightarrow (y_2 \wedge \neg x_2)) \\ & \wedge (y_2 \leftrightarrow (y_3 \vee y_4)) \\ & \wedge (y_3 \leftrightarrow (x_1 \rightarrow x_2)) \\ & \wedge (y_4 \leftrightarrow \neg y_5) \\ & \wedge (y_5 \leftrightarrow (y_6 \vee x_4)) \\ & \wedge (y_6 \leftrightarrow (\neg x_1 \leftrightarrow x_3))\end{aligned}$$

Acquistato da Michele Michele su Webster (12025-07-07 23:12 - Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

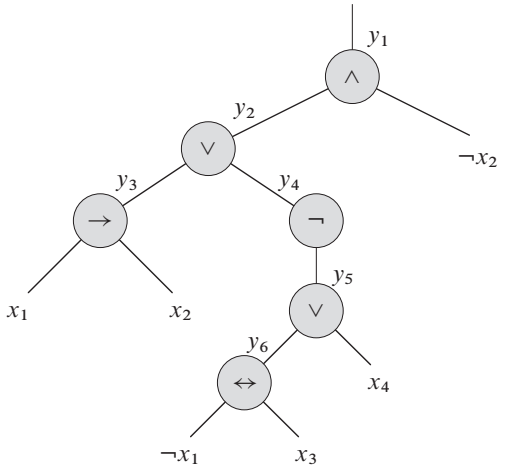


Figura 34.11 L'albero corrispondente alla formula $\phi = ((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$

Notate che la formula ϕ' così ottenuta è una congiunzione di clausole ϕ'_i, ciascuna delle quali ha al più 3 letterali. L'unico requisito aggiuntivo è che ogni clausola deve essere un OR di letterali.

La seconda fase della riduzione converte ogni clausola ϕ'_i nella forma normale congiuntiva. Costruiamo una tavola di verità per ϕ'_i valutando tutte le possibili assegnazioni alle sue variabili. Ogni riga della tavola di verità è formata da una possibile assegnazione delle variabili della clausola, insieme al valore della clausola sotto tale assegnazione. Utilizzando gli elementi della tavola di verità che valgono 0, costruiamo una formula nella ***forma normale disgiuntiva*** (o ***DNF***) – un OR di clausole AND – che è equivalente a $\neg\phi'_i$. Poi convertiamo questa formula in una formula CNF ϕ''_i utilizzando le ***leggi di DeMorgan*** per ottenere il complemento di tutti i letterali e cambiare gli OR in AND e gli AND in OR:

$$\begin{aligned}\neg(a \wedge b) &= \neg a \vee \neg b\\ \neg(a \vee b) &= \neg a \wedge \neg b\end{aligned}$$

Nell'esempio in esame, convertiamo la clausola $\phi'_1 = (y_1 \leftrightarrow (y_2 \wedge \neg x_2))$ nella forma CNF nel modo seguente. La tavola di verità per ϕ'_1 è riportata nella Figura 34.12. La formula DNF equivalente a $\neg\phi'_1$ è

$$(y_1 \wedge y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge \neg x_2) \vee (\neg y_1 \wedge y_2 \wedge \neg x_2)$$

Applicando le leggi di DeMorgan, otteniamo la formula CNF

$$\begin{aligned}\phi''_1 = {} & (\neg y_1 \vee \neg y_2 \vee \neg x_2) \wedge (\neg y_1 \vee y_2 \vee \neg x_2)\\ & \wedge (\neg y_1 \vee y_2 \vee x_2) \wedge (y_1 \vee \neg y_2 \vee x_2)\end{aligned}$$

che è equivalente alla clausola originale ϕ'_1.

Ogni clausola ϕ'_i della formula ϕ' adesso è convertita in una formula CNF ϕ''_i, e quindi ϕ' è equivalente alla formula CNF ϕ'' formata dalla congiunzione delle ϕ''_i. Inoltre, ogni clausola di ϕ'' ha al più 3 letterali.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 19503016-220707-0 Copyright © 2022. McGraw-Hill Education (Italy)

La terza e ultima fase della riduzione trasforma ulteriormente la formula in modo che ogni clausola abbia *esattamente* 3 letterali distinti. La formula 3-CNF finale ϕ''' è ottenuta dalle clausole della formula CNF ϕ''; utilizza anche due variabili ausiliarie che chiameremo p e q. Per ogni clausola C_i di ϕ'', includiamo le seguenti clausole in ϕ''':

- Se C_i ha 3 letterali distinti, allora includiamo semplicemente C_i come una clausola di ϕ'''.

y_1	y_2	x_2	$(y_1 \leftrightarrow (y_2 \wedge \neg x_2))$
1	1	1	0
1	1	0	1
1	0	1	0
1	0	0	0
0	1	1	1
0	1	0	0
0	0	1	1
0	0	0	1

Figura 34.12 La tavola di verità per la clausola $(y_1 \leftrightarrow (y_2 \wedge \neg x_2))$.

- Se C_i ha 2 letterali distinti, cioè, se $C_i = (l_1 \vee l_2)$, dove l_1 e l_2 sono letterali, allora includiamo $(l_1 \vee l_2 \vee p) \wedge (l_1 \vee l_2 \vee \neg p)$ come clausole di ϕ'''. I letterali p e $\neg p$ servono solamente per soddisfare il requisito sintattico di avere esattamente 3 letterali distinti per clausola. Se $p = 0$ o $p = 1$, una della clausole è equivalente a $l_1 \vee l_2$, e l'altra vale 1, che è l'identità di AND.
- Se C_i ha un unico letterale l, allora includiamo $(l \vee p \vee q) \wedge (l \vee p \vee \neg q) \wedge (l \vee \neg p \vee q) \wedge (l \vee \neg p \vee \neg q)$ come clausole di ϕ'''. Indipendentemente dai valori di p e q, una della quattro clausole è equivalente a l, e le altre tre valgono 1.

Se analizziamo ciascuna delle tre fasi, possiamo notare che la formula 3-CNF ϕ''' è soddisfattibile se e soltanto se ϕ è soddisfattibile. Analogamente alla riduzione da CIRCUIT-SAT a SAT, la costruzione di ϕ' da ϕ nella prima fase preserva la soddisfattibilità. La seconda fase produce una formula CNF ϕ'' che è algebricamente equivalente a ϕ'. La terza fase produce una formula 3-CNF ϕ''' che è effettivamente equivalente a ϕ'', perché qualsiasi assegnazione alle variabili p e q produce una formula che è algebricamente equivalente a ϕ''.

Dobbiamo anche dimostrare che la riduzione può essere calcolata in tempo polinomiale. La costruzione di ϕ' da ϕ introduce al più una variabile e una clausola per ogni connettivo in ϕ. La costruzione di ϕ'' da ϕ' può introdurre al più 8 clausole in ϕ'' per ogni clausola di ϕ', perché ogni clausola di ϕ' ha al più 3 variabili e la tavola di verità per ogni clausola ha al più $2^3 = 8$ righe. La costruzione di ϕ''' da ϕ'' introduce al più 4 clausole in ϕ''' per ogni clausola di ϕ''. Quindi, la dimensione della formula risultante ϕ''' è polinomiale nella lunghezza della formula originale. Ciascuna delle costruzioni può essere facilmente realizzata in tempo polinomiale. ■

Esercizi

34.5-1

Considerate il semplice metodo di riduzione (in tempo non polinomiale) utilizzato nella dimostrazione del Teorema 34.9. Descrivete un circuito di dimensione n che, quando viene convertito in una formula con questo metodo, fornisce una formula la cui dimensione è esponenziale in n.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

34.5-2

Calcolate la formula 3-CNF che si ottiene applicando il metodo del Teorema 34.10 alla formula (34.3).

34.5-3

Il professor Jagger propone di dimostrare che SAT $\leq_P$ 3-CNF-SAT utilizzando soltanto la tecnica della tavola di verità nella dimostrazione del Teorema 34.10,

escludendo le altre fasi. Ovvero il professore propone di prendere la formula booleana ϕ, formare una tavola di verità per le sue variabili, derivare dalla tavola di verità una formula nella forma 3-DNF che è equivalente a $\neg\phi$, poi negare e applicare le formule di DeMorgan per produrre una formula 3-CNF equivalente a ϕ. Dimostrate che questa strategia non effettua una riduzione in tempo polinomiale.

34.5-4
Dimostrate che il problema di determinare se una formula booleana è una tautologia è completo per co-NP (*suggerimento:* vedere l'Esercizio 34.4-7).

34.5-5
Dimostrate che il problema di determinare la soddisfattibilità delle formule booleane nella forma normale disgiuntiva è risolvibile in tempo polinomiale.

34.5-6
Supponete che qualcuno vi abbia dato un algoritmo con tempo polinomiale per decidere la soddisfattibilità delle formule. Descrivete come utilizzare questo algoritmo per trovare le assegnazioni che soddisfano una formula in tempo polinomiale.

34.5-7
Sia 2-CNF-SAT l'insieme delle formule booleane soddisfattibili in CNF con esattamente 2 letterali per clausola. Dimostrate che 2-CNF-SAT $\in$ P. Rendete il vostro algoritmo il più efficiente possibile. (*Suggerimento:* notate che $x \vee y$ è equivalente a $\neg x \rightarrow y$. Riducete 2-CNF-SAT a un problema in un grafo orientato che è risolvibile in modo efficiente.)

34.6 Problemi NP-completi

I problemi NP-completi si presentano in diverse aree: logica booleana, grafi, aritmetica, progettazione di reti, insiemi e partizioni, gestione della memoria, sequenzializzazione e scheduling, programmazione matematica, algebra e teoria dei numeri, giochi e puzzle, teoria dei linguaggi e degli automi, ottimizzazione dei programmi, biologia, chimica, fisica e altro ancora. In questo paragrafo, utilizzeremo il metodo della riduzione per fornire le dimostrazioni della NP-completezza per vari problemi scelti dalla teoria dei grafi e dal partizionamento degli insiemi.

La Figura 34.13 illustra la struttura delle dimostrazioni della NP-completezza descritte in questo paragrafo e nel Paragrafo 34.5. Ogni linguaggio nella figura è dimostrato essere NP-completo riducendo il linguaggio che lo punta. Alla radice c'è CIRCUIT-SAT, che abbiamo dimostrato essere NP-completo nel Teorema 34.7.

34.6.1 Il problema della cricca

Una ***cricca*** o ***clique*** in un grafo non orientato $G = (V, E)$ è un sottoinsieme $V' \subseteq V$ di vertici, ogni coppia dei quali è collegata da un arco in E. In altre parole, una cricca è un sottografo completo di G. La ***dimensione*** di una cricca è il numero di vertici che contiene. Il ***problema della cricca*** è un problema di ottimizzazione che consiste nel trovare una cricca di dimensione massima in un grafo. Considerandolo come problema di decisione, ci chiediamo semplicemente se esiste nel grafo una cricca di una data dimensione k. La definizione formale è

CLIQUE $= \{\langle G, k\rangle : G$ è un grafo con una cricca di dimensione $k\}$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1998059016-2204707-0 Copyright © 2022, McGraw-Hill Education (Italy)

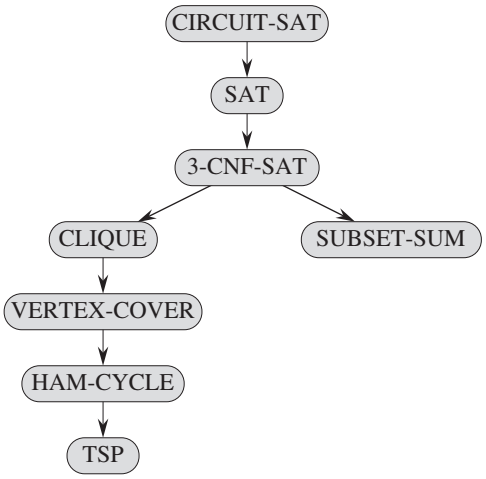


Figura 34.13 La struttura delle dimostrazioni della NP-completezza descritte nei Paragrafi 34.5 e 34.6. Tutte le dimostrazioni, in definitiva, si ottengono per riduzione dalla NP-completezza di CIRCUIT-SAT.

Per determinare se un grafo $G = (V, E)$ con $|V|$ vertici ha una cricca di dimensione k, un algoritmo ingenuo potrebbe elencare tutti i k-sottoinsiemi di V e verificare se ciascuno di essi forma una cricca. Il tempo di esecuzione di questo algoritmo è $\Omega(k^2\binom{|V|}{k})$, che è polinomiale se k è una costante. In generale, però, il valore di k può essere prossimo a $|V|/2$, nel qual caso l'algoritmo sarebbe eseguito in tempo superpolinomiale. Infatti, è poco probabile che esista un algoritmo efficiente per il problema della cricca.

Teorema 34.11
Il problema della cricca è NP-completo.

Dimostrazione Per dimostrare che CLIQUE $\in$ NP per un dato grafo $G = (V, E)$, utilizziamo l'insieme $V' \subseteq V$ dei vertici nella cricca come certificato per G. La verifica che V' è una cricca può essere effettuata in tempo polinomiale controllando, per ogni coppia $u, v \in V'$, se l'arco (u, v) appartiene a E.

Poi dimostriamo che 3-CNF-SAT $\leq_P$ CLIQUE, che prova che il problema della cricca è NP-difficile. Il fatto che sia possibile dimostrare questo è alquanto sorprendente, perché le formule logiche apparentemente hanno poco a che fare con i grafi.

L'algoritmo di riduzione inizia con un'istanza di 3-CNF-SAT. Sia $\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_k$ una formula booleana in 3-CNF con k clausole. Per $r = 1, 2, \ldots, k$, ogni clausola C_r ha esattamente tre letterali distinti l_1^r, l_2^r e l_3^r. Realizzeremo un grafo G tale che ϕ sia soddisfattibile se e soltanto se G ha una cricca di dimensione k.

Il grafo $G = (V, E)$ è realizzato nel modo seguente. Per ogni clausola $C_r = (l_1^r \vee l_2^r \vee l_3^r)$ in ϕ, inseriamo una tripla di vertici v_1^r, v_2^r e v_3^r in V. Poniamo un arco tra due vertici v_i^r e v_j^s se sono soddisfatte entrambe queste condizioni:

- v_i^r e v_j^s si trovano in triple differenti, cioè $r \neq s$.
- I corrispondenti letterali sono ***coerenti***, cioè l_i^r non è la negazione di l_j^s.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Questo grafo può essere facilmente calcolato da ϕ in tempo polinomiale. Come esempio di questa costruzione, se abbiamo

$$\phi = (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$$

allora G è il grafo illustrato nella Figura 34.14.

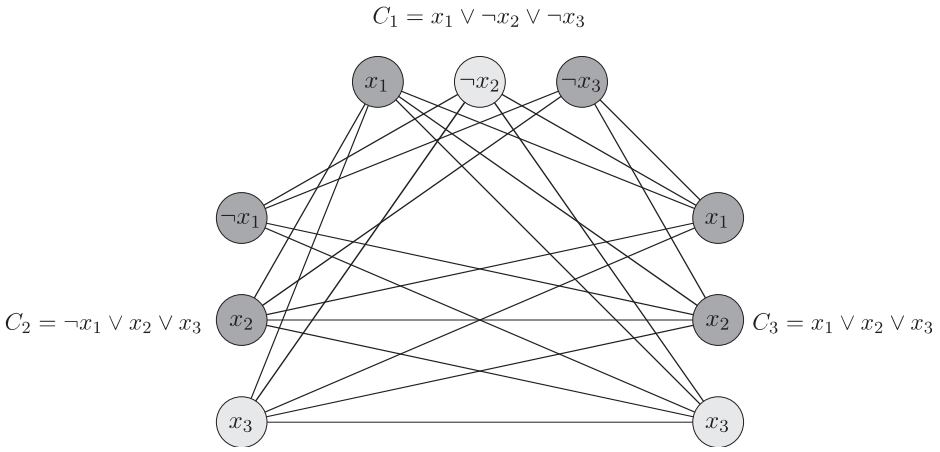


Figura 34.14 Il grafo G ottenuto dalla formula 3-CNF $\phi = C_1 \wedge C_2 \wedge C_3$, dove $C_1 = (x_1 \vee \neg x_2 \vee \neg x_3)$, $C_2 = (\neg x_1 \vee x_2 \vee x_3)$ e $C_3 = (x_1 \vee x_2 \vee x_3)$, riducendo 3-CNF-SAT a CLIQUE. Un'assegnazione che soddisfa la formula ha $x_2 = 0$ e $x_3 = 1$, mentre x_1 può essere 0 o 1. Questa assegnazione soddisfa C_1 con $\neg x_2$; soddisfa anche C_2 e C_3 con x_3; corrisponde alla cricca con i vertici di colore grigio chiaro.

Dobbiamo dimostrare che questa trasformazione di ϕ in G è una riduzione. Innanzi tutto, supponiamo che ϕ abbia un'assegnazione che la soddisfa. Allora ogni clausola C_r contiene almeno un letterale l_i^r cui è assegnato 1, e ciascuno di tali letterali corrisponde a un vertice v_i^r. Prendendo un letterale "true" da ogni clausola si ottiene un insieme V' di k vertici. Asseriamo che V' è una cricca. Per una coppia qualsiasi di vertici $v_i^r, v_j^s \in V'$, dove $r \neq s$, entrambi i corrispondenti letterali l_i^r e l_j^s sono associati a 1 dall'assegnazione di soddisfattibilità, e quindi i letterali non possono essere complementari. Dunque, per la costruzione di G, l'arco (v_i^r, v_j^s) appartiene a E.

Viceversa, supponiamo che G abbia una cricca V' di dimensione k. Nessun arco in G collega i vertici nella stessa tripla, e quindi V' contiene esattamente un vertice per tripla. Possiamo assegnare 1 a ogni letterale l_i^r in modo che $v_i^r \in V'$, senza temere di assegnare 1 sia a un letterale che al suo complemento, perché G non contiene archi tra letterali che non sono coerenti. Ogni clausola è soddisfatta, e quindi ϕ è soddisfatta (a ogni variabile che non corrisponde a un vertice nella cricca può essere assegnato un valore arbitrario.) ■

Nell'esempio della Figura 34.14, un'assegnazione che soddisfa ϕ ha $x_2 = 0$ e $x_3 = 1$. Una cricca corrispondente di dimensione $k = 3$ è formata dai vertici che corrispondono a $\neg x_2$ della prima clausola, a x_3 della seconda clausola e a x_3 della terza clausola. Poiché la cricca non contiene vertici che corrispondono a x_1 o a $\neg x_1$, possiamo porre x_1 a 0 o a 1 in questa assegnazione.

Notate che nella dimostrazione del Teorema 34.11, abbiamo ridotto un'istanza arbitraria di 3-CNF-SAT a un'istanza di CLIQUE con una particolare struttura. Potrebbe sembrare di avere dimostrato che CLIQUE è NP-difficile soltanto nei grafi in cui i vertici sono limitati a presentarsi in triple e in cui non ci sono archi tra vertici nella stessa tripla. In effetti, abbiamo dimostrato che CLIQUE è NP-difficile soltanto in questo caso limitato, ma questa dimostrazione è sufficiente per dimostrare che CLIQUE è NP-difficile anche nei grafi generali. Perché? Se avessimo un algoritmo con tempo polinomiale che risolve CLIQUE nei grafi generali, risolverebbe CLIQUE anche nei grafi limitati.

Acquistato da Michele Michele su Webster il 2022-07-05 18:19:00 - Copia personale - Vietata la distribuzione - Copyright © 2022 McGraw-Hill Education (Italy)

Non sarebbe stato sufficiente, invece, ridurre le istanze di 3-CNF-SAT con una struttura speciale a istanze generali di CLIQUE. Perché? Sarebbe potuto accadere che le istanze di 3-CNF-SAT che abbiamo selezionato per effettuare la ridu-

zione fossero "facili", e quindi non avremmo ridotto un problema NP-difficile a CLIQUE.

Notate inoltre che la riduzione ha utilizzato l'istanza di 3-CNF-SAT, non la soluzione. Sarebbe stato un errore per la riduzione in tempo polinomiale basarsi sulla conoscenza che la formula ϕ è soddisfattibile, perché non sappiamo determinare questa informazione in tempo polinomiale.

34.6.2 Il problema della copertura di vertici

Una ***copertura di vertici*** di un grafo non orientato $G = (V, E)$ è un sottoinsieme $V' \subseteq V$ tale che, se $(u, v) \in E$, allora $u \in V'$ e\o $v \in V'$. Ovvero ciascun vertice "copre" i suoi archi incidenti, e una copertura di vertici per G è un insieme di vertici che copre tutti gli archi in E. La ***dimensione*** di una copertura di vertici è il numero di vertici che appartengono alla copertura. Per esempio, il grafo nella Figura 34.15(b) ha una copertura di vertici $\{w, z\}$ di dimensione 2.

Il ***problema della copertura di vertici*** consiste nel trovare una copertura di vertici di dimensione minima in un dato grafo. Riformulando questo problema di ottimizzazione come un problema di decisione, vogliamo determinare se un grafo ha una copertura di vertici di una data dimensione k.

In termini di linguaggio formale, definiamo

$$\text{VERTEX-COVER} = \{\langle G, k\rangle : \text{il grafo } G \text{ ha una copertura di vertici di dimensione } k\}$$

Il seguente teorema dimostra che questo problema è NP-completo.

Teorema 34.12
Il problema della copertura di vertici è NP-completo.

Dimostrazione Dimostriamo prima che VERTEX-COVER $\in$ NP. Supponiamo di avere un grafo $G = (V, E)$ e un intero k. Il certificato che scegliamo è la stessa copertura di vertici $V' \subseteq V$. L'algoritmo di verifica accerta che $|V'| = k$ e poi controlla, per ogni arco $(u, v) \in E$, che $u \in V'$ o $v \in V'$. Questa verifica può essere effettuata facilmente in tempo polinomiale.

Dimostriamo che il problema della copertura di vertici è NP-difficile dimostrando che CLIQUE $\leq_P$ VERTEX-COVER. Questa riduzione si basa sul concetto di "complemento" di un grafo. Dato un grafo non orientato $G = (V, E)$, definiamo il ***complemento*** di G come $\overline{G} = (V, \overline{E})$, dove $\overline{E} = \{(u, v) : u, v \in V, u \neq v \text{ e } (u, v) \notin E\}$. In altre parole, $\overline{G}$ è il grafo che contiene esattamente quegli archi che non si trovano in G. La Figura 34.15 illustra un grafo e il suo complemento e la riduzione da CLIQUE a VERTEX-COVER.

L'algoritmo di riduzione riceve come input un'istanza $\langle G, k\rangle$ del problema della cricca. Calcola il complemento $\overline{G}$ (operazione facilmente eseguibile in tempo polinomiale). L'output dell'algoritmo di riduzione è l'istanza $\langle \overline{G}, |V| - k\rangle$ del problema della copertura di vertici. Per completare la dimostrazione, proviamo che questa trasformazione è effettivamente una riduzione: il grafo G ha una cricca di dimensione k se e soltanto se il grafo $\overline{G}$ ha una copertura di vertici di dimensione $|V| - k$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 19950301 6-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Supponiamo che G abbia una cricca $V' \subseteq V$ con $|V'| = k$. Asseriamo che $V - V'$ è una copertura di vertici in $\overline{G}$. Sia (u, v) un arco qualsiasi in $\overline{E}$. Allora $(u, v) \notin E$, e questo implica che almeno uno dei vertici u e v non appartiene

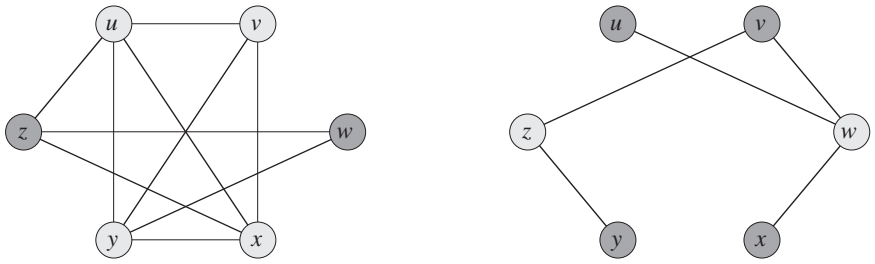


Figura 34.15 Riduzione di CLIQUE a VERTEX-COVER. **(a)** Un grafo non orientato $G = (V, E)$ con la cricca $V' = \{u, v, x, y\}$. **(b)** Il grafo $\overline{G}$ prodotto dall'algoritmo di riduzione che ha la copertura di vertici $V - V' = \{w, z\}$.

a V', perché ogni coppia di vertici in V' è collegata da un arco di E. In modo equivalente, almeno uno dei vertici u e v appartiene a $V - V'$, e questo significa che l'arco (u, v) è coperto da $V - V'$. Poiché (u, v) è stato scelto in modo arbitrario da $\overline{E}$, ogni arco di $\overline{E}$ è coperto da un vertice in $V - V'$. Quindi, l'insieme $V - V'$, che ha dimensione $|V| - k$, forma una copertura di vertici per $\overline{G}$.

Viceversa, supponiamo che $\overline{G}$ abbia una copertura di vertici $V' \subseteq V$, dove $|V'| = |V| - k$. Allora, per ogni $u, v \in V$, se $(u, v) \in \overline{E}$, si ha $u \in V'$ e\o $v \in V'$. La contrapposizione di questa implicazione è che per ogni $u, v \in V$, se $u \notin V'$ e $v \notin V'$, allora $(u, v) \in E$. In altre parole, $V - V'$ è una cricca e ha dimensione $|V| - |V'| = k$. ■

Poiché VERTEX-COVER è NP-completo, non ci aspettiamo di trovare un algoritmo con tempo polinomiale per cercare una copertura di vertici di dimensione minima. Tuttavia, il Paragrafo 35.1 presenta un "algoritmo di approssimazione" con tempo polinomiale, che produce soluzioni "approssimate" per il problema della copertura di vertici. La dimensione di una copertura di vertici prodotta dall'algoritmo è al più due volte la dimensione minima di una copertura di vertici.

Quindi, non dovremmo perdere la speranza solamente perché un problema è NP-completo. Potrebbe esistere un algoritmo di approssimazione con tempo polinomiale che fornisce soluzioni quasi ottime, anche se trovare una soluzione ottima è un problema NP-completo. Il Capitolo 35 presenta diversi algoritmi di approssimazione per i problemi NP-completi.

34.6.3 Il problema del ciclo hamiltoniano

Torniamo al problema del ciclo hamiltoniano definito nel Paragrafo 34.3.

Teorema 34.13
Il problema del ciclo hamiltoniano è NP-completo.

Dimostrazione Dimostriamo prima che HAM-CYCLE appartiene a NP. Dato un grafo $G = (V, E)$, il nostro certificato è la sequenza di $|V|$ vertici che formano il ciclo hamiltoniano. L'algoritmo di verifica controlla che questa sequenza contenga ogni vertice in V una sola volta e che, con il primo vertice ripetuto alla fine, formi un ciclo in G. Ovvero, verifica che ci sia un arco tra ogni coppia di vertici consecutivi e tra il primo e l'ultimo vertice. Questa verifica può essere effettuata in tempo polinomiale.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

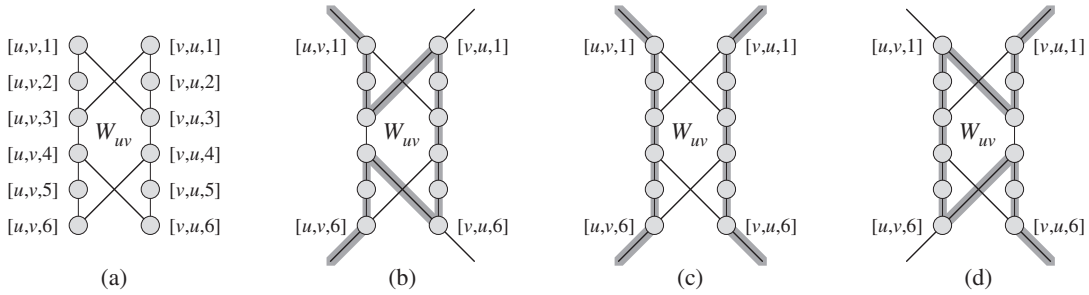


Figura 34.16 Il widget utilizzato per ridurre il problema della copertura di vertici al problema del ciclo hamiltoniano. Un arco (u, v) del grafo G corrisponde al widget W_{uv} nel grafo G' creato nella riduzione. **(a)** Il widget con le etichette nei vertici. **(b)–(d)** I cammini ombreggiati sono gli unici possibili attraverso il widget che includono tutti i vertici, supponendo che le uniche connessioni dal widget al resto di G' passino per i vertici $[u, v, 1]$, $[u, v, 6]$, $[v, u, 1]$ e $[v, u, 6]$.

Dimostriamo adesso che VERTEX-COVER $\leq_P$ HAM-CYCLE, che prova che HAM-CYCLE è NP-completo. Dato un grafo non orientato $G = (V, E)$ e un intero k, costruiamo un grafo non orientato $G' = (V', E')$ che ha un ciclo hamiltoniano se e soltanto se G ha una copertura di vertici di dimensione k.

La nostra costruzione di basa su un ***widget***, che è un pezzo di un grafo che impone alcune proprietà. La Figura 34.16(a) illustra il widget utilizzato. Per ogni arco $(u, v) \in E$, il grafo G' che costruiamo conterrà una copia di questo widget, che indichiamo con W_{uv}. Indichiamo ciascun vertice in W_{uv} con $[u, v, i]$ o $[v, u, i]$, dove $1 \leq i \leq 6$, in modo che ogni widget W_{uv} contenga 12 vertici. Il widget W_{uv} contiene anche i 14 archi illustrati nella Figura 34.16(a).

Assieme alla struttura interna del widget, imponiamo le proprietà che ci interessano, limitando le connessioni tra il widget e il resto del grafo G' che costruiamo. In particolare, soltanto i vertici $[u, v, 1]$, $[u, v, 6]$, $[v, u, 1]$ e $[v, u, 6]$ avranno archi incidenti dall'esterno di W_{uv}. Qualsiasi ciclo hamiltoniano di G' dovrà attraversare gli archi di W_{uv} in uno dei tre modi illustrati nelle Figure 34.16(b)–(d). Se il ciclo entra nel vertice $[u, v, 1]$, deve uscire dal vertice $[u, v, 6]$, e visita tutti i 12 vertici del widget (Figura 34.16(b)) o i sei vertici da $[u, v, 1]$ a $[u, v, 6]$ (Figura 34.16(c)). Nel secondo caso, il ciclo dovrà rientrare nel widget per visitare i vertici da $[v, u, 1]$ a $[v, u, 6]$. Analogamente, se il ciclo entra nel vertice $[v, u, 1]$, deve uscire dal vertice $[v, u, 6]$, e visita tutti i 12 vertici del widget (Figura 34.16(d)) o i sei vertici da $[v, u, 1]$ a $[v, u, 6]$ (Figura 34.16(c)). Non sono possibili altri cammini attraverso il widget che visitano tutti i 12 vertici. In particolare, è impossibile costruire due cammini disgiunti sui vertici, uno dei quali collega $[u, v, 1]$ a $[v, u, 6]$ e l'altro collega $[v, u, 1]$ a $[u, v, 6]$, in modo che l'unione dei due cammini contenga tutti i vertici del widget.

Gli unici altri vertici in V' diversi da quelli dei widget sono i ***vertici selettori*** $s_1, s_2, \ldots, s_k$. Utilizziamo gli archi incidenti nei vertici selettori in G' per selezionare k vertici della copertura in G.

Acquistato da Michele Nicolazzo Waterstile 2022-07-01 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Oltre agli archi nei widget, ci sono altri due tipi di archi in E', come illustra la Figura 34.17. In primo luogo, per ogni vertice $u \in V$, aggiungiamo degli archi per unire le coppie di widget in modo da formare un cammino contenente tutti i widget che corrispondono agli archi incidenti in u nel grafo G. Ordiniamo arbitrariamente i vertici adiacenti a ciascun vertice $u \in V$ come $u^{(1)}, u^{(2)}, \ldots, u^{(\text{degree}(u))}$,

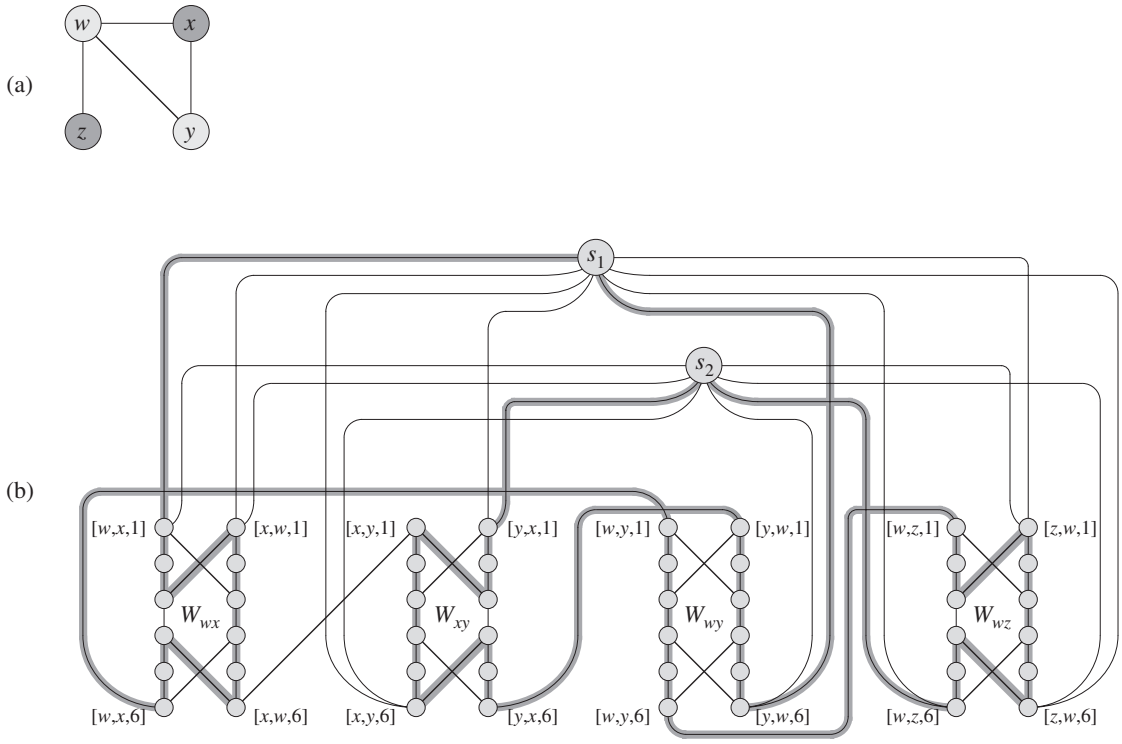


Figura 34.17 La riduzione di un'istanza del problema della copertura di vertici a un'istanza del problema del ciclo hamiltoniano. **(a)** Un grafo non orientato G con una copertura di vertici di dimensione 2, formata dai vertici w e y di colore grigio chiaro. **(b)** Il grafo non orientato G' prodotto dalla riduzione; il cammino hamiltoniano corrispondente alla copertura di vertici è ombreggiato. La copertura di vertici $\{w, y\}$ corrisponde agli archi $(s_1, [w, x, 1])$ e $(s_2, [y, x, 1])$ che appaiono nel ciclo hamiltoniano.

dove degree(u) è il numero di vertici adiacenti a u. Creiamo un cammino in G' attraverso tutti i widget che corrispondono agli archi incidenti in u aggiungendo a E' gli archi $\{([u, u^{(i)}, 6], [u, u^{(i+1)}, 1]) : 1 \le i \le \text{degree}(u) - 1\}$. Nella Figura 34.17, per esempio, ordiniamo i vertici adiacenti a w come x, y, z; quindi il grafo G' nella parte (b) della figura include gli archi $([w, x, 6], [w, y, 1])$ e $([w, y, 6], [w, z, 1])$. Per ogni vertice $u \in V$, questi archi in G' completano un cammino contenente tutti i widget che corrispondono agli archi incidenti in u nel grafo G.

Il concetto intuitivo che sta dietro questi archi è che se scegliamo un vertice $u \in V$ nella copertura di vertici di G, possiamo costruire un cammino da $[u, u^{(1)}, 1]$ a $[u, u^{(\text{degree}(u))}, 6]$ in G' che "copre" tutti i widget che corrispondono agli archi incidenti in u. Ovvero, per ciascuno di questi widget, per esempio $W_{u,u^{(i)}}$, il cammino include o tutti i 12 vertici (se u è nella copertura di vertici, ma $u^{(i)}$ non lo è) o soltanto i 6 vertici $[u, u^{(i)}, 1], [u, u^{(i)}, 2], \ldots, [u, u^{(i)}, 6]$ (se u e $u^{(i)}$ sono entrambi nella copertura di vertici).

Acquistato da Michele Michele su Webster il 2022-07-23 per uso esclusivo [illegible] Copyright © 2022, McGraw-Hill Education (Italy)

L'ultimo tipo di arco in E' è quello che unisce il primo vertice $[u, u^{(1)}, 1]$ e l'ultimo vertice $[u, u^{(\text{degree}(u))}, 6]$ di ciascuno di questi cammini a ogni vertice selettore. Ovvero, includiamo gli archi

$$\{(s_j, [u, u^{(1)}, 1]) : u \in V \text{ e } 1 \le j \le k\}$$
$$\cup \{(s_j, [u, u^{(\text{degree}(u))}, 6]) : u \in V \text{ e } 1 \le j \le k\}$$

Adesso dimostriamo che la dimensione di G' è polinomiale nella dimensione di G; quindi possiamo costruire G' in tempo polinomiale nella dimensione di G. I vertici di G' sono quelli nei widget, più i vertici selettori. Ogni widget contiene 12 vertici e ci sono $k \leq |V|$ vertici selettori, per un totale di

$$\begin{aligned} |V'| &= 12\,|E| + k \\ &\leq 12\,|E| + |V| \end{aligned}$$

vertici. Gli archi di G' sono quelli nei widget, quelli che uniscono i widget e quelli che collegano i vertici selettori ai widget. Ci sono 14 archi in ogni widget, o $14\,|E|$ in tutti i widget. Per ogni vertice $u \in V$, ci sono $\text{degree}(u) - 1$ archi tra i widget, quindi sommando per tutti i vertici in V, ci sono

$$\sum_{u \in V} (\text{degree}(u) - 1) = 2\,|E| - |V|$$

archi tra i widget. Infine, ci sono due archi per ogni coppia formata da un vertice selettore e un vertice di V, o $2k\,|V|$ di tali archi. Il numero totale di archi di G' è quindi

$$\begin{aligned} |E'| &= (14\,|E|) + (2\,|E| - |V|) + (2k\,|V|) \\ &= 16\,|E| + (2k - 1)\,|V| \\ &\leq 16\,|E| + (2\,|V| - 1)\,|V| \end{aligned}$$

Adesso dimostriamo che la trasformazione dal grafo G a G' è una riduzione. Ovvero dobbiamo dimostrare che G ha una copertura di vertici di dimensione k se e soltanto se G' ha un ciclo hamiltoniano.

Supponiamo che $G = (V, E)$ abbia una copertura di vertici $V^* \subseteq V$ di dimensione k. Sia $V^* = \{u_1, u_2, \ldots, u_k\}$. Come illustra la Figura 34.17, formiamo un ciclo hamiltoniano in G' includendo i seguenti archi[10] per ogni vertice $u_j \in V^*$. Includiamo gli archi $\left\{([u_j, u_j^{(i)}, 6], [u_j, u_j^{(i+1)}, 1]) : 1 \leq i \leq \text{degree}(u_j)\right\}$, che collegano tutti i widget che corrispondono agli archi incidenti in u_j. Includiamo anche gli archi all'interno di questi widget, come illustrano le Figure 34.16(b)–(d), a seconda che l'arco sia coperto da uno o due vertici in V^*. Il ciclo hamiltoniano include anche gli archi

$$\begin{aligned} &\{(s_j, [u_j, u_j^{(1)}, 1]) : 1 \leq j \leq k\} \\ &\qquad \cup \{(s_{j+1}, [u_j, u_j^{(\text{degree}(u_j))}, 6]) : 1 \leq j \leq k - 1\} \\ &\qquad \cup \{(s_1, [u_k, u_k^{(\text{degree}(u_k))}, 6])\} \end{aligned}$$

Esaminando la Figura 34.17, il lettore può verificare che questi archi formano un ciclo. Il ciclo inizia in s_1, visita tutti i widget che corrispondono agli archi incidenti in u_1, poi passa per s_2, visita tutti i widget che corrispondono agli archi incidenti in u_2, e così via fino a ritornare in s_1. Ogni widget è visitato una sola volta o due volte, a seconda che uno o due vertici di V^* coprano il suo arco corrispondente. Poiché V^* è una copertura di vertici per G, ogni arco in E è incidente in qualche vertice in V^*; quindi il ciclo visita ogni vertice di G'. Poiché il ciclo visita anche ogni vertice selettore, esso è hamiltoniano.

Acquistato da Michele Michels su Webster il 2023-07-07 23:12 Numero Ordine: ibrexia: 19950316-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

[10] Tecnicamente, un ciclo è definito in termini di vertici anziché di archi (Paragrafo B.4). Per ragioni di chiarezza, abusiamo della notazione e definiamo il ciclo hamiltoniano in termini di archi.

Viceversa, supponiamo che $G' = (V', E')$ abbia un ciclo hamiltoniano $C \subseteq E'$. Asseriamo che l'insieme

$$V^* = \{u \in V : (s_j, [u, u^{(1)}, 1]) \in C \text{ per qualche } 1 \le j \le k\} \tag{34.4}$$

è una copertura di vertici per G. Per capire perché, dividiamo C in cammini massimali che iniziano in qualche vertice selettore s_i, attraversano un arco $(s_i, [u, u^{(1)}, 1])$ per qualche $u \in V$, e arrivano in un vertice selettore s_j senza passare per altri vertici selettori. Ciascuno di questi cammini è detto "cammino di copertura". Dal modo in cui è costruito G', ogni cammino di copertura deve iniziare da qualche s_i, prendere l'arco $(s_i, [u, u^{(1)}, 1])$ per qualche vertice $u \in V$, attraversare tutti i widget che corrispondono agli archi in E incidenti in u e poi terminare in qualche vertice selettore s_j. Indichiamo con p_u questo cammino di copertura; per l'equazione (34.4), poniamo u in V^*. Ogni widget visitato da p_u deve essere W_{uv} o W_{vu} per qualche $v \in V$. Per ogni widget visitato da p_u, i suoi vertici sono visitati da uno o due cammini di copertura. Se sono visitati da un cammino di copertura, allora l'arco $(u, v) \in E$ è coperto in G dal vertice u. Se due cammini di copertura visitano il widget, allora l'altro cammino di copertura deve essere p_v, e questo implica che $v \in V^*$, e l'arco $(u, v) \in E$ è coperto da u e v. Poiché ogni vertice in ciascun widget è visitato da qualche cammino di copertura, notiamo che ogni arco in E è coperto da qualche vertice in V^*. ■

34.6.4 Il problema del commesso viaggiatore

Nel ***problema del commesso viaggiatore***, che è strettamente correlato al problema del ciclo hamiltoniano, un commesso deve visitare n città. Modellando il problema come un grafo completo con n vertici, possiamo dire che il commesso vuole fare un ***giro***, o ciclo hamiltoniano, visitando ogni città una sola volta e ritornando alla città di partenza. C'è un costo $c(i, j)$ (di valore intero) per viaggiare dalla città i alla città j; il commesso vuole fare il giro il cui costo totale è minimo, dove il costo totale è la somma dei singoli costi lungo gli archi del giro. Per esempio, nella Figura 34.18, un giro di costo minimo è $\langle u, w, v, x, u\rangle$, cha ha un costo 7. Il linguaggio formale per il corrispondente problema di decisione è

$$\begin{aligned}\text{TSP} = \{\langle G, c, k\rangle : \ & G = (V, E) \text{ è un grafo completo,} \\ & c \text{ è una funzione da } V \times V \to \mathbb{Z}, \\ & k \in \mathbb{Z} \text{ e} \\ & G \text{ ha un giro del commesso viaggiatore} \\ & \quad \text{con un costo al più } k\}\end{aligned}$$

Il seguente teorema dimostra che è poco probabile che esista un algoritmo veloce per il problema del commesso viaggiatore.

Teorema 34.14

Il problema del commesso viaggiatore è NP-completo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Dimostrazione Dimostriamo prima che TSP appartiene alla classe NP. Data un'istanza del problema, utilizziamo come certificato la sequenza di n vertici nel giro. L'algoritmo di verifica controlla se questa sequenza contiene ogni vertice una sola volta, somma i costi degli archi e controlla se la somma è al più k. Questo processo può certamente essere eseguito in tempo polinomiale.

Per dimostrare che TSP è NP-difficile, dimostriamo che HAM-CYCLE $\le_P$ TSP. Sia $G = (V, E)$ un'istanza di HAM-CYCLE. Costruiamo un'istanza di

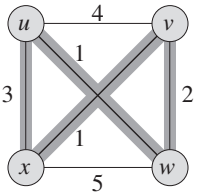


Figura 34.18 Un'istanza del problema del commesso viaggiatore. Gli archi ombreggiati rappresentano un giro di costo minimo (in questo caso il costo è 7).

TSP nel modo seguente. Formiamo il grafo completo $G' = (V, E')$, dove $E' = \{(i, j) : i, j \in V \text{ e } i \neq j\}$, e definiamo la funzione costo c come

$$c(i, j) = \begin{cases} 0 & \text{se } (i, j) \in E \\ 1 & \text{se } (i, j) \notin E \end{cases}$$

(Notiamo che, poiché G non è orientato, non ha cappi, e quindi $c(v, v) = 1$ per tutti i vertici $v \in V$.) L'istanza di TSP è allora $\langle G', c, 0 \rangle$, che è facilmente formata in tempo polinomiale.

Adesso dimostriamo che il grafo G ha un ciclo hamiltoniano se e soltanto se il grafo G' ha un giro di costo al più 0. Supponiamo che il grafo G abbia un ciclo hamiltoniano h. Ogni arco in h appartiene a E e quindi ha costo 0 in G'. Ne consegue che h è un giro in G' con costo 0. Viceversa, supponiamo che il grafo G' abbia un giro h' di costo al più 0. Poiché i costi degli archi in E' sono 0 e 1, il costo del giro h' è esattamente 0 e ogni arco nel giro deve avere costo 0. Di conseguenza, h' contiene soltanto archi in E. Concludiamo che h' è un ciclo hamiltoniano nel grafo G. ■

34.6.5 Il problema della somma di sottoinsieme

Il prossimo problema NP-completo che analizzeremo è aritmetico. Nel ***problema della somma di sottoinsieme***, sono dati un insieme finito S di interi positivi e un ***obiettivo*** intero $t > 0$. Ci chiediamo se esiste un sottoinsieme $S' \subseteq S$ tale che la somma dei suoi elementi sia t. Per esempio, se $S = \{1, 2, 7, 14, 49, 98, 343, 686, 2409, 2793, 16808, 17206, 117705, 117993\}$ e $t = 138457$, allora il sottoinsieme $S' = \{1, 2, 7, 98, 343, 686, 2409, 17206, 117705\}$ è una soluzione.

Come al solito, definiamo il problema come un linguaggio:

$$\text{SUBSET-SUM} = \{\langle S, t \rangle : \text{esiste un sottoinsieme } S' \subseteq S \text{ tale che } t = \textstyle\sum_{s \in S'} s\}$$

Come per qualsiasi problema aritmetico, è importante ricordare che la nostra codifica standard suppone che gli interi di input siano codificati in binario. Tenendo presente questa ipotesi, possiamo dimostrare che il problema della somma di sottoinsieme è poco probabile che abbia un algoritmo rapido.

Teorema 34.15

Il problema della somma di sottoinsieme è NP-completo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Dimostrazione Per dimostrare che SUBSET-SUM è in classe NP, per un'istanza $\langle S, t \rangle$ del problema, utilizziamo il sottoinsieme S' come certificato. Il controllo che $t = \sum_{s \in S'} s$ può essere effettuato da un algoritmo di verifica in tempo polinomiale.

Adesso dimostriamo che 3-CNF-SAT $\leq_P$ SUBSET-SUM. Data una formula 3-CNF ϕ nelle variabili $x_1, x_2, \ldots, x_n$ con le clausole $C_1, C_2, \ldots, C_k$, ciascuna

contenente esattamente tre letterali distinti, l'algoritmo di riduzione costruisce un'istanza $\langle S, t \rangle$ del problema della somma di sottoinsieme tale che ϕ sia soddisfattibile se e soltanto se esiste un sottoinsieme di S la cui somma è esattamente t. Senza perdere in generalità, facciamo due ipotesi semplificative sulla formula ϕ: a) nessuna clausola può contenere contemporaneamente una variabile e la sua negazione, altrimenti una clausola sarebbe automaticamente soddisfatta da qualsiasi assegnazione di valori alle variabili; b) ciascuna variabile deve apparire almeno in una clausola, altrimenti non avrebbe importanza quale valore sia assegnato alla variabile.

La riduzione crea due numeri nell'insieme S per ogni variabile x_i e due numeri in S per ogni clausola C_j. Creeremo i numeri in base 10, dove ogni numero contiene $n + k$ cifre e ogni cifra corrisponde a una variabile o a una clausola. La base 10 (e altre basi, come vedremo) ha una proprietà che ci serve: impedire i riporti dalle cifre meno significative a quelle più significative.

Come illustra la Figura 34.19, costruiamo l'insieme S e l'obiettivo t nel modo seguente. Etichettiamo ogni posizione delle cifre con una variabile o con una clausola. Le k cifre meno significative sono etichettate con le clausole; le n cifre più significative sono etichettate con le variabili.

- L'obiettivo t ha un 1 in ogni cifra etichettata con una variabile e un 4 in ogni cifra etichettata con una clausola.
- Per ogni variabile x_i, ci sono due interi, v_i e v'_i, in S. Ogni intero ha un 1 nella cifra etichettata con x_i e uno 0 nelle cifre delle altre variabili. Se il letterale x_i appare nella clausola C_j, allora la cifra etichettata con C_j in v_i contiene un 1. Se il letterale $\neg x_i$ appare nella clausola C_j, allora la cifra etichettata con C_j in v'_i contiene un 1. Tutte le altre cifre etichettate con le clausole in v_i e v'_i sono 0.

  Tutti i valori di v_i e v'_i nell'insieme S sono unici. Perché? Per $l \neq i$, i valori di v_l e v'_l non possono essere uguali a v_i e v'_i nelle n cifre più significative. Inoltre, per le precedenti ipotesi semplificative, i valori di v_i e v'_i non possono essere uguali in tutte le k cifre meno significative. Se v_i e v'_i fossero uguali, allora x_i e $\neg x_i$ dovrebbero apparire esattamente nello stesso insieme di clausole. Ma noi supponiamo che nessuna clausola contenga sia x_i che $\neg x_i$ e che in qualche clausola ci sia o x_i o $\neg x_i$; quindi deve esistere qualche clausola C_j per la quale v_i e v'_i sono differenti.
- Per ogni clausola C_j, ci sono due interi, s_j e s'_j in S. Ogni intero ha uno 0 in tutte le cifre diverse da quella etichettata con C_j. Per s_j, c'è un 1 nella cifra C_j, mentre s'_j ha un 2 in questa cifra. Questi interi sono "variabili slack", che utilizziamo per ottenere il valore 4 in ciascuna cifra dell'obiettivo etichettata con una clausola.

  Da un semplice esame della Figura 34.19, notiamo che tutti i valori di s_j e s'_j sono unici nell'insieme S.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria 1985020 a 2207072 Copyright © 2022 McGraw-Hill Education (Italy)

Notate che la somma massima delle cifre in qualsiasi posizione è 6, che si ottiene in corrispondenza delle cifre etichettate con le clausole (tre 1 dai valori di v_i e v'_i, più 1 e 2 dai valori di s_j e s'_j). Interpretando questi numeri in base 10, quindi,

		x_1	x_2	x_3	C_1	C_2	C_3	C_4
v_1	=	1	0	0	1	0	0	1
v'_1	=	1	0	0	0	1	1	0
v_2	=	0	1	0	0	0	0	1
v'_2	=	0	1	0	1	1	1	0
v_3	=	0	0	1	0	0	1	1
v'_3	=	0	0	1	1	1	0	0
s_1	=	0	0	0	1	0	0	0
s'_1	=	0	0	0	2	0	0	0
s_2	=	0	0	0	0	1	0	0
s'_2	=	0	0	0	0	2	0	0
s_3	=	0	0	0	0	0	1	0
s'_3	=	0	0	0	0	0	2	0
s_4	=	0	0	0	0	0	0	1
s'_4	=	0	0	0	0	0	0	2
t	=	1	1	1	4	4	4	4

Figura 34.19 La riduzione di 3-CNF-SAT a SUBSET-SUM. La formula in 3-CNF è $\phi = C_1 \wedge C_2 \wedge C_3 \wedge C_4$, dove $C_1 = (x_1 \vee \neg x_2 \vee \neg x_3)$, $C_2 = (\neg x_1 \vee \neg x_2 \vee \neg x_3)$, $C_3 = (\neg x_1 \vee \neg x_2 \vee x_3)$ e $C_4 = (x_1 \vee x_2 \vee x_3)$. Un'assegnazione che soddisfa ϕ è $\langle x_1 = 0, x_2 = 0, x_3 = 1\rangle$. L'insieme S prodotto dalla riduzione è formato dai numeri in base 10 illustrati nella figura; leggendo dall'alto in basso, $S = \{1001001, 1000110, 100001, 101110, 10011, 11100, 1000, 2000, 100, 200, 10, 20, 1, 2\}$. L'obiettivo t è 1114444. Il sottoinsieme $S' \subseteq S$, di colore grigio chiaro, contiene gli elementi v'_1, v'_2 e v_3, che corrispondono all'assegnazione che soddisfa ϕ; contiene anche le variabili slack s_1, s'_1, s'_2, s_3, s_4 ed s'_4 per ottenere il valore 4 nelle cifre dell'obiettivo etichettate da C_1 a C_4.

non si ha alcun riporto dalle cifre meno significative a quelle più significative.[11]

La riduzione può essere eseguita in tempo polinomiale. L'insieme S contiene $2n + 2k$ valori, ciascuno dei quali ha $n + k$ cifre; il tempo per produrre ciascuna cifra è polinomiale in $n + k$. L'obiettivo t ha $n + k$ cifre, ciascuna delle quali è prodotta dalla riduzione in tempo costante.

Adesso dimostriamo che la formula 3-CNF ϕ è soddisfattibile se e soltanto se esiste un sottoinsieme $S' \subseteq S$ la cui somma è t. Innanzi tutto, supponiamo che ϕ abbia un'assegnazione che la soddisfa. Per $i = 1, 2, \ldots, n$, se $x_i = 1$ in questa assegnazione, allora includiamo v_i in S', altrimenti includiamo v'_i. In altre parole, includiamo in S' esattamente i valori di v_i e v'_i che corrispondono ai letterali con il valore 1 nell'assegnazione che soddisfa ϕ. Avendo incluso v_i o v'_i, ma non entrambi, per ogni i, e avendo posto 0 nelle cifre etichettate con le variabili in ogni s_j ed s'_j, osserviamo che per ogni cifra etichettata con una variabile, la somma dei valori di S' deve essere 1, che corrisponde a quelle cifre dell'obiettivo t. Poiché ogni clausola è soddisfatta, esiste qualche letterale nella clausola con il valore 1. Pertanto, ogni cifra etichettata con una clausola ha almeno un 1 nella sua somma proveniente da un valore di v_i o v'_i in S'. In effetti, 1, 2 o 3 letterali possono essere 1 in ogni clausola; quindi ogni cifra etichettata con una clausola ha una somma pari a 1, 2 o 3 dai valori v_i e v'_i in S'. (Per esempio, nella Figura 34.19 i letterali $\neg x_1$, $\neg x_2$ e x_3 hanno il valore 1 in un'assegnazione che soddisfa la formula. Ciascuna delle clausole C_1 e C_4 contiene uno solo di questi letterali; quindi v'_1, v'_2 e v_3 insieme contribuiscono con 1 alla somma nelle cifre per C_1 e C_4. La clausola C_2 contiene due di questi letterali, quindi v'_1, v'_2 e v_3 contribuiscono con 2 alla somma nella cifra per C_2. La clausola C_3 contiene tutti e tre questi letterali, quindi v'_1, v'_2 e v_3 contribuiscono con 3 alla somma nella cifra per C_3.) Otteniamo il valore 4 per l'obiettivo in ogni cifra etichettata con la clausola C_j,

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995[illegible]016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

[11] In effetti, andrebbe bene qualsiasi base b, dove $b \geq 7$. L'istanza all'inizio di questo sottoparagrafo è l'insieme S e l'obiettivo t nella Figura 34.19 interpretati in base 7, con gli elementi di S ordinati.

includendo in S' un appropriato sottoinsieme non vuoto di variabili slack $\{s_j, s'_j\}$. (Nella Figura 34.19, S' include s_1, s'_1, s'_2, s_3, s_4 e s'_4.) Poiché abbiamo ottenuto i valori dell'obiettivo in tutte le cifre della somma e non possono esserci riporti, la somma dei valori di S' è t.

Adesso, supponiamo che ci sia un sottoinsieme $S' \subseteq S$ la cui somma è t. Il sottoinsieme S' deve includere uno solo dei valori v_i e v'_i per ogni $i = 1, 2, \ldots, n$, altrimenti la somma delle cifre etichettate con le variabili non sarebbe 1. Se $v_i \in S'$, poniamo $x_i = 1$; altrimenti $v'_i \in S'$, e poniamo $x_i = 0$. Asseriamo che ogni clausola C_j, per $j = 1, 2, \ldots, k$, è soddisfatta da questa assegnazione. Per dimostrare questa asserzione, notiamo che per ottenere una somma pari a 4 nella cifra etichettata con C_j, il sottoinsieme S' deve includere almeno un valore di v_i o v'_i che ha un 1 nella cifra etichettata con C_j, perché il contributo complessivo delle variabili slack s_j ed s'_j è al massimo 3. Se S' include un valore di v_i che ha un 1 in quella posizione, allora il letterale x_i appare nella clausola C_j. Poiché abbiamo posto $x_i = 1$ quando $v_i \in S'$, la clausola C_j è soddisfatta. Se S' include un valore di v'_i che ha un 1 in quella posizione, allora il letterale $\neg x_i$ appare in C_j. Poiché abbiamo posto $x_i = 0$ quando $v'_i \in S'$, la clausola C_j è ancora soddisfatta. Quindi, tutte le clausole di ϕ sono soddisfatte, e questo completa la dimostrazione. ■

Esercizi

34.6-1

Dati due grafi G_1 e G_2, il ***problema dell'isomorfismo dei sottografi*** consiste nel determinare se G_1 è isomorfo a un sottografo di G_2. Formulate il corrispondente problema di decisione e dimostrate che il problema di decisione è NP-completo.

34.6-2

Sia A una matrice di interi, $m \times n$, e sia b un vettore di m interi, il ***problema della programmazione intera 0-1*** consiste nel determinare se esiste un vettore x di n interi, che appartengono all'insieme $\{0, 1\}$, tali che $Ax \leq b$. Dimostrate che questo problema è NP-completo (*suggerimento:* riducete da 3-CNF-SAT).

34.6-3

Il ***problema della programmazione lineare intera*** è simile al problema della programmazione intera 0-1 descritto nell'Esercizio 34.6-2, con la differenza che i valori del vettore x possono essere interi qualsiasi, non soltanto 0 e 1. Supponendo che il problema della programmazione intera 0-1 sia NP-difficile, dimostrate che il problema della programmazione lineare intera è NP-completo.

34.6-4

Dimostrate che il problema della somma di sottoinsieme è risolvibile in tempo polinomiale se il valore dell'obiettivo t è espresso in unario.

34.6-5

Dato un insieme S di numeri, il ***problema del partizionamento dell'insieme*** consiste nel determinare se i numeri possono essere partizionati in due insiemi A e $\overline{A} = S - A$ tali che $\sum_{x \in A} x = \sum_{x \in \overline{A}} x$. Dimostrate che questo problema è NP-completo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:11 Maresca Cirdino Libreria: 19950316 230707 0 Copyright © 2022 The McGraw-Hill Education (Italy)

34.6-6

Dimostrate che il problema del cammino hamiltoniano è NP-completo.

34.6-7

Il ***problema del ciclo semplice più lungo*** consiste nel determinare un ciclo semplice (senza vertici ripetuti) di lunghezza massima in un grafo. Formulate il corrispondente problema di decisione e dimostrate che il problema di decisione è NP-completo.

34.6-8

Nel ***problema della soddisfattibilità a metà di una formula 3-CNF*** sono date una formula 3-CNF ϕ con n variabili ed m clausole, dove m è pari; vogliamo determinare se esiste un'assegnazione di verità alle variabili di ϕ tale che metà delle clausole risulti pari a 0 e l'altra metà risulti pari a 1. Dimostrate che questo problema è NP-completo.

Problemi

34-1 Insieme indipendente

Un ***insieme indipendente*** di un grafo $G = (V, E)$ è un sottoinsieme $V' \subseteq V$ di vertici tali che ogni arco in E sia incidente al più in un vertice di V'. Il ***problema dell'insieme indipendente*** consiste nel trovare un insieme indipendente di dimensione massima nel grafo G.

a. Formulate un problema di decisione associato al problema dell'insieme indipendente e dimostrate che esso è NP-completo (*suggerimento:* riducete dal problema della cricca).

b. Supponete di avere a disposizione una subroutine che risolve il problema di decisione definito nel punto (a). Create un algoritmo per trovare un insieme indipendente di dimensione massima. Il tempo di esecuzione del vostro algoritmo dovrebbe essere polinomiale in $|V|$ e $|E|$, dove le chiamate della subroutine sono conteggiate come un singolo passo.

Sebbene il problema di decisione per l'insieme indipendente sia NP-completo, alcuni casi speciali sono risolvibili in tempo polinomiale.

c. Create un algoritmo efficiente per risolvere il problema dell'insieme indipendente quando ogni vertice di G ha grado 2. Analizzate il tempo di esecuzione e dimostrate che l'algoritmo funziona correttamente.

d. Create un algoritmo efficiente per risolvere il problema dell'insieme indipendente quando G è un grafo bipartito. Analizzate il tempo di esecuzione e dimostrate che l'algoritmo funziona correttamente (*suggerimento:* utilizzate i risultati del Paragrafo 26.3).

34-2 Bonnie e Clyde

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Bonnie e Clyde hanno appena rapinato una banca. Hanno un sacco di soldi che vogliono dividersi. Per ciascuno dei seguenti casi, create un algoritmo con tempo polinomiale oppure dimostrate che il problema è NP-completo. L'input in ogni caso è una lista degli n elementi che si trovano nel sacco, con il valore di ciascun elemento.

a. Ci sono n monete di due valori differenti: alcune monete valgono x dollari e altre y dollari. Bonnie e Clyde vogliono dividersi i soldi esattamente a metà.

b. Ci sono n monete con un numero arbitrario di valori differenti, ma ogni valore è una potenza intera non negativa di 2, cioè, i possibili valori sono 1 dollaro, 2 dollari, 4 dollari e così via. Bonnie e Clyde vogliono dividersi i soldi esattamente a metà.

c. Ci sono n assegni che, per una sorprendente coincidenza, sono intestati "a Bonnie o a Clyde". I due ladri vogliono dividersi gli assegni in modo che ciascuno abbia esattamente la stessa quantità di soldi.

d. Ci sono n assegni come nel punto (c), ma stavolta Bonnie e Clyde sono disposti ad accettare una spartizione con una differenza non maggiore di 100 dollari.

34-3 Colorazione di un grafo

Una ***k-colorazione*** di un grafo non orientato $G = (V, E)$ è una funzione $c : V \to \{1, 2, \ldots, k\}$ tale che $c(u) \neq c(v)$ per ogni arco $(u, v) \in E$. In altre parole, i numeri $1, 2, \ldots, k$ rappresentano i k colori; inoltre i vertici adiacenti devono avere colori differenti. Il ***problema della colorazione di un grafo*** consiste nel determinare il numero minimo di colori richiesti per colorare un dato grafo.

a. Create un algoritmo efficiente per determinare una 2-colorazione di un grafo, se esiste.

b. Formulate il problema della colorazione di un grafo come un problema di decisione. Dimostrate che il problema di decisione è risolvibile in tempo polinomiale se e soltanto se il problema della colorazione del grafo è risolvibile in tempo polinomiale.

c. Sia il linguaggio 3-COLOR l'insieme dei grafi che possono essere 3-colorati. Dimostrate che se 3-COLOR è NP-completo, allora il problema di decisione del punto (b) è NP-completo.

Per dimostrare che 3-COLOR è NP-completo, utilizziamo una riduzione da 3-CNF-SAT. Data una formula ϕ di m clausole in n variabili $x_1, x_2, \ldots, x_n$, realizziamo un grafo $G = (V, E)$ nel modo seguente. L'insieme V è formato da un vertice per ogni variabile, un vertice per la negazione di ogni variabile, 5 vertici per ogni clausola e 3 vertici speciali: TRUE, FALSE e RED. Gli archi del grafo sono di due tipi: gli archi dei "letterali" che sono indipendenti dalle clausole e gli archi delle "clausole" che dipendono dalle clausole. Gli archi dei letterali formano un triangolo sui vertici speciali e anche su x_i, $\neg x_i$ e RED per $i = 1, 2, \ldots, n$.

d. Dimostrate che in una qualsiasi 3-colorazione c di un grafo che contiene gli archi dei letterali, se una variabile è colorata $c(\text{TRUE})$, la sua negazione è colorata $c(\text{FALSE})$, e viceversa. Dimostrate che per una qualsiasi assegnazione di verità per ϕ, esiste una 3-colorazione del grafo che contiene proprio gli archi dei letterali.

Acquistato da Michele Michele su Webster il 2022-07-07 12:10. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Il widget illustrato nella Figura 34.20 è utilizzato per imporre la condizione corrispondente a una clausola $(x \vee y \vee z)$. Ogni clausola richiede una copia unica dei 5 vertici che sono rappresentati in grigio scuro nella figura; questi vertici sono collegati ai letterali della clausola e al vertice speciale TRUE.

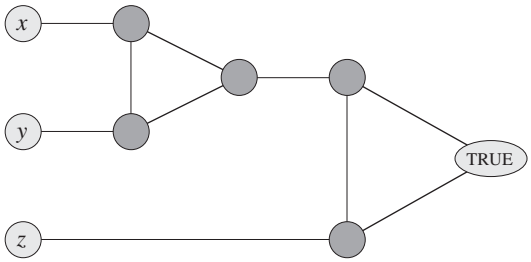


Figura 34.20 Il widget corrispondente alla clausola $(x \vee y \vee z)$ utilizzata nel Problema 34-3.

e. Dimostrate che, se ciascuna delle variabili x, y e z è colorata $c(\text{TRUE})$ o $c(\text{FALSE})$, allora il widget è 3-colorabile se e soltanto se almeno una delle variabili x, y e z è colorata $c(\text{TRUE})$.

f. Completate la dimostrazione che 3-COLOR è NP-completo.

34-4 Programmazione di lavori con premi e scadenze

Supponete di avere una macchina e un insieme di n lavori $a_1, a_2, \ldots, a_n$. Ogni lavoro a_j ha un tempo di esecuzione t_j, un premio p_j e una scadenza d_j. La macchina può eseguire un solo lavoro alla volta; un lavoro a_j deve essere eseguito ininterrottamente per t_j unità di tempo consecutive. Se completate il lavoro a_j entro la sua scadenza d_j, riceverete un premio p_j, ma se lo completate dopo la sua scadenza, perderete il premio. Come problema di ottimizzazione, conoscete i tempi di esecuzione, i premi e le scadenze per un insieme di n lavori, e volete trovare una programmazione che completa tutti i lavori con il massimo valore dei premi.

a. Formulate questo problema come un problema di decisione.

b. Dimostrate che il problema di decisione è NP-completo.

c. Create un algoritmo con tempo polinomiale per il problema di decisione, supponendo che tutti i tempi di esecuzione siano numeri interi compresi tra 1 ed n (*suggerimento:* utilizzate la programmazione dinamica).

d. Create un algoritmo con tempo polinomiale per il problema di ottimizzazione, supponendo che tutti i tempi di esecuzione siano numeri interi compresi tra 1 ed n.

Note

Il libro di Garey e Johnson [130] è un'ottima guida alla NP-completezza; tratta in modo approfondito la teoria e include un catalogo di vari problemi che nel 1979 erano noti come NP-completi. La dimostrazione del Teorema 34.13 è una versione adattata della loro dimostrazione e l'elenco delle aree in cui si presentano i problemi NP-completi, riportato all'inizio del Paragrafo 34.6, è tratto dall'indice del loro libro. Johnson ha scritto una serie di 23 colonne nel *Journal of Algorithms* tra il 1981 e il 1992, descrivendo i nuovi sviluppi nel campo della NP-completezza. Hopcroft, Motwani e Ullman [178], Lewis e Papadimitriou [237], Papadimitriou [271] e Sipser [318] hanno descritto molto bene la NP-completezza nel contesto della teoria della complessità. Anche Aho, Hopcroft e Ullman [5], Dasgupta, Papadimitriou e Vazirani [83], Johnsonbaugh e Schaefer [194], e Kleinberg e Tardos [209] hanno trattato la NP-completezza e hanno fornito numerose riduzioni.

continua

Acquistato da Michele Michelucci (mmichelucci@gmail.com) il 10/02/2022 con licenza Non cedibile a terzi - 10.15.0.16 - 20220210 - Copyright © 2022 Mondadori Education S.p.A.

La classe P fu introdotta nel 1964 da Cobham [73] e, in maniera indipendente, nel 1965 da Edmonds [101], che ha anche introdotto la classe NP e ipotizzato che P $\neq$ NP. Il concetto di NP-completezza fu proposto nel 1971 da Cook [76], che fornì le prime dimostrazioni della NP-completezza per la soddisfattibilità delle formule e la soddisfattibilità delle formule 3-CNF. Levin [235] ha scoperto in maniera indipendente il concetto, fornendo una dimostrazione della NP-completezza per un problema di tiling (copertura con piastrelle). Karp [200] ha introdotto la metodologia delle riduzioni nel 1972 e ha dimostrato la ricca varietà di problemi NP-completi. Il documento di Karp include le dimostrazioni originali della NP-completezza dei problemi della cricca, della copertura di vertici e del ciclo hamiltoniano. Da allora, molti ricercatori hanno dimostrato la NP-completezza di centinaia di problemi. Durante una riunione per festeggiare il sessantesimo compleanno di Karp nel 1995, Papadimitriou fece notare che circa 6000 documenti ogni anno riportavano il termine 'NP-completo' nei loro titoli, sommari o elenchi delle parole chiave e che ciò era più di quanto accadeva a ciascuno dei termini 'compilatore', 'database', 'esperto', 'rete neurale' o 'sistema operativo'.

Uno studio recente sulla teoria della complessità ha fatto luce sulla complessità del calcolo delle soluzioni approssimate; in questo studio viene data una nuova definizione di NP utilizzando "dimostrazioni verificabili in modo probabilistico". Questa nuova definizione implica che per problemi come quelli della cricca, della copertura di vertici, del commesso viaggiatore con la disuguaglianza triangolare e per molti altri, il calcolo delle soluzioni approssimate è NP-difficile e quindi non è più facile del calcolo delle soluzioni ottime. Un'introduzione a questo argomento si trova nella tesi di Arora [20], in un capitolo di Arora e Lund in [173], in un articolo di Arora [21], in un libro pubblicato da Mayr, Promel e Steger [247] e in un articolo di Johnson [192].

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Algoritmi di approssimazione 35

Molti problemi di interesse pratico purtroppo sono NP-completi, ma sono troppo importanti per rinunciarvi solamente perché non si riesce ad elaborare una loro soluzione ottima in tempo polinomiale. Se un problema è NP-completo, è poco probabile che riusciamo a trovare un algoritmo con tempo polinomiale che lo risolve con esattezza, ma nonostante ciò, non dobbiamo perdere tutte le speranze. Ci sono almeno tre approcci per affrontare la NP-completezza. In primo luogo, se gli input effettivi sono piccoli, un algoritmo con tempo di esecuzione esponenziale può fornire risultati molto soddisfacenti. In secondo luogo, potremmo riuscire a isolare importanti casi speciali che sono risolvibili in tempo polinomiale. In terzo luogo, potrebbe essere possibile trovare soluzioni *quasi ottime* in tempo polinomiale (nel caso peggiore o nel caso medio). Nella pratica, una soluzione quasi ottima spesso è accettabile. Un algoritmo che restituisce soluzioni quasi ottime è detto ***algoritmo di approssimazione***. Questo capitolo presenta gli algoritmi di approssimazione con tempo polinomiale per diversi problemi NP-completi.

Misura delle prestazioni degli algoritmi di approssimazione

Supponiamo di analizzare un problema di ottimizzazione in cui ogni possibile soluzione ha un costo positivo; vogliamo trovare una soluzione quasi ottima. A seconda del problema, una soluzione ottima può essere definita come una soluzione con il massimo costo possibile o con il minimo costo possibile; ovvero, il problema può essere un problema di massimizzazione o minimizzazione.

Diciamo che un algoritmo per un problema ha un ***rapporto di approssimazione*** $\rho(n)$ se, per un input qualsiasi di dimensione n, il costo C della soluzione prodotta dall'algoritmo è uguale al costo C^* di una soluzione ottima a meno di un fattore $\rho(n)$:

$$\max\left(\frac{C}{C^*}, \frac{C^*}{C}\right) \leq \rho(n) \tag{35.1}$$

Un algoritmo che garantisce un rapporto di approssimazione $\rho(n)$ è detto ***algoritmo $\rho(n)$-approssimato***. Le definizioni di rapporto di approssimazione e di algoritmo $\rho(n)$-approssimato si applicano a entrambi i problemi di minimizzazione e di massimizzazione. Per un problema di massimizzazione, si ha $0 < C \leq C^*$ e il rapporto C^*/C fornisce il fattore per il quale il costo di una soluzione ottima è maggiore del costo della soluzione approssimata. Analogamente, per un problema di minimizzazione, si ha $0 < C^* \leq C$ e il rapporto C/C^* fornisce il fattore per il quale il costo della soluzione approssimata è maggiore del costo di una soluzione ottima. Poiché si suppone che tutte le soluzioni abbiano un costo positivo, questi rapporti sono sempre ben definiti. Il rapporto di approssimazione di un algoritmo di approssimazione non è mai minore di 1, perché $C/C^* < 1$

Acquistato da Michele Michele Srl Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

implica $C^*/C > 1$. Di conseguenza, un algoritmo 1-approssimato[1] produce una soluzione ottima e un algoritmo di approssimazione con un grande rapporto di approssimazione potrebbe fornire una soluzione che è molto peggiore di quella ottima.

Per molti problemi, sono stati sviluppati algoritmi di approssimazione con tempo polinomiale che hanno piccoli rapporti di approssimazione costanti, mentre per altri problemi, i migliori algoritmi di approssimazione con tempo polinomiale che conosciamo hanno rapporti di approssimazione che crescono in funzione della dimensione dell'input n. Un esempio di tale problema è il problema della copertura di insiemi presentato nel Paragrafo 35.3.

Per alcuni problemi NP-completi sono stati sviluppati algoritmi di approssimazione con tempo polinomiale che possono raggiungere rapporti di approssimazione progressivamente più piccoli aumentando sempre di più il tempo di elaborazione. Questo significa che occorre un compromesso tra il tempo di elaborazione e la qualità dell'approssimazione. Un esempio è il problema della somma di sottoinsieme esaminato nel Paragrafo 35.5. Questa situazione è sufficientemente importante da meritarsi un nome tutto suo.

Uno ***schema di approssimazione*** per un problema di ottimizzazione è un algoritmo di approssimazione che riceve in input non soltanto un'istanza del problema, ma anche un valore $\epsilon > 0$ tale che, per ogni ϵ prefissato, lo schema è un algoritmo $(1+\epsilon)$-approssimato. Diciamo che uno schema di approssimazione è uno ***schema di approssimazione polinomiale*** se per ogni $\epsilon > 0$, lo schema viene eseguito in tempo polinomiale nella dimensione n della sua istanza di input.

Il tempo di esecuzione di uno schema di approssimazione polinomiale può aumentare molto rapidamente al diminuire di ϵ. Per esempio, il tempo di esecuzione di uno schema di approssimazione polinomiale potrebbe essere $O(n^{2/\epsilon})$. Teoricamente, se ϵ diminuisse di un fattore costante, il tempo di esecuzione per ottenere l'approssimazione desiderata non dovrebbe aumentare per più di un fattore costante (non necessariamente per lo stesso fattore costante di cui diminuisce ϵ).

Diciamo che uno schema di approssimazione è uno ***schema di approssimazione totalmente polinomiale*** se è uno schema di approssimazione e il suo tempo di esecuzione è polinomiale sia in $1/\epsilon$ sia nella dimensione n della sua istanza di input. Per esempio, lo schema potrebbe avere un tempo di esecuzione pari a $O((1/\epsilon)^2 n^3)$. Con un tale schema, qualsiasi diminuzione di ϵ di un fattore costante può essere ottenuta con un corrispondente aumento del tempo di esecuzione di un fattore costante.

Organizzazione del capitolo

I primi quattro paragrafi di questo capitolo presentano alcuni esempi di algoritmi di approssimazione con tempo polinomiale per problemi NP-completi; il quinto paragrafo descrive uno schema di approssimazione totalmente polinomiale. Il Paragrafo 35.1 inizia analizzando il problema della copertura di vertici, un problema di minimizzazione NP-completo che ha un algoritmo di approssimazione con un rapporto di approssimazione 2. Il Paragrafo 35.2 presenta un algoritmo di approssimazione con un rapporto di approssimazione 2 per il caso del proble-

Acquistato da Michele Michele su Webster il 2022-0[illegible] Vietata la riproduzione, anche parziale, con qualsiasi mezzo effettuata, compresa la fotocopia, anche ad uso interno o didattico

[1] Quando il rapporto di approssimazione non dipende da n, utilizzeremo i termini "rapporto di approssimazione ρ" e "algoritmo ρ-approssimato" per indicare l'indipendenza da n.

ma del commesso viaggiatore in cui la funzione costo soddisfa la disuguaglianza triangolare. Viene anche dimostrato che senza la disuguaglianza triangolare, per qualsiasi costante $\rho \geq 1$, non può esistere un algoritmo ρ-approssimato, a meno che P = NP. Nel Paragrafo 35.3 spieghiamo come utilizzare un metodo goloso per creare un algoritmo di approssimazione efficiente per il problema della copertura di insiemi, ottenendo una copertura il cui costo è, nel peggiore dei casi, più grande del costo ottimo per un fattore logaritmico. Il Paragrafo 35.4 presenta altri due algoritmi di approssimazione. Prima analizzeremo la versione di ottimizzazione della soddisfattibilità 3-CNF e forniremo un semplice algoritmo randomizzato che produce una soluzione con un rapporto di approssimazione atteso pari a $8/7$. Poi esamineremo una versione pesata del problema della copertura di vertici e dimostreremo come utilizzare la programmazione lineare per sviluppare un algoritmo 2-approssimato. Infine, il Paragrafo 35.5 presenta uno schema di approssimazione totalmente polinomiale per il problema della somma di sottoinsieme.

35.1 Il problema della copertura di vertici

Il problema della copertura di vertici è stato definito e provato NP-completo nel Paragrafo 34.6.2. Una ***copertura di vertici*** di un grafo non orientato $G = (V, E)$ è un sottoinsieme $V' \subseteq V$ tale che, se (u, v) è un arco di G, allora $u \in V'$ e\o $v \in V'$. La dimensione di una copertura di vertici è il numero di vertici che appartengono alla copertura.

Il ***problema della copertura di vertici*** consiste nel trovare una copertura di vertici di dimensione minima in un grafo non orientato. Una tale copertura di vertici è detta ***copertura di vertici ottima***. Questo problema è la versione di ottimizzazione di un problema di decisione NP-completo.

Sebbene possa essere difficile trovare una copertura di vertici ottima in un grafo G, tuttavia non è troppo difficile trovare una copertura di vertici che è quasi ottima. Il seguente algoritmo di approssimazione riceve in input un grafo non orientato G e restituisce una copertura di vertici la cui dimensione ha la garanzia di non essere più di due volte la dimensione di una copertura di vertici ottima.

```
APPROX-VERTEX-COVER(G)
1  C = ∅
2  E' = G.E
3  while E' ≠ ∅
4       sia (u, v) un arco arbitrario di E'
5       C = C ∪ {u, v}
6       rimuove da E' ogni arco incidente in u o v
7  return C
```

La Figura 35.1 illustra il funzionamento di APPROX-VERTEX-COVER. La variabile C contiene la copertura di vertici in costruzione. La riga 1 inizializza C con l'insieme vuoto. La riga 2 assegna a E' una copia dell'insieme degli archi $G.E$ del grafo. Il ciclo nelle righe 3–6 sceglie ripetutamente un arco (u, v) da E', aggiunge i suoi estremi u e v a C e cancella tutti gli archi in E' che sono coperti da u o da v. Il tempo di esecuzione di questo algoritmo è $O(V + E)$, se si usano le liste di adiacenza per rappresentare E'.

Acquistato da Michele Michele su Virasoft il 2022-07-07 20:12 Numero Ordine Libreria 1259998-1e-220718-0 Copyright © 2022, McGraw-Hill Education (Italy)

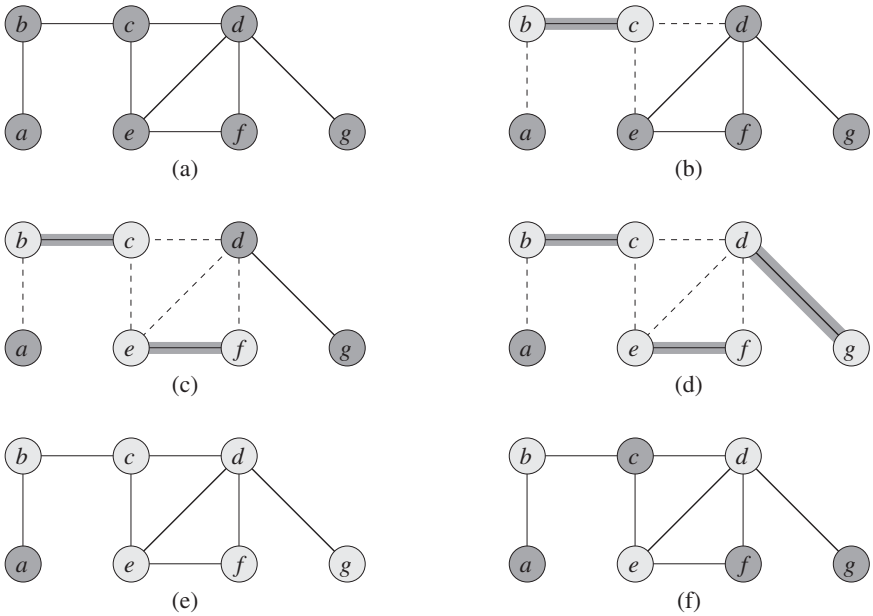


Figura 35.1
Il funzionamento della procedura APPROX-VERTEX-COVER. **(a)** Il grafo di input G, che ha 7 vertici e 8 archi. **(b)** L'arco (b, c), ombreggiato, è il primo arco scelto da APPROX-VERTEX-COVER. I vertici b e c, su sfondo grigio chiaro, vengono aggiunti all'insieme C che contiene la copertura di vertici che si sta creando. Gli archi (a, b), (c, e) e (c, d), ombreggiati, vengono rimossi perché adesso sono coperti da qualche vertice in C. **(c)** Viene scelto l'arco (e, f); i vertici e ed f vengono aggiunti a C. **(d)** Viene scelto l'arco (d, g); i vertici d e g vengono aggiunti a C. **(e)** L'insieme C, che è la copertura di vertici prodotta da APPROX-VERTEX-COVER, contiene i sei vertici b, c, d, e, f, g. **(f)** La copertura di vertici ottima per questo problema contiene soltanto tre vertici: b, d ed e.

Teorema 35.1
La procedura APPROX-VERTEX-COVER è un algoritmo 2-approssimato con tempo polinomiale.

Dimostrazione Abbiamo già dimostrato che la procedura APPROX-VERTEX-COVER viene eseguita in tempo polinomiale. L'insieme dei vertici C restituito da APPROX-VERTEX-COVER è una copertura di vertici, perché l'algoritmo ripete il ciclo finché tutti gli archi in $G.E$ non sono coperti da qualche vertice in C.

Per verificare che APPROX-VERTEX-COVER restituisce una copertura di vertici che è al più due volte la dimensione di una copertura ottima, indichiamo con A l'insieme degli archi che sono stati scelti nella riga 4 di APPROX-VERTEX-COVER. Per coprire gli archi in A, una qualsiasi copertura di vertici – in particolare, una copertura ottima C^* – deve includere almeno un estremo di ciascun arco in A. Due archi in A non possono avere un estremo in comune, perché dopo che un arco è stato scelto nella riga 4, tutti gli altri archi che sono incidenti nei suoi estremi vengono cancellati da E' nella riga 6. Quindi, non ci sono due archi in A che sono coperti dallo stesso vertice in C^*; pertanto si ha il seguente limite inferiore

$$|C^*| \geq |A| \tag{35.2}$$

per la dimensione di una copertura di vertici ottima. Ogni volta che viene eseguita la riga 4, viene scelto un arco i cui estremi non si trovano già in C; pertanto si ha un limite superiore (un limite superiore esatto, in effetti) per la dimensione della copertura di vertici prodotta:

$$|C| = 2|A| \tag{35.3}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Combinando le equazioni (35.2) e (35.3), si ottiene

$$\begin{aligned} |C| &= 2|A| \\ &\leq 2|C^*| \end{aligned}$$

che dimostra il teorema. ■

Riflettiamo su questa dimostrazione. Inizialmente qualcuno potrebbe chiedersi come sia possibile dimostrare che la dimensione della copertura di vertici restituita da APPROX-VERTEX-COVER sia al più due volte la dimensione di una copertura di vertici ottima, se non si conosce la dimensione della copertura ottima. La risposta è che noi utilizziamo un limite inferiore per la copertura ottima. Come l'Esercizio 35.1-2 chiede di dimostrare, l'insieme degli archi A che sono stati scelti nella riga 4 di APPROX-VERTEX-COVER è effettivamente un abbinamento massimale nel grafo G. (Un ***abbinamento massimale*** è un abbinamento che non è un sottoinsieme proprio di qualsiasi altro abbinamento.)

Come è stato detto nella dimostrazione del Teorema 35.1, la dimensione di un abbinamento massimale è un limite inferiore per la dimensione di una copertura di vertici ottima. L'algoritmo restituisce una copertura di vertici la cui dimensione è al più il doppio della dimensione dell'abbinamento massimale A. Mettendo in relazione la dimensione della soluzione restituita con il limite inferiore, otteniamo il nostro rapporto di approssimazione. Questa metodologia sarà utilizzata anche nei successivi paragrafi.

Esercizi

35.1-1

Trovate un esempio di grafo per il quale APPROX-VERTEX-COVER fornisce sempre una soluzione subottimale.

35.1-2

Sia A l'insieme degli archi che sono stati scelti nella riga 4 di APPROX-VERTEX-COVER. Dimostrate che l'insieme A è un abbinamento massimale nel grafo G.

35.1-3 ⋆

Il professor Nixon propone la seguente euristica per risolvere il problema della copertura di vertici. Selezionate ripetutamente un vertice con il grado più elevato ed eliminate tutti i suoi archi incidenti. Trovate un esempio per dimostrare che l'euristica del professore non ha un rapporto di approssimazione 2 (*suggerimento:* utilizzate un grafo bipartito con vertici di grado costante a sinistra e vertici di grado variabile a destra).

35.1-4

Create un algoritmo goloso efficiente che trova in tempo lineare una copertura di vertici ottima per un albero.

35.1-5

Dalla dimostrazione del Teorema 34.12, sappiamo che il problema della copertura di vertici e il problema NP-completo della cricca sono complementari nel senso che una copertura di vertici ottima è il complemento di una cricca di dimensione massima nel grafo complementare. Questa relazione implica che esiste un algoritmo di approssimazione con tempo polinomiale con un rapporto di approssimazione costante per il problema della cricca? Spiegate la vostra risposta.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

35.2 Il problema del commesso viaggiatore

Nel problema del commesso viaggiatore introdotto nel Paragrafo 34.6.4, dato un grafo completo non orientato $G = (V, E)$ che ha un costo intero non negativo $c(u, v)$ associato a ciascun arco $(u, v) \in E$, si vuole trovare un ciclo hamilto-

niano (un giro) di G con costo minimo. Come estensione della nostra notazione, indichiamo con $c(A)$ il costo totale degli archi nel sottoinsieme $A \subseteq E$:

$$c(A) = \sum_{(u,v) \in A} c(u, v)$$

In molte situazioni pratiche è sempre più economico andare direttamente da un posto u a un posto w; qualsiasi percorso con una tappa intermedia v non può essere meno costoso. In altri termini, eliminare una tappa intermedia non aumenta mai il costo. Formalizziamo questo concetto dicendo che la funzione costo c soddisfa la ***disuguaglianza triangolare*** se per ogni vertice $u, v, w \in V$, si ha

$$c(u, w) \leq c(u, v) + c(v, w)$$

La disuguaglianza triangolare è naturale e in molte applicazioni è automaticamente soddisfatta. Per esempio, se i vertici del grafo sono punti nel piano e il costo per spostarsi tra due vertici è la distanza euclidea tra di essi, allora la disuguaglianza triangolare è soddisfatta (ci sono molte funzioni costo diverse dalla distanza euclidea che soddisfano la disuguaglianza triangolare).

Come dimostra l'Esercizio 35.2-2, il problema del commesso viaggiatore è NP-completo anche se richiediamo che la funzione costo soddisfi la disuguaglianza triangolare. Quindi, è poco probabile che riusciamo a trovare un algoritmo con tempo polinomiale per risolvere questo problema con esattezza. Di conseguenza, cercheremo dei buoni algoritmi di approssimazione.

Nel Paragrafo 35.2.1 esaminiamo un algoritmo 2-approssimato per il problema del commesso viaggiatore con la disuguaglianza triangolare. Nel Paragrafo 35.2.2 dimostriamo che, senza la disuguaglianza triangolare, non esiste un algoritmo di approssimazione con tempo polinomiale con un rapporto di approssimazione costante, a meno che P = NP.

35.2.1 Il problema del commesso viaggiatore con la disuguaglianza triangolare

Applicando la metodologia del precedente paragrafo, calcoleremo prima una struttura – un albero di connessione minimo – il cui peso è un limite inferiore per la lunghezza di un giro ottimo del commesso viaggiatore. Poi utilizzeremo l'albero di connessione minimo per creare un giro il cui costo non è maggiore del doppio del peso dell'albero di connessione minimo, purché la funzione costo soddisfi la disuguaglianza triangolare. Il seguente algoritmo implementa questo approccio, chiamando come subroutine l'algoritmo MST-PRIM per gli alberi di connessione minimi (vedere il Paragrafo 23.2). Il parametro G è un grafo non orientato completo e la funzione costo c soddisfa la disuguaglianza triangolare.

APPROX-TSP-TOUR(G, c)

1 Seleziona un vertice $r \in G.V$ come "radice"
2 Calcola un albero di connessione minimo T per G dalla radice r utilizzando MST-PRIM(G, c, r)
3 Sia H una lista di vertici, ordinata in base a quando un vertice viene visitato per primo in un attraversamento anticipato di T
4 **return** il ciclo hamiltoniano H

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995035016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Ricordiamo dal Paragrafo 12.1 che l'attraversamento anticipato di un albero visita ricorsivamente ogni vertice dell'albero, elencando un vertice non appena lo incontra, prima di visitare uno qualsiasi dei suoi figli.

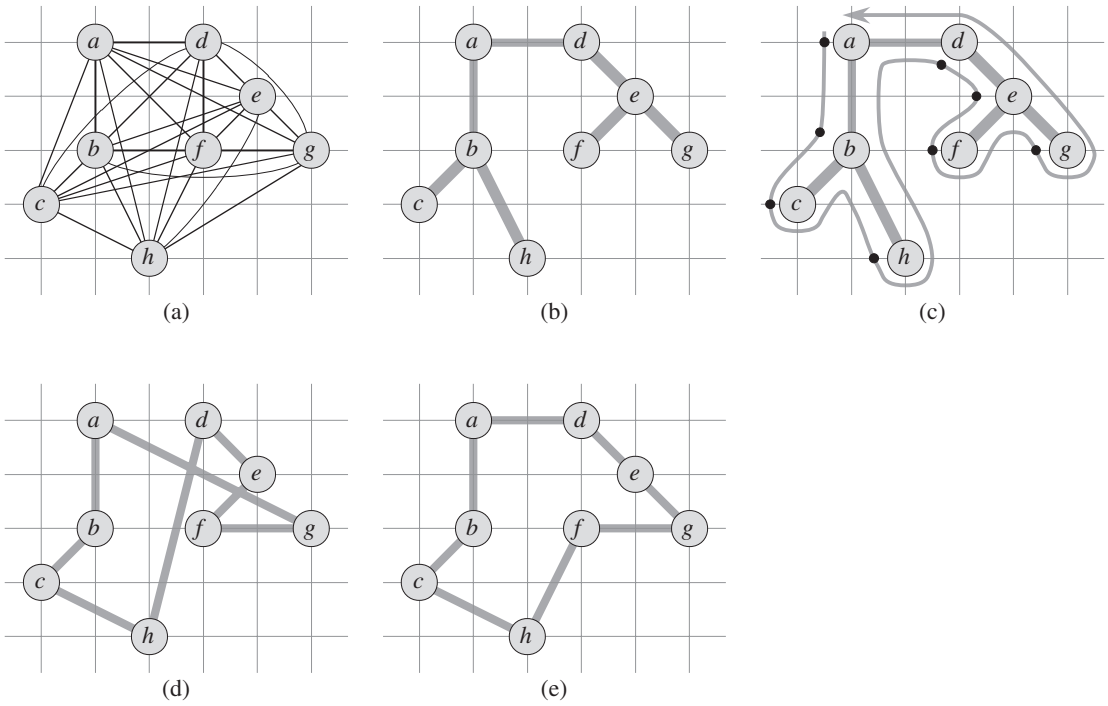


Figura 35.2 Il funzionamento di APPROX-TSP-TOUR. **(a)** Un grafo non orientato completo. L'insieme dei punti di input giacciono sui vertici di una griglia di interi. Per esempio, f è un'unità a destra e due unità in alto rispetto ad h. La distanza euclidea è utilizzata come funzione costo tra due punti. **(b)** Un albero di connessione minimo T di questi punti, come è calcolato da MST-PRIM. Il vertice a è la radice. I vertici sono etichettati nel modo in cui MST-PRIM li aggiunge in ordine alfabetico all'albero principale. **(c)** Un attraversamento di T, a partire da a. Un attraversamento completo dell'albero visita i vertici nel seguente ordine: $a, b, c, b, h, b, a, d, e, f, e, g, e, d, a$. Un attraversamento anticipato di T elenca un vertice la prima volta che lo incontra, come indica il punto accanto a ciascun vertice, generando l'ordinamento a, b, c, h, d, e, f, g. **(d)** Un giro di vertici ottenuto visitando i vertici nell'ordine dato dall'attraversamento anticipato. Questo è il giro H restituito da APPROX-TSP-TOUR. Il suo costo totale è approssimativamente 19,074. **(e)** Un giro ottimo H^* per un grafo non orientato completo. Il suo costo totale è approssimativamente 14,715.

La Figura 35.2 illustra il funzionamento di APPROX-TSP-TOUR. La parte (a) della figura mostra un grafo non orientato completo; la parte (b) illustra l'albero di connessione minimo T sviluppato a partire dalla radice a dall'algoritmo MST-PRIM. La parte (c) mostra come vengono visitati i vertici da un attraversamento anticipato di T; la parte (d) visualizza il giro corrispondente, che è il giro restituito da APPROX-TSP-TOUR. La parte (e) visualizza un giro ottimo, che è circa il 23% più corto.

Per l'Esercizio 23.2-2, anche con una semplice implementazione di MST-PRIM, il tempo di esecuzione di APPROX-TSP-TOUR è $\Theta(V^2)$. Adesso dimostriamo che, se la funzione costo per un'istanza del problema del commesso viaggiatore soddisfa la disuguaglianza triangolare, allora APPROX-TSP-TOUR restituisce un giro il cui costo non supera il doppio del costo di un giro ottimo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 1995030162207070. Copyright © 2022 McGraw-Hill Education (Italy)

Teorema 35.2

APPROX-TSP-TOUR è un algoritmo 2-approssimato con tempo polinomiale per il problema del commesso viaggiatore con la disuguaglianza triangolare.

Dimostrazione Abbiamo già dimostrato che l'algoritmo APPROX-TSP-TOUR viene eseguito in tempo polinomiale.

Indichiamo con H^* un giro ottimo per l'insieme dei vertici di input. Poiché cancellando un arco qualsiasi da un giro si ottiene un albero di connessione, il peso dell'albero di connessione minimo T è un limite inferiore per il costo di un giro ottimo, ovvero

$$c(T) \leq c(H^*) \tag{35.4}$$

Un ***attraversamento completo*** di T elenca i vertici quando li visita per la prima volta e ogni volta che li incontra dopo la visita di un sottoalbero. Chiamiamo questo attraversamento W. L'attraversamento completo del nostro esempio fornisce l'ordinamento

$$a, b, c, b, h, b, a, d, e, f, e, g, e, d, a$$

Poiché l'attraversamento completo passa per tutti gli archi di T esattamente due volte (estendendo la nostra definizione del costo c nella maniera naturale per gestire multiinsiemi di archi), si ha

$$c(W) = 2c(T) \tag{35.5}$$

Le equazioni (35.4) e (35.5) implicano che

$$c(W) \leq 2c(H^*) \tag{35.6}$$

e così il costo di W non supera il costo di un giro ottimo per un fattore 2.

Purtroppo, W di solito non è un giro, perché visita qualche vertice più di una volta. Per la disuguaglianza triangolare, tuttavia, possiamo cancellare da W una visita di un vertice qualsiasi, senza aumentare il costo. (Se un vertice v viene cancellato da W tra le visite di u e w, l'ordinamento risultante specifica di andare direttamente da u a w.) Applicando ripetutamente questa operazione, possiamo eliminare da W tutte le visite di un vertice, tranne la prima. Nell'esempio in esame, questo lascia l'ordinamento

$$a, b, c, h, d, e, f, g$$

Questo ordinamento è lo stesso di quello che si ottiene con un attraversamento anticipato dell'albero T. Sia H il ciclo che corrisponde a questo attraversamento anticipato. È un ciclo hamiltoniano, perché ogni vertice è visitato una sola volta e, in effetti, è il ciclo calcolato da APPROX-TSP-TOUR. Poiché H è ottenuto cancellando i vertici dall'attraversamento completo W, si ha

$$c(H) \leq c(W) \tag{35.7}$$

Combinando le disuguaglianze (35.6) e (35.7), si ottiene $c(H) \leq 2c(H^*)$, e questo completa la dimostrazione. ■

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Nonostante il buon rapporto di approssimazione fornito dal Teorema 35.2, APPROX-TSP-TOUR di solito non è la migliore soluzione pratica per questo problema. Ci sono altri algoritmi di approssimazione che tipicamente sono più efficienti nella pratica. Consultate i riferimenti bibliografici alla fine di questo capitolo.

35.2.2 Il problema generale del commesso viaggiatore

Se eliminiamo l'ipotesi che la funzione costo c soddisfi la disuguaglianza triangolare, non è possibile trovare dei buoni giri approssimati in tempo polinomiale, a meno che P = NP.

Teorema 35.3
Se P $\neq$ NP, allora per una costante qualsiasi $\rho \geq 1$, non esiste un algoritmo di approssimazione con tempo polinomiale con un rapporto di approssimazione ρ per il problema generale del commesso viaggiatore.

Dimostrazione La dimostrazione è per assurdo. Supponiamo per assurdo che per qualche numero $\rho \geq 1$ esista un algoritmo di approssimazione A con tempo polinomiale e rapporto di approssimazione ρ. Senza perdere in generalità, supponiamo che ρ sia un intero, arrotondandolo per eccesso se necessario. Poi dimostreremo come utilizzare A per risolvere in tempo polinomiale le istanze del problema del ciclo hamiltoniano (definito nel Paragrafo 34.3). Poiché il problema del ciclo hamiltoniano è NP-completo, per il Teorema 34.13, risolverlo in tempo polinomiale implica che P = NP, per il Teorema 34.4.

Sia $G = (V, E)$ un'istanza del problema del ciclo hamiltoniano. Vogliamo determinare in modo efficiente se G contiene un ciclo hamiltoniano utilizzando l'algoritmo di approssimazione A di cui abbiamo ipotizzato l'esistenza. Trasformiamo G in un'istanza del problema del commesso viaggiatore nel modo seguente. Sia $G' = (V, E')$ il grafo completo in V, cioè

$$E' = \{(u, v) : u, v \in V \text{ e } u \neq v\}$$

Assegniamo un costo intero a ciascun arco in E' nel modo seguente:

$$c(u, v) = \begin{cases} 1 & \text{se } (u, v) \in E \\ \rho|V| + 1 & \text{negli altri casi} \end{cases}$$

Le rappresentazioni di G' e c possono essere create da una rappresentazione di G in tempo polinomiale in $|E|$ e $|V|$.

Adesso consideriamo il problema del commesso viaggiatore (G', c). Se il grafo originale G ha un ciclo hamiltoniano H, allora la funzione costo c assegna a ciascun arco di H un costo 1, e quindi (G', c) contiene un giro di costo $|V|$. D'altra parte, se G non contiene un ciclo hamiltoniano, allora qualsiasi giro di G' deve utilizzare qualche arco che non appartiene a E. Ma qualsiasi giro che usa un arco che non appartiene a E ha un costo almeno pari a

$$\begin{aligned} (\rho|V| + 1) + (|V| - 1) &= \rho|V| + |V| \\ &> \rho|V| \end{aligned}$$

Poiché gli archi che non si trovano in G sono così costosi, c'è una differenza di almeno $\rho|V|$ tra il costo di un giro che è un ciclo hamiltoniano in G (costo $|V|$) e il costo di qualsiasi altro giro (costo minimo $\rho|V| + |V|$).

Acquistato da Michele Michelotto (michele.michelotto@pd.infn.it) - Licenza n. 1950401622707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Che cosa accade se applichiamo l'algoritmo di approssimazione A al problema del commesso viaggiatore (G', c)? Poiché è garantito che A restituisce un giro di costo non maggiore di ρ volte il costo di un giro ottimo, se G contiene un ciclo hamiltoniano, allora A deve restituirlo. Se G non ha un ciclo hamiltoniano, allora A restituisce un giro di costo maggiore di $\rho|V|$. Quindi, possiamo utilizzare A per risolvere il problema del ciclo hamiltoniano in tempo polinomiale. ■

La dimostrazione del Teorema 35.3 è un esempio di tecnica generale per dimostrare che un problema non può essere approssimato bene. Supponiamo che, dato un problema X NP-difficile, possiamo produrre in tempo polinomiale un problema di minimizzazione Y tale che le istanze "sì" di X corrispondano alle istanze di Y con valore al più k (per qualche k), ma che le istanze "no" di X corrispondano alle istanze di Y con valore maggiore di ρk. Allora abbiamo dimostrato che, a meno che P $=$ NP, non esiste un algoritmo ρ-approssimato con tempo polinomiale per il problema Y.

Esercizi

35.2-1
Supponete che un grafo completo non orientato $G = (V, E)$ con almeno 3 vertici abbia una funzione costo c che soddisfa la disuguaglianza triangolare. Dimostrate che $c(u, v) \geq 0$ per ogni $u, v \in V$.

35.2-2
Descrivete come trasformare in tempo polinomiale un'istanza del problema del commesso viaggiatore in un'altra istanza la cui funzione costo soddisfa la disuguaglianza triangolare. Le due istanze devono avere lo stesso insieme di giri ottimi. Spiegate perché una tale trasformazione in tempo polinomiale non contraddice il Teorema 35.3, supponendo che P $\neq$ NP.

35.2-3
Considerate la seguente ***euristica del punto più vicino*** per costruire un giro approssimato del commesso viaggiatore la cui funzione costo soddisfa la disuguaglianza triangolare. Iniziate con un ciclo banale formato da un solo vertice scelto arbitrariamente. A ogni passo, identificate il vertice u che non è nel ciclo, ma la cui distanza da un vertice qualsiasi del ciclo è minima. Supponete che il vertice del ciclo che è più vicino a u sia il vertice v. Estendete il ciclo per includere u, inserendolo subito dopo v. Ripetete l'operazione finché tutti i vertici non saranno nel ciclo. Dimostrate che questa euristica restituisce un giro il cui costo totale non è più del doppio del costo di un giro ottimo.

35.2-4
Il ***problema del commesso viaggiatore con collo di bottiglia*** consiste nel trovare il ciclo hamiltoniano tale che il costo del più costoso arco nel ciclo sia minimo. Supponendo che la funzione costo soddisfi la disuguaglianza triangolare, dimostrate che esiste un algoritmo di approssimazione con tempo polinomiale e rapporto di approssimazione 3 per questo problema. (*Suggerimento:* dimostrate ricorsivamente che è possibile visitare tutti i nodi in un albero di connessione collo di bottiglia, come descritto nel Problema 23-3, una sola volta effettuando un attraversamento completo dell'albero e saltando alcuni nodi, ma senza saltare più di due nodi intermedi consecutivi. Dimostrate che l'arco più costoso in un albero di connessione collo di bottiglia ha un costo che è al più il costo dell'arco più costoso in un ciclo hamiltoniano con collo di bottiglia.)

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995050162220707 © Copyright © 2022, McGraw-Hill Education (Italy)

35.2-5
Supponete che i vertici per un'istanza del problema del commesso viaggiatore siano punti nel piano e che il costo $c(u, v)$ sia la distanza euclidea tra i punti u e v. Dimostrate che un giro ottimo non incrocia mai sé stesso.

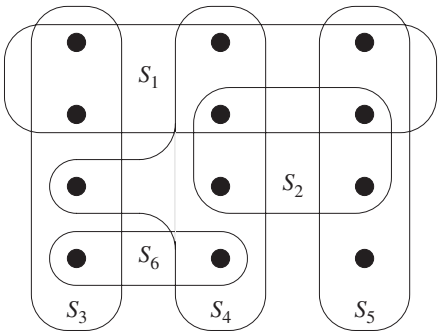


Figura 35.3
Un'istanza $(X, \mathcal{F})$ del problema della copertura di insiemi, dove X è formato dai 12 punti neri ed $\mathcal{F} = \{S_1, S_2, S_3, S_4, S_5, S_6\}$. Una copertura di dimensione minima è $\mathcal{C} = \{S_3, S_4, S_5\}$. L'algoritmo goloso produce una copertura di dimensione 4 selezionando nell'ordine gli insiemi S_1, S_4, S_5 ed S_3 o gli insiemi S_1, S_4, S_5 ed S_6.

35.3 Il problema della copertura di insiemi

Il problema della copertura di insiemi è un problema di ottimizzazione che è utilizzato per modellare vari problemi di assegnazione delle risorse. Il suo corrispondente problema di decisione generalizza il problema NP-completo della copertura di vertici e, quindi, è anche NP-difficile. Tuttavia, poiché l'algoritmo di approssimazione sviluppato per il problema della copertura di vertici non si può applicare, dobbiamo tentare altri approcci. Esamineremo una semplice euristica golosa con un rapporto di approssimazione logaritmico. Ovvero, al crescere della dimensione dell'istanza, la dimensione della soluzione approssimata può crescere rispetto alla dimensione di una soluzione ottima. Ciò nonostante, poiché la funzione logaritmo cresce molto lentamente, questo algoritmo di approssimazione può fornire risultati utili.

Un'istanza $(X, \mathcal{F})$ del ***problema della copertura di insiemi*** è formata da un insieme finito X e da una famiglia $\mathcal{F}$ di sottoinsiemi di X tali che ogni elemento di X appartenga almeno a un sottoinsieme in $\mathcal{F}$:

$$X = \bigcup_{S \in \mathcal{F}} S$$

Diciamo che un sottoinsieme $S \in \mathcal{F}$ ***copre*** i suoi elementi. Il problema consiste nel trovare un sottoinsieme $\mathcal{C} \subseteq \mathcal{F}$ di dimensione minima i cui membri coprono tutto X:

$$X = \bigcup_{S \in \mathcal{C}} S \tag{35.8}$$

Diciamo che qualsiasi $\mathcal{C}$ che soddisfa l'equazione (35.8) ***copre*** X. La Figura 35.3 illustra il problema della copertura di insiemi. La dimensione di $\mathcal{C}$ è definita come il numero di insiemi che contiene, anziché come il numero degli elementi contenuti in questi insiemi. Nella Figura 35.3 la copertura minima ha dimensione 3.

Il problema della copertura di insiemi è un'astrazione di molti fra i più comuni problemi combinatori. Come semplice esempio, supponiamo che X rappresenti un insieme di capacità che sono richieste per risolvere un problema; supponiamo inoltre di avere a disposizione un insieme di persone che possono studiare il problema. Vogliamo formare un gruppo di studio, con il minor numero di persone, tale che per ogni capacità richiesta in X, ci sia un elemento del gruppo che abbia tale capacità. Trattando il problema della copertura di insiemi come un problema di decisione, ci chiediamo se esiste oppure no una copertura con dimensione

Acquistato da Michele Mostarda (mostarda@gmail.com) - Copyright © 2005 The McGraw-Hill Companies, S.r.l. - Copyright © 2010 McGraw-Hill Education (Italy)

al più k, dove k è un parametro addizionale specificato nell'istanza del problema. Questa versione decisionale del problema è NP-completa, come l'Esercizio 35.3-2 chiede di dimostrare.

Un algoritmo di approssimazione goloso

Il metodo goloso funziona selezionando, in ogni fase, l'insieme S che copre il maggior numero di elementi che non sono ancora coperti.

```
GREEDY-SET-COVER(X, F)
1  U = X
2  C = ∅
3  while U ≠ ∅
4      seleziona un insieme S ∈ F che massimizza |S ∩ U|
5      U = U − S
6      C = C ∪ {S}
7  return C
```

Nell'esempio della Figura 35.3, GREEDY-SET-COVER aggiunge a $\mathcal{C}$, nell'ordine, gli insiemi S_1, S_4 ed S_5, seguiti da S_3 o S_6.

L'algoritmo funziona nel modo seguente. L'insieme U contiene, in ogni fase, l'insieme degli elementi che non sono ancora coperti. L'insieme $\mathcal{C}$ contiene la copertura in costruzione. La riga 4 è il passo in cui viene presa la decisione del metodo goloso. Viene scelto un sottoinsieme S che copre il maggior numero possibile di elementi non ancora coperti (le uguaglianze vengono risolte in modo arbitrario). Una volta selezionato S, i suoi elementi vengono rimossi da U ed S viene posto in $\mathcal{C}$. Quando l'algoritmo termina, l'insieme $\mathcal{C}$ contiene una sottofamiglia di $\mathcal{F}$ che copre X.

L'algoritmo GREEDY-SET-COVER può essere facilmente implementato per essere eseguito in tempo polinomiale in $|X|$ e $|\mathcal{F}|$. Poiché il numero di iterazioni del ciclo nelle righe 3–6 è limitato superiormente da $\min(|X|, |\mathcal{F}|)$ e il corpo del ciclo può essere implementato per essere eseguito nel tempo $O(|X|\,|\mathcal{F}|)$, esiste un'implementazione che viene eseguita nel tempo $O(|X|\,|\mathcal{F}|\min(|X|, |\mathcal{F}|))$. L'Esercizio 35.3-3 chiede di implementare un algoritmo con tempo lineare.

Analisi

Adesso dimostriamo che l'algoritmo goloso fornisce una copertura che non è molto più grande di una copertura ottima. Per comodità, in questo capitolo indicheremo con $H(d)$ il d-esimo numero armonico $H_d = \sum_{i=1}^{d} 1/i$ (vedere il Paragrafo A.1). Come condizione al contorno, definiamo $H(0) = 0$.

Teorema 35.4

GREEDY-SET-COVER è un algoritmo $\rho(n)$-approssimato con tempo polinomiale, dove

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\rho(n) = H(\max\{|S| : S \in \mathcal{F}\})$$

Dimostrazione Abbiamo già dimostrato che GREEDY-SET-COVER viene eseguito in tempo polinomiale.

Per dimostrare che GREEDY-SET-COVER è un algoritmo $\rho(n)$-approssimato, assegniamo un costo 1 a ciascun insieme selezionato dall'algoritmo, distribuiamo questo costo tra gli elementi coperti per la prima volta e poi utilizziamo questi costi per ottenere la relazione desiderata tra la dimensione di una copertura ottima $\mathcal{C}^*$ e la dimensione della copertura $\mathcal{C}$ fornita dall'algoritmo. Indichiamo con S_i l'i-esimo sottoinsieme selezionato da GREEDY-SET-COVER; l'algoritmo sostiene un costo 1 quando aggiunge S_i a $\mathcal{C}$. Distribuiamo questo costo per selezionare S_i equamente tra gli elementi coperti per la prima volta da S_i. Indichiamo con c_x il costo allocato all'elemento x, per ogni $x \in X$. A ogni elemento è assegnato un costo una sola volta, quando viene coperto per la prima volta. Se x è coperto per la prima volta da S_i, allora

$$c_x = \frac{1}{|S_i - (S_1 \cup S_2 \cup \cdots \cup S_{i-1})|}$$

A ogni passo dell'algoritmo viene assegnata una unità di costo, quindi

$$|\mathcal{C}| = \sum_{x \in X} c_x \tag{35.9}$$

Poiché ogni elemento $x \in X$ si trova almeno in un insieme nella copertura ottima $\mathcal{C}^*$, si ha

$$\sum_{S \in \mathcal{C}^*} \sum_{x \in S} c_x \geq \sum_{x \in X} c_x \tag{35.10}$$

Combininando le equazioni (35.9) e (35.10), si ha

$$|\mathcal{C}| \leq \sum_{S \in \mathcal{C}^*} \sum_{x \in S} c_x \tag{35.11}$$

La parte restante della dimostrazione si basa sulla seguente disuguaglianza fondamentale, che proveremo subito dopo. Per qualsiasi insieme S appartenente alla famiglia $\mathcal{F}$, si ha

$$\sum_{x \in S} c_x \leq H(|S|) \tag{35.12}$$

Dalle disuguaglianze (35.11) e (35.12) segue che

$$\begin{aligned} |\mathcal{C}| &\leq \sum_{S \in \mathcal{C}^*} H(|S|) \\ &\leq |\mathcal{C}^*| \cdot H(\max\{|S| : S \in \mathcal{F}\}) \end{aligned}$$

Questo dimostra il teorema.

Resta da provare la disuguaglianza (35.12). Consideriamo un insieme qualsiasi $S \in \mathcal{F}$ e $i = 1, 2, \ldots, |\mathcal{C}|$; sia

Acquistato da Michele Michele su Websta il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$u_i = |S - (S_1 \cup S_2 \cup \cdots \cup S_i)|$$

il numero di elementi in S che non sono stati ancora coperti dopo che gli insiemi $S_1, S_2, \ldots, S_i$ sono stati selezionati dall'algoritmo. Definiamo $u_0 = |S|$ come il numero degli elementi di S che inizialmente non sono coperti. Sia k l'indice più piccolo per cui $u_k = 0$, in modo che ogni elemento in S sia coperto da almeno uno degli insiemi $S_1, S_2, \ldots, S_k$ ma ci sia qualche elemento di S che non è coperto da

$S_1, S_2, \ldots, S_{k-1}$. Allora $u_{i-1} \geq u_i$ e $u_{i-1} - u_i$ elementi di S sono coperti per la prima volta da S_i, per $i = 1, 2, \ldots, k$. Quindi, si ha

$$\sum_{x \in S} c_x = \sum_{i=1}^{k} (u_{i-1} - u_i) \cdot \frac{1}{|S_i - (S_1 \cup S_2 \cup \cdots \cup S_{i-1})|}$$

Notate che

$$\begin{aligned} |S_i - (S_1 \cup S_2 \cup \cdots \cup S_{i-1})| &\geq |S - (S_1 \cup S_2 \cup \cdots \cup S_{i-1})| \\ &= u_{i-1} \end{aligned}$$

perché la scelta golosa di S_i garantisce che S non può coprire un numero di nuovi elementi maggiore di quelli che può coprire S_i (altrimenti sarebbe stato scelto S al posto di S_i). Di conseguenza, si ottiene

$$\sum_{x \in S} c_x \leq \sum_{i=1}^{k} (u_{i-1} - u_i) \cdot \frac{1}{u_{i-1}}$$

Adesso troviamo un limite per questa quantità nel modo seguente:

$$\begin{aligned} \sum_{x \in S} c_x &\leq \sum_{i=1}^{k} (u_{i-1} - u_i) \cdot \frac{1}{u_{i-1}} \\ &= \sum_{i=1}^{k} \sum_{j=u_i+1}^{u_{i-1}} \frac{1}{u_{i-1}} \\ &\leq \sum_{i=1}^{k} \sum_{j=u_i+1}^{u_{i-1}} \frac{1}{j} && \text{(perché } j \leq u_{i-1}\text{)} \\ &= \sum_{i=1}^{k} \left(\sum_{j=1}^{u_{i-1}} \frac{1}{j} - \sum_{j=1}^{u_i} \frac{1}{j} \right) \\ &= \sum_{i=1}^{k} (H(u_{i-1}) - H(u_i)) \\ &= H(u_0) - H(u_k) && \text{(perché gli altri termini della sommatoria si annullano a vicenda)} \\ &= H(u_0) - H(0) \\ &= H(u_0) && \text{(perché } H(0) = 0\text{)} \\ &= H(|S|) \end{aligned}$$

Questo completa la dimostrazione della disuguaglianza (35.12). ■

Corollario 35.5

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

GREEDY-SET-COVER è un algoritmo $(\ln |X| + 1)$-approssimato con tempo polinomiale.

Dimostrazione Utilizziamo la disuguaglianza (A.14) e il Teorema 35.4. ■

In alcune applicazioni $\max\{|S| : S \in \mathcal{F}\}$ è una costante piccola, quindi la soluzione restituita da GREEDY-SET-COVER è al più un piccolo numero di volte più grande di una soluzione ottima. Una di tali applicazioni si presenta quando

questa euristica è utilizzata per ottenere una copertura di vertici approssimata per un grafo i cui vertici hanno grado al più 3. In questo caso, la soluzione trovata da GREEDY-SET-COVER non è più grande di $H(3) = 11/6$ volte la soluzione ottima, una garanzia di prestazioni che è leggermente migliore di quella di APPROX-VERTEX-COVER.

Esercizi

35.3-1
Considerate ciascuna delle seguenti parole come un insieme di lettere: {`arid`, `dash`, `drain`, `heard`, `lost`, `nose`, `shun`, `slate`, `snare`, `thread`}. Trovate la copertura fornita da GREEDY-SET-COVER quando le uguaglianze sono risolte a favore della parola che appare per prima nel dizionario.

35.3-2
Dimostrate che la versione decisionale del problema della copertura di insiemi è NP-completa per riduzione dal problema della copertura di vertici.

35.3-3
Spiegate come implementare l'algoritmo GREEDY-SET-COVER in modo che possa essere eseguito nel tempo $O\left(\sum_{S \in \mathcal{F}} |S|\right)$.

35.3-4
Dimostrate che la seguente forma più debole del Teorema 35.4 è banalmente vera:

$$|\mathcal{C}| \leq |\mathcal{C}^*| \max\{|S| : S \in \mathcal{F}\}$$

35.3-5
GREEDY-SET-COVER può restituire un numero di soluzioni differenti, a seconda di come vengono risolte le uguaglianze nella riga 4. Create una procedura BAD-SET-COVER-INSTANCE(n) che restituisce un'istanza di n elementi del problema della copertura di insiemi per cui, a seconda di come vengono risolte le uguaglianze nella riga 4, GREEDY-SET-COVER può restituire un numero di soluzioni differenti che è esponenziale in n.

35.4 Randomizzazione e programmazione lineare

In questo paragrafo presenteremo due tecniche che sono utili nella progettazione degli algoritmi di approssimazione: la randomizzazione e la programmazione lineare. Descriveremo un semplice algoritmo randomizzato per una versione di ottimizzazione della soddisfattibilità 3-CNF; poi utilizzeremo la programmazione lineare per semplificare la progettazione di un algoritmo di approssimazione per una versione pesata del problema della copertura di vertici. Questo paragrafo è soltanto una presentazione superficiale di queste due potenti tecniche. Chi fosse interessato ad approfondire questi argomenti troverà alcuni riferimenti bibliografici nelle note alla fine di questo capitolo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Un algoritmo di approssimazione randomizzato per la soddisfattibilità MAX-3-CNF

Proprio come esistono algoritmi randomizzati che calcolano soluzioni esatte, così esistono algoritmi randomizzati che calcolano soluzioni approssimate. Diciamo che un algoritmo randomizzato per un problema ha un ***rapporto di approssimazione*** $\rho(n)$ se, per un input qualsiasi di dimensione n, il *costo atteso* C della solu-

zione prodotta dall'algoritmo randomizzato è uguale al costo C^* di una soluzione ottima a meno di un fattore $\rho(n)$:

$$\max\left(\frac{C}{C^*}, \frac{C^*}{C}\right) \leq \rho(n) \tag{35.13}$$

Un algoritmo randomizzato che garantisce un rapporto di approssimazione $\rho(n)$ è detto ***algoritmo randomizzato $\rho(n)$-approssimato***. In altre parole, un algoritmo di approssimazione randomizzato è come un algoritmo di approssimazione deterministico, con la differenza che il rapporto di approssimazione riguarda un valore atteso.

Una particolare istanza della soddisfattibilità 3-CNF, come è definita nel Paragrafo 34.5, può essere o no soddisfattibile. Per essere soddisfattibile, deve esistere un'assegnazione di valori per le variabili tale che ogni clausola risulti pari a 1. Se un'istanza non è soddisfattibile, possiamo calcolare quanto essa sia "vicina" a esserlo, ovvero possiamo calcolare un'assegnazione di valori per le variabili che soddisfa il maggior numero possibile di clausole. Il problema di massimizzazione risultante è detto ***soddisfattibilità MAX-3-CNF***. L'input per la soddisfattibilità MAX-3-CNF è lo stesso della soddisfattibilità 3-CNF, e l'obiettivo è restituire un'assegnazione per le variabili che massimizza il numero di clausole che risultano pari a 1. Adesso dimostriamo che assegnare casualmente a ciascuna variabile il valore 1 con probabilità $1/2$ e il valore 0 con probabilità $1/2$ è un algoritmo randomizzato $8/7$-approssimato. Secondo la definizione di soddisfattibilità 3-CNF (Paragrafo 34.5), richiediamo che ciascuna clausola sia formata esattamente da tre letterali distinti. Supponiamo inoltre che nessuna clausola possa contenere contemporaneamente una variabile e la sua negazione (l'Esercizio 35.4-1 chiede di rimuovere quest'ultima ipotesi).

Teorema 35.6
Data un'istanza di soddisfattibilità MAX-3-CNF con n variabili $x_1, x_2, \ldots, x_n$ ed m clausole, l'algoritmo randomizzato che assegna indipendentemente a ciascuna variabile il valore 1 con probabilità $1/2$ e il valore 0 con probabilità $1/2$ è un algoritmo randomizzato $8/7$-approssimato.

Dimostrazione Supponiamo di avere assegnato indipendentemente a ciascuna variabile il valore 1 con probabilità $1/2$ e il valore 0 con probabilità $1/2$. Per $i = 1, 2, \ldots, m$, definiamo la variabile casuale indicatrice

$$Y_i = \mathrm{I}\{\text{la clausola } i \text{ è soddisfatta}\}$$

in modo che $Y_i = 1$ se almeno uno dei letterali nella i-esima clausola vale 1. Poiché nessun letterale appare più di una volta nella stessa clausola e poiché abbiamo ipotizzato che una variabile e la sua negazione non possano apparire nella stessa clausola, i valori dei tre letterali in ogni clausola sono indipendenti. Una clausola non è soddisfatta soltanto se tutti e tre i suoi letterali sono posti a 0, e quindi $\Pr\{\text{la clausola } i \text{ non è soddisfatta}\} = (1/2)^3 = 1/8$. Di conseguenza, $\Pr\{\text{la clausola } i \text{ è soddisfatta}\} = 1 - 1/8 = 7/8$. Per il Lemma 5.1, quindi, $\mathrm{E}[Y_i] = 7/8$. Indichiamo con Y il numero totale di clausole soddisfatte, ovvero $Y = Y_1 + Y_2 + \cdots + Y_m$. Quindi si ha

$$\mathrm{E}[Y] = \mathrm{E}\left[\sum_{i=1}^{m} Y_i\right]$$

Acquistato da Michele Michele su Webster il 2022-07-23 a Mercatello sul Metauro (PU) - Vietata la copia e la distribuzione - © 2022 McGraw-Hill Education (Italy)

$$
\begin{aligned}
&= \sum_{i=1}^{m} \mathrm{E}[Y_i] \qquad \text{(per la linearità del valore atteso)} \\
&= \sum_{i=1}^{m} 7/8 \\
&= 7m/8
\end{aligned}
$$

Chiaramente m è un limite superiore per il numero di clausole soddisfatte, e quindi il rapporto di approssimazione è al più $m/(7m/8) = 8/7$. ■

Approssimare una copertura pesata di vertici utilizzando la programmazione lineare

Nel ***problema della copertura di vertici di peso minimo*** è dato un grafo non orientato $G = (V, E)$ in cui a ogni vertice $v \in V$ è associato un peso positivo $w(v)$. Per ogni copertura di vertici $V' \subseteq V$, definiamo il peso della copertura di vertici $w(V') = \sum_{v \in V'} w(v)$. L'obiettivo è trovare una copertura di vertici di peso minimo.

Non possiamo applicare l'algoritmo utilizzato per la copertura di vertici non pesata né possiamo utilizzare una soluzione casuale; entrambi i metodi possono fornire soluzioni che sono lontane da quella ottima. Calcoleremo invece un limite inferiore per il peso della copertura di vertici di peso minimo, utilizzando un programma lineare. Poi "arrotonderemo" questa soluzione e la utilizzeremo per ottenere una copertura di vertici.

Supponiamo di associare una variabile $x(v)$ a ogni vertice $v \in V$ e richiediamo che $x(v) \in \{0, 1\}$ per ogni $v \in V$. Se il vertice v appartiene alla copertura di vertici, scriveremo $x(v) = 1$; se il vertice v non appartiene alla copertura di vertici, scriveremo $x(v) = 0$. Allora il vincolo che, per un arco qualsiasi (u, v), almeno uno dei vertici u e v deve appartenere alla copertura di vertici può essere espresso come $x(u) + x(v) \geq 1$. Questo ci porta a definire il seguente ***programma intero 0-1*** per trovare una copertura di vertici di peso minimo:

$$\text{minimizzare} \qquad \sum_{v \in V} w(v)x(v) \tag{35.14}$$

con le condizioni

$$
\begin{aligned}
x(u) + x(v) &\geq 1 \quad \text{per ogni } (u, v) \in E && (35.15)\\
x(v) &\in \{0, 1\} \quad \text{per ogni } v \in V && (35.16)
\end{aligned}
$$

Nel caso speciale in cui tutti i pesi $w(v)$ siano uguali a 1, questa formulazione è la versione di ottimizzazione del problema NP-difficile della copertura di vertici. Supponiamo, invece, di rimuovere il vincolo che $x(v) \in \{0, 1\}$ e di sostituirlo con $0 \leq x(v) \leq 1$. Otteniamo il seguente programma lineare, che è noto come ***rilassamento nella programmazione lineare***:

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

$$\text{minimizzare} \qquad \sum_{v \in V} w(v)x(v) \tag{35.17}$$

con le condizioni

$$
\begin{aligned}
x(u) + x(v) &\geq 1 \quad \text{per ogni } (u, v) \in E && (35.18)\\
x(v) &\leq 1 \quad \text{per ogni } v \in V && (35.19)\\
x(v) &\geq 0 \quad \text{per ogni } v \in V && (35.20)
\end{aligned}
$$

Qualsiasi soluzione ammissibile del programma intero 0-1 nelle righe (35.14)–(35.16) è anche una soluzione ammissibile del programma lineare nelle righe (35.17)–(35.20). Di conseguenza, una soluzione ottima del programma lineare è un limite inferiore per la soluzione ottima del programma intero 0-1, e quindi un limite inferiore per una soluzione ottima del problema della copertura di vertici di peso minimo. La seguente procedura usa la soluzione del precedente programma lineare per costruire una soluzione approssimata del problema della copertura di vertici di peso minimo:

APPROX-MIN-WEIGHT-VC(G, w)

1 $C = \emptyset$
2 calcola $\bar{x}$, una soluzione ottima
  del programma lineare nelle righe (35.17)–(35.20)
3 **for** ogni $v \in V$
4 **if** $\bar{x}(v) \geq 1/2$
5 $C = C \cup \{v\}$
6 **return** C

La procedura APPROX-MIN-WEIGHT-VC funziona nel modo seguente. La riga 1 inizializza a zero la copertura di vertici. La riga 2 formula il programma lineare nelle righe (35.17)–(35.20) e poi risolve questo programma lineare. Una soluzione ottima assegna a ogni vertice v un valore associato $\bar{x}(v)$, dove $0 \leq \bar{x}(v) \leq 1$. Utilizziamo questo valore per guidare la scelta dei vertici da aggiungere alla copertura di vertici C nelle righe 3–5. Se $\bar{x}(v) \geq 1/2$, aggiungiamo il vertice v a C, altrimenti non lo aggiungiamo. In effetti, stiamo "arrotondando" a 0 o a 1 ogni variabile frazionaria nella soluzione del programma lineare, per ottenere una soluzione del programma intero 0-1 nelle righe (35.14)–(35.16). Infine, la riga 6 restituisce la copertura di vertici C.

Teorema 35.7
L'algoritmo APPROX-MIN-WEIGHT-VC è un algoritmo 2-approssimato con tempo polinomiale per il problema della copertura di vertici di peso minimo.

Dimostrazione Poiché esiste un algoritmo con tempo polinomiale per risolvere il programma lineare nella riga 2 e poiché il ciclo **for** nelle righe 3–5 viene eseguito in tempo polinomiale, APPROX-MIN-WEIGHT-VC è un algoritmo con tempo polinomiale. Dimostriamo che APPROX-MIN-WEIGHT-VC è un algoritmo 2-approssimato. Sia C^* una soluzione ottima del problema della copertura di vertici di peso minimo e sia z^* il valore di una soluzione ottima del programma lineare nelle righe (35.17)–(35.20). Poiché una copertura di vertici ottima è una soluzione ammissibile per il programma lineare, z^* deve essere un limite inferiore per $w(C^*)$, ovvero

$$z^* \leq w(C^*) \tag{35.21}$$

Acquistato da Michele Michele su Webster il 2022-0[illegible] Non riproducibile, distribuibile o trasmissibile. Copyright © 2022, The McGraw-Hill Education Italia

A questo punto, asseriamo che arrotondando i valori frazionari delle variabili $\bar{x}(v)$, otteniamo un insieme C che è una copertura di vertici e che soddisfa $w(C) \leq 2z^*$. Per verificare che C è una copertura di vertici, consideriamo un arco qualsiasi $(u, v) \in E$. Per il vincolo (35.18), sappiamo che $x(u) + x(v) \geq 1$; questo implica che almeno uno dei valori $\bar{x}(u)$ e $\bar{x}(v)$ è almeno pari a $1/2$. Di conseguenza, almeno uno dei vertici u e v sarà incluso nella copertura di vertici, e quindi ogni arco sarà coperto.

Consideriamo adesso il peso della copertura. Si ha

$$\begin{aligned} z^* &= \sum_{v \in V} w(v)\,\bar{x}(v) \\ &\geq \sum_{v \in V : \bar{x}(v) \geq 1/2} w(v)\,\bar{x}(v) \\ &\geq \sum_{v \in V : \bar{x}(v) \geq 1/2} w(v) \cdot \frac{1}{2} \\ &= \sum_{v \in C} w(v) \cdot \frac{1}{2} \\ &= \frac{1}{2} \sum_{v \in C} w(v) \\ &= \frac{1}{2} w(C) \end{aligned} \tag{35.22}$$

Combinando le disuguaglianze (35.21) e (35.22) si ottiene

$$w(C) \leq 2z^* \leq 2w(C^*)$$

Quindi APPROX-MIN-WEIGHT-VC è un algoritmo 2-approssimato. ■

Esercizi

35.4-1

Dimostrate che, anche se si consente a una clausola di contenere una variabile e la sua negazione, assegnare casualmente a ciascuna variabile il valore 1 con probabilità $1/2$ e il valore 0 con probabilità $1/2$ è ancora un algoritmo randomizzato $8/7$-approssimato.

35.4-2

Il ***problema della soddisfattibilità MAX-CNF*** è simile al problema della soddisfattibilità MAX-3-CNF, con la differenza che non limita ciascuna clausola ad avere esattamente 3 letterali. Create un algoritmo randomizzato 2-approssimato per il problema della soddisfattibilità MAX-CNF.

35.4-3

Nel problema MAX-CUT è dato un grafo non ordinato e non pesato $G = (V, E)$. Definiamo un taglio $(S, V - S)$ come nel Capitolo 23 e il ***peso*** di un taglio come il numero di archi che attraversano il taglio. L'obiettivo è trovare un taglio di peso massimo. Supponete di avere un algoritmo che, per ogni vertice v, inserisca in modo casuale e indipendente v in S con probabilità $1/2$ e in $V - S$ con probabilità $1/2$. Dimostrate che questo è un algoritmo randomizzato 2-approssimato.

35.4-4

Dimostrate che i vincoli nella riga (35.19) sono ridondanti nel senso che, se li escludete dal programma lineare nelle righe (35.17)–(35.20), qualsiasi soluzione ottima del programma lineare risultante deve soddisfare $x(v) \leq 1$ per ogni $v \in V$.

Acquistato da Michele Michele su Webster | 2022-07-07 23:12 | Numero Ordine Libreria 1995030162247070 | Copyright © 2022, McGraw-Hill Education (Italy)

35.5 Il problema della somma di sottoinsieme

Ricordiamo dal Paragrafo 34.6.5 che un'istanza del problema della somma di sottoinsieme è una coppia (S, t), dove S è un insieme $\{x_1, x_2, \ldots, x_n\}$ di interi positivi e t è un intero positivo. Questo problema di decisione chiede se esiste un

sottoinsieme di S tale che la somma dei suoi elementi sia esattamente pari al valore obiettivo t. Questo problema è NP-completo (vedere il Paragrafo 34.6.5). Il problema di ottimizzazione associato a questo problema di decisione si presenta in molte applicazioni pratiche. Nel problema di ottimizzazione, vogliamo trovare un sottoinsieme di $\{x_1, x_2, \ldots, x_n\}$ la cui somma sia la più grande possibile, ma non più grande di t. Per esempio, supponiamo di avere un autocarro che non può trasportare più di t quintali e di dovere spedire n pacchi differenti, l'i-esimo dei quali pesa x_i quintali. Vogliamo riempire l'autocarro con il massimo carico, senza superare il limite di peso t.

In questo paragrafo presentiamo un algoritmo con tempo esponenziale per questo problema di ottimizzazione e poi spieghiamo come modificare l'algoritmo in modo che diventi uno schema di approssimazione totalmente polinomiale (ricordiamo che uno schema di approssimazione totalmente polinomiale ha un tempo di esecuzione che è polinomiale sia in $1/\epsilon$ che nella dimensione dell'input).

Un algoritmo esatto con tempo esponenziale

Supponiamo di avere calcolato, per ciascun sottoinsieme S' di S, la somma degli elementi in S', e poi di avere selezionato, tra i sottoinsiemi la cui somma non supera t, quello la cui somma era più vicina a t. Chiaramente questo algoritmo restituirà la soluzione ottima, ma potrebbe richiedere un tempo esponenziale.

Per implementare questo algoritmo, potremmo utilizzare una procedura iterativa che, nell'iterazione i, calcola le somme di tutti i sottoinsiemi di $\{x_1, x_2, \ldots, x_i\}$, utilizzando come punto iniziale le somme di tutti i sottoinsiemi di $\{x_1, x_2, \ldots, x_{i-1}\}$. Così facendo, possiamo scoprire se un particolare sottoinsieme S' ha una somma maggiore di t; in tal caso, non ci sarebbe motivo di mantenerlo, perché nessun superinsieme di S' può essere la soluzione ottima. Adesso descriviamo un'implementazione di questa strategia. La procedura EXACT-SUBSET-SUM riceve in input l'insieme $S = \{x_1, x_2, \ldots, x_n\}$ e un valore obiettivo t; daremo il suo pseudocodice più avanti. Questa procedura calcola in modo iterativo L_i, la lista delle somme di tutti i sottoinsiemi di $\{x_1, \ldots, x_i\}$ che non superano t, e poi restituisce il valore massimo in L_n.

Se L è una lista di interi positivi e x è un altro intero positivo, allora indichiamo con $L + x$ la lista degli interi che si ottiene da L aumentando di x ogni elemento di L. Per esempio, se $L = \langle 1, 2, 3, 5, 9\rangle$, allora $L + 2 = \langle 3, 4, 5, 7, 11\rangle$. Noi utilizziamo questa notazione anche per gli insiemi, cosicché

$$S + x = \{s + x : s \in S\}$$

Utilizziamo anche una procedura ausiliaria MERGE-LISTS(L, L') che restituisce la lista ordinata che è l'unione delle sue due liste ordinate di input L ed L', con i valori duplicati rimossi. Come la procedura MERGE che abbiamo utilizzato in merge sort (Paragrafo 2.3.1), la procedura MERGE-LISTS viene eseguita nel tempo $O(|L| + |L'|)$. Omettiamo la pseudocodifica di MERGE-LISTS.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

```
EXACT-SUBSET-SUM(S, t)
1  n = |S|
2  L_0 = ⟨0⟩
3  for i = 1 to n
4       L_i = MERGE-LISTS(L_{i-1}, L_{i-1} + x_i)
5       elimina da L_i ogni elemento che è maggiore di t
6  return l'elemento massimo in L_n
```

Per spiegare il funzionamento di EXACT-SUBSET-SUM, indichiamo con P_i l'insieme di tutti i valori che possono essere ottenuti selezionando un sottoinsieme (eventualmente vuoto) di $\{x_1, x_2, \ldots, x_i\}$ e sommando i suoi elementi. Per esempio, se $S = \{1, 4, 5\}$, allora

$$\begin{aligned} P_1 &= \{0, 1\} \\ P_2 &= \{0, 1, 4, 5\} \\ P_3 &= \{0, 1, 4, 5, 6, 9, 10\} \end{aligned}$$

Data l'identità

$$P_i = P_{i-1} \cup (P_{i-1} + x_i) \tag{35.23}$$

possiamo provare per induzione su i (vedere l'Esercizio 35.5-1) che la lista L_i è una lista ordinata che contiene tutti gli elementi di P_i i cui valori non sono maggiori di t. Poiché la lunghezza di L_i può essere al più 2^i, EXACT-SUBSET-SUM è un algoritmo con tempo esponenziale in generale, sebbene sia un algoritmo con tempo polinomiale nei casi speciali in cui t sia polinomiale in $|S|$ o in cui tutti i numeri in S siano limitati da un polinomio in $|S|$.

Uno schema di approssimazione totalmente polinomiale

Possiamo ottenere uno schema di approssimazione totalmente polinomiale per il problema della somma di sottoinsieme "sfoltendo" ogni lista L_i dopo che è stata creata. L'idea è che se due valori in L sono vicini, allora ai fini di trovare una soluzione approssimata non c'è motivo di conservare esplicitamente entrambi i valori. Più precisamente, utilizziamo un parametro di sfoltimento δ tale che $0 < \delta < 1$. ***Sfoltire*** una lista L di δ significa rimuovere il maggior numero possibile di elementi da L, in modo che se L' è il risultato dello sfoltimento di L, allora per ogni elemento y che è stato rimosso da L, c'è un elemento z ancora in L' che approssima y, ovvero

$$\frac{y}{1+\delta} \le z \le y \tag{35.24}$$

L'elemento z può essere considerato come il "rappresentante" di y nella nuova lista L'. Ogni elemento rimosso y è rappresentato da un elemento rimasto z che soddisfa la disuguaglianza (35.24). Per esempio, se $\delta = 0.1$ ed

$$L = \langle 10, 11, 12, 15, 20, 21, 22, 23, 24, 29 \rangle$$

allora possiamo sfoltire L per ottenere

$$L' = \langle 10, 12, 15, 20, 23, 29 \rangle$$

dove il valore cancellato 11 è rappresentato da 10, i valori cancellati 21 e 22 sono rappresentati da 20 e il valore cancellato 24 è rappresentato da 23. Poiché ogni elemento della versione sfoltita della lista è anche un elemento della versione originale della lista, lo sfoltimento può ridurre notevolmente il numero degli elementi, anche se viene conservato un rappresentante con un valore vicino (e leggermente più piccolo) per ogni elemento cancellato.

Acquistato da Michele Mo[illegible] - Copyright © 2010 The McGraw-Hill Companies, S.r.l. - 2022 McGraw-Hill Education (Italy)

La seguente procedura sfoltisce la lista $L = \langle y_1, y_2, \ldots, y_m \rangle$ nel tempo $\Theta(m)$, supponendo che la lista L e il parametro δ siano noti e che L sia ordinata in ordine monotonicamente crescente. L'output della procedura è una lista ordinata sfoltita.

```
TRIM(L, δ)
1  Sia m la lunghezza di L
2  L' = ⟨y_1⟩
3  last = y_1
4  for i = 2 to m
5      if y_i > last · (1 + δ)        // y_i ≥ last perché L è ordinata
6          aggiunge y_i alla fine di L'
7          last = y_i
8  return L'
```

Gli elementi di L vengono esaminati in ordine monotonicamente crescente; un numero viene posto nella lista risultante L' soltanto se è il primo elemento di L oppure se non può essere rappresentato dall'ultimo numero che è stato inserito in L'. Data la procedura TRIM, possiamo costruire il nostro schema di approssimazione nel modo seguente. Questa procedura riceve in input un insieme $S = \{x_1, x_2, \ldots, x_n\}$ di n interi (in ordine arbitrario), un intero t (l'obiettivo) e un "parametro di approssimazione" ϵ, dove

$$0 < \epsilon < 1 \tag{35.25}$$

La procedura restituisce un elemento z il cui valore è uguale alla soluzione ottima a meno di un fattore $1 + \epsilon$.

```
APPROX-SUBSET-SUM(S, t, ε)
1  n = |S|
2  L_0 = ⟨0⟩
3  for i = 1 to n
4      L_i = MERGE-LISTS(L_{i-1}, L_{i-1} + x_i)
5      L_i = TRIM(L_i, ε/2n)
6      elimina da L_i ogni elemento che è maggiore di t
7  Sia z* il valore massimo in L_n
8  return z*
```

La riga 2 inizializza la lista L_0 come la lista che contiene soltanto l'elemento 0. Il ciclo **for** nelle righe 3–6 calcola L_i come una lista ordinata che contiene una versione opportunamente sfoltita dell'insieme P_i, eliminando tutti gli elementi maggiori di t. Poiché la lista L_i è ottenuta da L_{i-1}, dobbiamo assicurarci che lo sfoltimento ripetuto non introduca troppe inesattezze. Tra poco vedremo che APPROX-SUBSET-SUM restituisce un'approssimazione corretta, se esiste.

Come esempio, supponiamo di avere l'istanza

$$S = \langle 104, 102, 201, 101 \rangle$$

con $t = 308$ ed $\epsilon = 0,40$. Il parametro di sfoltimento δ è $\epsilon/8 = 0,05$. APPROX-SUBSET-SUM calcola i seguenti valori nelle righe indicate:

Acquistato da Michele Michele su Webster il 2022-0[illegible] Tutti i diritti riservati - McGraw-Hill Education (Italy)

riga 2: $L_0 = \langle 0 \rangle$

riga 4: $L_1 = \langle 0, 104 \rangle$

riga 5: $L_1 = \langle 0, 104 \rangle$

riga 6: $L_1 = \langle 0, 104 \rangle$

riga 4: $L_2 = \langle 0, 102, 104, 206 \rangle$
riga 5: $L_2 = \langle 0, 102, 206 \rangle$
riga 6: $L_2 = \langle 0, 102, 206 \rangle$

riga 4: $L_3 = \langle 0, 102, 201, 206, 303, 407 \rangle$
riga 5: $L_3 = \langle 0, 102, 201, 303, 407 \rangle$
riga 6: $L_3 = \langle 0, 102, 201, 303 \rangle$

riga 4: $L_4 = \langle 0, 101, 102, 201, 203, 302, 303, 404 \rangle$
riga 5: $L_4 = \langle 0, 101, 201, 302, 404 \rangle$
riga 6: $L_4 = \langle 0, 101, 201, 302 \rangle$

L'algoritmo restituisce $z^* = 302$ come soluzione, che si discosta molto meno del 40% dalla soluzione ottima $307 = 104 + 102 + 101$; in effetti, lo scostamento è minore del 2%.

Teorema 35.8
APPROX-SUBSET-SUM è uno schema di approssimazione totalmente polinomiale per il problema della somma di sottoinsieme.

Dimostrazione Le operazioni di sfoltimento di L_i nella riga 5 e la cancellazione da L_i di ogni elemento che è maggiore di t conservano la proprietà che ogni elemento di L_i è anche un elemento di P_i. Quindi, il valore z^* restituito nella riga 8 è davvero la somma di qualche sottoinsieme di S. Indichiamo con $y^* \in P_n$ una soluzione ottima del problema della somma di sottoinsieme. Allora, dalla riga 6 sappiamo che $z^* \leq y^*$. Per la disuguaglianza (35.1), dobbiamo provare che $y^*/z^* \leq 1 + \epsilon$. Dobbiamo provare inoltre che il tempo di esecuzione di questo algoritmo è polinomiale in $1/\epsilon$ e anche nella dimensione dell'input.

Come l'Esercizio 35.5-2 chiede di dimostrare, per ogni elemento y in P_i che è al più t, esiste un valore $z \in L_i$ tale che

$$\frac{y}{(1 + \epsilon/2n)^i} \leq z \leq y \tag{35.26}$$

La disuguaglianza (35.26) deve essere vera per $y^* \in P_n$; quindi esiste un elemento $z \in L_n$ tale che

$$\frac{y^*}{(1 + \epsilon/2n)^n} \leq z \leq y^*$$

e allora

$$\frac{y^*}{z} \leq \left(1 + \frac{\epsilon}{2n}\right)^n \tag{35.27}$$

Poiché esiste un elemento $z \in L_n$ che soddisfa la disuguaglianza (35.27), questa disuguaglianza deve essere vera per z^*, che è il massimo valore in L_n, ovvero

Acquistato da Michele Mastrogiovanni (mastrogiovanni.michele@gmail.com) - Proprietà letteraria riservata. Copyright © 2022 McGraw-Hill Education (Italy)

$$\frac{y^*}{z^*} \leq \left(1 + \frac{\epsilon}{2n}\right)^n \tag{35.28}$$

Resta da dimostrare che $y^*/z^* \leq 1+\epsilon$. Per farlo, dimostriamo che $(1 + \epsilon/2n)^n \leq 1 + \epsilon$. Per l'equazione (3.14), si ha $\lim_{n\to\infty}(1 + \epsilon/2n)^n = e^{\epsilon/2}$.

L'Esercizio 35.5-3 chiede di dimostrare che

$$\frac{d}{dn}\left(1+\frac{\epsilon}{2n}\right)^n > 0 \tag{35.29}$$

Quindi la funzione $(1+\epsilon/2n)^n$ aumenta con n quando si avvicina al suo limite $e^{\epsilon/2}$; quindi si ha

$$\begin{aligned}\left(1+\frac{\epsilon}{2n}\right)^n &\leq e^{\epsilon/2} \\ &\leq 1+\epsilon/2+(\epsilon/2)^2 \quad \text{(per la disuguaglianza (3.13))} \\ &\leq 1+\epsilon \quad \text{(per la disuguaglianza (35.25))}\end{aligned} \tag{35.30}$$

Combinando le disuguaglianze (35.28) e (35.30) si completa l'analisi del rapporto di approssimazione.

Per dimostrare che APPROX-SUBSET-SUM è uno schema di approssimazione totalmente polinomiale, calcoliamo un limite per la lunghezza di L_i. Dopo lo sfoltimento, gli elementi successivi z e z' di L_i devono avere la relazione $z'/z > 1+\epsilon/2n$. Ovvero, essi devono differire per un fattore di almeno $1+\epsilon/2n$. Di conseguenza, ogni lista contiene il valore 0, eventualmente il valore 1 e fino a $\lfloor \log_{1+\epsilon/2n} t \rfloor$ valori addizionali. Il numero di elementi in ogni lista L_i è al più

$$\begin{aligned}\log_{1+\epsilon/2n} t + 2 &= \frac{\ln t}{\ln(1+\epsilon/2n)} + 2 \\ &\leq \frac{2n(1+\epsilon/2n)\ln t}{\epsilon} + 2 \quad \text{(per la disuguaglianza (3.17))} \\ &\leq \frac{3n\ln t}{\epsilon} + 2 \quad \text{(per la disuguaglianza (35.25))}\end{aligned}$$

Questo vincolo è polinomiale nella dimensione dell'input – che è il numero di bit $\lg t$ richiesti per rappresentare t più il numero di bit richiesti per rappresentare l'insieme S, che a sua volta è polinomiale in n – e in $1/\epsilon$. Poiché il tempo di esecuzione di APPROX-SUBSET-SUM è polinomiale nelle lunghezze delle liste L_i, APPROX-SUBSET-SUM è uno schema di approssimazione totalmente polinomiale. ■

Esercizi

35.5-1

Dimostrate l'equazione (35.23). Poi dimostrate che, dopo l'esecuzione della riga 5 di EXACT-SUBSET-SUM, L_i è una lista ordinata che contiene tutti gli elementi di P_i i cui valori non superano t.

35.5-2

Dimostrate per induzione su i la disuguaglianza (35.26).

35.5-3

Dimostrate la disuguaglianza (35.29).

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

35.5-4

Come modifichereste lo schema di approssimazione presentato in questo paragrafo per trovare una buona approssimazione al valore minimo non minore di t che è una somma di qualche sottoinsieme di una data lista di input?

35.5-5

Modificate la procedura APPROX-SUBSET-SUM affinché restituisca anche il sottoinsieme di S la cui somma è z^*.

Problemi

35-1 Problema dell'impacchettamento

Supponiamo di avere un insieme di n oggetti, dove la dimensione s_i dell'i-esimo oggetto soddisfa la relazione $0 < s_i < 1$. Vogliamo impacchettare tutti gli oggetti utilizzando il minor numero possibile di pacchi di dimensione unitaria. Ogni pacco può contenere qualsiasi sottoinsieme di oggetti la cui dimensione totale non supera 1.

a. Dimostrate che il problema di determinare il numero minimo di pacchi richiesti è NP-difficile (*suggerimento:* riducete dal problema della somma di sottoinsieme).

L'euristica ***first-fit*** prende un oggetto alla volta e lo inserisce nel primo pacco che lo può contenere. Sia $S = \sum_{i=1}^{n} s_i$.

b. Dimostrate che il numero ottimo di pacchi richiesti è almeno $\lceil S \rceil$.

c. Dimostrate che l'euristica first-fit lascia al più un solo pacco pieno per meno della metà.

d. Dimostrate che il numero di pacchi utilizzati dall'euristica first-fit non è mai maggiore di $\lceil 2S \rceil$.

e. Dimostrate che l'euristica first-fit ha un rapporto di approssimazione 2.

f. Create una implementazione efficiente dell'euristica first-fit e analizzate il suo tempo di esecuzione.

35-2 Approssimare la dimensione di una cricca massima

Sia $G = (V, E)$ un grafo non orientato. Per ogni $k \geq 1$, definiamo $G^{(k)}$ come il grafo non orientato $(V^{(k)}, E^{(k)})$, dove $V^{(k)}$ è l'insieme di tutte le k-tuple ordinate di vertici in V ed $E^{(k)}$ è definito in modo che $(v_1, v_2, \ldots, v_k)$ sia adiacente a $(w_1, w_2, \ldots, w_k)$ se e soltanto se per ogni i, $1 \leq i \leq k$, il vertice v_i è adiacente a w_i in G oppure $v_i = w_i$.

a. Dimostrate che la dimensione della cricca massima in $G^{(k)}$ è uguale alla k-esima potenza della dimensione della cricca massima in G.

b. Dimostrate che, se esiste un algoritmo di approssimazione che ha un rapporto di approssimazione costante per trovare una cricca di dimensione massima, allora esiste uno schema di approssimazione totalmente polinomiale per il problema.

Acquistato da Michele Mazzucco su [illegible] 3016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

35-3 Problema della copertura pesata di insiemi

Supponiamo di generalizzare il problema della copertura di insiemi in modo che ogni insieme S_i nella famiglia $\mathcal{F}$ abbia un peso associato w_i e il peso di una copertura $\mathcal{C}$ sia $\sum_{S_i \in \mathcal{C}} w_i$. Vogliamo determinare una copertura di peso minimo (il Paragrafo 35.3 tratta il caso in cui $w_i = 1$ per ogni i).

Dimostrate che l'euristica golosa per la copertura di insiemi può essere generalizzata in una maniera naturale per ottenere una soluzione approssimata per

qualsiasi istanza del problema della copertura pesata di insiemi. Dimostrate che la vostra euristica ha un rapporto di approssimazione $H(d)$, dove d è la dimensione massima di un insieme qualsiasi S_i.

35-4 Abbinamento massimo

Ricordiamo che, per un grafo non orientato G, un abbinamento è un insieme di archi che non contiene due archi incidenti nello stesso vertice. Nel Paragrafo 26.3 abbiamo visto come trovare un abbinamento massimo in un grafo bipartito. In questo problema esamineremo gli abbinamenti nei grafi non orientati in generale (cioè non occorre che i grafi siano bipartiti).

a. Un ***abbinamento massimale*** è un abbinamento che non è un sottoinsieme proprio di qualsiasi altro abbinamento. Dimostrate che un abbinamento massimale non è necessariamente un abbinamento massimo, trovando un grafo non orientato G e un abbinamento massimale M in G che non è un abbinamento massimo (esiste un tale grafo con quattro vertici soltanto).

b. Considerate un grafo non orientato $G = (V, E)$. Create un algoritmo goloso con tempo $O(E)$ per trovare un abbinamento massimale in G.

In questo problema ci concentreremo su un algoritmo di approssimazione con tempo polinomiale per l'abbinamento massimo. Premesso che il più veloce algoritmo conosciuto per l'abbinamento massimo richiede un tempo superlineare (ma polinomiale), il nostro l'algoritmo di approssimazione qui sarà eseguito in tempo lineare. Voi dimostrerete che l'algoritmo goloso con tempo lineare per l'abbinamento massimale del punto (b) è un algoritmo 2-approssimato per l'abbinamento massimo.

c. Dimostrate che la dimensione di un abbinamento massimo in G è un limite inferiore per la dimensione di una copertura di vertici qualsiasi per G.

d. Considerate un abbinamento massimale M in $G = (V, E)$. Sia

$$T = \{v \in V : \text{qualche arco in } M \text{ è incidente in } v\}$$

Che cosa potreste dire sul sottografo di G indotto dai vertici di G che non si trovano in T?

e. Concludete dal punto (d) che $2|M|$ è la dimensione di una copertura di vertici per G.

f. Utilizzando i punti (c) ed (e), dimostrate che l'algoritmo goloso nel punto (b) è un algoritmo 2-approssimato per l'abbinamento massimo.

35-5 Schedulazione di macchine parallele

Nel ***problema della schedulazione di macchine parallele*** sono dati n lavori, $J_1, J_2, \ldots, J_n$, dove a ogni lavoro J_k è associato un tempo di elaborazione non negativo p_k. Sono date anche m macchine identiche, $M_1, M_2, \ldots, M_m$.

Acquistato da Michele Michele su Webster il 2022-0[illegible] Vietata la riproduzione non autorizzata - © 2010 The McGraw-Hill Education (Italy)

Una ***schedulazione*** specifica, per ogni lavoro J_k, la macchina sulla quale viene eseguito un lavoro e il periodo di tempo in cui il lavoro è eseguito. Ogni lavoro J_k deve essere eseguito in qualche macchina M_i per p_k unità di tempo consecutive e, durante questo periodo di tempo, nessun altro lavoro può essere eseguito su M_i. Indichiamo con C_k il ***tempo di completamento*** del lavoro J_k, ovvero il tempo in cui termina l'elaborazione del lavoro J_k.

Data una schedulazione, definiamo $C_{\max} = \max_{1 \le j \le n} C_j$ il ***tempo massimo di completamento*** (o ***makespan***) della schedulazione. L'obiettivo è trovare una schedulazione il cui makespan è minimo.

Per esempio, supponiamo di avere due macchine M_1 ed M_2 e quattro lavori J_1, J_2, J_3, J_4, con $p_1 = 2$, $p_2 = 12$, $p_3 = 4$ e $p_4 = 5$. Una possibile schedulazione esegue sulla macchina M_1 il lavoro J_1 seguito dal lavoro J_2 e sulla macchina M_2 il lavoro J_4 seguito dal lavoro J_3. Per questa schedulazione, $C_1 = 2$, $C_2 = 14$, $C_3 = 9$, $C_4 = 5$ e $C_{\max} = 14$. Una schedulazione ottima esegue J_2 sulla macchina M_1 e i lavori J_1, J_3 e J_4 sulla macchina M_2. Per questa schedulazione, $C_1 = 2$, $C_2 = 12$, $C_3 = 6$, $C_4 = 11$ e $C_{\max} = 12$.

Dato un problema di schedulazione di macchine parallele, indichiamo con $C^*_{\max}$ il makespan di una schedulazione ottima.

a. Dimostrate che il makespan ottimo è almeno grande quanto il tempo massimo di elaborazione, ovvero

$$C^*_{\max} \ge \max_{1 \le k \le n} p_k$$

b. Dimostrate che il makespan ottimo è almeno grande quanto il carico medio delle macchine, ovvero

$$C^*_{\max} \ge \frac{1}{m} \sum_{1 \le k \le n} p_k$$

Supponete di utilizzare il seguente algoritmo goloso per la schedulazione di macchine parallele: quando una macchina è inattiva, l'algoritmo programma un lavoro che non è stato ancora schedulato.

c. Scrivete lo pseudocodice per implementare questo algoritmo goloso. Qual è il tempo di esecuzione del vostro algoritmo?

d. Per la schedulazione fornita dall'algoritmo goloso, dimostrate che

$$C_{\max} \le \frac{1}{m} \sum_{1 \le k \le n} p_k + \max_{1 \le k \le n} p_k$$

Concludete che questo algoritmo è 2-approssimato con tempo polinomiale.

35-6 Approssimare un albero di copertura massimo

Sia $G = (V, E)$ un grafo non orientato con pesi degli archi distinti $w(u, v)$ in ogni arco $(u, v) \in E$. Per ogni vertice $v \in V$, sia $\max(v) = \max_{(u,v) \in E} \{w(u, v)\}$ l'arco di peso massimo che entra in un vertice. Sia $S_G = \{\max(v) : v \in V\}$ l'insieme degli archi di peso massimo che entrano in ciascun vertice; sia T_G l'albero di copertura di peso massimo di G, ovvero l'albero di copertura avente il peso massimo totale. Per ogni sottoinsieme di archi $E' \subseteq E$, definite $w(E') = \sum_{(u,v) \in E'} w(u, v)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995030l6-228707-0 Copyright © 2022, McGraw-Hill Education (Italy)

a. Indicate un esempio di un grafo con almeno 4 vertici per il quale $S_G = T_G$.

b. Indicate un esempio di un grafo con almeno 4 vertici per il quale $S_G \ne T_G$.

c. Dimostrate che $S_G \subseteq T_G$ per qualsiasi grafo G.

d. Dimostrate che $w(T_G) \geq w(S_G)/2$ per qualsiasi grafo G.

e. Create un algoritmo 2-approssimato con tempo $O(V + E)$ per calcolare un albero di copertura massimo.

35-7 Algoritmo di approssimazione per il problema dello zaino 0-1

Il problema dello zaino è descritto nel Paragrafo 16.2. Abbiamo n oggetti, dove l'i-esimo oggetto vale v_i dollari e pesa w_i libbre. Abbiamo anche uno zaino che non può sopportare un peso maggiore di W libbre. Qui, aggiungiamo le nuove condizioni che ciascun peso w_i sia al più W e che gli oggetti siano indicizzati in ordine monotonicamente decrescente dei loro valori: $v_1 \geq v_2 \geq \cdots \geq v_n$.

Nel problema dello zaino 0-1, vogliamo trovare un sottoinsieme di oggetti il cui peso totale sia al più W e il cui peso totale sia massimo. Il problema dello zaino frazionario è simile a quello dello zaino 0-1, con la differenza che è possibile prendere una frazione di ciascun oggetto, senza il vincolo di prendere o lasciare un oggetto intero. Se prendiamo una frazione x_i dell'oggetto i, dove $0 \leq x_i \leq 1$, incrementiamo di $x_i w_i$ il peso e di $x_i v_i$ il valore degli oggetti nello zaino. L'obiettivo è sviluppare un algoritmo 2-approssimato con tempo polinomiale per il problema dello zaino 0-1.

Per progettare un algoritmo con tempo polinomiale, consideriamo le istanze vincolate del problema dello zaino 0-1. Data un'istanza I del problema dello zaino, formiamo le istanze vincolate I_j, per $j = 1, 2, \ldots, n$, eliminando gli oggetti $1, 2, \ldots, j-1$ e imponendo che la soluzione includa l'oggetto j (tutto l'oggetto j sia nel problema dello zaino 0-1 sia in quello frazionario). Nessun oggetto viene eliminato nell'istanza I_1. Per l'istanza I_j, indichiamo con P_j una soluzione ottima del problema dello zaino 0-1 e con Q_j una soluzione ottima del problema dello zaino frazionario.

a. Dimostrate che una soluzione ottima dell'istanza I del problema dello zaino 0-1 appartiene all'insieme $\{P_1, P_2, \ldots, P_n\}$.

b. Dimostrate che è possibile trovare una soluzione ottima Q_j del problema dello zaino frazionario per l'istanza I_j, includendo l'oggetto j e poi utilizzando l'algoritmo goloso in cui, a ogni passo, prendiamo il maggior numero possibile di oggetti non selezionati nell'insieme $\{j+1, j+2, \ldots, n\}$ con il valore massimo per libbra v_i/w_i.

c. Dimostrate che è sempre possibile costruire una soluzione ottima Q_j del problema dello zaino frazionario per l'istanza I_j che include al più una frazione di oggetto. Ovvero, per tutti gli oggetti, tranne al più uno, includiamo o escludiamo gli oggetti interi.

d. Data una soluzione ottima Q_j del problema dello zaino frazionario per l'istanza I_j, formate la soluzione R_j da Q_j, eliminando tutti gli oggetti frazionari da Q_j. Indicate con $v(S)$ il valore totale degli oggetti di una soluzione S. Dimostrate che $v(R_j) \geq v(Q_j)/2 \geq v(P_j)/2$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995031016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

e. Create un algoritmo con tempo polinomiale che restituisce una soluzione del valore massimo dall'insieme $\{R_1, R_2, \ldots, R_n\}$, e dimostrate che il vostro è un algoritmo 2-approssimato con tempo polinomiale per il problema dello zaino 0-1.

Note

Sebbene i metodi che non necessariamente calcolano soluzioni esatte siano noti da migliaia di anni (per esempio, i metodi per approssimare il valore di π), il concetto di algoritmo di approssimazione è molto più recente. Hochbaum [173] attribuisce a Garey, Graham e Ullman [129] e Johnson [191] la formulazione del concetto di algoritmo di approssimazione con tempo polinomiale. Il primo algoritmo di approssimazione, di solito, è attribuito a Graham [150].

Dopo la pubblicazione di questo lavoro, sono stati sviluppati migliaia di algoritmi di approssimazione per una vasta gamma di problemi; su questo argomento c'è un'abbondante letteratura. I recenti libri di Ausiello e altri autori [26], Hochbaum [173] e Vazirani [346] trattano esclusivamente algoritmi di approssimazione, come gli articoli di Shmoys [316] e Klein e Young [208]. Molti altri testi, come Garey e Johnson [130] e Papadimitriou e Steiglitz [272], includono una buona trattazione degli algoritmi di approssimazione. Lawler, Lenstra, Rinnooy Kan e Shmoys [226] hanno studiato in modo particolare gli algoritmi di approssimazione per il problema del commesso viaggiatore.

Papadimitriou e Steiglitz attribuiscono l'algoritmo APPROX-VERTEX-COVER a F. Gavril e M. Yannakakis. Il problema della copertura di vertici è stato studiato a lungo (Hochbaum [173] elenca 16 differenti algoritmi di approssimazione per questo problema), ma tutti i rapporti di approssimazione sono almeno $2 - o(1)$.

L'algoritmo APPROX-TSP-TOUR appare in un documento di Rosenkrantz, Stearns e Lewis [299]. Christofides ha migliorato questo algoritmo e ha fornito un algoritmo $3/2$-approssimato per il problema del commesso viaggiatore con la disuguaglianza triangolare. Arora [22] e Mitchell [258] hanno dimostrato che, se i punti sono nel piano euclideo, esiste uno schema di approssimazione polinomiale. Il Teorema 35.3 è attribuito a Sahni e Gonzalez [302].

L'analisi dell'euristica golosa per il problema della copertura di insiemi si basa sulla dimostrazione di un metodo più generale pubblicata da Chvátal [69]; il metodo di base come è stato presentato qui è dovuto a Johnson [191] e Lovász [239].

L'algoritmo APPROX-SUBSET-SUM e la sua analisi si basano sugli algoritmi di approssimazione per i problemi dello zaino e della somma di sottoinsieme sviluppati da Ibarra e Kim [188].

Il Problema 35-7 è una versione combinatoria di un risultato più generale dei programmi di approssimazione creati da Bienstock e McClosky [45].

L'algoritmo randomizzato per la soddisfattibilità MAX-3-CNF è implicito nel lavoro di Johnson [191]. L'algoritmo per la copertura pesata di vertici è stato sviluppato da Hochbaum [172]. Il Paragrafo 35.4 accenna soltanto alla potenza della randomizzazione e della programmazione lineare nella progettazione degli algoritmi di approssimazione. La combinazione di questi due concetti ha dato origine a una tecnica detta "arrotondamento randomizzato" in cui un problema viene prima formulato come un programma lineare intero; poi viene risolto il rilassamento nella programmazione lineare e le variabili nella soluzione sono interpretate come probabilità. Queste probabilità sono poi utilizzate per agevolare la risoluzione del problema originale. I primi a utilizzare questa tecnica sono stati Raghavan e Thompson [291]; successivamente, la tecnica ha avuto molte altre applicazioni (consultate lo studio di Motwani, Naor e Raghavan [262]). Fra le numerose recenti idee sugli algoritmi di approssimazione figurano il metodo primale-duale (consultate [136]), il cercare tagli sparsi da utilizzare negli algoritmi divide et impera [230] e l'impiego della programmazione semidefinita [135].

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libraria: 899503010220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Come detto nelle note del Capitolo 34, le recenti scoperte nelle "dimostrazioni verificabili in modo probabilistico" hanno portato a limiti inferiori nell'approssimazione di vari problemi, inclusi molti di quelli descritti in questo capitolo. Oltre agli altri riferimenti bibliografici, il capitolo di Arora e Lund [23] contiene una buona descrizione della relazione tra le dimostrazioni verificabili in modo probabilistico e la difficoltà nell'approssimare vari problemi.

VIII Appendici: prerequisiti matematici

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Introduzione

L'analisi degli algoritmi spesso richiede l'impiego di un certo numero di strumenti matematici. Alcuni di questi strumenti sono semplici strumenti dell'algebra del liceo, ma altri potrebbero essere nuovi per voi. Nella Parte I abbiamo visto come manipolare le notazioni asintotiche e risolvere le ricorrenze. Queste appendici sono un compendio di molti altri concetti e metodi che vengono utilizzati per analizzare gli algoritmi. Come abbiamo notato nell'introduzione alla Parte I, è probabile che conosciate già molti degli argomenti descritti in queste appendici (sebbene le particolari notazioni da noi adottate possano differire in alcuni casi da quelle che conoscete). Quindi, potete considerare queste appendici come materiale di riferimento. Come nelle altre parti del libro, anche qui abbiamo incluso esercizi e problemi, in modo che possiate perfezionare le vostre tecniche in queste materie.

L'Appendice A descrive i metodi per calcolare e limitare le sommatorie, che si presentano spesso nell'analisi degli algoritmi. Molte delle formule in questa appendice si trovano nei testi di analisi matematica, tuttavia è più comodo poter disporre di questi metodi e formule in un solo libro.

L'Appendice B contiene le definizioni e le notazioni fondamentali per insiemi, relazioni, funzioni, grafi e alberi. L'appendice definisce anche alcune proprietà fondamentali di questi oggetti matematici.

L'Appendice C inizia con i principi elementari del calcolo combinatorio: permutazioni, combinazioni e così via. Il resto dell'appendice contiene definizioni e proprietà della teoria delle probabilità. L'analisi della maggior parte degli algoritmi descritti in questo libro non richiede il calcolo delle probabilità, quindi inizialmente potete ignorare gli ultimi paragrafi dell'appendice. Successivamente, quando incontrerete un'analisi probabilistica che vorrete capire meglio, potrete consultare l'Appendice C, che è stata appositamente organizzata per tale scopo.

L'Appendice D definisce le matrici, le loro operazioni e alcune delle loro proprietà fondamentali. Probabilmente avrete già studiato molti di questi argomenti in un corso di algebra lineare; tuttavia per vostra comodità abbiamo riunito in questa appendice le nostre notazioni e definizioni.

Acquistato da Michele Michele su Webster il 2022-07-07 28:12 - Numero Ordine Libreria: 1995|3016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

A Sommatorie

Quando un algoritmo contiene un costrutto di controllo iterativo, come un ciclo **while** o **for**, il suo tempo di esecuzione può essere espresso come la sommatoria dei tempi impiegati in ogni esecuzione del corpo del ciclo. Per esempio, nel Paragrafo 2.2 abbiamo visto che la j-esima iterazione dell'algoritmo insertion sort richiedeva un tempo proporzionale a j nel caso peggiore. Sommando il tempo impiegato in ciascuna iterazione, abbiamo ottenuto la sommatoria (o serie)

$$\sum_{j=2}^{n} j$$

Calcolando questa sommatoria, abbiamo ottenuto un limite $\Theta(n^2)$ per il tempo di esecuzione dell'algoritmo nel caso peggiore. Questo esempio indica l'importanza generale di saper manipolare e limitare le sommatorie.

Il Paragrafo A.1 elenca varie formule che riguardano le sommatorie. Il Paragrafo A.2 descrive delle utili tecniche per limitare le sommatorie. Le formule nel Paragrafo A.1 sono date senza dimostrazione. Alcune di queste formule sono dimostrate nel Paragrafo A.2 per illustrare i metodi descritti in questo paragrafo. Molte delle altre dimostrazioni possono essere trovate nei testi di analisi matematica.

A.1 Formule di sommatoria e loro proprietà

Data una sequenza di numeri $a_1, a_2, \ldots$, la somma finita $a_1 + a_2 + \cdots + a_n$, dove n è un intero non negativo, può essere scritta

$$\sum_{k=1}^{n} a_k$$

Se $n = 0$, il valore della sommatoria per definizione è 0. Il valore di una serie finita è sempre ben definito e i suoi termini possono essere sommati in qualsiasi ordine.

Data una sequenza di numeri $a_1, a_2, \ldots$, la somma infinita $a_1 + a_2 + \cdots$ può essere scritta

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\sum_{k=1}^{\infty} a_k$$

che può essere interpretata così

$$\lim_{n \to \infty} \sum_{k=1}^{n} a_k$$

Se il limite non esiste, la serie ***diverge***, altrimenti ***converge***. I termini di una serie convergente non sempre possono essere sommati in qualsiasi ordine. Tuttavia, è possibile riarrangiare i termini di una ***serie assolutamente convergente***, cioè una serie $\sum_{k=1}^{\infty} a_k$ per cui anche la serie $\sum_{k=1}^{\infty} |a_k|$ converge.

Linearità

Per qualsiasi numero reale c e due sequenze finite $a_1, a_2, \ldots, a_n$ e $b_1, b_2, \ldots, b_n$, si ha

$$\sum_{k=1}^{n}(ca_k + b_k) = c\sum_{k=1}^{n} a_k + \sum_{k=1}^{n} b_k$$

La proprietà della linearità si applica anche alle serie convergenti infinite.

La proprietà della linearità può essere utilizzata per manipolare le sommatorie che includono termini espressi nella notazione asintotica. Per esempio

$$\sum_{k=1}^{n} \Theta(f(k)) = \Theta\left(\sum_{k=1}^{n} f(k)\right)$$

In questa equazione, la notazione Θ sul lato sinistro si applica alla variabile k, ma sul lato destro si applica a n. Tali manipolazioni possono essere applicate anche alle serie convergenti infinite.

Serie aritmetiche

La sommatoria

$$\sum_{k=1}^{n} k = 1 + 2 + \cdots + n$$

è una ***serie aritmetica***, il cui valore è

$$\sum_{k=1}^{n} k = \frac{1}{2}n(n+1) \tag{A.1}$$

$$= \Theta(n^2) \tag{A.2}$$

Sommatorie di quadrati e cubi

Abbiamo le seguenti sommatorie di quadrati e cubi:

$$\sum_{k=0}^{n} k^2 = \frac{n(n+1)(2n+1)}{6} \tag{A.3}$$

$$\sum_{k=0}^{n} k^3 = \frac{n^2(n+1)^2}{4} \tag{A.4}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Serie geometriche

Per un numero reale $x \neq 1$, la sommatoria

$$\sum_{k=0}^{n} x^k = 1 + x + x^2 + \cdots + x^n$$

è una ***serie geometrica*** o ***di potenze***, il cui valore è

$$\sum_{k=0}^{n} x^k = \frac{x^{n+1} - 1}{x - 1} \tag{A.5}$$

Quando la sommatoria è infinita e $|x| < 1$, si ha la serie geometrica decrescente infinita

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1 - x} \tag{A.6}$$

Serie armoniche

Per n intero positivo, l'n-esimo ***numero armonico*** è

$$\begin{aligned} H_n &= 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \cdots + \frac{1}{n} \\ &= \sum_{k=1}^{n} \frac{1}{k} \\ &= \ln n + O(1) \end{aligned} \tag{A.7}$$

Dimostreremo questo limite nel Paragrafo A.2.

Integrazione e differenziazione delle serie

È possibile ottenere altre formule integrando e derivando le precedenti formule. Per esempio, derivando entrambi i lati della serie geometrica infinita (A.6) e moltiplicando per x, si ha

$$\sum_{k=0}^{\infty} k x^k = \frac{x}{(1 - x)^2} \tag{A.8}$$

per $|x| < 1$

Riduzione delle serie

Per una sequenza qualsiasi $a_0, a_1, \ldots, a_n$, si ha

$$\sum_{k=1}^{n} (a_k - a_{k-1}) = a_n - a_0 \tag{A.9}$$

perché ciascuno dei termini $a_1, a_2, \ldots, a_{n-1}$ una volta viene sommato e una volta viene sottratto. Diciamo che la sommatoria si ***riduce***. Analogamente, si ha

$$\sum_{k=0}^{n-1} (a_k - a_{k+1}) = a_0 - a_n$$

Come esempio di riduzione di sommatoria, consideriamo la serie

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\sum_{k=1}^{n-1} \frac{1}{k(k+1)}$$

Poiché possiamo riscrivere ciascun termine come

$$\frac{1}{k(k+1)} = \frac{1}{k} - \frac{1}{k+1}$$

otteniamo

$$\begin{aligned}\sum_{k=1}^{n-1} \frac{1}{k(k+1)} &= \sum_{k=1}^{n-1} \left(\frac{1}{k} - \frac{1}{k+1}\right) \\ &= 1 - \frac{1}{n}\end{aligned}$$

Produttorie

Il prodotto finito $a_1 a_2 \cdots a_n$ può essere scritto

$$\prod_{k=1}^{n} a_k$$

Se $n = 0$, il valore della produttoria per definizione è 1. Possiamo convertire una formula con una produttoria in una formula con una sommatoria utilizzando l'identità

$$\lg\left(\prod_{k=1}^{n} a_k\right) = \sum_{k=1}^{n} \lg a_k$$

Esercizi

A.1-1
Trovate una semplice formula per $\sum_{k=1}^{n}(2k-1)$.

A.1-2 ⋆
Dimostrate che $\sum_{k=1}^{n} 1/(2k-1) = \ln(\sqrt{n}) + O(1)$ manipolando la serie armonica.

A.1-3
Dimostrate che $\sum_{k=0}^{\infty} k^2 x^k = x(1+x)/(1-x)^3$ per $0 < |x| < 1$.

A.1-4 ⋆
Dimostrate che $\sum_{k=0}^{\infty}(k-1)/2^k = 0$.

A.1-5 ⋆
Calcolate la sommatoria $\sum_{k=1}^{\infty}(2k+1)x^{2k}$.

A.1-6
Dimostrate che $\sum_{k=1}^{n} O(f_k(n)) = O\left(\sum_{k=1}^{n} f_k(n)\right)$ utilizzando la proprietà della linearità delle sommatorie.

A.1-7
Calcolate la produttoria $\prod_{k=1}^{n} 2 \cdot 4^k$.

A.1-8 ⋆
Calcolate la produttoria $\prod_{k=2}^{n}(1 - 1/k^2)$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

A.2 Limitare le sommatorie

Ci sono varie tecniche che possono essere utilizzate per limitare le sommatorie che descrivono i tempi di esecuzione degli algoritmi. Ecco alcuni dei metodi più frequentemente utilizzati.

Induzione matematica

Il metodo più elementare per calcolare il valore di una serie consiste nell'utilizzare l'induzione matematica. Come esempio, dimostriamo che il valore della serie aritmetica $\sum_{k=1}^{n} k$ è $\frac{1}{2}n(n+1)$. Possiamo facilmente verificare ciò per $n = 1$; quindi facciamo l'ipotesi induttiva che ciò sia vero per n e dimostriamo che è vero per $n+1$. Si ha

$$\begin{aligned}\sum_{k=1}^{n+1} k &= \sum_{k=1}^{n} k + (n+1) \\ &= \frac{1}{2}n(n+1) + (n+1) \\ &= \frac{1}{2}(n+1)(n+2)\end{aligned}$$

Per utilizzare l'induzione matematica non è necessario ipotizzare il valore esatto di una sommatoria. L'induzione può essere utilizzata anche per dimostrare un limite. Come esempio, dimostriamo che la serie geometrica $\sum_{k=0}^{n} 3^k$ vale $O(3^n)$. Più precisamente, dimostriamo che $\sum_{k=0}^{n} 3^k \leq c3^n$ per qualche costante c. Per la condizione iniziale $n = 0$, si ha $\sum_{k=0}^{0} 3^k = 1 \leq c \cdot 1$ se $c \geq 1$. Supponendo che il limite sia vero per n, dimostriamo che è vero per $n+1$. Si ha

$$\begin{aligned}\sum_{k=0}^{n+1} 3^k &= \sum_{k=0}^{n} 3^k + 3^{n+1} \\ &\leq c3^n + 3^{n+1} && \text{(per l'ipotesi induttiva)} \\ &= \left(\frac{1}{3} + \frac{1}{c}\right) c3^{n+1} \\ &\leq c3^{n+1}\end{aligned}$$

se $(1/3 + 1/c) \leq 1$ ovvero $c \geq 3/2$. Quindi, $\sum_{k=0}^{n} 3^k = O(3^n)$, come volevamo dimostrare.

Dobbiamo stare attenti quando utilizziamo la notazione asintotica per dimostrare i limiti per induzione. Consideriamo la seguente dimostrazione falsa che $\sum_{k=1}^{n} k = O(n)$. Certamente, $\sum_{k=1}^{1} k = O(1)$. Supponendo che il limite sia vero per n, adesso possiamo provarlo per $n+1$:

$$\begin{aligned}\sum_{k=1}^{n+1} k &= \sum_{k=1}^{n} k + (n+1) \\ &= O(n) + (n+1) && \Longleftarrow \textit{sbagliato!} \\ &= O(n+1)\end{aligned}$$

L'errore sta nel fatto che la "costante" nascosta dalla notazione "O grande" cresce con n e quindi non è costante. Non abbiamo dimostrato che la stessa costante va bene per *ogni* n.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Limitare i termini

A volte, un buon limite superiore per una serie può essere ottenuto limitando ogni termine della serie; spesso è sufficiente utilizzare il termine più grande per limitare gli altri.

Per esempio, un limite superiore rapido per la serie aritmetica (A.1) è

$$\begin{aligned}\sum_{k=1}^{n} k &\leq \sum_{k=1}^{n} n \\ &= n^2\end{aligned}$$

In generale, per una serie $\sum_{k=1}^{n} a_k$, se poniamo $a_{\max} = \max_{1 \leq k \leq n} a_k$, allora

$$\sum_{k=1}^{n} a_k \leq n a_{\max}$$

La tecnica di limitare ciascun termine di una serie con il termine più grande non è efficiente quando la serie può essere limitata con una serie geometrica. Data la serie $\sum_{k=0}^{n} a_k$, supponiamo che $a_{k+1}/a_k \leq r$ per ogni $k \geq 0$, dove r è una costante $0 < r < 1$. La sommatoria può essere limitata da una serie geometrica decrescente infinita, in quanto $a_k \leq a_0 r^k$, e quindi

$$\begin{aligned}\sum_{k=0}^{n} a_k &\leq \sum_{k=0}^{\infty} a_0 r^k \\ &= a_0 \sum_{k=0}^{\infty} r^k \\ &= a_0 \frac{1}{1-r}\end{aligned}$$

Possiamo applicare questo metodo per limitare la sommatoria $\sum_{k=1}^{\infty}(k/3^k)$. Per iniziare la sommatoria da $k = 0$, riscriviamola così $\sum_{k=0}^{\infty}((k+1)/3^{k+1})$. Il primo termine (a_0) è $1/3$ e il rapporto (r) dei termini consecutivi è

$$\begin{aligned}\frac{(k+2)/3^{k+2}}{(k+1)/3^{k+1}} &= \frac{1}{3} \cdot \frac{k+2}{k+1} \\ &\leq \frac{2}{3}\end{aligned}$$

per ogni $k \geq 0$. Quindi, si ha

$$\begin{aligned}\sum_{k=1}^{\infty} \frac{k}{3^k} &= \sum_{k=0}^{\infty} \frac{k+1}{3^{k+1}} \\ &\leq \frac{1}{3} \cdot \frac{1}{1-2/3} \\ &= 1\end{aligned}$$

Un tipico errore nell'applicare questo metodo sta nel dimostrare che il rapporto dei termini consecutivi è minore di 1 e quindi nel supporre che la sommatoria sia limitata da una serie geometrica. Un esempio è la serie armonica infinita, che diverge perché

$$\begin{aligned}\sum_{k=1}^{\infty} \frac{1}{k} &= \lim_{n \to \infty} \sum_{k=1}^{n} \frac{1}{k} \\ &= \lim_{n \to \infty} \Theta(\lg n) \\ &= \infty\end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Il rapporto tra il $(k+1)$-esimo e il k-esimo termine in questa serie è $k/(k+1) < 1$, ma la serie non è limitata da una serie geometrica decrescente. Per limitare una serie con una serie geometrica, bisogna dimostrare che esiste una *costante* $r < 1$ tale che il rapporto di tutte le coppie di termini consecutivi non supera mai r. Nella serie armonica, non esiste una tale costante r perché il rapporto si avvicina in modo arbitrario a 1.

Scomposizione delle sommatorie

Una tecnica per trovare i limiti di una sommatoria complessa consiste nell'esprimere la serie come la somma di due o più serie, scomponendo l'intervallo di variazione dell'indice e poi limitando ciascuna delle serie risultanti. Per esempio, se volessimo calcolare un limite inferiore per la serie aritmetica $\sum_{k=1}^{n} k$ (che, come abbiamo dimostrato, ha un limite superiore pari a n^2), potremmo tentare di limitare ciascun termine della sommatoria con il termine più piccolo, ma poiché questo termine è 1, otterremmo un limite inferiore pari a n per la sommatoria – ben lontano dal nostro limite superiore n^2.

Possiamo ottenere un limite inferiore migliore scomponendo prima la sommatoria. Supponiamo per comodità che n sia pari. Si ha

$$\begin{aligned}
\sum_{k=1}^{n} k &= \sum_{k=1}^{n/2} k + \sum_{k=n/2+1}^{n} k \\
&\geq \sum_{k=1}^{n/2} 0 + \sum_{k=n/2+1}^{n} (n/2) \\
&= (n/2)^2 \\
&= \Omega(n^2)
\end{aligned}$$

che è un limite asintoticamente stretto, in quanto $\sum_{k=1}^{n} k = O(n^2)$.

Per una sommatoria che deriva dall'analisi di un algoritmo, spesso è possibile scomporre la sommatoria e ignorare un numero costante di termini iniziali. Generalmente, questa tecnica si applica quando ogni termine a_k di una sommatoria $\sum_{k=0}^{n} a_k$ è indipendente da n. Allora, per qualsiasi costante $k_0 > 0$, possiamo scrivere

$$\begin{aligned}
\sum_{k=0}^{n} a_k &= \sum_{k=0}^{k_0-1} a_k + \sum_{k=k_0}^{n} a_k \\
&= \Theta(1) + \sum_{k=k_0}^{n} a_k
\end{aligned}$$

perché i termini iniziali della sommatoria sono tutti costanti e c'è un numero costante di questi termini. A questo punto, possiamo utilizzare altri metodi per limitare $\sum_{k=k_0}^{n} a_k$. Questa tecnica si può applicare anche alle sommatorie infinite. Per esempio, per trovare un limite asintotico superiore per

$$\sum_{k=0}^{\infty} \frac{k^2}{2^k}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:11 Numero Ordine Libreria 1[illegible]. Copyright © 2022, McGraw-Hill Education (Italy)

osserviamo che il rapporto dei termini consecutivi è

$$\begin{aligned}\frac{(k+1)^2/2^{k+1}}{k^2/2^k} &= \frac{(k+1)^2}{2k^2} \\ &\leq \frac{8}{9}\end{aligned}$$

se $k \geq 3$. Quindi, la sommatoria può essere scomposta in

$$\begin{aligned}\sum_{k=0}^{\infty}\frac{k^2}{2^k} &= \sum_{k=0}^{2}\frac{k^2}{2^k} + \sum_{k=3}^{\infty}\frac{k^2}{2^k} \\ &\leq \sum_{k=0}^{2}\frac{k^2}{2^k} + \frac{9}{8}\sum_{k=0}^{\infty}\left(\frac{8}{9}\right)^k \\ &= O(1)\end{aligned}$$

perché la prima sommatoria ha un numero costante di termini e la seconda sommatoria è una serie geometrica decrescente.

La tecnica della scomposizione delle sommatorie può essere utilizzata per determinare i limiti asintotici in situazioni molto più difficili. Per esempio, possiamo ottenere un limite $O(\lg n)$ per la serie armonica (A.7):

$$H_n = \sum_{k=1}^{n}\frac{1}{k}$$

L'idea è quella di scomporre l'intervallo da 1 a n in $\lfloor \lg n \rfloor + 1$ parti e limitare superiormente a 1 il contributo di ciascuna parte. Ogni parte è formata dai termini che iniziano da $1/2^i$ e arrivano fino a $1/2^{i+1}$ escluso. Siccome l'ultima parte può contenere dei termini che non stavano nella serie armonica iniziale, si ottiene

$$\begin{aligned}\sum_{k=1}^{n}\frac{1}{k} &\leq \sum_{i=0}^{\lfloor \lg n \rfloor}\sum_{j=0}^{2^i-1}\frac{1}{2^i+j} \\ &\leq \sum_{i=0}^{\lfloor \lg n \rfloor}\sum_{j=0}^{2^i-1}\frac{1}{2^i} \\ &= \sum_{i=0}^{\lfloor \lg n \rfloor} 1 \\ &\leq \lg n + 1 \qquad \text{(A.10)}\end{aligned}$$

Approssimazione con integrali

Quando una sommatoria può essere espressa come $\sum_{k=m}^{n} f(k)$, dove $f(k)$ è una funzione monotonicamente crescente, è possibile approssimarla con gli integrali:

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995[illegible]10-2207076 Copyright © 2022, McGraw-Hill Education (Italy)

$$\int_{m-1}^{n} f(x)\,dx \leq \sum_{k=m}^{n} f(k) \leq \int_{m}^{n+1} f(x)\,dx \qquad \text{(A.11)}$$

La giustificazione di questa approssimazione è illustrata nella Figura A.1. La sommatoria è rappresentata dall'area dei rettangoli e l'integrale è la regione grigia sotto la curva.

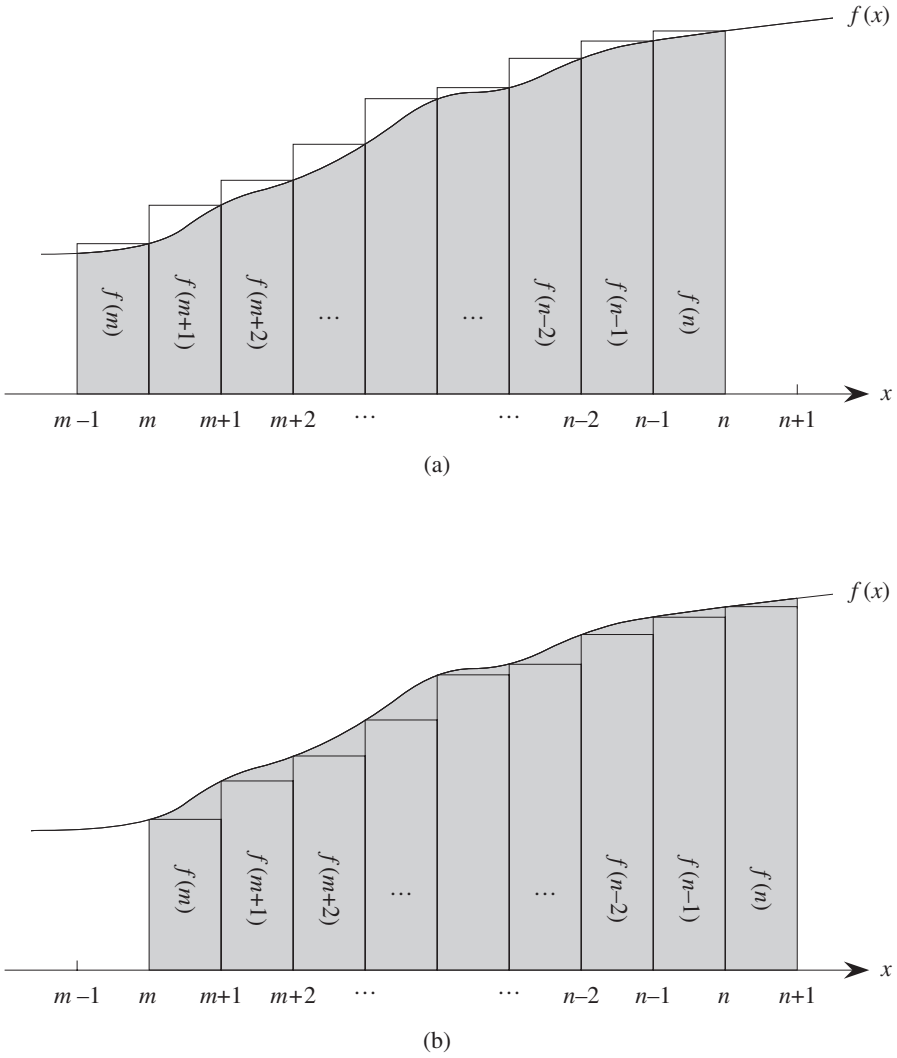


Figura A.1
Approssimazione di $\sum_{k=m}^{n} f(k)$ mediante integrali. All'interno di ogni rettangolo è indicata la sua area; l'area totale dei rettangoli rappresenta il valore della sommatoria. L'integrale è rappresentato dall'area grigia sotto la curva. Confrontando le aree in **(a)**, si ottiene $\int_{m-1}^{n} f(x)\,dx \leq \sum_{k=m}^{n} f(k)$; poi facendo scorrere i rettangoli di un'unità verso destra, si ottiene $\sum_{k=m}^{n} f(k) \leq \int_{m}^{n+1} f(x)\,dx$ in **(b)**.

Quando $f(k)$ è una funzione monotonicamente decrescente, possiamo utilizzare un metodo analogo per ottenere i seguenti limiti

$$\int_{m}^{n+1} f(x)\,dx \leq \sum_{k=m}^{n} f(k) \leq \int_{m-1}^{n} f(x)\,dx \tag{A.12}$$

L'approssimazione con integrali (A.12) fornisce una stima stretta dell'n-esimo numero armonico. Per un limite inferiore, otteniamo

$$\begin{aligned} \sum_{k=1}^{n} \frac{1}{k} &\geq \int_{1}^{n+1} \frac{dx}{x} \\ &= \ln(n+1) \end{aligned} \tag{A.13}$$

Per il limite superiore, deriviamo la disuguaglianza

$$\begin{aligned} \sum_{k=2}^{n} \frac{1}{k} &\leq \int_{1}^{n} \frac{dx}{x} \\ &= \ln n \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

che fornisce il limite

$$\sum_{k=1}^{n} \frac{1}{k} \leq \ln n + 1 \tag{A.14}$$

Esercizi

A.2-1
Dimostrate che $\sum_{k=1}^{n} 1/k^2$ è limitata superiormente da una costante.

A.2-2
Trovate un limite asintotico superiore per la sommatoria

$$\sum_{k=0}^{\lfloor \lg n \rfloor} \lceil n/2^k \rceil$$

A.2-3
Dimostrate che l'n-esimo numero armonico è $\Omega(\lg n)$ scomponendo la sommatoria.

A.2-4
Approssimate $\sum_{k=1}^{n} k^3$ con un integrale.

A.2-5
Perché non utilizziamo l'approssimazione con integrali (A.12) direttamente su $\sum_{k=1}^{n} 1/k$ per ottenere un limite superiore per l'n-esimo numero armonico?

Problemi

A-1 Limitare le sommatorie
Calcolate dei limiti asintotici stretti per le seguenti sommatorie. Supponete che $r \geq 0$ ed $s \geq 0$ siano costanti.

a. $\displaystyle\sum_{k=1}^{n} k^r$

b. $\displaystyle\sum_{k=1}^{n} \lg^s k$

c. $\displaystyle\sum_{k=1}^{n} k^r \lg^s k$

Acquistato da Michele Michela Su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Note

Knuth [210] è un eccellente testo di consultazione per il materiale presentato in questo capitolo. Le proprietà fondamentali delle serie si trovano nei testi di analisi matematica, come Apostol [18] o Thomas e altri [335].

B Insiemi e altro

Molti capitoli di questo libro descrivono solo superficialmente gli argomenti della matematica discreta. Questa appendice tratta in modo più esauriente le notazioni, le definizioni e le proprietà fondamentali degli insiemi, delle relazioni, delle funzioni, dei grafi e degli alberi. I lettori che conoscono già questi argomenti hanno bisogno soltanto di leggere rapidamente questa appendice.

B.1 Insiemi

Un ***insieme*** è una collezione di oggetti distinguibili, detti ***elementi*** o ***membri*** dell'insieme. Se un oggetto x è un elemento di un insieme S, scriviamo $x \in S$ (che si legge "x è un elemento di S" o, più semplicemente, "x è in S"). Se x non è un elemento di S, scriviamo $x \notin S$. Possiamo descrivere un insieme elencando esplicitamente i suoi elementi in una lista racchiusa tra parentesi graffe. Per esempio, possiamo definire un insieme S che contiene esattamente i numeri 1, 2 e 3, scrivendo $S = \{1, 2, 3\}$. Poiché 2 è un elemento dell'insieme S, possiamo scrivere $2 \in S$, e poiché 4 non è un elemento dell'insieme, si ha $4 \notin S$. Un insieme non può contenere lo stesso oggetto più di una volta,[1] e i suoi elementi non sono ordinati. Due insiemi A e B sono ***uguali***, se contengono gli stessi elementi; scriveremo $A = B$. Per esempio, $\{1, 2, 3, 1\} = \{1, 2, 3\} = \{3, 2, 1\}$.

Adotteremo delle notazioni speciali per gli insiemi più comuni.

- $\emptyset$ indica l'***insieme vuoto***, cioè l'insieme che non contiene alcun elemento.
- $\mathbb{Z}$ indica l'insieme dei ***numeri interi***, cioè l'insieme $\{\ldots, -2, -1, 0, 1, 2, \ldots\}$.
- $\mathbb{R}$ indica l'insieme dei ***numeri reali***.
- $\mathbb{N}$ indica l'insieme dei ***numeri naturali***, cioè l'insieme $\{0, 1, 2, \ldots\}$.[2]

Se tutti gli elementi di un insieme A sono contenuti in un insieme B, cioè se $x \in A$ implica $x \in B$, allora scriviamo $A \subseteq B$ e diciamo che A è un ***sottoinsieme*** di B. Un insieme A è un ***sottoinsieme proprio*** di B, e si scrive $A \subset B$, se $A \subseteq B$ ma $A \neq B$ (alcuni autori utilizzano il simbolo "$\subset$" per indicare la relazione di sottoinsieme ordinario, anziché la relazione di sottoinsieme proprio). Per qualsiasi insieme A, si ha $A \subseteq A$. Per due insiemi A e B, si ha $A = B$ se e soltanto se $A \subseteq B$ e $B \subseteq A$. Per tre insiemi qualsiasi A, B e C, se $A \subseteq B$ e $B \subseteq C$, allora $A \subseteq C$. Per qualsiasi insieme A, si ha $\emptyset \subseteq A$.

Acquistato da Michele Michele su Webster il 2022-01-05 23:12. Numero Ordine Libreria 1956730. Aquest text no pot ser copiat. © 2022, McGraw-Hill Education (Italy)

[1] Un insieme che può contenere lo stesso oggetto più di una volta è detto ***multinsieme***.

[2] Alcuni autori iniziano i numeri naturali da 1, anziché da 0. Attualmente si tende a includere lo 0 nei numeri naturali.

A volte gli insiemi sono definiti in funzione di altri insiemi. Dato un insieme A, possiamo definire un insieme $B \subseteq A$ stabilendo una proprietà che distingue gli elementi di B. Per esempio, possiamo definire l'insieme degli interi pari con $\{x : x \in \mathbb{Z} \text{ e } x/2 \text{ è un intero}\}$. I due punti in questa notazione si leggono "tale che" (alcuni autori usano una barra verticale al posto dei due punti).

Dati due insiemi A e B, possiamo definire nuovi insiemi applicando ***le operazioni sugli insiemi***:

- L'***intersezione*** degli insiemi A e B è l'insieme

  $$A \cap B = \{x : x \in A \text{ e } x \in B\}$$

- L'***unione*** degli insiemi A e B è l'insieme

  $$A \cup B = \{x : x \in A \text{ o } x \in B\}$$

- La ***differenza*** degli insiemi A e B è l'insieme

  $$A - B = \{x : x \in A \text{ e } x \notin B\}$$

Le operazioni sugli insiemi sono regolate dalle seguenti leggi e proprietà.

Leggi dell'insieme vuoto:

$$\begin{aligned} A \cap \emptyset &= \emptyset \\ A \cup \emptyset &= A \end{aligned}$$

Leggi dell'idempotenza:

$$\begin{aligned} A \cap A &= A \\ A \cup A &= A \end{aligned}$$

Commutatività:

$$\begin{aligned} A \cap B &= B \cap A \\ A \cup B &= B \cup A \end{aligned}$$

Associatività:

$$\begin{aligned} A \cap (B \cap C) &= (A \cap B) \cap C \\ A \cup (B \cup C) &= (A \cup B) \cup C \end{aligned}$$

Distributività:

$$\begin{aligned} A \cap (B \cup C) &= (A \cap B) \cup (A \cap C) \\ A \cup (B \cap C) &= (A \cup B) \cap (A \cup C) \end{aligned} \tag{B.1}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Assorbimento:

$$\begin{aligned} A \cap (A \cup B) &= A \\ A \cup (A \cap B) &= A \end{aligned}$$

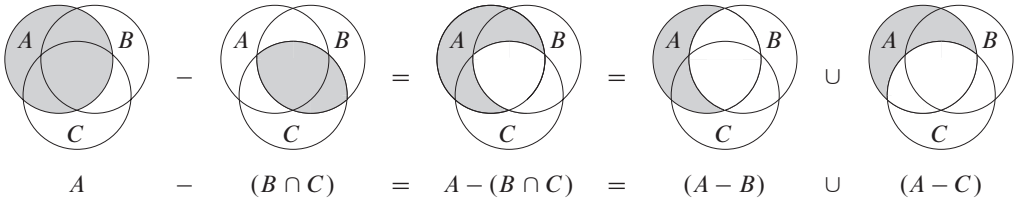


Figura B.1 Un diagramma di Venn che illustra la prima legge di DeMorgan (B.2). Ciascuno degli insiemi A, B e C è rappresentato da un cerchio.

Leggi di DeMorgan:

$$\begin{aligned} A - (B \cap C) &= (A - B) \cup (A - C) \\ A - (B \cup C) &= (A - B) \cap (A - C) \end{aligned} \tag{B.2}$$

La prima legge di DeMorgan è illustrata nella Figura B.1 con un ***diagramma di Venn***, una rappresentazione grafica in cui gli insiemi sono rappresentati come regioni del piano.

Spesso tutti gli insiemi in esame sono sottoinsiemi di qualche insieme U più grande, che è detto ***universo***. Per esempio, se stiamo considerando i vari insiemi composti soltanto dai numeri interi, l'insieme $\mathbb{Z}$ degli interi è un universo appropriato. Dato un universo U, definiamo il ***complemento*** di un insieme A come $\overline{A} = U - A$. Le seguenti leggi si applicano a qualsiasi insieme $A \subseteq U$:

$$\begin{aligned} \overline{\overline{A}} &= A \\ A \cap \overline{A} &= \emptyset \\ A \cup \overline{A} &= U \end{aligned}$$

Le leggi di DeMorgan (B.2) possono essere riscritte con i complementi. Per due insiemi qualsiasi $B, C \subseteq U$, si ha

$$\begin{aligned} \overline{B \cap C} &= \overline{B} \cup \overline{C} \\ \overline{B \cup C} &= \overline{B} \cap \overline{C} \end{aligned}$$

Due insiemi A e B sono ***disgiunti*** se non hanno elementi in comune, cioè se $A \cap B = \emptyset$. Una collezione $\mathcal{S} = \{S_i\}$ di insiemi non vuoti forma una ***partizione*** di un insieme S se

- gli insiemi sono ***disgiunti a coppie***, cioè $S_i, S_j \in \mathcal{S}$ e $i \neq j$ implicano che $S_i \cap S_j = \emptyset$, e
- la loro unione è S, ovvero

$$S = \bigcup_{S_i \in \mathcal{S}} S_i$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

In altre parole, $\mathcal{S}$ forma una partizione di S se ogni elemento di S appare esattamente in un solo $S_i \in \mathcal{S}$.

Il numero di elementi di un insieme è detto ***cardinalità*** o ***dimensione*** dell'insieme e si indica con $|S|$. Due insiemi hanno la stessa cardinalità se i loro elementi possono essere posti in una corrispondenza uno-a-uno. La cardinalità dell'insieme vuoto è $|\emptyset| = 0$. Se la cardinalità di un insieme è un numero naturale, diciamo che

l'insieme è ***finito***, altrimenti è ***infinito***. Un insieme infinito che può essere posto in una corrispondenza uno-a-uno con i numeri naturali $\mathbb{N}$ è detto ***insieme infinito numerabile***, altrimenti non è numerabile. Gli interi $\mathbb{Z}$ sono numerabili, mentre i reali $\mathbb{R}$ non sono numerabili.

Per due insiemi finiti qualsiasi A e B, si ha l'identità

$$|A \cup B| = |A| + |B| - |A \cap B| \tag{B.3}$$

dalla quale si deduce che

$$|A \cup B| \leq |A| + |B|$$

Se A e B sono disgiunti, allora $|A \cap B| = 0$ e quindi $|A \cup B| = |A| + |B|$. Se $A \subseteq B$, allora $|A| \leq |B|$.

Un insieme finito di n elementi è anche detto ***n-insieme***. Un 1-insieme è detto ***singoletto***. Un sottoinsieme di k elementi di un insieme è anche detto ***k-sottoinsieme***.

L'insieme di tutti i sottoinsiemi di un insieme S, incluso l'insieme vuoto e lo stesso S, è indicato con 2^S ed è detto ***insieme potenza*** di S. Per esempio, $2^{\{a,b\}} = \{\emptyset, \{a\}, \{b\}, \{a,b\}\}$. L'insieme potenza di un insieme finito S ha cardinalità $2^{|S|}$.

A volte siamo interessati a strutture simili agli insiemi i cui elementi sono ordinati. Una ***coppia ordinata*** di due elementi a e b è indicata con (a,b) e può essere definita formalmente come l'insieme $(a,b) = \{a, \{a,b\}\}$. Quindi, la coppia ordinata (a,b) *non* è uguale alla coppia ordinata (b,a).

Il ***prodotto cartesiano*** di due insiemi A e B, che si indica con $A \times B$, è l'insieme di tutte le coppie ordinate tali che il primo elemento di una coppia è un elemento di A e il secondo è un elemento di B. Più formalmente

$$A \times B = \{(a,b) : a \in A \text{ e } b \in B\}$$

Per esempio, $\{a,b\} \times \{a,b,c\} = \{(a,a),(a,b),(a,c),(b,a),(b,b),(b,c)\}$. Quando A e B sono insiemi finiti, la cardinalità del loro prodotto cartesiano è

$$|A \times B| = |A| \cdot |B| \tag{B.4}$$

Il prodotto cartesiano di n insiemi $A_1, A_2, \ldots, A_n$ è l'insieme di ***n-tuple***

$$A_1 \times A_2 \times \cdots \times A_n = \{(a_1, a_2, \ldots, a_n) : a_i \in A_i, i = 1, 2, \ldots, n\}$$

la cui cardinalità è

$$|A_1 \times A_2 \times \cdots \times A_n| = |A_1| \cdot |A_2| \cdots |A_n|$$

se tutti gli insiemi sono finiti. Indichiamo il prodotto cartesiano di n copie dello stesso insieme A con l'insieme

$$A^n = A \times A \times \cdots \times A$$

la cui cardinalità è $|A^n| = |A|^n$ se A è finito. Una n-tupla può essere considerata come una sequenza finita di lunghezza n (come descritto a pagina 969).

Acquistato da Michele Michela su Webster il 2023-07-07 23:12 All'mera Ordine di libreria 1T0050010 319707 3 Copyright © 2023 McGraw-Hill Education (Italy)

Esercizi

B.1-1

Disegnate i diagrammi di Venn che illustrano la prima proprietà distributiva (B.1).

B.1-2
Dimostrate la generalizzazione delle leggi di DeMorgan a una qualsiasi collezione finita di insiemi:

$$\begin{aligned}\overline{A_1 \cap A_2 \cap \cdots \cap A_n} &= \overline{A_1} \cup \overline{A_2} \cup \cdots \cup \overline{A_n} \\ \overline{A_1 \cup A_2 \cup \cdots \cup A_n} &= \overline{A_1} \cap \overline{A_2} \cap \cdots \cap \overline{A_n}\end{aligned}$$

B.1-3 ⋆
Dimostrate la generalizzazione dell'equazione (B.3), che è detta ***principio di inclusione ed esclusione***:

$$\begin{aligned}&|A_1 \cup A_2 \cup \cdots \cup A_n| = \\ &\quad |A_1| + |A_2| + \cdots + |A_n| \\ &\quad - |A_1 \cap A_2| - |A_1 \cap A_3| - \cdots \qquad \text{(tutte le coppie)} \\ &\quad + |A_1 \cap A_2 \cap A_3| + \cdots \qquad \text{(tutte triple)} \\ &\qquad\qquad \vdots \\ &\quad + (-1)^{n-1} |A_1 \cap A_2 \cap \cdots \cap A_n|\end{aligned}$$

B.1-4
Dimostrate che l'insieme dei numeri naturali dispari è numerabile.

B.1-5
Dimostrate che per qualsiasi insieme finito S, l'insieme potenza 2^S ha $2^{|S|}$ elementi (cioè ci sono $2^{|S|}$ sottoinsiemi distinti di S).

B.1-6
Date una definizione induttiva per una n-tupla estendendo la definizione di coppia ordinata della teoria degli insiemi.

B.2 Relazioni

Una ***relazione binaria*** R tra due insiemi A e B è un sottoinsieme del prodotto cartesiano $A \times B$. Se $(a, b) \in R$, a volte scriviamo $a\ R\ b$. Quando diciamo che R è una relazione binaria in un insieme A, intendiamo che R è un sottoinsieme di $A \times A$. Per esempio, la relazione "minore di" nei numeri naturali è l'insieme $\{(a, b) : a, b \in \mathbb{N} \text{ e } a < b\}$. Una relazione n-aria tra gli insiemi $A_1, A_2, \ldots, A_n$ è un sottoinsieme di $A_1 \times A_2 \times \cdots \times A_n$.

Una relazione binaria $R \subseteq A \times A$ è ***riflessiva*** se

$a\ R\ a$

per ogni $a \in A$. Per esempio, "$=$" e "$\leq$" sono relazioni riflessive in $\mathbb{N}$, ma "$<$" non lo è. La relazione R è ***simmetrica*** se

$a\ R\ b$ implica $b\ R\ a$

per ogni $a, b \in A$. Per esempio, "$=$" è una relazione simmetrica, ma "$<$" e "$\leq$" non lo sono. La relazione R è ***transitiva*** se

$a\ R\ b$ e $b\ R\ c$ implicano $a\ R\ c$

per ogni $a, b, c \in A$. Per esempio, le relazioni "$<$," "$\leq$," e "$=$" sono transitive, ma la relazione $R = \{(a, b) : a, b \in \mathbb{N} \text{ e } a = b - 1\}$ non lo è, perché $3\ R\ 4$ e $4\ R\ 5$ non implica $3\ R\ 5$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:13 Numero Ordine Libraria: 109503046-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

Una relazione che è riflessiva, simmetrica e transitiva è una ***relazione di equivalenza***. Per esempio, "=" è una relazione di equivalenza nei numeri naturali, ma "<" non lo è. Se R è una relazione di equivalenza in un insieme A, allora per $a \in A$, la ***classe di equivalenza*** di a è l'insieme $[a] = \{b \in A : a\, R\, b\}$, cioè l'insieme di tutti gli elementi equivalenti ad a. Per esempio, se definiamo $R = \{(a, b) : a, b \in \mathbb{N} \text{ e } a + b \text{ è un numero pari}\}$, allora R è una relazione di equivalenza, perché $a + a$ è pari (riflessiva), $a + b$ pari implica $b + a$ pari (simmetrica), e $a + b$ pari e $b + c$ pari implicano $a + c$ pari (transitiva). La classe di equivalenza di 4 è $[4] = \{0, 2, 4, 6, \ldots\}$ e la classe di equivalenza di 3 è $[3] = \{1, 3, 5, 7, \ldots\}$. Un teorema fondamentale delle classi di equivalenza è il seguente.

Teorema B.1 (Una relazione di equivalenza è lo stesso di una partizione)
Le classi di equivalenza di una qualsiasi relazione di equivalenza R in un insieme A formano una partizione di A, e una qualsiasi partizione di A determina una relazione di equivalenza in A di cui gli insiemi nella partizione sono le classi di equivalenza.

Dimostrazione Per la prima parte della dimostrazione, dobbiamo provare che le classi di equivalenza di R sono insiemi non vuoti, disgiunti a coppie, la cui unione è A. Poiché R è riflessiva, $a \in [a]$, e quindi le classi di equivalenza non sono vuote; inoltre, poiché ogni elemento $a \in A$ appartiene alla classe di equivalenza $[a]$, l'unione delle classi di equivalenza è A. Resta da dimostrare che le classi di equivalenza sono disgiunte a coppie, cioè, se due classi di equivalenza $[a]$ e $[b]$ hanno un elemento c in comune, allora esse in effetti sono lo stesso insieme. Infatti $a\, R\, c$ e $b\, R\, c$, per la simmetria e la transitività, implicano $a\, R\, b$. Quindi, per un elemento qualsiasi $x \in [a]$, $x\, R\, a$ implica $x\, R\, b$, e quindi $[a] \subseteq [b]$. Analogamente $[b] \subseteq [a]$, e quindi $[a] = [b]$.

Per la seconda parte della dimostrazione, sia $\mathcal{A} = \{A_i\}$ una partizione di A e definiamo $R = \{(a, b) : \text{esiste un } i \text{ tale che } a \in A_i \text{ e } b \in A_i\}$. Asseriamo che R è una relazione di equivalenza in A. La proprietà riflessiva è soddisfatta, perché $a \in A_i$ implica $a\, R\, a$. La simmetria è soddisfatta, perché se $a\, R\, b$, allora a e b sono nello stesso insieme A_i, e quindi $b\, R\, a$. Se $a\, R\, b$ e $b\, R\, c$, allora tutti e tre gli elementi sono nello stesso insieme, quindi $a\, R\, c$ e la proprietà transitiva è soddisfatta. Per vedere che gli insiemi nella partizione sono le classi di equivalenza di R, notiamo che se $a \in A_i$, allora $x \in [a]$ implica $x \in A_i$, e $x \in A_i$ implica $x \in [a]$. ■

Una relazione binaria R in un insieme A è ***antisimmetrica*** se

$$a\, R\, b \text{ e } b\, R\, a \text{ implicano } a = b$$

Per esempio, la relazione "$\leq$" nei numeri naturali è antisimmetrica, perché $a \leq b$ e $b \leq a$ implicano $a = b$. Una relazione che è riflessiva, antisimmetrica e transitiva è un ***ordine parziale***, un insieme in cui è definito un ordine parziale è detto ***insieme parzialmente ordinato***. Per esempio, la relazione "è un discendente di" è un ordine parziale nell'insieme di tutte le persone (se assumiamo che ogni individuo sia anche discendente di sé stesso).

Acquistato da Michele M... per uso esclusivo ... Copyright © 2007 The McGraw-Hill Companies ... per conto di McGraw-Hill Education (Italy)

In un insieme A parzialmente ordinato potrebbe non esserci un solo elemento "massimo" a tale che $b\, R\, a$ per ogni $b \in A$. Invece, ci possono essere più elementi ***massimali*** a tali che per nessun elemento $b \in A$, dove $b \neq a$, si abbia $a\, R\, b$. Per esempio, in una collezione di scatole di dimensioni differenti ci possono essere

molte scatole massimali che non entrano all'interno di un'altra scatola, ma non c'è una sola scatola "massima" che può contenere qualsiasi altra scatola.[3]

Una relazione R in un insieme A è una ***relazione totale*** se per ogni $a, b \in A$, si ha $a\ R\ b$ o $b\ R\ a$ (o entrambe), ovvero se ogni coppia di elementi di A è legata dalla relazione R. Un ordine parziale che è anche una relazione totale è un ***ordine totale*** o ***lineare***. Per esempio, la relazione "$\leq$" è un ordine totale nei numeri naturali, ma la relazione "è un discendente di" non è un ordine totale nell'insieme di tutte le persone, perché ci sono individui che non sono discendenti l'uno dell'altro. Una relazione totale che è transitiva, ma non necessariamente riflessiva e antisimmetrica, è un ***preordine totale***.

Esercizi

B.2-1
Dimostrate che la relazione di sottoinsieme "$\subseteq$" tra i sottoinsiemi di $\mathbb{Z}$ è un ordine parziale, non un ordine totale.

B.2-2
Dimostrate che, per qualsiasi intero positivo n, la relazione "congruo modulo n" è una relazione di equivalenza negli interi (diciamo che $a \equiv b \pmod{n}$ se esiste un intero q tale che $a - b = qn$). In quali classi di equivalenza questa relazione suddivide i numeri interi?

B.2-3
Trovate degli esempi di relazioni che sono

a. riflessive e simmetriche, ma non transitive

b. riflessive e transitive, ma non simmetriche

c. simmetriche e transitive, ma non riflessive

B.2-4
Sia S un insieme finito e sia R una relazione di equivalenza in $S \times S$. Dimostrate che se R è anche antisimmetrica, allora le classi di equivalenza di S rispetto a R sono singoletti.

B.2-5
Il professor Narcissus ritiene che, se una relazione R è simmetrica e transitiva, allora è anche riflessiva. La sua dimostrazione è questa: per simmetria, $a\ R\ b$ implica $b\ R\ a$; di conseguenza, la transitività implica $a\ R\ a$. La dimostrazione è corretta?

B.3 Funzioni

Dati due insiemi A e B, una ***funzione*** f è una relazione binaria in $A \times B$ tale che per ogni $a \in A$, esiste un solo $b \in B$ tale che $(a, b) \in f$. L'insieme A è detto ***dominio*** di f e l'insieme B è detto ***codominio*** di f. A volte scriviamo $f : A \to B$; e se $(a, b) \in f$, scriviamo $b = f(a)$, perché b è determinato in modo unico dalla scelta di a.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995030163228707-0 Copyright © 2022 McGraw-Hill Education (Italy)

[3]Per precisione, affinché la relazione "entra all'interno" possa essere un ordine parziale, dobbiamo supporre che una scatola possa entrare all'interno di sé stessa.

Intuitivamente, la funzione f assegna un elemento di B a ogni elemento di A. Nessun elemento di A è assegnato a due elementi diversi di B, ma lo stesso elemento di B può essere assegnato a due elementi diversi di A. Per esempio, la relazione binaria

$$f = \{(a, b) : a, b \in \mathbb{N} \text{ e } b = a \bmod 2\}$$

è una funzione $f : \mathbb{N} \to \{0, 1\}$, perché per ogni numero naturale a, esiste un solo valore b in $\{0, 1\}$ tale che $b = a \bmod 2$. Per questo esempio, $0 = f(0), 1 = f(1)$, $0 = f(2)$ e così via. D'altra parte, la relazione binaria

$$g = \{(a, b) : a, b \in \mathbb{N} \text{ e } a + b \text{ è pari}\}$$

non è una funzione, perché $(1, 3)$ e $(1, 5)$ sono entrambi in g, e quindi per $a = 1$, non esiste un solo elemento b tale che $(a, b) \in g$.

Data una funzione $f : A \to B$, se $b = f(a)$, diciamo che a è l'***argomento*** di f e che b è il ***valore*** di f in a. Possiamo definire una funzione stabilendo il suo valore per ogni elemento del suo dominio. Per esempio, potremmo definire $f(n) = 2n$ per $n \in \mathbb{N}$, che significa $f = \{(n, 2n) : n \in \mathbb{N}\}$. Due funzioni f e g sono ***uguali*** se hanno lo stesso dominio, lo stesso codominio e se, per ogni a nel dominio, $f(a) = g(a)$.

Una ***sequenza finita*** di lunghezza n è una funzione f il cui dominio è l'insieme di n interi $\{0, 1, \ldots, n - 1\}$. Di solito, indichiamo una sequenza finita elencando i suoi valori: $\langle f(0), f(1), \ldots, f(n-1)\rangle$. Una ***sequenza infinita*** è una funzione il cui dominio è l'insieme $\mathbb{N}$ dei numeri naturali. Per esempio, la sequenza di Fibonacci, definita della ricorrenza (3.22), è la sequenza infinita $\langle 0, 1, 1, 2, 3, 5, 8, 13, 21, \ldots\rangle$.

Quando il dominio di una funzione f è un prodotto cartesiano, di solito omettiamo le parentesi extra che racchiudono l'argomento di f. Per esempio, se avessimo una funzione $f : A_1 \times A_2 \times \cdots \times A_n \to B$, scriveremmo $b = f(a_1, a_2, \ldots, a_n)$, anziché $b = f((a_1, a_2, \ldots, a_n))$. Chiamiamo ciascun elemento a_i ***argomento*** della funzione f, sebbene tecnicamente l'argomento (unico) di f sia la n-tupla $(a_1, a_2, \ldots, a_n)$.

Se $f : A \to B$ è una funzione e $b = f(a)$, allora a volte diciamo che b è l'***immagine*** di a secondo f. L'immagine di un insieme $A' \subseteq A$ secondo f è definita da

$$f(A') = \{b \in B : b = f(a) \text{ per qualche } a \in A'\}$$

L'***immagine*** (*range*) di f è l'immagine del suo dominio, cioè $f(A)$. Per esempio, il rango della funzione $f : \mathbb{N} \to \mathbb{N}$ definita da $f(n) = 2n$ è $f(\mathbb{N}) = \{m : m = 2n \text{ per qualche } n \in \mathbb{N}\}$, ovvero l'insieme dei numeri interi pari non negativi.

Una funzione è ***suriettiva*** se la sua immagine è tutto il suo codominio. Per esempio, la funzione $f(n) = \lfloor n/2 \rfloor$ è una funzione suriettiva da $\mathbb{N}$ a $\mathbb{N}$, perché ogni elemento in $\mathbb{N}$ appare come il valore di f per qualche argomento. La funzione $f(n) = 2n$ invece non è una funzione suriettiva da $\mathbb{N}$ a $\mathbb{N}$, perché nessun argomento di f può produrre 3 come valore. Tuttavia, la funzione $f(n) = 2n$ è una funzione suriettiva dai numeri naturali ai numeri pari. Una funzione suriettiva $f : A \to B$ a volte è descritta come una mappatura di A ***in*** B. Quando diciamo che "f è in", intendiamo che la funzione è suriettiva.

Acquistato da Michele Mischiatti su (ebook.mheducation.it) il 2022-07-23 12:12 Numero Ordine: 2227702 Copyright © 2022 - McGraw-Hill Education (Italy)

Una funzione $f : A \to B$ è ***iniettiva*** se argomenti distinti di f producono valori distinti, ovvero se $a \neq a'$ implica $f(a) \neq f(a')$. Per esempio, la funzione $f(n) = 2n$ è una funzione iniettiva da $\mathbb{N}$ a $\mathbb{N}$, perché ogni numero pari b è

l'immagine secondo f al più di un elemento del dominio, cioè $b/2$. La funzione $f(n) = \lfloor n/2 \rfloor$ non è iniettiva, perché il valore 1 è prodotto da due argomenti: 2 e 3. Una funzione iniettiva è anche detta ***funzione uno-a-uno***.

Una funzione $f : A \to B$ è ***biiettiva*** se è iniettiva e suriettiva. Per esempio, $f(n) = (-1)^n \lceil n/2 \rceil$ è una funzione biiettiva da $\mathbb{N}$ a $\mathbb{Z}$:

$$\begin{array}{ccc} 0 & \to & 0 \\ 1 & \to & -1 \\ 2 & \to & 1 \\ 3 & \to & -2 \\ 4 & \to & 2 \\ & \vdots & \end{array}$$

La funzione è iniettiva, perché nessun elemento di $\mathbb{Z}$ è l'immagine di più elementi di $\mathbb{N}$. È suriettiva, perché ogni elemento di $\mathbb{Z}$ appare come l'immagine di qualche elemento di $\mathbb{N}$. Quindi, la funzione è biiettiva. Una funzione biiettiva è anche detta ***corrispondenza uno-a-uno***, perché abbina elementi del dominio e del codominio. Una funzione biiettiva da un insieme A allo stesso A è detta anche ***permutazione***.

Quando una funzione f è biiettiva, la sua ***inversa*** f^{-1} è definita come

$$f^{-1}(b) = a \text{ se e soltanto se } f(a) = b$$

Per esempio, l'inversa della funzione $f(n) = (-1)^n \lceil n/2 \rceil$ è

$$f^{-1}(m) = \begin{cases} 2m & \text{se } m \geq 0 \\ -2m - 1 & \text{se } m < 0 \end{cases}$$

Esercizi

B.3-1

Siano A e B due insiemi finiti e sia $f : A \to B$ una funzione. Dimostrate che

a. se f è iniettiva, allora $|A| \leq |B|$.

b. se f è suriettiva, allora $|A| \geq |B|$.

B.3-2

La funzione $f(x) = x + 1$ è biiettiva quando il dominio e il codominio sono $\mathbb{N}$? È biiettiva quando il dominio e il codominio sono $\mathbb{Z}$?

B.3-3

Date una definizione naturale per la relazione inversa di una relazione binaria tale che, se una relazione in effetti è una funzione biiettiva, la sua relazione inversa è la sua funzione inversa.

B.3-4 ⋆

Trovate una funzione biiettiva da $\mathbb{Z}$ a $\mathbb{Z} \times \mathbb{Z}$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

B.4 Grafi

Questo paragrafo presenta due tipi di grafi: orientati e non orientati. Alcune definizioni nella letteratura differiscono da quelle date in questo libro, ma nella maggior parte dei casi le differenze sono piccole. Il Paragrafo 22.1 spiega come possono essere rappresentati i grafi nella memoria dei calcolatori.

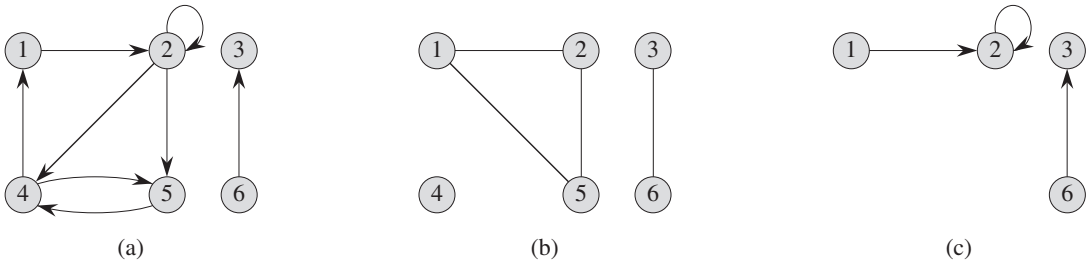


Figura B.2 Grafi orientati e non orientati. **(a)** Un grafo orientato $G = (V, E)$, dove $V = \{1, 2, 3, 4, 5, 6\}$ ed $E = \{(1, 2), (2, 2), (2, 4), (2, 5), (4, 1), (4, 5), (5, 4), (6, 3)\}$. L'arco $(2, 2)$ è un cappio. **(b)** Un grafo non orientato $G = (V, E)$, dove $V = \{1, 2, 3, 4, 5, 6\}$ ed $E = \{(1, 2), (1, 5), (2, 5), (3, 6)\}$. Il vertice 4 è isolato. **(c)** Il sottografo del grafo nella parte (a) indotto dall'insieme dei vertici $\{1, 2, 3, 6\}$.

Un ***grafo orientato*** G è una coppia (V, E), dove V è un insieme finito ed E è una relazione binaria in V. L'insieme V è detto ***insieme dei vertici*** di G e i suoi elementi sono detti ***vertici***. L'insieme E è detto ***insieme degli archi*** di G e i suoi elementi sono detti ***archi***. La Figura B.2(a) è la rappresentazione di un grafo orientato nell'insieme dei vertici $\{1, 2, 3, 4, 5, 6\}$. I vertici sono rappresentati da cerchi nella figura; gli archi sono rappresentati da frecce. Notate che sono ammessi i ***cappi*** – archi che escono ed entrano nello stesso vertice.

In un ***grafo non orientato*** $G = (V, E)$ l'insieme degli archi E è composto da coppie di vertici *non ordinate*, anziché da coppie ordinate. Ovvero un arco è un insieme $\{u, v\}$, dove $u, v \in V$ e $u \neq v$. Per convenzione, utilizziamo la notazione (u, v) per un arco, anziché la notazione di insieme $\{u, v\}$; inoltre le notazioni (u, v) e (v, u) identificano lo stesso arco. In un grafo non orientato, i cappi non sono ammessi, e quindi ogni arco è composto esattamente da due vertici distinti. La Figura B.2(b) è la rappresentazione di un grafo non orientato nell'insieme dei vertici $\{1, 2, 3, 4, 5, 6\}$. Molte definizioni per i grafi orientati e non orientati sono le stesse, sebbene alcuni termini abbiano significati leggermente differenti nei due contesti. Se (u, v) è un arco in un grafo orientato $G = (V, E)$, diciamo che (u, v) ***esce*** dal vertice u ed ***entra*** nel vertice v. Per esempio, gli archi che escono dal vertice 2 nella Figura B.2(a) sono $(2, 2)$, $(2, 4)$ e $(2, 5)$. Gli archi che entrano nel vertice 2 sono $(1, 2)$ e $(2, 2)$. Se (u, v) è un arco in un grafo non orientato $G = (V, E)$, diciamo che (u, v) è ***incidente*** nei vertici u e v. Nella Figura B.2(b), gli archi incidenti nel vertice 2 sono $(1, 2)$ e $(2, 5)$.

Se (u, v) è un arco di un grafo $G = (V, E)$, diciamo che il vertice v è ***adiacente*** al vertice u. Se il grafo non è orientato, la relazione di adiacenza è simmetrica. Se il grafo è orientato, la relazione di adiacenza non è necessariamente simmetrica. Se v è adiacente a u in un grafo orientato, a volte scriviamo $u \rightarrow v$. Nelle parti (a) e (b) della Figura B.2, il vertice 2 è adiacente al vertice 1, perché l'arco $(1, 2)$ appartiene a entrambi i grafi. Il vertice 1 *non* è adiacente al vertice 2 nella Figura B.2(a), perché l'arco $(2, 1)$ non appartiene al grafo.

Il ***grado*** di un vertice in un grafo non orientato è il numero di archi che incidono nel vertice. Per esempio, il vertice 2 nella Figura B.2(b) ha grado 2. Un vertice il cui grado è 0, come il vertice 4 nella Figura B.2(b), è ***isolato***. In un grafo orientato, il ***grado uscente*** di un vertice è il numero di archi che escono dal vertice; il ***grado entrante*** di un vertice è il numero di archi che entrano nel vertice. Il ***grado*** di un vertice in un grafo orientato è la somma del suo grado entrante e del suo grado uscente. Il vertice 2 nella Figura B.2(a) ha grado entrante 2, grado uscente 3 e grado 5.

Acquistato da Michele Michelangeli (michelangeli@gmail.com) - Copyright © 2010 The McGraw-Hill Companies, S.r.l. - McGraw-Hill Education (Italy)

Un ***cammino*** di ***lunghezza*** k da un vertice u a un vertice u' in un grafo $G = (V, E)$ è una sequenza $\langle v_0, v_1, v_2, \ldots, v_k \rangle$ di vertici tali che $u = v_0$, $u' = v_k$ e $(v_{i-1}, v_i) \in E$ per $i = 1, 2, \ldots, k$. La lunghezza del cammino è il numero di archi nel cammino. Il cammino ***contiene*** i vertici $v_0, v_1, \ldots, v_k$ e gli archi $(v_0, v_1), (v_1, v_2), \ldots, (v_{k-1}, v_k)$ (c'è sempre un cammino di lunghezza 0 da u a u). Se c'è un cammino p da u a u', diciamo che u' è ***raggiungibile*** da u attraverso p; questo fatto a volte è indicato con $u \overset{p}{\rightsquigarrow} u'$ se G è orientato.

Un cammino è ***semplice***[4] se tutti i vertici nel cammino sono distinti. Nella Figura B.2(a), il cammino $\langle 1, 2, 5, 4 \rangle$ è un cammino semplice di lunghezza 3. Il cammino $\langle 2, 5, 4, 5 \rangle$ non è semplice.

Un ***sottocammino*** di un cammino $p = \langle v_0, v_1, \ldots, v_k \rangle$ è una sottosequenza contigua dei suoi vertici. Ovvero, per ogni $0 \le i \le j \le k$, la sottosequenza dei vertici $\langle v_i, v_{i+1}, \ldots, v_j \rangle$ è un sottocammino di p.

In un grafo orientato un cammino $\langle v_0, v_1, \ldots, v_k \rangle$ forma un ***ciclo*** se $v_0 = v_k$ e il cammino contiene almeno un arco. Il ciclo è ***semplice*** se $v_1, v_2, \ldots, v_k$ sono anche distinti. Un cappio è un ciclo di lunghezza 1. I due cammini $\langle v_0, v_1, v_2, \ldots, v_{k-1}, v_0 \rangle$ e $\langle v'_0, v'_1, v'_2, \ldots, v'_{k-1}, v'_0 \rangle$ formano lo stesso ciclo se esiste un intero j tale che $v'_i = v_{(i+j) \bmod k}$ per $i = 0, 1, \ldots, k-1$. Nella Figura B.2(a), il cammino $\langle 1, 2, 4, 1 \rangle$ forma lo stesso ciclo dei cammini $\langle 2, 4, 1, 2 \rangle$ e $\langle 4, 1, 2, 4 \rangle$. Questo ciclo è semplice, ma il ciclo $\langle 1, 2, 4, 5, 4, 1 \rangle$ non lo è. Il ciclo $\langle 2, 2 \rangle$ formato dall'arco $(2, 2)$ è un cappio. Un grafo orientato senza cappi è ***semplice***. In un grafo non orientato un cammino $\langle v_0, v_1, \ldots, v_k \rangle$ forma un ***ciclo (semplice)*** se $k \ge 3$, $v_0 = v_k$ e $v_1, v_2, \ldots, v_k$ sono distinti. Per esempio, nella Figura B.2(b) il cammino $\langle 1, 2, 5, 1 \rangle$ è un ciclo. Un grafo senza cicli è ***aciclico***.

Un grafo non orientato è ***connesso*** se ogni coppia di vertici è collegata attraverso un cammino. Le ***componenti connesse*** di un grafo sono le classi di equivalenza dei vertici secondo la relazione "è raggiungibile da". Il grafo nella Figura B.2(b) ha tre componenti connesse: $\{1, 2, 5\}$, $\{3, 6\}$ e $\{4\}$. Ogni vertice in $\{1, 2, 5\}$ è raggiungibile da ogni altro vertice in $\{1, 2, 5\}$. Un grafo non orientato è connesso se ha esattamente una componente connessa, ovvero se ogni vertice è raggiungibile da ogni altro vertice.

Un grafo orientato è ***fortemente connesso*** se due vertici qualsiasi sono raggiungibili l'uno dall'altro. Le ***componenti fortemente connesse*** di un grafo orientato sono le classi di equivalenza dei vertici secondo la relazione "sono mutuamente raggiungibili". Un grafo orientato è fortemente connesso se ha una sola componente fortemente connessa. Il grafo nella Figura B.2(a) ha tre componenti fortemente connesse: $\{1, 2, 4, 5\}$, $\{3\}$ e $\{6\}$. Tutte le coppie di vertici in $\{1, 2, 4, 5\}$ sono mutuamente raggiungibili. I vertici $\{3, 6\}$ non formano una componente fortemente connessa, perché il vertice 6 non può essere raggiunto dal vertice 3.

Due grafi $G = (V, E)$ e $G' = (V', E')$ sono ***isomorfi*** se esiste una funzione biiettiva $f : V \to V'$ tale che $(u, v) \in E$ se e soltanto se $(f(u), f(v)) \in E'$. In altre parole, possiamo rietichettare i vertici di G come i vertici di G', mantenendo gli archi corrispondenti in G e G'. La Figura B.3(a) illustra una coppia di grafi isomorfi G e G' con i rispettivi insiemi di vertici $V = \{1, 2, 3, 4, 5, 6\}$ e

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[4] Alcuni autori usano il termine "percorso" per indicare quello che noi chiamiamo cammino e riservano il termine "cammino" per indicare quello che noi chiamiamo cammino semplice. In questo testo noi useremo sempre "cammino" e "cammino semplice" in modo consistente con la definizione che ne abbiamo dato.

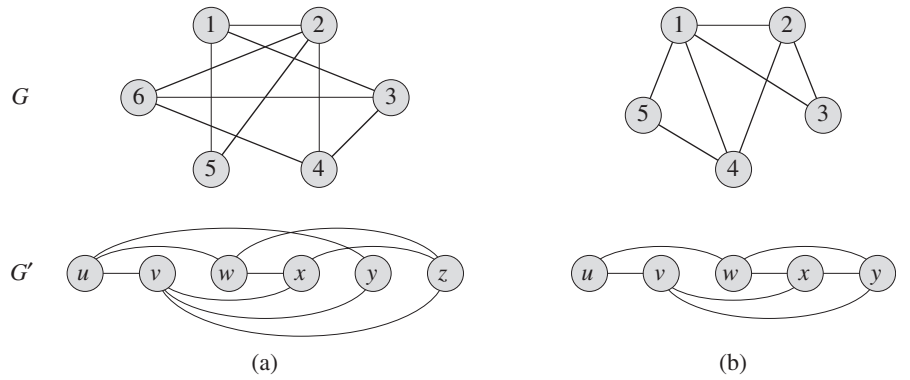


Figura B.3
(a) Una coppia di grafi isomorfi. I vertici del grafo in alto sono mappati nei vertici del grafo in basso da $f(1) = u$, $f(2) = v$, $f(3) = w$, $f(4) = x$, $f(5) = y$, $f(6) = z$. **(b)** I due grafi non sono isomorfi, perché il grafo in alto ha un vertice di grado 4 e il grafo in basso no.

$V' = \{u, v, w, x, y, z\}$. La mappatura da V a V' data da $f(1) = u$, $f(2) = v$, $f(3) = w$, $f(4) = x$, $f(5) = y$, $f(6) = z$ è la funzione biiettiva richiesta. I grafi nella Figura B.3(b) non sono isomorfi. Sebbene entrambi i grafi abbiamo 5 vertici e 7 archi, il grafo in alto ha un vertice di grado 4, che il grafo in basso non ha.

Diciamo che un grafo $G' = (V', E')$ è un ***sottografo*** di $G = (V, E)$ se $V' \subseteq V$ ed $E' \subseteq E$. Dato un insieme $V' \subseteq V$, il sottografo di G ***indotto*** da V' è il grafo $G' = (V', E')$, dove

$$E' = \{(u, v) \in E : u, v \in V'\}$$

Il sottografo indotto dall'insieme dei vertici $\{1, 2, 3, 6\}$ nella Figura B.2(a) appare nella Figura B.2(c) e ha l'insieme degli archi $\{(1, 2), (2, 2), (6, 3)\}$.

Dato un grafo non orientato $G = (V, E)$, la ***versione orientata*** di G è il grafo orientato $G' = (V, E')$, dove $(u, v) \in E'$ se e soltanto se $(u, v) \in E$. Ovvero ogni arco non orientato (u, v) in G è sostituito nella versione orientata con i due archi orientati (u, v) e (v, u). Dato un grafo orientato $G = (V, E)$, la ***versione non orientata*** di G è il grafo non orientato $G' = (V, E')$, dove $(u, v) \in E'$ se e soltanto se $u \neq v$ e $(u, v) \in E$. Ovvero la versione non orientata contiene gli archi di G "senza le loro direzioni" e senza i cappi. (Poiché (u, v) e (v, u) sono lo stesso arco in un grafo non orientato, la versione non orientata di un grafo orientato contiene questi archi una sola volta, anche se il grafo orientato contiene entrambi gli archi (u, v) e (v, u).) In un grafo orientato $G = (V, E)$, un ***vicino*** di un vertice u è un vertice qualsiasi che è adiacente a u nella versione non orientata di G. Ovvero v è un vicino di u se $u \neq v$ e $(u, v) \in E$ o $(v, u) \in E$. In un grafo non orientato, u e v sono vicini se sono adiacenti.

Alcuni tipi di grafi hanno dei nomi speciali. Un ***grafo completo*** è un grafo non orientato in cui ogni coppia di vertici è adiacente. Un ***grafo bipartito*** è un grafo non orientato $G = (V, E)$ in cui V può essere suddiviso in due insiemi V_1 e V_2 tali che $(u, v) \in E$ implica che $u \in V_1$ e $v \in V_2$ oppure che $u \in V_2$ e $v \in V_1$. Ovvero tutti gli archi stanno tra i due insiemi V_1 e V_2. Un grafo aciclico e non orientato è una ***foresta***; un grafo connesso, aciclico e non orientato è un ***albero (libero)*** (vedere il Paragrafo B.5). A volte, un grafo orientato aciclico viene chiamato semplicemente ***dag***; questo nome deriva dalle lettere iniziali di "directed acyclic graph".

Acquistato da Michele Mastrangelo (mastrangelo.michele@gmail.com) il 12/12/2023 per uso personale. Copyright © 2022, McGraw-Hill Education (Italy)

Ci sono due varianti di grafi che potreste occasionalmente incontrare nella letteratura. Un ***multigrafo*** è come un grafo non orientato, con la differenza che può avere cappi e più di un arco tra due vertici. Un ***ipergrafo*** è come un grafo non orientato, con la differenza che ciascun ***iperarco***, anziché collegare due vertici,

collega un sottoinsieme arbitrario di vertici. Molti algoritmi scritti per i normali grafi orientati e non orientati possono essere adattati per funzionare con queste strutture simili ai grafi.

La ***contrazione*** di un grafo non orientato $G = (V, E)$ in un arco $e = (u, v)$ è un grafo $G' = (V', E')$, dove $V' = V - \{u, v\} \cup \{x\}$ e x è un nuovo vertice. L'insieme degli archi E' è formato da E eliminando l'arco (u, v) e, per ogni vertice w incidente in u o v, cancellando l'arco (u, w) o (v, w) che si trova in E e aggiungendo il nuovo arco (x, w). In altre parole, u e v vengono "contratti" in un unico vertice.

Esercizi

B.4-1

I partecipanti a una festa accademica si stringono le mani per salutarsi; ogni professore ricorda quante volte ha stretto le mani. Alla fine della festa, il direttore del dipartimento somma il numero di volte che ogni professore ha stretto le mani. Dimostrate che il risultato è un numero pari provando il ***lemma della stretta di mano***: se $G = (V, E)$ è un grafo non orientato, allora

$$\sum_{v \in V} \text{grado}(v) = 2\,|E|$$

B.4-2

Dimostrate che, se un grafo orientato o non orientato contiene un cammino tra due vertici u e v, allora contiene un cammino semplice tra u e v. Dimostrate che se un grafo orientato contiene un ciclo, allora contiene un ciclo semplice.

B.4-3

Dimostrate che qualsiasi grafo $G = (V, E)$ connesso e non orientato soddisfa la relazione $|E| \geq |V| - 1$.

B.4-4

Verificate che in un grafo non orientato, la relazione "è raggiungibile da" è una relazione di equivalenza nei vertici del grafo. Quale delle tre proprietà di una relazione di equivalenza è soddisfatta in generale per la relazione "è raggiungibile da" nell'insieme dei vertici di un grafo orientato?

B.4-5

Qual è la versione non orientata del grafo orientato della Figura B.2(a)? Qual è la versione orientata del grafo non orientato della Figura B.2(b)?

B.4-6 ⋆

Dimostrate che un ipergrafo può essere rappresentato da un grafo bipartito se si suppone che l'incidenza nell'ipergrafo corrisponda all'adiacenza nel grafo bipartito. (*Suggerimento:* supponete che un insieme di vertici del grafo bipartito corrisponda ai vertici dell'ipergrafo e che l'altro insieme di vertici del grafo bipartito corrisponda agli iperarchi.)

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

B.5 Alberi

Come per i grafi, anche per gli alberi ci sono molti concetti correlati, anche se leggermente differenti. Questo paragrafo presenta le definizioni e le proprietà matematiche di vari tipi di alberi. I Paragrafi 10.4 e 22.1 descrivono come gli alberi possono essere rappresentati nella memoria dei calcolatori.

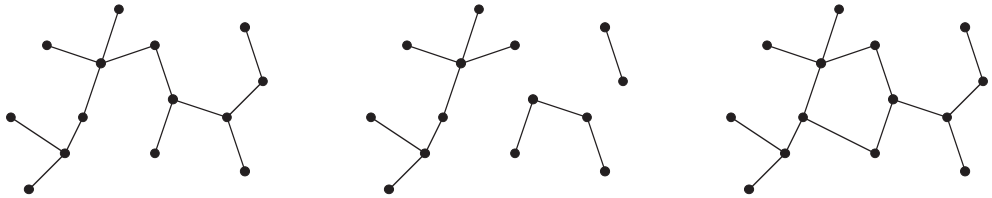


Figura B.4 **(a)** Un albero libero. **(b)** Una foresta. **(c)** Un grafo che contiene un ciclo e, quindi, non è un albero né una foresta.

B.5.1 Alberi liberi

Come è stato definito nel Paragrafo B.4, un ***albero libero*** è un grafo connesso, aciclico e non orientato. Di solito, omettiamo l'aggettivo "libero" quando diciamo che un grafo è un albero. Se un grafo non orientato è aciclico (eventualmente non connesso), allora è una ***foresta***. Molti algoritmi che funzionano per gli alberi, funzionano anche per le foreste. La Figura B.4(a) illustra un albero libero. La Figura B.4(b) illustra una foresta, che non è un albero perché il grafo non è connesso. Il grafo nella Figura B.4(c) è connesso ma non è né un albero né una foresta, perché contiene un ciclo.

Il seguente teorema riassume molte proprietà degli alberi liberi.

Teorema B.2 (Proprietà degli alberi liberi)
Sia $G = (V, E)$ un grafo non orientato. Le seguenti asserzioni sono equivalenti.

1. G è un albero libero.
2. Due vertici qualsiasi in G sono collegati da un cammino semplice unico.
3. G è connesso, ma se viene rimosso un arco qualsiasi da E, il grafo risultante non è connesso.
4. G è connesso e $|E| = |V| - 1$.
5. G è aciclico e $|E| = |V| - 1$.
6. G è aciclico, ma se un arco qualsiasi viene aggiunto a E, il grafo risultante contiene un ciclo.

Dimostrazione (1) $\Rightarrow$ (2): poiché un albero è connesso, due vertici qualsiasi in G sono connessi almeno da un cammino semplice. Siano u e v due vertici che sono connessi da due cammini semplici distinti p_1 e p_2, come illustra la Figura B.5. Sia w il vertice in cui i cammini divergono per la prima volta; ovvero w è il primo vertice in entrambi i cammini p_1 e p_2 il cui successore in p_1 è x e il cui successore in p_2 è y, dove $x \neq y$. Sia z il primo vertice in cui i cammini si ricongiungono; ovvero z è il primo vertice che segue w in p_1 e che è anche in p_2. Sia p' il sottocammino di p_1 da w per x fino a z; sia p'' il sottocammino di p_2 da w per y fino a z. I cammini p' e p'' non hanno vertici in comune, tranne i loro estremi. Quindi, il cammino ottenuto concatenando p' e l'inverso di p'' è un ciclo. Questo contraddice l'ipotesi che G sia un albero. Quindi, se G è un albero, ci può essere al più un cammino semplice tra due vertici.

Acquistato da Michele Mostarda in data 03/12/2022 20:03:21 e 22/12/2022 17:51:37. Copyright © 2022, McGraw-Hill Education (Italy)

(2) $\Rightarrow$ (3): se due vertici qualsiasi in G sono connessi da un cammino semplice unico, allora G è connesso. Sia (u, v) un arco qualsiasi in E. Questo arco è un cammino da u a v, e quindi deve essere l'unico cammino da u a v. Se rimuoviamo (u, v) da G, non c'è un cammino da u a v, e quindi la sua rimozione fa sì che G non sia più un grafo connesso.

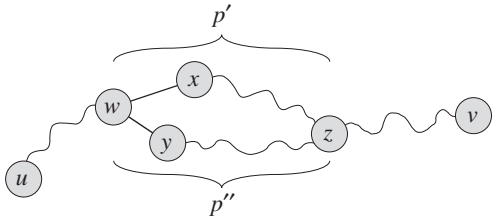


Figura B.5 Un passo della dimostrazione del Teorema B.2: se (1) G è un albero libero, allora (2) due vertici qualsiasi in G sono collegati da un cammino semplice unico. Supponiamo per assurdo che i vertici u e v siano collegati da due cammini semplici distinti p_1 e p_2. Questi cammini divergono per la prima volta nel vertice w e si riuniscono per la prima volta nel vertice z. Il cammino p' concatenato con l'inverso del cammino p'' forma un ciclo, che determina la contraddizione.

(3) $\Rightarrow$ (4): per ipotesi, il grafo G è connesso e, per l'Esercizio B.4-3, si ha $|E| \geq |V| - 1$. Dimostreremo $|E| \leq |V| - 1$ per induzione. Un grafo connesso con $n = 1$ o $n = 2$ vertici ha $n - 1$ archi. Supponiamo che G abbia $n \geq 3$ vertici e che tutti i grafi che soddisfano la (3) con meno di n vertici soddisfino anche $|E| \leq |V| - 1$. Eliminando un arco arbitrario da G, si divide il grafo in $k \geq 2$ componenti connesse (in effetti $k = 2$). Ogni componente soddisfa la (3), altrimenti G non soddisferebbe la (3). Se pensiamo ogni componente connessa V_i come albero libero allora, siccome ogni componente connessa ha meno di $|V|$ vertici, per ipotesi induttiva abbiamo $|E_i| \leq |V_i| - 1$. Quindi il numero complessivo di archi in tutte le componenti è al più $|V| - k \leq |V| - 2$. Aggiungendo l'arco rimosso si ha $|E| \leq |V| - 1$.

(4) $\Rightarrow$ (5): supponiamo che G sia connesso e che $|E| = |V| - 1$. Dobbiamo dimostrare che G è aciclico. Supponiamo che G abbia un ciclo che contiene k vertici $v_1, v_2, \ldots, v_k$ e, senza perdere in generalità, supponiamo che questo ciclo sia semplice. Sia $G_k = (V_k, E_k)$ il sottografo di G che è formato dal ciclo. Notate che $|V_k| = |E_k| = k$. Se $k < |V|$, deve esistere un vertice $v_{k+1} \in V - V_k$ che è adiacente a qualche vertice $v_i \in V_k$, perché G è connesso. Definiamo $G_{k+1} = (V_{k+1}, E_{k+1})$ come il sottografo di G con $V_{k+1} = V_k \cup \{v_{k+1}\}$ e $E_{k+1} = E_k \cup \{(v_i, v_{k+1})\}$. Notate che $|V_{k+1}| = |E_{k+1}| = k + 1$. Se $k + 1 < |V|$, possiamo continuare, definendo G_{k+2} nella stessa maniera, e così via, finché otteniamo $G_n = (V_n, E_n)$, dove $n = |V|$, $V_n = V$ e $|E_n| = |V_n| = |V|$. Poiché G_n è un sottografo di G, abbiamo $E_n \subseteq E$, e quindi $|E| \geq |V|$, che contraddice l'ipotesi che $|E| = |V| - 1$. Quindi il grafo G è aciclico.

(5) $\Rightarrow$ (6): supponiamo che G sia aciclico e che $|E| = |V| - 1$. Sia k il numero di componenti connesse di G. Ogni componente connessa è un albero libero per definizione, e poiché la (1) implica la (5), la somma di tutti gli archi in tutte le componenti connesse di G è $|V| - k$. Di conseguenza, dobbiamo avere $k = 1$, ed effettivamente G è un albero. Poiché la (1) implica la (2), due vertici qualsiasi in G sono connessi da un cammino semplice unico. Quindi, aggiungendo un arco qualsiasi a G si crea un ciclo.

(6) $\Rightarrow$ (1): supponiamo che G sia aciclico, ma che si crei un ciclo, se viene aggiunto un arco qualsiasi a E. Dobbiamo dimostrare che G è connesso. Siano u e v due vertici arbitrari in G. Se u e v non sono già adiacenti, l'aggiunta dell'arco (u, v) crea un ciclo in cui tutti gli archi, tranne (u, v), appartengono a G. Quindi, esiste un cammino da u a v e, poiché u e v sono stati scelti arbitrariamente, il grafo G è connesso. ■

Acquistato da Michele Michele su Webster il 2022-07-23 23:32:22 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

B.5.2 Alberi radicati e alberi ordinati

Un ***albero radicato*** è un albero libero in cui uno dei vertici si distingue da tutti gli altri; questo vertice si chiama ***radice*** dell'albero. Spesso facciamo riferimento

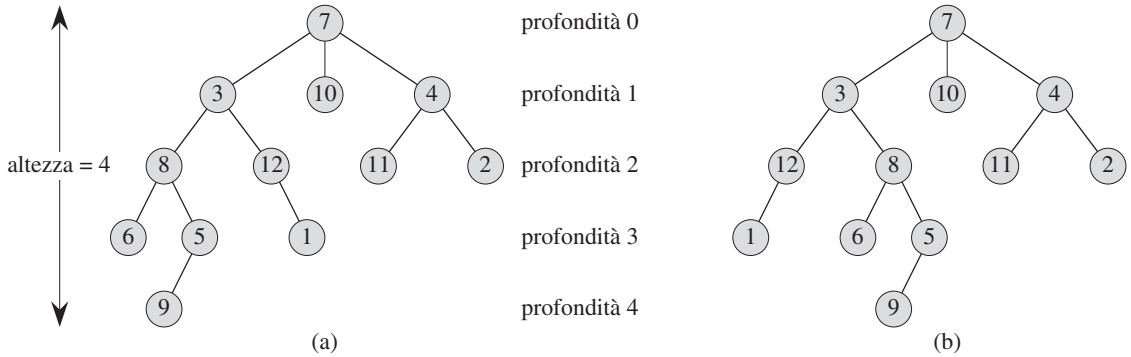


Figura B.6 Alberi radicati e alberi ordinati. **(a)** Un albero radicato con altezza 4. L'albero è disegnato nel modo standard: la radice (nodo 7) è in cima, i suoi figli (i nodi con profondità 1) sono sotto di essa, i loro figli (i nodi con profondità 2) sono sotto di essi e così via. Se l'albero è ordinato, è importante l'ordinamento relativo da sinistra a destra dei figli, altrimenti non lo è. **(b)** Un altro albero radicato. Come albero radicato è identico all'albero in (a), ma come albero ordinato è differente, perché i figli del nodo 3 sono ordinati in modo diverso.

a un vertice di un albero radicato come a un ***nodo***[5] dell'albero. La Figura B.6(a) illustra un albero radicato per un insieme di 12 nodi con radice 7.

Consideriamo un nodo x in albero radicato T con radice r. Un nodo qualsiasi y in un cammino semplice unico da r a x è detto ***antenato*** di x. Se y è un antenato di x, allora x è un ***discendente*** di y (ogni nodo è antenato e discendente di sé stesso). Se y è un antenato di x e $x \neq y$, allora y è un ***antenato proprio*** di x e x è un ***discendente proprio*** di y. Il ***sottoalbero con radice in x*** è l'albero indotto dai discendenti di x, con radice in x. Per esempio, il sottoalbero con radice nel nodo 8 nella Figura B.6(a) contiene i nodi 8, 6, 5 e 9.

Se l'ultimo arco nel cammino semplice dalla radice r di un albero T a un nodo x è (y, x), allora y è il ***padre*** di x e x è un ***figlio*** di y. La radice è l'unico nodo in T che non ha un padre. Se due nodi hanno lo stesso padre, sono ***fratelli***. Un nodo senza figli è un ***nodo esterno*** o ***foglia***. Un nodo non foglia è un ***nodo interno***.

Il numero di figli di un nodo x in un albero radicato T è detto ***grado*** di x.[6] La lunghezza del cammino semplice dalla radice r a un nodo x è la ***profondità*** di x in T. Un ***livello*** in un albero è costituito da tutti i nodi che stanno alla stessa profondità. L'***altezza*** di un nodo in un albero è il numero di archi nel più lungo cammino semplice che scende dal nodo a una foglia; l'altezza di un albero è l'altezza della sua radice. L'altezza di un albero è anche uguale alla profondità massima di un nodo qualsiasi dell'albero.

Un ***albero ordinato*** è un albero radicato in cui i figli di ogni nodo sono ordinati. Ovvero, se un nodo ha k figli, allora c'è un primo figlio, un secondo figlio, ..., e un k-esimo figlio. I due alberi nella Figura B.6 sono differenti se vengono considerati alberi ordinati, ma sono uguali se vengono considerati alberi radicati.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[5] Il termine "nodo" è utilizzato spesso nei libri di teoria dei grafi come sinonimo di "vertice". Riserveremo il termine "nodo" per indicare un vertice di un albero radicato.

[6] Notate che il grado di un nodo dipende dal fatto che T sia considerato albero radicato o albero libero. Il grado di un vertice in un albero libero è, come in qualsiasi grafo non orientato, il numero di vertici adiacenti. In un albero radicato, invece, il grado è il numero di figli – il padre di un nodo non influisce sul suo grado.

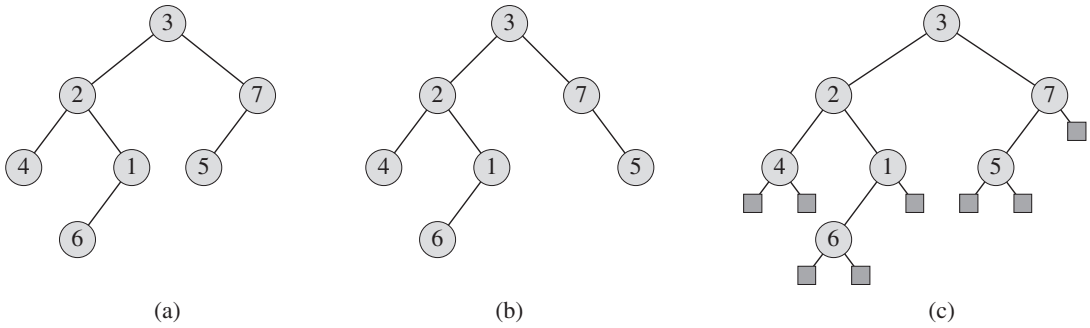


Figura B.7 Alberi binari. **(a)** Un albero binario disegnato nel modo standard. Il figlio sinistro di un nodo è disegnato sotto il nodo e a sinistra. Il figlio destro è disegnato sotto e a destra. **(b)** Un albero binario diverso da quello illustrato in (a). In (a), il figlio sinistro del nodo 7 è 5 e il figlio destro è assente. In (b), il figlio sinistro del nodo 7 è assente e il figlio destro è 5. Come alberi ordinati, questi alberi sono uguali, ma come alberi binari sono differenti. **(c)** L'albero binario in (a) rappresentato dai nodi interni di un albero binario pieno: un albero ordinato in cui ogni nodo interno ha grado 2. Le foglie dell'albero sono rappresentate dai quadrati.

B.5.3 Alberi binari e alberi posizionali

Gli alberi binari sono definiti in modo ricorsivo. Un ***albero binario*** T è una struttura definita in un insieme finito di nodi che

- non contiene nodi, oppure
- è composto da tre insiemi disgiunti di nodi: un nodo ***radice***, un albero binario detto ***sottoalbero sinistro*** della radice e un albero binario detto ***sottoalbero destro*** della radice.

L'albero binario che non contiene nodi è detto ***albero vuoto*** o ***nullo***; a volte è indicato con NIL. Se il sottoalbero sinistro non è vuoto, la sua radice è detta ***figlio sinistro*** della radice dell'intero albero. Analogamente, la radice di un sottoalbero destro non nullo è il ***figlio destro*** della radice dell'intero albero. Se un sottoalbero è l'albero nullo NIL, diciamo che il figlio è ***assente*** o ***mancante***. La Figura B.7(a) illustra un albero binario.

Un albero binario non è semplicemente un albero ordinato in cui ogni nodo ha grado al più 2. Per esempio, in un albero binario, se un nodo ha un solo figlio, la posizione del figlio è importante: ***figlio sinistro*** o ***figlio destro***. In un albero ordinato con un solo figlio non c'è distinzione tra figlio sinistro e figlio destro. La Figura B.7(b) illustra un albero binario che è diverso dall'albero della Figura B.7(a) per la posizione di un nodo. Tuttavia, se li trattiamo come alberi ordinati, i due alberi sono identici.

Le informazioni sulle posizioni in un albero binario possono essere rappresentate dai nodi interni di un albero ordinato, come illustra la Figura B.7(c). L'idea è quella di sostituire ogni nodo mancante nell'albero binario con un nodo senza figli. Questi nodi foglia sono rappresentati con dei quadrati nella figura. L'albero risultante è un ***albero binario pieno***: ogni nodo o è una foglia o ha grado 2. Non ci sono nodi di grado 1. Di conseguenza, l'ordine dei figli di un nodo conserva l'informazione sulla posizione.

Acquistato da Michele Michele su Webster il 2022-[illegible]

Le informazioni sulle posizioni che contraddistinguono gli alberi binari dagli alberi ordinati possono essere estese agli alberi con più di 2 figli per nodo. In un ***albero posizionale*** i figli di un nodo sono etichettati con interi positivi distinti.

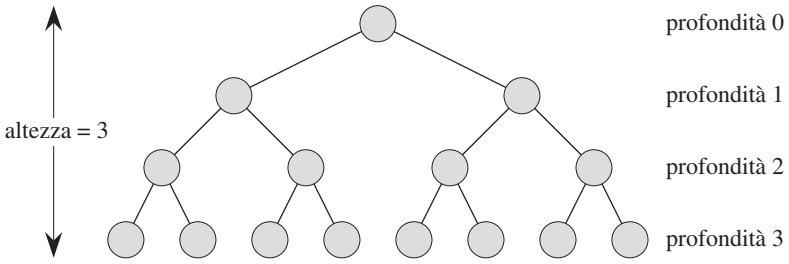


Figura B.8 Un albero binario completo di altezza 3 con 8 foglie e 7 nodi interni.

L'i-esimo figlio di un nodo è ***assente*** se nessun figlio è etichettato con l'intero i. Un ***albero k-ario*** è un albero posizionale in cui, per ogni nodo, tutti i figli con le etichette maggiori di k sono assenti. Quindi, un albero binario è un albero k-ario con $k = 2$.

Un ***albero k-ario completo*** è un albero k-ario in cui tutte le foglie hanno la stessa profondità e tutti i nodi interni hanno grado k. La Figura B.8 illustra un albero binario completo di altezza 3. Quante foglie ha un albero k-ario completo di altezza h? La radice ha k figli alla profondità 1, ciascuno dei quali ha k figli alla profondità 2 e così via. Quindi, il numero di foglie alla profondità h è k^h. Di conseguenza, l'altezza di un albero k-ario completo con n foglie è $\log_k n$. Il numero di nodi interni di un albero k-ario completo di altezza h è

$$\begin{aligned} 1 + k + k^2 + \cdots + k^{h-1} &= \sum_{i=0}^{h-1} k^i \\ &= \frac{k^h - 1}{k - 1} \end{aligned}$$

per l'equazione (A.5). Quindi, un albero binario completo ha $2^h - 1$ nodi interni.

Esercizi

B.5-1

Disegnate tutti gli alberi liberi composti dai 3 vertici x, y e z. Disegnate tutti gli alberi radicati con i nodi x, y e z, con x come radice. Disegnate tutti gli alberi ordinati con i nodi x, y e z, con x come radice. Disegnate tutti gli alberi binari con i nodi x, y e z, con x come radice.

B.5-2

Sia $G = (V, E)$ un grafo orientato aciclico, che contiene un vertice $v_0 \in V$ per il quale esiste un cammino unico che va da v_0 a ogni vertice $v \in V$. Dimostrate che la versione non orientata del grafo G forma un albero.

B.5-3

Dimostrate per induzione che il numero di nodi di grado 2 in qualsiasi albero binario non nullo è uguale al numero di foglie meno 1. Concludere che il numero di nodi interni in un albero binario pieno è 1 in meno del numero di foglie.

Acquistato da Michele Michela su Webster il 2022-07-07 23:13 Numero Ordine Libreria 19950301a 220707 0 Copyright © 2022, McGraw-Hill Education (Italy)

B.5-4

Utilizzate l'induzione per dimostrare che un albero binario non nullo con n nodi ha un'altezza almeno pari a $\lfloor \lg n \rfloor$.

B.5-5 ⋆
La ***lunghezza di cammino interna*** di un albero binario pieno è la somma delle profondità di tutti i nodi interni dell'albero. Analogamente, la ***lunghezza di cammino esterna*** è la somma delle profondità di tutte le foglie dell'albero. Considerate un albero binario pieno con n nodi interni, lunghezza di cammino interna i e lunghezza di cammino esterna e. Dimostrate che $e = i + 2n$.

B.5-6 ⋆
Associamo un "peso" $w(x) = 2^{-d}$ a ogni foglia x di profondità d in un albero binario T e sia L l'insieme delle foglie di T. Dimostrate che $\sum_{x \in L} w(x) \leq 1$. Questa relazione è nota come ***disuguaglianza di Kraft***.

B.5-7 ⋆
Dimostrate che, se $L \geq 2$, allora ogni albero binario con L foglie contiene un sottoalbero che ha un numero di foglie compreso tra $L/3$ e $2L/3$, estremi inclusi.

Problemi

B-1 Colorazione di grafi
Dato un grafo non orientato $G = (V, E)$, una ***k-colorazione*** di G è una funzione $c : V \to \{0, 1, \ldots, k-1\}$ tale che $c(u) \neq c(v)$ per ogni arco $(u, v) \in E$. In altre parole, i numeri $0, 1, \ldots, k-1$ rappresentano i k colori e i vertici adiacenti devono avere colori differenti.

a. Dimostrate che qualsiasi albero è 2-colorabile.

b. Dimostrate che le seguenti asserzioni sono equivalenti:

1. G è bipartito.
2. G è 2-colorabile.
3. G non ha cicli di lunghezza dispari.

c. Sia d il grado massimo di un vertice qualsiasi in un grafo G. Dimostrate che G può essere colorato con $d + 1$ colori.

d. Dimostrate che se il grafo G ha $O(|V|)$ archi, allora può essere colorato con $O(\sqrt{|V|})$ colori.

B-2 I grafi dell'amicizia
Riformulate ciascuna delle seguenti asserzioni come un teorema sui grafi non orientati, e poi dimostratele. Supponete che l'amicizia sia simmetrica, ma non riflessiva.

a. In qualsiasi gruppo di $n \geq 2$ persone, ci sono due persone che hanno lo stesso numero di amici nel gruppo.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

b. In ogni gruppo di sei persone o ci sono almeno tre amici reciproci oppure ci sono almeno tre sconosciuti reciproci.

c. Un gruppo qualsiasi di persone può essere suddiviso in due sottogruppi in modo tale che almeno metà degli amici di una persona qualsiasi appartenga al sottogruppo cui *non* appartiene quella persona.

d. Se ogni persona di un gruppo è amica di almeno metà delle persone del gruppo, allora tutto il gruppo potrà sedersi attorno a un tavolo in modo tale ogni persona sia seduta tra due amici.

B-3 Bisezione degli alberi

Molti algoritmi divide et impera che operano sui grafi richiedono che un grafo sia sezionato in due sottografi di dimensione pressoché uguale, che sono indotti da una partizione dei vertici. Questo problema esamina le bisezioni degli alberi formate rimuovendo un piccolo numero di archi. Richiediamo che, se due vertici finiscono nello stesso sottoalbero, dopo che sono stati rimossi gli archi, allora essi devono stare dalla stessa parte della partizione.

a. Dimostrate che eliminando un singolo arco, è possibile partizionare i vertici di qualsiasi albero binario di n vertici in due insiemi A e B tali che $|A| \leq 3n/4$ e $|B| \leq 3n/4$.

b. Dimostrate che la costante $3/4$ nel punto (a) è ottima nel caso peggiore, trovando un esempio di un semplice albero binario la cui partizione più equamente bilanciata dopo l'eliminazione di un singolo arco abbia $|A| = 3n/4$.

c. Dimostrate che eliminando al più $O(\lg n)$ archi, è possibile suddividere i vertici di qualsiasi albero binario di n vertici in due insiemi A e B tali che $|A| = \lfloor n/2 \rfloor$ e $|B| = \lceil n/2 \rceil$.

Note

G. Boole è stato un pioniere dello sviluppo della logica simbolica e ha presentato gran parte delle notazioni fondamentali dell'insiemistica in un libro pubblicato nel 1854. La moderna teoria degli insiemi è stata creata da G. Cantor nel periodo 1874–1895. Cantor si occupò principalmente degli insiemi di cardinalità infinita. Il termine "funzione" è attribuito a G. W. Leibniz, che lo utilizzò per fare riferimento a vari tipi di formule matematiche; la sua restrittiva definizione è stata più volte generalizzata. La teoria dei grafi risale al 1736, quando L. Eulero dimostrò che era impossibile attraversare una sola volta ciascuno dei sette ponti nella città di Königsberg e ritornare al punto di partenza.

Il libro di Harary [161] è un utile compendio di definizioni e proprietà della teoria dei grafi.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

C Calcolo combinatorio e delle probabilità

Questa appendice tratta i concetti fondamentali del calcolo combinatorio e delle probabilità. Se avete un buon bagaglio di conoscenze di questi argomenti, vi consigliamo di leggere rapidamente la parte iniziale dell'appendice e di concentrarvi sui paragrafi successivi. La maggior parte dei capitoli di questo libro non richiede il calcolo delle probabilità, ma per alcuni è essenziale. Il Paragrafo C.1 descrive le nozioni fondamentali del calcolo combinatorio, incluse le formule standard per calcolare il numero di permutazioni e combinazioni. Gli assiomi della teoria della probabilità e i concetti base che riguardano le distribuzioni di probabilità sono presentati nel Paragrafo C.2. Le variabili casuali sono introdotte nel Paragrafo C.3, insieme con le proprietà del valore atteso e della varianza. Il Paragrafo C.4 esamina le distribuzioni binomiali e geometriche che si presentano studiando le prove ripetute di Bernoulli. Lo studio della distribuzione binomiale prosegue nel Paragrafo C.5 con un'analisi approfondita delle "code" della distribuzione.

C.1 Calcolo combinatorio

Il calcolo combinatorio cerca di rispondere alla domanda "Quanti?", senza elencare effettivamente tutti gli elementi della risposta. Per esempio, potremmo chiederci "Quanti numeri distinti di n bit esistono?" o "Quante disposizioni di n elementi distinti esistono?" Questo paragrafo è dedicato al calcolo combinatorio. Poiché qualche argomento richiede la conoscenza degli insiemi, consigliamo ai lettori di riesaminare il Paragrafo B.1.

Regole della somma e del prodotto

Un insieme di cui vogliamo contare gli elementi, a volte, può essere espresso come un'unione di insiemi disgiunti o come un prodotto cartesiano di insiemi.

La ***regola della somma*** dice che il numero di volte in cui si può scegliere un elemento da uno di due insiemi *disgiunti* è pari alla somma delle cardinalità degli insiemi. Ovvero, se A e B sono due insiemi finiti senza elementi in comune, allora $|A \cup B| = |A| + |B|$, che segue dall'equazione (B.3). Per esempio, ogni posizione nella targa di un'automobile può essere una lettera o una cifra. Il numero di possibilità per ogni posizione è quindi $26 + 10 = 36$, perché ci sono 26 scelte per ogni lettera e 10 scelte per ogni cifra.

Acquistato da Michele Michele su Webster il 2022-07-21 per uso personale. Vietata la copia. ISBN 9788838620707-0 Copyright © 2022, McGraw-Hill Education (Italy)

La ***regola del prodotto*** stabilisce che il numero di modi in cui si può scegliere una coppia ordinata è pari al numero di modi in cui si può scegliere il primo elemento per il numero di modi in cui si può scegliere il secondo elemento. Ovvero, se A e B sono due insiemi finiti, allora $|A \times B| = |A| \cdot |B|$, che è l'equazione (B.4). Per esempio, se una gelateria prepara 28 gusti di gelati e 4 guarniture, il numero di coppe possibili con una pallina di gelato e una guarnitura è $28 \cdot 4 = 112$.

Stringhe

Una ***stringa*** in un insieme finito S è una sequenza di elementi di S. Per esempio, ci sono 8 stringhe binarie di lunghezza 3:

$000, 001, 010, 011, 100, 101, 110, 111$

Una stringa di lunghezza k è detta anche ***k-stringa***. Una ***sottostringa*** s' di una stringa s è una sequenza ordinata di elementi consecutivi di s. Una ***k-sottostringa*** di una stringa è una sottostringa di lunghezza k. Per esempio, 010 è una 3-sottostringa di 01101001 (la 3-sottostringa che inizia nella posizione 4), ma 111 non è una sottostringa di 01101001.

Una k-stringa in un insieme S può essere vista come un elemento del prodotto cartesiano S^k costituito dalle k-tuple; quindi, ci sono $|S|^k$ stringhe di lunghezza k. Per esempio, il numero di k-stringhe binarie è 2^k. Intuitivamente, per costruire una k-stringa in un n-insieme, ci sono n modi di scegliere il primo elemento; per ciascuna di queste scelte, ci sono n modi di scegliere il secondo elemento e così via per k volte. Questa costruzione porta al prodotto di k fattori $n \cdot n \cdots n = n^k$ come numero di k-stringhe.

Permutazioni

Una ***permutazione*** di un insieme finito S è una sequenza ordinata di tutti gli elementi di S, con ogni elemento che appare una sola volta. Per esempio, se $S = \{a, b, c\}$, ci sono 6 permutazioni di S:

$abc, acb, bac, bca, cab, cba$

Ci sono $n!$ permutazioni di un insieme di n elementi, perché il primo elemento della sequenza può essere scelto in n modi, il secondo in $n - 1$ modi, il terzo in $n - 2$ modi e così via.

Una ***k-permutazione*** di S è una sequenza ordinata di k elementi di S, che non contiene elementi duplicati (quindi, una permutazione ordinaria è proprio una n-permutazione di un n-insieme). Le dodici 2-permutazioni dell'insieme $\{a, b, c, d\}$ sono

$ab, ac, ad, ba, bc, bd, ca, cb, cd, da, db, dc$

Il numero di k-permutazioni di un n-insieme è

$$n(n-1)(n-2)\cdots(n-k+1) = \frac{n!}{(n-k)!} \tag{C.1}$$

perché ci sono n modi di scegliere il primo elemento, $n - 1$ modi di scegliere il secondo elemento e così via fino a scegliere k elementi, l'ultimo dei quali viene scelto tra $n - k + 1$ elementi.

Combinazioni

Acquistato da Michele Michele su Webster il 2022-07-07 23:12, Numero Ordine Libreria 119503016, 230707 - © Copyright © 2022, McGraw-Hill Education (Italy)

Una ***k-combinazione*** di un n-insieme S è semplicemente un k-sottoinsieme di S. Per esempio, ci sono sei 2-combinazioni del 4-insieme $\{a, b, c, d\}$:

ab, ac, ad, bc, bd, cd

(Qui possiamo usare l'abbreviazione ab per indicare il 2-insieme $\{a, b\}$ e così via.) Possiamo costruire una k-combinazione di un n-insieme scegliendo k distinti (differenti) elementi dall'n-insieme.

Il numero di k-combinazioni di un n-insieme può essere espresso in funzione del numero di k-permutazioni di un n-insieme. Per ogni k-combinazione, ci sono esattamente $k!$ permutazioni dei suoi elementi, ciascuno dei quali è una k-permutazione distinta dell'n-insieme. Quindi, il numero di k-combinazioni di un n-insieme è il numero di k-permutazioni diviso per $k!$; dall'equazione (C.1), questa quantità è

$$\frac{n!}{k!\,(n-k)!} \tag{C.2}$$

Per $k = 0$, questa formula ci dice che il numero di modi in cui possiamo scegliere 0 elementi da un n-insieme è 1 (non 0), perché $0! = 1$.

Coefficienti binomiali

Utilizziamo la notazione $\binom{n}{k}$ (che si legge "n su k") per indicare il numero di k-combinazioni di un n-insieme. Dall'equazione (C.2), si ha

$$\binom{n}{k} = \frac{n!}{k!\,(n-k)!}$$

Questa formula è simmetrica in k ed $n-k$:

$$\binom{n}{k} = \binom{n}{n-k} \tag{C.3}$$

Questi numeri sono anche detti ***coefficienti binomiali***, perché appaiono nello ***sviluppo in serie binomiale***:

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^k y^{n-k} \tag{C.4}$$

Un caso speciale di sviluppo in serie binomiale si ha quando $x = y = 1$:

$$2^n = \sum_{k=0}^{n} \binom{n}{k}$$

Questa formula equivale a contare le 2^n n-stringhe binarie raggruppandole per il numero di cifre 1 che contengono: ci sono $\binom{n}{k}$ n-stringhe binarie che contengono esattamente k cifre 1, perché ci sono $\binom{n}{k}$ modi di scegliere k delle n posizioni in cui porre le cifre 1.

Ci sono varie identità che riguardano i coefficienti binomiali. Gli esercizi alla fine di questo paragrafo vi offrono l'opportunità di provarne alcune.

Limiti per i binomiali

In alcuni casi è necessario limitare il valore di un coefficiente binomiale. Per $1 \le k \le n$, abbiamo il limite inferiore

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned}
\binom{n}{k} &= \frac{n(n-1)\cdots(n-k+1)}{k(k-1)\cdots 1} \\
&= \left(\frac{n}{k}\right)\left(\frac{n-1}{k-1}\right)\cdots\left(\frac{n-k+1}{1}\right) \\
&\ge \left(\frac{n}{k}\right)^k
\end{aligned}$$

Applicando la disuguaglianza $k! \geq (k/e)^k$ derivata dall'approssimazione di Stirling (3.18), otteniamo i limiti superiori

$$\begin{aligned}\binom{n}{k} &= \frac{n(n-1)\cdots(n-k+1)}{k(k-1)\cdots 1} \\ &\leq \frac{n^k}{k!} \\ &\leq \left(\frac{en}{k}\right)^k \qquad \text{(C.5)}\end{aligned}$$

Per ogni $0 \leq k \leq n$, utilizziamo l'induzione (vedere l'Esercizio C.1-12) per dimostrare il limite

$$\binom{n}{k} \leq \frac{n^n}{k^k (n-k)^{n-k}} \qquad \text{(C.6)}$$

dove per comodità supponiamo che $0^0 = 1$. Per $k = \lambda n$, dove $0 \leq \lambda \leq 1$, questo limite può essere riscritto così

$$\begin{aligned}\binom{n}{\lambda n} &\leq \frac{n^n}{(\lambda n)^{\lambda n}((1-\lambda)n)^{(1-\lambda)n}} \\ &= \left(\left(\frac{1}{\lambda}\right)^{\lambda}\left(\frac{1}{1-\lambda}\right)^{1-\lambda}\right)^n \\ &= 2^{n\,H(\lambda)}\end{aligned}$$

dove

$$H(\lambda) = -\lambda \lg \lambda - (1-\lambda)\lg(1-\lambda) \qquad \text{(C.7)}$$

è la ***funzione entropia (binaria)*** e dove, per comodità, supponiamo che $0 \lg 0 = 0$, in modo che $H(0) = H(1) = 0$.

Esercizi

C.1-1
Quante k-sottostringhe ha una n-stringa? (Considerate differenti le k-sottostringhe identiche che si trovano in posizioni differenti.) Quante sottostringhe ha in totale una n-stringa?

C.1-2
Una ***funzione booleana*** con n input ed m output è una funzione da $\{\text{TRUE}, \text{FALSE}\}^n$ a $\{\text{TRUE}, \text{FALSE}\}^m$. Quante sono le funzioni booleane con n input e 1 output? Quante sono le funzioni booleane con n input ed m output?

C.1-3
In quanti modi n professori possono sedersi attorno a un tavolo circolare? Considerate uguali due disposizioni, se una può essere ottenuta dall'altra con una rotazione.

C.1-4
Quanti modi ci sono di scegliere dall'insieme $\{1, 2, \ldots, 99\}$ tre numeri distinti tali che la loro somma sia pari?

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

C.1-5
Dimostrate l'identità

$$\binom{n}{k} = \frac{n}{k}\binom{n-1}{k-1} \tag{C.8}$$

per $0 < k \leq n$.

C.1-6
Dimostrate l'identità

$$\binom{n}{k} = \frac{n}{n-k}\binom{n-1}{k}$$

per $0 \leq k < n$.

C.1-7
Per scegliere k oggetti tra n, potete rendere distinguibile uno degli oggetti e considerare il caso in cui sia scelto questo oggetto. Utilizzate questo approccio per dimostrare che

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

C.1-8
Utilizzando il risultato ottenuto nell'Esercizio C.1-7 create, per $n = 0, 1, \ldots, 6$ e $0 \leq k \leq n$, una tabella di coefficienti binomiali $\binom{n}{k}$, con $\binom{0}{0}$ in alto, $\binom{1}{0}$ e $\binom{1}{1}$ nella riga successiva e così via. Questa tabella di coefficienti binomiali è detta ***triangolo di Pascal***.

C.1-9
Dimostrate che

$$\sum_{i=1}^{n} i = \binom{n+1}{2}$$

C.1-10
Dimostrate che per qualsiasi $n \geq 0$ e $0 \leq k \leq n$, il valore massimo di $\binom{n}{k}$ si ottiene quando $k = \lfloor n/2 \rfloor$ o $k = \lceil n/2 \rceil$.

C.1-11 ⋆
Dimostrate che per qualsiasi $n \geq 0$, $j \geq 0$, $k \geq 0$ e $j + k \leq n$, si ha

$$\binom{n}{j+k} \leq \binom{n}{j}\binom{n-j}{k} \tag{C.9}$$

Fornite una dimostrazione algebrica giustificandola con un ragionamento basato su un metodo per scegliere $j + k$ elementi tra n. Trovate un esempio in cui l'uguaglianza non è soddisfatta.

Acquistato da Michele Michele su Webster il 2022-07-05 23:43:13 (protocollo Licensing 1995301 - 2022 CC BY-NC-ND - Vietata ogni Distribuzione)

C.1-12 ⋆
Utilizzate l'induzione su ogni intero k, tale che $0 \leq k \leq n/2$, per dimostrare la disuguaglianza (C.6) e utilizzate l'equazione (C.3) per estenderla a ogni intero k, tale che $0 \leq k \leq n$.

C.1-13 ⋆
Utilizzate l'approssimazione di Stirling per dimostrare che

$$\binom{2n}{n} = \frac{2^{2n}}{\sqrt{\pi n}}(1 + O(1/n)) \tag{C.10}$$

C.1-14 ⋆
Derivate la funzione entropia $H(\lambda)$ per dimostrare che è massima in $\lambda = 1/2$. Quanto vale $H(1/2)$?

C.1-15 ⋆
Dimostrate che per qualsiasi intero $n \geq 0$, si ha

$$\sum_{k=0}^{n} \binom{n}{k} k = n2^{n-1} \tag{C.11}$$

C.2 Probabilità

Il calcolo delle probabilità è uno strumento essenziale per la progettazione e l'analisi degli algoritmi probabilistici e randomizzati. Questo paragrafo esamina i concetti fondamentali della teoria della probabilità.

Definiamo la probabilità nei termini di uno ***spazio dei campioni*** S, che è un insieme i cui elementi sono chiamati ***eventi elementari***. Ogni evento elementare può essere considerato come un possibile esito di un esperimento. Per l'esperimento del lancio di due monete distinguibili, lo spazio dei campioni sarà costituito dall'insieme di tutte le possibili 2-stringhe in $\{\text{T}, \text{C}\}$:

$$S = \{\text{TT}, \text{TC}, \text{CT}, \text{CC}\}$$

Un ***evento*** è un sottoinsieme[1] dello spazio dei campioni S. Per esempio, nell'esperimento del lancio delle due monete, l'evento di ottenere una testa e una croce è $\{\text{TC}, \text{CT}\}$. L'evento S è detto ***evento certo*** e l'evento $\emptyset$ è detto ***evento nullo***. Diciamo che due eventi A e B sono ***mutuamente esclusivi*** se $A \cap B = \emptyset$. A volte trattiamo un evento elementare $s \in S$ come l'evento $\{s\}$. Per definizione, tutti gli eventi elementari sono mutuamente esclusivi.

Assiomi di probabilità

Una ***distribuzione di probabilità*** $\Pr\{\}$ in uno spazio di campioni S è una relazione che associa gli eventi di S ai numeri reali, in modo tale che siano soddisfatti i seguenti ***assiomi di probabilità***:

[1] Per una distribuzione di probabilità generica, ci può essere qualche sottoinsieme dello spazio dei campioni S che non è considerato evento. Questa situazione, di solito, si presenta quando lo spazio dei campioni è un insieme infinito non numerabile. Il requisito principale è che l'insieme degli eventi dello spazio dei campioni sia chiuso rispetto alle operazioni per calcolare il complemento di un evento, per formare l'unione di un numero finito o numerabile di eventi e per determinare l'intersezione di un numero finito o numerabile di eventi. La maggior parte delle distribuzioni di probabilità che esamineremo si trovano in spazi di campioni finiti o numerabili e, in generale, considereremo eventi tutti i sottoinsiemi di un determinato spazio di campioni. Un'eccezione importante è la distribuzione di probabilità uniforme e continua, che sarà descritta più avanti.

Acquistato da Michele Mishelli su MyBesterly 2022-07-07 23:23:13 Numero Ordine Libraria 1005060016230707 © Copyright © 2022 McGraw-Hill Education (Italy)

1. $\Pr\{A\} \geq 0$ per qualsiasi evento A.
2. $\Pr\{S\} = 1$.
3. $\Pr\{A \cup B\} = \Pr\{A\} + \Pr\{B\}$ per due eventi qualsiasi mutuamente esclusivi A e B. Più in generale, per qualsiasi sequenza (finita o infinita numerabile) di eventi $A_1, A_2, \ldots$ che sono mutuamente esclusivi a coppie, si ha

$$\Pr\left\{\bigcup_i A_i\right\} = \sum_i \Pr\{A_i\}$$

Chiamiamo $\Pr\{A\}$ la ***probabilità*** dell'evento A. Notiamo che l'assioma 2 è un requisito di normalizzazione: in effetti non c'è alcun motivo di scegliere 1 come probabilità dell'evento certo, tranne che ciò è naturale e comodo.

Da questi assiomi e dalla teoria degli insiemi (vedere il Paragrafo B.1) derivano vari risultati. L'evento nullo $\emptyset$ ha probabilità $\Pr\{\emptyset\} = 0$. Se $A \subseteq B$, allora $\Pr\{A\} \leq \Pr\{B\}$. Utilizzando $\overline{A}$ per indicare l'evento $S - A$ (il ***complemento*** di A), si ha $\Pr\{\overline{A}\} = 1 - \Pr\{A\}$. Per due eventi qualsiasi A e B, abbiamo

$$\begin{aligned} \Pr\{A \cup B\} &= \Pr\{A\} + \Pr\{B\} - \Pr\{A \cap B\} && \text{(C.12)} \\ &\leq \Pr\{A\} + \Pr\{B\} && \text{(C.13)} \end{aligned}$$

Nell'esempio del lancio delle monete, supponiamo che ciascuno dei quattro eventi elementari abbia probabilità $1/4$. Allora la probabilità di ottenere almeno una testa è

$$\begin{aligned} \Pr\{\text{TT}, \text{TC}, \text{CT}\} &= \Pr\{\text{TT}\} + \Pr\{\text{TC}\} + \Pr\{\text{CT}\} \\ &= 3/4 \end{aligned}$$

In alternativa, poiché la probabilità di non ottenere una testa è $\Pr\{\text{CC}\} = 1/4$, la probabilità di ottenere almeno una testa è $1 - 1/4 = 3/4$.

Distribuzioni di probabilità discrete

Una distribuzione di probabilità è ***discreta*** se è definita in uno spazio di campioni finito o infinito numerabile. Sia S lo spazio dei campioni. Allora per ogni evento A, si ha

$$\Pr\{A\} = \sum_{s \in A} \Pr\{s\}$$

perché gli eventi elementari, specificatamente quelli in A, sono mutuamente esclusivi. Se S è finito e ogni evento elementare $s \in S$ ha probabilità

$$\Pr\{s\} = 1/|S|$$

allora si ha una ***distribuzione di probabilità uniforme*** in S. In questo caso, l'esperimento spesso è descritto così: "scegliere a caso un elemento di S".

Acquistato da Michele Michele su Webster il 2022-[illegible] - Copia personale non trasferibile - Copyright © 2022 The McGraw-Hill Education (Italy)

Come esempio consideriamo il lancio di una ***moneta perfetta***, una moneta per la quale la probabilità di ottenere testa è uguale a quella di ottenere croce, cioè $1/2$. Se lanciamo la moneta n volte, abbiamo la distribuzione di probabilità uniforme definita nello spazio dei campioni $S = \{\text{T}, \text{C}\}^n$, un insieme di dimensione 2^n. Ogni evento elementare in S può essere rappresentato come una stringa di lunghezza n in $\{\text{T}, \text{C}\}$; inoltre ogni evento si verifica con probabilità $1/2^n$. L'evento

$A = \{\text{si ottengono esattamente } k \text{ teste ed } n - k \text{ croci}\}$

è un sottoinsieme di S di dimensione $|A| = \binom{n}{k}$, perché ci sono $\binom{n}{k}$ stringhe di lunghezza n in $\{\text{T}, \text{C}\}$ che contengono esattamente k teste (T). La probabilità dell'evento A è quindi $\Pr\{A\} = \binom{n}{k}/2^n$.

Distribuzione di probabilità uniforme e continua

La distribuzione di probabilità uniforme e continua è un esempio di distribuzione di probabilità in cui non tutti i sottoinsiemi dello spazio dei campioni sono considerati eventi. La distribuzione di probabilità uniforme e continua è definita in un intervallo chiuso $[a, b]$ dei numeri reali, dove $a < b$. Intuitivamente, vogliamo che ogni punto nell'intervallo $[a, b]$ sia "ugualmente probabile". Tuttavia, c'è un insieme di punti non numerabile, quindi se diamo a tutti i punti la stessa probabilità positiva e finita, non possiamo soddisfare contemporaneamente gli assiomi 2 e 3. Per questo motivo, associamo una probabilità soltanto a *qualche* sottoinsieme di S in modo tale che gli assiomi siano soddisfatti per questi eventi.

Per qualsiasi intervallo chiuso $[c, d]$, dove $a \le c \le d \le b$, la ***distribuzione di probabilità uniforme e continua*** definisce la probabilità dell'evento $[c, d]$ come

$$\Pr\{[c, d]\} = \frac{d - c}{b - a}$$

Notate che per qualsiasi punto $x = [x, x]$, la probabilità di x è 0. Se eliminiamo gli estremi di un intervallo $[c, d]$, otteniamo l'intervallo aperto (c, d). Poiché $[c, d] = [c, c] \cup (c, d) \cup [d, d]$, l'assioma 3 fornisce $\Pr\{[c, d]\} = \Pr\{(c, d)\}$. In generale, l'insieme degli eventi per una distribuzione di probabilità uniforme e continua è qualsiasi sottoinsieme dello spazio dei campioni $[a, b]$ che può essere ottenuto dall'unione finita e numerabile di intervalli aperti e chiusi.

Probabilità condizionata e indipendenza

A volte è possibile avere a priori qualche informazione sull'esito di un esperimento. Per esempio, supponiamo che un amico abbia lanciato due monete perfette e ci abbia detto che almeno una delle monete ha presentato testa. Qual è la probabilità che entrambe le monete presentino testa? L'informazione che abbiamo ricevuto esclude la possibilità che le monete presentino due croci. I tre restanti eventi elementari sono ugualmente probabili, quindi possiamo dedurre che ciascun evento si verifichi con probabilità $1/3$. Poiché soltanto uno di questi eventi elementari mostra due teste, la risposta alla domanda è $1/3$.

La probabilità condizionata formalizza il concetto di avere a priori qualche informazione sull'esito di un esperimento. La ***probabilità condizionata*** di un evento A, sapendo che si è verificato l'evento B, è definita come

$$\Pr\{A \mid B\} = \frac{\Pr\{A \cap B\}}{\Pr\{B\}} \tag{C.14}$$

con $\Pr\{B\} \neq 0$. "$\Pr\{A \mid B\}$" si legge "la probabilità di A dato B". Intuitivamente, poiché sappiamo che l'evento B si verifica, l'evento che si verifichi anche A è $A \cap B$. Ovvero, $A \cap B$ è l'insieme dei risultati in cui si verificano entrambi gli eventi A e B. Poiché il risultato è uno degli eventi elementari in B, normalizziamo le probabilità di tutti gli eventi elementari in B dividendole per $\Pr\{B\}$, in modo che la loro somma sia 1. La probabilità condizionata di A dato B è, quindi, il rapporto tra la probabilità dell'evento $A \cap B$ e la probabilità

Acquistato da Michele Mostardi 2022-01-03 13:12 Numero licenza Libraria 2022010400 Copyright © 2010 McGraw-Hill Education (Italy)

dell'evento B. Nel precedente esempio, A è l'evento che entrambe le monete presentino testa e B è l'evento che almeno una moneta presenti testa. Quindi, $\Pr\{A \mid B\} = (1/4)/(3/4) = 1/3$.
Due eventi sono ***indipendenti*** se

$$\Pr\{A \cap B\} = \Pr\{A\}\Pr\{B\} \tag{C.15}$$

che equivale, se $\Pr\{B\} \neq 0$, alla condizione

$$\Pr\{A \mid B\} = \Pr\{A\}$$

Per esempio, supponiamo che vengano lanciate due monete perfette e che i risultati siano indipendenti; la probabilità che si presentino due teste è $(1/2)(1/2) = 1/4$. Adesso consideriamo un evento in cui la prima moneta presenta testa e un altro evento in cui le monete presentano risultati differenti. Ciascuno di questi eventi si verifica con probabilità $1/2$ e la probabilità che si verifichino entrambi gli eventi è $1/4$; quindi, secondo la definizione di indipendenza, gli eventi sono indipendenti – anche se possiamo ritenere che entrambi gli eventi dipendano dalla prima moneta. Infine, supponiamo che le due monete siano saldate insieme, in modo che esse presentino entrambe testa o entrambe croce e che i due eventi siano ugualmente probabili. Allora la probabilità che ogni moneta presenti testa è $1/2$, ma la probabilità che entrambe presentino testa è $1/2 \neq (1/2)(1/2)$. Di conseguenza, l'evento che una moneta presenti testa e l'evento che l'altra presenti testa non sono indipendenti.

Gli eventi di una collezione $A_1, A_2, \ldots, A_n$ sono ***indipendenti a coppie*** se

$$\Pr\{A_i \cap A_j\} = \Pr\{A_i\}\Pr\{A_j\}$$

per ogni $1 \leq i < j \leq n$. Diciamo che gli eventi della collezione sono ***(mutuamente) indipendenti*** se ogni k-sottoinsieme $A_{i_1}, A_{i_2}, \ldots, A_{i_k}$ della collezione, dove $2 \leq k \leq n$ e $1 \leq i_1 < i_2 < \cdots < i_k \leq n$, soddisfa la relazione

$$\Pr\{A_{i_1} \cap A_{i_2} \cap \cdots \cap A_{i_k}\} = \Pr\{A_{i_1}\}\Pr\{A_{i_2}\}\cdots\Pr\{A_{i_k}\}$$

Per esempio, supponiamo di lanciare due monete perfette. Sia A_1 l'evento in cui la prima moneta presenta testa, sia A_2 l'evento in cui la seconda moneta presenta testa e sia A_3 l'evento in cui le due monete sono differenti. Abbiamo

$$\begin{aligned}
\Pr\{A_1\} &= 1/2 \\
\Pr\{A_2\} &= 1/2 \\
\Pr\{A_3\} &= 1/2 \\
\Pr\{A_1 \cap A_2\} &= 1/4 \\
\Pr\{A_1 \cap A_3\} &= 1/4 \\
\Pr\{A_2 \cap A_3\} &= 1/4 \\
\Pr\{A_1 \cap A_2 \cap A_3\} &= 0
\end{aligned}$$

Poiché per $1 \leq i < j \leq 3$, si ha $\Pr\{A_i \cap A_j\} = \Pr\{A_i\}\Pr\{A_j\} = 1/4$, gli eventi A_1, A_2 e A_3 sono indipendenti a coppie, ma non sono mutuamente indipendenti, perché $\Pr\{A_1 \cap A_2 \cap A_3\} = 0$ e $\Pr\{A_1\}\Pr\{A_2\}\Pr\{A_3\} = 1/8 \neq 0$.

Acquistato da Michele Michele su Webster il 2022-07-07 28:12. Numero Ordine Libreria 1008302016-220707-0. Copyright © 2022, McGraw-Hill Education (Italy)

Teorema di Bayes

Dalla definizione di probabilità condizionata (C.14) e dalla legge commutativa $A \cap B = B \cap A$ segue che, per due eventi A e B, ciascuno con probabilità non nulla, si ha

$$\begin{aligned} \Pr\{A \cap B\} &= \Pr\{B\}\Pr\{A \mid B\} \\ &= \Pr\{A\}\Pr\{B \mid A\} \end{aligned} \tag{C.16}$$

Risolvendo rispetto a $\Pr\{A \mid B\}$, si ha

$$\Pr\{A \mid B\} = \frac{\Pr\{A\}\Pr\{B \mid A\}}{\Pr\{B\}} \tag{C.17}$$

Questa formula è nota come ***teorema di Bayes***. Il denominatore $\Pr\{B\}$ è una costante di normalizzazione che possiamo esprimere nel modo seguente. Poiché $B = (B \cap A) \cup (B \cap \overline{A})$ e $B \cap A$ e $B \cap \overline{A}$ sono eventi mutuamente esclusivi, si ha

$$\begin{aligned} \Pr\{B\} &= \Pr\{B \cap A\} + \Pr\left\{B \cap \overline{A}\right\} \\ &= \Pr\{A\}\Pr\{B \mid A\} + \Pr\left\{\overline{A}\right\}\Pr\left\{B \mid \overline{A}\right\} \end{aligned}$$

Sostituendo questa formula nell'equazione (C.17), otteniamo una forma equivalente del teorema di Bayes:

$$\Pr\{A \mid B\} = \frac{\Pr\{A\}\Pr\{B \mid A\}}{\Pr\{A\}\Pr\{B \mid A\} + \Pr\left\{\overline{A}\right\}\Pr\left\{B \mid \overline{A}\right\}} \tag{C.18}$$

Il teorema di Bayes può semplificare il calcolo delle probabilità condizionate. Per esempio, supponiamo di avere una moneta perfetta e una moneta truccata che presenta sempre testa. Eseguiamo un esperimento composto da tre eventi indipendenti: una delle due monete è scelta a caso, la moneta viene lanciata una prima volta e poi una seconda volta. Supponiamo che la moneta scelta presenti testa entrambe le volte. Qual è la probabilità che questa moneta sia quella truccata?

Risolviamo il problema utilizzanto il teorema di Bayes. Sia A l'evento in cui viene scelta la moneta truccata e sia B l'evento in cui la moneta presenta testa entrambe le volte. Vogliamo determinare $\Pr\{A \mid B\}$. Abbiamo $\Pr\{A\} = 1/2$, $\Pr\{B \mid A\} = 1$, $\Pr\left\{\overline{A}\right\} = 1/2$ e $\Pr\left\{B \mid \overline{A}\right\} = 1/4$; quindi

$$\begin{aligned} \Pr\{A \mid B\} &= \frac{(1/2) \cdot 1}{(1/2) \cdot 1 + (1/2) \cdot (1/4)} \\ &= 4/5 \end{aligned}$$

Esercizi

C.2-1

Il professor Rosencrantz lancia una moneta perfetta una sola volta. Il professor Guildenstern lancia una moneta perfetta due volte. Qual è la probabilità che il professor Rosencrantz ottenga più teste del professor Guildenstern?

C.2-2

Dimostrate la ***disuguaglianza di Boole***: per qualsiasi sequenza finita o infinita numerabile di eventi $A_1, A_2, \ldots$,

Acquistato da Michele Mu... - Licenza d'uso n. 2022-7-47-23423 - Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\Pr\{A_1 \cup A_2 \cup \cdots\} \leq \Pr\{A_1\} + \Pr\{A_2\} + \cdots \tag{C.19}$$

C.2-3

Un mazzo di 10 carte, ciascuna con un numero distinto da 1 a 10, viene mescolato a lungo per mischiare bene le carte. Tre carte vengono rimosse una alla volta dal mazzo. Qual è la probabilità che le tre carte siano scelte in ordine (crescente)?

C.2-4
Dimostrate che

$$\Pr\{A \mid B\} + \Pr\left\{\overline{A} \mid B\right\} = 1$$

C.2-5
Dimostrate che per qualsiasi collezione di eventi $A_1, A_2, \ldots, A_n$, si ha

$$\Pr\{A_1 \cap A_2 \cap \cdots \cap A_n\} = \Pr\{A_1\} \cdot \Pr\{A_2 \mid A_1\} \cdot \Pr\{A_3 \mid A_1 \cap A_2\} \cdots \Pr\{A_n \mid A_1 \cap A_2 \cap \cdots \cap A_{n-1}\}$$

C.2-6 ⋆
Descrivete una procedura che riceve come input due interi a e b tali che $0 < a < b$ e, utilizzando i lanci di una moneta perfetta, produce come output testa con probabilità a/b e croce con probabilità $(b-a)/b$. Calcolate un limite per il numero atteso di lanci della moneta, che dovrebbe essere $O(1)$ (*suggerimento:* rappresentate a/b in binario).

C.2-7 ⋆
Spiegate come costruire un insieme di n eventi che sono indipendenti a coppie, ma tale che nessun sottoinsieme con $k > 2$ di tali eventi sia mutuamente indipendente.

C.2-8 ⋆
Due eventi A e B sono ***indipendenti in modo condizionato***, dato C, se

$$\Pr\{A \cap B \mid C\} = \Pr\{A \mid C\} \cdot \Pr\{B \mid C\}$$

Trovate un semplice (non banale) esempio di due eventi che non sono indipendenti, ma che sono indipendenti in modo condizionato dato un terzo evento.

C.2-9 ⋆
Supponete di partecipare a un gioco televisivo in cui un premio è nascosto dietro tre tendine. Se scegliete la tendina giusta, vincerete il premio. Dopo che voi avete scelto una tendina, ma prima di alzarla, il conduttore del gioco alza una delle altre due tendine, sapendo di svelare una scena vuota, e vi chiede se volete cambiare la tendina che avete scelto con quella restante. Se cambiate tendina, come varia la vostra probabilità di vincere il premio?

C.2-10 ⋆
Il direttore di un carcere sceglie a caso un prigioniero fra tre per liberarlo; gli altri due saranno giustiziati. Il secondino sa chi sarà liberato, ma non gli è permesso dare a qualsiasi prigioniero alcuna informazione sulla sua sorte. Chiamiamo i prigionieri X, Y e Z. Il prigioniero X chiede in privato al secondino chi sarà giustiziato tra Y e Z, dimostrandogli che, se risponderà, egli non gli rivelerà alcuna informazione sulla sua sorte, poiché sa già che o Y o Z dovrà morire. Il secondino confida a X che Y sarà giustiziato. Il prigioniero X adesso è un po' più contento, perché sa che o lui o il prigioniero Z sarà liberato; ciò significa che la sua probabilità di essere liberato adesso è $1/2$. Ha ragione o tale probabilità resta $1/3$? Spiegate la risposta.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

C.3 Variabili casuali discrete

Una ***variabile casuale (discreta)*** X è una funzione da uno spazio di campioni S, finito o infinito numerabile, ai numeri reali. Associa un numero reale a ogni possibile risultato di un esperimento; questo ci consente di operare con la distribuzione

di probabilità indotta sull'insieme risultante dei numeri. Le variabili casuali possono essere definite anche per spazi di campioni infiniti non numerabili, ma ciò implica problemi tecnici che non è necessario risolvere per i nostri scopi. D'ora in poi, quindi, assumeremo che le variabili casuali siano discrete.

Per una variabile casuale X e un numero reale x, definiamo l'evento $X = x$ come $\{s \in S : X(s) = x\}$; quindi, si ha

$$\Pr\{X = x\} = \sum_{s \in S : X(s) = x} \Pr\{s\}$$

La funzione

$$f(x) = \Pr\{X = x\}$$

è la ***funzione densità di probabilità*** della variabile casuale X. Dagli assiomi di probabilità, si ha $\Pr\{X = x\} \geq 0$ e $\sum_x \Pr\{X = x\} = 1$. Come esempio consideriamo l'esperimento di lanciare due dadi normali a 6 facce. Ci sono 36 possibili eventi elementari nello spazio dei campioni. Supponiamo che la distribuzione di probabilità sia uniforme, cosicché ogni evento elementare $s \in S$ sia ugualmente probabile: $\Pr\{s\} = 1/36$. Definiamo la variabile casuale X come il *massimo* tra i due valori che presentano i dadi. Si ha $\Pr\{X = 3\} = 5/36$, perché X assegna il valore 3 a 5 dei 36 possibili eventi elementari, ovvero $(1, 3)$, $(2, 3)$, $(3, 3)$, $(3, 2)$ e $(3, 1)$. È normale che molte variabili casuali siano definite nello stesso spazio dei campioni. Se X e Y sono variabili casuali, la funzione

$$f(x, y) = \Pr\{X = x \text{ e } Y = y\}$$

è la ***funzione densità di probabilità congiunta*** di X e Y. Per un valore prefissato y, si ha

$$\Pr\{Y = y\} = \sum_x \Pr\{X = x \text{ e } Y = y\}$$

e analogamente, per un valore prefissato x, si ha

$$\Pr\{X = x\} = \sum_y \Pr\{X = x \text{ e } Y = y\}$$

Utilizzando la definizione (C.14) della probabilità condizionata, si ha

$$\Pr\{X = x \mid Y = y\} = \frac{\Pr\{X = x \text{ e } Y = y\}}{\Pr\{Y = y\}}$$

Definiamo ***indipendenti*** due variabili casuali X e Y, se per ogni x e y, gli eventi $X = x$ e $Y = y$ sono indipendenti ovvero se, per ogni x e y, si ha $\Pr\{X = x \text{ e } Y = y\} = \Pr\{X = x\}\Pr\{Y = y\}$.

Dato un insieme di variabili casuali definito nello stesso spazio dei campioni, è possibile definire nuove variabili casuali, come somme, prodotti o altre funzioni delle variabili originali.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Valore atteso di una variabile casuale

La più semplice e utile sintesi della distribuzione di una variabile casuale è la "media" dei valori che assume. Il ***valore atteso*** (***media*** o ***aspettazione***) di una variabile casuale discreta X è

$$\mathrm{E}[X] = \sum_x x \Pr\{X = x\} \tag{C.20}$$

Il valore atteso è ben definito se la sommatoria è finita o converge in modo assoluto. A volte il valore atteso di X è indicato con μ_X o, quando la variabile casuale è evidente dal contesto, semplicemente con μ.

Consideriamo un gioco in cui vengono lanciate due monete uguali. Vincete 3 euro per ogni testa, ma perdete 2 euro per ogni croce. Il valore atteso della variabile casuale X che rappresenta la vostra vincita è

$$\begin{aligned} \mathrm{E}[X] &= 6 \cdot \Pr\{2\ \mathrm{T}\} + 1 \cdot \Pr\{1\ \mathrm{T},\ 1\ \mathrm{C}\} - 4 \cdot \Pr\{2\ \mathrm{C}\} \\ &= 6(1/4) + 1(1/2) - 4(1/4) \\ &= 1 \end{aligned}$$

Il valore atteso della somma di due variabili casuali è la somma dei loro valori attesi, ovvero

$$\mathrm{E}[X + Y] = \mathrm{E}[X] + \mathrm{E}[Y] \tag{C.21}$$

se $\mathrm{E}[X]$ e $\mathrm{E}[Y]$ sono definiti. Questa proprietà è detta ***linearità del valore atteso***; è valida anche se X e Y non sono indipendenti. La proprietà si estende anche alle sommatorie, finite e convergenti in modo assoluto, di valori attesi. La linearità del valore atteso è la proprietà fondamentale che ci consente di eseguire l'analisi probabilistica utilizzando le variabili casuali indicatrici (vedere il Paragrafo 5.2).

Se X è una variabile casuale, qualsiasi funzione $g(x)$ definisce una nuova variabile casuale $g(X)$. Se il valore atteso di $g(X)$ è definito, allora

$$\mathrm{E}[g(X)] = \sum_x g(x)\ \Pr\{X = x\}$$

Ponendo $g(x) = ax$, per una qualsiasi costante a, si ha

$$\mathrm{E}[aX] = a\mathrm{E}[X] \tag{C.22}$$

Di conseguenza, i valori attesi sono lineari: per due variabili casuali qualsiasi X e Y e per una costante qualsiasi a, si ha

$$\mathrm{E}[aX + Y] = a\mathrm{E}[X] + \mathrm{E}[Y] \tag{C.23}$$

Quando due variabili casuali X e Y sono indipendenti e ciascuna ha un valore atteso definito, allora

$$\begin{aligned} \mathrm{E}[XY] &= \sum_x \sum_y xy \Pr\{X = x \text{ e } Y = y\} \\ &= \sum_x \sum_y xy \Pr\{X = x\} \Pr\{Y = y\} \\ &= \left(\sum_x x\ \Pr\{X = x\}\right)\left(\sum_y y\ \Pr\{Y = y\}\right) \\ &= \mathrm{E}[X]\mathrm{E}[Y] \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

In generale, quando n variabili casuali $X_1, X_2, \ldots, X_n$ sono mutuamente indipendenti, si ha

$$\mathrm{E}[X_1 X_2 \cdots X_n] = \mathrm{E}[X_1]\,\mathrm{E}[X_2] \cdots \mathrm{E}[X_n] \tag{C.24}$$

Quando una variabile casuale X assume i valori nell'insieme dei numeri naturali $\mathbb{N} = \{0, 1, 2, \ldots\}$, c'è un'interessante formula per il suo valore atteso

$$\begin{aligned} \mathrm{E}[X] &= \sum_{i=0}^{\infty} i \Pr\{X = i\} \\ &= \sum_{i=0}^{\infty} i(\Pr\{X \geq i\} - \Pr\{X \geq i+1\}) \\ &= \sum_{i=1}^{\infty} \Pr\{X \geq i\} \end{aligned} \tag{C.25}$$

perché ciascun termine $\Pr\{X \geq i\}$ viene aggiunto i volte e sottratto $i-1$ volte (tranne il termine $\Pr\{X \geq 0\}$, che viene aggiunto 0 volte e non viene mai sottratto).

Quando applichiamo una funzione convessa $f(x)$ a una variabile casuale X, la ***disuguaglianza di Jensen*** ci dice che

$$\mathrm{E}[f(X)] \geq f(\mathrm{E}[X]) \tag{C.26}$$

purché i valori attesi esistano e siano finiti. Una funzione $f(x)$ è ***convessa*** se, per ogni x e y e per ogni $0 \leq \lambda \leq 1$, si ha $f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda) f(y)$.

Varianza e deviazione standard

Il valore atteso di una variabile casuale non dice come sono "dispersi" i valori della variabile. Per esempio, se abbiamo le variabili casuali X e Y per cui $\Pr\{X = 1/4\} = \Pr\{X = 3/4\} = 1/2$ e $\Pr\{Y = 0\} = \Pr\{Y = 1\} = 1/2$, allora entrambi i valori $\mathrm{E}[X]$ e $\mathrm{E}[Y]$ sono $1/2$, anche se i valori effettivi assunti da Y sono più lontani dalla media rispetto ai valori effettivi assunti da X.

Il concetto di varianza esprime matematicamente come è probabile che siano dispersi i valori di una variabile casuale attorno alla media. La ***varianza*** di una variabile casuale X con media $\mathrm{E}[X]$ è

$$\begin{aligned} \mathrm{Var}[X] &= \mathrm{E}\left[(X - \mathrm{E}[X])^2\right] \\ &= \mathrm{E}\left[X^2 - 2X\mathrm{E}[X] + \mathrm{E}^2[X]\right] \\ &= \mathrm{E}\left[X^2\right] - 2\mathrm{E}[X\mathrm{E}[X]] + \mathrm{E}^2[X] \\ &= \mathrm{E}\left[X^2\right] - 2\mathrm{E}^2[X] + \mathrm{E}^2[X] \\ &= \mathrm{E}\left[X^2\right] - \mathrm{E}^2[X] \end{aligned} \tag{C.27}$$

La giustificazione delle uguaglianze $\mathrm{E}[\mathrm{E}^2[X]] = \mathrm{E}^2[X]$ e $\mathrm{E}[X\mathrm{E}[X]] = \mathrm{E}^2[X]$ è che $\mathrm{E}[X]$ non è una variabile casuale, ma semplicemente un numero reale, quindi si può applicare l'equazione (C.22) (con $a = \mathrm{E}[X]$). L'equazione (C.27) può essere riscritta per ottenere un'espressione per il valore atteso del quadrato di una variabile casuale:

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\mathrm{E}\left[X^2\right] = \mathrm{Var}[X] + \mathrm{E}^2[X] \tag{C.28}$$

La varianza di una variabile casuale X e la varianza di aX sono correlate (vedere l'Esercizio C.3-10) dall'equazione:

$$\mathrm{Var}[aX] = a^2 \mathrm{Var}[X]$$

Se X e Y sono variabili casuali indipendenti, si ha

$$\text{Var}[X + Y] = \text{Var}[X] + \text{Var}[Y]$$

In generale, se n variabili casuali $X_1, X_2, \ldots, X_n$ sono indipendenti a coppie, allora

$$\text{Var}\left[\sum_{i=1}^{n} X_i\right] = \sum_{i=1}^{n} \text{Var}[X_i] \tag{C.29}$$

La ***deviazione standard*** di una variabile casuale X è la radice quadrata non negativa della varianza di X. La deviazione standard di una variabile casuale X a volte è indicata con σ_X o semplicemente con σ, quando la variabile casuale X è evidente dal contesto. Secondo questa notazione, la varianza di X è indicata con σ^2.

Esercizi

C.3-1

Vengono lanciati due dadi normali a 6 facce. Qual è il valore atteso della somma dei due valori che presentano i dadi? Qual è il valore atteso del massimo tra i due valori che presentano i dadi?

C.3-2

Un array $A[1..n]$ contiene n numeri distinti che sono ordinati a caso, con ciascuna permutazione degli n numeri ugualmente probabile. Qual è il valore atteso dell'indice dell'elemento massimo nell'array? Qual è il valore atteso dell'indice dell'elemento minimo nell'array?

C.3-3

Un gioco di carnevale consiste nel lanciare tre dadi con un barattolo. Un giocatore può puntare un euro su un numero qualsiasi da 1 a 6. Il barattolo viene agitato e poi vengono lanciati i dadi. Se il numero puntato dal giocatore non appare in nessuno dei dadi, il giocatore perde l'euro. Se, invece, il suo numero appare su k dei tre dadi, per $k = 1, 2, 3$, il giocatore conserva il suo euro e vince altri k euro. Qual è il valore atteso della vincita se il giocatore fa una sola puntata?

C.3-4

Dimostrate che, se X e Y sono variabili casuali non negative, allora

$$\text{E}[\max(X, Y)] \leq \text{E}[X] + \text{E}[Y]$$

C.3-5 ⋆

Siano X e Y due variabili casuali indipendenti. Dimostrate che $f(X)$ e $g(Y)$ sono indipendenti per qualsiasi scelta delle funzioni f e g.

C.3-6 ⋆

Sia X una variabile casuale non negativa e supponete che $\text{E}[X]$ sia ben definito. Dimostrate la ***disequazione di Markov***

$$\Pr\{X \geq t\} \leq \text{E}[X]/t \tag{C.30}$$

per ogni $t > 0$.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

C.3-7 ⋆

Sia S uno spazio dei campioni e siano X e X' due variabili casuali tali che $X(s) \geq X'(s)$ per ogni $s \in S$. Dimostrate che per una costante reale qualsiasi t, si ha

$$\Pr\{X \geq t\} \geq \Pr\{X' \geq t\}$$

C.3-8
È più grande il valore atteso del quadrato di una variabile casuale o il quadrato del suo valore atteso?

C.3-9
Dimostrate che per qualsiasi variabile casuale X che assume soltanto i valori 0 e 1, si ha $\text{Var}[X] = \text{E}[X]\,\text{E}[1 - X]$.

C.3-10
Dimostrate che $\text{Var}[aX] = a^2\text{Var}[X]$ dalla definizione (C.27) di varianza.

C.4 Distribuzioni geometriche e binomiali

Il lancio di una moneta è un'istanza di una ***prova ripetuta di Bernoulli***, che è definita come un esperimento con due soli possibili risultati: un ***successo***, che si verifica con probabilità p, e un ***insuccesso***, che si verifica con probabilità $q = 1 - p$. Quando parliamo di ***prove ripetute di Bernoulli*** nel loro complesso, intendiamo che le prove sono mutuamente indipendenti e hanno la stessa probabilità di successo p, a meno che non sia diversamente specificato. Due importanti distribuzioni derivano dalle prove ripetute di Bernoulli: la distribuzione geometrica e la distribuzione binomiale.

La distribuzione geometrica

Supponiamo di avere una sequenza di prove ripetute di Bernoulli, ciascuna con una probabilità di successo p e una probabilità di insuccesso $q = 1 - p$. Quante prove ripetute devono essere effettuate prima di ottenere un successo? Indichiamo con la variabile casuale X il numero di prove ripetute che sono richieste per ottenere un successo. Allora la variabile X ha i valori nell'intervallo $\{1, 2, \ldots\}$ e, per $k \geq 1$, si ha

$$\Pr\{X = k\} = q^{k-1}p \tag{C.31}$$

perché abbiamo $k - 1$ insuccessi prima di un successo. Una distribuzione di probabilità che soddisfa l'equazione (C.31) è detta ***distribuzione geometrica***. La Figura C.1 illustra un esempio di tale distribuzione.

Supponendo che $q < 1$, il valore atteso di una distribuzione geometrica può essere calcolato utilizzando l'identità (A.8):

$$\begin{aligned}
\text{E}[X] &= \sum_{k=1}^{\infty} kq^{k-1}p \\
&= \frac{p}{q}\sum_{k=0}^{\infty} kq^k \\
&= \frac{p}{q}\cdot\frac{q}{(1-q)^2} \\
&= 1/p
\end{aligned} \tag{C.32}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Quindi, in media, occorrono $1/p$ prove ripetute prima di ottenere un successo, un risultato intuitivo. La varianza, che si può calcolare in modo analogo utilizzando l'Esercizio A.1-3, è

$$\text{Var}[X] = q/p^2 \tag{C.33}$$

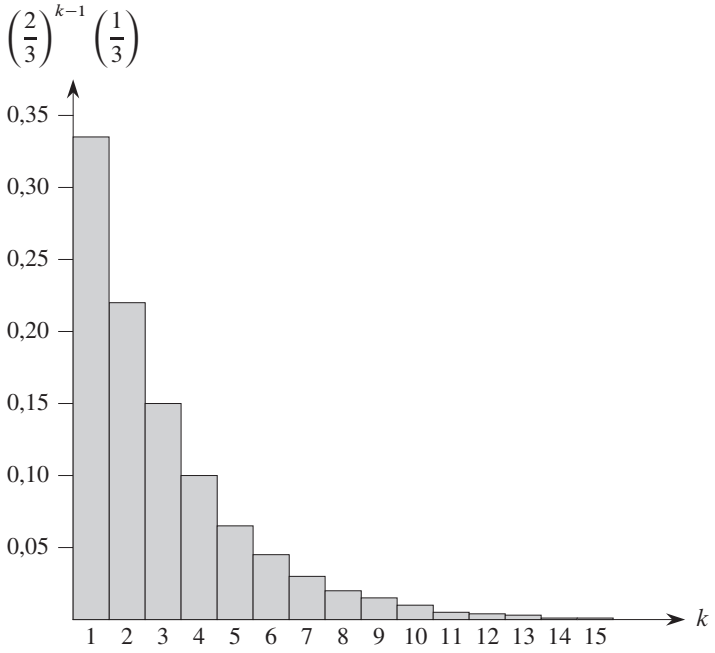


Figura C.1 Una distribuzione geometrica con probabilità di successo $p = 1/3$ e probabilità di insuccesso $q = 1 - p$. Il valore atteso della distribuzione è $1/p = 3$.

Come esempio, supponiamo di lanciare ripetutamente due dadi finché non otteniamo sette o undici. Dei 36 possibili risultati, 6 danno sette e 2 danno undici. Di conseguenza, la probabilità di successo è $p = 8/36 = 2/9$; quindi in media bisogna lanciare $1/p = 9/2 = 4,5$ volte i dadi per ottenere un sette o un undici.

La distribuzione binomiale

Quanti successi si verificano durante n prove ripetute di Bernoulli, se un successo si verifica con probabilità p e un insuccesso con probabilità $q = 1-p$? Definiamo la variabile casuale X come il numero di successi in n prove ripetute. Allora i valori di X sono nell'intervallo $\{0, 1, \ldots, n\}$ e, per $k = 0, \ldots, n$, si ha

$$\Pr\{X = k\} = \binom{n}{k} p^k q^{n-k} \tag{C.34}$$

perché ci sono $\binom{n}{k}$ modi di scegliere quali k delle n prove sono successi, e la probabilità che ciascuno di essi si verifichi è $p^k q^{n-k}$. Una distribuzione di probabilità che soddisfa l'equazione (C.34) è detta ***distribuzione binomiale***. Per comodità, definiamo la famiglia delle distribuzioni binomiali utilizzando la notazione

$$b(k; n, p) = \binom{n}{k} p^k (1 - p)^{n-k} \tag{C.35}$$

La Figura C.2 illustra una distribuzione binomiale. Il nome "binomiale" deriva dal fatto che l'equazione (C.34) rappresenta il k-esimo termine dello sviluppo in serie di $(p + q)^n$. Di conseguenza, poiché $p + q = 1$, si ha

$$\sum_{k=0}^{n} b(k; n, p) = 1 \tag{C.36}$$

come richiesto dall'assioma 2 degli assiomi di probabilità. Possiamo calcolare il valore atteso di una variabile casuale che ha una distribuzione binomiale utiliz-

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

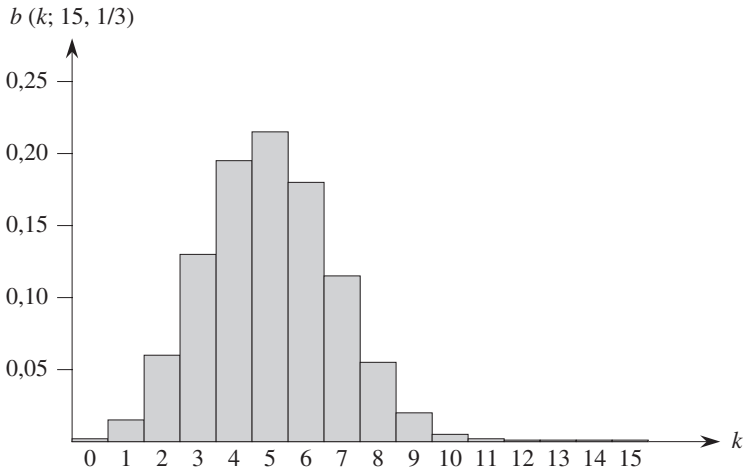


Figura C.2 La distribuzione binomiale $b(k; 15, 1/3)$ risultante da $n = 15$ prove ripetute di Bernoulli, ciascuna con probabilità di successo $p = 1/3$. Il valore atteso della distribuzione è $np = 5$.

zando le equazioni (C.8) e (C.36). Sia X una variabile casuale con distribuzione binomiale $b(k; n, p)$ e sia $q = 1 - p$. Per la definizione di valore atteso, si ha

$$\begin{aligned}
\mathrm{E}[X] &= \sum_{k=0}^{n} k \cdot \Pr\{X = k\} \\
&= \sum_{k=0}^{n} k \cdot b(k; n, p) \\
&= \sum_{k=1}^{n} k \binom{n}{k} p^k q^{n-k} \\
&= np \sum_{k=1}^{n} \binom{n-1}{k-1} p^{k-1} q^{n-k} \qquad \text{(per l'equazione (C.8))} \\
&= np \sum_{k=0}^{n-1} \binom{n-1}{k} p^k q^{(n-1)-k} \\
&= np \sum_{k=0}^{n-1} b(k; n-1, p) \\
&= np \qquad \text{(per l'equazione (C.36))} \qquad \text{(C.37)}
\end{aligned}$$

Applicando la linearità del valore atteso, possiamo ottenere lo stesso risultato riducendo il numero di calcoli algebrici. Sia X_i la variabile casuale che descrive il numero di successi nella i-esima prova ripetuta. Allora $\mathrm{E}[X_i] = p \cdot 1 + q \cdot 0 = p$ e, per la linearità del valore atteso (equazione (C.21)), il valore atteso dei successi per n prove ripetute è

$$\begin{aligned}
\mathrm{E}[X] &= \mathrm{E}\left[\sum_{i=1}^{n} X_i\right] \\
&= \sum_{i=1}^{n} \mathrm{E}[X_i] \\
&= \sum_{i=1}^{n} p \\
&= np \qquad \text{(C.38)}
\end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Lo stesso approccio può essere utilizzato per calcolare la varianza della distribuzione. Utilizzando l'equazione (C.27), si ottiene $\text{Var}[X_i] = \text{E}[X_i^2] - \text{E}^2[X_i]$. Poiché X_i assume soltanto i valori 0 e 1, si ha $\text{E}[X_i^2] = \text{E}[X_i] = p$, e quindi

$$\text{Var}[X_i] = p - p^2 = pq \tag{C.39}$$

Per calcolare la varianza di X, sfruttiamo l'indipendenza delle n prove ripetute; quindi, per l'equazione (C.29), si ha

$$\begin{aligned}
\text{Var}[X] &= \text{Var}\left[\sum_{i=1}^{n} X_i\right] \\
&= \sum_{i=1}^{n} \text{Var}[X_i] \\
&= \sum_{i=1}^{n} pq \\
&= npq
\end{aligned} \tag{C.40}$$

Come si può notare dalla Figura C.2, la distribuzione binomiale $b(k; n, p)$ aumenta quando k passa da 0 a n finché raggiunge il valore medio np, e poi diminuisce. Possiamo dimostrare che la distribuzione si comporta sempre in questo modo esaminando il rapporto dei termini successivi:

$$\begin{aligned}
\frac{b(k; n, p)}{b(k-1; n, p)} &= \frac{\binom{n}{k} p^k q^{n-k}}{\binom{n}{k-1} p^{k-1} q^{n-k+1}} \\
&= \frac{n!(k-1)!(n-k+1)!p}{k!(n-k)!n!q} \\
&= \frac{(n-k+1)p}{kq} \\
&= 1 + \frac{(n+1)p - k}{kq}
\end{aligned} \tag{C.41}$$

Questo rapporto è maggiore di 1 esattamente quando $(n+1)p - k$ è positivo. Di conseguenza, $b(k; n, p) > b(k-1; n, p)$ per $k < (n+1)p$ (la distribuzione aumenta) e $b(k; n, p) < b(k-1; n, p)$ per $k > (n+1)p$ (la distribuzione diminuisce). Se $k = (n+1)p$ è un intero, allora $b(k; n, p) = b(k-1; n, p)$, quindi la distribuzione ha due massimi: in $k = (n+1)p$ e in $k-1 = (n+1)p-1 = np-q$. Altrimenti, raggiunge un massimo nell'unico intero k che giace nell'intervallo $np - q < k < (n+1)p$.

Il seguente lemma fornisce un limite superiore per la distribuzione binomiale.

Lemma C.1
Sia $n \geq 0$, sia $0 < p < 1$, sia $q = 1 - p$ e sia $0 \leq k \leq n$. Allora

$$b(k; n, p) \leq \left(\frac{np}{k}\right)^k \left(\frac{nq}{n-k}\right)^{n-k}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria 1983503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Dimostrazione Utilizzando l'equazione (C.6), si ha

$$b(k; n, p) = \binom{n}{k} p^k q^{n-k}$$

$$\begin{aligned} &\leq \left(\frac{n}{k}\right)^k \left(\frac{n}{n-k}\right)^{n-k} p^k q^{n-k} \\ &= \left(\frac{np}{k}\right)^k \left(\frac{nq}{n-k}\right)^{n-k} \end{aligned}$$

■

Esercizi

C.4-1

Verificate l'assioma 2 per la distribuzione geometrica.

C.4-2

Quante volte in media occorre lanciare 6 monete perfette prima di ottenere 3 teste e 3 croci?

C.4-3

Dimostrate che $b(k; n, p) = b(n-k; n, q)$, dove $q = 1 - p$.

C.4-4

Dimostrate che il valore massimo della distribuzione binomiale $b(k; n, p)$ è circa $1/\sqrt{2\pi npq}$, dove $q = 1 - p$.

C.4-5 ⋆

Dimostrate che la probabilità di non avere successi in n prove ripetute di Bernoulli, ciascuna con probabilità $p = 1/n$, è circa $1/e$. Dimostrate che anche la probabilità di avere un solo successo è circa $1/e$.

C.4-6 ⋆

Il professor Rosencrantz lancia una moneta perfetta n volte; il professor Guildenstern fa la stessa cosa. Dimostrate che la probabilità che i professori ottengano lo stesso numero di teste è $\binom{2n}{n}/4^n$ (*suggerimento:* per il professor Rosencrantz è un successo ottenere una testa; per il professor Guildenstern è un successo ottenere una croce). Applicate lo stesso ragionamento per verificare l'identità

$$\sum_{k=0}^{n} \binom{n}{k}^2 = \binom{2n}{n}$$

C.4-7 ⋆

Dimostrate che per $0 \leq k \leq n$, si ha

$$b(k; n, 1/2) \leq 2^{n\,H(k/n)-n}$$

dove $H(x)$ è la funzione entropia (C.7).

C.4-8 ⋆

Considerate n prove ripetute di Bernoulli, dove per $i = 1, 2, \ldots, n$, la i-esima prova ha probabilità di successo p_i; sia X la variabile casuale che indica il numero totale di successi. Sia $p \geq p_i$ per ogni $i = 1, 2, \ldots, n$. Dimostrate che per $1 \leq k \leq n$, si ha

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\Pr\{X < k\} \geq \sum_{i=0}^{k-1} b(i; n, p)$$

C.4-9 ⋆

Sia X la variabile casuale che indica il numero totale di successi in un insieme A di n prove ripetute di Bernoulli, dove la i-esima prova ha una probabilità di

successo p_i; sia X' la variabile casuale che indica il numero totale di successi in un secondo insieme A' di n prove ripetute di Bernoulli, dove la i-esima prova ha una probabilità di successo $p'_i \geq p_i$. Dimostrate che per $0 \leq k \leq n$, si ha

$$\Pr\{X' \geq k\} \geq \Pr\{X \geq k\}$$

(*Suggerimento:* spiegate come ottenere le prove ripetute di Bernoulli in A' tramite un esperimento che si basa sulle prove di A e utilizzate il risultato dell'Esercizio C.3-7.)

⋆ C.5 Le code della distribuzione binomiale

La probabilità di avere almeno, o al più, k successi in n prove ripetute di Bernoulli, ciascuna con probabilità di successo p, spesso è più interessante della probabilità di avere esattamente k successi. In questo paragrafo, analizziamo le ***code*** della distribuzione binomiale: le due regioni della distribuzione $b(k; n, p)$ che sono lontane della media np. Dimostreremo vari limiti importanti per (la somma di tutti i termini in) una coda.

Prima forniremo un limite per la coda destra della distribuzione $b(k; n, p)$. I limiti per la coda sinistra possono essere determinati invertendo i ruoli di successo e insuccesso.

Teorema C.2
Consideriamo una sequenza di n prove ripetute di Bernoulli, dove il successo si verifica con probabilità p. Sia X la variabile casuale che indica il numero totale di successi. Allora per $0 \leq k \leq n$, la probabilità di avere almeno k successi è

$$\begin{aligned}\Pr\{X \geq k\} &= \sum_{i=k}^{n} b(i; n, p) \\ &\leq \binom{n}{k} p^k\end{aligned}$$

Dimostrazione Per $S \subseteq \{1, 2, \ldots, n\}$, indichiamo con A_S l'evento che la i-esima prova sia un successo per ogni $i \in S$. Chiaramente $\Pr\{A_S\} = p^k$ se $|S| = k$. Abbiamo

$$\begin{aligned}\Pr\{X \geq k\} &= \Pr\{\text{esiste } S \subseteq \{1, 2, \ldots, n\} : |S| = k \text{ and } A_S\} \\ &= \Pr\left\{\bigcup_{S \subseteq \{1,2,\ldots,n\}:|S|=k} A_S\right\} \\ &\leq \sum_{S \subseteq \{1,2,\ldots,n\}:|S|=k} \Pr\{A_S\} \qquad \text{(per la disuguaglianza (C.19))}\end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$= \binom{n}{k} p^k .$$

■

Il seguente corollario riformula il teorema per la coda sinistra della distribuzione binomiale. In generale, lasceremo al lettore il compito di adattare le dimostrazioni da una coda all'altra.

Corollario C.3
Consideriamo una sequenza di n prove ripetute di Bernoulli, dove il successo si verifica con probabilità p. Se X è la variabile casuale che indica il numero totale di successi, allora per $0 \le k \le n$, la probabilità di avere al più k successi è

$$\begin{aligned}\Pr\{X \le k\} &= \sum_{i=0}^{k} b(i;n,p) \\ &\le \binom{n}{n-k}(1-p)^{n-k} \\ &= \binom{n}{k}(1-p)^{n-k}\end{aligned}$$

■

Il prossimo limite riguarda la coda sinistra della distribuzione binomiale. Il suo corollario dimostra che, lontano dalla media, la coda sinistra diminuisce esponenzialmente.

Teorema C.4
Consideriamo una sequenza di n prove ripetute di Bernoulli, dove il successo si verifica con probabilità p e l'insuccesso con probabilità $q = 1 - p$. Sia X la variabile casuale che indica il numero totale di successi. Allora, per $0 < k < np$, la probabilità di avere meno di k successi è

$$\begin{aligned}\Pr\{X < k\} &= \sum_{i=0}^{k-1} b(i;n,p) \\ &< \frac{kq}{np-k}\, b(k;n,p)\end{aligned}$$

Dimostrazione Limitiamo la serie $\sum_{i=0}^{k-1} b(i;n,p)$ con una serie geometrica utilizzando la tecnica descritta nel Paragrafo A.2, pagina 957. Per $i = 1, 2, \ldots, k$, abbiamo dall'equazione (C.41)

$$\begin{aligned}\frac{b(i-1;n,p)}{b(i;n,p)} &= \frac{iq}{(n-i+1)p} \\ &< \frac{iq}{(n-i)p} \\ &\le \frac{kq}{(n-k)p}\end{aligned}$$

Se poniamo

$$\begin{aligned}x &= \frac{kq}{(n-k)p} \\ &< \frac{kq}{(n-np)p} \\ &= \frac{kq}{nqp} \\ &= \frac{k}{np} \\ &< 1\end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

otteniamo

$$b(i-1;n,p) < x\,b(i;n,p)$$

per $0 < i \leq k$. Applicando in maniera iterativa questa disuguaglianza $k-i$ volte, si ha

$$b(i;n,p) < x^{k-i}\,b(k;n,p)$$

per $0 \leq i < k$, e quindi

$$\begin{aligned}\sum_{i=0}^{k-1} b(i;n,p) &< \sum_{i=0}^{k-1} x^{k-i} b(k;n,p) \\ &< b(k;n,p)\sum_{i=1}^{\infty} x^i \\ &= \frac{x}{1-x}\,b(k;n,p) \\ &= \frac{kq}{np-k}\,b(k;n,p)\end{aligned}$$

■

Corollario C.5

Consideriamo una sequenza di n prove ripetute di Bernoulli, dove il successo si verifica con probabilità p e l'insuccesso con probabilità $q = 1-p$. Allora, per $0 < k \leq np/2$, la probabilità di avere meno di k successi è minore della metà della probabilità di avere meno di $k+1$ successi.

Dimostrazione Poiché $k \leq np/2$, si ha

$$\begin{aligned}\frac{kq}{np-k} &\leq \frac{(np/2)q}{np-(np/2)} \\ &= \frac{(np/2)q}{np/2} \\ &\leq 1\end{aligned}$$

in quanto $q \leq 1$. Se X è la variabile casuale che indica il numero di successi, il Teorema C.4 implica che la probabilità di avere meno di k successi è

$$\Pr\{X < k\} = \sum_{i=0}^{k-1} b(i;n,p) < b(k;n,p)$$

Quindi si ha

$$\begin{aligned}\frac{\Pr\{X < k\}}{\Pr\{X < k+1\}} &= \frac{\sum_{i=0}^{k-1} b(i;n,p)}{\sum_{i=0}^{k} b(i;n,p)} \\ &= \frac{\sum_{i=0}^{k-1} b(i;n,p)}{\sum_{i=0}^{k-1} b(i;n,p) + b(k;n,p)} \\ &< 1/2\end{aligned}$$

perché $\sum_{i=0}^{k-1} b(i;n,p) < b(k;n,p)$. ■

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

In modo analogo è possibile ottenere i limiti per la coda destra. Lasciamo al lettore il compito di dimostrarli (Esercizio C.5-2).

Corollario C.6
Consideriamo una sequenza di n prove ripetute di Bernoulli, dove il successo si verifica con probabilità p. Sia X la variabile casuale che indica il numero totale di successi. Allora per $np < k < n$, la probabilità di avere più di k successi è

$$\begin{aligned}\Pr\{X > k\} &= \sum_{i=k+1}^{n} b(i; n, p) \\ &< \frac{(n-k)p}{k-np}\, b(k; n, p)\end{aligned}$$

■

Corollario C.7
Consideriamo una sequenza di n prove ripetute di Bernoulli, dove il successo si verifica con probabilità p e l'insuccesso con probabilità $q = 1 - p$. Allora, per $(np + n)/2 < k < n$, la probabilità di avere più di k successi è minore della metà della probabilità di avere più di $k - 1$ successi. ■

Il prossimo teorema considera n prove ripetute di Bernoulli, ciascuna con una probabilità di successo p_i, per $i = 1, 2, \ldots, n$. Come dimostra il successivo corollario, possiamo utilizzare il teorema per fornire un limite per la coda destra della distribuzione binomiale, ponendo $p_i = p$ in ogni prova.

Teorema C.8
Consideriamo una sequenza di n prove ripetute di Bernoulli, dove nella i-esima prova, per $i = 1, 2, \ldots, n$, il successo si verifica con probabilità p_i e l'insuccesso con probabilità $q_i = 1 - p_i$. Sia X la variabile casuale che descrive il numero totale di successi e sia $\mu = \mathrm{E}[X]$. Allora, per $r > \mu$, si ha

$$\Pr\{X - \mu \geq r\} \leq \left(\frac{\mu e}{r}\right)^r$$

Dimostrazione Poiché per qualsiasi $\alpha > 0$, la funzione $e^{\alpha x}$ è strettamente crescente in x, si ha

$$\Pr\{X - \mu \geq r\} = \Pr\left\{e^{\alpha(X-\mu)} \geq e^{\alpha r}\right\} \tag{C.42}$$

dove α sarà calcolata più avanti. Applicando la disequazione di Markov (C.30), otteniamo

$$\Pr\left\{e^{\alpha(X-\mu)} \geq e^{\alpha r}\right\} \leq \mathrm{E}\left[e^{\alpha(X-\mu)}\right] e^{-\alpha r} \tag{C.43}$$

La parte cruciale della dimostrazione consiste nel limitare $\mathrm{E}\left[e^{\alpha(X-\mu)}\right]$ e nel sostituire α con un valore appropriato nella disuguaglianza (C.43).

Innanzi tutto, calcoliamo $\mathrm{E}\left[e^{\alpha(X-\mu)}\right]$. Utilizzando la notazione del Paragrafo 5.2, sia $X_i = \mathrm{I}\{\text{l'}i\text{-esima prova ripetuta di Bernoulli è un successo}\}$ per $i = 1, 2, \ldots, n$; ovvero, X_i è la variabile casuale che vale 1 se l'i-esima prova di Bernoulli è un successo, 0 se è un insuccesso. Quindi, si ottiene

$$X = \sum_{i=1}^{n} X_i$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Per la linearità del valore atteso, si ha

$$\mu = \mathrm{E}[X] = \mathrm{E}\left[\sum_{i=1}^{n} X_i\right] = \sum_{i=1}^{n} \mathrm{E}[X_i] = \sum_{i=1}^{n} p_i$$

che implica

$$X - \mu = \sum_{i=1}^{n} (X_i - p_i)$$

Per calcolare $\mathrm{E}\left[e^{\alpha(X-\mu)}\right]$, sostituiamo $X - \mu$, ottenendo

$$\begin{aligned}
\mathrm{E}\left[e^{\alpha(X-\mu)}\right] &= \mathrm{E}\left[e^{\alpha \sum_{i=1}^{n}(X_i - p_i)}\right] \\
&= \mathrm{E}\left[\prod_{i=1}^{n} e^{\alpha(X_i - p_i)}\right] \\
&= \prod_{i=1}^{n} \mathrm{E}\left[e^{\alpha(X_i - p_i)}\right]
\end{aligned}$$

che deriva da (C.24), perché l'indipendenza reciproca delle variabili casuali X_i implica l'indipendenza reciproca delle variabili casuali $e^{\alpha(X_i - p_i)}$ (vedere l'Esercizio C.3-5). Per la definizione di valore atteso, si ha

$$\begin{aligned}
\mathrm{E}\left[e^{\alpha(X_i - p_i)}\right] &= e^{\alpha(1-p_i)} p_i + e^{\alpha(0-p_i)} q_i \\
&= p_i e^{\alpha q_i} + q_i e^{-\alpha p_i} \\
&\leq p_i e^{\alpha} + 1 \qquad \text{(C.44)} \\
&\leq \exp(p_i e^{\alpha})
\end{aligned}$$

dove $\exp(x)$ indica la funzione esponenziale: $\exp(x) = e^x$. La disuguaglianza (C.44) segue dalle disuguaglianze $\alpha > 0$, $q_i \leq 1$, $e^{\alpha q_i} \leq e^{\alpha}$ e $e^{-\alpha p_i} \leq 1$; l'ultima riga segue dalla disuguaglianza (3.12). Di conseguenza, si ottiene

$$\begin{aligned}
\mathrm{E}\left[e^{\alpha(X-\mu)}\right] &= \prod_{i=1}^{n} \mathrm{E}\left[e^{\alpha(X_i - p_i)}\right] \\
&\leq \prod_{i=1}^{n} \exp(p_i e^{\alpha}) \\
&= \exp\left(\sum_{i=1}^{n} p_i e^{\alpha}\right) \\
&= \exp(\mu e^{\alpha}) \qquad \text{(C.45)}
\end{aligned}$$

perché $\mu = \sum_{i=1}^{n} p_i$. Quindi, per l'equazione (C.42) e le disuguaglianze (C.43) e (C.45), si ha

$$\Pr\{X - \mu \geq r\} \leq \exp(\mu e^{\alpha} - \alpha r) \qquad \text{(C.46)}$$

Scegliendo $\alpha = \ln(r/\mu)$ (vedere l'Esercizio C.5-7), si ottiene

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned}
\Pr\{X - \mu \geq r\} &\leq \exp(\mu e^{\ln(r/\mu)} - r \ln(r/\mu)) \\
&= \exp(r - r \ln(r/\mu)) \\
&= \frac{e^r}{(r/\mu)^r} \\
&= \left(\frac{\mu e}{r}\right)^r
\end{aligned}$$

■

Se viene applicato alle prove ripetute di Bernoulli quando ogni prova ha la stessa probabilità di successo, il Teorema C.8 fornisce il seguente corollario che limita la coda destra di una distribuzione binomiale.

Corollario C.9
Consideriamo una sequenza di n prove ripetute di Bernoulli, dove in ogni prova il successo si verifica con probabilità p e l'insuccesso con probabilità $q = 1 - p$. Allora, per $r > np$, si ha

$$\begin{aligned}\Pr\{X - np \geq r\} &= \sum_{k=\lceil np+r \rceil}^{n} b(k; n, p) \\ &\leq \left(\frac{npe}{r}\right)^r\end{aligned}$$

Dimostrazione Per l'equazione (C.37), si ha $\mu = \mathrm{E}[X] = np$. ■

Esercizi

C.5-1 ⋆
È meno probabile non ottenere testa lanciando una moneta perfetta n volte oppure ottenere meno di n teste lanciando la moneta $4n$ volte?

C.5-2 ⋆
Dimostrate i Corollari C.6 e C.7.

C.5-3 ⋆
Dimostrate che

$$\sum_{i=0}^{k-1} \binom{n}{i} a^i < (a+1)^n \frac{k}{na - k(a+1)} b(k; n, a/(a+1))$$

per ogni $a > 0$ e per ogni k tale che $0 < k < na/(a+1)$.

C.5-4 ⋆
Dimostrate che se $0 < k < np$, dove $0 < p < 1$ e $q = 1 - p$, allora

$$\sum_{i=0}^{k-1} p^i q^{n-i} < \frac{kq}{np - k} \left(\frac{np}{k}\right)^k \left(\frac{nq}{n-k}\right)^{n-k}$$

C.5-5 ⋆
Dimostrate che le condizioni del Teorema C.8 implicano che

$$\Pr\{\mu - X \geq r\} \leq \left(\frac{(n-\mu)e}{r}\right)^r$$

Analogamente, dimostrate che le condizioni del Corollario C.9 implicano che

$$\Pr\{np - X \geq r\} \leq \left(\frac{nqe}{r}\right)^r$$

Acquistato da Michele Marsiglia su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

C.5-6 ⋆
Considerate una sequenza di n prove ripetute di Bernoulli, dove nella i-esima prova, per $i = 1, 2, \ldots, n$, il successo si verifica con probabilità p_i e l'insuccesso con probabilità $q_i = 1 - p_i$. Sia X la variabile casuale che descrive il numero totale di successi e sia $\mu = \mathrm{E}[X]$. Dimostrate che, per $r \geq 0$, si ha

$$\Pr\{X - \mu \geq r\} \leq e^{-r^2/2n}$$

(*Suggerimento:* dimostrate che $p_i e^{\alpha q_i} + q_i e^{-\alpha p_i} \leq e^{\alpha^2/2}$. Poi seguite lo schema della dimostrazione del Teorema C.8, utilizzando questa disuguaglianza al posto della disuguaglianza (C.44).)

C.5-7 ⋆

Dimostrate che il lato destro della disuguaglianza (C.46) è minimizzato se si sceglie $\alpha = \ln(r/\mu)$.

Problemi

C-1 Palline e contenitori

In questo problema analizziamo gli effetti di varie ipotesi sul numero di modi in cui è possibile disporre n palline in b contenitori distinti.

a. Supponete che le n palline siano distinte e che il loro ordine all'interno di un contenitore non sia importante. Dimostrate che il numero di modi in cui è possibile disporre le palline nei contenitori è b^n.

b. Supponete che le palline siano distinte e che le palline in ogni contenitore siano ordinate. Dimostrate che ci sono esattamente $(b+n-1)!/(b-1)!$ modi di disporre le palline nei contenitori. (*Suggerimento:* considerate il numero di modi in cui è possibile mettere in fila n palline distinte e $b-1$ sbarrette indistinguibili.)

c. Supponete che le palline siano identiche; quindi il loro ordine all'interno di un contenitore non è importante. Dimostrate che il numero di modi in cui è possibile disporre le palline nei contenitori è $\binom{b+n-1}{n}$. (*Suggerimento:* quante disposizioni fra quelle calcolate nel punto (b) sono ripetute se le palline sono identiche?)

d. Supponete che le palline siano identiche e che nessun contenitore possa contenere più di una pallina. Dimostrate che il numero di modi in cui è possibile disporre le palline è $\binom{b}{n}$.

e. Supponete che le palline siano identiche e che nessun contenitore possa restare vuoto. Dimostrate che il numero di modi in cui è possibile disporre le palline è $\binom{n-1}{b-1}$.

Note

I primi metodi generali per risolvere i problemi di probabilità furono trattati in una famosa corrispondenza tra B. Pascal e P. de Fermat, che ebbe inizio nel 1654, e in un libro di C. Huygens nel 1657. La teoria rigorosa della probabilità iniziò con il lavoro di J. Bernoulli nel 1713 e di A. De Moivre nel 1730. Ulteriori sviluppi della teoria sono attribuiti a P. S. de Laplace, S.-D. Poisson e C. F. Gauss.

Le tecniche per sommare le variabili casuali furono studiate originariamente da P. L. Chebyshev e A. A. Markov. La teoria della probabilità è stata assiomatizzata da A. N. Kolmogorov nel 1933. I limiti per le code delle distribuzioni sono stati forniti da Chernoff [67] e Hoeffding [174]. Il lavoro di P. Erdös è stato alla base del successivo sviluppo delle strutture combinatorie casuali.

Knuth [210] e Liu [238] sono buoni testi di riferimento per i fondamenti del calcolo combinatorio. I libri di testo standard, come Billingsley [46], Chung [68], Drake [96], Feller [105] e Rozanov [301], offrono un'esauriente introduzione alla teoria della probabilità.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Matrici D

Le matrici si presentano in numerose applicazioni, in particolare nel calcolo scientifico. Se avete già studiato le matrici, molti degli argomenti di questa appendice vi saranno familiari, mentre altri potrebbero essere nuovi. Il Paragrafo D.1 descrive le definizioni e le operazioni fondamentali delle matrici; il Paragrafo D.2 presenta alcune proprietà fondamentali delle matrici.

D.1 Matrici e operazioni sulle matrici

In questo paragrafo presentiamo alcuni concetti di base della teoria delle matrici e alcune proprietà fondamentali delle matrici.

Matrici e vettori

Una ***matrice*** è un array rettangolare di numeri. Per esempio

$$\begin{aligned} A &= \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{pmatrix} \\ &= \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \end{aligned} \tag{D.1}$$

è una matrice $A = (a_{ij})$ di dimensioni 2×3, dove per $i = 1, 2$ e $j = 1, 2, 3$, l'elemento della matrice nella riga i e nella colonna j è a_{ij}. Utilizzeremo le lettere maiuscole per indicare le matrici (per esempio, A) e le corrispondenti lettere minuscole con pedici (per esempio, a_{ij}) per indicare i loro elementi. Indicheremo con $\mathbb{R}^{m \times n}$ l'insieme di tutte le matrici $m \times n$ con elementi a valori reali. In generale, l'insieme delle matrici $m \times n$ con elementi estratti da un insieme S è indicato con $S^{m \times n}$.

La ***trasposta*** di una matrice A è la matrice A^{T} ottenuta scambiando le righe con le colonne di A. Per la matrice A dell'equazione (D.1), la trasposta è

$$A^{\mathrm{T}} = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}$$

Un ***vettore*** è un array monodimensionale di numeri. Per esempio

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$x = \begin{pmatrix} 2 \\ 3 \\ 5 \end{pmatrix}$$

è un vettore di dimensione 3. Utilizziamo le lettere minuscole per indicare i vettori. L'i-esimo elemento di un vettore x di dimensione n è indicato con x_i, per

$i = 1, 2, \ldots, n$. Assumeremo che un vettore abbia la forma standard di un ***vettore colonna*** che è equivalente a una matrice $n \times 1$; il corrispondente ***vettore riga*** è ottenuto trasponendo il vettore colonna:

$$x^{\mathrm{T}} = (\,2 \quad 3 \quad 5\,)$$

Il ***vettore unità*** e_i è il vettore il cui i-esimo elemento è 1 e tutti gli altri elementi sono 0. Di solito, la dimensione di un vettore unità è evidente dal contesto.

La ***matrice zero*** è quella i cui elementi sono tutti 0. Tale matrice è spesso indicata con 0, perché il dubbio tra il numero 0 e una matrice 0 di solito viene risolto dal contesto. Quando si usa una matrice zero, anche la sua dimensione si deduce dal contesto.

Matrici quadrate

Nella pratica si usano spesso delle ***matrici quadrate*** $n \times n$. Molti casi speciali di matrici quadrate sono particolarmente interessanti:

1. Una ***matrice diagonale*** ha $a_{ij} = 0$ per ogni $i \neq j$. Poiché tutti gli elementi che non appartengono alla diagonale principale della matrice sono nulli, la matrice può essere specificata elencando soltanto gli elementi lungo la diagonale:

$$\mathrm{diag}(a_{11}, a_{22}, \ldots, a_{nn}) = \begin{pmatrix} a_{11} & 0 & \ldots & 0 \\ 0 & a_{22} & \ldots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & a_{nn} \end{pmatrix}$$

2. La ***matrice identità*** I_n è una matrice diagonale $n \times n$ i cui elementi lungo la diagonale sono tutti 1:

$$\begin{aligned} I_n &= \mathrm{diag}(1, 1, \ldots, 1) \\ &= \begin{pmatrix} 1 & 0 & \ldots & 0 \\ 0 & 1 & \ldots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & 1 \end{pmatrix} \end{aligned}$$

   Quando la matrice I appare senza pedice, la sua dimensione può essere dedotta dal contesto. L'i-esima colonna di una matrice identità è il vettore unità e_i.

3. Una ***matrice tridiagonale*** T è una matrice per la quale $t_{ij} = 0$ se $|i - j| > 1$. Gli elementi diversi da zero appaiono soltanto lungo la diagonale principale, immediatamente sopra la diagonale principale ($t_{i,i+1}$ per $i = 1, 2, \ldots, n-1$) o immediatamente sotto la diagonale principale ($t_{i+1,i}$ per $i = 1, 2, \ldots, n-1$):

$$T = \begin{pmatrix} t_{11} & t_{12} & 0 & 0 & \ldots & 0 & 0 & 0 \\ t_{21} & t_{22} & t_{23} & 0 & \ldots & 0 & 0 & 0 \\ 0 & t_{32} & t_{33} & t_{34} & \ldots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \ldots & t_{n-2,n-2} & t_{n-2,n-1} & 0 \\ 0 & 0 & 0 & 0 & \ldots & t_{n-1,n-2} & t_{n-1,n-1} & t_{n-1,n} \\ 0 & 0 & 0 & 0 & \ldots & 0 & t_{n,n-1} & t_{nn} \end{pmatrix}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 196503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

4. Una ***matrice triangolare superiore*** U è una matrice per la quale $u_{ij} = 0$ se $i > j$. Tutti gli elementi sotto la diagonale principale sono nulli:

$$U = \begin{pmatrix} u_{11} & u_{12} & \dots & u_{1n} \\ 0 & u_{22} & \dots & u_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & u_{nn} \end{pmatrix}$$

   Una matrice triangolare superiore è una ***matrice triangolare superiore unitaria*** se gli elementi della diagonale principale sono tutti 1.

5. Una ***matrice triangolare inferiore*** L è una matrice per la quale $l_{ij} = 0$ se $i < j$. Tutti gli elementi sopra la diagonale principale sono nulli:

$$L = \begin{pmatrix} l_{11} & 0 & \dots & 0 \\ l_{21} & l_{22} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ l_{n1} & l_{n2} & \dots & l_{nn} \end{pmatrix}$$

   Una matrice triangolare inferiore è una ***matrice triangolare inferiore unitaria*** se gli elementi della diagonale principale sono tutti 1.

6. Una ***matrice di permutazione*** P ha un solo 1 in ogni riga o colonna e 0 in tutte le altre posizioni. Ecco un esempio di matrice di permutazione:

$$P = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

   Tale matrice è detta matrice di permutazione perché moltiplicando un vettore x per una matrice di permutazione si ha l'effetto di permutare (riarrangiare) gli elementi di x. L'Esercizio D.1-4 esamina altre proprietà delle matrici di permutazione.

7. Una ***matrice simmetrica*** A soddisfa la condizione $A = A^{\mathrm{T}}$. Per esempio

$$\begin{pmatrix} 1 & 2 & 3 \\ 2 & 6 & 4 \\ 3 & 4 & 5 \end{pmatrix}$$

   è una matrice simmetrica.

Operazioni sulle matrici

Gli elementi di una matrice o di un vettore sono numeri appartenenti a un insieme numerico, come i numeri reali, i numeri complessi o gli interi modulo un numero primo. L'insieme numerico definisce come devono essere sommati e moltiplicati i numeri. Possiamo estendere queste definizioni in modo da includere la somma e il prodotto delle matrici.

Acquistato da Michele Mu... [illegible] ... McGraw-Hill Education (Italy)

Definiamo la ***somma delle matrici*** nel modo seguente. Se $A = (a_{ij})$ e $B = (b_{ij})$ sono due matrici $m \times n$, allora la loro somma $C = (c_{ij}) = A + B$ è la matrice $m \times n$ definita da

$$c_{ij} = a_{ij} + b_{ij}$$

per $i = 1, 2, \ldots, m$ e $j = 1, 2, \ldots, n$. Ovvero, la somma di due matrici viene eseguita elemento per elemento. La matrice zero è l'elemento neutro per la somma di matrici:

$$A + 0 = A = 0 + A$$

Se λ è un numero e $A = (a_{ij})$ è una matrice, allora $\lambda A = (\lambda a_{ij})$ è il ***multiplo scalare*** di A che si ottiene moltiplicando ogni elemento della matrice per λ. Come caso speciale, definiamo il ***negativo di una matrice*** $A = (a_{ij})$ come $-1 \cdot A = -A$, cosicché il generico elemento a_{ij} di $-A$ è $-a_{ij}$. Quindi

$$A + (-A) = 0 = (-A) + A$$

Sulla base di questa definizione, è possibile definire la ***sottrazione di matrici*** come la somma del negativo di una matrice: $A - B = A + (-B)$.

Definiamo il ***prodotto delle matrici*** nel modo seguente. Innanzi tutto, le matrici A e B da moltiplicare devono essere ***compatibili***, nel senso che il numero di colonne di A deve essere uguale al numero di righe di B (in generale, se un'espressione contiene il prodotto AB, si suppone implicitamente che le matrici A e B siano compatibili). Se $A = (a_{ij})$ è una matrice $m \times n$ e $B = (b_{jk})$ è una matrice $n \times p$, allora il loro prodotto $C = AB$ è la matrice $C = (c_{ik})$ di dimensioni $m \times p$, dove

$$c_{ik} = \sum_{j=1}^{n} a_{ij} b_{jk} \tag{D.2}$$

per $i = 1, 2, \ldots, m$ e $k = 1, 2, \ldots, p$. L'algoritmo di base MATRIX-MULTIPLY (Paragrafo 25.2) implementa in modo semplice la moltiplicazione delle matrici basata sull'equazione (D.2), supponendo che le matrici siano quadrate: $m = n = p$. Per moltiplicare le matrici $n \times n$, l'algoritmo di base MATRIX-MULTIPLY esegue n^3 moltiplicazioni e $n^2(n-1)$ somme; il suo tempo di esecuzione è $\Theta(n^3)$.

Le matrici godono di molte (ma non di tutte) proprietà algebriche tipiche dei numeri. La matrice identità è l'elemento neutro per la moltiplicazione delle matrici:

$$I_m A = A I_n = A$$

per qualsiasi matrice A di dimensione $m \times n$. Moltiplicando una matrice per la matrice zero si ottiene la matrice zero:

$$A\,0 = 0$$

La moltiplicazione delle matrici è associativa:

$$A(BC) = (AB)C$$

per matrici compatibili A, B e C. La moltiplicazione di matrici è distributiva rispetto alla somma:

Acquistato da Michele Michele su Webster il 2022-07-07 20:22 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$\begin{aligned} A(B + C) &= AB + AC \\ (B + C)D &= BD + CD \end{aligned}$$

Per $n > 1$, la moltiplicazione di matrici $n \times n$ non è commutativa. Per esempio, se $A = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ e $B = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}$, allora

$$AB = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$$

e

$$BA = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}$$

I prodotti matrice-vettore o vettore-vettore sono definiti come se il vettore fosse la matrice equivalente $n \times 1$ (o una matrice $1 \times n$, nel caso di un vettore riga). Quindi, se A è una matrice $m \times n$ e x è un vettore di dimensione n, allora Ax è un vettore di dimensione m. Se x e y sono vettori di dimensione n, allora

$$x^{\mathrm{T}} y = \sum_{i=1}^{n} x_i y_i$$

è un numero (in effetti una matrice 1×1) detto***prodotto interno*** di x e y. La matrice $x y^{\mathrm{T}}$ è una matrice Z di dimensioni $n \times n$ detta ***prodotto esterno*** di x e y, con $z_{ij} = x_i y_j$. La ***norma (euclidea)*** $\|x\|$ di un vettore x di dimensione n è così definita:

$$\begin{aligned} \|x\| &= (x_1^2 + x_2^2 + \cdots + x_n^2)^{1/2} \\ &= (x^{\mathrm{T}} x)^{1/2} \end{aligned}$$

Quindi, la norma di x è la sua lunghezza nello spazio euclideo n-dimensionale.

Esercizi

D.1-1

Dimostrate che, se A e B sono matrici simmetriche di dimensioni $n \times n$, lo sono anche $A + B$ e $A - B$.

D.1-2

Dimostrate che $(AB)^{\mathrm{T}} = B^{\mathrm{T}} A^{\mathrm{T}}$ e che $A^{\mathrm{T}} A$ è sempre una matrice simmetrica.

D.1-3

Dimostrate che il prodotto di due matrici triangolari inferiori è una matrice triangolare inferiore.

D.1-4

Dimostrate che, se P è una matrice di permutazione $n \times n$ e A è una matrice $n \times n$, allora PA può essere ottenuta da A permutando le sue righe, e AP può essere ottenuta da A permutando le sue colonne. Dimostrate che il prodotto di due matrici di permutazione è una matrice di permutazione.

D.2 Proprietà fondamentali delle matrici

In questo paragrafo definiamo alcune proprietà fondamentali delle matrici: le matrici inverse, la dipendenza e l'indipendenza lineare, il rango e i determinanti. Descriveremo inoltre la classe delle matrici definite positive.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Matrici inverse, ranghi e determinanti

Definiamo ***inversa*** di una matrice A di dimensioni $n \times n$ la matrice A^{-1} (se esiste) di dimensioni $n \times n$ tale che $AA^{-1} = I_n = A^{-1}A$. Per esempio

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{-1} = \begin{pmatrix} 0 & 1 \\ 1 & -1 \end{pmatrix}$$

Molte matrici $n \times n$ non nulle non hanno le matrici inverse. Una matrice senza l'inversa è detta ***matrice non invertibile*** o ***singolare***. Un esempio di matrice non nulla singolare è

$$\begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix}$$

Se una matrice ha l'inversa, è detta ***matrice invertibile*** o ***non singolare***. Le matrici inverse, quando esistono, sono uniche (vedere l'Esercizio D.2-1). Se A e B sono matrici $n \times n$ non singolari, allora

$$(BA)^{-1} = A^{-1}B^{-1}$$

La matrice trasposta dell'inversa è uguale alla matrice inversa della trasposta:

$$(A^{-1})^{\mathrm{T}} = (A^{\mathrm{T}})^{-1}$$

I vettori $x_1, x_2, \ldots, x_n$ sono ***linearmente dipendenti*** se esistono dei coefficienti $c_1, c_2, \ldots, c_n$, non tutti nulli, tali che $c_1x_1 + c_2x_2 + \cdots + c_nx_n = 0$. Per esempio, i vettori riga $x_1 = (\,1 \quad 2 \quad 3\,)$, $x_2 = (\,2 \quad 6 \quad 4\,)$ e $x_3 = (\,4 \quad 11 \quad 9\,)$ sono linearmente dipendenti, perché $2x_1 + 3x_2 - 2x_3 = 0$. Se i vettori non sono linearmente dipendenti, sono ***linearmente indipendenti***. Per esempio, le colonne di una matrice identità sono linearmente indipendenti.

Il ***rango di colonna*** di una matrice non nulla A, $m \times n$, è la dimensione del più grande insieme di colonne linearmente indipendenti di A. Analogamente, il ***rango di riga*** di A è la dimensione del più grande insieme di righe linearmente indipendenti di A. Una proprietà fondamentale di qualsiasi matrice A è che il suo rango di riga è sempre uguale al suo rango di colonna, quindi possiamo parlare semplicemente di ***rango*** di A. Il rango di una matrice $m \times n$ è un intero compreso tra 0 e $\min(m, n)$, estremi inclusi (il rango di una matrice zero è 0 e il rango di una matrice identità $n \times n$ è n). Una definizione alternativa, ma equivalente e spesso più utile, è che il rango di una matrice non nulla A, $m \times n$, è il più piccolo numero r tale che esistono due matrici B e C di dimensioni rispettivamente $m \times r$ ed $r \times n$ per le quali si ha

$$A = BC$$

Una matrice quadrata $n \times n$ ha ***rango massimo*** se il suo rango è n. Una matrice $m \times n$ ha ***rango di colonna massimo*** se il suo rango è n. Una proprietà fondamentale dei ranghi è data dal seguente teorema.

Teorema D.1

Una matrice quadrata ha rango massimo se e soltanto se non è singolare. ■

Un ***vettore nullo*** per una matrice A è un vettore non nullo x tale che $Ax = 0$. Il seguente teorema (la cui dimostrazione è richiesta nell'Esercizio D.2-7) e il suo corollario mettono in relazione i concetti di rango di colonna e di singolarità con i vettori nulli.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995030116229707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Teorema D.2

Una matrice A ha rango di colonna massimo se e soltanto se non ha un vettore nullo. ■

Corollario D.3
Una matrice quadrata A è singolare se e soltanto se ha un vettore nullo. ■

Il ***minore*** ij-esimo di una matrice A, $n \times n$, per $n > 1$, è la matrice $A_{[ij]}$, $(n-1) \times (n-1)$, che si ottiene cancellando la i-esima riga e la j-esima colonna di A. Il ***determinante*** di una matrice A, $n \times n$, può essere definito ricorsivamente in funzione dei suoi minori in questo modo:

$$\det(A) = \begin{cases} a_{11} & \text{se } n = 1 \\ \displaystyle\sum_{j=1}^{n} (-1)^{1+j} a_{1j} \det(A_{[1j]}) & \text{se } n > 1 \end{cases}$$

Il termine $(-1)^{i+j} \det(A_{[ij]})$ è detto ***cofattore*** dell'elemento a_{ij}.

I seguenti teoremi, le cui dimostrazioni sono omesse, enunciano le proprietà fondamentali del determinante.

Teorema D.4 (Proprietà del determinante)
Il determinante di una matrice quadrata A ha le seguenti proprietà:

- Se qualche riga o qualche colonna di A è zero, allora $\det(A) = 0$.
- Il determinante di A è moltiplicato per λ se gli elementi di una riga (o colonna) qualsiasi di A sono tutti moltiplicati per λ.
- Il determinante di A non cambia se gli elementi di una riga (o colonna) sono sommati a quelli di un'altra riga (o colonna).
- Il determinante di A è uguale al determinante della trasposta A^{T}.
- Il determinante di A è moltiplicato per -1 se due righe (o colonne) qualsiasi sono scambiate.

Inoltre, per due matrici quadrate qualsiasi A e B, si ha:
$\det(AB) = \det(A)\det(B)$ ■

Teorema D.5
Una matrice A, $n \times n$, è singolare se e soltanto se $\det(A) = 0$. ■

Matrici definite positive

Le matrici definite positive svolgono un ruolo importante in molte applicazioni. Una matrice A, $n \times n$, è ***definita positiva*** se $x^{\mathrm{T}}Ax > 0$ per tutti i vettori $x \neq 0$ di dimensione n. Per esempio, la matrice identità è definita positiva, perché per qualsiasi vettore non nullo $x = (\, x_1 \quad x_2 \quad \cdots \quad x_n \,)^{\mathrm{T}}$, si ha

$$\begin{aligned} x^{\mathrm{T}} I_n x &= x^{\mathrm{T}} x \\ &= \sum_{i=1}^{n} x_i^2 \\ &> 0 \end{aligned}$$

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Alcune delle matrici che si presentano nelle applicazioni sono definite positive a causa del seguente teorema.

Teorema D.6
Per qualsiasi matrice A con rango di colonna massimo, la matrice $A^{\mathrm{T}}A$ è definita positiva.

Dimostrazione Dobbiamo dimostrare che $x^{\mathrm{T}}(A^{\mathrm{T}}A)x > 0$ per qualsiasi vettore non nullo x. Per qualsiasi vettore x, si ha

$$\begin{aligned} x^{\mathrm{T}}(A^{\mathrm{T}}A)x &= (Ax)^{\mathrm{T}}(Ax) \quad \text{(per l'Esercizio D.1-2)} \\ &= \|Ax\|^2 \end{aligned}$$

Notate che $\|Ax\|^2$ è semplicemente la somma dei quadrati degli elementi del vettore Ax. Quindi, $\|Ax\|^2 \geq 0$. Se $\|Ax\|^2 = 0$, ogni elemento di Ax è 0, e questo equivale a dire che $Ax = 0$. Poiché la matrice A ha rango di colonna massimo, $Ax = 0$ implica $x = 0$, per il Teorema D.2. Dunque, $A^{\mathrm{T}}A$ è definita positiva. ■

Altre proprietà delle matrici definite positive sono esaminate nel Paragrafo 28.3.

Esercizi

D.2-1
Dimostrate che le matrici inverse sono uniche, ovvero, se B e C sono inverse di A, allora $B = C$.

D.2-2
Dimostrate che il determinante di una matrice triangolare inferiore o superiore è uguale al prodotto degli elementi della diagonale. Dimostrate che l'inversa di una matrice triangolare inferiore, se esiste, è una matrice triangolare inferiore.

D.2-3
Dimostrate che se P è una matrice di permutazione, allora P è invertibile, la sua inversa è P^{T} e P^{T} è una matrice di permutazione.

D.2-4
Siano A e B due matrici $n \times n$ tali che $AB = I$. Dimostrate che, se A' è ottenuta da A sommando la riga j alla riga i, allora l'inversa B' di A' può essere ottenuta sottraendo la colonna i dalla colonna j di B.

D.2-5
Sia A una matrice $n \times n$ non singolare con elementi complessi. Dimostrate che ogni elemento di A^{-1} è reale se e soltanto se ogni elemento di A è reale.

D.2-6
Dimostrate che, se A e una matrice $n \times n$ non singolare e simmetrica, allora A^{-1} è simmetrica. Dimostrate che, se B è una matrice $m \times n$ arbitraria, allora la matrice $m \times m$ ottenuta dal prodotto BAB^{T} è simmetrica.

D.2-7
Dimostrate il Teorema D.2. Cioè, dimostrate che una matrice A ha rango di colonna massimo se e soltanto se $Ax = 0$ implica $x = 0$ (*suggerimento:* esprimete la dipendenza lineare di una colonna dalle altre come un'equazione matrice-vettore).

Acquistato da Michele Michele su Webster il 2022-01-[illegible] Copyright © 2010 Edizioni Ambrosiana (Italy)

D.2-8
Dimostrate che per una coppia qualsiasi di matrici compatibili A e B, si ha

$$\mathrm{rank}(AB) \leq \min(\mathrm{rank}(A), \mathrm{rank}(B))$$

L'uguaglianza vale se A (o B) è una matrice quadrata non singolare (*suggerimento:* utilizzate la definizione alternativa del rango di una matrice).

Problemi

D-1 Matrice di Vandermonde

Dati i numeri $x_0, x_1, \ldots, x_{n-1}$, dimostrate che il determinante della ***matrice di Vandermonde***

$$V(x_0, x_1, \ldots, x_{n-1}) = \begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{pmatrix}$$

è

$$\det(V(x_0, x_1, \ldots, x_{n-1})) = \prod_{0 \le j < k \le n-1} (x_k - x_j)$$

(*Suggerimento:* moltiplicate la colonna i per $-x_0$ e sommatela alla colonna $i + 1$ per $i = n - 1, n - 2, \ldots, 1$; poi utilizzate l'induzione.)

D-2 Permutazioni definite dalla moltiplicazione matrice-vettore su GF(2)

Una classe di permutazioni degli interi nell'insieme $S_n = \{0, 1, 2, \ldots, 2^n - 1\}$ è definita dalla moltiplicazione di matrici su $GF(2)$. Per ogni intero x in S_n, indichiamo la sua rappresentazione binaria come un vettore di n bit

$$\begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_{n-1} \end{pmatrix}$$

dove $x = \sum_{i=0}^{n-1} x_i 2^i$. Se A è una matrice $n \times n$ nella quale ogni elemento è 0 o 1, allora possiamo definire una permutazione associando ciascun valore x in S_n al numero la cui rappresentazione binaria è il prodotto matrice-vettore Ax. Qui, eseguiamo tutte le operazioni aritmetiche su ***GF(2)***: tutti i valori sono 0 o 1, tranne un'eccezione, si applicano le solite regole della somma e della moltiplicazione. L'eccezione è che $1 + 1 = 0$. L'aritmetica su $GF(2)$ può essere considerata uguale a quella degli interi, ad eccezione del fatto che si usa soltanto il bit meno significativo. Per esempio, se $S_2 = \{0, 1, 2, 3\}$, la matrice

$$A = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$$

definisce la seguente permutazione π_A: $\pi_A(0) = 0$, $\pi_A(1) = 3$, $\pi_A(2) = 2$, $\pi_A(3) = 1$. Per capire perché $\pi_A(3) = 1$, notiamo che, operando in $GF(2)$, si ha

$$\begin{aligned} \pi_A(3) &= \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 \cdot 1 + 0 \cdot 1 \\ 1 \cdot 1 + 1 \cdot 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 \\ 0 \end{pmatrix} \end{aligned}$$

Questa è la rappresentazione binaria di 1.

Acquistato da Michele Michele su Webster il 2022-07-07 23:13 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Nella parte restante di questo problema opereremo su $GF(2)$ e tutti gli elementi dei vettori e delle matrici sono 0 o 1. Definiamo il rango di una matrice 0-1 (una matrice i cui elementi sono 0 o 1) su $GF(2)$ come per una matrice normale, ad eccezione del fatto che tutte le operazioni aritmetiche che determinano l'indipendenza lineare sono effettuate in $GF(2)$. Definiamo ***immagine*** di una matrice 0-1 A di dimensioni $n \times n$ come

$$R(A) = \{y : y = Ax \text{ per qualche } x \in S_n\}$$

Quindi $R(A)$ è l'insieme dei numeri in S_n che possiamo ottenere moltiplicando ciascun valore x in S_n per A.

a. Se r è il rango delle matrice A, dimostrate che $|R(A)| = 2^r$. Concludete che A definisce una permutazione su S_n se e soltanto se A ha rango massimo.

Per una data matrice A di dimensioni $n \times n$ e un determinato valore $y \in R(A)$, definiamo la ***preimmagine*** di y come

$$P(A, y) = \{x : Ax = y\}$$

Quindi $P(A, y)$ è l'insieme dei valori di S_n che si associano a y quando sono moltiplicati per A.

b. Se r è il rango di una matrice A di dimensioni $n \times n$ e $y \in R(A)$, dimostrate che $|P(A, y)| = 2^{n-r}$.

Se $0 \leq m \leq n$, supponete di suddividere l'insieme S_n in blocchi di numeri consecutivi, dove l'i-esimo blocco è formato dai 2^m numeri $i2^m, i2^m + 1$, $i2^m + 2, \ldots, (i+1)2^m - 1$. Per un sottoinsieme qualsiasi $S \subseteq S_n$, definite $B(S, m)$ come l'insieme dei blocchi di dimensione 2^m di S_n che contiene qualche elemento di S. Per esempio, se $n = 3$, $m = 1$ ed $S = \{1, 4, 5\}$, allora $B(S, m)$ è formato dai blocchi 0 (in quanto 1 è nel blocco 0) e 2 (in quanto 4 e 5 sono nel blocco 2).

c. Sia r il rango della sottomatrice inferiore sinistra di A, avente dimensioni $(n - m) \times m$, ovvero la matrice formata prendendo l'intersezione delle ultime $n - m$ righe in basso e delle prime m colonne a sinistra di A. Sia S un blocco di dimensione 2^m di S_n, e sia $S' = \{y : y = Ax \text{ per qualche } x \in S\}$. Dimostrate che $|B(S', m)| = 2^r$ e che per ciascun blocco in $B(S', m)$, 2^{m-r} numeri in S si associano a tale blocco.

Poiché moltiplicando il vettore zero per una matrice qualsiasi si ottiene un vettore zero, l'insieme delle permutazioni di S_n definito dal prodotto di matrici 0-1, $n \times n$, di rango massimo su $GF(2)$ non può includere tutte le permutazioni di S_n. Estendiamo la classe di permutazioni definita dalla moltiplicazione matrice-vettore per includere un termine additivo, in modo che $x \in S_n$ si associ a $Ax + c$, dove c è un vettore di n bit e la somma è eseguita su $GF(2)$. Per esempio, se

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

$$A = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$$

e

$$c = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

otteniamo la seguente permutazione $\pi_{A,c}$: $\pi_{A,c}(0) = 2$, $\pi_{A,c}(1) = 1$, $\pi_{A,c}(2) = 0$, $\pi_{A,c}(3) = 3$. Chiamiamo ***permutazione lineare*** qualsiasi permutazione che manda $x \in S_n$ in $Ax + c$, per qualche matrice 0-1 A, di dimensioni $n \times n$, di rango massimo e per qualche vettore c di n bit.

d. Utilizzate un contatore per dimostrare che il numero di permutazioni lineari di S_n è molto più piccolo del numero di permutazioni di S_n.

e. Trovate un esempio di un valore di n e di una permutazione di S_n che non possono essere ottenuti mediante una permutazione lineare. (*suggerimento:* per una data permutazione, pensate in che modo la moltiplicazione di una matrice per un vettore unità sia in relazione con le colonne della matrice.)

Note

I libri di algebra lineare forniscono i concetti di base sulle matrici. Strang [324, 325] ha fatto un'eccellente presentazione delle matrici simmetriche e definite positive e dell'algebra lineare in generale.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Bibliografia

[1] Milton Abramowitz and Irene A. Stegun, editors. *Handbook of Mathematical Functions*. Dover, 1965.

[2] G. M. Adel'son-Vel'skiĭ and E. M. Landis. An algorithm for the organization of information. *Soviet Mathematics Doklady*, 3(5):1259–1263, 1962.

[3] Alok Aggarwal and Jeffrey Scott Vitter. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988.

[4] Manindra Agrawal, Neeraj Kayal and Nitin Saxema. PRIMES is in P. *Annals of Mathematics*, (160)2:781–793, 2004.

[5] Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman. *The Design and Analysis of Computer Algorithms*. Addison-Wesley, 1974.

[6] Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman. *Data Structures and Algorithms*. Addison-Wesley, 1983.

[7] Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993.

[8] Ravindra K. Ahuja, Kurt Mehlhorn, James B. Orlin, and Robert E. Tarjan. Faster algorithms for the shortest path problem. *Journal of the ACM*, 37:213–223, 1990.

[9] Ravindra K. Ahuja and James B. Orlin. A fast and simple algorithm for the maximum flow problem. *Operations Research*, 37(5):748–759, 1989.

[10] Ravindra K. Ahuja, James B. Orlin, and Robert E. Tarjan. Improved time bounds for the maximum flow problem. *SIAM Journal on Computing*, 18(5):939–954, 1989.

[11] Miklós Ajtai, Nimrod Megiddo, and Orli Waarts. Improved algorithms and analysis for secretary problems and generalizations. In *Proceedings of the 36th Annual Symposium on Foundations of Computer Science*, pages 473–482, 1995.

[12] Selim G. Akl. *The Design and Analysis of Parallel Algorithms*. Prentice Hall, 1989.

[13] Mohamad Akra and Louay Bazzi. On the solution of linear recurrence equations. *Computational Optimization and Applications*, 10(2):195–210, 1998.

[14] Noga Alon. Generating pseudo-random permutations and maximum flow algorithms. *Information Processing Letters*, 35:201–204, 1990.

[15] Arne Andersson. Balanced search trees made simple. In *Proceedings of the Third Workshop on Algorithms and Data Structures*, volume 709 of *Lecture Notes in Computer Science*, pages 60–71. Springer, 1993.

[16] Arne Andersson. Faster deterministic sorting and searching in linear space. In *Proceedings of the 37th Annual Symposium on Foundations of Computer Science*, pages 135–141, 1996.

[17] Arne Andersson, Torben Hagerup, Stefan Nilsson, and Rajeev Raman. Sorting in linear time? *Journal of Computer and System Sciences*, 57:74–93, 1998.

[18] Tom M. Apostol. *Calculus*, volume 1. Blaisdell Publishing Company, second edition, 1967.

[19] Nimar S. Arora, Robert D. Blumofe, and C. Greg Plaxton. Thread scheduling for multiprogrammed multiprocessors. In *Proceedings of the 10th Annual ACM Symposium on Parallel Algorithms and Architectures*, pages 119–129, 1998.

[20] Sanjeev Arora. *Probabilistic checking of proofs and the hardness of approximation problems*. PhD thesis, University of California, Berkeley, 1994.

[21] Sanjeev Arora. The approximability of NP-hard problems. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing*, pages 337–348, 1998.

[22] Sanjeev Arora. Polynomial time approximation schemes for euclidean traveling salesman and other geometric problems. *Journal of the ACM*, 45(5):753–782, 1998.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 - Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[23] Sanjeev Arora and Carsten Lund. Hardness of approximations. In Dorit S. Hochbaum, editor, *Approximation Algorithms for NP-Hard Problems*, pages 399–446. PWS Publishing Company, 1997.

[24] Javed A. Aslam. A simple bound on the expected height of a randomly built binary search tree. Technical Report TR2001-387, Dartmouth College Department of Computer Science, 2001.

[25] Mikhail J. Atallah, editor. *Algorithms and Theory of Computation Handbook*. CRC Press, 1999.

[26] G. Ausiello, P. Crescenzi, G. Gambosi, V. Kann, A. Marchetti-Spaccamela, and M. Protasi. *Complexity and Approximation: Combinatorial Optimization Problems and Their Approximability Properties*. Springer, 1999.

[27] Shai Avidan and Ariel Shamir. Seam carving for content-aware image resizing. *ACM Transactions on Graphics*, 26(3), article 10, 2007.

[28] Sara Baase and Alan Van Gelder. *Computer Algorithms: Introduction to Design and Analysis*. Addison-Wesley, third edition, 2000.

[29] Eric Bach. Private communication, 1989.

[30] Eric Bach. Number-theoretic algorithms. In *Annual Review of Computer Science*, volume 4, pages 119–172. Annual Reviews, Inc., 1990.

[31] Eric Bach and Jeffrey Shallit. *Algorithmic Number Theory—Volume I: Efficient Algorithms*. The MIT Press, 1996.

[32] David H. Bailey, King Lee, and Horst D. Simon. Using Strassen's algorithm to accelerate the solution of linear systems. *The Journal of Supercomputing*, 4(4):357–371, 1990.

[33] Surender Baswana, Ramesh Hariharan, and Sandeep Sen. Improved decremental algorithms for maintaining transitive closure and all-pairs shortest paths. *Journal of Algorithms*, 62(2):74–92, 2007.

[34] R. Bayer. Symmetric binary B-trees: Data structure and maintenance algorithms. *Acta Informatica*, 1(4):290–306, 1972.

[35] R. Bayer and E. M. McCreight. Organization and maintenance of large ordered indexes. *Acta Informatica*, 1(3):173–189, 1972.

[36] Pierre Beauchemin, Gilles Brassard, Claude Crépeau, Claude Goutier, and Carl Pomerance. The generation of random numbers that are probably prime. *Journal of Cryptology*, 1(1):53–64, 1988.

[37] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[38] Richard Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16(1):87–90, 1958.

[39] Michael Ben-Or. Lower bounds for algebraic computation trees. In *Proceedings of the Fifteenth Annual ACM Symposium on Theory of Computing*, pages 80–86, 1983.

[40] Michael A. Bender, Erik D. Demaine, and Martin Farach-Colton. Cache-oblivious B-trees. In *Proceedings of the 41st Annual Symposium on Foundations of Computer Science*, pages 399–409, 2000.

[41] Samuel W. Bent and John W. John. Finding the median requires $2n$ comparisons. In *Proceedings of the Seventeenth Annual ACM Symposium on Theory of Computing*, pages 213–216, 1985.

[42] Jon L. Bentley. *Writing Efficient Programs*. Prentice Hall, 1982.

[43] Jon L. Bentley. *Programming Pearls*. Addison-Wesley, 1986.

[44] Jon L. Bentley, Dorothea Haken, and James B. Saxe. A general method for solving divide-and-conquer recurrences. *SIGACT News*, 12(3):36–44, 1980.

[45] Daniel Bienstock and Benjamin McClosky. Tightening simplex mixed-integer sets with guaranteed bounds. *Optimization Online*, July 2008.

[46] Patrick Billingsley. *Probability and Measure*. John Wiley & Sons, second edition, 1986.

[47] Guy E. Blelloch. *Scan Primitives and Parallel Vector Models*. PhD thesis, Department of Electrical Engineering and Computer Science, MIT, 1989. Available as MIT Laboratory for Computer Science Technical Report MIT/LCS/TR-463.

[48] Guy E. Blelloch. Programming parallel algorithms. *Communications of the ACM*, 39(3):85–97, 1996.

[49] Guy E. Blelloch, Phillip B. Gibbons, and Yossi Matias. Provably efficient scheduling for languages with fine-grained parallelism. In *Proceedings of the 7th Annual ACM Symposium on Parallel Algorithms and Architectures*, pages 1–12, 1995.

[50] Manuel Blum, Robert W. Floyd, Vaughan Pratt, Ronald L. Rivest, and Robert E. Tarjan. Time bounds for selection. *Journal of Computer and System Sciences*, 7(4):448–461, 1973.

[51] Robert D. Blumofe, Christopher F. Joerg, Bradley C. Kuszmaul, Charles E. Leiserson, Keith H. Randall, and Yuli Zhou. Cilk: An efficient multithreaded runtime system. *Journal of Parallel and Distributed Computing*, 37(1):55–69, 1996.

[52] Robert D. Blumofe and Charles E. Leiserson. Space-efficient scheduling of multithreaded computations. *SIAM Journal on Computing*, 27(1):202–229, 1998.

Acquistato da Michele Mastrangelo (mastrangelo.michele@gmail.com) - Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[53] Robert D. Blumofe and Charles E. Leiserson. Scheduling multithreaded computations by work stealing. *Journal of the ACM*, 46(5):720–748, 1999.

[54] Béla Bollobás. *Random Graphs*. Academic Press, 1985.

[55] Gilles Brassard and Paul Bratley. *Fundamentals of Algorithmics*. Prentice Hall, 1996.

[56] Richard P. Brent. The parallel evaluation of general arithmetic expressions. *Journal of the ACM*, 21(2):201–206, 1974.

[57] Richard P. Brent. An improved Monte Carlo factorization algorithm. *BIT*, 20(2):176–184, 1980.

[58] J. P. Buhler, H. W. Lenstra, Jr., and Carl Pomerance. Factoring integers with the number field sieve. In A. K. Lenstra and H. W. Lenstra, Jr., editors, *The Development of the Number Field Sieve*, volume 1554 of *Lecture Notes in Mathematics*, pages 50–94. Springer, 1993.

[59] J. Lawrence Carter and Mark N. Wegman. Universal classes of hash functions. *Journal of Computer and System Sciences*, 18(2):143–154, 1979.

[60] Barbara Chapman, Gabriele Jost, and Ruud van der Pas. *Using OpenMP: Portable Shared Memory Parallel Programming*. The MIT Press, 2007.

[61] Bernard Chazelle. A minimum spanning tree algorithm with inverse-Ackermann type complexity. *Journal of the ACM*, 47(6):1028–1047, 2000.

[62] Joseph Cheriyan and Torben Hagerup. A randomized maximum-flow algorithm. *SIAM Journal on Computing*, 24(2): 203–226, 1995.

[63] Joseph Cheriyan and S. N. Maheshwari. Analysis of preflow push algorithms for maximum network flow. *SIAM Journal on Computing*, 18(6):1057–1086, 1989.

[64] Boris V. Cherkassky and Andrew V. Goldberg. On implementing the push-relabel method for the maximum flow problem. *Algorithmica*, 19(4):390–410, 1997.

[65] Boris V. Cherkassky, Andrew V. Goldberg, and Tomasz Radzik. Shortest paths algorithms: Theory and experimental evaluation. *Mathematical Programming*, 73(2):129–174, 1996.

[66] Boris V. Cherkassky, Andrew V. Goldberg, and Craig Silverstein. Buckets, heaps, lists and monotone priority queues. *SIAM Journal on Computing*, 28(4):1326–1346, 1999.

[67] H. Chernoff. A measure of asymptotic efficiency for tests of a hypothesis based on the sum of observations. *Annals of Mathematical Statistics*, 23(4):493–507, 1952.

[68] Kai Lai Chung. *Elementary Probability Theory with Stochastic Processes*. Springer, 1974.

[69] V. Chvátal. A greedy heuristic for the set-covering problem. *Mathematics of Operations Research*, 4(3):233–235, 1979.

[70] V. Chvátal. *Linear Programming*. W. H. Freeman and Company, 1983.

[71] V. Chvátal, D. A. Klarner, and D. E. Knuth. Selected combinatorial research problems. Technical Report STAN-CS-72-292, Computer Science Department, Stanford University, 1972.

[72] Cilk Arts, Inc., Burlington, Massachusetts. *Cilk++ Programmer's Guide*, 2008.
Available at http://www.cilk.com/archive/docs/cilk1guide.

[73] Alan Cobham. The intrinsic computational difficulty of functions. In *Proceedings of the 1964 Congress for Logic, Methodology, and the Philosophy of Science*, pages 24–30. North-Holland, 1964.

[74] H. Cohen and H. W. Lenstra, Jr. Primality testing and Jacobi sums. *Mathematics of Computation*, 42(165):297–330, 1984.

[75] D. Comer. The ubiquitous B-tree. *ACM Computing Surveys*, 11(2):121–137, 1979.

[76] Stephen Cook. The complexity of theorem proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, pages 151–158, 1971.

[77] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301, 1965.

[78] Don Coppersmith. Modifications to the number field sieve. *Journal of Cryptology*, 6(3):169–180, 1993.

[79] Don Coppersmith and Shmuel Winograd. Matrix multiplication via arithmetic progression. *Journal of Symbolic Computation*, 9(3):251–280, 1990.

[80] Thomas H. Cormen, Thomas Sundquist, and Leonard F. Wisniewski. Asymptotically tight bounds for performing BMMC permutations on parallel disk systems. *SIAM Journal on Computing*, 28(1):105–136, 1998.

[81] Don Dailey and Charles E. Leiserson. Using Cilk to write multiprocessor chess programs. In H. J. van den Herik and B. Monien, editors, *Advances in Computer Games*, volume 9, pages 25–52. University of Maastricht, Netherlands, 2001.

[82] Paolo D'Alberto and Alexandru Nicolau. Adaptive Strassen's matrix multiplication. In *Proceedings of the 21st Annual International Conference on Supercomputing*, pages 284–292, June 2007.

[83] Sanjoy Dasgupta, Christos Papadimitriou, and Umesh Vazirani. *Algorithms*. McGraw-Hill, 2008.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[84] Roman Dementiev, Lutz Kettner, Jens Mehnert, and Peter Sanders. Engineering a sorted list data structure for 32 bit keys. In *Proceedings of the Sixth Workshop on Algorithm Engineering and Experiments and the First Workshop on Analytic Algorithmics and Combinatorics*, pages 142–151, January 2004.

[85] Camil Demetrescu and Giuseppe F. Italiano. Fully dynamic all pairs shortest paths with real edge weights. *Journal of Computer and System Sciences*, 72(5):813–837, 2006.

[86] Eric V. Denardo and Bennett L. Fox. Shortest-route methods: 1. Reaching, pruning, and buckets. *Operations Research*, 27(1):161–186, 1979.

[87] Martin Dietzfelbinger, Anna Karlin, Kurt Mehlhorn, Friedhelm Meyer auf der Heide, Hans Rohnert, and Robert E. Tarjan. Dynamic perfect hashing: Upper and lower bounds. *SIAM Journal on Computing*, 23(4):738–761, 1994.

[88] Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, IT-22(6):644–654, 1976.

[89] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959.

[90] E. A. Dinic. Algorithm for solution of a problem of maximum flow in a network with power estimation. *Soviet Mathematics Doklady*, 11(5):1277–1280, 1970.

[91] Brandon Dixon, Monika Rauch, and Robert E. Tarjan. Verification and sensitivity analysis of minimum spanning trees in linear time. *SIAM Journal on Computing*, 21(6):1184–1192, 1992.

[92] John D. Dixon. Factorization and primality tests. *The American Mathematical Monthly*, 91(6):333–352, 1984.

[93] Dorit Dor, Johan Håstad, Staffan Ulfberg, and Uri Zwick. On lower bounds for selecting the median. *SIAM Journal on Discrete Mathematics*, 14(3):299–311, 2001.

[94] Dorit Dor and Uri Zwick. Selecting the median. *SIAM Journal on Computing*, 28(5):1722–1758, 1999.

[95] Dorit Dor and Uri Zwick. Median selection requires $(2 + \epsilon)n$ comparisons. *SIAM Journal on Discrete Mathematics*, 14(3):312–325, 2001.

[96] Alvin W. Drake. *Fundamentals of Applied Probability Theory*. McGraw-Hill, 1967.

[97] James R. Driscoll, Harold N. Gabow, Ruth Shrairman, and Robert E. Tarjan. Relaxed heaps: An alternative to Fibonacci heaps with applications to parallel computation. *Communications of the ACM*, 31(11):1343–1354, 1988.

[98] James R. Driscoll, Neil Sarnak, Daniel D. Sleator, and Robert E. Tarjan. Making data structures persistent. *Journal of Computer and System Sciences*, 38(1):86–124, 1989.

[99] Derek L. Eager, John Zahorjan and Edward D. Lazowska. Speedup versus efficiency in parallel systems. *IEEE Transactions on Computer*, 33(3):408–423, 1989.

[100] Herbert Edelsbrunner. *Algorithms in Combinatorial Geometry*, volume 10 of *EATCS Monographs on Theoretical Computer Science*. Springer, 1987.

[101] Jack Edmonds. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17:449–467, 1965.

[102] Jack Edmonds. Matroids and the greedy algorithm. *Mathematical Programming*, 1(1):127–136, 1971.

[103] Jack Edmonds and Richard M. Karp. Theoretical improvements in the algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2):248–264, 1972.

[104] Shimon Even. *Graph Algorithms*. Computer Science Press, 1979.

[105] William Feller. *An Introduction to Probability Theory and Its Applications*. John Wiley & Sons, third edition, 1968.

[106] Robert W. Floyd. Algorithm 97 (SHORTEST PATH). *Communications of the ACM*, 5(6):345, 1962.

[107] Robert W. Floyd. Algorithm 245 (TREESORT). *Communications of the ACM*, 7(12):701, 1964.

[108] Robert W. Floyd. Permuting information in idealized two-level storage. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 105–109. Plenum Press, 1972.

[109] Robert W. Floyd and Ronald L. Rivest. Expected time bounds for selection. *Communications of the ACM*, 18(3):165–172, 1975.

[110] Lestor R. Ford, Jr. and D. R. Fulkerson. *Flows in Networks*. Princeton University Press, 1962.

[111] Lestor R. Ford, Jr. and Selmer M. Johnson. A tournament problem. *The American Mathematical Monthly*, 66(5):387–389, 1959.

[112] Michael L. Fredman. New bounds on the complexity of the shortest path problem. *SIAM Journal on Computing*, 5(1):83–89, 1976.

[113] Michael L. Fredman, János Komlós, and Endre Szemerédi. Storing a sparse table with $O(1)$ worst case access time. *Journal of the ACM*, 31(3):538–544, 1984.

[114] Michael L. Fredman and Michael E. Saks. The cell probe complexity of dynamic data structures. In *Proceedings of the Twenty First Annual ACM Symposium on Theory of Computing*, pages 345–354, 1989.

[115] Michael L. Fredman and Robert E. Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, 34(3):596–615, 1987.

Acquistato da Michele Mereta su weBook | 2022-07-07 23:12 | Napoli Online Libreria | 9788838619220 | Copyright © 2022, McGraw-Hill Education (Italy)

[116] Michael L. Fredman and Dan E. Willard. Surpassing the information theoretic bound with fusion trees. *Journal of Computer and System Sciences*, 47(3):424–436, 1993.

[117] Michael L. Fredman and Dan E. Willard. Trans-dichotomous algorithms for minimum spanning trees and shortest paths. *Journal of Computer and System Sciences*, 48(3):533–551, 1994.

[118] Matteo Frigo and Steven G. Johnson. The design and implementation of FFTW3. *Proceedings of the IEEE*, 93(2):216–231, 2005.

[119] Matteo Frigo, Charles E. Leiserson, and Keith H. Randall. The implementation of the Cilk-5 multithreaded language. In *Proceedings of the 1998 ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 212–223, 1998.

[120] Harold N. Gabow. Path-based depth-first search for strong and biconnected components. *Information Processing Letters*, 74(3–4):107–114, 2000.

[121] Harold N. Gabow, Z. Galil, T. Spencer, and Robert E. Tarjan. Efficient algorithms for finding minimum spanning trees in undirected and directed graphs. *Combinatorica*, 6(2):109–122, 1986.

[122] Harold N. Gabow and Robert E. Tarjan. A linear-time algorithm for a special case of disjoint set union. *Journal of Computer and System Sciences*, 30(2):209–221, 1985.

[123] Harold N. Gabow and Robert E. Tarjan. Faster scaling algorithms for network problems. *SIAM Journal on Computing*, 18(5):1013–1036, 1989.

[124] Zvi Galil and Oded Margalit. All pairs shortest distances for graphs with small integer length edges. *Information and Computation*, 134(2):103–139, 1997.

[125] Zvi Galil and Oded Margalit. All pairs shortest paths for graphs with small integer length edges. *Journal of Computer and System Sciences*, 54(2):243–254, 1997.

[126] Zvi Galil and Kunsoo Park. Dynamic programming with convexity, concavity and sparsity. *Theoretical Computer Science*, 92(1):49–76, 1992.

[127] Zvi Galil and Joel Seiferas. Time-space-optimal string matching. *Journal of Computer and System Sciences*, 26(3):280–294, 1983.

[128] Igal Galperin and Ronald L. Rivest. Scapegoat trees. In *Proceedings of the 4th ACM-SIAM Symposium on Discrete Algorithms*, pages 165–174, 1993.

[129] Michael R. Garey, R. L. Graham, and J. D. Ullman. Worst-case analyis of memory allocation algorithms. In *Proceedings of the Fourth Annual ACM Symposium on Theory of Computing*, pages 143–150, 1972.

[130] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[131] Saul Gass. *Linear Programming: Methods and Applications*. International Thomson Publishing, fourth edition, 1975.

[132] Fănică Gavril. Algorithms for minimum coloring, maximum clique, minimum covering by cliques, and maximum independent set of a chordal graph. *SIAM Journal on Computing*, 1(2):180–187, 1972.

[133] Alan George and Joseph W-H Liu. *Computer Solution of Large Sparse Positive Definite Systems*. Prentice Hall, 1981.

[134] E. N. Gilbert and E. F. Moore. Variable-length binary encodings. *Bell System Technical Journal*, 38(4):933–967, 1959.

[135] Michel X. Goemans and David P. Williamson. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *Journal of the ACM*, 42(6):1115–1145, 1995.

[136] Michel X. Goemans and David P. Williamson. The primal-dual method for approximation algorithms and its application to network design problems. In Dorit S. Hochbaum, editor, *Approximation Algorithms for NP-Hard Problems*, pages 144–191. PWS Publishing Company, 1997.

[137] Andrew V. Goldberg. *Efficient Graph Algorithms for Sequential and Parallel Computers*. PhD thesis, Department of Electrical Engineering and Computer Science, MIT, 1987.

[138] Andrew V. Goldberg. Scaling algorithms for the shortest paths problem. *SIAM Journal on Computing*, 24(3):494–504, 1995.

[139] Andrew V. Goldberg and Satish Rao. Beyond the flow decomposition barrier. *Journal of the ACM*, 45(5):783–797, 1998.

[140] Andrew V. Goldberg, Éva Tardos, and Robert E. Tarjan. Network flow algorithms. In Bernhard Korte, László Lovász, Hans Jürgen Prömel, and Alexander Schrijver, editors, *Paths, Flows, and VLSI-Layout*, pages 101–164. Springer, 1990.

[141] Andrew V. Goldberg and Robert E. Tarjan. A new approach to the maximum flow problem. *Journal of the ACM*, 35(4): 921-940, 1988.

[142] D. Goldfarb and M. J. Todd. Linear programming. In G. L. Nemhauser, A. H. G. Rinnooy-Kan, and M. J. Todd, editors, *Handbook in Operations Research and Management Science, Vol. 1, Optimization*, pages 73–170. Elsevier Science Publishers, 1989.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[143] Shafi Goldwasser and Silvio Micali. Probabilistic encryption. *Journal of Computer and System Sciences*, 28(2):270–299, 1984.

[144] Shafi Goldwasser, Silvio Micali, and Ronald L. Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM Journal on Computing*, 17(2):281–308, 1988.

[145] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. The Johns Hopkins University Press, third edition, 1996.

[146] G. H. Gonnet. *Handbook of Algorithms and Data Structures*. Addison-Wesley, 1984.

[147] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Addison-Wesley, 1992.

[148] Michael T. Goodrich and Roberto Tamassia. *Data Structures and Algorithms in Java*. John Wiley & Sons, 1998.

[149] Michael T. Goodrich and Roberto Tamassia. *Algorithm Design: Foundations, Analysis, and Internet Examples*. John Wiley & Sons, 2001.

[150] Ronald L. Graham. Bounds for certain multiprocessor anomalies. *Bell System Technical Journal*, 45(9):1563–1581, 1966.

[151] Ronald L. Graham. An efficient algorithm for determining the convex hull of a finite planar set. *Information Processing Letters*, 1(4):132–133, 1972.

[152] Ronald L. Graham and Pavol Hell. On the history of the minimum spanning tree problem. *Annals of the History of Computing*, 7(1):43–57, 1985.

[153] Ronald L. Graham, Donald E. Knuth, and Oren Patashnik. *Concrete Mathematics*. Addison-Wesley, second edition, 1994.

[154] David Gries. *The Science of Programming*. Springer, 1981.

[155] M. Grötschel, László Lovász, and Alexander Schrijver. *Geometric Algorithms and Combinatorial Optimization*. Springer, 1988.

[156] Leo J. Guibas and Robert Sedgewick. A dichromatic framework for balanced trees. In *Proceedings of the 19th Annual Symposium on Foundations of Computer Science*, pages 8–21, 1978.

[157] Dan Gusfield. *Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology*. Cambridge University Press, 1997.

[158] H. Halberstam and R. E. Ingram, editors. *The Mathematical Papers of Sir William Rowan Hamilton*, volume III (Algebra). Cambridge University Press, 1967.

[159] Yijie Han. Improved fast integer sorting in linear space. In *Proceedings of the 12th ACM-SIAM Symposium on Discrete Algorithms*, pages 793–796, 2001.

[160] Yijie Han. An $O(n^3(\log\log n/\log n)^{5/4})$ time algorithm for all pairs shortest path. *Algorithmica*, 51(4):428–434, 2008.

[161] Frank Harary. *Graph Theory*. Addison-Wesley, 1969.

[162] Gregory C. Harfst and Edward M. Reingold. A potential-based amortized analysis of the union-find data structure. *SIGACT News*, 31(3):86–95, 2000.

[163] J. Hartmanis and R. E. Stearns. On the computational complexity of algorithms. *Transactions of the American Mathematical Society*, 117:285–306, May 1965.

[164] Michael T. Heideman, Don H. Johnson, and C. Sidney Burrus. Gauss and the history of the Fast Fourier Transform. *IEEE ASSP Magazine*, 1(4):14–21, 1984.

[165] Monika R. Henzinger and Valerie King. Fully dynamic biconnectivity and transitive closure. In *Proceedings of the 36th Annual Symposium on Foundations of Computer Science*, pages 664–672, 1995.

[166] Monika R. Henzinger and Valerie King. Randomized fully dynamic graph algorithms with polylogarithmic time per operation. *Journal of the ACM*, 46(4):502–516, 1999.

[167] Monika R. Henzinger, Satish Rao, and Harold N. Gabow. Computing vertex connectivity: New bounds from old techniques. *Journal of Algorithms*, 34(2):222–250, 2000.

[168] Nicholas J. Higham. Exploiting fast matrix multiplication within the level 3 BLAS. *ACM Transactions on Mathematical Software*, 16(4):352–368, 1990.

[169] W. Daniel Hillis and Jr. Guy L. Steele. Data parallel algorithms. *Communications of the ACM*, 29(12):1170–1183, 1986.

[170] C. A. R. Hoare. Algorithm 63 (PARTITION) and algorithm 65 (FIND). *Communications of the ACM*, 4(7):321–322, 1961.

[171] C. A. R. Hoare. Quicksort. *Computer Journal*, 5(1):10–15, 1962.

[172] Dorit S. Hochbaum. Efficient bounds for the stable set, vertex cover and set packing problems. *Discrete Applied Mathematics*, 6(3):243–254, 1983.

[173] Dorit S. Hochbaum, editor. *Approximation Algorithms for NP-Hard Problems*. PWS Publishing Company, 1997.

[174] W. Hoeffding. On the distribution of the number of successes in independent trials. *Annals of Mathematical Statistics*, 27(3):713–721, 1956.

[175] Micha Hofri. *Probabilistic Analysis of Algorithms*. Springer, 1987.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 19950316-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[176] Micha Hofri. *Analysis of Algorithms*. Oxford University Press, 1995.

[177] John E. Hopcroft and Richard M. Karp. An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs. *SIAM Journal on Computing*, 2(4):225–231, 1973.

[178] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison Wesley, third edition, 2006.

[179] John E. Hopcroft and Robert E. Tarjan. Efficient algorithms for graph manipulation. *Communications of the ACM*, 16(6):372–378, 1973.

[180] John E. Hopcroft and Jeffrey D. Ullman. Set merging algorithms. *SIAM Journal on Computing*, 2(4):294–303, 1973.

[181] John E. Hopcroft and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley, 1979.

[182] Ellis Horowitz, Sartaj Sahni, and Sanguthevar Rajasekaran. *Computer Algorithms*. Computer Science Press, 1998.

[183] T. C. Hu and M. T. Shing. Computation of matrix chain products. Part I. *SIAM Journal on Computing*, 11(2):362–373, 1982.

[184] T. C. Hu and M. T. Shing. Computation of matrix chain products. Part II. *SIAM Journal on Computing*, 13(2):228–251, 1984.

[185] T. C. Hu and A. C. Tucker. Optimal computer search trees and variable-length alphabetic codes. *SIAM Journal on Applied Mathematics*, 21(4):514–532, 1971.

[186] David A. Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9):1098–1101, 1952.

[187] Steven Huss-Lederman, Elaine M. Jacobson, Jeremy R. Johnson, Anna Tsao, and Thomas Turnbull. Implementation of Strassen's algorithm for matrix multiplication. In *Proceedings of the 1996 ACM/IEEE Conference on Supercomputing*, article 32, 1996.

[188] Oscar H. Ibarra and Chul E. Kim. Fast approximation algorithms for the knapsack and sum of subset problems. *Journal of the ACM*, 22(4):463–468, 1975.

[189] E. J. Isaac and R. C. Singleton. Sorting by address calculation. *Journal of the ACM*, 3(3):169–174, 1956.

[190] R. A. Jarvis. On the identification of the convex hull of a finite set of points in the plane. *Information Processing Letters*, 2(1):18–21, 1973.

[191] David S. Johnson. Approximation algorithms for combinatorial problems. *Journal of Computer and System Sciences*, 9(3):256–278, 1974.

[192] David S. Johnson. The NP-completeness column: An ongoing guide—The tale of the second prover. *Journal of Algorithms*, 13(3):502–524, 1992.

[193] Donald B. Johnson. Efficient algorithms for shortest paths in sparse networks. *Journal of the ACM*, 24(1):1–13, 1977.

[194] Richard Johnsonbaugh and Marcus Schaefer. *Algorithms*. Pearson Prentice Hall, 2004.

[195] A. Karatsuba and Yu. Ofman. Multiplication of multidigit numbers on automata. *Soviet Physics—Doklady*, 7(7):595–596, 1963. Translation of an article in *Doklady Akademii Nauk SSSR*, 145(2), 1962.

[196] David R. Karger, Philip N. Klein, and Robert E. Tarjan. A randomized linear-time algorithm to find minimum spanning trees. *Journal of the ACM*, 42(2):321–328, 1995.

[197] David R. Karger, Daphne Koller, and Steven J. Phillips. Finding the hidden path: Time bounds for all-pairs shortest paths. *SIAM Journal on Computing*, 22(6):1199–1217, 1993.

[198] Howard Karloff. *Linear Programming*. Birkhäuser, 1991.

[199] N. Karmarkar. A new polynomial-time algorithm for linear programming. *Combinatorica*, 4(4):373–395, 1984.

[200] Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972.

[201] Richard M. Karp. An introduction to randomized algorithms. *Discrete Applied Mathematics*, 34(1–3):165–201, 1991.

[202] Richard M. Karp and Michael O. Rabin. Efficient randomized pattern-matching algorithms. *IBM Journal of Research and Development*, 31(2):249–260, 1987.

[203] A. V. Karzanov. Determining the maximal flow in a network by the method of preflows. *Soviet Mathematics Doklady*, 15(2):434–437, 1974.

[204] Valerie King. A simpler minimum spanning tree verification algorithm. *Algorithmica*, 18(2):263–270, 1997.

[205] Valerie King, Satish Rao, and Robert E. Tarjan. A faster deterministic maximum flow algorithm. *Journal of Algorithms*, 17(3):447–474, 1994.

[206] Jeffrey H. Kingston. *Algorithms and Data Structures: Design, Correctness, Analysis*. Addison-Wesley, second edition, 1997.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[207] D. G. Kirkpatrick and R. Seidel. The ultimate planar convex hull algorithm? *SIAM Journal on Computing*, 15(2):287–299, 1986.

[208] Philip N. Klein and Neal E. Young. Approximation algorithms for NP-hard optimization problems. In *CRC Handbook on Algorithms*, pages 34-1–34-19. CRC Press, 1999.

[209] Jon Kleinberg and Éva Tardos. *Algorithm Design*. Addison-Wesley, 2006.

[210] Donald E. Knuth. *Fundamental Algorithms*, volume 1 of *The Art of Computer Programming*. Addison-Wesley, 1968. Third edition, 1997.

[211] Donald E. Knuth. *Seminumerical Algorithms*, volume 2 of *The Art of Computer Programming*. Addison-Wesley, 1969. Third edition, 1997.

[212] Donald E. Knuth. *Sorting and Searching*, volume 3 of *The Art of Computer Programming*. Addison-Wesley, 1973. Second edition, 1998.

[213] Donald E. Knuth. Optimum binary search trees. *Acta Informatica*, 1(1):14–25, 1971.

[214] Donald E. Knuth. Big omicron and big omega and big theta. *SIGACT News*, 8(2):18–23, 1976.

[215] Donald E. Knuth, James H. Morris, Jr., and Vaughan R. Pratt. Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2):323–350, 1977.

[216] J. Komlós. Linear verification for spanning trees. *Combinatorica*, 5(1):57–65, 1985.

[217] Bernhard Korte and László Lovász. Mathematical structures underlying greedy algorithms. In F. Gecseg, editor, *Fundamentals of Computation Theory*, volume 117 of *Lecture Notes in Computer Science*, pages 205–209. Springer, 1981.

[218] Bernhard Korte and László Lovász. Structural properties of greedoids. *Combinatorica*, 3(3–4):359–374, 1983.

[219] Bernhard Korte and László Lovász. Greedoids—A structural framework for the greedy algorithm. In W. Pulleybank, editor, *Progress in Combinatorial Optimization*, pages 221–243. Academic Press, 1984.

[220] Bernhard Korte and László Lovász. Greedoids and linear objective functions. *SIAM Journal on Algebraic and Discrete Methods*, 5(2):229–238, 1984.

[221] Dexter C. Kozen. *The Design and Analysis of Algorithms*. Springer, 1992.

[222] David W. Krumme, George Cybenko, and K. N. Venkataraman. Gossiping in minimal time. *SIAM Journal on Computing*, 21(1):111–139, 1992.

[223] Joseph B. Kruskal, Jr. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7(1):48–50, 1956.

[224] Leslie Lamport. How to make a multiprocessor computer that correctly executes multiprocess programs. *IEEE Transactions on Computers*, C-28(9):690–691, 1979.

[225] Eugene L. Lawler. *Combinatorial Optimization: Networks and Matroids*. Holt, Rinehart, and Winston, 1976.

[226] Eugene L. Lawler, J. K. Lenstra, A. H. G. Rinnooy Kan, and D. B. Shmoys, editors. *The Traveling Salesman Problem*. John Wiley & Sons, 1985.

[227] C. Y. Lee. An algorithm for path connection and its applications. *IRE Transactions on Electronic Computers*, EC-10(3): 346-365, 1961.

[228] Tom Leighton. Tight bounds on the complexity of parallel sorting. *IEEE Transactions on Computers*, C-34(4):344–354, 1985.

[229] Tom Leighton. Notes on better master theorems for divide-and-conquer recurrences. Class notes. Available at http://citeseer.ist.psu.edu/252350.html, October 1996.

[230] Tom Leighton and Satish Rao. Multicommodity max-flow min-cut theorems and their use in designing approximation algorithms. *Journal of the ACM*, 46(6):787–832, 1999.

[231] Daan Leijen and Judd Hall. Optimize managed code for multi-core machines. *MSDN Magazine*, October 2007.

[232] Debra A. Lelewer and Daniel S. Hirschberg. Data compression. *ACM Computing Surveys*, 19(3):261–296, 1987.

[233] A. K. Lenstra, H. W. Lenstra, Jr., M. S. Manasse, and J. M. Pollard. The number field sieve. In A. K. Lenstra and H. W. Lenstra, Jr., editors, *The Development of the Number Field Sieve*, volume 1554 of *Lecture Notes in Mathematics*, pages 11–42. Springer, 1993.

[234] H. W. Lenstra, Jr. Factoring integers with elliptic curves. *Annals of Mathematics*, 126(3):649–673, 1987.

[235] L. A. Levin. Universal sorting problems. *Problemy Peredachi Informatsii*, 9(3):265–266, 1973. In Russian.

[236] Anany Levitin. *Introduction to the Design & Analysis of Algorithms*. Addison-Wesley, 2007.

[237] Harry R. Lewis and Christos H. Papadimitriou. *Elements of the Theory of Computation*. Prentice Hall, second edition, 1998.

[238] C. L. Liu. *Introduction to Combinatorial Mathematics*. McGraw-Hill, 1968.

[239] László Lovász. On the ratio of optimal integral and fractional covers. *Discrete Mathematics*, 13(4):383–390, 1975.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[240] László Lovász and M. D. Plummer. *Matching Theory*, volume 121 of *Annals of Discrete Mathematics*. North Holland, 1986.

[241] Bruce M. Maggs and Serge A. Plotkin. Minimum-cost spanning tree as a path-finding problem. *Information Processing Letters*, 26(6):291–293, 1988.

[242] Michael Main. *Data Structures and Other Objects Using Java*. Addison-Wesley, 1999.

[243] Udi Manber. *Introduction to Algorithms: A Creative Approach*. Addison-Wesley, 1989.

[244] Conrado Martínez and Salvador Roura. Randomized binary search trees. *Journal of the ACM*, 45(2):288–323, 1998.

[245] William J. Masek and Michael S. Paterson. A faster algorithm computing string edit distances. *Journal of Computer and System Sciences*, 20(1):18–31, 1980.

[246] H. A. Maurer, Th. Ottmann, and H.-W. Six. Implementing dictionaries using binary trees of very small height. *Information Processing Letters*, 5(1):11–14, 1976.

[247] Ernst W. Mayr, Hans Jürgen Prömel, and Angelika Steger, editors. *Lectures on Proof Verification and Approximation Algorithms*, volume 1367 of *Lecture Notes in Computer Science*. Springer, 1998.

[248] C. C. McGeoch. All pairs shortest paths and the essential subgraph. *Algorithmica*, 13(5):426–441, 1995.

[249] M. D. McIlroy. A killer adversary for quicksort. *Software—Practice and Experience*, 29(4):341–344, 1999.

[250] Kurt Mehlhorn. *Sorting and Searching*, volume 1 of *Data Structures and Algorithms*. Springer, 1984.

[251] Kurt Mehlhorn. *Graph Algorithms and NP-Completeness*, volume 2 of *Data Structures and Algorithms*. Springer, 1984.

[252] Kurt Mehlhorn. *Multidimensional Searching and Computational Geometry*, volume 3 of *Data Structures and Algorithms*. Springer, 1984.

[253] Kurt Mehlhorn and Stefan Näher. Bounded ordered dictionaries in $O(\log \log N)$ time and $O(n)$ space. *Information Processing Letters*, 35(4):183–189, 1990.

[254] Kurt Mehlhorn and Stefan Näher. *LEDA: A Platform for Combinatorial and Geometric Computing*. Cambridge University Press, 1999.

[255] Alfred J. Menezes, Paul C. van Oorschot, and Scott A. Vanstone. *Handbook of Applied Cryptography*. CRC Press, 1997.

[256] Gary L. Miller. Riemann's hypothesis and tests for primality. *Journal of Computer and System Sciences*, 13(3):300–317, 1976.

[257] John C. Mitchell. *Foundations for Programming Languages*. The MIT Press, 1996.

[258] Joseph S. B. Mitchell. Guillotine subdivisions approximate polygonal subdivisions: A simple polynomial-time approximation scheme for geometric TSP, k-MST, and related problems. *SIAM Journal on Computing*, 28(4):1298–1309, 1999.

[259] Louis Monier. *Algorithmes de Factorisation D'Entiers*. PhD thesis, L'Université Paris-Sud, 1980.

[260] Louis Monier. Evaluation and comparison of two efficient probabilistic primality testing algorithms. *Theoretical Computer Science*, 12(1):97–108, 1980.

[261] Edward F. Moore. The shortest path through a maze. In *Proceedings of the International Symposium on the Theory of Switching*, pages 285–292. Harvard University Press, 1959.

[262] Rajeev Motwani, Joseph (Seffi) Naor, and Prabakhar Raghavan. Randomized approximation algorithms in combinatorial optimization. In Dorit Hochbaum, editor, *Approximation Algorithms for NP-Hard Problems*, chapter 11, pages 447–481. PWS Publishing Company, 1997.

[263] Rajeev Motwani and Prabhakar Raghavan. *Randomized Algorithms*. Cambridge University Press, 1995.

[264] J. I. Munro and V. Raman. Fast stable in-place sorting with $O(n)$ data moves. *Algorithmica*, 16(2):151–160, 1996.

[265] J. Nievergelt and E. M. Reingold. Binary search trees of bounded balance. *SIAM Journal on Computing*, 2(1):33–43, 1973.

[266] Ivan Niven and Herbert S. Zuckerman. *An Introduction to the Theory of Numbers*. John Wiley & Sons, fourth edition, 1980.

[267] Alan V. Oppenheim and Ronald W. Schafer, with John R. Buck. *Discrete-Time Signal Processing*. Prentice Hall, second edition, 1998.

[268] Alan V. Oppenheim and Alan S. Willsky, with S. Hamid Nawab. *Signals and Systems*. Prentice Hall, second edition, 1997.

[269] James B. Orlin. A polynomial time primal network simplex algorithm for minimum cost flows. *Mathematical Programming*, 78(1):109–129, 1997.

[270] Joseph O'Rourke. *Computational Geometry in C*. Cambridge University Press, second edition, 1998.

[271] Christos H. Papadimitriou. *Computational Complexity*. Addison-Wesley, 1994.

[272] Christos H. Papadimitriou and Kenneth Steiglitz. *Combinatorial Optimization: Algorithms and Complexity*. Prentice Hall, 1982.

[273] Michael S. Paterson. Progress in selection. In *Proceedings of the Fifth Scandinavian Workshop on Algorithm Theory*, pages 368–379, 1996.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

[274] Mihai Pătraşcu and Mikkel Thorup. Time-space trade-offs for predecessor search. In *Proceedings of the 38th Annual ACM Symposium on Theory of Computing*, pages 232–240, 2006.

[275] Mihai Pătraşcu and Mikkel Thorup. Randomization does not help searching predecessors. In *Proceedings of the 18th ACM-SIAM Symposium on Discrete Algorithms*, pages 555–564, 2007.

[276] Pavel A. Pevzner. *Computational Molecular Biology: An Algorithmic Approach*. The MIT Press, 2000.

[277] Steven Phillips and Jeffery Westbrook. Online load balancing and network flow. In *Proceedings of the 25th Annual ACM Symposium on Theory of Computing*, pages 402–411, 1993.

[278] J. M. Pollard. A Monte Carlo method for factorization. *BIT*, 15(3):331–334, 1975.

[279] J. M. Pollard. Factoring with cubic integers. In A. K. Lenstra and H. W. Lenstra, Jr., editors, *The Development of the Number Field Sieve*, volume 1554 of *Lecture Notes in Mathematics*, pages 4–10. Springer, 1993.

[280] Carl Pomerance. On the distribution of pseudoprimes. *Mathematics of Computation*, 37(156):587–593, 1981.

[281] Carl Pomerance, editor. *Proceedings of the AMS Symposia in Applied Mathematics: Computational Number Theory and Cryptography*. American Mathematical Society, 1990.

[282] William K. Pratt. *Digital Image Processing*. John Wiley & Sons, fourth edition, 2007.

[283] Franco P. Preparata and Michael Ian Shamos. *Computational Geometry: An Introduction*. Springer, 1985.

[284] William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes in C++: The Art of Scientific Computing*. Cambridge University Press, second edition, 2002.

[285] William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press, third edition, 2007.

[286] R. C. Prim. Shortest connection networks and some generalizations. *Bell System Technical Journal*, 36(6):1389–1401, 1957.

[287] William Pugh. Skip lists: A probabilistic alternative to balanced trees. *Communications of the ACM*, 33(6):668–676, 1990.

[288] Paul W. Purdom, Jr. and Cynthia A. Brown. *The Analysis of Algorithms*. Holt, Rinehart, and Winston, 1985.

[289] Michael O. Rabin. Probabilistic algorithms. In J. F. Traub, editor, *Algorithms and Complexity: New Directions and Recent Results*, pages 21–39. Academic Press, 1976.

[290] Michael O. Rabin. Probabilistic algorithm for testing primality. *Journal of Number Theory*, 12(1):128–138, 1980.

[291] P. Raghavan and C. D. Thompson. Randomized rounding: A technique for provably good algorithms and algorithmic proofs. *Combinatorica*, 7(4):365–374, 1987.

[292] Rajeev Raman. Recent results on the single-source shortest paths problem. *SIGACT News*, 28(2):81–87, 1997.

[293] James Reinders. *Intel Threading Building Blocks: Outfitting C++ for Multi-core Processor Parallelism*. O'Reilly Media, Inc., 2007.

[294] Edward M. Reingold, Jürg Nievergelt, and Narsingh Deo. *Combinatorial Algorithms: Theory and Practice*. Prentice Hall, 1977.

[295] Edward M. Reingold, Kenneth J. Urban, and David Gries. K-M-P string matching revisited. *Information Processing Letters*, 64(5):217–223, 1997.

[296] Hans Riesel. *Prime Numbers and Computer Methods for Factorization*, volume 126 of *Progress in Mathematics*. Birkhäuser, second edition, 1994.

[297] Ronald L. Rivest, Adi Shamir, and Leonard M. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978. See also U.S. Patent 4,405,829.

[298] Herbert Robbins. A remark on Stirling's formula. *American Mathematical Monthly*, 62(1):26–29, 1955.

[299] D. J. Rosenkrantz, R. E. Stearns, and P. M. Lewis. An analysis of several heuristics for the traveling salesman problem. *SIAM Journal on Computing*, 6(3):563–581, 1977.

[300] Salvador Roura. An improved master theorem for divide-and-conquer recurrences. In *Proceedings of Automata, Languages and Programming, 24th International Colloquium, ICALP'97*, volume 1256 of *Lecture Notes in Computer Science*, pages 449–459. Springer, 1997.

[301] Y. A. Rozanov. *Probability Theory: A Concise Course*. Dover, 1969.

[302] S. Sahni and T. Gonzalez. P-complete approximation problems. *Journal of the ACM*, 23(3):555–565, 1976.

[303] A. Schönhage, M. Paterson, and N. Pippenger. Finding the median. *Journal of Computer and System Sciences*, 13(2): 184–199, 1976.

[304] Alexander Schrijver. *Theory of Linear and Integer Programming*. John Wiley & Sons, 1986.

[305] Alexander Schrijver. Paths and flows—A historical survey. *CWI Quarterly*, 6(3):169–183, 1993.

[306] Robert Sedgewick. Implementing quicksort programs. *Communications of the ACM*, 21(10):847–857, 1978.

[307] Robert Sedgewick. *Algorithms*. Addison-Wesley, second edition, 1988.

[308] Robert Sedgewick and Philippe Flajolet. *An Introduction to the Analysis of Algorithms*. Addison-Wesley, 1996.

[309] Raimund Seidel. On the all-pairs-shortest-path problem in unweighted undirected graphs. *Journal of Computer and System Sciences*, 51(3):400–403, 1995.

[310] Raimund Seidel and C. R. Aragon. Randomized search trees. *Algorithmica*, 16(4–5):464–497, 1996.

[311] João Setubal and João Meidanis. *Introduction to Computational Molecular Biology*. PWS Publishing Company, 1997.

Acquistato da Michele Michela su Webster il 2022-07-07 23:12. Numero Ordine Libreria: 199503916:220707:0 Copyright © 2022 McGraw-Hill Education (Italy)

[312] Clifford A. Shaffer. *A Practical Introduction to Data Structures and Algorithm Analysis*. Prentice Hall, second edition, 2001.

[313] Jeffrey Shallit. Origins of the analysis of the Euclidean algorithm. *Historia Mathematica*, 21(4):401–419, 1994.

[314] Michael I. Shamos and Dan Hoey. Geometric intersection problems. In *Proceedings of the 17th Annual Symposium on Foundations of Computer Science*, pages 208–215, 1976.

[315] M. Sharir. A strong-connectivity algorithm and its applications in data flow analysis. *Computers and Mathematics with Applications*, 7(1):67–72, 1981.

[316] David B. Shmoys. Computing near-optimal solutions to combinatorial optimization problems. In William Cook, László Lovász, and Paul Seymour, editors, *Combinatorial Optimization*, volume 20 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*. American Mathematical Society, 1995.

[317] Avi Shoshan and Uri Zwick. All pairs shortest paths in undirected graphs with integer weights. In *Proceedings of the 40th Annual Symposium on Foundations of Computer Science*, pages 605–614, 1999.

[318] Michael Sipser. *Introduction to the Theory of Computation*. Thomson Course Technology, second edition, 2006.

[319] Steven S. Skiena. *The Algorithm Design Manual*. Springer, second edition, 1998.

[320] Daniel D. Sleator and Robert E. Tarjan. A data structure for dynamic trees. *Journal of Computer and System Sciences*, 26(3):362–391, 1983.

[321] Daniel D. Sleator and Robert E. Tarjan. Self-adjusting binary search trees. *Journal of the ACM*, 32(3):652–686, 1985.

[322] Joel Spencer. *Ten Lectures on the Probabilistic Method*, volume 64 of *CBMS-NSF Regional Conference Series in Applied Mathematics*. Society for Industrial and Applied Mathematics, 1993.

[323] Daniel A. Spielman and Shang-Hua Teng. Smoothed analysis of algorithms: Why the simplex algorithm usually takes polynomial time. *Journal of the ACM*, 51(3):385–463, 2004.

[324] Gilbert Strang. *Introduction to Applied Mathematics*. Wellesley-Cambridge Press, 1986.

[325] Gilbert Strang. *Linear Algebra and Its Applications*. Thomson Brooks/Cole, fourth edition, 2006.

[326] Volker Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 14(3):354–356, 1969.

[327] T. G. Szymanski. A special case of the maximal common subsequence problem. Technical Report TR-170, Computer Science Laboratory, Princeton University, 1975.

[328] Robert E. Tarjan. Depth first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972.

[329] Robert E. Tarjan. Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, 22(2):215–225, 1975.

[330] Robert E. Tarjan. A class of algorithms which require nonlinear time to maintain disjoint sets. *Journal of Computer and System Sciences*, 18(2):110–127, 1979.

[331] Robert E. Tarjan. *Data Structures and Network Algorithms*. Society for Industrial and Applied Mathematics, 1983.

[332] Robert E. Tarjan. Amortized computational complexity. *SIAM Journal on Algebraic and Discrete Methods*, 6(2):306–318, 1985.

[333] Robert E. Tarjan. Class notes: Disjoint set union. COS 423, Princeton University, 1999.

[334] Robert E. Tarjan and Jan van Leeuwen. Worst-case analysis of set union algorithms. *Journal of the ACM*, 31(2):245–281, 1984.

[335] George B. Thomas, Jr., Maurice D. Weir, Joel Hass, and Frank R. Giordano. *Thomas' Calculus*. Addison-Wesley, eleventh edition, 2005.

[336] Mikkel Thorup. Faster deterministic sorting and priority queues in linear space. In *Proceedings of the 9th ACM-SIAM Symposium on Discrete Algorithms*, pages 550–555, 1998.

[337] Mikkel Thorup. Undirected single-source shortest paths with positive integer weights in linear time. *Journal of the ACM*, 46(3):362–394, 1999.

[338] Mikkel Thorup. On RAM priority queues. *SIAM Journal on Computing*, 30(1):86–109, 2000.

[339] Richard Tolimieri, Myoung An, and Chao Lu. *Mathematics of Multidimensional Fourier Transform Algorithms*. Springer, second edition, 1997.

Acquistato da Michele Michele su Webster n.2022-07-07 23:12 Numero Ordine Libreria: 1995059016-2287074 © Copyright © 2022, McGraw-Hill Education (Italy)

[340] P. van Emde Boas. Preserving order in a forest in less than logarithmic time. In *Proceedings of the 16th Annual Symposium on Foundations of Computer Science*, pages 75–84, 1975.

[341] P. van Emde Boas. Preserving order in a forest in less than logarithmic time and linear space. *Information Processing Letters*, 6(3):80–82, 1977.

[342] P. van Emde Boas, R. Kaas, and E. Zijlstra. Design and implementation of an efficient priority queue. *Mathematical Systems Theory*, 10(1):99–127, 1976.

[343] Jan van Leeuwen, editor. *Handbook of Theoretical Computer Science, Volume A: Algorithms and Complexity*. Elsevier Science Publishers and the MIT Press, 1990.

[344] Charles Van Loan. *Computational Frameworks for the Fast Fourier Transform*. Society for Industrial and Applied Mathematics, 1992.

[345] Robert J. Vanderbei. *Linear Programming: Foundations and Extensions*. Kluwer Academic Publishers, 1996.

[346] Vijay V. Vazirani. *Approximation Algorithms*. Springer, 2001.

[347] Rakesh M. Verma. General techniques for analyzing recursive algorithms with applications. *SIAM Journal on Computing*, 26(2):568–581, 1997.

[348] Hao Wang and Bill Lin. Pipelined van Emde Boas tree: Algorithms, analysis, and applications. In *26th IEEE International Conference on Computer Communications*, pages 2471–2475, 2007.

[349] Antony F. Ware. Fast approximate Fourier transforms for irregularly spaced data. *SIAM Review*, 40(4):838–856, 1998.

[350] Stephen Warshall. A theorem on boolean matrices. *Journal of the ACM*, 9(1):11–12, 1962.

[351] Michael S. Waterman. *Introduction to Computational Biology, Maps, Sequences and Genomes*. Chapman & Hall, 1995.

[352] Mark Allen Weiss. *Data Structures and Problem Solving Using C++*. Addison-Wesley, second edition, 2000.

[353] Mark Allen Weiss. *Data Structures and Problem Solving Using Java*. Addison-Wesley, third edition, 2006.

[354] Mark Allen Weiss. *Data Structures and Algorithm Analysis in C++*. Addison-Wesley, third edition, 2007.

[355] Mark Allen Weiss. *Data Structures and Algorithm Analysis in Java*. Addison-Wesley, second edition, 2007.

[356] Hassler Whitney. On the abstract properties of linear dependence. *American Journal of Mathematics*, 57(3):509–533, 1935.

[357] Herbert S. Wilf. *Algorithms and Complexity*. A K Peters, second edition, 2002.

[358] J. W. J. Williams. Algorithm 232 (HEAPSORT). *Communications of the ACM*, 7(6):347–348, 1964.

[359] Shmuel Winograd. On the algebraic complexity of functions. In *Actes du Congrès International des Mathématiciens*, volume 3, pages 283–288, 1970.

[360] Andrew C. C. Yao. A lower bound to finding convex hulls. *Journal of the ACM*, 28(4):780–787, 1981.

[361] Chee Yap. A real elementary approach to the master recurrence and generalizations. Unpublished manuscript. Available at http://cs.nyu.edu/yap/papers/, July 2008.

[362] Yinyu Ye. *Interior Point Algorithms: Theory and Analysis*. John Wiley & Sons, 1997.

[363] Daniel Zwillinger, editor. *CRC Standard Mathematical Tables and Formulae*. Chapman & Hall/CRC Press, 31st edition, 2003.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Indice analitico

L'indice analitico usa le seguenti convenzioni. Quando una voce fa riferimento a una posizione diversa dal testo principale, il numero della pagina è seguito da un codice: *es.* per esercizio, *fig.* per figura, *n.* per nota e *pr.* per problema. Alcuni riferimenti spesso indicano la prima pagina di un esercizio, figura, nota o problema; questa non è necessariamente la pagina dove si trova effettivamente il riferimento.

$\alpha(n)$, 479
o, notazione, 43–44
O, notazione, 39 fig., 40–41
O', notazione, 53 pr.
$\widetilde{O}$, notazione, 53 pr.
ω, notazione, 44
Ω, notazione, 39 fig., 41–42
$\overset{\infty}{\Omega}$, notazione, 53 pr.
$\widetilde{\Omega}$, notazione, 53 pr.
Θ, notazione, 38–40, 39 fig.
$\widetilde{\Theta}$, notazione, 53 pr.
{ } (insieme), 962
$\in$ (appartenenza a un insieme), 962
$\notin$ (non appartenenza a un insieme), 962
$\emptyset$ (insieme vuoto), 962
$\subseteq$ (sottoinsieme), 962
$\subset$ (sottoinsieme proprio), 962
: (tale che), 963
$\cap$ (intersezione di insiemi), 963
$\cup$ (unione di insiemi), 963
| |
  (cardinalità di un insieme), 964
  (lunghezza di una stringa), 823
  (valore del flusso), 594
$\times$
  (prodotto cartesiano), 965
  (prodotto vettoriale), 846
$\langle\,\rangle$
  (codifica standard), 881
  (sequenza), 969
$\binom{n}{k}$ (coefficiente binomiale), 984
! (fattoriale), 49
$\lceil\,\rceil$ (ceiling), 46
$\lfloor\,\rfloor$ (floor), 46
$\sqrt[\downarrow]{\ }$ (radice quadrata inferiore), 455
$\sqrt[\uparrow]{\ }$ (radice quadrata superiore), 455
$\sum$ (sommatoria), 952
$\prod$ (produttoria), 955
$\rightarrow$ (relazione di adiacenza), 971
$\rightsquigarrow$ (relazione di raggiungibilità), 972
$\wedge$ (AND), 584, 891
$\neg$ (NOT), 891
$\vee$ (OR), 584, 891
$\oplus$ (operatore di gruppo), 784
$\otimes$ (operatore di convoluzione), 752
| (relazione "divide"), 774
$\nmid$ (relazione "non divide"), 774
$\equiv$ (congruo modulo n), 46, 968 es.
$\not\equiv$ (non congruo modulo n), 46
$[a]_n$ (classe di congruenza modulo n), 775
$+_n$ (addizione modulo n), 784
$\cdot_n$ (moltiplicazione modulo n), 784
$(\frac{a}{p})$ (simbolo di Legendre), 820 pr.
ε (stringa nulla), 823
$\sqsubset$ (relazione di prefisso), 823
$\sqsupset$ (relazione di suffisso), 823
$\succcurlyeq_x$ (relazione "sopra"), 852
// (simbolo del commento), 18
$\gg$ (relazione "molto maggiore di"), 479
$\ll$ (relazione "molto minore"), 654
$\leq_P$ (riducibilità in tempo polinomiale), 889, 897 es.
2-3, albero, 279

AA-albero, 279
abbinamento, 612–616
  flusso massimo, 612–616
  massimale, 925
  massimo, 946 pr.
  massimo nei grafi bipartiti, 625 es.
    algoritmo di Hopcroft-Karp, 639 pr.
  perfetto, 615 es.
abeliano, gruppo, 784
ABOVE, 853
accettabile, coppia di interi, 812
accoppiato, vertice, 613
Ackermann, funzione di, 488
addensamento
  primario, 225
  secondario, 226
addizione
  modulo n ($+_n$), 784
ADD-SUBARRAY, 672 pr.
adiacenza
  liste di, 493
  matrice di, 495
aggregato
  flusso, 721
Akra-Bazzi, metodo, 94
albero, 974–980
  2-3, 279, 421
  2-3-4, 408
    congiunzione, 420 pr.
    spartizione, 420 pr.
  AA, 279
  altezza, 977
  arco, 503, 505, 510
  attraversamento, 282
  AVL, 275 pr.
  B-albero, 404–421
  bilanciato in altezza, 275 pr.
  bilanciato in peso, 279, 395 pr.
  binario, 978
    pieno, 978
    rappresentazione, 203
    sovrapposto a un vettore di bit, 444
  bisezione, 981 pr.
  completo, 979
  del capro espiatorio, 279
  DF, 505

Acquistato da Michele Michele su WebSter il 2022-07-07 23:12 Numero Ordine Libreria: 1995035015220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

diametro di un, 504 es.
di cammini minimi, 542, 565
di decisione, 157–158
di fusione, 174, 402
di intervalli, 288–293
dinamico, 402
di ricerca bilanciato, 395 pr., 404–421
treap, 276 pr.
di ricerca esponenziale, 174, 403
di van Emde Boas, 443–467
di visita, 497, 502
heap, 127–140
k-ario completo, 127
k-neighbor, 279
libero, 973, 975–976
nullo, 978
ordinato, 977
per statistiche d'ordine, 280–285
interrogazione, 287 es.
query, 287 es.
posizionale, 978
radicato, 203–205, 976
rappresentazione, 203–205
radix tree, 251 pr.
ricorsione, 33, 74–79
scapegoat, 279
splay, 279, 402
treap, 276 pr.
vuoto, 978
albero binario di ricerca, 237–254
analisi probabilistica, 248–251
AVL, 275 pr.
cancellazione, 244–247
capro espiatorio, 279
chiave massima, 241
chiave minima, 241
con chiavi uguali, 251 pr.
confronto con min-heap, 239 es.
costruito in modo casuale, 248–251
inserimento, 243
interrogazione, 239–243
ottimo, 329–336, 343
per l'ordinamento, 247 es.
predecessore, 241–242
proprietà, 237
query, 239
radix tree, 251 pr.
ricerca, 240
successore, 241–242
treap, 276 pr.
albero binomiale, 440 pr.
altezza, 440 pr.
dimensione, 440 pr.
grado della radice di un, 440 pr.
albero di connessione, 365, 523
collo di bottiglia, 537 pr.
minimo, 523–538
algoritmo di Borůvka, 538
algoritmo di Kruskal, 528–529
algoritmo di Prim, 530–533
algoritmo generico, 524–528
algoritmo goloso, 523
grafi dinamici, 534 es.
secondo migliore, 535 pr.
verifica, 538
albero di van Emde Boas
cancellazione, 462–463
cluster, 455
confronto con le protostrutture, 456
inserimento, 461–462
massimo, 458
minimo, 458
predecessore, 460–461
spazio ridotto, 464 pr.
successore, 459–460
summary, 455
albero rosso-nero, 255–279
altezza, 257
aumentare, 286–287
bilanciato, 255–279
cancellazione, 267–273
chiave massima, 257
chiave minima, 257
colore dei nodi, 255
enumerare le chiavi, 288 es.
inserimento, 260–267
predecessore, 257
proprietà, 255–258
ricerca, 257
rilassato, 258 es.
rotazione, 258–260
successore, 257
unione, 275 pr.
albero RS-vEB, 464 pr.
albero vEB, *vedere* albero di van Emde Boas
alfabeto, 831, 881
algoritmi golosi, 344–375
codici di Huffman, 356–363
confronto con la programmazione dinamica, 347–348, 352
elementi di base, 351–356
in un matroide pesato, 366–369
matroide, 364–369
per il caching off-line, 374 pr.
programmazione dei lavori, 369–372, 372 pr., 374 pr.
proprietà, 352–353
resto in monete, 372 pr.
selezione di attività, 345–351
sottostruttura ottima, 353
algoritmo, 5
cache-oblivious, 421
come tecnologia, 12
con tempo polinomiale, 773
correttezza di un, 5
decrementale, 592
del MCD binario, 819 pr.
deterministico, 102, 658
di Dijkstra, 552–557
implementato con min-heap, 555
di Euclide, 779–783, 819 pr.
di Kruskal, 528–529
con pesi interi degli archi, 534 es.
di Prim, 530–533
per grafi sparsi, 535 pr.
di verifica, 886
multithread, 645–678
non deterministico, 658
origine della parola, 36
parallelo, 10, 645
partizionamento, 142–143
push-relabel, 616–636
push-relabel generico, 619–626
randomizzato, 97, 101–108
caso peggiore, 148 es.
hashing universale, 219–222
input medi, 24
partizionamento, 148, 154 pr.
per la selezione, 177–180
per quicksort, 154–155 pr.
primalità di Miller-Rabin, 808–814
rho di Pollard, 814–818, 819 es.
scalare, 570 pr.
seriale, 645
tempo di esecuzione, 22
algoritmo di approssimazione, 921–949
commesso viaggiatore, 925–930
copertura di insiemi, 931–935
copertura di vertici, 923–925
copertura pesata di insiemi, 945 pr.
cricca massima, 945 pr.
impacchettamento, 945 pr.
MAX-CUT, 939 es.
problema dello zaino, 948 pr., 949
randomizzato, 936
schedulazione di macchine, 946 pr.
soddisfattibilità MAX-3-CNF, 935
somma di sottoinsieme, 939–945
algoritmo goloso
algoritmo di Dijkstra, 552–557
pianificazione multithread, 653–655
albero di connessione minimo, 523
algoritmo ingenuo, 824–826
algoritmo multithread
per i numeri di Fibonacci, 647–651
per il calcolo dei prefissi, 673 pr.
per la riduzione di un array, 673 pr.
per quicksort, 677 pr.
per trasporre le matrici, 665 es.
randomizzato, 677 pr.
Alice, 800
allocare gli oggetti, 200–202
ALLOCATE-NODE, 411

Acquistato da Michele Michela su VitalBook il 2022-07-07 23:12 Numero licenza: 2735013016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

ALLOCATE-OBJECT, 202
altezza
  di un albero di decisione, 158
  di un albero rosso-nero, 257
  di un B-albero, 408–409
  di un heap, 128, 129 es.
  di un heap d-ario, 139 pr.
  di un nodo in un albero, 977
  di un nodo in un heap, 128, 133 es.
  esponenziale, 248
  nera, 257
ammissibile
  arco, 627
  programma lineare, 711
  regione, 707
  rete, 627–628
  soluzione, 558, 707, 711
  soluzione di base, 723
ammissibilità
  problema di, 558, 747 pr.
ammortizzata, analisi, 376–399
ammortizzato, costo
  metodo degli accantonamenti, 380
  metodo dell'aggregazione, 377
analisi aggregata
  algoritmo di Knuth-Morris-Pratt, 838
  per il taglio delle aste, 303
analisi ammortizzata, 376–399
  algoritmo di Dijkstra, 555
  algoritmo di Knuth-Morris-Pratt, 838
  analisi aggregata, 303
  metodo degli accantonamenti, 380
  metodo dell'aggregazione, 377
  metodo del potenziale, 382–386
  per alberi bilanciati in peso, 395 pr.
  per algoritmi push-relabel, 624–625
  per cammini minimi in un dag, 550
  per gli heap di Fibonacci, 425–428, 432, 434–436, 436 es.
  per insiemi disgiunti, 472–473, 474 es., 477 es., 480–485, 485 es.
  per la visita in ampiezza, 499–500
  per la visita in profondità, 507
  per le tavole dinamiche, 386–393
  per liste auto-organizzanti, 397 pr.
analisi competitiva, 397 pr.
analisi degli algoritmi, 20–25
analisi probabilistica, 97, 108–118
  albero binario di ricerca, 248–251
  algoritmi randomizzati, 102–103
  algoritmo di Rabin-Karp, 829
  bucket sort, 165–167, 168 es.
  confronto di file, 830 es.
  contatore probabilistico, 118 pr.
  della selezione randomizzata, 178
  delle collisioni, 215 es., 233 es.
  del partizionamento, 147 es., 154 pr.
  di quicksort, 149, 154 pr., 251 es.
  euristica del rho di Pollard, 816–818
  hashing con concatenamento, 213
  hashing perfetto, 231–233
  hashing universale, 219–222
  indirizzamento aperto, 227–229
  input medio, 24
  lancio delle palline, 111–112
  paradosso del compleanno, 108–110
  primalità di Miller-Rabin, 810–814
  problema delle assunzioni, 100–101
  ricerca in una lista compatta, 206 pr.
  serie dello stesso evento, 112–115
AND, funzione ($\wedge$), 584, 891
and, nello pseudocodice, 19
annidamento di scatole, 569 pr.
annotazione, 302, 321–322
antenato, 977
  minimo comune, 487 pr.
anti-collisione, funzione hash, 805
antiparallelo, arco, 595–596
antisimmetria, 967
ANY-SEGMENTS-INTERSECT, 854
approssimazione
  ai minimi quadrati, 697–700
  di Stirling, 49
APPROX-MIN-WEIGHT-VC, 938
APPROX-SUBSET-SUM, 942
APPROX-TSP-TOUR, 926
APPROX-VERTEX-COVER, 923
arbitraggio, 569 pr.
arco
  all'indietro, 510, 513
  ammissibile, 627
  antiparallelo, 595–596
  attraversamento di un taglio, 525
  attributi, 495
  capacità di un, 594
  classificazione, 511 es., 520 pr.
  critico, 610
  d'albero, 503, 505, 510
  di chiamata, 650
  di continuazione, 650
  di generazione, 650
  di peso negativo, 541–542
  di ritorno, 650
  inammissibile, 627
  in avanti, 510
  in un dag multithread, 650
  leggero, 525
  peso di un, 494
  ponte, 520 pr.
  residuo, 599
  saturo, 619
  sicuro, 524
  trasversale, 510
arcoconnettività, 612 es.
area
  pulita, 171 pr.
  sporca, 171 pr.
aritmetica modulare, 46, 784–789
armonico, numero, 954
array, 18
  k-ordinato, 170 pr.
  fusione, 26
  Monge, 92 pr.
  permutare un, 103–106
arrotondamento randomizzato, 949
articolazione, punto di, 520 pr.
asintotica, efficienza, 37
assegnazione multipla, 18
assenza di un cammino, 545, 564
assiomi di probabilità, 987
associatività, 784
assunzioni, problema delle, 95, 115–117
attraversamento
  albero, 282
  anticipato, 238
  completo di un albero, 928
  di un albero, 238, 243 es.
  posticipato, 238
  simmetrico, 238, 243 es., 282
attraversamento inorder, *vedere* attraversamento simmetrico
attributo di un oggetto, 18
aumentante, cammino, 602–603, 639 pr.
ausiliaria, funzione hash, 225
autenticazione, 235 pr.
automa a stati finiti, 830, 835 es.
autovalori, fattorizzazione agli, 703
AVL, albero, 275 pr.
AVL-INSERT, 275 pr.

BAD-SET-COVER-INSTANCE, 935 es.
BALANCE, 275 pr.
B-albero, 404–421
  altezza, 408–409
  cancellazione, 416–419
  chiave mediana, 411
  chiave minima, 416 es.
  creare, 411
  dividere un nodo in un, 412
  grado minimo, 408
  inserire una chiave, 411–415
  nodo pieno, 408
  proprietà, 407–410
  ricerca in un, 410–411
banale, divisore, 774
base, 324
  soluzione di, 723
B*-albero, 408
BELLMAN-FORD, 546
Bellman-Ford, algoritmo, 546–549
  funzione obiettivo, 562 es.

Acquistato da Michele Michelangeli - Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36 Numero Ordine Licenza: 1195853016-220707-0 Copyright © 2024 McGraw Hill Education (Italy)

migliorato da Yen, 568 pr.
nell'algoritmo di Johnson, 588
per cammini minimi fra coppie, 573, 588
per risolvere i sistemi dei vincoli sulle differenze, 560–561
BELOW, 853
Bernoulli, prove ripetute, 111–112, 997
BFS, 498
bianco, cammino, 509
BIASED-RANDOM, 98 es.
biconnessa, componente, 520 pr.
bilanciato, albero, 275 pr.
binari, somma di interi, 20 es.
binario, codice, 356
BINARY-SEARCH, 667
binomiale, coefficiente, 984
bipartito, grafo, 442
bisezione di un albero, 981 pr.
bit, operazioni sui, 774
bitonico
cammino, 336 pr.
problema del commesso viaggiatore, 336 pr.
tour, 336 pr.
BIT-REVERSE-COPY, 767
BIT-REVERSED-INCREMENT, 393 pr.
Bland, regola di, 733
bloccante, flusso, 640
Bob, 800
Boole, disuguaglianza, 991 es.
booleana
formula, 874, 888 es.
funzione, 985 es.
Borůvka, algoritmo, 538
BOTTOM-UP-CUT-ROD, 303
B^+-albero, 407
braccio della testina, 405
breadth-first search, *vedere* visita in ampiezza
B-TREE-CREATE, 411
B-TREE-DELETE, 417
B-TREE-INSERT, 413
B-TREE-INSERT-NONFULL, 414
B-TREE-SEARCH, 410, 416 es.
B-TREE-SPLIT-CHILD, 413
BUBBLESORT, 34 pr.
bucket, 165
bucket sort, 164–168
BUCKET-SORT, 165
BUILD-MAX-HEAP, 131
BUILD-MAX-HEAP′, 139 pr.
BUILD-MIN-HEAP, 133
butterfly, operazione, 765
by, nella pseudocodifica, 18
cache
fallimento della, 374 pr.
memoria, 21
successo della, 374 pr.
cache-oblivious, algoritmo, 421
caching off-line, 374 pr.
calcolo
combinatorio, 982–987
di un polinomio, 35 pr.
multithread, 649, 651
cammino, 972
aumentante, 602–603, 639 pr.
bianco, 509
bitonico, 336 pr.
compressione, 475
critico, 551, 651
di ricerca, 474
hamiltoniano, 888 es., 916 es.
minimo e massimo, 316, 873
peso di un, 539
proprietà del rilassamento, 564–565
semplice, 336 pr.
massimo, 316
cammino minimo, 316, 539–592
albero, 542–543, 565–567
algoritmo di Bellman-Ford, 546–549
algoritmo di Dijkstra, 552–557
algoritmo di Floyd-Warshall, 580
algoritmo di Gabow, 570 pr.
algoritmo di Johnson, 586–591
cicli di peso negativo, 541
come un programma lineare, 718
con archi di peso negativo, 541–542
con cammini bitonici, 572 pr.
da sorgente unica, 539–572
algoritmo di Dijkstra, 552–557
con cammini bitonici, 572 pr.
nei grafi ϵ-densi, 591 pr.
vincoli sulle differenze, 557–562
destinazione unica, 540
disuguaglianza triangolare, 545, 563
fra coppie di vertici, 540, 573–592
algoritmo di Johnson, 586–591
elevazione al quadrato, 578–579
nei grafi ϵ-densi, 591 pr.
prodotto di matrici, 575–580
in un grafo orientato aciclico, 549
in un grafo pesato, 539
nei grafi ϵ-densi, 591 pr.
per una coppia di vertici, 540
peso di un, 539
proprietà dell'assenza di un, 545, 564
proprietà della convergenza, 545, 564
proprietà del limite superiore, 545, 563
proprietà del rilassamento del, 545
rilassamento, 543–545
sottografo dei predecessori, 545, 567
sottostruttura ottima, 540–541
stima del, 543
varianti del problema, 540
vincoli sulle differenze, 557–562
visita in ampiezza, 500–502, 539
campionamento casuale, 107 es., 148
cancellazione
alberi di van Emde Boas, 462–463
B-albero, 416–419
da alberi binari di ricerca, 244–247
da alberi rosso-neri, 267–273
dagli stack, 192
dalle code, 193
da tavole dinamiche, 391
da tavole hash, 212, 224–225
da un albero di intervalli, 289
da un albero per statistiche d'ordine, 284
da una lista concatenata, 197
da un heap, 139 es.
di flusso, 600
di una chiave da un heap 2-3-4, 442 pr.
di un nodo da un heap di Fibonacci, 436, 439 pr.
lemma, 758
protostrutture di van Emde Boas, 454
vettore di bit, 446
canonica, forma
piano di programmazione, 370
capacità
di un arco, 594
di un taglio, 603
di un vertice, 598 es.
residua, 599, 602
vincolo sulla, 594
cappelli, problema dei, 101 es.
cappio, 971
carattere jolly, 826 es.
cardinalità di un insieme, 964
carico, fattore di, 386
Carmichael, numeri di, 808
cartesiana, somma, 757 es.
CASCADING-CUT, 433
cascata, taglio in, 434
caso
base, 55, 71
medio, 24, 96
migliore, 23, 25 es., 42
peggiore, 24
algoritmo randomizzato, 148 es.
tempo di esecuzione, 42
ricorsivo, 55

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

casuale
numero, 97
permutazione, 97
catalano, numero, 253 pr., 308
ceiling
funzione, 46
istruzione, 20
teorema dell'esperto, 87–89
cella, 210
certificato
in un sistema di crittografia, 805
per algoritmi di verifica, 886
CHAINED-HASH-DELETE, 212
CHAINED-HASH-INSERT, 212
CHAINED-HASH-SEARCH, 212
chiamata
per valore, 19
in una procedura multithread, 649
chiave, 14, 123, 135, 189
diminuire il valore di una, 433–435
dummy, 330
mediana, 411
pubblica, 800, 803
crittografia a, 800–805
segreta, 800, 803
statica, 230
chiusura
proprietà di un gruppo, 784
transitiva, 584, 591 pr., 695 es.
ciclicità, 731
ciclico, gruppo, 797
ciclo
di peso medio minimo, 571 pr.
di peso negativo
cammini minimi, 541
rilassamento, 568 es.
vincoli sulle differenze, 559
cammini minimi, 542
euleriano, 521 pr., 873
hamiltoniano, 873, 885
problema, 908–912
infinito, 731
parallelo, 655–657, 672 pr.
semplice più lungo, 917 es.
Cilk, 647, 678
Cilk++, 647, 678
circolare
lista concatenata, 195
lista con una sentinella, 197
circolazione a costo minimo, 748 pr.
circuito
combinatorio booleano, 892
trasformata rapida di Fourier, 767
classe
di complessità, 883
co-NP, 887
NP, 874, 886
NPC, 874, 890
P, 874, 879
di congruenza
modulo n ($[a]_n$), 775
di equivalenza, 967
classificazione degli archi
visita in ampiezza, 520 pr.
visita in profondità, 509, 511 es.
clique, *vedere* cricca
CLIQUE, 904
cluster
albero di van Emde Boas, 455
computer paralleli, 645
in un vettore di bit, 446
protostruttture di van Emde Boas, 449
clustering, *vedere* addensamento
CNF, forma, 874, 901
coda, 192–194
di priorità, 135–139
algoritmo di Dijkstra, 555
con albero di van Emde Boas, 443
estrazioni monotone, 140
implementazione, 135–139
limiti inferiori, 443
max-priorità, 135
min-priorità, 136, 138 es.
nell'algoritmo di Prim, 531, 533
protostrutture van Emde Boas, 448
di una distribuzione binomiale, 1002
di una lista concatenata, 195
doppia, 195 es.
fine di una, 193
implementata con lista, 198 es.
implementata con uno stack, 195 es.
inizio di una, 193
min-priorità, 359
algoritmo di Dijkstra, 555
nell'algoritmo di Prim, 531, 533
negli algoritmi push-relabel, 636 es.
nella visita in ampiezza, 498
ricorsione in, 349
codice, 356–357
binario di caratteri, 356
di caratteri, 356
Huffman, 356–363
a lunghezza fissa, 356
a lunghezza variabile, 357
prefisso, 357
codominio, 968
coefficienti
binomiali, 984
di un polinomio, 750
nella forma standard, 715
coerenza sequenziale, 678
cofattore, 1015
collinearità, 847
collisione, 212-215, 223–230
colonna
rango di, 1014
vettore, 1010
colorazione
di un grafo, 351 es., 918 pr.
di un nodo in un albero RB, 255
columnsort, 171 pr.
combina, passo, 55
combinatorio, circuito, 892
combinazione, 983
convessa di punti, 845
commento, nella pseudocodifica (//), 18
commesso viaggiatore
algoritmo di approssimazione, 925
con collo di bottiglia, 930 es.
problema bitonico, 336 pr.
problema del, 912–913
problema euclideo, 336 pr.
commutatività, 784
COMPACTIFY-LIST, 203 es.
COMPACT-LIST-SEARCH, 207 pr.
COMPACT-LIST-SEARCH′, 207 pr.
COMPARE-EXCHANGE, 171 pr.
compatibili
attività, 345
matrici, 307, 1012
compatta, lista, 206 pr.
compilatore, tabella dei simboli, 217
compleanno, paradosso del, 108–110
complementari, scarti, 747 pr.
complemento
di un grafo, 907
di Schur, 685, 696
complessità
co-NP, 887
classe di, 883
classe NP, 874, 886
classe NPC, 874, 890
classe P, 874, 879
completamento
della visita in profondità, 507
tempo di, 507
componenti
biconnesse, 520 pr.
connesse, 469–470, 512 es., 515–520
grafo delle, 516
composizione di calcoli multithread, 655 fig.
composto, numero, 775
compressione
del cammino, 475
delle immagini, 340 pr.
compromesso tempo-memoria, 301
COMPUTE-PREFIX-FUNCTION, 838
computer
parallelo, 645
parallelo ideale, 650

Acquistato da Michele Miraldi su Webster il 2022-07-07 23:12 Numero Ordine Emporia: 2983016-220707-0 Copyright © 2022, The McGraw-Hill Education (Italy)

COMPUTE-TRANSITION-FUNCTION, 835
concatenamento, 212–215, 234 pr.
  di stringhe, 823
concorrenza, parole chiave, 648
condizione al contorno, 57, 71
conflitto, 658
  di determinatezza, 658
  situazioni di, 658–660
confronta-scambia, operazione, 171 pr.
confronti, ordinamento per, 157
congiunzione, negli alberi 2-3-4, 420 pr.
congruenza, classe di, 775
congruità modulare ($\equiv$), 46
coniugata, matrice, 695 es.
CONNECTED-COMPONENTS, 470
connettivo booleano, 898
co-NP, 887
conservazione del flusso, 594
CONSOLIDATE, 429
contatore
  binario, 382, 384
    a bit invertiti, 393 pr.
  probabilistico, 118 pr.
contrazione
  di una tavola dinamica, 389–391
  di un matroide, 368
convergenza, proprietà, 545, 564
conversione da binari a decimali, 779 es.
convesso, insieme, 597 es.
convoluzione ($\otimes$), 752
  teorema della, 763
copertura
  di cammini, 637 pr.
  di insiemi
    pesata, 945 pr.
    problema, 931–935
  di vertici, 907, 923, 937–939
    algoritmo di approssimazione, 923
    ottima, 923
coppia
  di punti più lontani, 858
  di punti più vicini, 865–870
  di vertici, 540, 573–592
copy, istruzione, 20
corda, 285 es.
correttezza di un algoritmo, 5
cortocircuitato, operatore, 19
costo
  ammortizzato
    metodo dell'aggregazione, 377
    metodo del potenziale, 383
  minimo, flusso di, 719–720
counting sort, 159–162
  in radix sort, 163
COUNTING-SORT, 160
CREATE-NEW-RS-vEB-TREE, 465 pr.
credito
  metodo degli accantonamenti, 380
  metodo del, 376, 380
cricca, 904–907
  algoritmo di approssimazione, 945 pr.
critico
  arco, 610
  cammino, 551
crittografia, 800–805
crivello per campi numerici, 821
currency exchange, 323 es.
curva interpolante, 697–700
curve ellittiche, 821
CUT, 433
CUT-ROD, 300

dag, 512
  calcolo multithread, 649
  di calcolo, 649
DAG-SHORTEST-PATHS, 549
d-ario, heap
dati
  record, 123
  satelliti, 123, 189
deallocare gli oggetti, 200–202
debole, dualità, 736
decisione
  albero di, 157
  problema di, 875, 879
DECREASE-KEY, 136, 422
DECREMENT, 380 es.
degenerazione, 731
DELETE, 190
DELETE-LARGER-HALF, 386 es.
DeMorgan, leggi, 964
densità
  dei numeri primi, 806
  di un'asta, 306 es.
  funzione, 993
denso, grafo, 493
depth-first search, *vedere*
  visita in profondità
deque, 195 es.
DEQUEUE, 194
derivata di un polinomio, 769 pr.
destinazione
  flusso, 593
  vertice, 540
determinante, 1015
  e moltiplicazione di matrici, 694 es.
DETERMINISTIC-SEARCH, 119 pr.
deviazione standard, 996
DF
  albero, 505
  foresta, 505
DFS, 505
DFS-VISIT, 506
DFT, 8, 759–760
diagonale, matrice, 1010
diagramma PERT, 551, 551 es.
diametro, di un albero, 504 es.
differenza simmetrica, 639 pr.
differenze, vincoli sulle, 557–562
DIJKSTRA, 552
Dijkstra, algoritmo, 552–557
  cammini minimi fra coppie, 573, 589
  coda di priorità, 555
  con archi di peso intero, 557 es.
  con un heap di Fibonacci, 555
  nell'algoritmo di Johnson, 588
  somiglianza con l'algoritmo di Prim, 530, 555
  somiglianza con la visita in ampiezza, 555, 556 es.
dimensione
  dell'input di un algoritmo, 21, 773
  di un sottoalbero in un heap di Fibonacci, 438
dimezzamento, lemma, 758
dinamico
  albero, 402
  grafo, 470 n.
dipendenza
  lineare, 1014
  variabili casuali indicatrici, 99
DIRECT-ADDRESS-DELETE, 210
DIRECT-ADDRESS-INSERT, 210
DIRECT-ADDRESS-SEARCH, 210
DIRECTION, 848
discendente, 977
DISCHARGE, 628
disco
  asse di rotazione, 405
  testina, 405
Discrete Fourier Transform, *vedere*
  trasformata discreta di Fourier
DISK-READ, 406
DISK-WRITE, 406
distanza
  di cammino minimo, 500
  di editing, 337 pr.
  euclidea, 865
  L_m, 869 es.
  Manhattan, 184 pr., 869 es.
distribuzione
  binomiale, 111, 998–1001, 1001 es.
  degli input, 96, 102
  funzione di, 806
  geometrica, 111, 997–998
disuguaglianza
  lineare, 706
    ammissibilità, 747 pr.
  triangolare, 926
  vincolo di, 712

Acquistato da Michele Nicolai su VitalSource il 2022-07-07 23:12 Numero Ordine Libreria: 2955303216-22107-0 Copyright © 2022, McGraw-Hill Education (Italy)

divide, passo, 55
divide et impera
  albero di ricorsione, 74
  algoritmo di Strassen, 67–70
  analisi del metodo, 30
  merge sort, 26-33, 665–672
  metodo, 26–30, 55
  metodo per la selezione, 177–183
  moltiplicazione di matrici, 64–70
  nel problema del massimo sottoarray, 57–63
  per la moltiplicazione multithread di matrici, 661–665
  quicksort, 141–156
  ricerca binaria, 33 es.
  risoluzione delle ricorrenze, 94
divisione
  di prova, 806
  di un nodo di un B-albero, 412
  metodo della, 217–218
  teorema della, 775
divisore, 774
  comune, 775
dizionario, 189
DNA, 7, 324, 337 pr.
DNF (forma normale disgiuntiva), 902
dominio, 968
  del tempo, 750
doppio hashing, 226–227, 229 es.
dorsale, 276 pr.
  destra, 276 pr.
  sinistra, 276 pr.
downto, nella pseudocodifica, 18
d-regolare, grafo, 616 es.
duale, programma lineare, 735
dualità, 735–740
  debole, 736
2-CNF-SAT, 904 es.
dummy, chiave, 330
durata
  legge della, 652
  metrica di un calcolo multithread, 651

e, 47
E [ ] (valore atteso), 993
editing, distanza di, 337 pr.
Edmonds-Karp, algoritmo, 609–611
efficienza asintotica, 37
elevazione
  a potenza, 21
  modulare, 798
  al quadrato, 578, 798
eliminazione di Gauss, 684
ellissoide, algoritmo, 710
else, pseudocodice, 17
elseif, nella pseudocodifica, 17 n.
ENQUEUE, 194
entrante, variabile, 724
entropia, funzione, 985
ϵ-denso, grafo, 591 pr.
equazioni
  lineari
    modulari, 790–793
    sistemi di, 679–690
    sistemi tridiagonali di, 701 pr.
  normali, 699
  notazione asintotica, 42–43
equivalenti, programmi lineari, 712
equivalenza, classe di, 967
ereditaria, famiglia, 364
error, nella pseudocodifica, 19
esclusione e inclusione, 966 es.
espansione di una tavola dinamica, 387
esperto, teorema, 79
esponenziale
  albero di ricerca, 403
  altezza, 248
  funzione, 47–48
estensione di un insieme, 365
estremi
  di un intervallo, 288
  di un segmento, 846
EUCLID, 780
euclidea, norma, 1013
euleriano, ciclo, 873
Eulero
  costante di, 787
  funzione fi di, 787
  teorema, 796, 814 es.
euristica
  del gap, 636 es.
  dell'unione per rango, 475
  dell'unione pesata, 472
  della compressione del cammino, 475
evento, 987
  certo, 987
  complemento di un, 988
  indipendente, 990
  nullo, 987
EXACT-SUBSET-SUM, 940
EXTENDED-BOTTOM-UP-CUT-ROD, 305
EXTENDED-EUCLID, 782
EXTEND-SHORTEST-PATHS, 576
EXTRACT-MAX, 136–137
EXTRACT-MIN, 136, 422

famiglia
  ereditaria di sottoinsiemi, 364
  indipendente di sottoinsiemi, 364
fan-out, 892
Farkas, lemma, 748 pr.
FASTER-ALL-PAIRS-SHORTEST-PATHS, 579, 580 es.
Fast Fourier Transform, *vedere* trasformata rapida di Fourier
fattore di carico
  di una tavola dinamica, 386
  di una tavola hash, 213
fattore di ripetizione di una stringa, 844 pr.
fattore di rotazione, 762
fattoriale (!), 49
fattorizzazione, 814–819
  crivello per campi numerici, 821
  di numeri interi, 777
  LU, 684–686
  LUP, 681
    calcolo di una, 687–690
    di una matrice, 690 es.
    nella inversione delle matrici, 691
    prodotto di matrici, 694 es.
    uso della, 680–684
Fermat, teorema, 797
fermata, problema della, 873
FFT (Fast Fourier Transform), *vedere* trasformata rapida di Fourier
FFTW, 772
FIB, 647
FIB-HEAP-CHANGE-KEY, 441 pr.
FIB-HEAP-DECREASE-KEY, 433
FIB-HEAP-DELETE, 436
FIB-HEAP-EXTRACT-MIN, 428
FIB-HEAP-INSERT, 426
FIB-HEAP-LINK, 429
FIB-HEAP-PRUNE, 441 pr.
FIB-HEAP-UNION, 427
Fibonacci
  calcolo dei numeri di, 647–651
  grado massimo di un heap, 425–426
  heap, 533
    nell'algoritmo di Johnson, 590
  numeri di, 50–51, 91 pr., 437
FIFO (First-In, First-Out), 192
  vedere anche coda
FIND-DEPTH, 486 pr.
FIND-MAX-CROSSING-SUBARRAY, 60
FIND-MAXIMUM-SUBARRAY, 61
FIND-SET, 469
  con foresta di insiemi disgiunti, 476
  con insiemi disgiunti, 488
  con liste concatenate, 471
FINITE-AUTOMATON-MATCHER, 833
firma digitale, 802
First-In, First-Out, 192
  vedere anche coda
floating-point, 20
floor
  funzione, 46
  istruzione, 20
  nel teorema dell'esperto, 87–89
FLOYD-WARSHALL, 582
FLOYD-WARSHALL′, 586 es.
Floyd-Warshall, algoritmo, 580, 585 es.

Acquistato da Michele Michelangeli Vivaldi il 2022-07-07 23:12 Numero Ordine Editoriale 1958301012-2207077 - Copyright © 2022 McGraw-Hill Education (Italy)

flusso, 594–598
  a valori interi, 614
  aggregato, 721
  aumento di, 600
  bloccante, 640
  cancellazione di, 600
  conservazione del, 594
  destinazione, 593
  di costo minimo, 719–720
  di più merci, 720–721, 722 es.
  in eccesso, 616
  massimo, 593-641, 718
    abbinamento nei grafi bipartiti, 612
    aggiornamento, 638 pr.
    algoritmo di Edmonds-Karp, 609
    algoritmo progressivo per, 638 pr.
    algoritmo push-relabel, 616–636
    algoritmo relabel-to-front, 626
    con taglio minimo, 605
    metodo di Ford-Fulkerson, 598
  netto attraverso un taglio, 603
  pozzo, 593
  prodotto scalare, 597 es.
  sorgente, 593
  su rete, *vedere* rete di flusso
  valore di un, 594
  vincolo sulla capacità, 594
foglia, 977
for
  invariante di ciclo, 16 n.
  pseudocodice, 17–18
FORD-FULKERSON, 606
Ford-Fulkerson, metodo, 598–612
FORD-FULKERSON-METHOD, 599
foresta, 973, 975
  DF, 505
  di insiemi disgiunti, 474–477
    analisi, 480–485, 485 es.
    rango di un nodo, 479–480, 485 es.
forma
  canonica, 706, 710–714
  standard, 706, 714–716
    unicità, 732
  normale
    congiuntiva, 874
    disgiuntiva, 902
formula
  booleana, 874, 898, 904 es.
  soddisfattibile, 899
free-agent, 342 pr.
free list, 201
FREE-OBJECT, 202
frequenze, dominio delle, 750
fuga, problema, 636 pr.
funzione, 968–970
  argomento, 969
  asintoticamente più grande, 45
  asintoticamente più piccola, 45
  biiettiva, 970
  ceiling, 46
  convessa, 995
  di Ackermann, 488
  di base, 697
  di distribuzione di probabilità, 168 es.
  di partizione, 298 n.
  di transizione, 831, 835
  esponenziale, 47–48
  fattoriale (!), 49
  floor, 46
  generatrice, 91 pr.
  hash, 211, 216–223
    anticollisioni, 805
    ausiliaria, 225
    ϵ-universale, 222 es.
    metodo della divisione, 217
    metodo della moltiplicazione, 218
  hashing universale, 219–222
  iniettiva, 969
  inversa, 970
  iter, 481
  iterata, 53 pr.
  iterazione, 49
  level, 481
  lineare, 23, 706
  livello di una, 478
  logaritmica, 48–49
  logaritmo iterato, 50
  monotonicamente crescente, 46
  monotonicamente decrescente, 46
  obiettivo, 557, 562 es., 707, 711
  peso per un grafo, 494
  polilogaritmicamente limitata, 48
  polinomialmente limitata, 47
  potenziale, 383
    limite inferiore, 399
  prefisso, 836–837
    iterazione, 839
  quadratica, 24
  strettamente crescente, 46
  strettamente decrescente, 46
  suffisso, 831
    disuguaglianza, 833
    ricorsione, 833
  suriettiva, 969
fusione
  albero di, 402
  di due array, 26
  di liste ordinate, 139 es.
  limite inferiore, 170 pr.
fuzzy, ordinamento, 156 pr.

Gabow, algoritmo, 570 pr.
gap, euristica, 636 es.
garbage collector, 127, 201

Gauss, processo di eliminazione di, 684
generatore
  di numeri, 97
  di un sottogruppo, 788
  di $\mathbb{Z}_n^*$, 797
GENERIC-MST, 524
GENERIC-PUSH-RELABEL, 620
geometria computazionale, 845–872
gestione della memoria, 200, 216 es.
Goldberg, algoritmo, *vedere*
  algoritmo push-relabel
golosi, algoritmi, 344–375
gossiping, 399
grado
  della radice di un albero binomiale, 440 pr.
  di ramificazione, 407
  di un nodo, 977
  di un polinomio, 750
  di un vertice, 971
  entrante, 971
  limite di un polinomio, 750
  massimo di un heap di Fibonacci, 425–426, 436–439
  minimo di un B-albero, 408
  uscente, 971
grafo, 970–974
  algoritmi, 491–641
  attributi, 495–496
  bipartito, 973
  cammino minimo, 500
  colorazione, 980 pr.
  complemento di un, 907
  completo, 973
  connesso, 972
  contrazione, 974
  dei sottoproblemi, 304
  dei vincoli, 559–560
  delle componenti, 516
  denso, 493
  di intervalli, 351 es.
  dinamico, 470 n.
    struttura dati per, 403
  d-regolare, 616 es.

Acquistato da Michele Mirelle su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright ©2022, McGraw-Hill Education (Italy)

ϵ-denso, 591 pr.
hamiltoniano, 885
isomorfo, 972
matrice di incidenza di un, 373 pr., 496 es.
matroidi, 373 pr.
non orientato
abbinamento, 613
albero di connessione minimo, 523
componente biconnessa, 520 pr.
conversione di un multigrafo in un, 496 es.
griglia, 636 pr.
ponte, 520 pr.
punto di articolazione, 520 pr.
orientato, 971
aciclico, 336 pr., 512–516
cammini minimi da sorgente unica, 539–572
cammini minimi fra coppie di vertici, 573–592
chiusura transitiva, 584
copertura di cammini, 637 pr.
dei vincoli, 559
diagramma PERT, 551, 551 es.
quadrato di un, 496 es.
semiconnesso, 520 es.
singolarmente connesso, 512 es.
trasposizione, 496 es.
pesato, 494
rappresentazione con liste di adiacenza, 493
rappresentazione con matrice di adiacenza, 495
singolarmente connesso, 512 es.
sparso, 493
statico, 470 n.
visita in ampiezza di un, 497–504
visita in profondità di un, 504–512
GRAFT, 486 pr.
Graham, algoritmo, 858–863
GRAHAM-SCAN, 859
GRAPH-ISOMORPHISM, 887 es.
greedoide, 375
GREEDY, 367
GREEDY-ACTIVITY-SELECTOR, 350
GREEDY-SET-COVER, 932
griglia, 636 pr.
gruppo, 784–789
abeliano, 784
additivo modulo n, 785
ciclico, 797
finito, 784
moltiplicativo modulo n, 785
HAM-CYCLE, 885
hamiltoniano
ciclo, 873, 885
grafo, 885
HAM-PATH, 888 es.
handle, 136
hash
ϵ-universale, 222 es.
valore, 211
HASH-DELETE, 229 es.
hashing, 209–236, 235 pr.
concatenamento, 212–215, 234 pr.
doppio, 226–227, 229 es.
indirizzamento aperto, 223–230
perfetto, 230–234
uniforme, 225
uniforme semplice, 214
universale, 219–222
HASH-INSERT, 224, 229 es.
HASH-SEARCH, 224, 229 es.
head
di una coda, 193
di una lista concatenata, 195
heap, 127–140
2-3-4, 442 pr.
algoritmo di Huffman, 359
altezza di un, 128
aumentare una chiave, 137
binomiale, 440 pr.
proprietà, 440 pr.
tempi di esecuzione, 423 fig.
cancellazione da un, 139 es.
chiave massima, 136
coda di priorità, 135–139
confronto con albero binario, 239 es.
confronto con heap di Fibonacci, 423
conservare la proprietà, 129–131
costruire, 131–133, 139 pr.
d-ario, 139 pr., 591 pr.
di Fibonacci, 422–442
cambiare una chiave, 441 pr.
cancellare un nodo, 436, 439 pr.
confronto con heap binari, 423
consolidare una lista di radici, 428
creare un, 426
estrarre il nodo minimo, 428–432
funzione potenziale, 425
grado massimo, 425–426, 436–439
inserimento di un nodo, 426–427
lista dei figli, 424
nell'algoritmo di Dijkstra, 555
nodo minimo, 427
tagliare un nodo, 434
unione, 427–428
estrarre la chiave massima, 136–137
garbage collector, 127, 201
implementare un heap riunibile, 422
inserimento in un, 137–138
max-heap, 128
metodo del potenziale, 385 es.
min-heap, 128
nell'algoritmo di Dijkstra, 555
nell'algoritmo di Johnson, 590
nell'algoritmo di Prim, 533
proprietà, 128
rilassato, 442
riunibile, 206 pr., 401, 422
binomiale, 440 pr.
heap 2-3-4, 442 pr.
treap, 276 pr.
unione, 423
HEAP-DECREASE-KEY, 138 es.
HEAP-DELETE, 139 es.
HEAP-EXTRACT-MAX, 137
HEAP-EXTRACT-MIN, 138 es.
HEAP-INCREASE-KEY, 137
HEAP-MAXIMUM, 136
HEAP-MINIMUM, 138 es.
heapsort, 127–140
HEAPSORT, 134
hermitiana, matrice, 695 es.
high, funzione, 448, 455
HIRE-ASSISTANT, 95
hit della cache, 374 pr.
HOARE-PARTITION, 153 pr.
HOPCROFT-KARP, 639 pr.
Hopcroft-Karp, algoritmo, 639 pr.
Horner, regola, 35 pr., 752
algoritmo di Rabin-Karp, 827
HUFFMAN, 359
Huffman, codici di, 356–363

identità, 784
matrice, 1010
if, pseudocodice, 17
immagine, 969
di una matrice, 1018 pr.
immagini, compressione, 340 pr., 343
impacchettamento, 945 pr.
impera, passo, 55
inammissibile
arco, 627
programma lineare, 711
soluzione, 711
incidenza, matrice di, 496 es., 559
inclusione ed esclusione, 966 es.
INCREASE-KEY, 136
INCREMENT, 378

Acquistato da Michele Michele su Weblibri il 2022-07-07 23:12 Numero Ordine Libreria 2018/0012 Copyright © 2022, McGraw-Hill Education (Italy)

incrementale, metodo, 25
indeterminato, sistema di equazioni, 680
index, funzione, 448, 455
indicatrice, variabile casuale, 98–101
indice di un elemento di $\mathbb{Z}_n^*$, 797
indipendenza
  dei sottoproblemi, 317–318
  lineare, 1014
indirizzamento
  aperto, 223–230
  diretto, 209–211, 444–447
infinite, grandezze, 545
ingenuo, algoritmo, 824–826
INITIALIZE-PREFLOW, 620
INITIALIZE-SIMPLEX, 727, 741
INITIALIZE-SINGLE-SOURCE, 543
innalzamento, di un vertice, 617, 619
inorder, attraversamento, *vedere* attraversamento simmetrico
INORDER-TREE-WALK, 238
input
  dimensione, 21
  distribuzione degli, 96, 102
  di un algoritmo, 5
inserimento, 135–136, 386 es.
  alberi di van Emde Boas, 461–462
  di una chiave in un heap 2-3-4, 442 pr.
  di un nodo in un heap di Fibonacci, 426–427
  in tavole hash, 223–224
  in tavole hash concatenate, 212
  in un albero di intervalli, 289
  in un albero per statistiche d'ordine, 283–284
  in un heap, 137–138
  in un vettore di bit, 446
  in una coda, 193
  in una lista concatenata, 196
  in una tavola dinamica, 387
  in uno stack, 192
  negli alberi binari di ricerca, 243
  negli alberi rosso-neri, 260–267
  protostrutture di van Emde Boas, 453–454
INSERT, 190
insertion sort, 11, 14–17, 22–24
  albero di decisione, 158 fig.
  in bucket sort, 165–167
  confronto con merge sort, 13 es.
  confronto con quicksort, 147 es.
  merge sort, 34 pr.
  quicksort, 152 es.
  ricerca binaria, 33 es.
INSERTION-SORT, 15, 22, 171 pr.
insiemi ({ }), 962–966
  associatività, 963
  assorbimento, 963
  cardinalità, 964
  commutatività, 963
  complemento, 964
  convessi, 597 es.
  dimensione, 964
  dinamici, 189–190
    vedere anche struttura dati
  disgiunti, 468–488, 964
    nell'algoritmo di Kruskal, 529
  distributività, 963
  estensione, 365
  finiti, 965
  idempotenza, 963
  indipendenti, 370, 917 pr.
  leggi di DeMorgan, 964
  numerabili, 965
  partizione, 964
  universo, 964
  vuoti (∅), 962
integer, 20
integralità, teorema, 614
intera, programmazione lineare, 748 pr.
intermedio, vertice, 580
interpolazione, 697–700
  con una spline cubica, 702 pr.
  polinomiale, 753, 756 es., 762
interrogazione, *vedere* query
intersezione
  di corde, 285 es.
  di due segmenti di retta, 851–857
  di insiemi (∩), 963
INTERVAL-DELETE, 288
INTERVAL-INSERT, 288
intervallo, 288
  albero, 288–293
  aperto, 288
  chiuso, 288
  estremi di un, 288
  ordinamento fuzzy, 156 pr.
  punto di massima sovrapposizione, 293 pr.
  semiaperto, 288
  sovrapposizione, 288, 292 es.
  tricotomia, 288
INTERVAL-SEARCH, 288–289
INTERVAL-SEARCH-EXACTLY, 292 es.
intorno, 615 es.
intrattabile, problema, 873
invariante di ciclo, 15
  algoritmo del simplesso, 729
  algoritmo di Dijkstra, 553
  algoritmo di Rabin-Karp, 829
  aumentare una chiave, 138 es.
  automa di string matching, 832, 834
  conclusione, 16
  fusione, 27
  heapsort, 134 es.
  inizializzazione, 16
  inserimento in un albero rosso-nero, 263
  mantenimento, 16
  origine, 36
  partizionamento, 142
  per consolidare la lista delle radici, 431
  per costruire un heap, 131
  per insertion sort, 15–17
  per l'algoritmo di Prim, 532–533
  per l'algoritmo relabel-to-front, 632
  per le potenze modulari, 799
  per un algoritmo push-relabel, 622
  permutazione casuale di array, 105
  regola di Horner, 35 pr.
  ricerca in un albero di intervalli, 291
  visita in ampiezza, 498
inversa, matrice, 691–694, 1013
inversione
  di bit, 393 pr., 766
  di matrice, 691–694
  in una lista auto-organizzante, 397 pr.
  in una sequenza, 35 pr., 101 es.
inverso
  moltiplicativo, modulo n, 792
  in un gruppo, 784
invio
  non saturante, 619, 624
  saturante, 619
involucro
  convesso, 7, 857–865
    catena, 864
    on-line, 865 es.
  sparso, 871 pr.
iperarco, 973
ipergrafo, 973
isomorfismo fra grafi, problema, 916 es.
ispezione, 223, 234 pr.
  lineare, 225
  quadratica, 225–226, 235 pr.
  sequenza di, 224
istanza di un problema, 5, 876
istruzioni
  aritmetiche, 20
  controllo, 20
iter, funzione, 481
ITERATIVE-FFT, 766
ITERATIVE-TREE-SEARCH, 240
iterazione di funzione, 49
Jarvis, algoritmo, 863–864
Jensen, disuguaglianza, 995
JOHNSON, 590
Johnson, algoritmo, 586–591
jolly, carattere, 826 es., 835 es.
Josephus, permutazione di, 293 pr.

Acquistato da Michele Gliozzi. su Webster il 2022-07-07 23:12 Numero ordine Libreria 1193930515-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Karmarkar, algoritmo, 749
Karp, algoritmo, 571 pr.
k-CNF, 874
k-colorazione, 980 pr.
k-combinazione, 983
k-esima potenza, 778 es.
Kleene, stella di, 882
KMP, algoritmo, 836–843
KMP-MATCHER, 837
k-neighbor, albero, 279
Knuth-Morris-Pratt, algoritmo, 836–843
k-ordinato, array, 170 pr.
k-permutazione, 983
Kraft, disuguaglianza, 980 es.
Kruskal, algoritmo, 529
k-sottoinsieme, 965
k-sottostringa, 983
k-stringa, 983

Lagrange
  formula, 753
  teorema, 788
Lamé, teorema, 781
lancio
  della moneta, 98
  delle palline, 111
lasco di parallelismo, 652
Last-In, First-Out, 192
  vedere anche stack
lavoro
  in anticipo, 370
  di durata unitaria, 369
  legge del, 652
  metrica, 651
  programmazione, 374 pr.
  in ritardo, 370
LCA, 488 pr.
LCS (Longest Common Subsequence), 324, 343
LCS-LENGTH, 327
LEFT, 127
LEFT-ROTATE, 259, 292 es.
Legendre, simbolo, 820 pr.
lessicografico, ordinamento, 251 pr.
letterale, 901
level, funzione, 481
lg (logaritmo binario), 48
$\lg^*$ (funzione logaritmo iterato), 50
$\lg^k$ (potenza di un logaritmo), 48
lg lg (composizione di logaritmi), 48
liberare gli oggetti, 200–202
LIFO (Last-In, First-Out), 192
  vedere anche stack
limite
  asintoticamente stretto, 38
  asintotico inferiore, 41
  asintotico superiore, 40
  inferiore, 56
  funzione potenziale, 399
  fusione di liste ordinate, 170 pr.
  ordinamento, 157, 170 pr., 443
  per le code di priorità, 443
  per serie dello stesso evento, 118 es.
  per trovare il predecessore, 467
  per trovare la mediana, 186
  per un calcolo multithread, 651
  polilogaritmico, 48
  superiore, 545, 563
lineare
  dipendenza, 1014
  disuguaglianza, 706
  funzione, 706
  indipendenza, 1014
  uguaglianza, 706
  vincolo, 706
linearità del valore atteso, 994
linguaggio, 881, 886
  NP-completezza, 897–898
  operazioni, 882
  vuoto, 882
LINK, 476
lista
  auto-organizzante, 397 pr.
  cancellazione da una, 197
  compatta, 202 es., 206 pr.
  concatenata, 195–199
    circolare, 195
    dei vicini, 628
    non ordinata, 195
    ordinata, 195
    per insiemi disgiunti, 471–474
    ricerca, 222 es.
  dei figli e delle radici
    in un heap di Fibonacci, 424
  doppiamente concatenata, 195
  inserimento in una, 196
  ricerca in una, 196
  singolarmente concatenata, 195
  unione, 199 es.
LIST-DELETE, 197
LIST-DELETE′, 197
LIST-INSERT, 196
LIST-INSERT′, 198
LIST-SEARCH, 196
LIST-SEARCH′, 198
livello
  di una funzione, 478
L_m-distanza, 869 es.
ln (logaritmo naturale), 48
load, istruzione, 20
logaritmo, 48–49
  discreto, 797
LONGEST-PATH, 884 es.
LONGEST-PATH-LENGTH, 884 es.
LOOKUP-CHAIN, 321
lottatori professionisti, 504 es.
low, funzione, 448, 455
LU-DECOMPOSITION, 686
lunghezza
  del percorso, 252 pr.
  di una dorsale, 276 pr.
  di una stringa, 823
LUP-DECOMPOSITION, 688
LUP-SOLVE, 683

MAKE-HEAP, 422
MAKE-SET, 468
  con foresta di insiemi disgiunti, 475
  con liste concatenate, 471
makespan, 946 pr.
MAKE-TREE, 486 pr.
Manhattan, distanza, 184 pr., 869 es.
Markov, disequazione, 996 es.
massimizzazione, programma lineare, 706, 712
massimo, 136, 175
  in alberi binari di ricerca, 241
  in alberi di van Emde Boas, 458
  in alberi per statistiche d'ordine, 287 es.
  in alberi rosso-neri, 257
  in un heap, 136
  in un vettore di bit, 446
  protostrutture di van Emde Boas, 454 es.
  rango, 1014
  sottoarray, 57–63, 93
  trovare, 175–177
massimo comun divisore (MCD), 776
  algoritmo del MCD binario, 819 pr.
  algoritmo di Euclide, 779–783
  con più di due argomenti, 783 es.
  teorema delle ricorsione, 779
matching
  di stringhe, 822–844
  pesato su grafo bipartito, 442
matrice
  booleana, moltiplicazione, 695 es.
  compatibile, 307, 1012
  definita positiva, 695–697, 1015
  dei predecessori, 574
  di adiacenza, 495
  diagonale, 1010
    fattorizzazione LUP di una, 690 es.
  di incidenza, 373 pr.
    di un grafo, 373 pr., 496 es.
  di permutazione, 1011, 1013 es., 1016 es.
  di Vandermonde, 1017 pr.
  hermitiana, 695 es.
  identità, 1010
  inversa, 673 pr., 691–694, 1016 es.
  invertibile, 1014
  minore di una, 1015

Acquistato da Michele Michele Sud Vesuvio | 2022-07-04 23:12 Numero Ordine Licenza: 1-1580010142287-0 Copyright © 2022, McGraw-Hill Education (Italy)

moltiplicazione, 64–70, 306–313
multiplo di una, 1012
negativo di una, 1012
non singolare, 1014
preimmagine, 1018 pr.
prodotto parentesizzato, 306
pseudoinversa, 699
quadrata, 1010
rango, 1016 es.
simmetrica, 1011, 1013 es., 1016 es.
sottrazione, 1012
Toeplitz, 769 pr.
trasposizione di una, 661 es.
trasposta, 665 es., 1009
trasposta coniugata, 695 es.
triangolare, 1011, 1013 es., 1016 es.
triangolare inferiore, 1016 es.
triangolare inferiore unitaria, 1011
triangolare superiore, 1011, 1016 es.
triangolare superiore unitaria, 1011
tridiagonale, 1010
zero, 1010
MATRIX-CHAIN-MULTIPLY
MATRIX-CHAIN-ORDER, 311
MATRIX-MULTIPLY, 307
matroide, 364–369, 373 pr.
grafico, 364, 538
matriciale, 364
pesato, 365–369
proprietà della scelta golosa, 367
sottoinsieme ottimo di un, 366
MAT-VEC, 656
MAT-VEC-MAIN-LOOP, 656
MAT-VEC-WRONG, 659
MAX-CUT, problema, 939 es.
MAX-FLOW-BY-SCALING, 638 pr.
max-heap, 128
aumentare una chiave, 137
cancellazione da un, 139 es.
chiave massima, 136
coda di max-priorità, 135–139
conservare la proprietà, 129–131
costruire, 131–133
estrarre la chiave massima, 136–137
heapsort, 134
inserimento in un, 137–138
proprietà, 128
riunibile, 401 n.
MAX-HEAPIFY, 130
MAX-HEAP-INSERT, 137
costruire un heap con, 139 pr.
max-heap riunibile, 206 n., 422 n.
MAXIMUM, 190
MAYBE-MST-A, 537 pr.
MAYBE-MST-B, 537 pr.
MAYBE-MST-C, 537 pr.
mcm (minimo comune multiplo), 783 es.
mediana, 175–186
di 3, metodo, 155 pr.
di liste ordinate, 183 es., 671 es.
inferiore, 175
ponderata, 184 pr.
superiore, 175
memoization, *vedere* annotazione
MEMOIZED-CUT-ROD, 302
MEMOIZED-CUT-ROD-AUX, 302
MEMOIZED-MATRIX-CHAIN, 321
memoria
cache, 21
condivisa, 645, 650
distribuita, 645
gestione, 127, 200, 202 es., 216 es.
locazione, 200
spazzini della, 127, 201
struttura gerarchica, 21
virtuale, 21
merci, flusso di più, 720
merge, 139 es., 170 pr.
MERGE, 27
mergeable heap, *vedere* heap riunibile
MERGE-LISTS, 940
merge sort, 11, 26–33
confronto con insertion sort, 13 es.
insertion sort, 34 pr.
multithread, 665–672
MERGE-SORT, 29
albero di ricorsione, 323 es.
MERGE-SORT′, 665
metodo
a doppio passaggio, 476
a singolo passaggio, 488
bottom-up, 302
degli aggregati
per la visita in ampiezza, 499–500
per la visita in profondità, 507
degli accantonamenti, 380–382
del credito, 376
del potenziale, 382–386
per contatori binari, 384–385
per gli heap di Fibonacci, 425–428, 432, 434–436
per un algoritmo push-relabel, 624
per i min-heap, 385 es.
per insiemi disgiunti, 480–485
per le operazioni su stack, 383–384
per le tavole dinamiche, 388, 391
per liste auto-organizzanti, 397 pr.
dell'esperto, 79–82
della divisione, 217–218
della moltiplicazione, 218–219
di aggregazione, 377–380
algoritmo di Dijkstra, 555
contatore binario, 378–379
operazioni su stack, 377–378
per gli heap di Fibonacci, 436 es.
per i cammini minimi, 550
per insiemi disgiunti, 472–473, 474 es., 477 es.
di sostituzione, 70–74
divide et impera
moltiplicazione di polinomi, 768 pr.
per la selezione, 177–183
programmazione dinamica, 297
trasformata rapida di Fourier, 760
mediana-di-3, 155 pr.
top-down, 302
MILLER-RABIN, 810
Miller-Rabin, test di primalità, 808–814
MIN-GAP, 292 es.
min-heap, 128
algoritmo di Huffman, 359
coda di min-priorità, 138 es.
confronto con albero binario, 239 es.
conservare la proprietà, 131 es.
costruire, 131–133
metodo del potenziale, 385 es.
nell'algoritmo di Dijkstra, 555
nell'algoritmo di Johnson, 590
nell'algoritmo di Prim, 533
ordinati, 424
proprietà, 128, 424
riunibile, 206 n., 401 n., 422
MIN-HEAPIFY, 131 es.
MIN-HEAP-INSERT, 138 es.
minimi quadrati, approssimazione, 697
minimizzazione, 706, 712
minimo, 136, 175
comune antenato, 487 pr.
in alberi binari di ricerca, 241
in alberi di van Emde Boas, 458
in alberi per statistiche d'ordine, 287 es.
in alberi rosso-neri, 257
in un vettore di bit, 446
nei B-alberi, 416 es.
nelle protostrutture di van Emde Boas, 451
off-line, 486 pr.
trovare, 175–177
minimo comune multiplo (mcm), 783 es.
MINIMUM, 176, 190
minimum spanning tree, *vedere* albero di connessione minimo
minore, di una matrice, 1015
min-priorità
coda, 136, 138 es.
miss della cache, 374 pr.
misura di complessità, 883
mod, 46, 775
modello RAM
istruzioni, 20

Acquistato da Michele Amoretti su Vapostel il 2022-07-07 23:12 Numero Ordine Ebraria: 3249503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

modulare, aritmetica, 46
MODULAR-EXPONENTIATION, 799
MODULAR-LINEAR-EQUATION-SOLVER, 792
modulo, 46, 775
moltiplicazione
  di matrici, 63–70, 673 pr., 1012, 1016 es.
    algoritmo di Strassen, 67–70
    booleane, 695 es.
    calcolo del determinante, 694 es.
    fattorizzazione LUP, 694 es.
    inversione, 691–694
    metodo di Pan, 70 es.
    metodo divide et impera, 64–70
  di numeri complessi, 70 es.
  di polinomi, 751
    metodo divide et impera, 768 pr.
  di una sequenza di matrici, 306–313
  matrice-vettore, 655, 659, 661 es.
  metodo della, 218–219
  modulo n ($\cdot_n$), 784
  multithread di matrici, 661–665
molto maggiore di ($\gg$), 479
molto minore ($\ll$), 654
moneta, lancio della, 98, 988
Monge, array di, 92 pr.
move-to-front, euristica, 397 pr., 399
MST-KRUSKAL, 529
MST-PRIM, 531
MST-REDUCE, 536 pr.
multigrafo, 973
  conversione in un grafo, 496 es.
multinsieme, 962 n.
multiplo, 774
  di un elemento modulo n, 790–793
  minimo comune, 783 es.
  scalare, 1012
MULTIPOP, 377
MULTIPUSH, 380 es.
multithread
  calcolo, 649
  composizione, 655 fig.
  per merge sort, 665–672
multithreading
  algoritmo di Strassen, 664
  concorrenza, 646
  dinamico, 646
  parole chiave nello pseudocodice, 646

$\mathbb{N}$ (numeri naturali), 962
naive algorithm, *vedere*
  algoritmo ingenuo
NAIVE-STRING-MATCHER, 824
negativo, di una matrice, 1012
nera, altezza, 257
NEXT-TO-TOP, 858
NIL, 19
n-insieme, 965
nodo
  altezza di un, 128
  colore, 255
  di una spline, 702 pr.
  esterno, 977
  interno, 977
  marcato, 424, 434
  minimo, 424, 427, 428–432
  pieno, 408
non invertibile, matrice, 1014
non-istanza, 880 n.
non negatività, vincolo, 711
non saturante, invio, 619
norma
  di un vettore, 1013
  euclidea, 1013
normale, equazione, 699
NOT, funzione ($\neg$), 891
notazione
  O grande, 39 fig., 40–41
  o piccolo, 43–44
  asintotica, 37–46, 53 pr.
    algoritmi per grafi, 491
  Omega grande, 39 fig., 41–42
  omega piccolo, 44
NP (classe di complessità), 874, 886, 888 es.
NPC (classe di complessità), 874, 890
NP-completezza, 873–920
  ciclo hamiltoniano, 908–912
  ciclo semplice più lungo, 917 es.
  commesso viaggiatore, 912–913
  copertura di vertici, 907–908
  cricca, 904–907
  isomorfismo fra grafi, 916 es.
  partizionamento di insieme, 916 es.
  programmazione di lavori con premi e scadenze, 919 pr.
  programmazione lineare, 916 es.
  somma di sottoinsieme, 913–916
NP-completo, 890
NP-difficile, 890
nullo, vettore, 1014
numeri
  catalani, 253 pr., 308
  complessi, moltiplicazione di, 70 es.
  composti, 775
  di Carmichael, 814 es.
  di Fibonacci, 819 pr.
  interi ($\mathbb{Z}$), 962
  naturali ($\mathbb{N}$), 962
  primi, 774
    densità, 806
    funzione di distribuzione dei, 806
    teorema dei, 806
  reali ($\mathbb{R}$), 962
numerica, stabilità, 679–680, 703

O', notazione, 53 pr.
$\widetilde{O}$, notazione, 53 pr.
obiettivo
  funzione, 557, 562 es., 707
  valore, 707, 711
occorrenza di un pattern, 822
off-line
  caching, 374 pr.
  minimo, 486 pr.
OFF-LINE-MINIMUM, 486 pr.
oggetto, 18
  allocare e liberare, 200–202
  implementato con array, 199–203
  passato come parametro, 19
ombra di un punto, 865 es.
Omega grande, notazione, 39 fig., 41–42
ON-LINE-MAXIMUM, 116
ON-SEGMENT, 848
OpenMP, 647
operatore cortocircuitato, 19
OPTIMAL-BST, 334
OR, funzione ($\vee$), 584, 891
or, nello pseudocodice, 19
ordinamento, 14–17, 26–33, 123
  algoritmi confronta-scambia, 171 pr.
  bubblesort, 34 pr.
  bucket sort, 164–168
  columnsort, 171 pr.
  counting sort, 159–162
  di elementi di lunghezza variabile, 169 pr.
  fuzzy, 156 pr.
  heapsort, 127–140
  insertion sort, 11, 14–17
  lessicografico, 251 pr.
  limite inferiore, 157–159, 168 pr., 443
  mediante un albero binario, 247 es.
  merge sort, 11, 26–33, 665–672
  ordinamento 0-1, 171 pr.
  per colonne principali, 171 pr.
  per confronti, 157, 168 pr.
  per confronti e alberi binari, 239 es.
  per confronti e selezione, 182
  per selezione, 25 es.
  problema, 5, 14, 123
  quicksort, 141–156
  radix sort, 162–164
  radix tree, 251 pr.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Cliente: 3053016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

Shell's sort, 36
stabilità degli algoritmi, 161
sul posto, 15, 124
tabella dei tempi di esecuzione, 125
in tempo lineare, 159–168, 169 pr.
topologico, 7, 512–515
ordine
di un gruppo, 789
lineare, 968
parziale, 967
topologico, 549
totale, 968
origine, 846
ortonormale, 703
OS-KEY-RANK, 285 es.
OS-RANK, 282
OS-SELECT, 281
ottima
soluzione, 711
sottostruttura, 314–318
ottimizzazione, 875
problema di, 297, 879
ottimo
albero binario di ricerca, 331, 343
valore obiettivo, 711
output di un algoritmo, 5
overflow, 192
di una coda, 194
P (classe), 874, 879
padre nell'albero BF, 497
pagina di un disco, 405, 419 pr.
palindroma, sottosequenza, 336 pr.
palline, lancio delle, 111–112, 1008 pr.
Pan, metodo, 70 es.
paradosso del compleanno, 110, 118 es.
parallelismo
annidato, 649, 672 pr.
di un algoritmo multithread, 677 pr.
di un calcolo multithread, 652
logico, 649
parametro, 19
costi per passare un, 90 pr.
PARENT, 127
parentesi
struttura di, 507
teorema delle, 507
parentesizzazione del prodotto di matrici, 306
parole chiave, nella pseudocodifica, 19, 648
parsing, albero di, 901
parte intera
inferiore (floor), 20
superiore (ceiling), 20
PARTITION, 142
partizionamento, 141
algoritmo randomizzato, 148
analisi probabilistica del, 147 es.
attorno alla mediana, 153 es.
di un insieme, 916 es.
Pascal, triangolo di, 986 es.
passo
completo, 653
incompleto, 653
PATH, 876, 882
pattern
matching, 822–844
nello string matching, 822
non sovrapponibile, 835 es.
penalità, 370
percorso, lunghezza del, 252 pr.
perfetto, hashing, 230–234
permutazione, 970, 983
casuale, 103–106
Josephus, 293 pr.
lineare, 1019 pr.
matrice di, 1011, 1013 es.
per inversione dei bit, 393 pr.
per inversione di bit, 766
sul posto, 105
PERMUTE-BY-CYCLIC, 107 es.
PERMUTE-BY-SORTING, 103
PERMUTE-WITH-ALL, 107 es.
PERMUTE-WITHOUT-IDENTITY, 106 es.
persistente, struttura dati, 274 pr., 402
PERSISTENT-TREE-INSERT, 274 pr.
PERT, diagramma, 551, 551 es.
peso
di un arco, 494
di un cammino, 539
di un cammino minimo, 539
funzione, 494
medio di un ciclo, 571 pr.
negativo, 541–542
P-FIB, 648
pianificazione multithread, 653–655
piano di programmazione, 369
forma canonica di un, 370
piattaforme concorrenti, 646
piatto di un disco, 405
pila, *vedere* stack
PISANO-DELETE, 439 pr.
pivot
nella fattorizzazione LU, 685
nella programmazione lineare, 726, 734 es.
quicksort, 142
PIVOT, 726
pivoting, 685
nella programmazione lineare, 724
P-MATRIX-MULTIPLY-RECURSIVE, 663
P-MERGE, 668
P-MERGE-SORT, 670
polare, angolo, 850 es.
poligono, 850 es.
a forma di stella, 865 es.
nucleo di un, 865 es.
polinomi, 750
derivate dei, 769 pr.
grado, 750
grado limite, 750
interpolazione, 753, 756 es.
moltiplicazione, 751, 755, 768 pr.
rappresentazione per coefficienti, 752
rappresentazione per punti, 752
somma, 750
valutazione, 752, 756 es., 770 pr.
polinomiale, condizione di crescita, 94
polinomio
calcolo, 35 pr.
coefficienti, 47
comportamento asintotico, 52 pr.
grado, 47
Pollard, euristica del rho di, 814, 819 es.
POLLARD-RHO, 815
ponderata, mediana, 184 pr.
ponte, 520 pr.
pop da uno stack, 155 pr.
POP, 193
porta logica, 891
postorder, attraversamento, *vedere* attraversamento posticipato
potenza
di un elemento, modulo n, 796–800
elevazione al quadrato ripetuta, 798
insieme, 965
k-esima, 778 es.
modulare, 798
non banale, 778 es.
potenziale di una struttura dati, 383
pozzo, 496 es., 594, 596
flusso, 593
Pr { } (distribuzione di probabilità), 987
PREDECESSOR, 190
predecessore
alberi BF, 497
alberi binari di ricerca, 241–242
alberi di cammini minimi, 542
alberi di van Emde Boas, 460–461
alberi per statistiche d'ordine, 287 es.
alberi rosso-neri, 257
B-alberi, 416 es.
liste concatenate, 195
protostrutture di van Emde Boas, 454 es.
vettori di bit, 446
predecessori
matrice dei, 574
sottografo dei, 574
prefisso
calcolo, 673 pr.
codice, 357

Acquistato da Pietro Grimaldi su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2021, McGraw-Hill Education (Italy)

di una sequenza, 325
di una stringa (⊏), 823
funzione, 836–837
parallelo, 673 pr.
preflusso, 616
preimmagine di una matrice, 1018 pr.
preorder, attraversamento, *vedere* attraversamento anticipato
preordine totale, 968
Prim, algoritmo
con pesi interi degli archi, 534 es.
con una matrice di adiacenza, 534 es.
implementato con un heap di Fibonacci, 533
implementato con un min-heap, 533
per grafi sparsi, 535 pr.
somiglianza con l'algoritmo di Dijkstra, 530, 555
primale, programma lineare, 735
primalità
test di, 805–814, 821
test di Miller-Rabin, 808–814
primaria, memoria, 404
primi, numeri, 774, 777
principale, memoria, 404
PRINT-ALL-PAIRS-SHORTEST-PATH, 574
PRINT-CUT-ROD-SOLUTION, 305
PRINT-INTERSECTING-SEGMENTS, 856 es.
PRINT-LCS, 327
PRINT-OPTIMAL-PARENS, 312, 313 es.
PRINT-PATH, 503
PRINT-SET, 477 es.
probabilistica, analisi, 96–97, 108–118
probabilistico, contatore, 118 pr.
probabilità, 987–992
condizionata, 989, 991
distribuzione, 987, 989
funzione densità, 993
probing, *vedere* ispezione
problema
astratto, 878
computazionale, 5
concreto, 879
del collezionista di figurine, 112
della colorazione di un grafo, 351 es.
della posizione dell'ufficio postale, 184 pr.
delle assunzioni
analisi probabilistica, 100
online, 115–117
variabile casuale indicatrice, 100
dello zaino, 354, 355–356 es., 948 pr.
dello zaino frazionario, 354
di ottimizzazione, 297
algoritmi di approssimazione, 921
intrattabile, 873
soluzione di un, 5
trattabile, 873
procedura, 6, 14
prodotto
cartesiano, 965
di matrici, 1012, 1016 es.
algoritmo di Strassen, 93–94
per cammini minimi, 575
di polinomi, 751
esterno, 1013
interno, 1013
regola del, 982
scalare del flusso, 597 es.
vettoriale (×), 846
profondità
dell'albero di ricorsione, 147 es.
di un nodo in un albero radicato, 977
di uno stack, 155 pr.
media di un nodo, 252 pr.
problema di determinare la, 486 pr.
programma lineare
ammissibile, 711
ausiliario, 740
di massimizzazione, 706
di minimizzazione, 706
duale, 735
illimitato, 711
primale, 735
programmazione
dei lavori, 369–372, 374 pr.
delle scorte, 341 pr.
intera 0-1, 916 es.
programmazione dinamica, 297–343
alberi binari di ricerca ottimi, 329
algoritmo di Viterbi, 339 pr.
annotazione, 321–322
confronto con gli algoritmi golosi, 316, 323 es., 347–348, 352
distanza di editing, 337 pr.
elementi della, 313–323
metodo bottom-up, 302
metodo top-down, 302
moltiplicazione di matrici, 306–313
per arruolare giocatori free-agent, 342 pr.
per cammini minimi fra coppie, 575
per la chiusura transitiva, 584–586
per la programmazione delle scorte, 341 pr.
per la sottosequenza palindroma più lunga, 336 pr.
per l'algoritmo di Floyd-Warshall, 580–584
problema del commesso viaggiatore, 336 pr.
problema dello zaino, 355 es.
ricostruzione di una soluzione ottima, 320–321
selezione di attività, 350 es.
sottoproblemi ripetuti, 318–320
sottosequenza comune più lunga, 324–329
sottostruttura ottima, 314–318
stampa accurata, 336 pr.
taglio delle aste, 298–306
taglio di una stringa, 340 pr.
programmazione lineare, 704–749
algoritmi, 710
algoritmo del simplesso, 722–734
algoritmo di Karmarkar, 749
applicazioni, 709–710
cammini minimi, 557–562, 718
circolazione a costo minimo, 748 pr.
dualità, 735–740
flusso di costo minimo, 719–720
flusso di più merci, 720–721, 722 es.
flusso massimo, 718–719
forma canonica, 710–714
forma standard, 714–716
intera, 710, 748 pr., 916 es.
metodo dei punti interni, 710, 749
rilassamento, 937
teorema fondamentale della, 745
trovare una soluzione iniziale, 740
protostrutture di van Emde Boas, 448
cancellazione, 454
cluster, 449
e alberi di van Emde Boas, 456
inserimento, 453–454
massimo, 454 es.
minimo, 451–452
predecessore, 454 es.
ricerca, 451
successore, 452–453
summary, 449
protostrutture vEB, *vedere* protostrutture di van Emde Boas
PROTO-VEB-INSERT, 454
PROTO-VEB-MEMBER, 451
PROTO-VEB-MINIMUM, 452
PROTO-VEB-SUCCESSOR, 453
prune-and-search, metodo, 858
P-SCAN-1, 674 pr.
P-SCAN-2, 674 pr.
P-SCAN-3, 675 pr.
P-SCAN-DOWN, 675 pr.
P-SCAN-UP, 675 pr.
pseudocasuali, numeri, 97
pseudocodice, 14
costrutti di ciclo, 17
pseudocodifica, 17–19
pseudoinversa, 699
pseudoprimalità, test di, 807–808
PSEUDOPRIME, 807
pseudoprimo, 807–808
in base a, 807

Acquistato da Michele Michele su VitalBook il 2021-07-07 23:12 Numero Ordine Libreria: 9788891220707-0 Copyright © 2021, McGraw-Hill Education (Italy)

P-SQUARE-MATRIX-MULTIPLY, 662
P-TRANSPOSE, 661 es.
puntatore, 18, 200
  implementato con array, 199–203
punti interni, metodo dei, 710
punto di articolazione di un grafo, 520 pr.
punto più vicino, euristica, 930 es.
push
  in uno stack, 155 pr.
  operazione, 618–619
PUSH
  operazione push-relabel, 618
  stack operation, 193
push-relabel, algoritmo, 616–636
  abbinamento massimo, 625 es.
  con coda di vertici traboccanti, 636 es.
  euristica del gap, 636 es.
  generico, 619–626
  operazioni di base, 617–619
  scarica di un vertice traboccante di altezza massima, 636 es.

quadratico, residuo, 820 pr.
quantile, 183 es.
query, 190, 239
  albero binario di ricerca, 239–243
quicksort, 141–156
  analisi, 144–153
  analisi del caso medio, 149–152
  analisi del caso peggiore, 149
  con elementi di valore uguale, 153 pr.
  con ricorsione in coda, 155 pr.
  confronto con insertion sort, 147 es.
  confronto con radix sort, 164
  descrizione, 141–144
  metodo della mediana-di-3, 155 pr.
  profondità dello stack per, 155 pr.
  usare insertion sort in, 152 es.
  versione randomizzata, 148, 154 pr.
QUICKSORT, 141
quoziente, 775

$\mathbb{R}$ (numeri reali), 962
Rabin-Karp, algoritmo, 826–830
RABIN-KARP-MATCHER, 828
RACE-EXAMPLE, 658
radice
  di $\mathbb{Z}_n^*$, 797
  grado della, 440 pr.
  primitiva di $\mathbb{Z}_n^*$, 797
  principale dell'unità, 757
  quadrata, 798, 820 pr.
  quadrata inferiore ($\downarrow\sqrt{\ }$), 455
  quadrata superiore ($\uparrow\sqrt{\ }$), 455
radici complesse dell'unità, 757
  interpolazione, 762–763
radix sort, 162–164
  confronto con quicksort, 164
RADIX-SORT, 163
radix tree, 251 pr.
raggiungibilità in un grafo ($\rightsquigarrow$), 972
ramificazione, 404
  grado di, 407
RANDOM, 97, 98 es.
random-access machine, 20–21
RANDOMIZED-HIRE-ASSISTANT, 103
RANDOMIZED-PARTITION, 148
RANDOMIZED-QUICKSORT, 148, 251 es.
RANDOMIZED-SELECT, 177
RANDOMIZE-IN-PLACE, 105
randomizzato, algoritmo, 97, 101–108
RANDOM-SAMPLE, 108 es.
RANDOM-SEARCH, 119 pr.
rango
  di colonna, 1014
  di riga, 1014
  di una matrice, 1014, 1016 es.
  di un nodo in una foresta di insiemi disgiunti, 475, 479–480, 485 es.
  di un numero in un insieme ordinato, 280
  in albero per statistiche d'ordine, 282, 285 es.
  massimo, 1014
rapporto
  aureo (ϕ), 50, 91 pr.
  di approssimazione, 921, 935
rappresentante di un insieme, 468
rappresentazione
  figlio-sinistro fratello-destro, 203
  per coefficienti dei polinomi, 752
  per punti dei polinomi, 752
RB-DELETE, 268
RB-DELETE-FIXUP, 270
RB-ENUMERATE, 288 es.
RB-INSERT, 261
RB-INSERT-FIXUP, 261
RB-JOIN, 275 pr.
RB-TRANSPLANT, 267
record, 123
RECURSIVE-ACTIVITY-SELECTOR, 348
RECURSIVE-FFT, 761
RECURSIVE-MATRIX-CHAIN, 319
REDUCE, 673 pr.
regione ammissibile, 707
RELABEL, 619
relabel, operazione, 619, 623
RELABEL-TO-FRONT, 632
relabel-to-front, algoritmo, 626–636
RELAX, 544
relazione, 966–968
  di equivalenza, 967
  "divide" ($|$), 774
  "non divide" ($\nmid$), 774
  "sopra" ($\succcurlyeq_x$), 852
  totale, 968
repeat, istruzione, 17
REPETITION-MATCHER, 844 pr.
RESET, 382 es.
residua
  capacità, 599, 602
  rete, 599–602
residuo, 46, 775, 820 pr.
  arco, 599
resto, 46, 775
  monete, 372 pr.
rete
  ammissibile, 627–628
  di flusso, 594–598
    associata a un grafo bipartito, 613
    taglio, 603–606
  residua, 599–602
retta di sweeping, 851
return, istruzione, 19–20
rho, euristica del, 814–818, 819 es.
ricerca
  albero binario ottimo, 329–336
  binaria, 33 es., 667
    con inserimento rapido, 394 pr.
    nei B-alberi, 416 es.
  di un intervallo esatto, 292 es.
  lineare, 19 es.
  negli alberi binari, 240
  negli alberi di Van Emde Boas, 458
  negli alberi rosso-neri, 257
  nei B-alberi, 410–411
  nelle protostrutture di van Emde Boas, 451
  problema di, 19 es.
  in tavole hash, 224
  in tavole hash concatenate, 212
  in un albero di intervalli, 289–292
  in una lista compatta, 206 pr.
  in una lista concatenata, 196
  in un array non ordinato, 119 pr.
ricorrenza, 30, 55–57, 70–94
  albero di ricorsione, 74–79
  metodo dell'esperto, 79–82
  metodo di Akra-Bazzi, 94
  metodo di sostituzione, 70–74

Acquistato da Michele Miraele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 1995303016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

ricorsione, 26
albero, 33, 74–79
per merge sort, 323 es.
in coda, 155 pr.
metodo di sostituzione, 77–78
riducibilità, 889–890
in tempo polinomiale ($\leq_P$), 889
riduzione
algoritmo di, 876, 889
di un array, 673 pr.
funzione di, 889
riga
rango di, 1014
vettore, 1010
RIGHT, 127
RIGHT-ROTATE, 259
rilascio, tempo di, 372 pr.
rilassamento
del cammino, 545, 564–565
di un arco, 543–545
programmazione lineare, 937
ripetizione di una stringa, 844 pr.
risoluzione di sistemi di equazioni lineari, 679–690
rispetto di un insieme di archi, 525
ritorno, arco di, 650
rotazione
ciclica, 843 es.
destra, 259
negli alberi rosso-neri, 258–260
sinistra, 259
RSA, crittografia a chiave pubblica, 800–805

sabermetrica, statistica, 342 pr.
SAME-COMPONENT, 470
SAT, 898
satelliti, dati, 123, 189
saturante, invio, 619, 624
saturo, arco, 619
scacchi, programma per il gioco, 660
scadenza, 370
scalare, algoritmo, 570 pr.
scambio, proprietà di, 364
scan, 673 pr.
SCAN, 674 pr.
scapegoat, albero, 279
scaricare un vertice traboccante, 628
scarto, variabile, 714
scatole, annidamento di, 569 pr.
scelta golosa, 346–348
schedulazione, 946 pr.
scheduler
goloso, 653
nel calcolo multithread, 649, 653–655, 678
scheduling, *vedere* piano di programmazione
schema di approssimazione, 922
Schur, complemento di, 696
scomposizione
in fattori primi, 777
LU, 673 pr.
LUP, 673 pr.
scoperta
nella visita in profondità, 507
tempo di, 507
scoperto, vertice, 497, 505
SCRAMBLE-SEARCH, 119 pr.
seam carving, 340 pr., 343
SEARCH, 190
segmento
di retta, 846
orientato, 847
SEGMENTS-INTERSECT, 848
SELECT, 180–181
selection sort, 25 es.
selettore, vertice, 909
selezione
di attività, 345–351
nel tempo lineare atteso, 177–180
ordinamenti per confronti, 182
problema della, 175
in un albero per statistiche d'ordine, 281–282
semianello chiuso, 592
semiconnesso, grafo, 520 es.
sentinella, 27, 197–198, 255
sequenza ($\langle\,\rangle$)
bitonica, 572 pr.
e cammini minimi, 572 pr.
finita, 969
infinita, 969
inversione in una, 35 pr., 101 es.
monotona, 140
serializzazione di un algoritmo multithread, 646, 648
serie, 91 pr., 953–954
aritmetica, 953
armonica, 954
convergente, 953
dello stesso evento, 112–115
differenziazione, 954
di Taylor, 253 pr.
divergente, 953
geometrica, 953
integrazione, 954
potenze, 91 pr.
riduzione, 954
Shell's sort, 36
shift, *vedere* spostamento
shift, istruzione, 21
SHORTEST-PATH, 875
sicuro, arco, 524
simbolo di Legendre ($\frac{a}{p}$), 820 pr.
simmetria
della notazione Θ, 44
trasposta, 44
simmetrica, differenza, 639 pr.
simmetrico, attraversamento, 282
simplesso, 708
algoritmo del, 708, 722–734
SIMPLEX, 728
singolare, matrice, 1014
singoletto, 965
sistemi di equazioni lineari, 673 pr., 679–690, 701 pr.
skip list, 279
slack, 714
slot, *vedere* cella
SLOW-ALL-PAIRS-SHORTEST-PATHS, 578
⋆Socrates, 660
soddisfattibilità, 874, 892, 898, 935
dei circuiti, 891–897
formule 2-CNF, 904 es.
formule 3-CNF, 900–903
MAX-CNF, 939 es.
a metà di formule 3-CNF, 917 es.
soluzione
ammissibile, 558, 707, 711
di base, 723
di un problema computazionale, 5
di un sistema di equazioni lineari, 679
inammissibile, 711
ottima, 711, 320–321
somma
cartesiana, 757 es.
di interi binari, 20 es.
di matrici, 1011
di polinomi, 750
di sottoinsieme, 939–945
problema della, 913–916, 916 es.
lemma, 759
regola della, 982
sommatoria, 952–961
formule e proprietà, 952–955
infinita, 952
limite di una, 955–961
linearità, 953
nella notazione asintotica, 43
riduzione, 954
scomposizione, 958–959
sorgente, 497, 540, 594, 596
flusso, 593
sorgenti e pozzi multipli, 596
sostituzione
alberi di ricorsione, 77–78

Acquistato da Michele Mercaldo - Webster - 2022-07-07 23:12 Numero Ordine Integrale: 2095-00016-220707-0 Copyright © 2022, The MIT Press, Cambridge (Italy)

all'indietro, 682
in avanti, 681–682
metodo di, 56
sottoalbero, 283–284, 977
sottoespressione comune, 764
sottografo, 973
dei predecessori, 502, 505, 542, 574
proprietà, 545, 567
sottogruppo, 787–789
proprio, 788
sottoinsieme ($\subseteq$), 962
famiglia ereditaria, 364
famiglia indipendente di, 364
massimale in un matroide, 365
ottimo di un matroide, 366
proprio ($\subset$), 962
sottomatrice principale, 695
sottoproblemi
grafo dei, 304
ripetuti, 318–320
sottosequenza, 324
comune, 7, 324
comune più lunga, 324–329, 343
sottostringa, 983
sottostruttura ottima
dei cammini minimi, 540, 575, 580
dei codici di Huffman, 362
del taglio delle aste, 299
della più lunga sottosequenza comune, 325
della selezione di attività, 345–346
di alberi binari di ricerca, 331–332
di cammini minimi non pesati, 316
di matroidi pesati, 368
moltiplicazione di matrici, 308–309
negli algoritmi golosi, 353
problema dello zaino, 354
programmazione dinamica, 314–318
sottrazione, di matrici, 1012
sovrapposizione
dei suffissi, 823
di intervalli, 288, 292 es.
di rettangoli, 293 es.
spanning tree, *vedere*
albero di connessione
sparso, grafo, 493
spartizione negli alberi 2-3-4, 420 pr.
spawn, nello pseudocodice, 648–649
spazzini della memoria, 127, 201
speedup
di un algoritmo multithread, 677 pr.
di un calcolo multithread, 652
lineare, 652
lineare perfetto, 652
splay, albero, 279, 402
spline, 702 pr.
cubica, 702 pr.
nodo, 702 pr.
spostamento, 822
SQUARE-MATRIX-MULTIPLY, 64, 577
SQUARE-MATRIX-MULTIPLY-RECURSIVE, 65
stabilità
algoritmi di ordinamento, 161, 164 es.
numerica, 679–680
stack, 192–193
fondo, 192
implementato con code, 195 es.
implementato con liste, 198 es.
metodo degli accantonamenti, 381
metodo del potenziale, 383–384
nella memoria secondaria, 419 pr.
per eseguire procedure, 155 pr.
vuoto, 192
STACK-EMPTY, 193
stampino, 676 pr.
statico
grafo, 470 n.
insieme di chiavi, 230
stati finiti, automa a, 830
statistica
d'ordine, 175–186
dinamica, 280–285
sabermetrica, 342 pr.
stencil, 676 pr.
Stirling, approssimazione di, 49
store, istruzione, 20
strand, 649
finale, 650
indipendente, 659
iniziale, 650
logicamente in parallelo, 650
logicamente in serie, 650
Strassen, algoritmo, 67–70, 93–94
multithread, 664
strato
convesso, 870 pr.
massimale, 870 pr.
stretta di mano, lemma della, 974 es.
stretto, vincolo, 723
stringa, 822, 983
lunghezza, 983
nulla (ε), 823
taglio di una, 340 pr.
vuota, 881
string matching, 822–844
algoritmo di Knuth-Morris-Pratt, 836–843
algoritmo di Rabin-Karp, 826–830
algoritmo ingenuo, 824–826
automi a stati finiti, 830–836
basato sui fattori di ripetizione, 844 pr.
con caratteri jolly, 826 es., 835 es.
STRONGLY-CONNECTED-COMPONENTS, 516
struttura dati, 8, 189–191, 280–293, 401–403, 468–488
alberi di van Emde Boas, 443–467
albero 2-3, 279, 421
albero 2-3-4, 408, 420 pr.
albero AA, 279
albero AVL, 275 pr.
albero bilanciato in peso, 279
albero binario di ricerca, 237–254
albero con radice, 203–205
albero del capro espiatorio, 279
albero di fusione, 174, 402
albero di intervalli, 288–293
albero dinamico, 402
albero di ricerca, 174, 403
albero k-neighbor, 279
albero per statistiche d'ordine, 280
albero rosso-nero, 255–279
albero splay, 279, 402
aumentare una, 280–293
B-albero, 404–421
coda, 192–194
coda di priorità, 135–139
coda doppia, 195 es.
deque, 195 es.
determinare la profondità, 486 pr.
di van Emde Boas, 448–455
dizionario, 189
foresta di insiemi disgiunti, 474–477
heap, 127–140
heap 2-3-4, 442 pr.
heap binomiale, 440 pr.
heap di Fibonacci, 422–442
heap rilassato, 442
heap riunibile, 422–423
insieme dinamico, 189–190
insiemi disgiunti, 468–488
analisi, 480–485, 485 es.
caso speciale di tempo lineare, 488
minimo off-line, 486 pr.
programmazione dei lavori, 374 pr.
lista concatenata, 195–199
nella memoria secondaria, 404–407
per grafi dinamici, 403
persistente, 274 pr., 402
potenziale di una, 383
radix tree, 251 pr.
skip list, 279
stack, 192–193

Acquistato da Michele Miregot su Virgilio 1 2022, 07 7 23:12 Numero Conto Utente: 2-9558016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)

tavola hash, 211–216
treap, 276 pr.
vettore di bit, 444–447
subroutine, 20
chiamata, 19–20, 22 n.
esecuzione, 22 n.
SUBSET-SUM, 913
SUCCESSOR, 190
successore
alberi di van Emde Boas, 459–460
in albero per statistiche d'ordine, 287 es.
negli alberi binari di ricerca, 241–242
negli alberi rosso-neri, 257
nella lista concatenata, 195
nelle protostrutture di van Emde Boas, 452–453
in un vettore di bit, 446
suffisso ($\sqsupset$), 823
funzione, 831
SUM-ARRAYS, 672 pr.
SUM-ARRAYS′, 672 pr.
summary
albero di van Emde Boas, 455
protostrutture di van Emde Boas, 449
supercomputer, 645
superpolinomiale, tempo, 873
superpozzo, 596
supersorgente, 596
SVD, metodo, 703
sweeping, 851–857, 870 pr.
rotazionale, 857–864
sync, nello pseudocodice, 648–649

tabella dei simboli di un compilatore, 209, 217, 219
tableau di Young, 140 pr.
TABLE-DELETE, 391
TABLE-INSERT, 387
taglio
attraversato da un arco, 525
delle aste, 298–306
di una rete di flusso, 603–606
di un grafo non ordinato, 525
in cascata, 434
minimo, 603, 612 es.
peso di un, 939 es.
tail, 193, 195
TAIL-RECURSIVE-QUICKSORT, 155 pr.
Task Parallel Library, 647
tasso di crescita, 25
tautologia, 888 es., 904 es.
tavola
dinamica
fattore di carico, 386
metodo degli accantonamenti, 388
metodo del potenziale, 388, 391
hash, 211–216
dinamica, 393 es.
doppio hashing, 226–227, 229 es.
indirizzamento aperto, 223–230
ispezione lineare, 225
ispezione quadratica, 225, 235 pr.
metodo di annotazione, 302
secondaria, 230
Taylor, serie, 253 pr.
tempo
di completamento dei lavori, 372 pr.
di completamento e componenti fortemente connesse, 517
di esecuzione, 22
atteso, 24, 97
caso migliore, 25 es., 42
caso peggiore, 24, 42
di un algoritmo per grafo, 491
nel caso medio, 96
tasso di crescita, 25
di rilascio, 372 pr.
dominio del, 750
teorema
cinese del resto, 793–796
dell'esperto, 79, 82–89
di Bayes, 991
di Hall, 615 es.
test
di primalità, 805–814, 821
di pseudoprimalità, 807–808
testa di una lista, 195
testina di un'unità a disco, 405
testo cifrato, 801
then, clausola, 17 n.
thread, 645
threading, 645
Threading Building Blocks, 647
3-CNF, 901
3-CNF-SAT, 901
to, nella pseudocodifica, 18
Toeplitz, matrice, 769 pr.
TOP, 858
top di uno stack, 192
TOPOLOGICAL-SORT, 513
tour bitonico, 336 pr.
traboccante, vertice, 616
traccia, 405
TRANSITIVE-CLOSURE, 584
transizione, funzione di, 831
TRANSPLANT, 245, 267
trasformata
chirp, 764 es.
discreta di Fourier, 8, 759
rapida di Fourier
circuito per la, 767
con aritmetica modulare, 771 pr.
implementazione iterativa, 764
implementazione ricorsiva, 760
multidimensionale, 769 pr.
treap, 276 pr.
TREAP-INSERT, 276 pr.
TREE-DELETE, 245, 267–268
TREE-INSERT, 244, 260
TREE-MAXIMUM, 241
TREE-MINIMUM, 241
TREE-PREDECESSOR, 242
TREE-SEARCH, 240
TREE-SUCCESSOR, 241
tricotomia
di intervalli, 288
dei numeri reali, 45
trie, *vedere* radix tree
TRIM, 942
TSP, 912
tupla, 223 es., 235 pr., 965
twiddle, fattore di, 762

uguaglianza
lineare, 706
vincolo, 562 es., 712, 723
unario, 880
underflow
di una coda, 194
di uno stack, 192
UNION, 422, 468
con foresta di insiemi disgiunti, 476
con liste concatenate, 472, 474 es.
unione
di heap 2-3-4, 442 pr.
di heap di Fibonacci, 427–428
di insiemi ($\cup$), 963
di liste concatenate, 199 es.
per rango, 475
pesata, 472
unità (1), 775
radice principale, 757
radici complesse, 757

valore
atteso, 993–995
di una distribuzione binomiale, 998
di una variabile casuale, 99
linearità del, 99
obiettivo, 707, 711
valutazione di un polinomio, 752, 756 es.
in più punti, 770 pr.
Vandermonde, matrice di, 1017 pr.
van Emde Boas, alberi, 443–467
Var [ ] (varianza), 995
variabile
casuale discreta, 992–997
casuale indicatrice, 98–101
analisi di quicksort, 150, 154 pr.
in bucket sort, 166–167

Acquistato da Michele Michele su Virgilio il 2022-07-07 23:12 Numero Ordine Libraccio 199503016-220707-0 Copyright © 2022 McGraw-Hill Education (Italy)

hashing, 214–215
hashing universale, 220
treap, 276 pr.
di base, 715
entrante, 724
globale, 18
locale, 18
nella pseudocodifica, 18
non di base, 715
scarto, 714
sostituzione, 73
uscente, 724
varianza, 995
di una distribuzione binomiale, 1000
di una distribuzione geometrica, 997
VEB-EMPTY-TREE-INSERT, 461
VEB-TREE-DELETE, 462
VEB-TREE-INSERT, 461
VEB-TREE-MAXIMUM, 458
VEB-TREE-MEMBER, 459
VEB-TREE-MINIMUM, 458
VEB-TREE-PREDECESSOR, 460
VEB-TREE-SUCCESSOR, 459
Venn, diagramma, 964
verifica
algoritmi di, 884–888
dell'albero di connessione, 538
verità
assegnazione di, 892, 899
tavola di, 891
vertice
accoppiato, 613
attributi, 495
bianco, 497, 505
capacità, 598 es.
completato, 505
grigio, 497, 505
innalzamento, 617
innalzato, 619
intermedio, 580
ispezionato, 507
nero, 497, 505
non accoppiato, 613
punto di articolazione, 520 pr.
scoperto, 497, 505
selettore, 909
traboccante, 616
vettori, 1009–1010
di bit, 211 es., 444–447
nel piano, 846
nulli, 1014
ortonormali, 703
prodotto di, 846
riga, 1010
unità, 1010
vicini, lista dei, 628
vincoli
di non negatività, 711–712
di uguaglianza, 712
grafo dei, 559–560
lineari, 706
stretti, 723
sulla capacità, 594
sulle differenze, 557–562
matrice di incidenza, 559
violazione dei, 723
visita
albero di, 497, 502
in ampiezza, 497–504
algoritmo di Dijkstra, 555, 556 es.
cammino minimo, 539
in profondità, 504–512
Viterbi, algoritmo, 339 pr.
VORP, 342 pr.

while, pseudocodice, 17
widget, 909
WITNESS, 808

y-fast trie, 465 pr.
Young, tableau di, 140 pr.

$\mathbb{Z}$ (numeri interi), 962
$\mathbb{Z}_n$ (classi di congruenza modulo n), 775
$\mathbb{Z}_n^*$ (elementi del gruppo moltiplicativo modulo n), 785
$\mathbb{Z}_n^+$ (elementi non nulli di $\mathbb{Z}_n^*$), 807
zaino
frazionario, 354
problema dello, 354, 355–356 es.

Acquistato da Michele Michele su Webster il 2022-07-07 23:12 Numero Ordine Libreria: 199503016-220707-0 Copyright © 2022, McGraw-Hill Education (Italy)